JavaScript

MDN Web Docs (Mozilla) — English

Fonte oficial: https://developer.mozilla.org/en-US/docs/Web/JavaScript
Gerado para o projeto doc-frog

JavaScript

JavaScript (**JS**) is a lightweight interpreted (or just-in-time compiled) programming language with first-class functions. While it is most well-known as the scripting language for Web pages, many non-browser environments also use it, such as Node.js, Apache CouchDB and Adobe Acrobat. JavaScript is a prototype-based, garbage-collected, dynamic language, supporting multiple paradigms such as imperative, functional, and object-oriented.

JavaScript's dynamic capabilities include runtime object construction, variable parameter lists, function variables, dynamic script creation (via `eval`), object introspection (via `for...in` and `Object` utilities), and source-code recovery (JavaScript functions store their source text and can be retrieved through `toString()`).

This section is dedicated to the JavaScript language itself, and not the parts that are specific to Web pages or other host environments. For information about APIs that are specific to Web pages, please see Web APIs and DOM.

The standards for JavaScript are the ECMAScript Language Specification (ECMA-262) and the ECMAScript Internationalization API specification (ECMA-402). As soon as one browser implements a feature, we try to document it. This means that cases where some proposals for new ECMAScript features have already been implemented in browsers, documentation and examples in MDN articles may use some of those new features. Most of the time, this happens between the stages 3 and 4, and is usually before the spec is officially published.

Do not confuse JavaScript with the Java programming language — **JavaScript is *not* "Interpreted Java"**. Both "Java" and "JavaScript" are trademarks or registered trademarks of Oracle in the U.S. and other countries. However, the two programming languages have very different syntax, semantics, and use.

JavaScript documentation of core language features (pure ECMAScript, for the most part) includes the following:

- The JavaScript guide
- The JavaScript reference

For more information about JavaScript specifications and related technologies, see JavaScript technologies overview.

Beginner's tutorials

Our learn web development core modules contain modern, up-to-date tutorials covering JavaScript fundamentals.

Your first website: Adding interactivity

> This article provides a brief tour of what JavaScript is and how to use it, aimed at people who are completely new to web development.

Dynamic scripting with JavaScript

> This module focuses on the essentials of the core JavaScript language, plus some key surrounding topics — learning these topics will give you a solid basis to work from.

JavaScript frameworks and libraries

> JavaScript frameworks are an essential part of modern front-end web development, providing developers with tried and tested tools for building scalable, interactive web applications. Many modern companies use frameworks as a standard part of their tooling, so many front-end development jobs now require framework experience. This set of articles provides a comfortable starting point to help you begin learning frameworks.

JavaScript guides

Fundamental language guides

JavaScript Guide

> A much more detailed guide to the JavaScript language, aimed at those with previous programming experience either in JavaScript or another language.

Intermediate

Advanced JavaScript objects

> The object-oriented nature of JavaScript is important to understand if you want to go further with your knowledge of the language and write more efficient code, therefore we've provided this module to help you.

Asynchronous JavaScript

> In this module, we take a look at asynchronous JavaScript, why it is important, and how it can be used to effectively handle potential blocking operations, such as fetching resources from a server.

Client-side web APIs

> Explores what APIs are, and how to use some of the most common APIs you'll come across often in your development work.

JavaScript language overview

> An overview of the basic syntax and semantics of JavaScript for those coming from other programming languages to get up to speed.

JavaScript data structures

> Overview of available data structures in JavaScript.

Equality comparisons and sameness

> JavaScript provides three different value comparison operations: strict equality using `===`, loose equality using `==`, and the `Object.is()` method.

Enumerability and ownership of properties

How different methods that visit a group of object properties one-by-one handle the enumerability and ownership of properties.

Closures

A closure is the combination of a function and the lexical environment within which that function was declared.

Advanced

Inheritance and the prototype chain

Explanation of the widely misunderstood and underestimated prototype-based inheritance.

Memory Management

Memory life cycle and garbage collection in JavaScript.

Reference

Browse the complete JavaScript reference documentation.

Standard objects

Get to know standard built-in objects: `Array`, `Boolean`, `Error`, `Function`, `JSON`, `Math`, `Number`, `Object`, `RegExp`, `String`, `Map`, `Set`, `WeakMap`, `WeakSet`, and others.

Expressions and operators

Learn more about the behavior of JavaScript's operators `instanceof`, `typeof`, `new`, `this`, the operator precedence, and more.

Statements and declarations

Learn how `do-while`, `for-in`, `for-of`, `try-catch`, `let`, `var`, `const`, `if-else`, `switch`, and more JavaScript statements and keywords work.

Functions

Learn how to work with JavaScript's functions to develop your applications.

Classes

JavaScript classes are the most appropriate way to do object-oriented programming.

JavaScript Guide

The JavaScript Guide shows you how to use JavaScript and gives an overview of the language. If you need exhaustive information about a language feature, have a look at the JavaScript reference.

This Guide is divided into the following chapters.

Introduction

Overview: Introduction

- About this guide
- About JavaScript
- JavaScript and Java
- ECMAScript
- Tools
- What's next

Grammar and types

Overview: Grammar and types

- Basic syntax & comments
- Declarations
- Variable scope
- Variable hoisting
- Data structures and types
- Literals

Control flow and error handling

Overview: Control flow and error handling

- `if...else`
- `switch`
- `try/catch/throw`
- Error objects

Loops and iteration

Overview: Loops and iteration

- `for`
- `while`
- `do...while`
- `continue`
- `break`
- `for...in`
- `for...of`

Functions

Overview: Functions

- Defining functions
- Calling functions
- Function scopes and closures
- Arguments & parameters
- Arrow functions

Expressions and operators

Overview: Expressions and operators

- Assignment & Comparisons
- Arithmetic operators
- Bitwise & logical operators
- Conditional (ternary) operator

Numbers and strings

Overview: Numbers and strings

- Numbers
- `Number` object
- `Math` object
- Strings
- `String` object
- Template literals

Representing dates & times

Overview: Representing dates & times

- `Date` object

Regular expressions

Overview: Regular expressions

- Creating a regular expression
- Writing a regular expression pattern
  - Assertions
  - Character classes
  - Groups and backreferences
  - Quantifiers

Indexed collections

Overview: Indexed collections

Keyed collections

Overview: Keyed collections

- `Map`
- `WeakMap`
- `Set`
- `WeakSet`

Working with objects

Overview: Working with objects

- Objects and properties
- Creating objects
- Defining methods
- Getter and setter

Using classes

Overview: Using classes

- Declaring a class
- Various class features
- Extends and inheritance
- Why classes?

Promises

Overview: Promises

- Guarantees
- Chaining
- Error handling
- Composition
- Timing

Typed arrays

Overview: Typed arrays

Iterators and generators

Overview: Iterators and generators

- Iterators
- Iterables
- Generators

Resource management

Overview: JavaScript resource management

- The `using` and `await using` declarations
- The `DisposableStack` and `AsyncDisposableStack` objects
- Error handling

Internationalization

Overview: Internationalization

- Date and time formatting
- Number formatting
- Collation

JavaScript modules

Overview: JavaScript modules

- Exporting
- Importing
- Default exports
- Renaming features
- Aggregating modules
- Dynamic module loading

Advanced topics

After you have learned all fundamental features of JavaScript, you can explore some more niche features, or dive deeper into the language's mechanisms and concepts.

- Language overview
- Data structures
- Enumerability and ownership of properties
- Inheritance and the prototype chain
- Equality comparisons and sameness
- Closures
- Meta programming
- Memory management

- Next

Closures

A **closure** is the combination of a function bundled together (enclosed) with references to its surrounding state (the **lexical environment**). In other words, a closure gives a function access to its outer scope. In JavaScript, closures are created every time a function is created, at function creation time.

Lexical scoping

Consider the following example code:

```
function init() {
  var name = "Mozilla"; // name is a local variable created by init
```

```
  function displayName() {
    // displayName() is the inner function, that forms a closure
    console.log(name); // use variable declared in the parent function
  }
  displayName();
}
init();
```

`init()` creates a local variable called `name` and a function called `displayName()`. The `displayName()` function is an inner function that is defined inside `init()` and is available only within the body of the `init()` function. Note that the `displayName()` function has no local variables of its own. However, since inner functions have access to the variables of outer scopes, `displayName()` can access the variable `name` declared in the parent function, `init()`.

If you run this code in your console, you can see that the `console.log()` statement within the `displayName()` function successfully displays the value of the `name` variable, which is declared in its parent function. This is an example of *lexical scoping*, which describes how a parser resolves variable names when functions are nested. The word *lexical* refers to the fact that lexical scoping uses the location where a variable is declared within the source code to determine where that variable is available. Nested functions have access to variables declared in their outer scope.

Scoping with let and const

Traditionally (before ES6), JavaScript variables only had two kinds of scopes: *function scope* and *global scope*. Variables declared with `var` are either function-scoped or global-scoped, depending on whether they are declared within a function or outside a function. This can be tricky, because blocks with curly braces do not create scopes:

```
if (Math.random() > 0.5) {
  var x = 1;
} else {
  var x = 2;
}
console.log(x);
```

For people from other languages (e.g., C, Java) where blocks create scopes, the above code should throw an error on the `console.log` line, because we are outside the scope of `x` in either block. However, because blocks don't create scopes for `var`, the `var` statements here actually create a global variable. There is also a practical example introduced below that illustrates how this can cause actual bugs when combined with closures.

In ES6, JavaScript introduced the `let` and `const` declarations, which, among other things like temporal dead zones, allow you to create block-scoped variables.

```
if (Math.random() > 0.5) {
  const x = 1;
} else {
  const x = 2;
}
console.log(x); // ReferenceError: x is not defined
```

In essence, blocks are finally treated as scopes in ES6, but only if you declare variables with `let` or `const`. In addition, ES6 introduced modules, which introduced another kind of scope. Closures are able to capture variables in all these scopes, which we will introduce later.

Closure

Consider the following code example:

```
function makeFunc() {
  const name = "Mozilla";
  function displayName() {
    console.log(name);
  }
  return displayName;
}

const myFunc = makeFunc();
myFunc();
```

Running this code has exactly the same effect as the previous example of the `init()` function above. What's different (and interesting) is that the `displayName()` inner function is returned from the outer function *before being executed*.

At first glance, it might seem unintuitive that this code still works. In some programming languages, the local variables within a function exist for just the duration of that function's execution. Once `makeFunc()` finishes executing, you might expect that the `name` variable would no longer be accessible. However, because the code still works, this is obviously not the case in JavaScript.

The reason is that functions in JavaScript form closures. A *closure* is the combination of a function and the lexical environment within which that function was declared. This environment consists of any variables that were in-scope at the time the closure was created. In this case, `myFunc` is a reference to the instance of the function `displayName` that is created when `makeFunc` is run. The instance of `displayName` maintains a reference to its lexical environment, within which the variable `name` exists. For this reason, when `myFunc` is invoked, the variable `name` remains available for use, and "Mozilla" is passed to `console.log`.

Here's a slightly more interesting example—a `makeAdder` function:

```
function makeAdder(x) {
  return function (y) {
    return x + y;
  };
}

const add5 = makeAdder(5);
```

```
const add10 = makeAdder(10);

console.log(add5(2)); // 7
console.log(add10(2)); // 12
```

In this example, we have defined a function `makeAdder(x)`, that takes a single argument `x`, and returns a new function. The function it returns takes a single argument `y`, and returns the sum of `x` and `y`.

In essence, `makeAdder` is a function factory. It creates functions that can add a specific value to their argument. In the above example, the function factory creates two new functions—one that adds five to its argument, and one that adds 10.

`add5` and `add10` both form closures. They share the same function body definition, but store different lexical environments. In `add5`'s lexical environment, `x` is 5, while in the lexical environment for `add10`, `x` is 10.

Practical closures

Closures are useful because they let you associate data (the lexical environment) with a function that operates on that data. This has obvious parallels to object-oriented programming, where objects allow you to associate data (the object's properties) with one or more methods.

Consequently, you can use a closure anywhere that you might normally use an object with only a single method.

Situations where you might want to do this are particularly common on the web. Much of the code written in front-end JavaScript is event-based. You define some behavior, and then attach it to an event that is triggered by the user (such as a click or a keypress). The code is attached as a callback (a single function that is executed in response to the event).

For instance, suppose we want to add buttons to a page to adjust the text size. One way of doing this is to specify the font-size of the `body` element (in pixels), and then set the size of the other elements on the page (such as headers) using the relative `em` unit:

```
body {
  font-family: "Helvetica", "Arial", sans-serif;
  font-size: 12px;
}

h1 {
  font-size: 1.5em;
}

h2 {
  font-size: 1.2em;
}
```

Such interactive text size buttons can change the `font-size` property of the `body` element, and the adjustments are picked up by other elements on the page thanks to the relative units.

Here's the JavaScript:

```
function makeSizer(size) {
  return () => {
    document.body.style.fontSize = `${size}px`;
  };
}

const size12 = makeSizer(12);
const size14 = makeSizer(14);
const size16 = makeSizer(16);
```

`size12`, `size14`, and `size16` are now functions that resize the body text to 12, 14, and 16 pixels, respectively. You can attach them to buttons as demonstrated in the following code example.

```
document.getElementById("size-12").onclick = size12;
document.getElementById("size-14").onclick = size14;
document.getElementById("size-16").onclick = size16;
```

```
<button id="size-12">12</button>
<button id="size-14">14</button>
<button id="size-16">16</button>
<p>This is some text that will change size when you click the buttons above.</p>
```

Emulating private methods with closures

Languages such as Java allow you to declare methods as private, meaning that they can be called only by other methods in the same class.

JavaScript, prior to classes, didn't have a native way of declaring private methods, but it was possible to emulate private methods using closures. Private methods aren't just useful for restricting access to code. They also provide a powerful way of managing your global namespace.

The following code illustrates how to use closures to define public functions that can access private functions and variables. Note that these closures follow the Module Design Pattern.

```
const counter = (function () {
  let privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
```

```
  }

  return {
    increment() {
      changeBy(1);
    },

    decrement() {
      changeBy(-1);
    },

    value() {
      return privateCounter;
    },
  };
})();

console.log(counter.value()); // 0.

counter.increment();
counter.increment();
console.log(counter.value()); // 2.

counter.decrement();
console.log(counter.value()); // 1.
```

In previous examples, each closure had its own lexical environment. Here though, there is a single lexical environment that is shared by the three functions: `counter.increment`, `counter.decrement`, and `counter.value`.

The shared lexical environment is created in the body of an anonymous function, *which is executed as soon as it has been defined* (also known as an IIFE). The lexical environment contains two private items: a variable called `privateCounter`, and a function called `changeBy`. You can't access either of these private members from outside the anonymous function. Instead, you indirectly access them using the three public functions that are returned from the anonymous wrapper.

Those three public functions form closures that share the same lexical environment. Thanks to JavaScript's lexical scoping, they each have access to the `privateCounter` variable and the `changeBy` function.

```
function makeCounter() {
  let privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
  }
  return {
    increment() {
      changeBy(1);
    },

    decrement() {
      changeBy(-1);
    },

    value() {
      return privateCounter;
    },
  };
}

const counter1 = makeCounter();
const counter2 = makeCounter();

console.log(counter1.value()); // 0.

counter1.increment();
counter1.increment();
console.log(counter1.value()); // 2.

counter1.decrement();
console.log(counter1.value()); // 1.
console.log(counter2.value()); // 0.
```

Notice how the two counters maintain their independence from one another. Each closure references a different version of the `privateCounter` variable through its own closure. Each time one of the counters is called, its lexical environment changes by changing the value of this variable. Changes to the variable value in one closure don't affect the value in the other closure.

> **Note:** Using closures in this way provides benefits that are normally associated with object-oriented programming. In particular, *data hiding* and *encapsulation*.

Closure scope chain

A nested function's access to the outer function's scope includes the enclosing scope of the outer function—effectively creating a chain of function scopes. To demonstrate, consider the following example code.

```
// global scope
const e = 10;
```

```
function sum(a) {
  return function (b) {
    return function (c) {
      // outer functions scope
      return function (d) {
        // local scope
        return a + b + c + d + e;
      };
    };
  };
}

console.log(sum(1)(2)(3)(4)); // 20
```

You can also write without anonymous functions:

```
// global scope
const e = 10;
function sum(a) {
  return function sum2(b) {
    return function sum3(c) {
      // outer functions scope
      return function sum4(d) {
        // local scope
        return a + b + c + d + e;
      };
    };
  };
}

const sum2 = sum(1);
const sum3 = sum2(2);
const sum4 = sum3(3);
const result = sum4(4);
console.log(result); // 20
```

In the example above, there's a series of nested functions, all of which have access to the outer functions' scope. In this context, we can say that closures have access to *all* outer scopes.

Closures can capture variables in block scopes and module scopes as well. For example, the following creates a closure over the block-scoped variable `y`:

```
function outer() {
  let getY;
  {
    const y = 6;
    getY = () => y;
  }
  console.log(typeof y); // undefined
  console.log(getY()); // 6
}

outer();
```

Closures over modules can be more interesting.

```
// myModule.js
let x = 5;
export const getX = () => x;
export const setX = (val) => {
  x = val;
};
```

Here, the module exports a pair of getter-setter functions, which close over the module-scoped variable `x`. Even when `x` is not directly accessible from other modules, it can be read and written with the functions.

```
import { getX, setX } from "./myModule.js";

console.log(getX()); // 5
setX(6);
console.log(getX()); // 6
```

Closures can close over imported values as well, which are regarded as *live bindings*, because when the original value changes, the imported one changes accordingly.

```
// myModule.js
export let x = 1;
export const setX = (val) => {
  x = val;
};
```

```
// closureCreator.js
import { x } from "./myModule.js";
```

```
export const getX = () => x; // Close over an imported live binding
```

```
import { getX } from "./closureCreator.js";
import { setX } from "./myModule.js";

console.log(getX()); // 1
setX(2);
console.log(getX()); // 2
```

Creating closures in loops: A common mistake

Prior to the introduction of the `let` keyword, a common problem with closures occurred when you created them inside a loop. To demonstrate, consider the following example code.

```
<p id="help">Helpful notes will appear here</p>
<p>Email: <input type="text" id="email" name="email" /></p>
<p>Name: <input type="text" id="name" name="name" /></p>
<p>Age: <input type="text" id="age" name="age" /></p>
```

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    // Culprit is the use of `var` on this line
    var item = helpText[i];
    document.getElementById(item.id).onfocus = function () {
      showHelp(item.help);
    };
  }
}

setupHelp();
```

The `helpText` array defines three helpful hints, each associated with the ID of an input field in the document. The loop cycles through these definitions, hooking up an `onfocus` event to each one that shows the associated help method.

If you try this code out, you'll see that it doesn't work as expected. No matter what field you focus on, the message about your age will be displayed.

The reason for this is that the functions assigned to `onfocus` form closures; they consist of the function definition and the captured environment from the `setupHelp` function's scope. Three closures have been created by the loop, but each one shares the same single lexical environment, which has a variable with changing values (`item`). This is because the variable `item` is declared with `var` and thus has function scope due to hoisting. The value of `item.help` is determined when the `onfocus` callbacks are executed. Because the loop has already run its course by that time, the `item` variable object (shared by all three closures) has been left pointing to the last entry in the `helpText` list.

One solution in this case is to use more closures: in particular, to use a function factory as described earlier:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function makeHelpCallback(help) {
  return function () {
    showHelp(help);
  };
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    var item = helpText[i];
    document.getElementById(item.id).onfocus = makeHelpCallback(item.help);
  }
}

setupHelp();
```

This works as expected. Rather than the callbacks all sharing a single lexical environment, the `makeHelpCallback` function creates *a new lexical environment* for each callback, in which `help` refers to the corresponding string from the `helpText` array.

One other way to write the above using anonymous closures is:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    (function () {
      var item = helpText[i];
      document.getElementById(item.id).onfocus = function () {
        showHelp(item.help);
      };
    })(); // Immediate event listener attachment with the current value of item (preserved until iteration).
  }
}

setupHelp();
```

If you don't want to use more closures, you can use the `let` or `const` keyword:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  const helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (let i = 0; i < helpText.length; i++) {
    const item = helpText[i];
    document.getElementById(item.id).onfocus = () => {
      showHelp(item.help);
    };
  }
}

setupHelp();
```

This example uses `const` instead of `var`, so every closure binds the block-scoped variable, meaning that no additional closures are required.

If you are writing modern JavaScript anyway, you can consider more alternatives to the plain `for` loop, such as using `for...of` loop and declaring `item` as `let` or `const`, or using the `forEach()` method, which both avoid the closure problem.

```
for (const item of helpText) {
  document.getElementById(item.id).onfocus = () => {
    document.getElementById("help").textContent = item.help;
  };
}

helpText.forEach((item) => {
  document.getElementById(item.id).onfocus = () => {
    showHelp(item.help);
  };
});
```

Performance considerations

As mentioned previously, each function instance manages its own scope and closure. Therefore, it is unwise to unnecessarily create functions within other functions if closures are not needed for a particular task, as it will negatively affect script performance both in terms of processing speed and memory consumption.

For instance, when creating a new object/class, methods should normally be associated to the object's prototype rather than defined into the object constructor. The reason is that whenever the constructor is called, the methods would get reassigned (that is, for every object creation).

Consider the following case:

```
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
  this.getName = function () {
    return this.name;
  };

  this.getMessage = function () {
    return this.message;
```

```
  };
}
```

Because the previous code does not take advantage of the benefits of using closures in this particular instance, we could instead rewrite it to avoid using closures as follows:

```
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
}
MyObject.prototype = {
  getName() {
    return this.name;
  },
  getMessage() {
    return this.message;
  },
};
```

However, redefining the prototype is not recommended. The following example instead appends to the existing prototype:

```
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
}
MyObject.prototype.getName = function () {
  return this.name;
};
MyObject.prototype.getMessage = function () {
  return this.message;
};
```

In the two previous examples, the inherited prototype can be shared by all objects and the method definitions need not occur at every object creation. See Inheritance and the prototype chain for more.

Control flow and error handling

- Previous
- Next

JavaScript supports a compact set of statements, specifically control flow statements, that you can use to incorporate a great deal of interactivity in your application. This chapter provides an overview of these statements.

The JavaScript reference contains exhaustive details about the statements in this chapter. The semicolon (`;`) character is used to separate statements in JavaScript code.

Any JavaScript expression is also a statement. See Expressions and operators for complete information about expressions.

Block statement

The most basic statement is a *block statement*, which is used to group statements. The block is delimited by a pair of curly braces:

```
{
  statement1;
  statement2;
  // …
  statementN;
}
```

Example

Block statements are commonly used with control flow statements (`if`, `for`, `while`).

```
while (x < 10) {
  x++;
}
```

Here, `{ x++; }` is the block statement.

> **Note:** `var`-declared variables are not block-scoped, but are scoped to the containing function or script, and the effects of setting them persist beyond the block itself. For example:
>
> ```
> var x = 1;
> {
>   var x = 2;
> }
> console.log(x); // 2
> ```
>
> This outputs `2` because the `var x` statement within the block is in the same scope as the `var x` statement before the block. (In C or Java, the equivalent code would have output `1`.)

> This scoping effect can be mitigated by using `let` or `const`.

Conditional statements

A conditional statement is a set of commands that executes if a specified condition is true. JavaScript supports two conditional statements: `if...else` and `switch`.

if...else statement

Use the `if` statement to execute a statement if a logical condition is `true`. Use the optional `else` clause to execute a statement if the condition is `false`.

An `if` statement looks like this:

```
if (condition) {
  statement1;
} else {
  statement2;
}
```

Here, the `condition` can be any expression that evaluates to `true` or `false`. (See Boolean for an explanation of what evaluates to `true` and `false`.)

If `condition` evaluates to `true`, `statement1` is executed. Otherwise, `statement2` is executed. `statement1` and `statement2` can be any statement, including further nested `if` statements.

You can also compound the statements using `else if` to have multiple conditions tested in sequence, as follows:

```
if (condition1) {
  statement1;
} else if (condition2) {
  statement2;
} else if (conditionN) {
  statementN;
} else {
  statementLast;
}
```

In the case of multiple conditions, only the first logical condition which evaluates to `true` will be executed. To execute multiple statements, group them within a block statement (`{ /* … */ }`).

Best practice

In general, it's good practice to always use block statements—*especially* when nesting `if` statements:

```
if (condition) {
  // Statements for when condition is true
  // …
} else {
  // Statements for when condition is false
  // …
}
```

In general it's good practice to not have an `if...else` with an assignment like `x = y` as a condition:

```
if (x = y) {
  // statements here
}
```

However, in the rare case you find yourself wanting to do something like that, the `while` documentation has a Using an assignment as a condition section with guidance on a general best-practice syntax you should know about and follow.

Falsy values

The following values evaluate to `false` (also known as Falsy values):

- `false`
- `undefined`
- `null`
- `0`
- `NaN`
- the empty string (`""`)

All other values—including all objects—evaluate to `true` when passed to a conditional statement.

> **Note:** Do not confuse the primitive boolean values `true` and `false` with the true and false values of the `Boolean` object!
>
> For example:
>
> ```
> const b = new Boolean(false);
> if (b) {
>   // this condition evaluates to true
> }
> ```

```
if (b == true) {
  // this condition evaluates to false
}
```

Example

In the following example, the function `checkData` returns `true` if the number of characters in a `Text` object is three. Otherwise, it displays an alert and returns `false`.

```
function checkData() {
  if (document.form1.threeChar.value.length === 3) {
    return true;
  }
  alert(
    `Enter exactly three characters. ${document.form1.threeChar.value} is not valid.`,
  );
  return false;
}
```

switch statement

A `switch` statement allows a program to evaluate an expression and attempt to match the expression's value to a `case` label. If a match is found, the program executes the associated statement.

A `switch` statement looks like this:

```
switch (expression) {
  case label1:
    statements1;
    break;
  case label2:
    statements2;
    break;
  // …
  default:
    statementsDefault;
}
```

JavaScript evaluates the above switch statement as follows:

- The program first looks for a `case` clause with a label matching the value of expression and then transfers control to that clause, executing the associated statements.
- If no matching label is found, the program looks for the optional `default` clause:
  - If a `default` clause is found, the program transfers control to that clause, executing the associated statements.
  - If no `default` clause is found, the program resumes execution at the statement following the end of `switch`.
  - (By convention, the `default` clause is written as the last clause, but it does not need to be so.)

break statements

The optional `break` statement associated with each `case` clause ensures that the program breaks out of `switch` once the matched statement is executed, and then continues execution at the statement following `switch`. If `break` is omitted, the program continues execution inside the `switch` statement (and will execute statements under the next `case`, and so on).

Example

In the following example, if `fruitType` evaluates to `"Bananas"`, the program matches the value with `case "Bananas"` and executes the associated statement. When `break` is encountered, the program exits the `switch` and continues execution from the statement following `switch`. If `break` were omitted, the statement for `case "Cherries"` would also be executed.

```
switch (fruitType) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Apples":
    console.log("Apples are $0.32 a pound.");
    break;
  case "Bananas":
    console.log("Bananas are $0.48 a pound.");
    break;
  case "Cherries":
    console.log("Cherries are $3.00 a pound.");
    break;
  case "Mangoes":
    console.log("Mangoes are $0.56 a pound.");
    break;
  case "Papayas":
    console.log("Papayas are $2.79 a pound.");
    break;
  default:
    console.log(`Sorry, we are out of ${fruitType}.`);
}
console.log("Is there anything else you'd like?");
```

Exception handling statements

You can throw exceptions using the `throw` statement and handle them using the `try...catch` statements.

- `throw` statement
- `try...catch` statement

Exception types

Just about any object can be thrown in JavaScript. Nevertheless, not all thrown objects are created equal. While it is common to throw numbers or strings as errors, it is frequently more effective to use one of the exception types specifically created for this purpose:

- ECMAScript exceptions
- `DOMException`

throw statement

Use the `throw` statement to throw an exception. A `throw` statement specifies the value to be thrown:

```
throw expression;
```

You may throw any expression, not just expressions of a specific type. The following code throws several exceptions of varying types:

```
throw "Error2"; // String type
throw 42; // Number type
throw true; // Boolean type
throw {
  toString() {
    return "I'm an object!";
  },
};
```

try...catch statement

The `try...catch` statement marks a block of statements to try, and specifies one or more responses should an exception be thrown. If an exception is thrown, the `try...catch` statement catches it.

The `try...catch` statement consists of a `try` block, which contains one or more statements, and a `catch` block, containing statements that specify what to do if an exception is thrown in the `try` block.

In other words, you want the `try` block to succeed—but if it does not, you want control to pass to the `catch` block. If any statement within the `try` block (or in a function called from within the `try` block) throws an exception, control *immediately* shifts to the `catch` block. If no exception is thrown in the `try` block, the `catch` block is skipped. The `finally` block executes after the `try` and `catch` blocks execute but before the statements following the `try...catch` statement.

The following example uses a `try...catch` statement. The example calls a function that retrieves a month name from an array based on the value passed to the function. If the value does not correspond to a month number (`1` – `12`), an exception is thrown with the value `'Invalid month code'` and the statements in the `catch` block set the `monthName` variable to `'unknown'`.

```
function getMonthName(mo) {
  mo--; // Adjust month number for array index (so that 0 = Jan, 11 = Dec)
  // prettier-ignore
  const months = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
  ];
  if (!months[mo]) {
    throw new Error("Invalid month code"); // throw keyword is used here
  }
  return months[mo];
}

try {
  // statements to try
  monthName = getMonthName(myMonth); // function could throw exception
} catch (e) {
  monthName = "unknown";
  logMyErrors(e); // pass exception object to error handler (i.e. your own function)
}
```

The catch block

You can use a `catch` block to handle all exceptions that may be generated in the `try` block.

```
catch (exception) {
  statements
}
```

The `catch` block specifies an identifier (`exception` in the preceding syntax) that holds the value specified by the `throw` statement. You can use this identifier to get information about the exception that was thrown.

JavaScript creates this identifier when the `catch` block is entered. The identifier lasts only for the duration of the `catch` block. Once the `catch` block finishes executing, the identifier no longer exists.

For example, the following code throws an exception. When the exception occurs, control transfers to the `catch` block.

```
try {
  throw "myException"; // generates an exception
} catch (err) {
  // statements to handle any exceptions
  logMyErrors(err); // pass exception object to error handler
}
```

> **Note:** When logging errors to the console inside a `catch` block, using `console.error()` rather than `console.log()` is advised for debugging. It formats the message as an error, and adds it to the list of error messages generated by the page.

The finally block

The `finally` block contains statements to be executed *after* the `try` and `catch` blocks execute. Additionally, the `finally` block executes *before* the code that follows the `try...catch...finally` statement.

It is also important to note that the `finally` block will execute *whether or not* an exception is thrown. If an exception is thrown, however, the statements in the `finally` block execute even if no `catch` block handles the exception that was thrown.

You can use the `finally` block to make your script fail gracefully when an exception occurs. For example, you may need to release a resource that your script has tied up.

The following example opens a file and then executes statements that use the file. (Server-side JavaScript allows you to access files.) If an exception is thrown while the file is open, the `finally` block closes the file before the script fails. Using `finally` here *ensures* that the file is never left open, even if an error occurs.

```
openMyFile();
try {
  writeMyFile(theData); // This may throw an error
} catch (e) {
  handleError(e); // If an error occurred, handle it
} finally {
  closeMyFile(); // Always close the resource
}
```

If the `finally` block returns a value, this value becomes the return value of the entire `try...catch...finally` production, regardless of any `return` statements in the `try` and `catch` blocks:

```
function f() {
  try {
    console.log(0);
    throw "bogus";
  } catch (e) {
    console.log(1);
    // This return statement is suspended
    // until finally block has completed
    return true;
    console.log(2); // not reachable
  } finally {
    console.log(3);
    return false; // overwrites the previous "return"
    // `f` exits here
    console.log(4); // not reachable
  }
  console.log(5); // not reachable
}
console.log(f()); // 0, 1, 3, false
```

Overwriting of return values by the `finally` block also applies to exceptions thrown or re-thrown inside of the `catch` block:

```
function f() {
  try {
    throw "bogus";
  } catch (e) {
    console.log('caught inner "bogus"');
    // This throw statement is suspended until
    // finally block has completed
    throw e;
  } finally {
    return false; // overwrites the previous "throw"
    // `f` exits here
  }
}

try {
  console.log(f());
} catch (e) {
  // this is never reached!
  // while f() executes, the `finally` block returns false,
  // which overwrites the `throw` inside the above `catch`
  console.log('caught outer "bogus"');
```

```
}

// Logs:
// caught inner "bogus"
// false
```

Nesting try...catch statements

You can nest one or more `try...catch` statements.

If an inner `try` block does *not* have a corresponding `catch` block:

1. it *must* contain a `finally` block, and
2. the enclosing `try...catch` statement's `catch` block is checked for a match.

For more information, see nested try-blocks on the `try...catch` reference page.

Utilizing Error objects

Depending on the type of error, you may be able to use the `name` and `message` properties to get a more refined message.

The `name` property provides the general class of `Error` (such as `DOMException` or `Error`), while `message` generally provides a more succinct message than one would get by converting the error object to a string.

If you are throwing your own exceptions, in order to take advantage of these properties (such as if your `catch` block doesn't discriminate between your own exceptions and system ones), you can use the `Error` constructor.

For example:

```
function doSomethingErrorProne() {
  if (ourCodeMakesAMistake()) {
    throw new Error("The message");
  }
  doSomethingToGetAJavaScriptError();
}

try {
  doSomethingErrorProne();
} catch (e) {
  // Now, we actually use `console.error()`
  console.error(e.name); // 'Error'
  console.error(e.message); // 'The message', or a JavaScript error message
}
```


- Previous
- Next

JavaScript data types and data structures

Programming languages all have built-in data structures, but these often differ from one language to another. This article attempts to list the built-in data structures available in JavaScript and what properties they have. These can be used to build other data structures.

The language overview offers a similar summary of the common data types, but with more comparisons to other languages.

Dynamic and weak typing

JavaScript is a dynamic language with dynamic types. Variables in JavaScript are not directly associated with any particular value type, and any variable can be assigned (and re-assigned) values of all types:

```
let foo = 42; // foo is now a number
foo = "bar"; // foo is now a string
foo = true; // foo is now a boolean
```

JavaScript is also a weakly typed language, which means it allows implicit type conversion when an operation involves mismatched types, instead of throwing type errors.

```
const foo = 42; // foo is a number
const result = foo + "1"; // JavaScript coerces foo to a string, so it can be concatenated with the other operand
console.log(result); // 421
```

Implicit coercions are very convenient, but can create subtle bugs when conversions happen where they are not expected, or where they are expected to happen in the other direction (for example, string to number instead of number to string). For symbols and BigInts, JavaScript has intentionally disallowed certain implicit type conversions.

Primitive values

All types except Object define immutable values represented directly at the lowest level of the language. We refer to values of these types as *primitive values*.

All primitive types, except `null`, can be tested by the `typeof` operator. `typeof null` returns `"object"`, so one has to use `=== null` to test for `null`.

All primitive types, except `null` and `undefined`, have their corresponding object wrapper types, which provide useful methods for working with the primitive values. For example, the `Number` object provides methods like `toExponential()`. When a property is accessed on a primitive value, JavaScript automatically wraps the value into the

corresponding wrapper object and accesses the property on the object instead. However, accessing a property on `null` or `undefined` throws a `TypeError` exception, which necessitates the introduction of the optional chaining operator.

Type	`typeof` return value	Object wrapper
Null	`"object"`	N/A
Undefined	`"undefined"`	N/A
Boolean	`"boolean"`	`Boolean`
Number	`"number"`	`Number`
BigInt	`"bigint"`	`BigInt`
String	`"string"`	`String`
Symbol	`"symbol"`	`Symbol`

The object wrapper classes' reference pages contain more information about the methods and properties available for each type, as well as detailed descriptions for the semantics of the primitive types themselves.

Null type

The Null type is inhabited by exactly one value: `null`.

Undefined type

The Undefined type is inhabited by exactly one value: `undefined`.

Conceptually, `undefined` indicates the absence of a *value*, while `null` indicates the absence of an *object* (which could also make up an excuse for `typeof null === "object"`). The language usually defaults to `undefined` when something is devoid of a value:

- A `return` statement with no value (`return;`) implicitly returns `undefined`.
- Accessing a nonexistent object property (`obj.iDontExist`) returns `undefined`.
- A variable declaration without initialization (`let x;`) implicitly initializes the variable to `undefined`.
- Many methods, such as `Array.prototype.find()` and `Map.prototype.get()`, return `undefined` when no element is found.

`null` is used much less often in the core language. The most important place is the end of the prototype chain — subsequently, methods that interact with prototypes, such as `Object.getPrototypeOf()`, `Object.create()`, etc., accept or return `null` instead of `undefined`.

`null` is a keyword, but `undefined` is a normal identifier that happens to be a global property. In practice, the difference is minor, since `undefined` should not be redefined or shadowed.

Boolean type

The `Boolean` type represents a logical entity and is inhabited by two values: `true` and `false`.

Boolean values are usually used for conditional operations, including ternary operators, `if...else`, `while`, etc.

Number type

The `Number` type is a double-precision 64-bit binary format IEEE 754 value. It is capable of storing positive floating-point numbers between 2^{-1074} (`Number.MIN_VALUE`) and $2^{1023} \times (2 - 2^{-52})$ (`Number.MAX_VALUE`) as well as negative floating-point numbers of the same magnitude, but it can only safely store integers in the range $-(2^{53} - 1)$ (`Number.MIN_SAFE_INTEGER`) to $2^{53} - 1$ (`Number.MAX_SAFE_INTEGER`). Outside this range, JavaScript can no longer safely represent integers; they will instead be represented by a double-precision floating point approximation. You can check if a number is within the range of safe integers using `Number.isSafeInteger()`.

Values outside the representable range are automatically converted:

- Positive values greater than `Number.MAX_VALUE` are converted to `Infinity`.
- Positive values smaller than `Number.MIN_VALUE` are converted to `0`.
- Negative values smaller than `-Number.MAX_VALUE` are converted to `-Infinity`.
- Negative values greater than `-Number.MIN_VALUE` are converted to `-0`.

`Infinity` and `-Infinity` behave similarly to mathematical infinity, but with some slight differences; see `Number.POSITIVE_INFINITY` and `Number.NEGATIVE_INFINITY` for details.

The Number type has only one value with multiple representations: `0` is represented as both `-0` and `+0` (where `0` is an alias for `+0`). In practice, there is almost no difference between the different representations; for example, `+0 === -0` is `true`. However, you are able to notice this when you divide by zero:

```
console.log(42 / +0); // Infinity
console.log(42 / -0); // -Infinity
```

`NaN` ("**N**ot **a** **N**umber") is a special kind of number value that's typically encountered when the result of an arithmetic operation cannot be expressed as a number. It is also the only value in JavaScript that is not equal to itself.

Although a number is conceptually a "mathematical value" and is always implicitly floating-point-encoded, JavaScript provides bitwise operators. When applying bitwise operators, the number is first converted to a 32-bit integer.

> **Note:** Although bitwise operators *can* be used to represent several Boolean values within a single number using bit masking, this is usually considered a bad practice. JavaScript offers other means to represent a set of Booleans (like an array of Booleans, or an object with Boolean values assigned to named properties). Bit masking also tends to make the code more difficult to read, understand, and maintain.

It may be necessary to use such techniques in very constrained environments, like when trying to cope with the limitations of local storage, or in extreme cases (such as when each bit over the network counts). This technique should only be considered when it is the last measure that can be taken to optimize size.

BigInt type

The `BigInt` type is a numeric primitive in JavaScript that can represent integers with arbitrary magnitude. With BigInts, you can safely store and operate on large integers even beyond the safe integer limit (`Number.MAX_SAFE_INTEGER`) for Numbers.

A BigInt is created by appending `n` to the end of an integer or by calling the `BigInt()` function.

This example demonstrates where incrementing the `Number.MAX_SAFE_INTEGER` returns the expected result:

```
// BigInt
const x = BigInt(Number.MAX_SAFE_INTEGER); // 9007199254740991n
x + 1n === x + 2n; // false because 9007199254740992n and 9007199254740993n are unequal

// Number
Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2; // true because both are 9007199254740992
```

You can use most operators to work with BigInts, including `+`, `*`, `-`, `**`, and `%` — the only forbidden one is `>>>`. A BigInt is not strictly equal to a Number with the same mathematical value, but it is loosely so.

BigInt values are neither always more precise nor always less precise than numbers, since BigInts cannot represent fractional numbers, but can represent big integers more accurately. Neither type entails the other, and they are not mutually substitutable. A `TypeError` is thrown if BigInt values are mixed with regular numbers in arithmetic expressions, or if they are implicitly converted to each other.

String type

The `String` type represents textual data and is encoded as a sequence of 16-bit unsigned integer values representing UTF-16 code units. Each element in the string occupies a position in the string. The first element is at index `0`, the next at index `1`, and so on. The length of a string is the number of UTF-16 code units in it, which may not correspond to the actual number of Unicode characters; see the `String` reference page for more details.

JavaScript strings are immutable. This means that once a string is created, it is not possible to modify it. String methods create new strings based on the content of the current string — for example:

- A substring of the original using `substring()`.
- A concatenation of two strings using the concatenation operator (+) or `concat()`.

Beware of "stringly-typing" your code!

It can be tempting to use strings to represent complex data. Doing this comes with short-term benefits:

- It is easy to build complex strings with concatenation.
- Strings are easy to debug (what you see printed is always what is in the string).
- Strings are the common denominator of a lot of APIs (input fields, local storage values, `fetch()` responses when using `Response.text()`, etc.) and it can be tempting to only work with strings.

With conventions, it is possible to represent any data structure in a string. This does not make it a good idea. For instance, with a separator, one could emulate a list (while a JavaScript array would be more suitable). Unfortunately, when the separator is used in one of the "list" elements, then, the list is broken. An escape character can be chosen, etc. All of this requires conventions and creates an unnecessary maintenance burden.

Use strings for textual data. When representing complex data, *parse* strings, and use the appropriate abstraction.

Symbol type

A `Symbol` is a **unique** and **immutable** primitive value and may be used as the key of an Object property (see below). In some programming languages, Symbols are called "atoms". The purpose of symbols is to create unique property keys that are guaranteed not to clash with keys from other code.

Objects

In computer science, an object is a value in memory which is possibly referenced by an identifier. In JavaScript, objects are the only mutable values. Functions are, in fact, also objects with the additional capability of being *callable*.

Properties

In JavaScript, objects can be seen as a collection of properties. With the object literal syntax, a limited set of properties are initialized; then properties can be added and removed. Object properties are equivalent to key-value pairs. Property keys are either strings or symbols. When other types (such as numbers) are used to index objects, the values are implicitly converted to strings. Property values can be values of any type, including other objects, which enables building complex data structures.

There are two types of object properties: The *data* property and the *accessor* property. Each property has corresponding *attributes*. Each attribute is accessed internally by the JavaScript engine, but you can set them through `Object.defineProperty()`, or read them through `Object.getOwnPropertyDescriptor()`. You can read more about the various nuances on the `Object.defineProperty()` page.

Data property

Data properties associate a key with a value. It can be described by the following attributes:

`value`

> The value retrieved by a get access of the property. Can be any JavaScript value.

`writable`

A boolean value indicating if the property can be changed with an assignment.

`enumerable`

A boolean value indicating if the property can be enumerated by a `for...in` loop. See also Enumerability and ownership of properties for how enumerability interacts with other functions and syntaxes.

`configurable`

A boolean value indicating if the property can be deleted, can be changed to an accessor property, and can have its attributes changed.

Accessor property

Associates a key with one of two accessor functions (`get` and `set`) to retrieve or store a value.

> **Note:** It's important to recognize it's accessor *property* — not accessor *method*. We can give a JavaScript object class-like accessors by using a function as a value — but that doesn't make the object a class.

An accessor property has the following attributes:

`get`

A function called with an empty argument list to retrieve the property value whenever a get access to the value is performed. See also getters. May be `undefined`.

`set`

A function called with an argument that contains the assigned value. Executed whenever a specified property is attempted to be changed. See also setters. May be `undefined`.

`enumerable`

A boolean value indicating if the property can be enumerated by a `for...in` loop. See also Enumerability and ownership of properties for how enumerability interacts with other functions and syntaxes.

`configurable`

A boolean value indicating if the property can be deleted, can be changed to a data property, and can have its attributes changed.

The prototype of an object points to another object or to `null` — it's conceptually a hidden property of the object, commonly represented as `[[Prototype]]`. Properties of the object's `[[Prototype]]` can also be accessed on the object itself.

Objects are ad-hoc key-value pairs, so they are often used as maps. However, there can be ergonomics, security, and performance issues. Use a `Map` for storing arbitrary data instead. The `Map` reference contains a more detailed discussion of the pros & cons between plain objects and maps for storing key-value associations.

Dates

JavaScript provides two sets of APIs for representing dates: the legacy `Date` object and the modern `Temporal` object. `Date` has many undesirable design choices and should be avoided in new code if possible.

Indexed collections: Arrays and typed Arrays

Arrays are regular objects for which there is a particular relationship between integer-keyed properties and the `length` property.

Additionally, arrays inherit from `Array.prototype`, which provides a handful of convenient methods to manipulate arrays. For example, `indexOf()` searches a value in the array and `push()` appends an element to the array. This makes Arrays a perfect candidate to represent ordered lists.

Typed Arrays present an array-like view of an underlying binary data buffer, and offer many methods that have similar semantics to the array counterparts. "Typed array" is an umbrella term for a range of data structures, including `Int8Array`, `Float32Array`, etc. Check the typed array page for more information. Typed arrays are often used in conjunction with `ArrayBuffer` and `DataView`.

Keyed collections: Maps, Sets, WeakMaps, WeakSets

These data structures take object references as keys. `Set` and `WeakSet` represent a collection of unique values, while `Map` and `WeakMap` represent a collection of key-value associations.

You could implement `Maps` and `Sets` yourself. However, since objects cannot be compared (in the sense of `<` "less than", for instance), neither does the engine expose its hash function for objects, look-up performance would necessarily be linear. Native implementations of them (including `WeakMaps`) can have look-up performance that is approximately logarithmic to constant time.

Usually, to bind data to a DOM node, one could set properties directly on the object, or use `data-*` attributes. This has the downside that the data is available to any script running in the same context. `Maps` and `WeakMaps` make it easy to *privately* bind data to an object.

`WeakMap` and `WeakSet` only allow garbage-collectable values as keys, which are either objects or non-registered symbols, and the keys may be collected even when they remain in the collection. They are specifically used for memory usage optimization.

Structured data: JSON

JSON (**J**ava**S**cript **O**bject **N**otation) is a lightweight data-interchange format, derived from JavaScript, but used by many programming languages. JSON builds universal data structures that can be transferred between different environments and even across languages. See `JSON` for more details.

More objects in the standard library

JavaScript has a standard library of built-in objects. Read the reference to find out more about the built-in objects.

Type coercion

As mentioned above, JavaScript is a weakly typed language. This means that you can often use a value of one type where another type is expected, and the language will convert it to the right type for you. To do so, JavaScript defines a handful of coercion rules.

Primitive coercion

The primitive coercion process is used where a primitive value is expected, but there's no strong preference for what the actual type should be. This is usually when a string, a number, or a BigInt are equally acceptable. For example:

- The `Date()` constructor, when it receives one argument that's not a `Date` instance — strings represent date strings, while numbers represent timestamps.
- The `+` operator — if one operand is a string, string concatenation is performed; otherwise, numeric addition is performed.
- The `==` operator — if one operand is a primitive while the other is an object, the object is converted to a primitive value with no preferred type.

This operation does not do any conversion if the value is already a primitive. Objects are converted to primitives by calling its `[Symbol.toPrimitive]()` (with `"default"` as hint), `valueOf()`, and `toString()` methods, in that order. Note that primitive conversion calls `valueOf()` before `toString()`, which is similar to the behavior of number coercion but different from string coercion.

The `[Symbol.toPrimitive]()` method, if present, must return a primitive — returning an object results in a `TypeError`. For `valueOf()` and `toString()`, if one returns an object, the return value is ignored and the other's return value is used instead; if neither is present, or neither returns a primitive, a `TypeError` is thrown. For example, in the following code:

```
console.log({} + []); // "[object Object]"
```

Neither `{}` nor `[]` have a `[Symbol.toPrimitive]()` method. Both `{}` and `[]` inherit `valueOf()` from `Object.prototype.valueOf`, which returns the object itself. Since the return value is an object, it is ignored. Therefore, `toString()` is called instead. `{}.toString()` returns `"[object Object]"`, while `[].toString()` returns `""`, so the result is their concatenation: `"[object Object]"`.

The `[Symbol.toPrimitive]()` method always takes precedence when doing conversion to any primitive type. Primitive conversion generally behaves like number conversion, because `valueOf()` is called in priority; however, objects with custom `[Symbol.toPrimitive]()` methods can choose to return any primitive. `Date` and `Symbol` objects are the only built-in objects that override the `[Symbol.toPrimitive]()` method. `Date.prototype[Symbol.toPrimitive]()` treats the `"default"` hint as if it's `"string"`, while `Symbol.prototype[Symbol.toPrimitive]()` ignores the hint and always returns a symbol.

Numeric coercion

There are two numeric types: Number and BigInt. Sometimes the language specifically expects a number or a BigInt (such as `Array.prototype.slice()`, where the index must be a number); other times, it may tolerate either and perform different operations depending on the operand's type. For strict coercion processes that do not allow implicit conversion from the other type, see number coercion and BigInt coercion.

Numeric coercion is nearly the same as number coercion, except that BigInts are returned as-is instead of causing a `TypeError`. Numeric coercion is used by all arithmetic operators, since they are overloaded for both numbers and BigInts. The only exception is unary plus, which always does number coercion.

Other coercions

All data types, except Null, Undefined, and Symbol, have their respective coercion process. See string coercion, boolean coercion, and object coercion for more details.

As you may have noticed, there are three distinct paths through which objects may be converted to primitives:

- Primitive coercion: `[Symbol.toPrimitive]("default")` → `valueOf()` → `toString()`
- Numeric coercion, number coercion, BigInt coercion: `[Symbol.toPrimitive]("number")` → `valueOf()` → `toString()`
- String coercion: `[Symbol.toPrimitive]("string")` → `toString()` → `valueOf()`

In all cases, `[Symbol.toPrimitive]()`, if present, must be callable and return a primitive, while `valueOf` or `toString` will be ignored if they are not callable or return an object. At the end of the process, if successful, the result is guaranteed to be a primitive. The resulting primitive is then subject to further coercions depending on the context.

See also

- JavaScript Data Structures and Algorithms by Oleksii Trekhleb
- Computer Science in JavaScript by Nicholas C. Zakas

Enumerability and ownership of properties

Every property in JavaScript objects can be classified by three factors:

- Enumerable or non-enumerable;
- String or symbol;
- Own property or inherited property from the prototype chain.

Enumerable properties are those properties whose internal enumerable flag is set to true, which is the default for properties created via simple assignment or via a property initializer. Properties defined via `Object.defineProperty` and such are not enumerable by default. Most iteration means (such as `for...in` loops and `Object.keys`) only visit enumerable keys.

Ownership of properties is determined by whether the property belongs to the object directly and not to its prototype chain.

All properties, enumerable or not, string or symbol, own or inherited, can be accessed with dot notation or bracket notation. In this section, we will focus on the means provided by JavaScript to visit a group of object properties one-by-one.

Querying object properties

There are four built-in ways to query a property of an object. They all support both string and symbol keys. The following table summarizes when each method returns `true`.

	Enumerable, own	Enumerable, inherited	Non-enumerable, own	Non-enumerable, inherited
`propertyIsEnumerable()`	`true` ✅	`false` ❌	`false` ❌	`false` ❌
`hasOwnProperty()`	`true` ✅	`false` ❌	`true` ✅	`false` ❌
`Object.hasOwn()`	`true` ✅	`false` ❌	`true` ✅	`false` ❌
`in`	`true` ✅	`true` ✅	`true` ✅	`true` ✅

Traversing object properties

There are many methods in JavaScript that traverse a group of properties of an object. Sometimes, these properties are returned as an array; sometimes, they are iterated one-by-one in a loop; sometimes, they are used for constructing or mutating another object. The following table summarizes when a property may be visited.

Methods that only visit string properties or only symbol properties will have an extra note. ✅ means a property of this type will be visited; ❌ means it will not.

	Enumerable, own	Enumerable, inherited	Non-enumerable, own	Non-enumerable, inherited

`Object.keys`
`Object.values`
`Object.entries`
	✅ (strings)	❌	❌	❌

`Object.getOwnPropertyNames`	✅ (strings)	❌	✅ (strings)	❌
`Object.getOwnPropertySymbols`	✅ (symbols)	❌	✅ (symbols)	❌
`Object.getOwnPropertyDescriptors`	✅	❌	✅	❌
`Reflect.ownKeys`	✅	❌	✅	❌
`for...in`	✅ (strings)	✅ (strings)	❌	❌
`Object.assign` (After the first parameter)	✅	❌	❌	❌
Object spread	✅	❌	❌	❌

Obtaining properties by enumerability/ownership

Note that this is not the most efficient algorithm for all cases, but useful for a quick demonstration.

- Detection can occur by `SimplePropertyRetriever.theGetMethodYouWant(obj).includes(prop)`
- Iteration can occur by `SimplePropertyRetriever.theGetMethodYouWant(obj).forEach((value, prop) => {});` (or use `filter()`, `map()`, etc.)

```
const SimplePropertyRetriever = {
  getOwnEnumProps(obj) {
    return this._getPropertyNames(obj, true, false, this._enumerable);
    // Or could use for...in filtered with Object.hasOwn or just this: return Object.keys(obj);
  },
  getOwnNonEnumProps(obj) {
    return this._getPropertyNames(obj, true, false, this._notEnumerable);
  },
  getOwnProps(obj) {
    return this._getPropertyNames(
      obj,
      true,
      false,
      this._enumerableAndNotEnumerable,
    );
    // Or just use: return Object.getOwnPropertyNames(obj);
  },
  getPrototypeEnumProps(obj) {
    return this._getPropertyNames(obj, false, true, this._enumerable);
  },
  getPrototypeNonEnumProps(obj) {
    return this._getPropertyNames(obj, false, true, this._notEnumerable);
  },
  getPrototypeProps(obj) {
    return this._getPropertyNames(
      obj,
      false,
      true,
      this._enumerableAndNotEnumerable,
    );
  },
  getOwnAndPrototypeEnumProps(obj) {
    return this._getPropertyNames(obj, true, true, this._enumerable);
    // Or could use unfiltered for...in
  },
  getOwnAndPrototypeNonEnumProps(obj) {
```

```
    return this._getPropertyNames(obj, true, true, this._notEnumerable);
  },
  getOwnAndPrototypeEnumAndNonEnumProps(obj) {
    return this._getPropertyNames(
      obj,
      true,
      true,
      this._enumerableAndNotEnumerable,
    );
  },
  // Private static property checker callbacks
  _enumerable(obj, prop) {
    return Object.prototype.propertyIsEnumerable.call(obj, prop);
  },
  _notEnumerable(obj, prop) {
    return !Object.prototype.propertyIsEnumerable.call(obj, prop);
  },
  _enumerableAndNotEnumerable(obj, prop) {
    return true;
  },
  // Inspired by http://stackoverflow.com/a/8024294/271577
  _getPropertyNames(obj, iterateSelf, iteratePrototype, shouldInclude) {
    const props = [];
    do {
      if (iterateSelf) {
        Object.getOwnPropertyNames(obj).forEach((prop) => {
          if (props.indexOf(prop) === -1 && shouldInclude(obj, prop)) {
            props.push(prop);
          }
        });
      }
      if (!iteratePrototype) {
        break;
      }
      iterateSelf = true;
      obj = Object.getPrototypeOf(obj);
    } while (obj);
    return props;
  },
};
```

See also

- `in`
- `for...in`
- `Object.prototype.hasOwnProperty()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.getOwnPropertyNames()`
- `Object.getOwnPropertySymbols()`
- `Object.keys()`
- `Object.getOwnPropertyDescriptors()`
- `Object.hasOwn()`
- `Reflect.ownKeys()`

Equality comparisons and sameness

JavaScript provides three different value-comparison operations:

- `===` — strict equality (triple equals)
- `==` — loose equality (double equals)
- `Object.is()`

Which operation you choose depends on what sort of comparison you are looking to perform. Briefly:

- Double equals (`==`) will perform a type conversion when comparing two things, and will handle `NaN`, `-0`, and `+0` specially to conform to IEEE 754 (so `NaN != NaN`, and `-0 == +0`);
- Triple equals (`===`) will do the same comparison as double equals (including the special handling for `NaN`, `-0`, and `+0`) but without type conversion; if the types differ, `false` is returned.
- `Object.is()` does no type conversion and no special handling for `NaN`, `-0`, and `+0` (giving it the same behavior as `===` except on those special numeric values).

They correspond to three of four equality algorithms in JavaScript:

- IsLooselyEqual: `==`
- IsStrictlyEqual: `===`
- SameValue: `Object.is()`
- SameValueZero: used by many built-in operations

Note that the distinctions between these all have to do with their handling of primitives; none of them compares whether the parameters are conceptually similar in structure. For any non-primitive objects `x` and `y` which have the same structure but are distinct objects themselves, all of the above forms will evaluate to `false`.

Comparing the contents of distinct objects or arrays recursively is called deep equality. JavaScript does not provide a general deep comparison operator; libraries and host APIs can provide comparison utilities with different rules.

Strict equality using ===

Strict equality compares two values for equality. Neither value is implicitly converted to some other value before being compared. If the values have different types, the values are considered unequal. If the values have the same type, are not numbers, and have the same value, they're considered equal. Finally, if both values are numbers, they're considered equal if they're both not `NaN` and are the same value, or if one is `+0` and one is `-0`.

```
const num = 0;
const obj = new String("0");
const str = "0";

console.log(num === num); // true
console.log(obj === obj); // true
console.log(str === str); // true

console.log(num === obj); // false
console.log(num === str); // false
console.log(obj === str); // false
console.log(null === undefined); // false
console.log(obj === null); // false
console.log(obj === undefined); // false
```

Strict equality is almost always the correct comparison operation to use. For all values except numbers, it uses the obvious semantics: a value is only equal to itself. For numbers it uses slightly different semantics to gloss over two different edge cases. The first is that floating point zero is either positively or negatively signed. This is useful in representing certain mathematical solutions, but as most situations don't care about the difference between `+0` and `-0`, strict equality treats them as the same value. The second is that floating point includes the concept of a not-a-number value, `NaN`, to represent the solution to certain ill-defined mathematical problems: negative infinity added to positive infinity, for example. Strict equality treats `NaN` as unequal to every other value — including itself. (The only case in which `(x !== x)` is `true` is when `x` is `NaN`.)

Besides `===`, strict equality is also used by array index-finding methods including `Array.prototype.indexOf()`, `Array.prototype.lastIndexOf()`, `TypedArray.prototype.indexOf()`, `TypedArray.prototype.lastIndexOf()`, and `case`-matching. This means you cannot use `indexOf(NaN)` to find the index of a `NaN` value in an array, or use `NaN` as a `case` value in a `switch` statement and make it match anything.

```
console.log([NaN].indexOf(NaN)); // -1
switch (NaN) {
  case NaN:
    console.log("Surprise"); // Nothing is logged
}
```

Loose equality using ==

Loose equality is *symmetric*: `A == B` always has identical semantics to `B == A` for any values of `A` and `B` (except for the order of applied conversions). The behavior for performing loose equality using `==` is as follows:

1. If the operands have the same type, they are compared as follows:
   - Object: return `true` only if both operands reference the same object.
   - String: return `true` only if both operands have the same characters in the same order.
   - Number: return `true` only if both operands have the same value. `+0` and `-0` are treated as the same value. If either operand is `NaN`, return `false`; so `NaN` is never equal to `NaN`.
   - Boolean: return `true` only if operands are both `true` or both `false`.
   - BigInt: return `true` only if both operands have the same value.
   - Symbol: return `true` only if both operands reference the same symbol.
2. If one of the operands is `null` or `undefined`, the other must also be `null` or `undefined` to return `true`. Otherwise return `false`.
3. If one of the operands is an object and the other is a primitive, convert the object to a primitive.
4. At this step, both operands are converted to primitives (one of String, Number, Boolean, Symbol, and BigInt). The rest of the conversion is done case-by-case.
   - If they are of the same type, compare them using step 1.
   - If one of the operands is a Symbol but the other is not, return `false`.
   - If one of the operands is a Boolean but the other is not, convert the boolean to a number: `true` is converted to 1, and `false` is converted to 0. Then compare the two operands loosely again.
   - Number to String: convert the string to a number. Conversion failure results in `NaN`, which will guarantee the equality to be `false`.
   - Number to BigInt: compare by their mathematical value. If the number is ±Infinity or `NaN`, return `false`.
   - String to BigInt: convert the string to a BigInt using the same algorithm as the `BigInt()` constructor. If conversion fails, return `false`.

Traditionally, and according to ECMAScript, all primitives and objects are loosely unequal to `undefined` and `null`. But most browsers permit a very narrow class of objects (specifically, the `document.all` object for any page), in some contexts, to act as if they *emulate* the value `undefined`. Loose equality is one such context: `null == A` and `undefined == A` evaluate to true if, and only if, A is an object that *emulates* `undefined`. In all other cases an object is never loosely equal to `undefined` or `null`.

In most cases, using loose equality is discouraged. The result of a comparison using strict equality is easier to predict, and may evaluate more quickly due to the lack of type coercion.

The following example demonstrates loose equality comparisons involving the number primitive `0`, the bigint primitive `0n`, the string primitive `'0'`, and an object whose `toString()` value is `'0'`.

```
const num = 0;
const big = 0n;
const str = "0";
const obj = new String("0");

console.log(num == str); // true
console.log(big == num); // true
console.log(str == big); // true
```

```
console.log(num == obj); // true
console.log(big == obj); // true
console.log(str == obj); // true
```

Loose equality is only used by the == operator.

Same-value equality using Object.is()

Same-value equality determines whether two values are *functionally identical* in all contexts. (This use case demonstrates an instance of the Liskov substitution principle.) One instance occurs when an attempt is made to mutate an immutable property:

```
// Add an immutable NEGATIVE_ZERO property to the Number constructor.
Object.defineProperty(Number, "NEGATIVE_ZERO", {
  value: -0,
  writable: false,
  configurable: false,
  enumerable: false,
});

function attemptMutation(v) {
  Object.defineProperty(Number, "NEGATIVE_ZERO", { value: v });
}
```

`Object.defineProperty` will throw an exception when attempting to change an immutable property, but it does nothing if no actual change is requested. If `v` is `-0`, no change has been requested, and no error will be thrown. Internally, when an immutable property is redefined, the newly-specified value is compared against the current value using same-value equality.

Same-value equality is provided by the `Object.is` method. It's used almost everywhere in the language where a value of equivalent identity is expected.

Same-value-zero equality

Similar to same-value equality, but +0 and -0 are considered equal.

Same-value-zero equality is not exposed as a JavaScript API, but can be implemented with custom code:

```
function sameValueZero(x, y) {
  if (typeof x === "number" && typeof y === "number") {
    // x and y are equal (may be -0 and 0) or they are both NaN
    return x === y || (x !== x && y !== y);
  }
  return x === y;
}
```

Same-value-zero only differs from strict equality by treating `NaN` as equivalent, and only differs from same-value equality by treating `-0` as equivalent to `0`. This makes it usually have the most sensible behavior during searching, especially when working with `NaN`. It's used by `Array.prototype.includes()`, `TypedArray.prototype.includes()`, as well as `Map` and `Set` methods for comparing key equality.

Comparing equality methods

People often compare double equals and triple equals by saying one is an "enhanced" version of the other. For example, double equals could be said as an extended version of triple equals, because the former does everything that the latter does, but with type conversion on its operands — for example, `6 == "6"`. Alternatively, it can be claimed that double equals is the baseline, and triple equals is an enhanced version, because it requires the two operands to be the same type, so it adds an extra constraint.

However, this way of thinking implies that the equality comparisons form a one-dimensional "spectrum" where "totally strict" lies on one end and "totally loose" lies on the other. This model falls short with `Object.is`, because it isn't "looser" than double equals or "stricter" than triple equals, nor does it fit somewhere in between (i.e., being both stricter than double equals, but looser than triple equals). We can see from the sameness comparisons table below that this is due to the way that `Object.is` handles `NaN`. Notice that if `Object.is(NaN, NaN)` evaluated to `false`, we *could* say that it fits on the loose/strict spectrum as an even stricter form of triple equals, one that distinguishes between `-0` and `+0`. The `NaN` handling means this is untrue, however. Unfortunately, `Object.is` has to be thought of in terms of its specific characteristics, rather than its looseness or strictness with regard to the equality operators.

x	y	==	===	Object.is	SameValueZero
undefined	undefined	✅ true	✅ true	✅ true	✅ true
null	null	✅ true	✅ true	✅ true	✅ true
true	true	✅ true	✅ true	✅ true	✅ true
false	false	✅ true	✅ true	✅ true	✅ true
'foo'	'foo'	✅ true	✅ true	✅ true	✅ true
0	0	✅ true	✅ true	✅ true	✅ true
+0	-0	✅ true	✅ true	❌ false	✅ true
+0	0	✅ true	✅ true	✅ true	✅ true
-0	0	✅ true	✅ true	❌ false	✅ true
0n	-0n	✅ true	✅ true	✅ true	✅ true
0	false	✅ true	❌ false	❌ false	❌ false
""	false	✅ true	❌ false	❌ false	❌ false

x	y	==	===	Object.is	SameValueZero
""	0	✅ true	❌ false	❌ false	❌ false
'0'	0	✅ true	❌ false	❌ false	❌ false
'17'	17	✅ true	❌ false	❌ false	❌ false
[1, 2]	'1,2'	✅ true	❌ false	❌ false	❌ false
new String('foo')	'foo'	✅ true	❌ false	❌ false	❌ false
null	undefined	✅ true	❌ false	❌ false	❌ false
null	false	❌ false	❌ false	❌ false	❌ false
undefined	false	❌ false	❌ false	❌ false	❌ false
{ foo: 'bar' }	{ foo: 'bar' }	❌ false	❌ false	❌ false	❌ false
new String('foo')	new String('foo')	❌ false	❌ false	❌ false	❌ false
0	null	❌ false	❌ false	❌ false	❌ false
0	NaN	❌ false	❌ false	❌ false	❌ false
'foo'	NaN	❌ false	❌ false	❌ false	❌ false
NaN	NaN	❌ false	❌ false	✅ true	✅ true

When to use Object.is() versus triple equals

In general, the only time `Object.is`'s special behavior towards zeros is likely to be of interest is in the pursuit of certain meta-programming schemes, especially regarding property descriptors, when it is desirable for your work to mirror some of the characteristics of `Object.defineProperty`. If your use case does not require this, it is suggested to avoid `Object.is` and use `===` instead. Even if your requirements involve having comparisons between two `NaN` values evaluate to `true`, generally it is easier to special-case the `NaN` checks (using the `isNaN` method available from previous versions of ECMAScript) than it is to work out how surrounding computations might affect the sign of any zeros you encounter in your comparison.

Here's a non-exhaustive list of built-in methods and operators that might cause a distinction between `-0` and `+0` to manifest itself in your code:

`-` (unary negation)

> Consider the following example:
>
> ```
> const stoppingForce = obj.mass * -obj.velocity;
> ```
>
> If `obj.velocity` is `0` (or computes to `0`), a `-0` is introduced at that place and propagates out into `stoppingForce`.

`Math.atan2`, `Math.ceil`, `Math.pow`, `Math.round`

> In some cases, it's possible for a `-0` to be introduced into an expression as a return value of these methods even when no `-0` exists as one of the parameters. For example, using `Math.pow` to raise `-Infinity` to the power of any negative, odd exponent evaluates to `-0`. Refer to the documentation for the individual methods.

`Math.floor`, `Math.max`, `Math.min`, `Math.sin`, `Math.sqrt`, `Math.tan`

> It's possible to get a `-0` return value out of these methods in some cases where a `-0` exists as one of the parameters. E.g., `Math.min(-0, +0)` evaluates to `-0`. Refer to the documentation for the individual methods.

`~`, `<<`, `>>`

> Each of these operators uses the ToInt32 algorithm internally. Since there is only one representation for 0 in the internal 32-bit integer type, `-0` will not survive a round trip after an inverse operation. E.g., both `Object.is(~~(-0), -0)` and `Object.is(-0 << 2 >> 2, -0)` evaluate to `false`.

Relying on `Object.is` when the sign of zeros is not taken into account can be hazardous. Of course, when the intent is to distinguish between `-0` and `+0`, it does exactly what's desired.

Caveat: Object.is() and NaN

`Object.is()` treats all instances of `NaN` as the same value. However, since typed arrays are available, we can have distinct floating point representations of `NaN` which don't behave identically in all contexts. For example:

```
const f2b = (x) => new Uint8Array(new Float64Array([x]).buffer);
const b2f = (x) => new Float64Array(x.buffer)[0];
// Get a byte representation of NaN
const n = f2b(NaN);
// Change the first bit, which is the sign bit and doesn't matter for NaN
n[7] |= 0x80;
const nan2 = b2f(n);
console.log(nan2); // NaN
console.log(Object.is(nan2, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan2)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 255]
```

> **Note:** Implementations are allowed to canonicalize the bit representation of `NaN`, so `nan2`, when converted back to floating point, may have the same bit representation as the original `NaN`.

See also

- JS Comparison Table by dorey

Expressions and operators

- Previous
- Next

This chapter describes JavaScript's expressions and operators, including assignment, comparison, arithmetic, bitwise, logical, string, ternary and more.

At a high level, an *expression* is a valid unit of code that resolves to a value. There are two types of expressions: those that have side effects (such as assigning values) and those that purely *evaluate*.

The expression `x = 7` is an example of the first type. This expression uses the `=` *operator* to assign the value seven to the variable `x`. The expression itself evaluates to `7`.

The expression `3 + 4` is an example of the second type. This expression uses the `+` operator to add `3` and `4` together and produces a value, `7`. However, if it's not eventually part of a bigger construct (for example, a variable declaration like `const z = 3 + 4`), its result will be immediately discarded — this is usually a programmer mistake because the evaluation doesn't produce any effects.

As the examples above also illustrate, all complex expressions are joined by *operators*, such as `=` and `+`. In this section, we will introduce the following operators:

- Assignment operators
- Comparison operators
- Arithmetic operators
- Bitwise operators
- Logical operators
- BigInt operators
- String operators
- Conditional (ternary) operator
- Comma operator
- Unary operators
- Relational operators

These operators join operands either formed by higher-precedence operators or one of the basic expressions. A complete and detailed list of operators and expressions is also available in the reference.

The *precedence* of operators determines the order they are applied when evaluating an expression. For example:

```
const x = 1 + 2 * 3;
const y = 2 * 3 + 1;
```

Despite `*` and `+` coming in different orders, both expressions would result in `7` because `*` has precedence over `+`, so the `*`-joined expression will always be evaluated first. You can override operator precedence by using parentheses (which creates a grouped expression — the basic expression). To see a complete table of operator precedence as well as various caveats, see the Operator Precedence Reference page.

JavaScript has both *binary* and *unary* operators, and one special ternary operator, the conditional operator. A binary operator requires two operands, one before the operator and one after the operator:

```
operand1 operator operand2
```

For example, `3 + 4` or `x * y`. This form is called an *infix* binary operator, because the operator is placed between two operands. All binary operators in JavaScript are infix.

A unary operator requires a single operand, either before or after the operator:

```
operator operand
operand operator
```

For example, `x++` or `++x`. The `operator operand` form is called a *prefix* unary operator, and the `operand operator` form is called a *postfix* unary operator. `++` and `--` are the only postfix operators in JavaScript — all other operators, like `!`, `typeof`, etc. are prefix.

Assignment operators

An assignment operator assigns a value to its left operand based on the value of its right operand. The simple assignment operator is equal (`=`), which assigns the value of its right operand to its left operand. That is, `x = f()` is an assignment expression that assigns the value of `f()` to `x`.

There are also compound assignment operators that are shorthand for the operations listed in the following table:

Name	Shorthand operator	Meaning
Assignment	`x = f()`	`x = f()`
Addition assignment	`x += f()`	`x = x + f()`
Subtraction assignment	`x -= f()`	`x = x - f()`
Multiplication assignment	`x *= f()`	`x = x * f()`
Division assignment	`x /= f()`	`x = x / f()`
Remainder assignment	`x %= f()`	`x = x % f()`
Exponentiation assignment	`x **= f()`	`x = x ** f()`
Left shift assignment	`x <<= f()`	`x = x << f()`

Name	Shorthand operator	Meaning
Right shift assignment	`x >>= f()`	`x = x >> f()`
Unsigned right shift assignment	`x >>>= f()`	`x = x >>> f()`
Bitwise AND assignment	`x &= f()`	`x = x & f()`
Bitwise XOR assignment	`x ^= f()`	`x = x ^ f()`
Bitwise OR assignment	`x |= f()`	`x = x | f()`
Logical AND assignment	`x &&= f()`	`x && (x = f())`
Logical OR assignment	`x ||= f()`	`x || (x = f())`
Nullish coalescing assignment	`x ??= f()`	`x ?? (x = f())`

Assigning to properties

If an expression evaluates to an object, then the left-hand side of an assignment expression may make assignments to properties of that expression. For example:

```
const obj = {};

obj.x = 3;
console.log(obj.x); // Prints 3.
console.log(obj); // Prints { x: 3 }.

const key = "y";
obj[key] = 5;
console.log(obj[key]); // Prints 5.
console.log(obj); // Prints { x: 3, y: 5 }.
```

For more information about objects, read Working with Objects.

If an expression does not evaluate to an object, then assignments to properties of that expression do not assign:

```
const val = 0;
val.x = 3;

console.log(val.x); // Prints undefined.
console.log(val); // Prints 0.
```

In strict mode, the code above throws, because one cannot assign properties to primitives.

It is an error to assign values to unmodifiable properties or to properties of an expression without properties (`null` or `undefined`).

Destructuring

For more complex assignments, the destructuring syntax is a JavaScript expression that makes it possible to extract data from arrays or objects using a syntax that mirrors the construction of array and object literals.

Without destructuring, it takes multiple statements to extract values from arrays and objects:

```
const foo = ["one", "two", "three"];

const one = foo[0];
const two = foo[1];
const three = foo[2];
```

With destructuring, you can extract multiple values into distinct variables using a single statement:

```
const [one, two, three] = foo;
```

Evaluation and nesting

In general, assignments are used within a variable declaration (i.e., with `const`, `let`, or `var`) or as standalone statements.

```
// Declares a variable x and initializes it to the result of f().
// The result of the x = f() assignment expression is discarded.
let x = f();

x = g(); // Reassigns the variable x to the result of g().
```

However, like other expressions, assignment expressions like `x = f()` evaluate into a result value. Although this result value is usually not used, it can then be used by another expression.

Chaining assignments or nesting assignments in other expressions can result in surprising behavior. For this reason, some JavaScript style guides discourage chaining or nesting assignments. Nevertheless, assignment chaining and nesting may occur sometimes, so it is important to be able to understand how they work.

By chaining or nesting an assignment expression, its result can itself be assigned to another variable. It can be logged, it can be put inside an array literal or function call, and so on.

```
let x;
const y = (x = f()); // Or equivalently: const y = x = f();
```

```
console.log(y); // Logs the return value of the assignment x = f().

console.log(x = f()); // Logs the return value directly.

// An assignment expression can be nested in any place
// where expressions are generally allowed,
// such as array literals' elements or as function calls' arguments.
console.log([0, x = f(), 0]);
console.log(f(0, x = f(), 0));
```

The evaluation result matches the expression to the right of the `=` sign in the "Meaning" column of the table above. That means that `x = f()` evaluates into whatever `f()`'s result is, `x += f()` evaluates into the resulting sum `x + f()`, `x **= f()` evaluates into the resulting power `x ** f()`, and so on.

In the case of logical assignments, `x &&= f()`, `x ||= f()`, and `x ??= f()`, the return value is that of the logical operation without the assignment, so `x && f()`, `x || f()`, and `x ?? f()`, respectively.

When chaining these expressions without parentheses or other grouping operators like array literals, the assignment expressions are **grouped right to left** (they are right-associative), but they are **evaluated left to right**.

Note that, for all assignment operators other than `=` itself, the resulting values are always based on the operands' values *before* the operation.

For example, assume that the following functions `f` and `g` and the variables `x` and `y` have been declared:

```
function f() {
  console.log("F!");
  return 2;
}
function g() {
  console.log("G!");
  return 3;
}
let x, y;
```

Consider these three examples:

```
y = x = f();
y = [f(), x = g()];
x[f()] = g();
```

Evaluation example 1

`y = x = f()` is equivalent to `y = (x = f())`, because the assignment operator `=` is right-associative. However, it evaluates from left to right:

1. The assignment expression `y = x = f()` starts to evaluate.
    1. The `y` on this assignment's left-hand side evaluates into a reference to the variable named `y`.
    2. The assignment expression `x = f()` starts to evaluate.
        1. The `x` on this assignment's left-hand side evaluates into a reference to the variable named `x`.
        2. The function call `f()` prints "F!" to the console and then evaluates to the number `2`.
        3. That `2` result from `f()` is assigned to `x`.
    3. The assignment expression `x = f()` has now finished evaluating; its result is the new value of `x`, which is `2`.
    4. That `2` result in turn is also assigned to `y`.
2. The assignment expression `y = x = f()` has now finished evaluating; its result is the new value of `y` – which happens to be `2`. `x` and `y` are assigned to `2`, and the console has printed "F!".

Evaluation example 2

`y = [ f(), x = g() ]` also evaluates from left to right:

1. The assignment expression `y = [ f(), x = g() ]` starts to evaluate.
    1. The `y` on this assignment's left-hand evaluates into a reference to the variable named `y`.
    2. The inner array literal `[ f(), x = g() ]` starts to evaluate.
        1. The function call `f()` prints "F!" to the console and then evaluates to the number `2`.
        2. The assignment expression `x = g()` starts to evaluate.
            1. The `x` on this assignment's left-hand side evaluates into a reference to the variable named `x`.
            2. The function call `g()` prints "G!" to the console and then evaluates to the number `3`.
            3. That `3` result from `g()` is assigned to `x`.
        3. The assignment expression `x = g()` has now finished evaluating; its result is the new value of `x`, which is `3`. That `3` result becomes the next element in the inner array literal (after the `2` from the `f()`).
    3. The inner array literal `[ f(), x = g() ]` has now finished evaluating; its result is an array with two values: `[ 2, 3 ]`.
    4. That `[ 2, 3 ]` array is now assigned to `y`.
2. The assignment expression `y = [ f(), x = g() ]` has now finished evaluating; its result is the new value of `y` – which happens to be `[ 2, 3 ]`. `x` is now assigned to `3`, `y` is now assigned to `[ 2, 3 ]`, and the console has printed "F!" then "G!".

Evaluation example 3

`x[f()] = g()` also evaluates from left to right. (This example assumes that `x` is already assigned to some object. For more information about objects, read Working with Objects.)

1. The assignment expression `x[f()] = g()` starts to evaluate.
    1. The `x[f()]` property access on this assignment's left-hand starts to evaluate.

    1. The `x` in this property access evaluates into a reference to the variable named `x`.
    2. Then the function call `f()` prints "F!" to the console and then evaluates to the number `2`.
  2. The `x[f()]` property access on this assignment has now finished evaluating; its result is a variable property reference: `x[2]`.
  3. Then the function call `g()` prints "G!" to the console and then evaluates to the number `3`.
  4. That `3` is now assigned to `x[2]`. (This step will succeed only if `x` is assigned to an object.)
2. The assignment expression `x[f()] = g()` has now finished evaluating; its result is the new value of `x[2]` – which happens to be `3`. `x[2]` is now assigned to `3`, and the console has printed "F!" then "G!".

Avoid assignment chains

Chaining assignments or nesting assignments in other expressions can result in surprising behavior. For this reason, chaining assignments in the same statement is discouraged.

In particular, putting a variable chain in a `const`, `let`, or `var` statement often does *not* work. Only the outermost/leftmost variable would get declared; other variables within the assignment chain are *not* declared by the `const`/`let`/`var` statement. For example:

```
const z = y = x = f();
```

This statement seemingly declares the variables `x`, `y`, and `z`. However, it only actually declares the variable `z`. `y` and `x` are either invalid references to nonexistent variables (in strict mode) or, worse, would implicitly create global variables for `x` and `y` in sloppy mode.

Comparison operators

A comparison operator compares its operands and returns a logical value based on whether the comparison is true. The operands can be numerical, string, logical, or object values. Strings are compared based on standard lexicographical ordering, using Unicode values. In most cases, if the two operands are not of the same type, JavaScript attempts to convert them to an appropriate type for the comparison. This behavior generally results in comparing the operands numerically. The sole exceptions to type conversion within comparisons involve the `===` and `!==` operators, which perform strict equality and inequality comparisons. These operators do not attempt to convert the operands to compatible types before checking equality. The following table describes the comparison operators in terms of this sample code:

```
const var1 = 3;
const var2 = 4;
```

Comparison operators

Operator	Description	Examples returning true

Equal (`==`)	Returns `true` if the operands are equal.	`3 == var1`
		`"3" == var1`
		`3 == '3'`

Not equal (`!=`)	Returns `true` if the operands are not equal.	`var1 != 4` `var2 != "3"`
Strict equal (`===`)	Returns `true` if the operands are equal and of the same type. See also `Object.is` and sameness in JS.	`3 === var1`
Strict not equal (`!==`)	Returns `true` if the operands are of the same type but not equal, or are of different type.	`var1 !== "3"` `3 !== '3'`
Greater than (`>`)	Returns `true` if the left operand is greater than the right operand.	`var2 > var1` `"12" > 2`
Greater than or equal (`>=`)	Returns `true` if the left operand is greater than or equal to the right operand.	`var2 >= var1` `var1 >= 3`
Less than (`<`)	Returns `true` if the left operand is less than the right operand.	`var1 < var2` `"2" < 12`
Less than or equal (`<=`)	Returns `true` if the left operand is less than or equal to the right operand.	`var1 <= var2` `var2 <= 5`

> **Note:** `=>` is not a comparison operator but rather is the notation for Arrow functions.

Arithmetic operators

An arithmetic operator takes numerical values (either literals or variables) as their operands and returns a single numerical value. The standard arithmetic operators are addition (`+`), subtraction (`-`), multiplication (`*`), and division (`/`). These operators work as they do in most other programming languages when used with floating point numbers (in particular, note that division by zero produces `Infinity`). For example:

```
1 / 2; // 0.5
1 / 2 === 1.0 / 2.0; // this is true
```

In addition to the standard arithmetic operations (`+`, `-`, `*`, `/`), JavaScript provides the arithmetic operators listed in the following table:

Arithmetic operators

Operator	Description	Example
Remainder (`%`)	Binary operator. Returns the integer remainder of dividing the two operands.	12 % 5 returns 2.

Operator	Description	Example
Increment (++)	Unary operator. Adds one to its operand. If used as a prefix operator (++x), returns the value of its operand after adding one; if used as a postfix operator (x++), returns the value of its operand before adding one.	If x is 3, then ++x sets x to 4 and returns 4, whereas x++ returns 3 and, only then, sets x to 4.
Decrement (--)	Unary operator. Subtracts one from its operand. The return value is analogous to that for the increment operator.	If x is 3, then --x sets x to 2 and returns 2, whereas x-- returns 3 and, only then, sets x to 2.
Unary negation (-)	Unary operator. Returns the negation of its operand.	If x is 3, then -x returns -3.
Unary plus (+)	Unary operator. Attempts to convert the operand to a number, if it is not already.	+"3" returns 3. +true returns 1.
Exponentiation operator (**)	Calculates the base to the exponent power, that is, base^exponent	2 ** 3 returns 8. 10 ** -1 returns 0.1.

Bitwise operators

A bitwise operator treats their operands as a set of 32 bits (zeros and ones), rather than as decimal, hexadecimal, or octal numbers. For example, the decimal number nine has a binary representation of 1001. Bitwise operators perform their operations on such binary representations, but they return standard JavaScript numerical values.

The following table summarizes JavaScript's bitwise operators.

Operator	Usage	Description
Bitwise AND	a & b	Returns a one in each bit position for which the corresponding bits of both operands are ones.
Bitwise OR	a | b	Returns a zero in each bit position for which the corresponding bits of both operands are zeros.
Bitwise XOR	a ^ b	Returns a zero in each bit position for which the corresponding bits are the same. [Returns a one in each bit position for which the corresponding bits are different.]
Bitwise NOT	~ a	Inverts the bits of its operand.
Left shift	a << b	Shifts a in binary representation b bits to the left, shifting in zeros from the right.
Sign-propagating right shift	a >> b	Shifts a in binary representation b bits to the right, discarding bits shifted off.
Zero-fill right shift	a >>> b	Shifts a in binary representation b bits to the right, discarding bits shifted off, and shifting in zeros from the left.

Bitwise logical operators

Conceptually, the bitwise logical operators work as follows:

- The operands are converted to thirty-two-bit integers and expressed by a series of bits (zeros and ones). Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 1110 0110 1111 1010 0000 0000 0000 0110 0000 0000 0001
After:              1010 0000 0000 0000 0110 0000 0000 0001
```

- Each bit in the first operand is paired with the corresponding bit in the second operand: first bit to first bit, second bit to second bit, and so on.
- The operator is applied to each pair of bits, and the result is constructed bitwise.

For example, the binary representation of nine is 1001, and the binary representation of fifteen is 1111. So, when the bitwise operators are applied to these values, the results are as follows:

Expression	Result	Binary Description
15 & 9	9	1111 & 1001 = 1001
15 | 9	15	1111 | 1001 = 1111
15 ^ 9	6	1111 ^ 1001 = 0110
~15	-16	~ 0000 0000 … 0000 1111 = 1111 1111 … 1111 0000
~9	-10	~ 0000 0000 … 0000 1001 = 1111 1111 … 1111 0110

Note that all 32 bits are inverted using the Bitwise NOT operator, and that values with the most significant (left-most) bit set to 1 represent negative numbers (two's-complement representation). ~x evaluates to the same value that -x - 1 evaluates to.

Bitwise shift operators

The bitwise shift operators take two operands: the first is a quantity to be shifted, and the second specifies the number of bit positions by which the first operand is to be shifted. The direction of the shift operation is controlled by the operator used.

Shift operators convert their operands to thirty-two-bit integers and return a result of either type Number or BigInt: specifically, if the type of the left operand is BigInt, they return BigInt; otherwise, they return Number.

The shift operators are listed in the following table.

Bitwise shift operators

Operator	Description	Example
Left shift (<<)	This operator shifts the first operand the specified number of bits to the left. Excess bits shifted off to the left are discarded. Zero bits are shifted in from the right.	`9<<2` yields 36, because 1001 shifted 2 bits to the left becomes 100100, which is 36.
Sign-propagating right shift (>>)	This operator shifts the first operand the specified number of bits to the right. Excess bits shifted off to the right are discarded. Copies of the leftmost bit are shifted in from the left.	`9>>2` yields 2, because 1001 shifted 2 bits to the right becomes 10, which is 2. Likewise, `-9>>2` yields -3, because the sign is preserved.
Zero-fill right shift (>>>)	This operator shifts the first operand the specified number of bits to the right. Excess bits shifted off to the right are discarded. Zero bits are shifted in from the left.	`19>>>2` yields 4, because 10011 shifted 2 bits to the right becomes 100, which is 4. For non-negative numbers, zero-fill right shift and sign-propagating right shift yield the same result.

Logical operators

Logical operators are typically used with Boolean (logical) values; when they are, they return a Boolean value. However, the `&&`, `||`, and `??` operators actually return the value of one of the specified operands, so if these operators are used with non-Boolean values, they may return a non-Boolean value. As such, they are more adequately called "value selection operators". The logical operators are described in the following table.

Logical operators

Operator	Usage	Description
Logical AND (`&&`)	`expr1 && expr2`	Returns `expr1` if it can be converted to `false`; otherwise, returns `expr2`. Thus, when used with Boolean values, `&&` returns `true` if both operands are true; otherwise, returns `false`.
Logical OR (`||`)	`expr1 || expr2`	Returns `expr1` if it can be converted to `true`; otherwise, returns `expr2`. Thus, when used with Boolean values, `||` returns `true` if either operand is true; if both are false, returns `false`.
Nullish coalescing operator (`??`)	`expr1 ?? expr2`	Returns `expr1` if it is neither `null` nor `undefined`; otherwise, returns `expr2`.
Logical NOT (`!`)	`!expr`	Returns `false` if its single operand can be converted to `true`; otherwise, returns `true`.

Examples of expressions that can be converted to `false` are those that evaluate to `null`, `0`, `0n`, `NaN`, the empty string (`""`), or `undefined`.

The following code shows examples of the `&&` (logical AND) operator.

```
const a1 = true && true; // t && t returns true
const a2 = true && false; // t && f returns false
const a3 = false && true; // f && t returns false
const a4 = false && 3 === 4; // f && f returns false
const a5 = "Cat" && "Dog"; // t && t returns Dog
const a6 = false && "Cat"; // f && t returns false
const a7 = "Cat" && false; // t && f returns false
```

The following code shows examples of the `||` (logical OR) operator.

```
const o1 = true || true; // t || t returns true
const o2 = false || true; // f || t returns true
const o3 = true || false; // t || f returns true
const o4 = false || 3 === 4; // f || f returns false
const o5 = "Cat" || "Dog"; // t || t returns Cat
const o6 = false || "Cat"; // f || t returns Cat
const o7 = "Cat" || false; // t || f returns Cat
```

The following code shows examples of the `??` (nullish coalescing) operator.

```
const n1 = null ?? 1; // 1
const n2 = undefined ?? 2; // 2
const n3 = false ?? 3; // false
const n4 = 0 ?? 4; // 0
```

Note how `??` works like `||`, but it only returns the second expression when the first one is "nullish", i.e., `null` or `undefined`. `??` is a better alternative than `||` for setting defaults for values that might be `null` or `undefined`, in particular when values like `''` or `0` are valid values and the default should not apply.

The following code shows examples of the `!` (logical NOT) operator.

```
const n1 = !true; // !t returns false
const n2 = !false; // !f returns true
const n3 = !"Cat"; // !t returns false
```

Short-circuit evaluation

As logical expressions are evaluated left to right, they are tested for possible "short-circuit" evaluation using the following rules:

- `falsy && anything` is short-circuit evaluated to the falsy value.
- `truthy || anything` is short-circuit evaluated to the truthy value.
- `nonNullish ?? anything` is short-circuit evaluated to the non-nullish value.

The rules of logic guarantee that these evaluations are always correct. Note that the *anything* part of the above expressions is not evaluated, so any side effects of doing so do not take effect.

BigInt operators

Most operators that can be used between numbers can be used between `BigInt` values as well.

```
// BigInt addition
const a = 1n + 2n; // 3n
// Division with BigInts round towards zero
const b = 1n / 2n; // 0n
// Bitwise operations with BigInts do not truncate either side
const c = 4000000000000000n >> 2n; // 1000000000000000n
```

One exception is unsigned right shift (>>>), which is not defined for BigInt values. This is because a BigInt does not have a fixed width, so technically it does not have a "highest bit".

```
const d = 8n >>> 2n; // TypeError: BigInts have no unsigned right shift, use >> instead
```

BigInts and numbers are not mutually replaceable — you cannot mix them in calculations.

```
const a = 1n + 2; // TypeError: Cannot mix BigInt and other types
```

This is because BigInt is neither a subset nor a superset of numbers. BigInts have higher precision than numbers when representing large integers, but cannot represent decimals, so implicit conversion on either side might lose precision. Use explicit conversion to signal whether you wish the operation to be a number operation or a BigInt one.

```
const a = Number(1n) + 2; // 3
const b = 1n + BigInt(2); // 3n
```

You can compare BigInts with numbers.

```
const a = 1n > 2; // false
const b = 3 > 2n; // true
```

String operators

In addition to the comparison operators, which can be used on string values, the concatenation operator (+) concatenates two string values together, returning another string that is the union of the two operand strings.

For example,

```
console.log("my " + "string"); // console logs the string "my string".
```

The shorthand assignment operator += can also be used to concatenate strings.

For example,

```
let myString = "alpha";
myString += "bet"; // evaluates to "alphabet" and assigns this value to myString.
```

Conditional (ternary) operator

The conditional operator is the only JavaScript operator that takes three operands. The operator can have one of two values based on a condition. The syntax is:

```
condition ? val1 : val2
```

If `condition` is true, the operator has the value of `val1`. Otherwise it has the value of `val2`. You can use the conditional operator anywhere you would use a standard operator.

For example,

```
const status = age >= 18 ? "adult" : "minor";
```

This statement assigns the value "adult" to the variable `status` if `age` is eighteen or more. Otherwise, it assigns the value "minor" to `status`.

Comma operator

The comma operator (`,`) evaluates both of its operands and returns the value of the last operand. This operator is primarily used inside a `for` loop, to allow multiple variables to be updated each time through the loop. It is regarded bad style to use it elsewhere, when it is not necessary. Often two separate statements can and should be used instead.

For example, if `a` is a 2-dimensional array with 10 elements on a side, the following code uses the comma operator to update two variables at once. The code prints the values of the diagonal elements in the array:

```
const x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];
const a = [x, x, x, x, x];

for (let i = 0, j = 9; i <= j; i++, j--) {
  //              ^
  console.log(`a[${i}][${j}]= ${a[i][j]}`);
}
```

Unary operators

A unary operation is an operation with only one operand.

delete

The `delete` operator deletes an object's property. The syntax is:

```
delete object.property;
delete object[propertyKey];
delete objectName[index];
```

where `object` is the name of an object, `property` is an existing property, and `propertyKey` is a string or symbol referring to an existing property.

If the `delete` operator succeeds, it removes the property from the object. Trying to access it afterwards will yield `undefined`. The `delete` operator returns `true` if the operation is possible; it returns `false` if the operation is not possible.

```
delete Math.PI; // returns false (cannot delete non-configurable properties)

const myObj = { h: 4 };
delete myObj.h; // returns true (can delete user-defined properties)
```

Deleting array elements

Since arrays are just objects, it's technically possible to `delete` elements from them. This is, however, regarded as a bad practice — try to avoid it. When you delete an array property, the array length is not affected and other elements are not re-indexed. To achieve that behavior, it is much better to just overwrite the element with the value `undefined`. To actually manipulate the array, use the various array methods such as `splice`.

typeof

The `typeof` operator returns a string indicating the type of the unevaluated operand. `operand` is the string, variable, keyword, or object for which the type is to be returned. The parentheses are optional.

Suppose you define the following variables:

```
const myFun = () => 5 + 2;
const shape = "round";
const size = 1;
const foo = ["Apple", "Mango", "Orange"];
const today = new Date();
```

The `typeof` operator returns the following results for these variables:

```
typeof myFun; // returns "function"
typeof shape; // returns "string"
typeof size; // returns "number"
typeof foo; // returns "object"
typeof today; // returns "object"
typeof doesntExist; // returns "undefined"
```

For the keywords `true` and `null`, the `typeof` operator returns the following results:

```
typeof true; // returns "boolean"
typeof null; // returns "object"
```

For a number or string, the `typeof` operator returns the following results:

```
typeof 62; // returns "number"
typeof "Hello world"; // returns "string"
```

For property values, the `typeof` operator returns the type of value the property contains:

```
typeof document.lastModified; // returns "string"
typeof window.length; // returns "number"
typeof Math.LN2; // returns "number"
```

For methods and functions, the `typeof` operator returns results as follows:

```
typeof blur; // returns "function"
typeof parseInt; // returns "function"
typeof shape.split; // returns "function"
```

For predefined objects, the `typeof` operator returns results as follows:

```
typeof Date; // returns "function"
typeof Function; // returns "function"
typeof Math; // returns "object"
typeof Option; // returns "function"
typeof String; // returns "function"
```

void

The `void` operator specifies an expression to be evaluated without returning a value. `expression` is a JavaScript expression to evaluate. The parentheses surrounding the expression are optional, but it is good style to use them to avoid precedence issues.

Relational operators

A relational operator compares its operands and returns a Boolean value based on whether the comparison is true.

in

The `in` operator returns `true` if the specified property is in the specified object. The syntax is:

```
propNameOrNumber in objectName
```

where `propNameOrNumber` is a string, numeric, or symbol expression representing a property name or array index, and `objectName` is the name of an object.

The following examples show some uses of the `in` operator.

```
// Arrays
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
0 in trees; // returns true
3 in trees; // returns true
6 in trees; // returns false
"bay" in trees; // returns false
// (you must specify the index number, not the value at that index)
"length" in trees; // returns true (length is an Array property)

// built-in objects
"PI" in Math; // returns true
const myString = new String("coral");
"length" in myString; // returns true

// Custom objects
const myCar = { make: "Honda", model: "Accord", year: 1998 };
"make" in myCar; // returns true
"model" in myCar; // returns true
```

instanceof

The `instanceof` operator returns `true` if the specified object is of the specified object type. The syntax is:

```
object instanceof objectType
```

where `object` is the object to test against `objectType`, and `objectType` is a constructor representing a type, such as `Map` or `Array`.

Use `instanceof` when you need to confirm the type of an object at runtime. For example, when catching exceptions, you can branch to different exception-handling code depending on the type of exception thrown.

For example, the following code uses `instanceof` to determine whether `obj` is a `Map` object. Because `obj` is a `Map` object, the statements inside the `if` block execute.

```
const obj = new Map();
if (obj instanceof Map) {
  // statements to execute
}
```

Basic expressions

All operators eventually operate on one or more basic expressions. These basic expressions include identifiers and literals, but there are a few other kinds as well. They are briefly introduced below, and their semantics are described in detail in their respective reference sections.

this

The `this` keyword is usually used within a function. In general, when the function is attached to an object as a method, `this` refers to the object that the method is called on. It functions like a hidden parameter that is passed to the function. `this` is an expression that evaluates to the object, so you can use all the object operations we introduced.

```
this["propertyName"];
this.propertyName;
doSomething(this);
```

For example, suppose a function is defined as follows:

```
function getFullName() {
  return `${this.firstName} ${this.lastName}`;
}
```

We can now attach this function to an object, and it will use the properties of that object when called:

```
const person1 = {
  firstName: "Chris",
```

```
  lastName: "Martin",
};

const person2 = {
  firstName: "Chester",
  lastName: "Bennington",
};

// Attach the same function
person1.getFullName = getFullName;
person2.getFullName = getFullName;

console.log(person1.getFullName()); // "Chris Martin"
console.log(person2.getFullName()); // "Chester Bennington"
```

Grouping operator

The grouping operator `( )` controls the precedence of evaluation in expressions. For example, you can override multiplication and division first, then addition and subtraction to evaluate addition first.

```
const a = 1;
const b = 2;
const c = 3;

// default precedence
a + b * c; // 7
// evaluated by default like this
a + (b * c); // 7

// now overriding precedence
// addition before multiplication
(a + b) * c; // 9

// which is equivalent to
a * c + b * c; // 9
```

Property accessor

The property accessor syntax gets property values on objects, using either dot notation or bracket notation.

```
object.property;
object["property"];
```

The working with objects guide goes into more details about object properties.

Optional chaining

The optional chaining syntax (`?.`) performs the chained operation on an object if it is defined and non-`null`, and otherwise short-circuits the operation and returns `undefined`. This allows you to operate on a value that may be `null` or `undefined` without causing a `TypeError`.

```
maybeObject?.property;
maybeObject?.[property];
maybeFunction?.();
```

new

You can use the `new` operator to create an instance of a user-defined object type or of one of the built-in object types. Use `new` as follows:

```
const objectName = new ObjectType(param1, param2, /* …, */ paramN);
```

super

The `super` keyword is used to call functions on an object's parent. It is useful with classes to call the parent constructor, for example.

```
super(args); // calls the parent constructor.
super.functionOnParent(args);
```


- Previous
- Next

Functions

- Previous
- Next

Functions are one of the fundamental building blocks in JavaScript. A function in JavaScript is similar to a procedure—a set of statements that performs a task or calculates a value, but for a procedure to qualify as a function, it should take some input and return an output where there is some obvious relationship between the input and the output. To use a function, you must define it somewhere in the scope from which you wish to call it.

See also the exhaustive reference chapter about JavaScript functions to get to know the details.

Defining functions

Function declarations

A **function definition** (also called a **function declaration**, or **function statement**) consists of the `function` keyword, followed by:

- The name of the function.
- A list of parameters to the function, enclosed in parentheses and separated by commas.
- The JavaScript statements that define the function, enclosed in curly braces, `{ /* … */ }`.

For example, the following code defines a function named `square`:

```
function square(number) {
  return number * number;
}
```

The function `square` takes one parameter, called `number`. The function consists of one statement that says to return the parameter of the function (that is, `number`) multiplied by itself. The `return` statement specifies the value returned by the function, which is `number * number`.

Parameters are essentially passed to functions **by value** — so if the code within the body of a function assigns a completely new value to a parameter that was passed to the function, **the change is not reflected globally or in the code which called that function**.

When you pass an object as a parameter, if the function changes the object's properties, that change is visible outside the function, as shown in the following example:

```
function myFunc(theObject) {
  theObject.make = "Toyota";
}

const myCar = {
  make: "Honda",
  model: "Accord",
  year: 1998,
};

console.log(myCar.make); // "Honda"
myFunc(myCar);
console.log(myCar.make); // "Toyota"
```

When you pass an array as a parameter, if the function changes any of the array's values, that change is visible outside the function, as shown in the following example:

```
function myFunc(theArr) {
  theArr[0] = 30;
}

const arr = [45];

console.log(arr[0]); // 45
myFunc(arr);
console.log(arr[0]); // 30
```

Function declarations and expressions can be nested, which forms a *scope chain*. For example:

```
function addSquares(a, b) {
  function square(x) {
    return x * x;
  }
  return square(a) + square(b);
}
```

See function scopes and closures for more information.

Function expressions

While the function declaration above is syntactically a statement, functions can also be created by a function expression.

Such a function can be **anonymous**; it does not have to have a name. For example, the function `square` could have been defined as:

```
const square = function (number) {
  return number * number;
};

console.log(square(4)); // 16
```

However, a name *can* be provided with a function expression. Providing a name allows the function to refer to itself, and also makes it easier to identify the function in a debugger's stack traces:

```
const factorial = function fac(n) {
  return n < 2 ? 1 : n * fac(n - 1);
};

console.log(factorial(3)); // 6
```

Function expressions are convenient when passing a function as an argument to another function. The following example defines a `map` function that should receive a function as first argument and an array as second argument. Then, it is called with a function defined by a function expression:

```
function map(f, a) {
  const result = new Array(a.length);
  for (let i = 0; i < a.length; i++) {
    result[i] = f(a[i]);
  }
  return result;
}

const numbers = [0, 1, 2, 5, 10];
const cubedNumbers = map(function (x) {
  return x * x * x;
}, numbers);
console.log(cubedNumbers); // [0, 1, 8, 125, 1000]
```

In JavaScript, a function can be defined based on a condition. For example, the following function definition defines `myFunc` only if `num` equals `0`:

```
let myFunc;
if (num === 0) {
  myFunc = function (theObject) {
    theObject.make = "Toyota";
  };
}
```

In addition to defining functions as described here, you can also use the `Function` constructor to create functions from a string at runtime, much like `eval()`.

A **method** is a function that is a property of an object. Read more about objects and methods in Working with objects.

Calling functions

Defining a function does not *execute* it. Defining it names the function and specifies what to do when the function is called.

Calling the function actually performs the specified actions with the indicated parameters. For example, if you define the function `square`, you could call it as follows:

```
square(5);
```

The preceding statement calls the function with an argument of `5`. The function executes its statements and returns the value `25`.

Functions must be *in scope* when they are called, but the function declaration can be hoisted (appear below the call in the code). The scope of a function declaration is the function in which it is declared (or the entire program, if it is declared at the top level).

The arguments of a function are not limited to strings and numbers. You can pass whole objects to a function. The `showProps()` function (defined in Working with objects) is an example of a function that takes an object as an argument.

A function can call itself. For example, here is a function that computes factorials recursively:

```
function factorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  }
  return n * factorial(n - 1);
}
```

You could then compute the factorials of `1` through `5` as follows:

```
console.log(factorial(1)); // 1
console.log(factorial(2)); // 2
console.log(factorial(3)); // 6
console.log(factorial(4)); // 24
console.log(factorial(5)); // 120
```

There are other ways to call functions. There are often cases where a function needs to be called dynamically, or the number of arguments to a function vary, or in which the context of the function call needs to be set to a specific object determined at runtime.

It turns out that *functions are themselves objects* — and in turn, these objects have methods. (See the `Function` object.) The `call()` and `apply()` methods can be used to achieve this goal.

Function hoisting

Consider the example below:

```
console.log(square(5)); // 25

function square(n) {
  return n * n;
}
```

This code runs without any error, despite the `square()` function being called before it's declared. This is because the JavaScript interpreter hoists the entire function declaration to the top of the current scope, so the code above is equivalent to:

```
// All function declarations are effectively at the top of the scope
function square(n) {
  return n * n;
}

console.log(square(5)); // 25
```

Function hoisting only works with function *declarations* — not with function *expressions*. The following code will not work:

```
console.log(square(5)); // ReferenceError: Cannot access 'square' before initialization
const square = function (n) {
  return n * n;
};
```

Recursion

A function can refer to and call itself. It can be referred to either by the function expression or declaration's name, or via any in-scope variable that refers to the function object. For example, consider the following function definition:

```
const foo = function bar() {
  // statements go here
};
```

Within the function body, you can refer to the function itself either as `bar` or `foo`, and call itself using `bar()` or `foo()`.

A function that calls itself is called a *recursive function*. In some ways, recursion is analogous to a loop. Both execute the same code multiple times, and both require a condition (to avoid an infinite loop, or rather, infinite recursion in this case).

For example, consider the following loop:

```
let x = 0;
// "x < 10" is the loop condition
while (x < 10) {
  // do stuff
  x++;
}
```

It can be converted into a recursive function declaration, followed by a call to that function:

```
function loop(x) {
  // "x >= 10" is the exit condition (equivalent to "!(x < 10)")
  if (x >= 10) {
    return;
  }
  // do stuff
  loop(x + 1); // the recursive call
}
loop(0);
```

However, some algorithms cannot be simple iterative loops. For example, getting all the nodes of a tree structure (such as the DOM) is easier via recursion:

```
function walkTree(node) {
  if (node === null) {
    return;
  }
  // do something with node
  for (const child of node.childNodes) {
    walkTree(child);
  }
}
```

Compared to the function `loop`, each recursive call itself makes many recursive calls here.

It is possible to convert any recursive algorithm to a non-recursive one, but the logic is often much more complex, and doing so requires the use of a stack.

In fact, recursion itself uses a stack: the function stack. The stack-like behavior can be seen in the following example:

```
function foo(i) {
  if (i < 0) {
    return;
  }
  console.log(`begin: ${i}`);
  foo(i - 1);
  console.log(`end: ${i}`);
}
foo(3);

// Logs:
// begin: 3
// begin: 2
// begin: 1
// begin: 0
```

```
// end: 0
// end: 1
// end: 2
// end: 3
```

Immediately Invoked Function Expressions (IIFE)

An Immediately Invoked Function Expression (IIFE) is a code pattern that directly calls a function defined as an expression. It looks like this:

```
(function () {
  // Do something
})();

const value = (function () {
  // Do something
  return someValue;
})();
```

Instead of saving the function in a variable, the function is immediately invoked. This is almost equivalent to just writing the function body, but there are a few unique benefits:

- It creates an extra scope of variables, which helps to confine variables to the place where they are useful.
- It is now an *expression* instead of a sequence of *statements*. This allows you to write complex computation logic when initializing variables.

For more information, see the IIFE glossary entry.

Function scopes and closures

Functions form a scope for variables—this means variables defined inside a function cannot be accessed from anywhere outside the function. The function scope inherits from all the upper scopes. For example, a function defined in the global scope can access all variables defined in the global scope. A function defined inside another function can also access all variables defined in its parent function, and any other variables to which the parent function has access. On the other hand, the parent function (and any other parent scope) does *not* have access to the variables and functions defined inside the inner function. This provides a sort of encapsulation for the variables in the inner function.

```
// The following variables are defined in the global scope
const num1 = 20;
const num2 = 3;
const name = "Chamakh";

// This function is defined in the global scope
function multiply() {
  return num1 * num2;
}

console.log(multiply()); // 60

// A nested function example
function getScore() {
  const num1 = 2;
  const num2 = 3;

  function add() {
    return `${name} scored ${num1 + num2}`;
  }

  return add();
}

console.log(getScore()); // "Chamakh scored 5"
```

Closures

We also refer to the function body as a *closure*. A closure is any piece of source code (most commonly, a function) that refers to some variables, and the closure "remembers" these variables even when the scope in which these variables were declared has exited.

Closures are usually illustrated with nested functions to show that they remember variables beyond the lifetime of its parent scope; but in fact, nested functions are unnecessary. Technically speaking, all functions in JavaScript form closures—some just don't capture anything, and closures don't even have to be functions. The key ingredients for a *useful* closure are the following:

- A parent scope that defines some variables or functions. It should have a clear lifetime, which means it should finish execution at some point. Any scope that's not the global scope satisfies this requirement; this includes blocks, functions, modules, and more.
- An inner scope defined within the parent scope, which refers to some variables or functions defined in the parent scope.
- The inner scope manages to survive beyond the lifetime of the parent scope. For example, it is saved to a variable that's defined outside the parent scope, or it's returned from the parent scope (if the parent scope is a function).
- Then, when you call the function outside of the parent scope, you can still access the variables or functions that were defined in the parent scope, even though the parent scope has finished execution.

The following is a typical example of a closure:

```
// The outer function defines a variable called "name"
const pet = function (name) {
  const getName = function () {
```

```
    // The inner function has access to the "name" variable of the outer function
    return name;
  };
  return getName; // Return the inner function, thereby exposing it to outer scopes
};
const myPet = pet("Vivie");

console.log(myPet()); // "Vivie"
```

It can be much more complex than the code above. An object containing methods for manipulating the inner variables of the outer function can be returned.

```
const createPet = function (name) {
  let sex;

  const pet = {
    // setName(newName) is equivalent to setName: function (newName)
    // in this context
    setName(newName) {
      name = newName;
    },

    getName() {
      return name;
    },

    getSex() {
      return sex;
    },

    setSex(newSex) {
      if (
        typeof newSex === "string" &&
        (newSex.toLowerCase() === "male" || newSex.toLowerCase() === "female")
      ) {
        sex = newSex;
      }
    },
  };

  return pet;
};

const pet = createPet("Vivie");
console.log(pet.getName()); // Vivie

pet.setName("Oliver");
pet.setSex("male");
console.log(pet.getSex()); // male
console.log(pet.getName()); // Oliver
```

In the code above, the `name` variable of the outer function is accessible to the inner functions, and there is no other way to access the inner variables except through the inner functions. The inner variables of the inner functions act as safe stores for the outer arguments and variables. They hold "persistent" and "encapsulated" data for the inner functions to work with. The functions do not even have to be assigned to a variable, or have a name.

```
const getCode = (function () {
  const apiCode = "0]Eal(eh&2"; // A code we do not want outsiders to be able to modify…

  return function () {
    return apiCode;
  };
})();

console.log(getCode()); // "0]Eal(eh&2"
```

In the code above, we use the IIFE pattern. Within this IIFE scope, two values exist: a variable `apiCode` and an unnamed function that gets returned and gets assigned to the variable `getCode`. `apiCode` is in the scope of the returned unnamed function but not in the scope of any other part of the program, so there is no way for reading the value of `apiCode` apart from via the `getCode` function.

Multiply-nested functions

Functions can be multiply-nested. For example:

- A function (`A`) contains a function (`B`), which itself contains a function (`C`).
- Both functions `B` and `C` form closures here. So, `B` can access `A`, and `C` can access `B`.
- In addition, since `C` can access `B` which can access `A`, `C` can also access `A`.

Thus, the closures can contain multiple scopes; they recursively contain the scope of the functions containing it. This is called *scope chaining*. Consider the following example:

```
function A(x) {
  function B(y) {
    function C(z) {
      console.log(x + y + z);
    }
```

```
    C(3);
  }
  B(2);
}
A(1); // Logs 6 (which is 1 + 2 + 3)
```

In this example, `C` accesses `B`'s `y` and `A`'s `x`. This can be done because:

1. `B` forms a closure including `A` (i.e., `B` can access `A`'s arguments and variables).
2. `C` forms a closure including `B`.
3. Because `C`'s closure includes `B` and `B`'s closure includes `A`, then `C`'s closure also includes `A`. This means `C` can access *both* `B` *and* `A`'s arguments and variables. In other words, `C` *chains* the scopes of `B` and `A`, *in that order*.

The reverse, however, is not true. `A` cannot access `C`, because `A` cannot access any argument or variable of `B`, which `C` is a variable of. Thus, `C` remains private to only `B`.

Name conflicts

When two arguments or variables in the scopes of a closure have the same name, there is a *name conflict*. More nested scopes take precedence. So, the innermost scope takes the highest precedence, while the outermost scope takes the lowest. This is the scope chain. The first on the chain is the innermost scope, and the last is the outermost scope. Consider the following:

```
function outside() {
  const x = 5;
  function inside(x) {
    return x * 2;
  }
  return inside;
}

console.log(outside()(10)); // 20 (instead of 10)
```

The name conflict happens at the statement `return x * 2` and is between `inside`'s parameter `x` and `outside`'s variable `x`. The scope chain here is `inside` => `outside` => global object. Therefore, `inside`'s `x` takes precedences over `outside`'s `x`, and `20` (`inside`'s `x`) is returned instead of `10` (`outside`'s `x`).

Using the arguments object

The arguments of a function are maintained in an array-like object. Within a function, you can address the arguments passed to it as follows:

```
arguments[i];
```

where `i` is the ordinal number of the argument, starting at `0`. So, the first argument passed to a function would be `arguments[0]`. The total number of arguments is indicated by `arguments.length`.

Using the `arguments` object, you can call a function with more arguments than it is formally declared to accept. This is often useful if you don't know in advance how many arguments will be passed to the function. You can use `arguments.length` to determine the number of arguments actually passed to the function, and then access each argument using the `arguments` object.

For example, consider a function that concatenates several strings. The only formal argument for the function is a string that specifies the characters that separate the items to concatenate. The function is defined as follows:

```
function myConcat(separator) {
  let result = ""; // initialize list
  // iterate through arguments
  for (let i = 1; i < arguments.length; i++) {
    result += arguments[i] + separator;
  }
  return result;
}
```

You can pass any number of arguments to this function, and it concatenates each argument into a string "list":

```
console.log(myConcat(", ", "red", "orange", "blue"));
// "red, orange, blue, "

console.log(myConcat("; ", "elephant", "giraffe", "lion", "cheetah"));
// "elephant; giraffe; lion; cheetah; "

console.log(myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley"));
// "sage. basil. oregano. pepper. parsley. "
```

> **Note:** The `arguments` variable is "array-like", but not an array. It is array-like in that it has a numbered index and a `length` property. However, it does *not* possess all of the array-manipulation methods.

See the `Function` object in the JavaScript reference for more information.

Function parameters

There are two special kinds of parameter syntax: *default parameters* and *rest parameters*.

Default parameters

In JavaScript, parameters of functions default to `undefined`. However, in some situations it might be useful to set a different default value. This is exactly what default parameters do.

In the past, the general strategy for setting defaults was to test parameter values in the body of the function and assign a value if they are `undefined`.

In the following example, if no value is provided for `b`, its value would be `undefined` when evaluating `a*b`, and a call to `multiply` would normally have returned `NaN`. However, this is prevented by the second line in this example:

```
function multiply(a, b) {
  b = typeof b !== "undefined" ? b : 1;
  return a * b;
}

console.log(multiply(5)); // 5
```

With *default parameters*, a manual check in the function body is no longer necessary. You can put `1` as the default value for `b` in the function head:

```
function multiply(a, b = 1) {
  return a * b;
}

console.log(multiply(5)); // 5
```

For more details, see default parameters in the reference.

Rest parameters

The rest parameter syntax allows us to represent an indefinite number of arguments as an array.

In the following example, the function `multiply` uses *rest parameters* to collect arguments from the second one to the end. The function then multiplies these by the first argument.

```
function multiply(multiplier, ...theArgs) {
  return theArgs.map((x) => multiplier * x);
}

const arr = multiply(2, 1, 2, 3);
console.log(arr); // [2, 4, 6]
```

Arrow functions

An arrow function expression (also called a *fat arrow* to distinguish from a hypothetical `->` syntax in future JavaScript) has a shorter syntax compared to function expressions and does not have its own `this`, `arguments`, `super`, or `new.target`. Arrow functions are always anonymous.

Two factors influenced the introduction of arrow functions: *shorter functions* and *non-binding* of `this`.

Shorter functions

In some functional patterns, shorter functions are welcome. Compare:

```
const a = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

const a2 = a.map(function (s) {
  return s.length;
});

console.log(a2); // [8, 6, 7, 9]

const a3 = a.map((s) => s.length);

console.log(a3); // [8, 6, 7, 9]
```

No separate this

Until arrow functions, every new function defined its own `this` value (a new object in the case of a constructor, undefined in strict mode function calls, the base object if the function is called as an "object method", etc.). This proved to be less than ideal with an object-oriented style of programming.

```
function Person() {
  // The Person() constructor defines `this` as itself.
  this.age = 0;

  setInterval(function growUp() {
    // In nonstrict mode, the growUp() function defines `this`
    // as the global object, which is different from the `this`
    // defined by the Person() constructor.
    this.age++;
  }, 1000);
}

const p = new Person();
```

In ECMAScript 3/5, this issue was fixed by assigning the value in `this` to a variable that could be closed over.

```
function Person() {
  // Some choose `that` instead of `self`.
  // Choose one and be consistent.
  const self = this;
  self.age = 0;

  setInterval(function growUp() {
    // The callback refers to the `self` variable of which
    // the value is the expected object.
    self.age++;
  }, 1000);
}
```

Alternatively, a bound function could be created so that the proper `this` value would be passed to the `growUp()` function.

An arrow function does not have its own `this`; the `this` value of the enclosing execution context is used. Thus, in the following code, the `this` within the function that is passed to `setInterval` has the same value as `this` in the enclosing function:

```
function Person() {
  this.age = 0;

  setInterval(() => {
    this.age++; // `this` properly refers to the person object
  }, 1000);
}

const p = new Person();
```

- Previous
- Next

Grammar and types

- Previous
- Next

This chapter discusses JavaScript's basic grammar, variable declarations, data types and literals.

Basics

JavaScript borrows most of its syntax from Java, C, and C++, but it has also been influenced by Awk, Perl, and Python.

JavaScript is **case-sensitive** and uses the **Unicode** character set. For example, the word Früh (which means "early" in German) could be used as a variable name.

```
const Früh = "foobar";
```

But, the variable `früh` is not the same as `Früh` because JavaScript is case sensitive.

In JavaScript, instructions are called statements and are separated by semicolons (;).

A semicolon is not necessary after a statement if it is written on its own line. But if more than one statement on a line is desired, then they *must* be separated by semicolons.

> **Note:** ECMAScript also has rules for automatic insertion of semicolons (ASI) to end statements. (For more information, see the detailed reference about JavaScript's lexical grammar.)

It is considered best practice, however, to always write a semicolon after a statement, even when it is not strictly needed. This practice reduces the chances of bugs getting into the code.

The source text of JavaScript script gets scanned from left to right, and is converted into a sequence of input elements which are *tokens*, *control characters*, *line terminators*, *comments*, or whitespace. (Spaces, tabs, and newline characters are considered whitespace.)

Comments

The syntax of **comments** is the same as in C++ and in many other languages:

```
// a one line comment

/* this is a longer,
 * multi-line comment
 */
```

You can't nest block comments. This often happens when you accidentally include a `*/` sequence in your comment, which will terminate the comment.

```
/* You can't, however, /* nest comments */ SyntaxError */
```

In this case, you need to break up the `*/` pattern. For example, by inserting a backslash:

```
/* You can /* nest comments *\/ by escaping slashes */
```

Comments behave like whitespace, and are discarded during script execution.

> **Note:** You might also see a third type of comment syntax at the start of some JavaScript files, which looks something like this: `#!/usr/bin/env node`.
>
> This is called **hashbang comment** syntax, and is a special comment used to specify the path to a particular JavaScript engine that should execute the script. See Hashbang comments for more details.

Declarations

JavaScript has five kinds of variable declarations.

`var`

: Declares a variable, optionally initializing it to a value.

`let`

: Declares a block-scoped variable, optionally initializing it to a value.

`const`

: Declares a block-scoped variable that cannot be re-assigned, which must be initialized at declaration.

`using`

: Declares a variable like `const` that is *synchronously disposed*.

`await using`

: Declares a variable like `const` that is *asynchronously disposed*.

We will only talk about the first three: `var`, `let`, and `const`, in this article. The `using` and `await using` declarations will be introduced in Resource management.

Variables

You use variables as symbolic names for values in your application. The names of variables, called identifiers, conform to certain rules.

A JavaScript identifier usually starts with a letter, underscore (_), or dollar sign ($). Subsequent characters can also be digits (`0` – `9`). Because JavaScript is case sensitive, letters include the characters `A` through `Z` (uppercase) as well as `a` through `z` (lowercase).

You can use most Unicode letters such as `å` and `ü` in identifiers. (For more details, see the lexical grammar reference.) You can also use Unicode escape sequences to represent characters in identifiers.

Some examples of legal names are `Number_hits`, `temp99`, `$credit`, and `_name`.

Declaring variables

You can declare a variable in two ways:

- With the keyword `var`. For example, `var x = 42`. This syntax can be used to declare both **local** and **global** variables, depending on the *execution context*.
- With the keyword `const` or `let`. For example, `let y = 13`. This syntax can be used to declare a block-scope local variable. (See Variable scope below.)

You can declare variables to unpack values using the destructuring syntax. For example, `const { bar } = foo`. This will create a variable named `bar` and assign to it the value corresponding to the key of the same name from our object `foo`.

Variables should always be declared before they are used. JavaScript used to allow assigning to undeclared variables, which creates an **undeclared global** variable. This is an error in strict mode and should be avoided altogether.

Declaration and initialization

In a statement like `let x = 42`, the `let x` part is called a *declaration*, and the `= 42` part is called an *initializer*. The declaration allows the variable to be accessed later in code without throwing a `ReferenceError`, while the initializer assigns a value to the variable. In `var` and `let` declarations, the initializer is optional. If a variable is declared without an initializer, it is assigned the value `undefined`.

```
let x;
console.log(x); // logs "undefined"
```

In essence, `let x = 42` is equivalent to `let x; x = 42`.

`const` declarations always need an initializer, because they forbid any kind of assignment after declaration, and implicitly initializing it with `undefined` is likely a programmer mistake.

```
const x; // SyntaxError: Missing initializer in const declaration
```

Variable scope

A variable may belong to one of the following scopes:

- Global scope: The default scope for all code running in script mode.
- Module scope: The scope for code running in module mode.
- Function scope: The scope created with a function.

In addition, variables declared with `let` or `const` can belong to an additional scope:

- Block scope: The scope created with a pair of curly braces (a block).

When you declare a variable outside of any function, it is called a *global* variable, because it is available to any other code in the current document. When you declare a variable within a function, it is called a *local* variable, because it is available only within that function.

`let` and `const` declarations can also be scoped to the block statement that they are declared in.

```
if (Math.random() > 0.5) {
  const y = 5;
}
console.log(y); // ReferenceError: y is not defined
```

However, variables created with `var` are not block-scoped, but only local to the *function (or global scope)* that the block resides within.

For example, the following code will log `5`, because the scope of `x` is the global context (or the function context if the code is part of a function). The scope of `x` is not limited to the immediate `if` statement block.

```
if (true) {
  var x = 5;
}
console.log(x); // x is 5
```

Variable hoisting

`var`-declared variables are hoisted, meaning you can refer to the variable anywhere in its scope, even if its declaration isn't reached yet. You can see `var` declarations as being "lifted" to the top of its function or global scope. However, if you access a variable before it's declared, the value is always `undefined`, because only its *declaration* and *default initialization (with `undefined`)* is hoisted, but not its *value assignment*.

```
console.log(x === undefined); // true
var x = 3;

(function () {
  console.log(x); // undefined
  var x = "local value";
})();
```

The above examples will be interpreted the same as:

```
var x;
console.log(x === undefined); // true
x = 3;

(function () {
  var x;
  console.log(x); // undefined
  x = "local value";
})();
```

Because of hoisting, all `var` statements in a function should be placed as near to the top of the function as possible. This best practice increases the clarity of the code.

Whether `let` and `const` are hoisted is a matter of definition debate. Referencing the variable in the block before the variable declaration always results in a `ReferenceError`, because the variable is in a "temporal dead zone" from the start of the block until the declaration is processed.

```
console.log(x); // ReferenceError
const x = 3;

console.log(y); // ReferenceError
let y = 3;
```

Unlike `var` declarations, which only hoist the declaration but not its value, function declarations are hoisted entirely — you can safely call the function anywhere in its scope. See the hoisting glossary entry for more discussion.

Global variables

Global variables are in fact properties of the *global object*.

In web pages, the global object is `window`, so you can read and set global variables using the `window.variable` syntax. In all environments, the `globalThis` variable (which itself is a global variable) may be used to read and set global variables. This is to provide a consistent interface among various JavaScript runtimes.

Consequently, you can access global variables declared in one window or frame from another window or frame by specifying the `window` or `frame` name. For example, if a variable called `phoneNumber` is declared in a document, you can refer to this variable from an `iframe` as `parent.phoneNumber`.

Constants

You can create a read-only, named constant with the `const` keyword. The syntax of a constant identifier is the same as any variable identifier: it must start with a letter, underscore, or dollar sign (`$`), and can contain alphabetic, numeric, or underscore characters.

```
const PI = 3.14;
```

A constant cannot change value through assignment or be re-declared while the script is running. It must be initialized to a value. The scope rules for constants are the same as those for `let` block-scope variables.

You cannot declare a constant with the same name as a function or variable in the same scope. For example:

```
// THIS WILL CAUSE AN ERROR
function f() {}
const f = 5;

// THIS WILL CAUSE AN ERROR TOO
function f() {
  const g = 5;
  var g;
}
```

However, `const` only prevents *re-assignments*, but doesn't prevent *mutations*. The properties of objects assigned to constants are not protected, so the following statement is executed without problems.

```
const MY_OBJECT = { key: "value" };
MY_OBJECT.key = "otherValue";
```

Also, the contents of an array are not protected, so the following statement is executed without problems.

```
const MY_ARRAY = ["HTML", "CSS"];
MY_ARRAY.push("JAVASCRIPT");
console.log(MY_ARRAY); // ['HTML', 'CSS', 'JAVASCRIPT'];
```

Data structures and types

Data types

The latest ECMAScript standard defines eight data types:

- Seven data types that are primitives:
    1. Boolean. `true` and `false`.
    2. null. A special keyword denoting a null value. (Because JavaScript is case-sensitive, `null` is not the same as `Null`, `NULL`, or any other variant.)
    3. undefined. A top-level property whose value is not defined.
    4. Number. An integer or floating point number. For example: `42` or `3.14159`.
    5. BigInt. An integer with arbitrary precision. For example: `900719925474099 2n`.
    6. String. A sequence of characters that represent a text value. For example: `"Howdy"`.
    7. Symbol. A data type whose instances are unique and immutable.
- and Object

Although these data types are relatively few, they enable you to perform useful operations with your applications. Functions are the other fundamental elements of the language. While functions are technically a kind of object, you can think of objects as named containers for values, and functions as procedures that your script can perform.

Data type conversion

JavaScript is a *dynamically typed* language. This means you don't have to specify the data type of a variable when you declare it. It also means that data types are automatically converted as-needed during script execution.

So, for example, you could define a variable as follows:

```
let answer = 42;
```

And later, you could assign the same variable a string value, for example:

```
answer = "Thanks for all the fish!";
```

Because JavaScript is dynamically typed, this assignment does not cause an error message.

Numbers and the '+' operator

In expressions involving numeric and string values with the + operator, JavaScript converts numeric values to strings. For example, consider the following statements:

```
x = "The answer is " + 42; // "The answer is 42"
y = 42 + " is the answer"; // "42 is the answer"
z = "37" + 7; // "377"
```

With all other operators, JavaScript does *not* convert numeric values to strings. For example:

```
"37" - 7; // 30
"37" * 7; // 259
```

Converting strings to numbers

In the case that a value representing a number is in memory as a string, there are methods for conversion.

- `parseInt()`
- `parseFloat()`
- `Number()`

`parseInt` only returns whole numbers, so its use is diminished for decimals.

> **Note:** Additionally, a best practice for `parseInt` is to always include the *radix* parameter. The radix parameter is used to specify which numerical system is to be used.

```
parseInt("101", 2); // 5
```

An alternative method of retrieving a number from a string is with the + (unary plus) operator. This implicitly performs number conversion, which is the same process as the `Number()` function.

```
"1.1" + "1.1"; // '1.11.1'
(+"1.1") + (+"1.1"); // 2.2
// Note: the parentheses are added for clarity, not required.
```

Literals

Literals represent values in JavaScript. These are fixed values—not variables—that you *literally* provide in your script. This section describes the following types of literals:

- Array literals
- Boolean literals
- Numeric literals
- Object literals
- RegExp literals
- String literals

Array literals

An array literal is a list of zero or more expressions, each of which represents an array element, enclosed in square brackets (`[]`). When you create an array using an array literal, it is initialized with the specified values as its elements, and its `length` is set to the number of arguments specified.

The following example creates the `coffees` array with three elements and a `length` of three:

```
const coffees = ["French Roast", "Colombian", "Kona"];
```

An array literal creates a new array object every time the literal is evaluated. For example, an array defined with a literal in the global scope is created once when the script loads. However, if the array literal is inside a function, a new array is instantiated every time that function is called.

> **Note:** Array literals create `Array` objects. See `Array` and Indexed collections for details on `Array` objects.

Extra commas in array literals

If you put two commas in a row in an array literal, the array leaves an empty slot for the unspecified element. The following example creates the `fish` array:

```
const fish = ["Lion", , "Angel"];
```

When you log this array, you will see:

```
console.log(fish);
// [ 'Lion', <1 empty item>, 'Angel' ]
```

Note that the second item is "empty", which is not exactly the same as the actual `undefined` value. When using array-traversing methods like `Array.prototype.map`, empty slots are skipped. However, index-accessing `fish[1]` still returns `undefined`.

If you include a trailing comma at the end of the list of elements, the comma is ignored.

In the following example, the `length` of the array is three. There is no `myList[3]` and `myList[1]` is empty. All other commas in the list indicate a new element.

```
const myList = ["home", , "school"];
```

In the following example, the `length` of the array is four, and `myList[0]` and `myList[2]` are missing.

```
const myList = [, "home", , "school"];
```

In the following example, the `length` of the array is four, and `myList[1]` and `myList[3]` are missing. **Only the last comma is ignored.**

```
const myList = ["home", , "school", ,];
```

> **Note:** Trailing commas help keep git diffs clean when you have a multi-line array, because appending an item to the end only adds one line, but does not modify the previous line.

```
const myList = [
  "home",
  "school",
+ "hospital",
];
```

Understanding the behavior of extra commas is important to understanding JavaScript as a language.

However, when writing your own code, you should explicitly declare the missing elements as `undefined`, or at least insert a comment to highlight its absence. Doing this increases your code's clarity and maintainability.

```
const myList = ["home", /* empty */, "school", /* empty */, ];
```

Boolean literals

The Boolean type has two literal values: `true` and `false`.

> **Note:** Do not confuse the primitive Boolean values `true` and `false` with the true and false values of the `Boolean` object.
>
> The Boolean object is a wrapper around the primitive Boolean data type. See `Boolean` for more information.

Numeric literals

JavaScript numeric literals include integer literals in different bases as well as floating-point literals in base-10.

Note that the language specification requires numeric literals to be unsigned. Nevertheless, code fragments like `-123.4` are fine, being interpreted as a unary `-` operator applied to the numeric literal `123.4`.

Integer literals

Integer and `BigInt` literals can be written in decimal (base 10), hexadecimal (base 16), octal (base 8) and binary (base 2).

- A *decimal* integer literal is a sequence of digits without a leading `0` (zero).
- A leading `0` (zero) on an integer literal, or a leading `0o` (or `0O`) indicates it is in *octal*. Octal integer literals can include only the digits `0` – `7`.
- A leading `0x` (or `0X`) indicates a *hexadecimal* integer literal. Hexadecimal integers can include digits (`0` – `9`) and the letters `a` – `f` and `A` – `F`. (The case of a character does not change its value. Therefore: `0xa` = `0xA` = `10` and `0xf` = `0xF` = `15`.)
- A leading `0b` (or `0B`) indicates a *binary* integer literal. Binary integer literals can only include the digits `0` and `1`.
- A trailing `n` suffix on an integer literal indicates a `BigInt` literal. The `BigInt` literal can use any of the above bases. Note that leading-zero octal syntax like `0123n` is not allowed, but `0o123n` is fine.

Some examples of integer literals are:

```
0, 117, 123456789123456789n            (decimal, base 10)
015, 0001, 0o777777777777n             (octal, base 8)
0x1123, 0x00111, 0x123456789ABCDEFn    (hexadecimal, "hex" or base 16)
0b11, 0b0011, 0b1110100101010101010101n   (binary, base 2)
```

For more information, see Numeric literals in the Lexical grammar reference.

Floating-point literals

A floating-point literal can have the following parts:

- An unsigned decimal integer,
- A decimal point (`.`),
- A fraction (another decimal number),
- An exponent.

The exponent part is an `e` or `E` followed by an integer, which can be signed (preceded by `+` or `-`). A floating-point literal must have at least one digit, and either a decimal point or `e` (or `E`).

More succinctly, the syntax is:

```
[digits].[digits][(E|e)[(+|-)]digits]
```

For example:

```
3.1415926
.123456789
3.1E+12
.1e-23
```

Object literals

An object literal is a list of zero or more pairs of property names and associated values of an object, enclosed in curly braces (`{}`).

> **Warning:** Do not use an object literal at the beginning of a statement! This will lead to an error (or not behave as you expect), because the `{` will be interpreted as the beginning of a block.

The following is an example of an object literal. The first element of the `car` object defines a property, `myCar`, and assigns to it a new string, `"Saturn"`; the second element, the `getCar` property, is immediately assigned the result of invoking the function `(carTypes("Honda"))`; the third element, the `special` property, uses an existing variable (`sales`).

```
const sales = "Toyota";

function carTypes(name) {
  return name === "Honda" ? name : `Sorry, we don't sell ${name}.`;
}

const car = { myCar: "Saturn", getCar: carTypes("Honda"), special: sales };

console.log(car.myCar); // Saturn
console.log(car.getCar); // Honda
console.log(car.special); // Toyota
```

Additionally, you can use a numeric or string literal for the name of a property or nest an object inside another. The following example uses these options.

```
const car = { manyCars: { a: "Saab", b: "Jeep" }, 7: "Mazda" };

console.log(car.manyCars.b); // Jeep
console.log(car[7]); // Mazda
```

Object property names can be any string, including the empty string. If the property name would not be a valid JavaScript identifier or number, it must be enclosed in quotes.

Property names that are not valid identifiers cannot be accessed as a dot (`.`) property.

```
const unusualPropertyNames = {
  "": "An empty string",
  "!": "Bang!",
};
console.log(unusualPropertyNames.""); // SyntaxError: Unexpected string
console.log(unusualPropertyNames.!); // SyntaxError: Unexpected token !
```

Instead, they must be accessed with the bracket notation (`[]`).

```
console.log(unusualPropertyNames[""]); // An empty string
console.log(unusualPropertyNames["!"]); // Bang!
```

Enhanced Object literals

Object literals support a range of shorthand syntaxes that include setting the prototype at construction, shorthand for `foo: foo` assignments, defining methods, making `super` calls, and computing property names with expressions.

Together, these also bring object literals and class declarations closer together, and allow object-based design to benefit from some of the same conveniences.

```
const theProtoObj = {};
const handler = {};
const obj = {
  // __proto__
  __proto__: theProtoObj,
  // Shorthand for 'handler: handler'
  handler,
  // Methods
  toString() {
    // Super calls
    return `d ${super.toString()}`;
  },
  // Computed (dynamic) property names
  ["prop_" + (() => 42)()]: 42,
};
```

RegExp literals

A regex literal (which is defined in detail later) is a pattern enclosed between slashes. The following is an example of a regex literal.

```
const re = /ab+c/;
```

String literals

A string literal is zero or more characters enclosed in double (`"`) or single (`'`) quotation marks. A string must be delimited by quotation marks of the same type (that is, either both single quotation marks, or both double quotation marks).

The following are examples of string literals:

```
'foo'
"bar"
'1234'
'one line \n another line'
"Joyo's cat"
```

You should use string literals unless you specifically need to use a `String` object. See `String` for details on `String` objects.

You can call any of the `String` object's methods on a string literal value. JavaScript automatically converts the string literal to a temporary String object, calls the method, then discards the temporary String object. You can also use the `length` property with a string literal:

```
// Will print the number of symbols in the string including whitespace.
console.log("Joyo's cat".length); // In this case, 10.
```

Template literals are also available. Template literals are enclosed by the back-tick (`` ` ``) (grave accent) character instead of double or single quotes.

Template literals provide syntactic sugar for constructing strings. (This is similar to string interpolation features in Perl, Python, and more.)

```
// Basic literal string creation
`In JavaScript '\n' is a line-feed.`;

// Multiline strings
`In JavaScript, template strings can run
 over multiple lines, but double and single
 quoted strings cannot.`;

// String interpolation
const name = "Lev",
  time = "today";
`Hello ${name}, how are you ${time}?`;
```

Tagged templates are a compact syntax for specifying a template literal along with a call to a "tag" function for parsing it. A tagged template is just a more succinct and semantic way to invoke a function that processes a string and a set of relevant values. The name of the template tag function precedes the template literal — as in the following example, where the template tag function is named `print`. The `print` function will interpolate the arguments and serialize any objects or arrays that may come up, avoiding the pesky `[object Object]`.

```
const formatArg = (arg) => {
  if (Array.isArray(arg)) {
    // Print a bulleted list
    return arg.map((part) => `- ${part}`).join("\n");
  }
  if (arg.toString === Object.prototype.toString) {
    // This object will be serialized to "[object Object]".
    // Let's print something nicer.
    return JSON.stringify(arg);
  }
  return arg;
};

const print = (segments, ...args) => {
  // For any well-formed template literal, there will always be N args and
  // (N+1) string segments.
  let message = segments[0];
  segments.slice(1).forEach((segment, index) => {
    message += formatArg(args[index]) + segment;
  });
  console.log(message);
};

const todos = [
  "Learn JavaScript",
  "Learn Web APIs",
  "Set up my website",
  "Profit!",
];

const progress = { javascript: 20, html: 50, css: 10 };

print`I need to do:
${todos}
My current progress is: ${progress}
`;

// I need to do:
// - Learn JavaScript
// - Learn Web APIs
// - Set up my website
// - Profit!
// My current progress is: {"javascript":20,"html":50,"css":10}
```

Since tagged template literals are just sugar of function calls, you can re-write the above as an equivalent function call:

```
print(["I need to do:\n", "\nMy current progress is: ", "\n"], todos, progress);
```

This may be reminiscent of the `console.log`-style interpolation:

```
console.log("I need to do:\n%o\nMy current progress is: %o\n", todos, progress);
```

You can see how the tagged template reads more naturally than a traditional "formatter" function, where the variables and the template itself have to be declared separately.

Using special characters in strings

In addition to ordinary characters, you can also include special characters in strings, as shown in the following example.

```
"one line \n another line";
```

The following table lists the special characters that you can use in JavaScript strings.

Character	Meaning
`\0`	Null Byte
`\b`	Backspace
`\f`	Form Feed
`\n`	New Line
`\r`	Carriage Return
`\t`	Tab
`\v`	Vertical tab
`\'`	Apostrophe or single quote
`\"`	Double quote
`\\`	Backslash character
`\XXX`	The character with the Latin-1 encoding specified by up to three octal digits `XXX` between `0` and `377`. For example, `\251` is the octal sequence for the copyright symbol.
`\xXX`	The character with the Latin-1 encoding specified by the two hexadecimal digits `XX` between `00` and `FF`. For example, `\xA9` is the hexadecimal sequence for the copyright symbol.
`\uXXXX`	The Unicode character specified by the four hexadecimal digits `XXXX`. For example, `\u00A9` is the Unicode sequence for the copyright symbol. See Unicode escape sequences.
`\u{XXXXX}`	Unicode code point escapes. For example, `\u{2F804}` is the same as the Unicode escapes `\uD87E\uDC04`.

Escaping characters

For characters not listed in the table, a preceding backslash is ignored, but this usage is deprecated and should be avoided.

You can insert a quotation mark inside a string by preceding it with a backslash. This is known as *escaping* the quotation mark. For example:

```
const quote = "He read \"The Cremation of Sam McGee\" by R.W. Service.";
console.log(quote);
```

The result of this would be:

```
He read "The Cremation of Sam McGee" by R.W. Service.
```

To include a literal backslash inside a string, you must escape the backslash character. For example, to assign the file path `c:\temp` to a string, use the following:

```
const home = "c:\\temp";
```

You can also escape line breaks by preceding them with backslash. The backslash and line break are both removed from the value of the string.

```
const str =
  "this string \
is broken \
across multiple \
lines.";
console.log(str); // this string is broken across multiple lines.
```

More information

This chapter focuses on basic syntax for declarations and types. To learn more about JavaScript's language constructs, see also the following chapters in this guide:

- Control flow and error handling guide
- Loops and iteration
- Functions
- Expressions and operators guide

In the next chapter, we will have a look at control flow constructs and error handling.

- Previous
- Next

Indexed collections

- Previous
- Next

This chapter introduces collections of data which are ordered by an index value. This includes arrays and array-like constructs such as `Array` objects and `TypedArray` objects.

An *array* is an ordered list of values that you refer to with a name and an index.

For example, consider an array called `emp`, which contains employees' names indexed by their numerical employee number. So `emp[0]` would be employee number zero, `emp[1]` employee number one, and so on.

JavaScript does not have an explicit array data type. However, you can use the predefined `Array` object and its methods to work with arrays in your applications. The `Array` object has methods for manipulating arrays in various ways, such as joining, reversing, and sorting them. It has a property for determining the array length and other properties for use with regular expressions.

We will be focusing on arrays in this article, but many of the same concepts apply to typed arrays as well, since arrays and typed arrays share many similar methods. For more information on typed arrays, see the typed array guide.

Creating an array

The following statements create equivalent arrays:

```
const arr1 = new Array(element0, element1, /* …, */ elementN);
const arr2 = Array(element0, element1, /* …, */ elementN);
const arr3 = [element0, element1, /* …, */ elementN];
```

`element0, element1, …, elementN` is a list of values for the array's elements. When these values are specified, the array is initialized with them as the array's elements. The array's `length` property is set to the number of arguments.

The bracket syntax is called an "array literal" or "array initializer." It's shorter than other forms of array creation, and so is generally preferred. See Array literals for details.

To create an array with non-zero length, but without any items, either of the following can be used:

```
// This...
const arr1 = new Array(arrayLength);

// … results in the same array as this
const arr2 = Array(arrayLength);

// This has exactly the same effect
const arr3 = [];
arr3.length = arrayLength;
```

> **Note:** In the above code, `arrayLength` must be a `Number`. Otherwise, an array with a single element (the provided value) will be created. Calling `arr.length` will return `arrayLength`, but the array doesn't contain any elements. A `for...in` loop will not find any property on the array.

In addition to a newly defined variable as shown above, arrays can also be assigned as a property of a new or an existing object:

```
const obj = {};
// …
obj.prop = [element0, element1, /* …, */ elementN];

// OR
const obj = { prop: [element0, element1, /* …, */ elementN] };
```

If you wish to initialize an array with a single element, and the element happens to be a `Number`, you must use the bracket syntax. When a single `Number` value is passed to the `Array()` constructor or function, it is interpreted as an `arrayLength`, not as a single element.

This creates an array with only one element: the number 42.

```
const arr = [42];
```

This creates an array with no elements and `arr.length` set to 42.

```
const arr = Array(42);
```

This is equivalent to:

```
const arr = [];
arr.length = 42;
```

Calling `Array(N)` results in a `RangeError`, if `N` is a non-whole number whose fractional portion is non-zero. The following example illustrates this behavior.

```
const arr = Array(9.3); // RangeError: Invalid array length
```

If your code needs to create arrays with single elements of an arbitrary data type, it is safer to use array literals. Alternatively, create an empty array first before adding the single element to it.

You can also use the `Array.of` static method to create arrays with single element.

```
const arr = Array.of(9.3); // arr contains only one element 9.3
```

Referring to array elements

Because elements are also properties, you can access them using property accessors. Suppose you define the following array:

```
const myArray = ["Wind", "Rain", "Fire"];
```

You can refer to the first element of the array as `myArray[0]`, the second element of the array as `myArray[1]`, etc. The index of the elements begins with zero.

> **Note:** You can also use property accessors to access other properties of the array, like with an object.
>
> ```
> const arr = ["one", "two", "three"];
> arr[2]; // three
> arr["length"]; // 3
> ```

Populating an array

You can populate an array by assigning values to its elements. For example:

```
const emp = [];
emp[0] = "Casey Jones";
emp[1] = "Phil Lesh";
emp[2] = "August West";
```

> **Note:** If you supply a non-integer value to the array operator in the code above, a property will be created in the object representing the array, instead of an array element.
>
> ```
> const arr = [];
> arr[3.4] = "Oranges";
> console.log(arr.length); // 0
> console.log(Object.hasOwn(arr, 3.4)); // true
> ```

You can also populate an array when you create it:

```
const myArray = new Array("Hello", myVar, 3.14159);
// OR
const myArray = ["Mango", "Apple", "Orange"];
```

Understanding length

At the implementation level, JavaScript's arrays actually store their elements as standard object properties, using the array index as the property name.

The `length` property is special. Its value is always a positive integer greater than the index of the last element if one exists. (In the example below, `'Dusty'` is indexed at `30`, so `cats.length` returns `30 + 1`).

Remember, JavaScript Array indexes are 0-based: they start at `0`, not `1`. This means that the `length` property will be one more than the highest index stored in the array:

```
const cats = [];
cats[30] = ["Dusty"];
console.log(cats.length); // 31
```

You can also assign to the `length` property.

Writing a value that is shorter than the number of stored items truncates the array. Writing `0` empties it entirely:

```
const cats = ["Dusty", "Misty", "Twiggy"];
console.log(cats.length); // 3

cats.length = 2;
console.log(cats); // [ 'Dusty', 'Misty' ] - Twiggy has been removed

cats.length = 0;
console.log(cats); // []; the cats array is empty

cats.length = 3;
console.log(cats); // [ <3 empty items> ]
```

Iterating over arrays

A common operation is to iterate over the values of an array, processing each one in some way, as follows:

```
const colors = ["red", "green", "blue"];
for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}
```

If you know that none of the elements in your array evaluate to `false` in a boolean context—if your array consists only of DOM nodes, for example—you can use a more efficient idiom:

```
const divs = document.getElementsByTagName("div");
for (let i = 0, div; (div = divs[i]); i++) {
  /* Process div in some way */
}
```

This avoids the overhead of checking the length of the array, and ensures that the `div` variable is reassigned to the current item each time around the loop for added convenience.

The `forEach()` method provides another way of iterating over an array:

```
const colors = ["red", "green", "blue"];
colors.forEach((color) => console.log(color));
// red
// green
// blue
```

The function passed to `forEach` is executed once for every item in the array, with the array item passed as the argument to the function. Unassigned values are not iterated in a `forEach` loop.

Note that the elements of an array that are omitted when the array is defined are not listed when iterating by `forEach`, but *are* listed when `undefined` has been manually assigned to the element:

```
const sparseArray = ["first", "second", , "fourth"];

sparseArray.forEach((element) => {
  console.log(element);
});
// Logs:
// first
// second
// fourth

if (sparseArray[2] === undefined) {
  console.log("sparseArray[2] is undefined"); // true
}

const nonsparseArray = ["first", "second", undefined, "fourth"];

nonsparseArray.forEach((element) => {
  console.log(element);
});
// Logs:
// first
// second
// undefined
// fourth
```

Since JavaScript array elements are saved as standard object properties, it is not advisable to iterate through JavaScript arrays using `for...in` loops, because normal elements and all enumerable properties will be listed.

Array methods

The `Array` object has the following methods:

The `concat()` method joins two or more arrays and returns a new array.

```
let myArray = ["1", "2", "3"];
myArray = myArray.concat("a", "b", "c");
// myArray is now ["1", "2", "3", "a", "b", "c"]
```

The `join()` method joins all elements of an array into a string.

```
const myArray = ["Wind", "Rain", "Fire"];
const list = myArray.join(" - "); // list is "Wind - Rain - Fire"
```

The `push()` method adds one or more elements to the end of an array and returns the resulting `length` of the array.

```
const myArray = ["1", "2"];
myArray.push("3"); // myArray is now ["1", "2", "3"]
```

The `pop()` method removes the last element from an array and returns that element.

```
const myArray = ["1", "2", "3"];
const last = myArray.pop();
// myArray is now ["1", "2"], last = "3"
```

The `shift()` method removes the first element from an array and returns that element.

```
const myArray = ["1", "2", "3"];
const first = myArray.shift();
// myArray is now ["2", "3"], first is "1"
```

The `unshift()` method adds one or more elements to the front of an array and returns the new length of the array.

```
const myArray = ["1", "2", "3"];
myArray.unshift("4", "5");
// myArray becomes ["4", "5", "1", "2", "3"]
```

The `slice()` method extracts a section of an array and returns a new array.

```
let myArray = ["a", "b", "c", "d", "e"];
myArray = myArray.slice(1, 4); // [ "b", "c", "d"]
// starts at index 1 and extracts all elements
// until index 3
```

The `at()` method returns the element at the specified index in the array, or `undefined` if the index is out of range. It's notably used for negative indices that access elements from the end of the array.

```
const myArray = ["a", "b", "c", "d", "e"];
myArray.at(-2); // "d", the second-last element of myArray
```

The `splice()` method removes elements from an array and (optionally) replaces them. It returns the items which were removed from the array.

```
const myArray = ["1", "2", "3", "4", "5"];
myArray.splice(1, 3, "a", "b", "c", "d");
// myArray is now ["1", "a", "b", "c", "d", "5"]
// This code started at index one (or where the "2" was),
// removed 3 elements there, and then inserted all consecutive
// elements in its place.
```

The `reverse()` method transposes the elements of an array, in place: the first array element becomes the last and the last becomes the first. It returns a reference to the array.

```
const myArray = ["1", "2", "3"];
myArray.reverse();
// transposes the array so that myArray = ["3", "2", "1"]
```

The `flat()` method returns a new array with all sub-array elements concatenated into it recursively up to the specified depth.

```
let myArray = [1, 2, [3, 4]];
myArray = myArray.flat();
// myArray is now [1, 2, 3, 4], since the [3, 4] subarray is flattened
```

The `sort()` method sorts the elements of an array in place, and returns a reference to the array.

```
const myArray = ["Wind", "Rain", "Fire"];
myArray.sort();
// sorts the array so that myArray = ["Fire", "Rain", "Wind"]
```

`sort()` can also take a callback function to determine how array elements are compared. The callback function is called with two arguments, which are two values from the array. The function compares these two values and returns a positive number, negative number, or zero, indicating the order of the two values. For instance, the following will sort the array by the last letter of a string:

```
const sortFn = (a, b) => {
  if (a[a.length - 1] < b[b.length - 1]) {
    return -1; // Negative number => a < b, a comes before b
  } else if (a[a.length - 1] > b[b.length - 1]) {
    return 1; // Positive number => a > b, a comes after b
  }
  return 0; // Zero => a = b, a and b keep their original order
};
myArray.sort(sortFn);
// sorts the array so that myArray = ["Wind","Fire","Rain"]
```

- if `a` is less than `b` by the sorting system, return `-1` (or any negative number)
- if `a` is greater than `b` by the sorting system, return `1` (or any positive number)
- if `a` and `b` are considered equivalent, return `0`.

The `indexOf()` method searches the array for `searchElement` and returns the index of the first match.

```
const a = ["a", "b", "a", "b", "a"];
console.log(a.indexOf("b")); // 1

// Now try again, starting from after the last match
console.log(a.indexOf("b", 2)); // 3
console.log(a.indexOf("z")); // -1, because 'z' was not found
```

The `lastIndexOf()` method works like `indexOf`, but starts at the end and searches backwards.

```
const a = ["a", "b", "c", "d", "a", "b"];
console.log(a.lastIndexOf("b")); // 5

// Now try again, starting from before the last match
console.log(a.lastIndexOf("b", 4)); // 1
console.log(a.lastIndexOf("z")); // -1
```

The `forEach()` method executes `callback` on every array item and returns `undefined`.

```
const a = ["a", "b", "c"];
a.forEach((element) => {
  console.log(element);
});
// Logs:
// a
// b
// c
```

The `forEach` method (and others below) that take a callback are known as *iterative methods*, because they iterate over the entire array in some fashion. Each one takes an optional second argument called `thisArg`. If provided, `thisArg` becomes the value of the `this` keyword inside the body of the callback function. If not provided, as with other cases where a function is invoked outside of an explicit object context, `this` will refer to the global object (`window`, `globalThis`, etc.) when the function is not strict, or `undefined` when the function is strict.

> **Note:** The `sort()` method introduced above is not an iterative method, because its callback function is only used for comparison and may not be called in any particular order based on element order. `sort()` does not accept the `thisArg` parameter either.

The `map()` method returns a new array of the return value from executing `callback` on every array item.

```
const a1 = ["a", "b", "c"];
const a2 = a1.map((item) => item.toUpperCase());
console.log(a2); // ['A', 'B', 'C']
```

The `flatMap()` method runs `map()` followed by a `flat()` of depth 1.

```
const a1 = ["a", "b", "c"];
const a2 = a1.flatMap((item) => [item.toUpperCase(), item.toLowerCase()]);
console.log(a2); // ['A', 'a', 'B', 'b', 'C', 'c']
```

The `filter()` method returns a new array containing the items for which `callback` returned `true`.

```
const a1 = ["a", 10, "b", 20, "c", 30];
const a2 = a1.filter((item) => typeof item === "number");
console.log(a2); // [10, 20, 30]
```

The `find()` method returns the first item for which `callback` returned `true`.

```
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.find((item) => typeof item === "number");
console.log(i); // 10
```

The `findLast()` method returns the last item for which `callback` returned `true`.

```
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.findLast((item) => typeof item === "number");
console.log(i); // 30
```

The `findIndex()` method returns the index of the first item for which `callback` returned `true`.

```
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.findIndex((item) => typeof item === "number");
console.log(i); // 1
```

The `findLastIndex()` method returns the index of the last item for which `callback` returned `true`.

```
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.findLastIndex((item) => typeof item === "number");
console.log(i); // 5
```

The `every()` method returns `true` if `callback` returns `true` for every item in the array.

```
function isNumber(value) {
  return typeof value === "number";
}
const a1 = [1, 2, 3];
console.log(a1.every(isNumber)); // true
const a2 = [1, "2", 3];
console.log(a2.every(isNumber)); // false
```

The `some()` method returns `true` if `callback` returns `true` for at least one item in the array.

```
function isNumber(value) {
  return typeof value === "number";
}
const a1 = [1, 2, 3];
console.log(a1.some(isNumber)); // true
const a2 = [1, "2", 3];
console.log(a2.some(isNumber)); // true
const a3 = ["1", "2", "3"];
console.log(a3.some(isNumber)); // false
```

The `reduce()` method applies `callback(accumulator, currentValue, currentIndex, array)` for each value in the array for the purpose of reducing the list of items down to a single value. The `reduce` function returns the final value returned by `callback` function.

If `initialValue` is specified, then `callback` is called with `initialValue` as the first parameter value and the value of the first item in the array as the second parameter value.

If `initialValue` is *not* specified, then `callback`'s first two parameter values will be the first and second elements of the array. On *every* subsequent call, the first parameter's value will be whatever `callback` returned on the previous call, and the second parameter's value will be the next value in the array.

If `callback` needs access to the index of the item being processed, or access to the entire array, they are available as optional parameters.

```
const a = [10, 20, 30];
const total = a.reduce(
  (accumulator, currentValue) => accumulator + currentValue,
  0,
);
console.log(total); // 60
```

The `reduceRight()` method works like `reduce()`, but starts with the last element.

`reduce` and `reduceRight` are the least obvious of the iterative array methods. They should be used for algorithms that combine two values recursively in order to reduce a sequence down to a single value.

Array transformations

You can transform back and forth between arrays and other data structures.

Grouping the elements of an array

The `Object.groupBy()` method can be used to group the elements of an array, using a test function that returns a string indicating the group of the current element.

Here we have an inventory array that contains "food" objects that have a `name` and a `type`.

```
const inventory = [
  { name: "asparagus", type: "vegetables" },
  { name: "bananas", type: "fruit" },
  { name: "goat", type: "meat" },
  { name: "cherries", type: "fruit" },
  { name: "fish", type: "meat" },
];
```

To use `Object.groupBy()`, you supply a callback function that is called with the current element, and optionally the current index and array, and returns a string indicating the group of the element.

The code below uses an arrow function to return the `type` of each array element (this uses object destructuring syntax for function arguments to unpack the `type` element from the passed object). The result is an object that has properties named after the unique strings returned by the callback. Each property is assigned an array containing the elements in the group.

```
const result = Object.groupBy(inventory, ({ type }) => type);
console.log(result);
// Logs
// {
//   vegetables: [{ name: 'asparagus', type: 'vegetables' }],
//   fruit: [
//     { name: 'bananas', type: 'fruit' },
//     { name: 'cherries', type: 'fruit' }
//   ],
//   meat: [
//     { name: 'goat', type: 'meat' },
//     { name: 'fish', type: 'meat' }
//   ]
// }
```

Note that the returned object references the *same* elements as the original array (not deep copies). Changing the internal structure of these elements will be reflected in both the original array and the returned object.

If you can't use a string as the key, for example, if the information to group is associated with an object that might change, then you can instead use `Map.groupBy()`. This is very similar to `Object.groupBy()` except that it groups the elements of the array into a `Map` that can use an arbitrary value (object or primitive) as a key.

Sparse arrays

Arrays can contain "empty slots", which are not the same as slots filled with the value `undefined`. Empty slots can be created in one of the following ways:

```
// Array constructor:
const a = Array(5); // [ <5 empty items> ]

// Consecutive commas in array literal:
const b = [1, 2, , , 5]; // [ 1, 2, <2 empty items>, 5 ]

// Directly setting a slot with index greater than array.length:
const c = [1, 2];
c[4] = 5; // [ 1, 2, <2 empty items>, 5 ]

// Elongating an array by directly setting .length:
const d = [1, 2];
d.length = 5; // [ 1, 2, <3 empty items> ]

// Deleting an element:
const e = [1, 2, 3, 4, 5];
delete e[2]; // [ 1, 2, <1 empty item>, 4, 5 ]
```

In some operations, empty slots behave as if they are filled with `undefined`.

```
const arr = [1, 2, , , 5]; // Create a sparse array

// Indexed access
console.log(arr[2]); // undefined

// For...of
for (const i of arr) {
  console.log(i);
}
// Logs: 1 2 undefined undefined 5

// Spreading
const another = [...arr]; // "another" is [ 1, 2, undefined, undefined, 5 ]
```

But in others (most notably array iteration methods), empty slots are skipped.

```
const mapped = arr.map((i) => i + 1); // [ 2, 3, <2 empty items>, 6 ]
arr.forEach((i) => console.log(i)); // 1 2 5
const filtered = arr.filter(() => true); // [ 1, 2, 5 ]
const hasFalsy = arr.some((k) => !k); // false

// Property enumeration
const keys = Object.keys(arr); // [ '0', '1', '4' ]
for (const key in arr) {
  console.log(key);
}
// Logs: '0' '1' '4'
// Spreading into an object uses property enumeration, not the array's iterator
const objectSpread = { ...arr }; // { '0': 1, '1': 2, '4': 5 }
```

For a complete list of how array methods behave with sparse arrays, see the `Array` reference page.

Multi-dimensional arrays

Arrays can be nested, meaning that an array can contain another array as an element. Using this characteristic of JavaScript arrays, multi-dimensional arrays can be created.

The following code creates a two-dimensional array.

```
const a = new Array(4);
for (let i = 0; i < 4; i++) {
  a[i] = new Array(4);
  for (let j = 0; j < 4; j++) {
    a[i][j] = `[${i}, ${j}]`;
  }
}
```

This example creates an array with the following rows:

```
Row 0: [0, 0] [0, 1] [0, 2] [0, 3]
Row 1: [1, 0] [1, 1] [1, 2] [1, 3]
Row 2: [2, 0] [2, 1] [2, 2] [2, 3]
Row 3: [3, 0] [3, 1] [3, 2] [3, 3]
```

Using arrays to store other properties

Arrays can also be used like objects, to store related information.

```
const arr = [1, 2, 3];
arr.property = "value";
console.log(arr.property); // "value"
```

For example, when an array is the result of a match between a regular expression and a string, the array returns properties and elements that provide information about the match. An array is the return value of `RegExp.prototype.exec()`, `String.prototype.match()`, and `String.prototype.split()`. For information on using arrays with regular expressions, see Regular Expressions.

Working with array-like objects

Some JavaScript objects, such as the `NodeList` returned by `document.getElementsByTagName()` or the `arguments` object made available within the body of a function, look and behave like arrays on the surface but do not share all of their methods. The `arguments` object provides a `length` attribute but does not implement array methods like `forEach()`.

Array methods cannot be called directly on array-like objects.

```
function printArguments() {
  arguments.forEach((item) => {
    console.log(item);
  }); // TypeError: arguments.forEach is not a function
}
```

But you can call them indirectly using `Function.prototype.call()`.

```
function printArguments() {
  Array.prototype.forEach.call(arguments, (item) => {
    console.log(item);
  });
}
```

Array prototype methods can be used on strings as well, since they provide sequential access to their characters in a similar way to arrays:

```
Array.prototype.forEach.call("a string", (chr) => {
  console.log(chr);
});
```


- Previous
- Next

Inheritance and the prototype chain

In programming, *inheritance* refers to passing down characteristics from a parent to a child so that a new piece of code can reuse and build upon the features of an existing one. JavaScript implements inheritance by using objects. Each object has an internal link to another object called its *prototype*. That prototype object has a prototype of its own, and so on until an object is reached with `null` as its prototype. By definition, `null` has no prototype and acts as the final link in this **prototype chain**. It is possible to mutate any member of the prototype chain or even swap out the prototype at runtime, so concepts like static dispatching do not exist in JavaScript.

JavaScript is a bit confusing for developers experienced in class-based languages (like Java or C++), as it is dynamic and does not have static types. While this confusion is often considered to be one of JavaScript's weaknesses, the prototypal inheritance model itself is, in fact, more powerful than the classic model. It is, for example, fairly trivial to build a classic model on top of a prototypal model — which is how classes are implemented.

Although classes are now widely adopted and have become a new paradigm in JavaScript, classes do not bring a new inheritance pattern. While classes abstract most of the prototypal mechanism away, understanding how prototypes work under the hood is still useful.

Inheritance with the prototype chain

Inheriting properties

JavaScript objects are dynamic "bags" of properties (referred to as **own properties**). JavaScript objects have a link to a prototype object. When trying to access a property of an object, the property will not only be sought on the object, but also on the prototype of the object, the prototype of the prototype, and so on, until either a property with a matching name is found or the end of the prototype chain is reached.

> **Note:** Following the ECMAScript standard, the notation `someObject.[[Prototype]]` is used to designate the prototype of `someObject`. The `[[Prototype]]` internal slot can be accessed and modified with the `Object.getPrototypeOf()` and `Object.setPrototypeOf()` functions respectively. This is equivalent to the JavaScript accessor `__proto__` which is non-standard but de-facto implemented by many JavaScript engines. To prevent confusion while keeping it succinct, in our notation we will avoid using `obj.__proto__` but use `obj.[[Prototype]]` instead. This corresponds to `Object.getPrototypeOf(obj)`.
>
> It should not be confused with the `func.prototype` property of functions, which instead specifies the `[[Prototype]]` to be assigned to all *instances* of objects created by the given function when used as a constructor. We will discuss the `prototype` property of constructor functions in a later section.

There are several ways to specify the `[[Prototype]]` of an object, which are listed in a later section. For now, we will use the `__proto__` syntax for illustration. It's worth noting that the `{ __proto__: ... }` syntax is different from the `obj.__proto__` accessor: the former is standard and not deprecated.

In an object literal like `{ a: 1, b: 2, __proto__: c }`, the value `c` (which has to be either `null` or another object) will become the `[[Prototype]]` of the object represented by the literal, while the other keys like `a` and `b` will become the *own properties* of the object. This syntax reads very naturally, since `[[Prototype]]` is just an "internal property" of the object.

Here is what happens when trying to access a property:

```
const o = {
  a: 1,
  b: 2,
  // __proto__ sets the [[Prototype]]. It's specified here
  // as another object literal.
  __proto__: {
    b: 3,
    c: 4,
  },
};

// o.[[Prototype]] has properties b and c.
// o.[[Prototype]].[[Prototype]] is Object.prototype (we will explain
// what that means later).
// Finally, o.[[Prototype]].[[Prototype]].[[Prototype]] is null.
// This is the end of the prototype chain, as null,
// by definition, has no [[Prototype]].
// Thus, the full prototype chain looks like:
// { a: 1, b: 2 } ---> { b: 3, c: 4 } ---> Object.prototype ---> null

console.log(o.a); // 1
// Is there an 'a' own property on o? Yes, and its value is 1.

console.log(o.b); // 2
// Is there a 'b' own property on o? Yes, and its value is 2.
// The prototype also has a 'b' property, but it's not visited.
// This is called Property Shadowing

console.log(o.c); // 4
// Is there a 'c' own property on o? No, check its prototype.
// Is there a 'c' own property on o.[[Prototype]]? Yes, its value is 4.

console.log(o.d); // undefined
// Is there a 'd' own property on o? No, check its prototype.
// Is there a 'd' own property on o.[[Prototype]]? No, check its prototype.
// o.[[Prototype]].[[Prototype]] is Object.prototype and
// there is no 'd' property by default, check its prototype.
// o.[[Prototype]].[[Prototype]].[[Prototype]] is null, stop searching,
// no property found, return undefined.
```

Setting a property to an object creates an own property. The only exception to the getting and setting behavior rules is when it's intercepted by a getter or setter.

Similarly, you can create longer prototype chains, and a property will be sought on all of them.

```
const o = {
  a: 1,
  b: 2,
  // __proto__ sets the [[Prototype]]. It's specified here
  // as another object literal.
  __proto__: {
    b: 3,
    c: 4,
    __proto__: {
      d: 5,
    },
  },
};

// { a: 1, b: 2 } ---> { b: 3, c: 4 } ---> { d: 5 } ---> Object.prototype ---> null

console.log(o.d); // 5
```

Inheriting "methods"

JavaScript does not have "methods" in the form that class-based languages define them. In JavaScript, any function can be added to an object in the form of a property. An inherited function acts just as any other property, including property shadowing as shown above (in this case, a form of *method overriding*).

When an inherited function is executed, the value of `this` points to the inheriting object, not to the prototype object where the function is an own property.

```
const parent = {
  value: 2,
  method() {
    return this.value + 1;
  },
};

console.log(parent.method()); // 3
// When calling parent.method in this case, 'this' refers to parent
```

```
// child is an object that inherits from parent
const child = {
  __proto__: parent,
};
console.log(child.method()); // 3
// When child.method is called, 'this' refers to child.
// So when child inherits the method of parent,
// The property 'value' is sought on child. However, since child
// doesn't have an own property called 'value', the property is
// found on the [[Prototype]], which is parent.value.

child.value = 4; // assign the value 4 to the property 'value' on child.
// This shadows the 'value' property on parent.
// The child object now looks like:
// { value: 4, __proto__: { value: 2, method: [Function] } }
console.log(child.method()); // 5
// Since child now has the 'value' property, 'this.value' means
// child.value instead
```

Constructors

The power of prototypes is that we can reuse a set of properties if they should be present on every instance — especially for methods. Suppose we are to create a series of boxes, where each box is an object that contains a value which can be accessed through a `getValue` function. A naive implementation would be:

```
const boxes = [
  { value: 1, getValue() { return this.value; } },
  { value: 2, getValue() { return this.value; } },
  { value: 3, getValue() { return this.value; } },
];
```

This is subpar, because each instance has its own function property that does the same thing, which is redundant and unnecessary. Instead, we can move `getValue` to the `[[Prototype]]` of all boxes:

```
const boxPrototype = {
  getValue() {
    return this.value;
  },
};

const boxes = [
  { value: 1, __proto__: boxPrototype },
  { value: 2, __proto__: boxPrototype },
  { value: 3, __proto__: boxPrototype },
];
```

This way, all boxes' `getValue` method will refer to the same function, lowering memory usage. However, manually binding the `__proto__` for every object creation is still very inconvenient. This is when we would use a *constructor* function, which automatically sets the `[[Prototype]]` for every object manufactured. Constructors are functions called with `new`.

```
// A constructor function
function Box(value) {
  this.value = value;
}

// Properties all boxes created from the Box() constructor
// will have
Box.prototype.getValue = function () {
  return this.value;
};

const boxes = [new Box(1), new Box(2), new Box(3)];
```

We say that `new Box(1)` is an *instance* created from the `Box` constructor function. `Box.prototype` is not much different from the `boxPrototype` object we created previously — it's just a plain object. Every instance created from a constructor function will automatically have the constructor's `prototype` property as its `[[Prototype]]` — that is, `Object.getPrototypeOf(new Box()) === Box.prototype`. `Constructor.prototype` by default has one own property: `constructor`, which references the constructor function itself — that is, `Box.prototype.constructor === Box`. This allows one to access the original constructor from any instance.

> **Note:** If a non-primitive is returned from the constructor function, that value will become the result of the `new` expression. In this case the `[[Prototype]]` may not be correctly bound — but this should not happen much in practice.

The above constructor function can be rewritten in classes as:

```
class Box {
  constructor(value) {
    this.value = value;
  }

  // Methods are created on Box.prototype
  getValue() {
    return this.value;
```

```
  }
}
```

Classes are syntax sugar over constructor functions, which means you can still manipulate `Box.prototype` to change the behavior of all instances. However, because classes are designed to be an abstraction over the underlying prototype mechanism, we will use the more-lightweight constructor function syntax for this tutorial to fully demonstrate how prototypes work.

Because `Box.prototype` references the same object as the `[[Prototype]]` of all instances, we can change the behavior of all instances by mutating `Box.prototype`.

```
function Box(value) {
  this.value = value;
}
Box.prototype.getValue = function () {
  return this.value;
};
const box = new Box(1);

// Mutate Box.prototype after an instance has already been created
Box.prototype.getValue = function () {
  return this.value + 1;
};
box.getValue(); // 2
```

A corollary is, *re-assigning* `Constructor.prototype` (`Constructor.prototype = ...`) is a bad idea for two reasons:

- The `[[Prototype]]` of instances created before the reassignment is now referencing a different object from the `[[Prototype]]` of instances created after the reassignment — mutating one's `[[Prototype]]` no longer mutates the other.
- Unless you manually re-set the `constructor` property, the constructor function can no longer be traced from `instance.constructor`, which may break user expectation. Some built-in operations will read the `constructor` property as well, and if it is not set, they may not work as expected.

`Constructor.prototype` is only useful when constructing instances. It has nothing to do with `Constructor.[[Prototype]]`, which is the constructor function's *own* prototype, which is `Function.prototype` — that is, `Object.getPrototypeOf(Constructor) === Function.prototype`.

Implicit constructors of literals

Some literal syntaxes in JavaScript create instances that implicitly set the `[[Prototype]]`. For example:

```
// Object literals (without the `__proto__` key) automatically
// have `Object.prototype` as their `[[Prototype]]`
const object = { a: 1 };
Object.getPrototypeOf(object) === Object.prototype; // true

// Array literals automatically have `Array.prototype` as their `[[Prototype]]`
const array = [1, 2, 3];
Object.getPrototypeOf(array) === Array.prototype; // true

// RegExp literals automatically have `RegExp.prototype` as their `[[Prototype]]`
const regexp = /abc/;
Object.getPrototypeOf(regexp) === RegExp.prototype; // true
```

We can "de-sugar" them into their constructor form.

```
const array = new Array(1, 2, 3);
const regexp = new RegExp("abc");
```

For example, "array methods" like `map()` are simply methods defined on `Array.prototype`, which is why they are automatically available on all array instances.

> **Warning:** There is one misfeature that used to be prevalent — extending `Object.prototype` or one of the other built-in prototypes. An example of this misfeature is, defining `Array.prototype.myMethod = function () {...}` and then using `myMethod` on all array instances.
>
> This misfeature is called *monkey patching*. Doing monkey patching risks forward compatibility, because if the language adds this method in the future but with a different signature, your code will break. It has led to incidents like the SmooshGate, and can be a great nuisance for the language to advance since JavaScript tries to "not break the web".
>
> The **only** good reason for extending a built-in prototype is to backport the features of newer JavaScript engines, like `Array.prototype.forEach`.

It may be interesting to note that due to historical reasons, some built-in constructors' `prototype` property are instances themselves. For example, `Number.prototype` is a number 0, `Array.prototype` is an empty array, and `RegExp.prototype` is `/(?:)/`.

```
Number.prototype + 1; // 1
Array.prototype.map((x) => x + 1); // []
String.prototype + "a"; // "a"
RegExp.prototype.source; // "(?:)"
Function.prototype(); // Function.prototype is a no-op function by itself
```

However, this is not the case for user-defined constructors, nor for modern constructors like `Map`.

```
Map.prototype.get(1);
// Uncaught TypeError: get method called on incompatible Map.prototype
```

Building longer inheritance chains

The `Constructor.prototype` property will become the `[[Prototype]]` of the constructor's instances, as-is — including `Constructor.prototype`'s own `[[Prototype]]`. By default, `Constructor.prototype` is a *plain object* — that is, `Object.getPrototypeOf(Constructor.prototype) === Object.prototype`. The only exception is `Object.prototype` itself, whose `[[Prototype]]` is `null` — that is, `Object.getPrototypeOf(Object.prototype) === null`. Therefore, a typical constructor will build the following prototype chain:

```
function Constructor() {}

const obj = new Constructor();
// obj ---> Constructor.prototype ---> Object.prototype ---> null
```

To build longer prototype chains, we can set the `[[Prototype]]` of `Constructor.prototype` via the `Object.setPrototypeOf()` function.

```
function Base() {}
function Derived() {}
// Set the `[[Prototype]]` of `Derived.prototype`
// to `Base.prototype`
Object.setPrototypeOf(Derived.prototype, Base.prototype);

const obj = new Derived();
// obj ---> Derived.prototype ---> Base.prototype ---> Object.prototype ---> null
```

In class terms, this is equivalent to using the `extends` syntax.

```
class Base {}
class Derived extends Base {}

const obj = new Derived();
// obj ---> Derived.prototype ---> Base.prototype ---> Object.prototype ---> null
```

You may also see some legacy code using `Object.create()` to build the inheritance chain. However, because this reassigns the `prototype` property and removes the `constructor` property, it can be more error-prone, while performance gains may not be apparent if the constructors haven't created any instances yet.

```
function Base() {}
function Derived() {}
// Re-assigns `Derived.prototype` to a new object
// with `Base.prototype` as its `[[Prototype]]`
// DON'T DO THIS — use Object.setPrototypeOf to mutate it instead
Derived.prototype = Object.create(Base.prototype);
```

Inspecting prototypes: a deeper dive

Let's look at what happens behind the scenes in a bit more detail.

In JavaScript, as mentioned above, functions are able to have properties. All functions have a special property named `prototype`. Please note that the code below is free-standing (it is safe to assume there is no other JavaScript on the webpage other than the below code). For the best learning experience, it is highly recommended that you open a console, navigate to the "console" tab, copy-and-paste in the below JavaScript code, and run it by pressing the Enter/Return key. (The console is included in most web browser's Developer Tools. More information is available for Firefox Developer Tools, Chrome DevTools, and Edge DevTools.)

```
function doSomething() {}
console.log(doSomething.prototype);
// It does not matter how you declare the function; a
// function in JavaScript will always have a default
// prototype property — with one exception: an arrow
// function doesn't have a default prototype property:
const doSomethingFromArrowFunction = () => {};
console.log(doSomethingFromArrowFunction.prototype);
```

As seen above, `doSomething()` has a default `prototype` property, as demonstrated by the console. After running this code, the console should have displayed an object that looks similar to this.

```
{
  constructor: ƒ doSomething(),
  [[Prototype]]: {
    constructor: ƒ Object(),
    hasOwnProperty: ƒ hasOwnProperty(),
    isPrototypeOf: ƒ isPrototypeOf(),
    propertyIsEnumerable: ƒ propertyIsEnumerable(),
    toLocaleString: ƒ toLocaleString(),
    toString: ƒ toString(),
    valueOf: ƒ valueOf()
  }
}
```

> **Note:** The Chrome console uses `[[Prototype]]` to denote the object's prototype, following the spec's terms; Firefox uses `<prototype>`. For consistency we will use `[[Prototype]]`.

We can add properties to the prototype of `doSomething()`, as shown below.

```
function doSomething() {}
doSomething.prototype.foo = "bar";
console.log(doSomething.prototype);
```

This results in:

```
{
  foo: "bar",
  constructor: ƒ doSomething(),
  [[Prototype]]: {
    constructor: ƒ Object(),
    hasOwnProperty: ƒ hasOwnProperty(),
    isPrototypeOf: ƒ isPrototypeOf(),
    propertyIsEnumerable: ƒ propertyIsEnumerable(),
    toLocaleString: ƒ toLocaleString(),
    toString: ƒ toString(),
    valueOf: ƒ valueOf()
  }
}
```

We can now use the `new` operator to create an instance of `doSomething()` based on this prototype. To use the new operator, call the function normally except prefix it with `new`. Calling a function with the `new` operator returns an object that is an instance of the function. Properties can then be added onto this object.

Try the following code:

```
function doSomething() {}
doSomething.prototype.foo = "bar"; // add a property onto the prototype
const doSomeInstancing = new doSomething();
doSomeInstancing.prop = "some value"; // add a property onto the object
console.log(doSomeInstancing);
```

This results in an output similar to the following:

```
{
  prop: "some value",
  [[Prototype]]: {
    foo: "bar",
    constructor: ƒ doSomething(),
    [[Prototype]]: {
      constructor: ƒ Object(),
      hasOwnProperty: ƒ hasOwnProperty(),
      isPrototypeOf: ƒ isPrototypeOf(),
      propertyIsEnumerable: ƒ propertyIsEnumerable(),
      toLocaleString: ƒ toLocaleString(),
      toString: ƒ toString(),
      valueOf: ƒ valueOf()
    }
  }
}
```

As seen above, the `[[Prototype]]` of `doSomeInstancing` is `doSomething.prototype`. But, what does this do? When you access a property of `doSomeInstancing`, the runtime first looks to see if `doSomeInstancing` has that property.

If `doSomeInstancing` does not have the property, then the runtime looks for the property in `doSomeInstancing.[[Prototype]]` (a.k.a. `doSomething.prototype`). If `doSomeInstancing.[[Prototype]]` has the property being looked for, then that property on `doSomeInstancing.[[Prototype]]` is used.

Otherwise, if `doSomeInstancing.[[Prototype]]` does not have the property, then `doSomeInstancing.[[Prototype]].[[Prototype]]` is checked for the property. By default, the `[[Prototype]]` of any function's `prototype` property is `Object.prototype`. So, `doSomeInstancing.[[Prototype]].[[Prototype]]` (a.k.a. `doSomething.prototype.[[Prototype]]` (a.k.a. `Object.prototype`)) is then looked through for the property being searched for.

If the property is not found in `doSomeInstancing.[[Prototype]].[[Prototype]]`, then `doSomeInstancing.[[Prototype]].[[Prototype]].[[Prototype]]` is looked through. However, there is a problem: `doSomeInstancing.[[Prototype]].[[Prototype]].[[Prototype]]` does not exist, because `Object.prototype.[[Prototype]]` is `null`. Then, and only then, after the entire prototype chain of `[[Prototype]]`'s is looked through, the runtime asserts that the property does not exist and conclude that the value at the property is `undefined`.

Let's try entering some more code into the console:

```
function doSomething() {}
doSomething.prototype.foo = "bar";
const doSomeInstancing = new doSomething();
doSomeInstancing.prop = "some value";
console.log("doSomeInstancing.prop:     ", doSomeInstancing.prop);
console.log("doSomeInstancing.foo:      ", doSomeInstancing.foo);
console.log("doSomething.prop:          ", doSomething.prop);
console.log("doSomething.foo:           ", doSomething.foo);
console.log("doSomething.prototype.prop:", doSomething.prototype.prop);
console.log("doSomething.prototype.foo: ", doSomething.prototype.foo);
```

This results in the following:

```
doSomeInstancing.prop:      some value
doSomeInstancing.foo:       bar
```

```
doSomething.prop:           undefined
doSomething.foo:            undefined
doSomething.prototype.prop: undefined
doSomething.prototype.foo:  bar
```

Different ways of creating and mutating prototype chains

We have encountered many ways to create objects and change their prototype chains. We will systematically summarize the different ways, comparing each approach's pros and cons.

Objects created with syntax constructs

```
const o = { a: 1 };
// The newly created object o has Object.prototype as its [[Prototype]]
// Object.prototype has null as its [[Prototype]].
// o ---> Object.prototype ---> null

const b = ["yo", "sup", "?"];
// Arrays inherit from Array.prototype
// (which has methods indexOf, forEach, etc.)
// The prototype chain looks like:
// b ---> Array.prototype ---> Object.prototype ---> null

function f() {
  return 2;
}
// Functions inherit from Function.prototype
// (which has methods call, bind, etc.)
// f ---> Function.prototype ---> Object.prototype ---> null

const p = { b: 2, __proto__: o };
// It is possible to point the newly created object's [[Prototype]] to
// another object via the __proto__ literal property. (Not to be confused
// with Object.prototype.__proto__ accessors)
// p ---> o ---> Object.prototype ---> null
```

When using the `__proto__` key in object initializers, pointing the `__proto__` key to something that is not an object only fails silently without throwing an exception. Contrary to the `Object.prototype.__proto__` setter, `__proto__` in object literal initializers is standardized and optimized, and can even be more performant than `Object.create`. Declaring extra own properties on the object at creation is more ergonomic than `Object.create`.

With constructor functions

```
function Graph() {
  this.vertices = [];
  this.edges = [];
}

Graph.prototype.addVertex = function (v) {
  this.vertices.push(v);
};

const g = new Graph();
// g is an object with own properties 'vertices' and 'edges'.
// g.[[Prototype]] is the value of Graph.prototype when new Graph() is executed.
```

Constructor functions have been available since very early JavaScript. Therefore, it is very fast, very standard, and very JIT-optimizable. However, it's also hard to "do properly" because methods added this way are enumerable by default, which is inconsistent with the class syntax or how built-in methods behave. Doing longer inheritance chains is also error-prone, as previously demonstrated.

With Object.create()

Calling `Object.create()` creates a new object. The `[[Prototype]]` of this object is the first argument of the function:

```
const a = { a: 1 };
// a ---> Object.prototype ---> null

const b = Object.create(a);
// b ---> a ---> Object.prototype ---> null
console.log(b.a); // 1 (inherited)

const c = Object.create(b);
// c ---> b ---> a ---> Object.prototype ---> null

const d = Object.create(null);
// d ---> null (d is an object that has null directly as its prototype)
console.log(d.hasOwnProperty);
// undefined, because d doesn't inherit from Object.prototype
```

Similar to the `__proto__` key in object initializers, `Object.create()` allows directly setting the prototype of an object at creation time, which permits the runtime to further optimize the object. It also allows the creation of objects with `null` prototype, using `Object.create(null)`. The second parameter of `Object.create()` allows you to precisely specify the attributes of each property in the new object, which can be a double-edged sword:

- It allows you to create non-enumerable properties, etc., during object creation, which is not possible with object literals.

- It is much more verbose and error-prone than object literals.
- It may be slower than object literals, especially when creating many properties.

With classes

```
class Rectangle {
  constructor(height, width) {
    this.name = "Rectangle";
    this.height = height;
    this.width = width;
  }
}

class FilledRectangle extends Rectangle {
  constructor(height, width, color) {
    super(height, width);
    this.name = "Filled rectangle";
    this.color = color;
  }
}

const filledRectangle = new FilledRectangle(5, 10, "blue");
// filledRectangle ---> FilledRectangle.prototype ---> Rectangle.prototype ---> Object.prototype ---> null
```

Classes offer the highest readability and maintainability when defining complex inheritance structures. Private elements are a feature with no trivial replacement in prototypal inheritance. However, classes are less optimized than traditional constructor functions and are not supported in older environments.

With Object.setPrototypeOf()

While all methods above will set the prototype chain at object creation time, `Object.setPrototypeOf()` allows mutating the `[[Prototype]]` internal property of an existing object. It can even force a prototype on a prototype-less object created with `Object.create(null)` or remove the prototype of an object by setting it to `null`.

```
const obj = { a: 1 };
const anotherObj = { b: 2 };
Object.setPrototypeOf(obj, anotherObj);
// obj ---> anotherObj ---> Object.prototype ---> null
```

However, you should set the prototype during creation if possible, because setting the prototype dynamically disrupts all optimizations that engines have made to the prototype chain. It might cause some engines to recompile your code for de-optimization, to make it work according to the specs.

With the __proto__ accessor

All objects inherit the `Object.prototype.__proto__` setter, which can be used to set the `[[Prototype]]` of an existing object (if the `__proto__` key is not overridden on the object).

> **Warning:** `Object.prototype.__proto__` accessors are **non-standard** and deprecated. You should almost always use `Object.setPrototypeOf` instead.

```
const obj = {};
// DON'T USE THIS: for example only.
obj.__proto__ = { barProp: "bar val" };
obj.__proto__.__proto__ = { fooProp: "foo val" };
console.log(obj.fooProp);
console.log(obj.barProp);
```

Compared to `Object.setPrototypeOf`, setting `__proto__` to something that is not an object fails silently without throwing an exception. It also has slightly better browser support. However, it is non-standard and deprecated. You should almost always use `Object.setPrototypeOf` instead.

Performance

The lookup time for properties that are high up on the prototype chain can have a negative impact on the performance, and this may be significant in the code where performance is critical. Additionally, trying to access nonexistent properties will always traverse the full prototype chain.

Also, when iterating over the properties of an object, **every** enumerable property that is on the prototype chain will be enumerated. To check whether an object has a property defined on *itself* and not somewhere on its prototype chain, it is necessary to use the `hasOwnProperty` or `Object.hasOwn` methods. All objects, except those with `null` as `[[Prototype]]`, inherit `hasOwnProperty` from `Object.prototype` — unless it has been overridden further down the prototype chain. To give you a concrete example, let's take the above graph example code to illustrate it:

```
function Graph() {
  this.vertices = [];
  this.edges = [];
}

Graph.prototype.addVertex = function (v) {
  this.vertices.push(v);
};

const g = new Graph();
// g ---> Graph.prototype ---> Object.prototype ---> null

g.hasOwnProperty("vertices"); // true
Object.hasOwn(g, "vertices"); // true
```

```
g.hasOwnProperty("nope"); // false
Object.hasOwn(g, "nope"); // false

g.hasOwnProperty("addVertex"); // false
Object.hasOwn(g, "addVertex"); // false

Object.getPrototypeOf(g).hasOwnProperty("addVertex"); // true
```

Note: It is **not** enough to check whether a property is `undefined`. The property might very well exist, but its value just happens to be set to `undefined`.

Conclusion

JavaScript may be a bit confusing for developers coming from Java or C++, as it's all dynamic, all runtime, and it has no static types at all. Everything is either an object (instance) or a function (constructor), and even functions themselves are instances of the `Function` constructor. Even the "classes" as syntax constructs are just constructor functions at runtime.

All constructor functions in JavaScript have a special property called `prototype`, which works with the `new` operator. The reference to the prototype object is copied to the internal `[[Prototype]]` property of the new instance. For example, when you do `const a1 = new A()`, JavaScript (after creating the object in memory and before running function `A()` with `this` defined to it) sets `a1.[[Prototype]] = A.prototype`. When you then access properties of the instance, JavaScript first checks whether they exist on that object directly, and if not, it looks in `[[Prototype]]`. `[[Prototype]]` is looked at *recursively*, i.e., `a1.doSomething`, `Object.getPrototypeOf(a1).doSomething`, `Object.getPrototypeOf(Object.getPrototypeOf(a1)).doSomething` etc., until it's found or `Object.getPrototypeOf` returns `null`. This means that all properties defined on `prototype` are effectively shared by all instances, and you can even later change parts of `prototype` and have the changes appear in all existing instances.

If, in the example above, you do `const a1 = new A(); const a2 = new A();`, then `a1.doSomething` would actually refer to `Object.getPrototypeOf(a1).doSomething` — which is the same as the `A.prototype.doSomething` you defined, i.e., `Object.getPrototypeOf(a1).doSomething === Object.getPrototypeOf(a2).doSomething === A.prototype.doSomething`.

It is essential to understand the prototypal inheritance model before writing complex code that makes use of it. Also, be aware of the length of the prototype chains in your code and break them up if necessary to avoid possible performance problems. Further, the native prototypes should **never** be extended unless it is for the sake of compatibility with newer JavaScript features.

Internationalization

- Previous
- Next

The `Intl` object is the namespace for the ECMAScript Internationalization API, which provides a wide range of locale- and culture-sensitive data and operations.

Overview

The `Intl` object is very use-case driven. It provides a separate object for each use case that requires locale-specific logic. Currently, it provides the following functionalities:

- Getting information about a locale using `Intl.Locale`.
- Formatting data using `Intl.DateTimeFormat`, `Intl.DurationFormat`, `Intl.ListFormat`, `Intl.NumberFormat`, and `Intl.RelativeTimeFormat`.
- Collation (i.e., comparing strings for sorting or searching) using `Intl.Collator`.
- Selecting plural forms using `Intl.PluralRules`.
- Segmenting text into units such as words, sentences, or graphemes using `Intl.Segmenter`.
- Getting the displayed name for currencies, languages, scripts, regions, and time zones using `Intl.DisplayNames`.

Most `Intl` APIs share a similar design (`Intl.Locale` is the only exception). You start by constructing an instance with the desired locale and options. This defines a set of rules for the desired operation (formatting, collation, segmentation, etc.). Then, when you call the method on the instance, such as `format()`, `compare()`, or `segment()`, the object applies the specified rule to the passed data.

```
// 1. Construct a formatter object, specifying the locale and formatting options:
const price = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
});

// 2. Use the `format` method of the formatter object to format a number:
console.log(price.format(5.259)); // $5.26
```

The general signature of the constructor is:

```
new Intl.SomeObject(locales, options)
```

`locales` Optional

> A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object containing properties that customize specific aspects of the operation, which is key to understanding how to use each `Intl` object.

Locale information

Locales underlie all behaviors of `Intl`. A *locale* is a set of conventions, represented in the `Intl` API by the `Intl.Locale` object. All `Intl` constructors that accept language tags also accept `Intl.Locale` objects.

Each locale is primarily defined by four things: a `language`, a `script`, a `region`, and sometimes some `variants`. When connected together by `-` in that order, they form a BCP 47 language tag.

- The language is the most important part of the locale and is mandatory. When given a single language, like `en` or `fr`, there are algorithms to infer the rest of the information (see `Intl.Locale.prototype.maximize()`).
- However, you often want to specify the region as well, because conventions can differ drastically between regions that speak the same language. For example, the date format in the US is MM/DD/YYYY, whereas in the UK it is DD/MM/YYYY, so specifying `en-US` or `en-GB` is important.
- You can also specify a script. The script is the writing system, or the characters used to transcribe the language. In practice, the script is often unnecessary, because the language used in a certain region is only written in one script. However, there are exceptions such as the Serbian language, which can be written in both the Latin and Cyrillic scripts (`sr-Latn` and `sr-Cyrl`), or the Chinese language, which can be written in both the Simplified and Traditional scripts (`zh-Hans` and `zh-Hant`).
- The variants are rarely used. Usually, they denote different orthographies; for example, German has the `1901` and `1996` orthography variants, which are written as `de-1901` and `de-1996`.

```
// These two are equivalent when passed to other Intl APIs
const locale1 = new Intl.Locale("en-US");
const locale2 = new Intl.Locale("en-Latn-US");

console.log(locale1.language, locale1.script, locale1.region); // "en", undefined, "US"
console.log(locale2.language, locale2.script, locale2.region); // "en", "Latn", "US"
```

A locale also contains a set of conventions used by that particular culture:

Use case	Property	Description	Extension subtag
Date/time formatting	`calendar`	Used to group days into years, months, and weeks, and to assign names to them. For example, the `gregory` date "2022-01-01" becomes "28 Tevet 5782" in the `hebrew` calendar.	`ca`
	`hourCycle`	Decides if times are displayed in 12-hour or 24-hour format, and whether the smallest hour number is 0 or 1.	`hc`
Number formatting, including dates, times, durations, etc.	`numberingSystem`	Transforms numbers into a locale-specific notation. The regular `0123456789` system is called `latn` (Latin). Often, each script has a numbering system that is just a digit-by-digit translation, but some scripts have more than one numbering system, some may not typically write numbers in that script (for example, Chinese has its own `hanidec` numbering system, but most texts use the standard `latn` system), and others may require special conversion algorithms (for example, Roman numerals — `roman`).	`nu`
Collation	`collation`	Defines the generic collation algorithm. For example, if you use the German `phonebk` collation, then "ä" is treated as "ae", and will be sorted between "ad" and "af".	`co`
	`caseFirst`	Decides whether uppercase or lowercase letters should be sorted first, or if casing is ignored.	`kf`
	`numeric`	Decides whether numbers are sorted as numbers or as strings. For example, if true, "10" will be sorted after "2".	`kn`

You can explicitly specify these properties when constructing the `Intl.Locale` or passing language tags to other `Intl` constructors. There are two ways to do this — append them to the language tag or specify them as options.

- To append them to the language tag, you first append the string `-u` (meaning "Unicode extension"), then the extension subtag as given above, then the value.
- To specify them as options, you just add the property name as given above, together with its value, to the `options` object.

Using `Intl.DateTimeFormat` as an example, both of the following lines create a formatter that formats dates in the Hebrew calendar:

```
const df1 = new Intl.DateTimeFormat("en-US-u-ca-hebrew");
const df2 = new Intl.DateTimeFormat("en-US", { calendar: "hebrew" });
```

Unrecognized properties are ignored, so you can use the same syntax as above with `Intl.NumberFormat`, but it won't do anything different than just passing `en-US`, because number formatting doesn't use the `calendar` property.

It is tricky to get the default values of these locale conventions. `new Intl.Locale("en-US").calendar` returns `undefined`, because the `Locale` object only contains the information you passed to it. The default calendar is theoretically dependent on what API you are using the calendar with, so to get the default calendar of `en-US` as used by `Intl.DateTimeFormat`, you can use its `resolvedOptions()` method. The same goes for other properties.

```
const locale = new Intl.Locale("en-US");
console.log(locale.calendar); // undefined; it's not provided
console.log(new Intl.DateTimeFormat(locale).resolvedOptions().calendar); // "gregory"
```

`Intl.Locale` objects do two things simultaneously: they represent a parsed BCP 47 language tag (as demonstrated above), and they provide information about that locale. All of its properties, like `calendar`, are extracted from the input only, without querying any data source about default values. On the other hand, it has a group of methods for querying real-world information about the locale. For example, the `getCalendars()`, `getHourCycles()`, `getNumberingSystems()`, and `getCollations()` methods complement the `calendar`, `hourCycle`, `numberingSystem`, and `collation` properties, and each returns an array of preferred values for that property.

```
const locale = new Intl.Locale("ar-EG");
console.log(locale.getCalendars()); // ['gregory', 'coptic', 'islamic', 'islamic-civil', 'islamic-tbla']
```

`Intl.Locale` instances also contain other methods that expose useful information, such as `getTextInfo()`, `getTimeZones()`, and `getWeekInfo()`.

Figuring out the locale

A shared concern for internationalization is: how do I know what locale to use?

The most obvious answer is "what the user prefers." Browsers expose the user's language preferences through the `navigator.languages` property. This is an array of language identifiers that can be directly passed to the formatter constructor—more on this later. The user can configure this list in their browser settings. You can also pass an empty array or `undefined`, which both cause the browser's default locale to be used.

```
const numberFormatter = new Intl.NumberFormat(navigator.languages);
console.log(numberFormatter.format(1234567.89));

const numberFormatter2 = new Intl.NumberFormat([]);
```

However, this may not always provide the most desirable result. Strings formatted by `Intl` formatters represent a tiny fraction of text displayed on your site; most localized content is provided by you, the site developer. For example, suppose your site is only offered in two languages: English and French. If a Japanese user visits your site and expects to use your site in English, they will be baffled when they see the English text interleaved with numbers and dates in Japanese!

Usually, you don't want to use the browser's default language. Rather, you want to use the same language that the rest of your site is offered in. Suppose your site has a language switcher that stores the user's choice somewhere — you could directly use that.

```
// Suppose this can be changed by some site-wide control
const userSettings = {
  locale: "en-US",
  colorMode: "dark",
};
const numberFormatter = new Intl.NumberFormat(userSettings.locale);
console.log(numberFormatter.format(1234567.89));
```

If your site has a backend that dynamically selects the language based on the user's `Accept-Language` header and sends back different HTML based on that, you could also use the HTML element's `HTMLElement.lang` property: `new Intl.NumberFormat(document.documentElement.lang)`.

If your site is only offered in one language, you could also hardcode the locale in your code: `new Intl.NumberFormat("en-US")`.

As previously mentioned, you can also pass an array of locales to the constructor, representing a list of fallback choices. The first example using `navigator.languages` is an example of this: if the first user-configured locale is not supported for the particular operation, the next one is tried, and so on, until we find a requested locale for which the runtime has data. You can do this manually as well. In the example below, we specify a list of locales in decreasing order of specificity which all represent languages likely to be understood by a Hong Kong Chinese speaker, so the formatter picks the most specific one that it supports.

```
const numberFormatter = new Intl.NumberFormat([
  "yue-Hant",
  "zh-Hant-HK",
  "zh-Hant",
  "zh",
]);
```

There is no API for listing all supported locales, but there are a few methods for handling the locale list:

- `Intl.getCanonicalLocales()`: This function takes a list of locale identifiers and returns a list of canonicalized locale identifiers. This is useful for understanding the canonicalization process for each `Intl` constructor.
- The `supportedLocalesOf()` static method on each `Intl` object (like `Intl.DateTimeFormat.supportedLocalesOf()`): This method takes the same arguments as the constructor (`locales` and `options`) and returns a subset of the given locale tags that match the given data. This is useful for understanding which locales are supported by the runtime for a particular operation, for example, to show a language switcher containing only the supported languages.

Understanding the return value

The second shared concern for all objects is "what does the method return?" This is a hard question to answer beyond the structure or type of the returned value because there's no normative specification saying what *exactly* should be returned. Most of the time, the result of a method is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string returned by `format()` may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of any `Intl` method to hardcoded constants; they should only be displayed to users.

Of course, this answer seems unsatisfactory, because most developers do wish to control what the output looks like—at least, you don't want your user to be confused by nonsensical output. Here are some guidelines if you do want to conduct testing, whether automated or manual:

- Test all locales that your user may use. This is easier if you have a fixed set of supported locales (for example via a language switcher). If you are using whatever the user prefers, you may pick a few common ones for your users, but bear in mind that what the user sees could vary. You can usually mock user preference via the test runner's configuration or mocking the `Intl` constructors.
- Test on multiple JavaScript engines. The `Intl` API is directly implemented by the JavaScript engine, so for example you should expect Node.js and Chrome (which both use V8) to have the same output, while Firefox (which uses SpiderMonkey) may have a different output. Although all engines likely use the CLDR data, they usually postprocess it in different ways. Some browser build settings (for reducing the install size, for example) may also affect what locales and options are supported.
- Don't assume the output. This means you shouldn't write the output by hand, such as `expect(result).toBe("foo")`. Instead, use snapshot testing or copy the string value from the output of a test run.

Formatting data

A major use case of `Intl` is to output locale-specific texts representing structured data. This is similar to translation software, but instead of letting you translate arbitrary text, it takes data such as dates, numbers, and lists, and formats them according to locale-specific rules.

The `Intl.DateTimeFormat`, `Intl.DurationFormat`, `Intl.ListFormat`, `Intl.NumberFormat`, and `Intl.RelativeTimeFormat` objects each format one kind of data. Each instance provides two methods:

- `format()`: Takes a piece of data and returns a string using the formatting rule as determined by the locale and options.
- `formatToParts()`: Takes the same data and returns the same string, but broken down into parts, each part being an object with a `type` and a `value`. This is useful for more advanced use cases, such as interleaving the formatted text with other texts.

For example, here's a typical use of the `Intl.NumberFormat` object:

```
// 1. Construct a formatter object, specifying the locale and formatting options:
const price = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
});

// 2. Use the `format` method of the formatter object to format a number:
console.log(price.format(5.259)); // $5.26

// Or, use the `formatToParts` method to get the formatted number
// broken down into parts:
console.table(price.formatToParts(5.259));
// |   | type       | value |
// | 0 | "currency" | "$"   |
// | 1 | "integer"  | "5"   |
// | 2 | "decimal"  | "."   |
// | 3 | "fraction" | "26"  |
```

You don't always have to construct a formatter object in order to format strings. For casual use, you can also directly call the `toLocaleString()` method on the data, passing the locale and options as arguments. The `toLocaleString()` method is implemented by `Temporal.PlainDate.prototype.toLocaleString()`, `Temporal.Duration.prototype.toLocaleString()`, `Number.prototype.toLocaleString()`, and so on. Read the documentation for the data you are formatting to see if it supports `toLocaleString()`, and what formatter options it corresponds to.

```
console.log(
  (5.259).toLocaleString("en-US", {
    style: "currency",
    currency: "USD",
  }),
); // $5.26
```

Note that `toLocaleString()` is potentially less efficient than using a formatter object, because every time `toLocaleString` is called, it has to perform a search in a big database of localization strings. When the method is called many times with the same arguments, it is better to create a formatter object and use its `format()` method, because a formatter object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Date and time formatting

`Intl.DateTimeFormat` formats dates and times, as well as ranges of dates and times. The `DateTimeFormat` object takes date/time inputs in one of the following forms: `Date`, `Temporal.PlainDateTime`, `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay`.

> **Note:** You can't pass a `Temporal.ZonedDateTime` object directly, because the time zone is already fixed in the object. You should use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object first.

Common use cases of localized date and time formatting are as follows:

- Output the same date and time in another calendar system, such as the Islamic, Hebrew, or Chinese calendar.
- Output the same real-world time (instant) but in another time zone.
- Selectively output certain components of the date and time, such as only the year and month, and the specific representation of them (such as "Thursday" or "Thu").
- Output the date according to locale-specific conventions, such as MM/DD/YYYY as in the U.S., DD/MM/YYYY as in the U.K., or YYYY/MM/DD as in Japan.
- Output the time according to locale-specific conventions, such as 12-hour or 24-hour clock.

To decide what the formatted string looks like, you first choose the calendar (which affects the year, month, week, and day calculation) and time zone (which affects the exact time as well as possibly the date). This is done using the aforementioned `calendar` option (or the `-ca-` extension key in the locale identifier), and the `timeZone` option.

- `Date` objects represent a unique instant in the user's time zone and in the ISO 8601 calendar (as reported by methods like `Date.prototype.getHours()` and `Date.prototype.getMonth()`). They are converted to the given `calendar` and `timeZone` by preserving the instant, so the date and time components may change.
- The various `Temporal` objects already have a calendar built-in, so the `calendar` option must be consistent with the calendar of the object—unless the date's calendar is `"iso8601"`, in which case it is converted to the requested `calendar`. These objects don't have a time zone, so they are directly displayed in the given `timeZone` without conversion.

Here we demonstrate how the combination of `calendar` and `timeZone` configurations result in different representations of the same instant.

```
// Assume that the local time zone is UTC
const targetDate = new Date(2022, 0, 1); // 2022-01-01 midnight in the local time zone
const results = [];

for (const calendar of ["gregory", "hebrew"]) {
  for (const timeZone of ["America/New_York", "Asia/Tokyo"]) {
```

```
    const df = new Intl.DateTimeFormat("en-US", {
      calendar,
      timeZone,
      // More on these later
      dateStyle: "full",
      timeStyle: "full",
    });
    results.push({ calendar, timeZone, output: df.format(targetDate) });
  }
}

console.table(results);
```

The output looks like this:

calendar	timeZone	output
'gregory'	'America/New_York'	'Friday, December 31, 2021 at 7:00:00 PM Eastern Standard Time'
'gregory'	'Asia/Tokyo'	'Saturday, January 1, 2022 at 9:00:00 AM Japan Standard Time'
'hebrew'	'America/New_York'	'Friday, 27 Tevet 5782 at 7:00:00 PM Eastern Standard Time'
'hebrew'	'Asia/Tokyo'	'Saturday, 28 Tevet 5782 at 9:00:00 AM Japan Standard Time'

A date/time is comprised of the following components: `weekday`, `era`, `year`, `month`, `day`, `dayPeriod`, `hour`, `minute`, `second`, `fractionalSecondDigits`, and `timeZoneName`. Your next decision is what components to include in the output, and what forms they should take. You have two choices:

- You can manually configure each component, using options with the same name as the component. Only the components you specify will be included in the output, with the specified form.
- You can use the shortcuts `dateStyle` and `timeStyle`, which are predefined sets of components. They expand to a set of component options depending on the locale.

You should pick one of these two methods, because they are mutually exclusive. Using both methods simultaneously will result in an error.

Underlyingly, after requesting a combination of components, the `DateTimeFormat` object looks for a "template" that matches the requested components, so it only needs to fill in the values one by one. Not every combination of components has a predefined template. `DateTimeFormat` has a `formatMatcher` option that decides how to negotiate, by making components longer or shorter than requested, or by omitting or adding components. It gets quite technical, so you should read the `Intl.DateTimeFormat()` reference to better understand how it handles this.

Here, we demonstrate a few common ways to format the components:

```
const df1 = new Intl.DateTimeFormat("en-US", {
  // Include all components (usually)
  dateStyle: "full",
  timeStyle: "full",
});

const df2 = new Intl.DateTimeFormat("en-US", {
  // Display the calendar date
  era: "short",
  year: "numeric",
  month: "long",
  day: "numeric",
});

const df3 = new Intl.DateTimeFormat("en-US", {
  // Display a time like on a digital clock
  hour: "2-digit",
  minute: "2-digit",
  second: "2-digit",
  timeZoneName: "shortOffset",
});

const targetDate = new Date(2022, 0, 1, 12, 34, 56); // 2022-01-01 12:34:56 in the local time zone
console.log(df1.format(targetDate));
// Saturday, January 1, 2022 at 12:34:56 PM Coordinated Universal Time
// January 1, 2022 AD
// 12:34:56 PM GMT
```

There are other customization options. For example, you can use the `hourCycle` option to display the time in 12-hour or 24-hour and show midnight/noon as 12:00 or 0:00. You can also use the `numberingSystem` option to display any numbers in another numbering system.

Apart from `format()`, there's a second important method, `formatRange()`, which formats a range of dates or times. It takes two date-times of the same type, formats each one, joins them with a range separator (like the en-dash), and deduplicates the common parts.

```
const springBreak = {
  start: new Date(2023, 2, 10),
  end: new Date(2023, 2, 26),
};

const df = new Intl.DateTimeFormat("en-US", { dateStyle: "long" });
console.log(df.formatRange(springBreak.start, springBreak.end));
// March 10 – 26, 2023
```

Number formatting

Number formatting is done with the `Intl.NumberFormat` object. The `NumberFormat` object accepts inputs in the form of numbers, strings, or `BigInt` values. Passing a string or `BigInt` instead of a number allows you to format numbers that are too large or too small to be represented precisely as a JavaScript number.

Common use cases of localized number formatting are as follows:

- Output the number in another numbering system (script), such as Chinese, Arabic, or Roman.
- Output the number with locale-specific conventions, such as the decimal symbol ("." in English, but "," in many European cultures), or digit grouping (3 digits in English, but may be 4 or 2 in other cultures, and may use ",", " ", or ".").
- Output the number with exponential notation such as "3.7 million" or "2 thousand".
- Output the number as a currency, applying specific currency symbols and rounding rules. For example, monetary values less than one cent in the U.S. or less than one yen in Japan may not be meaningful to display.
- Output the number as a percentage, applying locale-specific conversion and formatting rules.
- Output the number with units, such as "meters" or "liters", with translated unit names.

To decide what the formatted string looks like, you first choose the numbering system (which affects the characters used for the digits). The purpose of a numbering system is already discussed in locale information. Another option you need to decide is the `style`, which sets the context for what the number represents, and potentially affects other options' default values. It is one of `"decimal"`, `"percent"`, `"currency"`, or `"unit"`. If you want to format currencies, then you also need to provide the `currency` option. If you want to format units, then you also need to provide the `unit` option.

```
const results = [];
for (const options of [
  { style: "decimal" }, // Format the number as a dimensionless decimal
  { style: "percent" }, // Format the number as a percentage; it is multiplied by 100
  { style: "currency", currency: "USD" }, // Format the number as a US dollar amount
  { style: "unit", unit: "meter" }, // Format the number as a length in meters
]) {
  const nf = new Intl.NumberFormat("en-US", options);
  results.push({ style: options.style, output: nf.format(1234567.89) });
}
console.table(results);
```

The output looks like:

style	output
'decimal'	'1,234,567.89'
'percent'	'123,456,789%'
'currency'	'$1,234,567.89'
'unit'	'1,234,567.89 m'

The next group of options all specify what the numeric part should look like. First, you may want to represent extremely large values in a more readable way. You can set the `notation` option to `"scientific"` or `"engineering"`, which both use the `1.23e+6` notation. The difference is that the latter uses multiples of 3 for the exponent, keeping the mantissa (the part before the `e` symbol) between 1 and 1000, while the former can use any integer for the exponent, keeping the mantissa between 1 and 10. You can also set `notation` to `"compact"` to use a more human-readable notation.

```
const results = [];
for (const options of [
  { notation: "scientific" },
  { notation: "engineering" },
  { notation: "compact", compactDisplay: "short" }, // "short" is default
  { notation: "compact", compactDisplay: "long" },
]) {
  const nf = new Intl.NumberFormat("en-US", options);
  results.push({
    notation: options.compactDisplay
      ? `${options.notation}-${options.compactDisplay}`
      : options.notation,
    output: nf.format(12000),
  });
}
console.table(results);
```

The output looks like:

notation	output
'scientific'	'1.2E4'
'engineering'	'12E3'
'compact-short'	'12K'
'compact-long'	'12 thousand'

Then, you may want to round the number (if you specified `notation`, then just the mantissa part), so you don't display a number that's too long. These are the digit options, which include:

- `minimumIntegerDigits`
- `minimumFractionDigits`
- `maximumFractionDigits`

- `minimumSignificantDigits`
- `maximumSignificantDigits`
- `roundingPriority`
- `roundingIncrement`
- `roundingMode`

The exact interaction of these options is quite complex and not worth covering here. You should read the digit options reference for more detail. Nevertheless, the general idea is straightforward: we first find the number of decimal places we want to keep, then we round excess decimal places off, either rounding down or up, depending on the value of the last digit.

```
const results = [];
for (const options of [
  { minimumFractionDigits: 4, maximumFractionDigits: 4 },
  { minimumSignificantDigits: 4, maximumSignificantDigits: 4 },
  { minimumFractionDigits: 0, maximumFractionDigits: 0, roundingMode: "floor" },
  {
    minimumFractionDigits: 0,
    maximumFractionDigits: 0,
    roundingMode: "floor",
    roundingIncrement: 10,
  },
]) {
  const nf = new Intl.NumberFormat("en-US", options);
  results.push({
    options,
    output: nf.format(1234.56789),
  });
}
console.table(results);
```

The output looks like this:

options	output
`{ minimumFractionDigits: 4, maximumFractionDigits: 4 }`	'1,234.5679'
`{ minimumSignificantDigits: 4, maximumSignificantDigits: 4 }`	'1,235'
`{ minimumFractionDigits: 0, maximumFractionDigits: 0, roundingMode: "floor" }`	'1,234'
`{ minimumFractionDigits: 0, maximumFractionDigits: 0, roundingMode: "floor", roundingIncrement: 10 }`	'1,230'

There are other customization options. For example, you can use the `useGrouping` and `signDisplay` options to customize whether and how to show the group separators (like "," in "1,234,567.89") and the sign. However, note that the characters used for the group separator, decimal point, and sign are locale-specific, so you can't customize them directly.

Apart from `format()`, there's a second important method, `formatRange()`, which formats a range of numbers. It takes two number representations, formats each one, joins them with a range separator (like the en-dash), and potentially deduplicates the common parts.

```
const heightRange = {
  min: 1.63,
  max: 1.95,
};

const nf = new Intl.NumberFormat("en-US", { style: "unit", unit: "meter" });
console.log(nf.formatRange(heightRange.min, heightRange.max));
// 1.63–1.95 m
```

List formatting

You may already have written code that does this:

```
const fruits = ["apple", "banana", "cherry"];
console.log(`I like ${fruits.join(", ")}.`);
// I like apple, banana, cherry.
```

This code is not internationalized. In some languages, the list separator is not a comma. In *most* languages (including English), you need a conjunction before the last item. But even just adding an "and" manually doesn't make it correct among all English speakers, because there's the debate of Oxford commas in English: "apple, banana, and cherry" vs. "apple, banana and cherry".

The `Intl.ListFormat` object solves this problem. It takes an array of strings and joins them in a locale-specific way such that the result represents a conjunction (and), disjunction (or), or a list of units.

```
const fruits = ["apple", "banana", "cherry"];
const lf = new Intl.ListFormat("en-US", { style: "long", type: "conjunction" });
console.log(`I like ${lf.format(fruits)}.`);
// I like apple, banana, and cherry.

const lf = new Intl.ListFormat("en-US", { style: "long", type: "disjunction" });
console.log(`I can give you ${lf.format(fruits)}.`);
// I can give you apple, banana, or cherry.
```

Check `Intl.ListFormat()` for more examples and options.

Relative time formatting

`Intl.RelativeTimeFormat` formats a time difference. The `RelativeTimeFormat` object takes relative times in the form of two arguments: a number (with any sign) and a time unit, such as `"day"`, `"hour"`, or `"minute"`.

It does several things at once:

- It localizes and pluralizes the time unit, such as "1 day" vs. "2 days", like in number formatting.
- It chooses the appropriate phrase for past and future times, such as "in 1 day" vs. "1 day ago".
- It may select a special phrase for some time units, such as "1 day ago" vs. "yesterday".

```
const rtf = new Intl.RelativeTimeFormat("en-US", { numeric: "auto" });
console.log(rtf.format(1, "day")); // tomorrow
console.log(rtf.format(2, "day")); // in 2 days
console.log(rtf.format(-1, "hour")); // 1 hour ago
```

Check `Intl.RelativeTimeFormat()` for more examples and options.

Duration formatting

`Intl.DurationFormat` provides duration formatting, such as "3 hours, 4 minutes, 5 seconds". It is not a primitive operation with its own formatter: it uses `Intl.NumberFormat` and `Intl.ListFormat` internally to format each duration component, then joins them with a list separator. The `DurationFormat` object takes durations in the form of a `Temporal.Duration` object, or a plain object with the same properties.

Apart from customizing the numbering system, the duration formatting options decides whether or not to show each component, and how long they should be.

```
console.log(
  new Intl.DurationFormat("en-US", {
    style: "long",
  }).format({ hours: 3, minutes: 4, seconds: 5 }),
);
// 3 hours, 4 minutes, and 5 seconds
```

Check `Intl.DurationFormat()` for more examples and options.

Collation

The `Intl.Collator` object is useful for comparing and sorting strings. It takes two strings and returns a number indicating their relative order, in the same way as the `compareFn` argument of the `Array.prototype.sort` method.

There are many reasons why you should not use JavaScript operators like `===` or `>` to compare user-facing strings:

- Irrelevant orthographic variants: For example, in English, "naïve" and "naive" are just alternative spellings of the same word and should be treated as equal.
- Ignoring case: Often, you want to ignore case when comparing strings. For example, "apple" and "Apple" should be treated as equal.
- Unicode code point order doesn't make sense: Comparison operators like `>` compare by Unicode code point order, which is not the same as the order of characters in a dictionary. For example, "ï" comes after "z" in code point order, but you would want it to be ordered next to "i" in a dictionary.
- Unicode normalization: The same character may have multiple representations in Unicode. For example, "ñ" can be represented as a single character or as "n" followed by a combining tilde. (See `String.prototype.normalize()`.) These should be treated as equal.
- Number comparison: Numbers in strings should be compared as numbers, not as strings. For example, you would want "test-10" to come after "test-2".

There are two distinct use cases for collation: **sorting** and **searching**. Sorting is when you have a list of strings and you want to order them according to some rule. Searching is when you have a list of strings and you want to find a string that matches a query. When searching, you should only pay attention to whether the comparison result is zero (equal) or not, not the sign of the result.

There are many different ways to sort, even within the same locale. For example, there are two different sort orders in German, *phonebook* and *dictionary*. Phonebook sort emphasizes sound — as if "ä", "ö", and so on were expanded to "ae", "oe", and so on before sorting.

```
const names = ["Hochberg", "Hönigswald", "Holzman"];

const germanPhonebook = new Intl.Collator("de-DE-u-co-phonebk");

// as if sorting ["Hochberg", "Hoenigswald", "Holzman"]:
console.log(names.sort(germanPhonebook.compare));
// ['Hochberg', 'Hönigswald', 'Holzman']
```

Some German words conjugate with extra umlauts, so in dictionaries it's sensible to order ignoring umlauts (except when ordering words differing *only* by umlauts: *schon* before *schön*).

```
const germanDictionary = new Intl.Collator("de-DE-u-co-dict");

// as if sorting ["Hochberg", "Honigswald", "Holzman"]:
console.log(names.sort(germanDictionary.compare).join(", "));
// "Hochberg, Holzman, Hönigswald"
```

Plural rules

The `Intl.PluralRules` object is useful for selecting the correct plural form of a word. It doesn't automatically pluralize words for you (for example, you can't pass it "apple" and expect "apples" back), but it tells you which plural form to use based on a number. You may already be doing this:

```
function formatMessage(n) {
  return `You have ${n} ${n === 1 ? "apple" : "apples"}.`;
```

```
}
```

But this is hard to generalize across languages, especially those with many plural forms. You can see `Intl.PluralRules` for a general introduction to plural rules. Here, we just demonstrate a few common use cases.

```
const prCard = new Intl.PluralRules("en-US");
const prOrd = new Intl.PluralRules("en-US", { type: "ordinal" });

const englishOrdinalSuffixes = {
  one: "st",
  two: "nd",
  few: "rd",
  other: "th",
};

const catPlural = {
  one: "cat",
  other: "cats",
};

function formatMessage(n1, n2) {
  return `The ${n1}${englishOrdinalSuffixes[prOrd.select(n1)]} U.S. president had ${n2} ${catPlural[prCard.select(n2)]}.`;
}

console.log(formatMessage(42, 1)); // The 42nd U.S. president had 1 cat.
console.log(formatMessage(45, 0)); // The 45th U.S. president had 0 cats.
```

Segmentation

The `Intl.Segmenter` object is useful for breaking a string into segments. Without `Intl`, you are already able to split a string by UTF-16 code units and Unicode code points:

```
const str = "USCNRUGBFR";
console.log(str.split(""));
// Array(20) ['\uD83C', '\uDDFA', '\uD83C', ...]
console.log([...str]);
// Array(10) ['U', 'S', 'C', 'N', 'R', 'U', 'G', 'B', 'F', 'R']
```

But as you can see, Unicode code points are not the same as what human users perceive as discrete characters. This often happens with emojis, where a single emoji can be represented by multiple code points. When the user interacts with text, a grapheme is the smallest unit of text they can manipulate, such as delete or select. The `Segmenter` object enables grapheme-level segmentation, which is useful for counting characters, measuring text width, and so on. It takes a string and returns an iterable segments object, each element of which has a `segment` property representing the text of the segment.

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "grapheme" });
console.log([...segmenter.segment("USCNRUGBFR")].map((s) => s.segment));
// ['US', 'CN', 'RU', 'GB', 'FR']
```

The segmenter can also do higher-level segmentation, including word-level and sentence-level splitting. These use cases are necessarily language-specific. For example, the following is a very poor implementation of word-counting:

```
const wordCount = (str) => str.split(/\s+/).length;
console.log(wordCount("Hello, world!")); // 2
```

There are several problems with this: not all languages use spaces to separate words, not all spaces are word-separating, and not all words are separated by spaces. To solve this, use `Segmenter` with `granularity: "word"`. The result is the input string, split into segments of words and non-words. If you are counting words, you should filter out the non-words by checking each segment's `isWordLike` property.

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "word" });
const str = "It can even split non-space-separated words";
console.table([...segmenter.segment(str)]);
// ┌─────────────┬───────┬─────────────┐
// │ segment     │ index │ isWordLike  │
// ├─────────────┼───────┼─────────────┤
// │ 'It'        │ 0     │ true        │
// │ ' '         │ 2     │ false       │
// │ 'can'       │ 3     │ true        │
// │ ' '         │ 6     │ false       │
// │ 'even'      │ 7     │ true        │
// │ ' '         │ 11    │ false       │
// │ 'split'     │ 12    │ true        │
// │ ' '         │ 17    │ false       │
// │ 'non'       │ 18    │ true        │
// │ '-'         │ 21    │ false       │
// │ 'space'     │ 22    │ true        │
// │ '-'         │ 27    │ false       │
// │ 'separated' │ 28    │ true        │
// │ ' '         │ 37    │ false       │
// │ 'words'     │ 38    │ true        │
// └─────────────┴───────┴─────────────┘

console.log(
  [...segmenter.segment(str)].filter((s) => s.isWordLike).map((s) => s.segment),
```

```
);
// ['It', 'can', 'even', 'split', 'non', 'space', 'separated', 'words']
```

Word segmentation works for character-based languages too. For example, in Chinese, several characters can represent a single word, but there's no space between them. The segmenter implements the same behavior as the browser's built-in word segmentation, triggered by double-clicking a word.

```
const segmenter = new Intl.Segmenter("zh-Hans", { granularity: "word" });
console.log([...segmenter.segment("我是这篇文档的作者")].map((s) => s.segment));
// ['我是', '这', '篇', '文', '档', '的', '作者']
```

Sentence segmentation is similarly complex. For example, in English, there are many punctuation marks that could mark the end of a sentence (".", "!", "?", and so on).

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "sentence" });
console.log(
  [...segmenter.segment("I ate a sandwich. Then I went to bed.")].map(
    (s) => s.segment,
  ),
);
// ['I ate a sandwich. ', 'Then I went to bed.']
```

Note that the segmenter doesn't remove any characters. It just splits the string into segments, each of which is a sentence. You can then remove the punctuation marks if you want. Also, the current implementation of the segmenter doesn't support sentence segmentation suppressions (preventing sentence breaks after periods like "Mr." or "Approx."), but there's ongoing work to support this.

Display names

After introducing so many options and behaviors, you may wonder how to present them to the user. `Intl` comes with two useful APIs for building user interfaces: `Intl.supportedValuesOf()` and `Intl.DisplayNames`.

The `Intl.supportedValuesOf()` function returns an array of supported values for a given option. For example, you can use it to populate a dropdown list of supported calendars, from which users can select to display dates.

```
const supportedCal = Intl.supportedValuesOf("calendar");
console.log(supportedCal);
// ['buddhist', 'chinese', 'coptic', 'dangi', ...]
```

But often, these identifiers are not user-friendly. For example, you may want to show the calendars in the user's language, or unabbreviate them. The `Intl.DisplayNames` object is useful for this. It's like a formatter, but it's not template-based. Instead, it is a direct mapping from language-agnostic identifiers to localized names. It supports formatting languages, regions, scripts (the three subfields of a BCP 47 language tag), currency, calendar, and date-time fields.

Try the demo below:

```
<select id="lang"></select>
<select id="calendar"></select>
<output id="output"></output>
```

```
const langSelect = document.getElementById("lang");
const calSelect = document.getElementById("calendar");
const fieldset = document.querySelector("fieldset");
const output = document.getElementById("output");

// A few examples
const langs = [
  "en-US",
  "zh-Hans-CN",
  "ja-JP",
  "ar-EG",
  "ru-RU",
  "es-ES",
  "fr-FR",
  "de-DE",
  "hi-IN",
  "pt-BR",
  "bn-BD",
  "he-IL",
];
const calendars = Intl.supportedValuesOf("calendar");

for (const lang of langs) {
  const option = document.createElement("option");
  option.value = lang;
  option.textContent = new Intl.DisplayNames(lang, { type: "language" }).of(
    lang,
  );
  langSelect.appendChild(option);
}

function renderCalSelect() {
  const lang = langSelect.value;
  calSelect.innerHTML = "";
  const dn = new Intl.DisplayNames(lang, { type: "calendar" });
  const preferredCalendars = new Intl.Locale(lang).getCalendars?.() ?? [
    "gregory",
```

```
  ];
  for (const cal of [
    ...preferredCalendars,
    ...calendars.filter((c) => !preferredCalendars.includes(c)),
  ]) {
    const option = document.createElement("option");
    option.value = cal;
    option.textContent = dn.of(cal);
    calSelect.appendChild(option);
  }
}

function renderFieldInputs() {
  const lang = langSelect.value;
  fieldset.querySelectorAll("label").forEach((label) => label.remove());
  const dn = new Intl.DisplayNames(lang, { type: "dateTimeField" });
  for (const field of fields) {
    const label = document.createElement("label");
    label.textContent = dn.of(field);
    const input = document.createElement("input");
    input.type = "checkbox";
    input.value = field;
    label.appendChild(input);
    fieldset.appendChild(label);
  }
}

function renderTime() {
  const lang = langSelect.value;
  const cal = calSelect.value;
  const df = new Intl.DateTimeFormat(lang, {
    calendar: cal,
    dateStyle: "full",
    timeStyle: "full",
  });
  const now = new Date();
  const dn = new Intl.DisplayNames(lang, { type: "dateTimeField" });
  output.innerHTML = "";
  for (const component of df.formatToParts(now)) {
    const text = document.createElement("span");
    text.textContent = component.value;
    if (
      ![
        "era",
        "year",
        "quarter",
        "month",
        "weekOfYear",
        "weekday",
        "day",
        "dayPeriod",
        "hour",
        "minute",
        "second",
        "timeZoneName",
      ].includes(component.type)
    ) {
      output.appendChild(text);
      continue;
    }
    const title = dn.of(component.type);
    const field = document.createElement("ruby");
    field.appendChild(text);
    const rt = document.createElement("rt");
    rt.textContent = title;
    field.appendChild(rt);
    output.appendChild(field);
  }
}

renderCalSelect();
renderTime();
langSelect.addEventListener("change", renderCalSelect);
langSelect.addEventListener("change", renderTime);
calSelect.addEventListener("change", renderTime);
setInterval(renderTime, 500);
```


- Previous
- Next

Introduction

- Previous
- Next

This chapter introduces JavaScript and discusses some of its fundamental concepts.

What you should already know

This guide assumes you have the following basic background:

- A general understanding of the Internet and the World Wide Web (WWW).
- Good working knowledge of HyperText Markup Language (HTML).
- Some programming experience. If you are new to programming, try one of the tutorials linked on the main page about JavaScript.

Where to find JavaScript information

The JavaScript documentation on MDN includes the following:

- Dynamic scripting with JavaScript provides structured JavaScript guides for beginners and introduces basic concepts of programming and the Internet.
- JavaScript Guide (this guide) provides an overview about the JavaScript language and its objects.
- JavaScript Reference provides detailed reference material for JavaScript.

What is JavaScript?

JavaScript is a cross-platform, object-oriented scripting language used to make webpages interactive (e.g., having complex animations, clickable buttons, popup menus, etc.). There are also more advanced server side versions of JavaScript such as Node.js, which allow you to add more functionality to a website than downloading files (such as realtime collaboration between multiple computers). Inside a host environment (for example, a web browser), JavaScript can be connected to the objects of its environment to provide programmatic control over them.

JavaScript contains a standard library of objects, such as `Array`, `Map`, and `Math`, and a core set of language elements such as operators, control structures, and statements. Core JavaScript can be extended for a variety of purposes by supplementing it with additional objects; for example:

- *Client-side JavaScript* extends the core language by supplying objects to control a browser and its *Document Object Model* (DOM). For example, client-side extensions allow an application to place elements on an HTML form and respond to user events such as mouse clicks, form input, and page navigation.
- *Server-side JavaScript* extends the core language by supplying objects relevant to running JavaScript on a server. For example, server-side extensions allow an application to communicate with a database, provide continuity of information from one invocation to another of the application, or perform file manipulations on a server.

This means that in the browser, JavaScript can change the way the webpage (DOM) looks. And, likewise, Node.js JavaScript on the server can respond to custom requests sent by code executed in the browser.

JavaScript and Java

JavaScript and Java are similar in some ways but fundamentally different in some others. The JavaScript language resembles Java but does not have Java's static typing and strong type checking. JavaScript follows most Java expression syntax, naming conventions and basic control-flow constructs which was the reason why it was renamed from LiveScript to JavaScript.

In contrast to Java's compile-time system of classes built by declarations, JavaScript supports a runtime system based on a small number of data types representing numeric, Boolean, and string values. JavaScript has a prototype-based object model instead of the more common class-based object model. The prototype-based model provides dynamic inheritance; that is, what is inherited can vary for individual objects. JavaScript also supports functions without any special declarative requirements. Functions can be properties of objects, executing as loosely typed methods.

JavaScript is a very free-form language compared to Java. You do not have to declare all variables, classes, and methods. You do not have to be concerned with whether methods are public, private, or protected, and you do not have to implement interfaces. Variables, parameters, and function return types are not explicitly typed.

Java is a class-based programming language designed for fast execution and type safety. Type safety means, for instance, that you can't cast a Java integer into an object reference or access private memory by corrupting the Java bytecode. Java's class-based model means that programs consist exclusively of classes and their methods. Java's class inheritance and strong typing generally require tightly coupled object hierarchies. These requirements make Java programming more complex than JavaScript programming.

In contrast, JavaScript descends in spirit from a line of smaller, dynamically typed languages such as HyperTalk and dBase. These scripting languages offer programming tools to a much wider audience because of their easier syntax, specialized built-in functionality, and minimal requirements for object creation.

JavaScript	Java
Object-oriented. No distinction between types of objects. Inheritance is through the prototype mechanism, and properties and methods can be added to any object dynamically.	Class-based. Objects are divided into classes and instances with all inheritance through the class hierarchy. Classes and instances cannot have properties or methods added dynamically.
Variable data types are not declared (dynamic typing, loosely typed).	Variable data types must be declared (static typing, strongly typed).
Cannot automatically write to hard disk.	Can automatically write to hard disk.

JavaScript and the ECMAScript specification

JavaScript is standardized at Ecma International — the European association for standardizing information and communication systems (ECMA was formerly an acronym for the European Computer Manufacturers Association) to deliver a standardized, international programming language based on JavaScript. This standardized version of JavaScript, called ECMAScript, behaves the same way in all applications that support the standard. Companies can use the open standard language to develop their implementation of JavaScript. The ECMAScript standard is documented in the ECMA-262 specification.

The ECMA-262 standard is also approved by the ISO (International Organization for Standardization) as ISO-16262. You can also find the specification on the Ecma International website. The ECMAScript specification does not describe the Document Object Model (DOM), which is standardized by the World Wide Web Consortium (W3C) and/or WHATWG (Web Hypertext Application Technology Working Group). The DOM defines the way in which HTML document objects are exposed to your script. To get a better idea about the different technologies that are used when programming with JavaScript, consult the article JavaScript technologies overview.

JavaScript documentation versus the ECMAScript specification

The ECMAScript specification is a set of requirements for implementing ECMAScript. It is useful if you want to implement standards-compliant language features in your ECMAScript implementation or engine (such as SpiderMonkey in Firefox, or V8 in Chrome).

The ECMAScript document is *not* intended to help script programmers. Use the JavaScript documentation for information when writing scripts.

The ECMAScript specification uses terminology and syntax that may be unfamiliar to a JavaScript programmer. Although the description of the language may differ in ECMAScript, the language itself remains the same. JavaScript supports all functionality outlined in the ECMAScript specification.

The JavaScript documentation describes aspects of the language that are appropriate for a JavaScript programmer.

Getting started with JavaScript

To get started with JavaScript, all you need is a modern web browser. Recent versions of Firefox, Chrome, Microsoft Edge, and Safari all support the features discussed in this guide.

A very useful tool for exploring JavaScript is the JavaScript Console (sometimes called the Web Console, or just the console): this is a tool which enables you to enter JavaScript and run it in the current page.

The screenshots here show the Firefox Web Console, but all modern browsers ship with a console that works in a similar way.

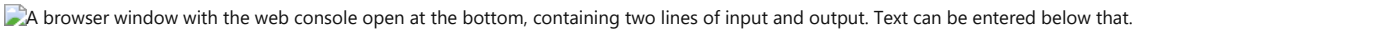
Opening the console

The exact instructions for opening the console vary from one browser to another:

- Opening the console in Firefox
- Opening the console in Chrome
- Opening the console in Microsoft Edge

Entering and running JavaScript

The console appears at the bottom of the browser window. Along the bottom of the console is an input line that you can use to enter JavaScript, and the output appears in the panel above:



The console works the exact same way as `eval`: the last expression entered is returned. For the sake of simplicity, it can be imagined that every time something is entered into the console, it is actually surrounded by `console.log` around `eval`, like so:

```
console.log(eval("3 + 5"));
```

Multi-line input in the console

By default, if you press `Enter` (or `Return`, depending on your keyboard) after entering a line of code, then the string you typed is executed. To enter multi-line input:

- If the string you typed was incomplete (for example, you typed `function foo() {`) then the console will treat `Enter` as a line break, and let you type another line.
- If you hold down `Shift` while pressing `Enter`, then the console will treat this as a line break, and let you type another line.
- In Firefox only, you can activate multi-line input mode, in which you can enter multiple lines in a mini-editor, then run the whole thing when you are ready.

To get started with writing JavaScript, open the console, copy the following code, and paste it in at the prompt:

```
(function () {
  "use strict";
  /* Start of your code */
  function greetMe(yourName) {
    alert(`Hello ${yourName}`);
  }

  greetMe("World");
  /* End of your code */
})();
```

Press `Enter` to watch it unfold in your browser!

What's next

In the following pages, this guide introduces you to the JavaScript syntax and language features, so that you will be able to write more complex applications.

But for now, remember to always include the `(function(){"use strict";` before your code, and add `})();` to the end of your code. The strict mode and IIFE articles explain what those do, but for now they can be thought of as doing the following:

1. Prevent semantics in JavaScript that trip up beginners.
2. Prevent code snippets executed in the console from interacting with one another (e.g., having something created in one console execution being used for a different console execution).

- Previous
- Next

Iterators and generators

- Previous

- Next

Iterators and Generators bring the concept of iteration directly into the core language and provide a mechanism for customizing the behavior of `for...of` loops.

For details, see also:

- Iteration protocols
- `for...of`
- `function*` and `Generator`
- `yield` and `yield*`

Iterators

In JavaScript an **iterator** is an object which defines a sequence and potentially a return value upon its termination.

Specifically, an iterator is any object which implements the Iterator protocol by having a `next()` method that returns an object with two properties:

`value`

The next value in the iteration sequence.

`done`

This is `true` if the last value in the sequence has already been consumed. If `value` is present alongside `done`, it is the iterator's return value.

Once created, an iterator object can be iterated explicitly by repeatedly calling `next()`. Iterating over an iterator is said to consume the iterator, because it is generally only possible to do once. After a terminating value has been yielded additional calls to `next()` should continue to return `{done: true}`.

The most common iterator in JavaScript is the Array iterator, which returns each value in the associated array in sequence.

While it is easy to imagine that all iterators could be expressed as arrays, this is not true. Arrays must be allocated in their entirety, but iterators are consumed only as necessary. Because of this, iterators can express sequences of unlimited size, such as the range of integers between `0` and `Infinity`.

Here is an example which can do just that. It allows creation of a range iterator which defines a sequence of integers from `start` (inclusive) to `end` (exclusive) spaced `step` apart. Its final return value is the size of the sequence it created, tracked by the variable `iterationCount`.

```
function makeRangeIterator(start = 0, end = Infinity, step = 1) {
  let nextIndex = start;
  let iterationCount = 0;

  const rangeIterator = {
    next() {
      let result;
      if (nextIndex < end) {
        result = { value: nextIndex, done: false };
        nextIndex += step;
        iterationCount++;
        return result;
      }
      return { value: iterationCount, done: true };
    },
  };
  return rangeIterator;
}
```

Using the iterator then looks like this:

```
const iter = makeRangeIterator(1, 10, 2);

let result = iter.next();
while (!result.done) {
  console.log(result.value); // 1 3 5 7 9
  result = iter.next();
}

console.log("Iterated over sequence of size:", result.value); // [5 numbers returned, that took interval in between: 0 to 10]
```

> **Note:** It is not possible to know reflectively whether a particular object is an iterator. If you need to do this, use Iterables.

Generator functions

While custom iterators are a useful tool, their creation requires careful programming due to the need to explicitly maintain their internal state. **Generator functions** provide a powerful alternative: they allow you to define an iterative algorithm by writing a single function whose execution is not continuous. Generator functions are written using the `function*` syntax.

When called, generator functions do not initially execute their code. Instead, they return a special type of iterator, called a **Generator**. When a value is consumed by calling the generator's `next` method, the Generator function executes until it encounters the `yield` keyword.

The function can be called as many times as desired, and returns a new Generator each time. Each Generator may only be iterated once.

We can now adapt the example from above. The behavior of this code is identical, but the implementation is much easier to write and read.

```
function* makeRangeIterator(start = 0, end = Infinity, step = 1) {
  let iterationCount = 0;
  for (let i = start; i < end; i += step) {
    iterationCount++;
    yield i;
  }
  return iterationCount;
}
```

Iterables

An object is **iterable** if it defines its iteration behavior, such as what values are looped over in a `for...of` construct. Some built-in types, such as `Array` or `Map`, have a default iteration behavior, while other types (such as `Object`) do not.

In order to be **iterable**, an object must implement the `[Symbol.iterator]()` method. This means that the object (or one of the objects up its prototype chain) must have a property with a `Symbol.iterator` key.

It may be possible to iterate over an iterable more than once, or only once. It is up to the programmer to know which is the case.

Iterables which can iterate only once (such as Generators) customarily return `this` from their `[Symbol.iterator]()` method, whereas iterables which can be iterated many times must return a new iterator on each invocation of `[Symbol.iterator]()`.

```
function* makeIterator() {
  yield 1;
  yield 2;
}

const iter = makeIterator();

for (const itItem of iter) {
  console.log(itItem);
}

console.log(iter[Symbol.iterator]() === iter); // true

// This example shows us generator(iterator) is an iterable object,
// which has the [Symbol.iterator]() method return the `iter` (itself),
// and consequently, the `iter` object can iterate only _once_.

// If we change the [Symbol.iterator]() method of `iter` to a function/generator
// which returns a new iterator/generator object, `iter`
// can iterate many times

iter[Symbol.iterator] = function* () {
  yield 2;
  yield 1;
};
```

User-defined iterables

You can make your own iterables like this:

```
const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};
```

User-defined iterables can be used in `for...of` loops or the spread syntax as usual.

```
for (const value of myIterable) {
  console.log(value);
}
// 1
// 2
// 3

[...myIterable]; // [1, 2, 3]
```

Built-in iterables

`String`, `Array`, `TypedArray`, `Map` and `Set` are all built-in iterables, because their prototype objects all have a `Symbol.iterator` method.

Syntaxes expecting iterables

Some statements and expressions expect iterables. For example: the `for...of` loops, spread syntax, `yield*`, and destructuring syntax.

```
for (const value of ["a", "b", "c"]) {
  console.log(value);
}
// "a"
```

```
// "b"
// "c"

[..."abc"];
// ["a", "b", "c"]

function* gen() {
  yield* ["a", "b", "c"];
}

gen().next();
// { value: "a", done: false }

[a, b, c] = new Set(["a", "b", "c"]);
a;
// "a"
```

Advanced generators

Generators compute their `yield`ed values *on demand*, which allows them to efficiently represent sequences that are expensive to compute (or even infinite sequences, as demonstrated above).

The `next()` method also accepts a value, which can be used to modify the internal state of the generator. A value passed to `next()` will be received by `yield`.

> **Note:** A value passed to the *first* invocation of `next()` is always ignored.

Here is the fibonacci generator using `next(x)` to restart the sequence:

```
function* fibonacci() {
  let current = 0;
  let next = 1;
  while (true) {
    const reset = yield current;
    [current, next] = [next, next + current];
    if (reset) {
      current = 0;
      next = 1;
    }
  }
}

const sequence = fibonacci();
console.log(sequence.next().value); // 0
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
console.log(sequence.next().value); // 3
console.log(sequence.next().value); // 5
console.log(sequence.next().value); // 8
console.log(sequence.next(true).value); // 0
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
```

You can force a generator to throw an exception by calling its `throw()` method and passing the exception value it should throw. This exception will be thrown from the current suspended context of the generator, as if the `yield` that is currently suspended were instead a `throw value` statement.

If the exception is not caught from within the generator, it will propagate up through the call to `throw()`, and subsequent calls to `next()` will result in the `done` property being `true`.

Generators have a `return()` method that returns the given value and finishes the generator itself.

- Previous
- Next

Keyed collections

- Previous
- Next

This chapter introduces collections of data which are indexed by a key; `Map` and `Set` objects contain elements which are iterable in the order of insertion.

Maps

Map object

A `Map` object is a key/value map that can iterate its elements in insertion order.

The following code shows some basic operations with a `Map`. See also the `Map` reference page for more examples and the complete API. You can use a `for...of` loop to return an array of `[key, value]` for each iteration.

```
const sayings = new Map();
sayings.set("dog", "woof");
sayings.set("cat", "meow");
sayings.set("elephant", "toot");
sayings.size; // 3
sayings.get("dog"); // woof
sayings.get("fox"); // undefined
sayings.has("bird"); // false
sayings.delete("dog");
sayings.has("dog"); // false

for (const [key, value] of sayings) {
  console.log(`${key} goes ${value}`);
}
// "cat goes meow"
// "elephant goes toot"

sayings.clear();
sayings.size; // 0
```

Object and Map compared

Traditionally, objects have been used to map strings to values. Objects allow you to set keys to values, retrieve those values, delete keys, and detect whether something is stored at a key. `Map` objects, however, have a few more advantages that make them better maps.

- The keys of an `Object` are strings or symbols, whereas they can be of any value for a `Map`.
- You can get the `size` of a `Map` easily, while you have to manually keep track of size for an `Object`.
- The iteration of maps is in insertion order of the elements.
- An `Object` has a prototype, so there are default keys in the map. (This can be bypassed using `map = Object.create(null)`.)

These three tips can help you to decide whether to use a `Map` or an `Object`:

- Use maps over objects when keys are unknown until run time, especially if the keys come from external input.
- Maps support keys and values of any type and do not require keys to be serializable to strings or symbols.
- Use objects when the shape is known ahead of time and all keys are expressible as strings.

WeakMap object

A `WeakMap` is a collection of key/value pairs whose keys must be objects or non-registered symbols, with values of any arbitrary JavaScript type, and which does not create strong references to its keys. That is, an object's presence as a key in a `WeakMap` does not prevent the object from being garbage collected. Once an object used as a key has been collected, its corresponding values in any `WeakMap` become candidates for garbage collection as well — as long as they aren't strongly referred to elsewhere. The only primitive type that can be used as a `WeakMap` key is symbol — more specifically, non-registered symbols — because non-registered symbols are guaranteed to be unique and cannot be re-created.

The `WeakMap` API is essentially the same as the `Map` API. However, a `WeakMap` doesn't allow observing the liveness of its keys, which is why it doesn't allow enumeration. So there is no method to obtain a list of the keys in a `WeakMap`. If there were, the list would depend on the state of garbage collection, introducing non-determinism.

For more information and example code, see also "Why WeakMap?" on the `WeakMap` reference page.

One use case of `WeakMap` objects is to store private data for an object, or to hide implementation details. In the following example, the private data and methods belong inside the object and are stored in the `privates` object, which is a `WeakMap`. Everything exposed on the instance and prototype is public; everything else is inaccessible from the outside world because `privates` is not exported from the module.

```
const privates = new WeakMap();

export default function Public() {
  const me = {
    // Private data goes here
  };
  privates.set(this, me);
}

Public.prototype.method = function () {
  const me = privates.get(this);
  // Do stuff with private data in `me`
  // …
};
```

> **Note:** This use case can now be implemented with classes and private fields.

Sets

Set object

`Set` objects are collections of unique values. You can iterate its elements in insertion order. A value in a `Set` may only occur once; it is unique in the `Set`'s collection.

The following code shows some basic operations with a `Set`. See also the `Set` reference page for more examples and the complete API.

```
const mySet = new Set();
mySet.add(1);
mySet.add("some text");
```

```
mySet.add("foo");

mySet.has(1); // true
mySet.delete("foo");
mySet.size; // 2

for (const item of mySet) {
  console.log(item);
}
// 1
// "some text"
```

Converting between Array and Set

You can create an `Array` from a Set using `Array.from` or the spread syntax. Also, the `Set` constructor accepts an `Array` to convert in the other direction.

> **Note:** `Set` objects store *unique values*—so any duplicate elements from an Array are deleted when converting!

```
Array.from(mySet);
[...mySet2];

mySet2 = new Set([1, 2, 3, 4]);
```

Array and Set compared

Traditionally, a set of elements has been stored in arrays in JavaScript in a lot of situations. The `Set` object, however, has some advantages:

- Deleting Array elements by value (`arr.splice(arr.indexOf(val), 1)`) is very slow.
- `Set` objects let you delete elements by their value. With an array, you would have to `splice` based on an element's index.
- The value `NaN` cannot be found with `indexOf` in an array.
- `Set` objects store unique values. You don't have to manually keep track of duplicates.

WeakSet object

`WeakSet` objects are collections of garbage-collectable values, including objects and non-registered symbols. A value in the `WeakSet` may only occur once. It is unique in the `WeakSet`'s collection.

The main differences to the `Set` object are:

- In contrast to `Sets`, `WeakSets` are **collections of *objects or symbols only***, and not of arbitrary values of any type.
- The `WeakSet` is *weak*: References to objects in the collection are held weakly. If there is no other reference to an object stored in the `WeakSet`, they can be garbage collected. That also means that there is no list of current objects stored in the collection.
- `WeakSets` are not enumerable.

The use cases of `WeakSet` objects are limited. They will not leak memory, so it can be safe to use DOM elements as a key and mark them for tracking purposes, for example.

Key and value equality of Map and Set

Both the key equality of `Map` objects and the value equality of `Set` objects are based on the SameValueZero algorithm:

- Equality works like the identity comparison operator `===`.
- `-0` and `+0` are considered equal.
- `NaN` is considered equal to itself (contrary to `===`).

- Previous
- Next

JavaScript language overview

JavaScript is a multi-paradigm, dynamic language with types and operators, standard built-in objects, and methods. Its syntax is based on the Java and C languages — many structures from those languages apply to JavaScript as well. JavaScript supports object-oriented programming with object prototypes and classes. It also supports functional programming since functions are first-class objects that can be easily created via expressions and passed around like any other object.

This page serves as a quick overview of various JavaScript language features, written for readers with background in other languages, such as C or Java.

Data types

Let's start off by looking at the building blocks of any language: the types. JavaScript programs manipulate values, and those values all belong to a type. JavaScript offers seven *primitive types*:

- Number: used for all number values (integer and floating point) except for *very* big integers.
- BigInt: used for arbitrarily large integers.
- String: used to store text.
- Boolean: `true` and `false` — usually used for conditional logic.
- Symbol: used for creating unique identifiers that won't collide.
- Undefined: indicating that a variable has not been assigned a value.
- Null: indicating a deliberate non-value.

Everything else is known as an Object. Common object types include:

- `Function`
- `Array`
- `Map`
- `RegExp`
- `Error`

Functions aren't special data structures in JavaScript — they are just a special type of object that can be called.

Numbers

JavaScript has two built-in numeric types: Number and BigInt.

The Number type is an IEEE 754 64-bit double-precision floating point value, which means integers can be safely represented between $-(2^{53} - 1)$ and $2^{53} - 1$ without loss of precision, and floating point numbers can be stored all the way up to 1.79×10^{308}. Within numbers, JavaScript does not distinguish between floating point numbers and integers.

```
console.log(3 / 2); // 1.5, not 1
```

So an *apparent integer* is in fact *implicitly a float*. Because of IEEE 754 encoding, sometimes floating point arithmetic can be imprecise.

```
console.log(0.1 + 0.2); // 0.30000000000000004
```

For operations that expect integers, such as bitwise operations, the number will be converted to a 32-bit integer.

Number literals can also have prefixes to indicate the base (binary, octal, decimal, or hexadecimal), or an exponent suffix.

```
console.log(0b111110111); // 503
console.log(0o767); // 503
console.log(0x1f7); // 503
console.log(5.03e2); // 503
```

The BigInt type is an arbitrary length integer. Its behavior is similar to C's integer types (e.g., division truncates to zero), except it can grow indefinitely. BigInts are specified with a number literal and an `n` suffix.

```
console.log(-3n / 2n); // -1n
```

The standard arithmetic operators are supported, including addition, subtraction, remainder arithmetic, etc. BigInts and numbers cannot be mixed in arithmetic operations.

The `Math` object provides standard mathematical functions and constants.

```
Math.sin(3.5);
const circumference = 2 * Math.PI * r;
```

There are three ways to convert a string to a number:

- `parseInt()`, which parses the string for an integer.
- `parseFloat()`, which parses the string for a floating-point number.
- The `Number()` function, which parses a string as if it's a number literal and supports many different number representations.

You can also use the unary plus `+` as a shorthand for `Number()`.

Number values also include `NaN` (short for "Not a Number") and `Infinity`. Many "invalid math" operations will return `NaN` — for example, if attempting to parse a non-numeric string, or using `Math.log()` on a negative value. Division by zero produces `Infinity` (positive or negative).

`NaN` is contagious: if you provide it as an operand to any mathematical operation, the result will also be `NaN`. `NaN` is the only value in JavaScript that's not equal to itself (per IEEE 754 specification).

Strings

Strings in JavaScript are sequences of Unicode characters. This should be welcome news to anyone who has had to deal with internationalization. More accurately, they are UTF-16 encoded.

```
console.log("Hello, world");
console.log("你好, 世界! "); // Nearly all Unicode characters can be written literally in string literals
```

Strings can be written with either single or double quotes — JavaScript does not have the distinction between characters and strings. If you want to represent a single character, you just use a string consisting of that single character.

```
console.log("Hello"[1] === "e"); // true
```

To find the length of a string (in code units), access its `length` property.

Strings have utility methods to manipulate the string and access information about the string. Because all primitives are immutable by design, these methods return new strings.

The `+` operator is overloaded for strings: when one of the operands is a string, it performs string concatenation instead of number addition. A special template literal syntax allows you to write strings with embedded expressions more succinctly. Unlike Python's f-strings or C#'s interpolated strings, template literals use backticks (not single or double quotes).

```
const age = 25;
console.log("I am " + age + " years old."); // String concatenation
console.log(`I am ${age} years old.`); // Template literal
```

Other types

JavaScript distinguishes between `null`, which indicates a deliberate non-value (and is only accessible through the `null` keyword), and `undefined`, which indicates absence of value. There are many ways to obtain `undefined`:

- A `return` statement with no value (`return;`) implicitly returns `undefined`.
- Accessing a nonexistent object property (`obj.iDontExist`) returns `undefined`.
- A variable declaration without initialization (`let x;`) will implicitly initialize the variable to `undefined`.

JavaScript has a Boolean type, with possible values `true` and `false` — both of which are keywords. Any value can be converted to a boolean according to the following rules:

1. `false`, `0`, empty strings (`""`), `NaN`, `null`, and `undefined` all become `false`.
2. All other values become `true`.

You can perform this conversion explicitly using the `Boolean()` function:

```
Boolean(""); // false
Boolean(234); // true
```

However, this is rarely necessary, as JavaScript will silently perform this conversion when it expects a boolean, such as in an `if` statement (see Control structures). For this reason, we sometimes speak of "truthy" and "falsy", meaning values that become `true` and `false`, respectively, when used in boolean contexts.

Boolean operations such as `&&` (logical *and*), `||` (logical *or*), and `!` (logical *not*) are supported; see Operators.

The Symbol type is often used to create unique identifiers. Every symbol created with the `Symbol()` function is guaranteed to be unique. In addition, there are registered symbols, which are shared constants, and well-known symbols, which are utilized by the language as "protocols" for certain operations. You can read more about them in the symbol reference.

Variables

Variables in JavaScript are declared using one of three keywords: `let`, `const`, or `var`.

`let` allows you to declare block-level variables. The declared variable is available from the *block* it is enclosed in.

```
let a;
let name = "Simon";

// myLetVariable is *not* visible out here

for (let myLetVariable = 0; myLetVariable < 5; myLetVariable++) {
  // myLetVariable is only visible in here
}

// myLetVariable is *not* visible out here
```

`const` allows you to declare variables whose values are never intended to change. The variable is available from the *block* it is declared in.

```
const Pi = 3.14; // Declare variable Pi
console.log(Pi); // 3.14
```

A variable declared with `const` cannot be reassigned.

```
const Pi = 3.14;
Pi = 1; // will throw an error because you cannot change a constant variable.
```

`const` declarations only prevent *reassignments* — they don't prevent *mutations* of the variable's value, if it's an object.

```
const obj = {};
obj.a = 1; // no error
console.log(obj); // { a: 1 }
```

`var` declarations can have surprising behaviors (for example, they are not block-scoped), and they are discouraged in modern JavaScript code.

If you declare a variable without assigning any value to it, its value is `undefined`. You can't declare a `const` variable without an initializer, because you can't change it later anyway.

`let` and `const` declared variables still occupy the entire scope they are defined in, and are in a region known as the temporal dead zone before the actual line of declaration. This has some interesting interactions with variable shadowing, which don't occur in other languages.

```
function foo(x, condition) {
  if (condition) {
    console.log(x);
    const x = 2;
    console.log(x);
  }
}
```

```
foo(1, true);
```

In most other languages, this would log "1" and "2", because before the `const x = 2` line, `x` should still refer to the parameter `x` in the upper scope. In JavaScript, because each declaration occupies the entire scope, this would throw an error on the first `console.log`: "Cannot access 'x' before initialization". For more information, see the reference page of `let`.

JavaScript is *dynamically typed*. Types (as described in the previous section) are only associated with values, but not with variables. For `let`-declared variables, you can always change its type through reassignment.

```
let a = 1;
a = "foo";
```

Operators

JavaScript's numeric operators include `+`, `-`, `*`, `/`, `%` (remainder), and `**` (exponentiation). Values are assigned using `=`. Each binary operator also has a compound assignment counterpart such as `+=` and `-=`, which extend out to `x = x operator y`.

```
x += 5;
x = x + 5;
```

You can use `++` and `--` to increment and decrement respectively. These can be used as a prefix or postfix operators.

The `+` operator also does string concatenation:

```
"hello" + " world"; // "hello world"
```

If you add a string to a number (or other value) everything is converted into a string first. This might trip you up:

```
"3" + 4 + 5; // "345"
3 + 4 + "5"; // "75"
```

Adding an empty string to something is a useful way of converting it to a string itself.

Comparisons in JavaScript can be made using `<`, `>`, `<=` and `>=`, which work for both strings and numbers. For equality, the double-equals operator performs type coercion if you give it different types, with sometimes interesting results. On the other hand, the triple-equals operator does not attempt type coercion, and is usually preferred.

```
123 == "123"; // true
1 == true; // true

123 === "123"; // false
1 === true; // false
```

The double-equals and triple-equals also have their inequality counterparts: `!=` and `!==`.

JavaScript also has bitwise operators and logical operators. Notably, logical operators don't work with boolean values only — they work by the "truthiness" of the value.

```
const a = 0 && "Hello"; // 0 because 0 is "falsy"
const b = "Hello" || "world"; // "Hello" because both "Hello" and "world" are "truthy"
```

The `&&` and `||` operators use short-circuit logic, which means whether they will execute their second operand is dependent on the first. This is useful for checking for null objects before accessing their attributes:

```
const name = o && o.getName();
```

Or for caching values (when falsy values are invalid):

```
const name = cachedName || (cachedName = getName());
```

For a comprehensive list of operators, see the guide page or reference section. You may be especially interested in the operator precedence.

Grammar

JavaScript grammar is very similar to the C family. There are a few points worth mentioning:

- Identifiers can have Unicode characters, but they cannot be one of the reserved words.
- Comments are commonly `//` or `/* */`, while many other scripting languages like Perl, Python, and Bash use `#`.
- Semicolons are optional in JavaScript — the language automatically inserts them when needed. However, there are certain caveats to watch out for, since unlike Python, semicolons are still part of the syntax.

For an in-depth look at the JavaScript grammar, see the reference page for lexical grammar.

Control structures

JavaScript has a similar set of control structures to other languages in the C family. Conditional statements are supported by `if` and `else`; you can chain them together:

```
let name = "kittens";
if (name === "puppies") {
  name += " woof";
} else if (name === "kittens") {
  name += " meow";
} else {
  name += "!";
}
name === "kittens meow";
```

JavaScript doesn't have `elif`, and `else if` is really just an `else` branch comprised of a single `if` statement.

JavaScript has `while` loops and `do...while` loops. The first is good for basic looping; the second is for loops where you wish to ensure that the body of the loop is executed at least once:

```
while (true) {
  // an infinite loop!
}

let input;
do {
  input = get_input();
} while (inputIsNotValid(input));
```

JavaScript's `for` loop is the same as that in C and Java: it lets you provide the control information for your loop on a single line.

```
for (let i = 0; i < 5; i++) {
  // Will execute 5 times
}
```

JavaScript also contains two other prominent for loops: `for...of`, which iterates over iterables, most notably arrays, and `for...in`, which visits all enumerable properties of an object.

```
for (const value of array) {
  // do something with value
}

for (const property in object) {
  // do something with object property
}
```

The `switch` statement can be used for multiple branches based on equality checking.

```
switch (action) {
  case "draw":
    drawIt();
    break;
  case "eat":
    eatIt();
    break;
  default:
    doNothing();
}
```

Similar to C, case clauses are conceptually the same as labels, so if you don't add a `break` statement, execution will "fall through" to the next level. However, they are not actually jump tables — any expression can be part of the `case` clause, not just string or number literals, and they would be evaluated one-by-one until one equals the value being matched. Comparison takes place between the two using the `===` operator.

Unlike some languages like Rust, control-flow structures are statements in JavaScript, meaning you can't assign them to a variable, like `const a = if (x) { 1 } else { 2 }`.

JavaScript errors are handled using the `try...catch` statement.

```
try {
  buildMySite("./website");
} catch (e) {
  console.error("Building site failed:", e);
}
```

Errors can be thrown using the `throw` statement. Many built-in operations may throw as well.

```
function buildMySite(siteDirectory) {
  if (!pathExists(siteDirectory)) {
    throw new Error("Site directory does not exist");
  }
}
```

In general, you can't tell the type of the error you just caught, because anything can be thrown from a `throw` statement. However, you can usually assume it's an `Error` instance, as is the example above. There are some subclasses of `Error` built-in, like `TypeError` and `RangeError`, that you can use to provide extra semantics about the error. There's no conditional catch in JavaScript — if you only want to handle one type of error, you need to catch everything, identify the type of error using `instanceof`, and then rethrow the other cases.

```
try {
  buildMySite("./website");
} catch (e) {
  if (e instanceof RangeError) {
    console.error("Seems like a parameter is out of range:", e);
    console.log("Retrying...");
    buildMySite("./website");
  } else {
    // Don't know how to handle other error types; throw them so
    // something else up in the call stack may catch and handle it
    throw e;
  }
}
```

If an error is uncaught by any `try...catch` in the call stack, the program will exit.

For a comprehensive list of control flow statements, see the reference section.

Objects

JavaScript objects can be thought of as collections of key-value pairs. As such, they are similar to:

- Dictionaries in Python.
- Hashes in Perl and Ruby.
- Hash tables in C and C++.
- HashMaps in Java.
- Associative arrays in PHP.

JavaScript objects are hashes. Unlike objects in statically typed languages, objects in JavaScript do not have fixed shapes — properties can be added, deleted, re-ordered, mutated, or dynamically queried at any time. Object keys are always strings or symbols — even array indices, which are canonically integers, are actually strings under the hood.

Objects are usually created using the literal syntax:

```
const obj = {
  name: "Carrot",
  for: "Max",
  details: {
    color: "orange",
    size: 12,
  },
};
```

Object properties can be accessed using dot (`.`) or square brackets (`[]`). When using the dot notation, the key must be a valid identifier. Square brackets, on the other hand, allow indexing the object with a dynamic key value.

```
// Dot notation
obj.name = "Simon";
const name = obj.name;

// Bracket notation
obj["name"] = "Simon";
const name = obj["name"];

// Can use a variable to define a key
const userName = prompt("what is your key?");
obj[userName] = prompt("what is its value?");
```

Property access can be chained together:

```
obj.details.color; // orange
obj["details"]["size"]; // 12
```

Objects are always references, so unless something is explicitly copying the object, mutations to an object would be visible to the outside.

```
const obj = {};
function doSomething(o) {
  o.x = 1;
}
doSomething(obj);
console.log(obj.x); // 1
```

This also means two separately created objects will never be equal (`!==`), because they are different references. If you hold two references of the same object, mutating one would be observable through the other.

```
const me = {};
const stillMe = me;
me.x = 1;
console.log(stillMe.x); // 1
```

For more on objects and prototypes, see the `Object` reference page. For more information on the object initializer syntax, see its reference page.

This page has omitted all details about object prototypes and inheritance because you can usually achieve inheritance with classes without touching the underlying mechanism (which you may have heard to be abstruse). To learn about them, see Inheritance and the prototype chain.

Arrays

Arrays in JavaScript are actually a special type of object. They work very much like regular objects (numerical properties can naturally be accessed only using `[]` syntax) but they have one magic property called `length`. This is always one more than the highest index in the array.

Arrays are usually created with array literals:

```
const a = ["dog", "cat", "hen"];
a.length; // 3
```

JavaScript arrays are still objects — you can assign any properties to them, including arbitrary number indices. The only "magic" is that `length` will be automatically updated when you set a particular index.

```
const a = ["dog", "cat", "hen"];
a[100] = "fox";
console.log(a.length); // 101
console.log(a); // ['dog', 'cat', 'hen', empty × 97, 'fox']
```

The array we got above is called a *sparse array* because there are uninhabited slots in the middle, and will cause the engine to deoptimize it from an array to a hash table. Make sure your array is densely populated!

Out-of-bounds indexing doesn't throw. If you query a non-existent array index, you'll get a value of `undefined` in return:

```
const a = ["dog", "cat", "hen"];
console.log(typeof a[90]); // undefined
```

Arrays can have any elements and can grow or shrink arbitrarily.

```
const arr = [1, "foo", true];
arr.push({});
// arr = [1, "foo", true, {}]
```

Arrays can be iterated with the `for` loop, as you can in other C-like languages:

```
for (let i = 0; i < a.length; i++) {
  // Do something with a[i]
}
```

Or, since arrays are iterable, you can use the `for...of` loop, which is synonymous to C++/Java's `for (int x : arr)` syntax:

```
for (const currentValue of a) {
  // Do something with currentValue
}
```

Arrays come with a plethora of array methods. Many of them would iterate the array — for example, `map()` would apply a callback to every array element, and return a new array:

```
const babies = ["dog", "cat", "hen"].map((name) => `baby ${name}`);
// babies = ['baby dog', 'baby cat', 'baby hen']
```

Functions

Along with objects, functions are the core component in understanding JavaScript. The most basic function declaration looks like this:

```
function add(x, y) {
  const total = x + y;
  return total;
}
```

A JavaScript function can take 0 or more parameters. The function body can contain as many statements as you like and can declare its own variables which are local to that function. The `return` statement can be used to return a value at any time, terminating the function. If no return statement is used (or an empty return with no value), JavaScript returns `undefined`.

Functions can be called with more or fewer parameters than it specifies. If you call a function without passing the parameters it expects, they will be set to `undefined`. If you pass more parameters than it expects, the function will ignore the extra parameters.

```
add(); // NaN
// Equivalent to add(undefined, undefined)

add(2, 3, 4); // 5
// added the first two; 4 was ignored
```

There are a number of other parameter syntaxes available. For example, the rest parameter syntax allows collecting all the extra parameters passed by the caller into an array, similar to Python's `*args`. (Since JS doesn't have named parameters on the language level, there's no `**kwargs`.)

```
function avg(...args) {
  let sum = 0;
  for (const item of args) {
    sum += item;
  }
  return sum / args.length;
}

avg(2, 3, 4, 5); // 3.5
```

In the above code, the variable `args` holds all the values that were passed into the function.

The rest parameter will store all arguments *after* where it's declared, but not before. In other words, `function avg(firstValue, ...args)` will store the first value passed into the function in the `firstValue` variable and the remaining arguments in `args`.

If a function accepts a list of arguments and you already hold them in an array, you can use the spread syntax in the function call to *spread* the array as a list of elements. For instance: `avg(...numbers)`.

We mentioned that JavaScript doesn't have named parameters. It's possible, though, to implement them using object destructuring, which allows objects to be conveniently packed and unpacked.

```
// Note the { } braces: this is destructuring an object
function area({ width, height }) {
  return width * height;
}

// The { } braces here create a new object
console.log(area({ width: 2, height: 3 }));
```

There's also the *default parameter* syntax, which allows omitted parameters (or those passed as `undefined`) to have a default value.

```
function avg(firstValue, secondValue, thirdValue = 0) {
  return (firstValue + secondValue + thirdValue) / 3;
}

avg(1, 2); // 1, instead of NaN
```

Anonymous functions

JavaScript lets you create anonymous functions — that is, functions without names. In practice, anonymous functions are typically used as arguments to other functions, immediately assigned to a variable that can be used to invoke the function, or returned from another function.

```
// Note that there's no function name before the parentheses
const avg = function (...args) {
  let sum = 0;
  for (const item of args) {
    sum += item;
  }
  return sum / args.length;
};
```

That makes the anonymous function invocable by calling `avg()` with some arguments — that is, it's semantically equivalent to declaring the function using the `function avg() {}` declaration syntax.

There's another way to define anonymous functions — using an arrow function expression.

```
// Note that there's no function name before the parentheses
const avg = (...args) => {
  let sum = 0;
  for (const item of args) {
    sum += item;
  }
  return sum / args.length;
};

// You can omit the `return` when simply returning an expression
const sum = (a, b, c) => a + b + c;
```

Arrow functions are not semantically equivalent to function expressions — for more information, see its reference page.

There's another way that anonymous functions can be useful: it can be simultaneously declared and invoked in a single expression, called an Immediately invoked function expression (IIFE):

```
(function () {
  // …
})();
```

For use-cases of IIFEs, you can read emulating private methods with closures.

Recursive functions

JavaScript allows you to call functions recursively. This is particularly useful for dealing with tree structures, such as those found in the browser DOM.

```
function countChars(elm) {
  if (elm.nodeType === 3) {
    // TEXT_NODE
    return elm.nodeValue.length;
  }
  let count = 0;
  for (let i = 0, child; (child = elm.childNodes[i]); i++) {
    count += countChars(child);
  }
  return count;
}
```

Function expressions can be named as well, which allows them to be recursive.

```
const charsInBody = (function counter(elm) {
  if (elm.nodeType === 3) {
    // TEXT_NODE
    return elm.nodeValue.length;
  }
  let count = 0;
  for (let i = 0, child; (child = elm.childNodes[i]); i++) {
    count += counter(child);
  }
  return count;
})(document.body);
```

The name provided to a function expression as above is only available to the function's own scope. This allows more optimizations to be done by the engine and results in more readable code. The name also shows up in the debugger and some stack traces, which can save you time when debugging.

If you are used to functional programming, beware of the performance implications of recursion in JavaScript. Although the language specification specifies tail-call optimization, only JavaScriptCore (used by Safari) has implemented it, due to the difficulty of recovering stack traces and debuggability. For deep recursion, consider using iteration instead to avoid stack overflow.

Functions are first-class objects

JavaScript functions are first-class objects. This means that they can be assigned to variables, passed as arguments to other functions, and returned from other functions. In addition, JavaScript supports closures out-of-the-box without explicit capturing, allowing you to conveniently apply functional programming styles.

```
// Function returning function
const add = (x) => (y) => x + y;
// Function accepting function
const babies = ["dog", "cat", "hen"].map((name) => `baby ${name}`);
```

Note that JavaScript functions are themselves objects — like everything else in JavaScript — and you can add or change properties on them just like we've seen earlier in the Objects section.

Inner functions

JavaScript function declarations are allowed inside other functions. An important detail of nested functions in JavaScript is that they can access variables in their parent function's scope:

```
function parentFunc() {
  const a = 1;

  function nestedFunc() {
    const b = 4; // parentFunc can't use this
    return a + b;
  }
  return nestedFunc(); // 5
}
```

This provides a great deal of utility in writing more maintainable code. If a called function relies on one or two other functions that are not useful to any other part of your code, you can nest those utility functions inside it. This keeps the number of functions that are in the global scope down.

This is also a great counter to the lure of global variables. When writing complex code, it is often tempting to use global variables to share values between multiple functions, which leads to code that is hard to maintain. Nested functions can share variables in their parent, so you can use that mechanism to couple functions together without polluting your global namespace.

Classes

JavaScript offers the class syntax that's very similar to languages like Java.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  sayHello() {
    return `Hello, I'm ${this.name}!`;
  }
}
```

```
const p = new Person("Maria");
console.log(p.sayHello());
```

JavaScript classes are just functions that must be instantiated with the `new` operator. Every time a class is instantiated, it returns an object containing the methods and properties that the class specified. Classes don't enforce any code organization — for example, you can have functions returning classes, or you can have multiple classes per file. Here's an example of how ad hoc the creation of a class can be: it's just an expression returned from an arrow function. This pattern is called a mixin.

```
const withAuthentication = (cls) =>
  class extends cls {
    authenticate() {
      // …
    }
  };

class Admin extends withAuthentication(Person) {
  // …
}
```

Static properties are created by prepending `static`. Private fields and methods are created by prepending a hash `#` (not `private`). The hash is an integral part of the element's name, and distinguishes it from a regular string-keyed property. (Think about `#` as _ in Python.) Unlike most other languages, there's absolutely no way to read a private element outside the class body — not even in derived classes.

For a detailed guide on various class features, you can read the guide page.

Asynchronous programming

JavaScript is single-threaded by nature. There's no paralleling; only concurrency. Asynchronous programming is powered by an event loop, which allows a set of tasks to be queued and polled for completion.

There are three idiomatic ways to write asynchronous code in JavaScript:

- Callback-based (such as `setTimeout()`)
- `Promise`-based
- `async`/`await`, which is a syntactic sugar for Promises

For example, here's what a file-read operation might look like in JavaScript:

```
// Callback-based
fs.readFile(filename, (err, content) => {
  // This callback is invoked when the file is read, which could be after a while
  if (err) {
    throw err;
  }
  console.log(content);
});
// Code here will be executed while the file is waiting to be read

// Promise-based
fs.readFile(filename)
  .then((content) => {
    // What to do when the file is read
    console.log(content);
  })
  .catch((err) => {
    throw err;
  });
// Code here will be executed while the file is waiting to be read

// Async/await
async function readFile(filename) {
  const content = await fs.readFile(filename);
  console.log(content);
}
```

The core language doesn't specify any asynchronous programming features, but it's crucial when interacting with the external environment — from asking user permissions, to fetching data, to reading files. Keeping the potentially long-running operations async ensures that other processes can still run while this one waits — for example, the browser will not freeze while waiting for the user to click a button to grant permission.

If you have an async value, it's not possible to get its value synchronously. For example, if you have a promise, you can only access the eventual result via the `then()` method. Similarly, `await` can only be used in an async context, which is usually an async function or a module. Promises are *never blocking* — only the logic depending on the promise's result will be deferred; everything else continues to execute in the meantime. If you are a functional programmer, you may recognize promises as monads which can be mapped with `then()` (however, they are not *proper* monads because they auto-flatten; i.e., you can't have a `Promise<Promise<T>>`).

In fact, the single-threaded model has made Node.js a popular choice for server-side programming due to its non-blocking IO, making handling a large number of database or file-system requests very performant. However, CPU-bound (computationally intensive) tasks that are pure JavaScript will still block the main thread. To achieve real paralleling, you may need to use workers.

To learn more about asynchronous programming, you can read about using promises or follow the asynchronous JavaScript tutorial.

Modules

JavaScript also specifies a module system supported by most runtimes. A module is usually a file, identified by its file path or URL. You can use the `import` and `export` statements to exchange data between modules:

```
import { foo } from "./foo.js";

// Unexported variables are local to the module
const b = 2;

export const a = 1;
```

Unlike Haskell, Python, Java, etc., JavaScript module resolution is entirely host-defined — it's usually based on URLs or file paths, so relative file paths "just work" and are relative to the current module's path instead of some project root path.

However, the JavaScript language doesn't offer standard library modules — all core functionalities are powered by global variables like `Math` and `Intl` instead. This is due to the long history of JavaScript lacking a module system, and the fact that opting into the module system involves some changes to the runtime setup.

Different runtimes may use different module systems. For example, Node.js uses the package manager npm and is mostly file-system based, while Deno and browsers are fully URL-based and modules can be resolved from HTTP URLs.

For more information, see the modules guide page.

Language and runtime

Throughout this page, we've constantly mentioned that certain features are *language-level* while others are *runtime-level*.

JavaScript is a general-purpose scripting language. The core language specification focuses on pure computational logic. It doesn't deal with any input/output — in fact, without extra runtime-level APIs (most notably `console.log()`), a JavaScript program's behavior is entirely unobservable.

A runtime, or a host, is something that feeds data to the JavaScript engine (the interpreter), provides extra global properties, and provides hooks for the engine to interact with the outside world. Module resolution, reading data, printing messages, sending network requests, etc. are all runtime-level operations. Since its inception, JavaScript has been adopted in various environments, such as browsers (which provide APIs like DOM), Node.js (which provides APIs like file system access), etc. JavaScript has been successfully integrated in web (which was its primary purpose), mobile apps, desktop apps, server-side apps, serverless, embedded systems, and more. While you learn about JavaScript core features, it's also important to understand host-provided features in order to put the knowledge to use. For example, you can read about all web platform APIs, which are implemented by browsers, and sometimes non-browsers.

Further exploration

This page offers a very basic insight into how various JavaScript features compare with other languages. If you want to learn more about the language itself and the nuances of each feature, you can read the JavaScript guide and the JavaScript reference.

There are some essential parts of the language that we have omitted due to space and complexity, but you can explore on your own:

- Inheritance and the prototype chain
- Closures
- Regular expressions
- Iteration

Loops and iteration

- Previous
- Next

Loops offer a quick and easy way to do something repeatedly. This chapter of the JavaScript Guide introduces the different iteration statements available to JavaScript.

You can think of a loop as a computerized version of the game where you tell someone to take *X* steps in one direction, then *Y* steps in another. For example, the idea "Go five steps to the east" could be expressed this way as a loop:

```
for (let step = 0; step < 5; step++) {
  // Runs 5 times, with values of step 0 through 4.
  console.log("Walking east one step");
}
```

There are many different kinds of loops, but they all essentially do the same thing: they repeat an action some number of times. (Note that it's possible that number could be zero!)

The various loop mechanisms offer different ways to determine the start and end points of the loop. There are various situations that are more easily served by one type of loop over the others.

The statements for loops provided in JavaScript are:

- for statement
- do...while statement
- while statement
- labeled statement
- break statement
- continue statement
- for...in statement
- for...of statement

for statement

A `for` loop repeats until a specified condition evaluates to false. The JavaScript `for` loop is similar to the Java and C `for` loop.

A `for` statement looks as follows:

```
for (initialization; condition; afterthought)
  statement
```

When a `for` loop executes, the following occurs:

1. The initializing expression `initialization`, if any, is executed. This expression usually initializes one or more loop counters, but the syntax allows an expression of any degree of complexity. This expression can also declare variables.
2. The `condition` expression is evaluated. If the value of `condition` is true, the loop statements execute. Otherwise, the `for` loop terminates. (If the `condition` expression is omitted entirely, the condition is assumed to be true.)
3. The `statement` executes. To execute multiple statements, use a block statement (`{ }`) to group those statements.
4. If present, the update expression `afterthought` is executed.
5. Control returns to Step 2.

Example

In the example below, the function contains a `for` statement that counts the number of selected options in a scrolling list (a `<select>` element that allows multiple selections).

HTML

```
<form name="selectForm">
  <label for="musicTypes"
    >Choose some music types, then click the button below:</label
  >
  <select id="musicTypes" name="musicTypes" multiple>
    <option selected>R&amp;B</option>
    <option>Jazz</option>
    <option>Blues</option>
    <option>New Age</option>
    <option>Classical</option>
    <option>Opera</option>
  </select>
  <button id="btn" type="button">How many are selected?</button>
</form>
```

JavaScript

Here, the `for` statement declares the variable `i` and initializes it to `0`. It checks that `i` is less than the number of options in the `<select>` element, performs the succeeding `if` statement, and increments `i` by 1 after each pass through the loop.

```
function countSelected(selectObject) {
  let numberSelected = 0;
  for (let i = 0; i < selectObject.options.length; i++) {
    if (selectObject.options[i].selected) {
      numberSelected++;
    }
  }
  return numberSelected;
}

const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
  const musicTypes = document.selectForm.musicTypes;
  console.log(`You have selected ${countSelected(musicTypes)} option(s).`);
});
```

do...while statement

The `do...while` statement repeats until a specified condition evaluates to false.

A `do...while` statement looks as follows:

```
do
  statement
while (condition);
```

`statement` is always executed once before the condition is checked. (To execute multiple statements, use a block statement (`{ }`) to group those statements.)

If `condition` is `true`, the statement executes again. At the end of every execution, the condition is checked. When the condition is `false`, execution stops, and control passes to the statement following `do...while`.

Example

In the following example, the `do` loop iterates at least once and reiterates until `i` is no longer less than `5`.

```
let i = 0;
do {
  i += 1;
  console.log(i);
} while (i < 5);
```

while statement

A `while` statement executes its statements as long as a specified condition evaluates to `true`. A `while` statement looks as follows:

```
while (condition)
  statement
```

If the `condition` becomes `false`, `statement` within the loop stops executing and control passes to the statement following the loop.

The condition test occurs *before* `statement` in the loop is executed. If the condition returns `true`, `statement` is executed and the `condition` is tested again. If the condition returns `false`, execution stops, and control is passed to the statement following `while`.

To execute multiple statements, use a block statement (`{ }`) to group those statements.

Example 1

The following `while` loop iterates as long as `n` is less than `3`:

```
let n = 0;
let x = 0;
while (n < 3) {
  n++;
  x += n;
}
```

With each iteration, the loop increments `n` and adds that value to `x`. Therefore, `x` and `n` take on the following values:

- After the first pass: `n = 1` and `x = 1`
- After the second pass: `n = 2` and `x = 3`
- After the third pass: `n = 3` and `x = 6`

After completing the third pass, the condition `n < 3` is no longer `true`, so the loop terminates.

Example 2

Avoid infinite loops. Make sure the condition in a loop eventually becomes `false`—otherwise, the loop will never terminate! The statements in the following `while` loop execute forever because the condition never becomes `false`:

```
// Infinite loops are bad!
while (true) {
  console.log("Hello, world!");
}
```

labeled statement

A `label` provides a statement with an identifier that lets you refer to it elsewhere in your program. For example, you can use a label to identify a loop, and then use the `break` or `continue` statements to indicate whether a program should interrupt the loop or continue its execution.

The syntax of the labeled statement looks like the following:

```
label:
  statement
```

The value of `label` may be any JavaScript identifier that is not a reserved word. The `statement` that you identify with a label may be any statement. For examples of using labeled statements, see the examples of `break` and `continue` below.

break statement

Use the `break` statement to terminate a loop, `switch`, or in conjunction with a labeled statement.

- When you use `break` without a label, it terminates the innermost enclosing `while`, `do-while`, `for`, or `switch` immediately and transfers control to the following statement.
- When you use `break` with a label, it terminates the specified labeled statement.

The syntax of the `break` statement looks like this:

```
break;
break label;
```

1. The first form of the syntax terminates the innermost enclosing loop or `switch`.
2. The second form of the syntax terminates the specified enclosing labeled statement.

Example 1

The following example iterates through the elements in an array until it finds the index of an element whose value is `theValue`:

```
for (let i = 0; i < a.length; i++) {
  if (a[i] === theValue) {
    break;
  }
}
```

Example 2: Breaking to a label

```
let x = 0;
let z = 0;
labelCancelLoops: while (true) {
  console.log("Outer loops:", x);
  x += 1;
  z = 1;
  while (true) {
    console.log("Inner loops:", z);
    z += 1;
    if (z === 10 && x === 10) {
      break labelCancelLoops;
    } else if (z === 10) {
      break;
    }
  }
}
```

continue statement

The `continue` statement can be used to restart a `while`, `do-while`, `for`, or `label` statement.

- When you use `continue` without a label, it terminates the current iteration of the innermost enclosing `while`, `do-while`, or `for` statement and continues execution of the loop with the next iteration. In contrast to the `break` statement, `continue` does not terminate the execution of the loop entirely. In a `while` loop, it jumps back to the condition. In a `for` loop, it jumps to the `increment-expression`.
- When you use `continue` with a label, it applies to the looping statement identified with that label.

The syntax of the `continue` statement looks like the following:

```
continue;
continue label;
```

Example 1

The following example shows a `while` loop with a `continue` statement that executes when the value of `i` is `3`. Thus, `n` takes on the values `1`, `3`, `7`, and `12`.

```
let i = 0;
let n = 0;
while (i < 5) {
  i++;
  if (i === 3) {
    continue;
  }
  n += i;
  console.log(n);
}
// Logs:
// 1 3 7 12
```

If you comment out the `continue;`, the loop would run till the end and you would see `1,3,6,10,15`.

Example 2

A statement labeled `checkIandJ` contains a statement labeled `checkJ`. If `continue` is encountered, the program terminates the current iteration of `checkJ` and begins the next iteration. Each time `continue` is encountered, `checkJ` reiterates until its condition returns `false`. When `false` is returned, the remainder of the `checkIandJ` statement is completed, and `checkIandJ` reiterates until its condition returns `false`. When `false` is returned, the program continues at the statement following `checkIandJ`.

If `continue` had a label of `checkIandJ`, the program would continue at the top of the `checkIandJ` statement.

```
let i = 0;
let j = 10;
checkIandJ: while (i < 4) {
  console.log(i);
  i += 1;
  checkJ: while (j > 4) {
    console.log(j);
    j -= 1;
    if (j % 2 === 0) {
      continue;
    }
    console.log(j, "is odd.");
  }
  console.log("i =", i);
  console.log("j =", j);
}
```

for...in statement

The `for...in` statement iterates a specified variable over all the enumerable properties of an object. For each distinct property, JavaScript executes the specified statements. A `for...in` statement looks as follows:

```
for (variable in object)
  statement
```

Example

The following function takes as its argument an object and the object's name. It then iterates over all the object's properties and returns a string that lists the property names and their values.

```
function dumpProps(obj, objName) {
  let result = "";
  for (const i in obj) {
    result += `${objName}.${i} = ${obj[i]}<br>`;
  }
  result += "<hr>";
  return result;
}
```

For an object `car` with properties `make` and `model`, `result` would be:

```
car.make = Ford
car.model = Mustang
```

Arrays

Although it may be tempting to use this as a way to iterate over `Array` elements, the `for...in` statement will return the name of your user-defined properties in addition to the numeric indexes.

Therefore, it is better to use a traditional `for` loop with a numeric index when iterating over arrays, because the `for...in` statement iterates over user-defined properties in addition to the array elements, if you modify the Array object (such as adding custom properties or methods).

for...of statement

The `for...of` statement creates a loop Iterating over iterable objects (including `Array`, `Map`, `Set`, `arguments` object and so on), invoking a custom iteration hook with statements to be executed for the value of each distinct property.

```
for (variable of iterable)
  statement
```

The following example shows the difference between a `for...of` loop and a `for...in` loop. While `for...in` iterates over property names, `for...of` iterates over property values:

```
const arr = [3, 5, 7];
arr.foo = "hello";

for (const i in arr) {
  console.log(i);
}
// "0" "1" "2" "foo"

for (const i of arr) {
  console.log(i);
}
// Logs: 3 5 7
```

The `for...of` and `for...in` statements can also be used with destructuring. For example, you can simultaneously loop over the keys and values of an object using `Object.entries()`.

```
const obj = { foo: 1, bar: 2 };

for (const [key, val] of Object.entries(obj)) {
  console.log(key, val);
}
// "foo" 1
// "bar" 2
```


- Previous
- Next

Memory management

Low-level languages like C, have manual memory management primitives such as `malloc()` and `free()`. In contrast, JavaScript automatically allocates memory when objects are created and frees it when they are not used anymore (*garbage collection*). This automaticity is a potential source of confusion: it can give developers the false impression that they don't need to worry about memory management.

Memory life cycle

Regardless of the programming language, the memory life cycle is pretty much always the same:

1. Allocate the memory you need
2. Use the allocated memory (read, write)

3. Release the allocated memory when it is not needed anymore

The second part is explicit in all languages. The first and last parts are explicit in low-level languages but are mostly implicit in high-level languages like JavaScript.

Allocation in JavaScript

Value initialization

In order to not bother the programmer with allocations, JavaScript will automatically allocate memory when values are initially declared.

```
const n = 123; // allocates memory for a number
const s = "string"; // allocates memory for a string

const o = {
  a: 1,
  b: null,
}; // allocates memory for an object and contained values

// (like object) allocates memory for the array and
// contained values
const a = [1, null, "str2"];

function f(a) {
  return a + 2;
} // allocates a function (which is a callable object)

// function expressions also allocate an object
someElement.addEventListener("click", () => {
  someElement.style.backgroundColor = "blue";
});
```

Allocation via function calls

Some function calls result in object allocation.

```
const d = new Date(); // allocates a Date object

const e = document.createElement("div"); // allocates a DOM element
```

Some methods allocate new values or objects:

```
const s = "string";
const s2 = s.substring(0, 3); // s2 is a new string
// Since strings are immutable values,
// JavaScript may decide to not allocate memory,
// but just store the [0, 3] range.

const a = ["yeah yeah", "no no"];
const a2 = ["generation", "no no"];
const a3 = a.concat(a2);
// new array with 4 elements being
// the concatenation of a and a2 elements.
```

Using values

Using values basically means reading and writing in allocated memory. This can be done by reading or writing the value of a variable or an object property or even passing an argument to a function.

Release when the memory is not needed anymore

The majority of memory management issues occur at this phase. The most difficult aspect of this stage is determining when the allocated memory is no longer needed.

Low-level languages require the developer to manually determine at which point in the program the allocated memory is no longer needed and to release it.

Some high-level languages, such as JavaScript, utilize a form of automatic memory management known as garbage collection (GC). The purpose of a garbage collector is to monitor memory allocation and determine when a block of allocated memory is no longer needed and reclaim it. This automatic process is an approximation since the general problem of determining whether or not a specific piece of memory is still needed is undecidable.

Garbage collection

As stated above, the general problem of automatically finding whether some memory "is not needed anymore" is undecidable. As a consequence, garbage collectors implement a restriction of a solution to the general problem. This section will explain the concepts that are necessary for understanding the main garbage collection algorithms and their respective limitations.

References

The main concept that garbage collection algorithms rely on is the concept of *reference*. Within the context of memory management, an object is said to reference another object if the former has access to the latter (either implicitly or explicitly). For instance, a JavaScript object has a reference to its prototype (implicit reference) and to its properties values (explicit reference).

In this context, the notion of an "object" is extended to something broader than regular JavaScript objects and also contain function scopes (or the global lexical scope).

Reference-counting garbage collection

> **Note:** No modern JavaScript engine uses reference-counting for garbage collection anymore.

This is the most naïve garbage collection algorithm. This algorithm reduces the problem from determining whether or not an object is still needed to determining if an object still has any other objects referencing it. An object is said to be "garbage", or collectible if there are zero references pointing to it.

For example:

```
let x = {
  a: {
    b: 2,
  },
};
// 2 objects are created. One is referenced by the other as one of its properties.
// The other is referenced by virtue of being assigned to the 'x' variable.
// Obviously, none can be garbage-collected.

let y = x;
// The 'y' variable is the second thing that has a reference to the object.

x = 1;
// Now, the object that was originally in 'x' has a unique reference
// embodied by the 'y' variable.

let z = y.a;
// Reference to 'a' property of the object.
// This object now has 2 references: one as a property,
// the other as the 'z' variable.

y = "mozilla";
// The object that was originally in 'x' has now zero
// references to it. It can be garbage-collected.
// However its 'a' property is still referenced by
// the 'z' variable, so it cannot be freed.

z = null;
// The 'a' property of the object originally in x
// has zero references to it. It can be garbage collected.
```

There is a limitation when it comes to circular references. In the following example, two objects are created with properties that reference one another, thus creating a cycle. They will go out of scope after the function call has completed. At that point they become unneeded and their allocated memory should be reclaimed. However, the reference-counting algorithm will not consider them reclaimable since each of the two objects has at least one reference pointing to them, resulting in neither of them being marked for garbage collection. Circular references are a common cause of memory leaks.

```
function f() {
  const x = {};
  const y = {};
  x.a = y; // x references y
  y.a = x; // y references x

  return "azerty";
}

f();
```

Mark-and-sweep algorithm

This algorithm reduces the definition of "an object is no longer needed" to "an object is unreachable".

This algorithm assumes the knowledge of a set of objects called *roots*. In JavaScript, the root is the global object. Periodically, the garbage collector will start from these roots, find all objects that are referenced from these roots, then all objects referenced from these, etc. Starting from the roots, the garbage collector will thus find all *reachable* objects and collect all non-reachable objects.

This algorithm is an improvement over the previous one since an object having zero references is effectively unreachable. The opposite does not hold true as we have seen with circular references.

Currently, all modern engines ship a mark-and-sweep garbage collector. All improvements made in the field of JavaScript garbage collection (generational/incremental/concurrent/parallel garbage collection) over the last few years are implementation improvements of this algorithm, but not improvements over the garbage collection algorithm itself nor its reduction of the definition of when "an object is no longer needed".

The immediate benefit of this approach is that cycles are no longer a problem. In the first example above, after the function call returns, the two objects are no longer referenced by any resource that is reachable from the global object. Consequently, they will be found unreachable by the garbage collector and have their allocated memory reclaimed.

However, the inability to manually control garbage collection remains. There are times when it would be convenient to manually decide when and what memory is released. In order to release the memory of an object, it needs to be made explicitly unreachable. It is also not possible to programmatically trigger garbage collection in JavaScript — and will likely never be within the core language, although engines may expose APIs behind opt-in flags.

Configuring an engine's memory model

JavaScript engines typically offer flags that expose the memory model. For example, Node.js offers additional options and tools that expose the underlying V8 mechanisms for configuring and debugging memory issues. This configuration may not be available in browsers, and even less so for web pages (via HTTP headers, etc.).

The max amount of available heap memory can be increased with a flag:

```
node --max-old-space-size=6000 index.js
```

We can also expose the garbage collector for debugging memory issues using a flag and the Chrome Debugger:

```
node --expose-gc --inspect index.js
```

Data structures aiding memory management

Although JavaScript does not directly expose the garbage collector API, the language offers several data structures that indirectly observe garbage collection and can be used to manage memory usage.

WeakMaps and WeakSets

`WeakMap` and `WeakSet` are data structures whose APIs closely mirror their non-weak counterparts: `Map` and `Set`. `WeakMap` allows you to maintain a collection of key-value pairs, while `WeakSet` allows you to maintain a collection of unique values, both with performant addition, deletion, and querying.

`WeakMap` and `WeakSet` got the name from the concept of *weakly held* values. If `x` is weakly held by `y`, it means that although you can access the value of `x` via `y`, the mark-and-sweep algorithm won't consider `x` as reachable if nothing else *strongly holds* to it. Most data structures, except the ones discussed here, strongly hold to the objects passed in so that you can retrieve them at any time. The keys of `WeakMap` and `WeakSet` can be garbage-collected (for `WeakMap` objects, the values would then be eligible for garbage collection as well) as long as nothing else in the program is referencing the key. This is ensured by two characteristics:

- `WeakMap` and `WeakSet` can only store objects or symbols. This is because only objects are garbage collected — primitive values can always be forged (that is, `1 === 1` but `{} !== {}`), making them stay in the collection forever. Registered symbols (like `Symbol.for("key")`) can also be forged and thus are not garbage collectable, but symbols created with `Symbol("key")` are garbage collectable. Well-known symbols like `Symbol.iterator` come in a fixed set and are unique throughout the lifetime of the program, similar to intrinsic objects such as `Array.prototype`, so they are also allowed as keys.
- `WeakMap` and `WeakSet` are not iterable. This prevents you from using `Array.from(map.keys()).length` to observe the liveliness of objects, or get hold of an arbitrary key which should otherwise be eligible for garbage collection. (Garbage collection should be as invisible as possible.)

In typical explanations of `WeakMap` and `WeakSet` (such as the one above), it's often implied that the key is garbage-collected first, freeing the value for garbage collection as well. However, consider the case of the value referencing the key:

```
const wm = new WeakMap();
const key = {};
wm.set(key, { key });
// Now `key` cannot be garbage collected,
// because the value holds a reference to the key,
// and the value is strongly held in the map!
```

If `key` is stored as an actual reference, it would create a cyclic reference and make both the key and value ineligible for garbage collection, even when nothing else references `key` — because if `key` is garbage collected, it means that at some particular instant, `value.key` would point to a non-existent address, which is not legal. To fix this, the entries of `WeakMap` and `WeakSet` aren't actual references, but ephemerons, an enhancement to the mark-and-sweep mechanism. Barros et al. offers a good summary of the algorithm (page 4). To quote a paragraph:

> Ephemerons are a refinement of weak pairs where neither the key nor the value can be classified as weak or strong. The connectivity of the key determines the connectivity of the value, but the connectivity of the value does not affect the connectivity of the key. [...] when the garbage collection offers support to ephemerons, it occurs in three phases instead of two (mark and sweep).

As a rough mental model, think of a `WeakMap` as the following implementation:

> **Warning:** This is not a polyfill, nor is it anywhere close to how it's implemented in the engine (which hooks into the garbage collection mechanism).

```
class MyWeakMap {
  #marker = Symbol("MyWeakMapData");
  get(key) {
    return key[this.#marker];
  }
  set(key, value) {
    key[this.#marker] = value;
  }
  has(key) {
    return this.#marker in key;
  }
  delete(key) {
    delete key[this.#marker];
  }
}
```

As you can see, the `MyWeakMap` never actually holds a collection of keys. It simply adds metadata to each object being passed in. The object is then garbage-collectable via mark-and-sweep. Therefore, it's not possible to iterate over the keys in a `WeakMap`, nor clear the `WeakMap` (as that also relies on the knowledge of the entire key collection).

For more information on their APIs, see the keyed collections guide.

WeakRefs and FinalizationRegistry

> **Note:** `WeakRef` and `FinalizationRegistry` offer direct introspection into the garbage collection machinery. Avoid using them where possible because the runtime semantics are almost completely unguaranteed.

All variables with an object as value are references to that object. However, such references are *strong* — their existence would prevent the garbage collector from marking the object as eligible for collection. A `WeakRef` is a *weak reference* to an object that allows the object to be garbage collected, while still retaining the ability to read the object's content during its lifetime.

One use case for `WeakRef` is a cache system which maps string URLs to large objects. We cannot use a `WeakMap` for this purpose, because `WeakMap` objects have their *keys* weakly held, but not their *values* — if you access a key, you would always deterministically get the value (since having access to the key means it's still alive). Here, we are okay to get `undefined` for a key (if the corresponding value is no longer alive) since we can just re-compute it, but we don't want unreachable objects to stay in the cache. In this case, we can use a normal `Map`, but with each value being a `WeakRef` of the object instead of the actual object value.

```
function cached(getter) {
  // A Map from string URLs to WeakRefs of results
  const cache = new Map();
  return async (key) => {
    if (cache.has(key)) {
      const dereferencedValue = cache.get(key).deref();
      if (dereferencedValue !== undefined) {
        return dereferencedValue;
      }
    }
    const value = await getter(key);
    cache.set(key, new WeakRef(value));
    return value;
  };
}

const getImage = cached((url) => fetch(url).then((res) => res.blob()));
```

`FinalizationRegistry` provides an even stronger mechanism to observe garbage collection. It allows you to register objects and be notified when they are garbage collected. For example, for the cache system exemplified above, even when the blobs themselves are free for collection, the `WeakRef` objects that hold them are not — and over time, the `Map` may accumulate a lot of useless entries. Using a `FinalizationRegistry` allows one to perform cleanup in this case.

```
function cached(getter) {
  // A Map from string URLs to WeakRefs of results
  const cache = new Map();
  // Every time after a value is garbage collected, the callback is
  // called with the key in the cache as argument, allowing us to remove
  // the cache entry
  const registry = new FinalizationRegistry((key) => {
    // Note: it's important to test that the WeakRef is indeed empty.
    // Otherwise, the callback may be called after a new object has been
    // added with this key, and that new, alive object gets deleted
    if (!cache.get(key)?.deref()) {
      cache.delete(key);
    }
  });
  return async (key) => {
    if (cache.has(key)) {
      return cache.get(key).deref();
    }
    const value = await getter(key);
    cache.set(key, new WeakRef(value));
    registry.register(value, key);
    return value;
  };
}

const getImage = cached((url) => fetch(url).then((res) => res.blob()));
```

Due to performance and security concerns, there is no guarantee of when the callback will be called, or if it will be called at all. It should only be used for cleanup — and non-critical cleanup. There are other ways for more deterministic resource management, such as `try...finally`, which will always execute the `finally` block. `WeakRef` and `FinalizationRegistry` exist solely for optimization of memory usage in long-running programs.

For more information on the API of `WeakRef` and `FinalizationRegistry`, see their reference pages.

Meta programming

The `Proxy` and `Reflect` objects allow you to intercept and define custom behavior for fundamental language operations (e.g., property lookup, assignment, enumeration, function invocation, etc.). With the help of these two objects you are able to program at the meta level of JavaScript.

Proxies

`Proxy` objects allow you to intercept certain operations and to implement custom behaviors.

For example, getting a property on an object:

```
const handler = {
  get(target, name) {
    return name in target ? target[name] : 42;
  },
};

const p = new Proxy({}, handler);
p.a = 1;
console.log(p.a, p.b); // 1, 42
```

The `Proxy` object defines a `target` (an empty object here) and a `handler` object, in which a `get` *trap* is implemented. Here, an object that is proxied will not return `undefined` when getting undefined properties, but will instead return the number `42`.

Additional examples are available on the `Proxy` reference page.

Terminology

The following terms are used when talking about the functionality of proxies.

handler

Placeholder object which contains traps.

traps

The methods that provide property access. (This is analogous to the concept of *traps* in operating systems.)

target

Object which the proxy virtualizes. It is often used as storage backend for the proxy. Invariants (semantics that remain unchanged) regarding object non-extensibility or non-configurable properties are verified against the target.

invariants

Semantics that remain unchanged when implementing custom operations are called *invariants*. If you violate the invariants of a handler, a `TypeError` will be thrown.

Handlers and traps

The following table summarizes the available traps available to `Proxy` objects. See the reference pages for detailed explanations and examples.

Handler / trap	Interceptions

`handler.getPrototypeOf()`	`Object.getPrototypeOf()`
	`Reflect.getPrototypeOf()`
	`__proto__`
	`Object.prototype.isPrototypeOf()`
	`instanceof`

`handler.setPrototypeOf()`	`Object.setPrototypeOf()` `Reflect.setPrototypeOf()`
`handler.isExtensible()`	`Object.isExtensible()` `Reflect.isExtensible()`
`handler.preventExtensions()`	`Object.preventExtensions()` `Reflect.preventExtensions()`
`handler.getOwnPropertyDescriptor()`	`Object.getOwnPropertyDescriptor()` `Reflect.getOwnPropertyDescriptor()`
`handler.defineProperty()`	`Object.defineProperty()` `Reflect.defineProperty()`

`handler.has()`	Property query
	`foo in proxy`
	Inherited property query
	`foo in Object.create(proxy)`
	`Reflect.has()`

`handler.get()`	Property access
	`proxy[foo]`
	`proxy.bar`
	Inherited property access
	`Object.create(proxy)[foo]`
	`Reflect.get()`

`handler.set()`	Property assignment
	`proxy[foo] = bar`
	`proxy.foo = bar`
	Inherited property assignment

Handler / trap	Interceptions
	`Object.create(proxy)[foo] = bar` `Reflect.set()`

`handler.deleteProperty()`	Property deletion
	`delete proxy[foo]`
	`delete proxy.foo`
	`Reflect.deleteProperty()`

`handler.ownKeys()`	`Object.getOwnPropertyNames()`
	`Object.getOwnPropertySymbols()`
	`Object.keys()`
	`Reflect.ownKeys()`

`handler.apply()`	`proxy(..args)`
	`Function.prototype.apply()` and `Function.prototype.call()`
	`Reflect.apply()`

`handler.construct()`	`new proxy(...args)` `Reflect.construct()`

Revocable <code>Proxy</code>

The `Proxy.revocable()` method is used to create a revocable `Proxy` object. This means that the proxy can be revoked via the function `revoke` and switches the proxy off.

Afterwards, any operation on the proxy leads to a `TypeError`.

```
const revocable = Proxy.revocable(
  {},
  {
    get(target, name) {
      return `[[${name}]]`;
    },
  },
);
const proxy = revocable.proxy;
console.log(proxy.foo); // "[[foo]]"

revocable.revoke();

console.log(proxy.foo); // TypeError: Cannot perform 'get' on a proxy that has been revoked
proxy.foo = 1; // TypeError: Cannot perform 'set' on a proxy that has been revoked
delete proxy.foo; // TypeError: Cannot perform 'deleteProperty' on a proxy that has been revoked
console.log(typeof proxy); // "object", typeof doesn't trigger any trap
```

Reflection

`Reflect` is a built-in object that provides methods for interceptable JavaScript operations. The methods are the same as those of the proxy handler's.

`Reflect` is not a function object.

`Reflect` helps with forwarding default operations from the handler to the `target`.

With `Reflect.has()` for example, you get the `in` operator as a function:

```
Reflect.has(Object, "assign"); // true
```

A better apply() function

Before `Reflect`, you typically use the `Function.prototype.apply()` method to call a function with a given `this` value and `arguments` provided as an array (or an array-like object).

```
Function.prototype.apply.call(Math.floor, undefined, [1.75]);
```

With `Reflect.apply` this becomes less verbose and easier to understand:

```
Reflect.apply(Math.floor, undefined, [1.75]);
// 1

Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]);
// "hello"

Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index;
// 4

Reflect.apply("".charAt, "ponies", [3]);
// "i"
```

Checking if property definition has been successful

With `Object.defineProperty`, which returns an object if successful, or throws a `TypeError` otherwise, you would use a `try...catch` block to catch any error that occurred while defining a property. Because `Reflect.defineProperty()` returns a Boolean success status, you can just use an `if...else` block here:

```
if (Reflect.defineProperty(target, property, attributes)) {
  // success
} else {
  // failure
}
```

JavaScript modules

- Previous

This guide gives you all you need to get started with JavaScript module syntax.

A background on modules

JavaScript programs started off pretty small — most of its usage in the early days was to do isolated scripting tasks, providing a bit of interactivity to your web pages where needed, so large scripts were generally not needed. Fast forward a few years and we now have complete applications being run in browsers with a lot of JavaScript, as well as JavaScript being used in other contexts (Node.js, for example).

Complex projects necessitate a mechanism for splitting JavaScript programs into separate modules that can be imported when needed. Node.js has had this ability for a long time, and there are a number of JavaScript libraries and frameworks that enable module usage (for example, other CommonJS and AMD-based module systems like RequireJS, webpack, and Babel).

All modern browsers support module features natively without needing transpilation. It can only be a good thing — browsers can optimize loading of modules, making it more efficient than having to use a library and do all of that extra client-side processing and extra round trips. It does not obsolete bundlers like webpack, though — bundlers still do a good job at partitioning code into reasonably sized chunks, and are able to do other optimizations like minification, dead code elimination, and tree-shaking.

Introducing an example

To demonstrate usage of modules, we've created a set of examples that you can find on GitHub. These examples demonstrate a set of modules that create a `<canvas>` element on a webpage, and then draw (and report information about) different shapes on the canvas.

These are fairly trivial, but have been kept deliberately simple to demonstrate modules clearly.

> **Note:** If you want to download the examples and run them locally, you'll need to run them through a local web server.

Basic example structure

In our first example (see basic-modules) we have a file structure as follows:

```
index.html
main.js
modules/
    canvas.js
    square.js
```

> **Note:** All of the examples in this guide have basically the same structure; the above should start getting pretty familiar.

The modules directory's two modules are described below:

- `canvas.js` — contains functions related to setting up the canvas:
  - `create()` — creates a canvas with a specified `width` and `height` inside a wrapper `<div>` with a specified ID, which is itself appended inside a specified parent element. Returns an object containing the canvas's 2D context and the wrapper's ID.
  - `createReportList()` — creates an unordered list appended inside a specified wrapper element, which can be used to output report data into. Returns the list's ID.
- `square.js` — contains:
  - `name` — a constant containing the string 'square'.
  - `draw()` — draws a square on a specified canvas, with a specified size, position, and color. Returns an object containing the square's size, position, and color.
  - `reportArea()` — writes a square's area to a specific report list, given its length.
  - `reportPerimeter()` — writes a square's perimeter to a specific report list, given its length.

Aside — .mjs versus .js

Throughout this article, we've used `.js` extensions for our module files, but in other resources you may see the `.mjs` extension used instead. V8's documentation recommends this, for example. The reasons given are:

- It is good for clarity, i.e., it makes it clear which files are modules, and which are regular JavaScript.
- It ensures that your module files are parsed as a module by runtimes such as Node.js, and build tools such as Babel.

However, we decided to keep using `.js`, at least for the moment. To get modules to work correctly in a browser, you need to make sure that your server is serving them with a `Content-Type` header that contains a JavaScript MIME type such as `text/javascript`. If you don't, you'll get a strict MIME type checking error along the

lines of "The server responded with a non-JavaScript MIME type" and the browser won't run your JavaScript. Most servers already set the correct type for `.js` files, but not yet for `.mjs` files. Servers that already serve `.mjs` files correctly include GitHub Pages and `http-server` for Node.js.

This is OK if you are using such an environment already, or if you aren't but you know what you are doing and have access (i.e., you can configure your server to set the correct `Content-Type` for `.mjs` files). It could however cause confusion if you don't control the server you are serving files from, or are publishing files for public use, as we are here.

For learning and portability purposes, we decided to keep to `.js`.

If you really value the clarity of using `.mjs` for modules versus using `.js` for "normal" JavaScript files, but don't want to run into the problem described above, you could always use `.mjs` during development and convert them to `.js` during your build step.

It is also worth noting that:

- Some tools may never support `.mjs`.
- The `<script type="module">` attribute is used to denote when a module is being pointed to, as described in Applying the module to your HTML.

Exporting module features

The first thing you do to get access to module features is export them. This is done using the `export` statement.

The easiest way to use it is to place it in front of any items you want exported out of the module, for example:

```
export const name = "square";

export function draw(ctx, length, x, y, color) {
  ctx.fillStyle = color;
  ctx.fillRect(x, y, length, length);

  return { length, x, y, color };
}
```

You can export functions, `var`, `let`, `const`, and — as we'll see later — classes. They need to be top-level items: for example, you can't use `export` inside a function.

A more convenient way of exporting all the items you want to export is to use a single export statement at the end of your module file, followed by a comma-separated list of the features you want to export wrapped in curly braces. For example:

```
export { name, draw, reportArea, reportPerimeter };
```

Importing features into your script

Once you've exported some features out of your module, you need to import them into your script to be able to use them. The simplest way to do this is as follows:

```
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";
```

You use the `import` statement, followed by a comma-separated list of the features you want to import wrapped in curly braces, followed by the keyword `from`, followed by the *module specifier*.

The *module specifier* provides a string that the JavaScript environment can resolve to a path to the module file. In a browser, this could be a path relative to the site root, which for our `basic-modules` example would be `/js-examples/module-examples/basic-modules`. However, here we are instead using the dot (`.`) syntax to mean "the current location", followed by the relative path to the file we are trying to find. This is much better than writing out the entire absolute path each time, as relative paths are shorter and make the URL portable — the example will still work if you move it to a different location in the site hierarchy.

So for example:

```
/js-examples/module-examples/basic-modules/modules/square.js
```

becomes

```
./modules/square.js
```

You can see such lines in action in `main.js`.

> **Note:** In some module systems, you can use a module specifier like `modules/square` that isn't a relative or absolute path, and that doesn't have a file extension. This kind of specifier can be used in a browser environment if you first define an import map.

Once you've imported the features into your script, you can use them just like they were defined inside the same file. The following is found in `main.js`, below the import lines:

```
const myCanvas = create("myCanvas", document.body, 480, 320);
const reportList = createReportList(myCanvas.id);

const square = draw(myCanvas.ctx, 50, 50, 100, "blue");
reportArea(square.length, reportList);
reportPerimeter(square.length, reportList);
```

> **Note:** The imported values are read-only views of the features that were exported. Similar to `const` variables, you cannot re-assign the variable that was imported,

> but you can still modify properties of object values. The value can only be re-assigned by the module exporting it. See the `import` reference for an example.

Importing modules using import maps

Above we saw how a browser can import a module using a module specifier that is either an absolute URL, or a relative URL that is resolved using the base URL of the document:

```
import { name as circleName } from "https://example.com/shapes/circle.js";
import { name as squareName, draw } from "./shapes/square.js";
```

Import maps allow developers to instead specify almost any text they want in the module specifier when importing a module; the map provides a corresponding value that will replace the text when the module URL is resolved.

For example, the `imports` key in the import map below defines a "module specifier map" JSON object where the property names can be used as module specifiers, and the corresponding values will be substituted when the browser resolves the module URL. The values must be absolute or relative URLs. Relative URLs are resolved to absolute URL addresses using the base URL of the document containing the import map.

```
<script type="importmap">
  {
    "imports": {
      "shapes": "./shapes/square.js",
      "shapes/square": "./modules/shapes/square.js",
      "https://example.com/shapes/square.js": "./shapes/square.js",
      "https://example.com/shapes/": "/shapes/square/",
      "../shapes/square": "./shapes/square.js"
    }
  }
</script>
```

The import map is defined using a JSON object inside a `<script>` element with the `type` attribute set to `importmap`. Note that an import map only applies to the document — the specification does not cover how to apply an import map in a worker or worklet context.

With this map you can now use the property names above as module specifiers. If there is no trailing forward slash on the module specifier key then the whole module specifier key is matched and substituted. For example, below we match bare module names, and remap a URL to another path.

```
// Bare module names as module specifiers
import { name as squareNameOne } from "shapes";
import { name as squareNameTwo } from "shapes/square";

// Remap a URL to another URL
import { name as squareNameThree } from "https://example.com/shapes/square.js";
```

If the module specifier has a trailing forward slash then the value must have one as well, and the key is matched as a "path prefix". This allows remapping of whole classes of URLs.

```
// Remap a URL as a prefix ( https://example.com/shapes/)
import { name as squareNameFour } from "https://example.com/shapes/moduleshapes/square.js";
```

It is possible for multiple keys in an import map to be valid matches for a module specifier. For example, a module specifier of `shapes/circle/` could match the module specifier keys `shapes/` and `shapes/circle/`. In this case the browser will select the most specific (longest) matching module specifier key.

Import maps allow modules to be imported using bare module names (as in Node.js), and can also simulate importing modules from packages, both with and without file extensions. While not shown above, they also allow particular versions of a library to be imported, based on the path of the script that is importing the module. Generally they let developers write more ergonomic import code, and make it easier to manage the different versions and dependencies of modules used by a site. This can reduce the effort required to use the same JavaScript libraries in both browser and server.

The following sections expand on the various features outlined above.

Feature detection

You can check support for import maps using the `HTMLScriptElement.supports()` static method (which is itself broadly supported):

```
if (HTMLScriptElement.supports?.("importmap")) {
  console.log("Browser supports import maps.");
}
```

Importing modules as bare names

In some JavaScript environments, such as Node.js, you can use bare names for the module specifier. This works because the environment can resolve module names to a standard location in the file system. For example, you might use the following syntax to import the "square" module.

```
import { name, draw, reportArea, reportPerimeter } from "square";
```

To use bare names on a browser you need an import map, which provides the information needed by the browser to resolve module specifiers to URLs (JavaScript will throw a `TypeError` if it attempts to import a module specifier that can't be resolved to a module location).

Below you can see a map that defines a `square` module specifier key, which in this case maps to a relative address value.

```
<script type="importmap">
  {
    "imports": {
      "square": "./shapes/square.js"
    }
  }
</script>
```

With this map we can now use a bare name when we import the module:

```
import { name as squareName, draw } from "square";
```

Remapping module paths

Module specifier map entries, where both the specifier key and its associated value have a trailing forward slash (`/`), can be used as a path-prefix. This allows the remapping of a whole set of import URLs from one location to another. It can also be used to emulate working with "packages and modules", such as you might see in the Node ecosystem.

> **Note:** The trailing `/` indicates that the module specifier key can be substituted as *part* of a module specifier. If this is not present, the browser will only match (and substitute) the whole module specifier key.

Packages of modules

The following JSON import map definition maps `lodash` as a bare name, and the module specifier prefix `lodash/` to the path `/node_modules/lodash-es/` (resolved to the document base URL):

```
{
  "imports": {
    "lodash": "/node_modules/lodash-es/lodash.js",
    "lodash/": "/node_modules/lodash-es/"
  }
}
```

With this mapping you can import both the whole "package", using the bare name, and modules within it (using the path mapping):

```
import _ from "lodash";
import fp from "lodash/fp.js";
```

It is possible to import `fp` above without the `.js` file extension, but you would need to create a bare module specifier key for that file, such as `lodash/fp`, rather than using the path. This may be reasonable for just one module, but scales poorly if you wish to import many modules.

General URL remapping

A module specifier key doesn't have to be a path — it can also be an absolute URL (or a URL-like relative path like `./`, `../`, `/`). This may be useful if you want to remap a module that has absolute paths to a resource with your own local resources.

```
{
  "imports": {
    "https://www.unpkg.com/moment/": "/node_modules/moment/"
  }
}
```

Scoped modules for version management

Ecosystems like Node use package managers such as npm to manage modules and their dependencies. The package manager ensures that each module is separated from other modules and their dependencies. As a result, while a complex application might include the same module multiple times with several different versions in different parts of the module graph, users do not need to think about this complexity.

> **Note:** You can also achieve version management using relative paths, but this is subpar because, among other things, this forces a particular structure on your project, and prevents you from using bare module names.

Import maps similarly allow you to have multiple versions of dependencies in your application and refer to them using the same module specifier. You implement this with the `scopes` key, which allows you to provide module specifier maps that will be used depending on the path of the script performing the import. The example below demonstrates this.

```
{
  "imports": {
    "cool-module": "/node_modules/cool-module/index.js"
  },
  "scopes": {
    "/node_modules/dependency/": {
      "cool-module": "/node_modules/some/other/location/cool-module/index.js"
    }
  }
}
```

With this mapping, if a script with a URL that contains `/node_modules/dependency/` imports `cool-module`, the version in `/node_modules/some/other/location/cool-module/index.js` will be used. The map in `imports` is used as a fallback if there is no matching scope in the scoped map, or the matching scopes don't contain a matching specifier. For example, if `cool-module` is imported from a script with a non-matching scope path, then the module specifier map in `imports` will be used instead, mapping to the version in `/node_modules/cool-module/index.js`.

Note that the path used to select a scope does not affect how the address is resolved. The value in the mapped path does not have to match the scopes path, and relative paths are still resolved to the base URL of the script that contains the import map.

Just as for module specifier maps, you can have many scope keys, and these may contain overlapping paths. If multiple scopes match the referrer URL, then the most specific scope path is checked first (the longest scope key) for a matching specifier. The browsers will fall back to the next most specific matching scoped path if there is no matching specifier, and so on. If there is no matching specifier in any of the matching scopes, the browser checks for a match in the module specifier map in the `imports` key.

Improve caching by mapping away hashed filenames

Script files used by websites often have hashed filenames to simplify caching. The downside of this approach is that if a module changes, any modules that import it using its hashed filename will also need to be updated/regenerated. This potentially results in a cascade of updates, which is wasteful of network resources.

Import maps provide a convenient solution to this problem. Rather than depending on specific hashed filenames, applications and scripts instead depend on an un-hashed version of the module name (address). An import map like the one below then provides a mapping to the actual script file.

```
{
  "imports": {
    "main_script": "/node/srcs/application-fg7744e1b.js",
    "dependency_script": "/node/srcs/dependency-3qn7e4b1q.js"
  }
}
```

If `dependency_script` changes, then its hash contained in the file name changes as well. In this case, we only need to update the import map to reflect the changed name of the module. We don't have to update the source of any JavaScript code that depends on it, because the specifier in the import statement does not change.

Loading non-JavaScript resources

One exciting feature that a unified module architecture brings is the ability to load non-JavaScript resources as modules. For example, you can import JSON as a JavaScript object, or import CSS as a `CSSStyleSheet` object.

You must explicitly declare what kind of resource you are importing. By default, the browser assumes that the resource is JavaScript, and will throw an error if the resolved resource is something else. To import JSON, CSS, or other types of resource, use the import attributes syntax:

```
import colors from "./colors.json" with { type: "json" };
import styles from "./styles.css" with { type: "css" };
```

Browsers will also perform validation on the module type, and fail if, for example, `./data.json` does not resolve to a JSON file. This ensures that you don't accidentally execute code when you just intend to import data. Once imported successfully, you can now use the imported value as a normal JavaScript object or `CSSStyleSheet` object.

```
console.log(colors.map((color) => color.value));
document.adoptedStyleSheets = [styles];
```

Applying the module to your HTML

Now we just need to apply the `main.js` module to our HTML page. This is very similar to how we apply a regular script to a page, with a few notable differences.

First of all, you need to include `type="module"` in the `<script>` element, to declare this script as a module. To import the `main.js` script, we use this:

```
<script type="module" src="https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules/main.js"></script>
```

You can also embed the module's script directly into the HTML file by placing the JavaScript code within the body of the `<script>` element:

```
<script type="module">
  /* JavaScript module code here */
</script>
```

You can only use `import` and `export` statements inside modules, not regular scripts. An error will be thrown if your `<script>` element doesn't have the `type="module"` attribute and attempts to import other modules. For example:

```
<script>
  import _ from "lodash"; // SyntaxError: import declarations may only appear at top level of a module
  // …
</script>
<script src="https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules/a-module-using-import-statements.js"></script>
<!-- SyntaxError: import declarations may only appear at top level of a module -->
```

You should generally define all your modules in separate files. Modules declared inline in HTML can only import other modules, but anything they export will not be accessible by other modules (because they don't have a URL).

> **Note:** Modules and their dependencies can be preloaded by specifying them in `<link>` elements with `rel="modulepreload"`. This can significantly reduce load time when the modules are used.

Other differences between modules and classic scripts

- You need to pay attention to local testing — if you try to load the HTML file locally (i.e., with a `file://` URL), you'll run into CORS errors due to JavaScript module security requirements. You need to do your testing through a server.
- Also, note that you might get different behavior from sections of script defined inside modules as opposed to in classic scripts. This is because modules use strict mode automatically.
- There is no need to use the `defer` attribute (see `<script>` attributes) when loading a module script; modules are deferred automatically.
- Modules are only executed once, even if they have been referenced in multiple `<script>` tags.
- Last but not least, let's make this clear — module features are imported into the scope of a single script — they aren't available in the global scope. Therefore, you will only be able to access imported features in the script they are imported into, and you won't be able to access them from the JavaScript console, for example. You'll still get syntax errors shown in the DevTools, but you'll not be able to use some of the debugging techniques you might have expected to use.

Module-defined variables are scoped to the module unless explicitly attached to the global object. On the other hand, globally-defined variables are available within the module. For example, given the following code:

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>Example page</title>
    <link rel="stylesheet" href="" />
  </head>
  <body>
    <div id="main"></div>
    <script>
      // A var statement creates a global variable.
      var text = "Hello";
    </script>
    <script type="module" src="https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules/./render.js"></script>
  </body>
</html>
```

```
/* render.js */
document.getElementById("main").innerText = text;
```

The page would still render `Hello`, because the global variables `text` and `document` are available in the module. (Also note from this example that a module doesn't necessarily need an import/export statement — the only thing needed is for the entry point to have `type="module"`.)

Default exports versus named exports

The functionality we've exported so far has been comprised of **named exports** — each item (be it a function, `const`, etc.) has been referred to by its name upon export, and that name has been used to refer to it on import as well.

There is also a type of export called the **default export** — this is designed to make it easy to have a default function provided by a module, and also helps JavaScript modules to interoperate with existing CommonJS and AMD module systems (as explained nicely in ES6 In Depth: Modules by Jason Orendorff; search for "Default exports").

Let's look at an example as we explain how it works. In our basic-modules `square.js` you can find a function called `randomSquare()` that creates a square with a random color, size, and position. We want to export this as our default, so at the bottom of the file we write this:

```
export default randomSquare;
```

Note the lack of curly braces.

We could instead prepend `export default` onto the function and define it as an anonymous function, like this:

```
export default function (ctx) {
  // …
}
```

Over in our `main.js` file, we import the default function using this line:

```
import randomSquare from "./modules/square.js";
```

Again, note the lack of curly braces. This is because there is only one default export allowed per module, and we know that `randomSquare` is it. The above line is basically shorthand for:

```
import { default as randomSquare } from "./modules/square.js";
```

> **Note:** The as syntax for renaming exported items is explained below in the Renaming imports and exports section.

Avoiding naming conflicts

So far, our canvas shape drawing modules seem to be working OK. But what happens if we try to add a module that deals with drawing another shape, like a circle or triangle? These shapes would probably have associated functions like `draw()`, `reportArea()`, etc. too; if we tried to import different functions of the same name into the same top-level module file, we'd end up with conflicts and errors.

Fortunately there are a number of ways to get around this. We'll look at these in the following sections.

Renaming imports and exports

Inside your `import` and `export` statement's curly braces, you can use the keyword `as` along with a new feature name, to change the identifying name you will use for a feature inside the top-level module.

So for example, both of the following would do the same job, albeit in a slightly different way:

```
// -- module.js --
export { function1 as newFunctionName, function2 as anotherNewFunctionName };

// -- main.js --
import { newFunctionName, anotherNewFunctionName } from "./modules/module.js";
```

```
// -- module.js --
export { function1, function2 };

// -- main.js --
import {
  function1 as newFunctionName,
  function2 as anotherNewFunctionName,
} from "./modules/module.js";
```

Let's look at a real example. In our renaming directory you'll see the same module system as in the previous example, except that we've added `circle.js` and `triangle.js` modules to draw and report on circles and triangles.

Inside each of these modules, we've got features with the same names being exported, and therefore each has the same `export` statement at the bottom:

```
export { name, draw, reportArea, reportPerimeter };
```

When importing these into `main.js`, if we tried to use

```
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";
import { name, draw, reportArea, reportPerimeter } from "./modules/circle.js";
import { name, draw, reportArea, reportPerimeter } from "./modules/triangle.js";
```

The browser would throw an error such as "SyntaxError: redeclaration of import name" (Firefox).

Instead we need to rename the imports so that they are unique:

```
import {
  name as squareName,
  draw as drawSquare,
  reportArea as reportSquareArea,
  reportPerimeter as reportSquarePerimeter,
} from "./modules/square.js";

import {
  name as circleName,
  draw as drawCircle,
  reportArea as reportCircleArea,
  reportPerimeter as reportCirclePerimeter,
} from "./modules/circle.js";

import {
  name as triangleName,
  draw as drawTriangle,
  reportArea as reportTriangleArea,
  reportPerimeter as reportTrianglePerimeter,
} from "./modules/triangle.js";
```

Note that you could solve the problem in the module files instead, e.g.

```
// in square.js
export {
  name as squareName,
  draw as drawSquare,
  reportArea as reportSquareArea,
  reportPerimeter as reportSquarePerimeter,
};
```

```
// in main.js
import {
  squareName,
  drawSquare,
  reportSquareArea,
  reportSquarePerimeter,
} from "./modules/square.js";
```

And it would work just the same. What style you use is up to you, however it arguably makes more sense to leave your module code alone, and make the changes in the imports. This especially makes sense when you are importing from third party modules that you don't have any control over.

Creating a module object

The above method works OK, but it's a little messy and long-winded. An even better solution is to import each module's features inside a module object. The following syntax form does that:

```
import * as Module from "./modules/module.js";
```

This grabs all the exports available inside `module.js`, and makes them available as members of an object `Module`, effectively giving it its own namespace. So for example:

```
Module.function1();
Module.function2();
```

Again, let's look at a real example. If you go to our module-objects directory, you'll see the same example again, but rewritten to take advantage of this new syntax. In the modules, the exports are all in the following simple form:

```
export { name, draw, reportArea, reportPerimeter };
```

The imports on the other hand look like this:

```
import * as Canvas from "./modules/canvas.js";

import * as Square from "./modules/square.js";
import * as Circle from "./modules/circle.js";
import * as Triangle from "./modules/triangle.js";
```

In each case, you can now access the module's imports underneath the specified object name, for example:

```
const square = Square.draw(myCanvas.ctx, 50, 50, 100, "blue");
Square.reportArea(square.length, reportList);
Square.reportPerimeter(square.length, reportList);
```

So you can now write the code just the same as before (as long as you include the object names where needed), and the imports are much neater.

Modules and classes

As we hinted at earlier, you can also export and import classes; this is another option for avoiding conflicts in your code, and is especially useful if you've already got your module code written in an object-oriented style.

You can see an example of our shape drawing module rewritten with ES classes in our classes directory. As an example, the `square.js` file now contains all its functionality in a single class:

```
class Square {
  constructor(ctx, listId, length, x, y, color) {
    // …
  }

  draw() {
    // …
  }

  // …
}
```

which we then export:

```
export { Square };
```

Over in `main.js`, we import it like this:

```
import { Square } from "./modules/square.js";
```

And then use the class to draw our square:

```
const square = new Square(myCanvas.ctx, myCanvas.listId, 50, 50, 100, "blue");
square.draw();
square.reportArea();
square.reportPerimeter();
```

Aggregating modules

There will be times where you'll want to aggregate modules together. You might have multiple levels of dependencies, where you want to simplify things, combining several submodules into one parent module. This is possible using export syntax of the following forms in the parent module:

```
export * from "x.js";
export { name } from "x.js";
```

For an example, see our module-aggregation directory. In this example (based on our earlier classes example) we've got an extra module called `shapes.js`, which aggregates all the functionality from `circle.js`, `square.js`, and `triangle.js` together. We've also moved our submodules inside a subdirectory inside the `modules` directory called `shapes`. So the module structure in this example is:

```
modules/
  canvas.js
  shapes.js
  shapes/
    circle.js
    square.js
    triangle.js
```

In each of the submodules, the export is of the same form, e.g.

```
export { Square };
```

Next up comes the aggregation part. Inside `shapes.js`, we include the following lines:

```
export { Square } from "./shapes/square.js";
export { Triangle } from "./shapes/triangle.js";
export { Circle } from "./shapes/circle.js";
```

These grab the exports from the individual submodules and effectively make them available from the `shapes.js` module.

> **Note:** The exports referenced in `shapes.js` basically get redirected through the file and don't really exist there, so you won't be able to write any useful related code inside the same file.

So now in the `main.js` file, we can get access to all three module classes by replacing

```
import { Square } from "./modules/square.js";
import { Circle } from "./modules/circle.js";
import { Triangle } from "./modules/triangle.js";
```

with the following single line:

```
import { Square, Circle, Triangle } from "./modules/shapes.js";
```

Dynamic module loading

A recent addition to JavaScript modules functionality is dynamic module loading. This allows you to dynamically load modules only when they are needed, rather than having to load everything up front. This has some obvious performance advantages; let's read on and see how it works.

This new functionality allows you to call `import()` as a function, passing it the path to the module as a parameter. It returns a `Promise`, which fulfills with a module object (see Creating a module object) giving you access to that object's exports. For example:

```
import("./modules/myModule.js").then((module) => {
  // Do something with the module.
});
```

> **Note:** Dynamic import is permitted in the browser main thread, and in shared and dedicated workers. However `import()` will throw if called in a service worker or worklet.

Let's look at an example. In the dynamic-module-imports directory we've got another example based on our classes example. This time however we are not drawing anything on the canvas when the example loads. Instead, we include three buttons — "Circle", "Square", and "Triangle" — that, when pressed, dynamically load the required module and then use it to draw the associated shape.

In this example we've only made changes to our `index.html` and `main.js` files — the module exports remain the same as before.

Over in `main.js` we've grabbed a reference to each button using a `document.querySelector()` call, for example:

```
const squareBtn = document.querySelector(".square");
```

We then attach an event listener to each button so that when pressed, the relevant module is dynamically loaded and used to draw the shape:

```
squareBtn.addEventListener("click", () => {
  import("./modules/square.js").then((Module) => {
    const square = new Module.Square(
      myCanvas.ctx,
      myCanvas.listId,
      50,
      50,
      100,
      "blue",
    );
    square.draw();
    square.reportArea();
    square.reportPerimeter();
  });
});
```

Note that, because the promise fulfillment returns a module object, the class is then made a subfeature of the object, hence we now need to access the constructor with `Module.` prepended to it, e.g., `Module.Square( /* … */ )`.

Another advantage of dynamic imports is that they are always available, even in script environments. Therefore, if you have an existing `<script>` tag in your HTML that doesn't have `type="module"`, you can still reuse code distributed as modules by dynamically importing it.

```
<script>
  import("./modules/square.js").then((module) => {
    // Do something with the module.
  });
  // Other code that operates on the global scope and is not
  // ready to be refactored into modules yet.
  var btn = document.querySelector(".square");
</script>
```

Top level await

Top level await is a feature available within modules. This means the `await` keyword can be used. It allows modules to act as big asynchronous functions meaning code can be evaluated before use in parent modules, but without blocking sibling modules from loading.

Let's take a look at an example. You can find all the files and code described in this section within the `top-level-await` directory, which extends from the previous examples.

Firstly we'll declare our color palette in a separate `colors.json` file:

```
{
  "yellow": "#F4D03F",
  "green": "#52BE80",
  "blue": "#5499C7",
  "red": "#CD6155",
  "orange": "#F39C12"
}
```

Then we'll create a module called `getColors.js` which uses a fetch request to load the `colors.json` file and return the data as an object.

```
// fetch request
const colors = fetch("../data/colors.json").then((response) => response.json());

export default await colors;
```

Notice the last export line here.

We're using the keyword `await` before specifying the constant `colors` to export. This means any other modules which include this one will wait until `colors` has been downloaded and parsed before using it.

Let's include this module in our `main.js` file:

```
import colors from "./modules/getColors.js";
import { Canvas } from "./modules/canvas.js";

const circleBtn = document.querySelector(".circle");

// …
```

We'll use `colors` instead of the previously used strings when calling our shape functions:

```
const square = new Module.Square(
  myCanvas.ctx,
  myCanvas.listId,
  50,
  50,
  100,
  colors.blue,
);

const circle = new Module.Circle(
  myCanvas.ctx,
  myCanvas.listId,
  75,
  200,
  100,
  colors.green,
);

const triangle = new Module.Triangle(
  myCanvas.ctx,
  myCanvas.listId,
  100,
  75,
  190,
  colors.yellow,
);
```

This is useful because the code within `main.js` won't execute until the code in `getColors.js` has run. However it won't block other modules being loaded. For instance our `canvas.js` module will continue to load while `colors` is being fetched.

Import declarations are hoisted

Import declarations are hoisted. In this case, it means that the imported values are available in the module's code even before the place that declares them, and that the imported module's side effects are produced before the rest of the module's code starts running.

So for example, in `main.js`, importing `Canvas` in the middle of the code would still work:

```
// …
const myCanvas = new Canvas("myCanvas", document.body, 480, 320);
myCanvas.create();
import { Canvas } from "./modules/canvas.js";
myCanvas.createReportList();
// …
```

Still, it is considered good practice to put all your imports at the top of the code, which makes it easier to analyze dependencies.

Cyclic imports

Modules can import other modules, and those modules can import other modules, and so on. This forms a directed graph called the "dependency graph". In an ideal world, this graph is acyclic. In this case, the graph can be evaluated using a depth-first traversal.

However, cycles are often inevitable. Cyclic import arises if module `a` imports module `b`, but `b` directly or indirectly depends on `a`. For example:

```
// -- a.js --
import { b } from "./b.js";

// -- b.js --
import { a } from "./a.js";

// Cycle:
// a.js ——> b.js
//  ^         |
//  └─────────┘
```

Cyclic imports don't always fail. The imported variable's value is only retrieved when the variable is actually used (hence allowing live bindings), and only if the variable remains uninitialized at that time will a `ReferenceError` be thrown.

```
// -- a.js --
import { b } from "./b.js";

setTimeout(() => {
  console.log(b); // 1
}, 10);

export const a = 2;

// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);

export const b = 1;
```

In this example, both `a` and `b` are used asynchronously. Therefore, at the time the module is evaluated, neither `b` nor `a` is actually read, so the rest of the code is executed as normal, and the two `export` declarations produce the values of `a` and `b`. Then, after the timeout, both `a` and `b` are available, so the two `console.log` statements also execute as normal.

If you change the code to use `a` synchronously, the module evaluation fails:

```
// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;

// -- b.js --
import { a } from "./a.js";

console.log(a); // ReferenceError: Cannot access 'a' before initialization
export const b = 1;
```

This is because when JavaScript evaluates `a.js`, it needs to first evaluate `b.js`, the dependency of `a.js`. However, `b.js` uses `a`, which is not yet available.

On the other hand, if you change the code to use `b` synchronously but `a` asynchronously, the module evaluation succeeds:

```
// -- a.js (entry module) --
import { b } from "./b.js";

console.log(b); // 1
```

```
export const a = 2;

// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);
export const b = 1;
```

This is because the evaluation of `b.js` completes normally, so the value of `b` is available when `a.js` is evaluated.

You should usually avoid cyclic imports in your project, because they make your code more error-prone. Some common cycle-elimination techniques are:

- Merge the two modules into one.
- Move the shared code into a third module.
- Move some code from one module to the other.

However, cyclic imports can also occur if the libraries depend on each other, which is harder to fix.

Authoring "isomorphic" modules

The introduction of modules encourages the JavaScript ecosystem to distribute and reuse code in a modular fashion. However, that doesn't necessarily mean a piece of JavaScript code can run in every environment. Suppose you discovered a module that generates SHA hashes of your user's password. Can you use it in the browser front end? Can you use it on your Node.js server? The answer is: it depends.

Modules still have access to global variables, as demonstrated previously. If the module references globals like `window`, it can run in the browser, but will throw an error in your Node.js server, because `window` is not available there. Similarly, if the code requires access to `process` to be functional, it can only be used in Node.js.

In order to maximize the reusability of a module, it is often advised to make the code "isomorphic" — that is, exhibits the same behavior in every runtime. This is commonly achieved in three ways:

- Separate your modules into "core" and "binding". For the "core", focus on pure JavaScript logic like computing the hash, without any DOM, network, filesystem access, and expose utility functions. For the "binding" part, you can read from and write to the global context. For example, the "browser binding" may choose to read the value from an input box, while the "Node binding" may read it from `process.env`, but values read from either place will be piped to the same core function and handled in the same way. The core can be imported in every environment and used in the same way, while only the binding, which is usually lightweight, needs to be platform-specific.
- Detect whether a particular global exists before using it. For example, if you test that `typeof window === "undefined"`, you know that you are probably in a Node.js environment, and should not read DOM.

  ```
  // myModule.js
  let password;
  if (typeof process !== "undefined") {
    // We are running in Node.js; read it from `process.env`
    password = process.env.PASSWORD;
  } else if (typeof window !== "undefined") {
    // We are running in the browser; read it from the input box
    password = document.getElementById("password").value;
  }
  ```

  This is preferable if the two branches actually end up with the same behavior ("isomorphic"). If it's impossible to provide the same functionality, or if doing so involves loading significant amounts of code while a large part remains unused, better use different "bindings" instead.
- Use a polyfill to provide a fallback for missing features. For example, if you want to use the `fetch` function, which is only supported in Node.js since v18, you can use a similar API, like the one provided by `node-fetch`. You can do so conditionally through dynamic imports:

  ```
  // myModule.js
  if (typeof fetch === "undefined") {
    // We are running in Node.js; use node-fetch
    globalThis.fetch = (await import("node-fetch")).default;
  }
  // …
  ```

  The `globalThis` variable is a global object that is available in every environment and is useful if you want to read or create global variables within modules.

These practices are not unique to modules. Still, with the trend of code reusability and modularization, you are encouraged to make your code cross-platform so that it can be enjoyed by as many people as possible. Runtimes like Node.js are also actively implementing web APIs where possible to improve interoperability with the web.

Troubleshooting

Here are a few tips that may help you if you are having trouble getting your modules to work. Feel free to add to the list if you discover more!

- We mentioned this before, but to reiterate: `.mjs` files need to be loaded with a MIME-type of `text/javascript` (or another JavaScript-compatible MIME-type, but `text/javascript` is recommended), otherwise you'll get a strict MIME type checking error like "The server responded with a non-JavaScript MIME type".
- If you try to load the HTML file locally (i.e., with a `file://` URL), you'll run into CORS errors due to JavaScript module security requirements. You need to do your testing through a server. GitHub pages is ideal as it also serves `.mjs` files with the correct MIME type.
- Because `.mjs` is a non-standard file extension, some operating systems might not recognize it, or try to replace it with something else. For example, we found that macOS was silently adding on `.js` to the end of `.mjs` files and then automatically hiding the file extension. So all of our files were actually coming out as `x.mjs.js`. Once we turned off automatically hiding file extensions, and trained it to accept `.mjs`, it was OK.

See also

- JavaScript modules on v8.dev (2018)
- ES modules: A cartoon deep-dive on hacks.mozilla.org (2018)
- ES6 in Depth: Modules on hacks.mozilla.org (2015)
- Exploring JS, Ch.16: Modules by Dr. Axel Rauschmayer

- Previous

Numbers and strings

- Previous
- Next

This chapter introduces the two most fundamental data types in JavaScript: numbers and strings. We will introduce their underlying representations, and functions used to work with and perform calculations on them.

Numbers

In JavaScript, numbers are implemented in double-precision 64-bit binary format IEEE 754 (i.e., a number between ±2^−1022 and ±2^+1023, or about ±10^−308 to ±10^+308, with a numeric precision of 53 bits). Integer values up to ±2^53 − 1 can be represented exactly.

In addition to being able to represent floating-point numbers, the number type has three symbolic values: `Infinity`, `-Infinity`, and `NaN` (not-a-number).

See also JavaScript data types and structures for context with other primitive types in JavaScript.

You can use four types of number literals: decimal, binary, octal, and hexadecimal.

Decimal numbers

```
1234567890
42
```

Decimal literals can start with a zero (`0`) followed by another decimal digit, but if all digits after the leading `0` are smaller than 8, the number is interpreted as an octal number. This is considered a legacy syntax, and number literals prefixed with `0`, whether interpreted as octal or decimal, cause a syntax error in strict mode — so, use the `0o` prefix instead.

```
0888 // 888 parsed as decimal
0777 // parsed as octal, 511 in decimal
```

Binary numbers

Binary number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "B" (`0b` or `0B`). If the digits after the `0b` are not 0 or 1, the following `SyntaxError` is thrown: "Missing binary digits after 0b".

```
0b10000000000000000000000000000000 // 2147483648
0b01111111100000000000000000000000 // 2139095040
0B00000000011111111111111111111111 // 8388607
```

Octal numbers

The standard syntax for octal numbers is to prefix them with `0o`. For example:

```
0O755 // 493
0o644 // 420
```

There's also a legacy syntax for octal numbers — by prefixing the octal number with a zero: `0644 === 420` and `"\045" === "%"`. If the digits after the `0` are outside the range 0 through 7, the number will be interpreted as a decimal number.

```
const n = 0755; // 493
const m = 0644; // 420
```

Strict mode forbids this octal syntax.

Hexadecimal numbers

Hexadecimal number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "X" (`0x` or `0X`). If the digits after 0x are outside the range (0123456789ABCDEF), the following `SyntaxError` is thrown: "Identifier starts immediately after numeric literal".

```
0xFFFFFFFFFFFFF // 4503599627370495
0xabcdef123456  // 188900967593046
0XA             // 10
```

Exponentiation

```
0e-5   // 0
0e+5   // 0
5e1    // 50
175e-2 // 1.75
1e3    // 1000
```

```
1e-3   // 0.001
1E3    // 1000
```

Numeric separators

For all literal syntaxes shown above, an underscore (_) can be inserted between digits to improve readability.

```
1_000_000_000_000
1_050.95
0b1010_0001_1000_0101
0o2_2_5_6
0xA0_B0_C0
1_000_000_000_000_000_000_000n
```

See the lexical grammar reference for more details about number literals.

Number object

The built-in `Number` object has properties for numerical constants, such as maximum value, not-a-number, and infinity. You cannot change the values of these properties and you use them as follows:

```
const biggestNum = Number.MAX_VALUE;
const smallestNum = Number.MIN_VALUE;
const infiniteNum = Number.POSITIVE_INFINITY;
const negInfiniteNum = Number.NEGATIVE_INFINITY;
const notANum = Number.NaN;
```

You always refer to a property of the predefined `Number` object as shown above, and not as a property of a `Number` object you create yourself.

The following table summarizes the `Number` object's properties.

Property	Description
`Number.MAX_VALUE`	The largest positive representable number (`1.7976931348623157e+308`)
`Number.MIN_VALUE`	The smallest positive representable number (`5e-324`)
`Number.NaN`	Special "not a number" value
`Number.NEGATIVE_INFINITY`	Special negative infinite value; returned on overflow
`Number.POSITIVE_INFINITY`	Special positive infinite value; returned on overflow
`Number.EPSILON`	Difference between `1` and the smallest value greater than `1` that can be represented as a `Number` (`2.220446049250313e-16`)
`Number.MIN_SAFE_INTEGER`	Minimum safe integer in JavaScript (−2^53 + 1, or `-9007199254740991`)
`Number.MAX_SAFE_INTEGER`	Maximum safe integer in JavaScript (+2^53 − 1, or `+9007199254740991`)

Method	Description
`Number.parseFloat()`	Parses a string argument and returns a floating point number. Same as the global `parseFloat()` function.
`Number.parseInt()`	Parses a string argument and returns an integer of the specified radix or base. Same as the global `parseInt()` function.
`Number.isFinite()`	Determines whether the passed value is a finite number.
`Number.isInteger()`	Determines whether the passed value is an integer.
`Number.isNaN()`	Determines whether the passed value is `NaN`. More robust version of the original global `isNaN()`.
`Number.isSafeInteger()`	Determines whether the provided value is a number that is a *safe integer*.

The `Number` prototype provides methods for retrieving information from `Number` objects in various formats. The following table summarizes the methods of `Number.prototype`.

Method	Description
`toExponential()`	Returns a string representing the number in exponential notation.
`toFixed()`	Returns a string representing the number in fixed-point notation.
`toPrecision()`	Returns a string representing the number to a specified precision in fixed-point notation.

Math object

The built-in `Math` object has properties and methods for mathematical constants and functions. For example, the `Math` object's `PI` property has the value of pi (3.141...), which you would use in an application as

```
Math.PI;
```

Similarly, standard mathematical functions are methods of `Math`. These include trigonometric, logarithmic, exponential, and other functions. For example, if you want to use the trigonometric function sine, you would write

```
Math.sin(1.56);
```

Note that all trigonometric methods of `Math` take arguments in radians.

The following table summarizes the `Math` object's methods.

Methods of `Math`

Method	Description
`abs()`	Absolute value
`sin()`, `cos()`, `tan()`	Standard trigonometric functions; with the argument in radians.
`asin()`, `acos()`, `atan()`, `atan2()`	Inverse trigonometric functions; return values in radians.
`sinh()`, `cosh()`, `tanh()`	Hyperbolic functions; argument in hyperbolic angle.
`asinh()`, `acosh()`, `atanh()`	Inverse hyperbolic functions; return values in hyperbolic angle.
`pow()`, `exp()`, `expm1()`, `log()`, `log10()`, `log1p()`, `log2()`	Exponential and logarithmic functions.
`floor()`, `ceil()`	Returns the largest/smallest integer less/greater than or equal to an argument.
`min()`, `max()`	Returns the minimum or maximum (respectively) value of a comma separated list of numbers as arguments.
`random()`	Returns a random number between 0 and 1.
`round()`, `fround()`, `trunc()`,	Rounding and truncation functions.
`sqrt()`, `cbrt()`, `hypot()`	Square root, cube root, Square root of the sum of square arguments.
`sign()`	The sign of a number, indicating whether the number is positive, negative or zero.
`clz32()`, `imul()`	Number of leading zero bits in the 32-bit binary representation. The result of the C-like 32-bit multiplication of the two arguments.

Unlike many other objects, you never create a `Math` object of your own. You always use the built-in `Math` object.

BigInts

One shortcoming of number values is they only have 64 bits. In practice, due to using IEEE 754 encoding, they cannot represent any integer larger than `Number.MAX_SAFE_INTEGER` (which is 2^{53} - 1) accurately. To solve the need of encoding binary data and to interoperate with other languages that offer wide integers like `i64` (64-bit integers) and `i128` (128-bit integers), JavaScript also offers another data type to represent *arbitrarily large integers*: `BigInt`.

A BigInt can be defined as an integer literal suffixed by `n`:

```
const b1 = 123n;
// Can be arbitrarily large.
const b2 = -1234567890987654321n;
```

BigInts can also be constructed from number values or string values using the `BigInt` constructor.

```
const b1 = BigInt(123);
// Using a string prevents loss of precision, since long number
// literals don't represent what they seem like.
const b2 = BigInt("-1234567890987654321");
```

Conceptually, a BigInt is just an arbitrarily long sequence of bits which encodes an integer. You can safely do any arithmetic operations without losing precision or over-/underflowing.

```
const integer = 12 ** 34; // 4.9222352429520264e+36; only has limited precision
const bigint = 12n ** 34n; // 4922235242952026704037113243122008064n
```

Compared to numbers, BigInt values yield higher precision when representing large *integers*; however, they cannot represent *floating-point numbers*. For example, division would round to zero:

```
const bigintDiv = 5n / 2n; // 2n, because there's no 2.5 in BigInt
```

`Math` functions cannot be used on BigInt values; they only work with numbers.

Choosing between BigInt and number depends on your use-case and your input's range. The precision of numbers should be able to accommodate most day-to-day tasks already, and BigInts are most suitable for handling binary data.

Read more about what you can do with BigInt values in the Expressions and Operators section, or the BigInt reference.

Strings

JavaScript's String type is used to represent textual data. It is a set of "elements" of 16-bit unsigned integer values (UTF-16 code units). Each element in the String occupies a position in the String. The first element is at index 0, the next at index 1, and so on. The length of a String is the number of elements in it. You can create strings using string literals or string objects.

String literals

You can declare strings in source code using either single or double quotes:

```
'foo'
"bar"
```

Within a string literal, most characters can be entered literally. The only exceptions are the backslash (`\`, which starts an escape sequence), the quote character being used to enclose the string, which terminates the string, and the newline character, which is a syntax error if not preceded by a backslash.

More advanced strings can be created using escape sequences:

Hexadecimal escape sequences

The number after `\x` is interpreted as a hexadecimal number.

```
"\xA9" // "©"
```

Unicode escape sequences

The Unicode escape sequences require at least four hexadecimal digits following `\u`.

```
"\u00A9" // "©"
```

Unicode code point escapes

With Unicode code point escapes, any character can be escaped using hexadecimal numbers so that it is possible to use Unicode code points up to `0x10FFFF`. With the four-digit Unicode escapes it is often necessary to write the surrogate halves separately to achieve the same result.

See also `String.fromCodePoint()` or `String.prototype.codePointAt()`.

```
"\u{2F804}"

// the same with simple Unicode escapes
"\uD87E\uDC04"
```

String object

You can call methods directly on a string value:

```
console.log("hello".toUpperCase()); // HELLO
```

The following methods are available on `String` values:

- Query: get the character or character code at a particular string index. Methods include `at()`, `charAt()`, `charCodeAt()`, and `codePointAt()`.
- Search: get information about a substring that conforms to a pattern, or test if a particular substring exists. Methods include `indexOf()`, `lastIndexOf()`, `startsWith()`, `endsWith()`, `includes()`, `match()`, `matchAll()`, and `search()`
- Composition: combine strings into a longer string. Methods include `padStart()`, `padEnd()`, `concat()`, and `repeat()`.
- Decomposition: break a string into smaller strings. Methods include `split()`, `slice()`, `substring()`, `substr()`, `trim()`, `trimStart()`, and `trimEnd()`.
- Transformation: return a new string based on the current string's content. Methods include `toLowerCase()`, `toUpperCase()`, `toLocaleLowerCase()`, `toLocaleUpperCase()`, `normalize()`, and `toWellFormed()`.

When working with strings, there are two other objects that provide important functionality for string manipulation: `RegExp` and `Intl`. They are introduced in regular expressions and internationalization respectively.

Template literals

Template literals are string literals allowing embedded expressions. You can use multi-line strings and string interpolation features with them.

Template literals are enclosed by backtick (grave accent) characters (`` ` ``) instead of double or single quotes. Template literals can contain placeholders. These are indicated by the dollar sign and curly braces (`${expression}`).

Multi-lines

Any new line characters inserted in the source are part of the template literal. Using normal strings, you would have to use the following syntax in order to get multi-line strings:

```
console.log(
  "string text line 1\n\
string text line 2",
);
// "string text line 1
// string text line 2"
```

To get the same effect with multi-line strings, you can now write:

```
console.log(`string text line 1
string text line 2`);
// "string text line 1
// string text line 2"
```

Embedded expressions

In order to embed expressions within normal strings, you would use the following syntax:

```
const five = 5;
const ten = 10;
```

```
console.log(
  "Fifteen is " + (five + ten) + " and not " + (2 * five + ten) + ".",
);
// "Fifteen is 15 and not 20."
```

Now, with template literals, you are able to make use of the syntactic sugar making substitutions like this more readable:

```
const five = 5;
const ten = 10;
console.log(`Fifteen is ${five + ten} and not ${2 * five + ten}.`);
// "Fifteen is 15 and not 20."
```

For more information, read about Template literals in the JavaScript reference.

- Previous
- Next

Regular expressions

- Previous
- Next

Regular expressions are patterns used to match character combinations in strings. In JavaScript, regular expressions are also objects. These patterns are used with the `exec()` and `test()` methods of `RegExp`, and with the `match()`, `matchAll()`, `replace()`, `replaceAll()`, `search()`, and `split()` methods of `String`. This chapter describes JavaScript regular expressions. It provides a brief overview of each syntax element. For a detailed explanation of each one's semantics, read the regular expressions reference.

Creating a regular expression

You construct a regular expression in one of two ways:

- Using a regular expression literal, which consists of a pattern enclosed between slashes, as follows:

  ```
  const re = /ab+c/;
  ```

  Regular expression literals provide compilation of the regular expression when the script is loaded. If the regular expression remains constant, using this can improve performance.

- Or calling the constructor function of the `RegExp` object, as follows:

  ```
  const re = new RegExp("ab+c");
  ```

  Using the constructor function provides runtime compilation of the regular expression. Use the constructor function when you know the regular expression pattern will be changing, or you don't know the pattern and are getting it from another source, such as user input.

Writing a regular expression pattern

A regular expression pattern is composed of simple characters, such as `/abc/`, or a combination of simple and special characters, such as `/ab*c/` or `/Chapter (\d+)\.\d*/`. The last example includes parentheses, which are used as a memory device. The match made with this part of the pattern is remembered for later use, as described in Using groups.

Using simple patterns

Simple patterns are constructed of characters for which you want to find a direct match. For example, the pattern `/abc/` matches character combinations in strings only when the exact sequence `"abc"` occurs (all characters together and in that order). Such a match would succeed in the strings `"Hi, do you know your abc's?"` and `"The latest airplane designs evolved from slabcraft."`. In both cases the match is with the substring `"abc"`. There is no match in the string `"Grab crab"` because while it contains the substring `"ab c"`, it does not contain the exact substring `"abc"`.

Using special characters

When the search for a match requires something more than a direct match, such as finding one or more b's, or finding white space, you can include special characters in the pattern. For example, to match *a single `"a"` followed by zero or more `"b"`s followed by `"c"`*, you'd use the pattern `/ab*c/`: the `*` after `"b"` means "0 or more occurrences of the preceding item." In the string `"cbbabbbbcdebc"`, this pattern will match the substring `"abbbbc"`.

The following pages provide lists of the different special characters that fit into each category, along with descriptions and examples.

Assertions guide

> Assertions include boundaries, which indicate the beginnings and endings of lines and words, and other patterns indicating in some way that a match is possible (including look-ahead, look-behind, and conditional expressions).

Character classes guide

> Distinguish different types of characters. For example, distinguishing between letters and digits.

Groups and backreferences guide

> Groups group multiple patterns as a whole, and capturing groups provide extra submatch information when using a regular expression pattern to match against a string. Backreferences refer to a previously captured group in the same regular expression.

Quantifiers guide

Indicate numbers of characters or expressions to match.

If you want to look at all the special characters that can be used in regular expressions in a single table, see the following:

Special characters in regular expressions.

Characters / constructs	Corresponding article
`[xyz]`, `[^xyz]`, `.`, `\d`, `\D`, `\w`, `\W`, `\s`, `\S`, `\t`, `\r`, `\n`, `\v`, `\f`, `[\b]`, `\0`, `\cX`, `\xHH`, `\uHHHH`, `\u{H…H}`, `x|y`	Character classes
`^`, `$`, `\b`, `\B`, `x(?=y)`, `x(?!y)`, `(?<=y)x`, `(?<!y)x`	Assertions
`(x)`, `(?<Name>x)`, `(?:x)`, `\n`, `\k<Name>`	Groups and backreferences
`x*`, `x+`, `x?`, `x{n}`, `x{n,}`, `x{n,m}`	Quantifiers

> **Note:** A larger cheat sheet is also available (only aggregating parts of those individual articles).

Escaping

If you need to use any of the special characters literally (actually searching for a `"*"`, for instance), you should escape it by putting a backslash in front of it. For instance, to search for `"a"` followed by `"*"` followed by `"b"`, you'd use `/a\*b/` — the backslash "escapes" the `"*"`, making it literal instead of special.

> **Note:** In many cases, when trying match a special character, you can wrap it in a character class as an alternative to escaping, for example `/a[*]b/`.

Similarly, if you're writing a regular expression literal and need to match a slash ("/"), you need to escape that (otherwise, it terminates the pattern). For instance, to search for the string "/example/" followed by one or more alphabetic characters, you'd use `/\/example\/[a-z]+/i`—the backslashes before each slash make them literal.

To match a literal backslash, you need to escape the backslash. For instance, to match the string "C:\" where "C" can be any letter, you'd use `/[A-Z]:\\/` — the first backslash escapes the one after it, so the expression searches for a single literal backslash.

If using the `RegExp` constructor with a string literal, remember that the backslash is an escape in string literals, so to use it in the regular expression, you need to escape it at the string literal level. `/a\*b/` and `new RegExp("a\\*b")` create the same expression, which searches for "a" followed by a literal "*" followed by "b".

The `RegExp.escape()` function returns a new string where all special characters in regex syntax are escaped. This allows you to do `new RegExp(RegExp.escape("a*b"))` to create a regular expression that matches only the string `"a*b"`.

Using parentheses

Parentheses around any part of the regular expression pattern causes that part of the matched substring to be remembered. Once remembered, the substring can be recalled for other use. See Groups and backreferences for more details.

Using regular expressions in JavaScript

Regular expressions are used with the `RegExp` methods `test()` and `exec()` and with the `String` methods `match()`, `matchAll()`, `replace()`, `replaceAll()`, `search()`, and `split()`.

Method	Description
`exec()`	Executes a search for a match in a string. It returns an array of information or `null` on a mismatch.
`test()`	Tests for a match in a string. It returns `true` or `false`.
`match()`	Returns an array containing all of the matches, including capturing groups, or `null` if no match is found.
`matchAll()`	Returns an iterator containing all of the matches, including capturing groups.
`search()`	Tests for a match in a string. It returns the index of the match, or `-1` if the search fails.
`replace()`	Executes a search for a match in a string, and replaces the matched substring with a replacement substring.
`replaceAll()`	Executes a search for all matches in a string, and replaces the matched substrings with a replacement substring.
`split()`	Uses a regular expression or a fixed string to break a string into an array of substrings.

When you want to know whether a pattern is found in a string, use the `test()` or `search()` methods; for more information (but slower execution) use the `exec()` or `match()` methods. If you use `exec()` or `match()` and if the match succeeds, these methods return an array and update properties of the associated regular expression object and also of the predefined regular expression object, `RegExp`. If the match fails, the `exec()` method returns `null` (which coerces to `false`).

In the following example, the script uses the `exec()` method to find a match in a string.

```
const myRe = /d(b+)d/g;
const myArray = myRe.exec("cdbbdbsbz");
```

If you do not need to access the properties of the regular expression, an alternative way of creating `myArray` is with this script:

```
const myArray = /d(b+)d/g.exec("cdbbdbsbz");
// similar to 'cdbbdbsbz'.match(/d(b+)d/g); however,
// 'cdbbdbsbz'.match(/d(b+)d/g) outputs [ "dbbd" ]
// while /d(b+)d/g.exec('cdbbdbsbz') outputs [ 'dbbd', 'bb', index: 1, input: 'cdbbdbsbz' ]
```

(See Using the global search flag with `exec()` for further info about the different behaviors.)

If you want to construct the regular expression from a string, yet another alternative is this script:

```
const myRe = new RegExp("d(b+)d", "g");
const myArray = myRe.exec("cdbbdbsbz");
```

With these scripts, the match succeeds and returns the array and updates the properties shown in the following table.

Results of regular expression execution.

Object	Property or index	Description	In this example
`myArray`		The matched string and all remembered substrings.	`['dbbd', 'bb', index: 1, input: 'cdbbdbsbz']`
	`index`	The 0-based index of the match in the input string.	`1`
	`input`	The original string.	`'cdbbdbsbz'`
	`[0]`	The last matched characters.	`'dbbd'`
`myRe`	`lastIndex`	The index at which to start the next match. (This property is set only if the regular expression uses the g option, described in Advanced Searching With Flags.)	`5`
	`source`	The text of the pattern. Updated at the time that the regular expression is created, not executed.	`'d(b+)d'`

As shown in the second form of this example, you can use a regular expression created with an object initializer without assigning it to a variable. If you do, however, every occurrence is a new regular expression. For this reason, if you use this form without assigning it to a variable, you cannot subsequently access the properties of that regular expression. For example, assume you have this script:

```
const myRe = /d(b+)d/g;
const myArray = myRe.exec("cdbbdbsbz");
console.log(`The value of lastIndex is ${myRe.lastIndex}`);

// "The value of lastIndex is 5"
```

However, if you have this script:

```
const myArray = /d(b+)d/g.exec("cdbbdbsbz");
console.log(`The value of lastIndex is ${/d(b+)d/g.lastIndex}`);

// "The value of lastIndex is 0"
```

The occurrences of `/d(b+)d/g` in the two statements are different regular expression objects and hence have different values for their `lastIndex` property. If you need to access the properties of a regular expression created with an object initializer, you should first assign it to a variable.

Advanced searching with flags

Regular expressions have optional flags that allow for functionality like global searching and case-insensitive searching. These flags can be used separately or together in any order, and are included as part of the regular expression.

Flag	Description	Corresponding property
`d`	Generate indices for substring matches.	`hasIndices`
`g`	Global search.	`global`
`i`	Case-insensitive search.	`ignoreCase`
`m`	Makes `^` and `$` match the start and end of each line instead of those of the entire string.	`multiline`
`s`	Allows `.` to match newline characters.	`dotAll`
`u`	"Unicode"; treat a pattern as a sequence of Unicode code points.	`unicode`
`v`	An upgrade to the `u` mode with more Unicode features.	`unicodeSets`
`y`	Perform a "sticky" search that matches starting at the current position in the target string.	`sticky`

To include a flag with the regular expression, use this syntax:

```
const re = /pattern/flags;
```

or

```
const re = new RegExp("pattern", "flags");
```

Note that the flags are an integral part of a regular expression. They cannot be added or removed later.

For example, `re = /\w+\s/g` creates a regular expression that looks for one or more characters followed by a space, and it looks for this combination throughout the string.

```
const re = /\w+\s/g;
const str = "fee fi fo fum";
const myArray = str.match(re);
console.log(myArray);

// ["fee ", "fi ", "fo "]
```

You could replace the line:

```
const re = /\w+\s/g;
```

with:

```
const re = new RegExp("\\w+\\s", "g");
```

and get the same result.

The `m` flag is used to specify that a multiline input string should be treated as multiple lines. If the `m` flag is used, `^` and `$` match at the start or end of any line within the input string instead of the start or end of the entire string.

The `i`, `m`, and `s` flags can be enabled or disabled for specific parts of a regex using the modifier syntax.

Using the global search flag with exec()

`RegExp.prototype.exec()` method with the `g` flag returns each match and its position iteratively.

```
const str = "fee fi fo fum";
const re = /\w+\s/g;

console.log(re.exec(str)); // ["fee ", index: 0, input: "fee fi fo fum"]
console.log(re.exec(str)); // ["fi ", index: 4, input: "fee fi fo fum"]
console.log(re.exec(str)); // ["fo ", index: 7, input: "fee fi fo fum"]
console.log(re.exec(str)); // null
```

In contrast, `String.prototype.match()` method returns all matches at once, but without their position.

```
console.log(str.match(re)); // ["fee ", "fi ", "fo "]
```

Using unicode regular expressions

The `u` flag is used to create "unicode" regular expressions; that is, regular expressions which support matching against unicode text. An important feature that's enabled in unicode mode is Unicode property escapes. For example, the following regular expression might be used to match against an arbitrary unicode "word":

```
/\p{L}*/u;
```

Unicode regular expressions have different execution behavior as well. `RegExp.prototype.unicode` contains more explanation about this.

Examples

> **Note:** Several examples are also available in:
>
> - The reference pages for `exec()`, `test()`, `match()`, `matchAll()`, `search()`, `replace()`, `split()`
> - The guide articles: character classes, assertions, groups and backreferences, quantifiers

Using special characters to verify input

In the following example, the user is expected to enter a phone number. When the user presses the "Check" button, the script checks the validity of the number. If the number is valid (matches the character sequence specified by the regular expression), the script shows a message thanking the user and confirming the number. If the number is invalid, the script informs the user that the phone number is not valid.

The regular expression looks for:

1. the beginning of the line of data: `^`
2. followed by three numeric characters `\d{3}` OR | a left parenthesis `\(`, followed by three digits `\d{3}`, followed by a close parenthesis `\)`, in a non-capturing group `(?:)`
3. followed by one dash, forward slash, or decimal point in a capturing group `()`
4. followed by three digits `\d{3}`
5. followed by the match remembered in the (first) captured group `\1`
6. followed by four digits `\d{4}`
7. followed by the end of the line of data: `$`

HTML

```
<p>
  Enter your phone number (with area code) and then click "Check".
  <br />
```

```
  The expected format is like ###-###-####.
</p>
<form id="form">
  <input id="phone" />
  <button type="submit">Check</button>
</form>
<p id="output"></p>
```

JavaScript

```
const form = document.querySelector("#form");
const input = document.querySelector("#phone");
const output = document.querySelector("#output");

const re = /^(?:\d{3}|\(\d{3}\))([-/.])\d{3}\1\d{4}$/;

function testInfo(phoneInput) {
  const ok = re.exec(phoneInput.value);

  output.textContent = ok
    ? `Thanks, your phone number is ${ok[0]}`
    : `${phoneInput.value} isn't a phone number with area code!`;
}

form.addEventListener("submit", (event) => {
  event.preventDefault();
  testInfo(input);
});
```

Result

Tools

RegExr

: An online tool to learn, build, & test Regular Expressions.

Regex tester

: An online regex builder/debugger

Regex interactive tutorial

: An online interactive tutorials, Cheat sheet, & Playground.

Regex visualizer

: An online visual regex tester.

- Previous
- Next

Assertions

Assertions include boundaries, which indicate the beginnings and endings of lines and words, and other patterns indicating in some way that a match is possible (including look-ahead, look-behind, and conditional expressions).

Try it

```
const text = "A quick fox";

const regexpLastWord = /\w+$/;
console.log(text.match(regexpLastWord));
// Expected output: Array ["fox"]

const regexpWords = /\b\w+\b/g;
console.log(text.match(regexpWords));
// Expected output: Array ["A", "quick", "fox"]

const regexpFoxQuality = /\w+(?= fox)/;
console.log(text.match(regexpFoxQuality));
// Expected output: Array ["quick"]
```

Types

Boundary-type assertions

Characters	Meaning
^	**Input boundary beginning assertion:** Matches the beginning of input. If the `multiline` (m) flag is enabled, also matches immediately after a line break character. For example, `/^A/` does not match the "A" in "an A", but does match the first "A" in "An A".

Characters	Meaning
	Note: This character has a different meaning when it appears at the start of a character class.
`$`	**Input boundary end assertion:** Matches the end of input. If the `multiline` (m) flag is enabled, also matches immediately before a line break character. For example, `/t$/` does not match the "t" in "eater", but does match it in "eat".
`\A`	**Buffer boundary start assertion:** Matches the start of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
`\z`	**Buffer boundary end assertion:** Matches the end of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
`\Z`	**Buffer boundary end assertion with optional newline:** Matches the end of the entire string, but allows an optional trailing newline character sequence (either a line terminator or a `\r\n` sequence). Only valid in Unicode-aware mode.
`\b`	**Word boundary assertion:** Matches a word boundary. This is the position where a word character is not followed or preceded by another word-character, such as between a letter and a space. Note that a matched word boundary is not included in the match. In other words, the length of a matched word boundary is zero. Examples: • `/\bm/` matches the "m" in "moon". • `/oo\b/` does not match the "oo" in "moon", because "oo" is followed by "n" which is a word character. • `/oon\b/` matches the "oon" in "moon", because "oon" is the end of the string, thus not followed by a word character. • `/\w\b\w/` will never match anything, because a word character can never be followed by both a non-word and a word character. To match a backspace character (`[\b]`), see Character Classes.
`\B`	**Non-word-boundary assertion:** Matches a non-word boundary. This is a position where the previous and next character are of the same type: Either both must be words, or both must be non-words, for example between two letters or between two spaces. The beginning and end of a string are considered non-words. Same as the matched word boundary, the matched non-word boundary is also not included in the match. For example, `/\Bon/` matches "on" in "at noon", and `/ye\B/` matches "ye" in "possibly yesterday".

Other assertions

> **Note:** The `?` character may also be used as a quantifier.

Characters	Meaning
`x(?=y)`	**Lookahead assertion:** Matches "x" only if "x" is followed by "y". For example, `/Jack(?=Sprat)/` matches "Jack" only if it is followed by "Sprat". `/Jack(?=Sprat|Frost)/` matches "Jack" only if it is followed by "Sprat" or "Frost". However, neither "Sprat" nor "Frost" is part of the match results.
`x(?!y)`	**Negative lookahead assertion:** Matches "x" only if "x" is not followed by "y". For example, `/\d+(?!\.)/` matches a number only if it is not followed by a decimal point. `/\d+(?!\.)/.exec('3.141')` matches "141" but not "3".
`(?<=y)x`	**Lookbehind assertion:** Matches "x" only if "x" is preceded by "y". For example, `/(?<=Jack)Sprat/` matches "Sprat" only if it is preceded by "Jack". `/(?<=Jack|Tom)Sprat/` matches "Sprat" only if it is preceded by "Jack" or "Tom". However, neither "Jack" nor "Tom" is part of the match results.
`(?<!y)x`	**Negative lookbehind assertion:** Matches "x" only if "x" is not preceded by "y". For example, `/(?<!-)\d+/` matches a number only if it is not preceded by a minus sign. `/(?<!-)\d+/.exec('3')` matches "3". `/(?<!-)\d+/.exec('-3')` match is not found because the number is preceded by the minus sign.

Examples

General boundary-type overview example

```
// Using Regex boundaries to fix buggy string.
buggyMultiline = `tey, ihe light-greon apple
tangs on ihe greon traa`;

// 1) Use ^ to fix the matching at the beginning of the string, and right after newline.
buggyMultiline = buggyMultiline.replace(/^t/gim, "h");
```

```
console.log(1, buggyMultiline); // fix 'tey' => 'hey' and 'tangs' => 'hangs' but do not touch 'traa'.

// 2) Use $ to fix matching at the end of the text.
buggyMultiline = buggyMultiline.replace(/aa$/gim, "ee.");
console.log(2, buggyMultiline); // fix 'traa' => 'tree.'.

// 3) Use \b to match characters right on border between a word and a space.
buggyMultiline = buggyMultiline.replace(/\bi/gim, "t");
console.log(3, buggyMultiline); // fix 'ihe' => 'the' but do not touch 'light'.

// 4) Use \B to match characters inside borders of an entity.
fixedMultiline = buggyMultiline.replace(/\Bo/gim, "e");
console.log(4, fixedMultiline); // fix 'greon' => 'green' but do not touch 'on'.
```

Matching the beginning of input using a ^ control character

Use `^` for matching at the beginning of input. In this example, we can get the fruits that start with 'A' by a `/^A/` regex. For selecting appropriate fruits we can use the `filter` method with an arrow function.

```
const fruits = ["Apple", "Watermelon", "Orange", "Avocado", "Strawberry"];

// Select fruits started with 'A' by /^A/ Regex.
// Here '^' control symbol used only in one role: Matching beginning of an input.

const fruitsStartsWithA = fruits.filter((fruit) => /^A/.test(fruit));
console.log(fruitsStartsWithA); // [ 'Apple', 'Avocado' ]
```

In the second example `^` is used both for matching at the beginning of input and for creating negated or complemented character class when used within character classes.

```
const fruits = ["Apple", "Watermelon", "Orange", "Avocado", "Strawberry"];

// Selecting fruits that do not start by 'A' with a /^[^A]/ regex.
// In this example, two meanings of '^' control symbol are represented:
// 1) Matching beginning of the input
// 2) A negated or complemented character class: [^A]
// That is, it matches anything that is not enclosed in the square brackets.

const fruitsStartsWithNotA = fruits.filter((fruit) => /^[^A]/.test(fruit));

console.log(fruitsStartsWithNotA); // [ 'Watermelon', 'Orange', 'Strawberry' ]
```

See more examples in the input boundary assertion reference.

Matching a word boundary

In this example, we match fruit names containing a word that ends in "en" or "ed".

```
const fruitsWithDescription = ["Red apple", "Orange orange", "Green Avocado"];

// Select descriptions that contains 'en' or 'ed' words endings:
const enEdSelection = fruitsWithDescription.filter((description) =>
  /(?:en|ed)\b/.test(description),
);

console.log(enEdSelection); // [ 'Red apple', 'Green Avocado' ]
```

See more examples in the word boundary assertion reference.

Lookahead assertion

In this example, we match the word "First" only if it is followed by the word "test", but we do not include "test" in the match results.

```
const regex = /First(?= test)/g;

console.log("First test".match(regex)); // [ 'First' ]
console.log("First peach".match(regex)); // null
console.log("This is a First test in a year.".match(regex)); // [ 'First' ]
console.log("This is a First peach in a month.".match(regex)); // null
```

See more examples in the lookahead assertion reference.

Basic negative lookahead assertion

For example, `/\d+(?!\.)/` matches a number only if it is not followed by a decimal point. `/\d+(?!\.)/.exec('3.141')` matches "141" but not "3".

```
console.log(/\d+(?!\.)/g.exec("3.141")); // [ '141', index: 2, input: '3.141' ]
```

See more examples in the lookahead assertion reference.

Different meaning of '?!' combination usage in assertions and character classes

The `?!` combination has different meanings in assertions like `/x(?!y)/` and character classes like `[^?!]`.

```
const orangeNotLemon =
  "Do you want to have an orange? Yes, I do not want to have a lemon!";

// Different meaning of '?!' combination usage in Assertions /x(?!y)/ and Ranges /[^?!]/
const selectNotLemonRegex = /[^?!]+have(?! a lemon)[^?!]+[?!]/gi;
console.log(orangeNotLemon.match(selectNotLemonRegex)); // [ 'Do you want to have an orange?' ]

const selectNotOrangeRegex = /[^?!]+have(?! an orange)[^?!]+[?!]/gi;
console.log(orangeNotLemon.match(selectNotOrangeRegex)); // [ ' Yes, I do not want to have a lemon!' ]
```

Lookbehind assertion

In this example, we replace the word "orange" with "apple" only if it is preceded by the word "ripe".

```
const oranges = ["ripe orange A", "green orange B", "ripe orange C"];

const newFruits = oranges.map((fruit) =>
  fruit.replace(/(?<=ripe )orange/, "apple"),
);
console.log(newFruits); // ['ripe apple A', 'green orange B', 'ripe apple C']
```

See more examples in the lookbehind assertion reference.

See also

- Regular expressions guide
- Character classes guide
- Quantifiers guide
- Groups and backreferences guide
- `RegExp`
- Regular expressions reference
- Input boundary assertion: `^`, `$`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Word boundary assertion: `\b`, `\B`

Character classes

Character classes distinguish kinds of characters such as, for example, distinguishing between letters and digits.

Try it

```
const chessStory = "He played the King in a8 and she moved her Queen in c2.";
const regexpCoordinates = /\w\d/g;
console.log(chessStory.match(regexpCoordinates));
// Expected output: Array [ 'a8', 'c2']

const moods = "happy 😊, confused 😳, sad 😳";
const regexpEmoticons = /[\u{1F600}-\u{1F64F}]/gu;
console.log(moods.match(regexpEmoticons));
// Expected output: Array ['😊', '😳', '😳']
```

Types

Characters	Meaning
`[xyz]` `[a-c]`	**Character class:** Matches any one of the enclosed characters. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first or last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[abcd]` is the same as `[a-d]`. They match the "b" in "brisket", and the "c" in "chop". For example, `[abcd-]` and `[-abcd]` match the "b" in "brisket", the "c" in "chop", and the "-" (hyphen) in "non-profit". For example, `[\w-]` is the same as `[A-Za-z0-9_-]`. They both match the "b" in "brisket", the "c" in "chop", and the "n" in "non-profit". When the `unicodeSets` (`v`) flag is enabled, the character class has some additional features. See the character class reference for more information.
`[^xyz]` `[^a-c]`	**Negated character class:** Matches anything that is not enclosed in the square brackets. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first character after the `^` or the last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[^abc]` is the same as `[^a-c]`. They initially match "o" in "bacon" and "h" in "chop". > **Note:** The ^ character may also indicate the beginning of input.

Characters	Meaning
`.`	**Wildcard:** Matches any single character *except* line terminators: `\n`, `\r`, `\u2028` or `\u2029`. For example, `/.y/` matches "my" and "ay", but not "yes", in "yes make my day", as there is no character before "y" in "yes". If the `dotAll` (s) flag is enabled, also matches line terminators. Inside a character class, the dot loses its special meaning and matches a literal dot.
`\d`	**Digit character class escape:** Matches any digit (Arabic numeral). Equivalent to `[0-9]`. For example, `/\d/` or `/[0-9]/` matches "2" in "B2 is the suite number".
`\D`	**Non-digit character class escape:** Matches any character that is not a digit (Arabic numeral). Equivalent to `[^0-9]`. For example, `/\D/` or `/[^0-9]/` matches "B" in "B2 is the suite number".
`\w`	**Word character class escape:** Matches any alphanumeric character from the basic Latin alphabet, including the underscore. Equivalent to `[A-Za-z0-9_]`. For example, `/\w/` matches "a" in "apple", "5" in "$5.28", "3" in "3D" and "m" in "Émanuel".
`\W`	**Non-word character class escape:** Matches any character that is not a word character from the basic Latin alphabet. Equivalent to `[^A-Za-z0-9_]`. For example, `/\W/` or `/[^A-Za-z0-9_]/` matches "%" in "50%" and "É" in "Émanuel".
`\s`	**White space character class escape:** Matches a single white space character, including space, tab, form feed, line feed, and other Unicode spaces. Equivalent to `[\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\s\w*/` matches " bar" in "foo bar".
`\S`	**Non-white space character class escape:** Matches a single character other than white space. Equivalent to `[^\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\S\w*/` matches "foo" in "foo bar".
`\t`	Matches a horizontal tab.
`\r`	Matches a carriage return.
`\n`	Matches a linefeed.
`\v`	Matches a vertical tab.
`\f`	Matches a form-feed.
`[\b]`	Matches a backspace. If you're looking for the word-boundary assertion (`\b`), see Assertions.
`\0`	Matches a NUL character. Do not follow this with another digit.
`\c`*`X`*	Matches a control character using caret notation, where "X" is a letter from A–Z or a–z (corresponding to code points `U+0001`–`U+001A`). For example, `/\cM\cJ/` matches "\r\n".
`\x`*`hh`*	**Hex escape:** Matches the character with the code *`hh`* (two hexadecimal digits).
`\u`*`HHHH`*	**Unicode escape:** Matches a UTF-16 code-unit with the value *`HHHH`* (four hexadecimal digits).
`\u{`*`H…H`*`}`	**Unicode code point escape:** Matches the character with the Unicode value `U+`*`H…H`* (1 to 6 hexadecimal digits). Only valid in Unicode-aware mode.
`\p{`*`UnicodeProperty`*`}`, `\P{`*`UnicodeProperty`*`}`	**Unicode character class escape:** Matches a character based on its Unicode character properties: for example, emoji characters, or Japanese *katakana* characters, or Chinese/Japanese Han/Kanji characters, etc.). Only valid in Unicode-aware mode.
`\`	Indicates that the following character should be treated specially, or "escaped". It behaves one of two ways. • For characters that are usually treated literally, indicates that the next character is special and not to be interpreted literally. For example, `/b/` matches the character "b". By placing a backslash in front of "b", that is by using `/\b/`, the character becomes special to mean match a word boundary. • For characters that are usually treated specially, indicates that the next character is not special and should be interpreted literally. For example, "*" is a special character that means 0 or more occurrences of the preceding character should be matched; for example, `/a*/` means match 0 or more "a"s. To match `*` literally, precede it with a backslash; for example, `/a\*/` matches "a*". **Note:** To match this character literally, escape it with itself. In other words to search for `\` use `/\\/`.

Characters	Meaning
x|*y*	**Disjunction:** Matches either "x" or "y". Each component, separated by a pipe (`|`), is called an *alternative*. For example, `/green|red/` matches "green" in "green apple" and "red" in "red apple". > **Note:** A disjunction is another way to specify "a set of choices", but it's not a character class. Disjunctions are not atoms — you need to use a group to make it part of a bigger pattern. `[abc]` is functionally equivalent to `(?:a|b|c)`.

Examples

Looking for a series of digits

In this example, we match a sequence of 4 digits with `\d{4}`. `\b` indicates a word boundary (i.e., do not start or end matching in the middle of a number sequence).

```
const randomData = "015 354 8787 687351 3512 8735";
const regexpFourDigits = /\b\d{4}\b/g;

console.table(randomData.match(regexpFourDigits));
// ['8787', '3512', '8735']
```

See more examples in the character class escape reference.

Looking for a word (from the latin alphabet) starting with A

In this example, we match a word starting with the letter A. `\b` indicates a word boundary (i.e., do not start matching in the middle of a word). `[aA]` indicates the letter "a" or "A". `\w+` indicates any character *from the Latin alphabet*, multiple times (`+` is a quantifier). Note that because we already match until there are no more word characters, an end `\b` boundary is not necessary.

```
const aliceExcerpt =
  "I'm sure I'm not Ada,' she said, 'for her hair goes in such long ringlets, and mine doesn't go in ringlets at all.";
const regexpWordStartingWithA = /\b[aA]\w+/g;

console.table(aliceExcerpt.match(regexpWordStartingWithA));
// ['Ada', 'and', 'at', 'all']
```

See more examples in the character class escape reference.

Looking for a word (from Unicode characters)

Instead of the Latin alphabet, we can use a range of Unicode characters to identify a word (thus being able to deal with text in other languages like Russian or Arabic). The "Basic Multilingual Plane" of Unicode contains most of the characters used around the world and we can use character classes and ranges to match words written with those characters.

```
const nonEnglishText = "Приключения Алисы в Стране чудес";
const regexpBMPWord = /([\u0000-\u0019\u0021-\uFFFF])+/gu;
// BMP goes through U+0000 to U+FFFF but space is U+0020

console.table(nonEnglishText.match(regexpBMPWord));
["Приключения", "Алисы", "в", "Стране", "чудес"];
```

See more examples in the Unicode character class escape reference.

Counting vowels

In this example, we count the number of vowels (A, E, I, O, U, Y) in a text. The `g` flag is used to match all occurrences of the pattern in the text. The `i` flag is used to make the pattern case-insensitive, so it matches both uppercase and lowercase vowels.

```
const aliceExcerpt =
  "There was a long silence after this, and Alice could only hear whispers now and then.";
const regexpVowels = /[aeiouy]/gi;

console.log("Number of vowels:", aliceExcerpt.match(regexpVowels).length);
// Number of vowels: 26
```

See also

- Regular expressions guide
- Assertions guide
- Quantifiers guide
- Groups and backreferences guide
- `RegExp`
- Regular expressions reference
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`
- Unicode character class escape: `\p{...}`, `\P{...}`

- Wildcard: .

Regular expression syntax cheat sheet

This page provides an overall cheat sheet of all the capabilities of `RegExp` syntax by aggregating the content of the articles in the `RegExp` guide. If you need more information on a specific topic, please follow the link on the corresponding heading to access the full article or head to the guide.

Character classes

Character classes distinguish kinds of characters such as, for example, distinguishing between letters and digits.

Characters	Meaning
`[xyz]` `[a-c]`	**Character class:** Matches any one of the enclosed characters. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first or last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[abcd]` is the same as `[a-d]`. They match the "b" in "brisket", and the "c" in "chop". For example, `[abcd-]` and `[-abcd]` match the "b" in "brisket", the "c" in "chop", and the "-" (hyphen) in "non-profit". For example, `[\w-]` is the same as `[A-Za-z0-9_-]`. They both match the "b" in "brisket", the "c" in "chop", and the "n" in "non-profit".
`[^xyz]` `[^a-c]`	**Negated character class:** Matches anything that is not enclosed in the square brackets. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first character after the `^` or the last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[^abc]` is the same as `[^a-c]`. They initially match "o" in "bacon" and "h" in "chop". **Note:** The ^ character may also indicate the beginning of input.
`.`	**Wildcard:** Matches any single character *except* line terminators: `\n`, `\r`, `\u2028` or `\u2029`. For example, `/.y/` matches "my" and "ay", but not "yes", in "yes make my day", as there is no character before "y" in "yes". If the `dotAll` (s) flag is enabled, also matches line terminators. Inside a character class, the dot loses its special meaning and matches a literal dot.
`\d`	**Digit character class escape:** Matches any digit (Arabic numeral). Equivalent to `[0-9]`. For example, `/\d/` or `/[0-9]/` matches "2" in "B2 is the suite number".
`\D`	**Non-digit character class escape:** Matches any character that is not a digit (Arabic numeral). Equivalent to `[^0-9]`. For example, `/\D/` or `/[^0-9]/` matches "B" in "B2 is the suite number".
`\w`	**Word character class escape:** Matches any alphanumeric character from the basic Latin alphabet, including the underscore. Equivalent to `[A-Za-z0-9_]`. For example, `/\w/` matches "a" in "apple", "5" in "$5.28", "3" in "3D" and "m" in "Émanuel".
`\W`	**Non-word character class escape:** Matches any character that is not a word character from the basic Latin alphabet. Equivalent to `[^A-Za-z0-9_]`. For example, `/\W/` or `/[^A-Za-z0-9_]/` matches "%" in "50%" and "É" in "Émanuel".
`\s`	**White space character class escape:** Matches a single white space character, including space, tab, form feed, line feed, and other Unicode spaces. Equivalent to `[\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\s\w*/` matches " bar" in "foo bar".
`\S`	**Non-white space character class escape:** Matches a single character other than white space. Equivalent to `[^\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\S\w*/` matches "foo" in "foo bar".
`\t`	Matches a horizontal tab.
`\r`	Matches a carriage return.
`\n`	Matches a linefeed.
`\v`	Matches a vertical tab.
`\f`	Matches a form-feed.
`[\b]`	Matches a backspace. If you're looking for the word-boundary assertion (`\b`), see Assertions.
`\0`	Matches a NUL character. Do not follow this with another digit.
`\c`*X*	Matches a control character using caret notation, where "X" is a letter from A–Z or a–z (corresponding to code points `U+0001`–`U+001A`). For example, `/\cM\cJ/` matches "\r\n".
`\x`*hh*	**Hex escape:** Matches the character with the code *hh* (two hexadecimal digits).

Characters	Meaning
`\uHHHH`	**Unicode escape:** Matches a UTF-16 code-unit with the value *HHHH* (four hexadecimal digits).
`\u{H…H}`	**Unicode code point escape:** Matches the character with the Unicode value *U+H…H* (1 to 6 hexadecimal digits). Only valid in Unicode-aware mode.
`\p{UnicodeProperty}`, `\P{UnicodeProperty}`	**Unicode character class escape:** Matches a character based on its Unicode character properties: for example, emoji characters, or Japanese *katakana* characters, or Chinese/Japanese Han/Kanji characters, etc.). Only valid in Unicode-aware mode.
`\`	Indicates that the following character should be treated specially, or "escaped". It behaves one of two ways. • For characters that are usually treated literally, indicates that the next character is special and not to be interpreted literally. For example, `/b/` matches the character "b". By placing a backslash in front of "b", that is by using `/\b/`, the character becomes special to mean match a word boundary. • For characters that are usually treated specially, indicates that the next character is not special and should be interpreted literally. For example, "*" is a special character that means 0 or more occurrences of the preceding character should be matched; for example, `/a*/` means match 0 or more "a"s. To match `*` literally, precede it with a backslash; for example, `/a\*/` matches "a*". **Note:** To match this character literally, escape it with itself. In other words to search for `\` use `/\\/`.
x|*y*	**Disjunction:** Matches either "x" or "y". Each component, separated by a pipe (`|`), is called an *alternative*. For example, `/green|red/` matches "green" in "green apple" and "red" in "red apple". **Note:** A disjunction is another way to specify "a set of choices", but it's not a character class. Disjunctions are not atoms — you need to use a group to make it part of a bigger pattern. `[abc]` is functionally equivalent to `(?:a|b|c)`.

Assertions

Assertions include boundaries, which indicate the beginnings and endings of lines and words, and other patterns indicating in some way that a match is possible (including look-ahead, look-behind, and conditional expressions).

Boundary-type assertions

Characters	Meaning
`^`	**Input boundary beginning assertion:** Matches the beginning of input. If the `multiline` (m) flag is enabled, also matches immediately after a line break character. For example, `/^A/` does not match the "A" in "an A", but does match the first "A" in "An A". **Note:** This character has a different meaning when it appears at the start of a character class.
`$`	**Input boundary end assertion:** Matches the end of input. If the `multiline` (m) flag is enabled, also matches immediately before a line break character. For example, `/t$/` does not match the "t" in "eater", but does match it in "eat".
`\A`	**Buffer boundary start assertion:** Matches the start of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
`\z`	**Buffer boundary end assertion:** Matches the end of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
`\Z`	**Buffer boundary end assertion with optional newline:** Matches the end of the entire string, but allows an optional trailing newline character sequence (either a line terminator or a `\r\n` sequence). Only valid in Unicode-aware mode.
`\b`	**Word boundary assertion:** Matches a word boundary. This is the position where a word character is not followed or preceded by another word-character, such as between a letter and a space. Note that a matched word boundary is not included in the match. In other words, the length of a matched word boundary is zero. Examples: • `/\bm/` matches the "m" in "moon". • `/oo\b/` does not match the "oo" in "moon", because "oo" is followed by "n" which is a word character. • `/oon\b/` matches the "oon" in "moon", because "oon" is the end of the string, thus not followed by a word character. • `/\w\b\w/` will never match anything, because a word character can never be followed by both a non-word and a word character. To match a backspace character (`[\b]`), see Character Classes.

Characters	Meaning
`\B`	**Non-word-boundary assertion:** Matches a non-word boundary. This is a position where the previous and next character are of the same type: Either both must be words, or both must be non-words, for example between two letters or between two spaces. The beginning and end of a string are considered non-words. Same as the matched word boundary, the matched non-word boundary is also not included in the match. For example, `/\Bon/` matches "on" in "at noon", and `/ye\B/` matches "ye" in "possibly yesterday".

Other assertions

> **Note:** The `?` character may also be used as a quantifier.

Characters	Meaning
`x(?=y)`	**Lookahead assertion:** Matches "x" only if "x" is followed by "y". For example, `/Jack(?=Sprat)/` matches "Jack" only if it is followed by "Sprat". `/Jack(?=Sprat|Frost)/` matches "Jack" only if it is followed by "Sprat" or "Frost". However, neither "Sprat" nor "Frost" is part of the match results.
`x(?!y)`	**Negative lookahead assertion:** Matches "x" only if "x" is not followed by "y". For example, `/\d+(?!\.)/` matches a number only if it is not followed by a decimal point. `/\d+(?!\.)/.exec('3.141')` matches "141" but not "3".
`(?<=y)x`	**Lookbehind assertion:** Matches "x" only if "x" is preceded by "y". For example, `/(?<=Jack)Sprat/` matches "Sprat" only if it is preceded by "Jack". `/(?<=Jack|Tom)Sprat/` matches "Sprat" only if it is preceded by "Jack" or "Tom". However, neither "Jack" nor "Tom" is part of the match results.
`(?<!y)x`	**Negative lookbehind assertion:** Matches "x" only if "x" is not preceded by "y". For example, `/(?<!-)\d+/` matches a number only if it is not preceded by a minus sign. `/(?<!-)\d+/.exec('3')` matches "3". `/(?<!-)\d+/.exec('-3')` match is not found because the number is preceded by the minus sign.

Groups and backreferences

Groups and backreferences indicate groups of expression characters.

Characters	Meaning
`(x)`	**Capturing group:** Matches *x* and remembers the match. For example, `/(foo)/` matches and remembers "foo" in "foo bar". A regular expression may have multiple capturing groups. In results, matches to capturing groups typically in an array whose members are in the same order as the left parentheses in the capturing group. This is usually just the order of the capturing groups themselves. This becomes important when capturing groups are nested. Matches are accessed using the index of the result's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`). Capturing groups have a performance penalty. If you don't need the matched substring to be recalled, prefer non-capturing parentheses (see below). `String.prototype.match()` won't return groups if the `/.../g` flag is set. However, you can still use `String.prototype.matchAll()` to get all matches.
`(?<Name>x)`	**Named capturing group:** Matches "x" and stores it on the groups property of the returned matches under the name specified by `<Name>`. The angle brackets (`<` and `>`) are required for group name. For example, to extract the United States area code from a phone number, we could use `/\((?<area>\d\d\d)\)/`. The resulting number would appear under `matches.groups.area`.
`(?:x)`	**Non-capturing group:** Matches "x" but does not remember the match. The matched substring cannot be recalled from the resulting array's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`).
`(?flags:x)`, `(?flags-flags:x)`	**Modifier:** Enables or disables the specified flags only to the enclosed pattern. Only the `i`, `m`, and `s` flags can be used in a modifier.
`\n`	**Backreference:** Where "n" is a positive integer. Matches the same substring matched by the nth capturing group in the regular expression (counting left parentheses). For example, `/apple(,)\sorange\1/` matches "apple, orange," in "apple, orange, cherry, peach".
`\k<Name>`	**Named backreference:** A back reference to the last substring matching the **Named capture group** specified by `<Name>`. For example, `/(?<title>\w+), yes \k<title>/` matches "Sir, yes Sir" in "Do you copy? Sir, yes Sir!".

Characters	Meaning
	Note: `\k` is used literally here to indicate the beginning of a back reference to a Named capture group.

Quantifiers

Quantifiers indicate numbers of characters or expressions to match.

Note: In the following, *item* refers not only to singular characters, but also includes character classes and groups and backreferences.

Characters	Meaning
*x**	Matches the preceding item "x" 0 or more times. For example, `/bo*/` matches "boooo" in "A ghost booooed" and "b" in "A bird warbled", but nothing in "A goat grunted".
x+	Matches the preceding item "x" 1 or more times. Equivalent to `{1,}`. For example, `/a+/` matches the "a" in "candy" and all the "a"'s in "caaaaaaandy".
x?	Matches the preceding item "x" 0 or 1 times. For example, `/e?le?/` matches the "el" in "angel" and the "le" in "angle." If used immediately after any of the quantifiers `*`, `+`, `?`, or `{}`, makes the quantifier non-greedy (matching the minimum number of times), as opposed to the default, which is greedy (matching the maximum number of times).
x{*n*}	Where "n" is a non-negative integer, matches exactly "n" occurrences of the preceding item "x". For example, `/a{2}/` doesn't match the "a" in "candy", but it matches all of the "a"'s in "caandy", and the first two "a"'s in "caaandy".
x{*n*,}	Where "n" is a non-negative integer, matches at least "n" occurrences of the preceding item "x". For example, `/a{2,}/` doesn't match the "a" in "candy", but matches all of the a's in "caandy" and in "caaaaaaandy".
x{*n*,*m*}	Where "n" and "m" are non-negative integers and `m >= n`, matches at least "n" and at most "m" occurrences of the preceding item "x". For example, `/a{1,3}/` matches nothing in "cndy", the "a" in "candy", the two "a"'s in "caandy", and the first three "a"'s in "caaaaaaandy". Notice that when matching "caaaaaaandy", the match is "aaa", even though the original string had more "a"s in it.
*x**? *x*+? *x*?? *x*{n}? *x*{n,}? *x*{n,m}?	By default quantifiers like `*` and `+` are "greedy", meaning that they try to match as many times as possible. The `?` character after the quantifier makes the quantifier "non-greedy": meaning that it will stop as soon as it finds the minimum number of matches. For example, given a string like "some <foo> <bar> new </bar> </foo> thing": • `/<.*>/` will match "<foo> <bar> new </bar> </foo>" • `/<.*?>/` will match "<foo>" **Note:** Adding `?` after `{n}` is syntactically valid but practically useless. Since `{n}` always matches exactly n times, `x{n}?` behaves the same as `x{n}`.

Groups and backreferences

Groups group multiple patterns as a whole, and capturing groups provide extra submatch information when using a regular expression pattern to match against a string. Backreferences refer to a previously captured group in the same regular expression.

Try it

```
// Groups
const imageDescription = "This image has a resolution of 1440×900 pixels.";
const regexpSize = /(\d+)×(\d+)/;
const match = imageDescription.match(regexpSize);
console.log(`Width: ${match[1]} / Height: ${match[2]}.`);
// Expected output: "Width: 1440 / Height: 900."

// Backreferences
const findDuplicates = "foo foo bar";
const regex = /\b(\w+)\s+\1\b/g;
console.log(findDuplicates.match(regex));
// Expected output: Array ["foo foo"]
```

Types

Characters	Meaning
(*x*)	**Capturing group:** Matches *x* and remembers the match. For example, `/(foo)/` matches and remembers "foo" in "foo bar".

Characters	Meaning
	A regular expression may have multiple capturing groups. In results, matches to capturing groups typically in an array whose members are in the same order as the left parentheses in the capturing group. This is usually just the order of the capturing groups themselves. This becomes important when capturing groups are nested. Matches are accessed using the index of the result's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`). Capturing groups have a performance penalty. If you don't need the matched substring to be recalled, prefer non-capturing parentheses (see below). `String.prototype.match()` won't return groups if the `/.../g` flag is set. However, you can still use `String.prototype.matchAll()` to get all matches.
`(?<Name>x)`	**Named capturing group:** Matches "x" and stores it on the groups property of the returned matches under the name specified by `<Name>`. The angle brackets (`<` and `>`) are required for group name. For example, to extract the United States area code from a phone number, we could use `/\((?<area>\d\d\d)\)/`. The resulting number would appear under `matches.groups.area`.
`(?:x)`	**Non-capturing group:** Matches "x" but does not remember the match. The matched substring cannot be recalled from the resulting array's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`).
`(?flags:x)`, `(?flags-flags:x)`	**Modifier:** Enables or disables the specified flags only to the enclosed pattern. Only the `i`, `m`, and `s` flags can be used in a modifier.
`\n`	**Backreference:** Where "n" is a positive integer. Matches the same substring matched by the nth capturing group in the regular expression (counting left parentheses). For example, `/apple(,)\sorange\1/` matches "apple, orange," in "apple, orange, cherry, peach".
`\k<Name>`	**Named backreference:** A back reference to the last substring matching the **Named capture group** specified by `<Name>`. For example, `/(?<title>\w+), yes \k<title>/` matches "Sir, yes Sir" in "Do you copy? Sir, yes Sir!". > **Note:** `\k` is used literally here to indicate the beginning of a back reference to a Named capture group.

Examples

Using groups

In this example, we match two words in a structured format by using capturing groups to remember them. `\w+` matches one or more word characters, and the parentheses `()` create a capturing group. The `g` flag is used to match all occurrences.

```
const personList = `First_Name: John, Last_Name: Doe
First_Name: Jane, Last_Name: Smith`;

const regexpNames = /First_Name: (\w+), Last_Name: (\w+)/g;
for (const match of personList.matchAll(regexpNames)) {
  console.log(`Hello ${match[1]} ${match[2]}`);
}
```

See more examples in the capturing group reference.

Using named groups

This example is the same as above, but we use named capturing groups to remember the matched words instead. This way, we can access the matched words by their meanings.

```
const personList = `First_Name: John, Last_Name: Doe
First_Name: Jane, Last_Name: Smith`;

const regexpNames =
  /First_Name: (?<firstName>\w+), Last_Name: (?<lastName>\w+)/g;
for (const match of personList.matchAll(regexpNames)) {
  console.log(`Hello ${match.groups.firstName} ${match.groups.lastName}`);
}
```

See more examples in the named capturing group reference.

Using groups and back references

In this example, we first match a single or double quote character with `['"]`, remember it, match an arbitrary number of characters with `.*?` (`*?` is a non-greedy quantifier), until we match the remembered quote character again with `\1`. The `\1` is a backreference to the first capturing group, which matches the same type of quote. The result will therefore be two strings: `"'"` and `'"'`.

```
const quote = `Single quote "'" and double quote '"'`;
const regexpQuotes = /(['"]).*?\1/g;
```

```
for (const match of quote.matchAll(regexpQuotes)) {
  console.log(match[0]);
}
```

See more examples in the backreference reference.

Using groups and match indices

By providing the `d` flag, the indices of each capturing group is returned. This is especially useful if you are correlating each matched group with the original text — for example, to provide compiler diagnostics.

```
const code = `function add(x, y) {
  return x + y;
}`;
const functionRegexp =
  /(function\s+)(?<name>[$_\p{ID_Start}][$\p{ID_Continue}]*)/du;
const match = functionRegexp.exec(code);
const lines = code.split("\n");
lines.splice(
  1,
  0,
  " ".repeat(match.indices[1][1] - match.indices[1][0]) +
    "^".repeat(match.indices.groups.name[1] - match.indices.groups.name[0]),
);
console.log(lines.join("\n"));
// function add(x, y) {
//          ^^^
//   return x + y;
// }
```

See also

- Regular expressions guide
- Character classes guide
- Assertions guide
- Quantifiers guide
- `RegExp`
- Regular expressions reference
- Backreference: `\1`, `\2`
- Capturing group: `(...)`
- Named backreference: `\k<name>`
- Named capturing group: `(?<name>...)`
- Non-capturing group: `(?:...)`

Quantifiers

Quantifiers indicate numbers of characters or expressions to match.

Try it

```
const ghostSpeak = "booh booooooooh";
const regexpSpooky = /bo{3,}h/;
console.log(ghostSpeak.match(regexpSpooky));
// Expected output: Array ["booooooooh"]

const modifiedQuote = "[He] ha[s] to go read this novel [Alice in Wonderland].";
const regexpModifications = /\[.*?\]/g;
console.log(modifiedQuote.match(regexpModifications));
// Expected output: Array ["[He]", "[s]", "[Alice in Wonderland]"]

const regexpTooGreedy = /\[.*\]/g;
console.log(modifiedQuote.match(regexpTooGreedy));
// Expected output: Array ["[He] ha[s] to go read this novel [Alice in Wonderland]"]
```

Types

> **Note:** In the following, *item* refers not only to singular characters, but also includes character classes and groups and backreferences.

Characters	Meaning
*x**	Matches the preceding item "x" 0 or more times. For example, `/bo*/` matches "boooo" in "A ghost booooed" and "b" in "A bird warbled", but nothing in "A goat grunted".
x+	Matches the preceding item "x" 1 or more times. Equivalent to `{1,}`. For example, `/a+/` matches the "a" in "candy" and all the "a"'s in "caaaaaaandy".
x?	Matches the preceding item "x" 0 or 1 times. For example, `/e?le?/` matches the "el" in "angel" and the "le" in "angle."

Characters	Meaning
	If used immediately after any of the quantifiers `*`, `+`, `?`, or `{}`, makes the quantifier non-greedy (matching the minimum number of times), as opposed to the default, which is greedy (matching the maximum number of times).
x{*n*}	Where "n" is a non-negative integer, matches exactly "n" occurrences of the preceding item "x". For example, `/a{2}/` doesn't match the "a" in "candy", but it matches all of the "a"'s in "caandy", and the first two "a"'s in "caaandy".
x{*n*,}	Where "n" is a non-negative integer, matches at least "n" occurrences of the preceding item "x". For example, `/a{2,}/` doesn't match the "a" in "candy", but matches all of the a's in "caandy" and in "caaaaaaandy".
x{*n*,*m*}	Where "n" and "m" are non-negative integers and `m >= n`, matches at least "n" and at most "m" occurrences of the preceding item "x". For example, `/a{1,3}/` matches nothing in "cndy", the "a" in "candy", the two "a"'s in "caandy", and the first three "a"'s in "caaaaaaandy". Notice that when matching "caaaaaaandy", the match is "aaa", even though the original string had more "a"s in it.
*x**? *x*+? *x*?? *x*{n}? *x*{n,}? *x*{n,m}?	By default quantifiers like `*` and `+` are "greedy", meaning that they try to match as many times as possible. The `?` character after the quantifier makes the quantifier "non-greedy": meaning that it will stop as soon as it finds the minimum number of matches. For example, given a string like "some <foo> <bar> new </bar> </foo> thing": • `/<.*>/` will match "<foo> <bar> new </bar> </foo>" • `/<.*?>/` will match "<foo>" **Note:** Adding `?` after `{n}` is syntactically valid but practically useless. Since `{n}` always matches exactly n times, `x{n}?` behaves the same as `x{n}`.

Examples

Repeated pattern

In this example, we match one or more word characters with `\w+`, then one or more characters "a" with `a+`, and finally end at a word boundary with `\b`.

```
const wordEndingWithAs = /\w+a+\b/;
const delicateMessage = "This is Spartaaaaaaa";

console.table(delicateMessage.match(wordEndingWithAs)); // [ "Spartaaaaaaa" ]
```

Counting characters

In this example, we match words that have a single letter, words that have between 2 and 6 letters, and words that have 13 or more letters.

```
const singleLetterWord = /\b\w\b/g;
const notSoLongWord = /\b\w{2,6}\b/g;
const longWord = /\b\w{13,}\b/g;

const sentence = "Why do I have to learn multiplication table?";

console.table(sentence.match(singleLetterWord)); // ["I"]
console.table(sentence.match(notSoLongWord)); // [ "Why", "do", "have", "to", "learn", "table" ]
console.table(sentence.match(longWord)); // ["multiplication"]
```

Optional character

In this example, we match words that either end with "our" or "or".

```
const britishText = "He asked his neighbour a favour.";
const americanText = "He asked his neighbor a favor.";

const regexpEnding = /\w+ou?r/g;
// \w+ One or several letters
// o   followed by an "o",
// u?  optionally followed by a "u"
// r   followed by an "r"

console.table(britishText.match(regexpEnding));
// ["neighbour", "favour"]

console.table(americanText.match(regexpEnding));
// ["neighbor", "favor"]
```

Greedy versus non-greedy

In this example, we match one or more word characters or spaces with `[\w ]+` and `[\w ]+?`. The first one is greedy and the second one is non-greedy. Note how the second one stops as soon as it meets the minimal requirement.

```
const text = "I must be getting somewhere near the center of the earth.";
const greedyRegexp = /[\w ]+/;

console.log(text.match(greedyRegexp)[0]);
// "I must be getting somewhere near the center of the earth"
// almost all of the text matches (leaves out the dot character)

const nonGreedyRegexp = /[\w ]+?/; // Notice the question mark
console.log(text.match(nonGreedyRegexp));
// "I"
// The match is the smallest one possible
```

See also

- Regular expressions guide
- Character classes guide
- Assertions guide
- Groups and backreferences guide
- `RegExp`
- Regular expressions reference
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

Representing dates & times

- Previous
- Next

> **Note:** The `Date` object is now considered legacy and should be avoided in new code. We will update this page with modern alternatives soon.

Date object

JavaScript does not have a date data type. However, you can use the `Date` object and its methods to work with dates and times in your applications. The `Date` object has a large number of methods for setting, getting, and manipulating dates. It does not have any properties.

JavaScript handles dates similarly to Java. The two languages have many of the same date methods, and both languages store dates as the number of milliseconds since midnight at the beginning of January 1, 1970, UTC, with a Unix Timestamp being the number of seconds since the same instant. The instant at the midnight at the beginning of January 1, 1970, UTC is called the epoch.

The `Date` object range is -100,000,000 days to 100,000,000 days relative to the epoch.

To create a `Date` object:

```
const dateObjectName = new Date([parameters]);
```

where `dateObjectName` is the name of the `Date` object being created; it can be a new object or a property of an existing object.

Calling `Date` without the `new` keyword returns a string representing the current date and time.

The `parameters` in the preceding syntax can be any of the following:

- Nothing: creates today's date and time. For example, `today = new Date();`.
- A string representing a date, in many different forms. The exact forms supported differ among engines, but the following form is always supported: `YYYY-MM-DDTHH:mm:ss.sssZ`. For example, `xmas95 = new Date("1995-12-25")`. If you omit hours, minutes, or seconds, the value will be set to zero.
- A set of integer values for year, month, and day. For example, `xmas95 = new Date(1995, 11, 25)`.
- A set of integer values for year, month, day, hour, minute, and seconds. For example, `xmas95 = new Date(1995, 11, 25, 9, 30, 0);`.

Methods of the Date object

The `Date` object methods for handling dates and times fall into these broad categories:

- "set" methods, for setting date and time values in `Date` objects.
- "get" methods, for getting date and time values from `Date` objects.
- "to" methods, for returning string values from `Date` objects.
- parse and UTC methods, for parsing `Date` strings.

With the "get" and "set" methods you can get and set seconds, minutes, hours, day of the month, day of the week, months, and years separately. There is a `getDay` method that returns the day of the week, but no corresponding `setDay` method, because the day of the week is set automatically. These methods use integers to represent these values as follows:

- Seconds and minutes: 0 to 59
- Hours: 0 to 23
- Day: 0 (Sunday) to 6 (Saturday)
- Date: 1 to 31 (day of the month)
- Months: 0 (January) to 11 (December)
- Year: years since 1900

For example, suppose you define the following date:

```
const xmas95 = new Date("1995-12-25");
```

Then `xmas95.getMonth()` returns 11, and `xmas95.getFullYear()` returns 1995.

The `getTime` and `setTime` methods are useful for comparing dates. The `getTime` method returns the number of milliseconds since the epoch for a `Date` object.

For example, the following code displays the number of days left in the current year:

```
const today = new Date();
const endYear = new Date(1995, 11, 31, 23, 59, 59, 999); // Set day and month
endYear.setFullYear(today.getFullYear()); // Set year to this year
const msPerDay = 24 * 60 * 60 * 1000; // Number of milliseconds per day
let daysLeft = (endYear.getTime() - today.getTime()) / msPerDay;
daysLeft = Math.round(daysLeft); // Returns days left in the year
```

This example creates a `Date` object named `today` that contains today's date. It then creates a `Date` object named `endYear` and sets the year to the current year. Then, using the number of milliseconds per day, it computes the number of days between `today` and `endYear`, using `getTime` and rounding to a whole number of days.

The `parse` method is useful for assigning values from date strings to existing `Date` objects. For example, the following code uses `parse` and `setTime` to assign a date value to the `ipoDate` object:

```
const ipoDate = new Date();
ipoDate.setTime(Date.parse("Aug 9, 1995"));
```

Example

In the following example, the function `JSClock()` returns the time in the format of a digital clock.

```
function JSClock() {
  const time = new Date();
  const hour = time.getHours();
  const minute = time.getMinutes();
  const second = time.getSeconds();
  let temp = String(hour % 12);
  if (temp === "0") {
    temp = "12";
  }
  temp += (minute < 10 ? ":0" : ":") + minute;
  temp += (second < 10 ? ":0" : ":") + second;
  temp += hour >= 12 ? " P.M." : " A.M.";
  return temp;
}
```

The `JSClock` function first creates a new `Date` object called `time`; since no arguments are given, time is created with the current date and time. Then calls to the `getHours`, `getMinutes`, and `getSeconds` methods assign the value of the current hour, minute, and second to `hour`, `minute`, and `second`.

The following statements build a string value based on the time. The first statement creates a variable `temp`. Its value is `hour % 12`, which is `hour` in the 12-hour system. Then, if the hour is `0`, it gets re-assigned to `12`, so that midnights and noons are displayed as `12:00` instead of `0:00`.

The next statement appends a `minute` value to `temp`. If the value of `minute` is less than 10, the conditional expression adds a string with a preceding zero; otherwise it adds a string with a demarcating colon. Then a statement appends a seconds value to `temp` in the same way.

Finally, a conditional expression appends "P.M." to `temp` if `hour` is 12 or greater; otherwise, it appends "A.M." to `temp`.

- Previous
- Next

JavaScript resource management

- Previous
- Next

This guide talks about how to do *resource management* in JavaScript. Resource management is not exactly the same as memory management, which is a more advanced topic and usually handled automatically by JavaScript. Resource management is about managing resources that are *not* automatically cleaned up by JavaScript. Sometimes, it's okay to have some unused objects in memory, because they don't interfere with application logic, but resource leaks often lead to things not working, or a lot of excess memory usage. Therefore, this is not an optional feature about optimization, but a core feature to write correct programs!

> **Note:** While memory management and resource management are two separate topics, sometimes you can hook into the memory management system to do resource management, as a last resort. For example, if you have a JavaScript object representing a handle of an external resource, you can create a `FinalizationRegistry` to clean up the resource when the handle is garbage collected, because there is definitely no way to access the resource afterwards. However, there is no guarantee that the finalizer will run, so it's not a good idea to rely on it for critical resources.

Problem

Let's first look at a few examples of resources that need to be managed:

- **File handles**: A file handle is used to read and write bytes in a file. When you are done with it, you must call `fileHandle.close()`, otherwise the file will remain open, even when the JS object is no longer accessible. As the linked Node.js docs say:

> If a `<FileHandle>` is not closed using the `fileHandle.close()` method, it will try to automatically close the file descriptor and emit a process warning, helping to prevent memory leaks. Please do not rely on this behavior because it can be unreliable and the file may not be closed. Instead, always explicitly close `<FileHandle>`s. Node.js may change this behavior in the future.

- **Network connections**: Some connections, such as `WebSocket` and `RTCPeerConnection`, need to be closed if no messages are transmitted. Otherwise, the connection remains open, and connection pools are often very limited in size.
- **Stream readers**: If you don't call `ReadableStreamDefaultReader.releaseLock()`, the stream will be locked and does not permit another reader to consume it.

Here is one concrete example, using a readable stream:

```
const stream = new ReadableStream({
  start(controller) {
    controller.enqueue("a");
    controller.enqueue("b");
    controller.enqueue("c");
    controller.close();
  },
});

async function readUntil(stream, text) {
  const reader = stream.getReader();
  let chunk = await reader.read();

  while (!chunk.done && chunk.value !== text) {
    console.log(chunk);
    chunk = await reader.read();
  }
  // We forgot to release the lock here
}

readUntil(stream, "b").then(() => {
  const anotherReader = stream.getReader();
  // TypeError: ReadableStreamDefaultReader constructor can only
  // accept readable streams that are not yet locked to a reader
});
```

Here, we have a stream that emits three chunks of data. We read from the stream until we find the letter "b". When `readUntil` returns, the stream is only partially consumed, so we should be able to continue to read from it using another reader. However, we forgot to release the lock, so although `reader` is no longer available, the stream is still locked and we cannot create another reader.

The solution in this case is straightforward: call `reader.releaseLock()` at the end of `readUntil`. But, a few issues still remain:

- Inconsistency: different resources have different ways to release them. For example, we have `close()`, `releaseLock()`, `disconnect()`, etc. The pattern does not generalize.
- Error handling: what happens if the `reader.read()` call fails? Then `readUntil` would terminate and never get to the `reader.releaseLock()` call. We can fix this using `try...finally`:

  ```
  async function readUntil(stream, text) {
    const reader = stream.getReader();
    try {
      let chunk = await reader.read();

      while (!chunk.done && chunk.value !== text) {
        console.log(chunk);
        chunk = await reader.read();
      }
    } finally {
      reader.releaseLock();
    }
  }
  ```

  But you have to remember to do this every time you have some important resource to release.
- Scoping: in the above example, `reader` is already closed when we exit the `try...finally` statement, but it continues to be available in its scope. This means you may accidentally use it after it's closed.
- Multiple resources: if we have two readers on different streams, we have to remember to release both of them. This is a respectable attempt to do so:

  ```
  const reader1 = stream1.getReader();
  const reader2 = stream2.getReader();
  try {
    // do something with reader1 and reader2
  } finally {
    reader1.releaseLock();
    reader2.releaseLock();
  }
  ```

  However, this introduces more error handling troubles. If `stream2.getReader()` throws, then `reader1` does not get released; if `reader1.releaseLock()` throws an error, then `reader2` does not get released. This means we actually have to wrap each resource acquisition-release pair in its own `try...finally`:

```
const reader1 = stream1.getReader();
try {
  const reader2 = stream2.getReader();
  try {
    // do something with reader1 and reader2
  } finally {
    reader2.releaseLock();
  }
} finally {
  reader1.releaseLock();
}
```

You see how a seemingly benign task of calling `releaseLock` can quickly lead to nested boilerplate code. This is why JavaScript provides integrated language support for resource management.

The <code>using</code> and <code>await using</code> declarations

The solution we have is two special kinds of variable declaration: `using` and `await using`. They are similar to `const`, but they automatically release the resource when the variable goes out of scope as long as the resource is *disposable*. Using the same example as above, we can rewrite it as:

```
{
  using reader1 = stream1.getReader();
  using reader2 = stream2.getReader();

  // do something with reader1 and reader2

  // Before we exit the block, reader1 and reader2 are automatically released
}
```

> **Note:** At the time of writing, `ReadableStreamDefaultReader` does not implement the disposable protocol. This is a hypothetical example.

First, notice the extra braces around the code. This creates a new block scope for the `using` declarations. Resources declared with `using` are automatically freed when they go out of the scope of `using`, which, in this case, is whenever we are exiting the block, either because all statements have executed, or because an error or `return`/`break`/`continue` was encountered somewhere.

This means `using` can only be used in a scope that has a clear lifetime—namely, it cannot be used at the top level of a script, because variables at the top level of a script are in scope for all future scripts on the page, which practically means the resource can never be freed if the page never unloads. However, you can use it at the top level of a module, because the module scope ends when the module finishes executing.

Now we know *when* `using` does cleanup. But *how* is it done? `using` requires the resource to implement the *disposable* protocol. An object is disposable if it has the `[Symbol.dispose]()` method. This method is called with no arguments to perform cleanup. For example, in the reader case, the `[Symbol.dispose]` property can be a simple alias or wrapper of `releaseLock`:

```
// For demonstration
class MyReader {
  // A wrapper
  [Symbol.dispose]() {
    this.releaseLock();
  }
  releaseLock() {
    // Logic to release resources
  }
}

// OR, an alias
MyReader.prototype[Symbol.dispose] = MyReader.prototype.releaseLock;
```

Through the disposable protocol, `using` can dispose all resources in a consistent fashion without understanding what type of resource it is.

Every scope has a list of resources associated with it, in the order they were declared. When the scope exits, the resources are disposed in reverse order, by calling their `[Symbol.dispose]()` method. For example, in the example above, `reader1` is declared before `reader2`, so `reader2` is disposed first, then `reader1`. Errors thrown when attempting to dispose one resource will not prevent disposal of other resources. This is consistent with the `try...finally` pattern, and respects possible dependencies between resources.

`await using` is very similar to `using`. The syntax tells you that an `await` happens somewhere—not when the resource is declared, but actually when it's disposed. `await using` requires the resource to be *async disposable*, which means it has a `[Symbol.asyncDispose]()` method. This method is called with no arguments and returns a promise that resolves when the cleanup is done. This is useful when the cleanup is asynchronous, such as `fileHandle.close()`, in which case the result of the disposal can only be known asynchronously.

```
{
  await using fileHandle = open("file.txt", "w");
  await fileHandle.write("Hello");

  // fileHandle.close() is called and awaited
}
```

Because `await using` requires doing an `await`, it is only permitted in contexts where `await` is, which includes inside `async` functions and top-level `await` in modules.

Resources are cleaned up sequentially, not concurrently: the return value of one resource's `[Symbol.asyncDispose]()` method will be `await`ed before the next resource's `[Symbol.asyncDispose]()` method is called.

Some things to note:

- `using` and `await using` are *opt in*. If you declare your resource using `let`, `const`, or `var`, no automatic disposal happens, just like any other non-disposable values.
- `using` and `await using` require the resource to be disposable (or async disposable). If the resource does not have the `[Symbol.dispose]()` or `[Symbol.asyncDispose]()` method respectively, you will get a `TypeError` at the line of declaration. The resource can be `null` or `undefined`, however, allowing you to conditionally acquire resources.
- Like `const`, `using` and `await using` variables cannot be re-assigned, although the properties of the objects they hold can be changed. However, the `[Symbol.dispose]()`/`[Symbol.asyncDispose]()` method is already saved at the time of declaration, so changing the method after the declaration does not affect the cleanup.
- There are some gotchas when conflating scopes with resource lifetime. See `using` for a few examples.

The <code>DisposableStack</code> and <code>AsyncDisposableStack</code> objects

`using` and `await using` are special syntaxes. Syntaxes are convenient and hide a lot of the complexity, but sometimes you need to do things manually.

For one common example: what if you don't want to dispose the resource at the end of *this* scope, but at some *later* scope? Consider this:

```
let reader;
if (someCondition) {
  reader = stream.getReader();
} else {
  reader = stream.getReader({ mode: "byob" });
}
```

As we said, `using` is like `const`: it must be initialized and can't be reassigned, so you may attempt this:

```
if (someCondition) {
  using reader = stream.getReader();
} else {
  using reader = stream.getReader({ mode: "byob" });
}
```

However, this means all logic has to be written inside the `if` or `else`, causing a lot of duplication. What we want to do is to acquire and register the resource in one scope but dispose it in another. We can use a `DisposableStack` for this purpose, which is an object which holds a collection of disposable resources and which itself is disposable:

```
{
  using disposer = new DisposableStack();
  let reader;
  if (someCondition) {
    reader = disposer.use(stream.getReader());
  } else {
    reader = disposer.use(stream.getReader({ mode: "byob" }));
  }
  // Do something with reader
  // Before scope exit, disposer is disposed, which disposes reader
}
```

You may have a resource that does not yet implement the disposable protocol, so it will be rejected by `using`. In this case, you can use `adopt()`.

```
{
  using disposer = new DisposableStack();
  // Suppose reader does not have the [Symbol.dispose]() method,
  // then it cannot be used with using.
  // However, we can manually pass a disposer function to disposer.adopt
  const reader = disposer.adopt(stream.getReader(), (reader) =>
    reader.releaseLock(),
  );
  // Do something with reader
  // Before scope exit, disposer is disposed, which disposes reader
}
```

You may have a disposal action to perform but it's not "tethered" to any resource in particular. Maybe you just want to log a message saying "All database connections closed" when there are multiple connections open simultaneously. In this case, you can use `defer()`.

```
{
  using disposer = new DisposableStack();
  disposer.defer(() => console.log("All database connections closed"));
  const connection1 = disposer.use(openConnection());
  const connection2 = disposer.use(openConnection());
  // Do something with connection1 and connection2
  // Before scope exit, disposer is disposed, which first disposes connection1
  // and connection2 and then logs the message
}
```

You may want to do *conditional* disposal—for example, only dispose claimed resources when an error occurred. In this case, you can use `move()` to preserve the resources which would otherwise be disposed.

```
class MyResource {
  #resource1;
  #resource2;
```

```
  #disposables;
  constructor() {
    using disposer = new DisposableStack();
    this.#resource1 = disposer.use(getResource1());
    this.#resource2 = disposer.use(getResource2());
    // If we made it here, then there were no errors during construction and
    // we can safely move the disposables out of `disposer` and into `#disposables`.
    this.#disposables = disposer.move();
    // If construction failed, then `disposer` would be disposed before reaching
    // the line above, disposing `#resource1` and `#resource2`.
  }
  [Symbol.dispose]() {
    this.#disposables.dispose(); // Dispose `#resource2` and `#resource1`.
  }
}
```

`AsyncDisposableStack` is like `DisposableStack`, but for use with async disposable resources. Its `use()` method expects an async disposable, its `adopt()` method expects an async cleanup function, and its `dispose()` method expects an async callback. It provides a `[Symbol.asyncDispose]()` method. You can still pass it sync resources if you have a mix of both sync and async.

The reference for `DisposableStack` contains more examples and details.

Error handling

A major use case of the resource management feature is to ensure that resources are always disposed, even when an error occurs. Let us investigate some complex error handling scenarios.

We start with the following code, which, by using `using`, is robust against errors:

```
async function readUntil(stream, text) {
  // Use `using` instead of `await using` because `releaseLock` is synchronous
  using reader = stream.getReader();
  let chunk = await reader.read();

  while (!chunk.done && chunk.value !== text) {
    console.log(chunk.toUpperCase());
    chunk = await reader.read();
  }
}
```

Suppose that `chunk` turns out to be `null`. Then `!chunk.done` will throw a `TypeError`, causing the function to terminate. Before the function exits, `stream[Symbol.dispose]()` is called, which releases the lock on the stream.

```
const stream = new ReadableStream({
  start(controller) {
    controller.enqueue("a");
    controller.enqueue(null);
    controller.enqueue("b");
    controller.enqueue("c");
    controller.close();
  },
});

readUntil(stream, "b")
  .catch((e) => console.error(e)) // TypeError: chunk.toUpperCase is not a function
  .then(() => {
    const anotherReader = stream.getReader();
    // Successfully creates another reader
  });
```

So, `using` does not swallow any errors: all errors that occur are still thrown, but the resources get closed right before that. Now, what happens if the resource cleanup itself also throws an error? Let's use a more contrived example:

```
class MyReader {
  [Symbol.dispose]() {
    throw new Error("Failed to release lock");
  }
}

function doSomething() {
  using reader = new MyReader();
  throw new Error("Failed to read");
}

try {
  doSomething();
} catch (e) {
  console.error(e); // SuppressedError: An error was suppressed during disposal
}
```

There are two errors generated in the `doSomething()` call: an error thrown during `doSomething`, and an error thrown during disposal of `reader` because of the first error. Both errors are thrown together, so what you caught is a `SuppressedError`. This is a special error that wraps two errors: the `error` property contains the later error, and the `suppressed` property contains the earlier error, which gets "suppressed" by the later error.

If we have more than one resource, and *both* of them throw an error during disposal (this should be exceedingly rare–it's already rare for disposal to fail!), then each earlier error is suppressed by the later error, forming a chain of suppressed errors.

```
class MyReader {
  [Symbol.dispose]() {
    throw new Error("Failed to release lock on reader");
  }
}

class MyWriter {
  [Symbol.dispose]() {
    throw new Error("Failed to release lock on writer");
  }
}

function doSomething() {
  using reader = new MyReader();
  using writer = new MyWriter();
  throw new Error("Failed to read");
}

try {
  doSomething();
} catch (e) {
  console.error(e); // SuppressedError: An error was suppressed during disposal
  console.error(e.suppressed); // SuppressedError: An error was suppressed during disposal
  console.error(e.error); // Error: Failed to release lock on reader
  console.error(e.suppressed.suppressed); // Error: Failed to read
  console.error(e.suppressed.error); // Error: Failed to release lock on writer
}
```

- The `reader` is released last, so its error is the latest and therefore suppresses everything else: it shows up as `e.error`.
- The `writer` is released first, so its error is later than the original exiting error, but earlier than the `reader` error: it shows up as `e.suppressed.error`.
- The original error about "Failed to read" is the earliest error, so it shows up as `e.suppressed.suppressed`.

Examples

Automatically releasing object URLs

In the following example, we create an object URL to a blob (in a real application, this blob would be fetched from somewhere, such as a file or a fetch response) so we can download the blob as a file. In order to prevent a resource leak, we must free the object URL with `URL.revokeObjectURL()` when it is no longer needed (that is, when the download has successfully started). Because the URL itself is just a string and therefore doesn't implement the disposable protocol, we cannot directly declare `url` with `using`; therefore, we create a `DisposableStack` to serve as the disposer for `url`. The object URL is revoked as soon as `disposer` goes out of scope, which is when either `link.click()` finishes or an error occurs somewhere.

```
const downloadButton = document.getElementById("download-button");
const exampleBlob = new Blob(["example data"]);

downloadButton.addEventListener("click", () => {
  using disposer = new DisposableStack();
  const link = document.createElement("a");
  const url = disposer.adopt(
    URL.createObjectURL(exampleBlob),
    URL.revokeObjectURL,
  );

  link.href = url;
  link.download = "example.txt";
  link.click();
});
```

Automatically canceling in-progress requests

In the following example, we fetch a list of resources concurrently using `Promise.all()`. `Promise.all()` fails and rejects the resulting promise as soon as one request failed; however, the other pending requests continue to run, despite their results being inaccessible to the program. To avoid these remaining requests needlessly consuming resources, we need to automatically cancel in-progress requests whenever `Promise.all()` settles. We implement cancellation with an `AbortController`, and pass its `signal` to every `fetch()` call. If `Promise.all()` fulfills, then the function returns normally and the controller aborts, which is harmless because there's no pending request to cancel; if `Promise.all()` rejects and the function throws, then the controller aborts and cancels all pending requests.

```
async function getAllData(urls) {
  using disposer = new DisposableStack();
  const { signal } = disposer.adopt(new AbortController(), (controller) =>
    controller.abort(),
  );

  // Fetch all URLs in parallel
  // Automatically cancel any incomplete requests if any request fails
  const pages = await Promise.all(
    urls.map((url) =>
      fetch(url, { signal }).then((response) => {
        if (!response.ok)
          throw new Error(
            `Response error: ${response.status} - ${response.statusText}`,
```

```
          );
        return response.text();
      }),
    ),
  );
  return pages;
}
```

Pitfalls

The resource disposal syntax offers a lot of strong error handling guarantees that ensure the resources are always cleaned up no matter what happens, but there are some pitfalls you may still encounter:

- Forgetting to use `using` or `await using`. The resource management syntax is only there to help you when you know you need it, but there's nothing to yell at you if you forget to use it! Unfortunately, there's no good way to prevent this before-the-fact, because there are no syntactic clues that something is a disposable resource, and even for disposable resources, you may want to declare them without automatic disposal. You probably need a type checker combined with a linter to catch these issues, such as typescript-eslint (which is still planning to work on this feature).
- Use-after-free. Generally, the `using` syntax ensures that a resource is freed when it goes out of scope, but there are many ways to persist a value beyond its binding variable. JavaScript does not have an ownership mechanism like Rust, so you can declare an alias that doesn't use `using`, or preserve the resource in a closure, etc. The `using` reference contains many examples of such pitfalls. Again, there's no good way to properly detect this in a complicated control flow, so you need to be careful.

The resource management feature is not a silver bullet. It is definitely an improvement over manually invoking the disposal methods, but it is not smart enough to prevent all resource management bugs. You still need to be careful and understand the semantics of the resources you are using.

Conclusion

Here are the key components of the resource management system:

- `using` and `await using` declarations for automatic resource disposal.
- The *disposable* and *async disposable* protocols, using the `Symbol.dispose` and `Symbol.asyncDispose` respectively, for resources to implement.
- The `DisposableStack` and `AsyncDisposableStack` objects for cases where `using` and `await using` are not suitable.

With proper usage of these APIs, you can create systems interacting with external resources that remain strong and robust against all error conditions without lots of boilerplate code.

- Previous
- Next

JavaScript typed arrays

- Previous
- Next

JavaScript typed arrays are array-like objects that provide a mechanism for reading and writing raw binary data in memory buffers.

Typed arrays are not intended to replace arrays for any kind of functionality. Instead, they provide developers with a familiar interface for manipulating binary data. This is useful when interacting with platform features, such as audio and video manipulation, access to raw data using WebSockets, and so forth. Each entry in a JavaScript typed array is a raw binary value in one of a number of supported formats, from 8-bit integers to 64-bit floating-point numbers.

Typed array objects share many of the same methods as arrays with similar semantics. However, typed arrays are *not* to be confused with normal arrays, as calling `Array.isArray()` on a typed array returns `false`. Moreover, not all methods available for normal arrays are supported by typed arrays (e.g., push and pop).

To achieve maximum flexibility and efficiency, JavaScript typed arrays split the implementation into *buffers* and *views*. A buffer is an object representing a chunk of data; it has no format to speak of, and offers no mechanism for accessing its contents. In order to access the memory contained in a buffer, you need to use a view. A view provides a *context* — that is, a data type, starting offset, and number of elements.

A diagram showing how different typed arrays may be views of the same underlying buffer. Each one has a different element number and width.

Buffers

There are two types of buffers: `ArrayBuffer` and `SharedArrayBuffer`. Both are low-level representations of a memory span. They have "array" in their names, but they don't have much to do with arrays — you cannot read or write to them directly. Instead, buffers are generic objects that just contain raw data. In order to access the memory represented by a buffer, you need to use a view.

Buffers support the following actions:

- *Allocate*: As soon as a new buffer is created, a new memory span is allocated and initialized to `0`.
- *Copy*: Using the `slice()` method, you can efficiently copy a portion of the memory without creating views to manually copy each byte.
- *Transfer*: Using the `transfer()` and `transferToFixedLength()` methods, you can transfer ownership of the memory span to a new buffer object. This is useful when transferring data between different execution contexts without copying. After the transfer, the original buffer is no longer usable. A `SharedArrayBuffer` cannot be transferred (as the buffer is already shared by all execution contexts).
- *Resize*: Using the `resize()` method, you can resize the memory span (either claim more memory space, as long as it doesn't pass the pre-set `maxByteLength` limit, or release some memory space). `SharedArrayBuffer` can only be grown but not shrunk.

The difference between `ArrayBuffer` and `SharedArrayBuffer` is that the former is always owned by a single execution context at a time. If you pass an `ArrayBuffer` to a different execution context, it is *transferred* and the original `ArrayBuffer` becomes unusable. This ensures that only one execution context can access the memory at a time. A `SharedArrayBuffer` is not transferred when passed to a different execution context, so it can be accessed by multiple execution contexts at the same time. This may introduce race conditions when multiple threads access the same memory span, so operations such as `Atomics` methods become useful.

Views

There are currently two main kinds of views: typed array views and `DataView`. Typed arrays provide utility methods that allow you to conveniently transform binary data. `DataView` is more low-level and allows granular control of how data is accessed. The ways to read and write data using the two views are very different.

Both kinds of views cause `ArrayBuffer.isView()` to return `true`. They both have the following properties:

`buffer`

: The underlying buffer that the view references.

`byteOffset`

: The offset, in bytes, of the view from the start of its buffer.

`byteLength`

: The length, in bytes, of the view.

Both constructors accept the above three as separate arguments, although typed array constructors accept `length` as the number of elements rather than the number of bytes.

Typed array views

Typed array views have self-descriptive names and provide views for all the usual numeric types like `Int8`, `Uint32`, `Float64` and so forth. There is one special typed array view, `Uint8ClampedArray`, which clamps the values between `0` and `255`. This is useful for Canvas data processing, for example.

Type	Value Range	Size in bytes	Web IDL type
`Int8Array`	-128 to 127	1	`byte`
`Uint8Array`	0 to 255	1	`octet`
`Uint8ClampedArray`	0 to 255	1	`octet`
`Int16Array`	-32768 to 32767	2	`short`
`Uint16Array`	0 to 65535	2	`unsigned short`
`Int32Array`	-2147483648 to 2147483647	4	`long`
`Uint32Array`	0 to 4294967295	4	`unsigned long`
`Float16Array`	`-65504` to `65504`	2	N/A
`Float32Array`	`-3.4e38` to `3.4e38`	4	`unrestricted float`
`Float64Array`	`-1.8e308` to `1.8e308`	8	`unrestricted double`
`BigInt64Array`	-2^{63} to $2^{63} - 1$	8	`bigint`
`BigUint64Array`	0 to $2^{64} - 1$	8	`bigint`

All typed array views have the same methods and properties, as defined by the `TypedArray` class. They only differ in the underlying data type and the size in bytes. This is discussed in more detail in Value encoding and normalization.

Typed arrays are, in principle, fixed-length, so array methods that may change the length of an array are not available. This includes `pop`, `push`, `shift`, `splice`, and `unshift`. In addition, `flat` is unavailable because there are no nested typed arrays, and related methods including `concat` and `flatMap` do not have great use cases so are unavailable. As `splice` is unavailable, so too is `toSpliced`. All other array methods are shared between `Array` and `TypedArray`.

On the other hand, `TypedArray` has the extra `set` and `subarray` methods that optimize working with multiple typed arrays that view the same buffer. The `set()` method allows setting multiple typed array indices at once, using data from another array or typed array. If the two typed arrays share the same underlying buffer, the operation may be more efficient as it's a fast memory move. The `subarray()` method creates a new typed array view that references the same buffer as the original typed array, but with a narrower span.

There's no way to directly change the length of a typed array without changing the underlying buffer. However, when the typed array views a resizable buffer and does not have a fixed `byteLength`, it is *length-tracking*, and will automatically resize to fit the underlying buffer as the resizable buffer is resized. See Behavior when viewing a resizable buffer for details.

Similar to regular arrays, you can access typed array elements using bracket notation. The corresponding bytes in the underlying buffer are retrieved and interpreted as a number. Any property access using a number (or the string representation of a number, since numbers are always converted to strings when accessing properties) will be proxied by the typed array — they never interact with the object itself. This means, for example:

- Out-of-bounds index access always returns `undefined`, without actually accessing the property on the object.
- Any attempt to write to such an out-of-bounds property has no effect: it does not throw an error but doesn't change the buffer or typed array either.
- Typed array indices appear to be configurable and writable, but any attempt to change their attributes will fail.

```
const uint8 = new Uint8Array([1, 2, 3]);
console.log(uint8[0]); // 1

// For illustrative purposes only. Not for production code.
uint8[-1] = 0;
uint8[2.5] = 0;
uint8[NaN] = 0;
console.log(Object.keys(uint8)); // ["0", "1", "2"]
console.log(uint8[NaN]); // undefined

// Non-numeric access still works
uint8[true] = 0;
console.log(uint8[true]); // 0
```

```
Object.freeze(uint8); // TypeError: Cannot freeze array buffer views with elements
```

DataView

The `DataView` is a low-level interface that provides a getter/setter API to read and write arbitrary data to the buffer. This is useful when dealing with different types of data, for example. Typed array views are in the native byte-order (see Endianness) of your platform. With a `DataView`, the byte-order can be controlled. By default, it's big-endian—the bytes are ordered from most significant to least significant. This can be reversed, with the bytes ordered from least significant to most significant (little-endian), using getter/setter methods.

`DataView` does not require alignment; multi-byte read and write can be started at any specified offset. The setter methods work the same way.

The following example uses a `DataView` to get the binary representation of any number:

```
function toBinary(
  x,
  { type = "Float64", littleEndian = false, separator = " ", radix = 16 } = {},
) {
  const bytesNeeded = globalThis[`${type}Array`].BYTES_PER_ELEMENT;
  const dv = new DataView(new ArrayBuffer(bytesNeeded));
  dv[`set${type}`](0, x, littleEndian);
  const bytes = Array.from({ length: bytesNeeded }, (_, i) =>
    dv
      .getUint8(i)
      .toString(radix)
      .padStart(8 / Math.log2(radix), "0"),
  );
  return bytes.join(separator);
}

console.log(toBinary(1.1)); // 3f f1 99 99 99 99 99 9a
console.log(toBinary(1.1, { littleEndian: true })); // 9a 99 99 99 99 99 f1 3f
console.log(toBinary(20, { type: "Int8", radix: 2 })); // 00010100
```

Web APIs using typed arrays

These are some examples of APIs that make use of typed arrays; there are others, and more are being added all the time.

`FileReader.prototype.readAsArrayBuffer()`

> The `FileReader.prototype.readAsArrayBuffer()` method starts reading the contents of the specified `Blob` or `File`.

`fetch()`

> The `body` option to `fetch()` can be a typed array or `ArrayBuffer`, enabling you to send these objects as the payload of a `POST` request.

`ImageData.data`

> Is a `Uint8ClampedArray` representing a one-dimensional array containing the data in the RGBA order, with integer values between `0` and `255` inclusive.

Examples

Using views with buffers

First of all, we will need to create a buffer, here with a fixed length of 16-bytes:

```
const buffer = new ArrayBuffer(16);
```

At this point, we have a chunk of memory whose bytes are all pre-initialized to 0. There's not a lot we can do with it, though. For example, we can confirm that the buffer is the right size:

```
if (buffer.byteLength === 16) {
  console.log("Yes, it's 16 bytes.");
} else {
  console.log("Oh no, it's the wrong size!");
}
```

Before we can really work with this buffer, we need to create a view. Let's create a view that treats the data in the buffer as an array of 32-bit signed integers:

```
const int32View = new Int32Array(buffer);
```

Now we can access the fields in the array just like a normal array:

```
for (let i = 0; i < int32View.length; i++) {
  int32View[i] = i * 2;
}
```

This fills out the 4 entries in the array (4 entries at 4 bytes each makes 16 total bytes) with the values `0`, `2`, `4`, and `6`.

Multiple views on the same data

Things start to get really interesting when you consider that you can create multiple views onto the same data. For example, given the code above, we can continue like this:

```
const int16View = new Int16Array(buffer);

for (let i = 0; i < int16View.length; i++) {
  console.log(`Entry ${i}: ${int16View[i]}`);
}
```

Here we create a 16-bit integer view that shares the same buffer as the existing 32-bit view and we output all the values in the buffer as 16-bit integers. Now we get the output `0`, `0`, `2`, `0`, `4`, `0`, `6`, `0` (assuming little-endian encoding):

```
Int16Array  |   0  |  0   |   2  |  0   |   4  |  0   |   6  |  0   |
Int32Array  |      0      |      2      |      4      |      6      |
ArrayBuffer | 00 00 00 00 | 02 00 00 00 | 04 00 00 00 | 06 00 00 00 |
```

You can go a step farther, though. Consider this:

```
int16View[0] = 32;
console.log(`Entry 0 in the 32-bit array is now ${int32View[0]}`);
```

The output from this is `"Entry 0 in the 32-bit array is now 32"`.

In other words, the two arrays are indeed viewed on the same data buffer, treating it as different formats.

```
Int16Array  |  32  |  0   |   2  |  0   |   4  |  0   |   6  |  0   |
Int32Array  |     32      |      2      |      4      |      6      |
ArrayBuffer | 20 00 00 00 | 02 00 00 00 | 04 00 00 00 | 06 00 00 00 |
```

You can do this with any view type, although if you set an integer and then read it as a floating-point number, you will probably get a strange result because the bits are interpreted differently.

```
const float32View = new Float32Array(buffer);
console.log(float32View[0]); // 4.484155085839415e-44
```

Reading text from a buffer

Buffers don't always represent numbers. For example, reading a file can give you a text data buffer. You can read this data out of the buffer using a typed array.

The following reads UTF-8 text using the `TextDecoder` web API:

```
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
// Data manually written here, but pretend it was already in the buffer
uint8.set([228, 189, 160, 229, 165, 189]);
const text = new TextDecoder().decode(uint8);
console.log(text); // "你好"
```

The following reads UTF-16 text using the `String.fromCharCode()` method:

```
const buffer = new ArrayBuffer(8);
const uint16 = new Uint16Array(buffer);
// Data manually written here, but pretend it was already in the buffer
uint16.set([0x4f60, 0x597d]);
const text = String.fromCharCode(...uint16);
console.log(text); // "你好"
```

Working with complex data structures

By combining a single buffer with multiple views of different types, starting at different offsets into the buffer, you can interact with data objects containing multiple data types. This lets you, for example, interact with complex data structures from WebGL or data files.

Consider this C structure:

```
struct someStruct {
    unsigned long id;
    char username[16];
    float amountDue;
};
```

You can access a buffer containing data in this format like this:

```
const buffer = new ArrayBuffer(24);

// … read the data into the buffer …

const idView = new Uint32Array(buffer, 0, 1);
const usernameView = new Uint8Array(buffer, 4, 16);
const amountDueView = new Float32Array(buffer, 20, 1);
```

Then you can access, for example, the amount due with `amountDueView[0]`.

> **Note:** The data structure alignment in a C structure is platform-dependent. Take precautions and considerations for these padding differences.

Conversion to normal arrays

After processing a typed array, it is sometimes useful to convert it back to a normal array in order to benefit from the `Array` prototype. This can be done using `Array.from()`:

```
const typedArray = new Uint8Array([1, 2, 3, 4]);
const normalArray = Array.from(typedArray);
```

as well as the spread syntax:

```
const typedArray = new Uint8Array([1, 2, 3, 4]);
const normalArray = [...typedArray];
```

See also

- Faster Canvas Pixel Manipulation with Typed Arrays on hacks.mozilla.org (2011)
- Typed arrays - Binary data in the browser on web.dev (2012)
- Endianness
- `ArrayBuffer`
- `DataView`
- `TypedArray`
- `SharedArrayBuffer`

- Previous
- Next

Using classes

- Previous
- Next

JavaScript is a prototype-based language — an object's behaviors are specified by its own properties and its prototype's properties. However, with the addition of classes, the creation of hierarchies of objects and the inheritance of properties and their values are much more in line with other object-oriented languages such as Java. In this section, we will demonstrate how objects can be created from classes.

In many other languages, *classes*, or constructors, are clearly distinguished from *objects*, or instances. In JavaScript, classes are mainly an abstraction over the existing prototypical inheritance mechanism — all patterns are convertible to prototype-based inheritance. Classes themselves are normal JavaScript values as well, and have their own prototype chains. In fact, most plain JavaScript functions can be used as constructors — you use the `new` operator with a constructor function to create a new object.

We will be playing with the well-abstracted class model in this tutorial, and discuss what semantics classes offer. If you want to dive deep into the underlying prototype system, you can read the Inheritance and the prototype chain guide.

This chapter assumes that you are already somewhat familiar with JavaScript and that you have used ordinary objects.

Overview of classes

If you have some hands-on experience with JavaScript, or have followed along with the guide, you probably have already used classes, even if you haven't created one. For example, this may seem familiar to you:

```
const bigDay = new Date(2019, 6, 19);
console.log(bigDay.toLocaleDateString());
if (bigDay.getTime() < Date.now()) {
  console.log("Once upon a time...");
}
```

On the first line, we created an instance of the class `Date`, and called it `bigDay`. On the second line, we called a method `toLocaleDateString()` on the `bigDay` instance, which returns a string. Then, we compared two numbers: one returned from the `getTime()` method, the other directly called from the `Date` class *itself*, as `Date.now()`.

`Date` is a built-in class of JavaScript. From this example, we can get some basic ideas of what classes do:

- Classes create objects through the `new` operator.
- Each object has some properties (data or method) added by the class.
- The class stores some properties (data or method) itself, which are usually used to interact with instances.

These correspond to the three key features of classes:

- Constructor;
- Instance methods and instance fields;
- Static methods and static fields.

Declaring a class

Classes are usually created with *class declarations*.

```
class MyClass {
  // class body...
}
```

Within a class body, there are a range of features available.

```
class MyClass {
  // Constructor
  constructor() {
    // Constructor body
  }
  // Instance field
  myField = "foo";
  // Instance method
  myMethod() {
    // myMethod body
  }
  // Static field
  static myStaticField = "bar";
  // Static method
  static myStaticMethod() {
    // myStaticMethod body
  }
  // Static block
  static {
    // Static initialization code
  }
  // Fields, methods, static fields, and static methods all have
  // "private" forms
  #myPrivateField = "bar";
}
```

If you came from a pre-ES6 world, you may be more familiar with using functions as constructors. The pattern above would roughly translate to the following with function constructors:

```
function MyClass() {
  this.myField = "foo";
  // Constructor body
}
MyClass.myStaticField = "bar";
MyClass.myStaticMethod = function () {
  // myStaticMethod body
};
MyClass.prototype.myMethod = function () {
  // myMethod body
};

(function () {
  // Static initialization code
})();
```

> **Note:** Private fields and methods are new features in classes with no trivial equivalent in function constructors.

Constructing a class

After a class has been declared, you can create instances of it using the `new` operator.

```
const myInstance = new MyClass();
console.log(myInstance.myField); // 'foo'
myInstance.myMethod();
```

Typical function constructors can both be constructed with `new` and called without `new`. However, attempting to "call" a class without `new` will result in an error.

```
const myInstance = MyClass(); // TypeError: Class constructor MyClass cannot be invoked without 'new'
```

Class declaration hoisting

Unlike function declarations, class declarations are not hoisted (or, in some interpretations, hoisted but with the temporal dead zone restriction), which means you cannot use a class before it is declared.

```
new MyClass(); // ReferenceError: Cannot access 'MyClass' before initialization

class MyClass {}
```

This behavior is similar to variables declared with `let` and `const`.

Class expressions

Similar to functions, class declarations also have their expression counterparts.

```
const MyClass = class {
  // Class body...
};
```

Class expressions can have names as well. The expression's name is only visible to the class's body.

```
const MyClass = class MyClassLongerName {
  // Class body. Here MyClass and MyClassLongerName point to the same class.
};
new MyClassLongerName(); // ReferenceError: MyClassLongerName is not defined
```

Constructor

Perhaps the most important job of a class is to act as a "factory" for objects. For example, when we use the `Date` constructor, we expect it to give a new object which represents the date data we passed in — which we can then manipulate with other methods the instance exposes. In classes, the instance creation is done by the constructor.

As an example, we would create a class called `Color`, which represents a specific color. Users create colors through passing in an RGB triplet.

```
class Color {
  constructor(r, g, b) {
    // Assign the RGB values as a property of `this`.
    this.values = [r, g, b];
  }
}
```

Open your browser's devtools, paste the above code into the console, and then create an instance:

```
const red = new Color(255, 0, 0);
console.log(red);
```

You should see some output like this:

```
Object { values: (3) […] }
  values: Array(3) [ 255, 0, 0 ]
```

You have successfully created a `Color` instance, and the instance has a `values` property, which is an array of the RGB values you passed in. That is pretty much equivalent to the following:

```
function createColor(r, g, b) {
  return {
    values: [r, g, b],
  };
}
```

The constructor's syntax is exactly the same as a normal function — which means you can use other syntaxes, like rest parameters:

```
class Color {
  constructor(...values) {
    this.values = values;
  }
}

const red = new Color(255, 0, 0);
// Creates an instance with the same shape as above.
```

Each time you call `new`, a different instance is created.

```
const red = new Color(255, 0, 0);
const anotherRed = new Color(255, 0, 0);
console.log(red === anotherRed); // false
```

Within a class constructor, the value of `this` points to the newly created instance. You can assign properties to it, or read existing properties (especially methods — which we will cover next).

The `this` value will be automatically returned as the result of `new`. You are advised to not return any value from the constructor — because if you return a non-primitive value, it will become the value of the `new` expression, and the value of `this` is dropped. (You can read more about what `new` does in its description.)

```
class MyClass {
  constructor() {
    this.myField = "foo";
    return {};
  }
}

console.log(new MyClass().myField); // undefined
```

Instance methods

If a class only has a constructor, it is not much different from a `createX` factory function which just creates plain objects. However, the power of classes is that they can be used as "templates" which automatically assign methods to instances.

For example, for `Date` instances, you can use a range of methods to get different information from a single date value, such as the year, month, day of the week, etc. You can also set those values through the `setX` counterparts like `setFullYear`.

For our own `Color` class, we can add a method called `getRed` which returns the red value of the color.

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  getRed() {
    return this.values[0];
  }
}

const red = new Color(255, 0, 0);
console.log(red.getRed()); // 255
```

Without methods, you may be tempted to define the function within the constructor:

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
    this.getRed = function () {
      return this.values[0];
    };
  }
}
```

This also works. However, a problem is that this creates a new function every time a `Color` instance is created, even when they all do the same thing!

```
console.log(new Color().getRed === new Color().getRed); // false
```

In contrast, if you use a method, it will be shared between all instances. A function can be shared between all instances, but still have its behavior differ when different instances call it, because the value of `this` is different. If you are curious *where* this method is stored in — it's defined on the prototype of all instances, or `Color.prototype`, which is explained in more detail in Inheritance and the prototype chain.

Similarly, we can create a new method called `setRed`, which sets the red value of the color.

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  getRed() {
    return this.values[0];
  }
  setRed(value) {
    this.values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.setRed(0);
console.log(red.getRed()); // 0; of course, it should be called "black" at this stage!
```

Private fields

You might be wondering: why do we want to go to the trouble of using `getRed` and `setRed` methods, when we can directly access the `values` array on the instance?

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
}

const red = new Color(255, 0, 0);
red.values[0] = 0;
console.log(red.values[0]); // 0
```

There is a philosophy in object-oriented programming called "encapsulation". This means you should not access the underlying implementation of an object, but instead use well-abstracted methods to interact with it. For example, if we suddenly decided to represent colors as HSL instead:

```
class Color {
  constructor(r, g, b) {
    // values is now an HSL array!
    this.values = rgbToHSL([r, g, b]);
  }
  getRed() {
    return hslToRGB(this.values)[0];
  }
```

```
  setRed(value) {
    const rgb = hslToRGB(this.values);
    rgb[0] = value;
    this.values = rgbToHSL(rgb);
  }
}

const red = new Color(255, 0, 0);
console.log(red.values[0]); // 0; It's not 255 anymore, because the H value for pure red is 0
```

The user assumption that `values` means the RGB value suddenly collapses, and it may cause their logic to break. So, if you are an implementor of a class, you would want to hide the internal data structure of your instance from your user, both to keep the API clean and to prevent the user's code from breaking when you do some "harmless refactors". In classes, this is done through *private fields*.

A private field is an identifier prefixed with `#` (the hash symbol). The hash is an integral part of the field's name, which means a private field can never clash names with a public field or method. In order to refer to a private field anywhere in the class, you must *declare* it in the class body (you can't create a private element on the fly). Apart from this, a private field is pretty much equivalent to a normal property.

```
class Color {
  // Declare: every Color instance has a private field called #values.
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  getRed() {
    return this.#values[0];
  }
  setRed(value) {
    this.#values[0] = value;
  }
}

const red = new Color(255, 0, 0);
console.log(red.getRed()); // 255
```

Accessing private fields outside the class is an early syntax error. The language can guard against this because `#privateField` is a special syntax, so it can do some static analysis and find all usage of private fields before even evaluating the code.

```
console.log(red.#values); // SyntaxError: Private field '#values' must be declared in an enclosing class
```

> **Note:** Code run in the Chrome console can access private elements outside the class. This is a DevTools-only relaxation of the JavaScript syntax restriction.

Private fields in JavaScript are *hard private*: if the class does not implement methods that expose these private fields, there's absolutely no mechanism to retrieve them from outside the class. This means you are safe to do any refactors to your class's private fields, as long as the behavior of exposed methods stay the same.

After we've made the `values` field private, we can add some more logic in the `getRed` and `setRed` methods, instead of making them simple pass-through methods. For example, we can add a check in `setRed` to see if it's a valid R value:

```
class Color {
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  getRed() {
    return this.#values[0];
  }
  setRed(value) {
    if (value < 0 || value > 255) {
      throw new RangeError("Invalid R value");
    }
    this.#values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.setRed(1000); // RangeError: Invalid R value
```

If we leave the `values` property exposed, our users can easily circumvent that check by assigning to `values[0]` directly, and create invalid colors. But with a well-encapsulated API, we can make our code more robust and prevent logic errors downstream.

A class method can read the private fields of other instances, as long as they belong to the same class.

```
class Color {
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  redDifference(anotherColor) {
    // #values doesn't necessarily need to be accessed from this:
    // you can access private fields of other instances belonging
    // to the same class.
```

```
    return this.#values[0] - anotherColor.#values[0];
  }
}

const red = new Color(255, 0, 0);
const crimson = new Color(220, 20, 60);
red.redDifference(crimson); // 35
```

However, if `anotherColor` is not a Color instance, `#values` won't exist. (Even if another class has an identically named `#values` private field, it's not referring to the same thing and cannot be accessed here.) Accessing a nonexistent private element throws an error instead of returning `undefined` like normal properties do. If you don't know if a private field exists on an object and you wish to access it without using `try/catch` to handle the error, you can use the `in` operator.

```
class Color {
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  redDifference(anotherColor) {
    if (!(#values in anotherColor)) {
      throw new TypeError("Color instance expected");
    }
    return this.#values[0] - anotherColor.#values[0];
  }
}
```

> **Note:** Keep in mind that the `#` is a special identifier syntax, and you can't use the field name as if it's a string. `"#values" in anotherColor` would look for a property name literally called `"#values"`, instead of a private field.

There are some limitations in using private elements: the same name can't be declared twice in a single class, and they can't be deleted. Both lead to early syntax errors.

```
class BadIdeas {
  #firstName;
  #firstName; // syntax error occurs here
  #lastName;
  constructor() {
    delete this.#lastName; // also a syntax error
  }
}
```

Methods, getters, and setters can be private as well. They're useful when you have something complex that the class needs to do internally but no other part of the code should be allowed to call.

For example, imagine creating HTML custom elements that should do something somewhat complicated when clicked/tapped/otherwise activated. Furthermore, the somewhat complicated things that happen when the element is clicked should be restricted to this class, because no other part of the JavaScript will (or should) ever access it.

```
class Counter extends HTMLElement {
  #xValue = 0;
  constructor() {
    super();
    this.onclick = this.#clicked.bind(this);
  }
  get #x() {
    return this.#xValue;
  }
  set #x(value) {
    this.#xValue = value;
    window.requestAnimationFrame(this.#render.bind(this));
  }
  #clicked() {
    this.#x++;
  }
  #render() {
    this.textContent = this.#x.toString();
  }
  connectedCallback() {
    this.#render();
  }
}

customElements.define("num-counter", Counter);
```

In this case, pretty much every field and method is private to the class. Thus, it presents an interface to the rest of the code that's essentially just like a built-in HTML element. No other part of the program has the power to affect any of the internals of `Counter`.

Accessor fields

`color.getRed()` and `color.setRed()` allow us to read and write to the red value of a color. If you come from languages like Java, you will be very familiar with this pattern. However, using methods to simply access a property is still somewhat unergonomic in JavaScript. *Accessor fields* allow us to manipulate something as if it is an "actual property".

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  get red() {
    return this.values[0];
  }
  set red(value) {
    this.values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.red = 0;
console.log(red.red); // 0
```

It looks as if the object has a property called `red` — but actually, no such property exists on the instance! There are only two methods, but they are prefixed with `get` and `set`, which allows them to be manipulated as if they were properties.

If a field only has a getter but no setter, it will be effectively read-only.

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  get red() {
    return this.values[0];
  }
}

const red = new Color(255, 0, 0);
red.red = 0;
console.log(red.red); // 255
```

In strict mode, the `red.red = 0` line will throw a type error: "Cannot set property red of #<Color> which has only a getter". In non-strict mode, the assignment is silently ignored.

Public fields

Private fields also have their public counterparts, which allow every instance to have a property. Fields are usually designed to be independent of the constructor's parameters.

```
class MyClass {
  luckyNumber = Math.random();
}
console.log(new MyClass().luckyNumber); // 0.5
console.log(new MyClass().luckyNumber); // 0.3
```

Public fields are almost equivalent to assigning a property to `this`. For example, the above example can also be converted to:

```
class MyClass {
  constructor() {
    this.luckyNumber = Math.random();
  }
}
```

Static properties

With the `Date` example, we have also encountered the `Date.now()` method, which returns the current date. This method does not belong to any date instance — it belongs to the class itself. However, it's put on the `Date` class instead of being exposed as a global `DateNow()` function, because it's mostly useful when dealing with date instances.

> **Note:** Prefixing utility methods with what they deal with is called "namespacing" and is considered a good practice. For example, in addition to the older, unprefixed `parseInt()` method, JavaScript also later added the prefixed `Number.parseInt()` method to indicate that it's for dealing with numbers.

Static properties are a group of class features that are defined on the class itself, rather than on individual instances of the class. These features include:

- Static methods
- Static fields
- Static getters and setters

Everything also has private counterparts. For example, for our `Color` class, we can create a static method that checks whether a given triplet is a valid RGB value:

```
class Color {
  static isValid(r, g, b) {
    return r >= 0 && r <= 255 && g >= 0 && g <= 255 && b >= 0 && b <= 255;
  }
}
```

```
Color.isValid(255, 0, 0); // true
Color.isValid(1000, 0, 0); // false
```

Static properties are very similar to their instance counterparts, except that:

- They are all prefixed with `static`, and
- They are not accessible from instances.

```
console.log(new Color(0, 0, 0).isValid); // undefined
```

There is also a special construct called a *static initialization block*, which is a block of code that runs when the class is first loaded.

```
class MyClass {
  static {
    MyClass.myStaticProperty = "foo";
  }
}

console.log(MyClass.myStaticProperty); // 'foo'
```

Static initialization blocks are almost equivalent to immediately executing some code after a class has been declared. The only difference is that they have access to static private elements.

Extends and inheritance

A key feature that classes bring about (in addition to ergonomic encapsulation with private fields) is *inheritance*, which means one object can "borrow" a large part of another object's behaviors, while overriding or enhancing certain parts with its own logic.

For example, suppose our `Color` class now needs to support transparency. We may be tempted to add a new field that indicates its transparency:

```
class Color {
  #values;
  constructor(r, g, b, a = 1) {
    this.#values = [r, g, b, a];
  }
  get alpha() {
    return this.#values[3];
  }
  set alpha(value) {
    if (value < 0 || value > 1) {
      throw new RangeError("Alpha value must be between 0 and 1");
    }
    this.#values[3] = value;
  }
}
```

However, this means every instance — even the vast majority which aren't transparent (those with an alpha value of 1) — will have to have the extra alpha value, which is not very elegant. Plus, if the features keep growing, our `Color` class will become very bloated and hard to maintain.

Instead, in object-oriented programming, we would create a *derived class*. The derived class has access to all public properties of the parent class. In JavaScript, derived classes are declared with an `extends` clause, which indicates the class it extends from.

```
class ColorWithAlpha extends Color {
  #alpha;
  constructor(r, g, b, a) {
    super(r, g, b);
    this.#alpha = a;
  }
  get alpha() {
    return this.#alpha;
  }
  set alpha(value) {
    if (value < 0 || value > 1) {
      throw new RangeError("Alpha value must be between 0 and 1");
    }
    this.#alpha = value;
  }
}
```

There are a few things that have immediately come to attention. First is that in the constructor, we are calling `super(r, g, b)`. It is a language requirement to call `super()` before accessing `this`. The `super()` call calls the parent class's constructor to initialize `this` — here it's roughly equivalent to `this = new Color(r, g, b)`. You can have code before `super()`, but you cannot access `this` before `super()` — the language prevents you from accessing the uninitialized `this`.

After the parent class is done with modifying `this`, the derived class can do its own logic. Here we added a private field called `#alpha`, and also provided a pair of getter/setters to interact with them.

A derived class inherits all methods from its parent. For example, consider the `get red()` accessor we added to the `Color` in the Accessor fields section—even though we haven't declared one in `ColorWithAlpha`, we can still access `red` because this behavior is specified by the parent class:

```
const color = new ColorWithAlpha(255, 0, 0, 0.5);
console.log(color.red); // 255
```

Derived classes can also override methods from the parent class. For example, all classes implicitly inherit the `Object` class, which defines some basic methods like `toString()`. However, the base `toString()` method is notoriously useless, because it prints `[object Object]` in most cases:

```
console.log(red.toString()); // [object Object]
```

Instead, our class can override it to print the color's RGB values:

```
class Color {
  #values;
  // …
  toString() {
    return this.#values.join(", ");
  }
}

console.log(new Color(255, 0, 0).toString()); // '255, 0, 0'
```

Within derived classes, you can access the parent class's methods by using `super`. This allows you to build enhancement methods and avoid code duplication.

```
class ColorWithAlpha extends Color {
  #alpha;
  // …
  toString() {
    // Call the parent class's toString() and build on the return value
    return `${super.toString()}, ${this.#alpha}`;
  }
}

console.log(new ColorWithAlpha(255, 0, 0, 0.5).toString()); // '255, 0, 0, 0.5'
```

When you use `extends`, the static methods inherit from each other as well, so you can also override or enhance them.

```
class ColorWithAlpha extends Color {
  // …
  static isValid(r, g, b, a) {
    // Call the parent class's isValid() and build on the return value
    return super.isValid(r, g, b) && a >= 0 && a <= 1;
  }
}

console.log(ColorWithAlpha.isValid(255, 0, 0, -1)); // false
```

Derived classes don't have access to the parent class's private fields — this is another key aspect to JavaScript private fields being "hard private". Private fields are scoped to the class body itself and do not grant access to *any* outside code.

```
class ColorWithAlpha extends Color {
  log() {
    console.log(this.#values); // SyntaxError: Private field '#values' must be declared in an enclosing class
  }
}
```

A class can only extend from one class. This prevents problems in multiple inheritance like the diamond problem. However, due to the dynamic nature of JavaScript, it's still possible to achieve the effect of multiple inheritance through class composition and mixins.

Instances of derived classes are also instances of the base class.

```
const color = new ColorWithAlpha(255, 0, 0, 0.5);
console.log(color instanceof Color); // true
console.log(color instanceof ColorWithAlpha); // true
```

Why classes?

The guide has been pragmatic so far: we are focusing on *how* classes can be used, but there's one question unanswered: *why* would one use a class? The answer is: it depends.

Classes introduce a *paradigm*, or a way to organize your code. Classes are the foundations of object-oriented programming, which is built on concepts like inheritance and polymorphism (especially *subtype polymorphism*). However, many people are philosophically against certain OOP practices and don't use classes as a result.

For example, one thing that makes `Date` objects infamous is that they're *mutable*.

```
function incrementDay(date) {
  return new Date(date.setDate(date.getDate() + 1));
}
const date = new Date(); // 2019-06-19
const newDay = incrementDay(date);
console.log(newDay); // 2019-06-20
// The old date is modified as well!?
console.log(date); // 2019-06-20
```

Mutability and internal state are important aspects of object-oriented programming, but often make code hard to reason with — because any seemingly innocent operation may have unexpected side effects and change the behavior in other parts of the program.

In order to reuse code, we usually resort to extending classes, which can create big hierarchies of inheritance patterns.

However, it is often hard to describe inheritance cleanly when one class can only extend one other class. Often, we want the behavior of multiple classes. In Java, this is done through interfaces; in JavaScript, it can be done through mixins. But at the end of the day, it's still not very convenient.

On the brighter side, classes are a very powerful way to organize our code on a higher level. For example, without the `Color` class, we may need to create a dozen of utility functions:

```
function isRed(color) {
  return color.red === 255;
}
function isValidColor(color) {
  return (
    color.red >= 0 &&
    color.red <= 255 &&
    color.green >= 0 &&
    color.green <= 255 &&
    color.blue >= 0 &&
    color.blue <= 255
  );
}
// …
```

But with classes, we can congregate them all under the `Color` namespace, which improves readability. In addition, the introduction of private fields allows us to hide certain data from downstream users, creating a clean API.

In general, you should consider using classes when you want to create objects that store their own internal data and expose a lot of behavior. Take built-in JavaScript classes as examples:

- The `Map` and `Set` classes store a collection of elements and allow you to access them by key using `get()`, `set()`, `has()`, etc.
- The `Date` class stores a date as a Unix timestamp (a number) and allows you to format, update, and read individual date components.
- The `Error` class stores information about a particular exception, including the error message, stack trace, cause, etc. It's one of the few classes that come with a rich inheritance structure: there are multiple built-in classes like `TypeError` and `ReferenceError` that extend `Error`. In the case of errors, this inheritance allows refining the semantics of errors: each error class represents a specific type of error, which can be easily checked with `instanceof`.

JavaScript offers the mechanism to organize your code in a canonical object-oriented way, but whether and how to use it is entirely up to the programmer's discretion.

- Previous
- Next

Using promises

- Previous
- Next

A `Promise` is an object representing the eventual completion or failure of an asynchronous operation. Since most people are consumers of already-created promises, this guide will explain consumption of returned promises before explaining how to create them.

Essentially, a promise is a returned object to which you attach callbacks, instead of passing callbacks into a function. Imagine a function, `createAudioFileAsync()`, which asynchronously generates a sound file given a configuration record and two callback functions: one called if the audio file is successfully created, and the other called if an error occurs.

Here's some code that uses `createAudioFileAsync()`:

```
function successCallback(result) {
  console.log(`Audio file ready at URL: ${result}`);
}

function failureCallback(error) {
  console.error(`Error generating audio file: ${error}`);
}

createAudioFileAsync(audioSettings, successCallback, failureCallback);
```

If `createAudioFileAsync()` were rewritten to return a promise, you would attach your callbacks to it instead:

```
createAudioFileAsync(audioSettings).then(successCallback, failureCallback);
```

This convention has several advantages. We will explore each one.

Chaining

A common need is to execute two or more asynchronous operations back to back, where each subsequent operation starts when the previous operation succeeds, with the result from the previous step. In the old days, doing several asynchronous operations in a row would lead to the classic callback hell:

```
doSomething(function (result) {
  doSomethingElse(result, function (newResult) {
    doThirdThing(newResult, function (finalResult) {
      console.log(`Got the final result: ${finalResult}`);
    }, failureCallback);
```

```
  }, failureCallback);
}, failureCallback);
```

With promises, we accomplish this by creating a promise chain. The API design of promises makes this great, because callbacks are attached to the returned promise object, instead of being passed into a function.

Here's the magic: the `then()` function returns a **new promise**, different from the original:

```
const promise = doSomething();
const promise2 = promise.then(successCallback, failureCallback);
```

This second promise (`promise2`) represents the completion not just of `doSomething()`, but also of the `successCallback` or `failureCallback` you passed in — which can be other asynchronous functions returning a promise. When that's the case, any callbacks added to `promise2` get queued behind the promise returned by either `successCallback` or `failureCallback`.

> **Note:** If you want a working example to play with, you can use the following template to create any function returning a promise:
>
> ```
> function doSomething() {
>   return new Promise((resolve) => {
>     setTimeout(() => {
>       // Other things to do before completion of the promise
>       console.log("Did something");
>       // The fulfillment value of the promise
>       resolve("https://example.com/");
>     }, 200);
>   });
> }
> ```
>
> The implementation is discussed in the Creating a Promise around an old callback API section below.

With this pattern, you can create longer chains of processing, where each promise represents the completion of one asynchronous step in the chain. In addition, the arguments to `then` are optional, and `catch(failureCallback)` is short for `then(null, failureCallback)` — so if your error handling code is the same for all steps, you can attach it to the end of the chain:

```
doSomething()
  .then(function (result) {
    return doSomethingElse(result);
  })
  .then(function (newResult) {
    return doThirdThing(newResult);
  })
  .then(function (finalResult) {
    console.log(`Got the final result: ${finalResult}`);
  })
  .catch(failureCallback);
```

You might see this expressed with arrow functions instead:

```
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => {
    console.log(`Got the final result: ${finalResult}`);
  })
  .catch(failureCallback);
```

> **Note:** Arrow function expressions can have an implicit return; so, `() => x` is short for `() => { return x; }`.

`doSomethingElse` and `doThirdThing` can return any value — if they return promises, that promise is first waited until it settles, and the next callback receives the fulfillment value, not the promise itself. It is important to always return promises from `then` callbacks, even if the promise always resolves to `undefined`. If the previous handler started a promise but did not return it, there's no way to track its settlement anymore, and the promise is said to be "floating".

```
doSomething()
  .then((url) => {
    // Missing `return` keyword in front of fetch(url).
    fetch(url);
  })
  .then((result) => {
    // result is undefined, because nothing is returned from the previous
    // handler. There's no way to know the return value of the fetch()
    // call anymore, or whether it succeeded at all.
  });
```

By returning the result of the `fetch` call (which is a promise), we can both track its completion and receive its value when it completes.

```
doSomething()
  .then((url) => {
    // `return` keyword added
```

```
    return fetch(url);
  })
  .then((result) => {
    // result is a Response object
  });
```

Floating promises could be worse if you have race conditions — if the promise from the last handler is not returned, the next `then` handler will be called early, and any value it reads may be incomplete.

```
const listOfIngredients = [];

doSomething()
  .then((url) => {
    // Missing `return` keyword in front of fetch(url).
    fetch(url)
      .then((res) => res.json())
      .then((data) => {
        listOfIngredients.push(data);
      });
  })
  .then(() => {
    console.log(listOfIngredients);
    // listOfIngredients will always be [], because the fetch request hasn't completed yet.
  });
```

Therefore, as a rule of thumb, whenever your operation encounters a promise, return it and defer its handling to the next `then` handler.

```
const listOfIngredients = [];

doSomething()
  .then((url) => {
    // `return` keyword now included in front of fetch call.
    return fetch(url)
      .then((res) => res.json())
      .then((data) => {
        listOfIngredients.push(data);
      });
  })
  .then(() => {
    console.log(listOfIngredients);
    // listOfIngredients will now contain data from fetch call.
  });
```

Even better, you can flatten the nested chain into a single chain, which is simpler and makes error handling easier. The details are discussed in the Nesting section below.

```
doSomething()
  .then((url) => fetch(url))
  .then((res) => res.json())
  .then((data) => {
    listOfIngredients.push(data);
  })
  .then(() => {
    console.log(listOfIngredients);
  });
```

Using `async/await` can help you write code that's more intuitive and resembles synchronous code. Below is the same example using `async/await`:

```
async function logIngredients() {
  const url = await doSomething();
  const res = await fetch(url);
  const data = await res.json();
  listOfIngredients.push(data);
  console.log(listOfIngredients);
}
```

Note how the code looks exactly like synchronous code, except for the `await` keywords in front of promises. One of the only tradeoffs is that it may be easy to forget the `await` keyword, which can only be fixed when there's a type mismatch (e.g., trying to use a promise as a value).

`async/await` builds on promises — for example, `doSomething()` is the same function as before, so there's minimal refactoring needed to change from promises to `async/await`. You can read more about the `async/await` syntax in the async functions and `await` references.

> **Note:** `async/await` has the same concurrency semantics as normal promise chains. `await` within one async function does not stop the entire program, only the parts that depend on its value, so other async jobs can still run while the `await` is pending.

Error handling

You might recall seeing `failureCallback` three times in the pyramid of doom earlier, compared to only once at the end of the promise chain:

```
doSomething()
  .then((result) => doSomethingElse(result))
```

```
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => console.log(`Got the final result: ${finalResult}`))
  .catch(failureCallback);
```

If there's an exception, the browser will look down the chain for `.catch()` handlers or `onRejected`. This is very much modeled after how synchronous code works:

```
try {
  const result = syncDoSomething();
  const newResult = syncDoSomethingElse(result);
  const finalResult = syncDoThirdThing(newResult);
  console.log(`Got the final result: ${finalResult}`);
} catch (error) {
  failureCallback(error);
}
```

This symmetry with asynchronous code culminates in the `async/await` syntax:

```
async function foo() {
  try {
    const result = await doSomething();
    const newResult = await doSomethingElse(result);
    const finalResult = await doThirdThing(newResult);
    console.log(`Got the final result: ${finalResult}`);
  } catch (error) {
    failureCallback(error);
  }
}
```

Promises solve a fundamental flaw with the callback pyramid of doom, by catching all errors, even thrown exceptions and programming errors. This is essential for functional composition of asynchronous operations. All errors are now handled by the `catch()` method at the end of the chain, and you should almost never need to use `try/catch` without using `async/await`.

Nesting

In the examples above involving `listOfIngredients`, the first one has one promise chain nested in the return value of another `then()` handler, while the second one uses an entirely flat chain. Simple promise chains are best kept flat without nesting, as nesting can be a result of careless composition.

Nesting is a control structure to limit the scope of `catch` statements. Specifically, a nested `catch` only catches failures in its scope and below, not errors higher up in the chain outside the nested scope. When used correctly, this gives greater precision in error recovery:

```
doSomethingCritical()
  .then((result) =>
    doSomethingOptional(result)
      .then((optionalResult) => doSomethingExtraNice(optionalResult))
      .catch((e) => {}),
  ) // Ignore if optional stuff fails; proceed.
  .then(() => moreCriticalStuff())
  .catch((e) => console.error(`Critical failure: ${e.message}`));
```

Note that the optional steps here are nested — with the nesting caused not by the indentation, but by the placement of the outer `(` and `)` parentheses around the steps.

The inner error-silencing `catch` handler only catches failures from `doSomethingOptional()` and `doSomethingExtraNice()`, after which the code resumes with `moreCriticalStuff()`. Importantly, if `doSomethingCritical()` fails, its error is caught by the final (outer) `catch` only, and does not get swallowed by the inner `catch` handler.

In `async/await`, this code looks like:

```
async function main() {
  try {
    const result = await doSomethingCritical();
    try {
      const optionalResult = await doSomethingOptional(result);
      await doSomethingExtraNice(optionalResult);
    } catch (e) {
      // Ignore failures in optional steps and proceed.
    }
    await moreCriticalStuff();
  } catch (e) {
    console.error(`Critical failure: ${e.message}`);
  }
}
```

> **Note:** If you don't have sophisticated error handling, you very likely don't need nested `then` handlers. Instead, use a flat chain and put the error handling logic at the end.

Chaining after a catch

It's possible to chain *after* a failure, i.e., a `catch`, which is useful to accomplish new actions even after an action failed in the chain. Read the following example:

```
doSomething()
  .then(() => {
    throw new Error("Something failed");

    console.log("Do this");
  })
  .catch(() => {
    console.error("Do that");
  })
  .then(() => {
    console.log("Do this, no matter what happened before");
  });
```

This will output the following text:

```
Do that
Do this, no matter what happened before
```

> **Note:** The text "Do this" is not displayed because the "Something failed" error caused a rejection.

In `async/await`, this code looks like:

```
async function main() {
  try {
    await doSomething();
    throw new Error("Something failed");
    console.log("Do this");
  } catch (e) {
    console.error("Do that");
  }
  console.log("Do this, no matter what happened before");
}
```

Promise rejection events

If a promise rejection event is not handled by any handler, it bubbles to the top of the call stack, and the host needs to surface it. On the web, whenever a promise is rejected, one of two events is sent to the global scope (generally, this is either the `window` or, if being used in a web worker, it's the `Worker` or other worker-based interface). The two events are:

`unhandledrejection`

: Sent when a promise is rejected but there is no rejection handler available.

`rejectionhandled`

: Sent when a handler is attached to a rejected promise that has already caused an `unhandledrejection` event.

In both cases, the event (of type `PromiseRejectionEvent`) has as members a `promise` property indicating the promise that was rejected, and a `reason` property that provides the reason given for the promise to be rejected.

These make it possible to offer fallback error handling for promises, as well as to help debug issues with your promise management. These handlers are global per context, so all errors will go to the same event handlers, regardless of source.

In Node.js, handling promise rejection is slightly different. You capture unhandled rejections by adding a handler for the Node.js `unhandledRejection` event (notice the difference in capitalization of the name), like this:

```
process.on("unhandledRejection", (reason, promise) => {
  // Add code here to examine the "promise" and "reason" values
});
```

For Node.js, to prevent the error from being logged to the console (the default action that would otherwise occur), adding that `process.on()` listener is all that's necessary; there's no need for an equivalent of the browser runtime's `preventDefault()` method.

However, if you add that `process.on` listener but don't also have code within it to handle rejected promises, they will just be dropped on the floor and silently ignored. So ideally, you should add code within that listener to examine each rejected promise and make sure it was not caused by an actual code bug.

Composition

There are four composition tools for running asynchronous operations concurrently: `Promise.all()`, `Promise.allSettled()`, `Promise.any()`, and `Promise.race()`.

We can start operations at the same time and wait for them all to finish like this:

```
Promise.all([func1(), func2(), func3()]).then(([result1, result2, result3]) => {
  // use result1, result2 and result3
});
```

If one of the promises in the array rejects, `Promise.all()` immediately rejects the returned promise. The other operations continue to run, but their outcomes are not available via the return value of `Promise.all()`. This may cause unexpected state or behavior. `Promise.allSettled()` is another composition tool that ensures all operations are complete before resolving.

These methods all run promises concurrently — a sequence of promises are started simultaneously and do not wait for each other. Sequential composition is possible using some clever JavaScript:

```
[func1, func2, func3]
  .reduce((p, f) => p.then(f), Promise.resolve())
  .then((result3) => {
    /* use result3 */
  });
```

In this example, we reduce an array of asynchronous functions down to a promise chain. The code above is equivalent to:

```
Promise.resolve()
  .then(func1)
  .then(func2)
  .then(func3)
  .then((result3) => {
    /* use result3 */
  });
```

This can be made into a reusable compose function, which is common in functional programming:

```
const applyAsync = (acc, val) => acc.then(val);
const composeAsync =
  (...funcs) =>
  (x) =>
    funcs.reduce(applyAsync, Promise.resolve(x));
```

The `composeAsync()` function accepts any number of functions as arguments and returns a new function that accepts an initial value to be passed through the composition pipeline:

```
const transformData = composeAsync(func1, func2, func3);
const result3 = transformData(data);
```

Sequential composition can also be done more succinctly with async/await:

```
let result;
for (const f of [func1, func2, func3]) {
  result = await f(result);
}
/* use last result (i.e. result3) */
```

However, before you compose promises sequentially, consider if it's really necessary — it's always better to run promises concurrently so that they don't unnecessarily block each other unless one promise's execution depends on another's result.

Cancellation

`Promise` itself has no first-class protocol for cancellation, but you may be able to directly cancel the underlying asynchronous operation, typically using `AbortController`.

Creating a Promise around an old callback API

A `Promise` can be created from scratch using its constructor. This should be needed only to wrap old APIs.

In an ideal world, all asynchronous functions would already return promises. Unfortunately, some APIs still expect success and/or failure callbacks to be passed in the old way. The most obvious example is the `setTimeout()` function:

```
setTimeout(() => saySomething("10 seconds passed"), 10 * 1000);
```

Mixing old-style callbacks and promises is problematic. If `saySomething()` fails or contains a programming error, nothing catches it. This is intrinsic to the design of `setTimeout()`.

Luckily we can wrap `setTimeout()` in a promise. The best practice is to wrap the callback-accepting functions at the lowest possible level, and then never call them directly again:

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait(10 * 1000)
  .then(() => saySomething("10 seconds"))
  .catch(failureCallback);
```

The promise constructor takes an executor function that lets us resolve or reject a promise manually. Since `setTimeout()` doesn't really fail, we left out reject in this case. For more information on how the executor function works, see the `Promise()` reference.

Timing

Lastly, we will look into the more technical details, about when the registered callbacks get called.

Guarantees

In the callback-based API, when and how the callback gets called depends on the API implementor. For example, the callback may be called synchronously or asynchronously:

```
function doSomething(callback) {
  if (Math.random() > 0.5) {
    callback();
  } else {
    setTimeout(() => callback(), 1000);
  }
}
```

The above design is strongly discouraged because it leads to the so-called "state of Zalgo". In the context of designing asynchronous APIs, this means a callback is called synchronously in some cases but asynchronously in other cases, creating ambiguity for the caller. For further background, see the article Designing APIs for Asynchrony, where the term was first formally presented. This API design makes side effects hard to analyze:

```
let value = 1;
doSomething(() => {
  value = 2;
});
console.log(value); // 1 or 2?
```

On the other hand, promises are a form of inversion of control — the API implementor does not control when the callback gets called. Instead, the job of maintaining the callback queue and deciding when to call the callbacks is delegated to the promise implementation, and both the API user and API developer automatically get strong semantic guarantees, including:

- Callbacks added with `then()` will never be invoked before the completion of the current run of the JavaScript event loop.
- These callbacks will be invoked even if they were added *after* the success or failure of the asynchronous operation that the promise represents.
- Multiple callbacks may be added by calling `then()` several times. They will be invoked one after another, in the order in which they were inserted.

To avoid surprises, functions passed to `then()` will never be called synchronously, even with an already-resolved promise:

```
Promise.resolve().then(() => console.log(2));
console.log(1);
// Logs: 1, 2
```

Instead of running immediately, the passed-in function is put on a microtask queue, which means it runs later (only after the function which created it exits, and when the JavaScript execution stack is empty), just before control is returned to the event loop; i.e., pretty soon:

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait(0).then(() => console.log(4));
Promise.resolve()
  .then(() => console.log(2))
  .then(() => console.log(3));
console.log(1); // 1, 2, 3, 4
```

Task queues vs. microtasks

Promise callbacks are handled as a microtask whereas `setTimeout()` callbacks are handled as task queues.

```
const promise = new Promise((resolve, reject) => {
  console.log("Promise callback");
  resolve();
}).then((result) => {
  console.log("Promise callback (.then)");
});

setTimeout(() => {
  console.log("event-loop cycle: Promise (fulfilled)", promise);
}, 0);

console.log("Promise (pending)", promise);
```

The code above will output:

```
Promise callback
Promise (pending) Promise {<pending>}
Promise callback (.then)
event-loop cycle: Promise (fulfilled) Promise {<fulfilled>}
```

For more details, refer to Tasks vs. microtasks.

When promises and tasks collide

If you run into situations in which you have promises and tasks (such as events or callbacks) which are firing in unpredictable orders, it's possible you may benefit from using a microtask to check status or balance out your promises when promises are created conditionally.

If you think microtasks may help solve this problem, see the microtask guide to learn more about how to use `queueMicrotask()` to enqueue a function as a microtask.

See also

- `Promise`
- `async function`
- `await`

- Promises/A+ specification
- We have a problem with promises on pouchdb.com (2015)

- Previous
- Next

Working with objects

- Previous
- Next

JavaScript is designed on an object-based paradigm. An object is a collection of properties, and a property is an association between a name (or *key*) and a value. A property's value can be a function, in which case the property is known as a method.

Objects in JavaScript, just as in many other programming languages, can be compared to objects in real life. In JavaScript, an object is a standalone entity, with properties and type. Compare it with a cup, for example. A cup is an object with properties. A cup has a color, a design, weight, a material it is made of, etc. In the same way, JavaScript objects can have properties, which define their characteristics.

In addition to objects that are predefined in the browser, you can define your own objects. This chapter describes how to use objects, properties, and methods, and how to create your own objects.

Creating new objects

You can create an object using an object initializer. Alternatively, you can first create a constructor function and then instantiate an object by invoking that function with the `new` operator.

Using object initializers

Object initializers are also called *object literals*. "Object initializer" is consistent with the terminology used by C++.

The syntax for an object using an object initializer is:

```
const obj = {
  property1: value1, // property name may be an identifier
  2: value2, // or a number
  "property n": value3, // or a string
};
```

Each property name before the colon is either an identifier, a number literal, or a string literal, and each `valueN` is an expression whose value is assigned to the property name. The property name can also be an expression; computed keys need to be wrapped in square brackets. The object initializer reference contains a more detailed explanation of the syntax.

In this example, the newly created object is assigned to a variable `obj` — this is optional. If you do not need to refer to this object elsewhere, you do not need to assign it to a variable. (Note that you may need to wrap the object literal in parentheses if the object appears where a statement is expected, so as not to have the literal be confused with a block statement.)

Object initializers are expressions, and each object initializer results in a new object being created whenever the statement in which it appears is executed. Identical object initializers create distinct objects that do not compare to each other as equal.

The following statement creates an object and assigns it to the variable `x` if and only if the expression `cond` is true:

```
let x;
if (cond) {
  x = { greeting: "hi there" };
}
```

The following example creates `myHonda` with three properties. Note that the `engine` property is also an object with its own properties.

```
const myHonda = {
  color: "red",
  wheels: 4,
  engine: { cylinders: 4, size: 2.2 },
};
```

Objects created with initializers are called *plain objects*, because they are instances of `Object`, but not any other object type. Some object types have special initializer syntaxes — for example, array initializers and regex literals.

Using a constructor function

Alternatively, you can create an object with these two steps:

1. Define the object type by writing a constructor function. There is a strong convention, with good reason, to use a capital initial letter.
2. Create an instance of the object with `new`.

To define an object type, create a function for the object type that specifies its name, properties, and methods. For example, suppose you want to create an object type for cars. You want this type of object to be called `Car`, and you want it to have properties for make, model, and year. To do this, you would write the following function:

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
```

```
  this.year = year;
}
```

Notice the use of `this` to assign values to the object's properties based on the values passed to the function.

Now you can create an object called `myCar` as follows:

```
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

This statement creates `myCar` and assigns it the specified values for its properties. Then the value of `myCar.make` is the string `"Eagle"`, `myCar.model` is the string `"Talon TSi"`, `myCar.year` is the integer `1993`, and so on. The order of arguments and parameters should be the same.

You can create any number of `Car` objects by calls to `new`. For example,

```
const randCar = new Car("Nissan", "300ZX", 1992);
const kenCar = new Car("Mazda", "Miata", 1990);
```

An object can have a property that is itself another object. For example, suppose you define an object called `Person` as follows:

```
function Person(name, age, sex) {
  this.name = name;
  this.age = age;
  this.sex = sex;
}
```

and then instantiate two new `Person` objects as follows:

```
const rand = new Person("Rand McKinnon", 33, "M");
const ken = new Person("Ken Jones", 39, "M");
```

Then, you can rewrite the definition of `Car` to include an `owner` property that takes a `Person` object, as follows:

```
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
  this.year = year;
  this.owner = owner;
}
```

To instantiate the new objects, you then use the following:

```
const car1 = new Car("Eagle", "Talon TSi", 1993, rand);
const car2 = new Car("Nissan", "300ZX", 1992, ken);
```

Notice that instead of passing a literal string or integer value when creating the new objects, the above statements pass the objects `rand` and `ken` as the arguments for the owners. Then if you want to find out the name of the owner of `car2`, you can access the following property:

```
car2.owner.name;
```

You can always add a property to a previously defined object. For example, the statement

```
car1.color = "black";
```

adds a property `color` to `car1`, and assigns it a value of `"black"`. However, this does not affect any other objects. To add the new property to all objects of the same type, you have to add the property to the definition of the `Car` object type.

You can also use the `class` syntax instead of the `function` syntax to define a constructor function. For more information, see the class guide.

Using the Object.create() method

Objects can also be created using the `Object.create()` method. This method can be very useful, because it allows you to choose the prototype object for the object you want to create, without having to define a constructor function.

```
// Animal properties and method encapsulation
const animalProto = {
  type: "Invertebrates", // Default value of properties
  displayType() {
    // Method which will display the type of animal
    console.log(this.type);
  },
};

// Create a new animal type called `animal`
const animal = Object.create(animalProto);
animal.displayType(); // Logs: Invertebrates

// Create a new animal type called fish
const fish = Object.create(animalProto);
fish.type = "Fishes";
fish.displayType(); // Logs: Fishes
```

Objects and properties

A JavaScript object has properties associated with it. Object properties are basically the same as variables, except that they are associated with objects, not scopes. The properties of an object define the characteristics of the object.

For example, this example creates an object named `myCar`, with properties named `make`, `model`, and `year`, with their values set to `"Ford"`, `"Mustang"`, and `1969`:

```
const myCar = {
  make: "Ford",
  model: "Mustang",
  year: 1969,
};
```

Like JavaScript variables, property names are case sensitive. Property names can only be strings or Symbols — all keys are converted to strings unless they are Symbols. Array indices are, in fact, properties with string keys that contain integers.

Accessing properties

You can access a property of an object by its property name. Property accessors come in two syntaxes: *dot notation* and *bracket notation*. For example, you could access the properties of the `myCar` object as follows:

```
// Dot notation
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;

// Bracket notation
myCar["make"] = "Ford";
myCar["model"] = "Mustang";
myCar["year"] = 1969;
```

An object property name can be any JavaScript string or symbol, including an empty string. However, you cannot use dot notation to access a property whose name is not a valid JavaScript identifier. For example, a property name that has a space or a hyphen, that starts with a number, or that is held inside a variable can only be accessed using the bracket notation. This notation is also very useful when property names are to be dynamically determined, i.e., not determinable until runtime. Examples are as follows:

```
const myObj = {};
const str = "myString";
const rand = Math.random();
const anotherObj = {};

// Create additional properties on myObj
myObj.type = "Dot syntax for a key named type";
myObj["date created"] = "This key has a space";
myObj[str] = "This key is in variable str";
myObj[rand] = "A random number is the key here";
myObj[anotherObj] = "This key is object anotherObj";
myObj[""] = "This key is an empty string";

console.log(myObj);
// {
//   type: 'Dot syntax for a key named type',
//   'date created': 'This key has a space',
//   myString: 'This key is in variable str',
//   '0.6398914448618778': 'A random number is the key here',
//   '[object Object]': 'This key is object anotherObj',
//   '': 'This key is an empty string'
// }
console.log(myObj.myString); // 'This key is in variable str'
```

In the above code, the key `anotherObj` is an object, which is neither a string nor a symbol. When it is added to the `myObj`, JavaScript calls the `toString()` method of `anotherObj`, and use the resulting string as the new key.

You can also access properties with a string value stored in a variable. The variable must be passed in bracket notation. In the example above, the variable `str` held `"myString"` and it is `"myString"` that is the property name. Therefore, `myObj.str` will return as undefined.

```
str = "myString";
myObj[str] = "This key is in variable str";

console.log(myObj.str); // undefined

console.log(myObj[str]); // 'This key is in variable str'
console.log(myObj.myString); // 'This key is in variable str'
```

This allows accessing any property as determined at runtime:

```
let propertyName = "make";
myCar[propertyName] = "Ford";

// access different properties by changing the contents of the variable
propertyName = "model";
myCar[propertyName] = "Mustang";
```

```
console.log(myCar); // { make: 'Ford', model: 'Mustang' }
```

However, beware of using square brackets to access properties whose names are given by external input. This may make your code susceptible to object injection attacks.

Nonexistent properties of an object have the value `undefined` (and not `null`).

```
myCar.nonexistentProperty; // undefined
```

Enumerating properties

There are three native ways to list/traverse object properties:

- `for...in` loops. This method traverses all of the enumerable string properties of an object as well as its prototype chain.
- `Object.keys()`. This method returns an array with only the enumerable own string property names ("keys") in the object `myObj`, but not those in the prototype chain.
- `Object.getOwnPropertyNames()`. This method returns an array containing all the own string property names in the object `myObj`, regardless of if they are enumerable or not.

You can use the bracket notation with `for...in` to iterate over all the enumerable properties of an object. To illustrate how this works, the following function displays the properties of the object when you pass the object and the object's name as arguments to the function:

```
function showProps(obj, objName) {
  let result = "";
  for (const i in obj) {
    // Object.hasOwn() is used to exclude properties from the object's
    // prototype chain and only show "own properties"
    if (Object.hasOwn(obj, i)) {
      result += `${objName}.${i} = ${obj[i]}\n`;
    }
  }
  console.log(result);
}
```

The term "own property" refers to the properties of the object, but excluding those of the prototype chain. So, the function call `showProps(myCar, 'myCar')` would print the following:

```
myCar.make = Ford
myCar.model = Mustang
myCar.year = 1969
```

The above is equivalent to:

```
function showProps(obj, objName) {
  let result = "";
  Object.keys(obj).forEach((i) => {
    result += `${objName}.${i} = ${obj[i]}\n`;
  });
  console.log(result);
}
```

There is no native way to list all inherited properties, including non-enumerable ones. However, this can be achieved with the following function:

```
function listAllProperties(myObj) {
  let objectToInspect = myObj;
  let result = [];

  while (objectToInspect !== null) {
    result = result.concat(Object.getOwnPropertyNames(objectToInspect));
    objectToInspect = Object.getPrototypeOf(objectToInspect);
  }

  return result;
}
```

For more information, see Enumerability and ownership of properties.

Deleting properties

You can remove a non-inherited property using the `delete` operator. The following code shows how to remove a property.

```
// Creates a new object, myObj, with two properties, a and b.
const myObj = { a: 5, b: 12 };

// Removes the a property, leaving myObj with only the b property.
delete myObj.a;
console.log("a" in myObj); // false
```

Inheritance

All objects in JavaScript inherit from at least one other object. The object being inherited from is known as the prototype, and the inherited properties can be found in the `prototype` object of the constructor. See Inheritance and the prototype chain for more information.

Defining properties for all objects of one type

You can add a property to all objects created through a certain constructor using the `prototype` property. This defines a property that is shared by all objects of the specified type, rather than by just one instance of the object. The following code adds a `color` property to all objects of type `Car`, and then reads the property's value from an instance `car1`.

```
Car.prototype.color = "red";
console.log(car1.color); // "red"
```

Defining methods

A *method* is a function associated with an object, or, put differently, a method is a property of an object that is a function. Methods are defined the way normal functions are defined, except that they have to be assigned as the property of an object. See also method definitions for more details. An example is:

```
objectName.methodName = functionName;

const myObj = {
  myMethod: function (params) {
    // do something
  },

  // this works too!
  myOtherMethod(params) {
    // do something else
  },
};
```

where `objectName` is an existing object, `methodName` is the name you are assigning to the method, and `functionName` is the name of the function.

You can then call the method in the context of the object as follows:

```
objectName.methodName(params);
```

Methods are typically defined on the `prototype` object of the constructor, so that all objects of the same type share the same method. For example, you can define a function that formats and displays the properties of the previously-defined `Car` objects.

```
Car.prototype.displayCar = function () {
  const result = `A Beautiful ${this.year} ${this.make} ${this.model}`;
  console.log(result);
};
```

Notice the use of `this` to refer to the object to which the method belongs. Then you can call the `displayCar` method for each of the objects as follows:

```
car1.displayCar();
car2.displayCar();
```

Using this for object references

JavaScript has a special keyword, `this`, that you can use within a method to refer to the current object. For example, suppose you have 2 objects, `manager` and `intern`. Each object has its own `name`, `age` and `job`. In the function `sayHi()`, notice the use of `this.name`. When added to the 2 objects, the same function will print the message with the name of the respective object it's attached to.

```
const manager = {
  name: "Karina",
  age: 27,
  job: "Software Engineer",
};
const intern = {
  name: "Tyrone",
  age: 21,
  job: "Software Engineer Intern",
};

function sayHi() {
  console.log(`Hello, my name is ${this.name}`);
}

// Add sayHi function to both objects
manager.sayHi = sayHi;
intern.sayHi = sayHi;

manager.sayHi(); // Hello, my name is Karina
intern.sayHi(); // Hello, my name is Tyrone
```

`this` is a "hidden parameter" of a function call that's passed in by specifying the object before the function that was called. For example, in `manager.sayHi()`, `this` is the `manager` object, because `manager` comes before the function `sayHi()`. If you access the same function from another object, `this` will change as well. If you use other methods to call the function, like `Function.prototype.call()` or `Reflect.apply()`, you can explicitly pass the value of `this` as an argument.

Defining getters and setters

A getter is a function associated with a property that gets the value of a specific property. A setter is a function associated with a property that sets the value of a specific property. Together, they can indirectly represent the value of a property.

Getters and setters can be either

- defined within object initializers, or
- added later to any existing object.

Within object initializers, getters and setters are defined like regular methods, but prefixed with the keywords `get` or `set`. The getter method must not expect a parameter, while the setter method expects exactly one parameter (the new value to set). For instance:

```
const myObj = {
  a: 7,
  get b() {
    return this.a + 1;
  },
  set c(x) {
    this.a = x / 2;
  },
};

console.log(myObj.a); // 7
console.log(myObj.b); // 8, returned from the get b() method
myObj.c = 50; // Calls the set c(x) method
console.log(myObj.a); // 25
```

The `myObj` object's properties are:

- `myObj.a` — a number
- `myObj.b` — a getter that returns `myObj.a` plus 1
- `myObj.c` — a setter that sets the value of `myObj.a` to half of the value `myObj.c` is being set to

Getters and setters can also be added to an object at any time after creation using the `Object.defineProperties()` method. This method's first parameter is the object on which you want to define the getter or setter. The second parameter is an object whose property names are the getter or setter names, and whose property values are objects for defining the getter or setter functions. Here's an example that defines the same getter and setter used in the previous example:

```
const myObj = { a: 0 };

Object.defineProperties(myObj, {
  b: {
    get() {
      return this.a + 1;
    },
  },
  c: {
    set(x) {
      this.a = x / 2;
    },
  },
});

myObj.c = 10; // Runs the setter, which assigns 10 / 2 (5) to the 'a' property
console.log(myObj.b); // Runs the getter, which yields a + 1 or 6
```

Which of the two forms to choose depends on your programming style and task at hand. If you can change the definition of the original object, you will probably define getters and setters through the original initializer. This form is more compact and natural. However, if you need to add getters and setters later — maybe because you did not write the particular object — then the second form is the only possible form. The second form better represents the dynamic nature of JavaScript, but it can make the code hard to read and understand.

Comparing objects

In JavaScript, objects are a reference type. Two distinct objects are never equal, even if they have the same properties. Only comparing the same object reference with itself yields true.

```
// Two variables, two distinct objects with the same properties
const fruit = { name: "apple" };
const anotherFruit = { name: "apple" };

fruit == anotherFruit; // return false
fruit === anotherFruit; // return false
```

```
// Two variables, a single object
const fruit = { name: "apple" };
const anotherFruit = fruit; // Assign fruit object reference to anotherFruit

// Here fruit and anotherFruit are pointing to same object
fruit == anotherFruit; // return true
fruit === anotherFruit; // return true
```

```
fruit.name = "grape";
console.log(anotherFruit); // { name: "grape" }; not { name: "apple" }
```

For more information about comparison operators, see equality operators.

See also

- Inheritance and the prototype chain
- Classes

- Previous
- Next

JavaScript reference

The JavaScript reference serves as a repository of facts about the JavaScript language. The entire language is described here in detail. As you write JavaScript code, you'll refer to these pages often (thus the title "JavaScript reference").

The JavaScript language is intended to be used within some larger environment, be it a browser, server-side scripts, or similar. For the most part, this reference attempts to be environment-agnostic and does not target a web browser environment.

If you are new to JavaScript, start with the guide. Once you have a firm grasp of the fundamentals, you can use the reference to get more details on individual objects and language constructs.

Built-ins

JavaScript standard built-in objects, along with their methods and properties.

Value properties

- `globalThis`
- `Infinity`
- `NaN`
- `undefined`

Function properties

- `eval()`
- `isFinite()`
- `isNaN()`
- `parseFloat()`
- `parseInt()`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`
- `escape()`
- `unescape()`

Fundamental objects

- `Object`
- `Function`
- `Boolean`
- `Symbol`

Error objects

- `Error`
- `AggregateError`
- `EvalError`
- `RangeError`
- `ReferenceError`
- `SuppressedError`
- `SyntaxError`
- `TypeError`
- `URIError`
- `InternalError`

Numbers and dates

- `Number`
- `BigInt`
- `Math`
- `Date`
- `Temporal`

Text processing

- `String`
- `RegExp`

Indexed collections

- `Array`
- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`
- `BigInt64Array`
- `BigUint64Array`
- `Float16Array`
- `Float32Array`
- `Float64Array`

Keyed collections

- `Map`
- `Set`
- `WeakMap`
- `WeakSet`

Structured data

- `ArrayBuffer`
- `SharedArrayBuffer`
- `DataView`
- `Atomics`
- `JSON`

Managing memory

- `WeakRef`
- `FinalizationRegistry`

Control abstraction objects

- `Iterator`
- `AsyncIterator`
- `Promise`
- `GeneratorFunction`
- `AsyncGeneratorFunction`
- `Generator`
- `AsyncGenerator`
- `AsyncFunction`
- `DisposableStack`
- `AsyncDisposableStack`

Reflection

- `AbstractModuleSource`
- `Reflect`
- `Proxy`

Internationalization

- `Intl`
- `Intl.Collator`
- `Intl.DateTimeFormat`
- `Intl.DisplayNames`
- `Intl.DurationFormat`
- `Intl.ListFormat`
- `Intl.Locale`
- `Intl.NumberFormat`
- `Intl.PluralRules`
- `Intl.RelativeTimeFormat`
- `Intl.Segmenter`

Statements

JavaScript statements and declarations

Control flow

- `return`

- `break`
- `continue`
- `throw`
- `if...else`
- `switch`
- `try...catch`

Declaring variables

- `var`
- `let`
- `const`
- `using`
- `await using`

Functions and classes

- `function`
- `function*`
- `async function`
- `async function*`
- `class`

Iterations

- `do...while`
- `for`
- `for...in`
- `for...of`
- `for await...of`
- `while`

Others

- Empty
- Block
- Expression statement
- `debugger`
- `export`
- `import`
- `import defer`
- `import source`
- label
- `with`

Expressions and operators

JavaScript expressions and operators.

Primary expressions

- `this`
- Literals
- `[]`
- `{}`
- `function`
- `class`
- `function*`
- `async function`
- `async function*`
- `/ab+c/i`
- `` `string` ``
- `( )`

Left-hand-side expressions

- Property accessors
- `?.`
- `new`
- `new.target`
- `import.meta`
- `super`
- `import()`
- `import.defer()`
- `import.source()`

Increment and decrement

- `A++`
- `A--`
- `++A`
- `--A`

Unary operators

- `delete`
- `void`
- `typeof`
- `+`
- `-`
- `~`
- `!`
- `await`

Arithmetic operators

- `**`
- `*`
- `/`
- `%`
- `+` (Plus)
- `-`

Relational operators

- `<` (Less than)
- `>` (Greater than)
- `<=`
- `>=`
- `instanceof`
- `in`

Equality operators

- `==`
- `!=`
- `===`
- `!==`

Bitwise shift operators

- `<<`
- `>>`
- `>>>`

Binary bitwise operators

- `&`
- `|`
- `^`

Binary logical operators

- `&&`
- `||`
- `??`

Conditional (ternary) operator

- `(condition ? ifTrue : ifFalse)`

Assignment operators

- `=`
- `*=`
- `/=`
- `%=`
- `+=`
- `-=`
- `<<=`
- `>>=`
- `>>>=`
- `&=`
- `^=`
- `|=`
- `**=`
- `&&=`

- `||=`
- `??=`
- `[a, b] = arr`, `{ a, b } = obj`

Yield operators

- `yield`
- `yield*`

Spread syntax

- `...obj`

Comma operator

- `,`

Functions

JavaScript functions.

- Arrow Functions
- Default parameters
- Rest parameters
- `arguments`
- Method definitions
- getter
- setter

Classes

JavaScript classes.

- `constructor`
- `extends`
- Private elements
- Public class fields
- `static`
- Static initialization blocks

Regular expressions

JavaScript regular expressions.

- Backreference: `\1`, `\2`
- Capturing group: `(...)`
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`
- Input boundary assertion: `^`, `$`
- Literal character: `a`, `b`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Modifier: `(?ims-ims:...)`
- Named backreference: `\k<name>`
- Named capturing group: `(?<name>...)`
- Non-capturing group: `(?:...)`
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Wildcard: `.`
- Word boundary assertion: `\b`, `\B`

Additional reference pages

- JavaScript technologies overview
- Execution model
- Lexical grammar
- Data types and data structures
- Iteration protocols
- Trailing commas
- Errors
- Strict mode
- Deprecated features

Classes

Classes are a template for creating objects. They encapsulate data with code to work on that data. Classes in JS are built on prototypes but also have some syntax and semantics that are unique to classes.

For more examples and explanations, see the Using classes guide.

Description

Defining classes

Classes are in fact "special functions", and just as you can define function expressions and function declarations, a class can be defined in two ways: a class expression or a class declaration.

```
// Declaration
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}

// Expression; the class is anonymous but assigned to a variable
const Rectangle = class {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
};

// Expression; the class has its own name
const Rectangle = class Rectangle2 {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
};
```

Like function expressions, class expressions may be anonymous, or have a name that's different from the variable that it's assigned to. However, unlike function declarations, class declarations have the same temporal dead zone restrictions as `let` or `const` and behave as if they are not hoisted.

Class body

The body of a class is the part that is in curly braces `{}`. This is where you define class members, such as methods or constructor.

The body of a class is executed in strict mode even without the `"use strict"` directive.

A class element can be characterized by three aspects:

- Kind: Getter, setter, method, or field
- Location: Static or instance
- Visibility: Public or private

Together, they add up to 16 possible combinations. To divide the reference more logically and avoid overlapping content, the different elements are introduced in detail in different pages:

Method definitions

: Public instance method

getter

: Public instance getter

setter

: Public instance setter

Public class fields

: Public instance field

`static`

: Public static method, getter, setter, and field

Private elements

: Everything that's private

> **Note:** Private elements have the restriction that all private names declared in the same class must be unique. All other public properties do not have this restriction — you can have multiple public properties with the same name, and the last one overwrites the others. This is the same behavior as in object initializers.

In addition, there are two special class element syntaxes: `constructor` and static initialization blocks, with their own references.

Constructor

The `constructor` method is a special method for creating and initializing an object created with a class. There can only be one special method with the name "constructor" in a class — a `SyntaxError` is thrown if the class contains more than one occurrence of a `constructor` method.

A constructor can use the `super` keyword to call the constructor of the super class.

You can create instance properties inside the constructor:

```
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}
```

Alternatively, if your instance properties' values do not depend on the constructor's arguments, you can define them as class fields.

Static initialization blocks

Static initialization blocks allow flexible initialization of static properties, including the evaluation of statements during initialization, while granting access to the private scope.

Multiple static blocks can be declared, and these can be interleaved with the declaration of static fields and methods (all static items are evaluated in declaration order).

Methods

Methods are defined on the prototype of each class instance and are shared by all instances. Methods can be plain functions, async functions, generator functions, or async generator functions. For more information, see method definitions.

```
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
  // Getter
  get area() {
    return this.calcArea();
  }
  // Method
  calcArea() {
    return this.height * this.width;
  }
  *getSides() {
    yield this.height;
    yield this.width;
    yield this.height;
    yield this.width;
  }
}

const square = new Rectangle(10, 10);

console.log(square.area); // 100
console.log([...square.getSides()]); // [10, 10, 10, 10]
```

Static methods and fields

The `static` keyword defines a static method or field for a class. Static properties (fields and methods) are defined on the class itself instead of each instance. Static methods are often used to create utility functions for an application, whereas static fields are useful for caches, fixed-configuration, or any other data that doesn't need to be replicated across instances.

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  static displayName = "Point";
  static distance(a, b) {
    const dx = a.x - b.x;
    const dy = a.y - b.y;

    return Math.hypot(dx, dy);
  }
}

const p1 = new Point(5, 5);
const p2 = new Point(10, 10);
p1.displayName; // undefined
p1.distance; // undefined
p2.displayName; // undefined
p2.distance; // undefined
```

```
console.log(Point.displayName); // "Point"
console.log(Point.distance(p1, p2)); // 7.0710678118654755
```

Field declarations

With the class field declaration syntax, the constructor example can be written as:

```
class Rectangle {
  height = 0;
  width;
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}
```

Class fields are similar to object properties, not variables, so we don't use keywords such as `const` to declare them. In JavaScript, private elements use a special identifier syntax, so modifier keywords like `public` and `private` should not be used either.

As seen above, the fields can be declared with or without a default value. Fields without default values default to `undefined`. By declaring fields up-front, class definitions become more self-documenting, and the fields are always present, which help with optimizations.

See public class fields for more information.

Private elements

Using private fields, the definition can be refined as below.

```
class Rectangle {
  #height = 0;
  #width;
  constructor(height, width) {
    this.#height = height;
    this.#width = width;
  }
}
```

It's an error to reference private fields from outside of the class; they can only be read or written within the class body. By defining things that are not visible outside of the class, you ensure that your classes' users can't depend on internals, which may change from version to version.

Private fields can only be declared up-front in a field declaration. They cannot be created later through assigning to them, the way that normal properties can.

Private methods and accessors can also be defined using the same syntax as their public counterparts, but with the identifier starting with `#`.

For more information, see private elements.

Inheritance

The `extends` keyword is used in *class declarations* or *class expressions* to create a class as a child of another constructor (either a class or a function).

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

class Dog extends Animal {
  constructor(name) {
    super(name); // call the super class constructor and pass in the name parameter
  }

  speak() {
    console.log(`${this.name} barks.`);
  }
}

const d = new Dog("Mitzie");
d.speak(); // Mitzie barks.
```

If there is a constructor present in the subclass, it needs to first call `super()` before using `this`. The `super` keyword can also be used to call corresponding methods of super class.

```
class Cat {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
```

```
  }
}

class Lion extends Cat {
  speak() {
    super.speak();
    console.log(`${this.name} roars.`);
  }
}

const l = new Lion("Fuzzy");
l.speak();
// Fuzzy makes a noise.
// Fuzzy roars.
```

Evaluation order

When a `class` declaration or `class` expression is evaluated, its various components are evaluated in the following order:

1. The `extends` clause, if present, is first evaluated. It must evaluate to a valid constructor function or `null`, or a `TypeError` is thrown.
2. The `constructor` method is extracted, substituted with a default implementation if `constructor` is not present. However, because the `constructor` definition is only a method definition, this step is not observable.
3. The class elements' property keys are evaluated in the order of declaration. If the property key is computed, the computed expression is evaluated, with the `this` value set to the `this` value surrounding the class (not the class itself). None of the property values are evaluated yet.
4. Methods and accessors are installed in the order of declaration. Instance methods and accessors are installed on the `prototype` property of the current class, and static methods and accessors are installed on the class itself. Private instance methods and accessors are saved to be installed on the instance directly later. This step is not observable.
5. The class is now initialized with the prototype specified by `extends` and implementation specified by `constructor`. For all steps above, if an evaluated expression tries to access the name of the class, a `ReferenceError` is thrown because the class is not initialized yet.
6. The class elements' values are evaluated in the order of declaration:
   - For each instance field (public or private), its initializer expression is saved. The initializer is evaluated during instance creation, at the start of the constructor (for base classes) or immediately before the `super()` call returns (for derived classes).
   - For each static field (public or private), its initializer is evaluated with `this` set to the class itself, and the property is created on the class.
   - Static initialization blocks are evaluated with `this` set to the class itself.
7. The class is now fully initialized and can be used as a constructor function.

For how instances are created, see the `constructor` reference.

Examples

Binding this with instance and static methods

When a static or instance method is called without a value for `this`, such as by assigning the method to a variable and then calling it, the `this` value will be `undefined` inside the method. This behavior is the same even if the `"use strict"` directive isn't present, because code within the `class` body is always executed in strict mode.

```
class Animal {
  speak() {
    return this;
  }
  static eat() {
    return this;
  }
}

const obj = new Animal();
obj.speak(); // the Animal object
const speak = obj.speak;
speak(); // undefined

Animal.eat(); // class Animal
const eat = Animal.eat;
eat(); // undefined
```

If we rewrite the above using traditional function-based syntax in non–strict mode, then `this` method calls are automatically bound to `globalThis`. In strict mode, the value of `this` remains as `undefined`.

```
function Animal() {}

Animal.prototype.speak = function () {
  return this;
};

Animal.eat = function () {
  return this;
};

const obj = new Animal();
const speak = obj.speak;
speak(); // global object (in non-strict mode)

const eat = Animal.eat;
eat(); // global object (in non-strict mode)
```

See also

- Using classes guide
- `class`
- `class` expression
- Functions
- ES6 In Depth: Classes on hacks.mozilla.org (2015)

constructor

The **`constructor`** method is a special method of a class for creating and initializing an object instance of that class.

> **Note:** This page introduces the `constructor` syntax. For the `constructor` property present on all objects, see `Object.prototype.constructor`.

Try it

```
class Polygon {
  constructor() {
    this.name = "Polygon";
  }
}

const poly = new Polygon();

console.log(poly.name);
// Expected output: "Polygon"
```

Syntax

```
constructor() { /* … */ }
constructor(argument0) { /* … */ }
constructor(argument0, argument1) { /* … */ }
constructor(argument0, argument1, /* …, */ argumentN) { /* … */ }
```

There are some additional syntax restrictions:

- A class method called `constructor` cannot be a getter, setter, async, or generator.
- A class cannot have more than one `constructor` method.

Description

A constructor enables you to provide any custom initialization that must be done before any other methods can be called on an instantiated object.

```
class Person {
  constructor(name) {
    this.name = name;
  }

  introduce() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

const otto = new Person("Otto");

otto.introduce(); // Hello, my name is Otto
```

If you don't provide your own constructor, then a default constructor will be supplied for you. If your class is a base class, the default constructor is empty:

```
constructor() {}
```

If your class is a derived class, the default constructor calls the parent constructor, passing along any arguments that were provided:

```
constructor(...args) {
  super(...args);
}
```

> **Note:** The difference between an explicit constructor like the one above and the default constructor is that the latter doesn't actually invoke the array iterator through argument spreading.

That enables code like this to work:

```
class ValidationError extends Error {
  printCustomerMessage() {
    return `Validation failed :-( (details: ${this.message})`;
  }
}
```

```
try {
  throw new ValidationError("Not a valid phone number");
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(error.name); // This is Error instead of ValidationError!
    console.log(error.printCustomerMessage());
  } else {
    console.log("Unknown error", error);
    throw error;
  }
}
```

The `ValidationError` class doesn't need an explicit constructor, because it doesn't need to do any custom initialization. The default constructor then takes care of initializing the parent `Error` from the argument it is given.

However, if you provide your own constructor, and your class derives from some parent class, then you must explicitly call the parent class constructor using `super()`. For example:

```
class ValidationError extends Error {
  constructor(message) {
    super(message); // call parent class constructor
    this.name = "ValidationError";
    this.code = "42";
  }

  printCustomerMessage() {
    return `Validation failed :-( (details: ${this.message}, code: ${this.code})`;
  }
}

try {
  throw new ValidationError("Not a valid phone number");
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(error.name); // Now this is ValidationError!
    console.log(error.printCustomerMessage());
  } else {
    console.log("Unknown error", error);
    throw error;
  }
}
```

Using `new` on a class goes through the following steps:

1. (If it's a derived class) The `constructor` body before the `super()` call is evaluated. This part should not access `this` because it's not yet initialized.
2. (If it's a derived class) The `super()` call is evaluated, which initializes the parent class through the same process.
3. The current class's fields are initialized.
4. The `constructor` body after the `super()` call (or the entire body, if it's a base class) is evaluated.

Within the `constructor` body, you can access the object being created through `this` and access the class that is called with `new` through `new.target`. Note that methods (including getters and setters) and the prototype chain are already initialized on `this` before the `constructor` is executed, so you can even access methods of the subclass from the constructor of the superclass. However, if those methods use `this`, the `this` will not have been fully initialized yet. This means reading public fields of the derived class will result in `undefined`, while reading private fields will result in a `TypeError`.

```
new (class C extends class B {
  constructor() {
    console.log(this.foo());
  }
} {
  #a = 1;
  foo() {
    return this.#a; // TypeError: Cannot read private member #a from an object whose class did not declare it
    // It's not really because the class didn't declare it,
    // but because the private field isn't initialized yet
    // when the superclass constructor is running
  }
})();
```

The `constructor` method may have a return value. While the base class may return anything from its constructor, the derived class must return an object or `undefined`, or a `TypeError` will be thrown.

```
class ParentClass {
  constructor() {
    return 1;
  }
}

console.log(new ParentClass()); // ParentClass {}
// The return value is ignored because it's not an object
// This is consistent with function constructors

class ChildClass extends ParentClass {
  constructor() {
```

```
    return 1;
  }
}

console.log(new ChildClass()); // TypeError: Derived constructors may only return object or undefined
```

If the parent class constructor returns an object, that object will be used as the `this` value on which class fields of the derived class will be defined. This trick is called "return overriding", which allows a derived class's fields (including private ones) to be defined on unrelated objects.

The `constructor` follows normal method syntax, so parameter default values, rest parameters, etc. can all be used.

```
class Person {
  constructor(name = "Anonymous") {
    this.name = name;
  }
  introduce() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

const person = new Person();
person.introduce(); // Hello, my name is Anonymous
```

The constructor must be a literal name. Computed properties cannot become constructors.

```
class Foo {
  // This is a computed property. It will not be picked up as a constructor.
  ["constructor"]() {
    console.log("called");
    this.a = 1;
  }
}

const foo = new Foo(); // No log
console.log(foo); // Foo {}
foo.constructor(); // Logs "called"
console.log(foo); // Foo { a: 1 }
```

Async methods, generator methods, accessors, and class fields are forbidden from being called `constructor`. Private names cannot be called `#constructor`. Any member named `constructor` must be a plain method.

Examples

Using the constructor

This code snippet is taken from the classes sample (live demo).

```
class Square extends Polygon {
  constructor(length) {
    // Here, it calls the parent class' constructor with lengths
    // provided for the Polygon's width and height
    super(length, length);
    // NOTE: In derived classes, `super()` must be called before you
    // can use `this`. Leaving this out will cause a ReferenceError.
    this.name = "Square";
  }

  get area() {
    return this.height * this.width;
  }

  set area(value) {
    this.height = value ** 0.5;
    this.width = value ** 0.5;
  }
}
```

Calling super in a constructor bound to a different prototype

`super()` calls the constructor that's the prototype of the current class. If you change the prototype of the current class itself, `super()` will call the constructor that's the new prototype. Changing the prototype of the current class's `prototype` property doesn't affect which constructor `super()` calls.

```
class Polygon {
  constructor() {
    this.name = "Polygon";
  }
}

class Rectangle {
  constructor() {
    this.name = "Rectangle";
  }
}
```

```
class Square extends Polygon {
  constructor() {
    super();
  }
}

// Make Square extend Rectangle (which is a base class) instead of Polygon
Object.setPrototypeOf(Square, Rectangle);

const newInstance = new Square();

// newInstance is still an instance of Polygon, because we didn't
// change the prototype of Square.prototype, so the prototype chain
// of newInstance is still
//   newInstance --> Square.prototype --> Polygon.prototype
console.log(newInstance instanceof Polygon); // true
console.log(newInstance instanceof Rectangle); // false

// However, because super() calls Rectangle as constructor, the name property
// of newInstance is initialized with the logic in Rectangle
console.log(newInstance.name); // Rectangle
```

See also

- Using classes guide
- Classes
- Static initialization blocks
- `class`
- `super()`
- `Object.prototype.constructor`

extends

The **`extends`** keyword is used in class declarations or class expressions to create a class that is a child of another class.

Try it

```
class DateFormatter extends Date {
  getFormattedDate() {
    const months = [
      "Jan",
      "Feb",
      "Mar",
      "Apr",
      "May",
      "Jun",
      "Jul",
      "Aug",
      "Sep",
      "Oct",
      "Nov",
      "Dec",
    ];
    return `${this.getDate()}-${months[this.getMonth()]}-${this.getFullYear()}`;
  }
}

console.log(new DateFormatter("August 19, 1975 23:15:30").getFormattedDate());
// Expected output: "19-Aug-1975"
```

Syntax

```
class ChildClass extends ParentClass { /* … */ }
```

`ParentClass`

An expression that evaluates to a constructor function (including a class) or `null`.

Description

The `extends` keyword can be used to subclass custom classes as well as built-in objects.

Any constructor that can be called with `new` and has the `prototype` property can be the candidate for the parent class. The two conditions must both hold — for example, bound functions and `Proxy` can be constructed, but they don't have a `prototype` property, so they cannot be subclassed.

```
function OldStyleClass() {
  this.someProperty = 1;
}
OldStyleClass.prototype.someMethod = function () {};

class ChildClass extends OldStyleClass {}

class ModernClass {
  someProperty = 1;
```

```
  someMethod() {}
}

class AnotherChildClass extends ModernClass {}
```

The `prototype` property of the `ParentClass` must be an `Object` or `null`, but you would rarely worry about this in practice, because a non-object `prototype` doesn't behave as it should anyway. (It's ignored by the `new` operator.)

```
function ParentClass() {}
ParentClass.prototype = 3;

class ChildClass extends ParentClass {}
// Uncaught TypeError: Class extends value does not have valid prototype property 3

console.log(Object.getPrototypeOf(new ParentClass()));
// [Object: null prototype] {}
// Not actually a number!
```

`extends` sets the prototype for both `ChildClass` and `ChildClass.prototype`.

	Prototype of `ChildClass`	**Prototype of `ChildClass.prototype`**
`extends` clause absent	`Function.prototype`	`Object.prototype`
`extends null`	`Function.prototype`	`null`
`extends ParentClass`	`ParentClass`	`ParentClass.prototype`

```
class ParentClass {}
class ChildClass extends ParentClass {}

// Allows inheritance of static properties
Object.getPrototypeOf(ChildClass) === ParentClass;
// Allows inheritance of instance properties
Object.getPrototypeOf(ChildClass.prototype) === ParentClass.prototype;
```

The right-hand side of `extends` does not have to be an identifier. You can use any expression that evaluates to a constructor. This is often useful to create mixins. The `this` value in the `extends` expression is the `this` surrounding the class definition, and referring to the class's name is a `ReferenceError` because the class is not initialized yet. `await` and `yield` work as expected in this expression.

```
class SomeClass extends class {
  constructor() {
    console.log("Base class");
  }
} {
  constructor() {
    super();
    console.log("Derived class");
  }
}

new SomeClass();
// Base class
// Derived class
```

While the base class may return anything from its constructor, the derived class must return an object or `undefined`, or a `TypeError` will be thrown.

```
class ParentClass {
  constructor() {
    return 1;
  }
}

console.log(new ParentClass()); // ParentClass {}
// The return value is ignored because it's not an object
// This is consistent with function constructors

class ChildClass extends ParentClass {
  constructor() {
    super();
    return 1;
  }
}

console.log(new ChildClass()); // TypeError: Derived constructors may only return object or undefined
```

If the parent class constructor returns an object, that object will be used as the `this` value for the derived class when further initializing class fields. This trick is called "return overriding", which allows a derived class's fields (including private ones) to be defined on unrelated objects.

Subclassing built-ins

> **Warning:** The standard committee now holds the position that the built-in subclassing mechanism in previous spec versions is over-engineered and causes non-negligible performance and security impacts. New built-in methods consider less about subclasses, and engine implementers are investigating whether to remove

> certain subclassing mechanisms. Consider using composition instead of inheritance when enhancing built-ins.

Here are some things you may expect when extending a class:

- When calling a static factory method (like `Promise.resolve()` or `Array.from()`) on a subclass, the returned instance is always an instance of the subclass.
- When calling an instance method that returns a new instance (like `Promise.prototype.then()` or `Array.prototype.map()`) on a subclass, the returned instance is always an instance of the subclass.
- Instance methods try to delegate to a minimal set of primitive methods where possible. For example, for a subclass of `Promise`, overriding `then()` automatically causes the behavior of `catch()` to change; or for a subclass of `Map`, overriding `set()` automatically causes the behavior of the `Map()` constructor to change.

However, the above expectations take non-trivial efforts to implement properly.

- The first one requires the static method to read the value of `this` to get the constructor for constructing the returned instance. This means `[p1, p2, p3].map(Promise.resolve)` throws an error because the `this` inside `Promise.resolve` is `undefined`. A way to fix this is to fall back to the base class if `this` is not a constructor, like `Array.from()` does, but that still means the base class is special-cased.
- The second one requires the instance method to read `this.constructor` to get the constructor function. However, `new this.constructor()` may break legacy code, because the `constructor` property is both writable and configurable and is not protected in any way. Therefore, many copying built-in methods use the constructor's `[Symbol.species]` property instead (which by default just returns `this`, the constructor itself). However, `[Symbol.species]` allows running arbitrary code and creating instances of arbitrary type, which poses a security concern and greatly complicates subclassing semantics.
- The third one leads to visible invocations of custom code, which makes a lot of optimizations harder to implement. For example, if the `Map()` constructor is called with an iterable of *x* elements, then it must visibly invoke the `set()` method *x* times, instead of just copying the elements into the internal storage.

These problems are not unique to built-in classes. For your own classes, you will likely have to make the same decisions. However, for built-in classes, optimizability and security are a much bigger concern. New built-in methods always construct the base class and call as few custom methods as possible. If you want to subclass built-ins while achieving the above expectations, you need to override all methods that have the default behavior baked into them. Any addition of new methods on the base class may also break the semantics of your subclass because they are inherited by default. Therefore, a better way to extend built-ins is to use *composition*.

Extending null

`extends null` was designed to allow easy creation of objects that do not inherit from `Object.prototype`. However, due to unsettled decisions about whether `super()` should be called within the constructor, it's not possible to construct such a class in practice using any constructor implementation that doesn't return an object. The TC39 committee is working on re-enabling this feature.

```
new (class extends null {})();
// TypeError: Super constructor null of anonymous class is not a constructor

new (class extends null {
  constructor() {}
})();
// ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from derived constructor

new (class extends null {
  constructor() {
    super();
  }
})();
// TypeError: Super constructor null of anonymous class is not a constructor
```

Instead, you need to explicitly return an instance from the constructor.

```
class NullClass extends null {
  constructor() {
    // Using new.target allows derived classes to
    // have the correct prototype chain
    return Object.create(new.target.prototype);
  }
}

const proto = Object.getPrototypeOf;
console.log(proto(proto(new NullClass()))); // null
```

Examples

Using extends

The first example creates a class called `Square` from a class called `Polygon`. This example is extracted from this live demo (source).

```
class Square extends Polygon {
  constructor(length) {
    // Here, it calls the parent class' constructor with lengths
    // provided for the Polygon's width and height
    super(length, length);
    // Note: In derived classes, super() must be called before you
    // can use 'this'. Leaving this out will cause a reference error.
    this.name = "Square";
  }

  get area() {
    return this.height * this.width;
  }
}
```

Extending plain objects

Classes cannot extend regular (non-constructible) objects. If you want to inherit from a regular object by making all properties of this object available on inherited instances, you can instead use `Object.setPrototypeOf()`:

```
const Animal = {
  speak() {
    console.log(`${this.name} makes a noise.`);
  },
};

class Dog {
  constructor(name) {
    this.name = name;
  }
}

Object.setPrototypeOf(Dog.prototype, Animal);

const d = new Dog("Mitzie");
d.speak(); // Mitzie makes a noise.
```

Extending built-in objects

This example extends the built-in `Date` object. This example is extracted from this live demo (source).

```
class MyDate extends Date {
  getFormattedDate() {
    const months = [
      "Jan", "Feb", "Mar", "Apr", "May", "Jun",
      "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
    ];
    return `${this.getDate()}-${months[this.getMonth()]}-${this.getFullYear()}`;
  }
}
```

Extending <code>Object</code>

All JavaScript objects inherit from `Object.prototype` by default, so writing `extends Object` at first glance seems redundant. The only difference from not writing `extends` at all is that the constructor itself inherits static methods from `Object`, such as `Object.keys()`. However, because no `Object` static method uses the `this` value, there's still no value in inheriting these static methods.

The `Object()` constructor special-cases the subclassing scenario. If it's implicitly called via `super()`, it always initializes a new object with `new.target.prototype` as its prototype. Any value passed to `super()` is ignored.

```
class C extends Object {
  constructor(v) {
    super(v);
  }
}

console.log(new C(1) instanceof Number); // false
console.log(C.keys({ a: 1, b: 2 })); // [ 'a', 'b' ]
```

Compare this behavior with a custom wrapper that does not special-case subclassing:

```
function MyObject(v) {
  return new Object(v);
}
class D extends MyObject {
  constructor(v) {
    super(v);
  }
}
console.log(new D(1) instanceof Number); // true
```

Species

You might want to return `Array` objects in your derived array class `MyArray`. The species pattern lets you override default constructors.

For example, when using methods such as `Array.prototype.map()` that return the default constructor, you want these methods to return a parent `Array` object, instead of the `MyArray` object. The `Symbol.species` symbol lets you do this:

```
class MyArray extends Array {
  // Overwrite species to the parent Array constructor
  static get [Symbol.species]() {
    return Array;
  }
}

const a = new MyArray(1, 2, 3);
const mapped = a.map((x) => x * x);
```

```
console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array); // true
```

This behavior is implemented by many built-in copying methods. For caveats of this feature, see the subclassing built-ins discussion.

Mix-ins

Abstract subclasses or *mix-ins* are templates for classes. A class can only have a single superclass, so multiple inheritance from tooling classes, for example, is not possible. The functionality must be provided by the superclass.

A function with a superclass as input and a subclass extending that superclass as output can be used to implement mix-ins:

```
const calculatorMixin = (Base) =>
  class extends Base {
    calc() {}
  };

const randomizerMixin = (Base) =>
  class extends Base {
    randomize() {}
  };
```

A class that uses these mix-ins can then be written like this:

```
class Foo {}
class Bar extends calculatorMixin(randomizerMixin(Foo)) {}
```

Avoiding inheritance

Inheritance is a very strong coupling relationship in object-oriented programming. It means all behaviors of the base class are inherited by the subclass by default, which may not always be what you want. For example, consider the implementation of a `ReadOnlyMap`:

```
class ReadOnlyMap extends Map {
  set() {
    throw new TypeError("A read-only map must be set at construction time.");
  }
}
```

It turns out that `ReadOnlyMap` is not constructible, because the `Map()` constructor calls the instance's `set()` method.

```
const m = new ReadOnlyMap([["a", 1]]); // TypeError: A read-only map must be set at construction time.
```

We may get around this by using a private flag to indicate whether the instance is being constructed. However, a more significant problem with this design is that it breaks the Liskov substitution principle, which states that a subclass should be substitutable for its superclass. If a function expects a `Map` object, it should be able to use a `ReadOnlyMap` object as well, which will break here.

Inheritance often leads to the circle-ellipse problem, because neither type perfectly entails the behavior of the other, although they share a lot of common traits. In general, unless there's a very good reason to use inheritance, it's better to use composition instead. Composition means that a class has a reference to an object of another class, and only uses that object as an implementation detail.

```
class ReadOnlyMap {
  #data;
  constructor(values) {
    this.#data = new Map(values);
  }
  get(key) {
    return this.#data.get(key);
  }
  has(key) {
    return this.#data.has(key);
  }
  get size() {
    return this.#data.size;
  }
  *keys() {
    yield* this.#data.keys();
  }
  *values() {
    yield* this.#data.values();
  }
  *entries() {
    yield* this.#data.entries();
  }
  *[Symbol.iterator]() {
    yield* this.#data[Symbol.iterator]();
  }
}
```

In this case, the `ReadOnlyMap` class is not a subclass of `Map`, but it still implements most of the same methods. This means more code duplication, but it also means that the `ReadOnlyMap` class is not strongly coupled to the `Map` class, and does not easily break if the `Map` class is changed, avoiding the semantic issues of built-in subclassing. For example, if the `Map` class adds a new utility method (such as `getOrInsert()`) that does not call `set()`, it would cause the `ReadOnlyMap` class to no longer be read-only

unless the latter is updated accordingly to override `getOrInsert()` as well. Moreover, `ReadOnlyMap` objects do not have the `set` method at all, which is more accurate than throwing an error at runtime.

See also

- Using classes guide
- Classes
- `constructor`
- `class`
- `super`

Private elements

Private elements are counterparts of the regular class elements which are public, including class fields, class methods, etc. Private elements get created by using a hash `#` prefix and cannot be legally referenced outside of the class. The privacy encapsulation of these class elements is enforced by JavaScript itself. The only way to access a private element is via dot notation, and you can only do so within the class that defines the private element.

Private elements were not native to the language before this syntax existed. In prototypal inheritance, its behavior may be emulated with `WeakMap` objects or closures, but they can't compare to the `#` syntax in terms of ergonomics.

> **Note:** On MDN, we avoid using the term "private property". A property in JavaScript has a string or symbol key, and has attributes like `writable`, `enumerable`, and `configurable`, but private elements have none. While private elements are accessed with the familiar dot notation, they cannot be proxied, enumerated, deleted, or interacted with using any `Object` method.

Syntax

```
class ClassWithPrivate {
  #privateField;
  #privateFieldWithInitializer = 42;

  #privateMethod() {
    // …
  }

  static #privateStaticField;
  static #privateStaticFieldWithInitializer = 42;

  static #privateStaticMethod() {
    // …
  }
}
```

There are some additional syntax restrictions:

- All private identifiers declared within a class must be unique. The namespace is shared between static and instance elements. The only exception is when the two declarations define a getter-setter pair.
- The private identifier cannot be `#constructor`.

Description

Most class elements have their private counterparts:

- Private fields
- Private methods
- Private static fields
- Private static methods
- Private getters
- Private setters
- Private static getters
- Private static setters

These features are collectively called *private elements*. However, constructors cannot be private in JavaScript. To prevent classes from being constructed outside of the class, you have to use a private flag.

Private elements are declared with **# names** (pronounced "hash names"), which are identifiers prefixed with `#`. The hash prefix is an inherent part of the property name — you can draw relationship with the old underscore prefix convention `_privateField` — but it's not an ordinary string property, so you can't dynamically access it with the bracket notation.

It is a syntax error to refer to `#` names from outside of the class. It is also a syntax error to refer to private elements that were not declared in the class body, or to attempt to remove declared elements with `delete`.

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    delete this.#privateField; // Syntax error
    this.#undeclaredField = 42; // Syntax error
  }
}
```

```
const instance = new ClassWithPrivateField();
instance.#privateField; // Syntax error
```

JavaScript, being a dynamic language, is able to perform this compile-time check because of the special hash identifier syntax, making it different from normal properties on the syntax level.

> **Note:** Code run in the Chrome console can access private elements outside the class. This is a DevTools-only relaxation of the JavaScript syntax restriction.

If you access a private element from an object that doesn't have the element, a `TypeError` is thrown, instead of returning `undefined` as normal properties do.

```
class C {
  #x;

  static getX(obj) {
    return obj.#x;
  }
}

console.log(C.getX(new C())); // undefined
console.log(C.getX({})); // TypeError: Cannot read private member #x from an object whose class did not declare it
```

This example also illustrates that you can access private elements within static functions too, and on externally defined instances of the class.

You can use the `in` operator to check whether an externally defined object possesses a private element. This will return `true` if the private field or method exists, and `false` otherwise.

```
class C {
  #x;
  constructor(x) {
    this.#x = x;
  }
  static getX(obj) {
    if (#x in obj) return obj.#x;

    return "obj must be an instance of C";
  }
}
console.log(C.getX(new C("foo"))); // "foo"
console.log(C.getX(new C(0.196))); // 0.196
console.log(C.getX(new C(new Date()))); // the current date and time
console.log(C.getX({})); // "obj must be an instance of C"
```

Note a corollary of private names being always pre-declared and non-deletable: if you found that an object possesses one private element of the current class (either from a `try...catch` or an `in` check), it must possess all other private elements. An object possessing the private elements of a class generally means it was constructed by that class (although not always).

Private elements are not part of the prototypical inheritance model since they can only be accessed within the current class's body and aren't inherited by subclasses. Private elements with the same name within different classes are entirely different and do not interoperate with each other. See them as external metadata attached to each instance, managed by the class. For this reason, `structuredClone()` does not clone private elements, and `Object.freeze()` and `Object.seal()` have no effect on private elements.

For more information on how and when private fields are initialized, see public class fields.

Examples

Private fields

Private fields include private instance fields and private static fields. Private fields are only accessible from inside the class declaration.

Private instance fields

Like their public counterparts, private instance fields:

- are added before the constructor runs in a base class, or immediately after `super()` is invoked in a subclass, and
- are only available on instances of the class.

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    this.#privateField = 42;
  }
}

class Subclass extends ClassWithPrivateField {
  #subPrivateField;

  constructor() {
    super();
    this.#subPrivateField = 23;
  }
```

```
}

new Subclass(); // In some dev tools, it shows Subclass {#privateField: 42, #subPrivateField: 23}
```

> **Note:** `#privateField` from the `ClassWithPrivateField` base class is private to `ClassWithPrivateField` and is not accessible from the derived `Subclass`.

Returning overriding object

A class's constructor can return a different object, which will be used as the new `this` for the derived class constructor. The derived class may then define private fields on that returned object — meaning it is possible to "stamp" private fields onto unrelated objects.

```
class Stamper extends class {
  // A base class whose constructor returns the object it's given
  constructor(obj) {
    return obj;
  }
} {
  // This declaration will "stamp" the private field onto the object
  // returned by the base class constructor
  #stamp = 42;
  static getStamp(obj) {
    return obj.#stamp;
  }
}

const obj = {};
new Stamper(obj);
// `Stamper` calls `Base`, which returns `obj`, so `obj` is
// now the `this` value. `Stamper` then defines `#stamp` on `obj`

console.log(obj); // In some dev tools, it shows {#stamp: 42}
console.log(Stamper.getStamp(obj)); // 42
console.log(obj instanceof Stamper); // false

// You cannot stamp private elements twice
new Stamper(obj); // Error: Initializing an object twice is an error with private fields
```

> **Warning:** This is a potentially very confusing thing to do. You are generally advised to avoid returning anything from the constructor — especially something unrelated to `this`.

Private static fields

Like their public counterparts, private static fields:

- are added to the class constructor at class evaluation time, and
- are only available on the class itself.

```
class ClassWithPrivateStaticField {
  static #privateStaticField = 42;

  static publicStaticMethod() {
    return ClassWithPrivateStaticField.#privateStaticField;
  }
}

console.log(ClassWithPrivateStaticField.publicStaticMethod()); // 42
```

There is a restriction on private static fields: only the class which defines the private static field can access the field. This can lead to unexpected behavior when using `this`. In the following example, `this` refers to the `Subclass` class (not the `ClassWithPrivateStaticField` class) when we try to call `Subclass.publicStaticMethod()`, and so causes a `TypeError`.

```
class ClassWithPrivateStaticField {
  static #privateStaticField = 42;

  static publicStaticMethod() {
    return this.#privateStaticField;
  }
}

class Subclass extends ClassWithPrivateStaticField {}

Subclass.publicStaticMethod(); // TypeError: Cannot read private member #privateStaticField from an object whose class did not declare it
```

This is the same if you call the method with `super`, because `super` methods are not called with the super class as `this`.

```
class ClassWithPrivateStaticField {
  static #privateStaticField = 42;

  static publicStaticMethod() {
```

```
    // When invoked through super, `this` still refers to Subclass
    return this.#privateStaticField;
  }
}

class Subclass extends ClassWithPrivateStaticField {
  static callSuperMethod() {
    return super.publicStaticMethod();
  }
}

Subclass.callSuperMethod(); // TypeError: Cannot read private member #privateStaticField from an object whose class did not declare it
```

You are advised to always access private static fields through the class name, not through `this`, so inheritance doesn't break the method.

Private methods

Private methods include private instance methods and private static methods. Private methods are only accessible from inside the class declaration.

Private instance methods

Unlike their public counterparts, private instance methods:

- are installed immediately before the instance fields are installed, and
- are only available on instances of the class, not on its `.prototype` property.

```
class ClassWithPrivateMethod {
  #privateMethod() {
    return 42;
  }

  publicMethod() {
    return this.#privateMethod();
  }
}

const instance = new ClassWithPrivateMethod();
console.log(instance.publicMethod()); // 42
```

Private instance methods may be generator, async, or async generator functions. Private getters and setters are also possible, and follow the same syntax requirements as their public getter and setter counterparts.

```
class ClassWithPrivateAccessor {
  #message;

  get #decoratedMessage() {
    return `🎬${this.#message}🔴`;
  }
  set #decoratedMessage(msg) {
    this.#message = msg;
  }

  constructor() {
    this.#decoratedMessage = "hello world";
    console.log(this.#decoratedMessage);
  }
}

new ClassWithPrivateAccessor(); // 🎬hello world🔴
```

Unlike public methods, private methods are not accessible on the `.prototype` property of their class.

```
class C {
  #method() {}

  static getMethod(x) {
    return x.#method;
  }
}

console.log(C.getMethod(new C())); // [Function: #method]
console.log(C.getMethod(C.prototype)); // TypeError: Receiver must be an instance of class C
```

Private static methods

Like their public counterparts, private static methods:

- are added to the class constructor at class evaluation time, and
- are only available on the class itself.

```
class ClassWithPrivateStaticMethod {
  static #privateStaticMethod() {
    return 42;
  }
```

```
  static publicStaticMethod() {
    return ClassWithPrivateStaticMethod.#privateStaticMethod();
  }
}

console.log(ClassWithPrivateStaticMethod.publicStaticMethod()); // 42
```

Private static methods may be generator, async, and async generator functions.

The same restriction previously mentioned for private static fields holds for private static methods, and similarly can lead to unexpected behavior when using `this`. In the following example, when we try to call `Subclass.publicStaticMethod()`, `this` refers to the `Subclass` class (not the `ClassWithPrivateStaticMethod` class) and so causes a `TypeError`.

```
class ClassWithPrivateStaticMethod {
  static #privateStaticMethod() {
    return 42;
  }

  static publicStaticMethod() {
    return this.#privateStaticMethod();
  }
}

class Subclass extends ClassWithPrivateStaticMethod {}

console.log(Subclass.publicStaticMethod()); // TypeError: Cannot read private member #privateStaticMethod from an object whose class did not declare it
```

Simulating private constructors

Many other languages include the capability to mark a constructor as private, which prevents the class from being instantiated outside of the class itself — you can only use static factory methods that create instances, or not be able to create instances at all. JavaScript does not have a native way to do this, but it can be accomplished by using a private static flag.

```
class PrivateConstructor {
  static #isInternalConstructing = false;

  constructor() {
    if (!PrivateConstructor.#isInternalConstructing) {
      throw new TypeError("PrivateConstructor is not constructable");
    }
    PrivateConstructor.#isInternalConstructing = false;
    // More initialization logic
  }

  static create() {
    PrivateConstructor.#isInternalConstructing = true;
    const instance = new PrivateConstructor();
    return instance;
  }
}

new PrivateConstructor(); // TypeError: PrivateConstructor is not constructable
PrivateConstructor.create(); // PrivateConstructor {}
```

Browser compatibility

See also

- Using classes guide
- Classes
- Public class fields
- `class`
- Private Syntax FAQ in the TC39 class-fields proposal
- The semantics of all JS class elements by Shu-yu Guo (2018)
- Public and private class fields on v8.dev (2018)

Public class fields

Public fields are writable, enumerable, and configurable properties defined on each class instance or class constructor.

Syntax

```
class ClassWithField {
  instanceField;
  instanceFieldWithInitializer = "instance field";
  static staticField;
  static staticFieldWithInitializer = "static field";
}
```

There are some additional syntax restrictions:

- The name of a static property (field or method) cannot be `prototype`.
- The name of a class field (static or instance) cannot be `constructor`.

Description

This page introduces public instance fields in detail.

- For public static fields, see `static`.
- For private fields, see private elements.
- For public methods, see method definitions.
- For public accessors, see getter and setter.

Public instance fields exist on every created instance of a class. By declaring a public field, you can ensure the field is always present, and the class definition is more self-documenting.

Public instance fields are added to the instance either at construction time in the base class (before the constructor body runs), or just after `super()` returns in a subclass. Fields without initializers are initialized to `undefined`. Like properties, field names may be computed.

```
const PREFIX = "prefix";

class ClassWithField {
  field;
  fieldWithInitializer = "instance field";
  [`${PREFIX}Field`] = "prefixed field";
}

const instance = new ClassWithField();
console.log(Object.hasOwn(instance, "field")); // true
console.log(instance.field); // undefined
console.log(instance.fieldWithInitializer); // "instance field"
console.log(instance.prefixField); // "prefixed field"
```

Computed field names are only evaluated once, at class definition time. This means that each class always has a fixed set of field names, and two instances cannot have different field names via computed names. The `this` value in the computed expression is the `this` surrounding the class definition, and referring to the class's name is a `ReferenceError` because the class is not initialized yet. `await` and `yield` work as expected in this expression.

```
class C {
  [Math.random()] = 1;
}

console.log(new C());
console.log(new C());
// Both instances have the same field name
```

In the field initializer, `this` refers to the class instance under construction, and `super` refers to the `prototype` property of the base class, which contains the base class's instance methods, but not its instance fields.

```
class Base {
  baseField = "base field";
  anotherBaseField = this.baseField;
  baseMethod() {
    return "base method output";
  }
}

class Derived extends Base {
  subField = super.baseMethod();
}

const base = new Base();
const sub = new Derived();

console.log(base.anotherBaseField); // "base field"

console.log(sub.subField); // "base method output"
```

The field initializer expression is evaluated each time a new instance is created. (Because the `this` value is different for each instance, the initializer expression can access instance-specific properties.)

```
class C {
  obj = {};
}

const instance1 = new C();
const instance2 = new C();
console.log(instance1.obj === instance2.obj); // false
```

The expression is evaluated synchronously. You cannot use `await` or `yield` in the initializer expression. (Think of the initializer expression as being implicitly wrapped in a function.)

Because instance fields of a class are added before the respective constructor runs, you can access the fields' values within the constructor. However, because instance fields of a derived class are defined after `super()` returns, the base class's constructor does not have access to the derived class's fields.

```
class Base {
  constructor() {
    console.log("Base constructor:", this.field);
  }
}

class Derived extends Base {
  field = 1;
  constructor() {
    super();
    console.log("Derived constructor:", this.field);
    this.field = 2;
  }
}

const instance = new Derived();
// Base constructor: undefined
// Derived constructor: 1
console.log(instance.field); // 2
```

Fields are added one-by-one. Field initializers can refer to field values above it, but not below it. All instance and static methods are added beforehand and can be accessed, although calling them may not behave as expected if they refer to fields below the one being initialized.

```
class C {
  a = 1;
  b = this.c;
  c = this.a + 1;
  d = this.c + 1;
}

const instance = new C();
console.log(instance.d); // 3
console.log(instance.b); // undefined
```

> **Note:** This is more important with private fields, because accessing a non-initialized private field throws a `TypeError`, even if the private field is declared below. (If the private field is not declared, it would be an early `SyntaxError`.)

Because class fields are added using the `[[DefineOwnProperty]]` semantic (which is essentially `Object.defineProperty()`), field declarations in derived classes do not invoke setters in the base class. This behavior differs from using `this.field = …` in the constructor.

```
class Base {
  set field(val) {
    console.log(val);
  }
}

class DerivedWithField extends Base {
  field = 1;
}

const instance = new DerivedWithField(); // No log

class DerivedWithConstructor extends Base {
  constructor() {
    super();
    this.field = 1;
  }
}

const instance2 = new DerivedWithConstructor(); // Logs 1
```

> **Note:** Before the class fields specification was finalized with the `[[DefineOwnProperty]]` semantic, most transpilers, including Babel and tsc, transformed class fields to the `DerivedWithConstructor` form, which has caused subtle bugs after class fields were standardized.

Examples

Using class fields

Class fields cannot depend on arguments of the constructor, so field initializers usually evaluate to the same value for each instance (unless the same expression can evaluate to different values each time, such as `Math.random()` or object initializers).

```
class Person {
  name = nameArg; // nameArg is out of scope of the constructor
  constructor(nameArg) {}
}
```

```
class Person {
  // All instances of Person will have the same name
```

```
  name = "Dragomir";
}
```

However, even declaring an empty class field is beneficial, because it indicates the existence of the field, which allows type checkers as well as human readers to statically analyze the shape of the class.

```
class Person {
  name;
  age;
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}
```

The code above seems repetitive, but consider the case where `this` is dynamically mutated: the explicit field declaration makes it clear which fields will definitely be present on the instance.

```
class Person {
  name;
  age;
  constructor(properties) {
    Object.assign(this, properties);
  }
}
```

Because initializers are evaluated after the base class has executed, you can access properties created by the base class constructor.

```
class Person {
  name;
  age;
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

class Professor extends Person {
  name = `Professor ${this.name}`;
}

console.log(new Professor("Radev", 54).name); // "Professor Radev"
```

See also

- Using classes guide
- Classes
- Private elements
- `class`
- The semantics of all JS class elements by Shu-yu Guo (2018)
- Public and private class fields on v8.dev (2018)

static

The **`static`** keyword defines a static method or field for a class, or a static initialization block (see the link for more information about this usage). Static properties cannot be directly accessed on instances of the class. Instead, they're accessed on the class itself.

Static methods are often utility functions, such as functions to create or clone objects, whereas static properties are useful for caches, fixed-configuration, or any other data you don't need to be replicated across instances.

> **Note:** In the context of classes, MDN Web Docs content uses the terms properties and fields interchangeably.

Try it

```
class ClassWithStaticMethod {
  static staticProperty = "someValue";
  static staticMethod() {
    return "static method has been called.";
  }
  static {
    console.log("Class static initialization block called");
  }
}

console.log(ClassWithStaticMethod.staticProperty);
// Expected output: "someValue"
console.log(ClassWithStaticMethod.staticMethod());
// Expected output: "static method has been called."
```

Syntax

```
class ClassWithStatic {
  static staticField;
  static staticFieldWithInitializer = value;
  static staticMethod() {
    // …
  }
}
```

There are some additional syntax restrictions:

- The name of a static property (field or method) cannot be `prototype`.
- The name of a class field (static or instance) cannot be `constructor`.

Description

This page introduces public static properties of classes, which include static methods, static accessors, and static fields.

- For private static features, see private elements.
- For instance features, see methods definitions, getter, setter, and public class fields.

Public static features are declared using the `static` keyword. They are added to the class constructor at the time of class evaluation using the `[[DefineOwnProperty]]` semantic (which is essentially `Object.defineProperty()`). They are accessed again from the class constructor.

Static methods are often utility functions, such as functions to create or clone instances. Public static fields are useful when you want a field to exist only once per class, not on every class instance you create. This is useful for caches, fixed-configuration, or any other data you don't need to be replicated across instances.

Static field names can be computed. The `this` value in the computed expression is the `this` surrounding the class definition, and referring to the class's name is a `ReferenceError` because the class is not initialized yet. `await` and `yield` work as expected in this expression.

Static fields can have an initializer. Static fields without initializers are initialized to `undefined`. Public static fields are not reinitialized on subclasses, but can be accessed via the prototype chain.

```
class ClassWithStaticField {
  static staticField;
  static staticFieldWithInitializer = "static field";
}

class SubclassWithStaticField extends ClassWithStaticField {
  static subStaticField = "subclass field";
}

console.log(Object.hasOwn(ClassWithStaticField, "staticField")); // true
console.log(ClassWithStaticField.staticField); // undefined
console.log(ClassWithStaticField.staticFieldWithInitializer); // "static field"
console.log(SubclassWithStaticField.staticFieldWithInitializer); // "static field"
console.log(SubclassWithStaticField.subStaticField); // "subclass field"
```

In the field initializer, `this` refers to the current class (which you can also access through its name), and `super` refers to the base class constructor.

```
class ClassWithStaticField {
  static baseStaticField = "base static field";
  static anotherBaseStaticField = this.baseStaticField;

  static baseStaticMethod() {
    return "base static method output";
  }
}

class SubClassWithStaticField extends ClassWithStaticField {
  static subStaticField = super.baseStaticMethod();
}

console.log(ClassWithStaticField.anotherBaseStaticField); // "base static field"
console.log(SubClassWithStaticField.subStaticField); // "base static method output"
```

The expression is evaluated synchronously. You cannot use `await` or `yield` in the initializer expression. (Think of the initializer expression as being implicitly wrapped in a function.)

Static field initializers and static initialization blocks are evaluated one-by-one. Field initializers can refer to field values above it, but not below it. All static methods are added beforehand and can be accessed, although calling them may not behave as expected if they refer to fields below the one being initialized.

> **Note:** This is more important with private static fields, because accessing a non-initialized private field throws a `TypeError`, even if the private field is declared below. (If the private field is not declared, it would be an early `SyntaxError`.)

Examples

Using static members in classes

The following example demonstrates several things:

1. How a static member (method or property) is defined on a class.

2. That a class with a static member can be sub-classed.
3. How a static member can and cannot be called.

```
class Triple {
  static customName = "Tripler";
  static description = "I triple any number you provide";
  static calculate(n = 1) {
    return n * 3;
  }
}

class SquaredTriple extends Triple {
  static longDescription;
  static description = "I square the triple of any number you provide";
  static calculate(n) {
    return super.calculate(n) * super.calculate(n);
  }
}

console.log(Triple.description); // 'I triple any number you provide'
console.log(Triple.calculate()); // 3
console.log(Triple.calculate(6)); // 18

const tp = new Triple();

console.log(SquaredTriple.calculate(3)); // 81 (not affected by parent's instantiation)
console.log(SquaredTriple.description); // 'I square the triple of any number you provide'
console.log(SquaredTriple.longDescription); // undefined
console.log(SquaredTriple.customName); // 'Tripler'

// This throws because calculate() is a static member, not an instance member.
console.log(tp.calculate()); // 'tp.calculate is not a function'
```

Calling static members from another static method

In order to call a static method or property within another static method of the same class, you can use the `this` keyword.

```
class StaticMethodCall {
  static staticProperty = "static property";
  static staticMethod() {
    return `Static method and ${this.staticProperty} has been called`;
  }
  static anotherStaticMethod() {
    return `${this.staticMethod()} from another static method`;
  }
}
StaticMethodCall.staticMethod();
// 'Static method and static property has been called'

StaticMethodCall.anotherStaticMethod();
// 'Static method and static property has been called from another static method'
```

Calling static members from a class constructor and other methods

Static members are not directly accessible using the `this` keyword from non-static methods. You need to call them using the class name: `CLASSNAME.STATIC_METHOD_NAME()` / `CLASSNAME.STATIC_PROPERTY_NAME` or by calling the method as a property of the `constructor`: `this.constructor.STATIC_METHOD_NAME()` / `this.constructor.STATIC_PROPERTY_NAME`

```
class StaticMethodCall {
  constructor() {
    console.log(StaticMethodCall.staticProperty); // 'static property'
    console.log(this.constructor.staticProperty); // 'static property'
    console.log(StaticMethodCall.staticMethod()); // 'static method has been called.'
    console.log(this.constructor.staticMethod()); // 'static method has been called.'
  }

  static staticProperty = "static property";
  static staticMethod() {
    return "static method has been called.";
  }
}
```

See also

- Using classes guide
- Classes
- Static initialization blocks
- `class`

Static initialization blocks

Static initialization blocks are declared within a `class`. It contains statements to be evaluated during class initialization. This permits more flexible initialization logic than `static` properties, such as using `try...catch` or setting multiple fields from a single value. Initialization is performed in the context of the current class declaration,

with access to private state, which allows the class to share information of its private elements with other classes or functions declared in the same scope (analogous to "friend" classes in C++).

Try it

```
class ClassWithStaticInitializationBlock {
  static staticProperty1 = "Property 1";
  static staticProperty2;
  static {
    this.staticProperty2 = "Property 2";
  }
}

console.log(ClassWithStaticInitializationBlock.staticProperty1);
// Expected output: "Property 1"
console.log(ClassWithStaticInitializationBlock.staticProperty2);
// Expected output: "Property 2"
```

Syntax

```
class ClassWithSIB {
  static {
    // …
  }
}
```

Description

Without static initialization blocks, complex static initialization might be achieved by calling a static method after the class declaration:

```
class MyClass {
  static init() {
    // Access to private static fields is allowed here
  }
}

MyClass.init();
```

However, this approach exposes an implementation detail (the `init()` method) to the user of the class. On the other hand, any initialization logic declared outside the class does not have access to private static fields. Static initialization blocks allow arbitrary initialization logic to be declared within the class and executed during class evaluation.

A `class` can have any number of `static {}` initialization blocks in its class body. These are evaluated, along with any interleaved static field initializers, in the order they are declared. Any static initialization of a super class is performed first, before that of its sub classes.

The scope of the variables declared inside the static block is local to the block. This includes `var`, `function`, `const`, and `let` declarations. `var` declarations will not be hoisted out of the static block.

```
var y = "Outer y";

class A {
  static field = "Inner y";
  static {
    // var y only hoisted inside block
    console.log(y); // undefined <-- not 'Outer y'

    var y = this.field;
  }
}

// var y defined in static block is not hoisted
// outside the block
console.log(y); // 'Outer y'
```

The `this` inside a static block refers to the constructor object of the class. `super.property` can be used to access static properties of the super class. Note however that it is a syntax error to call `super()` in a class static initialization block, or to use the `arguments` object.

The statements are evaluated synchronously. You cannot use `await` or `yield` in this block. (Think of the initialization statements as being implicitly wrapped in a function.)

The scope of the static block is nested *within* the lexical scope of the class body, and can access private names declared within the class without causing a syntax error.

Static field initializers and static initialization blocks are evaluated one-by-one. The initialization block can refer to field values above it, but not below it. All static methods are added beforehand and can be accessed, although calling them may not behave as expected if they refer to fields below the current block.

> **Note:** This is more important with private static fields, because accessing a non-initialized private field throws a `TypeError`, even if the private field is declared below. (If the private field is not declared, it would be an early `SyntaxError`.)

A static initialization block may not have decorators (the class itself may).

Examples

Multiple blocks

The code below demonstrates a class with static initialization blocks and interleaved static field initializers. The output shows that the blocks and fields are evaluated in execution order.

```
class MyClass {
  static field1 = console.log("static field1");
  static {
    console.log("static block1");
  }
  static field2 = console.log("static field2");
  static {
    console.log("static block2");
  }
}
// 'static field1'
// 'static block1'
// 'static field2'
// 'static block2'
```

Note that any static initialization of a super class is performed first, before that of its sub classes.

Using this and super

The `this` inside a static block refers to the constructor object of the class. This code shows how to access a public static field.

```
class A {
  static field = "static field";
  static {
    console.log(this.field);
  }
}
// 'static field'
```

The `super.property` syntax can be used inside a `static` block to reference static properties of a super class.

```
class A {
  static field = "static field";
}

class B extends A {
  static {
    console.log(super.field);
  }
}
// 'static field'
```

Access to private elements

This example below shows how access can be granted to a private instance field of a class from an object outside the class (example from the v8.dev blog):

```
let getDPrivateField;

class D {
  #privateField;
  constructor(v) {
    this.#privateField = v;
  }
  static {
    getDPrivateField = (d) => d.#privateField;
  }
}

console.log(getDPrivateField(new D("private"))); // 'private'
```

See also

- Using classes guide
- Classes
- `static`
- `class`
- Class static initialization blocks on v8.dev (2021)
- ES2022 feature: class static initialization blocks by Dr. Axel Rauschmayer (2021)

Deprecated and obsolete features

This page lists features of JavaScript that are deprecated (that is, still available but planned for removal) and obsolete (that is, no longer usable).

Deprecated features

These deprecated features can still be used, but should be used with caution because they are not required to be implemented by every JavaScript engine. You should work to remove their use from your code.

Some of these deprecated features are listed in the Annex B section of the ECMAScript specification. This section is described as normative optional — that is, web browser hosts must implement these features, while non-web hosts may not. These features are likely stable because removing them will cause backward compatibility issues and break legacy websites. (JavaScript has the design goal of "don't break the web".) Still, they are not cross-platform portable and may not be supported by all analysis tools, so you are advised to not use them, as the introduction of Annex B states:

> … All of the language features and behaviors specified in this annex have one or more undesirable characteristics and in the absence of legacy usage would be removed from this specification. …
>
> … Programmers should not use or assume the existence of these features and behaviors when writing new ECMAScript code. …

Some others, albeit in the main spec body, are also marked as normative optional and should not be depended on.

HTML comments

JavaScript source, if parsed as scripts, allows HTML-like comments, as if the script is part of a `<script>` tag.

The following is valid JavaScript when running in a web browser (or Node.js, which uses the V8 engine powering Chrome):

```
<!-- comment
console.log("a"); <!-- another comment
console.log("b");
--> More comment
// Logs "a" and "b"
```

`<!--` and `-->` both act like `//`, i.e., starting line comments. `-->` is only valid at the start of a line (to avoid ambiguity with a postfix decrement followed by a greater than operator), while `<!--` can occur anywhere in the line.

RegExp

The following properties are deprecated. This does not affect their use in replacement strings:

`$1–$9`

: Parenthesized substring matches, if any.

`input`, `$_`

: The string against which a regular expression is matched.

`lastMatch`, `$&`

: The last matched substring.

`lastParen`, `$+`

: The last parenthesized substring match, if any.

`leftContext`, `` $` ``

: The substring preceding the most recent match.

`rightContext`, `$'`

: The substring following the most recent match.

> **Warning:** Avoid using these static properties, as they can cause issues when interacting with external code!

The `compile()` method is deprecated. Construct a new `RegExp` instance instead.

The following regex syntaxes are deprecated and only available in Unicode-unaware mode. In Unicode-aware mode, they are all syntax errors:

- Lookahead assertions can have quantifiers.
- Backreferences that do not refer to an existing capturing group become legacy octal escapes.
- In character classes, character ranges where one boundary is a character class makes the `-` become a literal character.
- An escape sequence that's not recognized becomes an "identity escape".
- Escape sequences within character classes of the form `\cX` where X is a number or _ are decoded in the same way as those with ASCII letters: `\c0` is the same as `\cP` when taken modulo 32. In addition, if the form `\cX` is encountered anywhere where X is not one of the recognized characters, then the backslash is treated as a literal character.
- The sequence `\k` within a regex that doesn't have any named capturing groups is treated as an identity escape.
- The syntax characters `]`, `{`, and `}` may appear literally without escaping if they cannot be interpreted as the end of a character class or quantifier delimiters.

Function

- The `caller` property of functions and the `arguments.callee` property are deprecated and unavailable in strict mode.
- Instead of accessing `arguments` as a property of a function, you should use the `arguments` object inside function closures.

Object

- The `Object.prototype.__proto__` accessors are deprecated. Use `Object.getPrototypeOf` and `Object.setPrototypeOf` instead. This does not apply to the `__proto__` literal key in object literals.
- The `Object.prototype.__defineGetter__`, `Object.prototype.__defineSetter__`, `Object.prototype.__lookupGetter__`, and `Object.prototype.__lookupSetter__` methods are deprecated. Use `Object.getOwnPropertyDescriptor` and `Object.defineProperty` instead.

String

- HTML wrapper methods like `String.prototype.fontsize` and `String.prototype.big`.
- `String.prototype.substr` probably won't be removed anytime soon, but it's defined in Annex B and hence normative optional.
- `String.prototype.trimLeft` and `String.prototype.trimRight` should be replaced with `String.prototype.trimStart` and `String.prototype.trimEnd`.

Date

- The `getYear()` and `setYear()` methods are affected by the Year-2000-Problem and have been subsumed by `getFullYear` and `setFullYear`.
- The `toGMTString()` method is deprecated. Use `toUTCString()` instead.

Escape sequences

- Octal escape sequences (\ followed by one, two, or three octal digits) are deprecated in string and regular expression literals.
- The `escape()` and `unescape()` functions are deprecated. Use `encodeURI()`, `encodeURIComponent()`, `decodeURI()`, or `decodeURIComponent()` to encode and decode escape sequences for special characters.

Statements

The `with` statement is deprecated and unavailable in strict mode.

Initializers in `var` declarations of `for...in` loops headers are deprecated and produce syntax errors in strict mode. The initializer expression is evaluated and assigned to the variable, but the value would be immediately reassigned on the first iteration of the loop.

Normally, the `catch` block of a `try...catch` statement cannot contain any variable declaration with the same name as the variables bound in the `catch()`. An extension grammar allows the `catch` block to contain a `var` declared variable with the same name as the `catch`-bound identifier, but only if the `catch` binding is a simple identifier, not a destructuring pattern. However, this variable's initialization and assignment would only act on the `catch`-bound identifier, instead of the upper scope variable, and the behavior could be confusing.

```
var a = 2;
try {
  throw new Error();
} catch (a) {
  var a = 1; // This 1 is assigned to the caught `a`, not the outer `a`.
}
console.log(a); // 2

try {
  throw new Error();
  // Note: identifier changed to `err` to avoid conflict with
  // the inner declaration of `a`.
} catch (err) {
  var a = 1; // This 1 is assigned to the upper-scope `a`.
}
console.log(a); // 1
```

This is listed in Annex B of the spec and hence may not be implemented everywhere. Avoid any name conflicts between the `catch`-bound identifier and variables declared in the `catch` block.

Obsolete features

These obsolete features have been entirely removed from JavaScript and can no longer be used as of the indicated version of JavaScript.

RegExp

The following are now properties of `RegExp` instances, no longer of the `RegExp` constructor:

Property	Description
`global`	Whether or not to test the regular expression against all possible matches in a string, or only against the first.
`ignoreCase`	Whether or not to ignore case while attempting a match in a string.
`lastIndex`	The index at which to start the next match.
`multiline` (also via `RegExp.$*`)	Whether or not to search in strings across multiple lines.
`source`	The text of the pattern.

The `valueOf()` method is no longer specialized for `RegExp`. It uses `Object.prototype.valueOf()`, which returns itself.

Function

- Functions' `arity` property is obsolete. Use `length` instead.

Object

Property	Description	Alternative
`__count__`	Returns the number of enumerable properties directly available on a user-defined object.	`Object.keys()`
`__parent__`	Points to an object's context.	No direct replacement
`__iterator__`	Used with legacy iterators.	`Symbol.iterator` and the new iteration protocols
`__noSuchMethod__`	A method called when a non-existent property is called as method.	`Proxy`
`Object.prototype.eval()`	Evaluates a string of JavaScript code in the context of the specified object.	No direct replacement
`Object.observe()`	Asynchronously observing the changes to an object.	`Proxy`
`Object.unobserve()`	Remove observers.	`Proxy`
`Object.getNotifier()`	Create a notifier object that allows you to synthetically trigger a change observable with `Object.observe()`.	No direct replacement
`Object.prototype.watch()`	Attach a handler callback to a property that gets called when the property is assigned.	`Proxy`
`Object.prototype.unwatch()`	Remove watch handlers on a property.	`Proxy`

String

- Non-standard String generic methods like `String.slice(myStr, 0, 12)`, `String.replace(myStr, /\./g, "!")`, etc. have been introduced in Firefox 1.5 (JavaScript 1.6), deprecated in Firefox 53, and removed in Firefox 68. You can use methods on `String.prototype` together with `Function.call` instead.
- `String.prototype.quote` is removed from Firefox 37.
- Non-standard `flags` parameter in `String.prototype.search`, `String.prototype.match`, and `String.prototype.replace` are obsolete.

WeakMap

- `WeakMap.prototype.clear()` was added in Firefox 20 and removed in Firefox 46. It is not possible to traverse all keys in a `WeakMap`.

Date

- `Date.prototype.toLocaleFormat()`, which used a format string in the same format expected by the `strftime()` function in C, is obsolete. Use `toLocaleString()` or `Intl.DateTimeFormat` instead.

Array

- Non-standard Array generic methods like `Array.slice(myArr, 0, 12)`, `Array.forEach(myArr, myFn)`, etc. have been introduced in Firefox 1.5 (JavaScript 1.6), deprecated in Firefox 68, and removed in Firefox 71. You can use methods on `Array.prototype` together with `Function.call` instead.

Property	Description	Alternative
`Array.observe()`	Asynchronously observing changes to Arrays.	`Proxy`
`Array.unobserve()`	Remove observers.	`Proxy`

Number

- `Number.toInteger()` is obsolete. Use `Math.floor`, `Math.round`, or other methods instead.

Proxy

- `Proxy.create` and `Proxy.createFunction` are obsolete. Use the `Proxy()` constructor instead.
- The following traps are obsolete:
  - `hasOwn` (bug 980565, Firefox 33).
  - `getEnumerablePropertyKeys` (bug 783829, Firefox 37)
  - `getOwnPropertyNames` (bug 1007334, Firefox 33)
  - `keys` (bug 1007334, Firefox 33)

ParallelArray

- `ParallelArray` is obsolete.

Statements

- `for each...in` is obsolete. Use `for...of` instead.
- let blocks and let expressions are obsolete.
- Expression closures (`function () 1` as a shorthand of `function () { return 1; }`) are obsolete. Use regular `functions` or arrow functions instead.

Acquiring source text

The `toSource()` methods of arrays, numbers, strings, etc. and the `uneval()` global function are obsolete. Use `toString()`, or write your own serialization method instead.

Legacy generator and iterator

Legacy generator function statements and legacy generator function expressions are removed. The legacy generator function syntax reuses the `function` keyword, which automatically becomes a generator function when there are one or more `yield` expressions in the body — this is now a syntax error. Use `function*` statements and `function*` expressions instead.

Array comprehensions and generator comprehensions are removed.

```
// Legacy array comprehensions
[for (x of iterable) x]
[for (x of iterable) if (condition) x]
[for (x of iterable) for (y of iterable) x + y]

// Legacy generator comprehensions
(for (x of iterable) x)
(for (x of iterable) if (condition) x)
(for (x of iterable) for (y of iterable) x + y)
```

Firefox, prior to version 26, implemented another iterator protocol that is similar to the standard Iterator protocol. An object is a legacy iterator when it implements a `next()` method, which produces a value on each call and throws a `StopIteration` object at the end of iteration. This legacy iterator protocol differs from the standard iterator protocol:

- The value was returned directly as the return value of calls to `next()`, instead of the `value` property of the `IteratorResult` object.
- Iteration termination was expressed by throwing a `StopIteration` object, instead of through the `done` property of the `IteratorResult` object.

This feature, along with the `StopIteration` global constructor, was removed in Firefox 58+. For future-facing usages, consider using `for...of` loops and the iterator protocol.

Sharp variables

Sharp variables are obsolete. To create circular structures, use temporary variables instead.

JavaScript error reference

Below, you'll find a list of errors which are thrown by JavaScript. These errors can be a helpful debugging aid, but the reported problem isn't always immediately clear. The pages below will provide additional details about these errors. Each error is an object based upon the `Error` object, and has a `name` and a `message`.

Errors displayed in the Web console may include a link to the corresponding page below to help you quickly comprehend the problem in your code.

For a beginner's introductory tutorial on fixing JavaScript errors, see What went wrong? Troubleshooting JavaScript.

List of errors

In this list, each page is listed by name (the type of error) and message (a more detailed human-readable error message). Together, these two properties provide a starting point toward understanding and resolving the error. For more information, follow the links below!

- AggregateError: No Promise in Promise.any was resolved
- Error: Permission denied to access property "x"
- InternalError: too much recursion
- RangeError: argument is not a valid code point
- RangeError: BigInt division by zero
- RangeError: BigInt negative exponent
- RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD'
- RangeError: invalid array length
- RangeError: invalid date
- RangeError: precision is out of range
- RangeError: radix must be an integer
- RangeError: repeat count must be less than infinity
- RangeError: repeat count must be non-negative
- RangeError: x can't be converted to BigInt because it isn't an integer
- ReferenceError: "x" is not defined
- ReferenceError: assignment to undeclared variable "x"
- ReferenceError: can't access lexical declaration 'X' before initialization
- ReferenceError: must call super constructor before using 'this' in derived class constructor
- ReferenceError: super() called twice in derived class constructor
- SyntaxError: 'arguments'/'eval' can't be defined or assigned to in strict mode code
- SyntaxError: "0"-prefixed octal literals are deprecated
- SyntaxError: "use strict" not allowed in function with non-simple parameters
- SyntaxError: "x" is a reserved identifier
- SyntaxError: \ at end of pattern
- SyntaxError: a declaration in the head of a for-of loop can't have an initializer
- SyntaxError: applying the 'delete' operator to an unqualified name is deprecated
- SyntaxError: arguments is not valid in fields
- SyntaxError: await is only valid in async functions, async generators and modules
- SyntaxError: await/yield expression can't be used in parameter
- SyntaxError: cannot use ?? unparenthesized within || and && expressions
- SyntaxError: character class escape cannot be used in class range in regular expression
- SyntaxError: continue must be inside loop
- SyntaxError: duplicate capture group name in regular expression
- SyntaxError: duplicate formal argument x
- SyntaxError: for-in loop head declarations may not have initializers
- SyntaxError: function statement requires a name
- SyntaxError: functions cannot be labelled
- SyntaxError: getter and setter for private name #x should either be both static or non-static
- SyntaxError: getter functions must have no arguments
- SyntaxError: identifier starts immediately after numeric literal

- SyntaxError: illegal character
- SyntaxError: import declarations may only appear at top level of a module
- SyntaxError: incomplete quantifier in regular expression
- SyntaxError: invalid assignment left-hand side
- SyntaxError: invalid BigInt syntax
- SyntaxError: invalid capture group name in regular expression
- SyntaxError: invalid character in class in regular expression
- SyntaxError: invalid class set operation in regular expression
- SyntaxError: invalid decimal escape in regular expression
- SyntaxError: invalid identity escape in regular expression
- SyntaxError: invalid named capture reference in regular expression
- SyntaxError: invalid property name in regular expression
- SyntaxError: invalid range in character class
- SyntaxError: invalid regexp group
- SyntaxError: invalid regular expression flag "x"
- SyntaxError: invalid unicode escape in regular expression
- SyntaxError: JSON.parse: bad parsing
- SyntaxError: label not found
- SyntaxError: missing : after property id
- SyntaxError: missing) after argument list
- SyntaxError: missing) after condition
- SyntaxError: missing] after element list
- SyntaxError: missing } after function body
- SyntaxError: missing } after property list
- SyntaxError: missing = in const declaration
- SyntaxError: missing formal parameter
- SyntaxError: missing name after . operator
- SyntaxError: missing variable name
- SyntaxError: negated character class with strings in regular expression
- SyntaxError: new keyword cannot be used with an optional chain
- SyntaxError: nothing to repeat
- SyntaxError: numbers out of order in {} quantifier.
- SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code
- SyntaxError: parameter after rest parameter
- SyntaxError: private fields can't be deleted
- SyntaxError: property name __proto__ appears more than once in object literal
- SyntaxError: raw bracket is not allowed in regular expression with unicode flag
- SyntaxError: redeclaration of formal parameter "x"
- SyntaxError: reference to undeclared private field or method #x
- SyntaxError: rest parameter may not have a default
- SyntaxError: return not in function
- SyntaxError: setter functions must have one argument
- SyntaxError: string literal contains an unescaped line break
- SyntaxError: super() is only valid in derived class constructors
- SyntaxError: tagged template cannot be used with optional chain
- SyntaxError: Unexpected '#' used outside of class body
- SyntaxError: Unexpected token
- SyntaxError: unlabeled break must be inside loop or switch
- SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**'
- SyntaxError: use of super property/member accesses only valid within methods or eval code within methods
- SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead
- TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed
- TypeError: 'x' is not iterable
- TypeError: "x" is (not) "y"
- TypeError: "x" is not a constructor
- TypeError: "x" is not a function
- TypeError: "x" is not a non-null object
- TypeError: "x" is read-only
- TypeError: already executing generator
- TypeError: BigInt value can't be serialized in JSON
- TypeError: calling a builtin X constructor without new is forbidden
- TypeError: can't access/set private field or method: object is not the right class
- TypeError: can't assign to property "x" on "y": not an object
- TypeError: can't convert BigInt to number
- TypeError: can't convert x to BigInt
- TypeError: can't define property "x": "obj" is not extensible
- TypeError: can't delete non-configurable array element
- TypeError: can't redefine non-configurable property "x"
- TypeError: can't set prototype of this object
- TypeError: can't set prototype: it would cause a prototype chain cycle
- TypeError: cannot use 'in' operator to search for 'x' in 'y'
- TypeError: class constructors must be invoked with 'new'
- TypeError: cyclic object value
- TypeError: derived class constructor returned invalid value x

- TypeError: getting private setter-only property
- TypeError: Initializing an object twice is an error with private fields/methods
- TypeError: invalid 'instanceof' operand 'x'
- TypeError: invalid Array.prototype.sort argument
- TypeError: invalid assignment to const "x"
- TypeError: Iterator/AsyncIterator constructor can't be used directly
- TypeError: matchAll/replaceAll must be called with a global RegExp
- TypeError: More arguments needed
- TypeError: null/undefined has no properties
- TypeError: property "x" is non-configurable and can't be deleted
- TypeError: Reduce of empty array with no initial value
- TypeError: setting getter-only property "x"
- TypeError: WeakSet key/WeakMap value 'x' must be an object or an unregistered symbol
- TypeError: X.prototype.y called on incompatible type
- URIError: malformed URI sequence
- Warning: -file- is being assigned a //# sourceMappingURL, but already has one
- Warning: unreachable code after return statement

TypeError: already executing generator

The JavaScript exception "TypeError: already executing generator" occurs when a generator is continued using one of its methods (such as `next()`) while executing the generator function's body itself.

Message

```
TypeError: Generator is already running (V8-based)
TypeError: already executing generator (Firefox)
TypeError: Generator is executing (Safari)
```

Error type

`TypeError`

What went wrong?

The generator's methods, `next()`, `return()`, and `throw()`, are meant to continue the execution of a generator function when it's paused after a `yield` expression or before the first statement. If a call to one of these methods is made while executing the generator function, the error is thrown. If you want to return or throw within the generator function, use the `return` statement or the `throw` statement, respectively.

Examples

```
let it;
function* getNumbers(times) {
  if (times <= 0) {
    it.throw(new Error("times must be greater than 0"));
  }
  for (let i = 0; i < times; i++) {
    yield i;
  }
}
it = getNumbers(3);
it.next();
```

```
let it;
function* getNumbers(times) {
  if (times <= 0) {
    throw new Error("times must be greater than 0");
  }
  for (let i = 0; i < times; i++) {
    yield i;
  }
}
it = getNumbers(3);
it.next(); // { value: 0, done: false }
```

See also

- Iterators and generators
- Iteration protocols
- `function*`
- `Generator`

Warning: -file- is being assigned a //# sourceMappingURL, but already has one

The JavaScript warning "-file- is being assigned a //# sourceMappingURL, but already has one." occurs when a source map has been specified more than once for a given JavaScript source.

Message

```
Warning: -file- is being assigned a //# sourceMappingURL, but already has one.
```

Error type

A warning. JavaScript execution won't be halted.

What went wrong?

A source map has been specified more than once for a given JavaScript source.

JavaScript sources are often combined and minified to make delivering them from the server more efficient. With source maps, the debugger can map the code being executed to the original source files. There are two ways to assign a source map, either by using a comment or by setting a header to the JavaScript file.

Examples

Setting source maps

Setting a source map by using a comment in the file:

```
//# sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Or, alternatively, you can set a header to your JavaScript file:

```
X-SourceMap: /path/to/file.js.map
```

See also

- Use a source map in the Firefox source docs
- Introduction to JavaScript source maps on developer.chrome.com (2012)

SyntaxError: arguments is not valid in fields

The JavaScript exception "SyntaxError: arguments is not valid in fields" occurs when the `arguments` identifier is read in a class field initializer or in a static initialization block, outside of a non-arrow function.

Message

```
SyntaxError: 'arguments' is not allowed in class field initializer or static initialization block (V8-based)
SyntaxError: arguments is not valid in fields (Firefox)
SyntaxError: Unexpected identifier 'arguments'. Cannot reference 'arguments' in class field initializer. (Safari)
```

Error type

`SyntaxError`

What went wrong?

A class field initializer expression or a class static initialization block does not have `arguments` in its scope. Trying to access it is a syntax error.

- This is true even if `arguments` is bound in a parent scope (such as when the class is nested in a non-arrow function).
- A non-arrow function declared within this scope will still bind its own `arguments` and read it normally.

Examples

```
function makeOne() {
  class C {
    args = { ...arguments }; // SyntaxError: arguments is not valid in fields
  }
  return new C();
}
```

```
let CArgs;
class C {
  static {
    CArgs = arguments; // SyntaxError: arguments is not valid in fields
  }
}
```

```
class C {
  args = {};
  constructor() {
    this.args = arguments; // You can use arguments in constructors
  }
  myMethod() {
    this.args = arguments; // You can also use it in methods
  }
}
```

```
function makeOne() {
  const _arguments = arguments;
  class C {
    args = { ..._arguments }; // Only the identifier is forbidden
  }
```

```
  return new C();
}
```

See also

- Classes
- Public class fields
- Static initialization blocks

TypeError: invalid Array.prototype.sort argument

The JavaScript exception "invalid Array.prototype.sort argument" occurs when the argument of `Array.prototype.sort()` (and its related methods: `Array.prototype.toSorted()`, `TypedArray.prototype.sort()`, `TypedArray.prototype.toSorted()`) isn't either `undefined` or a function that compares its operands.

Message

```
TypeError: The comparison function must be either a function or undefined (V8-based)

TypeError: invalid Array.prototype.sort argument (Firefox)
TypeError: non-function passed to Array.prototype.toSorted (Firefox)
TypeError: invalid %TypedArray%.prototype.sort argument (Firefox)

TypeError: Array.prototype.sort requires the comparator argument to be a function or undefined (Safari)
TypeError: Array.prototype.toSorted requires the comparator argument to be a function or undefined (Safari)
TypeError: TypedArray.prototype.sort requires the comparator argument to be a function or undefined (Safari)
TypeError: TypedArray.prototype.toSorted requires the comparator argument to be a function or undefined (Safari)
```

Error type

`TypeError`

What went wrong?

The argument of `Array.prototype.sort()` (and its related methods: `Array.prototype.toSorted()`, `TypedArray.prototype.sort()`, `TypedArray.prototype.toSorted()`) is expected to be either `undefined` or a function which compares its operands.

Examples

Invalid cases

```
[1, 3, 2].sort(5); // TypeError
students.toSorted("name"); // TypeError
```

Valid cases

```
[1, 3, 2].sort(); // [1, 2, 3]
[1, 3, 2].sort((a, b) => a - b); // [1, 2, 3]
students.toSorted((a, b) => a.name.localeCompare(b.name));
```

See also

- `Array.prototype.sort()`
- `Array.prototype.toSorted()`
- `TypedArray.prototype.sort()`
- `TypedArray.prototype.toSorted()`

SyntaxError: await/yield expression can't be used in parameter

The JavaScript exception "await expression can't be used in parameter" or "yield expression can't be used in parameter" occurs when the default parameter expression contains the `await` or `yield` keyword and has the effect of pausing default parameter evaluation.

Message

```
SyntaxError: Illegal await-expression in formal parameters of async function (V8-based)
SyntaxError: await expression can't be used in parameter (Firefox)
SyntaxError: Cannot use 'await' within a parameter default expression. (Safari)

SyntaxError: Yield expression not allowed in formal parameter (V8-based)
SyntaxError: yield expression can't be used in parameter (Firefox)
SyntaxError: Unexpected keyword 'yield'. Cannot use yield expression within parameters. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The default expression must be able to evaluate *synchronously*. If it contains an `await` or `yield` expression, it will pause the evaluation of the default expression, which is not allowed.

> **Note:** This error is only generated when `await` or `yield` are valid operators in this function context. Otherwise, `await` or `yield` would be parsed as an identifier, and

> either not cause an error, or cause an error like "reserved identifier", or "unexpected token" if there's an expression following it.

Examples

Invalid cases

```
function *gen(a = yield 1) {}

async function f(a = await Promise.resolve(1)) {}
```

Valid cases

You can use the nullish coalescing assignment to provide a default value instead. If you want to treat `null` and `undefined` differently, you would need to use a condition.

```
function* gen(a) {
  a ??= yield 1;
}

async function f(a) {
  a ??= await Promise.resolve(1);
}
```

You are also allowed to use `await` or `yield` if the expression is contained in a function expression of the initializer and would not pause the evaluation of the default expression.

```
async function f(a = (async () => await Promise.resolve(1))()) {}
```

See also

- Default parameters
- `await`
- `yield`

SyntaxError: await is only valid in async functions, async generators and modules

The JavaScript exception "await is only valid in async functions, async generators and modules" occurs when an `await` expression is used outside of async functions or modules or other async contexts.

Message

```
SyntaxError: await is only valid in async functions and the top level bodies of modules (V8-based)
SyntaxError: await is only valid in async functions, async generators and modules (Firefox)
SyntaxError: Unexpected identifier (Safari)
```

Error type

`SyntaxError`.

What went wrong?

JavaScript execution is never blocking: an `await` can never block the execution of the program. Instead, it pauses the execution of the surrounding async task, while allowing other tasks to continue running. Therefore, `await` cannot be used in sync tasks, such as functions, generator functions, or top level of scripts. It is not always apparent whether the current file is a script or a module — see the Modules guide for more information.

Examples

Top-level await

You cannot use `await` at the top level of a script:

```
<script>
  await fetch("https://example.com");
  // SyntaxError: await is only valid in async functions, async generators and modules
</script>
```

Instead, make the script a module:

```
<script type="module">
  await fetch("https://example.com");
</script>
```

Async callbacks

You cannot use `await` in a sync callback:

```
urls.forEach((url) => {
  await fetch(url);
```

```
  // SyntaxError: await is only valid in async functions, async generators and modules
});
```

Instead, make the callback async. See more explanation in the Using promises guide.

```
Promise.all(
  urls.map(async (url) => {
    await fetch(url);
  }),
);
```

See also

- `await`

SyntaxError: unlabeled break must be inside loop or switch

The JavaScript exception "unlabeled break must be inside loop or switch" occurs when a `break` statement is not inside a loop or a `switch` statement.

Message

```
SyntaxError: Illegal break statement (V8-based)
SyntaxError: unlabeled break must be inside loop or switch (Firefox)
SyntaxError: 'break' is only valid inside a switch or loop statement. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

`break` statements can be used to exit a loop or a `switch` statement, and using them elsewhere is a syntax error. Alternatively, you can provide a label to the `break` statement to break out of any statement with that label — however, if the label does not reference a containing statement, another error SyntaxError: label not found will be thrown.

Examples

Unsyntactic break

`break` cannot be used outside `switch` or loops.

```
let score = 0;

function increment() {
  if (score === 100)
    break; // SyntaxError: unlabeled break must be inside loop or switch
  }
  score++;
}
```

Maybe instead of `break`, you intend to use `return` to early-terminate a function.

```
let score = 0;

function increment() {
  if (score === 100) {
    return;
  }
  score++;
}
```

Using break in callbacks

`break` cannot be used in callbacks, even if the callback is called from a loop.

```
let containingIndex = 0;
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

while (containingIndex < matrix.length) {
  matrix[containingIndex].forEach((value) => {
    if (value === 5) {
      break; // SyntaxError: unlabeled break must be inside loop or switch
    }
  });
  containingIndex++;
}
```

Instead, refactor the code so the `break` is used outside the callback.

```
let containingIndex = 0;
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

outer: while (containingIndex < matrix.length) {
  for (const value of matrix[containingIndex]) {
    if (value === 5) {
      break outer;
    }
  }
  containingIndex++;
}
```

```
let containingIndex = 0;
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

while (containingIndex < matrix.length) {
  if (matrix[containingIndex].includes(5)) {
    break;
  }
  containingIndex++;
}
```

There's no way to early-terminate a `forEach()` loop. You can use `some()` instead, or convert it to a `for...of` loop.

```
array.forEach((value) => {
  if (value === 5) {
    break; // SyntaxError: unlabeled break must be inside loop or switch
  }
  // do something with value
});
```

```
array.some((value) => {
  if (value === 5) {
    return true;
  }
  // do something with value
  return false;
});
```

```
for (const value of array) {
  if (value === 5) {
    break;
  }
  // do something with value
}
```

See also

- `break`

SyntaxError: continue must be inside loop

The JavaScript exception "continue must be inside loop" occurs when a `continue` statement is not inside a loop statement.

Message

```
SyntaxError: Illegal continue statement: no surrounding iteration statement (V8-based)
SyntaxError: Illegal continue statement: 'label' does not denote an iteration statement (V8-based)
SyntaxError: continue must be inside loop (Firefox)
SyntaxError: 'continue' is only valid inside a loop statement. (Safari)
SyntaxError: Cannot continue to the label 'label' as it is not targeting a loop. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

`continue` statements can be used to continue a loop, and using them elsewhere is a syntax error. Alternatively, you can provide a label to the `continue` statement to continue any loop with that label — however, if the label does not reference a containing statement, another error SyntaxError: label not found will be thrown, and if the label references a statement that is not a loop, a syntax error is still thrown.

Examples

Using continue in callbacks

If you want to proceed with the next iteration in a `forEach()` loop, use `return` instead, or convert it to a `for...of` loop.

```
array.forEach((value) => {
  if (value === 5) {
    continue; // SyntaxError: continue must be inside loop
  }
  // do something with value
});
```

```
array.forEach((value) => {
  if (value === 5) {
    return;
  }
  // do something with value
});
```

```
for (const value of array) {
  if (value === 5) {
    continue;
  }
  // do something with value
}
```

See also

- `continue`

SyntaxError: new keyword cannot be used with an optional chain

The JavaScript exception "new keyword cannot be used with an optional chain" occurs when the constructor of a `new` expression is an optional chain, or if there's an optional chain between the constructor and the parenthesized list of arguments.

Message

```
SyntaxError: Invalid optional chain from new expression (V8-based)
SyntaxError: new keyword cannot be used with an optional chain (Firefox)
SyntaxError: Cannot call constructor in an optional chain. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There are two ways to get this error. The first one is if the constructor expression is an optional chain expression, like this:

```
new Intl?.DateTimeFormat();
Number?.[parseMethod]`Hello, world!`;
```

The second one is if `?.` occurs between the constructor and the arguments list, like this:

```
new Intl.DateTimeFormat?.();
```

Optional `new` is specifically forbidden because its syntax is complicated (`new` with and without arguments), and the result is unclear (it would be the only case where `new` does not evaluate to an object value). You need to translate the optional chaining to its underlying condition (see optional chaining for more information).

```
const result =
  Intl.DateTimeFormat === null || Intl.DateTimeFormat === undefined
    ? undefined
    : new Intl.DateTimeFormat();
```

Remember that optional chaining only short-circuits within a parenthesized unit. If you parenthesize your constructor expression, the optional chaining will not cause an error, because now the constructor does not short-circuit and the result is clear (the constructor will produce `undefined` and then cause the `new` expression to throw).

```
new (Intl?.DateTimeFormat)(); // Throws if Intl?.DateTimeFormat is undefined
```

However this is a bit nonsensical anyway because optional chaining prevents errors inside the property access chain, but is then guaranteed to generate an error when calling `new`. You would probably still want to use a conditional check.

Note that optional chaining is only forbidden as the constructor expression. You can use optional chaining inside the argument list, or use optional chaining on the `new` expression as a whole.

```
new Intl.DateTimeFormat(navigator?.languages);
new Intl.DateTimeFormat().resolvedOptions?.();
```

Note that there's no needs to use `?.` on the `new` expression itself: `new a()?.b`, because `new` is guaranteed to produce a non-nullish object value.

See also

- `new`
- Optional chaining (`?.`)
- Original discussion on whether "optional new" should be allowed

SyntaxError: tagged template cannot be used with optional chain

The JavaScript exception "tagged template cannot be used with optional chain" occurs when the tag expression of a tagged template literal is an optional chain, or if there's an optional chain between the tag and the template.

Message

```
SyntaxError: Invalid tagged template on optional chain (V8-based)
SyntaxError: tagged template cannot be used with optional chain (Firefox)
SyntaxError: Cannot use tagged templates in an optional chain. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There are two ways to get this error. The first one is if the tag expression is an optional chain expression, like this:

```
String?.raw`Hello, world!`;
console.log?.()`Hello, world!`;
Number?.[parseMethod]`Hello, world!`;
```

The second one is if `?.` occurs between the tag and the template, like this:

```
String.raw?.`Hello, world!`;
```

Optional chaining in the tag is specifically forbidden because there's no great use case for it, and what the result is expected to be is unclear (should it be `undefined` or the template's value as if it's untagged?). You need to translate the optional chaining to its underlying condition (see optional chaining for more information).

```
const result =
  String.raw === null || String.raw === undefined
    ? undefined
    : String.raw`Hello, world!`;
```

Remember that optional chaining only short-circuits within a parenthesized unit. If you parenthesize your tag expression, the optional chaining will not cause an error, because now the tag does not short-circuit and the result is clear (the tag will produce `undefined` and then cause the tagged template to throw).

```
(console?.log)`Hello, world!`; // Throws if console?.log is undefined
```

However this is a bit nonsensical anyway because optional chaining prevents errors inside the property access chain, but is then guaranteed to generate an error when calling the template tag. You would probably still want to use a conditional check.

Note that optional chaining is only forbidden as the tag expression. You can use optional chaining inside the embedded expressions, or use optional chaining on the tagged template expression as a whole.

```
console.log`Hello, ${true.constructor?.name}!`; // ['Hello, ', '!', raw: Array(2)] 'Boolean'
console.log`Hello`?.toString(); // undefined
```

See also

- Template literals
- Optional chaining (`?.`)
- Original discussion on whether optional chaining should be allowed in tagged template literals

RangeError: radix must be an integer

The JavaScript exception "radix must be an integer at least 2 and no greater than 36" occurs when the optional `radix` parameter of the `Number.prototype.toString()` or the `BigInt.prototype.toString()` method was specified and is not between 2 and 36.

Message

```
RangeError: toString() radix argument must be between 2 and 36 (V8-based & Safari)
RangeError: radix must be an integer at least 2 and no greater than 36 (Firefox)
```

Error type

`RangeError`

What went wrong?

The optional `radix` parameter of the `Number.prototype.toString()` or the `BigInt.prototype.toString()` method was specified. Its value must be an integer (a number) between 2 and 36, specifying the base of the number system to be used for representing numeric values. For example, the decimal (base 10) number 169 is represented in hexadecimal (base 16) as A9.

Why is this parameter's value limited to 36? A radix that is larger than 10 uses alphabetical characters as digits; therefore, the radix can't be larger than 36, since the Latin alphabet (used by English and many other languages) only has 26 characters.

The most common radixes:

- 2 for binary numbers,
- 8 for octal numbers,
- 10 for decimal numbers,
- 16 for hexadecimal numbers.

Examples

Invalid cases

```
(42).toString(0);
(42).toString(1);
(42).toString(37);
(42).toString(150);
// You cannot use a string like this for formatting:
(12071989).toString("MM-dd-yyyy");
```

Valid cases

```
(42).toString(2); // "101010" (binary)
(13).toString(8); // "15" (octal)
(0x42).toString(10); // "66" (decimal)
(100000).toString(16); // "186a0" (hexadecimal)
```

See also

- `Number.prototype.toString()`
- `BigInt.prototype.toString()`

SyntaxError: invalid regular expression flag "x"

The JavaScript exception "invalid regular expression flag" occurs when the flags in a regular expression contain any flag that is not one of: `d`, `g`, `i`, `m`, `s`, `u`, `v`, or `y`. It may also be raised if the expression contains more than one instance of a valid flag, or when the `u` and `v` flags are used together.

Message

```
SyntaxError: Invalid flags supplied to RegExp constructor 'x' (V8-based)
SyntaxError: Invalid regular expression flags (V8-based)
SyntaxError: invalid regular expression flag x (Firefox)
SyntaxError: Invalid flags supplied to RegExp constructor. (Safari)
SyntaxError: Invalid regular expression: invalid flags (Safari)
```

Error type

`SyntaxError`

What went wrong?

The regular expression contains invalid flags, or valid flags have been used more than once in the expression.

The valid (allowed) flags are `d`, `g`, `i`, `m`, `s`, `u`, `v`, and `y`. They are introduced in more detail in Regular expressions > Advanced searching with flags.

The `u` and `v` flags are mutually exclusive, so they cannot be used together. You can read the references to understand what their behavior differences are.

Examples

In a regular expression literal, which consists of a pattern enclosed between slashes, the flags are defined after the second slash. Regular expression flags can be used separately or together in any order. This syntax shows how to declare the flags using the regular expression literal:

```
const re = /pattern/flags;
```

They can also be defined in the constructor function of the `RegExp` object (second parameter):

```
const re = new RegExp("pattern", "flags");
```

Here is an example showing use of only correct flags.

```
/foo/g;
/foo/gims;
/foo/uy;
```

Below is an example showing the use of some invalid flags `b`, `a` and `r`:

```
/foo/bar;

// SyntaxError: invalid regular expression flag "b"
```

The code below is incorrect, because `W`, `e` and `b` are not valid flags.

```
const obj = {
  url: /docs/Web,
};

// SyntaxError: invalid regular expression flag "W"
```

An expression containing two slashes is interpreted as a regular expression literal. Most likely the intent was to create a string literal, using single or double quotes as shown below:

```
const obj = {
  url: "/docs/Web",
};
```

See also

- Regular expressions guide

SyntaxError: return not in function

The JavaScript exception "return not in function" occurs when a `return` statement is called outside of a function.

Message

```
SyntaxError: Illegal return statement (V8-based)
SyntaxError: return not in function (Firefox)
SyntaxError: Return statements are only valid inside functions. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

A `return` statement is called outside of a function. Maybe there are missing curly braces somewhere? The `return` statement must be in a function, because it ends function execution and specifies a value to be returned to the function caller.

Examples

Missing curly braces

```
function cheer(score) {
  if (score === 147)
    return "Maximum!";
  }
  if (score > 100) {
    return "Century!";
  }
}

// SyntaxError: return not in function
```

The curly braces look correct at a first glance, but this code snippet is missing a `{` after the first `if` statement. Correct would be:

```
function cheer(score) {
  if (score === 147) {
    return "Maximum!";
  }
  if (score > 100) {
    return "Century!";
  }
}
```

See also

- `return`

SyntaxError: 'arguments'/'eval' can't be defined or assigned to in strict mode code

The JavaScript strict mode-only exception "'arguments' can't be defined or assigned to in strict mode code" or "'eval' can't be defined or assigned to in strict mode code" occurs when attempting to create a binding called `arguments` or `eval`, or assign to such a name.

Message

```
SyntaxError: Unexpected eval or arguments in strict mode (V8-based)
SyntaxError: 'arguments' can't be defined or assigned to in strict mode code (Firefox)
SyntaxError: Cannot modify 'arguments' in strict mode. (Safari)
SyntaxError: Cannot destructure to a parameter name 'arguments' in strict mode. (Safari)
SyntaxError: Cannot declare a variable named arguments in strict mode. (Safari)
SyntaxError: Cannot declare a catch variable named 'arguments' in strict mode. (Safari)
SyntaxError: 'arguments' is not a valid function name in strict mode. (Safari)
```

Error type

`SyntaxError`

What went wrong?

In strict mode, the names `arguments` and `eval` behave as if they are reserved words: you cannot make they refer to anything other than the `arguments` object in functions or the global `eval` function.

Examples

Invalid cases

```
"use strict";

const arguments = [1, 2, 3];
console.log(Math.max(...arguments));

function foo(...arguments) {
  console.log(arguments);
}
```

Valid cases

```
"use strict";

const args = [1, 2, 3];
console.log(Math.max(...args));

function foo(...args) {
  console.log(args);
}
```

See also

- Strict mode
- `arguments`
- `eval()`

SyntaxError: super() is only valid in derived class constructors

The JavaScript exception "super() is only valid in derived class constructors" occurs when the `super()` call is used somewhere that's not the body of a constructor in a class with `extends` keyword.

Message

```
SyntaxError: 'super' keyword unexpected here (V8-based)
SyntaxError: super() is only valid in derived class constructors (Firefox)
SyntaxError: super is not valid in this context. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `super()` call is used to invoke the base constructor of a derived class, so the base class can initialize the `this` object. Using it anywhere else doesn't make sense.

`super()` can also be defined in an arrow function that's nested within the constructor. However, it cannot be defined in any other kind of function.

Examples

Invalid cases

You cannot call `super()` if the class has no `extends`, because there's no base class to call:

```
class Base {
  constructor() {
    super();
  }
}
```

You cannot call `super()` in a class method, even if that method is called from the constructor:

```
class Base {}

class Derived extends Base {
  constructor() {
    this.init();
  }

  init() {
    super();
```

```
  }
}
```

You cannot call `super()` in a function, even if the function is used as a constructor:

```
function Base(x) {
  this.x = x;
}

function Derived() {
  super(1);
}

Object.setPrototypeOf(Derived.prototype, Base.prototype);
Object.setPrototypeOf(Derived, Base);
```

Valid cases

You can call `super()` before calling any other method in the constructor:

```
class Base {}

class Derived extends Base {
  constructor() {
    super();
    this.init();
  }

  init() {
    // …
  }
}
```

You can call `super()` in an arrow function that's nested within the constructor:

```
class Base {}

class Derived extends Base {
  constructor() {
    const init = () => {
      super();
    };

    init();
  }
}
```

See also

- Classes
- `super`

SyntaxError: use of super property/member accesses only valid within methods or eval code within methods

The JavaScript exception "use of super property/member accesses only valid within methods or eval code within methods" occurs when the `super.x` or `super[x]` syntax is used outside of a method.

Message

```
SyntaxError: 'super' keyword unexpected here (V8-based)
SyntaxError: use of super property accesses only valid within methods or eval code within methods (Firefox)
SyntaxError: super is not valid in this context. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `super.x` syntax is used to access properties on the prototype of the current object. It can be used in methods of both object literals and classes, field initializers, and static initialization blocks, but not in other contexts.

Examples

Invalid cases

You can't use `super.x` outside of a method in an object:

```
const obj = {
  __proto__: { x: 1 },
  x: super.x, // SyntaxError: use of super property accesses only valid within methods or eval code within methods
};
```

You can't use `super.x` in a function, even if that function has the effect of being a method:

```
function getX() {
  return super.x; // SyntaxError: use of super property accesses only valid within methods or eval code within methods
}

const obj = {
  getX,
  getX2: function () {
    return super.x; // SyntaxError: use of super property accesses only valid within methods or eval code within methods
  },
};

class Derived extends Base {
  getX = () => super.x;
}
```

Valid cases

You can use `super.x` in a method:

```
class Base {
  x = 1;
}

class Derived extends Base {
  getX() {
    return super.x;
  }
}
```

You can use `super.x` in a field initializer:

```
class Derived extends Base {
  x = super.x;
}
```

You can use `super.x` in object methods too:

```
const obj = {
  __proto__: { x: 1 },
  getX() {
    return super.x;
  },
};
```

See also

- Classes
- `super`

RangeError: BigInt division by zero

The JavaScript exception "BigInt division by zero" occurs when a `BigInt` is divided by `0n`.

Message

```
RangeError: Division by zero (V8-based)
RangeError: BigInt division by zero (Firefox)
RangeError: 0 is an invalid divisor value. (Safari)
```

Error type

`RangeError`.

What went wrong?

The divisor of a division or remainder operator is `0n`. In `Number` arithmetic, this produces `Infinity`, but there's no "infinity value" in BigInts, so an error is issued. Check if the divisor is `0n` before doing the division.

Examples

Division by 0n

```
const a = 1n;
const b = 0n;
const quotient = a / b;
// RangeError: BigInt division by zero
```

Instead, check if the divisor is `0n` first, and either issue an error with a better message, or fall back to a different value, like `Infinity` or `undefined`.

```
const a = 1n;
const b = 0n;
const quotient = b === 0n ? undefined : a / b;
```

See also

- `BigInt`
- Division (`/`)
- Remainder (`%`)

RangeError: BigInt negative exponent

The JavaScript exception "BigInt negative exponent" occurs when a `BigInt` is raised to the power of a negative BigInt value.

Message

```
RangeError: Exponent must be positive (V8-based)
RangeError: BigInt negative exponent (Firefox)
RangeError: Negative exponent is not allowed (Safari)
```

Error type

`RangeError`.

What went wrong?

The exponent of an exponentiation operation must be positive. Since negative exponents would take the reciprocal of the base, the result will be between -1 and 1 in almost all cases, which gets rounded to `0n`. To catch mistakes, negative exponents are not allowed. Check if the exponent is non-negative before doing exponentiation.

Examples

Using a negative BigInt as exponent

```
const a = 1n;
const b = -1n;
const c = a ** b;
// RangeError: BigInt negative exponent
```

Instead, check if the exponent is negative first, and either issue an error with a better message, or fall back to a different value, like `0n` or `undefined`.

```
const a = 1n;
const b = -1n;
const quotient = b >= 0n ? a ** b : 0n;
```

See also

- `BigInt`
- Exponentiation (`**`)

TypeError: BigInt value can't be serialized in JSON

The JavaScript exception "BigInt value can't be serialized in JSON" occurs when a `BigInt` is encountered in `JSON.stringify` with no custom serialization method provided.

Message

```
TypeError: Do not know how to serialize a BigInt (V8-based)
TypeError: BigInt value can't be serialized in JSON (Firefox)
TypeError: JSON.stringify cannot serialize BigInt. (Safari)
```

Error type

`TypeError`

What went wrong?

You are trying to serialize a BigInt value using `JSON.stringify`, which does not support BigInt values by default. Sometimes, JSON stringification happens implicitly in libraries, as part of data serialization. For example, sending data to the server, storing it in external storage, or transferring it between threads would all require serialization, which is often done using JSON.

There are several ways to handle this:

- If you can alter the data source, avoid using BigInt values and cast it to a number first (which may lose precision for large numbers).
- If you can alter the stringification process, pass a replacer function to `JSON.stringify` that converts BigInt values to strings or numbers.
- You can also provide a `BigInt.prototype.toJSON` method globally that gets called whenever a BigInt value is stringified.

For more information on various tradeoffs, see BigInt reference.

Examples

Providing a custom serialization method

By default, BigInt values are not serializable in JSON:

```
const data = { a: 1n };
JSON.stringify(data);
// TypeError: BigInt value can't be serialized in JSON
```

Assuming you intend for the JSON to contain a number value, here are a few approaches that work:

- Convert the BigInt to a number before stringifying:

  ```
  const data = { a: 1n };
  JSON.stringify({ ...data, a: Number(data.a) });
  // '{"a":1}'
  ```

- Provide a replacer function that converts BigInt values to numbers or raw JSON objects:

  ```
  const data = { a: 1n };
  JSON.stringify(data, (key, value) =>
    typeof value === "bigint" ? Number(value) : value,
  );
  // '{"a":1}'
  ```

  ```
  const data = { a: 1n };
  JSON.stringify(data, (key, value) =>
    typeof value === "bigint" ? JSON.rawJSON(value.toString()) : value,
  );
  // '{"a":1}'
  ```

- Provide a `BigInt.prototype.toJSON` method that gets called whenever a BigInt value is stringified:

  ```
  BigInt.prototype.toJSON = function () {
    return Number(this);
  };
  const data = { a: 1n };
  JSON.stringify(data);
  // '{"a":1}'
  ```

  ```
  BigInt.prototype.toJSON = function () {
    return JSON.rawJSON(this.toString());
  };
  const data = { a: 1n };
  JSON.stringify(data);
  // '{"a":1}'
  ```

See also

- `BigInt`
- `JSON.stringify()`
- `JSON.rawJSON()`

TypeError: calling a builtin X constructor without new is forbidden

The JavaScript exception "calling a builtin X constructor without new is forbidden" occurs when you try to call a builtin constructor without using the `new` keyword. All modern constructors, such as `Promise` and `Map`, must be called with `new`.

Message

```
TypeError: Constructor X requires 'new' (V8-based)
TypeError: Promise constructor cannot be invoked without 'new' (V8-based)
TypeError: calling a builtin X constructor without new is forbidden (Firefox)
TypeError: calling X constructor without new is invalid (Safari)
```

Error type

`TypeError`

What went wrong?

In JavaScript, *calling* a function without `new` and *constructing* a function with `new` are two distinct operations, and functions can behave differently depending on how they are called.

Apart from the following legacy constructors, all modern constructors must be called with `new`:

- `Object()`
- `Function()` (and its subclasses)
- `Error()` (and its subclasses)
- `RegExp()`
- `Array()`

Some other constructors, such as `Date()`, and primitive wrappers, such as `String()`, `Number()`, and `Boolean()`, can also be called with or without `new`, but the return types differ in the two cases.

On every constructor page, you can find information about whether the constructor must be called with `new`.

Examples

Invalid cases

```
const m = Map(); // TypeError: calling a builtin Map constructor without new is forbidden
```

Valid cases

```
const m = new Map();
```

See also

- `new`

TypeError: X.prototype.y called on incompatible type

The JavaScript exception "called on incompatible target (or object)" occurs when a function (on a given object), is called with a `this` not corresponding to the type expected by the function.

Message

```
TypeError: Method Set.prototype.add called on incompatible receiver undefined (V8-based)
TypeError: Bind must be called on a function (V8-based)
TypeError: Illegal invocation (V8-based)
TypeError: Function.prototype.toString requires that 'this' be a Function (V8-based)
TypeError: this is not a Date object. (V8-based)
TypeError: this is not a typed array. (V8-based)
TypeError: Function.prototype.toString called on incompatible object (Firefox)
TypeError: Function.prototype.bind called on incompatible target (Firefox)
TypeError: 'addEventListener' called on an object that does not implement interface EventTarget. (Firefox)
TypeError: Type error (Safari)
TypeError: undefined is not an object (Safari)
```

Error type

`TypeError`

What went wrong?

When this error is thrown, a function (on a given object), is called with a `this` not corresponding to the type expected by the function.

This issue can arise when using the `Function.prototype.call()` or `Function.prototype.apply()` methods, and providing a `this` argument which does not have the expected type.

This issue can also happen when providing a function that is stored as a property of an object as an argument to another function. In this case, the object that stores the function won't be the `this` target of that function when it is called by the other function. To work-around this issue, you will either need to wrap the callback function in another function, or use the `Function.prototype.bind()` method to force the `this` argument to the expected object.

Examples

Invalid cases

```
const mySet = new Set();
["bar", "baz"].forEach(mySet.add);
// mySet.add is a function, but "mySet" is not captured as this.

function myFun() {
  console.log(this);
}
["bar", "baz"].forEach(myFun.bind);
// myFun.bind is a function, but "myFun" is not captured as this.
```

Valid cases

```
const mySet = new Set();
["bar", "baz"].forEach(mySet.add.bind(mySet));
// This works due to binding "mySet" as this.

function myFun() {
  console.log(this);
}
["bar", "baz"].forEach((x) => myFun.bind(x));
// This works using the "bind" function. It creates a new function forwarding the argument.
```

See also

- `Function.prototype.call()`
- `Function.prototype.apply()`
- `Function.prototype.bind()`

ReferenceError: can't access lexical declaration 'X' before initialization

The JavaScript exception "can't access lexical declaration 'X' before initialization" occurs when a lexical variable was accessed before it was initialized. This happens within any scope (global, module, function, or block) when `let` or `const` variables are accessed before the place where they are declared is executed.

Message

```
ReferenceError: Cannot access 'X' before initialization (V8-based)
ReferenceError: can't access lexical declaration 'X' before initialization (Firefox)
ReferenceError: Cannot access uninitialized variable. (Safari)
```

Error type

`ReferenceError`

What went wrong?

A lexical variable was accessed before it was initialized. This happens within any scope (global, module, function, or block) when variables declared with `let` or `const` are accessed before the place where they are declared has been executed.

Note that it is the execution order of access and variable declaration that matters, not the order in which the statements appear in the code. For more information, see the description of Temporal Dead Zone.

This issue does not occur for variables declared using `var`, because they are initialized with a default value of `undefined` when they are hoisted.

This error can also occur in cyclic imports when a module uses a variable that depends on the module itself being evaluated.

Examples

Invalid cases

In this case, the variable `foo` is accessed before it is declared. At this point foo has not been initialized with a value, so accessing the variable throws a reference error.

```
function test() {
  // Accessing the 'const' variable foo before it's declared
  console.log(foo); // ReferenceError: foo is not initialized
  const foo = 33; // 'foo' is declared and initialized here using the 'const' keyword
}

test();
```

In this example, the imported variable `a` is accessed but is uninitialized, because the evaluation of `a.js` is blocked by the evaluation of the current module `b.js`.

```
// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;

// -- b.js --
import { a } from "./a.js";

console.log(a); // ReferenceError: Cannot access 'a' before initialization
export const b = 1;
```

Valid cases

In the following example, we correctly declare a variable using the `const` keyword before accessing it.

```
function test() {
  // Declaring variable foo
  const foo = 33;
  console.log(foo); // 33
}
test();
```

In this example, the imported variable `a` is asynchronously accessed, so both modules are evaluated before the access to `a` occurs.

```
// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;

// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);
export const b = 1;
```

See also

- `let`

- `const`
- `var`
- `class`

TypeError: can't assign to property "x" on "y": not an object

The JavaScript strict mode exception "can't assign to property" occurs when attempting to create a property on primitive value such as a symbol, a string, a number or a boolean. Primitive values cannot hold any property.

Message

```
TypeError: Cannot create property 'x' on number '1' (V8-based)
TypeError: can't assign to property "x" on 1: not an object (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError`.

What went wrong?

In strict mode, a `TypeError` is raised when attempting to create a property on primitive value such as a symbol, a string, a number or a boolean. Primitive values cannot hold any property.

The problem might be that an unexpected value is flowing at an unexpected place, or that an object variant of a `String` or a `Number` is expected.

Examples

Invalid cases

```
"use strict";

const foo = "my string";
// The following line does nothing if not in strict mode.
foo.bar = {}; // TypeError: can't assign to property "bar" on "my string": not an object
```

Fixing the issue

Either fix the code to prevent the primitive from being used in such places, or fix the issue by creating the object equivalent `Object`.

```
"use strict";

const foo = new String("my string");
foo.bar = {};
```

See also

- Strict mode
- primitive

RangeError: x can't be converted to BigInt because it isn't an integer

The JavaScript exception "x can't be converted to BigInt because it isn't an integer" occurs when the `BigInt()` function is used on a number that isn't an integer.

Message

```
RangeError: The number 1.5 cannot be converted to a BigInt because it is not an integer (V8-based & Firefox)
RangeError: Not an integer (Safari)
```

Error type

`RangeError`.

What went wrong?

When using the `BigInt()` function to convert a number to a BigInt, the number must be an integer (such that `Number.isInteger` returns true).

Examples

Invalid cases

```
const a = BigInt(1.5);
// RangeError: The number 1.5 cannot be converted to a BigInt because it is not an integer
const b = BigInt(NaN);
// RangeError: NaN cannot be converted to a BigInt because it is not an integer
```

Valid cases

```
const a = BigInt(1);
```

See also

- `BigInt()` constructor
- `Number.isInteger()`

TypeError: can't convert BigInt to number

The JavaScript exception "can't convert BigInt to number" occurs when an arithmetic operation involves a mix of `BigInt` and `Number` values.

Message

```
TypeError: Cannot convert a BigInt value to a number (V8-based)
TypeError: Cannot mix BigInt and other types, use explicit conversions (V8-based)
TypeError: BigInts have no unsigned right shift, use >> instead (V8-based)
TypeError: can't convert BigInt to number (Firefox)
TypeError: Conversion from 'BigInt' to 'number' is not allowed. (Safari)
TypeError: Invalid mix of BigInt and other type in addition/multiplication/…. (Safari)
TypeError: BigInt does not support >>> operator (Safari)
```

Error type

`TypeError`.

What went wrong?

The two sides of an arithmetic operator must both be BigInts or both not. If an operation involves a mix of BigInts and numbers, it's ambiguous whether the result should be a BigInt or number, since there may be loss of precision in both cases.

The error also happens when a BigInt is implicitly converted to a number via the number coercion process. For example, if a BigInt is passed to a built-in method that expects a number.

The error can also happen if the unsigned right shift operator (`>>>`) is used between two BigInts. In Firefox, the message is the same: "can't convert BigInt to number".

Examples

Mixing numbers and BigInts in operations

```
const sum = 1n + 1;
// TypeError: can't convert BigInt to number
```

Instead, explicitly coerce one side to a BigInt or number.

```
const sum = 1n + BigInt(1);
const sum2 = Number(1n) + 1;
```

Using unsigned right shift on BigInts

```
const a = 4n >>> 2n;
// TypeError: can't convert BigInt to number
```

Use normal right shift instead.

```
const a = 4n >> 2n;
```

See also

- `BigInt`

TypeError: can't convert x to BigInt

The JavaScript exception "x can't be converted to BigInt" occurs when attempting to convert a `Symbol`, `null`, or `undefined` value to a `BigInt`, or if an operation expecting a BigInt parameter receives a number.

Message

```
TypeError: Cannot convert null to a BigInt (V8-based)
TypeError: can't convert null to BigInt (Firefox)
TypeError: Invalid argument type in ToBigInt operation (Safari)
```

Error type

`TypeError`.

What went wrong?

When using the `BigInt()` function to convert a value to a BigInt, the value would first be converted to a primitive. Then, if it's not one of BigInt, string, number, and boolean, the error is thrown.

Some operations, like `BigInt.asIntN`, require the parameter to be a BigInt. Passing in a number in this case will also throw this error.

Examples

Using BigInt() on invalid values

```
const a = BigInt(null);
// TypeError: can't convert null to BigInt
const b = BigInt(undefined);
// TypeError: can't convert undefined to BigInt
const c = BigInt(Symbol("1"));
// TypeError: can't convert Symbol("1") to BigInt
```

```
const a = BigInt(1);
const b = BigInt(true);
const c = BigInt("1");
const d = BigInt(Symbol("1").description);
```

> **Note:** Simply coercing the value to a string or number using `String()` or `Number()` before passing it to `BigInt()` is usually not sufficient to avoid all errors. If the string is not a valid integer number string, a SyntaxError is thrown; if the number is not an integer (most notably, `NaN`), a RangeError is thrown. If the range of input is unknown, properly validate it before using `BigInt()`.

Passing a number to a function expecting a BigInt

```
const a = BigInt.asIntN(4, 8);
// TypeError: can't convert 8 to BigInt
const b = new BigInt64Array(3).fill(3);
// TypeError: can't convert 3 to BigInt
```

```
const a = BigInt.asIntN(4, 8n);
const b = new BigInt64Array(3).fill(3n);
```

See also

- `BigInt()` constructor

TypeError: can't define property "x": "obj" is not extensible

The JavaScript exception "can't define property "x": "obj" is not extensible" occurs when an object is marked as non-extensible, so that it will never have properties beyond the ones it had at the time it was marked as non-extensible. Objects can be made non-extensible by calling `Object.preventExtensions()`, `Object.seal()`, or `Object.freeze()`.

Message

```
TypeError: Cannot add property x, object is not extensible (V8-based)
TypeError: Cannot define property x, object is not extensible (V8-based)
TypeError: can't define property "x": Object is not extensible (Firefox)
TypeError: Attempting to define property on object that is not extensible. (Safari)
```

Error type

`TypeError`

What went wrong?

Usually, an object is extensible and new properties can be added to it. However, in this case the object isn't extensible, so that it will never have properties beyond the ones it had at the time it was marked as non-extensible. You could have marked the object as non-extensible by calling `Object.preventExtensions()`, `Object.seal()`, or `Object.freeze()`, or a library you are using could have done that for you.

Examples

Adding new properties to a non-extensible objects

In strict mode, attempting to add new properties to a non-extensible object via assignment throws a `TypeError`. In sloppy mode, the addition of the "x" property is silently ignored.

```
"use strict";

const obj = {};
Object.preventExtensions(obj);

obj.x = "foo";
// TypeError: can't define property "x": Object is not extensible
```

In both strict mode and sloppy mode, a call to `Object.defineProperty()` throws when adding a new property to a non-extensible object.

```
const obj = {};
Object.preventExtensions(obj);

Object.defineProperty(obj, "x", { value: "foo" });
// TypeError: can't define property "x": Object is not extensible
```

Non-extensible objects created by other means

The `Object.seal()` and `Object.freeze()` methods also create non-extensible objects—they just have additional restrictions on modifying existing properties as well.

```
"use strict";

const obj = { y: "bar" };
Object.seal(obj);
obj.x = "foo";
// TypeError: can't define property "x": Object is not extensible
```

Fixing the error

There are three ways to fix this error: you can remove the property addition entirely if you don't need it, you can copy the existing properties to a new extensible object, or you can add the property before making the object non-extensible.

```
"use strict";

const obj = {};
obj.x = "foo"; // add property first and only then prevent extensions

Object.preventExtensions(obj);
```

See also

- `Object.preventExtensions()`
- `Object.seal()`
- `Object.freeze()`

TypeError: property "x" is non-configurable and can't be deleted

The JavaScript exception "property is non-configurable and can't be deleted" occurs when it was attempted to delete a property, but that property is non-configurable.

Message

```
TypeError: Cannot delete property 'x' of #<Object> (V8-based)
TypeError: property "x" is non-configurable and can't be deleted (Firefox)
TypeError: Unable to delete property. (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

It was attempted to delete a property, but that property is non-configurable. The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `writable`) can be changed.

This error happens only in strict mode code. In non-strict code, the operation returns `false`.

Examples

Attempting to delete non-configurable properties

Non-configurable properties are not super common, but they can be created using `Object.defineProperty()` or `Object.freeze()`.

```
"use strict";
const obj = Object.freeze({ name: "Elsa", score: 157 });
delete obj.score; // TypeError
```

```
"use strict";
const obj = {};
Object.defineProperty(obj, "foo", { value: 2, configurable: false });
delete obj.foo; // TypeError
```

```
"use strict";
const frozenArray = Object.freeze([0, 1, 2]);
frozenArray.pop(); // TypeError
```

There are also a few non-configurable properties built into JavaScript. Maybe you tried to delete a mathematical constant.

```
"use strict";
delete Math.PI; // TypeError
```

See also

- `delete`
- `Object.defineProperty()`
- `Object.freeze()`

SyntaxError: private fields can't be deleted

The JavaScript exception "SyntaxError: private fields can't be deleted" occurs when `delete` is used on a private element of a class or an object.

Message

```
SyntaxError: Private fields can not be deleted (V8-based)
SyntaxError: private fields can't be deleted (Firefox)
SyntaxError: Cannot delete private field X (Safari)
```

Error type

`SyntaxError`

What went wrong?

There's code trying to `delete` a private element (field or method) of an object or a class. This is forbidden by JavaScript—private elements cannot be added or removed on the fly.

Examples

```
class MyClass {
  #myPrivateField;
  deleteIt() {
    delete this.#myPrivateField; // SyntaxError: private fields can't be deleted
  }
}
```

```
class MyClass {
  #myPrivateMethod() {
  }
  #deleteIt() {
    delete this.#myPrivateMethod; // SyntaxError: private fields can't be deleted
  }
}
```

See also

- Classes
- Private elements

TypeError: can't redefine non-configurable property "x"

The JavaScript exception "can't redefine non-configurable property" occurs when it was attempted to redefine a property, but that property is non-configurable.

Message

```
TypeError: Cannot redefine property: "x" (V8-based)
TypeError: can't redefine non-configurable property "x" (Firefox)
TypeError: Attempting to change value of a readonly property. (Safari)
```

Error type

`TypeError`

What went wrong?

It was attempted to redefine a property, but that property is non-configurable. The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `writable`) can be changed. Usually, properties in an object created by an object initializer are configurable. However, for example, when using `Object.defineProperty()`, the property isn't configurable by default.

Examples

Non-configurable properties created by Object.defineProperty

The `Object.defineProperty()` creates non-configurable properties if you haven't specified them as configurable.

```
const obj = Object.create({});
Object.defineProperty(obj, "foo", { value: "bar" });

Object.defineProperty(obj, "foo", { value: "baz" });
// TypeError: can't redefine non-configurable property "foo"
```

You will need to set the "foo" property to configurable, if you intend to redefine it later in the code.

```
const obj = Object.create({});
Object.defineProperty(obj, "foo", { value: "bar", configurable: true });
Object.defineProperty(obj, "foo", { value: "baz", configurable: true });
```

See also

- [[Configurable]]
- `Object.defineProperty()`

TypeError: can't set prototype of this object

The JavaScript exception "can't set prototype of this object" occurs when attempting to set the prototype of an object, but the object's prototype is frozen, either by being a built-in immutable prototype object, or by being non-extensible.

Message

```
TypeError: Immutable prototype object 'Object.prototype' cannot have their prototype set (V8-based)
TypeError: #<Object> is not extensible (V8-based)
TypeError: can't set prototype of this object (Firefox)
TypeError: Cannot set prototype of immutable prototype object (Safari)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError`

What went wrong?

You are using one of the prototype-mutating methods—most notably, `Object.setPrototypeOf()`—on an object whose prototype is immutable. Some built-in objects have immutable prototypes, such as `Object.prototype` and `window`, for security reasons. User objects can also prevent prototype changes by using `Object.preventExtensions()`, `Object.seal()`, or `Object.freeze()`.

Examples

Changing the prototype of a built-in object

A selected few built-in objects have immutable prototypes. For example, you cannot change the prototype of `Object.prototype`:

```
Object.setPrototypeOf(Object.prototype, {});
```

This prevents you from being able to arbitrarily change the behavior of all objects in the system. The prototype of `Object.prototype` is always `null`. However, other built-in prototype objects, such as `Function.prototype` and `Array.prototype`, are not protected by default in this regard.

Changing the prototype of a non-extensible object

If you make an object non-extensible, you cannot change its prototype either:

```
const obj = {};
Object.preventExtensions(obj);
Object.setPrototypeOf(obj, {});
// TypeError: can't set prototype of this object
```

See also

- `Object.setPrototypeOf()`
- `Object.preventExtensions()`

SyntaxError: cannot use ?? unparenthesized within || and && expressions

The JavaScript exception "cannot use `??` unparenthesized within `||` and `&&` expressions" occurs when a nullish coalescing operator is used with a logical OR or logical AND in the same expression without parentheses.

Message

```
SyntaxError: Unexpected token '??' (V8-based)
SyntaxError: cannot use `??` unparenthesized within `||` and `&&` expressions (Firefox)
SyntaxError: Unexpected token '??'. Coalescing and logical operators used together in the same expression; parentheses must be used to disambiguate.
(Safari)
```

Error type

`SyntaxError`

What went wrong?

The operator precedence chain looks like this:

```
|   >   &&   >   ||   >   =
|   >   ??   >   =
```

However, the precedence *between* `??` and `&&`/`||` is intentionally undefined, because the short circuiting behavior of logical operators can make the expression's evaluation counter-intuitive. Therefore, the following combinations are all syntax errors, because the language doesn't know how to parenthesize the operands:

```
a ?? b || c;
a || b ?? c;
a ?? b && c;
a && b ?? c;
```

Instead, make your intent clear by parenthesizing either side explicitly:

```
(a ?? b) || c;
a ?? (b && c);
```

Examples

When migrating legacy code that uses `||` and `&&` for guarding against `null` or `undefined`, you may often convert it partially:

```
function getId(user, fallback) {
  // Previously: user && user.id || fallback
  return user && user.id ?? fallback; // SyntaxError: cannot use `??` unparenthesized within `||` and `&&` expressions
}
```

Instead, consider parenthesizing the `&&`:

```
function getId(user, fallback) {
  return (user && user.id) ?? fallback;
}
```

Even better, consider using optional chaining instead of `&&`:

```
function getId(user, fallback) {
  return user?.id ?? fallback;
}
```

See also

- Issue about nullish coalescing precedence in the TC39 nullish-coalescing proposal
- Nullish coalescing operator (`??`)
- Operator precedence

TypeError: class constructors must be invoked with 'new'

The JavaScript exception "class constructors must be invoked with 'new'" occurs when a class constructor is called without the `new` keyword. All class constructors must be called with `new`.

Message

```
TypeError: Class constructor X cannot be invoked without 'new' (V8-based)
TypeError: Class constructors cannot be invoked without 'new' (V8-based)
TypeError: class constructors must be invoked with 'new' (Firefox)
TypeError: Cannot call a class constructor without |new| (Safari)
```

Error type

`TypeError`

What went wrong?

In JavaScript, *calling* a function without `new` and *constructing* a function with `new` are two distinct operations, and functions can behave differently depending on how they are called.

Traditionally, JavaScript functions have been used as both constructors and normal functions, and can detect how they were called using `new.target`. However, class constructors are always constructors and cannot be called as normal functions.

Examples

Invalid cases

```
class X {}

X(); // TypeError: class constructors must be invoked with 'new'
```

Valid cases

```
class X {}

new X();
```

See also

- Classes
- `new`

TypeError: Iterator/AsyncIterator constructor can't be used directly

The JavaScript exception "Iterator constructor can't be used directly" or "AsyncIterator constructor can't be used directly" occurs when you try to use the `Iterator()` or `AsyncIterator` constructors directly to create instances. These constructors are *abstract classes* and should only be inherited from.

Message

```
TypeError: Abstract class Iterator not directly constructable (V8-based)
TypeError: Iterator constructor can't be used directly (Firefox)
TypeError: Iterator cannot be constructed directly (Safari)
```

```
TypeError: Abstract class AsyncIterator not directly constructable (V8-based)
TypeError: AsyncIterator constructor can't be used directly (Firefox)
TypeError: AsyncIterator cannot be constructed directly (Safari)
```

Error type

`TypeError`

What went wrong?

The `Iterator` and `AsyncIterator` constructors are abstract classes and should not be used directly. They check the value of `new.target` and throw if it is the same as the constructor itself. The only way to use these constructors is to inherit from them in a subclass and call `super()` in the subclass constructor. The subclass must also define a `next()` method to be useful.

Examples

Invalid cases

```
new Iterator();
```

Valid cases

```
class MyIterator extends Iterator {
  #step;
  #end;
  constructor(start, end) {
    // Implicitly calls new Iterator(), but with a different `new.target`
    super();
    this.#step = start;
    this.#end = end;
  }
  next() {
    if (this.#step >= this.#end) {
      return { done: true };
    }
    return { value: this.#step++, done: false };
  }
}

new MyIterator();
```

See also

- `AsyncIterator`
- `Iterator`

TypeError: cyclic object value

The JavaScript exception "cyclic object value" occurs when object references were found in JSON. `JSON.stringify()` doesn't try to solve them and fails accordingly.

Message

```
TypeError: Converting circular structure to JSON (V8-based)
TypeError: cyclic object value (Firefox)
TypeError: JSON.stringify cannot serialize cyclic structures. (Safari)
```

Error type

`TypeError`

What went wrong?

The JSON format per se doesn't support object references (although an IETF draft exists), hence `JSON.stringify()` doesn't try to solve them and fails accordingly.

Examples

Circular references

In a circular structure like the following:

```
const circularReference = { otherData: 123 };
circularReference.myself = circularReference;
```

`JSON.stringify()` will fail

```
JSON.stringify(circularReference);
// TypeError: cyclic object value
```

To serialize circular references you can use a library that supports them (e.g., cycle.js) or implement a solution by yourself, which will require finding and replacing (or removing) the cyclic references by serializable values.

The snippet below illustrates how to find and filter (thus causing data loss) a cyclic reference by using the `replacer` parameter of `JSON.stringify()`:

```
function getCircularReplacer() {
  const ancestors = [];
  return function (key, value) {
    if (typeof value !== "object" || value === null) {
      return value;
    }
    // `this` is the object that value is contained in,
    // i.e., its direct parent.
    while (ancestors.length > 0 && ancestors.at(-1) !== this) {
      ancestors.pop();
    }
    if (ancestors.includes(value)) {
      return "[Circular]";
    }
    ancestors.push(value);
    return value;
  };
}

JSON.stringify(circularReference, getCircularReplacer());
// {"otherData":123,"myself":"[Circular]"}

const o = {};
const notCircularReference = [o, o];
JSON.stringify(notCircularReference, getCircularReplacer());
// [{},{}]
```

See also

- `JSON.stringify()`
- cycle.js on GitHub

TypeError: can't set prototype: it would cause a prototype chain cycle

The JavaScript exception "TypeError: can't set prototype: it would cause a prototype chain cycle" occurs when an object's prototype is set to an object such that the prototype chain becomes circular (`a` and `b` both have each other in their prototype chains).

Message

```
TypeError: Cyclic __proto__ value (V8-based)
TypeError: can't set prototype: it would cause a prototype chain cycle (Firefox)
TypeError: cyclic __proto__ value (Safari)
```

Error type

`TypeError`

What went wrong?

A loop, also called a cycle, was introduced in a prototype chain. That means that when walking this prototype chain, the same place would be accessed over and over again, instead of eventually reaching `null`.

This error is thrown at the time of setting the prototype. In an operation like `Object.setPrototypeOf(a, b)`, if `a` already exists in the prototype chain of `b`, this error will be thrown.

Examples

```
const a = {};
Object.setPrototypeOf(a, a);
// TypeError: can't set prototype: it would cause a prototype chain cycle
```

```
const a = {};
const b = {};
const c = {};
Object.setPrototypeOf(a, b);
Object.setPrototypeOf(b, c);
Object.setPrototypeOf(c, a);
// TypeError: can't set prototype: it would cause a prototype chain cycle
```

See also

- Object Prototypes
- Inheritance and the prototype chain

SyntaxError: applying the 'delete' operator to an unqualified name is deprecated

The JavaScript strict mode-only exception "applying the 'delete' operator to an unqualified name is deprecated" occurs when variables are attempted to be deleted using the `delete` operator.

Message

```
SyntaxError: Delete of an unqualified identifier in strict mode. (V8-based)
SyntaxError: applying the 'delete' operator to an unqualified name is deprecated (Firefox)
```

```
SyntaxError: Cannot delete unqualified property 'a' in strict mode. (Safari)
```

Error type

`SyntaxError` in strict mode only.

What went wrong?

Normal variables in JavaScript can't be deleted using the `delete` operator. In strict mode, an attempt to delete a variable will throw an error and is not allowed.

The `delete` operator can only delete properties on an object. Object properties are "qualified" if they are configurable.

Unlike what common belief suggests, the `delete` operator has **nothing** to do with directly freeing memory. Memory management is done indirectly via breaking references, see the memory management page and the `delete` operator page for more details.

This error only happens in strict mode code. In non-strict code, the operation just returns `false`.

Examples

Freeing the contents of a variable

Attempting to delete a plain variable throws an error in strict mode:

```
"use strict";

var x;

// …

delete x;

// SyntaxError: applying the 'delete' operator to an unqualified name
// is deprecated
```

To free the contents of a variable, you can set it to `null`:

```
"use strict";

var x;

// …

x = null;

// x can be garbage collected
```

See also

- `delete`
- Memory management
- TypeError: property "x" is non-configurable and can't be deleted

TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed

The JavaScript strict mode-only exception "'caller', 'callee', and 'arguments' properties may not be accessed on strict mode functions or the arguments objects for calls to them" occurs when the deprecated `arguments.callee`, `Function.prototype.caller`, or `Function.prototype.arguments` properties are used.

Message

```
TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed on strict mode functions or the arguments objects for calls to them (V8-
based & Firefox)
TypeError: 'arguments', 'callee', and 'caller' cannot be accessed in this context. (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

In strict mode, the `arguments.callee`, `Function.prototype.caller`, or `Function.prototype.arguments` properties are used and shouldn't be. They are deprecated, because they leak the function caller, are non-standard, hard to optimize and potentially a performance-harmful feature.

Examples

Deprecated function.caller or arguments.callee

`Function.prototype.caller` and `arguments.callee` are deprecated (see the reference articles for more information).

```
"use strict";

function myFunc() {
  if (myFunc.caller === null) {
    return "The function was called from the top!";
```

```
  }
  return `This function's caller was ${myFunc.caller}`;
}

myFunc();
// TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed on strict mode functions or the arguments objects for calls to them
```

Function.prototype.arguments

`Function.prototype.arguments` is deprecated (see the reference article for more information).

```
"use strict";

function f(n) {
  g(n - 1);
}

function g(n) {
  console.log(`before: ${g.arguments[0]}`);
  if (n > 0) {
    f(n);
  }
  console.log(`after: ${g.arguments[0]}`);
}

f(2);

console.log(`returned: ${g.arguments}`);
// TypeError: 'caller', 'callee', and 'arguments' properties may not be accessed on strict mode functions or the arguments objects for calls to them
```

See also

- Deprecated and obsolete features
- Strict mode
- `Function.prototype.arguments`
- `Function.prototype.caller`
- `arguments.callee`

SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code

The JavaScript exception "octal escape sequences can't be used in untagged template literals or in strict mode code" occurs when octal escape sequences are used in strict mode string literals or untagged template literals.

Message

```
SyntaxError: Octal escape sequences are not allowed in strict mode. (V8-based)
SyntaxError: \8 and \9 are not allowed in strict mode. (V8-based)
SyntaxError: Octal escape sequences are not allowed in template strings. (V8-based)
SyntaxError: \8 and \9 are not allowed in template strings. (V8-based)
SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code (Firefox)
SyntaxError: the escapes \8 and \9 can't be used in untagged template literals or in strict mode code (Firefox)
SyntaxError: The only valid numeric escape in strict mode is '\0' (Safari)
```

Error type

`SyntaxError`

What went wrong?

The string escape sequence of the form `\` followed by any number of digits, except a single `0`, is deprecated. If you want to represent a character by its code point value, you should use the `\x` or `\u` escape sequence instead, such as `\x01` or `\u0001` instead of `\1`.

Untagged template literals are never allowed to contain octal escape sequences, whether in strict mode or not. However, *tagged* template literals can contain any form of escape sequence, and will cause the template array received by the tag function to contain `undefined`.

Examples

Octal escape sequences

```
"use strict";

"\251";

// SyntaxError: octal escape sequences can't be used in untagged template literals or in strict mode code
```

Valid octal numbers

For octal escape sequences, you can use hexadecimal escape sequences instead:

```
"\xA9";
```

If you want to represent some source text literally without interpreting any escape sequence, use `String.raw`:

```
String.raw`\251`; // A string containing four characters
```

See also

- Lexical grammar

SyntaxError: "0"-prefixed octal literals are deprecated

The JavaScript strict mode-only exception "0-prefixed octal literals are deprecated; use the "0o" prefix instead" occurs when deprecated octal literals (`0` followed by digits) are used.

Message

```
SyntaxError: Octal literals are not allowed in strict mode. (V8-based)
SyntaxError: Decimals with leading zeros are not allowed in strict mode. (V8-based)
SyntaxError: Unexpected number (V8-based)
SyntaxError: "0"-prefixed octal literals are deprecated; use the "0o" prefix instead (Firefox)
SyntaxError: Decimal integer literals with a leading zero are forbidden in strict mode (Safari)
```

Error type

`SyntaxError` in strict mode only.

What went wrong?

Octal literals are deprecated. When you prefix a decimal integer with `0`, you actually change it to an octal literal, which may be surprising. The standardized syntax uses a leading zero followed by a lowercase or uppercase Latin letter "O" (`0o` or `0O`).

Leading zeros are always forbidden, even when the literal is not valid octal literal syntax (such as when the literal contains the digits `8` or `9`, or it has a decimal point). A number literal may only start with `0` if that 0 is its units place.

Examples

"0"-prefixed octal literals

```
"use strict";

03;

// SyntaxError: "0"-prefixed octal literals are deprecated; use the "0o" prefix instead
```

Valid octal numbers

Use a leading zero followed by the letter "o" or "O":

```
0o3;
```

See also

- Lexical grammar

SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead

The JavaScript warning "Using `//@` to indicate sourceURL pragmas is deprecated. Use `//#` instead" occurs when there is a deprecated source map syntax in a JavaScript source.

Message

```
Warning: SyntaxError: Using //@ to indicate sourceURL pragmas is deprecated. Use //# instead

Warning: SyntaxError: Using //@ to indicate sourceMappingURL pragmas is deprecated. Use //# instead
```

Error type

A warning that a `SyntaxError` occurred. JavaScript execution won't be halted.

What went wrong?

There is a deprecated source map syntax in a JavaScript source.

JavaScript sources are often combined and minified to make delivering them from the server more efficient. With source maps, the debugger can map the code being executed to the original source files.

The source map specification changed the syntax due to a conflict with IE whenever it was found in the page after `//@cc_on` was interpreted to turn on conditional compilation in the IE JScript engine. The conditional compilation comment in IE is a little known feature, but it broke source maps with jQuery and other libraries.

Examples

Deprecated syntax

Syntax with the "@" sign is deprecated.

```
//@ sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Standard syntax

Use the "#" sign instead.

```
//# sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Or, alternatively, you can set a `SourceMap` header to your JavaScript file to avoid having a comment at all:

```
SourceMap: /path/to/file.js.map
```

See also

- Use a source map in the Firefox source docs
- Introduction to JavaScript source maps on developer.chrome.com (2012)
- `SourceMap`

SyntaxError: duplicate formal argument x

The JavaScript exception "duplicate formal argument x" or "duplicate argument names not allowed in this context" occurs when a function creates two or more parameter bindings with the same name, and the function is not a non-strict function with only simple parameters.

Message

```
SyntaxError: Duplicate parameter name not allowed in this context (V8-based)
SyntaxError: duplicate formal argument x (Firefox)
SyntaxError: duplicate argument names not allowed in this context (Firefox)
SyntaxError: Cannot declare a parameter named 'x' in strict mode as it has already been declared. (Safari)
SyntaxError: Duplicate parameter 'x' not allowed in function with default parameter values. (Safari)
SyntaxError: Duplicate parameter 'x' not allowed in function with a rest parameter. (Safari)
SyntaxError: Duplicate parameter 'x' not allowed in function with destructuring parameters. (Safari)
```

Error type

`SyntaxError`

What went wrong?

Having two formal parameters of the same name is likely a mistake—the second occurrence would cause the first occurrence to be inaccessible through the parameter name. In legacy JavaScript, this was allowed. Therefore, to not break existing code, this is only an error if the code is guaranteed to not be legacy—either because it is in strict mode or it uses modern parameter syntax (rest, default, or destructured parameters).

Examples

Invalid cases

```
"use strict";

function add(x, x) {
  // How can you access both "x" parameters?
  // SyntaxError: duplicate formal argument x
}
```

```
function doSomething(name, { name }) {
  // How can you access both "name" parameters?
  // SyntaxError: duplicate argument names not allowed in this context
}
```

Valid cases

```
function doSomething(operationName, { name: userName }) {
  // You can access both "operationName" and "userName" parameters.
}

function doSomething(name, user) {
  // You can access both "name" and "user.name" parameters.
}
```

See also

- Functions
- Strict mode

SyntaxError: property name __proto__ appears more than once in object literal

The JavaScript exception "property name __proto__ appears more than once in object literal" occurs when an object literal contains multiple occurrences of the `__proto__` field, which is used to set the prototype of this new object.

Message

```
SyntaxError: Duplicate __proto__ fields are not allowed in object literals (V8-based)
SyntaxError: property name __proto__ appears more than once in object literal (Firefox)
SyntaxError: Attempted to redefine __proto__ property. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `__proto__` key, unlike other property keys, is a special syntax in an object literal. It is used to set the prototype of the object being created and is not allowed to appear more than once in an object literal. Note that this restriction only applies to the `__proto__` prototype setter syntax: if it actually has the effect of creating a property called `__proto__`, then it can appear multiple times. See prototype setter for the exact syntax restrictions.

Worth noting that the `__proto__` key in object literals is a special syntax and is not deprecated, unlike the `Object.prototype.__proto__` accessor property.

Examples

Invalid cases

```
const obj = { __proto__: {}, __proto__: { a: 1 } };
```

Valid cases

```
// Only setting the prototype once
const obj = { __proto__: { a: 1 } };

// These syntaxes all create a property called "__proto__" and can coexist
// They would overwrite each other and the last one is actually used
const __proto__ = null;
const obj2 = {
  ["__proto__"]: {},
  __proto__,
  __proto__() {},
  get __proto__() {
    return 1;
  },
};
```

See also

- Object initializer
- Inheritance and the prototype chain

SyntaxError: getter and setter for private name #x should either be both static or non-static

The JavaScript exception "mismatched placement" occurs when a private getter and setter are mismatched in whether or not they are `static`.

Message

```
SyntaxError: Identifier '#x' has already been declared (V8-based)
SyntaxError: getter and setter for private name #x should either be both static or non-static (Firefox)
SyntaxError: Cannot declare a private non-static getter if there is a static private setter with used name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

Private getters and setters for the same name must either be both `static`, or both non-static. This limitation does not exist for public methods.

Examples

Mismatched placement

```
class Test {
  static set #foo(_) {}
  get #foo() {}
}

// SyntaxError: getter and setter for private name #foo should either be both static or non-static
```

Since `foo` is private, the methods must be either both `static`:

```
class Test {
  static set #foo(_) {}
  static get #foo() {}
}
```

or non-static:

```
class Test {
  set #foo(_) {}
  get #foo() {}
}
```

See also

- `get`
- `set`
- `static`
- Private elements

RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD'

The JavaScript exception "form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD'" occurs when an unrecognized string is passed to the `String.prototype.normalize()` method.

Message

```
RangeError: The normalization form should be one of NFC, NFD, NFKC, NFKD. (V8-based)
RangeError: form must be one of 'NFC', 'NFD', 'NFKC', or 'NFKD' (Firefox)
RangeError: argument does not match any normalization form (Safari)
```

Error type

`RangeError`

What went wrong?

The `String.prototype.normalize()` method only accepts the following four values as its `form` argument: `"NFC"`, `"NFD"`, `"NFKC"`, or `"NFKD"`. If you pass any other value, an error will be thrown. Read the reference of `normalize()` to learn about different normalization forms.

Examples

Invalid cases

```
"foo".normalize("nfc"); // RangeError
"foo".normalize(" NFC "); // RangeError
```

Valid cases

```
"foo".normalize("NFC"); // 'foo'
```

See also

- `String.prototype.normalize()`

SyntaxError: functions cannot be labelled

The JavaScript exception "functions cannot be labelled" occurs when a `function` declaration has a label before it.

Message

```
SyntaxError: In non-strict mode code, functions can only be declared at top level, inside a block, or as the body of an if statement. (V8-based)
SyntaxError: In strict mode code, functions can only be declared at top level or inside a block. (V8-based)
SyntaxError: Generators can only be declared at the top level or inside a block. (V8-based)
SyntaxError: Async functions can only be declared at the top level or inside a block. (V8-based)

SyntaxError: functions can only be labelled inside blocks (Firefox)
SyntaxError: functions cannot be labelled (Firefox)
SyntaxError: generator functions cannot be labelled (Firefox)
SyntaxError: async function declarations can't appear in single-statement context (Firefox)

SyntaxError: Unexpected keyword 'function'. Function declarations are only allowed inside block statements or at the top level of a program. (Safari)
SyntaxError: Function declarations are only allowed inside blocks or switch statements in strict mode. (Safari)
SyntaxError: Unexpected token '*'. Cannot use generator function declaration in single-statement context. (Safari)
SyntaxError: Unexpected keyword 'function'. Cannot use async function declaration in single-statement context. (Safari)
```

Error type

`SyntaxError`

What went wrong?

Function declarations are never supposed to be labeled, because labels should only apply to statements, not declarations. There's no way to actually jump to this label. However, due to some legacy JavaScript syntax rules, the error condition is a bit more complicated than necessary:

- In strict mode, function declarations are never allowed to be labeled.
- In non-strict mode, function declarations are allowed to be labeled, but not when the function is the sole statement of an `if` statement (which itself is a deprecated feature).
- Async functions, generator functions, and async generator functions are never allowed to be labeled.

The error message may say something along the lines of "invalid place for a function declaration to appear", because when the parser sees a label, it expects a statement to follow, and a function declaration is not a statement. It depends on whether the error's perspective is that a label cannot be followed by a function, or that a function cannot be preceded by a label.

Examples

Misparsed object literal

While it's possible that you actually expect the label to do *something* along the lines of being a jump target, usually you didn't intend for it to be a label. The most common case is you actually want it to be a property key in an object literal:

```
const createObj = () => {
  greet: function greet() { // SyntaxError: functions cannot be labelled
    console.log("Hello");
  }
};
```

Here, `{...}` is actually not an object literal, but is instead the block body of the arrow function, so `greet:` becomes a label. To fix this, you need to wrap the object literal in parentheses:

```
const createObj = () => ({
  greet: function greet() {
    console.log("Hello");
  },
});
```

You may also want to use the method syntax for object literals, which avoids this pitfall:

```
const createObj = () => ({
  greet() {
    console.log("Hello");
  },
});
```

See also

- Labeled statement
- `function`
- Strict mode
- Deprecated and obsolete features

TypeError: can't access/set private field or method: object is not the right class

The JavaScript exception "can't access private field or method: object is not the right class" or "can't set private field: object is not the right class" occurs when a private field or method is get or set on an object that does not have this private element defined.

Message

```
TypeError: Cannot read private member #x from an object whose class did not declare it (V8-based)
TypeError: Cannot write private member #x to an object whose class did not declare it (V8-based)
TypeError: can't access private field or method: object is not the right class (Firefox)
TypeError: can't set private field: object is not the right class (Firefox)
TypeError: Cannot access invalid private field (evaluating 'this.#x') (Safari)
```

Error type

`TypeError`

What went wrong?

You are trying to get or set a private field or method on an object, but that object does not contain this private element. Private instance properties can only be accessed on instances of the class (including its subclasses) that declares them; private static properties can only be accessed on the class itself that declares them, and not on subclasses.

This error occurs when the private name exists in the class scope but the object it's accessed on is invalid. If the private name does not exist, you will get a syntax error instead.

Examples

Mismatched static/instance fields

You may have declared the field as a static field, but are trying to access it on an instance, or vice versa.

```
class MyClass {
  static #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}

const obj = new MyClass();
```

```
obj.doSomething();
// TypeError: can't access private field: object is not the right class
```

To fix this, either change the field to be an instance field, or access the field on the class itself, or declare another field on the instance. Note that the private namespace is shared between static and instance properties, so you cannot have a static and instance private element with the same name.

```
class MyClass {
  #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}

class MyClass2 {
  static #x = 0;
  doSomething() {
    console.log(MyClass2.#x);
  }
}
```

Wrong object used

Perhaps you have a method that access `this.#x`, but it is called with another `this` value.

```
class JSONReplacer {
  #count = 0;
  func(key, value) {
    if (typeof value === "object") {
      this.#count++;
    }
    return value;
  }
}

JSON.stringify({ a: 1, b: { c: 2 } }, new JSONReplacer().func);
// TypeError: can't access private field: object is not the right class
```

This is because `JSON.stringify()` calls the replacer function with the object containing `value` as `this`, so the private field is not accessible. To fix this, you can bind the method to the object, or use an arrow function, to ensure that `replacer.func` is called with the correct `this` value.

```
const replacer = new JSONReplacer();
JSON.stringify({ a: 1, b: { c: 2 } }, replacer.func.bind(replacer));
JSON.stringify({ a: 1, b: { c: 2 } }, (...args) => replacer.func(...args));
```

Most of the time, if you accidentally unbound a method, the method would be called with `undefined` as `this`, which would result in a different error (TypeError: can't convert undefined to object). This error only occurs when the method is called with a different object as `this`, either by using `call()` or `apply()`, or by passing the method as a callback to a function that calls it with a different `this` value.

If you don't know for sure that the object will contain the private element, as in the following code:

```
class MyClass {
  #x = 0;
  static doSomething(obj) {
    console.log(obj.#x); // Throws if obj is not an instance of MyClass
  }
}
```

You can use the `in` operator to perform a *branded check* first.

```
class MyClass {
  #x = 0;
  static doSomething(obj) {
    if (!(#x in obj)) {
      return;
    }
    console.log(obj.#x);
  }
}
```

Accessing static elements on subclasses

If you have a private static property, you can only access it on the class that declares it, not on subclasses.

```
class MyClass {
  static #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}

class MySubClass extends MyClass {}
```

```
MySubClass.doSomething();
// TypeError: can't access private field: object is not the right class
```

To fix this, never access private static properties through `this`. Instead, always explicitly specify the class's name.

```
class MyClass {
  static #x = 0;
  doSomething() {
    console.log(MyClass.#x);
  }
}
```

Accessing same-name private elements on another class

Unlike normal string or symbol properties, private names are not shared between classes. If you have a private element with the same name in two classes, they are still not the same element, and you cannot access one class's private element from another class.

```
class MyClass {
  #x = 0;
}

class MyOtherClass {
  #x = 1;
  doSomething(o) {
    console.log(o.#x);
  }
}

const obj = new MyClass();
new MyOtherClass().doSomething(obj);
// TypeError: can't access private field: object is not the right class
```

Adding private elements to unrelated objects

You cannot dynamically *add* private elements to unrelated objects.

```
class MyClass {
  #x = 0;
  static stamp(obj) {
    obj.#x = 1;
  }
}

MyClass.stamp({});
// TypeError: can't set private field: object is not the right class
```

If you really want to do this, consider the return override trick. However, in general, you would probably want to use a `WeakMap` instead.

```
class MyClass {
  static #objToX = new WeakMap();
  static stamp(obj) {
    MyClass.#objToX.set(obj, 1);
  }
}

MyClass.stamp({});
```

See also

- Classes
- Private elements

SyntaxError: getter functions must have no arguments

The JavaScript exception "getter functions must have no arguments" occurs when a getter is declared and the parameter list is non-empty.

Message

```
SyntaxError: Getter must not have any formal parameters. (V8-based)
SyntaxError: getter functions must have no arguments (Firefox)
SyntaxError: Unexpected identifier 'x'. getter functions must have no parameters. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `get` property syntax looks like a function, but it is stricter and not all function syntax is allowed. A getter is always invoked with no arguments, so defining it with any parameter is likely an error.

Note that this error only applies to property getters using the `get` syntax. If you define the getter using `Object.defineProperty()`, etc., the getter is defined as a normal function, although it's likely still an error if the getter expects any arguments, as it will be called without any.

Examples

Invalid cases

```
const obj = {
  get value(type) {
    return type === "string" ? String(Math.random()) : Math.random();
  },
};
```

Valid cases

```
// Remove the parameter
const obj = {
  get value() {
    return Math.random();
  },
};

// Use a normal method, if you need a parameter
const obj = {
  getValue(type) {
    return type === "string" ? String(Math.random()) : Math.random();
  },
};
```

See also

- Functions
- `get`

TypeError: setting getter-only property "x"

The JavaScript strict mode-only exception "setting getter-only property" occurs when there is an attempt to set a new value to a property for which only a getter is specified, or when setting a private accessor property that similarly only has a getter defined.

Message

```
TypeError: Cannot set property x of #<Object> which has only a getter (V8-based)
TypeError: '#x' was defined without a setter (V8-based)
TypeError: setting getter-only property "x" (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
TypeError: Trying to access an undefined private setter (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

There is an attempt to set a new value to a property for which only a getter is specified. While this will be silently ignored in non-strict mode, it will throw a `TypeError` in strict mode. Classes are always in strict mode, so assigning to a getter-only private element always throws this error.

Examples

Property with no setter

The example below shows how to set a getter for a property. It doesn't specify a setter, so a `TypeError` will be thrown upon trying to set the `temperature` property to `30`. For more details see also the `Object.defineProperty()` page.

```
"use strict";

function Archiver() {
  const temperature = null;
  Object.defineProperty(this, "temperature", {
    get() {
      console.log("get!");
      return temperature;
    },
  });
}

const arc = new Archiver();
arc.temperature; // 'get!'

arc.temperature = 30;
// TypeError: setting getter-only property "temperature"
```

To fix this error, you will either need to remove the `arc.temperature = 30` line, which attempts to set the temperature property, or you will need to implement a setter for it, for example like this:

```
"use strict";
```

```
function Archiver() {
  let temperature = null;
  const archive = [];

  Object.defineProperty(this, "temperature", {
    get() {
      console.log("get!");
      return temperature;
    },
    set(value) {
      temperature = value;
      archive.push({ val: temperature });
    },
  });

  this.getArchive = function () {
    return archive;
  };
}

const arc = new Archiver();
arc.temperature; // 'get!'
arc.temperature = 11;
arc.temperature = 13;
arc.getArchive(); // [{ val: 11 }, { val: 13 }]
```

See also

- `Object.defineProperty()`
- `Object.defineProperties()`

SyntaxError: Unexpected '#' used outside of class body

The JavaScript exception "Unexpected '#' used outside of class body" occurs when a hash ("#") is encountered in an unexpected context, most notably outside of a class declaration. Hashes are valid at the beginning of a file as a hashbang comment, or inside of a class as part of a private field. You may encounter this error if you forget the quotation marks when trying to access a DOM identifier as well.

Message

```
SyntaxError: Unexpected '#' used outside of class body.
```

Error type

`SyntaxError`

What went wrong?

We encountered a `#` somewhere unexpected. This may be due to code moving around and no longer being part of a class, a hashbang comment found on a line other than the first line of a file, or accidentally forgetting the quotation marks around a DOM identifier.

Examples

Missing quotation marks

For each case, there might be something slightly wrong. For example

```
document.querySelector(#some-element)
```

This can be fixed via

```
document.querySelector("#some-element");
```

Outside of a class

```
class ClassWithPrivateField {
  #privateField;

  constructor() {}
}

this.#privateField = 42;
```

This can be fixed by moving the private field back into the class

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    this.#privateField = 42;
  }
}
```

See also

- `SyntaxError`

SyntaxError: identifier starts immediately after numeric literal

The JavaScript exception "identifier starts immediately after numeric literal" occurs when an identifier started with a digit. Identifiers can only start with a letter, underscore (_), or dollar sign ($).

Message

```
SyntaxError: Invalid or unexpected token (V8-based)
SyntaxError: identifier starts immediately after numeric literal (Firefox)
SyntaxError: No identifiers allowed directly after numeric literal (Safari)
```

Error type

`SyntaxError`

What went wrong?

The names of variables, called identifiers, conform to certain rules, which your code must adhere to!

A JavaScript identifier must start with a letter, underscore (_), or dollar sign ($). They can't start with a digit! Only subsequent characters can be digits (0-9).

Examples

Variable names starting with numeric literals

Variable names can't start with numbers in JavaScript. The following fails:

```
const 1life = "foo";
// SyntaxError: identifier starts immediately after numeric literal

const foo = 1life;
// SyntaxError: identifier starts immediately after numeric literal
```

You will need to rename your variable to avoid the leading number.

```
const life1 = "foo";
const foo = life1;
```

In JavaScript, there is a syntactic peculiarity when calling properties or methods on numbers. If you want to call a method on an integer, you cannot immediately use a dot after the number because the dot is interpreted as the start of a decimal fraction, causing the parser to see the method's name as an identifier immediately after a number literal. To avoid this, you need to either wrap the number in parentheses or use a double dot, where the first dot is a decimal point for the number literal and the second dot is the property accessor.

```
alert(typeof 1.toString())
// SyntaxError: identifier starts immediately after numeric literal
```

Correct ways to call methods on numbers:

```
// Wrap the number in parentheses
alert(typeof (1).toString());

// Add an extra dot for the number literal
alert(typeof 2..toString());

// Use square brackets
alert(typeof 3["toString"]());
```

See also

- Lexical grammar
- Grammar and types guide

SyntaxError: illegal character

The JavaScript exception "illegal character" occurs when the lexer reads a character that's not part of a string literal, and the character cannot constitute a valid token in the language.

Message

```
SyntaxError: Invalid or unexpected token (V8-based)
SyntaxError: illegal character U+201C (Firefox)
SyntaxError: Invalid character '\u201c' (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is an invalid character that the interpreter doesn't understand. You should either put it in a string literal or replace it with another character. Use an editor that supports syntax highlighting and carefully check your code against mismatches like a minus sign (-) versus a dash (–) or simple quotes (") versus non-standard quotation marks (“).

Examples

Mismatched characters

Some characters look similar, but will cause the parser to fail interpreting your code. Famous examples of this are quotes, the minus or semicolon (greek question mark (U+37e) looks same).

```
“This looks like a string”; // SyntaxError: illegal character
// “ and ” are not " but look like it

42 – 13; // SyntaxError: illegal character
// – (en-dash) is not - but looks like it

const foo = "bar"; // SyntaxError: illegal character
// <37e> is not ; but looks like it
```

This should work:

```
"This is actually a string";
42 - 13;
const foo = "bar";
```

Some editors and IDEs will notify you or at least use a slightly different highlighting for it, but not all. When something like this happens to your code and you're not able to find the source of the problem, it's often best to just delete the problematic line and retype it.

Forgotten characters

It's easy to forget a character here or there.

```
const operators = ["+", "-", ×", "÷"];
// SyntaxError: illegal character U+00D7
```

Add the missing quote for `"×"`.

```
const operators = ["+", "-", "×", "÷"];
```

Hidden characters

When copy pasting code from external sources, there might be invalid characters. Watch out!

```
const foo = "bar";
// SyntaxError: illegal character
```

When inspecting this code in an editor like VIM, you can see that there is actually a zero-width space (ZWSP) (U+200B) character.

```
const foo = "bar";<200b>
```

See also

- Lexical grammar

SyntaxError: import declarations may only appear at top level of a module

The JavaScript exception "import declarations may only appear at top level of a module" occurs when an import declaration is not at the top level of a module. This might be because the import declaration is nested in other constructs (functions, blocks, etc.), or more often because the current file is not treated as a module.

Message

```
SyntaxError: Cannot use import statement outside a module (V8-based)
SyntaxError: import declarations may only appear at top level of a module (Firefox)
SyntaxError: Unexpected identifier 'x'. import call expects one or two arguments. (Safari)
```

Error type

`SyntaxError`

What went wrong?

You may have an `import` declaration that's nested in another construct, such as a function or a block. The `import` declaration must be at the top level of the module. If you want to conditionally import a module, or lazily import it on demand, use dynamic import instead.

If the `import` is already at the top level of the code, it may be because the file is not interpreted as a module. Runtimes require external hints to determine whether a file is a module or not, and here are several ways to provide such hints:

- If the file is directly loaded from HTML, make sure the `<script>` tag has the `type="module"` attribute.
- If the file is run in Node, make sure either the file has the `.mjs` extension, or the closest `package.json` file has the `"type": "module"` field.
- If the file is run as a worker, make sure you call the `Worker()` constructor with the `type: "module"` option.

- Import this file from another module.

One other cause might be when you are writing `import` with a compiler (such as TypeScript) and you accidentally ran the source file. Since `import` declarations usually appear at the very start of the program, they are the first thing the parser sees and then complains about. Make sure to compile the source file and execute the compiled file instead.

Examples

Conditional imports

You cannot use `import` inside other constructs, like you might do in Python.

```
if (writeOutput) {
  import fs from "fs"; // SyntaxError
}
```

Either move the `import` to the top level, or use dynamic import.

```
if (writeOutput) {
  import("fs").then((fs) => {
    // use fs
  });
}
```

Importing in a non-module script

If you are loading the script from HTML, make sure to add the `type="module"` attribute to the `<script>` tag.

```
<script type="module" src="https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/import_decl_module_top_level/main.js"></script>
```

If for some reason you cannot migrate the script to a module, you can use dynamic import.

```
async function main() {
  const myModule = await import("./my-module.js");
  // use myModule
}

main();
```

See also

- Using modules
- `import`
- `import()`

TypeError: cannot use 'in' operator to search for 'x' in 'y'

The JavaScript exception "right-hand side of 'in' should be an object" occurs when the `in` operator was used to search in strings, or in numbers, or other primitive types. It can only be used to check if a property is in an object.

Message

```
TypeError: Cannot use 'in' operator to search for 'x' in 'y' (V8-based & Firefox)
TypeError: right-hand side of 'in' should be an object, got null (Firefox)
TypeError: "y" is not an Object. (evaluating '"x" in "y"') (Safari)
```

Error type

`TypeError`

What went wrong?

The `in` operator can only be used to check if a property is in an object. You can't search in strings, or in numbers, or other primitive types.

Examples

Searching in strings

Unlike in other programming languages (e.g., Python), you can't search in strings using the `in` operator.

```
"Hello" in "Hello World";
// TypeError: cannot use 'in' operator to search for 'Hello' in 'Hello World'
```

Instead you will need to use `String.prototype.includes()`, for example.

```
"Hello World".includes("Hello");
// true
```

The operand can't be null or undefined

Make sure the object you are inspecting isn't actually `null` or `undefined`.

```
const foo = null;
"bar" in foo;
// TypeError: cannot use 'in' operator to search for 'bar' in 'foo' (Chrome)
// TypeError: right-hand side of 'in' should be an object, got null (Firefox)
```

The `in` operator always expects an object.

```
const foo = { baz: "bar" };
"bar" in foo; // false

"PI" in Math; // true
"pi" in Math; // false
```

Searching in arrays

Be careful when using the `in` operator to search in `Array` objects. The `in` operator checks the index number, not the value at that index.

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
3 in trees; // true
"oak" in trees; // false
```

See also

- `in`

RangeError: invalid array length

The JavaScript exception "Invalid array length" occurs when specifying an array length that is either negative, a floating number or exceeds the maximum supported by the platform (i.e., when creating an `Array` or `ArrayBuffer`, or when setting the `length` property).

The maximum allowed array length depends on the platform, browser and browser version. For `Array` the maximum length is 2^{32}-1. For `ArrayBuffer` the maximum is 2^{31}-1 (2GiB-1) on 32-bit systems. From Firefox version 89 the maximum value of `ArrayBuffer` is 2^{33} (8GiB) on 64-bit systems.

> **Note:** `Array` and `ArrayBuffer` are independent data structures (the implementation of one does not affect the other).

Message

```
RangeError: invalid array length (V8-based & Firefox)
RangeError: Array size is not a small enough positive integer. (Safari)

RangeError: Invalid array buffer length (V8-based)
RangeError: length too large (Safari)
```

Error type

`RangeError`

What went wrong?

The error might appear when attempting to produce an `Array` or `ArrayBuffer` with an invalid length, which includes:

- Negative length, via the constructor or setting the `length` property.
- Non-integer length, via the constructor or setting the `length` property. (The `ArrayBuffer` constructor coerces the length to an integer, but the `Array` constructor does not.)
- Exceeding the maximum length supported by the platform. For arrays, the maximum length is 2^{32}-1. For `ArrayBuffer`, the maximum length is 2^{31}-1 (2GiB-1) on 32-bit systems, or 2^{33} (8GiB) on 64-bit systems. This can happen via the constructor, setting the `length` property, or array methods that implicitly set the length property (such as `push` and `concat`).

If you are creating an `Array` using the constructor, you probably want to use the literal notation instead, as the first argument is interpreted as the length of the `Array`. Otherwise, you might want to clamp the length before setting the length property, or using it as an argument of the constructor.

Examples

Invalid cases

```
new Array(2 ** 40);
new Array(-1);
new ArrayBuffer(2 ** 32); // 32-bit system
new ArrayBuffer(-1);

const a = [];
a.length -= 1; // set the length property to -1

const b = new Array(2 ** 32 - 1);
b.length += 1; // set the length property to 2^32
b.length = 2.5; // set the length property to a floating-point number

const c = new Array(2.5); // pass a floating-point number

// Concurrent modification that accidentally grows the array infinitely
```

```
const arr = [1, 2, 3];
for (const e of arr) {
  arr.push(e * 10);
}
```

Valid cases

```
[2 ** 40]; // [ 1099511627776 ]
[-1]; // [ -1 ]
new ArrayBuffer(2 ** 31 - 1);
new ArrayBuffer(2 ** 33); // 64-bit systems after Firefox 89
new ArrayBuffer(0);

const a = [];
a.length = Math.max(0, a.length - 1);

const b = new Array(2 ** 32 - 1);
b.length = Math.min(0xffffffff, b.length + 1);
// 0xffffffff is the hexadecimal notation for 2^32 - 1
// which can also be written as (-1 >>> 0)

b.length = 3;

const c = new Array(3);

// Because array methods save the length before iterating, it is safe to grow
// the array during iteration
const arr = [1, 2, 3];
arr.forEach((e) => arr.push(e * 10));
```

See also

- `Array`
- `length`
- `ArrayBuffer`

SyntaxError: invalid assignment left-hand side

The JavaScript exception "invalid assignment left-hand side" occurs when there was an unexpected assignment somewhere. It may be triggered when a single `=` sign was used instead of `==` or `===`.

Message

```
SyntaxError: Invalid left-hand side in assignment (V8-based)
SyntaxError: invalid assignment left-hand side (Firefox)
SyntaxError: Left side of assignment is not a reference. (Safari)

ReferenceError: Invalid left-hand side in assignment (V8-based)
ReferenceError: cannot assign to function call (Firefox)
ReferenceError: Left side of assignment is not a reference. (Safari)
```

Error type

`SyntaxError` or `ReferenceError`, depending on the syntax.

What went wrong?

There was an unexpected assignment somewhere. This might be due to a mismatch of an assignment operator and an equality operator, for example. While a single `=` sign assigns a value to a variable, the `==` or `===` operators compare a value.

Examples

Typical invalid assignments

```
if (Math.PI + 1 = 3 || Math.PI + 1 = 4) {
  console.log("no way!");
}
// SyntaxError: invalid assignment left-hand side

const str = "Hello, "
+= "is it me "
+= "you're looking for?";
// SyntaxError: invalid assignment left-hand side
```

In the `if` statement, you want to use an equality operator (`===`), and for the string concatenation, the plus (`+`) operator is needed.

```
if (Math.PI + 1 === 3 || Math.PI + 1 === 4) {
  console.log("no way!");
}

const str = "Hello, "
  + "from the "
  + "other side!";
```

Assignments producing ReferenceErrors

Invalid assignments don't always produce syntax errors. Sometimes the syntax is almost correct, but at runtime, the left-hand side expression evaluates to a *value* instead of a *reference*, so the assignment is still invalid. Such errors occur later in execution, when the statement is actually executed.

```
function foo() {
  return { a: 1 };
}
foo() = 1; // ReferenceError: invalid assignment left-hand side
```

Function calls, `new` calls, `super()`, and `this` are all values instead of references. If you want to use them on the left-hand side, the assignment target needs to be a property of their produced values instead.

```
function foo() {
  return { a: 1 };
}
foo().a = 1;
```

> **Note:** In Firefox and Safari, the first example produces a `ReferenceError` in non-strict mode, and a `SyntaxError` in strict mode. Chrome throws a runtime `ReferenceError` for both strict and non-strict modes.

Using optional chaining as assignment target

Optional chaining is not a valid target of assignment.

```
obj?.foo = 1; // SyntaxError: invalid assignment left-hand side
```

Instead, you have to first guard the nullish case.

```
if (obj) {
  obj.foo = 1;
}
```

See also

- Assignment operators
- Equality operators

SyntaxError: invalid BigInt syntax

The JavaScript exception "invalid BigInt syntax" occurs when a string value is being coerced to a `BigInt` but it failed to be parsed as an integer.

Message

```
SyntaxError: Cannot convert x to a BigInt (V8-based)
SyntaxError: invalid BigInt syntax (Firefox)
SyntaxError: Failed to parse String to BigInt (Safari)
```

Error type

`SyntaxError`.

What went wrong?

When using the `BigInt()` function to convert a string to a BigInt, the string will be parsed in the same way as source code, and the resulting value must be an integer value.

Examples

Invalid cases

```
const a = BigInt("1.5");
const b = BigInt("1n");
const c = BigInt.asIntN(4, "8n");
// SyntaxError: invalid BigInt syntax
```

Valid cases

```
const a = BigInt("1");
const b = BigInt("  1   ");
const c = BigInt.asIntN(4, "8");
```

See also

- `BigInt`

TypeError: invalid assignment to const "x"

The JavaScript exception "invalid assignment to const" occurs when it was attempted to alter a constant value. JavaScript `const` declarations can't be re-assigned or redeclared.

Message

```
TypeError: Assignment to constant variable. (V8-based)
TypeError: invalid assignment to const 'x' (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError`

What went wrong?

A constant is a value that cannot be altered by the program during normal execution. It cannot change through re-assignment, and it can't be redeclared. In JavaScript, constants are declared using the `const` keyword.

Examples

Invalid redeclaration

Assigning a value to the same constant name in the same block-scope will throw.

```
const COLUMNS = 80;

// …

COLUMNS = 120; // TypeError: invalid assignment to const `COLUMNS'
```

Fixing the error

There are multiple options to fix this error. Check what was intended to be achieved with the constant in question.

Rename

If you meant to declare another constant, pick another name and re-name. This constant name is already taken in this scope.

```
const COLUMNS = 80;
const WIDE_COLUMNS = 120;
```

const, let or var?

Do not use const if you weren't meaning to declare a constant. Maybe you meant to declare a block-scoped variable with `let` or global variable with `var`.

```
let columns = 80;

// …

columns = 120;
```

Scoping

Check if you are in the correct scope. Should this constant appear in this scope or was it meant to appear in a function, for example?

```
const COLUMNS = 80;

function setupBigScreenEnvironment() {
  const COLUMNS = 120;
}
```

const and immutability

The `const` declaration creates a read-only reference to a value. It does **not** mean the value it holds is immutable, just that the variable identifier cannot be reassigned. For instance, in case the content is an object, this means the object itself can still be altered. This means that you can't mutate the value stored in a variable:

```
const obj = { foo: "bar" };
obj = { foo: "baz" }; // TypeError: invalid assignment to const `obj'
```

But you can mutate the properties in a variable:

```
obj.foo = "baz";
obj; // { foo: "baz" }
```

See also

- `const`
- `let`
- `var`

RangeError: invalid date

The JavaScript exception "invalid date" occurs when an invalid date is attempted to be converted to an ISO date string.

Message

```
RangeError: Invalid time value (V8-based)
RangeError: invalid date (Firefox)
RangeError: Invalid Date (Safari)
```

Error type

`RangeError`

What went wrong?

You are converting an invalid date value to an ISO date string by calling the `toISOString()` method.

An *invalid date* is produced when you attempt to parse an invalid date string, or set the timestamp to an out-of-bounds value. Invalid dates usually cause all date methods to return `NaN` or other special values. However, such dates do not have valid ISO string representations, so an error is thrown when you attempt to do so.

> **Note:** `toJSON()` does not throw this error. It checks whether the date is finite before formatting it, and returns `null` for an invalid date instead of calling `toISOString()`. `JSON.stringify()`, which calls `toJSON()`, therefore serializes an invalid date as `null` rather than throwing.

Examples

Invalid cases

```
const invalid = new Date("nothing");
invalid.toISOString(); // RangeError: invalid date
```

However, most other methods return special values:

```
invalid.toString(); // "Invalid Date"
invalid.getDate(); // NaN
invalid.toJSON(); // null
JSON.stringify({ date: invalid }); // '{"date":null}'
```

For more details, see the `Date.parse()` documentation.

Valid cases

```
new Date("05 October 2011 14:48 UTC").toISOString(); // "2011-10-05T14:48:00.000Z"
new Date(1317826080).toISOString(); // "2011-10-05T14:48:00.000Z"
```

See also

- `Date`
- `Date.prototype.parse()`
- `Date.prototype.toISOString()`

TypeError: derived class constructor returned invalid value x

The JavaScript exception "derived class constructor returned invalid value x" occurs when a derived class constructor returns a value that is not an object or `undefined`.

Message

```
TypeError: Derived constructors may only return object or undefined (V8-based)
TypeError: derived class constructor returned invalid value 1 (Firefox)
TypeError: Cannot return a non-object type in the constructor of a derived class. (Safari)
```

Error type

`TypeError`

What went wrong?

Typically, a constructor does not need to return anything—the value of `this` is automatically returned when the class is constructed. A constructor can also return an object, and this object will override `this` as the newly constructed instance. However, returning something that's neither an object nor `undefined` is usually a mistake, because that value is ignored. In base classes and function constructors (using the `function` syntax), returning such a value is silently ignored, while in derived classes, it throws an error.

Examples

Invalid cases

```
class Base {
  constructor() {}
}

class Derived extends Base {
  constructor() {
    return 2;
```

```
  }
}

new Derived(); // TypeError: derived class constructor returned invalid value 2
```

Valid cases

```
class Base {
  constructor() {}
}

class Derived extends Base {
  constructor() {
    return { x: 1 };
  }
}

new Derived(); // { x: 1 }
```

See also

- Classes
- `extends`
- `new`

SyntaxError: for-in loop head declarations may not have initializers

The JavaScript strict mode-only exception "for-in loop head declarations may not have initializers" occurs when the head of a for...in contains an initializer expression, such as `for (var i = 0 in obj)`. This is not allowed in for-in loops in strict mode. In addition, lexical declarations with initializers like `for (const i = 0 in obj)` are not allowed outside strict mode either.

Message

```
SyntaxError: for-in loop variable declaration may not have an initializer. (V8-based)
SyntaxError: for-in loop head declarations may not have initializers (Firefox)
SyntaxError: a lexical declaration in the head of a for-in loop can't have an initializer (Firefox)
SyntaxError: Cannot assign to the loop variable inside a for-in loop header. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

The head of a for...in loop contains an initializer expression. That is, a variable is declared and assigned a value `for (var i = 0 in obj)`. In non-strict mode, this head declaration is silently ignored and behaves like `for (var i in obj)`. In strict mode, however, a `SyntaxError` is thrown. In addition, lexical declarations with initializers like `for (const i = 0 in obj)` are not allowed outside strict mode either, and will always produce a `SyntaxError`.

Examples

This example throws a `SyntaxError`:

```
const obj = { a: 1, b: 2, c: 3 };

for (const i = 0 in obj) {
  console.log(obj[i]);
}

// SyntaxError: for-in loop head declarations may not have initializers
```

Valid for-in loop

You can remove the initializer (`i = 0`) in the head of the for-in loop.

```
const obj = { a: 1, b: 2, c: 3 };

for (const i in obj) {
  console.log(obj[i]);
}
```

Array iteration

The for...in loop shouldn't be used for Array iteration. Did you intend to use a `for` loop instead of a `for-in` loop to iterate an `Array`? The `for` loop allows you to set an initializer then as well:

```
const arr = ["a", "b", "c"];

for (let i = 2; i < arr.length; i++) {
  console.log(arr[i]);
}

// "c"
```

See also

- `for...in`
- `for...of`
- `for`

SyntaxError: a declaration in the head of a for-of loop can't have an initializer

The JavaScript exception "a declaration in the head of a for-of loop can't have an initializer" occurs when the head of a for...of loop contains an initializer expression such as `for (const i = 0 of iterable)`. This is not allowed in for-of loops.

Message

```
SyntaxError: for-of loop variable declaration may not have an initializer. (V8-based)
SyntaxError: a declaration in the head of a for-of loop can't have an initializer (Firefox)
SyntaxError: Cannot assign to the loop variable inside a for-of loop header. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The head of a for...of loop contains an initializer expression. That is, a variable is declared and assigned a value `for (const i = 0 of iterable)`. This is not allowed in for-of loops. You might want a `for` loop that does allow an initializer.

Examples

Invalid for-of loop

```
const iterable = [10, 20, 30];

for (const value = 50 of iterable) {
  console.log(value);
}

// SyntaxError: a declaration in the head of a for-of loop can't
// have an initializer
```

Valid for-of loop

You need to remove the initializer (`value = 50`) in the head of the `for-of` loop. Maybe you intended to make 50 an offset value, in that case you could add it to the loop body, for example.

```
const iterable = [10, 20, 30];

for (let value of iterable) {
  value += 50;
  console.log(value);
}
// 60
// 70
// 80
```

See also

- `for...of`
- `for...in`
- `for`

TypeError: invalid 'instanceof' operand 'x'

The JavaScript exception "invalid 'instanceof' operand" occurs when the right-hand side operands of the `instanceof` operator isn't used with a constructor object, i.e., an object which has a `prototype` property and is callable.

Message

```
TypeError: Right-hand side of 'instanceof' is not an object (V8-based)
TypeError: invalid 'instanceof' operand "x" (Firefox)
TypeError: Right hand side of instanceof is not an object (Safari)

TypeError: Right-hand side of 'instanceof' is not callable (V8-based)
TypeError: x is not a function (Firefox)
TypeError: x is not a function. (evaluating 'x instanceof y') (Safari)

TypeError: Function has non-object prototype 'undefined' in instanceof check (V8-based)
TypeError: 'prototype' property of x is not an object (Firefox)
TypeError: instanceof called on an object with an invalid prototype property. (Safari)
```

Error type

`TypeError`

What went wrong?

The `instanceof` operator expects the right-hand-side operands to be a constructor object, i.e., an object which has a `prototype` property and is callable. It can also be an object with a `Symbol.hasInstance` method. This error can occur if:

- The right-hand side operand is not an object.
- The right-hand side operand is not a callable and it has no `Symbol.hasInstance` method.
- The right-hand side operand is a callable, but its `prototype` property is not an object. (For example, arrow functions do not have a `prototype` property.)

Examples

instanceof vs. typeof

```
"test" instanceof ""; // TypeError: invalid 'instanceof' operand ""
42 instanceof 0; // TypeError: invalid 'instanceof' operand 0

function Foo() {}
const f = Foo(); // Foo() is called and returns undefined
const x = new Foo();

x instanceof f; // TypeError: invalid 'instanceof' operand f
x instanceof x; // TypeError: x is not a function
```

To fix these errors, you will either need to replace the `instanceof` operator with the `typeof` operator, or to make sure you use the function name, instead of the result of its evaluation.

```
typeof "test" === "string"; // true
typeof 42 === "number"; // true

function Foo() {}
const f = Foo; // Do not call Foo.
const x = new Foo();

x instanceof f; // true
x instanceof Foo; // true
```

See also

- `instanceof`
- `typeof`

TypeError: 'x' is not iterable

The JavaScript exception "is not iterable" occurs when the value which is spread into an array or function call, given as the right-hand side of `for...of`, as an argument of a function such as `Promise.all` or `Set()`, or as the right-hand side of an array destructuring, is not an iterable object. This error is also encountered when `Array.fromAsync()` or `for await...of` is used with a non-async iterable.

Message

```
TypeError: Spread syntax requires ...iterable[Symbol.iterator] to be a function (V8-based & Safari)
TypeError: %Array%.from requires that the property of the first argument, items[Symbol.iterator], when exists, be a function (V8-based & Safari)
TypeError: Array.fromAsync requires that the property of the first argument, items[Symbol.asyncIterator], when exists, be a function (V8-based & Safari)
TypeError: object is not iterable (cannot read property Symbol(Symbol.iterator)) (V8-based)
TypeError: x is not async iterable (V8-based)
TypeError: x is not iterable (V8-based & Firefox)
TypeError: undefined is not a function (near '...y of x...') (Safari)
TypeError: Array.from: no function (Safari)
TypeError: Type error (Safari)
```

Error type

`TypeError`

What went wrong?

The value which is spread into an array or function call, given as the right-hand side of `for...of`, or as an argument of a function such as `Promise.all` or `Set()`, or as the source of an array destructuring pattern, is not an iterable object. An iterable can be a built-in iterable type such as `Array`, `String` or `Map`, a generator result, or an object implementing the iterable protocol.

```
const nonIterable1 = {};
const nonIterable2 = { [Symbol.iterator]: 1 };

[...nonIterable1];
Math.max(...nonIterable1);
for (const x of nonIterable1);
new Set(nonIterable1);
Array.from(nonIterable2);
new Int8Array(nonIterable2);
const [] = nonIterable1;
```

Examples

Array destructuring a non-iterable

```
const myObj = { arrayOrObjProp1: {}, arrayOrObjProp2: [42] };

const {
  arrayOrObjProp1: [value1],
  arrayOrObjProp2: [value2],
} = myObj; // TypeError: object is not iterable

console.log(value1, value2);
```

The non-iterable might turn to be `undefined` in some runtime environments.

Iterating over Object properties

In JavaScript, `Object`s are not iterable unless they implement the iterable protocol. Therefore, you cannot use `for...of` to iterate over the properties of an object.

```
const obj = { France: "Paris", England: "London" };
for (const p of obj) {
  // …
} // TypeError: obj is not iterable
```

Instead you have to use `Object.keys` or `Object.entries`, to iterate over the properties or entries of an object.

```
const obj = { France: "Paris", England: "London" };
// Iterate over the property names:
for (const country of Object.keys(obj)) {
  const capital = obj[country];
  console.log(country, capital);
}

for (const [country, capital] of Object.entries(obj)) {
  console.log(country, capital);
}
```

Another option for this use case might be to use a `Map`:

```
const map = new Map();
map.set("France", "Paris");
map.set("England", "London");
// Iterate over the property names:
for (const country of map.keys()) {
  const capital = map.get(country);
  console.log(country, capital);
}

for (const capital of map.values()) {
  console.log(capital);
}

for (const [country, capital] of map.entries()) {
  console.log(country, capital);
}
```

Iterating over a generator

Generator functions are functions you call to produce an iterable object.

```
function* generate(a, b) {
  yield a;
  yield b;
}

for (const x of generate) {
  console.log(x);
} // TypeError: generate is not iterable
```

When they are not called, the `Function` object corresponding to the generator is callable, but not iterable. Calling a generator produces an iterable object which will iterate over the values yielded during the execution of the generator.

```
function* generate(a, b) {
  yield a;
  yield b;
}

for (const x of generate(1, 2)) {
  console.log(x);
}
```

Iterating over a custom iterable

Custom iterables can be created by implementing the `Symbol.iterator` method. You must be certain that your iterator method returns an object which is an iterator, which is to say it must have a next method.

```
const myEmptyIterable = {
  [Symbol.iterator]() {
    return []; // [] is iterable, but it is not an iterator — it has no next method.
  },
};

Array.from(myEmptyIterable); // TypeError: myEmptyIterable is not iterable
```

Here is a correct implementation:

```
const myEmptyIterable = {
  [Symbol.iterator]() {
    return [][Symbol.iterator]();
  },
};

Array.from(myEmptyIterable); // []
```

See also

- Iterable protocol
- `Object.keys`
- `Object.entries`
- `Map`
- Generator functions
- for...of

SyntaxError: JSON.parse: bad parsing

The JavaScript exceptions thrown by `JSON.parse()` occur when string failed to be parsed as JSON.

Message

```
SyntaxError: JSON.parse: unterminated string literal
SyntaxError: JSON.parse: bad control character in string literal
SyntaxError: JSON.parse: bad character in string literal
SyntaxError: JSON.parse: bad Unicode escape
SyntaxError: JSON.parse: bad escape character
SyntaxError: JSON.parse: unterminated string
SyntaxError: JSON.parse: no number after minus sign
SyntaxError: JSON.parse: unexpected non-digit
SyntaxError: JSON.parse: missing digits after decimal point
SyntaxError: JSON.parse: unterminated fractional number
SyntaxError: JSON.parse: missing digits after exponent indicator
SyntaxError: JSON.parse: missing digits after exponent sign
SyntaxError: JSON.parse: exponent part is missing a number
SyntaxError: JSON.parse: unexpected end of data
SyntaxError: JSON.parse: unexpected keyword
SyntaxError: JSON.parse: unexpected character
SyntaxError: JSON.parse: end of data while reading object contents
SyntaxError: JSON.parse: expected property name or '}'
SyntaxError: JSON.parse: end of data when ',' or ']' was expected
SyntaxError: JSON.parse: expected ',' or ']' after array element
SyntaxError: JSON.parse: end of data when property name was expected
SyntaxError: JSON.parse: expected double-quoted property name
SyntaxError: JSON.parse: end of data after property name when ':' was expected
SyntaxError: JSON.parse: expected ':' after property name in object
SyntaxError: JSON.parse: end of data after property value in object
SyntaxError: JSON.parse: expected ',' or '}' after property value in object
SyntaxError: JSON.parse: expected ',' or '}' after property-value pair in object literal
SyntaxError: JSON.parse: property names must be double-quoted strings
SyntaxError: JSON.parse: expected property name or '}'
SyntaxError: JSON.parse: unexpected character
SyntaxError: JSON.parse: unexpected non-whitespace character after JSON data
```

Error type

`SyntaxError`

What went wrong?

`JSON.parse()` parses a string as JSON. This string has to be valid JSON and will throw this error if incorrect syntax was encountered.

Examples

JSON.parse() does not allow trailing commas

Both lines will throw a SyntaxError:

```
JSON.parse("[1, 2, 3, 4,]");
JSON.parse('{"foo": 1,}');
// SyntaxError JSON.parse: unexpected character
// at line 1 column 14 of the JSON data
```

Omit the trailing commas to parse the JSON correctly:

```
JSON.parse("[1, 2, 3, 4]");
JSON.parse('{"foo": 1}');
```

Property names must be double-quoted strings

You cannot use single-quotes around properties, like 'foo'.

```
JSON.parse("{'foo': 1}");
// SyntaxError: JSON.parse: expected property name or '}'
// at line 1 column 2 of the JSON data
```

Instead write "foo":

```
JSON.parse('{"foo": 1}');
```

Leading zeros and decimal points

You cannot use leading zeros, like 01, and decimal points must be followed by at least one digit.

```
JSON.parse('{"foo": 01}');
// SyntaxError: JSON.parse: expected ',' or '}' after property value
// in object at line 1 column 2 of the JSON data

JSON.parse('{"foo": 1.}');
// SyntaxError: JSON.parse: unterminated fractional number
// at line 1 column 2 of the JSON data
```

Instead write just 1 without a zero and use at least one digit after a decimal point:

```
JSON.parse('{"foo": 1}');
JSON.parse('{"foo": 1.0}');
```

See also

- `JSON`
- `JSON.parse()`
- `JSON.stringify()`

TypeError: WeakSet key/WeakMap value 'x' must be an object or an unregistered symbol

The JavaScript exception "WeakSet key (or WeakMap value) 'x' must be an object or an unregistered symbol" occurs when a value of invalid type is used as a key in a `WeakSet` or as a value in a `WeakMap`.

Message

```
TypeError: Invalid value used as weak map key (V8-based)
TypeError: WeakMap key 1 must be an object or an unregistered symbol (Firefox)
TypeError: WeakMap keys must be objects or non-registered symbols (Safari)

TypeError: Invalid value used in weak set (V8-based)
TypeError: WeakSet value 1 must be an object or an unregistered symbol (Firefox)
TypeError: WeakSet values must be objects or non-registered symbols (Safari)
```

Error type

`TypeError`

What went wrong?

`WeakSet` and `WeakMap` require the keys to be *garbage collectable*. Only objects and non-registered symbols (that is, symbols not returned by `Symbol.for()`) are valid. For more information, see Memory management. If you want to add keys that are strings, numbers, or other primitive values, you should store them in a regular `Set` or `Map` instead.

Examples

Invalid cases

```
new WeakSet().add(1); // TypeError
new WeakMap().set(1, {}); // TypeError
new WeakSet([1]); // TypeError
new WeakMap([[1, {}]]); // TypeError
```

Valid cases

```
new WeakSet().add({}); // OK
new WeakMap().set({}, 1); // OK

new Set([1]); // OK
new Map([[1, {}]]); // OK
```

See also

- Memory management
- `WeakSet`
- `WeakMap`
- `Set`
- `Map`

SyntaxError: label not found

The JavaScript exception "label not found" occurs when a `break` or `continue` statement references a label that does not exist on any statement that contains the `break` or `continue` statement.

Message

```
SyntaxError: Undefined label 'label' (V8-based)
SyntaxError: label not found (Firefox)
SyntaxError: Cannot use the undeclared label 'label'. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

In JavaScript, labels are very limited: you can only use them with `break` and `continue` statements, and you can only jump to them from a statement contained within the labeled statement. You cannot jump to this label from anywhere in the program.

Examples

Unsyntactic jump

You cannot use labels as if they are `goto`.

```
start: console.log("Hello, world!");
console.log("Do it again");
break start;
```

Instead, you can only use labels to enhance the normal semantics of `break` and `continue` statements.

```
start: {
  console.log("Hello, world!");
  if (Math.random() > 0.5) {
    break start;
  }
  console.log("Maybe I'm logged");
}
```

See also

- Labeled statement

URIError: malformed URI sequence

The JavaScript exception "malformed URI sequence" occurs when URI encoding or decoding wasn't successful.

Message

```
URIError: URI malformed (V8-based)
URIError: malformed URI sequence (Firefox)
URIError: String contained an illegal UTF-16 sequence. (Safari)
```

Error type

`URIError`

What went wrong?

URI encoding or decoding wasn't successful. An argument given to either the `decodeURI`, `encodeURI`, `encodeURIComponent`, or `decodeURIComponent` function was not valid, so that the function was unable encode or decode properly.

Examples

Encoding

Encoding replaces each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character. An `URIError` will be thrown if there is an attempt to encode a surrogate which is not part of a high-low pair, for example:

```
encodeURI("\uD800");
// "URIError: malformed URI sequence"

encodeURI("\uDFFF");
// "URIError: malformed URI sequence"
```

A high-low pair is OK. For example:

```
encodeURI("\uD800\uDFFF");
// "%F0%90%8F%BF"
```

Decoding

Decoding replaces each escape sequence in the encoded URI component with the character that it represents. If there isn't such a character, an error will be thrown:

```
decodeURIComponent("%E0%A4%A");
// "URIError: malformed URI sequence"
```

With proper input, this should usually look like something like this:

```
decodeURIComponent("JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B");
// "JavaScript_шеллы"
```

See also

- `URIError`
- `decodeURI`
- `encodeURI`
- `encodeURIComponent`
- `decodeURIComponent`

SyntaxError: missing] after element list

The JavaScript exception "missing] after element list" occurs when there is an error with the array initializer syntax somewhere. Likely there is a closing square bracket (`]`) or a comma (`,`) missing.

Message

```
SyntaxError: missing ] after element list (Firefox)
SyntaxError: Unexpected token ';'. Expected either a closing ']' or a ',' following an array element. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

There is an error with the array initializer syntax somewhere. Likely there is a closing square bracket (`]`) or a comma (`,`) missing.

Examples

Incomplete array initializer

```
const list = [1, 2,

const instruments = [
  "Ukulele",
  "Guitar",
  "Piano",
};

const data = [{ foo: "bar" } { bar: "foo" }];
```

Correct would be:

```
const list = [1, 2];

const instruments = ["Ukulele", "Guitar", "Piano"];

const data = [{ foo: "bar" }, { bar: "foo" }];
```

See also

- `Array`

SyntaxError: missing : after property id

The JavaScript exception "missing : after property id" occurs when objects are created using the object initializer syntax. A colon (`:`) separates keys and values for the object's properties. Somehow, this colon is missing or misplaced.

Message

```
SyntaxError: Invalid shorthand property initializer (V8-based)
SyntaxError: missing : after property id (Firefox)
SyntaxError: Unexpected token '='. Expected a ':' following the property name 'x'. (Safari)
SyntaxError: Unexpected token '+'. Expected an identifier as property name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

When creating objects with the object initializer syntax, a colon (`:`) separates keys and values for the object's properties.

```
const obj = { propertyKey: "value" };
```

Examples

Colons vs. equal signs

This code fails, as the equal sign can't be used this way in this object initializer syntax.

```
const obj = { propertyKey = "value" };
// SyntaxError: missing : after property id
```

Correct would be to use a colon, or to use square brackets to assign a new property after the object has been created already.

```
const obj = { propertyKey: "value" };
```

Or alternatively:

```
const obj = {};
obj.propertyKey = "value";
```

Computed properties

If you create a property key from an expression, you need to use square brackets. Otherwise the property name can't be computed:

```
const obj = { "b"+"ar": "foo" };
// SyntaxError: missing : after property id
```

Put the expression in square brackets `[]`:

```
const obj = { ["b" + "ar"]: "foo" };
```

See also

- Object initializer

SyntaxError: missing } after function body

The JavaScript exception "missing } after function body" occurs when there is a syntax mistake when creating a function somewhere. Check if any closing curly braces or parenthesis are in the correct order.

Message

```
SyntaxError: missing } after function body (Firefox)
```

Error type

`SyntaxError`

What went wrong?

There is a syntax mistake when creating a function somewhere. Also check if any closing curly braces or parenthesis are in the correct order. Indenting or formatting the code a bit nicer might also help you to see through the jungle.

Examples

Forgotten closing curly bracket

Oftentimes, there is a missing curly bracket in your function code:

```
function charge() {
  if (sunny) {
    useSolarCells();
  } else {
```

```
    promptBikeRide();
}
```

Correct would be:

```
function charge() {
  if (sunny) {
    useSolarCells();
  } else {
    promptBikeRide();
  }
}
```

It can be more obscure when using IIFEs or other constructs that use a lot of different parenthesis and curly braces, for example.

```
(function () {
  if (Math.random() < 0.01) {
    doSomething();
  }
)();
```

Oftentimes, indenting differently or double checking indentation helps to spot these errors.

```
(function () {
  if (Math.random() < 0.01) {
    doSomething();
  }
})();
```

See also

- Functions guide

SyntaxError: missing } after property list

The JavaScript exception "missing } after property list" occurs when there is a mistake in the object initializer syntax somewhere. Might be in fact a missing curly bracket, but could also be a missing comma.

Message

```
SyntaxError: missing } after property list (Firefox)
SyntaxError: Unexpected identifier 'c'. Expected '}' to end an object literal. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is a mistake in the object initializer syntax somewhere. Might be in fact a missing curly bracket, but could also be a missing comma, for example. Also check if any closing curly braces or parenthesis are in the correct order. Indenting or formatting the code a bit nicer might also help you to see through the jungle.

Examples

Forgotten comma

Oftentimes, there is a missing comma in your object initializer code:

```
const obj = {
  a: 1,
  b: { myProp: 2 }
  c: 3
};
```

Correct would be:

```
const obj = {
  a: 1,
  b: { myProp: 2 },
  c: 3,
};
```

See also

- Object initializer

SyntaxError: missing formal parameter

The JavaScript exception "missing formal parameter" occurs when your function declaration is missing valid parameters.

Message

```
SyntaxError: missing formal parameter (Firefox)
SyntaxError: Unexpected number '3'. Expected a parameter pattern or a ')' in parameter list. (Safari)
SyntaxError: Unexpected string literal "x". Expected a parameter pattern or a ')' in parameter list. (Safari)
```

Error type

`SyntaxError`

What went wrong?

"Formal parameter" is a fancy way of saying "function parameter". Your function declaration is missing valid parameters. In the declaration of a function, the parameters must be identifiers, not any value like numbers, strings, or objects. Declaring functions and calling functions are two separate steps. Declarations require identifier as parameters, and only when calling (invoking) the function, you provide the values the function should use.

In JavaScript, identifiers can contain only alphanumeric characters (or "$" or "_"), and may not start with a digit. An identifier differs from a **string** in that a string is data, while an identifier is part of the code.

Examples

Provide proper function parameters

Function parameters must be identifiers when setting up a function. All these function declarations fail, as they are providing values for their parameters:

```
function square(3) {
  return number * number;
}
// SyntaxError: missing formal parameter

function greet("Howdy") {
  return greeting;
}
// SyntaxError: missing formal parameter

function log({ obj: "value"}) {
  console.log(arg)
}
// SyntaxError: missing formal parameter
```

You will need to use identifiers in function declarations:

```
function square(number) {
  return number * number;
}

function greet(greeting) {
  return greeting;
}

function log(arg) {
  console.log(arg);
}
```

You can then call these functions with the arguments you like:

```
square(2); // 4

greet("Howdy"); // "Howdy"

log({ obj: "value" }); // { obj: "value" }
```

See also

- SyntaxError: redeclaration of formal parameter "x"

SyntaxError: missing = in const declaration

The JavaScript exception "missing = in const declaration" occurs when a const declaration was not given a value in the same statement (like `const RED_FLAG;`). You need to provide one (`const RED_FLAG = true`).

Message

```
SyntaxError: Missing initializer in const declaration (V8-based)
SyntaxError: missing = in const declaration (Firefox)
SyntaxError: Unexpected token ';'. const declared variable 'x' must have an initializer. (Safari)
```

Error type

`SyntaxError`

What went wrong?

A constant is a value that cannot be altered by the program during normal execution. It cannot change through re-assignment, and it can't be redeclared. In JavaScript, constants are declared using the `const` keyword. An initializer for a constant is required; that is, you must specify its value in the same statement in which

it's declared (which makes sense, given that it can't be changed later).

Examples

Missing const initializer

Unlike `var` or `let`, you must specify a value for a `const` declaration. This throws:

```
const COLUMNS;
// SyntaxError: missing = in const declaration
```

Fixing the error

There are multiple options to fix this error. Check what was intended to be achieved with the constant in question.

Adding a constant value

Specify the constant value in the same statement in which it's declared:

```
const COLUMNS = 80;
```

`const`, `let` or `var`?

Do not use `const` if you weren't meaning to declare a constant. Maybe you meant to declare a block-scoped variable with `let` or global variable with `var`. Both don't require an initial value.

```
let columns;
```

See also

- `const`
- `let`
- `var`

SyntaxError: missing name after . operator

The JavaScript exception "missing name after . operator" occurs when there is a problem with how the dot operator (`.`) is used for property access.

Message

```
SyntaxError: missing name after . operator (Firefox)
SyntaxError: Unexpected token '['. Expected a property name after '.'. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The dot operator (`.`) is used for property access. You will have to specify the name of the property that you want to access. For computed property access, you might need to change your property access from using a dot to using square brackets. These will allow you to compute an expression. Maybe you intended to do concatenation instead? A plus operator (+) is needed in that case. Please see the examples below.

Examples

Property access

Property accessors in JavaScript use either the dot (.) or square brackets (`[]`), but not both. Square brackets allow computed property access.

```
const obj = { foo: { bar: "baz", bar2: "baz2" } };
const i = 2;

obj.[foo].[bar]
// SyntaxError: missing name after . operator

obj.foo."bar"+i;
// SyntaxError: missing name after . operator
```

To fix this code, you need to access the object like this:

```
obj.foo.bar; // "baz"
// or alternatively
obj["foo"]["bar"]; // "baz"

// computed properties require square brackets
obj.foo["bar" + i]; // "baz2"
// or as template literal
obj.foo[`bar${i}`]; // "baz2"
```

Property access vs. concatenation

If you are coming from another programming language (like PHP), it is also easy to mix up the dot operator (`.`) and the concatenation operator (+).

```
console.log("Hello" . "world");

// SyntaxError: missing name after . operator
```

Instead you need to use a plus sign for concatenation:

```
console.log("Hello" + "World");
```

See also

- Property accessors

SyntaxError: missing) after argument list

The JavaScript exception "missing) after argument list" occurs when there is an error with how a function is called. This might be a typo, a missing operator, or an unescaped string.

Message

```
SyntaxError: missing ) after argument list (V8-based & Firefox)
SyntaxError: Unexpected identifier 'x'. Expected ')' to end an argument list. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

There is an error with how a function is called. This might be a typo, a missing operator, or an unescaped string, for example.

Examples

Because there is no "+" operator to concatenate the string, JavaScript expects the argument for the `log` function to be just `"PI: "`. In that case, it should be terminated by a closing parenthesis.

```
console.log("PI: " Math.PI);
// SyntaxError: missing ) after argument list
```

You can correct the `log` call by adding the `+` operator:

```
console.log("PI: " + Math.PI);
// "PI: 3.141592653589793"
```

Alternatively, you can consider using a template literal, or take advantage of the fact that `console.log` accepts multiple parameters:

```
console.log(`PI: ${Math.PI}`);
console.log("PI:", Math.PI);
```

Unterminated strings

```
console.log('"Java" + "Script" = \"' + "Java" + 'Script\");
// SyntaxError: missing ) after argument list
```

Here JavaScript thinks that you meant to have `);` inside the string and ignores it, and it ends up not knowing that you meant the `);` to end the function `console.log`. To fix this, we could put a `'` after the "Script" string:

```
console.log('"Java" + "Script" = "' + "Java" + 'Script"');
// '"Java" + "Script" = "JavaScript"'
```

See also

- Functions guide

SyntaxError: missing) after condition

The JavaScript exception "missing) after condition" occurs when there is an error with how an `if` condition is written. It must appear in parenthesis after the `if` keyword.

Message

```
SyntaxError: missing ) after condition (Firefox)
SyntaxError: Unexpected token '{'. Expected ')' to end an 'if' condition. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is an error with how an `if` condition is written. In any programming language, code needs to make decisions and carry out actions accordingly depending on different inputs. The if statement executes a statement if a specified condition is truthy. In JavaScript, this condition must appear in parenthesis after the `if` keyword, like this:

```
if (condition) {
  // do something if the condition is true
}
```

Examples

Missing parenthesis

It might just be an oversight, carefully check all you parenthesis in your code.

```
if (Math.PI < 3 {
  console.log("wait what?");
}

// SyntaxError: missing ) after condition
```

To fix this code, you would need to add a parenthesis that closes the condition.

```
if (Math.PI < 3) {
  console.log("wait what?");
}
```

Misused is keyword

If you are coming from another programming language, it is also easy to add keywords that don't mean the same or have no meaning at all in JavaScript.

```
if (done is true) {
 console.log("we are done!");
}

// SyntaxError: missing ) after condition
```

Instead you need to use a correct comparison operator. For example:

```
if (done === true) {
  console.log("we are done!");
}
```

Or even better:

```
if (done) {
  console.log("we are done!");
}
```

See also

- `if...else`
- Relational operators
- Learn: Making decisions in your code — conditionals

TypeError: More arguments needed

The JavaScript exception "more arguments needed" occurs when there is an error with how a function is called. More arguments need to be provided.

Message

```
TypeError: Object prototype may only be an Object or null: undefined (V8-based)
TypeError: Object.create requires at least 1 argument, but only 0 were passed (Firefox)
TypeError: Object.setPrototypeOf requires at least 2 arguments, but only 0 were passed (Firefox)
TypeError: Object.defineProperties requires at least 1 argument, but only 0 were passed (Firefox)
TypeError: Object prototype may only be an Object or null. (Safari)
```

Error type

`TypeError`.

What went wrong?

There is an error with how a function is called. More arguments need to be provided.

Examples

Required arguments not provided

The `Object.create()` method requires at least one argument and the `Object.setPrototypeOf()` method requires at least two arguments:

```
const obj = Object.create();
// TypeError: Object.create requires at least 1 argument, but only 0 were passed

const obj2 = Object.setPrototypeOf({});
// TypeError: Object.setPrototypeOf requires at least 2 arguments, but only 1 were passed
```

You can fix this by setting `null` as the prototype, for example:

```
const obj = Object.create(null);

const obj2 = Object.setPrototypeOf({}, null);
```

See also

- Functions guide

RangeError: repeat count must be non-negative

The JavaScript exception "repeat count must be non-negative" occurs when the `String.prototype.repeat()` method is used with a `count` argument that is a negative number.

Message

```
RangeError: Invalid count value: -1 (V8-based)
RangeError: repeat count must be non-negative (Firefox)
RangeError: String.prototype.repeat argument must be greater than or equal to 0 and not be Infinity (Safari)
```

Error type

`RangeError`

What went wrong?

The `String.prototype.repeat()` method has been used. It has a `count` parameter indicating the number of times to repeat the string. It must be between 0 and less than positive `Infinity` and cannot be a negative number. The range of allowed values can be described like this: [0, +∞).

Examples

Invalid cases

```
"abc".repeat(-1); // RangeError
```

Valid cases

```
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count will be converted to integer)
```

See also

- `String.prototype.repeat()`

TypeError: "x" is not a non-null object

The JavaScript exception "is not a non-null object" occurs when an object is expected somewhere and wasn't provided. `null` is not an object and won't work.

Message

```
TypeError: Property description must be an object: x (V8-based)
TypeError: Property descriptor must be an object, got "x" (Firefox)
TypeError: Property description must be an object. (Safari)
```

Error type

`TypeError`

What went wrong?

An object is expected somewhere and wasn't provided. `null` is not an object and won't work. You must provide a proper object in the given situation.

Examples

Property descriptor expected

When methods like `Object.create()` or `Object.defineProperty()` and `Object.defineProperties()` are used, the optional descriptor parameter expects a property descriptor object. Providing no object (like just a number), will throw an error:

```
Object.defineProperty({}, "key", 1);
// TypeError: 1 is not a non-null object
```

```
Object.defineProperty({}, "key", null);
// TypeError: null is not a non-null object
```

A valid property descriptor object might look like this:

```
Object.defineProperty({}, "key", { value: "foo", writable: false });
```

See also

- `Object.create()`
- `Object.defineProperty()`
- `Object.defineProperties()`

TypeError: null/undefined has no properties

The JavaScript exception "null (or undefined) has no properties" occurs when you attempt to access properties of `null` and `undefined`. They don't have any.

Message

```
TypeError: Cannot read properties of undefined (reading 'x') (V8-based)
TypeError: Cannot destructure 'x' as it is undefined. (V8-based)
TypeError: Cannot destructure property 'x' of 'y' as it is undefined. (V8-based)
TypeError: null has no properties (Firefox)
TypeError: undefined has no properties (Firefox)
TypeError: undefined is not an object (evaluating 'undefined.x') (Safari)
TypeError: Right side of assignment cannot be destructured (Safari)
```

Error type

`TypeError`.

What went wrong?

Both `null` and `undefined` have no properties you could access. Therefore, you cannot use property accessors on them, or destructure them.

Examples

null and undefined have no properties

```
null.foo;
// TypeError: null has no properties

undefined.bar;
// TypeError: undefined has no properties
```

See also

- `null`
- `undefined`

SyntaxError: missing variable name

The JavaScript exception "missing variable name" is a common error. It is usually caused by omitting a variable name or a typographic error.

Message

```
SyntaxError: missing variable name (Firefox)
SyntaxError: Unexpected token '='. Expected a parameter pattern or a ')' in parameter list. (Safari)
```

Error type

`SyntaxError`

What went wrong?

A variable is missing a name. The cause is most likely a typo or a forgotten variable name. Make sure that you've provided the name of the variable before the `=` sign.

When declaring multiple variables at the same time, make sure that the previous lines/declaration does not end with a comma instead of a semicolon.

Examples

Missing a variable name

```
const = "foo";
```

It is easy to forget to assign a name for your variable!

```
const description = "foo";
```

Reserved keywords can't be variable names

There are a few variable names that are reserved keywords. You can't use these. Sorry :(

```
const debugger = "whoop";
// SyntaxError: missing variable name
```

Declaring multiple variables

Pay special attention to commas when declaring multiple variables. Is there an excess comma, or did you use commas instead of semicolons? Did you remember to assign values for all your `const` variables?

```
let x, y = "foo",
const z, = "foo"

const first = document.getElementById("one"),
const second = document.getElementById("two"),

// SyntaxError: missing variable name
```

The fixed version:

```
let x,
  y = "foo";
const z = "foo";

const first = document.getElementById("one");
const second = document.getElementById("two");
```

Arrays

`Array` literals in JavaScript need square brackets around the values. This won't work:

```
const arr = 1,2,3,4,5;
// SyntaxError: missing variable name
```

This would be correct:

```
const arr = [1, 2, 3, 4, 5];
```

See also

- Lexical grammar
- `var`
- Grammar and types guide

TypeError: can't delete non-configurable array element

The JavaScript exception "can't delete non-configurable array element" occurs when it was attempted to shorten the length of an array, but one of the array's elements is non-configurable.

Message

```
TypeError: Cannot delete property '1' of [object Array] (V8-based)
TypeError: can't delete non-configurable array element (Firefox)
TypeError: Unable to delete property. (Safari)
```

Error type

`TypeError`

What went wrong?

It was attempted to shorten the length of an array, but one of the array's elements is non-configurable. When shortening an array, the elements beyond the new array length will be deleted, which failed in this situation.

The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `writable`) can be changed.

Usually, properties in an object created by an array initializer are configurable. However, for example, when using `Object.defineProperty()`, the property isn't configurable by default.

Examples

Non-configurable properties created by Object.defineProperty

The `Object.defineProperty()` creates non-configurable properties by default if you haven't specified them as configurable.

```
"use strict";
const arr = [];
Object.defineProperty(arr, 0, { value: 0 });
Object.defineProperty(arr, 1, { value: "1" });

arr.length = 1;
// TypeError: can't delete non-configurable array element
```

You will need to set the elements as configurable, if you intend to shorten the array.

```
"use strict";
const arr = [];
Object.defineProperty(arr, 0, { value: 0, configurable: true });
Object.defineProperty(arr, 1, { value: "1", configurable: true });

arr.length = 1;
```

Sealed Arrays

The `Object.seal()` function marks all existing elements as non-configurable.

```
"use strict";
const arr = [1, 2, 3];
Object.seal(arr);

arr.length = 1;
// TypeError: can't delete non-configurable array element
```

You either need to remove the `Object.seal()` call, or make a copy of it. In case of a copy, shortening the copy of the array does not modify the original array length.

```
"use strict";
const arr = [1, 2, 3];
Object.seal(arr);

// Copy the initial array to shorten the copy
const copy = Array.from(arr);
copy.length = 1;
// arr.length === 3
```

See also

- [[Configurable]]
- `length`
- `Object.defineProperty()`
- `Object.seal()`

TypeError: "x" is not a constructor

The JavaScript exception "is not a constructor" occurs when there was an attempt to use an object or a variable as a constructor, but that object or variable is not a constructor.

Message

```
TypeError: x is not a constructor (V8-based & Firefox & Safari)
```

Error type

`TypeError`

What went wrong?

There was an attempt to use an object or a variable as a constructor, but that object or variable is not a constructor. See constructor or the `new` operator for more information on what a constructor is.

There are many global objects, like `String` or `Array`, which are constructable using `new`. However, some global objects are not and their properties and methods are static. The following JavaScript standard built-in objects are not a constructor: `Math`, `JSON`, `Symbol`, `Reflect`, `Intl`, `Atomics`.

Generator functions cannot be used as constructors either.

Examples

Invalid cases

```
const Car = 1;
new Car();
// TypeError: Car is not a constructor

new Math();
// TypeError: Math is not a constructor

new Symbol();
// TypeError: Symbol is not a constructor

function* f() {}
const obj = new f();
// TypeError: f is not a constructor
```

A car constructor

Suppose you want to create an object type for cars. You want this type of object to be called `Car`, and you want it to have properties for make, model, and year. To do this, you would write the following function:

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
```

Now you can create an object called `myCar` as follows:

```
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

In Promises

When returning an immediately-resolved or immediately-rejected Promise, you do not need to create a `new Promise(...)` and act on it. Instead, use the `Promise.resolve()` or `Promise.reject()` static methods.

This is not legal (the `Promise` constructor is not being called correctly) and will throw a `TypeError: this is not a constructor` exception:

```
function fn() {
  return new Promise.resolve(true);
}
```

This is legal, but unnecessarily long:

```
function fn() {
  return new Promise((resolve, reject) => {
    resolve(true);
  });
}
```

Instead, return the static method:

```
function resolveAlways() {
  return Promise.resolve(true);
}

function rejectAlways() {
  return Promise.reject(new Error());
}
```

See also

- constructor
- `new`

TypeError: "x" is not a function

The JavaScript exception "is not a function" occurs when there was an attempt to call a value from a function, but the value is not actually a function.

Message

```
TypeError: "x" is not a function. (V8-based & Firefox & Safari)
```

Error type

`TypeError`

What went wrong?

It attempted to call a value from a function, but the value is not actually a function. Some code expects you to provide a function, but that didn't happen.

Maybe there is a typo in the function name? Maybe the object you are calling the method on does not have this function? For example, JavaScript `Objects` have no `map` function, but the JavaScript `Array` object does.

There are many built-in functions in need of a (callback) function. You will have to provide a function in order to have these methods working properly:

- When working with `Array` or `TypedArray` objects:
  - `Array.prototype.every()`, `Array.prototype.some()`, `Array.prototype.forEach()`, `Array.prototype.map()`, `Array.prototype.filter()`, `Array.prototype.reduce()`, `Array.prototype.reduceRight()`, `Array.prototype.find()`
- When working with `Map` and `Set` objects:
  - `Map.prototype.forEach()` and `Set.prototype.forEach()`

Examples

A typo in the function name

In this case, which happens way too often, there is a typo in the method name:

```
const x = document.getElementByID("foo");
// TypeError: document.getElementByID is not a function
```

The correct function name is `getElementById`:

```
const x = document.getElementById("foo");
```

Function called on the wrong object

For certain methods, you have to provide a (callback) function and it will work on specific objects only. In this example, `Array.prototype.map()` is used, which will work with `Array` objects only.

```
const obj = { a: 13, b: 37, c: 42 };

obj.map((num) => num * 2);

// TypeError: obj.map is not a function
```

Use an array instead:

```
const numbers = [1, 4, 9];

numbers.map((num) => num * 2); // [2, 8, 18]
```

Function shares a name with a pre-existing property

Sometimes when making a class, you may have a property and a function with the same name. Upon calling the function, the compiler thinks that the function ceases to exist.

```
function Dog() {
  this.age = 11;
  this.color = "black";
  this.name = "Ralph";
  return this;
}

Dog.prototype.name = function (name) {
  this.name = name;
  return this;
};

const myNewDog = new Dog();
myNewDog.name("Cassidy"); // TypeError: myNewDog.name is not a function
```

Use a different property name instead:

```
function Dog() {
  this.age = 11;
  this.color = "black";
  this.dogName = "Ralph"; // Using this.dogName instead of .name
  return this;
}

Dog.prototype.name = function (name) {
  this.dogName = name;
  return this;
};

const myNewDog = new Dog();
myNewDog.name("Cassidy"); // Dog { age: 11, color: 'black', dogName: 'Cassidy' }
```

Using parentheses for multiplication

In math, you can write 2 × (3 + 5) as 2*(3 + 5) or just 2(3 + 5).

Using the latter will throw an error:

```
const sixteen = 2(3 + 5);
console.log(`2 x (3 + 5) is ${sixteen}`);
// Uncaught TypeError: 2 is not a function
```

You can correct the code by adding a `*` operator:

```
const sixteen = 2 * (3 + 5);
console.log(`2 x (3 + 5) is ${sixteen}`);
// 2 x (3 + 5) is 16
```

Import the exported module correctly

Ensure you are importing the module correctly.

An example helpers library (`helpers.js`)

```
function helpers() {}

helpers.groupBy = function (objectArray, property) {
  return objectArray.reduce((acc, obj) => {
    const key = obj[property];
    acc[key] ??= [];
    acc[key].push(obj);
    return acc;
  }, {});
};

export default helpers;
```

The correct import usage (`App.js`):

```
import helpers from "./helpers";
```

See also

- Functions

RangeError: argument is not a valid code point

The JavaScript exception "Invalid code point" occurs when `NaN` values, negative Integers (-1), non-Integers (5.4), or values larger than 0x10FFFF (1114111) are used with `String.fromCodePoint()`.

Message

```
RangeError: Invalid code point -1 (V8-based)
RangeError: -1 is not a valid code point (Firefox)
RangeError: Arguments contain a value that is out of range of code points (Safari)
```

Error type

`RangeError`

What went wrong?

`String.fromCodePoint()` throws this error when passed `NaN` values, negative Integers (-1), non-Integers (5.4), or values larger than 0x10FFFF (1114111).

A code point is a value in the Unicode codespace; that is, the range of integers from `0` to `0x10FFFF`.

Examples

Invalid cases

```
String.fromCodePoint("_"); // RangeError
String.fromCodePoint(Infinity); // RangeError
String.fromCodePoint(-1); // RangeError
String.fromCodePoint(3.14); // RangeError
String.fromCodePoint(3e-2); // RangeError
String.fromCodePoint(NaN); // RangeError
```

Valid cases

```
String.fromCodePoint(42); // "*"
String.fromCodePoint(65, 90); // "AZ"
String.fromCodePoint(0x404); // 'Є' (U+0404)
String.fromCodePoint(0x2f804); // '你' (U+2F804)
String.fromCodePoint(194564); // '你'
String.fromCodePoint(0x1d306, 0x61, 0x1d307); // '𝌆a𝌇'
```

See also

- `String.fromCodePoint()`

ReferenceError: "x" is not defined

The JavaScript exception "*variable* is not defined" occurs when there is a non-existent variable referenced somewhere.

Message

```
ReferenceError: "x" is not defined (V8-based & Firefox)
ReferenceError: Can't find variable: x (Safari)
```

Error type

`ReferenceError`.

What went wrong?

There is a non-existent variable referenced somewhere. This variable needs to be declared, or you need to make sure it is available in your current script or scope.

> **Note:** When loading a library (such as jQuery), make sure it is loaded before you access library variables, such as "$". Put the `<script>` element that loads the library before your code that uses it.

Examples

Variable not declared

```
foo.substring(1); // ReferenceError: foo is not defined
```

The "foo" variable isn't defined anywhere. It needs to be some string, so that the `String.prototype.substring()` method will work.

```
const foo = "bar";
foo.substring(1); // "ar"
```

Wrong scope

A variable needs to be available in the current context of execution. Variables defined inside a function cannot be accessed from anywhere outside the function, because the variable is defined only in the scope of the function

```
function numbers() {
  const num1 = 2;
  const num2 = 3;
  return num1 + num2;
}

console.log(num1); // ReferenceError num1 is not defined.
```

However, a function can access all variables and functions defined inside the scope in which it is defined. In other words, a function defined in the global scope can access all variables defined in the global scope.

```
const num1 = 2;
const num2 = 3;

function numbers() {
  return num1 + num2;
}

console.log(numbers()); // 5
```

See also

- Scope
- Declaring variables in the JavaScript Guide
- Function scope in the JavaScript Guide

SyntaxError: parameter after rest parameter

The JavaScript exception "parameter after rest parameter" occurs when a rest parameter is followed by anything else in a parameter list, including another rest parameter, a formal parameter, or a trailing comma.

Message

```
SyntaxError: Rest parameter must be last formal parameter (V8-based)
SyntaxError: parameter after rest parameter (Firefox)
SyntaxError: Unexpected token ','. Rest parameter should be the last parameter in a function declaration. (Safari)
```

Error type

`SyntaxError`

What went wrong?

A rest parameter must be the last parameter in a function definition. This is because the rest parameter collects all the remaining arguments passed to the function, so it doesn't make sense to have any parameters after it. The next non-whitespace character must be the closing parenthesis of the parameter list.

Examples

Invalid cases

```
function replacer(match, ...groups, offset, string) {}

function doSomething(
  arg1,
  arg2,
  ...otherArgs, // Accidental trailing comma
) {}
```

Valid cases

```
function replacer(match, ...args) {
  const offset = args.at(-2);
  const string = args.at(-1);
}

function doSomething(arg1, arg2, ...otherArgs) {}
```

See also

- Functions
- Rest parameters

RangeError: precision is out of range

The JavaScript exception "precision is out of range" occurs when a number that's outside of the allowed range was passed into `toExponential`, `toFixed`, or `toPrecision`.

Message

```
RangeError: toExponential() argument must be between 0 and 100 (V8-based & Safari)
RangeError: toFixed() digits argument must be between 0 and 100 (V8-based & Safari)
RangeError: toPrecision() argument must be between 1 and 100 (V8-based & Safari)
RangeError: precision -1 out of range (Firefox)
```

Error type

`RangeError`

What went wrong?

There was an out of range precision argument in one of these methods:

- `Number.prototype.toExponential()`, which requires the arguments to be between 0 and 100, inclusive.
- `Number.prototype.toFixed()`, which requires the arguments to be between 0 and 100, inclusive.
- `Number.prototype.toPrecision()`, which requires the arguments to be between 1 and 100, inclusive.

Examples

Invalid cases

```
(77.1234).toExponential(-1); // RangeError
(77.1234).toExponential(101); // RangeError

(2.34).toFixed(-100); // RangeError
(2.34).toFixed(1001); // RangeError

(1234.5).toPrecision(-1); // RangeError
(1234.5).toPrecision(101); // RangeError
```

Valid cases

```
(77.1234).toExponential(4); // 7.7123e+1
(77.1234).toExponential(2); // 7.71e+1

(2.34).toFixed(1); // 2.3
(2.35).toFixed(1); // 2.4 (note that it rounds up in this case)

(5.123456).toPrecision(5); // 5.1235
(5.123456).toPrecision(2); // 5.1
(5.123456).toPrecision(1); // 5
```

See also

- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`

TypeError: Initializing an object twice is an error with private fields/methods

The JavaScript exception "Initializing an object twice is an error with private fields/methods" occurs when an object that was created via a class constructor goes through the class construction again, and the class contains a private element. This is usually caused by the return override trick.

Message

```
TypeError: Cannot initialize #x twice on the same object (V8-based)
TypeError: Initializing an object twice is an error with private fields (Firefox)
TypeError: Cannot redefine existing private field (evaluating 'super(o)') (Safari)

TypeError: Cannot initialize private methods of class X twice on the same object (V8-based)
TypeError: Initializing an object twice is an error with private methods (Firefox)
TypeError: Cannot install same private methods on object more than once (evaluating 'super(o)') (Safari)
```

Error type

`TypeError`

What went wrong?

For any object, if it already contains a private field or method, it would be an error to install the same field again. Private elements are installed on the value of `this` when the class constructor is called, so this error could happen if the `this` value is an already-constructed instance of this class.

Usually, `this` in a constructor is a newly created object which doesn't have any preexisting properties. However, it can be overridden by the return value of the base class. If the base class returns another object, that object would replace the current object as the value of `this`:

```
class Base {
  constructor(o) {
    // This object will become the this value of any subclass
    return o;
  }
}

class Derived extends Base {
  #x = 0;
}
```

If you call `new Derived(anyObject)`, where `anyObject` is not an instance of `Derived`, the `Derived` constructor will be called with `anyObject` as the `this` value, and therefore install the `#x` private field on `anyObject`. This is the "return override" trick, which allows you to define arbitrary information on unrelated objects. However, if you call `new Derived(new Derived())`, or call `new Derived(anyObject)` again, the `Derived` constructor will try to install the `#x` private field again on an object which already has the `#x` private field, causing this error.

See also

- Classes
- Private elements

TypeError: getting private setter-only property

The JavaScript exception "getting private setter-only property" occurs when reading the value of a private element for which only a setter is defined.

Message

```
TypeError: '#x' was defined without a getter (V8-based)
TypeError: getting private setter-only property (Firefox)
TypeError: Trying to access an undefined private getter (Safari)
```

Error type

`TypeError`

What went wrong?

There is an attempt to get the value of a private element for which only a setter is specified. Unlike normal objects, where an undefined getter just means the property would always return undefined, for private elements, this is an error.

Examples

Private element with no getter

Here, `#name` has no getter, so trying to read it using `this.#name` will throw an error.

```
class Person {
  set #name(value) {}

  get name() {
    return this.#name;
  }
}

const person = new Person();
console.log(person.name);
```

It's not usual for a private element to have a setter without a getter. Either add a getter or refactor your program so the setter can be removed too.

See also

- Private elements
- `set`

AggregateError: No Promise in Promise.any was resolved

The JavaScript exception "No Promise in Promise.any was resolved" occurs when all promises passed to `Promise.any()` are rejected. It is the only built-in usage of `AggregateError`.

Message

```
AggregateError: All promises were rejected (V8-based)
AggregateError: No Promise in Promise.any was resolved (Firefox)
```

```
AggregateError (Safari)
```

Error type

`AggregateError`

What went wrong?

`Promise.any()` only rejects when all promises passed to it are rejected. You should access `errors` to get the array of rejection reasons. See Using promises for more information on how to handle asynchronously rejected promises. This error is also raised when `Promise.any()` receives an empty iterable.

Examples

Empty iterable

```
Promise.any([]).catch((error) => {
  console.error(error); // AggregateError: No Promise in Promise.any was resolved
});
```

Handling all rejections

```
const promises = [
  fetch("/data-location1"),
  fetch("/data-location1"),
  fetch("/data-location1"),
];

Promise.any(promises)
  .then((value) => console.log(value))
  .catch((error) => {
    // None of the fetches were successful
    for (const e of error.errors) {
      console.error(e);
    }
  });

// Using await
async function fetchFirstSuccessful() {
  try {
    const value = await Promise.any(promises);
    console.log(value);
  } catch (error) {
    for (const e of error.errors) {
      console.error(e);
    }
  }
}
```

See also

- `AggregateError`
- `Promise.any()`

Error: Permission denied to access property "x"

The JavaScript exception "Permission denied to access property" occurs when there was an attempt to access an object for which you have no permission.

Message

```
DOMException: Blocked a frame with origin "x" from accessing a cross-origin frame. (Chromium-based)
DOMException: Permission denied to access property "x" on cross-origin object (Firefox)
SecurityError: Blocked a frame with origin "x" from accessing a cross-origin frame. Protocols, domains, and ports must match. (Safari)
```

Error type

`DOMException`.

What went wrong?

There was attempt to access an object for which you have no permission. This is likely an `<iframe>` element loaded from a different domain for which you violated the same-origin policy.

Examples

No permission to access document

```
<iframe id="myframe" src="http://www1.w3c-test.org/common/blank.html"></iframe>
```

```
console.log(frames[0].document);
// Error: Permission denied to access property "document"
```

See also

- `<iframe>`
- Same-origin policy

TypeError: "x" is read-only

The JavaScript strict mode-only exception "is read-only" occurs when a global variable or object property that was assigned to is a read-only property.

Message

```
TypeError: Cannot assign to read only property 'x' of #<Object> (V8-based)
TypeError: "x" is read-only (Firefox)
TypeError: Attempted to assign to readonly property. (Safari)
```

Error type

`TypeError` in strict mode only.

What went wrong?

The global variable or object property that was assigned to is a read-only property. (Technically, it is a non-writable data property.)

This error happens only in strict mode code. In non-strict code, the assignment is silently ignored.

Examples

Invalid cases

Read-only properties are not super common, but they can be created using `Object.defineProperty()` or `Object.freeze()`.

```
"use strict";
const obj = Object.freeze({ name: "Elsa", score: 157 });
obj.score = 0; // TypeError

("use strict");
Object.defineProperty(this, "LUNG_COUNT", { value: 2, writable: false });
LUNG_COUNT = 3; // TypeError

("use strict");
const frozenArray = Object.freeze([0, 1, 2]);
frozenArray[0]++; // TypeError
```

There are also a few read-only properties built into JavaScript. Maybe you tried to redefine a mathematical constant.

```
"use strict";
Math.PI = 4; // TypeError
```

Sorry, you can't do that.

The global variable `undefined` is also read-only, so you can't silence the infamous "undefined is not a function" error by doing this:

```
"use strict";
undefined = function () {}; // TypeError: "undefined" is read-only
```

Valid cases

```
"use strict";
let obj = Object.freeze({ name: "Score", points: 157 });
obj = { name: obj.name, points: 0 }; // replacing it with a new object works
```

See also

- `Object.defineProperty()`
- `Object.freeze()`

SyntaxError: redeclaration of formal parameter "x"

The JavaScript exception "redeclaration of formal parameter" occurs when the same variable name occurs as a function parameter and is then redeclared using a `let` assignment in a function body again.

Message

```
SyntaxError: Identifier "x" has already been declared (V8-based)
SyntaxError: redeclaration of formal parameter "x" (Firefox)
SyntaxError: Cannot declare a let variable twice: 'x'. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The same variable name occurs as a function parameter and is then redeclared using a `let` assignment in a function body again. Redeclaring the same variable within the same function or block scope using `let` is not allowed in JavaScript.

Examples

Redeclared argument

In this case, the variable "arg" redeclares the argument.

```
function f(arg) {
  let arg = "foo";
}

// SyntaxError: redeclaration of formal parameter "arg"
```

If you want to change the value of "arg" in the function body, you can do so, but you do not need to declare the same variable again. In other words: you can omit the `let` keyword. If you want to create a new variable, you need to rename it as conflicts with the function parameter already.

```
function f(arg) {
  arg = "foo";
}

function g(arg) {
  let bar = "foo";
}
```

See also

- `let`
- `const`
- `var`
- Grammar and types guide

TypeError: Reduce of empty array with no initial value

The JavaScript exception "reduce of empty array with no initial value" occurs when a reduce function is used.

Message

```
TypeError: Reduce of empty array with no initial value (V8-based & Firefox & Safari)
```

Error type

`TypeError`

What went wrong?

In JavaScript, there are several reduce functions:

- `Array.prototype.reduce()`, `Array.prototype.reduceRight()` and
- `TypedArray.prototype.reduce()`, `TypedArray.prototype.reduceRight()`).

These functions optionally take an `initialValue` (which will be used as the first argument to the first call of the `callback`). However, if no initial value is provided, it will use the first element of the `Array` or `TypedArray` as the initial value. This error is raised when an empty array is provided because no initial value can be returned in that case.

Examples

Invalid cases

This problem appears frequently when combined with a filter (`Array.prototype.filter()`, `TypedArray.prototype.filter()`) which will remove all elements of the list. Thus leaving none to be used as the initial value.

```
const ints = [0, -1, -2, -3, -4, -5];
ints
  .filter((x) => x > 0) // removes all elements
  .reduce((x, y) => x + y); // no more elements to use for the initial value.
```

Similarly, the same issue can happen if there is a typo in a selector, or an unexpected number of elements in a list:

```
const names = document.getElementsByClassName("names");
const nameList = Array.prototype.reduce.call(
  names,
  (acc, name) => `${acc}, ${name}`,
);
```

Valid cases

These problems can be solved in two different ways.

One way is to actually provide an `initialValue` as the neutral element of the operator, such as 0 for the addition, 1 for a multiplication, or an empty string for a concatenation.

```
const ints = [0, -1, -2, -3, -4, -5];
ints
  .filter((x) => x > 0) // removes all elements
  .reduce((x, y) => x + y, 0); // the initial value is the neutral element of the addition
```

Another way would be to handle the empty case, either before calling `reduce`, or in the callback after adding an unexpected dummy initial value.

```
const names = document.getElementsByClassName("names");

let nameList1 = "";
if (names.length >= 1) {
  nameList1 = Array.prototype.reduce.call(
    names,
    (acc, name) => `${acc}, ${name}`,
  );
}
// nameList1 === "" when names is empty.

const nameList2 = Array.prototype.reduce.call(
  names,
  (acc, name) => {
    if (acc === "")
      // initial value
      return name;
    return `${acc}, ${name}`;
  },
  "",
);
// nameList2 === "" when names is empty.
```

See also

- `Array.prototype.reduce()`
- `Array.prototype.reduceRight()`
- `TypedArray.prototype.reduce()`
- `TypedArray.prototype.reduceRight()`
- `Array`
- `TypedArray`
- `Array.prototype.filter()`
- `TypedArray.prototype.filter()`

SyntaxError: \ at end of pattern

The JavaScript exception "\ at end of pattern" occurs when a regular expression pattern ends with an unescaped backslash (`\`). In a regex literal, the backslash would cause the closing slash `/` to be a literal character, so this can only appear when using the `RegExp()` constructor.

Message

```
SyntaxError: Invalid regular expression: /\/: \ at end of pattern (V8-based)
SyntaxError: \ at end of pattern (Firefox)
SyntaxError: Invalid regular expression: \ at end of pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

A backslash cannot appear literally in a regular expression. It either precedes another character to escape it, or is itself escaped by another backslash. A backslash at the end of a regular expression pattern is invalid because it is not escaping anything.

Examples

Double-escaping backslashes

This error can only happen when using the `RegExp()` constructor. Consider the following code, which intends to match a single backslash character:

```
const pattern = new RegExp("\\");
```

In JavaScript strings, backslashes are also escape sequences. Therefore, the two backslashes in the string literal `"\\"` are interpreted as a single backslash. The `RegExp()` constructor then only sees a single backslash character in the regex source. To fix this, you need to double-escape the backslash:

```
const pattern = new RegExp("\\\\");
```

The four backslashes in the string literal represent two backslashes in the regex source, which then becomes a character escape for a single backslash literal character.

See also

- Regular expressions
- Escape sequences

SyntaxError: character class escape cannot be used in class range in regular expression

The JavaScript exception "character class escape cannot be used in class range in regular expression" occurs when a Unicode-aware regular expression pattern contains a character class where a boundary of a character range is another character class, such as a character class escape.

Message

```
SyntaxError: Invalid regular expression: /[\s-1]/u: Invalid character class (V8-based)
SyntaxError: character class escape cannot be used in class range in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid range in character class for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

A character class can specify a range of characters by using a hyphen (`-`) between two characters. For example, `[a-z]` matches any lowercase letter from `a` to `z`. The two bounds of the range must represent single characters in order for the range to make sense. If one of the bounds actually represents multiple characters, an error is generated. In non-v-mode character classes, only character class escapes are allowed inside character classes; in v-mode character classes, this can also happen if one the bounds is another `[...]` character class.

In Unicode-unaware mode, this syntax causes the `-` to become a literal character instead of generating an error, but this is a deprecated syntax and you should not rely on it.

Examples

Invalid cases

```
/[\s-_]/u; // \s is a character class escape for whitespace
/[A-\D]/u; // \D is a character class escape for non-digits
/[\p{L}-\p{N}]/u; // \p{L} is a character class escape for Unicode letters
/[[A-z]-_]/v; // In unicodeSets mode, character classes can be nested
```

Valid cases

```
// Put the hyphen at the start of the character class,
// so it matches the literal character
/[-\s_]/u;
// Escape the hyphen so it also matches the literal character
/[\s\-_]/u;
// Remove the backslash so the bound is a literal character
/[A-D]/u;
// Remove the hyphen so the two bounds represent two alternatives
/[\p{L}\p{N}]/u;
// Use -- in unicodeSets mode, which represents set subtraction
/[[A-z]--_]/v;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

SyntaxError: duplicate capture group name in regular expression

The JavaScript exception "duplicate capture group name in regular expression" occurs when a regular expression pattern contains two or more named capturing groups with the same name, and these capture groups could be matched at the same time.

Message

```
SyntaxError: Invalid regular expression: /(?<a>)(?<a>)/: Duplicate capture group name (V8-based)
SyntaxError: duplicate capture group name in regular expression (Firefox)
SyntaxError: Invalid regular expression: duplicate group specifier name (Safari)
```

Error type

`SyntaxError`

What went wrong?

All named capturing groups in a regular expression pattern must have unique names. A more recent feature allows named capturing groups to share names, as long as they belong to different disjunction alternatives and cannot be matched at the same time (see browser compatibility for this). However, it is still an error if the named capturing groups with the same name could be matched at the same time, as that would make other features, such as named backreferences, ambiguous.

Examples

Invalid cases

```
/(?<name>\w+) (?<name>\w+)/;
```

Valid cases

```
/(?<firstName>\w+) (?<lastName>\w+)/;
/(?<year>\d{4})-\d{2}|\d{2}-(?<year>\d{4})/;
```

See also

- Regular expressions
- Named capturing group: `(?<name>...)`

SyntaxError: incomplete quantifier in regular expression

The JavaScript exception "incomplete quantifier in regular expression" occurs when a regular expression pattern contains a `{`, but it does not start a valid quantifier.

Message

```
SyntaxError: Invalid regular expression: /1{/u: Incomplete quantifier (V8-based)
SyntaxError: incomplete quantifier in regular expression (Firefox)
SyntaxError: Invalid regular expression: incomplete {} quantifier for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

A `{` character in a regular expression pattern starts a quantifier. A valid quantifier is in the form `{n}`, `{n,}`, or `{n,m}`, where `n` and `m` are non-negative integers and `m` is not less than `n`. If the `{` character does not start a valid quantifier, a `SyntaxError` occurs.

In Unicode-unaware mode, this syntax causes the `{` to become a literal character instead of generating an error, but this is a deprecated syntax and you should not rely on it.

Examples

Invalid cases

```
/1{/u;
/1{a}/u;
/1{}/u;
/1{1,2,3}/u;
/1{1, 2}/u;
```

Valid cases

```
/1{1}/u;
/1{1,}/u;
/1{1,2}/u;
```

See also

- Regular expressions
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

SyntaxError: invalid capture group name in regular expression

The JavaScript exception "invalid capture group name in regular expression" occurs when a named capturing group or named backreference contains an invalid identifier.

Message

```
SyntaxError: Invalid regular expression: /(?<1>)/: Invalid capture group name (V8-based)
SyntaxError: invalid capture group name in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid group specifier name (Safari)
```

Error type

`SyntaxError`

What went wrong?

Each named capturing group must have a name that is a valid identifier. You cannot use arbitrary strings as the group identifier.

Examples

Invalid cases

```
/(?<1>\d+) (?<2>\d+)/;
```

Or you might be building the regex dynamically:

```
const tokenTypes = {
  "number literal": /\d+/,
  "string literal": /".+?"/,
  identifier: /[a-zA-Z_]\w*/,
```

```
};

const tokenPattern = new RegExp(
  Object.entries(tokenTypes)
    .map(([name, pattern]) => `(?<${name}>${pattern.source})`)
    .join("|"),
);
```

Valid cases

```
/(?<group1>\d+) (?<group2>\d+)/;
```

If the regex is built dynamically, make sure the names are all valid identifiers. For example:

```
const tokenTypes = {
  numberLiteral: /\d+/,
  stringLiteral: /".+?"/,
  identifier: /[a-zA-Z_]\w*/,
};

const tokenPattern = new RegExp(
  Object.entries(tokenTypes)
    .map(([name, pattern]) => `(?<${name}>${pattern.source})`)
    .join("|"),
);
```

See also

- Regular expressions
- Named capturing group: `(?<name>...)`

SyntaxError: invalid character in class in regular expression

The JavaScript exception "invalid character in class in regular expression" occurs when a character appears in a v-mode character class but it's not allowed to appear literally.

Message

```
SyntaxError: Invalid regular expression: /[|]/v: Invalid character in character class (V8-based)
SyntaxError: invalid character in class in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid class set character (Safari)
```

Error type

`SyntaxError`

What went wrong?

Normally, character classes can contain almost all characters literally. However, the `v` mode made the character class syntax more sophisticated, and in order to leave room for future syntax extensions, some syntax characters are forbidden from appearing literally in a character class. They include: `(`, `)`, `[`, `]`, `{`, `}`, `/`, `-`, `|`. If you want to match these literal characters, escape them; for example: `/[\|]/v`.

Examples

Invalid cases

```
/[(){}]/v;
```

Valid cases

```
/[\(\)\{\}]/v;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

SyntaxError: invalid class set operation in regular expression

The JavaScript exception "invalid class set operation in regular expression" occurs when a double punctuator sequence appears in a v-mode character class but it is not recognized by the syntax.

Message

```
SyntaxError: Invalid regular expression: /[&&]/v: Invalid set operation in character class (V8-based)
SyntaxError: invalid class set operation in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid operation in class set (Safari)
```

Error type

`SyntaxError`

What went wrong?

There are three possible ways this could happen:

- You are trying to use `&&` or `--`, but the syntax is wrong. Each of these operators must join two characters or character sets.
- You are mixing operators on the same level. For example, `[\w&&[A-z]--_]` is invalid because it uses `&&` and `--` on the same level. You need to use nested character classes to disambiguate, such as `[\w&&[[A-z]--_]]`. Note that the union operation uses an operator that has no text; for example, `[AB&&C]` is invalid because `A` and `B` are implicitly joined by the union operator. You need to use `[A[B&&C]]` instead.
- You are using a double punctuator sequence that's not `&&` or `--`. These sequences are reserved for future syntax extensions. They include: `&&`, `!!`, `##`, `$$`, `%%`, `**`, `++`, `,,`, `..`, `::`, `;;`, `<<`, `==`, `>>`, `??`, `@@`, `^^`, `` `` ``, `~~`. However, these sequences don't make much sense anyway, and can either be replaced with a single character or cause two adjacent ranges to be merged.

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

SyntaxError: invalid decimal escape in regular expression

The JavaScript exception "invalid decimal escape in regular expression" occurs when a legacy octal escape sequence is used in a Unicode-aware regular expression pattern.

Message

```
SyntaxError: Invalid regular expression: /\00/u: Invalid decimal escape (V8-based)
SyntaxError: invalid decimal escape in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid octal escape for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

In a regular expression, `\0` followed by another digit is a *legacy octal escape sequence*. The same syntax is banned in template strings and strict-mode string literals. In regexes, this feature is disabled by the Unicode-aware modes (`u` and `v`). `\0` *not* followed by another digit is a valid escape sequence that represents the null character (U+0000).

`\` followed by a non-zero digit is a backreference, and is invalid in Unicode-aware mode if it doesn't refer to a capturing group; see invalid identity escape for more information.

Examples

Invalid cases

```
/\00/u;
/\01/u;
```

Valid cases

```
// If you want to match NULL followed by a digit, use a character class
/[\0]0/u;
// If you want to match a character by its character value, use \x
/\x01/u;
```

See also

- Regular expressions
- Escape sequences
- Character escape: `\n`, `\u{...}`

SyntaxError: invalid regexp group

The JavaScript exception "invalid regexp group" occurs when the sequence `(?` does not start a valid group syntax. Recognized group syntaxes that start with `(?` include:

- `(?:` for non-capturing groups
- `(?=` for positive lookahead
- `(?!` for negative lookahead
- `(?<=` for positive lookbehind
- `(?<!` for negative lookbehind
- `(?<` for named capturing groups
- `(?-i:`, `(?i:`, `(?m:`, `(?ims-:`, etc. for modifiers

`(?` followed by any other character would cause this error.

Message

```
SyntaxError: Invalid regular expression: /(?1)/: Invalid group (V8-based)
SyntaxError: invalid regexp group (Firefox)
SyntaxError: Invalid regular expression: unrecognized character after (? (Safari)
```

Error type

`SyntaxError`

What went wrong?

`?` is not an atom, so it does not make sense to appear at the start of a capturing group (`?` is a quantifier and should be placed after an atom). Maybe you want to match the `?` character literally, in which case you should escape it with a backslash (`\?`). Maybe you remembered the regex syntax wrong, and you intend to use one of the recognized group syntaxes listed above. Maybe you are using a feature that is not supported by the current JavaScript engine.

Examples

Invalid cases

```
/Hello(?|!)/;
// This is Perl syntax
/(?[\p{Thai}&\p{Digit}])/;
```

Valid cases

```
/Hello(\?|!)/;
// This is JavaScript syntax for character set operations
/[\p{Script=Thai}&&\p{Nd}]/v;
```

See also

- Regular expressions
- Capturing group: `(...)`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Modifier: `(?ims-ims:...)`
- Named capturing group: `(?<name>...)`
- Non-capturing group: `(?:...)`

SyntaxError: invalid identity escape in regular expression

The JavaScript exception "invalid identity escape in regular expression" occurs when a Unicode-aware regular expression pattern contains an escape sequence that does not represent a recognized escape sequence.

Message

```
SyntaxError: Invalid regular expression: /\q/u: Invalid escape (V8-based)
SyntaxError: invalid identity escape in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid escaped character for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

In Unicode-unaware mode, `\` could be used to escape any character, including those without a defined meaning. In these cases, the escaped character represents itself. For example, `\q` would match the character `q`. This severely limits the ability to add new escape sequences in the future, so in Unicode-aware mode, only recognized escape sequences are allowed. Do not add `\` redundantly.

This error is also thrown when the `\x` character escape is not followed by two hexadecimal digits.

For an exhaustive list of available escape sequences, see the regular expressions reference. For a table of which characters can appear literally escaped or unescaped in each context, see literal characters.

Examples

Invalid cases

```
/[\f\v\n\t\ ]/u;
```

Valid cases

```
// There's no need to escape the space
/[\f\v\n\t ]/u;
```

See also

- Regular expressions
- Escape sequences
- Character escape: `\n`, `\u{...}`
- Literal character: `a`, `b`

SyntaxError: invalid named capture reference in regular expression

The JavaScript exception "invalid named capture reference in regular expression" occurs when a regular expression pattern contains a named backreference that does not refer to a named capture group before it. The similar error message "invalid named reference in regular expression" is thrown when the sequence `\k` is encountered but is not followed by `<`.

Message

```
SyntaxError: Invalid regular expression: /\k<x>/u: Invalid named capture referenced (V8-based)
SyntaxError: invalid named capture reference in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid \k<> named backreference (Safari)

SyntaxError: Invalid regular expression: /\k/u: Invalid named reference (V8-based)
SyntaxError: invalid named reference in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid escaped character for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

You are using the escape sequence `\k` which is parsed as a named backreference, but the `\k` is either syntactically invalid or does not refer to a named capture group in the pattern.

`\k` only starts a named backreference when the pattern contains a named capturing group or when the pattern is Unicode-aware; otherwise, it is an identity escape for the character `k`.

Examples

Invalid cases

```
/\k<x>/u; // Does not refer to a named capture group
/(?<x>.+)\k<y>/u; // Does not refer to an existing named capture group
/(?<x>.+)\k{x}/u; // Invalid syntax
```

Valid cases

```
/(?<x>.+)\k<x>/u; // Refers to an existing named capture group
```

See also

- Regular expressions
- Named backreference: `\k<name>`
- Named capturing group: `(?<name>...)`

SyntaxError: invalid property name in regular expression

The JavaScript exception "invalid property name in regular expression" or "invalid class property name in regular expression" occurs when the `\p` and `\P` Unicode character class escapes are not followed by a valid Unicode property name and/or value.

Message

```
SyntaxError: Invalid regular expression: /\p{x}/u: Invalid property name (V8-based)
SyntaxError: Invalid regular expression: /[\p{x}]/u: Invalid property name in character class (V8-based)
SyntaxError: invalid property name in regular expression (Firefox)
SyntaxError: invalid class property name in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid property expression (Safari)
```

Error type

`SyntaxError`

What went wrong?

In Unicode-aware mode, the `\p` and `\P` escape sequences are used to match characters or character sequences based on their Unicode properties. The `\p` escape sequence matches characters with the specified Unicode property, while the `\P` escape sequence matches characters without the specified Unicode property. The syntax is:

```
\p{loneProperty}
\P{loneProperty}

\p{property=value}
\P{property=value}
```

If it's not followed by `{`, then a recognizable property name/value, then `}`, this syntax error is thrown. For more information on what lone properties and property names are allowed, see the Unicode character class escape reference.

Examples

Invalid cases

```
/\p{x}/u; // "x" is not a valid Unicode property name
/\p{Script=x}/u; // "x" is not a valid value for Script
```

```
/\property/u; // "\p" is not followed by the right syntax
/\p{RGI_Emoji_Flag_Sequence}/u;
// "RGI_Emoji_Flag_Sequence" is a property of strings, so the "v" flag must be used
/\P{RGI_Emoji_Flag_Sequence}/v; // \P cannot match properties of strings
```

Valid cases

```
/\p{Script=Latin}/u; // "Script=Latin" is a valid Unicode property
/\p{Letter}/u; // "Letter" is valid value for General_Category
/\p{RGI_Emoji_Flag_Sequence}/v; // Property of strings can only be used in "v" mode
```

See also

- Regular expressions
- Unicode character class escape: `\p{...}`, `\P{...}`

SyntaxError: invalid range in character class

The JavaScript exception "invalid range in character class" occurs when a character class in a regular expression uses a range, but the start of the range is greater than the end.

Message

```
SyntaxError: Invalid regular expression: /[2-1]/: Range out of order in character class (V8-based)
SyntaxError: invalid range in character class (Firefox)
SyntaxError: Invalid regular expression: range out of order in character class (Safari)
```

Error type

`SyntaxError`

What went wrong?

In character classes, you can join two characters with a hyphen `-` to represent an inclusive interval of characters based on their Unicode code points. For example, `[a-z]` matches any lowercase letter. However, if the end of the range is less than the start, the range cannot match anything and is likely a mistake.

Examples

Invalid cases

```
/[9-1]/; // The range is out of order
/[_-=]/; // _ has value 95, = has value 61
```

Valid cases

```
/[1-9]/; // Swap the range
/[_\-=]/; // Escape the hyphen so it matches the literal character
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`

SyntaxError: invalid unicode escape in regular expression

The JavaScript exception "invalid unicode escape in regular expression" occurs when the `\c` and `\u` character escapes are not followed by valid characters.

Message

```
SyntaxError: Invalid regular expression: /\u{123456}/u: Invalid Unicode escape (V8-based)
SyntaxError: invalid unicode escape in regular expression (Firefox)
SyntaxError: Invalid regular expression: invalid Unicode code point \u{} escape (Safari)
```

Error type

`SyntaxError`

What went wrong?

In Unicode-aware mode, the `\c` escape sequence must be followed by a letter from `A` to `Z` or `a` to `z`, and the `\u` escape sequence must either be followed by 4 hexadecimal digits, or 1 to 6 hexadecimal digits enclosed in curly braces (`{}`). Furthermore, when using the `\u{xxx}` escape sequence, the digits must represent a valid Unicode code point, which means its value cannot exceed `10FFFF`.

Examples

Invalid cases

```
/\u{123456}/u; // Unicode code point is too large
/\u65/u; // Not enough digits
/\c1/u; // Not a letter
```

Valid cases

```
/\u0065/u; // Lowercase "e"
/\u{1f600}/u; // Grinning face emoji
/\cA/u; // U+0001 (Start of Heading)
```

See also

- Regular expressions
- Character escape: `\n`, `\u{...}`

SyntaxError: negated character class with strings in regular expression

The JavaScript exception "negated character class with strings in regular expression" occurs when a v-mode character class is negated and may be able to match a string (more than one character).

Message

```
SyntaxError: Invalid regular expression: /[^\p{RGI_Emoji_Flag_Sequence}]/v: Negated character class may contain strings (V8-based)
SyntaxError: negated character class with strings in regular expression (Firefox)
SyntaxError: Invalid regular expression: negated class set may contain strings (Safari)
```

Error type

`SyntaxError`

What went wrong?

In `v` mode, character classes are able to match more than 1 character. For example, `/[\q{abc}]/v` would match the sequence `"abc"`, and `/[\p{RGI_Emoji_Flag_Sequence}]/v` would match any character sequence that represents an emoji flag. However, negated character classes `[^...]` are not allowed to match strings, so `/[^\p{RGI_Emoji_Flag_Sequence}]/v` is invalid, because it's unclear how many characters it should match. For more information, see the v-mode character class reference.

Examples

Invalid cases

```
/[^\p{RGI_Emoji_Flag_Sequence}]/v;
```

Valid cases

```
// Matches two characters that are not an emoji flag sequence
/(?!\p{RGI_Emoji_Flag_Sequence})../v;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`
- Unicode character class escape: `\p{...}`, `\P{...}`

SyntaxError: nothing to repeat

The JavaScript exception "nothing to repeat" or "invalid quantifier in regular expression" occurs when a quantifier in a regular expression is applied to nothing or applied to an assertion.

Message

```
SyntaxError: Invalid regular expression: /\b+/: Nothing to repeat (V8-based)
SyntaxError: Invalid regular expression: /(?=)+/u: Invalid quantifier (V8-based)
SyntaxError: nothing to repeat (Firefox)
SyntaxError: invalid quantifier in regular expression (Firefox)
SyntaxError: Invalid regular expression: nothing to repeat (Safari)
```

Error type

`SyntaxError`

What went wrong?

Quantifiers are used to specify how many times a character or group of characters can appear in a regular expression. For example, `a{3}` matches the character `a` exactly three times. Therefore, if the thing preceding the quantifier is not something that matches characters, the quantifier is invalid. For example: quantifiers at the start of a capturing group, at the start of a disjunction alternative, etc., cannot repeat anything. Assertions don't consume characters, so it also doesn't make sense to repeat them.

In Unicode-unaware mode, there's a deprecated syntax that allows the lookahead assertions to be quantified. This is a deprecated syntax and you should not rely on it.

Examples

Invalid cases

```
/\b+/; // \b is a word boundary assertion, it doesn't consume characters
/(*hello*)/;
```

Valid cases

```
/b+/; // b is a character, it can be repeated
/(\*hello\*)/; // Escape the asterisks to match them literally
```

See also

- Regular expressions
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

SyntaxError: numbers out of order in {} quantifier.

The JavaScript exception "numbers out of order in {} quantifier" occurs when a quantifier in a regular expression uses the `{n,m}` syntax but `m` is less than `n`.

Message

```
SyntaxError: Invalid regular expression: /1{2,1}/: numbers out of order in {} quantifier (V8-based)
SyntaxError: numbers out of order in {} quantifier. (Firefox)
SyntaxError: Invalid regular expression: numbers out of order in {} quantifier (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `{n,m}` syntax in a regular expression is used to specify that the preceding item is to be matched at least `n` times, but not more than `m` times. If `m` is less than `n`, the quantifier is nonsensical because, for example, a character cannot appear at least 2 times but not more than 1 time. Therefore, an error is thrown.

Examples

Invalid examples

```
/1{2,1}/;
```

Valid examples

```
/1{1,2}/;
```

See also

- Regular expressions
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

SyntaxError: raw bracket is not allowed in regular expression with unicode flag

The JavaScript exception "raw bracket is not allowed in regular expression with unicode flag" occurs when a Unicode-aware regular expression pattern contains a raw bracket (`{`, `}`, `]`) that is not part of a quantifier or character class.

Message

```
SyntaxError: Invalid regular expression: /{/u: Lone quantifier brackets (V8-based)
SyntaxError: raw bracket is not allowed in regular expression with unicode flag (Firefox)
SyntaxError: Invalid regular expression: incomplete {} quantifier for Unicode pattern (Safari)
SyntaxError: Invalid regular expression: unmatched ] or } bracket for Unicode pattern (Safari)
```

Error type

`SyntaxError`

What went wrong?

In Unicode-unaware mode, `{`, `}`, and `]` that are not part of a quantifier or character class are treated as literal characters. This may hide errors in your code and is therefore deprecated and disallowed in Unicode-aware mode. You should either check whether you have invalid syntax or you should escape the characters to match them literally.

If `{` appears in a context that accepts a quantifier, it is treated as the start of a quantifier. If what follows it is not a valid quantifier, another syntax error, incomplete quantifier, is thrown.

Examples

Invalid cases

```
/{{MDN_Macro}}/u;
/\[sic]/u;
```

Valid cases

```
// All { and } need to be escaped
/\{\{MDN_Macro\}\}/u;
```

```
// The ] needs to be escaped
/\[sic\]/u;
```

See also

- Regular expressions
- Character class: `[...]`, `[^...]`
- Character escape: `\n`, `\u{...}`
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

TypeError: matchAll/replaceAll must be called with a global RegExp

The JavaScript exception "TypeError: matchAll/replaceAll must be called with a global RegExp" occurs when the `String.prototype.matchAll()` or `String.prototype.replaceAll()` method is used with a `RegExp` object that does not have the `global` flag set.

Message

```
TypeError: String.prototype.matchAll called with a non-global RegExp argument (V8-based)
TypeError: String.prototype.replaceAll called with a non-global RegExp argument (V8-based)
TypeError: matchAll must be called with a global RegExp (Firefox)
TypeError: replaceAll must be called with a global RegExp (Firefox)
TypeError: String.prototype.matchAll argument must not be a non-global regular expression (Safari)
TypeError: String.prototype.replaceAll argument must not be a non-global regular expression (Safari)
```

Error type

`TypeError`

What went wrong?

The `String.prototype.matchAll()` and `String.prototype.replaceAll()` methods require a `RegExp` object with the `global` flag set. This flag indicates that the regular expression can match all locations of the input string, instead of stopping at the first match. Although the `g` flag is redundant when using these methods (because these methods always do a global replacement), they are still required to make the intention clear.

It's worth noting that the `g` flag validation is done in the `matchAll` and `replaceAll` methods. If you use the `[Symbol.matchAll]()` method of `RegExp` instead, you won't get this error, but there will only be a single match.

Examples

Invalid cases

```
"abc".matchAll(/./); // TypeError
"abc".replaceAll(/./, "f"); // TypeError
```

Valid cases

If you intend to do global matching/replacement: either add the `g` flag, or construct a new `RegExp` object with the `g` flag, if you want to keep the original regex unchanged.

```
[..."abc".matchAll(/./g)]; // [[ "a" ], [ "b" ], [ "c" ]]
"abc".replaceAll(/./g, "f"); // "fff"

const existingPattern = /./;
const newPattern = new RegExp(
  existingPattern.source,
  `${existingPattern.flags}g`,
);
"abc".replaceAll(newPattern, "f"); // "fff"
```

If you only intend to do a single matching/replacement: use `String.prototype.match()` or `String.prototype.replace()` instead. You can also use the `[Symbol.matchAll]()` method if you want an iterator like `matchAll` returns that only contains one match, but doing so will be very confusing.

```
"abc".match(/./); // [ "a" ]
"abc".replace(/./, "f"); // "fbc"

[..././[Symbol.matchAll]("abc")]; // [[ "a" ]]
```

See also

- `String.prototype.matchAll()`
- `String.prototype.replaceAll()`
- `RegExp.prototype.global`

SyntaxError: "x" is a reserved identifier

The JavaScript exception "*variable* is a reserved identifier" occurs when reserved keywords are used as identifiers.

Message

```
SyntaxError: Unexpected reserved word (V8-based)
SyntaxError: implements is a reserved identifier (Firefox)
```

```
SyntaxError: Cannot use the reserved word 'implements' as a variable name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

Reserved keywords will throw in if they are used as identifiers. These are reserved in strict mode and sloppy mode:

- `enum`

The following are only reserved when they are found in strict mode code:

- `implements`
- `interface`
- `let`
- `package`
- `private`
- `protected`
- `public`
- `static`

Examples

Strict and non-strict reserved keywords

The `enum` identifier is generally reserved.

```
const enum = { RED: 0, GREEN: 1, BLUE: 2 };
// SyntaxError: enum is a reserved identifier
```

In strict mode code, more identifiers are reserved.

```
"use strict";
const package = ["potatoes", "rice", "fries"];
// SyntaxError: package is a reserved identifier
```

You'll need to rename these variables.

```
const colorEnum = { RED: 0, GREEN: 1, BLUE: 2 };
const list = ["potatoes", "rice", "fries"];
```

Update older browsers

If you are using an older browser that does not yet implement `let` or `class`, for example, you should update to a more recent browser version that does support these new language features.

```
"use strict";
class DocArchiver {}

// SyntaxError: class is a reserved identifier
// (throws in older browsers only, e.g. Firefox 44 and older)
```

See also

- Lexical grammar

SyntaxError: rest parameter may not have a default

The JavaScript exception "rest parameter may not have a default" occurs when a rest parameter has a default value. Because the rest parameter always creates an array, the default value would never apply.

Message

```
SyntaxError: Rest parameter may not have a default initializer (V8-based)
SyntaxError: rest parameter may not have a default (Firefox)
SyntaxError: Unexpected token '='. Expected a ')' or a ',' after a parameter declaration. (Safari)
```

Error type

`SyntaxError`

What went wrong?

A default parameter gives a parameter a default value if the argument is not passed or passed as `undefined`. A rest parameter collects all the remaining arguments passed to the function and always creates an array. Therefore, it doesn't make sense to have a default value for a rest parameter.

Examples

Invalid cases

```
function doSomething(...args = []) {}
```

Valid cases

```
function doSomething(...args) {
  // args is always an array
}
```

See also

- Functions
- Default parameters
- Rest parameters

RangeError: repeat count must be less than infinity

The JavaScript exception "repeat count must be less than infinity" occurs when the `String.prototype.repeat()` method is used with a `count` argument that is infinity.

Message

```
RangeError: Invalid string length (V8-based)
RangeError: Invalid count value: Infinity (V8-based)
RangeError: repeat count must be less than infinity and not overflow maximum string size (Firefox)
RangeError: Out of memory (Safari)
RangeError: String.prototype.repeat argument must be greater than or equal to 0 and not be Infinity (Safari)
```

Error type

`RangeError`

What went wrong?

The `String.prototype.repeat()` method has been used. It has a `count` parameter indicating the number of times to repeat the string. It must be between 0 and less than positive `Infinity` and cannot be a negative number. The range of allowed values can be described like this: [0, +∞).

The resulting string can also not be larger than the maximum string size, which can differ in JavaScript engines. In Firefox (SpiderMonkey) the maximum string size is 2^{30} - 2 (~2GiB).

Examples

Invalid cases

```
"abc".repeat(Infinity); // RangeError
"a".repeat(2 ** 30); // RangeError
```

Valid cases

```
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count will be converted to integer)
```

See also

- `String.prototype.repeat()`

SyntaxError: setter functions must have one argument

The JavaScript exception "setter functions must have one argument" occurs when a setter is declared and the parameter list is not consisted of exactly one formal parameter.

Message

```
SyntaxError: Setter must have exactly one formal parameter. (V8-based)
SyntaxError: Setter function argument must not be a rest parameter (V8-based)
SyntaxError: setter functions must have one argument (Firefox)
SyntaxError: Unexpected token ','. setter functions must have one parameter. (Safari)
SyntaxError: Unexpected token '...'. Expected a parameter pattern or a ')' in parameter list. (Safari)
```

Error type

`SyntaxError`

What went wrong?

The `set` property syntax looks like a function, but it is stricter and not all function syntax is allowed. A setter is always invoked with exactly one argument, so defining it with any other number of parameters is likely an error. This parameter can be destructured or have a default value, but it cannot be a rest parameter.

Note that this error only applies to property setters using the `set` syntax. If you define the setter using `Object.defineProperty()`, etc., the setter is defined as a normal function, although it's likely still an error if the setter expects any other number of arguments, as it will be called with exactly one.

Examples

Invalid cases

```
const obj = {
  set value() {
    this._value = Math.random();
  },
};
```

Valid cases

```
// You must declare one parameter, even if you don't use it
const obj = {
  set value(_ignored) {
    this._value = Math.random();
  },
};

// You can also declare a normal method instead
const obj = {
  setValue() {
    this._value = Math.random();
  },
};
```

See also

- Functions
- `set`

Warning: unreachable code after return statement

The JavaScript warning "unreachable code after return statement" occurs when using an expression after a `return` statement, or when using a semicolon-less return statement but including an expression directly after.

Message

```
Warning: unreachable code after return statement (Firefox)
```

Error type

Warning

What went wrong?

Unreachable code after a return statement might occur in these situations:

- When using an expression after a `return` statement, or
- when using a semicolon-less return statement but including an expression directly after.

When an expression exists after a valid `return` statement, a warning is given to indicate that the code after the `return` statement is unreachable, meaning it can never be run.

Why should I have semicolons after `return` statements? In the case of semicolon-less `return` statements, it can be unclear whether the developer intended to return the statement on the following line, or to stop execution and return. The warning indicates that there is ambiguity in the way the `return` statement is written.

Warnings will not be shown for semicolon-less returns if these statements follow it:

- `throw`
- `break`
- `var`
- `function`

Examples

Invalid cases

```
function f() {
  let x = 3;
  x += 4;
  return x;   // return exits the function immediately
  x -= 3;     // so this line will never run; it is unreachable
}

function g() {
  return     // this is treated like `return;`
    3 + 4;   // so the function returns, and this line is never reached
}
```

Valid cases

```
function f() {
  let x = 3;
```

```
  x += 4;
  x -= 3;
  return x; // OK: return after all other statements
}

function g() {
  return 3 + 4 // OK: semicolon-less return with expression on the same line
}
```

See also

- Automatic semicolon insertion

SyntaxError: "use strict" not allowed in function with non-simple parameters

The JavaScript exception "`"use strict"` not allowed in function" occurs when a `"use strict"` directive is used at the top of a function with default parameters, rest parameters, or destructuring parameters.

Message

```
SyntaxError: Illegal 'use strict' directive in function with non-simple parameter list (V8-based)
SyntaxError: "use strict" not allowed in function with default parameter (Firefox)
SyntaxError: "use strict" not allowed in function with rest parameter (Firefox)
SyntaxError: "use strict" not allowed in function with destructuring parameter (Firefox)
SyntaxError: 'use strict' directive not allowed inside a function with a non-simple parameter list. (Safari)
```

Error type

`SyntaxError`.

What went wrong?

A `"use strict"` directive is written at the top of a function that has one of the following parameters:

- Default parameters
- Rest parameters
- Destructuring parameters

A `"use strict"` directive is not allowed at the top of such functions per the ECMAScript specification.

Examples

Function statement

In this case, the function `sum` has default parameters `a=1` and `b=2`:

```
function sum(a = 1, b = 2) {
  // SyntaxError: "use strict" not allowed in function with default parameter
  "use strict";
  return a + b;
}
```

If the function should be in strict mode, and the entire script or enclosing function is also okay to be in strict mode, you can move the `"use strict"` directive outside of the function:

```
"use strict";
function sum(a = 1, b = 2) {
  return a + b;
}
```

Function expression

A function expression can use yet another workaround:

```
const sum = function sum([a, b]) {
  // SyntaxError: "use strict" not allowed in function with destructuring parameter
  "use strict";
  return a + b;
};
```

This can be converted to the following expression:

```
const sum = (function () {
  "use strict";
  return function sum([a, b]) {
    return a + b;
  };
})();
```

Arrow function

If an arrow function needs to access the `this` variable, you can use the arrow function as the enclosing function:

```
const callback = (...args) => {
  // SyntaxError: "use strict" not allowed in function with rest parameter
  "use strict";
  return this.run(args);
};
```

This can be converted to the following expression:

```
const callback = (() => {
  "use strict";
  return (...args) => this.run(args);
})();
```

See also

- Strict mode
- function statement
- function expression
- Default parameters
- Rest parameters
- Destructuring parameters

SyntaxError: string literal contains an unescaped line break

The JavaScript error "string literal contains an unescaped line break" occurs when there is an unterminated string literal somewhere. String literals must be enclosed by single (') or double (") quotes and cannot split across multiple lines.

Message

```
SyntaxError: Invalid or unexpected token (V8-based)
SyntaxError: '' string literal contains an unescaped line break (Firefox)
SyntaxError: Unexpected EOF (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is an unterminated string literal somewhere. String literals must be enclosed by single (') or double (") quotes. JavaScript makes no distinction between single-quoted strings and double-quoted strings. Escape sequences work in strings created with either single or double quotes. To fix this error, check if:

- you have opening and closing quotes (single or double) for your string literal,
- you have escaped your string literal correctly,
- your string literal isn't split across multiple lines.

Examples

Multiple lines

You can't split a string across multiple lines like this in JavaScript:

```
const longString = "This is a very long string which needs
                   to wrap across multiple lines because
                   otherwise my code is unreadable.";
// SyntaxError: unterminated string literal
```

Instead, use the + operator, a backslash, or template literals. The + operator variant looks like this:

```
const longString =
  "This is a very long string which needs " +
  "to wrap across multiple lines because " +
  "otherwise my code is unreadable.";
```

Or you can use the backslash character ("\") at the end of each line to indicate that the string will continue on the next line. Make sure there is no space or any other character after the backslash (except for a line break), or as an indent; otherwise it will not work. That form looks like this:

```
const longString =
  "This is a very long string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable.";
```

Another possibility is to use template literals.

```
const longString = `This is a very long string which needs
to wrap across multiple lines because
otherwise my code is unreadable.`;
```

See also

- string literal

- Template literals

ReferenceError: super() called twice in derived class constructor

The JavaScript exception "super() called twice in derived class constructor" occurs when the `super()` is called a second time for a given derived class constructor.

Message

```
ReferenceError: Super constructor may only be called once (V8-based)
ReferenceError: super() called twice in derived class constructor (Firefox)
ReferenceError: 'super()' can't be called more than once in a constructor. (Safari)
```

Error type

`ReferenceError`

What went wrong?

The `super()` call can only be called at most once for each `new` call to a derived class constructor. This is because `super()` is responsible for initializing the parent class, and calling it more than once would result in the parent constructor being called multiple times.

The best way to prevent this is to ensure that `super()` is placed outside of any control flow structure. Otherwise, make sure that all code paths in the constructor lead to only one `super()` call.

The `super()` call can be "saved" in an arrow function nested within the constructor. Then, when you call the arrow function, you will also call `super()`, and the same rule applies: the arrow function can only be called at most once.

Examples

Invalid cases

```
class Base {}

class Derived extends Base {
  constructor() {
    super();
    super();
  }
}
```

Sometimes the bug may be more subtle.

```
class Base {
  constructor(flavor) {
    // Do something with the flavor
  }
}

class Derived extends Base {
  constructor(flavors) {
    if (flavors.includes("chocolate")) {
      super("chocolate");
    }
    if (flavors.includes("vanilla")) {
      super("vanilla");
    }
  }
}
```

Originally, `flavors` may never simultaneously include both "chocolate" and "vanilla", but if that ever happens, the constructor will call `super()` twice. You need to rethink about how your class should be structured to avoid this issue.

Valid cases

```
class Base {}

class Derived extends Base {
  constructor() {
    super();
    // More initialization logic
  }
}
```

See also

- Classes
- `super`

ReferenceError: must call super constructor before using 'this' in derived class constructor

The JavaScript exception "must call super constructor before using 'this' in derived class constructor" occurs when the `super()` is not called for a given derived class constructor, and the derived constructor tries to access the value of `this`, or the derived constructor has already returned and the return value is not an object.

Message

```
ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from derived constructor (V8-based)
ReferenceError: must call super constructor before using 'this' in derived class constructor (Firefox)
ReferenceError: 'super()' must be called in derived constructor before accessing |this| or returning non-object. (Safari)
```

Error type

`ReferenceError`

What went wrong?

The `super()` call can only be called at most once for each `new` call to a derived class constructor. Often, you need to call it exactly once, because if you don't call it, the parent constructor cannot initialize the value of `this`, so you cannot access `this` in the derived constructor and the `this` is not considered a valid constructed object (and throws if the derived constructor completes in this state). The way around it is to return an object from the derived class constructor, in which case the object returned will be used as the constructed object instead of `this`, allowing you to not call `super()`. This is rarely done though.

Examples

Invalid cases

```
class Base {
  constructor() {
    this.x = 1;
  }
}

class Derived extends Base {
  constructor() {
    console.log(this.x);
    // The Base constructor is not called yet, so this.x is undefined
    // ReferenceError: must call super constructor before using 'this' in derived class constructor
  }
}
```

Valid cases

```
class Base {
  constructor() {
    this.x = 1;
  }
}

class Derived extends Base {
  constructor() {
    super();
    console.log(this.x); // 1
  }
}
```

See also

- Classes
- `super`

InternalError: too much recursion

The JavaScript exception "too much recursion" or "Maximum call stack size exceeded" occurs when there are too many function calls, or a function is missing a base case.

Message

```
RangeError: Maximum call stack size exceeded (Chrome)
InternalError: too much recursion (Firefox)
RangeError: Maximum call stack size exceeded. (Safari)
```

Error type

`InternalError` in Firefox; `RangeError` in Chrome and Safari.

What went wrong?

A function that calls itself is called a *recursive function*. Once a condition is met, the function stops calling itself. This is called a *base case*.

In some ways, recursion is analogous to a loop. Both execute the same code multiple times, and both require a condition (to avoid an infinite loop, or rather, infinite recursion in this case). When there are too many function calls, or a function is missing a base case, JavaScript will throw this error.

Examples

This recursive function runs 10 times, as per the exit condition.

```
function loop(x) {
  if (x >= 10)
```

```
    // "x >= 10" is the exit condition
    return;
  // do stuff
  loop(x + 1); // the recursive call
}
loop(0);
```

Setting this condition to an extremely high value, won't work:

```
function loop(x) {
  if (x >= 1000000000000) return;
  // do stuff
  loop(x + 1);
}
loop(0);

// InternalError: too much recursion
```

This recursive function is missing a base case. As there is no exit condition, the function will call itself infinitely.

```
function loop(x) {
  // The base case is missing
  loop(x + 1); // Recursive call
}

loop(0);

// InternalError: too much recursion
```

Class error: too much recursion

```
class Person {
  constructor() {}
  set name(name) {
    this.name = name; // Recursive call
  }
}

const tony = new Person();
tony.name = "Tonisha"; // InternalError: too much recursion
```

When a value is assigned to the property name (this.name = name;) JavaScript needs to set that property. When this happens, the setter function is triggered.

In this example when the setter is triggered, it is told to do the same thing again: *to set the same property that it is meant to handle.* This causes the function to call itself, again and again, making it infinitely recursive.

This issue also appears if the same variable is used in the getter.

```
class Person {
  get name() {
    return this.name; // Recursive call
  }
}
```

To avoid this problem, make sure that the property being assigned to inside the setter function is different from the one that initially triggered the setter. The same goes for the getter.

```
class Person {
  constructor() {}
  set name(name) {
    this._name = name;
  }
  get name() {
    return this._name;
  }
}
const tony = new Person();
tony.name = "Tonisha";
console.log(tony);
```

See also

- Recursion
- Recursive functions

SyntaxError: reference to undeclared private field or method #x

The JavaScript exception "reference to undeclared private field or method #x" occurs when a private name is used, but this private name is not declared in the class scope.

Message

```
SyntaxError: Private field '#x' must be declared in an enclosing class (V8-based)
SyntaxError: reference to undeclared private field or method #x (Firefox)
SyntaxError: Cannot reference undeclared private names: "#x" (Safari)
```

Error type

`SyntaxError`

What went wrong?

Unlike normal string or symbol properties, which return `undefined` if the property does not exist, private names are very strict and can only be legally accessed if they actually exist. Accessing an undeclared private name will result in a syntax error, while accessing a private name that is declared but doesn't exist on the object will result in a type error.

Examples

Undeclared private field

You cannot access a private field that is not declared in the class scope.

```
class MyClass {
  doSomething() {
    console.log(this.#x);
  }
}
```

The same error occurs if you use the `in` operator to perform a check on an undeclared private field.

```
class MyClass {
  doSomething() {
    console.log(#x in this);
  }
}
```

These code are probably mistakes because it's impossible for `#x` to exist on `this` if it's not declared in the class scope. Note that you cannot dynamically *add* private elements to unrelated objects. You should either remove this code or declare the private field in the class scope.

```
class MyClass {
  #x = 0;
  doSomething() {
    console.log(this.#x);
  }
}
```

See also

- Classes
- Private elements

ReferenceError: assignment to undeclared variable "x"

The JavaScript strict mode-only exception "Assignment to undeclared variable" occurs when the value has been assigned to an undeclared variable.

Message

```
ReferenceError: x is not defined (V8-based)
ReferenceError: assignment to undeclared variable x (Firefox)
ReferenceError: Can't find variable: x (Safari)
```

Error type

`ReferenceError` in strict mode only.

What went wrong?

You have an assignment of the form `x = ...`, but `x` has not been previously declared with the `var`, `let`, or `const` keyword. This error occurs in strict mode code only. In non-strict code, assignment to an undeclared variable implicitly creates a property on the global scope.

Examples

Invalid cases

In this case, the variable "bar" is an undeclared variable.

```
function foo() {
  "use strict";
  bar = true;
}
foo(); // ReferenceError: assignment to undeclared variable bar
```

Valid cases

To make "bar" a declared variable, you can add a `let`, `const`, or `var` keyword in front of it.

```
function foo() {
  "use strict";
  const bar = true;
}
foo();
```

See also

- Strict mode
- `var`
- `let`
- `const`

SyntaxError: Unexpected token

The JavaScript exceptions "unexpected token" occur when the parser does not see a token it recognizes at the given position, so it cannot make sense of the structure of the program. This might be a simple typo.

Message

```
SyntaxError: Unexpected token ';' (V8-based)
SyntaxError: Unexpected identifier 'x' (V8-based)
SyntaxError: Unexpected number (V8-based)
SyntaxError: Unexpected string (V8-based)
SyntaxError: Unexpected regular expression (V8-based)
SyntaxError: Unexpected template string (V8-based)

SyntaxError: unexpected token: identifier (Firefox)
SyntaxError: expected expression, got "x" (Firefox)
SyntaxError: expected property name, got "x" (Firefox)
SyntaxError: expected target, got "x" (Firefox)
SyntaxError: expected meta, got "x" (Firefox)
SyntaxError: expected rest argument name, got "x" (Firefox)
SyntaxError: expected closing parenthesis, got "x" (Firefox)
```

Error type

`SyntaxError`

What went wrong?

A specific language construct was expected, but something else was provided. This might be a simple typo.

Examples

Expression expected

For example, when chaining expressions, trailing commas are not allowed.

```
for (let i = 0; i < 5,; ++i) {
  console.log(i);
}
// Uncaught SyntaxError: expected expression, got ';'
```

Correct would be omitting the comma or adding another expression:

```
for (let i = 0; i < 5; ++i) {
  console.log(i);
}
```

Not enough parentheses

Sometimes, you leave out parentheses around `if` statements:

```
function round(n, upperBound, lowerBound) {
if (n > upperBound) || (n < lowerBound) { // Missing parentheses here!
    throw new Error(`Number ${n} is more than ${upperBound} or less than ${lowerBound}`);
  } else if (n < (upperBound + lowerBound) / 2) {
    return lowerBound;
  } else {
    return upperBound;
  }
} // SyntaxError: expected expression, got '||'
```

The parentheses may look correct at first, but note how the `||` is outside the parentheses. Correct would be putting parentheses around the `||`:

```
function round(n, upperBound, lowerBound) {
  if ((n > upperBound) || (n < lowerBound)) {
    throw new Error(
      `Number ${n} is more than ${upperBound} or less than ${lowerBound}`,
    );
```

```
  } else if (n < (upperBound + lowerBound) / 2) {
    return lowerBound;
  } else {
    return upperBound;
  }
}
```

A structure error further up confused the meaning

Sometimes, the error is caused by some structure issues not directly next to the error location, so you need to look around for potential errors. For example, you intended to declare a method of an object, but you declared it as a property instead:

```
const MyComponent = {
  mounted: {
    document.getElementById("app").classList.add("loaded");
  }
}
```

The `.` after `document` is unexpected, because JavaScript is parsing the `{}` as an object literal instead of a function body, so it expects a `:`. The problem is solved by declaring `mounted` as a function.

```
const MyComponent = {
  mounted() {
    document.getElementById("app").classList.add("loaded");
  }
}
```

See also

- `SyntaxError`

TypeError: "x" is (not) "y"

The JavaScript exception "*x* is (not) *y*" occurs when there was an unexpected type. Oftentimes, unexpected `undefined` or `null` values.

Message

```
TypeError: Cannot read properties of undefined (reading 'x') (V8-based)
TypeError: "x" is undefined (Firefox)
TypeError: "undefined" is not an object (Firefox)
TypeError: undefined is not an object (evaluating 'obj.x') (Safari)

TypeError: "x" is not a symbol (V8-based & Firefox)
TypeError: Symbol.keyFor requires that the first argument be a symbol (Safari)
```

Error type

`TypeError`.

What went wrong?

There was an unexpected type. This occurs oftentimes with `undefined` or `null` values.

Also, certain methods, such as `Object.create()` or `Symbol.keyFor()`, require a specific type, that must be provided.

Examples

Invalid cases

You cannot invoke a method on an `undefined` or `null` variable.

```
const foo = undefined;
foo.substring(1); // TypeError: foo is undefined

const foo2 = null;
foo2.substring(1); // TypeError: foo2 is null
```

Certain methods might require a specific type.

```
const foo = {};
Symbol.keyFor(foo); // TypeError: foo is not a symbol

const foo2 = "bar";
Object.create(foo2); // TypeError: "foo2" is not an object or null
```

Fixing the issue

To fix null pointer to `undefined` or `null` values, you can test if the value is `undefined` or `null` first.

```
if (foo !== undefined && foo !== null) {
  // Now we know that foo is defined, we are good to go.
}
```

Or, if you are confident that `foo` will not be another falsy value like `""` or `0`, or if filtering those cases out is not an issue, you can simply test for its truthiness.

```
if (foo) {
  // Now we know that foo is truthy, it will necessarily not be null/undefined.
}
```

See also

- `undefined`
- `null`

SyntaxError: function statement requires a name

The JavaScript exception "function statement requires a name" occurs when there is a function statement in the code that requires a name.

Message

```
SyntaxError: Function statements require a function name (V8-based)
SyntaxError: function statement requires a name (Firefox)
SyntaxError: Function statements must have a name. (Safari)
```

Error type

`SyntaxError`

What went wrong?

There is a function statement in the code that requires a name. You'll need to check how functions are defined and if you need to provide a name for it, or if the function in question needs to be a function expression, an IIFE, or if the function code is placed correctly in this context at all.

Examples

Statements vs. expressions

A *function statement* (or *function declaration*) requires a name. This won't work:

```
function () {
  return "Hello world";
}
// SyntaxError: function statement requires a name
```

You can use a function expression (assignment) instead:

```
const greet = function () {
  return "Hello world";
};
```

If your function is intended to be an IIFE (Immediately Invoked Function Expression, which is a function that runs as soon as it is defined) you will need to add a few more braces:

```
(function () {
  // …
})();
```

Labeled functions

Labels are an entirely different feature from function names. You can't use a label as a function name.

```
function Greeter() {
  german: function () {
    return "Moin";
  }
}
// SyntaxError: function statement requires a name
```

In addition, labeled function declarations themselves are a deprecated feature. Use regular function declarations instead.

```
function Greeter() {
  function german() {
    return "Moin";
  }
}
```

Object methods

If you intended to create a method of an object, you will need to create an object. The following syntax without a name after the `function` keyword is valid then.

```
const greeter = {
  german: function () {
    return "Moin";
```

```
  },
};
```

You can also use the method syntax.

```
const greeter = {
  german() {
    return "Moin";
  },
};
```

Callback syntax

Also, check your syntax when using callbacks. Braces and commas can quickly get confusing.

```
promise.then(
  function () {
    console.log("success");
  });
  function () {
    console.log("error");
}
// SyntaxError: function statement requires a name
```

Correct would be:

```
promise.then(
  function () {
    console.log("success");
  },
  function () {
    console.log("error");
  },
);
```

See also

- Functions guide
- `function`
- `function` expression
- IIFE
- Labeled statement

SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**'

The JavaScript exception "unparenthesized unary expression can't appear on the left-hand side of '**'" occurs when a unary operator (one of `typeof`, `void`, `delete`, `await`, `!`, `~`, `+`, `-`) is used on the left operand of the exponentiation operator without parentheses.

Message

```
SyntaxError: Unary operator used immediately before exponentiation expression. Parenthesis must be used to disambiguate operator precedence (V8-based)
SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**' (Firefox)
SyntaxError: Unexpected token '**'. Ambiguous unary expression in the left hand side of the exponentiation expression; parentheses must be used to
disambiguate the expression. (Safari)
```

Error type

`SyntaxError`

What went wrong?

You likely wrote something like this:

```
-a ** b
```

Whether it should be evaluated as `(-a) ** b` or `-(a ** b)` is ambiguous. In mathematics, $-x^2$ means `-(x ** 2)` — and that's how many languages, including Python, Haskell, and PHP, handle it. But making the unary minus operator take precedence over `**` breaks symmetry with `a ** -b`, which is unambiguously `a ** (-b)`. Therefore, the language forbids this syntax and requires you to parenthesize either side to resolve the ambiguity.

```
(-a) ** b
-(a ** b)
```

Other unary operators cannot be the left-hand side of exponentiation either.

```
await a ** b
!a ** b
+a ** b
~a ** b
```

Examples

When writing complex math expressions involving exponentiation, you may write something like this:

```
function taylorSin(x) {
  return (n) => (-1 ** n * x ** (2 * n + 1)) / factorial(2 * n + 1);
  // SyntaxError: unparenthesized unary expression can't appear on the left-hand side of '**'
}
```

However, the `-1 ** n` part is illegal in JavaScript. Instead, parenthesize the left operand:

```
function taylorSin(x) {
  return (n) => ((-1) ** n * x ** (2 * n + 1)) / factorial(2 * n + 1);
}
```

This also makes the code's intent much clearer to other readers.

See also

- Exponentiation (`**`)
- Operator precedence
- Original discussion of exponentiation operator precedence on esdiscuss.org

JavaScript execution model

This page introduces the basic infrastructure of the JavaScript runtime environment. The model is largely theoretical and abstract, without any platform-specific or implementation-specific details. Modern JavaScript engines heavily optimize the described semantics.

This page is a reference. It assumes you are already familiar with the execution model of other programming languages, such as C and Java. It makes heavy references to existing concepts in operating systems and programming languages.

The engine and the host

JavaScript execution requires the cooperation of two pieces of software: the **JavaScript engine** and the **host environment**.

The JavaScript engine implements the ECMAScript (JavaScript) language, providing the core functionality. It takes source code, parses it, and executes it. However, in order to interact with the outside world, such as to produce any meaningful output, to interface with external resources, or to implement security- or performance-related mechanisms, we need additional environment-specific mechanisms provided by the host environment. For example, the HTML DOM is the host environment when JavaScript is executed in a web browser. Node.js is another host environment that allows JavaScript to be run on the server side.

While we focus primarily on the mechanisms defined in ECMAScript in this reference, we will occasionally talk about mechanisms defined in the HTML spec, which is often mimicked by other host environments like Node.js or Deno. This way, we can give a coherent picture of the JavaScript execution model as used on the web and beyond.

Agent execution model

In the JavaScript specification, each autonomous executor of JavaScript is called an **agent**, which maintains its facilities for code execution:

- **Heap** (of objects): this is just a name to denote a large (mostly unstructured) region of memory. It gets populated as objects get created in the program. Note that in the case of shared memory, each agent has its own heap with its own version of a `SharedArrayBuffer` object, but the underlying memory represented by the buffer is shared.
- **Queue** (of jobs): this is known in HTML (and also commonly) as the *event loop* which enables asynchronous programming in JavaScript while being single-threaded. It's called a queue because it's generally first-in-first-out: earlier jobs are executed before later ones.
- **Stack** (of execution contexts): this is what's known as a *call stack* and allows transferring control flow by entering and exiting execution contexts like functions. It's called a stack because it's last-in-first-out. Every job enters by pushing a new frame onto the (empty) stack, and exits by emptying the stack.

These are three distinct data structures that keep track of different data. We will introduce the queue and the stack in more detail in the following sections. To read more about how heap memory is allocated and freed, see memory management.

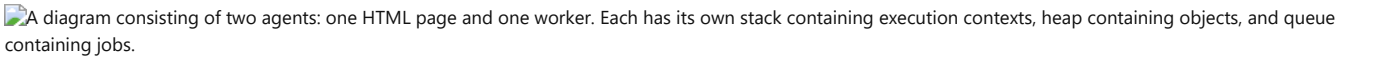
Each agent is analogous to a thread (note that the underlying implementation may or may not be an actual operating system thread). Each agent can own multiple realms (which 1-to-1 correlate with global objects) that can synchronously access each other, and thus needs to run in a single execution thread. An agent also has a single memory model, indicating whether it's little-endian, whether it can be synchronously blocked, whether atomic operations are lock-free, etc.

An agent on the web can be one of the following:

- A *Similar-origin window agent*, which contains various `Window` objects which can potentially reach each other, either directly or by using `document.domain`. If the window is origin-keyed, then only same-origin windows can reach each other.
- A *Dedicated worker agent* containing a single `DedicatedWorkerGlobalScope`.
- A *Shared worker agent* containing a single `SharedWorkerGlobalScope`.
- A *Service worker agent* containing a single `ServiceWorkerGlobalScope`.
- A *Worklet agent* containing a single `WorkletGlobalScope`.

In other words, each worker creates its own agent, while one or more windows may be within the same agent—usually a main document and its similar-origin iframes. In Node.js, a similar concept called worker threads is available.

The diagram below illustrates the execution model of agents:


A diagram consisting of two agents: one HTML page and one worker. Each has its own stack containing execution contexts, heap containing objects, and queue containing jobs.

Realms

Each agent owns one or more **realms**. Each piece of JavaScript code is associated with a realm when it's loaded, which remains the same even when called from another realm. A realm consists of the following information:

- A list of intrinsic objects like `Array`, `Array.prototype`, etc.
- Globally declared variables, the value of `globalThis`, and the global object
- A cache of template literal arrays, because evaluation of the same tagged template literal expression always causes the tag to receive the same array object

On the web, the realm and the global object are 1-to-1 corresponded. The global object is either a `Window`, a `WorkerGlobalScope`, or a `WorkletGlobalScope`. So for example, every `iframe` executes in a different realm, though it may be in the same agent as the parent window.

Realms are usually mentioned when talking about the identities of global objects. For example, we need methods such as `Array.isArray()` or `Error.isError()`, because an array constructed in another realm will have a different prototype object than the `Array.prototype` object in the current realm, so `instanceof Array` will wrongly return `false`.

Stack and execution contexts

We first consider synchronous code execution. Each job enters by calling its associated callback. Code inside this callback may create variables, call functions, or exit. Each function needs to keep track of its own variable environments and where to return to. To handle this, the agent needs a stack to keep track of the execution contexts. An **execution context**, also known generally as a *stack frame*, is the smallest unit of execution. It tracks the following information:

- Code evaluation state
- The module or script, the function (if applicable), and the currently executing generator that contains this code
- The current realm
- Bindings, including:
  - Variables defined with `var`, `let`, `const`, `function`, `class`, etc.
  - Private identifiers like `#foo` which are only valid in the current context
  - `this` reference

Imagine a program consisting of a single job defined by the following code:

```
function foo(b) {
  const a = 10;
  return a + b + 11;
}

function bar(x) {
  const y = 3;
  return foo(x * y);
}

const baz = bar(7); // assigns 42 to baz
```

1. When the job starts, the first frame is created, where the variables `foo`, `bar`, and `baz` are defined. It calls `bar` with the argument `7`.
2. A second frame is created for the `bar` call, containing bindings for the parameter `x` and the local variable `y`. It first performs the multiplication `x * y`, then calls `foo` with the result.
3. A third frame is created for the `foo` call, containing bindings for the parameter `b` and the local variable `a`. It first performs the addition `a + b + 11`, then returns the result.
4. When `foo` returns, the top frame element is popped out of the stack, and the call expression `foo(x * y)` resolves to the return value. It then continues execution, which is just to return this result.
5. When `bar` returns, the top frame element is popped out of the stack, and the call expression `bar(7)` resolves to the return value. This initializes `baz` with the return value.
6. We reach the end of the job's source code, so the stack frame for the entrypoint is popped out of the stack. The stack is empty, so the job is considered completed.

Generators and reentry

When a frame is popped, it's not necessarily gone forever, because sometimes we need to come back to it. For example, consider a generator function:

```
function* gen() {
  console.log(1);
  yield;
  console.log(2);
}

const g = gen();
g.next(); // logs 1
g.next(); // logs 2
```

In this case, calling `gen()` first creates an execution context which is suspended—no code inside `gen` gets executed yet. The generator `g` saves this execution context internally. The current running execution context remains to be the entrypoint. When `g.next()` is called, the execution context for `gen` is pushed onto the stack, and the code inside `gen` is executed until the `yield` expression. Then, the generator execution context gets suspended and removed from the stack, which returns control back to the entrypoint. When `g.next()` is called again, the generator execution context is pushed back onto the stack, and the code inside `gen` resumes from where it left off.

Tail calls

One mechanism defined in the specification is *proper tail call* (PTC). A function call is a tail call if the caller does nothing after the call except return the value:

```
function f() {
  return g();
}
```

In this case, the call to `g` is a tail call. If a function call is in tail position, the engine is required to discard the current execution context and replace it with the context of the tail call, instead of pushing a new frame for the `g()` call. This means that tail recursion is not subject to the stack size limits:

```
function factorial(n, acc = 1) {
  if (n <= 1) return acc;
  return factorial(n - 1, n * acc);
}
```

In reality, discarding the current frame causes debugging problems, because if `g()` throws an error, `f` is no longer on the stack and won't appear in the stack trace. Currently, only Safari (JavaScriptCore) implements PTC, and they have invented some specific infrastructure to address the debuggability issue.

Closures

Another interesting phenomenon related to variable scoping and function calls is closures. Whenever a function is created, it also memorizes internally the variable bindings of the current running execution context. Then, these variable bindings can outlive the execution context.

```
let f;
{
  let x = 10;
  f = () => x;
}
console.log(f()); // logs 10
```

Job queue and event loop

An agent is a thread, which means the interpreter can only process one statement at a time. When the code is all synchronous, this is fine because we can always make progress. But if the code needs to perform asynchronous action, then we cannot progress unless that action is completed. However, it would be detrimental to user experience if that halts the whole program—the nature of JavaScript as a web scripting language requires it to be never blocking. Therefore, the code that handles the completion of that asynchronous action is defined as a callback. This callback defines a **job**, which gets placed into a **job queue**—or, in HTML terminology, an event loop—once the action is completed.

Every time, the agent pulls a job from the queue and executes it. When the job is executed, it may create more jobs, which are added to the end of the queue. Jobs can also be added via the completion of asynchronous platform mechanisms, such as timers, I/O, and events. A job is considered completed when the stack is empty; then, the next job is pulled from the queue. Jobs might not be pulled with uniform priority—for example, HTML event loops split jobs into two categories: *tasks* and *microtasks*. Microtasks have higher priority and the microtask queue is drained first before the task queue is pulled. For more information, check the HTML microtask guide. If the job queue is empty, the agent waits for more jobs to be added.

"Run-to-completion"

Each job is processed completely before any other job is processed. This offers some nice properties when reasoning about your program, including the fact that whenever a function runs, it cannot be preempted and will run entirely before any other code runs (and can modify data the function manipulates). This differs from C, for instance, where if a function runs in a thread, it may be stopped at any point by the runtime system to run some other code in another thread.

For example, consider this example:

```
const promise = Promise.resolve();
let i = 0;
promise.then(() => {
  i += 1;
  console.log(i);
});
promise.then(() => {
  i += 1;
  console.log(i);
});
```

In this example, we create an already-resolved promise, which means any callback attached to it will be immediately scheduled as jobs. The two callbacks seem to cause a race condition, but actually, the output is fully predictable: `1` and `2` will be logged in order. This is because each job runs to completion before the next one is executed, so the overall order is always `i += 1; console.log(i); i += 1; console.log(i);` and never `i += 1; i += 1; console.log(i); console.log(i);`.

A downside of this model is that if a job takes too long to complete, the web application is unable to process user interactions like click or scroll. The browser mitigates this with the "a script is taking too long to run" dialog. A good practice to follow is to make job processing short and, if possible, cut down one job into several jobs.

Never blocking

Another important guarantee offered by the event loop model is that JavaScript execution is never blocking. Handling I/O is typically performed via events and callbacks, so when the application is waiting for an IndexedDB query to return or a `fetch()` request to return, it can still process other things like user input. The code that executes after the completion of an asynchronous action is always provided as a callback function (for example, the promise `then()` handler, the callback function in `setTimeout()`, or the event handler), which defines a job to be added to the job queue once the action completes.

Of course, the guarantee of "never-blocking" requires the platform API to be inherently asynchronous, but some legacy exceptions exist like `alert()` or synchronous XHR. It is considered good practice to avoid them to ensure the responsiveness of the application.

Agent clusters and memory sharing

Multiple agents can communicate via memory sharing, forming an **agent cluster**. Agents are within the same cluster if and only if they can share memory. There is no built-in mechanism for two agent clusters to exchange any information, so they can be regarded as completely isolated execution models.

When creating an agent (such as by spawning a worker), there are some criteria for whether it's in the same cluster as the current agent, or a new cluster is created. For example, the following pairs of global objects are each within the same agent cluster, and thus can share memory with each other:

- A `Window` object and a dedicated worker that it created.
- A worker (of any type) and a dedicated worker it created.
- A `Window` object A and the `Window` object of a same-origin `iframe` element that A created.
- A `Window` object and a same-origin `Window` object that opened it.
- A `Window` object and a worklet that it created.

The following pairs of global objects are not within the same agent cluster, and thus cannot share memory:

- A `Window` object and a shared worker it created.
- A worker (of any type) and a shared worker it created.
- A `Window` object and a service worker it created.
- A `Window` object A and the `Window` object of an `iframe` element that A created that cannot be same origin-domain with A.
- Any two `Window` objects with no opener or ancestor relationship. This holds even if the two `Window` objects are same origin.

For the exact algorithm, check the HTML specification.

Cross-agent communication and memory model

As aforementioned, agents communicate via memory sharing. On the web, memory is shared via the `postMessage()` method. The using web workers guide provides an overview of this. Typically, data is passed by value only (via structured cloning), and therefore does not involve any concurrency complications. To share memory, one must post a `SharedArrayBuffer` object, which can be simultaneously accessed by multiple agents. Once two agents share access to the same memory via a `SharedArrayBuffer`, they can synchronize executions via the `Atomics` object.

There are two ways to access shared memory: via normal memory access (which is not atomic) and via atomic memory access. The latter is sequentially consistent (which means there is a strict total ordering of events agreed upon by all agents in the cluster), while the former is unordered (which means no ordering exists); JavaScript does not provide operations with other ordering guarantees.

The spec provides the following guidelines for programmers working with shared memory:

> We recommend programs be kept data race free, i.e., make it so that it is impossible for there to be concurrent non-atomic operations on the same memory location. Data race free programs have interleaving semantics where each step in the evaluation semantics of each agent are interleaved with each other. For data race free programs, it is not necessary to understand the details of the memory model. The details are unlikely to build intuition that will help one to better write ECMAScript.
>
> More generally, even if a program is not data race free it may have predictable behavior, so long as atomic operations are not involved in any data races and the operations that race all have the same access size. The simplest way to arrange for atomics not to be involved in races is to ensure that different memory cells are used by atomic and non-atomic operations and that atomic accesses of different sizes are not used to access the same cells at the same time. Effectively, the program should treat shared memory as strongly typed as much as possible. One still cannot depend on the ordering and timing of non-atomic accesses that race, but if memory is treated as strongly typed the racing accesses will not "tear" (bits of their values will not be mixed).

Concurrency and ensuring forward progress

When multiple agents cooperate, the never-blocking guarantee does not always hold. An agent can become *blocked*, or paused, while waiting for another agent to perform some action. This is different from waiting on a promise in the same agent, because it halts the entire agent and does not allow any other code to run in the meantime—in other words, it cannot make *forward progress*.

To prevent deadlocks, there are some strong restrictions on when and which agents can become blocked.

- Every unblocked agent with a dedicated executing thread eventually makes forward progress.
- In a set of agents that share an executing thread, one agent eventually makes forward progress.
- An agent does not cause another agent to become blocked except via explicit APIs that provide blocking.
- Only certain agents can be blocked. On the web, this includes dedicated workers and shared workers, but not similar-origin windows or service workers.

The agent cluster ensures some level of integrity over the activeness of its agents, in the case of external pauses or terminations:

- An agent may be paused or resumed without its knowledge or cooperation. For example, navigating away from a window may suspend code execution but preserve its state. However, an agent cluster is not allowed to be partially deactivated, to avoid an agent starving because another agent has been deactivated. For example, shared workers are never in the same agent cluster as the creator window or other dedicated workers. This is because a shared worker's lifetime is independent of documents: if a document is deactivated while its dedicated worker holds a lock, the shared worker is blocked from acquiring the lock until the dedicated worker is reactivated, if ever. Meanwhile other workers trying to access the shared worker from other windows will starve.
- Similarly, an agent may be terminated by factors external to the cluster. For example, operating systems or users killing a browser process, or the browser force-terminating one agent because it's using too many resources. In this case, all the agents in the cluster get terminated. (The spec also allows a second strategy, which is an API that allows at least one remaining member of the cluster to identify the termination and the agent that was terminated, but this is not implemented on the web.)

See also

- Event loops in the HTML standard
- What is the Event Loop? in the Node.js docs

Functions

Generally speaking, a function is a "subprogram" that can be *called* by code external (or internal, in the case of recursion) to the function. Like the program itself, a function is composed of a sequence of statements called the *function body*. Values can be *passed* to a function as parameters, and the function will *return* a value.

In JavaScript, functions are first-class objects, because they can be passed to other functions, returned from functions, and assigned to variables and properties. They can also have properties and methods just like any other object. What distinguishes them from other objects is that functions can be called.

For more examples and explanations, see the JavaScript guide about functions.

Description

Function values are typically instances of `Function`. See `Function` for information on properties and methods of `Function` objects. Callable values cause `typeof` to return `"function"` instead of `"object"`.

> **Note:** Not all callable values are `instanceof Function`. For example, the `Function.prototype` object is callable but not an instance of `Function`. You can also manually set the prototype chain of your function so it no longer inherits from `Function.prototype`. However, such cases are extremely rare.

Return value

By default, if a function's execution doesn't end at a `return` statement, or if the `return` keyword doesn't have an expression after it, then the return value is `undefined`. The `return` statement allows you to return an arbitrary value from the function. One function call can only return one value, but you can simulate the effect of returning multiple values by returning an object or array and destructuring the result.

> **Note:** Constructors called with `new` have a different set of logic to determine their return values.

Passing arguments

Parameters and arguments have slightly different meanings, but in MDN web docs, we often use them interchangeably. For a quick reference:

```
function formatNumber(num) {
  return num.toFixed(2);
}

formatNumber(2);
```

In this example, the `num` variable is called the function's *parameter*: it's declared in the parenthesis-enclosed list of the function's definition. The function expects the `num` parameter to be a number — although this is not enforceable in JavaScript without writing runtime validation code. In the `formatNumber(2)` call, the number `2` is the function's *argument*: it's the value that is actually passed to the function in the function call. The argument value can be accessed inside the function body through the corresponding parameter name or the `arguments` object.

Arguments are always *passed by value* and never *passed by reference*. This means that if a function reassigns a parameter, the value won't change outside the function. More precisely, object arguments are *passed by sharing*, which means if the object's properties are mutated, the change will impact the outside of the function. For example:

```
function updateBrand(obj) {
  // Mutating the object is visible outside the function
  obj.brand = "Toyota";
  // Try to reassign the parameter, but this won't affect
  // the variable's value outside the function
  obj = null;
}

const car = {
  brand: "Honda",
  model: "Accord",
  year: 1998,
};

console.log(car.brand); // Honda

// Pass object reference to the function
updateBrand(car);

// updateBrand mutates car
console.log(car.brand); // Toyota
```

The `this` keyword refers to the object that the function is accessed on — it does not refer to the currently executing function, so you must refer to the function value by name, even within the function body.

Defining functions

Broadly speaking, JavaScript has four kinds of functions:

- Regular function: can return anything; always runs to completion after invocation
- Generator function: returns a `Generator` object; can be paused and resumed with the `yield` operator
- Async function: returns a `Promise`; can be paused and resumed with the `await` operator
- Async generator function: returns an `AsyncGenerator` object; both the `await` and `yield` operators can be used

For every kind of function, there are multiple ways to define it:

Declaration

`function`, `function*`, `async function`, `async function*`

Expression

`function`, `function*`, `async function`, `async function*`

Constructor

`Function()`, `GeneratorFunction()`, `AsyncFunction()`, `AsyncGeneratorFunction()`

In addition, there are special syntaxes for defining arrow functions and methods, which provide more precise semantics for their usage. Classes are conceptually not functions (because they throw an error when called without `new`), but they also inherit from `Function.prototype` and have `typeof MyClass === "function"`.

```
// Constructor
const multiply = new Function("x", "y", "return x * y");

// Declaration
function multiply(x, y) {
  return x * y;
} // No need for semicolon here

// Expression; the function is anonymous but assigned to a variable
const multiply = function (x, y) {
  return x * y;
};
// Expression; the function has its own name
const multiply = function funcName(x, y) {
  return x * y;
};

// Arrow function
const multiply = (x, y) => x * y;

// Method
const obj = {
  multiply(x, y) {
    return x * y;
  },
};
```

All syntaxes do approximately the same thing, but there are some subtle behavior differences.

- The `Function()` constructor, `function` expression, and `function` declaration syntaxes create full-fledged function objects, which can be constructed with `new`. However, arrow functions and methods cannot be constructed. Async functions, generator functions, and async generator functions are not constructible regardless of syntax.
- The `function` declaration creates functions that are *hoisted*. Other syntaxes do not hoist the function and the function value is only visible after the definition.
- The arrow function and `Function()` constructor always create *anonymous* functions, which means they can't easily call themselves recursively. One way to call an arrow function recursively is by assigning it to a variable.
- The arrow function syntax does not have access to `arguments` or `this`.
- The `Function()` constructor cannot access any local variables — it only has access to the global scope.
- The `Function()` constructor causes runtime compilation and is often slower than other syntaxes.

For `function` expressions, there is a distinction between the function name and the variable the function is assigned to. The function name cannot be changed, while the variable the function is assigned to can be reassigned. The function name can be different from the variable the function is assigned to — they have no relation to each other. The function name can be used only within the function's body. Attempting to use it outside the function's body results in an error (or gets another value, if the same name is declared elsewhere). For example:

```
const y = function x() {};
console.log(x); // ReferenceError: x is not defined
```

On the other hand, the variable the function is assigned to is limited only by its scope, which is guaranteed to include the scope in which the function is declared.

A function declaration also creates a variable with the same name as the function name. Thus, unlike those defined by function expressions, functions defined by function declarations can be accessed by their name in the scope they were defined in, as well as in their own body.

A function defined by `new Function` will dynamically have its source assembled, which is observable when you serialize it. For example, `console.log(new Function().toString())` gives:

```
function anonymous(
) {

}
```

This is the actual source used to compile the function. However, although the `Function()` constructor will create the function with name `anonymous`, this name is not added to the scope of the body. The body only ever has access to global variables. For example, the following would result in an error:

```
new Function("alert(anonymous);")();
```

A function defined by a function expression or by a function declaration inherits the current scope. That is, the function forms a closure. On the other hand, a function defined by a `Function` constructor does not inherit any scope other than the global scope (which all functions inherit).

```
// p is a global variable
globalThis.p = 5;
function myFunc() {
  // p is a local variable
```

```
  const p = 9;

  function decl() {
    console.log(p);
  }
  const expr = function () {
    console.log(p);
  };
  const cons = new Function("\tconsole.log(p);");

  decl();
  expr();
  cons();
}
myFunc();

// Logs:
// 9 (for 'decl' by function declaration (current scope))
// 9 (for 'expr' by function expression (current scope))
// 5 (for 'cons' by Function constructor (global scope))
```

Functions defined by function expressions and function declarations are parsed only once, while a function defined by the `Function` constructor parses the string passed to it each and every time the constructor is called. Although a function expression creates a closure every time, the function body is not reparsed, so function expressions are still faster than `new Function(...)`. Therefore the `Function` constructor should generally be avoided whenever possible.

A function declaration may be unintentionally turned into a function expression when it appears in an expression context.

```
// A function declaration
function foo() {
  console.log("FOO!");
}

doSomething(
  // A function expression passed as an argument
  function foo() {
    console.log("FOO!");
  },
);
```

On the other hand, a function expression may also be turned into a function declaration. An expression statement cannot begin with the `function` or `async function` keywords, which is a common mistake when implementing IIFEs (Immediately Invoked Function Expressions).

```
function () { // SyntaxError: Function statements require a function name
  console.log("FOO!");
}();

function foo() {
  console.log("FOO!");
}(); // SyntaxError: Unexpected token ')'
```

Instead, start the expression statement with something else, so that the `function` keyword unambiguously starts a function expression. Common options include grouping and using `void`.

```
(function () {
  console.log("FOO!");
})();

void function () {
  console.log("FOO!");
}();
```

Function parameters

Each function parameter is a simple identifier that you can access in the local scope.

```
function myFunc(a, b, c) {
  // You can access the values of a, b, and c here
}
```

There are three special parameter syntaxes:

- *Default parameters* allow formal parameters to be initialized with default values if no value or `undefined` is passed.
- The *rest parameter* allows representing an indefinite number of arguments as an array.
- *Destructuring* allows unpacking elements from arrays, or properties from objects, into distinct variables.

```
function myFunc({ a, b }, c = 1, ...rest) {
  // You can access the values of a, b, c, and rest here
}
```

There are some consequences if one of the above non-simple parameter syntaxes is used:

- You cannot apply `"use strict"` to the function body — this causes a syntax error.

- Even if the function is not in strict mode, certain strict mode function features apply, including that the `arguments` object stops syncing with the named parameters, `arguments.callee` throws an error when accessed, and duplicate parameter names are not allowed.

The arguments object

You can refer to a function's arguments within the function by using the `arguments` object.

`arguments`

: An array-like object containing the arguments passed to the currently executing function.

`arguments.callee`

: The currently executing function.

`arguments.length`

: The number of arguments passed to the function.

Getter and setter functions

You can define accessor properties on any standard built-in object or user-defined object that supports the addition of new properties. Within object literals and classes, you can use special syntaxes to define the getter and setter of an accessor property.

get

: Binds an object property to a function that will be called when that property is looked up.

set

: Binds an object property to a function to be called when there is an attempt to set that property.

Note that these syntaxes create an *object property*, not a *method*. The getter and setter functions themselves can only be accessed using reflective APIs such as `Object.getOwnPropertyDescriptor()`.

Block-level functions

In strict mode, functions inside blocks are scoped to that block. Prior to ES2015, block-level functions were forbidden in strict mode.

```
"use strict";

function f() {
  return 1;
}

{
  function f() {
    return 2;
  }
}

f() === 1; // true

// f() === 2 in non-strict mode
```

Block-level functions in non-strict code

In a word: **Don't.**

In non-strict code, function declarations inside blocks behave strangely. For example:

```
if (shouldDefineZero) {
  function zero() {
    // DANGER: compatibility risk
    console.log("This is zero.");
  }
}
```

The semantics of this in strict mode are well-specified — `zero` only ever exists within that scope of the `if` block. If `shouldDefineZero` is false, then `zero` should never be defined, since the block never executes. However, historically, this was left unspecified, so different browsers implemented it differently in non-strict mode. For more information, see the `function` declaration reference.

A safer way to define functions conditionally is to assign a function expression to a variable:

```
// Using a var makes it available as a global variable,
// with closer behavior to a top-level function declaration
var zero;
if (shouldDefineZero) {
  zero = function () {
    console.log("This is zero.");
  };
}
```

Examples

Returning a formatted number

The following function returns a string containing the formatted representation of a number padded with leading zeros.

```
// This function returns a string padded with leading zeros
function padZeros(num, totalLen) {
  let numStr = num.toString(); // Initialize return value as string
  const numZeros = totalLen - numStr.length; // Calculate no. of zeros
  for (let i = 1; i <= numZeros; i++) {
    numStr = `0${numStr}`;
  }
  return numStr;
}
```

The following statements call the `padZeros` function.

```
let result;
result = padZeros(42, 4); // returns "0042"
result = padZeros(42, 2); // returns "42"
result = padZeros(5, 4); // returns "0005"
```

Determining whether a function exists

You can determine whether a function exists by using the `typeof` operator. In the following example, a test is performed to determine if the `window` object has a property called `noFunc` that is a function. If so, it is used; otherwise, some other action is taken.

```
if (typeof window.noFunc === "function") {
  // use noFunc()
} else {
  // do something else
}
```

Note that in the `if` test, a reference to `noFunc` is used — there are no parentheses `()` after the function name so the actual function is not called.

See also

- Functions guide
- Classes
- `function`
- `function` expression
- `Function`

The arguments object

`arguments` is an array-like object accessible inside functions that contains the values of the arguments passed to that function.

Try it

```
function func1(a, b, c) {
  console.log(arguments[0]);
  // Expected output: 1

  console.log(arguments[1]);
  // Expected output: 2

  console.log(arguments[2]);
  // Expected output: 3
}

func1(1, 2, 3);
```

Description

> **Note:** In modern code, rest parameters should be preferred.

The `arguments` object is a local variable available within all non-arrow functions. You can refer to a function's arguments inside that function by using its `arguments` object. It has entries for each argument the function was called with, with the first entry's index at `0`.

For example, if a function is passed 3 arguments, you can access them as follows:

```
arguments[0]; // first argument
arguments[1]; // second argument
arguments[2]; // third argument
```

The `arguments` object is useful for functions called with more arguments than they are formally declared to accept, called *variadic functions*, such as `Math.min()`. This example function accepts any number of string arguments and returns the longest one:

```
function longestString() {
  let longest = "";
  if (arguments.length === 0) {
```

```
    throw new TypeError("At least one string is required");
  }
  for (const arg of arguments) {
    if (arg.length > longest.length) {
      longest = arg;
    }
  }
  return longest;
}
```

You can use `arguments.length` to count how many arguments the function was called with. If you instead want to count how many parameters a function is declared to accept, inspect that function's `length` property.

Assigning to indices

Each argument index can also be set or reassigned:

```
arguments[1] = "new value";
```

Non-strict functions that only have simple parameters (that is, no rest, default, or destructured parameters) will sync the new value of parameters with the `arguments` object, and vice versa:

```
function func(a) {
  arguments[0] = 99; // updating arguments[0] also updates a
  console.log(a);
}
func(10); // 99

function func2(a) {
  a = 99; // updating a also updates arguments[0]
  console.log(arguments[0]);
}
func2(10); // 99
```

Non-strict functions that *are* passed rest, default, or destructured parameters will not sync new values assigned to parameters in the function body with the `arguments` object. Instead, the `arguments` object in non-strict functions with complex parameters will always reflect the values passed to the function when the function was called.

```
function funcWithDefault(a = 55) {
  arguments[0] = 99; // updating arguments[0] does not also update a
  console.log(a);
}
funcWithDefault(10); // 10

function funcWithDefault2(a = 55) {
  a = 99; // updating a does not also update arguments[0]
  console.log(arguments[0]);
}
funcWithDefault2(10); // 10

// An untracked default parameter
function funcWithDefault3(a = 55) {
  console.log(arguments[0]);
  console.log(arguments.length);
}
funcWithDefault3(); // undefined; 0
```

This is the same behavior exhibited by all strict-mode functions, regardless of the type of parameters they are passed. That is, assigning new values to parameters in the body of the function never affects the `arguments` object, nor will assigning new values to the `arguments` indices affect the value of parameters, even when the function only has simple parameters.

> **Note:** You cannot write a `"use strict";` directive in the body of a function definition that accepts rest, default, or destructured parameters. Doing so will throw a syntax error.

arguments is an array-like object

`arguments` is an array-like object, which means that `arguments` has a `length` property and properties indexed from zero, but it doesn't have `Array`'s built-in methods like `forEach()` or `map()`. However, it can be converted to a real `Array`, using one of `slice()`, `Array.from()`, or spread syntax.

```
const args = Array.prototype.slice.call(arguments);
// or
const args = Array.from(arguments);
// or
const args = [...arguments];
```

For common use cases, using it as an array-like object is sufficient, since it both is iterable and has `length` and number indices. For example, `Function.prototype.apply()` accepts array-like objects.

```
function midpoint() {
  return (
    (Math.min.apply(null, arguments) + Math.max.apply(null, arguments)) / 2
```

```
  );
}

console.log(midpoint(3, 1, 4, 1, 5)); // 3
```

Properties

`arguments.callee`

Reference to the currently executing function that the arguments belong to. Forbidden in strict mode.

`arguments.length`

The number of arguments that were passed to the function.

`arguments[Symbol.iterator]()`

Returns a new array iterator object that contains the values for each index in `arguments`.

Examples

Defining a function that concatenates several strings

This example defines a function that concatenates several strings. The function's only formal argument is a string containing the characters that separate the items to concatenate.

```
function myConcat(separator) {
  const args = Array.prototype.slice.call(arguments, 1);
  return args.join(separator);
}
```

You can pass as many arguments as you like to this function. It returns a string list using each argument in the list:

```
myConcat(", ", "red", "orange", "blue");
// "red, orange, blue"

myConcat("; ", "elephant", "giraffe", "lion", "cheetah");
// "elephant; giraffe; lion; cheetah"

myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley");
// "sage. basil. oregano. pepper. parsley"
```

Defining a function that creates HTML lists

This example defines a function that creates a string containing HTML for a list. The only formal argument for the function is a string that is `"u"` if the list is to be unordered (bulleted), or `"o"` if the list is to be ordered (numbered). The function is defined as follows:

```
function list(type) {
  let html = `<${type}l><li>`;
  const args = Array.prototype.slice.call(arguments, 1);
  html += args.join("</li><li>");
  html += `</li></${type}l>`; // end list
  return html;
}
```

You can pass any number of arguments to this function, and it adds each argument as a list item to a list of the type indicated. For example:

```
list("u", "One", "Two", "Three");
// "<ul><li>One</li><li>Two</li><li>Three</li></ul>"
```

Using typeof with arguments

The `typeof` operator returns `'object'` when used with `arguments`

```
console.log(typeof arguments); // 'object'
```

The type of individual arguments can be determined by indexing `arguments`:

```
console.log(typeof arguments[0]); // returns the type of the first argument
```

See also

- Functions guide
- Functions
- Rest parameters

arguments.callee

> **Note:** Accessing `arguments.callee` in strict mode will throw a `TypeError`. If a function must reference itself, either give the function expression a name or use a function declaration.

The **`arguments.callee`** data property contains the currently executing function that the arguments belong to.

Value

A reference to the currently executing function.

Property attributes of `arguments.callee`	
Writable	yes
Enumerable	no
Configurable	yes

> **Note:** `callee` is a data property only in non-strict functions with simple parameters (in which case the `arguments` object is also auto-syncing). Otherwise, it is an accessor property whose getter and setter both throw a `TypeError`.

Description

`callee` is a property of the `arguments` object. It can be used to refer to the currently executing function inside the function body of that function. This is useful when the name of the function is unknown, such as within a function expression with no name (also called "anonymous functions").

(The text below is largely adapted from a Stack Overflow answer by olliej)

Early versions of JavaScript did not allow named function expressions, and for this reason you could not make a recursive function expression.

For example, this syntax worked:

```
function factorial(n) {
  return n <= 1 ? 1 : factorial(n - 1) * n;
}

[1, 2, 3, 4, 5].map(factorial);
```

but:

```
[1, 2, 3, 4, 5].map(function (n) {
  return n <= 1 ? 1 : /* what goes here? */ (n - 1) * n;
});
```

did not. To get around this `arguments.callee` was added so you could do

```
[1, 2, 3, 4, 5].map(function (n) {
  return n <= 1 ? 1 : arguments.callee(n - 1) * n;
});
```

However, the design of `arguments.callee` has multiple issues. The first problem is that the recursive call will get a different `this` value. For example:

```
function sillyFunction(recursed) {
  if (this !== globalThis) {
    console.log("This is:", this);
  } else {
    console.log("This is the global");
  }

  if (!recursed) {
    return arguments.callee(true);
  }
}

sillyFunction();
// This is the global
// This is: [object Arguments]
```

In addition, references to `arguments.callee` make inlining and tail recursion impossible in the general case. (You can achieve it in select cases through tracing, etc., but even the best code is suboptimal due to checks that would not otherwise be necessary.)

ECMAScript 3 resolved these issues by allowing named function expressions. For example:

```
[1, 2, 3, 4, 5].map(function factorial(n) {
  return n <= 1 ? 1 : factorial(n - 1) * n;
});
```

This has numerous benefits:

- the function can be called like any other from inside your code
- it does not create a variable in the outer scope (except for IE 8 and below)
- it has better performance than accessing the arguments object

Strict mode has banned other properties that leak stack information, like the `caller` property of functions. This is because looking at the call stack has one single major effect: it makes a large number of optimizations impossible, or much more difficult. For example, if you cannot guarantee that a function `f` will not call an unknown

function, it is not possible to inline `f`.

```
function f(a, b, c, d, e) {
  return a ? b * c : d * e;
}
```

If the JavaScript interpreter cannot guarantee that all the provided arguments are numbers at the point that the call is made, it needs to either insert checks for all the arguments before the inlined code, or it cannot inline the function. This means any call site that may have been trivially inlinable accumulates a large number of guards. Now in this particular case a smart interpreter should be able to rearrange the checks to be more optimal and not check any values that would not be used. However in many cases that's just not possible and therefore it becomes impossible to inline.

Examples

Using arguments.callee in an anonymous recursive function

A recursive function must be able to refer to itself. Typically, a function refers to itself by its name. However, an anonymous function (which can be created by a function expression or the `Function` constructor) does not have a name. Therefore if there is no accessible variable referring to it, the only way the function can refer to itself is by `arguments.callee`.

The following example defines a function, which, in turn, defines and returns a factorial function. This example isn't very practical, and there are nearly no cases where the same result cannot be achieved with named function expressions.

```
function create() {
  return function (n) {
    if (n <= 1) {
      return 1;
    }
    return n * arguments.callee(n - 1);
  };
}

const result = create()(5); // returns 120 (5 * 4 * 3 * 2 * 1)
```

Recursion of anonymous functions with a Y-combinator

Although function expressions can now be named, arrow functions always remain anonymous, which means they cannot reference themselves without being assigned to a variable first. Fortunately, in Lambda calculus there's a very good solution which allows a function to both be anonymous and self-referential. The technique is called a Y-combinator. Here we will not explain *how* it works, only *that* it works.

```
// The Y-combinator: a utility function!
const Y = (hof) => ((x) => x(x))((x) => hof((y) => x(x)(y)));

console.log(
  [1, 2, 3, 4, 5].map(
    // Wrap the higher-order function in the Y-combinator
    // "factorial" is not a function's name: it's introduced as a parameter
    Y((factorial) => (n) => (n <= 1 ? 1 : factorial(n - 1) * n)),
  ),
);
// [ 1, 2, 6, 24, 120 ]
```

> **Note:** This method allocates a new closure for every iteration, which may significantly increase memory usage. It's only here to demonstrate the possibility, but should be avoided in production. Use a temporary variable or a named function expression instead.

See also

- Functions guide
- Functions
- `arguments`
- `Function.prototype.caller`

arguments.length

The **`arguments.length`** data property contains the number of arguments passed to the function.

Value

A non-negative integer.

Property attributes of `arguments.length`	
Writable	yes
Enumerable	no
Configurable	yes

Description

The `arguments.length` property provides the number of arguments actually passed to a function. This can be more or less than the defined parameter's count (see `Function.prototype.length`). For example, for the function below:

```
function func1(a, b, c) {
  console.log(arguments.length);
}
```

`func1.length` returns `3`, because `func1` declares three formal parameters. However, `func1(1, 2, 3, 4, 5)` logs `5`, because `func1` was called with five arguments. Similarly, `func1(1)` logs `1`, because `func1` was called with one argument.

Examples

Using arguments.length

In this example, we define a function that can add two or more numbers together.

```
function adder(base /*, num1, …, numN */) {
  base = Number(base);
  for (let i = 1; i < arguments.length; i++) {
    base += Number(arguments[i]);
  }
  return base;
}
```

See also

- Functions guide
- Functions
- `arguments`
- `Function: length`

arguments[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `arguments` objects implements the iterable protocol and allows `arguments` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns an array iterator object that yields the value of each index in the `arguments` object.

The initial value of this property is the same function object as the initial value of the `Array.prototype.values` property (and also the same as `Array.prototype[Symbol.iterator]()`).

Syntax

```
arguments[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Array.prototype.values()`: a new iterable iterator object that yields the value of each index in the `arguments` object.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `arguments` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
function f() {
  for (const letter of arguments) {
    console.log(letter);
  }
}
f("w", "y", "k", "o", "p");
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
function f() {
  const argsIter = arguments[Symbol.iterator]();
  console.log(argsIter.next().value); // w
  console.log(argsIter.next().value); // y
  console.log(argsIter.next().value); // k
  console.log(argsIter.next().value); // o
  console.log(argsIter.next().value); // p
}
f("w", "y", "k", "o", "p");
```

See also

- Functions guide
- Functions
- `arguments`

- `Array.prototype.values()`
- `Symbol.iterator`
- Iteration protocols

Arrow function expressions

An **arrow function expression** is a compact alternative to a traditional function expression, with some semantic differences and deliberate limitations in usage:

- Arrow functions don't have their own bindings to `this`, `arguments`, or `super`, and should not be used as methods.
- Arrow functions cannot be used as constructors. Calling them with `new` throws a `TypeError`. They also don't have access to the `new.target` keyword.
- Arrow functions cannot use `yield` within their body and cannot be created as generator functions.

Try it

```
const materials = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

console.log(materials.map((material) => material.length));
// Expected output: Array [8, 6, 7, 9]
```

Syntax

```
() => expression

param => expression

(param) => expression

(param1, paramN) => expression

() => {
  statements
}

param => {
  statements
}

(param1, paramN) => {
  statements
}
```

Rest parameters, default parameters, and destructuring within params are supported, and always require parentheses:

```
(a, b, ...r) => expression
(a = 400, b = 20, c) => expression
([a, b] = [10, 20]) => expression
({ a, b } = { a: 10, b: 20 }) => expression
```

Arrow functions can be `async` by prefixing the expression with the `async` keyword.

```
async param => expression
async (param1, param2, ...paramN) => {
  statements
}
```

Description

Let's decompose a traditional anonymous function down to the simplest arrow function step-by-step. Each step along the way is a valid arrow function.

> **Note:** Traditional function expressions and arrow functions have more differences than their syntax. We will introduce their behavior differences in more detail in the next few subsections.

```
// Traditional anonymous function
(function (a) {
  return a + 100;
});

// 1. Remove the word "function" and place arrow between the argument and opening body brace
(a) => {
  return a + 100;
};

// 2. Remove the body braces and word "return" — the return is implied.
(a) => a + 100;

// 3. Remove the parameter parentheses
a => a + 100;
```

In the example above, both the parentheses around the parameter and the braces around the function body may be omitted. However, they can only be omitted in certain cases.

The parentheses can only be omitted if the function has a single simple parameter. If it has multiple parameters, no parameters, or default, destructured, or rest parameters, the parentheses around the parameter list are required.

```
// Traditional anonymous function
(function (a, b) {
  return a + b + 100;
});

// Arrow function
(a, b) => a + b + 100;

const a = 4;
const b = 2;

// Traditional anonymous function (no parameters)
(function () {
  return a + b + 100;
});

// Arrow function (no parameters)
() => a + b + 100;
```

The braces can only be omitted if the function directly returns an expression. If the body has statements, the braces are required. In this case, return values must be explicitly specified with the `return` keyword. Arrow functions cannot guess what or when you want to return.

```
// Traditional anonymous function
(function (a, b) {
  const chuck = 42;
  return a + b + chuck;
});

// Arrow function
(a, b) => {
  const chuck = 42;
  return a + b + chuck;
};
```

Arrow functions are not inherently associated with a name. If the arrow function needs to call itself, use a named function expression instead. You can also assign the arrow function to a variable, allowing you to refer to it through that variable.

```
// Traditional Function
function bob(a) {
  return a + 100;
}

// Arrow Function
const bob2 = (a) => a + 100;
```

Function body

Arrow functions can have either an *expression body* or the usual *block body*.

In an expression body, only a single expression is specified, which becomes the implicit return value. The block body is analogous to traditional function bodies, where return values must be explicitly specified with the `return` keyword. Arrow functions are not required to return a value. If execution of the block body reaches the end without encountering a `return` statement, the function returns `undefined` like other functions.

```
// Expression body
const add = (a, b) => a + b; // Implicitly returns a + b

// Block body
const add2 = (a, b) => {
  console.log(a, b);
  return a + b; // Must explicitly return a value
};

// No return value
const add3 = (b) => {
  a += b;
  // No return statement, so returns undefined
};
```

Returning object literals using the expression body syntax `(params) => { object: literal }` does not work as expected.

```
const func = () => { foo: 1 };
// Calling func() returns undefined!

const func2 = () => { foo: function () {} };
// SyntaxError: function statement requires a name

const func3 = () => { foo() {} };
// SyntaxError: Unexpected token '{'
```

This is because JavaScript only sees the arrow function as having an expression body if the token following the arrow is not a left brace, so the code inside braces ({}) is parsed as a sequence of statements, where `foo` is a label, not a key in an object literal.

To fix this, wrap the object literal in parentheses:

```
const func = () => ({ foo: 1 });
```

Cannot be used as methods

Arrow function expressions should only be used for non-method functions because they do not have their own `this`. Let's see what happens when we try to use them as methods:

```
"use strict";

const obj = {
  i: 10,
  b: () => console.log(this.i, this),
  c() {
    console.log(this.i, this);
  },
};

obj.b(); // logs undefined, Window { /* … */ } (or the global object)
obj.c(); // logs 10, Object { /* … */ }
```

Another example involving `Object.defineProperty()`:

```
"use strict";

const obj = {
  a: 10,
};

Object.defineProperty(obj, "b", {
  get: () => {
    console.log(this.a, typeof this.a, this); // undefined 'undefined' Window { /* … */ } (or the global object)
    return this.a + 10; // represents global object 'Window', therefore 'this.a' returns 'undefined'
  },
});
```

Because a class's body has a `this` context, arrow functions as class fields close over the class's `this` context, and the `this` inside the arrow function's body will correctly point to the instance (or the class itself, for static fields). However, because it is a closure, not the function's own binding, the value of `this` will not change based on the execution context.

```
class C {
  a = 1;
  autoBoundMethod = () => {
    console.log(this.a);
  };
}

const c = new C();
c.autoBoundMethod(); // 1
const { autoBoundMethod } = c;
autoBoundMethod(); // 1
// If it were a normal method, it should be undefined in this case
```

Arrow function properties are often said to be "auto-bound methods", because the equivalent with normal methods is:

```
class C {
  a = 1;
  constructor() {
    this.method = this.method.bind(this);
  }
  method() {
    console.log(this.a);
  }
}
```

> **Note:** Class fields are defined on the *instance*, not on the *prototype*, so every instance creation would create a new function reference and allocate a new closure, potentially leading to more memory usage than a normal unbound method.

For similar reasons, the `call()`, `apply()`, and `bind()` methods are not useful when called on arrow functions, because arrow functions establish `this` based on the scope the arrow function is defined within, and the `this` value does not change based on how the function is invoked.

No binding of arguments

Arrow functions do not have their own `arguments` object. Thus, in this example, `arguments` is a reference to the arguments of the enclosing scope:

```
function foo(n) {
  const f = () => arguments[0] + n; // foo's implicit arguments binding. arguments[0] is n
  return f();
}

foo(3); // 3 + 3 = 6
```

In most cases, using rest parameters is a good alternative to using an `arguments` object.

```
function foo(n) {
  const f = (...args) => args[0] + n;
  return f(10);
}

foo(1); // 11
```

Cannot be used as constructors

Arrow functions cannot be used as constructors and will throw an error when called with `new`. They also do not have a `prototype` property.

```
const Foo = () => {};
const foo = new Foo(); // TypeError: Foo is not a constructor
console.log("prototype" in Foo); // false
```

Cannot be used as generators

The `yield` keyword cannot be used in an arrow function's body (except when used within generator functions further nested within the arrow function). As a consequence, arrow functions cannot be used as generators.

Line break before arrow

An arrow function cannot contain a line break between its parameters and its arrow.

```
const func = (a, b, c)
  => 1;
// SyntaxError: Unexpected token '=>'
```

For the purpose of formatting, you may put the line break after the arrow or use parentheses/braces around the function body, as shown below. You can also put line breaks between parameters.

```
const func = (a, b, c) =>
  1;

const func2 = (a, b, c) => (
  1
);

const func3 = (a, b, c) => {
  return 1;
};

const func4 = (
  a,
  b,
  c,
) => 1;
```

Precedence of arrow

Although the arrow in an arrow function is not an operator, arrow functions have special parsing rules that interact differently with operator precedence compared to regular functions.

```
let callback;

callback = callback || () => {};
// SyntaxError: invalid arrow-function arguments
```

Because `=>` has a lower precedence than most operators, parentheses are necessary to avoid `callback || ()` being parsed as the arguments list of the arrow function.

```
callback = callback || (() => {});
```

Examples

Using arrow functions

```
// An empty arrow function returns undefined
const empty = () => {};

(() => "foobar")();
// Returns "foobar"
// (this is an Immediately Invoked Function Expression)
```

```
const simple = (a) => (a > 15 ? 15 : a);
simple(16); // 15
simple(10); // 10

const max = (a, b) => (a > b ? a : b);

// Easy array filtering, mapping, etc.
const arr = [5, 6, 13, 0, 1, 18, 23];

const sum = arr.reduce((a, b) => a + b);
// 66

const even = arr.filter((v) => v % 2 === 0);
// [6, 0, 18]

const double = arr.map((v) => v * 2);
// [10, 12, 26, 0, 2, 36, 46]

// More concise promise chains
promise
  .then((a) => {
    // …
  })
  .then((b) => {
    // …
  });

// Arrow functions without parameters
setTimeout(() => {
  console.log("I happen sooner");
  setTimeout(() => {
    // deeper code
    console.log("I happen later");
  }, 1);
}, 1);
```

Using call, bind, and apply

The `call()`, `apply()`, and `bind()` methods work as expected with traditional functions, because we establish the scope for each of the methods:

```
const obj = {
  num: 100,
};

// Setting "num" on globalThis to show how it is NOT used.
globalThis.num = 42;

// A traditional function to operate on "this"
function add(a, b, c) {
  return this.num + a + b + c;
}

console.log(add.call(obj, 1, 2, 3)); // 106
console.log(add.apply(obj, [1, 2, 3])); // 106
const boundAdd = add.bind(obj);
console.log(boundAdd(1, 2, 3)); // 106
```

With arrow functions, since our `add` function is essentially created on the `globalThis` (global) scope, it will assume `this` is the `globalThis`.

```
const obj = {
  num: 100,
};

// Setting "num" on globalThis to show how it gets picked up.
globalThis.num = 42;

// Arrow function
const add = (a, b, c) => this.num + a + b + c;

console.log(add.call(obj, 1, 2, 3)); // 48
console.log(add.apply(obj, [1, 2, 3])); // 48
const boundAdd = add.bind(obj);
console.log(boundAdd(1, 2, 3)); // 48
```

Perhaps the greatest benefit of using arrow functions is with methods like `setTimeout()` and `EventTarget.prototype.addEventListener()` that usually require some kind of closure, `call()`, `apply()`, or `bind()` to ensure that the function is executed in the proper scope.

With traditional function expressions, code like this does not work as expected:

```
const obj = {
  count: 10,
  doSomethingLater() {
    setTimeout(function () {
      // the function executes on the window scope
```

```
      this.count++;
      console.log(this.count);
    }, 300);
  },
};

obj.doSomethingLater(); // logs "NaN", because the property "count" is not in the window scope.
```

With arrow functions, the `this` scope is more easily preserved:

```
const obj = {
  count: 10,
  doSomethingLater() {
    // The method syntax binds "this" to the "obj" context.
    setTimeout(() => {
      // Since the arrow function doesn't have its own binding and
      // setTimeout (as a function call) doesn't create a binding
      // itself, the "obj" context of the outer method is used.
      this.count++;
      console.log(this.count);
    }, 300);
  },
};

obj.doSomethingLater(); // logs 11
```

See also

- Functions guide
- Functions
- `function`
- `function` expression
- ES6 In Depth: Arrow functions on hacks.mozilla.org (2015)

Default parameters

Default function parameters allow named parameters to be initialized with default values if no value or `undefined` is passed.

Try it

```
function multiply(a, b = 1) {
  return a * b;
}

console.log(multiply(5, 2));
// Expected output: 10

console.log(multiply(5));
// Expected output: 5
```

Syntax

```
function fnName(param1 = defaultValue1, /* …, */ paramN = defaultValueN) {
  // …
}
```

Description

In JavaScript, function parameters default to `undefined`. However, it's often useful to set a different default value. This is where default parameters can help.

In the following example, if no value is provided for `b` when `multiply` is called, `b`'s value would be `undefined` when evaluating `a * b` and `multiply` would return `NaN`.

```
function multiply(a, b) {
  return a * b;
}

multiply(5, 2); // 10
multiply(5); // NaN !
```

In the past, the general strategy for setting defaults was to test parameter values in the function body and assign a value if they are `undefined`. In the following example, `b` is set to `1` if `multiply` is called with only one argument:

```
function multiply(a, b) {
  b = typeof b !== "undefined" ? b : 1;
  return a * b;
}

multiply(5, 2); // 10
multiply(5); // 5
```

With default parameters, checks in the function body are no longer necessary. Now, you can assign `1` as the default value for `b` in the function head:

```
function multiply(a, b = 1) {
  return a * b;
}

multiply(5, 2); // 10
multiply(5); // 5
multiply(5, undefined); // 5
```

Parameters are still set left-to-right, overwriting default parameters even if there are later parameters without defaults.

```
function f(x = 1, y) {
  return [x, y];
}

f(); // [1, undefined]
f(2); // [2, undefined]
```

> **Note:** The first default parameter and all parameters after it will not contribute to the function's `length`.

The default parameter initializers live in their own scope, which is a parent of the scope created for the function body.

This means that earlier parameters can be referred to in the initializers of later parameters. However, functions and variables declared in the function body cannot be referred to from default value parameter initializers; attempting to do so throws a run-time `ReferenceError`. This also includes `var`-declared variables in the function body.

For example, the following function will throw a `ReferenceError` when invoked, because the default parameter value does not have access to the child scope of the function body:

```
function f(a = go()) {
  function go() {
    return ":P";
  }
}

f(); // ReferenceError: go is not defined
```

This function will print the value of the *parameter* `a`, because the variable `var a` is hoisted only to the top of the scope created for the function body, not the parent scope created for the parameter list, so its value is not visible to `b`.

```
function f(a, b = () => console.log(a)) {
  var a = 1;
  b();
}

f(); // undefined
f(5); // 5
```

The default parameter allows any expression, but you cannot use `await` or `yield` that would pause the evaluation of the default expression. The parameter must be initialized *synchronously*.

```
async function f(a = await Promise.resolve(1)) {
  return a;
}
```

> **Note:** Because the default parameter is evaluated when the function is called, not when the function is defined, the validity of the `await` and `yield` operators depends on the function itself, not its surrounding function. For example, if the current function is not `async`, `await` will be parsed as an identifier and follow normal identifier syntax rules, even when this function is nested in an `async` function.

Examples

Passing undefined vs. other falsy values

In the second call in this example, even if the first argument is set explicitly to `undefined` (though not `null` or other falsy values), the value of the `num` argument is still the default.

```
function test(num = 1) {
  console.log(typeof num);
}

test(); // 'number' (num is set to 1)
test(undefined); // 'number' (num is set to 1 too)

// test with other falsy values:
test(""); // 'string' (num is set to '')
test(null); // 'object' (num is set to null)
```

Evaluated at call time

The default argument is evaluated at *call time*. Unlike with Python (for example), a new object is created each time the function is called.

```
function append(value, array = []) {
  array.push(value);
  return array;
}

append(1); // [1]
append(2); // [2], not [1, 2]
```

This even applies to functions and variables:

```
function callSomething(thing = something()) {
  return thing;
}

let numberOfTimesCalled = 0;
function something() {
  numberOfTimesCalled += 1;
  return numberOfTimesCalled;
}

callSomething(); // 1
callSomething(); // 2
```

Earlier parameters are available to later default parameters

Parameters defined earlier (to the left) are available to later default parameters:

```
function greet(name, greeting, message = `${greeting} ${name}`) {
  return [name, greeting, message];
}

greet("David", "Hi"); // ["David", "Hi", "Hi David"]
greet("David", "Hi", "Happy Birthday!"); // ["David", "Hi", "Happy Birthday!"]
```

This functionality can be approximated like this, which demonstrates how many edge cases are handled:

```
function go() {
  return ":P";
}

function withDefaults(
  a,
  b = 5,
  c = b,
  d = go(),
  e = this,
  f = arguments,
  g = this.value,
) {
  return [a, b, c, d, e, f, g];
}

function withoutDefaults(a, b, c, d, e, f, g) {
  switch (arguments.length) {
    case 0:
    case 1:
      b = 5;
    case 2:
      c = b;
    case 3:
      d = go();
    case 4:
      e = this;
    case 5:
      f = arguments;
    case 6:
      g = this.value;
  }
  return [a, b, c, d, e, f, g];
}

withDefaults.call({ value: "=^_^=" });
// [undefined, 5, 5, ":P", {value:"=^_^="}, arguments, "=^_^="]

withoutDefaults.call({ value: "=^_^=" });
// [undefined, 5, 5, ":P", {value:"=^_^="}, arguments, "=^_^="]
```

Destructured parameter with default value assignment

You can use default value assignment with the destructuring syntax.

A common way of doing that is to set an empty object/array as the default value for the destructured parameter; for example: `[x = 1, y = 2] = []`. This makes it possible to pass nothing to the function and still have those values prefilled:

```
function preFilledArray([x = 1, y = 2] = []) {
  return x + y;
}

preFilledArray(); // 3
preFilledArray([]); // 3
preFilledArray([2]); // 4
preFilledArray([2, 3]); // 5

// Works the same for objects:
function preFilledObject({ z = 3 } = {}) {
  return z;
}

preFilledObject(); // 3
preFilledObject({}); // 3
preFilledObject({ z: 2 }); // 2
```

See also

- Functions guide
- Functions
- Rest parameters
- Nullish coalescing operator (`??`)

get

The **`get`** syntax binds an object property to a function that will be called when that property is looked up. It can also be used in classes.

Try it

```
const obj = {
  log: ["a", "b", "c"],
  get latest() {
    return this.log[this.log.length - 1];
  },
};

console.log(obj.latest);
// Expected output: "c"
```

Syntax

```
{ get prop() { /* … */ } }
{ get [expression]() { /* … */ } }
```

There are some additional syntax restrictions:

- A getter must have exactly zero parameters.

Parameters

`prop`

> The name of the property to bind to the given function. In the same way as other properties in object initializers, it can be a string literal, a number literal, or an identifier.

`expression`

> You can also use expressions for a computed property name to bind to the given function.

Description

Sometimes, it is desirable to allow access to a property that returns a dynamically computed value, or you may want to reflect the status of an internal variable without requiring the use of explicit method calls. In JavaScript, this can be accomplished with the use of a *getter*.

An object property is either a data property or an accessor property, but it cannot simultaneously be both. Read `Object.defineProperty()` for more information. The getter syntax allows you to specify the getter function in an object initializer.

```
const obj = {
  get prop() {
    // getter, the code executed when reading obj.prop
    return someValue;
  },
};
```

Properties defined using this syntax are own properties of the created object, and they are configurable and enumerable.

Examples

Defining a getter on new objects in object initializers

This will create a pseudo-property `latest` for object `obj`, which will return the last array item in `log`.

```
const obj = {
  log: ["example", "test"],
  get latest() {
    return this.log.at(-1);
  },
};
console.log(obj.latest); // "test"
```

Note that attempting to assign a value to `latest` will not change it.

Using getters in classes

You can use the exact same syntax to define public instance getters that are available on class instances. In classes, you don't need the comma separator between methods.

```
class ClassWithGetSet {
  #msg = "hello world";
  get msg() {
    return this.#msg;
  }
  set msg(x) {
    this.#msg = `hello ${x}`;
  }
}

const instance = new ClassWithGetSet();
console.log(instance.msg); // "hello world"

instance.msg = "cake";
console.log(instance.msg); // "hello cake"
```

Getter properties are defined on the `prototype` property of the class and are thus shared by all instances of the class. Unlike getter properties in object literals, getter properties in classes are not enumerable.

Static getters and private getters use similar syntaxes, which are described in the `static` and private elements pages.

Deleting a getter using the <code>delete</code> operator

If you want to remove the getter, you can just `delete` it:

```
delete obj.latest;
```

Defining a getter on existing objects using <code>defineProperty</code>

To append a getter to an existing object later at any time, use `Object.defineProperty()`.

```
const o = { a: 0 };

Object.defineProperty(o, "b", {
  get() {
    return this.a + 1;
  },
});

console.log(o.b); // Runs the getter, which yields a + 1 (which is 1)
```

Using a computed property name

```
const expr = "foo";

const obj = {
  get [expr]() {
    return "bar";
  },
};

console.log(obj.foo); // "bar"
```

Defining static getters

```
class MyConstants {
  static get foo() {
    return "foo";
  }
}

console.log(MyConstants.foo); // 'foo'
```

```
MyConstants.foo = "bar";
console.log(MyConstants.foo); // 'foo', a static getter's value cannot be changed
```

Smart / self-overwriting / lazy getters

Getters give you a way to *define* a property of an object, but they do not *calculate* the property's value until it is accessed. A getter defers the cost of calculating the value until the value is needed. If it is never needed, you never pay the cost.

An additional optimization technique to lazify or delay the calculation of a property value and cache it for later access are *smart* (or *memoized*) getters. The value is calculated the first time the getter is called and is then cached so subsequent accesses return the cached value without recalculating it. This is useful in the following situations:

- If the calculation of a property value is expensive (takes much RAM or CPU time, spawns worker threads, retrieves remote file, etc.).
- If the value isn't needed just now. It will be used later, or in some cases, it's not used at all.
- If it's used, it will be accessed several times, and there is no need to re-calculate that value will never be changed or shouldn't be re-calculated.

> **Note:** This means that you shouldn't write a lazy getter for a property whose value you expect to change, because if the getter is lazy, then it will not recalculate the value.
>
> Note that getters are not "lazy" or "memoized" by nature; you must implement this technique if you desire this behavior.

In the following example, the object has a getter as its own property. On getting the property, the property is removed from the object and re-added, but implicitly as a data property this time. Finally, the value gets returned.

```
const obj = {
  get notifier() {
    delete this.notifier;
    this.notifier = document.getElementById("bookmarked-notification-anchor");
    return this.notifier;
  },
};
```

Feature detection

Many functions accept an object and retrieve individual properties from it as separate parameters (this object parameter is known as an *options bag*). You can detect whether a specific option is supported by using a getter to track if the property has been retrieved. This example checks if the `colorType` option is supported by the `HTMLCanvasElement.getContext()` method.

```
function isColorTypeSupported() {
  let supported = false;
  const obj = {
    get colorType() {
      supported = true;
      return undefined;
    },
  };
  document.createElement("canvas").getContext("2d", obj);
  return supported;
}
```

get vs. defineProperty

While using the `get` keyword and `Object.defineProperty()` have similar results, there is a subtle difference between the two when used on classes. The `get` syntax defines the property on the instance's prototype, while `Object.defineProperty()` defines the property on the instance it is applied to.

```
class Example {
  get hello() {
    return "world";
  }
}

const obj = new Example();
console.log(obj.hello);
// "world"

console.log(Object.getOwnPropertyDescriptor(obj, "hello"));
// undefined

console.log(
  Object.getOwnPropertyDescriptor(Object.getPrototypeOf(obj), "hello"),
);
// { configurable: true, enumerable: false, get: function get hello() { return 'world'; }, set: undefined }
```

See also

- Working with objects guide
- Functions
- `set`
- `Object.defineProperty()`
- Object initializer

- `class`
- Property accessors
- Incompatible ES5 change: literal getter and setter functions must now have exactly zero or one arguments by Jeff Walden (2010)
- More SpiderMonkey changes: ancient, esoteric, very rarely used syntax for creating getters and setters is being removed by Jeff Walden (2010)

Method definitions

Method definition is a shorter syntax for defining a function property in an object initializer. It can also be used in classes.

Try it

```
const obj = {
  foo() {
    return "bar";
  },
};

console.log(obj.foo());
// Expected output: "bar"
```

Syntax

```
({
  property(parameters) {},
  *generator(parameters) {},
  async property(parameters) {},
  async *generator(parameters) {},

  // with computed keys
  [expression](parameters) {},
  *[expression](parameters) {},
  async [expression](parameters) {},
  async *[expression](parameters) {},
})
```

Description

The shorthand syntax is similar to the getter and setter syntax.

Given the following code:

```
const obj = {
  foo: function () {
    // …
  },
  bar: function () {
    // …
  },
};
```

You are now able to shorten this to:

```
const obj = {
  foo() {
    // …
  },
  bar() {
    // …
  },
};
```

Properties defined using this syntax are own properties of the created object, and they are configurable, enumerable, and writable, just like normal properties.

`function*`, `async function`, and `async function*` properties all have their respective method syntaxes; see examples below.

However, note that the method syntax is not equivalent to a normal property with a function as its value — there are semantic differences. This makes methods defined in object literals more consistent with methods in classes.

Method definitions are not constructable

Methods cannot be constructors! They will throw a `TypeError` if you try to instantiate them. On the other hand, a property created as a function can be used as a constructor.

```
const obj = {
  method() {},
};
new obj.method(); // TypeError: obj.method is not a constructor
```

Using super in method definitions

Only functions defined as methods have access to the `super` keyword. `super.prop` looks up the property on the prototype of the object that the method was initialized on.

```
const obj = {
  __proto__: {
    prop: "foo",
  },
  notAMethod: function () {
    console.log(super.prop); // SyntaxError: 'super' keyword unexpected here
  },
};
```

Examples

Using method definitions

```
const obj = {
  a: "foo",
  b() {
    return this.a;
  },
};
console.log(obj.b()); // "foo"
```

Method definitions in classes

You can use the exact same syntax to define public instance methods that are available on class instances. In classes, you don't need the comma separator between methods.

```
class ClassWithPublicInstanceMethod {
  publicMethod() {
    return "hello world";
  }
  secondPublicMethod() {
    return "goodbye world";
  }
}

const instance = new ClassWithPublicInstanceMethod();
console.log(instance.publicMethod()); // "hello world"
```

Public instance methods are defined on the `prototype` property of the class and are thus shared by all instances of the class. They are writable, non-enumerable, and configurable.

Inside instance methods, `this` and `super` work like in normal methods. Usually, `this` refers to the instance itself. In subclasses, `super` lets you access the prototype of the object that the method is attached to, allowing you to call methods from the superclass.

```
class BaseClass {
  msg = "hello world";
  basePublicMethod() {
    return this.msg;
  }
}

class SubClass extends BaseClass {
  subPublicMethod() {
    return super.basePublicMethod();
  }
}

const instance = new SubClass();
console.log(instance.subPublicMethod()); // "hello world"
```

Static methods and private methods use similar syntaxes, which are described in the `static` and private elements pages.

Computed property names

The method syntax also supports computed property names.

```
const bar = {
  foo0: function () {
    return 0;
  },
  foo1() {
    return 1;
  },
  ["foo".toUpperCase()]() {
    return 2;
  },
};

console.log(bar.foo0()); // 0
console.log(bar.foo1()); // 1
console.log(bar.FOO()); // 2
```

Generator methods

Note that the asterisk (`*`) in the generator method syntax must be *before* the generator property name. (That is, `* g(){}` will work, but `g *(){}` will not.)

```
// Using a named property
const obj = {
  g: function* () {
    let index = 0;
    while (true) {
      yield index++;
    }
  },
};

// The same object using shorthand syntax
const obj2 = {
  *g() {
    let index = 0;
    while (true) {
      yield index++;
    }
  },
};

const it = obj2.g();
console.log(it.next().value); // 0
console.log(it.next().value); // 1
```

Async methods

```
// Using a named property
const obj = {
  f: async function () {
    await somePromise;
  },
};

// The same object using shorthand syntax
const obj2 = {
  async f() {
    await somePromise;
  },
};
```

Async generator methods

```
// Using a named property
const obj = {
  f: async function* () {
    yield 1;
    yield 2;
    yield 3;
  },
};

// The same object using shorthand syntax
const obj2 = {
  async *f() {
    yield 1;
    yield 2;
    yield 3;
  },
};
```

See also

- Working with objects guide
- Functions
- `get`
- `set`
- Object initializer
- `class`

Rest parameters

The **rest parameter** syntax allows a function to accept an indefinite number of arguments as an array, providing a way to represent variadic functions in JavaScript.

Try it

```
function sum(...theArgs) {
  let total = 0;
  for (const arg of theArgs) {
    total += arg;
  }
  return total;
```

```
}

console.log(sum(1, 2, 3));
// Expected output: 6

console.log(sum(1, 2, 3, 4));
// Expected output: 10
```

Syntax

```
function f(a, b, ...theArgs) {
  // …
}
```

There are some additional syntax restrictions:

- A function definition can only have one rest parameter.
- The rest parameter must be the last parameter in the function definition.
- Trailing commas are not allowed after the rest parameter.
- The rest parameter cannot have a default value.

Description

A function definition's last parameter can be prefixed with `...` (three U+002E FULL STOP characters), which will cause all remaining (user supplied) parameters to be placed within an `Array` object.

```
function myFun(a, b, ...manyMoreArgs) {
  console.log("a", a);
  console.log("b", b);
  console.log("manyMoreArgs", manyMoreArgs);
}

myFun("one", "two", "three", "four", "five", "six");

// Console Output:
// a, one
// b, two
// manyMoreArgs, ["three", "four", "five", "six"]
```

The rest parameter may be destructured, which allows you to ignore certain parameter positions.

```
function ignoreFirst(...[, b, c]) {
  return b + c;
}
```

However, the following are all syntax errors:

```
function wrong1(...one, ...wrong) {}
function wrong2(...wrong, arg2, arg3) {}
function wrong3(...wrong,) {}
function wrong4(...wrong = []) {}
```

The rest parameter is not counted towards the function's `length` property.

The difference between rest parameters and the arguments object

There are four main differences between rest parameters and the `arguments` object:

- The `arguments` object is **not a real array**, while rest parameters are `Array` instances, meaning methods like `sort()`, `map()`, `forEach()` or `pop()` can be applied on it directly.
- The `arguments` object has the additional (deprecated) `callee` property.
- In a non-strict function with simple parameters, the `arguments` object syncs its indices with the values of parameters. The rest parameter array never updates its value when the named parameters are re-assigned.
- The rest parameter bundles all the *extra* parameters into a single array, but does not contain any named argument defined *before* the `...restParam`. The `arguments` object contains all of the parameters — including the parameters in the `...restParam` array — bundled into one array-like object.

Examples

Using rest parameters

In this example, the first argument is mapped to `a` and the second to `b`, so these named arguments are used as normal.

However, the third argument, `manyMoreArgs`, will be an array that contains the third, fourth, fifth, sixth, …, nth — as many arguments as the user specifies.

```
function myFun(a, b, ...manyMoreArgs) {
  console.log("a", a);
  console.log("b", b);
  console.log("manyMoreArgs", manyMoreArgs);
}

myFun("one", "two", "three", "four", "five", "six");
```

```
// a, "one"
// b, "two"
// manyMoreArgs, ["three", "four", "five", "six"] <-- an array
```

Below, even though there is just one value, the last argument still gets put into an array.

```
// Using the same function definition from example above

myFun("one", "two", "three");

// a, "one"
// b, "two"
// manyMoreArgs, ["three"] <-- an array with just one value
```

Below, the third argument isn't provided, but `manyMoreArgs` is still an array (albeit an empty one).

```
// Using the same function definition from example above

myFun("one", "two");

// a, "one"
// b, "two"
// manyMoreArgs, [] <-- still an array
```

Below, only one argument is provided, so `b` gets the default value `undefined`, but `manyMoreArgs` is still an empty array.

```
// Using the same function definition from example above

myFun("one");

// a, "one"
// b, undefined
// manyMoreArgs, [] <-- still an array
```

Argument length

Since `theArgs` is an array, a count of its elements is given by the `length` property. If the function's only parameter is a rest parameter, `restParams.length` will be equal to `arguments.length`.

```
function fun1(...theArgs) {
  console.log(theArgs.length);
}

fun1(); // 0
fun1(5); // 1
fun1(5, 6, 7); // 3
```

Using rest parameters in combination with ordinary parameters

In the next example, a rest parameter is used to collect all parameters after the first parameter into an array. Each one of the parameter values collected into the array is then multiplied by the first parameter, and the array is returned:

```
function multiply(multiplier, ...theArgs) {
  return theArgs.map((element) => multiplier * element);
}

const arr = multiply(2, 15, 25, 42);
console.log(arr); // [30, 50, 84]
```

From arguments to an array

`Array` methods can be used on rest parameters, but not on the `arguments` object:

```
function sortRestArgs(...theArgs) {
  const sortedArgs = theArgs.sort();
  return sortedArgs;
}

console.log(sortRestArgs(5, 3, 7, 1)); // 1, 3, 5, 7

function sortArguments() {
  const sortedArgs = arguments.sort();
  return sortedArgs; // this will never happen
}

console.log(sortArguments(5, 3, 7, 1));
// throws a TypeError (arguments.sort is not a function)
```

Rest parameters were introduced to reduce the boilerplate code that was commonly used for converting a set of arguments to an array.

Before rest parameters, `arguments` need to be converted to a normal array before calling array methods on them:

```
function fn(a, b) {
  const normalArray = Array.prototype.slice.call(arguments);
  // — or —
  const normalArray2 = [].slice.call(arguments);
  // — or —
  const normalArrayFrom = Array.from(arguments);

  const first = normalArray.shift(); // OK, gives the first argument
  const firstBad = arguments.shift(); // ERROR (arguments is not a normal array)
}
```

Now, you can easily gain access to a normal array using a rest parameter:

```
function fn(...args) {
  const normalArray = args;
  const first = normalArray.shift(); // OK, gives the first argument
}
```

See also

- Functions guide
- Functions
- Spread syntax (`...`)
- Default parameters
- `arguments`
- `Array`

set

The **`set`** syntax binds an object property to a function to be called when there is an attempt to set that property. It can also be used in classes.

Try it

```
const language = {
  set current(name) {
    this.log.push(name);
  },
  log: [],
};

language.current = "EN";
language.current = "FA";

console.log(language.log);
// Expected output: Array ["EN", "FA"]
```

Syntax

```
{ set prop(val) { /* … */ } }
{ set [expression](val) { /* … */ } }
```

There are some additional syntax restrictions:

- A setter must have exactly one parameter.

Parameters

`prop`

The name of the property to bind to the given function. In the same way as other properties in object initializers, it can be a string literal, a number literal, or an identifier.

`val`

An alias for the variable that holds the value attempted to be assigned to `prop`.

`expression`

You can also use expressions for a computed property name to bind to the given function.

Description

In JavaScript, a setter can be used to execute a function whenever an attempt is made to change a property's value. Setters are most often used in conjunction with getters.

An object property is either a data property or an accessor property, but it cannot simultaneously be both. Read `Object.defineProperty()` for more information. The setter syntax allows you to specify the setter function in an object initializer.

```
const obj = {
  set prop(val) {
    // setter, the code executed when setting obj.prop
```

```
  },
};
```

Properties defined using this syntax are own properties of the created object, and they are configurable and enumerable.

Examples

Defining a setter on new objects in object initializers

The following example defines a pseudo-property `current` of object `language`. When `current` is assigned a value, it updates `log` with that value:

```
const language = {
  set current(name) {
    this.log.push(name);
  },
  log: [],
};

language.current = "EN";
console.log(language.log); // ['EN']

language.current = "FA";
console.log(language.log); // ['EN', 'FA']
```

Note that `current` is not defined, and any attempts to access it will result in `undefined`.

Using setters in classes

You can use the exact same syntax to define public instance setters that are available on class instances. In classes, you don't need the comma separator between methods.

```
class ClassWithGetSet {
  #msg = "hello world";
  get msg() {
    return this.#msg;
  }
  set msg(x) {
    this.#msg = `hello ${x}`;
  }
}

const instance = new ClassWithGetSet();
console.log(instance.msg); // "hello world"

instance.msg = "cake";
console.log(instance.msg); // "hello cake"
```

Setter properties are defined on the `prototype` property of the class and are thus shared by all instances of the class. Unlike setter properties in object literals, setter properties in classes are not enumerable.

Static setters and private setters use similar syntaxes, which are described in the `static` and private elements pages.

Removing a setter with the <code>delete</code> operator

If you want to remove the setter, you can just `delete` it:

```
delete language.current;
```

Defining a setter on existing objects using <code>defineProperty</code>

To append a setter to an *existing* object, use `Object.defineProperty()`.

```
const o = { a: 0 };

Object.defineProperty(o, "b", {
  set(x) {
    this.a = x / 2;
  },
});

o.b = 10;
// Runs the setter, which assigns 10 / 2 (5) to the 'a' property

console.log(o.a); // 5
```

Using a computed property name

```
const expr = "foo";

const obj = {
  baz: "bar",
  set [expr](v) {
    this.baz = v;
```

```
  },
};

console.log(obj.baz); // "bar"

obj.foo = "baz";
// Run the setter

console.log(obj.baz); // "baz"
```

See also

- Working with objects guide
- Functions
- `get`
- `Object.defineProperty()`
- Object initializer
- `class`
- Property accessors
- Incompatible ES5 change: literal getter and setter functions must now have exactly zero or one arguments by Jeff Walden (2010)
- More SpiderMonkey changes: ancient, esoteric, very rarely used syntax for creating getters and setters is being removed by Jeff Walden (2010)

Standard built-in objects

This chapter documents all of JavaScript's standard, built-in objects, including their methods and properties.

The term "global objects" (or standard built-in objects) here is not to be confused with **the global object**. Here, "global objects" refer to **objects in the global scope**.

The **global object** itself can be accessed using the `this` operator in the global scope. In fact, the global scope **consists of** the properties of the global object, including inherited properties, if any.

Other objects in the global scope are either created by the user script or provided by the host application. The host objects available in browser contexts are documented in the API reference.

For more information about the distinction between the DOM and core JavaScript, see JavaScript technologies overview.

Standard objects by category

Value properties

These global properties return a simple value. They have no properties or methods.

- `globalThis`
- `Infinity`
- `NaN`
- `undefined`

Function properties

These global functions—functions which are called globally, rather than on an object—directly return their results to the caller.

- `eval()`
- `isFinite()`
- `isNaN()`
- `parseFloat()`
- `parseInt()`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`
- `escape()`
- `unescape()`

Fundamental objects

These objects represent fundamental language constructs.

- `Object`
- `Function`
- `Boolean`
- `Symbol`

Error objects

Error objects are a special type of fundamental object. They include the basic `Error` type, as well as several specialized error types.

- `Error`
- `AggregateError`
- `EvalError`
- `RangeError`

- `ReferenceError`
- `SuppressedError`
- `SyntaxError`
- `TypeError`
- `URIError`
- `InternalError`

Numbers and dates

These are the base objects representing numbers, dates, and mathematical calculations.

- `Number`
- `BigInt`
- `Math`
- `Date`
- `Temporal`

Text processing

These objects represent strings and support manipulating them.

- `String`
- `RegExp`

Indexed collections

These objects represent collections of data which are ordered by an index value. This includes (typed) arrays and array-like constructs.

- `Array`
- `TypedArray`
- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`
- `BigInt64Array`
- `BigUint64Array`
- `Float16Array`
- `Float32Array`
- `Float64Array`

Keyed collections

These objects represent collections which use keys. The iterable collections (`Map` and `Set`) contain elements which are easily iterated in the order of insertion.

- `Map`
- `Set`
- `WeakMap`
- `WeakSet`

Structured data

These objects represent and interact with structured data buffers and data coded using JavaScript Object Notation (JSON).

- `ArrayBuffer`
- `SharedArrayBuffer`
- `DataView`
- `Atomics`
- `JSON`

Managing memory

These objects interact with the garbage collection mechanism.

- `WeakRef`
- `FinalizationRegistry`

Control abstraction objects

Control abstractions can help to structure code, especially async code (without using deeply nested callbacks, for example).

- `Iterator`
- `AsyncIterator`
- `Promise`
- `GeneratorFunction`
- `AsyncGeneratorFunction`
- `Generator`
- `AsyncGenerator`

- `AsyncFunction`
- `DisposableStack`
- `AsyncDisposableStack`

Reflection

- `AbstractModuleSource`
- `Reflect`
- `Proxy`

Internationalization

Additions to the ECMAScript core for language-sensitive functionalities.

- `Intl`
- `Intl.Collator`
- `Intl.DateTimeFormat`
- `Intl.DisplayNames`
- `Intl.DurationFormat`
- `Intl.ListFormat`
- `Intl.Locale`
- `Intl.NumberFormat`
- `Intl.PluralRules`
- `Intl.RelativeTimeFormat`
- `Intl.Segmenter`

AbstractModuleSource

An ***AbstractModuleSource*** object represents the compiled source of a module obtained with `import source` or `import.source()`. There is no directly visible `AbstractModuleSource` constructor.

Description

The `AbstractModuleSource` constructor (often referred to as `%AbstractModuleSource%` to indicate its "intrinsicness", since it does not correspond to any global exposed to a JavaScript program) serves as the superclass of all module source subclasses, providing a common interface of utility methods. This constructor is not directly exposed: there is no global `AbstractModuleSource` property. It is only accessible through `Object.getPrototypeOf(moduleSourceObject.constructor)` and similar.

AbstractModuleSource objects

- `WebAssembly.Module`

JavaScript module source objects will be added by the ECMAScript Module Phase Imports proposal.

Constructor

This object cannot be instantiated directly — attempting to construct it with `new` throws a `TypeError`.

```
import source modSource from "./module.wasm";

new (Object.getPrototypeOf(modSource.constructor))();
// TypeError: Abstract class AbstractModuleSource not directly constructable
```

Instead, obtain a module source object using `import source`, `import.source()`, or a concrete module type's API, such as `WebAssembly.Module()`. These mechanisms do not call the abstract constructor.

Instance properties

These properties are defined on `AbstractModuleSource.prototype` and shared by all `AbstractModuleSource` subclass instances.

`AbstractModuleSource.prototype.constructor`

> The constructor function that created the instance object. `AbstractModuleSource.prototype.constructor` is the hidden `AbstractModuleSource` constructor function, but each module source subclass also defines its own `constructor` property.

`AbstractModuleSource.prototype[Symbol.toStringTag]`

> The initial value of the `AbstractModuleSource.prototype[Symbol.toStringTag]` property is a getter that returns a string based on the constructor's identity, such as `"WebAssembly.Module"` or `"ModuleSource"`. It returns `undefined` if the `this` value is not an instance of one of the module source subclasses. This property is used in `Object.prototype.toString()`.

See also

- `WebAssembly.Module`
- `import.source()`
- `import source`

AggregateError

The **`AggregateError`** object represents an error when several errors need to be wrapped in a single error. It is thrown when multiple errors need to be reported by an operation, for example by `Promise.any()`, when all promises passed to it reject.

Compared to `SuppressedError`, `AggregateError` represents a list of unrelated errors, while `SuppressedError` represents an error that happened during the handling of another error.

`AggregateError` is a subclass of `Error`.

Constructor

`AggregateError()`

Creates a new `AggregateError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `AggregateError.prototype` and shared by all `AggregateError` instances.

`AggregateError.prototype.constructor`

The constructor function that created the instance object. For `AggregateError` instances, the initial value is the `AggregateError` constructor.

`AggregateError.prototype.name`

Represents the name for the type of error. For `AggregateError.prototype.name`, the initial value is `"AggregateError"`.

These properties are own properties of each `AggregateError` instance.

`errors`

An array representing the errors that were aggregated.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching an AggregateError

```
Promise.any([Promise.reject(new Error("some error"))]).catch((e) => {
  console.log(e instanceof AggregateError); // true
  console.log(e.message); // "All Promises rejected"
  console.log(e.name); // "AggregateError"
  console.log(e.errors); // [ Error: "some error" ]
});
```

Creating an AggregateError

```
try {
  throw new AggregateError([new Error("some error")], "Hello");
} catch (e) {
  console.log(e instanceof AggregateError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "AggregateError"
  console.log(e.errors); // [ Error: "some error" ]
}
```

See also

- Polyfill of `AggregateError` in `core-js`
- es-shims polyfill of `AggregateError`
- `Error`
- `Promise.any`

AggregateError() constructor

The **`AggregateError()`** constructor creates `AggregateError` objects.

Syntax

```
new AggregateError(errors)
new AggregateError(errors, message)
new AggregateError(errors, message, options)

AggregateError(errors)
AggregateError(errors, message)
AggregateError(errors, message, options)
```

> **Note:** `AggregateError()` can be called with or without `new`. Both create a new `AggregateError` instance.

Parameters

`errors`

An iterable of errors, may not actually be `Error` instances.

`message` Optional

An optional human-readable description of the aggregate error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

Examples

Creating an AggregateError

```
try {
  throw new AggregateError([new Error("some error")], "Hello");
} catch (e) {
  console.log(e instanceof AggregateError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "AggregateError"
  console.log(e.errors); // [ Error: "some error" ]
}
```

See also

- Polyfill of `AggregateError` in `core-js`
- es-shims polyfill of `AggregateError`
- `Promise.any`

AggregateError: errors

The **`errors`** data property of an `AggregateError` instance contains an array representing the errors that were aggregated.

Value

An `Array` containing values in the same order as the iterable passed as the first argument of the `AggregateError()` constructor.

Property attributes of `AggregateError: errors`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using errors

```
try {
  throw new AggregateError(
    // An iterable of errors
    new Set([new Error("some error"), new Error("another error")]),
    "Multiple errors thrown",
  );
} catch (err) {
  console.log(err.errors);
  // [
  //   Error: some error,
  //   Error: another error
  // ]
}
```

See also

- Control flow and error handling guide
- `AggregateError`
- `Error: cause`

Array

The **`Array`** object, as with arrays in other programming languages, enables storing a collection of multiple items under a single variable name, and has members for performing common array operations.

Description

In JavaScript, arrays aren't primitives but are instead `Array` objects with the following core characteristics:

- **JavaScript arrays are resizable** and **can contain a mix of different data types**. (When those characteristics are undesirable, use typed arrays instead.)
- **JavaScript arrays are not associative arrays** and so, array elements cannot be accessed using arbitrary strings as indexes, but must be accessed using nonnegative integers (or their respective string form) as indexes.
- **JavaScript arrays are zero-indexed**: the first element of an array is at index `0`, the second is at index `1`, and so on — and the last element is at the value of the array's `length` property minus `1`.
- **JavaScript array-copy operations create shallow copies**. (All standard built-in copy operations with *any* JavaScript objects create shallow copies, rather than deep copies).

Array indices

`Array` objects cannot use arbitrary strings as element indexes (as in an associative array) but must use nonnegative integers (or their respective string form). Setting or accessing via non-integers will not set or retrieve an element from the array list itself, but will set or access a variable associated with that array's object property collection. The array's object properties and list of array elements are separate, and the array's traversal and mutation operations cannot be applied to these named properties.

Array elements are object properties in the same way that `toString` is a property (to be specific, however, `toString()` is a method). Nevertheless, trying to access an element of an array as follows throws a syntax error because the property name is not valid:

```
arr.0; // a syntax error
```

JavaScript syntax requires properties beginning with a digit to be accessed using bracket notation instead of dot notation. It's also possible to quote the array indices (e.g., `years['2']` instead of `years[2]`), although usually not necessary.

The `2` in `years[2]` is coerced into a string by the JavaScript engine through an implicit `toString` conversion. As a result, `'2'` and `'02'` would refer to two different slots on the `years` object, and the following example could be `true`:

```
console.log(years["2"] !== years["02"]);
```

Only `years['2']` is an actual array index. `years['02']` is an arbitrary string property that will not be visited in array iteration.

Relationship between length and numerical properties

A JavaScript array's `length` property and numerical properties are connected.

Several of the built-in array methods (e.g., `join()`, `slice()`, `indexOf()`, etc.) take into account the value of an array's `length` property when they're called.

Other methods (e.g., `push()`, `splice()`, etc.) also result in updates to an array's `length` property.

```
const fruits = [];
fruits.push("banana", "apple", "peach");
console.log(fruits.length); // 3
```

When setting a property on a JavaScript array when the property is a valid array index and that index is outside the current bounds of the array, the engine will update the array's `length` property accordingly:

```
fruits[5] = "mango";
console.log(fruits[5]); // 'mango'
console.log(Object.keys(fruits)); // ['0', '1', '2', '5']
console.log(fruits.length); // 6
```

Increasing the `length` extends the array by adding empty slots without creating any new elements — not even `undefined`.

```
fruits.length = 10;
console.log(fruits); // ['banana', 'apple', 'peach', empty x 2, 'mango', empty x 4]
console.log(Object.keys(fruits)); // ['0', '1', '2', '5']
console.log(fruits.length); // 10
console.log(fruits[8]); // undefined
```

Decreasing the `length` property does, however, delete elements.

```
fruits.length = 2;
console.log(Object.keys(fruits)); // ['0', '1']
console.log(fruits.length); // 2
```

This is explained further on the `length` page.

Array methods and empty slots

Array methods have different behaviors when encountering empty slots in sparse arrays. In general, older methods (e.g., `forEach`) treat empty slots differently from indices that contain `undefined`.

Methods that have special treatment for empty slots include the following: `concat()`, `copyWithin()`, `every()`, `filter()`, `flat()`, `flatMap()`, `forEach()`, `indexOf()`, `lastIndexOf()`, `map()`, `reduce()`, `reduceRight()`, `reverse()`, `slice()`, `some()`, `sort()`, and `splice()`. Iteration methods such as `forEach` don't visit empty slots at all. Other methods, such as `concat`, `copyWithin`, etc., preserve empty slots when doing the copying, so in the end the array is still sparse.

```
const colors = ["red", "yellow", "blue"];
colors[5] = "purple";
colors.forEach((item, index) => {
  console.log(`${index}: ${item}`);
});
```

```
// Output:
// 0: red
// 1: yellow
// 2: blue
// 5: purple

colors.reverse(); // ['purple', empty × 2, 'blue', 'yellow', 'red']
```

Newer methods (e.g., `keys`) do not treat empty slots specially and treat them as if they contain `undefined`. Methods that conflate empty slots with `undefined` elements include the following: `entries()`, `fill()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, `includes()`, `join()`, `keys()`, `toLocaleString()`, `toReversed()`, `toSorted()`, `toSpliced()`, `values()`, and `with()`.

```
const colors = ["red", "yellow", "blue"];
colors[5] = "purple";
const iterator = colors.keys();
for (const key of iterator) {
  console.log(`${key}: ${colors[key]}`);
}
// Output
// 0: red
// 1: yellow
// 2: blue
// 3: undefined
// 4: undefined
// 5: purple

const newColors = colors.toReversed(); // ['purple', undefined, undefined, 'blue', 'yellow', 'red']
```

Copying methods and mutating methods

Some methods do not mutate the existing array that the method was called on, but instead return a new array. They do so by first constructing a new array and then populating it with elements. The copy always happens *shallowly* — the method never copies anything beyond the initially created array. Elements of the original array(s) are copied into the new array as follows:

- Objects: the object reference is copied into the new array. Both the original and new array refer to the same object. That is, if a referenced object is modified, the changes are visible to both the new and original arrays.
- Primitive types such as strings, numbers and booleans (not `String`, `Number`, and `Boolean` objects): their values are copied into the new array.

Other methods mutate the array that the method was called on, in which case their return value differs depending on the method: sometimes a reference to the same array, sometimes the length of the new array.

The following methods create new arrays by accessing `this.constructor[Symbol.species]` to determine the constructor to use: `concat()`, `filter()`, `flat()`, `flatMap()`, `map()`, `slice()`, and `splice()` (to construct the array of removed elements that's returned).

The following methods always create new arrays with the `Array` base constructor: `toReversed()`, `toSorted()`, `toSpliced()`, and `with()`.

The following table lists the methods that mutate the original array, and the corresponding non-mutating alternative:

Mutating method	Non-mutating alternative
`copyWithin()`	No one-method alternative
`fill()`	No one-method alternative
`pop()`	`slice(0, -1)`
`push(v1, v2)`	`concat([v1, v2])`
`reverse()`	`toReversed()`
`shift()`	`slice(1)`
`sort()`	`toSorted()`
`splice()`	`toSpliced()`
`unshift(v1, v2)`	`toSpliced(0, 0, v1, v2)`

An easy way to change a mutating method into a non-mutating alternative is to use the spread syntax or `slice()` to create a copy first:

```
arr.copyWithin(0, 1, 2); // mutates arr
const arr2 = arr.slice().copyWithin(0, 1, 2); // does not mutate arr
const arr3 = [...arr].copyWithin(0, 1, 2); // does not mutate arr
```

Iterative methods

Many array methods take a callback function as an argument. The callback function is called sequentially and at most once for each element in the array, and the return value of the callback function is used to determine the return value of the method. They all share the same signature:

```
method(callbackFn, thisArg)
```

Where `callbackFn` takes three arguments:

`element`

The current element being processed in the array.

`index`

: The index of the current element being processed in the array.

`array`

: The array that the method was called upon.

What `callbackFn` is expected to return depends on the array method that was called.

The `thisArg` argument (defaults to `undefined`) will be used as the `this` value when calling `callbackFn`. The `this` value ultimately observable by `callbackFn` is determined according to the usual rules: if `callbackFn` is non-strict, primitive `this` values are wrapped into objects, and `undefined`/`null` is substituted with `globalThis`. The `thisArg` argument is irrelevant for any `callbackFn` defined with an arrow function, as arrow functions don't have their own `this` binding.

The `array` argument passed to `callbackFn` is most useful if you want to read another index during iteration, because you may not always have an existing variable that refers to the current array. You should generally not mutate the array during iteration (see mutating initial array in iterative methods), but you can also use this argument to do so. The `array` argument is *not* the array that is being built, in the case of methods like `map()`, `filter()`, and `flatMap()` — there is no way to access the array being built from the callback function.

All iterative methods are copying and generic, although they behave differently with empty slots.

The following methods are iterative: `every()`, `filter()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, `flatMap()`, `forEach()`, `map()`, and `some()`.

In particular, `every()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, and `some()` do not always invoke `callbackFn` on every element — they stop iteration as soon as the return value is determined.

The `reduce()` and `reduceRight()` methods also take a callback function and run it at most once for each element in the array, but they have slightly different signatures from typical iterative methods (for example, they don't accept `thisArg`).

The `sort()` method also takes a callback function, but it is not an iterative method. It mutates the array in-place, doesn't accept `thisArg`, and may invoke the callback multiple times on an index.

Iterative methods iterate the array like the following (with a lot of technical details omitted):

```
function method(callbackFn, thisArg) {
  const length = this.length;
  for (let i = 0; i < length; i++) {
    if (i in this) {
      const result = callbackFn.call(thisArg, this[i], i, this);
      // Do something with result; maybe return early
    }
  }
}
```

Note the following:

1. Not all methods do the `i in this` test. The `find`, `findIndex`, `findLast`, and `findLastIndex` methods do not, but other methods do.
2. The `length` is memorized before the loop starts. This affects how insertions and deletions during iteration are handled (see mutating initial array in iterative methods).
3. The method doesn't memorize the array contents, so if any index is modified during iteration, the new value might be observed.
4. The code above iterates the array in ascending order of index. Some methods iterate in descending order of index (`for (let i = length - 1; i >= 0; i--)`): `reduceRight()`, `findLast()`, and `findLastIndex()`.
5. `reduce` and `reduceRight` have slightly different signatures and do not always start at the first/last element.

Generic array methods

Array methods are always generic — they don't access any internal data of the array object. They only access the array elements through the `length` property and the indexed elements. This means that they can be called on array-like objects as well.

```
const arrayLike = {
  0: "a",
  1: "b",
  length: 2,
};
console.log(Array.prototype.join.call(arrayLike, "+")); // 'a+b'
```

Normalization of the length property

The `length` property is converted to an integer and then clamped to the range between 0 and 2^{53} - 1. `NaN` becomes `0`, so even when `length` is not present or is `undefined`, it behaves as if it has value `0`.

The language avoids setting `length` to an unsafe integer. All built-in methods will throw a `TypeError` if `length` will be set to a number greater than 2^{53} - 1. However, because the `length` property of arrays throws an error if it's set to greater than 2^{32} - 1, the safe integer threshold is usually not reached unless the method is called on a non-array object.

```
Array.prototype.flat.call({}); // []
```

Some array methods set the `length` property of the array object. They always set the value after normalization, so `length` always ends as an integer.

```
const a = { length: 0.7 };
Array.prototype.push.call(a);
```

```
console.log(a.length); // 0
```

Array-like objects

The term *array-like object* refers to any object that doesn't throw during the `length` conversion process described above. In practice, such object is expected to actually have a `length` property and to have indexed elements in the range `0` to `length - 1`. (If it doesn't have all indices, it will be functionally equivalent to a sparse array.) Any integer index less than zero or greater than `length - 1` is ignored when an array method operates on an array-like object.

Many DOM objects are array-like — for example, `NodeList` and `HTMLCollection`. The `arguments` object is also array-like. You can call array methods on them even if they don't have these methods themselves.

```
function f() {
  console.log(Array.prototype.join.call(arguments, "+"));
}

f("a", "b"); // 'a+b'
```

Constructor

`Array()`

Creates a new `Array` object.

Static properties

`Array[Symbol.species]`

Returns the `Array` constructor.

Static methods

`Array.from()`

Creates a new `Array` instance from an iterable or array-like object.

`Array.fromAsync()`

Creates a new `Array` instance from an async iterable, iterable, or array-like object.

`Array.isArray()`

Returns `true` if the argument is an array, or `false` otherwise.

`Array.of()`

Creates a new `Array` instance with a variable number of arguments, regardless of number or type of the arguments.

Instance properties

These properties are defined on `Array.prototype` and shared by all `Array` instances.

`Array.prototype.constructor`

The constructor function that created the instance object. For `Array` instances, the initial value is the `Array` constructor.

`Array.prototype[Symbol.unscopables]`

Contains property names that were not included in the ECMAScript standard prior to the ES2015 version and that are ignored for `with` statement-binding purposes.

These properties are own properties of each `Array` instance.

`length`

Reflects the number of elements in an array.

Instance methods

`Array.prototype.at()`

Returns the array item at the given index. Accepts negative integers, which count back from the last item.

`Array.prototype.concat()`

Returns a new array that is the calling array joined with other array(s) and/or value(s).

`Array.prototype.copyWithin()`

Copies a sequence of array elements within an array.

`Array.prototype.entries()`

Returns a new *array iterator* object that contains the key/value pairs for each index in an array.

`Array.prototype.every()`

Returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`.

`Array.prototype.fill()`

Fills all the elements of an array from a start index to an end index with a static value.

`Array.prototype.filter()`

Returns a new array containing all elements of the calling array for which the provided filtering function returns `true`.

`Array.prototype.find()`

Returns the value of the first element in the array that satisfies the provided testing function, or `undefined` if no appropriate element is found.

`Array.prototype.findIndex()`

Returns the index of the first element in the array that satisfies the provided testing function, or `-1` if no appropriate element was found.

`Array.prototype.findLast()`

Returns the value of the last element in the array that satisfies the provided testing function, or `undefined` if no appropriate element is found.

`Array.prototype.findLastIndex()`

Returns the index of the last element in the array that satisfies the provided testing function, or `-1` if no appropriate element was found.

`Array.prototype.flat()`

Returns a new array with all sub-array elements concatenated into it recursively up to the specified depth.

`Array.prototype.flatMap()`

Returns a new array formed by applying a given callback function to each element of the calling array, and then flattening the result by one level.

`Array.prototype.forEach()`

Calls a function for each element in the calling array.

`Array.prototype.includes()`

Determines whether the calling array contains a value, returning `true` or `false` as appropriate.

`Array.prototype.indexOf()`

Returns the first (least) index at which a given element can be found in the calling array.

`Array.prototype.join()`

Returns a new string that is the concatenation of all elements in this array, separated by commas or a specified separator string.

`Array.prototype.keys()`

Returns a new *array iterator* that contains the keys for each index in the calling array.

`Array.prototype.lastIndexOf()`

Returns the last (greatest) index at which a given element can be found in the calling array, or `-1` if none is found.

`Array.prototype.map()`

Returns a new array containing the results of invoking a function on every element in the calling array.

`Array.prototype.pop()`

Removes the last element from an array and returns that element.

`Array.prototype.push()`

Adds one or more elements to the end of an array, and returns the new `length` of the array.

`Array.prototype.reduce()`

Executes a user-supplied "reducer" callback function on each element of the array (from left to right), to reduce it to a single value.

`Array.prototype.reduceRight()`

Executes a user-supplied "reducer" callback function on each element of the array (from right to left), to reduce it to a single value.

`Array.prototype.reverse()`

Reverses the order of the elements of an array *in place*. (First becomes the last, last becomes first.)

`Array.prototype.shift()`

Removes the first element from an array and returns that element.

`Array.prototype.slice()`

Extracts a section of the calling array and returns a new array.

`Array.prototype.some()`

Returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`.

`Array.prototype.sort()`

Sorts the elements of an array in place and returns the array.

`Array.prototype.splice()`

Adds and/or removes elements from an array.

`Array.prototype.toLocaleString()`

Returns a localized string representing the calling array and its elements. Overrides the `Object.prototype.toLocaleString()` method.

`Array.prototype.toReversed()`

Returns a new array with the elements in reversed order, without modifying the original array.

`Array.prototype.toSorted()`

Returns a new array with the elements sorted in ascending order, without modifying the original array.

`Array.prototype.toSpliced()`

Returns a new array with some elements removed and/or replaced at a given index, without modifying the original array.

`Array.prototype.toString()`

Returns a string representing the calling array and its elements. Overrides the `Object.prototype.toString()` method.

`Array.prototype.unshift()`

Adds one or more elements to the front of an array, and returns the new `length` of the array.

`Array.prototype.values()`

Returns a new *array iterator* object that contains the values for each index in the array.

`Array.prototype.with()`

Returns a new array with the element at the given index replaced with the given value, without modifying the original array.

`Array.prototype[Symbol.iterator]()`

An alias for the `values()` method by default.

Examples

This section provides some examples of common array operations in JavaScript.

> **Note:** If you're not yet familiar with array basics, consider first reading JavaScript First Steps: Arrays, which explains what arrays are, and includes other examples of common array operations.

Create an array

This example shows three ways to create new array: first using array literal notation, then using the `Array()` constructor, and finally using `String.prototype.split()` to build the array from a string.

```
// 'fruits' array created using array literal notation.
const fruits = ["Apple", "Banana"];
console.log(fruits.length);
// 2

// 'fruits2' array created using the Array() constructor.
const fruits2 = new Array("Apple", "Banana");
console.log(fruits2.length);
// 2

// 'fruits3' array created using String.prototype.split().
const fruits3 = "Apple, Banana".split(", ");
console.log(fruits3.length);
// 2
```

Create a string from an array

This example uses the `join()` method to create a string from the `fruits` array.

```
const fruits = ["Apple", "Banana"];
const fruitsString = fruits.join(", ");
```

```
console.log(fruitsString);
// "Apple, Banana"
```

Access an array item by its index

This example shows how to access items in the `fruits` array by specifying the index number of their position in the array.

```
const fruits = ["Apple", "Banana"];

// The index of an array's first element is always 0.
fruits[0]; // Apple

// The index of an array's second element is always 1.
fruits[1]; // Banana

// The index of an array's last element is always one
// less than the length of the array.
fruits[fruits.length - 1]; // Banana

// Using an index number larger than the array's length
// returns 'undefined'.
fruits[99]; // undefined
```

Find the index of an item in an array

This example uses the `indexOf()` method to find the position (index) of the string `"Banana"` in the `fruits` array.

```
const fruits = ["Apple", "Banana"];
console.log(fruits.indexOf("Banana"));
// 1
```

Check if an array contains a certain item

This example shows two ways to check if the `fruits` array contains `"Banana"` and `"Cherry"`: first with the `includes()` method, and then with the `indexOf()` method to test for an index value that's not `-1`.

```
const fruits = ["Apple", "Banana"];

fruits.includes("Banana"); // true
fruits.includes("Cherry"); // false

// If indexOf() doesn't return -1, the array contains the given item.
fruits.indexOf("Banana") !== -1; // true
fruits.indexOf("Cherry") !== -1; // false
```

Append an item to an array

This example uses the `push()` method to append a new string to the `fruits` array.

```
const fruits = ["Apple", "Banana"];
const newLength = fruits.push("Orange");
console.log(fruits);
// ["Apple", "Banana", "Orange"]
console.log(newLength);
// 3
```

Remove the last item from an array

This example uses the `pop()` method to remove the last item from the `fruits` array.

```
const fruits = ["Apple", "Banana", "Orange"];
const removedItem = fruits.pop();
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItem);
// Orange
```

> **Note:** `pop()` can only be used to remove the last item from an array. To remove multiple items from the end of an array, see the next example.

Remove multiple items from the end of an array

This example uses the `splice()` method to remove the last 3 items from the `fruits` array.

```
const fruits = ["Apple", "Banana", "Strawberry", "Mango", "Cherry"];
const start = -3;
const removedItems = fruits.splice(start);
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItems);
// ["Strawberry", "Mango", "Cherry"]
```

Truncate an array down to just its first N items

This example uses the `splice()` method to truncate the `fruits` array down to just its first 2 items.

```
const fruits = ["Apple", "Banana", "Strawberry", "Mango", "Cherry"];
const start = 2;
const removedItems = fruits.splice(start);
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItems);
// ["Strawberry", "Mango", "Cherry"]
```

Remove the first item from an array

This example uses the `shift()` method to remove the first item from the `fruits` array.

```
const fruits = ["Apple", "Banana"];
const removedItem = fruits.shift();
console.log(fruits);
// ["Banana"]
console.log(removedItem);
// Apple
```

> **Note:** `shift()` can only be used to remove the first item from an array. To remove multiple items from the beginning of an array, see the next example.

Remove multiple items from the beginning of an array

This example uses the `splice()` method to remove the first 3 items from the `fruits` array.

```
const fruits = ["Apple", "Strawberry", "Cherry", "Banana", "Mango"];
const start = 0;
const deleteCount = 3;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Banana", "Mango"]
console.log(removedItems);
// ["Apple", "Strawberry", "Cherry"]
```

Add a new first item to an array

This example uses the `unshift()` method to add, at index `0`, a new item to the `fruits` array — making it the new first item in the array.

```
const fruits = ["Banana", "Mango"];
const newLength = fruits.unshift("Strawberry");
console.log(fruits);
// ["Strawberry", "Banana", "Mango"]
console.log(newLength);
// 3
```

Remove a single item by index

This example uses the `splice()` method to remove the string `"Banana"` from the `fruits` array — by specifying the index position of `"Banana"`.

```
const fruits = ["Strawberry", "Banana", "Mango"];
const start = fruits.indexOf("Banana");
const deleteCount = 1;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Strawberry", "Mango"]
console.log(removedItems);
// ["Banana"]
```

Remove multiple items by index

This example uses the `splice()` method to remove the strings `"Banana"` and `"Strawberry"` from the `fruits` array — by specifying the index position of `"Banana"`, along with a count of the number of total items to remove.

```
const fruits = ["Apple", "Banana", "Strawberry", "Mango"];
const start = 1;
const deleteCount = 2;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Apple", "Mango"]
console.log(removedItems);
// ["Banana", "Strawberry"]
```

Replace multiple items in an array

This example uses the `splice()` method to replace the last 2 items in the `fruits` array with new items.

```
const fruits = ["Apple", "Banana", "Strawberry"];
const start = -2;
const deleteCount = 2;
const removedItems = fruits.splice(start, deleteCount, "Mango", "Cherry");
console.log(fruits);
// ["Apple", "Mango", "Cherry"]
console.log(removedItems);
// ["Banana", "Strawberry"]
```

Iterate over an array

This example uses a `for...of` loop to iterate over the `fruits` array, logging each item to the console.

```
const fruits = ["Apple", "Mango", "Cherry"];
for (const fruit of fruits) {
  console.log(fruit);
}
// Apple
// Mango
// Cherry
```

But `for...of` is just one of many ways to iterate over any array; for more ways, see Loops and iteration, and see the documentation for the `every()`, `filter()`, `flatMap()`, `map()`, `reduce()`, and `reduceRight()` methods — and see the next example, which uses the `forEach()` method.

Call a function on each element in an array

This example uses the `forEach()` method to call a function on each element in the `fruits` array; the function causes each item to be logged to the console, along with the item's index number.

```
const fruits = ["Apple", "Mango", "Cherry"];
fruits.forEach((item, index, array) => {
  console.log(item, index);
});
// Apple 0
// Mango 1
// Cherry 2
```

Merge multiple arrays together

This example uses the `concat()` method to merge the `fruits` array with a `moreFruits` array, to produce a new `combinedFruits` array. Notice that `fruits` and `moreFruits` remain unchanged.

```
const fruits = ["Apple", "Banana", "Strawberry"];
const moreFruits = ["Mango", "Cherry"];
const combinedFruits = fruits.concat(moreFruits);
console.log(combinedFruits);
// ["Apple", "Banana", "Strawberry", "Mango", "Cherry"]

// The 'fruits' array remains unchanged.
console.log(fruits);
// ["Apple", "Banana", "Strawberry"]

// The 'moreFruits' array also remains unchanged.
console.log(moreFruits);
// ["Mango", "Cherry"]
```

Copy an array

This example shows three ways to create a new array from the existing `fruits` array: first by using spread syntax, then by using the `from()` method, and then by using the `slice()` method.

```
const fruits = ["Strawberry", "Mango"];

// Create a copy using spread syntax.
const fruitsCopy = [...fruits];
// ["Strawberry", "Mango"]

// Create a copy using the from() method.
const fruitsCopy2 = Array.from(fruits);
// ["Strawberry", "Mango"]

// Create a copy using the slice() method.
const fruitsCopy3 = fruits.slice();
// ["Strawberry", "Mango"]
```

All built-in array-copy operations (spread syntax, `Array.from()`, `Array.prototype.slice()`, and `Array.prototype.concat()`) create shallow copies. If you instead want a deep copy of an array, you can use `JSON.stringify()` to convert the array to a JSON string, and then `JSON.parse()` to convert the string back into a new array that's completely independent from the original array.

```
const fruitsDeepCopy = JSON.parse(JSON.stringify(fruits));
```

You can also create deep copies using the `structuredClone()` method, which has the advantage of allowing transferable objects in the source to be *transferred* to the new copy, rather than just cloned.

Finally, it's important to understand that assigning an existing array to a new variable doesn't create a copy of either the array or its elements. Instead the new variable is just a reference, or alias, to the original array; that is, the original array's name and the new variable name are just two names for the exact same object (and so will always evaluate as strictly equivalent). Therefore, if you make any changes at all either to the value of the original array or to the value of the new variable, the other will change, too:

```
const fruits = ["Strawberry", "Mango"];
const fruitsAlias = fruits;
// 'fruits' and 'fruitsAlias' are the same object, strictly equivalent.
fruits === fruitsAlias; // true
// Any changes to the 'fruits' array change 'fruitsAlias' too.
fruits.unshift("Apple", "Banana");
console.log(fruits);
// ['Apple', 'Banana', 'Strawberry', 'Mango']
console.log(fruitsAlias);
// ['Apple', 'Banana', 'Strawberry', 'Mango']
```

Creating a two-dimensional array

The following creates a chessboard as a two-dimensional array of strings. The first move is made by copying the `'p'` in `board[6][4]` to `board[4][4]`. The old position at `[6][4]` is made blank.

```
const board = [
  ["R", "N", "B", "Q", "K", "B", "N", "R"],
  ["P", "P", "P", "P", "P", "P", "P", "P"],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  ["p", "p", "p", "p", "p", "p", "p", "p"],
  ["r", "n", "b", "q", "k", "b", "n", "r"],
];

console.log(`${board.join("\n")}\n\n`);

// Move King's Pawn forward 2
board[4][4] = board[6][4];
board[6][4] = " ";
console.log(board.join("\n"));
```

Here is the output:

```
R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
 , , , , , , ,
 , , , , , , ,
 , , , , , , ,
 , , , , , , ,
p,p,p,p,p,p,p,p
r,n,b,q,k,b,n,r

R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
 , , , , , , ,
 , , , , , , ,
 , , , ,p, , ,
 , , , , , , ,
p,p,p,p, ,p,p,p
r,n,b,q,k,b,n,r
```

Using an array to tabulate a set of values

```
const values = [];
for (let x = 0; x < 10; x++) {
  values.push([2 ** x, 2 * x ** 2]);
}
console.table(values);
```

Results in

```
// The first column is the index
0  1    0
1  2    2
2  4    8
3  8    18
4  16   32
5  32   50
6  64   72
7  128  98
8  256  128
9  512  162
```

Creating an array using the result of a match

The result of a match between a `RegExp` and a string can create a JavaScript array that has properties and elements which provide information about the match. Such an array is returned by `RegExp.prototype.exec()` and `String.prototype.match()`.

For example:

```
// Match one d followed by one or more b's followed by one d
// Remember matched b's and the following d
// Ignore case

const myRe = /d(b+)(d)/i;
const execResult = myRe.exec("cdbBdbsbz");

console.log(execResult.input); // 'cdbBdbsbz'
console.log(execResult.index); // 1
console.log(execResult); // [ "dbBd", "bB", "d" ]
```

For more information about the result of a match, see the `RegExp.prototype.exec()` and `String.prototype.match()` pages.

Mutating initial array in iterative methods

Iterative methods do not mutate the array on which it is called, but the function provided as `callbackFn` can. The key principle to remember is that only indexes between 0 and `arrayLength - 1` are visited, where `arrayLength` is the length of the array at the time the array method was first called, but the element passed to the callback is the value at the time the index is visited. Therefore:

- `callbackFn` will not visit any elements added beyond the array's initial length when the call to the iterative method began.
- Changes to already-visited indexes do not cause `callbackFn` to be invoked on them again.
- If an existing, yet-unvisited element of the array is changed by `callbackFn`, its value passed to the `callbackFn` will be the value at the time that element gets visited. Removed elements are not visited.

> **Warning:** Concurrent modifications of the kind described above frequently lead to hard-to-understand code and are generally to be avoided (except in special cases).

The following examples use the `forEach` method as an example, but other methods that visit indexes in ascending order work in the same way. We will first define a helper function:

```
function testSideEffect(effect) {
  const arr = ["e1", "e2", "e3", "e4"];
  arr.forEach((elem, index, arr) => {
    console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
    effect(arr, index);
  });
  console.log(`Final array: [${arr.join(", ")}]`);
}
```

Modification to indexes not visited yet will be visible once the index is reached:

```
testSideEffect((arr, index) => {
  if (index + 1 < arr.length) arr[index + 1] += "*";
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2*, e3, e4], index: 1, elem: e2*
// array: [e1, e2*, e3*, e4], index: 2, elem: e3*
// array: [e1, e2*, e3*, e4*], index: 3, elem: e4*
// Final array: [e1, e2*, e3*, e4*]
```

Modification to already visited indexes does not change iteration behavior, although the array will be different afterwards:

```
testSideEffect((arr, index) => {
  if (index > 0) arr[index - 1] += "*";
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1*, e2, e3, e4], index: 2, elem: e3
// array: [e1*, e2*, e3, e4], index: 3, elem: e4
// Final array: [e1*, e2*, e3*, e4]
```

Inserting *n* elements at unvisited indexes that are less than the initial array length will make them be visited. The last *n* elements in the original array that now have index greater than the initial array length will not be visited:

```
testSideEffect((arr, index) => {
  if (index === 1) arr.splice(2, 0, "new");
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, new, e3, e4], index: 2, elem: new
// array: [e1, e2, new, e3, e4], index: 3, elem: e3
// Final array: [e1, e2, new, e3, e4]
// e4 is not visited because it now has index 4
```

Inserting *n* elements with index greater than the initial array length will not make them be visited:

```
testSideEffect((arr) => arr.push("new"));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4, new], index: 1, elem: e2
// array: [e1, e2, e3, e4, new, new], index: 2, elem: e3
// array: [e1, e2, e3, e4, new, new, new], index: 3, elem: e4
// Final array: [e1, e2, e3, e4, new, new, new, new]
```

Inserting *n* elements at already visited indexes will not make them be visited, but it shifts remaining elements back by *n*, so the current index and the *n* - *1* elements before it are visited again:

```
testSideEffect((arr, index) => arr.splice(index, 0, "new"));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [new, e1, e2, e3, e4], index: 1, elem: e1
// array: [new, new, e1, e2, e3, e4], index: 2, elem: e1
// array: [new, new, new, e1, e2, e3, e4], index: 3, elem: e1
// Final array: [new, new, new, new, e1, e2, e3, e4]
// e1 keeps getting visited because it keeps getting shifted back
```

Deleting *n* elements at unvisited indexes will make them not be visited anymore. Because the array has shrunk, the last *n* iterations will visit out-of-bounds indexes. If the method ignores non-existent indexes (see array methods and empty slots), the last *n* iterations will be skipped; otherwise, they will receive `undefined`:

```
testSideEffect((arr, index) => {
  if (index === 1) arr.splice(2, 1);
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, e4], index: 2, elem: e4
// Final array: [e1, e2, e4]
// Does not visit index 3 because it's out-of-bounds

// Compare this with find(), which treats nonexistent indexes as undefined:
const arr2 = ["e1", "e2", "e3", "e4"];
arr2.find((elem, index, arr) => {
  console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
  if (index === 1) arr.splice(2, 1);
  return false;
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, e4], index: 2, elem: e4
// array: [e1, e2, e4], index: 3, elem: undefined
```

Deleting *n* elements at already visited indexes does not change the fact that they were visited before they get deleted. Because the array has shrunk, the next *n* elements after the current index are skipped. If the method ignores non-existent indexes, the last *n* iterations will be skipped; otherwise, they will receive `undefined`:

```
testSideEffect((arr, index) => arr.splice(index, 1));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// Does not visit e2 because e2 now has index 0, which has already been visited
// array: [e2, e3, e4], index: 1, elem: e3
// Does not visit e4 because e4 now has index 1, which has already been visited
// Final array: [e2, e4]
// Index 2 is out-of-bounds, so it's not visited

// Compare this with find(), which treats nonexistent indexes as undefined:
const arr2 = ["e1", "e2", "e3", "e4"];
arr2.find((elem, index, arr) => {
  console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
  arr.splice(index, 1);
  return false;
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e2, e3, e4], index: 1, elem: e3
// array: [e2, e4], index: 2, elem: undefined
// array: [e2, e4], index: 3, elem: undefined
```

For methods that iterate in descending order of index, insertion causes elements to be skipped, and deletion causes elements to be visited multiple times. Adjust the code above yourself to see the effects.

See also

- Indexed collections guide
- `TypedArray`
- `ArrayBuffer`

Array() constructor

The **`Array()`** constructor creates `Array` objects.

Syntax

```
new Array()
new Array(element1)
new Array(element1, element2)
new Array(element1, element2, /* …, */ elementN)
new Array(arrayLength)

Array()
Array(element1)
Array(element1, element2)
Array(element1, element2, /* …, */ elementN)
Array(arrayLength)
```

> **Note:** `Array()` can be called with or without `new`. Both create a new `Array` instance.

Parameters

`element1, ..., elementN`

A JavaScript array is initialized with the given elements, except in the case where a single argument is passed to the `Array` constructor and that argument is a number (see the `arrayLength` parameter below). Note that this special case only applies to JavaScript arrays created with the `Array` constructor, not array literals created with the square bracket syntax.

`arrayLength`

If the only argument passed to the `Array` constructor is an integer between 0 and 2^{32} - 1 (inclusive), this returns a new JavaScript array with its `length` property set to that number.

> **Note:** This implies an array of `arrayLength` empty slots, not slots with actual `undefined` values — see sparse arrays.

Exceptions

`RangeError`

Thrown if there's only one argument (`arrayLength`) that is a number, but its value is not an integer or not between 0 and 2^{32} - 1 (inclusive).

Examples

Array literal notation

Arrays can be created using the literal notation:

```
const fruits = ["Apple", "Banana"];

console.log(fruits.length); // 2
console.log(fruits[0]); // "Apple"
```

Array constructor with a single parameter

Arrays can be created using a constructor with a single number parameter. An array is created with its `length` property set to that number, and the array elements are empty slots.

```
const arrayEmpty = new Array(2);

console.log(arrayEmpty.length); // 2
console.log(arrayEmpty[0]); // undefined; actually, it is an empty slot
console.log(0 in arrayEmpty); // false
console.log(1 in arrayEmpty); // false
```

```
const arrayOfOne = new Array("2"); // Not the number 2 but the string "2"

console.log(arrayOfOne.length); // 1
console.log(arrayOfOne[0]); // "2"
```

Array constructor with multiple parameters

If more than one argument is passed to the constructor, a new `Array` with the given elements is created.

```
const fruits = new Array("Apple", "Banana");

console.log(fruits.length); // 2
console.log(fruits[0]); // "Apple"
```

See also

- Indexed collections guide
- `Array`

Array.prototype.at()

The **`at()`** method of `Array` instances takes an integer value and returns the item at that index, allowing for positive and negative integers. Negative integers count back from the last item in the array.

Try it

```
const array = [5, 12, 8, 130, 44];

let index = 2;

console.log(`An index of ${index} returns ${array.at(index)}`);
// Expected output: "An index of 2 returns 8"

index = -2;

console.log(`An index of ${index} returns ${array.at(index)}`);
// Expected output: "An index of -2 returns 130"
```

Syntax

```
at(index)
```

Parameters

`index`

> Zero-based index of the array element to be returned, converted to an integer. Negative index counts back from the end of the array — if `index < 0`, `index + array.length` is accessed.

Return value

The element in the array matching the given index. Always returns `undefined` if `index < -array.length` or `index >= array.length` without attempting to access the corresponding property.

Description

The `at()` method is equivalent to the bracket notation when `index` is a non-negative integer. For example, `array[0]` and `array.at(0)` both return the first item. However, when counting elements from the end of the array, you cannot use `array[-1]` like you may in Python or R, because all values inside the square brackets are treated literally as string properties, so you will end up reading `array["-1"]`, which is just a normal string property instead of an array index.

The usual practice is to access `length` and calculate the index from that — for example, `array[array.length - 1]`. The `at()` method allows relative indexing, so this can be shortened to `array.at(-1)`.

By combining `at()` with `with()`, you can both read and write (respectively) an array using negative indices.

The `at()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Return the last value of an array

The following example provides a function which returns the last element found in a specified array.

```
// Our array with items
const cart = ["apple", "banana", "pear"];

// A function which returns the last item of a given array
function returnLast(arr) {
  return arr.at(-1);
}

// Get the last item of our array 'cart'
const item1 = returnLast(cart);
console.log(item1); // 'pear'

// Add an item to our 'cart' array
cart.push("orange");
const item2 = returnLast(cart);
console.log(item2); // 'orange'
```

Comparing methods

This example compares different ways to select the penultimate (last but one) item of an `Array`. While all the methods shown below are valid, this example highlights the succinctness and readability of the `at()` method.

```
// Our array with items
const colors = ["red", "green", "blue"];

// Using length property
const lengthWay = colors[colors.length - 2];
console.log(lengthWay); // 'green'

// Using slice() method. Note an array is returned
const sliceWay = colors.slice(-2, -1);
```

```
console.log(sliceWay[0]); // 'green'

// Using at() method
const atWay = colors.at(-2);
console.log(atWay); // 'green'
```

Calling at() on non-array objects

The `at()` method reads the `length` property of `this` and calculates the index to access.

```
const arrayLike = {
  length: 2,
  0: "a",
  1: "b",
  2: "c", // ignored by at() since length is 2
};
console.log(Array.prototype.at.call(arrayLike, 0)); // "a"
console.log(Array.prototype.at.call(arrayLike, 2)); // undefined
```

See also

- Polyfill of `Array.prototype.at` in `core-js`
- es-shims polyfill of `Array.prototype.at`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.indexOf()`
- `Array.prototype.with()`
- `TypedArray.prototype.at()`
- `String.prototype.at()`

Array.prototype.concat()

The **`concat()`** method of `Array` instances is used to merge two or more arrays. This method does not change the existing arrays, but instead returns a new array.

Try it

```
const array1 = ["a", "b", "c"];
const array2 = ["d", "e", "f"];
const array3 = array1.concat(array2);

console.log(array3);
// Expected output: Array ["a", "b", "c", "d", "e", "f"]
```

Syntax

```
concat()
concat(value1)
concat(value1, value2)
concat(value1, value2, /* … */ valueN)
```

Parameters

`value1, ..., valueN` Optional

> Arrays and/or values to concatenate into a new array. If all `valueN` parameters are omitted, `concat` returns a shallow copy of the existing array on which it is called. See the description below for more details.

Return value

A new `Array` instance.

Description

The `concat` method creates a new array. The array will first be populated by the elements in the object on which it is called. Then, for each argument, its value will be concatenated into the array — for normal objects or primitives, the argument itself will become an element of the final array; for arrays or array-like objects with the property `Symbol.isConcatSpreadable` set to a truthy value, each element of the argument will be independently added to the final array. The `concat` method does not recurse into nested array arguments.

The `concat()` method is a copying method. It does not alter `this` or any of the arrays provided as arguments but instead returns a shallow copy that contains the same elements as the ones from the original arrays.

The `concat()` method preserves empty slots if any of the source arrays is sparse.

The `concat()` method is generic. The `this` value is treated in the same way as the other arguments (except it will be converted to an object first), which means plain objects will be directly prepended to the resulting array, while array-like objects with truthy `[Symbol.isConcatSpreadable]` will be spread into the resulting array.

Examples

Concatenating two arrays

The following code concatenates two arrays:

```
const letters = ["a", "b", "c"];
const numbers = [1, 2, 3];

const alphaNumeric = letters.concat(numbers);
console.log(alphaNumeric);
// results in ['a', 'b', 'c', 1, 2, 3]
```

Concatenating three arrays

The following code concatenates three arrays:

```
const num1 = [1, 2, 3];
const num2 = [4, 5, 6];
const num3 = [7, 8, 9];

const numbers = num1.concat(num2, num3);

console.log(numbers);
// results in [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Concatenating values to an array

The following code concatenates three values to an array:

```
const letters = ["a", "b", "c"];

const alphaNumeric = letters.concat(1, [2, 3]);

console.log(alphaNumeric);
// results in ['a', 'b', 'c', 1, 2, 3]
```

Concatenating nested arrays

The following code concatenates nested arrays and demonstrates retention of references:

```
const num1 = [[1]];
const num2 = [2, [3]];

const numbers = num1.concat(num2);

console.log(numbers);
// results in [[1], 2, [3]]

// modify the first element of num1
num1[0].push(4);

console.log(numbers);
// results in [[1, 4], 2, [3]]
```

Concatenating array-like objects with Symbol.isConcatSpreadable

`concat` does not treat all array-like objects as arrays by default — only if `Symbol.isConcatSpreadable` is set to a truthy value (e.g., `true`).

```
const obj1 = { 0: 1, 1: 2, 2: 3, length: 3 };
const obj2 = { 0: 1, 1: 2, 2: 3, length: 3, [Symbol.isConcatSpreadable]: true };
console.log([0].concat(obj1, obj2));
// [ 0, { '0': 1, '1': 2, '2': 3, length: 3 }, 1, 2, 3 ]
```

Using concat() on sparse arrays

If any of the source arrays is sparse, the resulting array will also be sparse:

```
console.log([1, , 3].concat([4, 5])); // [1, empty, 3, 4, 5]
console.log([1, 2].concat([3, , 5])); // [1, 2, 3, empty, 5]
```

Calling concat() on non-array objects

If the `this` value is not an array, it is converted to an object and then treated in the same way as the arguments for `concat()`. In this case the return value is always a plain new array.

```
console.log(Array.prototype.concat.call({}, 1, 2, 3)); // [{}, 1, 2, 3]
console.log(Array.prototype.concat.call(1, 2, 3)); // [ [Number: 1], 2, 3 ]
const arrayLike = {
  [Symbol.isConcatSpreadable]: true,
  length: 2,
  0: 1,
  1: 2,
  2: 99, // ignored by concat() since length is 2
};
console.log(Array.prototype.concat.call(arrayLike, 3, 4)); // [1, 2, 3, 4]
```

See also

- Polyfill of `Array.prototype.concat` in `core-js` with fixes and implementation of modern behavior like `Symbol.isConcatSpreadable` support
- es-shims polyfill of `Array.prototype.concat`
- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.unshift()`
- `Array.prototype.splice()`
- `String.prototype.concat()`
- `Symbol.isConcatSpreadable`

Array.prototype.copyWithin()

The **`copyWithin()`** method of `Array` instances shallow copies part of this array to another location in the same array and returns this array without modifying its length.

Try it

```
const array = ["a", "b", "c", "d", "e"];

// Copy to index 0 the element at index 3
console.log(array.copyWithin(0, 3, 4));
// Expected output: Array ["d", "b", "c", "d", "e"]

// Copy to index 1 all elements from index 3 to the end
console.log(array.copyWithin(1, 3));
// Expected output: Array ["d", "d", "e", "d", "e"]
```

Syntax

```
copyWithin(target, start)
copyWithin(target, start, end)
```

Parameters

`target`

Zero-based index at which to copy the sequence to, converted to an integer. This corresponds to where the element at `start` will be copied to, and all elements between `start` and `end` are copied to succeeding indices.

- Negative index counts back from the end of the array — if `-array.length <= target < 0`, `target + array.length` is used.
- If `target < -array.length`, `0` is used.
- If `target >= array.length`, nothing is copied.
- If `target` is positioned after `start` after normalization, copying only happens until the end of `array.length` (in other words, `copyWithin()` never extends the array).

`start`

Zero-based index at which to start copying elements from, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length`, `0` is used.
- If `start >= array.length`, nothing is copied.

`end` Optional

Zero-based index at which to end copying elements from, converted to an integer. `copyWithin()` copies up to but not including `end`.

- Negative index counts back from the end of the array — if `-array.length <= end < 0`, `end + array.length` is used.
- If `end < -array.length`, `0` is used.
- If `end >= array.length` or `end` is omitted or `undefined`, `array.length` is used, causing all elements until the end to be copied.
- If `end` implies a position before or at the position that `start` implies, nothing is copied.

Return value

The modified array.

Description

The `copyWithin()` method works like C and C++'s `memmove`, and is a high-performance method to shift the data of an `Array`. This especially applies to the `TypedArray` method of the same name. The sequence is copied and pasted as one operation; the pasted sequence will have the copied values even when the copy and paste region overlap.

Because `undefined` becomes `0` when converted to an integer, omitting the `start` parameter has the same effect as passing `0`, which copies the entire array to the target position, equivalent to a right shift where the right boundary is clipped off and the left boundary is duplicated. This behavior may confuse readers of your code, so you should explicitly pass `0` as `start` instead.

```
console.log([1, 2, 3, 4, 5].copyWithin(2));
// [1, 2, 1, 2, 3]; move all elements to the right by 2 positions
```

The `copyWithin()` method is a mutating method. It does not alter the length of `this`, but it will change the content of `this` and create new properties or delete existing properties, if necessary.

The `copyWithin()` method preserves empty slots. If the region to be copied from is sparse, the empty slots' corresponding new indices are deleted and also become empty slots.

The `copyWithin()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Using copyWithin()

```
console.log([1, 2, 3, 4, 5].copyWithin(0, 3));
// [4, 5, 3, 4, 5]

console.log([1, 2, 3, 4, 5].copyWithin(0, 3, 4));
// [4, 2, 3, 4, 5]

console.log([1, 2, 3, 4, 5].copyWithin(-2, -3, -1));
// [1, 2, 3, 3, 4]
```

Using copyWithin() on sparse arrays

`copyWithin()` will propagate empty slots.

```
console.log([1, , 3].copyWithin(2, 1, 2)); // [1, empty, empty]
```

Calling copyWithin() on non-array objects

The `copyWithin()` method reads the `length` property of `this` and then manipulates the integer indices involved.

```
const arrayLike = {
  length: 5,
  3: 1,
};
console.log(Array.prototype.copyWithin.call(arrayLike, 0, 3));
// { '0': 1, '3': 1, length: 5 }
console.log(Array.prototype.copyWithin.call(arrayLike, 3, 1));
// { '0': 1, length: 5 }
// The '3' property is deleted because the copied source is an empty slot
```

See also

- Polyfill of `Array.prototype.copyWithin` in `core-js`
- es-shims polyfill of `Array.prototype.copyWithin`
- Indexed collections guide
- `Array`
- `TypedArray.prototype.copyWithin()`

Array.prototype.entries()

The **`entries()`** method of `Array` instances returns a new *array iterator* object that contains the key/value pairs for each index in the array.

Try it

```
const array = ["a", "b", "c"];

const iterator = array.entries();

console.log(iterator.next().value);
// Expected output: Array [0, "a"]

console.log(iterator.next().value);
// Expected output: Array [1, "b"]
```

Syntax

```
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

When used on sparse arrays, the `entries()` method iterates empty slots as if they have the value `undefined`.

The `entries()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Iterating with index and element

```
const a = ["a", "b", "c"];

for (const [index, element] of a.entries()) {
  console.log(index, element);
}

// 0 'a'
// 1 'b'
// 2 'c'
```

Using a for...of loop

```
const array = ["a", "b", "c"];
const arrayEntries = array.entries();

for (const element of arrayEntries) {
  console.log(element);
}

// [0, 'a']
// [1, 'b']
// [2, 'c']
```

Iterating sparse arrays

`entries()` will visit empty slots as if they are `undefined`.

```
for (const element of [, "a"].entries()) {
  console.log(element);
}
// [0, undefined]
// [1, 'a']
```

Calling entries() on non-array objects

The `entries()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: "d", // ignored by entries() since length is 3
};
for (const entry of Array.prototype.entries.call(arrayLike)) {
  console.log(entry);
}
// [ 0, 'a' ]
// [ 1, 'b' ]
// [ 2, 'c' ]
```

See also

- Polyfill of `Array.prototype.entries` in `core-js`
- es-shims polyfill of `Array.prototype.entries`
- Indexed collections guide
- `Array`
- `Array.prototype.keys()`
- `Array.prototype.values()`
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype.entries()`
- Iteration protocols

Array.prototype.every()

The **`every()`** method of `Array` instances returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`.

Try it

```
const isBelowThreshold = (currentValue) => currentValue < 40;

const array1 = [1, 30, 39, 29, 10, 13];

console.log(array1.every(isBelowThreshold));
// Expected output: true
```

Syntax

```
every(callbackFn)
```

```
every(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `every()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`true` unless `callbackFn` returns a falsy value for an array element, in which case `false` is immediately returned.

Description

The `every()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array, until the `callbackFn` returns a falsy value. If such an element is found, `every()` immediately returns `false` and stops iterating through the array. Otherwise, if `callbackFn` returns a truthy value for all elements, `every()` returns `true`. Read the iterative methods section for more information about how these methods work in general.

`every` acts like the "for all" quantifier in mathematics. In particular, for an empty array, it returns `true`. (It is vacuously true that all elements of the empty set satisfy any given condition.)

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `every()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Testing size of all array elements

The following example tests whether all elements in the array are 10 or bigger.

```
function isBigEnough(element, index, array) {
  return element >= 10;
}
[12, 5, 8, 130, 44].every(isBigEnough); // false
[12, 54, 18, 130, 44].every(isBigEnough); // true
```

Check if one array is a subset of another array

The following example tests if all the elements of an array are present in another array.

```
const isSubset = (array1, array2) =>
  array2.every((element) => array1.includes(element));

console.log(isSubset([1, 2, 3, 4, 5, 6, 7], [5, 7, 6])); // true
console.log(isSubset([1, 2, 3, 4, 5, 6, 7], [5, 8, 7])); // false
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array. The following example first uses `filter()` to extract the positive values and then uses `every()` to check whether the array is strictly increasing.

```
const numbers = [-2, 4, -8, 16, -32];
const isIncreasing = numbers
  .filter((num) => num > 0)
  .every((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx === 0) return true;
    return num > arr[idx - 1];
  });
console.log(isIncreasing); // true
```

Using every() on sparse arrays

`every()` will not run its predicate on empty slots.

```
console.log([1, , 3].every((x) => x !== undefined)); // true
console.log([2, , 2].every((x) => x === 2)); // true
```

Calling every() on non-array objects

The `every()` method reads the `length` property of `this` and then accesses each property with a nonnegative integer key less than `length` until they all have been accessed or `callbackFn` returns `false`.

```
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: 345, // ignored by every() since length is 3
};
console.log(
  Array.prototype.every.call(arrayLike, (x) => typeof x === "string"),
); // true
```

See also

- Polyfill of `Array.prototype.every` in `core-js`
- es-shims polyfill of `Array.prototype.every`
- Indexed collections guide
- `Array`
- `Array.prototype.forEach()`
- `Array.prototype.some()`
- `Array.prototype.find()`
- `TypedArray.prototype.every()`

Array.prototype.fill()

The **`fill()`** method of `Array` instances changes all elements within a range of indices in an array to a static value. It returns the modified array.

Try it

```
const array = [1, 2, 3, 4];

// Fill with 0 from position 2 until position 4
console.log(array.fill(0, 2, 4));
// Expected output: Array [1, 2, 0, 0]

// Fill with 5 from position 1
console.log(array.fill(5, 1));
// Expected output: Array [1, 5, 5, 5]

console.log(array.fill(6));
// Expected output: Array [6, 6, 6, 6]
```

Syntax

```
fill(value)
fill(value, start)
fill(value, start, end)
```

Parameters

`value`

Value to fill the array with. Note all elements in the array will be this exact value: if `value` is an object, each slot in the array will reference that object.

`start` Optional

Zero-based index at which to start filling, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length` or `start` is omitted, `0` is used.
- If `start >= array.length`, no index is filled.

`end` Optional

Zero-based index at which to end filling, converted to an integer. `fill()` fills up to but not including `end`.

- Negative index counts back from the end of the array — if `-array.length <= end < 0`, `end + array.length` is used.
- If `end < -array.length`, `0` is used.
- If `end >= array.length` or `end` is omitted or `undefined`, `array.length` is used, causing all indices until the end to be filled.
- If `end` implies a position before or at the position that `start` implies, nothing is filled.

Return value

The modified array, filled with `value`.

Description

The `fill()` method is a mutating method. It does not alter the length of `this`, but it will change the content of `this`.

The `fill()` method fills empty slots in sparse arrays with `value` as well.

The `fill()` method is generic. It only expects the `this` value to have a `length` property. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

> **Note:** Using `Array.prototype.fill()` on an empty array (`length = 0`) would not modify it as the array has nothing to be modified. To use `Array.prototype.fill()` when declaring an array, make sure the array has non-zero `length`. See example.

Examples

Using fill()

```
console.log([1, 2, 3].fill(4)); // [4, 4, 4]
console.log([1, 2, 3].fill(4, 1)); // [1, 4, 4]
console.log([1, 2, 3].fill(4, 1, 2)); // [1, 4, 3]
console.log([1, 2, 3].fill(4, 1, 1)); // [1, 2, 3]
console.log([1, 2, 3].fill(4, 3, 3)); // [1, 2, 3]
console.log([1, 2, 3].fill(4, -3, -2)); // [4, 2, 3]
console.log([1, 2, 3].fill(4, NaN, NaN)); // [1, 2, 3]
console.log([1, 2, 3].fill(4, 3, 5)); // [1, 2, 3]
console.log(Array(3).fill(4)); // [4, 4, 4]

// A single object, referenced by each slot of the array:
const arr = Array(3).fill({}); // [{}, {}, {}]
arr[0].hi = "hi"; // [{ hi: "hi" }, { hi: "hi" }, { hi: "hi" }]
```

Using fill() to create a matrix of all 1

This example shows how to create a matrix of all 1, like the `ones()` function of Octave or MATLAB.

```
const arr = new Array(3);
for (let i = 0; i < arr.length; i++) {
  arr[i] = new Array(4).fill(1); // Creating an array of size 4 and filled of 1
}
arr[0][0] = 10;
console.log(arr[0][0]); // 10
console.log(arr[1][0]); // 1
console.log(arr[2][0]); // 1
```

Using fill() to populate an empty array

This example shows how to populate an array, setting all elements to a specific value. The `end` parameter does not have to be specified.

```
const tempGirls = Array(5).fill("girl", 0);
```

Note that the array was initially a sparse array with no assigned indices. `fill()` is still able to fill this array.

Calling fill() on non-array objects

The `fill()` method reads the `length` property of `this` and sets the value of each integer-keyed property from `start` to `end`.

```
const arrayLike = { length: 2 };
console.log(Array.prototype.fill.call(arrayLike, 1));
// { '0': 1, '1': 1, length: 2 }
```

See also

- Polyfill of `Array.prototype.fill` in `core-js`
- Indexed collections guide
- `Array`
- `TypedArray.prototype.fill()`

Array.prototype.filter()

The **`filter()`** method of `Array` instances creates a shallow copy of a portion of a given array, filtered down to just the elements from the given array that pass the test implemented by the provided function.

Try it

```
const words = ["spray", "elite", "exuberant", "destruction", "present"];

const result = words.filter((word) => word.length > 6);

console.log(result);
// Expected output: Array ["exuberant", "destruction", "present"]
```

Syntax

```
filter(callbackFn)
filter(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to keep the element in the resulting array, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `filter()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A shallow copy of the given array containing just the elements that pass the test. If no elements pass the test, an empty array is returned.

Description

The `filter()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array, and constructs a new array of all the values for which `callbackFn` returns a truthy value. Array elements which do not pass the `callbackFn` test are not included in the new array. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `filter()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Filtering out all small values

The following example uses `filter()` to create a filtered array that has all elements with values less than 10 removed.

```
function isBigEnough(value) {
  return value >= 10;
}

const filtered = [12, 5, 8, 130, 44].filter(isBigEnough);
// filtered is [12, 130, 44]
```

Find all prime numbers in an array

The following example returns all prime numbers in the array:

```
const array = [-3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13];

function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log(array.filter(isPrime)); // [2, 3, 5, 7, 11, 13]
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Filtering invalid objects from an array of objects

The following example uses `filter()` to create a filtered array of all objects with non-zero, numeric `id`.

```
const arr = [
  { id: 15 },
  { id: -1 },
  { id: 0 },
  { id: 3 },
  { id: 12.2 },
  {},
  { id: null },
  { id: NaN },
  { id: "undefined" },
];

let invalidEntries = 0;

function filterByID(item) {
  if (Number.isFinite(item.id) && item.id !== 0) {
    return true;
  }
  invalidEntries++;
  return false;
}

const arrByID = arr.filter(filterByID);

console.log("Filtered Array\n", arrByID);
// Filtered Array
// [{ id: 15 }, { id: -1 }, { id: 3 }, { id: 12.2 }]

console.log("Number of Invalid Entries =", invalidEntries);
// Number of Invalid Entries = 5
```

Searching in array

The following example uses `filter()` to filter array content based on search criteria.

```
const fruits = ["apple", "banana", "grapes", "mango", "orange"];

/**
 * Filter array items based on search criteria (query)
 */
function filterItems(arr, query) {
  return arr.filter((el) => el.toLowerCase().includes(query.toLowerCase()));
}

console.log(filterItems(fruits, "ap")); // ['apple', 'grapes']
console.log(filterItems(fruits, "an")); // ['banana', 'mango', 'orange']
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `map()` to extract the numerical ID from each name and then uses `filter()` to select the ones that are greater than its neighbors.

```
const names = ["JC63", "Bob132", "Ursula89", "Ben96"];
const greatIDs = names
  .map((name) => parseInt(name.match(/\d+/)[0], 10))
  .filter((id, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && id <= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && id <= arr[idx + 1]) return false;
    return true;
  });
console.log(greatIDs); // [132, 96]
```

The `array` argument is *not* the array that is being built — there is no way to access the array being built from the callback function.

Using filter() on sparse arrays

`filter()` will skip empty slots.

```
console.log([1, , undefined].filter((x) => x === undefined)); // [undefined]
console.log([1, , undefined].filter((x) => x !== 2)); // [1, undefined]
```

Calling filter() on non-array objects

The `filter()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
```

```
  2: "c",
  3: "a", // ignored by filter() since length is 3
};
console.log(Array.prototype.filter.call(arrayLike, (x) => x <= "b"));
// [ 'a', 'b' ]
```

See also

- Polyfill of `Array.prototype.filter` in `core-js`
- es-shims polyfill of `Array.prototype.filter`
- Indexed collections guide
- `Array`
- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `Array.prototype.map()`
- `Array.prototype.some()`
- `Array.prototype.reduce()`
- `TypedArray.prototype.filter()`

Array.prototype.find()

The **`find()`** method of `Array` instances returns the first element in the provided array that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

- If you need the **index** of the found element in the array, use `findIndex()`.
- If you need to find the **index of a value**, use `indexOf()`. (It's similar to `findIndex()`, but checks each element for equality with the value instead of using a testing function.)
- If you need to find if a value **exists** in an array, use `includes()`. Again, it checks each element for equality with the value instead of using a testing function.
- If you need to find if any element satisfies the provided testing function, use `some()`.
- If you need to find all elements that satisfy the provided testing function, use `filter()`.

Try it

```
const array = [5, 12, 8, 130, 44];

const found = array.find((element) => element > 10);

console.log(found);
// Expected output: 12
```

Syntax

```
find(callbackFn)
find(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `find()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The first element in the array that satisfies the provided testing function. Otherwise, `undefined` is returned.

Description

The `find()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in ascending-index order, until `callbackFn` returns a truthy value. `find()` then returns that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `find()` returns `undefined`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `find()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find an object in an array by one of its properties

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "cherries", quantity: 5 },
];

function isCherries(fruit) {
  return fruit.name === "cherries";
}

console.log(inventory.find(isCherries));
// { name: 'cherries', quantity: 5 }
```

Using arrow function and destructuring

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "cherries", quantity: 5 },
];

const result = inventory.find(({ name }) => name === "cherries");

console.log(result); // { name: 'cherries', quantity: 5 }
```

Find the first prime number in an array

The following example returns the first element in the array that is a prime number, or `undefined` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log([4, 6, 8, 12].find(isPrime)); // undefined, not found
console.log([4, 5, 8, 12].find(isPrime)); // 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `find()` to find the first element that is less than its neighbors.

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const firstTrough = numbers
  .filter((num) => num > 0)
  .find((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(firstTrough); // 1
```

Using find() on sparse arrays

Empty slots in sparse arrays *are* visited, and are treated the same as `undefined`.

```
// Declare array with no elements at indexes 2, 3, and 4
const array = [0, 1, , , , 5, 6];

// Shows all indexes, not just those with assigned values
array.find((value, index) => {
  console.log("Visited index", index, "with value", value);
```

```
  return false;
});
// Visited index 0 with value 0
// Visited index 1 with value 1
// Visited index 2 with value undefined
// Visited index 3 with value undefined
// Visited index 4 with value undefined
// Visited index 5 with value 5
// Visited index 6 with value 6

// Shows all indexes, including deleted
array.find((value, index) => {
  // Delete element 5 on first iteration
  if (index === 0) {
    console.log("Deleting array[5] with value", array[5]);
    delete array[5];
  }
  // Element 5 is still visited even though deleted
  console.log("Visited index", index, "with value", value);
  return false;
});
// Deleting array[5] with value 5
// Visited index 0 with value 0
// Visited index 1 with value 1
// Visited index 2 with value undefined
// Visited index 3 with value undefined
// Visited index 4 with value undefined
// Visited index 5 with value undefined
// Visited index 6 with value 6
```

Calling find() on non-array objects

The `find()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  "-1": 0.1, // ignored by find() since -1 < 0
  0: 2,
  1: 7.3,
  2: 4,
};
console.log(Array.prototype.find.call(arrayLike, (x) => !Number.isInteger(x)));
// 7.3
```

See also

- Polyfill of `Array.prototype.find` in `core-js`
- es-shims polyfill of `Array.prototype.find`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.findLast()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.includes()`
- `Array.prototype.filter()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `TypedArray.prototype.find()`

Array.prototype.findIndex()

The **`findIndex()`** method of `Array` instances returns the index of the first element in an array that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned.

See also the `find()` method, which returns the first element that satisfies the testing function (rather than its index).

Try it

```
const array = [5, 12, 8, 130, 44];

const isLargeNumber = (element) => element > 13;

console.log(array.findIndex(isLargeNumber));
// Expected output: 3
```

Syntax

```
findIndex(callbackFn)
findIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `findIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the first element in the array that passes the test. Otherwise, `-1`.

Description

The `findIndex()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in ascending-index order, until `callbackFn` returns a truthy value. `findIndex()` then returns the index of that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `findIndex()` returns `-1`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `findIndex()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find the index of the first prime number in an array

The following example returns the index of the first element in the array that is a prime number, or `-1` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log([4, 6, 8, 9, 12].findIndex(isPrime)); // -1, not found
console.log([4, 6, 7, 9, 12].findIndex(isPrime)); // 2 (array[2] is 7)
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `findIndex()` to find the first element that is less than its neighbors.

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const firstTrough = numbers
  .filter((num) => num > 0)
  .findIndex((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(firstTrough); // 1
```

Using findIndex() on sparse arrays

You can search for `undefined` in a sparse array and get the index of an empty slot.

```
console.log([1, , 3].findIndex((x) => x === undefined)); // 1
```

Calling findIndex() on non-array objects

The `findIndex()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  "-1": 0.1, // ignored by findIndex() since -1 < 0
  0: 2,
  1: 7.3,
  2: 4,
};
console.log(
  Array.prototype.findIndex.call(arrayLike, (x) => !Number.isInteger(x)),
); // 1
```

See also

- Polyfill of `Array.prototype.findIndex` in `core-js`
- es-shims polyfill of `Array.prototype.findIndex`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.findLast()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.findIndex()`

Array.prototype.findLast()

The **`findLast()`** method of `Array` instances iterates the array in reverse order and returns the value of the first element that satisfies the provided testing function. If no elements satisfy the testing function, `undefined` is returned.

If you need to find:

- the *first* element that matches, use `find()`.
- the *index* of the last matching element in the array, use `findLastIndex()`.
- the *index of a value*, use `indexOf()`. (It's similar to `findIndex()`, but checks each element for equality with the value instead of using a testing function.)
- whether a value *exists* in an array, use `includes()`. Again, it checks each element for equality with the value instead of using a testing function.
- if any element satisfies the provided testing function, use `some()`.

Try it

```
const array = [5, 12, 50, 130, 44];

const found = array.findLast((element) => element > 45);

console.log(found);
// Expected output: 130
```

Syntax

```
findLast(callbackFn)
findLast(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `findLast()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The last (highest-index) element in the array that satisfies the provided testing function; `undefined` if no matching element is found.

Description

The `findLast()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in descending-index order, until `callbackFn` returns a truthy value. `findLast()` then returns that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `findLast()` returns `undefined`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `findLast()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find last object in an array matching on element properties

This example shows how you might create a test based on the properties of array elements.

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "fish", quantity: 1 },
  { name: "cherries", quantity: 5 },
];

// return true inventory stock is low
function isNotEnough(item) {
  return item.quantity < 2;
}

console.log(inventory.findLast(isNotEnough));
// { name: "fish", quantity: 1 }
```

Using arrow function and destructuring

The previous example might be written using an arrow function and object destructuring:

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "fish", quantity: 1 },
  { name: "cherries", quantity: 5 },
];

const result = inventory.findLast(({ quantity }) => quantity < 2);

console.log(result);
// { name: "fish", quantity: 1 }
```

Find the last prime number in an array

The following example returns the last element in the array that is a prime number, or `undefined` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log([4, 6, 8, 12].findLast(isPrime)); // undefined, not found
console.log([4, 5, 7, 8, 9, 11, 12].findLast(isPrime)); // 11
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Find the most recent completed task

This example demonstrates a practical use case for `findLast()` when working with time-ordered data. It finds the most recent completed task from a list already sorted by `timestamp`.

```
const tasks = [
  { name: "Setup project", completed: true, timestamp: 1609459200000 },
```

```
  { name: "Write tests", completed: false, timestamp: 1609545600000 },
  { name: "Fix bug #42", completed: true, timestamp: 1609632000000 },
  { name: "Deploy to staging", completed: true, timestamp: 1609718400000 },
  { name: "Review PR", completed: false, timestamp: 1609804800000 },
];

const lastCompletedTask = tasks.findLast((task) => task.completed);

console.log(lastCompletedTask.name); // Deploy to staging
```

This is more efficient than using `find()` with a reversed array, because it avoids creating a new array.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `findLast()` to find the last element that is less than its neighbors.

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const lastTrough = numbers
  .filter((num) => num > 0)
  .findLast((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(lastTrough); // 2
```

Using findLast() on sparse arrays

Empty slots in sparse arrays *are* visited, and are treated the same as `undefined`.

```
// Declare array with no elements at indexes 2, 3, and 4
const array = [0, 1, , , , 5, 6];

// Shows all indexes, not just those with assigned values
array.findLast((value, index) => {
  console.log(`Visited index ${index} with value ${value}`);
  return false;
});
// Visited index 6 with value 6
// Visited index 5 with value 5
// Visited index 4 with value undefined
// Visited index 3 with value undefined
// Visited index 2 with value undefined
// Visited index 1 with value 1
// Visited index 0 with value 0

// Shows all indexes, including deleted
array.findLast((value, index) => {
  // Delete element 5 on first iteration
  if (index === 6) {
    console.log(`Deleting array[5] with value ${array[5]}`);
    delete array[5];
  }
  // Element 5 is still visited even though deleted
  console.log(`Visited index ${index} with value ${value}`);
  return false;
});
// Deleting array[5] with value 5
// Visited index 6 with value 6
// Visited index 5 with value undefined
// Visited index 4 with value undefined
// Visited index 3 with value undefined
// Visited index 2 with value undefined
// Visited index 1 with value 1
// Visited index 0 with value 0
```

Calling findLast() on non-array objects

The `findLast()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 7.3,
  2: 4,
  3: 3, // ignored by findLast() since length is 3
};
console.log(
  Array.prototype.findLast.call(arrayLike, (x) => Number.isInteger(x)),
); // 4
```

See also

- Polyfill of `Array.prototype.findLast` in `core-js`
- es-shims polyfill of `Array.prototype.findLast`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.includes()`
- `Array.prototype.filter()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `TypedArray.prototype.findLast()`

Array.prototype.findLastIndex()

The **`findLastIndex()`** method of `Array` instances iterates the array in reverse order and returns the index of the first element that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned.

See also the `findLast()` method, which returns the value of last element that satisfies the testing function (rather than its index).

Try it

```
const array = [5, 12, 50, 130, 44];

const isLargeNumber = (element) => element > 45;

console.log(array.findLastIndex(isLargeNumber));
// Expected output: 3
// Index of element with value: 130
```

Syntax

```
findLastIndex(callbackFn)
findLastIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `findLastIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the last (highest-index) element in the array that passes the test. Otherwise `-1` if no matching element is found.

Description

The `findLastIndex()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in descending-index order, until `callbackFn` returns a truthy value. `findLastIndex()` then returns the index of that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `findLastIndex()` returns `-1`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `findLastIndex()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find the index of the last prime number in an array

The following example returns the index of the last element in the array that is a prime number, or `-1` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
```

```
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log([4, 6, 8, 12].findLastIndex(isPrime)); // -1, not found
console.log([4, 5, 7, 8, 9, 11, 12].findLastIndex(isPrime)); // 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `findLastIndex()` to find the last element that is less than its neighbors.

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const lastTrough = numbers
  .filter((num) => num > 0)
  .findLastIndex((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(lastTrough); // 6
```

Using findLastIndex() on sparse arrays

You can search for `undefined` in a sparse array and get the index of an empty slot.

```
console.log([1, , 3].findLastIndex((x) => x === undefined)); // 1
```

Calling findLastIndex() on non-array objects

The `findLastIndex()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 7.3,
  2: 4,
  3: 3, // ignored by findLastIndex() since length is 3
};
console.log(
  Array.prototype.findLastIndex.call(arrayLike, (x) => Number.isInteger(x)),
); // 2
```

See also

- Polyfill of `Array.prototype.findLastIndex` in `core-js`
- es-shims polyfill of `Array.prototype.findLastIndex`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `Array.prototype.findLast()`
- `Array.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.findLastIndex()`

Array.prototype.flat()

The **`flat()`** method of `Array` instances creates a new array with all sub-array elements concatenated into it recursively up to the specified depth.

Try it

```
const arr1 = [0, 1, 2, [3, 4]];

console.log(arr1.flat());
// expected output: Array [0, 1, 2, 3, 4]
```

```
const arr2 = [0, 1, [2, [3, [4, 5]]]];

console.log(arr2.flat());
// expected output: Array [0, 1, 2, Array [3, Array [4, 5]]]

console.log(arr2.flat(2));
// expected output: Array [0, 1, 2, 3, Array [4, 5]]

console.log(arr2.flat(Infinity));
// expected output: Array [0, 1, 2, 3, 4, 5]
```

Syntax

```
flat()
flat(depth)
```

Parameters

`depth` Optional

The depth level specifying how deep a nested array structure should be flattened. Defaults to 1.

Return value

A new array with the sub-array elements concatenated into it.

Description

The `flat()` method is a copying method. It does not alter `this` but instead returns a shallow copy that contains the same elements as the ones from the original array.

The `flat()` method removes empty slots if the array being flattened is sparse. For example, if `depth` is 1, both empty slots in the root array and in the first level of nested arrays are ignored, but empty slots in further nested arrays are preserved with the arrays themselves.

The `flat()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. However, its elements must be arrays if they are to be flattened.

Examples

Flattening nested arrays

```
const arr1 = [1, 2, [3, 4]];
arr1.flat();
// [1, 2, 3, 4]

const arr2 = [1, 2, [3, 4, [5, 6]]];
arr2.flat();
// [1, 2, 3, 4, [5, 6]]

const arr3 = [1, 2, [3, 4, [5, 6]]];
arr3.flat(2);
// [1, 2, 3, 4, 5, 6]

const arr4 = [1, 2, [3, 4, [5, 6, [7, 8, [9, 10]]]]];
arr4.flat(Infinity);
// [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Using flat() on sparse arrays

The `flat()` method removes empty slots in arrays:

```
const arr5 = [1, 2, , 4, 5];
console.log(arr5.flat()); // [1, 2, 4, 5]

const array = [1, , 3, ["a", , "c"]];
console.log(array.flat()); // [ 1, 3, "a", "c" ]

const array2 = [1, , 3, undefined, ["a", , ["d", , "e"]], null];
console.log(array2.flat()); // [ 1, 3, undefined, "a", ["d", empty, "e"], null ]
console.log(array2.flat(2)); // [ 1, 3, undefined, "a", "d", "e", null ]
```

Calling flat() on non-array objects

The `flat()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`. If the element is not an array, it's directly appended to the result. If the element is an array, it's flattened according to the `depth` parameter.

```
const arrayLike = {
  length: 3,
  0: [1, 2],
  // Array-like objects aren't flattened
  1: { length: 2, 0: 3, 1: 4 },
  2: 5,
  3: 3, // ignored by flat() since length is 3
```

```
};
console.log(Array.prototype.flat.call(arrayLike));
// [ 1, 2, { '0': 3, '1': 4, length: 2 }, 5 ]
```

See also

- Polyfill of `Array.prototype.flat` in `core-js`
- es-shims polyfill of `Array.prototype.flat`
- Indexed collections guide
- `Array`
- `Array.prototype.concat()`
- `Array.prototype.flatMap()`
- `Array.prototype.map()`
- `Array.prototype.reduce()`

Array.prototype.flatMap()

The **`flatMap()`** method of `Array` instances returns a new array formed by applying a given callback function to each element of the array, and then flattening the result by one level. It is identical to a `map()` followed by a `flat()` of depth 1 (`arr.map(...args).flat()`), but slightly more efficient than calling those two methods separately.

Try it

```
const arr = [1, 2, 1];

const result = arr.flatMap((num) => (num === 2 ? [2, 2] : 1));

console.log(result);
// Expected output: Array [1, 2, 2, 1]
```

Syntax

```
flatMap(callbackFn)
flatMap(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return an array containing new elements of the new array, or a single non-array value to be added to the new array. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `flatMap()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A new array with each element being the result of the callback function and flattened by a depth of 1.

Description

The `flatMap()` method is an iterative method. See `Array.prototype.map()` for a detailed description of the callback function. The `flatMap()` method is identical to `map(callbackFn, thisArg)` followed by `flat(1)` — for each element, it produces an array of new elements, and concatenates the resulting arrays together to form a new array. Read the iterative methods section for more information about how these methods work in general.

The `flatMap()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. However, the value returned from `callbackFn` must be an array if it is to be flattened.

Alternative

Pre-allocate and explicitly iterate

```
const arr = [1, 2, 3, 4];

arr.flatMap((x) => [x, x * 2]);
// is equivalent to
const n = arr.length;
const acc = new Array(n * 2);
for (let i = 0; i < n; i++) {
  const x = arr[i];
  acc[i * 2] = x;
```

```
  acc[i * 2 + 1] = x * 2;
}
// [1, 2, 2, 4, 3, 6, 4, 8]
```

Note that in this particular case the `flatMap` approach is slower than the for-loop approach — due to the creation of temporary arrays that must be garbage collected, as well as the return array not needing to be frequently resized. However, `flatMap` may still be the correct solution in cases where its flexibility and readability are desired.

Examples

map() and flatMap()

```
const arr = [1, 2, 3, 4];

arr.map((x) => [x * 2]);
// [[2], [4], [6], [8]]

arr.flatMap((x) => [x * 2]);
// [2, 4, 6, 8]

// only one level is flattened
arr.flatMap((x) => [[x * 2]]);
// [[2], [4], [6], [8]]
```

While the above could have been achieved by using map itself, here is an example that better showcases the use of `flatMap()`.

Let's generate a list of words from a list of sentences.

```
const arr = ["it's Sunny in", "", "California"];

arr.map((x) => x.split(" "));
// [["it's","Sunny","in"],[""],["California"]]

arr.flatMap((x) => x.split(" "));
// ["it's","Sunny","in", "", "California"]
```

Notice, the output list length can be different from the input list length.

For adding and removing items during a map()

`flatMap` can be used as a way to add and remove items (modify the number of items) during a `map`. In other words, it allows you to map *many items to many items* (by handling each input item separately), rather than always *one-to-one*. In this sense, it works like the opposite of filter. Return a 1-element array to keep the item, a multiple-element array to add items, or a 0-element array to remove the item.

```
// Let's say we want to remove all the negative numbers
// and split the odd numbers into an even number and a 1
const a = [5, 4, -3, 20, 17, -33, -4, 18];
//         |\  \  x   |  | \   x   x   |
//        [4,1, 4,   20, 16, 1,       18]

const result = a.flatMap((n) => {
  if (n < 0) {
    return [];
  }
  return n % 2 === 0 ? [n] : [n - 1, 1];
});
console.log(result); // [4, 1, 4, 20, 16, 1, 18]
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract operational stations and then uses `flatMap()` to create a new array where each element contains a station and its next station. On the last station, it returns an empty array to exclude it from the final array.

```
const stations = ["New Haven", "West Haven", "Milford (closed)", "Stratford"];
const line = stations
  .filter((name) => !name.endsWith("(closed)"))
  .flatMap((name, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx === arr.length - 1) return []; // last station has no next station
    return [`${name} - ${arr[idx + 1]}`];
  });
console.log(line); // ['New Haven - West Haven', 'West Haven - Stratford']
```

The `array` argument is *not* the array that is being built — there is no way to access the array being built from the callback function.

Using flatMap() on sparse arrays

The `callbackFn` won't be called for empty slots in the source array because `map()` doesn't, while `flat()` ignores empty slots in the returned arrays.

```
console.log([1, 2, , 4, 5].flatMap((x) => [x, x * 2])); // [1, 2, 2, 4, 4, 8, 5, 10]
console.log([1, 2, 3, 4].flatMap((x) => [, x * 2])); // [2, 4, 6, 8]
```

Calling flatMap() on non-array objects

The `flatMap()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`. If the return value of the callback function is not an array, it is always directly appended to the result array.

```
const arrayLike = {
  length: 3,
  0: 1,
  1: 2,
  2: 3,
  3: 4, // ignored by flatMap() since length is 3
};
console.log(Array.prototype.flatMap.call(arrayLike, (x) => [x, x * 2]));
// [1, 2, 2, 4, 3, 6]

// Array-like objects returned from the callback won't be flattened
console.log(
  Array.prototype.flatMap.call(arrayLike, (x) => ({
    length: 1,
    0: x,
  })),
);
// [ { '0': 1, length: 1 }, { '0': 2, length: 1 }, { '0': 3, length: 1 } ]
```

See also

- Polyfill of `Array.prototype.flatMap` in `core-js`
- es-shims polyfill of `Array.prototype.flatMap`
- Indexed collections guide
- `Array`
- `Array.prototype.concat()`
- `Array.prototype.flat()`
- `Array.prototype.map()`
- `Array.prototype.reduce()`

Array.prototype.forEach()

The **`forEach()`** method of `Array` instances executes a provided function once for each array element.

Try it

```
const array = ["a", "b", "c"];

array.forEach((element) => console.log(element));

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value is discarded. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `forEach()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

None (`undefined`).

Description

The `forEach()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in ascending-index order. Unlike `map()`, `forEach()` always returns `undefined` and is not chainable. The typical use case is to execute side effects at the end of a chain. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `forEach()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

There is no way to stop or break a `forEach()` loop other than by throwing an exception. If you need such behavior, the `forEach()` method is the wrong tool.

Early termination may be accomplished with looping statements like `for`, `for...of`, and `for...in`. Array methods like `every()`, `some()`, `find()`, and `findIndex()` also stops iteration immediately when further iteration is not necessary.

`forEach()` expects a synchronous function — it does not wait for promises. Make sure you are aware of the implications while using promises (or async functions) as `forEach` callbacks.

```
const ratings = [5, 4, 5];
let sum = 0;

const sumFunction = async (a, b) => a + b;

ratings.forEach(async (rating) => {
  sum = await sumFunction(sum, rating);
});

console.log(sum);
// Naively expected output: 14
// Actual output: 0
```

To run a series of asynchronous operations sequentially or concurrently, see promise composition.

Examples

Converting a for loop to forEach

```
const items = ["item1", "item2", "item3"];
const copyItems = [];

// before
for (let i = 0; i < items.length; i++) {
  copyItems.push(items[i]);
}

// after
items.forEach((item) => {
  copyItems.push(item);
});
```

Printing the contents of an array

> **Note:** In order to display the content of an array in the console, you can use `console.table()`, which prints a formatted version of the array.
>
> The following example illustrates an alternative approach, using `forEach()`.

The following code logs a line for each element in an array:

```
const logArrayElements = (element, index /*, array */) => {
  console.log(`a[${index}] = ${element}`);
};

// Notice that index 2 is skipped, since there is no item at
// that position in the array.
[2, 5, , 9].forEach(logArrayElements);
// Logs:
// a[0] = 2
// a[1] = 5
// a[3] = 9
```

Using thisArg

The following (contrived) example updates an object's properties from each entry in the array:

```
class Counter {
  constructor() {
    this.sum = 0;
    this.count = 0;
  }
  add(array) {
    // Only function expressions have their own this bindings.
    array.forEach(function countEntry(entry) {
```

```
      this.sum += entry;
      ++this.count;
    }, this);
  }
}

const obj = new Counter();
obj.add([2, 5, 9]);
console.log(obj.count); // 3
console.log(obj.sum); // 16
```

Since the `thisArg` parameter (`this`) is provided to `forEach()`, it is passed to `callback` each time it's invoked. The callback uses it as its `this` value.

> **Note:** If passing the callback function used an arrow function expression, the `thisArg` parameter could be omitted, since all arrow functions lexically bind the `this` value.

An object copy function

The following code creates a copy of a given object.

There are different ways to create a copy of an object. The following is just one way and is presented to explain how `Array.prototype.forEach()` works by using `Object.*` utility functions.

```
const copy = (obj) => {
  const copy = Object.create(Object.getPrototypeOf(obj));
  const propNames = Object.getOwnPropertyNames(obj);
  propNames.forEach((name) => {
    const desc = Object.getOwnPropertyDescriptor(obj, name);
    Object.defineProperty(copy, name, desc);
  });
  return copy;
};

const obj1 = { a: 1, b: 2 };
const obj2 = copy(obj1); // obj2 looks like obj1 now
```

Flatten an array

The following example is only here for learning purpose. If you want to flatten an array using built-in methods, you can use `Array.prototype.flat()`.

```
const flatten = (arr) => {
  const result = [];
  arr.forEach((item) => {
    if (Array.isArray(item)) {
      result.push(...flatten(item));
    } else {
      result.push(item);
    }
  });
  return result;
};

// Usage
const nested = [1, 2, 3, [4, 5, [6, 7], 8, 9]];
console.log(flatten(nested)); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `forEach()` to log its neighbors.

```
const numbers = [3, -1, 1, 4, 1, 5];
numbers
  .filter((num) => num > 0)
  .forEach((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    console.log(arr[idx - 1], num, arr[idx + 1]);
  });
// undefined 3 1
// 3 1 4
// 1 4 1
// 4 1 5
// 1 5 undefined
```

Using forEach() on sparse arrays

```
const arraySparse = [1, 3, /* empty */, 7];
let numCallbackRuns = 0;

arraySparse.forEach((element) => {
  console.log({ element });
```

```
  numCallbackRuns++;
});

console.log({ numCallbackRuns });

// { element: 1 }
// { element: 3 }
// { element: 7 }
// { numCallbackRuns: 3 }
```

The callback function is not invoked for the missing value at index 2.

Calling forEach() on non-array objects

The `forEach()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by forEach() since length is 3
};
Array.prototype.forEach.call(arrayLike, (x) => console.log(x));
// 2
// 3
// 4
```

See also

- Polyfill of `Array.prototype.forEach` in `core-js`
- es-shims polyfill of `Array.prototype.forEach`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.map()`
- `Array.prototype.filter()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `TypedArray.prototype.forEach()`
- `Map.prototype.forEach()`
- `Set.prototype.forEach()`

Array.from()

The **`Array.from()`** static method creates a new, shallow-copied `Array` instance from an iterable or array-like object.

Try it

```
console.log(Array.from("foo"));
// Expected output: Array ["f", "o", "o"]

console.log(Array.from([1, 2, 3], (x) => x + x));
// Expected output: Array [2, 4, 6]
```

Syntax

```
Array.from(items)
Array.from(items, mapFn)
Array.from(items, mapFn, thisArg)
```

Parameters

`items`

An iterable or array-like object to convert to an array.

`mapFn` Optional

A function to call on every element of the array. If provided, every value to be added to the array is first passed through this function, and `mapFn`'s return value is added to the array instead. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`thisArg` Optional

Value to use as `this` when executing `mapFn`.

Return value

A new `Array` instance.

Description

`Array.from()` lets you create `Array`s from:

- iterable objects (objects such as `Map` and `Set`); or, if the object is not iterable,
- array-like objects (objects with a `length` property and indexed elements).

To convert an ordinary object that's not iterable or array-like to an array (by enumerating its property keys, values, or both), use `Object.keys()`, `Object.values()`, or `Object.entries()`. To convert an async iterable to an array, use `Array.fromAsync()`.

`Array.from()` never creates a sparse array. If the `items` object is missing some index properties, they become `undefined` in the new array.

`Array.from()` has an optional parameter `mapFn`, which allows you to execute a function on each element of the array being created, similar to `map()`. More clearly, `Array.from(obj, mapFn, thisArg)` has the same result as `Array.from(obj).map(mapFn, thisArg)`, except that it does not create an intermediate array, and `mapFn` only receives two arguments (`element`, `index`) without the whole array, because the array is still under construction.

> **Note:** This behavior is more important for typed arrays, since the intermediate array would necessarily have values truncated to fit into the appropriate type. `Array.from()` is implemented to have the same signature as `TypedArray.from()`.

The `Array.from()` method is a generic factory method. For example, if a subclass of `Array` inherits the `from()` method, the inherited `from()` method will return new instances of the subclass instead of `Array` instances. In fact, the `this` value can be any constructor function that accepts a single argument representing the length of the new array. When an iterable is passed as `items`, the constructor is called with no arguments; when an array-like object is passed, the constructor is called with the normalized length of the array-like object. The final `length` will be set again when iteration finishes. If the `this` value is not a constructor function, the plain `Array` constructor is used instead.

Examples

Array from a String

```
Array.from("foo");
// [ "f", "o", "o" ]
```

Array from a Set

```
const set = new Set(["foo", "bar", "baz", "foo"]);
Array.from(set);
// [ "foo", "bar", "baz" ]
```

Array from a Map

```
const map = new Map([
  [1, 2],
  [2, 4],
  [4, 8],
]);
Array.from(map);
// [[1, 2], [2, 4], [4, 8]]

const mapper = new Map([
  ["1", "a"],
  ["2", "b"],
]);
Array.from(mapper.values());
// ['a', 'b'];

Array.from(mapper.keys());
// ['1', '2'];
```

Array from a NodeList

```
// Create an array based on a property of DOM Elements
const images = document.querySelectorAll("img");
const sources = Array.from(images, (image) => image.src);
const insecureSources = sources.filter((link) => link.startsWith("http://"));
```

Array from an Array-like object (arguments)

```
function f() {
  return Array.from(arguments);
}

f(1, 2, 3);

// [ 1, 2, 3 ]
```

Using arrow functions and Array.from()

```
// Using an arrow function as the map function to
// manipulate the elements
Array.from([1, 2, 3], (x) => x + x);
// [2, 4, 6]

// Generate a sequence of numbers
// Since the array is initialized with `undefined` on each position,
// the value of `v` below will be `undefined`
Array.from({ length: 5 }, (v, i) => i);
// [0, 1, 2, 3, 4]
```

Sequence generator (range)

```
// Sequence generator function (commonly referred to as "range", cf. Python, Clojure, etc.)
const range = (start, stop, step) =>
  Array.from(
    { length: Math.ceil((stop - start) / step) },
    (_, i) => start + i * step,
  );

// Generate a sequence of numbers from 0 (inclusive) to 5 (exclusive), incrementing by 1
range(0, 5, 1);
// [0, 1, 2, 3, 4]

// Generate a sequence of numbers from 1 (inclusive) to 10 (exclusive), incrementing by 2
range(1, 10, 2);
// [1, 3, 5, 7, 9]

// Generate the Latin alphabet making use of it being ordered as a sequence
range("A".charCodeAt(0), "Z".charCodeAt(0) + 1, 1).map((x) =>
  String.fromCharCode(x),
);
// ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z"]
```

Calling from() on non-array constructors

The `from()` method can be called on any constructor function that accepts a single argument representing the length of the new array.

```
function NotArray(len) {
  console.log("NotArray called with length", len);
}

// Iterable
console.log(Array.from.call(NotArray, new Set(["foo", "bar", "baz"])));
// NotArray called with length undefined
// NotArray { '0': 'foo', '1': 'bar', '2': 'baz', length: 3 }

// Array-like
console.log(Array.from.call(NotArray, { length: 1, 0: "foo" }));
// NotArray called with length 1
// NotArray { '0': 'foo', length: 1 }
```

When the `this` value is not a constructor, a plain `Array` object is returned.

```
console.log(Array.from.call({}, { length: 1, 0: "foo" })); // [ 'foo' ]
```

See also

- Polyfill of `Array.from` in `core-js`
- es-shims polyfill of `Array.from`
- Indexed collections guide
- `Array`
- `Array()`
- `Array.of()`
- `Array.fromAsync()`
- `Array.prototype.map()`
- `TypedArray.from()`

Array.fromAsync()

The **`Array.fromAsync()`** static method creates a new, shallow-copied `Array` instance from an async iterable, iterable, or array-like object.

Syntax

```
Array.fromAsync(items)
Array.fromAsync(items, mapFn)
Array.fromAsync(items, mapFn, thisArg)
```

Parameters

`items`

An async iterable, iterable, or array-like object to convert to an array.

`mapFn` Optional

A function to call on every element of the array. If provided, every value to be added to the array is first passed through this function, and `mapFn`'s return value is added to the array instead (after being awaited). The function is called with the following arguments:

`element`

The current element being processed in the array. If `items` is a sync iterable or array-like object, then all elements are first awaited, and `element` will never be a thenable. If `items` is an async iterable, then each yielded value is passed as-is.

`index`

The index of the current element being processed in the array.

`thisArg` Optional

Value to use as `this` when executing `mapFn`.

Return value

A new `Promise` whose fulfillment value is a new `Array` instance.

Description

`Array.fromAsync()` lets you create arrays from:

- async iterable objects (objects such as `ReadableStream` and `AsyncGenerator`); or, if the object is not async iterable,
- iterable objects (objects such as `Map` and `Set`); or, if the object is not iterable,
- array-like objects (objects with a `length` property and indexed elements).

`Array.fromAsync()` iterates the async iterable in a fashion very similar to `for await...of`. `Array.fromAsync(items)` is generally equivalent to the following code, if `items` is an async iterable or sync iterable:

```
const result = [];
for await (const element of items) {
  result.push(element);
}
```

`Array.fromAsync()` is almost equivalent to `Array.from()` in terms of behavior, except the following:

- `Array.fromAsync()` handles async iterable objects.
- `Array.fromAsync()` returns a `Promise` that fulfills to the array instance.
- If `Array.fromAsync()` is called with a non-async iterable object, each element to be added to the array is first awaited.
- If a `mapFn` is provided, its output is also internally awaited.

`Array.fromAsync()` and `Promise.all()` can both turn an iterable of promises into a promise of an array. However, there are two key differences:

- `Array.fromAsync()` awaits each value yielded from the object sequentially. `Promise.all()` awaits all values concurrently.
- `Array.fromAsync()` iterates the iterable lazily, and doesn't retrieve the next value until the current one is settled. `Promise.all()` retrieves all values in advance and awaits them all.

Examples

Array from an async iterable

```
const asyncIterable = (async function* () {
  for (let i = 0; i < 5; i++) {
    await new Promise((resolve) => setTimeout(resolve, 10 * i));
    yield i;
  }
})();

Array.fromAsync(asyncIterable).then((array) => console.log(array));
// [0, 1, 2, 3, 4]
```

When `items` is an async iterable where each result's `value` is also a promise, then those promises are added to the resulting array without being awaited. This is consistent with the behavior of `for await...of`.

```
function createAsyncIter() {
  let i = 0;
  return {
    [Symbol.asyncIterator]() {
      return {
        async next() {
          if (i > 2) return { done: true };
          i++;
          return { value: Promise.resolve(i), done: false };
        },
      };
```

```
    },
  };
}

Array.fromAsync(createAsyncIter()).then((array) => console.log(array));
// (3) [Promise, Promise, Promise]
```

> **Note:** In practice, you will rarely encounter an async iterable that yields promises, because if you implement it using an async generator function, then the `yield` expression automatically unwraps promises.

Array from a sync iterable

```
Array.fromAsync(
  new Map([
    [1, 2],
    [3, 4],
  ]),
).then((array) => console.log(array));
// [[1, 2], [3, 4]]
```

Array from a sync iterable that yields promises

```
Array.fromAsync(
  new Set([Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)]),
).then((array) => console.log(array));
// [1, 2, 3]
```

Array from an array-like object of promises

```
Array.fromAsync({
  length: 3,
  0: Promise.resolve(1),
  1: Promise.resolve(2),
  2: Promise.resolve(3),
}).then((array) => console.log(array));
// [1, 2, 3]
```

Using mapFn with a sync iterable

When `items` is a sync iterable or array-like object, both the input and output of `mapFn` are awaited internally by `Array.fromAsync()`.

```
function delayedValue(v) {
  return new Promise((resolve) => setTimeout(() => resolve(v), 100));
}

Array.fromAsync(
  [delayedValue(1), delayedValue(2), delayedValue(3)],
  (element) => delayedValue(element * 2),
).then((array) => console.log(array));
// [2, 4, 6]
```

Using mapFn with an async iterable

When `items` is an async iterable, the input to `mapFn` is not awaited, but the output is. Using the same `createAsyncIter` function as above:

```
Array.fromAsync(createAsyncIter(), async (element) => (await element) * 2).then(
  (array) => console.log(array),
);
// [2, 4, 6]
```

Curiously, this means that `Array.fromAsync(createAsyncIter())` is not equivalent to `Array.fromAsync(createAsyncIter(), (element) => element)`, because the latter awaits each yielded value, while the former does not.

```
Array.fromAsync(createAsyncIter(), (element) => element).then((array) =>
  console.log(array),
);
// [1, 2, 3]
```

Comparison with Promise.all()

`Array.fromAsync()` awaits each value yielded from the object sequentially. `Promise.all()` awaits all values concurrently.

```
function* makeIterableOfPromises() {
  for (let i = 0; i < 5; i++) {
    yield new Promise((resolve) => setTimeout(resolve, 100));
  }
}

(async () => {
  console.time("Array.fromAsync() time");
```

```
  await Array.fromAsync(makeIterableOfPromises());
  console.timeEnd("Array.fromAsync() time");
  // Array.fromAsync() time: 503.610ms

  console.time("Promise.all() time");
  await Promise.all(makeIterableOfPromises());
  console.timeEnd("Promise.all() time");
  // Promise.all() time: 101.728ms
})();
```

Closing sync iterables on rejection

Similar to `for await...of`, if the object being iterated is a sync iterable, and a yielded promise rejects, the `return()` method of the underlying iterator will be called, if one exists, to allow the iterator to perform cleanup.

```
function* generatorWithRejectedPromises() {
  try {
    yield 0;
    yield Promise.reject(new Error("error"));
  } finally {
    console.log("called finally");
  }
}

(async () => {
  try {
    await Array.fromAsync(generatorWithRejectedPromises());
  } catch (e) {
    console.log("caught", e);
  }
})();
// called finally
// caught Error: error
```

See also

- Polyfill of `Array.fromAsync` in `core-js`
- Indexed collections guide
- `Array`
- `Array()`
- `Array.of()`
- `Array.from()`
- `for await...of`

Array.prototype.includes()

The **`includes()`** method of `Array` instances determines whether an array includes a certain value among its entries, returning `true` or `false` as appropriate.

Try it

```
const array = [1, 2, 3];

console.log(array.includes(2));
// Expected output: true

const pets = ["cat", "dog", "bat"];

console.log(pets.includes("cat"));
// Expected output: true

console.log(pets.includes("at"));
// Expected output: false
```

Syntax

```
includes(searchElement)
includes(searchElement, fromIndex)
```

Parameters

`searchElement`

The value to search for.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= fromIndex < 0`, `fromIndex + array.length` is used. However, the array is still searched from front to back in this case.
- If `fromIndex < -array.length` or `fromIndex` is omitted, `0` is used, causing the entire array to be searched.
- If `fromIndex >= array.length`, the array is not searched and `false` is returned.

Return value

A boolean value which is `true` if the value `searchElement` is found within the array (or the part of the array indicated by the index `fromIndex`, if specified).

Description

The `includes()` method compares `searchElement` to elements of the array using the SameValueZero algorithm. This algorithm works like strict equality `===` (where `-0` and `+0` are considered equal), with the exception that `NaN` is considered equal to itself.

When used on sparse arrays, the `includes()` method iterates empty slots as if they have the value `undefined`.

The `includes()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using includes()

```
[1, 2, 3].includes(2); // true
[1, 2, 3].includes(4); // false
[1, 2, 3].includes(3, 3); // false
[1, 2, 3].includes(3, -1); // true
[1, 2, NaN].includes(NaN); // true
["1", "2", "3"].includes(3); // false
```

fromIndex is greater than or equal to the array length

If `fromIndex` is greater than or equal to the length of the array, `false` is returned. The array will not be searched.

```
const arr = ["a", "b", "c"];

arr.includes("c", 3); // false
arr.includes("c", 100); // false
```

Computed index is less than 0

If `fromIndex` is negative, the computed index is calculated to be used as a position in the array at which to begin searching for `searchElement`. If the computed index is less than or equal to `0`, the entire array will be searched.

```
// array length is 3
// fromIndex is -100
// computed index is 3 + (-100) = -97

const arr = ["a", "b", "c"];

arr.includes("a", -100); // true
arr.includes("b", -100); // true
arr.includes("c", -100); // true
arr.includes("a", -2); // false
```

Using includes() on sparse arrays

You can search for `undefined` in a sparse array and get `true`.

```
console.log([1, , 3].includes(undefined)); // true
```

Calling includes() on non-array objects

The `includes()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 1, // ignored by includes() since length is 3
};
console.log(Array.prototype.includes.call(arrayLike, 2));
// true
console.log(Array.prototype.includes.call(arrayLike, 1));
// false
```

See also

- Polyfill of `Array.prototype.includes` in `core-js`
- es-shims polyfill of `Array.prototype.includes`
- Indexed collections guide
- `Array`
- `Array.prototype.indexOf()`
- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `TypedArray.prototype.includes()`

- `String.prototype.includes()`

Array.prototype.indexOf()

The **`indexOf()`** method of `Array` instances returns the first index at which a given element can be found in the array, or -1 if it is not present.

Try it

```
const beasts = ["ant", "bison", "camel", "duck", "bison"];

console.log(beasts.indexOf("bison"));
// Expected output: 1

// Start from index 2
console.log(beasts.indexOf("bison", 2));
// Expected output: 4

console.log(beasts.indexOf("giraffe"));
// Expected output: -1
```

Syntax

```
indexOf(searchElement)
indexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the array.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= fromIndex < 0`, `fromIndex + array.length` is used. Note, the array is still searched from front to back in this case.
- If `fromIndex < -array.length` or `fromIndex` is omitted, `0` is used, causing the entire array to be searched.
- If `fromIndex >= array.length`, the array is not searched and `-1` is returned.

Return value

The first index of `searchElement` in the array; `-1` if not found.

Description

The `indexOf()` method compares `searchElement` to elements of the array using strict equality (the same algorithm used by the `===` operator). `NaN` values are never compared as equal, so `indexOf()` always returns `-1` when `searchElement` is `NaN`.

The `indexOf()` method skips empty slots in sparse arrays.

The `indexOf()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using indexOf()

The following example uses `indexOf()` to locate values in an array.

```
const array = [2, 9, 9];
array.indexOf(2); // 0
array.indexOf(7); // -1
array.indexOf(9, 2); // 2
array.indexOf(2, -1); // -1
array.indexOf(2, -3); // 0
```

You cannot use `indexOf()` to search for `NaN`.

```
const array = [NaN];
array.indexOf(NaN); // -1
```

Finding all the occurrences of an element

```
const indices = [];
const array = ["a", "b", "a", "c", "a", "d"];
const element = "a";
let idx = array.indexOf(element);
while (idx !== -1) {
  indices.push(idx);
  idx = array.indexOf(element, idx + 1);
}
console.log(indices);
// [0, 2, 4]
```

Finding if an element exists in the array or not and updating the array

```
function updateVegetablesCollection(veggies, veggie) {
  if (veggies.indexOf(veggie) === -1) {
    veggies.push(veggie);
    console.log(`New veggies collection is: ${veggies}`);
  } else {
    console.log(`${veggie} already exists in the veggies collection.`);
  }
}

const veggies = ["potato", "tomato", "chillies", "green-pepper"];

updateVegetablesCollection(veggies, "spinach");
// New veggies collection is: potato,tomato,chillies,green-pepper,spinach
updateVegetablesCollection(veggies, "spinach");
// spinach already exists in the veggies collection.
```

Using indexOf() on sparse arrays

You cannot use `indexOf()` to search for empty slots in sparse arrays.

```
console.log([1, , 3].indexOf(undefined)); // -1
```

Calling indexOf() on non-array objects

The `indexOf()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by indexOf() since length is 3
};
console.log(Array.prototype.indexOf.call(arrayLike, 2));
// 0
console.log(Array.prototype.indexOf.call(arrayLike, 5));
// -1
```

See also

- Polyfill of `Array.prototype.indexOf` in `core-js`
- es-shims polyfill of `Array.prototype.indexOf`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.indexOf()`
- `String.prototype.indexOf()`

Array.isArray()

The **`Array.isArray()`** static method determines whether the passed value is an `Array`.

Try it

```
console.log(Array.isArray([1, 3, 5]));
// Expected output: true

console.log(Array.isArray("[]"));
// Expected output: false

console.log(Array.isArray(new Array(5)));
// Expected output: true

console.log(Array.isArray(new Int16Array([15, 33])));
// Expected output: false
```

Syntax

```
Array.isArray(value)
```

Parameters

`value`

The value to be checked.

Return value

`true` if `value` is an `Array`; otherwise, `false`. `false` is always returned if `value` is a `TypedArray` instance.

Description

`Array.isArray()` checks if the passed value is an `Array`. It performs a *branded check*, similar to the `in` operator, for a private field initialized by the `Array()` constructor.

It is a more robust alternative to `instanceof Array` because it avoids false positives and false negatives:

- `Array.isArray()` rejects values that aren't actual `Array` instances, even if they have `Array.prototype` in their prototype chain — `instanceof Array` would accept these as it does check the prototype chain.
- `Array.isArray()` accepts `Array` objects constructed in another realm — `instanceof Array` returns `false` for these because the identity of the `Array` constructor is different across realms.

See the article "Determining with absolute accuracy whether or not a JavaScript object is an array" for more details.

Examples

Using Array.isArray()

```
// all following calls return true
Array.isArray([]);
Array.isArray([1]);
Array.isArray(new Array());
Array.isArray(new Array("a", "b", "c", "d"));
Array.isArray(new Array(3));
// Little known fact: Array.prototype itself is an array:
Array.isArray(Array.prototype);

// all following calls return false
Array.isArray();
Array.isArray({});
Array.isArray(null);
Array.isArray(undefined);
Array.isArray(17);
Array.isArray("Array");
Array.isArray(true);
Array.isArray(false);
Array.isArray(new Uint8Array(32));
// This is not an array, because it was not created using the
// array literal syntax or the Array constructor
Array.isArray({ __proto__: Array.prototype });
```

instanceof vs. Array.isArray()

When checking for `Array` instance, `Array.isArray()` is preferred over `instanceof` because it works across realms.

```
const iframe = document.createElement("iframe");
document.body.appendChild(iframe);
const xArray = window.frames[window.frames.length - 1].Array;
const arr = new xArray(1, 2, 3); // [1, 2, 3]

// Correctly checking for Array
Array.isArray(arr); // true
// The prototype of arr is xArray.prototype, which is a
// different object from Array.prototype
arr instanceof Array; // false
```

See also

- Polyfill of `Array.isArray` in `core-js`
- es-shims polyfill of `Array.isArray`
- Indexed collections guide
- `Array`

Array.prototype.join()

The **`join()`** method of `Array` instances returns a new string that is the concatenation of all elements in this array, separated by commas or a specified separator string. If the array has only one item, that item's stringification is returned without using the separator.

Try it

```
const elements = ["Fire", "Air", "Water"];

console.log(elements.join());
// Expected output: "Fire,Air,Water"

console.log(elements.join(""));
// Expected output: "FireAirWater"

console.log(elements.join("-"));
// Expected output: "Fire-Air-Water"
```

Syntax

```
join()
join(separator)
```

Parameters

`separator` Optional

A string to separate each pair of adjacent elements of the array. If omitted, the array elements are separated with a comma (",").

Return value

A string with all array elements joined. If `array.length` is `0`, the empty string is returned.

Description

The string conversions of all array elements are joined into one string. If an element is `undefined` or `null`, it is converted to an empty string instead of the string `"null"` or `"undefined"`.

The `join` method is accessed internally by `Array.prototype.toString()` with no arguments. Overriding `join` of an array instance will override its `toString` behavior as well.

`Array.prototype.join` recursively converts each element, including other arrays, to strings. Because the string returned by `Array.prototype.toString` (which is the same as calling `join()`) does not have delimiters, nested arrays look like they are flattened. You can only control the separator of the first level, while deeper levels always use the default comma.

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

console.log(matrix.join()); // 1,2,3,4,5,6,7,8,9
console.log(matrix.join(";")); // 1,2,3;4,5,6;7,8,9
```

When an array is cyclic (it contains an element that is itself), browsers avoid infinite recursion by ignoring the cyclic reference.

```
const arr = [];
arr.push(1, [3, arr, 4], 2);
console.log(arr.join(";")); // 1;3,,4;2
```

When used on sparse arrays, the `join()` method iterates empty slots as if they have the value `undefined`.

The `join()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Joining an array four different ways

The following example creates an array, `a`, with three elements, then joins the array four times: using the default separator, then a comma and a space, then a plus and an empty string.

```
const a = ["Wind", "Water", "Fire"];
a.join(); // 'Wind,Water,Fire'
a.join(", "); // 'Wind, Water, Fire'
a.join(" + "); // 'Wind + Water + Fire'
a.join(""); // 'WindWaterFire'
```

Using join() on sparse arrays

`join()` treats empty slots the same as `undefined` and produces an extra separator:

```
console.log([1, , 3].join()); // '1,,3'
console.log([1, undefined, 3].join()); // '1,,3'
```

Calling join() on non-array objects

The `join()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by join() since length is 3
};
console.log(Array.prototype.join.call(arrayLike));
// 2,3,4
console.log(Array.prototype.join.call(arrayLike, "."));
// 2.3.4
```

See also

- Polyfill of `Array.prototype.join` in `core-js`
- es-shims polyfill of `Array.prototype.join`
- Indexed collections guide
- `Array`
- `Array.prototype.toString()`
- `TypedArray.prototype.join()`
- `String.prototype.split()`

Array.prototype.keys()

The **`keys()`** method of `Array` instances returns a new *array iterator* object that contains the keys for each index in the array.

Try it

```
const array = ["a", "b", "c"];
const iterator = array.keys();

for (const key of iterator) {
  console.log(key);
}

// Expected output: 0
// Expected output: 1
// Expected output: 2
```

Syntax

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

When used on sparse arrays, the `keys()` method iterates empty slots as if they have the value `undefined`.

The `keys()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using keys() on sparse arrays

Unlike `Object.keys()`, which only includes keys that actually exist in the array, the `keys()` iterator doesn't ignore holes representing missing properties.

```
const arr = ["a", , "c"];
const sparseKeys = Object.keys(arr);
const denseKeys = [...arr.keys()];
console.log(sparseKeys); // ['0', '2']
console.log(denseKeys); // [0, 1, 2]
```

Calling keys() on non-array objects

The `keys()` method reads the `length` property of `this` and then yields all integer indices between 0 and `length - 1`. No index access actually happens.

```
const arrayLike = {
  length: 3,
};
for (const entry of Array.prototype.keys.call(arrayLike)) {
  console.log(entry);
}
// 0
// 1
// 2
```

See also

- Polyfill of `Array.prototype.keys` in `core-js`
- es-shims polyfill of `Array.prototype.keys`
- Indexed collections guide
- `Array`
- `Array.prototype.entries()`
- `Array.prototype.values()`
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype.keys()`
- Iteration protocols

Array.prototype.lastIndexOf()

The **`lastIndexOf()`** method of `Array` instances returns the last index at which a given element can be found in the array, or -1 if it is not present. The array is searched backwards, starting at `fromIndex`.

Try it

```
const animals = ["Dodo", "Tiger", "Penguin", "Dodo"];

console.log(animals.lastIndexOf("Dodo"));
// Expected output: 3

console.log(animals.lastIndexOf("Tiger"));
// Expected output: 1
```

Syntax

```
lastIndexOf(searchElement)
lastIndexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the array.

`fromIndex` Optional

Zero-based index at which to start searching backwards, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= fromIndex < 0`, `fromIndex + array.length` is used.
- If `fromIndex < -array.length`, the array is not searched and `-1` is returned. You can think of it conceptually as starting at a nonexistent position before the beginning of the array and going backwards from there. There are no array elements on the way, so `searchElement` is never found.
- If `fromIndex >= array.length` or `fromIndex` is omitted or `undefined`, `array.length - 1` is used, causing the entire array to be searched. You can think of it conceptually as starting at a nonexistent position beyond the end of the array and going backwards from there. It eventually reaches the real end position of the array, at which point it starts searching backwards through the actual array elements.

Return value

The last index of `searchElement` in the array; `-1` if not found.

Description

The `lastIndexOf()` method compares `searchElement` to elements of the array using strict equality (the same algorithm used by the `===` operator). `NaN` values are never compared as equal, so `lastIndexOf()` always returns `-1` when `searchElement` is `NaN`.

The `lastIndexOf()` method skips empty slots in sparse arrays.

The `lastIndexOf()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using lastIndexOf()

The following example uses `lastIndexOf()` to locate values in an array.

```
const numbers = [2, 5, 9, 2];
numbers.lastIndexOf(2); // 3
numbers.lastIndexOf(7); // -1
numbers.lastIndexOf(2, 3); // 3
numbers.lastIndexOf(2, 2); // 0
numbers.lastIndexOf(2, -2); // 0
numbers.lastIndexOf(2, -1); // 3
```

You cannot use `lastIndexOf()` to search for `NaN`.

```
const array = [NaN];
array.lastIndexOf(NaN); // -1
```

Finding all the occurrences of an element

The following example uses `lastIndexOf` to find all the indices of an element in a given array, using `push()` to add them to another array as they are found.

```
const indices = [];
const array = ["a", "b", "a", "c", "a", "d"];
const element = "a";
let idx = array.lastIndexOf(element);
while (idx !== -1) {
  indices.push(idx);
  idx = idx > 0 ? array.lastIndexOf(element, idx - 1) : -1;
}
```

```
console.log(indices);
// [4, 2, 0]
```

Note that we have to handle the case `idx === 0` separately here because the element will always be found regardless of the `fromIndex` parameter if it is the first element of the array. This is different from the `indexOf()` method.

Using lastIndexOf() on sparse arrays

You cannot use `lastIndexOf()` to search for empty slots in sparse arrays.

```
console.log([1, , 3].lastIndexOf(undefined)); // -1
```

Calling lastIndexOf() on non-array objects

The `lastIndexOf()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 2,
  3: 5, // ignored by lastIndexOf() since length is 3
};
console.log(Array.prototype.lastIndexOf.call(arrayLike, 2));
// 2
console.log(Array.prototype.lastIndexOf.call(arrayLike, 5));
// -1
```

See also

- Polyfill of `Array.prototype.lastIndexOf` in `core-js`
- es-shims polyfill of `Array.prototype.lastIndexOf`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.indexOf()`
- `TypedArray.prototype.lastIndexOf()`
- `String.prototype.lastIndexOf()`

Array: length

The **`length`** data property of an `Array` instance represents the number of slots in that array. The value is an unsigned, 32-bit integer that is always numerically greater than the highest index in the array. It may be greater than the number of elements if the array is sparse.

Try it

```
const clothing = ["shoes", "shirts", "socks", "sweaters"];

console.log(clothing.length);
// Expected output: 4
```

Value

A nonnegative integer less than 2^{32}.

Property attributes of `Array: length`	
Writable	yes
Enumerable	no
Configurable	no

Description

The value of the `length` property is a nonnegative integer with a value less than 2^{32}.

```
const listA = [1, 2, 3];
const listB = new Array(6);

console.log(listA.length);
// 3

console.log(listB.length);
// 6

listB.length = 2 ** 32; // 4294967296
// RangeError: Invalid array length

const listC = new Array(-100); // Negative numbers are not allowed
// RangeError: Invalid array length
```

The array object observes the `length` property, and automatically syncs the `length` value with the array's content. This means:

- Setting `length` to a value smaller than the current length truncates the array — elements beyond the new `length` are deleted.
- Setting any array index (a nonnegative integer smaller than 2^{32}) beyond the current `length` extends the array — the `length` property is increased to reflect the new highest index.
- Setting `length` to an invalid value (e.g., a negative number or a non-integer) throws a `RangeError` exception.

When `length` is set to a bigger value than the current length, the array is extended by adding empty slots, not actual `undefined` values. Empty slots have some special interactions with array methods; see array methods and empty slots.

```
const arr = [1, 2];
console.log(arr);
// [ 1, 2 ]

arr.length = 5; // set array length to 5 while currently 2.
console.log(arr);
// [ 1, 2, <3 empty items> ]

arr.forEach((element) => console.log(element));
// 1
// 2
```

See also Relationship between `length` and numerical properties.

Examples

Iterating over an array

In the following example, the array `numbers` is iterated through by looking at the `length` property. The value in each element is then doubled.

```
const numbers = [1, 2, 3, 4, 5];
const length = numbers.length;
for (let i = 0; i < length; i++) {
  numbers[i] *= 2;
}
// numbers is now [2, 4, 6, 8, 10]
```

Shortening an array

The following example shortens the array `numbers` to a length of 3 if the current length is greater than 3.

```
const numbers = [1, 2, 3, 4, 5];

if (numbers.length > 3) {
  numbers.length = 3;
}

console.log(numbers); // [1, 2, 3]
console.log(numbers.length); // 3
console.log(numbers[3]); // undefined; the extra elements are deleted
```

Create empty array of fixed length

Setting `length` to a value greater than the current length creates a sparse array.

```
const numbers = [];
numbers.length = 3;
console.log(numbers); // [empty x 3]
```

Array with non-writable length

The `length` property is automatically updated by the array when elements are added beyond the current length. If the `length` property is made non-writable, the array will not be able to update it. This causes an error in strict mode.

```
"use strict";

const numbers = [1, 2, 3, 4, 5];
Object.defineProperty(numbers, "length", { writable: false });
numbers[5] = 6; // TypeError: Cannot assign to read only property 'length' of object '[object Array]'
numbers.push(5); // // TypeError: Cannot assign to read only property 'length' of object '[object Array]'
```

See also

- Indexed collections guide
- `Array`
- `TypedArray.prototype.length`
- `String: length`
- RangeError: invalid array length

Array.prototype.map()

The **`map()`** method of `Array` instances creates a new array populated with the results of calling a provided function on every element in the calling array.

Try it

```
const array = [1, 4, 9, 16];

// Pass a function to map
const mapped = array.map((x) => x * 2);

console.log(mapped);
// Expected output: Array [2, 8, 18, 32]
```

Syntax

```
map(callbackFn)
map(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value is added as a single element in the new array. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `map()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A new array with each element being the result of the callback function.

Description

The `map()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array and constructs a new array from the results. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `map()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Since `map` builds a new array, calling it without using the returned array is an anti-pattern; use `forEach` or `for...of` instead.

Examples

Mapping an array of numbers to an array of square roots

The following code takes an array of numbers and creates a new array containing the square roots of the numbers in the first array.

```
const numbers = [1, 4, 9];
const roots = numbers.map((num) => Math.sqrt(num));

// roots is now     [1, 2, 3]
// numbers is still [1, 4, 9]
```

Using map to reformat objects in an array

The following code takes an array of objects and creates a new array containing the newly reformatted objects.

```
const kvArray = [
  { key: 1, value: 10 },
  { key: 2, value: 20 },
  { key: 3, value: 30 },
];

const reformattedArray = kvArray.map(({ key, value }) => ({ [key]: value }));

console.log(reformattedArray); // [{ 1: 10 }, { 2: 20 }, { 3: 30 }]
console.log(kvArray);
// [
//   { key: 1, value: 10 },
//   { key: 2, value: 20 },
```

```
//   { key: 3, value: 30 }
// ]
```

Using parseInt() with map()

It is common to use the callback with one argument (the element being traversed). Certain functions are also commonly used with one argument, even though they take additional optional arguments. These habits may lead to confusing behaviors. Consider:

```
["1", "2", "3"].map(parseInt);
```

While one might expect `[1, 2, 3]`, the actual result is `[1, NaN, NaN]`.

`parseInt` is often used with one argument, but takes two. The first is an expression and the second is the radix to the callback function, `Array.prototype.map` passes 3 arguments: the element, the index, and the array. The third argument is ignored by `parseInt` — but *not* the second one! This is the source of possible confusion.

Here is a concise example of the iteration steps:

```
/* first iteration  (index is 0): */ parseInt("1", 0); // 1
/* second iteration (index is 1): */ parseInt("2", 1); // NaN
/* third iteration  (index is 2): */ parseInt("3", 2); // NaN
```

To solve this, define another function that only takes one argument:

```
["1", "2", "3"].map((str) => parseInt(str, 10)); // [1, 2, 3]
```

You can also use the `Number` function, which only takes one argument:

```
["1", "2", "3"].map(Number); // [1, 2, 3]

// But unlike parseInt(), Number() will also return a float or (resolved) exponential notation:
["1.1", "2.2e2", "3e300"].map(Number); // [1.1, 220, 3e+300]

// For comparison, if we use parseInt() on the array above:
["1.1", "2.2e2", "3e300"].map((str) => parseInt(str, 10)); // [1, 2, 3]
```

See A JavaScript optional argument hazard by Allen Wirfs-Brock for more discussions.

Mapped array contains undefined

When `undefined` or nothing is returned, the resulting array contains `undefined`. If you want to delete the element instead, chain a `filter()` method, or use the `flatMap()` method and return an empty array to signify deletion.

```
const numbers = [1, 2, 3, 4];
const filteredNumbers = numbers.map((num, index) => {
  if (index < 3) {
    return num;
  }
});

// index goes from 0, so the filteredNumbers are 1,2,3 and undefined.
// filteredNumbers is [1, 2, 3, undefined]
// numbers is still [1, 2, 3, 4]
```

Side-effectful mapping

The callback can have side effects.

```
const cart = [5, 15, 25];
let total = 0;
const withTax = cart.map((cost) => {
  total += cost;
  return cost * 1.2;
});
console.log(withTax); // [6, 18, 30]
console.log(total); // 45
```

This is not recommended, because copying methods are best used with pure functions. In this case, we can choose to iterate the array twice.

```
const cart = [5, 15, 25];
const total = cart.reduce((acc, cost) => acc + cost, 0);
const withTax = cart.map((cost) => cost * 1.2);
```

Sometimes this pattern goes to its extreme and the *only* useful thing that `map()` does is causing side effects.

```
const products = [
  { name: "sports car" },
  { name: "laptop" },
  { name: "phone" },
];

products.map((product) => {
```

```
  product.price = 100;
});
```

As mentioned previously, this is an anti-pattern. If you don't use the return value of `map()`, use `forEach()` or a `for...of` loop instead.

```
products.forEach((product) => {
  product.price = 100;
});
```

Or, if you want to create a new array instead:

```
const productsWithPrice = products.map((product) => ({
  ...product,
  price: 100,
}));
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `map()` to create a new array where each element is the average of its neighbors and itself.

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const averaged = numbers
  .filter((num) => num > 0)
  .map((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    const prev = arr[idx - 1];
    const next = arr[idx + 1];
    let count = 1;
    let total = num;
    if (prev !== undefined) {
      count++;
      total += prev;
    }
    if (next !== undefined) {
      count++;
      total += next;
    }
    const average = total / count;
    // Keep two decimal places
    return Math.round(average * 100) / 100;
  });
console.log(averaged); // [2, 2.67, 2, 3.33, 5, 5.33, 5.67, 4]
```

The `array` argument is *not* the array that is being built — there is no way to access the array being built from the callback function.

Using map() on sparse arrays

A sparse array remains sparse after `map()`. The indices of empty slots are still empty in the returned array, and the callback function won't be called on them.

```
console.log(
  [1, , 3].map((x, index) => {
    console.log(`Visit ${index}`);
    return x * 2;
  }),
);
// Visit 0
// Visit 2
// [2, empty, 6]
```

Calling map() on non-array objects

The `map()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by map() since length is 3
};
console.log(Array.prototype.map.call(arrayLike, (x) => x ** 2));
// [ 4, 9, 16 ]
```

This example shows how to iterate through a collection of objects collected by `querySelectorAll()`, returning an array containing the values of all the selected `option` elements. Because `querySelectorAll()` returns a `NodeList`, which is an array-like object without the `map()` method, we use `Array.prototype.map.call()`, passing `elems` as the `this` value and the callback as the second argument.

```
const elems = document.querySelectorAll("select option:checked");
const values = Array.prototype.map.call(elems, ({ value }) => value);
```

You can also use `Array.from()` to transform `elems` to an array, and then access the `map()` method.

See also

- Polyfill of `Array.prototype.map` in `core-js`
- es-shims polyfill of `Array.prototype.map`
- Indexed collections guide
- `Array`
- `Array.prototype.forEach()`
- `Array.from()`
- `TypedArray.prototype.map()`
- `Map`

Array.of()

The **`Array.of()`** static method creates a new `Array` instance from a variable number of arguments, regardless of number or type of the arguments.

Try it

```
console.log(Array.of("foo", 2, "bar", true));
// Expected output: Array ["foo", 2, "bar", true]

console.log(Array.of());
// Expected output: Array []
```

Syntax

```
Array.of()
Array.of(element1)
Array.of(element1, element2)
Array.of(element1, element2, /* …, */ elementN)
```

Parameters

`element1, ..., elementN`

Elements used to create the array.

Return value

A new `Array` instance.

Description

The difference between `Array.of()` and the `Array()` constructor is in the handling of single arguments: `Array.of(7)` creates an array with a single element, `7`, whereas `Array(7)` creates an empty array with a `length` property of `7`. (That implies an array of 7 empty slots, not slots with actual `undefined` values.)

```
Array.of(7); // [7]
Array(7); // array of 7 empty slots

Array.of(1, 2, 3); // [1, 2, 3]
Array(1, 2, 3); // [1, 2, 3]
```

The `Array.of()` method is a generic factory method. For example, if a subclass of `Array` inherits the `of()` method, the inherited `of()` method will return new instances of the subclass instead of `Array` instances. In fact, the `this` value can be any constructor function that accepts a single argument representing the length of the new array, and the constructor will be called with the number of arguments passed to `of()`. The final `length` will be set again when all elements are assigned. If the `this` value is not a constructor function, the plain `Array` constructor is used instead.

Examples

Using Array.of()

```
Array.of(1); // [1]
Array.of(1, 2, 3); // [1, 2, 3]
Array.of(undefined); // [undefined]
```

Calling of() on non-array constructors

The `of()` method can be called on any constructor function that accepts a single argument representing the length of the new array.

```
function NotArray(len) {
  console.log("NotArray called with length", len);
}

console.log(Array.of.call(NotArray, 1, 2, 3));
// NotArray called with length 3
// NotArray { '0': 1, '1': 2, '2': 3, length: 3 }

console.log(Array.of.call(Object)); // [Number: 0] { length: 0 }
```

When the `this` value is not a constructor, a plain `Array` object is returned.

```
console.log(Array.of.call({}, 1)); // [ 1 ]
```

See also

- Polyfill of `Array.of` in `core-js`
- es-shims polyfill of `Array.of`
- Indexed collections guide
- `Array`
- `Array()`
- `Array.from()`
- `TypedArray.of()`

Array.prototype.pop()

The **`pop()`** method of `Array` instances removes the **last** element from an array and returns that element. This method changes the length of the array.

Try it

```
const plants = ["broccoli", "cauliflower", "cabbage", "kale", "tomato"];

console.log(plants.pop());
// Expected output: "tomato"

console.log(plants);
// Expected output: Array ["broccoli", "cauliflower", "cabbage", "kale"]

plants.pop();

console.log(plants);
// Expected output: Array ["broccoli", "cauliflower", "cabbage"]
```

Syntax

```
pop()
```

Parameters

None.

Return value

The removed element from the array; `undefined` if the array is empty.

Description

The `pop()` method removes the last element from an array and returns that value to the caller. If you call `pop()` on an empty array, it returns `undefined`.

`Array.prototype.shift()` has similar behavior to `pop()`, but applied to the first element in an array.

The `pop()` method is a mutating method. It changes the length and the content of `this`. In case you want the value of `this` to be the same, but return a new array with the last element removed, you can use `arr.slice(0, -1)` instead.

The `pop()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Removing the last element of an array

The following code creates the `myFish` array containing four elements, then removes its last element.

```
const myFish = ["angel", "clown", "mandarin", "sturgeon"];

const popped = myFish.pop();

console.log(myFish); // ['angel', 'clown', 'mandarin' ]

console.log(popped); // 'sturgeon'
```

Calling pop() on non-array objects

The `pop()` method reads the `length` property of `this`. If the normalized length is 0, `length` is set to `0` again (whereas it may be negative or `undefined` before). Otherwise, the property at `length - 1` is returned and deleted.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
console.log(Array.prototype.pop.call(arrayLike));
// 4
console.log(arrayLike);
```

```
// { length: 2, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.pop.call(plainObj);
console.log(plainObj);
// { length: 0 }
```

Using an object in an array-like fashion

`push` and `pop` are intentionally generic, and we can use that to our advantage — as the following example shows.

Note that in this example, we don't create an array to store a collection of objects. Instead, we store the collection on the object itself and use `call` on `Array.prototype.push` and `Array.prototype.pop` to trick those methods into thinking we're dealing with an array.

```
const collection = {
  length: 0,
  addElements(...elements) {
    // obj.length will be incremented automatically
    // every time an element is added.

    // Returning what push returns; that is
    // the new value of length property.
    return [].push.call(this, ...elements);
  },
  removeElement() {
    // obj.length will be decremented automatically
    // every time an element is removed.

    // Returning what pop returns; that is
    // the removed element.
    return [].pop.call(this);
  },
};

collection.addElements(10, 20, 30);
console.log(collection.length); // 3
collection.removeElement();
console.log(collection.length); // 2
```

See also

- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.shift()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

Array.prototype.push()

The **`push()`** method of `Array` instances adds the specified elements to the end of an array and returns the new length of the array.

Try it

```
const animals = ["pigs", "goats", "sheep"];

const count = animals.push("cows");
console.log(count);
// Expected output: 4
console.log(animals);
// Expected output: Array ["pigs", "goats", "sheep", "cows"]

animals.push("chickens", "cats", "dogs");
console.log(animals);
// Expected output: Array ["pigs", "goats", "sheep", "cows", "chickens", "cats", "dogs"]
```

Syntax

```
push()
push(element1)
push(element1, element2)
push(element1, element2, /* …, */ elementN)
```

Parameters

`element1, ..., elementN`

The element(s) to add to the end of the array.

Return value

The new `length` property of the object upon which the method was called.

Description

The `push()` method appends values to an array.

`Array.prototype.unshift()` has similar behavior to `push()`, but applied to the start of an array.

The `push()` method is a mutating method. It changes the length and the content of `this`. In case you want the value of `this` to be the same, but return a new array with elements appended to the end, you can use `arr.concat([element0, element1, /* ... ,*/ elementN])` instead. Notice that the elements are wrapped in an extra array — otherwise, if the element is an array itself, it would be spread instead of pushed as a single element due to the behavior of `concat()`.

The `push()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Adding elements to an array

The following code creates the `sports` array containing two elements, then appends two elements to it. The `total` variable contains the new length of the array.

```
const sports = ["soccer", "baseball"];
const total = sports.push("football", "swimming");

console.log(sports); // ['soccer', 'baseball', 'football', 'swimming']
console.log(total); // 4
```

Merging two arrays

This example uses spread syntax to push all elements from a second array into the first one.

```
const vegetables = ["parsnip", "potato"];
const moreVegs = ["celery", "beetroot"];

// Merge the second array into the first one
vegetables.push(...moreVegs);

console.log(vegetables); // ['parsnip', 'potato', 'celery', 'beetroot']
```

Merging two arrays can also be done with the `concat()` method, which will make a new combined array instead of adding to the original. The spread syntax only works if the number of elements in the array is less than the maximum number of function arguments allowed by the engine. For longer arrays, use `concat()` or call `push()` multiple times in a loop.

Calling push() on non-array objects

The `push()` method reads the `length` property of `this`. It then sets each index of `this` starting at `length` with the arguments passed to `push()`. Finally, it sets the `length` to the previous length plus the number of pushed elements.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
Array.prototype.push.call(arrayLike, 1, 2);
console.log(arrayLike);
// { '2': 4, '3': 1, '4': 2, length: 5, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.push.call(plainObj, 1, 2);
console.log(plainObj);
// { '0': 1, '1': 2, length: 2 }
```

Using an object in an array-like fashion

As mentioned above, `push` is intentionally generic, and we can use that to our advantage. `Array.prototype.push` can work on an object just fine, as this example shows.

Note that we don't create an array to store a collection of objects. Instead, we store the collection on the object itself and use `call` on `Array.prototype.push` to trick the method into thinking we are dealing with an array—and it just works, thanks to the way JavaScript allows us to establish the execution context in any way we want.

```
const obj = {
  length: 0,

  addElem(elem) {
    // obj.length is automatically incremented
    // every time an element is added.
    [].push.call(this, elem);
  },
};

// Let's add some empty objects just to illustrate.
obj.addElem({});
```

```
obj.addElem({});
console.log(obj.length); // 2
```

Note that although `obj` is not an array, the method `push` successfully incremented `obj`'s `length` property just like if we were dealing with an actual array.

See also

- Polyfill of `Array.prototype.push` in `core-js` with fixes of this method
- es-shims polyfill of `Array.prototype.push`
- Indexed collections guide
- `Array`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

Array.prototype.reduce()

The **`reduce()`** method of `Array` instances executes a user-supplied "reducer" callback function on each element of the array, in order, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements of the array is a single value.

The first time that the callback is run there is no "return value of the previous calculation". If supplied, an initial value may be used in its place. Otherwise the array element at index 0 is used as the initial value and iteration starts from the next element (index 1 instead of index 0).

Try it

```
const array = [1, 2, 3, 4];

// 0 + 1 + 2 + 3 + 4
const initialValue = 0;
const sumWithInitial = array.reduce(
  (accumulator, currentValue) => accumulator + currentValue,
  initialValue,
);

console.log(sumWithInitial);
// Expected output: 10
```

Syntax

```
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduce()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is `array[0]`.

`currentValue`

The value of the current element. On the first call, its value is `array[0]` if `initialValue` is specified; otherwise its value is `array[1]`.

`currentIndex`

The index position of `currentValue` in the array. On the first call, its value is `0` if `initialValue` is specified, otherwise `1`.

`array`

The array `reduce()` was called upon.

`initialValue` Optional

A value to which `accumulator` is initialized the first time the callback is called. If `initialValue` is specified, `callbackFn` starts executing with the first value in the array as `currentValue`. If `initialValue` is *not* specified, `accumulator` is initialized to the first value in the array, and `callbackFn` starts executing with the second value in the array as `currentValue`. In this case, if the array is empty (so that there's no first value to return as `accumulator`), an error is thrown.

Return value

The value that results from running the "reducer" callback function to completion over the entire array.

Exceptions

`TypeError`

Thrown if the array contains no elements and `initialValue` is not provided.

Description

The `reduce()` method is an iterative method. It runs a "reducer" callback function over all elements in the array, in ascending-index order, and accumulates them into a single value. Every time, the return value of `callbackFn` is passed into `callbackFn` again on next invocation as `accumulator`. The final value of `accumulator` (which is the value returned from `callbackFn` on the final iteration of the array) becomes the return value of `reduce()`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

Unlike other iterative methods, `reduce()` does not accept a `thisArg` argument. `callbackFn` is always called with `undefined` as `this`, which gets substituted with `globalThis` if `callbackFn` is non-strict.

`reduce()` is a central concept in functional programming, where it's not possible to mutate any value, so in order to accumulate all values in an array, one must return a new accumulator value on every iteration. This convention propagates to JavaScript's `reduce()`: you should use spreading or other copying methods where possible to create new arrays and objects as the accumulator, rather than mutating the existing one. If you decided to mutate the accumulator instead of copying it, remember to still return the modified object in the callback, or the next iteration will receive undefined. However, note that copying the accumulator may in turn lead to increased memory usage and degraded performance — see When to not use reduce() for more details. In such cases, to avoid bad performance and unreadable code, it's better to use a `for` loop instead.

The `reduce()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Edge cases

If the array only has one element (regardless of position) and no `initialValue` is provided, or if `initialValue` is provided but the array is empty, the solo value will be returned *without* calling `callbackFn`.

If `initialValue` is provided and the array is not empty, then the reduce method will always invoke the callback function starting at index 0.

If `initialValue` is not provided then the reduce method will act differently for arrays with length larger than 1, equal to 1 and 0, as shown in the following example:

```
const getMax = (a, b) => Math.max(a, b);

// callback is invoked for each element in the array starting at index 0
[1, 100].reduce(getMax, 50); // 100
[50].reduce(getMax, 10); // 50

// callback is invoked once for element at index 1
[1, 100].reduce(getMax); // 100

// callback is not invoked
[50].reduce(getMax); // 50
[].reduce(getMax, 1); // 1

[].reduce(getMax); // TypeError
```

Examples

How reduce() works without an initial value

The code below shows what happens if we call `reduce()` with an array and no initial value.

```
const array = [15, 16, 17, 18, 19];

function reducer(accumulator, currentValue, index) {
  const returns = accumulator + currentValue;
  console.log(
    `accumulator: ${accumulator}, currentValue: ${currentValue}, index: ${index}, returns: ${returns}`,
  );
  return returns;
}

array.reduce(reducer);
```

The callback would be invoked four times, with the arguments and return values in each call being as follows:

	`accumulator`	`currentValue`	`index`	**Return value**
First call	15	16	1	31
Second call	31	17	2	48
Third call	48	18	3	66
Fourth call	66	19	4	85

The `array` parameter never changes through the process — it's always `[15, 16, 17, 18, 19]`. The value returned by `reduce()` would be that of the last callback invocation (`85`).

How reduce() works with an initial value

Here we reduce the same array using the same algorithm, but with an `initialValue` of `10` passed as the second argument to `reduce()`:

```
[15, 16, 17, 18, 19].reduce(
  (accumulator, currentValue) => accumulator + currentValue,
```

```
  10,
);
```

The callback would be invoked five times, with the arguments and return values in each call being as follows:

	`accumulator`	`currentValue`	`index`	**Return value**
First call	`10`	`15`	`0`	`25`
Second call	`25`	`16`	`1`	`41`
Third call	`41`	`17`	`2`	`58`
Fourth call	`58`	`18`	`3`	`76`
Fifth call	`76`	`19`	`4`	`95`

The value returned by `reduce()` in this case would be `95`.

Sum of values in an object array

To sum up the values contained in an array of objects, you **must** supply an `initialValue`, so that each item passes through your function.

```
const objects = [{ x: 1 }, { x: 2 }, { x: 3 }];
const sum = objects.reduce(
  (accumulator, currentValue) => accumulator + currentValue.x,
  0,
);

console.log(sum); // 6
```

Function sequential piping

The `pipe` function takes a sequence of functions and returns a new function. When the new function is called with an argument, the sequence of functions are called in order, which each one receiving the return value of the previous function.

```
const pipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce((acc, fn) => fn(acc), initialValue);

// Building blocks to use for composition
const double = (x) => 2 * x;
const triple = (x) => 3 * x;
const quadruple = (x) => 4 * x;

// Composed functions for multiplication of specific values
const multiply6 = pipe(double, triple);
const multiply9 = pipe(triple, triple);
const multiply16 = pipe(quadruple, quadruple);
const multiply24 = pipe(double, triple, quadruple);

// Usage
multiply6(6); // 36
multiply9(9); // 81
multiply16(16); // 256
multiply24(10); // 240
```

Running promises in sequence

Promise sequencing is essentially function piping demonstrated in the previous section, except done asynchronously.

```
// Compare this with pipe: fn(acc) is changed to acc.then(fn),
// and initialValue is ensured to be a promise
const asyncPipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce((acc, fn) => acc.then(fn), Promise.resolve(initialValue));

// Building blocks to use for composition
const p1 = async (a) => a * 5;
const p2 = async (a) => a * 2;
// The composed functions can also return non-promises, because the values are
// all eventually wrapped in promises
const f3 = (a) => a * 3;
const p4 = async (a) => a * 4;

asyncPipe(p1, p2, f3, p4)(10).then(console.log); // 1200
```

`asyncPipe` can also be implemented using `async`/`await`, which better demonstrates its similarity with `pipe`:

```
const asyncPipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce(async (acc, fn) => fn(await acc), initialValue);
```

Using reduce() with sparse arrays

`reduce()` skips missing elements in sparse arrays, but it does not skip `undefined` values.

```
console.log([1, 2, , 4].reduce((a, b) => a + b)); // 7
console.log([1, 2, undefined, 4].reduce((a, b) => a + b)); // NaN
```

Calling reduce() on non-array objects

The `reduce()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 99, // ignored by reduce() since length is 3
};
console.log(Array.prototype.reduce.call(arrayLike, (x, y) => x + y));
// 9
```

When to not use reduce()

Multipurpose higher-order functions like `reduce()` can be powerful but sometimes difficult to understand, especially for less-experienced JavaScript developers. If code becomes clearer when using other array methods, developers must weigh the readability tradeoff against the other benefits of using `reduce()`.

Note that `reduce()` is always equivalent to a `for...of` loop, except that instead of mutating a variable in the upper scope, we now return the new value for each iteration:

```
const val = array.reduce((acc, cur) => update(acc, cur), initialValue);

// Is equivalent to:
let val = initialValue;
for (const cur of array) {
  val = update(val, cur);
}
```

As previously stated, the reason why people may want to use `reduce()` is to mimic functional programming practices of immutable data. Therefore, developers who uphold the immutability of the accumulator often copy the entire accumulator for each iteration, like this:

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = names.reduce((allNames, name) => {
  const currCount = Object.hasOwn(allNames, name) ? allNames[name] : 0;
  return {
    ...allNames,
    [name]: currCount + 1,
  };
}, {});
```

This code is ill-performing, because each iteration has to copy the entire `allNames` object, which could be big, depending how many unique names there are. This code has worst-case `O(N^2)` performance, where `N` is the length of `names`.

A better alternative is to *mutate* the `allNames` object on each iteration. However, if `allNames` gets mutated anyway, you may want to convert the `reduce()` to a `for` loop instead, which is much clearer:

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = names.reduce((allNames, name) => {
  const currCount = allNames[name] ?? 0;
  allNames[name] = currCount + 1;
  // return allNames, otherwise the next iteration receives undefined
  return allNames;
}, Object.create(null));
```

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = Object.create(null);
for (const name of names) {
  const currCount = countedNames[name] ?? 0;
  countedNames[name] = currCount + 1;
}
```

Therefore, if your accumulator is an array or an object and you are copying the array or object on each iteration, you may accidentally introduce quadratic complexity into your code, causing performance to quickly degrade on large data. This has happened in real-world code — see for example Making Tanstack Table 1000x faster with a 1 line change.

Some of the acceptable use cases of `reduce()` are given above (most notably, summing an array, promise sequencing, and function piping). There are other cases where better alternatives than `reduce()` exist.

- Flattening an array of arrays. Use `flat()` instead.

  ```
  const flattened = array.reduce((acc, cur) => acc.concat(cur), []);
  ```

```
const flattened = array.flat();
```

- Grouping objects by a property. Use `Object.groupBy()` instead.

```
const groups = array.reduce((acc, obj) => {
  const key = obj.name;
  const curGroup = acc[key] ?? [];
  return { ...acc, [key]: [...curGroup, obj] };
}, {});
```

```
const groups = Object.groupBy(array, (obj) => obj.name);
```

- Concatenating arrays contained in an array of objects. Use `flatMap()` instead.

```
const friends = [
  { name: "Anna", books: ["Bible", "Harry Potter"] },
  { name: "Bob", books: ["War and peace", "Romeo and Juliet"] },
  { name: "Alice", books: ["The Lord of the Rings", "The Shining"] },
];
const allBooks = friends.reduce((acc, cur) => [...acc, ...cur.books], []);
```

```
const allBooks = friends.flatMap((person) => person.books);
```

- Removing duplicate items in an array. Use `Set` and `Array.from()` instead.

```
const uniqArray = array.reduce(
  (acc, cur) => (acc.includes(cur) ? acc : [...acc, cur]),
  [],
);
```

```
const uniqArray = Array.from(new Set(array));
```

- Eliminating or adding elements in an array. Use `flatMap()` instead.

```
// Takes an array of numbers and splits perfect squares into its square roots
const roots = array.reduce((acc, cur) => {
  if (cur < 0) return acc;
  const root = Math.sqrt(cur);
  if (Number.isInteger(root)) return [...acc, root, root];
  return [...acc, cur];
}, []);
```

```
const roots = array.flatMap((val) => {
  if (val < 0) return [];
  const root = Math.sqrt(val);
  if (Number.isInteger(root)) return [root, root];
  return [val];
});
```

  If you are only eliminating elements from an array, you also can use `filter()`.

- Searching for elements or testing if elements satisfy a condition. Use `find()` and `findIndex()`, or `some()` and `every()` instead. These methods have the additional benefit that they return as soon as the result is certain, without iterating the entire array.

```
const allEven = array.reduce((acc, cur) => acc && cur % 2 === 0, true);
```

```
const allEven = array.every((val) => val % 2 === 0);
```

In cases where `reduce()` is the best choice, documentation and semantic variable naming can help mitigate readability drawbacks.

See also

- Polyfill of `Array.prototype.reduce` in `core-js`
- es-shims polyfill of `Array.prototype.reduce`
- Indexed collections guide
- `Array`
- `Array.prototype.map()`
- `Array.prototype.flat()`
- `Array.prototype.flatMap()`
- `Array.prototype.reduceRight()`
- `TypedArray.prototype.reduce()`
- `Object.groupBy()`
- `Map.groupBy()`

Array.prototype.reduceRight()

The **`reduceRight()`** method of `Array` instances applies a function against an accumulator and each value of the array (from right-to-left) to reduce it to a single value.

See also `Array.prototype.reduce()` for left-to-right.

Try it

```
const array = [
  [0, 1],
  [2, 3],
  [4, 5],
];

const result = array.reduceRight((accumulator, currentValue) =>
  accumulator.concat(currentValue),
);

console.log(result);
// Expected output: Array [4, 5, 2, 3, 0, 1]
```

Syntax

```
reduceRight(callbackFn)
reduceRight(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduceRight()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is the last element of the array.

`currentValue`

The value of the current element. On the first call, its value is the last element if `initialValue` is specified; otherwise its value is the second-to-last element.

`currentIndex`

The index position of `currentValue` in the array. On the first call, its value is `array.length - 1` if `initialValue` is specified, otherwise `array.length - 2`.

`array`

The array `reduceRight()` was called upon.

`initialValue` Optional

Value to use as accumulator to the first call of the `callbackFn`. If no initial value is supplied, the last element in the array will be used and skipped. Calling `reduceRight()` on an empty array without an initial value creates a `TypeError`.

Return value

The value that results from the reduction.

Description

The `reduceRight()` method is an iterative method. It runs a "reducer" callback function over all elements in the array, in descending-index order, and accumulates them into a single value. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

Unlike other iterative methods, `reduceRight()` does not accept a `thisArg` argument. `callbackFn` is always called with `undefined` as `this`, which gets substituted with `globalThis` if `callbackFn` is non-strict.

The `reduceRight()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

All caveats about `reduce` discussed in when to not use reduce() apply to `reduceRight` as well. Because JavaScript has no lazy evaluation semantics, there is no performance difference between `reduce` and `reduceRight`.

Examples

How reduceRight() works without an initial value

The call to the reduceRight `callbackFn` would look something like this:

```
arr.reduceRight((accumulator, currentValue, index, array) => {
  // …
});
```

The first time the function is called, the `accumulator` and `currentValue` can be one of two values. If an `initialValue` was provided in the call to `reduceRight`, then `accumulator` will be equal to `initialValue` and `currentValue` will be equal to the last value in the array. If no `initialValue` was provided, then `accumulator` will be equal to the last value in the array and `currentValue` will be equal to the second-to-last value.

If the array is empty and no `initialValue` was provided, `TypeError` would be thrown. If the array has only one element (regardless of position) and no `initialValue` was provided, or if `initialValue` is provided but the array is empty, the solo value would be returned without calling `callbackFn`.

Some example run-throughs of the function would look like this:

```
[0, 1, 2, 3, 4].reduceRight(
  (accumulator, currentValue, index, array) => accumulator + currentValue,
);
```

The callback would be invoked four times, with the arguments and return values in each call being as follows:

	`accumulator`	`currentValue`	`index`	**Return value**
First call	`4`	`3`	`3`	`7`
Second call	`7`	`2`	`2`	`9`
Third call	`9`	`1`	`1`	`10`
Fourth call	`10`	`0`	`0`	`10`

The `array` parameter never changes through the process — it's always `[0, 1, 2, 3, 4]`. The value returned by `reduceRight` would be that of the last callback invocation (`10`).

How reduceRight() works with an initial value

Here we reduce the same array using the same algorithm, but with an `initialValue` of `10` passed as the second argument to `reduceRight()`:

```
[0, 1, 2, 3, 4].reduceRight(
  (accumulator, currentValue, index, array) => accumulator + currentValue,
  10,
);
```

	`accumulator`	`currentValue`	`index`	**Return value**
First call	`10`	`4`	`4`	`14`
Second call	`14`	`3`	`3`	`17`
Third call	`17`	`2`	`2`	`19`
Fourth call	`19`	`1`	`1`	`20`
Fifth call	`20`	`0`	`0`	`20`

The value returned by `reduceRight` this time would be, of course, `20`.

Sum up all values within an array

```
const sum = [0, 1, 2, 3].reduceRight((a, b) => a + b);
// sum is 6
```

Run a list of asynchronous functions with callbacks in series each passing their results to the next

```
const waterfall =
  (...functions) =>
  (callback, ...args) =>
    functions.reduceRight(
      (composition, fn) =>
        (...results) =>
          fn(composition, ...results),
      callback,
    )(...args);

const randInt = (max) => Math.floor(Math.random() * max);

const add5 = (callback, x) => {
  setTimeout(callback, randInt(1000), x + 5);
};
const mul3 = (callback, x) => {
  setTimeout(callback, randInt(1000), x * 3);
};
const sub2 = (callback, x) => {
  setTimeout(callback, randInt(1000), x - 2);
};
const split = (callback, x) => {
  setTimeout(callback, randInt(1000), x, x);
};
const add = (callback, x, y) => {
  setTimeout(callback, randInt(1000), x + y);
};
const div4 = (callback, x) => {
  setTimeout(callback, randInt(1000), x / 4);
};

const computation = waterfall(add5, mul3, sub2, split, add, div4);
computation(console.log, 5); // Logs 14
```

```
// same as:

const computation2 = (input, callback) => {
  const f6 = (x) => div4(callback, x);
  const f5 = (x, y) => add(f6, x, y);
  const f4 = (x) => split(f5, x);
  const f3 = (x) => sub2(f4, x);
  const f2 = (x) => mul3(f3, x);
  add5(f2, input);
};
```

Difference between reduce and reduceRight

```
const a = ["1", "2", "3", "4", "5"];
const left = a.reduce((prev, cur) => prev + cur);
const right = a.reduceRight((prev, cur) => prev + cur);

console.log(left); // "12345"
console.log(right); // "54321"
```

Defining composable functions

Function composition is a mechanism for combining functions, in which the output of each function is passed into the next one, and the output of the last function is the final result. In this example we use `reduceRight()` to implement function composition.

See also Function composition on Wikipedia.

```
const compose =
  (...args) =>
  (value) =>
    args.reduceRight((acc, fn) => fn(acc), value);

// Increment passed number
const inc = (n) => n + 1;

// Doubles the passed value
const double = (n) => n * 2;

// using composition function
console.log(compose(double, inc)(2)); // 6

// using composition function
console.log(compose(inc, double)(2)); // 5
```

Using reduceRight() with sparse arrays

`reduceRight()` skips missing elements in sparse arrays, but it does not skip `undefined` values.

```
console.log([1, 2, , 4].reduceRight((a, b) => a + b)); // 7
console.log([1, 2, undefined, 4].reduceRight((a, b) => a + b)); // NaN
```

Calling reduceRight() on non-array objects

The `reduceRight()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 99, // ignored by reduceRight() since length is 3
};
console.log(Array.prototype.reduceRight.call(arrayLike, (x, y) => x - y));
// -1, which is 4 - 3 - 2
```

See also

- Polyfill of `Array.prototype.reduceRight` in `core-js`
- es-shims polyfill of `Array.prototype.reduceRight`
- Indexed collections guide
- `Array`
- `Array.prototype.map()`
- `Array.prototype.flat()`
- `Array.prototype.flatMap()`
- `Array.prototype.reduce()`
- `TypedArray.prototype.reduceRight()`
- `Object.groupBy()`
- `Map.groupBy()`

Array.prototype.reverse()

The **`reverse()`** method of `Array` instances reverses an array *in place* and returns the reference to the same array, the first array element now becoming the last, and the last array element becoming the first. In other words, elements order in the array will be turned towards the direction opposite to that previously stated.

To reverse the elements in an array without mutating the original array, use `toReversed()`.

Try it

```
const array = ["one", "two", "three"];
console.log("array:", array);
// Expected output: "array:" Array ["one", "two", "three"]

const reversed = array.reverse();
console.log("reversed:", reversed);
// Expected output: "reversed:" Array ["three", "two", "one"]

// Careful: reverse is destructive -- it changes the original array.
console.log("array:", array);
// Expected output: "array:" Array ["three", "two", "one"]
```

Syntax

```
reverse()
```

Parameters

None.

Return value

The reference to the original array, now reversed. Note that the array is reversed *in place*, and no copy is made.

Description

The `reverse()` method transposes the elements of the calling array object in place, mutating the array, and returning a reference to the array.

The `reverse()` method preserves empty slots. If the source array is sparse, the empty slots' corresponding new indices are deleted and also become empty slots.

The `reverse()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Reversing the elements in an array

The following example creates an array `items`, containing three elements, then reverses the array. The call to `reverse()` returns a reference to the reversed array `items`.

```
const items = [1, 2, 3];
console.log(items); // [1, 2, 3]

items.reverse();
console.log(items); // [3, 2, 1]
```

The reverse() method returns the reference to the same array

The `reverse()` method returns reference to the original array, so mutating the returned array will mutate the original array as well.

```
const numbers = [3, 2, 4, 1, 5];
const reversed = numbers.reverse();
// numbers and reversed are both in reversed order [5, 1, 4, 2, 3]
reversed[0] = 5;
console.log(numbers[0]); // 5
```

In case you want `reverse()` to not mutate the original array, but return a shallow-copied array like other array methods (e.g., `map()`) do, use the `toReversed()` method. Alternatively, you can do a shallow copy before calling `reverse()`, using the spread syntax or `Array.from()`.

```
const numbers = [3, 2, 4, 1, 5];
// [...numbers] creates a shallow copy, so reverse() does not mutate the original
const reverted = [...numbers].reverse();
reverted[0] = 5;
console.log(numbers[0]); // 3
```

Using reverse() on sparse arrays

Sparse arrays remain sparse after calling `reverse()`. Empty slots are copied over to their respective new indices as empty slots.

```
console.log([1, , 3].reverse()); // [3, empty, 1]
console.log([1, , 3, 4].reverse()); // [4, 3, empty, 1]
```

Calling reverse() on non-array objects

The `reverse()` method reads the `length` property of `this`. It then visits each property having an integer key between `0` and `length / 2`, and swaps the two corresponding indices on both ends, deleting any destination property for which the source property did not exist.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
  3: 33, // ignored by reverse() since length is 3
};
console.log(Array.prototype.reverse.call(arrayLike));
// { 0: 4, 3: 33, length: 3, unrelated: 'foo' }
// The index 2 is deleted because there was no index 0 present originally
// The index 3 is unchanged since the length is 3
```

See also

- Polyfill of `Array.prototype.reverse` in `core-js`
- es-shims polyfill of `Array.prototype.reverse`
- Indexed collections guide
- `Array`
- `Array.prototype.join()`
- `Array.prototype.sort()`
- `Array.prototype.toReversed()`
- `TypedArray.prototype.reverse()`

Array.prototype.shift()

The **`shift()`** method of `Array` instances removes the **first** element from an array and returns that removed element. This method changes the length of the array.

Try it

```
const array = [1, 2, 3];

const firstElement = array.shift();

console.log(array);
// Expected output: Array [2, 3]

console.log(firstElement);
// Expected output: 1
```

Syntax

```
shift()
```

Parameters

None.

Return value

The removed element from the array; `undefined` if the array is empty.

Description

The `shift()` method shifts all values to the left by 1 and decrements the length by 1, resulting in the first element being removed. If the `length` property is 0, `undefined` is returned.

The `pop()` method has similar behavior to `shift()`, but applied to the last element in an array.

The `shift()` method is a mutating method. It changes the length and the content of `this`. In case you want the value of `this` to be the same, but return a new array with the first element removed, you can use `arr.slice(1)` instead.

The `shift()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Removing an element from an array

The following code displays the `myFish` array before and after removing its first element. It also displays the removed element:

```
const myFish = ["angel", "clown", "mandarin", "surgeon"];

console.log("myFish before:", myFish);
// myFish before: ['angel', 'clown', 'mandarin', 'surgeon']

const shifted = myFish.shift();

console.log("myFish after:", myFish);
// myFish after: ['clown', 'mandarin', 'surgeon']

console.log("Removed this element:", shifted);
// Removed this element: angel
```

Using shift() method in while loop

The shift() method is often used in condition inside while loop. In the following example every iteration will remove the next element from an array, until it is empty:

```
const names = ["Andrew", "Tyrone", "Paul", "Maria", "Gayatri"];

while (typeof (i = names.shift()) !== "undefined") {
  console.log(i);
}
// Andrew, Tyrone, Paul, Maria, Gayatri
```

Calling shift() on non-array objects

The `shift()` method reads the `length` property of `this`. If the normalized length is 0, `length` is set to `0` again (whereas it may be negative or `undefined` before). Otherwise, the property at `0` is returned, and the rest of the properties are shifted left by one. The property at `length - 1` is deleted, and the `length` property is decremented by one.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
console.log(Array.prototype.shift.call(arrayLike));
// undefined, because it is an empty slot
console.log(arrayLike);
// { '1': 4, length: 2, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.shift.call(plainObj);
console.log(plainObj);
// { length: 0 }
```

See also

- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

Array.prototype.slice()

The **`slice()`** method of `Array` instances returns a shallow copy of a portion of an array into a new array object selected from `start` to `end` (`end` not included) where `start` and `end` represent the index of items in that array. The original array will not be modified.

Try it

```
const animals = ["ant", "bison", "camel", "duck", "elephant"];

console.log(animals.slice(2));
// Expected output: Array ["camel", "duck", "elephant"]

console.log(animals.slice(2, 4));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice(1, 5));
// Expected output: Array ["bison", "camel", "duck", "elephant"]

console.log(animals.slice(-2));
// Expected output: Array ["duck", "elephant"]

console.log(animals.slice(2, -1));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice());
// Expected output: Array ["ant", "bison", "camel", "duck", "elephant"]
```

Syntax

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length` or `start` is omitted, `0` is used.
- If `start >= array.length`, an empty array is returned.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

- Negative index counts back from the end of the array — if `-array.length <= end < 0`, `end + array.length` is used.
- If `end < -array.length`, `0` is used.
- If `end >= array.length` or `end` is omitted or `undefined`, `array.length` is used, causing all elements until the end to be extracted.
- If `end` implies a position before or at the position that `start` implies, an empty array is returned.

Return value

A new array containing the extracted elements.

Description

The `slice()` method is a copying method. It does not alter `this` but instead returns a shallow copy that contains some of the same elements as the ones from the original array.

The `slice()` method preserves empty slots. If the sliced portion is sparse, the returned array is sparse as well.

The `slice()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Return a portion of an existing array

```
const fruits = ["Banana", "Orange", "Lemon", "Apple", "Mango"];
const citrus = fruits.slice(1, 3);

// fruits contains ['Banana', 'Orange', 'Lemon', 'Apple', 'Mango']
// citrus contains ['Orange','Lemon']
```

In this example, `slice(1, 3)` extracts elements from index `1` up to, but not including, index `3`, resulting in a new array `['Orange', 'Lemon']`.

Omitting the end parameter

```
const fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

const tropical = fruits.slice(2);
console.log(tropical); // ['Orange', 'Mango', 'Pineapple']
```

In this example, `slice(2)` extracts elements from index `2` to the end of the array.

Using negative indices

```
const fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

const lastTwo = fruits.slice(-2);
console.log(lastTwo); // ['Mango', 'Pineapple']
```

In this example, `slice(-2)` extracts the last two elements of the array. When using a negative index with the `slice` method, negative indices are counted from the end of the array, starting at `-1` for the last element, `-2` for the second-to-last element, and so on. The negative index `-2` itself is included because it is the starting point of the extraction.

```
|     |     |     |     |     |
|  S  |  L  |  I  |  C  |  E  |
|     |     |     |     |     |
  -5    -4    -3    -2    -1

<--- read from reverse
```

Using a positive start index and a negative end index

```
const fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

// Using positive start index and negative end index
const sliceExample = fruits.slice(1, -1);
console.log(sliceExample); // ['Banana', 'Orange', 'Mango']
```

In this example, `slice(1, -1)` starts extracting from index `1` and goes up to, but does not include, the element at index `-1` (which is the last element). This results in a new array with `['Banana', 'Orange', 'Mango']`. The `slice` method always excludes the element at the final index specified, regardless of whether it is positive or negative.

```
read from start --->

   0     1     2     3     4
|     |     |     |     |     |
|  S  |  L  |  I  |  C  |  E  |
```

```
|    |    |    |    |    |
  -5   -4   -3   -2   -1

<--- read from reverse
```

Using slice with arrays of objects

In the following example, `slice` creates a new array, `newCar`, from `myCar`. Both include a reference to the object `myHonda`. When the color of `myHonda` is changed to purple, both arrays reflect the change.

```
// Using slice, create newCar from myCar.
const myHonda = {
  color: "red",
  wheels: 4,
  engine: { cylinders: 4, size: 2.2 },
};
const myCar = [myHonda, 2, "cherry condition", "purchased 1997"];
const newCar = myCar.slice(0, 2);

console.log("myCar =", myCar);
console.log("newCar =", newCar);
console.log("myCar[0].color =", myCar[0].color);
console.log("newCar[0].color =", newCar[0].color);

// Change the color of myHonda.
myHonda.color = "purple";
console.log("The new color of my Honda is", myHonda.color);

console.log("myCar[0].color =", myCar[0].color);
console.log("newCar[0].color =", newCar[0].color);
```

This script writes:

```
myCar = [
  { color: 'red', wheels: 4, engine: { cylinders: 4, size: 2.2 } },
  2,
  'cherry condition',
  'purchased 1997'
]
newCar = [ { color: 'red', wheels: 4, engine: { cylinders: 4, size: 2.2 } }, 2 ]
myCar[0].color = red
newCar[0].color = red
The new color of my Honda is purple
myCar[0].color = purple
newCar[0].color = purple
```

Calling slice() on non-array objects

The `slice()` method reads the `length` property of `this`. It then reads the integer-keyed properties from `start` to `end` and defines them on a newly created array.

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 33, // ignored by slice() since length is 3
};
console.log(Array.prototype.slice.call(arrayLike, 1, 3));
// [ 3, 4 ]
```

Using slice() to convert array-like objects to arrays

The `slice()` method is often used with `bind()` and `call()` to create a utility method that converts an array-like object into an array.

```
// slice() is called with `this` passed as the first argument
const slice = Function.prototype.call.bind(Array.prototype.slice);

function list() {
  return slice(arguments);
}

const listResult = list(1, 2, 3); // [1, 2, 3]
```

Using slice() on sparse arrays

The array returned from `slice()` may be sparse if the source is sparse.

```
console.log([1, 2, , 4, 5].slice(1, 4)); // [2, empty, 4]
```

See also

- Polyfill of `Array.prototype.slice` in `core-js`
- es-shims polyfill of `Array.prototype.slice`

- Indexed collections guide
- `Array`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`
- `TypedArray.prototype.slice()`
- `String.prototype.slice()`

Array.prototype.some()

The **`some()`** method of `Array` instances returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`.

Try it

```
const array = [1, 2, 3, 4, 5];

// Checks whether an element is even
const even = (element) => element % 2 === 0;

console.log(array.some(even));
// Expected output: true
```

Syntax

```
some(callbackFn)
some(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `some()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`false` unless `callbackFn` returns a truthy value for an array element, in which case `true` is immediately returned.

Description

The `some()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array, until the `callbackFn` returns a truthy value. If such an element is found, `some()` immediately returns `true` and stops iterating through the array. Otherwise, if `callbackFn` returns a falsy value for all elements, `some()` returns `false`. Read the iterative methods section for more information about how these methods work in general.

`some()` acts like the "there exists" quantifier in mathematics. In particular, for an empty array, it returns `false` for any condition.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

`some()` does not mutate the array on which it is called, but the function provided as `callbackFn` can. Note, however, that the length of the array is saved *before* the first invocation of `callbackFn`. Therefore:

- `callbackFn` will not visit any elements added beyond the array's initial length when the call to `some()` began.
- Changes to already-visited indexes do not cause `callbackFn` to be invoked on them again.
- If an existing, yet-unvisited element of the array is changed by `callbackFn`, its value passed to the `callbackFn` will be the value at the time that element gets visited. Deleted elements are not visited.

> **Warning:** Concurrent modifications of the kind described above frequently lead to hard-to-understand code and are generally to be avoided (except in special cases).

The `some()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Testing value of array elements

The following example tests whether any element in the array is bigger than 10.

```
function isBiggerThan10(element, index, array) {
  return element > 10;
}

[2, 5, 8, 1, 4].some(isBiggerThan10); // false
[12, 5, 8, 1, 4].some(isBiggerThan10); // true
```

Testing array elements using arrow functions

Arrow functions provide a shorter syntax for the same test.

```
[2, 5, 8, 1, 4].some((x) => x > 10); // false
[12, 5, 8, 1, 4].some((x) => x > 10); // true
```

Checking whether a value exists in an array

To mimic the function of the `includes()` method, this custom function returns `true` if the element exists in the array:

```
const fruits = ["apple", "banana", "mango", "guava"];

function checkAvailability(arr, val) {
  return arr.some((arrVal) => val === arrVal);
}

checkAvailability(fruits, "grapefruit"); // false
checkAvailability(fruits, "banana"); // true
```

Converting any value to Boolean

```
const TRUTHY_VALUES = [true, "true", 1];

function getBoolean(value) {
  if (typeof value === "string") {
    value = value.toLowerCase().trim();
  }

  return TRUTHY_VALUES.some((t) => t === value);
}

getBoolean(false); // false
getBoolean("false"); // false
getBoolean(1); // true
getBoolean("true"); // true
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `some()` to check whether the array is strictly increasing.

```
const numbers = [3, -1, 1, 4, 1, 5];
const isIncreasing = !numbers
  .filter((num) => num > 0)
  .some((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx === 0) return false;
    return num <= arr[idx - 1];
  });
console.log(isIncreasing); // false
```

Using some() on sparse arrays

`some()` will not run its predicate on empty slots.

```
console.log([1, , 3].some((x) => x === undefined)); // false
console.log([1, , 1].some((x) => x !== 1)); // false
console.log([1, undefined, 1].some((x) => x !== 1)); // true
```

Calling some() on non-array objects

The `some()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length` until they all have been accessed or `callbackFn` returns `true`.

```
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: 3, // ignored by some() since length is 3
};
```

```
console.log(Array.prototype.some.call(arrayLike, (x) => typeof x === "number"));
// false
```

See also

- Polyfill of `Array.prototype.some` in `core-js`
- es-shims polyfill of `Array.prototype.some`
- Indexed collections guide
- `Array`
- `Array.prototype.every()`
- `Array.prototype.forEach()`
- `Array.prototype.find()`
- `Array.prototype.includes()`
- `TypedArray.prototype.some()`

Array.prototype.sort()

The **`sort()`** method of `Array` instances sorts the elements of an array *in place* and returns the reference to the same array, now sorted. The default sort order is ascending, built upon converting the elements into strings, then comparing their sequences of UTF-16 code unit values.

The time and space complexity of the sort cannot be guaranteed as it depends on the implementation.

To sort the elements in an array without mutating the original array, use `toSorted()`.

Try it

```
const months = ["March", "Jan", "Feb", "Dec"];
months.sort();
console.log(months);
// Expected output: Array ["Dec", "Feb", "Jan", "March"]

const array = [1, 30, 4, 21, 100000];
array.sort();
console.log(array);
// Expected output: Array [1, 100000, 21, 30, 4]
```

Syntax

```
sort()
sort(compareFn)
```

Parameters

`compareFn` Optional

A function that determines the order of the elements. The function is called with the following arguments:

`a`

The first element for comparison. Will never be `undefined`.

`b`

The second element for comparison. Will never be `undefined`.

It should return a number where:

- A negative value indicates that `a` should come before `b`.
- A positive value indicates that `a` should come after `b`.
- Zero or `NaN` indicates that `a` and `b` are considered equal.

To memorize this, remember that `(a, b) => a - b` sorts numbers in ascending order.

If omitted, the array elements are converted to strings, then sorted according to each character's Unicode code point value.

Return value

The reference to the original array, now sorted. Note that the array is sorted *in place*, and no copy is made.

Description

If `compareFn` is not supplied, all non-`undefined` array elements are sorted by converting them to strings and comparing strings in UTF-16 code units order. For example, "banana" comes before "cherry". In a numeric sort, 9 comes before 80, but because numbers are converted to strings, "80" comes before "9" in the Unicode order. All `undefined` elements are sorted to the end of the array.

The `sort()` method preserves empty slots. If the source array is sparse, the empty slots are moved to the end of the array, and always come after all the `undefined`.

> **Note:** In UTF-16, Unicode characters above `\uFFFF` are encoded as two surrogate code units, of the range `\uD800` - `\uDFFF`. The value of each code unit is taken separately into account for the comparison. Thus the character formed by the surrogate pair `\uD855\uDE51` will be sorted before the character `\uFF3A`.

If `compareFn` is supplied, all non-`undefined` array elements are sorted according to the return value of the compare function (all `undefined` elements are sorted to the end of the array, with no call to `compareFn`).

`compareFn(a, b)` return value	sort order
> 0	sort `a` after `b`, e.g., `[b, a]`
< 0	sort `a` before `b`, e.g., `[a, b]`
=== 0	keep original order of `a` and `b`

So, the compare function has the following form:

```
function compareFn(a, b) {
  if (a is less than b by some ordering criterion) {
    return -1;
  } else if (a is greater than b by the ordering criterion) {
    return 1;
  }
  // a must be equal to b
  return 0;
}
```

More formally, the comparator is expected to have the following properties, in order to ensure proper sort behavior:

- *Pure*: The comparator does not mutate the objects being compared or any external state. (This is important because there's no guarantee *when* and *how* the comparator will be called, so any particular call should not produce visible effects to the outside.)
- *Stable*: The comparator returns the same result with the same pair of input.
- *Reflexive*: `compareFn(a, a) === 0`.
- *Anti-symmetric*: `compareFn(a, b)` and `compareFn(b, a)` must both be `0` or have opposite signs.
- *Transitive*: If `compareFn(a, b)` and `compareFn(b, c)` are both positive, zero, or negative, then `compareFn(a, c)` has the same positivity as the previous two.

A comparator conforming to the constraints above will always be able to return all of `1`, `0`, and `-1`, or consistently return `0`. For example, if a comparator only returns `1` and `0`, or only returns `0` and `-1`, it will not be able to sort reliably because *anti-symmetry* is broken. A comparator that always returns `0` will cause the array to not be changed at all, but is reliable nonetheless.

The default lexicographic comparator satisfies all constraints above.

To compare numbers instead of strings, the compare function can subtract `b` from `a`. The following function will sort the array in ascending order (if it doesn't contain `NaN`):

```
function compareNumbers(a, b) {
  return a - b;
}
```

The `sort()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Creating, displaying, and sorting an array

The following example creates four arrays and displays the original array, then the sorted arrays. The numeric arrays are sorted without a compare function, then sorted using one.

```
const stringArray = ["Blue", "Humpback", "Beluga"];
const numberArray = [40, 1, 5, 200];
const numericStringArray = ["80", "9", "700"];
const mixedNumericArray = ["80", "9", "700", 40, 1, 5, 200];

function compareNumbers(a, b) {
  return a - b;
}

stringArray.join(); // 'Blue,Humpback,Beluga'
stringArray.sort(); // ['Beluga', 'Blue', 'Humpback']

numberArray.join(); // '40,1,5,200'
numberArray.sort(); // [1, 200, 40, 5]
numberArray.sort(compareNumbers); // [1, 5, 40, 200]

numericStringArray.join(); // '80,9,700'
numericStringArray.sort(); // ['700', '80', '9']
numericStringArray.sort(compareNumbers); // ['9', '80', '700']

mixedNumericArray.join(); // '80,9,700,40,1,5,200'
mixedNumericArray.sort(); // [1, 200, 40, 5, '700', '80', '9']
mixedNumericArray.sort(compareNumbers); // [1, 5, '9', 40, '80', 200, '700']
```

Sorting array of objects

Arrays of objects can be sorted by comparing the value of one of their properties.

```
const items = [
  { name: "Edward", value: 21 },
  { name: "Sharpe", value: 37 },
  { name: "And", value: 45 },
  { name: "The", value: -12 },
  { name: "Magnetic", value: 13 },
  { name: "Zeros", value: 37 },
];

// sort by value
items.sort((a, b) => a.value - b.value);

// sort by name
items.sort((a, b) => {
  const nameA = a.name.toUpperCase(); // ignore upper and lowercase
  const nameB = b.name.toUpperCase(); // ignore upper and lowercase
  if (nameA < nameB) {
    return -1;
  }
  if (nameA > nameB) {
    return 1;
  }

  // names must be equal
  return 0;
});
```

Sorting non-ASCII characters

For sorting strings with non-ASCII characters, i.e., strings with accented characters (e, é, è, a, ä, etc.), strings from languages other than English, use `String.prototype.localeCompare()`. This function can compare those characters so they appear in the right order.

```
const items = ["réservé", "premier", "communiqué", "café", "adieu", "éclair"];
items.sort((a, b) => a.localeCompare(b));

// items is ['adieu', 'café', 'communiqué', 'éclair', 'premier', 'réservé']
```

Sorting with map

The `compareFn` can be invoked multiple times per element within the array. Depending on the `compareFn`'s nature, this may yield a high overhead. The more work a `compareFn` does and the more elements there are to sort, it may be more efficient to use `map()` for sorting. The idea is to traverse the array once to extract the actual values used for sorting into a temporary array, sort the temporary array, and then traverse the temporary array to achieve the right order.

```
// the array to be sorted
const data = ["delta", "alpha", "charlie", "bravo"];

// temporary array holds objects with position and sort-value
const mapped = data.map((v, i) => ({ i, value: someSlowOperation(v) }));

// sorting the mapped array containing the reduced values
mapped.sort((a, b) => {
  if (a.value > b.value) {
    return 1;
  }
  if (a.value < b.value) {
    return -1;
  }
  return 0;
});

const result = mapped.map((v) => data[v.i]);
```

There is an open source library available called mapsort which applies this approach.

sort() returns the reference to the same array

The `sort()` method returns a reference to the original array, so mutating the returned array will mutate the original array as well.

```
const numbers = [3, 1, 4, 1, 5];
const sorted = numbers.sort((a, b) => a - b);
// numbers and sorted are both [1, 1, 3, 4, 5]
sorted[0] = 10;
console.log(numbers[0]); // 10
```

In case you want `sort()` to not mutate the original array, but return a shallow-copied array like other array methods (e.g., `map()`) do, use the `toSorted()` method. Alternatively, you can do a shallow copy before calling `sort()`, using the spread syntax or `Array.from()`.

```
const numbers = [3, 1, 4, 1, 5];
// [...numbers] creates a shallow copy, so sort() does not mutate the original
const sorted = [...numbers].sort((a, b) => a - b);
sorted[0] = 10;
console.log(numbers[0]); // 3
```

Sort stability

Since version 10 (or ECMAScript 2019), the specification dictates that `Array.prototype.sort` is stable.

For example, say you had a list of students alongside their grades. Note that the list of students is already pre-sorted by name in alphabetical order:

```
const students = [
  { name: "Alex", grade: 15 },
  { name: "Devlin", grade: 15 },
  { name: "Eagle", grade: 13 },
  { name: "Sam", grade: 14 },
];
```

After sorting this array by `grade` in ascending order:

```
students.sort((firstItem, secondItem) => firstItem.grade - secondItem.grade);
```

The `students` variable will then have the following value:

```
[
  { name: "Eagle", grade: 13 },
  { name: "Sam", grade: 14 },
  { name: "Alex", grade: 15 }, // original maintained for similar grade (stable sorting)
  { name: "Devlin", grade: 15 }, // original maintained for similar grade (stable sorting)
];
```

It's important to note that students that have the same grade (for example, Alex and Devlin), will remain in the same order as before calling the sort. This is what a stable sorting algorithm guarantees.

Before version 10 (or ECMAScript 2019), sort stability was not guaranteed, meaning that you could end up with the following:

```
[
  { name: "Eagle", grade: 13 },
  { name: "Sam", grade: 14 },
  { name: "Devlin", grade: 15 }, // original order not maintained
  { name: "Alex", grade: 15 }, // original order not maintained
];
```

Sorting with non-well-formed comparator

If a comparing function does not satisfy all of purity, stability, reflexivity, anti-symmetry, and transitivity rules, as explained in the description, the program's behavior is not well-defined.

For example, consider this code:

```
const arr = [3, 1, 4, 1, 5, 9];
const compareFn = (a, b) => (a > b ? 1 : 0);
arr.sort(compareFn);
```

The `compareFn` function here is not well-formed, because it does not satisfy anti-symmetry: if `a > b`, it returns `1`; but by swapping `a` and `b`, it returns `0` instead of a negative value. Therefore, the resulting array will be different across engines. For example, V8 (used by Chrome, Node.js, etc.) and JavaScriptCore (used by Safari) would not sort the array at all and return `[3, 1, 4, 1, 5, 9]`, while SpiderMonkey (used by Firefox) will return the array sorted ascendingly, as `[1, 1, 3, 4, 5, 9]`.

However, if the `compareFn` function is changed slightly so that it returns `-1` or `0`:

```
const arr = [3, 1, 4, 1, 5, 9];
const compareFn = (a, b) => (a > b ? -1 : 0);
arr.sort(compareFn);
```

Then V8 and JavaScriptCore sorts it descendingly, as `[9, 5, 4, 3, 1, 1]`, while SpiderMonkey returns it as-is: `[3, 1, 4, 1, 5, 9]`.

Due to this implementation inconsistency, you are always advised to make your comparator well-formed by following the five constraints.

Using sort() on sparse arrays

Empty slots are moved to the end of the array.

```
console.log(["a", "c", , "b"].sort()); // ['a', 'b', 'c', empty]
console.log([, undefined, "a", "b"].sort()); // ["a", "b", undefined, empty]
```

Calling sort() on non-array objects

The `sort()` method reads the `length` property of `this`. It then collects all existing integer-keyed properties in the range of `0` to `length - 1`, sorts them, and writes them back. If there are missing properties in the range, the corresponding trailing properties are deleted, as if the non-existent properties are sorted towards the end.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
};
```

```
console.log(Array.prototype.sort.call(arrayLike));
// { '0': 4, '1': 5, length: 3, unrelated: 'foo' }
```

See also

- Polyfill of `Array.prototype.sort` with modern behavior like stable sort in `core-js`
- Indexed collections guide
- `Array`
- `Array.prototype.reverse()`
- `Array.prototype.toSorted()`
- `String.prototype.localeCompare()`
- `TypedArray.prototype.sort()`
- Getting things sorted in V8 on v8.dev (2018)
- Stable `Array.prototype.sort` on v8.dev (2019)
- `Array.prototype.sort` stability by Mathias Bynens

Array.prototype.splice()

The **`splice()`** method of `Array` instances changes the contents of an array by removing or replacing existing elements and/or adding new elements in place.

To create a new array with a segment removed and/or replaced without mutating the original array, use `toSpliced()`. To access part of an array without modifying it, see `slice()`.

Try it

```
const months = ["Jan", "March", "April", "June"];
months.splice(1, 0, "Feb");
// Inserts at index 1
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, "May");
// Replaces 1 element at index 4
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "May"]
```

Syntax

```
splice(start)
splice(start, deleteCount)
splice(start, deleteCount, item1)
splice(start, deleteCount, item1, item2)
splice(start, deleteCount, item1, item2, /* …, */ itemN)
```

Parameters

`start`

Zero-based index at which to start changing the array, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length`, `0` is used.
- If `start >= array.length`, no element will be deleted, but the method will behave as an adding function, adding as many elements as provided.
- If `start` is omitted (and `splice()` is called with no arguments), nothing is deleted. This is different from passing `undefined`, which is converted to `0`.

`deleteCount` Optional

An integer indicating the number of elements in the array to remove from `start`.

If `deleteCount` is omitted, or if its value is greater than or equal to the number of elements after the position specified by `start`, then all the elements from `start` to the end of the array will be deleted. However, if you wish to pass any `itemN` parameter, you should pass `Infinity` as `deleteCount` to delete all elements after `start`, because an explicit `undefined` gets converted to `0`.

If `deleteCount` is `0` or negative, no elements are removed. In this case, you should specify at least one new element (see below).

`item1`, …, `itemN` Optional

The elements to add to the array, beginning from `start`.

If you do not specify any elements, `splice()` will only remove elements from the array.

Return value

An array containing the deleted elements.

If only one element is removed, an array of one element is returned.

If no elements are removed, an empty array is returned.

Description

The `splice()` method is a mutating method. It may change the content of `this`. If the specified number of elements to insert differs from the number of elements being removed, the array's `length` will be changed as well. At the same time, it uses `[Symbol.species]` to create a new array instance to be returned.

If the deleted portion is sparse, the array returned by `splice()` is sparse as well, with those corresponding indices being empty slots.

The `splice()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Remove 0 (zero) elements before index 2, and insert "drum"

```
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(2, 0, "drum");

// myFish is ["angel", "clown", "drum", "mandarin", "sturgeon"]
// removed is [], no elements removed
```

Remove 0 (zero) elements before index 2, and insert "drum" and "guitar"

```
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(2, 0, "drum", "guitar");

// myFish is ["angel", "clown", "drum", "guitar", "mandarin", "sturgeon"]
// removed is [], no elements removed
```

Remove 0 (zero) elements at index 0, and insert "angel"

`splice(0, 0, ...elements)` inserts elements at the start of the array like `unshift()`.

```
const myFish = ["clown", "mandarin", "sturgeon"];
const removed = myFish.splice(0, 0, "angel");

// myFish is ["angel", "clown", "mandarin", "sturgeon"]
// no items removed
```

Remove 0 (zero) elements at last index, and insert "sturgeon"

`splice(array.length, 0, ...elements)` inserts elements at the end of the array like `push()`.

```
const myFish = ["angel", "clown", "mandarin"];
const removed = myFish.splice(myFish.length, 0, "sturgeon");

// myFish is ["angel", "clown", "mandarin", "sturgeon"]
// no items removed
```

Remove 1 element at index 3

```
const myFish = ["angel", "clown", "drum", "mandarin", "sturgeon"];
const removed = myFish.splice(3, 1);

// myFish is ["angel", "clown", "drum", "sturgeon"]
// removed is ["mandarin"]
```

Remove 1 element at index 2, and insert "trumpet"

```
const myFish = ["angel", "clown", "drum", "sturgeon"];
const removed = myFish.splice(2, 1, "trumpet");

// myFish is ["angel", "clown", "trumpet", "sturgeon"]
// removed is ["drum"]
```

Remove 2 elements from index 0, and insert "parrot", "anemone" and "blue"

```
const myFish = ["angel", "clown", "trumpet", "sturgeon"];
const removed = myFish.splice(0, 2, "parrot", "anemone", "blue");

// myFish is ["parrot", "anemone", "blue", "trumpet", "sturgeon"]
// removed is ["angel", "clown"]
```

Remove 2 elements, starting from index 2

```
const myFish = ["parrot", "anemone", "blue", "trumpet", "sturgeon"];
const removed = myFish.splice(2, 2);

// myFish is ["parrot", "anemone", "sturgeon"]
// removed is ["blue", "trumpet"]
```

Remove 1 element from index -2

```
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(-2, 1);
```

```
// myFish is ["angel", "clown", "sturgeon"]
// removed is ["mandarin"]
```

Remove all elements, starting from index 2

```
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(2);

// myFish is ["angel", "clown"]
// removed is ["mandarin", "sturgeon"]
```

Using splice() on sparse arrays

The `splice()` method preserves the array's sparseness.

```
const arr = [1, , 3, 4, , 6];
console.log(arr.splice(1, 2)); // [empty, 3]
console.log(arr); // [1, 4, empty, 6]
```

Calling splice() on non-array objects

The `splice()` method reads the `length` property of `this`. It then updates the integer-keyed properties and the `length` property as needed.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
};
console.log(Array.prototype.splice.call(arrayLike, 0, 1, 2, 3));
// [ 5 ]
console.log(arrayLike);
// { '0': 2, '1': 3, '3': 4, length: 4, unrelated: 'foo' }
```

See also

- Indexed collections guide
- `Array`
- `Array.prototype.concat()`
- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.slice()`
- `Array.prototype.toSpliced()`
- `Array.prototype.unshift()`

Array.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `Array` instances implements the iterable protocol and allows arrays to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns an array iterator object that yields the value of each index in the array.

The initial value of this property is the same function object as the initial value of the `Array.prototype.values` property.

Try it

```
const array = ["a", "b", "c"];
const iterator = array[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
}

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

```
array[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Array.prototype.values()`: a new iterable iterator object that yields the value of each index in the array.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes arrays iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

HTML

```
<ul id="letterResult"></ul>
```

JavaScript

```
const arr = ["a", "b", "c"];
const letterResult = document.getElementById("letterResult");
for (const letter of arr) {
  const li = document.createElement("li");
  li.textContent = letter;
  letterResult.appendChild(li);
}
```

Result

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
const arr = ["a", "b", "c", "d", "e"];
const arrIter = arr[Symbol.iterator]();
console.log(arrIter.next().value); // a
console.log(arrIter.next().value); // b
console.log(arrIter.next().value); // c
console.log(arrIter.next().value); // d
console.log(arrIter.next().value); // e
```

Handling strings and string arrays with the same function

Because both strings and arrays implement the iterable protocol, a generic function can be designed to handle both inputs in the same fashion. This is better than calling `Array.prototype.values()` directly, which requires the input to be an array, or at least an object with such a method.

```
function logIterable(it) {
  if (typeof it[Symbol.iterator] !== "function") {
    console.log(it, "is not iterable.");
    return;
  }
  for (const letter of it) {
    console.log(letter);
  }
}

// Array
logIterable(["a", "b", "c"]);
// a
// b
// c

// String
logIterable("abc");
// a
// b
// c

// Number
logIterable(123);
// 123 is not iterable.
```

See also

- Polyfill of `Array.prototype[Symbol.iterator]` in `core-js`
- Indexed collections guide
- `Array`
- `Array.prototype.keys()`
- `Array.prototype.entries()`
- `Array.prototype.values()`
- `TypedArray.prototype[Symbol.iterator]()`
- `String.prototype[Symbol.iterator]()`
- `Symbol.iterator`
- Iteration protocols

Array[Symbol.species]

The **`Array[Symbol.species]`** static accessor property returns the constructor used to construct return values from array methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible. Modern array methods, such as `toReversed()`, do not

> use `[Symbol.species]` and always return a new `Array` base class instance.

Syntax

```
Array[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from array methods that create new arrays.

Description

The `[Symbol.species]` accessor property returns the default constructor for `Array` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class Array {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubArray extends Array {}
SubArray[Symbol.species] === SubArray; // true
```

When calling array methods that do not mutate the existing array but return a new array instance (for example, `filter()` and `map()`), the array's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the array method. This makes it technically possible to make array methods return objects unrelated to arrays.

```
class NotAnArray {
  constructor(length) {
    this.length = length;
  }
}

const arr = [0, 1, 2];
arr.constructor = { [Symbol.species]: NotAnArray };
arr.map((i) => i); // NotAnArray { '0': 0, '1': 1, '2': 2, length: 3 }
arr.filter((i) => i); // NotAnArray { '0': 1, '1': 2, length: 0 }
arr.concat([1, 2]); // NotAnArray { '0': 0, '1': 1, '2': 2, '3': 1, '4': 2, length: 5 }
```

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `Array` constructor for `Array`.

```
Array[Symbol.species]; // [Function: Array]
```

Species in derived objects

In an instance of a custom `Array` subclass, such as `MyArray`, the `MyArray` species is the `MyArray` constructor. However, you might want to overwrite this, in order to return parent `Array` objects in your derived class methods:

```
class MyArray extends Array {
  // Overwrite MyArray species to the parent Array constructor
  static get [Symbol.species]() {
    return Array;
  }
}
```

See also

- Polyfill of `Array[Symbol.species]` and support of `[Symbol.species]` in all affected `Array` methods in `core-js`
- Indexed collections guide
- `Array`
- `Symbol.species`

Array.prototype[Symbol.unscopables]

The **`[Symbol.unscopables]`** data property of `Array.prototype` is shared by all `Array` instances. It contains property names that were not included in the ECMAScript standard prior to the ES2015 version and that are ignored for `with` statement-binding purposes.

Value

A `null`-prototype object with property names given below and their values set to `true`.

Property attributes of `Array.prototype[Symbol.unscopables]`	
Writable	no
Enumerable	no
Configurable	yes

Description

The default `Array` properties that are ignored for `with` statement-binding purposes are:

- `at()`
- `copyWithin()`
- `entries()`
- `fill()`
- `find()`
- `findIndex()`
- `findLast()`
- `findLastIndex()`
- `flat()`
- `flatMap()`
- `includes()`
- `keys()`
- `toReversed()`
- `toSorted()`
- `toSpliced()`
- `values()`

`Array.prototype[Symbol.unscopables]` is an empty object only containing all the above property names with the value `true`. Its prototype is `null`, so `Object.prototype` properties like `toString` won't accidentally be made unscopable, and a `toString()` within the `with` statement will continue to be called on the array.

See `Symbol.unscopables` for how to set unscopable properties for your own objects.

Examples

Imagine the `values.push('something')` call below is in code that was written prior to ECMAScript 2015.

```
var values = [];

with (values) {
  values.push("something");
}
```

When ECMAScript 2015 introduced the `Array.prototype.values()` method, the `with` statement in the above code started to interpret `values` as the `values.values` array method instead of the external `values` variable. The `values.push('something')` call would break because it's now accessing `push` on the `values.values` method. This caused a bug to be reported to Firefox (Firefox Bug 883914).

So the `[Symbol.unscopables]` data property for `Array.prototype` causes the `Array` properties introduced in ECMAScript 2015 to be ignored for `with` statement-binding purposes — allowing code that was written prior to ECMAScript 2015 to continue working as expected, rather than breaking.

See also

- Polyfill of `Array.prototype[Symbol.unscopables]` in `core-js`
- Indexed collections guide
- `Array`
- `with`
- `Symbol.unscopables`

Array.prototype.toLocaleString()

The **`toLocaleString()`** method of `Array` instances returns a string representing the elements of the array. The elements are converted to strings using their `toLocaleString` methods and these strings are separated by a locale-specific string (such as a comma ",").

Try it

```
const array = [1, "a", new Date("21 Dec 1997 14:12:00 UTC")];
const localeString = array.toLocaleString("en", { timeZone: "UTC" });

console.log(localeString);
// Expected output: "1,a,12/21/1997, 2:12:00 PM",
// This assumes "en" locale and UTC timezone - your results may vary
```

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object with configuration properties. What you can pass here depends on what elements are being converted. For example, for numbers, see `Number.prototype.toLocaleString()`.

Return value

A string representing the elements of the array.

Description

The `Array.prototype.toLocaleString` method traverses its content, calling the `toLocaleString` method of every element with the `locales` and `options` parameters provided, and concatenates them with an implementation-defined separator (such as a comma ",").

> **Note:** The `locales` or `options` arguments do not control the separator used between array elements; they are simply passed to the `toLocaleString()` method of each element. The actual separator (usually a comma) depends solely on the host's current locale. If you expect localized list formatting, consider using `Intl.ListFormat` instead.

If an element is `undefined`, `null`, it is converted to an empty string instead of the string `"null"` or `"undefined"`.

When used on sparse arrays, the `toLocaleString()` method iterates empty slots as if they have the value `undefined`.

The `toLocaleString()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using locales and options

The elements of the array are converted to strings using their `toLocaleString` methods. For example, this snippet implicitly calls the `Number.prototype.toLocaleString()` method to display the currency for the strings and numbers in the `prices` array:

```
const prices = ["￥7", 500, 8123, 12];
prices.toLocaleString("ja-JP", { style: "currency", currency: "JPY" });

// "￥7,￥500,￥8,123,￥12"
```

List separators

The list separator is not affected by the `locales` parameter. To configure it, use `Intl.ListFormat` instead.

```
const nums = [8888, 9999];
console.log(nums.toLocaleString("zh")); // "8,888,9,999"

const formatter = new Intl.ListFormat("zh", {
  type: "conjunction",
  style: "narrow",
});
console.log(formatter.format(nums.map((x) => x.toLocaleString("zh"))));
// "8,888、9,999"
```

Using toLocaleString() on sparse arrays

`toLocaleString()` treats empty slots the same as `undefined` and produces an extra separator:

```
console.log([1, , 3].toLocaleString()); // '1,,3'
```

Calling toLocaleString() on non-array objects

The `toLocaleString()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: 1,
  1: 2,
  2: 3,
  3: 4, // ignored by toLocaleString() since length is 3
};
console.log(Array.prototype.toLocaleString.call(arrayLike));
// 1,2,3
```

See also

- Indexed collections guide
- `Array`
- `Array.prototype.toString()`

- `TypedArray.prototype.toLocaleString()`
- `Intl`
- `Intl.ListFormat`
- `Object.prototype.toLocaleString()`
- `Number.prototype.toLocaleString()`
- `Temporal.PlainDate.prototype.toLocaleString()`

Array.prototype.toReversed()

The **`toReversed()`** method of `Array` instances is the copying counterpart of the `reverse()` method. It returns a new array with the elements in reversed order.

Syntax

```
toReversed()
```

Parameters

None.

Return value

A new array containing the elements in reversed order.

Description

The `toReversed()` method transposes the elements of the calling array object in reverse order and returns a new array.

When used on sparse arrays, the `toReversed()` method iterates empty slots as if they have the value `undefined`.

The `toReversed()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Reversing the elements in an array

The following example creates an array `items`, containing three elements, then creates a new array that's the reverse of `items`. The `items` array remains unchanged.

```
const items = [1, 2, 3];
console.log(items); // [1, 2, 3]

const reversedItems = items.toReversed();
console.log(reversedItems); // [3, 2, 1]
console.log(items); // [1, 2, 3]
```

Using toReversed() on sparse arrays

The return value of `toReversed()` is never sparse. Empty slots become `undefined` in the returned array.

```
console.log([1, , 3].toReversed()); // [3, undefined, 1]
console.log([1, , 3, 4].toReversed()); // [4, 3, undefined, 1]
```

Calling toReversed() on non-array objects

The `toReversed()` method reads the `length` property of `this`. It then visits each property having an integer key between `length - 1` and `0` in descending order, adding the value of the current property to the end of the array to be returned.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
console.log(Array.prototype.toReversed.call(arrayLike));
// [4, undefined, undefined]
// The '0' and '1' indices are not present so they become undefined
```

See also

- Polyfill of `Array.prototype.toReversed` in `core-js`
- es-shims polyfill of `Array.prototype.toReversed`
- Indexed collections guide
- `Array.prototype.reverse()`
- `Array.prototype.toSorted()`
- `Array.prototype.toSpliced()`
- `Array.prototype.with()`
- `TypedArray.prototype.toReversed()`

Array.prototype.toSorted()

The **`toSorted()`** method of `Array` instances is the copying version of the `sort()` method. It returns a new array with the elements sorted in ascending order.

Syntax

```
toSorted()
toSorted(compareFn)
```

Parameters

`compareFn` Optional

> A function that determines the order of the elements. If omitted, the array elements are converted to strings, then sorted according to each character's Unicode code point value. See `sort()` for more information.

Return value

A new array with the elements sorted in ascending order.

Description

See `sort()` for more information on the `compareFn` parameter.

When used on sparse arrays, the `toSorted()` method iterates empty slots as if they have the value `undefined`.

The `toSorted()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Sorting an array

```
const months = ["Mar", "Jan", "Feb", "Dec"];
const sortedMonths = months.toSorted();
console.log(sortedMonths); // ['Dec', 'Feb', 'Jan', 'Mar']
console.log(months); // ['Mar', 'Jan', 'Feb', 'Dec']

const values = [1, 10, 21, 2];
const sortedValues = values.toSorted((a, b) => a - b);
console.log(sortedValues); // [1, 2, 10, 21]
console.log(values); // [1, 10, 21, 2]
```

For more usage examples, see `sort()`.

Using toSorted() on sparse arrays

Empty slots are sorted as if they have the value `undefined`. They are always sorted to the end of the array and `compareFn` is not called for them.

```
console.log(["a", "c", , "b"].toSorted()); // ['a', 'b', 'c', undefined]
console.log([, undefined, "a", "b"].toSorted()); // ["a", "b", undefined, undefined]
```

Calling toSorted() on non-array objects

The `toSorted()` method reads the `length` property of `this`. It then collects all existing integer-keyed properties in the range of `0` to `length - 1`, sorts them, and writes them into a new array.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
  3: 3, // ignored by toSorted() since length is 3
};
console.log(Array.prototype.toSorted.call(arrayLike));
// [4, 5, undefined]
```

See also

- Polyfill of `Array.prototype.toSorted` in `core-js`
- es-shims polyfill of `Array.prototype.toSorted`
- Indexed collections guide
- `Array.prototype.sort()`
- `Array.prototype.toReversed()`
- `Array.prototype.toSpliced()`
- `Array.prototype.with()`
- `TypedArray.prototype.toSorted()`

Array.prototype.toSpliced()

The **`toSpliced()`** method of `Array` instances is the copying version of the `splice()` method. It returns a new array with some elements removed and/or replaced at a given index.

Syntax

```
toSpliced(start)
toSpliced(start, skipCount)
toSpliced(start, skipCount, item1)
```

```
toSpliced(start, skipCount, item1, item2)
toSpliced(start, skipCount, item1, item2, /* …, */ itemN)
```

Parameters

`start`

Zero-based index at which to start changing the array, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length` or `start` is omitted, `0` is used.
- If `start >= array.length`, no element will be deleted, but the method will behave as an adding function, adding as many elements as provided.

`skipCount` Optional

An integer indicating the number of elements in the array to remove (or, to skip) from `start`.

If `skipCount` is omitted, or if its value is greater than or equal to the number of elements after the position specified by `start`, then all the elements from `start` to the end of the array will be deleted. However, if you wish to pass any `itemN` parameter, you should pass `Infinity` as `skipCount` to delete all elements after `start`, because an explicit `undefined` gets converted to `0`.

If `skipCount` is `0` or negative, no elements are removed. In this case, you should specify at least one new element (see below).

`item1`, ..., `itemN` Optional

The elements to add to the array, beginning from `start`.

If you do not specify any elements, `toSpliced()` will only remove elements from the array.

Return value

A new array that consists of all elements before `start`, `item1`, `item2`, ..., `itemN`, and all elements after `start + skipCount`.

Description

The `toSpliced()` method, like `splice()`, does multiple things at once: it removes the given number of elements from the array, starting at a given index, and then inserts the given elements at the same index. However, it returns a new array instead of modifying the original array. The deleted elements therefore are not returned from this method, but they remain accessible in the original array.

The `toSpliced()` method never produces a sparse array. If the source array is sparse, the empty slots will be replaced with `undefined` in the new array.

The `toSpliced()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Deleting, adding, and replacing elements

You can use `toSpliced()` to delete, add, and replace elements in an array and create a new array more efficiently than using `slice()` and `concat()`.

```
const months = ["Jan", "Mar", "Apr", "May"];

// Inserting an element at index 1
const months2 = months.toSpliced(1, 0, "Feb");
console.log(months2); // ["Jan", "Feb", "Mar", "Apr", "May"]

// Deleting two elements starting from index 2
const months3 = months2.toSpliced(2, 2);
console.log(months3); // ["Jan", "Feb", "May"]

// Replacing one element at index 1 with two new elements
const months4 = months3.toSpliced(1, 1, "Feb", "Mar");
console.log(months4); // ["Jan", "Feb", "Mar", "May"]

// Original array is not modified
console.log(months); // ["Jan", "Mar", "Apr", "May"]
```

Using toSpliced() on sparse arrays

The `toSpliced()` method always creates a dense array.

```
const arr = [1, , 3, 4, , 6];
console.log(arr.toSpliced(1, 2)); // [1, 4, undefined, 6]
```

Calling toSpliced() on non-array objects

The `toSpliced()` method reads the `length` property of `this`. It then reads the integer-keyed properties needed and writes them into the new array.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
};
```

```
console.log(Array.prototype.toSpliced.call(arrayLike, 0, 1, 2, 3));
// [2, 3, undefined, 4]
```

See also

- Polyfill of `Array.prototype.toSpliced` in `core-js`
- es-shims polyfill of `Array.prototype.toSpliced`
- `Array.prototype.splice()`
- `Array.prototype.toReversed()`
- `Array.prototype.toSorted()`
- `Array.prototype.with()`

Array.prototype.toString()

The **`toString()`** method of `Array` instances returns a string representing the specified array and its elements.

Try it

```
const array = [1, 2, "a", "1a"];

console.log(array.toString());
// Expected output: "1,2,a,1a"
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the elements of the array.

Description

The `Array` object overrides the `toString` method of `Object`. The `toString` method of arrays calls `join()` internally, which joins the array and returns one string containing each array element separated by commas. If the `join` method is unavailable or is not a function, `Object.prototype.toString` is used instead, returning `[object Array]`.

```
const arr = [];
arr.join = 1; // re-assign `join` with a non-function
console.log(arr.toString()); // [object Array]

console.log(Array.prototype.toString.call({ join: () => 1 })); // 1
```

JavaScript calls the `toString` method automatically when an array is to be represented as a text value or when an array is referred to in a string concatenation.

`Array.prototype.toString` recursively converts each element, including other arrays, to strings. Because the string returned by `Array.prototype.toString` does not have delimiters, nested arrays look like they are flattened.

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

console.log(matrix.toString()); // 1,2,3,4,5,6,7,8,9
```

When an array is cyclic (it contains an element that is itself), browsers avoid infinite recursion by ignoring the cyclic reference.

```
const arr = [];
arr.push(1, [3, arr, 4], 2);
console.log(arr.toString()); // 1,3,,4,2
```

Examples

Using toString()

```
const array = [1, 2, "a", "1a"];

console.log(array.toString()); // "1,2,a,1a"
```

Using toString() on sparse arrays

Following the behavior of `join()`, `toString()` treats empty slots the same as `undefined` and produces an extra separator:

```
console.log([1, , 3].toString()); // '1,,3'
```

Calling toString() on non-array objects

`toString()` is generic. It expects `this` to have a `join()` method; or, failing that, uses `Object.prototype.toString()` instead.

```
console.log(Array.prototype.toString.call({ join: () => 1 }));
// 1; a number
console.log(Array.prototype.toString.call({ join: () => undefined }));
// undefined
console.log(Array.prototype.toString.call({ join: "not function" }));
// "[object Object]"
```

See also

- Indexed collections guide
- `Array`
- `Array.prototype.join()`
- `Array.prototype.toLocaleString()`
- `TypedArray.prototype.toString()`
- `String.prototype.toString()`

Array.prototype.unshift()

The **`unshift()`** method of `Array` instances adds the specified elements to the beginning of an array and returns the new length of the array.

Try it

```
const array = [1, 2, 3];

console.log(array.unshift(4, 5));
// Expected output: 5

console.log(array);
// Expected output: Array [4, 5, 1, 2, 3]
```

Syntax

```
unshift()
unshift(element1)
unshift(element1, element2)
unshift(element1, element2, /* …, */ elementN)
```

Parameters

`element1, ..., elementN`

The elements to add to the front of the `arr`.

Return value

The new `length` property of the object upon which the method was called.

Description

The `unshift()` method inserts the given values to the beginning of an array-like object.

`Array.prototype.push()` has similar behavior to `unshift()`, but applied to the end of an array.

Please note that, if multiple elements are passed as parameters, they're inserted in chunk at the beginning of the object, in the exact same order they were passed as parameters. Hence, calling `unshift()` with `n` arguments **once**, or calling it `n` times with **1** argument (with a loop, for example), don't yield the same results.

See example:

```
let arr = [4, 5, 6];

arr.unshift(1, 2, 3);
console.log(arr);
// [1, 2, 3, 4, 5, 6]

arr = [4, 5, 6]; // resetting the array

arr.unshift(1);
arr.unshift(2);
arr.unshift(3);

console.log(arr);
// [3, 2, 1, 4, 5, 6]
```

The `unshift()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Using unshift()

```
const arr = [1, 2];

arr.unshift(0); // result of the call is 3, which is the new array length
// arr is [0, 1, 2]

arr.unshift(-2, -1); // the new array length is 5
// arr is [-2, -1, 0, 1, 2]

arr.unshift([-4, -3]); // the new array length is 6
// arr is [[-4, -3], -2, -1, 0, 1, 2]

arr.unshift([-7, -6], [-5]); // the new array length is 8
// arr is [ [-7, -6], [-5], [-4, -3], -2, -1, 0, 1, 2 ]
```

Calling unshift() on non-array objects

The `unshift()` method reads the `length` property of `this`. It shifts all indices in the range `0` to `length - 1` right by the number of arguments (incrementing their values by this number). Then, it sets each index starting at `0` with the arguments passed to `unshift()`. Finally, it sets the `length` to the previous length plus the number of prepended elements.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
Array.prototype.unshift.call(arrayLike, 1, 2);
console.log(arrayLike);
// { '0': 1, '1': 2, '4': 4, length: 5, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.unshift.call(plainObj, 1, 2);
console.log(plainObj);
// { '0': 1, '1': 2, length: 2 }
```

See also

- Polyfill of `Array.prototype.unshift` in `core-js` with fixes of this method
- es-shims polyfill of `Array.prototype.unshift`
- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

Array.prototype.values()

The **`values()`** method of `Array` instances returns a new *array iterator* object that iterates the value of each item in the array.

Try it

```
const array = ["a", "b", "c"];
const iterator = array.values();

for (const value of iterator) {
  console.log(value);
}

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

`Array.prototype.values()` is the default implementation of `Array.prototype[Symbol.iterator]()`.

```
Array.prototype.values === Array.prototype[Symbol.iterator]; // true
```

When used on sparse arrays, the `values()` method iterates empty slots as if they have the value `undefined`.

The `values()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Iteration using for...of loop

Because `values()` returns an iterable iterator, you can use a `for...of` loop to iterate it.

```
const arr = ["a", "b", "c", "d", "e"];
const iterator = arr.values();

for (const letter of iterator) {
  console.log(letter);
} // "a" "b" "c" "d" "e"
```

Iteration using next()

Because the return value is also an iterator, you can directly call its `next()` method.

```
const arr = ["a", "b", "c", "d", "e"];
const iterator = arr.values();
iterator.next(); // { value: "a", done: false }
iterator.next(); // { value: "b", done: false }
iterator.next(); // { value: "c", done: false }
iterator.next(); // { value: "d", done: false }
iterator.next(); // { value: "e", done: false }
iterator.next(); // { value: undefined, done: true }
console.log(iterator.next().value); // undefined
```

Reusing the iterable

> **Warning:** The array iterator object should be a one-time use object. Do not reuse it.

The iterable returned from `values()` is not reusable. When `next().done = true` or `currentIndex > length`, the `for...of` loop ends, and further iterating it has no effect.

```
const arr = ["a", "b", "c", "d", "e"];
const values = arr.values();
for (const letter of values) {
  console.log(letter);
}
// "a" "b" "c" "d" "e"
for (const letter of values) {
  console.log(letter);
}
// No output
```

If you use a `break` statement to end the iteration early, the iterator can resume from the current position when continuing to iterate it.

```
const arr = ["a", "b", "c", "d", "e"];
const values = arr.values();
for (const letter of values) {
  console.log(letter);
  if (letter === "b") {
    break;
  }
}
// "a" "b"

for (const letter of values) {
  console.log(letter);
}
// "c" "d" "e"
```

Mutations during iteration

There are no values stored in the array iterator object returned from `values()`; instead, it stores the address of the array used in its creation, and reads the currently visited index on each iteration. Therefore, its iteration output depends on the value stored in that index at the time of stepping. If the values in the array changed, the array iterator object's values change too.

```
const arr = ["a", "b", "c", "d", "e"];
const iterator = arr.values();
console.log(iterator); // Array Iterator { }
console.log(iterator.next().value); // "a"
arr[1] = "n";
console.log(iterator.next().value); // "n"
```

Unlike iterative methods, the array iterator object does not save the array's length at the time of its creation, but reads it once on each iteration. Therefore, if the array grows during iteration, the iterator will visit the new elements too. This may lead to infinite loops.

```
const arr = [1, 2, 3];
for (const e of arr) {
  arr.push(e * 10);
}
// RangeError: invalid array length
```

Iterating sparse arrays

`values()` will visit empty slots as if they are `undefined`.

```
for (const element of [, "a"].values()) {
  console.log(element);
}
// undefined
// 'a'
```

Calling values() on non-array objects

The `values()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: "d", // ignored by values() since length is 3
};
for (const entry of Array.prototype.values.call(arrayLike)) {
  console.log(entry);
}
// a
// b
// c
```

See also

- Polyfill of `Array.prototype.values` in `core-js`
- es-shims polyfill of `Array.prototype.values`
- Indexed collections guide
- `Array`
- `Array.prototype.entries()`
- `Array.prototype.keys()`
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype.values()`
- Iteration protocols

Array.prototype.with()

The **`with()`** method of `Array` instances is the copying version of using the bracket notation to change the value of a given index. It returns a new array with the element at the given index replaced with the given value.

Syntax

```
arrayInstance.with(index, value)
```

Parameters

`index`

Zero-based index at which to change the array, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= index < 0`, `index + array.length` is used.
- If the index after normalization is out of bounds, a `RangeError` is thrown.

`value`

Any value to be assigned to the given index.

Return value

A new array with the element at `index` replaced with `value`.

Exceptions

`RangeError`

Thrown if `index >= array.length` or `index < -array.length`.

Description

The `with()` method changes the value of a given index in the array, returning a new array with the element at the given index replaced with the given value. The original array is not modified. This allows you to chain array methods while doing manipulations.

By combining `with()` with `at()`, you can both write and read (respectively) an array using negative indices.

The `with()` method never produces a sparse array. If the source array is sparse, the empty slots will be replaced with `undefined` in the new array.

The `with()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Creating a new array with a single element changed

```
const arr = [1, 2, 3, 4, 5];
console.log(arr.with(2, 6)); // [1, 2, 6, 4, 5]
console.log(arr); // [1, 2, 3, 4, 5]
```

Chaining array methods

With the `with()` method, you can update a single element in an array and then apply other array methods.

```
const arr = [1, 2, 3, 4, 5];
console.log(arr.with(2, 6).map((x) => x ** 2)); // [1, 4, 36, 16, 25]
```

Using with() on sparse arrays

The `with()` method always creates a dense array.

```
const arr = [1, , 3, 4, , 6];
console.log(arr.with(0, 2)); // [2, undefined, 3, 4, undefined, 6]
```

Calling with() on non-array objects

The `with()` method creates and returns a new array. It reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`. As each property of `this` is accessed, the array element having an index equal to the key of the property is set to the value of the property. Finally, the array value at `index` is set to `value`.

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
  3: 3, // ignored by with() since length is 3
};
console.log(Array.prototype.with.call(arrayLike, 0, 1));
// [ 1, undefined, 4 ]
```

See also

- Polyfill of `Array.prototype.with` in `core-js`
- es-shims polyfill of `Array.prototype.with`
- Indexed collections guide
- `Array.prototype.toReversed()`
- `Array.prototype.toSorted()`
- `Array.prototype.toSpliced()`
- `Array.prototype.at()`
- `TypedArray.prototype.with()`

ArrayBuffer

The **`ArrayBuffer`** object is used to represent a generic raw binary data buffer.

It is an array of bytes, often referred to in other languages as a "byte array". You cannot directly manipulate the contents of an `ArrayBuffer`; instead, you create one of the typed array objects or a `DataView` object which represents the buffer in a specific format, and use that to read and write the contents of the buffer.

The `ArrayBuffer()` constructor creates a new `ArrayBuffer` of the given length in bytes. You can also get an array buffer from existing data, for example, from a Base64 string or from a local file.

`ArrayBuffer` is a transferable object.

Description

Resizing ArrayBuffers

`ArrayBuffer` objects can be made resizable by including the `maxByteLength` option when calling the `ArrayBuffer()` constructor. You can query whether an `ArrayBuffer` is resizable and what its maximum size is by accessing its `resizable` and `maxByteLength` properties, respectively. You can assign a new size to a resizable `ArrayBuffer` with a `resize()` call. New bytes are initialized to 0.

These features make resizing `ArrayBuffer`s more efficient — otherwise, you have to make a copy of the buffer with a new size. It also gives JavaScript parity with WebAssembly in this regard (Wasm linear memory can be resized with `WebAssembly.Memory.prototype.grow()`).

Transferring ArrayBuffers

`ArrayBuffer` objects can be transferred between different execution contexts, like Web Workers or Service Workers, using the structured clone algorithm. This is done by passing the `ArrayBuffer` as a transferable object in a call to `Worker.postMessage()` or `ServiceWorker.postMessage()`. In pure JavaScript, you can also transfer the ownership of memory from one `ArrayBuffer` to another using its `transfer()` or `transferToFixedLength()` method.

When an `ArrayBuffer` is transferred, its original copy becomes *detached* — this means it is no longer usable. At any moment, there will only be one copy of the `ArrayBuffer` that actually has access to the underlying memory. Detached buffers have the following behaviors:

- `byteLength` becomes 0 (in both the buffer and the associated typed array views).
- Methods, such as `resize()` and `slice()`, throw a `TypeError` when invoked. The associated typed array views' methods also throw a `TypeError`.

You can check whether an `ArrayBuffer` is detached by its `detached` property.

Constructor

`ArrayBuffer()`

Creates a new `ArrayBuffer` object.

Static properties

`ArrayBuffer[Symbol.species]`

The constructor function that is used to create derived objects.

Static methods

`ArrayBuffer.isView()`

Returns `true` if `arg` is one of the ArrayBuffer views, such as typed array objects or a `DataView`. Returns `false` otherwise.

Instance properties

These properties are defined on `ArrayBuffer.prototype` and shared by all `ArrayBuffer` instances.

`ArrayBuffer.prototype.byteLength`

The size, in bytes, of the `ArrayBuffer`. This is established when the array is constructed and can only be changed using the `ArrayBuffer.prototype.resize()` method if the `ArrayBuffer` is resizable.

`ArrayBuffer.prototype.constructor`

The constructor function that created the instance object. For `ArrayBuffer` instances, the initial value is the `ArrayBuffer` constructor.

`ArrayBuffer.prototype.detached`

Read-only. Returns `true` if the `ArrayBuffer` has been detached (transferred), or `false` if not.

`ArrayBuffer.prototype.maxByteLength`

The read-only maximum length, in bytes, that the `ArrayBuffer` can be resized to. This is established when the array is constructed and cannot be changed.

`ArrayBuffer.prototype.resizable`

Read-only. Returns `true` if the `ArrayBuffer` can be resized, or `false` if not.

`ArrayBuffer.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"ArrayBuffer"`. This property is used in `Object.prototype.toString()`.

Instance methods

`ArrayBuffer.prototype.resize()`

Resizes the `ArrayBuffer` to the specified size, in bytes.

`ArrayBuffer.prototype.slice()`

Returns a new `ArrayBuffer` whose contents are a copy of this `ArrayBuffer`'s bytes from `begin` (inclusive) up to `end` (exclusive). If either `begin` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

`ArrayBuffer.prototype.transfer()`

Creates a new `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

`ArrayBuffer.prototype.transferToFixedLength()`

Creates a new non-resizable `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

Examples

Creating an ArrayBuffer

In this example, we create a 8-byte buffer with an `Int32Array` view referring to the buffer:

```
const buffer = new ArrayBuffer(8);
const view = new Int32Array(buffer);
```

See also

- Polyfill of `ArrayBuffer` in `core-js`
- JavaScript typed arrays guide
- `SharedArrayBuffer`
- RangeError: invalid array length

ArrayBuffer() constructor

The **`ArrayBuffer()`** constructor creates `ArrayBuffer` objects.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);

console.log(buffer.byteLength);
// Expected output: 8
```

Syntax

```
new ArrayBuffer(length)
new ArrayBuffer(length, options)
```

> **Note:** `ArrayBuffer()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`length`

The size, in bytes, of the array buffer to create.

`options` Optional

An object, which can contain the following properties:

`maxByteLength` Optional

The maximum size, in bytes, that the array buffer can be resized to.

Return value

A new `ArrayBuffer` object of the specified size, with its `maxByteLength` property set to the specified `maxByteLength` if one was specified. Its contents are initialized to 0.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `length` or `maxByteLength` is larger than `Number.MAX_SAFE_INTEGER` ($\geq 2^{53}$) or negative.
- `length` is larger than `maxByteLength`.

Examples

Creating an ArrayBuffer

In this example, we create a 8-byte buffer with an `Int32Array` view referring to the buffer:

```
const buffer = new ArrayBuffer(8);
const view = new Int32Array(buffer);
```

Creating a resizable ArrayBuffer

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then `resize()` it to 12 bytes:

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

buffer.resize(12);
```

> **Note:** It is recommended that `maxByteLength` is set to the smallest value possible for your use case. It should never exceed `1073741824` (1GB) to reduce the risk of out-of-memory errors.

See also

- Polyfill of `ArrayBuffer` in `core-js`
- JavaScript typed arrays guide
- `SharedArrayBuffer`

ArrayBuffer.prototype.byteLength

The **`byteLength`** accessor property of `ArrayBuffer` instances returns the length (in bytes) of this array buffer.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);

// Use byteLength to check the size
const bytes = buffer.byteLength;

console.log(bytes);
// Expected output: 8
```

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed and cannot be changed. This property returns 0 if this `ArrayBuffer` has been detached.

Examples

Using byteLength

```
const buffer = new ArrayBuffer(8);
buffer.byteLength; // 8
```

See also

- `ArrayBuffer`

ArrayBuffer.prototype.detached

The **`detached`** accessor property of `ArrayBuffer` instances returns a boolean indicating whether or not this buffer has been detached (transferred).

Description

The `detached` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is `false` when the `ArrayBuffer` is first created. The value becomes `true` if the `ArrayBuffer` is transferred, which detaches the instance from its underlying memory. Once a buffer becomes detached, it is no longer usable.

Examples

Using detached

```
const buffer = new ArrayBuffer(8);
console.log(buffer.detached); // false
const newBuffer = buffer.transfer();
console.log(buffer.detached); // true
console.log(newBuffer.detached); // false
```

See also

- Polyfill of `ArrayBuffer.prototype.detached` in `core-js`
- es-shims polyfill of `ArrayBuffer.prototype.detached`
- `ArrayBuffer`
- `ArrayBuffer.prototype.transfer()`
- `ArrayBuffer.prototype.transferToFixedLength()`

ArrayBuffer.isView()

The **`ArrayBuffer.isView()`** static method determines whether the passed value is one of the `ArrayBuffer` views, such as typed array objects or a `DataView`.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

console.log(ArrayBuffer.isView(new Int32Array()));
// Expected output: true
```

Syntax

```
ArrayBuffer.isView(value)
```

Parameters

`value`

The value to be checked.

Return value

`true` if the given argument is one of the `ArrayBuffer` views; otherwise, `false`.

Examples

Using isView

```
ArrayBuffer.isView(); // false
ArrayBuffer.isView([]); // false
ArrayBuffer.isView({}); // false
ArrayBuffer.isView(null); // false
ArrayBuffer.isView(undefined); // false
ArrayBuffer.isView(new ArrayBuffer(10)); // false

ArrayBuffer.isView(new Uint8Array()); // true
ArrayBuffer.isView(new Float32Array()); // true
ArrayBuffer.isView(new Int8Array(10).subarray(0, 3)); // true

const buffer = new ArrayBuffer(2);
const dv = new DataView(buffer);
ArrayBuffer.isView(dv); // true
```

See also

- JavaScript typed arrays guide

ArrayBuffer.prototype.maxByteLength

The **`maxByteLength`** accessor property of `ArrayBuffer` instances returns the maximum length (in bytes) that this array buffer can be resized to.

Try it

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

console.log(buffer.byteLength);
// Expected output: 8

console.log(buffer.maxByteLength);
// Expected output: 16
```

Description

The `maxByteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed, set via the `maxByteLength` option of the `ArrayBuffer()` constructor, and cannot be changed.

This property returns 0 if this `ArrayBuffer` has been detached. If this `ArrayBuffer` was constructed without specifying a `maxByteLength` value, this property returns a value equal to the value of the `ArrayBuffer`'s `byteLength`.

Examples

Using maxByteLength

In this example, we create an 8-byte buffer that is resizable to a max length of 16 bytes, then return its `maxByteLength`:

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

buffer.maxByteLength; // 16
```

See also

- `ArrayBuffer`
- `ArrayBuffer.prototype.byteLength`
- `ArrayBuffer.prototype.resize()`

ArrayBuffer.prototype.resizable

The **`resizable`** accessor property of `ArrayBuffer` instances returns whether this array buffer can be resized or not.

Try it

```
const buffer1 = new ArrayBuffer(8, { maxByteLength: 16 });
const buffer2 = new ArrayBuffer(8);

console.log(buffer1.resizable);
// Expected output: true

console.log(buffer2.resizable);
// Expected output: false
```

Description

The `resizable` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed. If the `maxByteLength` option was set in the constructor, `resizable` will return `true`; if not, it will return `false`.

Examples

Using resizable

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then check its `resizable` property, resizing it if `resizable` returns `true`:

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

if (buffer.resizable) {
  console.log("Buffer is resizable!");
  buffer.resize(12);
}
```

See also

- `ArrayBuffer`
- `ArrayBuffer.prototype.maxByteLength`
- `ArrayBuffer.prototype.resize()`

ArrayBuffer.prototype.resize()

The **`resize()`** method of `ArrayBuffer` instances resizes the `ArrayBuffer` to the specified size, in bytes.

Try it

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

console.log(buffer.byteLength);
// Expected output: 8

buffer.resize(12);

console.log(buffer.byteLength);
// Expected output: 12
```

Syntax

```
resize(newLength)
```

Parameters

`newLength`

The new length, in bytes, to resize the `ArrayBuffer` to.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if the `ArrayBuffer` is detached or is not resizable.

`RangeError`

Thrown if `newLength` is larger than the `maxByteLength` of the `ArrayBuffer`.

Description

The `resize()` method resizes an `ArrayBuffer` to the size specified by the `newLength` parameter, provided that the `ArrayBuffer` is resizable and the new size is less than or equal to the `maxByteLength` of the `ArrayBuffer`. New bytes are initialized to 0.

Note that you can use `resize()` to shrink as well as grow an `ArrayBuffer` — it is permissible for `newLength` to be smaller than the `ArrayBuffer`'s current `byteLength`.

Examples

Using resize()

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then check its `resizable` property, resizing it if `resizable` returns `true`:

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

if (buffer.resizable) {
  console.log("Buffer is resizable!");
  buffer.resize(12);
}
```

See also

- `ArrayBuffer`
- `ArrayBuffer.prototype.resizable`
- `ArrayBuffer.prototype.maxByteLength`

ArrayBuffer.prototype.slice()

The **`slice()`** method of `ArrayBuffer` instances returns a new `ArrayBuffer` whose contents are a copy of this `ArrayBuffer`'s bytes from `start`, inclusive, up to `end`, exclusive. If either `start` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);
const int32View = new Int32Array(buffer);
// Produces Int32Array [0, 0, 0, 0]

int32View[1] = 42;
const sliced = new Int32Array(buffer.slice(4, 12));
// Produces Int32Array [42, 0]

console.log(sliced[0]);
// Expected output: 42
```

Syntax

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

- Negative index counts back from the end of the buffer — if `-buffer.length <= start < 0`, `start + buffer.length` is used.
- If `start < -buffer.length` or `start` is omitted, `0` is used.
- If `start >= buffer.length`, an empty buffer is returned.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

- Negative index counts back from the end of the buffer — if `-buffer.length <= end < 0`, `end + buffer.length` is used.
- If `end < -buffer.length`, `0` is used.
- If `end >= buffer.length` or `end` is omitted is `undefined`, `buffer.length` is used, causing all elements until the end to be extracted.
- If `end` implies a position before or at the position that `start` implies, an empty buffer is returned.

Return value

A new `ArrayBuffer` containing the extracted elements. It is not resizable, even if the original was.

Examples

Copying an ArrayBuffer

```
const buf1 = new ArrayBuffer(8);
const buf2 = buf1.slice(0);
```

See also

- `ArrayBuffer`
- `SharedArrayBuffer.prototype.slice()`

ArrayBuffer[Symbol.species]

The **`ArrayBuffer[Symbol.species]`** static accessor property returns the constructor used to construct return values from array buffer methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
ArrayBuffer[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from array buffer methods that create new array buffers.

Description

The `[Symbol.species]` accessor property returns the default constructor for `ArrayBuffer` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class ArrayBuffer {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubArrayBuffer extends ArrayBuffer {}
SubArrayBuffer[Symbol.species] === SubArrayBuffer; // true
```

When calling array buffer methods that do not mutate the existing object but return a new array buffer instance (for example, `slice()`), the object's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the array buffer method.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `ArrayBuffer` constructor for `ArrayBuffer`.

```
ArrayBuffer[Symbol.species]; // function ArrayBuffer()
```

Species in derived objects

In an instance of a custom `ArrayBuffer` subclass, such as `MyArrayBuffer`, the `MyArrayBuffer` species is the `MyArrayBuffer` constructor. However, you might want to overwrite this, in order to return parent `ArrayBuffer` objects in your derived class methods:

```
class MyArrayBuffer extends ArrayBuffer {
  // Overwrite MyArrayBuffer species to the parent ArrayBuffer constructor
  static get [Symbol.species]() {
    return ArrayBuffer;
  }
}
```

See also

- `ArrayBuffer`
- `Symbol.species`

ArrayBuffer.prototype.transfer()

The **`transfer()`** method of `ArrayBuffer` instances creates a new `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

Syntax

```
transfer()
transfer(newByteLength)
```

Parameters

`newByteLength` Optional

The `byteLength` of the new `ArrayBuffer`. Defaults to the `byteLength` of this `ArrayBuffer`.

- If `newByteLength` is smaller than the `byteLength` of this `ArrayBuffer`, the "overflowing" bytes are dropped.
- If `newByteLength` is larger than the `byteLength` of this `ArrayBuffer`, the extra bytes are filled with zeros.
- If this `ArrayBuffer` is resizable, `newByteLength` must not be greater than its `maxByteLength`.

Return value

A new `ArrayBuffer` object. Its contents are initialized to the contents of this `ArrayBuffer`, and extra bytes, if any, are filled with zeros. The new `ArrayBuffer` is resizable if and only if this `ArrayBuffer` is resizable, in which case its `maxByteLength` is the same as this `ArrayBuffer`'s. The original `ArrayBuffer` is detached.

Exceptions

`RangeError`

Thrown if this `ArrayBuffer` is resizable and `newByteLength` is greater than the `maxByteLength` of this `ArrayBuffer`.

`TypeError`

Thrown if this `ArrayBuffer` is already detached, or if it can only be detached by designated operations. Currently, only certain web APIs are capable of creating `ArrayBuffer` objects with designated detaching methods, such as `GPUBuffer.getMappedRange()` and `WebAssembly.Memory.buffer`.

Description

The `transfer()` method performs the same operation as the structured clone algorithm. It copies the bytes of this `ArrayBuffer` into a new `ArrayBuffer` object, then detaches this `ArrayBuffer` object. See transferring ArrayBuffers for more information.

`transfer()` preserves the resizability of this `ArrayBuffer`. If you want the new `ArrayBuffer` to be non-resizable, use `transferToFixedLength()` instead. There's no way to transfer a buffer that makes a fixed-length buffer become resizable.

`transfer()` is very efficient because implementations may implement this method as a zero-copy move or a `realloc` — there does not need to be an actual copy of the data.

Examples

Transferring an ArrayBuffer

```
// Create an ArrayBuffer and write a few bytes
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

// Copy the buffer to the same size
const buffer2 = buffer.transfer();
console.log(buffer.detached); // true
console.log(buffer2.byteLength); // 8
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // 4

// Copy the buffer to a smaller size
const buffer3 = buffer2.transfer(4);
console.log(buffer3.byteLength); // 4
const view3 = new Uint8Array(buffer3);
console.log(view3[1]); // 2
console.log(view3[7]); // undefined

// Copy the buffer to a larger size
const buffer4 = buffer3.transfer(8);
console.log(buffer4.byteLength); // 8
const view4 = new Uint8Array(buffer4);
console.log(view4[1]); // 2
console.log(view4[7]); // 0

// Already detached, throws TypeError
buffer.transfer(); // TypeError: Cannot perform ArrayBuffer.prototype.transfer on a detached ArrayBuffer
```

Transferring a resizable ArrayBuffer

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

// Copy the buffer to a smaller size
const buffer2 = buffer.transfer(4);
console.log(buffer2.byteLength); // 4
console.log(buffer2.maxByteLength); // 16
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // undefined
buffer2.resize(8);
console.log(view2[7]); // 0

// Copy the buffer to a larger size within maxByteLength
const buffer3 = buffer2.transfer(12);
console.log(buffer3.byteLength); // 12

// Copy the buffer to a larger size than maxByteLength
buffer3.transfer(20); // RangeError: Invalid array buffer length
```

See also

- Polyfill of `ArrayBuffer.prototype.transfer` in `core-js`
- es-shims polyfill of `ArrayBuffer.prototype.transfer`
- `ArrayBuffer`
- `ArrayBuffer.prototype.detached`
- `ArrayBuffer.prototype.transferToFixedLength()`

ArrayBuffer.prototype.transferToFixedLength()

The **`transferToFixedLength()`** method of `ArrayBuffer` instances creates a new non-resizable `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

Syntax

```
transferToFixedLength()
transferToFixedLength(newByteLength)
```

Parameters

`newByteLength`

The `byteLength` of the new `ArrayBuffer`. Defaults to the `byteLength` of this `ArrayBuffer`.

- If `newByteLength` is smaller than the `byteLength` of this `ArrayBuffer`, the "overflowing" bytes are dropped.
- If `newByteLength` is larger than the `byteLength` of this `ArrayBuffer`, the extra bytes are filled with zeros.

Return value

A new `ArrayBuffer` object. Its contents are initialized to the contents of this `ArrayBuffer`, and extra bytes, if any, are filled with zeros. The new `ArrayBuffer` is always non-resizable. The original `ArrayBuffer` is detached.

Exceptions

`TypeError`

Thrown if this `ArrayBuffer` is already detached, or if it can only be detached by designated operations. Currently, only certain web APIs are capable of creating `ArrayBuffer` objects with designated detaching methods, such as `GPUBuffer.getMappedRange()` and `WebAssembly.Memory.buffer`.

Description

Unlike `transfer()`, `transferToFixedLength()` always creates a non-resizable `ArrayBuffer`. This means `newByteLength` can be larger than the `maxByteLength`, even if this `ArrayBuffer` is resizable. See transferring ArrayBuffers for more information.

Examples

Transferring a resizable ArrayBuffer to fixed-length

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

const buffer2 = buffer.transferToFixedLength();
console.log(buffer2.byteLength); // 8
console.log(buffer2.resizable); // false
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // 4
```

Using `transferToFixedLength`, `newByteLength` can be larger than the `maxByteLength` of the original `ArrayBuffer`.

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

const buffer2 = buffer.transferToFixedLength(20);
console.log(buffer2.byteLength); // 20
console.log(buffer2.resizable); // false
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // 4
```

See also

- Polyfill of `ArrayBuffer.prototype.transferToFixedLength` in `core-js`
- es-shims polyfill of `ArrayBuffer.prototype.transferToFixedLength`
- `ArrayBuffer`
- `ArrayBuffer.prototype.detached`
- `ArrayBuffer.prototype.transfer()`

AsyncDisposableStack

The **`AsyncDisposableStack`** object represents a stack of async disposers to run when the stack itself is disposed. Disposer functions are executed in reverse order of registration, with strong error handling guarantees. Calling its `move()` method will transfer responsibility for calling the current registered disposers to a new `AsyncDisposableStack` and prevent registering any additional disposers.

See `DisposableStack` for general information about using disposable stacks.

Constructor

`AsyncDisposableStack()`

Creates a new `AsyncDisposableStack` object.

Instance properties

These properties are defined on `AsyncDisposableStack.prototype` and shared by all `AsyncDisposableStack` instances.

`AsyncDisposableStack.prototype.constructor`

The constructor function that created the instance object. For `AsyncDisposableStack` instances, the initial value is the `AsyncDisposableStack` constructor.

`AsyncDisposableStack.prototype.disposed`

Read-only. Returns `true` if the `AsyncDisposableStack` has been disposed, or `false` if not.

`AsyncDisposableStack.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncDisposableStack"`. This property is used in `Object.prototype.toString()`.

Instance methods

`AsyncDisposableStack.prototype.adopt()`

Registers a value that doesn't implement the async disposable protocol to the stack by providing a custom disposer function.

`AsyncDisposableStack.prototype.disposeAsync()`

Disposes this stack by calling all disposers registered to it in reverse order of registration.

`AsyncDisposableStack.prototype.defer()`

Takes a callback function to be called when the stack is disposed.

`AsyncDisposableStack.prototype.move()`

Creates a new `AsyncDisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

`AsyncDisposableStack.prototype.use()`

Registers a value that implements the async disposable protocol to the stack.

`AsyncDisposableStack.prototype[Symbol.asyncDispose]`

An alias for the `disposeAsync()` method.

See also

- Polyfill of `AsyncDisposableStack` in `core-js`
- JavaScript resource management
- `Symbol.asyncDispose`
- `await using`
- `DisposableStack`

AsyncDisposableStack.prototype.adopt()

The **`adopt()`** method of `AsyncDisposableStack` instances registers a value that doesn't implement the async disposable protocol to the stack by providing a custom disposer function.

See `DisposableStack.prototype.adopt()` for general information about the `adopt()` method.

Syntax

```
adopt(value, onDispose)
```

Parameters

`value`

Any value to be registered to the stack.

`onDispose`

A function that will be called when the stack is disposed. The function receives `value` as its only argument, and it can return a promise which gets awaited.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Using adopt()

This function creates a file handle (as a Node.js `FileHandle`), that gets closed when the function completes. We suppose that the file handle does not implement the async disposable protocol (in reality it does), so we use `adopt()` to register it to the stack. Because the `handle.close()` method returns a promise, we need to use an `AsyncDisposableStack` so that the disposal gets awaited.

```
async function readFile(path) {
  await using disposer = new AsyncDisposableStack();
  const handle = disposer.adopt(
    await fs.open(path),
    async (handle) => await handle.close(),
  );
  const data = await handle.read();
  // The handle.close() method is called and awaited here before exiting
  return data;
}
```

See also

- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.defer()`
- `AsyncDisposableStack.prototype.use()`

AsyncDisposableStack() constructor

The **`AsyncDisposableStack()`** constructor creates `AsyncDisposableStack` objects.

Syntax

```
new AsyncDisposableStack()
```

> **Note:** `AsyncDisposableStack()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

None.

Return value

A new `AsyncDisposableStack` object.

Examples

Creating an AsyncDisposableStack

```
const disposer = new AsyncDisposableStack();
disposer.defer(() => console.log("Disposed!"));
await disposer.disposeAsync();
// Logs: Disposed!
```

See also

- JavaScript resource management

AsyncDisposableStack.prototype.defer()

The **`defer()`** method of `AsyncDisposableStack` instances takes a callback function to be called and awaited when the stack is disposed.

See `DisposableStack.prototype.defer()` for general information about the `defer()` method.

Syntax

```
defer(onDispose)
```

Parameters

`onDispose`

A function that will be called when the stack is disposed. The function receives no arguments and can return a promise which gets awaited.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Using defer()

One use case of `defer()` is to do something unrelated to resource freeing during scope exit, such as logging a message.

```
async function doSomething() {
  await using disposer = new AsyncDisposableStack();
  disposer.defer(async () => {
    await fs.writeFile("log.txt", "All resources freed successfully");
  });
  // Other code that claims and frees more data
}
```

See also

- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.adopt()`
- `AsyncDisposableStack.prototype.use()`

AsyncDisposableStack.prototype.disposeAsync()

The **`disposeAsync()`** method of `AsyncDisposableStack` instances disposes this stack by calling all disposers registered to it in reverse order of registration, awaiting for each one's completion before calling the next one. If the stack is already disposed, this method does nothing.

It performs the same action as `await using disposer = new AsyncDisposableStack()` at scope exit. It can be used if you need to clean up at a point other than scope exit.

Syntax

```
disposeAsync()
```

Parameters

None.

Return value

A new `Promise` that resolves with `undefined` when all registered disposers have completed in sequence.

Exceptions

`disposeAsync()` never synchronously throws an error. The returned promise may reject with one of the following errors:

`SuppressedError`

Thrown if multiple disposers in the stack threw an error. If only one error is thrown, it is rethrown as-is. Otherwise, for each additional error, a new `SuppressedError` is created, with the original error as the `suppressed` property, and the new error as the `error` property.

Examples

Disposing a stack

Here we push three disposers to the stack, using the `use()`, `adopt()`, and `defer()` methods. When `disposeAsync()` is called, the disposers are called in reverse order of registration.

Note that usually you don't need to call `disposeAsync()` manually. Declare the stack with `await using`, and its `[Symbol.asyncDispose]()` method will be automatically called when the stack goes out of scope.

```
class Resource {
  #doDisposal() {
    // Imagine more meaningful disposal logic here
    return new Promise((resolve) => {
      setTimeout(resolve, 1000);
    });
  }
  async dispose() {
    await this.#doDisposal();
    console.log("Resource disposed");
  }
  async [Symbol.asyncDispose]() {
    await this.#doDisposal();
    console.log("Resource disposed via Symbol.asyncDispose");
  }
}

async function doSomething() {
  const disposer = new AsyncDisposableStack();
  const resource = disposer.use(new Resource());
  const resource2 = disposer.adopt(new Resource(), (resource) =>
```

```
    resource.dispose(),
  );
  disposer.defer(() => console.log("Deferred disposer"));
  disposer.disposeAsync();
  // Logs in order:
  // Deferred disposer
  // Resource disposed
  // Resource disposed via Symbol.dispose
}

doSomething();
```

See also

- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.adopt()`
- `AsyncDisposableStack.prototype.defer()`
- `AsyncDisposableStack.prototype.use()`
- `AsyncDisposableStack.prototype[Symbol.asyncDispose]()`

AsyncDisposableStack.prototype.disposed

The **`disposed`** accessor property of `AsyncDisposableStack` instances returns a boolean indicating whether or not this `AsyncDisposableStack` has been disposed or moved by doing any of the following:

- Calling its `disposeAsync()` method
- Calling its `move()` method
- Declaring it with `await using` and letting the variable go out of scope, which automatically calls the `[Symbol.asyncDispose]()` method.

Examples

Checking if a stack is disposed

```
const disposer = new AsyncDisposableStack();
console.log(disposer.disposed); // false
await disposer.disposeAsync();
console.log(disposer.disposed); // true
```

See also

- JavaScript resource management

AsyncDisposableStack.prototype.move()

The **`move()`** method of `AsyncDisposableStack` instances creates a new `AsyncDisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

Syntax

```
move()
```

Parameters

None.

Return value

A new `AsyncDisposableStack` instance.

Exceptions

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Claiming ownership of a stack

```
async function consumeStack(stack) {
  await using newStack = stack.move(); // newStack now owns the disposers
  console.log(stack.disposed); // true
  console.log(newStack.disposed); // false
  // newStack is disposed here immediately before the function exits
}

const stack = new AsyncDisposableStack();
console.log(stack.disposed); // false
await consumeStack(stack);
console.log(stack.disposed); // true
```

Allowing resources to be disposed within two code paths

The major use case of `move()` is when you have one or more resources which could either be disposed right here or could be persisted for later use. In this case, you can put the resources in an `AsyncDisposableStack` and then call `move()` when you need to persist the resources for later usage.

```
class PluginHost {
  #disposed = false;
  #disposables;
  #channel;
  #socket;

  static async init() {
    // Create an AsyncDisposableStack that is disposed when init exits.
    // If construction succeeds, we move everything out of `stack` and into
    // `#disposables` to be disposed later.
    await using stack = new AsyncDisposableStack();

    const channel = stack.use(await getChannel());

    const socket = stack.use(await getSocket());

    // If we made it here, then there were no errors during construction and
    // we can safely move the disposables out of `stack`.
    return new PluginHost(channel, socket, stack.move());

    // If construction failed, then `stack` would be disposed before reaching
    // the line above, which would dispose `channel` and `socket` in turn.
  }

  constructor(channel, socket, disposables) {
    this.#channel = channel;
    this.#socket = socket;
    this.#disposables = disposables;
  }

  async [Symbol.asyncDispose]() {
    if (this.#disposed) {
      return;
    }
    this.#disposed = true;
    // Put `this.#disposables` into a `using` variable, so it is automatically
    // disposed when the function exits.
    await using disposables = this.#disposables;

    // NOTE: we can free `#socket` and `#channel` here since they will be
    // disposed by the call to `disposables[Symbol.asyncDispose]()`, below.
    // This isn't strictly a requirement for every disposable, but is
    // good housekeeping since these objects will no longer be useable.
    this.#socket = undefined;
    this.#channel = undefined;
    this.#disposables = undefined;
  }
}
```

See also

- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.disposeAsync()`

AsyncDisposableStack.prototype[Symbol.asyncDispose]()

The **`[Symbol.asyncDispose]()`** method of `AsyncDisposableStack` instances implements the *async disposable protocol* and allows it to be disposed when used with `await using`. It is an alias for the `disposeAsync()` method.

Syntax

```
asyncDisposableStack[Symbol.asyncDispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring a stack with <code>await using</code>

The `Symbol.asyncDispose` method is intended to be automatically called in an `await using` declaration.

```
async function doSomething() {
  await using disposer = new AsyncDisposableStack();
  const resource = disposer.use(new Resource());
```

```
  resource.doSomething();
  // disposer is disposed here immediately before the function exits
  // which causes the resource to be disposed
}
```

See also

- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.disposeAsync()`

AsyncDisposableStack.prototype.use()

The **`use()`** method of `AsyncDisposableStack` instances registers a value that implements the async disposable protocol to the stack.

See `DisposableStack.prototype.use()` for general information about the `use()` method.

Syntax

```
use(value)
```

Parameters

`value`

The value to register to the stack. Must either contain a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method, or be `null` or `undefined`.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `value` is not `null` or `undefined`, and does not contain a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method.

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Using use()

This function reads a file (as a Node.js `FileHandle`) and returns its contents. The file handle is automatically closed when the function completes, given that the `FileHandle` class implements a `[Symbol.asyncDispose]()` method that asynchronously closes the file.

```
async function readFileContents(path) {
  await using disposer = new AsyncDisposableStack();
  const handle = disposer.use(await fs.open(path));
  const data = await handle.read();
  return data;
  // The disposer is disposed here, which causes handle to be closed too
}
```

See also

- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.adopt()`
- `AsyncDisposableStack.prototype.defer()`

AsyncFunction

The **`AsyncFunction`** object provides methods for async functions. In JavaScript, every async function is actually an `AsyncFunction` object.

Note that `AsyncFunction` is *not* a global object. It can be obtained with the following code:

```
const AsyncFunction = async function () {}.constructor;
```

`AsyncFunction` is a subclass of `Function`.

Constructor

`AsyncFunction()`

Creates a new `AsyncFunction` object.

Instance properties

Also inherits instance properties from its parent `Function`.

These properties are defined on `AsyncFunction.prototype` and shared by all `AsyncFunction` instances.

`AsyncFunction.prototype.constructor`

The constructor function that created the instance object. For `AsyncFunction` instances, the initial value is the `AsyncFunction` constructor.

`AsyncFunction.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncFunction"`. This property is used in `Object.prototype.toString()`.

> **Note:** `AsyncFunction` instances do not have the `prototype` property.

Instance methods

Inherits instance methods from its parent `Function`.

See also

- `async function`
- `async function` expression
- `Function`
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Functions

AsyncFunction() constructor

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations in the `Function()` constructor reference for more information.

The **`AsyncFunction()`** constructor creates `AsyncFunction` objects.

Note that `AsyncFunction` is *not* a global object. It can be obtained with the following code:

```
const AsyncFunction = async function () {}.constructor;
```

The `AsyncFunction()` constructor is not intended to be used directly, and all caveats mentioned in the `Function()` description apply to `AsyncFunction()`.

Syntax

```
new AsyncFunction(functionBody)
new AsyncFunction(arg1, functionBody)
new AsyncFunction(arg1, arg2, functionBody)
new AsyncFunction(arg1, arg2, /* …, */ argN, functionBody)

AsyncFunction(functionBody)
AsyncFunction(arg1, functionBody)
AsyncFunction(arg1, arg2, functionBody)
AsyncFunction(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `AsyncFunction()` can be called with or without `new`. Both create a new `AsyncFunction` instance.

Parameters

See `Function()`.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Creating an async function from an AsyncFunction() constructor

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

const AsyncFunction = async function () {}.constructor;

const fn = new AsyncFunction(
  "a",
```

```
  "b",
  "return await resolveAfter2Seconds(a) + await resolveAfter2Seconds(b);",
);

fn(10, 20).then((v) => {
  console.log(v); // prints 30 after 4 seconds
});
```

See also

- `async function`
- `async function` expression
- `Function()` constructor

AsyncGenerator

The **`AsyncGenerator`** object is returned by an async generator function and it conforms to both the async iterable protocol and the async iterator protocol.

Async generator methods always yield `Promise` objects.

`AsyncGenerator` is a subclass of the hidden `AsyncIterator` class.

Constructor

There's no JavaScript entity that corresponds to the `AsyncGenerator` constructor. Instances of `AsyncGenerator` must be returned from async generator functions:

```
async function* createAsyncGenerator() {
  yield Promise.resolve(1);
  yield await Promise.resolve(2);
  yield 3;
}
const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res.value)); // 1
asyncGen.next().then((res) => console.log(res.value)); // 2
asyncGen.next().then((res) => console.log(res.value)); // 3
```

There's only a hidden object which is the prototype object shared by all objects created by async generator functions. This object is often stylized as `AsyncGenerator.prototype` to make it look like a class, but it should be more appropriately called `AsyncGeneratorFunction.prototype.prototype`, because `AsyncGeneratorFunction` is an actual JavaScript entity. To understand the prototype chain of `AsyncGenerator` instances, see `AsyncGeneratorFunction.prototype.prototype`.

Instance properties

These properties are defined on `AsyncGenerator.prototype` and shared by all `AsyncGenerator` instances.

`AsyncGenerator.prototype.constructor`

> The constructor function that created the instance object. For `AsyncGenerator` instances, the initial value is `AsyncGeneratorFunction.prototype`.
>
> > **Note:** `AsyncGenerator` objects do not store a reference to the async generator function that created them.

`AsyncGenerator.prototype[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncGenerator"`. This property is used in `Object.prototype.toString()`.

Instance methods

Also inherits instance methods from its parent `AsyncIterator`.

`AsyncGenerator.prototype.next()`

> Returns a `Promise` which will be resolved with the given value yielded by the `yield` expression.

`AsyncGenerator.prototype.return()`

> Acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

`AsyncGenerator.prototype.throw()`

> Acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Examples

Async generator iteration

The following example iterates over an async generator, logging values 1–6 to the console at decreasing time intervals. Notice how each time a Promise is yielded, but it's automatically resolved within the `for await...of` loop.

```
// An async task. Pretend it's doing something more useful
// in practice.
function delayedValue(time, value) {
```

```
  return new Promise((resolve /*, reject */) => {
    setTimeout(() => resolve(value), time);
  });
}

async function* generate() {
  yield delayedValue(2000, 1);
  yield delayedValue(1000, 2);
  yield delayedValue(500, 3);
  yield delayedValue(250, 4);
  yield delayedValue(125, 5);
  yield delayedValue(50, 6);
  console.log("All done!");
}

async function main() {
  for await (const value of generate()) {
    console.log("value", value);
  }
}

main().catch((e) => console.error(e));
```

See also

- function*
- async function*
- `function*` expression
- Generator Function
- Async Generator Function
- Iterators and generators guide

AsyncGenerator.prototype.next()

The **`next()`** method of `AsyncGenerator` instances returns the next value in the sequence.

Syntax

```
next()
next(value)
```

Parameters

`value` Optional

An optional value used to modify the internal state of the generator. A value passed to the `next()` method will be received by `yield`

Return value

A `Promise` which when resolved returns an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator is past the end of its control flow. In this case `value` specifies the *return value* of the generator (which may be undefined).
- `false` if the generator is able to produce more values.

`value`

Any JavaScript value yielded or returned by the generator.

Examples

Using next()

The following example shows a generator and the object that the `next` method returns:

```
// An async task. Pretend it's doing something more useful
// in practice.
function delayedValue(time, value) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(value), time);
  });
}

async function* createAsyncGenerator() {
  yield delayedValue(500, 1);
  yield delayedValue(500, 2);
  yield delayedValue(500, 3);
}

const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res)); // { value: 1, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 2, done: false }
```

```
asyncGen.next().then((res) => console.log(res)); // { value: 3, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: undefined, done: true }
```

Sending values to the generator

In this example, `next` is called with a value.

> **Note:** The first call does not log anything, because the generator was not yielding anything initially.

```
// An async task. Pretend it's doing something more useful
// in practice.
function sleep(time) {
  return new Promise((resolve, reject) => {
    setTimeout(resolve, time);
  });
}

async function* createAsyncGenerator() {
  while (true) {
    await sleep(500);
    const value = yield;
    console.log(value);
  }
}

async function main() {
  const asyncGen = createAsyncGenerator();
  // No log at this step: the first value sent through `next` is lost
  console.log(await asyncGen.next(1)); // { value: undefined, done: false }
  // Logs 2: the value sent through `next`
  console.log(await asyncGen.next(2)); // { value: undefined, done: false }
}

main();
```

See also

- `async function*`
- Iterators and generators guide

AsyncGenerator.prototype.return()

The **`return()`** method of `AsyncGenerator` instances acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

Syntax

```
asyncGeneratorInstance.return()
asyncGeneratorInstance.return(value)
```

Parameters

`value` Optional

The value to return.

Return value

A `Promise` which resolves with an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function's control flow hasn't reached the end and can produce more values. This can only happen if the `return` is captured in a `try...finally` and there are more `yield` expressions in the `finally` block.

`value`

The value that is given as an argument, or, if the `yield` expression is wrapped in a `try...finally`, the value yielded/returned from the `finally` block.

Description

The `return()` method, when called, can be seen as if a `return value;` statement is inserted in the generator's body at the current suspended position, where `value` is the value passed to the `return()` method. Therefore, in a typical flow, calling `return(value)` will return `{ done: true, value: value }`. However, if the `yield` expression is wrapped in a `try...finally` block, the control flow doesn't exit the function body, but proceeds to the `finally` block instead. In this case, the value returned may be different, and `done` may even be `false`, if there are more `yield` expressions within the `finally` block.

Examples

Using return()

The following example shows an async generator and the `return` method.

```
// An async task. Pretend it's doing something more useful
// in practice.
function delayedValue(time, value) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(value), time);
  });
}

async function* createAsyncGenerator() {
  yield delayedValue(500, 1);
  yield delayedValue(500, 2);
  yield delayedValue(500, 3);
}

const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res)); // { value: 1, done: false }
asyncGen.return("foo").then((res) => console.log(res)); // { value: "foo", done: true }
asyncGen.next().then((res) => console.log(res)); // { value: undefined, done: true }
```

Using return() once a generator is complete

If no `value` argument is passed into the `return()` method, the promise will resolve as if the next() method has been called. In this example the generator has completed, so the value returned is `undefined`.

`return()` can still be called after the generator is in a "completed" state, however the generator will stay in this state.

```
async function* createAsyncGenerator() {
  yield Promise.resolve(1);
  yield await Promise.resolve(2);
  yield 3;
}
const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res)); // { value: 1, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 2, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 3, done: false }
// value is returned undefined, as no value is passed and generator is 'done'
asyncGen.return().then((res) => console.log(res)); // { value: undefined, done: true }
// we can still return a value once the generator is complete
asyncGen.return(1).then((res) => console.log(res)); // { value: 1, done: true }
```

See also

- `async function*`
- Iterators and generators guide

AsyncGenerator.prototype.throw()

The **`throw()`** method of `AsyncGenerator` instances acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Syntax

```
asyncGeneratorInstance.throw(exception)
```

Parameters

`exception`

The exception to throw. For debugging purposes, it is useful to make it an `instanceof` `Error`.

Return value

If the thrown error is not caught, it will return a `Promise` which rejects with the exception passed in.

If the exception is caught by a `try...catch` and the generator resumes to yield more values, it will return a `Promise` which resolves with an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function is able to produce more values.

`value`

The value yielded from the next `yield` expression.

Examples

Using throw()

The following example shows a generator and an error that is thrown using the `throw` method. An error can be caught by a `try...catch` block as usual.

```
// An async task. Pretend it's doing something more useful
// in practice.
function sleep(time) {
  return new Promise((resolve, reject) => {
    setTimeout(resolve, time);
  });
}

async function* createAsyncGenerator() {
  while (true) {
    try {
      await sleep(500);
      yield 42;
    } catch (e) {
      console.error(e);
    }
  }
}

const asyncGen = createAsyncGenerator();
asyncGen.next(1).then((res) => console.log(res)); // { value: 42, done: false }
asyncGen
  .throw(new Error("Something went wrong")) // Error: Something went wrong
  .then((res) => console.log(res)); // { value: 42, done: false }
```

See also

- `async function*`
- Iterators and generators guide

AsyncGeneratorFunction

The **`AsyncGeneratorFunction`** object provides methods for async generator functions. In JavaScript, every async generator function is actually an `AsyncGeneratorFunction` object.

Note that `AsyncGeneratorFunction` is *not* a global object. It can be obtained with the following code:

```
const AsyncGeneratorFunction = async function* () {}.constructor;
```

`AsyncGeneratorFunction` is a subclass of `Function`.

Try it

```
const AsyncGeneratorFunction = async function* () {}.constructor;

const foo = new AsyncGeneratorFunction(`
  yield await Promise.resolve('a');
  yield await Promise.resolve('b');
  yield await Promise.resolve('c');
`);

let str = "";

async function generate() {
  for await (const val of foo()) {
    str += val;
  }
  console.log(str);
}

generate();
// Expected output: "abc"
```

Constructor

`AsyncGeneratorFunction()`

Creates a new `AsyncGeneratorFunction` object.

Instance properties

Also inherits instance properties from its parent `Function`.

These properties are defined on `AsyncGeneratorFunction.prototype` and shared by all `AsyncGeneratorFunction` instances.

`AsyncGeneratorFunction.prototype.constructor`

The constructor function that created the instance object. For `AsyncGeneratorFunction` instances, the initial value is the `AsyncGeneratorFunction` constructor.

`AsyncGeneratorFunction.prototype.prototype`

All async generator functions share the same `prototype` property, which is `AsyncGenerator.prototype`. Each async generator function created with the `async function*` syntax or the `AsyncGeneratorFunction()` constructor also has its own `prototype` property, whose prototype is

`AsyncGeneratorFunction.prototype.prototype`. When the async generator function is called, its `prototype` property becomes the prototype of the returned async generator object.

`AsyncGeneratorFunction.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncGeneratorFunction"`. This property is used in `Object.prototype.toString()`.

These properties are own properties of each `AsyncGeneratorFunction` instance.

`prototype`

Used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

Instance methods

Inherits instance methods from its parent `Function`.

See also

- `async function*`
- `async function*` expression
- `Function`
- `AsyncFunction`
- `GeneratorFunction`
- Functions

AsyncGeneratorFunction() constructor

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations in the `Function()` constructor reference for more information.

The **`AsyncGeneratorFunction()`** constructor creates `AsyncGeneratorFunction` objects.

Note that `AsyncGeneratorFunction` is not a global object. It could be obtained by evaluating the following code.

```
const AsyncGeneratorFunction = async function* () {}.constructor;
```

The `AsyncGeneratorFunction()` constructor is not intended to be used directly, and all caveats mentioned in the `Function()` description apply to `AsyncGeneratorFunction()`.

Syntax

```
new AsyncGeneratorFunction(functionBody)
new AsyncGeneratorFunction(arg1, functionBody)
new AsyncGeneratorFunction(arg1, arg2, functionBody)
new AsyncGeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)

AsyncGeneratorFunction(functionBody)
AsyncGeneratorFunction(arg1, functionBody)
AsyncGeneratorFunction(arg1, arg2, functionBody)
AsyncGeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `AsyncGeneratorFunction()` can be called with or without `new`. Both create a new `AsyncGeneratorFunction` instance.

Parameters

See `Function()`.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Using the constructor

The following example uses the `AsyncGeneratorFunction` constructor to create an async generator function.

```
const AsyncGeneratorFunction = async function* () {}.constructor;
const createAsyncGenerator = new AsyncGeneratorFunction("a", "yield a * 2");
const asyncGen = createAsyncGenerator(10);
asyncGen.next().then((res) => console.log(res.value)); // 20
```

See also

- `async function*`
- `async function*` expression

- `Function()` constructor
- Iterators and generators guide
- Functions

AsyncGeneratorFunction.prototype.prototype

The **`prototype`** property of `AsyncGeneratorFunction.prototype` is shared by all async generator functions. Its value is `AsyncGenerator.prototype`. Each async generator function created with the `async function*` syntax or the `AsyncGeneratorFunction()` constructor also has its own `prototype` property, whose prototype is `AsyncGeneratorFunction.prototype.prototype`. When the async generator function is called, its `prototype` property becomes the prototype of the returned async generator object.

Value

The same object as `AsyncGenerator.prototype`. `AsyncGeneratorFunction.prototype.prototype` is the technically more accurate name, but `AsyncGenerator.prototype` appeals to the intuition that it's the prototype of async generator objects.

Property attributes of `AsyncGeneratorFunction.prototype.prototype`	
Writable	no
Enumerable	no
Configurable	yes

The `prototype` property of each `AsyncGeneratorFunction` instance is an empty object with no properties, whose prototype is `AsyncGeneratorFunction.prototype.prototype`. It has the following property attributes:

Property attributes of `AsyncGeneratorFunction.prototype.prototype`	
Writable	yes
Enumerable	no
Configurable	no

Description

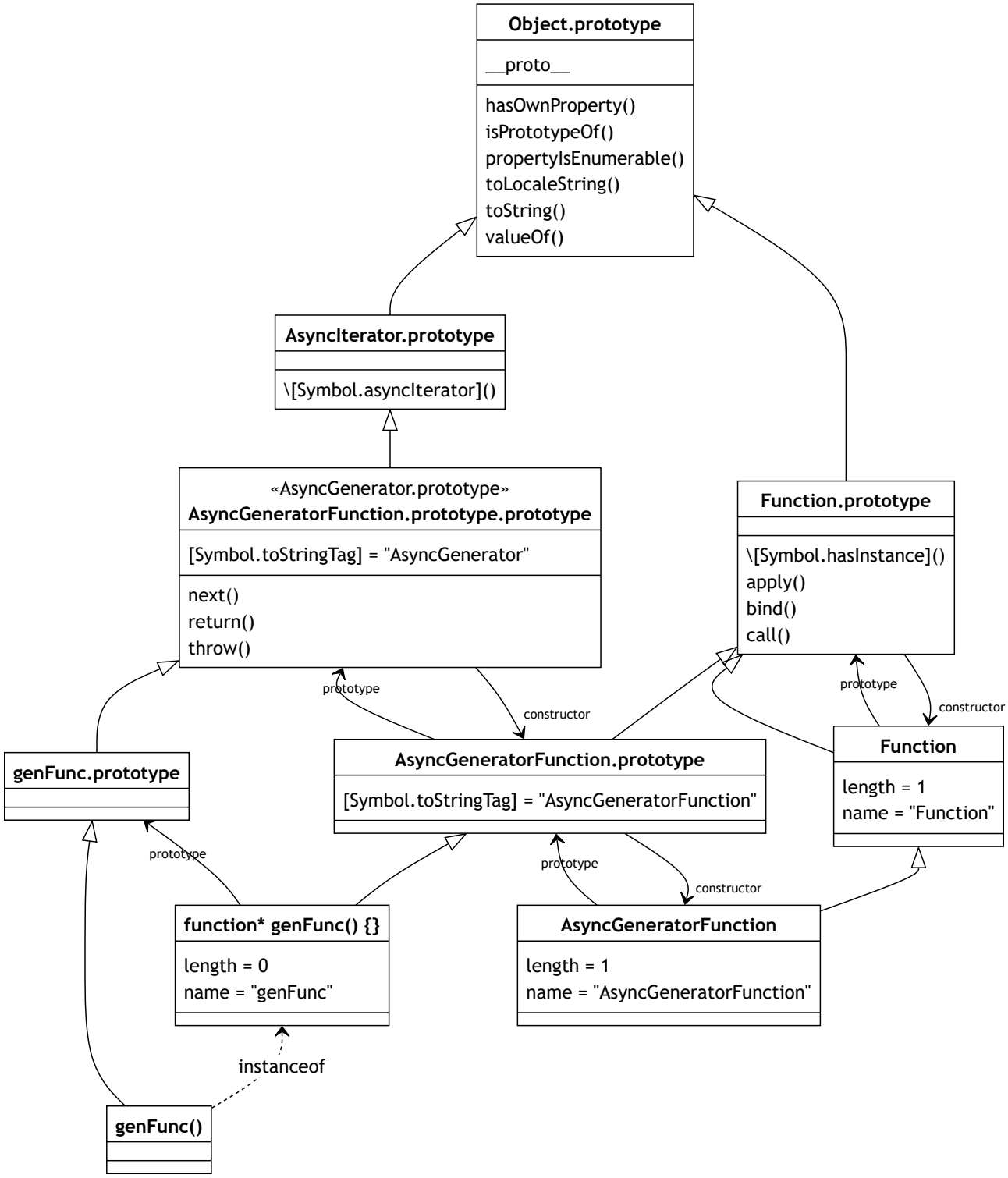
An async generator function instance has two `prototype` properties. The first one is its own `prototype` property. The second one is the `prototype` property on its prototype, which is `AsyncGeneratorFunction.prototype`. (Remember that every async generator function is an instance of `AsyncGeneratorFunction`, so it has `AsyncGeneratorFunction.prototype` as its prototype.)

```
async function* genFunc() {}
const AsyncGeneratorFunctionPrototype = Object.getPrototypeOf(genFunc);
console.log(Object.hasOwn(genFunc, "prototype")); // true
console.log(Object.hasOwn(AsyncGeneratorFunctionPrototype, "prototype")); // true
```

When an async generator function is called, the async generator function's `prototype` property becomes the prototype of the returned async generator object.

```
const gen = genFunc();
const proto = Object.getPrototypeOf;
console.log(proto(gen) === genFunc.prototype); // true
console.log(proto(proto(gen)) === AsyncGeneratorFunctionPrototype.prototype); // true
```

The following diagram illustrates the prototype chain of an async generator function and its instances. Each hollow arrow indicates an inheritance relationship (i.e., a prototype link), and each solid arrow indicates a property relationship. Note that there's no way to access `genFunc` from `gen` — they only have an `instanceof` relationship.



See also

- `async function*`
- `async function*` expression
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Inheritance and the prototype chain
- Iterators and generators

AsyncIterator

An **`AsyncIterator`** object is an object that conforms to the async iterator protocol by providing a `next()` method that returns a promise fulfilling to an iterator result object. The `AsyncIterator.prototype` object is a hidden global object that all built-in async iterators inherit from. It provides a `[Symbol.asyncIterator]()` method that returns the async iterator object itself, making the async iterator also async iterable.

Note that `AsyncIterator` is *not* a global object, although it will be in the future with the async iterator helpers proposal. The `AsyncIterator.prototype` object shared by all built-in async iterators can be obtained with the following code:

```
const AsyncIteratorPrototype = Object.getPrototypeOf(
  Object.getPrototypeOf(Object.getPrototypeOf((async function* () {})())),
);
```

Description

Currently, the only built-in JavaScript async iterator is the `AsyncGenerator` object returned by async generator functions. There are some other built-in async iterators in web API, such as the one of a `ReadableStream`.

Each of these async iterators have a distinct prototype object, which defines the `next()` method used by the particular async iterator. All of these prototype objects inherit from `AsyncIterator.prototype`, which provides a `[Symbol.asyncIterator]()` method that returns the async iterator object itself, making the async iterator also async iterable.

> **Note:** `AsyncIterator.prototype` does not implement `[Symbol.iterator]()`, so async iterators are not sync iterable by default.

Instance methods

`AsyncIterator.prototype[Symbol.asyncDispose]()`

Calls and awaits the `return()` method of `this`, if it exists. This implements the *async disposable protocol* and allows it to be disposed when used with `await using`.

`AsyncIterator.prototype[Symbol.asyncIterator]()`

Returns the async iterator object itself. This allows async iterator objects to also be async iterable.

Examples

Using an async iterator as an async iterable

All built-in async iterators are also async iterable, so you can use them in a `for await...of` loop:

```
const asyncIterator = (async function* () {
  yield 1;
  yield 2;
  yield 3;
})();
(async () => {
  for await (const value of asyncIterator) {
    console.log(value);
  }
})();
// Logs: 1, 2, 3
```

See also

- `async function*`
- Iteration protocols

AsyncIterator.prototype[Symbol.asyncDispose]()

The **`[Symbol.asyncDispose]()`** method of `AsyncIterator` instances implements the *async disposable protocol* and allows it to be disposed when used with `await using`. It calls and awaits the `return()` method of `this`, if it exists.

Syntax

```
asyncIterator[Symbol.asyncDispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring an async iterator with <code>await using</code>

The `Symbol.asyncDispose` method is intended to be automatically called in an `await using` declaration. This is useful if you have an async iterator that you manually iterate over by calling its `next()` method; if you iterate it with `for await...of` or something similar, then error handling and cleanup is done automatically.

```
async function* generateNumbers() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("Cleaning up");
  }
}
```

```
async function doSomething() {
  await using numbers = generateNumbers();
  const res1 = await numbers.next();
  // Not iterating the rest of the numbers
  // Before the function exits, the async iterator is disposed
  // Logs "Cleaning up"
}

doSomething();
```

See also

- Polyfill of `AsyncIterator.prototype[Symbol.asyncDispose]` in `core-js`
- JavaScript resource management
- `Symbol.asyncDispose`
- `await using`

AsyncIterator.prototype[Symbol.asyncIterator]()

The **`[Symbol.asyncIterator]()`** method of `AsyncIterator` instances implements the async iterable protocol and allows built-in async iterators to be consumed by most syntaxes expecting async iterables, such as `for await...of` loops. It returns the value of `this`, which is the async iterator object itself.

Syntax

```
asyncIterator[Symbol.asyncIterator]()
```

Parameters

None.

Return value

The value of `this`, which is the async iterator object itself.

Examples

Iteration using for await...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.asyncIterator]()` method makes all built-in async iterators async iterable, and iterating syntaxes like the `for await...of` loop automatically calls this method to obtain the async iterator to loop over.

```
const asyncIterator = (async function* () {
  yield 1;
  yield 2;
  yield 3;
})();
(async () => {
  for await (const value of asyncIterator) {
    console.log(value);
  }
})();
// Logs: 1, 2, 3
```

See also

- `for await...of`

Atomics

The **`Atomics`** namespace object contains static methods for carrying out atomic operations. They are used with `SharedArrayBuffer` and `ArrayBuffer` objects.

Description

Unlike most global objects, `Atomics` is not a constructor. You cannot use it with the `new` operator or invoke the `Atomics` object as a function. All properties and methods of `Atomics` are static (just like the `Math` object).

Atomic operations

When memory is shared, multiple threads can read and write the same data in memory. Atomic operations make sure that predictable values are written and read, that operations are finished before the next operation starts and that operations are not interrupted.

Wait and notify

The `wait()` and `notify()` methods are modeled on Linux futexes ("fast user-space mutex") and provide ways for waiting until a certain condition becomes true and are typically used as blocking constructs.

Static properties

`Atomics[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"Atomics"`. This property is used in `Object.prototype.toString()`.

Static methods

`Atomics.add()`

Adds the provided value to the existing value at the specified index of the array. Returns the old value at that index.

`Atomics.and()`

Computes a bitwise AND on the value at the specified index of the array with the provided value. Returns the old value at that index.

`Atomics.compareExchange()`

Stores a value at the specified index of the array, if it equals a value. Returns the old value.

`Atomics.exchange()`

Stores a value at the specified index of the array. Returns the old value.

`Atomics.isLockFree()`

An optimization primitive that can be used to determine whether to use locks or atomic operations. Returns `true` if an atomic operation on arrays of the given element size will be implemented using a hardware atomic operation (as opposed to a lock). Experts only.

`Atomics.load()`

Returns the value at the specified index of the array.

`Atomics.notify()`

Notifies agents that are waiting on the specified index of the array. Returns the number of agents that were notified.

`Atomics.or()`

Computes a bitwise OR on the value at the specified index of the array with the provided value. Returns the old value at that index.

`Atomics.pause()`

Provides a micro-wait primitive that hints to the CPU that the caller is spinning while waiting on access to a shared resource. This allows the system to reduce the resources allocated to the core (such as power) or thread, without yielding the current thread.

`Atomics.store()`

Stores a value at the specified index of the array. Returns the value.

`Atomics.sub()`

Subtracts a value at the specified index of the array. Returns the old value at that index.

`Atomics.wait()`

Verifies that the specified index of the array still contains a value and sleeps awaiting or times out. Returns either `"ok"`, `"not-equal"`, or `"timed-out"`. If waiting is not allowed in the calling agent then it throws an exception. (Most browsers will not allow `wait()` on the browser's main thread.)

`Atomics.waitAsync()`

Waits asynchronously (i.e., without blocking, unlike `Atomics.wait`) on a shared memory location and returns an object representing the result of the operation.

`Atomics.xor()`

Computes a bitwise XOR on the value at the specified index of the array with the provided value. Returns the old value at that index.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics

```
const sab = new SharedArrayBuffer(1024);
const ta = new Uint8Array(sab);

ta[0]; // 0
ta[0] = 5; // 5

Atomics.add(ta, 0, 12); // 5
Atomics.load(ta, 0); // 17

Atomics.and(ta, 0, 1); // 17
Atomics.load(ta, 0); // 1

Atomics.compareExchange(ta, 0, 5, 12); // 1
Atomics.load(ta, 0); // 1

Atomics.exchange(ta, 0, 12); // 1
Atomics.load(ta, 0); // 12
```

```
Atomics.isLockFree(1); // true
Atomics.isLockFree(2); // true
Atomics.isLockFree(3); // false
Atomics.isLockFree(4); // true

Atomics.or(ta, 0, 1); // 12
Atomics.load(ta, 0); // 13

Atomics.store(ta, 0, 12); // 12

Atomics.sub(ta, 0, 2); // 12
Atomics.load(ta, 0); // 10

Atomics.xor(ta, 0, 1); // 10
Atomics.load(ta, 0); // 11
```

Waiting and notifying

Given a shared `Int32Array`:

```
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 because the provided value matches what is stored at the provided index. The reading thread will not move on until the writing thread has called `Atomics.notify()` on position 0 of the provided typed array. Note that if, after being woken up, the value of location 0 has not been changed by the writing thread, the reading thread will **not** go back to sleep, but will continue on.

```
Atomics.wait(int32, 0, 0);
console.log(int32[0]); // 123
```

A writing thread stores a new value and notifies the waiting thread once it has written:

```
console.log(int32[0]); // 0;
Atomics.store(int32, 0, 123);
Atomics.notify(int32, 0, 1);
```

See also

- `ArrayBuffer`
- JavaScript typed arrays guide
- Web Workers
- Shared Memory – a brief tutorial in the TC39 ecmascript-sharedmem proposal
- A Taste of JavaScript's New Parallel Primitives on hacks.mozilla.org (2016)

Atomics.add()

The **`Atomics.add()`** static method adds a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```
Atomics.add(typedArray, index, value)
```

Parameters

`typedArray`

: An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

: The position in the `typedArray` to add a `value` to.

`value`

: The number to add.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

: Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

: Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.add()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 + 12 = 19
console.log(Atomics.add(ta, 0, 12)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 19, the new/current value
```

See also

- `Atomics`
- `Atomics.sub()`

Atomics.and()

The **`Atomics.and()`** static method computes a bitwise AND with a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```
Atomics.and(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to compute the bitwise AND.

`value`

The number to compute the bitwise AND with.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Description

The bitwise AND operation only yields 1, if both `a` and `b` are 1. The truth table for the AND operation is:

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

For example, a bitwise AND of `5 & 1` results in `0001` which is 1 in decimal.

```
5  0101
1  0001
   ----
1  0001
```

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.and()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 (0111) AND 10 (1010) = 2 (0010)
console.log(Atomics.and(ta, 0, 10)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 2, the new/current value
```

See also

- `Atomics`
- `Atomics.or()`
- `Atomics.xor()`

Atomics.compareExchange()

The **`Atomics.compareExchange()`** static method exchanges a given replacement value at a given position in the array, if a given expected value equals the old value. It returns the old value at that position whether it was equal to the expected value or not. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```
Atomics.compareExchange(typedArray, index, expectedValue, replacementValue)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to exchange a `replacementValue`.

`expectedValue`

The value to check for equality.

`replacementValue`

The number to exchange.

Return value

The old value at the given position (`typedArray[index]`). If the return value is equal to `expectedValue`, the exchange was successful; otherwise, the exchange failed.

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.compareExchange()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

console.log(Atomics.compareExchange(ta, 0, 7, 12)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 12, the new/current value
```

Checking the return value

Compare-and-swap guarantees that the new value is calculated based on up-to-date information; if the value had been updated by another thread in the meantime, the write would fail. Therefore, you should check the return value of `compareExchange()` to check if it has failed, and retry if necessary.

Here is one example of an atomic adder (same functionality as `Atomics.add()`), adapted from the linked Wikipedia article:

```
function add(mem, index, a) {
  let done = false;
```

```
  while (!done) {
    const value = Atomics.load(mem, index);
    done = Atomics.compareExchange(mem, index, value, value + a) === value;
  }
  return value + a;
}
```

It first reads the value at the given index, then tries to update it with the new value. It keeps retrying until it successfully updates the value.

See also

- `Atomics`
- `Atomics.exchange()`

Atomics.exchange()

The **`Atomics.exchange()`** static method exchanges a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens between the read of the old value and the write of the new value.

Syntax

```
Atomics.exchange(typedArray, index, value)
```

Parameters

`typedArray`

: An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

: The position in the `typedArray` to exchange a `value`.

`value`

: The number to exchange.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

: Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

: Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.exchange()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

console.log(Atomics.exchange(ta, 0, 12)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 12, the new/current value
```

See also

- `Atomics`
- `Atomics.compareExchange()`
- `Atomics.store()`

Atomics.isLockFree()

The **`Atomics.isLockFree()`** static method is used to determine whether the `Atomics` methods use locks or atomic hardware operations when applied to typed arrays with the given element byte size. It is intended as an optimization primitive, so that high-performance algorithms can determine whether to use locks or atomic operations in critical sections. If an atomic primitive is not lock-free, it is often more efficient for an algorithm to provide its own locking.

Try it

```
console.log(Atomics.isLockFree(3));
// 3 is not one of the BYTES_PER_ELEMENT values
// Expected output: false
```

```
console.log(Atomics.isLockFree(4));
// 4 is one of the BYTES_PER_ELEMENT values
// Expected output: true
```

Syntax

```
Atomics.isLockFree(size)
```

Parameters

`size`

The size in bytes to check.

Return value

A `true` or `false` value indicating whether the operation is lock free.

- Always `true` if `size` is 4, because all known platforms support 4-byte atomic operations.
- Always `false` if the given size is not one of the `BYTES_PER_ELEMENT` property of integer TypedArray types.

Examples

Using Atomics.isLockFree()

```
Atomics.isLockFree(1); // true (platform-dependent)
Atomics.isLockFree(2); // true (platform-dependent)
Atomics.isLockFree(3); // false
Atomics.isLockFree(4); // true
Atomics.isLockFree(5); // false
Atomics.isLockFree(6); // false
Atomics.isLockFree(7); // false
Atomics.isLockFree(8); // true (platform-dependent)
```

See also

- `Atomics`

Atomics.load()

The **`Atomics.load()`** static method returns a value at a given position in the array. This atomic operation guarantees that the read is tear-free, and that all atomic reads are sequentially consistent.

Syntax

```
Atomics.load(typedArray, index)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to load from.

Return value

The value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.load()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;
```

```
Atomics.add(ta, 0, 12); // Add 12 to index 0
console.log(Atomics.load(ta, 0)); // 19, the new/current value
```

See also

- `Atomics`
- `Atomics.store()`

Atomics.notify()

The **`Atomics.notify()`** static method notifies up some agents that are sleeping in the wait queue.

> **Note:** This operation only works with an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`. It will return `0` on non-shared `ArrayBuffer` objects.

Syntax

```
Atomics.notify(typedArray, index, count)
```

Parameters

`typedArray`

: An `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`index`

: The position in the `typedArray` to wake up on.

`count` Optional

: The number of sleeping agents to notify. Defaults to `Infinity`.

Return value

Returns the number of woken up agents, or `0` if `typedArray` is a view on a non-shared `ArrayBuffer`.

Exceptions

`TypeError`

: Thrown if `typedArray` is not an `Int32Array` or `BigInt64Array`.

`RangeError`

: Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.notify()

Given a shared `Int32Array`:

```
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 because the provided `value` matches what is stored at the provided `index`. The reading thread will not move on until the writing thread has called `Atomics.notify()` on position 0 of the provided `typedArray`. Note that if, after being woken up, the value of location 0 has not been changed by the writing thread, the reading thread will **not** go back to sleep, but will continue on.

```
Atomics.wait(int32, 0, 0);
console.log(int32[0]); // 123
```

A writing thread stores a new value and notifies the waiting thread once it has written:

```
console.log(int32[0]); // 0;
Atomics.store(int32, 0, 123);
Atomics.notify(int32, 0, 1);
```

See also

- `Atomics`
- `Atomics.wait()`
- `Atomics.waitAsync()`

Atomics.or()

The **`Atomics.or()`** static method computes a bitwise OR with a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```
Atomics.or(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to compute the bitwise OR.

`value`

The number to compute the bitwise OR with.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Description

The bitwise OR operation yields 1, if either `a` or `b` is 1. The truth table for the OR operation is:

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

For example, a bitwise OR of `5 | 1` results in `0101` which is 5 in decimal.

```
5  0101
1  0001
   ----
5  0101
```

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.or()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 (0111) OR 10 (1010) = 15 (1111)
console.log(Atomics.or(ta, 0, 10)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 15, the new/current value
```

See also

- `Atomics`
- `Atomics.and()`
- `Atomics.xor()`

Atomics.pause()

The **`Atomics.pause()`** static method provides a micro-wait primitive that hints to the CPU that the caller is spinning while waiting on access to a shared resource. This allows the system to reduce the resources allocated to the core (such as power) or thread, without yielding the current thread.

`pause()` has no observable behavior other than timing. The exact behavior is dependent on the CPU architecture and the operating system. For example, in Intel x86, it may be a `pause` instruction as per Intel's optimization manual. It could be a no-op in certain platforms.

Syntax

```
Atomics.pause()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.pause()

Calling `Atomics.wait()` or `Atomics.waitAsync()` in order to wait for access to shared memory causes the thread to be scheduled out of the core and then back in again after the wait. This is efficient during times of high contention, where access to the shared memory could take some time. When contention is low, then it is often more efficient to poll on the lock without yielding the thread: this approach is known as busy waiting or spinlocking. The `pause()` method allows you to spinlock more efficiently while waiting, by providing a hint to the CPU about what the thread is doing, and hence its low need for resources.

To cater for both conditions, a common approach is to first spinlock in the hope that contention is low, and then wait if the lock is not gained after a short time. If we acquired the lock via spinlocking already, then the `wait()` call will be a no-op.

The example below shows how this approach can be used with `Atomics.pause()` and `Atomics.wait()`.

> **Warning:** Using spinlocking on the main thread is not recommended, as it will freeze the entire page. In general, unless designed very carefully, spinlocks may not actually be more performant than a regular wait.

```
// Imagine another thread also has access to this shared memory
const sab = new SharedArrayBuffer(1024);
const i32 = new Int32Array(sab);

// Fast path: spin the CPU for a short while
let spin = 0;
do {
  if (Atomics.compareExchange(i32, 0, 0, 1) === 0) {
    break;
  }
  Atomics.pause();
  spin++;
} while (spin < 10);

// Slow path: wait for the lock
// This can only be called in a worker thread,
// because the main thread cannot be blocked
Atomics.wait(i32, 0, 1);
```

See also

- `Atomics`
- `Atomics.wait()`
- `Atomics.waitAsync()`

Atomics.store()

The **`Atomics.store()`** static method stores a given value at a given position in the array and returns that value. This atomic operation guarantees that the write is tear-free, and that all atomic writes are sequentially consistent.

Note that unlike many other `Atomic` methods, `store()` returns the new value instead of the old value. To get the old value, use `Atomics.exchange()` instead.

Syntax

```
Atomics.store(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to store a `value` in.

`value`

The number to store.

Return value

The value that has been stored.

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.store()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
const ta = new Uint8Array(sab);

console.log(Atomics.store(ta, 0, 12)); // 12, the new/current value
console.log(Atomics.load(ta, 0)); // 12, the new/current value
```

See also

- `Atomics`
- `Atomics.load()`
- `Atomics.exchange()`

Atomics.sub()

The **`Atomics.sub()`** static method subtracts a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```
Atomics.sub(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to subtract a `value` from.

`value`

The number to subtract.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.sub()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
```

```
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 48;

// 48 - 12 = 36
console.log(Atomics.sub(ta, 0, 12)); // 48, the old value
console.log(Atomics.load(ta, 0)); // 36, the new/current value
```

See also

- `Atomics`
- `Atomics.add()`

Atomics.wait()

The **`Atomics.wait()`** static method verifies that a shared memory location contains a given value and if so sleeps, awaiting a wake-up notification or a timeout. It returns a string which is `"not-equal"` if the memory location does not match the given value, `"ok"` if woken by `Atomics.notify()`, or `"timed-out"` if the timeout expires.

`Atomics.wait()` and `Atomics.notify()` are used together to enable thread synchronization based on a value in shared memory. A thread can proceed immediately if the synchronization value has changed, or it can wait for notification from another thread when it reaches the synchronization point.

This method only works with an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`. It is blocking and cannot be used in the main thread. For a non-blocking, asynchronous version of this method, see `Atomics.waitAsync()`.

Syntax

```
Atomics.wait(typedArray, index, value)
Atomics.wait(typedArray, index, value, timeout)
```

Parameters

`typedArray`

An `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`index`

The position in the `typedArray` to wait on.

`value`

The expected value to test.

`timeout` Optional

Time to wait in milliseconds. `NaN` (and values that get converted to `NaN`, such as `undefined`) becomes `Infinity`. Negative values become `0`.

Return value

A string which is either `"not-equal"`, `"ok"`, or `"timed-out"`.

- `"not-equal"` is returned immediately if the initial `value` does not equal what is stored at `index`.
- `"ok"` is returned if woken up by a call to `Atomics.notify()`, **regardless of whether the expected value has changed**.
- `"timed-out"` is returned if a sleeping wait exceeds the specified `timeout` without being woken up by `Atomics.notify()`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- If `typedArray` is not an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.
- If the current thread cannot be blocked (for example, because it's the main thread).

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.wait()

Given a shared `Int32Array`:

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 because the provided `value` matches what is stored at the provided `index`. The reading thread will not move on until the writing thread has called `Atomics.notify()` on position 0 of the provided `typedArray`. Note that if, after being woken up, the value of location 0 has not been changed by the writing thread, the reading thread will **not** go back to sleep, but will continue on.

```
Atomics.wait(int32, 0, 0);
console.log(int32[0]); // 123
```

A writing thread stores a new value and notifies the waiting thread once it has written:

```
console.log(int32[0]); // 0;
Atomics.store(int32, 0, 123);
Atomics.notify(int32, 0, 1);
```

See also

- `Atomics`
- `Atomics.waitAsync()`
- `Atomics.notify()`

Atomics.waitAsync()

The **`Atomics.waitAsync()`** static method verifies that a shared memory location contains a given value, immediately returning an object with the `value` property containing the string `"not-equal"` if the memory location does not match the given value, or `"timed-out"` if the timeout was set to zero. Otherwise the method returns an object where the `value` property is a `Promise` that fulfills with either `"ok"` when `Atomics.notify()` is called, or `"timed-out"` if the timeout expires.

`Atomics.waitAsync()` and `Atomics.notify()` are used together to enable thread synchronization based on a value in shared memory. A thread can proceed immediately if the synchronization value has changed, or it can wait for notification from another thread when it reaches the synchronization point.

This method only works with an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`. It is non-blocking and, unlike `Atomics.wait()`, can be used on the main thread. Because it does not block the whole thread, you still need to be careful not to access the shared memory before the promise settles.

Syntax

```
Atomics.waitAsync(typedArray, index, value)
Atomics.waitAsync(typedArray, index, value, timeout)
```

Parameters

`typedArray`

: An `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`index`

: The position in the `typedArray` to wait on.

`value`

: The expected value to test.

`timeout` Optional

: Time to wait in milliseconds. `NaN` (and values that get converted to `NaN`, such as `undefined`) becomes `Infinity`. Negative values become `0`.

Return value

An `Object` with the following properties:

`async`

: A boolean indicating whether the `value` property is a `Promise` or not.

`value`

: If `async` is `false`, it will be a string which is either `"not-equal"` or `"timed-out"` (only when the `timeout` parameter is `0`). If `async` is `true`, it will be a `Promise` which is fulfilled with a string value, either `"ok"` or `"timed-out"`. The promise is never rejected.

Exceptions

`TypeError`

: Thrown if `typedArray` is not an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`RangeError`

: Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.waitAsync()

Given a shared `Int32Array`:

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 which is expected to be 0. The `result.value` will be a promise.

```
const result = Atomics.waitAsync(int32, 0, 0, 1000);
// { async: true, value: Promise {<pending>} }
```

In the reading thread or in another thread, the memory location 0 is called and the promise can be resolved with `"ok"`.

```
Atomics.notify(int32, 0);
// { async: true, value: Promise {<fulfilled>: 'ok'} }
```

If it isn't resolving to `"ok"`, the value in the shared memory location wasn't the expected (the `value` would be `"not-equal"` instead of a promise) or the timeout was reached (the promise will resolve to `"time-out"`).

See also

- `Atomics`
- `Atomics.wait()`
- `Atomics.notify()`

Atomics.xor()

The **`Atomics.xor()`** static method computes a bitwise XOR with a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```
Atomics.xor(typedArray, index, value)
```

Parameters

`typedArray`

: An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

: The position in the `typedArray` to compute the bitwise XOR.

`value`

: The number to compute the bitwise XOR with.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

: Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

: Thrown if `index` is out of bounds in the `typedArray`.

Description

The bitwise XOR operation yields 1, if `a` and `b` are different. The truth table for the XOR operation is:

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

For example, a bitwise XOR of `5 ^ 1` results in `0100` which is 4 in decimal.

```
5  0101
1  0001
   ----
4  0100
```

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.xor()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 (0111) XOR 2 (0010) = 5 (0101)
console.log(Atomics.xor(ta, 0, 2)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 5, the new/current value
```

See also

- `Atomics`
- `Atomics.and()`
- `Atomics.or()`

BigInt

`BigInt` values represent integer values which are too high or too low to be represented by the `number` primitive.

Description

A **BigInt value**, also sometimes just called a **BigInt**, is a `bigint` primitive, created by appending `n` to the end of an integer literal, or by calling the `BigInt()` function (without the `new` operator) and giving it an integer value or string value.

```
const previouslyMaxSafeInteger = 9007199254740991n;

const alsoHuge = BigInt(9007199254740991);
// 9007199254740991n

const hugeString = BigInt("9007199254740991");
// 9007199254740991n

const hugeHex = BigInt("0x1fffffffffffff");
// 9007199254740991n

const hugeOctal = BigInt("0o377777777777777777");
// 9007199254740991n

const hugeBin = BigInt(
  "0b11111111111111111111111111111111111111111111111111111",
);
// 9007199254740991n
```

BigInt values are similar to Number values in some ways, but also differ in a few key matters: A BigInt value cannot be used with methods in the built-in `Math` object and cannot be mixed with a Number value in operations; they must be coerced to the same type. Be careful coercing values back and forth, however, as the precision of a BigInt value may be lost when it is coerced to a Number value.

Type information

When tested against `typeof`, a BigInt value (`bigint` primitive) will give `"bigint"`:

```
typeof 1n === "bigint"; // true
typeof BigInt("1") === "bigint"; // true
```

A BigInt value can also be wrapped in an `Object`:

```
typeof Object(1n) === "object"; // true
```

Operators

Most operators support BigInts, however most do not permit operands to be of mixed types — both operands must be BigInt or neither:

- Arithmetic operators: `+`, `-`, `*`, `/`, `%`, `**`
- Bitwise operators: `>>`, `<<`, `&`, `|`, `^`, `~`
- Unary negation (`-`)
- Increment/decrement: `++`, `--`

The boolean-returning operators allow mixing numbers and BigInts as operands:

- Relational operators and equality operators: `>`, `<`, `>=`, `<=`, `==`, `!=`, `===`, `!==`
- Logical operators only rely on the truthiness of operands

A couple of operators do not support BigInt at all:

- Unary plus (`+`) cannot be supported due to conflicting usage in asm.js, so it has been left out in order to not break asm.js.
- Unsigned right shift (`>>>`) is the only bitwise operator that's unsupported, as every BigInt value is signed.

Special cases:

- Addition (+) involving a string and a BigInt returns a string.
- Division (/) truncates fractional components towards zero, since BigInt is unable to represent fractional quantities.

```
const previousMaxSafe = BigInt(Number.MAX_SAFE_INTEGER); // 9007199254740991n
const maxPlusOne = previousMaxSafe + 1n; // 9007199254740992n
const theFuture = previousMaxSafe + 2n; // 9007199254740993n, this works now!
const prod = previousMaxSafe * 2n; // 18014398509481982n
const diff = prod - 10n; // 18014398509481972n
const mod = prod % 10n; // 2n
const bigN = 2n ** 54n; // 18014398509481984n
bigN * -1n; // -18014398509481984n
const expected = 4n / 2n; // 2n
const truncated = 5n / 2n; // 2n, not 2.5n
```

Comparisons

A BigInt value is not strictly equal to a Number value, but it *is* loosely so:

```
0n === 0; // false
0n == 0; // true
```

A Number value and a BigInt value may be compared as usual:

```
1n < 2; // true
2n > 1; // true
2 > 2; // false
2n > 2; // false
2n >= 2; // true
```

BigInt values and Number values may be mixed in arrays and sorted:

```
const mixed = [4n, 6, -12n, 10, 4, 0, 0n];
// [4n, 6, -12n, 10, 4, 0, 0n]

mixed.sort(); // default sorting behavior
// [ -12n, 0, 0n, 10, 4n, 4, 6 ]

mixed.sort((a, b) => a - b);
// won't work since subtraction will not work with mixed types
// TypeError: can't convert BigInt value to Number value

// sort with an appropriate numeric comparator
mixed.sort((a, b) => (a < b ? -1 : a > b ? 1 : 0));
// [ -12n, 0, 0n, 4n, 4, 6, 10 ]
```

Note that comparisons with `Object`-wrapped BigInt values act as with other objects, only indicating equality when the same object instance is compared:

```
Object(0n) === 0n; // false
Object(0n) === Object(0n); // false

const o = Object(0n);
o === o; // true
```

Because coercing between Number values and BigInt values can lead to loss of precision, the following are recommended:

- Only use a BigInt value when values greater than 2^{53} are reasonably expected.
- Don't coerce between BigInt values and Number values.

Conditionals

A BigInt value follows the same conversion rules as Numbers when:

- it is converted to a `Boolean`: via the `Boolean` function;
- when used with logical operators `||`, `&&`, and `!`; or
- within a conditional test like an `if` statement.

Namely, only `0n` is falsy; everything else is truthy.

```
if (0n) {
  console.log("Hello from the if!");
} else {
  console.log("Hello from the else!");
}
// "Hello from the else!"

0n || 12n; // 12n
0n && 12n; // 0n
Boolean(0n); // false
Boolean(12n); // true
```

```
!12n; // false
!0n; // true
```

Cryptography

The operations supported on BigInt values are not constant-time and are thus open to timing attacks. JavaScript BigInts therefore could be dangerous for use in cryptography without mitigating factors. As a very generic example, an attacker could measure the time difference between `101n ** 65537n` and `17n ** 9999n`, and deduce the magnitude of secrets, such as private keys, based on the time elapsed. If you still have to use BigInts, take a look at the Timing attack FAQ for general advice regarding the issue.

Use within JSON

Using `JSON.stringify()` with any BigInt value will raise a `TypeError`, as BigInt values aren't serialized in JSON by default. However, `JSON.stringify()` specifically leaves a backdoor for BigInt values: it would try to call the BigInt's `toJSON()` method. (It doesn't do so for any other primitive values.) Therefore, you can implement your own `toJSON()` method (which is one of the few cases where patching built-in objects is not explicitly discouraged):

```
BigInt.prototype.toJSON = function () {
  return { $bigint: this.toString() };
};
```

Instead of throwing, `JSON.stringify()` now produces a string like this:

```
console.log(JSON.stringify({ a: 1n }));
// {"a":{"$bigint":"1"}}
```

If you do not wish to patch `BigInt.prototype`, you can use the `replacer` parameter of `JSON.stringify` to serialize BigInt values:

```
const replacer = (key, value) =>
  typeof value === "bigint" ? { $bigint: value.toString() } : value;

const data = {
  number: 1,
  big: 18014398509481982n,
};
const stringified = JSON.stringify(data, replacer);

console.log(stringified);
// {"number":1,"big":{"$bigint":"18014398509481982"}}
```

You can then use the `reviver` parameter of `JSON.parse` to handle them:

```
const reviver = (key, value) =>
  value !== null &&
  typeof value === "object" &&
  "$bigint" in value &&
  typeof value.$bigint === "string"
    ? BigInt(value.$bigint)
    : value;

const payload = '{"number":1,"big":{"$bigint":"18014398509481982"}}';
const parsed = JSON.parse(payload, reviver);

console.log(parsed);
// { number: 1, big: 18014398509481982n }
```

> **Note:** While it's possible to make the replacer of `JSON.stringify()` generic and properly serialize BigInt values for all objects as shown above, the reviver of `JSON.parse()` has to be used with caution, because the serialization is *irreversible*: it's not possible to distinguish between an object that happens to have a property called `$bigint` and an actual BigInt.
>
> In addition, the example above creates an entire object during replacing and reviving, which may have performance or storage implications for larger objects containing many BigInts. If you know the shape of the payload, it may be better to just serialize them as strings and revive them based on the property key's name instead.

In fact, JSON allows number literals that are arbitrarily long; they just cannot be parsed to full precision in JavaScript. If you are communicating with another program in a language that supports longer integers (such as 64-bit integers), and you want to transmit the BigInt as a JSON number instead of a JSON string, see Lossless number serialization.

BigInt coercion

Many built-in operations that expect BigInts first coerce their arguments to BigInts. The operation can be summarized as follows:

- BigInts are returned as-is.
- `undefined` and `null` throw a `TypeError`.
- `true` turns into `1n`; `false` turns into `0n`.
- Strings are converted by parsing them as if they contain an integer literal. Any parsing failure results in a `SyntaxError`. The syntax is a subset of string numeric literals, where decimal points or exponent indicators are not allowed.
- Numbers throw a `TypeError` to prevent unintended implicit coercion causing loss of precision.
- Symbols throw a `TypeError`.

- Objects are first converted to a primitive by calling their `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. The resulting primitive is then converted to a BigInt.

The best way to achieve nearly the same effect in JavaScript is through the `BigInt()` function: `BigInt(x)` uses the same algorithm to convert `x`, except that Numbers don't throw a `TypeError`, but are converted to BigInts if they are integers.

Note that built-in operations expecting BigInts often truncate the BigInt to a fixed width after coercion. This includes `BigInt.asIntN()`, `BigInt.asUintN()`, and methods of `BigInt64Array` and `BigUint64Array`.

Constructor

`BigInt()`

Returns primitive values of type BigInt. Throws an error when called with `new`.

Static methods

`BigInt.asIntN()`

Truncates a `BigInt` value to the given number of least significant bits and returns that value as a signed integer.

`BigInt.asUintN()`

Truncates a `BigInt` value to the given number of least significant bits and returns that value as an unsigned integer.

Instance properties

These properties are defined on `BigInt.prototype` and shared by all `BigInt` instances.

`BigInt.prototype.constructor`

The constructor function that created the instance object. For `BigInt` instances, the initial value is the `BigInt` constructor.

`BigInt.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"BigInt"`. This property is used in `Object.prototype.toString()`. However, because `BigInt` also has its own `toString()` method, this property is not used unless you call `Object.prototype.toString.call()` with a BigInt as `thisArg`.

Instance methods

`BigInt.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this BigInt value. Overrides the `Object.prototype.toLocaleString()` method.

`BigInt.prototype.toString()`

Returns a string representing this BigInt value in the specified radix (base). Overrides the `Object.prototype.toString()` method.

`BigInt.prototype.valueOf()`

Returns this BigInt value. Overrides the `Object.prototype.valueOf()` method.

Examples

Calculating Primes

```
function isPrime(n) {
  if (n < 2n) {
    return false;
  }
  if (n % 2n === 0n) {
    return n === 2n;
  }
  for (let factor = 3n; factor * factor <= n; factor += 2n) {
    if (n % factor === 0n) {
      return false;
    }
  }
  return true;
}

// Takes a BigInt value as an argument, returns nth prime number as a BigInt value
function nthPrime(nth) {
  let maybePrime = 2n;
  let prime = 0n;

  while (nth >= 0n) {
    if (isPrime(maybePrime)) {
      nth--;
      prime = maybePrime;
    }
    maybePrime++;
  }

  return prime;
}
```

```
nthPrime(20n);
// 73n
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

See also

- `Number`
- `Number.MAX_SAFE_INTEGER`

BigInt.asIntN()

The **`BigInt.asIntN()`** static method truncates a `BigInt` value to the given number of least significant bits and returns that value as a signed integer.

Try it

```
const I64_CEIL = 2n ** 63n;

console.log(BigInt.asIntN(64, I64_CEIL - 1n));
// 9223372036854775807n (2n ** 64n - 1n, the maximum non-wrapping value)
console.log(BigInt.asIntN(64, I64_CEIL));
// -9223372036854775808n (wraps to min value)
console.log(BigInt.asIntN(64, I64_CEIL + 1n));
// -9223372036854775807n (min value + 1n)
console.log(BigInt.asIntN(64, I64_CEIL * 2n));
// 0n (wrapped around to zero)
console.log(BigInt.asIntN(64, -I64_CEIL * -42n));
// 0n (also wraps on negative multiples)
```

Syntax

```
BigInt.asIntN(bits, bigint)
```

Parameters

`bits`

The number of bits available for the returned BigInt. Should be an integer between 0 and 2^{53} - 1, inclusive.

`bigint`

The BigInt value to truncate to fit into the supplied bits.

Return value

A BigInt containing the bit representation of `bigint` truncated to `bits` bits, interpreted as a signed integer.

Exceptions

`RangeError`

Thrown if `bits` is negative or greater than 2^{53} - 1.

Description

The `BigInt.asIntN` method truncates a `BigInt` value to the given number of bits, and interprets the result as a signed integer. For example, for `BigInt.asIntN(3, 25n)`, the value `25n` is truncated to `1n`:

```
25n = 00011001 (base 2)
          ^=== Use only the three remaining bits
===>       001 (base 2) = 1n
```

If the leading bit of the remaining number is `1`, the result is negative. For example, `BigInt.asIntN(4, 25n)` yields `-7n`, because `1001` is the encoding of `-7` under two's complement:

```
25n = 00011001 (base 2)
         ^==== Use only the four remaining bits
===>      1001 (base 2) = -7n
```

> **Note:** `BigInt` values are always encoded as two's complement in binary.

Unlike similar language APIs such as `Number.prototype.toExponential()`, `asIntN` is a static property of `BigInt`, so you always use it as `BigInt.asIntN()`, rather than as a method of a BigInt value. Exposing `asIntN()` as a "standard library function" allows interop with asm.js.

Examples

Staying in 64-bit ranges

The `BigInt.asIntN()` method can be useful to stay in the range of 64-bit arithmetic.

```
const max = 2n ** (64n - 1n) - 1n;

BigInt.asIntN(64, max); // 9223372036854775807n

BigInt.asIntN(64, max + 1n); // -9223372036854775808n
// negative because the 64th bit of 2^63 is 1
```

See also

- `BigInt`
- `BigInt.asUintN()`

BigInt.asUintN()

The **`BigInt.asUintN()`** static method truncates a `BigInt` value to the given number of least significant bits and returns that value as an unsigned integer.

Try it

```
const U64_CEIL = 2n ** 64n;

console.log(BigInt.asUintN(64, U64_CEIL - 1n));
// 18446744073709551615n (2n ** 64n - 1n, the maximum non-wrapping value)
console.log(BigInt.asUintN(64, U64_CEIL));
// 0n (wraps to zero)
console.log(BigInt.asUintN(64, U64_CEIL + 1n));
// 1n
console.log(BigInt.asUintN(64, U64_CEIL * 2n));
// 0n (wraps on multiples)
console.log(BigInt.asUintN(64, U64_CEIL * -42n));
// 0n (also wraps on negative multiples)
```

Syntax

```
BigInt.asUintN(bits, bigint)
```

Parameters

`bits`

The number of bits available for the returned BigInt. Should be an integer between 0 and 2^{53} - 1, inclusive.

`bigint`

The BigInt value to truncate to fit into the supplied bits.

Return value

A BigInt containing the bit representation of `bigint` truncated to `bits` bits, interpreted as an unsigned integer.

Exceptions

`RangeError`

Thrown if `bits` is negative or greater than 2^{53} - 1.

Description

The `BigInt.asUintN` method truncates a `BigInt` value to the given number of bits, and interprets the result as an unsigned integer. Unsigned integers have no sign bits and are always non-negative. For example, for `BigInt.asUintN(4, 25n)`, the value `25n` is truncated to `9n`:

```
25n = 00011001 (base 2)
         ^==== Use only the four remaining bits
===>      1001 (base 2) = 9n
```

> **Note:** `BigInt` values are always encoded as two's complement in binary.

Unlike similar language APIs such as `Number.prototype.toExponential()`, `asUintN` is a static property of `BigInt`, so you always use it as `BigInt.asUintN()`, rather than as a method of a BigInt value. Exposing `asUintN()` as a "standard library function" allows interop with asm.js.

Examples

Staying in 64-bit ranges

The `BigInt.asUintN()` method can be useful to stay in the range of 64-bit arithmetic.

```
const max = 2n ** 64n - 1n;
```

```
BigInt.asUintN(64, max); // 18446744073709551615n

BigInt.asUintN(64, max + 1n); // 0n
// zero because of overflow: the lowest 64 bits are all zeros
```

See also

- `BigInt`
- `BigInt.asIntN()`

BigInt() constructor

The **`BigInt()`** function returns primitive values of type BigInt.

Syntax

```
BigInt(value)
```

> **Note:** `BigInt()` can only be called without `new`. Attempting to construct it with `new` throws a `TypeError`.

Parameters

`value`

The value to be converted to a BigInt value. It may be a string, an integer, a boolean, or another `BigInt`.

Return value

A `BigInt` value. Number values must be integers and are converted to BigInts. The boolean value `true` becomes `1n`, and `false` becomes `0n`. Strings are parsed as if they are source text for integer literals, which means they can have leading and trailing whitespaces and can be prefixed with `0b`, `0o`, or `0x`.

Exceptions

`RangeError`

Thrown if the parameter is a non-integral number.

`TypeError`

Thrown in one of the following cases:

- The parameter cannot be converted to a primitive.
- After conversion to a primitive, the result is `undefined`, `null`, or `Symbol`.

`SyntaxError`

Thrown if the parameter is a string that cannot be parsed as a `BigInt`.

Examples

Using BigInt() to convert a number to a BigInt

`BigInt()` is the only case where a number can be converted to a BigInt without throwing, because it's very explicit. However, only integers are allowed.

```
BigInt(123); // 123n
BigInt(123.3); // RangeError: The number 123.3 cannot be converted to a BigInt because it is not an integer
```

Using string values

```
BigInt("123"); // 123n
BigInt("0b10101"); // 21n, which is 10101 in binary
BigInt("0o123"); // 83n, which is 123 in octal
BigInt("0x123"); // 291n, which is 123 in hexadecimal
BigInt("  123  "); // 123n, leading and trailing whitespaces are allowed
```

See also

- `BigInt`

BigInt.prototype.toLocaleString()

The **`toLocaleString()`** method of `BigInt` values returns a string with a language-sensitive representation of this BigInt. In implementations with `Intl.NumberFormat` API support, this method delegates to `Intl.NumberFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.NumberFormat` object and use its `format()` method, because a `NumberFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

```
const bigint = 123456789123456789n;
```

```
// German uses period for thousands
console.log(bigint.toLocaleString("de-DE"));
// Expected output: "123.456.789.123.456.789"

// Request a currency format
console.log(
  bigint.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// Expected output: "123.456.789.123.456.789,00 €"
```

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.NumberFormat` API, these parameters correspond exactly to the `Intl.NumberFormat()` constructor's parameters. Implementations without `Intl.NumberFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

> A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.NumberFormat()` constructor.
>
> In implementations without `Intl.NumberFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

> An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.NumberFormat()` constructor.
>
> In implementations without `Intl.NumberFormat` support, this parameter is ignored.

See the `Intl.NumberFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given BigInt according to language-specific conventions.

In implementations with `Intl.NumberFormat`, this is equivalent to `new Intl.NumberFormat(locales, options).format(number)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const bigint = 3500n;

console.log(bigint.toLocaleString());
// "3,500" if in U.S. English locale
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```
function toLocaleStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.NumberFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const bigint = 123456789123456789n;

// German uses period for thousands
```

```
console.log(bigint.toLocaleString("de-DE"));
// 123.456.789.123.456.789

// Arabic in most Arabic speaking countries uses Eastern Arabic digits
console.log(bigint.toLocaleString("ar-EG"));
// ١٢٣,٤٥٦,٧٨٩,١٢٣,٤٥٦,٧٨٩

// India uses thousands/lakh/crore separators
console.log(bigint.toLocaleString("en-IN"));
// 1,23,45,67,89,12,34,56,789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(bigint.toLocaleString("zh-Hans-CN-u-nu-hanidec"));
// 一二三,四五六,七八九,一二三,四五六,七八九

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(bigint.toLocaleString(["ban", "id"]));
// 123.456.789.123.456.789
```

Using options

The results provided by `toLocaleString()` can be customized using the `options` parameter:

```
const bigint = 123456789123456789n;

// request a currency format
console.log(
  bigint.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// 123.456.789.123.456.789,00 €

// the Japanese yen doesn't use a minor unit
console.log(
  bigint.toLocaleString("ja-JP", { style: "currency", currency: "JPY" }),
);
// ￥123,456,789,123,456,789

// limit to three significant digits
console.log(bigint.toLocaleString("en-IN", { maximumSignificantDigits: 3 }));
// 1,23,00,00,00,00,00,00,000
```

See also

- `Intl.NumberFormat`
- `BigInt.prototype.toString()`

BigInt.prototype.toString()

The **`toString()`** method of `BigInt` values returns a string representing the specified `BigInt` value. The trailing "n" is not part of the string.

Try it

```
console.log(1024n.toString());
// Expected output: "1024"

console.log(1024n.toString(2));
// Expected output: "10000000000"

console.log(1024n.toString(16));
// Expected output: "400"
```

Syntax

```
toString()
toString(radix)
```

Parameters

`radix` Optional

An integer in the range 2 through 36 specifying the base to use for representing the BigInt value. Defaults to 10.

Return value

A string representing the specified `BigInt` value.

Exceptions

`RangeError`

Thrown if `radix` is less than 2 or greater than 36.

Description

The `BigInt` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `BigInt` values, the `toString()` method returns a string representation of the value in the specified radix.

For radixes above 10, the letters of the alphabet indicate digits greater than 9. For example, for hexadecimal numbers (base 16) `a` through `f` are used.

If the specified BigInt value is negative, the sign is preserved. This is the case even if the radix is 2; the string returned is the positive binary representation of the BigInt value preceded by a `-` sign, **not** the two's complement of the BigInt value.

The `toString()` method requires its `this` value to be a `BigInt` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to BigInt values.

Because `BigInt` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `BigInt` *object* is used in a context expecting a string, such as in a template literal. However, BigInt *primitive* values do not consult the `toString()` method to be coerced to strings — rather, they are directly converted using the same algorithm as the initial `toString()` implementation.

```
BigInt.prototype.toString = () => "Overridden";
console.log(`${1n}`); // "1"
console.log(`${Object(1n)}`); // "Overridden"
```

Examples

Using toString()

```
17n.toString(); // "17"
66n.toString(2); // "1000010"
254n.toString(16); // "fe"
(-10n).toString(2); // "-1010"
(-0xffn).toString(2); // "-11111111"
```

Negative-zero BigInt

There is no negative-zero `BigInt` as there are no negative zeros in integers. `-0.0` is an IEEE floating-point concept that only appears in the JavaScript `Number` type.

```
(-0n).toString(); // "0"
BigInt(-0).toString(); // "0"
```

See also

- `BigInt.prototype.toLocaleString()`
- `BigInt.prototype.valueOf()`
- `Number.prototype.toString()`

BigInt.prototype.valueOf()

The **`valueOf()`** method of `BigInt` values returns the wrapped primitive value of a `BigInt` object.

Try it

```
console.log(typeof Object(1n));
// Expected output: "object"

console.log(typeof Object(1n).valueOf());
// Expected output: "bigint"
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

A BigInt representing the primitive value of the specified `BigInt` object.

Examples

Using <code>valueOf</code>

```
typeof Object(1n); // object
typeof Object(1n).valueOf(); // bigint
```

See also

- `BigInt.prototype.toString()`

BigInt64Array

The **`BigInt64Array`** typed array represents an array of 64-bit signed integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0n` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or

using standard array index syntax (that is, using bracket notation).

`BigInt64Array` is a subclass of the hidden `TypedArray` class.

Try it

```
const buffer = new ArrayBuffer(24);
const bigint64 = new BigInt64Array(buffer);
bigint64[0] = 5886014448488689n;
bigint64[1] = 1881938909131133n;
bigint64[2] = 1898875537769492n;

bigint64[0] = 6118793953620967n;
console.log(bigint64);
// Expected Output: BigInt64Array [6118793953620967n, 1881938909131133n, 1898875537769492n]

console.log(bigint64[2]);
// Expected Output: 1898875537769492n

console.log("Array length:", bigint64.length);
// Expected Output: Array length: 3

console.log("Array byte length:", bigint64.byteLength);
// Expected Output: Array byte length: 24

console.log("Array byte offset:", bigint64.byteOffset);
// Expected Output: Array byte offset: 0

bigint64.set([100n, 200n], 1);
console.log(bigint64);
// Expected Output: BigInt64Array [6118793953620967n, 100n, 200n]
```

Constructor

`BigInt64Array()`

Creates a new `BigInt64Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`BigInt64Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of `BigInt64Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `BigInt64Array.prototype` and shared by all `BigInt64Array` instances.

`BigInt64Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of a `BigInt64Array`.

`BigInt64Array.prototype.constructor`

The constructor function that created the instance object. For `BigInt64Array` instances, the initial value is the `BigInt64Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a BigInt64Array

```
// From a length
const bigint64 = new BigInt64Array(2);
bigint64[0] = 42n;
console.log(bigint64[0]); // 42n
console.log(bigint64.length); // 2
console.log(bigint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigInt64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigInt64Array(x);
```

```
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigInt64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const bigint64FromIterable = new BigInt64Array(iterable);
console.log(bigint64FromIterable);
// BigInt64Array [1n, 2n, 3n]
```

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

BigInt64Array() constructor

The **`BigInt64Array()`** constructor creates `BigInt64Array` objects. The contents are initialized to `0n` unless initialization data is explicitly provided.

Syntax

```
new BigInt64Array()
new BigInt64Array(length)
new BigInt64Array(typedArray)
new BigInt64Array(object)

new BigInt64Array(buffer)
new BigInt64Array(buffer, byteOffset)
new BigInt64Array(buffer, byteOffset, length)
```

> **Note:** `BigInt64Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a BigInt64Array

```
// From a length
const bigint64 = new BigInt64Array(2);
bigint64[0] = 42n;
console.log(bigint64[0]); // 42n
console.log(bigint64.length); // 2
console.log(bigint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigInt64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigInt64Array(x);
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigInt64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const bigint64FromIterable = new BigInt64Array(iterable);
console.log(bigint64FromIterable);
// BigInt64Array [1n, 2n, 3n]
```

See also

- JavaScript typed arrays guide

- `TypedArray`
- `ArrayBuffer`
- `DataView`

BigUint64Array

The **`BigUint64Array`** typed array represents an array of 64-bit unsigned integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0n` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`BigUint64Array` is a subclass of the hidden `TypedArray` class.

Constructor

`BigUint64Array()`

Creates a new `BigUint64Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`BigUint64Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of `BigUint64Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `BigUint64Array.prototype` and shared by all `BigUint64Array` instances.

`BigUint64Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of a `BigUint64Array`.

`BigUint64Array.prototype.constructor`

The constructor function that created the instance object. For `BigUint64Array` instances, the initial value is the `BigUint64Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a BigUint64Array

```
// From a length
const biguint64 = new BigUint64Array(2);
biguint64[0] = 42n;
console.log(biguint64[0]); // 42n
console.log(biguint64.length); // 2
console.log(biguint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigUint64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigUint64Array(x);
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigUint64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const biguint64FromIterable = new BigUint64Array(iterable);
console.log(biguint64FromIterable);
// BigUint64Array [1n, 2n, 3n]
```

See also

- JavaScript typed arrays guide
- `TypedArray`

- `ArrayBuffer`
- `DataView`

BigUint64Array() constructor

The **`BigUint64Array()`** constructor creates `BigUint64Array` objects. The contents are initialized to `0n` unless initialization data is explicitly provided.

Syntax

```
new BigUint64Array()
new BigUint64Array(length)
new BigUint64Array(typedArray)
new BigUint64Array(object)

new BigUint64Array(buffer)
new BigUint64Array(buffer, byteOffset)
new BigUint64Array(buffer, byteOffset, length)
```

> **Note:** `BigUint64Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a BigUint64Array

```
// From a length
const biguint64 = new BigUint64Array(2);
biguint64[0] = 42n;
console.log(biguint64[0]); // 42n
console.log(biguint64.length); // 2
console.log(biguint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigUint64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigUint64Array(x);
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigUint64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const biguint64FromIterable = new BigUint64Array(iterable);
console.log(biguint64FromIterable);
// BigUint64Array [1n, 2n, 3n]
```

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Boolean

`Boolean` values can be one of two values: `true` or `false`, representing the truth value of a logical proposition.

Description

Boolean values are typically produced by relational operators, equality operators, and logical NOT (`!`). They can also be produced by functions that represent conditions, such as `Array.isArray()`. Note that binary logical operators such as `&&` and `||` return the values of the operands, which may or may not be boolean values.

Boolean values are typically used in conditional testing, such as the condition for `if...else` and `while` statements, the conditional operator (`? :`), or the predicate return value of `Array.prototype.filter()`.

You would rarely need to explicitly convert something to a boolean value, as JavaScript does this automatically in boolean contexts, so you can use any value as if it's a boolean, based on its truthiness. You are also encouraged to use `if (condition)` and `if (!condition)` instead of `if (condition === true)` or `if (condition === false)`

in your own code so you can take advantage of this convention. However, making sure that values representing conditions are always booleans can help clarify the intent of your code.

```
// Do this:
// This always returns a boolean value
const isObject = (obj) => !!obj && typeof obj === "object";

// Or this:
const isObject = (obj) => Boolean(obj) && typeof obj === "object";

// Or this:
const isObject = (obj) => obj !== null && typeof obj === "object";

// Instead of this:
// This may return falsy values that are not equal to false
const isObject = (obj) => obj && typeof obj === "object";
```

Boolean primitives and Boolean objects

For converting non-boolean values to boolean, use `Boolean` as a function or use the double NOT operator. Do not use the `Boolean()` constructor with `new`.

```
const good = Boolean(expression);
const good2 = !!expression;
```

```
const bad = new Boolean(expression); // don't use this!
```

This is because *all* objects, including a `Boolean` object whose wrapped value is `false`, are truthy and evaluate to `true` in places such as conditional statements. (See also the boolean coercion section below.)

```
if (new Boolean(true)) {
  console.log("This log is printed.");
}

if (new Boolean(false)) {
  console.log("This log is ALSO printed.");
}

const myFalse = new Boolean(false); // myFalse is a Boolean object (not the primitive value false)
const g = Boolean(myFalse); // g is true
const myString = new String("Hello"); // myString is a String object
const s = Boolean(myString); // s is true
```

> **Warning:** You should rarely find yourself using `Boolean` as a constructor.

Boolean coercion

Many built-in operations that expect booleans first coerce their arguments to booleans. The operation can be summarized as follows:

- Booleans are returned as-is.
- `undefined` turns into `false`.
- `null` turns into `false`.
- `0`, `-0`, and `NaN` turn into `false`; other numbers turn into `true`.
- `0n` turns into `false`; other BigInts turn into `true`.
- The empty string `""` turns into `false`; other strings turn into `true`.
- Symbols turn into `true`.
- All objects become `true`.

> **Note:** A legacy behavior makes `document.all` return `false` when used as a boolean, despite it being an object. This property is legacy and non-standard and should not be used.

> **Note:** Unlike other type conversions like string coercion or number coercion, boolean coercion does not attempt to convert objects to primitives by calling user methods.

In other words, there are only a handful of values that get coerced to `false` — these are called falsy values. All other values are called truthy values. A value's truthiness is important when used with logical operators, conditional statements, or any boolean context.

There are two ways to achieve the same effect in JavaScript.

- Double NOT: `!!x` negates `x` twice, which converts `x` to a boolean using the same algorithm as above.
- The `Boolean()` function: `Boolean(x)` uses the same algorithm as above to convert `x`.

Note that truthiness is not the same as being loosely equal to `true` or `false`.

```
if ([]) {
  console.log("[] is truthy");
}
```

```
if ([] == false) {
  console.log("[] == false");
}
// [] is truthy
// [] == false
```

`[]` is truthy, but it's also loosely equal to `false`. It's truthy, because all objects are truthy. However, when comparing with `false`, which is a primitive, `[]` is also converted to a primitive, which is `""` via `Array.prototype.toString()`. Comparing strings and booleans results in both being converted to numbers, and they both become `0`, so `[] == false` is `true`. In general, falsiness and `== false` differ in the following cases:

- `NaN`, `undefined`, and `null` are falsy but not loosely equal to `false`.
- `"0"` (and other string literals that are not `""` but get coerced to 0) is truthy but loosely equal to `false`.
- Objects are always truthy, but their primitive representation may be loosely equal to `false`.

Truthy values are even more unlikely to be loosely equal to `true`. All values are either truthy or falsy, but most values are loosely equal to neither `true` nor `false`.

Constructor

`Boolean()`

Creates `Boolean` objects. When called as a function, it returns primitive values of type Boolean.

Instance properties

These properties are defined on `Boolean.prototype` and shared by all `Boolean` instances.

`Boolean.prototype.constructor`

The constructor function that created the instance object. For `Boolean` instances, the initial value is the `Boolean` constructor.

Instance methods

`Boolean.prototype.toString()`

Returns a string of either `true` or `false` depending upon the value of the object. Overrides the `Object.prototype.toString()` method.

`Boolean.prototype.valueOf()`

Returns the primitive value of the `Boolean` object. Overrides the `Object.prototype.valueOf()` method.

Examples

Creating false values

```
const bNoParam = Boolean();
const bZero = Boolean(0);
const bNull = Boolean(null);
const bEmptyString = Boolean("");
const bfalse = Boolean(false);
```

Creating true values

```
const btrue = Boolean(true);
const btrueString = Boolean("true");
const bfalseString = Boolean("false");
const bSuLin = Boolean("Su Lin");
const bArrayProto = Boolean([]);
const bObjProto = Boolean({});
```

See also

- Boolean
- Boolean primitives
- Boolean data type on Wikipedia

Boolean() constructor

The **`Boolean()`** constructor creates `Boolean` objects. When called as a function, it returns primitive values of type Boolean.

Try it

```
const flag = new Boolean();
console.log(typeof flag);
// Expected output: object
console.log(flag === false);
// Expected output: false

const flag2 = Boolean();
console.log(typeof flag2);
// Expected output: boolean
console.log(flag2 === false);
// Expected output: true
```

Syntax

```
new Boolean(value)
Boolean(value)
```

> **Note:** `Boolean()` can be called with or without `new`, but with different effects. See Return value.

Parameters

`value`

The initial value of the `Boolean` object.

Return value

When `Boolean()` is called as a function (without `new`), it returns `value` coerced to a boolean primitive.

When `Boolean()` is called as a constructor (with `new`), it coerces `value` to a boolean primitive and returns a wrapping `Boolean` object, which is **not** a primitive.

> **Warning:** You should rarely find yourself using `Boolean` as a constructor.

Description

The value passed as the first parameter is converted to a boolean value. If the value is omitted or is `0`, `-0`, `0n`, `null`, `false`, `NaN`, `undefined`, or the empty string (`""`), then the object has an initial value of `false`. All other values, including any object, an empty array (`[]`), or the string `"false"`, create an object with an initial value of `true`.

> **Note:** When the non-standard property `document.all` is used as an argument for this constructor, the result is a `Boolean` object with the value `false`. This property is legacy and non-standard and should not be used.

Examples

Creating Boolean objects with an initial value of false

```
const bZero = new Boolean(0);
const bNull = new Boolean(null);
const bEmptyString = new Boolean("");
const bfalse = new Boolean(false);

typeof bfalse; // "object"
Boolean(bfalse); // true
```

Note how converting a `Boolean` object to a primitive with `Boolean()` always yields `true`, even if the object holds a value of `false`. You are therefore always advised to avoid constructing `Boolean` wrapper objects.

If you need to take the primitive value out from the wrapper object, instead of using the `Boolean()` function, use the object's `valueOf()` method instead.

```
const bfalse = new Boolean(false);

bfalse.valueOf(); // false
```

Creating <code>Boolean</code> objects with an initial value of <code>true</code>

```
const btrue = new Boolean(true);
const btrueString = new Boolean("true");
const bfalseString = new Boolean("false");
const bSuLin = new Boolean("Su Lin");
const bArrayProto = new Boolean([]);
const bObjProto = new Boolean({});
```

See also

- Boolean

Boolean.prototype.toString()

The **`toString()`** method of `Boolean` values returns a string representing the specified boolean value.

Try it

```
const flag1 = new Boolean(true);

console.log(flag1.toString());
// Expected output: "true"

const flag2 = new Boolean(1);
```

```
console.log(flag2.toString());
// Expected output: "true"
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the specified boolean value.

Description

The `Boolean` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `Boolean` values, the `toString` method returns a string representation of the boolean value, which is either `"true"` or `"false"`.

The `toString()` method requires its `this` value to be a `Boolean` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to boolean values.

Because `Boolean` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `Boolean` *object* is used in a context expecting a string, such as in a template literal. However, boolean *primitive* values do not consult the `toString()` method to be coerced to strings — rather, they are directly converted using the same algorithm as the initial `toString()` implementation.

```
Boolean.prototype.toString = () => "Overridden";
console.log(`${true}`); // "true"
console.log(`${new Boolean(true)}`); // "Overridden"
```

Examples

Using toString()

```
const flag = new Boolean(true);
console.log(flag.toString()); // "true"
console.log(false.toString()); // "false"
```

See also

- `Object.prototype.toString()`

Boolean.prototype.valueOf()

The **`valueOf()`** method of `Boolean` values returns the primitive value of a `Boolean` object.

Try it

```
const x = new Boolean();

console.log(x.valueOf());
// Expected output: false

const y = new Boolean("Mozilla");

console.log(y.valueOf());
// Expected output: true
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

The primitive value of the given `Boolean` object.

Description

The `valueOf()` method of `Boolean` returns the primitive value of a `Boolean` object or literal `Boolean` as a Boolean data type.

This method is usually called internally by JavaScript and not explicitly in code.

Examples

Using <code>valueOf()</code>

```
const x = new Boolean();
const myVar = x.valueOf(); // assigns false to myVar
```

See also

- `Object.prototype.valueOf()`

DataView

The **`DataView`** view provides a low-level interface for reading and writing multiple number types in a binary `ArrayBuffer`, without having to care about the platform's endianness.

Description

Endianness

Multi-byte number formats are represented in memory differently depending on machine architecture — see Endianness for an explanation. `DataView` accessors provide explicit control of how data is accessed, regardless of the executing computer's endianness. For example, WebAssembly memory is always little-endian, so you should use `DataView` instead of typed arrays to read and write multi-byte values. See `WebAssembly.Memory` for an example.

```
const littleEndian = (() => {
  const buffer = new ArrayBuffer(2);
  new DataView(buffer).setInt16(0, 256, true /* littleEndian */);
  // Int16Array uses the platform's endianness.
  return new Int16Array(buffer)[0] === 256;
})();
console.log(littleEndian); // true or false
```

> **Note:** `DataView` defaults to big-endian read and write, but most platforms use little-endian.

Constructor

`DataView()`

Creates a new `DataView` object.

Instance properties

These properties are defined on `DataView.prototype` and shared by all `DataView` instances.

`DataView.prototype.buffer`

Returns the `ArrayBuffer` referenced by the `DataView`.

`DataView.prototype.byteLength`

Returns the length (in bytes) of the `DataView`.

`DataView.prototype.byteOffset`

Returns the offset (in bytes) of the `DataView` from the start of its `ArrayBuffer`.

`DataView.prototype.constructor`

The constructor function that created the instance object. For `DataView` instances, the initial value is the `DataView` constructor.

`DataView.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"DataView"`. This property is used in `Object.prototype.toString()`.

Instance methods

`DataView.prototype.getBigInt64()`

Reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit signed integer.

`DataView.prototype.getBigUint64()`

Reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit unsigned integer.

`DataView.prototype.getFloat16()`

Reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit floating point number.

`DataView.prototype.getFloat32()`

Reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit floating point number.

`DataView.prototype.getFloat64()`

Reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit floating point number.

`DataView.prototype.getInt16()`

Reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit signed integer.

`DataView.prototype.getInt32()`

Reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit signed integer.

`DataView.prototype.getInt8()`

Reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit signed integer.

`DataView.prototype.getUint16()`

Reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit unsigned integer.

`DataView.prototype.getUint32()`

Reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit unsigned integer.

`DataView.prototype.getUint8()`

Reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit unsigned integer.

`DataView.prototype.setBigInt64()`

Takes a BigInt and stores it as a 64-bit signed integer in the 8 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setBigUint64()`

Takes a BigInt and stores it as a 64-bit unsigned integer in the 8 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setFloat16()`

Takes a number and stores it as a 16-bit float in the 2 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setFloat32()`

Takes a number and stores it as a 32-bit float in the 4 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setFloat64()`

Takes a number and stores it as a 64-bit float in the 8 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setInt16()`

Takes a number and stores it as a 16-bit signed integer in the 2 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setInt32()`

Takes a number and stores it as a 32-bit signed integer in the 4 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setInt8()`

Takes a number and stores it as an 8-bit signed integer in the byte at the specified byte offset of this `DataView`.

`DataView.prototype.setUint16()`

Takes a number and stores it as a 16-bit unsigned integer in the 2 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setUint32()`

Takes a number and stores it as a 32-bit unsigned integer in the 4 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setUint8()`

Takes a number and stores it as an 8-bit unsigned integer in the byte at the specified byte offset of this `DataView`.

Examples

Using DataView

```
const buffer = new ArrayBuffer(16);
const view = new DataView(buffer, 0);

view.setInt16(1, 42);
view.getInt16(1); // 42
```

See also

- Polyfill of `DataView` in `core-js`
- `ArrayBuffer`
- `SharedArrayBuffer`

DataView.prototype.buffer

The **`buffer`** accessor property of `DataView` instances returns the `ArrayBuffer` or `SharedArrayBuffer` referenced by this view at construction time.

Try it

```
// Create an ArrayBuffer
const buffer = new ArrayBuffer(123);

// Create a view
const view = new DataView(buffer);

console.log(view.buffer.byteLength);
// Expected output: 123
```

Description

The `buffer` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the `DataView` is constructed and cannot be changed.

Examples

Using the buffer property

```
const buffer = new ArrayBuffer(8);
const dataview = new DataView(buffer);
dataview.buffer; // ArrayBuffer { byteLength: 8 }
```

See also

- `DataView`
- `ArrayBuffer`
- `SharedArrayBuffer`

DataView.prototype.byteLength

The **`byteLength`** accessor property of `DataView` instances returns the length (in bytes) of this view.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view1 = new DataView(buffer);
const view2 = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes

console.log(view1.byteLength + view2.byteLength); // 16 + 4
// Expected output: 20
```

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. If the `DataView` is length-tracking, then its length depends on the length of the underlying buffer, and may change if the buffer is resized. Otherwise, the value is established when the `DataView` is constructed and cannot be changed. Whether length-tracking or not, the `byteLength` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteLength property

```
const buffer = new ArrayBuffer(8);
const dataview = new DataView(buffer);
dataview.byteLength; // 8 (matches the byteLength of the buffer)

const dataview2 = new DataView(buffer, 1, 5);
dataview2.byteLength; // 5 (as specified when constructing the DataView)

const dataview3 = new DataView(buffer, 2);
dataview3.byteLength; // 6 (due to the offset of the constructed DataView)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const dataviewLengthTracking = new DataView(buffer2, 4);
dataviewLengthTracking.byteLength; // 12 (16 - 4)
buffer2.resize(20);
dataviewLengthTracking.byteLength; // 16 (20 - 4)
buffer2.resize(3);
dataviewLengthTracking.byteLength; // 0 (viewed range is no longer valid)
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `SharedArrayBuffer`

DataView.prototype.byteOffset

The **`byteOffset`** accessor property of `DataView` instances returns the offset (in bytes) of this view from the start of its `ArrayBuffer` or `SharedArrayBuffer`.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes

console.log(view.byteOffset);
// Expected output: 12
```

Description

The `byteOffset` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the `DataView` is constructed and cannot be changed. However, the `byteOffset` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteOffset property

```
const buffer = new ArrayBuffer(8);
const dataview = new DataView(buffer);
dataview.byteOffset; // 0 (no offset specified)

const dataview2 = new DataView(buffer, 3);
dataview2.byteOffset; // 3 (as specified when constructing the DataView)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const dataviewLengthTracking = new DataView(buffer2, 4);
dataviewLengthTracking.byteOffset; // 4
buffer2.resize(3);
dataviewLengthTracking.byteOffset; // 0 (viewed range is no longer valid)
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `SharedArrayBuffer`

DataView() constructor

The **`DataView()`** constructor creates `DataView` objects.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Create a couple of views
const view1 = new DataView(buffer);
const view2 = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes
view1.setInt8(12, 42); // Put 42 in slot 12

console.log(view2.getInt8(0));
// Expected output: 42
```

Syntax

```
new DataView(buffer)
new DataView(buffer, byteOffset)
new DataView(buffer, byteOffset, byteLength)
```

> **Note:** `DataView()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`buffer`

An existing `ArrayBuffer` or `SharedArrayBuffer` to use as the storage backing the new `DataView` object.

`byteOffset` Optional

The offset, in bytes, to the first byte in the above buffer for the new view to reference. If unspecified, the buffer view starts with the first byte.

`byteLength` Optional

The number of elements in the byte array. If unspecified, the view's length will match the buffer's length.

Return value

A new `DataView` object representing the specified data buffer.

Exceptions

`RangeError`

Thrown if the `byteOffset` or `byteLength` parameter values result in the view extending past the end of the buffer. In other words, `byteOffset + byteLength > buffer.byteLength`.

Examples

Using DataView

```
const buffer = new ArrayBuffer(16);
const view = new DataView(buffer, 0);

view.setInt16(1, 42);
view.getInt16(1); // 42
```

See also

- Polyfill of `DataView` in `core-js`
- `DataView`

DataView.prototype.getBigInt64()

The **`getBigInt64()`** method of `DataView` instances reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit signed integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in a signed 64-bit integer
const max = 2n ** (64n - 1n) - 1n;

const view = new DataView(buffer);
view.setBigInt64(1, max);

console.log(view.getBigInt64(1));
// Expected output: 92233720368547758O7n
```

Syntax

```
getBigInt64(byteOffset)
getBigInt64(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A `BigInt` from -2^{63} to $2^{63}-1$, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getBigInt64()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getBigInt64(1)); // 72623859790382856n
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`

- `BigInt64Array`

DataView.prototype.getBigUint64()

The **`getBigUint64()`** method of `DataView` instances reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit unsigned integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in an unsigned 64-bit integer
const max = 2n ** 64n - 1n;

const view = new DataView(buffer);
view.setBigUint64(1, max);

console.log(view.getBigUint64(1));
// Expected output: 18446744073709551615n
```

Syntax

```
getBigUint64(byteOffset)
getBigUint64(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A `BigInt` from 0 to 2^{64}-1, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getBigUint64()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getBigUint64(1)); // 72623859790382856n
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `BigUint64Array`

DataView.prototype.getFloat16()

The **`getFloat16()`** method of `DataView` instances reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit floating point number. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat16(1, Math.PI);

console.log(view.getFloat16(1));
// Expected output: 3.140625
```

Syntax

```
getFloat16(byteOffset)
getFloat16(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A floating point number from `-65504` to `65504`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getFloat16()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getFloat16(1)); // 0.00091537799835205078
```

See also

- Polyfill of `DataView.prototype.getFloat16` in `core-js`
- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float16Array`

DataView.prototype.getFloat32()

The **`getFloat32()`** method of `DataView` instances reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit floating point number. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat32(1, Math.PI);

console.log(view.getFloat32(1));
// Expected output: 3.1415927410125732
```

Syntax

```
getFloat32(byteOffset)
getFloat32(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A floating point number from `-3.4e38` to `3.4e38`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getFloat32()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
```

```
console.log(dataview.getFloat32(1)); // 2.387939260590663e-38
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float32Array`

DataView.prototype.getFloat64()

The **`getFloat64()`** method of `DataView` instances reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit floating point number. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat64(1, Math.PI);

console.log(view.getFloat64(1));
// Expected output: 3.141592653589793
```

Syntax

```
getFloat64(byteOffset)
getFloat64(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

Any number value.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getFloat64()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getFloat64(1)); // 8.20788039913184e-304
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float64Array`

DataView.prototype.getInt16()

The **`getInt16()`** method of `DataView` instances reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit signed integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt16(1, 32767); // Max signed 16-bit integer

console.log(view.getInt16(1));
// Expected output: 32767
```

Syntax

```
getInt16(byteOffset)
getInt16(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from -32768 to 32767, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getInt16()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getInt16(1)); // 258
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int16Array`

DataView.prototype.getInt32()

The **`getInt32()`** method of `DataView` instances reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit signed integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt32(1, 2147483647); // Max signed 32-bit integer

console.log(view.getInt32(1));
// Expected output: 2147483647
```

Syntax

```
getInt32(byteOffset)
getInt32(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from -2147483648 to 2147483647, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getInt32()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getInt32(1)); // 16909060
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int32Array`

DataView.prototype.getInt8()

The **`getInt8()`** method of `DataView` instances reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit signed integer.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt8(1, 127); // Max signed 8-bit integer

console.log(view.getInt8(1));
// Expected output: 127
```

Syntax

```
getInt8(byteOffset)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

Return value

An integer from -128 to 127, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getInt8()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getInt8(1)); // 1
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int8Array`

DataView.prototype.getUint16()

The **`getUint16()`** method of `DataView` instances reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit unsigned integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint16(1, 65535); // Max unsigned 16-bit integer

console.log(view.getUint16(1));
// Expected output: 65535
```

Syntax

```
getUint16(byteOffset)
getUint16(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from 0 to 65535, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getUint16()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getUint16(1)); // 258
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint16Array`

DataView.prototype.getUint32()

The **`getUint32()`** method of `DataView` instances reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit unsigned integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint32(1, 4294967295); // Max unsigned 32-bit integer

console.log(view.getUint32(1));
// Expected output: 4294967295
```

Syntax

```
getUint32(byteOffset)
getUint32(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from 0 to 4294967295, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getUint32()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getUint32(1)); // 16909060
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint32Array`

DataView.prototype.getUint8()

The **`getUint8()`** method of `DataView` instances reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit unsigned integer.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint8(1, 255); // Max unsigned 8-bit integer

console.log(view.getUint8(1));
// Expected output: 255
```

Syntax

```
getUint8(byteOffset)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

Return value

An integer from 0 to 255, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getUint8()

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getUint8(1)); // 1
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint8Array`

DataView.prototype.setBigInt64()

The **`setBigInt64()`** method of `DataView` instances takes a BigInt and stores it as a 64-bit signed integer in the 8 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in a signed 64-bit integer
const max = 2n ** (64n - 1n) - 1n;

const view = new DataView(buffer);
view.setBigInt64(1, max);

console.log(view.getBigInt64(1));
// Expected output: 9223372036854775807n
```

Syntax

```
setBigInt64(byteOffset, value)
setBigInt64(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set as a `BigInt`. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setBigInt64()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setBigInt64(0, 3n);
dataview.getBigInt64(1); // 768n
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `BigInt64Array`

DataView.prototype.setBigUint64()

The **`setBigUint64()`** method of `DataView` instances takes a BigInt and stores it as a 64-bit unsigned integer in the 8 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in an unsigned 64-bit integer
const max = 2n ** 64n - 1n;

const view = new DataView(buffer);
view.setBigUint64(1, max);

console.log(view.getBigUint64(1));
// Expected output: 18446744073709551615n
```

Syntax

```
setBigUint64(byteOffset, value)
setBigUint64(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set as a `BigInt`. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setBigUint64()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setBigUint64(0, 3n);
dataview.getBigUint64(1); // 768n
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `BigUint64Array`

DataView.prototype.setFloat16()

The **`setFloat16()`** method of `DataView` instances takes a number and stores it as a 16-bit floating point number in the 2 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat16(1, Math.PI);

console.log(view.getFloat16(1));
// Expected output: 3.140625
```

Syntax

```
setFloat16(byteOffset, value)
setFloat16(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setFloat16()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setFloat16(0, 3);
dataview.getFloat16(1); // 0
```

See also

- Polyfill of `DataView.prototype.setFloat16` in `core-js`
- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float16Array`

DataView.prototype.setFloat32()

The **`setFloat32()`** method of `DataView` instances takes a number and stores it as a 32-bit floating point number in the 4 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat32(1, Math.PI);

console.log(view.getFloat32(1));
// Expected output: 3.1415927410125732
```

Syntax

```
setFloat32(byteOffset, value)
setFloat32(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setFloat32()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setFloat32(0, 3);
dataview.getFloat32(1); // 2
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float32Array`

DataView.prototype.setFloat64()

The **`setFloat64()`** method of `DataView` instances takes a number and stores it as a 64-bit floating point number in the 8 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat64(1, Math.PI);

console.log(view.getFloat64(1));
// Expected output: 3.141592653589793
```

Syntax

```
setFloat64(byteOffset, value)
setFloat64(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setFloat64()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setFloat64(0, 3);
dataview.getFloat64(1); // 3.785766995733679e-270
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float64Array`

DataView.prototype.setInt16()

The **`setInt16()`** method of `DataView` instances takes a number and stores it as a 16-bit signed integer in the 2 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt16(1, 32767); // Max signed 16-bit integer

console.log(view.getInt16(1));
// Expected output: 32767
```

Syntax

```
setInt16(byteOffset, value)
setInt16(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setInt16()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setInt16(0, 3);
dataview.getInt16(1); // 768
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int16Array`

DataView.prototype.setInt32()

The **`setInt32()`** method of `DataView` instances takes a number and stores it as a 32-bit signed integer in the 4 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt32(1, 2147483647); // Max signed 32-bit integer

console.log(view.getInt32(1));
// Expected output: 2147483647
```

Syntax

```
setInt32(byteOffset, value)
setInt32(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setInt32()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setInt32(0, 3);
dataview.getInt32(1); // 768
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int32Array`

DataView.prototype.setInt8()

The **`setInt8()`** method of `DataView` instances takes a number and stores it as an 8-bit signed integer in the byte at the specified byte offset of this `DataView`.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
```

```
view.setInt8(1, 127); // Max signed 8-bit integer

console.log(view.getInt8(1));
// Expected output: 127
```

Syntax

```
setInt8(byteOffset, value)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setInt8()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setInt8(0, 3);
dataview.getInt8(0); // 3
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int8Array`

DataView.prototype.setUint16()

The **`setUint16()`** method of `DataView` instances takes a number and stores it as a 16-bit unsigned integer in the 2 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint16(1, 65535); // Max unsigned 16-bit integer

console.log(view.getUint16(1));
// Expected output: 65535
```

Syntax

```
setUint16(byteOffset, value)
setUint16(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setUint16()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setUint16(0, 3);
dataview.getUint16(1); // 768
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint16Array`

DataView.prototype.setUint32()

The **`setUint32()`** method of `DataView` instances takes a number and stores it as a 32-bit unsigned integer in the 4 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint32(1, 4294967295); // Max unsigned 32-bit integer

console.log(view.getUint32(1));
// Expected output: 4294967295
```

Syntax

```
setUint32(byteOffset, value)
setUint32(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setUint32()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setUint32(0, 3);
dataview.getUint32(1); // 768
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint32Array`

DataView.prototype.setUint8()

The **`setUint8()`** method of `DataView` instances takes a number and stores it as an 8-bit unsigned integer in the byte at the specified byte offset of this `DataView`.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint8(1, 255); // Max unsigned 8-bit integer

console.log(view.getUint8(1));
// Expected output: 255
```

Syntax

```
setUint8(byteOffset, value)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setUint8()

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setUint8(0, 3);
dataview.getUint8(0); // 3
```

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint8Array`

Date

JavaScript **`Date`** objects represent a single moment in time in a platform-independent format. `Date` objects encapsulate an integral number that represents milliseconds since the midnight at the beginning of January 1, 1970, UTC (the *epoch*).

> **Note:** With the introduction of the `Temporal` API, the `Date` object is considered a legacy feature. Consider using `Temporal` for new code and migrate existing code over to it if possible (check the browser compatibility. We will be writing a usage guide soon!

Description

The epoch, timestamps, and invalid date

A JavaScript date is fundamentally specified as the time in milliseconds that has elapsed since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC (equivalent to the UNIX epoch). This timestamp is *timezone-agnostic* and uniquely defines an instant in history.

> **Note:** While the time value at the heart of a Date object is UTC, the basic methods to fetch the date and time or its components all work in the local (i.e., host system) time zone and offset.

The maximum timestamp representable by a `Date` object is slightly smaller than the maximum safe integer (`Number.MAX_SAFE_INTEGER`, which is 9,007,199,254,740,991). A `Date` object can represent a maximum of ±8,640,000,000,000,000 milliseconds, or ±100,000,000 (one hundred million) days, relative to the epoch. This is the range from April 20, 271821 BC to September 13, 275760 AD. Any attempt to represent a time outside this range results in the `Date` object holding a timestamp value of `NaN`, which is an "Invalid Date".

```
console.log(new Date(8.64e15).toString()); // "Sat Sep 13 275760 00:00:00 GMT+0000 (Coordinated Universal Time)"
console.log(new Date(8.64e15 + 1).toString()); // "Invalid Date"
```

There are various methods that allow you to interact with the timestamp stored in the date:

- You can interact with the timestamp value directly using the `getTime()` and `setTime()` methods.
- The `valueOf()` and `[Symbol.toPrimitive]()` (when passed `"number"`) methods — which are automatically called in number coercion — return the timestamp, causing `Date` objects to behave like their timestamps when used in number contexts.
- All static methods (`Date.now()`, `Date.parse()`, and `Date.UTC()`) return timestamps instead of `Date` objects.
- The `Date()` constructor can be called with a timestamp as the only argument.

Date components and time zones

A date is represented internally as a single number, the *timestamp*. When interacting with it, the timestamp needs to be interpreted as a structured date-and-time representation. There are always two ways to interpret a timestamp: as a local time or as a Coordinated Universal Time (UTC), the global standard time defined by the World Time Standard. The local timezone is not stored in the date object, but is determined by the host environment (user's device).

> **Note:** UTC should not be confused with the Greenwich Mean Time (GMT), because they are not always equal — this is explained in more detail in the linked Wikipedia page.

For example, the timestamp 0 represents a unique instant in history, but it can be interpreted in two ways:

- As a UTC time, it is midnight at the beginning of January 1, 1970, UTC,
- As a local time in New York (UTC-5), it is 19:00:00 on December 31, 1969.

The `getTimezoneOffset()` method returns the difference between UTC and the local time in minutes. Note that the timezone offset does not only depend on the current timezone, but also on the time represented by the `Date` object, because of daylight saving time and historical changes. In essence, the timezone offset is the offset from UTC time, at the time represented by the `Date` object and at the location of the host environment.

There are two groups of `Date` methods: one group gets and sets various date components by interpreting the timestamp as a local time, while the other uses UTC.

Component	Local		UTC	
	Get	Set	Get	Set
Year	`getFullYear()`	`setFullYear()`	`getUTCFullYear()`	`setUTCFullYear()`
Month	`getMonth()`	`setMonth()`	`getUTCMonth()`	`setUTCMonth()`
Date (of month)	`getDate()`	`setDate()`	`getUTCDate()`	`setUTCDate()`
Hours	`getHours()`	`setHours()`	`getUTCHours()`	`setUTCHours()`
Minutes	`getMinutes()`	`setMinutes()`	`getUTCMinutes()`	`setUTCMinutes()`
Seconds	`getSeconds()`	`setSeconds()`	`getUTCSeconds()`	`setUTCSeconds()`
Milliseconds	`getMilliseconds()`	`setMilliseconds()`	`getUTCMilliseconds()`	`setUTCMilliseconds()`
Day (of week)	`getDay()`	N/A	`getUTCDay()`	N/A

The `Date()` constructor can be called with two or more arguments, in which case they are interpreted as the year, month, day, hour, minute, second, and millisecond, respectively, in local time. `Date.UTC()` works similarly, but it interprets the components as UTC time and also accepts a single argument representing the year.

> **Note:** Some methods, including the `Date()` constructor, `Date.UTC()`, and the deprecated `getYear()`/`setYear()` methods, interpret a two-digit year as a year in the 1900s. For example, `new Date(99, 5, 24)` is interpreted as June 24, 1999, not June 24, 99. See Interpretation of two-digit years for more information.

When a segment overflows or underflows its expected range, it usually "carries over to" or "borrows from" the higher segment. For example, if the month is set to 12 (months are zero-based, so December is 11), it becomes the January of the next year. If the day of month is set to 0, it becomes the last day of the previous month. This also applies to dates specified with the date time string format.

When attempting to set the local time to a time falling within an offset transition (usually daylight saving time), the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

```
// Assume America/New_York local time zone
// 2024-03-10 02:30 is within the spring-forward transition and does not exist
// 01:59 (UTC-5) jumps to 03:00 (UTC-4), so 02:30 moves forward by one hour
console.log(new Date(2024, 2, 10, 2, 30).toString());
// Sun Mar 10 2024 03:30:00 GMT-0400 (Eastern Daylight Time)

// 2024-11-03 01:30 is within the fall-back transition and exists twice
// 01:59 (UTC-4) jumps to 01:00 (UTC-5), so the earlier 01:30 (UTC-4) is chosen
console.log(new Date(2024, 10, 3, 1, 30).toString());
// Sun Nov 03 2024 01:30:00 GMT-0400 (Eastern Daylight Time)
```

Date time string format

There are many ways to format a date as a string. The JavaScript specification only specifies one format to be universally supported: the *date time string format*, a simplification of the ISO 8601 calendar date extended format. The format is as follows:

```
YYYY-MM-DDTHH:mm:ss.sssZ
```

- `YYYY` is the year, with four digits (`0000` to `9999`), or as an *expanded year* of `+` or `-` followed by six digits. The sign is required for expanded years. `-000000` is explicitly disallowed as a valid year.
- `MM` is the month, with two digits (`01` to `12`). Defaults to `01`.
- `DD` is the day of the month, with two digits (`01` to `31`). Defaults to `01`.
- `T` is a literal character, which indicates the beginning of the *time* part of the string. The `T` is required when specifying the time part.
- `HH` is the hour, with two digits (`00` to `23`). As a special case, `24:00:00` is allowed, and is interpreted as midnight at the beginning of the next day. Defaults to `00`.
- `mm` is the minute, with two digits (`00` to `59`). Defaults to `00`.
- `ss` is the second, with two digits (`00` to `59`). Defaults to `00`.
- `sss` is the millisecond, with three digits (`000` to `999`). Defaults to `000`.
- `Z` is the timezone offset, which can either be the literal character `Z` (indicating UTC), or `+` or `-` followed by `HH:mm`, the offset in hours and minutes from UTC.

Various components can be omitted, so the following are all valid:

- Date-only form: `YYYY`, `YYYY-MM`, `YYYY-MM-DD`
- Date-time form: one of the above date-only forms, followed by `T`, followed by `HH:mm`, `HH:mm:ss`, or `HH:mm:ss.sss`. Each combination can be followed by a time zone offset.

For example, `"2011-10-10"` (*date-only* form), `"2011-10-10T14:48:00"` (*date-time* form), or `"2011-10-10T14:48:00.000+09:00"` (*date-time* form with milliseconds and time zone) are all valid date time strings.

When the time zone offset is absent, **date-only forms are interpreted as a UTC time and date-time forms are interpreted as a local time.** The interpretation as a UTC time is due to a historical spec error that was not consistent with ISO 8601 but could not be changed due to web compatibility. See Broken Parser – A Web Reality Issue.

`Date.parse()` and the `Date()` constructor both accept strings in the date time string format as input. Furthermore, implementations are allowed to support other date formats when the input fails to match this format.

The `toISOString()` method returns a string representation of the date in the date time string format, with the time zone offset always set to `Z` (UTC).

> **Note:** You are encouraged to make sure your input conforms to the date time string format above for maximum compatibility, because support for other formats is not guaranteed. However, there are some formats that are supported in all major implementations — like RFC 2822 format — in which case their usage can be acceptable. Always conduct cross-browser tests to ensure your code works in all target browsers. A library can help if many different formats are to be accommodated.

Non-standard strings can be parsed in any way as desired by the implementation, including the time zone — most implementations use the local time zone by default. Implementations are not required to return invalid date for out-of-bounds date components, although they usually do. A string may have in-bounds date components (with the bounds defined above), but does not represent a date in reality (for example, "February 30"). Implementations behave inconsistently in this case. The `Date.parse()` page offers more examples about these non-standard cases.

Other ways to format a date

- `toISOString()` returns a string in the format `1970-01-01T00:00:00.000Z` (the date time string format introduced above, which is simplified ISO 8601). `toJSON()` calls `toISOString()` and returns the result.
- `toString()` returns a string in the format `Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)`, while `toDateString()` and `toTimeString()` return the date and time parts of the string, respectively. `[Symbol.toPrimitive]()` (when passed `"string"` or `"default"`) calls `toString()` and returns the result.
- `toUTCString()` returns a string in the format `Thu, 01 Jan 1970 00:00:00 GMT` (generalized RFC 7231).
- `toLocaleDateString()`, `toLocaleTimeString()`, and `toLocaleString()` use locale-specific date and time formats, usually provided by the `Intl` API.

See the Formats of `toString` method return values section for examples.

Constructor

`Date()`

When called as a constructor, returns a new `Date` object. When called as a function, returns a string representation of the current date and time.

Static methods

`Date.now()`

Returns the numeric value corresponding to the current time—the number of milliseconds since January 1, 1970 00:00:00 UTC, with leap seconds ignored.

`Date.parse()`

Parses a string representation of a date and returns the number of milliseconds since January 1, 1970 00:00:00 UTC, with leap seconds ignored.

`Date.UTC()`

Accepts the same parameters as the longest form of the constructor (i.e., 2 to 7) and returns the number of milliseconds since January 1, 1970 00:00:00 UTC, with leap seconds ignored.

Instance properties

These properties are defined on `Date.prototype` and shared by all `Date` instances.

`Date.prototype.constructor`

The constructor function that created the instance object. For `Date` instances, the initial value is the `Date` constructor.

Instance methods

`Date.prototype.getDate()`

Returns the day of the month (`1` – `31`) for the specified date according to local time.

`Date.prototype.getDay()`

Returns the day of the week (`0` – `6`) for the specified date according to local time.

`Date.prototype.getFullYear()`

Returns the year (4 digits for 4-digit years) of the specified date according to local time.

`Date.prototype.getHours()`

Returns the hour (`0` – `23`) in the specified date according to local time.

`Date.prototype.getMilliseconds()`

Returns the milliseconds (`0` – `999`) in the specified date according to local time.

`Date.prototype.getMinutes()`

Returns the minutes (`0` – `59`) in the specified date according to local time.

`Date.prototype.getMonth()`

Returns the month (`0` – `11`) in the specified date according to local time.

`Date.prototype.getSeconds()`

Returns the seconds (`0` – `59`) in the specified date according to local time.

`Date.prototype.getTime()`

Returns the numeric value of the specified date as the number of milliseconds since January 1, 1970 00:00:00 UTC. (Negative values are returned for prior times.)

`Date.prototype.getTimezoneOffset()`

Returns the time-zone offset in minutes for the current locale.

`Date.prototype.getUTCDate()`

Returns the day (date) of the month (`1` – `31`) in the specified date according to universal time.

`Date.prototype.getUTCDay()`

Returns the day of the week (`0` – `6`) in the specified date according to universal time.

`Date.prototype.getUTCFullYear()`

Returns the year (4 digits for 4-digit years) in the specified date according to universal time.

`Date.prototype.getUTCHours()`

Returns the hours (`0` – `23`) in the specified date according to universal time.

`Date.prototype.getUTCMilliseconds()`

Returns the milliseconds (`0` – `999`) in the specified date according to universal time.

`Date.prototype.getUTCMinutes()`

Returns the minutes (`0` – `59`) in the specified date according to universal time.

`Date.prototype.getUTCMonth()`

Returns the month (`0` – `11`) in the specified date according to universal time.

`Date.prototype.getUTCSeconds()`

Returns the seconds (`0` – `59`) in the specified date according to universal time.

`Date.prototype.getYear()`

Returns the year (usually 2–3 digits) in the specified date according to local time. Use `getFullYear()` instead.

`Date.prototype.setDate()`

Sets the day of the month for a specified date according to local time.

`Date.prototype.setFullYear()`

Sets the full year (e.g., 4 digits for 4-digit years) for a specified date according to local time.

`Date.prototype.setHours()`

Sets the hours for a specified date according to local time.

`Date.prototype.setMilliseconds()`

Sets the milliseconds for a specified date according to local time.

`Date.prototype.setMinutes()`

Sets the minutes for a specified date according to local time.

`Date.prototype.setMonth()`

Sets the month for a specified date according to local time.

`Date.prototype.setSeconds()`

Sets the seconds for a specified date according to local time.

`Date.prototype.setTime()`

Sets the `Date` object to the time represented by the number of milliseconds since January 1, 1970 00:00:00 UTC. Use negative numbers for times prior.

`Date.prototype.setUTCDate()`

Sets the day of the month for a specified date according to universal time.

`Date.prototype.setUTCFullYear()`

Sets the full year (e.g., 4 digits for 4-digit years) for a specified date according to universal time.

`Date.prototype.setUTCHours()`

Sets the hour for a specified date according to universal time.

`Date.prototype.setUTCMilliseconds()`

Sets the milliseconds for a specified date according to universal time.

`Date.prototype.setUTCMinutes()`

Sets the minutes for a specified date according to universal time.

`Date.prototype.setUTCMonth()`

Sets the month for a specified date according to universal time.

`Date.prototype.setUTCSeconds()`

Sets the seconds for a specified date according to universal time.

`Date.prototype.setYear()`

Sets the year (usually 2–3 digits) for a specified date according to local time. Use `setFullYear()` instead.

`Date.prototype.toDateString()`

Returns the "date" portion of the `Date` as a human-readable string like `'Thu Apr 12 2018'`.

`Date.prototype.toISOString()`

Converts a date to a string following the ISO 8601 Extended Format.

`Date.prototype.toJSON()`

Returns a string representing the `Date` using `toISOString()`. Intended to be implicitly called by `JSON.stringify()`.

`Date.prototype.toLocaleDateString()`

Returns a string with a locality sensitive representation of the date portion of this date based on system settings.

`Date.prototype.toLocaleString()`

Returns a string with a locality-sensitive representation of this date. Overrides the `Object.prototype.toLocaleString()` method.

`Date.prototype.toLocaleTimeString()`

Returns a string with a locality-sensitive representation of the time portion of this date, based on system settings.

`Date.prototype.toString()`

Returns a string representing the specified `Date` object. Overrides the `Object.prototype.toString()` method.

`Date.prototype.toTemporalInstant()`

Returns a new `Temporal.Instant` object with the same `epochMilliseconds` value as this date's timestamp.

`Date.prototype.toTimeString()`

Returns the "time" portion of the `Date` as a human-readable string.

`Date.prototype.toUTCString()`

Converts a date to a string using the UTC timezone.

`Date.prototype.valueOf()`

Returns the primitive value of a `Date` object. Overrides the `Object.prototype.valueOf()` method.

`Date.prototype[Symbol.toPrimitive]()`

Converts this `Date` object to a primitive value.

Examples

Several ways to create a Date object

The following examples show several ways to create JavaScript dates:

> **Note:** Creating a date from a string has a lot of behavior inconsistencies. See date time string format for caveats on using different formats.

```
const today = new Date();
const birthday = new Date("December 17, 1995 03:24:00"); // DISCOURAGED: may not work in all runtimes
const birthday2 = new Date("1995-12-17T03:24:00"); // This is standardized and will work reliably
const birthday3 = new Date(1995, 11, 17); // the month is 0-indexed
const birthday4 = new Date(1995, 11, 17, 3, 24, 0);
const birthday5 = new Date(628021800000); // passing epoch timestamp
```

Formats of toString method return values

```
const date = new Date("2020-05-12T23:50:21.817Z");
date.toString(); // Tue May 12 2020 18:50:21 GMT-0500 (Central Daylight Time)
date.toDateString(); // Tue May 12 2020
date.toTimeString(); // 18:50:21 GMT-0500 (Central Daylight Time)
date[Symbol.toPrimitive]("string"); // Tue May 12 2020 18:50:21 GMT-0500 (Central Daylight Time)

date.toISOString(); // 2020-05-12T23:50:21.817Z
date.toJSON(); // 2020-05-12T23:50:21.817Z

date.toUTCString(); // Tue, 12 May 2020 23:50:21 GMT

date.toLocaleString(); // 5/12/2020, 6:50:21 PM
date.toLocaleDateString(); // 5/12/2020
date.toLocaleTimeString(); // 6:50:21 PM
```

To get Date, Month and Year or Time

```
const date = new Date("2000-01-17T16:45:30");
const [month, day, year] = [
  date.getMonth(),
  date.getDate(),
  date.getFullYear(),
];
// [0, 17, 2000] as month are 0-indexed
const [hour, minutes, seconds] = [
  date.getHours(),
  date.getMinutes(),
  date.getSeconds(),
];
// [16, 45, 30]
```

Interpretation of two-digit years

`new Date()` exhibits legacy undesirable, inconsistent behavior with two-digit year values; specifically, when a `new Date()` call is given a two-digit year value, that year value does not get treated as a literal year and used as-is but instead gets interpreted as a relative offset — in some cases as an offset from the year `1900`, but in other cases, as an offset from the year `2000`.

```
let date = new Date(98, 1); // Sun Feb 01 1998 00:00:00 GMT+0000 (GMT)
date = new Date(22, 1); // Wed Feb 01 1922 00:00:00 GMT+0000 (GMT)
date = new Date("2/1/22"); // Tue Feb 01 2022 00:00:00 GMT+0000 (GMT)

// Legacy method; always interprets two-digit year values as relative to 1900
date.setYear(98);
date.toString(); // Sun Feb 01 1998 00:00:00 GMT+0000 (GMT)
date.setYear(22);
date.toString(); // Wed Feb 01 1922 00:00:00 GMT+0000 (GMT)
```

So, to create and get dates between the years `0` and `99`, instead use the preferred `setFullYear()` and `getFullYear()` methods:.

```
// Preferred method; never interprets any value as being a relative offset,
// but instead uses the year value as-is
date.setFullYear(98);
date.getFullYear(); // 98 (not 1998)
date.setFullYear(22);
date.getFullYear(); // 22 (not 1922, not 2022)
```

Calculating elapsed time

The following examples show how to determine the elapsed time between two JavaScript dates in milliseconds.

Due to the differing lengths of days (due to daylight saving changeover), months, and years, expressing elapsed time in units greater than hours, minutes, and seconds requires addressing a number of issues, and should be thoroughly researched before being attempted.

```
// Using Date objects
const start = Date.now();

// The event to time goes here:
doSomethingForALongTime();
const end = Date.now();
const elapsed = end - start; // elapsed time in milliseconds
```

```
// Using built-in methods
const start = new Date();

// The event to time goes here:
doSomethingForALongTime();
const end = new Date();
const elapsed = end.getTime() - start.getTime(); // elapsed time in milliseconds
```

```
// To test a function and get back its return
function printElapsedTime(testFn) {
  const startTime = Date.now();
  const result = testFn();
  const endTime = Date.now();

  console.log(`Elapsed time: ${String(endTime - startTime)} milliseconds`);
  return result;
}

const yourFunctionReturn = printElapsedTime(yourFunction);
```

> **Note:** In browsers that support the Performance API's high-resolution time feature, `Performance.now()` can provide more reliable and precise measurements of elapsed time than `Date.now()`.

Get the number of seconds since the ECMAScript Epoch

```
const seconds = Math.floor(Date.now() / 1000);
```

In this case, it's important to return only an integer—so a simple division won't do. It's also important to only return actually elapsed seconds. (That's why this code uses `Math.floor()`, and *not* `Math.round()`.)

See also

- `Date()`

Date() constructor

The **`Date()`** constructor creates `Date` objects. When called as a function, it returns a string representing the current time.

Try it

```
const date1 = new Date("December 17, 1995 03:24:00");
// Sun Dec 17 1995 03:24:00 GMT...

const date2 = new Date("1995-12-17T03:24:00");
// Sun Dec 17 1995 03:24:00 GMT...

console.log(date1.getTime() === date2.getTime());
// Expected output: true
```

Syntax

```
new Date()
new Date(value)
new Date(dateString)
new Date(dateObject)

new Date(year, monthIndex)
new Date(year, monthIndex, day)
```

```
new Date(year, monthIndex, day, hours)
new Date(year, monthIndex, day, hours, minutes)
new Date(year, monthIndex, day, hours, minutes, seconds)
new Date(year, monthIndex, day, hours, minutes, seconds, milliseconds)

Date()
```

> **Note:** `Date()` can be called with or without `new`, but with different effects. See Return value.

Parameters

There are five basic forms for the `Date()` constructor:

No parameters

When no parameters are provided, the newly-created `Date` object represents the current date and time as of the time of instantiation. The returned date's timestamp is the same as the number returned by `Date.now()`.

Time value or timestamp number

`value`

An integer value representing the timestamp (the number of milliseconds since midnight at the beginning of January 1, 1970, UTC — a.k.a. the epoch).

Date string

`dateString`

A string value representing a date, parsed and interpreted using the same algorithm implemented by `Date.parse()`. See date time string format for caveats on using different formats.

Date object

`dateObject`

An existing `Date` object. This effectively makes a copy of the existing `Date` object with the same date and time. This is equivalent to `new Date(dateObject.valueOf())`, except the `valueOf()` method is not called.

When one parameter is passed to the `Date()` constructor, `Date` instances are specially treated. All other values are converted to primitives. If the result is a string, it will be parsed as a date string. Otherwise, the resulting primitive is further coerced to a number and treated as a timestamp.

Individual date and time component values

Given at least a year and month, this form of `Date()` returns a `Date` object whose component values (year, month, day, hour, minute, second, and millisecond) all come from the following parameters. Any missing fields are given the lowest possible value (`1` for `day` and `0` for every other component). The parameter values are all evaluated against the local time zone, rather than UTC. `Date.UTC()` accepts similar parameters but interprets the components as UTC and returns a timestamp.

If any parameter overflows its defined bounds, it "carries over". For example, if a `monthIndex` greater than `11` is passed in, those months will cause the year to increment; if a `minutes` greater than `59` is passed in, `hours` will increment accordingly, etc. Therefore, `new Date(1990, 12, 1)` will return January 1st, 1991; `new Date(2020, 5, 19, 25, 65)` will return 2:05 A.M. June 20th, 2020.

Similarly, if any parameter underflows, it "borrows" from the higher positions. For example, `new Date(2020, 5, 0)` will return May 31st, 2020.

`year`

Integer value representing the year. Values from `0` to `99` map to the years `1900` to `1999`. All other values are the actual year. See the example.

`monthIndex`

Integer value representing the month, beginning with `0` for January to `11` for December.

`day` Optional

Integer value representing the day of the month. Defaults to `1`.

`hours` Optional

Integer value between `0` and `23` representing the hour of the day. Defaults to `0`.

`minutes` Optional

Integer value representing the minute segment of a time. Defaults to `0`.

`seconds` Optional

Integer value representing the second segment of a time. Defaults to `0`.

`milliseconds` Optional

Integer value representing the millisecond segment of a time. Defaults to `0`.

Return value

Calling `new Date()` (the `Date()` constructor) returns a `Date` object. If called with an invalid date string, or if the date to be constructed will have a timestamp less than `-8,640,000,000,000,000` or greater than `8,640,000,000,000,000` milliseconds, it returns an invalid date (a `Date` object whose `toString()` method returns `"Invalid Date"` and `valueOf()` method returns `NaN`).

Calling the `Date()` function (without the `new` keyword) returns a string representation of the current date and time, exactly as `new Date().toString()` does. Any arguments given in a `Date()` function call (without the `new` keyword) are ignored; regardless of whether it's called with an invalid date string — or even called with any arbitrary object or other primitive as an argument — it always returns a string representation of the current date and time.

Description

Reduced time precision

To offer protection against timing attacks and fingerprinting, the precision of `new Date()` may be reduced depending on browser settings.

When a timestamp, date string, date object, or date component values are supplied to the constructor, the browser does not apply timer rounding to the supplied time.

When called without arguments, the constructor obtains the current time in the same way as `Date.now()`. It inherits the precision of that clock reading without introducing additional inaccuracy.

Examples

Several ways to create a Date object

The following examples show several ways to create JavaScript dates:

```
const today = new Date();
const birthday = new Date("December 17, 1995 03:24:00"); // DISCOURAGED: may not work in all runtimes
const birthday = new Date("1995-12-17T03:24:00"); // This is standardized and will work reliably
const birthday = new Date(1995, 11, 17); // the month is 0-indexed
const birthday = new Date(1995, 11, 17, 3, 24, 0);
const birthday = new Date(628021800000); // passing epoch timestamp
```

Passing a non-Date, non-string, non-number value

If the `Date()` constructor is called with one parameter which is not a `Date` instance, it will be coerced to a primitive and then checked whether it's a string. For example, `new Date(undefined)` is different from `new Date()`:

```
console.log(new Date(undefined)); // Invalid Date
```

This is because `undefined` is already a primitive but not a string, so it will be coerced to a number, which is `NaN` and therefore not a valid timestamp. On the other hand, `null` will be coerced to `0`.

```
console.log(new Date(null)); // 1970-01-01T00:00:00.000Z
```

Arrays would be coerced to a string via `Array.prototype.toString()`, which joins the elements with commas. However, the resulting string for any array with more than one element is not a valid ISO 8601 date string, so its parsing behavior would be implementation-defined. **Do not pass arrays to the `Date()` constructor.**

```
console.log(new Date(["2020-06-19", "17:13"]));
// 2020-06-19T17:13:00.000Z in Chrome, since it recognizes "2020-06-19,17:13"
// "Invalid Date" in Firefox
```

See also

- `Date`

Date.prototype.getDate()

The **`getDate()`** method of `Date` instances returns the day of the month for this date according to local time.

Try it

```
const birthday = new Date("August 19, 1975 23:15:30");
const date = birthday.getDate();

console.log(date);
// Expected output: 19
```

Syntax

```
getDate()
```

Parameters

None.

Return value

An integer, between 1 and 31, representing the day of the month for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getDate()

The `day` variable has value `25`, based on the value of the `Date` object `xmas95`.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const day = xmas95.getDate();

console.log(day); // 25
```

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setDate()`

Date.prototype.getDay()

The **`getDay()`** method of `Date` instances returns the day of the week for this date according to local time, where 0 represents Sunday. For the day of the month, see `Date.prototype.getDate()`.

Try it

```
const birthday = new Date("August 19, 1975 23:15:30");
const day1 = birthday.getDay();
// Sunday - Saturday : 0 - 6

console.log(day1);
// Expected output: 2
```

Syntax

```
getDay()
```

Parameters

None.

Return value

An integer, between 0 and 6, representing the day of the week for the given date according to local time: 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on. Returns `NaN` if the date is invalid.

Description

The return value of `getDay()` is zero-based, which is useful for indexing into arrays of days, for example:

```
const valentines = new Date("1995-02-14");
const day = valentines.getDay();
const dayNames = ["Sunday", "Monday", "Tuesday" /* , … */];

console.log(dayNames[day]); // "Monday"
```

However, for the purpose of internationalization, you should prefer using `Intl.DateTimeFormat` with the `options` parameter instead.

```
const options = { weekday: "long" };
console.log(new Intl.DateTimeFormat("en-US", options).format(valentines));
// "Monday"
console.log(new Intl.DateTimeFormat("de-DE", options).format(valentines));
// "Montag"
```

Examples

Using getDay()

The `weekday` variable has value `1`, based on the value of the `Date` object `xmas95`, because December 25, 1995 is a Monday.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const weekday = xmas95.getDay();

console.log(weekday); // 1
```

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setDate()`

Date.prototype.getFullYear()

The **`getFullYear()`** method of `Date` instances returns the year for this date according to local time.

Use this method instead of the `getYear()` method.

Try it

```
const moonLanding = new Date("July 20, 69 00:20:18");

console.log(moonLanding.getFullYear());
// Expected output: 1969
```

Syntax

```
getFullYear()
```

Parameters

None.

Return value

An integer representing the year for the given date according to local time. Returns `NaN` if the date is invalid.

Description

Unlike `getYear()`, the value returned by `getFullYear()` is an absolute number. For dates between the years 1000 and 9999, `getFullYear()` returns a four-digit number, for example, 1995. Use this function to make sure a year is compliant with years after 2000.

Examples

Using getFullYear()

The `fullYear` variable has value `1995`, based on the value of the `Date` object `xmas95`.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const fullYear = xmas95.getFullYear();

console.log(fullYear); // 1995
```

See also

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`
- `Date.prototype.getYear()`

Date.prototype.getHours()

The **`getHours()`** method of `Date` instances returns the hours for this date according to local time.

Try it

```
const birthday = new Date("March 13, 08 04:20");

console.log(birthday.getHours());
// Expected output: 4
```

Syntax

```
getHours()
```

Parameters

None.

Return value

An integer, between 0 and 23, representing the hours for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getHours()

The `hours` variable has value `23`, based on the value of the `Date` object `xmas95`.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const hours = xmas95.getHours();

console.log(hours); // 23
```

See also

- `Date.prototype.getUTCHours()`
- `Date.prototype.setHours()`

Date.prototype.getMilliseconds()

The **`getMilliseconds()`** method of `Date` instances returns the milliseconds for this date according to local time.

Try it

```
const moonLanding = new Date("July 20, 69 00:20:18");
moonLanding.setMilliseconds(123);

console.log(moonLanding.getMilliseconds());
// Expected output: 123
```

Syntax

```
getMilliseconds()
```

Parameters

None.

Return value

An integer, between 0 and 999, representing the milliseconds for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getMilliseconds()

The `milliseconds` variable has value `0`, based on the value of the `Date` object `xmas95`, which doesn't specify the milliseconds component, so it defaults to 0.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const milliseconds = xmas95.getMilliseconds();

console.log(milliseconds); // 0
```

See also

- `Date.prototype.getUTCMilliseconds()`
- `Date.prototype.setMilliseconds()`

Date.prototype.getMinutes()

The **`getMinutes()`** method of `Date` instances returns the minutes for this date according to local time.

Try it

```
const birthday = new Date("March 13, 08 04:20");

console.log(birthday.getMinutes());
// Expected output: 20
```

Syntax

```
getMinutes()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the minutes for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getMinutes()

The `minutes` variable has value `15`, based on the value of the `Date` object `xmas95`.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const minutes = xmas95.getMinutes();

console.log(minutes); // 15
```

See also

- `Date.prototype.getUTCMinutes()`
- `Date.prototype.setMinutes()`

Date.prototype.getMonth()

The **`getMonth()`** method of `Date` instances returns the month for this date according to local time, as a zero-based value (where zero indicates the first month of the year).

Try it

```
const moonLanding = new Date("July 20, 69 00:20:18");

console.log(moonLanding.getMonth()); // (January gives 0)
// Expected output: 6
```

Syntax

```
getMonth()
```

Parameters

None.

Return value

An integer, between 0 and 11, representing the month for the given date according to local time: 0 for January, 1 for February, and so on. Returns `NaN` if the date is invalid.

Description

The return value of `getMonth()` is zero-based, which is useful for indexing into arrays of months, for example:

```
const valentines = new Date("1995-02-14");
const month = valentines.getMonth();
const monthNames = ["January", "February", "March" /* , … */];

console.log(monthNames[month]); // "February"
```

However, for the purpose of internationalization, you should prefer using `Intl.DateTimeFormat` with the `options` parameter instead.

```
const options = { month: "long" };
console.log(new Intl.DateTimeFormat("en-US", options).format(valentines));
// "February"
console.log(new Intl.DateTimeFormat("de-DE", options).format(valentines));
// "Februar"
```

Examples

Using getMonth()

The `month` variable has value `11`, based on the value of the `Date` object `xmas95`.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const month = xmas95.getMonth();

console.log(month); // 11
```

See also

- `Date.prototype.getUTCMonth()`
- `Date.prototype.setMonth()`

Date.prototype.getSeconds()

The **`getSeconds()`** method of `Date` instances returns the seconds for this date according to local time.

Try it

```
const moonLanding = new Date("July 20, 69 00:20:18");

console.log(moonLanding.getSeconds());
// Expected output: 18
```

Syntax

```
getSeconds()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the seconds for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getSeconds()

The `seconds` variable has value `30`, based on the value of the `Date` object `xmas95`.

```
const xmas95 = new Date("1995-12-25T23:15:30");
const seconds = xmas95.getSeconds();

console.log(seconds); // 30
```

See also

- `Date.prototype.getUTCSeconds()`
- `Date.prototype.setSeconds()`

Date.prototype.getTime()

The **`getTime()`** method of `Date` instances returns the number of milliseconds for this date since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

```
const moonLanding = new Date("July 20, 69 20:17:40 GMT+00:00");

// Milliseconds since Jan 1, 1970, 00:00:00.000 GMT
console.log(moonLanding.getTime());
// Expected output: -14182940000
```

Syntax

```
getTime()
```

Parameters

None.

Return value

A number representing the timestamp, in milliseconds, of this date. Returns `NaN` if the date is invalid.

Description

`Date` objects are fundamentally represented by a timestamp, and this method allows you to retrieve the timestamp. You can use this method to help assign a date and time to another `Date` object. This method is functionally equivalent to the `valueOf()` method.

Examples

Using getTime() for copying dates

Constructing a date object with the identical time value.

```
// Since month is zero based, birthday will be January 10, 1995
const birthday = new Date(1994, 12, 10);
const copy = new Date();
copy.setTime(birthday.getTime());
```

Measuring execution time

Subtracting two subsequent `getTime()` calls on newly generated `Date` objects, give the time span between these two calls. This can be used to calculate the executing time of some operations. See also `Date.now()` to prevent instantiating unnecessary `Date` objects.

```
let end, start;

start = new Date();
for (let i = 0; i < 1000; i++) {
  Math.sqrt(i);
}
end = new Date();

console.log(`Operation took ${end.getTime() - start.getTime()} msec`);
```

> **Note:** In browsers that support the Performance API's high-resolution time feature, `Performance.now()` can provide more reliable and precise measurements of elapsed time than `Date.now()`.

See also

- `Date.prototype.setTime()`
- `Date.prototype.valueOf()`
- `Date.now()`

Date.prototype.getTimezoneOffset()

The **`getTimezoneOffset()`** method of `Date` instances returns the difference, in minutes, between this date as evaluated in the UTC time zone, and the same date as evaluated in the local time zone.

Try it

```
const date1 = new Date("August 19, 1975 23:15:30 GMT+07:00");
const date2 = new Date("August 19, 1975 23:15:30 GMT-02:00");

console.log(date1.getTimezoneOffset());
// Expected output: your local timezone offset in minutes
// (e.g., -120). NOT the timezone offset of the date object.

console.log(date1.getTimezoneOffset() === date2.getTimezoneOffset());
// Expected output: true
```

Syntax

```
getTimezoneOffset()
```

Parameters

None.

Return value

A number representing the difference, in minutes, between the date as evaluated in the UTC time zone and as evaluated in the local time zone. The actual local time algorithm is implementation-defined, and the return value is allowed to be zero in runtimes without appropriate data. Returns `NaN` if the date is invalid.

Description

`date.getTimezoneOffset()` returns the difference, in minutes, between `date` as evaluated in the UTC time zone and as evaluated in the local time zone — that is, the time zone of the host system in which the browser is being used (if the code is run from the Web in a browser), or otherwise the host system of whatever JavaScript runtime (for example, a Node.js environment) the code is executed in.

Negative values and positive values

The number of minutes returned by `getTimezoneOffset()` is positive if the local time zone is behind UTC, and negative if the local time zone is ahead of UTC. For example, for UTC+10, `-600` will be returned.

Current time zone	Return value
UTC-8	480
UTC	0
UTC+3	-180

Varied results in Daylight Saving Time (DST) regions

In a region that annually shifts in and out of Daylight Saving Time (DST), as `date` varies, the number of minutes returned by calling `getTimezoneOffset()` can be non-uniform.

> **Note:** `getTimezoneOffset()`'s behavior will never differ based on the time when the code is run — its behavior is always consistent when running in the same region. Only the value of `date` affects the result.

> **Note:** Many countries have experimented with not changing the time twice a year and this has meant that DST has continued over the winter too. For example in the UK DST lasted from 2:00AM 18 February 1968 to 3:00AM 31 October 1971, so during the winter the clocks were not set back.

In most implementations, the IANA time zone database (tzdata) is used to precisely determine the offset of the local timezone at the moment of the `date`. However, if such information is unavailable, an implementation may return zero.

Examples

Using getTimezoneOffset()

```
// Create a Date instance for the current time
const currentLocalDate = new Date();
// Create a Date instance for 03:24 GMT-0200 on May 1st in 2016
const laborDay2016at0324GMTminus2 = new Date("2016-05-01T03:24:00-02:00");
currentLocalDate.getTimezoneOffset() ===
  laborDay2016at0324GMTminus2.getTimezoneOffset();
// true, always, in any timezone that doesn't annually shift in and out of DST
// false, sometimes, in any timezone that annually shifts in and out of DST
```

getTimezoneOffset() and DST

In regions that use DST, the return value may change based on the time of the year `date` is in. Below is the output in a runtime in New York, where the timezone is UTC-05:00.

```
const nyOffsetSummer = new Date("2022-02-01").getTimezoneOffset(); // 300
const nyOffsetWinter = new Date("2022-08-01").getTimezoneOffset(); // 240
```

getTimezoneOffset() and historical data

Due to historical reasons, the timezone a region is in can be constantly changing, even disregarding DST. For example, below is the output in a runtime in Shanghai, where the timezone is UTC+08:00.

```
const shModernOffset = new Date("2022-01-27").getTimezoneOffset(); // -480
const shHistoricalOffset = new Date("1943-01-27").getTimezoneOffset(); // -540
```

This is because during the Sino-Japanese War when Shanghai was under Japanese control, the timezone was changed to UTC+09:00 to align with Japan's (in effect, it was a "year-round DST"), and this was recorded in the IANA database.

See also

- `Date`

Date.prototype.getUTCDate()

The **`getUTCDate()`** method of `Date` instances returns the day of the month for this date according to universal time.

Try it

```
const date1 = new Date("August 19, 1975 23:15:30 GMT+11:00");
const date2 = new Date("August 19, 1975 23:15:30 GMT-11:00");

console.log(date1.getUTCDate());
// Expected output: 19

console.log(date2.getUTCDate());
// Expected output: 20
```

Syntax

```
getUTCDate()
```

Parameters

None.

Return value

An integer, between 1 and 31, representing day of month for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCDate()

The following example assigns the day of month of the current date to the variable `dayOfMonth`.

```
const today = new Date();
const dayOfMonth = today.getUTCDate();
```

See also

- `Date.prototype.getUTCDay()`
- `Date.prototype.getDay()`
- `Date.prototype.setUTCDate()`

Date.prototype.getUTCDay()

The **`getUTCDay()`** method of `Date` instances returns the day of the week for this date according to universal time, where 0 represents Sunday.

Try it

```
const date1 = new Date("August 19, 1975 23:15:30 GMT+11:00");
const date2 = new Date("August 19, 1975 23:15:30 GMT-11:00");

// Tuesday
console.log(date1.getUTCDay());
// Expected output: 2

// Wednesday
console.log(date2.getUTCDay());
// Expected output: 3
```

Syntax

```
getUTCDay()
```

Parameters

None.

Return value

An integer corresponding to the day of the week for the given date according to universal time: 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on. Returns `NaN` if the date is invalid.

Examples

Using getUTCDay()

The following example assigns the weekday portion of the current date to the variable `weekday`.

```
const today = new Date();
const weekday = today.getUTCDay();
```

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.getDay()`
- `Date.prototype.setUTCDate()`

Date.prototype.getUTCFullYear()

The **`getUTCFullYear()`** method of `Date` instances returns the year for this date according to universal time.

Try it

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

console.log(date1.getUTCFullYear());
// Expected output: 1975

console.log(date2.getUTCFullYear());
// Expected output: 1976
```

Syntax

```
getUTCFullYear()
```

Parameters

None.

Return value

An integer representing the year for the given date according to universal time. Returns `NaN` if the date is invalid.

Description

Unlike `getYear()`, the value returned by `getUTCFullYear()` is an absolute number. For dates between the years 1000 and 9999, `getFullYear()` returns a four-digit number, for example, 1995. Use this function to make sure a year is compliant with years after 2000.

Examples

Using getUTCFullYear()

The following example assigns the four-digit value of the current year to the variable `year`.

```
const today = new Date();
const year = today.getUTCFullYear();
```

See also

- `Date.prototype.getFullYear()`
- `Date.prototype.setFullYear()`

Date.prototype.getUTCHours()

The **`getUTCHours()`** method of `Date` instances returns the hours for this date according to universal time.

Try it

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

console.log(date1.getUTCHours());
// Expected output: 12
```

```
console.log(date2.getUTCHours());
// Expected output: 10
```

Syntax

```
getUTCHours()
```

Parameters

None.

Return value

An integer, between 0 and 23, representing the hours for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCHours()

The following example assigns the hours portion of the current time to the variable `hours`.

```
const today = new Date();
const hours = today.getUTCHours();
```

See also

- `Date.prototype.getHours()`
- `Date.prototype.setUTCHours()`

Date.prototype.getUTCMilliseconds()

The **`getUTCMilliseconds()`** method of `Date` instances returns the milliseconds for this date according to universal time.

Try it

```
const exampleDate = new Date("2018-01-02T03:04:05.678Z"); // 2 January 2018, 03:04:05.678 (UTC)

console.log(exampleDate.getUTCMilliseconds());
// Expected output: 678
```

Syntax

```
getUTCMilliseconds()
```

Parameters

None.

Return value

An integer, between 0 and 999, representing the milliseconds for the given date according to universal time. Returns `NaN` if the date is invalid.

Not to be confused with the timestamp. To get the total milliseconds since the epoch, use the `getTime()` method.

Examples

Using getUTCMilliseconds()

The following example assigns the milliseconds portion of the current time to the variable `milliseconds`.

```
const today = new Date();
const milliseconds = today.getUTCMilliseconds();
```

See also

- `Date.prototype.getMilliseconds()`
- `Date.prototype.setUTCMilliseconds()`

Date.prototype.getUTCMinutes()

The **`getUTCMinutes()`** method of `Date` instances returns the minutes for this date according to universal time.

Try it

```
const date1 = new Date("1 January 2000 03:15:30 GMT+07:00");
const date2 = new Date("1 January 2000 03:15:30 GMT+03:30");

console.log(date1.getUTCMinutes()); // 31 Dec 1999 20:15:30 GMT
// Expected output: 15

console.log(date2.getUTCMinutes()); // 31 Dec 1999 23:45:30 GMT
// Expected output: 45
```

Syntax

```
getUTCMinutes()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the minutes for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCMinutes()

The following example assigns the minutes portion of the current time to the variable `minutes`.

```
const today = new Date();
const minutes = today.getUTCMinutes();
```

See also

- `Date.prototype.getMinutes()`
- `Date.prototype.setUTCMinutes()`

Date.prototype.getUTCMonth()

The **`getUTCMonth()`** method of `Date` instances returns the month for this date according to universal time, as a zero-based value (where zero indicates the first month of the year).

Try it

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

// December
console.log(date1.getUTCMonth());
// Expected output: 11

// January
console.log(date2.getUTCMonth());
// Expected output: 0
```

Syntax

```
getUTCMonth()
```

Parameters

None.

Return value

An integer, between 0 and 11, representing the month for the given date according to universal time: 0 for January, 1 for February, and so on. Returns `NaN` if the date is invalid.

Examples

Using getUTCMonth()

The following example assigns the month portion of the current date to the variable `month`.

```
const today = new Date();
const month = today.getUTCMonth();
```

See also

- `Date.prototype.getMonth()`
- `Date.prototype.setUTCMonth()`

Date.prototype.getUTCSeconds()

The **`getUTCSeconds()`** method of `Date` instances returns the seconds in the specified date according to universal time.

Try it

```
const moonLanding = new Date("July 20, 1969, 20:18:04 UTC");

console.log(moonLanding.getUTCSeconds());
// Expected output: 4
```

Syntax

```
getUTCSeconds()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the seconds for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCSeconds()

The following example assigns the seconds portion of the current time to the variable `seconds`.

```
const today = new Date();
const seconds = today.getUTCSeconds();
```

See also

- `Date.prototype.getSeconds()`
- `Date.prototype.setUTCSeconds()`

Date.prototype.getYear()

The **`getYear()`** method of `Date` instances returns the year for this date according to local time. Because `getYear()` does not return full years ("year 2000 problem"), it is deprecated and has been replaced by the `getFullYear()` method.

Syntax

```
getYear()
```

Parameters

None.

Return value

An integer representing the year for the given date according to local time, minus 1900. Returns `NaN` if the date is invalid.

- For years greater than or equal to 2000, the value is 100 or greater. For example, if the year is 2026, `getYear()` returns 126.
- For years between and including 1900 and 1999, the value returned by `getYear()` is between 0 and 99. For example, if the year is 1976, `getYear()` returns 76.
- For years less than 1900, the value returned by `getYear()` is less than 0. For example, if the year is 1800, `getYear()` returns -100.

This method essentially returns the value of `getFullYear()` minus 1900. You should use `getFullYear()` instead, so that the year is specified in full.

Examples

Years between 1900 and 1999

The second statement assigns the value 95 to the variable `year`.

```
const xmas = new Date("1995-12-25");
const year = xmas.getYear(); // returns 95
```

Years above 1999

The second statement assigns the value 100 to the variable `year`.

```
const xmas = new Date("2000-12-25");
const year = xmas.getYear(); // returns 100
```

Years below 1900

The second statement assigns the value -100 to the variable `year`.

```
const xmas = new Date("1800-12-25");
const year = xmas.getYear(); // returns -100
```

Setting and getting a year between 1900 and 1999

The third statement assigns the value 95 to the variable `year`, representing the year 1995.

```
const xmas = new Date("2015-12-25");
xmas.setYear(95);
const year = xmas.getYear(); // returns 95
```

See also

- Polyfill of `Date.prototype.getYear` in `core-js`
- es-shims polyfill of `Date.prototype.getYear`
- `Date.prototype.getFullYear()`
- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setYear()`

Date.now()

The **`Date.now()`** static method returns the number of milliseconds elapsed since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

```
// This example takes 2 seconds to run
const start = Date.now();

console.log("starting timer...");
// Expected output: "starting timer..."

setTimeout(() => {
  const ms = Date.now() - start;

  console.log(`seconds elapsed = ${Math.floor(ms / 1000)}`);
  // Expected output: "seconds elapsed = 2"
}, 2000);
```

Syntax

```
Date.now()
```

Parameters

None.

Return value

A number representing the timestamp, in milliseconds, of the current time.

Description

Reduced time precision

To offer protection against timing attacks and fingerprinting, the precision of `Date.now()` may be reduced depending on browser settings.

The timestamp is always an integer number of milliseconds, so its resolution is limited to 1 ms in all contexts. This already satisfies certain basic security and privacy needs.

In Firefox, the `privacy.reduceTimerPrecision` preference is enabled by default. With default settings, the timestamp has 1 ms resolution. If `privacy.resistFingerprinting` is enabled, the rounding interval is 16.667 ms or the interval configured by `privacy.resistFingerprinting.reduceTimerPrecision.microseconds`, whichever is larger. The final output is rounded to an integer.

For example, these are possible values in Firefox:

```
// Reduced time precision (1 ms) with default settings
Date.now();
// Might be:
// 1519211809934
// 1519211810363
// 1519211811671
// …

// Reduced time precision with `privacy.resistFingerprinting` enabled
Date.now();
// Might be:
// 1519129853489
// 1519129853506
// 1519129853522
// …
```

Examples

Measuring time elapsed

You can use `Date.now()` to get the current time in milliseconds, then subtract a previous time to find out how much time elapsed between the two calls.

```
const start = Date.now();
doSomeLongRunningProcess();
console.log(`Time elapsed: ${Date.now() - start} ms`);
```

For more complex scenarios, you may want to use the performance API instead.

See also

- Polyfill of `Date.now` in `core-js`

- `Performance.now()`
- `console.time()`
- `console.timeEnd()`

Date.parse()

The **`Date.parse()`** static method parses a string representation of a date, and returns the date's timestamp.

Try it

```
// Standard date-time string format
const unixTimeZero = Date.parse("1970-01-01T00:00:00Z");
// Non-standard format resembling toUTCString()
const javaScriptRelease = Date.parse("04 Dec 1995 00:12:00 GMT");

console.log(unixTimeZero);
// Expected output: 0

console.log(javaScriptRelease);
// Expected output: 818035920000
```

Syntax

```
Date.parse(dateString)
```

Parameters

`dateString`

A string in the date time string format. See the linked reference for caveats on using different formats.

Return value

A number representing the timestamp of the given date. If `dateString` fails to be parsed as a valid date, `NaN` is returned.

Description

This function is useful for setting date values based on string values, for example in conjunction with the `setTime()` method.

The formats that `parse()` can handle are not explicitly specified, but there are a few invariants:

- The date time string format (produced by `toISOString()`) must be supported.
- If `x` is any Date whose milliseconds amount is zero, then `x.valueOf()` should be equal to any of the following: `Date.parse(x.toString())`, `Date.parse(x.toUTCString())`, `Date.parse(x.toISOString())`. This means the formats produced by `toString()` and `toUTCString()` should be supported too.
- The spec does *not* require support for the format produced by `toLocaleString()`. However, major engines all try to support `toLocaleString("en-US")` format.

Other formats are implementation-defined and may not work across all browsers. A library can help if many different formats are to be accommodated. In fact, the unreliability of `Date.parse()` is one of the motivations for the `Temporal` API to be introduced.

Because `parse()` is a static method of `Date`, you always use it as `Date.parse()`, rather than as a method of a `Date` object you created.

Examples

Using Date.parse()

The following calls all return `1546300800000`. The first will imply UTC time because it's date-only, and the others explicitly specify the UTC timezone.

```
Date.parse("2019-01-01");
Date.parse("2019-01-01T00:00:00.000Z");
Date.parse("2019-01-01T00:00:00.000+00:00");
```

The following call, which does not specify a time zone will be set to 2019-01-01 at 00:00:00 in the local timezone of the system, because it has both date and time.

```
Date.parse("2019-01-01T00:00:00");
```

toString() and toUTCString() formats

Apart from the standard date time string format, the `toString()` and `toUTCString()` formats are supported:

```
// toString() format
Date.parse("Thu Jan 01 1970 00:00:00 GMT-0500 (Eastern Standard Time)");
// 18000000 in all implementations in all timezones

// toUTCString() format
Date.parse("Thu, 01 Jan 1970 00:00:00 GMT");
// 0 in all implementations in all timezones
```

Non-standard date strings

> **Note:** This section contains implementation-specific behavior that may be inconsistent across browsers or different versions of browsers. It is not meant to be a

> comprehensive browser compatibility table and you should always conduct your own tests before using any format in your code.

Implementations usually default to the local time zone when the date string is non-standard. For consistency, we will assume that the runtime uses the UTC timezone, and unless specified otherwise, the output will vary with the device's time zone. Daylight Saving Time (DST), of the local time zone, can also have an effect on this.

Here are some more examples of non-standard date strings. Browsers are very lenient when parsing date strings and may discard any part of a string that they cannot parse. For compatibility reasons, browsers often copy each other's behavior, so these handling patterns tend to propagate cross-browser. As previously stated, the following examples are for illustration only, and are not exhaustive by any means:

Description	Example	Chrome	Firefox	Safari
Single number	`0` (single-digit)	946684800000 (Jan 01 2000); NaN in Firefox ≤122		-62167219200000 (Jan 01 0000)
	`31` (two-digit)	NaN		-61188912000000 (Jan 01 0031)
	`999` (three-/four-digit)	-30641733102000 (Jan 01 0999)		
Date strings that use different separators	`1970-01-01` (standard)	0 in all time zones		
	`1970/01/01`	0		
	`1970,01,01`	0		NaN
	`1970 01 01`	0		NaN

Strings that look like `toString()`	`Thu Jan 01 1970 00:00:00`
	`Thu Jan 01 1970`
	`Jan 01 1970 00:00:00`
	`Jan 01 1970`
		0		

Strings that look like `toUTCString()`	`Thu, 01 Jan 1970 00:00:00`
	`Thu, 01 Jan 1970`
	`01 Jan 1970 00:00:00`
	`01 Jan 1970`
		0		

First date component is 2-digit	`01-02-03` (first segment can be valid month)	1041465600000 (Jan 02 2003)		-62132745600000 (Feb 03 0001) Note: Safari always assumes YY-MM-DD, but MM/DD/YY.
	`27-02-03` (first segment can be valid day but not month)	NaN		-61312291200000 (Feb 03 0027)
	`49-02-03` (first segment cannot be valid day and is <50)	2495923200000 (Feb 03 2049)		-60617980800000 (Feb 03 0049)
	`50-02-03` (first segment cannot be valid day and is ≥50)	-628300800000 (Feb 03 1950)		-60586444800000 (Feb 03 0050)

Out-of-bounds date components	`2014-25-23`
	`Mar 32, 2014`
	`2014/25/23`
		NaN		

	`2014-02-30`	1393718400000 (Mar 02 2014)		NaN
	`02/30/2014`	1393718400000		

Extraneous characters after the month name	`04 Dec 1995`
	`04 Decem 1995`
	`04 December 1995`
		818031600000		

	`04 DecFoo 1995`	818031600000 Only the first three characters are read. Firefox ≤121 reads up to the valid month name, thus returning NaN when it sees "F".		
	`04 De 1995`	NaN		

See also

- `Date.UTC()`

Date.prototype.setDate()

The **`setDate()`** method of `Date` instances changes the day of the month for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

event.setDate(24);

console.log(event.getDate());
// Expected output: 24

event.setDate(32);
// Only 31 days in August!
```

```
console.log(event.getDate());
// Expected output: 1
```

Syntax

```
setDate(dateValue)
```

Parameters

`dateValue`

An integer representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `dateValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you specify a number outside the expected range, the date information in the `Date` object is updated accordingly. For example, if the `Date` object holds June 1st, a `dateValue` of 40 changes the date to July 10th, while a `dateValue` of 0 changes the date to the last day of the previous month, May 31st.

Because `setDate()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the date crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal day difference multiplied by 24 hours. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCDate()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setDate()

```
const theBigDay = new Date(1962, 6, 7, 12); // noon of 1962-07-07 (7th of July 1962, month is 0-indexed)
const theBigDay2 = new Date(theBigDay).setDate(24); // 1962-07-24 (24th of July 1962)
const theBigDay3 = new Date(theBigDay).setDate(32); // 1962-08-01 (1st of August 1962)
const theBigDay4 = new Date(theBigDay).setDate(22); // 1962-07-22 (22nd of July 1962)
const theBigDay5 = new Date(theBigDay).setDate(0); // 1962-06-30 (30th of June 1962)
const theBigDay6 = new Date(theBigDay).setDate(98); // 1962-10-06 (6th of October 1962)
const theBigDay7 = new Date(theBigDay).setDate(-50); // 1962-05-11 (11th of May 1962)
```

See also

- `Date()`
- `Date.prototype.getDate()`
- `Date.prototype.setUTCDate()`

Date.prototype.setFullYear()

The **`setFullYear()`** method of `Date` instances changes the year, month, and/or day of month for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

event.setFullYear(1969);

console.log(event.getFullYear());
// Expected output: 1969

event.setFullYear(0);

console.log(event.getFullYear());
// Expected output: 0
```

Syntax

```
setFullYear(yearValue)
setFullYear(yearValue, monthValue)
setFullYear(yearValue, monthValue, dateValue)
```

Parameters

`yearValue`

An integer representing the year. For example, 1995.

`monthValue` Optional

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer between 1 and 31 representing the day of the month. If you specify `dateValue`, you must also specify `monthValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `monthValue` and `dateValue` parameters, the same values as what are returned by `getMonth()` and `getDate()` are used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 15 for `monthValue`, the year is incremented by 1 (`yearValue + 1`), and 3 is used for the month.

Because `setFullYear()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the date crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal day difference multiplied by 24 hours. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCFullYear()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setFullYear()

```
const theBigDay = new Date();
theBigDay.setFullYear(1997);
```

See also

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setUTCFullYear()`
- `Date.prototype.setYear()`

Date.prototype.setHours()

The **`setHours()`** method of `Date` instances changes the hours, minutes, seconds, and/or milliseconds for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");
event.setHours(20);

console.log(event);
// Expected output: "Tue Aug 19 1975 20:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary

event.setHours(20, 21, 22);

console.log(event);
// Expected output: "Tue Aug 19 1975 20:21:22 GMT+0200 (CEST)"
```

Syntax

```
setHours(hoursValue)
setHours(hoursValue, minutesValue)
setHours(hoursValue, minutesValue, secondsValue)
setHours(hoursValue, minutesValue, secondsValue, msValue)
```

Parameters

`hoursValue`

An integer between 0 and 23 representing the hours.

`minutesValue` Optional

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `minutesValue`, `secondsValue`, and `msValue` parameters, the same values as what are returned by `getMinutes()`, `getSeconds()`, and `getMilliseconds()` are used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 100 for `secondsValue`, the minutes are incremented by 1 (`minutesValue + 1`), and 40 is used for seconds.

Because `setHours()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the hours crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal hour difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCHours()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setHours()

```
const theBigDay = new Date();
theBigDay.setHours(7);
```

See also

- `Date.prototype.getHours()`
- `Date.prototype.setUTCHours()`

Date.prototype.setMilliseconds()

The **`setMilliseconds()`** method of `Date` instances changes the milliseconds for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.getMilliseconds());
// Expected output: 0

event.setMilliseconds(456);

console.log(event.getMilliseconds());
// Expected output: 456
```

Syntax

```
setMilliseconds(millisecondsValue)
```

Parameters

`millisecondsValue`

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `millisecondsValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you specify a number outside the expected range, the date information in the `Date` object is updated accordingly. For example, if you specify 1005, the number of seconds is incremented by 1, and 5 is used for the milliseconds.

Because `setMilliseconds()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the milliseconds crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal time difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCMilliseconds()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setMilliseconds()

```
const theBigDay = new Date();
theBigDay.setMilliseconds(100);
```

See also

- `Date.prototype.getMilliseconds()`
- `Date.prototype.setUTCMilliseconds()`

Date.prototype.setMinutes()

The **`setMinutes()`** method of `Date` instances changes the minutes for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

event.setMinutes(45);

console.log(event.getMinutes());
// Expected output: 45

console.log(event);
// Expected output: "Tue Aug 19 1975 23:45:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

```
setMinutes(minutesValue)
setMinutes(minutesValue, secondsValue)
setMinutes(minutesValue, secondsValue, msValue)
```

Parameters

`minutesValue`

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `secondsValue` and `msValue` parameters, the same values as what are returned by `getSeconds()` and `getMilliseconds()` are used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 100 for `secondsValue`, the minutes is incremented by 1 (`minutesValue + 1`), and 40 is used for seconds.

Because `setMinutes()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the minutes crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal time difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCMinutes()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setMinutes()

```
const theBigDay = new Date();
theBigDay.setMinutes(45);
```

See also

- `Date.prototype.getMinutes()`
- `Date.prototype.setUTCMinutes()`

Date.prototype.setMonth()

The **`setMonth()`** method of `Date` instances changes the month and/or day of the month for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

event.setMonth(3);
```

```
console.log(event.getMonth());
// Expected output: 3

console.log(event);
// Expected output: "Sat Apr 19 1975 23:15:30 GMT+0100 (CET)"
// Note: your timezone may vary
```

Syntax

```
setMonth(monthValue)
setMonth(monthValue, dateValue)
```

Parameters

`monthValue`

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer from 1 to 31 representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `dateValue` parameter, the same value as what is returned by `getDate()` is used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 15 for `monthValue`, the year is incremented by 1, and 3 is used for month.

The current day of month will have an impact on the behavior of this method. Conceptually it will add the number of days given by the current day of the month to the 1st day of the new month specified as the parameter, to return the new date. For example, if the current value is 31st January 2016, calling setMonth with a value of 1 will return 2nd March 2016. This is because in 2016 February had 29 days.

Because `setMonth()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the month crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal day difference multiplied by 24 hours. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCMonth()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setMonth()

```
const theBigDay = new Date();
theBigDay.setMonth(6);

// Watch out for end of month transitions
const endOfMonth = new Date(2016, 7, 31);
endOfMonth.setMonth(1);
console.log(endOfMonth); // Wed Mar 02 2016 00:00:00
```

See also

- `Date.prototype.getMonth()`
- `Date.prototype.setUTCMonth()`

Date.prototype.setSeconds()

The **`setSeconds()`** method of `Date` instances changes the seconds and/or milliseconds for this date according to local time.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

event.setSeconds(42);

console.log(event.getSeconds());
// Expected output: 42

console.log(event);
// Expected output: "Tue Aug 19 1975 23:15:42 GMT+0100 (CET)"
// Note: your timezone may vary
```

Syntax

```
setSeconds(secondsValue)
setSeconds(secondsValue, msValue)
```

Parameters

`secondsValue`

An integer between 0 and 59 representing the seconds.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `msValue` parameter, the value returned from the `getMilliseconds()` method is used.

If a parameter you specify is outside of the expected range, `setSeconds()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes stored in the `Date` object will be incremented by 1, and 40 will be used for seconds.

Because `setSeconds()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the seconds crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal time difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCSeconds()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setSeconds()

```
const theBigDay = new Date();
theBigDay.setSeconds(30);
```

See also

- `Date.prototype.getSeconds()`
- `Date.prototype.setUTCSeconds()`

Date.prototype.setTime()

The **`setTime()`** method of `Date` instances changes the timestamp for this date, which is the number of milliseconds since the epoch, defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

```
const launchDate = new Date("July 1, 1999, 12:00:00");
const futureDate = new Date();
futureDate.setTime(launchDate.getTime());

console.log(futureDate);
// Expected output: "Thu Jul 01 1999 12:00:00 GMT+0200 (CEST)"

const fiveMinutesInMs = 5 * 60 * 1000;
futureDate.setTime(futureDate.getTime() + fiveMinutesInMs);

console.log(futureDate);
// Expected output: "Thu Jul 01 1999 12:05:00 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

```
setTime(timeValue)
```

Parameters

`timeValue`

An integer representing the new timestamp — the number of milliseconds since the midnight at the beginning of January 1, 1970, UTC.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `timeValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Examples

Using setTime()

```
const theBigDay = new Date("1999-07-01");
const sameAsBigDay = new Date();
sameAsBigDay.setTime(theBigDay.getTime());
```

See also

- `Date.prototype.getTime()`
- `Date.prototype.setUTCHours()`

Date.prototype.setUTCDate()

The **`setUTCDate()`** method of `Date` instances changes the day of the month for this date according to universal time.

Try it

```
const event = new Date("August 19, 1975 23:15:30 GMT-3:00");

console.log(event.getUTCDate());
// Expected output: 20

event.setUTCDate(19);

console.log(event.getUTCDate());
// Expected output: 19
```

Syntax

```
setUTCDate(dateValue)
```

Parameters

`dateValue`

An integer from 1 to 31 representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `dateValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If the `dateValue` is outside of the range of date values for the month, `setDate()` will update the `Date` object accordingly.

For example, if 0 is provided for `dateValue`, the date will be set to the last day of the previous month. If you use 40 for `dateValue`, and the month stored in the `Date` object is June, the day will be changed to 10 and the month will be incremented to July.

If a negative number is provided for `dateValue`, the date will be set counting backwards from the last day of the previous month. -1 would result in the date being set to 1 day before the last day of the previous month.

Examples

Using setUTCDate()

```
const theBigDay = new Date();
theBigDay.setUTCDate(20);
```

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.setDate()`

Date.prototype.setUTCFullYear()

The **`setUTCFullYear()`** method of `Date` instances changes the year for this date according to universal time.

Try it

```
const event = new Date("December 31, 1975 23:15:30 GMT-3:00");

console.log(event.getUTCFullYear());
// Expected output: 1976

console.log(event.toUTCString());
// Expected output: "Thu, 01 Jan 1976 02:15:30 GMT"

event.setUTCFullYear(1975);

console.log(event.toUTCString());
// Expected output: "Wed, 01 Jan 1975 02:15:30 GMT"
```

Syntax

```
setUTCFullYear(yearValue)
setUTCFullYear(yearValue, monthValue)
setUTCFullYear(yearValue, monthValue, dateValue)
```

Parameters

`yearValue`

An integer representing the year. For example, 1995.

`monthValue` Optional

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer between 1 and 31 representing the day of the month. If you specify `dateValue`, you must also specify `monthValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `monthValue` and `dateValue` parameters, the values returned from the `getUTCMonth()` and `getUTCDate()` methods are used.

If a parameter you specify is outside of the expected range, `setUTCFullYear()` attempts to update the other parameters and the date information in the `Date` object accordingly. For example, if you specify 15 for `monthValue`, the year is incremented by 1 (`yearValue + 1`), and 3 is used for the month.

Examples

Using setUTCFullYear()

```
const theBigDay = new Date();
theBigDay.setUTCFullYear(1997);
```

See also

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`

Date.prototype.setUTCHours()

The **`setUTCHours()`** method of `Date` instances changes the hours, minutes, seconds, and/or milliseconds for this date according to universal time.

Try it

```
const event = new Date("August 19, 1975 23:15:30 GMT-3:00");

console.log(event.toUTCString());
// Expected output: "Wed, 20 Aug 1975 02:15:30 GMT"

console.log(event.getUTCHours());
// Expected output: 2

event.setUTCHours(23);

console.log(event.toUTCString());
// Expected output: "Wed, 20 Aug 1975 23:15:30 GMT"
```

Syntax

```
setUTCHours(hoursValue)
setUTCHours(hoursValue, minutesValue)
setUTCHours(hoursValue, minutesValue, secondsValue)
setUTCHours(hoursValue, minutesValue, secondsValue, msValue)
```

Parameters

`hoursValue`

An integer between 0 and 23 representing the hours.

`minutesValue` Optional

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `minutesValue`, `secondsValue`, and `msValue` parameters, the values returned from the `getUTCMinutes()`, `getUTCSeconds()`, and `getUTCMilliseconds()` methods are used.

If a parameter you specify is outside of the expected range, `setUTCHours()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes will be incremented by 1 (`minutesValue + 1`), and 40 will be used for seconds.

Examples

Using setUTCHours()

```
const theBigDay = new Date();
theBigDay.setUTCHours(8);
```

See also

- `Date.prototype.getUTCHours()`
- `Date.prototype.setHours()`

Date.prototype.setUTCMilliseconds()

The **`setUTCMilliseconds()`** method of `Date` instances changes the milliseconds for this date according to universal time.

Try it

```
const date = new Date("2018-01-24T12:38:29.069Z");

console.log(date.getUTCMilliseconds());
// Expected output: 69

date.setUTCMilliseconds(420);

console.log(date.getUTCMilliseconds());
// Expected output: 420
```

Syntax

```
setUTCMilliseconds(millisecondsValue)
```

Parameters

`millisecondsValue`

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `millisecondsValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If a parameter you specify is outside of the expected range, `setUTCMilliseconds()` attempts to update the date information in the `Date` object accordingly. For example, if you use 1100 for `millisecondsValue`, the seconds stored in the `Date` object will be incremented by 1, and 100 will be used for milliseconds.

Examples

Using setUTCMilliseconds()

```
const theBigDay = new Date();
theBigDay.setUTCMilliseconds(500);
```

See also

- `Date.prototype.getUTCMilliseconds()`
- `Date.prototype.setMilliseconds()`

Date.prototype.setUTCMinutes()

The **`setUTCMinutes()`** method of `Date` instances changes the minutes for this date according to universal time.

Try it

```
const date = new Date("December 31, 1975, 23:15:30 GMT+11:00");

console.log(date.getUTCMinutes());
// Expected output: 15

date.setUTCMinutes(25);

console.log(date.getUTCMinutes());
// Expected output: 25
```

Syntax

```
setUTCMinutes(minutesValue)
setUTCMinutes(minutesValue, secondsValue)
setUTCMinutes(minutesValue, secondsValue, msValue)
```

Parameters

`minutesValue`

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `secondsValue` and `msValue` parameters, the values returned from `getUTCSeconds()` and `getUTCMilliseconds()` methods are used.

If a parameter you specify is outside of the expected range, `setUTCMinutes()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes will be incremented by 1 (`minutesValue + 1`), and 40 will be used for seconds.

Examples

Using setUTCMinutes()

```
const theBigDay = new Date();
theBigDay.setUTCMinutes(43);
```

See also

- `Date.prototype.getUTCMinutes()`
- `Date.prototype.setMinutes()`

Date.prototype.setUTCMonth()

The **`setUTCMonth()`** method of `Date` instances changes the month and/or day of the month for this date according to universal time.

Try it

```
const event = new Date("December 31, 1975 23:15:30 GMT-3:00");

console.log(event.toUTCString());
// Expected output: "Thu, 01 Jan 1976 02:15:30 GMT"

console.log(event.getUTCMonth());
// Expected output: 0

event.setUTCMonth(11);

console.log(event.toUTCString());
// Expected output: "Wed, 01 Dec 1976 02:15:30 GMT"
```

Syntax

```
setUTCMonth(monthValue)
setUTCMonth(monthValue, dateValue)
```

Parameters

`monthValue`

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer from 1 to 31 representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `dateValue` parameter, the value returned from the `getUTCDate()` method is used.

If a parameter you specify is outside of the expected range, `setUTCMonth()` attempts to update the date information in the `Date` object accordingly. For example, if you use 15 for `monthValue`, the year will be incremented by 1, and 3 will be used for month.

Examples

Using setUTCMonth()

```
const theBigDay = new Date();
theBigDay.setUTCMonth(11);
```

See also

- `Date.prototype.getUTCMonth()`
- `Date.prototype.setMonth()`

Date.prototype.setUTCSeconds()

The **`setUTCSeconds()`** method of `Date` instances changes the seconds and/or milliseconds for this date according to universal time.

Try it

```
const date = new Date("December 31, 1975, 23:15:30 GMT+11:00");

console.log(date.getUTCSeconds());
// Expected output: 30

date.setUTCSeconds(39);

console.log(date.getUTCSeconds());
// Expected output: 39
```

Syntax

```
setUTCSeconds(secondsValue)
setUTCSeconds(secondsValue, msValue)
```

Parameters

`secondsValue`

An integer between 0 and 59 representing the seconds.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `msValue` parameter, the value returned from the `getUTCMilliseconds()` method is used.

If a parameter you specify is outside of the expected range, `setUTCSeconds()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes stored in the `Date` object will be incremented by 1, and 40 will be used for seconds.

Examples

Using setUTCSeconds()

```
const theBigDay = new Date();
theBigDay.setUTCSeconds(20);
```

See also

- `Date.prototype.getUTCSeconds()`
- `Date.prototype.setSeconds()`

Date.prototype.setYear()

The **`setYear()`** method of `Date` instances sets the year for a specified date according to local time.

However, the way the legacy `setYear()` method sets year values is different from how the preferred `setFullYear()` method sets year values — and in some cases, also different from how `new Date()` and `Date.parse()` set year values. Specifically, given two-digit numbers, such as `22` and `61`:

- `setYear()` interprets any two-digit number as an offset to `1900`; so `date.setYear(22)` results in the year value being set to `1922`, and `date.setYear(61)` results in the year value being set to `1961`. (In contrast, while `new Date(61, 1)` also results in the year value being set to `1961`, `new Date("2/1/22")` results in the year value being set to `2022`; and similarly for `Date.parse()`).
- `setFullYear()` does no special interpretation but instead uses the literal two-digit value as-is to set the year; so `date.setFullYear(61)` results in the year value being set to `0061`, and `date.setFullYear(22)` results in the year value being set to `0022`.

Because of those differences in behavior, you should no longer use the legacy `setYear()` method, but should instead use the preferred `setFullYear()` method.

Syntax

```
setYear(yearValue)
```

Parameters

`yearValue`

An integer.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `yearValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If `yearValue` is a number between 0 and 99 (inclusive), then the year for `dateObj` is set to `1900 + yearValue`. Otherwise, the year for `dateObj` is set to `yearValue`.

Examples

Using setYear()

The first two lines set the year to 1996. The third sets the year to 2000.

```
const theBigDay = new Date();

theBigDay.setYear(96);
theBigDay.setYear(1996);
theBigDay.setYear(2000);
```

See also

- Polyfill of `Date.prototype.setYear` in `core-js`
- `Date.prototype.getFullYear()`
- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`
- `Date.prototype.setUTCFullYear()`

Date.prototype[Symbol.toPrimitive]()

The **`[Symbol.toPrimitive]()`** method of `Date` instances returns a primitive value representing this date. It may either be a string or a number, depending on the hint given.

Try it

```
// Depending on timezone, your results will vary
const date = new Date("20 December 2019 14:48");

console.log(date[Symbol.toPrimitive]("string"));
// Expected output: "Fri Dec 20 2019 14:48:00 GMT+0530 (India Standard Time)"

console.log(date[Symbol.toPrimitive]("number"));
// Expected output: 1576833480000
```

Syntax

```
date[Symbol.toPrimitive](hint)
```

Parameters

`hint`

A string representing the type of the primitive value to return. The following values are valid:

- `"string"` or `"default"`: The method should return a string.

- `"number"`: The method should return a number.

Return value

If `hint` is `"string"` or `"default"`, this method returns a string by coercing the `this` value to a string (first trying `toString()` then trying `valueOf()`).

If `hint` is `"number"`, this method returns a number by coercing the `this` value to a number (first trying `valueOf()` then trying `toString()`).

Exceptions

`TypeError`

Thrown if the `hint` argument is not one of the three valid values.

Description

The `[Symbol.toPrimitive]()` method is part of the type coercion protocol. JavaScript always calls the `[Symbol.toPrimitive]()` method in priority to convert an object to a primitive value. You rarely need to invoke the `[Symbol.toPrimitive]()` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

The `[Symbol.toPrimitive]()` method of the `Date` object returns a primitive value by either invoking `this.valueOf()` and returning a number, or invoking `this.toString()` and returning a string. It exists to override the default primitive coercion process to return a string instead of a number, because primitive coercion, by default, calls `valueOf()` before `toString()`. With the custom `[Symbol.toPrimitive]()`, `new Date(0) + 1` returns `"Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)1"` (a string) instead of `1` (a number).

Examples

Using [Symbol.toPrimitive]()

```
const d = new Date(0); // 1970-01-01T00:00:00.000Z

d[Symbol.toPrimitive]("string"); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
d[Symbol.toPrimitive]("number"); // 0
d[Symbol.toPrimitive]("default"); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
```

See also

- `Symbol.toPrimitive`

Date.prototype.toDateString()

The **`toDateString()`** method of `Date` instances returns a string representing the date portion of this date interpreted in the local timezone.

Try it

```
const event = new Date(1993, 6, 28, 14, 39, 7);

console.log(event.toString());
// Expected output: "Wed Jul 28 1993 14:39:07 GMT+0200 (CEST)"
// Note: your timezone may vary

console.log(event.toDateString());
// Expected output: "Wed Jul 28 1993"
```

Syntax

```
toDateString()
```

Parameters

None.

Return value

A string representing the date portion of the given date (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

`Date` instances refer to a specific point in time. `toDateString()` interprets the date in the local timezone and formats the *date* part in English. It always uses the following format, separated by spaces:

1. First three letters of the week day name
2. First three letters of the month name
3. Two-digit day of the month, padded on the left a zero if necessary
4. Four-digit year (at least), padded on the left with zeros if necessary. May have a negative sign

For example: "Thu Jan 01 1970".

- If you only want to get the *time* part, use `toTimeString()`.
- If you want to get both the date and time, use `toString()`.
- If you want to make the date interpreted as UTC instead of local timezone, use `toUTCString()`.
- If you want to format the date in a more user-friendly format (e.g., localization), use `toLocaleDateString()`.

Examples

Using toDateString()

```
const d = new Date(0);

console.log(d.toString()); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
console.log(d.toDateString()); // "Thu Jan 01 1970"
```

See also

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toString()`

Date.prototype.toISOString()

The **`toISOString()`** method of `Date` instances returns a string representing this date in the date time string format, a *simplified* format based on ISO 8601, which is always 24 or 27 characters long (`YYYY-MM-DDTHH:mm:ss.sssZ` or `±YYYYYY-MM-DDTHH:mm:ss.sssZ`, respectively). The timezone is always UTC, as denoted by the suffix `Z`.

Try it

```
const event = new Date("05 October 2011 14:48 UTC");
console.log(event.toString());
// Expected output: "Wed Oct 05 2011 16:48:00 GMT+0200 (CEST)"
// Note: your timezone may vary

console.log(event.toISOString());
// Expected output: "2011-10-05T14:48:00.000Z"
```

Syntax

```
toISOString()
```

Parameters

None.

Return value

A string representing the given date in the date time string format according to universal time. It's the same format as the one required to be recognized by `Date.parse()`.

Exceptions

`RangeError`

Thrown if the date is invalid or if it corresponds to a year that cannot be represented in the date string format.

Examples

Using toISOString()

```
const d = new Date(0);

console.log(d.toISOString()); // "1970-01-01T00:00:00.000Z"
```

See also

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toString()`
- `Date.prototype.toUTCString()`

Date.prototype.toJSON()

The **`toJSON()`** method of `Date` instances returns a string representing this date in the same ISO format as `toISOString()`.

Try it

```
const event = new Date("August 19, 1975 23:15:30 UTC");

const jsonDate = event.toJSON();

console.log(jsonDate);
// Expected output: "1975-08-19T23:15:30.000Z"

console.log(new Date(jsonDate).toUTCString());
// Expected output: "Tue, 19 Aug 1975 23:15:30 GMT"
```

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given date in the date time string format according to universal time, or `null` when the date is invalid. For valid dates, the return value is the same as that of `toISOString()`.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Date` object is stringified. This method is generally intended to, by default, usefully serialize `Date` objects during JSON serialization, which can then be deserialized using the `Date()` constructor within the reviver of `JSON.parse()`.

The method first attempts to convert its `this` value to a primitive by calling its `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. If the result is a non-finite number, `null` is returned. (This generally corresponds to an invalid date, whose `valueOf()` returns `NaN`.) Otherwise, if the converted primitive is not a number or is a finite number, the return value of `this.toISOString()` is returned.

Note that the method does not check whether the `this` value is a valid `Date` object. However, calling `Date.prototype.toJSON()` on non-`Date` objects fails unless the object's number primitive representation is `NaN`, or the object also has a `toISOString()` method.

Examples

Using toJSON()

```
const jsonDate = new Date(0).toJSON(); // '1970-01-01T00:00:00.000Z'
const backToDate = new Date(jsonDate);

console.log(jsonDate); // 1970-01-01T00:00:00.000Z
```

Serialization round-tripping

When parsing JSON containing date strings, you can use the `Date()` constructor to revive them into the original date objects.

```
const fileData = {
  author: "Maria",
  title: "Date.prototype.toJSON()",
  createdAt: new Date(2019, 3, 15),
  updatedAt: new Date(2020, 6, 26),
};
const response = JSON.stringify(fileData);

// Imagine transmission through network

const data = JSON.parse(response, (key, value) => {
  if (key === "createdAt" || key === "updatedAt") {
    return new Date(value);
  }
  return value;
});

console.log(data);
```

> **Note:** The reviver of `JSON.parse()` must be specific to the payload shape you expect, because the serialization is *irreversible*: it's not possible to distinguish between a string that represents a Date and a normal string.

See also

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toUTCString()`

Date.prototype.toLocaleDateString()

The **`toLocaleDateString()`** method of `Date` instances returns a string with a language-sensitive representation of the date portion of this date in the local timezone. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleDateString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

```
const event = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
```

```
console.log(event.toLocaleDateString("de-DE", options));
// Expected output (varies according to local timezone): Donnerstag, 20. Dezember 2012

console.log(event.toLocaleDateString("ar-EG", options));
// Expected output (varies according to local timezone): الخميس، ٢٠ ديسمبر، ٢٠١٢

console.log(event.toLocaleDateString(undefined, options));
// Expected output (varies according to local timezone and default locale): Thursday, December 20, 2012
```

Syntax

```
toLocaleDateString()
toLocaleDateString(locales)
toLocaleDateString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

> A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.
>
> In implementations without `Intl.DateTimeFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

> An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. The `timeStyle` option must be undefined, or a `TypeError` would be thrown. If `weekday`, `year`, `month`, and `day` are all undefined, then `year`, `month`, and `day` will be set to `"numeric"`.
>
> In implementations without `Intl.DateTimeFormat` support, this parameter is ignored.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the date portion of the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleDateString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleDateString()` to hardcoded constants.

Examples

Using toLocaleDateString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

// toLocaleDateString() without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(date.toLocaleDateString());
// "12/11/2012" if run in en-US locale with time zone America/Los_Angeles
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleDateString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```
function toLocaleDateStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.DateTimeFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized date formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// formats below assume the local time zone of the locale;
// America/Los_Angeles for the US

// US English uses month-day-year order
console.log(date.toLocaleDateString("en-US"));
// "12/20/2012"

// British English uses day-month-year order
console.log(date.toLocaleDateString("en-GB"));
// "20/12/2012"

// Korean uses year-month-day order
console.log(date.toLocaleDateString("ko-KR"));
// "2012. 12. 20."

// Event for Persian, It's hard to manually convert date to Solar Hijri
console.log(date.toLocaleDateString("fa-IR"));
// "۱۳۹۱/۹/۳۰"

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(date.toLocaleDateString("ar-EG"));
// "٢٠١٢/١٢/٢٠"

// for Japanese, applications may want to use the Japanese calendar,
// where 2012 was the year 24 of the Heisei era
console.log(date.toLocaleDateString("ja-JP-u-ca-japanese"));
// "24/12/20"

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(date.toLocaleDateString(["ban", "id"]));
// "20/12/2012"
```

Using options

The results provided by `toLocaleDateString()` can be customized using the `options` parameter:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Request a weekday along with a long date
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(date.toLocaleDateString("de-DE", options));
// "Donnerstag, 20. Dezember 2012"

// An application may want to use UTC and make that visible
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(date.toLocaleDateString("en-US", options));
// "Thursday, December 20, 2012, UTC"
```

See also

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toString()`

Date.prototype.toLocaleString()

The **`toLocaleString()`** method of `Date` instances returns a string with a language-sensitive representation of this date in the local timezone. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

```
const event = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// British English uses day-month-year order and 24-hour time without AM/PM
console.log(event.toLocaleString("en-GB", { timeZone: "UTC" }));
// Expected output: "20/12/2012, 03:00:00"

// Korean uses year-month-day order and 12-hour time with AM/PM
```

```
console.log(event.toLocaleString("ko-KR", { timeZone: "UTC" }));
// Expected output: "2012. 12. 20. 오전 3:00:00"
```

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

> A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.
>
> In implementations without `Intl.DateTimeFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

> An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If `weekday`, `year`, `month`, `day`, `dayPeriod`, `hour`, `minute`, `second`, and `fractionalSecondDigits` are all undefined, then `year`, `month`, `day`, `hour`, `minute`, `second` will be set to `"numeric"`.
>
> In implementations without `Intl.DateTimeFormat` support, this parameter is ignored.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method – without specifying `locale` or `options` – depends on the implementation and returns a string formatted based on the default locale and time zone, and with default options.

```
const date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

console.log(date.toLocaleString());
// "12/11/2012, 7:00:00 PM" if run in en-US locale with time zone America/Los_Angeles
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```
function toLocaleStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.DateTimeFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized date and time formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const date = new Date(Date.UTC(2012, 1, 2, 3, 0, 0));

// Formats below assume the local time zone of the locale;
// America/Los_Angeles for the US

// US English uses month-day-year order and 12-hour time with AM/PM
console.log(date.toLocaleString("en-US"));
```

```
// "2/1/2012, 7:00:00 PM" (UTC-8 is the previous day)

// British English uses day-month-year order and 24-hour time without AM/PM
console.log(date.toLocaleString("en-GB"));
// "02/02/2012, 03:00:00" (UTC+0 or UTC+1 depending on time of the year)

// Korean uses year-month-day order and 12-hour time with AM/PM
console.log(date.toLocaleString("ko-KR"));
// "2012. 2. 2. 오후 12:00:00"

// Arabic in most Arabic-speaking countries uses Eastern Arabic numerals
console.log(date.toLocaleString("ar-EG"));
// "٢/٢/٢٠١٢ ٥:٠٠:٠٠ ص"

// For Japanese, applications may want to use the Japanese calendar,
// where 2012 was the year 24 of the Heisei era
console.log(date.toLocaleString("ja-JP-u-ca-japanese"));
// "H24/2/2 12:00:00"

// When requesting a language that may not be supported, such as
// Balinese, include a fallback language (in this case, Indonesian)
console.log(date.toLocaleString(["ban", "id"]));
// "2/2/2012 11.00.00"
```

Using options

The results provided by `toLocaleString()` can be customized using the `options` parameter:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Request a weekday along with a long date
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(date.toLocaleString("de-DE", options));
// Example output: "Donnerstag, 20. Dezember 2012"
// The exact date may shift depending on your local time zone.

// An application may want to use UTC and make that visible
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(date.toLocaleString("en-US", options));
// Example output: "Thursday, December 20, 2012 at UTC"

// Sometimes even the US needs 24-hour time
console.log(date.toLocaleString("en-US", { hour12: false }));
// Example output: "12/19/2012, 19:00:00"
// The exact date and time may shift depending on your local time zone.
```

See also

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toString()`

Date.prototype.toLocaleTimeString()

The **`toLocaleTimeString()`** method of `Date` instances returns a string with a language-sensitive representation of the time portion of this date in the local timezone. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleTimeString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

```
// Depending on timezone, your results will vary
const event = new Date("August 19, 1975 23:15:30 GMT+00:00");

console.log(event.toLocaleTimeString("en-US"));
// Expected output: "1:15:30 AM"

console.log(event.toLocaleTimeString("it-IT"));
// Expected output: "01:15:30"

console.log(event.toLocaleTimeString("ar-EG"));
// Expected output: "١:١٥:٣٠ ص"
```

Syntax

```
toLocaleTimeString()
toLocaleTimeString(locales)
toLocaleTimeString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

> A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.
>
> In implementations without `Intl.DateTimeFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

> An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If `dayPeriod`, `hour`, `minute`, `second`, and `fractionalSecondDigits` are all undefined, then `hour`, `minute`, `second` will be set to `"numeric"`.
>
> In implementations without `Intl.DateTimeFormat` support, this parameter is ignored.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the time portion of the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleTimeString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleTimeString()` to hardcoded constants.

Examples

Using toLocaleTimeString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

// toLocaleTimeString() without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(date.toLocaleTimeString());
// "7:00:00 PM" if run in en-US locale with time zone America/Los_Angeles
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleTimeString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```
function toLocaleTimeStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.DateTimeFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized time formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// formats below assume the local time zone of the locale;
// America/Los_Angeles for the US

// US English uses 12-hour time with AM/PM
console.log(date.toLocaleTimeString("en-US"));
// "7:00:00 PM"
```

```
// British English uses 24-hour time without AM/PM
console.log(date.toLocaleTimeString("en-GB"));
// "03:00:00"

// Korean uses 12-hour time with AM/PM
console.log(date.toLocaleTimeString("ko-KR"));
// "오후 12:00:00"

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(date.toLocaleTimeString("ar-EG"));
// "٩ ٧:٠٠:٠٠"

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(date.toLocaleTimeString(["ban", "id"]));
// "11.00.00"
```

Using options

The results provided by `toLocaleTimeString()` can be customized using the `options` parameter:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// An application may want to use UTC and make that visible
const options = { timeZone: "UTC", timeZoneName: "short" };
console.log(date.toLocaleTimeString("en-US", options));
// "3:00:00 AM GMT"

// Sometimes even the US needs 24-hour time
console.log(date.toLocaleTimeString("en-US", { hour12: false }));
// "19:00:00"

// Show only hours and minutes, use options with the default locale - use an empty array
console.log(
  date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" }),
);
// "20:01"
```

See also

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toString()`

Date.prototype.toString()

The **`toString()`** method of `Date` instances returns a string representing this date interpreted in the local timezone.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.toString());
// Expected output: "Tue Aug 19 1975 23:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the given date (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

The `toString()` method is part of the type coercion protocol. Because `Date` has a `[Symbol.toPrimitive]()` method, that method always takes priority over `toString()` when a `Date` object is implicitly coerced to a string. However, `Date.prototype[Symbol.toPrimitive]()` still calls `this.toString()` internally.

The `Date` object overrides the `toString()` method of `Object`. `Date.prototype.toString()` returns a string representation of the Date as interpreted in the local timezone, containing both the date and the time — it joins the string representation specified in `toDateString()` and `toTimeString()` together, adding a space in between. For example: "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)".

`Date.prototype.toString()` must be called on `Date` instances. If the `this` value does not inherit from `Date.prototype`, a `TypeError` is thrown.

- If you only want to get the *date* part, use `toDateString()`.
- If you only want to get the *time* part, use `toTimeString()`.

- If you want to make the date interpreted as UTC instead of local timezone, use `toUTCString()`.
- If you want to format the date in a more user-friendly format (e.g., localization), use `toLocaleString()`.

Examples

Using toString()

```
const d = new Date(0);
console.log(d.toString()); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
```

See also

- `Object.prototype.toString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toTimeString()`

Date.prototype.toTemporalInstant()

The **`toTemporalInstant()`** method of `Date` instances returns a new `Temporal.Instant` object with the same `epochMilliseconds` value as this date's timestamp.

Use this method to convert legacy `Date` values to the `Temporal` API, then further convert it to other `Temporal` classes as necessary.

Syntax

```
toTemporalInstant()
```

Parameters

None.

Return value

A new `Temporal.Instant` object with the same `epochMilliseconds` value as this date's timestamp. Its microsecond and nanosecond components are always `0`.

Exceptions

`RangeError`

Thrown if the date is invalid (it has a timestamp of `NaN`).

Examples

Using toTemporalInstant()

```
const legacyDate = new Date("2021-07-01T12:34:56.789Z");
const instant = legacyDate.toTemporalInstant();

// Further convert it to other objects
const zdt = instant.toZonedDateTimeISO("UTC");
const date = zdt.toPlainDate();
console.log(date.toString()); // 2021-07-01
```

See also

- `Temporal.Instant`
- `Temporal.ZonedDateTime`
- `Temporal.Instant.fromEpochMilliseconds()`

Date.prototype.toTimeString()

The **`toTimeString()`** method of `Date` instances returns a string representing the time portion of this date interpreted in the local timezone.

Try it

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.toTimeString());
// Expected output: "23:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

```
toTimeString()
```

Parameters

None.

Return value

A string representing the time portion of the given date (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

`Date` instances refer to a specific point in time. `toTimeString()` interprets the date in the local timezone and formats the *time* part in English. It always uses the format of `HH:mm:ss GMT±xxxx (TZ)`, where:

Format String	Description
`HH`	Hour, as two digits with leading zero if required
`mm`	Minute, as two digits with leading zero if required
`ss`	Seconds, as two digits with leading zero if required
`±xxxx`	The local timezone's offset — two digits for hours and two digits for minutes (e.g., `-0500`, `+0800`)
`TZ`	The timezone's name (e.g., `PDT`, `PST`)

For example: "04:42:04 GMT+0000 (Coordinated Universal Time)".

- If you only want to get the *date* part, use `toDateString()`.
- If you want to get both the date and time, use `toString()`.
- If you want to make the date interpreted as UTC instead of local timezone, use `toUTCString()`.
- If you want to format the date in a more user-friendly format (e.g., localization), use `toLocaleTimeString()`.

Examples

Using toTimeString()

```
const d = new Date(0);

console.log(d.toString()); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
console.log(d.toTimeString()); // "00:00:00 GMT+0000 (Coordinated Universal Time)"
```

See also

- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toString()`

Date.prototype.toUTCString()

The **`toUTCString()`** method of `Date` instances returns a string representing this date in the RFC 7231 format, with negative years allowed. The timezone is always UTC. `toGMTString()` is an alias of this method.

Try it

```
const event = new Date("14 Jun 2017 00:00:00 PDT");

console.log(event.toUTCString());
// Expected output: "Wed, 14 Jun 2017 07:00:00 GMT"
```

Syntax

```
toUTCString()
```

Parameters

None.

Return value

A string representing the given date using the UTC time zone (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

The value returned by `toUTCString()` is a string in the form `Www, dd Mmm yyyy HH:mm:ss GMT`, where:

Format String	Description
`Www`	Day of week, as three letters (e.g., `Sun`, `Mon`)
`dd`	Day of month, as two digits with leading zero if required
`Mmm`	Month, as three letters (e.g., `Jan`, `Feb`)
`yyyy`	Year, as four or more digits with leading zeroes if required
`HH`	Hour, as two digits with leading zero if required
`mm`	Minute, as two digits with leading zero if required
`ss`	Seconds, as two digits with leading zero if required

Aliasing

JavaScript's `Date` API was inspired by Java's `java.util.Date` library (while the latter had become de facto legacy since Java 1.1 in 1997). In particular, the Java `Date` class had a method called `toGMTString` — which was poorly named, because the Greenwich Mean Time is not equivalent to the Coordinated Universal Time, while JavaScript

dates always operate by UTC time. For web compatibility reasons, `toGMTString` remains as an alias to `toUTCString`, and they refer to the exact same function object. This means:

```
Date.prototype.toGMTString.name === "toUTCString";
```

Examples

Using toUTCString()

```
const d = new Date(0);
console.log(d.toUTCString()); // 'Thu, 01 Jan 1970 00:00:00 GMT'
```

See also

- `Date.prototype.toLocaleString()`
- `Date.prototype.toString()`
- `Date.prototype.toISOString()`

Date.UTC()

The **`Date.UTC()`** static method accepts parameters representing the date and time components similar to the `Date` constructor, but treats them as UTC. It returns the number of milliseconds since January 1, 1970, 00:00:00 UTC.

Try it

```
const utcDate1 = new Date(Date.UTC(96, 1, 2, 3, 4, 5));
const utcDate2 = new Date(Date.UTC(0, 0, 0, 0, 0, 0));

console.log(utcDate1.toUTCString());
// Expected output: "Fri, 02 Feb 1996 03:04:05 GMT"

console.log(utcDate2.toUTCString());
// Expected output: "Sun, 31 Dec 1899 00:00:00 GMT"
```

Syntax

```
Date.UTC(year)
Date.UTC(year, monthIndex)
Date.UTC(year, monthIndex, day)
Date.UTC(year, monthIndex, day, hours)
Date.UTC(year, monthIndex, day, hours, minutes)
Date.UTC(year, monthIndex, day, hours, minutes, seconds)
Date.UTC(year, monthIndex, day, hours, minutes, seconds, milliseconds)
```

Parameters

`year`

Integer value representing the year. Values from `0` to `99` map to the years `1900` to `1999`. All other values are the actual year. See the example.

`monthIndex` Optional

Integer value representing the month, beginning with `0` for January to `11` for December. Defaults to `0`.

`day` Optional

Integer value representing the day of the month. Defaults to `1`.

`hours` Optional

Integer value between `0` and `23` representing the hour of the day. Defaults to `0`.

`minutes` Optional

Integer value representing the minute segment of a time. Defaults to `0`.

`seconds` Optional

Integer value representing the second segment of a time. Defaults to `0`.

`milliseconds` Optional

Integer value representing the millisecond segment of a time. Defaults to `0`.

Return value

A number representing the timestamp of the given date. Returns `NaN` if the date is invalid.

Description

Years between `0` and `99` are converted to a year in the 20th century `(1900 + year)`. For example, `95` is converted to the year `1995`.

The `UTC()` method differs from the `Date()` constructor in three ways:

1. `Date.UTC()` uses universal time instead of the local time.
2. `Date.UTC()` returns a time value as a number instead of creating a `Date` object.
3. When passed a single number, `Date.UTC()` interprets it as a year instead of a timestamp.

If a parameter is outside of the expected range, the `UTC()` method updates the other parameters to accommodate the value. For example, if `15` is used for `monthIndex`, the year will be incremented by 1 (`year + 1`) and `3` will be used for the month.

Because `UTC()` is a static method of `Date`, you always use it as `Date.UTC()`, rather than as a method of a `Date` object you created.

Examples

Using Date.UTC()

The following statement creates a `Date` object with the arguments treated as UTC instead of local:

```
const utcDate = new Date(Date.UTC(2018, 11, 1, 0, 0, 0));
```

Behavior of Date.UTC() with one argument

`Date.UTC()` when passed one argument used to have inconsistent behavior, because implementations only kept the behavior consistent with the `Date()` constructor, which does not interpret a single argument as the year number. Implementations are now required to treat omitted `monthIndex` as `0`, instead of coercing it to `NaN`.

```
Date.UTC(2017); // 1483228800000
```

See also

- `Date.parse()`
- `Date`

Date.prototype.valueOf()

The **`valueOf()`** method of `Date` instances returns the number of milliseconds for this date since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

```
const date1 = new Date(Date.UTC(96, 1, 2, 3, 4, 5));

console.log(date1.valueOf());
// Expected output: 823230245000

const date2 = new Date("02 Feb 1996 03:04:05 GMT");

console.log(date2.valueOf());
// Expected output: 823230245000
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

A number representing the timestamp, in milliseconds, of this date. Returns `NaN` if the date is invalid.

Description

The `valueOf()` method is part of the type coercion protocol. Because `Date` has a `[Symbol.toPrimitive]()` method, that method always takes priority over `valueOf()` when a `Date` object is implicitly coerced to a number. However, `Date.prototype[Symbol.toPrimitive]()` still calls `this.valueOf()` internally.

The `Date` object overrides the `valueOf()` method of `Object`. `Date.prototype.valueOf()` returns the timestamp of the date, which is functionally equivalent to the `Date.prototype.getTime()` method.

Examples

Using valueOf()

```
const d = new Date(0); // 1970-01-01T00:00:00.000Z
console.log(d.valueOf()); // 0
```

See also

- `Object.prototype.valueOf()`
- `Date.prototype.getTime()`

decodeURI()

The **`decodeURI()`** function decodes a Uniform Resource Identifier (URI) previously created by `encodeURI()` or a similar routine.

Try it

```
const uri = "https://mozilla.org/?x=шеллы";
const encoded = encodeURI(uri);
console.log(encoded);
// Expected output: "https://mozilla.org/?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"

try {
  console.log(decodeURI(encoded));
  // Expected output: "https://mozilla.org/?x=шеллы"
} catch (e) {
  // Catches a malformed URI
  console.error(e);
}
```

Syntax

```
decodeURI(encodedURI)
```

Parameters

`encodedURI`

A complete, encoded Uniform Resource Identifier.

Return value

A new string representing the unencoded version of the given encoded Uniform Resource Identifier (URI).

Exceptions

`URIError`

Thrown if `encodedURI` contains a `%` not followed by two hexadecimal digits, or if the escape sequence does not encode a valid UTF-8 character.

Description

`decodeURI()` is a function property of the global object.

The `decodeURI()` function decodes the URI by treating each escape sequence in the form `%XX` as one UTF-8 code unit (one byte). In UTF-8, the number of leading 1 bits in the first byte, which may be 0 (for 1-byte ASCII characters), 2, 3, or 4, indicates the number of bytes in the character. So by reading the first escape sequence, `decodeURI()` can determine how many more escape sequences to consume. If `decodeURI()` fails to find the expected number of sequences, or if the escape sequences don't encode a valid UTF-8 character, a `URIError` is thrown.

`decodeURI()` decodes all escape sequences, but if the escape sequence encodes one of the following characters, the escape sequence is preserved in the output string (because they are part of the URI syntax):

```
; / ? : @ & = + $ , #
```

Examples

Decoding a Cyrillic URL

```
decodeURI(
  "https://developer.mozilla.org/ru/docs/JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B",
);
// "https://developer.mozilla.org/ru/docs/JavaScript_шеллы"
```

decodeURI() vs. decodeURIComponent()

`decodeURI()` assumes the input is a full URI, so it does not decode characters that are part of the URI syntax.

```
decodeURI(
  "https://developer.mozilla.org/docs/JavaScript%3A%20a_scripting_language",
);
// "https://developer.mozilla.org/docs/JavaScript%3A a_scripting_language"

decodeURIComponent(
  "https://developer.mozilla.org/docs/JavaScript%3A%20a_scripting_language",
);
// "https://developer.mozilla.org/docs/JavaScript: a_scripting_language"
```

Catching errors

```
try {
  const a = decodeURI("%E0%A4%A");
} catch (e) {
  console.error(e);
}

// URIError: malformed URI sequence
```

See also

- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`

decodeURIComponent()

The **`decodeURIComponent()`** function decodes a Uniform Resource Identifier (URI) component previously created by `encodeURIComponent()` or by a similar routine.

Try it

```
function containsEncodedComponents(x) {
  // ie ?,=,&,/ etc
  return decodeURI(x) !== decodeURIComponent(x);
}

console.log(containsEncodedComponents("%3Fx%3Dtest")); // ?x=test
// Expected output: true

console.log(containsEncodedComponents("%D1%88%D0%B5%D0%BB%D0%BB%D1%8B")); // шеллы
// Expected output: false
```

Syntax

```
decodeURIComponent(encodedURI)
```

Parameters

`encodedURI`

An encoded component of a Uniform Resource Identifier.

Return value

A new string representing the decoded version of the given encoded Uniform Resource Identifier (URI) component.

Exceptions

`URIError`

Thrown if `encodedURI` contains a `%` not followed by two hexadecimal digits, or if the escape sequence does not encode a valid UTF-8 character.

Description

`decodeURIComponent()` is a function property of the global object.

`decodeURIComponent()` uses the same decoding algorithm as described in `decodeURI()`. It decodes *all* escape sequences, including those that are not created by `encodeURIComponent`, like `-.!~*'()`.

Examples

Decoding a Cyrillic URL component

```
decodeURIComponent("JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B");
// "JavaScript_шеллы"
```

Catching errors

```
try {
  const a = decodeURIComponent("%E0%A4%A");
} catch (e) {
  console.error(e);
}

// URIError: malformed URI sequence
```

Decoding query parameters from a URL

`decodeURIComponent()` cannot be used directly to parse query parameters from a URL. It needs a bit of preparation.

```
function decodeQueryParam(p) {
  return decodeURIComponent(p.replace(/\+/g, " "));
}

decodeQueryParam("search+query%20%28correct%29");
// 'search query (correct)'
```

See also

- `decodeURI`
- `encodeURI`

- `encodeURIComponent`

DisposableStack

The **`DisposableStack`** object represents a stack of disposers to run when the stack itself is disposed. Disposer functions are executed in reverse order of registration, with strong error handling guarantees. Calling its `move()` method will transfer responsibility for calling the current registered disposers to a new `DisposableStack` and prevent registering any additional disposers.

Description

A `DisposableStack` is not exactly a "stack" in terms of its interface. It has several methods for pushing disposers to it, but it has no way to pop one disposer off. Rather, *all* disposers are popped and executed one-by-one when the stack is disposed.

You register disposable resources to the `DisposableStack` using its `use()`, `adopt()`, or `defer()` methods.

```
using disposer = new DisposableStack();
const reader = disposer.use(stream.getReader());
```

Then, when the `disposer` goes out of scope, all resources registered to it are disposed in reverse order of registration, unless they have been moved out with `move()`.

It is good practice to *not* extract the resource acquisition expression to a separate statement, no matter how long the expression is. You should always wrap the `use()` or `adopt()` call around the resource acquisition expression to ensure that the resource is registered to the stack immediately.

```
using disposer = new DisposableStack();
const reader = stream.getReader();
disposer.use(reader);
```

Functionally, these two code snippets are equivalent. However, the first one is less error-prone because the resource is declared and registered in a single line. If someone puts more code between the second and third lines of the second snippet, an error could occur, causing the resource to leak.

Constructor

`DisposableStack()`

Creates a new `DisposableStack` object.

Instance properties

These properties are defined on `DisposableStack.prototype` and shared by all `DisposableStack` instances.

`DisposableStack.prototype.constructor`

The constructor function that created the instance object. For `DisposableStack` instances, the initial value is the `DisposableStack` constructor.

`DisposableStack.prototype.disposed`

Read-only. Returns `true` if the `DisposableStack` has been disposed, or `false` if not.

`DisposableStack.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"DisposableStack"`. This property is used in `Object.prototype.toString()`.

Instance methods

`DisposableStack.prototype.adopt()`

Registers a value that doesn't implement the disposable protocol to the stack by providing a custom disposer function.

`DisposableStack.prototype.defer()`

Takes a callback function to be called when the stack is disposed.

`DisposableStack.prototype.dispose()`

Disposes this stack by calling all disposers registered to it in reverse order of registration.

`DisposableStack.prototype.move()`

Creates a new `DisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

`DisposableStack.prototype.use()`

Registers a value that implements the disposable protocol to the stack.

`DisposableStack.prototype[Symbol.dispose]`

An alias for the `dispose()` method.

See also

- Polyfill of `DisposableStack` in `core-js`
- JavaScript resource management
- `Symbol.dispose`
- `using`

- `AsyncDisposableStack`

DisposableStack.prototype.adopt()

The **`adopt()`** method of `DisposableStack` instances registers a value that doesn't implement the disposable protocol to the stack by providing a custom disposer function.

Syntax

```
adopt(value, onDispose)
```

Parameters

`value`

Any value to be registered to the stack.

`onDispose`

A function that will be called when the stack is disposed. The function receives `value` as its only argument.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `adopt()` is to register a value that doesn't implement the disposable protocol to the stack. If the value is already disposable, you can use `use()` instead, which automatically uses the value's `[Symbol.dispose]()` method as the disposer.

`adopt(value, onDispose)` is almost the same as `defer(() => onDispose(value))`, but it allows you to declare the resource and register it on the same line. This way, there's minimal chance of an error happening between the resource creation and registration, which will cause the resource to leak.

```
using disposer = new DisposableStack();
const reader = disposer.adopt(stream.getReader(), (reader) =>
  reader.releaseLock(),
);
```

```
using disposer = new DisposableStack();
const reader = stream.getReader();
// If someone adds code in between these lines and an error occurs,
// the stream will be locked forever.
disposer.defer(() => reader.close());
```

In the same spirit of "make your resource registered as soon as it's declared", you should always wrap your resource acquisition expression in `adopt()`, instead of extracting it to a separate statement.

```
using disposer = new DisposableStack();
const reader = stream.getReader();
disposer.adopt(reader, (reader) => reader.close());
```

Examples

Using adopt()

This code consumes a `ReadableStream` via a `ReadableStreamDefaultReader`. The reader does not implement the disposable protocol, so we use `adopt()` to register it to the stack.

```
{
  using disposer = new DisposableStack();
  const reader = disposer.adopt(stream.getReader(), (reader) =>
    reader.releaseLock(),
  );
  const { value, done } = reader.read();
  if (!done) {
    // Process the value
  }
  // The reader.releaseLock() method is called here before exiting
}
```

See also

- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.defer()`
- `DisposableStack.prototype.use()`

DisposableStack.prototype.defer()

The **`defer()`** method of `DisposableStack` instances takes a callback function to be called when the stack is disposed.

Syntax

```
defer(onDispose)
```

Parameters

`onDispose`

A function that will be called when the stack is disposed. The function receives no arguments.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `defer()` is to register a cleanup callback that's not specific to the disposal of a particular resource. If the callback is specific to a resource, you should use `use()` or `adopt()` instead. You can also use `defer` when the resource is not claimed within your code:

```
function consumeReader(reader) {
  using disposer = new DisposableStack();
  disposer.defer(() => reader.releaseLock());
  // Do something with reader
}
```

Examples

Using defer()

This function sets a simple lock to prevent multiple async operations from running at the same time. The lock is released when the function completes.

```
let isLocked = false;

async function requestWithLock(url, options) {
  if (isLocked) {
    return undefined;
  }
  using disposer = new DisposableStack();
  isLocked = true;
  disposer.defer(() => (isLocked = false));
  const data = await fetch(url, options).then((res) => res.json());
  return data;
}
```

See also

- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.adopt()`
- `DisposableStack.prototype.use()`

DisposableStack() constructor

The **`DisposableStack()`** constructor creates `DisposableStack` objects.

Syntax

```
new DisposableStack()
```

> **Note:** `DisposableStack()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

None.

Return value

A new `DisposableStack` object.

Examples

Creating a DisposableStack

```
const disposer = new DisposableStack();
disposer.defer(() => console.log("Disposed!"));
disposer.dispose();
// Logs: Disposed!
```

See also

- JavaScript resource management

DisposableStack.prototype.dispose()

The **`dispose()`** method of `DisposableStack` instances disposes this stack by calling all disposers registered to it in reverse order of registration. If the stack is already disposed, this method does nothing.

It performs the same action as `using disposer = new DisposableStack()` at scope exit. It can be used if you need to clean up at a point other than scope exit.

Syntax

```
dispose()
```

Parameters

None.

Return value

None (`undefined`).

Exceptions

`SuppressedError`

> Thrown if multiple disposers in the stack threw an error. If only one error is thrown, it is rethrown as-is. Otherwise, for each additional error, a new `SuppressedError` is created, with the original error as the `suppressed` property, and the new error as the `error` property.

Examples

Disposing a stack

Here we push three disposers to the stack, using the `use()`, `adopt()`, and `defer()` methods. When `dispose()` is called, the disposers are called in reverse order of registration.

Note that usually you don't need to call `dispose()` manually. Declare the stack with `using`, and its `[Symbol.dispose]()` method will be automatically called when the stack goes out of scope.

```
class Resource {
  dispose() {
    console.log("Resource disposed");
  }
  [Symbol.dispose]() {
    console.log("Resource disposed via Symbol.dispose");
  }
}

{
  const disposer = new DisposableStack();
  const resource = disposer.use(new Resource());
  const resource2 = disposer.adopt(new Resource(), (resource) =>
    resource.dispose(),
  );
  disposer.defer(() => console.log("Deferred disposer"));
  disposer.dispose();
  // Logs in order:
  // Deferred disposer
  // Resource disposed
  // Resource disposed via Symbol.dispose
}
```

See also

- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.adopt()`

- `DisposableStack.prototype.defer()`
- `DisposableStack.prototype.use()`
- `DisposableStack.prototype[Symbol.dispose]()`

DisposableStack.prototype.disposed

The **`disposed`** accessor property of `DisposableStack` instances returns a boolean indicating whether or not this `DisposableStack` has been disposed or moved by doing any of the following:

- Calling its `dispose()` method
- Calling its `move()` method
- Declaring it with `using` and letting the variable go out of scope, which automatically calls the `[Symbol.dispose]()` method.

Examples

Checking if a stack is disposed

```
const disposer = new DisposableStack();
console.log(disposer.disposed); // false
disposer.dispose();
console.log(disposer.disposed); // true
```

See also

- JavaScript resource management

DisposableStack.prototype.move()

The **`move()`** method of `DisposableStack` instances creates a new `DisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

Syntax

```
move()
```

Parameters

None.

Return value

A new `DisposableStack` instance.

Exceptions

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `move()` is to enable transferring responsibility for disposal out of the current scope. For example, your function can claim ownership of some resources and dispose them if an error occurs; if everything completes successfully, then you return these resources and transfer ownership to the caller.

When using `move()` to transfer ownership, calling `move()` should be the very last step in your control flow, because there will be no owner in between your code dropping ownership via `move()` and the caller picking up ownership from the return value.

```
let resource1;

function init() {
  using disposer = new DisposableStack();
  resource1 = disposer.use(getResource1());
  // ...
  // Drop ownership immediately before returning
  return disposer.move();
}

// Pick up ownership immediately after returning
using disposer = init();
```

```
let resource1;

function init() {
  using disposer = new DisposableStack();
  resource1 = disposer.use(getResource1());
  // ...
  const newDisposer = disposer.move();
  // If someone adds code in between these lines and an error occurs,
  // there would be no owner to free resource1
  return newDisposer;
}

using disposer = init();
```

Also be cautious with the following pattern, although using the "good" pattern may be very awkward in many cases:

```
function init() {
  using disposer = new DisposableStack();
  const resource1 = disposer.use(getResource1());
  // ...
  return { disposer: disposer.move(), resource1 };
}

const { resource1, ...rest } = init();
// If someone adds code in between these lines and an error occurs,
// there would be no owner to free resource1
using disposer = rest.disposer;
```

`move()` can also be used for conditional disposal in cases where you may sometimes not want to dispose the resources at all. For example:

```
using disposer = new DisposableStack();
const server = disposer.use(makeServer());
await server.init();
if (server.ready) {
  // Successfully initialized server; it now should live through the rest
  // of the program. Drop its disposer and don't pick it up. It will no
  // longer be disposed at all.
  disposer.move();
}
// If we reach the end of the scope without running disposer.move(),
// then server isn't ready for any reason, and we dispose its resources
// by disposing the disposer.
```

Examples

Claiming ownership of a stack

```
function consumeStack(stack) {
  using newStack = stack.move(); // newStack now owns the disposers
  console.log(stack.disposed); // true
  console.log(newStack.disposed); // false
  // newStack is disposed here immediately before the function exits
}

const stack = new DisposableStack();
console.log(stack.disposed); // false
consumeStack(stack);
console.log(stack.disposed); // true
```

Allowing resources to be disposed within two code paths

The major use case of `move()` is when you have one or more resources which could either be disposed right here or could be persisted for later use. In this case, you can put the resources in a `DisposableStack` and then call `move()` when you need to persist the resources for later usage.

```
class PluginHost {
  #disposed = false;
  #disposables;
  #channel;
  #socket;

  constructor() {
    // Create a DisposableStack that is disposed when the constructor exits.
    // If construction succeeds, we move everything out of `disposer` and into
    // `#disposables` to be disposed later.
    using disposer = new DisposableStack();

    // Create an IPC adapter around process.send/process.on("message").
    // When disposed, it unsubscribes from process.on("message").
    this.#channel = disposer.use(new NodeProcessIpcChannelAdapter(process));

    // Create a pseudo-websocket that sends and receives messages over
    // a NodeJS IPC channel.
    this.#socket = disposer.use(new NodePluginHostIpcSocket(this.#channel));

    // If we made it here, then there were no errors during construction and
    // we can safely move the disposables out of `disposer` and into `#disposables`.
    this.#disposables = disposer.move();

    // If construction failed, then `disposer` would be disposed before reaching
    // the line above. Event handlers would be removed, allowing `#channel` and
    // `#socket` to be GC'd.
  }

  [Symbol.dispose]() {
    if (this.#disposed) {
      return;
    }
    this.#disposed = true;
```

```
    // Put `this.#disposables` into a `using` variable, so it is automatically
    // disposed when the function exits.
    using disposables = this.#disposables;

    // NOTE: we can free `#socket` and `#channel` here since they will be
    // disposed by the call to `disposables[Symbol.dispose]()`, below.
    // This isn't strictly a requirement for every disposable, but is
    // good housekeeping since these objects will no longer be useable.
    this.#socket = undefined;
    this.#channel = undefined;
    this.#disposables = undefined;
  }
}
```

See also

- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.dispose()`

DisposableStack.prototype[Symbol.dispose]()

The **`[Symbol.dispose]()`** method of `DisposableStack` instances implements the *disposable protocol* and allows it to be disposed when used with `using` or `await using`. It is an alias for the `dispose()` method.

Syntax

```
disposableStack[Symbol.dispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring a stack with <code>using</code>

The `Symbol.dispose` method is intended to be automatically called in a `using` declaration.

```
{
  using disposer = new DisposableStack();
  const resource = disposer.use(new Resource());
  resource.doSomething();
  // stack is disposed here immediately before the function exits
  // which causes the resource to be disposed
}
```

See also

- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.dispose()`

DisposableStack.prototype.use()

The **`use()`** method of `DisposableStack` instances registers a value that implements the disposable protocol to the stack.

Syntax

```
use(value)
```

Parameters

`value`

The value to register to the stack. Must either contain a `[Symbol.dispose]()` method, or be `null` or `undefined`.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `value` is not `null` or `undefined`, and does not contain a `[Symbol.dispose]()` method.

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `use()` is to register a value that implements the disposable protocol to the stack, as the equivalent of the `using` declaration. If the value does not implement the disposable protocol (it doesn't have the `[Symbol.dispose]()` method), use `adopt()` instead, passing a callback that calls the resource's cleanup method.

You should make your resource registered as soon as it's declared. This means you should always wrap your resource acquisition expression in `use()`, instead of extracting it to a separate statement.

```
using disposer = new DisposableStack();
const reader = stream.getReader();
disposer.use(reader);
```

Examples

Using use()

This code consumes a `ReadableStream` via a `ReadableStreamDefaultReader`. The reader is automatically closed when the function completes, assuming it implements a `[Symbol.dispose]()` method that synchronously releases the lock on the stream.

```
{
  using disposer = new DisposableStack();
  const reader = disposer.use(stream.getReader());
  const { value, done } = reader.read();
  if (!done) {
    // Process the value
  }
  // The reader.releaseLock() method is called here before exiting
}
```

See also

- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.adopt()`
- `DisposableStack.prototype.defer()`

encodeURI()

The **`encodeURI()`** function encodes a URI by replacing each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character (will only be four escape sequences for characters composed of two surrogate characters). Compared to `encodeURIComponent()`, this function encodes fewer characters, preserving those that are part of the URI syntax.

Try it

```
const uri = "https://mozilla.org/?x=шеллы";
const encoded = encodeURI(uri);
console.log(encoded);
// Expected output: "https://mozilla.org/?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"

try {
  console.log(decodeURI(encoded));
  // Expected output: "https://mozilla.org/?x=шеллы"
} catch (e) {
  // Catches a malformed URI
  console.error(e);
}
```

Syntax

```
encodeURI(uri)
```

Parameters

`uri`

A string to be encoded as a URI.

Return value

A new string representing the provided string encoded as a URI.

Exceptions

`URIError`

Thrown if `uri` contains a lone surrogate.

Description

`encodeURI()` is a function property of the global object.

The `encodeURI()` function escapes characters by UTF-8 code units, with each octet encoded in the format `%xx`, left-padded with 0 if necessary. Because lone surrogates in UTF-16 do not encode any valid Unicode character, they cause `encodeURI()` to throw a `URIError`.

`encodeURI()` escapes all characters **except**:

```
A–Z a–z 0–9 - _ . ! ~ * ' ( )

; / ? : @ & = + $ , #
```

The characters on the second line are characters that may be part of the URI syntax, and are only escaped by `encodeURIComponent()`. Both `encodeURI()` and `encodeURIComponent()` do not encode the characters `-.!~*'()`, known as "unreserved marks", which do not have a reserved purpose but are allowed in a URI "as is". (See RFC2396)

The `encodeURI()` function does not encode characters that have special meaning (reserved characters) for a URI. The following example shows all the parts that a URI can possibly contain. Note how certain characters are used to signify special meaning:

```
http://username:password@www.example.com:80/path/to/file.php?foo=316&bar=this+has+spaces#anchor
```

`encodeURI`, as the name implies, is used to encode a URL as a whole, assuming it is already well-formed. If you want to dynamically assemble string values into a URL, you probably want to use `encodeURIComponent()` on each dynamic segment instead, to avoid URL syntax characters in unwanted places.

```
const name = "Ben & Jerry's";

// This is bad:
const badLink = encodeURI(`https://example.com/?choice=${name}`); // "https://example.com/?choice=Ben%20&%20Jerry's"
console.log([...new URL(badLink).searchParams]); // [['choice', 'Ben '], [" Jerry's", '']]

// Instead:
const goodLink = `https://example.com/?choice=${encodeURIComponent(name)}`;
// "https://example.com/?choice=Ben%20%26%20Jerry's"
console.log([...new URL(goodLink).searchParams]); // [['choice', "Ben & Jerry's"]]
```

Examples

encodeURI() vs. encodeURIComponent()

`encodeURI()` differs from `encodeURIComponent()` as follows:

```
const set1 = ";/?:@&=+$,#"; // Reserved Characters
const set2 = "-.!~*'()"; // Unreserved Marks
const set3 = "ABC abc 123"; // Alphanumeric Characters + Space

console.log(encodeURI(set1)); // ;/?:@&=+$,#
console.log(encodeURI(set2)); // -.!~*'()
console.log(encodeURI(set3)); // ABC%20abc%20123 (the space gets encoded as %20)

console.log(encodeURIComponent(set1)); // %3B%2C%2F%3F%3A%40%26%3D%2B%24%23
console.log(encodeURIComponent(set2)); // -.!~*'()
console.log(encodeURIComponent(set3)); // ABC%20abc%20123 (the space gets encoded as %20)
```

Encoding a lone surrogate throws

A `URIError` will be thrown if one attempts to encode a surrogate which is not part of a high-low pair. For example:

```
// High-low pair OK
encodeURI("\uD800\uDFFF"); // "%F0%90%8F%BF"

// Lone high-surrogate code unit throws "URIError: malformed URI sequence"
encodeURI("\uD800");

// Lone low-surrogate code unit throws "URIError: malformed URI sequence"
encodeURI("\uDFFF");
```

You can use `String.prototype.toWellFormed()`, which replaces lone surrogates with the Unicode replacement character (U+FFFD), to avoid this error. You can also use `String.prototype.isWellFormed()` to check if a string contains lone surrogates before passing it to `encodeURI()`.

Encoding for RFC3986

The more recent RFC3986 makes square brackets reserved (for IPv6) and thus not encoded when forming something which could be part of a URL (such as a host). It also reserves !, ', (,), and *, even though these characters have no formalized URI delimiting uses. The following function encodes a string for RFC3986-compliant URL format.

```
function encodeRFC3986URI(str) {
  return encodeURI(str)
    .replace(/%5B/g, "[")
    .replace(/%5D/g, "]")
    .replace(
      /[!'()*]/g,
      (c) => `%${c.charCodeAt(0).toString(16).toUpperCase()}`,
```

```
    );
}
```

See also

- `decodeURI()`
- `encodeURIComponent()`
- `decodeURIComponent()`

encodeURIComponent()

The **`encodeURIComponent()`** function encodes a URI by replacing each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character (will only be four escape sequences for characters composed of two surrogate characters). Compared to `encodeURI()`, this function encodes more characters, including those that are part of the URI syntax.

Try it

```
// Encodes characters such as ?,=,/,&,:
console.log(`?x=${encodeURIComponent("test?")}`);
// Expected output: "?x=test%3F"

console.log(`?x=${encodeURIComponent("шеллы")}`);
// Expected output: "?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"
```

Syntax

```
encodeURIComponent(uriComponent)
```

Parameters

`uriComponent`

A string to be encoded as a URI component (a path, query string, fragment, etc.). Other values are converted to strings.

Return value

A new string representing the provided `uriComponent` encoded as a URI component.

Exceptions

`URIError`

Thrown if `uriComponent` contains a lone surrogate.

Description

`encodeURIComponent()` is a function property of the global object.

`encodeURIComponent()` uses the same encoding algorithm as described in `encodeURI()`. It escapes all characters **except**:

```
A–Z a–z 0–9 - _ . ! ~ * ' ( )
```

Compared to `encodeURI()`, `encodeURIComponent()` escapes a larger set of characters. Use `encodeURIComponent()` on user-entered fields from forms sent to the server — this will encode `&` symbols that may inadvertently be generated during data entry for character references or other characters that require encoding/decoding. For example, if a user writes `Jack & Jill`, without `encodeURIComponent()`, the ampersand could be interpreted on the server as the start of a new field and jeopardize the integrity of the data.

For `application/x-www-form-urlencoded`, spaces are to be replaced by `+`, so one may wish to follow an `encodeURIComponent()` replacement with an additional replacement of `%20` with `+`.

Examples

Encoding for Content-Disposition and Link headers

The following example provides the special encoding required within UTF-8 `Content-Disposition` and `Link` server response header parameters (e.g., UTF-8 filenames):

```
const fileName = "my file(2).txt";
const header = `Content-Disposition: attachment; filename*=UTF-8''${encodeRFC5987ValueChars(
  fileName,
)}`;

console.log(header);
// "Content-Disposition: attachment; filename*=UTF-8''my%20file%282%29.txt"

function encodeRFC5987ValueChars(str) {
  return (
    encodeURIComponent(str)
      // The following creates the sequences %27 %28 %29 %2A (Note that
      // the valid encoding of "*" is %2A, which necessitates calling
      // toUpperCase() to properly encode). Although RFC3986 reserves "!",
      // RFC5987 does not, so we do not need to escape it.
      .replace(
```

```
        /['()*]/g,
        (c) => `%${c.charCodeAt(0).toString(16).toUpperCase()}`,
      )
      // The following are not required for percent-encoding per RFC5987,
      // so we can allow for a little better readability over the wire: |`^
      .replace(/%(7C|60|5E)/g, (str, hex) =>
        String.fromCharCode(parseInt(hex, 16)),
      )
  );
}
```

Encoding for RFC3986

The more recent RFC3986 reserves `!`, `'`, `(`, `)`, and `*`, even though these characters have no formalized URI delimiting uses. The following function encodes a string for RFC3986-compliant URL component format. It also encodes `[` and `]`, which are part of the IPv6 URI syntax. An RFC3986-compliant `encodeURI` implementation should not escape them, which is demonstrated in the `encodeURI()` example.

```
function encodeRFC3986URIComponent(str) {
  return encodeURIComponent(str).replace(
    /[!'()*]/g,
    (c) => `%${c.charCodeAt(0).toString(16).toUpperCase()}`,
  );
}
```

Encoding a lone surrogate throws

A `URIError` will be thrown if one attempts to encode a surrogate which is not part of a high-low pair. For example:

```
// High-low pair OK
encodeURIComponent("\uD800\uDFFF"); // "%F0%90%8F%BF"

// Lone high-surrogate code unit throws "URIError: malformed URI sequence"
encodeURIComponent("\uD800");

// Lone high-surrogate code unit throws "URIError: malformed URI sequence"
encodeURIComponent("\uDFFF");
```

You can use `String.prototype.toWellFormed()`, which replaces lone surrogates with the Unicode replacement character (U+FFFD), to avoid this error. You can also use `String.prototype.isWellFormed()` to check if a string contains lone surrogates before passing it to `encodeURIComponent()`.

See also

- `decodeURI()`
- `encodeURI()`
- `decodeURIComponent()`

Error

`Error` objects are thrown when runtime errors occur. The `Error` object can also be used as a base object for user-defined exceptions. See below for standard built-in error types.

Description

Runtime errors result in new `Error` objects being created and thrown.

`Error` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

Error types

Besides the generic `Error` constructor, there are other core error constructors in JavaScript. For client-side exceptions, see Exception handling statements.

`EvalError`

: Creates an instance representing an error that occurs regarding the global function `eval()`.

`RangeError`

: Creates an instance representing an error that occurs when a numeric variable or parameter is outside its valid range.

`ReferenceError`

: Creates an instance representing an error that occurs when de-referencing an invalid reference.

`SyntaxError`

: Creates an instance representing a syntax error.

`TypeError`

: Creates an instance representing an error that occurs when a variable or parameter is not of a valid type.

`URIError`

: Creates an instance representing an error that occurs when `encodeURI()` or `decodeURI()` are passed invalid parameters.

`AggregateError`

Creates an instance representing several errors wrapped in a single error when multiple errors need to be reported by an operation, for example by `Promise.any()`.

`InternalError`

Creates an instance representing an error that occurs when an internal error in the JavaScript engine is thrown. E.g. "too much recursion".

Constructor

`Error()`

Creates a new `Error` object.

Static properties

`Error.stackTraceLimit`

A non-standard numerical property that limits how many stack frames to include in an error stack trace.

Static methods

`Error.captureStackTrace()`

A non-standard function that creates the `stack` property on the provided object.

`Error.isError()`

Returns `true` if the argument is an error, or `false` otherwise.

`Error.prepareStackTrace()` Optional

A non-standard function that, if provided by user code, is called by the JavaScript engine for thrown exceptions, allowing the user to provide custom formatting for stack traces. See the V8 Stack Trace API docs.

Instance properties

These properties are defined on `Error.prototype` and shared by all `Error` instances.

`Error.prototype.constructor`

The constructor function that created the instance object. For `Error` instances, the initial value is the `Error` constructor.

`Error.prototype.name`

Represents the name for the type of error. For `Error.prototype.name`, the initial value is `"Error"`. Subclasses like `TypeError` and `SyntaxError` provide their own `name` properties.

`Error.prototype.stack`

A non-standard property for a stack trace.

These properties are own properties of each `Error` instance.

`cause`

Error cause indicating the reason why the current error is thrown — usually another caught error. For user-created `Error` objects, this is the value provided as the `cause` property of the constructor's second argument.

`columnNumber`

A non-standard Mozilla property for the column number in the line that raised this error.

`fileName`

A non-standard Mozilla property for the path to the file that raised this error.

`lineNumber`

A non-standard Mozilla property for the line number in the file that raised this error.

`message`

Error message. For user-created `Error` objects, this is the string provided as the constructor's first argument.

Instance methods

`Error.prototype.toString()`

Returns a string representing the specified object. Overrides the `Object.prototype.toString()` method.

Examples

Throwing a generic error

Usually you create an `Error` object with the intention of raising it using the `throw` keyword. You can handle the error using the `try...catch` construct:

```
try {
  throw new Error("Whoops!");
} catch (e) {
  console.error(`${e.name}: ${e.message}`);
}
```

Handling a specific error type

You can choose to handle only specific error types by testing the error type with the `instanceof` keyword:

```
try {
  foo.bar();
} catch (e) {
  if (e instanceof EvalError) {
    console.error(`${e.name}: ${e.message}`);
  } else if (e instanceof RangeError) {
    console.error(`${e.name}: ${e.message}`);
  }
  // etc.
  else {
    // If none of our cases matched leave the Error unhandled
    throw e;
  }
}
```

Differentiate between similar errors

Sometimes a block of code can fail for reasons that require different handling, but which throw very similar errors (i.e., with the same type and message).

If you don't have control over the original errors that are thrown, one option is to catch them and throw new `Error` objects that have more specific messages. The original error should be passed to the new `Error` in the constructor's `options` parameter as its `cause` property. This ensures that the original error and stack trace are available to higher-level try/catch blocks.

The example below shows this for two methods that would otherwise fail with similar errors (`doFailSomeWay()` and `doFailAnotherWay()`):

```
function doWork() {
  try {
    doFailSomeWay();
  } catch (err) {
    throw new Error("Failed in some way", { cause: err });
  }
  try {
    doFailAnotherWay();
  } catch (err) {
    throw new Error("Failed in another way", { cause: err });
  }
}

try {
  doWork();
} catch (err) {
  switch (err.message) {
    case "Failed in some way":
      handleFailSomeWay(err.cause);
      break;
    case "Failed in another way":
      handleFailAnotherWay(err.cause);
      break;
  }
}
```

> **Note:** If you are making a library, you should prefer to use error cause to discriminate between different errors emitted — rather than asking your consumers to parse the error message. See the error cause page for an example.

Custom error types can also use the `cause` property, provided the subclasses' constructor passes the `options` parameter when calling `super()`. The `Error()` base class constructor will read `options.cause` and define the `cause` property on the new error instance.

```
class MyError extends Error {
  constructor(message, options) {
    // Need to pass `options` as the second parameter to install the "cause" property.
    super(message, options);
  }
}

console.log(new MyError("test", { cause: new Error("cause") }).cause);
// Error: cause
```

Custom error types

You might want to define your own error types deriving from `Error` to be able to `throw new MyError()` and use `instanceof MyError` to check the kind of error in the exception handler. This results in cleaner and more consistent error handling code.

See "What's a good way to extend Error in JavaScript?" on Stack Overflow for an in-depth discussion.

> **Warning:** Builtin subclassing cannot be reliably transpiled to pre-ES6 code, because there's no way to construct the base class with a particular `new.target` without `Reflect.construct()`. You need additional configuration or manually call `Object.setPrototypeOf(this, CustomError.prototype)` at the end of the constructor; otherwise, the constructed instance will not be a `CustomError` instance. See the TypeScript FAQ for more information.

> **Note:** Some browsers include the `CustomError` constructor in the stack trace when using ES2015 classes.

```
class CustomError extends Error {
  constructor(foo = "bar", ...params) {
    // Pass remaining arguments (including vendor specific ones) to parent constructor
    super(...params);

    // Maintains proper stack trace for where our error was thrown (non-standard)
    if (Error.captureStackTrace) {
      Error.captureStackTrace(this, CustomError);
    }

    this.name = "CustomError";
    // Custom debugging information
    this.foo = foo;
    this.date = new Date();
  }
}

try {
  throw new CustomError("baz", "bazMessage");
} catch (e) {
  console.error(e.name); // CustomError
  console.error(e.foo); // baz
  console.error(e.message); // bazMessage
  console.error(e.stack); // stack trace
}
```

See also

- Polyfill of `Error` with `cause` support in `core-js`
- es-shims polyfill of Error `cause`
- `throw`
- `try...catch`
- Stack trace API in the V8 docs

Error.captureStackTrace()

The **`Error.captureStackTrace()`** static method installs stack trace information on a provided object as the `stack` property.

Syntax

```
Error.captureStackTrace(object)
Error.captureStackTrace(object, constructor)
```

Parameters

`object`

The object on which to add the `stack` property.

`constructor` Optional

A function, typically the constructor where the `object` was created. When collecting the stack trace, all frames above the topmost call to this function, including that call, are left out of the stack trace.

Return value

None (`undefined`).

The `object` is modified in-place with an extra own property called `stack` defined, whose string value follows the same format as `Error.prototype.stack`. This property is non-enumerable and configurable. In V8, it is a getter-setter pair. In SpiderMonkey and JavaScriptCore, it is a data property that is writable.

Examples

Using Error.captureStackTrace()

The `getStack()` utility function returns the current stack trace at the point it is called, removing itself from the stack. This serves the same debugging purpose as `console.trace()`, but allows you to output the string elsewhere. Note that it does not construct an `Error` instance for this purpose, but installs `stack` on a plain object,

which would be more efficient for our purposes. Normally, you would call `Error.captureStackTrace` on objects intended to be thrown as errors, as shown in the next example.

```
function getStack() {
  const obj = {};
  if ("captureStackTrace" in Error) {
    // Avoid getStack itself in the stack trace
    Error.captureStackTrace(obj, getStack);
  }
  return obj.stack;
}

function foo() {
  console.log(getStack());
}

foo();
// Error
//     at foo (<anonymous>:8:15)
//     at <anonymous>:11:1
```

Installing stack trace on a custom error object

The main use case for `Error.captureStackTrace()` is to install a stack trace on a custom error object. Typically, you define custom errors by extending the `Error` class, which automatically makes the `stack` property available via inheritance. However, the problem with the default stack trace is that it includes the constructor call itself, which leaks implementation details. You can avoid this by using `Error.captureStackTrace()`, which allows the stack trace to be installed even for custom errors that do not inherit from `Error`.

```
class MyError extends Error {
  constructor(message, options) {
    super(message, options);
    if ("captureStackTrace" in Error) {
      // Avoid MyError itself in the stack trace
      Error.captureStackTrace(this, MyError);
    }
  }
}

const myError = new MyError("Something went wrong");
console.log(myError.stack);
// Error: Something went wrong
//     at <anonymous>:8:17
```

Note that even if you don't call `Error.captureStackTrace()` here, some engines are still smart enough to avoid `MyError` in the stack trace if the constructor inherits from `Error`. Calling `Error.captureStackTrace()` is more important for custom errors that, for some reason, do not inherit from `Error`.

```
class MyError {
  constructor(message) {
    this.message = message;
    if ("captureStackTrace" in Error) {
      // Avoid MyError itself in the stack trace
      Error.captureStackTrace(this, MyError);
    }
  }
}

const myError = new MyError("Something went wrong");
console.log(myError.stack);
// Error: Something went wrong
//     at <anonymous>:8:17
```

See also

- `Error.prototype.stack`
- `Error.stackTraceLimit`
- Stack trace API in the V8 docs

Error: cause

The **`cause`** data property of an `Error` instance indicates the specific original cause of the error.

It is used when catching and re-throwing an error with a more-specific or useful error message in order to still have access to the original error.

Value

The value that was passed to the `Error()` constructor in the `options.cause` argument. It may not be present.

Property attributes of `Error: cause`	
Writable	yes
Enumerable	no
Configurable	yes

Description

The value of `cause` can be of any type. You should not make assumptions that the error you caught has an `Error` as its `cause`, in the same way that you cannot be sure the variable bound in the `catch` statement is an `Error` either. The "Providing structured data as the error cause" example below shows a case where a non-error is deliberately provided as the cause.

The `SuppressedError` and `AggregateError` subclasses both serve the purpose of associating multiple errors. They both represent multiple failing sites: `SuppressedError` represents an error that occurred while handling another error, while `AggregateError` represents a collection of multiple, unrelated errors that occurred during the same operation. The `cause` property represents a single failing site, with the wrapper error only adding context to the cause and not representing an additional failure.

The following is typical usage of `cause`. There is a single failing site, which originates within `mainLogic()`. The `throw new Error()` statement merely wraps that original error to add context and does not constitute an additional failure.

```
try {
  mainLogic();
} catch (err) {
  throw new Error("Main logic failed", { cause: err });
}
```

The following is typical usage of `SuppressedError`. There are two failing sites: one within `mainLogic()` and one within `cleanup()`. The `SuppressedError` instance associates the two errors.

```
try {
  mainLogic();
} catch (primaryError) {
  try {
    cleanup();
  } catch (cleanupError) {
    throw new SuppressedError(
      cleanupError,
      primaryError,
      "Main logic failed; while handling that, cleanup also failed",
    );
  }
}
```

The following is typical usage of `AggregateError`. There are multiple failing sites within `mainLogic()`. The `AggregateError` instance associates all the errors.

```
function mainLogic() {
  const errors = [];
  try {
    operation1();
  } catch (e1) {
    errors.push(e1);
  }
  try {
    operation2();
  } catch (e2) {
    errors.push(e2);
  }
  if (errors.length > 0) {
    throw new AggregateError(errors, "Multiple operations failed");
  }
}
```

Examples

Rethrowing an error with a cause

It is sometimes useful to catch an error and re-throw it with a new message. In this case you should pass the original error into the constructor for the new `Error`, as shown.

```
try {
  connectToDatabase();
} catch (err) {
  throw new Error("Connecting to database failed.", { cause: err });
}
```

For a more detailed example see Error > Differentiate between similar errors.

Providing structured data as the error cause

Error messages written for human consumption may be inappropriate for machine parsing — since they're subject to rewording or punctuation changes that may break any existing parsing written to consume them. So when throwing an error from a function, as an alternative to a human-readable error message, you can instead provide the cause as structured data, for machine parsing.

```
function makeRSA(p, q) {
  if (!Number.isInteger(p) || !Number.isInteger(q)) {
    throw new Error("RSA key generation requires integer inputs.", {
      cause: { code: "NonInteger", values: [p, q] },
    });
  }
```

```
  if (!areCoprime(p, q)) {
    throw new Error("RSA key generation requires two co-prime integers.", {
      cause: { code: "NonCoprime", values: [p, q] },
    });
  }
  // rsa algorithm…
}
```

See also

- `Error.prototype.message`
- `Error.prototype.toString()`

Error: columnNumber

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`columnNumber`** data property of an `Error` instance contains the column number in the line of the file that raised this error.

Value

A positive integer.

Property attributes of `Error: columnNumber`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using columnNumber

```
try {
  throw new Error("Could not parse input");
} catch (err) {
  console.log(err.columnNumber); // 9
}
```

Specifications

Not part of any standard.

See also

- `Error.prototype.stack`
- `Error.prototype.lineNumber`
- `Error.prototype.fileName`

Error() constructor

The **`Error()`** constructor creates `Error` objects.

Syntax

```
new Error()
new Error(message)
new Error(message, options)
new Error(message, fileName)
new Error(message, fileName, lineNumber)

Error()
Error(message)
Error(message, options)
Error(message, fileName)
Error(message, fileName, lineNumber)
```

> **Note:** `Error()` can be called with or without `new`. Both create a new `Error` instance.

Parameters

`message` Optional

A human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A value indicating the specific cause of the error, reflected in the `cause` property. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The path to the file that raised this error, reflected in the `fileName` property. Defaults to the name of the file containing the code that called the `Error()` constructor.

`lineNumber` Optional

The line number within the file on which the error was raised, reflected in the `lineNumber` property. Defaults to the line number containing the `Error()` constructor invocation.

Examples

Function call or new construction

When `Error` is used like a function, that is without `new`, it will return an `Error` object. Therefore, a mere call to `Error` will produce the same output that constructing an `Error` object via the `new` keyword would.

```
const x = Error("I was created using a function call!");

// above has the same functionality as following
const y = new Error('I was constructed via the "new" keyword!');
```

Rethrowing an error with a cause

It is sometimes useful to catch an error and re-throw it with a new message. In this case you should pass the original error into the constructor for the new `Error`, as shown.

```
try {
  frameworkThatCanThrow();
} catch (err) {
  throw new Error("New error message", { cause: err });
}
```

For a more detailed example see Error > Differentiate between similar errors.

Omitting options argument

JavaScript only tries to read `options.cause` if `options` is an object — this avoids ambiguity with the other non-standard `Error(message, fileName, lineNumber)` signature, which requires the second parameter to be a string. If you omit `options`, pass a primitive value as `options`, or pass an object without the `cause` property, then the created `Error` object will have no `cause` property.

```
// Omitting options
const error1 = new Error("Error message");
console.log("cause" in error1); // false

// Passing a primitive value
const error2 = new Error("Error message", "");
console.log("cause" in error2); // false

// Passing an object without a cause property
const error3 = new Error("Error message", { details: "http error" });
console.log("cause" in error3); // false
```

See also

- Polyfill of `Error` with `cause` support in `core-js`
- es-shims polyfill of Error `cause`
- `throw`
- `try...catch`
- Error causes on v8.dev (2021)

Error: fileName

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`fileName`** data property of an `Error` instance contains the path to the file that raised this error.

Value

A string.

Property attributes of `Error: fileName`	
Writable	yes

Property attributes of `Error: fileName`	
Enumerable	no
Configurable	yes

Description

This non-standard property contains the path to the file that raised this error. If called from a debugger context, the Firefox Developer Tools for example, "debugger eval code" is returned.

Examples

Using fileName

```
const e = new Error("Could not parse input");
throw e;
// e.fileName could look like "file:///C:/example.html"
```

Specifications

Not part of any standard.

See also

- `Error.prototype.stack`
- `Error.prototype.columnNumber`
- `Error.prototype.lineNumber`

Error.isError()

The **`Error.isError()`** static method determines whether the passed value is an `Error`.

Syntax

```
Error.isError(value)
```

Parameters

`value`

The value to be checked.

Return value

`true` if `value` is an `Error`; otherwise, `false`.

Description

`Error.isError()` checks if the passed value is an `Error`. It does so by performing a *branded check* for a private field initialized by the `Error()` constructor. This is the same mechanism used by `Array.isArray()`, which is in turn similar to the mechanism used by the `in` operator.

It is a more robust alternative to `instanceof Error` because it avoids false positives and false negatives:

- `Error.isError()` rejects values that aren't actual `Error` instances, even if they have `Error.prototype` in their prototype chain — `instanceof Error` would accept these as it does check the prototype chain.
- `Error.isError()` accepts `Error` objects constructed in another realm — `instanceof Error` returns `false` for these because the identity of the `Error` constructor is different across realms.

`Error.isError()` returns `true` for `DOMException` instances. This is because, although `DOMException` is not specified as a real subclass of `Error` (the `Error` constructor is not the prototype of the `DOMException` constructor), `DOMException` still behaves like `Error` for all branded checking purposes.

Examples

Using Error.isError()

```
// all following calls return true
Error.isError(new Error());
Error.isError(new TypeError());
Error.isError(new DOMException());
try {
  1 + 1n;
} catch (e) {
  console.log(Error.isError(e)); // The operation threw a TypeError, so this returns true
}

// all following calls return false
Error.isError();
Error.isError({});
Error.isError(null);
Error.isError(undefined);
Error.isError(17);
Error.isError("Error");
Error.isError(true);
```

```
Error.isError(false);
// This is not an error, because the object does not have the private field
// initialized by the Error constructor
Error.isError({ __proto__: Error.prototype });
```

instanceof vs. Error.isError()

When checking for `Error` instance, `Error.isError()` is preferred over `instanceof` because it works across realms.

```
const iframe = document.createElement("iframe");
document.body.appendChild(iframe);
const xError = window.frames[window.frames.length - 1].Error;
const error = new xError();

// Correctly checking for Error
Error.isError(error); // true
// The prototype of error is xError.prototype, which is a
// different object from Error.prototype
error instanceof Error; // false
```

Normalizing caught errors

You can use `Error.isError()` to detect if the caught value is an error and normalize it to an error object.

```
try {
  throw "Oops; this is not an Error object";
} catch (e) {
  if (!Error.isError(e)) {
    e = new Error(e);
  }
  console.error(e.message);
}
```

See also

- Polyfill of `Error.isError` in `core-js`
- es-shims polyfill of `Error.isError`
- `Error`

Error: lineNumber

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`lineNumber`** data property of an `Error` instance contains the line number in the file that raised this error.

Value

A positive integer.

Property attributes of `Error: lineNumber`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using lineNumber

```
try {
  throw new Error("Could not parse input");
} catch (err) {
  console.log(err.lineNumber); // 2
}
```

Alternative example using error event

```
window.addEventListener("error", (e) => {
  console.log(e.lineNumber); // 5
});
const e = new Error("Could not parse input");
throw e;
```

This is not a standard feature and lacks widespread support. See the browser compatibility table below.

Specifications

Not part of any standard.

See also

- `Error.prototype.stack`
- `Error.prototype.columnNumber`
- `Error.prototype.fileName`

Error: message

The **`message`** data property of an `Error` instance is a human-readable description of the error.

Value

A string corresponding to the value passed to the `Error()` constructor as the first argument.

Property attributes of `Error: message`	
Writable	yes
Enumerable	no
Configurable	yes

Description

This property contains a brief description of the error if one is available or has been set. The `message` property combined with the `name` property is used by the `Error.prototype.toString()` method to create a string representation of the Error.

By default, the `message` property is an empty string, but this behavior can be overridden for an instance by specifying a message as the first argument to the `Error` constructor.

Examples

Throwing a custom error

```
const e = new Error("Could not parse input");
// e.message is 'Could not parse input'
throw e;
```

See also

- `Error.prototype.name`
- `Error.prototype.toString()`

Error.prototype.name

The **`name`** data property of `Error.prototype` is shared by all `Error` instances. It represents the name for the type of error. For `Error.prototype.name`, the initial value is `"Error"`. Subclasses like `TypeError` and `SyntaxError` provide their own `name` properties.

Value

A string. For `Error.prototype.name`, the initial value is `"Error"`.

Property attributes of `Error.prototype.name`	
Writable	yes
Enumerable	no
Configurable	yes

Description

By default, `Error` instances are given the name "Error". The `name` property, in addition to the `message` property, is used by the `Error.prototype.toString()` method to create a string representation of the error.

Examples

Throwing a custom error

```
const e = new Error("Malformed input"); // e.name is 'Error'

e.name = "ParseError";
throw e;
// e.toString() would return 'ParseError: Malformed input'
```

See also

- `Error.prototype.message`
- `Error.prototype.toString()`

Error.prototype.stack

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may

change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

Note: The `stack` property is de facto implemented by all major JavaScript engines, and the JavaScript standards committee is looking to standardize it. You cannot rely on the precise content of the stack string due to implementation inconsistencies, but you can generally assume it exists and use it for debugging purposes.

The non-standard **`stack`** property of an `Error` instance offers a trace of which functions were called, in what order, from which line and file, and with what arguments. The stack string proceeds from the most recent calls to earlier ones, leading back to the original global scope call.

Value

A string.

Because the `stack` property is non-standard, implementations differ about where it's installed.

- In Firefox, it's an accessor property on `Error.prototype`.
- In Chrome and Safari, it's a data property on each `Error` instance, with the descriptor:

Property attributes of `Error.prototype.stack`	
Writable	yes
Enumerable	no
Configurable	yes

Description

Each JavaScript engine uses its own format for stack traces, but they are fairly consistent in their high-level structure. Every implementation uses a separate line in the stack to represent each function call. The call that directly caused the error is placed at the top, and the call that started the whole call chain is placed at the bottom. Below are some examples of stack traces:

```
function foo() {
  bar();
}

function bar() {
  baz();
}

function baz() {
  console.log(new Error().stack);
}

foo();
```

```
#### JavaScriptCore
baz@filename.js:10:24
bar@filename.js:6:6
foo@filename.js:2:6
global code@filename.js:13:4

#### SpiderMonkey
baz@filename.js:10:15
bar@filename.js:6:3
foo@filename.js:2:3
@filename.js:13:1

#### V8
Error
    at baz (filename.js:10:15)
    at bar (filename.js:6:3)
    at foo (filename.js:2:3)
    at filename.js:13:1
```

V8 provides the non-standard stack trace API for customizing the stack trace, including `Error.captureStackTrace()`, `Error.stackTraceLimit`, and `Error.prepareStackTrace()`. Other engines support this API to varying extents.

Different engines set this value at different times. Most modern engines set it when the `Error` object is created. This means you can get the full call stack information within a function using the following:

```
function foo() {
  console.log(new Error().stack);
}
```

Without having to throw an error and then catch it.

Stack frames can be things other than explicit function calls, too. For example, event listeners, timeout jobs, and promise handlers all begin their own call chain. Source code within `eval()` and `Function` constructor calls also appear in the stack:

```
console.log(new Function("return new Error('Function failed')")().stack);
console.log("====");
```

```
console.log(eval("new Error('eval failed')").stack);
```

```
#### JavaScriptCore
anonymous@
global code@filename.js:1:65
====
eval code@
eval@[native code]
global code@filename.js:3:17

#### SpiderMonkey
anonymous@filename.js line 1 > Function:1:8
@filename.js:1:65

====
@filename.js line 3 > eval:1:1
@filename.js:3:13

#### V8
Error: Function failed
    at eval (eval at <anonymous> (filename.js:1:13), <anonymous>:1:8)
    at filename.js:1:65
====
Error: eval failed
    at eval (eval at <anonymous> (filename.js:3:13), <anonymous>:1:1)
    at filename.js:3:13
```

In Firefox, you can use the `//# sourceURL` directive to name an eval source. See the Firefox Debug eval sources docs.

Examples

Using the stack property

The following script demonstrates how to use the `stack` property to output a stack trace into your browser window. You can use this to check what your browser's stack structure looks like.

```
function trace() {
  throw new Error("trace() failed");
}
function b() {
  trace();
}
function a() {
  b(3, 4, "\n\n", undefined, {});
}
try {
  a("first call, first arg");
} catch (e) {
  document.getElementById("output").textContent = e.stack;
}
```

Specifications

Not part of any standard.

See also

- TraceKit on GitHub
- stacktrace.js on GitHub
- Stack trace API in the V8 docs

Error.stackTraceLimit

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** This feature is part of the currently non-standard V8 stack trace API. However, for compatibility reasons, it is also implemented by JavaScriptCore.

The **`Error.stackTraceLimit`** static data property indicates the maximum number of stack frames captured by the stack trace of an error. It can be set by user code to change the engine's behavior.

Generally, *reading* this property is not very useful, but you can *set* it to a new value. Setting it to a larger value can be useful for debugging, as it allows you to see more of the call stack. Setting it to a smaller value can improve performance as it reduces the amount of stack captured.

Value

An integer representing the maximum number of stack frames captured by the stack trace of an error.

Property attributes of `Error.stackTraceLimit`	
Writable	yes
Enumerable	yes
Configurable	yes

Description

Because `stackTraceLimit` is a static property of `Error`, you always use it as `Error.stackTraceLimit`, rather than as a property of an `Error` object you created. If you want to customize the stack trace for a single error only, you may need to set the property, create the error, and then reset the property to its original value.

Examples

Setting Error.stackTraceLimit

This code is safe to run even in environments that don't support `Error.stackTraceLimit`, because it doesn't read the property, only sets it, and engines that don't support it will ignore the setting.

```
Error.stackTraceLimit = 2;
const a = () => b();
const b = () => c();
const c = () => d();
const d = () => e();
const e = () => {
  throw new Error("My error");
};
try {
  a();
} catch (e) {
  console.log(e.stack);
}
// Only two frames in supporting engines; all frames in others
```

See also

- `Error.prototype.stack`
- `Error.captureStackTrace()`
- Stack trace API in the V8 docs

Error.prototype.toString()

The **`toString()`** method of `Error` instances returns a string representing this error.

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the specified `Error` object.

Description

The `Error` object overrides the `Object.prototype.toString()` method inherited by all objects. Its semantics are as follows:

```
Error.prototype.toString = function () {
  if (
    this === null ||
    (typeof this !== "object" && typeof this !== "function")
  ) {
    throw new TypeError();
  }
  let name = this.name;
  name = name === undefined ? "Error" : `${name}`;
  let msg = this.message;
  msg = msg === undefined ? "" : `${msg}`;
  if (name === "") {
    return msg;
  }
  if (msg === "") {
    return name;
  }
  return `${name}: ${msg}`;
};
```

Examples

Using toString()

```
const e1 = new Error("fatal error");
console.log(e1.toString()); // "Error: fatal error"

const e2 = new Error("fatal error");
e2.name = undefined;
console.log(e2.toString()); // "Error: fatal error"

const e3 = new Error("fatal error");
e3.name = "";
console.log(e3.toString()); // "fatal error"

const e4 = new Error("fatal error");
e4.name = "";
e4.message = undefined;
console.log(e4.toString()); // ""

const e5 = new Error("fatal error");
e5.name = "hello";
e5.message = undefined;
console.log(e5.toString()); // "hello"
```

See also

- Polyfill of `Error.prototype.toString` with many bug fixes in `core-js`

escape()

> **Note:** `escape()` is a non-standard function implemented by browsers and was only standardized for cross-engine compatibility. It is not required to be implemented by all JavaScript engines and may not work everywhere. Use `encodeURIComponent()` or `encodeURI()` if possible.

The **`escape()`** function computes a new string in which certain characters have been replaced by hexadecimal escape sequences.

Syntax

```
escape(str)
```

Parameters

`str`

A string to be encoded.

Return value

A new string in which certain characters have been escaped.

Description

`escape()` is a function property of the global object.

The `escape()` function replaces all characters with escape sequences, with the exception of ASCII word characters (A–Z, a–z, 0–9, _) and `@\*_+-./`. Characters are escaped by UTF-16 code units. If the code unit's value is less than 256, it is represented by a two-digit hexadecimal number in the format `%xx`, left-padded with 0 if necessary. Otherwise, it is represented by a four-digit hexadecimal number in the format `%uxxxx`, left-padded with 0 if necessary.

> **Note:** This function was used mostly for percent-encoding and is partly based on the escape format in RFC 1738. The escape format is *not* an escape sequence in string literals. You can replace `%xx` with `\xxx` and `%uxxxx` with `\uxxxx` to get a string containing actual string-literal escape sequences.

Examples

Using escape()

```
escape("abc123"); // "abc123"
escape("äöü"); // "%E4%F6%FC"
escape("ć"); // "%u0107"

// special characters
escape("@*_+-./"); // "@*_+-./"
```

See also

- Polyfill of `escape` in `core-js`
- `encodeURI`
- `encodeURIComponent`
- `unescape`

eval()

> **Warning:** The argument passed to this function is dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations for more information.

The **`eval()`** function evaluates JavaScript code represented as a string and returns its completion value. The source is parsed as a script.

Try it

```
console.log(eval("2 + 2"));
// Expected output: 4

console.log(eval(new String("2 + 2")));
// Expected output: 2 + 2

console.log(eval("2 + 2") === eval("4"));
// Expected output: true

console.log(eval("2 + 2") === eval(new String("2 + 2")));
// Expected output: false
```

Syntax

```
eval(script)
```

Parameters

`script`

A `TrustedScript` instance or string representing a JavaScript expression, statement, or sequence of statements. The expression can include variables and properties of existing objects. It will be parsed as a script, so `import` declarations (which can only exist in modules) are not allowed.

Return value

The completion value of evaluating the given code. If the completion value is empty, `undefined` is returned. If `script` is not a `TrustedScript` or string primitive, `eval()` returns the argument unchanged.

Exceptions

`SyntaxError`

The `script` parameter cannot be parsed as a script.

`TypeError`

`script` is a string when Trusted Types are enforced by a CSP and no default policy is defined.

The method also throws any exception that occurs during evaluation of the code.

Description

`eval()` is a function property of the global object.

The argument of the `eval()` function is a string. It will evaluate the source string as a script body, which means both statements and expressions are allowed. It returns the completion value of the code. For expressions, it's the value the expression evaluates to. Many statements and declarations have completion values as well, but the result may be surprising (for example, the completion value of an assignment is the assigned value, but the completion value of `let` is undefined), so it's recommended to not rely on statements' completion values.

In strict mode, declaring a variable named `eval` or re-assigning `eval` is a `SyntaxError`.

```
"use strict";

const eval = 1; // SyntaxError: Unexpected eval or arguments in strict mode
```

If the argument of `eval()` is not a `TrustedScript` or string, `eval()` returns the argument unchanged. In the following example, passing a `String` object instead of a primitive causes `eval()` to return the `String` object rather than evaluating the string.

```
eval(new String("2 + 2")); // returns a String object containing "2 + 2"
eval("2 + 2"); // returns 4
```

To work around the issue in a generic fashion, you can coerce the argument to a string yourself before passing it to `eval()`.

```
const expression = new String("2 + 2");
eval(String(expression)); // returns 4
```

Direct and indirect eval

There are two modes of `eval()` calls: *direct* eval and *indirect* eval. Direct eval, as the name implies, refers to *directly* calling the global `eval` function with `eval(...)`. Everything else, including invoking it via an aliased variable, via a member access or other expression, or through the optional chaining `?.` operator, is indirect.

```
// Direct call
eval("x + y");

// Indirect call using the comma operator to return eval
(0, eval)("x + y");

// Indirect call through optional chaining
eval?.("x + y");

// Indirect call using a variable to store and return eval
const myEval = eval;
myEval("x + y");

// Indirect call through member access
const obj = { eval };
obj.eval("x + y");
```

Indirect eval can be seen as if the code is evaluated within a separate `<script>` tag. This means:

- Indirect eval works in the global scope rather than the local scope, and the code being evaluated doesn't have access to local variables within the scope where it's being called.

  ```
  function test() {
    const x = 2;
    const y = 4;
    // Direct call, uses local scope
    console.log(eval("x + y")); // Result is 6
    // Indirect call, uses global scope
    console.log(eval?.("x + y")); // Throws because x is not defined in global scope
  }
  ```

- Indirect `eval` does not inherit the strictness of the surrounding context, and is only in strict mode if the source string itself has a `"use strict"` directive.

  ```
  function nonStrictContext() {
    eval?.(`with (Math) console.log(PI);`);
  }
  function strictContext() {
    "use strict";
    eval?.(`with (Math) console.log(PI);`);
  }
  function strictContextStrictEval() {
    "use strict";
    eval?.(`"use strict"; with (Math) console.log(PI);`);
  }
  nonStrictContext(); // Logs 3.141592653589793
  strictContext(); // Logs 3.141592653589793
  strictContextStrictEval(); // Uncaught SyntaxError: Strict mode code may not include a with statement
  ```

  On the other hand, direct eval inherits the strictness of the invoking context.

  ```
  function nonStrictContext() {
    eval(`with (Math) console.log(PI);`);
  }
  function strictContext() {
    "use strict";
    eval(`with (Math) console.log(PI);`);
  }
  function strictContextStrictEval() {
    "use strict";
    eval(`"use strict"; with (Math) console.log(PI);`);
  }
  nonStrictContext(); // Logs 3.141592653589793
  strictContext(); // Uncaught SyntaxError: Strict mode code may not include a with statement
  strictContextStrictEval(); // Uncaught SyntaxError: Strict mode code may not include a with statement
  ```

- `var`-declared variables and function declarations would go into the surrounding scope if the source string is not interpreted in strict mode — for indirect eval, they become global variables. If it's a direct eval in a strict mode context, or if the `eval` source string itself is in strict mode, then `var` and function declarations do not "leak" into the surrounding scope.

  ```
  // Neither context nor source string is strict,
  // so var creates a variable in the surrounding scope
  eval("var a = 1;");
  console.log(a); // 1
  // Context is not strict, but eval source is strict,
  // so b is scoped to the evaluated script
  eval("'use strict'; var b = 1;");
  console.log(b); // ReferenceError: b is not defined

  function strictContext() {
    "use strict";
  ```

```
  // Context is strict, but this is indirect and the source
  // string is not strict, so c is still global
  eval?.("var c = 1;");
  // Direct eval in a strict context, so d is scoped
  eval("var d = 1;");
}
strictContext();
console.log(c); // 1
console.log(d); // ReferenceError: d is not defined
```

`let` and `const` declarations within the evaluated string are always scoped to that script.

- Direct eval may have access to additional contextual expressions. For example, in a function's body, one can use `new.target`:

```
function Ctor() {
  eval("console.log(new.target)");
}
new Ctor(); // [Function: Ctor]
```

Never use direct eval()!

Using direct `eval()` suffers from multiple problems:

- `eval()` executes the code it's passed with the privileges of the caller. If you run `eval()` with a string that could be affected by a malicious party, you may end up running malicious code on the user's machine with the permissions of your webpage / extension. More importantly, allowing third-party code to access the scope in which `eval()` was invoked (if it's a direct eval) can lead to possible attacks that reads or changes local variables. See Security considerations for approaches that mitigate these risks.
- `eval()` is slower than the alternatives, since it has to invoke the JavaScript interpreter, while many other constructs are optimized by modern JS engines.
- Modern JavaScript interpreters convert JavaScript to machine code. This means that any concept of variable naming gets obliterated. Thus, any use of `eval()` will force the browser to do long expensive variable name lookups to figure out where the variable exists in the machine code and set its value. Additionally, new things can be introduced to that variable through `eval()`, such as changing the type of that variable, forcing the browser to re-evaluate all of the generated machine code to compensate.
- Minifiers give up on any minification if the scope is transitively depended on by `eval()`, because otherwise `eval()` cannot read the correct variable at runtime.

There are many cases where the use of `eval()` or related methods can be optimized or avoided altogether.

Using indirect eval()

Consider this code:

```
function looseJsonParse(obj) {
  return eval(`(${obj})`);
}
console.log(looseJsonParse("{ a: 4 - 1, b: function () {}, c: new Map() }"));
```

Simply using indirect eval and forcing strict mode can make the code much better:

```
function looseJsonParse(obj) {
  return eval?.(`"use strict";(${obj})`);
}
console.log(looseJsonParse("{ a: 4 - 1, b: function () {}, c: new Map() }"));
```

The two code snippets above may seem to work the same way, but they do not; the first one using direct eval suffers from multiple problems.

- It is a great deal slower, due to more scope inspections. Notice `c: new Map()` in the evaluated string. In the indirect eval version, the object is being evaluated in the global scope, so it is safe for the interpreter to assume that `Map` refers to the global `Map()` constructor instead of a local variable called `Map`. However, in the code using direct eval, the interpreter cannot assume this. For example, in the following code, `Map` in the evaluated string doesn't refer to `window.Map()`.

  ```
  function looseJsonParse(obj) {
    class Map {}
    return eval(`(${obj})`);
  }
  console.log(looseJsonParse(`{ a: 4 - 1, b: function () {}, c: new Map() }`));
  ```

  Thus, in the `eval()` version of the code, the browser is forced to make the expensive lookup call to check to see if there are any local variables called `Map()`.

- If not using strict mode, `var` declarations within the `eval()` source becomes variables in the surrounding scope. This leads to hard-to-debug issues if the string is acquired from external input, especially if there's an existing variable with the same name.
- Direct eval can read and mutate bindings in the surrounding scope, which may lead to external input corrupting local data.
- When using direct `eval`, especially when the eval source cannot be proven to be in strict mode, the engine — and build tools — have to disable all optimizations related to inlining, because the `eval()` source can depend on any variable name in its surrounding scope.

However, using indirect `eval()` does not allow passing extra bindings other than existing global variables for the evaluated source to read. If you need to specify additional variables that the evaluated source should have access to, consider using the `Function()` constructor.

Using the Function() constructor

The `Function()` constructor is very similar to the indirect eval example above: it also evaluates the JavaScript source passed to it in the global scope without reading or mutating any local bindings, and therefore allows engines to do more optimizations than direct `eval()`.

The difference between `eval()` and `Function()` is that the source string passed to `Function()` is parsed as a function body, not as a script. There are a few nuances — for example, you can use `return` statements at the top level of a function body, but not in a script.

The `Function()` constructor is useful if you wish to create local bindings within your eval source, by passing the variables as parameter bindings.

```
function add(a, b) {
  return a + b;
}
function runCodeWithAddFunction(obj) {
  return Function("add", `"use strict";return (${obj});`)(add);
}
console.log(runCodeWithAddFunction("add(5, 7)")); // 12
```

Both `eval()` and `Function()` implicitly evaluate arbitrary code, and are forbidden in strict CSP settings. There are also additional safer (and faster!) alternatives to `eval()` or `Function()` for common use-cases.

Using bracket accessors

You should not use `eval()` to access properties dynamically. Consider the following example where the property of the object to be accessed is not known until the code is executed. This can be done with `eval()`:

```
const obj = { a: 20, b: 30 };
const propName = getPropName(); // returns "a" or "b"

const result = eval(`obj.${propName}`);
```

However, `eval()` is not necessary here — in fact, it's more error-prone, because if `propName` is not a valid identifier, it leads to a syntax error. Moreover, if `getPropName` is not a function you control, this may lead to execution of arbitrary code. Instead, use the property accessors, which are much faster and safer:

```
const obj = { a: 20, b: 30 };
const propName = getPropName(); // returns "a" or "b"
const result = obj[propName]; // obj["a"] is the same as obj.a
```

You can even use this method to access descendant properties. Using `eval()`, this would look like:

```
const obj = { a: { b: { c: 0 } } };
const propPath = getPropPath(); // suppose it returns "a.b.c"

const result = eval(`obj.${propPath}`); // 0
```

Avoiding `eval()` here could be done by splitting the property path and looping through the different properties:

```
function getDescendantProp(obj, desc) {
  const arr = desc.split(".");
  while (arr.length) {
    obj = obj[arr.shift()];
  }
  return obj;
}

const obj = { a: { b: { c: 0 } } };
const propPath = getPropPath(); // suppose it returns "a.b.c"
const result = getDescendantProp(obj, propPath); // 0
```

Setting a property that way works similarly:

```
function setDescendantProp(obj, desc, value) {
  const arr = desc.split(".");
  while (arr.length > 1) {
    obj = obj[arr.shift()];
  }
  return (obj[arr[0]] = value);
}

const obj = { a: { b: { c: 0 } } };
const propPath = getPropPath(); // suppose it returns "a.b.c"
const result = setDescendantProp(obj, propPath, 1); // obj.a.b.c is now 1
```

However, beware that using bracket accessors with unconstrained input is not safe either — it may lead to object injection attacks.

Using callbacks

JavaScript has first-class functions, which means you can pass functions as arguments to other APIs, store them in variables and objects' properties, and so on. Many DOM APIs are designed with this in mind, so you can (and should) write:

```
// Instead of setTimeout("…", 1000) use:
setTimeout(() => {
  // …
}, 1000);

// Instead of elt.setAttribute("onclick", "…") use:
elt.addEventListener("click", () => {
```

```
  // …
});
```

Closures are also helpful as a way to create parameterized functions without concatenating strings.

Using JSON

If the string you're calling `eval()` on contains data (for example, an array: `"[1, 2, 3]"`), as opposed to code, you should consider switching to JSON, which allows the string to use a subset of JavaScript syntax to represent data.

Note that since JSON syntax is limited compared to JavaScript syntax, many valid JavaScript literals will not parse as JSON. For example, trailing commas are not allowed in JSON, and property names (keys) in object literals must be enclosed in quotes. Be sure to use a JSON serializer to generate strings that will be later parsed as JSON.

Passing carefully constrained data instead of arbitrary code is a good idea in general. For example, an extension designed to scrape contents of web-pages could have the scraping rules defined in XPath instead of JavaScript code.

Security considerations

The method can be used to execute arbitrary input with the privileges of the caller. If the input is a potentially unsafe string provided by a user, this is a possible vector for Cross-site-scripting (XSS) attacks.

For example, the following code shows how `eval()` might execute `untrustedCode` provided by a user:

```
const untrustedCode = "alert('Potentially evil code!');";
const adder = eval(untrustedCode);
```

Websites with a Content Security Policy (CSP) that specifies `script-src` or `default-src` will prevent such code running by default. If you must allow the scripts to run via `eval()`, you can mitigate the risks by always assigning a `TrustedScript` instance instead of a string, and enforcing trusted types using the `require-trusted-types-for` CSP directive. This ensures that the input is passed through a transformation function.

To allow `eval()` to run, you additionally need to specify the `trusted-types-eval` keyword in your CSP `script-src` directive. The `unsafe-eval` keyword also allows `eval()`, but is much less safe than `trusted-types-eval` because it would allow execution even on browsers that do not support trusted types.

For example, the required CSP for your site might look like this:

```
Content-Security-Policy: require-trusted-types-for 'script'; script-src '<your_allowlist>' 'trusted-types-eval'
```

The behavior of the transformation function implemented in your trusted types policy depends on the specific use case that requires a user-provided script. If possible, you should lock the allowed scripts to exactly the code that you trust to run. If that is not possible, you might allow or block the use of certain functions within the provided input.

Examples

Note that the first example shows how to use the method with trusted types. The other examples omit this step for brevity.

Using TrustedScript

To mitigate the risk of XSS, we should always assign `TrustedScript` instances to the `script` parameter. We also need to do this if we're enforcing trusted types for other reasons and we want to allow some script sources that have been permitted (by `CSP: script-src`).

Trusted types are not yet supported on all browsers, so first we define the trusted types tinyfill. This acts as a transparent replacement for the Trusted Types JavaScript API:

```
if (typeof trustedTypes === "undefined")
  trustedTypes = { createPolicy: (n, rules) => rules };
```

Next we create a `TrustedTypePolicy` that defines a `createScript()` method for transforming input strings into `TrustedScript` instances.

For the purpose of this example, we'll assume that we have a function `transformedScript()` that defines our transformation/filtering logic.

```
const policy = trustedTypes.createPolicy("script-policy", {
  createScript(input) {
    const transformed = transformedScript(input); // Our filter method
    return transformed;
  },
});
```

Then we use the `policy` object to create a `TrustedScript` object from a potentially unsafe input string:

```
// The potentially malicious string
const untrustedScript = "alert('Potentially evil code!');";

// Create a TrustedScriptURL instance using the policy
const trustedScript = policy.createScript(untrustedScript);
```

The `TrustedScript` object can now be passed to `eval()`:

```
eval(trustedScript);
```

Using eval()

In the following code, both of the statements containing `eval()` return 42. The first evaluates the string `"x + y + 1"`; the second evaluates the string `"42"`.

```
const x = 2;
const y = 39;
const z = "42";
eval("x + y + 1"); // 42
eval(z); // 42
```

eval() returns the completion value of statements

`eval()` returns the completion value of statements. For `if`, it would be the last expression or statement evaluated.

```
const str = "if (a) { 1 + 1 } else { 1 + 2 }";
let a = true;
let b = eval(str);

console.log(`b is: ${b}`); // b is: 2

a = false;
b = eval(str);

console.log(`b is: ${b}`); // b is: 3
```

The following example uses `eval()` to evaluate the string `str`. This string consists of JavaScript statements that assign `z` a value of 42 if `x` is five, and assign 0 to `z` otherwise. When the second statement is executed, `eval()` will cause these statements to be performed, and it will also evaluate the set of statements and return the value that is assigned to `z`, because the completion value of an assignment is the assigned value.

```
const x = 5;
const str = `if (x === 5) {
  console.log("z is 42");
  z = 42;
} else {
  z = 0;
}`;

console.log("z is ", eval(str)); // z is 42  z is 42
```

If you assign multiple values then the last value is returned.

```
let x = 5;
const str = `if (x === 5) {
  console.log("z is 42");
  z = 42;
  x = 420;
} else {
  z = 0;
}`;

console.log("x is", eval(str)); // z is 42  x is 420
```

eval() as a string defining function requires "(" and ")" as prefix and suffix

```
// This is a function declaration
const fctStr1 = "function a() {}";
// This is a function expression
const fctStr2 = "(function b() {})";
const fct1 = eval(fctStr1); // return undefined, but `a` is available as a global function now
const fct2 = eval(fctStr2); // return the function `b`
```

See also

- Property accessors
- WebExtensions: Using eval in content scripts

EvalError

The **`EvalError`** object indicates an error regarding the global `eval()` function. This exception is not thrown by JavaScript anymore, however the `EvalError` object remains for compatibility.

`EvalError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`EvalError` is a subclass of `Error`.

Constructor

`EvalError()`

Creates a new `EvalError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `EvalError.prototype` and shared by all `EvalError` instances.

`EvalError.prototype.constructor`

The constructor function that created the instance object. For `EvalError` instances, the initial value is the `EvalError` constructor.

`EvalError.prototype.name`

Represents the name for the type of error. For `EvalError.prototype.name`, the initial value is `"EvalError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Creating an EvalError

```
try {
  throw new EvalError("Hello");
} catch (e) {
  console.log(e instanceof EvalError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "EvalError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`
- `eval()`

EvalError() constructor

The **`EvalError()`** constructor creates `EvalError` objects.

Syntax

```
new EvalError()
new EvalError(message)
new EvalError(message, options)
new EvalError(message, fileName)
new EvalError(message, fileName, lineNumber)

EvalError()
EvalError(message)
EvalError(message, options)
EvalError(message, fileName)
EvalError(message, fileName, lineNumber)
```

> **Note:** `EvalError()` can be called with or without `new`. Both create a new `EvalError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception

`lineNumber` Optional

The line number of the code that caused the exception

Examples

`EvalError` is not used in the current ECMAScript specification and will thus not be thrown by the runtime. However, the object itself remains for backwards compatibility with earlier versions of the specification.

Creating an EvalError

```
try {
  throw new EvalError("Hello");
} catch (e) {
  console.log(e instanceof EvalError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "EvalError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`
- `eval()`

FinalizationRegistry

A **`FinalizationRegistry`** object lets you request a callback when a value is garbage-collected.

Description

`FinalizationRegistry` provides a way to request that a *cleanup callback* get called at some point when a value registered with the registry has been *reclaimed* (garbage-collected). (Cleanup callbacks are sometimes called *finalizers*.)

> **Note:** Cleanup callbacks should not be used for essential program logic. See Notes on cleanup callbacks for details.

You create the registry passing in the callback:

```
const registry = new FinalizationRegistry((heldValue) => {
  // …
});
```

Then you register any value you want a cleanup callback for by calling the `register` method, passing in the value and a *held value* for it:

```
registry.register(target, "some value");
```

The registry does not keep a strong reference to the value, as that would defeat the purpose (if the registry held it strongly, the value would never be reclaimed). In JavaScript, objects and non-registered symbols are garbage collectable, so they can be registered in a `FinalizationRegistry` object as the target or the token.

If `target` is reclaimed, your cleanup callback may be called at some point with the *held value* you provided for it (`"some value"` in the above). The held value can be any value you like: a primitive or an object, even `undefined`. If the held value is an object, the registry keeps a *strong* reference to it (so it can pass it to your cleanup callback later).

If you might want to unregister a registered target value later, you pass a third value, which is the *unregistration token* you'll use later when calling the registry's `unregister` function to unregister the value. The registry only keeps a weak reference to the unregister token.

It's common to use the target value itself as the unregister token, which is just fine:

```
registry.register(target, "some value", target);
// …

// some time later, if you don't care about `target` anymore, unregister it
registry.unregister(target);
```

It doesn't have to be the same value, though; it can be a different one:

```
registry.register(target, "some value", token);
// …

// some time later
registry.unregister(token);
```

Avoid where possible

Correct use of `FinalizationRegistry` takes careful thought, and it's best avoided if possible. It's also important to avoid relying on any specific behaviors not guaranteed by the specification. When, how, and whether garbage collection occurs is down to the implementation of any given JavaScript engine. Any behavior you observe in one engine may be different in another engine, in another version of the same engine, or even in a slightly different situation with the same version of the same engine. Garbage collection is a hard problem that JavaScript engine implementers are constantly refining and improving their solutions to.

Here are some specific points included by the authors in the proposal that introduced `FinalizationRegistry`:

> Garbage collectors are complicated. If an application or library depends on GC cleaning up a WeakRef or calling a finalizer [cleanup callback] in a timely, predictable manner, it's likely to be disappointed: the cleanup may happen much later than expected, or not at all. Sources of variability include:
>
> - One object might be garbage-collected much sooner than another object, even if they become unreachable at the same time, e.g., due to generational collection.
> - Garbage collection work can be split up over time using incremental and concurrent techniques.
> - Various runtime heuristics can be used to balance memory usage, responsiveness.
> - The JavaScript engine may hold references to things which look like they are unreachable (e.g., in closures, or inline caches).

> - Different JavaScript engines may do these things differently, or the same engine may change its algorithms across versions.
> - Complex factors may lead to objects being held alive for unexpected amounts of time, such as use with certain APIs.

Notes on cleanup callbacks

- Developers shouldn't rely on cleanup callbacks for essential program logic. Cleanup callbacks may be useful for reducing memory usage across the course of a program, but are unlikely to be useful otherwise.
- If your code has just registered a value to the registry, that target will not be reclaimed until the end of the current JavaScript job. See notes on WeakRefs for details.
- A conforming JavaScript implementation, even one that does garbage collection, is not required to call cleanup callbacks. When and whether it does so is entirely down to the implementation of the JavaScript engine. When a registered object is reclaimed, any cleanup callbacks for it may be called then, or some time later, or not at all.
- It's likely that major implementations will call cleanup callbacks at some point during execution, but those calls may be substantially after the related object was reclaimed. Furthermore, if there is an object registered in two registries, there is no guarantee that the two callbacks are called next to each other — one may be called and the other never called, or the other may be called much later.
- There are also situations where even implementations that normally call cleanup callbacks are unlikely to call them:
  - When the JavaScript program shuts down entirely (for instance, closing a tab in a browser).
  - When the `FinalizationRegistry` instance itself is no longer reachable by JavaScript code.
- If the target of a `WeakRef` is also in a `FinalizationRegistry`, the `WeakRef`'s target is cleared at the same time or before any cleanup callback associated with the registry is called; if your cleanup callback calls `deref` on a `WeakRef` for the object, it will receive `undefined`.

Constructor

`FinalizationRegistry()`

Creates a new `FinalizationRegistry` object.

Instance properties

These properties are defined on `FinalizationRegistry.prototype` and shared by all `FinalizationRegistry` instances.

`FinalizationRegistry.prototype.constructor`

The constructor function that created the instance object. For `FinalizationRegistry` instances, the initial value is the `FinalizationRegistry` constructor.

`FinalizationRegistry.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"FinalizationRegistry"`. This property is used in `Object.prototype.toString()`.

Instance methods

`FinalizationRegistry.prototype.register()`

Registers an object with the registry in order to get a cleanup callback when/if the object is garbage-collected.

`FinalizationRegistry.prototype.unregister()`

Unregisters an object from the registry.

Examples

Creating a new registry

You create the registry passing in the callback:

```
const registry = new FinalizationRegistry((heldValue) => {
  // …
});
```

Registering objects for cleanup

Then you register any objects you want a cleanup callback for by calling the `register` method, passing in the object and a *held value* for it:

```
registry.register(theObject, "some value");
```

Callbacks never called synchronously

No matter how much pressure you put on the garbage collector, the cleanup callback will never be called synchronously. The object may be reclaimed synchronously, but the callback will always be called sometime after the current job finishes:

```
let counter = 0;
const registry = new FinalizationRegistry(() => {
  console.log(`Array gets garbage collected at ${counter}`);
});

registry.register(["foo"]);

(function allocateMemory() {
  // Allocate 50000 functions — a lot of memory!
  Array.from({ length: 50000 }, () => () => {});
  if (counter > 5000) return;
```

```
  counter++;
  allocateMemory();
})();

console.log("Main job ends");
// Logs:
// Main job ends
// Array gets garbage collected at 5001
```

However, if you allow a little break between each allocation, the callback may be called sooner:

```
let arrayCollected = false;
let counter = 0;
const registry = new FinalizationRegistry(() => {
  console.log(`Array gets garbage collected at ${counter}`);
  arrayCollected = true;
});

registry.register(["foo"]);

(function allocateMemory() {
  // Allocate 50000 functions — a lot of memory!
  Array.from({ length: 50000 }, () => () => {});
  if (counter > 5000 || arrayCollected) return;
  counter++;
  // Use setTimeout to make each allocateMemory a different job
  setTimeout(allocateMemory);
})();

console.log("Main job ends");
```

There's no guarantee that the callback will be called sooner or if it will be called at all, but there's a possibility that the logged message has a counter value smaller than 5000.

See also

- `WeakRef`
- `WeakSet`
- `WeakMap`

FinalizationRegistry() constructor

The **`FinalizationRegistry()`** constructor creates `FinalizationRegistry` objects.

Syntax

```
new FinalizationRegistry(callbackFn)
```

> **Note:** `FinalizationRegistry()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`callback`

A function to be invoked each time a registered target value is garbage collected. Its return value is ignored. The function is called with the following arguments:

`heldValue`

The value that was passed to the second parameter of the `register()` method when the `target` object was registered.

Examples

Creating a new registry

You create the registry passing in the callback:

```
const registry = new FinalizationRegistry((heldValue) => {
  // …
});
```

See also

- `FinalizationRegistry`

FinalizationRegistry.prototype.register()

The **`register()`** method of `FinalizationRegistry` instances registers a value with this `FinalizationRegistry` so that if the value is garbage-collected, the registry's callback may get called.

Syntax

```
register(target, heldValue)
register(target, heldValue, unregisterToken)
```

Parameters

`target`

The target value to register.

`heldValue`

The value to pass to the finalizer for this `target`. This cannot be the `target` itself but can be anything else, including functions and primitives.

`unregisterToken` Optional

A token that may be used with the `unregister` method later to unregister the target value. If provided (and not `undefined`), this must be an object or a non-registered symbol. If not provided, the target cannot be unregistered.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown in one of the following cases:

- `target` is not an object or a non-registered symbol (object as opposed to primitives; functions are objects as well)
- `target` is the same as `heldValue` (`target === heldValue`)
- `unregisterToken` is not an object or a non-registered symbol

Description

See the Avoid where possible and Notes on cleanup callbacks sections of the `FinalizationRegistry` page for important caveats.

Examples

Using register

The following registers the value referenced by `target`, passing in the held value `"some value"` and passing the target itself as the unregistration token:

```
registry.register(target, "some value", target);
```

The following registers the value referenced by `target`, passing in another object as the held value, and not passing in any unregistration token (which means `target` can't be unregistered):

```
registry.register(target, { useful: "info about target" });
```

See also

- `FinalizationRegistry`

FinalizationRegistry.prototype.unregister()

The **`unregister()`** method of `FinalizationRegistry` instances unregisters a target value from this `FinalizationRegistry`.

Syntax

```
unregister(unregisterToken)
```

Parameters

`unregisterToken`

The token used with the `register()` method when registering the target value. Multiple cells registered with the same `unregisterToken` will be unregistered together.

Return value

A boolean value that is `true` if at least one cell was unregistered and `false` if no cell was unregistered.

Exceptions

`TypeError`

Thrown if `unregisterToken` is not an object or a non-registered symbol.

Description

When a target value has been reclaimed, it is no longer registered in the registry. There is no need to call `unregister` in your cleanup callback. Only call `unregister` if you haven't received a cleanup callback and no longer need to receive one.

Examples

Using unregister

This example shows registering a target object using that same object as the unregister token, then later unregistering it via `unregister`:

```
class Thingy {
  static #cleanup = (label) => {
    //              ^^^^^----- held value
    console.error(
      `The "release" method was never called for the object with the label "${label}"`,
    );
  };
  #registry = new FinalizationRegistry(Thingy.#cleanup);

  /**
   * Constructs a `Thingy` instance.
   * Be sure to call `release` when you're done with it.
   *
   * @param label A label for the `Thingy`.
   */
  constructor(label) {
    //                            vvvvv----- held value
    this.#registry.register(this, label, this);
    //          target -----^^^^         ^^^^----- unregister token
  }

  /**
   * Releases resources held by this `Thingy` instance.
   */
  release() {
    this.#registry.unregister(this);
    //                       ^^^^----- unregister token
  }
}
```

This example shows registering a target object using a different object as its unregister token:

```
class Thingy {
  static #cleanup = (file) => {
    //              ^^^^----- held value
    console.error(
      `The "release" method was never called for the "Thingy" for the file "${file.name}"`,
    );
  };
  #registry = new FinalizationRegistry(Thingy.#cleanup);
  #file;

  /**
   * Constructs a `Thingy` instance for the given file.
   * Be sure to call `release` when you're done with it.
   *
   * @param filename The name of the file.
   */
  constructor(filename) {
    this.#file = File.open(filename);
    //                            vvvvv----- held value
    this.#registry.register(this, label, this.#file);
    //          target -----^^^^         ^^^^^^^^^^----- unregister token
  }

  /**
   * Releases resources held by this `Thingy` instance.
   */
  release() {
    if (this.#file) {
      this.#registry.unregister(this.#file);
      //                       ^^^^^^^^^^----- unregister token
      File.close(this.#file);
      this.#file = null;
    }
  }
}
```

See also

- `FinalizationRegistry`

Float16Array

The **`Float16Array`** typed array represents an array of 16-bit floating point numbers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Float16Array` is a subclass of the hidden `TypedArray` class.

> **Note:** Float16 support is not universal, both in the JavaScript API and the underlying CPU architecture. Using it may result in slower performance on some platforms. It is intended for interacting with highly optimized and performance-sensitive systems such as float-backed canvases, WebGPU, WebGL, and deep learning models including stable diffusion.

Constructor

`Float16Array()`

Creates a new `Float16Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Float16Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of `Float16Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Float16Array.prototype` and shared by all `Float16Array` instances.

`Float16Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of a `Float16Array`.

`Float16Array.prototype.constructor`

The constructor function that created the instance object. For `Float16Array` instances, the initial value is the `Float16Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Float16Array

```
// From a length
const float16 = new Float16Array(2);
float16[0] = 42;
console.log(float16[0]); // 42
console.log(float16.length); // 2
console.log(float16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Float16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float16Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float16FromIterable = new Float16Array(iterable);
console.log(float16FromIterable);
// Float16Array [1, 2, 3]
```

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float16Array() constructor

The **`Float16Array()`** constructor creates `Float16Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Float16Array()
new Float16Array(length)
new Float16Array(typedArray)
new Float16Array(object)

new Float16Array(buffer)
new Float16Array(buffer, byteOffset)
new Float16Array(buffer, byteOffset, length)
```

> **Note:** `Float16Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Float16Array

```
// From a length
const float16 = new Float16Array(2);
float16[0] = 42;
console.log(float16[0]); // 42
console.log(float16.length); // 2
console.log(float16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Float16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float16Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float16FromIterable = new Float16Array(iterable);
console.log(float16FromIterable);
// Float16Array [1, 2, 3]
```

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float32Array

The **`Float32Array`** typed array represents an array of 32-bit floating point numbers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Float32Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Float32Array()`

Creates a new `Float32Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Float32Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of `Float32Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Float32Array.prototype` and shared by all `Float32Array` instances.

`Float32Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of a `Float32Array`.

`Float32Array.prototype.constructor`

The constructor function that created the instance object. For `Float32Array` instances, the initial value is the `Float32Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Float32Array

```
// From a length
const float32 = new Float32Array(2);
float32[0] = 42;
console.log(float32[0]); // 42
console.log(float32.length); // 2
console.log(float32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Float32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float32FromIterable = new Float32Array(iterable);
console.log(float32FromIterable);
// Float32Array [1, 2, 3]
```

See also

- Polyfill of `Float32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float32Array() constructor

The **`Float32Array()`** constructor creates `Float32Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Float32Array()
new Float32Array(length)
new Float32Array(typedArray)
new Float32Array(object)

new Float32Array(buffer)
new Float32Array(buffer, byteOffset)
new Float32Array(buffer, byteOffset, length)
```

> **Note:** `Float32Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Float32Array

```
// From a length
const float32 = new Float32Array(2);
float32[0] = 42;
console.log(float32[0]); // 42
console.log(float32.length); // 2
console.log(float32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Float32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float32FromIterable = new Float32Array(iterable);
console.log(float32FromIterable);
// Float32Array [1, 2, 3]
```

See also

- Polyfill of `Float32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float64Array

The **`Float64Array`** typed array represents an array of 64-bit floating point numbers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Float64Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Float64Array()`

Creates a new `Float64Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Float64Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of `Float64Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Float64Array.prototype` and shared by all `Float64Array` instances.

`Float64Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of a `Float64Array`.

`Float64Array.prototype.constructor`

The constructor function that created the instance object. For `Float64Array` instances, the initial value is the `Float64Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Float64Array

```
// From a length
const float64 = new Float64Array(2);
float64[0] = 42;
console.log(float64[0]); // 42
console.log(float64.length); // 2
console.log(float64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new Float64Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float64Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new Float64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float64FromIterable = new Float64Array(iterable);
console.log(float64FromIterable);
// Float64Array [1, 2, 3]
```

See also

- Polyfill of `Float64Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float64Array() constructor

The **`Float64Array()`** constructor creates `Float64Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Float64Array()
new Float64Array(length)
new Float64Array(typedArray)
new Float64Array(object)

new Float64Array(buffer)
new Float64Array(buffer, byteOffset)
new Float64Array(buffer, byteOffset, length)
```

> **Note:** `Float64Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Float64Array

```
// From a length
const float64 = new Float64Array(2);
float64[0] = 42;
console.log(float64[0]); // 42
console.log(float64.length); // 2
console.log(float64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new Float64Array([21, 31]);
```

```
console.log(x[1]); // 31

// From another TypedArray
const y = new Float64Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new Float64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float64FromIterable = new Float64Array(iterable);
console.log(float64FromIterable);
// Float64Array [1, 2, 3]
```

See also

- Polyfill of `Float64Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Function

The **`Function`** object provides methods for functions. In JavaScript, every function is actually a `Function` object.

Constructor

`Function()`

Creates a new `Function` object. Calling the constructor directly can create functions dynamically but suffers from security and similar (but far less significant) performance issues to `eval()`. However, unlike `eval()`, the `Function` constructor creates functions that execute in the global scope only.

Instance properties

These properties are defined on `Function.prototype` and shared by all `Function` instances.

`Function.prototype.arguments`

Represents the arguments passed to this function. For strict, arrow, async, and generator functions, accessing the `arguments` property throws a `TypeError`. Use the `arguments` object inside function closures instead.

`Function.prototype.caller`

Represents the function that invoked this function. For strict, arrow, async, and generator functions, accessing the `caller` property throws a `TypeError`.

`Function.prototype.constructor`

The constructor function that created the instance object. For `Function` instances, the initial value is the `Function` constructor.

These properties are own properties of each `Function` instance.

`displayName` Optional

The display name of the function.

`length`

Specifies the number of arguments expected by the function.

`name`

The name of the function.

`prototype`

Used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

Instance methods

`Function.prototype.apply()`

Calls a function with a given `this` value and optional arguments provided as an array (or an array-like object).

`Function.prototype.bind()`

Creates a new function that, when called, has its `this` keyword set to a provided value, optionally with a given sequence of arguments preceding any provided when the new function is called.

`Function.prototype.call()`

Calls a function with a given `this` value and optional arguments.

`Function.prototype.toString()`

Returns a string representing the source code of the function. Overrides the `Object.prototype.toString` method.

`Function.prototype[Symbol.hasInstance]()`

Specifies the default procedure for determining if a constructor function recognizes an object as one of the constructor's instances. Called by the `instanceof` operator.

Examples

Difference between Function constructor and function declaration

Functions created with the `Function` constructor do not create closures to their creation contexts; they always are created in the global scope. When running them, they will only be able to access their own local variables and global ones, not the ones from the scope in which the `Function` constructor was created. This is different from using `eval()` with code for a function expression.

```
// Create a global property with `var`
var x = 10;

function createFunction1() {
  const x = 20;
  return new Function("return x;"); // this `x` refers to global `x`
}

function createFunction2() {
  const x = 20;
  function f() {
    return x; // this `x` refers to the local `x` above
  }
  return f;
}

const f1 = createFunction1();
console.log(f1()); // 10
const f2 = createFunction2();
console.log(f2()); // 20
```

While this code works in web browsers, `f1()` will produce a `ReferenceError` in Node.js, as `x` will not be found. This is because the top-level scope in Node is not the global scope, and `x` will be local to the module.

See also

- `function`
- `function` expression
- `AsyncFunction`
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Functions

Function.prototype.apply()

The **`apply()`** method of `Function` instances calls this function with a given `this` value, and `arguments` provided as an array (or an array-like object).

Try it

```
const numbers = [5, 6, 2, 3, 7];

const max = Math.max.apply(null, numbers);

console.log(max);
// Expected output: 7

const min = Math.min.apply(null, numbers);

console.log(min);
// Expected output: 2
```

Syntax

```
apply(thisArg)
apply(thisArg, argsArray)
```

Parameters

`thisArg`

The value of `this` provided for the call to `func`. If the function is not in strict mode, `null` and `undefined` will be replaced with the global object, and primitive values will be converted to objects.

`argsArray` Optional

An array-like object, specifying the arguments with which `func` should be called, or `null` or `undefined` if no arguments should be provided to the function.

Return value

The result of calling the function with the specified `this` value and arguments.

Description

> **Note:** This function is almost identical to `call()`, except that the function arguments are passed to `call()` individually as a list, while for `apply()` they are combined in one object, typically an array — for example, `func.call(this, "eat", "bananas")` vs. `func.apply(this, ["eat", "bananas"])`.

Normally, when calling a function, the value of `this` inside the function is the object that the function was accessed on. With `apply()`, you can assign an arbitrary value as `this` when calling an existing function, without first attaching the function to the object as a property. This allows you to use methods of one object as generic utility functions.

You can also use any kind of object which is array-like as the second parameter. In practice, this means that it needs to have a `length` property, and integer ("index") properties in the range `(0..length - 1)`. For example, you could use a `NodeList`, or a custom object like `{ 'length': 2, '0': 'eat', '1': 'bananas' }`. You can also use `arguments`, for example:

```
function wrapper() {
  return anotherFn.apply(null, arguments);
}
```

With the rest parameters and parameter spread syntax, this can be rewritten as:

```
function wrapper(...args) {
  return anotherFn(...args);
}
```

In general, `fn.apply(null, args)` is equivalent to `fn(...args)` with the parameter spread syntax, except `args` is expected to be an array-like object in the former case with `apply()`, and an iterable object in the latter case with spread syntax.

> **Warning:** Do not use `apply()` to chain constructors (for example, to implement inheritance). This invokes the constructor function as a plain function, which means `new.target` is `undefined`, and classes throw an error because they can't be called without `new`. Use `Reflect.construct()` or `extends` instead.

Examples

Using apply() to append an array to another

You can use `Array.prototype.push()` to append an element to an array. Because `push()` accepts a variable number of arguments, you can also push multiple elements at once. But if you pass an array to `push()`, it will actually add that array as a single element, instead of adding the elements individually, ending up with an array inside an array. On the other hand, `Array.prototype.concat()` does have the desired behavior in this case, but it does not append to the *existing* array — it creates and returns a new array.

In this case, you can use `apply` to implicitly "spread" an array as a series of arguments.

```
const array = ["a", "b"];
const elements = [0, 1, 2];
array.push.apply(array, elements);
console.info(array); // ["a", "b", 0, 1, 2]
```

The same effect can be achieved with the spread syntax.

```
const array = ["a", "b"];
const elements = [0, 1, 2];
array.push(...elements);
console.info(array); // ["a", "b", 0, 1, 2]
```

Using apply() and built-in functions

Clever usage of `apply()` allows you to use built-in functions for some tasks that would probably otherwise require manually looping over a collection (or using the spread syntax).

For example, we can use `Math.max()` and `Math.min()` to find out the maximum and minimum value in an array.

```
// min/max number in an array
const numbers = [5, 6, 2, 3, 7];

// using Math.min/Math.max apply
let max = Math.max.apply(null, numbers);
// This about equal to Math.max(numbers[0], …)
// or Math.max(5, 6, …)

let min = Math.min.apply(null, numbers);

// vs. loop based algorithm
max = -Infinity;
min = Infinity;
```

```
for (const n of numbers) {
  if (n > max) {
    max = n;
  }
  if (n < min) {
    min = n;
  }
}
```

But beware: by using `apply()` (or the spread syntax) with an arbitrarily long arguments list, you run the risk of exceeding the JavaScript engine's argument length limit.

The consequences of calling a function with too many arguments (that is, more than tens of thousands of arguments) is unspecified and varies across engines. (The JavaScriptCore engine has a hard-coded argument limit of 65536.) Most engines throw an exception; but there's no normative specification preventing other behaviors, such as arbitrarily limiting the number of arguments actually passed to the applied function. To illustrate this latter case: if such an engine had a limit of four arguments (actual limits are of course significantly higher), it would be as if the arguments `5, 6, 2, 3` had been passed to `apply` in the examples above, rather than the full array.

If your value array might grow into the tens of thousands, use a hybrid strategy: apply your function to chunks of the array at a time:

```
function minOfArray(arr) {
  let min = Infinity;
  const QUANTUM = 32768;

  for (let i = 0; i < arr.length; i += QUANTUM) {
    const subMin = Math.min.apply(
      null,
      arr.slice(i, Math.min(i + QUANTUM, arr.length)),
    );
    min = Math.min(subMin, min);
  }

  return min;
}

const min = minOfArray([5, 6, 2, 3, 7]);
```

See also

- `arguments`
- `Function.prototype.bind()`
- `Function.prototype.call()`
- `Reflect.apply()`
- Functions
- Spread syntax (`...`)

Function.prototype.arguments

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** The `arguments` property of `Function` objects is deprecated. The recommended way to access the `arguments` object is to refer to the variable `arguments` available within functions.

The **`arguments`** accessor property of `Function` instances returns the arguments passed to this function. For strict, arrow, async, and generator functions, accessing the `arguments` property throws a `TypeError`.

Description

The value of `arguments` is an array-like object corresponding to the arguments passed to a function.

In the case of recursion, i.e., if function `f` appears several times on the call stack, the value of `f.arguments` represents the arguments corresponding to the most recent invocation of the function.

The value of the `arguments` property is normally `null` if there is no outstanding invocation of the function in progress (that is, the function has been called but has not yet returned).

Note that the only behavior specified by the ECMAScript specification is that `Function.prototype` has an initial `arguments` accessor that unconditionally throws a `TypeError` for any `get` or `set` request (known as a "poison pill accessor"), and that implementations are not allowed to change this semantic for any function except non-strict plain functions. The actual behavior of the `arguments` property, if it's anything other than throwing an error, is implementation-defined. For example, Chrome defines it as an own data property, while Firefox and Safari extend the initial poison-pill `Function.prototype.arguments` accessor to specially handle `this` values that are non-strict functions.

```
(function f() {
  if (Object.hasOwn(f, "arguments")) {
    console.log(
      "arguments is an own property with descriptor",
      Object.getOwnPropertyDescriptor(f, "arguments"),
    );
```

```
  } else {
    console.log(
      "f doesn't have an own property named arguments. Trying to get f.[[Prototype]].arguments",
    );
    console.log(
      Object.getOwnPropertyDescriptor(
        Object.getPrototypeOf(f),
        "arguments",
      ).get.call(f),
    );
  }
})();

// In Chrome:
// arguments is an own property with descriptor {value: Arguments(0), writable: false, enumerable: false, configurable: false}

// In Firefox:
// f doesn't have an own property named arguments. Trying to get f.[[Prototype]].arguments
// Arguments { … }
```

Examples

Using the arguments property

```
function f(n) {
  g(n - 1);
}

function g(n) {
  console.log(`before: ${g.arguments[0]}`);
  if (n > 0) {
    f(n);
  }
  console.log(`after: ${g.arguments[0]}`);
}

f(2);

console.log(`returned: ${g.arguments}`);

// Logs:
// before: 1
// before: 0
// after: 0
// after: 1
// returned: null
```

Specifications

Not part of any standard.

See also

- `arguments`
- Functions

Function.prototype.bind()

The **`bind()`** method of `Function` instances creates a new function that, when called, calls this function with its `this` keyword set to the provided value, and a given sequence of arguments preceding any provided when the new function is called.

Try it

```
const module = {
  x: 42,
  getX() {
    return this.x;
  },
};

const unboundGetX = module.getX;
console.log(unboundGetX()); // The function gets invoked at the global scope
// Expected output: undefined

const boundGetX = unboundGetX.bind(module);
console.log(boundGetX());
// Expected output: 42
```

Syntax

```
bind(thisArg)
bind(thisArg, arg1)
bind(thisArg, arg1, arg2)
bind(thisArg, arg1, arg2, /* …, */ argN)
```

Parameters

`thisArg`

> The value to be passed as the `this` parameter to the target function `func` when the bound function is called. If the function is not in strict mode, `null` and `undefined` will be replaced with the global object, and primitive values will be converted to objects. The value is ignored if the bound function is constructed using the `new` operator.

`arg1, ..., argN` Optional

> Arguments to prepend to arguments provided to the bound function when invoking `func`.

Return value

A copy of the given function with the specified `this` value, and initial arguments (if provided).

Description

The `bind()` function creates a new *bound function*. Calling the bound function generally results in the execution of the function it wraps, which is also called the *target function*. The bound function will store the parameters passed — which include the value of `this` and the first few arguments — as its internal state. These values are stored in advance, instead of being passed at call time. You can generally see `const boundFn = fn.bind(thisArg, arg1, arg2)` as being equivalent to `const boundFn = (...restArgs) => fn.call(thisArg, arg1, arg2, ...restArgs)` for the effect when it's called (but not when `boundFn` is constructed).

A bound function can be further bound by calling `boundFn.bind(thisArg, /* more args */)`, which creates another bound function `boundFn2`. The newly bound `thisArg` value is ignored, because the target function of `boundFn2`, which is `boundFn`, already has a bound `this`. When `boundFn2` is called, it would call `boundFn`, which in turn calls `fn`. The arguments that `fn` ultimately receives are, in order: the arguments bound by `boundFn`, arguments bound by `boundFn2`, and the arguments received by `boundFn2`.

```
"use strict"; // prevent `this` from being boxed into the wrapper object

function log(...args) {
  console.log(this, ...args);
}
const boundLog = log.bind("this value", 1, 2);
const boundLog2 = boundLog.bind("new this value", 3, 4);
boundLog2(5, 6); // "this value", 1, 2, 3, 4, 5, 6
```

A bound function may also be constructed using the `new` operator if its target function is constructable. Doing so acts as though the target function had instead been constructed. The prepended arguments are provided to the target function as usual, while the provided `this` value is ignored (because construction prepares its own `this`, as seen by the parameters of `Reflect.construct`). If the bound function is directly constructed, `new.target` will be the target function instead. (That is, the bound function is transparent to `new.target`.)

```
class Base {
  constructor(...args) {
    console.log(new.target === Base);
    console.log(args);
  }
}

const BoundBase = Base.bind(null, 1, 2);

new BoundBase(3, 4); // true, [1, 2, 3, 4]
```

However, because a bound function does not have the `prototype` property, it cannot be used as a base class for `extends`.

```
class Derived extends class {}.bind(null) {}
// TypeError: Class extends value does not have valid prototype property undefined
```

When using a bound function as the right-hand side of `instanceof`, `instanceof` would reach for the target function (which is stored internally in the bound function) and read its `prototype` instead.

```
class Base {}
const BoundBase = Base.bind(null, 1, 2);
console.log(new Base() instanceof BoundBase); // true
```

The bound function has the following properties:

`length`

> The `length` of the target function minus the number of arguments being bound (not counting the `thisArg` parameter), with 0 being the minimum value.

`name`

> The `name` of the target function plus a `"bound "` prefix.

The bound function also inherits the prototype chain of the target function. However, it doesn't have other own properties of the target function (such as static properties if the target function is a class).

Examples

Creating a bound function

The most common use of `bind()` is to make a function that, no matter how it is called, is called with a particular `this` value.

A common mistake for new JavaScript programmers is to extract a method from an object, then to later call that function and expect it to use the original object as its `this` (e.g., by using the method in callback-based code).

Without special care, however, the original object is usually lost. Creating a bound function from the function, using the original object, neatly solves this problem:

```
// Top-level 'this' is bound to 'globalThis' in scripts.
this.x = 9;
const module = {
  x: 81,
  getX() {
    return this.x;
  },
};

// The 'this' parameter of 'getX' is bound to 'module'.
console.log(module.getX()); // 81

const retrieveX = module.getX;
// The 'this' parameter of 'retrieveX' is bound to 'globalThis' in non-strict mode.
console.log(retrieveX()); // 9

// Create a new function 'boundGetX' with the 'this' parameter bound to 'module'.
const boundGetX = retrieveX.bind(module);
console.log(boundGetX()); // 81
```

> **Note:** If you run this example in strict mode, the `this` parameter of `retrieveX` will be bound to `undefined` instead of `globalThis`, causing the `retrieveX()` call to fail.
>
> If you run this example in an ECMAScript module, top-level `this` will be bound to `undefined` instead of `globalThis`, causing the `this.x = 9` assignment to fail.
>
> If you run this example in a Node CommonJS module, top-level `this` will be bound to `module.exports` instead of `globalThis`. However, the `this` parameter of `retrieveX` will still be bound to `globalThis` in non-strict mode and to `undefined` in strict mode. Therefore, in non-strict mode (the default), the `retrieveX()` call will return `undefined` because `this.x = 9` is writing to a different object (`module.exports`) from what `getX` is reading from (`globalThis`).

In fact, some built-in "methods" are also getters that return bound functions — one notable example being `Intl.NumberFormat.prototype.format()`, which, when accessed, returns a bound function that you can directly pass as a callback.

Partially applied functions

Another use of `bind()` is to make a function with pre-specified initial arguments.

These arguments (if any) follow the provided `this` value and are then inserted at the start of the arguments passed to the target function, followed by whatever arguments are passed to the bound function at the time it is called.

```
function list(...args) {
  return args;
}

function addArguments(arg1, arg2) {
  return arg1 + arg2;
}

console.log(list(1, 2, 3)); // [1, 2, 3]

console.log(addArguments(1, 2)); // 3

// Create a function with a preset leading argument
const leadingThirtySevenList = list.bind(null, 37);

// Create a function with a preset first argument.
const addThirtySeven = addArguments.bind(null, 37);

console.log(leadingThirtySevenList()); // [37]
console.log(leadingThirtySevenList(1, 2, 3)); // [37, 1, 2, 3]
console.log(addThirtySeven(5)); // 42
console.log(addThirtySeven(5, 10)); // 42
// (the last argument 10 is ignored)
```

With setTimeout()

By default, within `setTimeout()`, the `this` keyword will be set to `globalThis`, which is `window` in browsers. When working with class methods that require `this` to refer to class instances, you may explicitly bind `this` to the callback function, in order to maintain the instance.

```
class LateBloomer {
  constructor() {
    this.petalCount = Math.floor(Math.random() * 12) + 1;
  }
  bloom() {
    // Declare bloom after a delay of 1 second
    setTimeout(this.declare.bind(this), 1000);
  }
  declare() {
    console.log(`I am a beautiful flower with ${this.petalCount} petals!`);
```

```
  }
}

const flower = new LateBloomer();
flower.bloom();
// After 1 second, calls 'flower.declare()'
```

You can also use arrow functions for this purpose.

```
class LateBloomer {
  bloom() {
    // Declare bloom after a delay of 1 second
    setTimeout(() => this.declare(), 1000);
  }
}
```

Bound functions used as constructors

Bound functions are automatically suitable for use with the `new` operator to construct new instances created by the target function. When a bound function is used to construct a value, the provided `this` is ignored. However, provided arguments are still prepended to the constructor call.

```
function Point(x, y) {
  this.x = x;
  this.y = y;
}

Point.prototype.toString = function () {
  return `${this.x},${this.y}`;
};

const p = new Point(1, 2);
p.toString();
// '1,2'

// The thisArg's value doesn't matter because it's ignored
const YAxisPoint = Point.bind(null, 0 /* x */);

const axisPoint = new YAxisPoint(5);
axisPoint.toString(); // '0,5'

axisPoint instanceof Point; // true
axisPoint instanceof YAxisPoint; // true
new YAxisPoint(17, 42) instanceof Point; // true
```

Note that you need not do anything special to create a bound function for use with `new`. `new.target`, `instanceof`, `this` etc. all work as expected, as if the constructor was never bound. The only difference is that it can no longer be used for `extends`.

The corollary is that you need not do anything special to create a bound function to be called plainly, even if you would rather require the bound function to only be called using `new`. If you call it without `new`, the bound `this` is suddenly not ignored.

```
const emptyObj = {};
const YAxisPoint = Point.bind(emptyObj, 0 /* x */);

// Can still be called as a normal function
// (although usually this is undesirable)
YAxisPoint(13);

// The modifications to `this` is now observable from the outside
console.log(emptyObj); // { x: 0, y: 13 }
```

If you wish to restrict a bound function to only be callable with `new`, or only be callable without `new`, the target function must enforce that restriction, such as by checking `new.target !== undefined` or using a class instead.

Binding classes

Using `bind()` on classes preserves most of the class's semantics, except that all static own properties of the current class are lost. However, because the prototype chain is preserved, you can still access static properties inherited from the parent class.

```
class Base {
  static baseProp = "base";
}

class Derived extends Base {
  static derivedProp = "derived";
}

const BoundDerived = Derived.bind(null);
console.log(BoundDerived.baseProp); // "base"
console.log(BoundDerived.derivedProp); // undefined
console.log(new BoundDerived() instanceof Derived); // true
```

Transforming methods to utility functions

`bind()` is also helpful in cases where you want to transform a method which requires a specific `this` value to a plain utility function that accepts the previous `this` parameter as a normal parameter. This is similar to how general-purpose utility functions work: instead of calling `array.map(callback)`, you use `map(array, callback)`, which allows you to use `map` with array-like objects that are not arrays (for example, `arguments`) without mutating `Object.prototype`.

Take `Array.prototype.slice()`, for example, which you want to use for converting an array-like object to a real array. You could create a shortcut like this:

```
const slice = Array.prototype.slice;

// …

slice.call(arguments);
```

Note that you can't save `slice.call` and call it as a plain function, because the `call()` method also reads its `this` value, which is the function it should call. In this case, you can use `bind()` to bind the value of `this` for `call()`. In the following piece of code, `slice()` is a bound version of `Function.prototype.call()`, with the `this` value bound to `Array.prototype.slice()`. This means that additional `call()` calls can be eliminated:

```
// Same as "slice" in the previous example
const unboundSlice = Array.prototype.slice;
const slice = Function.prototype.call.bind(unboundSlice);

// …

slice(arguments);
```

See also

- Polyfill of `Function.prototype.bind` in `core-js`
- `Function.prototype.apply()`
- `Function.prototype.call()`
- Functions

Function.prototype.call()

The **`call()`** method of `Function` instances calls this function with a given `this` value and arguments provided individually.

Try it

```
function Product(name, price) {
  this.name = name;
  this.price = price;
}

function Food(name, price) {
  Product.call(this, name, price);
  this.category = "food";
}

console.log(new Food("cheese", 5).name);
// Expected output: "cheese"
```

Syntax

```
call(thisArg)
call(thisArg, arg1)
call(thisArg, arg1, arg2)
call(thisArg, arg1, arg2, /* …, */ argN)
```

Parameters

`thisArg`

> The value to use as `this` when calling `func`. If the function is not in strict mode, `null` and `undefined` will be replaced with the global object, and primitive values will be converted to objects.

`arg1, ..., argN` Optional

> Arguments for the function.

Return value

The result of calling the function with the specified `this` value and arguments.

Description

> **Note:** This function is almost identical to `apply()`, except that the function arguments are passed to `call()` individually as a list, while for `apply()` they are combined in one object, typically an array — for example, `func.call(this, "eat", "bananas")` vs. `func.apply(this, ["eat", "bananas"])`.

Normally, when calling a function, the value of `this` inside the function is the object that the function was accessed on. With `call()`, you can assign an arbitrary value as `this` when calling an existing function, without first attaching the function to the object as a property. This allows you to use methods of one object as generic utility functions.

> **Warning:** Do not use `call()` to chain constructors (for example, to implement inheritance). This invokes the constructor function as a plain function, which means `new.target` is `undefined`, and classes throw an error because they can't be called without `new`. Use `Reflect.construct()` or `extends` instead.

Examples

Using call() to invoke a function and specifying the this value

In the example below, when we call `greet`, the value of `this` will be bound to object `obj`, even when `greet` is not a method of `obj`.

```
function greet() {
  console.log(this.animal, "typically sleep between", this.sleepDuration);
}

const obj = {
  animal: "cats",
  sleepDuration: "12 and 16 hours",
};

greet.call(obj); // cats typically sleep between 12 and 16 hours
```

Using call() to invoke a function without specifying the first argument

If the first `thisArg` parameter is omitted, it defaults to `undefined`. In non-strict mode, the `this` value is then substituted with `globalThis` (which is akin to the global object).

```
globalThis.globProp = "foo";

function display() {
  console.log(`globProp value is ${this.globProp}`);
}

display.call(); // Logs "globProp value is foo"
```

In strict mode, the value of `this` is not substituted, so it stays as `undefined`.

```
"use strict";

globalThis.globProp = "foo";

function display() {
  console.log(`globProp value is ${this.globProp}`);
}

display.call(); // throws TypeError: Cannot read the property of 'globProp' of undefined
```

Transforming methods to utility functions

`call()` is almost equivalent to a normal function call, except that `this` is passed as a normal parameter instead of as the value that the function was accessed on. This is similar to how general-purpose utility functions work: instead of calling `array.map(callback)`, you use `map(array, callback)`, which allows you to use `map` with array-like objects that are not arrays (for example, `arguments`) without mutating `Object.prototype`.

Take `Array.prototype.slice()`, for example, which you want to use for converting an array-like object to a real array. You could create a shortcut like this:

```
const slice = Array.prototype.slice;

// …

slice.call(arguments);
```

Note that you can't save `slice.call` and call it as a plain function, because the `call()` method also reads its `this` value, which is the function it should call. In this case, you can use `bind()` to bind the value of `this` for `call()`. In the following piece of code, `slice()` is a bound version of `Function.prototype.call()`, with the `this` value bound to `Array.prototype.slice()`. This means that additional `call()` calls can be eliminated:

```
// Same as "slice" in the previous example
const unboundSlice = Array.prototype.slice;
const slice = Function.prototype.call.bind(unboundSlice);

// …

slice(arguments);
```

See also

- `Function.prototype.bind()`
- `Function.prototype.apply()`
- `Reflect.apply()`
- Spread syntax (`...`)
- Introduction to Object-Oriented JavaScript

Function.prototype.caller

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** In strict mode, accessing `caller` of a function throws an error — the API is removed with no replacement. This is to prevent code from being able to "walk the stack", which both poses security risks and severely limits the possibility of optimizations like inlining and tail-call optimization. For more explanation, you can read the rationale for the deprecation of `arguments.callee`.

The **`caller`** accessor property of `Function` instances returns the function that invoked this function. For strict, arrow, async, and generator functions, accessing the `caller` property throws a `TypeError`.

Description

If the function `f` was invoked by the top-level code, the value of `f.caller` is `null`; otherwise it's the function that called `f`. If the function that called `f` is a strict mode function, the value of `f.caller` is also `null`.

Note that the only behavior specified by the ECMAScript specification is that `Function.prototype` has an initial `caller` accessor that unconditionally throws a `TypeError` for any `get` or `set` request (known as a "poison pill accessor"), and that implementations are not allowed to change this semantic for any function except non-strict plain functions, in which case it must not have the value of a strict mode function. The actual behavior of the `caller` property, if it's anything other than throwing an error, is implementation-defined. For example, Chrome defines it as an own data property, while Firefox and Safari extend the initial poison-pill `Function.prototype.caller` accessor to specially handle `this` values that are non-strict functions.

```
(function f() {
  if (Object.hasOwn(f, "caller")) {
    console.log(
      "caller is an own property with descriptor",
      Object.getOwnPropertyDescriptor(f, "caller"),
    );
  } else {
    console.log(
      "f doesn't have an own property named caller. Trying to get f.[[Prototype]].caller",
    );
    console.log(
      Object.getOwnPropertyDescriptor(
        Object.getPrototypeOf(f),
        "caller",
      ).get.call(f),
    );
  }
})();

// In Chrome:
// caller is an own property with descriptor {value: null, writable: false, enumerable: false, configurable: false}

// In Firefox:
// f doesn't have an own property named caller. Trying to get f.[[Prototype]].caller
// null
```

This property replaces the obsolete `arguments.caller` property of the `arguments` object.

The special property `__caller__`, which returned the activation object of the caller thus allowing you to reconstruct the stack, was removed for security reasons.

Examples

Checking the value of a function's caller property

The following code checks the value a function's `caller` property.

```
function myFunc() {
  if (myFunc.caller === null) {
    return "The function was called from the top!";
  }
  return `This function's caller was ${myFunc.caller}`;
}
```

Reconstructing the stack and recursion

Note that in case of recursion, you can't reconstruct the call stack using this property. Consider:

```
function f(n) {
  g(n - 1);
}
function g(n) {
  if (n > 0) {
    f(n);
  } else {
    stop();
  }
```

```
}
f(2);
```

At the moment `stop()` is called the call stack will be:

```
f(2) -> g(1) -> f(1) -> g(0) -> stop()
```

The following is true:

```
stop.caller === g && f.caller === g && g.caller === f;
```

so if you tried to get the stack trace in the `stop()` function like this:

```
let f = stop;
let stack = "Stack trace:";
while (f) {
  stack += `\n${f.name}`;
  f = f.caller;
}
```

the loop would never stop.

Strict mode caller

If the caller is a strict mode function, the value of `caller` is `null`.

```
function callerFunc() {
  calleeFunc();
}

function strictCallerFunc() {
  "use strict";
  calleeFunc();
}

function calleeFunc() {
  console.log(calleeFunc.caller);
}

(function () {
  callerFunc();
})();
// Logs [Function: callerFunc]

(function () {
  strictCallerFunc();
})();
// Logs null
```

Specifications

Not part of any standard.

See also

- `Function.prototype.name`
- `arguments`

Function: displayName

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The optional **`displayName`** property of a `Function` instance specifies the display name of the function.

Value

The `displayName` property is not initially present on any function — it's added by the code authors. For the purpose of display, it should be a string.

Description

The `displayName` property, if present, may be preferred by consoles and profilers over the `name` property to be displayed as the name of a function.

Among browsers, only the Firefox console utilizes this property. React devtools also use the `displayName` property when displaying the component tree.

Firefox does some basic attempts to decode the `displayName` that's possibly generated by the anonymous JavaScript functions naming convention algorithm. The following patterns are detected:

- If `displayName` ends with a sequence of alphanumeric characters, `_`, and `$`, the longest such suffix is displayed.
- If `displayName` ends with a sequence of `[]`-enclosed characters, that sequence is displayed without the square brackets.

- If `displayName` ends with a sequence of alphanumeric characters and _ followed by some `/`, `.`, or `<`, the sequence is returned without the trailing `/`, `.`, or `<` characters.
- If `displayName` ends with a sequence of alphanumeric characters and _ followed by `(^)`, the sequence is displayed without the `(^)`.

If none of the above patterns match, the entire `displayName` is displayed.

Examples

Setting a displayName

By entering the following in a Firefox console, it should display as something like `function MyFunction()`:

```
function a() {}
a.displayName = "MyFunction";

a; // function MyFunction()
```

Changing displayName dynamically

You can dynamically change the `displayName` of a function:

```
const object = {
  // anonymous
  someMethod: function someMethod(value) {
    someMethod.displayName = `someMethod (${value})`;
  },
};

console.log(object.someMethod.displayName); // undefined

object.someMethod("123");
console.log(object.someMethod.displayName); // "someMethod (123)"
```

Cleaning of displayName

Firefox devtools would clean up a few common patterns in the `displayName` property before displaying it.

```
function foo() {}

function testName(name) {
  foo.displayName = name;
  console.log(foo);
}

testName("$foo$"); // function $foo$()
testName("foo bar"); // function bar()
testName("Foo.prototype.add"); // function add()
testName("foo ."); // function foo .()
testName("foo <"); // function foo <()
testName("foo?"); // function foo?()
testName("foo()"); // function foo()()

testName("[...]"); // function ...()
testName("foo<"); // function foo()
testName("foo..."); // function foo()
testName("foo(^)"); // function foo()
```

Specifications

Not part of any standard.

See also

- `Function.prototype.name`

Function() constructor

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations for more information.

The **`Function()`** constructor creates `Function` objects. Calling the constructor directly can create functions dynamically, but suffers from security and similar (but far less significant) performance issues as `eval()`. However, unlike `eval` (which may have access to the local scope), the `Function` constructor creates functions which execute in the global scope only.

Try it

```
const sum = new Function("a", "b", "return a + b");
```

```
console.log(sum(2, 6));
// Expected output: 8
```

Syntax

```
new Function(functionBody)
new Function(arg1, functionBody)
new Function(arg1, arg2, functionBody)
new Function(arg1, arg2, /* …, */ argN, functionBody)

Function(functionBody)
Function(arg1, functionBody)
Function(arg1, arg2, functionBody)
Function(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `Function()` can be called with or without `new`. Both create a new `Function` instance.

Parameters

`arg1, ..., argN` Optional

> `TrustedScript` instances or strings specifying names to be used by the function as formal argument names. The value must correspond to a valid JavaScript parameter (any of plain identifier, rest parameter, or destructured parameter, optionally with a default), or a list of such strings separated with commas.
>
> As the parameters are parsed in the same way as function expressions, whitespace and comments are accepted. For example: `"x", "theValue = 42", "[a, b] /* numbers */"` — or `"x, theValue = 42, [a, b] /* numbers */"`. (`"x, theValue = 42", "[a, b]"` is also correct, though very confusing to read.)

`functionBody`

> A `TrustedScript` or a string containing the JavaScript statements comprising the function definition.

Exceptions

`SyntaxError`

> Function parameter arguments can't be parsed as a valid parameter list, or the `functionBody` can't be parsed as valid JavaScript statements.

`TypeError`

> Any parameter is a string when Trusted Types are enforced by a CSP and no default policy is defined.

Description

`Function` objects created with the `Function` constructor are parsed when the function is created. This is less efficient than creating a function with a function expression or function declaration and calling it within your code, because such functions are parsed with the rest of the code.

All arguments passed to the function, except the last, are treated as the names of the identifiers of the parameters in the function to be created, in the order in which they are passed. The function will be dynamically compiled as a function expression, with the source assembled in the following fashion:

```
`function anonymous(${args.join(",")}
) {
${functionBody}
}`;
```

This is observable by calling the function's `toString()` method.

However, unlike normal function expressions, the name `anonymous` is not added to the `functionBody`'s scope, since `functionBody` only has access the global scope. If `functionBody` is not in strict mode (the body itself needs to have the `"use strict"` directive since it doesn't inherit the strictness from the context), you may use `arguments.callee` to refer to the function itself. Alternatively, you can define the recursive part as an inner function:

```
const recursiveFn = new Function(
  "count",
  `
(function recursiveFn(count) {
  if (count < 0) {
    return;
  }
  console.log(count);
  recursiveFn(count - 1);
})(count);
`,
);
```

Note that the two dynamic parts of the assembled source — the parameters list `args.join(",")` and `functionBody` — will first be parsed separately to ensure they are each syntactically valid. This prevents injection-like attempts.

```
new Function("/*", "*/) {");
// SyntaxError: Unexpected end of arg string
// Doesn't become "function anonymous(/*) {*/) {}"
```

Security considerations

The method can be used to execute arbitrary input passed to any parameter. If the input is a potentially unsafe string provided by a user, this is a possible vector for Cross-site-scripting (XSS) attacks. For example, the following example assumes the `untrustedCode` was provided by a user:

```
const untrustedCode = "alert('Potentially evil code!');";
const adder = new Function("a", "b", untrustedCode);
```

Websites with a Content Security Policy (CSP) that specifies `script-src` or `default-src` will prevent such code running by default. If you must allow the scripts to run via `Function()`, you can mitigate these issues by always assigning `TrustedScript` objects instead of strings, and enforcing trusted types using the `require-trusted-types-for` CSP directive. This ensures that the input is passed through a transformation function.

To allow `Function()` to run, you additionally need to specify the `trusted-types-eval` keyword in your CSP `script-src` directive. The `unsafe-eval` keyword also allows `Function()`, but is much less safe than `trusted-types-eval` because it would allow execution even on browsers that do not support trusted types.

For example, the required CSP for your site might look like this:

```
Content-Security-Policy: require-trusted-types-for 'script'; script-src '<your_allowlist>' 'trusted-types-eval'
```

The behavior of the transformation function depends on the specific use case that requires a user-provided script. If possible, you should lock the allowed scripts to exactly the code that you trust to run. If that is not possible, you might allow or block the use of certain functions within the provided string.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Specifying arguments with the Function constructor

The following code creates a `Function` object that takes two arguments.

```
// Example can be run directly in your JavaScript console

// Create a function that takes two arguments, and returns the sum of those arguments
const adder = new Function("a", "b", "return a + b");

// Call the function
adder(2, 6);
// 8
```

The arguments `a` and `b` are formal argument names that are used in the function body, `return a + b`.

Creating a function object from a function declaration or function expression

```
// The function constructor can take in multiple statements separated by a semicolon. Function expressions require a return statement with the function's
name

// Observe that new Function is called. This is so we can call the function we created directly afterwards
const sumOfArray = new Function(
  "const sumArray = (arr) => arr.reduce((previousValue, currentValue) => previousValue + currentValue); return sumArray",
)();

// call the function
sumOfArray([1, 2, 3, 4]);
// 10

// If you don't call new Function at the point of creation, you can use the Function.call() method to call it
const findLargestNumber = new Function(
  "function findLargestNumber (arr) { return Math.max(...arr) }; return findLargestNumber",
);

// call the function
findLargestNumber.call({}).call({}, [2, 4, 1, 8, 5]);
// 8

// Function declarations do not require a return statement
const sayHello = new Function(
  "return function (name) { return `Hello, ${name}` }",
)();

// call the function
sayHello("world");
// Hello, world
```

See also

- Using the function constructor in `eval()`
- `function`
- `function` expression
- Functions

Function: length

The **`length`** data property of a `Function` instance indicates the number of parameters expected by the function.

Try it

```
function func1() {}

function func2(a, b) {}

console.log(func1.length);
// Expected output: 0

console.log(func2.length);
// Expected output: 2
```

Value

A number.

Property attributes of `Function: length`	
Writable	no
Enumerable	no
Configurable	yes

Description

A `Function` object's `length` property indicates how many arguments the function expects, i.e., the number of formal parameters:

- Only parameters before the first one with a default value are counted.
- A destructuring pattern counts as a single parameter.
- The rest parameter is excluded.

By contrast, `arguments.length` is local to a function and provides the number of arguments actually passed to the function.

The `Function` constructor is itself a `Function` object. Its `length` data property has a value of `1`.

Due to historical reasons, `Function.prototype` is a callable itself. The `length` property of `Function.prototype` has a value of `0`.

Examples

Using function length

```
console.log(Function.length); // 1

console.log((() => {}).length); // 0
console.log(((a) => {}).length); // 1
console.log(((a, b) => {}).length); // 2 etc.

console.log(((...args) => {}).length);
// 0, rest parameter is not counted

console.log(((a, b = 1, c) => {}).length);
// 1, only parameters before the first one with
// a default value are counted

console.log((({ a, b }, [c, d]) => {}).length);
// 2, destructuring patterns each count as
// a single parameter
```

See also

- `Function`

Function: name

The **`name`** data property of a `Function` instance indicates the function's name as specified when it was created, or it may be either `anonymous` or `''` (an empty string) for functions created anonymously.

Try it

```
const func1 = function () {};

const object = {
  func2: function () {},
};

console.log(func1.name);
// Expected output: "func1"

console.log(object.func2.name);
// Expected output: "func2"
```

Value

A string.

Property attributes of `Function: name`	
Writable	no
Enumerable	no
Configurable	yes

> **Note:** In non-standard, pre-ES2015 implementations the `configurable` attribute was `false` as well.

Description

The function's `name` property can be used to identify the function in debugging tools or error messages. It has no semantic significance to the language itself.

The `name` property is read-only and cannot be changed by the assignment operator:

```
function someFunction() {}

someFunction.name = "otherFunction";
console.log(someFunction.name); // someFunction
```

To change it, use `Object.defineProperty()`.

The `name` property is typically inferred from how the function is defined. In the following sections, we will describe the various ways in which it can be inferred.

Function declaration

The `name` property returns the name of a function declaration.

```
function doSomething() {}
doSomething.name; // "doSomething"
```

Default-exported function declaration

An `export default` declaration exports the function as a declaration instead of an expression. If the declaration is anonymous, the name is `"default"`.

```
// -- someModule.js --
export default function () {}

// -- main.js --
import someModule from "./someModule.js";

someModule.name; // "default"
```

Function constructor

Functions created with the `Function()` constructor have name "anonymous".

```
new Function().name; // "anonymous"
```

Function expression

If the function expression is named, that name is used as the `name` property.

```
const someFunction = function someFunctionName() {};
someFunction.name; // "someFunctionName"
```

Anonymous function expressions, created using either the `function` keyword or the arrow function syntax, have `""` (an empty string) as their name by default.

```
(function () {}).name; // ""
(() => {}).name; // ""
```

However, such cases are rare — usually, in order to call the function elsewhere, the function expression is associated with an identifier. The name of an anonymous function expression can be inferred within certain syntactic contexts, including: variable declaration, method, initializer, and default value.

One practical case where the name cannot be inferred is a function returned from another function:

```
function getFoo() {
  return () => {};
}
getFoo().name; // ""
```

Variable declaration and method

Variables and methods can infer the name of an anonymous function from its syntactic position.

```
const f = function () {};
const object = {
  someMethod: function () {},
};
```

```
console.log(f.name); // "f"
console.log(object.someMethod.name); // "someMethod"
```

The same applies to assignment:

```
let f;
f = () => {};
f.name; // "f"
```

Initializer and default value

Functions in initializers (default values) of destructuring, default parameters, class fields, etc., will inherit the name of the bound identifier as their `name`.

```
const [f = () => {}] = [];
f.name; // "f"

const { someMethod: m = () => {} } = {};
m.name; // "m"

function foo(f = () => {}) {
  console.log(f.name);
}
foo(); // "f"

class Foo {
  static someMethod = () => {};
}
Foo.someMethod.name; // someMethod
```

Shorthand method

```
const o = {
  foo() {},
};
o.foo.name; // "foo";
```

Bound function

`Function.prototype.bind()` produces a function whose name is "bound " plus the function name.

```
function foo() {}
foo.bind({}).name; // "bound foo"
```

Getter and setter

When using `get` and `set` accessor properties, "get" or "set" will appear in the function name.

```
const o = {
  get foo() {
    return 1;
  },
  set foo(x) {},
};

const descriptor = Object.getOwnPropertyDescriptor(o, "foo");
descriptor.get.name; // "get foo"
descriptor.set.name; // "set foo";
```

Class

A class's name follows the same algorithm as function declarations and expressions.

```
class Foo {}
Foo.name; // "Foo"
```

> **Warning:** JavaScript will set the function's `name` property only if a function does not have an own property called `name`. However, classes' static members will be set as own properties of the class constructor function, and thus prevent the built-in `name` from being applied. See an example below.

Symbol as function name

If a `Symbol` is used a function name and the symbol has a description, the method's name is the description in square brackets.

```
const sym1 = Symbol("foo");
const sym2 = Symbol();

const o = {
  [sym1]() {},
  [sym2]() {},
};
```

```
o[sym1].name; // "[foo]"
o[sym2].name; // "[]"
```

Private fields and methods

Private fields and private methods have the hash (`#`) as part of their names.

```
class Foo {
  #field = () => {};
  #method() {}
  getNames() {
    console.log(this.#field.name);
    console.log(this.#method.name);
  }
}

new Foo().getNames();
// "#field"
// "#method"
```

Examples

Telling the constructor name of an object

You can use `obj.constructor.name` to check the "class" of an object.

```
function Foo() {} // Or: class Foo {}

const fooInstance = new Foo();
console.log(fooInstance.constructor.name); // "Foo"
```

However, because static members will become own properties of the class, we can't obtain the class name for virtually any class with a static method property `name()`:

```
class Foo {
  constructor() {}
  static name() {}
}
```

With a `static name()` method `Foo.name` no longer holds the actual class name but a reference to the `name()` function object. Trying to obtain the class of `fooInstance` via `fooInstance.constructor.name` won't give us the class name at all, but instead a reference to the static class method. Example:

```
const fooInstance = new Foo();
console.log(fooInstance.constructor.name); // ƒ name() {}
```

Due to the existence of static fields, `name` may not be a function either.

```
class Foo {
  static name = 123;
}
console.log(new Foo().constructor.name); // 123
```

If a class has a static property called `name`, it will also become *writable*. The built-in definition in the absence of a custom static definition is *read-only*.

```
Foo.name = "Hello";
console.log(Foo.name); // "Hello" if class Foo has a static "name" property, but "Foo" if not.
```

Therefore you may not rely on the built-in `name` property to always hold a class's name.

JavaScript compressors and minifiers

> **Warning:** Be careful when using the `name` property with source-code transformations, such as those carried out by JavaScript compressors (minifiers) or obfuscators. These tools are often used as part of a JavaScript build pipeline to reduce the size of a program prior to deploying it to production. Such transformations often change a function's name at build time.

Source code such as:

```
function Foo() {}
const foo = new Foo();

if (foo.constructor.name === "Foo") {
  console.log("'foo' is an instance of 'Foo'");
} else {
  console.log("Oops!");
}
```

may be compressed to:

```
function a() {}
const b = new a();
if (b.constructor.name === "Foo") {
  console.log("'foo' is an instance of 'Foo'");
} else {
  console.log("Oops!");
}
```

In the uncompressed version, the program runs into the truthy branch and logs "'foo' is an instance of 'Foo'" — whereas, in the compressed version it behaves differently, and runs into the else branch. If you rely on the `name` property, like in the example above, make sure your build pipeline doesn't change function names, or don't assume a function has a particular name.

See also

- Polyfill for `Function: name` in `core-js`
- es-shims polyfill of `Function.prototype.name`
- `Function`

Function: prototype

The **`prototype`** data property of a `Function` instance is used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

> **Note:** Not all `Function` objects have the `prototype` property — see description.

Value

An object.

Property attributes of `Function: prototype`	
Writable	yes
Enumerable	no
Configurable	no

> **Note:** Classes are a type of function, so most of the description here applies to the `prototype` property of classes too. The only salient difference is that the `prototype` property of a class is not writable.

Description

When a function is called with `new`, the constructor's `prototype` property will become the resulting object's prototype.

```
function Ctor() {}
const inst = new Ctor();
console.log(Object.getPrototypeOf(inst) === Ctor.prototype); // true
```

You can read Inheritance and the prototype chain for more information about the interactions between a constructor function's `prototype` property and the resulting object's prototype.

A function having a `prototype` property is not sufficient for it to be eligible as a constructor. Generator functions have a `prototype` property, but cannot be called with `new`:

```
async function* asyncGeneratorFunction() {}
function* generatorFunction() {}
```

Instead, generator functions' `prototype` property is used when they are called *without* `new`. The `prototype` property will become the returned `Generator` object's prototype.

In addition, some functions may have a `prototype` but throw unconditionally when called with `new`. For example, the `Symbol()` and `BigInt()` functions throw when called with `new`, because `Symbol.prototype` and `BigInt.prototype` are only intended to provide methods for the primitive values, but the wrapper objects should not be directly constructed.

The following functions do not have `prototype`, and are therefore ineligible as constructors, even if a `prototype` property is later manually assigned:

```
const method = { foo() {} }.foo;
const arrowFunction = () => {};
async function asyncFunction() {}
```

The following are valid constructors that have `prototype`:

```
class Class {}
function fn() {}
```

A bound function does not have a `prototype` property, but may be constructable. When it's constructed, the target function is constructed instead, and if the target function is constructable, it would return a normal instance.

```
const boundFunction = function () {}.bind(null);
```

A function's `prototype` property, by default, is a plain object with one property: `constructor`, which is a reference to the function itself. The `constructor` property is writable, non-enumerable, and configurable.

If the `prototype` of a function is reassigned with something other than an `Object`, when the function is called with `new`, the returned object's prototype would be `Object.prototype` instead. (In other words, `new` ignores the `prototype` property and constructs a plain object.)

```
function Ctor() {}
Ctor.prototype = 3;
console.log(Object.getPrototypeOf(new Ctor()) === Object.prototype); // true
```

Examples

Changing the prototype of all instances by mutating the prototype property

```
function Ctor() {}
const p1 = new Ctor();
const p2 = new Ctor();
Ctor.prototype.prop = 1;
console.log(p1.prop); // 1
console.log(p2.prop); // 1
```

Adding a non-method property to a class's prototype property

Class fields add properties to each instance. Class methods declare *function* properties on the prototype. However, there's no way to add a non-function property to the prototype. In case you want to share static data between all instances (for example, `Error.prototype.name` is the same between all error instances), you can manually assign it on the `prototype` of a class.

```
class Dog {
  constructor(name) {
    this.name = name;
  }
}

Dog.prototype.species = "dog";

console.log(new Dog("Jack").species); // "dog"
```

This can be made more ergonomic using static initialization blocks, which are called when the class is initialized.

```
class Dog {
  static {
    Dog.prototype.species = "dog";
  }
  constructor(name) {
    this.name = name;
  }
}

console.log(new Dog("Jack").species); // "dog"
```

See also

- `Function`
- Inheritance and the prototype chain

Function.prototype[Symbol.hasInstance]()

The **`[Symbol.hasInstance]()`** method of `Function` instances specifies the default procedure for determining if a constructor function recognizes an object as one of the constructor's instances. It is called by the `instanceof` operator.

Syntax

```
func[Symbol.hasInstance](value)
```

Parameters

`value`

The object to test. Primitive values always return `false`.

Return value

`true` if `func.prototype` is in the prototype chain of `value`; otherwise, `false`. Always returns `false` if `value` is not an object or `this` is not a function. If `this` is a bound function, returns the result of an `instanceof` test on `value` and the underlying target function.

Exceptions

`TypeError`

Thrown if `this` is not a bound function and `this.prototype` is not an object.

Description

The `instanceof` operator calls the `[Symbol.hasInstance]()` method of the right-hand side whenever such a method exists. Because all functions inherit from `Function.prototype` by default, they would all have the `[Symbol.hasInstance]()` method, so most of the time, the `Function.prototype[Symbol.hasInstance]()` method specifies the behavior of `instanceof` when the right-hand side is a function. This method implements the default behavior of the `instanceof` operator (the same algorithm when `constructor` has no `[Symbol.hasInstance]()` method).

Unlike most methods, the `Function.prototype[Symbol.hasInstance]()` property is non-configurable and non-writable. This is a security feature to prevent the underlying target function of a bound function from being obtainable. See this Stack Overflow answer for an example.

Examples

Reverting to default instanceof behavior

You would rarely need to call this method directly. Instead, this method is called by the `instanceof` operator. You should expect the two results to usually be equivalent.

```
class Foo {}
const foo = new Foo();
console.log(foo instanceof Foo === Foo[Symbol.hasInstance](foo)); // true
```

You may want to use this method if you want to invoke the default `instanceof` behavior, but you don't know if a constructor has an overridden `[Symbol.hasInstance]()` method.

```
class Foo {
  static [Symbol.hasInstance](value) {
    // A custom implementation
    return false;
  }
}

const foo = new Foo();
console.log(foo instanceof Foo); // false
console.log(Function.prototype[Symbol.hasInstance].call(Foo, foo)); // true
```

See also

- `instanceof`
- `Symbol.hasInstance`

Function.prototype.toString()

The **`toString()`** method of `Function` instances returns a string representing the source code of this function.

Try it

```
function sum(a, b) {
  return a + b;
}

console.log(sum.toString());
// Expected output: "function sum(a, b) {
//                     return a + b;
//                   }"

console.log(Math.abs.toString());
// Expected output: "function abs() { [native code] }"
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the source code of the function.

Description

The `Function` object overrides the `toString()` method inherited from `Object`; it does not inherit `Object.prototype.toString`. For user-defined `Function` objects, the `toString` method returns a string containing the source text segment which was used to define the function.

JavaScript calls the `toString` method automatically when a `Function` is to be represented as a text value, e.g., when a function is concatenated with a string.

The `toString()` method will throw a `TypeError` exception ("Function.prototype.toString called on incompatible object"), if its `this` value object is not a `Function` object.

```
Function.prototype.toString.call("foo"); // throws TypeError
```

If the `toString()` method is called on built-in function objects, a function created by `Function.prototype.bind()`, or other non-JavaScript functions, then `toString()` returns a *native function string* which looks like

```
function someName() { [native code] }
```

For intrinsic object methods and functions, `someName` is the initial name of the function; otherwise its content may be implementation-defined, but will always be in property name syntax, like `[1 + 1]`, `someName`, or `1`.

> **Note:** This means using `eval()` on native function strings is a guaranteed syntax error.

If the `toString()` method is called on a function created by the `Function` constructor, `toString()` returns the source code of a synthesized function declaration named "anonymous" using the provided parameters and function body. For example, `Function("a", "b", "return a + b").toString()` will return:

```
function anonymous(a,b
) {
return a + b
}
```

Since ES2018, the spec requires the return value of `toString()` to be the exact same source code as it was declared, including any whitespace and/or comments — or, if the host doesn't have the source code available for some reason, requires returning a native function string. Support for this revised behavior can be found in the compatibility table.

Examples

Comparing actual source code and toString results

```
function test(fn) {
  console.log(fn.toString());
}

function f() {}
class A {
  a() {}
}
function* g() {}

test(f); // "function f() {}"
test(A); // "class A { a() {} }"
test(g); // "function* g() {}"
test((a) => a); // "(a) => a"
test({ a() {} }.a); // "a() {}"
test({ *a() {} }.a); // "*a() {}"
test({ [0]() {} }[0]); // "[0]() {}"
test(Object.getOwnPropertyDescriptor({ get a() {} }, "a").get); // "get a() {}"
test(Object.getOwnPropertyDescriptor({ set a(x) {} }, "a").set); // "set a(x) {}"
test(Function.prototype.toString); // "function toString() { [native code] }"
test(function f() {}.bind(0)); // "function () { [native code] }"
test(Function("a", "b")); // function anonymous(a\n) {\nb\n}
```

Note that after the `Function.prototype.toString()` revision, when `toString()` is called, implementations are never allowed to synthesize a function's source that is not a native function string. The method always returns the exact source code used to create the function — including the getter and setter examples above. The `Function` constructor itself has the capability of synthesizing the source code for the function (and is therefore a form of implicit `eval()`).

Getting source text of a function

It is possible to get the source text of a function by coercing it to a string — for example, by wrapping it in a template literal:

```
function foo() {
  return "bar";
}
console.log(`${foo}`);
// function foo() {
//   return "bar";
// }
```

This source text is *exact*, including any interspersed comments (which won't be stored by the engine's internal representation otherwise).

```
function foo /* a comment */() {
  return "bar";
}
console.log(foo.toString());
// function foo /* a comment */() {
//   return "bar";
// }
```

See also

- `Object.prototype.toString()`

Generator

The **`Generator`** object is returned by a generator function and it conforms to both the iterable protocol and the iterator protocol.

`Generator` is a subclass of the `Iterator` class.

Constructor

There's no JavaScript entity that corresponds to the `Generator` constructor. Instances of `Generator` must be returned from generator functions:

```
function* generator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = generator(); // "Generator { }"

console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

There's only a hidden object which is the prototype object shared by all objects created by generator functions. This object is often stylized as `Generator.prototype` to make it look like a class, but it should be more appropriately called `GeneratorFunction.prototype.prototype`, because `GeneratorFunction` is an actual JavaScript entity. To understand the prototype chain of `Generator` instances, see `GeneratorFunction.prototype.prototype`.

Instance properties

These properties are defined on `Generator.prototype` and shared by all `Generator` instances.

`Generator.prototype.constructor`

> The constructor function that created the instance object. For `Generator` instances, the initial value is `GeneratorFunction.prototype`.
>
> > **Note:** `Generator` objects do not store a reference to the generator function that created them.

`Generator.prototype[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"Generator"`. This property is used in `Object.prototype.toString()`.

Instance methods

Also inherits instance methods from its parent `Iterator`.

`Generator.prototype.next()`

> Returns a value yielded by the `yield` expression.

`Generator.prototype.return()`

> Acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

`Generator.prototype.throw()`

> Acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Examples

An infinite iterator

With a generator function, values are not evaluated until they are needed. Therefore a generator allows us to define a potentially infinite data structure.

```
function* infinite() {
  let index = 0;

  while (true) {
    yield index++;
  }
}

const generator = infinite(); // "Generator { }"

console.log(generator.next().value); // 0
console.log(generator.next().value); // 1
console.log(generator.next().value); // 2
// …
```

See also

- `function*`
- `function*` expression
- `GeneratorFunction`
- Iteration protocols

Generator.prototype.next()

The **`next()`** method of `Generator` instances returns an object with two properties `done` and `value`. You can also provide a parameter to the `next` method to send a value to the generator.

Syntax

```
next()
next(value)
```

Parameters

`value` Optional

The value to send to the generator.

The value will be assigned as a result of a `yield` expression. For example, in `variable = yield expression`, the value passed to the `.next()` function will be assigned to `variable`.

Return value

An `Object` with two properties:

`done`

A boolean value:

- `true` if the generator is past the end of its control flow. In this case `value` specifies the *return value* of the generator (which may be undefined).
- `false` if the generator is able to produce more values.

`value`

Any JavaScript value yielded or returned by the generator.

Exceptions

`TypeError`

Thrown if the generator is already running.

Examples

Using next()

The following example shows a generator and the object that the `next` method returns:

```
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen(); // Generator { }
g.next(); // { value: 1, done: false }
g.next(); // { value: 2, done: false }
g.next(); // { value: 3, done: false }
g.next(); // { value: undefined, done: true }
```

Using next() with a list

In this example, `getPage` takes a list and "paginates" it into chunks of size `pageSize`. Each call to `next` will yield one such chunk.

```
function* getPage(list, pageSize = 1) {
  for (let index = 0; index < list.length; index += pageSize) {
    yield list.slice(index, index + pageSize);
  }
}

const list = [1, 2, 3, 4, 5, 6, 7, 8];
const page = getPage(list, 3); // Generator { }

page.next(); // { value: [1, 2, 3], done: false }
page.next(); // { value: [4, 5, 6], done: false }
page.next(); // { value: [7, 8], done: false }
page.next(); // { value: undefined, done: true }
```

Sending values to the generator

In this example, `next` is called with a value.

> **Note:** The first call does not log anything, because the generator was not yielding anything initially.

```
function* gen() {
  while (true) {
    const value = yield;
    console.log(value);
  }
}

const g = gen();
g.next(1); // Returns { value: undefined, done: false }
// No log at this step: the first value sent through `next` is lost
g.next(2); // Returns { value: undefined, done: false }
// Logs 2
```

See also

- `function*`
- Iterators and generators guide

Generator.prototype.return()

The **`return()`** method of `Generator` instances acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

Syntax

```
generatorInstance.return()
generatorInstance.return(value)
```

Parameters

`value` Optional

The value to return.

Return value

An `Object` with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function's control flow hasn't reached the end and can produce more values. This can only happen if the `return` is captured in a `try...finally` and there are more `yield` expressions in the `finally` block.

`value`

The value that is given as an argument, or, if the `yield` expression is wrapped in a `try...finally`, the value yielded/returned from the `finally` block.

Exceptions

`TypeError`

Thrown if the generator is already running.

Description

The `return()` method, when called, can be seen as if a `return value;` statement is inserted in the generator's body at the current suspended position, where `value` is the value passed to the `return()` method. Therefore, in a typical flow, calling `return(value)` will return `{ done: true, value: value }`. However, if the `yield` expression is wrapped in a `try...finally` block, the control flow doesn't exit the function body, but proceeds to the `finally` block instead. In this case, the value returned may be different, and `done` may even be `false`, if there are more `yield` expressions within the `finally` block.

Examples

Using return()

The following example shows a generator and the `return` method.

```
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen();

g.next(); // { value: 1, done: false }
g.return("foo"); // { value: "foo", done: true }
g.next(); // { value: undefined, done: true }
```

If `return(value)` is called on a generator that is already in "completed" state, the generator will remain in "completed" state.

If no argument is provided, the `value` property of the returned object will be `undefined`. If an argument is provided, it will become the value of the `value` property of the returned object, unless the `yield` expression is wrapped in a `try...finally`.

```
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen();
g.next(); // { value: 1, done: false }
g.next(); // { value: 2, done: false }
g.next(); // { value: 3, done: false }
g.next(); // { value: undefined, done: true }
g.return(); // { value: undefined, done: true }
g.return(1); // { value: 1, done: true }
```

Using return() with try...finally

The fact that the `return` method has been called can only be made known to the generator itself if the `yield` expression is wrapped in a `try...finally` block.

When the `return` method is called on a generator that is suspended within a `try` block, execution in the generator proceeds to the `finally` block — since the `finally` block of `try...finally` statements always executes.

```
function* gen() {
  yield 1;
  try {
    yield 2;
    yield 3;
  } finally {
    yield "cleanup";
  }
}

const g1 = gen();
g1.next(); // { value: 1, done: false }

// Execution is suspended before the try...finally.
g1.return("early return"); // { value: 'early return', done: true }

const g2 = gen();
g2.next(); // { value: 1, done: false }
g2.next(); // { value: 2, done: false }

// Execution is suspended within the try...finally.
g2.return("early return"); // { value: 'cleanup', done: false }

// The completion value is preserved
g2.next(); // { value: 'early return', done: true }

// Generator is in the completed state
g2.return("not so early return"); // { value: 'not so early return', done: true }
```

The return value of the finally block can also become the `value` of the result returned from the `return` call.

```
function* gen() {
  try {
    yield 1;
  } finally {
    return "cleanup";
  }
}

const generator = gen();
generator.next(); // { value: 1, done: false }
generator.return("early return"); // { value: 'cleanup', done: true }
```

See also

- `function*`

Generator.prototype.throw()

The **`throw()`** method of `Generator` instances acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Syntax

```
generatorInstance.throw(exception)
```

Parameters

`exception`

The exception to throw. For debugging purposes, it is useful to make it an `instanceof` `Error`.

Return value

If the thrown exception is caught by a `try...catch` and the generator resumes to yield more values, it will return an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function is able to produce more values.

`value`

The value yielded from the next `yield` expression.

Exceptions

`TypeError`

Thrown if the generator is already running.

If the `exception` is not caught by a `try...catch` within the generator function, it is also thrown to the caller of `throw()`.

Description

The `throw()` method, when called, can be seen as if a `throw exception;` statement is inserted in the generator's body at the current suspended position, where `exception` is the exception passed to the `throw()` method. Therefore, in a typical flow, calling `throw(exception)` will cause the generator to throw. However, if the `yield` expression is wrapped in a `try...catch` block, the error may be caught and control flow can either resume after error handling, or exit gracefully.

Examples

Using throw()

The following example shows a generator and an error that is thrown using the `throw` method. An error can be caught by a `try...catch` block as usual.

```
function* gen() {
  while (true) {
    try {
      yield 42;
    } catch (e) {
      console.log("Error caught!");
    }
  }
}

const g = gen();
g.next();
// { value: 42, done: false }
g.throw(new Error("Something went wrong"));
// "Error caught!"
// { value: 42, done: false }
```

See also

- `function*`

GeneratorFunction

The **`GeneratorFunction`** object provides methods for generator functions. In JavaScript, every generator function is actually a `GeneratorFunction` object.

Note that `GeneratorFunction` is *not* a global object. It can be obtained with the following code:

```
const GeneratorFunction = function* () {}.constructor;
```

`GeneratorFunction` is a subclass of `Function`.

Try it

```
const GeneratorFunction = function* () {}.constructor;

const foo = new GeneratorFunction(`
  yield 'a';
  yield 'b';
  yield 'c';
`);

let str = "";
for (const val of foo()) {
  str += val;
}

console.log(str);
// Expected output: "abc"
```

Constructor

`GeneratorFunction()`

> Creates a new `GeneratorFunction` object.

Instance properties

Also inherits instance properties from its parent `Function`.

These properties are defined on `GeneratorFunction.prototype` and shared by all `GeneratorFunction` instances.

`GeneratorFunction.prototype.constructor`

> The constructor function that created the instance object. For `GeneratorFunction` instances, the initial value is the `GeneratorFunction` constructor.

`GeneratorFunction.prototype.prototype`

> All generator functions share the same `prototype` property, which is `Generator.prototype`. Each generator function created with the `function*` syntax or the `GeneratorFunction()` constructor also has its own `prototype` property, whose prototype is `GeneratorFunction.prototype.prototype`. When the generator function is called, its `prototype` property becomes the prototype of the returned generator object.

`GeneratorFunction.prototype[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"GeneratorFunction"`. This property is used in `Object.prototype.toString()`.

These properties are own properties of each `GeneratorFunction` instance.

`prototype`

> Used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

Instance methods

Inherits instance methods from its parent `Function`.

See also

- `function*`
- `function*` expression
- `Function`
- `AsyncFunction`
- `AsyncGeneratorFunction`
- Functions

GeneratorFunction() constructor

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations in the `Function()` constructor reference for more information.

The **`GeneratorFunction()`** constructor creates `GeneratorFunction` objects.

Note that `GeneratorFunction` is *not* a global object. It can be obtained with the following code:

```
const GeneratorFunction = function* () {}.constructor;
```

The `GeneratorFunction()` constructor is not intended to be used directly, and all caveats mentioned in the `Function()` description apply to `GeneratorFunction()`.

Syntax

```
new GeneratorFunction(functionBody)
new GeneratorFunction(arg1, functionBody)
new GeneratorFunction(arg1, arg2, functionBody)
new GeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)

GeneratorFunction(functionBody)
GeneratorFunction(arg1, functionBody)
GeneratorFunction(arg1, arg2, functionBody)
GeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `GeneratorFunction()` can be called with or without `new`. Both create a new `GeneratorFunction` instance.

Parameters

See `Function()`.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Creating and using a GeneratorFunction() constructor

```
const GeneratorFunction = function* () {}.constructor;
const g = new GeneratorFunction("a", "yield a * 2");
const iterator = g(10);
console.log(iterator.next().value); // 20
```

See also

- `function*`
- `function*` expression
- `Function()` constructor
- Iterators and generators guide
- Functions

GeneratorFunction.prototype.prototype

The **`prototype`** property of `GeneratorFunction.prototype` is shared by all generator functions. Its value is `Generator.prototype`. Each generator function created with the `function*` syntax or the `GeneratorFunction()` constructor also has its own `prototype` property, whose prototype is `GeneratorFunction.prototype.prototype`. When the generator function is called, its `prototype` property becomes the prototype of the returned generator object.

Value

The same object as `Generator.prototype`. `GeneratorFunction.prototype.prototype` is the technically more accurate name, but `Generator.prototype` appeals to the intuition that it's the prototype of generator objects.

Property attributes of `GeneratorFunction.prototype.prototype`	
Writable	yes
Enumerable	no
Configurable	no

The `prototype` property of each `GeneratorFunction` instance is an empty object with no properties, whose prototype is `GeneratorFunction.prototype.prototype`. It has the following property attributes:

Property attributes of `GeneratorFunction.prototype.prototype`	
Writable	no
Enumerable	no
Configurable	yes

Description

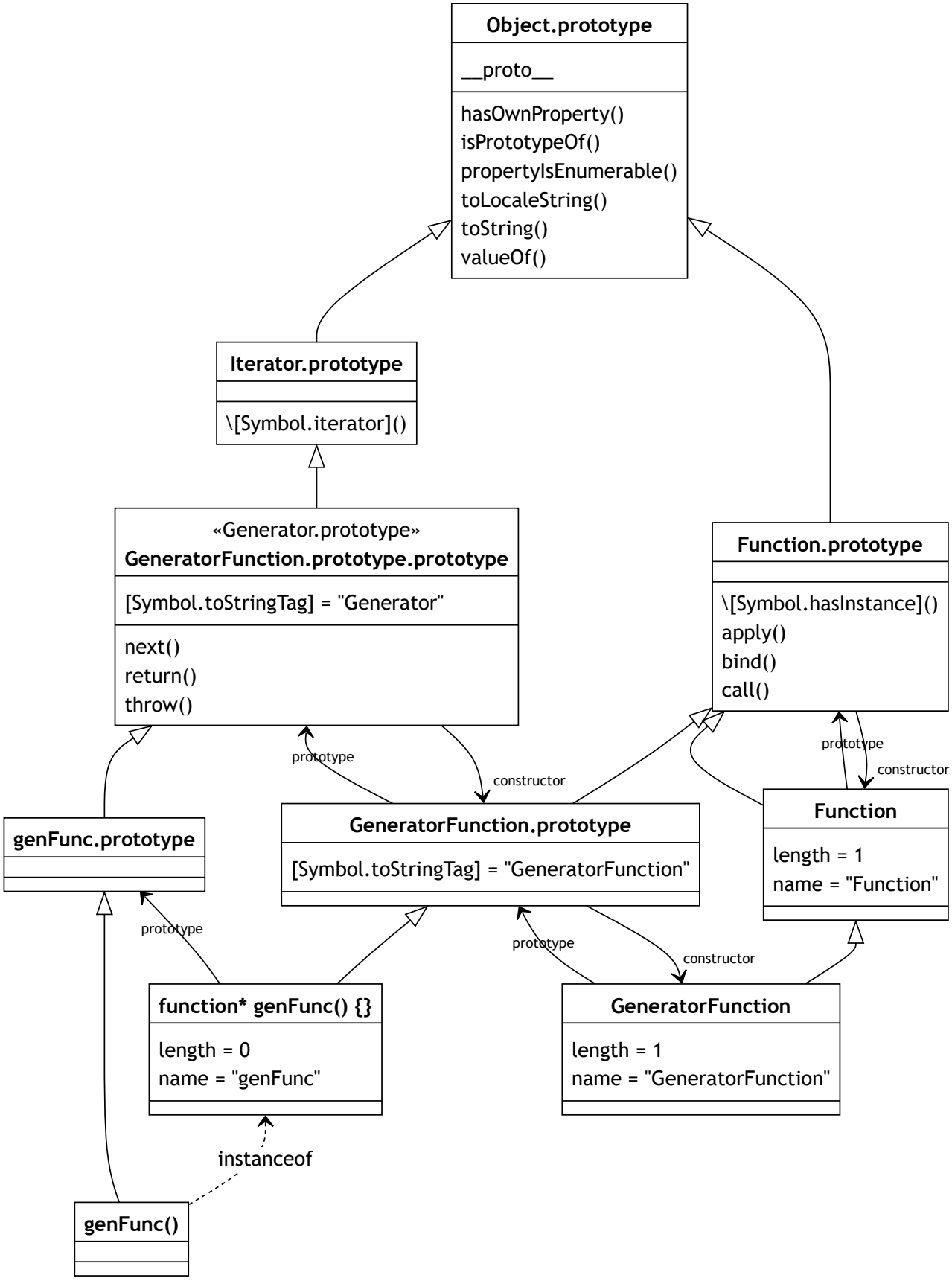
A generator function instance has two `prototype` properties. The first one is its own `prototype` property. The second one is the `prototype` property on its prototype, which is `GeneratorFunction.prototype`. (Remember that every generator function is an instance of `GeneratorFunction`, so it has `GeneratorFunction.prototype` as its prototype.)

```
function* genFunc() {}
const GeneratorFunctionPrototype = Object.getPrototypeOf(genFunc);
console.log(Object.hasOwn(genFunc, "prototype")); // true
console.log(Object.hasOwn(GeneratorFunctionPrototype, "prototype")); // true
```

When a generator function is called, the generator function's `prototype` property becomes the prototype of the returned generator object.

```
const gen = genFunc();
const proto = Object.getPrototypeOf;
console.log(proto(gen) === genFunc.prototype); // true
console.log(proto(proto(gen)) === GeneratorFunctionPrototype.prototype); // true
```

The following diagram illustrates the prototype chain of a generator function and its instances. Each hollow arrow indicates an inheritance relationship (i.e., a prototype link), and each solid arrow indicates a property relationship. Note that there's no way to access `genFunc` from `gen` — they only have an `instanceof` relationship.



See also

- `function*`
- `function*` expression
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Inheritance and the prototype chain
- Iterators and generators

globalThis

The **`globalThis`** global property contains the global `this` value, which is usually akin to the global object.

Try it

```
function canMakeHTTPRequest() {
  return typeof globalThis.XMLHttpRequest === "function";
}

console.log(canMakeHTTPRequest());
// Expected output (in a browser): true
```

Value

The global `this` object.

Property attributes of `globalThis`	
Writable	yes
Enumerable	no
Configurable	yes

> **Note:** The `globalThis` property is configurable and writable so that code authors can hide it when executing untrusted code and prevent exposing the global object.

Description

Historically, accessing the global object has required different syntax in different JavaScript environments. On the web you can use `window`, `self`, or `frames` - but in Web Workers only `self` will work. In Node.js none of these work, and you must instead use `global`. The `this` keyword could be used inside functions running in non–strict mode, but `this` will be `undefined` in modules and inside functions running in strict mode. You can also use `Function('return this')()`, but environments that disable `eval()`, like CSP in browsers, prevent use of `Function` in this way.

The `globalThis` property provides a standard way of accessing the global `this` value (and hence the global object itself) across environments. Unlike similar properties such as `window` and `self`, it's guaranteed to work in window and non-window contexts. In this way, you can access the global object in a consistent manner without having to know which environment the code is being run in. To help you remember the name, just remember that in global scope the `this` value is `globalThis`.

> **Note:** `globalThis` is generally the same concept as the global object (i.e., adding properties to `globalThis` makes them global variables) — this is the case for browsers and Node — but hosts are allowed to provide a different value for `globalThis` that's unrelated to the global object.

HTML and the WindowProxy

In many engines `globalThis` will be a reference to the actual global object, but in web browsers, due to iframe and cross-window security considerations, it references a `Proxy` around the actual global object (which you can't directly access). This distinction is rarely relevant in common usage, but important to be aware of.

Naming

Several other popular name choices such as `self` and `global` were removed from consideration because of their potential to break compatibility with existing code. See the language proposal's "naming" document for more details.

`globalThis` is, quite literally, the global `this` value. It's the same value as the `this` value in a non-strict function called without an object. It's also the value of `this` in the global scope of a script.

Examples

Search for the global across environments

Usually, the global object does not need to be explicitly specified — its properties are automatically accessible as global variables.

```
console.log(window.Math === Math); // true
```

However, one case where one needs to explicitly access the global object is when *writing* to it, usually for the purpose of polyfills.

Prior to `globalThis`, the only reliable cross-platform way to get the global object for an environment was `Function('return this')()`. However, this causes CSP violations in some settings, so authors would use a piecewise definition like this (slightly adapted from the original core-js source):

```
function check(it) {
  // Math is known to exist as a global in every environment.
  return it && it.Math === Math && it;
}

const globalObject =
  check(typeof window === "object" && window) ||
  check(typeof self === "object" && self) ||
  check(typeof global === "object" && global) ||
  // This returns undefined when running in strict mode
  (function () {
    return this;
```

```
  })() ||
  Function("return this")();
```

After obtaining the global object, we can define new globals on it. For example, adding an implementation for `Intl`:

```
if (typeof globalObject.Intl === "undefined") {
  // No Intl in this environment; define our own on the global scope
  Object.defineProperty(globalObject, "Intl", {
    value: {
      // Our Intl implementation
    },
    enumerable: false,
    configurable: true,
    writable: true,
  });
}
```

With `globalThis` available, the additional search for the global across environments is not necessary anymore:

```
if (typeof globalThis.Intl === "undefined") {
  Object.defineProperty(globalThis, "Intl", {
    value: {
      // Our Intl implementation
    },
    enumerable: false,
    configurable: true,
    writable: true,
  });
}
```

See also

- Polyfill of `globalThis` in `core-js`
- es-shims polyfill of `globalThis`
- `this`

Infinity

The **`Infinity`** global property is a numeric value representing infinity.

Try it

```
const maxNumber = 10 ** 1000; // Max positive number

if (maxNumber === Infinity) {
  console.log("Let's call it Infinity!");
  // Expected output: "Let's call it Infinity!"
}

console.log(1 / maxNumber);
// Expected output: 0
```

Value

The same number value as `Number.POSITIVE_INFINITY`.

Property attributes of `Infinity`	
Writable	no
Enumerable	no
Configurable	no

Description

`Infinity` is a property of the *global object*. In other words, it is a variable in global scope.

The value `Infinity` (positive infinity) is greater than any other number.

This value behaves slightly differently than mathematical infinity; see `Number.POSITIVE_INFINITY` for details.

Examples

Using Infinity

```
console.log(Infinity); /* Infinity */
console.log(Infinity + 1); /* Infinity */
console.log(10 ** 1000); /* Infinity */
console.log(Math.log(0)); /* -Infinity */
console.log(1 / Infinity); /* 0 */
console.log(1 / 0); /* Infinity */
```

See also

- `Number.NEGATIVE_INFINITY`
- `Number.POSITIVE_INFINITY`
- `Number.isFinite`

Int16Array

The **`Int16Array`** typed array represents an array of 16-bit signed integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Int16Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Int16Array()`

Creates a new `Int16Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Int16Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of `Int16Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Int16Array.prototype` and shared by all `Int16Array` instances.

`Int16Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of an `Int16Array`.

`Int16Array.prototype.constructor`

The constructor function that created the instance object. For `Int16Array` instances, the initial value is the `Int16Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create an Int16Array

```
// From a length
const int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42
console.log(int16.length); // 2
console.log(int16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Int16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Int16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int16FromIterable = new Int16Array(iterable);
console.log(int16FromIterable);
// Int16Array [1, 2, 3]
```

See also

- Polyfill of `Int16Array` in `core-js`
- JavaScript typed arrays guide

- `TypedArray`
- `ArrayBuffer`
- `DataView`

Int16Array() constructor

The **`Int16Array()`** constructor creates `Int16Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Int16Array()
new Int16Array(length)
new Int16Array(typedArray)
new Int16Array(object)

new Int16Array(buffer)
new Int16Array(buffer, byteOffset)
new Int16Array(buffer, byteOffset, length)
```

> **Note:** `Int16Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create an Int16Array

```
// From a length
const int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42
console.log(int16.length); // 2
console.log(int16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Int16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Int16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int16FromIterable = new Int16Array(iterable);
console.log(int16FromIterable);
// Int16Array [1, 2, 3]
```

See also

- Polyfill of `Int16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Int32Array

The **`Int32Array`** typed array represents an array of 32-bit signed integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Int32Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Int32Array()`

Creates a new `Int32Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Int32Array.BYTES_PER_ELEMENT`

> Returns a number value of the element size. `4` in the case of `Int32Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Int32Array.prototype` and shared by all `Int32Array` instances.

`Int32Array.prototype.BYTES_PER_ELEMENT`

> Returns a number value of the element size. `4` in the case of an `Int32Array`.

`Int32Array.prototype.constructor`

> The constructor function that created the instance object. For `Int32Array` instances, the initial value is the `Int32Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create an Int32Array

```
// From a length
const int32 = new Int32Array(2);
int32[0] = 42;
console.log(int32[0]); // 42
console.log(int32.length); // 2
console.log(int32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Int32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Int32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int32FromIterable = new Int32Array(iterable);
console.log(int32FromIterable);
// Int32Array [1, 2, 3]
```

See also

- Polyfill of `Int32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Int32Array() constructor

The **`Int32Array()`** constructor creates `Int32Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Int32Array()
new Int32Array(length)
new Int32Array(typedArray)
new Int32Array(object)

new Int32Array(buffer)
new Int32Array(buffer, byteOffset)
new Int32Array(buffer, byteOffset, length)
```

> **Note:** `Int32Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create an Int32Array

```
// From a length
const int32 = new Int32Array(2);
int32[0] = 42;
console.log(int32[0]); // 42
console.log(int32.length); // 2
console.log(int32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Int32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Int32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int32FromIterable = new Int32Array(iterable);
console.log(int32FromIterable);
// Int32Array [1, 2, 3]
```

See also

- Polyfill of `Int32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Int8Array

The **`Int8Array`** typed array represents an array of 8-bit signed integers. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Int8Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Int8Array()`

Creates a new `Int8Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Int8Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of `Int8Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Int8Array.prototype` and shared by all `Int8Array` instances.

`Int8Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of an `Int8Array`.

`Int8Array.prototype.constructor`

The constructor function that created the instance object. For `Int8Array` instances, the initial value is the `Int8Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create an Int8Array

```
// From a length
const int8 = new Int8Array(2);
int8[0] = 42;
console.log(int8[0]); // 42
console.log(int8.length); // 2
console.log(int8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Int8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Int8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int8FromIterable = new Int8Array(iterable);
console.log(int8FromIterable);
// Int8Array [1, 2, 3]
```

See also

- Polyfill of `Int8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Int8Array() constructor

The **`Int8Array()`** constructor creates `Int8Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Int8Array()
new Int8Array(length)
new Int8Array(typedArray)
new Int8Array(object)

new Int8Array(buffer)
new Int8Array(buffer, byteOffset)
new Int8Array(buffer, byteOffset, length)
```

> **Note:** `Int8Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create an Int8Array

```
// From a length
const int8 = new Int8Array(2);
int8[0] = 42;
```

```
console.log(int8[0]); // 42
console.log(int8.length); // 2
console.log(int8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Int8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Int8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int8FromIterable = new Int8Array(iterable);
console.log(int8FromIterable);
// Int8Array [1, 2, 3]
```

See also

- Polyfill of `Int8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

InternalError

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`InternalError`** object indicates an error that occurred internally in the JavaScript engine.

Example cases are mostly when something is too large, e.g.:

- "too many switch cases",
- "too many parentheses in regular expression",
- "array initializer too large",
- "too much recursion".

`InternalError` is a subclass of `Error`.

Constructor

`InternalError()`

Creates a new `InternalError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `InternalError.prototype` and shared by all `InternalError` instances.

`InternalError.prototype.constructor`

The constructor function that created the instance object. For `InternalError` instances, the initial value is the `InternalError` constructor.

`InternalError.prototype.name`

Represents the name for the type of error. For `InternalError.prototype.name`, the initial value is `"InternalError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Too much recursion

This recursive function runs 10 times, as per the exit condition.

```
function loop(x) {
  // "x >= 10" is the exit condition
  if (x >= 10) return;
```

```
  // do stuff
  loop(x + 1); // the recursive call
}
loop(0);
```

Setting this condition to an extremely high value, may not work:

```
function loop(x) {
  if (x >= 1000000000000) return;

  // do stuff
  loop(x + 1);
}
loop(0);

// InternalError: too much recursion
```

For more information, see InternalError: too much recursion.

Specifications

Not part of any standard.

See also

- `Error`
- InternalError: too much recursion

InternalError() constructor

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`InternalError()`** constructor creates `InternalError` objects.

Syntax

```
new InternalError()
new InternalError(message)
new InternalError(message, options)
new InternalError(message, fileName)
new InternalError(message, fileName, lineNumber)

InternalError()
InternalError(message)
InternalError(message, options)
InternalError(message, fileName)
InternalError(message, fileName, lineNumber)
```

> **Note:** `InternalError()` can be called with or without `new`. Both create a new `InternalError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception

`lineNumber` Optional

The line number of the code that caused the exception

Examples

Creating a new InternalError

```
new InternalError("Engine failure");
```

Specifications

Not part of any standard.

See also

- `Error`

Intl

The **`Intl`** namespace object contains several constructors as well as functionality common to the internationalization constructors and other language sensitive functions. Collectively, they comprise the ECMAScript Internationalization API, which provides language sensitive string comparison, number formatting, date and time formatting, and more.

Description

Unlike most global objects, `Intl` is not a constructor. You cannot use it with the `new` operator or invoke the `Intl` object as a function. All properties and methods of `Intl` are static (just like the `Math` object).

The internationalization constructors as well as several language sensitive methods of other constructors (listed under See also) use a common pattern for identifying locales and determining the one they will actually use: they all accept `locales` and `options` arguments, and negotiate the requested locale(s) against the locales they support using an algorithm specified in the `options.localeMatcher` property.

locales argument

The `locales` argument is used to determine the locale used in a given operation. The JavaScript implementation examines `locales`, and then computes a locale it understands that comes closest to satisfying the expressed preference. `locales` may be:

- `undefined` (or omitted): The implementation's default locale will be used.
- A locale: A locale identifier or an `Intl.Locale` object that wraps a locale identifier.
- A list of locales: Any other value, that will be converted into an object and then treated as an array of locales.

In the latter two cases, the actual locale used is the best-supported locale determined by locale negotiation. If a locale identifier is not a string or an object, a `TypeError` is thrown. If a locale identifier is a string that's syntactically invalid, a `RangeError` is thrown. If a locale identifier is well-formed but the implementation doesn't recognize it, it is ignored and the next locale in the list is considered, eventually falling back to the system's locale. However, you shouldn't rely on a particular locale name being ignored, because the implementation may add data for any locale in the future. For example, `new Intl.DateTimeFormat("default")` uses the implementation's default locale only because `"default"` is syntactically valid but not recognized as any locale.

A locale identifier is a string that consists of:

1. A language subtag with 2–3 or 5–8 letters
2. A script subtag with 4 letters Optional
3. A region subtag with either 2 letters or 3 digits Optional
4. One or more variant subtags (all of which must be unique), each with either 5–8 alphanumerals or a digit followed by 3 alphanumerals Optional
5. One or more BCP 47 extension sequences Optional
6. A private-use extension sequence Optional

Each subtag and sequence are separated by hyphens. Locale identifiers are case-insensitive ASCII. However, it's conventional to use title case (the first letter is capitalized, successive letters are lower case) for script subtags, upper case for region subtags, and lower case for everything else. For example:

- `"hi"`: Hindi (language)
- `"de-AT"`: German (language) as used in Austria (region)
- `"zh-Hans-CN"`: Chinese (language) written in simplified characters (script) as used in China (region)
- `"en-emodeng"`: English (language) in the "Early modern English" dialect (variant)

Subtags identifying languages, scripts, regions (including countries), and (rarely used) variants are registered in the IANA Language Subtag Registry. This registry is periodically updated over time, and implementations may not always be up to date, so don't rely too much on subtags being universally supported.

BCP 47 extension sequences consist of a single digit or letter (other than `"x"`) and one or more two- to eight-letter or digit subtags separated by hyphens. Only one sequence is permitted for each digit or letter: `"de-a-foo-a-foo"` is invalid. BCP 47 extension subtags are defined in the Unicode CLDR Project. Currently only two extensions have defined semantics:

- The `"u"` (Unicode) extension can be used to request additional customization of `Intl` API objects. Examples:
  - `"de-DE-u-co-phonebk"`: Use the phonebook variant of the German sort order, which interprets umlauted vowels as corresponding character pairs: ä → ae, ö → oe, ü → ue.
  - `"th-TH-u-nu-thai"`: Use Thai digits (๐, ๑, ๒, ๓, ๔, ๕, ๖, ๗, ๘, ๙) in number formatting.
  - `"ja-JP-u-ca-japanese"`: Use the Japanese calendar in date and time formatting, so that 2013 is expressed as the year 25 of the Heisei period, or 平成 25.
  - `"en-GB-u-ca-islamic-umalqura"`: use British English with the Umm al-Qura (Hijri) calendar, where the Gregorian date 14 October, 2017 is the Hijri date 24 Muharram, 1439.
- The `"t"` (transformed) extension indicates transformed content: for example, text that was translated from another locale. No `Intl` functionality currently considers the `"t"` extension. However, this extension sometimes contains a nested locale (with no extensions): for example, the transformed extension in `"de-t-en"` contains the locale identifier for English. If a nested locale is present, it must be a valid locale identifier. For example, because `"en-emodeng-emodeng"` is invalid (because it contains a duplicate `emodeng` variant subtag), `"de-t-en-emodeng-emodeng"` is also invalid.

Finally, a private-use extension sequence using the letter `"x"` may appear, followed by one or more one- to eight-letter or digit subtags separated by hyphens. This allows applications to encode information for their own private use, that will be ignored by all `Intl` operations.

options argument

The `options` argument must be an object with properties that vary between constructors and functions. If the `options` argument is not provided or is undefined, default values are used for all properties.

One property is supported by all language sensitive constructors and functions: The `localeMatcher` property, whose value must be a string `"lookup"` or `"best fit"` and which selects one of the locale matching algorithms described below.

Locale identification and negotiation

The list of locales specified by the `locales` argument, after Unicode extensions have been removed from them, is interpreted as a prioritized request from the application. The runtime compares it against the locales it has available and picks the best one available. Two matching algorithms exist: the `"lookup"` matcher follows the Lookup algorithm specified in BCP 47; the `"best fit"` matcher lets the runtime provide a locale that's at least, but possibly more, suited for the request than the result of the Lookup algorithm. If the application doesn't provide a `locales` argument, or the runtime doesn't have a locale that matches the request, then the runtime's default locale is used. The matcher can be selected using a property of the `options` argument (see below).

If the selected locale identifier had a Unicode extension sequence, that extension is now used to customize the constructed object or the behavior of the function. Each constructor or function supports only a subset of the keys defined for the Unicode extension, and the supported values often depend on the locale identifier. For example, the `"co"` key (collation) is only supported by `Intl.Collator`, and its `"phonebk"` value is only supported for German.

Static properties

`Intl.Collator`

: Constructor for collators, which are objects that enable language-sensitive string comparison.

`Intl.DateTimeFormat`

: Constructor for objects that enable language-sensitive date and time formatting.

`Intl.DisplayNames`

: Constructor for objects that enable the consistent translation of language, region and script display names.

`Intl.DurationFormat`

: Constructor for objects that enable locale-sensitive duration formatting.

`Intl.ListFormat`

: Constructor for objects that enable language-sensitive list formatting.

`Intl.Locale`

: Constructor for objects that represents a Unicode locale identifier.

`Intl.NumberFormat`

: Constructor for objects that enable language-sensitive number formatting.

`Intl.PluralRules`

: Constructor for objects that enable plural-sensitive formatting and language-specific rules for plurals.

`Intl.RelativeTimeFormat`

: Constructor for objects that enable language-sensitive relative time formatting.

`Intl.Segmenter`

: Constructor for objects that enable locale-sensitive text segmentation.

`Intl[Symbol.toStringTag]`

: The initial value of the `[Symbol.toStringTag]` property is the string `"Intl"`. This property is used in `Object.prototype.toString()`.

Static methods

`Intl.getCanonicalLocales()`

: Returns canonical locale names.

`Intl.supportedValuesOf()`

: Returns a sorted array containing the supported unique calendar, collation, currency, numbering systems, or unit values supported by the implementation.

Examples

Formatting dates and numbers

You can use `Intl` to format dates and numbers in a form that's conventional for a specific language and region:

```
const count = 26254.39;
const date = new Date("2012-05-24");

function log(locale) {
  console.log(
```

```
    `${new Intl.DateTimeFormat(locale).format(date)} ${new Intl.NumberFormat(
      locale,
    ).format(count)}`,
  );
}

log("en-US"); // 5/24/2012 26,254.39

log("de-DE"); // 24.5.2012 26.254,39
```

Using the browser's preferred language

Instead of passing a hardcoded locale name to the `Intl` methods, you can use the user's preferred language provided by `navigator.language`:

```
const date = new Date("2012-05-24");

const formattedDate = new Intl.DateTimeFormat(navigator.language).format(date);
```

Alternatively, the `navigator.languages` property provides a sorted list of the user's preferred languages. This list can be passed directly to the `Intl` constructors to implement preference-based fallback selection of locales. The locale negotiation process is used to pick the most appropriate locale available:

```
const count = 26254.39;

const formattedCount = new Intl.NumberFormat(navigator.languages).format(count);
```

See also

- `Keyboard.getLayoutMap()`
- `navigator.language`
- `navigator.languages`
- The ECMAScript Internationalization API by Norbert Lindenberg (2012)

Intl.Collator

The **`Intl.Collator`** object enables language-sensitive string comparison.

Try it

```
console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("de").compare));
// Expected output: Array ["a", "ä", "z", "Z"]

console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("sv").compare));
// Expected output: Array ["a", "z", "Z", "ä"]

console.log(
  ["Z", "a", "z", "ä"].sort(
    new Intl.Collator("de", { caseFirst: "upper" }).compare,
  ),
);
// Expected output: Array ["a", "ä", "Z", "z"]
```

Constructor

`Intl.Collator()`

Creates a new `Collator` object.

Static methods

`Intl.Collator.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.Collator.prototype` and shared by all `Intl.Collator` instances.

`Intl.Collator.prototype.constructor`

The constructor function that created the instance object. For `Intl.Collator` instances, the initial value is the `Intl.Collator` constructor.

`Intl.Collator.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.Collator"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.Collator.prototype.compare()`

Getter function that compares two strings according to the sort order of this `Intl.Collator` object.

`Intl.Collator.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

Examples

Using Collator

The following example demonstrates the different potential results for a string occurring before, after, or at the same level as another:

```
console.log(new Intl.Collator().compare("a", "c")); // -1, or some other negative value
console.log(new Intl.Collator().compare("c", "a")); // 1, or some other positive value
console.log(new Intl.Collator().compare("a", "a")); // 0
```

Note that the results shown in the code above can vary between browsers and browser versions. This is because the values are implementation-specific. That is, the specification requires only that the before and after values are negative and positive.

Using locales

The results provided by `Intl.Collator.prototype.compare()` vary between languages. In order to get the sort order of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
// in German, ä sorts with a
console.log(new Intl.Collator("de").compare("ä", "z"));
// -1, or some other negative value

// in Swedish, ä sorts after z
console.log(new Intl.Collator("sv").compare("ä", "z"));
// 1, or some other positive value
```

Using options

The results provided by `Intl.Collator.prototype.compare()` can be customized using the `options` argument:

```
// in German, ä has a as the base letter
console.log(new Intl.Collator("de", { sensitivity: "base" }).compare("ä", "a"));
// 0

// in Swedish, ä and a are separate base letters
console.log(new Intl.Collator("sv", { sensitivity: "base" }).compare("ä", "a"));
// 1, or some other positive value
```

See also

- `Intl`
- `String.prototype.localeCompare()`

Intl.Collator() constructor

The **`Intl.Collator()`** constructor creates `Intl.Collator` objects.

Try it

```
console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("de").compare));
// Expected output: Array ["a", "ä", "z", "Z"]

console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("sv").compare));
// Expected output: Array ["a", "z", "Z", "ä"]

console.log(
  ["Z", "a", "z", "ä"].sort(
    new Intl.Collator("de", { caseFirst: "upper" }).compare,
  ),
);
// Expected output: Array ["a", "ä", "Z", "z"]
```

Syntax

```
new Intl.Collator()
new Intl.Collator(locales)
new Intl.Collator(locales, options)

Intl.Collator()
Intl.Collator(locales)
Intl.Collator(locales, options)
```

> **Note:** `Intl.Collator()` can be called with or without `new`. Both create a new `Intl.Collator` instance.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter

description on the `Intl` main page.

The following Unicode extension keys are allowed:

`co`

See `collation`.

`kn`

See `numeric`.

`kf`

See `caseFirst`.

These keys can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`usage`

Whether the comparison is for sorting a list of strings or fuzzy (for the Latin script diacritic-insensitive and case-insensitive) filtering a list of strings by key. Possible values are:

`"sort"` (default)

For sorting a list of strings.

`"search"`

For filtering a list of strings by testing each list item for a full-string match against a key. With `"search"`, the caller should only pay attention to whether `compare()` returns zero or non-zero and should not distinguish the non-zero return values from each other. That is, it is inappropriate to use `"search"` for sorting/ordering.

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`collation`

Variant collations for certain locales, such as `"emoji"`, `"pinyin"`, `"stroke"`, and so on. Only has an effect when `usage` is `"sort"` (because `"search"` is underlyingly its own collation type). For a list of supported collation types, see `Intl.supportedValuesOf()`; the default is `"default"`. This option can also be set through the `co` Unicode extension key; if both are provided, this `options` property takes precedence.

`numeric`

Whether numeric collation should be used, such that "1" < "2" < "10". Possible values are `true` and `false`; the default is `false`. This option can also be set through the `kn` Unicode extension key; if both are provided, this `options` property takes precedence.

`caseFirst`

Whether upper case or lower case should sort first. Possible values are `"upper"`, `"lower"`, and `"false"` (use the locale's default); the default is `"false"`. This option can also be set through the `kf` Unicode extension key; if both are provided, this `options` property takes precedence.

`sensitivity`

Which differences in the strings should lead to non-zero result values. Possible values are:

`"base"`

Only strings that differ in base letters compare as unequal. Examples: a ≠ b, a = á, a = A. In the Unicode collation algorithm, this is equivalent to the primary strength level.

`"accent"`

Only strings that differ in base letters or accents and other diacritic marks compare as unequal. Examples: a ≠ b, a ≠ á, a = A. In the Unicode collation algorithm, this is equivalent to the secondary strength level.

`"case"`

Only strings that differ in base letters or case compare as unequal. Examples: a ≠ b, a = á, a ≠ A. In the Unicode collation algorithm, this is equivalent to the primary strength level with case level handling.

`"variant"`

Strings that differ in base letters, accents and other diacritic marks, or case compare as unequal. Other differences may also be taken into consideration. Examples: a ≠ b, a ≠ á, a ≠ A. In the Unicode collation algorithm, this is equivalent to the tertiary strength level.

The default is `"variant"` for usage `"sort"`; it's locale dependent for usage `"search"` per spec, but is usually also `"variant"`. Because the core functionality of `"search"` is accent-insensitive and case-insensitive filtering, setting it to `"base"` makes the most sense (and perhaps `"case"`).

`ignorePunctuation`

Whether punctuation should be ignored. Possible values are `true` and `false`. The default is `true` for Thai (`th`) and `false` for all other languages.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Using Collator

The following example demonstrates the different potential results for a string occurring before, after, or at the same level as another:

```
console.log(new Intl.Collator().compare("a", "c")); // -1, or some other negative value
console.log(new Intl.Collator().compare("c", "a")); // 1, or some other positive value
console.log(new Intl.Collator().compare("a", "a")); // 0
```

Note that the results shown in the code above can vary between browsers and browser versions. This is because the values are implementation-specific. That is, the specification requires only that the before and after values are negative and positive.

See also

- `Intl.Collator`
- `Intl`

Intl.Collator.prototype.compare()

The **`compare()`** method of `Intl.Collator` instances compares two strings according to the sort order of this collator object.

Try it

```
const enCollator = new Intl.Collator("en");
const deCollator = new Intl.Collator("de");
const svCollator = new Intl.Collator("sv");

console.log(enCollator.compare("z", "a") > 0);
// Expected output: true

console.log(deCollator.compare("z", "ä") > 0);
// Expected output: true

console.log(svCollator.compare("z", "ä") > 0);
// Expected output: false
```

Syntax

```
compare(string1, string2)
```

Parameters

`string1`, `string2`

The strings to compare against each other.

Return value

A number indicating how `string1` and `string2` compare to each other according to the sort order of this `Intl.Collator` object:

- A negative value if `string1` comes before `string2`;
- A positive value if `string1` comes after `string2`;
- 0 if they are considered equal.

Examples

Using compare for array sort

Use the `compare` function for sorting arrays. Note that the function is bound to the collator from which it was obtained, so it can be passed directly to `Array.prototype.sort()`.

```
const a = ["Offenbach", "Österreich", "Odenwald"];
const collator = new Intl.Collator("de-u-co-phonebk");
a.sort(collator.compare);
console.log(a.join(", ")); // "Odenwald, Österreich, Offenbach"
```

Using compare for array search

Use the `compare` function for finding matching strings in arrays:

```
const a = ["Congrès", "congres", "Assemblée", "poisson"];
const collator = new Intl.Collator("fr", {
```

```
  usage: "search",
  sensitivity: "base",
});
const s = "congres";
const matches = a.filter((v) => collator.compare(v, s) === 0);
console.log(matches.join(", ")); // "Congrès, congres"
```

See also

- `Intl.Collator`
- `String.prototype.localeCompare()`

Intl.Collator.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.Collator` instances returns a new object with properties reflecting the options computed during initialization of this `Collator` object.

Try it

```
const numberDe = new Intl.NumberFormat("de-DE");
const numberAr = new Intl.NumberFormat("ar");

console.log(numberDe.resolvedOptions().numberingSystem);
// Expected output: "latn"

console.log(numberAr.resolvedOptions().numberingSystem);
// Expected output: "arab"
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `Collator` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `co`, `kn`, and `kf` Unicode extension keys, if requested and supported, may be included in the output.

`usage`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"sort"` or `"search"`. The default is `"sort"`.

`sensitivity`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"base"`, `"accent"`, `"case"`, or `"variant"`. The default is `"variant"` for usage `"sort"`; it's locale dependent for usage `"search"`.

`ignorePunctuation`

> The value provided for this property in the `options` argument, with default filled in as needed. It is a boolean. The default is `true` for Thai (`th`) and `false` for all other languages.

`collation`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"co"`, with default filled in as needed. It is a supported collation type for this locale. The default is `"default"`.

`numeric`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"kn"`, with default filled in as needed. It is a boolean. The default is `false`. If the implementation does not support this Unicode extension key, this property is omitted.

`caseFirst`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"kf"`, with default filled in as needed. It is either `"upper"`, `"lower"`, or `"false"`. The default is `"false"`. If the implementation does not support this Unicode extension key, this property is omitted.

Examples

Using the resolvedOptions method

```
const de = new Intl.Collator("de", { sensitivity: "base" });
const usedOptions = de.resolvedOptions();

usedOptions.locale; // "de"
```

```
usedOptions.usage; // "sort"
usedOptions.sensitivity; // "base"
usedOptions.ignorePunctuation; // false
usedOptions.collation; // "default"
usedOptions.numeric; // false
```

See also

- `Intl.Collator`

Intl.Collator.supportedLocalesOf()

The **`Intl.Collator.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in collation without having to fall back to the runtime's default locale.

Try it

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.Collator.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

```
Intl.Collator.supportedLocalesOf(locales)
Intl.Collator.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in collation without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in collation, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is not used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.Collator.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.Collator`

Intl.DateTimeFormat

The **`Intl.DateTimeFormat`** object enables language-sensitive date and time formatting.

Try it

```
const date = new Date(Date.UTC(2020, 11, 20, 3, 23, 16, 738));
// Results below assume UTC timezone - your results may vary

// Specify default date formatting for language (locale)
console.log(new Intl.DateTimeFormat("en-US").format(date));
// Expected output: "12/20/2020"

// Specify default date formatting for language with a fallback language (in this case Indonesian)
console.log(new Intl.DateTimeFormat(["ban", "id"]).format(date));
// Expected output: "20/12/2020"
```

```
// Specify date and time format using "style" options (i.e. full, long, medium, short)
console.log(
  new Intl.DateTimeFormat("en-GB", {
    dateStyle: "full",
    timeStyle: "long",
    timeZone: "Australia/Sydney",
  }).format(date),
);
// Expected output: "Sunday, 20 December 2020 at 14:23:16 GMT+11"
```

Constructor

`Intl.DateTimeFormat()`

Creates a new `Intl.DateTimeFormat` object.

Static methods

`Intl.DateTimeFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.DateTimeFormat.prototype` and shared by all `Intl.DateTimeFormat` instances.

`Intl.DateTimeFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.DateTimeFormat` instances, the initial value is the `Intl.DateTimeFormat` constructor.

`Intl.DateTimeFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.DateTimeFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.DateTimeFormat.prototype.format()`

Getter function that formats a date according to the locale and formatting options of this `DateTimeFormat` object.

`Intl.DateTimeFormat.prototype.formatRange()`

This method receives two Dates and formats the date range in the most concise way based on the locale and options provided when instantiating `DateTimeFormat`.

`Intl.DateTimeFormat.prototype.formatRangeToParts()`

This method receives two Dates and returns an Array of objects containing the locale-specific tokens representing each part of the formatted date range.

`Intl.DateTimeFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the date string in parts that can be used for custom locale-aware formatting.

`Intl.DateTimeFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Using DateTimeFormat

In basic use without specifying a locale, `DateTimeFormat` uses the default locale and default options.

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// toLocaleString without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(new Intl.DateTimeFormat().format(date));
// "12/19/2012" if run with en-US locale (language) and time zone America/Los_Angeles (UTC-0800)
```

Using locales

This example shows some of the variations in localized date and time formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Results below use the time zone of America/Los_Angeles (UTC-0800, Pacific Standard Time)

// US English uses month-day-year order
console.log(new Intl.DateTimeFormat("en-US").format(date));
// "12/19/2012"

// British English uses day-month-year order
```

```
console.log(new Intl.DateTimeFormat("en-GB").format(date));
// "19/12/2012"

// Korean uses year-month-day order
console.log(new Intl.DateTimeFormat("ko-KR").format(date));
// "2012. 12. 19."

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(new Intl.DateTimeFormat("ar-EG").format(date));
// "٢٠١٢/١٢/١٩"

// for Japanese, applications may want to use the Japanese calendar,
// where 2012 was the year 24 of the Heisei era
console.log(new Intl.DateTimeFormat("ja-JP-u-ca-japanese").format(date));
// "24/12/19"

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(new Intl.DateTimeFormat(["ban", "id"]).format(date));
// "19/12/2012"
```

Using options

The date and time formats can be customized using the `options` argument:

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0, 200));

// request a weekday along with a long date
let options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(new Intl.DateTimeFormat("de-DE", options).format(date));
// "Donnerstag, 20. Dezember 2012"

// an application may want to use UTC and make that visible
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(new Intl.DateTimeFormat("en-US", options).format(date));
// "Thursday, December 20, 2012, GMT"

// sometimes you want to be more precise
options = {
  hour: "numeric",
  minute: "numeric",
  second: "numeric",
  timeZone: "Australia/Sydney",
  timeZoneName: "short",
};
console.log(new Intl.DateTimeFormat("en-AU", options).format(date));
// "2:00:00 pm AEDT"

// sometimes you want to be very precise
options.fractionalSecondDigits = 3; // number digits for fraction-of-seconds
console.log(new Intl.DateTimeFormat("en-AU", options).format(date));
// "2:00:00.200 pm AEDT"

// sometimes even the US needs 24-hour time
options = {
  year: "numeric",
  month: "numeric",
  day: "numeric",
  hour: "numeric",
  minute: "numeric",
  second: "numeric",
  hour12: false,
  timeZone: "America/Los_Angeles",
};
console.log(new Intl.DateTimeFormat("en-US", options).format(date));
// "12/19/2012, 19:00:00"

// to specify options but use the browser's default locale, use undefined
console.log(new Intl.DateTimeFormat(undefined, options).format(date));
// "12/19/2012, 19:00:00"

// sometimes it's helpful to include the period of the day
options = { hour: "numeric", dayPeriod: "short" };
console.log(new Intl.DateTimeFormat("en-US", options).format(date));
// 10 at night
```

The used calendar and numbering formats can also be set independently via `options` arguments:

```
const options = { calendar: "chinese", numberingSystem: "arab" };
const dateFormat = new Intl.DateTimeFormat(undefined, options);
const usedOptions = dateFormat.resolvedOptions();

console.log(usedOptions.calendar);
// "chinese"

console.log(usedOptions.numberingSystem);
// "arab"

console.log(usedOptions.timeZone);
// "America/New_York" (the users default timezone)
```

See also

- Polyfill of `Intl.DateTimeFormat` in FormatJS
- `Intl`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleTimeString()`
- `Temporal.Instant.prototype.toLocaleString()`
- `Temporal.PlainDate.prototype.toLocaleString()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`
- `Temporal.PlainTime.prototype.toLocaleString()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

Intl.DateTimeFormat() constructor

The **`Intl.DateTimeFormat()`** constructor creates `Intl.DateTimeFormat` objects.

Try it

```
const date = new Date(Date.UTC(2020, 11, 20, 3, 23, 16, 738));
// Results below assume UTC timezone - your results may vary

// Specify default date formatting for language (locale)
console.log(new Intl.DateTimeFormat("en-US").format(date));
// Expected output: "12/20/2020"

// Specify default date formatting for language with a fallback language (in this case Indonesian)
console.log(new Intl.DateTimeFormat(["ban", "id"]).format(date));
// Expected output: "20/12/2020"

// Specify date and time format using "style" options (i.e. full, long, medium, short)
console.log(
  new Intl.DateTimeFormat("en-GB", {
    dateStyle: "full",
    timeStyle: "long",
    timeZone: "Australia/Sydney",
  }).format(date),
);
// Expected output: "Sunday, 20 December 2020 at 14:23:16 GMT+11"
```

Syntax

```
new Intl.DateTimeFormat()
new Intl.DateTimeFormat(locales)
new Intl.DateTimeFormat(locales, options)

Intl.DateTimeFormat()
Intl.DateTimeFormat(locales)
Intl.DateTimeFormat(locales, options)
```

> **Note:** `Intl.DateTimeFormat()` can be called with or without `new`. Both create a new `Intl.DateTimeFormat` instance. However, there's a special behavior when it's called without `new` and the `this` value is another `Intl.DateTimeFormat` instance; see Return value.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension keys are allowed:

`nu`

See `numberingSystem`.

`ca`

See `calendar`.

`hc`

See `hourCycle`.

These keys can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object. For ease of reading, the property list is broken into sections based on their purposes, including locale options, date-time component options, and style shortcuts.

Locale options

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`calendar`

The calendar to use, such as `"chinese"`, `"gregory"`, `"persian"`, and so on. For a list of supported calendar types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `ca` Unicode extension key; if both are provided, this `options` property takes precedence.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

`hour12`

Whether to use 12-hour time (as opposed to 24-hour time). Possible values are `true` and `false`; the default is locale dependent. When `true`, this option sets `hourCycle` to either `"h11"` or `"h12"`, depending on the locale. When `false`, it sets `hourCycle` to `"h23"`. `hour12` overrides both the `hc` locale extension tag and the `hourCycle` option, should either or both of those be present.

`hourCycle`

The hour cycle to use. Possible values are `"h11"`, `"h12"`, `"h23"`, and `"h24"`; the default is inferred from `hour12` and locale. This option can also be set through the `hc` Unicode extension key; if both are provided, this `options` property takes precedence.

`timeZone`

The time zone to use. Can be any IANA time zone name, including named identifiers such as `"UTC"`, `"America/New_York"`, and `"Etc/GMT+8"`, and offset identifiers such as `"+01:00"`, `"-2359"`, and `"+23"`. The default is the runtime's time zone, the same time zone used by `Date.prototype.toString()`.

Date-time component options

`weekday`

The representation of the weekday. Possible values are:

`"long"`

E.g., `Thursday`

`"short"`

E.g., `Thu`

`"narrow"`

E.g., `T`. Two weekdays may have the same narrow style for some locales (e.g., both `Tuesday`'s and `Thursday`'s narrow styles are `T` in the `en-US` locale).

`era`

The representation of the era. Possible values are:

`"long"`

E.g., `Anno Domini`

`"short"`

E.g., `AD`

`"narrow"`

E.g., `A`

`year`

The representation of the year. Possible values are `"numeric"` and `"2-digit"`.

`month`

The representation of the month. Possible values are:

`"numeric"`

E.g., `3`

`"2-digit"`

E.g., `03`

`"long"`

E.g., `March`

`"short"`

E.g., `Mar`

`"narrow"`

E.g., `M`. Two months may have the same narrow style for some locales (e.g., both `March`'s and `May`'s narrow styles are `M` in the `en-US` locale).

`day`

The representation of the day. Possible values are `"numeric"` and `"2-digit"`.

`dayPeriod`

The formatting style used for day periods like "in the morning", "am", "noon", "n" etc. Possible values are `"narrow"`, `"short"`, and `"long"`.

> **Note:** This option only has an effect if a 12-hour clock (`hourCycle: "h12"` or `hourCycle: "h11"`) is used. Many locales use the same string irrespective of the width specified.

`hour`

The representation of the hour. Possible values are `"numeric"` and `"2-digit"`.

`minute`

The representation of the minute. Possible values are `"numeric"` and `"2-digit"`.

`second`

The representation of the second. Possible values are `"numeric"` and `"2-digit"`.

`fractionalSecondDigits`

The number of digits used to represent fractions of a second (any additional digits are truncated). Possible values are from `1` to `3`.

`timeZoneName`

The localized representation of the time zone name. Possible values are:

`"long"`

Long localized form (e.g., `Pacific Standard Time`, `Nordamerikanische Westküsten-Normalzeit`)

`"short"`

Short localized form (e.g.: `PST`, `GMT-8`)

`"shortOffset"`

Short localized GMT format (e.g., `GMT-8`)

`"longOffset"`

Long localized GMT format (e.g., `GMT-08:00`)

`"shortGeneric"`

Short generic non-location format (e.g.: `PT`, `Los Angeles Zeit`).

`"longGeneric"`

Long generic non-location format (e.g.: `Pacific Time`, `Nordamerikanische Westküstenzeit`)

> **Note:** Timezone display may fall back to another format if a required string is unavailable. For example, the non-location formats should display the timezone without a specific country/city location like "Pacific Time", but may fall back to a timezone like "Los Angeles Time".

Date-time component default values

If any of the date-time component options are specified, then `dateStyle` and `timeStyle` must be `undefined`. If all date-time component options and `dateStyle`/`timeStyle` are `undefined`, some default options for date-time components are set, which depend on the object that the formatting method was called with:

- When formatting `Temporal.PlainDate` and `Date`, `year`, `month`, and `day` default to `"numeric"`.
- When formatting `Temporal.PlainTime`, `hour`, `minute`, and `second` default to `"numeric"`.
- When formatting `Temporal.PlainYearMonth`, `year` and `month` default to `"numeric"`.
- When formatting `Temporal.PlainMonthDay`, `month` and `day` default to `"numeric"`.
- When formatting `Temporal.PlainDateTime` and `Temporal.Instant`, `year`, `month`, `day`, `hour`, `minute`, and `second` default to `"numeric"`.

Format matching

Implementations are required to support displaying at least the following subsets of date-time components:

- `weekday`, `year`, `month`, `day`, `hour`, `minute`, `second`
- `weekday`, `year`, `month`, `day`
- `year`, `month`, `day`
- `year`, `month`
- `month`, `day`
- `hour`, `minute`, `second`
- `hour`, `minute`

The date-time component styles requested might not directly correspond to a valid format supported by the locale, so the format matcher allows you to specify how to match the requested styles to the closest supported format.

`formatMatcher`

> The format matching algorithm to use. Possible values are `"basic"` and `"best fit"`; the default is `"best fit"`. The algorithm for `"best fit"` is implementation-defined, and `"basic"` is defined by the spec. This option is only used when both `dateStyle` and `timeStyle` are `undefined` (so that each date-time component's format is individually customizable).

Style shortcuts

`dateStyle`

> The date formatting style to use. Possible values are `"full"`, `"long"`, `"medium"`, and `"short"`. It expands to styles for `weekday`, `day`, `month`, `year`, and `era`, with the exact combination of values depending on the locale. When formatting objects such as `Temporal.PlainDate`, `Temporal.PlainYearMonth`, and `Temporal.PlainMonthDay`, `dateStyle` will resolve to only those fields relevant to the object.

`timeStyle`

> The time formatting style to use. Possible values are `"full"`, `"long"`, `"medium"`, and `"short"`. It expands to styles for `hour`, `minute`, `second`, and `timeZoneName`, with the exact combination of values depending on the locale.

> **Note:** `dateStyle` and `timeStyle` can be used with each other, but not with other date-time component options (e.g., `weekday`, `hour`, `month`, etc.).

You can format different object types depending on which of the style shortcut options you include:

- If the `dateStyle` is specified, then you can format `Temporal.PlainDate`, `Temporal.PlainYearMonth`, and `Temporal.PlainMonthDay` objects.
- If the `timeStyle` is specified, then you can format `Temporal.PlainTime` objects.
- If either `dateStyle` or `timeStyle` is specified, then you can format `Temporal.PlainDateTime` and `Date` objects.

Return value

A new `Intl.DateTimeFormat` object.

> **Note:** The text below describes behavior that is marked by the specification as "optional". It may not work in all environments. Check the browser compatibility table.

Normally, `Intl.DateTimeFormat()` can be called with or without `new`, and a new `Intl.DateTimeFormat` instance is returned in both cases. However, if the `this` value is an object that is `instanceof` `Intl.DateTimeFormat` (doesn't necessarily mean it's created via `new Intl.DateTimeFormat`; just that it has `Intl.DateTimeFormat.prototype` in its prototype chain), then the value of `this` is returned instead, with the newly created `Intl.DateTimeFormat` object hidden in a `[Symbol(IntlLegacyConstructedSymbol)]` property (a unique symbol that's reused between instances).

```
const formatter = Intl.DateTimeFormat.call(
  { __proto__: Intl.DateTimeFormat.prototype },
  "en-US",
  { dateStyle: "full" },
);
console.log(Object.getOwnPropertyDescriptors(formatter));
// {
//   [Symbol(IntlLegacyConstructedSymbol)]: {
//     value: DateTimeFormat [Intl.DateTimeFormat] {},
//     writable: false,
//     enumerable: false,
//     configurable: false
```

```
//   }
// }
```

Note that there's only one actual `Intl.DateTimeFormat` instance here: the one hidden in `[Symbol(IntlLegacyConstructedSymbol)]`. Calling the `format()` and `resolvedOptions()` methods on `formatter` would correctly use the options stored in that instance, but calling all other methods (e.g., `formatRange()`) would fail: "TypeError: formatRange method called on incompatible Object", because those methods don't consult the hidden instance's options.

This behavior, called `ChainDateTimeFormat`, does not happen when `Intl.DateTimeFormat()` is called without `new` but with `this` set to anything else that's not an `instanceof Intl.DateTimeFormat`. If you call it directly as `Intl.DateTimeFormat()`, the `this` value is `Intl`, and a new `Intl.DateTimeFormat` instance is created normally.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Using DateTimeFormat

In basic use without specifying a locale, `DateTimeFormat` uses the default locale and default options.

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// toLocaleString without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(new Intl.DateTimeFormat().format(date));
// "12/19/2012" if run with en-US locale (language) and time zone America/Los_Angeles (UTC-0800)
```

Using timeStyle and dateStyle

`dateStyle` and `timeStyle` provide a shortcut for setting multiple date-time component options at once. For example, for `en-US`, `dateStyle: "short"` is equivalent to setting `year: "2-digit", month: "numeric", day: "numeric"`, and `timeStyle: "short"` is equivalent to setting `hour: "numeric", minute: "numeric"`.

```
const shortTime = new Intl.DateTimeFormat("en-US", {
  timeStyle: "short",
});
console.log(shortTime.format(Date.now())); // "1:31 PM"

const shortDate = new Intl.DateTimeFormat("en-US", {
  dateStyle: "short",
});
console.log(shortDate.format(Date.now())); // "7/7/20"

const mediumTime = new Intl.DateTimeFormat("en-US", {
  timeStyle: "medium",
  dateStyle: "short",
});
console.log(mediumTime.format(Date.now())); // "7/7/20, 1:31:55 PM"
```

However, the exact (locale dependent) component styles they resolve to are not included in the resolved options. This ensures the result of `resolvedOptions()` can be passed directly to the `Intl.DateTimeFormat()` constructor (because an `options` object with both `dateStyle` or `timeStyle` and individual date or time component styles is not valid).

```
console.log(shortDate.resolvedOptions().year); // undefined
```

Using dayPeriod

Use the `dayPeriod` option to output a string for the times of day ("in the morning", "at night", "noon", etc.). Note, that this only works when formatting for a 12 hour clock (`hourCycle: 'h12'` or `hourCycle: 'h11'`) and that for many locales the strings are the same irrespective of the value passed for the `dayPeriod`.

```
const date = Date.UTC(2012, 11, 17, 4, 0, 42);

console.log(
  new Intl.DateTimeFormat("en-GB", {
    hour: "numeric",
    hourCycle: "h12",
    dayPeriod: "short",
    timeZone: "UTC",
  }).format(date),
);
// 4 at night"  (same formatting in en-GB for all dayPeriod values)

console.log(
  new Intl.DateTimeFormat("fr", {
    hour: "numeric",
    hourCycle: "h12",
    dayPeriod: "narrow",
    timeZone: "UTC",
  }).format(date),
);
// "4 mat."  (same output in French for both narrow/short dayPeriod)
```

```
console.log(
  new Intl.DateTimeFormat("fr", {
    hour: "numeric",
    hourCycle: "h12",
    dayPeriod: "long",
    timeZone: "UTC",
  }).format(date),
);
// "4 du matin"
```

Using timeZoneName

Use the `timeZoneName` option to output a string for the timezone ("GMT", "Pacific Time", etc.).

```
const date = Date.UTC(2021, 11, 17, 3, 0, 42);
const timezoneNames = [
  "short",
  "long",
  "shortOffset",
  "longOffset",
  "shortGeneric",
  "longGeneric",
];

for (const zoneName of timezoneNames) {
  // Do something with currentValue
  const formatter = new Intl.DateTimeFormat("en-US", {
    timeZone: "America/Los_Angeles",
    timeZoneName: zoneName,
  });
  console.log(`${zoneName}: ${formatter.format(date)}`);
}

// Logs:
// short: 12/16/2021, PST
// long: 12/16/2021, Pacific Standard Time
// shortOffset: 12/16/2021, GMT-8
// longOffset: 12/16/2021, GMT-08:00
// shortGeneric: 12/16/2021, PT
// longGeneric: 12/16/2021, Pacific Time
```

See also

- `Intl.DateTimeFormat`
- `Intl.supportedValuesOf()`
- `Intl`

Intl.DateTimeFormat.prototype.format()

The **`format()`** method of `Intl.DateTimeFormat` instances formats a date according to the locale and formatting options of this `Intl.DateTimeFormat` object.

Try it

```
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const date = new Date(2012, 5);

const dateTimeFormat1 = new Intl.DateTimeFormat("sr-RS", options);
console.log(dateTimeFormat1.format(date));
// Expected output: "петак, 1. јун 2012."

const dateTimeFormat2 = new Intl.DateTimeFormat("en-GB", options);
console.log(dateTimeFormat2.format(date));
// Expected output: "Friday, 1 June 2012"

const dateTimeFormat3 = new Intl.DateTimeFormat("en-US", options);
console.log(dateTimeFormat3.format(date));
// Expected output: "Friday, June 1, 2012"
```

Syntax

```
format(date)
```

Parameters

`date`

> The date to format. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

Omitting it results in formatting the current date (as returned by `Date.now()`), which could be slightly confusing, so it is advisable to always explicitly pass a date.

Return value

A string representing the given `date` formatted according to the locale and formatting options of this `Intl.DateTimeFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Using format

Use the `format` getter function for formatting a single date, here for Serbia:

```
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const dateTimeFormat = new Intl.DateTimeFormat("sr-RS", options);
console.log(dateTimeFormat.format(new Date()));
// "недеља, 7. април 2013."
```

Using format with map

Use the `format` getter function for formatting all dates in an array. Note that the function is bound to the `Intl.DateTimeFormat` from which it was obtained, so it can be passed directly to `Array.prototype.map()`.

```
const a = [new Date(2012, 8), new Date(2012, 11), new Date(2012, 3)];
const options = { year: "numeric", month: "long" };
const dateTimeFormat = new Intl.DateTimeFormat("pt-BR", options);
const formatted = a.map(dateTimeFormat.format);
console.log(formatted.join("; "));
// "setembro de 2012; dezembro de 2012; abril de 2012"
```

See also

- `Intl.DateTimeFormat`

Intl.DateTimeFormat.prototype.formatRange()

The **`formatRange()`** method of `Intl.DateTimeFormat` instances formats a date range in the most concise way based on the locales and options provided when instantiating this `Intl.DateTimeFormat` object.

Try it

```
const options1 = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const options2 = { year: "2-digit", month: "numeric", day: "numeric" };

const startDate = new Date(Date.UTC(2007, 0, 10, 10, 0, 0));
const endDate = new Date(Date.UTC(2008, 0, 10, 11, 0, 0));

const dateTimeFormat = new Intl.DateTimeFormat("en", options1);
console.log(dateTimeFormat.formatRange(startDate, endDate));
// Expected output: "Wednesday, January 10, 2007 – Thursday, January 10, 2008"

const dateTimeFormat2 = new Intl.DateTimeFormat("en", options2);
console.log(dateTimeFormat2.formatRange(startDate, endDate));
// Expected output: "1/10/07 – 1/10/08"
```

Syntax

```
formatRange(startDate, endDate)
```

Parameters

`startDate`

The start of the date range. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

`endDate`

The end of the date range. Must have the same type as `startDate`.

Return value

A string representing the given date range formatted according to the locale and formatting options of this `Intl.DateTimeFormat` object. If the start and end dates are equivalent at the precision of the output, the output will only contain a single date.

Examples

Basic formatRange usage

This method receives two `Date`s and formats the date range in the most concise way based on the `locale` and `options` provided when instantiating `Intl.DateTimeFormat`.

```
const date1 = new Date(Date.UTC(1906, 0, 10, 10, 0, 0)); // Wed, 10 Jan 1906 10:00:00 GMT
const date2 = new Date(Date.UTC(1906, 0, 10, 11, 0, 0)); // Wed, 10 Jan 1906 11:00:00 GMT
const date3 = new Date(Date.UTC(1906, 0, 20, 10, 0, 0)); // Sat, 20 Jan 1906 10:00:00 GMT

const fmt1 = new Intl.DateTimeFormat("en", {
  year: "2-digit",
  month: "numeric",
  day: "numeric",
  hour: "numeric",
  minute: "numeric",
});
console.log(fmt1.format(date1)); // '1/10/06, 10:00 AM'
console.log(fmt1.formatRange(date1, date2)); // '1/10/06, 10:00 – 11:00 AM'
console.log(fmt1.formatRange(date1, date3)); // '1/10/06, 10:00 AM – 1/20/07, 10:00 AM'

const fmt2 = new Intl.DateTimeFormat("en", {
  year: "numeric",
  month: "short",
  day: "numeric",
});
console.log(fmt2.format(date1)); // 'Jan 10, 1906'
console.log(fmt2.formatRange(date1, date2)); // 'Jan 10, 1906'
console.log(fmt2.formatRange(date1, date3)); // 'Jan 10 – 20, 1906'
```

See also

- `Intl.DateTimeFormat`

Intl.DateTimeFormat.prototype.formatRangeToParts()

The **`formatRangeToParts()`** method of `Intl.DateTimeFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `formatRange()`. It is useful for building custom strings from the locale-specific tokens.

Try it

```
const startDate = new Date(Date.UTC(2007, 0, 10, 10, 0, 0)); // > 'Wed, 10 Jan 2007 10:00:00 GMT'
const endDate = new Date(Date.UTC(2007, 0, 10, 11, 0, 0)); // > 'Wed, 10 Jan 2007 11:00:00 GMT'

const dateTimeFormat = new Intl.DateTimeFormat("en", {
  hour: "numeric",
  minute: "numeric",
});

const parts = dateTimeFormat.formatRangeToParts(startDate, endDate);
for (const part of parts) {
  console.log(part);
}
// Expected output (in GMT timezone):
// Object { type: "hour", value: "2", source: "startRange" }
// Object { type: "literal", value: ":", source: "startRange" }
// Object { type: "minute", value: "00", source: "startRange" }
// Object { type: "literal", value: " – ", source: "shared" }
// Object { type: "hour", value: "3", source: "endRange" }
// Object { type: "literal", value: ":", source: "endRange" }
// Object { type: "minute", value: "00", source: "endRange" }
// Object { type: "literal", value: " ", source: "shared" }
// Object { type: "dayPeriod", value: "AM", source: "shared" }
```

Syntax

```
formatRangeToParts(startDate, endDate)
```

Parameters

`startDate`

> The start of the date range. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.
>
> > **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

`endDate`

> The end of the date range. Must have the same type as `startDate`.

Return value

An `Array` of objects containing the formatted date range in parts. Each object has three properties, `type`, `value`, and `source`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `formatRange()`. The `type` may have the same values as `formatToParts()`. The `source` can be one of the following:

`startRange`

> The token is a part of the start date.

`endRange`

> The token is a part of the end date.

`shared`

> The token is shared between the start and end; for example, if the start and end dates share the same day period, that token may get reused. All literals that are part of the range pattern itself, such as the `" – "` separator, are also marked as `shared`.

If the start and end dates are equivalent at the precision of the output, then the output has the same list of tokens as calling `formatToParts()` on the start date, with all tokens marked as `source: "shared"`.

Examples

Using formatRangeToParts()

The `formatRange()` method outputs localized, opaque strings that cannot be manipulated directly:

```
const date1 = new Date(Date.UTC(1906, 0, 10, 10, 0, 0)); // Wed, 10 Jan 1906 10:00:00 GMT
const date2 = new Date(Date.UTC(1906, 0, 10, 11, 0, 0)); // Wed, 10 Jan 1906 11:00:00 GMT

const fmt = new Intl.DateTimeFormat("en", {
  hour: "numeric",
  minute: "numeric",
});

console.log(fmt.formatRange(date1, date2)); // '10:00 – 11:00 AM'
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatRangeToParts()` method produces the same information in parts:

```
console.log(fmt.formatRangeToParts(date1, date2));

// return value:
[
  { type: "hour", value: "10", source: "startRange" },
  { type: "literal", value: ":", source: "startRange" },
  { type: "minute", value: "00", source: "startRange" },
  { type: "literal", value: " – ", source: "shared" },
  { type: "hour", value: "11", source: "endRange" },
  { type: "literal", value: ":", source: "endRange" },
  { type: "minute", value: "00", source: "endRange" },
  { type: "literal", value: " ", source: "shared" },
  { type: "dayPeriod", value: "AM", source: "shared" },
];
```

See also

- `Intl.DateTimeFormat`
- `Intl.DateTimeFormat.prototype.formatRange()`

Intl.DateTimeFormat.prototype.formatToParts()

The **`formatToParts()`** method of `Intl.DateTimeFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

```
const date = new Date(2012, 5);
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const dateTimeFormat = new Intl.DateTimeFormat("en-US", options);

const parts = dateTimeFormat.formatToParts(date);
const partValues = parts.map((p) => p.value);

console.log(partValues);
// Expected output: "["Friday", ", ", "June", " ", "1", ", ", "2012"]"
```

Syntax

```
formatToParts(date)
```

Parameters

`date` Optional

The date to format. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

Omitting it results in formatting the current date (as returned by `Date.now()`), which could be slightly confusing, so it is advisable to always explicitly pass a date.

Return value

An `Array` of objects containing the formatted date in parts. Each object has two properties, `type` and `value`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The `type` may be one of the date-time components:

`weekday`

For example `"M"`, `"Monday"`, or `"Montag"`.

`era`

For example `"BC"` or `"AD"`.

`year`

For example `"2012"` or `"96"`.

`month`

For example `"12"` or `"January"`.

`day`

For example `"17"`.

`dayPeriod`

For example `"AM"`, `"PM"`, `"in the morning"`, or `"noon"`.

`hour`

For example `"3"` or `"03"`.

`minute`

For example `"00"`.

`second`

For example `"07"` or `"42"`.

`fractionalSecond`

For example `"0"`, `"00"`, or `"000"`.

`timeZoneName`

For example `"UTC"`, `"CET"`, or `"Central European Time"`.

The `type` may also be one of the following:

`literal`

Any string that's a part of the format pattern and not influenced by the `date`; for example `"/"`, `", "`, `"o'clock"`, `"de"`, `" "`, etc.

`relatedYear`

A 4-digit Gregorian year, in the event that the calendar's representation would be a `yearName` instead of a year; for example `"2019"`. See named years for more details.

`yearName`

The name given to the year, usually in calendars without the concept of continuous years; for example `"geng-zi"`.

`unknown`

Reserved for any token that's not recognized as any of the above; should be rarely encountered.

Examples

Using formatToParts()

The `format()` method outputs localized, opaque strings that cannot be manipulated directly:

```
const date = Date.UTC(2012, 11, 17, 3, 0, 42);

const formatter = new Intl.DateTimeFormat("en-us", {
  weekday: "long",
  year: "numeric",
  month: "numeric",
  day: "numeric",
  hour: "numeric",
  minute: "numeric",
  second: "numeric",
  fractionalSecondDigits: 3,
  hour12: true,
  timeZone: "UTC",
});

formatter.format(date);
// "Monday, 12/17/2012, 3:00:42.000 AM"
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatToParts()` method produces the same information in parts:

```
formatter.formatToParts(date);

// return value:
[
  { type: "weekday", value: "Monday" },
  { type: "literal", value: ", " },
  { type: "month", value: "12" },
  { type: "literal", value: "/" },
  { type: "day", value: "17" },
  { type: "literal", value: "/" },
  { type: "year", value: "2012" },
  { type: "literal", value: ", " },
  { type: "hour", value: "3" },
  { type: "literal", value: ":" },
  { type: "minute", value: "00" },
  { type: "literal", value: ":" },
  { type: "second", value: "42" },
  { type: "fractionalSecond", value: "000" },
  { type: "literal", value: " " },
  { type: "dayPeriod", value: "AM" },
];
```

Now the information is available separately and it can be formatted and concatenated again in a customized way. For example by using `Array.prototype.map()`, arrow functions, a switch statement, template literals, and `Array.prototype.join()`, to insert additional markup for certain components.

```
const dateString = formatter
  .formatToParts(date)
  .map(({ type, value }) => {
    switch (type) {
      case "dayPeriod":
        return `<em>${value}</em>`;
      default:
        return value;
    }
  })
  .join("");

console.log(dateString);
// "Monday, 12/17/2012, 3:00:42.000 <em>AM</em>"
```

Named years

Some calendars use named years; for example, the Chinese and Tibetan calendars use a 60-year sexagenary cycle of named years. These calendars do not have a universal way to unambiguously number each year, so years are disambiguated by relationship to corresponding years on the Gregorian calendar. In this case, when the `DateTimeFormat` is configured to output the year component, a token for `relatedYear` is output instead of `year`.

```
const df = new Intl.DateTimeFormat("zh-u-ca-chinese");
df.formatToParts(Date.UTC(2012, 11, 17, 3, 0, 42));

// return value:
[
  { type: "relatedYear", value: "2012" },
  { type: "literal", value: "年" },
  { type: "month", value: "十一月" },
  { type: "day", value: "4" },
];
```

Sometimes, the combination of date-time component options maps to a format that also includes a `yearName`. There isn't a separate option that controls whether `yearName` is displayed or not. For example, the options below sets `month` to `"long"` and results in a `yearName` token, despite `year` still being `"numeric"`:

```
const opts = { year: "numeric", month: "long", day: "numeric" };
const df = new Intl.DateTimeFormat("zh-u-ca-chinese", opts);
df.formatToParts(Date.UTC(2012, 11, 17, 3, 0, 42));

// return value:
[
  { type: "relatedYear", value: "2012" },
  { type: "yearName", value: "壬辰" },
  { type: "literal", value: "年" },
  { type: "month", value: "十一月" },
  { type: "day", value: "4" },
];
```

Because `format()` just concatenates all the `value` strings together, you will see the Gregorian year and the year name together in the output in this case.

See also

- `Intl.DateTimeFormat`
- `Intl.DateTimeFormat.prototype.format()`

Intl.DateTimeFormat.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.DateTimeFormat` instances returns a new object with properties reflecting the options computed during initialization of this `DateTimeFormat` object.

Try it

```
const region = new Intl.DateTimeFormat("zh-CN", { timeZone: "UTC" });
const options = region.resolvedOptions();

console.log(options.locale);
// Expected output: "zh-CN"

console.log(options.calendar);
// Expected output: "gregory"

console.log(options.numberingSystem);
// Expected output: "latn"
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `DateTimeFormat` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `ca`, `hc`, and `nu` Unicode extension keys, if requested, may be included in the output.

`calendar`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"ca"`, with default filled in as needed. It is a supported calendar type for this locale. The default is locale dependent.

`numberingSystem`

The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

`timeZone`

The value provided for this property in the `options` argument, with default filled in as needed. It is an IANA time zone name. The default is the runtime's default time zone.

> **Note:** The standardization of `Temporal` requires browsers to use the same identifier as originally specified, without canonicalization to a different alias. See time zones and offsets for more information.

`hourCycle` Optional

The value provided for this property in the `options` argument, or using the Unicode extension key `"hc"`, with default filled in as needed. If `hour12` was provided in the `options`, then that overrides other `hourCycle` settings. It is only present if the resolved options also include `hour` or `timeStyle`. It is either `"h11"`, `"h12"`, `"h23"`, or `"h24"`. The default is locale dependent, although `"h24"` is never a default.

`hour12` Optional

Calculated from `hourCycle`. It is only present if the resolved options also include `hour` or `timeStyle`. It is `true` if `hourCycle` is `"h11"` or `"h12"`, and `false` if `hourCycle` is `"h23"` or `"h24"`.

`weekday`, `era`, `year`, `month`, `day`, `dayPeriod`, `hour`, `minute`, `second`, `fractionalSecondDigits`, `timeZoneName` Optional

The values resulting from format matching between the corresponding properties in the `options` argument and the available combinations and representations for date-time formatting in the selected locale. Some of these properties may not be present, indicating that the corresponding components will not be represented in formatted output. `weekday`, `era`, and `dayPeriod` are either `"narrow"`, `"short"`, or `"long"`; `year`, `day`, `hour`, `minute`, and `second` are either `"numeric"`, `"2-digit"`, or `"narrow"`; `month` is either `"numeric"`, `"2-digit"`, `"narrow"`, `"short"`, or `"long"`; `fractionalSecondDigits` is either `1`, `2`, or `3`; `timeZoneName` is either `"short"`, `"long"`, `"shortOffset"`, `"longOffset"`, `"shortGeneric"`, or `"longGeneric"`.

If these properties were requested in `options`, the constructor prevents `dateStyle` and `timeStyle` from being specified, so the below group will never be present.

`dateStyle`, `timeStyle` Optional

The values provided for these properties in the `options` argument. They are either `"full"`, `"long"`, `"medium"`, `"short"`, or `"none"`. Some of these properties may not be present, indicating that the corresponding components will not be represented in formatted output.

If these properties were requested in `options`, the constructor prevents individual date time component options from being specified, so the above group will never be present.

> **Note:** Although `dateStyle` and `timeStyle` are shortcuts for individual date and time component styles, the exact (locale dependent) component styles they resolve to are not included in the resolved options. This ensures the result of `resolvedOptions()` can be passed directly to the `Intl.DateTimeFormat()` constructor (because an `options` object with both `dateStyle` or `timeStyle` and individual date or time component styles is not valid).

Examples

Using the resolvedOptions method

```
const germanFakeRegion = new Intl.DateTimeFormat("de-XX", { timeZone: "UTC" });
const usedOptions = germanFakeRegion.resolvedOptions();

usedOptions.locale; // "de" (because "de-XX" does not exist)
usedOptions.calendar; // "gregory"
usedOptions.numberingSystem; // "latn"
usedOptions.timeZone; // "UTC"
usedOptions.month; // "numeric"
```

Getting the user's time zone and locale preferences

The `Intl.DateTimeFormat` constructor without any options uses the current system settings. You can use `resolvedOptions()` to get the user's current time zone and locale's preferred calendar and numbering system:

```
const systemOptions = new Intl.DateTimeFormat().resolvedOptions();
systemOptions.timeZone; // e.g., "Europe/Brussels" or "Asia/Riyadh"
systemOptions.calendar; // e.g., "gregory" or "islamic-umalqura"
systemOptions.numberingSystem; // e.g., "latn" or "arab"
systemOptions.locale; // e.g., "nl-BE" or "ar-SA"
```

See also

- `Intl.DateTimeFormat`

Intl.DateTimeFormat.supportedLocalesOf()

The **`Intl.DateTimeFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in date and time formatting without having to fall back to the runtime's default locale.

Try it

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.DateTimeFormat.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

```
Intl.DateTimeFormat.supportedLocalesOf(locales)
Intl.DateTimeFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in date and time formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in date and time formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to date and time formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.DateTimeFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.DateTimeFormat`

Intl.DisplayNames

The **`Intl.DisplayNames`** object enables the consistent translation of language, region and script display names.

Try it

```
const regionNamesInEnglish = new Intl.DisplayNames(["en"], { type: "region" });
const regionNamesInTraditionalChinese = new Intl.DisplayNames(["zh-Hant"], {
  type: "region",
});

console.log(regionNamesInEnglish.of("US"));
// Expected output: "United States"

console.log(regionNamesInTraditionalChinese.of("US"));
// Expected output: "美國"
```

Constructor

`Intl.DisplayNames()`

Creates a new `Intl.DisplayNames` object.

Static methods

`Intl.DisplayNames.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.DisplayNames.prototype` and shared by all `Intl.DisplayNames` instances.

`Intl.DisplayNames.prototype.constructor`

The constructor function that created the instance object. For `Intl.DisplayNames` instances, the initial value is the `Intl.DisplayNames` constructor.

`Intl.DisplayNames.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.DisplayNames"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.DisplayNames.prototype.of()`

This method receives a `code` and returns a string based on the locale and options provided when instantiating `Intl.DisplayNames`.

`Intl.DisplayNames.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Region Code Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for a region code.

```
// Get display names of region in English
let regionNames = new Intl.DisplayNames(["en"], { type: "region" });
regionNames.of("419"); // "Latin America"
regionNames.of("BZ"); // "Belize"
regionNames.of("US"); // "United States"
regionNames.of("BA"); // "Bosnia & Herzegovina"
regionNames.of("MM"); // "Myanmar (Burma)"

// Get display names of region in Traditional Chinese
regionNames = new Intl.DisplayNames(["zh-Hant"], { type: "region" });
regionNames.of("419"); // "拉丁美洲"
regionNames.of("BZ"); // "貝里斯"
regionNames.of("US"); // "美國"
regionNames.of("BA"); // "波士尼亞與赫塞哥維納"
regionNames.of("MM"); // "緬甸"
```

Language Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for a language-script-region sequence.

```
// Get display names of language in English
let languageNames = new Intl.DisplayNames(["en"], { type: "language" });
languageNames.of("fr"); // "French"
languageNames.of("de"); // "German"
languageNames.of("fr-CA"); // "Canadian French"
languageNames.of("zh-Hant"); // "Traditional Chinese"
languageNames.of("en-US"); // "American English"
languageNames.of("zh-TW"); // "Chinese (Taiwan)"]

// Get display names of language in Traditional Chinese
languageNames = new Intl.DisplayNames(["zh-Hant"], { type: "language" });
languageNames.of("fr"); // "法文"
languageNames.of("zh"); // "中文"
languageNames.of("de"); // "德文"
```

Script Code Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for a script code.

```
// Get display names of script in English
let scriptNames = new Intl.DisplayNames(["en"], { type: "script" });
// Get script names
scriptNames.of("Latn"); // "Latin"
scriptNames.of("Arab"); // "Arabic"
scriptNames.of("Kana"); // "Katakana"

// Get display names of script in Traditional Chinese
scriptNames = new Intl.DisplayNames(["zh-Hant"], { type: "script" });
scriptNames.of("Latn"); // "拉丁文"
scriptNames.of("Arab"); // "阿拉伯文"
scriptNames.of("Kana"); // "片假名"
```

Currency Code Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for currency code.

```
// Get display names of currency code in English
let currencyNames = new Intl.DisplayNames(["en"], { type: "currency" });
// Get currency names
currencyNames.of("USD"); // "US Dollar"
```

```
currencyNames.of("EUR"); // "Euro"
currencyNames.of("TWD"); // "New Taiwan Dollar"
currencyNames.of("CNY"); // "Chinese Yuan"

// Get display names of currency code in Traditional Chinese
currencyNames = new Intl.DisplayNames(["zh-Hant"], { type: "currency" });
currencyNames.of("USD"); // "美元"
currencyNames.of("EUR"); // "歐元"
currencyNames.of("TWD"); // "新台幣"
currencyNames.of("CNY"); // "人民幣"
```

See also

- Polyfill of `Intl.DisplayNames` in FormatJS
- `Intl`

Intl.DisplayNames() constructor

The **`Intl.DisplayNames()`** constructor creates `Intl.DisplayNames` objects.

Try it

```
const regionNamesInEnglish = new Intl.DisplayNames(["en"], { type: "region" });
const regionNamesInTraditionalChinese = new Intl.DisplayNames(["zh-Hant"], {
  type: "region",
});

console.log(regionNamesInEnglish.of("US"));
// Expected output: "United States"

console.log(regionNamesInTraditionalChinese.of("US"));
// Expected output: "美國"
```

Syntax

```
new Intl.DisplayNames(locales, options)
```

> **Note:** `Intl.DisplayNames()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales`

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options`

An object containing the following properties, in the order they are retrieved:

`localeMatcher` Optional

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`style` Optional

The formatting style to use. Possible values are `"narrow"`, `"short"`, and `"long"`; the default is `"long"`.

`type`

The type of display names to return from `of()`. Possible values are `"language"`, `"region"`, `"script"`, `"currency"`, `"calendar"`, and `"dateTimeField"`.

`fallback` Optional

What to return from `of()` if the input is structurally valid but there's no matching display name. Possible values are:

`"code"` (default)

Return the input code itself.

`"none"`

Return `undefined`.

`languageDisplay` Optional

How language names should be displayed. Only usable along with `type: "language"`. Possible values are:

`"dialect"` (default)

Display special regional dialects using their own name. E.g. `"nl-BE"` will be displayed as `"Flemish"`.

`"standard"`

Display all languages using standard format. E.g. `"nl-BE"` will be displayed as `"Dutch (Belgium)"`.

Exceptions

`TypeError`

Thrown if `options.type` is not provided.

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

```
console.log(new Intl.DisplayNames([], { type: "language" }).of("US"));
// 'us'
```

Using type <code>dateTimeField</code>

Example using `dateTimeField` as a type option, will return the localized date time names strings.

```
const dn = new Intl.DisplayNames("pt", { type: "dateTimeField" });
console.log(dn.of("era")); // 'era'
console.log(dn.of("year")); // 'ano'
console.log(dn.of("month")); // 'mês'
console.log(dn.of("quarter")); // 'trimestre'
console.log(dn.of("weekOfYear")); // 'semana'
console.log(dn.of("weekday")); // 'dia da semana'
console.log(dn.of("dayPeriod")); // 'AM/PM'
console.log(dn.of("day")); // 'dia'
console.log(dn.of("hour")); // 'hora'
console.log(dn.of("minute")); // 'minuto'
console.log(dn.of("second")); // 'segundo'
```

Using type <code>calendar</code>

Example using `calendar` as a type option, will return the localized calendar names strings.

```
const dn = new Intl.DisplayNames("en", { type: "calendar" });
console.log(dn.of("roc")); // 'Minguo Calendar'
console.log(dn.of("gregory")); // 'Gregorian Calendar'
console.log(dn.of("chinese")); // 'Chinese Calendar'
```

Using type <code>language</code> with <code>languageDisplay</code>

Example using `language` as a type with `languageDisplay` options.

```
// Using `dialect` option
const dnDialect = new Intl.DisplayNames("en", {
  type: "language",
  languageDisplay: "dialect",
});
console.log(dnDialect.of("en-GB")); // 'British English'

// Using `standard` option
const dnStd = new Intl.DisplayNames("en", {
  type: "language",
  languageDisplay: "standard",
});
console.log(dnStd.of("en-GB")); // 'English (United Kingdom)'
```

See also

- `Intl.DisplayNames`
- `Intl.supportedValuesOf()`
- `Intl`

Intl.DisplayNames.prototype.of()

The **`of()`** method of `Intl.DisplayNames` instances receives a code and returns a string based on the locale and options provided when instantiating this `Intl.DisplayNames` object.

Try it

```
const regionNamesInEnglish = new Intl.DisplayNames(["en"], { type: "region" });
const regionNamesInTraditionalChinese = new Intl.DisplayNames(["zh-Hant"], {
  type: "region",
```

```
});

console.log(regionNamesInEnglish.of("US"));
// Expected output: "United States"

console.log(regionNamesInTraditionalChinese.of("US"));
// Expected output: "美國"
```

Syntax

```
of(code)
```

Parameters

`code`

The `code` to provide depends on the `type`:

- If the type is "region", `code` should be either a two-letter ISO 3166 region code, or a three-digit UN M49 geographic region. It is required to follow the `unicode_region_subtag` grammar. Use uppercase codes (e.g., `"US"`), because lowercase ones (e.g., `"us"`) may not work reliably everywhere.
- If the type is "script", `code` should be a four-letter ISO 15924 script code. It is required to follow the `unicode_script_subtag` grammar.
- If the type is "language", `code` should be matched by the `unicode_language_id` nonterminal.
- If the type is "currency", `code` should be a three-letter ISO 4217 currency code. It is required to have exactly three alphabetic characters.
- If the type is "dateTimeField", `code` should be one of: `"era"`, `"year"`, `"quarter"`, `"month"`, `"weekOfYear"`, `"weekday"`, `"day"`, `"dayPeriod"`, `"hour"`, `"minute"`, `"second"`, `"timeZoneName"`.
- If the type is "calendar", `code` should be a calendar key. It is required to follow the `type` grammar of a Unicode locale identifier.

Return value

A language-specific formatted string, or `undefined` if there's no data for the input and `fallback` is `"none"`.

> **Note:** `fallback` is only used if `code` is structurally valid. See using fallback.

Exceptions

`RangeError`

Thrown if `code` is not structurally valid for the given `type`.

Examples

Using the of method

```
const regionNames = new Intl.DisplayNames("en", { type: "region" });
regionNames.of("419"); // "Latin America"

const languageNames = new Intl.DisplayNames("en", { type: "language" });
languageNames.of("fr"); // "French"

const currencyNames = new Intl.DisplayNames("en", { type: "currency" });
currencyNames.of("EUR"); // "Euro"

const languageNamesStandard = new Intl.DisplayNames("fr", {
  type: "language",
  languageDisplay: "standard",
});
languageNamesStandard.of("fr-CA"); // "français (Canada)"

const languageNamesDialect = new Intl.DisplayNames("fr", {
  type: "language",
  languageDisplay: "dialect",
});
languageNamesDialect.of("fr-CA"); // "français canadien"
```

Using fallback

When the `Intl.DisplayNames` is constructed with `fallback: "code"`, the `of()` method will return the `code` if the input looks structurally valid but there's no data for the input. If `fallback` is `"none"`, `undefined` is returned.

```
console.log(
  new Intl.DisplayNames("en", { type: "region", fallback: "code" }).of("ZL"),
); // "ZL"

console.log(
  new Intl.DisplayNames("en", { type: "region", fallback: "none" }).of("ZL"),
); // undefined
```

However, this only applies if the `code` is structurally valid. For example, if `type` is `"region"` but `code` does not follow the `unicode_region_subtag` grammar (2 alphabetic characters or 3 numeric characters), a `RangeError` is directly thrown instead of using the fallback.

```
console.log(
  new Intl.DisplayNames("en", { type: "region", fallback: "code" }).of("ZLC"),
); // throws RangeError: invalid value "ZLC" for option region
```

See also

- `Intl.DisplayNames`

Intl.DisplayNames.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.DisplayNames` instances returns a new object with properties reflecting the options computed during initialization of this `DisplayNames` object.

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `DisplayNames` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`style`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"narrow"`, `"short"`, or `"long"`. The default is `"long"`.

`type`

The value provided for this property in the `options` argument. It is either `"language"`, `"region"`, `"script"`, `"currency"`, `"calendar"`, or `"dateTimeField"`. It is required so there is no default.

`fallback`

The value provided for this property in the `options` argument. It is either `"code"` or `"none"`. The default is `"code"`.

`languageDisplay`

The value provided for this property in the `options` argument. It is either `"dialect"` or `"standard"`. The default is `"dialect"`.

Examples

Using resolvedOptions

```
const displayNames = new Intl.DisplayNames(["de-DE"], { type: "region" });

const usedOptions = displayNames.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.style); // "long"
console.log(usedOptions.type); // "region"
console.log(usedOptions.fallback); // "code"
```

```
const displayNames = new Intl.DisplayNames("en", {
  type: "language",
  languageDisplay: "standard",
});

const usedOptions = displayNames.resolvedOptions();
console.log(usedOptions.type); // "language"
console.log(usedOptions.languageDisplay); // "standard"
```

See also

- `Intl.DisplayNames`

Intl.DisplayNames.supportedLocalesOf()

The **`Intl.DisplayNames.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in display names without having to fall back to the runtime's default locale.

Syntax

```
Intl.DisplayNames.supportedLocalesOf(locales)
Intl.DisplayNames.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in display names without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in display names, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to display names nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.DisplayNames.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.DisplayNames`

Intl.DurationFormat

The **`Intl.DurationFormat`** object enables language-sensitive duration formatting.

Constructor

`Intl.DurationFormat()`

Creates a new `Intl.DurationFormat` object.

Static methods

`Intl.DurationFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.DurationFormat.prototype` and shared by all `Intl.DurationFormat` instances.

`Intl.DurationFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.DurationFormat` instances, the initial value is the `Intl.DurationFormat` constructor.

`Intl.DurationFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.DurationFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.DurationFormat.prototype.format()`

Getter function that formats a duration according to the locale and formatting options of this `DurationFormat` object.

`Intl.DurationFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the formatted duration in parts.

`Intl.DurationFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Using Intl.DurationFormat

The examples below show how to use the `Intl.DurationFormat` object to format a duration object with various locales and styles.

```
const duration = {
  hours: 1,
  minutes: 46,
  seconds: 40,
};

// With style set to "long" and locale "fr-FR"
new Intl.DurationFormat("fr-FR", { style: "long" }).format(duration);
// "1 heure, 46 minutes et 40 secondes"

// With style set to "short" and locale "en"
new Intl.DurationFormat("en", { style: "short" }).format(duration);
// "1 hr, 46 min and 40 sec"

// With style set to "narrow" and locale "pt"
new Intl.DurationFormat("pt", { style: "narrow" }).format(duration);
// "1 h 46 min 40 s"
```

See also

- Polyfill of `Intl.DurationFormat` in FormatJS
- `Intl`
- `Temporal.Duration.prototype.toLocaleString()`

Intl.DurationFormat() constructor

The **`Intl.DurationFormat()`** constructor creates `Intl.DurationFormat` objects.

Syntax

```
new Intl.DurationFormat()
new Intl.DurationFormat(locales)
new Intl.DurationFormat(locales, options)
```

> **Note:** `Intl.DurationFormat()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension key is allowed:

`nu`

See `numberingSystem`.

This key can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

`style`

The style of the formatted duration. This value is used as the default for all other unit options, and also corresponds to the `style` option of `Intl.ListFormat()` when concatenating the list of duration units. Possible values are:

`"long"`

E.g., 1 hour and 50 minutes

`"short"` (default)

E.g., 1 hr, 50 min

`"narrow"`

E.g., 1h 50m

`"digital"`

E.g., 1:50:00

`years`

The style of the formatted years. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`yearsDisplay`

Whether to always display years, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `years` is unspecified, and `"always"` otherwise.

`months`

The style of the formatted months. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`monthsDisplay`

Whether to always display months, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `months` is unspecified, and `"always"` otherwise.

`weeks`

The style of the formatted weeks. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`weeksDisplay`

Whether to always display weeks, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `weeks` is unspecified, and `"always"` otherwise.

`days`

The style of the formatted days. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`daysDisplay`

Whether to always display days, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `days` is unspecified, and `"always"` otherwise.

`hours`

The style of the formatted hours. Possible values are `"long"`, `"short"`, `"narrow"`, `"numeric"`, and `"2-digit"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`hoursDisplay`

Whether to always display hours, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `hours` is unspecified and `options.style` is not `"digital"`, and `"always"` otherwise.

`minutes`

The style of the formatted minutes.

- If `hours` is `"numeric"` or `"2-digit"`, possible values are `"numeric"` and `"2-digit"`, and `"numeric"` is normalized to `"2-digit"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, `"numeric"`, and `"2-digit"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`minutesDisplay`

Whether to always display minutes, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `minutes` is unspecified and `options.style` is not `"digital"`, and `"always"` otherwise.

`seconds`

The style of the formatted seconds.

- If `minutes` is `"numeric"` or `"2-digit"`, possible values are `"numeric"` and `"2-digit"`, and `"numeric"` is normalized to `"2-digit"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, `"numeric"`, and `"2-digit"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`secondsDisplay`

Whether to always display seconds, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `seconds` is unspecified and `options.style` is not `"digital"`, and `"always"` otherwise.

`milliseconds`

The style of the formatted milliseconds.

- If `seconds` is `"numeric"` or `"2-digit"`, the only possible value is `"numeric"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, and `"numeric"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`millisecondsDisplay`

Whether to always display milliseconds, or only if nonzero.

- If `seconds` is `"numeric"` or `"2-digit"`, the only possible value is `"auto"`; the default is only `"auto"` when `milliseconds` is unspecified.
- Otherwise, possible values are `"always"` and `"auto"`; the default is `"auto"` if `milliseconds` is unspecified, and `"always"` otherwise.

`microseconds`

The style of the formatted microseconds.

- If `milliseconds` is `"numeric"`, the only possible value is `"numeric"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, and `"numeric"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`microsecondsDisplay`

Whether to always display microseconds, or only if nonzero.

- If `milliseconds` is `"numeric"`, the only possible value is `"auto"`; the default is only `"auto"` when `microseconds` is unspecified.
- Otherwise, possible values are `"always"` and `"auto"`; the default is `"auto"` if `microseconds` is unspecified, and `"always"` otherwise.

`nanoseconds`

The style of the formatted nanoseconds.

- If `microseconds` is `"numeric"`, the only possible value is `"numeric"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, and `"numeric"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`nanosecondsDisplay`

Whether to always display nanoseconds, or only if nonzero.

- If `microseconds` is `"numeric"`, the only possible value is `"auto"`; the default is only `"auto"` when `nanoseconds` is unspecified.
- Otherwise, possible values are `"always"` and `"auto"`; the default is `"auto"` if `nanoseconds` is unspecified, and `"always"` otherwise.

`fractionalDigits`

Number of how many fractional second digits to display in the output. Possible values are from `0` to `9`; the default is `undefined` (include as many fractional digits as necessary).

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Description

For each time segment, an `Intl.NumberFormat` object is constructed under the hood. It uses the following options (see `Intl.NumberFormat()` for details):

- `numberingSystem`: the value of `options.numberingSystem`

When `milliseconds`, `microseconds`, or `nanoseconds` uses the `"numeric"` style, the following options are also used:

- `minimumFractionDigits`: `0` when `options.fractionalDigits` is `undefined`, `options.fractionalDigits` otherwise
- `maximumFractionDigits`: `9` when `options.fractionalDigits` is `undefined`, `options.fractionalDigits` otherwise
- `roundingMode`: `"trunc"`

When the time segment uses the `"2-digit"` style, the following options are also used:

- `minimumIntegerDigits`: `2`

When the time segment uses the `"long"`, `"short"`, or `"narrow"` style, the following options are also used:

- `style`: `"unit"` when `"long"`, `"short"`, or `"narrow"` is specified, `undefined` otherwise
- `unit`: the currently formatted unit (`"years"`, `"days"`, `"nanoseconds"`, etc.)
- `unitDisplay`: the value of the time segment style (`"long"`, `"short"`, or `"narrow"`)

Examples

Using the Intl.DurationFormat() constructor

```
const duration = {
  hours: 2,
  minutes: 20,
  seconds: 35,
};

console.log(new Intl.DurationFormat("pt", { style: "long" }).format(duration));
// "2 horas, 20 minutos e 35 segundos"
```

See also

- `Intl.DurationFormat`
- `Intl.supportedValuesOf()`
- `Intl`

Intl.DurationFormat.prototype.format()

The **`format()`** method of `Intl.DurationFormat` instances formats a duration according to the locale and formatting options of this `Intl.DurationFormat` object.

Syntax

```
format(duration)
```

Parameters

`duration`

> The duration object to be formatted. It should include some or all of the following properties: `years`, `months`, `weeks`, `days`, `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, `nanoseconds`. Each property's value should be an integer, and their signs should be consistent. This can be a `Temporal.Duration` object; see the `Temporal.Duration` documentation for more information about these properties.

Return value

A string representing the given `duration` formatted according to the locale and formatting options of this `Intl.DurationFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Using format()

The following example shows how to create a Duration formatter using the English language.

```
const duration = {
  years: 1,
  months: 2,
  weeks: 3,
  days: 3,
  hours: 4,
  minutes: 5,
  seconds: 6,
  milliseconds: 7,
  microseconds: 8,
  nanoseconds: 9,
};

// Without options, style defaults to "short"
new Intl.DurationFormat("en").format(duration);
// "1 yr, 2 mths, 3 wks, 3 days, 4 hr, 5 min, 6 sec, 7 ms, 8 µs, 9 ns"

// With style set to "long"
new Intl.DurationFormat("en", { style: "long" }).format(duration);
// "1 year, 2 months, 3 weeks, 3 days, 4 hours, 5 minutes, 6 seconds, 7 milliseconds, 8 microseconds, 9 nanoseconds"

// With style set to "narrow"
new Intl.DurationFormat("en", { style: "narrow" }).format(duration);
// "1y 2mo 3w 3d 4h 5m 6s 7ms 8µs 9ns"
```

Using format() with different locales and styles

```
const duration = {
  hours: 1,
  minutes: 46,
  seconds: 40,
};

// With style set to "long" and locale "fr-FR"
new Intl.DurationFormat("fr-FR", { style: "long" }).format(duration);
// "1 heure, 46 minutes et 40 secondes"

// With style set to "short" and locale set to "en"
new Intl.DurationFormat("en", { style: "short" }).format(duration);
// "1 hr, 46 min and 40 sec"

// With style set to "short" and locale set to "pt"
new Intl.DurationFormat("pt", { style: "narrow" }).format(duration);
// "1 h 46 min 40 s"

// With style set to "digital" and locale set to "en"
new Intl.DurationFormat("en", { style: "digital" }).format(duration);
// "1:46:40"

// With style set to "digital", locale set to "en", and hours set to "long"
new Intl.DurationFormat("en", { style: "digital", hours: "long" }).format(
  duration,
);
// "1 hour, 46:40"
```

Using format() with the fractionalDigits option

```
const duration = {
  hours: 11,
  minutes: 30,
  seconds: 12,
  milliseconds: 345,
  microseconds: 600,
};

new Intl.DurationFormat("en", { style: "digital" }).format(duration);
// "11:30:12.3456"

new Intl.DurationFormat("en", { style: "digital", fractionalDigits: 5 }).format(
  duration,
);
// "11:30:12.34560"

new Intl.DurationFormat("en", { style: "digital", fractionalDigits: 3 }).format(
  duration,
);
// "11:30:12.346"
```

See also

- `Intl.DurationFormat`
- `Temporal.Duration`

Intl.DurationFormat.prototype.formatToParts()

The **`formatToParts()`** method of `Intl.DurationFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Syntax

```
formatToParts(duration)
```

Parameters

`duration` Optional

> The duration object to be formatted. It should include some or all of the following properties: `years`, `months`, `weeks`, `days`, `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, `nanoseconds`. Each property's value should be an integer, and their signs should be consistent. This can be a `Temporal.Duration` object; see the `Temporal.Duration` documentation for more information about these properties.

Return value

An `Array` of objects containing the formatted duration in parts. Each object has two or three properties, `type`, `value`, and optionally `unit`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The parts can be thought of as directly obtained from calling `Intl.NumberFormat.prototype.formatToParts()` with the numeric value and their respective units. All tokens that are produced by the `NumberFormat` have an additional `unit` property, which is the singular form of the input `unit`; this is for programmatic usage and is not localized. The localized unit is output as a separate `unit` token as part of the `NumberFormat` result. The parts from each duration unit are concatenated together in the same fashion as calling `Intl.ListFormat.prototype.formatToParts()` with `{ type: "unit" }`, so additional literal tokens are inserted.

Examples

The `formatToParts` method enables locale-aware formatting of strings produced by `DurationFormat` formatters by providing you the string in parts:

```
const duration = {
  hours: 7,
  minutes: 8,
  seconds: 9,
  milliseconds: 123,
  microseconds: 456,
  nanoseconds: 789,
};

new Intl.DurationFormat("en", { style: "long" }).formatToParts(duration);

// Returned value:
[
  { type: "integer", value: "7", unit: "hour" },
  { type: "literal", value: " ", unit: "hour" },
  { type: "unit", value: "hours", unit: "hour" },
  { type: "literal", value: ", " },
  { type: "integer", value: "8", unit: "minute" },
  { type: "literal", value: " ", unit: "minute" },
  { type: "unit", value: "minutes", unit: "minute" },
  { type: "literal", value: ", " },
  { type: "integer", value: "9", unit: "second" },
  { type: "literal", value: " ", unit: "second" },
  { type: "unit", value: "seconds", unit: "second" },
  { type: "literal", value: ", " },
```

```
  { type: "integer", value: "123", unit: "millisecond" },
  { type: "literal", value: " ", unit: "millisecond" },
  { type: "unit", value: "milliseconds", unit: "millisecond" },
  { type: "literal", value: ", " },
  { type: "integer", value: "456", unit: "microsecond" },
  { type: "literal", value: " ", unit: "microsecond" },
  { type: "unit", value: "microseconds", unit: "microsecond" },
  { type: "literal", value: ", " },
  { type: "integer", value: "789", unit: "nanosecond" },
  { type: "literal", value: " ", unit: "nanosecond" },
  { type: "unit", value: "nanoseconds", unit: "nanosecond" },
];
```

See also

- `Intl.DurationFormat`
- `Intl.DurationFormat.prototype.format()`
- `Temporal.Duration`

Intl.DurationFormat.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.DurationFormat` instances returns a new object with properties reflecting the options computed during initialization of this `DurationFormat` object.

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `DurationFormat` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `nu` Unicode extension key, if requested, may be included in the output.

`numberingSystem`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

`style`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"long"`, `"short"`, `"narrow"`, or `"digital"`. The default is `"short"`.

`years`, `yearsDisplay`, `months`, `monthsDisplay`, `weeks`, `weeksDisplay`, `days`, `daysDisplay`, `hours`, `hoursDisplay`, `minutes`, `minutesDisplay`, `seconds`, `secondsDisplay`, `milliseconds`, `millisecondsDisplay`, `nanoseconds`, `nanosecondsDisplay`

> The values provided for these properties in the `options` argument, with defaults filled in as needed. For the valid values and defaults for each, see the `options` argument of the constructor.

`fractionalDigits` Optional

> The value provided for this property in the `options` argument. It is only present if specified in `options`. It is an integer from 0 to 9, inclusive.

Examples

Using the resolvedOptions method

```
const duration = new Intl.DurationFormat("en");
const usedOptions = duration.resolvedOptions();

usedOptions.locale; // "en"
usedOptions.numberingSystem; // "latn"
usedOptions.years; // "long"
usedOptions.yearsDisplay; // "auto"
usedOptions.style; // "long"
```

See also

- `Intl.DurationFormat`
- `Intl.supportedValuesOf()`
- `Intl`

Intl.DurationFormat.supportedLocalesOf()

The **`Intl.DurationFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in duration formatting without having to fall back to the runtime's default locale.

Syntax

```
Intl.DurationFormat.supportedLocalesOf(locales)
Intl.DurationFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

> A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object that may have the following property:
>
> `localeMatcher`
>
> > The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in duration formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in duration formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to duration formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.DurationFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.DurationFormat`

Intl.getCanonicalLocales()

The **`Intl.getCanonicalLocales()`** static method returns an array containing the canonical locale names. Duplicates will be omitted and elements will be validated as structurally valid language tags.

Try it

```
console.log(Intl.getCanonicalLocales("EN-US"));
// Expected output: Array ["en-US"]

console.log(Intl.getCanonicalLocales(["EN-US", "Fr"]));
// Expected output: Array ["en-US", "fr"]

try {
  Intl.getCanonicalLocales("EN_US");
} catch (err) {
  console.log(err.toString());
  // Expected output: RangeError: invalid language tag: "EN_US"
}
```

Syntax

```
Intl.getCanonicalLocales(locales)
```

Parameters

`locales`

> A list of `String` values for which to get the canonical locale names.

Return value

An array containing the canonical locale names.

Examples

Using getCanonicalLocales

```
Intl.getCanonicalLocales("EN-US"); // ["en-US"]
Intl.getCanonicalLocales(["EN-US", "Fr"]); // ["en-US", "fr"]

Intl.getCanonicalLocales("EN_US");
// RangeError: invalid language tag: "EN_US"
```

See also

- Polyfill of `Intl.getCanonicalLocales` in FormatJS
- `Intl.NumberFormat.supportedLocalesOf()`
- `Intl.DateTimeFormat.supportedLocalesOf()`
- `Intl.Collator.supportedLocalesOf()`

Intl.ListFormat

The **`Intl.ListFormat`** object enables language-sensitive list formatting.

Try it

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Constructor

`Intl.ListFormat()`

Creates a new `Intl.ListFormat` object.

Static methods

`Intl.ListFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.ListFormat.prototype` and shared by all `Intl.ListFormat` instances.

`Intl.ListFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.ListFormat` instances, the initial value is the `Intl.ListFormat` constructor.

`Intl.ListFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.ListFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.ListFormat.prototype.format()`

Returns a language-specific formatted string representing the elements of the list.

`Intl.ListFormat.prototype.formatToParts()`

Returns an array of objects representing the different components that can be used to format a list of values in a locale-aware fashion.

`Intl.ListFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and style formatting options computed during the construction of the current `Intl.ListFormat` object.

Examples

Using format

The following example shows how to create a List formatter using the English language.

```
const list = ["Motorcycle", "Bus", "Car"];

console.log(
  new Intl.ListFormat("en-GB", { style: "long", type: "conjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus and Car

console.log(
  new Intl.ListFormat("en-GB", { style: "short", type: "disjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus or Car

console.log(
  new Intl.ListFormat("en-GB", { style: "narrow", type: "unit" }).format(list),
);
// Motorcycle Bus Car
```

Using formatToParts

The following example shows how to create a List formatter returning formatted parts

```
const list = ["Motorcycle", "Bus", "Car"];
console.log(
  new Intl.ListFormat("en-GB", {
    style: "long",
    type: "conjunction",
  }).formatToParts(list),
);

// [ { "type": "element", "value": "Motorcycle" },
//   { "type": "literal", "value": ", " },
//   { "type": "element", "value": "Bus" },
//   { "type": "literal", "value": ", and " },
//   { "type": "element", "value": "Car" } ];
```

See also

- Polyfill of `Intl.ListFormat` in FormatJS
- `Intl`

Intl.ListFormat.prototype.format()

The **`format()`** method of `Intl.ListFormat` instances returns a string with a language-specific representation of the list.

Try it

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Syntax

```
format(list)
```

Parameters

`list`

An iterable object, such as an Array, containing strings. Omitting it results in formatting the empty array, which could be slightly confusing, so it is advisable to always explicitly pass a list.

Return value

A language-specific formatted string representing the elements of the list.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Using format

The following example shows how to create a List formatter using the English language.

```
const list = ["Motorcycle", "Bus", "Car"];

console.log(
  new Intl.ListFormat("en-GB", { style: "long", type: "conjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus and Car

console.log(
  new Intl.ListFormat("en-GB", { style: "short", type: "disjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus or Car

console.log(
  new Intl.ListFormat("en-GB", { style: "narrow", type: "unit" }).format(list),
);
// Motorcycle Bus Car
```

See also

- `Intl.ListFormat`

Intl.ListFormat.prototype.formatToParts()

The **`formatToParts()`** method of `Intl.ListFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatterEn = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});

const formatterFr = new Intl.ListFormat("fr", {
  style: "long",
  type: "conjunction",
});

const partValuesEn = formatterEn.formatToParts(vehicles).map((p) => p.value);
const partValuesFr = formatterFr.formatToParts(vehicles).map((p) => p.value);

console.log(partValuesEn);
// Expected output: "["Motorcycle", ", ", "Bus", ", and ", "Car"]"
console.log(partValuesFr);
// Expected output: "["Motorcycle", ", ", "Bus", " et ", "Car"]"
```

Syntax

```
formatToParts(list)
```

Parameters

`list`

> An iterable object, such as an Array, containing strings. Omitting it results in formatting the empty array, which could be slightly confusing, so it is advisable to always explicitly pass a list.

Return value

An `Array` of objects containing the formatted list in parts. Each object has two properties, `type` and `value`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The `type` may be one of the following:

`literal`

> Any string that's a part of the format pattern; for example `", "`, `", and"`, etc.

`element`

An element of the list, exactly as provided.

Examples

Using formatToParts()

```
const fruits = ["Apple", "Orange", "Pineapple"];
const myListFormat = new Intl.ListFormat("en-GB", {
  style: "long",
  type: "conjunction",
});

console.table(myListFormat.formatToParts(fruits));
// [
//  { "type": "element", "value": "Apple" },
//  { "type": "literal", "value": ", " },
//  { "type": "element", "value": "Orange" },
//  { "type": "literal", "value": " and " },
//  { "type": "element", "value": "Pineapple" }
// ]
```

See also

- `Intl.ListFormat`
- `Intl.ListFormat.prototype.format()`

Intl.ListFormat() constructor

The **`Intl.ListFormat()`** constructor creates `Intl.ListFormat` objects.

Try it

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Syntax

```
new Intl.ListFormat()
new Intl.ListFormat(locales)
new Intl.ListFormat(locales, options)
```

> **Note:** `Intl.ListFormat()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`type`

Indicates the type of grouping. Possible values are:

`"conjunction"` (default)

For "and"-based grouping of the list items: "A, B, and C"

`"disjunction"`

For "or"-based grouping of the list items: "A, B, or C"

`"unit"`

For grouping the list items as a compound unit (neither "and"-based nor "or"-based): "A, B, C"

`style`

The grouping style (for example, whether list separators and conjunctions are included). Possible values are:

`"long"` (default)

The typical list format. For example, "A, B, and C"

`"short"`

The spacing, the length or presence of a conjunction, and the separators may change. Usually, you would want the input elements to be abbreviated too. For example, "A, B, & C"

`"narrow"`

Where possible, the list format is further abbreviated, so that the output is as short as possible. For example, "A, B, C"

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Using format

The following example shows how to create a List formatter using the English language.

```
const list = ["Motorcycle", "Bus", "Car"];

console.log(new Intl.ListFormat("en-GB", { type: "conjunction" }).format(list));
// Motorcycle, Bus and Car

console.log(new Intl.ListFormat("en-GB", { type: "disjunction" }).format(list));
// Motorcycle, Bus or Car
```

Oxford comma

Oxford comma is a comma placed immediately before the coordinating conjunction (usually "and" or "or") in a list of three or more terms. Somewhat controversially, the `en-US` locale uses the Oxford comma, while the `en-GB` locale does not.

```
const list = ["Motorcycle", "Bus", "Car"];

console.log(new Intl.ListFormat("en-GB", { type: "conjunction" }).format(list));
// Motorcycle, Bus and Car

console.log(new Intl.ListFormat("en-US", { type: "conjunction" }).format(list));
// Motorcycle, Bus, and Car
```

Unit formatting

Use `style: "unit"` to format the list items as a compound unit. In fact, `Intl.DurationFormat` uses unit-style list formatting under the hood to format durations.

```
const marathon = [
  [42, "kilometer"],
  [195, "meter"],
];

console.log(
  new Intl.ListFormat("en-US", { type: "unit" }).format(
    marathon.map((component) =>
      component[0].toLocaleString("en-US", {
        style: "unit",
        unit: component[1],
        unitDisplay: "long",
      }),
    ),
  ),
);
// 42 kilometers, 195 meters
```

Short and narrow style

The `"short"` and `"narrow"` styles are useful for compact representations of lists.

```
const list = ["Motorcycle", "Bus", "Car"];
console.log(new Intl.ListFormat("en-US", { style: "short" }).format(list));
// Motorcycle, Bus, & Car

console.log(new Intl.ListFormat("en-US", { style: "narrow" }).format(list));
// Motorcycle, Bus, Car

console.log(new Intl.ListFormat("en-GB", { style: "short" }).format(list));
// Motorcycle, Bus and Car

console.log(new Intl.ListFormat("en-GB", { style: "narrow" }).format(list));
// Motorcycle, Bus, Car
```

The input elements are not transformed, but you will often want to abbreviate them too.

```
const marathon = [
  [42, "kilometer"],
  [195, "meter"],
];

function formatDistance(locale, distance, style) {
  return new Intl.ListFormat(locale, { type: "unit", style }).format(
    marathon.map((component) =>
      component[0].toLocaleString(locale, {
        style: "unit",
        unit: component[1],
        unitDisplay: style,
      }),
    ),
  );
}

console.log(formatDistance("en-US", marathon, "long"));
// 42 kilometers, 195 meters
console.log(formatDistance("en-US", marathon, "short"));
// 42 km, 195 m
console.log(formatDistance("en-US", marathon, "narrow"));
// 42km 195m
```

Selection of conjunction

The conjunction word used may depend on the list items' string values. For example, in Spanish, the conjunction is `"y"` for most words, but `"e"` for words starting with the vowel `"i"`.

```
const words = ["fuerte", "indomable"];
const formatter = new Intl.ListFormat("es-ES", { type: "conjunction" });

console.log(formatter.format(words));
// fuerte e indomable
console.log(formatter.format(words.toReversed()));
// indomable y fuerte
```

The algorithm used to determine the conjunction is not perfect (for example, it can't always tell a word's pronunciation from its spelling), but it should work in the general case.

See also

- `Intl.ListFormat`
- `Intl`

Intl.ListFormat.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.ListFormat` instances returns a new object with properties reflecting the options computed during initialization of this `ListFormat` object.

Try it

```
const deListFormatter = new Intl.ListFormat("de-DE", { type: "disjunction" });
const options = deListFormatter.resolvedOptions();

console.log(options.locale);
// Expected output: "de-DE"

console.log(options.style);
// Expected output: "long"

console.log(options.type);
// Expected output: "disjunction"
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `ListFormat` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`type`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"conjunction"`, `"disjunction"`, or `"unit"`. The default is `"conjunction"`.

`style`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"long"`, `"short"`, or `"narrow"`. The default is `"long"`.

Examples

Using resolvedOptions

```
const deListFormatter = new Intl.ListFormat("de-DE", { style: "short" });

const usedOptions = de.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.style); // "short"
console.log(usedOptions.type); // "conjunction" (the default value)
```

See also

- `Intl.ListFormat`
- `Intl.NumberFormat.prototype.resolvedOptions()`
- `Intl.Collator.prototype.resolvedOptions()`
- `Intl.DateTimeFormat.prototype.resolvedOptions()`
- `Intl.PluralRules.prototype.resolvedOptions()`

Intl.ListFormat.supportedLocalesOf()

The **`Intl.ListFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in list formatting without having to fall back to the runtime's default locale.

Syntax

```
Intl.ListFormat.supportedLocalesOf(locales)
Intl.ListFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

> A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object that may have the following property:
>
> `localeMatcher`
>
> > The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in list formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in list formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to list formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be

supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.ListFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.ListFormat`

Intl.Locale

The **`Intl.Locale`** object is a standard built-in property of the Intl object that represents a Unicode locale identifier.

Try it

```
const korean = new Intl.Locale("ko", {
  script: "Kore",
  region: "KR",
  hourCycle: "h23",
  calendar: "gregory",
});

const japanese = new Intl.Locale("ja-Jpan-JP-u-ca-japanese-hc-h12");

console.log(korean.baseName, japanese.baseName);
// Expected output: "ko-Kore-KR" "ja-Jpan-JP"

console.log(korean.hourCycle, japanese.hourCycle);
// Expected output: "h23" "h12"
```

Description

The **`Intl.Locale`** object was created to allow for easier manipulation of Unicode locales. Unicode represents locales with a string, called a *locale identifier*. The locale identifier consists of a *language identifier* and *extension tags*. Language identifiers are the core of the locale, consisting of *language*, *script*, *region*, and *variants* subtags. Additional information about the locale is stored in the optional *extension tags*. Extension tags hold information about locale aspects such as calendar type, clock type, and numbering system type.

Traditionally, the Intl API used strings to represent locales, just as Unicode does. This is a simple and lightweight solution that works well. Adding a Locale class, however, adds ease of parsing and manipulating the language, script, and region, as well as extension tags. The following properties of `Intl.Locale` correspond to Unicode locale identifier subtags:

Property	Corresponding subtag
`language`	Language ID, first part
`script`	Language ID, part after `language`
`region`	Language ID, part after `script`
`variants`	Language ID, part after `region`
`calendar`	`ca` (extension)
`caseFirst`	`kf` (extension)
`collation`	`co` (extension)
`hourCycle`	`hc` (extension)
`numberingSystem`	`nu` (extension)
`numeric`	`kn` (extension)

The information above is exactly provided as-is when the `Locale` object is constructed, without consulting any external database. The `Intl.Locale` object additionally provides some methods that return information about the locale's real-world information, such as available calendars, collations, and numbering systems.

Constructor

`Intl.Locale()`

> Creates a new `Locale` object.

Instance properties

These properties are defined on `Intl.Locale.prototype` and shared by all `Intl.Locale` instances.

`Intl.Locale.prototype.baseName`

> Returns basic, core information about the `Locale` in the form of a substring of the complete data string.

`Intl.Locale.prototype.calendar`

> Returns the part of the `Locale` that indicates the Locale's calendar era.

`Intl.Locale.prototype.caseFirst`

Returns whether case is taken into account for the locale's collation rules.

`Intl.Locale.prototype.collation`

Returns the collation type for the `Locale`, which is used to order strings according to the locale's rules.

`Intl.Locale.prototype.constructor`

The constructor function that created the instance object. For `Intl.Locale` instances, the initial value is the `Intl.Locale` constructor.

`Intl.Locale.prototype.hourCycle`

Returns the time keeping format convention used by the locale.

`Intl.Locale.prototype.language`

Returns the language associated with the locale.

`Intl.Locale.prototype.numberingSystem`

Returns the numeral system used by the locale.

`Intl.Locale.prototype.numeric`

Returns whether the locale has special collation handling for numeric characters.

`Intl.Locale.prototype.region`

Returns the region of the world (usually a country) associated with the locale.

`Intl.Locale.prototype.script`

Returns the script used for writing the particular language used in the locale.

`Intl.Locale.prototype.variants`

Returns the variants subtags (such as different orthographies) associated with the locale.

`Intl.Locale.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.Locale"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.Locale.prototype.getCalendars()`

Returns an `Array` of available calendar identifiers, according to the locale's rules.

`Intl.Locale.prototype.getCollations()`

Returns an `Array` of the collation types for the `Locale`.

`Intl.Locale.prototype.getHourCycles()`

Returns an `Array` of hour cycle identifiers, indicating either the 12-hour clock ("h12"), the Japanese 12-hour clock ("h11"), the 24-hour clock ("h23"), or the unused format "h24".

`Intl.Locale.prototype.getNumberingSystems()`

Returns an `Array` of numbering system identifiers available according to the locale's rules.

`Intl.Locale.prototype.getTextInfo()`

Returns the part indicating the ordering of characters `ltr` (left-to-right) or `rtl` (right-to-left).

`Intl.Locale.prototype.getTimeZones()`

Returns an `Array` of time zone identifiers, associated with the `Locale`.

`Intl.Locale.prototype.getWeekInfo()`

Returns UTS 35's Week Elements according to the locale rules.

`Intl.Locale.prototype.maximize()`

Gets the most likely values for the language, script, and region of the locale based on existing values.

`Intl.Locale.prototype.minimize()`

Attempts to remove information about the locale that would be added by calling `maximize()`.

`Intl.Locale.prototype.toString()`

Returns the Locale's full locale identifier string.

Examples

Basic usage

At its very simplest, the `Intl.Locale()` constructor takes a locale identifier string as its argument:

```
const us = new Intl.Locale("en-US");
```

Using the Locale constructor with an options object

The constructor also takes an optional configuration object argument, which can contain any of several extension types. For example, set the `hourCycle` property of the configuration object to your desired hour cycle type, and then pass it into the constructor:

```
const us12hour = new Intl.Locale("en-US", { hourCycle: "h12" });
console.log(us12hour.hourCycle); // Prints "h12"
```

See also

- Polyfill of `Intl.Locale` in FormatJS
- `Intl`
- Canonical Unicode Locale Identifiers in the Unicode locale data markup language spec

Intl.Locale.prototype.baseName

The **`baseName`** accessor property of `Intl.Locale` instances returns a substring of this locale's string representation, containing core information about this locale, including the language, script, region, and variants, if available.

Description

`baseName` returns the `language ["-" script] ["-" region] *("-" variant)` subsequence of the unicode_language_id grammar. It only includes information explicitly specified in the constructor, either through the locale identifier string or the options object.

The set accessor of `baseName` is `undefined`. You cannot change this property directly.

Examples

Basic example

```
const myLoc = new Intl.Locale("fr-Latn-CA"); // Sets locale to Canadian French
console.log(myLoc.toString()); // "fr-Latn-CA-u-ca-gregory"
console.log(myLoc.baseName); // "fr-Latn-CA"
```

Example with extension tags in the input string

```
// Sets language to Japanese, region to Japan,
// calendar to Gregorian, hour cycle to 24 hours
const japan = new Intl.Locale("ja-JP-u-ca-gregory-hc-24");
console.log(japan.toString()); // "ja-JP-u-ca-gregory-hc-h24"
console.log(japan.baseName); // "ja-JP"
```

Example with options that override input string

```
// Input string indicates language as Dutch and region as Belgium,
// but options object overrides the region and sets it to the Netherlands
const dutch = new Intl.Locale("nl-Latn-BE", { region: "NL" });

console.log(dutch.baseName); // "nl-Latn-NL"
```

See also

- `Intl.Locale`

Intl.Locale.prototype.calendar

The **`calendar`** accessor property of `Intl.Locale` instances returns the calendar type for this locale.

Description

While most of the world uses the Gregorian calendar, there are several regional calendar eras used around the world. For a list of supported calendar types, see `Intl.supportedValuesOf()`.

The `calendar` property's value is set at construction time, either through the `ca` key of the locale identifier or through the `calendar` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `calendar` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the calendar type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a calendar type via the locale string

In the Unicode locale string spec, `calendar` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the calendar type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist.

Next, add the `-ca` extension to indicate that you are adding a calendar type. Finally, add the calendar era type.

```
const locale = new Intl.Locale("fr-FR-u-ca-buddhist");
console.log(locale.calendar); // "buddhist"
```

Adding a calendar type via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including calendars. Set the `calendar` property of the configuration object to your desired calendar era, and then pass it into the constructor.

```
const locale = new Intl.Locale("fr-FR", { calendar: "buddhist" });
console.log(locale.calendar); // "buddhist"
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getCalendars()`
- Unicode Calendar Identifier in the Unicode locale data markup language spec

Intl.Locale.prototype.caseFirst

The **`caseFirst`** accessor property of `Intl.Locale` instances returns whether case is taken into account for this locale's collation rules.

Description

A locale's collation rules are used to determine how strings are ordered in that locale. Certain locales use a character's case (UPPERCASE or lowercase) in the collation process. This additional rule can be expressed in an `Intl.Locale` object's `caseFirst` property. There are 3 values that the `caseFirst` property can have, outlined in the table below.

Value	Description
`upper`	Upper case to be sorted before lower case.
`lower`	Lower case to be sorted before upper case.
`false`	No special case ordering.

The `caseFirst` property's value is set at construction time, either through the `kf` key of the locale identifier or through the `caseFirst` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `caseFirst` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the `caseFirst` value can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a caseFirst value via the locale string

In the Unicode locale string spec, `caseFirst` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the `caseFirst` value to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-kf` extension to indicate that you are adding a value for `caseFirst`. Finally, add the `caseFirst` value.

```
const locale = new Intl.Locale("fr-Latn-FR-u-kf-upper");
console.log(locale.caseFirst); // "upper"
```

Adding a caseFirst value via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including `caseFirst`. Set the `caseFirst` property of the configuration object to your desired `caseFirst` value, and then pass it into the constructor.

```
const locale = new Intl.Locale("en-Latn-US", { caseFirst: "lower" });
console.log(locale.caseFirst); // "lower"
```

See also

- `Intl.Locale`
- Unicode case first collation spec

Intl.Locale.prototype.collation

The **`collation`** accessor property of `Intl.Locale` instances returns the collation type for this locale, which is used to order strings according to the locale's rules.

Description

Collation is the process of ordering strings of characters. It is used whenever strings must be sorted and placed into a certain order, from search query results to ordering records in a database. While the idea of placing strings in order might seem trivial, the idea of order can vary from region to region and language to language. For a list of supported collation types, see `Intl.supportedValuesOf()`.

The `collation` property's value is set at construction time, either through the `co` key of the locale identifier or through the `collation` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `collation` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the collation type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a collation type via the locale string

In the Unicode locale string spec, `collation` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the collation type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-co` extension to indicate that you are adding a collation type. Finally, add the collation type.

```
const locale = new Intl.Locale("zh-Hant-u-co-zhuyin");
console.log(locale.collation); // "zhuyin"
```

Adding a collation type via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including collation types. Set the `collation` property of the configuration object to your desired collation type, and then pass it into the constructor.

```
const locale = new Intl.Locale("zh-Hant", { collation: "zhuyin" });
console.log(locale.collation); // "zhuyin"
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getCollations()`

Intl.Locale.prototype.getCalendars()

The **`getCalendars()`** method of `Intl.Locale` instances returns a list of one or more unique calendar identifiers for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `calendars`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.calendars === locale.calendars` returning `false`. Check the browser compatibility table for details.

Syntax

```
getCalendars()
```

Parameters

None.

Return value

An array of strings representing all calendars commonly used for the `Locale`, sorted in descending preference. If the `Locale` already has a `calendar`, then the returned array contains that single value.

For a list of supported calendar types, see `Intl.supportedValuesOf()`.

Examples

Obtaining supported calendars

If the `Locale` object doesn't have a `calendar` already, `getCalendars()` lists all commonly-used calendars for the given `Locale`. For examples of explicitly setting a `calendar`, see `calendar` examples.

```
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getCalendars()); // ["gregory", "coptic", "islamic", "islamic-civil", "islamic-tbla"]
```

```
const jaJP = new Intl.Locale("ja-JP");
console.log(jaJP.getCalendars()); // ["gregory", "japanese"]
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.calendar`
- Unicode Calendar Identifier in the Unicode locale data markup language spec

Intl.Locale.prototype.getCollations()

The **`getCollations()`** method of `Intl.Locale` instances returns a list of one or more collation types for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `collations`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.collations === locale.collations` returning `false`. Check the browser compatibility table for details.

Syntax

```
getCollations()
```

Parameters

None.

Return value

An array of strings representing all collation types commonly used for the `Locale`, sorted in alphabetical order, with the `standard` and `search` values always excluded. If the `Locale` already has a `collation`, then the returned array contains that single value.

For a list of supported collation types, see `Intl.supportedValuesOf()`.

Examples

Obtaining supported collation types

If the `Locale` object doesn't have a `collation` already, `getCollations()` lists all commonly-used collation types for the given `Locale`. For examples of explicitly setting a `collation`, see `collation` examples.

```
const locale = new Intl.Locale("zh");
console.log(locale.getCollations()); // ["pinyin", "stroke", "zhuyin", "emoji", "eor"]
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.collation`

Intl.Locale.prototype.getHourCycles()

The **`getHourCycles()`** method of `Intl.Locale` instances returns a list of one or more unique hour cycle identifiers for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `hourCycles`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.hourCycles === locale.hourCycles` returning `false`. Check the browser compatibility table for details.

Syntax

```
getHourCycles()
```

Parameters

None.

Return value

An array of strings representing all hour cycle types commonly used for the `Locale`, sorted in descending preference. If the `Locale` already has an `hourCycle`, then the returned array contains that single value.

Below is a list of supported hour cycle types.

Supported hour cycle types

`h12`

: Hour system using 1–12; corresponds to 'h' in patterns. The 12 hour clock, with midnight starting at 12:00 am. As used, for example, in the United States.

`h23`

: Hour system using 0–23; corresponds to 'H' in patterns. The 24 hour clock, with midnight starting at 0:00.

`h11`

: Hour system using 0–11; corresponds to 'K' in patterns. The 12 hour clock, with midnight starting at 0:00 am. Mostly used in Japan.

`h24`

: Hour system using 1–24; corresponds to 'k' in pattern. The 24 hour clock, with midnight starting at 24:00. Not used anywhere.

Examples

Obtaining supported hour cycles

If the `Locale` object doesn't have a `hourCycle` already, `getHourCycles()` lists all commonly-used hour cycle identifiers for the given `Locale`. For examples of explicitly setting a `hourCycle`, see `hourCycle` examples.

```
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getHourCycles()); // ["h12"]
```

```
const jaJP = new Intl.Locale("ja-JP");
console.log(jaJP.getHourCycles()); // ["h23"]
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.hourCycle`
- Unicode Hour Cycle Identifier in the Unicode locale data markup language spec

Intl.Locale.prototype.getNumberingSystems()

The **`getNumberingSystems()`** method of `Intl.Locale` instances returns a list of one or more unique numbering system identifiers for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `numberingSystems`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.numberingSystems === locale.numberingSystems` returning `false`. Check the browser compatibility table for details.

Syntax

```
getNumberingSystems()
```

Parameters

None.

Return value

An array of strings representing all numbering systems commonly used for the `Locale`, sorted in descending preference. If the `Locale` already has a `numberingSystem`, then the returned array contains that single value.

For a list of supported numbering system types, see `Intl.supportedValuesOf()`.

Examples

Obtaining supported numbering systems

If the `Locale` object doesn't have a `numberingSystem` already, `getNumberingSystems()` lists all commonly-used numbering systems for the given `Locale`. For examples of explicitly setting a `numberingSystem`, see `numberingSystem` examples.

```
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getNumberingSystems()); // ["arab"]
```

```
const ja = new Intl.Locale("ja");
console.log(ja.getNumberingSystems()); // ["latn"]
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.numberingSystem`
- Details on the standard Unicode numeral systems

Intl.Locale.prototype.getTextInfo()

The **`getTextInfo()`** method of `Intl.Locale` instances returns the ordering of characters indicated by either `ltr` (left-to-right) or by `rtl` (right-to-left) for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `textInfo`. However, because it returns a new object on each access, it is now implemented as a method to prevent the situation of `locale.textInfo === locale.textInfo` returning `false`. Check the browser compatibility table for details.

Syntax

```
getTextInfo()
```

Parameters

None.

Return value

An object representing text typesetting information associated with the Locale data specified in UTS 35's Layouts Elements. It has the following properties:

`direction`

A string indicating the direction of text for the locale. Can be either `"ltr"` (left-to-right) or `"rtl"` (right-to-left).

Examples

Obtaining text info

Return the supported text directions for a given `Locale`.

```
const ar = new Intl.Locale("ar");
console.log(ar.getTextInfo()); // { direction: "rtl" }
console.log(ar.getTextInfo().direction); // "rtl"
```

```
const es = new Intl.Locale("es");
console.log(es.getTextInfo()); // { direction: "ltr" }
console.log(es.getTextInfo().direction); // "ltr"
```

See also

- `Intl.Locale`

Intl.Locale.prototype.getTimeZones()

The **`getTimeZones()`** method of `Intl.Locale` instances returns a list of supported time zones for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `timeZones`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.timeZones === locale.timeZones` returning `false`. Check the browser compatibility table for details.

Syntax

```
getTimeZones()
```

Parameters

None.

Return value

An array of strings representing supported time zones for the associated `Locale`, where each value is an IANA time zone canonical name, sorted in alphabetical order. If the locale identifier does not contain a region subtag, the returned value is `undefined`.

> **Note:** The standardization of `Temporal` requires browsers to always return the primary identifier in the IANA database, which may change over time. See time zones and offsets for more information.

Examples

Obtaining supported time zones

List supported time zones for a given `Locale`.

```
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getTimeZones()); // ["Africa/Cairo"]
```

```
const jaJP = new Intl.Locale("ja-JP");
console.log(jaJP.getTimeZones()); // ["Asia/Tokyo"]
```

```
const ar = new Intl.Locale("ar");
console.log(ar.getTimeZones()); // undefined
```

See also

- `Intl.Locale`
- IANA time zone database on Wikipedia

Intl.Locale.prototype.getWeekInfo()

The **`getWeekInfo()`** method of `Intl.Locale` instances returns a `weekInfo` object with the properties `firstDay`, `weekend` and `minimalDays` for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `weekInfo`. However, because it returns a new object on each access, it is now implemented as a method to prevent the situation of `locale.weekInfo === locale.weekInfo` returning `false`. Check the browser compatibility table for details.

Syntax

```
getWeekInfo()
```

Parameters

None.

Return value

An object representing week information associated with the Locale data specified in UTS 35's Week Elements. It has the following properties:

`firstDay`

An integer between 1 (Monday) and 7 (Sunday) indicating the first day of the week for the locale. Commonly 1, 5, 6, or 7.

`weekend`

An array of integers between 1 and 7 indicating the weekend days for the locale. This is usually continuous because UTS 35 stores `weekendStart` and `weekendEnd` instead.

`minimalDays`

An integer between 1 and 7 (commonly 1 and 4) indicating the minimal days required in the first week of a month or year, for week-of-year or week-of-month calculations (e.g., The 20th week of the year). For example, in the ISO 8601 calendar, the first week of a year must have at least 4 days in this year, so if January 1 is a Friday, Saturday, or Sunday, it will be numbered as part of the last week of the previous year.

Examples

Obtaining the Week Information

Return the week information for a given `Locale`.

```
const he = new Intl.Locale("he"); // Hebrew (Israel)
console.log(he.getWeekInfo()); // { firstDay: 7, weekend: [5, 6], minimalDays: 1 }

const af = new Intl.Locale("af"); // Afrikaans (South Africa)
console.log(af.getWeekInfo()); // { firstDay: 7, weekend: [6, 7], minimalDays: 1 }

const enGB = new Intl.Locale("en-GB"); // English (United Kingdom)
console.log(enGB.getWeekInfo()); // { firstDay: 1, weekend: [6, 7], minimalDays: 4 }

const arAF = new Intl.Locale("ar-AF"); // Arabic (Afghanistan)
console.log(arAF.getWeekInfo()); // { firstDay: 6, weekend: [4, 5], minimalDays: 1 }

const dvMV = new Intl.Locale("dv-MV"); // Divehi (Maldives)
console.log(dvMV.getWeekInfo()); // { firstDay: 5, weekend: [6, 7], minimalDays: 1 }
```

See also

- `Intl.Locale`

Intl.Locale.prototype.hourCycle

The **`hourCycle`** accessor property of `Intl.Locale` instances returns the hour cycle type for this locale.

Description

There are 2 main types of time keeping conventions (clocks) used around the world: the 12 hour clock and the 24 hour clock. For a list of supported hour cycle types, see `Intl.Locale.prototype.getHourCycles()`.

The `hourCycle` property's value is set at construction time, either through the `hc` key of the locale identifier or through the `hourCycle` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `hourCycle` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the hour cycle type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding an hour cycle via the locale string

In the Unicode locale string spec, `hourCycle` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the hour cycle type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-hc` extension to indicate that you are adding an hour cycle. Finally, add the hour cycle type.

```
const locale = new Intl.Locale("fr-FR-u-hc-h23");
console.log(locale.hourCycle); // "h23"
```

Adding an hour cycle via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including hour cycle types. Set the `hourCycle` property of the configuration object to your desired hour cycle type, and then pass it into the constructor.

```
const locale = new Intl.Locale("en-US", { hourCycle: "h12" });
console.log(locale.hourCycle); // "h12"
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getHourCycles()`

- Unicode Hour Cycle Identifier in the Unicode locale data markup language spec

Intl.Locale.prototype.language

The **`language`** accessor property of `Intl.Locale` instances returns the language associated with this locale.

Description

Language is one of the core attributes of a locale. The Unicode specification treats the language identifier of a locale as the language and the region together (to make a distinction between dialects and variations, e.g., British English vs. American English). The `language` property of an `Intl.Locale` returns strictly the locale's language subtag.

The `language` property's value is set at construction time, either through the first part of the locale identifier or through the `language` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present.

The set accessor of `language` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the language can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Setting the language via the locale string

In order to be a valid Unicode locale identifier, a string must start with the language subtag. The main argument to the `Intl.Locale()` constructor must be a valid Unicode locale identifier, so whenever the constructor is used, it must be passed an identifier with a language subtag.

```
const locale = new Intl.Locale("en-Latn-US");
console.log(locale.language); // "en"
```

Overriding language via the configuration object argument

While the language subtag must be specified, the `Intl.Locale()` constructor has an optional configuration object argument, which can override the language subtag.

```
const locale = new Intl.Locale("en-Latn-US", { language: "es" });
console.log(locale.language); // "es"
```

See also

- `Intl.Locale`
- Unicode language subtag in the Unicode locale data markup language spec

Intl.Locale() constructor

The **`Intl.Locale()`** constructor creates `Intl.Locale` objects.

Try it

```
const korean = new Intl.Locale("ko", {
  script: "Kore",
  region: "KR",
  hourCycle: "h23",
  calendar: "gregory",
});

const japanese = new Intl.Locale("ja-Jpan-JP-u-ca-japanese-hc-h12");

console.log(korean.baseName, japanese.baseName);
// Expected output: "ko-Kore-KR" "ja-Jpan-JP"

console.log(korean.hourCycle, japanese.hourCycle);
// Expected output: "h23" "h12"
```

Syntax

```
new Intl.Locale(tag)
new Intl.Locale(tag, options)
```

> **Note:** `Intl.Locale()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`tag`

The Unicode locale identifier string. For the syntax of locale identifier strings, see the Intl main page. Note that the `Intl.Locale` constructor, unlike most other `Intl` constructors, does not accept an array of locales or `undefined`.

`options`

An object that contains configuration for the Locale. Option values here take priority over extension keys in the locale identifier. Possible properties are:

`language`

The language. Any syntactically valid string following the `unicode_language_subtag` grammar (2–3 or 5–8 letters) is accepted, but the implementation only recognizes certain kinds.

`script`

The script. Any syntactically valid string following the `unicode_script_subtag` grammar (4 letters) is accepted, but the implementation only recognizes certain kinds.

`region`

The region. Any syntactically valid string following the `unicode_region_subtag` grammar (either 2 letters or 3 digits) is accepted, but the implementation only recognizes certain kinds.

`variants`

The variants. It should be a dash (`-`) separated list of *unique* variant tags, where each tag is any syntactically valid string following the `unicode_variant_subtag` grammar (either 5–8 alphanumerals or a digit followed by 3 alphanumerals), but the implementation only recognizes certain kinds.

`calendar`

The calendar. Any syntactically valid string following the `type` grammar (one or more segments of 3–8 alphanumerals, joined by hyphens) is accepted, but the implementation only recognizes certain kinds, which are listed in `Intl.supportedValuesOf()`.

`collation`

The collation. Any syntactically valid string following the `type` grammar is accepted, but the implementation only recognizes certain kinds, which are listed in `Intl.supportedValuesOf()`.

`numberingSystem`

The numbering system. Any syntactically valid string following the `type` grammar is accepted, but the implementation only recognizes certain kinds, which are listed in `Intl.supportedValuesOf()`.

`caseFirst`

The case-first sort option. Possible values are `"upper"`, `"lower"`, or `"false"`.

`hourCycle`

The hour cycle. Possible values are `"h23"`, `"h12"`, `"h11"`, or the practically unused `"h24"`, which are explained in `Intl.Locale.prototype.getHourCycles`

`numeric`

The numeric sort option. A boolean.

Examples

Basic usage

At its very simplest, the `Intl.Locale()` constructor takes a locale identifier string as its argument:

```
const us = new Intl.Locale("en-US");
```

Using the Locale constructor with an options object

The constructor also takes an optional configuration object argument, which can contain any of several extension types. For example, set the `hourCycle` property of the configuration object to your desired hour cycle type, and then pass it into the constructor:

```
const locale = new Intl.Locale("en-US", { hourCycle: "h12" });
console.log(locale.hourCycle); // "h12"
```

See also

- `Intl.Collator`
- Canonical Unicode Locale Identifiers in the Unicode locale data markup language spec

Intl.Locale.prototype.maximize()

The **`maximize()`** method of `Intl.Locale` instances gets the most likely values for the language, script, and region of this locale based on existing values.

Try it

```
const english = new Intl.Locale("en");
const korean = new Intl.Locale("ko");
const arabic = new Intl.Locale("ar");

console.log(english.maximize().baseName);
// Expected output: "en-Latn-US"

console.log(korean.maximize().baseName);
// Expected output: "ko-Kore-KR"
```

```
console.log(arabic.maximize().baseName);
// Expected output: "ar-Arab-EG"
```

Syntax

```
maximize()
```

Parameters

None.

Return value

An `Intl.Locale` instance whose `baseName` property returns the result of the Add Likely Subtags algorithm executed against *`Locale.baseName`*.

Description

Sometimes, it is convenient to be able to identify the most likely locale language identifier subtags based on an incomplete language ID. The Add Likely Subtags algorithm gives us this functionality. For instance, given the language ID "en", the algorithm would return "en-Latn-US", since English can only be written in the Latin script, and is most likely to be used in the United States, as it is the largest English-speaking country in the world. This functionality is provided to JavaScript programmers via the `maximize()` method. `maximize()` only affects the main subtags that comprise the language identifier: language, script, and region subtags. Other subtags after the "-u" in the locale identifier are called extension subtags and are not affected by the `maximize()` method. Examples of these subtags include `hourCycle`, `calendar`, and `numeric`.

Examples

Using maximize

```
const myLocale = new Intl.Locale("fr", {
  hourCycle: "h12",
  calendar: "gregory",
});
console.log(myLocale.baseName); // Prints "fr"
console.log(myLocale.toString()); // Prints "fr-u-ca-gregory-hc-h12"
const myLocMaximized = myLocale.maximize();

// Prints "fr-Latn-FR". The "Latn" and "FR" tags are added,
// since French is only written in the Latin script and is most likely to be spoken in France.
console.log(myLocMaximized.baseName);

// Prints "fr-Latn-FR-u-ca-gregory-hc-h12".
// Note that the extension tags (after "-u") remain unchanged.
console.log(myLocMaximized.toString());
```

See also

- `Intl.Locale`
- `baseName`
- Likely Subtags in the Unicode locale data markup language spec

Intl.Locale.prototype.minimize()

The **`minimize()`** method of `Intl.Locale` instances attempts to remove information about this locale that would be added by calling `maximize()`.

Try it

```
const english = new Intl.Locale("en-Latn-US");
const korean = new Intl.Locale("ko-Kore-KR");
const arabic = new Intl.Locale("ar-Arab-EG");

console.log(english.minimize().baseName);
// Expected output: "en"

console.log(korean.minimize().baseName);
// Expected output: "ko"

console.log(arabic.minimize().baseName);
// Expected output: "ar"
```

Syntax

```
minimize()
```

Parameters

None.

Return value

An `Intl.Locale` instance whose `baseName` property returns the result of the Remove Likely Subtags algorithm executed against *`Locale.baseName`*.

Description

This method carries out the reverse of `maximize()`, removing any language, script, or region subtags from the locale language identifier (essentially the contents of `baseName`). This is useful when there are superfluous subtags in the language identifier; for instance, "en-Latn" can be simplified to "en", since "Latn" is the only script used to write English. `minimize()` only affects the main subtags that comprise the language identifier: language, script, and region subtags. Other subtags after the "-u" in the locale identifier are called extension subtags and are not affected by the `minimize()` method. Examples of these subtags include `hourCycle`, `calendar`, and `numeric`.

Examples

Using minimize

```
const myLocale = new Intl.Locale("fr-Latn-FR", {
  hourCycle: "h12",
  calendar: "gregory",
});
console.log(myLocale.baseName); // Prints "fr-Latn-FR"
console.log(myLocale.toString()); // Prints "fr-Latn-FR-u-ca-gregory-hc-h12"

const myLocMinimized = myLocale.minimize();

// Prints "fr", since French is only written in the Latin script
// and is most likely to be spoken in France.
console.log(myLocMinimized.baseName);

// Prints "fr-u-ca-gregory-hc-h12".
// Note that the extension tags (after "-u") remain unchanged.
console.log(myLocMinimized.toString());
```

See also

- `Intl.Locale`
- `baseName`

Intl.Locale.prototype.numberingSystem

The **`numberingSystem`** accessor property of `Intl.Locale` instances returns the numeral system for this locale.

Description

A numeral system is a system for expressing numbers. For a list of supported numbering system types, see `Intl.supportedValuesOf()`.

The `numberingSystem` property's value is set at construction time, either through the `nu` key of the locale identifier or through the `numberingSystem` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `numberingSystem` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the numbering system type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a numbering system via the locale string

In the Unicode locale string spec, `numberingSystem` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the numbering system type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-nu` extension to indicate that you are adding a numbering system. Finally, add the numbering system type.

```
const locale = new Intl.Locale("fr-Latn-FR-u-nu-mong");
console.log(locale.numberingSystem); // "mong"
```

Adding a numbering system via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including numbering system types. Set the `numberingSystem` property of the configuration object to your desired numbering system type, and then pass it into the constructor.

```
const locale = new Intl.Locale("en-Latn-US", { numberingSystem: "latn" });
console.log(locale.numberingSystem); // "latn"
```

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getNumberingSystems()`
- Details on the standard Unicode numeral systems

Intl.Locale.prototype.numeric

The **`numeric`** accessor property of `Intl.Locale` instances returns whether this locale has special collation handling for numeric characters.

Description

Like `caseFirst`, `numeric` represents a modification to the collation rules utilized by the locale. `numeric` is a boolean value, which means that it can be either `true` or `false`. If `numeric` is set to `false`, there will be no special handling of numeric values in strings. If `numeric` is set to `true`, then the locale will take numeric characters into

account when collating strings. This special numeric handling means that sequences of decimal digits will be compared as numbers. For example, the string "A-21" will be considered less than "A-123".

The `numeric` property's value is set at construction time, either through the `kn` key of the locale identifier or through the `numeric` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `numeric` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the `numeric` value can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a <code>numeric</code> value via the locale string

In the Unicode locale string spec, `numeric` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the `numeric` value to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-kn` extension to indicate that you are adding a value for `numeric`. Finally, add the `numeric` value. If you want to set `numeric` to `true`, adding the `kn` key will suffice. To set the value to `false`, you must specify in by adding `"false"` after the `kn` key.

```
const locale = new Intl.Locale("fr-Latn-FR-u-kn-false");
console.log(locale.numeric); // "false"
```

Adding a <code>numeric</code> value via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including `numeric`. Set the `numeric` property of the configuration object to your desired `numeric` value, and then pass it into the constructor.

```
const locale = new Intl.Locale("en-Latn-US", { numeric: true });
console.log(locale.numeric); // "true"
```

See also

- `Intl.Locale`

Intl.Locale.prototype.region

The **`region`** accessor property of `Intl.Locale` instances returns the region of the world (usually a country) associated with this locale.

Description

Region is one of the core attributes of a locale. It allows selection for differences between the same language in, say, different countries. For example, English is spoken in the United Kingdom and the United States of America, but there are differences in spelling and other language conventions between those two countries. Knowing the locale's region helps JavaScript programmers make sure that the content from their sites and applications is correctly displayed when viewed from different areas of the world.

The `region` property's value is set at construction time, either through the part of the locale identifier after `script` or through the `region` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `region` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the region can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a region via the locale string

The region, if present, is the third part (if `script` is present, second part otherwise) of a valid Unicode language identifier string, and can be added to the initial locale identifier string that is passed into the `Intl.Locale()` constructor. Note that the region is not a required part of a locale identifier.

```
const locale = new Intl.Locale("en-Latn-US");
console.log(locale.region); // "US"
```

Adding a region via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument. Set the `region` property of the configuration object to your desired region, and then pass it into the constructor.

```
const locale = new Intl.Locale("fr-Latn", { region: "FR" });
console.log(locale.region); // "FR"
```

See also

- `Intl.Locale`
- Unicode region chart

Intl.Locale.prototype.script

The **`script`** accessor property of `Intl.Locale` instances returns the script used for writing the particular language used in this locale.

Description

Script, sometimes called writing system, is one of the core attributes of a locale. It indicates the set of symbols, or glyphs, that are used to write a particular language. For instance, the script associated with English is Latin, whereas the script typically associated with Korean is Hangul. In many cases, denoting a script is not strictly necessary, since the language (which is necessary) is only written in a single script. There are exceptions to this rule, however, and it is important to indicate the script when multiple scripts are applicable.

The `script` property's value is set at construction time, either through the part of the locale identifier after `language` or through the `script` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `script` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the script can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a script via the locale string

The script, if present, is the second part of a valid Unicode language identifier string, and can be added to the initial locale identifier string that is passed into the `Intl.Locale()` constructor. Note that the script is not a required part of a locale identifier.

```
const locale = new Intl.Locale("en-Latn-US");
console.log(locale.script); // "Latn"
```

Adding a script via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument. Set the `script` property of the configuration object to your desired script, and then pass it into the constructor.

```
const locale = new Intl.Locale("fr-FR", { script: "Latn" });
console.log(locale.script); // "Latn"
```

See also

- `Intl.Locale`
- Unicode script subtag in the Unicode locale data markup language spec

Intl.Locale.prototype.toString()

The **`toString()`** method of `Intl.Locale` instances returns this Locale's full locale identifier string.

Try it

```
const french = new Intl.Locale("fr-Latn-FR", {
  calendar: "gregory",
  hourCycle: "h12",
});
const korean = new Intl.Locale("ko-Kore-KR", {
  numeric: true,
  caseFirst: "upper",
});

console.log(french.toString());
// Expected output: "fr-Latn-FR-u-ca-gregory-hc-h12"

console.log(korean.toString());
// Expected output: "ko-Kore-KR-u-kf-upper-kn"
```

Syntax

```
toString()
```

Parameters

None.

Return value

The *locale*'s Unicode locale identifier string.

Description

The `Locale` object is a JavaScript representation of a concept Unicode locale identifier. Information about a particular locale (language, script, calendar type, etc.) can be encoded in a locale identifier string. To make it easier to work with these locale identifiers, the `Locale` object was introduced to JavaScript. Calling the `toString` method on a Locale object will return the identifier string for that particular Locale. The `toString` method allows `Locale` instances to be provided as an argument to existing `Intl` constructors, serialized in JSON, or any other context where an exact string representation is useful.

Examples

Using toString

```
const myLocale = new Intl.Locale("fr-Latn-FR", {
  hourCycle: "h12",
  calendar: "gregory",
```

```
});
console.log(myLocale.baseName); // Prints "fr-Latn-FR"
console.log(myLocale.toString()); // Prints "fr-Latn-FR-u-ca-gregory-hc-h12"
```

See also

- `Intl.Locale`
- `baseName`

Intl.Locale.prototype.variants

The **`variants`** accessor property of `Intl.Locale` instances returns the variants associated with this locale, as a string of dash (`-`) separated identifiers in the originally specified order.

Description

Variants are a part of the main language ID. They select variants of a language that the (language, region, script) triple cannot differentiate. Usually, they represent the same language in different eras or different orthographies. For example, German has the `1901` and `1996` orthography variants, which are written as `de-1901` and `de-1996`; the "Early Modern English (1500-1700)" variant is written as `en-emodeng`. The subtag can contain multiple identifiers separated by dashes (`-`). These identifiers are technically unordered, although in practice they often have a semantic hierarchy—for example, the Resian dialect of Slovenian is written as `sl-rozaj`, and the San Giorgio/Bila dialect of Resian is written as `sl-rozaj-biske`.

The `variants` property's value is set at construction time, either through the part of the locale identifier after `region` or through the `variants` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `variants` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the variants can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding variants via the locale string

The variants, if present, are the last parts of a valid Unicode language identifier string, and can be added to the initial locale identifier string that is passed into the `Intl.Locale()` constructor. Note that the variants are not a required part of a locale identifier.

```
const locale = new Intl.Locale("sl-rozaj-biske");
console.log(locale.variants); // "rozaj-biske"
```

Adding variants via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument. Set the `variants` property of the configuration object to your desired variants, and then pass it into the constructor.

```
const locale = new Intl.Locale("sl", { variants: "rozaj-biske" });
console.log(locale.variants); // "rozaj-biske"
```

See also

- `Intl.Locale`
- Unicode variant subtag in the Unicode locale data markup language spec
- IANA language subtag registry

Intl.NumberFormat

The **`Intl.NumberFormat`** object enables language-sensitive number formatting.

Try it

```
const number = 123456.789;

console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// Expected output: "123.456,79 €"

// The Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// Expected output: "￥123,457"

// Limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
```

```
);
// Expected output: "1,23,000"
```

Constructor

`Intl.NumberFormat()`

Creates a new `NumberFormat` object.

Static methods

`Intl.NumberFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.NumberFormat.prototype` and shared by all `Intl.NumberFormat` instances.

`Intl.NumberFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.NumberFormat` instances, the initial value is the `Intl.NumberFormat` constructor.

`Intl.NumberFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.NumberFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.NumberFormat.prototype.format()`

Getter function that formats a number according to the locale and formatting options of this `Intl.NumberFormat` object.

`Intl.NumberFormat.prototype.formatRange()`

Getter function that formats a range of numbers according to the locale and formatting options of the `Intl.NumberFormat` object from which the method is called.

`Intl.NumberFormat.prototype.formatRangeToParts()`

Returns an `Array` of objects representing the range of number strings in parts that can be used for custom locale-aware formatting.

`Intl.NumberFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the number string in parts that can be used for custom locale-aware formatting.

`Intl.NumberFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

```
const number = 3500;

console.log(new Intl.NumberFormat().format(number));
// '3,500' if in US English locale
```

Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const number = 123456.789;

// German uses comma as decimal separator and period for thousands
console.log(new Intl.NumberFormat("de-DE").format(number));
// 123.456,789

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(new Intl.NumberFormat("ar-EG").format(number));
// ١٢٣٤٥٦٫٧٨٩

// India uses thousands/lakh/crore separators
console.log(new Intl.NumberFormat("en-IN").format(number));
// 1,23,456.789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(new Intl.NumberFormat("zh-Hans-CN-u-nu-hanidec").format(number));
// 一二三,四五六.七八九

// when requesting a language that may not be supported, such as
```

```
// Balinese, include a fallback language, in this case Indonesian
console.log(new Intl.NumberFormat(["ban", "id"]).format(number));
// 123.456,789
```

Using options

The results can be customized using the `options` argument:

```
const number = 123456.789;

// request a currency format
console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// 123.456,79 €

// the Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// ￥123,457

// limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
);
// 1,23,000

// Formatting with units
console.log(
  new Intl.NumberFormat("pt-PT", {
    style: "unit",
    unit: "kilometer-per-hour",
  }).format(50),
);
// 50 km/h

console.log(
  (16).toLocaleString("en-GB", {
    style: "unit",
    unit: "liter",
    unitDisplay: "long",
  }),
);
// 16 litres
```

For an exhaustive list of options, see the `Intl.NumberFormat()` constructor page.

See also

- Polyfill of `Intl.NumberFormat` in FormatJS
- `Intl`
- `Number.prototype.toLocaleString()`

Intl.NumberFormat.prototype.format()

The **`format()`** method of `Intl.NumberFormat` instances formats a number according to the locale and formatting options of this `Intl.NumberFormat` object.

Try it

```
const amount = 654321.987;

const options1 = { style: "currency", currency: "RUB" };
const numberFormat1 = new Intl.NumberFormat("ru-RU", options1);

console.log(numberFormat1.format(amount));
// Expected output: "654 321,99 ₽"

const options2 = { style: "currency", currency: "USD" };
const numberFormat2 = new Intl.NumberFormat("en-US", options2);

console.log(numberFormat2.format(amount));
// Expected output: "$654,321.99"
```

Syntax

```
format(number)
```

Parameters

`number`

> A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `format()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

> **Note:** Older versions of the specification parsed strings as a `Number`. Check the compatibility table for your browser.

Return value

A string representing the given `number` formatted according to the locale and formatting options of this `Intl.NumberFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Description

`Number` values in JavaScript suffer from loss of precision if they are too big or too small, making the text representation inaccurate. If you are performing calculations with integers larger than `Number.MAX_SAFE_INTEGER` you should use a `BigInt` instead, which will format correctly:

```
new Intl.NumberFormat("en-US").format(1234567891234567891); // 1,234,567,891,234,568,000
new Intl.NumberFormat("en-US").format(1234567891234567891n); // 1,234,567,891,234,567,891
```

You can also pass through very large strings to be formatted as an arbitrary-precision decimal string (if you're performing calculations on the data you will still need to work with `BigInt`):

```
new Intl.NumberFormat("en-US").format("1234567891234567891"); // 1,234,567,891,234,567,891
```

Examples

Using format

Use the `format` getter function for formatting a single currency value. The code below shows how to format the roubles currency for a Russian locale:

```
const options = { style: "currency", currency: "RUB" };
const numberFormat = new Intl.NumberFormat("ru-RU", options);
console.log(numberFormat.format(654321.987));
// "654 321,99 ₽"
```

Using format with map

Use the `format` getter function for formatting all numbers in an array. Note that the function is bound to the `Intl.NumberFormat` from which it was obtained, so it can be passed directly to `Array.prototype.map`. This is considered a historical artifact, as part of a convention which is no longer followed for new features, but is preserved to maintain compatibility with existing programs.

```
const a = [123456.789, 987654.321, 456789.123];
const numberFormat = new Intl.NumberFormat("es-ES");
const formatted = a.map((n) => numberFormat.format(n));
console.log(formatted.join("; "));
// "123.456,789; 987.654,321; 456.789,123"
```

Using format with a string

Using a string we can specify numbers that are larger than `Number.MAX_SAFE_INTEGER` without losing precision.

```
const numberFormat = new Intl.NumberFormat("en-US");

// Here the value is converted to a Number
console.log(numberFormat.format(987654321987654321));
// 987,654,321,987,654,300

// Here we use a string and don't lose precision
console.log(numberFormat.format("987654321987654321"));
// 987,654,321,987,654,321
```

We can also use the general "E" exponent syntax for decimal strings: `#.#E#`. The code below creates a `BigInt`, coerces it to a string with the suffix `E-6`, and then formats it.

```
const numberFormat = new Intl.NumberFormat("en-US");
const bigNum = 1000000000000000110000n;
console.log(numberFormat.format(bigNum));
// "1,000,000,000,000,000,110,000"

// Format as a string using the `E` syntax:
```

```
console.log(numberFormat.format(`${bigNum}E-6`));
// "1,000,000,000,000,000.11"
```

See also

- `Intl.NumberFormat`
- `Number.prototype.toLocaleString()`

Intl.NumberFormat.prototype.formatRange()

The **`formatRange()`** method of `Intl.NumberFormat` instances formats a range of numbers according to the locale and formatting options of this `Intl.NumberFormat` object.

Syntax

```
formatRange(startRange, endRange)
```

Parameters

`startRange`

A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `formatRange()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

`endRange`

A `Number`, `BigInt`, or string, to format.

Return value

A string representing the given range of numbers formatted according to the locale and formatting options of this `Intl.NumberFormat` object. If the start and end values are formatted to the same string, the output will only contain a single value, possibly prefixed with an "approximately equals" symbol (e.g., "~$3"). The insertion of this symbol only depends on the locale settings, and is inserted even when `startRange === endRange`.

Exceptions

`RangeError`

Thrown if either `startRange` or `endRange` is `NaN` or an inconvertible string.

`TypeError`

Thrown if either `startRange` or `endRange` is undefined.

Description

The `formatRange` getter function formats a range of numbers into a string according to the locale and formatting options of this `Intl.NumberFormat` object from which it is called.

Examples

Using formatRange

Use the `formatRange` getter function for formatting a range of currency values:

```
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 0,
});

console.log(nf.formatRange(3, 5)); // "$3 – $5"

// Note: the "approximately equals" symbol is added if
// startRange and endRange round to the same values.
console.log(nf.formatRange(2.9, 3.1)); // "~$3"
```

```
const nf = new Intl.NumberFormat("es-ES", {
  style: "currency",
  currency: "EUR",
  maximumFractionDigits: 0,
});

console.log(nf.formatRange(3, 5)); // "3-5 €"
console.log(nf.formatRange(2.9, 3.1)); // "~3 €"
```

See also

- `Intl.NumberFormat`
- `Number.prototype.toLocaleString()`

Intl.NumberFormat.prototype.formatRangeToParts()

The **`formatRangeToParts()`** method of `Intl.NumberFormat` instances returns an `Array` of objects containing the locale-specific tokens from which it is possible to build custom strings while preserving the locale-specific parts. This makes it possible to provide locale-aware custom formatting ranges of number strings.

Syntax

```
formatRangeToParts(startRange, endRange)
```

Parameters

`startRange`

A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `formatRangeToParts()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

`endRange`

A `Number`, `BigInt`, or string, to format.

Return value

An `Array` of objects containing the formatted range in parts. Each object has three properties, `type`, `value`, and `source`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `formatRange()`. The `type` may have the same values as `formatToParts()`, or the additional value `"approximatelySign"` (see below). The `source` can be one of the following:

`startRange`

The token is a part of the start number.

`endRange`

The token is a part of the end number.

`shared`

The token is shared between the start and end; for example, the currency symbol. All literals that are part of the range pattern itself, such as the `"–"` separator, are also marked as `shared`.

If the start and end numbers are formatted to the same string, then the output has the same list of tokens as calling `formatToParts()` on the start number, with all tokens marked as `source: "shared"`. In addition, the first token may be an "approximately equals" symbol (e.g., `"~"`) with `type: "approximatelySign"`. The insertion of this symbol only depends on the locale settings, and is inserted even when `startRange === endRange`.

Exceptions

`RangeError`

Thrown if either `startRange` or `endRange` is `NaN` or an inconvertible string.

`TypeError`

Thrown if either `startRange` or `endRange` is undefined.

Examples

Using formatRangeToParts()

The `formatRange()` method outputs localized, opaque strings that cannot be manipulated directly:

```
const startRange = 3500;
const endRange = 9500;

const formatter = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "EUR",
});

console.log(formatter.formatRange(startRange, endRange));
// "3.500,00–9.500,00 €"
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatRangeToParts()` method produces the same information in parts:

```
console.log(formatter.formatRangeToParts(startRange, endRange));

// return value:
[
  { type: "integer", value: "3", source: "startRange" },
  { type: "group", value: ".", source: "startRange" },
  { type: "integer", value: "500", source: "startRange" },
  { type: "decimal", value: ",", source: "startRange" },
  { type: "fraction", value: "00", source: "startRange" },
  { type: "literal", value: "–", source: "shared" },
  { type: "integer", value: "9", source: "endRange" },
  { type: "group", value: ".", source: "endRange" },
  { type: "integer", value: "500", source: "endRange" },
```

```
  { type: "decimal", value: ",", source: "endRange" },
  { type: "fraction", value: "00", source: "endRange" },
  { type: "literal", value: " ", source: "shared" },
  { type: "currency", value: "€", source: "shared" },
];
```

See also

- `Intl.NumberFormat`
- `Intl.NumberFormat.prototype.format()`

Intl.NumberFormat.prototype.formatToParts()

The **`formatToParts()`** method of `Intl.NumberFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

```
const amount = 654321.987;
const options = { style: "currency", currency: "USD" };
const numberFormat = new Intl.NumberFormat("en-US", options);

const parts = numberFormat.formatToParts(amount);
const partValues = parts.map((p) => p.value);

console.log(partValues);
// Expected output: "["$", "654", ",", "321", ".", "99"]"
```

Syntax

```
formatToParts(number)
```

Parameters

`number`

> A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `formatToParts()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

Return value

An `Array` of objects containing the formatted number in parts. Each object has two properties, `type` and `value`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The `type` may be one of the following:

`literal`

> Any string that's a part of the format pattern; for example `" "`. Note that common tokens like the decimal separator or the plus/minus signs have their own token types.

`integer`

> The integral part of the number, or a segment of it if using grouping (controlled by `options.useGrouping`).

`group`

> The group separator string, such as `","`. Only present when using grouping (controlled by `options.useGrouping`).

`decimal`

> The decimal separator string, such as `"."`. Only present when `fraction` is present.

`fraction`

> The fractional part of the number.

`compact`

> The compact exponent, such as `"M"` or `"thousands"`. Only present when `options.notation` is `"compact"`. The form (`"short"` or `"long"`) can be controlled via `options.compactDisplay`.

`exponentSeparator`

> The exponent separator, such as `"E"`. Only present when `options.notation` is `"scientific"` or `"engineering"`.

`exponentMinusSign`

> The exponent minus sign string, such as `"-"`. Only present when `options.notation` is `"scientific"` or `"engineering"` and the exponent is negative.

`exponentInteger`

> The exponent's integer value. Only present when `options.notation` is `"scientific"` or `"engineering"`.

`nan`

A string representing `NaN`, such as `"NaN"`. This is the sole token representing the number itself when the number is `NaN`.

`infinity`

A string representing `Infinity` or `-Infinity`, such as `"∞"`. This is the sole token representing the number itself when the number is `Infinity` or `-Infinity`.

`plusSign`

The plus sign, such as `"+"`.

`minusSign`

The minus sign, such as `"-"`.

`percentSign`

The percent sign, such as `"%"`. Only present when `options.style` is `"percent"`.

`unit`

The unit string, such as `"l"` or `"litres"`. Only present when `options.style` is `"unit"`. The form (`"short"`, `"narrow"`, or `"long"`) can be controlled via `options.unitDisplay`.

`currency`

The currency string, such as `"$"`, `"€"`, `"Dollar"`, or `"Euro"`. Only present when `options.style` is `"currency"`. The form (`"code"`, `"symbol"`, `"narrowSymbol"`, or `"name"`) can be controlled via `options.currencyDisplay`.

`unknown`

Reserved for any token that's not recognized as one of the above; should be rarely encountered.

Examples

Using formatToParts()

The `format()` method outputs localized, opaque strings that cannot be manipulated directly:

```
const number = 3500;

const formatter = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "EUR",
});

formatter.format(number);
// "3.500,00 €"
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatToParts()` method produces the same information in parts:

```
formatter.formatToParts(number);

// return value:
[
  { type: "integer", value: "3" },
  { type: "group", value: "." },
  { type: "integer", value: "500" },
  { type: "decimal", value: "," },
  { type: "fraction", value: "00" },
  { type: "literal", value: " " },
  { type: "currency", value: "€" },
];
```

Now the information is available separately and it can be formatted and concatenated again in a customized way. For example by using `Array.prototype.map()`, arrow functions, a switch statement, template literals, and `Array.prototype.join()`, to insert additional markup for certain components.

```
const numberString = formatter
  .formatToParts(number)
  .map(({ type, value }) => {
    switch (type) {
      case "currency":
        return `<strong>${value}</strong>`;
      default:
        return value;
    }
  })
  .join("");

console.log(numberString);
// "3.500,00 <strong>€</strong>"
```

See also

- `Intl.NumberFormat`
- `Intl.NumberFormat.prototype.format()`

Intl.NumberFormat() constructor

The **`Intl.NumberFormat()`** constructor creates `Intl.NumberFormat` objects.

Try it

```
const number = 123456.789;

console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// Expected output: "123.456,79 €"

// The Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// Expected output: "￥123,457"

// Limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
);
// Expected output: "1,23,000"
```

Syntax

```
new Intl.NumberFormat()
new Intl.NumberFormat(locales)
new Intl.NumberFormat(locales, options)

Intl.NumberFormat()
Intl.NumberFormat(locales)
Intl.NumberFormat(locales, options)
```

> **Note:** `Intl.NumberFormat()` can be called with or without `new`. Both create a new `Intl.NumberFormat` instance. However, there's a special behavior when it's called without `new` and the `this` value is another `Intl.NumberFormat` instance; see Return value.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension key is allowed:

`nu`

See `numberingSystem`.

This key can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object. For ease of reading, the property list is broken into sections based on their purposes, including locale options, style options, digit options, and other options.

Locale options

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

Style options

Depending on the `style` used, some of them may be ignored, and others may be required:

`style`

The formatting style to use.

`"decimal"` (default)

For plain number formatting.

`"currency"`

For currency formatting.

`"percent"`

For percent formatting.

`"unit"`

For unit formatting.

`currency`

The currency to use in currency formatting. Possible values are the ISO 4217 currency codes, such as `"USD"` for the US dollar, `"EUR"` for the euro, or `"CNY"` for the Chinese RMB — see `Intl.supportedValuesOf()`. There is no default value; if the `style` is `"currency"`, the `currency` property must be provided. It is normalized to uppercase.

`currencyDisplay`

How to display the currency in currency formatting.

`"code"`

Use the ISO currency code.

`"symbol"` (default)

Use a localized currency symbol such as €.

`"narrowSymbol"`

Use a narrow format symbol ("$100" rather than "US$100").

`"name"`

Use a localized currency name such as `"dollar"`.

`currencySign`

In many locales, accounting format means to wrap the number with parentheses instead of appending a minus sign. Possible values are `"standard"` and `"accounting"`; the default is `"standard"`.

`unit`

The unit to use in `unit` formatting. Possible values are listed in `Intl.supportedValuesOf()`. Pairs of simple units can be concatenated with "-per-" to make a compound unit. There is no default value; if the `style` is `"unit"`, the `unit` property must be provided.

`unitDisplay`

The unit formatting style to use in `unit` formatting. Possible values are:

`"short"` (default)

E.g., `16 l`.

`"narrow"`

E.g., `16l`.

`"long"`

E.g., `16 litres`.

Digit options

The following properties are also supported by `Intl.PluralRules`.

`minimumIntegerDigits`

The minimum number of integer digits to use. A value with a smaller number of integer digits than this number will be left-padded with zeros (to the specified length) when formatted. Possible values are from `1` to `21`; the default is `1`.

`minimumFractionDigits`

The minimum number of fraction digits to use. Possible values are from `0` to `100`; the default for plain number and percent formatting is `0`; the default for currency formatting is the number of minor unit digits provided by the ISO 4217 currency code list (2 if the list doesn't provide that information). See SignificantDigits/FractionDigits default values for when this default gets applied.

`maximumFractionDigits`

The maximum number of fraction digits to use. Possible values are from `0` to `100`; the default for plain number formatting is the larger of `minimumFractionDigits` and `3`; the default for currency formatting is the larger of `minimumFractionDigits` and the number of minor unit digits provided by the ISO 4217 currency code list (2 if the list doesn't provide that information); the default for percent formatting is the larger of `minimumFractionDigits` and 0. See SignificantDigits/FractionDigits default values for when this default gets applied.

`minimumSignificantDigits`

The minimum number of significant digits to use. Possible values are from `1` to `21`; the default is `1`. See SignificantDigits/FractionDigits default values for when this default gets applied.

`maximumSignificantDigits`

The maximum number of significant digits to use. Possible values are from `1` to `21`; the default is `21`. See SignificantDigits/FractionDigits default values for when this default gets applied.

`roundingPriority`

Specify how rounding conflicts will be resolved if both "FractionDigits" (`minimumFractionDigits`/`maximumFractionDigits`) and "SignificantDigits" (`minimumSignificantDigits`/`maximumSignificantDigits`) are specified. Possible values are:

`"auto"` (default)

The result from the significant digits property is used.

`"morePrecision"`

The result from the property that results in more precision is used.

`"lessPrecision"`

The result from the property that results in less precision is used.

The value `"auto"` is normalized to `"morePrecision"` if `notation` is `"compact"` and none of the four "FractionDigits"/"SignificantDigits" options are set.

Note that for values other than `auto` the result with more precision is calculated from the `maximumSignificantDigits` and `maximumFractionDigits` (minimum fractional and significant digit settings are ignored).

`roundingIncrement`

Indicates the increment at which rounding should take place relative to the calculated rounding magnitude. Possible values are `1`, `2`, `5`, `10`, `20`, `25`, `50`, `100`, `200`, `250`, `500`, `1000`, `2000`, `2500`, and `5000`; the default is `1`. It cannot be mixed with significant-digits rounding or any setting of `roundingPriority` other than `auto`.

`roundingMode`

How decimals should be rounded. Possible values are:

`"ceil"`

Round toward +∞. Positive values round up. Negative values round "more positive".

`"floor"`

Round toward -∞. Positive values round down. Negative values round "more negative".

`"expand"`

Round away from 0. The *magnitude* of the value is always increased by rounding. Positive values round up. Negative values round "more negative".

`"trunc"`

Round toward 0. This *magnitude* of the value is always reduced by rounding. Positive values round down. Negative values round "less negative".

`"halfCeil"`

Ties toward +∞. Values above the half-increment round like `"ceil"` (towards +∞), and below like `"floor"` (towards -∞). On the half-increment, values round like `"ceil"`.

`"halfFloor"`

Ties toward -∞. Values above the half-increment round like `"ceil"` (towards +∞), and below like `"floor"` (towards -∞). On the half-increment, values round like `"floor"`.

`"halfExpand"` (default)

Ties away from 0. Values above the half-increment round like `"expand"` (away from zero), and below like `"trunc"` (towards 0). On the half-increment, values round like `"expand"`.

`"halfTrunc"`

Ties toward 0. Values above the half-increment round like `"expand"` (away from zero), and below like `"trunc"` (towards 0). On the half-increment, values round like `"trunc"`.

`"halfEven"`

Ties towards the nearest even integer. Values above the half-increment round like `"expand"` (away from zero), and below like `"trunc"` (towards 0). On the half-increment values round towards the nearest even digit.

These options reflect the ICU user guide, where "expand" and "trunc" map to ICU "UP" and "DOWN", respectively. The rounding modes example below demonstrates how each mode works.

`trailingZeroDisplay`

The strategy for displaying trailing zeros on whole numbers. Possible values are:

`"auto"` (default)

Keep trailing zeros according to `minimumFractionDigits` and `minimumSignificantDigits`.

`"stripIfInteger"`

Remove the fraction digits *if* they are all zero. This is the same as `"auto"` if any of the fraction digits is non-zero.

SignificantDigits/FractionDigits default values

For the four options above (the `FractionDigits` and `SignificantDigits` options), we mentioned their defaults; however, these defaults are *not unconditionally applied*. They are only applied when the property is actually going to be used, which depends on the `roundingPriority` and `notation` settings. Specifically:

- If `roundingPriority` is not `"auto"`, then all four options apply.
- If `roundingPriority` is `"auto"` and at least one `SignificantDigits` option is set, then the `SignificantDigits` options apply and the `FractionDigits` options are ignored.
- If `roundingPriority` is `"auto"`, and either at least one `FractionDigits` option is set or `notation` is not `"compact"`, then the `FractionDigits` options apply and the `SignificantDigits` options are ignored.
- If `roundingPriority` is `"auto"`, `notation` is `"compact"`, and none of the four options are set, then they are set to `{ minimumFractionDigits: 0, maximumFractionDigits: 0, minimumSignificantDigits: 1, maximumSignificantDigits: 2 }`, regardless of the defaults mentioned above, and `roundingPriority` is set to `"morePrecision"`.

Other options

`notation`

The formatting that should be displayed for the number. Possible values are:

`"standard"` (default)

Plain number formatting.

`"scientific"`

Return the order-of-magnitude for formatted number.

`"engineering"`

Return the exponent of ten when divisible by three.

`"compact"`

String representing exponent; defaults to using the "short" form.

`compactDisplay`

Only used when `notation` is `"compact"`. Possible values are `"short"` and `"long"`; the default is `"short"`.

`useGrouping`

Whether to use grouping separators, such as thousands separators or thousand/lakh/crore separators.

`"always"`

Display grouping separators even if the locale prefers otherwise.

`"auto"`

Display grouping separators based on the locale preference, which may also be dependent on the currency.

`"min2"`

Display grouping separators when there are at least 2 digits in a group.

`true`

Same as `"always"`.

`false`

Display no grouping separators.

The default is `"min2"` if `notation` is `"compact"`, and `"auto"` otherwise. The string values `"true"` and `"false"` are accepted, but are always converted to the default value.

`signDisplay`

When to display the sign for the number. Possible values are:

`"auto"` (default)

Sign display for negative numbers only, including negative zero.

`"always"`

Always display sign.

`"exceptZero"`

Sign display for positive and negative numbers, but not zero.

`"negative"`

Sign display for negative numbers only, excluding negative zero.

`"never"`

Never display sign.

Return value

A new `Intl.NumberFormat` object.

> **Note:** The text below describes behavior that is marked by the specification as "optional". It may not work in all environments. Check the browser compatibility table.

Normally, `Intl.NumberFormat()` can be called with or without `new`, and a new `Intl.NumberFormat` instance is returned in both cases. However, if the `this` value is an object that is `instanceof` `Intl.NumberFormat` (doesn't necessarily mean it's created via `new Intl.NumberFormat`; just that it has `Intl.NumberFormat.prototype` in its prototype chain), then the value of `this` is returned instead, with the newly created `Intl.NumberFormat` object hidden in a `[Symbol(IntlLegacyConstructedSymbol)]` property (a unique symbol that's reused between instances).

```
const formatter = Intl.NumberFormat.call(
  { __proto__: Intl.NumberFormat.prototype },
  "en-US",
  { notation: "scientific" },
);
console.log(Object.getOwnPropertyDescriptors(formatter));
// {
//   [Symbol(IntlLegacyConstructedSymbol)]: {
//     value: NumberFormat [Intl.NumberFormat] {},
//     writable: false,
//     enumerable: false,
//     configurable: false
//   }
// }
```

Note that there's only one actual `Intl.NumberFormat` instance here: the one hidden in `[Symbol(IntlLegacyConstructedSymbol)]`. Calling the `format()` and `resolvedOptions()` methods on `formatter` would correctly use the options stored in that instance, but calling all other methods (e.g., `formatRange()`) would fail with "TypeError: formatRange method called on incompatible Object", because those methods don't consult the hidden instance's options.

This behavior, called `ChainNumberFormat`, does not happen when `Intl.NumberFormat()` is called without `new` but with `this` set to anything else that's not an `instanceof` `Intl.NumberFormat`. If you call it directly as `Intl.NumberFormat()`, the `this` value is `Intl`, and a new `Intl.NumberFormat` instance is created normally.

Exceptions

`RangeError`

Thrown in one of the following cases:

- A property that takes enumerated values (such as `style`, `units`, `currency`, and so on) is set to an invalid value.
- Both `maximumFractionDigits` and `minimumFractionDigits` are set, and they are set to different values. Note that depending on various formatting options, these properties can have default values. It is therefore possible to get this error even if you only set one of the properties.

`TypeError`

Thrown if the `options.style` property is set to "unit" or "currency", and no value has been set for the corresponding property `options.unit` or `options.currency`.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

```
const amount = 3500;

console.log(new Intl.NumberFormat().format(amount));
// '3,500' if in US English locale
```

Decimal and percent formatting

```
const amount = 3500;

new Intl.NumberFormat("en-US", {
  style: "decimal",
}).format(amount); // '3,500'
new Intl.NumberFormat("en-US", {
  style: "percent",
}).format(amount); // '350,000%'
```

Unit formatting

If the `style` is `'unit'`, a `unit` property must be provided. Optionally, `unitDisplay` controls the unit formatting.

```
const amount = 3500;

new Intl.NumberFormat("en-US", {
  style: "unit",
  unit: "liter",
}).format(amount); // '3,500 L'

new Intl.NumberFormat("en-US", {
  style: "unit",
  unit: "liter",
  unitDisplay: "long",
}).format(amount); // '3,500 liters'
```

Currency formatting

If the `style` is `'currency'`, a `currency` property must be provided. Optionally, `currencyDisplay` and `currencySign` control the unit formatting.

```
const amount = -3500;
new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
}).format(amount); // '-$3,500.00'

new Intl.NumberFormat("bn", {
  style: "currency",
  currency: "USD",
  currencyDisplay: "name",
}).format(amount); // '-3,500.00 US dollars'

new Intl.NumberFormat("bn", {
  style: "currency",
  currency: "USD",
  currencySign: "accounting",
}).format(amount); // '($3,500.00)'
```

Scientific, engineering or compact notations

Scientific and compact notation are represented by the `notation` option and can be formatted like this:

```
new Intl.NumberFormat("en-US", {
  notation: "scientific",
}).format(987654321);
// 9.877E8

new Intl.NumberFormat("pt-PT", {
  notation: "scientific",
}).format(987654321);
// 9,877E8

new Intl.NumberFormat("en-GB", {
  notation: "engineering",
}).format(987654321);
// 987.654E6

new Intl.NumberFormat("de", {
  notation: "engineering",
}).format(987654321);
// 987,654E6

new Intl.NumberFormat("zh-CN", {
  notation: "compact",
}).format(987654321);
// 9.9亿
```

```
new Intl.NumberFormat("fr", {
  notation: "compact",
  compactDisplay: "long",
}).format(987654321);
// 988 millions

new Intl.NumberFormat("en-GB", {
  notation: "compact",
  compactDisplay: "short",
}).format(987654321);
// 988M
```

Displaying signs

Display a sign for positive and negative numbers, but not zero:

```
new Intl.NumberFormat("en-US", {
  style: "percent",
  signDisplay: "exceptZero",
}).format(0.55);
// '+55%'
```

Note that when the currency sign is "accounting", parentheses might be used instead of a minus sign:

```
new Intl.NumberFormat("bn", {
  style: "currency",
  currency: "USD",
  currencySign: "accounting",
  signDisplay: "always",
}).format(-3500);
// '($3,500.00)'
```

FractionDigits, SignificantDigits and IntegerDigits

You can specify the minimum or maximum number of fractional, integer or significant digits to display when formatting a number.

> **Note:** If both significant and fractional digit limits are specified, then the actual formatting depends on the `roundingPriority`.

Using FractionDigits and IntegerDigits

The integer and fraction digit properties indicate the number of digits to display before and after the decimal point, respectively. If the value to display has fewer integer digits than specified, it will be left-padded with zeros to the expected number. If it has fewer fractional digits, it will be right-padded with zeros. Both cases are shown below:

```
// Formatting adds zeros to display minimum integers and fractions
console.log(
  new Intl.NumberFormat("en", {
    minimumIntegerDigits: 3,
    minimumFractionDigits: 4,
  }).format(4.33),
);
// "004.3300"
```

If a value has more fractional digits than the specified maximum number, it will be rounded. The *way* that it is rounded depends on the `roundingMode` property (more details are provided in the rounding modes section). Below the value is rounded from five fractional digits (`4.33145`) to two (`4.33`):

```
// Display value shortened to maximum number of digits
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 2,
  }).format(4.33145),
);
// "4.33"
```

The minimum fractional digits have no effect if the value already has more than 2 fractional digits:

```
// Minimum fractions have no effect if value is higher precision.
console.log(
  new Intl.NumberFormat("en", {
    minimumFractionDigits: 2,
  }).format(4.33145),
);
// "4.331"
```

> **Warning:** Watch out for default values as they may affect formatting even if not specified in your code. The default maximum digit value is `3` for plain values, `2` for currency, and may have different values for other predefined types.

The formatted value above is rounded to 3 digits, even though we didn't specify the maximum digits! This is because a default value of `maximumFractionDigits` is set when we specify `minimumFractionDigits`, and visa versa. The default values of `maximumFractionDigits` and `minimumFractionDigits` are `3` and `0`, respectively.

You can use `resolvedOptions()` to inspect the formatter.

```
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumFractionDigits: 0,
//   maximumFractionDigits: 2,
//   …
// }

console.log(
  new Intl.NumberFormat("en", {
    minimumFractionDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumFractionDigits: 2,
//   maximumFractionDigits: 3,
//   …
// }
```

Using SignificantDigits

The number of *significant digits* is the total number of digits including both integer and fractional parts. The `maximumSignificantDigits` is used to indicate the total number of digits from the original value to display.

The examples below show how this works. Note in particular the last case: only the first digit is retained and the others are discarded/set to zero.

```
// Display 5 significant digits
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 5,
  }).format(54.33145),
);
// "54.331"

// Max 2 significant digits
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(54.33145),
);
// "54"

// Max 1 significant digits
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 1,
  }).format(54.33145),
);
// "50"
```

The `minimumSignificantDigits` ensures that at least the specified number of digits are displayed, adding zeros to the end of the value if needed.

```
// Minimum 10 significant digits
console.log(
  new Intl.NumberFormat("en", {
    minimumSignificantDigits: 10,
  }).format(54.33145),
);
// "54.33145000"
```

> **Warning:** Watch out for default values as they may affect formatting. If only one `SignificantDigits` property is used, then its counterpart will automatically be applied with the default value. The default maximum and minimum significant digit values are 21 and 1, respectively.

Specifying significant and fractional digits at the same time

The fraction digits (`minimumFractionDigits`/`maximumFractionDigits`) and significant digits (`minimumSignificantDigits`/`maximumSignificantDigits`) are both ways of controlling how many fractional and leading digits should be formatted. If both are used at the same time, it is possible for them to conflict.

These conflicts are resolved using the `roundingPriority` property. By default, this has a value of `"auto"`, which means that if either `minimumSignificantDigits` or `maximumSignificantDigits` is specified, the fractional and integer digit properties will be ignored.

For example, the code below formats the value of `4.33145` with `maximumFractionDigits: 3`, and then `maximumSignificantDigits: 2`, and then both. The value with both is the one set with `maximumSignificantDigits`.

```
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
  }).format(4.33145),
);
// "4.331"
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(4.33145),
);
// "4.3"
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
    maximumSignificantDigits: 2,
  }).format(4.33145),
);
// "4.3"
```

Using `resolvedOptions()` to inspect the formatter, we can see that the returned object does not include `maximumFractionDigits` when `maximumSignificantDigits` or `minimumSignificantDigits` are specified.

```
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
    maximumSignificantDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumSignificantDigits: 1,
//   maximumSignificantDigits: 2,
//   …
// }
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
    minimumSignificantDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumSignificantDigits: 2,
//   maximumSignificantDigits: 21,
//   …
// }
```

In addition to `"auto"`, you can resolve conflicts by specifying `roundingPriority` as `"morePrecision"` or `"lessPrecision"`. The formatter calculates the precision using the values of `maximumSignificantDigits` and `maximumFractionDigits`.

The code below shows the format being selected for the three different rounding priorities:

```
const maxFracNF = new Intl.NumberFormat("en", {
  maximumFractionDigits: 3,
});
console.log(`maximumFractionDigits:3 - ${maxFracNF.format(1.23456)}`);
// "maximumFractionDigits:2 - 1.235"

const maxSigNS = new Intl.NumberFormat("en", {
  maximumSignificantDigits: 3,
});
console.log(`maximumSignificantDigits:3 - ${maxSigNS.format(1.23456)}`);
// "maximumSignificantDigits:3 - 1.23"

const bothAuto = new Intl.NumberFormat("en", {
  maximumSignificantDigits: 3,
  maximumFractionDigits: 3,
});
console.log(`auto - ${bothAuto.format(1.23456)}`);
// "auto - 1.23"

const bothLess = new Intl.NumberFormat("en", {
  roundingPriority: "lessPrecision",
  maximumSignificantDigits: 3,
  maximumFractionDigits: 3,
});
console.log(`lessPrecision - ${bothLess.format(1.23456)}`);
// "lessPrecision - 1.23"
```

```
const bothMore = new Intl.NumberFormat("en", {
  roundingPriority: "morePrecision",
  maximumSignificantDigits: 3,
  maximumFractionDigits: 3,
});
console.log(`morePrecision - ${bothMore.format(1.23456)}`);
// "morePrecision - 1.235"
```

Note that the algorithm can behave in an unintuitive way if a minimum value is specified without a maximum value. The example below formats the value `1` specifying `minimumFractionDigits: 2` (formatting to `1.00`) and `minimumSignificantDigits: 2` (formatting to `1.0`). Since `1.00` has more digits than `1.0`, this should be the result when prioritizing `morePrecision`, but in fact the opposite is true:

```
const bothLess = new Intl.NumberFormat("en", {
  roundingPriority: "lessPrecision",
  minimumFractionDigits: 2,
  minimumSignificantDigits: 2,
});
console.log(`lessPrecision - ${bothLess.format(1)}`);
// "lessPrecision - 1.00"

const bothMore = new Intl.NumberFormat("en", {
  roundingPriority: "morePrecision",
  minimumFractionDigits: 2,
  minimumSignificantDigits: 2,
});
console.log(`morePrecision - ${bothMore.format(1)}`);
// "morePrecision - 1.0"
```

The reason for this is that only the "maximum precision" values are used for the calculation, and the default value of `maximumSignificantDigits` is much higher than `maximumFractionDigits`.

> **Note:** The working group have proposed a modification of the algorithm where the formatter should evaluate the result of using the specified fractional and significant digits independently (taking account of both minimum and maximum values). It will then select the option that displays more fractional digits if `morePrecision` is set, and fewer if `lessPrecision` is set. This will result in more intuitive behavior for this case.

Rounding modes

If a value has more fractional digits than allowed by the constructor options, the formatted value will be *rounded* to the specified number of fractional digits. The *way* in which the value is rounded depends on the `roundingMode` property.

Number formatters use `halfExpand` rounding by default, which rounds values "away from zero" at the half-increment (in other words, the *magnitude* of the value is rounded up).

For a positive number, if the fractional digits to be removed are closer to the next increment (or on the half way point) then the remaining fractional digits will be rounded up, otherwise they are rounded down. This is shown below: 2.23 rounded to two significant digits is truncated to 2.2 because 2.23 is less than the half increment 2.25, while values of 2.25 and greater are rounded up to 2.3:

```
// Value below half-increment: round down.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(2.23),
);
// "2.2"

// Value on or above half-increment: round up.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(2.25),
);
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(2.28),
);
// "2.3"
// "2.3"
```

A negative number on or below the half-increment point is also rounded away from zero (becomes more negative):

```
// Value below half-increment: round down.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(-2.23),
);
// "-2.2"

// Value on or above half-increment: round up.
console.log(
```

```
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(-2.25),
);
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(-2.28),
);
// "-2.3"
// "-2.3"
```

The table below shows the effect of different rounding modes for positive and negative values that are on and around the half-increment.

rounding mode	2.23	2.25	2.28	-2.23	-2.25	-2.28
`ceil`	2.3	2.3	2.3	-2.2	-2.2	-2.2
`floor`	2.2	2.2	2.2	-2.3	-2.3	-2.3
`expand`	2.3	2.3	2.3	-2.3	-2.3	-2.3
`trunc`	2.2	2.2	2.2	-2.2	-2.2	-2.2
`halfCeil`	2.2	2.3	2.3	-2.2	-2.2	-2.3
`halfFloor`	2.2	2.2	2.3	-2.2	-2.3	-2.3
`halfExpand`	2.2	2.3	2.3	-2.2	-2.3	-2.3
`halfTrunc`	2.2	2.2	2.3	-2.2	-2.2	-2.3
`halfEven`	2.2	2.2	2.3	-2.2	-2.2	-2.3

When using `halfEven`, its behavior also depends on the parity (odd or even) of the last digit of the rounded number. For example, the behavior of `halfEven` in the table above is the same as `halfTrunc`, because the magnitudes of all numbers are between a smaller "even" number (2.2) and a larger "odd" number (2.3). If the numbers are between ±2.3 and ±2.4, `halfEven` will behave like `halfExpand` instead. This behavior avoids consistently under- or over-estimating half-increments in a large data sample.

Using roundingIncrement

Sometimes we want to round the remaining fractional digits to some other increment than the next integer. For example, currencies for which the smallest coin is 5 cents might want to round the value to increments of 5, reflecting amounts that can actually be paid in cash.

This kind of rounding can be achieved with the `roundingIncrement` property.

For example, if `maximumFractionDigits` is 2 and `roundingIncrement` is 5, then the number is rounded to the nearest 0.05:

```
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 5,
});

console.log(nf.format(11.29)); // "$11.30"
console.log(nf.format(11.25)); // "$11.25"
console.log(nf.format(11.22)); // "$11.20"
```

This particular pattern is referred to as "nickel rounding", where nickel is the colloquial name for a USA 5 cent coin. To round to the nearest 10 cents ("dime rounding"), you could change `roundingIncrement` to `10`.

```
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 10,
});

console.log(nf.format(11.29)); // "$11.30"
console.log(nf.format(11.25)); // "$11.30"
console.log(nf.format(11.22)); // "$11.20"
```

You can also use `roundingMode` to change the rounding algorithm. The example below shows how `halfCeil` rounding can be used to round the value "less positive" below the half-rounding increment and "more positive" if above or on the half-increment. The incremented digit is "0.05" so the half-increment is at .025 (below, this is shown at 11.225).

```
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 5,
  roundingMode: "halfCeil",
});

console.log(nf.format(11.21)); // "$11.20"
console.log(nf.format(11.22)); // "$11.20"
```

```
console.log(nf.format(11.224)); // "$11.20"
console.log(nf.format(11.225)); // "$11.25"
console.log(nf.format(11.23)); // "$11.25"
```

If you need to change the number of digits, remember that `minimumFractionDigits` and `maximumFractionDigits` must both be set to the same value, or a `RangeError` is thrown.

`roundingIncrement` cannot be mixed with significant-digits rounding or any setting of `roundingPriority` other than `auto`.

See also

- `Intl.NumberFormat`
- `Intl.supportedValuesOf()`
- `Intl`

Intl.NumberFormat.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.NumberFormat` instances returns a new object with properties reflecting the options computed during initialization of this `NumberFormat` object.

Try it

```
const numberFormat = new Intl.NumberFormat("de-DE");
const options = numberFormat.resolvedOptions();

console.log(options.locale);
// Expected output: "de-DE"

console.log(options.numberingSystem);
// Expected output: "latn"

console.log(options.style);
// Expected output: "decimal"
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `NumberFormat` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `nu` Unicode extension key, if requested, may be included in the output.

`numberingSystem`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

`style`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"decimal"`, `"percent"`, `"currency"`, or `"unit"`. The default is `"decimal"`.

`currency` Optional

> The value provided for this property in the `options` argument. It is only present if `style` is `"currency"`. It is an ISO 4217 currency code; see `Intl.supportedValuesOf()`. It is required if `style` is `"currency"` so there is no default.

`currencyDisplay` Optional

> The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `style` is `"currency"`. It is either `"code"`, `"symbol"`, `"narrowSymbol"`, or `"name"`. The default is `"symbol"`.

`currencySign` Optional

> The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `style` is `"currency"`. It is either `"standard"` or `"accounting"`. The default is `"standard"`.

`unit` Optional

> The value provided for this property in the `options` argument. It is only present if `style` is `"unit"`. It is a sanctioned unit identifier from the full CLDR list. It is required if `style` is `"unit"` so there is no default.

`unitDisplay` Optional

The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `style` is `"unit"`. It is either `"short"`, `"narrow"`, or `"long"`. The default is `"short"`.

`minimumIntegerDigits`

The value provided for this property in the `options` argument, with default filled in as needed. It is an integer between `1` and `21`. The default is `1`.

`minimumFractionDigits`, `maximumFractionDigits` Optional

The value provided for these properties in the `options` argument, with defaults filled in as needed. They are only present if necessary; see digit options. It is an integer between `0` and `100`.

`minimumSignificantDigits`, `maximumSignificantDigits` Optional

The value provided for these properties in the `options` argument, with defaults filled in as needed. They are only present if necessary; see digit options. It is an integer between `1` and `21`.

`useGrouping`

The value provided for this property in the `options` argument, with default filled in as needed, and with some values normalized. It is either `"always"`, `"auto"`, `"min2"`, or the boolean `false`. The default is `"min2"` if `notation` is `"compact"`, and `"auto"` otherwise.

`notation`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"standard"`, `"scientific"`, `"engineering"`, or `"compact"`. The default is `"standard"`.

`compactDisplay` Optional

The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `notation` is `"compact"`. It is either `"short"` or `"long"`. The default is `"short"`.

`signDisplay`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"`, `"always"`, `"exceptZero"`, `"negative"`, or `"never"`. The default is `"auto"`.

`roundingIncrement`

The value provided for this property in the `options` argument, with default filled in as needed. It is one of `1`, `2`, `5`, `10`, `20`, `25`, `50`, `100`, `200`, `250`, `500`, `1000`, `2000`, `2500`, and `5000`. The default is `1`.

`roundingMode`

The value provided for this property in the `options` argument, with default filled in as needed. It is one of `"ceil"`, `"floor"`, `"expand"`, `"trunc"`, `"halfCeil"`, `"halfFloor"`, `"halfExpand"`, `"halfTrunc"`, and `"halfEven"`. The default is `"halfExpand"`.

`roundingPriority`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"`, `"morePrecision"`, or `"lessPrecision"`. The default is `"auto"`.

`trailingZeroDisplay`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"` or `"stripIfInteger"`. The default is `"auto"`.

Examples

Using the <code>resolvedOptions</code> method

```
// Create a NumberFormat
const de = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 5,
  roundingMode: "halfCeil",
});

// Resolve the options
const usedOptions = de.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.numberingSystem); // "latn"
console.log(usedOptions.compactDisplay); // undefined ("notation" not set to "compact")
console.log(usedOptions.currency); // "USD"
console.log(usedOptions.currencyDisplay); // "symbol"
console.log(usedOptions.currencySign); // "standard"
console.log(usedOptions.minimumIntegerDigits); // 1
console.log(usedOptions.minimumFractionDigits); // 2
console.log(usedOptions.maximumFractionDigits); // 2
console.log(usedOptions.minimumSignificantDigits); // undefined (maximumFractionDigits is set)
console.log(usedOptions.maximumSignificantDigits); // undefined (maximumFractionDigits is set)
console.log(usedOptions.notation); // "standard"
console.log(usedOptions.roundingIncrement); // 5
console.log(usedOptions.roundingMode); // halfCeil
```

```
console.log(usedOptions.roundingPriority); // auto
console.log(usedOptions.signDisplay); // "auto"
console.log(usedOptions.style); // "currency"
console.log(usedOptions.trailingZeroDisplay); // auto
console.log(usedOptions.useGrouping); // auto
```

See also

- `Intl.NumberFormat`

Intl.NumberFormat.supportedLocalesOf()

The **`Intl.NumberFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in number formatting without having to fall back to the runtime's default locale.

Try it

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.NumberFormat.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

```
Intl.NumberFormat.supportedLocalesOf(locales)
Intl.NumberFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

> A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object that may have the following property:
>
> `localeMatcher`
>
> > The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in number formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in number formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to number formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.NumberFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.NumberFormat`

Intl.PluralRules

The **`Intl.PluralRules`** object enables plural-sensitive formatting and plural-related language rules.

Description

Languages use different patterns for expressing both plural numbers of items (cardinal numbers) and for expressing the order of items (ordinal numbers). English has two forms for expressing cardinal numbers: one for the singular "item" (1 hour, 1 dog, 1 fish) and the other for zero or any other number of "items" (0 hours, 2 lemmings, 100000.5 fish), while Chinese has only one form, and Arabic has six! Similarly, English has four forms for expressing ordinal numbers: "th", "st", "nd", "rd", giving the sequence: 0th, 1st, 2nd, 3rd, 4th, 5th, ..., 21st, 22nd, 23rd, 24th, 25th, and so on, while both Chinese and Arabic only have one form for ordinal numbers.

Given a particular language and set of formatting options, the methods `Intl.PluralRules.prototype.select()` and `Intl.PluralRules.prototype.selectRange()` return a *tag* that represents the plural form of a single or a range of numbers, cardinal or ordinal. Code can use the returned tags to represent numbers appropriately for the given language. The full set of tags that might be returned are: `zero`, `one`, `two`, `few`, `many`, and `other` (the "general" plural form, also used if the language only has one form).

As English only has two forms for cardinal numbers, the `select()` method returns only two tags: `"one"` for the singular case, and `"other"` for all other cardinal numbers. This allows construction of sentences that make sense in English for each case, such as: "1 dog is happy; do you want to play with it?" and "10 dogs are happy; do you want to play with them?".

Creating appropriate sentences for each form depends on the language, and even in English may not be as simple as just adding "s" to a noun to make the plural form. Using the example above, we see that the form may affect:

- **Nouns**: 1 dog, 2 dogs (but not "fish" or "sheep", which have the same singular and plural form).
- **Verbs**: 1 dog *is* happy, 2 dogs *are* happy.
- **Pronouns** (and other referents): Do you want to play with *it* / *them*.

Other languages have more forms, and choosing appropriate sentences can be even more complex.

`select()` can return any of four tags for ordinal numbers in English, representing each of the allowed forms: `one` for "st" numbers (1, 21, 31, ...), `two` for "nd" numbers (2, 22, 32, ...), `few` for "rd" numbers (3, 33, 43, ...), and `other` for "th" numbers (0, 4-20, etc.). Again, the returned tags allow appropriate formatting of strings describing an ordinal number.

For more information about the rules and how they are used, see Plural Rules. For a list of the rules and how they apply for different languages, see the LDML Language Plural Rules.

Constructor

`Intl.PluralRules()`

Creates a new `Intl.PluralRules` object.

Static methods

`Intl.PluralRules.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.PluralRules.prototype` and shared by all `Intl.PluralRules` instances.

`Intl.PluralRules.prototype.constructor`

The constructor function that created the instance object. For `Intl.PluralRules` instances, the initial value is the `Intl.PluralRules` constructor.

`Intl.PluralRules.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.PluralRules"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.PluralRules.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

`Intl.PluralRules.prototype.select()`

Returns a string indicating which plural rule to use for locale-aware formatting.

`Intl.PluralRules.prototype.selectRange()`

This method receives two values and returns a string indicating which plural rule to use for locale-aware formatting.

Examples

Using locales

This example shows some of the variations in localized plural rules for cardinal numbers.

In order to get the format for the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the constructor `locales` argument:

```
// US English
const enCardinalRules = new Intl.PluralRules("en-US");
console.log(enCardinalRules.select(0)); // "other"
console.log(enCardinalRules.select(1)); // "one"
console.log(enCardinalRules.select(2)); // "other"
console.log(enCardinalRules.select(3)); // "other"

// Arabic
const arCardinalRules = new Intl.PluralRules("ar-EG");
console.log(arCardinalRules.select(0)); // "zero"
console.log(arCardinalRules.select(1)); // "one"
console.log(arCardinalRules.select(2)); // "two"
console.log(arCardinalRules.select(6)); // "few"
console.log(arCardinalRules.select(18)); // "many"
```

Using options

The plural form of the specified number may also depend on constructor `options`, such as how the number is rounded, and whether it is cardinal or ordinal.

This example shows how you can set the type of rules to "ordinal", and how this affects the form for some numbers in US English.

```
// US English - ordinal
const enOrdinalRules = new Intl.PluralRules("en-US", { type: "ordinal" });
console.log(enOrdinalRules.select(0)); // "other" (0th)
console.log(enOrdinalRules.select(1)); // "one"   (1st)
console.log(enOrdinalRules.select(2)); // "two"   (2nd)
console.log(enOrdinalRules.select(3)); // "few"   (3rd)
console.log(enOrdinalRules.select(4)); // "other" (4th)
console.log(enOrdinalRules.select(21)); // "one"  (21st)
```

Formatting text using the returned tag

The code below extends the previous example, showing how you might use the returned tag for an ordinal number to format text in English.

```
const enOrdinalRules = new Intl.PluralRules("en-US", { type: "ordinal" });

const suffixes = new Map([
  ["one", "st"],
  ["two", "nd"],
  ["few", "rd"],
  ["other", "th"],
]);
const formatOrdinals = (n) => {
  const rule = enOrdinalRules.select(n);
  const suffix = suffixes.get(rule);
  return `${n}${suffix}`;
};

formatOrdinals(0); // '0th'
formatOrdinals(1); // '1st'
formatOrdinals(2); // '2nd'
formatOrdinals(3); // '3rd'
formatOrdinals(4); // '4th'
formatOrdinals(11); // '11th'
formatOrdinals(21); // '21st'
formatOrdinals(42); // '42nd'
formatOrdinals(103); // '103rd'
```

See also

- Polyfill of `Intl.PluralRules` in FormatJS
- `Intl`

Intl.PluralRules() constructor

The **`Intl.PluralRules()`** constructor creates `Intl.PluralRules` objects.

Syntax

```
new Intl.PluralRules()
new Intl.PluralRules(locales)
new Intl.PluralRules(locales, options)
```

> **Note:** `Intl.PluralRules()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

> A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object containing the following properties, in the order they are retrieved (all of them are optional):
>
> `localeMatcher`
>
> > The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.
>
> `type`
>
> > The type to use. Possible values are:
> >
> > `"cardinal"` (default)
> >
> > > For cardinal numbers (referring to the quantity of things).

`"ordinal"`

For ordinal number (referring to the ordering or ranking of things, e.g., "1st", "2nd", "3rd" in English).

`Intl.PluralRules` also supports the `Intl.NumberFormat()` digit options (see `Intl.NumberFormat()` for details):

- `minimumIntegerDigits`
- `minimumFractionDigits`
- `maximumFractionDigits`
- `minimumSignificantDigits`
- `maximumSignificantDigits`
- `roundingPriority`
- `roundingIncrement`
- `roundingMode`

These options are interpreted as if the `notation` option from `Intl.NumberFormat` is `"standard"` and `style` is `"decimal"`.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned. This is useful to distinguish between singular and plural forms, e.g., "dog" and "dogs".

```
const pr = new Intl.PluralRules();

pr.select(0); // 'other' if in US English locale

pr.select(1); // 'one' if in US English locale

pr.select(2); // 'other' if in US English locale
```

Using options

The results can be customized using the `options` argument, which has one property called `type` which you can set to `ordinal`. This is useful to figure out the ordinal indicator, e.g., "1st", "2nd", "3rd", "4th", "42nd", and so forth.

```
const pr = new Intl.PluralRules("en-US", { type: "ordinal" });

const suffixes = new Map([
  ["one", "st"],
  ["two", "nd"],
  ["few", "rd"],
  ["other", "th"],
]);
const formatOrdinals = (n) => {
  const rule = pr.select(n);
  const suffix = suffixes.get(rule);
  return `${n}${suffix}`;
};

formatOrdinals(0); // '0th'
formatOrdinals(1); // '1st'
formatOrdinals(2); // '2nd'
formatOrdinals(3); // '3rd'
formatOrdinals(4); // '4th'
formatOrdinals(11); // '11th'
formatOrdinals(21); // '21st'
formatOrdinals(42); // '42nd'
formatOrdinals(103); // '103rd'
```

See also

- `Intl.PluralRules`
- `Intl`

Intl.PluralRules.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.PluralRules` instances returns a new object with properties reflecting the options computed during initialization of this `PluralRules` object.

Try it

```
const pluralRules1 = new Intl.PluralRules("uk");
const options1 = pluralRules1.resolvedOptions();

const pluralRules2 = new Intl.PluralRules("bn");
```

```
const options2 = pluralRules2.resolvedOptions();

console.log(options1.pluralCategories);
// Expected output: Array ["few", "many", "one", "other"]

console.log(options2.pluralCategories);
// Expected output: Array ["one", "other"]
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `PluralRules` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`type`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"cardinal"` or `"ordinal"`. The default is `"cardinal"`.

`minimumIntegerDigits`, `minimumFractionDigits`, `maximumFractionDigits` Optional

> The value provided for these properties in the `options` argument, with defaults filled in as needed. These properties are present only if neither `minimumSignificantDigits` nor `maximumSignificantDigits` was provided in the `options` argument.

`minimumSignificantDigits`, `maximumSignificantDigits` Optional

> The value provided for these properties in the `options` argument, with defaults filled in as needed. These properties are present only if at least one of them was provided in the `options` argument.

`pluralCategories`

> An `Array` of plural categories used by the given locale, selected from the list `"zero"`, `"one"`, `"two"`, `"few"`, `"many"` and `"other"`.

`roundingIncrement`

> The value provided for this property in the `options` argument, with default filled in as needed. It is one of `1`, `2`, `5`, `10`, `20`, `25`, `50`, `100`, `200`, `250`, `500`, `1000`, `2000`, `2500`, and `5000`. The default is `1`.

`roundingMode`

> The value provided for this property in the `options` argument, with default filled in as needed. It is one of `"ceil"`, `"floor"`, `"expand"`, `"trunc"`, `"halfCeil"`, `"halfFloor"`, `"halfExpand"`, `"halfTrunc"`, and `"halfEven"`. The default is `"halfExpand"`.

`roundingPriority`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"`, `"morePrecision"`, or `"lessPrecision"`. The default is `"auto"`.

`trailingZeroDisplay`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"` or `"stripIfInteger"`. The default is `"auto"`.

Examples

Using the resolvedOptions() method

The code below shows the construction of a `PluralRules` object, followed by logging of each of the resolved options.

```
// Create a PluralRules instance
const de = new Intl.PluralRules("de-DE", {
  maximumSignificantDigits: 2,
  trailingZeroDisplay: "auto",
});

// Resolve the options
const usedOptions = de.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.pluralCategories); // Array ["one", "other"]
console.log(usedOptions.type); // "cardinal"
console.log(usedOptions.minimumIntegerDigits); // 1
console.log(usedOptions.minimumFractionDigits); // undefined (maximumSignificantDigits is set)
console.log(usedOptions.maximumFractionDigits); // undefined (maximumSignificantDigits is set)
console.log(usedOptions.minimumSignificantDigits); // 1
console.log(usedOptions.maximumSignificantDigits); // 2
console.log(usedOptions.roundingIncrement); // 1
```

```
console.log(usedOptions.roundingMode); // "halfExpand"
console.log(usedOptions.roundingPriority); // "auto"
console.log(usedOptions.trailingZeroDisplay); // "auto"
```

See also

- `Intl.PluralRules`

Intl.PluralRules.prototype.select()

The **`select()`** method of `Intl.PluralRules` instances returns a string indicating which plural rule to use for locale-aware formatting of a number.

Try it

```
console.log(new Intl.PluralRules("ar-EG").select(0));
// Expected output: "zero"

console.log(new Intl.PluralRules("ar-EG").select(5));
// Expected output: "few"

console.log(new Intl.PluralRules("ar-EG").select(55));
// Expected output: "many"

console.log(new Intl.PluralRules("en").select(0));
// Expected output: "other"
```

Syntax

```
select(number)
```

Parameters

`number`

The number to get a plural rule for.

Return value

A string representing the pluralization category of the `number`. This can be one of `zero`, `one`, `two`, `few`, `many`, or `other`.

Description

This function selects a pluralization category according to the locale and formatting options of an `Intl.PluralRules` object. These options are set in the `Intl.PluralRules()` constructor.

Examples

Using select()

First, create an `Intl.PluralRules` object, passing the appropriate `locales` and `options` parameters. Here we create a plural rules object for Arabic in the Egyptian dialect. Because the `type` is not specified the rules object will provide formatting for cardinal numbers (the default).

```
const pr = new Intl.PluralRules("ar-EG");
```

Then call `select()` on the rules object, specifying the number for which the plural form is required. Note that Arabic has 5 forms for cardinal numbers, as shown.

```
pr.select(0); // 'zero'
pr.select(1); // 'one'
pr.select(2); // 'two'
pr.select(6); // 'few'
pr.select(18); // 'many'
```

See also

- `Intl.PluralRules`

Intl.PluralRules.prototype.selectRange()

The **`selectRange()`** method of `Intl.PluralRules` instances receives two values and returns a string indicating which plural rule to use for locale-aware formatting of the indicated range.

Syntax

```
selectRange(startRange, endRange)
```

Parameters

`startRange`

A number representing the start of the range.

`endRange`

A number representing the end of the range.

Return value

A string representing the pluralization category of the specified range. This can be one of `zero`, `one`, `two`, `few`, `many` or `other`, that are relevant for the locale whose localization is specified in LDML Language Plural Rules.

Description

This function selects a pluralization category according to the locale and formatting options of an `Intl.PluralRules` object.

Conceptually the behavior is the same as getting plural rules for a single cardinal or ordinal number. Languages have one or more forms for describing ranges, and this method returns the appropriate form given the supplied locale and formatting options. In English there is only one plural form, such as "1–10 apples", and the method will return `other`. Other languages can have many forms.

Examples

Using selectRange()

```
new Intl.PluralRules("sl").selectRange(102, 201); // 'few'

new Intl.PluralRules("pt").selectRange(102, 102); // 'other'
```

See also

- `Intl.PluralRules`

Intl.PluralRules.supportedLocalesOf()

The **`Intl.PluralRules.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in plural rules without having to fall back to the runtime's default locale.

Try it

```
const locales = ["en-US", "ban", "ar-OM", "de-DE"];
const options = { localeMatcher: "lookup" };

console.log(Intl.PluralRules.supportedLocalesOf(locales, options));
// Expected output: Array ["en-US", "ar-OM", "de-DE"]
```

Syntax

```
Intl.PluralRules.supportedLocalesOf(locales)
Intl.PluralRules.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in plural rules without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in plural rules, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to plural rules nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.PluralRules.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.PluralRules`

Intl.RelativeTimeFormat

The **`Intl.RelativeTimeFormat`** object enables language-sensitive relative time formatting.

Try it

```
const rtf1 = new Intl.RelativeTimeFormat("en", { style: "short" });

console.log(rtf1.format(3, "quarter"));
// Expected output: "in 3 qtrs."

console.log(rtf1.format(-1, "day"));
// Expected output: "1 day ago"

const rtf2 = new Intl.RelativeTimeFormat("es", { numeric: "auto" });

console.log(rtf2.format(2, "day"));
// Expected output: "pasado mañana"
```

Constructor

`Intl.RelativeTimeFormat()`

Creates a new `Intl.RelativeTimeFormat` object.

Static methods

`Intl.RelativeTimeFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.RelativeTimeFormat.prototype` and shared by all `Intl.RelativeTimeFormat` instances.

`Intl.RelativeTimeFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.RelativeTimeFormat` instances, the initial value is the `Intl.RelativeTimeFormat` constructor.

`Intl.RelativeTimeFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.RelativeTimeFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.RelativeTimeFormat.prototype.format()`

Formats a `value` and a `unit` according to the locale and formatting options of the given `Intl.RelativeTimeFormat` object.

`Intl.RelativeTimeFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the relative time format in parts that can be used for custom locale-aware formatting.

`Intl.RelativeTimeFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Basic format usage

The following example shows how to use a relative time formatter for the English language.

```
// Create a relative time formatter in your locale
// with default values explicitly passed in.
const rtf = new Intl.RelativeTimeFormat("en", {
  localeMatcher: "best fit", // other values: "lookup"
  numeric: "always", // other values: "auto"
  style: "long", // other values: "short" or "narrow"
});

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "1 day ago"

// Format relative time using positive value (1).
rtf.format(1, "day"); // "in 1 day"
```

Using formatToParts

The following example shows how to create a relative time formatter returning formatted parts.

```
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Format relative time using the day unit.
rtf.formatToParts(-1, "day");
```

```
// [{ type: "literal", value: "yesterday"}]

rtf.formatToParts(100, "day");
// [
//   { type: "literal", value: "in " },
//   { type: "integer", value: "100", unit: "day" },
//   { type: "literal", value: " days" }
// ]
```

See also

- Polyfill of `Intl.RelativeTimeFormat` in FormatJS
- `Intl`
- `Intl.RelativeTimeFormat` on v8.dev (2018)

Intl.RelativeTimeFormat.prototype.format()

The **`format()`** method of `Intl.RelativeTimeFormat` instances formats a `value` and `unit` according to the locale and formatting options of this `Intl.RelativeTimeFormat` object.

Try it

```
const rtf = new Intl.RelativeTimeFormat("en", { style: "short" });

console.log(rtf.format(3, "quarter"));
// Expected output: "in 3 qtrs."

console.log(rtf.format(-1, "day"));
// Expected output: "1 day ago"

console.log(rtf.format(10, "seconds"));
// Expected output: "in 10 sec."
```

Syntax

```
format(value, unit)
```

Parameters

`value`

> Numeric value to use in the internationalized relative time message.

`unit`

> Unit to use in the relative time internationalized message. Possible values are: `"year"`, `"quarter"`, `"month"`, `"week"`, `"day"`, `"hour"`, `"minute"`, `"second"`. Plural forms are also permitted.

Return value

A string representing the given `value` and `unit` formatted according to the locale and formatting options of this `Intl.RelativeTimeFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Basic format usage

The following example shows how to create a relative time formatter using the English language.

```
// Create a relative time formatter in your locale
// with default values explicitly passed in.
const rtf = new Intl.RelativeTimeFormat("en", {
  localeMatcher: "best fit", // other values: "lookup"
  numeric: "always", // other values: "auto"
  style: "long", // other values: "short" or "narrow"
});

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "1 day ago"

// Format relative time using positive value (1).
rtf.format(1, "day"); // "in 1 day"
```

Using the auto option

If the `numeric: "auto"` option is passed, it will produce the string `yesterday`, `today`, or `tomorrow` instead of `1 day ago`, `in 0 days`, or `in 1 day`. This allows you to avoid using numeric values in the output.

```
// Create a relative time formatter in your locale
// with numeric: "auto" option value passed in.
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "yesterday"

rtf.format(0, "day"); // "today"

// Format relative time using positive day unit (1).
rtf.format(1, "day"); // "tomorrow"
```

See also

- `Intl.RelativeTimeFormat`

Intl.RelativeTimeFormat.prototype.formatToParts()

The **`formatToParts()`** method of `Intl.RelativeTimeFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

```
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });
const parts = rtf.formatToParts(10, "seconds");

console.log(parts[0].value);
// Expected output: "in "

console.log(parts[1].value);
// Expected output: "10"

console.log(parts[2].value);
// Expected output: " seconds"
```

Syntax

```
formatToParts(value, unit)
```

Parameters

`value`

> Numeric value to use in the internationalized relative time message.

`unit`

> Unit to use in the relative time internationalized message. Possible values are: `"year"`, `"quarter"`, `"month"`, `"week"`, `"day"`, `"hour"`, `"minute"`, `"second"`. Plural forms are also permitted.

Return value

An `Array` of objects containing the formatted relative time in parts. Each object has two or three properties, `type`, `value`, and optionally `unit`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The parts can be thought of as directly obtained from calling `Intl.NumberFormat.prototype.formatToParts()` with the numeric value, passing only the `numberingSystem` option, and then adding additional `type: "literal"` tokens, such as `"in "`, `" days ago"`, etc. All tokens that are produced by the `NumberFormat` have an additional `unit` property, which is the singular form of the input `unit`; this is for programmatic usage and is not localized. The localized unit is output as a part of a literal token.

When `options.numeric` is `"auto"`, and there's a special string for the value, the returned array is a single literal token.

Examples

Using formatToParts()

```
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Format relative time using the day unit
rtf.formatToParts(-1, "day");
// [{ type: "literal", value: "yesterday"}]

rtf.formatToParts(100, "day");
// [
//   { type: "literal", value: "in " },
//   { type: "integer", value: "100", unit: "day" },
//   { type: "literal", value: " days" }
// ]
```

See also

- `Intl.RelativeTimeFormat`
- `Intl.RelativeTimeFormat.prototype.format()`

Intl.RelativeTimeFormat() constructor

The **`Intl.RelativeTimeFormat()`** constructor creates `Intl.RelativeTimeFormat` objects.

Syntax

```
new Intl.RelativeTimeFormat()
new Intl.RelativeTimeFormat(locales)
new Intl.RelativeTimeFormat(locales, options)
```

> **Note:** `Intl.RelativeTimeFormat()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension key is allowed:

`nu`

See `numberingSystem`.

This key can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

`style`

The style of the formatted relative time. Possible values are:

`"long"` (default)

E.g., "in 1 month"

`"short"`

E.g., "in 1 mo."

`"narrow"`

E.g., "in 1 mo.". The narrow style could be similar to the short style for some locales.

`numeric`

Whether to use numeric values in the output. Possible values are `"always"` and `"auto"`; the default is `"always"`. When set to `"auto"`, the output may use more idiomatic phrasing such as `"yesterday"` instead of `"1 day ago"`.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic format usage

The following example shows how to create a relative time formatter using the English language.

```
// Create a relative time formatter in your locale
// with default values explicitly passed in.
const rtf = new Intl.RelativeTimeFormat("en-US", {
  numeric: "always", // other values: "auto"
  style: "long", // other values: "short" or "narrow"
});

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "1 day ago"
```

```
// Format relative time using positive value (1).
rtf.format(1, "day"); // "in 1 day"
```

Using the auto option

If the `numeric: "auto"` option is passed, it will produce the string `yesterday` or `tomorrow` instead of `1 day ago` or `in 1 day`. This is useful when you don't want to use numeric values in the output.

```
// Create a relative time formatter in your locale
// with numeric: "auto" option value passed in.
const rtf = new Intl.RelativeTimeFormat("en-US", { numeric: "auto" });

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "yesterday"

// Format relative time using positive day unit (1).
rtf.format(1, "day"); // "tomorrow"
```

When the value is `0`, the output may be dependent on the unit. "0 seconds" is represented by the localized version of "now".

```
rtf.format(0, "second"); // "now"
rtf.format(0, "day"); // "today"
rtf.format(0, "minute"); // "this minute"
```

See also

- `Intl.RelativeTimeFormat`
- `Intl`
- `Intl.RelativeTimeFormat` on v8.dev (2018)

Intl.RelativeTimeFormat.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.RelativeTimeFormat` instances returns a new object with properties reflecting the options computed during initialization of this `RelativeTimeFormat` object.

Try it

```
const rtf1 = new Intl.RelativeTimeFormat("en", { style: "narrow" });
const options1 = rtf1.resolvedOptions();

const rtf2 = new Intl.RelativeTimeFormat("es", { numeric: "auto" });
const options2 = rtf2.resolvedOptions();

console.log(`${options1.locale}, ${options1.style}, ${options1.numeric}`);
// Expected output: "en, narrow, always"

console.log(`${options2.locale}, ${options2.style}, ${options2.numeric}`);
// Expected output: "es, long, auto"
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `RelativeTimeFormat` object. The object has the following properties, in the order they are listed:

`locale`

> The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `nu` Unicode extension key, if requested, may be included in the output.

`style`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"long"`, `"short"`, or `"narrow"`. The default is `"long"`.

`numeric`

> The value provided for this property in the `options` argument, with default filled in as needed. It is either `"always"` or `"auto"`. The default is `"always"`.

`numberingSystem`

> The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

Examples

Using the resolvedOptions() method

```
const de = new Intl.RelativeTimeFormat("de-DE");
const usedOptions = de.resolvedOptions();

usedOptions.locale; // "de-DE"
usedOptions.style; // "long"
usedOptions.numeric; // "always"
usedOptions.numberingSystem; // "latn"
```

See also

- `Intl.RelativeTimeFormat`

Intl.RelativeTimeFormat.supportedLocalesOf()

The **`Intl.RelativeTimeFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in relative time formatting without having to fall back to the runtime's default locale.

Try it

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.RelativeTimeFormat.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

```
Intl.RelativeTimeFormat.supportedLocalesOf(locales)
Intl.RelativeTimeFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

> A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object that may have the following property:
>
> `localeMatcher`
>
> > The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in relative time formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in relative time formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to relative time formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.RelativeTimeFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.RelativeTimeFormat`

Intl.Segmenter

The **`Intl.Segmenter`** object enables locale-sensitive text segmentation, enabling you to get meaningful items (graphemes, words or sentences) from a string.

Try it

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const iterator = segmenterFr.segment(string)[Symbol.iterator]();
```

```
console.log(iterator.next().value.segment);
// Expected output: 'Que'

console.log(iterator.next().value.segment);
// Expected output: ' '
```

Constructor

`Intl.Segmenter()`

Creates a new `Intl.Segmenter` object.

Static methods

`Intl.Segmenter.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.Segmenter.prototype` and shared by all `Intl.Segmenter` instances.

`Intl.Segmenter.prototype.constructor`

The constructor function that created the instance object. For `Intl.Segmenter` instances, the initial value is the `Intl.Segmenter` constructor.

`Intl.Segmenter.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.Segmenter"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.Segmenter.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and granularity options computed during initialization of this `Intl.Segmenter` object.

`Intl.Segmenter.prototype.segment()`

Returns a new iterable `Segments` instance representing the segments of a string according to the locale and granularity of this `Intl.Segmenter` instance.

Examples

Basic usage and difference from String.prototype.split()

If we were to use `String.prototype.split(" ")` to segment a text in words, we would not get the correct result if the locale of the text does not use whitespaces between words (which is the case for Japanese, Chinese, Thai, Lao, Khmer, Myanmar, etc.).

```
const str = "吾輩は猫である。名前はたぬき。";
console.table(str.split(" "));
// ['吾輩は猫である。名前はたぬき。']
// The two sentences are not correctly segmented.
```

```
const str = "吾輩は猫である。名前はたぬき。";
const segmenterJa = new Intl.Segmenter("ja-JP", { granularity: "word" });

const segments = segmenterJa.segment(str);
console.table(Array.from(segments));
// [{segment: '吾輩', index: 0, input: '吾輩は猫である。名前はたぬき。', isWordLike: true},
// etc.
// ]
```

See also

- Polyfill of `Intl.Segmenter` in FormatJS
- `Intl`

Intl.Segmenter.prototype.resolvedOptions()

The **`resolvedOptions()`** method of `Intl.Segmenter` instances returns a new object with properties reflecting the options computed during initialization of this `Segmenter` object.

Try it

```
const segmenter = new Intl.Segmenter("fr-FR");
const options = segmenter.resolvedOptions();

console.log(options.locale);
// Expected output: "fr-FR"

console.log(options.granularity);
// Expected output: "grapheme"
```

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `Segmenter` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`granularity`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"grapheme"`, `"word"`, or `"sentence"`. The default is `"grapheme"`.

Examples

Basic usage

```
const spanishSegmenter = new Intl.Segmenter("es", { granularity: "sentence" });
const options = spanishSegmenter.resolvedOptions();
console.log(options.locale); // "es"
console.log(options.granularity); // "sentence"
```

Default granularity

```
const spanishSegmenter = new Intl.Segmenter("es");
const options = spanishSegmenter.resolvedOptions();
console.log(options.locale); // "es"
console.log(options.granularity); // "grapheme"
```

Fallback locale

```
const banSegmenter = new Intl.Segmenter("ban");
const options = banSegmenter.resolvedOptions();
console.log(options.locale);
// "fr" on a runtime where the Balinese locale
// is not supported and French is the default locale
console.log(options.granularity); // "grapheme"
```

Intl.Segmenter.prototype.segment()

The **`segment()`** method of `Intl.Segmenter` instances segments a string according to the locale and granularity of this `Intl.Segmenter` object.

Try it

```
const string1 = "Que ma joie demeure";

const segmenterFrGrapheme = new Intl.Segmenter("fr", {
  granularity: "grapheme",
});
const graphemeSegments = segmenterFrGrapheme.segment(string1);

console.log(Array.from(graphemeSegments)[0]);
// Expected output:
// Object {segment: 'Q', index: 0, input: 'Que ma joie demeure'}
```

Syntax

```
segment(input)
```

Parameters

`input`

The text to be segmented as a string.

Return value

A new iterable `Segments` object containing the segments of the input string, using the segmenter's locale and granularity.

Examples

```
// Create a locale-specific word segmenter
const segmenter = new Intl.Segmenter("fr", { granularity: "word" });

// Use it to get an iterator over the segments of a string
```

```
const input = "Moi ? N'est-ce pas ?";
const segments = segmenter.segment(input);

// Use that for segmentation
for (const { segment, index, isWordLike } of segments) {
  console.log(
    "segment at code units [%d, %d]: «%s»%s",
    index,
    index + segment.length,
    segment,
    isWordLike ? " (word-like)" : "",
  );
}
// segment at code units [0, 3]: «Moi» (word-like)
// segment at code units [3, 4]: « »
// segment at code units [4, 5]: «?»
// segment at code units [5, 6]: « »
// segment at code units [6, 11]: «N'est» (word-like)
// segment at code units [11, 12]: «-»
// segment at code units [12, 14]: «ce» (word-like)
// segment at code units [14, 15]: « »
// segment at code units [15, 18]: «pas» (word-like)
// segment at code units [18, 19]: « »
// segment at code units [19, 20]: «?»
```

Segments

A **`Segments`** object is an iterable collection of the segments of a text string. It is returned by a call to the `segment()` method of an `Intl.Segmenter` object.

Instance methods

`Segments.prototype.containing()`

Returns an object describing the segment in the original string that includes the code unit at a specified index.

`Segments.prototype[Symbol.iterator]()`

Returns an iterator to iterate over the segments.

See also

- `Intl.Segmenter`
- `Intl.Segmenter.prototype.segment()`

Segments.prototype.containing()

The **`containing()`** method of `Segments` instances returns an object describing the segment in the string that includes the code unit at the specified index.

Try it

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const segments = segmenterFr.segment(string);

console.log(segments.containing(5));
// Expected output:
// Object {segment: 'ma', index: 4, input: 'Que ma joie demeure', isWordLike: true}
```

Syntax

```
containing(codeUnitIndex)
```

Parameters

`codeUnitIndex` Optional

A number specifying the index of the code unit in the original input string. If the value is omitted, it defaults to `0`.

Return value

An object describing the segment of the original string with the following properties, or `undefined` if the supplied index value is out of bounds.

`segment`

A string containing the segment extracted from the original input string.

`index`

The code unit index in the original input string at which the segment begins.

`input`

The complete input string that was segmented.

`isWordLike`

A boolean value only if `granularity` is `"word"`; otherwise, `undefined`. If `granularity` is `"word"`, then `isWordLike` is `true` when the segment is word-like (i.e., consists of letters/numbers/ideographs/etc.); otherwise, `false`.

Examples

```
// ┃0 1 2 3 4 5┃6┃7┃8┃9  ← code unit index
// ┃A 1 1 o n s┃-┃y┃!┃   ← code unit
const input = "Allons-y!";

const segmenter = new Intl.Segmenter("fr", { granularity: "word" });
const segments = segmenter.segment(input);

let current = segments.containing();
// { index: 0, segment: "Allons", isWordLike: true }

current = segments.containing(4);
// { index: 0, segment: "Allons", isWordLike: true }

current = segments.containing(6);
// { index: 6, segment: "-", isWordLike: false }

current = segments.containing(current.index + current.segment.length);
// { index: 7, segment: "y", isWordLike: true }

current = segments.containing(current.index + current.segment.length);
// { index: 8, segment: "!", isWordLike: false }

current = segments.containing(current.index + current.segment.length);
// undefined
```

See also

- `Intl.Segmenter`
- `Intl.Segmenter.prototype.segment()`

Segments.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `Segments` instances implements the iterable protocol and allows `Segments` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a segments iterator object that yields data about each segment.

Try it

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const iterator = segmenterFr.segment(string)[Symbol.iterator]();

for (const segment of iterator) {
  if (segment.segment.length > 4) {
    console.log(segment.segment);
  }
}

// Expected output: "demeure"
```

Syntax

```
segments[Symbol.iterator]()
```

Parameters

None.

Return value

A new iterable iterator object that yields data about each segment. Each yielded object has the same properties as the object returned by the `containing()` method.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `Segments` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const segmenter = new Intl.Segmenter("zh-CN", { granularity: "word" });
const input = "你好，世界！我爱编程。";

for (const value of segmenter.segment(input)) {
  console.log(value);
}

/*
```

```
{segment: '你好', index: 0, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '，', index: 2, input: '你好，世界！我爱编程。', isWordLike: false}
{segment: '世界', index: 3, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '！', index: 5, input: '你好，世界！我爱编程。', isWordLike: false}
{segment: '我', index: 6, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '爱', index: 7, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '编', index: 8, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '程', index: 9, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '。', index: 10, input: '你好，世界！我爱编程。', isWordLike: false}
*/
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
const segmenter = new Intl.Segmenter("fr", { granularity: "word" });
const input = "Moi ? N'est-ce pas ?";
const segments = segmenter.segment(input);
const iterator = segments[Symbol.iterator]();

let result = iterator.next();

while (!result.done) {
  console.log(result.value);
  result = iterator.next();
}

/*
{segment: 'Moi', index: 0, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: ' ', index: 3, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: '?', index: 4, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: ' ', index: 5, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: "N'est", index: 6, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: '-', index: 11, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: 'ce', index: 12, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: ' ', index: 14, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: 'pas', index: 15, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: ' ', index: 18, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: '?', index: 19, input: "Moi ? N'est-ce pas ?", isWordLike: false}
*/
```

See also

- `Intl.Segmenter`
- `Intl.Segmenter.prototype.segment()`
- `Symbol.iterator`
- Iteration protocols

Intl.Segmenter() constructor

The **`Intl.Segmenter()`** constructor creates `Intl.Segmenter` objects.

Try it

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const iterator = segmenterFr.segment(string)[Symbol.iterator]();

console.log(iterator.next().value.segment);
// Expected output: 'Que'

console.log(iterator.next().value.segment);
// Expected output: ' '
```

Syntax

```
new Intl.Segmenter()
new Intl.Segmenter(locales)
new Intl.Segmenter(locales, options)
```

> **Note:** `Intl.Segmenter()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

> A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`granularity`

How granularly should the input be split. Possible values are:

`"grapheme"` (default)

Split the input into segments at grapheme cluster (user-perceived character) boundaries, as determined by the locale.

`"word"`

Split the input into segments at word boundaries, as determined by the locale.

`"sentence"`

Split the input into segments at sentence boundaries, as determined by the locale.

Return value

A new `Intl.Segmenter` instance.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic usage

The following example shows how to count words in a string using the Japanese language (where splitting the string using `String` methods would have given an incorrect result).

```
const text = "吾輩は猫である。名前はたぬき。";
const japaneseSegmenter = new Intl.Segmenter("ja-JP", { granularity: "word" });
console.log(
  [...japaneseSegmenter.segment(text)].filter((segment) => segment.isWordLike)
    .length,
);
// 8, as the text is segmented as '吾輩'|'は'|'猫'|'で'|'ある'|'。'|'名前'|'は'|'たぬき'|'。'
```

Intl.Segmenter.supportedLocalesOf()

The **`Intl.Segmenter.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in segmentation without having to fall back to the runtime's default locale.

Try it

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup", granularity: "string" };

console.log(Intl.Segmenter.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

```
Intl.Segmenter.supportedLocalesOf(locales)
Intl.Segmenter.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in segmentation without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in segmentation, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to segmentation nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.Segmenter.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

See also

- `Intl.Segmenter`

Intl.supportedValuesOf()

The **`Intl.supportedValuesOf()`** static method returns an array containing the supported calendar, collation, currency, numbering systems, or unit values supported by the implementation.

Duplicates are omitted and the array is sorted in ascending lexicographical order (or more precisely, using `Array.prototype.sort()` with an `undefined` compare function).

The method can be used to feature-test whether values are supported in a particular implementation and download a polyfill only if necessary. It can also be used to build UIs that allow users to select their preferred localized values, for example when the UI is created from WebGL or server-side.

This method is locale-unaware: it is possible that certain identifiers are only supported or preferred in certain locales. If you want to determine the preferred values for a specific locale, you should use the `Intl.Locale` object, such as `Intl.Locale.prototype.getCalendars()`.

Try it

```
console.log(Intl.supportedValuesOf("calendar"));
console.log(Intl.supportedValuesOf("collation"));
console.log(Intl.supportedValuesOf("currency"));
console.log(Intl.supportedValuesOf("numberingSystem"));
console.log(Intl.supportedValuesOf("timeZone"));
console.log(Intl.supportedValuesOf("unit"));
// Expected output: Array ['key'] (for each key)

try {
  Intl.supportedValuesOf("someInvalidKey");
} catch (err) {
  console.log(err.toString());
  // Expected output: RangeError: invalid key: "someInvalidKey"
}
```

Syntax

```
Intl.supportedValuesOf(key)
```

Parameters

`key`

A key string indicating the category of values to be returned. This is one of:

- `"calendar"`: see supported calendar types
- `"collation"`: see supported collation types
- `"currency"`: see supported currency identifiers
- `"numberingSystem"`: see supported numbering system types
- `"timeZone"`: see supported time zone identifiers
- `"unit"`: see supported unit identifiers

Return value

A sorted array of unique string values indicating the values supported by the implementation for the given key. The values that could be returned are listed below.

Supported calendar types

Below are all values that are commonly supported by browsers for the `calendar` key. These values can be used for the `calendar` option or the `ca` Unicode extension key when creating objects such as `Intl.DateTimeFormat`, as well as for creating `Temporal` date objects. This list is explicitly sanctioned by the ECMA-402 specification, so all implementations should be consistent.

Value	Description
`buddhist`	Thai Buddhist calendar, proleptic. Month numbers, month codes, and days are the same as in the ISO 8601 calendar, but the epoch year is different. There is one era.

Value	Description
`chinese`	Traditional Chinese calendar, proleptic. Lunisolar calendar used in China based on data published by the Purple Mountain Observatory between 1900 and 2100 (which compiles with GB/T 33661-2017 between 1912 and 2100), falling back to an implementation-defined approximation outside that range. The arithmetic year is identical to `gregory`, and there are no eras.
`coptic`	Coptic calendar, proleptic. Similar solar algorithm to `ethioaa` and `ethiopic`, with one era and a different epoch year.
`dangi`	Traditional Korean calendar, proleptic. Lunisolar calendar using months published by the Korea Astronomy and Space Science Institute (KASI) between 1900 and 2050, falling back to an implementation-defined approximation outside that range. The arithmetic year is identical to `gregory`, and there are no eras.
`ethioaa`	Ethiopic calendar, Amete Alem, proleptic. Similar solar algorithm to `coptic` and `ethiopic`, with one era and a different epoch year.
`ethiopic`	Ethiopic calendar, Amete Mihret, proleptic. Similar solar algorithm to `coptic` and `ethioaa`, with two eras and a different epoch year.
`gregory`	Gregorian calendar, proleptic. Solar calendar almost identical to the ISO 8601 calendar, except that it does not define week numbering and it contains two eras, one before the epoch year.
`hebrew`	Hebrew calendar, proleptic. Civil calendar with Tishrei as the first month of the year. Lunisolar calendar with one leap month inserted after month 5. There is one era.
`indian`	Indian national (or Śaka) calendar, proleptic. Solar calendar with one era.
`islamic-civil`	Hijri calendar, proleptic, tabular/rule-based with leap year rule II (leap years 2,5,7,10,13,16,18,21,24,26,29 in the 30-year cycle (1-based numbering)) and civil epoch (Friday July 16, 622 Julian / 0622-07-19 ISO)
`islamic-tbla`	Hijri calendar, proleptic, tabular/rule-based with leap year rule II (leap years 2,5,7,10,13,16,18,21,24,26,29 in the 30-year cycle (1-based numbering)) and astronomical epoch (Thursday July 15, 622 Julian / 0622-07-18 ISO)
`islamic-umalqura`	Hijri calendar, proleptic, Umm al-Qura. Lunar calendar using KACST-calculated months from the start of 1300 AH (1882-11-12 ISO) to the end of 1600 AH (2174-11-25 ISO), falling back to `islamic-civil` outside that range.
`iso8601`	ISO calendar (variant of the Gregorian calendar with week rules and formatting parameters made region-independent)
`japanese`	Japanese Imperial calendar (this calendar adds an era for each new emperor, so the output year and era for a future date may not match the input year and era when your code runs on a future engine version. **Note:** See the remarks below this table about dates prior to 1868-10-23 ISO.)
`persian`	Persian (or Solar Hijri) calendar, proleptic. There is one era.
`roc`	Republic of China (or Minguo) calendar, proleptic. Month numbers, month codes, and days are the same as in the ISO 8601 calendar, but the epoch year is different. There are two eras, one before the epoch year and one after.

As of October 2025, in the `japanese` calendar, dates prior to 1868-10-23 ISO (the start date of the year 1 Meiji) don't work as expected in browsers in two ways. First, CLDR had the wrong start date for the Meiji era, which causes calendar implementations to extend the Meiji era further to the past than it actually did. Second, the upcoming Intl era and monthCode Proposal specifies that dates prior to 1873-01-01 ISO should use Gregorian eras, but browsers have traditionally used approximations of prior Japanese eras instead. The `japanese` calendar was taken into use on January 1, 6 Meiji / 1873-01-01 ISO, so these problems only affect proleptic dates.

The types below are specified in CLDR but do not have implementations distinct from the above calendars in browsers.

Value	Description	Notes
`ethiopic-amete-alem`	Ethiopic calendar, Amete Alem, proleptic.	This is an alias for `ethioaa` and therefore is not returned by `supportedValuesOf()`. Use `ethioaa` instead.
`islamic`	Hijri calendar, unspecified algorithm.	As of April 2025, this is an astronomical simulation whose parameters are undocumented and that is not known to match a specific Hijri calendar variant from non-software contexts. It is specified to be canonicalized to a different calendar, usually one of `islamic-umalqura`, `islamic-tbla`, or `islamic-civil`, and raise a warning.
`islamicc`	Civil (algorithmic) Arabic calendar.	This is an alias for `islamic-civil` and therefore is not returned by `supportedValuesOf()`. Use `islamic-civil` instead.

The `Temporal.PlainDate.prototype.era` and `Temporal.PlainDate.prototype.monthCode` docs provide more information about different calendars.

References:

- CLDR Calendar type keys
- UTS 35, Dates
- Islamic calendar types (CLDR design proposal)

Supported collation types

Below are all values that are commonly supported by browsers for the `collation` key. These values can be used for the `collation` option or the `co` Unicode extension key when creating objects such as `Intl.Collator`.

Value	Description
`compat`	A previous version of the ordering, for compatibility (for Arabic)
`dict`	Dictionary style ordering (for Sinhala). Also recognized as `dictionary`.
`emoji`	Recommended ordering for emoji characters. The requested language has no effect, so combining this with a language that requires language-specific rules breaks sorting for text in that language!
`eor`	European ordering rules (ENV 13710). This order is designed not to be language-specific. While the API requires attaching this to a language, the requested language has no effect on the order.
`phonebk`	Phonebook style ordering (for German). Also recognized as `phonebook`.

Value	Description
`phonetic`	Phonetic ordering (sorting based on pronunciation; for Lingala)
`pinyin`	Pinyin ordering for Latin and for Han characters (for Chinese)
`searchjl`	Special collation type for Korean initial consonant search. **Warning:** This collation is not for sorting, even though you can only use it with `Intl.Collator` of `usage: "sort"`.
`stroke`	Pinyin ordering for Latin, stroke order for Han characters (for Chinese)
`trad`	Traditional style ordering (such as in Spanish). Also recognized as `traditional`.
`unihan`	Unihan radical-stroke ordering for Han characters (for Chinese, Japanese, and Korean), Pinyin ordering for Latin for Chinese and Japanese standard kana ordering for Japanese
`zhuyin`	Pinyin ordering for Latin, zhuyin order for Bopomofo and Han characters (for Chinese)

The types below are specified in CLDR data, but are deprecated, are discouraged from explicit usage, and/or may not be indicated by browsers as supported for various reasons. Avoid using them:

Value	Description	Notes
`big5han`	Pinyin ordering for Latin, big5 charset ordering for CJK characters (used in Chinese)	Deprecated.
`direct`	Binary code point order (used in Hindi)	Deprecated.
`ducet`	The default Unicode collation element table order	The `ducet` collation type is not available to the Web.
`gb2312`	Pinyin ordering for Latin, gb2312han charset ordering for CJK characters (for Chinese). Also recognized as `gb2312han`.	Deprecated.
`reformed`	Reformed ordering (such as Swedish)	Deprecated. This is the old name for the default ordering for Swedish whose collation naming used to differ from other languages. Since this was the default, request `sv` instead of requesting `sv-u-co-reformed`.
`search`	Special collation type for string search	Do not use as a collation type, since in `Intl.Collator`, this collation is activated via the `usage: "search"` option. There is currently no API for substring search, so this is currently only good for filtering a list of strings by trying a full-string match of the key against each list item.
`standard`	Default ordering for each language, except Chinese (and, previously, Swedish)	Do not use explicitly. In general, it's unnecessary to specify this explicitly and specifying this for Swedish is problematic due to the different meaning for Swedish in the past.

References:

- CLDR Collation type keys
- UTS 35, Collation

Supported currency identifiers

Currency identifiers are three-letter uppercase codes defined in ISO 4217. These values can be used for the `currency` option when creating objects such as `Intl.NumberFormat`, as well as for `Intl.DisplayNames.prototype.of()`. There are over 300 identifiers in common use so we won't list them. For an exhaustive list of possible identifiers, see the Wikipedia article.

References:

- CLDR Currency type keys
- ISO 4217 Currency codes
- UTS 35, Currencies

Supported numbering system types

Below are all values that are commonly supported by browsers for the `numberingSystem` key. These values can be used for the `numberingSystem` option or the `nu` Unicode extension key when creating objects such as `Intl.NumberFormat`. For the rows with "digit characters", the runtime translates the digits one-by-one without extra actions. The others marked as "algorithmic" need additional algorithms to translate the digits. The higher the Unicode code point is, the newer the numbering system is and the more likely it is unsupported by all browsers.

Value	Description	Digit characters
`adlm`	Adlam digits	𞥐𞥑𞥒𞥓𞥔𞥕𞥖𞥗𞥘𞥙 (U+1E950 to U+1E959)
`ahom`	Ahom digits	𑜰𑜱𑜲𑜳𑜴𑜵𑜶𑜷𑜸𑜹 (U+11730 to U+11739)
`arab`	Arabic-Indic digits	٠١٢٣٤٥٦٧٨٩ (U+0660 to U+0669)
`arabext`	Extended Arabic-Indic digits	۰۱۲۳۴۵۶۷۸۹ (U+06F0 to U+06F9)
`armn`	Armenian upper case numerals	algorithmic
`armnlow`	Armenian lower case numerals	algorithmic
`bali`	Balinese digits	᭐᭑᭒᭓᭔᭕᭖᭗᭘᭙ (U+1B50 to U+1B59)
`beng`	Bengali digits	০১২৩৪৫৬৭৮৯ (U+09E6 to U+09EF)
`bhks`	Bhaiksuki digits	𑱐𑱑𑱒𑱓𑱔𑱕𑱖𑱗𑱘𑱙 (U+11C50 to U+11C59)
`brah`	Brahmi digits	𑁦𑁧𑁨𑁩𑁪𑁫𑁬𑁭𑁮𑁯 (U+11066 to U+1106F)

Value	Description	Digit characters
cakm	Chakma digits	𑄶𑄷𑄸𑄹𑄺𑄻𑄼𑄽𑄾𑄿 (U+11136 to U+1113F)
cham	Cham digits	꩐꩑꩒꩓꩔꩕꩖꩗꩘꩙ (U+AA50 to U+AA59)
cyrl	Cyrillic numerals	algorithmic
deva	Devanagari digits	०१२३४५६७८९ (U+0966 to U+096F)
diak	Dives Akuru digits	𑥐𑥑𑥒𑥓𑥔𑥕𑥖𑥗𑥘𑥙 (U+11950 to U+11959)
ethi	Ethiopic numerals	algorithmic
fullwide	Full width digits	０１２３４５６７８９ (U+FF10 to U+FF19)
gara	Garay digits	𐵀𐵁𐵂𐵃𐵄𐵅𐵆𐵇𐵈𐵉 (U+10D40 to U+10D49)
geor	Georgian numerals	algorithmic
gong	Gunjala Gondi digits	𑶠𑶡𑶢𑶣𑶤𑶥𑶦𑶧𑶨𑶩 (U+11DA0 to U+11DA9)
gonm	Masaram Gondi digits	𑵐𑵑𑵒𑵓𑵔𑵕𑵖𑵗𑵘𑵙 (U+11D50 to U+11D59)
grek	Greek upper case numerals	algorithmic
greklow	Greek lower case numerals	algorithmic
gujr	Gujarati digits	૦૧૨૩૪૫૬૭૮૯ (U+0AE6 to U+0AEF)
gukh	Gurung Khema digits	𖄰𖄱𖄲𖄳𖄴𖄵𖄶𖄷𖄸𖄹 (U+16130 to U+16139)
guru	Gurmukhi digits	੦੧੨੩੪੫੬੭੮੯ (U+0A66 to U+0A6F)
hanidays	Han-character day-of-month numbering for lunar/other traditional calendars	
hanidec	Positional decimal system using Chinese number ideographs as digits	〇一二三四五六七八九 (U+3007, U+4E00, U+4E8C, U+4E09, U+56DB, U+4E94, U+516D, U+4E03, U+516B, U+4E5D)
hans	Simplified Chinese numerals	algorithmic
hansfin	Simplified Chinese financial numerals	algorithmic
hant	Traditional Chinese numerals	algorithmic
hantfin	Traditional Chinese financial numerals	algorithmic
hebr	Hebrew numerals	algorithmic
hmng	Pahawh Hmong digits	𖭐𖭑𖭒𖭓𖭔𖭕𖭖𖭗𖭘𖭙 (U+16B50 to U+16B59)
hmnp	Nyiakeng Puachue Hmong digits	𞅀𞅁𞅂𞅃𞅄𞅅𞅆𞅇𞅈𞅉 (U+1E140 to U+1E149)
java	Javanese digits	꧐꧑꧒꧓꧔꧕꧖꧗꧘꧙ (U+A9D0 to U+A9D9)
jpan	Japanese numerals	algorithmic
jpanfin	Japanese financial numerals	algorithmic
jpanyear	Japanese first-year Gannen numbering for Japanese calendar	algorithmic
kali	Kayah Li digits	꤀꤁꤂꤃꤄꤅꤆꤇꤈꤉ (U+A900 to U+A909)
kawi	Kawi digits	𑽐𑽑𑽒𑽓𑽔𑽕𑽖𑽗𑽘𑽙 (U+11F50 to U+11F59)
khmr	Khmer digits	០១២៣៤៥៦៧៨៩ (U+17E0 to U+17E9)
knda	Kannada digits	೦೧೨೩೪೫೬೭೮೯ (U+0CE6 to U+0CEF)
krai	Kirat Rai digits	𖵰𖵱𖵲𖵳𖵴𖵵𖵶𖵷𖵸𖵹 (U+16D70 to U+16D79)
lana	Tai Tham Hora (secular) digits	᪀᪁᪂᪃᪄᪅᪆᪇᪈᪉ (U+1A80 to U+1A89)
lanatham	Tai Tham (ecclesiastical) digits	᪐᪑᪒᪓᪔᪕᪖᪗᪘᪙ (U+1A90 to U+1A99)
laoo	Lao digits	๐๑๒๓๔๕๖๗๘๙ (U+0ED0 to U+0ED9)
latn	Latin digits	0123456789 (U+0030 to U+0039)
lepc	Lepcha digits	᱀᱁᱂᱃᱄᱅᱆᱇᱈᱉ (U+1C40 to U+1C49)
limb	Limbu digits	᥆᥇᥈᥉᥊᥋᥌᥍᥎᥏ (U+1946 to U+194F)
mathbold	Mathematical bold digits	**0123456789** (U+1D7CE to U+1D7D7)
mathdbl	Mathematical double-struck digits	𝟘𝟙𝟚𝟛𝟜𝟝𝟞𝟟𝟠𝟡 (U+1D7D8 to U+1D7E1)
mathmono	Mathematical monospace digits	𝟶𝟷𝟸𝟹𝟺𝟻𝟼𝟽𝟾𝟿 (U+1D7F6 to U+1D7FF)
mathsanb	Mathematical sans-serif bold digits	**0123456789** (U+1D7EC to U+1D7F5)
mathsans	Mathematical sans-serif digits	0123456789 (U+1D7E2 to U+1D7EB)
mlym	Malayalam digits	൦൧൨൩൪൫൬൭൮൯ (U+0D66 to U+0D6F)
modi	Modi digits	𑙐𑙑𑙒𑙓𑙔𑙕𑙖𑙗𑙘𑙙 (U+11650 to U+11659)
mong	Mongolian digits	᠐᠑᠒᠓᠔᠕᠖᠗᠘᠙ (U+1810 to U+1819)
mroo	Mro digits	𖩠𖩡𖩢𖩣𖩤𖩥𖩦𖩧𖩨𖩩 (U+16A60 to U+16A69)
mtei	Meetei Mayek digits	꯰꯱꯲꯳꯴꯵꯶꯷꯸꯹ (U+ABF0 to U+ABF9)
mymr	Myanmar digits	၀၁၂၃၄၅၆၇၈၉ (U+1040 to U+1049)
mymrepka	Myanmar Eastern Pwo Karen digits	𑋰𑋱𑋲𑋳𑋴𑋵𑋶𑋷𑋸𑋹 (U+116DA to U+116E3)

Value	Description	Digit characters
`mymrpao`	Myanmar Pao digits	𑵐𑵑𑵒𑵓𑵔𑵕𑵖𑵗𑵘𑵙 (U+116D0 to U+116D9)
`mymrshan`	Myanmar Shan digits	၀၁၂၃၄၅၆၇၈၉ (U+1090 to U+1099)
`mymrtlng`	Myanmar Tai Laing digits	꧐꧑꧒꧓꧔꧕꧖꧗꧘꧙ (U+A9F0 to U+A9F9)
`nagm`	Nag Mundari digits	𞓰𞓱𞓲𞓳𞓴𞓵𞓶𞓷𞓸𞓹 (U+1E4F0 to U+1E4F9)
`newa`	Newa digits	𑐐𑐑𑐒𑐓𑐔𑐕𑐖𑐗𑐘𑐙 (U+11450 to U+11459)
`nkoo`	N'Ko digits	߀߁߂߃߄߅߆߇߈߉ (U+07C0 to U+07C9)
`olck`	Ol Chiki digits	᱐᱑᱒᱓᱔᱕᱖᱗᱘᱙ (U+1C50 to U+1C59)
`onao`	Ol Onal digits	𞗱𞗲𞗳𞗴𞗵𞗶𞗷𞗸𞗹𞗺 (U+1E5F1 to U+1E5FA)
`orya`	Oriya digits	୦୧୨୩୪୫୬୭୮୯ (U+0B66 to U+0B6F)
`osma`	Osmanya digits	𐒠𐒡𐒢𐒣𐒤𐒥𐒦𐒧𐒨𐒩 (U+104A0 to U+104A9)
`outlined`	Legacy computing outlined digits	𜳰𜳱𜳲𜳳𜳴𜳵𜳶𜳷𜳸𜳹 (U+1CCF0 to U+1CCF9)
`rohg`	Hanifi Rohingya digits	𐴰𐴱𐴲𐴳𐴴𐴵𐴶𐴷𐴸𐴹 (U+10D30 to U+10D39)
`roman`	Roman upper case numerals	algorithmic
`romanlow`	Roman lowercase numerals	algorithmic
`saur`	Saurashtra digits	꣐꣑꣒꣓꣔꣕꣖꣗꣘꣙ (U+A8D0 to U+A8D9)
`segment`	Legacy computing segmented digits	🯰🯱🯲🯳🯴🯵🯶🯷🯸🯹 (U+1FBF0 to U+1FBF9)
`shrd`	Sharada digits	𑇐𑇑𑇒𑇓𑇔𑇕𑇖𑇗𑇘𑇙 (U+111D0 to U+111D9)
`sind`	Khudawadi digits	𑋰𑋱𑋲𑋳𑋴𑋵𑋶𑋷𑋸𑋹 (U+112F0 to U+112F9)
`sinh`	Sinhala Lith digits	෦෧෨෩෪෫෬෭෮෯ (U+0DE6 to U+0DEF)
`sora`	Sora_Sompeng digits	𑃰𑃱𑃲𑃳𑃴𑃵𑃶𑃷𑃸𑃹 (U+110F0 to U+110F9)
`sund`	Sundanese digits	᮰᮱᮲᮳᮴᮵᮶᮷᮸᮹ (U+1BB0 to U+1BB9)
`sunu`	Sunuwar digits	𑯰𑯱𑯲𑯳𑯴𑯵𑯶𑯷𑯸𑯹 (U+11BF0 to U+11BF9)
`takr`	Takri digits	𑛀𑛁𑛂𑛃𑛄𑛅𑛆𑛇𑛈𑛉 (U+116C0 to U+116C9)
`talu`	New Tai Lue digits	᧐᧑᧒᧓᧔᧕᧖᧗᧘᧙ (U+19D0 to U+19D9)
`taml`	Tamil numerals	algorithmic
`tamldec`	Modern Tamil decimal digits	௦௧௨௩௪௫௬௭௮௯ (U+0BE6 to U+0BEF)
`telu`	Telugu digits	౦౧౨౩౪౫౬౭౮౯ (U+0C66 to U+0C6F)
`thai`	Thai digits	๐๑๒๓๔๕๖๗๘๙ (U+0E50 to U+0E59)
`tibt`	Tibetan digits	༠༡༢༣༤༥༦༧༨༩ (U+0F20 to U+0F29)
`tirh`	Tirhuta digits	𑓐𑓑𑓒𑓓𑓔𑓕𑓖𑓗𑓘𑓙 (U+114D0 to U+114D9)
`tnsa`	Tangsa digits	𖩠𖩡𖩢𖩣𖩤𖩥𖩦𖩧𖩨𖩩 (U+16AC0 to U+16AC9)
`vaii`	Vai digits	꘠꘡꘢꘣꘤꘥꘦꘧꘨꘩ (U+A620 to U+A629)
`wara`	Warang Citi digits	𑣠𑣡𑣢𑣣𑣤𑣥𑣦𑣧𑣨𑣩 (U+118E0 to U+118E9)
`wcho`	Wancho digits	𞋰𞋱𞋲𞋳𞋴𞋵𞋶𞋷𞋸𞋹 (U+1E2F0 to U+1E2F9)

There are three special values: `native`, `traditio`, and `finance`, whose meanings are locale-dependent, and will be resolved to the right system depending on the locale. Therefore, the `resolvedOptions()` methods will never return these values, but `Intl.Locale.prototype.numberingSystem` will (if provided as input).

References:

- CLDR Numbering system type keys
- CLDR Numbering system definitions
- UTS 35, Numbering systems

Supported time zone identifiers

Supported time zone identifiers can be used for the `timeZone` option when creating objects such as `Intl.DateTimeFormat`, as well as for creating `Temporal` date objects. There are over 400 identifiers in common use so we won't list them. For an exhaustive list of possible identifiers, see the Wikipedia article or the IANA time zone database.

As you browse the list, note that the standardization of `Temporal` requires browsers to always return the primary identifier in the IANA database, which may change over time. See time zones and offsets for more information. For example, the returned array should contain `"Asia/Kolkata"` instead of `"Asia/Calcutta"` because the latter is an alias of the former and they both correspond to India; however, it should contain both `"Africa/Abidjan"` and `"Atlantic/Reykjavik"` because they are in different countries, despite the latter also being an alias of the former.

References:

- IANA Time Zone Database
- UTS 35, Time Zone Identifiers

Supported unit identifiers

Below are all values that are commonly supported by browsers for the `unit` key. These values can be used for the `unit` option when creating objects such as `Intl.NumberFormat`. This list is a subset of the CLDR explicitly sanctioned by the ECMA-402 specification, so all implementations should be consistent.

- `acre`
- `bit`
- `byte`
- `celsius`
- `centimeter`
- `day`
- `degree`
- `fahrenheit`
- `fluid-ounce`
- `foot`
- `gallon`
- `gigabit`
- `gigabyte`
- `gram`
- `hectare`
- `hour`
- `inch`
- `kilobit`
- `kilobyte`
- `kilogram`
- `kilometer`
- `liter`
- `megabit`
- `megabyte`
- `meter`
- `microsecond`
- `mile`
- `mile-scandinavian`
- `milliliter`
- `millimeter`
- `millisecond`
- `minute`
- `month`
- `nanosecond`
- `ounce`
- `percent`
- `petabyte`
- `pound`
- `second`
- `stone`
- `terabit`
- `terabyte`
- `week`
- `yard`
- `year`

When specifying units, you can also combine two units with the "-per-" separator. For example, `meter-per-second` or `liter-per-megabyte`.

References:

- ECMA-402 sanctioned single units
- CLDR Unit validity data
- UTS 35, Unit identifiers

Exceptions

`RangeError`

Thrown if an unsupported key was passed as a parameter.

Examples

Feature testing

You can check that the method is supported by comparing to `undefined`:

```
if (typeof Intl.supportedValuesOf !== "undefined") {
  // method is supported
}
```

Get all values for key

To get the supported values for calendar you call the method with the key `"calendar"`. You can then iterate through the returned array as shown below:

```
Intl.supportedValuesOf("calendar").forEach((calendar) => {
  // "buddhist", "chinese", "coptic", "dangi", etc.
});
```

The other values are all obtained in the same way:

```
Intl.supportedValuesOf("collation").forEach((collation) => {
  // "compat", "dict", "emoji", etc.
});

Intl.supportedValuesOf("currency").forEach((currency) => {
  // "ADP", "AED", "AFA", "AFN", "ALK", "ALL", "AMD", etc.
});

Intl.supportedValuesOf("numberingSystem").forEach((numberingSystem) => {
  // "adlm", "ahom", "arab", "arabext", "bali", etc.
});

Intl.supportedValuesOf("timeZone").forEach((timeZone) => {
  // "Africa/Abidjan", "Africa/Accra", "Africa/Addis_Ababa", "Africa/Algiers", etc.
});

Intl.supportedValuesOf("unit").forEach((unit) => {
  // "acre", "bit", "byte", "celsius", "centimeter", etc.
});
```

Invalid key throws RangeError

```
try {
  Intl.supportedValuesOf("someInvalidKey");
} catch (err) {
  // RangeError: invalid key: "someInvalidKey"
}
```

See also

- Polyfill of `Intl.supportedValuesOf` in FormatJS
- `Intl`

isFinite()

The **`isFinite()`** function determines whether a value is finite, first converting the value to a number if necessary. A finite number is one that's not `NaN` or `±Infinity`. Because coercion inside the `isFinite()` function can be surprising, you may prefer to use `Number.isFinite()`.

Try it

```
function div(x) {
  if (isFinite(1000 / x)) {
    return "Number is NOT Infinity.";
  }
  return "Number is Infinity!";
}

console.log(div(0));
// Expected output: "Number is Infinity!""

console.log(div(1));
// Expected output: "Number is NOT Infinity."
```

Syntax

```
isFinite(value)
```

Parameters

`value`

The value to be tested.

Return value

`false` if the given value is `NaN`, `Infinity`, or `-Infinity` after being converted to a number; otherwise, `true`.

Description

`isFinite()` is a function property of the global object.

When the argument to the `isFinite()` function is not of type Number, the value is first coerced to a number, and the resulting value is then compared against `NaN` and ±Infinity. This is as confusing as the behavior of `isNaN` — for example, `isFinite("1")` is `true`.

`Number.isFinite()` is a more reliable way to test whether a value is a finite number value, because it returns `false` for any non-number input.

Examples

Using isFinite()

```
isFinite(Infinity); // false
isFinite(NaN); // false
isFinite(-Infinity); // false

isFinite(0); // true
isFinite(2e64); // true
isFinite(910); // true

// Would've been false with the more robust Number.isFinite():
isFinite(null); // true
isFinite("0"); // true
```

See also

- `Number.isFinite()`
- `Number.NaN`
- `Number.POSITIVE_INFINITY`
- `Number.NEGATIVE_INFINITY`

isNaN()

The **`isNaN()`** function determines whether a value is `NaN`, first converting the value to a number if necessary. Because coercion inside the `isNaN()` function can be surprising, you may prefer to use `Number.isNaN()`.

Try it

```
function milliseconds(x) {
  if (isNaN(x)) {
    return "Not a Number!";
  }
  return x * 1000;
}

console.log(milliseconds("100F"));
// Expected output: "Not a Number!"

console.log(milliseconds("0.0314E+2"));
// Expected output: 3140
```

Syntax

```
isNaN(value)
```

Parameters

`value`

The value to be tested.

Return value

`true` if the given value is `NaN` after being converted to a number; otherwise, `false`.

Description

`isNaN()` is a function property of the global object.

For number values, `isNaN()` tests if the number is the value `NaN`. When the argument to the `isNaN()` function is not of type Number, the value is first coerced to a number, and the resulting value is then compared against `NaN`.

This behavior of `isNaN()` for non-numeric arguments can be confusing! For example, an empty string is coerced to 0, while a boolean is coerced to 0 or 1; both values are intuitively "not numbers", but they don't evaluate to `NaN`, so `isNaN()` returns `false`. Therefore, `isNaN()` answers neither the question "is the input the floating point `NaN` value" nor the question "is the input not a number".

`Number.isNaN()` is a more reliable way to test whether a value is the number value `NaN` or not. Alternatively, the expression `x !== x` can be used, and neither of the solutions is subject to the false positives that make the global `isNaN()` unreliable. To test if a value is a number, use `typeof x === "number"`.

The `isNaN()` function answers the question "is the input functionally equivalent to `NaN` when used in a number context". If `isNaN(x)` returns `false`, you can use `x` in an arithmetic expression as if it's a valid number that's not `NaN`. If `isNaN(x)` returns `true`, `x` will get coerced to `NaN` and make most arithmetic expressions return `NaN` (because `NaN` propagates). You can use this, for example, to test whether an argument to a function is arithmetically processable (usable "like" a number), and handle values that are not number-like by throwing an error, providing a default value, etc. This way, you can have a function that makes use of the full versatility JavaScript provides by implicitly converting values depending on context.

> **Note:** The + operator performs both number addition and string concatenation. Therefore, even if `isNaN()` returns `false` for both operands, the + operator may still return a string, because it's not used as an arithmetic operator. For example, `isNaN("1")` returns `false`, but `"1" + 1` returns `"11"`. To be sure that you are working with numbers, coerce the value to a number and use `Number.isNaN()` to test the result.

Examples

Note how `isNaN()` returns `true` for values that are not the value `NaN` but are not numbers either:

```
isNaN(NaN); // true
isNaN(undefined); // true
isNaN({}); // true

isNaN(true); // false
isNaN(null); // false
isNaN(37); // false

// Strings
isNaN("37"); // false: "37" is converted to the number 37 which is not NaN
isNaN("37.37"); // false: "37.37" is converted to the number 37.37 which is not NaN
isNaN("37,5"); // true
isNaN("123ABC"); // true: Number("123ABC") is NaN
isNaN(""); // false: the empty string is converted to 0 which is not NaN
isNaN(" "); // false: a string with spaces is converted to 0 which is not NaN

// Dates
isNaN(new Date()); // false; Date objects can be converted to a number (timestamp)
isNaN(new Date().toString()); // true; the string representation of a Date object cannot be parsed as a number

// Arrays
isNaN([]); // false; the primitive representation is "", which converts to the number 0
isNaN([1]); // false; the primitive representation is "1"
isNaN([1, 2]); // true; the primitive representation is "1,2", which cannot be parsed as number
```

See also

- `NaN`
- `Number.isNaN()`

Iterator

An **`Iterator`** object is an object that conforms to the iterator protocol by providing a `next()` method that returns an iterator result object. All built-in iterators inherit from the `Iterator` class. The `Iterator` class provides a `[Symbol.iterator]()` method that returns the iterator object itself, making the iterator also iterable. It also provides some helper methods for working with iterators.

Description

The following are all built-in JavaScript iterators:

- The *Array Iterator* returned by `Array.prototype.values()`, `Array.prototype.keys()`, `Array.prototype.entries()`, `Array.prototype[Symbol.iterator]()`, `TypedArray.prototype.values()`, `TypedArray.prototype.keys()`, `TypedArray.prototype.entries()`, `TypedArray.prototype[Symbol.iterator]()`, and `arguments[Symbol.iterator]()`.
- The *String Iterator* returned by `String.prototype[Symbol.iterator]()`.
- The *Map Iterator* returned by `Map.prototype.values()`, `Map.prototype.keys()`, `Map.prototype.entries()`, and `Map.prototype[Symbol.iterator]()`.
- The *Set Iterator* returned by `Set.prototype.values()`, `Set.prototype.keys()`, `Set.prototype.entries()`, and `Set.prototype[Symbol.iterator]()`.
- The *RegExp String Iterator* returned by `RegExp.prototype[Symbol.matchAll]()` and `String.prototype.matchAll()`.
- The `Generator` object returned by generator functions.
- The *Segments Iterator* returned by the `[Symbol.iterator]()` method of the `Segments` object returned by `Intl.Segmenter.prototype.segment()`.
- The *Iterator Helper* returned by iterator helper methods such as `Iterator.prototype.filter()` and `Iterator.prototype.map()`.

Web APIs may return iterators too. Some reuse core JavaScript iterators while others define their own iterators. For example:

- Array-Like objects such as `NodeList` return an *Array Iterator* from their respective methods `keys()`, `values()`, `entries()`, and `[Symbol.iterator]()`.
- Map-Like objects from Web APIs such as `Headers` return their own iterator type like *Headers Iterator* from their respective methods `keys()`, `values()`, `entries()`, and `[Symbol.iterator]()`.
- Set-Like objects from Web APIs such as `FontFaceSet` return their own iterator type like *FontFaceSet Iterator* from their respective methods `keys()`, `values()`, `entries()`, and `[Symbol.iterator]()`.

> **Note:** `NodeIterator` and other old interfaces are named as such but do not conform to the iterator protocol or iterable protocol.

Each of these iterators have a distinct prototype object, which defines the `next()` method used by the particular iterator. For example, all string iterator objects inherit from a hidden object `StringIteratorPrototype`, which has a `next()` method that iterates this string by code points. `StringIteratorPrototype` also has a `[Symbol.toStringTag]` property whose initial value is the string `"String Iterator"`. This property is used in `Object.prototype.toString()`. Similarly, other iterator prototypes also have their own `[Symbol.toStringTag]` values, which are the same as the names given above.

All of these prototype objects inherit from `Iterator.prototype`, which provides a `[Symbol.iterator]()` method that returns the iterator object itself, making the iterator also iterable.

Iterator helper methods

> **Note:** These methods are *iterator* helpers, not *iterable* helpers, because the only requirement for an object to be iterable is just the presence of a `[Symbol.iterator]()` method. There is no shared prototype to install these methods on.

The `Iterator` class itself provides some helper methods for working with iterators. For example, you may be tempted to do the following:

```
const nameToDeposit = new Map([
  ["Anne", 1000],
```

```
  ["Bert", 1500],
  ["Carl", 2000],
]);

const totalDeposit = [...nameToDeposit.values()].reduce((a, b) => a + b);
```

This first converts the iterator returned by `Map.prototype.values()` to an array, then uses the `Array.prototype.reduce()` method to calculate the sum. However, this both creates an intermediate array and iterates the array twice. Instead, you can use the `reduce()` method of the iterator itself:

```
const totalDeposit = nameToDeposit.values().reduce((a, b) => a + b);
```

This method may be more efficient, especially memory-wise, because it only iterates the iterator once, without memorizing any intermediate values. Iterator helper methods are necessary to work with infinite iterators:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci();
const firstThreeDigitTerm = seq.find((n) => n >= 100);
```

You cannot convert `seq` to an array, because it is infinite. Instead, you can use the `find()` method of the iterator itself, which only iterates `seq` as far as necessary to find the first value that satisfies the condition.

You will find many iterator methods analogous to array methods, such as:

Iterator method	Array method
`Iterator.prototype.every()`	`Array.prototype.every()`
`Iterator.prototype.filter()`	`Array.prototype.filter()`
`Iterator.prototype.find()`	`Array.prototype.find()`
`Iterator.prototype.flatMap()`	`Array.prototype.flatMap()`
`Iterator.prototype.forEach()`	`Array.prototype.forEach()`
`Iterator.prototype.includes()`	`Array.prototype.includes()`
`Iterator.prototype.join()`	`Array.prototype.join()`
`Iterator.prototype.map()`	`Array.prototype.map()`
`Iterator.prototype.reduce()`	`Array.prototype.reduce()`
`Iterator.prototype.some()`	`Array.prototype.some()`

`Iterator.prototype.drop()` and `Iterator.prototype.take()` combined are somewhat analogous to `Array.prototype.slice()`.

Iterator helper objects

> **Note:** *Iterator helper objects* and *iterator helper methods* are two different concepts. An Iterator helper object is detectable at runtime, while "iterator helper method" is just a name for a set of methods for comprehension. *Iterator helper* may refer to either the object or the method, depending on the context.

Among the iterator helper methods, `filter()`, `flatMap()`, `map()`, `drop()`, and `take()` return a new *Iterator Helper* object. The iterator helper is also an `Iterator` instance, making these helper methods chainable. All iterator helper objects inherit from a common prototype object, which implements the iterator protocol:

`next()`

Calls the `next()` method of the underlying iterator, applies the helper method to the result, and returns the result.

`return()`

Calls the `return()` method of the underlying iterator, and returns the result.

The iterator helper shares the same data source as the underlying iterator, so iterating the iterator helper causes the underlying iterator to be iterated as well. There is no way to "fork" an iterator to allow it to be iterated multiple times.

```
const it = [1, 2, 3].values();
const it2 = it.drop(0); // Essentially a copy
console.log(it.next().value); // 1
console.log(it2.next().value); // 2
console.log(it.next().value); // 3
```

Proper iterators

There are two kinds of "iterators": objects that conform to the iterator protocol (which, at its minimum, only requires the presence of a `next()` method), and objects that inherit from the `Iterator` class, which enjoy the helper methods. They do not entail each other — objects that inherit from `Iterator` do not automatically become iterators, because the `Iterator` class does not define a `next()` method. Instead, the object needs to define a `next()` method itself. A *proper iterator* is one that both

conforms to the iterator protocol and inherits from `Iterator`, and most code expect iterators to be proper iterators and iterables to return proper iterators. To create proper iterators, define a class that extends `Iterator`, or use the `Iterator.from()` method.

```
class MyIterator extends Iterator {
  next() {
    // …
  }
}

const myIterator = Iterator.from({
  next() {
    // …
  },
});
```

Constructor

`Iterator()`

Intended to be extended by other classes that create iterators. Throws an error when constructed by itself.

Static methods

`Iterator.concat()`

Creates a new `Iterator` object from a list of iterable objects. The new iterator yields the values from each of the input iterables in sequence.

`Iterator.from()`

Creates a new `Iterator` object from an iterator or iterable object.

`Iterator.zip()`

Creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding arrays containing elements at the same position.

`Iterator.zipKeyed()`

Creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding objects containing elements at the same position, with keys specified by the input.

Instance properties

These properties are defined on `Iterator.prototype` and shared by all `Iterator` instances.

`Iterator.prototype.constructor`

The constructor function that created the instance object. For `Iterator` instances, the initial value is the `Iterator` constructor.

`Iterator.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Iterator"`. This property is used in `Object.prototype.toString()`.

> **Note:** Unlike the `[Symbol.toStringTag]` on most built-in classes, `Iterator.prototype[Symbol.toStringTag]` is writable for web compatibility reasons.

Instance methods

`Iterator.prototype.chunks()`

Returns a new iterator helper object that splits the elements from the original iterator into consecutive array chunks. Each time the helper is iterated, it gets the specified number of elements from the underlying iterator and yields them together.

`Iterator.prototype.drop()`

Returns a new iterator helper object that skips the given number of elements at the start of this iterator.

`Iterator.prototype.every()`

Returns `false` if it finds an element that does not satisfy the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `true`.

`Iterator.prototype.filter()`

Returns a new iterator helper object that yields only those elements of the iterator for which the provided callback function returns `true`.

`Iterator.prototype.find()`

Returns the first element produced by the iterator that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

`Iterator.prototype.flatMap()`

Returns a new iterator helper object that takes each element in the original iterator, runs it through a mapping function, and yields elements returned by the mapping function (which are contained in another iterator or iterable).

`Iterator.prototype.forEach()`

Executes a provided function once for each element produced by the iterator.

`Iterator.prototype.includes()`

Returns `true` if an element produced by the iterator is equal to the given value. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

`Iterator.prototype.join()`

Returns a string that is the concatenation of all elements produced by the iterator, separated by commas or a specified separator string.

`Iterator.prototype.map()`

Returns a new iterator helper object that yields elements of the iterator, each transformed by a mapping function.

`Iterator.prototype.reduce()`

Executes a user-supplied "reducer" callback function on each element produced by the iterator, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements is a single value.

`Iterator.prototype.some()`

Returns `true` if it finds an element that satisfies the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

`Iterator.prototype.take()`

Returns a new iterator helper object that yields the given number of elements in this iterator and then terminates.

`Iterator.prototype.toArray()`

Creates a new `Array` instance populated with the elements yielded from the iterator.

`Iterator.prototype.windows()`

Returns a new iterator helper object that yields a sliding window of elements. Each time the helper is iterated, it yields an array removing the first element from the previous iteration and adding the next element from the original iterator.

`Iterator.prototype[Symbol.dispose]()`

Calls the `return()` method of `this`, if it exists. This implements the *disposable protocol* and allows it to be disposed when used with `using` or `await using`.

`Iterator.prototype[Symbol.iterator]()`

Returns the iterator object itself. This allows iterator objects to also be iterable.

Examples

Using an iterator as an iterable

All built-in iterators are also iterable, so you can use them in a `for...of` loop:

```
const arrIterator = [1, 2, 3].values();
for (const value of arrIterator) {
  console.log(value);
}
// Logs: 1, 2, 3
```

See also

- Polyfill of `Iterator` in `core-js`
- es-shims polyfill of `Iterator` and associated helpers
- `function*`
- Iteration protocols

Iterator.prototype.chunks()

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`chunks()`** method of `Iterator` instances returns a new iterator helper object that splits the elements from the original iterator into consecutive array chunks. Each time the helper is iterated, it gets the specified number of elements from the underlying iterator and yields them together.

To yield overlapping sequences (i.e., sliding window), see `Iterator.prototype.windows()`.

Syntax

```
chunks(chunkSize)
```

Parameters

`chunkSize`

The number of elements in each chunk. Must be a positive integer less than 2^{32} (the maximum array length).

Return value

A new iterator helper object. Each time the returned iterator helper's `next()` method is called, the original iterator is advanced by `chunkSize` elements, and those elements are yielded together as an array.

If the original iterator has some but fewer than `chunkSize` elements remaining, those elements are still yielded as an array (so the length is less than `chunkSize`), and the iterator helper will be immediately completed the next time `next()` is called.

If the original iterator has no elements remaining, the iterator helper is immediately completed without yielding an empty array.

Exceptions

`TypeError`

Thrown if `chunkSize` is not an integer.

`RangeError`

Thrown if `chunkSize` is less than 1 or greater than $2^{32} - 1$.

Examples

Printing elements in a grid

The following example creates an iterator that yields terms in the Fibonacci sequence. Then, they are printed in a grid layout by retrieving 5 elements each time.

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const rows = fibonacci().chunks(5);

for (const row of rows.take(5)) {
  console.log(row.join("\t"));
}
```

Output:

```
1       1       2       3       5
8       13      21      34      55
89      144     233     377     610
987     1597    2584    4181    6765
10946   17711   28657   46368   75025
```

See also

- Polyfill of `Iterator.prototype.chunks` in `core-js`
- es-shims polyfill of `Iterator.prototype.chunks`
- `Iterator`
- `Iterator.prototype.windows()`

Iterator.concat()

The **`Iterator.concat()`** static method creates a new `Iterator` object from a list of iterable objects. The new iterator yields the values from each of the input iterables in sequence.

Syntax

```
Iterator.concat(it)
Iterator.concat(it1, it2)
Iterator.concat(it1, it2, /* …, */ itN)
```

Parameters

`it1, it2, ..., itN`

An object that implements the iterable protocol. Iterators which are not also iterable cannot be passed directly to this method; they must first be wrapped using `Iterator.from()`.

Return value

A new `Iterator` object that yields the values from each of the input iterables in sequence.

Description

The `Iterator.concat()` method is conceptually similar to `Array`'s `concat()` method, but it operates on any kind of iterable, and returns an iterator instead of an array. This means that the iterables can be lazily iterated, avoiding unnecessary allocation or computation. It also means that, technically, you can concatenate *infinite iterables*, but results from iterables after the first infinite iterable will never be reached.

While each iterable can be infinite, the list of iterables must be finite—and quite limited in number because engines impose a very low limit on the number of function arguments. If you need to concatenate a large—even infinite—number of iterables, use `Iterator.prototype.flatMap()` instead.

```
function* infiniteIterables() {
  for (let i = 1; ; i++) {
    yield Array(i).fill(i);
  }
}

// BAD:
// Iterator.concat(...infiniteIterables());
// The spread operator never finishes

// GOOD:
const it = infiniteIterables().flatMap((x) => x);
// Infinite sequence of numbers: 1, 2, 2, 3, 3, 3, 4, 4, 4, 4, ...
```

The `Iterator.concat()` method is similar in functionality to the following function, which uses the `yield*` operator to yield values from each of the input iterables in sequence:

```
function* concat(...iterables) {
  for (const iterable of iterables) {
    yield* iterable;
  }
}
```

Like `yield*`, `Iterator.concat()` doesn't support arguments that are not iterable (i.e., don't have the `[Symbol.iterator]()` method). This is because `Iterator.concat()` always takes ownership over its iterators and closes open iterators when the method exits. With iterable arguments, `Iterator.concat()` acquires iterators one-by-one and just closes the current iterator when the iteration is stopped. With iterator arguments, it's not clear whether the caller or `Iterator.concat()` should be responsible for closing the iterators, especially the ones that `Iterator.concat()` hasn't reached, so the method simply disallows non-iterable arguments.

Examples

Combining maps

In this example, we create a new `Map` that's the union of three other maps. The `Map()` constructor accepts an iterable of key-value pairs, while the Map iterator yields key-value pairs from the map, so just using `Iterator.concat()` on the maps is sufficient to create the new map.

```
const map1 = new Map([
  ["a", 1],
  ["b", 2],
]);
const map2 = new Map([
  ["c", 3],
  ["d", 4],
]);
const map3 = new Map([
  ["a", 5],
  ["e", 6],
]);

const map = new Map(Iterator.concat(map1, map2, map3));
console.log(map);
// Map(5) {'a' => 5, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 6}
```

Note that the value of key `"a"` in the resulting map is `5`. This is because the `Map` constructor uses the last value for each key.

Concatenating infinite iterables

When any of the input iterables is infinite, the resulting iterator will also be infinite. This isn't immediately a problem because iterators can be consumed lazily and closed at any time, but it does mean that iterables after the first infinite iterable will never be reached.

```
function* it1() {
  yield 1;
  yield 2;
}

function* it2() {
  let i = 3;
  while (true) {
    yield i++;
  }
}

function* it3() {
  yield "done";
}

const it = Iterator.concat(it1(), it2(), it3());
```

```
for (const value of it.take(10)) {
  console.log(value); // 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
}
// "done" is never reached
```

Concatenating iterables of different types

You can concatenate iterables of different types.

```
const array = [1, 2, 3];
const set = new Set([4, 5, 6]);
function* gen() {
  yield 7;
  yield 8;
  yield 9;
}

const it = Iterator.concat(array, set, gen());
console.log([...it]); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The elements of each iterable can also be of different types, just like arrays.

```
const array = [1, "two", 3];
const set = new Set([true, {}]);

const it = Iterator.concat(array, set);
console.log([...it]); // [1, "two", 3, true, {}]
```

Concatenating non-iterable objects

Non-iterable objects throw a `TypeError` when passed to `Iterator.concat()` because they don't have the `[Symbol.iterator]()` method.

```
const nonIterable = {
  next() {
    return { done: true };
  },
};

const it = Iterator.concat(nonIterable); // TypeError: object is not iterable
```

Because all built-in iterators extend the `Iterator` class, they are all iterable and can be passed to `Iterator.concat()`.

```
const arrayIterator = [1, 2, 3][Symbol.iterator]();

const it = Iterator.concat(arrayIterator); // No error
```

To pass an iterator that isn't also iterable, you can wrap it using `Iterator.from()`.

```
const nonIterable = {
  next() {
    return { done: true };
  },
};

const it = Iterator.concat(Iterator.from(nonIterable)); // No error
```

Another option is to use `Iterator.prototype.flatMap()` instead, which automatically calls `Iterator.from()`. But be careful: you need to call `flatMap()` on an iterator, not on an array, because `Array.prototype.flatMap()` only supports array return values.

```
const nonIterable = {
  next() {
    return { done: true };
  },
};

const it = [nonIterable].values().flatMap((x) => x); // No error
```

When implementing your own iterators, consider making them iterable by or subclassing `Iterator` or adding a `[Symbol.iterator]()` method that returns `this`.

See also

- Polyfill of `Iterator.concat` in `core-js`
- es-shims polyfill of `Iterator.concat`
- `Iterator`
- `Iterator.from()`
- `Array.prototype.concat()`

Iterator.prototype.drop()

The **`drop()`** method of `Iterator` instances returns a new iterator helper object that skips the given number of elements at the start of this iterator.

Syntax

```
drop(limit)
```

Parameters

`limit`

The number of elements to drop from the start of the iteration.

Return value

A new iterator helper object. The first time the returned iterator helper's `next()` method is called, the current iterator is immediately advanced by `limit` elements, and then the next element (the `limit+1`-th element) is yielded. The iterator helper then yields the remaining elements one-by-one. If the current iterator has fewer than `limit` elements, the new iterator helper will be immediately completed the first time `next()` is called.

Exceptions

`RangeError`

Thrown if `limit` becomes `NaN` or negative when converted to an integer.

Examples

Using drop()

The following example creates an iterator that yields terms in the Fibonacci sequence, starting from the 3rd term by dropping the first two terms:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().drop(2);
console.log(seq.next().value); // 2
console.log(seq.next().value); // 3
```

This is equivalent to:

```
const seq = fibonacci();
seq.next();
seq.next();
```

Using drop() with a for...of loop

`drop()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```
for (const n of fibonacci().drop(2)) {
  console.log(n);
  if (n > 30) {
    break;
  }
}

// Logs:
// 2
// 3
// 5
// 8
// 13
// 21
// 34
```

Combining drop() with take()

You can combine `drop()` with `Iterator.prototype.take()` to get a slice of an iterator:

```
for (const n of fibonacci().drop(2).take(5)) {
  // Drops the first two elements, then takes the next five
  console.log(n);
}
// Logs:
// 2
// 3
// 5
// 8
// 13

for (const n of fibonacci().take(5).drop(2)) {
  // Takes the first five elements, then drops the first two
  console.log(n);
```

```
}
// Logs:
// 2
// 3
// 5
```

Lower and upper bounds of drop count

When the `limit` is negative or `NaN`, a `RangeError` is thrown:

```
fibonacci().drop(-1); // RangeError: -1 must be positive
fibonacci().drop(undefined); // RangeError: undefined must be positive
```

When the `limit` is larger than the total number of elements the iterator can produce (such as `Infinity`), the returned iterator helper will instantly drop all elements and then be completed the first time `next()` is called. If the current iterator is infinite, the returned iterator helper will never complete.

```
fibonacci().drop(Infinity).next(); // Never ends
new Set([1, 2, 3]).values().drop(Infinity).next(); // { value: undefined, done: true }
new Set([1, 2, 3]).values().drop(4).next(); // { value: undefined, done: true }
```

See also

- Polyfill of `Iterator.prototype.drop` in `core-js`
- es-shims polyfill of `Iterator.prototype.drop`
- `Iterator`
- `Iterator.prototype.take()`

Iterator.prototype.every()

The **`every()`** method of `Iterator` instances is similar to `Array.prototype.every()`: it returns `false` if it finds an element that does not satisfy the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `true`.

Syntax

```
every(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

`true` if `callbackFn` returns a truthy value for every element. Otherwise, `false`.

Description

`every()` iterates the iterator and invokes the `callbackFn` function once for each element. It returns `false` immediately if the callback function returns a falsy value. Otherwise, it iterates until the end of the iterator and returns `true`. If `every()` returns `false`, the underlying iterator is closed by calling its `return()` method.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `every()` returns `false` as soon as the first falsy value is found. If the `callbackFn` always returns a truthy value, the method never returns.

Calling `every()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using every()

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const isEven = (x) => x % 2 === 0;
console.log(fibonacci().every(isEven)); // false

const isPositive = (x) => x > 0;
```

```
console.log(fibonacci().take(10).every(isPositive)); // true
console.log(fibonacci().every(isPositive)); // Never completes
```

The method closes the iterator after returning.

```
const seq = fibonacci();
console.log(seq.every(isEven)); // false
console.log(seq.next()); // { value: undefined, done: true }
```

See also

- Polyfill of `Iterator.prototype.every` in `core-js`
- es-shims polyfill of `Iterator.prototype.every`
- `Iterator`
- `Iterator.prototype.find()`
- `Iterator.prototype.some()`
- `Array.prototype.every()`

Iterator.prototype.filter()

The **`filter()`** method of `Iterator` instances returns a new iterator helper object that yields only those elements of the iterator for which the provided callback function returns `true`.

Syntax

```
filter(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to make the element yielded by the iterator helper, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

A new iterator helper object. Each time the iterator helper's `next()` method is called, it returns the next element in the iterator for which the callback function returns `true`. When the underlying iterator is completed, the iterator helper object is also completed (the `next()` method produces `{ value: undefined, done: true }`).

Description

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators.

Examples

Using filter()

The following example creates an iterator that yields terms in the Fibonacci sequence, and then reads the first few terms that are even:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().filter((x) => x % 2 === 0);
console.log(seq.next().value); // 2
console.log(seq.next().value); // 8
console.log(seq.next().value); // 34
```

Using filter() with a for...of loop

`filter()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```
for (const n of fibonacci().filter((x) => x % 2 === 0)) {
  console.log(n);
  if (n > 30) {
    break;
  }
}
```

```
// Logs:
// 2
// 8
// 34
```

This is equivalent to:

```
for (const n of fibonacci()) {
  if (n % 2 !== 0) {
    continue;
  }
  console.log(n);
  if (n > 30) {
    break;
  }
}
```

See also

- Polyfill of `Iterator.prototype.filter` in `core-js`
- es-shims polyfill of `Iterator.prototype.filter`
- `Iterator`
- `Iterator.prototype.forEach()`
- `Iterator.prototype.every()`
- `Iterator.prototype.map()`
- `Iterator.prototype.some()`
- `Iterator.prototype.reduce()`
- `Array.prototype.filter()`

Iterator.prototype.find()

The **`find()`** method of `Iterator` instances is similar to `Array.prototype.find()`: it returns the first element produced by the iterator that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

Syntax

```
find(callbackFn)
```

Parameters

`callbackFn`

> A function to execute for each element produced by the iterator. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:
>
> `element`
>
> > The current element being processed.
>
> `index`
>
> > The index of the current element being processed.

Return value

The first element produced by the iterator that satisfies the provided testing function. Otherwise, `undefined` is returned.

Description

`find()` iterates the iterator and invokes the `callbackFn` function once for each element. It returns the element immediately if the callback function returns a truthy value. Otherwise, it iterates until the end of the iterator and returns `undefined`. If `find()` returns an element, the underlying iterator is closed by calling its `return()` method.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `find()` returns the first satisfying element as soon as it is found. If the `callbackFn` always returns a falsy value, the method never returns.

Calling `find()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using find()

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}
```

```
const isEven = (x) => x % 2 === 0;
console.log(fibonacci().find(isEven)); // 2

const isNegative = (x) => x < 0;
console.log(fibonacci().take(10).find(isNegative)); // undefined
console.log(fibonacci().find(isNegative)); // Never completes
```

The method closes the iterator after returning.

```
const seq = fibonacci();
console.log(seq.find(isEven)); // 2
console.log(seq.next()); // { value: undefined, done: true }
```

See also

- Polyfill of `Iterator.prototype.find` in `core-js`
- es-shims polyfill of `Iterator.prototype.find`
- `Iterator`
- `Iterator.prototype.every()`
- `Iterator.prototype.some()`
- `Array.prototype.find()`

Iterator.prototype.flatMap()

The **`flatMap()`** method of `Iterator` instances returns a new iterator helper object that takes each element in the original iterator, runs it through a mapping function, and yields elements returned by the mapping function (which are contained in another iterator or iterable).

Syntax

```
flatMap(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return an iterator or iterable that yields elements to be yielded by `flatMap()`. Note that unlike `Array.prototype.flatMap()`, you cannot return single non-iterator/iterable values. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

Return value

A new iterator helper object. The first time the iterator helper's `next()` method is called, it calls `callbackFn` on the first element produced by the underlying iterator, and the return value, which should be an iterator or iterable, is yielded one-by-one by the iterator helper (like `yield*`). The next element is fetched from the underlying iterator when the previous one returned by `callbackFn` is completed. When the underlying iterator is completed, the iterator helper is also completed (the `next()` method produces `{ value: undefined, done: true }`).

Exceptions

`TypeError`

Thrown if `callbackFn` returns a non-iterator/iterable value or a string primitive.

Description

`flatMap` accepts two kinds of return values from `callbackFn`: an iterator or iterable. They are handled in the same way as `Iterator.from()`: if the return value is iterable, the `[Symbol.iterator]()` method is called and the return value is used; otherwise, the return value is treated as an iterator and its `next()` method is called.

```
[1, 2, 3]
  .values()
  .flatMap((x) => {
    let itDone = false;
    const it = {
      next() {
        if (itDone) {
          return { value: undefined, done: true };
        }
        itDone = true;
        return { value: x, done: false };
      },
    };
    switch (x) {
      case 1:
        // An iterable that's not an iterator
        return { [Symbol.iterator]: () => it };
      case 2:
```

```
        // An iterator that's not an iterable
        return it;
      case 3:
        // An iterable iterator is treated as an iterable
        return {
          ...it,
          [Symbol.iterator]() {
            console.log("Symbol.iterator called");
            return it;
          },
        };
      default:
        return undefined;
    }
  })
  .toArray();
// Logs "Symbol.iterator called"
// Returns [1, 2, 3]
```

Examples

Merging maps

The following example merges two `Map` objects into one:

```
const map1 = new Map([
  ["a", 1],
  ["b", 2],
  ["c", 3],
]);
const map2 = new Map([
  ["d", 4],
  ["e", 5],
  ["f", 6],
]);

const merged = new Map([map1, map2].values().flatMap((x) => x));
console.log(merged.get("a")); // 1
console.log(merged.get("e")); // 5
```

This avoids creating any temporary copies of the map's content. Note that the array `[map1, map2]` must first be converted to an iterator (using `Array.prototype.values()`), because `Array.prototype.flatMap()` only flattens arrays, not iterables.

```
new Map([map1, map2].flatMap((x) => x)); // Map(1) {undefined => undefined}
```

Returning strings

Strings are iterable, but `flatMap()` specifically rejects string primitives returned from `callbackFn`, this is because the behavior of iterating by code points is often not what you want.

```
[1, 2, 3]
  .values()
  .flatMap((x) => String(x))
  .toArray(); // TypeError: Iterator.prototype.flatMap called on non-object
```

You may want to wrap it in an array instead so the entire string is yielded as one:

```
[1, 2, 3]
  .values()
  .flatMap((x) => [String(x)])
  .toArray(); // ['1', '2', '3']
```

Or, if the behavior of iterating by code points is intended, you can use `Iterator.from()` to convert it to a proper iterator:

```
[1, 2, 3]
  .values()
  .flatMap((x) => Iterator.from(String(x * 10)))
  .toArray();
// ['1', '0', '2', '0', '3', '0']
```

See also

- Polyfill of `Iterator.prototype.flatMap` in `core-js`
- es-shims polyfill of `Iterator.prototype.flatMap`

Iterator.prototype.forEach()

The **`forEach()`** method of `Iterator` instances is similar to `Array.prototype.forEach()`: it executes a provided function once for each element produced by the iterator.

Syntax

```
forEach(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. Its return value is discarded. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

`undefined`.

Description

`forEach()` iterates the iterator and invokes the `callbackFn` function once for each element. Unlike most other iterator helper methods, it does not work with infinite iterators, because it is not lazy.

Examples

Using forEach()

```
new Set([1, 2, 3]).values().forEach((v) => console.log(v));

// Logs:
// 1
// 2
// 3
```

This is equivalent to:

```
for (const v of new Set([1, 2, 3]).values()) {
  console.log(v);
}
```

See also

- Polyfill of `Iterator.prototype.forEach` in `core-js`
- es-shims polyfill of `Iterator.prototype.forEach`
- `Iterator`
- `Iterator.prototype.find()`
- `Iterator.prototype.map()`
- `Iterator.prototype.filter()`
- `Iterator.prototype.every()`
- `Iterator.prototype.some()`
- `Array.prototype.forEach()`

Iterator.from()

The **`Iterator.from()`** static method creates a new `Iterator` object from an iterator or iterable object.

Syntax

```
Iterator.from(object)
```

Parameters

`object`

An object that implements the iterable protocol or the iterator protocol.

Return value

If `object` is an iterable, its `[Symbol.iterator]()` method is called to obtain the iterator. Otherwise, `object` is assumed to be an iterator. If the iterator is already `instanceof Iterator` (which means it has `Iterator.prototype` in its prototype chain), it is returned directly. Otherwise, a new `Iterator` object is created that wraps the original iterator.

Description

This method exists to convert custom iterators, probably exported by libraries, to proper iterators. All iterator objects returned by `Iterator.from()` inherit from a common prototype object, which has the following methods:

`next()`

Calls the underlying iterator's `next()` method and returns the result.

`return()`

Calls the underlying iterator's `return()` method and returns the result, or returns `{ value: undefined, done: true }` if the underlying iterator doesn't have a `return()` method.

Examples

Converting an iterable to a proper iterator

Because `obj` is already an iterable that returns a proper iterator when its `[Symbol.iterator]()` method is called, `Iterator.from(obj)` returns the same iterator.

```
const iterator = (function* () {
  yield 1;
  yield 2;
  yield 3;
})();

const obj = {
  [Symbol.iterator]() {
    return iterator;
  },
};

const iterator2 = Iterator.from(obj);
console.log(iterator2 === iterator); // true
```

Because `obj2` is an iterable that returns a non-proper iterator when its `[Symbol.iterator]()` method is called, `Iterator.from(obj2)` returns a new iterator that wraps the original iterator.

```
const iterator = {
  current: 0,
  next() {
    return { value: this.current++, done: false };
  },
};

const obj2 = {
  [Symbol.iterator]() {
    return iterator;
  },
};

const iterator2 = Iterator.from(obj2);
console.log(iterator2 === iterator); // false
console.log(iterator2.next()); // { value: 0, done: false }
console.log(iterator.next()); // { value: 1, done: false }
```

Converting an iterator to a proper iterator

Because `obj` is already a proper iterator, `Iterator.from(obj)` returns itself.

```
const obj = (function* () {
  yield 1;
  yield 2;
  yield 3;
})();

const iterator = Iterator.from(obj);
console.log(iterator === obj); // true
```

Because `obj2` is a non-proper iterator, `Iterator.from(obj2)` returns a new iterator that wraps the original iterator.

```
const obj2 = {
  current: 0,
  next() {
    return { value: this.current++, done: false };
  },
};

const iterator = Iterator.from(obj2);
console.log(iterator === obj2); // false
console.log(iterator.next()); // { value: 0, done: false }
console.log(obj2.next()); // { value: 1, done: false }
```

See also

- Polyfill of `Iterator.from` in `core-js`
- es-shims polyfill of `Iterator.from`
- `Iterator`
- `Array.from()`

Iterator.prototype.includes()

The **`includes()`** method of `Iterator` instances is similar to `Array.prototype.includes()`: it returns `true` if it finds an element that is equal to the given value. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

Syntax

```
includes(searchElement)
includes(searchElement, fromIndex)
```

Parameters

`searchElement`

The value to search for.

`fromIndex` Optional

Zero-based index at which to start searching. Must be a non-negative integer, `Infinity`, or `undefined`. If `fromIndex` is greater than or equal to the number of elements produced by the iterator (including when `fromIndex` is `Infinity`), the method always returns `false` after exhausting the iterator. If `fromIndex` is `undefined`, it defaults to `0`.

Return value

A boolean value which is `true` if the value `searchElement` is found within the iterator (or the part of the iterator starting at `fromIndex`, if specified).

Exceptions

`TypeError`

Thrown if `fromIndex` is not one of: an integer, `Infinity`, `-Infinity`, or `undefined`.

`RangeError`

Thrown if `fromIndex` is negative.

Description

The `includes()` method compares `searchElement` to elements of the array using the SameValueZero algorithm. This algorithm works like strict equality `===` (where `-0` and `+0` are considered equal), with the exception that `NaN` is considered equal to itself.

Unlike `Array.prototype.includes()`, the `fromIndex` parameter of `Iterator.prototype.includes()` is not allowed to be negative, because the iterator does not have a known length. The type validation is also stricter: non-integer values are not truncated to integers.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `includes()` returns `true` as soon as the first match is found. If the value is never encountered, the method never returns.

Calling `includes()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using includes()

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(fibonacci().includes(8)); // true
console.log(fibonacci().take(10).includes(7)); // false
console.log(fibonacci().includes(7)); // Never completes
```

The method closes the iterator after returning.

```
const seq = fibonacci();
console.log(seq.includes(8)); // true
console.log(seq.next()); // { value: undefined, done: true }
```

Using fromIndex

`fromIndex` specifies the number of elements to be skipped from the beginning. It is equivalent to calling `drop(fromIndex).includes(searchElement)`.

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(fibonacci().includes(8, 3)); // true
console.log(fibonacci().includes(8, 10)); // false
```

If `fromIndex` is greater than or equal to the elements available, `false` is returned.

```
["a", "b", "c"].values().includes("a", 3); // false
```

See also

- es-shims polyfill of `Iterator.prototype.includes`
- `Iterator`
- `Iterator.prototype.every()`
- `Iterator.prototype.find()`
- `Iterator.prototype.some()`
- `Array.prototype.includes()`

Iterator() constructor

The **`Iterator()`** constructor is intended to be used as the superclass of other classes that create iterators. It throws an error when constructed by itself.

Syntax

```
new Iterator()
```

> **Note:** `Iterator()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`. In addition, `Iterator()` cannot actually be constructed itself — it's usually implicitly constructed through `super()` calls inside the constructor of a subclass.

Parameters

None.

Return value

A new `Iterator` object.

Exceptions

`TypeError`

When `new.target` is the `Iterator` function itself, i.e., when the `Iterator` constructor itself is constructed.

Description

`Iterator` represents an *abstract class* — a class that provides common utilities for its subclasses, but is not intended to be instantiated itself. It is the superclass of all other iterator classes, and is used to create subclasses that implement specific iteration algorithms — namely, all subclasses of `Iterator` must implement a `next()` method as required by the iterator protocol. Because `Iterator` doesn't actually provide the `next()` method, it doesn't make sense to construct an `Iterator` directly.

You can also use `Iterator.from()` to create an `Iterator` instance from an existing iterable or iterator object.

Examples

Subclassing Iterator

The following example defines a custom data structure, `Range`, which allows iteration. To make an object iterable, we can provide a `[Symbol.iterator]()` method in the form of a generator function:

```
class Range {
  #start;
  #end;
  #step;

  constructor(start, end, step = 1) {
    this.#start = start;
    this.#end = end;
    this.#step = step;
  }

  *[Symbol.iterator]() {
    for (let value = this.#start; value <= this.#end; value += this.#step) {
      yield value;
    }
  }
}

const range = new Range(1, 5);
for (const num of range) {
  console.log(num);
}
```

This works, but it isn't as nice as how built-in iterators work. There are two problems:

- The returned iterator inherits from `Generator`, which means modifications to `Generator.prototype` are going to affect the returned iterator, which is a leak of abstraction.
- The returned iterator does not inherit from a custom prototype, which makes it harder if we intend to add extra methods to the iterator.

We can mimic the implementation of built-in iterators, such as map iterators, by subclassing `Iterator`. This enables us to define extra properties, such as `[Symbol.toStringTag]`, while making the iterator helper methods available on the returned iterator.

```
class Range {
  #start;
  #end;
  #step;

  constructor(start, end, step = 1) {
    this.#start = start;
    this.#end = end;
    this.#step = step;
  }

  static #RangeIterator = class extends Iterator {
    #cur;
    #s;
    #e;
    constructor(range) {
      super();
      this.#cur = range.#start;
      this.#s = range.#step;
      this.#e = range.#end;
    }
    static {
      Object.defineProperty(this.prototype, Symbol.toStringTag, {
        value: "Range Iterator",
        configurable: true,
        enumerable: false,
        writable: false,
      });

      // Avoid #RangeIterator from being accessible outside
      delete this.prototype.constructor;
    }
    next() {
      if (this.#cur > this.#e) {
        return { value: undefined, done: true };
      }
      const res = { value: this.#cur, done: false };
      this.#cur += this.#s;
      return res;
    }
  };

  [Symbol.iterator]() {
    return new Range.#RangeIterator(this);
  }
}

const range = new Range(1, 5);
for (const num of range) {
  console.log(num);
}
```

The subclassing pattern is useful if you want to create many custom iterators. If you have an existing iterable or iterator object which doesn't inherit from `Iterator`, and you just want to call iterator helper methods on it, you can use `Iterator.from()` to create a one-time `Iterator` instance.

See also

- Polyfill of `Iterator` in `core-js`
- es-shims polyfill of `Iterator` and associated helpers
- `Iterator`
- `Iterator.from()`

Iterator.prototype.join()

The **`join()`** method of `Iterator` instances is similar to `Array.prototype.join()`: it returns a string that is the concatenation of all elements produced by the iterator, separated by commas or a specified separator string. If the iterator has only one item, that item's stringification is returned without using the separator.

Syntax

```
join()
join(separator)
```

Parameters

`separator` Optional

A string to separate each pair of adjacent elements of the iterator. If omitted, the elements are separated with a comma (",").

Return value

A string joining all yielded elements. The elements are converted to strings. If an element is `undefined` or `null`, it is converted to an empty string instead of the string `"null"` or `"undefined"`. If the iterator is empty, the empty string is returned.

Description

See `Array.prototype.join()` for details about how `join()` works. Unlike most other iterator helper methods, it does not work well with infinite iterators, because it is not lazy.

Examples

Using join()

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(fibonacci().take(5).join()); // "1,1,2,3,5"
console.log(fibonacci().take(5).join(" - ")); // "1 - 1 - 2 - 3 - 5"
```

See also

- Polyfill of `Iterator.prototype.join` in `core-js`
- es-shims polyfill of `Iterator.prototype.join`
- `Iterator`
- `Iterator.prototype.reduce()`
- `Array.prototype.join()`

Iterator.prototype.map()

The **`map()`** method of `Iterator` instances returns a new iterator helper object that yields elements of the iterator, each transformed by a mapping function.

Syntax

```
map(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. Its return value is yielded by the iterator helper. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

A new iterator helper object. Each time the iterator helper's `next()` method is called, it gets the next element from the underlying iterator, applies `callbackFn`, and yields the return value. When the underlying iterator is completed, the iterator helper is also completed (the `next()` method produces `{ value: undefined, done: true }`).

Description

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. The `map()` method allows you to create a new iterator that, when iterated, produces transformed elements.

Examples

Using map()

The following example creates an iterator that yields terms in the Fibonacci sequence, transforms it into a new sequence with each term squared, and then reads the first few terms:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}
```

```
const seq = fibonacci().map((x) => x ** 2);
console.log(seq.next().value); // 1
console.log(seq.next().value); // 1
console.log(seq.next().value); // 4
```

Using map() with a for...of loop

`map()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```
for (const n of fibonacci().map((x) => x ** 2)) {
  console.log(n);
  if (n > 30) {
    break;
  }
}

// Logs:
// 1
// 1
// 4
// 9
// 25
// 64
```

This is equivalent to:

```
for (const n of fibonacci()) {
  const n2 = n ** 2;
  console.log(n2);
  if (n2 > 30) {
    break;
  }
}
```

See also

- Polyfill of `Iterator.prototype.map` in `core-js`
- es-shims polyfill of `Iterator.prototype.map`
- `Iterator`
- `Iterator.prototype.flatMap()`
- `Array.prototype.reduce()`

Iterator.prototype.reduce()

The **`reduce()`** method of `Iterator` instances is similar to `Array.prototype.reduce`: it executes a user-supplied "reducer" callback function on each element produced by the iterator, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements is a single value.

Syntax

```
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduce()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is the first element of the iterator.

`currentValue`

The value of the current element. On the first call, its value is the first element of the iterator if `initialValue` is specified; otherwise its value is the second element.

`currentIndex`

The index position of `currentValue`. On the first call, its value is `0` if `initialValue` is specified, otherwise `1`.

`initialValue` Optional

A value to which `accumulator` is initialized the first time the callback is called. If `initialValue` is specified, `callbackFn` starts executing with the first element as `currentValue`. If `initialValue` is *not* specified, `accumulator` is initialized to the first element, and `callbackFn` starts executing with the second element as `currentValue`. In this case, if the iterator is empty (so that there's no first value to return as `accumulator`), an error is thrown.

Return value

The value that results from running the "reducer" callback function to completion over the entire iterator.

Exceptions

`TypeError`

Thrown if the iterator contains no elements and `initialValue` is not provided.

Description

See `Array.prototype.reduce()` for details about how `reduce()` works. Unlike most other iterator helper methods, it does not work well with infinite iterators, because it is not lazy.

Examples

Using reduce()

The following example creates an iterator that yields terms in the Fibonacci sequence, and then sums the first ten terms:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(
  fibonacci()
    .take(10)
    .reduce((a, b) => a + b),
); // 143
```

See also

- Polyfill of `Iterator.prototype.reduce` in `core-js`
- es-shims polyfill of `Iterator.prototype.reduce`
- `Iterator`
- `Iterator.prototype.map()`
- `Iterator.prototype.flatMap()`
- `Array.prototype.reduce()`

Iterator.prototype.some()

The **`some()`** method of `Iterator` instances is similar to `Array.prototype.some()`: it returns `true` if it finds an element that satisfies the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

Syntax

```
some(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

`true` if the callback function returns a truthy value for at least one element. Otherwise, `false`.

Description

`some()` iterates the iterator and invokes the `callbackFn` function once for each element. It returns `true` immediately if the callback function returns a truthy value. Otherwise, it iterates until the end of the iterator and returns `false`. If `some()` returns `true`, the underlying iterator is closed by calling its `return()` method.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `some()` returns `true` as soon as the first truthy value is found. If the `callbackFn` always returns a falsy value, the method never returns.

Calling `some()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using some()

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const isEven = (x) => x % 2 === 0;
console.log(fibonacci().some(isEven)); // true

const isNegative = (x) => x < 0;
console.log(fibonacci().take(10).some(isNegative)); // false
console.log(fibonacci().some(isNegative)); // Never completes
```

The method closes the iterator after returning.

```
const seq = fibonacci();
console.log(seq.some(isEven)); // true
console.log(seq.next()); // { value: undefined, done: true }
```

See also

- Polyfill of `Iterator.prototype.some` in `core-js`
- es-shims polyfill of `Iterator.prototype.some`
- `Iterator`
- `Iterator.prototype.every()`
- `Iterator.prototype.find()`
- `Array.prototype.some()`

Iterator.prototype[Symbol.dispose]()

The **`[Symbol.dispose]()`** method of `Iterator` instances implements the *disposable protocol* and allows it to be disposed when used with `using`. It calls the `return()` method of `this`, if it exists.

Syntax

```
iterator[Symbol.dispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring an iterator with <code>using</code>

The `Symbol.dispose` method is intended to be automatically called in a `using` declaration. This is useful if you have an iterator that you manually iterate over by calling its `next()` method; if you iterate it with `for...of` or something similar, then error handling and cleanup is done automatically.

```
function* generateNumbers() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("Cleaning up");
  }
}

function doSomething() {
  using numbers = generateNumbers();
  const res1 = numbers.next();
  // Not iterating the rest of the numbers
  // Before the function exits, the async iterator is disposed
  // Logs "Cleaning up"
}

doSomething();
```

See also

- Polyfill of `Iterator.prototype[Symbol.dispose]` in `core-js`
- JavaScript resource management
- `Symbol.dispose`
- `using`

Iterator.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `Iterator` instances implements the iterable protocol and allows built-in iterators to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns the value of `this`, which is the iterator object itself.

Syntax

```
iterator[Symbol.iterator]()
```

Parameters

None.

Return value

The value of `this`, which is the iterator object itself.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes built-in iterators iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const arrIterator = [1, 2, 3].values();
for (const value of arrIterator) {
  console.log(value);
}
// Logs: 1, 2, 3
```

See also

- `Iterator`
- `Symbol.iterator`
- Iteration protocols

Iterator.prototype.take()

The **`take()`** method of `Iterator` instances returns a new iterator helper object that yields the given number of elements in this iterator and then terminates.

Syntax

```
take(limit)
```

Parameters

`limit`

The number of elements to take from the start of the iteration.

Return value

A new iterator helper object. The returned iterator helper yields the elements in the original iterator one-by-one, and then completes (the `next()` method produces `{ value: undefined, done: true }`) once `limit` elements have been yielded, or when the original iterator is exhausted, whichever comes first.

Exceptions

`RangeError`

Thrown if `limit` becomes `NaN` or negative when converted to an integer.

Examples

Using take()

The following example creates an iterator that yields terms in the Fibonacci sequence, and then logs the first three terms:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().take(3);
console.log(seq.next().value); // 1
console.log(seq.next().value); // 1
console.log(seq.next().value); // 2
console.log(seq.next().value); // undefined
```

Using take() with a for...of loop

`take()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```
for (const n of fibonacci().take(5)) {
  console.log(n);
}

// Logs:
// 1
// 1
// 2
// 3
// 5
```

Because `fibonacci()` is an infinite iterator, using a `for` loop to iterate it without any logic to exit early (such as a `break` statement) would result in an infinite loop.

Combining drop() with take()

You can combine `take()` with `Iterator.prototype.drop()` to get a slice of an iterator:

```
for (const n of fibonacci().drop(2).take(5)) {
  // Drops the first two elements, then takes the next five
  console.log(n);
}
// Logs:
// 2
// 3
// 5
// 8
// 13

for (const n of fibonacci().take(5).drop(2)) {
  // Takes the first five elements, then drops the first two
  console.log(n);
}
// Logs:
// 2
// 3
// 5
```

Lower and upper bounds of take count

When the `limit` is negative or `NaN`, a `RangeError` is thrown:

```
fibonacci().take(-1); // RangeError: -1 must be positive
fibonacci().take(undefined); // RangeError: undefined must be positive
```

When the `limit` is larger than the total number of elements the iterator can produce (such as `Infinity`), the returned iterator helper has essentially the same behavior as the original iterator:

```
for (const n of new Set([1, 2, 3]).values().take(Infinity)) {
  console.log(n);
}

// Logs:
// 1
// 2
// 3
```

See also

- Polyfill of `Iterator.prototype.take` in `core-js`
- es-shims polyfill of `Iterator.prototype.take`
- `Iterator`
- `Iterator.prototype.drop()`

Iterator.prototype.toArray()

The **`toArray()`** method of `Iterator` instances creates a new `Array` instance populated with the elements yielded from the iterator.

Syntax

```
toArray()
```

Parameters

None.

Return value

A new `Array` instance containing the elements from the iterator in the order they were produced.

Examples

Using toArray()

`iterator.toArray()` is equivalent to `Array.from(iterator)` and `[...iterator]`, except that it's easier to chain when multiple iterator helper methods are involved. The following example creates an iterator that yields terms in the Fibonacci sequence, takes the first 10 terms, filters out the odd numbers, and converts the result to an array:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const array = fibonacci()
  .take(10)
  .filter((x) => x % 2 === 0)
  .toArray();

console.log(array); // [2, 8, 34]
```

Note that it's a good idea to call `toArray()` as a last step of your processing. For example, `fibonacci().take(10).toArray().filter(...)` is less efficient, because iterator helpers are lazy and avoids creating a temporary array.

See also

- Polyfill of `Iterator.prototype.toArray` in `core-js`
- es-shims polyfill of `Iterator.prototype.toArray`
- `Iterator`
- `Array.from()`

Iterator.prototype.windows()

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`windows()`** method of `Iterator` instances returns a new iterator helper object that yields a sliding window of elements. Each time the helper is iterated, it yields an array removing the first element from the previous iteration and adding the next element from the original iterator.

To yield distinct chunks, see `Iterator.prototype.chunks()`.

Syntax

```
windows(windowSize)
windows(windowSize, underSized)
```

Parameters

`windowSize`

The number of elements in the sliding window. Must be a positive integer less than 2^{32} (the maximum array length).

`underSized` Optional

A string specifying what to do when the number of elements in the iterator is less than `windowSize` but greater than 0. It can be one of the following:

`"only-full"` (default)

Ignore the partial window. The returned iterator immediately completes as if the input is empty.

`"allow-partial"`

Yield all elements in the partial window as an array (with length less than `windowSize`).

Note that when the input is empty, the returned iterator always immediately completes without yielding an empty array.

Return value

A new iterator helper object. The first time the returned iterator helper's `next()` method is called, the current iterator is immediately advanced by `windowSize` elements. If that many elements are available, they are yielded together as an array. Otherwise, the behavior depends on the `underSized` argument: the iterator either immediately completes without yielding anything (with `"only-full"` or with a fully empty input), or an array of length less than `windowSize` is yielded.

Afterwards, each time, the yielded array consists of the previous array, the leftmost element removed, a new element retrieved from the input iterable appended. When the input iterable is exhausted, the iterator helper also completes.

Exceptions

`TypeError`

Thrown if `limit` is not an integer, or if `underSized` is not one of the allowed values.

`RangeError`

Thrown if `limit` is less than 1 or greater than 2^{32} - 1.

Examples

Moving average

The following example creates an iterator that yields terms in the Fibonacci sequence. Then, a new iterator is created, each time yielding the average of two adjacent elements.

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const averaged = fibonacci()
  .windows(2)
  .map((window) => (window[0] + window[1]) / 2);

console.log(averaged.next().value); // 1 (1, 1)
console.log(averaged.next().value); // 1.5 (1, 2)
console.log(averaged.next().value); // 2.5 (2, 3)
console.log(averaged.next().value); // 4 (3, 5)
```

This process, known as a moving average, is very useful in signal processing.

Enumerating substrings

The following example enumerates all substrings of a given length in a string. The `windows()` method is used to create a sliding window of characters, which are then joined together to form substrings.

> **Note:** This is probably much less efficient than manually iterating a number index and calling the `String.prototype.substring()` method.

```
const chars = Iterator.from("MASSACHUSETTS");

const substrings = chars.windows(3).map((window) => window.join(""));

for (const substring of substrings) {
  console.log(substring);
}
// MAS
// ASS
// SSA
// ...
```

See also

- Polyfill of `Iterator.prototype.windows` in `core-js`
- es-shims polyfill of `Iterator.prototype.windows`
- `Iterator`
- `Iterator.prototype.chunks()`

Iterator.zip()

The **`Iterator.zip()`** static method creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding arrays containing elements at the same position. It essentially "zips" the input iterables together, allowing simultaneous iteration over them.

The `Iterator.zipKeyed()` method is similar, but yields objects instead of arrays with keys you can specify.

Syntax

```
Iterator.zip(iterables)
Iterator.zip(iterables, options)
```

Parameters

`iterables`

An iterable of iterables whose elements are aggregated. It must be iterable and cannot be an iterator. It should be finite, although its elements can be infinite iterables. Each element must implement either the iterable protocol or, failing that, the iterator protocol. Strings are rejected: to zip strings, convert them to iterators explicitly using `Iterator.from()`.

`options` Optional

An object specifying behavior in case of inconsistent input lengths. It may have the following properties:

`mode` Optional

One of the following:

- `"shortest"` (default): The resulting iterator stops when one input iterable is exhausted.
- `"longest"`: The resulting iterator stops when all input iterables are exhausted. Missing values from shorter iterables are filled according to the `padding` option.
- `"strict"`: A `TypeError` is thrown if not all input iterables finish at the same time.

`padding` Optional

An iterable object (not iterator). Only retrieved and validated when `mode` is `"longest"`. If `undefined` or absent, missing values from shorter iterables are filled with `undefined` (which is equivalent to passing an empty iterable). If an iterable is provided, it is iterated for the number of times equal to the number of elements in `iterables` *as soon as* `Iterator.zip()` *is called*. `padding[i]` is used for missing values for `iterables[i]` (assuming `padding` and `iterables` are provided as arrays; they don't have to be). If `padding` is shorter than `iterables`, `undefined` is used for the remaining iterables.

Return value

A new `Iterator` object. Each of its elements is an array with length equal to the number of input iterables, containing the elements from each input iterable at the corresponding position. If the `iterables` object is empty, the resulting iterator is created as completed.

Description

The `Iterator.zip()` function behaves like a transpose operation, yielding arrays containing the elements at matching positions in each of the inputs. If we represent iterables as arrays, the input may looks like this:

```
[
  [a1, a2, a3, a4], // Iterable a
  [b1, b2, b3], // Iterable b
  [c1, c2, c3, c4, c5], // Iterable c
];
```

The resulting iterator, regardless of the options, will start by yielding the following arrays:

```
[a1, b1, c1];
[a2, b2, c2];
[a3, b3, c3];
```

After the first three arrays are yielded, the input iterable `b` is exhausted on the fourth `next()` call—it returns `{ done: true }`. What happens next depends on the `mode` option. If `mode` is `"shortest"` (the default), the resulting iterator stops here: the other two input iterators are closed. If `mode` is `"strict"`, an error is thrown because the other two iterables are *not* finished when the second one yields the result `{ done: true }`. If `mode` is `"longest"`, the resulting iterator continues yielding arrays, filling missing values. For example, if `padding` is not provided, it defaults to `undefined`:

```
[a4, undefined, c4];
[undefined, undefined, c5];
```

If `padding` is provided as an iterable, because there are three input iterables, the first three values from the `padding` iterable are used to fill missing values. Let's suppose that `padding` is an array with values `[p1, p2, p3]`. Then `p2` is used to fill the missing value from the input iterable `b`, and `p1` is used to fill the missing value from the input iterable `a`:

```
[a4, p2, c4];
[p1, p2, c5];
```

If the `padding` iterable has fewer than three values, the remaining missing values are filled with `undefined`.

Examples

Iteration over a map with indices

Using `Iterator.zip()`, you can iterate over any iterable object (strings aren't supported by default) while also having access to an incrementing counter:

```
const ages = new Map([
  ["Caroline", 30],
  ["Danielle", 25],
  ["Evelyn", 35],
]);

const numbers = (function* () {
  let n = 0;
  while (true) {
    yield n++;
  }
})();
for (const [index, [name, age]] of Iterator.zip([numbers, ages])) {
  console.log(`${index}: ${name} is ${age} years old.`);
}

// Output:
// 0: Caroline is 30 years old.
// 1: Danielle is 25 years old.
// 2: Evelyn is 35 years old.
```

`numbers` is an infinite iterator that generates incrementing numbers starting from `0`. Because `Iterator.zip()` stops by default when the shortest input iterable is exhausted, the loop iterates exactly three times. The `numbers` iterator is properly closed after the loop ends; it doesn't cause an infinite loop.

Creating a Map from lists of keys and values

Suppose you have two arrays: one with keys and another with values. You can use `Iterator.zip()` to combine them into a `Map`:

```
const days = ["Mon", "Tue", "Wed", "Thu", "Fri"];
const temperatures = [22, 21, 23, 20, 19];

const dayTemperatureMap = new Map(Iterator.zip([days, temperatures]));
console.log(dayTemperatureMap);
// Map(5) { 'Mon' => 22, 'Tue' => 21, 'Wed' => 23, 'Thu' => 20, 'Fri' => 19 }
```

Joint iteration over multiple data sources

Suppose you have data coming from multiple sources, such as multiple microservices or databases. You know that each source provides related data in the same order, and you want to process them together. You can use `Iterator.zip()` to achieve this:

```
const names = fetchNames(); // e.g., ["Caroline", "Danielle", "Evelyn"]
const ages = fetchAges(); // e.g., [30, 25, 35]
const cities = fetchCities(); // e.g., ["New York", "London", "Hong Kong"]

for (const [name, age, city] of Iterator.zip([names, ages, cities])) {
  console.log(`${name}, aged ${age}, lives in ${city}.`);
}

// Output:
// Caroline, aged 30, lives in New York.
// Danielle, aged 25, lives in London.
// Evelyn, aged 35, lives in Hong Kong.
```

Providing padding for uneven iterables

When zipping iterables of different lengths with `mode` set to `"longest"`, you can provide a `padding` iterable to specify the values used to fill in missing entries:

```
const letters = ["a", "b", "c", "d", "e"];
const numbers = [1, 2, 3];

// One padding value per iterable
const padding = ["[Letter missing]", "[Number missing]"];
const it = Iterator.zip([letters, numbers], { mode: "longest", padding });
for (const [letter, number] of it) {
  console.log(`${letter}: ${number}`);
}
// Output:
// a: 1
// b: 2
// c: 3
// d: [Number missing]
// e: [Number missing]
```

Zipping strings

Strings are not accepted as input iterables to `Iterator.zip()`, because it is now considered a mistake to make strings implicitly iterable. To zip strings, convert them to iterators explicitly using `Iterator.from()`:

```
const str1 = "abc";
const str2 = "1234";
const it = Iterator.zip([Iterator.from(str1), Iterator.from(str2)]);
for (const [char1, char2] of it) {
  console.log(`${char1} - ${char2}`);
}
// Output:
// a - 1
// b - 2
// c - 3
```

In some cases, you may wish to split by grapheme clusters instead of code units. In that case, you can use the `Intl.Segmenter` API:

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "grapheme" });
const str1 = "🤷🏼‍♀️🤷🏽‍♀️🤷🏾‍♀️";
const str2 = "123";
const it = Iterator.zip([
  segmenter.segment(str1).map(({ segment }) => segment),
  segmenter.segment(str2).map(({ segment }) => segment),
]);
for (const [char1, char2] of it) {
  console.log(`${char1} - ${char2}`);
}
// Output:
// 🤷🏼‍♀️ - 1
```

```
// 🤷 - 2
// 🤷 - 3
```

See also

- Polyfill of `Iterator.zip` in `core-js`
- es-shims polyfill of `Iterator.zip`
- `Iterator`
- `Iterator.zipKeyed()`
- `Iterator.from()`
- `Iterator.concat()`

Iterator.zipKeyed()

The **`Iterator.zipKeyed()`** static method creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding objects containing elements at the same position, with keys specified by the input. It essentially "zips" the input iterables together, allowing simultaneous iteration over them.

The `Iterator.zip()` method is similar, but yields arrays instead of objects.

Syntax

```
Iterator.zipKeyed(iterables)
Iterator.zipKeyed(iterables, options)
```

Parameters

`iterables`

> An object. Each property's key is used as the key in the resulting objects. The property's value must implement either the iterable protocol or, failing that, the iterator protocol. These iterables may be infinite. Strings are rejected: to zip strings, convert them to iterators explicitly using `Iterator.from()`.

`options` Optional

> An object specifying behavior in case of inconsistent input lengths. It may have the following properties:
>
> `mode` Optional
>
> > One of the following:
> >
> > - `"shortest"` (default): The resulting iterator stops when one input iterable is exhausted.
> > - `"longest"`: The resulting iterator stops when all input iterables are exhausted. Missing values from shorter iterables are filled according to the `padding` option.
> > - `"strict"`: A `TypeError` is thrown if not all input iterables finish at the same time.
>
> `padding` Optional
>
> > An object. Only retrieved and validated when `mode` is `"longest"`. If `undefined` or absent, missing values from shorter iterables are filled with `undefined` (which is equivalent to passing an empty object). If an object is provided, each key of the `iterables` argument is retrieved *as soon as `Iterator.zipKeyed()` is called*. `padding[key]` is used for missing values for `iterables[key]`. If the `padding` object is missing some keys, those keys are filled with `undefined`.

Return value

A new `Iterator` object. Each of its elements is an object with the same keys as the `iterables` argument, containing the elements from each input iterable at the corresponding position.

Description

The `Iterator.zipKeyed()` function behaves like `Iterator.zip()`; the only difference is that you can specify the keys used in the resulting objects, while `Iterator.zip()` always uses numeric indices (by yielding arrays).

If we represent iterables as arrays, the input may looks like this:

```
({
  a: [a1, a2, a3, a4],
  b: [b1, b2, b3],
  c: [c1, c2, c3, c4, c5],
});
```

The resulting iterator, regardless of the options, will start by yielding the following arrays:

```
({ a: a1, b: b1, c: c1 });
({ a: a2, b: b2, c: c2 });
({ a: a3, b: b3, c: c3 });
```

After the first three objects are yielded, the input iterable `b` is exhausted on the fourth `next()` call—it returns `{ done: true }`. What happens next depends on the `mode` option. If `mode` is `"shortest"` (the default), the resulting iterator stops here: the other two input iterators are closed. If `mode` is `"strict"`, an error is thrown because the other two iterables are *not* finished when the second one yields the result `{ done: true }`. If `mode` is `"longest"`, the resulting iterator continues yielding objects, filling missing values. For example, if `padding` is not provided, it defaults to `undefined`:

```
({ a: a4, b: undefined, c: c4 });
({ a: undefined, b: undefined, c: c5 });
```

If `padding` is provided as an object, it may look like `{ a: p1, b: p2, c: p3 }`. Then `p2` is used to fill the missing value from the input iterable `b`, and `p1` is used to fill the missing value from the input iterable `a`:

```
({ a: a4, b: p2, c: c4 });
({ a: p1, b: p2, c: c5 });
```

Examples

Transposing tabular data

There are two common ways to represent tabular data: as an object where each property is a column, or as an array of objects where each object is a row. This example shows how you can iterate the column-based representation by rows using `Iterator.zipKeyed()`.

```
const table = {
  name: ["Caroline", "Danielle", "Evelyn"],
  age: [30, 25, 35],
  city: ["New York", "London", "Hong Kong"],
};

for (const { name, age, city } of Iterator.zipKeyed(table)) {
  console.log(`${name}, aged ${age}, lives in ${city}.`);
}

// Output:
// Caroline, aged 30, lives in New York.
// Danielle, aged 25, lives in London.
// Evelyn, aged 35, lives in Hong Kong.
```

Most use cases of `Iterator.zipKeyed()` are identical to those of `Iterator.zip()`. Which one to use depends on whether you already have an object of iterables (use `zipKeyed()`) or an array of iterables (use `zip()`). We recommend using `zipKeyed()` when possible, because specifying explicit keys makes it harder to accidentally mix up the order of iterables.

See also

- Polyfill of `Iterator.zipKeyed` in `core-js`
- es-shims polyfill of `Iterator.zipKeyed`
- `Iterator`
- `Iterator.zip()`
- `Iterator.from()`
- `Iterator.concat()`

JSON

The **`JSON`** namespace object contains static methods for parsing values from and converting values to JavaScript Object Notation (JSON).

Description

Unlike most global objects, `JSON` is not a constructor. You cannot use it with the `new` operator or invoke the `JSON` object as a function. All properties and methods of `JSON` are static (just like the `Math` object).

JavaScript and JSON differences

JSON is a syntax for serializing objects, arrays, numbers, strings, booleans, and `null`. It is based upon JavaScript syntax, but is distinct from JavaScript: most of JavaScript is *not* JSON. For example:

Objects and Arrays

> Property names must be double-quoted strings; trailing commas are forbidden.

Numbers

> Leading zeros are prohibited. A decimal point must be followed by at least one digit. `NaN` and `Infinity` are unsupported.

Any JSON text is a valid JavaScript expression, but only after the JSON superset revision. Before the revision, U+2028 LINE SEPARATOR and U+2029 PARAGRAPH SEPARATOR are allowed in string literals and property keys in JSON; but the same use in JavaScript string literals is a `SyntaxError`.

Other differences include allowing only double-quoted strings and no support for `undefined` or comments. For those who wish to use a more human-friendly configuration format based on JSON, there is JSON5, used by the Babel compiler, and the more commonly used YAML.

The same text may represent different values in JavaScript object literals vs. JSON as well. For more information, see Object literal syntax vs. JSON.

Full JSON grammar

Valid JSON syntax is formally defined by the following grammar, expressed in ABNF, and copied from IETF JSON standard (RFC):

```
JSON-text = ws value ws
begin-array     = ws %x5B ws  ; [ left square bracket
begin-object    = ws %x7B ws  ; { left curly bracket
end-array       = ws %x5D ws  ; ] right square bracket
end-object      = ws %x7D ws  ; } right curly bracket
name-separator  = ws %x3A ws  ; : colon
value-separator = ws %x2C ws  ; , comma
```

```
ws = *(
     %x20 /              ; Space
     %x09 /              ; Horizontal tab
     %x0A /              ; Line feed or New line
     %x0D                ; Carriage return
     )
value = false / null / true / object / array / number / string
false = %x66.61.6c.73.65   ; false
null  = %x6e.75.6c.6c      ; null
true  = %x74.72.75.65      ; true
object = begin-object [ member *( value-separator member ) ]
         end-object
member = string name-separator value
array = begin-array [ value *( value-separator value ) ] end-array
number = [ minus ] int [ frac ] [ exp ]
decimal-point = %x2E       ; .
digit1-9 = %x31-39         ; 1-9
e = %x65 / %x45            ; e E
exp = e [ minus / plus ] 1*DIGIT
frac = decimal-point 1*DIGIT
int = zero / ( digit1-9 *DIGIT )
minus = %x2D               ; -
plus = %x2B                ; +
zero = %x30                ; 0
string = quotation-mark *char quotation-mark
char = unescaped /
    escape (
        %x22 /          ; "    quotation mark  U+0022
        %x5C /          ; \    reverse solidus U+005C
        %x2F /          ; /    solidus         U+002F
        %x62 /          ; b    backspace       U+0008
        %x66 /          ; f    form feed       U+000C
        %x6E /          ; n    line feed       U+000A
        %x72 /          ; r    carriage return U+000D
        %x74 /          ; t    tab             U+0009
        %x75 4HEXDIG )  ; uXXXX                U+XXXX
escape = %x5C              ; \
quotation-mark = %x22      ; "
unescaped = %x20-21 / %x23-5B / %x5D-10FFFF
HEXDIG = DIGIT / %x41-46 / %x61-66   ; 0-9, A-F, or a-f
       ; HEXDIG equivalent to HEXDIG rule in [RFC5234]
DIGIT = %x30-39            ; 0-9
      ; DIGIT equivalent to DIGIT rule in [RFC5234]
```

Insignificant whitespace may be present anywhere except within a `JSONNumber` (numbers must contain no whitespace) or `JSONString` (where it is interpreted as the corresponding character in the string, or would cause an error). The tab (U+0009), carriage return (U+000D), line feed (U+000A), and space (U+0020) characters are the only valid whitespace characters.

Static properties

`JSON[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"JSON"`. This property is used in `Object.prototype.toString()`.

Static methods

`JSON.isRawJSON()`

Tests whether a value is an object returned by `JSON.rawJSON()`.

`JSON.parse()`

Parse a piece of string text as JSON, optionally transforming the produced value and its properties, and return the value.

`JSON.rawJSON()`

Creates a "raw JSON" object containing a piece of JSON text. When serialized to JSON, the raw JSON object is treated as if it is already a piece of JSON. This text is required to be valid JSON.

`JSON.stringify()`

Return a JSON string corresponding to the specified value, optionally including only certain properties or replacing property values in a user-defined manner.

Examples

Example JSON

```
{
  "browsers": {
    "firefox": {
      "name": "Firefox",
      "pref_url": "about:config",
      "releases": {
        "1": {
          "release_date": "2004-11-09",
          "status": "retired",
```

```
          "engine": "Gecko",
          "engine_version": "1.7"
        }
      }
    }
  }
}
```

You can use the `JSON.parse()` method to convert the above JSON string into a JavaScript object:

```
const jsonText = `{
  "browsers": {
    "firefox": {
      "name": "Firefox",
      "pref_url": "about:config",
      "releases": {
        "1": {
          "release_date": "2004-11-09",
          "status": "retired",
          "engine": "Gecko",
          "engine_version": "1.7"
        }
      }
    }
  }
}`;

console.log(JSON.parse(jsonText));
```

Lossless number serialization

JSON can contain number literals of arbitrary precision. However, it is not possible to represent all JSON numbers exactly in JavaScript, because JavaScript uses floating point representation which has a fixed precision. For example, `12345678901234567890 === 12345678901234567000` in JavaScript because they have the same floating point representation. This means there is no JavaScript number that corresponds precisely to the `12345678901234567890` JSON number.

Let's assume you have an exact representation of some number (either via `BigInt` or a custom library):

```
const data = {
  // Using a BigInt here to store the exact value,
  // but it can also be a custom high-precision number library,
  // if the number might not be an integer.
  gross_gdp: 12345678901234567890n,
};
```

You want to serialize it and then parse to the same exact number. There are several difficulties:

- On the serialization side, in order to obtain a number in JSON, you have to pass a number to `JSON.stringify`, either via the `replacer` function or via the `toJSON` method. But, in either case, you have already lost precision during number conversion. If you pass a string to `JSON.stringify`, it will be serialized as a string, not a number.
- On the parsing side, not all numbers can be represented exactly. For example, `JSON.parse("12345678901234567890")` returns `12345678901234568000` because the number is rounded to the nearest representable number. Even if you use a `reviver` function, the number will already be rounded before the `reviver` function is called.

There are, in general, two ways to ensure that numbers are losslessly converted to JSON and parsed back: one involves a JSON number, another involves a JSON string. JSON is a *communication format*, so if you use JSON, you are likely communicating with another system (HTTP request, storing in database, etc.). The best solution to choose depends on the recipient system.

Using JSON strings

If the recipient system does not have same JSON-handling capabilities as JavaScript, and does not support high precision numbers, you may want to serialize the number as a string, and then handle it as a string on the recipient side. This is also the only option in older JavaScript.

To specify how custom data types (including `BigInt`) should be serialized to JSON, either add a `toJSON` method to your data type, or use the `replacer` function of `JSON.stringify()`.

```
// Using toJSON() method
BigInt.prototype.toJSON = function () {
  return this.toString();
};
const str1 = JSON.stringify(data);

// Using JSON.stringify() with replacer
const str2 = JSON.stringify(data, (key, value) => {
  if (key === "gross_gdp") {
    return value.toString();
  }
  return value;
});
```

In either case, the JSON text will look like `{"gross_gdp":"12345678901234567890"}`, where the value is a string, not a number. Then, on the recipient side, you can parse the JSON and handle the string.

Using JSON numbers

If the recipient of this message natively supports high precision numbers (such as Python integers), passing numbers as JSON numbers is obviously better, because they can directly parse to the high precision type instead of parsing a string from JSON, and then parsing a number from the string. In JavaScript, you can serialize arbitrary data types to JSON numbers without producing a number value first (resulting in loss of precision) by using `JSON.rawJSON()` to precisely specify what the JSON source text should be.

```
// Using toJSON() method
BigInt.prototype.toJSON = function () {
  return JSON.rawJSON(this.toString());
};
const str1 = JSON.stringify(data);

// Using JSON.stringify() with replacer
const str2 = JSON.stringify(data, (key, value) => {
  if (key === "gross_gdp") {
    return JSON.rawJSON(value.toString());
  }
  return value;
});
```

The text passed to `JSON.rawJSON` is treated as if it is already a piece of JSON, so it won't be serialized again as a string. Therefore, the JSON text will look like `{"gross_gdp":12345678901234567890}`, where the value is a number. This JSON can then be parsed by the recipient without any extra processing, provided that the recipient system does not have the same precision limitations as JavaScript.

When parsing JSON containing high-precision numbers in JavaScript, take extra care because when `JSON.parse()` invokes the `reviver` function, the value you receive is already parsed (and has lost precision). You can use the `context.source` parameter of the `JSON.parse()` `reviver` function to re-parse the number yourself.

```
const parsedData = JSON.parse(str, (key, value, context) => {
  if (key === "gross_gdp") {
    // Or use the constructor of your custom high-precision number library
    return BigInt(context.source);
  }
  return value;
});
// { gross_gdp: 1234567890123456789n }
```

See also

- JSON Diff
- JSON Beautifier/editor
- JSON Parser
- JSON Validator

JSON.isRawJSON()

The **`JSON.isRawJSON()`** static method tests whether a value is an object returned by `JSON.rawJSON()`.

Syntax

```
JSON.isRawJSON(value)
```

Parameters

`value`

The value to test.

Return value

`true` if `value` is created by `JSON.rawJSON()`; otherwise, `false`.

Description

"Raw JSON" objects, when serialized to JSON, are treated as if they are already a piece of JSON. Furthermore, because of the way `JSON.rawJSON()` works, the raw JSON is guaranteed to be syntactically valid JSON. For more information on the shape and behavior of raw JSON objects, see `JSON.rawJSON()`. This method exists to allow other serialization libraries to implement similar behavior to `JSON.stringify()` for raw JSON objects.

Examples

Using JSON.isRawJSON()

The following example demonstrates how to use `JSON.isRawJSON()` to test whether an object was returned by `JSON.rawJSON()`. It implements a custom serializer that serializes data to a YAML-like format.

```
function mySerializer(value, indent = "") {
  if (typeof value !== "object" || value === null) {
    return JSON.stringify(value);
  }
  if (JSON.isRawJSON(value)) {
    return value.rawJSON;
  }
```

```
  const subIndent = `${indent}  `;
  if (Array.isArray(value)) {
    return `- ${value.map((v) => mySerializer(v, subIndent)).join(`\n${indent}- `)}`;
  }
  return Object.entries(value)
    .map(([key, value]) => {
      const subValue = mySerializer(value, subIndent);
      if (subValue.includes("\n")) {
        return `${key}:\n${subIndent}${subValue}`;
      }
      return `${key}: ${subValue}`;
    })
    .join(`\n${indent}`);
}

console.log(
  mySerializer({
    name: "Josh",
    userId: JSON.rawJSON("12345678901234567890"),
    friends: [
      { name: "Alice", userId: JSON.rawJSON("9876543210987654321") },
      { name: "Bob", userId: JSON.rawJSON("56789012345678901234") },
    ],
  }),
);

// name: "Josh"
// userId: 12345678901234567890
// friends:
//   - name: "Alice"
//     userId: 9876543210987654321
//   - name: "Bob"
//     userId: 56789012345678901234
```

If in the above example, the `userId` values were not created by `JSON.rawJSON()`, but passed as numbers directly, then we will get loss of precision upfront because of JS floating point precision limitations.

```
console.log(
  mySerializer({
    name: "Josh",
    userId: 12345678901234567890,
    friends: [
      { name: "Alice", userId: 9876543210987654321 },
      { name: "Bob", userId: 56789012345678901234 },
    ],
  }),
);

// name: "Josh"
// userId: 12345678901234567000
// friends:
//   - name: "Alice"
//     userId: 9876543210987655000
//   - name: "Bob"
//     userId: 56789012345678900000
```

See also

- Polyfill of `JSON.isRawJSON` in `core-js`
- `JSON`
- `JSON.stringify()`
- `JSON.rawJSON()`

JSON.parse()

The **`JSON.parse()`** static method parses a JSON string, constructing the JavaScript value or object described by the string. An optional *reviver* function can be provided to perform a transformation on the resulting object before it is returned.

Try it

```
const json = '{"result":true, "count":42}';
const obj = JSON.parse(json);

console.log(obj.count);
// Expected output: 42

console.log(obj.result);
// Expected output: true
```

Syntax

```
JSON.parse(text)
JSON.parse(text, reviver)
```

Parameters

`text`

The string to parse as JSON. See the `JSON` object for a description of JSON syntax.

`reviver` Optional

If a function, this prescribes how each value originally produced by parsing is transformed before being returned. Non-callable values are ignored. The function is called with the following arguments:

`key`

The key associated with the value.

`value`

The value produced by parsing.

`context` Optional

A context object that holds state relevant to the current expression being revived. It is a new object for each invocation of the reviver function. It is only passed when reviving primitive values, but not when `value` is an object or array. It contains the following property:

`source`

The original JSON string representing this value.

Return value

The `Object`, `Array`, string, number, boolean, or `null` value corresponding to the given JSON `text`.

Exceptions

`SyntaxError`

Thrown if the string to parse is not valid JSON.

Description

`JSON.parse()` parses a JSON string according to the JSON grammar, then evaluates the string as if it's a JavaScript expression. The only instance where a piece of JSON text represents a different value from the same JavaScript expression is when dealing with the `"__proto__"` key — see Object literal syntax vs. JSON.

The reviver parameter

If a `reviver` is specified, the value computed by parsing is *transformed* before being returned. Specifically, the computed value and all its properties (in a depth-first fashion, beginning with the most nested properties and proceeding to the original value itself) are individually run through the `reviver`.

The `reviver` is called with the object containing the property being processed as `this` (unless you define the `reviver` as an arrow function, in which case there's no separate `this` binding) and two arguments: `key` and `value`, representing the property name as a string (even for arrays) and the property value. For primitive values, an additional `context` parameter is passed, which contains the source text of this value. If the `reviver` function returns `undefined` (or returns no value — for example, if execution falls off the end of the function), the property is deleted from the object. Otherwise, the property is redefined to be the return value. If the `reviver` only transforms some values and not others, be certain to return all untransformed values as-is — otherwise, they will be deleted from the resulting object.

Similar to the `replacer` parameter of `JSON.stringify()`, for arrays and objects, `reviver` will be last called on the root value with an empty string as the `key` and the root object as the `value`. For other valid JSON values, `reviver` works similarly and is called once with an empty string as the `key` and the value itself as the `value`.

If you return another value from `reviver`, that value will completely replace the originally parsed value. This even applies to the root value. For example:

```
const transformedObj = JSON.parse('[1,5,{"s":1}]', (key, value) =>
  typeof value === "object" ? undefined : value,
);

console.log(transformedObj); // undefined
```

There is no way to work around this generically. You cannot specially handle the case where `key` is an empty string, because JSON objects can also contain keys that are empty strings. You need to know very precisely what kind of transformation is needed for each key when implementing the reviver.

Note that `reviver` is run after the value is parsed. So, for example, numbers in JSON text will have already been converted to JavaScript numbers, and may lose precision in the process. One way to transfer large numbers without loss of precision is to serialize them as strings, and revive them to BigInts, or other appropriate arbitrary precision formats.

You can also use the `context.source` property to access the original JSON source text representing the value, as shown below:

```
const bigJSON = '{"gross_gdp": 12345678901234567890}';
const bigObj = JSON.parse(bigJSON, (key, value, context) => {
  if (key === "gross_gdp") {
    // Ignore the value because it has already lost precision
    return BigInt(context.source);
  }
  return value;
});
```

Examples

Using JSON.parse()

```
JSON.parse("{}"); // {}
JSON.parse("true"); // true
JSON.parse('"foo"'); // "foo"
JSON.parse('[1, 5, "false"]'); // [1, 5, "false"]
JSON.parse("null"); // null
```

Using the reviver parameter

```
JSON.parse(
  '{"p": 5}',
  (key, value) =>
    typeof value === "number"
      ? value * 2 // return value * 2 for numbers
      : value, // return everything else unchanged
);
// { p: 10 }

JSON.parse('{"1": 1, "2": 2, "3": {"4": 4, "5": {"6": 6}}}', (key, value) => {
  console.log(key);
  return value;
});
// 1
// 2
// 4
// 6
// 5
// 3
// ""
```

Using reviver when paired with the replacer of JSON.stringify()

In order for a value to properly round-trip (that is, it gets deserialized to the same original object), the serialization process must preserve the type information. For example, you can use the `replacer` parameter of `JSON.stringify()` for this purpose:

```
// Maps are normally serialized as objects with no properties.
// We can use the replacer to specify the entries to be serialized.
const map = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const jsonText = JSON.stringify(map, (key, value) =>
  value instanceof Map ? Array.from(value.entries()) : value,
);

console.log(jsonText);
// [[1,"one"],[2,"two"],[3,"three"]]

const map2 = JSON.parse(jsonText, (key, value) =>
  Array.isArray(value) && value.every(Array.isArray) ? new Map(value) : value,
);

console.log(map2);
// Map { 1 => "one", 2 => "two", 3 => "three" }
```

Because JSON has no syntax space for annotating type metadata, in order to revive values that are not plain objects, you have to consider one of the following:

- Serialize the entire object to a string and prefix it with a type tag.
- "Guess" based on the structure of the data (for example, an array of two-member arrays)
- If the shape of the payload is fixed, based on the property name (for example, all properties called `registry` hold `Map` objects).

Illegal JSON

When `JSON.parse` receives a string that does not conform to the JSON grammar, it throws a `SyntaxError`.

Arrays and objects cannot have trailing commas in JSON:

```
JSON.parse("[1, 2, 3, 4, ]");
// SyntaxError: Unexpected token ] in JSON at position 13

JSON.parse('{"foo": 1, }');
// SyntaxError: Unexpected token } in JSON at position 12
```

JSON strings must be delimited by double (not single) quotes:

```
JSON.parse("{'foo': 1}");
// SyntaxError: Unexpected token ' in JSON at position 1
```

```
JSON.parse("'string'");
// SyntaxError: Unexpected token ' in JSON at position 0
```

If you are writing JSON inside a JavaScript string literal, you should either use single quotes to delimit the JavaScript string literal, or escape the double quotes that delimit the JSON string:

```
JSON.parse('{"foo": 1}'); // OK
JSON.parse("{\"foo\": 1}"); // OK
```

See also

- Polyfill of modern `JSON.parse` behavior (reviver's `context` parameter) in `core-js`
- `JSON.stringify()`

JSON.rawJSON()

The **`JSON.rawJSON()`** static method creates a "raw JSON" object containing a piece of JSON text. When serialized to JSON, the raw JSON object is treated as if it is already a piece of JSON. This text is required to be valid JSON.

Syntax

```
JSON.rawJSON(string)
```

Parameters

`string`

The JSON text. Must be valid JSON **representing a primitive value**.

Return value

An object that can be used to create JSON text with the exact same content as the `string` provided, without quotes around the string itself. This object has `null` prototype and is frozen (so it never gets accidentally serialized as a regular object by any kind of primitive conversion), and the following property:

`rawJSON`

The original JSON `string` provided.

Furthermore, it has a private field that marks itself as a raw JSON object. This allows it to be identified by `JSON.stringify()` and `JSON.isRawJSON()`.

Exceptions

`SyntaxError`

Thrown if the `string` is not valid JSON, or if it represents an object or array.

Description

A raw JSON object can be seen as an immutable, atomic data structure like any kind of primitive. It is not a regular object and it contains no data other than the raw JSON text. It is used to "pre-serialize" data to formats that `JSON.stringify` itself cannot produce for various reasons. The most typical use case is the floating point number loss of precision problem. For example:

```
JSON.stringify({ value: 1234567890123456789 });
// {"value":1234567890123457000}
```

The value is not exactly equivalent to the original number any more! This is because JavaScript uses floating point representation for all numbers, so it cannot represent all integers exactly. The number literal `1234567890123456789` itself is already rounded to the nearest representable number when it is parsed by JavaScript.

Without `JSON.rawJSON`, there is no way to tell `JSON.stringify` to produce the number literal `1234567890123456789`, because there is simply no corresponding JavaScript number value. With raw JSON, you can directly tell `JSON.stringify()` what a particular value should be stringified as:

```
const rawJSON = JSON.rawJSON("1234567890123456789");
JSON.stringify({ value: rawJSON });
// {"value":1234567890123456789}
```

For a more complete example of this, see Lossless number serialization.

Note that although we passed a string to `JSON.rawJSON()`, it still becomes a number in the final JSON. This is because the string represents the verbatim JSON text. If you want to serialize a string, you should use `JSON.rawJSON()` with a quotes-enclosed string value:

```
const rawJSON = JSON.rawJSON('"Hello world"');
JSON.stringify({ value: rawJSON });
// {"value":"Hello world"}
```

`JSON.rawJSON` allows you to insert arbitrary JSON text, but does not allow you to create invalid JSON. Anything that was not permitted by the JSON syntax is not permitted by `JSON.rawJSON()` either:

```
const rawJSON = JSON.rawJSON('"Hello\nworld"'); // Syntax error, because line breaks are not allowed in JSON strings
```

Furthermore, you cannot use `JSON.rawJSON()` to create JSON objects or arrays.

Examples

Using JSON.rawJSON() to create JSON expressions of different types

```
const numJSON = JSON.rawJSON("123");
const strJSON = JSON.rawJSON('"Hello world"');
const boolJSON = JSON.rawJSON("true");
const nullJSON = JSON.rawJSON("null");

console.log(
  JSON.stringify({
    age: numJSON,
    message: strJSON,
    isActive: boolJSON,
    nothing: nullJSON,
  }),
);

// {"age":123,"message":"Hello world","isActive":true,"nothing":null}
```

However, you cannot use `JSON.rawJSON()` to create JSON objects or arrays:

```
const arrJSON = JSON.rawJSON("[1, 2, 3]");
const objJSON = JSON.rawJSON('{"a": 1, "b": 2}');
// SyntaxError
```

Using JSON.rawJSON() to create escaped string literals

Apart from numbers, there is only one other type that does not have a one-to-one correspondence between JavaScript values and JSON text: strings. When strings are serialized to JSON, all code points, other than those that are not legal inside JSON string literals (such as line breaks), are printed literally:

```
console.log(JSON.stringify({ value: "\ud83d\ude04" })); // {"value":"😄"}
```

This may not be desirable, because the receiver of this string may handle Unicode differently. To improve interoperability, you can explicitly specify the string to be serialized with escape sequences:

```
const rawJSON = JSON.rawJSON('"\\ud83d\\ude04"');
const objStr = JSON.stringify({ value: rawJSON });
console.log(objStr); // {"value":"\ud83d\ude04"}
console.log(JSON.parse(objStr).value); // 😄
```

Note that the double backslashes in the `rawJSON` actually represents a single slash character.

See also

- Polyfill of `JSON.rawJSON` in `core-js`
- `JSON`
- `JSON.isRawJSON()`
- `JSON.stringify()`

JSON.stringify()

The **`JSON.stringify()`** static method converts a JavaScript value to a JSON string, optionally replacing values if a replacer function is specified or optionally including only the specified properties if a replacer array is specified.

Try it

```
console.log(JSON.stringify({ x: 5, y: 6 }));
// Expected output: '{"x":5,"y":6}'

console.log(
  JSON.stringify([new Number(3), new String("false"), new Boolean(false)]),
);
// Expected output: '[3,"false",false]'

console.log(JSON.stringify({ x: [10, undefined, function () {}, Symbol("")] }));
// Expected output: '{"x":[10,null,null,null]}'

console.log(JSON.stringify(new Date(2006, 0, 2, 15, 4, 5)));
// Expected output: '"2006-01-02T15:04:05.000Z"'
```

Syntax

```
JSON.stringify(value)
JSON.stringify(value, replacer)
JSON.stringify(value, replacer, space)
```

Parameters

`value`

The value to convert to a JSON string.

`replacer` Optional

A function that alters the behavior of the stringification process, or an array of strings and numbers that specifies properties of `value` to be included in the output. If `replacer` is an array, all elements in this array that are not strings or numbers (either primitives or wrapper objects), including `Symbol` values, are completely ignored. If `replacer` is anything other than a function or an array (e.g., `null` or not provided), all string-keyed properties of the object are included in the resulting JSON string.

`space` Optional

A string or number that's used to insert white space (including indentation, line break characters, etc.) into the output JSON string for readability purposes.

If this is a number, it indicates the number of space characters to be used as indentation, clamped to 10 (that is, any number greater than `10` is treated as if it were `10`). Values less than 1 indicate that no space should be used.

If this is a string, the string (or the first 10 characters of the string, if it's longer than that) is inserted before every nested object or array.

If `space` is anything other than a string or number (can be either a primitive or a wrapper object) — for example, is `null` or not provided — no white space is used.

Return value

A JSON string representing the given value, or undefined.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `value` contains a circular reference.
- A `BigInt` value is encountered.

Description

`JSON.stringify()` converts a value to the JSON notation that the value represents. Values are stringified in the following manner:

- `Boolean`, `Number`, `String`, and `BigInt` (obtainable via `Object()`) objects are converted to the corresponding primitive values during stringification, in accordance with the traditional conversion semantics. `Symbol` objects (obtainable via `Object()`) are treated as plain objects.
- Attempting to serialize `BigInt` values will throw. However, if the BigInt has a `toJSON()` method (through monkey patching: `BigInt.prototype.toJSON = ...`), that method can provide the serialization result. This constraint ensures that a proper serialization (and, very likely, its accompanying deserialization) behavior is always explicitly provided by the user.
- `undefined`, `Function`, and `Symbol` values are not valid JSON values. If any such values are encountered during conversion, they are either omitted (when found in an object) or changed to `null` (when found in an array). `JSON.stringify()` can return `undefined` when passing in "pure" values like `JSON.stringify(() => {})` or `JSON.stringify(undefined)`.
- The numbers `Infinity` and `NaN`, as well as the value `null`, are all considered `null`. (But unlike the values in the previous point, they would never be omitted.)
- Arrays are serialized as arrays (enclosed by square brackets). Only array indices between 0 and `length - 1` (inclusive) are serialized; other properties are ignored.
- The special raw JSON object created with `JSON.rawJSON()` is serialized as the raw JSON text it contains (by accessing its `rawJSON` property).
- For other objects:
  - All `Symbol`-keyed properties will be completely ignored, even when using the `replacer` parameter.
  - If the value has a `toJSON()` method, it's responsible for defining what data will be serialized. Instead of the object being serialized, the value returned by the `toJSON()` method when called will be serialized. `JSON.stringify()` calls `toJSON` with one parameter, the `key`, which has the same semantic as the `key` parameter of the `replacer` function:
    - if this object is a property value, the property name
    - if it is in an array, the index in the array, as a string
    - if `JSON.stringify()` was directly called on this object, an empty string

    All `Temporal` objects implement the `toJSON()` method, which returns a string (the same as calling `toString()`). Thus, they will be serialized as strings. Similarly, `Date` objects implement `toJSON()`, which returns the same as `toISOString()`.
  - Only enumerable own properties are visited. This means `Map`, `Set`, etc. will become `"{}"`. You can use the `replacer` parameter to serialize them to something more useful.

    Properties are visited using the same algorithm as `Object.keys()`, which has a well-defined order and is stable across implementations. For example, `JSON.stringify` on the same object will always produce the same string, and `JSON.parse(JSON.stringify(obj))` would produce an object with the same key ordering as the original (assuming the object is completely JSON-serializable).

The replacer parameter

The `replacer` parameter can be either a function or an array.

As an array, its elements indicate the names of the properties in the object that should be included in the resulting JSON string. Only string and number values are taken into account; symbol keys are ignored.

As a function, it takes two parameters: the `key` and the `value` being stringified. The object in which the key was found is provided as the `replacer`'s `this` context.

The `replacer` function is called for the initial object being stringified as well, in which case the `key` is an empty string (`""`). It is then called for each property on the object or array being stringified. Array indices will be provided in its string form as `key`. The current property value will be replaced with the `replacer`'s return value for stringification. This means:

- If you return a number, string, boolean, or `null`, that value is directly serialized and used as the property's value. (Returning a BigInt will throw as well.)
- If you return a `Function`, `Symbol`, or `undefined`, the property is not included in the output.
- If you return any other object, the object is recursively stringified, calling the `replacer` function on each property.

> **Note:** When parsing JSON generated with `replacer` functions, you would likely want to use the `reviver` parameter to perform the reverse operation.

Typically, array elements' index would never shift (even when the element is an invalid value like a function, it will become `null` instead of omitted). Using the `replacer` function allows you to control the order of the array elements by returning a different array.

The space parameter

The `space` parameter may be used to control spacing in the final string.

- If it is a number, successive levels in the stringification will each be indented by this many space characters.
- If it is a string, successive levels will be indented by this string.

Each level of indentation will never be longer than 10. Number values of `space` are clamped to 10, and string values are truncated to 10 characters.

Examples

Using JSON.stringify

```
JSON.stringify({}); // '{}'
JSON.stringify(true); // 'true'
JSON.stringify("foo"); // '"foo"'
JSON.stringify([1, "false", false]); // '[1,"false",false]'
JSON.stringify([NaN, null, Infinity]); // '[null,null,null]'
JSON.stringify({ x: 5 }); // '{"x":5}'

JSON.stringify(new Date(1906, 0, 2, 15, 4, 5));
// '"1906-01-02T15:04:05.000Z"'

JSON.stringify({ x: 5, y: 6 });
// '{"x":5,"y":6}'
JSON.stringify([new Number(3), new String("false"), new Boolean(false)]);
// '[3,"false",false]'

// String-keyed array elements are not enumerable and make no sense in JSON
const a = ["foo", "bar"];
a["baz"] = "quux"; // a: [ 0: 'foo', 1: 'bar', baz: 'quux' ]
JSON.stringify(a);
// '["foo","bar"]'

JSON.stringify({ x: [10, undefined, function () {}, Symbol("")] });
// '{"x":[10,null,null,null]}'

// Standard data structures
JSON.stringify([
  new Set([1]),
  new Map([[1, 2]]),
  new WeakSet([{ a: 1 }]),
  new WeakMap([[{ a: 1 }, 2]]),
]);
// '[{},{},{},{}]'

// TypedArray
JSON.stringify([new Int8Array([1]), new Int16Array([1]), new Int32Array([1])]);
// '[{"0":1},{"0":1},{"0":1}]'
JSON.stringify([
  new Uint8Array([1]),
  new Uint8ClampedArray([1]),
  new Uint16Array([1]),
  new Uint32Array([1]),
]);
// '[{"0":1},{"0":1},{"0":1},{"0":1}]'
JSON.stringify([new Float32Array([1]), new Float64Array([1])]);
// '[{"0":1},{"0":1}]'

// toJSON()
JSON.stringify({
  x: 5,
  y: 6,
  toJSON() {
    return this.x + this.y;
  },
});
// '11'

// Symbols:
JSON.stringify({ x: undefined, y: Object, z: Symbol("") });
// '{}'
JSON.stringify({ [Symbol("foo")]: "foo" });
// '{}'
```

```
JSON.stringify({ [Symbol.for("foo")]: "foo" }, [Symbol.for("foo")]);
// '{}'
JSON.stringify({ [Symbol.for("foo")]: "foo" }, (k, v) => {
  if (typeof k === "symbol") {
    return "a symbol";
  }
});
// undefined

// Non-enumerable properties:
JSON.stringify(
  Object.create(null, {
    x: { value: "x", enumerable: false },
    y: { value: "y", enumerable: true },
  }),
);
// '{"y":"y"}'

// BigInt values throw
JSON.stringify({ x: 2n });
// TypeError: BigInt value can't be serialized in JSON
```

Using a function as replacer

```
function replacer(key, value) {
  // Filtering out properties
  if (typeof value === "string") {
    return undefined;
  }
  return value;
}

const foo = {
  foundation: "Mozilla",
  model: "box",
  week: 45,
  transport: "car",
  month: 7,
};
JSON.stringify(foo, replacer);
// '{"week":45,"month":7}'
```

If you wish the `replacer` to distinguish an initial object from a key with an empty string property (since both would give the empty string as key and potentially an object as value), you will have to keep track of the iteration count (if it is beyond the first iteration, it is a genuine empty string key).

```
function makeReplacer() {
  let isInitial = true;

  return (key, value) => {
    if (isInitial) {
      isInitial = false;
      return value;
    }
    if (key === "") {
      // Omit all properties with name "" (except the initial object)
      return undefined;
    }
    return value;
  };
}

const replacer = makeReplacer();
console.log(JSON.stringify({ "": 1, b: 2 }, replacer)); // "{"b":2}"
```

Using an array as replacer

```
const foo = {
  foundation: "Mozilla",
  model: "box",
  week: 45,
  transport: "car",
  month: 7,
};

JSON.stringify(foo, ["week", "month"]);
// '{"week":45,"month":7}', only keep "week" and "month" properties
```

Using the space parameter

Indent the output with one space:

```
console.log(JSON.stringify({ a: 2 }, null, " "));
/*
{
```

```
 "a": 2
}
*/
```

Using a tab character mimics standard pretty-print appearance:

```
console.log(JSON.stringify({ uno: 1, dos: 2 }, null, "\t"));
/*
{
        "uno": 1,
        "dos": 2
}
*/
```

toJSON() behavior

Defining `toJSON()` for an object allows overriding its serialization behavior.

```
const obj = {
  data: "data",

  toJSON(key) {
    return key ? `Now I am a nested object under key '${key}'` : this;
  },
};

JSON.stringify(obj);
// '{"data":"data"}'

JSON.stringify({ obj });
// '{"obj":"Now I am a nested object under key 'obj'"}'

JSON.stringify([obj]);
// '["Now I am a nested object under key '0'"]'
```

Issue with serializing circular references

Since the JSON format doesn't support object references (although an IETF draft exists), a `TypeError` will be thrown if one attempts to encode an object with circular references.

```
const circularReference = {};
circularReference.myself = circularReference;

// Serializing circular references throws "TypeError: cyclic object value"
JSON.stringify(circularReference);
```

To serialize circular references, you can use a library that supports them (e.g., cycle.js by Douglas Crockford) or implement a solution yourself, which will require finding and replacing (or removing) the cyclic references by serializable values.

If you are using `JSON.stringify()` to deep-copy an object, you may instead want to use `structuredClone()`, which supports circular references. JavaScript engine APIs for binary serialization, such as `v8.serialize()`, also support circular references.

Using JSON.stringify() with localStorage

In a case where you want to store an object created by your user and allow it to be restored even after the browser has been closed, the following example is a model for the applicability of `JSON.stringify()`:

```
// Creating an example of JSON
const session = {
  screens: [],
  state: true,
};
session.screens.push({ name: "screenA", width: 450, height: 250 });
session.screens.push({ name: "screenB", width: 650, height: 350 });
session.screens.push({ name: "screenC", width: 750, height: 120 });
session.screens.push({ name: "screenD", width: 250, height: 60 });
session.screens.push({ name: "screenE", width: 390, height: 120 });
session.screens.push({ name: "screenF", width: 1240, height: 650 });

// Converting the JSON string with JSON.stringify()
// then saving with localStorage in the name of session
localStorage.setItem("session", JSON.stringify(session));

// Example of how to transform the String generated through
// JSON.stringify() and saved in localStorage in JSON object again
const restoredSession = JSON.parse(localStorage.getItem("session"));

// Now restoredSession variable contains the object that was saved
// in localStorage
console.log(restoredSession);
```

Well-formed JSON.stringify()

Engines implementing the well-formed JSON.stringify specification will stringify lone surrogates (any code point from U+D800 to U+DFFF) using Unicode escape sequences rather than literally (outputting lone surrogates). Before this change, such strings could not be encoded in valid UTF-8 or UTF-16:

```
JSON.stringify("\uD800"); // '"\ud800"'
```

But with this change `JSON.stringify()` represents lone surrogates using JSON escape sequences that *can* be encoded in valid UTF-8 or UTF-16:

```
JSON.stringify("\uD800"); // '"\\ud800"'
```

This change should be backwards-compatible as long as you pass the result of `JSON.stringify()` to APIs such as `JSON.parse()` that will accept any valid JSON text, because they will treat Unicode escapes of lone surrogates as identical to the lone surrogates themselves. *Only* if you are directly interpreting the result of `JSON.stringify()` do you need to carefully handle `JSON.stringify()`'s two possible encodings of these code points.

See also

- Polyfill of modern `JSON.stringify` behavior (symbol, well-formed unicode, raw JSON) in `core-js`
- `JSON.parse()`
- `JSON.rawJSON()`

Map

The **`Map`** object holds key-value pairs and remembers the original insertion order of the keys. Any value (both objects and primitive values) may be used as either a key or a value.

Try it

```
const map = new Map();

map.set("a", 1);
map.set("b", 2);
map.set("c", 3);

console.log(map.get("a"));
// Expected output: 1

map.set("a", 97);

console.log(map.get("a"));
// Expected output: 97

console.log(map.size);
// Expected output: 3

map.delete("b");

console.log(map.size);
// Expected output: 2
```

Description

`Map` objects are collections of key-value pairs. A key in the `Map` **may only occur once**; it is unique in the `Map`'s collection. A `Map` object is iterated by key-value pairs — a `for...of` loop returns a 2-member array of `[key, value]` for each iteration. Iteration happens in *insertion order*, which corresponds to the order in which each key-value pair was first inserted into the map by the `set()` method (that is, there wasn't a key with the same value already in the map when `set()` was called).

The specification requires maps to be implemented "that, on average, provide access times that are sublinear on the number of elements in the collection". Therefore, it could be represented internally as a hash table (with O(1) lookup), a search tree (with O(log(N)) lookup), or any other data structure, as long as the complexity is better than O(N).

Key equality

Value equality is based on the SameValueZero algorithm. (It used to use SameValue, which treated `0` and `-0` as different. Check browser compatibility.) This means `NaN` is considered the same as `NaN` (even though `NaN !== NaN`) and all other values are considered equal according to the semantics of the `===` operator. Also, for object keys, equality is based on object identity. They are compared by reference, not by value. See Using the Map object for examples.

Objects vs. Maps

`Object` is similar to `Map`—both let you set keys to values, retrieve those values, delete keys, and detect whether something is stored at a key. For this reason (and because there were no built-in alternatives), `Object` has been used as `Map` historically.

However, there are important differences that make `Map` preferable in some cases:

	Map	Object
Accidental Keys	A `Map` does not contain any keys by default. It only contains what is explicitly put into it.	An `Object` has a prototype, so it contains default keys that could collide with your own keys if you're not careful. **Note:** This can be bypassed by using `Object.create(null)`, but this is seldom done.

	Map	Object
Security	A `Map` is safe to use with user-provided keys and values.	Setting user-provided key-value pairs on an `Object` may allow an attacker to override the object's prototype, which can lead to object injection attacks or prototype pollution attacks. Like the accidental keys issue, this can also be mitigated by using a `null`-prototype object.
Key Types	A `Map`'s keys can be any value (including functions, objects, or any primitive).	The keys of an `Object` must be either a `String` or a `Symbol`.
Key Order	The keys in `Map` are ordered in a straightforward way: A `Map` object iterates entries, keys, and values in the order of entry insertion.	Although the keys of an ordinary `Object` are ordered now, this was not always the case, and the order is complex. As a result, it's best not to rely on property order. The order was first defined for own properties only in ECMAScript 2015; ECMAScript 2020 defines order for inherited properties as well. But note that no single mechanism iterates **all** of an object's properties; the various mechanisms each include different subsets of properties. (`for-in` includes only enumerable string-keyed properties; `Object.keys` includes only own, enumerable, string-keyed properties; `Object.getOwnPropertyNames` includes own, string-keyed properties even if non-enumerable; `Object.getOwnPropertySymbols` does the same for just `Symbol`-keyed properties, etc.)
Size	The number of items in a `Map` is easily retrieved from its `size` property.	Determining the number of items in an `Object` is more roundabout and less efficient. A common way to do it is through the `length` of the array returned from `Object.keys()`.
Iteration	A `Map` is an iterable, so it can be directly iterated.	`Object` does not implement an iteration protocol, and so objects are not directly iterable using the JavaScript for...of statement (by default). **Note:** • An object can implement the iteration protocol, or you can get an iterable for an object using `Object.keys` or `Object.entries`. • The for...in statement allows you to iterate over the *enumerable* properties of an object.
Performance	Performs better in scenarios involving frequent additions and removals of key-value pairs.	Not optimized for frequent additions and removals of key-value pairs.
Serialization and parsing	No native support for serialization or parsing. (But you can build your own serialization and parsing support for `Map` by using `JSON.stringify()` with its *replacer* argument, and by using `JSON.parse()` with its *reviver* argument. See the Stack Overflow question How do you JSON.stringify an ES6 Map?).	Native support for serialization from `Object` to JSON, using `JSON.stringify()`. Native support for parsing from JSON to `Object`, using `JSON.parse()`.

Setting object properties

Setting Object properties works for Map objects as well, and can cause considerable confusion.

Therefore, this appears to work in a way:

```
const wrongMap = new Map();
wrongMap["bla"] = "blaa";
wrongMap["bla2"] = "blaaa2";

console.log(wrongMap); // Map { bla: 'blaa', bla2: 'blaaa2' }
```

But that way of setting a property does not interact with the Map data structure. It uses the feature of the generic object. The value of 'bla' is not stored in the Map for queries. Other operations on the data fail:

```
wrongMap.has("bla"); // false
wrongMap.delete("bla"); // false
console.log(wrongMap); // Map { bla: 'blaa', bla2: 'blaaa2' }
```

The correct usage for storing data in the Map is through the `set(key, value)` method.

```
const contacts = new Map();
contacts.set("Jessie", { phone: "213-555-1234", address: "123 N 1st Ave" });
contacts.has("Jessie"); // true
```

```
contacts.get("Hilary"); // undefined
contacts.set("Hilary", { phone: "617-555-4321", address: "321 S 2nd St" });
contacts.get("Jessie"); // {phone: "213-555-1234", address: "123 N 1st Ave"}
contacts.delete("Raymond"); // false
contacts.delete("Jessie"); // true
console.log(contacts.size); // 1
```

Map-like browser APIs

Browser `Map`-like objects (or "maplike objects") are Web API interfaces that behave in many ways like a `Map`.

Just like `Map`, entries can be iterated in the same order that they were added to the object. `Map`-like objects and `Map` also have properties and methods that share the same name and behavior. However unlike `Map` they only allow specific predefined types for the keys and values of each entry.

The allowed types are set in the specification IDL definition. For example, `RTCStatsReport` is a `Map`-like object that must use strings for keys and objects for values. This is defined in the specification IDL below:

```
interface RTCStatsReport {
  readonly maplike<DOMString, object>;
};
```

`Map`-like objects are either read-only or read-writable (see the `readonly` keyword in the IDL above).

- Read-only `Map`-like objects have the property `size`, and the methods: `entries()`, `forEach()`, `get()`, `has()`, `keys()`, `values()`, and `[Symbol.iterator]()`.
- Writeable `Map`-like objects additionally have the methods: `clear()`, `delete()`, and `set()`.

The methods and properties have the same behavior as the equivalent entities in `Map`, except for the restriction on the types of the keys and values.

The following are examples of read-only `Map`-like browser objects:

- `AudioParamMap`
- `CSSFontFeatureValuesMap`
- `EventCounts`
- `KeyboardLayoutMap`
- `MIDIInputMap`
- `MIDIOutputMap`
- `RTCStatsReport`

Constructor

`Map()`

Creates a new `Map` object.

Static properties

`Map[Symbol.species]`

The constructor function that is used to create derived objects.

Static methods

`Map.groupBy()`

Groups the elements of a given iterable using the values returned by a provided callback function. The final returned `Map` uses the unique values from the test function as keys, which can be used to get the array of elements in each group.

Instance properties

These properties are defined on `Map.prototype` and shared by all `Map` instances.

`Map.prototype.constructor`

The constructor function that created the instance object. For `Map` instances, the initial value is the `Map` constructor.

`Map.prototype.size`

Returns the number of key/value pairs in the `Map` object.

`Map.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Map"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Map.prototype.clear()`

Removes all key-value pairs from the `Map` object.

`Map.prototype.delete()`

Removes the entry specified by the key from this `Map`.

`Map.prototype.entries()`

Returns a new Iterator object that contains a two-member array of `[key, value]` for each element in the `Map` object in insertion order.

`Map.prototype.forEach()`

Calls `callbackFn` once for each key-value pair present in the `Map` object, in insertion order. If a `thisArg` parameter is provided to `forEach`, it will be used as the `this` value for each callback.

`Map.prototype.get()`

Returns the value corresponding to the key in this `Map`, or `undefined` if there is none.

`Map.prototype.getOrInsert()`

Returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

`Map.prototype.getOrInsertComputed()`

Returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

`Map.prototype.has()`

Returns a boolean indicating whether an entry with the specified key exists in this `Map` or not.

`Map.prototype.keys()`

Returns a new Iterator object that contains the keys for each element in the `Map` object in insertion order.

`Map.prototype.set()`

Adds a new entry with a specified key and value to this `Map`, or updates an existing entry if the key already exists.

`Map.prototype.values()`

Returns a new Iterator object that contains the values for each element in the `Map` object in insertion order.

`Map.prototype[Symbol.iterator]()`

Returns a new Iterator object that contains a two-member array of `[key, value]` for each element in the `Map` object in insertion order.

Examples

Using the Map object

```
const myMap = new Map();

const keyString = "a string";
const keyObj = {};
const keyFunc = () => {};

// setting the values
myMap.set(keyString, "value associated with 'a string'");
myMap.set(keyObj, "value associated with keyObj");
myMap.set(keyFunc, "value associated with keyFunc");

console.log(myMap.size); // 3

// getting the values
console.log(myMap.get(keyString)); // "value associated with 'a string'"
console.log(myMap.get(keyObj)); // "value associated with keyObj"
console.log(myMap.get(keyFunc)); // "value associated with keyFunc"

console.log(myMap.get("a string")); // "value associated with 'a string'", because keyString === 'a string'
console.log(myMap.get({})); // undefined, because keyObj !== {}
console.log(myMap.get(() => {})); // undefined, because keyFunc !== () => {}
```

Using NaN as Map keys

`NaN` can also be used as a key. Even though every `NaN` is not equal to itself (`NaN !== NaN` is true), the following example works because `NaN`s are indistinguishable from each other:

```
const myMap = new Map();
myMap.set(NaN, "not a number");

myMap.get(NaN);
// "not a number"

const otherNaN = Number("foo");
myMap.get(otherNaN);
// "not a number"
```

Iterating Map with for...of

Maps can be iterated using a `for...of` loop:

```
const myMap = new Map();
myMap.set(0, "zero");
myMap.set(1, "one");

for (const [key, value] of myMap) {
  console.log(`${key} = ${value}`);
}
// 0 = zero
// 1 = one

for (const key of myMap.keys()) {
  console.log(key);
}
// 0
// 1

for (const value of myMap.values()) {
  console.log(value);
}
// zero
// one

for (const [key, value] of myMap.entries()) {
  console.log(`${key} = ${value}`);
}
// 0 = zero
// 1 = one
```

Iterating Map with forEach()

Maps can be iterated using the `forEach()` method:

```
myMap.forEach((value, key) => {
  console.log(`${key} = ${value}`);
});
// 0 = zero
// 1 = one
```

Relation with Array objects

```
const kvArray = [
  ["key1", "value1"],
  ["key2", "value2"],
];

// Use the regular Map constructor to transform a 2D key-value Array into a map
const myMap = new Map(kvArray);

console.log(myMap.get("key1")); // "value1"

// Use Array.from() to transform a map into a 2D key-value Array
console.log(Array.from(myMap)); // Will show you exactly the same Array as kvArray

// A succinct way to do the same, using the spread syntax
console.log([...myMap]);

// Or use the keys() or values() iterators, and convert them to an array
console.log(Array.from(myMap.keys())); // ["key1", "key2"]
```

Cloning and merging Maps

Just like `Array`s, `Map`s can be cloned:

```
const original = new Map([[1, "one"]]);

const clone = new Map(original);

console.log(clone.get(1)); // one
console.log(original === clone); // false (useful for shallow comparison)
```

> **Note:** Keep in mind that *the data itself* is not cloned. In other words, it is only a shallow copy of the `Map`.

Maps can be merged, maintaining key uniqueness:

```
const first = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const second = new Map([
  [1, "uno"],
```

```
  [2, "dos"],
]);

// Merge two maps. The last repeated key wins.
// Spread syntax essentially converts a Map to an Array
const merged = new Map([...first, ...second]);

console.log(merged.get(1)); // uno
console.log(merged.get(2)); // dos
console.log(merged.get(3)); // three
```

Maps can be merged with Arrays, too:

```
const first = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const second = new Map([
  [1, "uno"],
  [2, "dos"],
]);

// Merge maps with an array. The last repeated key wins.
const merged = new Map([...first, ...second, [1, "un"]]);

console.log(merged.get(1)); // un
console.log(merged.get(2)); // dos
console.log(merged.get(3)); // three
```

See also

- Polyfill for `Map` in `core-js`
- es-shims polyfill of `Map`
- `Set`
- `WeakMap`
- `WeakSet`

Map.prototype.clear()

The **`clear()`** method of `Map` instances removes all elements from this map.

Try it

```
const map = new Map();

map.set("bar", "baz");
map.set(1, "foo");

console.log(map.size);
// Expected output: 2

map.clear();

console.log(map.size);
// Expected output: 0
```

Syntax

```
clear()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Using clear()

```
const myMap = new Map();
myMap.set("bar", "baz");
myMap.set(1, "foo");

console.log(myMap.size); // 2
console.log(myMap.has("bar")); // true

myMap.clear();
```

```
console.log(myMap.size); // 0
console.log(myMap.has("bar")); // false
```

See also

- `Map`

Map.prototype.delete()

The **`delete()`** method of `Map` instances removes the entry specified by the key from this `Map`.

Try it

```
const map = new Map();
map.set("bar", "foo");

console.log(map.delete("bar"));
// Expected result: true
// True indicates successful removal

console.log(map.has("bar"));
// Expected result: false
```

Syntax

```
mapInstance.delete(key)
```

Parameters

`key`

The key of the entry to remove from the `Map` object. Object keys are compared by reference, not by value.

Return value

`true` if an entry in the `Map` object has been removed successfully. `false` if the key is not found in the `Map`.

Examples

Using delete()

```
const myMap = new Map();
myMap.set("bar", "foo");

console.log(myMap.delete("bar")); // Returns true. Successfully removed.
console.log(myMap.has("bar")); // Returns false. The "bar" element is no longer present.
```

See also

- `Map`
- `Map.prototype.clear()`
- `Map.prototype.get()`
- `Map.prototype.set()`
- `Map.prototype.has()`

Map.prototype.entries()

The **`entries()`** method of `Map` instances returns a new *map iterator* object that contains the `[key, value]` pairs for each element in this map in insertion order.

Try it

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map.entries();

console.log(iterator.next().value);
// Expected output: Array ["0", "foo"]

console.log(iterator.next().value);
// Expected output: Array [1, "bar"]
```

Syntax

```
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using entries()

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap.entries();

console.log(mapIter.next().value); // ["0", "foo"]
console.log(mapIter.next().value); // [1, "bar"]
console.log(mapIter.next().value); // [Object, "baz"]
```

See also

- `Map.prototype.keys()`
- `Map.prototype.values()`

Map.prototype.forEach()

The **`forEach()`** method of `Map` instances executes a provided function once per each key/value pair in this map, in insertion order.

Try it

```
function logMapElements(value, key, map) {
  console.log(`m[${key}] = ${value}`);
}

new Map([
  ["foo", 3],
  ["bar", {}],
  ["baz", undefined],
]).forEach(logMapElements);

// Expected output: "m[foo] = 3"
// Expected output: "m[bar] = [object Object]"
// Expected output: "m[baz] = undefined"
```

Syntax

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each entry in the map. The function is called with the following arguments:

`value`

Value of each iteration.

`key`

Key of each iteration.

`map`

The map being iterated.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`.

Return value

None (`undefined`).

Description

The `forEach` method executes the provided `callback` once for each key of the map which actually exist. It is not invoked for keys which have been deleted. However, it is executed for values which are present but have the value `undefined`.

`callback` is invoked with **three arguments**:

- the entry's `value`
- the entry's `key`

- the `Map` **object** being traversed

If a `thisArg` parameter is provided to `forEach`, it will be passed to `callback` when invoked, for use as its `this` value. Otherwise, the value `undefined` will be passed for use as its `this` value. The `this` value ultimately observable by `callback` is determined according to the usual rules for determining the `this` seen by a function.

Each value is visited once, except in the case when it was deleted and re-added before `forEach` has finished. `callback` is not invoked for values deleted before being visited. New values added before `forEach` has finished will be visited.

Examples

Printing the contents of a Map object

The following code logs a line for each element in a `Map` object:

```
function logMapElements(value, key, map) {
  console.log(`map.get('${key}') = ${value}`);
}
new Map([
  ["foo", 3],
  ["bar", {}],
  ["baz", undefined],
]).forEach(logMapElements);
// Logs:
// "map.get('foo') = 3"
// "map.get('bar') = [object Object]"
// "map.get('baz') = undefined"
```

See also

- `Array.prototype.forEach()`
- `Set.prototype.forEach()`

Map.prototype.get()

The **`get()`** method of `Map` instances returns the value corresponding to the key in this `Map`, or `undefined` if there is none. Object values are returned as the same reference that was originally stored, not as a copy, so mutations to the returned object will be reflected anywhere that reference is held, including inside the `Map`.

Try it

```
const map = new Map();
map.set("bar", "foo");

console.log(map.get("bar"));
// Expected output: "foo"

console.log(map.get("baz"));
// Expected output: undefined
```

Syntax

```
get(key)
```

Parameters

`key`

The key of the value to return from the `Map` object. Object keys are compared by reference, not by value.

Return value

The value associated with the specified key in the `Map` object. If the key can't be found, `undefined` is returned.

Examples

Using get()

```
const myMap = new Map();
myMap.set("bar", "foo");

console.log(myMap.get("bar")); // Returns "foo"
console.log(myMap.get("baz")); // Returns undefined
```

Using get() to retrieve a reference to an object

```
const arr = [];
const myMap = new Map();
myMap.set("bar", arr);

myMap.get("bar").push("foo");

console.log(arr); // ["foo"]
console.log(myMap.get("bar")); // ["foo"]
```

See also

- `Map`
- `Map.prototype.delete()`
- `Map.prototype.set()`
- `Map.prototype.has()`

Map.prototype.getOrInsert()

The **`getOrInsert()`** method of `Map` instances returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

If the computation of the default value is expensive, consider using `Map.prototype.getOrInsertComputed()` instead, which takes a callback to compute the default value only if it's actually needed.

Try it

```
const map = new Map([["bar", "foo"]]);
console.log(map.getOrInsert("bar", "default"));
// Expected output: "foo"

console.log(map.getOrInsert("baz", "default"));
// Expected output: "default"
```

Syntax

```
getOrInsert(key, defaultValue)
```

Parameters

`key`

The key of the value to return from the `Map` object. Object keys are compared by reference, not by value.

`defaultValue`

The value to insert and return if the key is not already present in the `Map` object.

Return value

The value associated with the specified key in the `Map` object. If the key can't be found, `defaultValue` is inserted and returned.

Description

The `getOrInsert()` method is equivalent to the following:

```
if (map.has(key)) {
  return map.get(key);
}
map.set(key, defaultValue);
return defaultValue;
```

It is also similar to the following pattern (which is slightly less reliable if `null` or `undefined` are valid values in your map):

```
map.set(key, map.get(key) ?? defaultValue);
```

Examples

Multimap

In a map where each key is mapped to an array of values, you can use `getOrInsert()` to ensure that the array exists for a given key before attempting to push a new value to the array.

```
map.getOrInsert(key, []).push(value);
```

Applying default values

You can use `getOrInsert()` to ensure that a key exists in a map, even if you currently don't need its value. This is usually to normalize user input.

Imagine you have a map of user preferences, and you want to ensure that a certain preference is always set to a default value if the user hasn't specified it:

```
const options = readConfig();
options.getOrInsert("theme", "light");
options.getOrInsert("fontSize", 14);

// Later in your code, you can safely assume these options exist
document.body.dataset.theme = options.get("theme");
```

See also

- Polyfill of `Map.prototype.getOrInsert` in `core-js`

- es-shims polyfill of `Map.prototype.getOrInsert`
- `Map`
- `Map.prototype.get()`
- `Map.prototype.set()`
- `Map.prototype.has()`
- `Map.prototype.getOrInsertComputed()`

Map.prototype.getOrInsertComputed()

The **`getOrInsertComputed()`** method of `Map` instances returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

Use this method instead of `Map.prototype.getOrInsert()` when the default value is expensive to compute, and you want to avoid computing it unless it's actually needed.

Try it

```
const map = new Map([["bar", "foo"]]);
const defaultCreator = (key) => `default for ${key}`;

console.log(map.getOrInsertComputed("bar", defaultCreator));
// Expected output: "foo"

console.log(map.getOrInsertComputed("baz", defaultCreator));
// Expected output: "default for baz"
```

Syntax

```
getOrInsertComputed(key, callback)
```

Parameters

`key`

The key of the element to return from the `Map` object. Object keys are compared by reference, not by value.

`callback`

A function that returns the value to insert and return if the key is not already present in the `Map` object. The function is called with the following argument:

`key`

The same key that was passed to `getOrInsertComputed()`.

Return value

The value associated with the specified key in the `Map` object. If the key can't be found, the result of `callback(key)` is inserted and returned.

Exceptions

`TypeError`

Thrown if `callback` isn't callable.

Examples

Avoiding unnecessary default computation

When using `Map.prototype.getOrInsert()`, the default value is computed every time, even if it is not needed. With `getOrInsertComputed()`, the default value is only computed when necessary.

```
const map = new Map([["bar", "foo"]]);
const defaultCreator = (key) => {
  console.log(`Creating default for ${key}`);
  return `default for ${key}`;
};

map.getOrInsert("bar", defaultCreator("bar")); // Logs "Creating default for bar"
map.getOrInsertComputed("bar", defaultCreator); // No log
```

See also

- Polyfill of `Map.prototype.getOrInsertComputed` in `core-js`
- es-shims polyfill of `Map.prototype.getOrInsertComputed`
- `Map`
- `Map.prototype.get()`
- `Map.prototype.set()`
- `Map.prototype.has()`
- `Map.prototype.getOrInsert()`

Map.groupBy()

> **Note:** In some versions of some browsers, this method was implemented as the method `Array.prototype.groupToMap()`. Due to web compatibility issues, it is now implemented as a static method. Check the browser compatibility table for details.

The **`Map.groupBy()`** static method groups the elements of a given iterable using the values returned by a provided callback function. The final returned `Map` uses the unique values from the test function as keys, which can be used to get the array of elements in each group.

The method is primarily useful when grouping elements that are associated with an object, and in particular when that object might change over time. If the object is invariant, you might instead represent it using a string, and group elements with `Object.groupBy()`.

Try it

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 9 },
  { name: "bananas", type: "fruit", quantity: 5 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 12 },
  { name: "fish", type: "meat", quantity: 22 },
];

const restock = { restock: true };
const sufficient = { restock: false };
const result = Map.groupBy(inventory, ({ quantity }) =>
  quantity < 6 ? restock : sufficient,
);
console.log(result.get(restock));
// [{ name: "bananas", type: "fruit", quantity: 5 }]
```

Syntax

```
Map.groupBy(items, callbackFn)
```

Parameters

`items`

An iterable (such as an `Array`) whose elements will be grouped.

`callbackFn`

A function to execute for each element in the iterable. It should return a value (object or primitive) indicating the group of the current element. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

A `Map` object with keys for each group, each assigned to an array containing the elements of the associated group.

Description

`Map.groupBy()` calls a provided `callbackFn` function once for each element in an iterable. The callback function should return a value indicating the group of the associated element. The values returned by `callbackFn` are used as keys for the `Map` returned by `Map.groupBy()`. Each key has an associated array containing all the elements for which the callback returned the same value.

The elements in the returned `Map` and the original iterable are the same (not deep copies). Changing the internal structure of the elements will be reflected in both the original iterable and the returned `Map`.

This method is useful when you need to group information that is related to a particular object that might potentially change over time. This is because even if the object is modified, it will continue to work as a key to the returned `Map`. If you instead create a string representation for the object and use that as a grouping key in `Object.groupBy()`, you must maintain the mapping between the original object and its representation as the object changes.

> **Note:** To access the groups in the returned `Map`, you must use the same object that was originally used as a key in the `Map` (although you may modify its properties). You can't use another object that just happens to have the same name and properties.

`Map.groupBy` does not read the value of `this`. It can be called on any object and a new `Map` instance will be returned.

Examples

Using Map.groupBy()

First we define an array containing objects representing an inventory of different foodstuffs. Each food has a `type` and a `quantity`.

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 9 },
  { name: "bananas", type: "fruit", quantity: 5 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 12 },
  { name: "fish", type: "meat", quantity: 22 },
];
```

The code below uses `Map.groupBy()` with an arrow function that returns the object keys named `restock` or `sufficient`, depending on whether the element has `quantity < 6`. The returned `result` object is a `Map` so we need to call `get()` with the key to obtain the array.

```
const restock = { restock: true };
const sufficient = { restock: false };
const result = Map.groupBy(inventory, ({ quantity }) =>
  quantity < 6 ? restock : sufficient,
);
console.log(result.get(restock));
// [{ name: "bananas", type: "fruit", quantity: 5 }]
```

Note that the function argument `{ quantity }` is a basic example of object destructuring syntax for function arguments. This unpacks the `quantity` property of an object passed as a parameter, and assigns it to a variable named `quantity` in the body of the function. This is a very succinct way to access the relevant values of elements within a function.

The key to a `Map` can be modified and still used. However you can't recreate the key and still use it. For this reason it is important that anything that needs to use the map keeps a reference to its keys.

```
// The key can be modified and still used
restock["fast"] = true;
console.log(result.get(restock));
// [{ name: "bananas", type: "fruit", quantity: 5 }]

// A new key can't be used, even if it has the same structure!
const restock2 = { restock: true };
console.log(result.get(restock2)); // undefined
```

See also

- Polyfill of `Map.groupBy` in `core-js`
- es-shims polyfill of `Map.groupBy`
- Indexed collections guide
- `Array.prototype.reduce()`
- `Map()`
- `Object.groupBy()`

Map.prototype.has()

The **`has()`** method of `Map` instances returns a boolean indicating whether an entry with the specified key exists in this `Map` or not.

Try it

```
const map = new Map();
map.set("bar", "foo");

console.log(map.has("bar"));
// Expected output: true

console.log(map.has("baz"));
// Expected output: false
```

Syntax

```
has(key)
```

Parameters

`key`

The key of the entry to test for presence in the `Map` object. Object keys are compared by reference, not by value.

Return value

Returns `true` if an entry with the specified key exists in the `Map` object; otherwise `false`.

Examples

Using has()

```
const myMap = new Map();
myMap.set("bar", "foo");
```

```
console.log(myMap.has("bar")); // true
console.log(myMap.has("baz")); // false
```

See also

- `Map`
- `Map.prototype.delete()`
- `Map.prototype.get()`
- `Map.prototype.set()`

Map.prototype.keys()

The **`keys()`** method of `Map` instances returns a new *map iterator* object that contains the keys for each element in this map in insertion order.

Try it

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map.keys();

console.log(iterator.next().value);
// Expected output: "0"

console.log(iterator.next().value);
// Expected output: 1
```

Syntax

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using keys()

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap.keys();

console.log(mapIter.next().value); // "0"
console.log(mapIter.next().value); // 1
console.log(mapIter.next().value); // {}
```

See also

- `Map.prototype.entries()`
- `Map.prototype.values()`

Map() constructor

The **`Map()`** constructor creates `Map` objects.

Syntax

```
new Map()
new Map(iterable)
```

> **Note:** `Map()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable` Optional

> If an iterable object (such as an array) is passed, all of its elements will be added to the new `Map`. Each element must be an object with two properties: `0` and `1`, which correspond to the key and value (for example, `[[1, "one"],[2, "two"]]`). If you don't specify this parameter, or its value is `null` or `undefined`, the new `Map` is empty.

Examples

Creating a new Map

```
const myMap = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);
```

See also

- Polyfill for `Map` in `core-js`
- es-shims polyfill of `Map`
- `Set`
- `WeakMap`
- `WeakSet`

Map.prototype.set()

The **`set()`** method of `Map` instances adds a new entry with a specified key and value to this `Map`, or updates an existing entry if the key already exists.

Try it

```
const map = new Map();
map.set("bar", "foo");

console.log(map.get("bar"));
// Expected output: "foo"

console.log(map.get("baz"));
// Expected output: undefined
```

Syntax

```
set(key, value)
```

Parameters

`key`

The key of the entry to add to or modify within the `Map` object. Can be any value. Object keys are compared by reference, not by value.

`value`

The value of the entry to add to or modify within the `Map` object. Can be any value.

Return value

The `Map` object.

Examples

Using set()

```
const myMap = new Map();

// Add new elements to the map
myMap.set("bar", "foo");
myMap.set(1, "foobar");

// Update an element in the map
myMap.set("bar", "baz");
```

Using the set() with chaining

Since the `set()` method returns the same `Map` object, you can chain the method call like below:

```
// Add new elements to the map with chaining.
myMap.set("bar", "foo").set(1, "foobar").set(2, "baz");
```

See also

- `Map`
- `Map.prototype.delete()`
- `Map.prototype.get()`
- `Map.prototype.has()`

Map.prototype.size

The **`size`** accessor property of `Map` instances returns the number of elements in this map.

Try it

```
const map = new Map();

map.set("a", "alpha");
map.set("b", "beta");
map.set("g", "gamma");

console.log(map.size);
// Expected output: 3
```

Description

The value of `size` is an integer representing how many entries the `Map` object has. A set accessor function for `size` is `undefined`; you cannot change this property.

Examples

Using size

```
const myMap = new Map();
myMap.set("a", "alpha");
myMap.set("b", "beta");
myMap.set("g", "gamma");

console.log(myMap.size); // 3
```

See also

- `Map`

Map.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `Map` instances implements the iterable protocol and allows `Map` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a map iterator object that yields the key-value pairs of the map in insertion order.

The initial value of this property is the same function object as the initial value of the `Map.prototype.entries` property.

Try it

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map[Symbol.iterator]();

for (const item of iterator) {
  console.log(item);
}
// Expected output: Array ["0", "foo"]
// Expected output: Array [1, "bar"]
```

Syntax

```
map[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Map.prototype.entries()`: a new iterable iterator object that yields the key-value pairs of the map.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `Map` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

for (const entry of myMap) {
  console.log(entry);
}
// ["0", "foo"]
// [1, "bar"]
// [{}, "baz"]

for (const [key, value] of myMap) {
  console.log(`${key}: ${value}`);
```

```
}
// 0: foo
// 1: bar
// [Object]: baz
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap[Symbol.iterator]();

console.log(mapIter.next().value); // ["0", "foo"]
console.log(mapIter.next().value); // [1, "bar"]
console.log(mapIter.next().value); // [Object, "baz"]
```

See also

- `Map`
- `Map.prototype.entries()`
- `Map.prototype.keys()`
- `Map.prototype.values()`
- `Symbol.iterator`
- Iteration protocols

Map[Symbol.species]

The **`Map[Symbol.species]`** static accessor property is an unused accessor property specifying how to copy `Map` objects.

Syntax

```
Map[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct copied `Map` instances.

Description

The `[Symbol.species]` accessor property returns the default constructor for `Map` objects. Subclass constructors may override it to change the constructor assignment.

> **Note:** This property is currently unused by all `Map` methods.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `Map` constructor for `Map`.

```
Map[Symbol.species]; // function Map()
```

Species in derived objects

In an instance of a custom `Map` subclass, such as `MyMap`, the `MyMap` species is the `MyMap` constructor. However, you might want to overwrite this, in order to return parent `Map` objects in your derived class methods:

```
class MyMap extends Map {
  // Overwrite MyMap species to the parent Map constructor
  static get [Symbol.species]() {
    return Map;
  }
}
```

See also

- `Map`
- `Symbol.species`

Map.prototype.values()

The **`values()`** method of `Map` instances returns a new *map iterator* object that contains the values for each element in this map in insertion order.

Try it

```
const map = new Map();
```

```
map.set("0", "foo");
map.set(1, "bar");

const iterator = map.values();

console.log(iterator.next().value);
// Expected output: "foo"

console.log(iterator.next().value);
// Expected output: "bar"
```

Syntax

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using values()

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap.values();

console.log(mapIter.next().value); // "foo"
console.log(mapIter.next().value); // "bar"
console.log(mapIter.next().value); // "baz"
```

See also

- `Map.prototype.entries()`
- `Map.prototype.keys()`

Math

The **`Math`** namespace object contains static properties and methods for mathematical constants and functions.

`Math` works with the `Number` type. It doesn't work with `BigInt`.

Description

Unlike most global objects, `Math` is not a constructor. You cannot use it with the `new` operator or invoke the `Math` object as a function. All properties and methods of `Math` are static.

> **Note:** Many `Math` functions have a precision that's *implementation-dependent*.
>
> This means that different browsers can give a different result. Even the same JavaScript engine on a different OS or architecture can give different results!

Static properties

`Math.E`

Euler's number and the base of natural logarithms; approximately `2.718`.

`Math.LN10`

Natural logarithm of `10`; approximately `2.303`.

`Math.LN2`

Natural logarithm of `2`; approximately `0.693`.

`Math.LOG10E`

Base-10 logarithm of `E`; approximately `0.434`.

`Math.LOG2E`

Base-2 logarithm of `E`; approximately `1.443`.

`Math.PI`

Ratio of a circle's circumference to its diameter; approximately `3.14159`.

`Math.SQRT1_2`

Square root of ½; approximately `0.707`.

`Math.SQRT2`

Square root of 2; approximately `1.414`.

`Math[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Math"`. This property is used in `Object.prototype.toString()`.

Static methods

`Math.abs()`

Returns the absolute value of the input.

`Math.acos()`

Returns the arccosine of the input.

`Math.acosh()`

Returns the hyperbolic arccosine of the input.

`Math.asin()`

Returns the arcsine of the input.

`Math.asinh()`

Returns the hyperbolic arcsine of a number.

`Math.atan()`

Returns the arctangent of the input.

`Math.atan2()`

Returns the arctangent of the quotient of its arguments.

`Math.atanh()`

Returns the hyperbolic arctangent of the input.

`Math.cbrt()`

Returns the cube root of the input.

`Math.ceil()`

Returns the smallest integer greater than or equal to the input.

`Math.clz32()`

Returns the number of leading zero bits of the 32-bit integer input.

`Math.cos()`

Returns the cosine of the input.

`Math.cosh()`

Returns the hyperbolic cosine of the input.

`Math.exp()`

Returns e^x, where x is the argument, and e is Euler's number (`2.718...`, the base of the natural logarithm).

`Math.expm1()`

Returns subtracting `1` from `exp(x)`.

`Math.floor()`

Returns the largest integer less than or equal to the input.

`Math.f16round()`

Returns the nearest half precision float representation of the input.

`Math.fround()`

Returns the nearest single precision float representation of the input.

`Math.hypot()`

Returns the square root of the sum of squares of its arguments.

`Math.imul()`

Returns the result of the 32-bit integer multiplication of the inputs.

`Math.log()`

Returns the natural logarithm ($\log_e$; also, $\ln$) of the input.

`Math.log10()`

Returns the base-10 logarithm of the input.

`Math.log1p()`

Returns the natural logarithm ($\log_e$; also $\ln$) of `1 + x` for the number `x`.

`Math.log2()`

Returns the base-2 logarithm of the input.

`Math.max()`

Returns the largest of zero or more numbers.

`Math.min()`

Returns the smallest of zero or more numbers.

`Math.pow()`

Returns base `x` to the exponent power `y` (that is, x^y).

`Math.random()`

Returns a pseudo-random number between `0` and `1`.

`Math.round()`

Returns the value of the input rounded to the nearest integer.

`Math.sign()`

Returns the sign of the input, indicating whether it is positive, negative, or zero.

`Math.sin()`

Returns the sine of the input.

`Math.sinh()`

Returns the hyperbolic sine of the input.

`Math.sqrt()`

Returns the positive square root of the input.

`Math.sumPrecise()`

Returns the sum of a passed iterable of numbers, avoiding floating point precision loss in intermediate results.

`Math.tan()`

Returns the tangent of the input.

`Math.tanh()`

Returns the hyperbolic tangent of the input.

`Math.trunc()`

Returns the integer portion of the input, removing any fractional digits.

Examples

Converting between degrees and radians

The trigonometric functions `sin()`, `cos()`, `tan()`, `asin()`, `acos()`, `atan()`, and `atan2()` expect (and return) angles in *radians*.

Since humans tend to think in degrees, and some functions (such as CSS transforms) can accept degrees, it is a good idea to keep functions handy that convert between the two:

```
function degToRad(degrees) {
  return degrees * (Math.PI / 180);
}

function radToDeg(rad) {
  return rad / (Math.PI / 180);
}
```

Calculating the height of an equilateral triangle

If we want to calculate the height of an equilateral triangle, and we know its side length is 100, we can use the formulae *length of the adjacent multiplied by the tangent of the angle is equal to the opposite*.

An equilateral triangle where a perpendicular of one edge is drawn from the opposite vertex, forming a right triangle with three sides marked as "adjacent", "opposite", and "hypotenuse". The angle between the "adjacent" and "hypotenuse" sides is 60 degrees.

In JavaScript, we can do this with the following:

```
50 * Math.tan(degToRad(60));
```

We use our `degToRad()` function to convert 60 degrees to radians, as `Math.tan()` expects an input value in radians.

Returning a random integer between two bounds

This can be achieved with a combination of `Math.random()` and `Math.floor()`:

```
function random(min, max) {
  const num = Math.floor(Math.random() * (max - min + 1)) + min;
  return num;
}

random(1, 10);
```

See also

- `Number`

Math.abs()

The **`Math.abs()`** static method returns the absolute value of a number.

Try it

```
function difference(a, b) {
  return Math.abs(a - b);
}

console.log(difference(3, 5));
// Expected output: 2

console.log(difference(5, 3));
// Expected output: 2

console.log(difference(1.23456, 7.89012));
// Expected output: 6.6555599999999995
```

Syntax

```
Math.abs(x)
```

Parameters

`x`

A number.

Return value

The absolute value of `x`. If `x` is negative or `-0`, returns its opposite number `-x` (which is non-negative). Otherwise, returns `x` itself. The result is therefore always a positive number or `0`.

Description

Because `abs()` is a static method of `Math`, you always use it as `Math.abs()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.abs()

```
Math.abs(-Infinity); // Infinity
Math.abs(-1); // 1
Math.abs(-0); // 0
```

```
Math.abs(0); // 0
Math.abs(1); // 1
Math.abs(Infinity); // Infinity
```

Coercion of parameter

`Math.abs()` coerces its parameter to a number. Non-coercible values will become `NaN`, making `Math.abs()` also return `NaN`.

```
Math.abs("-1"); // 1
Math.abs(-2); // 2
Math.abs(null); // 0
Math.abs(""); // 0
Math.abs([]); // 0
Math.abs([2]); // 2
Math.abs([1, 2]); // NaN
Math.abs({}); // NaN
Math.abs("string"); // NaN
Math.abs(); // NaN
```

See also

- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

Math.acos()

The **`Math.acos()`** static method returns the inverse cosine (in radians) of a number. That is,

$$\forall x \in [-1, 1],\ \texttt{Math.acos}\ (x) = \arccos(x) = \text{the unique } y \in [0, \pi] \text{ such that } \cos(y) = x$$

Try it

```
// Calculates angle of a right-angle triangle in radians
function calcAngle(adjacent, hypotenuse) {
  return Math.acos(adjacent / hypotenuse);
}

console.log(calcAngle(8, 10));
// Expected output: 0.6435011087932843

console.log(calcAngle(5, 3));
// Expected output: NaN
```

Syntax

```
Math.acos(x)
```

Parameters

`x`

A number between -1 and 1, inclusive, representing the angle's cosine value.

Return value

The inverse cosine (angle in radians between 0 and π, inclusive) of `x`. If `x` is less than -1 or greater than 1, returns `NaN`.

Description

Because `acos()` is a static method of `Math`, you always use it as `Math.acos()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.acos()

```
Math.acos(-2); // NaN
Math.acos(-1); // 3.141592653589793 (π)
Math.acos(0); // 1.5707963267948966 (π/2)
Math.acos(0.5); // 1.0471975511965979 (π/3)
Math.acos(1); // 0
Math.acos(2); // NaN
```

See also

- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`

- `Math.tan()`
- CSS `acos()` function

Math.acosh()

The **`Math.acosh()`** static method returns the inverse hyperbolic cosine of a number. That is,

$$\forall x \geq 1,\ \texttt{Math.acosh}(x) = \operatorname{arcosh}(x) = \text{the unique } y \geq 0 \text{ such that } \cosh(y) = x = \ln\left(x + \sqrt{x^2 - 1}\right)$$

Try it

```
console.log(Math.acosh(0.999999999999));
// Expected output: NaN

console.log(Math.acosh(1));
// Expected output: 0

console.log(Math.acosh(2));
// Expected output: 1.3169578969248166

console.log(Math.acosh(2.5));
// Expected output: 1.566799236972411
```

Syntax

```
Math.acosh(x)
```

Parameters

`x`

A number greater than or equal to 1.

Return value

The inverse hyperbolic cosine of `x`. If `x` is less than 1, returns `NaN`.

Description

Because `acosh()` is a static method of `Math`, you always use it as `Math.acosh()`, rather than as a method of a `Math` object you created (`Math` is no constructor).

Examples

Using Math.acosh()

```
Math.acosh(0); // NaN
Math.acosh(1); // 0
Math.acosh(2); // 1.3169578969248166
Math.acosh(Infinity); // Infinity
```

See also

- Polyfill of `Math.acosh` in `core-js`
- es-shims polyfill of `Math.acosh`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

Math.asin()

The **`Math.asin()`** static method returns the inverse sine (in radians) of a number. That is,

$$\forall x \in [-1, 1],\ \texttt{Math.asin}(x) = \arcsin(x) = \text{the unique } y \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \text{ such that } \sin(y) = x$$

Try it

```
// Calculates angle of a right-angle triangle in radians
function calcAngle(opposite, hypotenuse) {
  return Math.asin(opposite / hypotenuse);
}

console.log(calcAngle(6, 10));
// Expected output: 0.6435011087932844

console.log(calcAngle(5, 3));
// Expected output: NaN
```

Syntax

```
Math.asin(x)
```

Parameters

`x`

A number between -1 and 1, inclusive, representing the angle's sine value.

Return value

The inverse sine (angle in radians between $-\frac{\pi}{2}$ and $\frac{\pi}{2}$, inclusive) of `x`. If `x` is less than -1 or greater than 1, returns `NaN`.

Description

Because `asin()` is a static method of `Math`, you always use it as `Math.asin()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.asin()

```
Math.asin(-2); // NaN
Math.asin(-1); // -1.5707963267948966 (-π/2)
Math.asin(-0); // -0
Math.asin(0); // 0
Math.asin(0.5); // 0.5235987755982989 (π/6)
Math.asin(1); // 1.5707963267948966 (π/2)
Math.asin(2); // NaN
```

See also

- `Math.acos()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`
- CSS `asin()` function

Math.asinh()

The **`Math.asinh()`** static method returns the inverse hyperbolic sine of a number. That is,

$$\begin{aligned}\texttt{Math.asinh}\,(\texttt{x}) &= \operatorname{arsinh}(x) = \text{the unique } y \text{ such that } \sinh(y) = x \\ &= \ln\left(x + \sqrt{x^2+1}\right)\end{aligned}$$

Try it

```
console.log(Math.asinh(1));
// Expected output: 0.881373587019543

console.log(Math.asinh(0));
// Expected output: 0

console.log(Math.asinh(-1));
// Expected output: -0.881373587019543

console.log(Math.asinh(2));
// Expected output: 1.4436354751788103
```

Syntax

```
Math.asinh(x)
```

Parameters

`x`

A number.

Return value

The inverse hyperbolic sine of `x`.

Description

Because `asinh()` is a static method of `Math`, you always use it as `Math.asinh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.asinh()

```
Math.asinh(-Infinity); // -Infinity
Math.asinh(-1); // -0.881373587019543
Math.asinh(-0); // -0
Math.asinh(0); // 0
Math.asinh(1); // 0.881373587019543
Math.asinh(Infinity); // Infinity
```

See also

- Polyfill of `Math.asinh` in `core-js`
- es-shims polyfill of `Math.asinh`
- `Math.acosh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

Math.atan()

The **`Math.atan()`** static method returns the inverse tangent (in radians) of a number, that is

$$\mathrm{Math.atan}(x) = \arctan(x) = \text{the unique } y \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \text{ such that } \tan(y) = x$$

Try it

```
// Calculates angle of a right-angle triangle in radians
function calcAngle(opposite, adjacent) {
  return Math.atan(opposite / adjacent);
}

console.log(calcAngle(8, 10));
// Expected output: 0.6747409422235527

console.log(calcAngle(5, 3));
// Expected output: 1.0303768265243125
```

Syntax

```
Math.atan(x)
```

Parameters

`x`

A number.

Return value

The inverse tangent (angle in radians between $-\frac{\pi}{2}$ and $\frac{\pi}{2}$, inclusive) of `x`. If `x` is `Infinity`, it returns $\frac{\pi}{2}$. If `x` is `-Infinity`, it returns $-\frac{\pi}{2}$.

Description

Because `atan()` is a static method of `Math`, you always use it as `Math.atan()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.atan()

```
Math.atan(-Infinity); // -1.5707963267948966 (-π/2)
Math.atan(-0); // -0
Math.atan(0); // 0
Math.atan(1); // 0.7853981633974483  (π/4)
Math.atan(Infinity); // 1.5707963267948966  (π/2)

// The angle that the line (0,0) -- (x,y) forms with the x-axis in a Cartesian coordinate system
const theta = (x, y) => Math.atan(y / x);
```

Note that you may want to avoid the `theta` function and use `Math.atan2()` instead, which has a wider range (between -π and π) and avoids outputting `NaN` for cases such as when `x` is `0`.

See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`
- CSS `atan()` function

Math.atan2()

The **`Math.atan2()`** static method returns the angle in the plane (in radians) between the positive x-axis and the ray from (0, 0) to the point (x, y), for `Math.atan2(y, x)`.

Try it

```
function calcAngleDegrees(x, y) {
  return (Math.atan2(y, x) * 180) / Math.PI;
}

console.log(calcAngleDegrees(5, 5));
// Expected output: 45

console.log(calcAngleDegrees(10, 10));
// Expected output: 45

console.log(calcAngleDegrees(0, 10));
// Expected output: 90
```

Syntax

```
Math.atan2(y, x)
```

Parameters

`y`

The y coordinate of the point.

`x`

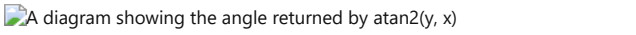
The x coordinate of the point.

Return value

The angle in radians (between -π and π, inclusive) between the positive x-axis and the ray from (0, 0) to the point (x, y).

Description

The `Math.atan2()` method measures the counterclockwise angle θ, in radians, between the positive x-axis and the point `(x, y)`. Note that the arguments to this function pass the y-coordinate first and the x-coordinate second.



`Math.atan2()` is passed separate `x` and `y` arguments, while `Math.atan()` is passed the ratio of those two arguments. `Math.atan2(y, x)` differs from `Math.atan(y / x)` in the following cases:

x	y	Math.atan2(y, x)	Math.atan(y / x)
Infinity	Infinity	π / 4	NaN
Infinity	-Infinity	-π / 4	NaN
-Infinity	Infinity	3π / 4	NaN
-Infinity	-Infinity	-3π / 4	NaN
0	0	0	NaN
0	-0	-0	NaN
< 0 (including -0)	0	π	0
< 0 (including -0)	-0	-π	0
-Infinity	> 0	π	-0
-0	> 0	π / 2	-π / 2
-Infinity	< 0	-π	0
-0	< 0	-π / 2	π / 2

In addition, for points in the second and third quadrants (`x < 0`), `Math.atan2()` would output an angle less than $-\frac{\pi}{2}$ or greater than $\frac{\pi}{2}$.

Because `atan2()` is a static method of `Math`, you always use it as `Math.atan2()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.atan2()

```
Math.atan2(90, 15); // 1.4056476493802699
Math.atan2(15, 90); // 0.16514867741462683
```

Difference between Math.atan2(y, x) and Math.atan(y / x)

The following script prints all inputs that produce a difference between `Math.atan2(y, x)` and `Math.atan(y / x)`.

```
const formattedNumbers = new Map([
  [-Math.PI, "-π"],
  [(-3 * Math.PI) / 4, "-3π/4"],
```

```
  [-Math.PI / 2, "-π/2"],
  [-Math.PI / 4, "-π/4"],
  [Math.PI / 4, "π/4"],
  [Math.PI / 2, "π/2"],
  [(3 * Math.PI) / 4, "3π/4"],
  [Math.PI, "π"],
  [-Infinity, "-∞"],
  [Infinity, "∞"],
]);

function format(template, ...args) {
  return String.raw(
    { raw: template },
    ...args.map((num) =>
      (Object.is(num, -0)
        ? "-0"
        : (formattedNumbers.get(num) ?? String(num))
      ).padEnd(5),
    ),
  );
}

console.log(`| x     | y     | atan2 | atan  |
|-------|-------|-------|-------|`);

for (const x of [-Infinity, -1, -0, 0, 1, Infinity]) {
  for (const y of [-Infinity, -1, -0, 0, 1, Infinity]) {
    const atan2 = Math.atan2(y, x);
    const atan = Math.atan(y / x);
    if (!Object.is(atan2, atan)) {
      console.log(format`| ${x} | ${y} | ${atan2} | ${atan} |`);
    }
  }
}
```

The output is:

```
| x     | y     | atan2 | atan  |
|-------|-------|-------|-------|
| -∞    | -∞    | -3π/4 | NaN   |
| -∞    | -1    | -π    | 0     |
| -∞    | -0    | -π    | 0     |
| -∞    | 0     | π     | -0    |
| -∞    | 1     | π     | -0    |
| -∞    | ∞     | 3π/4  | NaN   |
| -1    | -∞    | -π/2  | π/2   |
| -1    | -1    | -3π/4 | π/4   |
| -1    | -0    | -π    | 0     |
| -1    | 0     | π     | -0    |
| -1    | 1     | 3π/4  | -π/4  |
| -1    | ∞     | π/2   | -π/2  |
| -0    | -∞    | -π/2  | π/2   |
| -0    | -1    | -π/2  | π/2   |
| -0    | -0    | -π    | NaN   |
| -0    | 0     | π     | NaN   |
| -0    | 1     | π/2   | -π/2  |
| -0    | ∞     | π/2   | -π/2  |
| 0     | -0    | -0    | NaN   |
| 0     | 0     | 0     | NaN   |
| ∞     | -∞    | -π/4  | NaN   |
| ∞     | ∞     | π/4   | NaN   |
```

See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`
- CSS `atan2()` function

Math.atanh()

The **`Math.atanh()`** static method returns the inverse hyperbolic tangent of a number. That is,

$$\forall x \in (-1, 1),\ \mathrm{Math.atanh}(x) = \operatorname{artanh}(x) = \text{the unique } y \text{ such that } \tanh(y) = x = \frac{1}{2}\ln\left(\frac{1+x}{1-x}\right)$$

Try it

```
console.log(Math.atanh(-1));
// Expected output: -Infinity
```

```
console.log(Math.atanh(0));
// Expected output: 0

console.log(Math.atanh(0.5));
// Expected output: 0.549306144334055 (approximately)

console.log(Math.atanh(1));
// Expected output: Infinity
```

Syntax

```
Math.atanh(x)
```

Parameters

`x`

A number between -1 and 1, inclusive.

Return value

The inverse hyperbolic tangent of `x`. If `x` is 1, returns `Infinity`. If `x` is -1, returns `-Infinity`. If `x` is less than -1 or greater than 1, returns `NaN`.

Description

Because `atanh()` is a static method of `Math`, you always use it as `Math.atanh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.atanh()

```
Math.atanh(-2); // NaN
Math.atanh(-1); // -Infinity
Math.atanh(-0); // -0
Math.atanh(0); // 0
Math.atanh(0.5); // 0.5493061443340548
Math.atanh(1); // Infinity
Math.atanh(2); // NaN
```

See also

- Polyfill of `Math.atanh` in `core-js`
- es-shims polyfill of `Math.atanh`
- `Math.acosh()`
- `Math.asinh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

Math.cbrt()

The **`Math.cbrt()`** static method returns the cube root of a number. That is

$$\texttt{Math.cbrt}\,(x) = \sqrt[3]{x} = \text{the unique } y \text{ such that } y^3 = x$$

Try it

```
console.log(Math.cbrt(-1));
// Expected output: -1

console.log(Math.cbrt(1));
// Expected output: 1

console.log(Math.cbrt(Infinity));
// Expected output: Infinity

console.log(Math.cbrt(64));
// Expected output: 4
```

Syntax

```
Math.cbrt(x)
```

Parameters

`x`

A number.

Return value

The cube root of `x`.

Description

Because `cbrt()` is a static method of `Math`, you always use it as `Math.cbrt()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.cbrt()

```
Math.cbrt(-Infinity); // -Infinity
Math.cbrt(-1); // -1
Math.cbrt(-0); // -0
Math.cbrt(0); // 0
Math.cbrt(1); // 1
Math.cbrt(2); // 1.2599210498948732
Math.cbrt(Infinity); // Infinity
```

See also

- Polyfill of `Math.cbrt` in `core-js`
- es-shims polyfill of `Math.cbrt`
- `Math.pow()`
- `Math.sqrt()`

Math.ceil()

The **`Math.ceil()`** static method always rounds up and returns the smallest integer greater than or equal to a given number.

Try it

```
console.log(Math.ceil(0.95));
// Expected output: 1

console.log(Math.ceil(4));
// Expected output: 4

console.log(Math.ceil(7.004));
// Expected output: 8

console.log(Math.ceil(-7.004));
// Expected output: -7
```

Syntax

```
Math.ceil(x)
```

Parameters

`x`

A number.

Return value

The smallest integer greater than or equal to `x`. It's the same value as `-Math.floor(-x)`.

Description

Because `ceil()` is a static method of `Math`, you always use it as `Math.ceil()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.ceil()

```
Math.ceil(-Infinity); // -Infinity
Math.ceil(-7.004); // -7
Math.ceil(-4); // -4
Math.ceil(-0.95); // -0
Math.ceil(-0); // -0
Math.ceil(0); // 0
Math.ceil(0.95); // 1
Math.ceil(4); // 4
Math.ceil(7.004); // 8
Math.ceil(Infinity); // Infinity
```

See also

- `Math.abs()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

Math.clz32()

The **`Math.clz32()`** static method returns the number of leading zero bits in the 32-bit binary representation of a number.

Try it

```
// 00000000000000000000000000000001
console.log(Math.clz32(1));
// Expected output: 31

// 00000000000000000000000000000100
console.log(Math.clz32(4));
// Expected output: 29

// 00000000000000000000001111101000
console.log(Math.clz32(1000));
// Expected output: 22
```

Syntax

```
Math.clz32(x)
```

Parameters

`x`

A number.

Return value

The number of leading zero bits in the 32-bit binary representation of `x`.

Description

`clz32` is short for **C**ount**L**eading**Z**eros**32**.

If `x` is not a number, it will be converted to a number first, then converted to a 32-bit unsigned integer.

If the converted 32-bit unsigned integer is `0`, `32` is returned, because all bits are `0`. If the most significant bit is `1` (i.e., the number is greater than or equal to 2^{31}), `0` is returned.

This function is particularly useful for systems that compile to JS, like Emscripten.

Examples

Using Math.clz32()

```
Math.clz32(1); // 31
Math.clz32(1000); // 22
Math.clz32(); // 32

const stuff = [
  NaN,
  Infinity,
  -Infinity,
  0,
  -0,
  false,
  null,
  undefined,
  "foo",
  {},
  [],
];
stuff.every((n) => Math.clz32(n) === 32); // true

Math.clz32(true); // 31
Math.clz32(3.5); // 30
```

Implementing Count Leading Ones and beyond

At present, there is no `Math.clon` for "Count Leading Ones" (named "clon", not "clo", because "clo" and "clz" are too similar especially for non-English-speaking people). However, a `clon` function can easily be created by inverting the bits of a number and passing the result to `Math.clz32`. Doing this will work because the inverse of 1 is 0 and vice versa. Thus, inverting the bits will inverse the measured quantity of 0's (from `Math.clz32`), thereby making `Math.clz32` count the number of ones instead of counting the number of zeros.

Consider the following 32-bit word:

```
const a = 32776; // 00000000000000001000000000001000 (16 leading zeros)
Math.clz32(a); // 16

const b = ~32776; // 11111111111111110111111111110111 (32776 inverted, 0 leading zeros)
Math.clz32(b); // 0 (this is equal to how many leading one's there are in a)
```

Using this logic, a `clon` function can be created as follows:

```
const clz = Math.clz32;

function clon(integer) {
  return clz(~integer);
}
```

Further, this technique could be extended to create a jumpless "Count Trailing Zeros" function, as seen below. The `ctrz` function takes a bitwise AND of the integer with its two's complement. By how two's complement works, all trailing zeros will be converted to ones, and then when adding 1, it would be carried over until the first `0` (which was originally a `1`) is reached. All bits higher than this one stay the same and are inverses of the original integer's bits. Therefore, when doing bitwise AND with the original integer, all higher bits become `0`, which can be counted with `clz`. The number of trailing zeros, plus the first `1` bit, plus the leading bits that were counted by `clz`, total to 32.

```
function ctrz(integer) {
  integer >>>= 0; // coerce to Uint32
  if (integer === 0) {
    // skipping this step would make it return -1
    return 32;
  }
  integer &= -integer; // equivalent to `int = int & (~int + 1)`
  return 31 - clz(integer);
}
```

Then we can define a "Count Trailing Ones" function like so:

```
function ctron(integer) {
  return ctrz(~integer);
}
```

These helper functions can be made into an asm.js module for a potential performance improvement.

```
const countTrailsMethods = (function (stdlib, foreign, heap) {
  "use asm";
  const clz = stdlib.Math.clz32;

  // count trailing zeros
  function ctrz(integer) {
    integer = integer | 0; // coerce to an integer
    if ((integer | 0) == 0) {
      // skipping this step would make it return -1
      return 32;
    }
    // Note: asm.js doesn't have compound assignment operators such as &=
    integer = integer & -integer; // equivalent to `int = int & (~int + 1)`
    return (31 - clz(integer)) | 0;
  }

  // count trailing ones
  function ctron(integer) {
    integer = integer | 0; // coerce to an integer
    return ctrz(~integer) | 0;
  }

  // asm.js demands plain objects:
  return { ctrz: ctrz, ctron: ctron };
})(window, null, null);

const { ctrz, ctron } = countTrailsMethods;
```

See also

- Polyfill of `Math.clz32` in `core-js`
- es-shims polyfill of `Math.clz32`
- `Math`
- `Math.imul`

Math.cos()

The **`Math.cos()`** static method returns the cosine of a number in radians.

Try it

```
function getCircleX(radians, radius) {
  return Math.cos(radians) * radius;
}

console.log(getCircleX(1, 10));
// Expected output: 5.403023058681398

console.log(getCircleX(2, 10));
// Expected output: -4.161468365471424
```

```
console.log(getCircleX(Math.PI, 10));
// Expected output: -10
```

Syntax

```
Math.cos(x)
```

Parameters

`x`

A number representing an angle in radians.

Return value

The cosine of `x`, between -1 and 1, inclusive. If `x` is `Infinity`, `-Infinity`, or `NaN`, returns `NaN`.

Description

Because `cos()` is a static method of `Math`, you always use it as `Math.cos()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.cos()

```
Math.cos(-Infinity); // NaN
Math.cos(-0); // 1
Math.cos(0); // 1
Math.cos(1); // 0.5403023058681398
Math.cos(Math.PI); // -1
Math.cos(2 * Math.PI); // 1
Math.cos(Infinity); // NaN
```

See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.sin()`
- `Math.tan()`
- CSS `cos()` function

Math.cosh()

The **`Math.cosh()`** static method returns the hyperbolic cosine of a number. That is,

$$\mathrm{Math.cosh}\left(\mathrm{x}\right) = \cosh(x) = \frac{\mathrm{e}^{x} + \mathrm{e}^{-x}}{2}$$

Try it

```
console.log(Math.cosh(0));
// Expected output: 1

console.log(Math.cosh(1));
// Expected output: 1.543080634815244 (approximately)

console.log(Math.cosh(-1));
// Expected output: 1.543080634815244 (approximately)

console.log(Math.cosh(2));
// Expected output: 3.7621956910836314
```

Syntax

```
Math.cosh(x)
```

Parameters

`x`

A number.

Return value

The hyperbolic cosine of `x`.

Description

Because `cosh()` is a static method of `Math`, you always use it as `Math.cosh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.cosh()

```
Math.cosh(-Infinity); // Infinity
Math.cosh(-1); // 1.5430806348152437
Math.cosh(-0); // 1
Math.cosh(0); // 1
Math.cosh(1); // 1.5430806348152437
Math.cosh(Infinity); // Infinity
```

See also

- Polyfill of `Math.cosh` in `core-js`
- es-shims polyfill of `Math.cosh`
- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.sinh()`
- `Math.tanh()`

Math.E

The **`Math.E`** static data property represents Euler's number, the base of natural logarithms, e, which is approximately 2.718.

Try it

```
function compoundOneYear(interestRate, currentVal) {
  return currentVal * Math.E ** interestRate;
}

console.log(Math.E);
// Expected output: 2.718281828459045

console.log((1 + 1 / 1000000) ** 1000000);
// Expected output: 2.718280469 (approximately)

console.log(compoundOneYear(0.05, 100));
// Expected output: 105.12710963760242
```

Value

$$\texttt{Math.E} = e \approx 2.718$$

Property attributes of `Math.E`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `E` is a static property of `Math`, you always use it as `Math.E`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.E

The following function returns e:

```
function getNapier() {
  return Math.E;
}

getNapier(); // 2.718281828459045
```

See also

- `Math.exp()`
- `Math.log()`
- `Math.log1p()`

Math.exp()

The **`Math.exp()`** static method returns e raised to the power of a number. That is

$$\texttt{Math.exp}(x) = e^x$$

Try it

```
console.log(Math.exp(0));
// Expected output: 1

console.log(Math.exp(1));
// Expected output: 2.718281828459 (approximately)
```

```
console.log(Math.exp(-1));
// Expected output: 0.36787944117144233

console.log(Math.exp(2));
// Expected output: 7.38905609893065
```

Syntax

```
Math.exp(x)
```

Parameters

`x`

A number.

Return value

A nonnegative number representing e^x, where e is the base of the natural logarithm.

Description

Because `exp()` is a static method of `Math`, you always use it as `Math.exp()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Beware that `e` to the power of a number very close to 0 will be very close to 1 and suffer from loss of precision. In this case, you may want to use `Math.expm1` instead, and obtain a much higher-precision fractional part of the answer.

Examples

Using Math.exp()

```
Math.exp(-Infinity); // 0
Math.exp(-1); // 0.36787944117144233
Math.exp(0); // 1
Math.exp(1); // 2.718281828459045
Math.exp(Infinity); // Infinity
```

See also

- `Math.E`
- `Math.expm1()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

Math.expm1()

The **`Math.expm1()`** static method returns e raised to the power of a number, subtracted by 1. That is

$$\texttt{Math.expm1}\,(x) = e^x - 1$$

Try it

```
console.log(Math.expm1(0));
// Expected output: 0

console.log(Math.expm1(1));
// Expected output: 1.718281828459045

console.log(Math.expm1(-1));
// Expected output: -0.6321205588285577

console.log(Math.expm1(2));
// Expected output: 6.38905609893065
```

Syntax

```
Math.expm1(x)
```

Parameters

`x`

A number.

Return value

A number representing e^x - 1, where e is the base of the natural logarithm.

Description

For very small values of *x*, adding 1 can reduce or eliminate precision. The double floats used in JS give you about 15 digits of precision. 1 + 1e-15 = 1.000000000000001, but 1 + 1e-16 = 1.000000000000000 and therefore exactly 1.0 in that arithmetic, because digits past 15 are rounded off.

When you calculate e^x, where x is a number very close to 0, you should get an answer very close to 1 + x because: $\lim_{x \to 0} \frac{e^x - 1}{x} = 1$. If you calculate `Math.exp(1.1111111111e-15) - 1`, you should get an answer close to `1.1111111111e-15`. Instead, due to the highest significant figure in the result of `Math.exp` being the units digit `1`, the final value ends up being `1.1102230246251565e-15`, with only 3 correct digits. If you calculate `Math.expm1(1.1111111111e-15)` instead, you will get a much more accurate answer, `1.1111111111000007e-15`, with 11 correct digits of precision.

Because `expm1()` is a static method of `Math`, you always use it as `Math.expm1()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.expm1()

```
Math.expm1(-Infinity); // -1
Math.expm1(-1); // -0.6321205588285577
Math.expm1(-0); // -0
Math.expm1(0); // 0
Math.expm1(1); // 1.718281828459045
Math.expm1(Infinity); // Infinity
```

See also

- Polyfill of `Math.expm1` in `core-js`
- `Math.E`
- `Math.exp()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

Math.f16round()

The **`Math.f16round()`** static method returns the nearest 16-bit half precision float representation of a number.

Try it

```
console.log(Math.f16round(5.5));
// Expected output: 5.5

console.log(Math.f16round(5.05));
// Expected output: 5.05078125

console.log(Math.f16round(5));
// Expected output: 5

console.log(Math.f16round(-5.05));
// Expected output: -5.05078125
```

Syntax

```
Math.f16round(doubleFloat)
```

Parameters

`doubleFloat`

A number.

Return value

The nearest 16-bit half precision float representation of `doubleFloat`.

Description

`Math.f16round` is the 16-bit counterpart of `Math.fround()`. It is intended to smooth some rough edges when interacting with float16 numbers, such as when reading from a `Float16Array`. Internally, JavaScript continues to treat the number as a 64-bit float, it just performs a "round to even" on the 10th bit of the mantissa, and sets all following mantissa bits to `0`. If the number is outside the range of a 16-bit float, `Infinity` or `-Infinity` is returned.

Because `f16round()` is a static method of `Math`, you always use it as `Math.f16round()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.f16round()

The number 1.5 can be precisely represented in the binary numeral system, and is identical in 16-bit and 64-bit:

```
Math.f16round(1.5); // 1.5
```

```
Math.f16round(1.5) === 1.5; // true
```

However, the number 1.337 cannot be precisely represented in the binary numeral system, so it differs in 16-bit and 64-bit:

```
Math.f16round(1.337); // 1.3369140625
Math.f16round(1.337) === 1.337; // false
```

100000 is too big for a 16-bit float, so `Infinity` is returned:

```
Math.f16round(100000); // Infinity
```

See also

- Polyfill of `Math.f16round` in `core-js`
- es-shims polyfill of `Math.f16round`
- `Math.fround()`
- `Math.round()`

Math.floor()

The **`Math.floor()`** static method always rounds down and returns the largest integer less than or equal to a given number.

Try it

```
console.log(Math.floor(5.95));
// Expected output: 5

console.log(Math.floor(5.05));
// Expected output: 5

console.log(Math.floor(5));
// Expected output: 5

console.log(Math.floor(-5.05));
// Expected output: -6
```

Syntax

```
Math.floor(x)
```

Parameters

`x`

A number.

Return value

The largest integer smaller than or equal to `x`. It's the same value as `-Math.ceil(-x)`.

Description

Because `floor()` is a static method of `Math`, you always use it as `Math.floor()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.floor()

```
Math.floor(-Infinity); // -Infinity
Math.floor(-45.95); // -46
Math.floor(-45.05); // -46
Math.floor(-0); // -0
Math.floor(0); // 0
Math.floor(4); // 4
Math.floor(45.05); // 45
Math.floor(45.95); // 45
Math.floor(Infinity); // Infinity
```

Decimal adjustment

In this example, we implement a method called `decimalAdjust()` that is an enhancement method of `Math.floor()`, `Math.ceil()`, and `Math.round()`. While the three `Math` functions always adjust the input to the units digit, `decimalAdjust` accepts an `exp` parameter that specifies the number of digits to the left of the decimal point to which the number should be adjusted. For example, `-1` means it would leave one digit after the decimal point (as in "$\times 10^{-1}$"). In addition, it allows you to select the means of adjustment — `round`, `floor`, or `ceil` — through the `type` parameter.

It does so by multiplying the number by a power of 10, then rounding the result to the nearest integer, then dividing by the power of 10. To better preserve precision, it takes advantage of Number's `toString()` method, which represents large or small numbers in scientific notation (like `6.02e23`).

```
/**
 * Adjusts a number to the specified digit.
 *
```

```
 * @param {"round" | "floor" | "ceil"} type The type of adjustment.
 * @param {number} value The number.
 * @param {number} exp The exponent (the 10 logarithm of the adjustment base).
 * @returns {number} The adjusted value.
 */
function decimalAdjust(type, value, exp) {
  type = String(type);
  if (!["round", "floor", "ceil"].includes(type)) {
    throw new TypeError(
      "The type of decimal adjustment must be one of 'round', 'floor', or 'ceil'.",
    );
  }
  exp = Number(exp);
  value = Number(value);
  if (exp % 1 !== 0 || Number.isNaN(value)) {
    return NaN;
  } else if (exp === 0) {
    return Math[type](value);
  }
  const [magnitude, exponent = 0] = value.toString().split("e");
  const adjustedValue = Math[type](`${magnitude}e${exponent - exp}`);
  // Shift back
  const [newMagnitude, newExponent = 0] = adjustedValue.toString().split("e");
  return Number(`${newMagnitude}e${Number(newExponent) + exp}`);
}

// Decimal round
const round10 = (value, exp) => decimalAdjust("round", value, exp);
// Decimal floor
const floor10 = (value, exp) => decimalAdjust("floor", value, exp);
// Decimal ceil
const ceil10 = (value, exp) => decimalAdjust("ceil", value, exp);

// Round
round10(55.55, -1); // 55.6
round10(55.549, -1); // 55.5
round10(55, 1); // 60
round10(54.9, 1); // 50
round10(-55.55, -1); // -55.5
round10(-55.551, -1); // -55.6
round10(-55, 1); // -50
round10(-55.1, 1); // -60
// Floor
floor10(55.59, -1); // 55.5
floor10(59, 1); // 50
floor10(-55.51, -1); // -55.6
floor10(-51, 1); // -60
// Ceil
ceil10(55.51, -1); // 55.6
ceil10(51, 1); // 60
ceil10(-55.59, -1); // -55.5
ceil10(-59, 1); // -50
```

See also

- `Math.abs()`
- `Math.ceil()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

Math.fround()

The **`Math.fround()`** static method returns the nearest 32-bit single precision float representation of a number.

Try it

```
console.log(Math.fround(5.5));
// Expected output: 5.5

console.log(Math.fround(5.05));
// Expected output: 5.050000190734863

console.log(Math.fround(5));
// Expected output: 5

console.log(Math.fround(-5.05));
// Expected output: -5.050000190734863
```

Syntax

```
Math.fround(doubleFloat)
```

Parameters

`doubleFloat`

A number.

Return value

The nearest 32-bit single precision float representation of `doubleFloat`.

Description

JavaScript uses 64-bit double floating-point numbers internally, which offer a very high precision. However, sometimes you may be working with 32-bit floating-point numbers, for example if you are reading values from a `Float32Array`. This can create confusion: checking a 64-bit float and a 32-bit float for equality may fail even though the numbers are seemingly identical.

To solve this, `Math.fround()` can be used to cast the 64-bit float to a 32-bit float. Internally, JavaScript continues to treat the number as a 64-bit float, it just performs a "round to even" on the 23rd bit of the mantissa, and sets all following mantissa bits to `0`. If the number is outside the range of a 32-bit float, `Infinity` or `-Infinity` is returned.

Because `fround()` is a static method of `Math`, you always use it as `Math.fround()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.fround()

The number 1.5 can be precisely represented in the binary numeral system, and is identical in 32-bit and 64-bit:

```
Math.fround(1.5); // 1.5
Math.fround(1.5) === 1.5; // true
```

However, the number 1.337 cannot be precisely represented in the binary numeral system, so it differs in 32-bit and 64-bit:

```
Math.fround(1.337); // 1.3370000123977661
Math.fround(1.337) === 1.337; // false
```

2^{150} is too big for a 32-bit float, so `Infinity` is returned:

```
2 ** 150; // 1.42724769270596e+45
Math.fround(2 ** 150); // Infinity
```

See also

- Polyfill of `Math.fround` in `core-js`
- es-shims polyfill of `Math.fround`
- `Math.round()`

Math.hypot()

The **`Math.hypot()`** static method returns the square root of the sum of squares of its arguments. That is,

$$\text{Math.hypot}\,(v_1, v_2, \ldots, v_n) = \sqrt{\sum_{i=1}^{n} v_i^2} = \sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}$$

Try it

```
console.log(Math.hypot(3, 4));
// Expected output: 5

console.log(Math.hypot(5, 12));
// Expected output: 13

console.log(Math.hypot(3, 4, 5));
// Expected output: 7.0710678118654755

console.log(Math.hypot(-5));
// Expected output: 5
```

Syntax

```
Math.hypot()
Math.hypot(value1)
Math.hypot(value1, value2)
Math.hypot(value1, value2, /* …, */ valueN)
```

Parameters

`value1, ..., valueN`

Numbers.

Return value

The square root of the sum of squares of the given arguments. Returns `Infinity` if any of the arguments is ±Infinity. Otherwise, if at least one of the arguments is or is converted to `NaN`, returns `NaN`. Returns `0` if no arguments are given or all arguments are ±0.

Description

Calculating the hypotenuse of a right triangle, or the magnitude of a complex number, uses the formula `Math.sqrt(v1*v1 + v2*v2)`, where v1 and v2 are the lengths of the triangle's legs, or the complex number's real and complex components. The corresponding distance in 2 or more dimensions can be calculated by adding more squares under the square root: `Math.sqrt(v1*v1 + v2*v2 + v3*v3 + v4*v4)`.

This function makes this calculation easier and faster; you call `Math.hypot(v1, v2)`, or `Math.hypot(v1, /* …, */, vN)`.

`Math.hypot` also avoids overflow/underflow problems if the magnitude of your numbers is very large. The largest number you can represent in JS is `Number.MAX_VALUE`, which is around 10^{308}. If your numbers are larger than about 10^{154}, taking the square of them will result in Infinity. For example, `Math.sqrt(1e200*1e200 + 1e200*1e200) = Infinity`. If you use `hypot()` instead, you get a better answer: `Math.hypot(1e200, 1e200) = 1.4142...e+200`. This is also true with very small numbers. `Math.sqrt(1e-200*1e-200 + 1e-200*1e-200) = 0`, but `Math.hypot(1e-200, 1e-200) = 1.4142...e-200`.

With one argument, `Math.hypot()` is equivalent to `Math.abs()`. `Math.hypot.length` is 2, which weakly signals that it's designed to handle at least two parameters.

Because `hypot()` is a static method of `Math`, you always use it as `Math.hypot()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.hypot()

```
Math.hypot(3, 4); // 5
Math.hypot(3, 4, 5); // 7.0710678118654755
Math.hypot(); // 0
Math.hypot(NaN); // NaN
Math.hypot(NaN, Infinity); // Infinity
Math.hypot(3, 4, "foo"); // NaN, since +'foo' => NaN
Math.hypot(3, 4, "5"); // 7.0710678118654755, +'5' => 5
Math.hypot(-3); // 3, the same as Math.abs(-3)
```

See also

- Polyfill of `Math.hypot` in `core-js`
- `Math.abs()`
- `Math.pow()`
- `Math.sqrt()`

Math.imul()

The **`Math.imul()`** static method returns the result of the C-like 32-bit multiplication of the two parameters.

Try it

```
console.log(Math.imul(3, 4));
// Expected output: 12

console.log(Math.imul(-5, 12));
// Expected output: -60

console.log(Math.imul(0xffffffff, 5));
// Expected output: -5

console.log(Math.imul(0xfffffffe, 5));
// Expected output: -10
```

Syntax

```
Math.imul(a, b)
```

Parameters

`a`

First number.

`b`

Second number.

Return value

The result of the C-like 32-bit multiplication of the given arguments.

Description

`Math.imul()` allows for 32-bit integer multiplication with C-like semantics. This feature is useful for projects like Emscripten.

Because `imul()` is a static method of `Math`, you always use it as `Math.imul()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

If you use normal JavaScript floating point numbers in `imul()`, you will experience a degrade in performance. This is because of the costly conversion from a floating point to an integer for multiplication, and then converting the multiplied integer back into a floating point. However, with asm.js, which allows JIT-optimizers to more confidently use integers in JavaScript, multiplying two numbers stored internally as integers (which is only possible with asm.js) with `imul()` could be potentially more performant.

Examples

Using Math.imul()

```
Math.imul(2, 4); // 8
Math.imul(-1, 8); // -8
Math.imul(-2, -2); // 4
Math.imul(0xffffffff, 5); // -5
Math.imul(0xfffffffe, 5); // -10
```

See also

- Polyfill of `Math.imul` in `core-js`
- es-shims polyfill of `Math.imul`
- Emscripten on Wikipedia

Math.LN10

The **`Math.LN10`** static data property represents the natural logarithm of 10, approximately 2.303.

Try it

```
function getNatLog10() {
  return Math.LN10;
}

console.log(getNatLog10());
// Expected output: 2.302585092994046
```

Value

$$\texttt{Math.LN10} = \ln(10) \approx 2.303$$

Property attributes of `Math.LN10`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `LN10` is a static property of `Math`, you always use it as `Math.LN10`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.LN10

The following function returns the natural log of 10:

```
function getNatLog10() {
  return Math.LN10;
}

getNatLog10(); // 2.302585092994046
```

See also

- `Math.exp()`
- `Math.log()`
- `Math.log10()`

Math.LN2

The **`Math.LN2`** static data property represents the natural logarithm of 2, approximately 0.693:

Try it

```
function getNatLog2() {
  return Math.LN2;
}

console.log(getNatLog2());
// Expected output: 0.6931471805599453
```

Value

$$\texttt{Math.LN2} = \ln(2) \approx 0.693$$

Property attributes of `Math.LN2`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `LN2` is a static property of `Math`, you always use it as `Math.LN2`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.LN2

The following function returns the natural log of 2:

```
function getNatLog2() {
  return Math.LN2;
}

getNatLog2(); // 0.6931471805599453
```

See also

- `Math.exp()`
- `Math.log()`
- `Math.log2()`

Math.log()

The **`Math.log()`** static method returns the natural logarithm (base e) of a number. That is

$$\forall x > 0,\ \mathrm{Math.log}\,(\mathrm{x}) = \ln(x) = \text{the unique } y \text{ such that } e^y = x$$

Try it

```
function getBaseLog(x, y) {
  return Math.log(y) / Math.log(x);
}

// 2 x 2 x 2 = 8
console.log(getBaseLog(2, 8));
// Expected output: 3

// 5 x 5 x 5 x 5 = 625
console.log(getBaseLog(5, 625));
// Expected output: 4
```

Syntax

```
Math.log(x)
```

Parameters

`x`

A number greater than or equal to 0.

Return value

The natural logarithm (base e) of `x`. If `x` is ±0, returns `-Infinity`. If `x < 0`, returns `NaN`.

Description

Because `log()` is a static method of `Math`, you always use it as `Math.log()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

If you need the natural log of 2 or 10, use the constants `Math.LN2` or `Math.LN10`. If you need a logarithm to base 2 or 10, use `Math.log2()` or `Math.log10()`. If you need a logarithm to other bases, use `Math.log(x) / Math.log(otherBase)` as in the example below; you might want to precalculate `1 / Math.log(otherBase)` since multiplication in `Math.log(x) * constant` is much faster.

Beware that positive numbers very close to 1 can suffer from loss of precision and make its natural logarithm less accurate. In this case, you may want to use `Math.log1p` instead.

Examples

Using Math.log()

```
Math.log(-1); // NaN
Math.log(-0); // -Infinity
Math.log(0); // -Infinity
Math.log(1); // 0
```

```
Math.log(10); // 2.302585092994046
Math.log(Infinity); // Infinity
```

Using Math.log() with a different base

The following function returns the logarithm of `y` with base `x` (i.e., $\log_x y$):

```
function getBaseLog(x, y) {
  return Math.log(y) / Math.log(x);
}
```

If you run `getBaseLog(10, 1000)`, it returns `2.9999999999999996` due to floating-point rounding, but still very close to the actual answer of 3.

See also

- `Math.exp()`
- `Math.log1p()`
- `Math.log10()`
- `Math.log2()`
- `Math.pow()`

Math.log10()

The **`Math.log10()`** static method returns the base 10 logarithm of a number. That is

$$\forall x > 0,\ \texttt{Math.log10}\,(x) = \log_{10}(x) = \text{the unique } y \text{ such that } 10^y = x$$

Try it

```
console.log(Math.log10(100000));
// Expected output: 5

console.log(Math.log10(2));
// Expected output: 0.3010299956639812

console.log(Math.log10(1));
// Expected output: 0

console.log(Math.log10(0));
// Expected output: -Infinity
```

Syntax

```
Math.log10(x)
```

Parameters

`x`

A number greater than or equal to 0.

Return value

The base 10 logarithm of `x`. If `x < 0`, returns `NaN`.

Description

Because `log10()` is a static method of `Math`, you always use it as `Math.log10()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

This function is the equivalent of `Math.log(x) / Math.log(10)`. For `log10(e)`, use the constant `Math.LOG10E`, which is 1 / `Math.LN10`.

Examples

Using Math.log10()

```
Math.log10(-2); // NaN
Math.log10(-0); // -Infinity
Math.log10(0); // -Infinity
Math.log10(1); // 0
Math.log10(2); // 0.3010299956639812
Math.log10(100000); // 5
Math.log10(Infinity); // Infinity
```

See also

- Polyfill of `Math.log10` in `core-js`
- es-shims polyfill of `Math.log10`
- `Math.exp()`
- `Math.log()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

Math.LOG10E

The **`Math.LOG10E`** static data property represents the base 10 logarithm of e, approximately 0.434.

Try it

```
function getLog10e() {
  return Math.LOG10E;
}

console.log(getLog10e());
// Expected output: 0.4342944819032518
```

Value

$$\mathtt{Math.LOG10E} = \log_{10}(e) \approx 0.434$$

Property attributes of `Math.LOG10E`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `LOG10E` is a static property of `Math`, you always use it as `Math.LOG10E`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.LOG10E

The following function returns the base 10 logarithm of e:

```
function getLog10e() {
  return Math.LOG10E;
}

getLog10e(); // 0.4342944819032518
```

See also

- `Math.exp()`
- `Math.log()`
- `Math.log10()`

Math.log1p()

The **`Math.log1p()`** static method returns the natural logarithm (base e) of `1 + x`, where `x` is the argument. That is:

$$\forall x > -1,\ \mathtt{Math.log1p}(x) = \ln(1 + x)$$

Try it

```
console.log(Math.log1p(1));
// Expected output: 0.6931471805599453

console.log(Math.log1p(0));
// Expected output: 0

console.log(Math.log1p(-1));
// Expected output: -Infinity

console.log(Math.log1p(-2));
// Expected output: NaN
```

Syntax

```
Math.log1p(x)
```

Parameters

`x`

A number greater than or equal to -1.

Return value

The natural logarithm (base e) of `x + 1`. If `x` is -1, returns `-Infinity`. If `x < -1`, returns `NaN`.

Description

For very small values of *x*, adding 1 can reduce or eliminate precision. The double floats used in JS give you about 15 digits of precision. 1 + 1e-15 = 1.000000000000001, but 1 + 1e-16 = 1.000000000000000 and therefore exactly 1.0 in that arithmetic, because digits past 15 are rounded off.

When you calculate log(1 + *x*), where *x* is a small positive number, you should get an answer very close to *x* because: $\lim_{x \to 0} \frac{\log(1+x)}{x} = 1$. If you calculate `Math.log(1 + 1.1111111111e-15)`, you should get an answer close to `1.1111111111e-15`. Instead, you will end up taking the logarithm of `1.0000000000000111022` (the roundoff is in binary, so sometimes it gets ugly), and get the answer 1.11022...e-15, with only 3 correct digits. If you calculate `Math.log1p(1.1111111111e-15)` instead, you will get a much more accurate answer, `1.1111111110999995e-15`, with 15 correct digits of precision (actually 16 in this case).

If the value of `x` is less than -1, the return value is always `NaN`.

Because `log1p()` is a static method of `Math`, you always use it as `Math.log1p()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.log1p()

```
Math.log1p(-2); // NaN
Math.log1p(-1); // -Infinity
Math.log1p(-0); // -0
Math.log1p(0); // 0
Math.log1p(1); // 0.6931471805599453
Math.log1p(Infinity); // Infinity
```

See also

- Polyfill of `Math.log1p` in `core-js`
- es-shims polyfill of `Math.log1p`
- `Math.exp()`
- `Math.log()`
- `Math.expm1()`
- `Math.log10()`
- `Math.log2()`
- `Math.pow()`

Math.log2()

The **`Math.log2()`** static method returns the base 2 logarithm of a number. That is

$$\forall x > 0,\ \texttt{Math.log2}\ (x) = \log_2(x) = \text{the unique } y \text{ such that } 2^y = x$$

Try it

```
console.log(Math.log2(3));
// Expected output: 1.584962500721156

console.log(Math.log2(2));
// Expected output: 1

console.log(Math.log2(1));
// Expected output: 0

console.log(Math.log2(0));
// Expected output: -Infinity
```

Syntax

```
Math.log2(x)
```

Parameters

`x`

A number greater than or equal to 0.

Return value

The base 2 logarithm of `x`. If `x < 0`, returns `NaN`.

Description

Because `log2()` is a static method of `Math`, you always use it as `Math.log2()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

This function is the equivalent of `Math.log(x) / Math.log(2)`. For `log2(e)`, use the constant `Math.LOG2E`, which is 1 / `Math.LN2`.

Examples

Using Math.log2()

```
Math.log2(-2); // NaN
Math.log2(-0); // -Infinity
Math.log2(0); // -Infinity
Math.log2(1); // 0
Math.log2(2); // 1
Math.log2(3); // 1.584962500721156
```

```
Math.log2(1024); // 10
Math.log2(Infinity); // Infinity
```

See also

- Polyfill of `Math.log2` in `core-js`
- `Math.exp()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`
- `Math.pow()`

Math.LOG2E

The **`Math.LOG2E`** static data property represents the base 2 logarithm of e, approximately 1.443.

Try it

```
function getLog2e() {
  return Math.LOG2E;
}

console.log(getLog2e());
// Expected output: 1.4426950408889634
```

Value

$$\mathrm{Math.LOG2E} = \log_2(e) \approx 1.443$$

Property attributes of `Math.LOG2E`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `LOG2E` is a static property of `Math`, you always use it as `Math.LOG2E`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.LOG2E

The following function returns the base 2 logarithm of e:

```
function getLog2e() {
  return Math.LOG2E;
}

getLog2e(); // 1.4426950408889634
```

See also

- `Math.exp()`
- `Math.log()`
- `Math.log2()`

Math.max()

The **`Math.max()`** static method returns the largest of the numbers given as input parameters, or `-Infinity` if there are no parameters.

Try it

```
console.log(Math.max(1, 3, 2));
// Expected output: 3

console.log(Math.max(-1, -3, -2));
// Expected output: -1

const array = [1, 3, 2];

console.log(Math.max(...array));
// Expected output: 3
```

Syntax

```
Math.max()
Math.max(value1)
Math.max(value1, value2)
Math.max(value1, value2, /* …, */ valueN)
```

Parameters

`value1, ..., valueN`

Zero or more numbers among which the largest value will be selected and returned.

Return value

The largest of the given numbers. Returns `NaN` if any of the parameters is or is converted into `NaN`. Returns `-Infinity` if no parameters are provided.

Description

Because `max()` is a static method of `Math`, you always use it as `Math.max()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

`Math.max.length` is 2, which weakly signals that it's designed to handle at least two parameters.

Examples

Using Math.max()

```
Math.max(10, 20); // 20
Math.max(-10, -20); // -10
Math.max(-10, 20); // 20
```

Getting the maximum element of an array

`Array.prototype.reduce()` can be used to find the maximum element in a numeric array, by comparing each value:

```
const arr = [1, 2, 3];
const max = arr.reduce((a, b) => Math.max(a, b), -Infinity);
```

The following function uses `Function.prototype.apply()` to get the maximum of an array. `getMaxOfArray([1, 2, 3])` is equivalent to `Math.max(1, 2, 3)`, but you can use `getMaxOfArray()` on programmatically constructed arrays. This should only be used for arrays with relatively few elements.

```
function getMaxOfArray(numArray) {
  return Math.max.apply(null, numArray);
}
```

The spread syntax is a shorter way of writing the `apply` solution to get the maximum of an array:

```
const arr = [1, 2, 3];
const max = Math.max(...arr);
```

However, both spread (`...`) and `apply` will either fail or return the wrong result if the array has too many elements, because they try to pass the array elements as function parameters. See Using apply and built-in functions for more details. The `reduce` solution does not have this problem.

See also

- `Math.min()`

Math.min()

The **`Math.min()`** static method returns the smallest of the numbers given as input parameters, or `Infinity` if there are no parameters.

Try it

```
console.log(Math.min(2, 3, 1));
// Expected output: 1

console.log(Math.min(-2, -3, -1));
// Expected output: -3

const array = [2, 3, 1];

console.log(Math.min(...array));
// Expected output: 1
```

Syntax

```
Math.min()
Math.min(value1)
Math.min(value1, value2)
Math.min(value1, value2, /* …, */ valueN)
```

Parameters

`value1, ..., valueN`

Zero or more numbers among which the lowest value will be selected and returned.

Return value

The smallest of the given numbers. Returns `NaN` if any of the parameters is or is converted into `NaN`. Returns `Infinity` if no parameters are provided.

Description

Because `min()` is a static method of `Math`, you always use it as `Math.min()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

`Math.min.length` is 2, which weakly signals that it's designed to handle at least two parameters.

Examples

Using Math.min()

This finds the min of `x` and `y` and assigns it to `z`:

```
const x = 10;
const y = -20;
const z = Math.min(x, y); // -20
```

Clipping a value with Math.min()

`Math.min()` is often used to clip a value so that it is always less than or equal to a boundary. For instance, this

```
let x = f(foo);

if (x > boundary) {
  x = boundary;
}
```

may be written as this

```
const x = Math.min(f(foo), boundary);
```

`Math.max()` can be used in a similar way to clip a value at the other end.

See also

- `Math.max()`

Math.PI

The **`Math.PI`** static data property represents the ratio of the circumference of a circle to its diameter, approximately 3.14159.

Try it

```
function calculateCircumference(radius) {
  return 2 * Math.PI * radius;
}

console.log(Math.PI);
// Expected output: 3.141592653589793

console.log(calculateCircumference(10));
// Expected output: 62.83185307179586
```

Value

$$\texttt{Math.PI} = \pi \approx 3.14159$$

Property attributes of `Math.PI`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `PI` is a static property of `Math`, you always use it as `Math.PI`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.PI

The following function uses `Math.PI` to calculate the circumference of a circle with a passed radius.

```
function calculateCircumference(radius) {
  return Math.PI * (radius + radius);
}

calculateCircumference(1); // 6.283185307179586
```

See also

- Math

Math.pow()

The **`Math.pow()`** static method returns the value of a base raised to a power. That is

$$\texttt{Math.pow}(x, y) = x^y$$

Try it

```
console.log(Math.pow(7, 3));
// Expected output: 343

console.log(Math.pow(4, 0.5));
// Expected output: 2

console.log(Math.pow(7, -2));
// Expected output: 0.02040816326530612
//                  (1/49)

console.log(Math.pow(-7, 0.5));
// Expected output: NaN
```

Syntax

```
Math.pow(base, exponent)
```

Parameters

`base`

The base number.

`exponent`

The exponent number.

Return value

A number representing `base` taken to the power of `exponent`. Returns `NaN` in one of the following cases:

- `exponent` is `NaN`.
- `base` is `NaN` and `exponent` is not `0`.
- `base` is ±1 and `exponent` is `±Infinity`.
- `base < 0` and `exponent` is not an integer.

Description

`Math.pow()` is equivalent to the `**` operator, except `Math.pow()` only accepts numbers.

`Math.pow(NaN, 0)` (and the equivalent `NaN ** 0`) is the only case where `NaN` doesn't propagate through mathematical operations — it returns `1` despite the operand being `NaN`. In addition, the behavior where `base` is 1 and `exponent` is non-finite (±Infinity or `NaN`) is different from IEEE 754, which specifies that the result should be 1, whereas JavaScript returns `NaN` to preserve backward compatibility with its original behavior.

Because `pow()` is a static method of `Math`, use it as `Math.pow()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.pow()

```
// Basic cases
Math.pow(7, 2); // 49
Math.pow(7, 3); // 343
Math.pow(2, 10); // 1024

// Fractional exponents
Math.pow(4, 0.5); // 2 (square root of 4)
Math.pow(8, 1 / 3); // 2 (cube root of 8)
Math.pow(2, 0.5); // 1.4142135623730951 (square root of 2)
Math.pow(2, 1 / 3); // 1.2599210498948732 (cube root of 2)

// Signed exponents
Math.pow(7, -2); // 0.02040816326530612 (1/49)
Math.pow(8, -1 / 3); // 0.5

// Signed bases
Math.pow(-7, 2); // 49 (squares are positive)
Math.pow(-7, 3); // -343 (cubes can be negative)
Math.pow(-7, 0.5); // NaN (negative numbers don't have a real square root)
// Due to "even" and "odd" roots laying close to each other,
// and limits in the floating number precision,
// negative bases with fractional exponents always return NaN,
// even when the mathematical result is real
Math.pow(-7, 1 / 3); // NaN
```

```
// Zero and infinity
Math.pow(0, 0); // 1 (anything ** ±0 is 1)
Math.pow(Infinity, 0.1); // Infinity (positive exponent)
Math.pow(Infinity, -1); // 0 (negative exponent)
Math.pow(-Infinity, 1); // -Infinity (positive odd integer exponent)
Math.pow(-Infinity, 1.5); // Infinity (positive exponent)
Math.pow(-Infinity, -1); // -0 (negative odd integer exponent)
Math.pow(-Infinity, -1.5); // 0 (negative exponent)
Math.pow(0, 1); // 0 (positive exponent)
Math.pow(0, -1); // Infinity (negative exponent)
Math.pow(-0, 1); // -0 (positive odd integer exponent)
Math.pow(-0, 1.5); // 0 (positive exponent)
Math.pow(-0, -1); // -Infinity (negative odd integer exponent)
Math.pow(-0, -1.5); // Infinity (negative exponent)
Math.pow(0.9, Infinity); // 0
Math.pow(1, Infinity); // NaN
Math.pow(1.1, Infinity); // Infinity
Math.pow(0.9, -Infinity); // Infinity
Math.pow(1, -Infinity); // NaN
Math.pow(1.1, -Infinity); // 0

// NaN: only Math.pow(NaN, 0) does not result in NaN
Math.pow(NaN, 0); // 1
Math.pow(NaN, 1); // NaN
Math.pow(1, NaN); // NaN
```

See also

- `Math.cbrt()`
- `Math.exp()`
- `Math.log()`
- `Math.sqrt()`
- Exponentiation (`**`)

Math.random()

The **`Math.random()`** static method returns a floating-point, pseudo-random number that's greater than or equal to 0 and less than 1, with approximately uniform distribution over that range — which you can then scale to your desired range. The implementation selects the initial seed to the random number generation algorithm; it cannot be chosen or reset by the user.

> **Note:** `Math.random()` *does not* provide cryptographically secure random numbers. Do not use them for anything related to security. Use the Web Crypto API instead, and more precisely the `Crypto.getRandomValues()` method.

Try it

```
function getRandomInt(max) {
  return Math.floor(Math.random() * max);
}

console.log(getRandomInt(3));
// Expected output: 0, 1 or 2

console.log(getRandomInt(1));
// Expected output: 0

console.log(Math.random());
// Expected output: a number from 0 to <1
```

Syntax

```
Math.random()
```

Parameters

None.

Return value

A floating-point, pseudo-random number between 0 (inclusive) and 1 (exclusive).

Examples

Note that as numbers in JavaScript are IEEE 754 floating point numbers with round-to-nearest-even behavior, the ranges claimed for the functions below (excluding the one for `Math.random()` itself) aren't exact. Usually, the claimed upper bound is not attainable, but if `Math.random()` returns a number very close to 1, the tiny difference may not be representable at the requested maximum, therefore causing the upper bound to be attained.

Getting a random number between 0 (inclusive) and 1 (exclusive)

```
function getRandom() {
  return Math.random();
}
```

Getting a random number between two values

This example returns a random number between the specified values. The returned value is no lower than (and may possibly equal) `min`, and is less than (and not equal) `max`.

```
function getRandomArbitrary(min, max) {
  return Math.random() * (max - min) + min;
}
```

Getting a random integer between two values

This example returns a random *integer* between the specified values. The value is no lower than `min` (or the next integer greater than `min` if `min` isn't an integer), and is less than (but not equal to) `max`.

```
function getRandomInt(min, max) {
  const minCeiled = Math.ceil(min);
  const maxFloored = Math.floor(max);
  return Math.floor(Math.random() * (maxFloored - minCeiled) + minCeiled); // The maximum is exclusive and the minimum is inclusive
}
```

> **Note:** It might be tempting to use `Math.round()` to accomplish that, but doing so would cause your random numbers to follow a non-uniform distribution, which may not be acceptable for your needs.

Getting a random integer between two values, inclusive

While the `getRandomInt()` function above is inclusive at the minimum, it's exclusive at the maximum. What if you need the results to be inclusive at both the minimum and the maximum? The `getRandomIntInclusive()` function below accomplishes that.

```
function getRandomIntInclusive(min, max) {
  const minCeiled = Math.ceil(min);
  const maxFloored = Math.floor(max);
  return Math.floor(Math.random() * (maxFloored - minCeiled + 1) + minCeiled); // The maximum is inclusive and the minimum is inclusive
}
```

See also

- `Crypto.getRandomValues()`

Math.round()

The **`Math.round()`** static method returns the value of a number rounded to the nearest integer.

Try it

```
console.log(Math.round(0.9));
// Expected output: 1

console.log(Math.round(5.95), Math.round(5.5), Math.round(5.05));
// Expected output: 6 6 5

console.log(Math.round(-5.05), Math.round(-5.5), Math.round(-5.95));
// Expected output: -5 -5 -6
```

Syntax

```
Math.round(x)
```

Parameters

`x`

A number.

Return value

The value of `x` rounded to the nearest integer.

Description

If the fractional portion of the argument is greater than 0.5, the argument is rounded to the integer with the next higher absolute value. If it is less than 0.5, the argument is rounded to the integer with the lower absolute value. If the fractional portion is exactly 0.5, the argument is rounded to the next integer in the direction of $+\infty$.

> **Note:** This differs from many languages' `round()` functions, which often round half-increments *away from zero*, giving a different result in the case of negative numbers with a fractional part of exactly 0.5.

`Math.round(x)` is not exactly the same as `Math.floor(x + 0.5)`. When `x` is -0, or $-0.5 \le x < 0$, `Math.round(x)` returns -0, while `Math.floor(x + 0.5)` returns 0. However, neglecting that difference and potential precision errors, `Math.round(x)` and `Math.floor(x + 0.5)` are generally equivalent.

Because `round()` is a static method of `Math`, you always use it as `Math.round()`, rather than as a method of a `Math` object you created (`Math` has no constructor).

Examples

Using round

```
Math.round(-Infinity); // -Infinity
Math.round(-20.51); // -21
Math.round(-20.5); // -20
Math.round(-0.1); // -0
Math.round(0); // 0
Math.round(20.49); // 20
Math.round(20.5); // 21
Math.round(42); // 42
Math.round(Infinity); // Infinity
```

See also

- `Number.prototype.toPrecision()`
- `Number.prototype.toFixed()`
- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.sign()`
- `Math.trunc()`

Math.sign()

The **`Math.sign()`** static method returns 1 or -1, indicating the sign of the number passed as argument. If the input is 0 or -0, it will be returned as-is.

Try it

```
console.log(Math.sign(3));
// Expected output: 1

console.log(Math.sign(-3));
// Expected output: -1

console.log(Math.sign(0));
// Expected output: 0

console.log(Math.sign("-3"));
// Expected output: -1
```

Syntax

```
Math.sign(x)
```

Parameters

`x`

A number.

Return value

A number representing the sign of `x`:

- If `x` is positive, returns `1`.
- If `x` is negative, returns `-1`.
- If `x` is positive zero, returns `0`.
- If `x` is negative zero, returns `-0`.
- Otherwise, returns `NaN`.

Description

Because `sign()` is a static method of `Math`, you always use it as `Math.sign()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.sign()

```
Math.sign(3); // 1
Math.sign(-3); // -1
Math.sign("-3"); // -1
Math.sign(0); // 0
Math.sign(-0); // -0
Math.sign(NaN); // NaN
Math.sign("foo"); // NaN
Math.sign(); // NaN
```

See also

- Polyfill of `Math.sign` in `core-js`
- es-shims polyfill of `Math.sign`
- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.trunc()`

Math.sin()

The **`Math.sin()`** static method returns the sine of a number in radians.

Try it

```
function getCircleY(radians, radius) {
  return Math.sin(radians) * radius;
}

console.log(getCircleY(1, 10));
// Expected output: 8.414709848078965

console.log(getCircleY(2, 10));
// Expected output: 9.092974268256818

console.log(getCircleY(Math.PI, 10));
// Expected output: 1.2246467991473533e-15
```

Syntax

```
Math.sin(x)
```

Parameters

`x`

A number representing an angle in radians.

Return value

The sine of `x`, between -1 and 1, inclusive. If `x` is `Infinity`, `-Infinity`, or `NaN`, returns `NaN`.

Description

Because `sin()` is a static method of `Math`, you always use it as `Math.sin()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.sin()

```
Math.sin(-Infinity); // NaN
Math.sin(-0); // -0
Math.sin(0); // 0
Math.sin(1); // 0.8414709848078965
Math.sin(Math.PI / 2); // 1
Math.sin(Infinity); // NaN
```

See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.tan()`
- CSS `sin()` function

Math.sinh()

The **`Math.sinh()`** static method returns the hyperbolic sine of a number. That is,

$$\mathrm{Math.sinh}\,(x) = \sinh(x) = \frac{e^x - e^{-x}}{2}$$

Try it

```
console.log(Math.sinh(0));
// Expected output: 0

console.log(Math.sinh(1));
// Expected output: 1.1752011936438014

console.log(Math.sinh(-1));
// Expected output: -1.1752011936438014
```

```
console.log(Math.sinh(2));
// Expected output: 3.626860407847019
```

Syntax

```
Math.sinh(x)
```

Parameters

`x`

A number.

Return value

The hyperbolic sine of `x`.

Description

Because `sinh()` is a static method of `Math`, you always use it as `Math.sinh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.sinh()

```
Math.sinh(-Infinity); // -Infinity
Math.sinh(-0); // -0
Math.sinh(0); // 0
Math.sinh(1); // 1.1752011936438014
Math.sinh(Infinity); // Infinity
```

See also

- Polyfill of `Math.sinh` in `core-js`
- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.tanh()`

Math.sqrt()

The **`Math.sqrt()`** static method returns the square root of a number. That is

$$\forall x \geq 0,\ \mathrm{Math.sqrt}\,(x) = \sqrt{x} = \text{the unique } y \geq 0 \text{ such that } y^2 = x$$

Try it

```
function calcHypotenuse(a, b) {
  return Math.sqrt(a * a + b * b);
}

console.log(calcHypotenuse(3, 4));
// Expected output: 5

console.log(calcHypotenuse(5, 12));
// Expected output: 13

console.log(calcHypotenuse(0, 0));
// Expected output: 0
```

Syntax

```
Math.sqrt(x)
```

Parameters

`x`

A number greater than or equal to 0.

Return value

The square root of `x`, a nonnegative number. If `x < 0`, returns `NaN`.

Description

Because `sqrt()` is a static method of `Math`, you always use it as `Math.sqrt()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.sqrt()

```
Math.sqrt(-1); // NaN
Math.sqrt(-0); // -0
Math.sqrt(0); // 0
Math.sqrt(1); // 1
Math.sqrt(2); // 1.414213562373095
Math.sqrt(9); // 3
Math.sqrt(Infinity); // Infinity
```

See also

- `Math.cbrt()`
- `Math.exp()`
- `Math.log()`
- `Math.pow()`

Math.SQRT1_2

The **`Math.SQRT1_2`** static data property represents the square root of 1/2, which is approximately 0.707.

Try it

```
function getRoot1Over2() {
  return Math.SQRT1_2;
}

console.log(getRoot1Over2());
// Expected output: 0.7071067811865476
```

Value

$$\texttt{Math.SQRT1_2} = \sqrt{\frac{1}{2}} \approx 0.707$$

Property attributes of `Math.SQRT1_2`	
Writable	no
Enumerable	no
Configurable	no

Description

`Math.SQRT1_2` is a constant and a more performant equivalent to `Math.sqrt(0.5)`.

Because `SQRT1_2` is a static property of `Math`, you always use it as `Math.SQRT1_2`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.SQRT1_2

The following function calculates the side length of a square given its diagonal length:

```
function getSquareSideLength(diagonalLength) {
  return diagonalLength * Math.SQRT1_2;
}
```

See also

- `Math.pow()`
- `Math.sqrt()`

Math.SQRT2

The **`Math.SQRT2`** static data property represents the square root of 2, approximately 1.414.

Try it

```
function getRoot2() {
  return Math.SQRT2;
}

console.log(getRoot2());
// Expected output: 1.4142135623730951
```

Value

$$\texttt{Math.SQRT2} = \sqrt{2} \approx 1.414$$

Property attributes of `Math.SQRT2`	
Writable	no
Enumerable	no

Property attributes of `Math.SQRT2`	
Configurable	no

Description

`Math.SQRT2` is a constant and a more performant equivalent to `Math.sqrt(2)`.

Because `SQRT2` is a static property of `Math`, you always use it as `Math.SQRT2`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.SQRT2

The following function returns the square root of 2:

```
function getRoot2() {
  return Math.SQRT2;
}

getRoot2(); // 1.4142135623730951
```

See also

- `Math.pow()`
- `Math.sqrt()`

Math.sumPrecise()

The **`Math.sumPrecise()`** static method takes an iterable of numbers and returns the sum of them. It is more precise than summing them up in a loop, because it avoids floating point precision loss in intermediate results.

Try it

```
console.log(Math.sumPrecise([1, 2]));
// Expected output: 3

console.log(Math.sumPrecise([1e20, 0.1, -1e20]));
// Expected output: 0.1
```

Syntax

```
Math.sumPrecise(numbers)
```

Parameters

`numbers`

An iterable (such as an `Array`) of numbers.

Return value

A number that is the sum of the numbers in the `numbers` iterable. If the iterable is empty, the return value is `-0` (*not* `0`).

Exceptions

`TypeError`

If `numbers` is not an iterable, or if any of the numbers in the iterable is not of the number type.

Description

Because `sumPrecise()` is a static method of `Math`, you always use it as `Math.sumPrecise()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

The method is called `Math.sumPrecise()` because it is more precise than naïvely summing up numbers in a loop. Consider the following example:

```
let sum = 0;
const numbers = [1e20, 0.1, -1e20];
for (const number of numbers) {
  sum += number;
}
console.log(sum); // 0
```

The output is 0. This is because `1e20 + 0.1` cannot be represented precisely in 64-bit floats, so the intermediate result is rounded to `1e20`. Then, the sum of `1e20` and `-1e20` is `0`, so the final result is `0`.

`Math.sumPrecise()` avoids this issue by using some specialized summing algorithm. It works as if the floating point numbers are summed up using their precise mathematical values, and the final result is then converted to the nearest representable 64-bit float. This still cannot avoid the `0.1 + 0.2` precision problem:

```
console.log(Math.sumPrecise([0.1, 0.2])); // 0.30000000000000004
```

Because the floating point literals `0.1` and `0.2` already represent mathematical values greater than `0.1` and `0.2`, and their sum's closest 64-bit float representation is actually `0.30000000000000004`.

Examples

Using Math.sumPrecise()

```
console.log(Math.sumPrecise([1, 2, 3])); // 6
console.log(Math.sumPrecise([1e20, 0.1, -1e20])); // 0.1
```

See also

- Polyfill of `Math.sumPrecise` in `core-js`
- es-shims polyfill of `Math.sumPrecise`
- `Array.prototype.reduce()`

Math.tan()

The **`Math.tan()`** static method returns the tangent of a number in radians.

Try it

```
function getTanFromDegrees(degrees) {
  return Math.tan((degrees * Math.PI) / 180);
}

console.log(getTanFromDegrees(0));
// Expected output: 0

console.log(getTanFromDegrees(45));
// Expected output: 0.9999999999999999

console.log(getTanFromDegrees(90));
// Expected output: 16331239353195370
```

Syntax

```
Math.tan(x)
```

Parameters

`x`

A number representing an angle in radians.

Return value

The tangent of `x`. If `x` is `Infinity`, `-Infinity`, or `NaN`, returns `NaN`.

> **Note:** Due to floating point precision, it's not possible to obtain the exact value π/2, so the result is always finite if not `NaN`.

Description

Because `tan()` is a static method of `Math`, you always use it as `Math.tan()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.tan()

```
Math.tan(-Infinity); // NaN
Math.tan(-0); // -0
Math.tan(0); // 0
Math.tan(1); // 1.5574077246549023
Math.tan(Math.PI / 4); // 0.9999999999999999 (Floating point error)
Math.tan(Infinity); // NaN
```

Math.tan() and π/2

It's not possible to calculate `tan(π/2)` exactly.

```
Math.tan(Math.PI / 2); // 16331239353195370
Math.tan(Math.PI / 2 + Number.EPSILON); // -6218431163823738
```

Using Math.tan() with a degree value

Because the `Math.tan()` function accepts radians, but it is often easier to work with degrees, the following function accepts a value in degrees, converts it to radians and returns the tangent.

```
function getTanDeg(deg) {
  const rad = (deg * Math.PI) / 180;
```

```
  return Math.tan(rad);
}
```

See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- CSS `tan()` function

Math.tanh()

The **`Math.tanh()`** static method returns the hyperbolic tangent of a number. That is,

$$\operatorname{Math.tanh}(x) = \tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$

Try it

```
console.log(Math.tanh(-1));
// Expected output: -0.7615941559557649

console.log(Math.tanh(0));
// Expected output: 0

console.log(Math.tanh(Infinity));
// Expected output: 1

console.log(Math.tanh(1));
// Expected output: 0.7615941559557649
```

Syntax

```
Math.tanh(x)
```

Parameters

`x`

A number.

Return value

The hyperbolic tangent of `x`.

Description

Because `tanh()` is a static method of `Math`, you always use it as `Math.tanh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.tanh()

```
Math.tanh(-Infinity); // -1
Math.tanh(-0); // -0
Math.tanh(0); // 0
Math.tanh(1); // 0.7615941559557649
Math.tanh(Infinity); // 1
```

See also

- Polyfill of `Math.tanh` in `core-js`
- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`

Math.trunc()

The **`Math.trunc()`** static method returns the integer part of a number by removing any fractional digits.

Try it

```
console.log(Math.trunc(13.37));
// Expected output: 13

console.log(Math.trunc(42.84));
// Expected output: 42
```

```
console.log(Math.trunc(0.123));
// Expected output: 0

console.log(Math.trunc(-0.123));
// Expected output: -0
```

Syntax

```
Math.trunc(x)
```

Parameters

`x`

A number.

Return value

The integer part of `x`.

Description

The way `Math.trunc()` works is more straightforward than the other three `Math` methods: `Math.floor()`, `Math.ceil()` and `Math.round()`; it *truncates* (cuts off) the dot and the digits to the right of it, no matter whether the argument is a positive or negative number.

Because `trunc()` is a static method of `Math`, you always use it as `Math.trunc()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using Math.trunc()

```
Math.trunc(-Infinity); // -Infinity
Math.trunc("-1.123"); // -1
Math.trunc(-0.123); // -0
Math.trunc(-0); // -0
Math.trunc(0); // 0
Math.trunc(0.123); // 0
Math.trunc(13.37); // 13
Math.trunc(42.84); // 42
Math.trunc(Infinity); // Infinity
```

Using bitwise no-ops to truncate numbers

> **Warning:** This is not a polyfill for `Math.trunc()` because of non-negligible edge cases.

Bitwise operations convert their operands to 32-bit integers, which people have historically taken advantage of to truncate float-point numbers. Common techniques include:

```
const original = 3.14;
const truncated1 = ~~original; // Double negation
const truncated2 = original & -1; // Bitwise AND with -1
const truncated3 = original | 0; // Bitwise OR with 0
const truncated4 = original ^ 0; // Bitwise XOR with 0
const truncated5 = original >> 0; // Bitwise shifting by 0
```

Beware that this is essentially `toInt32`, which is not the same as `Math.trunc`. When the value does not satisfy $-2^{31} - 1 <$ `value` $< 2^{31}$ ($-2147483649 <$ `value` < 2147483648), the conversion would overflow.

```
const a = ~~2147483648; // -2147483648
const b = ~~-2147483649; // 2147483647
const c = ~~4294967296; // 0
```

Only use `~~` as a substitution for `Math.trunc()` when you are confident that the range of input falls within the range of 32-bit integers.

See also

- Polyfill of `Math.trunc` in `core-js`
- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`

NaN

The **`NaN`** global property is a value representing Not-A-Number.

Try it

```
function sanitize(x) {
  if (isNaN(x)) {
    return NaN;
  }
  return x;
}

console.log(sanitize("1"));
// Expected output: "1"

console.log(sanitize("NotANumber"));
// Expected output: NaN
```

Value

The same number value as `Number.NaN`.

Property attributes of `NaN`	
Writable	no
Enumerable	no
Configurable	no

Description

`NaN` is a property of the *global object*. In other words, it is a variable in global scope.

In modern browsers, `NaN` is a non-configurable, non-writable property. Even when this is not the case, avoid overriding it.

There are five different types of operations that return `NaN`:

- Failed number conversion (e.g., explicit ones like `parseInt("blabla")`, `Number(undefined)`, or implicit ones like `Math.abs(undefined)`)
- Math operation where the result is not a real number (e.g., `Math.sqrt(-1)`)
- Indeterminate form (e.g., `0 * Infinity`, `1 ** Infinity`, `Infinity / Infinity`, `Infinity - Infinity`)
- A method or expression whose operand is or gets coerced to `NaN` (e.g., `7 ** NaN`, `7 * "blabla"`) — this means `NaN` is contagious
- Other cases where an invalid value is to be represented as a number (e.g., an invalid Date `new Date("blabla").getTime()`, `"".charCodeAt(1)`)

`NaN` and its behaviors are not invented by JavaScript. Its semantics in floating point arithmetic (including that `NaN !== NaN`) are specified by IEEE 754. `NaN`'s behaviors include:

- If `NaN` is involved in a mathematical operation (but not bitwise operations), the result is usually also `NaN`. (See counter-example below.)
- When `NaN` is one of the operands of any relational comparison (`>`, `<`, `>=`, `<=`), the result is always `false`.
- `NaN` compares unequal (via `==`, `!=`, `===`, and `!==`) to any other value — including to another `NaN` value.

`NaN` is also one of the falsy values in JavaScript.

Examples

Testing against NaN

To tell if a value is `NaN`, use `Number.isNaN()` or `isNaN()` to most clearly determine whether a value is `NaN` — or, since `NaN` is the only value that compares unequal to itself, you can perform a self-comparison like `x !== x`.

```
NaN === NaN; // false
Number.NaN === NaN; // false
isNaN(NaN); // true
isNaN(Number.NaN); // true
Number.isNaN(NaN); // true

function valueIsNaN(v) {
  return v !== v;
}
valueIsNaN(1); // false
valueIsNaN(NaN); // true
valueIsNaN(Number.NaN); // true
```

However, do note the difference between `isNaN()` and `Number.isNaN()`: the former will return `true` if the value is currently `NaN`, or if it is going to be `NaN` after it is coerced to a number, while the latter will return `true` only if the value is currently `NaN`:

```
isNaN("hello world"); // true
Number.isNaN("hello world"); // false
```

For the same reason, using a BigInt value will throw an error with `isNaN()` and not with `Number.isNaN()`:

```
isNaN(1n); // TypeError: Conversion from 'BigInt' to 'number' is not allowed.
Number.isNaN(1n); // false
```

Additionally, some array methods cannot find `NaN`, while others can. Namely, the index-finding ones (`indexOf()`, `lastIndexOf()`) cannot find `NaN`, while the value-finding ones (`includes()`) can:

```
const arr = [2, 4, NaN, 12];
arr.indexOf(NaN); // -1
arr.includes(NaN); // true

// Methods accepting a properly defined predicate can always find NaN
arr.findIndex((n) => Number.isNaN(n)); // 2
```

For more information about `NaN` and its comparison, see Equality comparison and sameness.

Observably distinct NaN values

It's possible to produce two floating point numbers with different binary representations but are both `NaN`, because in IEEE 754 encoding, any floating point number with exponent `0x7ff` and a non-zero mantissa is `NaN`. In JavaScript, you can do bit-level manipulation using typed arrays.

```
const f2b = (x) => new Uint8Array(new Float64Array([x]).buffer);
const b2f = (x) => new Float64Array(x.buffer)[0];
// Get a byte representation of NaN
const n = f2b(NaN);
const m = f2b(NaN);
// Change the sign bit, which doesn't matter for NaN
n[7] += 2 ** 7;
// n[0] += 2**7; for big endian processors
const nan2 = b2f(n);
console.log(nan2); // NaN
console.log(Object.is(nan2, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan2)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 255]
// Change the first bit, which is the least significant bit of the mantissa and doesn't matter for NaN
m[0] = 1;
// m[7] = 1; for big endian processors
const nan3 = b2f(m);
console.log(nan3); // NaN
console.log(Object.is(nan3, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan3)); // Uint8Array(8) [1, 0, 0, 0, 0, 0, 248, 127]
```

Silently escaping NaN

`NaN` propagates through mathematical operations, so it's usually sufficient to test for `NaN` once at the end of calculation to detect error conditions. The only case where `NaN` gets silently escaped is when using exponentiation with an exponent of `0`, which immediately returns `1` without testing the base's value.

```
NaN ** 0 === 1; // true
```

See also

- `Number.NaN`
- `Number.isNaN()`
- `isNaN()`

Number

`Number` values represent floating-point numbers like `37` or `-9.25`.

The `Number` constructor contains constants and methods for working with numbers. Values of other types can be converted to numbers using the `Number()` function.

Description

Numbers are most commonly expressed in literal forms like `255` or `3.14159`. The lexical grammar contains a more detailed reference.

```
255; // two-hundred and fifty-five
255.0; // same number
255 === 255.0; // true
255 === 0xff; // true (hexadecimal notation)
255 === 0b11111111; // true (binary notation)
255 === 0.255e3; // true (decimal exponential notation)
```

A number literal like `37` in JavaScript code is a floating-point value, not an integer. There is no separate integer type in common everyday use. (JavaScript also has a `BigInt` type, but it's not designed to replace Number for everyday uses. `37` is still a number, not a BigInt.)

When used as a function, `Number(value)` converts a string or other value to the Number type. If the value can't be converted, it returns `NaN`.

```
Number("123"); // returns the number 123
Number("123") === 123; // true

Number("unicorn"); // NaN
Number(undefined); // NaN
```

Number encoding

The JavaScript `Number` type is a double-precision 64-bit binary format IEEE 754 value, like `double` in Java or C#. This means it can represent fractional values, but there are some limits to the stored number's magnitude and precision. Very briefly, an IEEE 754 double-precision number uses 64 bits to represent 3 parts:

- 1 bit for the *sign* (positive or negative)
- 11 bits for the *exponent* (-1022 to 1023)
- 52 bits for the *mantissa* (representing a number between 0 and 1)

The mantissa (also called *significand*) is the part of the number representing the actual value (significant digits). The exponent is the power of 2 that the mantissa should be multiplied by. Thinking about it as scientific notation:

$$\text{Number} = (-1)^{\text{sign}} \cdot (1 + \text{mantissa}) \cdot 2^{\text{exponent}}$$

The mantissa is stored with 52 bits, interpreted as digits after `1.`... in a binary fractional number. Therefore, the mantissa's precision is 2^{-52} (obtainable via `Number.EPSILON`), or about 15 to 17 significant decimal digits; arithmetic above that level of precision is subject to rounding.

The largest value a number can hold is $2^{1023} \times (2 - 2^{-52})$ (with the exponent being 1023 and the mantissa being 0.1111... in base 2), which is obtainable via `Number.MAX_VALUE`. Values higher than that are replaced with the special number constant `Infinity`.

Integers can only be represented without loss of precision in the range $-2^{53} + 1$ to $2^{53} - 1$, inclusive (obtainable via `Number.MIN_SAFE_INTEGER` and `Number.MAX_SAFE_INTEGER`), because the mantissa can only hold 53 bits (including the leading 1).

More details on this are described in the ECMAScript standard.

Number coercion

Many built-in operations that expect numbers first coerce their arguments to numbers (which is largely why `Number` objects behave similarly to number primitives). The operation can be summarized as follows:

- Numbers are returned as-is.
- `undefined` turns into `NaN`.
- `null` turns into `0`.
- `true` turns into `1`; `false` turns into `0`.
- Strings are converted by parsing them as if they contain a number literal. Parsing failure results in `NaN`. There are some minor differences compared to an actual number literal:
  - Leading and trailing whitespace/line terminators are ignored.
  - A leading `0` digit does not cause the number to become an octal literal (or get rejected in strict mode).
  - `+` and `-` are allowed at the start of the string to indicate its sign. (In actual code, they "look like" part of the literal, but are actually separate unary operators.) However, the sign can only appear once, and must not be followed by whitespace.
  - `Infinity` and `-Infinity` are recognized as literals. In actual code, they are global variables.
  - Empty or whitespace-only strings are converted to `0`.
  - Numeric separators are not allowed.
- BigInts throw a `TypeError` to prevent unintended implicit coercion causing loss of precision.
- Symbols throw a `TypeError`.
- Objects are first converted to a primitive by calling their `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. The resulting primitive is then converted to a number.

There are two ways to achieve nearly the same effect in JavaScript.

- Unary plus: `+x` does exactly the number coercion steps explained above to convert `x`.
- The `Number()` function: `Number(x)` uses the same algorithm to convert `x`, except that BigInts don't throw a `TypeError`, but return their number value, with possible loss of precision.

`Number.parseFloat()` and `Number.parseInt()` are similar to `Number()` but only convert strings, and have slightly different parsing rules. For example, `parseInt()` doesn't recognize the decimal point, and `parseFloat()` doesn't recognize the `0x` prefix.

Integer conversion

Some operations expect integers, most notably those that work with array/string indices, date/time components, and number radixes. After performing the number coercion steps above, the result is truncated to an integer (by discarding the fractional part). If the number is ±Infinity, it's returned as-is. If the number is `NaN` or `-0`, it's returned as `0`. The result is therefore always an integer (which is not `-0`) or ±Infinity.

Notably, when converted to integers, both `undefined` and `null` become `0`, because `undefined` is converted to `NaN`, which also becomes `0`.

Fixed-width number conversion

JavaScript has some lower-level functions that deal with the binary encoding of integer numbers, most notably bitwise operators and `TypedArray` objects. Bitwise operators always convert the operands to 32-bit integers. In these cases, after converting the value to a number, the number is then normalized to the given width by first truncating the fractional part and then taking the lowest bits in the integer's two's complement encoding.

```
new Int32Array([1.1, 1.9, -1.1, -1.9]); // Int32Array(4) [ 1, 1, -1, -1 ]

new Int8Array([257, -257]); // Int8Array(2) [ 1, -1 ]
// 257 = 0001 0000 0001
//     =      0000 0001 (mod 2^8)
//     = 1
// -257 = 1110 1111 1111
//      =      1111 1111 (mod 2^8)
//      = -1 (as signed integer)

new Uint8Array([257, -257]); // Uint8Array(2) [ 1, 255 ]
// -257 = 1110 1111 1111
//      =      1111 1111 (mod 2^8)
//      = 255 (as unsigned integer)
```

Constructor

`Number()`

Creates `Number` objects. When called as a function, it returns primitive values of type Number.

Static properties

`Number.EPSILON`

The smallest interval between two representable numbers.

`Number.MAX_SAFE_INTEGER`

The maximum safe integer in JavaScript (2^{53} - 1).

`Number.MAX_VALUE`

The largest positive representable number.

`Number.MIN_SAFE_INTEGER`

The minimum safe integer in JavaScript (-(2^{53} - 1)).

`Number.MIN_VALUE`

The smallest positive representable number—that is, the positive number closest to zero (without actually being zero).

`Number.NaN`

Special "**N**ot **a** **N**umber" value.

`Number.NEGATIVE_INFINITY`

Special value representing negative infinity. Returned on overflow.

`Number.POSITIVE_INFINITY`

Special value representing infinity. Returned on overflow.

Static methods

`Number.isFinite()`

Determine whether the passed value is a finite number.

`Number.isInteger()`

Determine whether the passed value is an integer.

`Number.isNaN()`

Determine whether the passed value is `NaN`.

`Number.isSafeInteger()`

Determine whether the passed value is a safe integer (number between -(2^{53} - 1) and 2^{53} - 1).

`Number.parseFloat()`

This is the same as the global `parseFloat()` function.

`Number.parseInt()`

This is the same as the global `parseInt()` function.

Instance properties

These properties are defined on `Number.prototype` and shared by all `Number` instances.

`Number.prototype.constructor`

The constructor function that created the instance object. For `Number` instances, the initial value is the `Number` constructor.

Instance methods

`Number.prototype.toExponential()`

Returns a string representing the number in exponential notation.

`Number.prototype.toFixed()`

Returns a string representing the number in fixed-point notation.

`Number.prototype.toLocaleString()`

Returns a string with a language sensitive representation of this number. Overrides the `Object.prototype.toLocaleString()` method.

`Number.prototype.toPrecision()`

Returns a string representing the number to a specified precision in fixed-point or exponential notation.

`Number.prototype.toString()`

Returns a string representing the specified object in the specified *radix* ("base"). Overrides the `Object.prototype.toString()` method.

`Number.prototype.valueOf()`

Returns the primitive value of the specified object. Overrides the `Object.prototype.valueOf()` method.

Examples

Using the Number object to assign values to numeric variables

The following example uses the `Number` object's properties to assign values to several numeric variables:

```
const biggestNum = Number.MAX_VALUE;
const smallestNum = Number.MIN_VALUE;
const infiniteNum = Number.POSITIVE_INFINITY;
const negInfiniteNum = Number.NEGATIVE_INFINITY;
const notANum = Number.NaN;
```

Integer range for Number

The following example shows the minimum and maximum integer values that can be represented as `Number` object.

```
const biggestInt = Number.MAX_SAFE_INTEGER; // (2**53 - 1) => 9007199254740991
const smallestInt = Number.MIN_SAFE_INTEGER; // -(2**53 - 1) => -9007199254740991
```

When parsing data that has been serialized to JSON, integer values falling outside of this range can be expected to become corrupted when JSON parser coerces them to `Number` type.

A possible workaround is to use `String` instead.

Larger numbers can be represented using the `BigInt` type.

Using Number() to convert a Date object

The following example converts the `Date` object to a numerical value using `Number` as a function:

```
const d = new Date("1995-12-17T03:24:00");
console.log(Number(d));
```

This logs `819199440000`.

Convert numeric strings and null to numbers

```
Number("123"); // 123
Number("123") === 123; // true
Number("12.3"); // 12.3
Number("12.00"); // 12
Number("123e-1"); // 12.3
Number(""); // 0
Number(null); // 0
Number("0x11"); // 17
Number("0b11"); // 3
Number("0o11"); // 9
Number("foo"); // NaN
Number("100a"); // NaN
Number("-Infinity"); // -Infinity
```

See also

- Polyfill of modern `Number` behavior (with support binary and octal literals) in `core-js`
- `NaN`
- Arithmetic operators
- `Math`
- `BigInt`

Number.EPSILON

The **`Number.EPSILON`** static data property represents the difference between 1 and the smallest floating point number greater than 1.

Try it

```
const result = Math.abs(0.2 - 0.3 + 0.1);

console.log(result);
// Expected output: 2.7755575615628914e-17
```

```
console.log(result < Number.EPSILON);
// Expected output: true
```

Value

2^{-52}, or approximately `2.220446049250313080847263336181 6E-16`.

Property attributes of `Number.EPSILON`	
Writable	no
Enumerable	no
Configurable	no

Description

`Number.EPSILON` is the difference between 1 and the next greater number representable in the Number format, because double precision floating point format only has 52 bits to represent the mantissa, and the lowest bit has a significance of 2^{-52}.

Note that the absolute accuracy of floating numbers decreases as the number gets larger, because the exponent grows while the mantissa's accuracy stays the same. `Number.MIN_VALUE` is the smallest representable positive number, which is much smaller than `Number.EPSILON`.

Because `EPSILON` is a static property of `Number`, you always use it as `Number.EPSILON`, rather than as a property of a number value.

Examples

Testing equality

Any number encoding system occupying a finite number of bits, of whatever base you choose (e.g., decimal or binary), will *necessarily* be unable to represent all numbers exactly, because you are trying to represent an infinite number of points on the number line using a finite amount of memory. For example, a base-10 (decimal) system cannot represent 1/3 exactly, and a base-2 (binary) system cannot represent `0.1` exactly. Thus, for example, `0.1 + 0.2` is not exactly equal to `0.3`:

```
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

For this reason, it is often advised that **floating point numbers should never be compared with `===`**. Instead, we can deem two numbers as equal if they are *close enough* to each other. The `Number.EPSILON` constant is usually a reasonable threshold for errors if the arithmetic is around the magnitude of `1`, because `EPSILON`, in essence, specifies how accurate the number "1" is.

```
function equal(x, y) {
  return Math.abs(x - y) < Number.EPSILON;
}

const x = 0.2;
const y = 0.3;
const z = 0.1;
console.log(equal(x + z, y)); // true
```

However, `Number.EPSILON` is inappropriate for any arithmetic operating on a larger magnitude. If your data is on the 10^3 order of magnitude, the decimal part will have a much smaller accuracy than `Number.EPSILON`:

```
function equal(x, y) {
  return Math.abs(x - y) < Number.EPSILON;
}

const x = 1000.1;
const y = 1000.2;
const z = 2000.3;
console.log(x + y); // 2000.3000000000002; error of 10^-13 instead of 10^-16
console.log(equal(x + y, z)); // false
```

In this case, a larger tolerance is required. As the numbers compared have a magnitude of approximately `2000`, a multiplier such as `2000 * Number.EPSILON` creates enough tolerance for this instance.

```
function equal(x, y, tolerance = Number.EPSILON) {
  return Math.abs(x - y) < tolerance;
}

const x = 1000.1;
const y = 1000.2;
const z = 2000.3;
console.log(equal(x + y, z, 2000 * Number.EPSILON)); // true
```

In addition to magnitude, it is important to consider the *accuracy* of your input. For example, if the numbers are collected from a form input and the input value can only be adjusted by steps of `0.1` (i.e., `<input type="number" step="0.1">`), it usually makes sense to allow a much larger tolerance, such as `0.01`, since the data only has a precision of `0.1`.

> **Note:** Important takeaway: do not simply use `Number.EPSILON` as a threshold for equality testing. Use a threshold that is appropriate for the magnitude and

> accuracy of the numbers you are comparing.

See also

- Polyfill of `Number.EPSILON` in `core-js`
- es-shims polyfill of `Number.EPSILON`
- `Number`

Number.isFinite()

The **`Number.isFinite()`** static method determines whether the passed value is a finite number — that is, it checks that a given value is a number, and the number is neither positive `Infinity`, negative `Infinity`, nor `NaN`.

Try it

```
console.log(Number.isFinite(1 / 0));
// Expected output: false

console.log(Number.isFinite(10 / 5));
// Expected output: true

console.log(Number.isFinite(0 / 0));
// Expected output: false
```

Syntax

```
Number.isFinite(value)
```

Parameters

`value`

The value to be tested for finiteness.

Return value

The boolean value `true` if the given value is a finite number. Otherwise `false`.

Examples

Using isFinite()

```
Number.isFinite(Infinity); // false
Number.isFinite(NaN); // false
Number.isFinite(-Infinity); // false

Number.isFinite(0); // true
Number.isFinite(2e64); // true
```

Difference between Number.isFinite() and global isFinite()

In comparison to the global `isFinite()` function, this method doesn't first convert the parameter to a number. This means only values of the type number *and* are finite return `true`, and non-numbers always return `false`.

```
isFinite("0"); // true; coerced to number 0
Number.isFinite("0"); // false
isFinite(null); // true; coerced to number 0
Number.isFinite(null); // false
```

See also

- Polyfill of `Number.isFinite` in `core-js`
- es-shims polyfill of `Number.isFinite`
- `Number`
- `isFinite()`

Number.isInteger()

The **`Number.isInteger()`** static method determines whether the passed value is an integer.

Try it

```
function fits(x, y) {
  if (Number.isInteger(y / x)) {
    return "Fits!";
  }
  return "Does NOT fit!";
}

console.log(fits(5, 10));
// Expected output: "Fits!"
```

```
console.log(fits(5, 11));
// Expected output: "Does NOT fit!"
```

Syntax

```
Number.isInteger(value)
```

Parameters

`value`

The value to be tested for being an integer.

Return value

The boolean value `true` if the given value is an integer. Otherwise `false`.

Description

If the target value is an integer, return `true`, otherwise return `false`. If the value is `NaN` or `Infinity`, return `false`. The method will also return `true` for floating point numbers that can be represented as integer. It will always return `false` if the value is not a number.

Note that some number literals, while looking like non-integers, actually represent integers — due to the precision limit of ECMAScript floating-point number encoding (IEEE-754). For example, `5.000000000000001` only differs from `5` by `1e-16`, which is too small to be represented. (For reference, `Number.EPSILON` stores the distance between 1 and the next representable floating-point number greater than 1, and that is about `2.22e-16`.) Therefore, `5.0000000000000001` will be represented with the same encoding as `5`, thus making `Number.isInteger(5.0000000000000001)` return `true`.

In a similar sense, numbers around the magnitude of `Number.MAX_SAFE_INTEGER` will suffer from loss of precision and make `Number.isInteger` return `true` even when it's not an integer. (The actual threshold varies based on how many bits are needed to represent the decimal — for example, `Number.isInteger(4500000000000000.1)` is `true`, but `Number.isInteger(4500000000000000.5)` is `false`.)

Examples

Using isInteger

```
Number.isInteger(0); // true
Number.isInteger(1); // true
Number.isInteger(-100000); // true
Number.isInteger(99999999999999999999); // true

Number.isInteger(0.1); // false
Number.isInteger(Math.PI); // false

Number.isInteger(NaN); // false
Number.isInteger(Infinity); // false
Number.isInteger(-Infinity); // false
Number.isInteger("10"); // false
Number.isInteger(true); // false
Number.isInteger(false); // false
Number.isInteger([1]); // false

Number.isInteger(5.0); // true
Number.isInteger(5.000000000000001); // false
Number.isInteger(5.0000000000000001); // true, because of loss of precision
Number.isInteger(4500000000000000.1); // true, because of loss of precision
```

See also

- Polyfill of `Number.isInteger` in `core-js`
- es-shims polyfill of `Number.isInteger`
- `Number`

Number.isNaN()

The **`Number.isNaN()`** static method determines whether the passed value is the number value `NaN`, and returns `false` if the input is not of the Number type. It is a more robust version of the original, global `isNaN()` function.

Try it

```
function typeOfNaN(x) {
  if (Number.isNaN(x)) {
    return "Number NaN";
  }
  if (isNaN(x)) {
    return "NaN";
  }
}

console.log(typeOfNaN("100F"));
// Expected output: "NaN"
```

```
console.log(typeOfNaN(NaN));
// Expected output: "Number NaN"
```

Syntax

```
Number.isNaN(value)
```

Parameters

`value`

The value to be tested for `NaN`.

Return value

The boolean value `true` if the given value is a number with value `NaN`. Otherwise, `false`.

Description

The function `Number.isNaN()` provides a convenient way to check for equality with `NaN`. Note that you cannot test for equality with `NaN` using either the `==` or `===` operators, because unlike all other value comparisons in JavaScript, these evaluate to `false` whenever one operand is `NaN`, even if the other operand is also `NaN`.

Since `x !== x` is only true for `NaN` among all possible JavaScript values, `Number.isNaN(x)` can also be replaced with a test for `x !== x`, despite the latter being less readable.

As opposed to the global `isNaN()` function, the `Number.isNaN()` method doesn't force-convert the parameter to a number. This makes it safe to pass values that would normally convert to `NaN` but aren't actually the same value as `NaN`. This also means that only values of the Number type that are also `NaN` return `true`.

Examples

Using isNaN()

```
Number.isNaN(NaN); // true
Number.isNaN(Number.NaN); // true
Number.isNaN(0 / 0); // true
Number.isNaN(37); // false
```

Difference between Number.isNaN() and global isNaN()

`Number.isNaN()` doesn't attempt to convert the parameter to a number, so non-numbers always return `false`. The following are all `false`:

```
Number.isNaN("NaN");
Number.isNaN(undefined);
Number.isNaN({});
Number.isNaN("blabla");
Number.isNaN(true);
Number.isNaN(null);
Number.isNaN("37");
Number.isNaN("37.37");
Number.isNaN("");
Number.isNaN(" ");
```

The global `isNaN()` coerces its parameter to a number:

```
isNaN("NaN"); // true
isNaN(undefined); // true
isNaN({}); // true
isNaN("blabla"); // true
isNaN(true); // false, this is coerced to 1
isNaN(null); // false, this is coerced to 0
isNaN("37"); // false, this is coerced to 37
isNaN("37.37"); // false, this is coerced to 37.37
isNaN(""); // false, this is coerced to 0
isNaN(" "); // false, this is coerced to 0
```

See also

- Polyfill of `Number.isNaN` in `core-js`
- es-shims polyfill of `Number.isNaN`
- `Number`
- `isNaN()`

Number.isSafeInteger()

The **`Number.isSafeInteger()`** static method determines whether the provided value is a number that is a *safe integer*.

Try it

```
function warn(x) {
  if (Number.isSafeInteger(x)) {
    return "Precision safe.";
  }
```

```
  return "Precision may be lost!";
}

console.log(warn(2 ** 53));
// Expected output: "Precision may be lost!"

console.log(warn(2 ** 53 - 1));
// Expected output: "Precision safe."
```

Syntax

```
Number.isSafeInteger(testValue)
```

Parameters

`testValue`

The value to be tested for being a safe integer.

Return value

The boolean value `true` if the given value is a number that is a safe integer. Otherwise `false`.

Description

The safe integers consist of all integers from $-(2^{53} - 1)$ to $2^{53} - 1$, inclusive (±9,007,199,254,740,991). A safe integer is an integer that:

- can be exactly represented as an IEEE-754 double precision number, and
- whose IEEE-754 representation cannot be the result of rounding any other integer to fit the IEEE-754 representation.

For example, $2^{53} - 1$ is a safe integer: it can be exactly represented, and no other integer rounds to it under any IEEE-754 rounding mode. In contrast, 2^{53} is *not* a safe integer: it can be exactly represented in IEEE-754, but the integer $2^{53} + 1$ can't be directly represented in IEEE-754 but instead rounds to 2^{53} under round-to-nearest and round-to-zero rounding.

Handling values larger or smaller than ~9 quadrillion with full precision requires using an arbitrary precision arithmetic library. See What Every Programmer Needs to Know about Floating Point Arithmetic for more information on floating point representations of numbers.

For larger integers, consider using the `BigInt` type.

Examples

Using isSafeInteger()

```
Number.isSafeInteger(3); // true
Number.isSafeInteger(2 ** 53); // false
Number.isSafeInteger(2 ** 53 - 1); // true
Number.isSafeInteger(NaN); // false
Number.isSafeInteger(Infinity); // false
Number.isSafeInteger("3"); // false
Number.isSafeInteger(3.1); // false
Number.isSafeInteger(3.0); // true
```

See also

- Polyfill of `Number.isSafeInteger` in `core-js`
- es-shims polyfill of `Number.isSafeInteger`
- `Number`
- `Number.MIN_SAFE_INTEGER`
- `Number.MAX_SAFE_INTEGER`
- `BigInt`

Number.MAX_SAFE_INTEGER

The **`Number.MAX_SAFE_INTEGER`** static data property represents the maximum safe integer in JavaScript ($2^{53} – 1$).

For larger integers, consider using `BigInt`.

Try it

```
const x = Number.MAX_SAFE_INTEGER + 1;
const y = Number.MAX_SAFE_INTEGER + 2;

console.log(Number.MAX_SAFE_INTEGER);
// Expected output: 9007199254740991

console.log(x);
// Expected output: 9007199254740992

console.log(x === y);
// Expected output: true
```

Value

`9007199254740991` (9,007,199,254,740,991, or ~9 quadrillion).

Property attributes of `Number.MAX_SAFE_INTEGER`	
Writable	no
Enumerable	no
Configurable	no

Description

Double precision floating point format only has 52 bits to represent the mantissa, so it can only safely represent integers between -(2^{53} – 1) and 2^{53} – 1. "Safe" in this context refers to the ability to represent integers exactly and to compare them correctly. For example, `Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2` will evaluate to true, which is mathematically incorrect. See `Number.isSafeInteger()` for more information.

As mentioned in `Number.EPSILON`, the precision of numbers depends on their magnitude. `Number.MAX_SAFE_INTEGER` represents the largest value at which integer-level operations can be performed precisely, but you can still perform meaningful arithmetic on numbers larger than that, just without integer-level precision. The largest representable number in JavaScript is actually `Number.MAX_VALUE`, which is approximately $1.7976931348623157 \times 10^{308}$.

Because `MAX_SAFE_INTEGER` is a static property of `Number`, you always use it as `Number.MAX_SAFE_INTEGER`, rather than as a property of a number value.

Examples

Return value of MAX_SAFE_INTEGER

```
Number.MAX_SAFE_INTEGER; // 9007199254740991
```

Relationship between MAX_SAFE_INTEGER and EPSILON

`Number.EPSILON` is 2^{-52}, while `MAX_SAFE_INTEGER` is $2^{53} - 1$ — both of them are derived from the width of the mantissa, which is 53 bits (with the highest bit always being 1). Multiplying them will give a value very close — but not equal — to 2.

```
Number.MAX_SAFE_INTEGER * Number.EPSILON; // 1.9999999999999998
```

See also

- Polyfill of `Number.MAX_SAFE_INTEGER` in `core-js`
- es-shims polyfill of `Number.MAX_SAFE_INTEGER`
- `Number.MIN_SAFE_INTEGER`
- `Number.MAX_VALUE`
- `Number.isSafeInteger()`
- `BigInt`

Number.MAX_VALUE

The **`Number.MAX_VALUE`** static data property represents the maximum numeric value representable in JavaScript.

Try it

```
function multiply(x, y) {
  if (x * y > Number.MAX_VALUE) {
    return "Process as Infinity";
  }
  return x * y;
}

console.log(multiply(1.7976931348623157e308, 1));
// Expected output: 1.7976931348623157e+308

console.log(multiply(1.7976931348623157e308, 2));
// Expected output: "Process as Infinity"
```

Value

2^{1024} - 2^{971}, or approximately `1.7976931348623157E+308`.

Property attributes of `Number.MAX_VALUE`	
Writable	no
Enumerable	no
Configurable	no

Description

Values larger than `MAX_VALUE` are represented as `Infinity` and will lose their actual value. As mentioned in `Number.EPSILON`, the precision of numbers depends on their magnitude. Integers can only be represented precisely up to `Number.MAX_SAFE_INTEGER`, which is 2^{53} - 1.

Because `MAX_VALUE` is a static property of `Number`, you always use it as `Number.MAX_VALUE`, rather than as a property of a number value.

Examples

Using MAX_VALUE

The following code multiplies two numeric values. If the result is less than or equal to `MAX_VALUE`, the `func1` function is called; otherwise, the `func2` function is called.

```
if (num1 * num2 <= Number.MAX_VALUE) {
  func1();
} else {
  func2();
}
```

See also

- `Number.MIN_VALUE`
- `Number.MAX_SAFE_INTEGER`
- `Number`

Number.MIN_SAFE_INTEGER

The **`Number.MIN_SAFE_INTEGER`** static data property represents the minimum safe integer in JavaScript, or $-(2^{53} - 1)$.

To represent integers smaller than this, consider using `BigInt`.

Try it

```
const x = Number.MIN_SAFE_INTEGER - 1;
const y = Number.MIN_SAFE_INTEGER - 2;

console.log(Number.MIN_SAFE_INTEGER);
// Expected output: -9007199254740991

console.log(x);
// Expected output: -9007199254740992

console.log(x === y);
// Expected output: true
```

Value

`-9007199254740991` (-9,007,199,254,740,991, or about -9 quadrillion).

Property attributes of `Number.MIN_SAFE_INTEGER`	
Writable	no
Enumerable	no
Configurable	no

Description

Double precision floating point format only has 52 bits to represent the mantissa, so it can only safely represent integers between $-(2^{53} - 1)$ and $2^{53} - 1$. Safe in this context refers to the ability to represent integers exactly and to correctly compare them. For example, `Number.MIN_SAFE_INTEGER - 1 === Number.MIN_SAFE_INTEGER - 2` will evaluate to true, which is mathematically incorrect. See `Number.isSafeInteger()` for more information.

As mentioned in `Number.EPSILON`, the precision of numbers depends on their magnitude. `Number.MIN_SAFE_INTEGER` represents the smallest value at which integer-level operations can be performed precisely, but you can still perform meaningful arithmetic on numbers more negative than that, just without integer-level precision. The largest representable number in JavaScript is actually `Number.MAX_VALUE`, which is approximately $1.7976931348623157 \times 10^{308}$.

Because `MIN_SAFE_INTEGER` is a static property of `Number`, you always use it as `Number.MIN_SAFE_INTEGER`, rather than as a property of a number value.

Examples

Using MIN_SAFE_INTEGER

```
Number.MIN_SAFE_INTEGER; // -9007199254740991
-(2 ** 53 - 1); // -9007199254740991
```

See also

- Polyfill of `Number.MIN_SAFE_INTEGER` in `core-js`
- es-shims polyfill of `Number.MIN_SAFE_INTEGER`
- `Number.MAX_SAFE_INTEGER`
- `Number.isSafeInteger()`
- `BigInt`

Number.MIN_VALUE

The **`Number.MIN_VALUE`** static data property represents the smallest positive numeric value representable in JavaScript.

Try it

```
function divide(x, y) {
  if (x / y < Number.MIN_VALUE) {
```

```
    return "Process as 0";
  }
  return x / y;
}

console.log(divide(5e-324, 1));
// Expected output: 5e-324

console.log(divide(5e-324, 2));
// Expected output: "Process as 0"
```

Value

2^{-1074}, or `5E-324`.

Property attributes of `Number.MIN_VALUE`	
Writable	no
Enumerable	no
Configurable	no

Description

`Number.MIN_VALUE` is the smallest positive number (not the most negative number) that can be represented within float precision — in other words, the number closest to 0. The ECMAScript spec doesn't define a precise value that implementations are required to support — instead the spec says, "*must be the smallest non-zero positive value that can actually be represented by the implementation*". This is because small IEEE-754 floating point numbers are denormalized, but implementations are not required to support this representation, in which case `Number.MIN_VALUE` may be larger.

In practice, its precise value in mainstream engines like V8 (used by Chrome, Edge, Node.js), SpiderMonkey (used by Firefox), and JavaScriptCore (used by Safari) is 2^{-1074}, or `5E-324`.

Because `MIN_VALUE` is a static property of `Number`, you always use it as `Number.MIN_VALUE`, rather than as a property of a number value.

Examples

Using MIN_VALUE

The following code divides two numeric values. If the result is greater than or equal to `MIN_VALUE`, the `func1` function is called; otherwise, the `func2` function is called.

```
if (num1 / num2 >= Number.MIN_VALUE) {
  func1();
} else {
  func2();
}
```

See also

- `Number.MAX_VALUE`

Number.NaN

The **`Number.NaN`** static data property represents Not-A-Number, which is equivalent to `NaN`. For more information about the behaviors of `NaN`, see the description for the global property.

Try it

```
function clean(x) {
  if (x === Number.NaN) {
    // Can never be true
    return null;
  }
  if (isNaN(x)) {
    return 0;
  }
}

console.log(clean(Number.NaN));
// Expected output: 0
```

Value

The number value `NaN`.

Property attributes of `Number.NaN`	
Writable	no
Enumerable	no
Configurable	no

Description

Because `NaN` is a static property of `Number`, you always use it as `Number.NaN`, rather than as a property of a number value.

Examples

Checking whether values are numeric

```
function sanitize(x) {
  if (isNaN(x)) {
    return Number.NaN;
  }
  return x;
}
```

See also

- `NaN`
- `Number.isNaN()`

Number.NEGATIVE_INFINITY

The **`Number.NEGATIVE_INFINITY`** static data property represents the negative Infinity value.

Try it

```
function checkNumber(smallNumber) {
  if (smallNumber === Number.NEGATIVE_INFINITY) {
    return "Process number as -Infinity";
  }
  return smallNumber;
}

console.log(checkNumber(-Number.MAX_VALUE));
// Expected output: -1.7976931348623157e+308

console.log(checkNumber(-Number.MAX_VALUE * 2));
// Expected output: "Process number as -Infinity"
```

Value

The same as the negative value of the global `Infinity` property.

Property attributes of `Number.NEGATIVE_INFINITY`	
Writable	no
Enumerable	no
Configurable	no

Description

The `Number.NEGATIVE_INFINITY` value behaves slightly differently than mathematical infinity:

- Any positive value, including `POSITIVE_INFINITY`, multiplied by `NEGATIVE_INFINITY` is `NEGATIVE_INFINITY`.
- Any negative value, including `NEGATIVE_INFINITY`, multiplied by `NEGATIVE_INFINITY` is `POSITIVE_INFINITY`.
- Any positive value divided by `NEGATIVE_INFINITY` is negative zero (as defined in IEEE 754).
- Any negative value divided by `NEGATIVE_INFINITY` is positive zero (as defined in IEEE 754).
- Zero multiplied by `NEGATIVE_INFINITY` is `NaN`.
- `NaN` multiplied by `NEGATIVE_INFINITY` is `NaN`.
- `NEGATIVE_INFINITY`, divided by any negative value except `NEGATIVE_INFINITY`, is `POSITIVE_INFINITY`.
- `NEGATIVE_INFINITY`, divided by any positive value except `POSITIVE_INFINITY`, is `NEGATIVE_INFINITY`.
- `NEGATIVE_INFINITY`, divided by either `NEGATIVE_INFINITY` or `POSITIVE_INFINITY`, is `NaN`.
- `x > Number.NEGATIVE_INFINITY` is true for any number *x* that isn't `NEGATIVE_INFINITY`.

You might use the `Number.NEGATIVE_INFINITY` property to indicate an error condition that returns a finite number in case of success. Note, however, that `NaN` would be more appropriate in such a case.

Because `NEGATIVE_INFINITY` is a static property of `Number`, you always use it as `Number.NEGATIVE_INFINITY`, rather than as a property of a number value.

Examples

Using NEGATIVE_INFINITY

In the following example, the variable `smallNumber` is assigned a value that is smaller than the minimum value. When the `if` statement executes, `smallNumber` has the value `-Infinity`, so `smallNumber` is set to a more manageable value before continuing.

```
let smallNumber = -Number.MAX_VALUE * 2;

if (smallNumber === Number.NEGATIVE_INFINITY) {
  smallNumber = returnFinite();
}
```

See also

- `Number.POSITIVE_INFINITY`
- `Number.isFinite()`
- `Infinity`
- `isFinite()`

Number() constructor

The **`Number()`** constructor creates `Number` objects. When called as a function, it returns primitive values of type Number.

Syntax

```
new Number(value)
Number(value)
```

> **Note:** `Number()` can be called with or without `new`, but with different effects. See Return value.

Parameters

`value`

The numeric value of the object being created.

Return value

When `Number()` is called as a function (without `new`), it returns `value` coerced to a number primitive. Specially, BigInts values are converted to numbers instead of throwing. If `value` is absent, it becomes `0`.

When `Number()` is called as a constructor (with `new`), it uses the coercion process above and returns a wrapping `Number` object, which is **not** a primitive.

> **Warning:** You should rarely find yourself using `Number` as a constructor.

Examples

Creating Number objects

```
const a = new Number("123"); // a === 123 is false
const b = Number("123"); // b === 123 is true
a instanceof Number; // is true
b instanceof Number; // is false
typeof a; // "object"
typeof b; // "number"
```

Using Number() to convert a BigInt to a number

`Number()` is the only case where a BigInt can be converted to a number without throwing, because it's very explicit.

```
+1n; // TypeError: Cannot convert a BigInt value to a number
0 + 1n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

```
Number(1n); // 1
```

Note that this may result in loss of precision, if the BigInt is too large to be safely represented.

```
BigInt(Number(2n ** 54n + 1n)) === 2n ** 54n + 1n; // false
```

See also

- Polyfill of modern `Number` behavior (with support binary and octal literals) in `core-js`
- `NaN`
- `Math`
- `BigInt`

Number.parseFloat()

The **`Number.parseFloat()`** static method parses an argument and returns a floating point number. If a number cannot be parsed from the argument, it returns `NaN`.

Try it

```
function circumference(r) {
  if (Number.isNaN(Number.parseFloat(r))) {
    return 0;
  }
  return parseFloat(r) * 2.0 * Math.PI;
}

console.log(circumference("4.567abcdefgh"));
// Expected output: 28.695307297889173
```

```
console.log(circumference("abcdefgh"));
// Expected output: 0
```

Syntax

```
Number.parseFloat(string)
```

Parameters

`string`

The value to parse, coerced to a string. Leading whitespace in this argument is ignored.

Return value

A floating point number parsed from the given `string`.

Or `NaN` when the first non-whitespace character cannot be converted to a number.

Examples

Number.parseFloat vs. parseFloat

This method has the same functionality as the global `parseFloat()` function:

```
Number.parseFloat === parseFloat; // true
```

Its purpose is modularization of globals.

See `parseFloat()` for more detail and examples.

See also

- Polyfill of `Number.parseFloat` in `core-js`
- es-shims polyfill of `Number.parseFloat`
- `Number`
- `parseFloat()`

Number.parseInt()

The **`Number.parseInt()`** static method parses a string argument and returns an integer of the specified radix or base.

Try it

```
function roughScale(x, base) {
  const parsed = Number.parseInt(x, base);
  if (Number.isNaN(parsed)) {
    return 0;
  }
  return parsed * 100;
}

console.log(roughScale(" 0xF", 16));
// Expected output: 1500

console.log(roughScale("321", 2));
// Expected output: 0
```

Syntax

```
Number.parseInt(string)
Number.parseInt(string, radix)
```

Parameters

`string`

The value to parse, coerced to a string. Leading whitespace in this argument is ignored.

`radix` Optional

An integer between `2` and `36` that represents the *radix* (the base in mathematical numeral systems) of the `string`.

If `radix` is undefined or `0`, it is assumed to be `10` except when the number begins with the code unit pairs `0x` or `0X`, in which case a radix of `16` is assumed.

Return value

An integer parsed from the given `string`.

If the `radix` is smaller than `2` or bigger than `36`, or the first non-whitespace character cannot be converted to a number, `NaN` is returned.

Examples

Number.parseInt vs. parseInt

This method has the same functionality as the global `parseInt()` function:

```
Number.parseInt === parseInt; // true
```

Its purpose is modularization of globals. Please see `parseInt()` for more detail and examples.

See also

- Polyfill of `Number.parseInt` in `core-js`
- es-shims polyfill of `Number.parseInt`
- `Number`
- `parseInt()`

Number.POSITIVE_INFINITY

The **`Number.POSITIVE_INFINITY`** static data property represents the positive Infinity value.

Try it

```
function checkNumber(bigNumber) {
  if (bigNumber === Number.POSITIVE_INFINITY) {
    return "Process number as Infinity";
  }
  return bigNumber;
}

console.log(checkNumber(Number.MAX_VALUE));
// Expected output: 1.7976931348623157e+308

console.log(checkNumber(Number.MAX_VALUE * 2));
// Expected output: "Process number as Infinity"
```

Value

The same as the value of the global `Infinity` property.

Property attributes of `Number.POSITIVE_INFINITY`	
Writable	no
Enumerable	no
Configurable	no

Description

The `Number.POSITIVE_INFINITY` value behaves slightly differently than mathematical infinity:

- Any positive value, including `POSITIVE_INFINITY`, multiplied by `POSITIVE_INFINITY` is `POSITIVE_INFINITY`.
- Any negative value, including `NEGATIVE_INFINITY`, multiplied by `POSITIVE_INFINITY` is `NEGATIVE_INFINITY`.
- Any positive number divided by `POSITIVE_INFINITY` is positive zero (as defined in IEEE 754).
- Any negative number divided by `POSITIVE_INFINITY` is negative zero (as defined in IEEE 754.
- Zero multiplied by `POSITIVE_INFINITY` is `NaN`.
- `NaN` multiplied by `POSITIVE_INFINITY` is `NaN`.
- `POSITIVE_INFINITY`, divided by any negative value except `NEGATIVE_INFINITY`, is `NEGATIVE_INFINITY`.
- `POSITIVE_INFINITY`, divided by any positive value except `POSITIVE_INFINITY`, is `POSITIVE_INFINITY`.
- `POSITIVE_INFINITY`, divided by either `NEGATIVE_INFINITY` or `POSITIVE_INFINITY`, is `NaN`.
- `Number.POSITIVE_INFINITY > x` is true for any number *x* that isn't `POSITIVE_INFINITY`.

You might use the `Number.POSITIVE_INFINITY` property to indicate an error condition that returns a finite number in case of success. Note, however, that `NaN` would be more appropriate in such a case.

Because `POSITIVE_INFINITY` is a static property of `Number`, you always use it as `Number.POSITIVE_INFINITY`, rather than as a property of a number value.

Examples

Using POSITIVE_INFINITY

In the following example, the variable `bigNumber` is assigned a value that is larger than the maximum value. When the `if` statement executes, `bigNumber` has the value `Infinity`, so `bigNumber` is set to a more manageable value before continuing.

```
let bigNumber = Number.MAX_VALUE * 2;

if (bigNumber === Number.POSITIVE_INFINITY) {
  bigNumber = returnFinite();
}
```

See also

- `Number.NEGATIVE_INFINITY`
- `Number.isFinite()`

- `Infinity`
- `isFinite()`

Number.prototype.toExponential()

The **`toExponential()`** method of `Number` values returns a string representing this number in exponential notation.

Try it

```
function expo(x, f) {
  return Number.parseFloat(x).toExponential(f);
}

console.log(expo(123456, 2));
// Expected output: "1.23e+5"

console.log(expo("123456"));
// Expected output: "1.23456e+5"

console.log(expo("oink"));
// Expected output: "NaN"
```

Syntax

```
toExponential()
toExponential(fractionDigits)
```

Parameters

`fractionDigits` Optional

Optional. An integer specifying the number of digits after the decimal point. Defaults to as many digits as necessary to specify the number.

Return value

A string representing the given `Number` object in exponential notation with one digit before the decimal point, rounded to `fractionDigits` digits after the decimal point.

Exceptions

`RangeError`

Thrown if `fractionDigits` is not between `0` and `100` (inclusive).

`TypeError`

Thrown if this method is invoked on an object that is not a `Number`.

Description

If the `fractionDigits` argument is omitted, the number of digits after the decimal point defaults to the number of digits necessary to represent the value uniquely.

If you use the `toExponential()` method for a numeric literal and the numeric literal has no exponent and no decimal point, leave whitespace(s) before the dot that precedes the method call to prevent the dot from being interpreted as a decimal point.

If a number has more digits than requested by the `fractionDigits` parameter, the number is rounded to the nearest number represented by `fractionDigits` digits. See the discussion of rounding in the description of the `toFixed()` method, which also applies to `toExponential()`.

Examples

Using toExponential

```
const numObj = 77.1234;

console.log(numObj.toExponential()); // 7.71234e+1
console.log(numObj.toExponential(4)); // 7.7123e+1
console.log(numObj.toExponential(2)); // 7.71e+1
console.log((77.1234).toExponential()); // 7.71234e+1
console.log((77).toExponential()); // 7.7e+1
```

See also

- Polyfill of `Number.prototype.toExponential` with many bug fixes in `core-js`
- es-shims polyfill of `Number.prototype.toExponential`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`
- `Number.prototype.toString()`

Number.prototype.toFixed()

The **`toFixed()`** method of `Number` values returns a string representing this number using fixed-point notation with the specified number of decimal places.

Try it

```
function financial(x) {
  return Number.parseFloat(x).toFixed(2);
}

console.log(financial(123.456));
// Expected output: "123.46"

console.log(financial(0.004));
// Expected output: "0.00"

console.log(financial("1.23e+5"));
// Expected output: "123000.00"
```

Syntax

```
toFixed()
toFixed(digits)
```

Parameters

`digits` Optional

The number of digits to appear after the decimal point; should be a value between `0` and `100`, inclusive. If this argument is omitted, it is treated as `0`.

Return value

A string representing the given number using fixed-point notation. Scientific notation is used if the number's magnitude (ignoring sign) is greater than or equal to 10^{21} (same return value as `Number.prototype.toString()`).

Exceptions

`RangeError`

Thrown if `digits` is not between `0` and `100` (inclusive).

`TypeError`

Thrown if this method is invoked on an object that is not a `Number`.

Description

The `toFixed()` method returns a string representation of a number without using exponential notation and with exactly `digits` digits after the decimal point. The number is rounded if necessary, and the fractional part is padded with zeros if necessary so that it has the specified length.

If the absolute value of the number is greater or equal to 10^{21}, this method uses the same algorithm as `Number.prototype.toString()` and returns a string in exponential notation. `toFixed()` returns `"Infinity"`, `"NaN"`, or `"-Infinity"` if the value of the number is non-finite.

The output of `toFixed()` may be more precise than `toString()` for some values, because `toString()` only prints enough significant digits to distinguish the number from adjacent number values. For example:

```
(1000000000000000128).toString(); // '1000000000000000100'
(1000000000000000128).toFixed(0); // '1000000000000000128'
```

However, choosing a `digits` precision that's too high can return unexpected results, because decimal fractional numbers cannot be represented precisely in floating point. For example:

```
(0.3).toFixed(50); // '0.29999999999999998889776975374843459576368331909180'
```

Examples

Using toFixed()

```
const numObj = 12345.6789;

numObj.toFixed(); // '12346'; rounding, no fractional part
numObj.toFixed(1); // '12345.7'; it rounds up
numObj.toFixed(6); // '12345.678900'; additional zeros
(1.23e20).toFixed(2); // '123000000000000000000.00'
(1.23e-10).toFixed(2); // '0.00'
(2.34).toFixed(1); // '2.3'
(2.35).toFixed(1); // '2.4'; it rounds up
(2.55).toFixed(1); // '2.5'
// it rounds down as it can't be represented exactly by a float and the
// closest representable float is lower
(2.449999999999999999).toFixed(1); // '2.5'
// it rounds up as it's less than Number.EPSILON away from 2.45.
// This literal actually encodes the same number value as 2.45

(6.02 * 10 ** 23).toFixed(50); // '6.019999999999999e+23'; large numbers still use exponential notation
```

Using toFixed() with negative numbers

Because member access has higher precedence than unary minus, you need to group the negative number expression to get a string.

```
-2.34.toFixed(1); // -2.3; a number
(-2.34).toFixed(1); // '-2.3'
```

See also

- `Number.prototype.toExponential()`
- `Number.prototype.toPrecision()`
- `Number.prototype.toString()`
- `Number.EPSILON`

Number.prototype.toLocaleString()

The **`toLocaleString()`** method of `Number` values returns a string with a language-sensitive representation of this number. In implementations with `Intl.NumberFormat` API support, this method delegates to `Intl.NumberFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.NumberFormat` object and use its `format()` method, because a `NumberFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

```
function eArabic(x) {
  return x.toLocaleString("ar-EG");
}

console.log(eArabic(123456.789));
// Expected output: "١٢٣٬٤٥٦٫٧٨٩"

console.log(eArabic("123456.789"));
// Expected output: "123456.789"

console.log(eArabic(NaN));
// Expected output: "ليس رقم"
```

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.NumberFormat` API, these parameters correspond exactly to the `Intl.NumberFormat()` constructor's parameters. Implementations without `Intl.NumberFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.NumberFormat()` constructor.

In implementations without `Intl.NumberFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.NumberFormat()` constructor.

In implementations without `Intl.NumberFormat` support, this parameter is ignored.

See the `Intl.NumberFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given number according to language-specific conventions.

In implementations with `Intl.NumberFormat`, this is equivalent to `new Intl.NumberFormat(locales, options).format(number)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const number = 3500;

console.log(number.toLocaleString()); // "3,500" if in U.S. English locale
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```
function toLocaleStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.NumberFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
const number = 123456.789;

// German uses comma as decimal separator and period for thousands
console.log(number.toLocaleString("de-DE"));
// 123.456,789

// Arabic in most Arabic speaking countries uses Eastern Arabic digits
console.log(number.toLocaleString("ar-EG"));
// ١٢٣٤٥٦٫٧٨٩

// India uses thousands/lakh/crore separators
console.log(number.toLocaleString("en-IN"));
// 1,23,456.789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(number.toLocaleString("zh-Hans-CN-u-nu-hanidec"));
// 一二三,四五六.七八九

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(number.toLocaleString(["ban", "id"]));
// 123.456,789
```

Using options

The results provided by `toLocaleString()` can be customized using the `options` parameter:

```
const number = 123456.789;

// request a currency format
console.log(
  number.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// 123.456,79 €

// the Japanese yen doesn't use a minor unit
console.log(
  number.toLocaleString("ja-JP", { style: "currency", currency: "JPY" }),
);
// ￥123,457

// limit to three significant digits
console.log(number.toLocaleString("en-IN", { maximumSignificantDigits: 3 }));
// 1,23,000

// Use the host default language with options for number formatting
const num = 30000.65;
console.log(
  num.toLocaleString(undefined, {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2,
  }),
);
// "30,000.65" where English is the default language, or
// "30.000,65" where German is the default language, or
// "30 000,65" where French is the default language
```

See also

- `Intl.NumberFormat`

- `Number.prototype.toString()`

Number.prototype.toPrecision()

The **`toPrecision()`** method of `Number` values returns a string representing this number to the specified number of significant digits.

Try it

```
function precise(x) {
  return x.toPrecision(4);
}

console.log(precise(123.456));
// Expected output: "123.5"

console.log(precise(0.004));
// Expected output: "0.004000"

console.log(precise(1.23e5));
// Expected output: "1.230e+5"
```

Syntax

```
toPrecision()
toPrecision(precision)
```

Parameters

`precision` Optional

An integer specifying the number of significant digits.

Return value

A string representing the given number, using the given number of significant digits. Scientific notation is used if the exponent is greater than or equal to `precision` or less than -6. Has the same behavior as `Number.prototype.toString()` if the `precision` argument is omitted.

Exceptions

`RangeError`

Thrown if `precision` is not between `1` and `100` (inclusive).

`TypeError`

Thrown if this method is invoked on an object that is not a `Number`.

Examples

Using <code>toPrecision</code>

```
// This number has exponent 0, so it will never use exponential notation
let num = 5.123456;

console.log(num.toPrecision()); // '5.123456'
console.log(num.toPrecision(5)); // '5.1235'
console.log(num.toPrecision(2)); // '5.1'
console.log(num.toPrecision(1)); // '5'

// This number has exponent -4, so it will never use exponential notation
num = 0.000123;

console.log(num.toPrecision()); // '0.000123'
console.log(num.toPrecision(5)); // '0.00012300'
console.log(num.toPrecision(2)); // '0.00012'
console.log(num.toPrecision(1)); // '0.0001'

// This number has exponent 3, so it will use exponential notation if precision is less than 4
num = 1234.5;
console.log(num.toPrecision(1)); // '1e+3'
console.log(num.toPrecision(2)); // '1.2e+3'
console.log(num.toPrecision(6)); // '1234.50'

// This number has exponent -7, so it will always use exponential notation
num = 0.00000012345;
console.log(num.toPrecision(1)); // '1e-7'
console.log(num.toPrecision(10)); // '1.234500000e-7'
```

See also

- `Number.prototype.toFixed()`
- `Number.prototype.toExponential()`
- `Number.prototype.toString()`

Number.prototype.toString()

The **`toString()`** method of `Number` values returns a string representing this number value.

Try it

```
function hexColor(c) {
  if (c < 256) {
    return Math.abs(c).toString(16);
  }
  return 0;
}

console.log(hexColor(233));
// Expected output: "e9"

console.log(hexColor("11"));
// Expected output: "b"
```

Syntax

```
toString()
toString(radix)
```

Parameters

`radix` Optional

An integer in the range `2` through `36` specifying the base to use for representing the number value. Defaults to 10.

Return value

A string representing the specified number value. Scientific notation is used if radix is 10 and the number's magnitude (ignoring sign) is greater than or equal to 10^{21} or less than 10^{-6}.

Exceptions

`RangeError`

Thrown if `radix` is less than 2 or greater than 36.

`TypeError`

Thrown if this method is invoked on an object that is not a `Number`.

Description

The `Number` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `Number` values, the `toString` method returns a string representation of the value in the specified radix.

For radixes above 10, the letters of the alphabet indicate digits greater than 9. For example, for hexadecimal numbers (base 16) `a` through `f` are used.

If the specified number value is negative, the sign is preserved. This is the case even if the radix is 2; the string returned is the positive binary representation of the number value preceded by a `-` sign, **not** the two's complement of the number value.

Both `0` and `-0` have `"0"` as their string representation. `Infinity` returns `"Infinity"` and `NaN` returns `"NaN"`.

If the number is not a whole number, the decimal point `.` is used to separate the decimal places. Scientific notation is used if the radix is 10 and the number's magnitude (ignoring sign) is greater than or equal to 10^{21} or less than 10^{-6}. In this case, the returned string always explicitly specifies the sign of the exponent.

```
console.log((10 ** 21.5).toString()); // "3.1622776601683794e+21"
console.log((10 ** 21.5).toString(8)); // "526665530627250154000000"
```

The underlying representation for floating point numbers is base-2 scientific notation (see number encoding). However, the `toString()` method doesn't directly use this most precise representation of the number value. Rather, the algorithm uses the least number of significant figures necessary to distinguish the output from adjacent number values. For example, if the number is large, there will be many equivalent string representations of the same floating point number, and `toString()` will choose the one with the most 0s to the right (for any given radix).

```
console.log((100000000000000128).toString()); // "100000000000000100"
console.log(100000000000000100 === 100000000000000128); // true
```

On the other hand, `Number.prototype.toFixed()` and `Number.prototype.toPrecision()` allow you to specify the precision and can be more precise than `toString()`.

The `toString()` method requires its `this` value to be a `Number` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to number values.

Because `Number` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `Number` *object* is used in a context expecting a string, such as in a template literal. However, Number *primitive* values do not consult the `toString()` method to be coerced to strings — rather, they are directly converted using the same algorithm as the initial `toString()` implementation.

```
Number.prototype.toString = () => "Overridden";
console.log(`${1}`); // "1"
```

```
console.log(`${new Number(1)}`); // "Overridden"
```

Examples

Using toString()

```
const count = 10;
console.log(count.toString()); // "10"

console.log((17).toString()); // "17"
console.log((17.2).toString()); // "17.2"

const x = 6;
console.log(x.toString(2)); // "110"
console.log((254).toString(16)); // "fe"
console.log((-10).toString(2)); // "-1010"
console.log((-0xff).toString(2)); // "-11111111"
```

Converting radix of number strings

If you have a string representing a number in a non-decimal radix, you can use `parseInt()` and `toString()` to convert it to a different radix.

```
const hex = "CAFEBABE";
const bin = parseInt(hex, 16).toString(2); // "11001010111111101011101010111110"
```

Beware of loss of precision: if the original number string is too large (larger than `Number.MAX_SAFE_INTEGER`, for example), you should use a `BigInt` instead. However, the `BigInt` constructor only has support for strings representing number literals (i.e., strings starting with `0b`, `0o`, `0x`). In case your original radix is not one of binary, octal, decimal, or hexadecimal, you may need to hand-write your radix converter, or use a library.

See also

- `Number.prototype.toFixed()`
- `Number.prototype.toExponential()`
- `Number.prototype.toPrecision()`

Number.prototype.valueOf()

The **`valueOf()`** method of `Number` values returns the value of this number.

Try it

```
const numObj = new Number(42);
console.log(typeof numObj);
// Expected output: "object"

const num = numObj.valueOf();
console.log(num);
// Expected output: 42

console.log(typeof num);
// Expected output: "number"
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

A number representing the primitive value of the specified `Number` object.

Description

This method is usually called internally by JavaScript and not explicitly in web code.

Examples

Using valueOf

```
const numObj = new Number(10);
console.log(typeof numObj); // object

const num = numObj.valueOf();
console.log(num); // 10
console.log(typeof num); // number
```

See also

- `Object.prototype.valueOf()`

Object

The **`Object`** type represents one of JavaScript's data types. It is used to store various keyed collections and more complex entities. Objects can be created using the `Object()` constructor or the object initializer / literal syntax.

Description

Nearly all objects in JavaScript are instances of `Object`; a typical object inherits properties (including methods) from `Object.prototype`, although these properties may be shadowed (a.k.a. overridden). The only objects that don't inherit from `Object.prototype` are those with `null` prototype, or descended from other `null` prototype objects.

Changes to the `Object.prototype` object are seen by **all** objects through prototype chaining, unless the properties and methods subject to those changes are overridden further along the prototype chain. This provides a very powerful although potentially dangerous mechanism to override or extend object behavior. To make it more secure, `Object.prototype` is the only object in the core JavaScript language that has immutable prototype — the prototype of `Object.prototype` is always `null` and not changeable.

Object prototype properties

You should avoid calling any of the `Object.prototype` methods directly from the instance, especially those that are not intended to be polymorphic (i.e., only its initial behavior makes sense and no descending object could override it in a meaningful way). All objects descending from `Object.prototype` may define a custom own property that has the same name, but with entirely different semantics from what you expect. Furthermore, these properties are not inherited by `null`-prototype objects. All modern JavaScript utilities for working with objects are static. More specifically:

- `valueOf()`, `toString()`, and `toLocaleString()` exist to be polymorphic and you should expect the object to define its own implementation with sensible behaviors, so you can call them as instance methods. However, `valueOf()` and `toString()` are usually implicitly called through type conversion and you don't need to call them yourself in your code.
- `__defineGetter__()`, `__defineSetter__()`, `__lookupGetter__()`, and `__lookupSetter__()` are deprecated and should not be used. Use the static alternatives `Object.defineProperty()` and `Object.getOwnPropertyDescriptor()` instead.
- The `__proto__` property is deprecated and should not be used. The `Object.getPrototypeOf()` and `Object.setPrototypeOf()` alternatives are static methods.
- The `propertyIsEnumerable()` and `hasOwnProperty()` methods can be replaced with the `Object.getOwnPropertyDescriptor()` and `Object.hasOwn()` static methods, respectively.
- The `isPrototypeOf()` method can usually be replaced with `instanceof`, if you are checking the `prototype` property of a constructor.

In case where a semantically equivalent static method doesn't exist, or if you really want to use the `Object.prototype` method, you should directly `call()` the `Object.prototype` method on your target object instead, to prevent the object from having an overriding property that produces unexpected results.

```
const obj = {
  foo: 1,
  // You should not define such a method on your own object,
  // but you may not be able to prevent it from happening if
  // you are receiving the object from external input
  propertyIsEnumerable() {
    return false;
  },
};

obj.propertyIsEnumerable("foo"); // false; unexpected result
Object.prototype.propertyIsEnumerable.call(obj, "foo"); // true; expected result
```

Deleting a property from an object

There isn't any method in an Object itself to delete its own properties (such as `Map.prototype.delete()`). To do so, one must use the `delete` operator.

null-prototype objects

Almost all objects in JavaScript ultimately inherit from `Object.prototype` (see inheritance and the prototype chain). However, you may create `null`-prototype objects using `Object.create(null)` or the object initializer syntax with `__proto__: null` (note: the `__proto__` key in object literals is different from the deprecated `Object.prototype.__proto__` property). You can also change the prototype of an existing object to `null` by calling `Object.setPrototypeOf(obj, null)`.

```
const obj = Object.create(null);
const obj2 = { __proto__: null };
```

An object with a `null` prototype can behave in unexpected ways, because it doesn't inherit any object methods from `Object.prototype`. This is especially true when debugging, since common object-property converting/detecting utility functions may generate errors, or lose information (especially if using silent error-traps that ignore errors).

For example, the lack of `Object.prototype.toString()` often makes debugging intractable:

```
const normalObj = {}; // create a normal object
const nullProtoObj = Object.create(null); // create an object with "null" prototype

console.log(`normalObj is: ${normalObj}`); // shows "normalObj is: [object Object]"
console.log(`nullProtoObj is: ${nullProtoObj}`); // throws error: Cannot convert object to primitive value

alert(normalObj); // shows [object Object]
alert(nullProtoObj); // throws error: Cannot convert object to primitive value
```

Other methods will fail as well.

```
normalObj.valueOf(); // shows {}
nullProtoObj.valueOf(); // throws error: nullProtoObj.valueOf is not a function
```

```
normalObj.hasOwnProperty("p"); // shows "true"
nullProtoObj.hasOwnProperty("p"); // throws error: nullProtoObj.hasOwnProperty is not a function

normalObj.constructor; // shows "Object() { [native code] }"
nullProtoObj.constructor; // shows "undefined"
```

We can add the `toString` method back to the null-prototype object by assigning it one:

```
nullProtoObj.toString = Object.prototype.toString; // since new object lacks toString, add the original generic one back

console.log(nullProtoObj.toString()); // shows "[object Object]"
console.log(`nullProtoObj is: ${nullProtoObj}`); // shows "nullProtoObj is: [object Object]"
```

Unlike normal objects, in which `toString()` is on the object's prototype, the `toString()` method here is an own property of `nullProtoObj`. This is because `nullProtoObj` has no (`null`) prototype.

You can also revert a null-prototype object back to an ordinary object using `Object.setPrototypeOf(nullProtoObj, Object.prototype)`.

In practice, objects with `null` prototype are usually used as a cheap substitute for maps. The presence of `Object.prototype` properties will cause some bugs:

```
const ages = { alice: 18, bob: 27 };

function hasPerson(name) {
  return name in ages;
}

function getAge(name) {
  return ages[name];
}

hasPerson("hasOwnProperty"); // true
getAge("toString"); // [Function: toString]
```

Using a null-prototype object removes this hazard without introducing too much complexity to the `hasPerson` and `getAge` functions:

```
const ages = Object.create(null, {
  alice: { value: 18, enumerable: true },
  bob: { value: 27, enumerable: true },
});

hasPerson("hasOwnProperty"); // false
getAge("toString"); // undefined
```

In such case, the addition of any method should be done cautiously, as they can be confused with the other key-value pairs stored as data.

Making your object not inherit from `Object.prototype` also prevents prototype pollution attacks. If a malicious script adds a property to `Object.prototype`, it will be accessible on every object in your program, except objects that have null prototype.

```
const user = {};

// A malicious script:
Object.prototype.authenticated = true;

// Unexpectedly allowing unauthenticated user to pass through
if (user.authenticated) {
  // access confidential data
}
```

JavaScript also has built-in APIs that produce `null`-prototype objects, especially those that use objects as ad hoc key-value collections. For example:

- The return value of `Object.groupBy()`
- The `groups` and `indices.groups` properties of the result of `RegExp.prototype.exec()`
- `Array.prototype[Symbol.unscopables]` (all `[Symbol.unscopables]` objects should have `null`-prototype)
- `import.meta`
- Module namespace objects, obtained through `import * as ns from "module";` or `import()`

The term "`null`-prototype object" often also includes any object without `Object.prototype` in its prototype chain. Such objects can be created with `extends null` when using classes.

Object coercion

Many built-in operations that expect objects first coerce their arguments to objects. The operation can be summarized as follows:

- Objects are returned as-is.
- `undefined` and `null` throw a `TypeError`.
- `Number`, `String`, `Boolean`, `Symbol`, `BigInt` primitives are wrapped into their corresponding object wrappers.

There are two ways to achieve nearly the same effect in JavaScript.

- `Object.prototype.valueOf()`: `Object.prototype.valueOf.call(x)` does exactly the object coercion steps explained above to convert `x`.
- The `Object()` function: `Object(x)` uses the same algorithm to convert `x`, except that `undefined` and `null` don't throw a `TypeError`, but return a plain object.

Places that use object coercion include:

- The `object` parameter of `for...in` loops.
- The `this` value of `Array` methods.
- Parameters of `Object` methods such as `Object.keys()`.
- Auto-boxing when a property is accessed on a primitive value, since primitives do not have properties.
- The `this` value when calling a non-strict function. Primitives are boxed while `null` and `undefined` are replaced with the global object.

Unlike conversion to primitives, the object coercion process itself is not observable in any way, since it doesn't invoke custom code like `toString` or `valueOf` methods.

Constructor

`Object()`

Turns the input into an object.

Static methods

`Object.assign()`

Copies the values of all enumerable own properties from one or more source objects to a target object.

`Object.create()`

Creates a new object with the specified prototype object and properties.

`Object.defineProperties()`

Adds the named properties described by the given descriptors to an object.

`Object.defineProperty()`

Adds the named property described by a given descriptor to an object.

`Object.entries()`

Returns an array containing all of the `[key, value]` pairs of a given object's **own** enumerable string properties.

`Object.freeze()`

Freezes an object. Other code cannot delete or change its properties.

`Object.fromEntries()`

Returns a new object from an iterable of `[key, value]` pairs. (This is the reverse of `Object.entries`).

`Object.getOwnPropertyDescriptor()`

Returns a property descriptor for a named property on an object.

`Object.getOwnPropertyDescriptors()`

Returns an object containing all own property descriptors for an object.

`Object.getOwnPropertyNames()`

Returns an array containing the names of all of the given object's **own** enumerable and non-enumerable properties.

`Object.getOwnPropertySymbols()`

Returns an array of all symbol properties found directly upon a given object.

`Object.getPrototypeOf()`

Returns the prototype (internal `[[Prototype]]` property) of the specified object.

`Object.groupBy()`

Groups the elements of a given iterable according to the string values returned by a provided callback function. The returned object has separate properties for each group, containing arrays with the elements in the group.

`Object.hasOwn()`

Returns `true` if the specified object has the indicated property as its *own* property, or `false` if the property is inherited or does not exist.

`Object.is()`

Compares if two values are the same value. Equates all `NaN` values (which differs from both `IsLooselyEqual` used by `==` and `IsStrictlyEqual` used by `===`).

`Object.isExtensible()`

Determines if extending of an object is allowed.

`Object.isFrozen()`

Determines if an object was frozen.

`Object.isSealed()`

Determines if an object is sealed.

`Object.keys()`

Returns an array containing the names of all of the given object's **own** enumerable string properties.

`Object.preventExtensions()`

Prevents any extensions of an object.

`Object.seal()`

Prevents other code from deleting properties of an object.

`Object.setPrototypeOf()`

Sets the object's prototype (its internal `[[Prototype]]` property).

`Object.values()`

Returns an array containing the values that correspond to all of a given object's **own** enumerable string properties.

Instance properties

These properties are defined on `Object.prototype` and shared by all `Object` instances.

`Object.prototype.__proto__`

Points to the object which was used as prototype when the object was instantiated.

`Object.prototype.constructor`

The constructor function that created the instance object. For plain `Object` instances, the initial value is the `Object` constructor. Instances of other constructors each inherit the `constructor` property from their respective `Constructor.prototype` object.

Instance methods

`Object.prototype.__defineGetter__()`

Associates a function with a property that, when accessed, executes that function and returns its return value.

`Object.prototype.__defineSetter__()`

Associates a function with a property that, when set, executes that function which modifies the property.

`Object.prototype.__lookupGetter__()`

Returns the function bound as a getter to the specified property.

`Object.prototype.__lookupSetter__()`

Returns the function bound as a setter to the specified property.

`Object.prototype.hasOwnProperty()`

Returns a boolean indicating whether an object contains the specified property as a direct property of that object and not inherited through the prototype chain.

`Object.prototype.isPrototypeOf()`

Returns a boolean indicating whether the object this method is called upon is in the prototype chain of the specified object.

`Object.prototype.propertyIsEnumerable()`

Returns a boolean indicating whether the specified property is the object's enumerable own property.

`Object.prototype.toLocaleString()`

Calls `toString()`.

`Object.prototype.toString()`

Returns a string representation of the object.

`Object.prototype.valueOf()`

Returns the primitive value of the specified object.

Examples

Constructing empty objects

The following example creates empty objects using the `new` keyword with different arguments:

```
const o1 = new Object();
const o2 = new Object(undefined);
const o3 = new Object(null);
```

Using Object() constructor to turn primitives into an Object of their respective type

You can use the `Object()` constructor to create an object wrapper of a primitive value.

The following examples create variables `o1` and `o2` which are objects storing `Boolean` and `BigInt` values:

```
// Equivalent to const o1 = new Boolean(true)
const o1 = new Object(true);

// No equivalent because BigInt() can't be called as a constructor,
// and calling it as a regular function won't create an object
const o2 = new Object(1n);
```

Object prototypes

When altering the behavior of existing `Object.prototype` methods, consider injecting code by wrapping your extension before or after the existing logic. For example, this (untested) code will pre-conditionally execute custom logic before the built-in logic or someone else's extension is executed.

When modifying prototypes with hooks, pass `this` and the arguments (the call state) to the current behavior by calling `apply()` on the function. This pattern can be used for any prototype, such as `Node.prototype`, `Function.prototype`, etc.

```
const current = Object.prototype.valueOf;

// Since my property "-prop-value" is cross-cutting and isn't always
// on the same prototype chain, I want to modify Object.prototype:
Object.prototype.valueOf = function (...args) {
  if (Object.hasOwn(this, "-prop-value")) {
    return this["-prop-value"];
  }
  // It doesn't look like one of my objects, so let's fall back on
  // the default behavior by reproducing the current behavior as best we can.
  // The apply behaves like "super" in some other languages.
  // Even though valueOf() doesn't take arguments, some other hook may.
  return current.apply(this, args);
};
```

> **Warning:** Modifying the `prototype` property of any built-in constructor is considered a bad practice and risks forward compatibility.

You can read more about prototypes in Inheritance and the prototype chain.

See also

- Object initializer
- Prototype pollution attack

Object.prototype.__defineGetter__()

> **Note:** This feature is deprecated in favor of defining getters using the object initializer syntax or the `Object.defineProperty()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__defineGetter__()`** method of `Object` instances binds an object's property to a function to be called when that property is looked up.

Syntax

```
__defineGetter__(prop, func)
```

Parameters

`prop`

A string containing the name of the property that the getter `func` is bound to.

`func`

A function to be bound to a lookup of the specified property.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if `func` is not a function.

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__defineGetter__()` method. This method allows a getter to be defined on a pre-existing object. This is equivalent to `Object.defineProperty(obj, prop, { get: func, configurable: true, enumerable: true })`, which means the property is enumerable and configurable, and any existing setter, if present, is preserved.

`__defineGetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__defineGetter__()`, it also needs to implement the `__lookupGetter__()`, `__lookupSetter__()`, and `__defineSetter__()` methods.

Examples

Using __defineGetter__()

```
const o = {};
o.__defineGetter__("gimmeFive", function () {
  return 5;
});
console.log(o.gimmeFive); // 5
```

Defining a getter property in standard ways

You can use the `get` syntax to define a getter when the object is first initialized.

```
const o = {
  get gimmeFive() {
    return 5;
  },
};
console.log(o.gimmeFive); // 5
```

You may also use `Object.defineProperty()` to define a getter on an object after it's been created. Compared to `__defineGetter__()`, this method allows you to control the getter's enumerability and configurability, as well as defining symbol properties. The `Object.defineProperty()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__defineGetter__()` method.

```
const o = {};
Object.defineProperty(o, "gimmeFive", {
  get() {
    return 5;
  },
  configurable: true,
  enumerable: true,
});
console.log(o.gimmeFive); // 5
```

See also

- Polyfill of `Object.prototype.__defineGetter__` in `core-js`
- `Object.prototype.__defineSetter__()`
- `get`
- `Object.defineProperty()`
- `Object.prototype.__lookupGetter__()`
- `Object.prototype.__lookupSetter__()`
- JS Guide: Defining Getters and Setters
- Firefox bug 647423

Object.prototype.__defineSetter__()

> **Note:** This feature is deprecated in favor of defining setters using the object initializer syntax or the `Object.defineProperty()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__defineSetter__()`** method of `Object` instances binds an object's property to a function to be called when an attempt is made to set that property.

Syntax

```
__defineSetter__(prop, func)
```

Parameters

`prop`

A string containing the name of the property that the setter `func` is bound to.

`func`

A function to be called when there is an attempt to set the specified property. This function receives the following parameter:

`val`

The value attempted to be assigned to `prop`.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if `func` is not a function.

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__defineSetter__()` method. This method allows a setter to be defined on a pre-existing object. This is equivalent to `Object.defineProperty(obj, prop, { set: func, configurable: true, enumerable: true })`, which means the property is enumerable and configurable, and any existing getter, if present, is preserved.

`__defineSetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__defineSetter__()`, it also needs to implement the `__lookupGetter__()`, `__lookupSetter__()`, and `__defineGetter__()` methods.

Examples

Using __defineSetter__()

```
const o = {};
o.__defineSetter__("value", function (val) {
  this.anotherValue = val;
});
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

Defining a setter property in standard ways

You can use the `set` syntax to define a setter when the object is first initialized.

```
const o = {
  set value(val) {
    this.anotherValue = val;
  },
};
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

You may also use `Object.defineProperty()` to define a setter on an object after it's been created. Compared to `__defineSetter__()`, this method allows you to control the setter's enumerability and configurability, as well as defining symbol properties. The `Object.defineProperty()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__defineSetter__()` method.

```
const o = {};
Object.defineProperty(o, "value", {
  set(val) {
    this.anotherValue = val;
  },
  configurable: true,
  enumerable: true,
});
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

See also

- Polyfill of `Object.prototype.__defineSetter__` in `core-js`
- `Object.prototype.__defineGetter__()`
- `set`
- `Object.defineProperty()`
- `Object.prototype.__lookupGetter__()`
- `Object.prototype.__lookupSetter__()`
- JS Guide: Defining Getters and Setters
- Firefox bug 647423

Object.prototype.__lookupGetter__()

> **Note:** This feature is deprecated in favor of the `Object.getOwnPropertyDescriptor()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__lookupGetter__()`** method of `Object` instances returns the function bound as a getter to the specified property.

Syntax

```
__lookupGetter__(prop)
```

Parameters

`prop`

A string containing the name of the property whose getter should be returned.

Return value

The function bound as a getter to the specified property. Returns `undefined` if no such property is found, or the property is a data property.

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__lookupGetter__()` method. If a getter has been defined for an object's property, it's not possible to reference the getter function through that property, because that property refers to the return value of that function. `__lookupGetter__()` can be used to obtain a reference to the getter function.

`__lookupGetter__()` walks up the prototype chain to find the specified property. If any object along the prototype chain has the specified own property, the `get` attribute of the property descriptor for that property is returned. If that property is a data property, `undefined` is returned. If the property is not found along the entire prototype chain, `undefined` is also returned.

`__lookupGetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__lookupGetter__()`, it also needs to implement the `__lookupSetter__()`, `__defineGetter__()`, and `__defineSetter__()` methods.

Examples

Using __lookupGetter__()

```
const obj = {
  get foo() {
    return Math.random() > 0.5 ? "foo" : "bar";
  },
};

obj.__lookupGetter__("foo");
// [Function: get foo]
```

Looking up a property's getter in the standard way

You should use the `Object.getOwnPropertyDescriptor()` API to look up a property's getter. Compared to `__lookupGetter__()`, this method allows looking up symbol properties. The `Object.getOwnPropertyDescriptor()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__lookupGetter__()` method. If `__lookupGetter__()`'s behavior of walking up the prototype chain is important, you may implement it yourself with `Object.getPrototypeOf()`.

```
const obj = {
  get foo() {
    return Math.random() > 0.5 ? "foo" : "bar";
  },
};

Object.getOwnPropertyDescriptor(obj, "foo").get;
// [Function: get foo]
```

```
const obj2 = {
  __proto__: {
    get foo() {
      return Math.random() > 0.5 ? "foo" : "bar";
    },
  },
};

function findGetter(obj, prop) {
  while (obj) {
    const desc = Object.getOwnPropertyDescriptor(obj, prop);
    if (desc) {
      return desc.get;
    }
    obj = Object.getPrototypeOf(obj);
  }
}

console.log(findGetter(obj2, "foo")); // [Function: get foo]
```

See also

- Polyfill of `Object.prototype.__lookupGetter__` in `core-js`
- `Object.prototype.__lookupSetter__()`
- `get`
- `Object.getOwnPropertyDescriptor()`
- `Object.prototype.__defineGetter__()`

- `Object.prototype.__defineSetter__()`
- JS Guide: Defining Getters and Setters

Object.prototype.__lookupSetter__()

> **Note:** This feature is deprecated in favor of the `Object.getOwnPropertyDescriptor()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__lookupSetter__()`** method of `Object` instances returns the function bound as a setter to the specified property.

Syntax

```
__lookupSetter__(prop)
```

Parameters

`prop`

A string containing the name of the property whose setter should be returned.

Return value

The function bound as a setter to the specified property. Returns `undefined` if no such property is found, or the property is a data property.

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__lookupSetter__()` method. If a setter has been defined for an object's property, it's not possible to reference the setter function through that property, because that property only calls the function when it's being set. `__lookupSetter__()` can be used to obtain a reference to the setter function.

`__lookupSetter__()` walks up the prototype chain to find the specified property. If any object along the prototype chain has the specified own property, the `set` attribute of the property descriptor for that property is returned. If that property is a data property, `undefined` is returned. If the property is not found along the entire prototype chain, `undefined` is also returned.

`__lookupSetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__lookupSetter__()`, it also needs to implement the `__lookupGetter__()`, `__defineGetter__()`, and `__defineSetter__()` methods.

Examples

Using __lookupSetter__()

```
const obj = {
  set foo(value) {
    this.bar = value;
  },
};

obj.__lookupSetter__("foo");
// [Function: set foo]
```

Looking up a property's setter in the standard way

You should use the `Object.getOwnPropertyDescriptor()` API to look up a property's setter. Compared to `__lookupSetter__()`, this method allows looking up symbol properties. The `Object.getOwnPropertyDescriptor()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__lookupSetter__()` method. If `__lookupSetter__()`'s behavior of walking up the prototype chain is important, you may implement it yourself with `Object.getPrototypeOf()`.

```
const obj = {
  set foo(value) {
    this.bar = value;
  },
};

Object.getOwnPropertyDescriptor(obj, "foo").set;
// [Function: set foo]
```

```
const obj2 = {
  __proto__: {
    set foo(value) {
      this.bar = value;
    },
  },
};

function findSetter(obj, prop) {
  while (obj) {
    const desc = Object.getOwnPropertyDescriptor(obj, prop);
    if (desc) {
      return desc.set;
    }
```

```
    obj = Object.getPrototypeOf(obj);
  }
}

console.log(findSetter(obj2, "foo")); // [Function: set foo]
```

See also

- Polyfill of `Object.prototype.__lookupSetter__` in `core-js`
- `Object.prototype.__lookupGetter__()`
- `set`
- `Object.getOwnPropertyDescriptor()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- JS Guide: Defining Getters and Setters

Object.assign()

The **`Object.assign()`** static method copies all enumerable own properties from one or more *source objects* to a *target object*. It returns the modified target object.

Try it

```
const target = { a: 1, b: 2 };
const source = { b: 4, c: 5 };

const returnedTarget = Object.assign(target, source);

console.log(target);
// Expected output: Object { a: 1, b: 4, c: 5 }

console.log(returnedTarget === target);
// Expected output: true
```

Syntax

```
Object.assign(target)
Object.assign(target, source1)
Object.assign(target, source1, source2)
Object.assign(target, source1, source2, /* …, */ sourceN)
```

Parameters

`target`

> The target object — what to apply the sources' properties to, which is returned after it is modified. If a primitive value is provided as the target, it will be converted to an object.

`source1, ..., sourceN`

> The source object(s) — objects containing the properties you want to apply.

Return value

The target object.

Exceptions

`TypeError`

> Thrown in one of the following cases:
>
> - The `target` parameter is `null` or `undefined`.
> - Assignment of a property on the target object fails; for example, because the property is non-writable on the target object, or because its setter throws an error.

Description

Properties in the target object are overwritten by properties in the sources if they have the same key. Later sources' properties overwrite earlier ones.

The `Object.assign()` method only copies *enumerable* and *own* properties from a source object to a target object. It uses `[[Get]]` on the source and `[[Set]]` on the target, so it will invoke getters and setters. Therefore it *assigns* properties, versus copying or defining new properties. This may make it unsuitable for merging new properties into a prototype if the merge sources contain getters.

For copying property definitions (including their enumerability) into prototypes, use `Object.getOwnPropertyDescriptor()` and `Object.defineProperty()` instead.

Both `String` and `Symbol` properties are copied.

In case of an error, for example if a property is non-writable, a `TypeError` is raised, and the `target` object is changed if any properties are added before the error is raised.

> **Note:** `Object.assign()` does not throw on `null` or `undefined` sources.

Examples

Cloning an object

```
const obj = { a: 1 };
const copy = Object.assign({}, obj);
console.log(copy); // { a: 1 }
```

Warning for deep clone

For deep cloning, we need to use alternatives like `structuredClone()`, because `Object.assign()` copies property values.

If the source value is a reference to an object, it only copies the reference value.

```
const obj1 = { a: 0, b: { c: 0 } };
const obj2 = Object.assign({}, obj1);
console.log(obj2); // { a: 0, b: { c: 0 } }

obj1.a = 1;
console.log(obj1); // { a: 1, b: { c: 0 } }
console.log(obj2); // { a: 0, b: { c: 0 } }

obj2.a = 2;
console.log(obj1); // { a: 1, b: { c: 0 } }
console.log(obj2); // { a: 2, b: { c: 0 } }

obj2.b.c = 3;
console.log(obj1); // { a: 1, b: { c: 3 } }
console.log(obj2); // { a: 2, b: { c: 3 } }

// Deep Clone
const obj3 = { a: 0, b: { c: 0 } };
const obj4 = structuredClone(obj3);
obj3.a = 4;
obj3.b.c = 4;
console.log(obj4); // { a: 0, b: { c: 0 } }
```

Merging objects

```
const o1 = { a: 1 };
const o2 = { b: 2 };
const o3 = { c: 3 };

const obj = Object.assign(o1, o2, o3);
console.log(obj); // { a: 1, b: 2, c: 3 }
console.log(o1); // { a: 1, b: 2, c: 3 }, target object itself is changed.
```

Merging objects with same properties

```
const o1 = { a: 1, b: 1, c: 1 };
const o2 = { b: 2, c: 2 };
const o3 = { c: 3 };

const obj = Object.assign({}, o1, o2, o3);
console.log(obj); // { a: 1, b: 2, c: 3 }
```

The properties are overwritten by other objects that have the same properties later in the parameters order.

Copying symbol-typed properties

```
const o1 = { a: 1 };
const o2 = { [Symbol("foo")]: 2 };

const obj = Object.assign({}, o1, o2);
console.log(obj); // { a : 1, [Symbol("foo")]: 2 } (cf. bug 1207182 on Firefox)
Object.getOwnPropertySymbols(obj); // [Symbol(foo)]
```

Properties on the prototype chain and non-enumerable properties cannot be copied

```
const obj = Object.create(
  // foo is on obj's prototype chain.
  { foo: 1 },
  {
    bar: {
      value: 2, // bar is a non-enumerable property.
    },
    baz: {
      value: 3,
      enumerable: true, // baz is an own enumerable property.
    },
  },
);
```

```
const copy = Object.assign({}, obj);
console.log(copy); // { baz: 3 }
```

Primitives will be wrapped to objects

```
const v1 = "abc";
const v2 = true;
const v3 = 10;
const v4 = Symbol("foo");

const obj = Object.assign({}, v1, null, v2, undefined, v3, v4);
// Primitives will be wrapped, null and undefined will be ignored.
// Note, only string wrappers can have own enumerable properties.
console.log(obj); // { "0": "a", "1": "b", "2": "c" }

// Primitives as the target are also wrapped to objects
const number = Object.assign(3, { a: 1 });
console.log(number); // Number {3, a: 1}
console.log(typeof number); // object
console.log(number.a); // 1

// null and undefined as targets throw TypeError
try {
  Object.assign(null, { a: 1 });
} catch (e) {
  console.log(e.message); // "Cannot convert undefined or null to object"
}
```

Exceptions will interrupt the ongoing copying task

```
const target = Object.defineProperty({}, "foo", {
  value: 1,
  writable: false,
}); // target.foo is a read-only property

Object.assign(target, { bar: 2 }, { foo2: 3, foo: 3, foo3: 3 }, { baz: 4 });
// TypeError: "foo" is read-only
// The Exception is thrown when assigning target.foo

console.log(target.bar); // 2, the first source was copied successfully.
console.log(target.foo2); // 3, the first property of the second source was copied successfully.
console.log(target.foo); // 1, exception is thrown here.
console.log(target.foo3); // undefined, assign method has finished, foo3 will not be copied.
console.log(target.baz); // undefined, the third source will not be copied either.
```

Copying accessors

```
const obj = {
  foo: 1,
  get bar() {
    return 2;
  },
};

let copy = Object.assign({}, obj);
console.log(copy);
// { foo: 1, bar: 2 }
// The value of copy.bar is obj.bar's getter's return value.

// This is an assign function that copies full descriptors
function completeAssign(target, ...sources) {
  sources.forEach((source) => {
    const descriptors = Object.keys(source).reduce((descriptors, key) => {
      descriptors[key] = Object.getOwnPropertyDescriptor(source, key);
      return descriptors;
    }, {});

    // By default, Object.assign copies enumerable Symbols, too
    Object.getOwnPropertySymbols(source).forEach((sym) => {
      const descriptor = Object.getOwnPropertyDescriptor(source, sym);
      if (descriptor.enumerable) {
        descriptors[sym] = descriptor;
      }
    });
    Object.defineProperties(target, descriptors);
  });
  return target;
}

copy = completeAssign({}, obj);
console.log(copy);
// { foo:1, get bar() { return 2 } }
```

See also

- Polyfill of `Object.assign` in `core-js`
- es-shims polyfill of `Object.assign`
- `Object.defineProperties()`
- Enumerability and ownership of properties
- Spread in object literals

Object.prototype.constructor

The **`constructor`** data property of an `Object` instance returns a reference to the constructor function that created the instance object. Note that the value of this property is a reference to *the function itself*, not a string containing the function's name.

> **Note:** This is a property of JavaScript objects. For the `constructor` method in classes, see its own reference page.

Value

A reference to the constructor function that created the instance object.

Property attributes of `Object.prototype.constructor`	
Writable	yes
Enumerable	no
Configurable	yes

> **Note:** This property is created by default on the `prototype` property of every constructor function and is inherited by all objects created by that constructor.

Description

Any object (with the exception of `null` prototype objects) will have a `constructor` property on its `[[Prototype]]`. Objects created with literals will also have a `constructor` property that points to the constructor type for that object — for example, array literals create `Array` objects, and object literals create plain objects.

```
const o1 = {};
o1.constructor === Object; // true

const o2 = new Object();
o2.constructor === Object; // true

const a1 = [];
a1.constructor === Array; // true

const a2 = new Array();
a2.constructor === Array; // true

const n = 3;
n.constructor === Number; // true
```

Note that `constructor` usually comes from the constructor's `prototype` property. If you have a longer prototype chain, you can usually expect every object in the chain to have a `constructor` property.

```
const o = new TypeError(); // Inheritance: TypeError -> Error -> Object
const proto = Object.getPrototypeOf;

Object.hasOwn(o, "constructor"); // false
proto(o).constructor === TypeError; // true
proto(proto(o)).constructor === Error; // true
proto(proto(proto(o))).constructor === Object; // true
```

Examples

Displaying the constructor of an object

The following example creates a constructor (`Tree`) and an object of that type (`theTree`). The example then displays the `constructor` property for the object `theTree`.

```
function Tree(name) {
  this.name = name;
}

const theTree = new Tree("Redwood");
console.log(`theTree.constructor is ${theTree.constructor}`);
```

This example displays the following output:

```
theTree.constructor is function Tree(name) {
  this.name = name;
}
```

Assigning the constructor property to an object

One can assign the `constructor` property of non-primitives.

```
const arr = [];
arr.constructor = String;
arr.constructor === String; // true
arr instanceof String; // false
arr instanceof Array; // true

const foo = new Foo();
foo.constructor = "bar";
foo.constructor === "bar"; // true

// etc.
```

This does not overwrite the old `constructor` property — it was originally present on the instance's `[[Prototype]]`, not as its own property.

```
const arr = [];
Object.hasOwn(arr, "constructor"); // false
Object.hasOwn(Object.getPrototypeOf(arr), "constructor"); // true

arr.constructor = String;
Object.hasOwn(arr, "constructor"); // true — the instance property shadows the one on its prototype
```

But even when `Object.getPrototypeOf(a).constructor` is re-assigned, it won't change other behaviors of the object. For example, the behavior of `instanceof` is controlled by `Symbol.hasInstance`, not `constructor`:

```
const arr = [];
arr.constructor = String;
arr instanceof String; // false
arr instanceof Array; // true
```

There is nothing protecting the `constructor` property from being re-assigned or shadowed, so using it to detect the type of a variable should usually be avoided in favor of less fragile ways like `instanceof` and `Symbol.toStringTag` for objects, or `typeof` for primitives.

Changing the constructor of a constructor function's prototype

Every constructor has a `prototype` property, which will become the instance's `[[Prototype]]` when called via the `new` operator. `ConstructorFunction.prototype.constructor` will therefore become a property on the instance's `[[Prototype]]`, as previously demonstrated.

However, if `ConstructorFunction.prototype` is re-assigned, the `constructor` property will be lost. For example, the following is a common way to create an inheritance pattern:

```
function Parent() {
  // …
}
Parent.prototype.parentMethod = function () {};

function Child() {
  Parent.call(this); // Make sure everything is initialized properly
}
// Pointing the [[Prototype]] of Child.prototype to Parent.prototype
Child.prototype = Object.create(Parent.prototype);
```

The `constructor` of instances of `Child` will be `Parent` due to `Child.prototype` being re-assigned.

This is usually not a big deal — the language almost never reads the `constructor` property of an object. The only exception is when using `[Symbol.species]` to create new instances of a class, but such cases are rare, and you should be using the `extends` syntax to subclass builtins anyway.

However, ensuring that `Child.prototype.constructor` always points to `Child` itself is crucial when some caller is using `constructor` to access the original class from an instance. Take the following case: the object has the `create()` method to create itself.

```
function Parent() {
  // …
}
function CreatedConstructor() {
  Parent.call(this);
}

CreatedConstructor.prototype = Object.create(Parent.prototype);

CreatedConstructor.prototype.create = function () {
  return new this.constructor();
};

new CreatedConstructor().create().create(); // TypeError: new CreatedConstructor().create().create is undefined, since constructor === Parent
```

In the example above, an exception is thrown, since the `constructor` links to `Parent`. To avoid this, just assign the necessary constructor you are going to use.

```
function Parent() {
  // …
}
function CreatedConstructor() {
```

```
  // …
}

CreatedConstructor.prototype = Object.create(Parent.prototype, {
  // Return original constructor to Child
  constructor: {
    value: CreatedConstructor,
    enumerable: false, // Make it non-enumerable, so it won't appear in `for...in` loop
    writable: true,
    configurable: true,
  },
});

CreatedConstructor.prototype.create = function () {
  return new this.constructor();
};

new CreatedConstructor().create().create(); // it's pretty fine
```

Note that when manually adding the `constructor` property, it's crucial to make the property non-enumerable, so `constructor` won't be visited in `for...in` loops — as it normally isn't.

If the code above looks like too much boilerplate, you may also consider using `Object.setPrototypeOf()` to manipulate the prototype chain.

```
function Parent() {
  // …
}
function CreatedConstructor() {
  // …
}

Object.setPrototypeOf(CreatedConstructor.prototype, Parent.prototype);

CreatedConstructor.prototype.create = function () {
  return new this.constructor();
};

new CreatedConstructor().create().create(); // still works without re-creating constructor property
```

`Object.setPrototypeOf()` comes with its potential performance downsides because all previously created objects involved in the prototype chain have to be re-compiled; but if the above initialization code happens before `Parent` or `CreatedConstructor` are constructed, the effect should be minimal.

Let's consider one more involved case.

```
function ParentWithStatic() {}

ParentWithStatic.startPosition = { x: 0, y: 0 }; // Static member property
ParentWithStatic.getStartPosition = function () {
  return this.startPosition;
};

function Child(x, y) {
  this.position = { x, y };
}

Child.prototype = Object.create(ParentWithStatic.prototype, {
  // Return original constructor to Child
  constructor: {
    value: Child,
    enumerable: false,
    writable: true,
    configurable: true,
  },
});

Child.prototype.getOffsetByInitialPosition = function () {
  const position = this.position;
  // Using this.constructor, in hope that getStartPosition exists as a static method
  const startPosition = this.constructor.getStartPosition();

  return {
    offsetX: startPosition.x - position.x,
    offsetY: startPosition.y - position.y,
  };
};

new Child(1, 1).getOffsetByInitialPosition();
// Error: this.constructor.getStartPosition is undefined, since the
// constructor is Child, which doesn't have the getStartPosition static method
```

For this example to work properly, we can reassign the `Parent`'s static properties to `Child`:

```
// …
Object.assign(Child, ParentWithStatic); // Notice that we assign it before we create() a prototype below
Child.prototype = Object.create(ParentWithStatic.prototype, {
```

```
  // Return original constructor to Child
  constructor: {
    value: Child,
    enumerable: false,
    writable: true,
    configurable: true,
  },
});
// …
```

But even better, we can make the constructor functions themselves extend each other, as classes' `extends` do.

```
function ParentWithStatic() {}

ParentWithStatic.startPosition = { x: 0, y: 0 }; // Static member property
ParentWithStatic.getStartPosition = function () {
  return this.startPosition;
};

function Child(x, y) {
  this.position = { x, y };
}

// Properly create inheritance!
Object.setPrototypeOf(Child.prototype, ParentWithStatic.prototype);
Object.setPrototypeOf(Child, ParentWithStatic);

Child.prototype.getOffsetByInitialPosition = function () {
  const position = this.position;
  const startPosition = this.constructor.getStartPosition();

  return {
    offsetX: startPosition.x - position.x,
    offsetY: startPosition.y - position.y,
  };
};

console.log(new Child(1, 1).getOffsetByInitialPosition()); // { offsetX: -1, offsetY: -1 }
```

Again, using `Object.setPrototypeOf()` may have adverse performance effects, so make sure it happens immediately after the constructor declaration and before any instances are created — to avoid objects being "tainted".

> **Note:** Manually updating or setting the constructor can lead to different and sometimes confusing consequences. To prevent this, just define the role of `constructor` in each specific case. In most cases, `constructor` is not used and reassigning it is not necessary.

See also

- `class`
- `constructor`
- Constructor

Object.create()

The **`Object.create()`** static method creates a new object, using an existing object as the prototype of the newly created object.

Try it

```
const person = {
  isHuman: false,
  printIntroduction() {
    console.log(`My name is ${this.name}. Am I human? ${this.isHuman}`);
  },
};

const me = Object.create(person);

me.name = "Matthew"; // "name" is a property set on "me", but not on "person"
me.isHuman = true; // Inherited properties can be overwritten

me.printIntroduction();
// Expected output: "My name is Matthew. Am I human? true"
```

Syntax

```
Object.create(proto)
Object.create(proto, propertiesObject)
```

Parameters

`proto`

The object which should be the prototype of the newly-created object.

`propertiesObject` Optional

If specified and not `undefined`, an object whose enumerable own properties specify property descriptors to be added to the newly-created object, with the corresponding property names. These properties correspond to the second argument of `Object.defineProperties()`.

Return value

A new object with the specified prototype object and properties.

Exceptions

`TypeError`

Thrown if `proto` is neither `null` nor an `Object`.

Examples

Classical inheritance with Object.create()

Below is an example of how to use `Object.create()` to achieve classical inheritance. This is for a single inheritance, which is all that JavaScript supports.

```
// Shape - superclass
function Shape() {
  this.x = 0;
  this.y = 0;
}

// superclass method
Shape.prototype.move = function (x, y) {
  this.x += x;
  this.y += y;
  console.info("Shape moved.");
};

// Rectangle - subclass
function Rectangle() {
  Shape.call(this); // call super constructor.
}

// subclass extends superclass
Rectangle.prototype = Object.create(Shape.prototype, {
  // If you don't set Rectangle.prototype.constructor to Rectangle,
  // it will take the prototype.constructor of Shape (parent).
  // To avoid that, we set the prototype.constructor to Rectangle (child).
  constructor: {
    value: Rectangle,
    enumerable: false,
    writable: true,
    configurable: true,
  },
});

const rect = new Rectangle();

console.log("Is rect an instance of Rectangle?", rect instanceof Rectangle); // true
console.log("Is rect an instance of Shape?", rect instanceof Shape); // true
rect.move(1, 1); // Logs 'Shape moved.'
```

Note that there are caveats to watch out for using `create()`, such as re-adding the `constructor` property to ensure proper semantics. Although `Object.create()` is believed to have better performance than mutating the prototype with `Object.setPrototypeOf()`, the difference is in fact negligible if no instances have been created and property accesses haven't been optimized yet. In modern code, the class syntax should be preferred in any case.

Using propertiesObject argument with Object.create()

`Object.create()` allows fine-tuned control over the object creation process. The object initializer syntax is, in fact, a syntax sugar of `Object.create()`. With `Object.create()`, we can create objects with a designated prototype and also some properties. Note that the second parameter maps keys to *property descriptors* — this means you can control each property's enumerability, configurability, etc. as well, which you can't do in object initializers.

```
o = {};
// Is equivalent to:
o = Object.create(Object.prototype);

o = Object.create(Object.prototype, {
  // foo is a regular data property
  foo: {
    writable: true,
    configurable: true,
    value: "hello",
  },
  // bar is an accessor property
  bar: {
    configurable: false,
    get() {
      return 10;
```

```
    },
    set(value) {
      console.log("Setting `o.bar` to", value);
    },
  },
});

// Create a new object whose prototype is a new, empty
// object and add a single property 'p', with value 42.
o = Object.create({}, { p: { value: 42 } });
```

With `Object.create()`, we can create an object with `null` as prototype. The equivalent syntax in object initializers would be the `__proto__` key.

```
o = Object.create(null);
// Is equivalent to:
o = { __proto__: null };
```

By default properties are *not* writable, enumerable or configurable.

```
o.p = 24; // throws in strict mode
o.p; // 42

o.q = 12;
for (const prop in o) {
  console.log(prop);
}
// 'q'

delete o.p;
// false; throws in strict mode
```

To specify a property with the same attributes as in an initializer, explicitly specify `writable`, `enumerable` and `configurable`.

```
o2 = Object.create(
  {},
  {
    p: {
      value: 42,
      writable: true,
      enumerable: true,
      configurable: true,
    },
  },
);
// This is not equivalent to:
// o2 = Object.create({ p: 42 })
// which will create an object with prototype { p: 42 }
```

You can use `Object.create()` to mimic the behavior of the `new` operator.

```
function Constructor() {}
o = new Constructor();
// Is equivalent to:
o = Object.create(Constructor.prototype);
```

Of course, if there is actual initialization code in the `Constructor` function, the `Object.create()` method cannot reflect it.

See also

- Polyfill of `Object.create` in `core-js`
- `Object.defineProperty()`
- `Object.defineProperties()`
- `Object.prototype.isPrototypeOf()`
- `Reflect.construct()`
- Object.getPrototypeOf by John Resig (2008)

Object.defineProperties()

The **`Object.defineProperties()`** static method defines new or modifies existing properties directly on an object, returning the object.

Try it

```
const object = {};

Object.defineProperties(object, {
  property1: {
    value: 42,
    writable: true,
  },
  property2: {},
});
```

```
console.log(object.property1);
// Expected output: 42
```

Syntax

```
Object.defineProperties(obj, props)
```

Parameters

`obj`

The object on which to define or modify properties.

`props`

An object whose keys represent the names of properties to be defined or modified and whose values are objects describing those properties. Each value in `props` must be either a data descriptor or an accessor descriptor; it cannot be both (see `Object.defineProperty()` for more details).

Data descriptors and accessor descriptors may optionally contain the following keys:

`configurable`

`true` if and only if the type of this property descriptor may be changed and if the property may be deleted from the corresponding object. **Defaults to `false`.**

`enumerable`

`true` if and only if this property shows up during enumeration of the properties on the corresponding object. **Defaults to `false`.**

A data descriptor also has the following optional keys:

`value`

The value associated with the property. Can be any valid JavaScript value (number, object, function, etc.). **Defaults to `undefined`.**

`writable`

`true` if and only if the value associated with the property may be changed with an assignment operator. **Defaults to `false`.**

An accessor descriptor also has the following optional keys:

`get`

A function which serves as a getter for the property, or `undefined` if there is no getter. The function's return value will be used as the value of the property. **Defaults to `undefined`.**

`set`

A function which serves as a setter for the property, or `undefined` if there is no setter. The function will receive as its only argument the new value being assigned to the property. **Defaults to `undefined`.**

If a descriptor has neither of `value`, `writable`, `get` and `set` keys, it is treated as a data descriptor. If a descriptor has both `value` or `writable` and `get` or `set` keys, an exception is thrown.

Return value

The object that was passed to the function.

Examples

Using Object.defineProperties

```
const obj = {};
Object.defineProperties(obj, {
  property1: {
    value: true,
    writable: true,
  },
  property2: {
    value: "Hello",
    writable: false,
  },
  // etc. etc.
});
```

See also

- Polyfill of `Object.defineProperties` in `core-js`
- es-shims polyfill of `Object.defineProperties`
- `Object.defineProperty()`
- `Object.keys()`
- Enumerability and ownership of properties

Object.defineProperty()

The **`Object.defineProperty()`** static method defines a new property directly on an object, or modifies an existing property on an object, and returns the object.

Try it

```
const object = {};

Object.defineProperty(object, "foo", {
  value: 42,
  writable: false,
});

object.foo = 77;
// Throws an error in strict mode

console.log(object.foo);
// Expected output: 42
```

Syntax

```
Object.defineProperty(obj, prop, descriptor)
```

Parameters

`obj`

The object on which to define the property.

`prop`

A string or `Symbol` specifying the key of the property to be defined or modified.

`descriptor`

The descriptor for the property being defined or modified.

Return value

The object that was passed to the function, with the specified property added or modified.

Description

`Object.defineProperty()` allows a precise addition to or modification of a property on an object. Normal property addition through assignment creates properties which show up during property enumeration (`for...in`, `Object.keys()`, etc.), whose values may be changed and which may be deleted. This method allows these extra details to be changed from their defaults. By default, properties added using `Object.defineProperty()` are not writable, not enumerable, and not configurable. In addition, `Object.defineProperty()` uses the `[[DefineOwnProperty]]` internal method, instead of `[[Set]]`, so it does not invoke setters, even when the property is already present.

Property descriptors present in objects come in two main flavors: data descriptors and accessor descriptors. A **data descriptor** is a property with a value that may or may not be writable. An **accessor descriptor** is a property described by a getter-setter pair of functions. A descriptor must be one of these two flavors; it cannot be both.

Both data and accessor descriptors are objects. They share the following optional keys (please note: the **defaults** mentioned here are in the case of defining properties using `Object.defineProperty()`):

`configurable`

when this is set to `false`,

- the type of this property cannot be changed between data property and accessor property, and
- the property may not be deleted, and
- other attributes of its descriptor cannot be changed (however, if it's a data descriptor with `writable: true`, the `value` can be changed, and `writable` can be changed to `false`).

Defaults to `false`.

`enumerable`

`true` if and only if this property shows up during enumeration of the properties on the corresponding object. **Defaults to `false`.**

A **data descriptor** also has the following optional keys:

`value`

The value associated with the property. Can be any valid JavaScript value (number, object, function, etc.). **Defaults to `undefined`.**

`writable`

`true` if the value associated with the property may be changed with an assignment operator. **Defaults to `false`.**

An **accessor descriptor** also has the following optional keys:

`get`

A function which serves as a getter for the property, or `undefined` if there is no getter. When the property is accessed, this function is called without arguments and with `this` set to the object through which the property is accessed (this may not be the object on which the property is defined due to inheritance). The return value will be used as the value of the property. **Defaults to `undefined`.**

`set`

A function which serves as a setter for the property, or `undefined` if there is no setter. When the property is assigned, this function is called with one argument (the value being assigned to the property) and with `this` set to the object through which the property is assigned. **Defaults to `undefined`.**

If a descriptor doesn't have any of the `value`, `writable`, `get`, and `set` keys, it is treated as a data descriptor. If a descriptor is both a data descriptor (because it has `value` or `writable`) and an accessor descriptor (because it has `get` or `set`), an exception is thrown.

These attributes are not necessarily the descriptor's own properties. Inherited properties will be considered as well. In order to ensure these defaults are preserved, you might freeze existing objects in the descriptor object's prototype chain upfront, specify all options explicitly, or create a `null`-prototype object.

```
const obj = {};
// 1. Using a null prototype: no inherited properties
const descriptor = Object.create(null);
descriptor.value = "static";

// not enumerable, not configurable, not writable as defaults
Object.defineProperty(obj, "key", descriptor);

// 2. Being explicit by using a throw-away object literal with all attributes present
Object.defineProperty(obj, "key2", {
  enumerable: false,
  configurable: false,
  writable: false,
  value: "static",
});

// 3. Prevents adding or removing the object prototype properties
// (value, get, set, enumerable, writable, configurable)
Object.freeze(Object.prototype);
```

When the property already exists, `Object.defineProperty()` attempts to modify the property according to the values in the descriptor and the property's current configuration.

If the old descriptor had its `configurable` attribute set to `false`, the property is said to be *non-configurable*. It is not possible to change any attribute of a non-configurable accessor property, and it is not possible to switch between data and accessor property types. For data properties with `writable: true`, it is possible to modify the value and change the `writable` attribute from `true` to `false`. A `TypeError` is thrown when attempts are made to change non-configurable property attributes (except `value` and `writable`, if permitted), except when defining a value same as the original value on a data property.

When the current property is configurable, defining an attribute to `undefined` effectively deletes it. For example, if `o.k` is an accessor property, `Object.defineProperty(o, "k", { set: undefined })` will remove the setter, making `k` only have a getter and become readonly. If an attribute is absent from the new descriptor, the old descriptor attribute's value is kept (it won't be implicitly re-defined to `undefined`). It is possible to toggle between data and accessor property by giving a descriptor of a different "flavor". For example, if the new descriptor is a data descriptor (with `value` or `writable`), the original descriptor's `get` and `set` attributes will both be dropped.

Examples

Creating a property

When the property specified doesn't exist in the object, `Object.defineProperty()` creates a new property as described. Fields may be omitted from the descriptor and default values for those fields are inputted.

```
const o = {}; // Creates a new object

// Example of an object property added
// with defineProperty with a data property descriptor
Object.defineProperty(o, "a", {
  value: 37,
  writable: true,
  enumerable: true,
  configurable: true,
});
// 'a' property exists in the o object and its value is 37

// Example of an object property added
// with defineProperty with an accessor property descriptor
let bValue = 38;
Object.defineProperty(o, "b", {
  get() {
    return bValue;
  },
  set(newValue) {
    bValue = newValue;
  },
  enumerable: true,
  configurable: true,
});
o.b; // 38
// 'b' property exists in the o object and its value is 38
```

```
// The value of o.b is now always identical to bValue,
// unless o.b is redefined

// You cannot try to mix both:
Object.defineProperty(o, "conflict", {
  value: 0x9f91102,
  get() {
    return 0xdeadbeef;
  },
});
// throws a TypeError: value appears
// only in data descriptors,
// get appears only in accessor descriptors
```

Modifying a property

When modifying an existing property, the current property configuration determines if the operator succeeds, does nothing, or throws a `TypeError`.

Writable attribute

When the `writable` property attribute is `false`, the property is said to be "non-writable". It cannot be reassigned. Trying to write to a non-writable property doesn't change it and results in an error in strict mode.

```
const o = {}; // Creates a new object

Object.defineProperty(o, "a", {
  value: 37,
  writable: false,
});

console.log(o.a); // 37
o.a = 25; // No error thrown
// (it would throw in strict mode,
// even if the value had been the same)
console.log(o.a); // 37; the assignment didn't work

// strict mode
(() => {
  "use strict";
  const o = {};
  Object.defineProperty(o, "b", {
    value: 2,
    writable: false,
  });
  o.b = 3; // throws TypeError: "b" is read-only
  return o.b; // returns 2 without the line above
})();
```

Enumerable attribute

The `enumerable` property attribute defines whether the property is considered by `Object.assign()` or the spread operator. For non-`Symbol` properties, it also defines whether it shows up in a `for...in` loop and `Object.keys()` or not. For more information, see Enumerability and ownership of properties.

```
const o = {};
Object.defineProperty(o, "a", {
  value: 1,
  enumerable: true,
});
Object.defineProperty(o, "b", {
  value: 2,
  enumerable: false,
});
Object.defineProperty(o, "c", {
  value: 3,
}); // enumerable defaults to false
o.d = 4; // enumerable defaults to true when creating a property by setting it
Object.defineProperty(o, Symbol.for("e"), {
  value: 5,
  enumerable: true,
});
Object.defineProperty(o, Symbol.for("f"), {
  value: 6,
  enumerable: false,
});

for (const i in o) {
  console.log(i);
}
// Logs 'a' and 'd' (always in that order)

Object.keys(o); // ['a', 'd']

o.propertyIsEnumerable("a"); // true
o.propertyIsEnumerable("b"); // false
o.propertyIsEnumerable("c"); // false
```

```
o.propertyIsEnumerable("d"); // true
o.propertyIsEnumerable(Symbol.for("e")); // true
o.propertyIsEnumerable(Symbol.for("f")); // false

const p = { ...o };
p.a; // 1
p.b; // undefined
p.c; // undefined
p.d; // 4
p[Symbol.for("e")]; // 5
p[Symbol.for("f")]; // undefined
```

Configurable attribute

The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `value` and `writable`) can be changed.

This example illustrates a non-configurable accessor property.

```
const o = {};
Object.defineProperty(o, "a", {
  get() {
    return 1;
  },
  configurable: false,
});

Object.defineProperty(o, "a", {
  configurable: true,
}); // throws a TypeError
Object.defineProperty(o, "a", {
  enumerable: true,
}); // throws a TypeError
Object.defineProperty(o, "a", {
  set() {},
}); // throws a TypeError (set was undefined previously)
Object.defineProperty(o, "a", {
  get() {
    return 1;
  },
}); // throws a TypeError
// (even though the new get does exactly the same thing)
Object.defineProperty(o, "a", {
  value: 12,
}); // throws a TypeError
// ('value' can be changed when 'configurable' is false, but only when the property is a writable data property)

console.log(o.a); // 1
delete o.a; // Nothing happens; throws an error in strict mode
console.log(o.a); // 1
```

If the `configurable` attribute of `o.a` had been `true`, none of the errors would be thrown and the property would be deleted at the end.

This example illustrates a non-configurable but writable data property. The property's `value` can still be changed, and `writable` can still be toggled from `true` to `false`.

```
const o = {};
Object.defineProperty(o, "b", {
  writable: true,
  configurable: false,
});
console.log(o.b); // undefined
Object.defineProperty(o, "b", {
  value: 1,
}); // Even when configurable is false, because the object is writable, we may still replace the value
console.log(o.b); // 1
o.b = 2; // We can change the value with assignment operators as well
console.log(o.b); // 2
// Toggle the property's writability
Object.defineProperty(o, "b", {
  writable: false,
});
Object.defineProperty(o, "b", {
  value: 1,
}); // TypeError: because the property is neither writable nor configurable, it cannot be modified
// At this point, there's no way to further modify 'b'
// or restore its writability
```

This example illustrates a configurable but non-writable data property. The property's `value` may still be replaced with `defineProperty` (but not with assignment operators), and `writable` may be toggled.

```
const o = {};
Object.defineProperty(o, "b", {
  writable: false,
  configurable: true,
});
Object.defineProperty(o, "b", {
```

```
  value: 1,
}); // We can replace the value with defineProperty
console.log(o.b); // 1
o.b = 2; // throws TypeError in strict mode: cannot change a non-writable property's value with assignment
```

This example illustrates a non-configurable and non-writable data property. There's no way to update any attribute of the property, including its `value`.

```
const o = {};
Object.defineProperty(o, "b", {
  writable: false,
  configurable: false,
});
Object.defineProperty(o, "b", {
  value: 1,
}); // TypeError: the property cannot be modified because it is neither writable nor configurable.
```

Adding properties and default values

It is important to consider the way default values of attributes are applied. There is often a difference between using property accessors to assign a value and using `Object.defineProperty()`, as shown in the example below.

```
const o = {};

o.a = 1;
// is equivalent to:
Object.defineProperty(o, "a", {
  value: 1,
  writable: true,
  configurable: true,
  enumerable: true,
});

// On the other hand,
Object.defineProperty(o, "a", { value: 1 });
// is equivalent to:
Object.defineProperty(o, "a", {
  value: 1,
  writable: false,
  configurable: false,
  enumerable: false,
});
```

Custom setters and getters

The example below shows how to implement a self-archiving object. When `temperature` property is set, the `archive` array gets a log entry.

```
function Archiver() {
  let temperature = null;
  const archive = [];

  Object.defineProperty(this, "temperature", {
    get() {
      console.log("get!");
      return temperature;
    },
    set(value) {
      temperature = value;
      archive.push({ val: temperature });
    },
  });

  this.getArchive = () => archive;
}

const arc = new Archiver();
arc.temperature; // 'get!'
arc.temperature = 11;
arc.temperature = 13;
arc.getArchive(); // [{ val: 11 }, { val: 13 }]
```

In this example, a getter always returns the same value.

```
const pattern = {
  get() {
    return "I always return this string, whatever you have assigned";
  },
  set() {
    this.myName = "this is my name string";
  },
};

function TestDefineSetAndGet() {
  Object.defineProperty(this, "myProperty", pattern);
}
```

```
const instance = new TestDefineSetAndGet();
instance.myProperty = "test";
console.log(instance.myProperty);
// I always return this string, whatever you have assigned

console.log(instance.myName); // this is my name string
```

Inheritance of properties

If an accessor property is inherited, its `get` and `set` methods will be called when the property is accessed and modified on descendant objects. If these methods use a variable to store the value, this value will be shared by all objects.

```
function MyClass() {}

let value;
Object.defineProperty(MyClass.prototype, "x", {
  get() {
    return value;
  },
  set(x) {
    value = x;
  },
});

const a = new MyClass();
const b = new MyClass();
a.x = 1;
console.log(b.x); // 1
```

This can be fixed by storing the value in another property. In `get` and `set` methods, `this` points to the object which is used to access or modify the property.

```
function MyClass() {}

Object.defineProperty(MyClass.prototype, "x", {
  get() {
    return this.storedX;
  },
  set(x) {
    this.storedX = x;
  },
});

const a = new MyClass();
const b = new MyClass();
a.x = 1;
console.log(b.x); // undefined
```

Unlike accessor properties, data properties are always set on the object itself, not on a prototype. However, if a non-writable data property is inherited, it is still prevented from being modified on the object.

```
function MyClass() {}

MyClass.prototype.x = 1;
Object.defineProperty(MyClass.prototype, "y", {
  writable: false,
  value: 1,
});

const a = new MyClass();
a.x = 2;
console.log(a.x); // 2
console.log(MyClass.prototype.x); // 1
a.y = 2; // Ignored, throws in strict mode
console.log(a.y); // 1
console.log(MyClass.prototype.y); // 1
```

See also

- Enumerability and ownership of properties
- `Object.defineProperties()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.getOwnPropertyDescriptor()`
- `get`
- `set`
- `Object.create()`
- `Reflect.defineProperty()`

Object.entries()

The **`Object.entries()`** static method returns an array of a given object's own enumerable string-keyed property key-value pairs.

Try it

```
const object = {
  a: "some string",
  b: 42,
};

for (const [key, value] of Object.entries(object)) {
  console.log(`${key}: ${value}`);
}

// Expected output:
// "a: some string"
// "b: 42"
```

Syntax

```
Object.entries(obj)
```

Parameters

`obj`

An object.

Return value

An array of the given object's own enumerable string-keyed property key-value pairs. Each key-value pair is an array with two elements: the first element is the property key (which is always a string), and the second element is the property value.

Description

`Object.entries()` returns an array whose elements are arrays corresponding to the enumerable string-keyed property key-value pairs found directly upon `object`. This is the same as iterating with a `for...in` loop, except that a `for...in` loop enumerates properties in the prototype chain as well. The order of the array returned by `Object.entries()` is the same as that provided by a `for...in` loop.

If you only need the property keys, use `Object.keys()` instead. If you only need the property values, use `Object.values()` instead.

Examples

Using Object.entries()

```
const obj = { foo: "bar", baz: 42 };
console.log(Object.entries(obj)); // [ ['foo', 'bar'], ['baz', 42] ]

const arrayLike = { 0: "a", 1: "b", 2: "c" };
console.log(Object.entries(arrayLike)); // [ ['0', 'a'], ['1', 'b'], ['2', 'c'] ]

const randomKeyOrder = { 100: "a", 2: "b", 7: "c" };
console.log(Object.entries(randomKeyOrder)); // [ ['2', 'b'], ['7', 'c'], ['100', 'a'] ]

// getFoo is a non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
    },
  },
);
myObj.foo = "bar";
console.log(Object.entries(myObj)); // [ ['foo', 'bar'] ]
```

Using Object.entries() on primitives

Non-object arguments are coerced to objects. `undefined` and `null` cannot be coerced to objects and throw a `TypeError` upfront. Only strings may have own enumerable properties, while all other primitives return an empty array.

```
// Strings have indices as enumerable own properties
console.log(Object.entries("foo")); // [ ['0', 'f'], ['1', 'o'], ['2', 'o'] ]

// Other primitives except undefined and null have no own properties
console.log(Object.entries(100)); // []
```

Converting an Object to a Map

The `Map()` constructor accepts an iterable of `entries`. With `Object.entries`, you can easily convert from `Object` to `Map`:

```
const obj = { foo: "bar", baz: 42 };
const map = new Map(Object.entries(obj));
console.log(map); // Map(2) {"foo" => "bar", "baz" => 42}
```

Iterating through an Object

Using array destructuring, you can iterate through objects easily.

```
// Using for...of loop
const obj = { a: 5, b: 7, c: 9 };
for (const [key, value] of Object.entries(obj)) {
  console.log(`${key} ${value}`); // "a 5", "b 7", "c 9"
}

// Using array methods
Object.entries(obj).forEach(([key, value]) => {
  console.log(`${key} ${value}`); // "a 5", "b 7", "c 9"
});
```

See also

- Polyfill of `Object.entries` in `core-js`
- es-shims polyfill of `Object.entries`
- Enumerability and ownership of properties
- `Object.keys()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.fromEntries()`
- `Object.getOwnPropertyNames()`
- `Map.prototype.entries()`

Object.freeze()

The **`Object.freeze()`** static method *freezes* an object. Freezing an object prevents extensions and makes existing properties non-writable and non-configurable. A frozen object can no longer be changed: new properties cannot be added, existing properties cannot be removed, their enumerability, configurability, writability, or value cannot be changed, and the object's prototype cannot be re-assigned. `freeze()` returns the same object that was passed in.

Freezing an object is the highest integrity level that JavaScript provides.

Try it

```
const obj = {
  prop: 42,
};

Object.freeze(obj);

obj.prop = 33;
// Throws an error in strict mode

console.log(obj.prop);
// Expected output: 42
```

Syntax

```
Object.freeze(obj)
```

Parameters

`obj`

The object to freeze.

Return value

The object that was passed to the function.

Description

Freezing an object is equivalent to preventing extensions and then changing all existing properties' descriptors' `configurable` to `false` — and for data properties, `writable` to `false` as well. Nothing can be added to or removed from the properties set of a frozen object. Any attempt to do so will fail, either silently or by throwing a `TypeError` exception (most commonly, but not exclusively, when in strict mode).

For data properties of a frozen object, their values cannot be changed since the `writable` and `configurable` attributes are set to `false`. Accessor properties (getters and setters) work the same — the property value returned by the getter may still change, and the setter can still be called without throwing errors when setting the property. Note that values that are objects can still be modified, unless they are also frozen. As an object, an array can be frozen; after doing so, its elements cannot be altered and no elements can be added to or removed from the array.

Private elements are not properties and do not have the concept of property descriptors. Freezing an object with private elements does not prevent the values of these private elements from being changed. (Freezing objects is usually meant as a security measure against external code, but external code cannot access private elements anyway.) Private elements cannot be added or removed from the object, whether the object is frozen or not.

`freeze()` returns the same object that was passed into the function. It *does not* create a frozen copy.

A `TypedArray` or a `DataView` with elements will cause a `TypeError`, as they are views over memory and will definitely cause other possible issues:

```
Object.freeze(new Uint8Array(0)); // No elements
// Uint8Array []

Object.freeze(new Uint8Array(1)); // Has elements
// TypeError: Cannot freeze array buffer views with elements

Object.freeze(new DataView(new ArrayBuffer(32))); // No elements
// DataView {}

Object.freeze(new Float64Array(new ArrayBuffer(64), 32, 0)); // No elements
// Float64Array []

Object.freeze(new Float64Array(new ArrayBuffer(64), 32, 2)); // Has elements
// TypeError: Cannot freeze array buffer views with elements
```

Note that as the standard three properties (`buf.byteLength`, `buf.byteOffset` and `buf.buffer`) are read-only (as are those of an `ArrayBuffer` or `SharedArrayBuffer`), there is no reason for attempting to freeze these properties.

Unlike `Object.seal()`, existing properties in objects frozen with `Object.freeze()` are made immutable and data properties cannot be re-assigned.

Examples

Freezing objects

```
const obj = {
  prop() {},
  foo: "bar",
};

// Before freezing: new properties may be added,
// and existing properties may be changed or removed
obj.foo = "baz";
obj.lumpy = "woof";
delete obj.prop;

// Freeze.
const o = Object.freeze(obj);

// The return value is just the same object we passed in.
o === obj; // true

// The object has become frozen.
Object.isFrozen(obj); // === true

// Now any changes will fail
obj.foo = "quux"; // silently does nothing
// silently doesn't add the property
obj.quaxxor = "the friendly duck";

// In strict mode such attempts will throw TypeErrors
function fail() {
  "use strict";
  obj.foo = "sparky"; // throws a TypeError
  delete obj.foo; // throws a TypeError
  delete obj.quaxxor; // returns true since attribute 'quaxxor' was never added
  obj.sparky = "arf"; // throws a TypeError
}

fail();

// Attempted changes through Object.defineProperty;
// both statements below throw a TypeError.
Object.defineProperty(obj, "ohai", { value: 17 });
Object.defineProperty(obj, "foo", { value: "eit" });

// It's also impossible to change the prototype
// both statements below will throw a TypeError.
Object.setPrototypeOf(obj, { x: 20 });
obj.__proto__ = { x: 20 };
```

Freezing arrays

```
const a = [0];
Object.freeze(a); // The array cannot be modified now.

a[0] = 1; // fails silently

// In strict mode such attempt will throw a TypeError
function fail() {
  "use strict";
  a[0] = 1;
}
```

```
fail();

// Attempted to push
a.push(2); // throws a TypeError
```

The object being frozen is *immutable*. However, it is not necessarily *constant*. The following example shows that a frozen object is not constant (freeze is shallow).

```
const obj1 = {
  internal: {},
};

Object.freeze(obj1);
obj1.internal.a = "value";

obj1.internal.a; // 'value'
```

To be a constant object, the entire reference graph (direct and indirect references to other objects) must reference only immutable frozen objects. The object being frozen is said to be immutable because the entire object *state* (values and references to other objects) within the whole object is fixed. Note that strings, numbers, and booleans are always immutable and that Functions and Arrays are objects.

Deep freezing

The result of calling `Object.freeze(object)` only applies to the immediate properties of `object` itself and will prevent future property addition, removal or value re-assignment operations *only* on `object`. If the value of those properties are objects themselves, those objects are not frozen and may be the target of property addition, removal or value re-assignment operations.

```
const employee = {
  name: "Mayank",
  designation: "Developer",
  address: {
    street: "Rohini",
    city: "Delhi",
  },
};

Object.freeze(employee);

employee.name = "Dummy"; // fails silently in non-strict mode
employee.address.city = "Noida"; // attributes of child object can be modified

console.log(employee.address.city); // "Noida"
```

To make an object immutable, recursively freeze each non-primitive property (deep freeze). Use the pattern on a case-by-case basis based on your design when you know the object contains no cycles in the reference graph, otherwise an endless loop will be triggered. For example, functions created with the `function` syntax have a `prototype` property with a `constructor` property that points to the function itself, so they have cycles by default. Other functions, such as arrow functions, can still be frozen.

An enhancement to `deepFreeze()` would be to store the objects it has already visited, so you can suppress calling `deepFreeze()` recursively when an object is in the process of being made immutable. For one example, see using `WeakSet` to detect circular references. You still run a risk of freezing an object that shouldn't be frozen, such as `window`.

```
function deepFreeze(object) {
  // Retrieve the property names defined on object
  const propNames = Reflect.ownKeys(object);

  // Freeze properties before freezing self
  for (const name of propNames) {
    const value = object[name];

    if ((value && typeof value === "object") || typeof value === "function") {
      deepFreeze(value);
    }
  }

  return Object.freeze(object);
}

const obj2 = {
  internal: {
    a: null,
  },
};

deepFreeze(obj2);

obj2.internal.a = "anotherValue"; // fails silently in non-strict mode
obj2.internal.a; // null
```

See also

- `Object.isFrozen()`
- `Object.preventExtensions()`

- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`

Object.fromEntries()

The **`Object.fromEntries()`** static method transforms a list of key-value pairs into an object.

Try it

```
const entries = new Map([
  ["foo", "bar"],
  ["baz", 42],
]);

const obj = Object.fromEntries(entries);

console.log(obj);
// Expected output: Object { foo: "bar", baz: 42 }
```

Syntax

```
Object.fromEntries(iterable)
```

Parameters

`iterable`

An iterable, such as an `Array` or `Map`, containing a list of objects. Each object should have two properties:

`0`

A string or symbol representing the property key.

`1`

The property value.

Typically, this object is implemented as a two-element array, with the first element being the property key and the second element being the property value.

Return value

A new object whose properties are given by the entries of the iterable.

Description

The `Object.fromEntries()` method takes a list of key-value pairs and returns a new object whose properties are given by those entries. The `iterable` argument is expected to be an object that implements a `[Symbol.iterator]()` method. The method returns an iterator object that produces two-element array-like objects. The first element is a value that will be used as a property key, and the second element is the value to associate with that property key.

`Object.fromEntries()` performs the reverse of `Object.entries()`, except that `Object.entries()` only returns string-keyed properties, while `Object.fromEntries()` can also create symbol-keyed properties.

> **Note:** Unlike `Array.from()`, `Object.fromEntries()` does not use the value of `this`, so calling it on another constructor does not create objects of that type.

Examples

Converting a Map to an Object

With `Object.fromEntries`, you can convert from `Map` to `Object`:

```
const map = new Map([
  ["foo", "bar"],
  ["baz", 42],
]);
const obj = Object.fromEntries(map);
console.log(obj); // { foo: "bar", baz: 42 }
```

Converting an Array to an Object

With `Object.fromEntries`, you can convert from `Array` to `Object`:

```
const arr = [
  ["0", "a"],
  ["1", "b"],
  ["2", "c"],
];
const obj = Object.fromEntries(arr);
console.log(obj); // { 0: "a", 1: "b", 2: "c" }
```

Object transformations

With `Object.fromEntries`, its reverse method `Object.entries()`, and array manipulation methods, you are able to transform objects like this:

```
const object1 = { a: 1, b: 2, c: 3 };

const object2 = Object.fromEntries(
  Object.entries(object1).map(([key, val]) => [key, val * 2]),
);

console.log(object2);
// { a: 2, b: 4, c: 6 }
```

See also

- Polyfill of `Object.fromEntries` in `core-js`
- es-shims polyfill of `Object.fromEntries`
- `Object.entries()`
- `Object.keys()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Map.prototype.entries()`
- `Map.prototype.keys()`
- `Map.prototype.values()`

Object.getOwnPropertyDescriptor()

The **`Object.getOwnPropertyDescriptor()`** static method returns an object describing the configuration of a specific property on a given object (that is, one directly present on an object and not in the object's prototype chain). The object returned is mutable but mutating it has no effect on the original property's configuration.

Try it

```
const object = {
  foo: 42,
};

const descriptor = Object.getOwnPropertyDescriptor(object, "foo");

console.log(descriptor.configurable);
// Expected output: true

console.log(descriptor.value);
// Expected output: 42
```

Syntax

```
Object.getOwnPropertyDescriptor(obj, prop)
```

Parameters

`obj`

The object in which to look for the property.

`prop`

The name or `Symbol` of the property whose description is to be retrieved.

Return value

A property descriptor of the given property if it exists on the object, `undefined` otherwise.

Description

This method permits examination of the precise description of a property. A *property* in JavaScript consists of either a string-valued name or a `Symbol` and a property descriptor. Further information about property descriptor types and their attributes can be found in `Object.defineProperty()`.

A *property descriptor* is a record with some of the following attributes:

`value`

The value associated with the property (data descriptors only).

`writable`

`true` if and only if the value associated with the property may be changed (data descriptors only).

`get`

A function which serves as a getter for the property, or `undefined` if there is no getter (accessor descriptors only).

`set`

A function which serves as a setter for the property, or `undefined` if there is no setter (accessor descriptors only).

`configurable`

`true` if and only if the type of this property descriptor may be changed and if the property may be deleted from the corresponding object.

`enumerable`

`true` if and only if this property shows up during enumeration of the properties on the corresponding object.

Examples

Using Object.getOwnPropertyDescriptor()

```
let o, d;

o = {
  get foo() {
    return 17;
  },
};
d = Object.getOwnPropertyDescriptor(o, "foo");
console.log(d);
// {
//   configurable: true,
//   enumerable: true,
//   get: [Function: get foo],
//   set: undefined
// }

o = { bar: 42 };
d = Object.getOwnPropertyDescriptor(o, "bar");
console.log(d);
// {
//   configurable: true,
//   enumerable: true,
//   value: 42,
//   writable: true
// }

o = { [Symbol.for("baz")]: 73 };
d = Object.getOwnPropertyDescriptor(o, Symbol.for("baz"));
console.log(d);
// {
//   configurable: true,
//   enumerable: true,
//   value: 73,
//   writable: true
// }

o = {};
Object.defineProperty(o, "qux", {
  value: 8675309,
  writable: false,
  enumerable: false,
});
d = Object.getOwnPropertyDescriptor(o, "qux");
console.log(d);
// {
//   value: 8675309,
//   writable: false,
//   enumerable: false,
//   configurable: false
// }
```

Non-object coercion

In ES5, if the first argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object first argument will be coerced to an object at first.

```
Object.getOwnPropertyDescriptor("foo", 0);
// TypeError: "foo" is not an object  // ES5 code

Object.getOwnPropertyDescriptor("foo", 0);
// Object returned by ES2015 code: {
//   configurable: false,
//   enumerable: true,
//   value: "f",
//   writable: false
// }
```

See also

- `Object.defineProperty()`
- `Reflect.getOwnPropertyDescriptor()`

Object.getOwnPropertyDescriptors()

The **`Object.getOwnPropertyDescriptors()`** static method returns all own property descriptors of a given object.

Try it

```
const object = {
  foo: 42,
};

const descriptors = Object.getOwnPropertyDescriptors(object);

console.log(descriptors.foo.writable);
// Expected output: true

console.log(descriptors.foo.value);
// Expected output: 42
```

Syntax

```
Object.getOwnPropertyDescriptors(obj)
```

Parameters

`obj`

The object for which to get all own property descriptors.

Return value

An object containing all own property descriptors of an object. Might be an empty object, if there are no properties.

Description

This method permits examination of the precise description of all own properties of an object. A *property* in JavaScript consists of either a string-valued name or a `Symbol` and a property descriptor. Further information about property descriptor types and their attributes can be found in `Object.defineProperty()`.

A *property descriptor* is a record with some of the following attributes:

`value`

The value associated with the property (data descriptors only).

`writable`

`true` if and only if the value associated with the property may be changed (data descriptors only).

`get`

A function which serves as a getter for the property, or `undefined` if there is no getter (accessor descriptors only).

`set`

A function which serves as a setter for the property, or `undefined` if there is no setter (accessor descriptors only).

`configurable`

`true` if and only if the type of this property descriptor may be changed and if the property may be deleted from the corresponding object.

`enumerable`

`true` if and only if this property shows up during enumeration of the properties on the corresponding object.

Examples

Creating a shallow copy

Whereas the `Object.assign()` method will only copy enumerable and own properties from a source object to a target object, you are able to use this method and `Object.create()` for a shallow copy between two unknown objects:

```
Object.create(
  Object.getPrototypeOf(obj),
  Object.getOwnPropertyDescriptors(obj),
);
```

Creating a subclass

A typical way of creating a subclass is to define the subclass, set its prototype to an instance of the superclass, and then define properties on that instance. This can get awkward especially for getters and setters. Instead, you can use this code to set the prototype:

```
function superclass() {}
superclass.prototype = {
  // Define the superclass constructor, methods, and properties here
};
function subclass() {}
subclass.prototype = Object.create(superclass.prototype, {
```

```
  // Define the subclass constructor, methods, and properties here
});
```

See also

- Polyfill of `Object.getOwnPropertyDescriptors` in `core-js`
- es-shims polyfill of `Object.getOwnPropertyDescriptors`
- `Object.getOwnPropertyDescriptor()`
- `Object.defineProperty()`

Object.getOwnPropertyNames()

The **`Object.getOwnPropertyNames()`** static method returns an array of all properties (including non-enumerable properties except for those which use Symbol) found directly in a given object.

Try it

```
const object = {
  a: 1,
  b: 2,
  c: 3,
};

console.log(Object.getOwnPropertyNames(object));
// Expected output: Array ["a", "b", "c"]
```

Syntax

```
Object.getOwnPropertyNames(obj)
```

Parameters

`obj`

The object whose enumerable and non-enumerable properties are to be returned.

Return value

An array of strings that corresponds to the properties found directly in the given object.

Description

`Object.getOwnPropertyNames()` returns an array whose elements are strings corresponding to the enumerable and non-enumerable properties found directly in a given object `obj`. The ordering of the enumerable properties in the array is consistent with the ordering exposed by a `for...in` loop (or by `Object.keys()`) over the properties of the object. The non-negative integer keys of the object (both enumerable and non-enumerable) are added in ascending order to the array first, followed by the string keys in the order of insertion.

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object argument will be coerced to an object.

```
Object.getOwnPropertyNames("foo");
// TypeError: "foo" is not an object (ES5 code)

Object.getOwnPropertyNames("foo");
// ["0", "1", "2", "length"]  (ES2015 code)
```

Examples

Using Object.getOwnPropertyNames()

```
const arr = ["a", "b", "c"];
console.log(Object.getOwnPropertyNames(arr).sort());
// ["0", "1", "2", "length"]

// Array-like object
const obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.getOwnPropertyNames(obj).sort());
// ["0", "1", "2"]

Object.getOwnPropertyNames(obj).forEach((val, idx, array) => {
  console.log(`${val} -> ${obj[val]}`);
});
// 0 -> a
// 1 -> b
// 2 -> c

// non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
```

```
      enumerable: false,
    },
  },
);
myObj.foo = 1;

console.log(Object.getOwnPropertyNames(myObj).sort()); // ["foo", "getFoo"]
```

If you want only the enumerable properties, see `Object.keys()` or use a `for...in` loop (note that this will also return enumerable properties found along the prototype chain for the object unless the latter is filtered with `Object.hasOwn()`).

Items on the prototype chain are not listed:

```
function ParentClass() {}
ParentClass.prototype.inheritedMethod = function () {};

function ChildClass() {
  this.prop = 5;
  this.method = function () {};
}
ChildClass.prototype = new ParentClass();
ChildClass.prototype.prototypeMethod = function () {};

console.log(Object.getOwnPropertyNames(new ChildClass()));
// ["prop", "method"]
```

Get non-enumerable properties only

This uses the `Array.prototype.filter()` function to remove the enumerable keys (obtained with `Object.keys()`) from a list of all keys (obtained with `Object.getOwnPropertyNames()`) thus giving only the non-enumerable keys as output.

```
const target = myObject;
const enumAndNonEnum = Object.getOwnPropertyNames(target);
const enumOnly = new Set(Object.keys(target));
const nonEnumOnly = enumAndNonEnum.filter((key) => !enumOnly.has(key));

console.log(nonEnumOnly);
```

See also

- Polyfill of `Object.getOwnPropertyNames` in `core-js`
- Enumerability and ownership of properties
- `Object.hasOwn()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.keys()`
- `Array.prototype.forEach()`

Object.getOwnPropertySymbols()

The **`Object.getOwnPropertySymbols()`** static method returns an array of all symbol properties found directly upon a given object.

Try it

```
const object = {};
const a = Symbol("a");
const b = Symbol.for("b");

object[a] = "localSymbol";
object[b] = "globalSymbol";

const objectSymbols = Object.getOwnPropertySymbols(object);

console.log(objectSymbols.length);
// Expected output: 2
```

Syntax

```
Object.getOwnPropertySymbols(obj)
```

Parameters

`obj`

The object whose symbol properties are to be returned.

Return value

An array of all symbol properties found directly upon the given object.

Description

Similar to `Object.getOwnPropertyNames()`, you can get all symbol properties of a given object as an array of symbols. Note that `Object.getOwnPropertyNames()` itself does not contain the symbol properties of an object and only the string properties.

As all objects have no own symbol properties initially, `Object.getOwnPropertySymbols()` returns an empty array unless you have set symbol properties on your object.

Examples

Using Object.getOwnPropertySymbols()

```
const obj = {};
const a = Symbol("a");
const b = Symbol.for("b");

obj[a] = "localSymbol";
obj[b] = "globalSymbol";

const objectSymbols = Object.getOwnPropertySymbols(obj);

console.log(objectSymbols.length); // 2
console.log(objectSymbols); // [Symbol(a), Symbol(b)]
console.log(objectSymbols[0]); // Symbol(a)
```

See also

- Polyfill of `Object.getOwnPropertySymbols` in `core-js`
- `Object.getOwnPropertyNames()`
- `Symbol`

Object.getPrototypeOf()

The **`Object.getPrototypeOf()`** static method returns the prototype (i.e., the value of the internal `[[Prototype]]` property) of the specified object.

Try it

```
const prototype = {};
const object = Object.create(prototype);

console.log(Object.getPrototypeOf(object) === prototype);
// Expected output: true
```

Syntax

```
Object.getPrototypeOf(obj)
```

Parameters

`obj`

The object whose prototype is to be returned.

Return value

The prototype of the given object, which may be `null`.

Examples

Using getPrototypeOf

```
const proto = {};
const obj = Object.create(proto);
Object.getPrototypeOf(obj) === proto; // true
```

Non-object coercion

In ES5, it will throw a `TypeError` exception if the `obj` parameter isn't an object. In ES2015, the parameter will be coerced to an `Object`.

```
Object.getPrototypeOf("foo");
// TypeError: "foo" is not an object (ES5 code)
Object.getPrototypeOf("foo");
// String.prototype                 (ES2015 code)
```

See also

- Polyfill of `Object.getPrototypeOf` in `core-js`
- es-shims polyfill of `Object.getPrototypeOf`
- `Object.prototype.isPrototypeOf()`
- `Object.setPrototypeOf()`
- `Object.prototype.__proto__`
- `Reflect.getPrototypeOf()`
- Object.getPrototypeOf by John Resig (2008)

Object.groupBy()

> **Note:** In some versions of some browsers, this method was implemented as the method `Array.prototype.group()`. Due to web compatibility issues, it is now implemented as a static method. Check the browser compatibility table for details.

The **`Object.groupBy()`** static method groups the elements of a given iterable according to the string values returned by a provided callback function. The returned object has separate properties for each group, containing arrays with the elements in the group.

This method should be used when group names can be represented by strings. If you need to group elements using a key that is some arbitrary value, use `Map.groupBy()` instead.

Try it

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 9 },
  { name: "bananas", type: "fruit", quantity: 5 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 12 },
  { name: "fish", type: "meat", quantity: 22 },
];

const result = Object.groupBy(inventory, ({ quantity }) =>
  quantity < 6 ? "restock" : "sufficient",
);
console.log(result.restock);
// [{ name: "bananas", type: "fruit", quantity: 5 }]
```

Syntax

```
Object.groupBy(items, callbackFn)
```

Parameters

`items`

An iterable (such as an `Array`) whose elements will be grouped.

`callbackFn`

A function to execute for each element in the iterable. It should return a value that can get coerced into a property key (string or symbol) indicating the group of the current element. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

A `null`-prototype object with properties for all groups, each assigned to an array containing the elements of the associated group.

Description

`Object.groupBy()` calls a provided `callbackFn` function once for each element in an iterable. The callback function should return a string or symbol (values that are neither type are coerced to strings) indicating the group of the associated element. The values returned by `callbackFn` are used as keys for the object returned by `Object.groupBy()`. Each key has an associated array containing all the elements for which the callback returned the same value.

The elements in the returned object and the original iterable are the same (not deep copies). Changing the internal structure of the elements will be reflected in both the original iterable and the returned object.

Examples

Using Object.groupBy()

First we define an array containing objects representing an inventory of different foodstuffs. Each food has a `type` and a `quantity`.

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 5 },
  { name: "bananas", type: "fruit", quantity: 0 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 5 },
  { name: "fish", type: "meat", quantity: 22 },
];
```

The code below groups the elements by the value of their `type` property.

```
const result = Object.groupBy(inventory, ({ type }) => type);

/* Result is:
{
```

```
  vegetables: [
    { name: 'asparagus', type: 'vegetables', quantity: 5 },
  ],
  fruit: [
    { name: "bananas", type: "fruit", quantity: 0 },
    { name: "cherries", type: "fruit", quantity: 5 }
  ],
  meat: [
    { name: "goat", type: "meat", quantity: 23 },
    { name: "fish", type: "meat", quantity: 22 }
  ]
}
*/
```

The arrow function just returns the `type` of each array element each time it is called. Note that the function argument `{ type }` is a basic example of object destructuring syntax for function arguments. This unpacks the `type` property of an object passed as a parameter, and assigns it to a variable named `type` in the body of the function. This is a very succinct way to access the relevant values of elements within a function.

We can also create groups inferred from values in one or more properties of the elements. Below is a very similar example that puts the items into `ok` or `restock` groups based on the value of the `quantity` field.

```
function myCallback({ quantity }) {
  return quantity > 5 ? "ok" : "restock";
}

const result2 = Object.groupBy(inventory, myCallback);

/* Result is:
{
  restock: [
    { name: "asparagus", type: "vegetables", quantity: 5 },
    { name: "bananas", type: "fruit", quantity: 0 },
    { name: "cherries", type: "fruit", quantity: 5 }
  ],
  ok: [
    { name: "goat", type: "meat", quantity: 23 },
    { name: "fish", type: "meat", quantity: 22 }
  ]
}
*/
```

See also

- Polyfill of `Object.groupBy` in `core-js`
- es-shims polyfill of `Object.groupBy`
- Indexed collections guide
- `Array.prototype.reduce()`
- `Object.fromEntries()`
- `Map.groupBy()`

Object.hasOwn()

The **`Object.hasOwn()`** static method returns `true` if the specified object has the indicated property as its *own* property. If the property is inherited, or does not exist, the method returns `false`.

> **Note:** `Object.hasOwn()` is intended as a replacement for `Object.prototype.hasOwnProperty()`.

Try it

```
const object = {
  prop: "exists",
};

console.log(Object.hasOwn(object, "prop"));
// Expected output: true

console.log(Object.hasOwn(object, "toString"));
// Expected output: false

console.log(Object.hasOwn(object, "undeclaredPropertyValue"));
// Expected output: false
```

Syntax

```
Object.hasOwn(obj, prop)
```

Parameters

`obj`

The JavaScript object instance to test.

`prop`

The `String` name or Symbol of the property to test.

Return value

`true` if the specified object has directly defined the specified property. Otherwise `false`

Description

The `Object.hasOwn()` method returns `true` if the specified property is a direct property of the object — even if the property value is `null` or `undefined`. The method returns `false` if the property is inherited, or has not been declared at all. Unlike the `in` operator, this method does not check for the specified property in the object's prototype chain.

It is recommended over `Object.prototype.hasOwnProperty()` because it works for `null`-prototype objects and with objects that have overridden the inherited `hasOwnProperty()` method. While it is possible to work around these problems by accessing `Object.prototype.hasOwnProperty()` on another object (like `Object.prototype.hasOwnProperty.call(obj, prop)`), `Object.hasOwn()` is more intuitive and concise.

Examples

Using Object.hasOwn() to test for a property's existence

The following code shows how to determine whether the `example` object contains a property named `prop`.

```
const example = {};
Object.hasOwn(example, "prop"); // false - 'prop' has not been defined

example.prop = "exists";
Object.hasOwn(example, "prop"); // true - 'prop' has been defined

example.prop = null;
Object.hasOwn(example, "prop"); // true - own property exists with value of null

example.prop = undefined;
Object.hasOwn(example, "prop"); // true - own property exists with value of undefined
```

Direct vs. inherited properties

The following example differentiates between direct properties and properties inherited through the prototype chain:

```
const example = {};
example.prop = "exists";

// `hasOwn` will only return true for direct properties:
Object.hasOwn(example, "prop"); // true
Object.hasOwn(example, "toString"); // false
Object.hasOwn(example, "hasOwnProperty"); // false

// The `in` operator will return true for direct or inherited properties:
"prop" in example; // true
"toString" in example; // true
"hasOwnProperty" in example; // true
```

Iterating over the properties of an object

To iterate over the enumerable properties of an object, you *should* use:

```
const example = { foo: true, bar: true };
for (const name of Object.keys(example)) {
  // …
}
```

But if you need to use `for...in`, you can use `Object.hasOwn()` to skip the inherited properties:

```
const example = { foo: true, bar: true };
for (const name in example) {
  if (Object.hasOwn(example, name)) {
    // …
  }
}
```

Checking if an Array index exists

The elements of an `Array` are defined as direct properties, so you can use `hasOwn()` method to check whether a particular index exists:

```
const fruits = ["Apple", "Banana", "Watermelon", "Orange"];
Object.hasOwn(fruits, 3); // true ('Orange')
Object.hasOwn(fruits, 4); // false - not defined
```

Problematic cases for hasOwnProperty()

This section demonstrates that `Object.hasOwn()` is immune to the problems that affect `hasOwnProperty()`. Firstly, it can be used with objects that have re-implemented `hasOwnProperty()`. In the example below, the re-implemented `hasOwnProperty()` method reports false for *every* property, but the behavior of `Object.hasOwn()` remains unaffected:

```
const foo = {
  hasOwnProperty() {
    return false;
  },
  bar: "The dragons be out of office",
};

console.log(foo.hasOwnProperty("bar")); // false

console.log(Object.hasOwn(foo, "bar")); // true
```

It can also be used with `null`-prototype objects. These do not inherit from `Object.prototype`, and so `hasOwnProperty()` is inaccessible.

```
const foo = Object.create(null);
foo.prop = "exists";

console.log(foo.hasOwnProperty("prop"));
// Uncaught TypeError: foo.hasOwnProperty is not a function

console.log(Object.hasOwn(foo, "prop")); // true
```

See also

- Polyfill of `Object.hasOwn` in `core-js`
- es-shims polyfill of `Object.hasOwn`
- `Object.prototype.hasOwnProperty()`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `for...in`
- `in`
- Inheritance and the prototype chain

Object.prototype.hasOwnProperty()

The **`hasOwnProperty()`** method of `Object` instances returns a boolean indicating whether this object has the specified property as its own property (as opposed to inheriting it).

> **Note:** `Object.hasOwn()` is recommended over `hasOwnProperty()`, in browsers where it is supported.

Try it

```
const object = {};
object.foo = 42;

console.log(object.hasOwnProperty("foo"));
// Expected output: true

console.log(object.hasOwnProperty("toString"));
// Expected output: false

console.log(object.hasOwnProperty("hasOwnProperty"));
// Expected output: false
```

Syntax

```
hasOwnProperty(prop)
```

Parameters

`prop`

The `String` name or Symbol of the property to test.

Return value

Returns `true` if the object has the specified property as own property; `false` otherwise.

Description

The **`hasOwnProperty()`** method returns `true` if the specified property is a direct property of the object — even if the value is `null` or `undefined`. The method returns `false` if the property is inherited, or has not been declared at all. Unlike the `in` operator, this method does not check for the specified property in the object's prototype chain.

The method can be called on *most* JavaScript objects, because most objects descend from `Object`, and hence inherit its methods. For example `Array` is an `Object`, so you can use `hasOwnProperty()` method to check whether an index exists:

```
const fruits = ["Apple", "Banana", "Watermelon", "Orange"];
fruits.hasOwnProperty(3); // true ('Orange')
fruits.hasOwnProperty(4); // false - not defined
```

The method will not be available in objects where it is reimplemented, or on `null`-prototype objects (as these don't inherit from `Object.prototype`). Examples for these cases are given below.

Examples

Using hasOwnProperty to test for an own property's existence

The following code shows how to determine whether the `example` object contains a property named `prop`.

```
const example = {};
example.hasOwnProperty("prop"); // false

example.prop = "exists";
example.hasOwnProperty("prop"); // true - 'prop' has been defined

example.prop = null;
example.hasOwnProperty("prop"); // true - own property exists with value of null

example.prop = undefined;
example.hasOwnProperty("prop"); // true - own property exists with value of undefined
```

Direct vs. inherited properties

The following example differentiates between direct properties and properties inherited through the prototype chain:

```
const example = {};
example.prop = "exists";

// `hasOwnProperty` will only return true for direct properties:
example.hasOwnProperty("prop"); // true
example.hasOwnProperty("toString"); // false
example.hasOwnProperty("hasOwnProperty"); // false

// The `in` operator will return true for direct or inherited properties:
"prop" in example; // true
"toString" in example; // true
"hasOwnProperty" in example; // true
```

Iterating over the properties of an object

The following example shows how to iterate over the enumerable properties of an object without executing on inherited properties.

```
const buz = {
  fog: "stack",
};

for (const name in buz) {
  if (buz.hasOwnProperty(name)) {
    console.log(`this is fog (${name}) for sure. Value: ${buz[name]}`);
  } else {
    console.log(name); // toString or something else
  }
}
```

Note that the `for...in` loop only iterates enumerable items: the absence of non-enumerable properties emitted from the loop does not imply that `hasOwnProperty` itself is confined strictly to enumerable items. You can iterate over non-enumerable properties with `Object.getOwnPropertyNames()`.

Using hasOwnProperty as a property name

JavaScript does not protect the property name `hasOwnProperty`; an object that has a property with this name may return incorrect results:

```
const foo = {
  hasOwnProperty() {
    return false;
  },
  bar: "Here be dragons",
};

foo.hasOwnProperty("bar"); // re-implementation always returns false
```

The recommended way to overcome this problem is to instead use `Object.hasOwn()` (in browsers that support it). Other alternatives include using an *external* `hasOwnProperty`:

```
const foo = { bar: "Here be dragons" };

// Use Object.hasOwn() method - recommended
Object.hasOwn(foo, "bar"); // true
```

```
// Use the hasOwnProperty property from the Object prototype
Object.prototype.hasOwnProperty.call(foo, "bar"); // true

// Use another Object's hasOwnProperty
// and call it with 'this' set to foo
({}).hasOwnProperty.call(foo, "bar"); // true
```

Note that in the first two cases there are no newly created objects.

Objects created with Object.create(null)

`null`-prototype objects do not inherit from `Object.prototype`, making `hasOwnProperty()` inaccessible.

```
const foo = Object.create(null);
foo.prop = "exists";
foo.hasOwnProperty("prop"); // Uncaught TypeError: foo.hasOwnProperty is not a function
```

The solutions in this case are the same as for the previous section: use `Object.hasOwn()` by preference, otherwise use an external object's `hasOwnProperty()`.

See also

- `Object.hasOwn()`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `for...in`
- `in`
- Inheritance and the prototype chain

Object.is()

The **`Object.is()`** static method determines whether two values are the same value.

Try it

```
console.log(Object.is("1", 1));
// Expected output: false

console.log(Object.is(NaN, NaN));
// Expected output: true

console.log(Object.is(-0, 0));
// Expected output: false

const obj = {};
console.log(Object.is(obj, {}));
// Expected output: false
```

Syntax

```
Object.is(value1, value2)
```

Parameters

`value1`

The first value to compare.

`value2`

The second value to compare.

Return value

A boolean indicating whether or not the two arguments are the same value.

Description

`Object.is()` determines whether two values are the same value. Two values are the same if one of the following holds:

- both `undefined`
- both `null`
- both `true` or both `false`
- both strings of the same length with the same characters in the same order
- both the same object (meaning both values reference the same object in memory)
- both BigInts with the same numeric value
- both symbols that reference the same symbol value
- both numbers and
  - both `+0`
  - both `-0`
  - both `NaN`
  - or both non-zero, not `NaN`, and have the same value

`Object.is()` is not equivalent to the `==` operator. The `==` operator applies various coercions to both sides (if they are not the same type) before testing for equality (resulting in such behavior as `"" == false` being `true`), but `Object.is()` doesn't coerce either value.

`Object.is()` is also *not* equivalent to the `===` operator. The only difference between `Object.is()` and `===` is in their treatment of signed zeros and `NaN` values. The `===` operator (and the `==` operator) treats the number values `-0` and `+0` as equal, but treats `NaN` as not equal to each other.

Examples

Using Object.is()

```
// Case 1: Evaluation result is the same as using ===
Object.is(25, 25); // true
Object.is("foo", "foo"); // true
Object.is("foo", "bar"); // false
Object.is(null, null); // true
Object.is(undefined, undefined); // true
Object.is(window, window); // true
Object.is([], []); // false
const foo = { a: 1 };
const bar = { a: 1 };
const sameFoo = foo;
Object.is(foo, foo); // true
Object.is(foo, bar); // false
Object.is(foo, sameFoo); // true

// Case 2: Signed zero
Object.is(0, -0); // false
Object.is(+0, -0); // false
Object.is(-0, -0); // true

// Case 3: NaN
Object.is(NaN, 0 / 0); // true
Object.is(NaN, Number.NaN); // true
```

See also

- Polyfill of `Object.is` in `core-js`
- es-shims polyfill of `Object.is`
- Equality comparisons and sameness

Object.isExtensible()

The **`Object.isExtensible()`** static method determines if an object is extensible (whether it can have new properties added to it).

Try it

```
const object = {};

console.log(Object.isExtensible(object));
// Expected output: true

Object.preventExtensions(object);

console.log(Object.isExtensible(object));
// Expected output: false
```

Syntax

```
Object.isExtensible(obj)
```

Parameters

`obj`

The object which should be checked.

Return value

A `Boolean` indicating whether or not the given object is extensible.

Description

Objects are extensible by default: they can have new properties added to them, and their `[[Prototype]]` can be re-assigned. An object can be marked as non-extensible using one of `Object.preventExtensions()`, `Object.seal()`, `Object.freeze()`, or `Reflect.preventExtensions()`.

Examples

Using Object.isExtensible

```
// New objects are extensible.
const empty = {};
Object.isExtensible(empty); // true
```

```
// They can be made un-extensible
Object.preventExtensions(empty);
Object.isExtensible(empty); // false

// Sealed objects are by definition non-extensible.
const sealed = Object.seal({});
Object.isExtensible(sealed); // false

// Frozen objects are also by definition non-extensible.
const frozen = Object.freeze({});
Object.isExtensible(frozen); // false
```

Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, it will return `false` without any errors if a non-object argument is passed, since primitives are, by definition, immutable.

```
Object.isExtensible(1);
// TypeError: 1 is not an object (ES5 code)

Object.isExtensible(1);
// false                        (ES2015 code)
```

See also

- `Object.preventExtensions()`
- `Object.seal()`
- `Object.isSealed()`
- `Object.freeze()`
- `Object.isFrozen()`
- `Reflect.isExtensible()`

Object.isFrozen()

The **`Object.isFrozen()`** static method determines if an object is frozen.

Try it

```
const object = {
  foo: 42,
};

console.log(Object.isFrozen(object));
// Expected output: false

Object.freeze(object);

console.log(Object.isFrozen(object));
// Expected output: true
```

Syntax

```
Object.isFrozen(obj)
```

Parameters

`obj`

The object which should be checked.

Return value

A `Boolean` indicating whether or not the given object is frozen.

Description

An object is frozen if and only if it is not extensible, all its properties are non-configurable, and all its data properties (that is, properties which are not accessor properties with getter or setter components) are non-writable.

Examples

Using Object.isFrozen

```
// A new object is extensible, so it is not frozen.
Object.isFrozen({}); // false

// An empty object which is not extensible
// is vacuously frozen.
const vacuouslyFrozen = Object.preventExtensions({});
Object.isFrozen(vacuouslyFrozen); // true

// A new object with one property is also extensible,
// ergo not frozen.
```

```
const oneProp = { p: 42 };
Object.isFrozen(oneProp); // false

// Preventing extensions to the object still doesn't
// make it frozen, because the property is still
// configurable (and writable).
Object.preventExtensions(oneProp);
Object.isFrozen(oneProp); // false

// Deleting that property makes the object vacuously frozen.
delete oneProp.p;
Object.isFrozen(oneProp); // true

// A non-extensible object with a non-writable
// but still configurable property is not frozen.
const nonWritable = { e: "plep" };
Object.preventExtensions(nonWritable);
Object.defineProperty(nonWritable, "e", {
  writable: false,
}); // make non-writable
Object.isFrozen(nonWritable); // false

// Changing that property to non-configurable
// then makes the object frozen.
Object.defineProperty(nonWritable, "e", {
  configurable: false,
}); // make non-configurable
Object.isFrozen(nonWritable); // true

// A non-extensible object with a non-configurable
// but still writable property also isn't frozen.
const nonConfigurable = { release: "the kraken!" };
Object.preventExtensions(nonConfigurable);
Object.defineProperty(nonConfigurable, "release", {
  configurable: false,
});
Object.isFrozen(nonConfigurable); // false

// Changing that property to non-writable
// then makes the object frozen.
Object.defineProperty(nonConfigurable, "release", {
  writable: false,
});
Object.isFrozen(nonConfigurable); // true

// A non-extensible object with a configurable
// accessor property isn't frozen.
const accessor = {
  get food() {
    return "yum";
  },
};
Object.preventExtensions(accessor);
Object.isFrozen(accessor); // false

// When we make that property non-configurable it becomes frozen.
Object.defineProperty(accessor, "food", {
  configurable: false,
});
Object.isFrozen(accessor); // true

// But the easiest way for an object to be frozen
// is if Object.freeze has been called on it.
const frozen = { 1: 81 };
Object.isFrozen(frozen); // false
Object.freeze(frozen);
Object.isFrozen(frozen); // true

// By definition, a frozen object is non-extensible.
Object.isExtensible(frozen); // false

// Also by definition, a frozen object is sealed.
Object.isSealed(frozen); // true
```

Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, it will return `true` without any errors if a non-object argument is passed, since primitives are, by definition, immutable.

```
Object.isFrozen(1);
// TypeError: 1 is not an object (ES5 code)

Object.isFrozen(1);
// true                          (ES2015 code)
```

See also

- `Object.freeze()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`

Object.prototype.isPrototypeOf()

The **`isPrototypeOf()`** method of `Object` instances checks if this object exists in another object's prototype chain.

> **Note:** `isPrototypeOf()` differs from the `instanceof` operator. In the expression `object instanceof AFunction`, `object`'s prototype chain is checked against `AFunction.prototype`, not against `AFunction` itself.

Try it

```
function Foo() {}
function Bar() {}

Bar.prototype = Object.create(Foo.prototype);

const bar = new Bar();

console.log(Foo.prototype.isPrototypeOf(bar));
// Expected output: true
console.log(Bar.prototype.isPrototypeOf(bar));
// Expected output: true
```

Syntax

```
isPrototypeOf(object)
```

Parameters

`object`

The object whose prototype chain will be searched.

Return value

A boolean indicating whether the calling object (`this`) lies in the prototype chain of `object`. Directly returns `false` when `object` is not an object (i.e., a primitive).

Exceptions

`TypeError`

Thrown if `this` is `null` or `undefined` (because it can't be converted to an object).

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `isPrototypeOf()` method. This method allows you to check whether or not the object exists within another object's prototype chain. If the `object` passed as the parameter is not an object (i.e., a primitive), the method directly returns `false`. Otherwise, the `this` value is converted to an object, and the prototype chain of `object` is searched for the `this` value, until the end of the chain is reached or the `this` value is found.

Examples

Using isPrototypeOf()

This example demonstrates that `Baz.prototype`, `Bar.prototype`, `Foo.prototype` and `Object.prototype` exist in the prototype chain for object `baz`:

```
class Foo {}
class Bar extends Foo {}
class Baz extends Bar {}

const foo = new Foo();
const bar = new Bar();
const baz = new Baz();

// prototype chains:
// foo: Foo --> Object
// bar: Bar --> Foo --> Object
// baz: Baz --> Bar --> Foo --> Object
console.log(Baz.prototype.isPrototypeOf(baz)); // true
console.log(Baz.prototype.isPrototypeOf(bar)); // false
console.log(Baz.prototype.isPrototypeOf(foo)); // false
console.log(Bar.prototype.isPrototypeOf(baz)); // true
console.log(Bar.prototype.isPrototypeOf(foo)); // false
console.log(Foo.prototype.isPrototypeOf(baz)); // true
```

```
console.log(Foo.prototype.isPrototypeOf(bar)); // true
console.log(Object.prototype.isPrototypeOf(baz)); // true
```

The `isPrototypeOf()` method — along with the `instanceof` operator — comes in particularly handy if you have code that can only function when dealing with objects descended from a specific prototype chain; e.g., to guarantee that certain methods or properties will be present on that object.

For example, to execute some code that's only safe to run if a `baz` object has `Foo.prototype` in its prototype chain, you can do this:

```
if (Foo.prototype.isPrototypeOf(baz)) {
  // do something safe
}
```

However, `Foo.prototype` existing in `baz`'s prototype chain doesn't imply `baz` was created using `Foo` as its constructor. For example, `baz` could be directly assigned with `Foo.prototype` as its prototype. In this case, if your code reads private fields of `Foo` from `baz`, it would still fail:

```
class Foo {
  #value = "foo";
  static getValue(x) {
    return x.#value;
  }
}

const baz = { __proto__: Foo.prototype };

if (Foo.prototype.isPrototypeOf(baz)) {
  console.log(Foo.getValue(baz)); // TypeError: Cannot read private member #value from an object whose class did not declare it
}
```

The same applies to `instanceof`. If you need to read private fields in a secure way, offer a branded check method using `in` instead.

```
class Foo {
  #value = "foo";
  static getValue(x) {
    return x.#value;
  }
  static isFoo(x) {
    return #value in x;
  }
}

const baz = { __proto__: Foo.prototype };

if (Foo.isFoo(baz)) {
  // Doesn't run, because baz is not a Foo
  console.log(Foo.getValue(baz));
}
```

See also

- `instanceof`
- `Object.getPrototypeOf()`
- `Object.setPrototypeOf()`
- Inheritance and the prototype chain

Object.isSealed()

The **`Object.isSealed()`** static method determines if an object is sealed.

Try it

```
const object = {
  foo: 42,
};

console.log(Object.isSealed(object));
// Expected output: false

Object.seal(object);

console.log(Object.isSealed(object));
// Expected output: true
```

Syntax

```
Object.isSealed(obj)
```

Parameters

`obj`

The object which should be checked.

Return value

A `Boolean` indicating whether or not the given object is sealed.

Description

Returns `true` if the object is sealed, otherwise `false`. An object is sealed if it is not extensible and if all its properties are non-configurable and therefore not removable (but not necessarily non-writable).

Examples

Using Object.isSealed

```
// Objects aren't sealed by default.
const empty = {};
Object.isSealed(empty); // false

// If you make an empty object non-extensible,
// it is vacuously sealed.
Object.preventExtensions(empty);
Object.isSealed(empty); // true

// The same is not true of a non-empty object,
// unless its properties are all non-configurable.
const hasProp = { fee: "fie foe fum" };
Object.preventExtensions(hasProp);
Object.isSealed(hasProp); // false

// But make them all non-configurable
// and the object becomes sealed.
Object.defineProperty(hasProp, "fee", {
  configurable: false,
});
Object.isSealed(hasProp); // true

// The easiest way to seal an object, of course,
// is Object.seal.
const sealed = {};
Object.seal(sealed);
Object.isSealed(sealed); // true

// A sealed object is, by definition, non-extensible.
Object.isExtensible(sealed); // false

// A sealed object might be frozen,
// but it doesn't have to be.
Object.isFrozen(sealed); // true
// (all properties also non-writable)

const s2 = Object.seal({ p: 3 });
Object.isFrozen(s2); // false
// ('p' is still writable)

const s3 = Object.seal({
  get p() {
    return 0;
  },
});
Object.isFrozen(s3); // true
// (only configurability matters for accessor properties)
```

Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, it will return `true` without any errors if a non-object argument is passed, since primitives are, by definition, immutable.

```
Object.isSealed(1);
// TypeError: 1 is not an object (ES5 code)

Object.isSealed(1);
// true                          (ES2015 code)
```

See also

- `Object.seal()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.freeze()`
- `Object.isFrozen()`

Object.keys()

The **`Object.keys()`** static method returns an array of a given object's own enumerable string-keyed property names.

Try it

```
const object = {
  a: "some string",
  b: 42,
  c: false,
};

console.log(Object.keys(object));
// Expected output: Array ["a", "b", "c"]
```

Syntax

```
Object.keys(obj)
```

Parameters

`obj`

An object.

Return value

An array of strings representing the given object's own enumerable string-keyed property keys.

Description

`Object.keys()` returns an array whose elements are strings corresponding to the enumerable string-keyed property names found directly upon `object`. This is the same as iterating with a `for...in` loop, except that a `for...in` loop enumerates properties in the prototype chain as well. The order of the array returned by `Object.keys()` is the same as that provided by a `for...in` loop.

If you need the property values, use `Object.values()` instead. If you need both the property keys and values, use `Object.entries()` instead.

Examples

Using Object.keys()

```
// Basic array
const arr = ["a", "b", "c"];
console.log(Object.keys(arr)); // ['0', '1', '2']

// Array-like object
const obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.keys(obj)); // ['0', '1', '2']

// Array-like object with random key ordering
const anObj = { 100: "a", 2: "b", 7: "c" };
console.log(Object.keys(anObj)); // ['2', '7', '100']

// getFoo is a non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
    },
  },
);
myObj.foo = 1;
console.log(Object.keys(myObj)); // ['foo']
```

If you want *all* string-keyed own properties, including non-enumerable ones, see `Object.getOwnPropertyNames()`.

Using Object.keys() on primitives

Non-object arguments are coerced to objects. `undefined` and `null` cannot be coerced to objects and throw a `TypeError` upfront. Only strings may have own enumerable properties, while all other primitives return an empty array.

```
// Strings have indices as enumerable own properties
console.log(Object.keys("foo")); // ['0', '1', '2']

// Other primitives except undefined and null have no own properties
console.log(Object.keys(100)); // []
```

> **Note:** In ES5, passing a non-object to `Object.keys()` threw a `TypeError`.

See also

- Polyfill of `Object.keys` in `core-js`

- es-shims polyfill of `Object.keys`
- Enumerability and ownership of properties
- `Object.entries()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`
- `Map.prototype.keys()`

Object() constructor

The **`Object()`** constructor turns the input into an object. Its behavior depends on the input's type.

Syntax

```
new Object()
new Object(value)

Object()
Object(value)
```

> **Note:** `Object()` can be called with or without `new`, but sometimes with different effects. See Return value.

Parameters

`value` Optional

Any value.

Return value

When the `Object()` constructor itself is called or constructed, its return value is an object:

- If the value is `null` or `undefined`, it creates and returns an empty object.
- If the value is an object already, it returns the value.
- Otherwise, it returns an object of a type that corresponds to the given value. For example, passing a `BigInt` primitive returns a `BigInt` wrapper object.

When `Object()` is constructed but `new.target` is not the `Object` constructor itself, the behavior is slightly different — it initializes a new object with `new.target.prototype` as its prototype. Any argument value is ignored. This may happen, for example, when `Object()` is implicitly called via `super()` in the constructor of a class that extends `Object`. In this case, even if you pass a number to `super()`, the `this` value inside the constructor does not become a `Number` instance.

Examples

Creating a new Object

```
const o = new Object();
o.foo = 42;

console.log(o);
// { foo: 42 }
```

Using Object given undefined and null types

The following examples store an empty `Object` object in `o`:

```
const o = new Object();
```

```
const o = new Object(undefined);
```

```
const o = new Object(null);
```

Obtaining wrapper objects for BigInt and Symbol

The `BigInt()` and `Symbol()` constructors throw an error when called with `new`, to prevent the common mistake of creating a wrapper object instead of the primitive value. The only way to create a wrapper object for these types is to call `Object()` with them:

```
const numberObj = new Number(1);
console.log(typeof numberObj); // "object"

const bigintObj = Object(1n);
console.log(typeof bigintObj); // "object"

const symbolObj = Object(Symbol("foo"));
console.log(typeof symbolObj); // "object"
```

See also

- Object initializer

Object.preventExtensions()

The **`Object.preventExtensions()`** static method prevents new properties from ever being added to an object (i.e., prevents future extensions to the object). It also prevents the object's prototype from being re-assigned.

Try it

```
const object = {};

Object.preventExtensions(object);

try {
  Object.defineProperty(object, "foo", {
    value: 42,
  });
} catch (e) {
  console.log(e);
  // Expected output: TypeError: Cannot define property foo, object is not extensible
}
```

Syntax

```
Object.preventExtensions(obj)
```

Parameters

`obj`

The object which should be made non-extensible.

Return value

The object being made non-extensible.

Description

An object is extensible if new properties can be added to it. `Object.preventExtensions()` marks an object as no longer extensible, so that it will never have properties beyond the ones it had at the time it was marked as non-extensible. Note that the properties of a non-extensible object, in general, may still be *deleted*. Attempting to add new properties to a non-extensible object will fail, either silently or, in strict mode, throwing a `TypeError`.

Unlike `Object.seal()` and `Object.freeze()`, `Object.preventExtensions()` invokes an intrinsic JavaScript behavior and cannot be replaced with a composition of several other operations. It also has its `Reflect` counterpart (which only exists for intrinsic operations), `Reflect.preventExtensions()`.

`Object.preventExtensions()` only prevents addition of own properties. Properties can still be added to the object prototype.

This method makes the `[[Prototype]]` of the target immutable; any `[[Prototype]]` re-assignment will throw a `TypeError`. This behavior is specific to the internal `[[Prototype]]` property; other properties of the target object will remain mutable.

There is no way to make an object extensible again once it has been made non-extensible.

Examples

Using Object.preventExtensions

```
// Object.preventExtensions returns the object
// being made non-extensible.
const obj = {};
const obj2 = Object.preventExtensions(obj);
obj === obj2; // true

// Objects are extensible by default.
const empty = {};
Object.isExtensible(empty); // true

// They can be made un-extensible
Object.preventExtensions(empty);
Object.isExtensible(empty); // false

// Object.defineProperty throws when adding
// a new property to a non-extensible object.
const nonExtensible = { removable: true };
Object.preventExtensions(nonExtensible);
Object.defineProperty(nonExtensible, "new", {
  value: 8675309,
}); // throws a TypeError

// In strict mode, attempting to add new properties
// to a non-extensible object throws a TypeError.
function fail() {
  "use strict";
  // throws a TypeError
  nonExtensible.newProperty = "FAIL";
}
fail();
```

A non-extensible object's prototype is immutable:

```
const fixed = Object.preventExtensions({});
// throws a 'TypeError'.
fixed.__proto__ = { oh: "hai" };
```

Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object argument will be returned as-is without any errors, since primitives are already, by definition, immutable.

```
Object.preventExtensions(1);
// TypeError: 1 is not an object (ES5 code)

Object.preventExtensions(1);
// 1                            (ES2015 code)
```

See also

- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`
- `Object.freeze()`
- `Object.isFrozen()`
- `Reflect.preventExtensions()`

Object.prototype.propertyIsEnumerable()

The **`propertyIsEnumerable()`** method of `Object` instances returns a boolean indicating whether the specified property is this object's enumerable own property.

Try it

```
const object = {};
const array = [];
object.foo = 42;
array[0] = 42;

console.log(object.propertyIsEnumerable("foo"));
// Expected output: true

console.log(array.propertyIsEnumerable(0));
// Expected output: true

console.log(array.propertyIsEnumerable("length"));
// Expected output: false
```

Syntax

```
propertyIsEnumerable(prop)
```

Parameters

`prop`

The name of the property to test. Can be a string or a `Symbol`.

Return value

A boolean value indicating whether the specified property is enumerable and is the object's own property.

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `propertyIsEnumerable()` method. This method determines if the specified property, string or symbol, is an enumerable own property of the object. If the object does not have the specified property, this method returns `false`.

This method is equivalent to `Object.getOwnPropertyDescriptor(obj, prop)?.enumerable ?? false`.

Examples

Using propertyIsEnumerable()

The following example shows the use of `propertyIsEnumerable()` on objects and arrays.

```
const o = {};
const a = [];
o.prop = "is enumerable";
a[0] = "is enumerable";

o.propertyIsEnumerable("prop"); // true
a.propertyIsEnumerable(0); // true
```

User-defined vs. built-in objects

Most built-in properties are non-enumerable by default, while user-created object properties are often enumerable, unless explicitly designated otherwise.

```
const a = ["is enumerable"];

a.propertyIsEnumerable(0); // true
a.propertyIsEnumerable("length"); // false

Math.propertyIsEnumerable("random"); // false
globalThis.propertyIsEnumerable("Math"); // false
```

Direct vs. inherited properties

Only enumerable own properties cause `propertyIsEnumerable()` to return `true`, although all enumerable properties, including inherited ones, are visited by the `for...in` loop.

```
const o1 = {
  enumerableInherited: "is enumerable",
};
Object.defineProperty(o1, "nonEnumerableInherited", {
  value: "is non-enumerable",
  enumerable: false,
});
const o2 = {
  // o1 is the prototype of o2
  __proto__: o1,
  enumerableOwn: "is enumerable",
};
Object.defineProperty(o2, "nonEnumerableOwn", {
  value: "is non-enumerable",
  enumerable: false,
});

o2.propertyIsEnumerable("enumerableInherited"); // false
o2.propertyIsEnumerable("nonEnumerableInherited"); // false
o2.propertyIsEnumerable("enumerableOwn"); // true
o2.propertyIsEnumerable("nonEnumerableOwn"); // false
```

Testing symbol properties

`Symbol` properties are also supported by `propertyIsEnumerable()`. Note that most enumeration methods only visit string properties; enumerability of symbol properties is only useful when using `Object.assign()` or spread syntax. For more information, see Enumerability and ownership of properties.

```
const sym = Symbol("enumerable");
const sym2 = Symbol("non-enumerable");
const o = {
  [sym]: "is enumerable",
};
Object.defineProperty(o, sym2, {
  value: "is non-enumerable",
  enumerable: false,
});

o.propertyIsEnumerable(sym); // true
o.propertyIsEnumerable(sym2); // false
```

Usage with null-prototype objects

Because `null`-prototype objects do not inherit from `Object.prototype`, they do not inherit the `propertyIsEnumerable()` method. You must call `Object.prototype.propertyIsEnumerable` with the object as `this` instead.

```
const o = {
  __proto__: null,
  enumerableOwn: "is enumerable",
};

o.propertyIsEnumerable("enumerableOwn"); // TypeError: o.propertyIsEnumerable is not a function
Object.prototype.propertyIsEnumerable.call(o, "enumerableOwn"); // true
```

Alternatively, you may use `Object.getOwnPropertyDescriptor()` instead, which also helps to distinguish between non-existent properties and actually non-enumerable properties.

```
const o = {
  __proto__: null,
  enumerableOwn: "is enumerable",
};

Object.getOwnPropertyDescriptor(o, "enumerableOwn")?.enumerable; // true
Object.getOwnPropertyDescriptor(o, "nonExistent")?.enumerable; // undefined
```

See also

- Enumerability and ownership of properties

- `for...in`
- `Object.keys()`
- `Object.defineProperty()`

Object.prototype.__proto__

> **Warning:** Changing the `[[Prototype]]` of an object is, by the nature of how modern JavaScript engines optimize property accesses, currently a very slow operation in every browser and JavaScript engine. In addition, the effects of altering inheritance are subtle and far-flung, and are not limited to the time spent in the `obj.__proto__ = ...` statement, but may extend to ***any*** code that has access to any object whose `[[Prototype]]` has been altered. You can read more in JavaScript engine fundamentals: optimizing prototypes.

> **Note:** The use of `__proto__` is controversial and discouraged. Its existence and exact behavior have only been standardized as a legacy feature to ensure web compatibility, while it presents several security issues and footguns. For better support, prefer `Object.getPrototypeOf()`/`Reflect.getPrototypeOf()` and `Object.setPrototypeOf()`/`Reflect.setPrototypeOf()` instead.

The **`__proto__`** accessor property of `Object` instances exposes the `[[Prototype]]` (either an object or `null`) of this object.

The `__proto__` property can also be used in an object literal definition to set the object `[[Prototype]]` on creation, as an alternative to `Object.create()`. See: object initializer / literal syntax. That syntax is standard and optimized for in implementations, and quite different from `Object.prototype.__proto__`.

Syntax

```
obj.__proto__
```

Return value

If used as a getter, returns the object's `[[Prototype]]`.

Exceptions

`TypeError`

Thrown if attempting to set the prototype of a non-extensible object or an immutable prototype exotic object, such as `Object.prototype` or `window`.

Description

The `__proto__` getter function exposes the value of the internal `[[Prototype]]` of an object. For objects created using an object literal (unless you use the prototype setter syntax), this value is `Object.prototype`. For objects created using array literals, this value is `Array.prototype`. For functions, this value is `Function.prototype`. You can read more about the prototype chain in Inheritance and the prototype chain.

The `__proto__` setter allows the `[[Prototype]]` of an object to be mutated. The value provided must be an object or `null`. Providing any other value will do nothing.

Unlike `Object.getPrototypeOf()` and `Object.setPrototypeOf()`, which are always available on `Object` as static properties and always reflect the `[[Prototype]]` internal property, the `__proto__` property doesn't always exist as a property on all objects, and as a result doesn't reflect `[[Prototype]]` reliably.

The `__proto__` property is just an accessor property on `Object.prototype` consisting of a getter and setter function. A property access for `__proto__` that eventually consults `Object.prototype` will find this property, but an access that does not consult `Object.prototype` will not. If some other `__proto__` property is found before `Object.prototype` is consulted, that property will hide the one found on `Object.prototype`.

`null`-prototype objects don't inherit any property from `Object.prototype`, including the `__proto__` accessor property, so if you try to read `__proto__` on such an object, the value is always `undefined` regardless of the object's actual `[[Prototype]]`, and any assignment to `__proto__` would create a new property called `__proto__` instead of setting the object's prototype. Furthermore, `__proto__` can be redefined as an own property on any object instance through `Object.defineProperty()` without triggering the setter. In this case, `__proto__` will no longer be an accessor for `[[Prototype]]`. Therefore, always prefer `Object.getPrototypeOf()` and `Object.setPrototypeOf()` for setting and getting the `[[Prototype]]` of an object.

Examples

Using __proto__

```
function Circle() {}
const shape = {};
const circle = new Circle();

// Set the object prototype.
// DEPRECATED. This is for example purposes only. DO NOT DO THIS in real code.
shape.__proto__ = circle;

// Get the object prototype
console.log(shape.__proto__ === Circle); // false
```

```
function ShapeA() {}
const ShapeB = {
  a() {
    console.log("aaa");
  },
};

ShapeA.prototype.__proto__ = ShapeB;
console.log(ShapeA.prototype.__proto__); // { a: [Function: a] }
```

```
const shapeA = new ShapeA();
shapeA.a(); // aaa
console.log(ShapeA.prototype === shapeA.__proto__); // true
```

```
function ShapeC() {}
const ShapeD = {
  a() {
    console.log("a");
  },
};

const shapeC = new ShapeC();
shapeC.__proto__ = ShapeD;
shapeC.a(); // a
console.log(ShapeC.prototype === shapeC.__proto__); // false
```

```
function Test() {}
Test.prototype.myName = function () {
  console.log("myName");
};

const test = new Test();
console.log(test.__proto__ === Test.prototype); // true
test.myName(); // myName

const obj = {};
obj.__proto__ = Test.prototype;
obj.myName(); // myName
```

See also

- `Object.prototype.isPrototypeOf()`
- `Object.getPrototypeOf()`
- `Object.setPrototypeOf()`
- Prototype pollution attack

Object.seal()

The **`Object.seal()`** static method *seals* an object. Sealing an object prevents extensions and makes existing properties non-configurable. A sealed object has a fixed set of properties: new properties cannot be added, existing properties cannot be removed, their enumerability and configurability cannot be changed, and its prototype cannot be re-assigned. Values of existing properties can still be changed as long as they are writable. `seal()` returns the same object that was passed in.

Try it

```
const object = {
  foo: 42,
};

Object.seal(object);
object.foo = 33;
console.log(object.foo);
// Expected output: 33

delete object.foo; // Cannot delete when sealed
console.log(object.foo);
// Expected output: 33
```

Syntax

```
Object.seal(obj)
```

Parameters

`obj`

The object which should be sealed.

Return value

The object being sealed.

Description

Sealing an object is equivalent to preventing extensions and then changing all existing properties' descriptors to `configurable: false`. This has the effect of making the set of properties on the object fixed. Making all properties non-configurable also prevents them from being converted from data properties to accessor properties and vice versa, but it does not prevent the values of data properties from being changed. Attempting to delete or add properties to a sealed object, or to convert a data property to accessor or vice versa, will fail, either silently or by throwing a `TypeError` (most commonly, although not exclusively, when in strict mode code).

Private elements are not properties and do not have the concept of property descriptors. Private elements cannot be added or removed from the object, whether the object is sealed or not.

The prototype chain remains untouched. However, due to the effect of preventing extensions, the `[[Prototype]]` cannot be reassigned.

Unlike `Object.freeze()`, objects sealed with `Object.seal()` may have their existing properties changed, as long as they are writable.

Examples

Using Object.seal

```
const obj = {
  prop() {},
  foo: "bar",
};

// New properties may be added, existing properties
// may be changed or removed.
obj.foo = "baz";
obj.lumpy = "woof";
delete obj.prop;

const o = Object.seal(obj);

o === obj; // true
Object.isSealed(obj); // true

// Changing property values on a sealed object
// still works.
obj.foo = "quux";

// But you can't convert data properties to accessors,
// or vice versa.
Object.defineProperty(obj, "foo", {
  get() {
    return "g";
  },
}); // throws a TypeError

// Now any changes, other than to property values,
// will fail.
obj.quaxxor = "the friendly duck";
// silently doesn't add the property
delete obj.foo;
// silently doesn't delete the property

// … and in strict mode such attempts
// will throw TypeErrors.
function fail() {
  "use strict";
  delete obj.foo; // throws a TypeError
  obj.sparky = "arf"; // throws a TypeError
}
fail();

// Attempted additions through
// Object.defineProperty will also throw.
Object.defineProperty(obj, "ohai", {
  value: 17,
}); // throws a TypeError
Object.defineProperty(obj, "foo", {
  value: "eit",
}); // changes existing property value
```

Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object argument will be returned as-is without any errors, since primitives are already, by definition, immutable.

```
Object.seal(1);
// TypeError: 1 is not an object (ES5 code)

Object.seal(1);
// 1                            (ES2015 code)
```

See also

- `Object.isSealed()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.freeze()`
- `Object.isFrozen()`

Object.setPrototypeOf()

The **`Object.setPrototypeOf()`** static method sets the prototype (i.e., the internal `[[Prototype]]` property) of a specified object to another object or `null`.

> **Warning:** Changing the `[[Prototype]]` of an object is, by the nature of how modern JavaScript engines optimize property accesses, currently a very slow operation in every browser and JavaScript engine. In addition, the effects of altering inheritance are subtle and far-flung, and are not limited to the time spent in the `Object.setPrototypeOf(...)` statement, but may extend to ***any*** code that has access to any object whose `[[Prototype]]` has been altered. You can read more in JavaScript engine fundamentals: optimizing prototypes.
>
> Because this feature is a part of the language, it is still the burden on engine developers to implement that feature performantly (ideally). Until engine developers address this issue, if you are concerned about performance, you should avoid setting the `[[Prototype]]` of an object. Instead, create a new object with the desired `[[Prototype]]` using `Object.create()`.

Try it

```
const obj = {};
const parent = { foo: "bar" };

console.log(obj.foo);
// Expected output: undefined

Object.setPrototypeOf(obj, parent);

console.log(obj.foo);
// Expected output: "bar"
```

Syntax

```
Object.setPrototypeOf(obj, prototype)
```

Parameters

`obj`

The object which is to have its prototype set.

`prototype`

The object's new prototype (an object or `null`).

Return value

The specified object.

Exceptions

`TypeError`

Thrown in one of the following cases:

- The `obj` parameter is `undefined` or `null`.
- The `obj` parameter is non-extensible, or it's an immutable prototype exotic object, such as `Object.prototype` or `window`. However, the error is not thrown if the new prototype is the same value as the original prototype of `obj`.
- The `prototype` parameter is not an object or `null`.

Description

`Object.setPrototypeOf()` is generally considered the proper way to set the prototype of an object. You should always use it in favor of the deprecated `Object.prototype.__proto__` accessor.

If the `obj` parameter is not an object (e.g., number, string, etc.), this method does nothing — without coercing it to an object or attempting to set its prototype — and directly returns `obj` as a primitive value. If `prototype` is the same value as the prototype of `obj`, then `obj` is directly returned, without causing a `TypeError` even when `obj` has immutable prototype.

For security concerns, there are certain built-in objects that are designed to have an *immutable prototype*. This prevents prototype pollution attacks, especially proxy-related ones. The core language only specifies `Object.prototype` as an immutable prototype exotic object, whose prototype is always `null`. In browsers, `window` and `location` are two other very common examples.

```
Object.isExtensible(Object.prototype); // true; you can add more properties
Object.setPrototypeOf(Object.prototype, {}); // TypeError: Immutable prototype object '#<Object>' cannot have their prototype set
Object.setPrototypeOf(Object.prototype, null); // No error; the prototype of `Object.prototype` is already `null`
```

Examples

Pseudoclassical inheritance using Object.setPrototypeOf()

Inheritance in JS using classes.

```
class Human {}
class SuperHero extends Human {}

const superMan = new SuperHero();
```

However, if we want to implement subclasses without using `class`, we can do the following:

```
function Human(name, level) {
  this.name = name;
  this.level = level;
}

function SuperHero(name, level) {
  Human.call(this, name, level);
}

Object.setPrototypeOf(SuperHero.prototype, Human.prototype);

// Set the `[[Prototype]]` of `SuperHero.prototype`
// to `Human.prototype`
// To set the prototypal inheritance chain

Human.prototype.speak = function () {
  return `${this.name} says hello.`;
};

SuperHero.prototype.fly = function () {
  return `${this.name} is flying.`;
};

const superMan = new SuperHero("Clark Kent", 1);

console.log(superMan.fly());
console.log(superMan.speak());
```

The similarity between classical inheritance (with classes) and pseudoclassical inheritance (with constructors' `prototype` property) as done above is mentioned in Inheritance chains.

Since function constructors' `prototype` property is writable, you can reassign it to a new object created with `Object.create()` to achieve the same inheritance chain as well. There are caveats to watch out when using `create()`, such as remembering to re-add the `constructor` property.

In the example below, which also uses classes, `SuperHero` is made to inherit from `Human` without using `extends` by using `setPrototypeOf()` instead.

> **Warning:** It is not advisable to use `setPrototypeOf()` instead of `extends` due to performance and readability reasons.

```
class Human {}
class SuperHero {}

// Set the instance properties
Object.setPrototypeOf(SuperHero.prototype, Human.prototype);

// Hook up the static properties
Object.setPrototypeOf(SuperHero, Human);

const superMan = new SuperHero();
```

Subclassing without `extends` is mentioned in ES-6 subclassing.

See also

- Polyfill of `Object.setPrototypeOf` in `core-js`
- `Reflect.setPrototypeOf()`
- `Object.prototype.isPrototypeOf()`
- `Object.getPrototypeOf()`
- `Object.prototype.__proto__`
- Inheritance chain
- ES6 In Depth: Subclassing on hacks.mozilla.org (2015)

Object.prototype.toLocaleString()

The **`toLocaleString()`** method of `Object` instances returns a string representing this object. This method is meant to be overridden by derived objects for locale-specific purposes.

Try it

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

console.log(date.toLocaleString("ar-EG"));
// Expected output: "٢٠‏/١٢‏/٢٠١٢ ٤:٠٠:٠٠ ص"

const number = 123456.789;

console.log(number.toLocaleString("de-DE"));
// Expected output: "123.456,789"
```

Syntax

```
toLocaleString()
```

Parameters

None. However, all objects that override this method are expected to accept at most two parameters, corresponding to `locales` and `options`, such as `Number.prototype.toLocaleString`. The parameter positions should not be used for any other purpose.

Return value

The return value of calling `this.toString()`.

Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `toLocaleString()` method. `Object`'s `toLocaleString` returns the result of calling `this.toString()`.

This function is provided to give objects a generic `toLocaleString` method, even though not all may use it. In the core language, these built-in objects override `toLocaleString` to provide locale-specific formatting:

- `Array`: `Array.prototype.toLocaleString()`
- `Number`: `Number.prototype.toLocaleString()`
- `Date`: `Date.prototype.toLocaleString()`
- `TypedArray`: `TypedArray.prototype.toLocaleString()`
- `BigInt`: `BigInt.prototype.toLocaleString()`

Examples

Using the base toLocaleString() method

The base `toLocaleString()` method simply calls `toString()`.

```
const obj = {
  toString() {
    return "My Object";
  },
};
console.log(obj.toLocaleString()); // "My Object"
```

Array toLocaleString() override

`Array.prototype.toLocaleString()` is used to print array values as a string by invoking each element's `toLocaleString()` method and joining the results with a locale-specific separator. For example:

```
const testArray = [4, 7, 10];

const euroPrices = testArray.toLocaleString("fr", {
  style: "currency",
  currency: "EUR",
});
// "4,00 €,7,00 €,10,00 €"
```

Date toLocaleString() override

`Date.prototype.toLocaleString()` is used to print out date displays more suitable for specific locales. For example:

```
const testDate = new Date();
// "Fri May 29 2020 18:04:24 GMT+0100 (British Summer Time)"

const deDate = testDate.toLocaleString("de");
// "29.5.2020, 18:04:24"

const frDate = testDate.toLocaleString("fr");
// "29/05/2020, 18:04:24"
```

Number toLocaleString() override

`Number.prototype.toLocaleString()` is used to print out number displays more suitable for specific locales, e.g., with the correct separators. For example:

```
const testNumber = 2901234564;
// "2901234564"

const deNumber = testNumber.toLocaleString("de");
// "2.901.234.564"

const frNumber = testNumber.toLocaleString("fr");
// "2 901 234 564"
```

See also

- `Object.prototype.toString()`

Object.prototype.toString()

The **`toString()`** method of `Object` instances returns a string representing this object. This method is meant to be overridden by derived objects for custom type coercion logic.

Try it

```
const map = new Map();

console.log(map.toString());
// Expected output: "[object Map]"
```

Syntax

```
toString()
```

Parameters

By default `toString()` takes no parameters. However, objects that inherit from `Object` may override it with their own implementations that do take parameters. For example, the `Number.prototype.toString()` and `BigInt.prototype.toString()` methods take an optional `radix` parameter.

Return value

A string representing the object.

Description

JavaScript calls the `toString` method to convert an object to a primitive value. You rarely need to invoke the `toString` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

This method is called in priority by string conversion, but numeric conversion and primitive conversion call `valueOf()` in priority. However, because the base `valueOf()` method returns an object, the `toString()` method is usually called in the end, unless the object overrides `valueOf()`. For example, `+[1]` returns `1`, because its `toString()` method returns `"1"`, which is then converted to a number.

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `toString()` method. When you create a custom object, you can override `toString()` to call a custom method, so that your custom object can be converted to a string value. Alternatively, you can add a `[Symbol.toPrimitive]()` method, which allows even more control over the conversion process, and will always be preferred over `valueOf` or `toString` for any type conversion.

To use the base `Object.prototype.toString()` with an object that has it overridden (or to invoke it on `null` or `undefined`), you need to call `Function.prototype.call()` or `Function.prototype.apply()` on it, passing the object you want to inspect as the first parameter (called `thisArg`).

```
const arr = [1, 2, 3];

arr.toString(); // "1,2,3"
Object.prototype.toString.call(arr); // "[object Array]"
```

`Object.prototype.toString()` returns `"[object Type]"`, where `Type` is the object type. If the object has a `Symbol.toStringTag` property whose value is a string, that value will be used as the `Type`. Many built-in objects, including `Map` and `Symbol`, have a `Symbol.toStringTag`. Some objects predating ES6 do not have `Symbol.toStringTag`, but have a special tag nonetheless. They include (the tag is the same as the type name given below):

- `Array`
- `Function` (anything whose `typeof` returns `"function"`)
- `Error`
- `Boolean`
- `Number`
- `String`
- `Date`
- `RegExp`

The `arguments` object returns `"[object Arguments]"`. Everything else, including user-defined classes, unless with a custom `Symbol.toStringTag`, will return `"[object Object]"`.

`Object.prototype.toString()` invoked on `null` and `undefined` returns `[object Null]` and `[object Undefined]`, respectively.

Examples

Overriding toString for custom objects

You can create a function to be called in place of the default `toString()` method. The `toString()` function you create should return a string value. If it returns an object and the method is called implicitly during type conversion, then its result is ignored and the value of a related method, `valueOf()`, is used instead, or a `TypeError` is thrown if none of these methods return a primitive.

The following code defines a `Dog` class.

```
class Dog {
  constructor(name, breed, color, sex) {
    this.name = name;
    this.breed = breed;
    this.color = color;
    this.sex = sex;
  }
}
```

If you call the `toString()` method, either explicitly or implicitly, on an instance of `Dog`, it returns the default value inherited from `Object`:

```
const theDog = new Dog("Gabby", "Lab", "chocolate", "female");

theDog.toString(); // "[object Object]"
`${theDog}`; // "[object Object]"
```

The following code overrides the default `toString()` method. This method generates a string containing the `name`, `breed`, `color`, and `sex` of the object.

```
class Dog {
  constructor(name, breed, color, sex) {
    this.name = name;
    this.breed = breed;
    this.color = color;
    this.sex = sex;
  }
  toString() {
    return `Dog ${this.name} is a ${this.sex} ${this.color} ${this.breed}`;
  }
}
```

With the preceding code in place, any time an instance of `Dog` is used in a string context, JavaScript automatically calls the `toString()` method.

```
const theDog = new Dog("Gabby", "Lab", "chocolate", "female");

`${theDog}`; // "Dog Gabby is a female chocolate Lab"
```

Using toString() to detect object class

`toString()` can be used with every object and (by default) allows you to get its class.

```
const toString = Object.prototype.toString;

toString.call(new Date()); // [object Date]
toString.call(new String()); // [object String]
// Math has its Symbol.toStringTag
toString.call(Math); // [object Math]

toString.call(undefined); // [object Undefined]
toString.call(null); // [object Null]
```

Using `toString()` in this way is unreliable; objects can change the behavior of `Object.prototype.toString()` by defining a `Symbol.toStringTag` property, leading to unexpected results. For example:

```
const myDate = new Date();
Object.prototype.toString.call(myDate); // [object Date]

myDate[Symbol.toStringTag] = "myDate";
Object.prototype.toString.call(myDate); // [object myDate]

Date.prototype[Symbol.toStringTag] = "prototype polluted";
Object.prototype.toString.call(new Date()); // [object prototype polluted]
```

See also

- Polyfill of `Object.prototype.toString` with `Symbol.toStringTag` support in `core-js`
- `Object.prototype.valueOf()`
- `Number.prototype.toString()`
- `Symbol.toPrimitive`
- `Symbol.toStringTag`

Object.prototype.valueOf()

The **`valueOf()`** method of `Object` instances converts the `this` value to an object. This method is meant to be overridden by derived objects for custom type conversion logic.

Try it

```
function MyNumberType(n) {
  this.number = n;
}

MyNumberType.prototype.valueOf = function () {
  return this.number;
};

const object = new MyNumberType(4);

console.log(object + 3);
// Expected output: 7
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

The `this` value, converted to an object.

> **Note:** In order for `valueOf` to be useful during type conversion, it must return a primitive. Because all primitive types have their own `valueOf()` methods, calling `primitiveValue.valueOf()` generally does not invoke `Object.prototype.valueOf()`.

Description

JavaScript calls the `valueOf` method to convert an object to a primitive value. You rarely need to invoke the `valueOf` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

This method is called in priority by numeric conversion and primitive conversion, but string conversion calls `toString()` in priority, and `toString()` is very likely to return a string value (even for the `Object.prototype.toString()` base implementation), so `valueOf()` is usually not called in this case.

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `toString()` method. The `Object.prototype.valueOf()` base implementation is deliberately useless: by returning an object, its return value will never be used by any primitive conversion algorithm. Many built-in objects override this method to return an appropriate primitive value. When you create a custom object, you can override `valueOf()` to call a custom method, so that your custom object can be converted to a primitive value. Generally, `valueOf()` is used to return a value that is most meaningful for the object — unlike `toString()`, it does not need to be a string. Alternatively, you can add a `[Symbol.toPrimitive]()` method, which allows even more control over the conversion process, and will always be preferred over `valueOf` or `toString` for any type conversion.

Examples

Using valueOf()

The base `valueOf()` method returns the `this` value itself, converted to an object if it isn't already. Therefore its return value will never be used by any primitive conversion algorithm.

```
const obj = { foo: 1 };
console.log(obj.valueOf() === obj); // true

console.log(Object.prototype.valueOf.call("primitive"));
// [String: 'primitive'] (a wrapper object)
```

Overriding valueOf for custom objects

You can create a function to be called in place of the default `valueOf` method. Your function should take no arguments, since it won't be passed any when called during type conversion.

For example, you can add a `valueOf` method to your custom class `Box`.

```
class Box {
  #value;
  constructor(value) {
    this.#value = value;
  }
  valueOf() {
    return this.#value;
  }
}
```

With the preceding code in place, any time an object of type `Box` is used in a context where it is to be represented as a primitive value (but not specifically a string), JavaScript automatically calls the function defined in the preceding code.

```
const box = new Box(123);
console.log(box + 456); // 579
console.log(box == 123); // true
```

An object's `valueOf` method is usually invoked by JavaScript, but you can invoke it yourself as follows:

```
box.valueOf();
```

Using unary plus on objects

Unary plus performs number coercion on its operand, which, for most objects without `[Symbol.toPrimitive]()`, means calling its `valueOf()`. However, if the object doesn't have a custom `valueOf()` method, the base implementation will cause `valueOf()` to be ignored and the return value of `toString()` to be used instead.

```
+new Date(); // the current timestamp; same as new Date().getTime()
+{}; // NaN (toString() returns "[object Object]")
+[]; // 0 (toString() returns an empty string list)
+[1]; // 1 (toString() returns "1")
+[1, 2]; // NaN (toString() returns "1,2")
```

```
+new Set([1]); // NaN (toString() returns "[object Set]")
+{ valueOf: () => 42 }; // 42
```

See also

- `Object.prototype.toString()`
- `parseInt()`
- `Symbol.toPrimitive`

Object.values()

The **`Object.values()`** static method returns an array of a given object's own enumerable string-keyed property values.

Try it

```
const object = {
  a: "some string",
  b: 42,
  c: false,
};

console.log(Object.values(object));
// Expected output: Array ["some string", 42, false]
```

Syntax

```
Object.values(obj)
```

Parameters

`obj`

An object.

Return value

An array containing the given object's own enumerable string-keyed property values.

Description

`Object.values()` returns an array whose elements are values of enumerable string-keyed properties found directly upon `object`. This is similar to accessing the values inside a `for...in` loop, except that a `for...in` loop enumerates properties in the prototype chain as well. The order of the array returned by `Object.values()` is the same as that provided by a `for...in` loop.

If you need the property keys, use `Object.keys()` instead. If you need both the property keys and values, use `Object.entries()` instead.

Examples

Using Object.values()

```
const obj = { foo: "bar", baz: 42 };
console.log(Object.values(obj)); // ['bar', 42]

// Array-like object
const arrayLikeObj1 = { 0: "a", 1: "b", 2: "c" };
console.log(Object.values(arrayLikeObj1)); // ['a', 'b', 'c']

// Array-like object with random key ordering
// When using numeric keys, the values are returned in the keys' numerical order
const arrayLikeObj2 = { 100: "a", 2: "b", 7: "c" };
console.log(Object.values(arrayLikeObj2)); // ['b', 'c', 'a']

// getFoo is a non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
    },
  },
);
myObj.foo = "bar";
console.log(Object.values(myObj)); // ['bar']
```

Using Object.values() on primitives

Non-object arguments are coerced to objects. `undefined` and `null` cannot be coerced to objects and throw a `TypeError` upfront. Only strings may have own enumerable properties, while all other primitives return an empty array.

```
// Strings have indices as enumerable own properties
console.log(Object.values("foo")); // ['f', 'o', 'o']
```

```
// Other primitives except undefined and null have no own properties
console.log(Object.values(100)); // []
```

See also

- Polyfill of `Object.values` in `core-js`
- es-shims polyfill of `Object.values`
- Enumerability and ownership of properties
- `Object.keys()`
- `Object.entries()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`
- `Map.prototype.values()`

parseFloat()

The **`parseFloat()`** function parses a string argument and returns a floating point number.

Try it

```
function circumference(r) {
  return parseFloat(r) * 2.0 * Math.PI;
}

console.log(circumference(4.567));
// Expected output: 28.695307297889173

console.log(circumference("4.567abcdefgh"));
// Expected output: 28.695307297889173

console.log(circumference("abcdefgh"));
// Expected output: NaN
```

Syntax

```
parseFloat(string)
```

Parameters

`string`

The value to parse, coerced to a string. Leading whitespace in this argument is ignored.

Return value

A floating point number parsed from the given `string`, or `NaN` when the first non-whitespace character cannot be converted to a number.

> **Note:** JavaScript does not have the distinction of "floating point numbers" and "integers" on the language level. `parseInt()` and `parseFloat()` only differ in their parsing behavior, but not necessarily their return values. For example, `parseInt("42")` and `parseFloat("42")` would return the same value: a `Number` 42.

Description

The `parseFloat` function converts its first argument to a string, parses that string as a decimal number literal, then returns a number or `NaN`. The number syntax it accepts can be summarized as:

- The characters accepted by `parseFloat()` are plus sign (`+`), minus sign (`-` U+002D HYPHEN-MINUS), decimal digits (`0` – `9`), decimal point (`.`), exponent indicator (`e` or `E`), and the `"Infinity"` literal.
- The `+`/`-` signs can only appear strictly at the beginning of the string, or immediately following the `e`/`E` character. The decimal point can only appear once, and only before the `e`/`E` character. The `e`/`E` character can only appear once, and only if there is at least one digit before it.
- Leading spaces in the argument are trimmed and ignored.
- `parseFloat()` can also parse and return `Infinity` or `-Infinity` if the string starts with `"Infinity"` or `"-Infinity"` preceded by none or more white spaces.
- `parseFloat()` picks the longest substring starting from the beginning that generates a valid number literal. If it encounters an invalid character, it returns the number represented up to that point, ignoring the invalid character and all characters following it.
- If the argument's first character can't start a legal number literal per the syntax above, `parseFloat` returns `NaN`.

Syntax-wise, `parseFloat()` parses a subset of the syntax that the `Number()` function accepts. Namely, `parseFloat()` does not support non-decimal literals with `0x`, `0b`, or `0o` prefixes but supports everything else. However, `parseFloat()` is more lenient than `Number()` because it ignores trailing invalid characters, which would cause `Number()` to return `NaN`.

Similar to number literals and `Number()`, the number returned from `parseFloat()` may not be exactly equal to the number represented by the string, due to floating point range and inaccuracy. For numbers outside the `-1.7976931348623158e+308` – `1.7976931348623158e+308` range (see `Number.MAX_VALUE`), `-Infinity` or `Infinity` is returned.

Examples

Using parseFloat()

The following examples all return `3.14`:

```
parseFloat(3.14);
parseFloat("3.14");
parseFloat("  3.14  ");
parseFloat("314e-2");
parseFloat("0.0314E+2");
parseFloat("3.14some non-digit characters");
parseFloat({
  toString() {
    return "3.14";
  },
});
```

parseFloat() returning NaN

The following example returns `NaN`:

```
parseFloat("FF2");
```

Anecdotally, because the string `NaN` itself is invalid syntax as accepted by `parseFloat()`, passing `"NaN"` returns `NaN` as well.

```
parseFloat("NaN"); // NaN
```

Returning Infinity

Infinity values are returned when the number is outside the double-precision 64-bit IEEE 754-2019 format range:

```
parseFloat("1.7976931348623159e+308"); // Infinity
parseFloat("-1.7976931348623159e+308"); // -Infinity
```

Infinity is also returned when the string starts with `"Infinity"` or `"-Infinity"`:

```
parseFloat("Infinity"); // Infinity
parseFloat("-Infinity"); // -Infinity
```

Interaction with BigInt values

`parseFloat()` does not handle `BigInt` values. It stops at the `n` character, and treats the preceding string as a normal integer, with possible loss of precision. If a BigInt value is passed to `parseFloat()`, it will be converted to a string, and the string will be parsed as a floating-point number, which may result in loss of precision as well.

```
parseFloat(900719925474099267n); // 900719925474099300
parseFloat("900719925474099267n"); // 900719925474099300
```

You should pass the string to the `BigInt()` function instead, without the trailing `n` character.

```
BigInt("900719925474099267");
// 900719925474099267n
```

See also

- `parseInt()`
- `Number.parseFloat()`
- `Number.parseInt()`
- `Number.prototype.toFixed()`

parseInt()

The **`parseInt()`** function parses a string argument and returns an integer of the specified radix (the base in mathematical numeral systems).

Try it

```
console.log(parseInt("123"));
// 123 (default base-10)
console.log(parseInt("123", 10));
// 123 (explicitly specify base-10)
console.log(parseInt("   123 "));
// 123 (whitespace is ignored)
console.log(parseInt("077"));
// 77 (leading zeros are ignored)
console.log(parseInt("1.9"));
// 1 (decimal part is truncated)
console.log(parseInt("ff", 16));
// 255 (lower-case hexadecimal)
console.log(parseInt("0xFF", 16));
// 255 (upper-case hexadecimal with "0x" prefix)
console.log(parseInt("xyz"));
// NaN (input can't be converted to an integer)
```

Syntax

```
parseInt(string)
parseInt(string, radix)
```

Parameters

`string`

> A string starting with an integer. Leading whitespace in this argument is ignored.

`radix` Optional

> An integer between `2` and `36` that represents the *radix* (the base in mathematical numeral systems) of the `string`. It is converted to a 32-bit integer; if it's nonzero and outside the range of [2, 36] after conversion, the function will always return `NaN`. If `0` or not provided, the radix will be inferred based on `string`'s value. Be careful — this does *not* always default to `10`! The description below explains in more detail what happens when `radix` is not provided.

Return value

An integer parsed from the given `string`, or `NaN` when

- the `radix` as a 32-bit integer is smaller than `2` or bigger than `36`, or
- the first non-whitespace character cannot be converted to a number.

> **Note:** JavaScript does not have the distinction of "floating point numbers" and "integers" on the language level. `parseInt()` and `parseFloat()` only differ in their parsing behavior, but not necessarily their return values. For example, `parseInt("42")` and `parseFloat("42")` would return the same value: a `Number` 42.

Description

The `parseInt` function converts its first argument to a string, parses that string, then returns an integer or `NaN`.

If not `NaN`, the return value will be the integer that is the first argument taken as a number in the specified `radix`. (For example, a `radix` of `10` converts from a decimal number, `8` converts from octal, `16` from hexadecimal, and so on.)

The `radix` argument is converted to a number. If it's unprovided, or if the value becomes 0, `NaN` or `Infinity` (`undefined` is coerced to `NaN`), JavaScript assumes the following:

1. If the input `string`, with leading whitespace and possible `+`/`-` signs removed, begins with `0x` or `0X` (a zero, followed by lowercase or uppercase X), `radix` is assumed to be `16` and the rest of the string is parsed as a hexadecimal number.
2. If the input `string` begins with any other value, the radix is `10` (decimal).

> **Note:** Other prefixes like `0b`, which are valid in number literals, are treated as normal digits by `parseInt()`. `parseInt()` does *not* treat strings beginning with a `0` character as octal values either. The only prefix that `parseInt()` recognizes is `0x` or `0X` for hexadecimal values — everything else is parsed as a decimal value if `radix` is missing. `Number()` or `BigInt()` can be used instead to parse these prefixes.

If the radix is `16`, `parseInt()` allows the string to be optionally prefixed by `0x` or `0X` after the optional sign character (`+`/`-`).

If the radix value (coerced if necessary) is not in range [2, 36] (inclusive) `parseInt` returns `NaN`.

For radices above `10`, letters of the English alphabet indicate numerals greater than `9`. For example, for hexadecimal numbers (base `16`), `A` through `F` are used. The letters are case-insensitive.

`parseInt` understands exactly two signs: `+` for positive, and `-` for negative. It is done as an initial step in the parsing after whitespace is removed. If no signs are found, the algorithm moves to the following step; otherwise, it removes the sign and runs the number-parsing on the rest of the string.

If `parseInt` encounters a character in the input string that is not a valid numeral in the specified `radix`, it ignores it and all succeeding characters and returns the integer value parsed up to that point. For example, `parseInt("2", 2)` returns `NaN` because `2` is not a valid numeral in the binary number system. Likewise, although `1e3` technically encodes an integer (and will be correctly parsed to the integer `1000` by `parseFloat()`), `parseInt("1e3", 10)` returns `1`, because `e` is not a valid numeral in base 10. Because `.` is not a numeral either, the return value will always be an integer.

If the first character cannot be converted to a number with the radix in use, `parseInt` returns `NaN`. Leading whitespace is allowed.

For arithmetic purposes, the `NaN` value is not a number in any radix. You can call the `Number.isNaN` function to determine if the result of `parseInt` is `NaN`. If `NaN` is passed on to arithmetic operations, the operation result will also be `NaN`.

Because large numbers use the `e` character in their string representation (e.g., `6.022e23` for 6.022×10^{23}), using `parseInt` to truncate numbers will produce unexpected results when used on very large or very small numbers. `parseInt` should *not* be used as a substitute for `Math.trunc()`.

To convert a number to its string literal in a particular radix, use `thatNumber.toString(radix)`.

Because `parseInt()` returns a number, it may suffer from loss of precision if the integer represented by the string is outside the safe range. The `BigInt()` function supports parsing integers of arbitrary length accurately, by returning a `BigInt`.

Examples

Using parseInt()

The following examples all return `15`:

```
parseInt("0xF", 16);
parseInt("F", 16);
parseInt("17", 8);
parseInt("015", 10);
parseInt("15,123", 10);
parseInt("FXX123", 16);
parseInt("1111", 2);
parseInt("15 * 3", 10);
parseInt("15e2", 10);
parseInt("15px", 10);
parseInt("12", 13);
```

The following examples all return `NaN`:

```
parseInt("Hello", 8); // Not a number at all
parseInt("546", 2); // Digits other than 0 or 1 are invalid for binary radix
```

The following examples all return `-15`:

```
parseInt("-F", 16);
parseInt("-0F", 16);
parseInt("-0XF", 16);
parseInt("-17", 8);
parseInt("-15", 10);
parseInt("-1111", 2);
parseInt("-15e1", 10);
parseInt("-12", 13);
```

The following example returns `224`:

```
parseInt("0e0", 16);
```

`parseInt()` does not handle `BigInt` values. It stops at the `n` character, and treats the preceding string as a normal integer, with possible loss of precision.

```
parseInt("900719925474099267n");
// 900719925474099300
```

You should pass the string to the `BigInt()` function instead, without the trailing `n` character.

```
BigInt("900719925474099267");
// 900719925474099267n
```

`parseInt` doesn't work with numeric separators:

```
parseInt("123_456"); // 123
```

Using parseInt() on non-strings

`parseInt()` can have interesting results when working on non-strings combined with a high radix; for example, `36` (which makes all alphanumeric characters valid numeric digits).

```
parseInt(null, 36); // 1112745: The string "null" is 1112745 in base 36
parseInt(undefined, 36); // 86464843759093: The string "undefined" is 86464843759093 in base 36
```

In general, it's a bad idea to use `parseInt()` on non-strings, especially to use it as a substitution for `Math.trunc()`. It may work on small numbers:

```
parseInt(15.99, 10); // 15
parseInt(-15.1, 10); // -15
```

However, it only happens to work because the string representation of these numbers uses basic fractional notation (`"15.99"`, `"-15.1"`), where `parseInt()` stops at the decimal point. Numbers greater than or equal to 1e+21 or less than or equal to 1e-7 use exponential notation (`"1.5e+22"`, `"1.51e-8"`) in their string representation, and `parseInt()` will stop at the `e` character or decimal point, which always comes after the first digit. This means for large and small numbers, `parseInt()` will return a one-digit integer:

```
parseInt(4.7 * 1e22, 10); // Very large number becomes 4
parseInt(0.00000000000434, 10); // Very small number becomes 4

parseInt(0.0000001, 10); // 1
parseInt(0.000000123, 10); // 1
parseInt(1e-7, 10); // 1
parseInt(100000000000000000000, 10); // 1
parseInt(123000000000000000000000, 10); // 1
parseInt(1e21, 10); // 1
```

See also

- `parseFloat()`
- `Number()` constructor
- `Number.parseFloat()`

- `Number.parseInt()`
- `isNaN()`
- `Number.prototype.toString()`
- `Object.prototype.valueOf()`
- `BigInt()` constructor

Promise

The **`Promise`** object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

To learn about the way promises work and how you can use them, we advise you to read Using promises first.

Description

A `Promise` is a proxy for a value not necessarily known when the promise is created. It allows you to associate handlers with an asynchronous action's eventual success value or failure reason. This lets asynchronous methods return values like synchronous methods: instead of immediately returning the final value, the asynchronous method returns a *promise* to supply the value at some point in the future.

A `Promise` is in one of these states:

- *pending*: initial state, neither fulfilled nor rejected.
- *fulfilled*: meaning that the operation was completed successfully.
- *rejected*: meaning that the operation failed.

The *eventual state* of a pending promise can either be *fulfilled* with a value or *rejected* with a reason (error). When either of these options occurs, the associated handlers queued up by a promise's `then` method are called. If the promise has already been fulfilled or rejected when a corresponding handler is attached, the handler will be called, so there is no race condition between an asynchronous operation completing and its handlers being attached.

A promise is said to be *settled* if it is either fulfilled or rejected, but not pending.

Flowchart showing how the Promise state transitions between pending, fulfilled, and rejected via then/catch handlers. A pending promise can become either fulfilled or rejected. If fulfilled, the "on fulfillment" handler, or first parameter of the then() method, is executed and carries out further asynchronous actions. If rejected, the error handler, either passed as the second parameter of the then() method or as the sole parameter of the catch() method, gets executed.

You will also hear the term *resolved* used with promises — this means that the promise is settled or "locked-in" to match the eventual state of another promise, and further resolving or rejecting it has no effect. The States and fates document from the original Promise proposal contains more details about promise terminology. Colloquially, "resolved" promises are often equivalent to "fulfilled" promises, but as illustrated in "States and fates", resolved promises can be pending or rejected as well. For example:

```
new Promise((resolveOuter) => {
  resolveOuter(
    new Promise((resolveInner) => {
      setTimeout(resolveInner, 1000);
    }),
  );
});
```

This promise is already *resolved* at the time when it's created (because the `resolveOuter` is called synchronously), but it is resolved with another promise, and therefore won't be *fulfilled* until 1 second later, when the inner promise fulfills. In practice, the "resolution" is often done behind the scenes and not observable, and only its fulfillment or rejection are.

> **Note:** Several other languages have mechanisms for lazy evaluation and deferring a computation, which they also call "promises", e.g., Scheme. Promises in JavaScript represent processes that are already happening, which can be chained with callback functions. If you are looking to lazily evaluate an expression, consider using a function with no arguments e.g., `f = () => expression` to create the lazily-evaluated expression, and `f()` to evaluate the expression immediately.

`Promise` itself has no first-class protocol for cancellation, but you may be able to directly cancel the underlying asynchronous operation, typically using `AbortController`.

Chained Promises

The promise methods `then()`, `catch()`, and `finally()` are used to associate further action with a promise that becomes settled. The `then()` method takes up to two arguments; the first argument is a callback function for the fulfilled case of the promise, and the second argument is a callback function for the rejected case. The `catch()` and `finally()` methods call `then()` internally and make error handling less verbose. For example, a `catch()` is really just a `then()` without passing the fulfillment handler. As these methods return promises, they can be chained. For example:

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 300);
});

myPromise
  .then(handleFulfilledA, handleRejectedA)
  .then(handleFulfilledB, handleRejectedB)
  .then(handleFulfilledC, handleRejectedC);
```

We will use the following terminology: *initial promise* is the promise on which `then` is called; *new promise* is the promise returned by `then`. The two callbacks passed to `then` are called *fulfillment handler* and *rejection handler*, respectively.

The settled state of the initial promise determines which handler to execute.

- If the initial promise is fulfilled, the fulfillment handler is called with the fulfillment value.
- If the initial promise is rejected, the rejection handler is called with the rejection reason.

The completion of the handler determines the settled state of the new promise.

- If the handler returns a thenable value, the new promise settles in the same state as the returned value.
- If the handler returns a non-thenable value, the new promise is fulfilled with the returned value.
- If the handler throws an error, the new promise is rejected with the thrown error.
- If the initial promise has no corresponding handler attached, the new promise will settle to the same state as the initial promise — that is, without a rejection handler, a rejected promise stays rejected with the same reason.

For example, in the code above, if `myPromise` rejects, `handleRejectedA` will be called, and if `handleRejectedA` completes normally (without throwing or returning a rejected promise), the promise returned by the first `then` will be fulfilled instead of staying rejected. Therefore, if an error must be handled immediately, but we want to maintain the error state down the chain, we must throw an error of some type in the rejection handler. On the other hand, in the absence of an immediate need, we can leave out error handling until the final `catch()` handler.

```
myPromise
  .then(handleFulfilledA)
  .then(handleFulfilledB)
  .then(handleFulfilledC)
  .catch(handleRejectedAny);
```

Using arrow functions for the callback functions, implementation of the promise chain might look something like this:

```
myPromise
  .then((value) => `${value} and bar`)
  .then((value) => `${value} and bar again`)
  .then((value) => `${value} and again`)
  .then((value) => `${value} and again`)
  .then((value) => {
    console.log(value);
  })
  .catch((err) => {
    console.error(err);
  });
```

> **Note:** For faster execution, all synchronous actions should preferably be done within one handler, otherwise it would take several ticks to execute all handlers in sequence.

JavaScript maintains a job queue. Each time, JavaScript picks a job from the queue and executes it to completion. The jobs are defined by the executor of the `Promise()` constructor, the handlers passed to `then`, or any platform API that returns a promise. The promises in a chain represent the dependency relationship between these jobs. When a promise settles, the respective handlers associated with it are added to the back of the job queue.

A promise can participate in more than one chain. For the following code, the fulfillment of `promiseA` will cause both `handleFulfilled1` and `handleFulfilled2` to be added to the job queue. Because `handleFulfilled1` is registered first, it will be invoked first.

```
const promiseA = new Promise(myExecutorFunc);
const promiseB = promiseA.then(handleFulfilled1, handleRejected1);
const promiseC = promiseA.then(handleFulfilled2, handleRejected2);
```

An action can be assigned to an already settled promise. In this case, the action is added immediately to the back of the job queue and will be performed when all existing jobs are completed. Therefore, an action for an already "settled" promise will occur only after the current synchronous code completes and at least one loop-tick has passed. This guarantees that promise actions are asynchronous.

```
const promiseA = new Promise((resolve, reject) => {
  resolve(777);
});
// At this point, "promiseA" is already settled.
promiseA.then((val) => console.log("asynchronous logging has val:", val));
console.log("immediate logging");

// produces output in this order:
// immediate logging
// asynchronous logging has val: 777
```

Thenables

The JavaScript ecosystem had made multiple Promise implementations long before it became part of the language. Despite being represented differently internally, at the minimum, all Promise-like objects implement the *Thenable* interface. A thenable implements the `.then()` method, which is called with two callbacks: one for when the promise is fulfilled, one for when it's rejected. Promises are thenables as well.

To interoperate with the existing Promise implementations, the language allows using thenables in place of promises. For example, `Promise.resolve` will not only resolve promises, but also trace thenables.

```
// This is not a Promises/A+ compliant thenable! It calls onFulfilled
// synchronously. For demonstration only.
const thenable = {
```

```
  then(onFulfilled, onRejected) {
    onFulfilled({
      // The thenable is fulfilled with another thenable
      then(onFulfilled, onRejected) {
        onFulfilled(42);
      },
    });
  },
};

Promise.resolve(thenable); // A promise fulfilled with 42
```

The `then()` method is responsible for scheduling the execution of the provided `onFulfilled` and `onRejected` callbacks. Its semantics, including error handling and asynchronicity, are precisely defined in the Promises/A+ specification, and we shall not repeat them here. It's very rare that you need to implement a thenable yourself; even if you are not using native promises, you would probably be using a Promise library such as Bluebird.

Promise concurrency

The `Promise` class offers four main static methods to facilitate async task concurrency:

`Promise.all()`

> Fulfills when **all** of the promises fulfill; rejects when **any** of the promises rejects.

`Promise.allSettled()`

> Fulfills when **all** promises settle.

`Promise.any()`

> Fulfills when **any** of the promises fulfills; rejects when **all** of the promises reject.

`Promise.race()`

> Settles when **any** of the promises settles. In other words, fulfills when any of the promises fulfills; rejects when any of the promises rejects.

All these methods take an iterable of promises (thenables, to be exact) and return a new promise. They all support subclassing, which means they can be called on subclasses of `Promise`, and the result will be a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters. The subclass must also have a `resolve` static method that can be called like `Promise.resolve()` to resolve values to promises.

There are two other convenience static methods: `Promise.allKeyed()` and `Promise.allSettledKeyed()`, that behave like `Promise.all()` and `Promise.allSettled()`, but take *objects* of promises and return promises that fulfill with *objects* of the same shape. By working with objects instead of arrays, you can associate results with semantically meaningful keys, instead of arbitrary array ordering which can be difficult to maintain.

These methods attach handlers to each input promise using `then()`. Even when the resulting promise has settled early (such as when one input in `Promise.race()` settles), the other handlers are not removed. Repeatedly passing the same pending promise to concurrency methods can accumulate handlers even when those handlers are never used:

```
const pendingPromise = new Promise(() => {});

for (let i = 0; i < 1000; i++) {
  await Promise.race([Promise.resolve(0), pendingPromise]);
}
// All tasks have completed, but pendingPromise retains the
// handlers attached by all 1000 races.
```

Promises do not provide a way to unsubscribe these handlers; they remain attached while the input promise is pending and reachable. Where possible, cancel the underlying operation by using an `AbortSignal` when the pending promise is no longer useful.

Note that JavaScript is single-threaded by nature, so at a given instant, only one task will be executing, although control can shift between different promises, making execution of the promises appear concurrent. Parallel execution in JavaScript can only be achieved through worker threads.

Constructor

`Promise()`

> Creates a new `Promise` object. The constructor is primarily used to wrap functions that do not already support promises.

Static properties

`Promise[Symbol.species]`

> Returns the constructor used to construct return values from promise methods.

Static methods

`Promise.all()`

> Takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises fulfill (including when an empty iterable is passed), with an array of the fulfillment values. It rejects when any of the input's promises reject, with this first rejection reason.

`Promise.allKeyed()`

Like `Promise.all()`, except that it takes an object of promises and returns a promise that fulfills with an object of the same shape, allowing you to associate results with semantically meaningful keys.

`Promise.allSettled()`

Takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises settle (including when an empty iterable is passed), with an array of objects that describe the outcome of each promise.

`Promise.allSettledKeyed()`

Like `Promise.allSettled()`, except that it takes an object of promises and returns a promise that fulfills with an object of the same shape, allowing you to associate results with semantically meaningful keys.

`Promise.any()`

Takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when any of the input's promises fulfill, with this first fulfillment value. It rejects when all of the input's promises reject (including when an empty iterable is passed), with an `AggregateError` containing an array of rejection reasons.

`Promise.race()`

Takes an iterable of promises as input and returns a single `Promise`. This returned promise settles with the eventual state of the first promise that settles.

`Promise.reject()`

Returns a new `Promise` object that is rejected with the given reason.

`Promise.resolve()`

Returns a `Promise` object that is resolved with the given value. If the value is a thenable (i.e., has a `then` method), the returned promise will "follow" that thenable, adopting its eventual state; otherwise, the returned promise will be fulfilled with the value.

`Promise.try()`

Takes a callback of any kind (returns or throws, synchronously or asynchronously) and resolves its result to a `Promise`.

`Promise.withResolvers()`

Returns an object containing a new `Promise` object and two functions to resolve or reject it, corresponding to the two parameters passed to the executor of the `Promise()` constructor.

Instance properties

These properties are defined on `Promise.prototype` and shared by all `Promise` instances.

`Promise.prototype.constructor`

The constructor function that created the instance object. For `Promise` instances, the initial value is the `Promise` constructor.

`Promise.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Promise"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Promise.prototype.catch()`

Appends a rejection handler callback to the promise, and returns a new promise resolving to the return value of the callback if it is called, or to its original fulfillment value if the promise is instead fulfilled.

`Promise.prototype.finally()`

Appends a handler to the promise, and returns a new promise that is resolved when the original promise is resolved. The handler is called when the promise is settled, whether fulfilled or rejected.

`Promise.prototype.then()`

Appends fulfillment and rejection handlers to the promise, and returns a new promise resolving to the return value of the called handler, or to its original settled value if the promise was not handled (i.e., if the relevant handler `onFulfilled` or `onRejected` is not a function).

Examples

Basic Example

In this example, we use `setTimeout(...)` to simulate async code. In reality, you will probably be using something like XHR or an HTML API.

```
const myFirstPromise = new Promise((resolve, reject) => {
  // We call resolve(...) when what we were doing asynchronously
  // was successful, and reject(...) when it failed.
  setTimeout(() => {
    resolve("Success!"); // Yay! Everything went well!
  }, 250);
});

myFirstPromise.then((successMessage) => {
```

```
  // successMessage is whatever we passed in the resolve(...) function above.
  // It doesn't have to be a string, but if it is only a succeed message, it probably will be.
  console.log(`Yay! ${successMessage}`);
});
```

Example with diverse situations

This example shows diverse techniques for using Promise capabilities and diverse situations that can occur. To understand this, start by scrolling to the bottom of the code block, and examine the promise chain. Upon provision of an initial promise, a chain of promises can follow. The chain is composed of `.then()` calls, and typically (but not necessarily) has a single `.catch()` at the end, optionally followed by `.finally()`. In this example, the promise chain is initiated by a custom-written `new Promise()` construct; but in actual practice, promise chains more typically start with an API function (written by someone else) that returns a promise.

The example function `tetheredGetNumber()` shows that a promise generator will utilize `reject()` while setting up an asynchronous call, or within the call-back, or both. The function `promiseGetWord()` illustrates how an API function might generate and return a promise in a self-contained manner.

Note that the function `troubleWithGetNumber()` ends with a `throw`. That is forced because a promise chain goes through all the `.then()` promises, even after an error, and without the `throw`, the error would seem "fixed". This is a hassle, and for this reason, it is common to omit `onRejected` throughout the chain of `.then()` promises, and just have a single `onRejected` in the final `catch()`.

This code can be run under Node.js. Comprehension is enhanced by seeing the errors actually occur. To force more errors, change the `threshold` values.

```
// To experiment with error handling, "threshold" values cause errors randomly
const THRESHOLD_A = 8; // can use zero 0 to guarantee error

function tetheredGetNumber(resolve, reject) {
  setTimeout(() => {
    const randomInt = Date.now();
    const value = randomInt % 10;
    if (value < THRESHOLD_A) {
      resolve(value);
    } else {
      reject(new RangeError(`Too large: ${value}`));
    }
  }, 500);
}

function determineParity(value) {
  const isOdd = value % 2 === 1;
  return { value, isOdd };
}

function troubleWithGetNumber(reason) {
  const err = new Error("Trouble getting number", { cause: reason });
  console.error(err);
  throw err;
}

function promiseGetWord(parityInfo) {
  return new Promise((resolve, reject) => {
    const { value, isOdd } = parityInfo;
    if (value >= THRESHOLD_A - 1) {
      reject(new RangeError(`Still too large: ${value}`));
    } else {
      parityInfo.wordEvenOdd = isOdd ? "odd" : "even";
      resolve(parityInfo);
    }
  });
}

new Promise(tetheredGetNumber)
  .then(determineParity, troubleWithGetNumber)
  .then(promiseGetWord)
  .then((info) => {
    console.log(`Got: ${info.value}, ${info.wordEvenOdd}`);
    return info;
  })
  .catch((reason) => {
    if (reason.cause) {
      console.error("Had previously handled error");
    } else {
      console.error(`Trouble with promiseGetWord(): ${reason}`);
    }
  })
  .finally((info) => console.log("All done"));
```

Advanced Example

This small example shows the mechanism of a `Promise`. The `testPromise()` method is called each time the `<button>` is clicked. It creates a promise that will be fulfilled, using `setTimeout()`, to the promise count (number starting from 1) every 1-3 seconds, at random. The `Promise()` constructor is used to create the promise.

The fulfillment of the promise is logged, via a fulfill callback set using `p1.then()`. A few logs show how the synchronous part of the method is decoupled from the asynchronous completion of the promise.

By clicking the button several times in a short amount of time, you'll even see the different promises being fulfilled one after another.

HTML

```
<button id="make-promise">Make a promise!</button>
<div id="log"></div>
```

JavaScript

```
"use strict";

let promiseCount = 0;

function testPromise() {
  const thisPromiseCount = ++promiseCount;
  const log = document.getElementById("log");
  // begin
  log.insertAdjacentHTML("beforeend", `${thisPromiseCount}) Started<br>`);
  // We make a new promise: we promise a numeric count of this promise,
  // starting from 1 (after waiting 3s)
  const p1 = new Promise((resolve, reject) => {
    // The executor function is called with the ability
    // to resolve or reject the promise
    log.insertAdjacentHTML(
      "beforeend",
      `${thisPromiseCount}) Promise constructor<br>`,
    );
    // This is only an example to create asynchronism
    setTimeout(
      () => {
        // We fulfill the promise
        resolve(thisPromiseCount);
      },
      Math.random() * 2000 + 1000,
    );
  });

  // We define what to do when the promise is resolved with the then() call,
  // and what to do when the promise is rejected with the catch() call
  p1.then((val) => {
    // Log the fulfillment value
    log.insertAdjacentHTML("beforeend", `${val}) Promise fulfilled<br>`);
  }).catch((reason) => {
    // Log the rejection reason
    console.log(`Handle rejected promise (${reason}) here.`);
  });
  // end
  log.insertAdjacentHTML("beforeend", `${thisPromiseCount}) Promise made<br>`);
}

const btn = document.getElementById("make-promise");
btn.addEventListener("click", testPromise);
```

Result

Loading an image with XHR

Another example using `Promise` and `XMLHttpRequest` to load an image is shown below. Each step is commented on and allows you to follow the Promise and XHR architecture closely.

```
function imgLoad(url) {
  // Create new promise with the Promise() constructor;
  // This has as its argument a function with two parameters, resolve and reject
  return new Promise((resolve, reject) => {
    // XHR to load an image
    const request = new XMLHttpRequest();
    request.open("GET", url);
    request.responseType = "blob";
    // When the request loads, check whether it was successful
    request.onload = () => {
      if (request.status === 200) {
        // If successful, resolve the promise by passing back the request response
        resolve(request.response);
      } else {
        // If it fails, reject the promise with an error message
        reject(
          Error(
            `Image didn't load successfully; error code: + ${request.statusText}`,
          ),
        );
      }
    };
    // Handle network errors
    request.onerror = () => reject(new Error("There was a network error."));
    // Send the request
    request.send();
  });
```

```
}

// Get a reference to the body element, and create a new image object
const body = document.querySelector("body");
const myImage = new Image();
const imgUrl =
  "https://mdn.github.io/shared-assets/images/examples/round-balloon.png";

// Call the function with the URL we want to load, then chain the
// promise then() method with two callbacks
imgLoad(imgUrl).then(
  (response) => {
    // The first runs when the promise resolves, with the request.response
    // specified within the resolve() method.
    const imageURL = URL.createObjectURL(response);
    myImage.src = imageURL;
    body.appendChild(myImage);
  },
  (error) => {
    // The second runs when the promise
    // is rejected, and logs the Error specified with the reject() method.
    console.log(error);
  },
);
```

Incumbent settings object tracking

A settings object is an environment that provides additional information when JavaScript code is running. This includes the realm and module map, as well as HTML specific information such as the origin. The incumbent settings object is tracked in order to ensure that the browser knows which one to use for a given piece of user code.

To better picture this, we can take a closer look at how the realm might be an issue. A **realm** can be roughly thought of as the global object. What is unique about realms is that they hold all of the necessary information to run JavaScript code. This includes objects like `Array` and `Error`. Each settings object has its own "copy" of these and they are not shared. That can cause some unexpected behavior in relation to promises. In order to get around this, we track something called the **incumbent settings object**. This represents information specific to the context of the user code responsible for a certain function call.

To illustrate this a bit further we can take a look at how an `<iframe>` embedded in a document communicates with its host. Since all web APIs are aware of the incumbent settings object, the following will work in all browsers:

```
<!doctype html>
<iframe></iframe>
<!-- we have a realm here -->
<script>
  // we have a realm here as well
  const bound = frames[0].postMessage.bind(frames[0], "some data", "*");
  // bound is a built-in function — there is no user
  // code on the stack, so which realm do we use?
  setTimeout(bound);
  // this still works, because we use the youngest
  // realm (the incumbent) on the stack
</script>
```

The same concept applies to promises. If we modify the above example a little bit, we get this:

```
<!doctype html>
<iframe></iframe>
<!-- we have a realm here -->
<script>
  // we have a realm here as well
  const bound = frames[0].postMessage.bind(frames[0], "some data", "*");
  // bound is a built in function — there is no user
  // code on the stack — which realm do we use?
  Promise.resolve(undefined).then(bound);
  // this still works, because we use the youngest
  // realm (the incumbent) on the stack
</script>
```

If we change this so that the `<iframe>` in the document is listening to post messages, we can observe the effect of the incumbent settings object:

```
<!-- y.html -->
<!doctype html>
<iframe src="https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/x.html"></iframe>
<script>
  const bound = frames[0].postMessage.bind(frames[0], "some data", "*");
  Promise.resolve(undefined).then(bound);
</script>
```

```
<!-- x.html -->
<!doctype html>
<script>
  window.addEventListener("message", (event) => {
    document.querySelector("#text").textContent = "hello";
    // this code will only run in browsers that track the incumbent settings object
```

```
    console.log(event);
  });
</script>
```

In the above example, the inner text of the `<iframe>` will be updated only if the incumbent settings object is tracked. This is because without tracking the incumbent, we may end up using the wrong environment to send the message.

> **Note:** Currently, incumbent realm tracking is fully implemented in Firefox, and has partial implementations in Chrome and Safari.

See also

- Polyfill of `Promise` in `core-js`
- Using promises guide
- Promises/A+ specification
- JavaScript Promises: an introduction on web.dev (2013)
- Callbacks, Promises, and Coroutines: Asynchronous Programming Patterns in JavaScript slide show by Domenic Denicola (2011)

Promise.all()

The **`Promise.all()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises fulfill (including when an empty iterable is passed), with an array of the fulfillment values. It rejects when any of the input's promises rejects, with this first rejection reason.

Try it

```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "foo");
});

Promise.all([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});
// Expected output: Array [3, 42, "foo"]
```

Syntax

```
Promise.all(iterable)
```

Parameters

`iterable`

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

Return value

A `Promise` that is:

- **Already fulfilled**, if the `iterable` passed is empty.
- **Asynchronously fulfilled**, when all the promises in the given `iterable` fulfill. The fulfillment value is an array of fulfillment values, in the order of the promises passed, regardless of completion order. If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.
- **Asynchronously rejected**, when any of the promises in the given `iterable` rejects. The rejection reason is the rejection reason of the first promise that was rejected.

Description

The `Promise.all()` method is one of the promise concurrency methods. It can be useful for aggregating the results of multiple promises. It is typically used when there are multiple related asynchronous tasks that the overall code relies on to work successfully — all of whom we want to fulfill before the code execution continues.

`Promise.all()` will reject immediately upon **any** of the input promises rejecting. In comparison, the promise returned by `Promise.allSettled()` will wait for all input promises to complete, regardless of whether or not one rejects. Use `allSettled()` if you need the final result of every promise in the input iterable.

Like other promise combinators, `Promise.all()` immediately marks all promises as "handled" when it is called (by calling their `.then()` methods). Subsequent rejections after the first rejection will be ignored, and will not trigger any `unhandledrejection` events.

Rejecting the returned promise does not cancel the remaining operations or unsubscribe the handlers attached to their promises. Repeatedly passing a long-lived pending promise to `Promise.all()` can accumulate handlers on that promise even when another input rejects each time.

Examples

Using Promise.all()

`Promise.all` waits for all fulfillments (or the first rejection).

```
const p1 = Promise.resolve(3);
const p2 = 1337;
const p3 = new Promise((resolve, reject) => {
  setTimeout(() => {
```

```
    resolve("foo");
  }, 100);
});

Promise.all([p1, p2, p3]).then((values) => {
  console.log(values); // [3, 1337, "foo"]
});
```

If the `iterable` contains non-promise values, they will be ignored, but still counted in the returned promise array value (if the promise is fulfilled):

```
// All values are non-promises, so the returned promise gets fulfilled
const p = Promise.all([1, 2, 3]);
// The only input promise is already fulfilled,
// so the returned promise gets fulfilled
const p2 = Promise.all([1, 2, 3, Promise.resolve(444)]);
// One (and the only) input promise is rejected,
// so the returned promise gets rejected
const p3 = Promise.all([1, 2, 3, Promise.reject(new Error("bad"))]);

// Using setTimeout, we can execute code after the queue is empty
setTimeout(() => {
  console.log(p);
  console.log(p2);
  console.log(p3);
});

// Logs:
// Promise { <state>: "fulfilled", <value>: Array[3] }
// Promise { <state>: "fulfilled", <value>: Array[4] }
// Promise { <state>: "rejected", <reason>: Error: bad }
```

Destructuring the result

You will find destructuring very useful if you are batching together a known number of tasks.

```
// With then()
Promise.all([p1, p2, p3]).then(([a, b, c]) => {
  console.log(a, b, c); // 3 1337 "foo"
});

// With await
const [a, b, c] = await Promise.all([p1, p2, p3]);
```

Be careful: if the original promises and the result variables' order don't match, you may run into subtle bugs. The `Promise.allKeyed()` method solves exactly this problem.

Asynchronicity or synchronicity of Promise.all

This following example demonstrates the asynchronicity of `Promise.all` when a non-empty `iterable` is passed:

```
// Passing an array of promises that are already resolved,
// to trigger Promise.all as soon as possible
const resolvedPromisesArray = [Promise.resolve(33), Promise.resolve(44)];

const p = Promise.all(resolvedPromisesArray);
// Immediately logging the value of p
console.log(p);

// Using setTimeout, we can execute code after the queue is empty
setTimeout(() => {
  console.log("the queue is now empty");
  console.log(p);
});

// Logs, in order:
// Promise { <state>: "pending" }
// the queue is now empty
// Promise { <state>: "fulfilled", <value>: Array[2] }
```

The same thing happens if `Promise.all` rejects:

```
const mixedPromisesArray = [
  Promise.resolve(33),
  Promise.reject(new Error("bad")),
];
const p = Promise.all(mixedPromisesArray);
console.log(p);
setTimeout(() => {
  console.log("the queue is now empty");
  console.log(p);
});

// Logs:
// Promise { <state>: "pending" }
```

```
// the queue is now empty
// Promise { <state>: "rejected", <reason>: Error: bad }
```

`Promise.all` resolves synchronously if and only if the `iterable` passed is empty:

```
const p = Promise.all([]); // Will be immediately resolved
const p2 = Promise.all([1337, "hi"]); // Non-promise values are ignored, but the evaluation is done asynchronously
console.log(p);
console.log(p2);
setTimeout(() => {
  console.log("the queue is now empty");
  console.log(p2);
});

// Logs:
// Promise { <state>: "fulfilled", <value>: Array[0] }
// Promise { <state>: "pending" }
// the queue is now empty
// Promise { <state>: "fulfilled", <value>: Array[2] }
```

Using Promise.all() with async functions

Within async functions, it's very common to "over-await" your code. For example, given the following functions:

```
function promptForDishChoice() {
  return new Promise((resolve, reject) => {
    const dialog = document.createElement("dialog");
    dialog.innerHTML = `
<form method="dialog">
  <p>What would you like to eat?</p>
  <select>
    <option value="pizza">Pizza</option>
    <option value="pasta">Pasta</option>
    <option value="salad">Salad</option>
  </select>
  <menu>
    <li><button value="cancel">Cancel</button></li>
    <li><button type="submit" value="ok">OK</button></li>
  </menu>
</form>
    `;
    dialog.addEventListener("close", () => {
      if (dialog.returnValue === "ok") {
        resolve(dialog.querySelector("select").value);
      } else {
        reject(new Error("User canceled dialog"));
      }
    });
    document.body.appendChild(dialog);
    dialog.showModal();
  });
}

async function fetchPrices() {
  const response = await fetch("/prices");
  return await response.json();
}
```

You may write a function like this:

```
async function getPrice() {
  const choice = await promptForDishChoice();
  const prices = await fetchPrices();
  return prices[choice];
}
```

However, note that the execution of `promptForDishChoice` and `fetchPrices` don't depend on the result of each other. While the user is choosing their dish, it's fine for the prices to be fetched in the background, but in the code above, the `await` operator causes the async function to pause until the choice is made, and then again until the prices are fetched. We can use `Promise.all` to run them concurrently, so that the user doesn't have to wait for the prices to be fetched before the result is given:

```
async function getPrice() {
  const [choice, prices] = await Promise.all([
    promptForDishChoice(),
    fetchPrices(),
  ]);
  return prices[choice];
}
```

`Promise.all` is the best choice of concurrency method here, because error handling is intuitive — if any of the promises reject, the result is no longer available, so the whole `await` expression throws.

`Promise.all` accepts an iterable of promises, so if you are using it to run several async functions concurrently, you need to call the async functions and use the returned promises. Directly passing the functions to `Promise.all` does not work, since they are not promises.

```
async function getPrice() {
  const [choice, prices] = await Promise.all([
    promptForDishChoice,
    fetchPrices,
  ]);
  // `choice` and `prices` are still the original async functions;
  // Promise.all() does nothing to non-promises
}
```

Note that the following, despite other undesirable aspects (like creating more variables and bug-prone error handling), also achieves concurrency:

```
async function getPrice() {
  // Fire all async operations upfront
  const choicePromise = promptForDishChoice();
  const pricesPromise = fetchPrices();

  // Wait for each promise (all in-progress async operations keep running)
  const choice = await choicePromise;
  const prices = await pricesPromise;
}
```

Promise.all fail-fast behavior

`Promise.all` is rejected if any of the elements are rejected. For example, if you pass in four promises that resolve after a timeout and one promise that rejects immediately, then `Promise.all` will reject immediately.

```
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("one"), 1000);
});
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("two"), 2000);
});
const p3 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("three"), 3000);
});
const p4 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("four"), 4000);
});
const p5 = new Promise((resolve, reject) => {
  reject(new Error("reject"));
});

// Using .catch:
Promise.all([p1, p2, p3, p4, p5])
  .then((values) => {
    console.log(values);
  })
  .catch((error) => {
    console.error(error.message);
  });

// Logs:
// "reject"
```

It is possible to change this behavior by handling possible rejections:

```
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("p1_delayed_resolution"), 1000);
});

const p2 = new Promise((resolve, reject) => {
  reject(new Error("p2_immediate_rejection"));
});

Promise.all([p1.catch((error) => error), p2.catch((error) => error)]).then(
  (values) => {
    console.log(values[0]); // "p1_delayed_resolution"
    console.error(values[1]); // "Error: p2_immediate_rejection"
  },
);
```

See also

- `Promise`
- `Promise.allKeyed()`
- `Promise.allSettled()`
- `Promise.any()`
- `Promise.race()`

Promise.allKeyed()

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`Promise.allKeyed()`** static method is like `Promise.all()`, except that instead of using arrays/iterables as input/output, it uses objects. It takes an object where each own key is associated with a promise, and returns a single `Promise`. This returned promise fulfills when all of the input's promises fulfill, with an object of the same keys mapped to the corresponding fulfillment values. It rejects when any of the input's promises rejects, with this first rejection reason.

Compared to `Promise.all()`, `Promise.allKeyed()` allows you to associate results with semantically meaningful keys, instead of arbitrary array ordering which can be difficult to maintain.

Syntax

```
Promise.allKeyed(object)
```

Parameters

`object`

An object. All of its own enumerable properties, whether the key is a string or a symbol, should have `Promise` values. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

Return value

A `Promise` that is:

- **Already fulfilled**, if the `object` passed has no own enumerable properties.
- **Asynchronously fulfilled**, when all the promises in the given `object` fulfill. The fulfillment value is an object of fulfillment values, with the same keys in the same order as the given `object`, regardless of completion order. If the `object` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.
- **Asynchronously rejected**, when any of the promises in the given `object` rejects. The rejection reason is the rejection reason of the first promise that was rejected.

Description

The `Promise.allKeyed()` method is one of the promise concurrency methods. It performs the same kind of task as `Promise.all()`. However, a lot of the times, you don't already have an array of promises and instead just have a few ad-hoc operations to batch up, so you put them in an array and then immediately destructure:

```
const [resultA, resultB, resultC] = await Promise.all([getA(), getB(), getC()]);
```

The problem with this is that you need to maintain the consistency of ordering on both sides: if you accidentally write `[resultA, resultC, resultB]`, your code will break.

The keyed method mitigates the problem by associating each async operation with a semantic key:

```
const {
  a: resultA,
  b: resultB,
  c: resultC,
} = await Promise.allKeyed({
  a: getA(),
  b: getB(),
  c: getC(),
});
```

This way, the ordering no longer matters, and any name mismatch is local: accidentally writing `b: resultC` is now much easier to catch!

Examples

Using Promise.allKeyed()

The `Promise.allKeyed()` method takes an object and processes all of its own enumerable properties.

```
function delayed(value, timeout) {
  return new Promise((res) => setTimeout(() => res(value), timeout));
}

const sym = Symbol("example");

const promises = {
  a: delayed("a", 500),
  // Symbol properties are processed
  [sym]: delayed("symbol", 300),
  // Nested properties are not processed; this whole object is treated as
  // an already-resolved value and returned as-is
  nested: {
    b: delayed("b", 100),
  },
};

const result = await Promise.allKeyed(promises);
```

```
console.log(result);
// {
//   a: "a",
//   [sym]: "symbol",
//   nested: {
//     b: <Promise>,
//   },
// }
```

For more examples related to concurrency behavior common to `Promise.all()` and `Promise.allKeyed()`, see `Promise.all()`.

See also

- Polyfill of `Promise.allKeyed` in `core-js`
- es-shims polyfill of `Promise.allKeyed`
- `Promise`
- `Promise.allSettledKeyed()`
- `Promise.all()`

Promise.allSettled()

The **`Promise.allSettled()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises settle (including when an empty iterable is passed), with an array of objects that describe the outcome of each promise.

Try it

```
const promise1 = Promise.resolve(3);
const promise2 = new Promise((resolve, reject) =>
  setTimeout(reject, 100, "foo"),
);
const promises = [promise1, promise2];

Promise.allSettled(promises).then((results) =>
  results.forEach((result) => console.log(result.status)),
);

// Expected output:
// "fulfilled"
// "rejected"
```

Syntax

```
Promise.allSettled(iterable)
```

Parameters

`iterable`

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

Return value

A `Promise` that is:

- **Already fulfilled**, if the `iterable` passed is empty.
- **Asynchronously fulfilled**, when all promises in the given `iterable` have settled (either fulfilled or rejected). The fulfillment value is an array of objects, each describing the outcome of one promise in the `iterable`, in the order of the promises passed, regardless of completion order. Each outcome object has the following properties:

  `status`

  A string, either `"fulfilled"` or `"rejected"`, indicating the eventual state of the promise.

  `value`

  Only present if `status` is `"fulfilled"`. The value that the promise was fulfilled with.

  `reason`

  Only present if `status` is `"rejected"`. The reason that the promise was rejected with.

  If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.

Description

The `Promise.allSettled()` method is one of the promise concurrency methods. `Promise.allSettled()` is typically used when you have multiple asynchronous tasks that are not dependent on one another to complete successfully, or you'd always like to know the result of each promise.

In comparison, the Promise returned by `Promise.all()` may be more appropriate if the tasks are dependent on each other, or if you'd like to immediately reject upon any of them rejecting.

Examples

Using Promise.allSettled()

```
Promise.allSettled([
  Promise.resolve(33),
  new Promise((resolve) => setTimeout(() => resolve(66), 0)),
  99,
  Promise.reject(new Error("an error")),
]).then((values) => console.log(values));

// [
//   { status: 'fulfilled', value: 33 },
//   { status: 'fulfilled', value: 66 },
//   { status: 'fulfilled', value: 99 },
//   { status: 'rejected', reason: Error: an error }
// ]
```

You may also destructure the results to have one variable per settling outcome.

```
const [status1, status2] = await Promise.allSettled([
  Promise.resolve(33),
  new Promise((resolve) => setTimeout(() => resolve(66), 0)),
]);

// status1 = { status: 'fulfilled', value: 33 }
// status2 = { status: 'fulfilled', value: 66 }
```

If you destructure the result, you must keep the result variables in the same order as the input promises; a mismatch may cause subtle bugs. The `Promise.allSettledKeyed()` method avoids this problem by associating each input and output with a key.

See also

- Polyfill of `Promise.allSettled` in `core-js`
- es-shims polyfill of `Promise.allSettled`
- Using promises guide
- Graceful asynchronous programming with promises
- `Promise`
- `Promise.all()`
- `Promise.allSettledKeyed()`
- `Promise.any()`
- `Promise.race()`

Promise.allSettledKeyed()

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`Promise.allSettledKeyed()`** static method is like `Promise.allSettled()`, except that instead of using arrays/iterables as input/output, it uses objects. It takes an object where each own key is associated with a promise, and returns a single `Promise`. This returned promise fulfills when all of the input's promises settle, with an object of the same keys mapped to objects that describe the outcome of the corresponding promise.

Compared to `Promise.allSettled()`, `Promise.allSettledKeyed()` allows you to associate results with semantically meaningful keys, instead of arbitrary array ordering which can be difficult to maintain.

Syntax

```
Promise.allSettledKeyed(object)
```

Parameters

`object`

> An object. All of its own enumerable properties, whether the key is a string or a symbol, should have `Promise` values. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

Return value

A `Promise` that is:

- **Already fulfilled**, if the `object` passed has no own enumerable properties.
- **Asynchronously fulfilled**, when all promises in the given `object` have settled (either fulfilled or rejected). The fulfillment value is an object, with the same keys in the same order as the given `object`, and each property's value being an object describing the outcome of the corresponding promise in `object`, regardless of completion order. Each outcome object has the following properties:

  `status`

  > A string, either `"fulfilled"` or `"rejected"`, indicating the eventual state of the promise.

  `value`

Only present if `status` is `"fulfilled"`. The value that the promise was fulfilled with.

`reason`

Only present if `status` is `"rejected"`. The reason that the promise was rejected with.

If the `object` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.

Description

The `Promise.allSettledKeyed()` method is one of the promise concurrency methods. It performs the same kind of task as `Promise.allSettled()`. `Promise.allSettledKeyed()` is preferred when you don't already have an array of promises and/or you are immediately destructuring the results; see `Promise.allKeyed()` for more description.

Examples

Using Promise.allSettledKeyed()

The `Promise.allSettledKeyed()` method takes an object and processes all of its own enumerable properties.

```
function delayed(value, timeout) {
  return new Promise((res) => setTimeout(() => res(value), timeout));
}

const sym = Symbol("example");

const promises = {
  a: delayed("a", 500),
  // Symbol properties are processed
  [sym]: delayed("symbol", 300),
  // Nested properties are not processed; this whole object is treated as
  // an already-resolved value and returned as-is
  nested: {
    b: delayed("b", 100),
  },
};

const result = await Promise.allSettledKeyed(promises);
console.log(result);
// {
//   a: { status: "fulfilled", value: "a" },
//   [sym]: { status: "fulfilled", value: "symbol" },
//   nested: { status: "fulfilled", value: { b: <Promise> } },
// }
```

For more examples related to concurrency behavior common to `Promise.allSettled()` and `Promise.allSettledKeyed()`, see `Promise.allSettled()`.

See also

- Polyfill of `Promise.allSettledKeyed` in `core-js`
- es-shims polyfill of `Promise.allSettledKeyed`
- `Promise`
- `Promise.allKeyed()`
- `Promise.allSettled()`

Promise.any()

The **`Promise.any()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when any of the input's promises fulfills, with this first fulfillment value. It rejects when all of the input's promises reject (including when an empty iterable is passed), with an `AggregateError` containing an array of rejection reasons.

Try it

```
const promise1 = Promise.reject(new Error("error"));
const promise2 = new Promise((resolve) => setTimeout(resolve, 100, "quick"));
const promise3 = new Promise((resolve) => setTimeout(resolve, 500, "slow"));

const promises = [promise1, promise2, promise3];

Promise.any(promises).then((value) => console.log(value));

// Expected output: "quick"
```

Syntax

```
Promise.any(iterable)
```

Parameters

`iterable`

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

Return value

A `Promise` that is:

- **Already rejected**, if the `iterable` passed is empty.
- **Asynchronously fulfilled**, when any of the promises in the given `iterable` fulfills. The fulfillment value is the fulfillment value of the first promise that was fulfilled.
- **Asynchronously rejected**, when all of the promises in the given `iterable` reject. The rejection reason is an `AggregateError` containing an array of rejection reasons in its `errors` property. The errors are in the order of the promises passed, regardless of completion order. If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) rejected.

Description

The `Promise.any()` method is one of the promise concurrency methods. This method is useful for returning the first promise that fulfills. It short-circuits after a promise fulfills, so it does not wait for the other promises to complete once it finds one.

Unlike `Promise.all()`, which returns an *array* of fulfillment values, we only get one fulfillment value (assuming at least one promise fulfills). This can be beneficial if we need only one promise to fulfill but we do not care which one does. Note another difference: this method rejects upon receiving an *empty iterable*, since, truthfully, the iterable contains no items that fulfill. You may compare `Promise.any()` and `Promise.all()` with `Array.prototype.some()` and `Array.prototype.every()`.

Also, unlike `Promise.race()`, which returns the first *settled* value (either fulfillment or rejection), this method returns the first *fulfilled* value. This method ignores all rejected promises up until the first promise that fulfills.

Like other promise combinators, `Promise.any()` immediately marks all promises as "handled" when it is called (by calling their `.then()` methods). Subsequent rejections after the first fulfillment will be ignored, and will not trigger any `unhandledrejection` events.

Fulfilling the returned promise does not cancel the remaining operations or unsubscribe the handlers attached to their promises. Repeatedly passing a long-lived pending promise to `Promise.any()` can accumulate handlers on that promise even when another input fulfills each time.

Examples

Using Promise.any()

`Promise.any()` fulfills with the first promise to fulfill, even if a promise rejects first. This is in contrast to `Promise.race()`, which fulfills or rejects with the first promise to settle.

```
const pErr = new Promise((resolve, reject) => {
  reject(new Error("Always fails"));
});

const pSlow = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "Done eventually");
});

const pFast = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "Done quick");
});

Promise.any([pErr, pSlow, pFast]).then((value) => {
  console.log(value);
  // pFast fulfills first
});
// Logs:
// Done quick
```

Rejections with AggregateError

`Promise.any()` rejects with an `AggregateError` if no promise fulfills.

```
const failure = new Promise((resolve, reject) => {
  reject(new Error("Always fails"));
});

Promise.any([failure]).catch((err) => {
  console.log(err);
});
// AggregateError: No Promise in Promise.any was resolved
```

Displaying the first image loaded

In this example, we have a function that fetches an image and returns a blob. We use `Promise.any()` to fetch a couple of images and display the first one available (i.e., whose promise has resolved).

```
async function fetchAndDecode(url, description) {
  const res = await fetch(url);
  if (!res.ok) {
    throw new Error(`HTTP error! status: ${res.status}`);
  }
  const data = await res.blob();
  return [data, description];
}

const coffee = fetchAndDecode("coffee.jpg", "Coffee");
```

```
const tea = fetchAndDecode("tea.jpg", "Tea");

Promise.any([coffee, tea])
  .then(([blob, description]) => {
    const objectURL = URL.createObjectURL(blob);
    const image = document.createElement("img");
    image.src = objectURL;
    image.alt = description;
    document.body.appendChild(image);
  })
  .catch((e) => {
    console.error(e);
  });
```

See also

- Polyfill of `Promise.any` in `core-js`
- es-shims polyfill of `Promise.any`
- `Promise`
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.race()`

Promise.prototype.catch()

The **`catch()`** method of `Promise` instances schedules a function to be called when the promise is rejected. It immediately returns another `Promise` object, allowing you to chain calls to other promise methods. It is a shortcut for `then(undefined, onRejected)`.

Try it

```
const promise = new Promise((resolve, reject) => {
  throw new Error("Uh-oh!");
});

promise.catch((error) => {
  console.error(error);
});
// Expected output: Error: Uh-oh!
```

Syntax

```
promiseInstance.catch(onRejected)
```

Parameters

`onRejected`

A function to asynchronously execute when this promise becomes rejected. Its return value becomes the fulfillment value of the promise returned by `catch()`. The function is called with the following arguments:

`reason`

The value that the promise was rejected with.

Return value

Returns a new `Promise`. This new promise is always pending when returned, regardless of the current promise's status. If `onRejected` is called, the returned promise will resolve based on the return value of this call, or reject with the thrown error from this call. If the current promise fulfills, `onRejected` is not called and the returned promise fulfills to the same value.

Description

The `catch` method is used for error handling in promise composition. Since it returns a `Promise`, it can be chained in the same way as its sister method, `then()`.

If a promise becomes rejected, and there are no rejection handlers to call (a handler can be attached through any of `then()`, `catch()`, or `finally()`), then the rejection event is surfaced by the host. In the browser, this results in an `unhandledrejection` event. If a handler is attached to a rejected promise whose rejection has already caused an unhandled rejection event, then another `rejectionhandled` event is fired.

`catch()` internally calls `then()` on the object upon which it was called, passing `undefined` and `onRejected` as arguments. The value of that call is directly returned. This is observable if you wrap the methods.

```
// overriding original Promise.prototype.then/catch just to add some logs
((Promise) => {
  const originalThen = Promise.prototype.then;
  const originalCatch = Promise.prototype.catch;

  Promise.prototype.then = function (...args) {
    console.log("Called .then on %o with arguments: %o", this, args);
    return originalThen.apply(this, args);
  };
  Promise.prototype.catch = function (...args) {
    console.error("Called .catch on %o with arguments: %o", this, args);
    return originalCatch.apply(this, args);
```

```
  };
})(Promise);

// calling catch on an already resolved promise
Promise.resolve().catch(function XXX() {});

// Logs:
// Called .catch on Promise{} with arguments: Arguments{1} [0: function XXX()]
// Called .then on Promise{} with arguments: Arguments{2} [0: undefined, 1: function XXX()]
```

This means that passing `undefined` still causes the returned promise to be rejected, and you have to pass a function to prevent the final promise from being rejected.

Because `catch()` just calls `then()`, it supports subclassing.

> **Note:** The examples below are throwing instances of `Error`. As with synchronous `throw` statements, this is considered a good practice; otherwise, the part doing the catching would have to perform checks to see if the argument was a string or an error, and you might lose valuable information such as stack traces.

Examples

Using and chaining the catch() method

```
const p1 = new Promise((resolve, reject) => {
  resolve("Success");
});

p1.then((value) => {
  console.log(value); // "Success!"
  throw new Error("oh, no!");
})
  .catch((e) => {
    console.error(e.message); // "oh, no!"
  })
  .then(
    () => console.log("after a catch the chain is restored"), // "after a catch the chain is restored"
    () => console.log("Not fired due to the catch"),
  );

// The following behaves the same as above
p1.then((value) => {
  console.log(value); // "Success!"
  return Promise.reject(new Error("oh, no!"));
})
  .catch((e) => {
    console.error(e); // Error: oh, no!
  })
  .then(
    () => console.log("after a catch the chain is restored"), // "after a catch the chain is restored"
    () => console.log("Not fired due to the catch"),
  );
```

Gotchas when throwing errors

Throwing an error will call the `catch()` method most of the time:

```
const p1 = new Promise((resolve, reject) => {
  throw new Error("Uh-oh!");
});

p1.catch((e) => {
  console.error(e); // "Uh-oh!"
});
```

Errors thrown inside asynchronous functions will act like uncaught errors:

```
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    throw new Error("Uncaught Exception!");
  }, 1000);
});

p2.catch((e) => {
  console.error(e); // This is never called
});
```

Errors thrown after `resolve` is called will be silenced:

```
const p3 = new Promise((resolve, reject) => {
  resolve();
  throw new Error("Silenced Exception!");
});

p3.catch((e) => {
```

```
  console.error(e); // This is never called
});
```

catch() is not called if the promise is fulfilled

```
// Create a promise which would not call onReject
const p1 = Promise.resolve("calling next");

const p2 = p1.catch((reason) => {
  // This is never called
  console.error("catch p1!");
  console.error(reason);
});

p2.then(
  (value) => {
    console.log("next promise's onFulfilled");
    console.log(value); // calling next
  },
  (reason) => {
    console.log("next promise's onRejected");
    console.log(reason);
  },
);
```

See also

- `Promise`
- `Promise.prototype.then()`
- `Promise.prototype.finally()`

Promise.prototype.finally()

The **`finally()`** method of `Promise` instances schedules a function to be called when the promise is settled (either fulfilled or rejected). It immediately returns another `Promise` object, allowing you to chain calls to other promise methods.

Like the `finally` block, this method is usually intended for cleanup actions, regardless of the promise's outcome. It lets you avoid duplicating code in both the promise's `then()` and `catch()` handlers.

Try it

```
function checkMail() {
  return new Promise((resolve, reject) => {
    if (Math.random() > 0.5) {
      resolve("Mail has arrived");
    } else {
      reject(new Error("Failed to arrive"));
    }
  });
}

checkMail()
  .then((mail) => {
    console.log(mail);
  })
  .catch((err) => {
    console.error(err);
  })
  .finally(() => {
    console.log("Experiment completed");
  });
```

Syntax

```
promiseInstance.finally(onFinally)
```

Parameters

`onFinally`

> A function to asynchronously execute when this promise becomes settled. If the function returns a promise, the resulting promise will wait for that promise to settle before continuing. If the returned promise is rejected, the resulting promise is rejected with the same reason. Any other returned value, or the fulfilled value of the returned promise, is ignored.

Return value

Returns a new `Promise` immediately. This new promise is always pending when returned, regardless of the current promise's status. If `onFinally` throws an error or returns a rejected promise, the new promise will reject with that value. Otherwise, the new promise will settle with the same state as the current promise.

Description

The `finally()` method can be useful if you want to do some processing or cleanup once the promise is settled, regardless of its outcome.

The `finally()` method is very similar to calling `then(onFinally, onFinally)`. However, there are a couple of differences:

- When creating a function inline, you can pass it once, instead of being forced to either declare it twice, or create a variable for it.
- The `onFinally` callback does not receive any argument. This use case is for precisely when you *do not care* about the rejection reason or the fulfillment value, and so there's no need to provide it.
- A `finally()` call is usually transparent and reflects the eventual state of the original promise. So for example:
  - Unlike `Promise.resolve(2).then(() => 77, () => 77)`, which returns a promise eventually fulfilled with the value `77`, `Promise.resolve(2).finally(() => 77)` returns a promise eventually fulfilled with the value `2`.
  - Similarly, unlike `Promise.reject(3).then(() => 88, () => 88)`, which returns a promise eventually fulfilled with the value `88`, `Promise.reject(3).finally(() => 88)` returns a promise eventually rejected with the reason `3`.

> **Note:** A `throw` (or returning a rejected promise) in the `finally` callback still rejects the returned promise. For example, both `Promise.reject(3).finally(() => { throw 99; })` and `Promise.reject(3).finally(() => Promise.reject(99))` reject the returned promise with the reason `99`.

Like `catch()`, `finally()` internally calls the `then` method on the object upon which it was called. If `onFinally` is not a function, `then()` is called with `onFinally` as both arguments — which, for `Promise.prototype.then()`, means that no useful handler is attached. Otherwise, `then()` is called with two internally created functions, which behave like the following:

> **Warning:** This is only for demonstration purposes and is not a polyfill.

```
promise.then(
  (value) => Promise.resolve(onFinally()).then(() => value),
  (reason) =>
    Promise.resolve(onFinally()).then(() => {
      throw reason;
    }),
);
```

Because `finally()` calls `then()`, it supports subclassing. Moreover, notice the `Promise.resolve()` call above — in reality, `onFinally()`'s return value is resolved using the same algorithm as `Promise.resolve()`, but the actual constructor used to construct the resolved promise will be the subclass. `finally()` gets this constructor through `promise.constructor[Symbol.species]`.

Examples

Using finally()

```
let isLoading = true;

fetch(myRequest)
  .then((response) => {
    const contentType = response.headers.get("content-type");
    if (contentType && contentType.includes("application/json")) {
      return response.json();
    }
    throw new TypeError("Oops, we haven't got JSON!");
  })
  .then((json) => {
    /* process your JSON further */
  })
  .catch((error) => {
    console.error(error); // this line can also throw, e.g. when console = {}
  })
  .finally(() => {
    isLoading = false;
  });
```

See also

- Polyfill of `Promise.prototype.finally` in `core-js`
- es-shims polyfill of `Promise.prototype.finally`
- `Promise`
- `Promise.prototype.then()`
- `Promise.prototype.catch()`

Promise() constructor

The **`Promise()`** constructor creates `Promise` objects. It is primarily used to wrap callback-based APIs that do not already support promises.

Try it

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 300);
});

promise1.then((value) => {
  console.log(value);
```

```
  // Expected output: "foo"
});

console.log(promise1);
// Expected output: [object Promise]
```

Syntax

```
new Promise(executor)
```

> **Note:** `Promise()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`executor`

> A `Function` to be executed by the constructor. It receives two functions as parameters: `resolveFunc` and `rejectFunc`. Any errors thrown in the `executor` will cause the promise to be rejected, and the return value will be neglected. The semantics of `executor` are detailed below.

Return value

When called via `new`, the `Promise` constructor returns a promise object. The promise object will become *resolved* when either of the functions `resolveFunc` or `rejectFunc` are invoked. Note that if you call `resolveFunc` and pass another promise object as an argument, the initial promise can be said to be "resolved", but still not "settled". See the Promise description for more explanation.

Description

Traditionally (before promises), asynchronous tasks were designed as callbacks.

```
readFile("./data.txt", (error, result) => {
  // This callback will be called when the task is done, with the
  // final `error` or `result`. Any operation dependent on the
  // result must be defined within this callback.
});
// Code here is immediately executed after the `readFile` request
// is fired. It does not wait for the callback to be called, hence
// making `readFile` "asynchronous".
```

To take advantage of the readability improvement and language features offered by promises, the `Promise()` constructor allows one to transform the callback-based API to a promise-based one.

> **Note:** If your task is already promise-based, you likely do not need the `Promise()` constructor.

The `executor` is custom code that ties an outcome in a callback to a promise. You, the programmer, write the `executor`. Its signature is expected to be:

```
function executor(resolveFunc, rejectFunc) {
  // Typically, some asynchronous operation that accepts a callback,
  // like the `readFile` function above
}
```

`resolveFunc` and `rejectFunc` are also functions, and you can give them whatever actual names you want. Their signatures are simple: they accept a single parameter of any type.

```
resolveFunc(value); // call on resolved
rejectFunc(reason); // call on rejected
```

The `value` parameter passed to `resolveFunc` can be another promise object, in which case the newly constructed promise's state will be "locked in" to the promise passed (as part of the resolution promise). The `rejectFunc` has semantics close to the `throw` statement, so `reason` is typically an `Error` instance. If either `value` or `reason` is omitted, the promise is fulfilled/rejected with `undefined`.

The `executor`'s completion state has limited effect on the promise's state:

- The `executor` return value is ignored. `return` statements within the `executor` merely impact control flow and alter whether a part of the function is executed, but do not have any impact on the promise's fulfillment value. If `executor` exits and it's impossible for `resolveFunc` or `rejectFunc` to be called in the future (for example, there are no async tasks scheduled), then the promise remains pending forever.
- If an error is thrown in the `executor`, the promise is rejected, unless `resolveFunc` or `rejectFunc` has already been called.

> **Note:** The existence of pending promises does not prevent the program from exiting. If the event loop is empty, the program exits despite any pending promises (because those are necessarily forever-pending).

Here's a summary of the typical flow:

1. At the time when the constructor generates the new `Promise` object, it also generates a corresponding pair of functions for `resolveFunc` and `rejectFunc`; these are "tethered" to the `Promise` object.

2. `executor` typically wraps some asynchronous operation which provides a callback-based API. The callback (the one passed to the original callback-based API) is defined within the `executor` code, so it has access to the `resolveFunc` and `rejectFunc`.
3. The `executor` is called synchronously (as soon as the `Promise` is constructed) with the `resolveFunc` and `rejectFunc` functions as arguments.
4. The code within the `executor` has the opportunity to perform some operation. The eventual completion of the asynchronous task is communicated with the promise instance via the side effect caused by `resolveFunc` or `rejectFunc`. The side effect is that the `Promise` object becomes "resolved".
    - If `resolveFunc` is called first, the value passed will be resolved. The promise may stay pending (in case another thenable is passed), become fulfilled (in most cases where a non-thenable value is passed), or become rejected (in case of an invalid resolution value).
    - If `rejectFunc` is called first, the promise instantly becomes rejected.
    - Once one of the resolving functions (`resolveFunc` or `rejectFunc`) is called, the promise stays resolved. Only the first call to `resolveFunc` or `rejectFunc` affects the promise's eventual state, and subsequent calls to either function can neither change the fulfillment value/rejection reason nor toggle its eventual state from "fulfilled" to "rejected" or opposite.
    - If `executor` exits by throwing an error, then the promise is rejected. However, the error is ignored if one of the resolving functions has already been called (so that the promise is already resolved).
    - Resolving the promise does not necessarily cause the promise to become fulfilled or rejected (i.e., settled). The promise may still be pending because it's resolved with another thenable, but its eventual state will match that of the resolved thenable.
5. Once the promise settles, it (asynchronously) invokes any further handlers associated through `then()`, `catch()`, or `finally()`. The eventual fulfillment value or rejection reason is passed to the invocation of fulfillment and rejection handlers as an input parameter (see Chained Promises).

For example, the callback-based `readFile` API above can be transformed into a promise-based one.

```
const readFilePromise = (path) =>
  new Promise((resolve, reject) => {
    readFile(path, (error, result) => {
      if (error) {
        reject(error);
      } else {
        resolve(result);
      }
    });
  });

readFilePromise("./data.txt")
  .then((result) => console.log(result))
  .catch((error) => console.error("Failed to read data"));
```

The `resolve` and `reject` callbacks are only available within the scope of the executor function, which means you can't access them after the promise is constructed. If you want to construct the promise before deciding how to resolve it, you can use the `Promise.withResolvers()` method instead, which exposes the `resolve` and `reject` functions.

The resolve function

The `resolve` function has the following behaviors:

- If it's called with the same value as the newly created promise (the promise it's "tethered to"), the promise is rejected with a `TypeError`.
- If it's called with a non-thenable value (a primitive, or an object whose `then` property is not callable, including when the property is not present), the promise is immediately fulfilled with that value.
- If it's called with a thenable value (including another `Promise` instance), then the thenable's `then` method is saved and called in the future (it's always called asynchronously). The `then` method will be called with two callbacks, which are two new functions with the exact same behaviors as the `resolveFunc` and `rejectFunc` passed to the `executor` function. If calling the `then` method throws, then the current promise is rejected with the thrown error.

In the last case, it means code like:

```
new Promise((resolve, reject) => {
  resolve(thenable);
});
```

Is roughly equivalent to:

```
new Promise((resolve, reject) => {
  try {
    thenable.then(
      (value) => resolve(value),
      (reason) => reject(reason),
    );
  } catch (e) {
    reject(e);
  }
});
```

Except that in the `resolve(thenable)` case:

1. `resolve` is called synchronously, so that calling `resolve` or `reject` again has no effect, even when the handlers attached through `anotherPromise.then()` are not called yet.
2. The `then` method is called asynchronously, so that the promise will never be instantly resolved if a thenable is passed.

Because `resolve` is called again with whatever `thenable.then()` passes to it as `value`, the resolver function is able to flatten nested thenables, where a thenable calls its `onFulfilled` handler with another thenable. The effect is that the fulfillment handler of a real promise will never receive a thenable as its fulfillment value.

Examples

Turning a callback-based API into a promise-based one

To provide a function with promise functionality, have it return a promise by calling the `resolve` and `reject` functions at the correct times.

```
function myAsyncFunction(url) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.open("GET", url);
    xhr.onload = () => resolve(xhr.responseText);
    xhr.onerror = () => reject(xhr.statusText);
    xhr.send();
  });
}
```

Effect of calling resolveFunc

Calling `resolveFunc` causes the promise to become resolved, so that calling `resolveFunc` or `rejectFunc` again has no effect. However, the promise may be in any of the states: pending, fulfilled, or rejected.

This `pendingResolved` promise is resolved the time it's created, because it has already been "locked in" to match the eventual state of the inner promise, and calling `resolveOuter` or `rejectOuter` or throwing an error later in the executor has no effect on its eventual state. However, the inner promise is still pending until 100ms later, so the outer promise is also pending:

```
const pendingResolved = new Promise((resolveOuter, rejectOuter) => {
  resolveOuter(
    new Promise((resolveInner) => {
      setTimeout(() => {
        resolveInner("inner");
      }, 100);
    }),
  );
});
```

This `fulfilledResolved` promise becomes fulfilled the moment it's resolved, because it's resolved with a non-thenable value. However, when it's created, it's unresolved, because neither `resolve` nor `reject` has been called yet. An unresolved promise is necessarily pending:

```
const fulfilledResolved = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("outer");
  }, 100);
});
```

Calling `rejectFunc` obviously causes the promise to reject. However, there are also two ways to cause the promise to instantly become rejected even when the `resolveFunc` callback is called.

```
// 1. Resolving with the promise itself
const rejectedResolved1 = new Promise((resolve) => {
  // Note: resolve has to be called asynchronously,
  // so that the rejectedResolved1 variable is initialized
  setTimeout(() => resolve(rejectedResolved1)); // TypeError: Chaining cycle detected for promise #<Promise>
});

// 2. Resolving with an object which throws when accessing the `then` property
const rejectedResolved2 = new Promise((resolve) => {
  resolve({
    get then() {
      throw new Error("Can't get then property");
    },
  });
});
```

See also

- Polyfill of `Promise` in `core-js`
- Using promises guide
- `Promise.withResolvers()`

Promise.race()

The **`Promise.race()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise settles with the eventual state of the first promise that settles.

Try it

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "two");
});
```

```
Promise.race([promise1, promise2]).then((value) => {
  console.log(value);
  // Both resolve, but promise2 is faster
});
// Expected output: "two"
```

Syntax

```
Promise.race(iterable)
```

Parameters

`iterable`

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

Return value

A `Promise` that **asynchronously settles** with the eventual state of the first promise in the `iterable` to settle. In other words, it fulfills if the first promise to settle is fulfilled, and rejects if the first promise to settle is rejected. The returned promise remains pending forever if the `iterable` passed is empty. If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) settled.

Description

The `Promise.race()` method is one of the promise concurrency methods. It's useful when you want the first async task to complete, but do not care about its eventual state (i.e., it can either succeed or fail).

If the iterable contains one or more non-promise values and/or an already settled promise, then `Promise.race()` will settle to the first of these values found in the iterable.

Like other promise combinators, `Promise.race()` immediately marks all promises as "handled" when it is called (by calling their `.then()` methods). Subsequent rejections after the first settlement will be ignored, and will not trigger any `unhandledrejection` events.

Settling the returned promise does not cancel the losing operations or unsubscribe the handlers attached to their promises. If you repeatedly race a long-lived pending promise against short-lived promises, handlers can accumulate on the pending promise even after each race settles.

Examples

Using Promise.race()

This example shows how `Promise.race()` can be used to race several timers implemented with `setTimeout()`. The timer with the shortest time always wins the race and becomes the resulting promise's state.

```
function sleep(time, value, state) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (state === "fulfill") {
        resolve(value);
      } else {
        reject(new Error(value));
      }
    }, time);
  });
}

const p1 = sleep(500, "one", "fulfill");
const p2 = sleep(100, "two", "fulfill");

Promise.race([p1, p2]).then((value) => {
  console.log(value); // "two"
  // Both fulfill, but p2 is faster
});

const p3 = sleep(100, "three", "fulfill");
const p4 = sleep(500, "four", "reject");

Promise.race([p3, p4]).then(
  (value) => {
    console.log(value); // "three"
    // p3 is faster, so it fulfills
  },
  (error) => {
    // Not called
  },
);

const p5 = sleep(500, "five", "fulfill");
const p6 = sleep(100, "six", "reject");

Promise.race([p5, p6]).then(
  (value) => {
    // Not called
  },
  (error) => {
```

```
    console.error(error.message); // "six"
    // p6 is faster, so it rejects
  },
);
```

Asynchronicity of Promise.race

This following example demonstrates the asynchronicity of `Promise.race`. Unlike other promise concurrency methods, `Promise.race` is always asynchronous: it never settles synchronously, even when the `iterable` is empty.

```
// Passing an array of promises that are already resolved,
// to trigger Promise.race as soon as possible
const resolvedPromisesArray = [Promise.resolve(33), Promise.resolve(44)];

const p = Promise.race(resolvedPromisesArray);
// Immediately logging the value of p
console.log(p);

// Using setTimeout, we can execute code after the stack is empty
setTimeout(() => {
  console.log("the stack is now empty");
  console.log(p);
});

// Logs, in order:
// Promise { <state>: "pending" }
// the stack is now empty
// Promise { <state>: "fulfilled", <value>: 33 }
```

An empty iterable causes the returned promise to be forever pending:

```
const foreverPendingPromise = Promise.race([]);
console.log(foreverPendingPromise);
setTimeout(() => {
  console.log("the stack is now empty");
  console.log(foreverPendingPromise);
});

// Logs, in order:
// Promise { <state>: "pending" }
// the stack is now empty
// Promise { <state>: "pending" }
```

If the iterable contains one or more non-promise value and/or an already settled promise, then `Promise.race` will settle to the first of these values found in the array:

```
const foreverPendingPromise = Promise.race([]);
const alreadyFulfilledProm = Promise.resolve(100);

const arr = [foreverPendingPromise, alreadyFulfilledProm, "non-Promise value"];
const arr2 = [foreverPendingPromise, "non-Promise value", Promise.resolve(100)];
const p = Promise.race(arr);
const p2 = Promise.race(arr2);

console.log(p);
console.log(p2);
setTimeout(() => {
  console.log("the stack is now empty");
  console.log(p);
  console.log(p2);
});

// Logs, in order:
// Promise { <state>: "pending" }
// Promise { <state>: "pending" }
// the stack is now empty
// Promise { <state>: "fulfilled", <value>: 100 }
// Promise { <state>: "fulfilled", <value>: "non-Promise value" }
```

Using Promise.race() to implement request timeout

You can race a potentially long-lasting request with a timer that rejects, so that when the time limit has elapsed, the resulting promise automatically rejects.

```
const data = Promise.race([
  fetch("/api"),
  new Promise((resolve, reject) => {
    // Reject after 5 seconds
    setTimeout(() => reject(new Error("Request timed out")), 5000);
  }),
])
  .then((res) => res.json())
  .catch((err) => displayError(err));
```

If the `data` promise fulfills, it will contain the data fetched from `/api`. `Promise.race` will capture and discard the settlement results of the losing promises, so the `"Request timed out"` rejection will not bubble up as unhandled. Otherwise, if `fetch` remains pending for 5 seconds and loses the race with the `setTimeout` timer, the final promise will reject.

One promise finishing does not automatically cancel the other; the other's result is simply ignored. This does not pose problems in this small example, but it keeps resources such as network connections and timers alive for longer than necessary. To release resources early, abort the fetch if the timeout wins, or clear the timeout if the fetch wins. Whenever possible—including fetch—prefer using the `AbortController` API instead.

Using Promise.race() to detect the status of a promise

Because `Promise.race()` resolves to the first non-pending promise in the iterable, we can check a promise's state, including if it's pending. This example is adapted from `promise-status-async`.

```
function promiseState(promise) {
  const pendingState = { status: "pending" };

  return Promise.race([promise, pendingState]).then(
    (value) =>
      value === pendingState ? value : { status: "fulfilled", value },
    (reason) => ({ status: "rejected", reason }),
  );
}
```

In this function, if `promise` is pending, the second value, `pendingState`, which is a non-promise, becomes the result of the race; otherwise, if `promise` is already settled, we may know its state through the `onFulfilled` and `onRejected` handlers. For example:

```
const p1 = new Promise((res) => setTimeout(() => res(100), 100));
const p2 = new Promise((res) => setTimeout(() => res(200), 200));
const p3 = new Promise((res, rej) =>
  setTimeout(() => rej(new Error("failed")), 100),
);

async function getStates() {
  console.log(await promiseState(p1));
  console.log(await promiseState(p2));
  console.log(await promiseState(p3));
}

console.log("Immediately after initiation:");
getStates();
setTimeout(() => {
  console.log("After waiting for 100ms:");
  getStates();
}, 100);

// Logs:
// Immediately after initiation:
// { status: 'pending' }
// { status: 'pending' }
// { status: 'pending' }
// After waiting for 100ms:
// { status: 'fulfilled', value: 100 }
// { status: 'pending' }
// { status: 'rejected', reason: Error: failed }
```

> **Note:** The `promiseState` function still runs asynchronously, because there is no way to synchronously get a promise's value (i.e., without `then()` or `await`), even when it is already settled. However, `promiseState()` always fulfills within one tick and never actually waits for any promise's settlement.

Comparison with Promise.any()

`Promise.race` takes the first settled `Promise`.

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(reject, 100, "two");
});

Promise.race([promise1, promise2])
  .then((value) => {
    console.log("succeeded with value:", value);
  })
  .catch((reason) => {
    // Only promise1 is fulfilled, but promise2 is faster
    console.error("failed with reason:", reason);
  });
// failed with reason: two
```

`Promise.any` takes the first fulfilled `Promise`.

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(reject, 100, "two");
});

Promise.any([promise1, promise2])
  .then((value) => {
    // Only promise1 is fulfilled, even though promise2 settled sooner
    console.log("succeeded with value:", value);
  })
  .catch((reason) => {
    console.error("failed with reason:", reason);
  });
// succeeded with value: one
```

See also

- `Promise`
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.any()`

Promise.reject()

The **`Promise.reject()`** static method returns a `Promise` object that is rejected with a given reason.

Try it

```
function resolved(result) {
  console.log("Resolved");
}

function rejected(result) {
  console.error(result);
}

Promise.reject(new Error("fail")).then(resolved, rejected);
// Expected output: Error: fail
```

Syntax

```
Promise.reject(reason)
```

Parameters

`reason`

Reason why this `Promise` rejected.

Return value

A `Promise` that is rejected with the given reason.

Description

The static `Promise.reject` function returns a `Promise` that is rejected. For debugging purposes and selective error catching, it is useful to make `reason` an `instanceof Error`.

`Promise.reject()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will be a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters. `Promise.reject()` is essentially a shorthand for `new Promise((resolve, reject) => reject(reason))`.

Unlike `Promise.resolve()`, `Promise.reject()` always wraps `reason` in a new `Promise` object, even when `reason` is already a `Promise`.

Examples

Using the static Promise.reject() method

```
Promise.reject(new Error("fail")).then(
  () => {
    // not called
  },
  (error) => {
    console.error(error); // Stacktrace
  },
);
```

Rejecting with a promise

Unlike `Promise.resolve`, the `Promise.reject` method does not reuse existing `Promise` instances. It always returns a new `Promise` instance that wraps `reason`.

```
const p = Promise.resolve(1);
const rejected = Promise.reject(p);
console.log(rejected === p); // false
rejected.catch((v) => {
  console.log(v === p); // true
});
```

Calling reject() on a non-Promise constructor

`Promise.reject()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes it `console.log` as `reject`:

```
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the
    // native promise's, but Promise.reject() calls them in the same way.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static reject = Promise.reject;
}

const p = NotPromise.reject("foo"); // Logs "Rejected foo"
// p is a NotPromise instance
```

See also

- `Promise`

Promise.resolve()

The **`Promise.resolve()`** static method "resolves" a given value to a `Promise`. If the value is a promise, that promise is returned; if the value is a thenable, `Promise.resolve()` will call the `then()` method with two callbacks it prepared; otherwise the returned promise will be fulfilled with the value.

This function flattens nested layers of promise-like objects (e.g., a promise that fulfills to a promise that fulfills to something) into a single layer — a promise that fulfills to a non-thenable value.

Try it

```
const promise1 = Promise.resolve(123);

promise1.then((value) => {
  console.log(value);
  // Expected output: 123
});
```

Syntax

```
Promise.resolve(value)
```

Parameters

`value`

Argument to be resolved by this `Promise`. Can also be a `Promise` or a thenable to resolve.

Return value

A `Promise` that is resolved with the given value, or the promise passed as value, if the value was a promise object. A resolved promise can be in any of the states — fulfilled, rejected, or pending. For example, resolving a rejected promise will still result in a rejected promise.

Description

`Promise.resolve()` *resolves* a promise, which is not the same as fulfilling or rejecting the promise. See Promise description for definitions of the terminology. In brief, `Promise.resolve()` returns a promise whose eventual state depends on another promise, thenable object, or other value.

> **Note:** If evaluating the `value` expression may synchronously throw an error, this error won't be caught and wrapped in a rejected promise by `Promise.resolve()`. Consider using `Promise.try(() => value)` in this case.

`Promise.resolve()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will be a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

`Promise.resolve()` special-cases native `Promise` instances. If `value` belongs to `Promise` or a subclass, and `value.constructor === Promise`, then `value` is directly returned by `Promise.resolve()`, without creating a new `Promise` instance. Otherwise, `Promise.resolve()` is essentially a shorthand for `new Promise((resolve) => resolve(value))`.

The bulk of the resolving logic is actually implemented by the `resolve` function passed by the `Promise()` constructor. In summary:

- If a non-thenable value is passed, the returned promise is already fulfilled with that value.
- If a thenable is passed, the returned promise will adopt the state of that thenable by calling the `then` method and passing a pair of resolving functions as arguments. (But because native promises directly pass through `Promise.resolve()` without creating a wrapper, the `then` method is not called on native promises.) If the `resolve` function receives another thenable object, it will be resolved again, so that the eventual fulfillment value of the promise will never be thenable.

Examples

Using the static Promise.resolve method

```
Promise.resolve("Success").then(
  (value) => {
    console.log(value); // "Success"
  },
  (reason) => {
    // not called
  },
);
```

Resolving an array

```
const p = Promise.resolve([1, 2, 3]);
p.then((v) => {
  console.log(v[0]); // 1
});
```

Resolving another Promise

`Promise.resolve()` reuses existing `Promise` instances. If it's resolving a native promise, it returns the same promise instance without creating a wrapper.

```
const original = Promise.resolve(33);
const cast = Promise.resolve(original);
cast.then((value) => {
  console.log(`value: ${value}`);
});
console.log(`original === cast ? ${original === cast}`);

// Logs, in order:
// original === cast ? true
// value: 33
```

The inverted order of the logs is due to the fact that the `then` handlers are called asynchronously. See the `then()` reference for more information.

Resolving thenables and throwing Errors

```
// Resolving a thenable object
const p1 = Promise.resolve({
  then(onFulfill, onReject) {
    onFulfill("fulfilled!");
  },
});
console.log(p1 instanceof Promise); // true, object casted to a Promise

p1.then(
  (v) => {
    console.log(v); // "fulfilled!"
  },
  (e) => {
    // not called
  },
);

// Thenable throws
// Promise rejects
const p2 = Promise.resolve({
  then() {
    throw new TypeError("Throwing");
  },
});
p2.then(
  (v) => {
    // not called
  },
  (e) => {
    console.error(e); // TypeError: Throwing
  },
);

// Thenable throws after callback
// Promise resolves
const p3 = Promise.resolve({
  then(onFulfilled) {
    onFulfilled("Resolving");
    throw new TypeError("Throwing");
```

```
  },
});
p3.then(
  (v) => {
    console.log(v); // "Resolving"
  },
  (e) => {
    // not called
  },
);
```

Nested thenables will be "deeply flattened" to a single promise.

```
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled({
      // The thenable is fulfilled with another thenable
      then(onFulfilled, onRejected) {
        onFulfilled(42);
      },
    });
  },
};

Promise.resolve(thenable).then((v) => {
  console.log(v); // 42
});
```

> **Warning:** Do not call `Promise.resolve()` on a thenable that resolves to itself. That leads to infinite recursion, because it attempts to flatten an infinitely-nested promise.

```
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled(thenable);
  },
};

Promise.resolve(thenable); // Will lead to infinite recursion.
```

Calling resolve() on a non-Promise constructor

`Promise.resolve()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes it `console.log` as `resolve`:

```
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the
    // native promise's, but Promise.resolve() calls them in the same way.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static resolve = Promise.resolve;
}

const p = NotPromise.resolve("foo"); // Logs "Resolved foo"
// p is a NotPromise instance
```

The ability to flatten nested thenables is implemented by the `resolve` function of the `Promise()` constructor, so if you call it on another constructor, nested thenables may not be flattened, depending on how that constructor implements its `resolve` function.

```
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled({
      // The thenable is fulfilled with another thenable
      then(onFulfilled, onRejected) {
        onFulfilled(42);
      },
    });
  },
};

const p = NotPromise.resolve(thenable); // Logs "Resolved { then: [Function: then] }"
// p is a NotPromise instance
```

See also

- `Promise`

Promise[Symbol.species]

The **`Promise[Symbol.species]`** static accessor property returns the constructor used to construct return values from promise methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
Promise[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from promise chaining methods that create new promises.

Description

The `[Symbol.species]` accessor property returns the default constructor for `Promise` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class Promise {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubPromise extends Promise {}
SubPromise[Symbol.species] === SubPromise; // true
```

Promise chaining methods — `then()`, `catch()`, and `finally()` — return new promise objects. They get the constructor to construct the new promise through `this.constructor[Symbol.species]`. If `this.constructor` is `undefined`, or if `this.constructor[Symbol.species]` is `undefined` or `null`, the default `Promise()` constructor is used. Otherwise, the constructor returned by `this.constructor[Symbol.species]` is used to construct the new promise object.

Examples

Species in ordinary objects

The `Symbol.species` property returns the default constructor function, which is the `Promise` constructor for `Promise`.

```
Promise[Symbol.species]; // [Function: Promise]
```

Species in derived objects

In an instance of a custom `Promise` subclass, such as `MyPromise`, the `MyPromise` species is the `MyPromise` constructor. However, you might want to override this, in order to return parent `Promise` objects in your derived class methods.

```
class MyPromise extends Promise {
  // Override MyPromise species to the parent Promise constructor
  static get [Symbol.species]() {
    return Promise;
  }
}
```

By default, promise methods would return promises with the type of the subclass.

```
class MyPromise extends Promise {
  someValue = 1;
}

console.log(MyPromise.resolve(1).then(() => {}).someValue); // 1
```

By overriding `[Symbol.species]`, the promise methods will return the base `Promise` type.

```
class MyPromise extends Promise {
  someValue = 1;
  static get [Symbol.species]() {
    return Promise;
  }
}

console.log(MyPromise.resolve(1).then(() => {}).someValue); // undefined
```

See also

- `Promise`

- `Symbol.species`

Promise.prototype.then()

The **`then()`** method of `Promise` instances takes up to two arguments: callback functions for the fulfilled and rejected cases of the `Promise`. It stores the callbacks within the promise it is called on and immediately returns another `Promise` object, allowing you to chain calls to other promise methods.

Try it

```
const promise1 = new Promise((resolve, reject) => {
  resolve("Success!");
});

promise1.then((value) => {
  console.log(value);
  // Expected output: "Success!"
});
```

Syntax

```
then(onFulfilled)
then(onFulfilled, onRejected)
```

Parameters

`onFulfilled`

A function to asynchronously execute when this promise becomes fulfilled. Its return value becomes the fulfillment value of the promise returned by `then()`. The function is called with the following arguments:

`value`

The value that the promise was fulfilled with.

If it is not a function, it is internally replaced with an *identity* function (`(x) => x`) which simply passes the fulfillment value forward.

`onRejected` Optional

A function to asynchronously execute when this promise becomes rejected. Its return value becomes the fulfillment value of the promise returned by `then()`. The function is called with the following arguments:

`reason`

The value that the promise was rejected with.

If it is not a function, it is internally replaced with a *thrower* function (`(x) => { throw x; }`) which throws the rejection reason it received.

Return value

Returns a new `Promise` immediately. This returned promise is always pending when returned, regardless of the current promise's status.

One of the `onFulfilled` and `onRejected` handlers will be executed to handle the current promise's fulfillment or rejection. The call always happens asynchronously, even when the current promise is already settled. The behavior of the promise returned by `then()` (referred to as `p` in the following list) depends on the handler's execution result, following a specific set of rules. If the handler function:

- returns a value: `p` gets fulfilled with the returned value as its value.
- doesn't return anything: `p` gets fulfilled with `undefined` as its value.
- throws an error: `p` gets rejected with the thrown error as its value.
- returns an already fulfilled promise: `p` gets fulfilled with that promise's value as its value.
- returns an already rejected promise: `p` gets rejected with that promise's value as its value.
- returns another pending promise: `p` is pending and becomes fulfilled/rejected with that promise's value as its value immediately after that promise becomes fulfilled/rejected.

Description

The `then()` method schedules callback functions for the eventual completion of a Promise — either fulfillment or rejection. It is the primitive method of promises: the thenable protocol expects all promise-like objects to expose a `then()` method, and the `catch()` and `finally()` methods both work by invoking the object's `then()` method.

For more information about the `onRejected` handler, see the `catch()` reference.

`then()` returns a new promise object but mutates the promise object it's called on, appending the handlers to an internal list if the promise is pending. Therefore the handler is retained by the original promise for as long as that promise remains pending and reachable. For example, the following example will eventually run out of memory even though the returned promise is not retained:

```
const pendingPromise = new Promise(() => {});
while (true) {
  pendingPromise.then(doSomething);
}
```

If you call the `then()` method twice on the same promise object (instead of chaining), then this promise object will have two pairs of settlement handlers. All handlers attached to the same promise object are always called in the order they were added. Moreover, the two promises returned by each call of `then()` start separate chains

and do not wait for each other's settlement.

Thenable objects that arise along the `then()` chain are always resolved — the `onFulfilled` handler never receives a thenable object, and any thenable returned by either handler are always resolved before being passed to the next handler. This is because when constructing the new promise, the `resolve` and `reject` functions passed by the `executor` are saved, and when the current promise settles, the respective function will be called with the fulfillment value or rejection reason. The resolving logic comes from the `resolve` function passed by the `Promise()` constructor.

`then()` supports subclassing, which means it can be called on instances of subclasses of `Promise`, and the result will be a promise of the subclass type. You can customize the type of the return value through the `[Symbol.species]` property.

Examples

Using the then() method

```
const p1 = new Promise((resolve, reject) => {
  resolve("Success!");
  // or
  // reject(new Error("Error!"));
});

p1.then(
  (value) => {
    console.log(value); // Success!
  },
  (reason) => {
    console.error(reason); // Error!
  },
);
```

Having a non-function as either parameter

```
Promise.resolve(1).then(2).then(console.log); // 1
Promise.reject(new Error("failed")).then(2, 2).then(console.log, console.log); // Error: failed
```

Chaining

The `then` method returns a new `Promise`, which allows for method chaining.

If the function passed as handler to `then` returns a `Promise`, an equivalent `Promise` will be exposed to the subsequent `then` in the method chain. The below snippet simulates asynchronous code with the `setTimeout` function.

```
Promise.resolve("foo")
  // 1. Receive "foo", concatenate "bar" to it, and resolve that to the next then
  .then(
    (string) =>
      new Promise((resolve, reject) => {
        setTimeout(() => {
          string += "bar";
          resolve(string);
        }, 1);
      }),
  )
  // 2. receive "foobar", register a callback function to work on that string
  // and print it to the console, but not before returning the unworked on
  // string to the next then
  .then((string) => {
    setTimeout(() => {
      string += "baz";
      console.log(string); // foobarbaz
    }, 1);
    return string;
  })
  // 3. print helpful messages about how the code in this section will be run
  // before the string is actually processed by the mocked asynchronous code in the
  // previous then block.
  .then((string) => {
    console.log(
      "Last Then: oops... didn't bother to instantiate and return a promise in the prior then so the sequence may be a bit surprising",
    );

    // Note that `string` will not have the 'baz' bit of it at this point. This
    // is because we mocked that to happen asynchronously with a setTimeout function
    console.log(string); // foobar
  });

// Logs, in order:
// Last Then: oops... didn't bother to instantiate and return a promise in the prior then so the sequence may be a bit surprising
// foobar
// foobarbaz
```

The value returned from `then()` is resolved in the same way as `Promise.resolve()`. This means thenable objects are supported, and if the return value is not a promise, it's implicitly wrapped in a `Promise` and then resolved.

```
const p2 = new Promise((resolve, reject) => {
  resolve(1);
});

p2.then((value) => {
  console.log(value); // 1
  return value + 1;
}).then((value) => {
  console.log(value, "- A synchronous value works"); // 2 - A synchronous value works
});

p2.then((value) => {
  console.log(value); // 1
});
```

A `then` call returns a promise that eventually rejects if the function throws an error or returns a rejected Promise.

```
Promise.resolve()
  .then(() => {
    // Makes .then() return a rejected promise
    throw new Error("Oh no!");
  })
  .then(
    () => {
      console.log("Not called.");
    },
    (error) => {
      console.error(`onRejected function called: ${error.message}`);
    },
  );
```

In practice, it is often desirable to `catch()` rejected promises rather than `then()`'s two-case syntax, as demonstrated below.

```
Promise.resolve()
  .then(() => {
    // Makes .then() return a rejected promise
    throw new Error("Oh no!");
  })
  .catch((error) => {
    console.error(`onRejected function called: ${error.message}`);
  })
  .then(() => {
    console.log("I am always called even if the prior then's promise rejects");
  });
```

In all other cases, the returned promise eventually fulfills. In the following example, the first `then()` returns `42` wrapped in a fulfilled Promise, even though the previous Promise in the chain was rejected.

```
Promise.reject(new Error("Oh no!"))
  .then(
    () => 99,
    () => 42,
  ) // onRejected returns 42 which is wrapped in a fulfilled Promise
  .then((solution) => console.log(`Resolved with ${solution}`)); // Fulfilled with 42
```

If `onFulfilled` returns a promise, the return value of `then` will be fulfilled/rejected based on the eventual state of that promise.

```
function resolveLater(resolve, reject) {
  setTimeout(() => {
    resolve(10);
  }, 1000);
}
function rejectLater(resolve, reject) {
  setTimeout(() => {
    reject(new Error("Error"));
  }, 1000);
}

const p1 = Promise.resolve("foo");
// Return promise here, that will be resolved to 10 after 1 second
const p2 = p1.then(() => new Promise(resolveLater));
p2.then(
  (v) => {
    console.log("resolved", v); // "resolved", 10
  },
  (e) => {
    // not called
    console.error("rejected", e);
  },
);

// Return promise here, that will be rejected with 'Error' after 1 second
const p3 = p1.then(() => new Promise(rejectLater));
p3.then(
```

```
  (v) => {
    // not called
    console.log("resolved", v);
  },
  (e) => {
    console.error("rejected", e); // "rejected", 'Error'
  },
);
```

You can use chaining to implement one function with a Promise-based API on top of another such function.

```
function fetchCurrentData() {
  // The fetch() API returns a Promise. This function
  // exposes a similar API, except the fulfillment
  // value of this function's Promise has had more
  // work done on it.
  return fetch("current-data.json").then((response) => {
    if (response.headers.get("content-type") !== "application/json") {
      throw new TypeError();
    }
    const j = response.json();
    // maybe do something with j

    // fulfillment value given to user of
    // fetchCurrentData().then()
    return j;
  });
}
```

Asynchronicity of then()

The following is an example to demonstrate the asynchronicity of the `then` method.

```
// Using a resolved promise 'resolvedProm' for example,
// the function call 'resolvedProm.then(...)' returns a new promise immediately,
// but its handler '(value) => {...}' will get called asynchronously as demonstrated by the console.logs.
// the new promise is assigned to 'thenProm',
// and thenProm will be resolved with the value returned by handler
const resolvedProm = Promise.resolve(33);
console.log(resolvedProm);

const thenProm = resolvedProm.then((value) => {
  console.log(
    `this gets called after the end of the main stack. the value received is: ${value}, the value returned is: ${
      value + 1
    }`,
  );
  return value + 1;
});
console.log(thenProm);

// Using setTimeout, we can postpone the execution of a function to the moment the stack is empty
setTimeout(() => {
  console.log(thenProm);
});

// Logs, in order:
// Promise {[[PromiseStatus]]: "resolved", [[PromiseResult]]: 33}
// Promise {[[PromiseStatus]]: "pending", [[PromiseResult]]: undefined}
// "this gets called after the end of the main stack. the value received is: 33, the value returned is: 34"
// Promise {[[PromiseStatus]]: "resolved", [[PromiseResult]]: 34}
```

See also

- `Promise`
- `Promise.prototype.catch()`

Promise.try()

The **`Promise.try()`** static method takes a callback of any kind (returns or throws, synchronously or asynchronously) and resolves its result to a `Promise`.

Syntax

```
Promise.try(func)
Promise.try(func, arg1)
Promise.try(func, arg1, arg2)
Promise.try(func, arg1, arg2, /* …, */ argN)
```

Parameters

`func`

A function that is called synchronously with the arguments provided (`arg1`, `arg2`, ..., `argN`). It can do anything—either return a value, throw an error, or return a promise.

`arg1, arg2, ..., argN`

Arguments to pass to `func`.

Return value

A `Promise` that is:

- Already fulfilled, if `func` synchronously returns a value.
- Already rejected, if `func` synchronously throws an error.
- Asynchronously fulfilled or rejected, if `func` returns a promise. The returned value is resolved to a promise, which means built-in `Promise` objects are returned as-is.

Description

You may have an API that takes a callback. The callback may be synchronous or asynchronous. You want to handle everything uniformly by wrapping the result in a promise. The most straightforward way might be `Promise.resolve(func())`. The problem is that if `func()` synchronously throws an error, this error would not be caught and turned into a rejected promise.

You can wrap this expression in `try...catch`:

```
let result;
try {
  result = Promise.resolve(func());
} catch (error) {
  result = Promise.reject(error);
}
```

The problem is that `try...catch` is not an expression, so you can't directly use it in expression positions like passing it to other functions.

Therefore, when lifting a function call result into a promise, fulfilled or rejected, people more commonly do this:

```
new Promise((resolve) => resolve(func()));
```

For the built-in `Promise()` constructor, errors thrown from the executor are automatically caught and turned into rejections, so this also prevents synchronous errors. The problem is that it unconditionally creates a new `Promise` object, which is unnecessary if `func()` already returns a `Promise`. `Promise.resolve()`, on the other hand, is smart enough to prevent that extra promise wrapping.

`Promise.try()` is almost exactly equivalent to the `try...catch` approach, except that it's shorter and can be used as an expression:

```
Promise.try(func);
```

Note that `Promise.try()` is *not* equivalent to this, despite being highly similar:

```
Promise.resolve().then(func);
```

The difference is that the callback passed to `then()` is always called asynchronously, while the executor of the `Promise()` constructor is called synchronously. `Promise.try` also calls the function synchronously, and resolves the promise immediately if possible.

`Promise.try()`, combined with `catch()` and `finally()`, can be used to handle both synchronous and asynchronous errors in a single chain, and make promise error handling appear almost like synchronous error handling.

Like `setTimeout()`, `Promise.try()` accepts extra arguments that are passed to the callback. This means instead of doing this:

```
Promise.try(() => func(arg1, arg2));
```

You can do this:

```
Promise.try(func, arg1, arg2);
```

Which are equivalent, but the latter avoids creating an extra closure and is more efficient.

`Promise.try()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will contain a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

Examples

Using Promise.try()

The following example takes a callback, "lifts" it into a promise, handles the result, and does some error handling:

```
function doSomething(action) {
  return Promise.try(action)
    .then((result) => console.log(result))
    .catch((error) => console.error(error))
    .finally(() => console.log("Done"));
}

doSomething(() => "Sync result");
```

```
doSomething(() => {
  throw new Error("Sync error");
});

doSomething(async () => "Async result");

doSomething(async () => {
  throw new Error("Async error");
});
```

In async/await, the same code would look like this:

```
async function doSomething(action) {
  try {
    const result = await action();
    console.log(result);
  } catch (error) {
    console.error(error);
  } finally {
    console.log("Done");
  }
}
```

Calling try() on a non-Promise constructor

`Promise.try()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor.

The following is a slightly more faithful approximation of the actual `Promise.try()` (although it should still not be used as a polyfill):

```
Promise.try = function (func, ...args) {
  let result;
  try {
    result = func(...args);
  } catch (error) {
    return Promise.reject.call(this, error);
  }
  return Promise.resolve.call(this, result);
};
```

`Promise.try()` delegates to `Promise.resolve()` and `Promise.reject()` to create the return value, and both of these functions are generic.

For example, we can call it on a constructor that passes `console.log` as the `resolve` and `reject` functions to `executor`:

```
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the native
    // promise's, but Promise.try() just calls resolve
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static try = Promise.try;
}

const p = NotPromise.try(() => "hello");
// Logs: Resolved hello
// p is a NotPromise instance

const p2 = NotPromise.try(() => {
  throw new Error("oops");
});
// Logs: Rejected Error: oops
// p2 is a NotPromise instance
```

See also

- Polyfill of `Promise.try` in `core-js`
- es-shims polyfill of `Promise.try`
- Using promises guide
- `Promise`
- `Promise()` constructor

Promise.withResolvers()

The **`Promise.withResolvers()`** static method returns an object containing a new `Promise` object and two functions to resolve or reject it, corresponding to the two parameters passed to the executor of the `Promise()` constructor.

Syntax

```
Promise.withResolvers()
```

Parameters

None.

Return value

A plain object containing the following properties:

`promise`

A `Promise` object.

`resolve`

A function that resolves the promise. For its semantics, see the `Promise()` constructor reference.

`reject`

A function that rejects the promise. For its semantics, see the `Promise()` constructor reference.

Description

`Promise.withResolvers()` is exactly equivalent to the following code:

```
let resolve, reject;
const promise = new Promise((res, rej) => {
  resolve = res;
  reject = rej;
});
```

Except that it is more concise and does not require the use of `let`.

The key difference when using `Promise.withResolvers()` is that the resolution and rejection functions now live in the same scope as the promise itself, instead of being created and used once within the executor. This may enable some more advanced use cases, such as when reusing them for recurring events, particularly with streams and queues. This also generally results in less nesting than wrapping a lot of logic within the executor.

`Promise.withResolvers()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will contain a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

Examples

Transforming a stream to an async iterable

The use case of `Promise.withResolvers()` is when you have a promise that should be resolved or rejected by some event listener that cannot be wrapped inside the promise executor. The following example transforms a Node.js readable stream to an async iterable. Each `promise` here represents a single batch of data available, and each time the current batch is read, a new promise is created for the next batch. Note how the event listeners are only attached once, but actually call a different version of the `resolve` and `reject` functions each time.

```
async function* readableToAsyncIterable(stream) {
  let { promise, resolve, reject } = Promise.withResolvers();
  stream.on("error", (error) => reject(error));
  stream.on("end", () => resolve());
  stream.on("readable", () => resolve());

  while (stream.readable) {
    await promise;
    let chunk;
    while ((chunk = stream.read())) {
      yield chunk;
    }
    ({ promise, resolve, reject } = Promise.withResolvers());
  }
}
```

Calling withResolvers() on a non-Promise constructor

`Promise.withResolvers()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes `console.log` as the `resolve` and `reject` functions to `executor`:

```
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the native
    // promise's, but Promise.withResolvers() just returns them, as is.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static withResolvers = Promise.withResolvers;
}
```

```
const { promise, resolve, reject } = NotPromise.withResolvers();
resolve("hello");
// Logs: Resolved hello
```

See also

- Polyfill of `Promise.withResolvers` in `core-js`
- es-shims polyfill of `Promise.withResolvers`
- Using promises guide
- `Promise`
- `Promise()` constructor

Proxy

The **`Proxy`** object enables you to create a proxy for another object, which can intercept and redefine fundamental operations for that object.

Description

The `Proxy` object allows you to create an object that can be used in place of the original object, but which may redefine fundamental `Object` operations like getting, setting, and defining properties. Proxy objects are commonly used to log property accesses, validate, format, or sanitize inputs, and so on.

You create a `Proxy` with two parameters:

- `target`: the original object which you want to proxy
- `handler`: an object that defines which operations will be intercepted and how to redefine intercepted operations.

For example, this code creates a proxy for the `target` object.

```
const target = {
  message1: "hello",
  message2: "everyone",
};

const handler1 = {};

const proxy1 = new Proxy(target, handler1);
```

Because the handler is empty, this proxy behaves just like the original target:

```
console.log(proxy1.message1); // hello
console.log(proxy1.message2); // everyone
```

To customize the proxy, we define functions on the handler object:

```
const target = {
  message1: "hello",
  message2: "everyone",
};

const handler2 = {
  get(target, prop, receiver) {
    return "world";
  },
};

const proxy2 = new Proxy(target, handler2);
```

Here we've provided an implementation of the `get()` handler, which intercepts attempts to access properties in the target.

Handler functions are sometimes called *traps*, presumably because they trap calls to the target object. The trap in `handler2` above redefines all property accessors:

```
console.log(proxy2.message1); // world
console.log(proxy2.message2); // world
```

Proxies are often used with the `Reflect` object, which provides some methods with the same names as the `Proxy` traps. The `Reflect` methods provide the reflective semantics for invoking the corresponding object internal methods. For example, we can call `Reflect.get` if we don't wish to redefine the object's behavior:

```
const target = {
  message1: "hello",
  message2: "everyone",
};

const handler3 = {
  get(target, prop, receiver) {
    if (prop === "message2") {
      return "world";
    }
    return Reflect.get(...arguments);
  },
};

const proxy3 = new Proxy(target, handler3);
```

```
console.log(proxy3.message1); // hello
console.log(proxy3.message2); // world
```

The `Reflect` method still interacts with the object through object internal methods — it doesn't "de-proxify" the proxy if it's invoked on a proxy. If you use `Reflect` methods within a proxy trap, and the `Reflect` method call gets intercepted by the trap again, there may be infinite recursion.

Terminology

The following terms are used when talking about the functionality of proxies.

handler

> The object passed as the second argument to the `Proxy` constructor. It contains the traps which define the behavior of the proxy.

trap

> The function that defines the behavior for the corresponding object internal method. (This is analogous to the concept of *traps* in operating systems.)

target

> Object which the proxy virtualizes. It is often used as storage backend for the proxy. Invariants (semantics that remain unchanged) regarding object non-extensibility or non-configurable properties are verified against the target.

invariants

> Semantics that remain unchanged when implementing custom operations. If your trap implementation violates the invariants of a handler, a `TypeError` will be thrown.

Object internal methods

Objects are collections of properties. However, the language doesn't provide any machinery to *directly* manipulate data stored in the object — rather, the object defines some internal methods specifying how it can be interacted with. For example, when you read `obj.x`, you may expect the following to happen:

- The `x` property is searched up the prototype chain until it is found.
- If `x` is a data property, the property descriptor's `value` attribute is returned.
- If `x` is an accessor property, the getter is invoked, and the return value of the getter is returned.

There isn't anything special about this process in the language — it's just because ordinary objects, by default, have a `[[Get]]` internal method that is defined with this behavior. The `obj.x` property access syntax simply invokes the `[[Get]]` method on the object, and the object uses its own internal method implementation to determine what to return.

As another example, arrays differ from normal objects, because they have a magic `length` property that, when modified, automatically allocates empty slots or removes elements from the array. Similarly, adding array elements automatically changes the `length` property. This is because arrays have a `[[DefineOwnProperty]]` internal method that knows to update `length` when an integer index is written to, or update the array contents when `length` is written to. Such objects whose internal methods have different implementations from ordinary objects are called *exotic objects*. `Proxy` enable developers to define their own exotic objects with full capacity.

All objects have the following internal methods:

Internal method	Corresponding trap
`[[GetPrototypeOf]]`	`getPrototypeOf()`
`[[SetPrototypeOf]]`	`setPrototypeOf()`
`[[IsExtensible]]`	`isExtensible()`
`[[PreventExtensions]]`	`preventExtensions()`
`[[GetOwnProperty]]`	`getOwnPropertyDescriptor()`
`[[DefineOwnProperty]]`	`defineProperty()`
`[[HasProperty]]`	`has()`
`[[Get]]`	`get()`
`[[Set]]`	`set()`
`[[Delete]]`	`deleteProperty()`
`[[OwnPropertyKeys]]`	`ownKeys()`

Function objects also have the following internal methods:

Internal method	Corresponding trap
`[[Call]]`	`apply()`
`[[Construct]]`	`construct()`

It's important to realize that all interactions with an object eventually boils down to the invocation of one of these internal methods, and that they are all customizable through proxies. This means almost no behavior (except certain critical invariants) is guaranteed in the language — everything is defined by the object itself. When you run `delete obj.x`, there's no guarantee that `"x" in obj` returns `false` afterwards — it depends on the object's implementations of `[[Delete]]` and `[[HasProperty]]`. A `delete obj.x` may log things to the console, modify some global state, or even define a new property instead of deleting the existing one, although these semantics should be avoided in your own code.

All internal methods are called by the language itself, and are not directly accessible in JavaScript code. The `Reflect` namespace offers methods that do little more than call the internal methods, besides some input normalization/validation. In each trap's page, we list several typical situations when the trap is invoked, but these internal methods are called in *a lot* of places. For example, array methods read and write to array through these internal methods, so methods like `push()` would also invoke `get()` and `set()` traps.

Most of the internal methods are straightforward in what they do. The only two that may be confusable are `[[Set]]` and `[[DefineOwnProperty]]`. For normal objects, the former invokes setters; the latter doesn't. (And `[[Set]]` calls `[[DefineOwnProperty]]` internally if there's no existing property or the property is a data property.) While you may know that the `obj.x = 1` syntax uses `[[Set]]`, and `Object.defineProperty()` uses `[[DefineOwnProperty]]`, it's not immediately apparent what semantics other built-in methods and syntaxes use. For example, class fields use the `[[DefineOwnProperty]]` semantic, which is why setters defined in the superclass are not invoked when a field is declared on the derived class.

Constructor

`Proxy()`

Creates a new `Proxy` object.

> **Note:** There's no `Proxy.prototype` property, so `Proxy` instances do not have any special properties or methods.

Static methods

`Proxy.revocable()`

Creates a revocable `Proxy` object.

Examples

Basic example

In this example, the number `37` gets returned as the default value when the property name is not in the object. It is using the `get()` handler.

```
const handler = {
  get(obj, prop) {
    return prop in obj ? obj[prop] : 37;
  },
};

const p = new Proxy({}, handler);
p.a = 1;
p.b = undefined;

console.log(p.a, p.b); // 1, undefined

console.log("c" in p, p.c); // false, 37
```

No-op forwarding proxy

In this example, we are using a native JavaScript object to which our proxy will forward all operations that are applied to it.

```
const target = {};
const p = new Proxy(target, {});

p.a = 37; // Operation forwarded to the target

console.log(target.a); // 37 (The operation has been properly forwarded!)
```

Note that while this "no-op" works for plain JavaScript objects, it does not work for native objects, such as DOM elements, `Map` objects, or anything that has internal slots. See no private field forwarding for more information.

No private field forwarding

A proxy is still another object with a different identity — it's a *proxy* that operates between the wrapped object and the outside. As such, the proxy does not have direct access to the original object's private elements.

```
class Secret {
  #secret;
  constructor(secret) {
    this.#secret = secret;
  }
  get secret() {
    return this.#secret.replace(/\d+/, "[REDACTED]");
  }
}

const secret = new Secret("123456");
console.log(secret.secret); // [REDACTED]
// Looks like a no-op forwarding...
const proxy = new Proxy(secret, {});
console.log(proxy.secret); // TypeError: Cannot read private member #secret from an object whose class did not declare it
```

This is because when the proxy's `get` trap is invoked, the `this` value is the `proxy` instead of the original `secret`, so `#secret` is not accessible. To fix this, use the original `secret` as `this`:

```
const proxy = new Proxy(secret, {
  get(target, prop, receiver) {
    // By default, it looks like Reflect.get(target, prop, receiver)
    // which has a different value of `this`
    return target[prop];
  },
});
console.log(proxy.secret);
```

For methods, this means you have to redirect the method's `this` value to the original object as well:

```
class Secret {
  #x = 1;
  x() {
    return this.#x;
  }
}

const secret = new Secret();
const proxy = new Proxy(secret, {
  get(target, prop, receiver) {
    const value = target[prop];
    if (value instanceof Function) {
      return function (...args) {
        return value.apply(this === receiver ? target : this, args);
      };
    }
    return value;
  },
});
console.log(proxy.x());
```

Some native JavaScript objects have properties called *internal slots*, which are not accessible from JavaScript code. For example, `Map` objects have an internal slot called `[[MapData]]`, which stores the key-value pairs of the map. As such, you cannot trivially create a forwarding proxy for a map:

```
const proxy = new Proxy(new Map(), {});
console.log(proxy.size); // TypeError: get size method called on incompatible Proxy
```

You have to use the "`this`-recovering" proxy illustrated above to work around this.

Validation

With a `Proxy`, you can easily validate the passed value for an object. This example uses the `set()` handler.

```
const validator = {
  set(obj, prop, value) {
    if (prop === "age") {
      if (!Number.isInteger(value)) {
        throw new TypeError("The age is not an integer");
      }
      if (value > 200) {
        throw new RangeError("The age seems invalid");
      }
    }

    // The default behavior to store the value
    obj[prop] = value;

    // Indicate success
    return true;
  },
};

const person = new Proxy({}, validator);

person.age = 100;
console.log(person.age); // 100
person.age = "young"; // Throws an exception
person.age = 300; // Throws an exception
```

Manipulating DOM nodes

In this example we use `Proxy` to toggle an attribute of two different elements: so when we set the attribute on one element, the attribute is unset on the other one.

We create a `view` object which is a proxy for an object with a `selected` property. The proxy handler defines the `set()` handler.

When we assign an HTML element to `view.selected`, the element's `'aria-selected'` attribute is set to `true`. If we then assign a different element to `view.selected`, this element's `'aria-selected'` attribute is set to `true` and the previous element's `'aria-selected'` attribute is automatically set to `false`.

```
const view = new Proxy(
  {
    selected: null,
  },
  {
    set(obj, prop, newVal) {
      const oldVal = obj[prop];

      if (prop === "selected") {
        if (oldVal) {
          oldVal.setAttribute("aria-selected", "false");
        }
        if (newVal) {
          newVal.setAttribute("aria-selected", "true");
        }
      }

      // The default behavior to store the value
      obj[prop] = newVal;

      // Indicate success
      return true;
    },
  },
);

const item1 = document.getElementById("item-1");
const item2 = document.getElementById("item-2");

// select item1:
view.selected = item1;

console.log(`item1: ${item1.getAttribute("aria-selected")}`);
// item1: true

// selecting item2 de-selects item1:
view.selected = item2;

console.log(`item1: ${item1.getAttribute("aria-selected")}`);
// item1: false

console.log(`item2: ${item2.getAttribute("aria-selected")}`);
// item2: true
```

Value correction and an extra property

The `products` proxy object evaluates the passed value and converts it to an array if needed. The object also supports an extra property called `latestBrowser` both as a getter and a setter.

```
const products = new Proxy(
  {
    browsers: ["Firefox", "Chrome"],
  },
  {
    get(obj, prop) {
      // An extra property
      if (prop === "latestBrowser") {
        return obj.browsers[obj.browsers.length - 1];
      }

      // The default behavior to return the value
      return obj[prop];
    },
    set(obj, prop, value) {
      // An extra property
      if (prop === "latestBrowser") {
        obj.browsers.push(value);
        return true;
      }

      // Convert the value if it is not an array
      if (typeof value === "string") {
        value = [value];
      }

      // The default behavior to store the value
      obj[prop] = value;

      // Indicate success
      return true;
    },
  },
);

console.log(products.browsers);
```

```
//  ['Firefox', 'Chrome']

products.browsers = "Safari";
//  pass a string (by mistake)

console.log(products.browsers);
//  ['Safari'] <- no problem, the value is an array

products.latestBrowser = "Edge";

console.log(products.browsers);
//  ['Safari', 'Edge']

console.log(products.latestBrowser);
//  'Edge'
```

See also

- Proxies are awesome presentation by Brendan Eich at JSConf (2014)

Proxy() constructor

The **`Proxy()`** constructor creates `Proxy` objects.

Syntax

```
new Proxy(target, handler)
```

> **Note:** `Proxy()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`target`

A target object to wrap with `Proxy`. It can be any sort of object, including a native array, a function, or even another proxy.

`handler`

An object whose properties are functions that define the behavior of the proxy when an operation is performed on it.

Description

Use the `Proxy()` constructor to create a new `Proxy` object. This constructor takes two mandatory arguments:

- `target` is the object for which you want to create the proxy
- `handler` is the object that defines the custom behavior of the proxy.

An empty handler will create a proxy that behaves, in almost all respects, exactly like the target. By defining any of a set group of functions on the `handler` object, you can customize specific aspects of the proxy's behavior. For example, by defining `get()` you can provide a customized version of the target's property accessor.

Handler functions

This section lists all the handler functions you can define. Handler functions are sometimes called *traps*, because they trap calls to the underlying target object.

`handler.apply()`

A trap for a function call.

`handler.construct()`

A trap for the `new` operator.

`handler.defineProperty()`

A trap for `Object.defineProperty`.

`handler.deleteProperty()`

A trap for the `delete` operator.

`handler.get()`

A trap for getting property values.

`handler.getOwnPropertyDescriptor()`

A trap for `Object.getOwnPropertyDescriptor`.

`handler.getPrototypeOf()`

A trap for `Object.getPrototypeOf`.

`handler.has()`

A trap for the `in` operator.

`handler.isExtensible()`

A trap for `Object.isExtensible`.

`handler.ownKeys()`

A trap for `Object.getOwnPropertyNames` and `Object.getOwnPropertySymbols`.

`handler.preventExtensions()`

A trap for `Object.preventExtensions`.

`handler.set()`

A trap for setting property values.

`handler.setPrototypeOf()`

A trap for `Object.setPrototypeOf`.

Examples

Selectively proxy property accessors

In this example the target has two properties, `notProxied` and `proxied`. We define a handler that returns a different value for `proxied`, and lets any other accesses through to the target.

```
const target = {
  notProxied: "original value",
  proxied: "original value",
};

const handler = {
  get(target, prop, receiver) {
    if (prop === "proxied") {
      return "replaced value";
    }
    return Reflect.get(...arguments);
  },
};

const proxy = new Proxy(target, handler);

console.log(proxy.notProxied); // "original value"
console.log(proxy.proxied); // "replaced value"
```

See also

- Meta programming guide
- `Reflect`

handler.apply()

The **`handler.apply()`** method is a trap for the `[[Call]]` object internal method, which is used by operations such as function calls.

Try it

```
function sum(a, b) {
  return a + b;
}

const handler = {
  apply(target, thisArg, argumentsList) {
    console.log(`Calculate sum: ${argumentsList}`);
    // Expected output: "Calculate sum: 1,2"

    return target(argumentsList[0], argumentsList[1]) * 10;
  },
};

const proxy = new Proxy(sum, handler);

console.log(sum(1, 2));
// Expected output: 3
console.log(proxy(1, 2));
// Expected output: 30
```

Syntax

```
new Proxy(target, {
  apply(target, thisArg, argumentsList) {
```

```
  }
})
```

Parameters

The following parameters are passed to the `apply()` method. `this` is bound to the handler.

`target`

: The target callable object.

`thisArg`

: The `this` argument for the call.

`argumentsList`

: An `Array` containing the arguments passed to the function.

Return value

The `apply()` method can return any value, representing the return value of the function call.

Description

Interceptions

This trap can intercept these operations:

- Function call: `proxy(...args)`
- `Function.prototype.apply()` and `Function.prototype.call()`
- `Reflect.apply()`

Or any other operation that invokes the `[[Call]]` internal method.

Invariants

The proxy's `[[Call]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The `target` must be a callable itself. That is, it must be a function object.

Examples

Trapping a function call

The following code traps a function call.

```
const p = new Proxy(function () {}, {
  apply(target, thisArg, argumentsList) {
    console.log(`called: ${argumentsList}`);
    return argumentsList[0] + argumentsList[1] + argumentsList[2];
  },
});

console.log(p(1, 2, 3)); // "called: 1,2,3"
// 6
```

See also

- `Proxy`
- `Proxy()` constructor
- `Function.prototype.apply()`
- `Function.prototype.call()`
- `Reflect.apply()`

handler.construct()

The **`handler.construct()`** method is a trap for the `[[Construct]]` object internal method, which is used by operations such as the `new` operator. In order for the new operation to be valid on the resulting Proxy object, the target used to initialize the proxy must itself be a valid constructor.

Try it

```
function Monster(disposition) {
  this.disposition = disposition;
}

const handler = {
  construct(target, args) {
    console.log(`Creating a ${target.name}`);
    // Expected output: "Creating a Monster"

    return new target(...args);
  },
};
```

```
const ProxiedMonster = new Proxy(Monster, handler);

console.log(new ProxiedMonster("fierce").disposition);
// Expected output: "fierce"
```

Syntax

```
new Proxy(target, {
  construct(target, argumentsList, newTarget) {
  }
})
```

Parameters

The following parameters are passed to the `construct()` method. `this` is bound to the handler.

`target`

The target constructor object.

`argumentsList`

An `Array` containing the arguments passed to the constructor.

`newTarget`

The constructor that was originally called.

Return value

The `construct()` method must return an object, representing the newly created object.

Description

Interceptions

This trap can intercept these operations:

- The `new` operator: `new myFunction(...args)`
- `Reflect.construct()`

Or any other operation that invokes the `[[Construct]]` internal method.

Invariants

The proxy's `[[Construct]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The `target` must be a constructor itself.
- The result must be an `Object`.

Examples

Trapping the new operator

The following code traps the `new` operator.

```
const p = new Proxy(function () {}, {
  construct(target, argumentsList, newTarget) {
    console.log(`called: ${argumentsList}`);
    return { value: argumentsList[0] * 10 };
  },
});

console.log(new p(1).value); // "called: 1"
// 10
```

The following code violates the invariant.

```
const p = new Proxy(function () {}, {
  construct(target, argumentsList, newTarget) {
    return 1;
  },
});

new p(); // TypeError is thrown
```

The following code improperly initializes the proxy. The `target` in Proxy initialization must itself be a valid constructor for the `new` operator.

```
const p = new Proxy(
  {},
  {
    construct(target, argumentsList, newTarget) {
      return {};
```

```
    },
  },
);

new p(); // TypeError is thrown, "p" is not a constructor
```

See also

- `Proxy`
- `Proxy()` constructor
- `new`
- `Reflect.construct()`

handler.defineProperty()

The **`handler.defineProperty()`** method is a trap for the `[[DefineOwnProperty]]` object internal method, which is used by operations such as `Object.defineProperty()`.

Try it

```
const handler = {
  defineProperty(target, key, descriptor) {
    invariant(key, "define");
    return true;
  },
};

function invariant(key, action) {
  if (key[0] === "_") {
    throw new Error(`Invalid attempt to ${action} private "${key}" property`);
  }
}

const monster = {};
const proxy = new Proxy(monster, handler);

console.log((proxy._secret = "easily scared"));
// Expected output: Error: Invalid attempt to define private "_secret" property
```

Syntax

```
new Proxy(target, {
  defineProperty(target, property, descriptor) {
  }
})
```

Parameters

The following parameters are passed to the `defineProperty()` method. `this` is bound to the handler.

`target`

The target object.

`property`

A string or `Symbol` representing the property name.

`descriptor`

The descriptor for the property being defined or modified.

Return value

The `defineProperty()` method must return a `Boolean` indicating whether or not the property has been successfully defined. Other values are coerced to booleans.

Many operations, including `Object.defineProperty()` and `Object.defineProperties()`, throw a `TypeError` if the `[[DefineOwnProperty]]` internal method returns `false`.

Description

Interceptions

This trap can intercept these operations:

- `Object.defineProperty()`, `Object.defineProperties()`
- `Reflect.defineProperty()`

Or any other operation that invokes the `[[DefineOwnProperty]]` internal method.

Invariants

The proxy's `[[DefineOwnProperty]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- A property cannot be added, if the target object is not extensible. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `Reflect.getOwnPropertyDescriptor()` returns `undefined` for the property on `target`, then the trap must return a falsy value.

- A property cannot be non-configurable, unless there exists a corresponding non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `undefined` or `configurable: true` for the property on `target`, and `descriptor.configurable` is `false`, then the trap must return a falsy value.
- A non-configurable property cannot be non-writable, unless there exists a corresponding non-configurable, non-writable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: true` for the property on `target`, and `descriptor.writable` is `false`, then the trap must return a falsy value.
- If a property has a corresponding property on the target object, then the target object property's descriptor must be compatible with `descriptor`. That is, pretending `target` is an ordinary object, and `Object.defineProperty(target, property, descriptor)` would throw an error, then the trap must return a falsy value. The `Object.defineProperty()` reference contains more information, but to summarize, when the target property is non-configurable, the following must hold:
  - `configurable`, `enumerable`, `get`, and `set` cannot be changed
  - the property cannot be switched between data and accessor
  - the `writable` attribute can only be changed from `true` to `false`
  - the `value` attribute can only be changed if `writable` is `true`

Examples

Trapping of defineProperty

The following code traps `Object.defineProperty()`.

```
const p = new Proxy(
  {},
  {
    defineProperty(target, prop, descriptor) {
      console.log(`called: ${prop}`);
      return true;
    },
  },
);

const desc = { configurable: true, enumerable: true, value: 10 };
Object.defineProperty(p, "a", desc); // "called: a"
```

When calling `Object.defineProperty()` or `Reflect.defineProperty()`, the `descriptor` passed to `defineProperty()` trap has one restriction—only following properties are usable (non-standard properties will be ignored):

- `enumerable`
- `configurable`
- `writable`
- `value`
- `get`
- `set`

```
const p = new Proxy(
  {},
  {
    defineProperty(target, prop, descriptor) {
      console.log(descriptor);
      return Reflect.defineProperty(target, prop, descriptor);
    },
  },
);

Object.defineProperty(p, "name", {
  value: "proxy",
  type: "custom",
}); // { value: 'proxy' }
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.defineProperty()`
- `Reflect.defineProperty()`

handler.deleteProperty()

The **`handler.deleteProperty()`** method is a trap for the `[[Delete]]` object internal method, which is used by operations such as the `delete` operator.

Try it

```
const monster = {
  texture: "scaly",
};

const handler = {
  deleteProperty(target, prop) {
    if (prop in target) {
      delete target[prop];
      console.log(`property removed: ${prop}`);
```

```
      // Expected output: "property removed: texture"
    }
  },
};

console.log(monster.texture);
// Expected output: "scaly"

const proxy = new Proxy(monster, handler);
delete proxy.texture;

console.log(monster.texture);
// Expected output: undefined
```

Syntax

```
new Proxy(target, {
  deleteProperty(target, property) {
  }
})
```

Parameters

The following parameters are passed to the `deleteProperty()` method. `this` is bound to the handler.

`target`

The target object.

`property`

A string or `Symbol` representing the property name.

Return value

The `deleteProperty()` method must return a `Boolean` indicating whether or not the property has been successfully deleted. Other values are coerced to booleans.

Many operations, including the `delete` operator when in strict mode, throw a `TypeError` if the `[[Delete]]` internal method returns `false`.

Description

Interceptions

This trap can intercept these operations:

- The `delete` operator: `delete proxy[foo]` and `delete proxy.foo`
- `Reflect.deleteProperty()`

Or any other operation that invokes the `[[Delete]]` internal method.

Invariants

The proxy's `[[Delete]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- A property cannot be reported as deleted, if it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false` for the property on `target`, then the trap must return a falsy value.
- A property cannot be reported as deleted, if it exists as an own property of the target object and the target object is non-extensible. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `Reflect.getOwnPropertyDescriptor()` returns a property descriptor for the property on `target`, then the trap must return a falsy value.

Examples

Trapping the delete operator

The following code traps the `delete` operator.

```
const p = new Proxy(
  {},
  {
    deleteProperty(target, prop) {
      if (!(prop in target)) {
        console.log(`property not found: ${prop}`);
        return false;
      }
      delete target[prop];
      console.log(`property removed: ${prop}`);
      return true;
    },
  },
);

p.a = 10;
console.log("a" in p); // true
```

```
const result1 = delete p.a; // "property removed: a"
console.log(result1); // true
console.log("a" in p); // false

const result2 = delete p.a; // "property not found: a"
console.log(result2); // false
```

See also

- `Proxy`
- `Proxy()` constructor
- `delete`
- `Reflect.deleteProperty()`

handler.get()

The **`handler.get()`** method is a trap for the `[[Get]]` object internal method, which is used by operations such as property accessors.

Try it

```
const monster = {
  secret: "easily scared",
  eyeCount: 4,
};

const handler = {
  get(target, prop, receiver) {
    if (prop === "secret") {
      return `${target.secret.substring(0, 4)} ... shhhh!`;
    }
    return Reflect.get(...arguments);
  },
};

const proxy = new Proxy(monster, handler);

console.log(proxy.eyeCount);
// Expected output: 4

console.log(proxy.secret);
// Expected output: "easi ... shhhh!"
```

Syntax

```
new Proxy(target, {
  get(target, property, receiver) {
  }
})
```

Parameters

The following parameters are passed to the `get()` method. `this` is bound to the handler.

`target`

The target object.

`property`

A string or `Symbol` representing the property name.

`receiver`

The `this` value for getters; see `Reflect.get()`. This is usually either the proxy itself or an object that inherits from the proxy.

Return value

The `get()` method can return any value, representing the property value.

Description

Interceptions

This trap can intercept these operations:

- Property access: `proxy[foo]` and `proxy.bar`
- `Reflect.get()`

Or any other operation that invokes the `[[Get]]` internal method.

Invariants

The proxy's `[[Get]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The value reported for a property must be the same as the value of the corresponding target object property, if the target object property is a non-writable, non-configurable own data property. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: false` for the property on `target`, then the trap must return the same value as the `value` attribute in the `target`'s property descriptor.
- The value reported for a property must be `undefined`, if the corresponding target object property is a non-configurable own accessor property that has an undefined getter. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, get: undefined` for the property on `target`, then the trap must return `undefined`.

Examples

Trap for getting a property value

The following code traps getting a property value.

```
const p = new Proxy(
  {},
  {
    get(target, property, receiver) {
      console.log(`called: ${property}`);
      return 10;
    },
  },
);

console.log(p.a);
// "called: a"
// 10
```

The following code violates an invariant.

```
const obj = {};
Object.defineProperty(obj, "a", {
  configurable: false,
  enumerable: false,
  value: 10,
  writable: false,
});

const p = new Proxy(obj, {
  get(target, property) {
    return 20;
  },
});

p.a; // TypeError is thrown
```

See also

- `Proxy`
- `Proxy()` constructor
- `Reflect.get()`

handler.getOwnPropertyDescriptor()

The **`handler.getOwnPropertyDescriptor()`** method is a trap for the `[[GetOwnProperty]]` object internal method, which is used by operations such as `Object.getOwnPropertyDescriptor()`.

Try it

```
const monster = {
  eyeCount: 4,
};

const handler = {
  getOwnPropertyDescriptor(target, prop) {
    console.log(`called: ${prop}`);
    // Expected output: "called: eyeCount"

    return { configurable: true, enumerable: true, value: 5 };
  },
};

const proxy = new Proxy(monster, handler);

console.log(Object.getOwnPropertyDescriptor(proxy, "eyeCount").value);
// Expected output: 5
```

Syntax

```
new Proxy(target, {
  getOwnPropertyDescriptor(target, property) {
  }
})
```

Parameters

The following parameters are passed to the `getOwnPropertyDescriptor()` method. `this` is bound to the handler.

`target`

The target object.

`property`

A string or `Symbol` representing the property name.

Return value

The `getOwnPropertyDescriptor()` method must return an object or `undefined`, representing the property descriptor. Missing attributes are normalized in the same way as `Object.defineProperty()`.

Description

Interceptions

This trap can intercept these operations:

- `Object.getOwnPropertyDescriptor()`
- `Reflect.getOwnPropertyDescriptor()`

Or any other operation that invokes the `[[GetOwnProperty]]` internal method.

Invariants

The proxy's `[[GetOwnProperty]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result must be either an `Object` or `undefined`.
- A property cannot be reported as non-existent, if it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false` for the property on `target`, then the trap must not return `undefined`.
- A property cannot be reported as non-existent, if it exists as an own property of a non-extensible target object. That is, if `Reflect.isExtensible()` returns `false` for the target object, then the trap must not return `undefined`.
- A property cannot be reported as existent, if it does not exist as an own property of the target object and the target object is not extensible. That is, if `Reflect.isExtensible()` returns `false` for the target object, and `Reflect.getOwnPropertyDescriptor()` returns `undefined` for the property on `target`, then the trap must return `undefined`.
- A property cannot be reported as non-configurable, unless it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `undefined` or `configurable: true` for the property on `target`, then the trap must not return `configurable: false`.
- A property cannot be reported as both non-configurable and non-writable, unless it exists as a non-configurable, non-writable own property of the target object. That is, in addition to the previous invariant, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: true` for the property on `target`, then the trap must not return `configurable: false, writable: false`.
- If a property has a corresponding property on the target object, then the target object property's descriptor must be compatible with `descriptor`. That is, pretending `target` is an ordinary object, then `Object.defineProperty(target, property, resultObject)` must not throw an error. The `Object.defineProperty()` reference contains more information, but to summarize, when the target property is non-configurable, the following must hold:
  - `configurable`, `enumerable`, `get`, and `set` must be the same as original. `writable` must also be the original by virtue of the previous invariant.
  - the property must stay as data or accessor
  - the `value` attribute can only be changed if `writable` is `true`

Examples

Trapping of getOwnPropertyDescriptor

The following code traps `Object.getOwnPropertyDescriptor()`.

```
const p = new Proxy(
  { a: 20 },
  {
    getOwnPropertyDescriptor(target, prop) {
      console.log(`called: ${prop}`);
      return { configurable: true, enumerable: true, value: 10 };
    },
  },
);

console.log(Object.getOwnPropertyDescriptor(p, "a").value);
// "called: a"
// 10
```

The following code violates an invariant.

```
const obj = { a: 10 };
Object.preventExtensions(obj);
const p = new Proxy(obj, {
  getOwnPropertyDescriptor(target, prop) {
    return undefined;
  },
});
```

```
Object.getOwnPropertyDescriptor(p, "a"); // TypeError is thrown
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.getOwnPropertyDescriptor()`
- `Reflect.getOwnPropertyDescriptor()`

handler.getPrototypeOf()

The **`handler.getPrototypeOf()`** method is a trap for the `[[GetPrototypeOf]]` object internal method, which is used by operations such as `Object.getPrototypeOf()`.

Try it

```
const monster = {
  eyeCount: 4,
};

const monsterPrototype = {
  eyeCount: 2,
};

const handler = {
  getPrototypeOf(target) {
    return monsterPrototype;
  },
};

const proxy = new Proxy(monster, handler);

console.log(Object.getPrototypeOf(proxy) === monsterPrototype);
// Expected output: true

console.log(Object.getPrototypeOf(proxy).eyeCount);
// Expected output: 2
```

Syntax

```
new Proxy(target, {
  getPrototypeOf(target) {
  }
})
```

Parameters

The following parameter is passed to the `getPrototypeOf()` method. `this` is bound to the handler.

`target`

The target object.

Return value

The `getPrototypeOf()` method must return an object or `null`, representing the prototype of the target object.

Description

Interceptions

This trap can intercept these operations:

- `Object.getPrototypeOf()`
- `Reflect.getPrototypeOf()`
- `__proto__`
- `Object.prototype.isPrototypeOf()`
- `instanceof`

Or any other operation that invokes the `[[GetPrototypeOf]]` internal method.

Invariants

The proxy's `[[GetPrototypeOf]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result must be either an `Object` or `null`.
- If the target object is not extensible (that is, `Reflect.isExtensible()` returns `false` on `target`), the result must be the same as the result of `Reflect.getPrototypeOf(target)`.

Examples

Basic usage

```
const obj = {};
const proto = {};
const handler = {
  getPrototypeOf(target) {
    console.log(target === obj); // true
    console.log(this === handler); // true
    return proto;
  },
};

const p = new Proxy(obj, handler);
console.log(Object.getPrototypeOf(p) === proto); // true
```

Five ways to trigger the getPrototypeOf trap

```
const obj = {};
const p = new Proxy(obj, {
  getPrototypeOf(target) {
    return Array.prototype;
  },
});
console.log(
  Object.getPrototypeOf(p) === Array.prototype, // true
  Reflect.getPrototypeOf(p) === Array.prototype, // true
  p.__proto__ === Array.prototype, // true
  Array.prototype.isPrototypeOf(p), // true
  p instanceof Array, // true
);
```

Two kinds of exceptions

```
const obj = {};
const p = new Proxy(obj, {
  getPrototypeOf(target) {
    return "foo";
  },
});
Object.getPrototypeOf(p); // TypeError: "foo" is not an object or null

const obj2 = Object.preventExtensions({});
const p2 = new Proxy(obj2, {
  getPrototypeOf(target) {
    return {};
  },
});
Object.getPrototypeOf(p2); // TypeError: expected same prototype value
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.getPrototypeOf()`
- `Reflect.getPrototypeOf()`

handler.has()

The **`handler.has()`** method is a trap for the `[[HasProperty]]` object internal method, which is used by operations such as the `in` operator.

Try it

```
const handler = {
  has(target, key) {
    if (key[0] === "_") {
      return false;
    }
    return key in target;
  },
};

const monster = {
  _secret: "easily scared",
  eyeCount: 4,
};

const proxy = new Proxy(monster, handler);
console.log("eyeCount" in proxy);
// Expected output: true

console.log("_secret" in proxy);
// Expected output: false

console.log("_secret" in monster);
// Expected output: true
```

Syntax

```
new Proxy(target, {
  has(target, property) {
  }
})
```

Parameters

The following parameters are passed to `has()` method. `this` is bound to the handler.

`target`

: The target object.

`property`

: A string or `Symbol` representing the property name.

Return value

The `has()` method must return a `Boolean` indicating whether or not the property exists. Other values are coerced to booleans.

Description

Interceptions

This trap can intercept these operations:

- The `in` operator: `foo in proxy`
- `with` check: `with(proxy) { (foo); }`
- `Reflect.has()`

Or any other operation that invokes the `[[HasProperty]]` internal method.

Invariants

The proxy's `[[HasProperty]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- A property cannot be reported as non-existent, if it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false` for the property on `target`, the trap must return `true`.
- A property cannot be reported as non-existent, if it exists as an own property of the target object and the target object is not extensible. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `Reflect.getOwnPropertyDescriptor()` returns a property descriptor for the property on `target`, the trap must return `true`.

Examples

Trapping the in operator

The following code traps the `in` operator.

```
const p = new Proxy(
  {},
  {
    has(target, prop) {
      console.log(`called: ${prop}`);
      return true;
    },
  },
);

console.log("a" in p);
// "called: a"
// true
```

The following code violates an invariant.

```
const obj = { a: 10 };
Object.preventExtensions(obj);

const p = new Proxy(obj, {
  has(target, prop) {
    return false;
  },
});

"a" in p; // TypeError is thrown
```

See also

- `Proxy`
- `Proxy()` constructor
- `in`

- `Reflect.has()`

handler.isExtensible()

The **`handler.isExtensible()`** method is a trap for the `[[IsExtensible]]` object internal method, which is used by operations such as `Object.isExtensible()`.

Try it

```
const monster = {
  canEvolve: true,
};

const handler = {
  isExtensible(target) {
    return Reflect.isExtensible(target);
  },
  preventExtensions(target) {
    target.canEvolve = false;
    return Reflect.preventExtensions(target);
  },
};

const proxy = new Proxy(monster, handler);

console.log(Object.isExtensible(proxy));
// Expected output: true

console.log(monster.canEvolve);
// Expected output: true

Object.preventExtensions(proxy);

console.log(Object.isExtensible(proxy));
// Expected output: false

console.log(monster.canEvolve);
// Expected output: false
```

Syntax

```
new Proxy(target, {
  isExtensible(target) {
  }
})
```

Parameters

The following parameter is passed to the `isExtensible()` method. `this` is bound to the handler.

`target`

The target object.

Return value

The `isExtensible()` method must return a `Boolean` indicating whether or not the target object is extensible. Other values are coerced to booleans.

Description

Interceptions

This trap can intercept these operations:

- `Object.isExtensible()`
- `Reflect.isExtensible()`

Or any other operation that invokes the `[[IsExtensible]]` internal method.

Invariants

The proxy's `[[IsExtensible]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result must be the same as `Reflect.isExtensible()` on the target object.

Examples

Trapping of isExtensible

The following code traps `Object.isExtensible()`.

```
const p = new Proxy(
  {},
  {
    isExtensible(target) {
      console.log("called");
```

```
      return true;
    },
  },
);

console.log(Object.isExtensible(p));
// "called"
// true
```

The following code violates the invariant.

```
const p = new Proxy(
  {},
  {
    isExtensible(target) {
      return false;
    },
  },
);

Object.isExtensible(p); // TypeError is thrown
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.isExtensible()`
- `Reflect.isExtensible()`
- `Reflect.preventExtensions()`

handler.ownKeys()

The **`handler.ownKeys()`** method is a trap for the `[[OwnPropertyKeys]]` object internal method, which is used by operations such as `Object.keys()`, `Reflect.ownKeys()`, etc.

Try it

```
const monster = {
  _age: 111,
  [Symbol("secret")]: "I am scared!",
  eyeCount: 4,
};

const handler = {
  ownKeys(target) {
    return Reflect.ownKeys(target);
  },
};

const proxy = new Proxy(monster, handler);

for (const key of Object.keys(proxy)) {
  console.log(key);
  // Expected output: "_age"
  // Expected output: "eyeCount"
}
```

Syntax

```
new Proxy(target, {
  ownKeys(target) {
  }
})
```

Parameters

The following parameter is passed to the `ownKeys()` method. `this` is bound to the handler.

`target`

The target object.

Return value

The `ownKeys()` method must return an array-like object where each element is either a `String` or a `Symbol` containing no duplicate items.

Description

Interceptions

This trap can intercept these operations:

- `Object.getOwnPropertyNames()`
- `Object.getOwnPropertySymbols()`

- `Object.keys()`
- `Reflect.ownKeys()`

Or any other operation that invokes the `[[OwnPropertyKeys]]` internal method.

Invariants

The proxy's `[[OwnPropertyKeys]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result is an `Object`.
- The list of keys contains no duplicate values.
- The type of each key is either a `String` or a `Symbol`.
- The result list must contain the keys of all non-configurable own properties of the target object. That is, for all keys returned by `Reflect.ownKeys()` on the target object, if the key reports `configurable: false` by `Reflect.getOwnPropertyDescriptor()`, then the key must be included in the result List.
- If the target object is not extensible, then the result list must contain all the keys of the own properties of the target object and no other values. That is, if `Reflect.isExtensible()` returns `false` on `target`, then the result list must contain the same values as the result of `Reflect.ownKeys()` on `target`.

Examples

Trapping of getOwnPropertyNames

The following code traps `Object.getOwnPropertyNames()`.

```
const p = new Proxy(
  {},
  {
    ownKeys(target) {
      console.log("called");
      return ["a", "b", "c"];
    },
  },
);

console.log(Object.getOwnPropertyNames(p));
// "called"
// [ 'a', 'b', 'c' ]
```

The following code violates an invariant.

```
const obj = {};
Object.defineProperty(obj, "a", {
  configurable: false,
  enumerable: true,
  value: 10,
});

const p = new Proxy(obj, {
  ownKeys(target) {
    return [123, 12.5, true, false, undefined, null, {}, []];
  },
});

console.log(Object.getOwnPropertyNames(p));

// TypeError: proxy [[OwnPropertyKeys]] must return an array
// with only string and symbol elements
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.getOwnPropertyNames()`
- `Reflect.ownKeys()`

handler.preventExtensions()

The **`handler.preventExtensions()`** method is a trap for the `[[PreventExtensions]]` object internal method, which is used by operations such as `Object.preventExtensions()`.

Try it

```
const monster = {
  canEvolve: true,
};

const handler = {
  preventExtensions(target) {
    target.canEvolve = false;
    Object.preventExtensions(target);
    return true;
  },
};
```

```
const proxy = new Proxy(monster, handler);

console.log(monster.canEvolve);
// Expected output: true

Object.preventExtensions(proxy);

console.log(monster.canEvolve);
// Expected output: false
```

Syntax

```
new Proxy(target, {
  preventExtensions(target) {
  }
})
```

Parameters

The following parameter is passed to the `preventExtensions()` method. `this` is bound to the handler.

`target`

The target object.

Return value

The `preventExtensions()` method must return a `Boolean` indicating whether or not the operation was successful. Other values are coerced to booleans.

Many operations, including `Object.preventExtensions()`, throw a `TypeError` if the `[[PreventExtensions]]` internal method returns `false`.

Description

Interceptions

This trap can intercept these operations:

- `Object.preventExtensions()`
- `Reflect.preventExtensions()`
- `Object.seal()`
- `Object.freeze()`

Or any other operation that invokes the `[[PreventExtensions]]` internal method.

Invariants

The proxy's `[[PreventExtensions]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result is only `true` if `Reflect.isExtensible()` on the target object returns `false` after calling `handler.preventExtensions()`.

Examples

Trapping of preventExtensions

The following code traps `Object.preventExtensions()`.

```
const p = new Proxy(
  {},
  {
    preventExtensions(target) {
      console.log("called");
      Object.preventExtensions(target);
      return true;
    },
  },
);

console.log(Object.preventExtensions(p));
// "called"
// false
```

The following code violates the invariant.

```
const p = new Proxy(
  {},
  {
    preventExtensions(target) {
      return true;
    },
  },
);

Object.preventExtensions(p); // TypeError is thrown
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.preventExtensions()`
- `Reflect.preventExtensions()`

handler.set()

The **`handler.set()`** method is a trap for the `[[Set]]` object internal method, which is used by operations such as using property accessors to set a property's value.

Try it

```
const monster = { eyeCount: 4 };

const handler = {
  set(obj, prop, value) {
    if (prop === "eyeCount" && value % 2 !== 0) {
      console.log("Monsters must have an even number of eyes");
    } else {
      return Reflect.set(...arguments);
    }
  },
};

const proxy = new Proxy(monster, handler);

proxy.eyeCount = 1;
// Expected output: "Monsters must have an even number of eyes"

console.log(proxy.eyeCount);
// Expected output: 4

proxy.eyeCount = 2;
console.log(proxy.eyeCount);
// Expected output: 2
```

Syntax

```
new Proxy(target, {
  set(target, property, value, receiver) {
  }
})
```

Parameters

The following parameters are passed to the `set()` method. `this` is bound to the handler.

`target`

The target object.

`property`

A string or `Symbol` representing the property name.

`value`

The new value of the property to set.

`receiver`

The `this` value for setters; see `Reflect.set()`. This is usually either the proxy itself or an object that inherits from the proxy.

Return value

The `set()` method must return a `Boolean` indicating whether or not the assignment succeeded. Other values are coerced to booleans.

Many operations, including using property accessors in strict mode, throw a `TypeError` if the `[[Set]]` internal method returns `false`.

Description

Interceptions

This trap can intercept these operations:

- Property assignment: `proxy[foo] = bar` and `proxy.foo = bar`
- `Reflect.set()`

Or any other operation that invokes the `[[Set]]` internal method.

Invariants

The proxy's `[[Set]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- Cannot change the value of a property to be different from the value of the corresponding target object property, if the corresponding target object property is a non-writable, non-configurable own data property. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: false` for the property on `target`, and `value` is different from the `value` attribute in the `target`'s property descriptor, then the trap must return a falsy value.
- Cannot set the value of a property if the corresponding target object property is a non-configurable own accessor property that has an undefined setter. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, set: undefined` for the property on `target`, then the trap must return a falsy value.

Examples

Trap setting of a property value

The following code traps setting a property value.

```
const p = new Proxy(
  {},
  {
    set(target, prop, value, receiver) {
      target[prop] = value;
      console.log(`property set: ${prop} = ${value}`);
      return true;
    },
  },
);

console.log("a" in p); // false

p.a = 10; // "property set: a = 10"
console.log("a" in p); // true
console.log(p.a); // 10
```

See also

- `Proxy`
- `Proxy()` constructor
- `Reflect.set()`

handler.setPrototypeOf()

The **`handler.setPrototypeOf()`** method is a trap for the `[[SetPrototypeOf]]` object internal method, which is used by operations such as `Object.setPrototypeOf()`.

Try it

```
const handler = {
  setPrototypeOf(monster, monsterProto) {
    monster.geneticallyModified = true;
    return false;
  },
};

const monsterProto = {};
const monster = {
  geneticallyModified: false,
};

const proxy = new Proxy(monster, handler);
// Object.setPrototypeOf(proxy, monsterProto); // Throws a TypeError

console.log(Reflect.setPrototypeOf(proxy, monsterProto));
// Expected output: false

console.log(monster.geneticallyModified);
// Expected output: true
```

Syntax

```
new Proxy(target, {
  setPrototypeOf(target, prototype) {
  }
})
```

Parameters

The following parameters are passed to the `setPrototypeOf()` method. `this` is bound to the handler.

`target`

The target object.

`prototype`

The object's new prototype or `null`.

Return value

The `setPrototypeOf()` method must return a `Boolean` indicating whether or not the prototype was successfully changed. Other values are coerced to booleans.

Many operations, including `Object.setPrototypeOf()`, throw a `TypeError` if the `[[SetPrototypeOf]]` internal method returns `false`.

Description

Interceptions

This trap can intercept these operations:

- `Object.setPrototypeOf()`
- `Reflect.setPrototypeOf()`

Or any other operation that invokes the `[[SetPrototypeOf]]` internal method.

Invariants

The proxy's `[[SetPrototypeOf]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- If the target object is not extensible, the prototype cannot be changed. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `prototype` is not the same as the result of `Reflect.getPrototypeOf(target)`, then the trap must return a falsy value.

Examples

If you want to disallow setting a new prototype for your object, your handler's `setPrototypeOf()` method can either return `false`, or it can throw an exception.

Approach 1: Returning false

This approach means that any mutating operation that throws an exception on failure to mutate, must create the exception itself.

For example, `Object.setPrototypeOf()` will create and throw a `TypeError` itself. If the mutation is performed by an operation that *doesn't* ordinarily throw in case of failure, such as `Reflect.setPrototypeOf()`, no exception will be thrown.

```
const handlerReturnsFalse = {
  setPrototypeOf(target, newProto) {
    return false;
  },
};

const newProto = {},
  target = {};

const p1 = new Proxy(target, handlerReturnsFalse);
Object.setPrototypeOf(p1, newProto); // throws a TypeError
Reflect.setPrototypeOf(p1, newProto); // returns false
```

Approach 2: Throwing an Exception

The latter approach will cause *any* operation that attempts to mutate, to throw. This approach is best if you want even non-throwing operations to throw on failure, or you want to throw a custom exception value.

```
const handlerThrows = {
  setPrototypeOf(target, newProto) {
    throw new Error("custom error");
  },
};

const newProto = {},
  target = {};

const p2 = new Proxy(target, handlerThrows);
Object.setPrototypeOf(p2, newProto); // throws new Error("custom error")
Reflect.setPrototypeOf(p2, newProto); // throws new Error("custom error")
```

See also

- `Proxy`
- `Proxy()` constructor
- `Object.setPrototypeOf()`
- `Reflect.setPrototypeOf()`

Proxy.revocable()

The **`Proxy.revocable()`** static method creates a revocable `Proxy` object.

Syntax

```
Proxy.revocable(target, handler)
```

Parameters

`target`

A target object to wrap with `Proxy`. It can be any sort of object, including a native array, a function, or even another proxy.

`handler`

An object whose properties are functions defining the behavior of `proxy` when an operation is performed on it.

Return value

A plain object with the following two properties:

`proxy`

A Proxy object exactly the same as one created with a `new Proxy(target, handler)` call.

`revoke`

A function with no parameters to revoke (switch off) the `proxy`.

Description

The `Proxy.revocable()` factory function is the same as the `Proxy()` constructor, except that in addition to creating a proxy object, it also creates a `revoke` function that can be called to disable the proxy. The proxy object and the `revoke` function are wrapped in a plain object.

The `revoke` function does not take any parameters, nor does it rely on the `this` value. The created `proxy` object is attached to the `revoke` function as a private field that the `revoke` function accesses on itself when called (the existence of the private field is not observable from the outside, but it has implications on how garbage collection happens). The `proxy` object is *not* captured within the closure of the `revoke` function (which will make garbage collection of `proxy` impossible if `revoke` is still alive).

After the `revoke()` function gets called, the proxy becomes unusable: any trap to a handler throws a `TypeError`. Once a proxy is revoked, it remains revoked, and calling `revoke()` again has no effect — in fact, the call to `revoke()` detaches the `proxy` object from the `revoke` function, so the `revoke` function will not be able to access the proxy again at all. If the proxy is not referenced elsewhere, it will then be eligible for garbage collection. The `revoke` function also detaches `target` and `handler` from the `proxy`, so if `target` is not referenced elsewhere, it will also be eligible for garbage collection, even when its proxy is still alive, since there's no longer a way to meaningfully interact with the target object.

Letting users interact with an object through a revocable proxy allows you to control the lifetime of the object exposed to the user — you can make the object garbage-collectable even when the user is still holding a reference to its proxy.

Examples

Using Proxy.revocable()

```
const revocable = Proxy.revocable(
  {},
  {
    get(target, name) {
      return `[[${name}]]`;
    },
  },
);
const proxy = revocable.proxy;
console.log(proxy.foo); // "[[foo]]"

revocable.revoke();

console.log(proxy.foo); // TypeError is thrown
proxy.foo = 1; // TypeError again
delete proxy.foo; // still TypeError
typeof proxy; // "object", typeof doesn't trigger any trap
```

See also

- `Proxy`

RangeError

The **`RangeError`** object indicates an error when a value is not in the set or range of allowed values.

Description

A `RangeError` is thrown when trying to pass a value as an argument to a function that does not allow a range that includes the value.

This can be encountered when:

- passing a value that is not one of the allowed string values to `String.prototype.normalize()`, or
- when attempting to create an array of an illegal length with the `Array` constructor, or
- when passing bad values to the numeric methods `Number.prototype.toExponential()`, `Number.prototype.toFixed()` or `Number.prototype.toPrecision()`.

`RangeError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`RangeError` is a subclass of `Error`.

Constructor

`RangeError()`

Creates a new `RangeError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `RangeError.prototype` and shared by all `RangeError` instances.

`RangeError.prototype.constructor`

The constructor function that created the instance object. For `RangeError` instances, the initial value is the `RangeError` constructor.

`RangeError.prototype.name`

Represents the name for the type of error. For `RangeError.prototype.name`, the initial value is `"RangeError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Using RangeError (for numeric values)

```
function check(n) {
  if (!(n >= -500 && n <= 500)) {
    throw new RangeError("The argument must be between -500 and 500.");
  }
}

try {
  check(2000);
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

Using RangeError (for non-numeric values)

```
function check(value) {
  if (!["apple", "banana", "carrot"].includes(value)) {
    throw new RangeError(
      'The argument must be an "apple", "banana", or "carrot".',
    );
  }
}

try {
  check("cabbage");
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

See also

- `Error`
- `Array`
- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`
- `String.prototype.normalize()`

RangeError() constructor

The **`RangeError()`** constructor creates `RangeError` objects.

Syntax

```
new RangeError()
new RangeError(message)
new RangeError(message, options)
new RangeError(message, fileName)
new RangeError(message, fileName, lineNumber)

RangeError()
RangeError(message)
RangeError(message, options)
RangeError(message, fileName)
RangeError(message, fileName, lineNumber)
```

> **Note:** `RangeError()` can be called with or without `new`. Both create a new `RangeError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception

`lineNumber` Optional

The line number of the code that caused the exception

Examples

Using RangeError (for numeric values)

```
function check(n) {
  if (!(n >= -500 && n <= 500)) {
    throw new RangeError("The argument must be between -500 and 500.");
  }
}

try {
  check(2000);
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

Using RangeError (for non-numeric values)

```
function check(value) {
  if (!["apple", "banana", "carrot"].includes(value)) {
    throw new RangeError(
      'The argument must be an "apple", "banana", or "carrot".',
    );
  }
}

try {
  check("cabbage");
} catch (error) {
  if (error instanceof RangeError) {
    // Handle the error
  }
}
```

See also

- `Error`
- `Array`
- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`
- `String.prototype.normalize()`

ReferenceError

The **`ReferenceError`** object represents an error when a variable that doesn't exist (or hasn't yet been initialized) in the current scope is referenced.

`ReferenceError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`ReferenceError` is a subclass of `Error`.

Constructor

`ReferenceError()`

Creates a new `ReferenceError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `ReferenceError.prototype` and shared by all `ReferenceError` instances.

`ReferenceError.prototype.constructor`

The constructor function that created the instance object. For `ReferenceError` instances, the initial value is the `ReferenceError` constructor.

`ReferenceError.prototype.name`

Represents the name for the type of error. For `ReferenceError.prototype.name`, the initial value is `"ReferenceError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a ReferenceError

```
try {
  let a = undefinedVariable;
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "undefinedVariable is not defined"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

Creating a ReferenceError

```
try {
  throw new ReferenceError("Hello");
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`

ReferenceError() constructor

The **`ReferenceError()`** constructor creates `ReferenceError` objects.

Syntax

```
new ReferenceError()
new ReferenceError(message)
new ReferenceError(message, options)
new ReferenceError(message, fileName)
new ReferenceError(message, fileName, lineNumber)

ReferenceError()
ReferenceError(message)
ReferenceError(message, options)
ReferenceError(message, fileName)
ReferenceError(message, fileName, lineNumber)
```

> **Note:** `ReferenceError()` can be called with or without `new`. Both create a new `ReferenceError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception.

`lineNumber` Optional

The line number of the code that caused the exception

Examples

Catching a ReferenceError

```
try {
  let a = undefinedVariable;
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "undefinedVariable is not defined"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

Creating a ReferenceError

```
try {
  throw new ReferenceError("Hello");
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "ReferenceError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`

Reflect

The **`Reflect`** namespace object contains static methods for invoking interceptable JavaScript object internal methods. The methods are the same as those of proxy handlers.

Description

Unlike most global objects, `Reflect` is not a constructor. You cannot use it with the `new` operator or invoke the `Reflect` object as a function. All properties and methods of `Reflect` are static (just like the `Math` object).

The `Reflect` object provides a collection of static functions which have the same names as the proxy handler methods.

The major use case of `Reflect` is to provide default forwarding behavior in `Proxy` handler traps. A trap is used to intercept an operation on an object — it provides a custom implementation for an object internal method. The `Reflect` API is used to invoke the corresponding internal method. For example, the code below creates a proxy `p` with a `deleteProperty` trap that intercepts the `[[Delete]]` internal method. `Reflect.deleteProperty()` is used to invoke the default `[[Delete]]` behavior on `targetObject` directly. You can replace it with `delete`, but using `Reflect` saves you from having to remember the syntax that each internal method corresponds to.

```
const p = new Proxy(
  {},
  {
    deleteProperty(targetObject, property) {
      // Custom functionality: log the deletion
      console.log("Deleting property:", property);

      // Execute the default introspection behavior
      return Reflect.deleteProperty(targetObject, property);
    },
  },
);
```

The `Reflect` methods also allow finer control of how the internal method is invoked. For example, `Reflect.construct()` is the only way to construct a target function with a specific `new.target` value. If you use the `new` operator to invoke a function, the `new.target` value is always the function itself. This has important effects with subclassing. For another example, `Reflect.get()` allows you to run a getter with a custom `this` value, while property accessors always use the current object as the `this` value.

Nearly every `Reflect` method's behavior can be done with some other syntax or method. Some of these methods have corresponding static methods of the same name on `Object`, although they do have some subtle differences. For the exact differences, see the description for each `Reflect` method.

Static properties

`Reflect[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Reflect"`. This property is used in `Object.prototype.toString()`.

Static methods

`Reflect.apply()`

Calls a `target` function with arguments as specified by the `argumentsList` parameter. See also `Function.prototype.apply()`.

`Reflect.construct()`

The `new` operator as a function. Equivalent to calling `new target(...argumentsList)`. Also provides the option to specify a different prototype.

`Reflect.defineProperty()`

Similar to `Object.defineProperty()`. Returns a boolean that is `true` if the property was successfully defined.

`Reflect.deleteProperty()`

The `delete` operator as a function. Equivalent to calling `delete target[propertyKey]`.

`Reflect.get()`

Returns the value of the property. Works like getting a property from an object (`target[propertyKey]`) as a function.

`Reflect.getOwnPropertyDescriptor()`

Similar to `Object.getOwnPropertyDescriptor()`. Returns a property descriptor of the given property if it exists on the object, `undefined` otherwise.

`Reflect.getPrototypeOf()`

Same as `Object.getPrototypeOf()`.

`Reflect.has()`

Returns a boolean indicating whether the target has the property. Either as own or inherited. Works like the `in` operator as a function.

`Reflect.isExtensible()`

Same as `Object.isExtensible()`. Returns a boolean that is `true` if the target is extensible.

`Reflect.ownKeys()`

Returns an array of the target object's own (not inherited) property keys.

`Reflect.preventExtensions()`

Similar to `Object.preventExtensions()`. Returns a boolean that is `true` if the update was successful.

`Reflect.set()`

A function that assigns values to properties. Returns a boolean that is `true` if the update was successful.

`Reflect.setPrototypeOf()`

A function that sets the prototype of an object. Returns a boolean that is `true` if the update was successful.

Examples

Detecting whether an object contains certain properties

```
const duck = {
  name: "Maurice",
  color: "white",
  greeting() {
    console.log(`Quaaaack! My name is ${this.name}`);
  },
};

Reflect.has(duck, "color");
// true
Reflect.has(duck, "haircut");
// false
```

Returning the object's own keys

```
Reflect.ownKeys(duck);
// [ "name", "color", "greeting" ]
```

Adding a new property to the object

```
Reflect.set(duck, "eyes", "black");
// returns "true" if successful
// "duck" now contains the property "eyes: 'black'"
```

See also

- `Proxy`

Reflect.apply()

The **`Reflect.apply()`** static method calls a target function with arguments as specified.

Try it

```
console.log(Reflect.apply(Math.floor, undefined, [1.75]));
// Expected output: 1

console.log(
  Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]),
);
// Expected output: "hello"

console.log(
  Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index,
);
// Expected output: 4

console.log(Reflect.apply("".charAt, "ponies", [3]));
// Expected output: "i"
```

Syntax

```
Reflect.apply(target, thisArgument, argumentsList)
```

Parameters

`target`

The target function to call.

`thisArgument`

The value of `this` provided for the call to `target`.

`argumentsList`

An array-like object specifying the arguments with which `target` should be called.

Return value

The result of calling the given `target` function with the specified `this` value and arguments.

Exceptions

`TypeError`

Thrown if `target` is not a function or `argumentsList` is not an object.

Description

`Reflect.apply()` provides the reflective semantic of a function call. That is, `Reflect.apply(target, thisArgument, argumentsList)` is semantically equivalent to:

```
Math.floor.apply(null, [1.75]);
Reflect.apply(Math.floor, null, [1.75]);
```

The only differences are:

- `Reflect.apply()` takes the function to call as the `target` parameter instead of the `this` context.
- `Reflect.apply()` throws if `argumentsList` is omitted instead of defaulting to calling with no parameters.

`Reflect.apply()` invokes the `[[Call]]` object internal method of `target`.

Examples

Using Reflect.apply()

```
Reflect.apply(Math.floor, undefined, [1.75]);
// 1;

Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]);
// "hello"

Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index;
// 4

Reflect.apply("".charAt, "ponies", [3]);
// "i"
```

See also

- Polyfill of `Reflect.apply` in `core-js`
- es-shims polyfill of `Reflect.apply`
- `Reflect`
- `Function.prototype.apply()`
- `handler.apply()`

Reflect.construct()

The **`Reflect.construct()`** static method is like the `new` operator, but as a function. It is equivalent to calling `new target(...args)`. It additionally allows you to specify a different `new.target` value.

Try it

```
function func1(a, b, c) {
  this.sum = a + b + c;
}

const args = [1, 2, 3];
const object1 = new func1(...args);
const object2 = Reflect.construct(func1, args);

console.log(object2.sum);
// Expected output: 6

console.log(object1.sum);
// Expected output: 6
```

Syntax

```
Reflect.construct(target, argumentsList)
Reflect.construct(target, argumentsList, newTarget)
```

Parameters

`target`

The target function to call.

`argumentsList`

An array-like object specifying the arguments with which `target` should be called.

`newTarget` Optional

The value of the `new.target` expression inside `target`. Defaults to `target`. Generally (see example), `target` specifies the *logic* to initialize the object, while `newTarget.prototype` specifies the *prototype* of the constructed object.

Return value

A new instance of `target` (or `newTarget`, if present), initialized by `target` as a constructor with the given `argumentsList`.

Exceptions

`TypeError`

Thrown if `target` or `newTarget` is not a constructor, or if `argumentsList` is not an object.

Description

`Reflect.construct()` provides the reflective semantic of a constructor call. That is, `Reflect.construct(target, argumentsList, newTarget)` is semantically equivalent to:

```
new target(...argumentsList);
```

Note that when using the `new` operator, `target` and `newTarget` are always the same constructor — but `Reflect.construct()` allows you to pass a different `new.target` value. Conceptually, `newTarget` is the function on which `new` was called, and `newTarget.prototype` will become the constructed object's prototype, while `target` is the constructor that is actually executed to initialize the object. For example, `new.target` may also be different from the currently executed constructor in class inheritance.

```
class A {
  constructor() {
    console.log(new.target.name);
  }
}
class B extends A {}

new B(); // "B"
```

`Reflect.construct()` allows you to invoke a constructor with a variable number of arguments. (This is also possible with the spread syntax in a normal constructor call.)

```
const obj = new Foo(...args);
const obj = Reflect.construct(Foo, args);
```

`Reflect.construct()` invokes the `[[Construct]]` object internal method of `target`.

Examples

Using Reflect.construct()

```
const d = Reflect.construct(Date, [1776, 6, 4]);
d instanceof Date; // true
```

```
d.getFullYear(); // 1776
```

Changing new.target

If `newTarget` is passed, it changes the value of `new.target` inside the constructor. The constructed object will be an instance of `newTarget`, not `target`.

```
function OneClass() {
  console.log("OneClass executed");
  console.log(`new.target is ${new.target.name}`);
}

function OtherClass() {
  console.log("OtherClass executed");
  console.log(`new.target is ${new.target.name}`);
}

const obj1 = Reflect.construct(OneClass, []);
// Logs:
// OneClass executed
// new.target is OneClass
console.log(obj1 instanceof OneClass); // true

const obj2 = Reflect.construct(OneClass, [], OtherClass);
// Logs:
// OneClass executed
// new.target is OtherClass
console.log(obj2 instanceof OtherClass); // true
console.log(obj2 instanceof OneClass); // false
```

Of course, there's no strong guarantee about the prototype chain of the constructed object, as it depends on the constructor's implementation. For example, if the `target` constructor returns an object, then that object will be the constructed object, regardless of the `newTarget` value. If `target` is a proxy with a `construct` trap, then the trap fully controls the construction process.

```
function OneClass() {
  return { name: "one" };
}

function OtherClass() {
  return { name: "other" };
}

const obj1 = Reflect.construct(OneClass, [], OtherClass);
console.log(obj1.name); // 'one'
console.log(obj1 instanceof OneClass); // false
console.log(obj1 instanceof OtherClass); // false
```

A valid `new.target` should be a constructor function with a `prototype` property, but the latter is not enforced. If the `prototype` property's value is not an object, the initialized object will inherit from `Object.prototype`.

```
function OneClass() {
  console.log("OneClass executed");
  console.log(`new.target is ${new.target.name}`);
}

function OtherClass() {
  console.log("OtherClass executed");
  console.log(`new.target is ${new.target.name}`);
}

OtherClass.prototype = null;

const obj = Reflect.construct(OneClass, [], OtherClass);
// Logs:
// OneClass executed
// new.target is OtherClass
console.log(Object.getPrototypeOf(obj) === Object.prototype); // true
```

Reflect.construct() vs. Object.create()

Prior to the introduction of `Reflect`, objects could be constructed using an arbitrary combination of constructors and prototypes using `Object.create()`.

```
function OneClass() {
  this.name = "one";
}

function OtherClass() {
  this.name = "other";
}

const args = [];
const obj1 = Reflect.construct(OneClass, args, OtherClass);
const obj2 = Object.create(OtherClass.prototype);
OneClass.apply(obj2, args);
```

```
console.log(obj1.name); // 'one'
console.log(obj2.name); // 'one'

console.log(obj1 instanceof OneClass); // false
console.log(obj2 instanceof OneClass); // false

console.log(obj1 instanceof OtherClass); // true
console.log(obj2 instanceof OtherClass); // true
```

However, while the end result is the same, there is one important difference in the process. When using `Object.create()` and `Function.prototype.apply()`, the `new.target` operator will point to `undefined` within the function used as the constructor, since the `new` keyword is not being used to create the object. (In fact, it uses the `apply` semantic, not `construct`, although normal functions happen to operate nearly the same.)

When invoking `Reflect.construct()`, on the other hand, the `new.target` operator will point to the `newTarget` parameter if supplied, or `target` if not.

```
function OneClass() {
  console.log("OneClass");
  console.log(new.target);
}
function OtherClass() {
  console.log("OtherClass");
  console.log(new.target);
}

const obj1 = Reflect.construct(OneClass, args);
// Logs:
// OneClass
// function OneClass { ... }

const obj2 = Reflect.construct(OneClass, args, OtherClass);
// Logs:
// OneClass
// function OtherClass { ... }

const obj3 = Object.create(OtherClass.prototype);
OneClass.apply(obj3, args);
// Output:
//     OneClass
//     undefined
```

See also

- Polyfill of `Reflect.construct` in `core-js`
- `Reflect`
- `new`
- `new.target`
- `handler.construct()`

Reflect.defineProperty()

The **`Reflect.defineProperty()`** static method is like `Object.defineProperty()` but returns a `Boolean`.

Try it

```
const object = {};

if (Reflect.defineProperty(object, "foo", { value: 42 })) {
  console.log("foo created!");
  // Expected output: "foo created!"
} else {
  console.log("problem creating foo");
}

console.log(object.foo);
// Expected output: 42
```

Syntax

```
Reflect.defineProperty(target, propertyKey, attributes)
```

Parameters

`target`

The target object on which to define the property.

`propertyKey`

The name of the property to be defined or modified.

`attributes`

The attributes for the property being defined or modified.

Return value

A boolean indicating whether or not the property was successfully defined.

Exceptions

`TypeError`

Thrown if `target` or `attributes` is not an object.

Description

`Reflect.defineProperty()` provides the reflective semantic of defining an own property on an object. At the very low level, defining a property returns a boolean (as is the case with the proxy handler). `Object.defineProperty()` provides nearly the same semantic, but it throws a `TypeError` if the status is `false` (the operation was unsuccessful), while `Reflect.defineProperty()` directly returns the status.

Many built-in operations would also define own properties on objects. The most significant difference between defining properties and setting them is that setters aren't invoked. For example, class fields directly define properties on the instance without invoking setters.

```
class B extends class A {
  set a(v) {
    console.log("Setter called");
  }
} {
  a = 1; // Nothing logged
}
```

`Reflect.defineProperty()` invokes the `[[DefineOwnProperty]]` object internal method of `target`.

Examples

Using Reflect.defineProperty()

```
const obj = {};
Reflect.defineProperty(obj, "x", { value: 7 }); // true
console.log(obj.x); // 7
```

Checking if property definition has been successful

With `Object.defineProperty()`, which returns an object if successful, or throws a `TypeError` otherwise, you would use a `try...catch` block to catch any error that occurred while defining a property.

Because `Reflect.defineProperty()` returns a Boolean success status, you can just use an `if...else` block here:

```
if (Reflect.defineProperty(target, property, attributes)) {
  // success
} else {
  // failure
}
```

See also

- Polyfill of `Reflect.defineProperty` in `core-js`
- `Reflect`
- `Object.defineProperty()`
- `handler.defineProperty()`

Reflect.deleteProperty()

The **`Reflect.deleteProperty()`** static method is like the `delete` operator, but as a function. It deletes a property from an object.

Try it

```
const object = {
  foo: 42,
};

Reflect.deleteProperty(object, "foo");

console.log(object.foo);
// Expected output: undefined

const array = [1, 2, 3, 4, 5];
Reflect.deleteProperty(array, "3");

console.log(array);
// Expected output: Array [1, 2, 3, <1 empty slot>, 5]
```

Syntax

```
Reflect.deleteProperty(target, propertyKey)
```

Parameters

`target`

The target object on which to delete the property.

`propertyKey`

The name of the property to be deleted.

Return value

A boolean indicating whether or not the property was successfully deleted.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.deleteProperty()` provides the reflective semantic of the `delete` operator. That is, `Reflect.deleteProperty(target, propertyKey)` is semantically equivalent to:

```
delete target.propertyKey;
```

At the very low level, deleting a property returns a boolean (as is the case with the proxy handler). `Reflect.deleteProperty()` directly returns the status, while `delete` would throw a `TypeError` in strict mode if the status is `false`. In non-strict mode, `delete` and `Reflect.deleteProperty()` have the same behavior.

`Reflect.deleteProperty()` invokes the `[[Delete]]` object internal method of `target`.

Examples

Using Reflect.deleteProperty()

```
const obj = { x: 1, y: 2 };
Reflect.deleteProperty(obj, "x"); // true
console.log(obj); // { y: 2 }

const arr = [1, 2, 3, 4, 5];
Reflect.deleteProperty(arr, "3"); // true
console.log(arr); // [1, 2, 3, <1 empty slot>, 5]

// Returns true if no such property exists
Reflect.deleteProperty({}, "foo"); // true

// Returns false if a property is unconfigurable
Reflect.deleteProperty(Object.freeze({ foo: 1 }), "foo"); // false
```

See also

- Polyfill of `Reflect.deleteProperty` in `core-js`
- `Reflect`
- `delete`
- `handler.deleteProperty()`

Reflect.get()

The **`Reflect.get()`** static method is like the property accessor syntax, but as a function.

Try it

```
const object = {
  x: 1,
  y: 2,
};

console.log(Reflect.get(object, "x"));
// Expected output: 1

const array = ["zero", "one"];

console.log(Reflect.get(array, 1));
// Expected output: "one"
```

Syntax

```
Reflect.get(target, propertyKey)
Reflect.get(target, propertyKey, receiver)
```

Parameters

`target`

The target object on which to get the property.

`propertyKey`

The name of the property to get.

`receiver` Optional

The value of `this` provided for the call to `target` if a getter is encountered. Defaults to `target`.

Return value

The value of the property.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.get()` provides the reflective semantic of a property access. That is, `Reflect.get(target, propertyKey, receiver)` is semantically equivalent to:

```
target[propertyKey];
```

Note that in a normal property access, `target` and `receiver` would observably be the same object.

`Reflect.get()` invokes the `[[Get]]` object internal method of `target`.

Examples

Using Reflect.get()

```
// Object
const obj1 = { x: 1, y: 2 };
Reflect.get(obj1, "x"); // 1

// Array
Reflect.get(["zero", "one"], 1); // "one"

// Proxy with a get handler
const obj2 = new Proxy(
  { p: 1 },
  {
    get(t, k, r) {
      return `${k}bar`;
    },
  },
);
Reflect.get(obj2, "foo"); // "foobar"

// Proxy with get handler and receiver
const obj3 = new Proxy(
  { p: 1, foo: 2 },
  {
    get(t, prop, receiver) {
      return `${receiver[prop]}bar`;
    },
  },
);
Reflect.get(obj3, "foo", { foo: 3 }); // "3bar"
```

See also

- Polyfill of `Reflect.get` in `core-js`
- `Reflect`
- Property accessors
- `handler.get()`

Reflect.getOwnPropertyDescriptor()

The **`Reflect.getOwnPropertyDescriptor()`** static method is like `Object.getOwnPropertyDescriptor()`. It returns a property descriptor of the given property if it exists on the object, `undefined` otherwise.

Try it

```
const object = {
  property1: 42,
};

console.log(Reflect.getOwnPropertyDescriptor(object, "property1").value);
// Expected output: 42
```

```
console.log(Reflect.getOwnPropertyDescriptor(object, "property2"));
// Expected output: undefined

console.log(Reflect.getOwnPropertyDescriptor(object, "property1").writable);
// Expected output: true
```

Syntax

```
Reflect.getOwnPropertyDescriptor(target, propertyKey)
```

Parameters

`target`

The target object in which to look for the property.

`propertyKey`

The name of the property to get an own property descriptor for.

Return value

A property descriptor object if the property exists as an own property of `target`; otherwise, `undefined`.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.getOwnPropertyDescriptor()` provides the reflective semantic of retrieving the property descriptor of an object. The only difference with `Object.getOwnPropertyDescriptor()` is how non-object targets are handled. `Reflect.getOwnPropertyDescriptor()` throws a `TypeError` if the target is not an object, while `Object.getOwnPropertyDescriptor()` coerces it to an object.

`Reflect.getOwnPropertyDescriptor()` invokes the `[[GetOwnProperty]]` object internal method of `target`.

Examples

Using Reflect.getOwnPropertyDescriptor()

```
Reflect.getOwnPropertyDescriptor({ x: "hello" }, "x");
// {value: "hello", writable: true, enumerable: true, configurable: true}

Reflect.getOwnPropertyDescriptor({ x: "hello" }, "y");
// undefined

Reflect.getOwnPropertyDescriptor([], "length");
// {value: 0, writable: true, enumerable: false, configurable: false}
```

Difference with Object.getOwnPropertyDescriptor()

If the `target` argument to this method is not an object (a primitive), then it will cause a `TypeError`. With `Object.getOwnPropertyDescriptor`, a non-object first argument will be coerced to an object at first.

```
Reflect.getOwnPropertyDescriptor("foo", 0);
// TypeError: "foo" is not non-null object

Object.getOwnPropertyDescriptor("foo", 0);
// { value: "f", writable: false, enumerable: true, configurable: false }
```

See also

- Polyfill of `Reflect.getOwnPropertyDescriptor` in `core-js`
- `Reflect`
- `Object.getOwnPropertyDescriptor()`
- `handler.getOwnPropertyDescriptor()`

Reflect.getPrototypeOf()

The **`Reflect.getPrototypeOf()`** static method is like `Object.getPrototypeOf()`. It returns the prototype of the specified object.

Try it

```
const object = {
  foo: 42,
};

const proto = Reflect.getPrototypeOf(object);

console.log(proto);
// Expected output: Object {  }
```

```
console.log(Reflect.getPrototypeOf(proto));
// Expected output: null
```

Syntax

```
Reflect.getPrototypeOf(target)
```

Parameters

`target`

The target object of which to get the prototype.

Return value

The prototype of the given object, which may be an object or `null`.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.getPrototypeOf()` provides the reflective semantic of retrieving the prototype of an object. The only difference with `Object.getPrototypeOf()` is how non-object targets are handled. `Reflect.getPrototypeOf()` throws a `TypeError` if the target is not an object, while `Object.getPrototypeOf()` coerces it to an object.

`Reflect.getPrototypeOf()` invokes the `[[GetPrototypeOf]]` object internal method of `target`.

Examples

Using Reflect.getPrototypeOf()

```
Reflect.getPrototypeOf({}); // Object.prototype
Reflect.getPrototypeOf(Object.prototype); // null
Reflect.getPrototypeOf(Object.create(null)); // null
```

Difference with Object.getPrototypeOf()

```
// Same result for Objects
Object.getPrototypeOf({}); // Object.prototype
Reflect.getPrototypeOf({}); // Object.prototype

// Both throw in ES5 for non-Objects
Object.getPrototypeOf("foo"); // Throws TypeError
Reflect.getPrototypeOf("foo"); // Throws TypeError

// In ES2015 only Reflect throws, Object coerces non-Objects
Object.getPrototypeOf("foo"); // String.prototype
Reflect.getPrototypeOf("foo"); // Throws TypeError

// To mimic the Object ES2015 behavior you need to coerce
Reflect.getPrototypeOf(Object("foo")); // String.prototype
```

See also

- Polyfill of `Reflect.getPrototypeOf` in `core-js`
- es-shims polyfill of `Reflect.getPrototypeOf`
- `Reflect`
- `Object.getPrototypeOf()`
- `handler.getPrototypeOf()`

Reflect.has()

The **`Reflect.has()`** static method is like the `in` operator, but as a function.

Try it

```
const object = {
  property1: 42,
};

console.log(Reflect.has(object, "property1"));
// Expected output: true

console.log(Reflect.has(object, "property2"));
// Expected output: false

console.log(Reflect.has(object, "toString"));
// Expected output: true
```

Syntax

```
Reflect.has(target, propertyKey)
```

Parameters

`target`

The target object in which to look for the property.

`propertyKey`

The name of the property to check.

Return value

A `Boolean` indicating whether or not the `target` has the property.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.has()` provides the reflective semantic of checking if a property is in an object. That is, `Reflect.has(target, propertyKey)` is semantically equivalent to:

```
propertyKey in target;
```

`Reflect.has()` invokes the `[[HasProperty]]` object internal method of `target`.

Examples

Using Reflect.has()

```
Reflect.has({ x: 0 }, "x"); // true
Reflect.has({ x: 0 }, "y"); // false

// returns true for properties in the prototype chain
Reflect.has({ x: 0 }, "toString");

// Proxy with .has() handler method
obj = new Proxy(
  {},
  {
    has(t, k) {
      return k.startsWith("door");
    },
  },
);
Reflect.has(obj, "doorbell"); // true
Reflect.has(obj, "dormitory"); // false
```

`Reflect.has` returns `true` for any inherited properties, like the `in` operator:

```
const a = { foo: 123 };
const b = { __proto__: a };
const c = { __proto__: b };
// The prototype chain is: c -> b -> a
Reflect.has(c, "foo"); // true
```

See also

- Polyfill of `Reflect.has` in `core-js`
- `Reflect`
- `in`
- `handler.has()`

Reflect.isExtensible()

The **`Reflect.isExtensible()`** static method is like `Object.isExtensible()`. It determines if an object is extensible (whether it can have new properties added to it).

Try it

```
const object1 = {};

console.log(Reflect.isExtensible(object1));
// Expected output: true

Reflect.preventExtensions(object1);

console.log(Reflect.isExtensible(object1));
```

```
// Expected output: false

const object2 = Object.seal({});

console.log(Reflect.isExtensible(object2));
// Expected output: false
```

Syntax

```
Reflect.isExtensible(target)
```

Parameters

`target`

The target object which to check if it is extensible.

Return value

A `Boolean` indicating whether or not the target is extensible.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.isExtensible()` provides the reflective semantic of checking if an object is extensible. The only difference with `Object.isExtensible()` is how non-object targets are handled. `Reflect.isExtensible()` throws a `TypeError` if the target is not an object, while `Object.isExtensible()` always returns `false` for non-object targets.

`Reflect.isExtensible()` invokes the `[[IsExtensible]]` object internal method of `target`.

Examples

Using Reflect.isExtensible()

See also `Object.isExtensible()`.

```
// New objects are extensible.
const empty = {};
Reflect.isExtensible(empty); // true

// … but that can be changed.
Reflect.preventExtensions(empty);
Reflect.isExtensible(empty); // false

// Sealed objects are by definition non-extensible.
const sealed = Object.seal({});
Reflect.isExtensible(sealed); // false

// Frozen objects are also by definition non-extensible.
const frozen = Object.freeze({});
Reflect.isExtensible(frozen); // false
```

Difference with Object.isExtensible()

If the `target` argument to this method is not an object (a primitive), then it will cause a `TypeError`. With `Object.isExtensible()`, a non-object `target` will return false without any errors.

```
Reflect.isExtensible(1);
// TypeError: 1 is not an object

Object.isExtensible(1);
// false
```

See also

- Polyfill of `Reflect.isExtensible` in `core-js`
- `Reflect`
- `Object.isExtensible()`
- `handler.isExtensible()`

Reflect.ownKeys()

The **`Reflect.ownKeys()`** static method returns an array of the `target` object's own property keys.

Try it

```
const object = {
  property1: 42,
  property2: 13,
```

```
};

const array = [];

console.log(Reflect.ownKeys(object));
// Expected output: Array ["property1", "property2"]

console.log(Reflect.ownKeys(array));
// Expected output: Array ["length"]
```

Syntax

```
Reflect.ownKeys(target)
```

Parameters

`target`

The target object from which to get the own keys.

Return value

An `Array` of the `target` object's own property keys, including strings and symbols. For most objects, the array will be in the order of:

1. Non-negative integer indexes in increasing numeric order (but as strings)
2. Other string keys in the order of property creation
3. Symbol keys in the order of property creation.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.ownKeys()` provides the reflective semantic of retrieving all property keys of an object. It is the only way to get all own properties – enumerable and not enumerable, strings and symbols — in one call, without extra filtering logic. For example, `Object.getOwnPropertyNames()` takes the return value of `Reflect.ownKeys()` and filters to only string values, while `Object.getOwnPropertySymbols()` filters to only symbol values. Because normal objects implement `[[OwnPropertyKeys]]` to return all string keys before symbol keys, `Reflect.ownKeys(target)` is usually equivalent to `Object.getOwnPropertyNames(target).concat(Object.getOwnPropertySymbols(target))`. However, if the object has a custom `[[OwnPropertyKeys]]` method (such as through a proxy's `ownKeys` handler), the order of the keys may be different.

`Reflect.ownKeys()` invokes the `[[OwnPropertyKeys]]` object internal method of `target`.

Examples

Using Reflect.ownKeys()

```
Reflect.ownKeys({ z: 3, y: 2, x: 1 }); // [ "z", "y", "x" ]
Reflect.ownKeys([]); // ["length"]

const sym = Symbol.for("comet");
const sym2 = Symbol.for("meteor");
const obj = {
  [sym]: 0,
  str: 0,
  773: 0,
  0: 0,
  [sym2]: 0,
  "-1": 0,
  8: 0,
  "second str": 0,
};
Reflect.ownKeys(obj);
// [ "0", "8", "773", "str", "-1", "second str", Symbol(comet), Symbol(meteor) ]
// Indexes in numeric order,
// strings in insertion order,
// symbols in insertion order
```

See also

- Polyfill of `Reflect.ownKeys` in `core-js`
- es-shims polyfill of `Reflect.ownKeys`
- `Reflect`
- `Object.getOwnPropertyNames()`
- `Object.getOwnPropertySymbols()`
- `handler.ownKeys()`

Reflect.preventExtensions()

The **`Reflect.preventExtensions()`** static method is like `Object.preventExtensions()`. It prevents new properties from ever being added to an object (i.e., prevents future extensions to the object).

Try it

```
const object = {};

console.log(Reflect.isExtensible(object));
// Expected output: true

Reflect.preventExtensions(object);

console.log(Reflect.isExtensible(object));
// Expected output: false
```

Syntax

```
Reflect.preventExtensions(target)
```

Parameters

`target`

The target object on which to prevent extensions.

Return value

A `Boolean` indicating whether or not the target was successfully set to prevent extensions.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.preventExtensions()` provides the reflective semantic of preventing extensions of an object. The differences with `Object.preventExtensions()` are:

- `Reflect.preventExtensions()` throws a `TypeError` if the target is not an object, while `Object.preventExtensions()` always returns non-object targets as-is.
- `Reflect.preventExtensions()` returns a `Boolean` indicating whether or not the target was successfully set to prevent extensions, while `Object.preventExtensions()` returns the target object.

`Reflect.preventExtensions()` invokes the `[[PreventExtensions]]` object internal method of `target`.

Examples

Using Reflect.preventExtensions()

See also `Object.preventExtensions()`.

```
// Objects are extensible by default.
const empty = {};
Reflect.isExtensible(empty); // true

// … but that can be changed.
Reflect.preventExtensions(empty);
Reflect.isExtensible(empty); // false
```

Difference with Object.preventExtensions()

If the `target` argument to this method is not an object (a primitive), then it will cause a `TypeError`. With `Object.preventExtensions()`, a non-object `target` will be returned as-is without any errors.

```
Reflect.preventExtensions(1);
// TypeError: 1 is not an object

Object.preventExtensions(1);
// 1
```

See also

- Polyfill of `Reflect.preventExtensions` in `core-js`
- `Reflect`
- `Object.preventExtensions()`
- `handler.preventExtensions()`

Reflect.set()

The **`Reflect.set()`** static method is like the property accessor and assignment syntax, but as a function.

Try it

```
const object = {};
Reflect.set(object, "foo", 42);
```

```
console.log(object.foo);
// Expected output: 42

const array = ["duck", "duck", "duck"];
Reflect.set(array, 2, "goose");

console.log(array[2]);
// Expected output: "goose"
```

Syntax

```
Reflect.set(target, propertyKey, value)
Reflect.set(target, propertyKey, value, receiver)
```

Parameters

`target`

The target object on which to set the property.

`propertyKey`

The name of the property to set.

`value`

The value to set.

`receiver` Optional

The value of `this` provided for the call to the setter for `propertyKey` on `target`. If provided and `target` does not have a setter for `propertyKey`, the property will be set on `receiver` instead. Defaults to `target`.

Return value

A `Boolean` indicating whether or not setting the property was successful.

Exceptions

`TypeError`

Thrown if `target` is not an object.

Description

`Reflect.set()` provides the reflective semantic of a property access. That is, `Reflect.set(target, propertyKey, value, receiver)` is semantically equivalent to:

```
target[propertyKey] = value;
```

Note that in a normal property access, `target` and `receiver` would observably be the same object.

`Reflect.set()` invokes the `[[Set]]` object internal method of `target`.

Examples

Using Reflect.set()

```
// Object
const obj = {};
Reflect.set(obj, "prop", "value"); // true
obj.prop; // "value"

// Array
const arr = ["duck", "duck", "duck"];
Reflect.set(arr, 2, "goose"); // true
arr[2]; // "goose"

// It can truncate an array.
Reflect.set(arr, "length", 1); // true
arr; // ["duck"]

// With just one argument, propertyKey and value are "undefined".
Reflect.set(obj); // true
Reflect.getOwnPropertyDescriptor(obj, "undefined");
// { value: undefined, writable: true, enumerable: true, configurable: true }
```

Different target and receiver

When the `target` and `receiver` are different, `Reflect.set` will use the property descriptor of `target` (to find the setter or determine if the property is writable), but set the property on `receiver`.

```
const target = {};
const receiver = {};
```

```
Reflect.set(target, "a", 2, receiver); // true
// target is {}; receiver is { a: 2 }

const target = { a: 1 };
const receiver = {};
Reflect.set(target, "a", 2, receiver); // true
// target is { a: 1 }; receiver is { a: 2 }

const target = {
  set a(v) {
    this.b = v;
  },
};
const receiver = {};
Reflect.set(target, "a", 2, receiver); // true
// target is { a: [Setter] }; receiver is { b: 2 }
```

See also

- Polyfill of `Reflect.set` in `core-js`
- `Reflect`
- Property accessors
- `handler.set()`

Reflect.setPrototypeOf()

The **`Reflect.setPrototypeOf()`** static method is like `Object.setPrototypeOf()` but returns a `Boolean`. It sets the prototype (i.e., the internal `[[Prototype]]` property) of a specified object.

Try it

```
const object1 = {};

console.log(Reflect.setPrototypeOf(object1, Object.prototype));
// Expected output: true

console.log(Reflect.setPrototypeOf(object1, null));
// Expected output: true

const object2 = {};

console.log(Reflect.setPrototypeOf(Object.freeze(object2), null));
// Expected output: false
```

Syntax

```
Reflect.setPrototypeOf(target, prototype)
```

Parameters

`target`

The target object of which to set the prototype.

`prototype`

The object's new prototype (an object or `null`).

Return value

A `Boolean` indicating whether or not the prototype was successfully set.

Exceptions

`TypeError`

Thrown if `target` is not an object or if `prototype` is neither an object nor `null`.

Description

`Reflect.setPrototypeOf()` provides the reflective semantic of setting the prototype of an object. At the very low level, setting the prototype returns a boolean (as is the case with the proxy handler). `Object.setPrototypeOf()` provides nearly the same semantic, but it throws a `TypeError` if the status is `false` (the operation was unsuccessful), while `Reflect.setPrototypeOf()` directly returns the status.

`Reflect.setPrototypeOf()` invokes the `[[SetPrototypeOf]]` object internal method of `target`.

Examples

Using Reflect.setPrototypeOf()

```
Reflect.setPrototypeOf({}, Object.prototype); // true

// It can change an object's [[Prototype]] to null.
Reflect.setPrototypeOf({}, null); // true
```

```
// Returns false if target is not extensible.
Reflect.setPrototypeOf(Object.freeze({}), null); // false

// Returns false if it cause a prototype chain cycle.
const target = {};
const proto = Object.create(target);
Reflect.setPrototypeOf(target, proto); // false
```

See also

- Polyfill of `Reflect.setPrototypeOf` in `core-js`
- `Reflect`
- `Object.setPrototypeOf()`
- `handler.setPrototypeOf()`

RegExp

The **`RegExp`** object is used for matching text with a pattern.

For an introduction to regular expressions, read the Regular expressions chapter in the JavaScript guide. For detailed information of regular expression syntax, read the regular expression reference.

Description

Literal notation and constructor

There are two ways to create a `RegExp` object: a *literal notation* and a *constructor*.

- The *literal notation* takes a pattern between two slashes, followed by optional flags, after the second slash.
- The *constructor function* takes either a string or a `RegExp` object as its first parameter and a string of optional flags as its second parameter.

The following three expressions create the same regular expression object:

```
const re = /ab+c/i; // literal notation
// OR
const re = new RegExp("ab+c", "i"); // constructor with string pattern as first argument
// OR
const re = new RegExp(/ab+c/, "i"); // constructor with regular expression literal as first argument
```

Before regular expressions can be used, they have to be compiled. This process allows them to perform matches more efficiently. More about the process can be found in dotnet docs.

The literal notation results in compilation of the regular expression when the expression is evaluated. On the other hand, the constructor of the `RegExp` object, `new RegExp('ab+c')`, results in runtime compilation of the regular expression.

Use a string as the first argument to the `RegExp()` constructor when you want to build the regular expression from dynamic input.

Flags in constructor

The expression `new RegExp(/ab+c/, flags)` will create a new `RegExp` using the source of the first parameter and the flags provided by the second.

When using the constructor function, the normal string escape rules (preceding special characters with `\` when included in a string) are necessary.

For example, the following are equivalent:

```
const re = /\w+/;
// OR
const re = new RegExp("\\w+");
```

Special handling for regexes

> **Note:** Whether something is a "regex" can be duck-typed. It doesn't have to be a `RegExp`!

Some built-in methods would treat regexes specially. They decide whether `x` is a regex through multiple steps:

1. `x` must be an object (not a primitive).
2. If `x[Symbol.match]` is not `undefined`, check if it's truthy.
3. Otherwise, if `x[Symbol.match]` is `undefined`, check if `x` had been created with the `RegExp` constructor. (This step should rarely happen, since if `x` is a `RegExp` object that have not been tampered with, it should have a `Symbol.match` property.)

Note that in most cases, it would go through the `Symbol.match` check, which means:

- An actual `RegExp` object whose `Symbol.match` property's value is falsy but not `undefined` (even with everything else intact, like `exec` and `[Symbol.replace]()`) can be used as if it's not a regex.
- A non-`RegExp` object with a `Symbol.match` property will be treated as if it's a regex.

This choice was made because `[Symbol.match]()` is the most indicative property that something is intended to be used for matching. (`exec` could also be used, but because it's not a symbol property, there would be too many false positives.) The places that treat regexes specially include:

- `String.prototype.endsWith()`, `startsWith()`, and `includes()` throw a `TypeError` if the first argument is a regex.

- `String.prototype.matchAll()` and `replaceAll()` check whether the global flag is set if the first argument is a regex before invoking its `[Symbol.matchAll]()` or `[Symbol.replace]()` method.
- The `RegExp()` constructor directly returns the `pattern` argument only if `pattern` is a regex (among a few other conditions). If `pattern` is a regex, it would also interrogate `pattern`'s `source` and `flags` properties instead of coercing `pattern` to a string.

For example, `String.prototype.endsWith()` would coerce all inputs to strings, but it would throw if the argument is a regex, because it's only designed to match strings, and using a regex is likely a developer mistake.

```
"foobar".endsWith({ toString: () => "bar" }); // true
"foobar".endsWith(/bar/); // TypeError: First argument to String.prototype.endsWith must not be a regular expression
```

You can get around the check by setting `[Symbol.match]` to a falsy value that's not `undefined`. This would mean that the regex cannot be used for `String.prototype.match()` (since without `[Symbol.match]`, `match()` would construct a new `RegExp` object with the two enclosing slashes added by `re.toString()`), but it can be used for virtually everything else.

```
const re = /bar/g;
re[Symbol.match] = false;
"/bar/g".endsWith(re); // true
re.exec("bar"); // [ 'bar', index: 0, input: 'bar', groups: undefined ]
"bar & bar".replace(re, "foo"); // 'foo & foo'
```

Perl-like RegExp properties

Note that several of the `RegExp` properties have both long and short (Perl-like) names. Both names always refer to the same value. (Perl is the programming language from which JavaScript modeled its regular expressions.) See also deprecated `RegExp` properties.

Constructor

`RegExp()`

Creates a new `RegExp` object.

Static properties

`RegExp.$1, ..., RegExp.$9`

Static read-only properties that contain parenthesized substring matches.

`RegExp.input ($_)`

A static property that contains the last string against which a regular expression was successfully matched.

`RegExp.lastMatch ($&)`

A static read-only property that contains the last matched substring.

`RegExp.lastParen ($+)`

A static read-only property that contains the last parenthesized substring match.

`RegExp.leftContext ($`)`

A static read-only property that contains the substring preceding the most recent match.

`RegExp.rightContext ($')`

A static read-only property that contains the substring following the most recent match.

`RegExp[Symbol.species]`

The constructor function that is used to create derived objects.

Static methods

`RegExp.escape()`

Escapes any potential regex syntax characters in a string, and returns a new string that can be safely used as a literal pattern for the `RegExp()` constructor.

Instance properties

These properties are defined on `RegExp.prototype` and shared by all `RegExp` instances.

`RegExp.prototype.constructor`

The constructor function that created the instance object. For `RegExp` instances, the initial value is the `RegExp` constructor.

`RegExp.prototype.dotAll`

Whether `.` matches newlines or not.

`RegExp.prototype.flags`

A string that contains the flags of the `RegExp` object.

`RegExp.prototype.global`

Whether to test the regular expression against all possible matches in a string, or only against the first.

`RegExp.prototype.hasIndices`

Whether the regular expression result exposes the start and end indices of captured substrings.

`RegExp.prototype.ignoreCase`

Whether to ignore case while attempting a match in a string.

`RegExp.prototype.multiline`

Whether or not to search in strings across multiple lines.

`RegExp.prototype.source`

The text of the pattern.

`RegExp.prototype.sticky`

Whether or not the search is sticky.

`RegExp.prototype.unicode`

Whether or not Unicode features are enabled.

`RegExp.prototype.unicodeSets`

Whether or not the `v` flag, an upgrade to the `u` mode, is enabled.

These properties are own properties of each `RegExp` instance.

`lastIndex`

The index at which to start the next match.

Instance methods

`RegExp.prototype.compile()`

(Re-)compiles a regular expression during execution of a script.

`RegExp.prototype.exec()`

Executes a search for a match in its string parameter.

`RegExp.prototype.test()`

Tests for a match in its string parameter.

`RegExp.prototype.toString()`

Returns a string representing the specified object. Overrides the `Object.prototype.toString()` method.

`RegExp.prototype[Symbol.match]()`

Performs match to given string and returns match result.

`RegExp.prototype[Symbol.matchAll]()`

Returns all matches of the regular expression against a string.

`RegExp.prototype[Symbol.replace]()`

Replaces matches in given string with new substring.

`RegExp.prototype[Symbol.search]()`

Searches the match in given string and returns the index the pattern found in the string.

`RegExp.prototype[Symbol.split]()`

Splits given string into an array by separating the string into substrings.

Examples

Using a regular expression to change data format

The following script uses the `String.prototype.replace()` method to match a name in the format *first last* and output it in the format *last, first*.

In the replacement text, the script uses `$1` and `$2` to indicate the results of the corresponding matching parentheses in the regular expression pattern.

```
const re = /(\w+)\s(\w+)/;
const str = "Maria Cruz";
```

```
const newStr = str.replace(re, "$2, $1");
console.log(newStr);
```

This displays `"Cruz, Maria"`.

Using regular expression to split lines with different line endings/ends of line/line breaks

The default line ending varies depending on the platform (Unix, Windows, etc.). The line splitting provided in this example works on all platforms.

```
const text = "Some text\nAnd some more\r\nAnd yet\nThis is the end";
const lines = text.split(/\r?\n/);
console.log(lines); // [ 'Some text', 'And some more', 'And yet', 'This is the end' ]
```

Note that the order of the patterns in the regular expression matters.

Using regular expression on multiple lines

By default, the `.` character does not match newlines. To make it match newlines, use the `s` flag (`dotAll` mode).

```
const s = "Please yes\nmake my day!";

s.match(/yes.*day/);
// Returns null

s.match(/yes.*day/s);
// Returns ["yes\nmake my day"]
```

Using a regular expression with the sticky flag

The `sticky` flag indicates that the regular expression performs sticky matching in the target string by attempting to match starting at `RegExp.prototype.lastIndex`.

```
const str = "#foo#";
const regex = /foo/y;

regex.lastIndex = 1;
regex.test(str); // true
regex.lastIndex = 5;
regex.test(str); // false (lastIndex is taken into account with sticky flag)
regex.lastIndex; // 0 (reset after match failure)
```

The difference between the sticky flag and the global flag

With the sticky flag `y`, the next match has to happen at the `lastIndex` position, while with the global flag `g`, the match can happen at the `lastIndex` position or later:

```
const re = /\d/y;
let r;
while ((r = re.exec("123 456"))) {
  console.log(r, "AND re.lastIndex", re.lastIndex);
}

// [ '1', index: 0, input: '123 456', groups: undefined ] AND re.lastIndex 1
// [ '2', index: 1, input: '123 456', groups: undefined ] AND re.lastIndex 2
// [ '3', index: 2, input: '123 456', groups: undefined ] AND re.lastIndex 3
//  … and no more match.
```

With the global flag `g`, all 6 digits would be matched, not just 3.

Regular expression and Unicode characters

`\w` and `\W` only matches ASCII based characters; for example, `a` to `z`, `A` to `Z`, `0` to `9`, and `_`.

To match characters from other languages such as Cyrillic or Hebrew, use `\uHHHH`, where `HHHH` is the character's Unicode value in hexadecimal.

This example demonstrates how one can separate out Unicode characters from a word.

```
const text = "Образец text на русском языке";
const regex = /[\u0400-\u04ff]+/g;

const match = regex.exec(text);
console.log(match[0]); // 'Образец'
console.log(regex.lastIndex); // 7

const match2 = regex.exec(text);
console.log(match2[0]); // 'на' (did not log 'text')
console.log(regex.lastIndex); // 15

// and so on
```

The Unicode property escapes feature provides a simpler way to target particular Unicode ranges, by allowing for statements like `\p{scx=Cyrl}` (to match any Cyrillic letter), or `\p{L}/u` (to match a letter from any language).

Extracting subdomain name from URL

```
const url = "http://xxx.example.com";
console.log(/^https?:\/\/(.+?)\./.exec(url)[1]); // 'xxx'
```

> **Note:** Instead of using regular expressions for parsing URLs, it is usually better to use the browsers built-in URL parser by using the URL API.

Building a regular expression from dynamic inputs

```
const breakfasts = ["bacon", "eggs", "oatmeal", "toast", "cereal"];
const order = "Let me get some bacon and eggs, please";

order.match(new RegExp(`\\b(${breakfasts.join("|")})\\b`, "g"));
// Returns ['bacon', 'eggs']
```

Firefox-specific notes

Starting with Firefox 34, in the case of a capturing group with quantifiers preventing its exercise, the matched text for a capturing group is now `undefined` instead of an empty string:

```
// Firefox 33 or older
"x".replace(/x(.)?/g, (m, group) => {
  console.log(`group: ${JSON.stringify(group)}`);
});
// group: ""

// Firefox 34 or newer
"x".replace(/x(.)?/g, (m, group) => {
  console.log(`group: ${group}`);
});
// group: undefined
```

Note that due to web compatibility, `RegExp.$N` will still return an empty string instead of `undefined` (bug 1053944).

See also

- Polyfill of many modern `RegExp` features (`dotAll`, `sticky` flags, named capture groups, etc.) in `core-js`
- Regular expressions guide
- Regular expressions
- `String.prototype.match()`
- `String.prototype.replace()`
- `String.prototype.split()`

RegExp.prototype.compile()

> **Note:** The `compile()` method is only specified for compatibility reasons. Using `compile()` causes the otherwise immutable regex source and flags to become mutable, which may break user expectations. You can use the `RegExp()` constructor to construct a new regular expression object instead.

The **`compile()`** method of `RegExp` instances is used to recompile a regular expression with new source and flags after the `RegExp` object has already been created.

Syntax

```
compile(pattern, flags)
```

Parameters

`pattern`

The text of the regular expression.

`flags`

Any combination of flag values.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if the `this` value is not an instance of the current realm's `RegExp` constructor. This includes a subclass of `RegExp` and the `RegExp` constructor from a different realm.

Examples

Using compile()

The following example shows how to recompile a regular expression with a new pattern and a new flag.

```
const regexObj = /foo/gi;
regexObj.compile("new foo", "g");
```

See also

- RegExp

RegExp.prototype.dotAll

The **`dotAll`** accessor property of RegExp instances returns whether or not the `s` flag is used with this regular expression.

Try it

```
const regex1 = /f.o/s;

console.log(regex1.dotAll);
// Expected output: true

const regex2 = /bar/;

console.log(regex2.dotAll);
// Expected output: false
```

Description

`RegExp.prototype.dotAll` has the value `true` if the `s` flag was used; otherwise, `false`. The `s` flag indicates that the dot special character (`.`) should additionally match the following line terminator ("newline") characters in a string, which it would not match otherwise:

- U+000A LINE FEED (LF) (`\n`)
- U+000D CARRIAGE RETURN (CR) (`\r`)
- U+2028 LINE SEPARATOR
- U+2029 PARAGRAPH SEPARATOR

This effectively means the dot will match any UTF-16 code unit. However, it will *not* match characters that are outside of the Unicode Basic Multilingual Plane (BMP), also known as astral characters, which are represented as surrogate pairs and necessitate matching with two `.` patterns instead of one.

```
"😄".match(/(.)(.)/s);
// Array(3) [ "😄", "\ud83d", "\ude04" ]
```

The `u` (unicode) flag can be used to allow the dot to match astral characters as a single character.

```
"😄".match(/./su);
// Array [ "😄" ]
```

Note that a pattern such as `.*` is still capable of *consuming* astral characters as part of a larger context, even without the `u` flag.

```
"😄".match(/.*/s);
// Array [ "😄" ]
```

Using both the `s` and `u` flags in conjunction allows the dot to match any Unicode character in a more intuitive manner.

The set accessor of `dotAll` is `undefined`. You cannot change this property directly.

Examples

Using dotAll

```
const str1 = "bar\nexample foo example";

const regex1 = /bar.example/s;

console.log(regex1.dotAll); // true

console.log(str1.replace(regex1, "")); // foo example

const str2 = "bar\nexample foo example";

const regex2 = /bar.example/;

console.log(regex2.dotAll); // false

console.log(str2.replace(regex2, ""));
// bar
// example foo example
```

See also

- Polyfill of the `dotAll` flag in `core-js`
- `RegExp.prototype.lastIndex`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`

- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

RegExp.escape()

The **`RegExp.escape()`** static method escapes any potential regex syntax characters in a string, and returns a new string that can be safely used as a literal pattern for the `RegExp()` constructor.

When dynamically creating a `RegExp` with user-provided content, consider using this function to sanitize the input (unless the input is actually intended to contain regex syntax). In addition, don't try to re-implement its functionality by, for example, using `String.prototype.replaceAll()` to insert a `\` before all syntax characters. `RegExp.escape()` is designed to use escape sequences that work in many more edge cases/contexts than hand-crafted code is likely to achieve.

Syntax

```
RegExp.escape(string)
```

Parameters

`string`

The string to escape.

Return value

A new string that can be safely used as a literal pattern for the `RegExp()` constructor. Namely, the following things in the input string are replaced:

- The first character of the string, if it's either a decimal digit (0–9) or ASCII letter (a–z, A–Z), is escaped using the `\x` character escape syntax. For example, `RegExp.escape("foo")` returns `"\\x66oo"` (here and after, the two backslashes in a string literal denote a single backslash character). This step ensures that if this escaped string is embedded into a bigger pattern where it's immediately preceded by `\1`, `\x0`, `\u000`, etc., the leading character doesn't get interpreted as part of the escape sequence.
- Regex syntax characters, including `^`, `$`, `\`, `.`, `*`, `+`, `?`, `(`, `)`, `[`, `]`, `{`, `}`, and `|`, as well as the `/` delimiter, are escaped by inserting a `\` character before them. For example, `RegExp.escape("foo.bar")` returns `"\\x66oo\\.bar"`, and `RegExp.escape("(foo)")` returns `"\\(foo\\)"`.
- Other punctuators, including `,`, `-`, `=`, `<`, `>`, `#`, `&`, `!`, `%`, `:`, `;`, `@`, `~`, `'`, `` ` ``, and `"`, are escaped using the `\x` syntax. For example, `RegExp.escape("foo-bar")` returns `"\\x66oo\\x2dbar"`. These characters cannot be escaped by prefixing with `\` because, for example, `/foo\-bar/u` is a syntax error.
- The characters with their own character escape sequences: `\f` (U+000C FORM FEED), `\n` (U+000A LINE FEED), `\r` (U+000D CARRIAGE RETURN), `\t` (U+0009 CHARACTER TABULATION), and `\v` (U+000B LINE TABULATION), are replaced with their escape sequences. For example, `RegExp.escape("foo\nbar")` returns `"\\x66oo\\nbar"`.
- The space character is escaped as `"\\x20"`.
- Other non-ASCII line break and white space characters are replaced with one or two `\uXXXX` escape sequences representing their UTF-16 code units. For example, `RegExp.escape("foo\u2028bar")` returns `"\\x66oo\\u2028bar"`.
- Lone surrogates are replaced with their `\uXXXX` escape sequences. For example, `RegExp.escape("foo\uD800bar")` returns `"\\x66oo\\ud800bar"`.

Exceptions

`TypeError`

Thrown if `string` is not a string.

Examples

Using RegExp.escape()

The following examples demonstrate various inputs and outputs for the `RegExp.escape()` method.

```
RegExp.escape("Buy it. use it. break it. fix it.");
// "\\x42uy\\x20it\\.\\x20use\\x20it\\.\\x20break\\x20it\\.\\x20fix\\x20it\\."
RegExp.escape("foo.bar"); // "\\x66oo\\.bar"
RegExp.escape("foo-bar"); // "\\x66oo\\x2dbar"
RegExp.escape("foo\nbar"); // "\\x66oo\\nbar"
RegExp.escape("foo\uD800bar"); // "\\x66oo\\ud800bar"
RegExp.escape("foo\u2028bar"); // "\\x66oo\\u2028bar"
```

Using RegExp.escape() with the RegExp constructor

The primary use case of `RegExp.escape()` is when you want to embed a string into a bigger regex pattern, and you want to ensure that the string is treated as a literal pattern, not as a regex syntax. Consider the following naïve example that replaces URLs:

```
function removeDomain(text, domain) {
  return text.replace(new RegExp(`https?://${domain}(?=/)`, "g"), "");
}

const input =
  "Consider using [RegExp.escape()](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/escape) to escape special
characters in a string.";
const domain = "developer.mozilla.org";
console.log(removeDomain(input, domain));
// Consider using [RegExp.escape()](/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp/escape) to escape special characters in a string.
```

Inserting the `domain` above results in the regular expression literal `https?://developer.mozilla.org(?=/)`, where the "." character is a regex wildcard character. This means the string will match the string with any character in place of the ".", such as `developer-mozilla-org`. Therefore, it would incorrectly also change the following text:

```
const input =
  "This is not an MDN link: https://developer-mozilla.org/, be careful!";
const domain = "developer.mozilla.org";
console.log(removeDomain(input, domain));
// This is not an MDN link: /, be careful!
```

To fix this, we can use `RegExp.escape()` to ensure that any user input is treated as a literal pattern:

```
function removeDomain(text, domain) {
  return text.replace(
    new RegExp(`https?://${RegExp.escape(domain)}(?=/)`, "g"),
    "",
  );
}
```

Now this function will do exactly what we intend to, and will not transform `developer-mozilla.org` URLs.

See also

- Polyfill of `RegExp.escape` in `core-js`
- es-shims polyfill of `RegExp.escape`
- `RegExp`

RegExp.prototype.exec()

The **`exec()`** method of `RegExp` instances executes a search with this regular expression for a match in a specified string and returns a result array, or `null`.

Try it

```
const regex = /fo+/g;
const str = "table football, foosball";
let array;

while ((array = regex.exec(str)) !== null) {
  console.log(`Found ${array[0]}. Next starts at ${regex.lastIndex}.`);
  // Expected output: "Found foo. Next starts at 9."
  // Expected output: "Found foo. Next starts at 19."
}
```

Syntax

```
exec(str)
```

Parameters

`str`

> The string against which to match the regular expression. All values are coerced to strings, so omitting it or passing `undefined` causes `exec()` to search for the string `"undefined"`, which is rarely what you want.

Return value

If the match fails, the `exec()` method returns `null`, and sets the regex's `lastIndex` to `0`.

If the match succeeds, the `exec()` method returns an array and updates the `lastIndex` property of the regular expression object. The returned array has the matched text as the first item, and then one item for each capturing group of the matched text. The array also has the following additional properties:

`index`

> The 0-based index of the match in the string.

`input`

> The original string that was matched against.

`groups`

> A `null`-prototype object of named capturing groups, whose keys are the names, and values are the capturing groups, or `undefined` if no named capturing groups were defined. See capturing groups for more information.

`indices` Optional

> This property is only present when the `d` flag is set. It is an array where each entry represents the bounds of a substring match. The index of each element in this array corresponds to the index of the respective substring match in the array returned by `exec()`. In other words, the first `indices` entry represents the entire match, the second `indices` entry represents the first capturing group, etc. Each entry itself is a two-element array, where the first number represents the match's start index, and the second number, its end index.

The `indices` array additionally has a `groups` property, which holds a `null`-prototype object of all named capturing groups. The keys are the names of the capturing groups, and each value is a two-element array, with the first number being the start index, and the second number being the end index of the capturing group. If the regular expression doesn't contain any named capturing groups, `groups` is `undefined`.

Description

JavaScript `RegExp` objects are *stateful* when they have the global or sticky flags set (e.g., `/foo/g` or `/foo/y`). They store a `lastIndex` from the previous match. Using this internally, `exec()` can be used to iterate over multiple matches in a string of text (with capture groups), as opposed to getting just the matching strings with `String.prototype.match()`.

When using `exec()`, the global flag has no effect when the sticky flag is set — the match is always sticky.

`exec()` is the primitive method of regexps. Many other regexp methods call `exec()` internally — including those called by string methods, like `[Symbol.replace]()`. While `exec()` itself is powerful (and is the most efficient), it often does not convey the intent more clearly.

- If you only care whether the regex matches a string, but not what is actually being matched, use `RegExp.prototype.test()` instead.
- If you are finding all occurrences of a global regex and you don't care about information like capturing groups, use `String.prototype.match()` instead. In addition, `String.prototype.matchAll()` helps to simplify matching multiple parts of a string (with capture groups) by allowing you to iterate over the matches.
- If you are executing a match to find its index position in the string, use the `String.prototype.search()` method instead.

`exec()` is useful for complex operations that cannot be easily achieved via any of the methods above, often when you need to manually adjust `lastIndex`. (`String.prototype.matchAll()` copies the regex, so changing `lastIndex` while iterating over `matchAll` does not affect the iteration.) For one such example, see rewinding `lastIndex`.

Examples

Using exec()

Consider the following example:

```
// Match "quick brown" followed by "jumps", ignoring characters in between
// Remember "brown" and "jumps"
// Ignore case
const re = /quick\s(?<color>brown).+?(jumps)/dgi;
const result = re.exec("The Quick Brown Fox Jumps Over The Lazy Dog");
```

The following table shows the state of `result` after running this script:

Property	Value
`[0]`	`"Quick Brown Fox Jumps"`
`[1]`	`"Brown"`
`[2]`	`"Jumps"`
`index`	`4`
`indices`	`[[4, 25], [10, 15], [20, 25]]` `groups: { color: [10, 15 ]}`
`input`	`"The Quick Brown Fox Jumps Over The Lazy Dog"`
`groups`	`{ color: "Brown" }`

In addition, `re.lastIndex` will be set to `25`, due to this regex being global.

Finding successive matches

If your regular expression uses the `g` flag, you can use the `exec()` method multiple times to find successive matches in the same string. When you do so, the search starts at the substring of `str` specified by the regular expression's `lastIndex` property (`test()` will also advance the `lastIndex` property). Note that the `lastIndex` property will not be reset when searching a different string, it will start its search at its existing `lastIndex`.

For example, assume you have this script:

```
const myRe = /ab*/g;
const str = "abbcdefabh";
let myArray;
while ((myArray = myRe.exec(str)) !== null) {
  let msg = `Found ${myArray[0]}. `;
  msg += `Next match starts at ${myRe.lastIndex}`;
  console.log(msg);
}
```

This script displays the following text:

```
Found abb. Next match starts at 3
Found ab. Next match starts at 9
```

> **Warning:** There are many pitfalls that can lead to this becoming an infinite loop!
>
> - Do *not* place the regular expression literal (or `RegExp` constructor) within the `while` condition — it will recreate the regex for every iteration and reset `lastIndex`.
> - Be sure that the global (g) flag is set, or `lastIndex` will never be advanced.

> - If the regex may match zero-length characters (e.g., `/^/gm`), increase its `lastIndex` manually each time to avoid being stuck in the same place.

You can usually replace this kind of code with `String.prototype.matchAll()` to make it less error-prone.

Using exec() with RegExp literals

You can also use `exec()` without creating a `RegExp` object explicitly:

```
const matches = /(hello \S+)/.exec("This is a hello world!");
console.log(matches[1]);
```

This will log a message containing `'hello world!'`.

See also

- Regular expressions guide
- `RegExp`

RegExp.prototype.flags

The **`flags`** accessor property of `RegExp` instances returns the flags of this regular expression.

Try it

```
// Outputs RegExp flags in alphabetical order

console.log(/foo/gi.flags);
// Expected output: "gi"

console.log(/^bar/muy.flags);
// Expected output: "muy"
```

Description

`RegExp.prototype.flags` has a string as its value. Flags in the `flags` property are sorted alphabetically (from left to right, e.g., `"dgimsuvy"`). It actually invokes the other flag accessors (`hasIndices`, `global`, etc.) one-by-one and concatenates the results.

All built-in functions read the `flags` property instead of reading individual flag accessors.

The set accessor of `flags` is `undefined`. You cannot change this property directly.

Examples

Using flags

```
/foo/ig.flags; // "gi"
/^bar/myu.flags; // "muy"
```

See also

- Polyfill of `RegExp.prototype.flags` in `core-js`
- es-shims polyfill of `RegExp.prototype.flags`
- Advanced searching with flags in the Regular expressions guide
- `RegExp.prototype.source`

RegExp.prototype.global

The **`global`** accessor property of `RegExp` instances returns whether or not the `g` flag is used with this regular expression.

Try it

```
const regex1 = /foo/g;

console.log(regex1.global);
// Expected output: true

const regex2 = /bar/i;

console.log(regex2.global);
// Expected output: false
```

Description

`RegExp.prototype.global` has the value `true` if the `g` flag was used; otherwise, `false`. The `g` flag indicates that the regular expression should be tested against all possible matches in a string. Each call to `exec()` will update its `lastIndex` property, so that the next call to `exec()` will start at the next character.

Some methods, such as `String.prototype.matchAll()` and `String.prototype.replaceAll()`, will validate that, if the parameter is a regex, it is global. The regex's `[Symbol.match]()` and `[Symbol.replace]()` methods (called by `String.prototype.match()` and `String.prototype.replace()`) would also have different behaviors when the regex is global.

The set accessor of `global` is `undefined`. You cannot change this property directly.

Examples

Using global

```
const globalRegex = /foo/g;

const str = "fooexamplefoo";
console.log(str.replace(globalRegex, "")); // example

const nonGlobalRegex = /foo/;
console.log(str.replace(nonGlobalRegex, "")); // examplefoo
```

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

RegExp.prototype.hasIndices

The **`hasIndices`** accessor property of `RegExp` instances returns whether or not the `d` flag is used with this regular expression.

Try it

```
const regex1 = /foo/d;

console.log(regex1.hasIndices);
// Expected output: true

const regex2 = /bar/;

console.log(regex2.hasIndices);
// Expected output: false
```

Description

`RegExp.prototype.hasIndices` has the value `true` if the `d` flag was used; otherwise, `false`. The `d` flag indicates that the result of a regular expression match should contain the start and end indices of the substrings of each capture group. It does not change the regex's interpretation or matching behavior in any way, but only provides additional information in the matching result.

This flag primarily affects the return value of `exec()`. If the `d` flag is present, the array returned by `exec()` has an additional `indices` property as described in the `exec()` method's return value. Because all other regex-related methods (such as `String.prototype.match()`) call `exec()` internally, they will also return the indices if the regex has the `d` flag.

The set accessor of `hasIndices` is `undefined`. You cannot change this property directly.

Examples

There's a more detailed usage example at Groups and backreferences > Using groups and match indices.

Using hasIndices

```
const str1 = "foo bar foo";

const regex1 = /foo/dg;

console.log(regex1.hasIndices); // true

console.log(regex1.exec(str1).indices[0]); // [0, 3]
console.log(regex1.exec(str1).indices[0]); // [8, 11]

const str2 = "foo bar foo";

const regex2 = /foo/;

console.log(regex2.hasIndices); // false

console.log(regex2.exec(str2).indices); // undefined
```

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.exec()`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`

- RegExp.prototype.source
- RegExp.prototype.sticky
- RegExp.prototype.unicode

RegExp.prototype.ignoreCase

The **ignoreCase** accessor property of RegExp instances returns whether or not the i flag is used with this regular expression.

Try it

```
const regex1 = /foo/;
const regex2 = /foo/i;

console.log(regex1.test("Football"));
// Expected output: false

console.log(regex2.ignoreCase);
// Expected output: true

console.log(regex2.test("Football"));
// Expected output: true
```

Description

RegExp.prototype.ignoreCase has the value true if the i flag was used; otherwise, false. The i flag indicates that case should be ignored while attempting a match in a string. Case-insensitive matching is done by mapping both the expected character set and the matched string to the same casing.

If the regex is Unicode-aware, the case mapping happens through *simple case folding* specified in CaseFolding.txt. The mapping always maps to a single code point, so it does not map, for example, ß (U+00DF LATIN SMALL LETTER SHARP S) to ss (which is *full case folding*, not *simple case folding*). It may however map code points outside the Basic Latin block to code points within it — for example, ſ (U+017F LATIN SMALL LETTER LONG S) case-folds to s (U+0073 LATIN SMALL LETTER S) and K (U+212A KELVIN SIGN) case-folds to k (U+006B LATIN SMALL LETTER K). Therefore, ſ and K can be matched by /[a-z]/ui.

If the regex is Unicode-unaware, case mapping uses the Unicode Default Case Conversion — the same algorithm used in String.prototype.toUpperCase(). This algorithm prevents code points outside the Basic Latin block to be mapped to code points within it, so ſ and K mentioned previously are not matched by /[a-z]/i.

Unicode-aware case folding generally folds to lower case, while Unicode-unaware case folding folds to upper case. These two are not perfect reverse operations, so there are some subtle behavior differences. For example, Ω (U+2126 OHM SIGN) and Ω (U+03A9 GREEK CAPITAL LETTER OMEGA) are both mapped by simple case folding to ω (U+03C9 GREEK SMALL LETTER OMEGA), so "\u2126" is matched by /[\u03c9]/ui and /[\u03a9]/ui; on the other hand, U+2126 is mapped by Default Case Conversion to itself, while the other two both map to U+03A9, so "\u2126" is matched by neither /[\u03c9]/i nor /[\u03a9]/i.

The set accessor of ignoreCase is undefined. You cannot change this property directly.

Examples

Using ignoreCase

```
const regex = /foo/i;

console.log(regex.ignoreCase); // true
```

See also

- RegExp.prototype.lastIndex
- RegExp.prototype.dotAll
- RegExp.prototype.global
- RegExp.prototype.hasIndices
- RegExp.prototype.multiline
- RegExp.prototype.source
- RegExp.prototype.sticky
- RegExp.prototype.unicode

RegExp.input ($_)

> **Note:** All RegExp static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **RegExp.input** static accessor property returns the string against which a regular expression is matched. RegExp.$_ is an alias for this property.

Description

Because input is a static property of RegExp, you always use it as RegExp.input or RegExp.$_, rather than as a property of a RegExp object you created.

The value of input updates whenever a RegExp (but not a RegExp subclass) instance makes a successful match. If no matches have been made, input is an empty string. You can set the value of input, but this does not affect other behaviors of the regex, and the value will be overwritten again when the next successful match is made.

Examples

Using input and $_

```
const re = /hi/g;
re.test("hi there!");
```

```
RegExp.input; // "hi there!"
re.test("foo"); // new test, non-matching
RegExp.$_; // "hi there!"
re.test("hi world!"); // new test, matching
RegExp.$_; // "hi world!"
```

See also

- `RegExp.lastMatch` ($&)
- `RegExp.lastParen` ($+)
- `RegExp.leftContext` ($`)
- `RegExp.rightContext` ($')
- `RegExp.$1`, ..., `RegExp.$9`

RegExp: lastIndex

The **`lastIndex`** data property of a `RegExp` instance specifies the index at which to start the next match.

Try it

```
const regex = /foo/g;
const str = "table football, foosball";

regex.test(str);

console.log(regex.lastIndex);
// Expected output: 9

regex.test(str);

console.log(regex.lastIndex);
// Expected output: 19
```

Value

A non-negative integer.

Property attributes of `RegExp: lastIndex`	
Writable	yes
Enumerable	no
Configurable	no

Description

This property is set only if the regular expression instance used the `g` flag to indicate a global search, or the `y` flag to indicate a sticky search. The following rules apply when `exec()` is called on a given input:

- If `lastIndex` is greater than the length of the input, `exec()` will not find a match, and `lastIndex` will be set to 0.
- If `lastIndex` is equal to or less than the length of the input, `exec()` will attempt to match the input starting from `lastIndex`.
  - If `exec()` finds a match, then `lastIndex` will be set to the position of the end of the matched string in the input.
  - If `exec()` does not find a match, then `lastIndex` will be set to 0.

Other regex-related methods, such as `RegExp.prototype.test()`, `String.prototype.match()`, `String.prototype.replace()`, etc., call `exec()` under the hood, so they have different effects on `lastIndex`. See their respective pages for details.

Examples

Using lastIndex

Consider the following sequence of statements:

```
const re = /(hi)?/g;
```

Matches the empty string.

```
console.log(re.exec("hi"));
console.log(re.lastIndex);
```

Returns `["hi", "hi"]` with `lastIndex` equal to 2.

```
console.log(re.exec("hi"));
console.log(re.lastIndex);
```

Returns `["", undefined]`, an empty array whose zeroth element is the match string. In this case, the empty string because `lastIndex` was 2 (and still is 2) and `hi` has length 2.

Using lastIndex with sticky regexes

The `lastIndex` property is writable. You can set it to make the regex start its next search at a given index.

The `y` flag almost always requires setting `lastIndex`. It always matches strictly at `lastIndex` and does not attempt any later positions. This is usually useful for writing parsers, when you want to match tokens only at the current position.

```
const stringPattern = /"[^"]*"/y;
const input = `const message = "Hello world";`;

stringPattern.lastIndex = 6;
console.log(stringPattern.exec(input)); // null

stringPattern.lastIndex = 16;
console.log(stringPattern.exec(input)); // ['"Hello world"']
```

Rewinding lastIndex

The `g` flag also benefits from setting `lastIndex`. One common use case is when the string is modified in the middle of a global search. In this case, we may miss a particular match if the string is shortened. We can avoid this by rewinding `lastIndex`.

```
const mdLinkPattern = /\[[^[\]]+\]\((?<link>[^()\s]+)\)/dg;

function resolveMDLink(line) {
  let match;
  let modifiedLine = line;
  while ((match = mdLinkPattern.exec(modifiedLine))) {
    const originalLink = match.groups.link;
    const resolvedLink = originalLink.replaceAll(/^files|\/index\.md$/g, "");
    modifiedLine =
      modifiedLine.slice(0, match.indices.groups.link[0]) +
      resolvedLink +
      modifiedLine.slice(match.indices.groups.link[1]);
    // Rewind the pattern to the end of the resolved link
    mdLinkPattern.lastIndex += resolvedLink.length - originalLink.length;
  }
  return modifiedLine;
}

console.log(
  resolveMDLink(
    "[`lastIndex`](files/en-us/web/javascript/reference/global_objects/regexp/lastindex/index.md)",
  ),
); // [`lastIndex`](/en-us/web/javascript/reference/global_objects/regexp/lastindex)
console.log(
  resolveMDLink(
    "[`ServiceWorker`](files/en-us/web/api/serviceworker/index.md) and [`SharedWorker`](files/en-us/web/api/sharedworker/index.md)",
  ),
); // [`ServiceWorker`](/en-us/web/api/serviceworker) and [`SharedWorker`](/en-us/web/api/sharedworker)
```

Try deleting the `mdLinkPattern.lastIndex += resolvedLink.length - originalLink.length` line and running the second example. You will find that the second link is not replaced correctly, because the `lastIndex` is already past the link's index after the string is shortened.

> **Warning:** This example is for demonstration only. To deal with Markdown, you should probably use a parsing library instead of regex.

Optimizing searching

You can optimize searching by setting `lastIndex` to a point where previous possible occurrences can be ignored. For example, instead of this:

```
const stringPattern = /"[^"]*"/g;
const input = `const message = "Hello " + "world";`;

// Pretend we've already dealt with the previous parts of the string
let offset = 26;
const remainingInput = input.slice(offset);
const nextString = stringPattern.exec(remainingInput);
console.log(nextString[0]); // "world"
offset += nextString.index + nextString.length;
```

Consider this:

```
stringPattern.lastIndex = offset;
const nextString = stringPattern.exec(remainingInput);
console.log(nextString[0]); // "world"
offset = stringPattern.lastIndex;
```

This is potentially more performant because we avoid string slicing.

Avoiding side effects

The side effects caused by `exec()` can be confusing, especially if the input is different for each `exec()`.

```
const re = /foo/g;
console.log(re.test("foo bar")); // true
console.log(re.test("foo baz")); // false, because lastIndex is non-zero
```

This is even more confusing when you are hand-modifying `lastIndex`. To contain the side effects, remember to reset `lastIndex` after each input is completely processed.

```
const re = /foo/g;
console.log(re.test("foo bar")); // true
re.lastIndex = 0;
console.log(re.test("foo baz")); // true
```

With some abstraction, you can require `lastIndex` to be set to a particular value before each `exec()` call.

```
function createMatcher(pattern) {
  // Create a copy, so that the original regex is never updated
  const regex = new RegExp(pattern, "g");
  return (input, offset) => {
    regex.lastIndex = offset;
    return regex.exec(input);
  };
}

const matchFoo = createMatcher(/foo/);
console.log(matchFoo("foo bar", 0)[0]); // "foo"
console.log(matchFoo("foo baz", 0)[0]); // "foo"
```

See also

- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

RegExp.lastMatch ($&)

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.lastMatch`** static accessor property returns the last matched substring. `RegExp["$&"]` is an alias for this property.

Description

Because `lastMatch` is a static property of `RegExp`, you always use it as `RegExp.lastMatch` or `RegExp["$&"]`, rather than as a property of a `RegExp` object you created.

The value of `lastMatch` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `lastMatch` is an empty string. The set accessor of `lastMatch` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$&`), because `&` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

`$&` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp["$&"]` legacy property.

Examples

Using lastMatch and $&

```
const re = /hi/g;
re.test("hi there!");
RegExp.lastMatch; // "hi"
RegExp["$&"]; // "hi"
```

See also

- `RegExp.input ($_)`
- `RegExp.lastParen ($+)`
- `RegExp.leftContext ($`)`
- `RegExp.rightContext ($')`
- `RegExp.$1, ..., RegExp.$9`

RegExp.lastParen ($+)

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.lastParen`** static accessor property returns the last parenthesized substring match, if any. `RegExp["$+"]` is an alias for this property.

Description

Because `lastParen` is a static property of `RegExp`, you always use it as `RegExp.lastParen` or `RegExp["$+"]`, rather than as a property of a `RegExp` object you created.

The value of `lastParen` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, or if the most recent regex execution contains no capturing groups, `lastParen` is an empty string. The set accessor of `lastParen` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$+`), because `+` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

Examples

Using lastParen and $+

```
const re = /(hi)/g;
re.test("hi there!");
RegExp.lastParen; // "hi"
RegExp["$+"]; // "hi"
```

See also

- `RegExp.input ($_)`
- `RegExp.lastMatch ($&)`
- `RegExp.leftContext ($`)`
- `RegExp.rightContext ($')`
- `RegExp.$1, ..., RegExp.$9`

RegExp.leftContext ($`)

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.leftContext`** static accessor property returns the substring preceding the most recent match. `RegExp["$`"]` is an alias for this property.

Description

Because `leftContext` is a static property of `RegExp`, you always use it as `RegExp.leftContext` or `RegExp["$`"]`, rather than as a property of a `RegExp` object you created.

The value of `leftContext` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `leftContext` is an empty string. The set accessor of `leftContext` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$` `), because `` ` `` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

`$`` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp["$`"]` legacy property.

Examples

Using leftContext and $`

```
const re = /world/g;
re.test("hello world!");
RegExp.leftContext; // "hello "
RegExp["$`"]; // "hello "
```

See also

- `RegExp.input ($_)`
- `RegExp.lastMatch ($&)`
- `RegExp.lastParen ($+)`
- `RegExp.rightContext ($')`
- `RegExp.$1, ..., RegExp.$9`

RegExp.prototype.multiline

The **`multiline`** accessor property of `RegExp` instances returns whether or not the `m` flag is used with this regular expression.

Try it

```
const regex1 = /^football/;
const regex2 = /^football/m;

console.log(regex1.multiline);
// Expected output: false

console.log(regex2.multiline);
// Expected output: true

console.log(regex1.test("rugby\nfootball"));
// Expected output: false

console.log(regex2.test("rugby\nfootball"));
// Expected output: true
```

Description

`RegExp.prototype.multiline` has the value `true` if the `m` flag was used; otherwise, `false`. The `m` flag indicates that a multiline input string should be treated as multiple lines. For example, if `m` is used, `^` and `$` change from matching at only the start or end of the entire string to the start or end of any line within the string.

> **Note:** To match the start and end of the entire string in `m` mode, use the buffer boundary assertions `\A`, `\z`, and `\Z`.

The set accessor of `multiline` is `undefined`. You cannot change this property directly.

Examples

Using multiline

```
const regex = /^foo/m;

console.log(regex.multiline); // true
```

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`

RegExp.$1, ..., RegExp.$9

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.$1, …, RegExp.$9`** static accessor properties return parenthesized substring matches.

Description

Because `$1`–`$9` are static properties of `RegExp`, you always use them as `RegExp.$1`, `RegExp.$2`, etc., rather than as properties of a `RegExp` object you created.

The values of `$1, …, $9` update whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, or if the last match does not have the corresponding capturing group, the respective property is an empty string. The set accessor of each property is `undefined`, so you cannot change the properties directly.

The number of possible parenthesized substrings is unlimited, but the `RegExp` object can only hold the first nine. You can access all parenthesized substrings through the returned array's indexes.

`$1, …, $9` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp.$n` legacy properties.

Examples

Using $n with RegExp.prototype.test()

The following script uses the `RegExp.prototype.test()` method to grab a number in a generic string.

```
const str = "Test 24";
const number = /(\d+)/.test(str) ? RegExp.$1 : "0";
number; // "24"
```

Please note that any operation involving the usage of other regular expressions between a `re.test(str)` call and the `RegExp.$n` property, might have side effects, so that accessing these special properties should be done instantly, otherwise the result might be unexpected.

See also

- `RegExp.input ($_)`
- `RegExp.lastMatch ($&)`
- `RegExp.lastParen ($+)`
- `RegExp.leftContext ($`)`
- `RegExp.rightContext ($')`

RegExp() constructor

The **`RegExp()`** constructor creates `RegExp` objects.

For an introduction to regular expressions, read the Regular Expressions chapter in the JavaScript Guide.

Try it

```
const regex1 = /\w+/;
const regex2 = new RegExp("\\w+");

console.log(regex1);
```

```
// Expected output: /\w+/

console.log(regex2);
// Expected output: /\w+/

console.log(regex1 === regex2);
// Expected output: false
```

Syntax

```
new RegExp(pattern)
new RegExp(pattern, flags)
RegExp(pattern)
RegExp(pattern, flags)
```

> **Note:** `RegExp()` can be called with or without `new`, but sometimes with different effects. See Return value.

Parameters

`pattern`

The text of the regular expression. This can also be another `RegExp` object.

`flags` Optional

If specified, `flags` is a string that contains the flags to add. Alternatively, if a `RegExp` object is supplied for the `pattern`, the `flags` string will replace any of that object's flags (and `lastIndex` will be reset to `0`).

`flags` may contain any combination of the following characters:

`d` (indices)

Generate indices for substring matches.

`g` (global)

Find all matches rather than stopping after the first match.

`i` (ignore case)

When matching, casing differences are ignored.

`m` (multiline)

Treat beginning and end assertions (`^` and `$`) as working over multiple lines. In other words, match the beginning or end of *each* line (delimited by `\n` or `\r`), not only the very beginning or end of the whole input string.

`s` (dotAll)

Allows `.` to match newlines.

`u` (unicode)

Treat `pattern` as a sequence of Unicode code points.

`v` (unicodeSets)

An upgrade to the `u` flag that enables set notation in character classes as well as properties of strings.

`y` (sticky)

Matches only from the index indicated by the `lastIndex` property of this regular expression in the target string. Does not attempt to match from any later indexes.

Return value

`RegExp(pattern)` returns `pattern` directly if all of the following are true:

- `RegExp()` is called without `new`;
- `pattern` is a regex;
- `pattern.constructor === RegExp` (usually meaning it's not a subclass);
- `flags` is `undefined`.

In all other cases, calling `RegExp()` with or without `new` both create a new `RegExp` object. If `pattern` is a regex, the new object's source is `pattern.source`; otherwise, its source is `pattern` coerced to a string. If the `flags` parameter is not `undefined`, the new object's `flags` is the parameter's value; otherwise, its `flags` is `pattern.flags` (if `pattern` is a regex).

Exceptions

`SyntaxError`

Thrown in one of the following cases:

- `pattern` cannot be parsed as a valid regular expression.
- `flags` contains repeated characters or any character outside of those allowed.

Examples

Literal notation and constructor

There are two ways to create a `RegExp` object: a *literal notation* and a *constructor*.

- The *literal notation* takes a pattern between two slashes, followed by optional flags, after the second slash.
- The *constructor function* takes either a string or a `RegExp` object as its first parameter and a string of optional flags as its second parameter.

The following three expressions create the same regular expression:

```
/ab+c/i;
new RegExp(/ab+c/, "i"); // literal notation
new RegExp("ab+c", "i"); // constructor
```

Before regular expressions can be used, they have to be compiled. This process allows them to perform matches more efficiently. There are two ways to compile and get a `RegExp` object.

The literal notation results in compilation of the regular expression when the expression is evaluated. On the other hand, the constructor of the `RegExp` object, `new RegExp('ab+c')`, results in runtime compilation of the regular expression.

Use a string as the first argument to the `RegExp()` constructor when you want to build the regular expression from dynamic input.

Building a regular expression from dynamic inputs

```
const breakfasts = ["bacon", "eggs", "oatmeal", "toast", "cereal"];
const order = "Let me get some bacon and eggs, please";

order.match(new RegExp(`\\b(${breakfasts.join("|")})\\b`, "g"));
// Returns ['bacon', 'eggs']
```

See also

- Polyfill of many modern `RegExp` features (`dotAll`, `sticky` flags, named capture groups, etc.) in `core-js`
- Regular expressions guide
- `String.prototype.match()`
- `String.prototype.replace()`

RegExp.rightContext ($')

> **Note:** All `RegExp` static properties that expose the last match state globally are deprecated. See deprecated RegExp features for more information.

The **`RegExp.rightContext`** static accessor property returns the substring following the most recent match. `RegExp["$'"]` is an alias for this property.

Description

Because `rightContext` is a static property of `RegExp`, you always use it as `RegExp.rightContext` or `RegExp["$'"]`, rather than as a property of a `RegExp` object you created.

The value of `rightContext` updates whenever a `RegExp` (but not a `RegExp` subclass) instance makes a successful match. If no matches have been made, `rightContext` is an empty string. The set accessor of `rightContext` is `undefined`, so you cannot change this property directly.

You cannot use the shorthand alias with the dot property accessor (`RegExp.$'`), because `'` is not a valid identifier part, so this causes a `SyntaxError`. Use the bracket notation instead.

`$'` can also be used in the replacement string of `String.prototype.replace()`, but that's unrelated to the `RegExp["$'"]` legacy property.

Examples

Using rightContext and $'

```
const re = /hello/g;
re.test("hello world!");
RegExp.rightContext; // " world!"
RegExp["$'"]; // " world!"
```

See also

- `RegExp.input` (`$_`)
- `RegExp.lastMatch` (`$&`)
- `RegExp.lastParen` (`$+`)
- `RegExp.leftContext` (`$`` `)
- `RegExp.$1`, ..., `RegExp.$9`

RegExp.prototype.source

The **`source`** accessor property of `RegExp` instances returns a string containing the source text of this regular expression, without the two forward slashes on both sides or any flags.

Try it

```
const regex = /fooBar/gi;

console.log(regex.source);
// Expected output: "fooBar"

console.log(new RegExp().source);
// Expected output: "(?:)"

console.log(new RegExp("\n").source === "\\n");
// Expected output: true (starting with ES5)
// Due to escaping
```

Description

Conceptually, the `source` property is the text between the two forward slashes in the regular expression literal. The language requires the returned string to be properly escaped, so that when the `source` is concatenated with a forward slash on both ends, it would form a parsable regex literal. For example, for `new RegExp("/")`, the `source` is `\/`, because if it generates `///`, which is a line comment. Similarly, all line terminators will be escaped because line terminator *characters* would break up the regex literal. There's no requirement for other characters, as long as the result is parsable. For empty regular expressions, the string `(?:)` is returned.

Examples

Using source

```
const regex = /fooBar/gi;

console.log(regex.source); // "fooBar", doesn't contain /.../ and "gi".
```

Empty regular expressions and escaping

```
new RegExp().source; // "(?:)"

new RegExp("\n").source === "\\n"; // true, starting with ES5
```

See also

- `RegExp.prototype.flags`

RegExp.prototype.sticky

The **`sticky`** accessor property of `RegExp` instances returns whether or not the `y` flag is used with this regular expression.

Try it

```
const str = "table football";
const regex = /foo/y;

regex.lastIndex = 6;

console.log(regex.sticky);
// Expected output: true

console.log(regex.test(str));
// Expected output: true

console.log(regex.test(str));
// Expected output: false
```

Description

`RegExp.prototype.sticky` has the value `true` if the `y` flag was used; otherwise, `false`. The `y` flag indicates that the regex attempts to match the target string only from the index indicated by the `lastIndex` property (and unlike a global regex, does not attempt to match from any later indexes).

The set accessor of `sticky` is `undefined`. You cannot change this property directly.

For both sticky regexes and global regexes:

- They start matching at `lastIndex`.
- When the match succeeds, `lastIndex` is advanced to the end of the match.
- When `lastIndex` is out of bounds of the currently matched string, `lastIndex` is reset to 0.

However, for the `exec()` method, the behavior when matching fails is different:

- When the `exec()` method is called on a sticky regex, if the regex fails to match at `lastIndex`, the regex immediately returns `null` and resets `lastIndex` to 0.
- When the `exec()` method is called on a global regex, if the regex fails to match at `lastIndex`, it tries to match from the next character, and so on until a match is found or the end of the string is reached.

For the `exec()` method, a regex that's both sticky and global behaves the same as a sticky and non-global regex. Because `test()` is a simple wrapper around `exec()`, `test()` would ignore the global flag and perform sticky matches as well. However, due to many other methods special-casing the behavior of global regexes, the global flag is, in general, orthogonal to the sticky flag.

- `String.prototype.matchAll()` (which calls `RegExp.prototype[Symbol.matchAll]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: `matchAll()` throws; `[Symbol.matchAll]()` yields the `exec()` result exactly once, without updating the regex's `lastIndex`.
  - For `g` or `gy` regexes: returns an iterator that yields a sequence of `exec()` results.
- `String.prototype.match()` (which calls `RegExp.prototype[Symbol.match]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: returns the `exec()` result and updates the regex's `lastIndex`.
  - For `g` or `gy` regexes: returns an array of all `exec()` results.
- `String.prototype.search()` (which calls `RegExp.prototype[Symbol.search]()`): the `g` flag is always irrelevant.
  - For `y` or `gy` regexes: always returns `0` (if the very beginning of the string matches) or `-1` (if the beginning doesn't match), without updating the regex's `lastIndex` when it exits.
  - For `g` regexes: returns the index of the first match in the string, or `-1` if no match is found.
- `String.prototype.split()` (which calls `RegExp.prototype[Symbol.split]()`): `y`, `g`, and `gy` all have the same behavior.
- `String.prototype.replace()` (which calls `RegExp.prototype[Symbol.replace]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: replaces once at the current `lastIndex` and updates `lastIndex`.
  - For `g` and `gy` regexes: replaces all occurrences matched by `exec()`.
- `String.prototype.replaceAll()` (which calls `RegExp.prototype[Symbol.replace]()`): `y`, `g` and `gy` are all different.
  - For `y` regexes: `replaceAll()` throws.
  - For `g` and `gy` regexes: replaces all occurrences matched by `exec()`.

Examples

Using a regular expression with the sticky flag

```
const str = "#foo#";
const regex = /foo/y;

regex.lastIndex = 1;
regex.test(str); // true
regex.lastIndex = 5;
regex.test(str); // false (lastIndex is taken into account with sticky flag)
regex.lastIndex; // 0 (reset after match failure)
```

Anchored sticky flag

For several versions, Firefox's SpiderMonkey engine had a bug with regard to the `^` assertion and the sticky flag which allowed expressions starting with the `^` assertion and using the sticky flag to match when they shouldn't. The bug was introduced some time after Firefox 3.6 (which had the sticky flag but not the bug) and fixed in 2015. Perhaps because of the bug, the specification specifically calls out the fact that:

> Even when the `y` flag is used with a pattern, `^` always matches only at the beginning of *Input*, or (if *rer*.[[Multiline]] is `true`) at the beginning of a line.

Examples of correct behavior:

```
const regex1 = /^foo/y;
regex1.lastIndex = 2;
regex1.test("..foo"); // false - index 2 is not the beginning of the string

const regex2 = /^foo/my;
regex2.lastIndex = 2;
regex2.test("..foo"); // false - index 2 is not the beginning of the string or line
regex2.lastIndex = 2;
regex2.test(".\nfoo"); // true - index 2 is the beginning of a line
```

See also

- Polyfill of the `sticky` flag in `core-js`
- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.unicode`

RegExp.prototype[Symbol.match]()

The **`[Symbol.match]()`** method of `RegExp` instances specifies how `String.prototype.match()` should behave. In addition, its presence (or absence) can influence whether an object is regarded as a regular expression.

Try it

```
class RegExp1 extends RegExp {
  [Symbol.match](str) {
    const result = RegExp.prototype[Symbol.match].call(this, str);
    if (result) {
      return "VALID";
    }
    return "INVALID";
  }
```

```
}

console.log("2012-07-02".match(new RegExp1("(\\d+)-(\\d+)-(\\d+)")));
// Expected output: "VALID"
```

Syntax

```
regexp[Symbol.match](str)
```

Parameters

`str`

A `String` that is a target of the match.

Return value

An `Array` whose contents depend on the presence or absence of the global (`g`) flag, or `null` if no matches are found.

- If the `g` flag is used, all results matching the complete regular expression will be returned, but capturing groups are not included.
- If the `g` flag is not used, only the first complete match and its related capturing groups are returned. In this case, `match()` will return the same result as `RegExp.prototype.exec()` (an array with some extra properties).

Description

This method exists for customizing match behavior within `RegExp` subclasses. It is called internally in `String.prototype.match()`. For example, the following two examples return the same result.

```
"abc".match(/a/);

/a/[Symbol.match]("abc");
```

If the regex is global (with the `g` flag), its `lastIndex` is first set to 0, so matching always starts from the beginning of the string, and the regex's `exec()` method is repeatedly called until `exec()` returns `null`. If the current match is an empty string, the `lastIndex` would still be advanced — if the regex is Unicode-aware, it would advance by one Unicode code point; otherwise, it advances by one UTF-16 code unit.

```
console.log("😄".match(/(?:)/g)); // [ '', '', '' ]
console.log("😄".match(/(?:)/gu)); // [ '', '' ]
```

If the regex is not global, `exec()` would only be called once and its result becomes the return value of `[Symbol.match]()`.

The `exec()` method automatically resets `lastIndex` to 0 when the last match fails, so for global regexes with `lastIndex` starting at 0, `[Symbol.match]()` generally produces no side-effects. However, when the regex is sticky but not global, `exec()` is only called once and therefore does not reset `lastIndex` if the match was successful. In this case, each call to `match()` may return a different result.

```
const re = /[abc]/y;
for (let i = 0; i < 5; i++) {
  console.log("abc".match(re), re.lastIndex);
}
// [ 'a' ] 1
// [ 'b' ] 2
// [ 'c' ] 3
// null 0
// [ 'a' ] 1
```

When the regex is sticky and global, it would still perform sticky matches — i.e., it would fail to match any occurrences beyond the `lastIndex`.

```
console.log("ab-c".match(/[abc]/gy)); // [ 'a', 'b' ]
```

In addition, the `[Symbol.match]` property is used to check whether an object is a regular expression.

Examples

Direct call

This method can be used in *almost* the same way as `String.prototype.match()`, except the different `this` and the different arguments order.

```
const re = /\d+/g;
const str = "2016-01-02";
const result = re[Symbol.match](str);
console.log(result); // ["2016", "01", "02"]
```

Using <code>[Symbol.match]()</code> in subclasses

Subclasses of `RegExp` can override the `[Symbol.match]()` method to modify the default behavior.

```
class MyRegExp extends RegExp {
  [Symbol.match](str) {
    const result = RegExp.prototype[Symbol.match].call(this, str);
    if (!result) return null;
    return {
```

```
      group(n) {
        return result[n];
      },
    };
  }
}

const re = new MyRegExp("(\\d+)-(\\d+)-(\\d+)");
const str = "2016-01-02";
const result = str.match(re); // String.prototype.match calls re[Symbol.match]().
console.log(result.group(1)); // 2016
console.log(result.group(2)); // 01
console.log(result.group(3)); // 02
```

See also

- Polyfill of `RegExp.prototype[Symbol.match]` in `core-js`
- `String.prototype.match()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype[Symbol.split]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.match`

RegExp.prototype[Symbol.matchAll]()

The **`[Symbol.matchAll]()`** method of `RegExp` instances specifies how `String.prototype.matchAll` should behave.

Try it

```
class MyRegExp extends RegExp {
  [Symbol.matchAll](str) {
    const result = RegExp.prototype[Symbol.matchAll].call(this, str);
    if (!result) {
      return null;
    }
    return Array.from(result);
  }
}

const re = new MyRegExp("-\\d+", "g");
console.log("2016-01-02|2019-03-07".matchAll(re));
// Expected output: Array [Array ["-01"], Array ["-02"], Array ["-03"], Array ["-07"]]
```

Syntax

```
regexp[Symbol.matchAll](str)
```

Parameters

`str`

A `String` that is a target of the match.

Return value

An iterable iterator object (which is not restartable) of matches. Each match is an array with the same shape as the return value of `RegExp.prototype.exec()`.

Description

This method exists for customizing the behavior of `matchAll()` in `RegExp` subclasses. It is called internally in `String.prototype.matchAll()`. For example, the following two examples return the same result.

```
"abc".matchAll(/a/g);

/a/g[Symbol.matchAll]("abc");
```

Like `[Symbol.split]()`, `[Symbol.matchAll]()` starts by using `[Symbol.species]` to construct a new regex, thus avoiding mutating the original regexp in any way. The constructor receives `this` and the original flags. `lastIndex` starts as the original regex's value.

```
const regexp = /[a-c]/g;
regexp.lastIndex = 1;
const str = "abc";
Array.from(str.matchAll(regexp), (m) => `${regexp.lastIndex} ${m[0]}`);
// [ "1 b", "1 c" ]
```

If the regex is global (with the `g` flag), each time the returned iterator's `next()` method is called, the regex's `exec()` is called and the result is yielded. If the current match is an empty string, the `lastIndex` will still be advanced. If the regex has the `u` flag, it advances by one Unicode code point; otherwise, it advances by one UTF-16 code point.

```
console.log(Array.from("😄".matchAll(/(?:)/g)));
// [ [ "" ], [ "" ], [ "" ] ]

console.log(Array.from("😄".matchAll(/(?:)/gu)));
// [ [ "" ], [ "" ] ]
```

If the regex is not global, the returned iterator yields the `exec()` result once and then completes. (The validation that the input is a global regex happens in `String.prototype.matchAll()`. `[Symbol.matchAll]()` does not validate the flags of `this`.)

When the regex is sticky and global, it will still perform sticky matches — i.e., it will not match any occurrences beyond the `lastIndex`.

```
console.log(Array.from("ab-c".matchAll(/[abc]/gy)));
// [ [ "a" ], [ "b" ] ]
```

Examples

Direct call

This method can be used in almost the same way as `String.prototype.matchAll()`, except for the different value of `this` and the different order of arguments.

```
const re = /\d+/g;
const str = "2016-01-02";
const result = re[Symbol.matchAll](str);

console.log(Array.from(result, (x) => x[0]));
// [ "2016", "01", "02" ]
```

Using <code>[Symbol.matchAll]()</code> in subclasses

Subclasses of `RegExp` can override the `[Symbol.matchAll]()` method to modify the default behavior.

For example, to return an `Array` instead of an iterator:

```
class MyRegExp extends RegExp {
  [Symbol.matchAll](str) {
    const result = RegExp.prototype[Symbol.matchAll].call(this, str);
    return result ? Array.from(result) : null;
  }
}

const re = new MyRegExp("(\\d+)-(\\d+)-(\\d+)", "g");
const str = "2016-01-02|2019-03-07";
const result = str.matchAll(re);

console.log(result[0]);
// [ "2016-01-02", "2016", "01", "02" ]

console.log(result[1]);
// [ "2019-03-07", "2019", "03", "07" ]
```

See also

- Polyfill of `RegExp.prototype[Symbol.matchAll]` in `core-js`
- es-shims polyfill of `RegExp.prototype[Symbol.matchAll]`
- `String.prototype.matchAll()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype[Symbol.split]()`
- `Symbol.matchAll`

RegExp.prototype[Symbol.replace]()

The **`[Symbol.replace]()`** method of `RegExp` instances specifies how `String.prototype.replace()` and `String.prototype.replaceAll()` should behave when the regular expression is passed in as the pattern.

Try it

```
class RegExp1 extends RegExp {
  [Symbol.replace](str) {
    return RegExp.prototype[Symbol.replace].call(this, str, "#!@?");
  }
}

console.log("football".replace(new RegExp1("foo")));
// Expected output: "#!@?tball"
```

Syntax

```
regexp[Symbol.replace](str, replacement)
```

Parameters

`str`

A `String` that is a target of the replacement.

`replacement`

Can be a string or a function.

- If it's a string, it will replace the substring matched by the current regexp. A number of special replacement patterns are supported; see the Specifying a string as the replacement section of `String.prototype.replace`.
- If it's a function, it will be invoked for every match and the return value is used as the replacement text. The arguments supplied to this function are described in the Specifying a function as the replacement section of `String.prototype.replace`.

Return value

A new string, with one, some, or all matches of the pattern replaced by the specified replacement.

Description

This method exists for customizing replace behavior in `RegExp` subclasses. It is called internally in `String.prototype.replace()` and `String.prototype.replaceAll()` if the `pattern` argument is a `RegExp` object. For example, the following two examples return the same result.

```
"abc".replace(/a/, "A");

/a/[Symbol.replace]("abc", "A");
```

If the regex is global (with the `g` flag), its `lastIndex` is first set to 0, so matching always starts from the beginning of the string, and the regex's `exec()` method is repeatedly called until `exec()` returns `null`. If the current match is an empty string, the `lastIndex` would still be advanced — if the regex is Unicode-aware, it would advance by one Unicode code point; otherwise, it advances by one UTF-16 code unit.

```
console.log("😄".replace(/(?:)/g, " ")); // " \ud83d \ude04 "
console.log("😄".replace(/(?:)/gu, " ")); // " 😄 "
```

If the regex is not global, `exec()` would only be called once.

Substitution happens after all matching substrings are identified. For each successful `exec()` result, a substitution string is created based on the `replacement` argument, the process of which is described in `String.prototype.replace()`.

The `exec()` method automatically resets `lastIndex` to 0 when the last match fails, so for global regexes with `lastIndex` starting at 0, `[Symbol.replace]()` generally produces no side-effects. However, when the regex is sticky but not global, `exec()` is only called once and therefore does not reset `lastIndex` if the match was successful. In this case, each call to `replace()` may return a different result.

```
const re = /a/y;

for (let i = 0; i < 5; i++) {
  console.log("aaa".replace(re, "b"), re.lastIndex);
}

// baa 1
// aba 2
// aab 3
// aaa 0
// baa 1
```

When the regex is sticky and global, it would still perform sticky matches — i.e., it would fail to match any occurrences beyond the `lastIndex`.

```
console.log("aa-a".replace(/a/gy, "b")); // "bb-a"
```

Examples

Direct call

This method can be used in almost the same way as `String.prototype.replace()`, except the different `this` and the different arguments order.

```
const re = /-/g;
const str = "2016-01-01";
const newStr = re[Symbol.replace](str, ".");
console.log(newStr); // 2016.01.01
```

Using <code>[Symbol.replace]()</code> in subclasses

Subclasses of `RegExp` can override the `[Symbol.replace]()` method to modify the default behavior.

```
class MyRegExp extends RegExp {
  constructor(pattern, flags, count) {
    super(pattern, flags);
    this.count = count;
  }
  [Symbol.replace](str, replacement) {
```

```
    // Perform [Symbol.replace]() `count` times.
    let result = str;
    for (let i = 0; i < this.count; i++) {
      result = RegExp.prototype[Symbol.replace].call(this, result, replacement);
    }
    return result;
  }
}

const re = new MyRegExp("\\d", "", 3);
const str = "01234567";
const newStr = str.replace(re, "#"); // String.prototype.replace calls re[Symbol.replace]().
console.log(newStr); // ###34567
```

See also

- Polyfill of `RegExp.prototype[Symbol.replace]` in `core-js`
- `String.prototype.replace()`
- `String.prototype.replaceAll()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype[Symbol.split]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.replace`

RegExp.prototype[Symbol.search]()

The **`[Symbol.search]()`** method of `RegExp` instances specifies how `String.prototype.search` should behave.

Try it

```
class RegExp1 extends RegExp {
  constructor(str) {
    super(str);
    this.pattern = str;
  }
  [Symbol.search](str) {
    return str.indexOf(this.pattern);
  }
}

console.log("table football".search(new RegExp1("foo")));
// Expected output: 6
```

Syntax

```
regexp[Symbol.search](str)
```

Parameters

`str`

A `String` that is a target of the search.

Return value

The index of the first match between the regular expression and the given string, or `-1` if no match was found.

Description

This method exists for customizing the search behavior in `RegExp` subclasses. It is called internally in `String.prototype.search()`. For example, the following two examples return the same result.

```
"abc".search(/a/);

/a/[Symbol.search]("abc");
```

`[Symbol.search]()` always calls the regex's `exec()` method exactly once, and returns the `index` property of the result, or `-1` if the result is `null`. The `g` flag has no effect with this method.

This method does not copy the regular expression, unlike `[Symbol.split]()` or `[Symbol.matchAll]()`. However, unlike `[Symbol.match]()` or `[Symbol.replace]()`, it will always set `lastIndex` to 0 when execution starts and restore it to the previous value when it exits, therefore generally avoiding side effects. This means that it always returns the first match in the string even when `lastIndex` is non-zero, and sticky regexes always search strictly at the beginning of the string.

```
const re = /[abc]/g;
re.lastIndex = 2;
console.log("abc".search(re)); // 0

const re2 = /[bc]/y;
re2.lastIndex = 1;
```

```
console.log("abc".search(re2)); // -1
console.log("abc".match(re2)); // [ 'b' ]
```

Examples

Direct call

This method can be used in almost the same way as `String.prototype.search()`, except for the different value of `this` and the different arguments order.

```
const re = /-/g;
const str = "2016-01-02";
const result = re[Symbol.search](str);
console.log(result); // 4
```

Using <code>[Symbol.search]()</code> in subclasses

Subclasses of `RegExp` can override `[Symbol.search]()` method to modify the behavior.

```
class MyRegExp extends RegExp {
  constructor(str) {
    super(str);
    this.pattern = str;
  }
  [Symbol.search](str) {
    return str.indexOf(this.pattern);
  }
}

const re = new MyRegExp("a+b");
const str = "ab a+b";
const result = str.search(re); // String.prototype.search calls re[Symbol.search]().
console.log(result); // 3
```

See also

- Polyfill of `RegExp.prototype[Symbol.search]` in `core-js`
- `String.prototype.search()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.split]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.search`

RegExp[Symbol.species]

The **`RegExp[Symbol.species]`** static accessor property returns the constructor used to construct copied regular expressions in certain `RegExp` methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
RegExp[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct copied `RegExp` instances.

Description

The `[Symbol.species]` accessor property returns the default constructor for `RegExp` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class RegExp {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubRegExp extends RegExp {}
SubRegExp[Symbol.species] === SubRegExp; // true
```

Some `RegExp` methods create a copy of the current regex instance before running `exec()`, so that side effects such as changes to `lastIndex` are not retained. The `[Symbol.species]` property is used to determine the constructor of the new instance. The methods that copy the current regex instance are:

- `[Symbol.matchAll]()`
- `[Symbol.split]()`

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `RegExp` constructor for `RegExp` objects:

```
RegExp[Symbol.species]; // function RegExp()
```

Species in derived objects

In an instance of a custom `RegExp` subclass, such as `MyRegExp`, the `MyRegExp` species is the `MyRegExp` constructor. However, you might want to overwrite this, in order to return parent `RegExp` objects in your derived class methods:

```
class MyRegExp extends RegExp {
  // Overwrite MyRegExp species to the parent RegExp constructor
  static get [Symbol.species]() {
    return RegExp;
  }
}
```

Or you can use this to observe the copying process:

```
class MyRegExp extends RegExp {
  constructor(...args) {
    console.log("Creating a new MyRegExp instance with args:", args);
    super(...args);
  }
  static get [Symbol.species]() {
    console.log("Copying MyRegExp");
    return this;
  }
  exec(value) {
    console.log("Executing with lastIndex:", this.lastIndex);
    return super.exec(value);
  }
}

Array.from("aabbccdd".matchAll(new MyRegExp("[ac]", "g")));
// Creating a new MyRegExp instance with args: [ '[ac]', 'g' ]
// Copying MyRegExp
// Creating a new MyRegExp instance with args: [ MyRegExp /[ac]/g, 'g' ]
// Executing with lastIndex: 0
// Executing with lastIndex: 1
// Executing with lastIndex: 2
// Executing with lastIndex: 5
// Executing with lastIndex: 6
```

See also

- `RegExp`
- `Symbol.species`

RegExp.prototype[Symbol.split]()

The **`[Symbol.split]()`** method of `RegExp` instances specifies how `String.prototype.split` should behave when the regular expression is passed in as the separator.

Try it

```
class RegExp1 extends RegExp {
  [Symbol.split](str, limit) {
    const result = RegExp.prototype[Symbol.split].call(this, str, limit);
    return result.map((x) => `(${x})`);
  }
}

console.log("2016-01-02".split(new RegExp1("-")));
// Expected output: Array ["(2016)", "(01)", "(02)"]

console.log("2016-01-02".split(/-/));
// Expected output: Array ["2016", "01", "02"]
```

Syntax

```
regexp[Symbol.split](str)
regexp[Symbol.split](str, limit)
```

Parameters

`str`

The target of the split operation.

`limit` Optional

Integer specifying a limit on the number of splits to be found. The `[Symbol.split]()` method still splits on every match of `this` RegExp pattern (or, in the Syntax above, `regexp`), until the number of split items match the `limit` or the string falls short of `this` pattern.

Return value

An `Array` containing substrings as its elements. Capturing groups are included.

Description

This method exists for customizing the behavior of `split()` in `RegExp` subclasses. It is called internally in `String.prototype.split()` when a `RegExp` is passed as the separator. For example, the following two examples return the same result.

```
"a-b-c".split(/-/);

/-/[Symbol.split]("a-b-c");
```

Like `[Symbol.matchAll]()`, `[Symbol.split]()` starts by using `[Symbol.species]` to construct a new regex, thus avoiding mutating the original regexp in any way. The constructor receives `this` and the original flags, plus the `y` ("sticky") flag if it was not originally present. The `g` ("global") flag is irrelevant for the method's behavior. By default, due to the `RegExp()` constructor's behavior, `lastIndex` starts as 0.

If the target string is empty, and the regexp can match empty strings (for example, `/a?/`), an empty array is returned. Otherwise, if the regexp can't match an empty string, `[""]` is returned.

The regex's `exec()` method is repeatedly called, advancing the `lastIndex` each time, until it is at the end of the string. If the current match is an empty string, or if the regexp doesn't match at the current position (since it's sticky), the `lastIndex` would still be advanced — if the regex is Unicode-aware, it would advance by one Unicode code point; otherwise, it advances by one UTF-16 code unit.

```
console.log("😄".split(/(?:)/g)); // [ '\ud83d', '\ude04' ]
console.log("😄".split(/(?:)/gu)); // [ '😄' ]
```

For each match, the substring between the last matched string's end and the current matched string's beginning is first appended to the result array. Then, the capturing groups' values are appended one-by-one. The returned array's length will never exceed the `limit` parameter, if provided, while trying to be as close as possible. Therefore, the last match and its capturing groups may not all be present in the returned array if the array is already filled.

If there was no successful match anywhere in the string, the target string is returned as-is, wrapped in an array.

Examples

Direct call

This method can be used in almost the same way as `String.prototype.split()`, except the different `this` and the different order of arguments.

```
const re = /-/g;
const str = "2016-01-02";
const result = re[Symbol.split](str);
console.log(result); // ["2016", "01", "02"]
```

Using <code>[Symbol.split]()</code> in subclasses

Subclasses of `RegExp` can override the `[Symbol.split]()` method to modify the default behavior.

```
class MyRegExp extends RegExp {
  [Symbol.split](str, limit) {
    const result = RegExp.prototype[Symbol.split].call(this, str, limit);
    return result.map((x) => `(${x})`);
  }
}

const re = new MyRegExp("-");
const str = "2016-01-02";
const result = str.split(re); // String.prototype.split calls re[Symbol.split]().
console.log(result); // ["(2016)", "(01)", "(02)"]
```

See also

- Polyfill of `RegExp.prototype[Symbol.split]` in `core-js`
- `String.prototype.split()`
- `RegExp.prototype[Symbol.match]()`
- `RegExp.prototype[Symbol.matchAll]()`
- `RegExp.prototype[Symbol.replace]()`
- `RegExp.prototype[Symbol.search]()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.split`

RegExp.prototype.test()

The **`test()`** method of `RegExp` instances executes a search with this regular expression for a match between a regular expression and a specified string. Returns `true` if there is a match; `false` otherwise.

JavaScript `RegExp` objects are **stateful** when they have the `global` or `sticky` flags set (e.g., `/foo/g` or `/foo/y`). They store a `lastIndex` from the previous match. Using this internally, `test()` can be used to iterate over multiple matches in a string of text (with capture groups).

Try it

```
const str = "table football";

const regex = /fo+/;
const globalRegex = /fo+/g;

console.log(regex.test(str));
// Expected output: true

console.log(globalRegex.lastIndex);
// Expected output: 0

console.log(globalRegex.test(str));
// Expected output: true

console.log(globalRegex.lastIndex);
// Expected output: 9

console.log(globalRegex.test(str));
// Expected output: false
```

Syntax

```
test(str)
```

Parameters

`str`

> The string against which to match the regular expression. All values are coerced to strings, so omitting it or passing `undefined` causes `test()` to search for the string `"undefined"`, which is rarely what you want.

Return value

`true` if there is a match between the regular expression and the string `str`. Otherwise, `false`.

Description

Use `test()` whenever you want to know whether a pattern is found in a string. `test()` returns a boolean, unlike the `String.prototype.search()` method (which returns the index of a match, or `-1` if not found).

To get more information (but with slower execution), use the `exec()` method. (This is similar to the `String.prototype.match()` method.)

As with `exec()` (or in combination with it), `test()` called multiple times on the same global regular expression instance will advance past the previous match.

Examples

Using test()

This example tests if `"hello"` is contained at the very beginning of a string, returning a boolean result.

```
const str = "hello world!";
const result = /^hello/.test(str);

console.log(result); // true
```

The following example logs a message which depends on the success of the test:

```
function testInput(re, str) {
  const midString = re.test(str) ? "contains" : "does not contain";
  console.log(`${str} ${midString} ${re.source}`);
}
```

Using test() on a regex with the "global" flag

When a regex has the global flag set, `test()` will advance the `lastIndex` of the regex. (`RegExp.prototype.exec()` also advances the `lastIndex` property.)

Further calls to `test(str)` will resume searching `str` starting from `lastIndex`. The `lastIndex` property will continue to increase each time `test()` returns `true`.

> **Note:** As long as `test()` returns `true`, `lastIndex` will *not* reset—even when testing a different string!

When `test()` returns `false`, the calling regex's `lastIndex` property will reset to `0`.

The following example demonstrates this behavior:

```
const regex = /foo/g; // the "global" flag is set

// regex.lastIndex is at 0
regex.test("foo"); // true

// regex.lastIndex is now at 3
regex.test("foo"); // false

// regex.lastIndex is at 0
regex.test("barfoo"); // true

// regex.lastIndex is at 6
regex.test("foobar"); // false

// regex.lastIndex is at 0
regex.test("foobarfoo"); // true

// regex.lastIndex is at 3
regex.test("foobarfoo"); // true

// regex.lastIndex is at 9
regex.test("foobarfoo"); // false

// regex.lastIndex is at 0
// (...and so on)
```

See also

- Regular expressions guide
- `RegExp`

RegExp.prototype.toString()

The **`toString()`** method of `RegExp` instances returns a string representing this regular expression.

Try it

```
console.log(new RegExp("a+b+c"));
// Expected output: /a+b+c/

console.log(new RegExp("a+b+c").toString());
// Expected output: "/a+b+c/"

console.log(new RegExp("bar", "g").toString());
// Expected output: "/bar/g"

console.log(new RegExp("\n", "g").toString());
// Expected output: "/\n/g"

console.log(new RegExp("\\n", "g").toString());
// Expected output: "/\n/g"
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the given object.

Description

The `RegExp` object overrides the `toString()` method of the `Object` object; it does not inherit `Object.prototype.toString()`. For `RegExp` objects, the `toString()` method returns a string representation of the regular expression.

In practice, it reads the regex's `source` and `flags` properties and returns a string in the form `/source/flags`. The `toString()` return value is guaranteed to be a parsable regex literal, although it may not be the exact same text as what was originally specified for the regex (for example, the flags may be reordered).

Examples

Using toString()

The following example displays the string value of a `RegExp` object:

```
const myExp = new RegExp("a+b+c");
console.log(myExp.toString()); // '/a+b+c/'

const foo = new RegExp("bar", "g");
console.log(foo.toString()); // '/bar/g'
```

Empty regular expressions and escaping

Since `toString()` accesses the `source` property, an empty regular expression returns the string `"/(?:)/"`, and line terminators such as `\n` are escaped. This makes the returned value always a valid regex literal.

```
new RegExp().toString(); // "/(?:)/"

new RegExp("\n").toString() === "/\\n/"; // true
```

See also

- `Object.prototype.toString()`

RegExp.prototype.unicode

The **`unicode`** accessor property of `RegExp` instances returns whether or not the `u` flag is used with this regular expression.

Try it

```
const regex1 = /\u{61}/;
const regex2 = /\u{61}/u;

console.log(regex1.unicode);
// Expected output: false

console.log(regex2.unicode);
// Expected output: true
```

Description

`RegExp.prototype.unicode` has the value `true` if the `u` flag was used; otherwise, `false`. The `u` flag enables various Unicode-related features. With the "u" flag:

- Any Unicode code point escapes (`\u{xxxx}`, `\p{UnicodePropertyValue}`) will be interpreted as such instead of identity escapes. For example `/\u{61}/u` matches `"a"`, but `/\u{61}/` (without `u` flag) matches `"u".repeat(61)`, where the `\u` is equivalent to a single `u`.
- Buffer boundary assertions (`\A`, `\z`, `\Z`) will be interpreted as such instead of identity escapes.
- Surrogate pairs will be interpreted as whole characters instead of two separate characters. For example, `/[😄]/u` would only match `"😄"` but not `"\ud83d"`.
- When `lastIndex` is automatically advanced (such as when calling `exec()`), unicode regexes advance by Unicode code points instead of UTF-16 code units.

There are other changes to the parsing behavior that prevent possible syntax mistakes (which are analogous to strict mode for regex syntax). These syntaxes are all deprecated and only kept for web compatibility, and you should not rely on them.

The set accessor of `unicode` is `undefined`. You cannot change this property directly.

Unicode-aware mode

When we refer to *Unicode-aware mode*, we mean the regex has either the `u` or the `v` flag, in which case the regex enables Unicode-related features (such as Unicode character class escape) and has much stricter syntax rules. Because `u` and `v` interpret the same regex in incompatible ways, using both flags results in a `SyntaxError`.

Similarly, a regex is *Unicode-unaware* if it has neither the `u` nor the `v` flag. In this case, the regex is interpreted as a sequence of UTF-16 code units, and there are many legacy syntaxes that do not become syntax errors.

Examples

Using the unicode property

```
const regex = /\u{61}/u;

console.log(regex.unicode); // true
```

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`

RegExp.prototype.unicodeSets

The **`unicodeSets`** accessor property of `RegExp` instances returns whether or not the `v` flag is used with this regular expression.

Try it

```
const regex1 = /[α-ω]/u;
const regex2 = /[\p{Lowercase}&&\p{Script=Greek}]/v;

console.log(regex1.unicodeSets);
// Expected output: false
```

```
console.log(regex2.unicodeSets);
// Expected output: true
```

Description

`RegExp.prototype.unicodeSets` has the value `true` if the `v` flag was used; otherwise, `false`. The `v` flag is an "upgrade" to the `u` flag that enables more Unicode-related features. ("v" is the next letter after "u" in the alphabet.) Because `u` and `v` interpret the same regex in incompatible ways, using both flags results in a `SyntaxError`. With the `v` flag, you get all features mentioned in the `u` flag description, plus:

- The `\p` escape sequence can be additionally used to match properties of strings, instead of just characters.
- The character class syntax is upgraded to allow intersection, union, and subtraction syntaxes, as well as matching multiple Unicode characters.
- The character class complement syntax `[^...]` constructs a complement class instead of negating the match result, avoiding some confusing behaviors with case-insensitive matching. For more information, see Complement classes and case-insensitive matching.

Some valid `u`-mode regexes become invalid in `v`-mode. Specifically, the character class syntax is different and some characters can no longer appear literally. For more information, see v-mode character class.

> **Note:** The `v` mode does not interpret grapheme clusters as single characters; they are still multiple code points. For example, `/[un]/v` is still able to match `"u"`.

The set accessor of `unicodeSets` is `undefined`. You cannot change this property directly.

Examples

Using the unicodeSets property

```
const regex = /[\p{Script_Extensions=Greek}&&\p{Letter}]/v;

console.log(regex.unicodeSets); // true
```

See also

- `RegExp.prototype.lastIndex`
- `RegExp.prototype.dotAll`
- `RegExp.prototype.global`
- `RegExp.prototype.hasIndices`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`
- `RegExp.prototype.unicode`
- RegExp v flag with set notation and properties of strings on v8.dev (2022)

Set

The **`Set`** object lets you store unique values of any type, whether primitive values or object references.

Description

`Set` objects are collections of values. A value in the set **may only occur once**; it is unique in the set's collection. You can iterate through the elements of a set in insertion order. The *insertion order* corresponds to the order in which each element was inserted into the set by the `add()` method successfully (that is, there wasn't an identical element already in the set when `add()` was called).

The specification requires sets to be implemented "that, on average, provide access times that are sublinear on the number of elements in the collection". Therefore, it could be represented internally as a hash table (with O(1) lookup), a search tree (with O(log(N)) lookup), or any other data structure, as long as the complexity is better than O(N).

Value equality

Value equality is based on the SameValueZero algorithm. (It used to use SameValue, which treated `0` and `-0` as different. Check browser compatibility.) This means `NaN` is considered the same as `NaN` (even though `NaN !== NaN`) and all other values are considered equal according to the semantics of the `===` operator. Also, for object keys, equality is based on object identity. They are compared by reference, not by value. See Using the Set object for examples.

Performance

The `has` method checks if a value is in the set, using an approach that is, on average, quicker than testing most of the elements that have previously been added to the set. In particular, it is, on average, faster than the `Array.prototype.includes` method when an array has a `length` equal to a set's `size`.

Set composition

The `Set` object provides some methods that allow you to compose sets like you would with mathematical operations. These methods include:

Method	Return type	Mathematical equivalent	Venn diagram
`A.difference(B)`	`Set`	$A \setminus B$	A Venn diagram where two circles overlap. The difference of A and B is the part of A that is not overlapping B.
`A.intersection(B)`	`Set`	$A \cap B$	A Venn diagram where two circles overlap. The intersection of A and B is the part where they overlap.

Method	Return type	Mathematical equivalent	Venn diagram
`A.symmetricDifference(B)`	`Set`	$(A \setminus B) \cup (B \setminus A)$	A Venn diagram where two circles overlap. The symmetric difference of A and B is the region contained by either circle but not both.
`A.union(B)`	`Set`	$A \cup B$	A Venn diagram where two circles overlap. The union of A and B is the region contained by either or both circles.
`A.isDisjointFrom(B)`	`Boolean`	$A \cap B = \emptyset$	A Venn diagram with two circles. A and B are disjoint because the circles have no region of overlap.
`A.isSubsetOf(B)`	`Boolean`	$A \subseteq B$	A Venn diagram with two circles. A is a subset of B because A is completely contained in B.
`A.isSupersetOf(B)`	`Boolean`	$A \supseteq B$	A Venn diagram with two circles. A is a superset of B because B is completely contained in A.

To make them more generalizable, these methods don't just accept `Set` objects, but anything that's set-like.

Set-like objects

All set composition methods require `this` to be an actual `Set` instance, but their arguments just need to be set-like. A *set-like object* is an object that provides the following:

- A `size` property that contains a number.
- A `has()` method that takes an element and returns a boolean.
- A `keys()` method that returns an iterator of the elements in the set.

For example, `Map` objects are set-like because they also have `size`, `has()`, and `keys()`, so they behave just like sets of keys when used in set methods:

```
const a = new Set([1, 2, 3]);
const b = new Map([
  [1, "one"],
  [2, "two"],
  [4, "four"],
]);
console.log(a.union(b)); // Set(4) {1, 2, 3, 4}
```

> **Note:** The set-like protocol invokes the `keys()` method instead of `[Symbol.iterator]()` to produce elements. This is to make maps valid set-like objects, because for maps, the iterator produces *entries* but the `has()` method takes *keys*.

Arrays are not set-like because they don't have a `has()` method or the `size` property, and their `keys()` method produces indices instead of elements. `WeakSet` objects are also not set-like because they don't have a `keys()` method.

Set-like browser APIs

Browser **`Set`-like objects** (or "setlike objects") are Web API interfaces that behave in many ways like a `Set`.

Just like `Set`, elements can be iterated in the same order that they were added to the object. `Set`-like objects and `Set` also have properties and methods that share the same name and behavior. However unlike `Set` they only allow a specific predefined type for each entry.

The allowed types are set in the specification IDL definition. For example, `GPUSupportedFeatures` is a `Set`-like object that must use strings as the key/value. This is defined in the specification IDL below:

```
interface GPUSupportedFeatures {
  readonly setlike<DOMString>;
};
```

`Set`-like objects are either read-only or read-writable (see the `readonly` keyword in the IDL above).

- Read-only `Set`-like objects have the property `size`, and the methods: `entries()`, `forEach()`, `has()`, `keys()`, `values()`, and `[Symbol.iterator]()`.
- Writeable `Set`-like objects additionally have the methods: `clear()`, `delete()`, and `add()`.

The methods and properties have the same behavior as the equivalent entities in `Set`, except for the restriction on the types of the entry.

The following are examples of read-only `Set`-like browser objects:

- `GPUSupportedFeatures`
- `XRAnchorSet`

The following are examples of writable `Set`-like browser objects:

- `CustomStateSet`
- `FontFaceSet`
- `Highlight`
- `ViewTransitionTypeSet`

Constructor

`Set()`

Creates a new `Set` object.

Static properties

`Set[Symbol.species]`

> The constructor function that is used to create derived objects.

Instance properties

These properties are defined on `Set.prototype` and shared by all `Set` instances.

`Set.prototype.constructor`

> The constructor function that created the instance object. For `Set` instances, the initial value is the `Set` constructor.

`Set.prototype.size`

> Returns the number of values in the `Set` object.

`Set.prototype[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"Set"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Set.prototype.add()`

> Inserts the specified value into this set, if it is not already present.

`Set.prototype.clear()`

> Removes all elements from the `Set` object.

`Set.prototype.delete()`

> Removes the specified value from this set, if it is in the set.

`Set.prototype.difference()`

> Takes a set and returns a new set containing elements in this set but not in the given set.

`Set.prototype.entries()`

> Returns a new iterator object that contains **an array of `[value, value]`** for each element in the `Set` object, in insertion order. This is similar to the `Map` object, so that each entry's *key* is the same as its *value* for a `Set`.

`Set.prototype.forEach()`

> Calls `callbackFn` once for each value present in the `Set` object, in insertion order. If a `thisArg` parameter is provided, it will be used as the `this` value for each invocation of `callbackFn`.

`Set.prototype.has()`

> Returns a boolean indicating whether the specified value exists in this `Set` or not.

`Set.prototype.intersection()`

> Takes a set and returns a new set containing elements in both this set and the given set.

`Set.prototype.isDisjointFrom()`

> Takes a set and returns a boolean indicating if this set has no elements in common with the given set.

`Set.prototype.isSubsetOf()`

> Takes a set and returns a boolean indicating if all elements of this set are in the given set.

`Set.prototype.isSupersetOf()`

> Takes a set and returns a boolean indicating if all elements of the given set are in this set.

`Set.prototype.keys()`

> An alias for `Set.prototype.values()`.

`Set.prototype.symmetricDifference()`

> Takes a set and returns a new set containing elements which are in either this set or the given set, but not in both.

`Set.prototype.union()`

> Takes a set and returns a new set containing elements which are in either or both of this set and the given set.

`Set.prototype.values()`

> Returns a new iterator object that yields the **values** for each element in the `Set` object in insertion order.

`Set.prototype[Symbol.iterator]()`

Returns a new iterator object that yields the **values** for each element in the `Set` object in insertion order.

Examples

Using the Set object

```
const mySet1 = new Set();

mySet1.add(1); // Set(1) { 1 }
mySet1.add(5); // Set(2) { 1, 5 }
mySet1.add(5); // Set(2) { 1, 5 }
mySet1.add("some text"); // Set(3) { 1, 5, 'some text' }
const o = { a: 1, b: 2 };
mySet1.add(o);

mySet1.add({ a: 1, b: 2 }); // o is referencing a different object, so this is okay

mySet1.has(1); // true
mySet1.has(3); // false, since 3 has not been added to the set
mySet1.has(5); // true
mySet1.has(Math.sqrt(25)); // true
mySet1.has("Some Text".toLowerCase()); // true
mySet1.has(o); // true

mySet1.size; // 5

mySet1.delete(5); // removes 5 from the set
mySet1.has(5); // false, 5 has been removed

mySet1.size; // 4, since we just removed one value

mySet1.add(5); // Set(5) { 1, 'some text', {...}, {...}, 5 } - a previously deleted item will be added as a new item, it will not retain its original
position before deletion

console.log(mySet1); // Set(5) { 1, "some text", {…}, {…}, 5 }
```

Iterating sets

The iteration over a set visits elements in insertion order.

```
for (const item of mySet1) {
  console.log(item);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

for (const item of mySet1.keys()) {
  console.log(item);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

for (const item of mySet1.values()) {
  console.log(item);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

// key and value are the same here
for (const [key, value] of mySet1.entries()) {
  console.log(key);
}
// 1, "some text", { "a": 1, "b": 2 }, { "a": 1, "b": 2 }, 5

// Convert Set object to an Array object, with Array.from
const myArr = Array.from(mySet1); // [1, "some text", {"a": 1, "b": 2}, {"a": 1, "b": 2}, 5]

// the following will also work if run in an HTML document
mySet1.add(document.body);
mySet1.has(document.querySelector("body")); // true

// converting between Set and Array
const mySet2 = new Set([1, 2, 3, 4]);
console.log(mySet2.size); // 4
console.log([...mySet2]); // [1, 2, 3, 4]

// intersect can be simulated via
const intersection = new Set([...mySet1].filter((x) => mySet2.has(x)));

// difference can be simulated via
const difference = new Set([...mySet1].filter((x) => !mySet2.has(x)));

// Iterate set entries with forEach()
mySet2.forEach((value) => {
  console.log(value);
});
// 1
// 2
```

```
// 3
// 4
```

Implementing basic set operations

```
function isSuperset(set, subset) {
  for (const elem of subset) {
    if (!set.has(elem)) {
      return false;
    }
  }
  return true;
}

function union(setA, setB) {
  const _union = new Set(setA);
  for (const elem of setB) {
    _union.add(elem);
  }
  return _union;
}

function intersection(setA, setB) {
  const _intersection = new Set();
  for (const elem of setB) {
    if (setA.has(elem)) {
      _intersection.add(elem);
    }
  }
  return _intersection;
}

function symmetricDifference(setA, setB) {
  const _difference = new Set(setA);
  for (const elem of setB) {
    if (_difference.has(elem)) {
      _difference.delete(elem);
    } else {
      _difference.add(elem);
    }
  }
  return _difference;
}

function difference(setA, setB) {
  const _difference = new Set(setA);
  for (const elem of setB) {
    _difference.delete(elem);
  }
  return _difference;
}

// Examples
const setA = new Set([1, 2, 3, 4]);
const setB = new Set([2, 3]);
const setC = new Set([3, 4, 5, 6]);

isSuperset(setA, setB); // returns true
union(setA, setC); // returns Set {1, 2, 3, 4, 5, 6}
intersection(setA, setC); // returns Set {3, 4}
symmetricDifference(setA, setC); // returns Set {1, 2, 5, 6}
difference(setA, setC); // returns Set {1, 2}
```

Relation to arrays

```
const myArray = ["value1", "value2", "value3"];

// Use the regular Set constructor to transform an Array into a Set
const mySet = new Set(myArray);

mySet.has("value1"); // returns true

// Use the spread syntax to transform a set into an Array.
console.log([...mySet]); // Will show you exactly the same Array as myArray
```

Remove duplicate elements from an array

```
// Use to remove duplicate elements from an array
const numbers = [2, 13, 4, 4, 2, 13, 13, 4, 4, 5, 5, 6, 6, 7, 5, 32, 13, 4, 5];

console.log([...new Set(numbers)]); // [2, 13, 4, 5, 6, 7, 32]
```

Relation to strings

```
// Case sensitive (set will contain "F" and "f")
new Set("Firefox"); // Set(7) [ "F", "i", "r", "e", "f", "o", "x" ]

// Duplicate omission ("f" occurs twice in the string but set will contain only one)
new Set("firefox"); // Set(6) [ "f", "i", "r", "e", "o", "x" ]
```

Use a set to ensure the uniqueness of a list of values

```
const array = Array.from(document.querySelectorAll("[id]")).map((e) => e.id);

const set = new Set(array);
console.assert(set.size === array.length);
```

See also

- Polyfill of `Set` in `core-js`
- es-shims polyfill of `Set`
- `Map`
- `WeakMap`
- `WeakSet`

Set.prototype.add()

The **`add()`** method of `Set` instances inserts the specified value into this set, if it is not already present.

Try it

```
const set = new Set();

set.add(42);
set.add(42);
set.add(13);

for (const item of set) {
  console.log(item);
  // Expected output: 42
  // Expected output: 13
}
```

Syntax

```
add(value)
```

Parameters

`value`

The value to add to the `Set` object. Objects are compared by reference, not by value.

Return value

The `Set` object.

Examples

Using add()

```
const mySet = new Set();

mySet.add(1);
mySet.add(5).add("some text"); // chainable

console.log(mySet);
// Set [1, 5, "some text"]
```

See also

- `Set`
- `Set.prototype.delete()`
- `Set.prototype.has()`

Set.prototype.clear()

The **`clear()`** method of `Set` instances removes all elements from this set.

Try it

```
const set = new Set();
set.add(1);
set.add("foo");

console.log(set.size);
```

```
// Expected output: 2

set.clear();

console.log(set.size);
// Expected output: 0
```

Syntax

```
clear()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Using the clear() method

```
const mySet = new Set();
mySet.add(1);
mySet.add("foo");

console.log(mySet.size); // 2
console.log(mySet.has("foo")); // true

mySet.clear();

console.log(mySet.size); // 0
console.log(mySet.has("foo")); // false
```

See also

- `Set`
- `Set.prototype.delete()`

Set.prototype.delete()

The **`delete()`** method of `Set` instances removes the specified value from this set, if it is in the set.

Try it

```
const set = new Set();
set.add({ x: 10, y: 20 }).add({ x: 20, y: 30 });

// Delete any point with `x > 10`.
set.forEach((point) => {
  if (point.x > 10) {
    set.delete(point);
  }
});

console.log(set.size);
// Expected output: 1
```

Syntax

```
setInstance.delete(value)
```

Parameters

`value`

The value to remove from the `Set` object. Objects are compared by reference, not by value.

Return value

`true` if a value in the `Set` object has been removed successfully. `false` if the value is not found in the `Set`.

Examples

Using delete()

```
const mySet = new Set();
mySet.add("foo");

console.log(mySet.delete("bar")); // false; no "bar" element found to be deleted.
console.log(mySet.delete("foo")); // true; successfully removed.
```

```
console.log(mySet.has("foo")); // false; the "foo" element is no longer present.
```

Deleting an object from a set

Because objects are compared by reference, you have to delete them by checking individual properties if you don't have a reference to the original object.

```
const setObj = new Set(); // Create a new set.

setObj.add({ x: 10, y: 20 }); // Add object in the set.

setObj.add({ x: 20, y: 30 }); // Add object in the set.

// Delete any point with `x > 10`.
setObj.forEach((point) => {
  if (point.x > 10) {
    setObj.delete(point);
  }
});
```

See also

- `Set`
- `Set.prototype.add()`
- `Set.prototype.clear()`
- `Set.prototype.has()`

Set.prototype.difference()

The **`difference()`** method of `Set` instances takes a set and returns a new set containing elements in this set but not in the given set.

Syntax

```
difference(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

A new `Set` object containing elements in this set but not in the `other` set.

Description

In mathematical notation, *difference* is defined as:

$$A \setminus B = \{x \in A \mid x \notin B\}$$

And using Venn diagram:

A Venn diagram where two circles overlap. The difference of A and B is the part of A that is not overlapping B.

`difference()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements in `this` that are not seen in `other`.
- Otherwise, it iterates over the elements in `this`, and constructs a new set with all elements `e` in `this` that cause `other.has(e)` to return a falsy value.

The order of elements in the returned set is the same as in `this`.

Examples

Using difference()

The following example computes the difference between the set of odd numbers (<10) and the set of perfect squares (<10). The result is the set of odd numbers that are not perfect squares.

```
const odds = new Set([1, 3, 5, 7, 9]);
const squares = new Set([1, 4, 9]);
console.log(odds.difference(squares)); // Set(3) { 3, 5, 7 }
```

See also

- Polyfill of `Set.prototype.difference` in `core-js`
- es-shims polyfill of `Set.prototype.difference`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`

- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

Set.prototype.entries()

The **`entries()`** method of `Set` instances returns a new *set iterator* object that contains **an array of `[value, value]`** for each element in this set, in insertion order. For `Set` objects there is no `key` like in `Map` objects. However, to keep the API similar to the `Map` object, each *entry* has the same value for its *key* and *value* here, so that an array `[value, value]` is returned.

Try it

```
const set = new Set();
set.add(42);
set.add("forty two");

const iterator = set.entries();

for (const entry of iterator) {
  console.log(entry);
  // Expected output: Array [42, 42]
  // Expected output: Array ["forty two", "forty two"]
}
```

Syntax

```
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using entries()

```
const mySet = new Set();
mySet.add("foobar");
mySet.add(1);
mySet.add("baz");

const setIter = mySet.entries();

console.log(setIter.next().value); // ["foobar", "foobar"]
console.log(setIter.next().value); // [1, 1]
console.log(setIter.next().value); // ["baz", "baz"]
```

See also

- `Set.prototype.keys()`
- `Set.prototype.values()`

Set.prototype.forEach()

The **`forEach()`** method of `Set` instances executes a provided function once for each value in this set, in insertion order.

Try it

```
function logSetElements(value1, value2, set) {
  console.log(`s[${value1}] = ${value2}`);
}

new Set(["foo", "bar", undefined]).forEach(logSetElements);

// Expected output: "s[foo] = foo"
// Expected output: "s[bar] = bar"
// Expected output: "s[undefined] = undefined"
```

Syntax

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callback`

A function to execute for each entry in the set. The function is called with the following arguments:

`value`

Value of each iteration.

`key`

Key of each iteration. This is always the same as `value`.

`set`

The set being iterated.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`.

Return value

None (`undefined`).

Description

The `forEach()` method executes the provided `callback` once for each value which actually exists in the `Set` object. It is not invoked for values which have been deleted. However, it is executed for values which are present but have the value `undefined`.

`callback` is invoked with **three arguments**:

- the **element value**
- the **element key**
- the `Set` **object being traversed**

There are no keys in `Set` objects, however, so the first two arguments are both **values** contained in the `Set`. This is to make it consistent with other `forEach()` methods for `Map` and `Array`.

If a `thisArg` parameter is provided to `forEach()`, it will be passed to `callback` when invoked, for use as its `this` value. Otherwise, the value `undefined` will be passed for use as its `this` value. The `this` value ultimately observable by `callback` is determined according to the usual rules for determining the `this` seen by a function.

Each value is visited once, except in the case when it was deleted and re-added before `forEach()` has finished. `callback` is not invoked for values deleted before being visited. New values added before `forEach()` has finished will be visited.

`forEach()` executes the `callback` function once for each element in the `Set` object; it does not return a value.

Examples

Logging the contents of a Set object

The following code logs a line for each element in a `Set` object:

```
function logSetElements(value1, value2, set) {
  console.log(`s[${value1}] = ${value2}`);
}

new Set(["foo", "bar", undefined]).forEach(logSetElements);

// Logs:
// "s[foo] = foo"
// "s[bar] = bar"
// "s[undefined] = undefined"
```

See also

- `Array.prototype.forEach()`
- `Map.prototype.forEach()`

Set.prototype.has()

The **`has()`** method of `Set` instances returns a boolean indicating whether the specified value exists in this `Set` or not.

Try it

```
const set = new Set([1, 2, 3, 4, 5]);

console.log(set.has(1));
// Expected output: true

console.log(set.has(5));
// Expected output: true

console.log(set.has(6));
// Expected output: false
```

Syntax

```
has(value)
```

Parameters

`value`

The value to test for presence in the `Set` object.

Return value

Returns `true` if the specified value exists in the `Set` object; otherwise `false`.

Examples

Using has()

```
const mySet = new Set();
mySet.add("foo");

console.log(mySet.has("foo")); // true
console.log(mySet.has("bar")); // false

const set = new Set();
const obj = { key1: 1 };
set.add(obj);

console.log(set.has(obj)); // true
console.log(set.has({ key1: 1 })); // false, because they are different object references
console.log(set.add({ key1: 1 })); // now set contains 2 entries
```

See also

- `Set`
- `Set.prototype.add()`
- `Set.prototype.delete()`

Set.prototype.intersection()

The **`intersection()`** method of `Set` instances takes a set and returns a new set containing elements in both this set and the given set.

Syntax

```
intersection(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

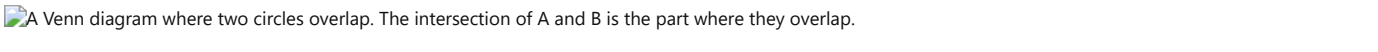
A new `Set` object containing elements in both this set and the `other` set.

Description

In mathematical notation, *intersection* is defined as:

$$A \cap B = \{x \in A \mid x \in B\}$$

And using Venn diagram:



`intersection()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements produced that are also present in `this`.
- Otherwise, it iterates over the elements in `this`, and constructs a new set with all elements `e` in `this` that cause `other.has(e)` to return a truthy value.

Because of this implementation, the efficiency of `intersection()` mostly depends on the size of the smaller set between `this` and `other` (assuming sets can be accessed in sublinear time). The order of elements in the returned set is the same as that of the smaller of `this` and `other`.

Examples

Using intersection()

The following example computes the intersection between the set of odd numbers (<10) and the set of perfect squares (<10). The result is the set of odd numbers that are perfect squares.

```
const odds = new Set([1, 3, 5, 7, 9]);
const squares = new Set([1, 4, 9]);
```

```
console.log(odds.intersection(squares)); // Set(2) { 1, 9 }
```

See also

- Polyfill of `Set.prototype.intersection` in `core-js`
- es-shims polyfill of `Set.prototype.intersection`
- `Set.prototype.difference()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

Set.prototype.isDisjointFrom()

The **`isDisjointFrom()`** method of `Set` instances takes a set and returns a boolean indicating if this set has no elements in common with the given set.

Syntax

```
isDisjointFrom(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

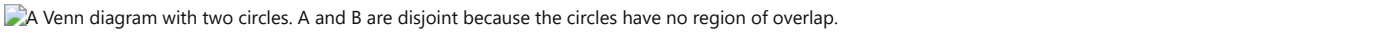
`true` if this set has no elements in common with the `other` set, and `false` otherwise.

Description

Two sets are *disjoint* if they have no elements in common. In mathematical notation:

$$A \text{ is disjoint from } B \Leftrightarrow A \cap B = \emptyset$$

And using Venn diagram:



`isDisjointFrom()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it iterates over `other` by calling its `keys()` method, and if any element in `other` is present in `this`, it returns `false` (and closes the `keys()` iterator by calling its `return()` method). Otherwise, it returns `true`.
- Otherwise, it iterates over the elements in `this`, and returns `false` if any element `e` in `this` causes `other.has(e)` to return a truthy value. Otherwise, it returns `true`.

Because of this implementation, the efficiency of `isDisjointFrom()` mostly depends on the size of the smaller set between `this` and `other` (assuming sets can be accessed in sublinear time).

Examples

Using isDisjointFrom()

The set of perfect squares (<20) is disjoint from the set of prime numbers (<20), because a perfect square is by definition decomposable into the product of two integers, while 1 is also not considered a prime number:

```
const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19]);
const squares = new Set([1, 4, 9, 16]);
console.log(primes.isDisjointFrom(squares)); // true
```

The set of perfect squares (<20) is not disjoint from the set of composite numbers (<20), because all non-1 perfect squares are by definition composite numbers:

```
const composites = new Set([4, 6, 8, 9, 10, 12, 14, 15, 16, 18]);
const squares = new Set([1, 4, 9, 16]);
console.log(composites.isDisjointFrom(squares)); // false
```

See also

- Polyfill of `Set.prototype.isDisjointFrom` in `core-js`
- es-shims polyfill of `Set.prototype.isDisjointFrom`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

Set.prototype.isSubsetOf()

The **`isSubsetOf()`** method of `Set` instances takes a set and returns a boolean indicating if all elements of this set are in the given set.

Syntax

```
isSubsetOf(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

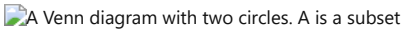
`true` if all elements in this set are also in the `other` set, and `false` otherwise.

Description

In mathematical notation, *subset* is defined as:

$$A \subseteq B \Leftrightarrow \forall x \in A,\ x \in B$$

And using Venn diagram:



> **Note:** The *subset* relationship is not *proper subset*, which means `isSubsetOf()` returns `true` if `this` and `other` contain the same elements.

`isSubsetOf()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are more elements in `this` than `other.size`, then it directly returns `false`.
- Otherwise, it iterates over the elements in `this`, and returns `false` if any element `e` in `this` causes `other.has(e)` to return a falsy value. Otherwise, it returns `true`.

Examples

Using isSubsetOf()

The set of multiples of 4 (<20) is a subset of even numbers (<20):

```
const fours = new Set([4, 8, 12, 16]);
const evens = new Set([2, 4, 6, 8, 10, 12, 14, 16, 18]);
console.log(fours.isSubsetOf(evens)); // true
```

The set of prime numbers (<20) is not a subset of all odd numbers (<20), because 2 is prime but not odd:

```
const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19]);
const odds = new Set([3, 5, 7, 9, 11, 13, 15, 17, 19]);
console.log(primes.isSubsetOf(odds)); // false
```

Equivalent sets are subsets of each other:

```
const set1 = new Set([1, 2, 3]);
const set2 = new Set([1, 2, 3]);
console.log(set1.isSubsetOf(set2)); // true
console.log(set2.isSubsetOf(set1)); // true
```

See also

- Polyfill of `Set.prototype.isSubsetOf` in `core-js`
- es-shims polyfill of `Set.prototype.isSubsetOf`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

Set.prototype.isSupersetOf()

The **`isSupersetOf()`** method of `Set` instances takes a set and returns a boolean indicating if all elements of the given set are in this set.

Syntax

```
isSupersetOf(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

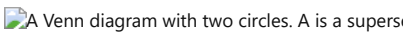
`true` if all elements in the `other` set are also in this set, and `false` otherwise.

Description

In mathematical notation, *superset* is defined as:

$$A \supseteq B \Leftrightarrow \forall x \in B,\ x \in A$$

And using Venn diagram:



> **Note:** The *superset* relationship is not *proper superset*, which means `isSupersetOf()` returns `true` if `this` and `other` contain the same elements.

`isSupersetOf()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, its behavior depends on the sizes of `this` and `other`:

- If there are fewer elements in `this` than `other.size`, then it directly returns `false`.
- Otherwise, it iterates over `other` by calling its `keys()` method, and if any element in `other` is not present in `this`, it returns `false` (and closes the `keys()` iterator by calling its `return()` method). Otherwise, it returns `true`.

Examples

Using isSupersetOf()

The set of even numbers (<20) is a superset of multiples of 4 (<20):

```
const evens = new Set([2, 4, 6, 8, 10, 12, 14, 16, 18]);
const fours = new Set([4, 8, 12, 16]);
console.log(evens.isSupersetOf(fours)); // true
```

The set of all odd numbers (<20) is not a superset of prime numbers (<20), because 2 is prime but not odd:

```
const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19]);
const odds = new Set([3, 5, 7, 9, 11, 13, 15, 17, 19]);
console.log(odds.isSupersetOf(primes)); // false
```

Equivalent sets are supersets of each other:

```
const set1 = new Set([1, 2, 3]);
const set2 = new Set([1, 2, 3]);
console.log(set1.isSupersetOf(set2)); // true
console.log(set2.isSupersetOf(set1)); // true
```

See also

- Polyfill of `Set.prototype.isSupersetOf` in `core-js`
- es-shims polyfill of `Set.prototype.isSupersetOf`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.symmetricDifference()`
- `Set.prototype.union()`

Set.prototype.keys()

The **`keys()`** method of `Set` instances is an alias for the `values()` method.

Syntax

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using keys()

The `keys()` method is exactly equivalent to the `values()` method.

```
const mySet = new Set();
mySet.add("foo");
mySet.add("bar");
mySet.add("baz");

const setIter = mySet.keys();

console.log(setIter.next().value); // "foo"
console.log(setIter.next().value); // "bar"
console.log(setIter.next().value); // "baz"
```

See also

- `Set.prototype.entries()`
- `Set.prototype.values()`

Set() constructor

The **`Set()`** constructor creates `Set` objects.

Try it

```
const set = new Set([1, 2, 3, 4, 5]);

console.log(set.has(1));
// Expected output: true

console.log(set.has(5));
// Expected output: true

console.log(set.has(6));
// Expected output: false
```

Syntax

```
new Set()
new Set(iterable)
```

> **Note:** `Set()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable` Optional

If an iterable object (such as an array) is passed, all of its elements will be added to the new `Set`. If you don't specify this parameter, or its value is `null` or `undefined`, the new `Set` is empty.

Return value

A new `Set` object.

Examples

Using the <code>Set</code> object

```
const mySet = new Set();

mySet.add(1); // Set [ 1 ]
mySet.add(5); // Set [ 1, 5 ]
mySet.add(5); // Set [ 1, 5 ]
mySet.add("some text"); // Set [ 1, 5, 'some text' ]
const o = { a: 1, b: 2 };
mySet.add(o);
```

See also

- Polyfill of `Set` in `core-js`
- es-shims polyfill of `Set`
- `Set`

Set.prototype.size

The **`size`** accessor property of `Set` instances returns the number of (unique) elements in this set.

Try it

```
const set = new Set();
const object = {};

set.add(42);
set.add("forty two");
```

```
set.add("forty two");
set.add(object);

console.log(set.size);
// Expected output: 3
```

Description

The value of `size` is an integer representing how many entries the `Set` object has. A set accessor function for `size` is `undefined`; you cannot change this property.

Examples

Using size

```
const mySet = new Set();
mySet.add(1);
mySet.add(5);
mySet.add("some text");

console.log(mySet.size); // 3
```

See also

- `Set`

Set.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `Set` instances implements the iterable protocol and allows `Set` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a set iterator object that yields the values of the set in insertion order.

The initial value of this property is the same function object as the initial value of the `Set.prototype.values` property.

Try it

```
const set = new Set();

set.add(42);
set.add("forty two");

const iterator = set[Symbol.iterator]();

console.log(iterator.next().value);
// Expected output: 42

console.log(iterator.next().value);
// Expected output: "forty two"
```

Syntax

```
set[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Set.prototype.values()`: a new iterable iterator object that yields the values of the set.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `Set` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const mySet = new Set();
mySet.add("0");
mySet.add(1);
mySet.add({});

for (const v of mySet) {
  console.log(v);
}
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
const mySet = new Set();
mySet.add("0");
mySet.add(1);
mySet.add({});
```

```
const setIter = mySet[Symbol.iterator]();

console.log(setIter.next().value); // "0"
console.log(setIter.next().value); // 1
console.log(setIter.next().value); // {}
```

See also

- `Set`
- `Set.prototype.entries()`
- `Set.prototype.keys()`
- `Set.prototype.values()`
- `Symbol.iterator`
- Iteration protocols

Set[Symbol.species]

The **`Set[Symbol.species]`** static accessor property is an unused accessor property specifying how to copy `Set` objects.

Syntax

```
Set[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct copied `Set` instances.

Description

The `[Symbol.species]` accessor property returns the default constructor for `Set` objects. Subclass constructors may override it to change the constructor assignment.

> **Note:** This property is currently unused by all `Set` methods.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `Set` constructor for `Set`.

```
Set[Symbol.species]; // function Set()
```

Species in derived objects

In an instance of a custom `Set` subclass, such as `MySet`, the `MySet` species is the `MySet` constructor. However, you might want to overwrite this, in order to return parent `Set` objects in your derived class methods:

```
class MySet extends Set {
  // Overwrite MySet species to the parent Set constructor
  static get [Symbol.species]() {
    return Set;
  }
}
```

See also

- `Set`
- `Symbol.species`

Set.prototype.symmetricDifference()

The **`symmetricDifference()`** method of `Set` instances takes a set and returns a new set containing elements which are in either this set or the given set, but not in both.

Syntax

```
symmetricDifference(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

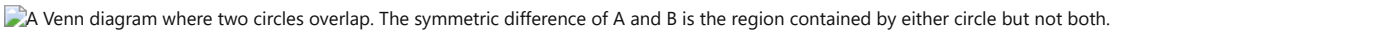
A new `Set` object containing elements which are in either this set or the `other` set, but not in both.

Description

In mathematical notation, *symmetric difference* is defined as:

$$A \ominus B = (A \setminus B) \cup (B \setminus A)$$

And using Venn diagram:



`symmetricDifference()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements in `this` that are not seen in `other`, and all elements in `other` that are not seen in `this`.

The order of elements in the returned set is first those in `this` followed by those in `other`.

Examples

Using symmetricDifference()

The following example computes the symmetric difference between the set of even numbers (<10) and the set of perfect squares (<10). The result is the set of numbers that are either even or a perfect square, but not both.

```
const evens = new Set([2, 4, 6, 8]);
const squares = new Set([1, 4, 9]);
console.log(evens.symmetricDifference(squares)); // Set(5) { 2, 6, 8, 1, 9 }
```

See also

- Polyfill of `Set.prototype.symmetricDifference` in `core-js`
- es-shims polyfill of `Set.prototype.symmetricDifference`
- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.union()`

Set.prototype.union()

The **`union()`** method of `Set` instances takes a set and returns a new set containing elements which are in either or both of this set and the given set.

Syntax

```
union(other)
```

Parameters

`other`

A `Set` object, or set-like object.

Return value

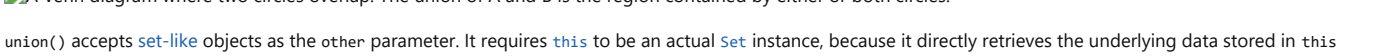
A new `Set` object containing elements which are in either or both of this set and the `other` set.

Description

In mathematical notation, *union* is defined as:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

And using Venn diagram:



`union()` accepts set-like objects as the `other` parameter. It requires `this` to be an actual `Set` instance, because it directly retrieves the underlying data stored in `this` without invoking any user code. Then, it iterates over `other` by calling its `keys()` method, and constructs a new set with all elements in `this`, followed by all elements in `other` that are not present in `this`.

The order of elements in the returned set is first those in `this` followed by those in `other`.

Examples

Using union()

The following example computes the union between the set of even numbers (<10) and the set of perfect squares (<10). The result is the set of numbers that are either even or a perfect square, or both.

```
const evens = new Set([2, 4, 6, 8]);
const squares = new Set([1, 4, 9]);
console.log(evens.union(squares)); // Set(6) { 2, 4, 6, 8, 1, 9 }
```

See also

- Polyfill of `Set.prototype.union` in `core-js`
- es-shims polyfill of `Set.prototype.union`

- `Set.prototype.difference()`
- `Set.prototype.intersection()`
- `Set.prototype.isDisjointFrom()`
- `Set.prototype.isSubsetOf()`
- `Set.prototype.isSupersetOf()`
- `Set.prototype.symmetricDifference()`

Set.prototype.values()

The **`values()`** method of `Set` instances returns a new *set iterator* object that contains the values for each element in this set in insertion order.

Try it

```
const set = new Set();
set.add(42);
set.add("forty two");

const iterator = set.values();

console.log(iterator.next().value);
// Expected output: 42

console.log(iterator.next().value);
// Expected output: "forty two"
```

Syntax

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Examples

Using values()

```
const mySet = new Set();
mySet.add("foo");
mySet.add("bar");
mySet.add("baz");

const setIter = mySet.values();

console.log(setIter.next().value); // "foo"
console.log(setIter.next().value); // "bar"
console.log(setIter.next().value); // "baz"
```

See also

- `Set.prototype.entries()`
- `Set.prototype.keys()`

SharedArrayBuffer

The **`SharedArrayBuffer`** object is used to represent a generic raw binary data buffer, similar to the `ArrayBuffer` object, but in a way that they can be used to create views on shared memory. A `SharedArrayBuffer` is not a Transferable Object, unlike an `ArrayBuffer` which is transferable.

Description

To share memory using `SharedArrayBuffer` objects from one agent in the cluster to another (an agent is either the web page's main program or one of its web workers), `postMessage` and structured cloning is used.

The structured clone algorithm accepts `SharedArrayBuffer` objects and typed arrays mapped onto `SharedArrayBuffer` objects. In both cases, the `SharedArrayBuffer` object is transmitted to the receiver resulting in a new, private `SharedArrayBuffer` object in the receiving agent (just as for `ArrayBuffer`). However, the shared data block referenced by the two `SharedArrayBuffer` objects is the same data block, and a side effect to the block in one agent will eventually become visible in the other agent.

```
const sab = new SharedArrayBuffer(1024);
worker.postMessage(sab);
```

Shared memory can be created and updated simultaneously in workers or the main thread. Depending on the system (the CPU, the OS, the Browser) it can take a while until the change is propagated to all contexts. To synchronize, atomic operations are needed.

`SharedArrayBuffer` objects are used by some web APIs, such as:

- `WebGLRenderingContext.bufferData()`
- `WebGLRenderingContext.bufferSubData()`
- `WebGL2RenderingContext.getBufferSubData()`

Security requirements

Shared memory and high-resolution timers were effectively disabled at the start of 2018 in light of Spectre. In 2020, a new, secure approach has been standardized to re-enable shared memory.

To use shared memory your document must be in a secure context and cross-origin isolated. You can use the `Window.crossOriginIsolated` and `WorkerGlobalScope.crossOriginIsolated` properties to check if the document is cross-origin isolated:

```
const myWorker = new Worker("worker.js");

if (crossOriginIsolated) {
  const buffer = new SharedArrayBuffer(16);
  myWorker.postMessage(buffer);
} else {
  const buffer = new ArrayBuffer(16);
  myWorker.postMessage(buffer);
}
```

When cross-origin isolated, `postMessage()` no longer throws for `SharedArrayBuffer` objects, and shared memory across threads is therefore available.

API availability

Depending on whether the above security measures are taken, the various memory-sharing APIs have different availabilities:

- The `Atomics` object is always available.
- `SharedArrayBuffer` objects are in principle always available, but unfortunately the constructor on the global object is hidden, unless the two headers mentioned above are set, for compatibility with web content. There is hope that this restriction can be removed in the future. `WebAssembly.Memory` can still be used to get an instance.
- Unless the two headers mentioned above are set, the various `postMessage()` APIs will throw for `SharedArrayBuffer` objects. If they are set, `postMessage()` on `Window` objects and dedicated workers will function and allow for memory sharing.

WebAssembly shared memory

`WebAssembly.Memory` objects can be created with the `shared` constructor flag. When this flag is set to `true`, the constructed `Memory` object can be shared between workers via `postMessage()`, just like `SharedArrayBuffer`, and the backing `buffer` of the `Memory` object is a `SharedArrayBuffer`. Therefore, the requirements listed above for sharing a `SharedArrayBuffer` between workers also apply to sharing a `WebAssembly.Memory`.

The WebAssembly Threads proposal also defines a new set of atomic instructions. Just as `SharedArrayBuffer` and its methods are unconditionally enabled (and only sharing between threads is gated on the new headers), the WebAssembly atomic instructions are also unconditionally allowed.

Growing SharedArrayBuffers

`SharedArrayBuffer` objects can be made growable by including the `maxByteLength` option when calling the `SharedArrayBuffer()` constructor. You can query whether a `SharedArrayBuffer` is growable and what its maximum size is by accessing its `growable` and `maxByteLength` properties, respectively. You can assign a new size to a growable `SharedArrayBuffer` with a `grow()` call. New bytes are initialized to 0.

These features make growing `SharedArrayBuffer`s more efficient — otherwise, you have to make a copy of the buffer with a new size. It also gives JavaScript parity with WebAssembly in this regard (Wasm linear memory can be resized with `WebAssembly.Memory.prototype.grow()`).

For security reasons, `SharedArrayBuffer`s cannot be reduced in size, only grown.

Constructor

`SharedArrayBuffer()`

> Creates a new `SharedArrayBuffer` object.

Static properties

`SharedArrayBuffer[Symbol.species]`

> Returns the constructor used to construct return values from `SharedArrayBuffer` methods.

Instance properties

These properties are defined on `SharedArrayBuffer.prototype` and shared by all `SharedArrayBuffer` instances.

`SharedArrayBuffer.prototype.byteLength`

> The size, in bytes, of the array. This is established when the array is constructed and can only be changed using the `SharedArrayBuffer.prototype.grow()` method if the `SharedArrayBuffer` is growable.

`SharedArrayBuffer.prototype.constructor`

> The constructor function that created the instance object. For `SharedArrayBuffer` instances, the initial value is the `SharedArrayBuffer` constructor.

`SharedArrayBuffer.prototype.growable`

> Read-only. Returns `true` if the `SharedArrayBuffer` can be grown, or `false` if not.

`SharedArrayBuffer.prototype.maxByteLength`

> The read-only maximum length, in bytes, that the `SharedArrayBuffer` can be grown to. This is established when the array is constructed and cannot be changed.

`SharedArrayBuffer.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"SharedArrayBuffer"`. This property is used in `Object.prototype.toString()`.

Instance methods

`SharedArrayBuffer.prototype.grow()`

Grows the `SharedArrayBuffer` to the specified size, in bytes.

`SharedArrayBuffer.prototype.slice()`

Returns a new `SharedArrayBuffer` whose contents are a copy of this `SharedArrayBuffer`'s bytes from `begin`, inclusive, up to `end`, exclusive. If either `begin` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

Examples

Note that these examples cannot be run directly from the console or and arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Creating a new SharedArrayBuffer

```
const sab = new SharedArrayBuffer(1024);
```

Slicing the SharedArrayBuffer

```
sab.slice(); // SharedArrayBuffer { byteLength: 1024 }
sab.slice(2); // SharedArrayBuffer { byteLength: 1022 }
sab.slice(-2); // SharedArrayBuffer { byteLength: 2 }
sab.slice(0, 1); // SharedArrayBuffer { byteLength: 1 }
```

Using it in a WebGL buffer

```
const canvas = document.querySelector("canvas");
const gl = canvas.getContext("webgl");
const buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
gl.bufferData(gl.ARRAY_BUFFER, sab, gl.STATIC_DRAW);
```

See also

- `Atomics`
- `ArrayBuffer`
- JavaScript typed arrays guide
- Web Workers
- Shared Memory – a brief tutorial in the TC39 ecmascript-sharedmem proposal
- A Taste of JavaScript's New Parallel Primitives on hacks.mozilla.org (2016)
- COOP and COEP explained by the Chrome team (2020)
- `Cross-Origin-Opener-Policy`
- `Cross-Origin-Embedder-Policy`
- `Cross-Origin-Resource-Policy`
- `Window.crossOriginIsolated` and `WorkerGlobalScope.crossOriginIsolated`
- SharedArrayBuffer updates in Android Chrome 88 and Desktop Chrome 92 on developer.chrome.com (2021)

SharedArrayBuffer.prototype.byteLength

The **`byteLength`** accessor property of `SharedArrayBuffer` instances returns the length (in bytes) of this `SharedArrayBuffer`.

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the shared array is constructed and cannot be changed.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using byteLength

```
const sab = new SharedArrayBuffer(1024);
sab.byteLength; // 1024
```

See also

- `SharedArrayBuffer`

SharedArrayBuffer.prototype.grow()

The **`grow()`** method of `SharedArrayBuffer` instances grows the `SharedArrayBuffer` to the specified size, in bytes.

Syntax

```
grow(newLength)
```

Parameters

`newLength`

The new length, in bytes, to resize the `SharedArrayBuffer` to.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if the `SharedArrayBuffer` is not growable.

`RangeError`

Thrown if `newLength` is larger than the `maxByteLength` of the `SharedArrayBuffer` or smaller than the `byteLength`.

Description

The `grow()` method grows a `SharedArrayBuffer` to the size specified by the `newLength` parameter, provided that the `SharedArrayBuffer` is growable and the new size is less than or equal to the `maxByteLength` of the `SharedArrayBuffer`. New bytes are initialized to 0.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using grow()

In this example, we create a 8-byte buffer that is growable to a max length of 16 bytes, then check its `growable` property, growing it if `growable` returns `true`:

```
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

if (buffer.growable) {
  console.log("SAB is growable!");
  buffer.grow(12);
}
```

See also

- `SharedArrayBuffer`
- `SharedArrayBuffer.prototype.growable`
- `SharedArrayBuffer.prototype.maxByteLength`

SharedArrayBuffer.prototype.growable

The **`growable`** accessor property of `SharedArrayBuffer` instances returns whether this `SharedArrayBuffer` can be grow or not.

Description

The `growable` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed. If a `maxByteLength` option was set in the constructor, `growable` will return `true`; if not, it will return `false`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using growable

In this example, we create a 8-byte buffer that is growable to a max length of 16 bytes, then check its `growable` property, growing it if `growable` returns `true`:

```
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

if (buffer.growable) {
  console.log("SAB is growable!");
  buffer.grow(12);
}
```

See also

- `SharedArrayBuffer`
- `SharedArrayBuffer.prototype.grow()`
- `SharedArrayBuffer.prototype.maxByteLength`

SharedArrayBuffer.prototype.maxByteLength

The **`maxByteLength`** accessor property of `SharedArrayBuffer` instances returns the maximum length (in bytes) that this `SharedArrayBuffer` can be grown to.

Description

The `maxByteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the shared array is constructed, set via the `maxByteLength` option of the `SharedArrayBuffer()` constructor, and cannot be changed.

If this `SharedArrayBuffer` was constructed without specifying a `maxByteLength` value, this property returns a value equal to the value of the `SharedArrayBuffer`'s `byteLength`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using maxByteLength

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then return its `maxByteLength`:

```
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

buffer.maxByteLength; // 16
```

See also

- `SharedArrayBuffer`

SharedArrayBuffer() constructor

> **Note:** The `SharedArrayBuffer` constructor may not always be globally available unless certain security requirements are met.

The **`SharedArrayBuffer()`** constructor creates `SharedArrayBuffer` objects.

Syntax

```
new SharedArrayBuffer(length)
new SharedArrayBuffer(length, options)
```

> **Note:** `SharedArrayBuffer()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`length`

The size, in bytes, of the array buffer to create.

`options` Optional

An object, which can contain the following properties:

`maxByteLength` Optional

The maximum size, in bytes, that the shared array buffer can be resized to.

Return value

A new `SharedArrayBuffer` object of the specified size, with its `maxByteLength` property set to the specified `maxByteLength` if one was specified. Its contents are initialized to 0.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Basic usage

Create a buffer specifying its size in bytes.

```
// Create a SharedArrayBuffer with a size in bytes
const buffer = new SharedArrayBuffer(8);

console.log(buffer.byteLength); // 8
```

Always use the new operator to create a SharedArrayBuffer

`SharedArrayBuffer` constructors are required to be constructed with a `new` operator. Calling a `SharedArrayBuffer` constructor as a function without `new` will throw a `TypeError`.

```
const sab = SharedArrayBuffer(1024);
// TypeError: calling a builtin SharedArrayBuffer constructor
// without new is forbidden
```

```
const sab = new SharedArrayBuffer(1024);
```

Growing a growable SharedArrayBuffer

In this example, we create an 8-byte buffer that is growable to a max length of 16 bytes, then `grow()` it to 12 bytes:

```
const buffer = new SharedArrayBuffer(8, { maxByteLength: 16 });

buffer.grow(12);
```

> **Note:** It is recommended that `maxByteLength` is set to the smallest value possible for your use case. It should never exceed `1073741824` (1GB), to reduce the risk of out-of-memory errors.

See also

- `Atomics`
- `ArrayBuffer`
- JavaScript typed arrays guide

SharedArrayBuffer.prototype.slice()

The **`slice()`** method of `SharedArrayBuffer` instances returns a new `SharedArrayBuffer` whose contents are a copy of this `SharedArrayBuffer`'s bytes from `start`, inclusive, up to `end`, exclusive. If either `start` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

Syntax

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

- Negative index counts back from the end of the buffer — if `-buffer.length <= start < 0`, `start + buffer.length` is used.
- If `start < -buffer.length` or `start` is omitted, `0` is used.
- If `start >= buffer.length`, an empty buffer is returned.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

- Negative index counts back from the end of the buffer — if `-buffer.length <= end < 0`, `end + buffer.length` is used.
- If `end < -buffer.length`, `0` is used.
- If `end >= buffer.length` or `end` is omitted or `undefined`, `buffer.length` is used, causing all elements until the end to be extracted.
- If `end` implies a position before or at the position that `start` implies, an empty buffer is returned.

Return value

A new `SharedArrayBuffer` containing the extracted elements.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using slice()

```
// Create a SharedArrayBuffer with a size in bytes
const buffer = new SharedArrayBuffer(16);
const int32View = new Int32Array(buffer);
int32View[1] = 42;
// Produces Int32Array [0, 42, 0, 0]

const sliced = new Int32Array(buffer.slice(4, 12));
console.log(sliced); // Int32Array [42, 0]
```

Using different start and end values

```
const sab = new SharedArrayBuffer(1024);
sab.slice(); // SharedArrayBuffer { byteLength: 1024 }
sab.slice(2); // SharedArrayBuffer { byteLength: 1022 }
sab.slice(-2); // SharedArrayBuffer { byteLength: 2 }
sab.slice(0, 1); // SharedArrayBuffer { byteLength: 1 }
```

See also

- `SharedArrayBuffer`
- `ArrayBuffer.prototype.slice()`

SharedArrayBuffer[Symbol.species]

The **`SharedArrayBuffer[Symbol.species]`** static accessor property returns the constructor used to construct return values from `SharedArrayBuffer` methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
SharedArrayBuffer[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from array buffer methods that create new array buffer.

Description

The `[Symbol.species]` accessor property returns the default constructor for `SharedArrayBuffer` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class SharedArrayBuffer {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubArrayBuffer extends SharedArrayBuffer {}
SubArrayBuffer[Symbol.species] === SubArrayBuffer; // true
```

When calling array buffer methods that do not mutate the existing array but return a new array buffer instance (for example, `slice()`), the array's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the array buffer method.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `SharedArrayBuffer` constructor for `SharedArrayBuffer`.

```
SharedArrayBuffer[Symbol.species]; // function SharedArrayBuffer()
```

Species in derived objects

In an instance of a custom `SharedArrayBuffer` subclass, such as `MySharedArrayBuffer`, the `MySharedArrayBuffer` species is the `MySharedArrayBuffer` constructor. However, you might want to overwrite this, in order to return parent `SharedArrayBuffer` objects in your derived class methods:

```
class MySharedArrayBuffer extends SharedArrayBuffer {
  // Overwrite MySharedArrayBuffer species to the parent SharedArrayBuffer constructor
  static get [Symbol.species]() {
    return SharedArrayBuffer;
  }
}
```

See also

- `SharedArrayBuffer`
- `Symbol.species`

String

The **`String`** object is used to represent and manipulate a sequence of characters.

Description

Strings are useful for holding data that can be represented in text form. Some of the most-used operations on strings are to check their `length`, to build and concatenate them using the `+` and `+=` string operators, checking for the existence or location of substrings with the `indexOf()` method, or extracting substrings with the `substring()` method.

Creating strings

Strings can be created as primitives, from string literals, or as objects, using the `String()` constructor:

```
const string1 = "A string primitive";
const string2 = 'Also a string primitive';
const string3 = `Yet another string primitive`;
```

```
const string4 = new String("A String object");
```

String primitives and string objects share many behaviors, but have other important differences and caveats. See "String primitives and String objects" below.

String literals can be specified using single or double quotes, which are treated identically, or using the backtick character `. This last form specifies a template literal: with this form you can interpolate expressions. For more information on the syntax of string literals, see lexical grammar.

Character access

There are two ways to access an individual character in a string. The first is the `charAt()` method:

```
"cat".charAt(1); // gives value "a"
```

The other way is to treat the string as an array-like object, where individual characters correspond to a numerical index:

```
"cat"[1]; // gives value "a"
```

When using bracket notation for character access, attempting to delete or assign a value to these properties will not succeed. The properties involved are neither writable nor configurable. (See `Object.defineProperty()` for more information.)

Comparing strings

Use the less-than and greater-than operators to compare strings:

```
const a = "a";
const b = "b";
if (a < b) {
  // true
  console.log(`${a} is less than ${b}`);
} else if (a > b) {
  console.log(`${a} is greater than ${b}`);
} else {
  console.log(`${a} and ${b} are equal.`);
}
```

Note that all comparison operators, including `===` and `==`, compare strings case-sensitively. A common way to compare strings case-insensitively is to convert both to the same case (upper or lower) before comparing them.

```
function areEqualCaseInsensitive(str1, str2) {
  return str1.toUpperCase() === str2.toUpperCase();
}
```

The choice of whether to transform by `toUpperCase()` or `toLowerCase()` is mostly arbitrary, and neither one is fully robust when extending beyond the Latin alphabet. For example, the German lowercase letter `ß` and `ss` are both transformed to `SS` by `toUpperCase()`, while the Turkish letter `ı` would be falsely reported as unequal to `I` by `toLowerCase()` unless specifically using `toLocaleLowerCase("tr")`.

```
const areEqualInUpperCase = (str1, str2) =>
  str1.toUpperCase() === str2.toUpperCase();
const areEqualInLowerCase = (str1, str2) =>
  str1.toLowerCase() === str2.toLowerCase();

areEqualInUpperCase("ß", "ss"); // true; should be false
areEqualInLowerCase("ı", "I"); // false; should be true
```

A locale-aware and robust solution for testing case-insensitive equality is to use the `Intl.Collator` API or the string's `localeCompare()` method — they share the same interface — with the `sensitivity` option set to `"accent"` or `"base"`.

```
const areEqual = (str1, str2, locale = "en-US") =>
  str1.localeCompare(str2, locale, { sensitivity: "accent" }) === 0;

areEqual("ß", "ss", "de"); // false
areEqual("ı", "I", "tr"); // true
```

The `localeCompare()` method enables string comparison in a similar fashion as `strcmp()` — it allows sorting strings in a locale-aware manner.

String primitives and String objects

Note that JavaScript distinguishes between `String` objects and primitive string values. (The same is true of `Boolean` and `Numbers`.)

String literals (denoted by double or single quotes) and strings returned from `String` calls in a non-constructor context (that is, called without using the `new` keyword) are primitive strings. In contexts where a method is to be invoked on a primitive string or a property lookup occurs, JavaScript will automatically wrap the string primitive and call the method or perform the property lookup on the wrapper object instead.

```
const strPrim = "foo"; // A literal is a string primitive
const strPrim2 = String(1); // Coerced into the string primitive "1"
const strPrim3 = String(true); // Coerced into the string primitive "true"
```

```
const strObj = new String(strPrim); // String with new returns a string wrapper object.

console.log(typeof strPrim); // "string"
console.log(typeof strPrim2); // "string"
console.log(typeof strPrim3); // "string"
console.log(typeof strObj); // "object"
```

> **Warning:** You should rarely find yourself using `String` as a constructor.

String primitives and `String` objects also give different results when using `eval()`. Primitives passed to `eval` are treated as source code; `String` objects are treated as all other objects are, by returning the object. For example:

```
const s1 = "2 + 2"; // creates a string primitive
const s2 = new String("2 + 2"); // creates a String object
console.log(eval(s1)); // returns the number 4
console.log(eval(s2)); // returns the string "2 + 2"
```

For these reasons, the code may break when it encounters `String` objects when it expects a primitive string instead, although generally, authors need not worry about the distinction.

A `String` object can always be converted to its primitive counterpart with the `valueOf()` method.

```
console.log(eval(s2.valueOf())); // returns the number 4
```

String coercion

Many built-in operations that expect strings first coerce their arguments to strings (which is largely why `String` objects behave similarly to string primitives). The operation can be summarized as follows:

- Strings are returned as-is.
- `undefined` turns into `"undefined"`.
- `null` turns into `"null"`.
- `true` turns into `"true"`; `false` turns into `"false"`.
- Numbers are converted with the same algorithm as `toString(10)`.
- BigInts are converted with the same algorithm as `toString(10)`.
- Symbols throw a `TypeError`.
- Objects are first converted to a primitive by calling its `[Symbol.toPrimitive]()` (with `"string"` as hint), `toString()`, and `valueOf()` methods, in that order. The resulting primitive is then converted to a string.

There are several ways to achieve nearly the same effect in JavaScript.

- Template literal: `` `${x}` `` does exactly the string coercion steps explained above for the embedded expression.
- The `String()` function: `String(x)` uses the same algorithm to convert `x`, except that Symbols don't throw a `TypeError`, but return `"Symbol(description)"`, where `description` is the description of the Symbol.
- Using the `+` operator: `"" + x` coerces its operand to a *primitive* instead of a *string*, and, for some objects, has entirely different behaviors from normal string coercion. See its reference page for more details.

Depending on your use case, you may want to use `` `${x}` `` (to mimic built-in behavior) or `String(x)` (to handle symbol values without throwing an error), but you should not use `"" + x`.

UTF-16 characters, Unicode code points, and grapheme clusters

Strings are represented fundamentally as sequences of UTF-16 code units. In UTF-16 encoding, every code unit is exact 16 bits long. This means there are a maximum of 2^{16}, or 65536 possible characters representable as single UTF-16 code units. This character set is called the basic multilingual plane (BMP), and includes the most common characters like the Latin, Greek, Cyrillic alphabets, as well as many East Asian characters. Each code unit can be written in a string with `\u` followed by exactly four hex digits.

However, the entire Unicode character set is much, much bigger than 65536. The extra characters are stored in UTF-16 as *surrogate pairs*, which are pairs of 16-bit code units that represent a single character. To avoid ambiguity, the two parts of the pair must be between `0xD800` and `0xDFFF`, and these code units are not used to encode single-code-unit characters. (More precisely, leading surrogates, also called high-surrogate code units, have values between `0xD800` and `0xDBFF`, inclusive, while trailing surrogates, also called low-surrogate code units, have values between `0xDC00` and `0xDFFF`, inclusive.) Each Unicode character, comprised of one or two UTF-16 code units, is also called a *Unicode code point*. Each Unicode code point can be written in a string with `\u{xxxxxx}` where `xxxxxx` represents 1–6 hex digits.

A "lone surrogate" is a 16-bit code unit satisfying one of the descriptions below:

- It is in the range `0xD800`–`0xDBFF`, inclusive (i.e., is a leading surrogate), but it is the last code unit in the string, or the next code unit is not a trailing surrogate.
- It is in the range `0xDC00`–`0xDFFF`, inclusive (i.e., is a trailing surrogate), but it is the first code unit in the string, or the previous code unit is not a leading surrogate.

Lone surrogates do not represent any Unicode character. Although most JavaScript built-in methods handle them correctly because they all work based on UTF-16 code units, lone surrogates are often not valid values when interacting with other systems — for example, `encodeURI()` will throw a `URIError` for lone surrogates, because URI encoding uses UTF-8 encoding, which does not have any encoding for lone surrogates. Strings not containing any lone surrogates are called *well-formed* strings, and are safe to be used with functions that do not deal with UTF-16 (such as `encodeURI()` or `TextEncoder`). You can check if a string is well-formed with the `isWellFormed()` method, or sanitize lone surrogates with the `toWellFormed()` method.

On top of Unicode characters, there are certain sequences of Unicode characters that should be treated as one visual unit, known as a *grapheme cluster*. The most common case is emojis: many emojis that have a range of variations are actually formed by multiple emojis, usually joined by the <ZWJ> (`U+200D`) character.

You must be careful which level of characters you are iterating on. For example, `split("")` will split by UTF-16 code units and will separate surrogate pairs. String indexes also refer to the index of each UTF-16 code unit. On the other hand, `[Symbol.iterator]()` iterates by Unicode code points. Iterating through grapheme clusters will require some custom code.

```
"😄".split(""); // ['\ud83d', '\ude04']; splits into two lone surrogates

// "Backhand Index Pointing Right: Dark Skin Tone"
[..."👉🏿"]; // ['👉', '🏿']
// splits into the basic "Backhand Index Pointing Right" emoji and
// the "Dark skin tone" emoji

// "Family: Man, Boy"
[..."👨‍👦"]; // [ '👨', '', '👦' ]
// splits into the "Man" and "Boy" emoji, joined by a ZWJ

// The United Nations flag
[..."🇺🇳"]; // [ '🇺', '🇳' ]
// splits into two "region indicator" letters "U" and "N".
// All flag emojis are formed by joining two region indicator letters
```

Constructor

`String()`

Creates `String` objects. When called as a function, it returns primitive values of type String.

Static methods

`String.fromCharCode()`

Returns a string created by using the specified sequence of Unicode values.

`String.fromCodePoint()`

Returns a string created by using the specified sequence of code points.

`String.raw()`

Returns a string created from a raw template string.

Instance properties

These properties are defined on `String.prototype` and shared by all `String` instances.

`String.prototype.constructor`

The constructor function that created the instance object. For `String` instances, the initial value is the `String` constructor.

These properties are own properties of each `String` instance.

`length`

Reflects the `length` of the string. Read-only.

Instance methods

`String.prototype.at()`

Returns the character (exactly one UTF-16 code unit) at the specified `index`. Accepts negative integers, which count back from the last string character.

`String.prototype.charAt()`

Returns the character (exactly one UTF-16 code unit) at the specified `index`.

`String.prototype.charCodeAt()`

Returns a number that is the UTF-16 code unit value at the given `index`.

`String.prototype.codePointAt()`

Returns a nonnegative integer Number that is the code point value of the UTF-16 encoded code point starting at the specified `pos`.

`String.prototype.concat()`

Combines the text of two (or more) strings and returns a new string.

`String.prototype.endsWith()`

Determines whether a string ends with the characters of the string `searchString`.

`String.prototype.includes()`

Determines whether the calling string contains `searchString`.

`String.prototype.indexOf()`

Returns the index within this string of the first occurrence of `searchValue`, or `-1` if not found.

`String.prototype.isWellFormed()`

Returns a boolean indicating whether this string contains any lone surrogates.

`String.prototype.lastIndexOf()`

Returns the index within this string of the last occurrence of `searchValue`, or `-1` if not found.

`String.prototype.localeCompare()`

Returns a number indicating whether the reference string `compareString` comes before, after, or is equivalent to the given string in sort order.

`String.prototype.match()`

Used to match regular expression `regexp` against a string.

`String.prototype.matchAll()`

Returns an iterator of all `regexp`'s matches.

`String.prototype.normalize()`

Returns the Unicode Normalization Form of the calling string value.

`String.prototype.padEnd()`

Pads the current string from the end with a given string and returns a new string of the length `targetLength`.

`String.prototype.padStart()`

Pads the current string from the start with a given string and returns a new string of the length `targetLength`.

`String.prototype.repeat()`

Returns a string consisting of the elements of the object repeated `count` times.

`String.prototype.replace()`

Used to replace occurrences of `searchFor` using `replaceWith`. `searchFor` may be a string or Regular Expression, and `replaceWith` may be a string or function.

`String.prototype.replaceAll()`

Used to replace all occurrences of `searchFor` using `replaceWith`. `searchFor` may be a string or Regular Expression, and `replaceWith` may be a string or function.

`String.prototype.search()`

Search for a match between a regular expression `regexp` and the calling string.

`String.prototype.slice()`

Extracts a section of a string and returns a new string.

`String.prototype.split()`

Returns an array of strings populated by splitting the calling string at occurrences of the substring `sep`.

`String.prototype.startsWith()`

Determines whether the calling string begins with the characters of string `searchString`.

`String.prototype.substr()`

Returns a portion of the string, starting at the specified index and extending for a given number of characters afterwards.

`String.prototype.substring()`

Returns a new string containing characters of the calling string from (or between) the specified index (or indices).

`String.prototype.toLocaleLowerCase()`

The characters within a string are converted to lowercase while respecting the current locale.

For most languages, this will return the same as `toLowerCase()`.

`String.prototype.toLocaleUpperCase()`

The characters within a string are converted to uppercase while respecting the current locale.

For most languages, this will return the same as `toUpperCase()`.

`String.prototype.toLowerCase()`

Returns the calling string value converted to lowercase.

`String.prototype.toString()`

Returns a string representing the specified object. Overrides the `Object.prototype.toString()` method.

`String.prototype.toUpperCase()`

Returns the calling string value converted to uppercase.

`String.prototype.toWellFormed()`

Returns a string where all lone surrogates of this string are replaced with the Unicode replacement character U+FFFD.

`String.prototype.trim()`

Trims whitespace from the beginning and end of the string.

`String.prototype.trimEnd()`

Trims whitespace from the end of the string.

`String.prototype.trimStart()`

Trims whitespace from the beginning of the string.

`String.prototype.valueOf()`

Returns the primitive value of the specified object. Overrides the `Object.prototype.valueOf()` method.

`String.prototype[Symbol.iterator]()`

Returns a new iterator object that iterates over the code points of a String value, returning each code point as a String value.

HTML wrapper methods

> **Warning:** Deprecated. Avoid these methods.
>
> They are of limited use, as they are based on a very old HTML standard and provide only a subset of the currently available HTML tags and attributes. Many of them create deprecated or non-standard markup today. In addition, they do string concatenation without any validation or sanitation, which makes them a potential security threat when directly inserted using `innerHTML`. Use DOM APIs such as `document.createElement()` instead.

`String.prototype.anchor()`

`<a name="name">` (hypertext target)

`String.prototype.big()`
`<big>`
`String.prototype.blink()`

`<blink>`

`String.prototype.bold()`
`<b>`
`String.prototype.fixed()`
`<tt>`
`String.prototype.fontcolor()`

`<font color="color">`

`String.prototype.fontsize()`

`<font size="size">`

`String.prototype.italics()`
`<i>`
`String.prototype.link()`

`<a href="url">` (link to URL)

`String.prototype.small()`
`<small>`
`String.prototype.strike()`
`<strike>`
`String.prototype.sub()`
`<sub>`
`String.prototype.sup()`
`<sup>`

Note that these methods do not check if the string itself contains HTML tags, so it's possible to create invalid HTML:

```
"</b>".bold(); // <b></b></b>
```

The only escaping they do is to replace `"` in the attribute value (for `anchor()`, `fontcolor()`, `fontsize()`, and `link()`) with `"`.

```
"foo".anchor('"Hello"'); // <a name="&quot;Hello&quot;">foo</a>
```

Examples

String conversion

The `String()` function is a more reliable way of converting values to strings than calling the `toString()` method of the value, as the former works when used on `null` and `undefined`. For example:

```
// You cannot access properties on null or undefined

const nullVar = null;
nullVar.toString(); // TypeError: Cannot read properties of null
String(nullVar); // "null"

const undefinedVar = undefined;
undefinedVar.toString(); // TypeError: Cannot read properties of undefined
String(undefinedVar); // "undefined"
```

See also

- Numbers and strings guide
- `RegExp`

String.prototype.anchor()

The **`anchor()`** method of `String` values creates a string that embeds this string in an `<a>` element with a name (`<a name="...">str</a>`).

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.
>
> The HTML specification no longer allows the `<a>` element to have a `name` attribute, so this method doesn't even create valid markup.

Syntax

```
anchor(name)
```

Parameters

`name`

A string representing a `name` value to put into the generated `<a name="...">` start tag.

Return value

A string beginning with an `<a name="name">` start tag (double quotes in `name` are replaced with `"`), then the text `str`, and then an `</a>` end tag.

Examples

Using anchor()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.anchor("hello");
```

This will create the following HTML:

```
<a name="hello">Hello, world</a>
```

> **Warning:** This markup is invalid, because `name` is no longer a valid attribute of the `<a>` element.

Instead of using `anchor()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("a");
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.anchor` in `core-js`
- es-shims polyfill of `String.prototype.anchor`
- HTML wrapper methods
- `<a>`

String.prototype.at()

The **`at()`** method of `String` values takes an integer value and returns a new `String` consisting of the single UTF-16 code unit located at the specified offset. This method allows for positive and negative integers. Negative integers count back from the last string character.

Try it

```
const sentence = "The quick brown fox jumps over the lazy dog.";

let index = 5;

console.log(`An index of ${index} returns the character ${sentence.at(index)}`);
// Expected output: "An index of 5 returns the character u"

index = -4;

console.log(`An index of ${index} returns the character ${sentence.at(index)}`);
// Expected output: "An index of -4 returns the character d"
```

Syntax

```
at(index)
```

Parameters

`index`

> The index (position) of the string character to be returned. Supports relative indexing from the end of the string when passed a negative index; i.e., if a negative number is used, the character returned will be found by counting back from the end of the string.

Return value

A `String` consisting of the single UTF-16 code unit located at the specified position. Returns `undefined` if the given index cannot be found.

Examples

Return the last character of a string

The following example provides a function which returns the last character found in a specified string.

```
// A function which returns the last character of a given string
function returnLast(str) {
  return str.at(-1);
}

let invoiceRef = "my-invoice01";

console.log(returnLast(invoiceRef)); // '1'

invoiceRef = "my-invoice02";

console.log(returnLast(invoiceRef)); // '2'
```

Comparing methods

Here we compare different ways to select the penultimate (last but one) character of a `String`. While all below methods are valid, it highlights the succinctness and readability of the `at()` method.

```
const myString = "Every green bus drives fast.";

// Using length property and charAt() method
const lengthWay = myString.charAt(myString.length - 2);
console.log(lengthWay); // 't'

// Using slice() method
const sliceWay = myString.slice(-2, -1);
console.log(sliceWay); // 't'

// Using at() method
const atWay = myString.at(-2);
console.log(atWay); // 't'
```

See also

- Polyfill of `String.prototype.at` in `core-js`
- es-shims polyfill of `String.prototype.at`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`
- `String.prototype.split()`

String.prototype.big()

The **`big()`** method of `String` values creates a string that embeds this string in a `<big>` element (`<big>str</big>`), which causes this string to be displayed in a big font.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `big()`, the `<big>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```
big()
```

Parameters

None.

Return value

A string beginning with a `<big>` start tag, then the text `str`, and then a `</big>` end tag.

Examples

Using big()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.big();
```

This will create the following HTML:

```
<big>Hello, world</big>
```

> **Warning:** This markup is invalid, because `big` is no longer a valid element.

Instead of using `big()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `font-size` through the `element.style` attribute:

```
document.getElementById("yourElemId").style.fontSize = "2em";
```

See also

- Polyfill of `String.prototype.big` in `core-js`
- es-shims polyfill of `String.prototype.big`
- HTML wrapper methods
- `<big>`

String.prototype.blink()

The **`blink()`** method of `String` values creates a string that embeds this string in a `<blink>` element (`<blink>str</blink>`), which used to cause a string to blink in old browsers.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `blink()`, the `<blink>` element itself is removed from modern browsers, and blinking text is frowned upon by several accessibility standards. Avoid using the element in any way.

Syntax

```
blink()
```

Parameters

None.

Return value

A string beginning with a `<blink>` start tag, then the text `str`, and then a `</blink>` end tag.

Examples

Using blink()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.blink();
```

This will create the following HTML:

```
<blink>Hello, world</blink>
```

> **Warning:** This markup is invalid, because `blink` is no longer a valid element.

You should avoid blinking elements altogether.

See also

- Polyfill of `String.prototype.blink` in `core-js`
- es-shims polyfill of `String.prototype.blink`
- HTML wrapper methods

String.prototype.bold()

The **`bold()`** method of `String` values creates a string that embeds this string in a `<b>` element (`<b>str</b>`), which causes this string to be displayed as bold.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
bold()
```

Parameters

None.

Return value

A string beginning with a `<b>` start tag, then the text `str`, and then a `</b>` end tag.

Examples

Using bold()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.bold();
```

This will create the following HTML:

```
<b>Hello, world</b>
```

Instead of using `bold()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("b");
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.bold` in `core-js`
- es-shims polyfill of `String.prototype.bold`
- HTML wrapper methods
- `<b>`

String.prototype.charAt()

The **`charAt()`** method of `String` values returns a new string consisting of the single UTF-16 code unit at the given index.

`charAt()` always indexes the string as a sequence of UTF-16 code units, so it may return lone surrogates. To get the full Unicode code point at the given index, use `String.prototype.codePointAt()` and `String.fromCodePoint()`.

Try it

```
const sentence = "The quick brown fox jumps over the lazy dog.";

const index = 4;

console.log(`The character at index ${index} is ${sentence.charAt(index)}`);
// Expected output: "The character at index 4 is q"
```

Syntax

```
charAt(index)
```

Parameters

`index`

Zero-based index of the character to be returned. Converted to an integer — `undefined` is converted to 0.

Return value

A string representing the character (exactly one UTF-16 code unit) at the specified `index`. If `index` is out of the range of `0` – `str.length - 1`, `charAt()` returns an empty string.

Description

Characters in a string are indexed from left to right. The index of the first character is `0`, and the index of the last character in a string called `str` is `str.length - 1`.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). `charAt()` always returns a character whose value is less than `65536`, because the higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, in order to get a full character with value greater than `65535`, it is necessary to retrieve not only `charAt(i)`, but also `charAt(i + 1)` (as if manipulating a string with two characters), or to use `codePointAt(i)` and `String.fromCodePoint()` instead. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

`charAt()` is very similar to using bracket notation to access a character at the specified index. The main differences are:

- `charAt()` attempts to convert `index` to an integer, while bracket notation does not, and directly uses `index` as a property name.
- `charAt()` returns an empty string if `index` is out of range, while bracket notation returns `undefined`.

Examples

Using charAt()

The following example displays characters at different locations in the string `"Brave new world"`:

```
const anyString = "Brave new world";
console.log(`The character at index 0   is '${anyString.charAt()}'`);
// No index was provided, used 0 as default

console.log(`The character at index 0   is '${anyString.charAt(0)}'`);
console.log(`The character at index 1   is '${anyString.charAt(1)}'`);
console.log(`The character at index 2   is '${anyString.charAt(2)}'`);
console.log(`The character at index 3   is '${anyString.charAt(3)}'`);
console.log(`The character at index 4   is '${anyString.charAt(4)}'`);
console.log(`The character at index 999 is '${anyString.charAt(999)}'`);
```

These lines display the following:

```
The character at index 0   is 'B'

The character at index 0   is 'B'
The character at index 1   is 'r'
The character at index 2   is 'a'
The character at index 3   is 'v'
The character at index 4   is 'e'
The character at index 999 is ''
```

`charAt()` may return lone surrogates, which are not valid Unicode characters.

```
const str = "吉吼";
console.log(str.charAt(0)); // "\ud842", which is not a valid Unicode character
console.log(str.charAt(1)); // "\udfb7", which is not a valid Unicode character
```

To get the full Unicode code point at the given index, use an indexing method that splits by Unicode code points, such as `String.prototype.codePointAt()` and spreading strings into an array of Unicode code points.

```
const str = "吉吼";
console.log(String.fromCodePoint(str.codePointAt(0))); // "吉"
console.log([...str][0]); // "吉"
```

> **Note:** Avoid re-implementing the solutions above using `charAt()`. The detection of lone surrogates and their pairing is complex, and built-in APIs may be more performant as they directly use the internal representation of the string. Install a polyfill for the APIs mentioned above if necessary.

See also

- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`
- `String.prototype.split()`
- `String.fromCodePoint()`

- JavaScript has a Unicode problem by Mathias Bynens (2013)

String.prototype.charCodeAt()

The **`charCodeAt()`** method of `String` values returns an integer between `0` and `65535` representing the UTF-16 code unit at the given index.

`charCodeAt()` always indexes the string as a sequence of UTF-16 code units, so it may return lone surrogates. To get the full Unicode code point at the given index, use `String.prototype.codePointAt()`.

Try it

```
const sentence = "The quick brown fox jumps over the lazy dog.";

const index = 4;

console.log(
  `Character code ${sentence.charCodeAt(index)} is equal to ${sentence.charAt(
    index,
  )}`,
);
// Expected output: "Character code 113 is equal to q"
```

Syntax

```
charCodeAt(index)
```

Parameters

`index`

Zero-based index of the character to be returned. Converted to an integer — `undefined` is converted to 0.

Return value

An integer between `0` and `65535` representing the UTF-16 code unit value of the character at the specified `index`. If `index` is out of range of `0` – `str.length - 1`, `charCodeAt()` returns `NaN`.

Description

Characters in a string are indexed from left to right. The index of the first character is `0`, and the index of the last character in a string called `str` is `str.length - 1`.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). `charCodeAt()` always returns a value that is less than `65536`, because the higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, in order to get a full character with value greater than `65535`, it is necessary to retrieve not only `charCodeAt(i)`, but also `charCodeAt(i + 1)` (as if manipulating a string with two characters), or to use `codePointAt(i)` instead. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Examples

Using charCodeAt()

The following example returns `65`, the Unicode value for A.

```
"ABC".charCodeAt(0); // returns 65
```

`charCodeAt()` may return lone surrogates, which are not valid Unicode characters.

```
const str = "𠮷";
console.log(str.charCodeAt(0)); // 55362, or d842, which is not a valid Unicode character
console.log(str.charCodeAt(1)); // 57271, or dfb7, which is not a valid Unicode character
```

To get the full Unicode code point at the given index, use `String.prototype.codePointAt()`.

```
const str = "𠮷";
console.log(str.codePointAt(0)); // 134071
```

> **Note:** Avoid re-implementing `codePointAt()` using `charCodeAt()`. The translation from UTF-16 surrogates to Unicode code points is complex, and `codePointAt()` may be more performant as it directly uses the internal representation of the string. Install a polyfill for `codePointAt()` if necessary.

Below is a possible algorithm to convert a pair of UTF-16 code units into a Unicode code point, adapted from the Unicode FAQ:

```
// constants
const LEAD_OFFSET = 0xd800 - (0x10000 >> 10);
const SURROGATE_OFFSET = 0x10000 - (0xd800 << 10) - 0xdc00;

function utf16ToUnicode(lead, trail) {
  return (lead << 10) + trail + SURROGATE_OFFSET;
}
function unicodeToUTF16(codePoint) {
  const lead = LEAD_OFFSET + (codePoint >> 10);
  const trail = 0xdc00 + (codePoint & 0x3ff);
```

```
  return [lead, trail];
}

const str = "吉";
console.log(utf16ToUnicode(str.charCodeAt(0), str.charCodeAt(1))); // 134071
console.log(str.codePointAt(0)); // 134071
```

See also

- `String.fromCharCode()`
- `String.prototype.charAt()`
- `String.fromCodePoint()`
- `String.prototype.codePointAt()`

String.prototype.codePointAt()

The **`codePointAt()`** method of `String` values returns a non-negative integer that is the Unicode code point value of the character starting at the given index. Note that the index is still based on UTF-16 code units, not Unicode code points.

Try it

```
const icons = "🐻★♘";

console.log(icons.codePointAt(1));
// Expected output: "9733"
```

Syntax

```
codePointAt(index)
```

Parameters

`index`

Zero-based index of the character to be returned. Converted to an integer — `undefined` is converted to 0.

Return value

A non-negative integer representing the code point value of the character at the given `index`.

- If `index` is out of the range of `0` – `str.length - 1`, `codePointAt()` returns `undefined`.
- If the element at `index` is a UTF-16 leading surrogate, returns the code point of the surrogate *pair*.
- If the element at `index` is a UTF-16 trailing surrogate, returns *only* the trailing surrogate code unit.

Description

Characters in a string are indexed from left to right. The index of the first character is `0`, and the index of the last character in a string called `str` is `str.length - 1`.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). In UTF-16, each string index is a code unit with value `0` – `65535`. Higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, `codePointAt()` returns a code point that may span two string indices. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Examples

Using codePointAt()

```
"ABC".codePointAt(0); // 65
"ABC".codePointAt(0).toString(16); // 41

"😍".codePointAt(0); // 128525
"\ud83d\ude0d".codePointAt(0); // 128525
"\ud83d\ude0d".codePointAt(0).toString(16); // 1f60d

"😍".codePointAt(1); // 56845
"\ud83d\ude0d".codePointAt(1); // 56845
"\ud83d\ude0d".codePointAt(1).toString(16); // de0d

"ABC".codePointAt(42); // undefined
```

Looping with codePointAt()

Because using string indices for looping causes the same code point to be visited twice (once for the leading surrogate, once for the trailing surrogate), and the second time `codePointAt()` returns *only* the trailing surrogate, it's better to avoid looping by index.

```
const str = "\ud83d\udc0e\ud83d\udc71\u2764";

for (let i = 0; i < str.length; i++) {
  console.log(str.codePointAt(i).toString(16));
}
// '1f40e', 'dc0e', '1f471', 'dc71', '2764'
```

Instead, use a `for...of` statement or spread the string, both of which invoke the string's `[Symbol.iterator]()`, which iterates by code points. Then, use `codePointAt(0)` to get the code point of each element.

```
for (const codePoint of str) {
  console.log(codePoint.codePointAt(0).toString(16));
}
// '1f40e', '1f471', '2764'

[...str].map((cp) => cp.codePointAt(0).toString(16));
// ['1f40e', '1f471', '2764']
```

See also

- Polyfill of `String.prototype.codePointAt` in `core-js`
- es-shims polyfill of `String.prototype.codePointAt`
- `String.fromCodePoint()`
- `String.fromCharCode()`
- `String.prototype.charCodeAt()`
- `String.prototype.charAt()`

String.prototype.concat()

The **`concat()`** method of `String` values concatenates the string arguments to this string and returns a new string.

Try it

```
const str1 = "Hello";
const str2 = "World";

console.log(str1.concat(" ", str2));
// Expected output: "Hello World"

console.log(str2.concat(", ", str1));
// Expected output: "World, Hello"
```

Syntax

```
concat(str1)
concat(str1, str2)
concat(str1, str2, /* …, */ strN)
```

Parameters

`str1, ..., strN`

> One or more strings to concatenate to `str`. Though technically permitted, calling `String.prototype.concat()` with no arguments is a useless operation, because it does not result in observable copying (like `Array.prototype.concat()`), since strings are immutable. It should only happen if you are spreading an array of strings as arguments, and that array happens to be empty.

Return value

A new string containing the combined text of the strings provided.

Description

The `concat()` function concatenates the string arguments to the calling string and returns a new string.

If the arguments are not of the type string, they are converted to string values before concatenating.

The `concat()` method is very similar to the addition/string concatenation operators (`+`, `+=`), except that `concat()` coerces its arguments directly to strings, while addition coerces its operands to primitives first. For more information, see the reference page for the `+` operator.

Examples

Using concat()

The following example combines strings into a new string.

```
const hello = "Hello, ";
console.log(hello.concat("Kevin", ". Have a nice day."));
// Hello, Kevin. Have a nice day.

const greetList = ["Hello", " ", "Venkat", "!"];
"".concat(...greetList); // "Hello Venkat!"

"".concat({}); // "[object Object]"
"".concat([]); // ""
"".concat(null); // "null"
"".concat(true); // "true"
"".concat(4, 5); // "45"
```

See also

- `Array.prototype.concat()`
- Addition (+)

String.prototype.endsWith()

The **`endsWith()`** method of `String` values determines whether a string ends with the characters of this string, returning `true` or `false` as appropriate.

Try it

```
const str1 = "Cats are the best!";

console.log(str1.endsWith("best!"));
// Expected output: true

console.log(str1.endsWith("best", 17));
// Expected output: true

const str2 = "Is this a question?";

console.log(str2.endsWith("question"));
// Expected output: false
```

Syntax

```
endsWith(searchString)
endsWith(searchString, endPosition)
```

Parameters

`searchString`

> The characters to be searched for at the end of `str`. Cannot be a regex. All values that are not regexes are coerced to strings, so omitting it or passing `undefined` causes `endsWith()` to search for the string `"undefined"`, which is rarely what you want.

`endPosition` Optional

> The end position at which `searchString` is expected to be found (the index of `searchString`'s last character plus 1). Defaults to `str.length`.

Return value

`true` if the given characters are found at the end of the string, including when `searchString` is an empty string; otherwise, **`false`**.

Exceptions

`TypeError`

> Thrown if `searchString` is a regex.

Description

This method lets you determine whether or not a string ends with another string. This method is case-sensitive.

Examples

Using endsWith()

```
const str = "To be, or not to be, that is the question.";

console.log(str.endsWith("question.")); // true
console.log(str.endsWith("to be")); // false
console.log(str.endsWith("to be", 19)); // true
```

See also

- Polyfill of `String.prototype.endsWith` in `core-js`
- es-shims polyfill of `String.prototype.endsWith`
- `String.prototype.startsWith()`
- `String.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`

String.prototype.fixed()

The **`fixed()`** method of `String` values creates a string that embeds this string in a `<tt>` element (`<tt>str</tt>`), which causes this string to be displayed in a fixed-width font.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `fixed()`, the `<tt>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```
fixed()
```

Parameters

None.

Return value

A string beginning with a `<tt>` start tag, then the text `str`, and then a `</tt>` end tag.

Examples

Using fixed()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.fixed();
```

This will create the following HTML:

```
<tt>Hello, world</tt>
```

> **Warning:** This markup is invalid, because `tt` is no longer a valid element.

Instead of using `fixed()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `font-family` through the `element.style` attribute:

```
document.getElementById("yourElemId").style.fontFamily = "monospace";
```

See also

- Polyfill of `String.prototype.fixed` in `core-js`
- es-shims polyfill of `String.prototype.fixed`
- HTML wrapper methods
- `<tt>`

String.prototype.fontcolor()

The **`fontcolor()`** method of `String` values creates a string that embeds this string in a `<font>` element (`<font color="...">str</font>`), which causes this string to be displayed in the specified font color.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `fontcolor()`, the `<font>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```
fontcolor(color)
```

Parameters

`color`

A string expressing the color as a hexadecimal RGB triplet or as a string literal. String literals for color names are listed in the CSS color reference.

Return value

A string beginning with a `<font color="color">` start tag (double quotes in `color` are replaced with `"`), then the text `str`, and then a `</font>` end tag.

Description

The `fontcolor()` method itself simply joins the string parts together without any validation or normalization. However, to create valid `<font>` elements, if you express color as a hexadecimal RGB triplet, you must use the format `rrggbb`. For example, the hexadecimal RGB values for salmon are red=FA, green=80, and blue=72, so the RGB triplet for salmon is `"FA8072"`.

Examples

Using fontcolor()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.fontcolor("red");
```

This will create the following HTML:

```
<font color="red">Hello, world</font>
```

> **Warning:** This markup is invalid, because `font` is no longer a valid element.

Instead of using `fontcolor()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `color` through the `element.style` attribute:

```
document.getElementById("yourElemId").style.color = "red";
```

See also

- Polyfill of `String.prototype.fontcolor` in `core-js`
- es-shims polyfill of `String.prototype.fontcolor`
- HTML wrapper methods
- `<font>`

String.prototype.fontsize()

The **`fontsize()`** method of `String` values creates a string that embeds this string in a `<font>` element (`<font size="...">str</font>`), which causes this string to be displayed in the specified font size.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `fontsize()`, the `<font>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use CSS properties instead.

Syntax

```
fontsize(size)
```

Parameters

`size`

An integer between 1 and 7, or a string representing a signed integer between 1 and 7.

Return value

A string beginning with a `<font size="size">` start tag (double quotes in `size` are replaced with `"`), then the text `str`, and then a `</font>` end tag.

Description

The `fontsize()` method itself simply joins the string parts together without any validation or normalization. However, to create valid `<font>` elements, When you specify size as an integer, you set the font size of `str` to one of the 7 defined sizes. You can specify `size` as a string such as `"-2"` or `"+3"` to adjust the font size of `str` relative to 3, the default value.

Examples

Using fontsize()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.fontsize(7);
```

This will create the following HTML:

```
<font size="7">Hello, world</font>
```

> **Warning:** This markup is invalid, because `font` is no longer a valid element.

Instead of using `fontsize()` and creating HTML text directly, you should use CSS to manipulate fonts. For example, you can manipulate `font-size` through the `element.style` attribute:

```
document.getElementById("yourElemId").style.fontSize = "7pt";
```

See also

- Polyfill of `String.prototype.fontsize` in `core-js`
- es-shims polyfill of `String.prototype.fontsize`
- HTML wrapper methods
- `<font>`

String.fromCharCode()

The **`String.fromCharCode()`** static method returns a string created from the specified sequence of UTF-16 code units.

Try it

```
console.log(String.fromCharCode(189, 43, 190, 61));
// Expected output: "½+¾="
```

Syntax

```
String.fromCharCode()
String.fromCharCode(num1)
String.fromCharCode(num1, num2)
String.fromCharCode(num1, num2, /* …, */ numN)
```

Parameters

`num1, ..., numN`

A number between `0` and `65535` (`0xFFFF`) representing a UTF-16 code unit. Numbers greater than `0xFFFF` are truncated to the last 16 bits. No validity checks are performed.

Return value

A string of length `N` consisting of the `N` specified UTF-16 code units.

Description

Because `fromCharCode()` is a static method of `String`, you always use it as `String.fromCharCode()`, rather than as a method of a `String` value you created.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). `charCodeAt()` always returns a value that is less than `65536`, because the higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, in order to produce a full character with value greater than `65535`, it is necessary to provide two code units (as if manipulating a string with two characters). For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Because `fromCharCode()` only works with 16-bit values (same as the `\u` escape sequence), a surrogate pair is required in order to return a supplementary character. For example, both `String.fromCharCode(0xd83c, 0xdf03)` and `"\ud83c\udf03"` return code point `U+1F303` "Night with Stars". While there is a mathematical relationship between the supplementary code point value (e.g., `0x1f303`) and both surrogate values that represent it (e.g., `0xd83c` and `0xdf03`), it does require an extra step to either calculate or look up the surrogate pair values every time a supplementary code point is to be used. For this reason, it's more convenient to use `String.fromCodePoint()`, which allows for returning supplementary characters based on their actual code point value. For example, `String.fromCodePoint(0x1f303)` returns code point `U+1F303` "Night with Stars".

Examples

Using fromCharCode()

BMP characters, in UTF-16, use a single code unit:

```
String.fromCharCode(65, 66, 67); // returns "ABC"
String.fromCharCode(0x2014); // returns "—"
String.fromCharCode(0x12014); // also returns "—"; the digit 1 is truncated and ignored
String.fromCharCode(8212); // also returns "—"; 8212 is the decimal form of 0x2014
```

Supplementary characters, in UTF-16, require two code units (i.e., a surrogate pair):

```
String.fromCharCode(0xd83c, 0xdf03); // Code Point U+1F303 "Night with
String.fromCharCode(55356, 57091); // Stars" === "\uD83C\uDF03"

String.fromCharCode(0xd834, 0xdf06, 0x61, 0xd834, 0xdf07); // "\uD834\uDF06a\uD834\uDF07"
```

See also

- `String.fromCodePoint()`
- `String.prototype.charAt()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`

String.fromCodePoint()

The **`String.fromCodePoint()`** static method returns a string created from the specified sequence of code points.

Try it

```
console.log(String.fromCodePoint(9731, 9733, 9842, 0x2f804));
// Expected output: "☃★♲你"
```

Syntax

```
String.fromCodePoint()
String.fromCodePoint(num1)
String.fromCodePoint(num1, num2)
String.fromCodePoint(num1, num2, /* …, */ numN)
```

Parameters

`num1, ..., numN`

An integer between `0` and `0x10FFFF` (inclusive) representing a Unicode code point.

Return value

A string created by using the specified sequence of code points.

Exceptions

`RangeError`

Thrown if `numN` is not an integer, is less than `0`, or is greater than `0x10FFFF` after being converted to a number.

Description

Because `fromCodePoint()` is a static method of `String`, you always use it as `String.fromCodePoint()`, rather than as a method of a `String` value you created.

Unicode code points range from `0` to `1114111` (`0x10FFFF`). In UTF-16, each string index is a code unit with value `0` – `65535`. Higher code points are represented by *a pair* of 16-bit surrogate pseudo-characters. Therefore, `fromCodePoint()` may return a string whose `length` (in UTF-16 code units) is larger than the number of arguments passed. For information on Unicode, see UTF-16 characters, Unicode code points, and grapheme clusters.

Examples

Using fromCodePoint()

Valid input:

```
String.fromCodePoint(42); // "*"
String.fromCodePoint(65, 90); // "AZ"
String.fromCodePoint(0x404); // "\u0404" === "Є"
String.fromCodePoint(0x2f804); // "\uD87E\uDC04"
String.fromCodePoint(194564); // "\uD87E\uDC04"
String.fromCodePoint(0x1d306, 0x61, 0x1d307); // "\uD834\uDF06a\uD834\uDF07"
```

Invalid input:

```
String.fromCodePoint("_"); // RangeError
String.fromCodePoint(Infinity); // RangeError
String.fromCodePoint(-1); // RangeError
String.fromCodePoint(3.14); // RangeError
String.fromCodePoint(3e-2); // RangeError
String.fromCodePoint(NaN); // RangeError
```

Compared to fromCharCode()

`String.fromCharCode()` cannot return supplementary characters (i.e., code points `0x010000` – `0x10FFFF`) by specifying their code point. Instead, it requires the UTF-16 surrogate pair in order to return a supplementary character:

```
String.fromCharCode(0xd83c, 0xdf03); // Code Point U+1F303 "Night with
String.fromCharCode(55356, 57091); // Stars" === "\uD83C\uDF03"
```

`String.fromCodePoint()`, on the other hand, can return 4-byte supplementary characters, as well as the more common 2-byte BMP characters, by specifying their code point (which is equivalent to the UTF-32 code unit):

```
String.fromCodePoint(0x1f303); // or 127747 in decimal
```

See also

- Polyfill of `String.fromCodePoint` in `core-js`
- es-shims polyfill of `String.fromCodePoint`
- `String.fromCharCode()`
- `String.prototype.charAt()`
- `String.prototype.codePointAt()`
- `String.prototype.charCodeAt()`

String.prototype.includes()

The **`includes()`** method of `String` values performs a case-sensitive search to determine whether a given string may be found within this string, returning `true` or `false` as appropriate.

Try it

```
const sentence = "The quick brown fox jumps over the lazy dog.";

const word = "fox";

console.log(
  `The word "${word}" ${
    sentence.includes(word) ? "is" : "is not"
  } in the sentence`,
```

```
);
// Expected output: "The word "fox" is in the sentence"
```

Syntax

```
includes(searchString)
includes(searchString, position)
```

Parameters

`searchString`

> A string to be searched for within `str`. Cannot be a regex. All values that are not regexes are coerced to strings, so omitting it or passing `undefined` causes `includes()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

> The position within the string at which to begin searching for `searchString`. (Defaults to `0`.)

Return value

`true` if the search string is found anywhere within the given string, including when `searchString` is an empty string; otherwise, **`false`**.

Exceptions

`TypeError`

> Thrown if `searchString` is a regex.

Description

This method lets you determine whether or not a string includes another string.

Case-sensitivity

The `includes()` method is case sensitive. For example, the following expression returns `false`:

```
"Blue Whale".includes("blue"); // returns false
```

You can work around this constraint by transforming both the original string and the search string to all lowercase:

```
"Blue Whale".toLowerCase().includes("blue"); // returns true
```

Examples

Using includes()

```
const str = "To be, or not to be, that is the question.";

console.log(str.includes("To be")); // true
console.log(str.includes("question")); // true
console.log(str.includes("nonexistent")); // false
console.log(str.includes("To be", 1)); // false
console.log(str.includes("TO BE")); // false
console.log(str.includes("")); // true
```

See also

- Polyfill of `String.prototype.includes` in `core-js`
- es-shims polyfill of `String.prototype.includes`
- `Array.prototype.includes()`
- `TypedArray.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.startsWith()`
- `String.prototype.endsWith()`

String.prototype.indexOf()

The **`indexOf()`** method of `String` values searches this string and returns the index of the first occurrence of the specified substring. It takes an optional starting position and returns the first occurrence of the specified substring at an index greater than or equal to the specified number.

Try it

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

const searchTerm = "dog";
const indexOfFirst = paragraph.indexOf(searchTerm);

console.log(`The index of the first "${searchTerm}" is ${indexOfFirst}`);
// Expected output: "The index of the first "dog" is 15"
```

```
console.log(
  `The index of the second "${searchTerm}" is ${paragraph.indexOf(
    searchTerm,
    indexOfFirst + 1,
  )}`,
);
// Expected output: "The index of the second "dog" is 38"
```

Syntax

```
indexOf(searchString)
indexOf(searchString, position)
```

Parameters

`searchString`

Substring to search for. All values are coerced to strings, so omitting it or passing `undefined` causes `indexOf()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

The method returns the index of the first occurrence of the specified substring at a position greater than or equal to `position`, which defaults to `0`. If `position` is greater than the length of the calling string, the method doesn't search the calling string at all. If `position` is less than zero, the method behaves as it would if `position` were `0`.

- `'hello world hello'.indexOf('o', -5)` returns `4` — because it causes the method to behave as if the second argument were `0`, and the first occurrence of `o` at a position greater or equal to `0` is at position `4`.
- `'hello world hello'.indexOf('world', 12)` returns `-1` — because, while it's true the substring `world` occurs at index `6`, that position is not greater than or equal to `12`.
- `'hello world hello'.indexOf('o', 99)` returns `-1` — because `99` is greater than the length of `hello world hello`, which causes the method to not search the string at all.

Return value

The index of the first occurrence of `searchString` found, or `-1` if not found.

Return value when using an empty search string

Searching for an empty search string produces strange results. With no second argument, or with a second argument whose value is less than the calling string's length, the return value is the same as the value of the second argument:

```
"hello world".indexOf(""); // returns 0
"hello world".indexOf("", 0); // returns 0
"hello world".indexOf("", 3); // returns 3
"hello world".indexOf("", 8); // returns 8
```

However, with a second argument whose value is greater than or equal to the string's length, the return value is the string's length:

```
"hello world".indexOf("", 11); // returns 11
"hello world".indexOf("", 13); // returns 11
"hello world".indexOf("", 22); // returns 11
```

In the former instance, the method behaves as if it found an empty string just after the position specified in the second argument. In the latter instance, the method behaves as if it found an empty string at the end of the calling string.

Description

Strings are zero-indexed: The index of a string's first character is `0`, and the index of a string's last character is the length of the string minus 1.

```
"Blue Whale".indexOf("Blue"); // returns  0
"Blue Whale".indexOf("Wale"); // returns -1
"Blue Whale".indexOf("Whale", 0); // returns  5
"Blue Whale".indexOf("Whale", 5); // returns  5
"Blue Whale".indexOf("Whale", 7); // returns -1
"Blue Whale".indexOf(""); // returns  0
"Blue Whale".indexOf("", 9); // returns  9
"Blue Whale".indexOf("", 10); // returns 10
"Blue Whale".indexOf("", 11); // returns 10
```

The `indexOf()` method is case sensitive. For example, the following expression returns `-1`:

```
"Blue Whale".indexOf("blue"); // returns -1
```

Checking occurrences

When checking if a specific substring occurs within a string, the correct way to check is test whether the return value is `-1`:

```
"Blue Whale".indexOf("Blue") !== -1; // true; found 'Blue' in 'Blue Whale'
```

```
"Blue Whale".indexOf("Wale") !== -1; // false; no 'Wale' in 'Blue Whale'
```

Examples

Using indexOf()

The following example uses `indexOf()` to locate substrings in the string `"Brave new world"`.

```
const str = "Brave new world";

console.log(str.indexOf("w")); // 8
console.log(str.indexOf("new")); // 6
```

indexOf() and case-sensitivity

The following example defines two string variables.

The variables contain the same string, except that the second string contains uppercase letters. The first `console.log()` method displays `19`. But because the `indexOf()` method is case sensitive, the string `"cheddar"` is not found in `myCapString`, so the second `console.log()` method displays `-1`.

```
const myString = "brie, pepper jack, cheddar";
const myCapString = "Brie, Pepper Jack, Cheddar";

console.log(myString.indexOf("cheddar")); // 19
console.log(myCapString.indexOf("cheddar")); // -1
```

Using indexOf() to count occurrences of a letter in a string

The following example sets `count` to the number of occurrences of the letter `e` in the string `str`:

```
const str = "To be, or not to be, that is the question.";
let count = 0;
let position = str.indexOf("e");

while (position !== -1) {
  count++;
  position = str.indexOf("e", position + 1);
}

console.log(count); // 4
```

See also

- `String.prototype.charAt()`
- `String.prototype.lastIndexOf()`
- `String.prototype.includes()`
- `String.prototype.split()`
- `Array.prototype.indexOf()`

String.prototype.isWellFormed()

The **`isWellFormed()`** method of `String` values returns a boolean indicating whether this string contains any lone surrogates.

Syntax

```
isWellFormed()
```

Parameters

None.

Return value

Returns `true` if this string does not contain any lone surrogates, `false` otherwise.

Description

Strings in JavaScript are UTF-16 encoded. UTF-16 encoding has the concept of *surrogate pairs*, which is introduced in detail in the UTF-16 characters, Unicode code points, and grapheme clusters section.

`isWellFormed()` allows you to test whether a string is well-formed (i.e., does not contain any lone surrogates). Compared to a custom implementation, `isWellFormed()` is more efficient, as engines can directly access the internal representation of strings. If you need to convert a string to a well-formed string, use the `toWellFormed()` method. `isWellFormed()` allows you to handle ill-formed strings differently from well-formed strings, such as throwing an error or marking it as invalid.

Examples

Using isWellFormed()

```
const strings = [
  // Lone leading surrogate
  "ab\uD800",
  "ab\uD800c",
```

```
  // Lone trailing surrogate
  "\uDFFFab",
  "c\uDFFFab",
  // Well-formed
  "abc",
  "ab\uD83D\uDE04c",
];

for (const str of strings) {
  console.log(str.isWellFormed());
}
// Logs:
// false
// false
// false
// false
// true
// true
```

Avoiding errors in encodeURI()

`encodeURI` throws an error if the string passed is not well-formed. This can be avoided by using `isWellFormed()` to test the string before passing it to `encodeURI()`.

```
const illFormed = "https://example.com/search?q=\uD800";

try {
  encodeURI(illFormed);
} catch (e) {
  console.log(e); // URIError: URI malformed
}

if (illFormed.isWellFormed()) {
  console.log(encodeURI(illFormed));
} else {
  console.warn("Ill-formed strings encountered."); // Ill-formed strings encountered.
}
```

See also

- Polyfill of `String.prototype.isWellFormed` in `core-js`
- es-shims polyfill of `String.prototype.isWellFormed`
- `String.prototype.toWellFormed()`
- `String.prototype.normalize()`

String.prototype.italics()

The **`italics()`** method of `String` values creates a string that embeds this string in an `<i>` element (`<i>str</i>`), which causes this string to be displayed as italic.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
italics()
```

Parameters

None.

Return value

A string beginning with an `<i>` start tag, then the text `str`, and then an `</i>` end tag.

Examples

Using italics()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.italics();
```

This will create the following HTML:

```
<i>Hello, world</i>
```

Instead of using `italics()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("i");
```

```
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.italics` in `core-js`
- es-shims polyfill of `String.prototype.italics`
- HTML wrapper methods
- `<i>`

String.prototype.lastIndexOf()

The **`lastIndexOf()`** method of `String` values searches this string and returns the index of the last occurrence of the specified substring. It takes an optional starting position and returns the last occurrence of the specified substring at an index less than or equal to the specified number.

Try it

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

const searchTerm = "dog";

console.log(
  `Index of the last "${searchTerm}" is ${paragraph.lastIndexOf(searchTerm)}`,
);
// Expected output: "Index of the last "dog" is 38"
```

Syntax

```
lastIndexOf(searchString)
lastIndexOf(searchString, position)
```

Parameters

`searchString`

> Substring to search for. All values are coerced to strings, so omitting it or passing `undefined` causes `lastIndexOf()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

> The method returns the index of the last occurrence of the specified substring at a position less than or equal to `position`, which defaults to `Infinity`. If `position` is greater than the length of the calling string, the method searches the entire string. If `position` is less than `0`, the behavior is the same as for `0` — that is, the method looks for the specified substring only at index `0`.
>
> - `'hello world hello'.lastIndexOf('world', 4)` returns `-1` — because, while the substring `world` does occurs at index `6`, that position is not less than or equal to `4`.
> - `'hello world hello'.lastIndexOf('hello', 99)` returns `12` — because the last occurrence of `hello` at a position less than or equal to `99` is at position `12`.
> - `'hello world hello'.lastIndexOf('hello', 0)` and `'hello world hello'.lastIndexOf('hello', -5)` both return `0` — because both cause the method to only look for `hello` at index `0`.

Return value

The index of the last occurrence of `searchString` found, or `-1` if not found.

Description

Strings are zero-indexed: The index of a string's first character is `0`, and the index of a string's last character is the length of the string minus 1.

```
"canal".lastIndexOf("a"); // returns 3
"canal".lastIndexOf("a", 2); // returns 1
"canal".lastIndexOf("a", 0); // returns -1
"canal".lastIndexOf("x"); // returns -1
"canal".lastIndexOf("c", -5); // returns 0
"canal".lastIndexOf("c", 0); // returns 0
"canal".lastIndexOf(""); // returns 5
"canal".lastIndexOf("", 2); // returns 2
```

Case-sensitivity

The `lastIndexOf()` method is case sensitive. For example, the following expression returns `-1`:

```
"Blue Whale, Killer Whale".lastIndexOf("blue"); // returns -1
```

Examples

Using indexOf() and lastIndexOf()

The following example uses `indexOf()` and `lastIndexOf()` to locate values in the string `"Brave, Brave New World"`.

```
const anyString = "Brave, Brave New World";

console.log(anyString.indexOf("Brave")); // 0
console.log(anyString.lastIndexOf("Brave")); // 7
```

See also

- `String.prototype.charAt()`
- `String.prototype.indexOf()`
- `String.prototype.split()`
- `Array.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`

String: length

The **`length`** data property of a `String` value contains the length of the string in UTF-16 code units.

Try it

```
const str = "Life, the universe and everything. Answer:";

console.log(`${str} ${str.length}`);
// Expected output: "Life, the universe and everything. Answer: 42"
```

Value

A non-negative integer.

Property attributes of `String: length`	
Writable	no
Enumerable	no
Configurable	no

Description

This property returns the number of code units in the string. JavaScript uses UTF-16 encoding, where each Unicode character may be encoded as one or two code units, so it's possible for the value returned by `length` to not match the actual number of Unicode characters in the string. For common scripts like Latin, Cyrillic, wellknown CJK characters, etc., this should not be an issue, but if you are working with certain scripts, such as emojis, mathematical symbols, or obscure Chinese characters, you may need to account for the difference between code units and characters.

The language specification requires strings to have a maximum length of 2^{53} - 1 elements, which is the upper limit for precise integers. However, a string with this length needs 16384TiB of storage, which cannot fit in any reasonable device's memory, so implementations tend to lower the threshold, which allows the string's length to be conveniently stored in a 32-bit integer.

- In V8 (used by Chrome and Node), the maximum length is 2^{29} - 24 (~1GiB). On 32-bit systems, the maximum length is 2^{28} - 16 (~512MiB).
- In Firefox, the maximum length is 2^{30} - 2 (~2GiB). Before Firefox 65, the maximum length was 2^{28} - 1 (~512MiB).
- In Safari, the maximum length is 2^{31} - 1 (~4GiB).

> **Note:** All memory sizes above assume strings are stored in UTF-16 encoding in memory, which may be an overestimate. The actual space used by a string is subject to JavaScript engine optimizations, and all major engines use a more compact internal representation, such as one-byte storage, for strings containing only Latin-1 characters.

If you are working with large strings in other encodings (such as UTF-8 files or blobs), note that when you load the data into a JS string, the encoding always becomes UTF-16. The size of the string may be different from the size of the source file.

```
const str1 = "a".repeat(2 ** 29 - 24); // Success
const str2 = "a".repeat(2 ** 29 - 23); // RangeError: Invalid string length

const buffer = new Uint8Array(2 ** 29 - 24).fill("a".codePointAt(0)); // This buffer is 512MiB in size
const str = new TextDecoder().decode(buffer); // This string is 1GiB in size unless optimized
```

For an empty string, `length` is 0.

The static property `String.length` is unrelated to the length of strings. It's the arity of the `String` function (loosely, the number of formal parameters it has), which is 1.

Since `length` counts code units instead of characters, if you want to get the number of characters, you can first split the string with its iterator, which iterates by characters:

```
function getCharacterLength(str) {
  // The string iterator that is used here iterates over characters,
  // not mere code units
  return [...str].length;
}

console.log(getCharacterLength("A\uD87E\uDC04Z")); // 3
```

If you want to count characters by *grapheme clusters*, use `Intl.Segmenter`. You can first pass the string you want to split to the `segment()` method, and then iterate over the returned `Segments` object to get the length:

```
function getGraphemeCount(str) {
  const segmenter = new Intl.Segmenter("en-US", { granularity: "grapheme" });
  // The Segments object iterator that is used here iterates over characters in grapheme clusters,
  // which may consist of multiple Unicode characters
  return [...segmenter.segment(str)].length;
}

console.log(getGraphemeCount("👩‍👩‍👦‍👦")); // 1
```

Examples

Basic usage

```
const x = "Mozilla";
const empty = "";

console.log(`${x} is ${x.length} code units long`);
// Mozilla is 7 code units long

console.log(`The empty string has a length of ${empty.length}`);
// The empty string has a length of 0
```

Strings with length not equal to the number of characters

```
const emoji = "😄";
console.log(emoji.length); // 2
console.log([...emoji].length); // 1
const adlam = "𞤤𞤵𞤲";
console.log(adlam.length); // 8
console.log([...adlam].length); // 4
const formula = "∀𝑥∈ℝ,𝑥²≥0";
console.log(formula.length); // 11
console.log([...formula].length); // 9
```

Assigning to length

Because string is a primitive, attempting to assign a value to a string's `length` property has no observable effect, and will throw in strict mode.

```
const myString = "bluebells";

myString.length = 4;
console.log(myString); // "bluebells"
console.log(myString.length); // 9
```

See also

- `Array: length`

String.prototype.link()

The **`link()`** method of `String` values creates a string that embeds this string in an `<a>` element (`<a href="...">str</a>`), to be used as a hypertext link to another URL.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
link(url)
```

Parameters

`url`

Any string that specifies the `href` attribute of the `<a>` element; it should be a valid URL (relative or absolute), with any `&` characters escaped as `&`.

Return value

A string beginning with an `<a href="url">` start tag (double quotes in `url` are replaced with `"`), then the text `str`, and then an `</a>` end tag.

Examples

Using link()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "MDN Web Docs";

document.body.innerHTML = contentString.link("https://developer.mozilla.org/");
```

This will create the following HTML:

```
<a href="https://developer.mozilla.org/">MDN Web Docs</a>
```

Instead of using `link()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "MDN Web Docs";
const elem = document.createElement("a");
elem.href = "https://developer.mozilla.org/";
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.link` in `core-js`
- es-shims polyfill of `String.prototype.link`
- HTML wrapper methods
- `<a>`

String.prototype.localeCompare()

The **`localeCompare()`** method of `String` values returns a number indicating whether this string comes before, or after, or is the same as the given string in sort order. In implementations with `Intl.Collator` API support, this method delegates to `Intl.Collator`.

When comparing large numbers of strings, such as in sorting large arrays, it is better to create an `Intl.Collator` object and use the function provided by its `compare()` method.

Try it

```
const a = "réservé"; // With accents, lowercase
const b = "RESERVE"; // No accents, uppercase

console.log(a.localeCompare(b));
// Expected output: 1
console.log(a.localeCompare(b, "en", { sensitivity: "base" }));
// Expected output: 0
```

Syntax

```
localeCompare(compareString)
localeCompare(compareString, locales)
localeCompare(compareString, locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.Collator` API, these parameters correspond exactly to the `Intl.Collator()` constructor's parameters. Implementations without `Intl.Collator` support are asked to ignore both parameters, making the comparison result returned entirely implementation-dependent — it's only required to be *consistent*.

`compareString`

> The string against which the `referenceStr` is compared. All values are coerced to strings, so omitting it or passing `undefined` causes `localeCompare()` to compare against the string `"undefined"`, which is rarely what you want.

`locales` Optional

> A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.Collator()` constructor.
>
> In implementations without `Intl.Collator` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

> An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.Collator()` constructor.
>
> In implementations without `Intl.Collator` support, this parameter is ignored.

See the `Intl.Collator()` constructor for details on the `locales` and `options` parameters and how to use them.

Return value

A **negative** number if `referenceStr` occurs before `compareString`; **positive** if the `referenceStr` occurs after `compareString`; `0` if they are equivalent.

In implementations with `Intl.Collator`, this is equivalent to `new Intl.Collator(locales, options).compare(referenceStr, compareString)`.

Description

Returns an integer indicating whether the `referenceStr` comes before, after or is equivalent to the `compareString`.

- Negative when the `referenceStr` occurs before `compareString`
- Positive when the `referenceStr` occurs after `compareString`

- Returns `0` if they are equivalent

> **Warning:** Do not rely on exact return values of `-1` or `1`!
>
> Negative and positive integer results vary between browsers (as well as between browser versions) because the ECMAScript specification only mandates negative and positive values. Some browsers may return `-2` or `2`, or even some other negative or positive value.

Examples

Using localeCompare()

```
// The letter "a" is before "c" yielding a negative value
"a".localeCompare("c"); // -2 or -1 (or some other negative value)

// Alphabetically the word "check" comes after "against" yielding a positive value
"check".localeCompare("against"); // 2 or 1 (or some other positive value)

// "a" and "a" are equivalent yielding a neutral value of zero
"a".localeCompare("a"); // 0
```

Sort an array

`localeCompare()` enables case-insensitive sorting for an array.

```
const items = ["réservé", "Premier", "Cliché", "communiqué", "café", "Adieu"];
items.sort((a, b) => a.localeCompare(b, "fr", { ignorePunctuation: true }));
// ['Adieu', 'café', 'Cliché', 'communiqué', 'Premier', 'réservé']
```

Check browser support for extended arguments

The `locales` and `options` arguments are not supported in all browsers yet.

To check whether an implementation supports them, use the `"i"` argument (a requirement that illegal language tags are rejected) and look for a `RangeError` exception:

```
function localeCompareSupportsLocales() {
  try {
    "foo".localeCompare("bar", "i");
  } catch (e) {
    return e.name === "RangeError";
  }
  return false;
}
```

Using locales

The results provided by `localeCompare()` vary between languages. In order to get the sort order of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```
console.log("ä".localeCompare("z", "de")); // a negative value: in German, ä sorts before z
console.log("ä".localeCompare("z", "sv")); // a positive value: in Swedish, ä sorts after z
```

Using options

The results provided by `localeCompare()` can be customized using the `options` argument:

```
// in German, ä has a as the base letter
console.log("ä".localeCompare("a", "de", { sensitivity: "base" })); // 0

// in Swedish, ä and a are separate base letters
console.log("ä".localeCompare("a", "sv", { sensitivity: "base" })); // a positive value
```

Numeric sorting

```
// by default, "2" > "10"
console.log("2".localeCompare("10")); // 1

// numeric using options:
console.log("2".localeCompare("10", undefined, { numeric: true })); // -1

// numeric using locales tag:
console.log("2".localeCompare("10", "en-u-kn-true")); // -1
```

See also

- `Intl.Collator`

String.prototype.match()

The **`match()`** method of `String` values retrieves the result of matching this string against a regular expression.

Try it

```
const paragraph = "The quick brown fox jumps over the lazy dog. It barked.";
const regex = /[A-Z]/g;
const found = paragraph.match(regex);

console.log(found);
// Expected output: Array ["T", "I"]
```

Syntax

```
match(regexp)
```

Parameters

`regexp`

A regular expression object, or any object that has a `Symbol.match` method.

If `regexp` is not a `RegExp` object and does not have a `Symbol.match` method, it is implicitly converted to a `RegExp` by using `new RegExp(regexp)`.

If you don't give any parameter and use the `match()` method directly, you will get an `Array` with an empty string: `[""]`, because this is equivalent to `match(/(?:)/)`.

Return value

An `Array` whose contents depend on the presence or absence of the global (`g`) flag, or `null` if no matches are found.

- If the `g` flag is used, all results matching the complete regular expression will be returned, but capturing groups are not included.
- If the `g` flag is not used, only the first complete match and its related capturing groups are returned. In this case, `match()` will return the same result as `RegExp.prototype.exec()` (an array with some extra properties).

Description

The implementation of `String.prototype.match` doesn't do much other than calling the `Symbol.match` method of the argument with the string as the first parameter. The actual implementation comes from `RegExp.prototype[Symbol.match]()`.

- If you need to know if a string matches a regular expression `RegExp`, use `RegExp.prototype.test()`.
- If you only want the first match found, you might want to use `RegExp.prototype.exec()` instead.
- If you want to obtain capture groups and the global flag is set, you need to use `RegExp.prototype.exec()` or `String.prototype.matchAll()` instead.

For more information about the semantics of `match()` when a regex is passed, see `RegExp.prototype[Symbol.match]()`.

Examples

Using match()

In the following example, `match()` is used to find `"Chapter"` followed by one or more numeric characters followed by a decimal point and numeric character zero or more times.

The regular expression includes the `i` flag so that upper/lower case differences will be ignored.

```
const str = "For more information, see Chapter 3.4.5.1";
const re = /see (chapter \d+(\.\d)*)/i;
const found = str.match(re);

console.log(found);
// [
//   'see Chapter 3.4.5.1',
//   'Chapter 3.4.5.1',
//   '.1',
//   index: 22,
//   input: 'For more information, see Chapter 3.4.5.1',
//   groups: undefined
// ]
```

In the match result above, `'see Chapter 3.4.5.1'` is the whole match. `'Chapter 3.4.5.1'` was captured by `(chapter \d+(\.\d)*)`. `'.1'` was the last value captured by `(\.\d)`. The `index` property (`22`) is the zero-based index of the whole match. The `input` property is the original string that was parsed.

Using global and ignoreCase flags with match()

The following example demonstrates the use of the global flag and ignore-case flag with `match()`. All letters `A` through `E` and `a` through `e` are returned, each its own element in the array.

```
const str = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
const regexp = /[a-e]/gi;
const matches = str.match(regexp);

console.log(matches);
// ['A', 'B', 'C', 'D', 'E', 'a', 'b', 'c', 'd', 'e']
```

> **Note:** See also `String.prototype.matchAll()` and Advanced searching with flags.

Using named capturing groups

In browsers which support named capturing groups, the following code captures `"fox"` or `"cat"` into a group named `animal`:

```
const paragraph = "The quick brown fox jumps over the lazy dog. It barked.";

const capturingRegex = /(?<animal>fox|cat) jumps over/;
const found = paragraph.match(capturingRegex);
console.log(found.groups); // {animal: "fox"}
```

Using match() with no parameter

```
const str = "Nothing will come of nothing.";

str.match(); // returns [""]
```

Using match() with a non-RegExp implementing <code>[Symbol.match]()</code>

If an object has a `Symbol.match` method, it can be used as a custom matcher. The return value of `Symbol.match` becomes the return value of `match()`.

```
const str = "Hmm, this is interesting.";

str.match({
  [Symbol.match](str) {
    return ["Yes, it's interesting."];
  },
}); // returns ["Yes, it's interesting."]
```

A non-RegExp as the parameter

When the `regexp` parameter is a string or a number, it is implicitly converted to a `RegExp` by using `new RegExp(regexp)`.

```
const str1 =
  "All numbers except NaN satisfy <= Infinity and >= -Infinity in JavaScript.";
const str2 =
  "My grandfather is 65 years old and My grandmother is 63 years old.";
const str3 = "The contract was declared null and void.";
str1.match("number"); // "number" is a string. returns ["number"]
str1.match(NaN); // the type of NaN is the number. returns ["NaN"]
str1.match(Infinity); // the type of Infinity is the number. returns ["Infinity"]
str1.match(-Infinity); // returns ["-Infinity"]
str2.match(65); // returns ["65"]
str3.match(null); // returns ["null"]
```

This may have unexpected results if special characters are not properly escaped.

```
console.log("123".match("1.3")); // [ "123" ]
```

This is a match because `.` in a regex matches any character. In order to make it only match specifically a dot character, you need to escape the input.

```
console.log("123".match("1\\.3")); // null
```

See also

- Polyfill of `String.prototype.match` in `core-js` with fixes and implementation of modern behavior like `Symbol.match` support
- Regular expressions guide
- `String.prototype.matchAll()`
- `RegExp`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

String.prototype.matchAll()

The **`matchAll()`** method of `String` values returns an iterator of all results matching this string against a regular expression, including capturing groups.

Try it

```
const regexp = /t(e)(st(\d?))/g;
const str = "test1test2";

const array = [...str.matchAll(regexp)];

console.log(array[0]);
// Expected output: Array ["test1", "e", "st1", "1"]

console.log(array[1]);
// Expected output: Array ["test2", "e", "st2", "2"]
```

Syntax

```
matchAll(regexp)
```

Parameters

`regexp`

> A regular expression object, or any object that has a `Symbol.matchAll` method.
>
> If `regexp` is not a `RegExp` object and does not have a `Symbol.matchAll` method, it is implicitly converted to a `RegExp` by using `new RegExp(regexp, 'g')`.
>
> If `regexp` is a regex, then it must have the global (`g`) flag set, or a `TypeError` is thrown.

Return value

An iterable iterator object (which is not restartable) of matches or an empty iterator if no matches are found. Each value yielded by the iterator is an array with the same shape as the return value of `RegExp.prototype.exec()`.

Exceptions

`TypeError`

> Thrown if the `regexp` is a regex that does not have the global (`g`) flag set (its `flags` property does not contain `"g"`).

Description

The implementation of `String.prototype.matchAll` doesn't do much other than calling the `Symbol.matchAll` method of the argument with the string as the first parameter (apart from the extra input validation that the regex is global). The actual implementation comes from `RegExp.prototype[Symbol.matchAll]()`.

Examples

Regexp.prototype.exec() and matchAll()

Without `matchAll()`, it's possible to use calls to `regexp.exec()` (and regexes with the `g` flag) in a loop to obtain all the matches:

```
const regexp = /foo[a-z]*/g;
const str = "table football, foosball";
let match;

while ((match = regexp.exec(str)) !== null) {
  console.log(
    `Found ${match[0]} start=${match.index} end=${regexp.lastIndex}.`,
  );
}
// Found football start=6 end=14.
// Found foosball start=16 end=24.
```

With `matchAll()` available, you can avoid the `while` loop and `exec` with `g`. Instead, you get an iterator to use with the more convenient `for...of`, array spreading, or `Array.from()` constructs:

```
const regexp = /foo[a-z]*/g;
const str = "table football, foosball";
const matches = str.matchAll(regexp);

for (const match of matches) {
  console.log(
    `Found ${match[0]} start=${match.index} end=${
      match.index + match[0].length
    }.`,
  );
}
// Found football start=6 end=14.
// Found foosball start=16 end=24.

// matches iterator is exhausted after the for...of iteration
// Call matchAll again to create a new iterator
Array.from(str.matchAll(regexp), (m) => m[0]);
// [ "football", "foosball" ]
```

`matchAll` will throw an exception if the `g` flag is missing.

```
const regexp = /[a-c]/;
const str = "abc";
str.matchAll(regexp);
// TypeError
```

`matchAll` internally makes a clone of the `regexp` — so, unlike `regexp.exec()`, `lastIndex` does not change as the string is scanned.

```
const regexp = /[a-c]/g;
regexp.lastIndex = 1;
const str = "abc";
```

```
Array.from(str.matchAll(regexp), (m) => `${regexp.lastIndex} ${m[0]}`);
// [ "1 b", "1 c" ]
```

However, this means that unlike using `regexp.exec()` in a loop, you can't mutate `lastIndex` to make the regex advance or rewind.

Better access to capturing groups (than String.prototype.match())

Another compelling reason for `matchAll` is the improved access to capture groups.

Capture groups are ignored when using `match()` with the global `g` flag:

```
const regexp = /t(e)(st(\d?))/g;
const str = "test1test2";

str.match(regexp); // ['test1', 'test2']
```

Using `matchAll`, you can access capture groups easily:

```
const array = [...str.matchAll(regexp)];

array[0];
// ['test1', 'e', 'st1', '1', index: 0, input: 'test1test2', length: 4]
array[1];
// ['test2', 'e', 'st2', '2', index: 5, input: 'test1test2', length: 4]
```

Using matchAll() with a non-RegExp implementing <code>[Symbol.matchAll]()</code>

If an object has a `Symbol.matchAll` method, it can be used as a custom matcher. The return value of `Symbol.matchAll` becomes the return value of `matchAll()`.

```
const str = "Hmm, this is interesting.";

str.matchAll({
  [Symbol.matchAll](str) {
    return [["Yes, it's interesting."]];
  },
}); // returns [["Yes, it's interesting."]]
```

See also

- Polyfill of `String.prototype.matchAll` in `core-js`
- es-shims polyfill of `String.prototype.matchAll`
- Regular expressions guide
- Groups and backreferences guide
- `String.prototype.match()`
- `RegExp`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `RegExp.prototype[Symbol.matchAll]()`

String.prototype.normalize()

The **`normalize()`** method of `String` values returns the Unicode Normalization Form of this string.

Try it

```
const name1 = "\u0041\u006d\u00e9\u006c\u0069\u0065";
const name2 = "\u0041\u006d\u0065\u0301\u006c\u0069\u0065";

console.log(`${name1}, ${name2}`);
// Expected output: "Amélie, Amélie"
console.log(name1 === name2);
// Expected output: false
console.log(name1.length === name2.length);
// Expected output: false

const name1NFC = name1.normalize("NFC");
const name2NFC = name2.normalize("NFC");

console.log(`${name1NFC}, ${name2NFC}`);
// Expected output: "Amélie, Amélie"
console.log(name1NFC === name2NFC);
// Expected output: true
console.log(name1NFC.length === name2NFC.length);
// Expected output: true
```

Syntax

```
normalize()
normalize(form)
```

Parameters

`form` Optional

One of `"NFC"`, `"NFD"`, `"NFKC"`, or `"NFKD"`, specifying the Unicode Normalization Form. If omitted or `undefined`, `"NFC"` is used.

These values have the following meanings:

`"NFC"`

Canonical Decomposition, followed by Canonical Composition.

`"NFD"`

Canonical Decomposition.

`"NFKC"`

Compatibility Decomposition, followed by Canonical Composition.

`"NFKD"`

Compatibility Decomposition.

Return value

A string containing the Unicode Normalization Form of the given string.

Exceptions

`RangeError`

Thrown if `form` isn't one of the values specified above.

Description

Unicode assigns a unique numerical value, called a *code point*, to each character. For example, the code point for `"A"` is given as U+0041. However, sometimes more than one code point, or sequence of code points, can represent the same abstract character — the character `"ñ"` for example can be represented by either of:

- The single code point U+00F1.
- The code point for `"n"` (U+006E) followed by the code point for the combining tilde (U+0303).

```
const string1 = "\u00F1";
const string2 = "\u006E\u0303";

console.log(string1); // ñ
console.log(string2); // ñ
```

However, since the code points are different, string comparison will not treat them as equal. And since the number of code points in each version is different, they even have different lengths.

```
const string1 = "\u00F1"; // ñ
const string2 = "\u006E\u0303"; // ñ

console.log(string1 === string2); // false
console.log(string1.length); // 1
console.log(string2.length); // 2
```

The `normalize()` method helps solve this problem by converting a string into a normalized form common for all sequences of code points that represent the same characters. There are two main normalization forms, one based on **canonical equivalence** and the other based on **compatibility**.

Canonical equivalence normalization

In Unicode, two sequences of code points have canonical equivalence if they represent the same abstract characters, and should always have the same visual appearance and behavior (for example, they should always be sorted in the same way).

You can use `normalize()` using the `"NFD"` or `"NFC"` arguments to produce a form of the string that will be the same for all canonically equivalent strings. In the example below we normalize two representations of the character `"ñ"`:

```
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

string1 = string1.normalize("NFD");
string2 = string2.normalize("NFD");

console.log(string1 === string2); // true
console.log(string1.length); // 2
console.log(string2.length); // 2
```

Composed and decomposed forms

Note that the length of the normalized form under `"NFD"` is `2`. That's because `"NFD"` gives you the **decomposed** version of the canonical form, in which single code points are split into multiple combining ones. The decomposed canonical form for `"ñ"` is `"\u006E\u0303"`.

You can specify `"NFC"` to get the **composed** canonical form, in which multiple code points are replaced with single code points where possible. The composed canonical form for `"ñ"` is `"\u00F1"`:

```
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

string1 = string1.normalize("NFC");
string2 = string2.normalize("NFC");

console.log(string1 === string2); // true
console.log(string1.length); // 1
console.log(string2.length); // 1
console.log(string2.codePointAt(0).toString(16)); // f1
```

Compatibility normalization

In Unicode, two sequences of code points are compatible if they represent the same abstract characters, and should be treated alike in some — but not necessarily all — applications.

All canonically equivalent sequences are also compatible, but not vice versa.

For example:

- the code point U+FB00 represents the ligature `"ﬀ"`. It is compatible with two consecutive U+0066 code points (`"ff"`).
- the code point U+24B9 represents the symbol `"Ⓓ"`. It is compatible with the U+0044 code point (`"D"`).

In some respects (such as sorting) they should be treated as equivalent—and in some (such as visual appearance) they should not, so they are not canonically equivalent.

You can use `normalize()` using the `"NFKD"` or `"NFKC"` arguments to produce a form of the string that will be the same for all compatible strings:

```
let string1 = "\uFB00";
let string2 = "\u0066\u0066";

console.log(string1); // ﬀ
console.log(string2); // ff
console.log(string1 === string2); // false
console.log(string1.length); // 1
console.log(string2.length); // 2

string1 = string1.normalize("NFKD");
string2 = string2.normalize("NFKD");

console.log(string1); // ff <- visual appearance changed
console.log(string2); // ff
console.log(string1 === string2); // true
console.log(string1.length); // 2
console.log(string2.length); // 2
```

When applying compatibility normalization it's important to consider what you intend to do with the strings, since the normalized form may not be appropriate for all applications. In the example above the normalization is appropriate for search, because it enables a user to find the string by searching for `"f"`. But it may not be appropriate for display, because the visual representation is different.

As with canonical normalization, you can ask for decomposed or composed compatible forms by passing `"NFKD"` or `"NFKC"`, respectively.

Examples

Using normalize()

```
// Initial string

// U+1E9B: LATIN SMALL LETTER LONG S WITH DOT ABOVE
// U+0323: COMBINING DOT BELOW
const str = "\u1E9B\u0323";

// Canonically-composed form (NFC)

// U+1E9B: LATIN SMALL LETTER LONG S WITH DOT ABOVE
// U+0323: COMBINING DOT BELOW
str.normalize("NFC"); // '\u1E9B\u0323'
str.normalize(); // same as above

// Canonically-decomposed form (NFD)

// U+017F: LATIN SMALL LETTER LONG S
// U+0323: COMBINING DOT BELOW
// U+0307: COMBINING DOT ABOVE
str.normalize("NFD"); // '\u017F\u0323\u0307'

// Compatibly-composed (NFKC)

// U+1E69: LATIN SMALL LETTER S WITH DOT BELOW AND DOT ABOVE
str.normalize("NFKC"); // '\u1E69'
```

```
// Compatibly-decomposed (NFKD)

// U+0073: LATIN SMALL LETTER S
// U+0323: COMBINING DOT BELOW
// U+0307: COMBINING DOT ABOVE
str.normalize("NFKD"); // '\u0073\u0323\u0307'
```

See also

- Unicode Standard Annex #15, Unicode Normalization Forms
- Unicode equivalence on Wikipedia

String.prototype.padEnd()

The **`padEnd()`** method of `String` values pads this string with a given string (repeated and/or truncated, if needed) so that the resulting string has a given length. The padding is applied from the end of this string.

Try it

```
const str1 = "Breaded Mushrooms";

console.log(str1.padEnd(25, "."));
// Expected output: "Breaded Mushrooms........"

const str2 = "200";

console.log(str2.padEnd(5));
// Expected output: "200  "
```

Syntax

```
padEnd(targetLength)
padEnd(targetLength, padString)
```

Parameters

`targetLength`

> The length of the resulting string once the current `str` has been padded. If the value is less than or equal to `str.length`, then `str` is returned as-is.

`padString` Optional

> The string to pad the current `str` with. If `padString` is too long to stay within `targetLength`, it will be truncated from the end. The default value is the space character (U+0020).

Return value

A `String` of the specified `targetLength` with `padString` applied at the end.

Examples

Using String.prototype.padEnd()

```
"abc".padEnd(10); // "abc       "
"abc".padEnd(10, "foo"); // "abcfoofoof"
"abc".padEnd(6, "123456"); // "abc123"
"abc".padEnd(1); // "abc"
```

See also

- Polyfill of `String.prototype.padEnd` in `core-js`
- es-shims polyfill of `String.prototype.padEnd`
- `String.prototype.padStart()`

String.prototype.padStart()

The **`padStart()`** method of `String` values pads this string with a given string (repeated and/or truncated, if needed) so that the resulting string has a given length. The padding is applied from the start of this string.

Try it

```
const str = "5";

console.log(str.padStart(2, "0"));
// Expected output: "05"

const fullNumber = "2034399002125581";
const last4Digits = fullNumber.slice(-4);
const maskedNumber = last4Digits.padStart(fullNumber.length, "*");

console.log(maskedNumber);
// Expected output: "************5581"
```

Syntax

```
padStart(targetLength)
padStart(targetLength, padString)
```

Parameters

`targetLength`

The length of the resulting string once the current `str` has been padded. If the value is less than or equal to `str.length`, then `str` is returned as-is.

`padString` Optional

The string to pad the current `str` with. If `padString` is too long to stay within `targetLength`, it will be truncated from the end. The default value is the space character (U+0020).

Return value

A `String` of the specified `targetLength` with `padString` applied at the start.

Examples

Using String.prototype.padStart()

```
"abc".padStart(10); // "       abc"
"abc".padStart(10, "foo"); // "foofoofabc"
"abc".padStart(6, "123465"); // "123abc"
"abc".padStart(8, "0"); // "00000abc"
"abc".padStart(1); // "abc"
```

Fixed width string number conversion

```
// JavaScript version of: (unsigned)
// printf "%0*d" width num
function leftFillNum(num, targetLength) {
  return num.toString().padStart(targetLength, "0");
}

const num = 123;
console.log(leftFillNum(num, 5)); // "00123"
```

See also

- Polyfill of `String.prototype.padStart` in `core-js`
- es-shims polyfill of `String.prototype.padStart`
- `String.prototype.padEnd()`

String.raw()

The **`String.raw()`** static method is a tag function of template literals. This is similar to the `r` prefix in Python, or the `@` prefix in C# for string literals. It's used to get the raw string form of template literals — that is, substitutions (e.g., `${foo}`) are processed, but escape sequences (e.g., `\n`) are not.

Try it

```
// Create a variable that uses a Windows
// path without escaping the backslashes:
const filePath = String.raw`C:\Development\profile\about.html`;

console.log(`The file was uploaded from: ${filePath}`);
// Expected output: "The file was uploaded from: C:\Development\profile\about.html"
```

Syntax

```
String.raw(strings)
String.raw(strings, sub1)
String.raw(strings, sub1, sub2)
String.raw(strings, sub1, sub2, /* …, */ subN)

String.raw`templateString`
```

Parameters

`strings`

Well-formed template literal array object, like `{ raw: ['foo', 'bar', 'baz'] }`. Should be an object with a `raw` property whose value is an array-like object of strings.

`sub1, ..., subN`

Contains substitution values.

`templateString`

A template literal, optionally with substitutions (`${...}`).

Return value

The raw string form of a given template literal.

Exceptions

`TypeError`

Thrown if the first argument doesn't have a `raw` property, or the `raw` property is `undefined` or `null`.

Description

In most cases, `String.raw()` is used with template literals. The first syntax mentioned above is only rarely used, because the JavaScript engine will call this with proper arguments for you, (just like with other tag functions).

`String.raw()` is the only built-in template literal tag. It has close semantics to an untagged literal since it concatenates all arguments and returns a string. You can even re-implement it with normal JavaScript code.

> **Warning:** You should not use `String.raw` directly as an "identity" tag. See Building an identity tag for how to implement this.

If `String.raw()` is called with an object whose `raw` property doesn't have a `length` property or a non-positive `length`, it returns an empty string `""`. If `substitutions.length < strings.raw.length - 1` (i.e., there are not enough substitutions to fill the placeholders — which can't happen in a well-formed tagged template literal), the rest of the placeholders are filled with empty strings.

Examples

Using String.raw()

```
String.raw`Hi\n${2 + 3}!`;
// 'Hi\\n5!', the character after 'Hi'
// is not a newline character,
// '\' and 'n' are two characters.

String.raw`Hi\u000A!`;
// 'Hi\\u000A!', same here, this time we will get the
// \, u, 0, 0, 0, A, 6 characters.
// All kinds of escape characters will be ineffective
// and backslashes will be present in the output string.
// You can confirm this by checking the .length property
// of the string.

const name = "Bob";
String.raw`Hi\n${name}!`;
// 'Hi\\nBob!', substitutions are processed.
```

Raw strings containing template literal syntax

`String.raw` is a function, so it cannot circumvent basic template literal syntax such as backticks as delimiters and `${` for substitutions. If you want to include these characters in the output string, you need to escape them with backslashes. However, since `String.raw` outputs raw strings, the backslashes will be preserved in the output.

```
String.raw`Hi \${name}!`;
// 'Hi \\${name}!', the dollar sign is escaped; there's no interpolation.
// However, the backslash is still present in the output string.

String.raw`This is a backtick: \``;
// 'This is a backtick: \\`', the backslash is still present.

String.raw`A trailing backslash: \\`;
// 'A trailing backslash: \\\\', both backslashes are present.
// If you use a single backslash at the end, it escapes the ending backtick,
// causing subsequent code to be included in the string.
```

To work around this, you can use a substitution to insert these characters.

```
String.raw`Hi ${"$"}{name}!`;
// 'Hi ${name}!', the substitution inserts a single dollar sign.
String.raw`This is a backtick: ${"`"}`;
// 'This is a backtick: `', the substitution inserts a single backtick.
String.raw`A trailing backslash: ${"\\"}`;
// 'A trailing backslash: \\', the substitution inserts a single backslash.
```

This approach works for `String.raw` because it just concatenates the raw strings and the substitutions. In general, unfortunately, there's no way for a template literal tag to receive a `raw` string that contains unescaped template literal syntax.

```
function tag(strings) {
  console.log(strings.raw[0]); // This will never contain unescaped `${` or backticks
}
```

Using String.raw with RegExp

Combining a `String.raw` template literal with the `RegExp()` constructor allows you to create regular expressions with dynamic parts (which is not possible with regex literals) without double-escaping (`\\`) regular expression escape sequences (which is not possible with normal string literals). This is also valuable in strings that contain a lot of slashes, such as file paths or URLs.

```
// A String.raw template allows a fairly readable regular expression matching a URL:
const reRawTemplate = new RegExp(
  String.raw`https://developer\.mozilla\.org/en-US/docs/Web/JavaScript/Reference/`,
);

// The same thing with a regexp literal looks like this, with \/ for
// each forward slash:
const reRegexpLiteral =
  /https:\/\/developer\.mozilla\.org\/en-US\/docs\/Web\/JavaScript\/Reference\//;

// And the same thing written with the RegExp constructor and a
// traditional string literal, with \\. for each period:
const reStringLiteral = new RegExp(
  "https://developer\\.mozilla\\.org/en-US/docs/Web/JavaScript/Reference/",
);

// String.raw also allows dynamic parts to be included
function makeURLRegExp(path) {
  return new RegExp(String.raw`https://developer\.mozilla\.org/${path}`);
}

const reDynamic = makeURLRegExp("en-US/docs/Web/JavaScript/Reference/");
const reWildcard = makeURLRegExp(".*");
```

Building an identity tag

Many tools give special treatment to literals tagged by a particular name.

```
// Some formatters will format this literal's content as HTML
const doc = html`<!doctype html>
  <html lang="en-US">
    <head>
      <title>Hello</title>
    </head>
    <body>
      <h1>Hello world!</h1>
    </body>
  </html>`;
```

One might naïvely implement the `html` tag as:

```
const html = String.raw;
```

This, in fact, works for the case above. However, because `String.raw` would concatenate the *raw* string literals instead of the "cooked" ones, escape sequences would not be processed.

```
const doc = html`<canvas>\n</canvas>`;
// "<canvas>\\n</canvas>"
```

This may not be what you want for a "true identity" tag, where the tag is purely for markup and doesn't change the literal's value. In this case, you can create a custom tag and pass the "cooked" (i.e., escape sequences are processed) literal array to `String.raw`, pretending they are raw strings.

```
const html = (strings, ...values) => String.raw({ raw: strings }, ...values);
// Some formatters will format this literal's content as HTML
const doc = html`<canvas>\n</canvas>`;
// "<canvas>\n</canvas>"; the "\n" becomes a line break
```

Notice the first argument is an object with a `raw` property, whose value is an array-like object (with a `length` property and integer indexes) representing the separated strings in the template literal. The rest of the arguments are the substitutions. Since the `raw` value can be any array-like object, it can even be a string! For example, `'test'` is treated as `['t', 'e', 's', 't']`. The following is equivalent to `` `t${0}e${1}s${2}t` ``:

```
String.raw({ raw: "test" }, 0, 1, 2); // 't0e1s2t'
```

See also

- Polyfill of `String.raw` in `core-js`
- es-shims polyfill of `String.raw`
- Template literals
- `String`
- Lexical grammar

String.prototype.repeat()

The **`repeat()`** method of `String` values constructs and returns a new string which contains the specified number of copies of this string, concatenated together.

Try it

```
const mood = "Happy! ";

console.log(`I feel ${mood.repeat(3)}`);
// Expected output: "I feel Happy! Happy! Happy! "
```

Syntax

```
repeat(count)
```

Parameters

`count`

An integer between `0` and `Infinity`, indicating the number of times to repeat the string.

Return value

A new string containing the specified number of copies of the given string.

Exceptions

`RangeError`

Thrown if `count` is negative or if `count` overflows maximum string length.

Examples

Using repeat()

```
"abc".repeat(-1); // RangeError
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count will be converted to integer)
"abc".repeat(1 / 0); // RangeError

({ toString: () => "abc", repeat: String.prototype.repeat }).repeat(2);
// 'abcabc' (repeat() is a generic method)
```

See also

- Polyfill of `String.prototype.repeat` in `core-js`
- es-shims polyfill of `String.prototype.repeat`
- `String.prototype.concat()`

String.prototype.replace()

The **`replace()`** method of `String` values returns a new string with one, some, or all matches of a `pattern` replaced by a `replacement`. The `pattern` can be a string or a `RegExp`, and the `replacement` can be a string or a function called for each match. If `pattern` is a string, only the first occurrence will be replaced. The original string is left unchanged.

Try it

```
const paragraph = "This dog's name is just Dog! Yes, that is the name.";

console.log(paragraph.replace("name", "nickname"));
// Expected output: "This dog's nickname is just Dog! Yes, that is the name."

console.log(paragraph.replace(/\bis\b/, "was"));
// Expected output: "This dog's name was just Dog! Yes, that is the name."

console.log(paragraph.replace(/\bis\b/g, "was"));
// Expected output: "This dog's name was just Dog! Yes, that was the name."
```

Syntax

```
replace(pattern, replacement)
```

Parameters

`pattern`

Can be a string or an object with a `Symbol.replace` method — the typical example being a regular expression. Any value that doesn't have the `Symbol.replace` method will be coerced to a string.

`replacement`

Can be a string or a function.

- If it's a string, it will replace the substring matched by `pattern`. A number of special replacement patterns are supported; see the Specifying a string as the replacement section below.
- If it's a function, it will be invoked for every match and its return value is used as the replacement text. The arguments supplied to this function are described in the Specifying a function as the replacement section below.

Return value

A new string, with one, some, or all matches of the pattern replaced by the specified replacement.

Description

This method does not mutate the string value it's called on. It returns a new string.

A string pattern will only be replaced once. To perform a global search and replace, use a regular expression with the `g` flag, or use `replaceAll()` instead.

If `pattern` is an object with a `Symbol.replace` method (including `RegExp` objects), that method is called with the target string and `replacement` as arguments. Its return value becomes the return value of `replace()`. In this case the behavior of `replace()` is entirely encoded by the `[Symbol.replace]()` method — for example, any mention of "capturing groups" in the description below is actually functionality provided by `RegExp.prototype[Symbol.replace]()`.

If the `pattern` is an empty string, the replacement is prepended to the start of the string.

```
"xxx".replace("", "_"); // "_xxx"
```

A regexp with the `g` flag is the only case where `replace()` replaces more than once. For more information about how regex properties (especially the sticky flag) interact with `replace()`, see `RegExp.prototype[Symbol.replace]()`.

Specifying a string as the replacement

The replacement string can include the following special replacement patterns:

Pattern	Inserts
`$$`	Inserts a "$".
`$&`	Inserts the matched substring.
`$``	Inserts the portion of the string that precedes the matched substring.
`$'`	Inserts the portion of the string that follows the matched substring.
`$n`	Inserts the nth (1-indexed) capturing group where `n` is a positive integer less than 100.
`$<Name>`	Inserts the named capturing group where `Name` is the group name.

`$n` and `$<Name>` are only available if the `pattern` argument is a `RegExp` object. If the `pattern` is a string, or if the corresponding capturing group isn't present in the regex, then the pattern will be replaced as a literal. If the group is present but isn't matched (because it's part of a disjunction), it will be replaced with an empty string.

```
"foo".replace(/(f)/, "$2");
// "$2oo"; the regex doesn't have the second group

"foo".replace("f", "$1");
// "$1oo"; the pattern is a string, so it doesn't have any groups

"foo".replace(/(f)|(g)/, "$2");
// "oo"; the second group exists but isn't matched
```

Specifying a function as the replacement

You can specify a function as the second parameter. In this case, the function will be invoked after the match has been performed. The function's result (return value) will be used as the replacement string.

> **Note:** The above-mentioned special replacement patterns do *not* apply for strings returned from the replacer function.

The function has the following signature:

```
function replacer(match, p1, p2, /* …, */ pN, offset, string, groups) {
  return replacement;
}
```

The arguments to the function are as follows:

`match`

The matched substring. (Corresponds to `$&` above.)

`p1, p2, ..., pN`

The nth string found by a capture group (including named capturing groups), provided the first argument to `replace()` is a `RegExp` object. (Corresponds to `$1`, `$2`, etc. above.) For example, if the `pattern` is `/(\a+)(\b+)/`, then `p1` is the match for `\a+`, and `p2` is the match for `\b+`. If the group is part of a disjunction (e.g., `"abc".replace(/(a)|(b)/, replacer)`), the unmatched alternative will be `undefined`.

`offset`

The offset of the matched substring within the whole string being examined. For example, if the whole string was `'abcd'`, and the matched substring was `'bc'`, then this argument will be `1`.

`string`

The whole string being examined.

`groups`

An object whose keys are the used group names, and whose values are the matched portions (`undefined` if not matched). Only present if the `pattern` contains at least one named capturing group.

The exact number of arguments depends on whether the first argument is a `RegExp` object — and, if so, how many capture groups it has.

The following example will set `newString` to `'abc - 12345 - #$*%'`:

```
function replacer(match, p1, p2, p3, offset, string) {
  // p1 is non-digits, p2 digits, and p3 non-alphanumerics
  return [p1, p2, p3].join(" - ");
}
const newString = "abc12345#$*%".replace(/(\D*)(\d*)(\W*)/, replacer);
console.log(newString); // abc - 12345 - #$*%
```

The function will be invoked multiple times for each full match to be replaced if the regular expression in the first parameter is global.

Examples

Defining the regular expression in replace()

In the following example, the regular expression is defined in `replace()` and includes the ignore case flag.

```
const str = "Twas the night before Xmas...";
const newStr = str.replace(/xmas/i, "Christmas");
console.log(newStr); // Twas the night before Christmas...
```

This logs `'Twas the night before Christmas...'`.

> **Note:** See the regular expression guide for more explanations about regular expressions.

Using the global and ignoreCase flags with replace()

Global replace can only be done with a regular expression. In the following example, the regular expression includes the global and ignore case flags which permits `replace()` to replace each occurrence of `'apples'` in the string with `'oranges'`.

```
const re = /apples/gi;
const str = "Apples are round, and apples are juicy.";
const newStr = str.replace(re, "oranges");
console.log(newStr); // oranges are round, and oranges are juicy.
```

This logs `'oranges are round, and oranges are juicy'`.

Switching words in a string

The following script switches the words in the string. For the replacement text, the script uses capturing groups and the `$1` and `$2` replacement patterns.

```
const re = /(\w+)\s(\w+)/;
const str = "Maria Cruz";
const newStr = str.replace(re, "$2, $1");
console.log(newStr); // Cruz, Maria
```

This logs `'Cruz, Maria'`.

Using an inline function that modifies the matched characters

In this example, all occurrences of capital letters in the string are converted to lower case, and a hyphen is inserted just before the match location. The important thing here is that additional operations are needed on the matched item before it is given back as a replacement.

The replacement function accepts the matched snippet as its parameter, and uses it to transform the case and concatenate the hyphen before returning.

```
function styleHyphenFormat(propertyName) {
  function upperToHyphenLower(match, offset, string) {
    return (offset > 0 ? "-" : "") + match.toLowerCase();
  }
  return propertyName.replace(/[A-Z]/g, upperToHyphenLower);
}
```

Given `styleHyphenFormat('borderTop')`, this returns `'border-top'`.

Because we want to further transform the *result* of the match before the final substitution is made, we must use a function. This forces the evaluation of the match prior to the `toLowerCase()` method. If we had tried to do this using the match without a function, the `toLowerCase()` would have no effect.

```
// Won't work
const newString = propertyName.replace(/[A-Z]/g, "-" + "$&".toLowerCase());
```

This is because `'$&'.toLowerCase()` would first be evaluated as a string literal (resulting in the same `'$&'`) before using the characters as a pattern.

Replacing a Fahrenheit degree with its Celsius equivalent

The following example replaces a Fahrenheit degree with its equivalent Celsius degree. The Fahrenheit degree should be a number ending with `"F"`. The function returns the Celsius number ending with `"C"`. For example, if the input number is `"212F"`, the function returns `"100C"`. If the number is `"0F"`, the function returns `"-17.77777777777778C"`.

The regular expression `test` checks for any number that ends with `F`. The number of Fahrenheit degrees is accessible to the function through its second parameter, `p1`. The function sets the Celsius number based on the number of Fahrenheit degrees passed in a string to the `f2c()` function. `f2c()` then returns the Celsius number. This function approximates Perl's `s///e` flag.

```
function f2c(x) {
  function convert(str, p1, offset, s) {
    return `${((p1 - 32) * 5) / 9}C`;
  }
  const s = String(x);
  const test = /(-?\d+(?:\.\d*)?)F\b/g;
  return s.replace(test, convert);
}
```

Making a generic replacer

Suppose we want to create a replacer that appends the offset data to every matched string. Because the replacer function already receives the `offset` parameter, it will be trivial if the regex is statically known.

```
"abcd".replace(/(bc)/, (match, p1, offset) => `${match} (${offset}) `);
// "abc (1) d"
```

However, this replacer would be hard to generalize if we want it to work with any regex pattern. The replacer is *variadic* — the number of arguments it receives depends on the number of capturing groups present. We can use rest parameters, but it would also collect `offset`, `string`, etc. into the array. The fact that `groups` may or may not be passed depending on the identity of the regex would also make it hard to generically know which argument corresponds to the `offset`.

```
function addOffset(match, ...args) {
  const offset = args.at(-2);
  return `${match} (${offset}) `;
}

console.log("abcd".replace(/(bc)/, addOffset)); // "abc (1) d"
console.log("abcd".replace(/(?<group>bc)/, addOffset)); // "abc (abcd) d"
```

The `addOffset` example above doesn't work when the regex contains a named group, because in this case `args.at(-2)` would be the `string` instead of the `offset`.

Instead, you need to extract the last few arguments based on type, because `groups` is an object while `string` is a string.

```
function addOffset(match, ...args) {
  const hasNamedGroups = typeof args.at(-1) === "object";
  const offset = hasNamedGroups ? args.at(-3) : args.at(-2);
  return `${match} (${offset}) `;
}

console.log("abcd".replace(/(bc)/, addOffset)); // "abc (1) d"
console.log("abcd".replace(/(?<group>bc)/, addOffset)); // "abc (1) d"
```

See also

- Polyfill of `String.prototype.replace` in `core-js` with fixes and implementation of modern behavior like `Symbol.replace` support
- Regular expressions guide
- `String.prototype.replaceAll()`
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`
- `Symbol.replace`
- `RegExp.prototype[Symbol.replace]()`

String.prototype.replaceAll()

The **`replaceAll()`** method of `String` values returns a new string with all matches of a `pattern` replaced by a `replacement`. The `pattern` can be a string or a `RegExp`, and the `replacement` can be a string or a function to be called for each match. The original string is left unchanged.

Try it

```
const paragraph = "This dog's name is just Dog! Yes, that is the name.";

console.log(paragraph.replaceAll("name", "nickname"));
// Expected output: "This dog's nickname is just Dog! Yes, that is the nickname."
```

```
console.log(paragraph.replaceAll(/\bis\b/g, "was"));
// Expected output: "This dog's name was just Dog! Yes, that was the name."
```

Syntax

```
replaceAll(pattern, replacement)
```

Parameters

`pattern`

> Can be a string or an object with a `Symbol.replace` method — the typical example being a regular expression. Any value that doesn't have the `Symbol.replace` method will be coerced to a string.
>
> If `pattern` is a regex, then it must have the global (`g`) flag set, or a `TypeError` is thrown.

`replacement`

> Can be a string or a function. The replacement has the same semantics as that of `String.prototype.replace()`.

Return value

A new string, with all matches of a pattern replaced by a replacement.

Exceptions

`TypeError`

> Thrown if the `pattern` is a regex that does not have the global (`g`) flag set (its `flags` property does not contain `"g"`).

Description

This method does not mutate the string value it's called on. It returns a new string.

Unlike `replace()`, this method replaces all occurrences of a string, not just the first one. While it is also possible to use `replace()` with a global regex dynamically constructed with `RegExp()` to replace all instances of a string, this can have unintended consequences if the string contains special characters that have meaning in regular expressions (which might happen if the replacement string comes from user input). While you can mitigate this case using `RegExp.escape()` to make the regular expression string into a literal pattern, it is simpler to pass the string to `replaceAll()` directly, without converting it to a regex.

```
function unsafeRedactName(text, name) {
  return text.replace(new RegExp(name, "g"), "[REDACTED]");
}
function semiSafeRedactName(text, name) {
  return text.replaceAll(name, "[REDACTED]");
}
function superSafeRedactName(text, name) {
  // only match at word boundaries
  return text.replaceAll(
    new RegExp(`\\b${RegExp.escape(name)}\\b`, "g"),
    "[REDACTED]",
  );
}

let report =
  "A hacker called ha.*er used special characters in their name to breach the system.";

console.log(unsafeRedactName(report, "ha.*er")); // "A [REDACTED]s in their name to breach the system."
console.log(semiSafeRedactName(report, "ha.*er")); // "A hacker called [REDACTED] used special characters in their name to breach the system."

report = "A hacker called acke breached the system.";

console.log(semiSafeRedactName(report, "acke")); // "A h[REDACTED]r called [REDACTED] breached the system."
console.log(superSafeRedactName(report, "acke")); // "A hacker called [REDACTED] breached the system."
```

If `pattern` is an object with a `Symbol.replace` method (including `RegExp` objects), that method is called with the target string and `replacement` as arguments. Its return value becomes the return value of `replaceAll()`. In this case the behavior of `replaceAll()` is entirely encoded by the `[Symbol.replace]()` method, and therefore will have the same result as `replace()` (apart from the extra input validation that the regex is global).

If the `pattern` is an empty string, the replacement will be inserted in between every UTF-16 code unit, similar to `split()` behavior.

```
"xxx".replaceAll("", "_"); // "_x_x_x_"
```

For more information about how regex properties (especially the sticky flag) interact with `replaceAll()`, see `RegExp.prototype[Symbol.replace]()`.

Examples

Using replaceAll()

```
"aabbcc".replaceAll("b", ".");
// 'aa..cc'
```

Non-global regex throws

When using a regular expression search value, it must be global. This won't work:

```
"aabbcc".replaceAll(/b/, ".");
// TypeError: replaceAll must be called with a global RegExp
```

This will work:

```
"aabbcc".replaceAll(/b/g, ".");
("aa..cc");
```

See also

- Polyfill of `String.prototype.replaceAll` in `core-js`
- es-shims polyfill of `String.prototype.replaceAll`
- Regular expressions guide
- `String.prototype.replace()`
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

String.prototype.search()

The **`search()`** method of `String` values executes a search for a match between a regular expression and this string, returning the index of the first match in the string.

Try it

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

// Anything not a word character, whitespace or apostrophe
const regex = /[^\w\s']/g;

console.log(paragraph.search(regex));
// Expected output: 41

console.log(paragraph[paragraph.search(regex)]);
// Expected output: "!"
```

Syntax

```
search(regexp)
```

Parameters

`regexp`

A regular expression object, or any object that has a `Symbol.search` method.

If `regexp` is not a `RegExp` object and does not have a `Symbol.search` method, it is implicitly converted to a `RegExp` by using `new RegExp(regexp)`.

Return value

The index of the first match between the regular expression and the given string, or `-1` if no match was found.

Description

The implementation of `String.prototype.search()` doesn't do much other than calling the `Symbol.search` method of the argument with the string as the first parameter. The actual implementation comes from `RegExp.prototype[Symbol.search]()`.

The `g` flag of `regexp` has no effect on the `search()` result, and the search always happens as if the regex's `lastIndex` is 0. For more information on the behavior of `search()`, see `RegExp.prototype[Symbol.search]()`.

When you want to know whether a pattern is found, and *also* know its index within a string, use `search()`.

- If you only want to know if it exists, use the `RegExp.prototype.test()` method, which returns a boolean.
- If you need the content of the matched text, use `String.prototype.match()` or `RegExp.prototype.exec()`.

Examples

Using search()

The following example searches a string with two different regex objects to show a successful search (positive value) vs. an unsuccessful search (`-1`).

```
const str = "hey JudE";
const re = /[A-Z]/;
const reDot = /[.]/;
console.log(str.search(re)); // returns 4, which is the index of the first capital letter "J"
console.log(str.search(reDot)); // returns -1 cannot find '.' dot punctuation
```

See also

- Polyfill of `String.prototype.search` in `core-js` with fixes and implementation of modern behavior like `Symbol.search` support

- Regular expressions guide
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype[Symbol.search]()`

String.prototype.slice()

The **`slice()`** method of `String` values extracts a section of this string and returns it as a new string, without modifying the original string.

Try it

```
const str = "The quick brown fox jumps over the lazy dog.";

console.log(str.slice(31));
// Expected output: "the lazy dog."

console.log(str.slice(4, 19));
// Expected output: "quick brown fox"

console.log(str.slice(-4));
// Expected output: "dog."

console.log(str.slice(-9, -5));
// Expected output: "lazy"
```

Syntax

```
slice(indexStart)
slice(indexStart, indexEnd)
```

Parameters

`indexStart`

The index of the first character to include in the returned substring.

`indexEnd` Optional

The index of the first character to exclude from the returned substring.

Return value

A new string containing the extracted section of the string.

Description

`slice()` extracts the text from one string and returns a new string.

`slice()` extracts up to but not including `indexEnd`. For example, `str.slice(4, 8)` extracts the fifth character through the eighth character (characters indexed `4`, `5`, `6`, and `7`):

```
             indexStart        indexEnd
                 ↓               ↓
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| T | h | e |   | m | i | r | r | o | r |

                m   i   r   r
                ‾‾‾‾‾‾‾‾‾‾‾
                    ↑
                  Result
```

- If `indexStart >= str.length`, an empty string is returned.
- If `indexStart < 0`, the index is counted from the end of the string. More formally, in this case, the substring starts at `max(indexStart + str.length, 0)`.
- If `indexStart` is omitted, `undefined`, or cannot be converted to a number, it's treated as `0`.
- If `indexEnd` is omitted or `undefined`, or if `indexEnd >= str.length`, `slice()` extracts to the end of the string.
- If `indexEnd < 0`, the index is counted from the end of the string. More formally, in this case, the substring ends at `max(indexEnd + str.length, 0)`.
- If `indexEnd <= indexStart` after normalizing negative values (i.e., `indexEnd` represents a character that's before `indexStart`), an empty string is returned.

Examples

Using slice() to create a new string

The following example uses `slice()` to create a new string.

```
const str1 = "The morning is upon us."; // The length of str1 is 23.
const str2 = str1.slice(1, 8);
const str3 = str1.slice(4, -2);
const str4 = str1.slice(12);
const str5 = str1.slice(30);
console.log(str2); // he morn
console.log(str3); // morning is upon u
```

```
console.log(str4); // is upon us.
console.log(str5); // ""
```

Using slice() with negative indexes

The following example uses `slice()` with negative indexes.

```
const str = "The morning is upon us.";
str.slice(-3); // 'us.'
str.slice(-3, -1); // 'us'
str.slice(0, -1); // 'The morning is upon us'
str.slice(4, -1); // 'morning is upon us'
```

This example counts backwards from the end of the string by `11` to find the start index and forwards from the start of the string by `16` to find the end index.

```
console.log(str.slice(-11, 16)); // "is u"
```

Here it counts forwards from the start by `11` to find the start index and backwards from the end by `7` to find the end index.

```
console.log(str.slice(11, -7)); // " is u"
```

These arguments count backwards from the end by `5` to find the start index and backwards from the end by `1` to find the end index.

```
console.log(str.slice(-5, -1)); // "n us"
```

See also

- `String.prototype.substr()`
- `String.prototype.substring()`
- `Array.prototype.slice()`

String.prototype.small()

The **`small()`** method of `String` values creates a string that embeds this string in a `<small>` element (`<small>str</small>`), which causes this string to be displayed in a small font.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
small()
```

Parameters

None.

Return value

A string beginning with a `<small>` start tag, then the text `str`, and then a `</small>` end tag.

Examples

Using small()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.small();
```

This will create the following HTML:

```
<small>Hello, world</small>
```

Instead of using `small()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("small");
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.small` in `core-js`
- es-shims polyfill of `String.prototype.small`
- HTML wrapper methods
- `<small>`

String.prototype.split()

The **`split()`** method of `String` values takes a pattern and divides this string into an ordered list of substrings by searching for the pattern, puts these substrings into an array, and returns the array.

Try it

```
const str = "The quick brown fox jumps over the lazy dog.";

const words = str.split(" ");
console.log(words[3]);
// Expected output: "fox"

const chars = str.split("");
console.log(chars[8]);
// Expected output: "k"

const strCopy = str.split();
console.log(strCopy);
// Expected output: Array ["The quick brown fox jumps over the lazy dog."]
```

Syntax

```
split(separator)
split(separator, limit)
```

Parameters

`separator`

> The pattern describing where each split should occur. Can be `undefined`, a string, or an object with a `Symbol.split` method — the typical example being a regular expression. Omitting `separator` or passing `undefined` causes `split()` to return an array with the calling string as a single element. All values that are not `undefined` or objects with a `[Symbol.split]()` method are coerced to strings.

`limit` Optional

> A non-negative integer specifying a limit on the number of substrings to be included in the array. If provided, splits the string at each occurrence of the specified `separator`, but stops when `limit` entries have been placed in the array. Any leftover text is not included in the array at all.
>
> - The array may contain fewer entries than `limit` if the end of the string is reached before the limit is reached.
> - If `limit` is `0`, `[]` is returned.

Return value

If `separator` is a string, an `Array` of strings is returned, split at each point where the `separator` occurs in the given string.

If `separator` is a regex, the returned `Array` also contains the captured groups for each separator match; see below for details. The capturing groups may be unmatched, in which case they are `undefined` in the array.

If `separator` has a custom `[Symbol.split]()` method, its return value is directly returned.

Description

If `separator` is a non-empty string, the target string is split by all matches of the `separator` without including `separator` in the results. For example, a string containing tab separated values (TSV) could be parsed by passing a tab character as the separator, like `myString.split("\t")`. If `separator` contains multiple characters, that entire character sequence must be found in order to split. If `separator` appears at the beginning (or end) of the string, it still has the effect of splitting, resulting in an empty (i.e., zero length) string appearing at the first (or last) position of the returned array. If `separator` does not occur in `str`, the returned array contains one element consisting of the entire string.

If `separator` is an empty string (`""`), `str` is converted to an array of each of its UTF-16 "characters", without empty strings on either ends of the resulting string.

> **Note:** `"".split("")` is therefore the only way to produce an empty array when a string is passed as `separator` and `limit` is not `0`.

> **Warning:** When the empty string (`""`) is used as a separator, the string is **not** split by *user-perceived characters* (grapheme clusters) or unicode characters (code points), but by UTF-16 code units. This destroys surrogate pairs. See "How do you get a string to a character array in JavaScript?" on Stack Overflow.

If `separator` is a regexp that matches empty strings, whether the match is split by UTF-16 code units or Unicode code points depends on if the regex is Unicode-aware.

```
"😄😄".split(/(?:)/); // [ "\ud83d", "\ude04", "\ud83d", "\ude04" ]
"😄😄".split(/(?:)/u); // [ "😄", "😄" ]
```

If `separator` is a regular expression with capturing groups, then each time `separator` matches, the captured groups (including any `undefined` results) are spliced into the output array. This behavior is specified by the regexp's `Symbol.split` method.

If `separator` is an object with a `Symbol.split` method, that method is called with the target string and `limit` as arguments, and `this` set to the object. Its return value becomes the return value of `split`.

Any other value will be coerced to a string before being used as separator.

Examples

Using split()

When the string is empty and a non-empty separator is specified, `split()` returns `[""]`. If the string and separator are both empty strings, an empty array is returned.

```
const emptyString = "";

// string is empty and separator is non-empty
console.log(emptyString.split("a"));
// [""]

// string and separator are both empty strings
console.log(emptyString.split(emptyString));
// []
```

The following example defines a function that splits a string into an array of strings using `separator`. After splitting the string, the function logs messages indicating the original string (before the split), the separator used, the number of elements in the array, and the individual array elements.

```
function splitString(stringToSplit, separator) {
  const arrayOfStrings = stringToSplit.split(separator);

  console.log("The original string is:", stringToSplit);
  console.log("The separator is:", separator);
  console.log(
    "The array has",
    arrayOfStrings.length,
    "elements:",
    arrayOfStrings.join(" / "),
  );
}

const tempestString = "Oh brave new world that has such people in it.";
const monthString = "Jan,Feb,Mar,Apr,May,Jun,Jul,Aug,Sep,Oct,Nov,Dec";

const space = " ";
const comma = ",";

splitString(tempestString, space);
splitString(tempestString);
splitString(monthString, comma);
```

This example produces the following output:

```
The original string is: "Oh brave new world that has such people in it."
The separator is: " "
The array has 10 elements: Oh / brave / new / world / that / has / such / people / in / it.

The original string is: "Oh brave new world that has such people in it."
The separator is: "undefined"
The array has 1 elements: Oh brave new world that has such people in it.

The original string is: "Jan,Feb,Mar,Apr,May,Jun,Jul,Aug,Sep,Oct,Nov,Dec"
The separator is: ","
The array has 12 elements: Jan / Feb / Mar / Apr / May / Jun / Jul / Aug / Sep / Oct / Nov / Dec
```

Removing spaces from a string

In the following example, `split()` looks for zero or more spaces, followed by a semicolon, followed by zero or more spaces—and, when found, removes the spaces and the semicolon from the string. `nameList` is the array returned as a result of `split()`.

```
const names = "Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ;Chris Hand ";

console.log(names);

const re = /\s*(?:;|$)\s*/;
const nameList = names.split(re);

console.log(nameList);
```

This logs two lines; the first line logs the original string, and the second line logs the resulting array.

```
Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ;Chris Hand
[ "Harry Trump", "Fred Barney", "Helen Rigby", "Bill Abel", "Chris Hand", "" ]
```

Returning a limited number of splits

In the following example, `split()` looks for spaces in a string and returns the first 3 splits that it finds.

```
const myString = "Hello World. How are you doing?";
const splits = myString.split(" ", 3);
```

```
console.log(splits); // [ "Hello", "World.", "How" ]
```

Splitting with a <code>RegExp</code> to include parts of the separator in the result

If `separator` is a regular expression that contains capturing parentheses `( )`, matched results are included in the array.

```
const myString = "Hello 1 word. Sentence number 2.";
const splits = myString.split(/(\d)/);

console.log(splits);
// [ "Hello ", "1", " word. Sentence number ", "2", "." ]
```

> **Note:** `\d` matches the character class for digits between 0 and 9.

Using a custom splitter

An object with a `Symbol.split` method can be used as a splitter with custom behavior.

The following example splits a string using an internal state consisting of an incrementing number:

```
const splitByNumber = {
  [Symbol.split](str) {
    let num = 1;
    let pos = 0;
    const result = [];
    while (pos < str.length) {
      const matchPos = str.indexOf(num, pos);
      if (matchPos === -1) {
        result.push(str.substring(pos));
        break;
      }
      result.push(str.substring(pos, matchPos));
      pos = matchPos + String(num).length;
      num++;
    }
    return result;
  },
};

const myString = "a1bc2c5d3e4f";
console.log(myString.split(splitByNumber)); // [ "a", "bc", "c5d", "e", "f" ]
```

The following example uses an internal state to enforce certain behavior, and to ensure a "valid" result is produced.

```
const DELIMITER = ";";

// Split the commands, but remove any invalid or unnecessary values.
const splitCommands = {
  [Symbol.split](str, lim) {
    const results = [];
    const state = {
      on: false,
      brightness: {
        current: 2,
        min: 1,
        max: 3,
      },
    };
    let pos = 0;
    let matchPos = str.indexOf(DELIMITER, pos);

    while (matchPos !== -1) {
      const subString = str.slice(pos, matchPos).trim();

      switch (subString) {
        case "light on":
          // If the `on` state is already true, do nothing.
          if (!state.on) {
            state.on = true;
            results.push(subString);
          }
          break;

        case "light off":
          // If the `on` state is already false, do nothing.
          if (state.on) {
            state.on = false;
            results.push(subString);
          }
          break;

        case "brightness up":
```

```
          // Enforce a brightness maximum.
          if (state.brightness.current < state.brightness.max) {
            state.brightness.current += 1;
            results.push(subString);
          }
          break;

        case "brightness down":
          // Enforce a brightness minimum.
          if (state.brightness.current > state.brightness.min) {
            state.brightness.current -= 1;
            results.push(subString);
          }
          break;
      }

      if (results.length === lim) {
        break;
      }

      pos = matchPos + DELIMITER.length;
      matchPos = str.indexOf(DELIMITER, pos);
    }

    // If we broke early due to reaching the split `lim`, don't add the remaining commands.
    if (results.length < lim) {
      results.push(str.slice(pos).trim());
    }

    return results;
  },
};

const commands =
  "light on; brightness up; brightness up; brightness up; light on; brightness down; brightness down; light off";
console.log(commands.split(splitCommands, 3)); // ["light on", "brightness up", "brightness down"]
```

See also

- Polyfill of `String.prototype.split` in `core-js` with fixes and implementation of modern behavior like `Symbol.split` support
- es-shims polyfill of `String.prototype.split`
- Regular expressions guide
- `String.prototype.charAt()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `Array.prototype.join()`

String.prototype.startsWith()

The **`startsWith()`** method of `String` values determines whether this string begins with the characters of a specified string, returning `true` or `false` as appropriate.

Try it

```
const str = "Saturday night plans";

console.log(str.startsWith("Sat"));
// Expected output: true

console.log(str.startsWith("Sat", 3));
// Expected output: false
```

Syntax

```
startsWith(searchString)
startsWith(searchString, position)
```

Parameters

`searchString`

> The characters to be searched for at the start of this string. Cannot be a regex. All values that are not regexes are coerced to strings, so omitting it or passing `undefined` causes `startsWith()` to search for the string `"undefined"`, which is rarely what you want.

`position` Optional

> The start position at which `searchString` is expected to be found (the index of `searchString`'s first character). Defaults to `0`.

Return value

`true` if the given characters are found at the beginning of the string, including when `searchString` is an empty string; otherwise, **`false`**.

Exceptions

`TypeError`

Thrown if `searchString` is a regex.

Description

This method lets you determine whether or not a string begins with another string. This method is case-sensitive.

Examples

Using startsWith()

```
const str = "To be, or not to be, that is the question.";

console.log(str.startsWith("To be")); // true
console.log(str.startsWith("not to be")); // false
console.log(str.startsWith("not to be", 10)); // true
```

See also

- Polyfill of `String.prototype.startsWith` in `core-js`
- es-shims polyfill of `String.prototype.startsWith`
- `String.prototype.endsWith()`
- `String.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`

String.prototype.strike()

The **`strike()`** method of `String` values creates a string that embeds this string in a `<strike>` element (`<strike>str</strike>`), which causes this string to be displayed as struck-out text.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. For the case of `strike()`, the `<strike>` element itself has been removed from the HTML specification and shouldn't be used anymore. Web developers should use the `<del>` for deleted content or the `<s>` for content that is no longer accurate or no longer relevant instead.

Syntax

```
strike()
```

Parameters

None.

Return value

A string beginning with a `<strike>` start tag, then the text `str`, and then a `</strike>` end tag.

Examples

Using strike()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.strike();
```

This will create the following HTML:

```
<strike>Hello, world</strike>
```

> **Warning:** This markup is invalid, because `strike` is no longer a valid element.

Instead of using `strike()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("s");
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.strike` in `core-js`
- es-shims polyfill of `String.prototype.strike`
- HTML wrapper methods
- `<strike>`

String() constructor

The **`String()`** constructor creates `String` objects. When called as a function, it returns primitive values of type String.

Syntax

```
new String(thing)
String(thing)
```

> **Note:** `String()` can be called with or without `new`, but with different effects. See Return value.

Parameters

`thing`

Anything to be converted to a string.

Return value

When `String()` is called as a function (without `new`), it returns `value` coerced to a string primitive. Specially, Symbol values are converted to `"Symbol(description)"`, where `description` is the description of the Symbol, instead of throwing.

When `String()` is called as a constructor (with `new`), it coerces `value` to a string primitive (without special symbol handling) and returns a wrapping `String` object, which is **not** a primitive.

> **Warning:** You should rarely find yourself using `String` as a constructor.

Examples

String constructor and String function

String function and String constructor produce different results:

```
const a = new String("Hello world"); // a === "Hello world" is false
const b = String("Hello world"); // b === "Hello world" is true
a instanceof String; // is true
b instanceof String; // is false
typeof a; // "object"
typeof b; // "string"
```

Here, the function produces a string (the primitive type) as promised. However, the constructor produces an instance of the type String (an object wrapper) and that's why you rarely want to use the String constructor at all.

Using String() to stringify a symbol

`String()` is the only case where a symbol can be converted to a string without throwing, because it's very explicit.

```
const sym = Symbol("example");
`${sym}`; // TypeError: Cannot convert a Symbol value to a string
"" + sym; // TypeError: Cannot convert a Symbol value to a string
"".concat(sym); // TypeError: Cannot convert a Symbol value to a string
```

```
const sym = Symbol("example");
String(sym); // "Symbol(example)"
```

See also

- Numbers and strings guide

String.prototype.sub()

The **`sub()`** method of `String` values creates a string that embeds this string in a `<sub>` element (`<sub>str</sub>`), which causes this string to be displayed as subscript.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
sub()
```

Parameters

None.

Return value

A string beginning with a `<sub>` start tag, then the text `str`, and then a `</sub>` end tag.

Examples

Using sub()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.sub();
```

This will create the following HTML:

```
<sub>Hello, world</sub>
```

Instead of using `sub()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("sub");
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.sub` in `core-js`
- es-shims polyfill of `String.prototype.sub`
- HTML wrapper methods
- `<sub>`

String.prototype.substr()

The **`substr()`** method of `String` values returns a portion of this string, starting at the specified index and extending for a given number of characters afterwards.

> **Note:** `substr()` is not part of the main ECMAScript specification — it's defined in Annex B: Additional ECMAScript Features for Web Browsers, which is normative optional for non-browser runtimes. Therefore, people are advised to use the standard `String.prototype.substring()` and `String.prototype.slice()` methods instead to make their code maximally cross-platform friendly. The `String.prototype.substring()` page has some comparisons between the three methods.

Try it

```
const str = "Mozilla";

console.log(str.substr(1, 2));
// Expected output: "oz"

console.log(str.substr(2));
// Expected output: "zilla"
```

Syntax

```
substr(start)
substr(start, length)
```

Parameters

`start`

The index of the first character to include in the returned substring.

`length` Optional

The number of characters to extract.

Return value

A new string containing the specified part of the given string.

Description

A string's `substr()` method extracts `length` characters from the string, counting from the `start` index.

- If `start >= str.length`, an empty string is returned.
- If `start < 0`, the index starts counting from the end of the string. More formally, in this case the substring starts at `max(start + str.length, 0)`.
- If `start` is omitted or `undefined`, it's treated as `0`.
- If `length` is omitted or `undefined`, or if `start + length >= str.length`, `substr()` extracts characters to the end of the string.
- If `length < 0`, an empty string is returned.
- For both `start` and `length`, `NaN` is treated as `0`.

Although you are encouraged to avoid using `substr()`, there is no trivial way to migrate `substr()` to either `slice()` or `substring()` in legacy code without essentially writing a polyfill for `substr()`. For example, `str.substr(a, l)`, `str.slice(a, a + l)`, and `str.substring(a, a + l)` all have different results when `str = "01234", a = 1, l = -2` — `substr()` returns an empty string, `slice()` returns `"123"`, while `substring()` returns `"0"`. The actual refactoring path depends on the knowledge of the range of `a` and `l`.

Examples

Using substr()

```
const string = "Mozilla";

console.log(string.substr(0, 1)); // 'M'
console.log(string.substr(1, 0)); // ''
console.log(string.substr(-1, 1)); // 'a'
console.log(string.substr(1, -1)); // ''
console.log(string.substr(-3)); // 'lla'
console.log(string.substr(1)); // 'ozilla'
console.log(string.substr(-20, 2)); // 'Mo'
console.log(string.substr(20, 2)); // ''
```

See also

- Polyfill of `String.prototype.substr` in `core-js`
- es-shims polyfill of `String.prototype.substr`
- `String.prototype.slice()`
- `String.prototype.substring()`

String.prototype.substring()

The **`substring()`** method of `String` values returns the part of this string from the start index up to and excluding the end index, or to the end of the string if no end index is supplied.

Try it

```
const str = "Mozilla";

console.log(str.substring(1, 3));
// Expected output: "oz"

console.log(str.substring(2));
// Expected output: "zilla"
```

Syntax

```
substring(indexStart)
substring(indexStart, indexEnd)
```

Parameters

`indexStart`

The index of the first character to include in the returned substring.

`indexEnd` Optional

The index of the first character to exclude from the returned substring.

Return value

A new string containing the specified part of the given string.

Description

`substring()` extracts characters from `indexStart` up to *but not including* `indexEnd`. In particular:

- If `indexEnd` is omitted or `undefined`, `substring()` extracts characters to the end of the string.
- If `indexStart` is equal to `indexEnd`, `substring()` returns an empty string.
- If `indexStart` is greater than `indexEnd`, then the effect of `substring()` is as if the two arguments were swapped; see example below.

Any argument value that is less than `0` or greater than `str.length` is treated as if it were `0` and `str.length`, respectively.

Any argument value that is `NaN` is treated as if it were `0`.

Examples

Using substring()

The following example uses `substring()` to display characters from the string `"Mozilla"`:

```
const anyString = "Mozilla";

console.log(anyString.substring(0, 1)); // "M"
console.log(anyString.substring(1, 0)); // "M"

console.log(anyString.substring(0, 6)); // "Mozill"

console.log(anyString.substring(4)); // "lla"
console.log(anyString.substring(4, 7)); // "lla"
```

```
console.log(anyString.substring(7, 4)); // "lla"

console.log(anyString.substring(0, 7)); // "Mozilla"
console.log(anyString.substring(0, 10)); // "Mozilla"
```

Using substring() with length property

The following example uses the `substring()` method and `length` property to extract the last characters of a particular string. This method may be easier to remember, given that you don't need to know the starting and ending indices as you would in the above examples.

```
const text = "Mozilla";

// Takes 4 last characters of string
console.log(text.substring(text.length - 4)); // prints "illa"

// Takes 5 last characters of string
console.log(text.substring(text.length - 5)); // prints "zilla"
```

The difference between substring() and substr()

There are subtle differences between the `substring()` and `substr()` methods, so you should be careful not to get them confused.

- The two parameters of `substr()` are `start` and `length`, while for `substring()`, they are `start` and `end`.
- `substr()`'s `start` index will wrap to the end of the string if it is negative, while `substring()` will clamp it to `0`.
- Negative lengths in `substr()` are treated as zero, while `substring()` will swap the two indexes if `end` is less than `start`.

Furthermore, `substr()` is considered a *legacy feature in ECMAScript*, so it is best to avoid using it if possible.

```
const text = "Mozilla";
console.log(text.substring(2, 5)); // "zil"
console.log(text.substr(2, 3)); // "zil"
```

Differences between substring() and slice()

The `substring()` and `slice()` methods are almost identical, but there are a couple of subtle differences between the two, especially in the way negative arguments are dealt with.

The `substring()` method swaps its two arguments if `indexStart` is greater than `indexEnd`, meaning that a string is still returned. The `slice()` method returns an empty string if this is the case.

```
const text = "Mozilla";
console.log(text.substring(5, 2)); // "zil"
console.log(text.slice(5, 2)); // ""
```

If either or both of the arguments are negative or `NaN`, the `substring()` method treats them as if they were `0`.

```
console.log(text.substring(-5, 2)); // "Mo"
console.log(text.substring(-5, -2)); // ""
```

`slice()` also treats `NaN` arguments as `0`, but when it is given negative values it counts backwards from the end of the string to find the indexes.

```
console.log(text.slice(-5, 2)); // ""
console.log(text.slice(-5, -2)); // "zil"
```

See the `slice()` page for more examples with negative numbers.

Replacing a substring within a string

The following example replaces a substring within a string. It will replace both individual characters and substrings. The function call at the end of the example creates a string `Brave New Web` from the original string `Brave New World`.

```
// Replaces oldS with newS in the string fullS
function replaceString(oldS, newS, fullS) {
  for (let i = 0; i < fullS.length; ++i) {
    if (fullS.substring(i, i + oldS.length) === oldS) {
      fullS =
        fullS.substring(0, i) +
        newS +
        fullS.substring(i + oldS.length, fullS.length);
    }
  }
  return fullS;
}

replaceString("World", "Web", "Brave New World");
```

Note that this can result in an infinite loop if `oldS` is itself a substring of `newS` — for example, if you attempted to replace `"World"` with `"OtherWorld"` here.

A better method for replacing strings is as follows:

```
function replaceString(oldS, newS, fullS) {
  return fullS.split(oldS).join(newS);
}
```

The code above serves as an example for substring operations. If you need to replace substrings, most of the time you will want to use `String.prototype.replace()`.

See also

- `String.prototype.substr()`
- `String.prototype.slice()`

String.prototype.sup()

The **`sup()`** method of `String` values creates a string that embeds this string in a `<sup>` element (`<sup>str</sup>`), which causes this string to be displayed as superscript.

> **Note:** All HTML wrapper methods are deprecated and only standardized for compatibility purposes. Use DOM APIs such as `document.createElement()` instead.

Syntax

```
sup()
```

Parameters

None.

Return value

A string beginning with a `<sup>` start tag, then the text `str`, and then a `</sup>` end tag.

Examples

Using sup()

The code below creates an HTML string and then replaces the document's body with it:

```
const contentString = "Hello, world";

document.body.innerHTML = contentString.sup();
```

This will create the following HTML:

```
<sup>Hello, world</sup>
```

Instead of using `sup()` and creating HTML text directly, you should use DOM APIs such as `document.createElement()`. For example:

```
const contentString = "Hello, world";
const elem = document.createElement("sup");
elem.innerText = contentString;
document.body.appendChild(elem);
```

See also

- Polyfill of `String.prototype.sup` in `core-js`
- es-shims polyfill of `String.prototype.sup`
- HTML wrapper methods
- `<sup>`

String.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `String` values implements the iterable protocol and allows strings to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a string iterator object that yields the Unicode code points of the string value as individual strings.

Try it

```
const str = "The quick red fox jumped over the lazy dog's back.";

const iterator = str[Symbol.iterator]();
let theChar = iterator.next();

while (!theChar.done && theChar.value !== " ") {
  console.log(theChar.value);
  theChar = iterator.next();
  // Expected output: "T"
  //                  "h"
  //                  "e"
}
```

Syntax

```
string[Symbol.iterator]()
```

Parameters

None.

Return value

A new iterable iterator object that yields the Unicode code points of the string value as individual strings.

Description

Strings are iterated by Unicode code points. This means grapheme clusters will be split, but surrogate pairs will be preserved.

```
// "Backhand Index Pointing Right: Dark Skin Tone"
[..."👉🏿"]; // ['👉', '🏿']
// splits into the basic "Backhand Index Pointing Right" emoji and
// the "Dark skin tone" emoji

// "Family: Man, Boy"
[..."👨‍👦"]; // [ '👨', '', '👦' ]
// splits into the "Man" and "Boy" emoji, joined by a ZWJ
```

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes strings iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const str = "A\uD835\uDC68B\uD835\uDC69C\uD835\uDC6A";

for (const v of str) {
  console.log(v);
}
// "A"
// "\uD835\uDC68"
// "B"
// "\uD835\uDC69"
// "C"
// "\uD835\uDC6A"
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
const str = "A\uD835\uDC68";

const strIter = str[Symbol.iterator]();

console.log(strIter.next().value); // "A"
console.log(strIter.next().value); // "\uD835\uDC68"
```

See also

- Polyfill of `String.prototype[Symbol.iterator]` in `core-js`
- Numbers and strings guide
- `Symbol.iterator`
- Iteration protocols

String.prototype.toLocaleLowerCase()

The **`toLocaleLowerCase()`** method of `String` values returns this string converted to lower case, according to any locale-specific case mappings.

Try it

```
const dotted = "İstanbul";

console.log(`EN-US: ${dotted.toLocaleLowerCase("en-US")}`);
// Expected output: "EN-US: istanbul"

console.log(`TR: ${dotted.toLocaleLowerCase("tr")}`);
// Expected output: "TR: istanbul"
```

Syntax

```
toLocaleLowerCase()
toLocaleLowerCase(locales)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Indicates the locale to be used to convert to lower case according to any locale-specific case mappings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

Unlike other methods that use the `locales` argument, `toLocaleLowerCase()` does not allow locale matching. Therefore, after checking the validity of the `locales` argument, `toLocaleLowerCase()` always uses the first locale in the list (or the default locale if the list is empty), even if this locale is not supported by the implementation.

Return value

A new string representing the calling string converted to lower case, according to any locale-specific case mappings.

Description

The `toLocaleLowerCase()` method returns the value of the string converted to lower case according to any locale-specific case mappings. `toLocaleLowerCase()` does not affect the value of the string itself. In most cases, this will produce the same result as `toLowerCase()`, but for some locales, such as Turkish, whose case mappings do not follow the default case mappings in Unicode, there may be a different result.

Examples

Using toLocaleLowerCase()

```
"ALPHABET".toLocaleLowerCase(); // 'alphabet'

"\u0130".toLocaleLowerCase("tr") === "i"; // true
"\u0130".toLocaleLowerCase("en-US") === "i"; // false

const locales = ["tr", "TR", "tr-TR", "tr-u-co-search", "tr-x-turkish"];
"\u0130".toLocaleLowerCase(locales) === "i"; // true
```

See also

- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toLowerCase()`
- `String.prototype.toUpperCase()`

String.prototype.toLocaleUpperCase()

The **`toLocaleUpperCase()`** method of `String` values returns this string converted to upper case, according to any locale-specific case mappings.

Try it

```
const city = "istanbul";

console.log(city.toLocaleUpperCase("en-US"));
// Expected output: "ISTANBUL"

console.log(city.toLocaleUpperCase("TR"));
// Expected output: "İSTANBUL"
```

Syntax

```
toLocaleUpperCase()
toLocaleUpperCase(locales)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Indicates the locale to be used to convert to upper case according to any locale-specific case mappings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

Unlike other methods that use the `locales` argument, `toLocaleUpperCase()` does not allow locale matching. Therefore, after checking the validity of the `locales` argument, `toLocaleUpperCase()` always uses the first locale in the list (or the default locale if the list is empty), even if this locale is not supported by the implementation.

Return value

A new string representing the calling string converted to upper case, according to any locale-specific case mappings.

Description

The `toLocaleUpperCase()` method returns the value of the string converted to upper case according to any locale-specific case mappings. `toLocaleUpperCase()` does not affect the value of the string itself. In most cases, this will produce the same result as `toUpperCase()`, but for some locales, such as Turkish, whose case mappings do not follow the default case mappings in Unicode, there may be a different result.

Also notice that conversion is not necessarily a 1:1 character mapping, as some characters might result in two (or even more) characters when transformed to upper-case. Therefore the length of the result string can differ from the input length. This also implies that the conversion is not stable, so for example the following can return `false`: `x.toLocaleLowerCase() === x.toLocaleUpperCase().toLocaleLowerCase()`

Examples

Using toLocaleUpperCase()

```
"alphabet".toLocaleUpperCase(); // 'ALPHABET'

"Gesäß".toLocaleUpperCase(); // 'GESÄSS'

"i\u0307".toLocaleUpperCase("lt-LT"); // 'I'

const locales = ["lt", "LT", "lt-LT", "lt-u-co-phonebk", "lt-x-lietuva"];
"i\u0307".toLocaleUpperCase(locales); // 'I'
```

See also

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLowerCase()`
- `String.prototype.toUpperCase()`

String.prototype.toLowerCase()

The **`toLowerCase()`** method of `String` values returns this string converted to lower case.

Try it

```
const sentence = "The quick brown fox jumps over the lazy dog.";

console.log(sentence.toLowerCase());
// Expected output: "the quick brown fox jumps over the lazy dog."
```

Syntax

```
toLowerCase()
```

Parameters

None.

Return value

A new string representing the calling string converted to lower case.

Description

The `toLowerCase()` method returns the value of the string converted to lower case. `toLowerCase()` does not affect the value of the string `str` itself.

Examples

Using <code>toLowerCase()</code>

```
console.log("ALPHABET".toLowerCase()); // 'alphabet'
```

See also

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toUpperCase()`

String.prototype.toString()

The **`toString()`** method of `String` values returns this string value.

Try it

```
const stringObj = new String("foo");

console.log(stringObj);
// Expected output: String { "foo" }

console.log(stringObj.toString());
// Expected output: "foo"
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the specified string value.

Description

The `String` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `String` values, the `toString` method returns the string itself (if it's a primitive) or the string that the `String` object wraps. It has the exact same implementation as `String.prototype.valueOf()`.

The `toString()` method requires its `this` value to be a `String` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to string values.

Because `String` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `String` *object* is used in a context expecting a string, such as in a template literal. However, string *primitive* values do not consult the `toString()` method to be coerced to strings — since they are already strings, no conversion is performed.

```
String.prototype.toString = () => "Overridden";
console.log(`${"foo"}`); // "foo"
console.log(`${new String("foo")}`); // "Overridden"
```

Examples

Using toString()

The following example displays the string value of a `String` object:

```
const x = new String("Hello world");

console.log(x.toString()); // "Hello world"
```

See also

- `String.prototype.valueOf()`

String.prototype.toUpperCase()

The **`toUpperCase()`** method of `String` values returns this string converted to uppercase.

Try it

```
const sentence = "The quick brown fox jumps over the lazy dog.";

console.log(sentence.toUpperCase());
// Expected output: "THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG."
```

Syntax

```
toUpperCase()
```

Parameters

None.

Return value

A new string representing the calling string converted to upper case.

Description

The `toUpperCase()` method returns the value of the string converted to uppercase. This method does not affect the value of the string itself since JavaScript strings are immutable.

Examples

Basic usage

```
console.log("alphabet".toUpperCase()); // 'ALPHABET'
```

Conversion of non-string <code>this</code> values to strings

This method will convert any non-string value to a string, when you set its `this` to a value that is not a string:

```
const a = String.prototype.toUpperCase.call({
  toString() {
    return "abcdef";
  },
});

const b = String.prototype.toUpperCase.call(true);

// prints out 'ABCDEF TRUE'.
console.log(a, b);
```

See also

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLocaleUpperCase()`

- `String.prototype.toLowerCase()`

String.prototype.toWellFormed()

The **`toWellFormed()`** method of `String` values returns a string where all lone surrogates of this string are replaced with the Unicode replacement character U+FFFD.

Syntax

```
toWellFormed()
```

Parameters

None.

Return value

A new string that is a copy of this string, with all lone surrogates replaced with the Unicode replacement character U+FFFD. If `str` is well formed, a new string is still returned (essentially a copy of `str`).

Description

Strings in JavaScript are UTF-16 encoded. UTF-16 encoding has the concept of *surrogate pairs*, which is introduced in detail in the UTF-16 characters, Unicode code points, and grapheme clusters section.

`toWellFormed()` iterates through the code units of this string, and replaces any lone surrogates with the Unicode replacement character U+FFFD `�`. This ensures that the returned string is well-formed and can be used in functions that expect well-formed strings, such as `encodeURI`. Compared to a custom implementation, `toWellFormed()` is more efficient, as engines can directly access the internal representation of strings.

When ill-formed strings are used in certain contexts, such as `TextEncoder`, they are automatically converted to well-formed strings using the same replacement character. When lone surrogates are rendered, they are also rendered as the replacement character (a diamond with a question mark inside).

Examples

Using toWellFormed()

```
const strings = [
  // Lone leading surrogate
  "ab\uD800",
  "ab\uD800c",
  // Lone trailing surrogate
  "\uDFFFab",
  "c\uDFFFab",
  // Well-formed
  "abc",
  "ab\uD83D\uDE04c",
];

for (const str of strings) {
  console.log(str.toWellFormed());
}
// Logs:
// "ab�"
// "ab�c"
// "�ab"
// "c�ab"
// "abc"
// "ab😄c"
```

Avoiding errors in encodeURI()

`encodeURI` throws an error if the string passed is not well-formed. This can be avoided by using `toWellFormed()` to convert the string to a well-formed string first.

```
const illFormed = "https://example.com/search?q=\uD800";

try {
  encodeURI(illFormed);
} catch (e) {
  console.log(e); // URIError: URI malformed
}

console.log(encodeURI(illFormed.toWellFormed())); // "https://example.com/search?q=%EF%BF%BD"
```

See also

- Polyfill of `String.prototype.toWellFormed` in `core-js`
- es-shims polyfill of `String.prototype.toWellFormed`
- `String.prototype.isWellFormed()`
- `String.prototype.normalize()`

String.prototype.trim()

The **`trim()`** method of `String` values removes whitespace from both ends of this string and returns a new string, without modifying the original string.

To return a new string with whitespace trimmed from just one end, use `trimStart()` or `trimEnd()`.

Try it

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trim());
// Expected output: "Hello world!";
```

Syntax

```
trim()
```

Parameters

None.

Return value

A new string representing `str` stripped of whitespace from both its beginning and end. Whitespace is defined as white space characters plus line terminators.

If neither the beginning nor the end of `str` has any whitespace, a new string is still returned (essentially a copy of `str`).

Examples

Using trim()

The following example trims whitespace from both ends of `str`.

```
const str = "   foo  ";
console.log(str.trim()); // 'foo'
```

See also

- `String.prototype.trimStart()`
- `String.prototype.trimEnd()`

String.prototype.trimEnd()

The **`trimEnd()`** method of `String` values removes whitespace from the end of this string and returns a new string, without modifying the original string. `trimRight()` is an alias of this method.

Try it

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trimEnd());
// Expected output: "   Hello world!";
```

Syntax

```
trimEnd()

trimRight()
```

Parameters

None.

Return value

A new string representing `str` stripped of whitespace from its end (right side). Whitespace is defined as white space characters plus line terminators.

If the end of `str` has no whitespace, a new string is still returned (essentially a copy of `str`).

Aliasing

After `trim()` was standardized, engines also implemented the non-standard method `trimRight`. However, for consistency with `padEnd()`, when the method got standardized, its name was chosen as `trimEnd`. For web compatibility reasons, `trimRight` remains as an alias to `trimEnd`, and they refer to the exact same function object. In some engines this means:

```
String.prototype.trimRight.name === "trimEnd";
```

Examples

Using trimEnd()

The following example trims whitespace from the end of `str`, but not from its start.

```
let str = "   foo  ";

console.log(str.length); // 8

str = str.trimEnd();
console.log(str.length); // 6
console.log(str); // '   foo'
```

See also

- Polyfill of `String.prototype.trimEnd` in `core-js`
- es-shims polyfill of `String.prototype.trimEnd`
- `String.prototype.trim()`
- `String.prototype.trimStart()`

String.prototype.trimStart()

The **`trimStart()`** method of `String` values removes whitespace from the beginning of this string and returns a new string, without modifying the original string. `trimLeft()` is an alias of this method.

Try it

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trimStart());
// Expected output: "Hello world!   ";
```

Syntax

```
trimStart()

trimLeft()
```

Parameters

None.

Return value

A new string representing `str` stripped of whitespace from its beginning (left side). Whitespace is defined as white space characters plus line terminators.

If the beginning of `str` has no whitespace, a new string is still returned (essentially a copy of `str`).

Aliasing

After `trim()` was standardized, engines also implemented the non-standard method `trimLeft`. However, for consistency with `padStart()`, when the method got standardized, its name was chosen as `trimStart`. For web compatibility reasons, `trimLeft` remains as an alias to `trimStart`, and they refer to the exact same function object. In some engines this means:

```
String.prototype.trimLeft.name === "trimStart";
```

Examples

Using trimStart()

The following example trims whitespace from the start of `str`, but not from its end.

```
let str = "   foo  ";

console.log(str.length); // 8

str = str.trimStart();
console.log(str.length); // 5
console.log(str); // 'foo  '
```

See also

- Polyfill of `String.prototype.trimStart` in `core-js`
- es-shims polyfill of `String.prototype.trimStart`
- `String.prototype.trim()`
- `String.prototype.trimEnd()`

String.prototype.valueOf()

The **`valueOf()`** method of `String` values returns this string value.

Try it

```
const stringObj = new String("foo");

console.log(stringObj);
// Expected output: String { "foo" }

console.log(stringObj.valueOf());
// Expected output: "foo"
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

A string representing the primitive value of a given `String` object.

Description

The `valueOf()` method of `String` returns the primitive value of a `String` object as a string data type. This value is equivalent to `String.prototype.toString()`.

This method is usually called internally by JavaScript and not explicitly in code.

Examples

Using <code>valueOf()</code>

```
const x = new String("Hello world");
console.log(x.valueOf()); // 'Hello world'
```

See also

- `String.prototype.toString()`
- `Object.prototype.valueOf()`

SuppressedError

The **`SuppressedError`** object represents an error generated while handing another error. It is generated during resource disposal using `using` or `await using`.

Compared to `AggregateError`, `SuppressedError` represents an error that happened during the handling of another error, while `AggregateError` represents a list of unrelated errors. It is possible, though, for a `SuppressedError` to contain a chain of suppressed errors (`e.suppressed.suppressed.suppressed...`). It is also semantically different from `cause` because the error is not *caused* by another error, but *happens when* handling another error.

`SuppressedError` is a subclass of `Error`.

Constructor

`SuppressedError()`

Creates a new `SuppressedError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `SuppressedError.prototype` and shared by all `SuppressedError` instances.

`SuppressedError.prototype.constructor`

The constructor function that created the instance object. For `SuppressedError` instances, the initial value is the `SuppressedError` constructor.

`SuppressedError.prototype.name`

Represents the name for the type of error. For `SuppressedError.prototype.name`, the initial value is `"SuppressedError"`.

> **Note:** `SuppressedError` never has the `cause` property, because the semantics of `cause` overlaps with `suppressed`.

These properties are own properties of each `SuppressedError` instance.

`error`

A reference to the error that results in the suppression.

`suppressed`

A reference to the error that is suppressed by `error`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a SuppressedError

A `SuppressedError` is thrown when an error occurs during resource disposal. Throwing an error causes scope cleanup, and each disposer during the cleanup can throw its own error. All these errors are collected into a chain of `SuppressedError` instances, with the original error as the `suppressed` property and the new error thrown by the next disposer as the `error` property.

```
try {
  using resource1 = {
    [Symbol.dispose]() {
      throw new Error("resource1 disposal failed");
    },
  };
  using resource2 = {
    [Symbol.dispose]() {
      throw new Error("resource2 disposal failed");
    },
  };
  throw new TypeError("Original error");
} catch (e) {
  console.log(e instanceof SuppressedError); // true
  console.log(e.message); // "An error was suppressed during disposal"
  console.log(e.name); // "SuppressedError"
  console.log(e.error); // Error: resource1 disposal failed
  console.log(e.suppressed); // SuppressedError: An error was suppressed during disposal
  console.log(e.suppressed.error); // Error: resource2 disposal failed
  console.log(e.suppressed.suppressed); // TypeError: Original error
}
```

The chain looks like this:

```
     SuppressedError --suppressed--> SuppressedError --suppressed--> TypeError
           |                               |
         error                           error
           |                               |
           v                               v
resource1 disposal failed       resource2 disposal failed
 (Disposal happens later)       (Disposal happens earlier)
```

Creating a SuppressedError

```
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e instanceof SuppressedError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "SuppressedError"
  console.log(e.error); // Error: "New error"
  console.log(e.suppressed); // Error: "Original error"
}
```

See also

- Polyfill of `SuppressedError` in `core-js`
- `Error`
- `using`
- `await using`
- `DisposableStack`
- `AsyncDisposableStack`

SuppressedError: error

The **`error`** data property of a `SuppressedError` instance contains a reference to the error that results in the suppression.

Value

Any value. Like `cause`, you cannot assume it's an `Error` instance, although it usually is the case.

Property attributes of `SuppressedError: error`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using error

```
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e.error); // Error: "New error"
}
```

See also

- Control flow and error handling guide
- `SuppressedError`
- `Error: cause`

SuppressedError: suppressed

The **`suppressed`** data property of a `SuppressedError` instance contains a reference to the original error that got suppressed because a new error was generated while handling it.

Value

Any value. Like `cause`, you cannot assume it's an `Error` instance, although it usually is the case.

Property attributes of `SuppressedError: suppressed`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using suppressed

```
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e.suppressed); // Error: "Original error"
}
```

See also

- Control flow and error handling guide
- `SuppressedError`
- `Error: cause`

SuppressedError() constructor

The **`SuppressedError()`** constructor creates `SuppressedError` objects.

Syntax

```
new SuppressedError(error, suppressed)
new SuppressedError(error, suppressed, message)

SuppressedError(error, suppressed)
SuppressedError(error, suppressed, message)
```

> **Note:** `SuppressedError()` can be called with or without `new`. Both create a new `SuppressedError` instance.

Parameters

`error`

The new error that results in the suppression of `suppressed`.

`suppressed`

The error that was originally thrown and is now suppressed.

`message` Optional

An optional human-readable description of the aggregate error.

> **Note:** `SuppressedError()` does not accept `options` like `Error()` and other subclasses do, because the semantics of `cause` overlaps with `suppressed`.

Examples

Creating a SuppressedError

```
try {
  throw new SuppressedError(
    new Error("New error"),
    new Error("Original error"),
    "Hello",
  );
} catch (e) {
  console.log(e.suppressed); // Error: "Original error"
  console.log(e.error); // Error: "New error"
}
```

See also

- Polyfill of `SuppressedError` in `core-js`

Symbol

`Symbol` is a built-in object whose constructor returns a `symbol` primitive — also called a **Symbol value** or just a **Symbol** — that's guaranteed to be unique. Symbols are often used to add unique property keys to an object that won't collide with keys any other code might add to the object, and which are hidden from any mechanisms other code will typically use to access the object. That enables a form of weak encapsulation, or a weak form of information hiding.

Every `Symbol()` call is guaranteed to return a unique Symbol. Every `Symbol.for("key")` call will always return the same Symbol for a given value of `"key"`. When `Symbol.for("key")` is called, if a Symbol with the given key can be found in the global Symbol registry, that Symbol is returned. Otherwise, a new Symbol is created, added to the global Symbol registry under the given key, and returned.

Description

To create a new primitive Symbol, you write `Symbol()` with an optional string as its description:

```
const sym1 = Symbol();
const sym2 = Symbol("foo");
const sym3 = Symbol("foo");
```

The above code creates three new Symbols. Note that `Symbol("foo")` does not coerce the string `"foo"` into a Symbol. It creates a new Symbol each time:

```
Symbol("foo") === Symbol("foo"); // false
```

The following syntax with the `new` operator will throw a `TypeError`:

```
const sym = new Symbol(); // TypeError
```

This prevents authors from creating an explicit `Symbol` wrapper object instead of a new Symbol value and might be surprising as creating explicit wrapper objects around primitive data types is generally possible (for example, `new Boolean`, `new String` and `new Number`).

If you really want to create a `Symbol` wrapper object, you can use the `Object()` function:

```
const sym = Symbol("foo");
typeof sym; // "symbol"
const symObj = Object(sym);
typeof symObj; // "object"
```

Because symbols are the only primitive data type that has reference identity (that is, you cannot create the same symbol twice), they behave like objects in some way. For example, they are garbage collectable and can therefore be stored in `WeakMap`, `WeakSet`, `WeakRef`, and `FinalizationRegistry` objects.

Shared Symbols in the global Symbol registry

The above syntax using the `Symbol()` function will create a Symbol whose value remains unique throughout the lifetime of the program. To create Symbols available across files and even across realms (each of which has its own global scope), use the methods `Symbol.for()` and `Symbol.keyFor()` to set and retrieve Symbols from the global Symbol registry.

Note that the "global Symbol registry" is only a fictitious concept and may not correspond to any internal data structure in the JavaScript engine — and even if such a registry exists, its content is not available to the JavaScript code, except through the `for()` and `keyFor()` methods.

The method `Symbol.for(tokenString)` takes a string key and returns a symbol value from the registry, while `Symbol.keyFor(symbolValue)` takes a symbol value and returns the string key corresponding to it. Each is the other's inverse, so the following is `true`:

```
Symbol.keyFor(Symbol.for("tokenString")) === "tokenString"; // true
```

Because registered symbols can be arbitrarily created anywhere, they behave almost exactly like the strings they wrap. Therefore, they are not guaranteed to be unique and are not garbage collectable. Therefore, registered symbols are disallowed in `WeakMap`, `WeakSet`, `WeakRef`, and `FinalizationRegistry` objects.

Well-known Symbols

All static properties of the `Symbol` constructor are Symbols themselves, whose values are constant across realms. They are known as *well-known Symbols*, and their purpose is to serve as "protocols" for certain built-in JavaScript operations, allowing users to customize the language's behavior. For example, if a constructor function has a method with `Symbol.hasInstance` as its name, this method will encode its behavior with the `instanceof` operator.

Prior to well-known Symbols, JavaScript used normal properties to implement certain built-in operations. For example, the `JSON.stringify` function will attempt to call each object's `toJSON()` method, and the `String` function will call the object's `toString()` and `valueOf()` methods. However, as more operations are added to the language, designating each operation a "magic property" can break backward compatibility and make the language's behavior harder to reason with. Well-known Symbols allow the customizations to be "invisible" from normal code, which typically only read string properties.

> **Note:** The spec used to use the notation `@@<symbol-name>` to denote well-known symbols. For example, `Symbol.hasInstance` was written as `@@hasInstance`, and the `Array.prototype[Symbol.iterator]()` method would be called `Array.prototype[@@iterator]()`. This notation is no longer used in the spec, but you may still see it in older documentation or discussions.

Well-known symbols do not have the concept of garbage collectability, because they come in a fixed set and are unique throughout the lifetime of the program, similar to intrinsic objects such as `Array.prototype`, so they are also allowed in `WeakMap`, `WeakSet`, `WeakRef`, and `FinalizationRegistry` objects.

Finding Symbol properties on objects

The method `Object.getOwnPropertySymbols()` returns an array of Symbols and lets you find Symbol properties on a given object. Note that every object is initialized with no own Symbol properties, so that this array will be empty unless you've set Symbol properties on the object.

Constructor

`Symbol()`

Returns primitive values of type Symbol. Throws an error when called with `new`.

Static properties

The static properties are all well-known Symbols. In these Symbols' descriptions, we will use language like "`Symbol.hasInstance` is a method determining...", but bear in mind that this is referring to the semantic of an object's method having this Symbol as the method name (because well-known Symbols act as "protocols"), not describing the value of the Symbol itself.

`Symbol.asyncDispose`

A method that disposes resources of the object asynchronously when the object goes out of scope. Used by the `await using` declaration.

`Symbol.asyncIterator`

A method that returns the default AsyncIterator for an object. Used by `for await...of`.

`Symbol.dispose`

A method that disposes resources of the object when the object goes out of scope. Used by the `using` declaration.

`Symbol.hasInstance`

A method determining if a constructor object recognizes an object as its instance. Used by `instanceof`.

`Symbol.isConcatSpreadable`

A Boolean value indicating if an object should be flattened to its array elements. Used by `Array.prototype.concat()`.

`Symbol.iterator`

A method returning the default iterator for an object. Used by `for...of`.

`Symbol.match`

A method that matches against a string, also used to determine if an object may be used as a regular expression. Used by `String.prototype.match()`.

`Symbol.matchAll`

A method that returns an iterator, that yields matches of the regular expression against a string. Used by `String.prototype.matchAll()`.

`Symbol.replace`

A method that replaces matched substrings of a string. Used by `String.prototype.replace()`.

`Symbol.search`

A method that returns the index within a string that matches the regular expression. Used by `String.prototype.search()`.

`Symbol.species`

A constructor function that is used to create derived objects.

`Symbol.split`

A method that splits a string at the indices that match a regular expression. Used by `String.prototype.split()`.

`Symbol.toPrimitive`

A method converting an object to a primitive value.

`Symbol.toStringTag`

A string value used for the default description of an object. Used by `Object.prototype.toString()`.

`Symbol.unscopables`

An object value of whose own and inherited property names are excluded from the `with` environment bindings of the associated object.

Static methods

`Symbol.for()`

Searches for existing registered Symbols in the global Symbol registry with the given `key` and returns it if found. Otherwise a new Symbol gets created and registered with `key`.

`Symbol.keyFor()`

Retrieves a shared Symbol key from the global Symbol registry for the given Symbol.

Instance properties

These properties are defined on `Symbol.prototype` and shared by all `Symbol` instances.

`Symbol.prototype.constructor`

The constructor function that created the instance object. For `Symbol` instances, the initial value is the `Symbol` constructor.

`Symbol.prototype.description`

A read-only string containing the description of the Symbol.

`Symbol.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Symbol"`. This property is used in `Object.prototype.toString()`. However, because `Symbol` also has its own `toString()` method, this property is not used unless you call `Object.prototype.toString.call()` with a symbol as `thisArg`.

Instance methods

`Symbol.prototype.toString()`

Returns a string containing the description of the Symbol. Overrides the `Object.prototype.toString()` method.

`Symbol.prototype.valueOf()`

Returns the Symbol. Overrides the `Object.prototype.valueOf()` method.

`Symbol.prototype[Symbol.toPrimitive]()`

Returns the Symbol.

Examples

Using the typeof operator with Symbols

The `typeof` operator can help you to identify Symbols.

```
typeof Symbol() === "symbol";
typeof Symbol("foo") === "symbol";
typeof Symbol.iterator === "symbol";
```

Symbol type conversions

Some things to note when working with type conversion of Symbols.

- When trying to convert a Symbol to a number, a `TypeError` will be thrown (e.g., `+sym` or `sym | 0`).
- When using loose equality, `Object(sym) == sym` returns `true`.
- `Symbol("foo") + "bar"` throws a `TypeError` (can't convert Symbol to string). This prevents you from silently creating a new string property name from a Symbol, for example.
- The "safer" `String(sym)` conversion works like a call to `Symbol.prototype.toString()` with Symbols, but note that `new String(sym)` will throw.

Symbols and for...in iteration

Symbols are not enumerable in `for...in` iterations. In addition, `Object.getOwnPropertyNames()` will not return Symbol object properties, however, you can use `Object.getOwnPropertySymbols()` to get these.

```
const obj = {};

obj[Symbol("a")] = "a";
obj[Symbol.for("b")] = "b";
obj["c"] = "c";
obj.d = "d";
```

```
for (const i in obj) {
  console.log(i);
}
// "c" "d"
```

Symbols and JSON.stringify()

Symbol-keyed properties will be completely ignored when using `JSON.stringify()`:

```
JSON.stringify({ [Symbol("foo")]: "foo" });
// '{}'
```

For more details, see `JSON.stringify()`.

Symbol wrapper objects as property keys

When a Symbol wrapper object is used as a property key, this object will be coerced to its wrapped Symbol:

```
const sym = Symbol("foo");
const obj = { [sym]: 1 };
obj[sym]; // 1
obj[Object(sym)]; // still 1
```

See also

- Polyfill of `Symbol` in `core-js`
- `typeof`
- JavaScript data types and data structures
- ES6 In Depth: Symbols on hacks.mozilla.org (2015)

Symbol.asyncDispose

The **`Symbol.asyncDispose`** static data property represents the well-known symbol `Symbol.asyncDispose`. The `await using` declaration looks up this symbol on the variable initializer for the method to call when the variable goes out of scope.

Value

The well-known symbol `Symbol.asyncDispose`.

Property attributes of `Symbol.asyncDispose`	
Writable	no
Enumerable	no
Configurable	no

Description

An object is async disposable if it has the `[Symbol.asyncDispose]()` method. The method is expected to have the following semantics:

- Invoking this method notifies the AsyncDisposable object that the caller does not intend to continue to use this object. This method should perform any necessary logic to explicit clean up the resource including, but not limited to, file system handles, streams, host objects, etc.
- This method can return a promise, which will be awaited before continuing.
- When an exception is thrown from this method, it typically means that the resource could not be explicitly freed. An AsyncDisposable object is not considered "disposed" until the resulting Promise has been fulfilled.
- If called more than once on the same object, the function should not throw an exception. However, this requirement is not enforced.

Examples

User defined async disposables

`[Symbol.asyncDispose]` allows the creation of custom async disposables. See the `await using` reference for more information.

```
class Disposable {
  #fileHandle;
  #disposed;

  constructor(handle) {
    this.#disposed = false;
    this.#fileHandle = handle;
  }

  async [Symbol.asyncDispose]() {
    await this.#fileHandle.close();
    this.disposed = true;
  }

  get isDisposed() {
    return this.disposed;
  }
}

const resource = new Disposable(await fs.open("my-file.txt", "r"));
```

```
{
  await using resourceUsed = resource;
  console.log(resource.isDisposed); // false
}
console.log(resource.isDisposed); // true
```

See also

- Polyfill of `Symbol.asyncDispose` in `core-js`
- JavaScript resource management
- `Symbol.dispose`
- `await using`
- `AsyncDisposableStack.prototype[Symbol.asyncDispose]()`
- `AsyncIterator.prototype[Symbol.asyncDispose]()`

Symbol.asyncIterator

The **`Symbol.asyncIterator`** static data property represents the well-known symbol `Symbol.asyncIterator`. The async iterable protocol looks up this symbol for the method that returns the async iterator for an object. In order for an object to be async iterable, it must have a `[Symbol.asyncIterator]` key.

Try it

```
const delayedResponses = {
  delays: [500, 1300, 3500],

  wait(delay) {
    return new Promise((resolve) => {
      setTimeout(resolve, delay);
    });
  },

  async *[Symbol.asyncIterator]() {
    for (const delay of this.delays) {
      await this.wait(delay);
      yield `Delayed response for ${delay} milliseconds`;
    }
  },
};

(async () => {
  for await (const response of delayedResponses) {
    console.log(response);
  }
})();

// Expected output: "Delayed response for 500 milliseconds"
// Expected output: "Delayed response for 1300 milliseconds"
// Expected output: "Delayed response for 3500 milliseconds"
```

Value

The well-known symbol `Symbol.asyncIterator`.

Property attributes of `Symbol.asyncIterator`	
Writable	no
Enumerable	no
Configurable	no

Examples

User-defined async iterables

You can define your own async iterable by setting the `[Symbol.asyncIterator]()` property on an object.

```
const myAsyncIterable = {
  async *[Symbol.asyncIterator]() {
    yield "hello";
    yield "async";
    yield "iteration!";
  },
};

(async () => {
  for await (const x of myAsyncIterable) {
    console.log(x);
  }
})();
// Logs:
// "hello"
// "async"
// "iteration!"
```

When creating an API, remember that async iterables are designed to represent something *iterable* — like a stream of data or a list —, not to completely replace callbacks and events in most situations.

Built-in async iterables

There is no object in the core JavaScript language that is async iterable. Some web APIs, such as `ReadableStream`, have the `Symbol.asyncIterator` method set by default.

See also

- Iteration protocols
- for await...of

Symbol.prototype.description

The **`description`** accessor property of `Symbol` values returns a string containing the description of this symbol, or `undefined` if the symbol has no description.

Try it

```
console.log(Symbol("desc").description);
// Expected output: "desc"

console.log(Symbol.iterator.description);
// Expected output: "Symbol.iterator"

console.log(Symbol.for("foo").description);
// Expected output: "foo"

console.log(`${Symbol("foo").description}bar`);
// Expected output: "foobar"
```

Description

`Symbol` objects can be created with an optional description which can be used for debugging but not to access the symbol itself. The `Symbol.prototype.description` property can be used to read that description. It is different to `Symbol.prototype.toString()` as it does not contain the enclosing `"Symbol()"` string. See the examples.

Examples

Using description

```
Symbol("desc").toString(); // "Symbol(desc)"
Symbol("desc").description; // "desc"
Symbol("").description; // ""
Symbol().description; // undefined

// well-known symbols
Symbol.iterator.toString(); // "Symbol(Symbol.iterator)"
Symbol.iterator.description; // "Symbol.iterator"

// global symbols
Symbol.for("foo").toString(); // "Symbol(foo)"
Symbol.for("foo").description; // "foo"
```

See also

- Polyfill of `Symbol.prototype.description` in `core-js`
- es-shims polyfill of `Symbol.prototype.description`
- `Symbol.prototype.toString()`

Symbol.dispose

The **`Symbol.dispose`** static data property represents the well-known symbol `Symbol.dispose`. The `using` declaration looks up this symbol on the variable initializer for the method to call when the variable goes out of scope.

Value

The well-known symbol `Symbol.dispose`.

Property attributes of `Symbol.dispose`	
Writable	no
Enumerable	no
Configurable	no

Description

An object is disposable if it has the `[Symbol.dispose]()` method. The method is expected to have the following semantics:

- Invoking this method notifies the Disposable object that the caller does not intend to continue to use this object. This method should perform any necessary logic to explicit clean up the resource including, but not limited to, file system handles, streams, host objects, etc.
- When an exception is thrown from this method, it typically means that the resource could not be explicitly freed.
- If called more than once on the same object, the function should not throw an exception. However, this requirement is not enforced.

This method should not return a promise, as promises returned by `[Symbol.dispose]()` are not awaited by `await using`. To declare async disposables, use `Symbol.asyncDispose`.

Examples

User defined disposables

`[Symbol.dispose]` allows the creation of custom disposables. See the `using` reference for more information.

```
class Disposable {
  constructor() {
    this.disposed = false;
  }

  [Symbol.dispose]() {
    this.disposed = true;
  }

  get isDisposed() {
    return this.disposed;
  }
}

const resource = new Disposable();
{
  using resourceUsed = resource;
  console.log(resource.isDisposed); // false
}
console.log(resource.isDisposed); // true
```

See also

- Polyfill of `Symbol.dispose` in `core-js`
- JavaScript resource management
- `Symbol.asyncDispose`
- `using`
- `DisposableStack.prototype[Symbol.dispose]()`
- `Iterator.prototype[Symbol.dispose]()`

Symbol.for()

The **`Symbol.for()`** static method searches for existing symbols in a runtime-wide symbol registry with the given key and returns it if found. Otherwise a new symbol gets created in the global symbol registry with this key.

Try it

```
console.log(Symbol.for("bar") === Symbol.for("bar"));
// Expected output: true

console.log(Symbol("bar") === Symbol("bar"));
// Expected output: false

const symbol1 = Symbol.for("foo");

console.log(symbol1.toString());
// Expected output: "Symbol(foo)"
```

Syntax

```
Symbol.for(key)
```

Parameters

`key`

String, required. The key for the symbol (and also used for the description of the symbol).

Return value

An existing symbol with the given key if found; otherwise, a new symbol is created and returned.

Description

In contrast to `Symbol()`, the `Symbol.for()` function creates a symbol available in a global symbol registry list. `Symbol.for()` does also not necessarily create a new symbol on every call, but checks first if a symbol with the given `key` is already present in the registry. In that case, that symbol is returned. If no symbol with the given key is found, `Symbol.for()` will create a new global symbol.

Examples

Using Symbol.for()

```
Symbol.for("foo"); // create a new global symbol
Symbol.for("foo"); // retrieve the already created symbol
```

```
// Same global symbol, but not locally
Symbol.for("bar") === Symbol.for("bar"); // true
Symbol("bar") === Symbol("bar"); // false

// The key is also used as the description
const sym = Symbol.for("mario");
sym.toString(); // "Symbol(mario)"
```

To avoid name clashes with your global symbol keys and other (library code) global symbols, it might be a good idea to prefix your symbols:

```
Symbol.for("mdn.foo");
Symbol.for("mdn.bar");
```

See also

- `Symbol.keyFor()`

Symbol.hasInstance

The **`Symbol.hasInstance`** static data property represents the well-known symbol `Symbol.hasInstance`. The `instanceof` operator looks up this symbol on its right-hand operand for the method used to determine if the constructor object recognizes an object as its instance.

Try it

```
class Array1 {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance);
  }
}

console.log([] instanceof Array1);
// Expected output: true
```

Value

The well-known symbol `Symbol.hasInstance`.

Property attributes of `Symbol.hasInstance`	
Writable	no
Enumerable	no
Configurable	no

Description

The `instanceof` operator uses the following algorithm to calculate the return value of `object instanceof constructor`:

1. If `constructor` has a `[Symbol.hasInstance]()` method, then call it with `object` as the first argument and return the result, coerced to a boolean. Throw a `TypeError` if `constructor` is not an object, or if `constructor[Symbol.hasInstance]` is not one of `null`, `undefined`, or a function.
2. Otherwise, if `constructor` doesn't have a `[Symbol.hasInstance]()` method (`constructor[Symbol.hasInstance]` is `null` or `undefined`), then determine the result using the same algorithm as `Function.prototype[Symbol.hasInstance]()`. Throw a `TypeError` if `constructor` is not a function.

Because all functions inherit from `Function.prototype` by default, most of the time, the `Function.prototype[Symbol.hasInstance]()` method specifies the behavior of `instanceof` when the right-hand side is a function.

Examples

Custom instanceof behavior

You could implement your custom `instanceof` behavior like this, for example:

```
class MyArray {
  static [Symbol.hasInstance](instance) {
    return Array.isArray(instance);
  }
}
console.log([] instanceof MyArray); // true
```

```
function MyArray() {}
Object.defineProperty(MyArray, Symbol.hasInstance, {
  value(instance) {
    return Array.isArray(instance);
  },
});
console.log([] instanceof MyArray); // true
```

Checking the instance of an object

Just in the same manner at which you can check if an object is an instance of a class using the `instanceof` keyword, we can also use `Symbol.hasInstance` for such checks.

```
class Animal {
  constructor() {}
}

const cat = new Animal();

console.log(Animal[Symbol.hasInstance](cat)); // true
```

See also

- `instanceof`
- `Function.prototype[Symbol.hasInstance]()`

Symbol.isConcatSpreadable

The **`Symbol.isConcatSpreadable`** static data property represents the well-known symbol `Symbol.isConcatSpreadable`. The `Array.prototype.concat()` method looks up this symbol on each object being concatenated to determine if it should be treated as an array-like object and flattened to its array elements.

Try it

```
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];
let alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric);
// Expected output: Array ["a", "b", "c", 1, 2, 3]

numeric[Symbol.isConcatSpreadable] = false;
alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric);
// Expected output: Array ["a", "b", "c", Array [1, 2, 3]]
```

Value

The well-known symbol `Symbol.isConcatSpreadable`.

Property attributes of `Symbol.isConcatSpreadable`	
Writable	no
Enumerable	no
Configurable	no

Description

The `[Symbol.isConcatSpreadable]` property can be defined as an own or inherited property and its value is a boolean. It can control behavior for arrays and array-like objects:

- For array objects, the default behavior is to spread (flatten) elements. `Symbol.isConcatSpreadable` can avoid flattening in these cases.
- For array-like objects, the default behavior is no spreading or flattening. `Symbol.isConcatSpreadable` can force flattening in these cases.

Examples

Arrays

By default, `Array.prototype.concat()` spreads (flattens) arrays into its result:

```
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];

const alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric); // Result: ['a', 'b', 'c', 1, 2, 3]
```

When setting `Symbol.isConcatSpreadable` to `false`, you can disable the default behavior:

```
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];

numeric[Symbol.isConcatSpreadable] = false;
const alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric); // Result: ['a', 'b', 'c', [1, 2, 3] ]
```

Array-like objects

For array-like objects, the default is to not spread. `Symbol.isConcatSpreadable` needs to be set to `true` in order to get a flattened array:

```
const x = [1, 2, 3];

const fakeArray = {
  [Symbol.isConcatSpreadable]: true,
```

```
  length: 2,
  0: "hello",
  1: "world",
};

x.concat(fakeArray); // [1, 2, 3, "hello", "world"]
```

> **Note:** The `length` property is used to control the number of object properties to be added. In the above example, `length:2` indicates that two properties have to be added.

See also

- Polyfill of `Symbol.isConcatSpreadable` in `core-js`
- `Array.prototype.concat()`

Symbol.iterator

The **`Symbol.iterator`** static data property represents the well-known symbol `Symbol.iterator`. The iterable protocol looks up this symbol for the method that returns the iterator for an object. In order for an object to be iterable, it must have a `[Symbol.iterator]` key.

Try it

```
const iterable = {};

iterable[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};

console.log([...iterable]);
// Expected output: Array [1, 2, 3]
```

Value

The well-known symbol `Symbol.iterator`.

Property attributes of `Symbol.iterator`	
Writable	no
Enumerable	no
Configurable	no

Description

Whenever an object needs to be iterated (such as at the beginning of a `for...of` loop), its `[Symbol.iterator]()` method is called with no arguments, and the returned **iterator** is used to obtain the values to be iterated.

Some built-in types have a default iteration behavior, while other types (such as `Object`) do not. Some built-in types with a `[Symbol.iterator]()` method are:

- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype[Symbol.iterator]()`
- `String.prototype[Symbol.iterator]()`
- `Map.prototype[Symbol.iterator]()`
- `Set.prototype[Symbol.iterator]()`

See also Iteration protocols for more information.

Examples

User-defined iterables

We can make our own iterables like this:

```
const myIterable = {};
myIterable[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};
[...myIterable]; // [1, 2, 3]
```

Or iterables can be defined directly inside a class or object using a computed property:

```
class Foo {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
```

```
}

const someObj = {
  *[Symbol.iterator]() {
    yield "a";
    yield "b";
  },
};

console.log(...new Foo()); // 1, 2, 3
console.log(...someObj); // 'a', 'b'
```

Non-well-formed iterables

If an iterable's `[Symbol.iterator]()` method does not return an iterator object, then it is a non-well-formed iterable. Using it as such is likely to result in runtime exceptions or buggy behavior:

```
const nonWellFormedIterable = {};
nonWellFormedIterable[Symbol.iterator] = () => 1;
[...nonWellFormedIterable]; // TypeError: [Symbol.iterator]() returned a non-object value
```

See also

- Polyfill of `Symbol.iterator` in `core-js`
- Iteration protocols
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype[Symbol.iterator]()`
- `String.prototype[Symbol.iterator]()`
- `Map.prototype[Symbol.iterator]()`
- `Set.prototype[Symbol.iterator]()`
- `arguments[Symbol.iterator]()`
- `Segments.prototype[Symbol.iterator]()`

Symbol.keyFor()

The **`Symbol.keyFor()`** static method retrieves a shared symbol key from the global symbol registry for the given symbol.

Try it

```
const globalSym = Symbol.for("foo"); // Global symbol

console.log(Symbol.keyFor(globalSym));
// Expected output: "foo"

const localSym = Symbol(); // Local symbol

console.log(Symbol.keyFor(localSym));
// Expected output: undefined

console.log(Symbol.keyFor(Symbol.iterator));
// Expected output: undefined
```

Syntax

```
Symbol.keyFor(sym)
```

Parameters

`sym`

Symbol, required. The symbol to find a key for.

Return value

A string representing the key for the given symbol if one is found on the global registry; otherwise, `undefined`.

Examples

Using keyFor()

```
const globalSym = Symbol.for("foo"); // create a new global symbol
Symbol.keyFor(globalSym); // "foo"

const localSym = Symbol();
Symbol.keyFor(localSym); // undefined

// well-known symbols are not symbols registered
// in the global symbol registry
Symbol.keyFor(Symbol.iterator); // undefined
```

See also

- `Symbol.for()`

Symbol.match

The **`Symbol.match`** static data property represents the well-known symbol `Symbol.match`. The `String.prototype.match()` method looks up this symbol on its first argument for the method used to match an input string against the current object. This symbol is also used to determine if an object should be treated as a regex.

For more information, see `RegExp.prototype[Symbol.match]()` and `String.prototype.match()`.

Try it

```
const regexp = /foo/;
// console.log('/foo/'.startsWith(regexp));
// Expected output (Chrome): Error: First argument to String.prototype.startsWith must not be a regular expression
// Expected output (Firefox): Error: Invalid type: first can't be a Regular Expression
// Expected output (Safari): Error: Argument to String.prototype.startsWith cannot be a RegExp

regexp[Symbol.match] = false;

console.log("/foo/".startsWith(regexp));
// Expected output: true

console.log("/baz/".endsWith(regexp));
// Expected output: false
```

Value

The well-known symbol `Symbol.match`.

Property attributes of `Symbol.match`	
Writable	no
Enumerable	no
Configurable	no

Description

This function is also used to identify if objects have the behavior of regular expressions. For example, the methods `String.prototype.startsWith()`, `String.prototype.endsWith()` and `String.prototype.includes()`, check if their first argument is a regular expression and will throw a `TypeError` if they are. Now, if the `match` symbol is set to `false` (or a Falsy value except `undefined`), it indicates that the object is not intended to be used as a regular expression object.

Examples

Marking a RegExp as not a regex

The following code will throw a `TypeError`:

```
"/bar/".startsWith(/bar/);

// Throws TypeError, as /bar/ is a regular expression
// and Symbol.match is not modified.
```

However, if you set `Symbol.match` to `false`, the object will be considered as not a regular expression object. The methods `startsWith` and `endsWith` won't throw a `TypeError` as a consequence.

```
const re = /foo/;
re[Symbol.match] = false;
"/foo/".startsWith(re); // true
"/baz/".endsWith(re); // false
```

See also

- Polyfill of `Symbol.match` in `core-js`
- `Symbol.matchAll`
- `Symbol.replace`
- `Symbol.search`
- `Symbol.split`
- `String.prototype.match()`
- `RegExp.prototype[Symbol.match]()`

Symbol.matchAll

The **`Symbol.matchAll`** static data property represents the well-known symbol `Symbol.matchAll`. The `String.prototype.matchAll()` method looks up this symbol on its first argument for the method that returns an iterator, that yields matches of the current object against a string.

For more information, see `RegExp.prototype[Symbol.matchAll]()` and `String.prototype.matchAll()`.

Try it

```
const re = /\d+/g;
const str = "2016-01-02|2019-03-07";
```

```
const result = re[Symbol.matchAll](str);

console.log(Array.from(result, (x) => x[0]));
// Expected output: Array ["2016", "01", "02", "2019", "03", "07"]
```

Value

The well-known symbol `Symbol.matchAll`.

Property attributes of `Symbol.matchAll`	
Writable	no
Enumerable	no
Configurable	no

Examples

Using Symbol.matchAll

```
const str = "2016-01-02|2019-03-07";

const numbers = {
  *[Symbol.matchAll](str) {
    for (const n of str.matchAll(/\d+/g)) yield n[0];
  },
};

console.log(Array.from(str.matchAll(numbers)));
// ["2016", "01", "02", "2019", "03", "07"]
```

See also

- Polyfill of `Symbol.matchAll` in `core-js`
- es-shims polyfill of `Symbol.matchAll`
- `Symbol.match`
- `Symbol.replace`
- `Symbol.search`
- `Symbol.split`
- `String.prototype.matchAll()`
- `RegExp.prototype[Symbol.matchAll]()`

Symbol.replace

The **`Symbol.replace`** static data property represents the well-known symbol `Symbol.replace`. The `String.prototype.replace()` and `String.prototype.replaceAll()` methods look up this symbol on their first argument for the method that replaces substrings matched by the current object.

For more information, see `RegExp.prototype[Symbol.replace]()`, `String.prototype.replace()`, and `String.prototype.replaceAll()`.

Try it

```
class Replace1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.replace](string) {
    return `s/${string}/${this.value}/g`;
  }
}

console.log("foo".replace(new Replace1("bar")));
// Expected output: "s/foo/bar/g"
```

Value

The well-known symbol `Symbol.replace`.

Property attributes of `Symbol.replace`	
Writable	no
Enumerable	no
Configurable	no

Examples

Using Symbol.replace

```
class CustomReplacer {
  constructor(value) {
    this.value = value;
  }
  [Symbol.replace](string) {
```

```
    return string.replace(this.value, "#!@?");
  }
}

console.log("football".replace(new CustomReplacer("foo"))); // "#!@?tball"
```

See also

- Polyfill of `Symbol.replace` in `core-js`
- `Symbol.match`
- `Symbol.matchAll`
- `Symbol.search`
- `Symbol.split`
- `String.prototype.replace()`
- `String.prototype.replaceAll()`
- `RegExp.prototype[Symbol.replace]()`

Symbol.search

The **`Symbol.search`** static data property represents the well-known symbol `Symbol.search`. The `String.prototype.search()` method looks up this symbol on its first argument for the method that returns the index within a string that matches the current object.

For more information, see `RegExp.prototype[Symbol.search]()` and `String.prototype.search()`.

Try it

```
class Search1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.search](string) {
    return string.indexOf(this.value);
  }
}

console.log("foobar".search(new Search1("bar")));
// Expected output: 3
```

Value

The well-known symbol `Symbol.search`.

Property attributes of `Symbol.search`	
Writable	no
Enumerable	no
Configurable	no

Examples

Custom string search

```
class CaseInsensitiveSearch {
  constructor(value) {
    this.value = value.toLowerCase();
  }
  [Symbol.search](string) {
    return string.toLowerCase().indexOf(this.value);
  }
}

console.log("foobar".search(new CaseInsensitiveSearch("BaR"))); // 3
```

See also

- Polyfill of `Symbol.search` in `core-js`
- `Symbol.match`
- `Symbol.matchAll`
- `Symbol.replace`
- `Symbol.split`
- `String.prototype.search()`
- `RegExp.prototype[Symbol.search]()`

Symbol.species

The **`Symbol.species`** static data property represents the well-known symbol `Symbol.species`. Methods that create copies of an object may look up this symbol on the object for the constructor function to use when creating the copy.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much

> harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Try it

```
class Array1 extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}

const a = new Array1(1, 2, 3);
const mapped = a.map((x) => x * x);

console.log(mapped instanceof Array1);
// Expected output: false

console.log(mapped instanceof Array);
// Expected output: true
```

Value

The well-known symbol `Symbol.species`.

Property attributes of `Symbol.species`	
Writable	no
Enumerable	no
Configurable	no

Description

The `[Symbol.species]` accessor property allows subclasses to override the default constructor for objects. This specifies a protocol about how instances should be copied. For example, when you use copying methods of arrays, such as `map()`, the `map()` method uses `instance.constructor[Symbol.species]` to get the constructor for constructing the new array. For more information, see subclassing built-ins.

All built-in implementations of `[Symbol.species]` return the `this` value, which is the current instance's constructor. This allows copying methods to create instances of derived classes rather than the base class — for example, `map()` will return an array of the same type as the original array.

Examples

Using species

You might want to return `Array` objects in your derived array class `MyArray`. For example, when using methods such as `map()` that return the default constructor, you want these methods to return a parent `Array` object, instead of the `MyArray` object. The `species` symbol lets you do this:

```
class MyArray extends Array {
  // Overwrite species to the parent Array constructor
  static get [Symbol.species]() {
    return Array;
  }
}
const a = new MyArray(1, 2, 3);
const mapped = a.map((x) => x * x);

console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array); // true
```

See also

- `Array[Symbol.species]`
- `ArrayBuffer[Symbol.species]`
- `Map[Symbol.species]`
- `Promise[Symbol.species]`
- `RegExp[Symbol.species]`
- `Set[Symbol.species]`
- `TypedArray[Symbol.species]`

Symbol.split

The **`Symbol.split`** static data property represents the well-known symbol `Symbol.split`. The `String.prototype.split()` method looks up this symbol on its first argument for the method that splits a string at the indices that match the current object.

For more information, see `RegExp.prototype[Symbol.split]()` and `String.prototype.split()`.

Try it

```
class Split1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.split](string) {
```

```
    const index = string.indexOf(this.value);
    return `${this.value}${string.substring(0, index)}/${string.substring(
      index + this.value.length,
    )}`;
  }
}

console.log("foobar".split(new Split1("foo")));
// Expected output: "foo/bar"
```

Value

The well-known symbol `Symbol.split`.

Property attributes of `Symbol.split`	
Writable	no
Enumerable	no
Configurable	no

Examples

Custom reverse split

```
class ReverseSplit {
  [Symbol.split](string) {
    const array = string.split(" ");
    return array.reverse();
  }
}

console.log("Another one bites the dust".split(new ReverseSplit()));
// [ "dust", "the", "bites", "one", "Another" ]
```

See also

- Polyfill of `Symbol.split` in `core-js`
- `Symbol.match`
- `Symbol.matchAll`
- `Symbol.replace`
- `Symbol.search`
- `String.prototype.split()`
- `RegExp.prototype[Symbol.split]()`

Symbol() constructor

The **`Symbol()`** function returns primitive values of type Symbol.

Try it

```
const symbol1 = Symbol();
const symbol2 = Symbol(42);
const symbol3 = Symbol("foo");

console.log(typeof symbol1);
// Expected output: "symbol"

console.log(symbol2 === 42);
// Expected output: false

console.log(symbol3.toString());
// Expected output: "Symbol(foo)"

console.log(Symbol("foo") === Symbol("foo"));
// Expected output: false
```

Syntax

```
Symbol()
Symbol(description)
```

> **Note:** `Symbol()` can only be called without `new`. Attempting to construct it with `new` throws a `TypeError`.

Parameters

`description` Optional

A string. A description of the symbol which can be used for debugging but not to access the symbol itself.

Examples

Creating symbols

To create a new primitive symbol, you write `Symbol()` with an optional string as its description:

```
const sym1 = Symbol();
const sym2 = Symbol("foo");
const sym3 = Symbol("foo");
```

The above code creates three new symbols. Note that `Symbol("foo")` does not coerce the string `"foo"` into a symbol. It creates a new symbol each time:

```
Symbol("foo") === Symbol("foo"); // false
```

new Symbol()

The following syntax with the `new` operator will throw a `TypeError`:

```
const sym = new Symbol(); // TypeError
```

This prevents authors from creating an explicit `Symbol` wrapper object instead of a new symbol value and might be surprising as creating explicit wrapper objects around primitive data types is generally possible (for example, `new Boolean`, `new String` and `new Number`).

If you really want to create a `Symbol` wrapper object, you can use the `Object()` function:

```
const sym = Symbol("foo");
const symObj = Object(sym);
typeof sym; // "symbol"
typeof symObj; // "object"
```

See also

- Polyfill of `Symbol` in `core-js`

Symbol.prototype[Symbol.toPrimitive]()

The **`[Symbol.toPrimitive]()`** method of `Symbol` values returns this symbol value.

Syntax

```
symbolValue[Symbol.toPrimitive](hint)
```

Parameters

`hint`

A string value indicating the primitive value to return. The value is ignored.

Return value

The primitive value of the specified `Symbol` object.

Description

The `[Symbol.toPrimitive]()` method of `Symbol` returns the primitive value of a Symbol object as a Symbol data type. The `hint` argument is not used.

JavaScript calls the `[Symbol.toPrimitive]()` method to convert an object to a primitive value. You rarely need to invoke the `[Symbol.toPrimitive]()` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

Examples

Using <code>[Symbol.toPrimitive]()</code>

```
const sym = Symbol("example");
sym === sym[Symbol.toPrimitive](); // true
```

See also

- `Symbol.toPrimitive`

Symbol.toPrimitive

The **`Symbol.toPrimitive`** static data property represents the well-known symbol `Symbol.toPrimitive`. All type coercion algorithms look up this symbol on objects for the method that accepts a preferred type and returns a primitive representation of the object, before falling back to using the object's `valueOf()` and `toString()` methods.

Try it

```
const object = {
  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return 42;
    }
    return null;
  },
```

```
};

console.log(+object);
// Expected output: 42
```

Value

The well-known symbol `Symbol.toPrimitive`.

Property attributes of `Symbol.toPrimitive`	
Writable	no
Enumerable	no
Configurable	no

Description

With the help of the `Symbol.toPrimitive` property (used as a function value), an object can be converted to a primitive value. The function is called with a string argument `hint`, which specifies the preferred type of the result primitive value. The `hint` argument can be one of `"number"`, `"string"`, and `"default"`.

The `"number"` hint is used by numeric coercion algorithms. The `"string"` hint is used by the string coercion algorithm. The `"default"` hint is used by the primitive coercion algorithm. The `hint` only acts as a weak signal of preference, and the implementation is free to ignore it (as `Symbol.prototype[Symbol.toPrimitive]()` does). The language does not enforce alignment between the `hint` and the result type, although `[Symbol.toPrimitive]()` must return a primitive, or a `TypeError` is thrown.

Objects without the `[Symbol.toPrimitive]` property are converted to primitives by calling the `valueOf()` and `toString()` methods in different orders, which is explained in more detail in the type coercion section. `[Symbol.toPrimitive]()` allows full control over the primitive conversion process. For example, `Date.prototype[Symbol.toPrimitive]()` treats `"default"` as if it's `"string"` and calls `toString()` instead of `valueOf()`. `Symbol.prototype[Symbol.toPrimitive]()` ignores the hint and always returns a symbol, which means even in string contexts, `Symbol.prototype.toString()` won't be called, and `Symbol` objects must always be explicitly converted to strings through `String()`.

Examples

Modifying primitive values converted from an object

Following example describes how `Symbol.toPrimitive` property can modify the primitive value converted from an object.

```
// An object without Symbol.toPrimitive property.
const obj1 = {};
console.log(+obj1); // NaN
console.log(`${obj1}`); // "[object Object]"
console.log(obj1 + ""); // "[object Object]"

// An object with Symbol.toPrimitive property.
const obj2 = {
  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return 10;
    }
    if (hint === "string") {
      return "hello";
    }
    return true;
  },
};
console.log(+obj2); // 10        — hint is "number"
console.log(`${obj2}`); // "hello"   — hint is "string"
console.log(obj2 + ""); // "true"    — hint is "default"
```

See also

- Polyfill of `Symbol.toPrimitive` in `core-js`
- `Date.prototype[Symbol.toPrimitive]()`
- `Symbol.prototype[Symbol.toPrimitive]()`
- `Object.prototype.toString()`
- `Object.prototype.valueOf()`

Symbol.prototype.toString()

The **`toString()`** method of `Symbol` values returns a string representing this symbol value.

Try it

```
console.log(Symbol("desc").toString());
// Expected output: "Symbol(desc)"

console.log(Symbol.iterator.toString());
// Expected output: "Symbol(Symbol.iterator)

console.log(Symbol.for("foo").toString());
// Expected output: "Symbol(foo)"
```

```
// console.log(Symbol('foo') + 'bar');
// Expected output: Error: Can't convert symbol to string
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the specified symbol value.

Description

The `Symbol` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `Symbol` values, the `toString` method returns a descriptive string in the form `"Symbol(description)"`, where `description` is the symbol's description.

The `toString()` method requires its `this` value to be a `Symbol` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to symbol values.

Because `Symbol` has a `[Symbol.toPrimitive]()` method, that method always takes priority over `toString()` when a `Symbol` object is coerced to a string. However, because `Symbol.prototype[Symbol.toPrimitive]()` returns a symbol primitive, and symbol primitives throw a `TypeError` when implicitly converted to a string, the `toString()` method is never implicitly called by the language. To stringify a symbol, you must explicitly call its `toString()` method or use the `String()` function.

Examples

Using toString()

```
Symbol("desc").toString(); // "Symbol(desc)"

// well-known symbols
Symbol.iterator.toString(); // "Symbol(Symbol.iterator)"

// global symbols
Symbol.for("foo").toString(); // "Symbol(foo)"
```

Implicitly calling toString()

The only way to make JavaScript implicitly call `toString()` instead of `[Symbol.toPrimitive]()` on a symbol wrapper object is by deleting the `[Symbol.toPrimitive]()` method first.

> **Warning:** You should not do this in practice. Never mutate built-in objects unless you know exactly what you're doing.

```
delete Symbol.prototype[Symbol.toPrimitive];
console.log(`${Object(Symbol("foo"))}`); // "Symbol(foo)"
```

See also

- `Object.prototype.toString()`

Symbol.toStringTag

The **`Symbol.toStringTag`** static data property represents the well-known symbol `Symbol.toStringTag`. `Object.prototype.toString()` looks up this symbol on the `this` value for the property containing a string that represents the type of the object.

Try it

```
class ValidatorClass {
  get [Symbol.toStringTag]() {
    return "Validator";
  }
}

console.log(Object.prototype.toString.call(new ValidatorClass()));
// Expected output: "[object Validator]"
```

Value

The well-known symbol `Symbol.toStringTag`.

Property attributes of `Symbol.toStringTag`	
Writable	no
Enumerable	no
Configurable	no

Examples

Default tags

Some values do not have `Symbol.toStringTag`, but have special `toString()` representations. For a complete list, see `Object.prototype.toString()`.

```
Object.prototype.toString.call("foo"); // "[object String]"
Object.prototype.toString.call([1, 2]); // "[object Array]"
Object.prototype.toString.call(3); // "[object Number]"
Object.prototype.toString.call(true); // "[object Boolean]"
Object.prototype.toString.call(undefined); // "[object Undefined]"
Object.prototype.toString.call(null); // "[object Null]"
// … and more
```

Built-in toStringTag symbols

Most built-in objects provide their own `[Symbol.toStringTag]` property. Almost all built-in objects' `[Symbol.toStringTag]` property is not writable, not enumerable, and configurable; the exception is `Iterator`, which is writable for compatibility reasons.

For constructor objects like `Promise`, the property is installed on `Constructor.prototype`, so that all instances of the constructor inherit `[Symbol.toStringTag]` and can be stringified. For non-constructor objects like `Math` and `JSON`, the property is installed as a static property, so that the namespace object itself can be stringified. Sometimes, the constructor also provides its own `toString` method (for example, `Intl.Locale`), in which case the `[Symbol.toStringTag]` property is only used when you explicitly call `Object.prototype.toString` on it.

```
Object.prototype.toString.call(new Map()); // "[object Map]"
Object.prototype.toString.call(function* () {}); // "[object GeneratorFunction]"
Object.prototype.toString.call(Promise.resolve()); // "[object Promise]"
// … and more
```

Custom tag with toStringTag

When creating your own class, JavaScript defaults to the "Object" tag:

```
class ValidatorClass {}

Object.prototype.toString.call(new ValidatorClass()); // "[object Object]"
```

Now, with the help of `toStringTag`, you are able to set your own custom tag:

```
class ValidatorClass {
  get [Symbol.toStringTag]() {
    return "Validator";
  }
}

Object.prototype.toString.call(new ValidatorClass()); // "[object Validator]"
```

toStringTag available on all DOM prototype objects

Due to a WebIDL spec change in mid-2020, browsers are adding a `Symbol.toStringTag` property to all DOM prototype objects. For example, to access the `Symbol.toStringTag` property on `HTMLButtonElement`:

```
const test = document.createElement("button");
test.toString(); // "[object HTMLButtonElement]"
test[Symbol.toStringTag]; // "HTMLButtonElement"
```

See also

- Polyfill of `Symbol.toStringTag` in `core-js`
- `Object.prototype.toString()`

Symbol.unscopables

The **`Symbol.unscopables`** static data property represents the well-known symbol `Symbol.unscopables`. The `with` statement looks up this symbol on the scope object for a property containing a collection of properties that should not become bindings within the `with` environment.

Try it

```
const object = {
  foo: 42,
};

object[Symbol.unscopables] = {
  foo: true,
};

with (object) {
  console.log(foo);
  // Expected output: Error: foo is not defined
}
```

Value

The well-known symbol `Symbol.unscopables`.

Property attributes of `Symbol.unscopables`	
Writable	no
Enumerable	no
Configurable	no

Description

The `[Symbol.unscopables]` symbol (accessed via `Symbol.unscopables`) can be defined on any object to exclude property names from being exposed as lexical variables in `with` environment bindings. Note that when using strict mode, `with` statements are not available, and this symbol is likely not needed.

Setting a property of the `[Symbol.unscopables]` object to `true` (or any truthy value) will make the corresponding property of the `with` scope object *unscopable* and therefore won't be introduced to the `with` body scope. Setting a property to `false` (or any falsy value) will make it *scopable* and thus appear as lexical scope variables.

When deciding whether `x` is unscopable, the entire prototype chain of the `[Symbol.unscopables]` property is looked up for a property called `x`. This means if you declared `[Symbol.unscopables]` as a plain object, `Object.prototype` properties like `toString` would become unscopable as well, which may cause backward incompatibility for legacy code assuming those properties are normally scoped (see an example below). You are advised to make your custom `[Symbol.unscopables]` property have `null` as its prototype, like `Array.prototype[Symbol.unscopables]` does.

This protocol is also utilized by DOM APIs, such as `Element.prototype.append()`.

Examples

Scoping in with statements

The following code works fine in ES5 and below. However, in ECMAScript 2015, the `Array.prototype.values()` method was introduced. That means that inside a `with` environment, "values" would now be the `Array.prototype.values()` method and not the variable outside the `with` statement.

```
var values = [];

with (values) {
  // If [Symbol.unscopables] did not exist, values would become
  // Array.prototype.values starting with ECMAScript 2015.
  // And an error would have occurred.
  values.push("something");
}
```

The code containing `with (values)` caused some websites to malfunction in Firefox when `Array.prototype.values()` was added (Firefox Bug 883914). Furthermore, this implies that any future array method addition may be breaking if it implicitly changes the `with` scope. Therefore, the `[Symbol.unscopables]` symbol was introduced and implemented on `Array` as `Array.prototype[Symbol.unscopables]` to prevent some of the Array methods being scoped into the `with` statement.

Unscopables in objects

You can also set `[Symbol.unscopables]` for your own objects.

```
const obj = {
  foo: 1,
  bar: 2,
  baz: 3,
};

obj[Symbol.unscopables] = {
  // Make the object have `null` prototype to prevent
  // `Object.prototype` methods from being unscopable
  __proto__: null,
  // `foo` will be scopable
  foo: false,
  // `bar` will be unscopable
  bar: true,
  // `baz` is omitted; because `undefined` is falsy, it is also scopable (default)
};

with (obj) {
  console.log(foo); // 1
  console.log(bar); // ReferenceError: bar is not defined
  console.log(baz); // 3
}
```

Avoid using a non-null-prototype object as <code>[Symbol.unscopables]</code>

Declaring `[Symbol.unscopables]` as a plain object without eliminating its prototype may cause subtle bugs. Consider the following code working before `[Symbol.unscopables]`:

```
const character = {
  name: "Yoda",
  toString: function () {
    return "Use with statements, you must not";
  },
```

```
};

with (character) {
  console.log(name + ' says: "' + toString() + '"'); // Yoda says: "Use with statements, you must not"
}
```

To preserve backward compatibility, you decided to add a `[Symbol.unscopables]` property when adding more properties to `character`. You may naïvely do it like:

```
const character = {
  name: "Yoda",
  toString: function () {
    return "Use with statements, you must not";
  },
  student: "Luke",
  [Symbol.unscopables]: {
    // Make `student` unscopable
    student: true,
  },
};
```

However, the code above now breaks:

```
with (character) {
  console.log(name + ' says: "' + toString() + '"'); // Yoda says: "[object Undefined]"
}
```

This is because when looking up `character[Symbol.unscopables].toString`, it returns `Object.prototype.toString()`, which is a truthy value, thus making the `toString()` call in the `with()` statement reference `globalThis.toString()` instead — and because it's called without a `this`, `this` is `undefined`, making it return `[object Undefined]`.

Even when the method is not overridden by `character`, making it unscopable will change the value of `this`.

```
const proto = {};
const obj = { __proto__: proto };

with (proto) {
  console.log(isPrototypeOf(obj)); // true; `isPrototypeOf` is scoped and `this` is `proto`
}

proto[Symbol.unscopables] = {};

with (proto) {
  console.log(isPrototypeOf(obj)); // TypeError: Cannot convert undefined or null to object
  // `isPrototypeOf` is unscoped and `this` is undefined
}
```

To fix this, always make sure `[Symbol.unscopables]` only contains properties you wish to be unscopable, without `Object.prototype` properties.

```
const character = {
  name: "Yoda",
  toString: function () {
    return "Use with statements, you must not";
  },
  student: "Luke",
  [Symbol.unscopables]: {
    // Make the object have `null` prototype to prevent
    // `Object.prototype` methods from being unscopable
    __proto__: null,
    // Make `student` unscopable
    student: true,
  },
};
```

See also

- `Array.prototype[Symbol.unscopables]`
- `with`
- `Element.prototype.append()`

Symbol.prototype.valueOf()

The **`valueOf()`** method of `Symbol` values returns this symbol value.

Try it

```
const symbol = Symbol("foo");

console.log(typeof Object(symbol));
// Expected output: "object"

console.log(typeof Object(symbol).valueOf());
// Expected output: "symbol"
```

Syntax

```
valueOf()
```

Parameters

None.

Return value

The primitive value of the specified `Symbol` object.

Description

The `valueOf()` method of `Symbol` returns the primitive value of a Symbol object as a Symbol data type.

JavaScript calls the `valueOf()` method to convert an object to a primitive value. You rarely need to invoke the `valueOf()` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

Examples

Using valueOf()

```
const sym = Symbol("example");
sym === sym.valueOf(); // true
```

See also

- `Object.prototype.valueOf()`

SyntaxError

The **`SyntaxError`** object represents an error when trying to interpret syntactically invalid code. It is thrown when the JavaScript engine encounters tokens or token order that does not conform to the syntax of the language when parsing code.

`SyntaxError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`SyntaxError` is a subclass of `Error`.

Constructor

`SyntaxError()`

Creates a new `SyntaxError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `SyntaxError.prototype` and shared by all `SyntaxError` instances.

`SyntaxError.prototype.constructor`

The constructor function that created the instance object. For `SyntaxError` instances, the initial value is the `SyntaxError` constructor.

`SyntaxError.prototype.name`

Represents the name for the type of error. For `SyntaxError.prototype.name`, the initial value is `"SyntaxError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a SyntaxError

```
try {
  eval("hoo bar");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message);
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

Creating a SyntaxError

```
try {
  throw new SyntaxError("Hello");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "SyntaxError"
```

```
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`

SyntaxError() constructor

The **`SyntaxError()`** constructor creates `SyntaxError` objects.

Syntax

```
new SyntaxError()
new SyntaxError(message)
new SyntaxError(message, options)
new SyntaxError(message, fileName)
new SyntaxError(message, fileName, lineNumber)

SyntaxError()
SyntaxError(message)
SyntaxError(message, options)
SyntaxError(message, fileName)
SyntaxError(message, fileName, lineNumber)
```

> **Note:** `SyntaxError()` can be called with or without `new`. Both create a new `SyntaxError` instance.

Parameters

`message` Optional

Human-readable description of the error

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception

`lineNumber` Optional

The line number of the code that caused the exception

Examples

Catching a SyntaxError

```
try {
  eval("hoo bar");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message);
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

Creating a SyntaxError

```
try {
  throw new SyntaxError("Hello");
} catch (e) {
  console.log(e instanceof SyntaxError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "SyntaxError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`

Temporal

The **`Temporal`** object enables date and time management in various scenarios, including built-in time zone and calendar representation, wall-clock time conversions, arithmetics, formatting, and more. It is designed as a full replacement for the `Date` object.

Description

Unlike most global objects, `Temporal` is not a constructor. You cannot use it with the `new` operator or invoke the `Temporal` object as a function. All properties and methods of `Temporal` are static (just like the `Math` object).

`Temporal` has an intricate and powerful API. It exposes over 200 utility methods via several classes, so it could appear very complex. We will provide a high-level overview of how these APIs are related to each other.

Background and concepts

JavaScript has had the `Date` object for handling date and time since its first days. However, the `Date` API is based on the poorly designed `java.util.Date` class from Java, which was replaced in the early 2010s; but, because of JavaScript's goal of backward compatibility, `Date` sticks around in the language.

The important lesson to preface the whole introduction is that **date handling is complex**. Most of the problems of `Date` are fixable by adding more methods, but a fundamental design flaw remains: it exposes so many methods on the same object that developers are often confused about what to use, leading to unexpected pitfalls. A well-designed API not only needs to do more, but also should do *less* with each level of abstraction, because preventing misuse is as important as enabling use cases.

`Date` objects wear two hats simultaneously:

- As a timestamp: the number of milliseconds or nanoseconds elapsed since a fixed point in time (known as the *epoch*).
- As a combination of components: year, month, day, hour, minute, second, millisecond, and nanosecond. The year, month, and day identifiers only make sense with reference to a *calendar system*. The whole combination maps to a unique instant in history when associated with a time zone. `Date` objects provide methods for reading and modifying these components.

Time zones underlie a significant number of date-related bugs. When interacting with a `Date` via the "combination of components" model, the time can only be in two time zones: UTC and local (device), and there's no way to specify an arbitrary time zone. Also lacking is the concept of "no time zone": this is known as a *calendar date* (for dates) or *wall-clock time* (for times), which is a time you "read off a calendar or clock". For example, if you are setting a daily wake up alarm, you will want to set it to "8:00AM" regardless of whether it is daylight saving time or not, whether you have traveled to a different time zone, etc.

A second feature lacking from `Date` is a calendar system. Most people may be familiar with the Gregorian calendar, where there are two eras, BC and AD; there are 12 months; each month has a different number of days; there's a leap year every 4 years; and so on. However, some of these concepts may not apply when you are working with another calendar system, such as the Hebrew calendar, the Chinese calendar, the Japanese calendar, etc. With `Date`, you can only work with the Gregorian calendar model.

There are many other undesirable legacies about `Date`, such as all setters being mutating (which often causes unwanted side effects), the date time string format being impossible to parse in a consistent way, etc. In the end, the best solution is to build a new API from scratch, which is what `Temporal` is.

API overview

`Temporal` is a namespace, like `Intl`. It contains several classes and namespaces, each of which is designed to handle a specific aspect of date and time management. The classes can be grouped as such:

- Representing a time duration (a difference between two time points): `Temporal.Duration`
- Representing a time point:
  - Representing a unique instant in history:
    - As a timestamp: `Temporal.Instant`
    - As a date-time component combination paired with a time zone: `Temporal.ZonedDateTime`
  - Representing a time-zone-unaware date/time (which are all prefixed with "Plain"):
    - Date (year, month, day) + time (hour, minute, second, millisecond, microsecond, nanosecond): `Temporal.PlainDateTime` (Note: `ZonedDateTime` is equivalent to `PlainDateTime` plus a time zone)
      - Date (year, month, day): `Temporal.PlainDate`
        - Year, month: `Temporal.PlainYearMonth`
        - Month, day: `Temporal.PlainMonthDay`
      - Time (hour, minute, second, millisecond, microsecond, nanosecond): `Temporal.PlainTime`

Furthermore, there's also another utility namespace, `Temporal.Now`, which provides methods for getting the current time in various formats.

Shared class interface

There are many classes in the `Temporal` namespace, but they share many similar methods. The following table lists all methods of each class (except conversion methods):

	`Instant`	`ZonedDateTime`	`PlainDateTime`	`PlainDate`	`PlainTime`	`Plain`

Construction	`Instant()`
	`Instant.from()`
	`Instant.fromEpochMilliseconds()`
	`Instant.fromEpochNanoseconds()`
		`ZonedDateTime()`
		`ZonedDateTime.from()`
			`PlainDateTime()`
			`PlainDateTime.from()`
				`PlainDate()`
				`PlainDate.from()`
					`PlainTime()`
					`PlainTime.from()`
						`PlainYearM`
						`PlainYearM`

Updater	N/A	`with()`
		`withCalendar()`
		`withTimeZone()`
		`withPlainTime()`
			`with()`
			`withCalendar()`
			`withPlainTime()`
				`with()`
				`withCalendar()`
					`with()`	`with()`

Arithmetic	`add()`
	`subtract()`
	`since()`
	`until()`
		`add()`
		`subtract()`
		`since()`
		`until()`
			`add()`
			`subtract()`
			`since()`
			`until()`
				`add()`
				`subtract()`
				`since()`
				`until()`
					`add()`
					`subtract()`
					`since()`
					`until()`
						`add()`
						`subtract()`
						`since()`
						`until()`

Rounding	`round()`	`round()`	`round()`	N/A	`round()`	N/A

	Instant	ZonedDateTime	PlainDateTime	PlainDate	PlainTime	Plain
Comparison	`equals()` `Instant.compare()`	`equals()` `ZonedDateTime.compare()`	`equals()` `PlainDateTime.compare()`	`equals()` `PlainDate.compare()`	`equals()` `PlainTime.compare()`	`equals()` `PlainYearM`

Serialization	`toJSON()`
	`toLocaleString()`
	`toString()`
	`valueOf()`
		`toJSON()`
		`toLocaleString()`
		`toString()`
		`valueOf()`
			`toJSON()`
			`toLocaleString()`
			`toString()`
			`valueOf()`
				`toJSON()`
				`toLocaleString()`
				`toString()`
				`valueOf()`
					`toJSON()`
					`toLocaleString()`
					`toString()`
					`valueOf()`
						`toJSON()`
						`toLocaleSt`
						`toString()`
						`valueOf()`

The following table summarizes which properties are available on each class, giving you a sense of what information each class can represent.

	Instant	ZonedDateTime	PlainDateTime	PlainDate	PlainTime	PlainYearMonth	PlainMonthDay
Calendar	N/A	`calendarId`	`calendarId`	`calendarId`	N/A	`calendarId`	`calendarId`

Year-related	N/A	`era`
		`eraYear`
		`year`
		`inLeapYear`
		`monthsInYear`
		`daysInYear`
			`era`
			`eraYear`
			`year`
			`inLeapYear`
			`monthsInYear`
			`daysInYear`
				`era`
				`eraYear`
				`year`
				`inLeapYear`
				`monthsInYear`
				`daysInYear`
					N/A	`era`
						`eraYear`
						`year`
						`inLeapYear`
						`monthsInYear`
						`daysInYear`
							N/A

Month-related	N/A	`month`
		`monthCode`
		`daysInMonth`
			`month`
			`monthCode`
			`daysInMonth`
				`month`
				`monthCode`
				`daysInMonth`
					N/A	`month`
						`monthCode`
						`daysInMonth`
							`monthCode`

Week-related	N/A	`weekOfYear`
		`yearOfWeek`
		`daysInWeek`
			`weekOfYear`
			`yearOfWeek`
			`daysInWeek`
				`weekOfYear`
				`yearOfWeek`
				`daysInWeek`
					N/A	N/A	N/A

Day-related	N/A	`day`
		`dayOfWeek`
		`dayOfYear`
			`day`
			`dayOfWeek`
			`dayOfYear`
				`day`
				`dayOfWeek`
				`dayOfYear`
					N/A	N/A	`day`

Time components	N/A	`hour`
		`minute`
		`second`
		`millisecond`
		`microsecond`
		`nanosecond`
			`hour`
			`minute`
			`second`
			`millisecond`
			`microsecond`
			`nanosecond`
				N/A	`hour`
					`minute`
					`second`
					`millisecond`
					`microsecond`
					`nanosecond`
						N/A	N/A

Time zone	N/A	`timeZoneId`
		`offset`
		`offsetNanoseconds`
		`hoursInDay`
		`getTimeZoneTransition()`
		`startOfDay()`
			N/A	N/A	N/A	N/A	N/A

Epoch time	`epochMilliseconds` `epochNanoseconds`	`epochMilliseconds` `epochNanoseconds`	N/A	N/A	N/A	N/A	N/A

Conversion between classes

The table below summarizes all conversion methods that exist on each class.

		How to convert from...						
		Instant	**ZonedDateTime**	**PlainDateTime**	**PlainDate**	**PlainTime**	**PlainYearMonth**	**Plai**
to...	**Instant**	/	`toInstant()`	Convert to `ZonedDateTime` first				
	ZonedDateTime	`toZonedDateTimeISO()`	/	`toZonedDateTime()`	`toZonedDateTime()`	`PlainDate#toZonedDateTime()` (pass as argument)	Convert to `PlainDate`	
	PlainDateTime	Convert to `ZonedDateTime` first	`toPlainDateTime()`	/	`toPlainDateTime()`	`PlainDate#toPlainDateTime()` (pass as argument)		
	PlainDate		`toPlainDate()`	`toPlainDate()`	/	No overlap in information	`toPlainDate()`	`toPl`
	PlainTime		`toPlainTime()`	`toPlainTime()`	No overlap in information	/	No overlap in informa	
	PlainYearMonth		Convert to `PlainDate` first		`toPlainYearMonth()`	No overlap in information	/	Con `Plai` first
	PlainMonthDay				`toPlainMonthDay()`		Convert to `PlainDate` first	/

With these tables, you should have a basic idea of how to navigate the `Temporal` API.

Calendars

A calendar is a way to organize days, typically into periods of weeks, months, years, and eras. Most of the world uses the Gregorian calendar, but there are many other calendars in use, especially in religious and cultural contexts. By default, all calendar-aware `Temporal` objects use the ISO 8601 calendar system, which is based on the

Gregorian calendar and defines additional week-numbering rules. `Intl.supportedValuesOf()` lists most of the calendars likely to be supported by browsers. Here we provide a brief overview of how calendar systems are formed to help you internalize what factors may vary between calendars.

There are three prominent periodic events on Earth: its rotation around the sun (365.242 days for one revolution), the moon's rotation around the Earth (29.53 days from new moon to new moon), and its rotation around its axis (24 hours from sunrise to sunrise). Every culture has the same measure of a "day", which is 24 hours. Occasional changes such as daylight saving time are not part of the calendar, but are part of the time zone's information.

- Some calendars primarily define one year as 365.242 days on average, by defining years to have 365 days, and adding an extra day, the *leap day*, about every 4 years. Then, the year may be further divided into parts called months. These calendars are called *solar calendars*. The Gregorian calendar and the Solar Hijri calendar are solar calendars.
- Some calendars primarily define one month as 29.5 days on average, by defining months to alternate between 29 and 30 days. Then, 12 months may be grouped into a year of 354 days. These calendars are called *lunar calendars*. The Islamic calendar is a lunar calendar. Because a lunar year is artificial and does not correlate with the season cycle, lunar calendars are generally rarer.
- Some calendars also primarily define months based on lunar cycles, like lunar calendars. Then, to compensate for the 11-day discrepancy with the solar year, an extra month, the *leap month*, is added about every 3 years. These calendars are called *lunisolar calendars*. The Hebrew calendar and the Chinese calendar are lunisolar calendars.

In `Temporal`, every date under one calendar system is uniquely identified by three components: `year`, `month`, and `day`. While `year` is typically a positive integer, it may also be zero or negative, and increases monotonically with time. The year `1` (or `0`, if it exists) is known as the calendar epoch, and is arbitrary for each calendar. `month` is a positive integer that increments by 1 every time, starting at `1` and ending at `date.monthsInYear`, then resetting back to `1` as the year advances. `day` is also a positive integer, but it may not start at 1 or increment by 1 every time, because political changes may cause days to be skipped or repeated. But in general, `day` monotonically increases and resets as the month advances.

In addition to `year`, a year can also be uniquely identified by the combination of `era` and `eraYear`, for calendars that use eras. For example, the Gregorian calendar uses the era "CE" (Common Era) and "BCE" (Before Common Era), and the year `-1` is the same as `{ era: "bce", eraYear: 2 }` (note that year `0` always exists for all calendars; for the Gregorian calendar, it corresponds to 1 BCE due to astronomical year numbering). `era` is a lowercase string, and `eraYear` is an arbitrary integer that may be zero or negative, or even decrease with time (usually for the oldest era).

> **Note:** Always use `era` and `eraYear` as a pair; don't use one property without also using the other. In addition, to avoid conflicts, don't combine `year` and `era`/`eraYear` when designating a year. Pick one year representation and use it consistently.
>
> Be careful of the following incorrect assumptions about years:
>
> - Don't assume that `era` and `eraYear` are always present; they may be `undefined`.
> - Don't assume that `era` is a user-friendly string; use `toLocaleString()` to format your date instead.
> - Don't assume that two `year` values from different calendars are comparable; use the `compare()` static method instead.
> - Don't assume that years have 365/366 days and 12 months; use `daysInYear` and `monthsInYear` instead.
> - Don't assume that leap years (`inLeapYear` is `true`) have one extra day; they may have an extra month.

In addition to `month`, a month in a year can also be uniquely identified by the `monthCode`. `monthCode` usually maps to the month's name, but `month` does not. For example, in the case of lunisolar calendars, two months with the same `monthCode`, where one belongs to a leap year and the other one does not, will have different `month` values if they come after the leap month, due to the insertion of an extra month.

> **Note:** To avoid conflicts, don't combine `month` and `monthCode` when designating a month. Pick one month representation and use it consistently. `month` is more useful if you need the order of months in a year (e.g., when looping through the months), while `monthCode` is more useful if you need the name of the month (e.g., when storing birthdays).
>
> Be careful of the following incorrect assumptions about months:
>
> - Don't assume that `monthCode` and `month` always correspond.
> - Don't assume the number of days in a month; use `daysInMonth` instead.
> - Don't assume that `monthCode` is a user-friendly string; use `toLocaleString()` to format your date instead.
> - Generally, don't cache the name of months in an array or object. Even though `monthCode` usually maps to the month's name within one calendar, we recommend always computing the month's name using, for example, `date.toLocaleString("en-US", { calendar: date.calendarId, month: "long" })`.

In addition to `day` (which is a month-based index), a day in a year can also be uniquely identified by the `dayOfYear`. `dayOfYear` is a positive integer that increments by 1 every time, starting at `1` and ending at `date.daysInYear`.

The concept of a "week" is not connected with any astronomical event, but is a cultural construct. While the most common length is `7` days, weeks can also have 4, 5, 6, 8, or more days — or even lack a fixed number of days altogether. To get the specific number of days of the week of a date, use the date's `daysInWeek`. `Temporal` identifies weeks by the combination of `weekOfYear` and `yearOfWeek`. `weekOfYear` is a positive integer that increments by 1 every time, starting at `1`, then resetting back to `1` as the year advances. `yearOfWeek` is generally the same as `year`, but may be different at the start or end of each year, because one week may cross two years, and `yearOfWeek` picks one of the two years based on the calendar's rules.

> **Note:** Always use `weekOfYear` and `yearOfWeek` as a pair; don't use `weekOfYear` and `year`.
>
> Be careful of the following incorrect assumptions about weeks:
>
> - Don't assume that `weekOfYear` and `yearOfWeek` are always present; they may be `undefined`.
> - Don't assume that weeks are always 7 days long; use `daysInWeek` instead.
> - Note that the current `Temporal` API does not support year-week dates, so you can't construct dates using these properties or serialize dates to year-week representations. They are only informational properties.

RFC 9557 format

All `Temporal` classes can be serialized and deserialized using the format specified in RFC 9557, which is based on ISO 8601 / RFC 3339. The format, in its full form, is as follows (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss Z/±HH:mm [time_zone_id] [u-ca=calendar_id]
```

Different classes have different requirements for the presence of each component, so you will find a section titled "RFC 9557 format" in each class's documentation, which specifies the format recognized by that class.

This is very similar to the date time string format used by `Date`, which is also based on ISO 8601. The main addition is the ability to specify micro- and nanosecond components, and the ability to specify the time zone and calendar system.

Representable dates

All `Temporal` objects that represent a specific calendar date impose a similar limit on the range of representable dates, which is $\pm 10^8$ days (inclusive) from the Unix epoch, or the range of instants from `-271821-04-20T00:00:00` to `+275760-09-13T00:00:00`. This is the same range as valid dates. More specifically:

- `Temporal.Instant` and `Temporal.ZonedDateTime` apply this limit directly on its `epochNanoseconds` value.
- `Temporal.PlainDateTime` interprets the date-time in the UTC time zone and requires it to be $\pm(10^8 + 1)$ days (exclusive) from the Unix epoch, so its valid range is `-271821-04-19T00:00:00` to `+275760-09-14T00:00:00`, exclusive. This allows any `ZonedDateTime` to be converted to a `PlainDateTime` regardless of its offset.
- `Temporal.PlainDate` applies the same check as `PlainDateTime` to the noon (`12:00:00`) of that date, so its valid range is `-271821-04-19` to `+275760-09-13`. This allows any `PlainDateTime` to be converted to a `PlainDate` regardless of its time, and vice versa.
- `Temporal.PlainYearMonth` has the valid range of `-271821-04` to `+275760-09`. This allows any `PlainDate` to be converted to a `PlainYearMonth` regardless of its date (except if a non-ISO month's first day falls in the ISO month `-271821-03`).

The `Temporal` objects will refuse to construct an instance representing a date/time beyond this limit. This includes:

- Using the constructor or `from()` static method.
- Using the `with()` method to update calendar fields.
- Using `add()`, `subtract()`, `round()`, or any other method to derive new instances.

Static properties

`Temporal.Duration`

> Represents a difference between two time points, which can be used in date/time arithmetic. It is fundamentally represented as a combination of years, months, weeks, days, hours, minutes, seconds, milliseconds, microseconds, and nanoseconds values.

`Temporal.Instant`

> Represents a unique point in time, with nanosecond precision. It is fundamentally represented as the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC), without any time zone or calendar system.

`Temporal.Now`

> Provides methods for getting the current time in various formats.

`Temporal.PlainDate`

> Represents a calendar date (a date without a time or time zone); for example, an event on a calendar which happens during the whole day no matter which time zone it's happening in. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system.

`Temporal.PlainDateTime`

> Represents a date (calendar date) and time (wall-clock time) without a time zone. It is fundamentally represented as a combination of a date (with an associated calendar system) and a time.

`Temporal.PlainMonthDay`

> Represents the month and day of a calendar date, without a year or time zone; for example, an event on a calendar that recurs every year and happens during the whole day. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The year is used to disambiguate the month-day in non-ISO calendar systems.

`Temporal.PlainTime`

> Represents a time without a date or time zone; for example, a recurring event that happens at the same time every day. It is fundamentally represented as a combination of hour, minute, second, millisecond, microsecond, and nanosecond values.

`Temporal.PlainYearMonth`

> Represents the year and month of a calendar date, without a day or time zone; for example, an event on a calendar that happens during the whole month. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The day is used to disambiguate the year-month in non-ISO calendar systems.

`Temporal.ZonedDateTime`

> Represents a date and time with a time zone. It is fundamentally represented as a combination of an instant, a time zone, and a calendar system.

`Temporal[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal"`. This property is used in `Object.prototype.toString()`.

See also

- `Intl.DateTimeFormat`
- `Intl.RelativeTimeFormat`
- `Intl.DurationFormat`
- Temporal polyfill by proposal champions
- Temporal polyfill by FullCalendar

Temporal.Duration

The **`Temporal.Duration`** object represents a difference between two time points, which can be used in date/time arithmetic. It is fundamentally represented as a combination of years, months, weeks, days, hours, minutes, seconds, milliseconds, microseconds, and nanoseconds values.

Description

ISO 8601 duration format

`Duration` objects can be serialized and parsed using the ISO 8601 duration format (with some extensions specified by ECMAScript). The string has the following form (spaces are only for readability and should not be present in the actual string):

```
±P nY nM nW nD T nH nM nS
```

`±` Optional

An optional sign character (`+` or `-`), which represents positive or negative duration. Default is positive.

`P`

A literal character `P` or `p` that stands for "period".

`nY`, `nM`, `nW`, `nD`, `nH`, `nM`, `nS`

A number followed by a literal character, which represents the number of years (`Y`), months (`M`), weeks (`W`), days (`D`), hours (`H`), minutes (`M`), or seconds (`S`), respectively. All except the last existing component must be an integer. The last component, if it is a time component (hours, minutes, or seconds), may have a fractional part of 1 to 9 digits, led by a dot or a comma, such as `PT0.0021S` or `PT1.1H`. Any zero components may be omitted, but at least one component should be present (even if it has value zero, in which case the duration is zero).

`T`

A literal character `T` or `t` that separates the date part from the time part, which should be present if and only if there's at least one component after it.

Here are some examples:

ISO 8601	Meaning
`P1Y1M1DT1H1M1.1S`	1 year, 1 month, 1 day, 1 hour, 1 minute, 1 second, and 100 milliseconds
`P40D`	40 days
`P1Y1D`	1 year and 1 day
`P3DT4H59M`	3 days, 4 hours and 59 minutes
`PT2H30M`	2 hours and 30 minutes
`P1M`	1 month
`PT1M`	1 minute
`PT0.0021S`	2.1 milliseconds (2 milliseconds and 100 microseconds)
`PT0S`	Zero (canonical representation)
`P0D`	Zero

> **Note:** According to the ISO 8601-1 standard, weeks are not allowed to appear together with any other units, and durations can only be positive. As extensions to the standard, ISO 8601-2, which Temporal uses, allows a sign character at the start of the string, and allows combining weeks with other units. Therefore, if your duration is serialized to a string like `P3W1D`, `+P1M`, or `-P1M`, note that other programs may not accept it.

When serializing, the output respects the stored components as much as possible, preserving unbalanced components. However, subsecond components are serialized as a single fractional second, so their precise values, if unbalanced may be lost. The plus sign is omitted for positive durations. The zero duration is always serialized as `PT0S`.

Calendar durations

A *calendar duration* is one that contains any of the calendar units: weeks, months, and years. A non-calendar duration is portable and can participate in date/time arithmetic without any calendar information, because they unambiguously represent a fixed amount of time. However, a calendar duration is not portable because the number of days in a month or year depends on the calendar system and the reference time point. Therefore, attempting to perform any arithmetic operation on a calendar durations throws an error because durations don't keep track of a calendar themselves. For example, if we are in May of the Gregorian calendar, then "1 month" is "31 days", but if we are in April, then "1 month" becomes "30 days". To add or subtract calendar durations, you need to add them to dates instead:

```
const dur1 = Temporal.Duration.from({ years: 1 });
const dur2 = Temporal.Duration.from({ months: 1 });

dur1.add(dur2); // RangeError: for calendar duration arithmetic, use date arithmetic relative to a starting point
```

```
const startingPoint = Temporal.PlainDate.from("2021-01-01"); // ISO 8601 calendar
startingPoint.add(dur1).add(dur2).since(startingPoint); // "P396D"
```

Other operations, `round()`, `total()`, and `compare()`, take a `relativeTo` option to provide the necessary calendar and reference time information. This option can be a `Temporal.PlainDate`, `Temporal.PlainDateTime`, `Temporal.ZonedDateTime`, or otherwise an object or string that's convertible using `Temporal.ZonedDateTime.from()` (if the `timeZone` option is provided or the string contains time zone annotation) or `Temporal.PlainDate.from()`.

Note that `days` to `hours` conversion is also technically ambiguous because the length of a day may vary due to offset changes, such as daylight saving time. You can provide a zoned `relativeTo` to account for these changes; otherwise 24-hour days are assumed.

Duration balancing

There are many ways to represent the same duration: for example, "1 minute and 30 seconds" and "90 seconds" are equivalent. However, depending on the context, one representation may be more appropriate than the other. Therefore, generally, the `Duration` object preserves the input values as much as possible, so that when you format it, it will be displayed as you expect.

Each component of a duration has its optimal range; hours should be from 0 to 23, minutes from 0 to 59, and so on. When a component overflows its optimal range, the excess may be "carried" into the next larger component. To carry over, we need to answer the question of "how many X are in a Y?", which is a complicated question for calendar units, so in this case a calendar is needed. Also note that by default, `days` are directly carried into `months`; the weeks unit is only carried into if explicitly requested. If we carry as much as possible, the eventual result where all components are within their optimal range is called a "balanced" duration. Unbalanced durations usually come in the "top-heavy" form, where the largest unit is unbalanced (e.g., "27 hours and 30 minutes"); other forms, such as "23 hours and 270 minutes", are rarely seen.

The `round()` method always balances the duration into the "top-heavy" form, up to the `largestUnit` option. With a manual `largestUnit` option that's large enough, you can fully balance the duration. Similarly, the `add()` and `subtract()` methods balance the result duration to the largest unit of the input durations.

Note that because the ISO 8601 duration format represents subsecond components as one single fraction number, it is not possible to preserve unbalanced subsecond components during serialization using the default format. For example, "1000 milliseconds" is serialized as `"PT1S"`, and then deserialized as "1 second". If you need to preserve the subsecond components' magnitudes, you need to manually serialize it as a JSON object instead (because by default the `toJSON()` method serializes the duration in the ISO 8601 format).

Duration sign

Because a duration is a difference between two time points, it can be positive, negative, or zero. For example, if you are displaying event times in relative time, then negative durations may represent events in the past, and positive durations for the future. In our representation using a combination of time components, the sign is stored within each component: a negative duration always has all components negative (or zero), and a positive duration always has all components positive (or zero). Constructing a duration with components of mixed signs is invalid and will be rejected by the constructor or the `with()` method. The `add()` and `subtract()` methods will balance the result duration to avoid mixed signs.

Constructor

`Temporal.Duration()`

> Creates a new `Temporal.Duration` object by directly supplying the underlying data.

Static methods

`Temporal.Duration.compare()`

> Returns a number (-1, 0, or 1) indicating whether the first duration is shorter, equal to, or longer than the second duration.

`Temporal.Duration.from()`

> Creates a new `Temporal.Duration` object from another `Temporal.Duration` object, an object with duration properties, or an ISO 8601 string.

Instance properties

These properties are defined on `Temporal.Duration.prototype` and shared by all `Temporal.Duration` instances.

`Temporal.Duration.prototype.blank`

> Returns a boolean that is `true` if this duration represents a zero duration, and `false` otherwise. Equivalent to `duration.sign === 0`.

`Temporal.Duration.prototype.constructor`

> The constructor function that created the instance object. For `Temporal.Duration` instances, the initial value is the `Temporal.Duration()` constructor.

`Temporal.Duration.prototype.days`

> Returns an integer representing the number of days in the duration.

`Temporal.Duration.prototype.hours`

> Returns an integer representing the number of hours in the duration.

`Temporal.Duration.prototype.microseconds`

> Returns an integer representing the number of microseconds in the duration.

`Temporal.Duration.prototype.milliseconds`

> Returns an integer representing the number of milliseconds in the duration.

`Temporal.Duration.prototype.minutes`

Returns an integer representing the number of minutes in the duration.

`Temporal.Duration.prototype.months`

Returns an integer representing the number of months in the duration.

`Temporal.Duration.prototype.nanoseconds`

Returns an integer representing the number of nanoseconds in the duration.

`Temporal.Duration.prototype.seconds`

Returns an integer representing the number of seconds in the duration.

`Temporal.Duration.prototype.sign`

Returns `1` if this duration is positive, `-1` if negative, and `0` if zero.

`Temporal.Duration.prototype.weeks`

Returns an integer representing the number of weeks in the duration.

`Temporal.Duration.prototype.years`

Returns an integer representing the number of years in the duration.

`Temporal.Duration.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.Duration"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.Duration.prototype.abs()`

Returns a new `Temporal.Duration` object with the absolute value of this duration (all fields keep the same magnitude, but sign becomes positive).

`Temporal.Duration.prototype.add()`

Returns a new `Temporal.Duration` object with the sum of this duration and a given duration (in a form convertible by `Temporal.Duration.from()`). The result is balanced.

`Temporal.Duration.prototype.negated()`

Returns a new `Temporal.Duration` object with the negated value of this duration (all fields keep the same magnitude, but sign becomes reversed).

`Temporal.Duration.prototype.round()`

Returns a new `Temporal.Duration` object with the duration rounded to the given smallest unit and/or balanced to the given largest unit.

`Temporal.Duration.prototype.subtract()`

Returns a new `Temporal.Duration` object with the difference between this duration and a given duration (in a form convertible by `Temporal.Duration.from()`). Equivalent to adding the negated value of the other duration.

`Temporal.Duration.prototype.toJSON()`

Returns a string representing this duration in the same ISO 8601 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.Duration.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this duration. In implementations with `Intl.DurationFormat` API support, this method delegates to `Intl.DurationFormat`.

`Temporal.Duration.prototype.toString()`

Returns a string representing this duration in the ISO 8601 format.

`Temporal.Duration.prototype.total()`

Returns a number representing the total duration in the given unit.

`Temporal.Duration.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.Duration` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.Duration.prototype.with()`

Returns a new `Temporal.Duration` object representing this duration with some fields replaced by new values.

See also

- `Temporal`

Temporal.Duration.prototype.abs()

The **`abs()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the absolute value of this duration (all fields have the same magnitude, but sign becomes positive).

Syntax

```
abs()
```

Parameters

None.

Return value

A new `Temporal.Duration` object with the absolute value of this duration, which is either the same as this duration if it is already positive, or its negation if it is negative.

Examples

Using abs()

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });

console.log(d1.abs().toString()); // "PT1H30M"
console.log(d2.abs().toString()); // "PT1H30M"
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.negated()`
- `Temporal.Duration.prototype.sign`

Temporal.Duration.prototype.add()

The **`add()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the sum of this duration and a given duration. The result is balanced.

Syntax

```
add(other)
```

Parameters

`other`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this duration. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Duration` object representing the sum of this duration and `other`.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Either `this` or `other` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), because calendar durations are ambiguous without a calendar and time reference.
- The sum of `this` and `other` overflows the maximum or underflows the minimum representable duration, which is $\pm 2^{53}$ seconds.

Description

Non-calendar durations unambiguously represent a fixed amount of time. Internally, `this` and `other` are both converted to nanoseconds (assuming 24-hour days) and added together. The result is then converted back to a `Temporal.Duration` object, so the result is always balanced or top-heavy with the largest possible unit being `days`.

If you want to perform addition or subtraction with a calendar duration, you can add both durations to a starting point and then figure out the difference between the two resulting instants; that is, `dur1 + dur2` is equivalent to `(start + dur1 + dur2) - start`.

To add a duration to a date or time, use the `add()` method of the date or time object instead.

Examples

Using add()

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -20 });

const d3 = d1.add(d2);
console.log(d3.toString()); // "PT10M"
```

Adding calendar durations

```
const d1 = Temporal.Duration.from({ days: 1 });
const d2 = Temporal.Duration.from({ months: 1 });

d1.add(d2); // RangeError: for calendar duration arithmetic, use date arithmetic relative to a starting point

const start = Temporal.PlainDateTime.from("2022-01-01T00:00"); // ISO 8601 calendar
const result = start.add(d1).add(d2).since(start);
console.log(result.toString()); // "P32D"
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.subtract()`

Temporal.Duration.prototype.blank

The **`blank`** accessor property of `Temporal.Duration` instances returns a boolean that is `true` if this duration represents a zero duration, and `false` otherwise. It is equivalent to `duration.sign === 0`.

Examples

Using blank

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ hours: 0 });

console.log(d1.blank); // false
console.log(d2.blank); // false
console.log(d3.blank); // true
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.sign`

Temporal.Duration.compare()

The **`Temporal.Duration.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first duration is shorter, equal to, or longer than the second duration.

Syntax

```
Temporal.Duration.compare(duration1, duration2)
Temporal.Duration.compare(duration1, duration2, options)
```

Parameters

`duration1`

A string, an object, or a `Temporal.Duration` instance representing the first duration to compare. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`duration2`

The second duration to compare, converted to a `Temporal.Duration` object using the same algorithm as `duration1`.

`options` Optional

An object containing the following property:

`relativeTo` Optional

A zoned or plain date(time) that provides the time and calendar information to resolve calendar durations (see the link for the general interpretation of this option). Required if either `duration1` or `duration2` is a calendar duration (unless they are equal component-wise, in which case `0` is returned without computations).

Return value

Returns `-1` if `duration1` is shorter than `duration2`, `0` if they are equal, and `1` if `duration1` is longer than `duration2`.

Exceptions

`RangeError`

Thrown if either `duration1` or `duration2` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), and `relativeTo` is not provided.

Description

If `relativeTo` is a zoned date-time, and either `duration1` or `duration2` is a calendar duration, the result is calculated by adding the durations to the starting point, and then comparing the resulting instants. Otherwise, the comparison is done by converting both of them to nanoseconds (assuming 24-hour days, and using the calendar

of `relativeTo` if necessary) and comparing the results.

Examples

Using Temporal.Duration.compare()

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ minutes: 100 });
console.log(Temporal.Duration.compare(d1, d2)); // -1

const d3 = Temporal.Duration.from({ hours: 2 });
const d4 = Temporal.Duration.from({ minutes: 110 });
console.log(Temporal.Duration.compare(d3, d4)); // 1

const d5 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d6 = Temporal.Duration.from({ seconds: 5400 });
console.log(Temporal.Duration.compare(d5, d6)); // 0
```

Comparing calendar durations

```
const d1 = Temporal.Duration.from({ days: 31 });
const d2 = Temporal.Duration.from({ months: 1 });

console.log(
  Temporal.Duration.compare(d1, d2, {
    relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
  }),
); // 0

console.log(
  Temporal.Duration.compare(d1, d2, {
    relativeTo: Temporal.PlainDate.from("2021-02-01"), // ISO 8601 calendar
  }),
); // 1; February has 28 days
```

Using zoned relativeTo

Using a zoned `relativeTo`, you can even take into account daylight saving time changes. On `2024-11-03`, the US shifts from daylight saving time to standard time, so that day has 25 hours because the clock is set back by 1 hour.

```
const d1 = Temporal.Duration.from({ days: 1 });
const d2 = Temporal.Duration.from({ hours: 24 });

console.log(
  Temporal.Duration.compare(d1, d2, {
    relativeTo: Temporal.ZonedDateTime.from(
      "2024-11-03T01:00-04:00[America/New_York]",
    ),
  }),
); // 1
```

Sorting an array of durations

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const durations = [
  Temporal.Duration.from({ hours: 1 }),
  Temporal.Duration.from({ hours: 2 }),
  Temporal.Duration.from({ hours: 1, minutes: 30 }),
  Temporal.Duration.from({ hours: 1, minutes: 45 }),
];

durations.sort(Temporal.Duration.compare);
console.log(durations.map((d) => d.toString()));
// [ 'PT1H', 'PT1H30M', 'PT1H45M', 'PT2H' ]
```

Pass options like this:

```
durations.sort((a, b) =>
  Temporal.Duration.compare(a, b, {
    relativeTo: Temporal.Now.zonedDateTimeISO(),
  }),
);
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.subtract()`

Temporal.Duration.prototype.days

The **`days`** accessor property of `Temporal.Duration` instances returns an integer representing the number of days in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above days, the `days` absolute value's range depends on the calendar (how many days are in a week or month).

The set accessor of `days` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using days

```
const d1 = Temporal.Duration.from({ weeks: 1, days: 1 });
const d2 = Temporal.Duration.from({ weeks: -1, days: -1 });
const d3 = Temporal.Duration.from({ weeks: 1 });
const d4 = Temporal.Duration.from({ days: 7 });

console.log(d1.days); // 1
console.log(d2.days); // -1
console.log(d3.days); // 0
console.log(d4.days); // 7

// Balance d4
const d4Balanced = d4.round({
  largestUnit: "weeks",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.days); // 0
console.log(d4Balanced.weeks); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration() constructor

The **`Temporal.Duration()`** constructor creates `Temporal.Duration` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.Duration` objects using the `Temporal.Duration.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.Duration()
new Temporal.Duration(years)
new Temporal.Duration(years, months)
new Temporal.Duration(years, months, weeks)
new Temporal.Duration(years, months, weeks, days)
new Temporal.Duration(years, months, weeks, days, hours)
new Temporal.Duration(years, months, weeks, days, hours, minutes)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds, milliseconds)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds, milliseconds, microseconds)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds)
```

> **Note:** `Temporal.Duration()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`years` Optional

Number of years, or `undefined` (which is treated as `0`).

`months` Optional

Number of months, or `undefined` (which is treated as `0`).

`weeks` Optional

Number of weeks, or `undefined` (which is treated as `0`).

`days` Optional

Number of days, or `undefined` (which is treated as `0`).

`hours` Optional

Number of hours, or `undefined` (which is treated as `0`).

`minutes` Optional

Number of minutes, or `undefined` (which is treated as `0`).

`seconds` Optional

Number of seconds, or `undefined` (which is treated as `0`).

`milliseconds` Optional

Number of milliseconds, or `undefined` (which is treated as `0`).

`microseconds` Optional

Number of microseconds, or `undefined` (which is treated as `0`).

`nanoseconds` Optional

Number of nanoseconds, or `undefined` (which is treated as `0`).

Return value

A new `Temporal.Duration` object, possibly unbalanced, with the specified components.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the parameters is not an integer (including non-finite values).
- A calendar unit (years, months, weeks) has an absolute value ≥ 2^{32}.
- The non-calendar part of the duration (days and below), when expressed in seconds, has an absolute value ≥ 2^{53}.

Examples

Using Temporal.Duration()

```
const d = new Temporal.Duration(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
console.log(d.toString()); // "P1Y2M3W4DT5H6M7.00800901S"
```

See also

- `Temporal.Duration`
- `Temporal.Duration.from()`

Temporal.Duration.from()

The **`Temporal.Duration.from()`** static method creates a new `Temporal.Duration` object from another `Temporal.Duration` object, an object with duration properties, or an ISO 8601 string.

Syntax

```
Temporal.Duration.from(info)
```

Parameters

`info`

One of the following:

- A `Temporal.Duration` instance, which creates a copy of the instance.
- An ISO 8601 string representing a duration.
- An object containing at least one of the following properties (in the order they are retrieved and validated):
  - `days`
  - `hours`
  - `microseconds`
  - `milliseconds`
  - `minutes`
  - `months`
  - `nanoseconds`
  - `seconds`
  - `weeks`
  - `years`

Each property should contain an integer number value. The resulting duration must not have mixed signs, so all of these properties must have the same sign (or zero). Missing properties are treated as zero.

Return value

A new `Temporal.Duration` object, possibly unbalanced, with the specified components.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the recognized properties in the `info` object is not an integer (including non-finite values).
- A calendar unit (years, months, weeks) has an absolute value ≥ 2^{32}.
- The non-calendar part of the duration (days and below), when expressed in seconds, has an absolute value ≥ 2^{53}.

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- All of the recognized properties in the `info` object are `undefined`.

Examples

Creating a duration from an object

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
console.log(d1.toString()); // "PT1H30M"

const d2 = Temporal.Duration.from({ months: 1, days: 2 });
console.log(d2.toString()); // "P1M2D"

// Uncommon because unbalanced, but valid
const unbalanced = Temporal.Duration.from({
  hours: 100,
  minutes: 100,
  seconds: 100,
});
console.log(unbalanced.toString()); // "PT100H100M100S"

const neg = Temporal.Duration.from({ hours: -1, minutes: -30 });
console.log(neg.toString()); // "-PT1H30M"
```

Creating a duration from a string

```
const d = Temporal.Duration.from("P1Y2M3W4DT5H6M7.00800901S");
console.log(d.hours); // 5
```

Creating a duration from another duration

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from(d1);
console.log(d2.toString()); // "PT1H30M"
```

See also

- `Temporal.Duration`
- `Temporal.Duration()`
- `Temporal.Duration.prototype.with()`

Temporal.Duration.prototype.hours

The **`hours`** accessor property of `Temporal.Duration` instances returns an integer representing the number of hours in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above hours, the `hours` absolute value will be between 0 and 23, inclusive.

The set accessor of `hours` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using hours

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ days: 1 });
const d4 = Temporal.Duration.from({ hours: 24 });

console.log(d1.hours); // 1
console.log(d2.hours); // -1
```

```
console.log(d3.hours); // 0
console.log(d4.hours); // 24

// Balance d4
const d4Balanced = d4.round({ largestUnit: "days" });
console.log(d4Balanced.hours); // 0
console.log(d4Balanced.days); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.microseconds

The **`microseconds`** accessor property of `Temporal.Duration` instances returns an integer representing the number of microseconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above microseconds, the `microseconds` absolute value will be between 0 and 999, inclusive.

The set accessor of `microseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using microseconds

```
const d1 = Temporal.Duration.from({ milliseconds: 1, microseconds: 500 });
const d2 = Temporal.Duration.from({ milliseconds: -1, microseconds: -500 });
const d3 = Temporal.Duration.from({ milliseconds: 1 });
const d4 = Temporal.Duration.from({ microseconds: 1000 });

console.log(d1.microseconds); // 500
console.log(d2.microseconds); // -500
console.log(d3.microseconds); // 0
console.log(d4.microseconds); // 1000

// Balance d4
const d4Balanced = d4.round({ largestUnit: "milliseconds" });
console.log(d4Balanced.microseconds); // 0
console.log(d4Balanced.milliseconds); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.milliseconds

The **`milliseconds`** accessor property of `Temporal.Duration` instances returns an integer representing the number of milliseconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above milliseconds, the `milliseconds` absolute value will be between 0 and 999, inclusive.

The set accessor of `milliseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using milliseconds

```
const d1 = Temporal.Duration.from({ seconds: 1, milliseconds: 500 });
const d2 = Temporal.Duration.from({ seconds: -1, milliseconds: -500 });
const d3 = Temporal.Duration.from({ seconds: 1 });
const d4 = Temporal.Duration.from({ milliseconds: 1000 });
```

```
console.log(d1.milliseconds); // 500
console.log(d2.milliseconds); // -500
console.log(d3.milliseconds); // 0
console.log(d4.milliseconds); // 1000

// Balance d4
const d4Balanced = d4.round({ largestUnit: "seconds" });
console.log(d4Balanced.milliseconds); // 0
console.log(d4Balanced.seconds); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.minutes

The **`minutes`** accessor property of `Temporal.Duration` instances returns an integer representing the number of minutes in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above minutes, the `minutes` absolute value will be between 0 and 59, inclusive.

The set accessor of `minutes` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using minutes

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ hours: 1 });
const d4 = Temporal.Duration.from({ minutes: 60 });

console.log(d1.minutes); // 30
console.log(d2.minutes); // -30
console.log(d3.minutes); // 0
console.log(d4.minutes); // 60

// Balance d4
const d4Balanced = d4.round({ largestUnit: "hours" });
console.log(d4Balanced.minutes); // 0
console.log(d4Balanced.hours); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.months

The **`months`** accessor property of `Temporal.Duration` instances returns an integer representing the number of months in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above months, the `months` absolute value's range depends on the calendar (how many months are in a year).

The set accessor of `months` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using months

```
const d1 = Temporal.Duration.from({ years: 1, months: 1 });
const d2 = Temporal.Duration.from({ years: -1, months: -1 });
const d3 = Temporal.Duration.from({ years: 1 });
```

```
const d4 = Temporal.Duration.from({ months: 12 });

console.log(d1.months); // 1
console.log(d2.months); // -1
console.log(d3.months); // 0
console.log(d4.months); // 12

// Balance d4
const d4Balanced = d4.round({
  largestUnit: "years",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.months); // 0
console.log(d4Balanced.years); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.nanoseconds

The **`nanoseconds`** accessor property of `Temporal.Duration` instances returns an integer representing the number of nanoseconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above nanoseconds, the `nanoseconds` absolute value will be between 0 and 999, inclusive.

The set accessor of `nanoseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using nanoseconds

```
const d1 = Temporal.Duration.from({ microseconds: 1, nanoseconds: 500 });
const d2 = Temporal.Duration.from({ microseconds: -1, nanoseconds: -500 });
const d3 = Temporal.Duration.from({ microseconds: 1 });
const d4 = Temporal.Duration.from({ nanoseconds: 1000 });

console.log(d1.nanoseconds); // 500
console.log(d2.nanoseconds); // -500
console.log(d3.nanoseconds); // 0
console.log(d4.nanoseconds); // 1000

// Balance d4
const d4Balanced = d4.round({ largestUnit: "microseconds" });
console.log(d4Balanced.nanoseconds); // 0
console.log(d4Balanced.microseconds); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`

Temporal.Duration.prototype.negated()

The **`negated()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the negated value of this duration (all fields keep the same magnitude, but sign becomes reversed).

Syntax

```
negated()
```

Parameters

None.

Return value

A new `Temporal.Duration` object, where all fields have the same magnitude as this duration, but the sign is reversed (positive fields become negative, and vice versa).

Examples

Using negated()

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });

console.log(d1.negated().toString()); // "-PT1H30M"
console.log(d2.negated().toString()); // "PT1H30M"
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.abs()`
- `Temporal.Duration.prototype.sign`

Temporal.Duration.prototype.round()

The **`round()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the duration rounded to the given smallest unit and/or balanced to the given largest unit.

Syntax

```
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`largestUnit` Optional

Any of the temporal units: `"years"`, `"months"`, `"weeks"`, `"days"`, `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms, or the value `"auto"` which means the largest non-zero component of this duration or `smallestUnit`, whichever is greater. Defaults to `"auto"`. The result will not contain units larger than this; for example, if the largest unit is `"minutes"`, then "1 hour 30 minutes" will become "90 minutes".

`relativeTo` Optional

A zoned or plain date(time) that provides the time and calendar information to resolve calendar durations (see the link for the general interpretation of this option). Required if either `this` or `other` is a calendar duration, or `smallestUnit` is a calendar unit.

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. Must be in the inclusive range of 1 to 1e9. If the smallest unit is hours, minutes, seconds, milliseconds, microseconds, or nanoseconds, the increment must be a divisor of the maximum value of the unit; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12.

`roundingMode` Optional

A string representing the rounding mode specifying to round up or down in various scenarios. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit` Optional

Any of the temporal units: `"years"`, `"months"`, `"weeks"`, `"days"`, `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms. Defaults to `"nanoseconds"`. For units larger than `"nanoseconds"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings. Must be smaller or equal to `largestUnit`. At least one of `smallestUnit` and `largestUnit` must be provided.

Return value

A new `Temporal.Duration` object, where the largest unit is no larger than the `largestUnit` option, and the smallest unit is no smaller than the `smallestUnit` option. The fractional parts of the `smallestUnit` are rounded according to the `roundingIncrement` and `roundingMode` settings.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Description

The `round()` method performs two operations: rounding and balancing. It does the following:

1. It makes sure the duration is balanced. If a component is above its preferred maximum (24 hours per day, 60 minutes per hour, etc.), the excess is carried over to the next larger unit, until we reach `largestUnit`. For example, "24 hours 90 minutes" becomes "25 hours 30 minutes" if `largestUnit` is `"auto"`, and "1 day 1 hour 30 minutes" if `largestUnit` is `"days"`.
2. For all components larger than `largestUnit`, they are carried down into `largestUnit`; for example, "2 hours 30 minutes" becomes "150 minutes" if `largestUnit` is `"minutes"`.
3. For all components smaller than `smallestUnit`, they are carried up into `smallestUnit` as a fractional part, and then rounded according to the `roundingIncrement` and `roundingMode` settings. For example, "1 hour 30 minutes" becomes "1.5 hours" if `smallestUnit` is `"hours"`, and then rounded to "2 hours" using the default settings.

Calendar units have uneven lengths. Their lengths are resolved relative to a starting point. For example, a duration of "2 years" in the Gregorian calendar may be 730 days or 731 days long, depending on whether it moves through a leap year or not. When rounding to a calendar unit, we first get the exact date-time represented by `relativeTo + duration`, then round it down and up according to `smallestUnit` and `roundingIncrement` to get two candidates. Then, we choose the candidate according to the `roundingMode` setting, and finally subtract `relativeTo` to get the final duration.

Examples

Rounding off small units

```
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
const roundedDuration = duration.round("minutes");
console.log(roundedDuration.toString()); // "PT1H30M"
```

Avoiding larger units

```
const duration = Temporal.Duration.from({
  days: 3,
  hours: 1,
  minutes: 41,
  seconds: 5,
});
const roundedDuration = duration.round({ largestUnit: "hours" });
console.log(
  `Time spent on this problem: ${roundedDuration.toLocaleString("en-US", { style: "digital" })}`,
);
// Time spent on this problem: 73:41:05
```

Rounding to a whole number of hours

```
const duration = Temporal.Duration.from({ days: 1, hours: 1, minutes: 30 });
const roundedDuration = duration.round({
  largestUnit: "hours",
  smallestUnit: "hours",
  roundingMode: "floor",
});
console.log(roundedDuration.hours); // 25
```

Rounding by 15-minute increments

```
const duration = Temporal.Duration.from({ hours: 1, minutes: 17 });
const roundedDuration = duration.round({
  smallestUnit: "minutes",
  roundingIncrement: 15,
});
console.log(
  `The queue will take approximately ${roundedDuration.toLocaleString("en-US")}`,
);
// The queue will take approximately 1 hr, 15 min
```

Resolving calendar durations

If either the initial duration or largest/smallest unit contains a calendar unit, you must provide a `relativeTo` option to resolve the calendar durations.

```
const duration = Temporal.Duration.from({ months: 1, days: 1, hours: 1 });
const roundedDuration = duration.round({
  largestUnit: "days",
  smallestUnit: "days",
  relativeTo: Temporal.PlainDateTime.from("2022-01-01"),
});
console.log(roundedDuration); // "P32D"
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.total()`

Temporal.Duration.prototype.seconds

The **`seconds`** accessor property of `Temporal.Duration` instances returns an integer representing the number of seconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above seconds, the `seconds` absolute value will be between 0 and 59, inclusive.

The set accessor of `seconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using seconds

```
const d1 = Temporal.Duration.from({ minutes: 1, seconds: 30 });
const d2 = Temporal.Duration.from({ minutes: -1, seconds: -30 });
const d3 = Temporal.Duration.from({ minutes: 1 });
const d4 = Temporal.Duration.from({ seconds: 60 });

console.log(d1.seconds); // 30
console.log(d2.seconds); // -30
console.log(d3.seconds); // 0
console.log(d4.seconds); // 60

// Balance d4
const d4Balanced = d4.round({ largestUnit: "minutes" });
console.log(d4Balanced.seconds); // 0
console.log(d4Balanced.minutes); // 1
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.sign

The **`sign`** accessor property of `Temporal.Duration` instances returns `1` if this duration is positive, `-1` if negative, and `0` if zero. Because a duration never has mixed signs, the sign of a duration is determined by the sign of any of its non-zero fields.

Examples

Using sign

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ hours: 0 });

console.log(d1.sign); // 1
console.log(d2.sign); // -1
console.log(d3.sign); // 0

console.log(d1.abs().sign); // 1
console.log(d2.abs().sign); // 1
console.log(d3.abs().sign); // 0

console.log(d1.negated().sign); // -1
console.log(d2.negated().sign); // 1
console.log(d3.negated().sign); // 0
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.abs()`
- `Temporal.Duration.prototype.negated()`
- `Temporal.Duration.prototype.blank`

Temporal.Duration.prototype.subtract()

The **`subtract()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the difference between this duration and a given duration. It is equivalent to adding the negated value of the other duration.

Syntax

```
subtract(other)
```

Parameters

`other`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this duration. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Duration` object representing the difference of this duration and `other`.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Either `this` or `other` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), because calendar durations are ambiguous without a calendar and time reference.
- The difference of `this` and `other` overflows the maximum or underflows the minimum representable duration, which is $\pm 2^{53}$ seconds.

Examples

Using subtract()

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -20 });

const d3 = d1.subtract(d2);
console.log(d3.toString()); // "PT2H50M"
```

For more examples and caveats, see the `add()` method.

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.add()`
- `Temporal.Duration.prototype.negated()`

Temporal.Duration.prototype.toJSON()

The **`toJSON()`** method of `Temporal.Duration` instances returns a string representing this duration in the same ISO 8601 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given duration in the ISO 8601 format, with as much subsecond precision as necessary to represent the duration accurately.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.Duration` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.Duration` objects during JSON serialization, which can then be deserialized using the `Temporal.Duration.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
const durationStr = duration.toJSON(); // 'PT1H30M15S'
const d2 = Temporal.Duration.from(durationStr);
```

JSON serialization and parsing

This example shows how `Temporal.Duration` can be serialized as JSON without extra effort, and how to parse it back.

```
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
const jsonStr = JSON.stringify({ data: duration }); // '{"data":"PT1H30M15S"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "data") {
    return Temporal.Duration.from(value);
  }
  return value;
});
```

See also

- `Temporal.Duration`

- `Temporal.Duration.from()`
- `Temporal.Duration.prototype.toString()`
- `Temporal.Duration.prototype.toLocaleString()`

Temporal.Duration.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.Duration` instances returns a string with a language-sensitive representation of this duration. In implementations with `Intl.DurationFormat` API support, this method delegates to `Intl.DurationFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DurationFormat` object and use its `format()` method, because a `DurationFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DurationFormat` API, these parameters correspond exactly to the `Intl.DurationFormat()` constructor's parameters. Implementations without `Intl.DurationFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DurationFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DurationFormat()` constructor.

See the `Intl.DurationFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given duration according to language-specific conventions.

In implementations with `Intl.DurationFormat`, this is equivalent to `new Intl.DurationFormat(locales, options).format(duration)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });

console.log(duration.toLocaleString()); // 1 hr, 30 min, 15 sec
```

See also

- `Temporal.Duration`
- `Intl.DurationFormat`
- `Temporal.Duration.prototype.toJSON()`
- `Temporal.Duration.prototype.toString()`

Temporal.Duration.prototype.toString()

The **`toString()`** method of `Temporal.Duration` instances returns a string representing this duration in the ISO 8601 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

Return value

A string representing the given duration in the ISO 8601 format, with subsecond components formatted according to the options. The zero duration is represented as `"PT0S"`.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using toString()

```
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
console.log(duration.toString()); // 'PT1H30M15S'

// Stringification implicitly calls toString()
console.log(`${duration}`); // 'PT1H30M15S'
```

Using options

```
const worldRecord = Temporal.Duration.from({ seconds: 9, milliseconds: 580 });
console.log(worldRecord.toString()); // 'PT9.58S'
console.log(worldRecord.toString({ fractionalSecondDigits: 1 })); // 'PT9.5S'
console.log(worldRecord.toString({ fractionalSecondDigits: 0 })); // 'PT9S'
console.log(worldRecord.toString({ smallestUnit: "millisecond" })); // 'PT9.580S'
console.log(
  worldRecord.toString({
    fractionalSecondDigits: 1,
    roundingMode: "halfExpand",
  }),
); // 'PT9.6S'
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.toJSON()`
- `Temporal.Duration.prototype.toLocaleString()`

Temporal.Duration.prototype.total()

The **`total()`** method of `Temporal.Duration` instances returns a number representing the total duration in the given unit.

Syntax

```
total(unit)
total(options)
```

Parameters

`unit`

A string representing the `unit` option. This is a convenience overload, so `total(unit)` is equivalent to `total({ unit })`, where `unit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`relativeTo` Optional

A zoned or plain date(time) that provides the time and calendar information to resolve calendar durations (see the link for the general interpretation of this option). Required if either `this` or `other` is a calendar duration, or `unit` is a calendar unit.

`unit`

Any of the temporal units: `"years"`, `"months"`, `"weeks"`, `"days"`, `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms.

Return value

A floating-point number representing the total duration in the given unit. May be inexact due to floating point precision limits.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `unit` is not provided or is not a valid unit.
- Either `this` or `other` is a calendar duration, or `unit` is a calendar unit, and `relativeTo` is not provided.

Description

If a `relativeTo` is provided, the result is calculated by adding the duration to the starting point, finding the difference between the result and the starting point (in nanoseconds), and then converting the difference to the requested unit by dividing by the appropriate number of nanoseconds per unit. Providing a zoned date-time allows daylight saving time and other time zone changes to be taken into account too; otherwise, 24-hour days are assumed.

If `relativeTo` is not provided, the result is calculated by converting the duration to nanoseconds and dividing by the appropriate number of nanoseconds per unit.

Examples

Using total()

```
const d = Temporal.Duration.from({ hours: 1, minutes: 30 });

console.log(d.total("minutes")); // 90
console.log(d.total("hours")); // 1.5
```

Total of a calendar duration

```
const d = Temporal.Duration.from({ months: 1 });

console.log(
  d.total({ unit: "days", relativeTo: Temporal.PlainDate.from("2021-01-01") }),
); // 31
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.round()`

Temporal.Duration.prototype.valueOf()

The **`valueOf()`** method of `Temporal.Duration` instances throws a `TypeError`, which prevents `Temporal.Duration` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `duration1 > duration2` would implicitly compare them as strings, which may have unexpected results such as `"PT3S" > "PT1M"`. By throwing a `TypeError`, `Temporal.Duration` instances prevent such implicit conversions. You need to explicitly convert them to numbers using `Temporal.Duration.prototype.total()`, or use the `Temporal.Duration.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.Duration

All arithmetic and comparison operations on `Temporal.Duration` instances should use the dedicated methods or convert them to primitives explicitly.

```
const duration1 = Temporal.Duration.from({ seconds: 3 });
const duration2 = Temporal.Duration.from({ minutes: 1 });
duration1 > duration2; // TypeError: can't convert Duration to primitive type
duration1.total("seconds") > duration2.total("seconds"); // false
Temporal.Duration.compare(duration1, duration2); // -1
```

```
duration1 + duration2; // TypeError: can't convert Duration to primitive type
duration1.total("seconds") + duration2.total("seconds"); // 63
duration1.add(duration2).toString(); // "PT1M3S"
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.toString()`
- `Temporal.Duration.prototype.toJSON()`
- `Temporal.Duration.prototype.toLocaleString()`

Temporal.Duration.prototype.weeks

The **`weeks`** accessor property of `Temporal.Duration` instances returns an integer representing the number of weeks in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above weeks, the `weeks` absolute value's range depends on the calendar (how many weeks are in a month or year).

The set accessor of `weeks` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using weeks

```
const d1 = Temporal.Duration.from({ weeks: 1, days: 1 });
const d2 = Temporal.Duration.from({ weeks: -1, days: -1 });
const d3 = Temporal.Duration.from({ weeks: 1 });
const d4 = Temporal.Duration.from({ days: 7 });

console.log(d1.weeks); // 1
console.log(d2.weeks); // -1
console.log(d3.weeks); // 1
console.log(d4.weeks); // 0

// Balance d4
const d4Balanced = d4.round({
  largestUnit: "weeks",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.weeks); // 1
console.log(d4Balanced.days); // 0
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

Temporal.Duration.prototype.with()

The **`with()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object representing this duration with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the duration's fields.

Syntax

```
with(info)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.Duration.from()`: `years`, `months`, `weeks`, `days`, `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, `nanoseconds`. Unspecified properties use the values from the original duration.

Return value

A new `Temporal.Duration` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original duration.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the recognized properties in the `info` object is not an integer (including non-finite values).
- A calendar unit (years, months, weeks) has an absolute value ≥ 2^{32}.
- The non-calendar part of the duration (days and below), when expressed in seconds, has an absolute value ≥ 2^{53}.

`TypeError`

Thrown in one of the following cases:

- The `info` object is not an object.
- All of the recognized properties in the `info` object are `undefined`.

Examples

Using with()

You can use `with()` to achieve fine-grained control over the fields of a `Temporal.Duration` object. For example, you can manually balance a duration only on one unit, which `round()` does not offer:

```
function balanceMinutes(duration) {
  const { hours, minutes } = duration;
  const totalMinutes = hours * 60 + minutes;
  const balancedMinutes = totalMinutes % 60;
  const balancedHours = (totalMinutes - balancedMinutes) / 60;
  return duration.with({ hours: balancedHours, minutes: balancedMinutes });
}

const d1 = Temporal.Duration.from({ hours: 100, minutes: 100, seconds: 100 });
const d2 = balanceMinutes(d1);
console.log(d2.hours); // 101
console.log(d2.minutes); // 40
console.log(d2.seconds); // 100; remains unbalanced
```

See also

- `Temporal.Duration`
- `Temporal.Duration.from()`
- `Temporal.Duration.prototype.add()`
- `Temporal.Duration.prototype.subtract()`
- `Temporal.Duration.prototype.round()`

Temporal.Duration.prototype.years

The **`years`** accessor property of `Temporal.Duration` instances returns an integer representing the number of years in the duration.

You can know the sign of `years` by checking the duration's `sign` property.

The set accessor of `years` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using years

```
const d1 = Temporal.Duration.from({ years: 1, months: 1 });
const d2 = Temporal.Duration.from({ years: -1, months: -1 });
const d3 = Temporal.Duration.from({ years: 1 });
const d4 = Temporal.Duration.from({ months: 12 });

console.log(d1.years); // 1
console.log(d2.years); // -1
console.log(d3.years); // 1
console.log(d4.years); // 0

// Balance d4
const d4Balanced = d4.round({
  largestUnit: "years",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.years); // 1
console.log(d4Balanced.months); // 0
```

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`

- `Temporal.Duration.prototype.nanoseconds`

Temporal.Instant

The **`Temporal.Instant`** object represents a unique point in time, with nanosecond precision. It is fundamentally represented as the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC), without any time zone or calendar system.

Description

`Temporal.Instant` is semantically the same as `Date`. They both encapsulate a single point in time, but `Temporal.Instant` is more precise because it stores nanoseconds rather than milliseconds. `Temporal.Instant` also avoids pitfalls of `Date` because it does not assume any calendar or time zone information—if you want to read any date or time information such as year or month, you need to convert it to a `Temporal.ZonedDateTime` first, using `toZonedDateTimeISO()`.

You can convert from `Date` to `Temporal.Instant` using the `Date.prototype.toTemporalInstant()` method, which should be preferred over other methods such as `Temporal.Instant.fromEpochMilliseconds()` because the former involves less user code and may be more optimized. You can also convert from `Temporal.Instant` to `Date` using its epoch milliseconds, such as `new Date(instant.epochMilliseconds)`.

RFC 9557 format

`Instant` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss Z/±HH:mm
```

`YYYY`

: Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

: A two-digit number from `01` to `12`.

`DD`

: A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`T`

: The date-time separator, which can be `T`, `t`, or a space.

`HH`

: A two-digit number from `00` to `23`.

`mm` Optional

: A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

: A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

`Z/±HH:mm`

: Either the UTC designator `Z` or `z`, or an offset from UTC in the form `+` or `-` followed by the same format as the time component. Note that subminute precision (`:ss.sssssssss`) may be unsupported by other systems, and is accepted but never output. If an offset is provided, the time is interpreted in the specified offset.

As an input, you may optionally include the time zone identifier and calendar, in the same format as `ZonedDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits and offset.

Constructor

`Temporal.Instant()`

: Creates a new `Temporal.Instant` object by directly supplying the underlying data.

Static methods

`Temporal.Instant.compare()`

: Returns a number (-1, 0, or 1) indicating whether the first instant comes before, is the same as, or comes after the second instant. Equivalent to comparing the `epochNanoseconds` of the two instants.

`Temporal.Instant.from()`

: Creates a new `Temporal.Instant` object from another `Temporal.Instant` object, or an RFC 9557 string.

`Temporal.Instant.fromEpochMilliseconds()`

: Creates a new `Temporal.Instant` object from the number of milliseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

`Temporal.Instant.fromEpochNanoseconds()`

Creates a new `Temporal.Instant` object from the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

Instance properties

These properties are defined on `Temporal.Instant.prototype` and shared by all `Temporal.Instant` instances.

`Temporal.Instant.prototype.constructor`

The constructor function that created the instance object. For `Temporal.Instant` instances, the initial value is the `Temporal.Instant()` constructor.

`Temporal.Instant.prototype.epochMilliseconds`

Returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. Equivalent to dividing `epochNanoseconds` by `1e6` and flooring it.

`Temporal.Instant.prototype.epochNanoseconds`

Returns a `BigInt` representing the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

`Temporal.Instant.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.Instant"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.Instant.prototype.add()`

Returns a new `Temporal.Instant` object representing this instant moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.Instant.prototype.equals()`

Returns `true` if this instant is equivalent in value to another instant (in a form convertible by `Temporal.Instant.from()`), and `false` otherwise. They are compared by their epoch nanoseconds. Equivalent to `Temporal.Instant.compare(this, other) === 0`.

`Temporal.Instant.prototype.round()`

Returns a new `Temporal.Instant` object representing this instant rounded to the given unit.

`Temporal.Instant.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another instant (in a form convertible by `Temporal.Instant.from()`) to this instant. The duration is positive if the other instant is before this instant, and negative if after.

`Temporal.Instant.prototype.subtract()`

Returns a new `Temporal.Instant` object representing this instant moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.Instant.prototype.toJSON()`

Returns a string representing this instant in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.Instant.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this instant. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

`Temporal.Instant.prototype.toString()`

Returns a string representing this instant in the RFC 9557 format using the specified time zone.

`Temporal.Instant.prototype.toZonedDateTimeISO()`

Returns a new `Temporal.ZonedDateTime` object representing this instant in the specified time zone using the ISO 8601 calendar system.

`Temporal.Instant.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this instant to another instant (in a form convertible by `Temporal.Instant.from()`). The duration is positive if the other instant is after this instant, and negative if before.

`Temporal.Instant.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.Instant` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.ZonedDateTime`

Temporal.Instant.prototype.add()

The **`add()`** method of `Temporal.Instant` instances returns a new `Temporal.Instant` object representing this instant moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```
add(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this instant. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Instant` object representing adding `duration` to this instant. If `duration` is positive, then the returned instant is later than this instant; if `duration` is negative, then the returned instant is earlier than this instant.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `duration` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), or has a non-zero `days`, because calendar durations are ambiguous without a calendar and time reference.
- The result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

In essence, the `add()` method first gets the number of nanoseconds represented by `duration`, adds it to this instant's `epochNanoseconds`, and then creates a new `Temporal.Instant` object from the result. Therefore, the duration must unambiguously represent a fixed amount of time.

If you want to add a calendar duration, the addition must be performed in the context of a calendar and a time zone to account for the variable lengths of months, years, and days (because of daylight saving time). In this case, convert the instant to a `Temporal.ZonedDateTime` object, add the duration, and then convert the result back to an instant.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a Temporal.Duration

```
const instant = Temporal.Instant.fromEpochMilliseconds(0);
const duration = Temporal.Duration.from("PT1S");
const newInstant = instant.add(duration);
console.log(newInstant.epochMilliseconds); // 1000
```

Adding an object or a string

```
const instant = Temporal.Instant.fromEpochMilliseconds(0);
const newInstant = instant.add({ seconds: 1 });
console.log(newInstant.epochMilliseconds); // 1000

const newInstant2 = instant.add("PT1S");
console.log(newInstant2.epochMilliseconds); // 1000
```

Adding a calendar duration

```
const instant = Temporal.Instant.fromEpochMilliseconds(1730610000000);
const duration = Temporal.Duration.from({ days: 1 });

// This instant is 2024-11-03T01:00:00-04:00[America/New_York],
// which is a DST transition day in the US.
const instant2 = instant
  .toZonedDateTimeISO("America/New_York")
  .add(duration)
  .toInstant();
console.log(instant2.epochMilliseconds); // 1730700000000

// The same instant is not a DST transition day in Paris.
const instant3 = instant
  .toZonedDateTimeISO("Europe/Paris")
  .add(duration)
  .toInstant();
console.log(instant3.epochMilliseconds); // 1730696400000
```

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.subtract()`

Temporal.Instant.compare()

The **`Temporal.Instant.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first instant comes before, is the same as, or comes after the second instant. It is equivalent to comparing the `epochNanoseconds` of the two instants.

Syntax

```
Temporal.Instant.compare(instant1, instant2)
```

Parameters

`instant1`

> A string or a `Temporal.Instant` instance representing the first instant to compare. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

`instant2`

> The second instant to compare, converted to a `Temporal.Instant` object using the same algorithm as `instant1`.

Return value

Returns `-1` if `instant1` comes before `instant2`, `0` if they are the same, and `1` if `instant1` comes after `instant2`.

Examples

Using Temporal.Instant.compare()

```
const instant1 = Temporal.Instant.from("2021-08-01T12:34:56Z");
const instant2 = Temporal.Instant.from("2021-08-01T12:34:56Z");

console.log(Temporal.Instant.compare(instant1, instant2)); // 0

const instant3 = Temporal.Instant.from("2021-08-01T13:34:56Z");
console.log(Temporal.Instant.compare(instant1, instant3)); // -1
```

Sorting an array of instants

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const instants = [
  Temporal.Instant.from("2021-08-01T12:34:56Z"),
  Temporal.Instant.from("2021-08-01T12:34:56+01:00"),
  Temporal.Instant.from("2021-08-01T12:34:56-01:00"),
];

instants.sort(Temporal.Instant.compare);
console.log(instants.map((instant) => instant.toString()));
// [ '2021-08-01T11:34:56Z', '2021-08-01T12:34:56Z', '2021-08-01T13:34:56Z' ]
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.equals()`

Temporal.Instant.prototype.epochMilliseconds

The **`epochMilliseconds`** accessor property of `Temporal.Instant` instances returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. It is equivalent to dividing `epochNanoseconds` by `1e6` and flooring the result.

The set accessor of `epochMilliseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.Instant` object with the desired new `epochMilliseconds` value, use the `Temporal.Instant.fromEpochMilliseconds()` static method instead.

Examples

Using epochMilliseconds

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
console.log(instant.epochMilliseconds); // 1627821296789

const instant2 = Temporal.Instant.from("1969-08-01T12:34:56.789Z");
console.log(instant2.epochMilliseconds); // -13173903211
```

Changing epochMilliseconds

This is the method that allows you to move by any amount of time:

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = instant.add({ hours: 1 });
console.log(instant1hourLater.epochMilliseconds); // 1627824896789
```

If you already know the change in milliseconds, you can also directly construct a new `Temporal.Instant` object:

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = Temporal.Instant.fromEpochMilliseconds(
  instant.epochMilliseconds + 3600000,
);
console.log(instant1hourLater.epochMilliseconds); // 1627824896789
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochNanoseconds`
- `Temporal.Instant.fromEpochMilliseconds()`

Temporal.Instant.prototype.epochNanoseconds

The **`epochNanoseconds`** accessor property of `Temporal.Instant` instances returns a `BigInt` representing the number of nanoseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

The set accessor of `epochNanoseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.Instant` object with the desired new `epochNanoseconds` value, use the `Temporal.Instant.fromEpochNanoseconds()` static method instead.

An instant can only represent $\pm 10^8$ days (about ±273,972.6 years) around the epoch, which is ±8.64e21 nanoseconds. Attempting to set `epochNanoseconds` beyond this boundary throws a `RangeError`.

Examples

Using epochNanoseconds

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
console.log(instant.epochNanoseconds); // 1627821296789000000n

const instant2 = Temporal.Instant.from("1969-08-01T12:34:56.789Z");
console.log(instant2.epochNanoseconds); // -13173903211000000n
```

Changing epochNanoseconds

This is the method that allows you to move by any amount of time:

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = instant.add({ hours: 1 });
console.log(instant1hourLater.epochNanoseconds); // 1627824896789000000n
```

If you already know the change in nanoseconds, you can also directly construct a new `Temporal.Instant` object:

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = Temporal.Instant.fromEpochNanoseconds(
  instant.epochNanoseconds + 3600000000000n,
);
console.log(instant1hourLater.epochNanoseconds); // 1627824896789000000n
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochMilliseconds`
- `Temporal.Instant.fromEpochNanoseconds()`

Temporal.Instant.prototype.equals()

The **`equals()`** method of `Temporal.Instant` instances returns `true` if this instant is equivalent in value to another instant (in a form convertible by `Temporal.Instant.from()`), and `false` otherwise. They are compared by their `epochNanoseconds`. It is equivalent to `Temporal.Instant.compare(this, other) === 0`.

Syntax

```
equals(other)
```

Parameters

`other`

> A string or a `Temporal.Instant` instance representing the other instant to compare. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

Return value

`true` if this instant is equal to `other` by nanoseconds, `false` otherwise.

Examples

Using equals()

```
const instant1 = Temporal.Instant.from("2021-08-01T12:34:56Z");
const instant2 = Temporal.Instant.fromEpochMilliseconds(1627821296000);
```

```
console.log(instant1.equals(instant2)); // true
```

See also

- `Temporal.Instant`
- `Temporal.Instant.compare()`

Temporal.Instant.from()

The **`Temporal.Instant.from()`** static method creates a new `Temporal.Instant` object from another `Temporal.Instant` object, or an RFC 9557 string.

Syntax

```
Temporal.Instant.from(info)
```

Parameters

`info`

One of the following:

- A `Temporal.Instant` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date, time, and time zone offset. The time zone name is ignored; only the offset is used.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `info`.

Exceptions

`TypeError`

Thrown if `info` is not a `Temporal.Instant` instance or a string.

`RangeError`

Thrown in one of the following cases:

- The string is not a valid RFC 9557 string.
- The info is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating an instant from a string

```
const instant = Temporal.Instant.from("1970-01-01T00Z");
console.log(instant.toString()); // 1970-01-01T00:00:00Z

const instant2 = Temporal.Instant.from("1970-01-01T00+08:00");
console.log(instant2.toString()); // 1969-12-31T16:00:00Z

// America/New_York is UTC-5 in January 1970, not UTC+8
const instant3 = Temporal.Instant.from("1970-01-01T00+08:00[America/New_York]");
console.log(instant3.toString()); // 1969-12-31T16:00:00Z; the time zone name is ignored
```

Creating an instant from another instant

```
const instant = Temporal.Instant.from("1970-01-01T00Z");
const instant2 = Temporal.Instant.from(instant);
console.log(instant2.toString()); // 1970-01-01T00:00:00Z
```

See also

- `Temporal.Instant`
- `Temporal.Instant.fromEpochMilliseconds()`
- `Temporal.Instant.fromEpochNanoseconds()`

Temporal.Instant.fromEpochMilliseconds()

The **`Temporal.Instant.fromEpochMilliseconds()`** static method creates a new `Temporal.Instant` object from the number of milliseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

To convert a `Date` object to a `Temporal.Instant` object, use `Date.prototype.toTemporalInstant()` instead.

Syntax

```
Temporal.Instant.fromEpochMilliseconds(epochMilliseconds)
```

Parameters

`epochMilliseconds`

A number representing the number of milliseconds since the Unix epoch. Internally, it is converted to a BigInt and multiplied by `1e6` to get the number of nanoseconds.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `epochMilliseconds`.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `epochMilliseconds` cannot be converted to a BigInt (e.g., not an integer).
- `epochMilliseconds` is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using Temporal.Instant.fromEpochMilliseconds()

```
const instant = Temporal.Instant.fromEpochMilliseconds(0);
console.log(instant.toString()); // 1970-01-01T00:00:00Z
const vostok1Liftoff = Temporal.Instant.fromEpochMilliseconds(-275248380000);
console.log(vostok1Liftoff.toString()); // 1961-04-12T06:07:00Z
const sts1Liftoff = Temporal.Instant.fromEpochMilliseconds(355924804000);
console.log(sts1Liftoff.toString()); // 1981-04-12T12:00:04Z
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochMilliseconds`
- `Temporal.Instant.from()`
- `Temporal.Instant.fromEpochNanoseconds()`
- `Date.prototype.toTemporalInstant()`

Temporal.Instant.fromEpochNanoseconds()

The **`Temporal.Instant.fromEpochNanoseconds()`** static method creates a new `Temporal.Instant` object from the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

To convert a `Date` object to a `Temporal.Instant` object, use `Date.prototype.toTemporalInstant()` instead.

Syntax

```
Temporal.Instant.fromEpochNanoseconds(epochNanoseconds)
```

Parameters

`epochNanoseconds`

A BigInt representing the number of nanoseconds since the Unix epoch.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `epochNanoseconds`.

Exceptions

`RangeError`

Thrown if `epochNanoseconds` is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using Temporal.Instant.fromEpochNanoseconds()

```
const instant = Temporal.Instant.fromEpochNanoseconds(0n);
console.log(instant.toString()); // 1970-01-01T00:00:00Z
const vostok1Liftoff =
  Temporal.Instant.fromEpochNanoseconds(-27524838000000000n);
console.log(vostok1Liftoff.toString()); // 1961-04-12T06:07:00Z
const sts1Liftoff = Temporal.Instant.fromEpochNanoseconds(355924804000000000n);
console.log(sts1Liftoff.toString()); // 1981-04-12T12:00:04Z
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochNanoseconds`
- `Temporal.Instant.from()`
- `Temporal.Instant.fromEpochMilliseconds()`
- `Date.prototype.toTemporalInstant()`

Temporal.Instant() constructor

The **`Temporal.Instant()`** constructor creates `Temporal.Instant` objects.

This constructor is exactly equivalent to calling `Temporal.Instant.fromEpochNanoseconds()`.

Syntax

```
new Temporal.Instant(epochNanoseconds)
```

> **Note:** `Temporal.Instant()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`epochNanoseconds`

A BigInt representing the number of nanoseconds since the Unix epoch.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `epochNanoseconds`.

Exceptions

`RangeError`

Thrown if `epochNanoseconds` represents an instant outside the range of representable instants, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using Temporal.Instant()

```
const instant = new Temporal.Instant(0n);
console.log(instant.toString()); // 1970-01-01T00:00:00Z
const vostok1Liftoff = new Temporal.Instant(-27524838000000000n);
console.log(vostok1Liftoff.toString()); // 1961-04-12T06:07:00Z
const sts1Liftoff = new Temporal.Instant(355924804000000000n);
console.log(sts1Liftoff.toString()); // 1981-04-12T12:00:04Z
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochNanoseconds`
- `Temporal.Instant.from()`
- `Temporal.Instant.fromEpochMilliseconds()`
- `Temporal.Instant.fromEpochNanoseconds()`

Temporal.Instant.prototype.round()

The **`round()`** method of `Temporal.Instant` instances returns a new `Temporal.Instant` object representing this instant rounded to the given unit.

Syntax

```
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. The increment and the `smallestUnit` must evenly divide 24 hours; for example, 45 seconds is a divisor of 86400 seconds, and 100 minutes is a divisor of 3600 minutes. This is slightly less strict than the `round()` method of the other classes, which all require the increment to be a divisor of the maximum value of the unit.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.Instant` object representing this instant rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```
const instant = Temporal.Instant.fromEpochMilliseconds(1000);
const roundedInstant = instant.round("second");
console.log(roundedInstant.epochMilliseconds); // 1000

const instant2 = instant.round("minute");
console.log(instant2.epochMilliseconds); // 0
```

See also

- `Temporal.Instant`

Temporal.Instant.prototype.since()

The **`since()`** method of `Temporal.Instant` instances returns a new `Temporal.Duration` object representing the duration from another instant (in a form convertible by `Temporal.Instant.from()`) to this instant. The duration is positive if the other instant is before this instant, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```
since(other)
since(other, options)
```

Parameters

`other`

A string or a `Temporal.Instant` instance representing an instant to subtract from this instant. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"seconds"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"nanoseconds"`.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this instant. The duration is positive if `other` is before this instant, and negative if after.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using since()

```
const lastUpdated = Temporal.Instant.fromEpochMilliseconds(1735235418000);
const now = Temporal.Now.instant();
const duration = now.since(lastUpdated, { smallestUnit: "minute" });
console.log(`Last updated ${duration.toLocaleString("en-US")} ago`);
```

Balancing the resulting duration

Because an instant does not carry calendar information, the resulting duration avoids calendar durations, which are ambiguous without a calendar and time reference. Therefore, the result is unbalanced because `hours` may be greater than 24. To balance the duration, round the resulting duration again with the desired `largestUnit`, passing a `relativeTo` that carries the calendar information.

```
const lastUpdated = Temporal.Instant.fromEpochMilliseconds(1735235418000);
const now = Temporal.Now.instant();
const duration = now.since(lastUpdated, { smallestUnit: "minutes" });
const roundedDuration = duration.round({
```

```
  largestUnit: "years",
  // Use the ISO calendar; you can convert to another calendar using
  // withCalendar()
  relativeTo: now.toZonedDateTimeISO("UTC"),
});
console.log(`Last updated ${roundedDuration.toLocaleString("en-US")} ago`);
```

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.add()`
- `Temporal.Instant.prototype.subtract()`
- `Temporal.Instant.prototype.until()`

Temporal.Instant.prototype.subtract()

The **`subtract()`** method of `Temporal.Instant` instances returns a new `Temporal.Instant` object representing this instant moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two instants and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this instant. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Instant` object representing subtracting `duration` from this instant. If `duration` is positive, then the returned instant is earlier than this instant; if `duration` is negative, then the returned instant is later than this instant.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `duration` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), or has a non-zero `days`, because calendar durations are ambiguous without a calendar and time reference.
- The result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a Temporal.Duration

```
const instant = Temporal.Instant.fromEpochMilliseconds(1000);
const duration = Temporal.Duration.from("PT1S"); // One-second duration
const newInstant = instant.subtract(duration);
console.log(newInstant.epochMilliseconds); // 0
```

For more examples, see `add()`.

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.add()`
- `Temporal.Instant.prototype.since()`
- `Temporal.Instant.prototype.until()`

Temporal.Instant.prototype.toJSON()

The **`toJSON()`** method of `Temporal.Instant` instances returns a string representing this instant in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given instant in the RFC 9557 format, with as much subsecond precision as necessary to represent the duration accurately, and with the UTC time zone designator `z`.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.Instant` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.Instant` objects during JSON serialization, which can then be deserialized using the `Temporal.Instant.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const instant = Temporal.Instant.fromEpochMilliseconds(1627821296000);
const instantStr = instant.toJSON(); // '2021-08-01T12:34:56Z'
const i2 = Temporal.Instant.from(instantStr);
```

JSON serialization and parsing

This example shows how `Temporal.Instant` can be serialized as JSON without extra effort, and how to parse it back.

```
const instant = Temporal.Instant.fromEpochMilliseconds(1627821296000);
const jsonStr = JSON.stringify({ time: instant }); // '{"time":"2021-08-01T12:34:56Z"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "time") {
    return Temporal.Instant.from(value);
  }
  return value;
});
```

See also

- `Temporal.Instant`
- `Temporal.Instant.from()`
- `Temporal.Instant.prototype.toString()`
- `Temporal.Instant.prototype.toLocaleString()`

Temporal.Instant.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.Instant` instances returns a string with a language-sensitive representation of this instant. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given instant according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(instant)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking

> spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56Z");

console.log(instant.toLocaleString()); // 8/1/2021, 12:34:56 AM (assuming en-US locale and device in UTC time zone)
```

See also

- `Temporal.Instant`
- `Intl.DateTimeFormat`
- `Temporal.Instant.prototype.toJSON()`
- `Temporal.Instant.prototype.toString()`

Temporal.Instant.prototype.toString()

The **`toString()`** method of `Temporal.Instant` instances returns a string representing this instant in the RFC 9557 format using the specified time zone.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information). Defaults to `"UTC"`.

Return value

A string in the RFC 9557 format representing this instant using the specified time zone. No annotations, such as time zone names, are included.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using toString()

```
const instant = Temporal.Instant.fromEpochMilliseconds(1627814412345);
console.log(instant.toString()); // '2021-08-01T10:40:12.345Z'

// Stringification implicitly calls toString()
console.log(`${instant}`); // '2021-08-01T10:40:12.345Z'
```

Using options

```
const instant = Temporal.Instant.fromEpochMilliseconds(1627814412345);
console.log(instant.toString({ fractionalSecondDigits: 1 })); // '2021-08-01T10:40:12.3Z'
console.log(instant.toString({ smallestUnit: "minute" })); // '2021-08-01T10:40Z'
console.log(instant.toString({ timeZone: "America/New_York" })); // '2021-08-01T06:40:12.345-04:00'
```

```
// The time zone name automatically resolves to the correct offset
// based on the instant; for example, America/New_York is UTC-4 in summer,
// but UTC-5 in winter.
const instant2 = Temporal.Instant.fromEpochMilliseconds(1577836800000);
console.log(instant2.toString({ timeZone: "UTC" })); // '2029-12-31T23:00:00Z'
console.log(instant2.toString({ timeZone: "America/New_York" })); // '2019-12-31T19:00:00-05:00'
```

See also

- `Temporal.Instant`
- `Temporal.Instant.from()`
- `Temporal.Instant.prototype.toJSON()`
- `Temporal.Instant.prototype.toLocaleString()`

Temporal.Instant.prototype.toZonedDateTimeISO()

The **`toZonedDateTimeISO()`** method of `Temporal.Instant` instances returns a new `Temporal.ZonedDateTime` object representing this instant in the specified time zone using the ISO 8601 calendar system.

Syntax

```
toZonedDateTimeISO(timeZone)
```

Parameters

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

A new `Temporal.ZonedDateTime` object representing this instant in the specified time zone using the ISO 8601 calendar system.

Exceptions

`RangeError`

Thrown if the time zone name is invalid.

`TypeError`

Thrown if `timeZone` is not a string or a `Temporal.ZonedDateTime` instance.

Examples

Using toZonedDateTimeISO()

```
const instant = Temporal.Instant.from("2021-08-01T12:34:56.123456789Z");
const zonedDateTime = instant.toZonedDateTimeISO("America/New_York");
console.log(zonedDateTime.toString()); // 2021-08-01T08:34:56.123456789-04:00[America/New_York]

const localDateTime = instant.toZonedDateTimeISO(Temporal.Now.timeZoneId());
console.log(localDateTime.toString()); // This instant in your timezone
```

See also

- `Temporal.Instant`
- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.toInstant()`

Temporal.Instant.prototype.until()

The **`until()`** method of `Temporal.Instant` instances returns a new `Temporal.Duration` object representing the duration from this instant to another instant (in a form convertible by `Temporal.Instant.from()`). The duration is positive if the other instant is after this instant, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

A string or a `Temporal.Instant` instance representing an instant to subtract this instant from. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this instant *until* `other`. The duration is positive if `other` is after this instant, and negative if before.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using until()

```
const launch = Temporal.Instant.fromEpochMilliseconds(2051222400000);
const now = Temporal.Now.instant();
const duration = now.until(launch, { smallestUnit: "minutes" });
console.log(`It will be ${duration.toLocaleString("en-US")} until the launch`);
```

For more examples, see `since()`.

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.add()`
- `Temporal.Instant.prototype.subtract()`
- `Temporal.Instant.prototype.since()`

Temporal.Instant.prototype.valueOf()

The **`valueOf()`** method of `Temporal.Instant` instances throws a `TypeError`, which prevents `Temporal.Instant` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `instant1 > instant2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.Instant` instances prevent such implicit conversions. You need to explicitly convert them to numbers using `Temporal.Instant.prototype.epochNanoseconds`, or use the `Temporal.Instant.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.Instant

All arithmetic and comparison operations on `Temporal.Instant` instances should use the dedicated methods or convert them to primitives explicitly.

```
const instant1 = Temporal.Instant.fromEpochMilliseconds(0);
const instant2 = Temporal.Instant.fromEpochMilliseconds(1000);
instant1 > instant2; // TypeError: can't convert Instant to primitive type
instant1.epochNanoseconds > instant2.epochNanoseconds; // false
Temporal.Instant.compare(instant1, instant2); // -1

instant2 - instant1; // TypeError: can't convert Instant to primitive type
instant2.since(instant1).toString(); // "PT1S"
```

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.toString()`
- `Temporal.Instant.prototype.toJSON()`
- `Temporal.Instant.prototype.toLocaleString()`

Temporal.Now

The **`Temporal.Now`** namespace object contains static methods for getting the current time in various formats.

Description

Unlike most global objects, `Temporal.Now` is not a constructor. You cannot use it with the `new` operator or invoke the `Temporal.Now` object as a function. All properties and methods of `Temporal.Now` are static (just like the `Math` object).

Most fundamentally, the system time is returned by the operating system as a time since the Unix epoch. `Temporal.Now.instant()` returns this time as a `Temporal.Instant` object. Although this object can represent nanoseconds, the resolution of the current time depends on the system clock and browser settings.

An instant can be interpreted in a time zone (which is the system time zone `Temporal.Now.timeZoneId()` by default) in the same fashion as `Temporal.Instant.prototype.toZonedDateTimeISO()`. To get a `Temporal.ZonedDateTime` object, you can use `Temporal.Now.zonedDateTimeISO()`. You can also get different parts of the date and time, using `Temporal.Now.plainDateISO()`, `Temporal.Now.plainTimeISO()`, and `Temporal.Now.plainDateTimeISO()`.

For example, if the computer is set to the time zone "America/New_York", `Temporal.Now.zonedDateTimeISO()` returns a zoned date-time like: `2021-08-01T10:40:12.345-04:00[America/New_York]`. In this case, `Temporal.Now.plainTimeISO()` would return the time part of this zoned date-time: `10:40:12.345`. However, if you call `Temporal.Now.plainTimeISO("UTC")`, it returns the time part of the zoned date-time in the UTC time zone: `14:40:12.345`. This is especially useful for cross-system communication where the other end may be expecting the time in a different time zone.

Reduced time precision

To offer protection against timing attacks and fingerprinting, the precision of the current time returned by the `Temporal.Now` methods may be reduced depending on browser settings.

In Chrome, the rounding interval is 0.1 ms, or 0.005 ms in cross-origin-isolated contexts.

In Firefox, these methods obtain the current time in the same way as `Date.now()`. They inherit the precision of that clock reading without introducing additional inaccuracy. The timestamp is always an integer number of milliseconds, so its resolution is limited to 1 ms in all contexts. Accessing `epochNanoseconds` does not increase its precision.

Static properties

`Temporal.Now[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.Now"`. This property is used in `Object.prototype.toString()`.

Static methods

`Temporal.Now.instant()`

> Returns the current time as a `Temporal.Instant` object.

`Temporal.Now.plainDateISO()`

> Returns the current date as a `Temporal.PlainDate` object, in the ISO 8601 calendar and the specified time zone.

`Temporal.Now.plainDateTimeISO()`

> Returns the current date and time as a `Temporal.PlainDateTime` object, in the ISO 8601 calendar and the specified time zone.

`Temporal.Now.plainTimeISO()`

> Returns the current time as a `Temporal.PlainTime` object, in the specified time zone.

`Temporal.Now.timeZoneId()`

> Returns a time zone identifier representing the system's current time zone.

`Temporal.Now.zonedDateTimeISO()`

> Returns the current date and time as a `Temporal.ZonedDateTime` object, in the ISO 8601 calendar and the specified time zone.

See also

- `Temporal`
- `Temporal.Instant`
- `Temporal.PlainDate`
- `Temporal.PlainDateTime`
- `Temporal.PlainTime`
- `Temporal.ZonedDateTime`

Temporal.Now.instant()

The **`Temporal.Now.instant()`** static method returns the current time as a `Temporal.Instant` object.

Syntax

```
Temporal.Now.instant()
```

Parameters

None.

Return value

A `Temporal.Instant` object representing the current time, with potentially reduced precision.

Examples

Measuring time elapsed

The following example measures two instants in time and calculates the duration between them, and gets the total duration in milliseconds:

```
const start = Temporal.Now.instant();
// Do something that takes time
const end = Temporal.Now.instant();
const duration = end.since(start);
console.log(duration.total("milliseconds"));
```

See also

- `Temporal.Now`
- `Temporal.Instant`

Temporal.Now.plainDateISO()

The **`Temporal.Now.plainDateISO()`** static method returns the current date as a `Temporal.PlainDate` object, in the ISO 8601 calendar and the specified time zone.

Syntax

```
Temporal.Now.plainDateISO()
Temporal.Now.plainDateISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current date in the specified time zone, as a `Temporal.PlainDate` object using the ISO 8601 calendar.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.plainDateISO()

```
// The current date in the system's time zone
const date = Temporal.Now.plainDateISO();
console.log(date); // e.g.: 2021-10-01

// The current date in the "America/New_York" time zone
const dateInNewYork = Temporal.Now.plainDateISO("America/New_York");
console.log(dateInNewYork); // e.g.: 2021-09-30
```

See also

- `Temporal.Now`
- `Temporal.PlainDate`

Temporal.Now.plainDateTimeISO()

The **`Temporal.Now.plainDateTimeISO()`** static method returns the current date and time as a `Temporal.PlainDateTime` object, in the ISO 8601 calendar and the specified time zone.

Syntax

```
Temporal.Now.plainDateTimeISO()
Temporal.Now.plainDateTimeISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset

(see time zones and offsets for more information).

Return value

The current date and time in the specified time zone, as a `Temporal.PlainDateTime` object using the ISO 8601 calendar. Has the same precision as `Temporal.Now.instant()`.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.plainDateTimeISO()

```
// The current date and time in the system's time zone
const dateTime = Temporal.Now.plainDateTimeISO();
console.log(dateTime); // e.g.: 2021-10-01T06:12:34.567890123

// The current date and time in the "America/New_York" time zone
const dateTimeInNewYork = Temporal.Now.plainDateTimeISO("America/New_York");
console.log(dateTimeInNewYork); // e.g.: 2021-09-30T23:12:34.567890123
```

See also

- `Temporal.Now`
- `Temporal.PlainDateTime`

Temporal.Now.plainTimeISO()

The **`Temporal.Now.plainTimeISO()`** static method returns the current time as a `Temporal.PlainTime` object, in the specified time zone.

Note that although the method contains "ISO" in its name, `Temporal.PlainTime` objects do not have associated calendars, as the time format is not calendar-dependent.

Syntax

```
Temporal.Now.plainTimeISO()
Temporal.Now.plainTimeISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current time in the specified time zone, as a `Temporal.PlainTime` object. Has the same precision as `Temporal.Now.instant()`.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.plainTimeISO()

```
// The current time in the system's time zone
const time = Temporal.Now.plainTimeISO();
console.log(time); // e.g.: 06:12:34.567890123

// The current time in the "America/New_York" time zone
const timeInNewYork = Temporal.Now.plainTimeISO("America/New_York");
console.log(timeInNewYork); // e.g.: 23:12:34.567890123
```

See also

- `Temporal.Now`
- `Temporal.PlainTime`

Temporal.Now.timeZoneId()

The **`Temporal.Now.timeZoneId()`** static method returns a time zone identifier representing the system's current time zone. Most systems will return a primary time zone identifier such as `"America/New_York"`, though offset time zone identifier such as `"-04:00"` is possible too. The time zone identifier returned is the default time zone used by the other `Temporal.Now` methods.

Syntax

```
Temporal.Now.timeZoneId()
```

Parameters

None.

Return value

A valid time zone identifier representing the system's current time zone. The returned time zone identifier is never a non-primary time zone identifier (alias). For example, it would always return `"Asia/Kolkata"` (new name) instead of `"Asia/Calcutta"` (old name). For more information, see time zones and offsets.

If the implementation does not support time zones, the method always returns `"UTC"`.

Examples

Getting the system's current time zone

```
console.log(Temporal.Now.timeZoneId()); // e.g.: "America/New_York"
```

See also

- `Temporal.Now`

Temporal.Now.zonedDateTimeISO()

The **`Temporal.Now.zonedDateTimeISO()`** static method returns the current date and time as a `Temporal.ZonedDateTime` object, in the ISO 8601 calendar and the specified time zone.

Syntax

```
Temporal.Now.zonedDateTimeISO()
Temporal.Now.zonedDateTimeISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current date and time in the specified time zone, as a `Temporal.ZonedDateTime` object using the ISO 8601 calendar. Has the same precision as `Temporal.Now.instant()`.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.zonedDateTimeISO()

```
// The current date and time in the system's time zone
const dateTime = Temporal.Now.zonedDateTimeISO();
console.log(dateTime); // e.g.: 2021-10-01T06:12:34.567890123+03:00[Africa/Nairobi]

// The current date and time in the "America/New_York" time zone
const dateTimeInNewYork = Temporal.Now.zonedDateTimeISO("America/New_York");
console.log(dateTimeInNewYork); // e.g.: 2021-09-30T23:12:34.567890123-04:00[America/New_York]
```

See also

- `Temporal.Now`
- `Temporal.ZonedDateTime`

Temporal.PlainDate

The **`Temporal.PlainDate`** object represents a calendar date (a date without a time or time zone); for example, an event on a calendar which happens during the whole day no matter which time zone it's happening in. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system.

Description

A `PlainDate` is essentially the date part of a `Temporal.PlainDateTime` object, with the time information removed. Because the date and time information don't have much interaction, all general information about date properties is documented here.

RFC 9557 format

`PlainDate` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD [u-ca=calendar_id]
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD`

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the time, offset, and time zone identifier, in the same format as `PlainDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure whether to display the calendar ID, and whether to add a critical flag for it.

Invalid date clamping

The `Temporal.PlainDate.from()`, `Temporal.PlainDate.prototype.with()`, `Temporal.PlainDate.prototype.add()`, `Temporal.PlainDate.prototype.subtract()` methods, and their counterparts in other `Temporal` objects, allow constructing dates using calendar-specific properties. The date components may be out of range. In the ISO calendar, this is always an *overflow*, such as the month being greater than 12 or the day being greater than the number of days, and fixing it would only involve clamping the value to the maximum allowed value. In other calendars, the invalid case may be more complex. When using the `overflow: "constrain"` option, invalid dates are fixed to a valid one in the following way:

- If the day doesn't exist but the month does: pick the closest day in the same month. If there are two equally-close dates in that month, pick the later one.
- If the month is a leap month that doesn't exist in the year: pick another date according to the cultural conventions of that calendar's users. Usually this will result in the same day in the month before or after where that month would normally fall in a leap year.
- If the month doesn't exist in the year for other reasons: pick the closest date that is still in the same year. If there are two equally-close dates in that year, pick the later one.
- If the entire year doesn't exist: pick the closest date in a different year. If there are two equally-close dates, pick the later one.

Constructor

`Temporal.PlainDate()`

Creates a new `Temporal.PlainDate` object by directly supplying the underlying data.

Static methods

`Temporal.PlainDate.compare()`

Returns a number (-1, 0, or 1) indicating whether the first date comes before, is the same as, or comes after the second date. Equivalent to comparing the year, month, and day fields of the underlying ISO 8601 dates.

`Temporal.PlainDate.from()`

Creates a new `Temporal.PlainDate` object from another `Temporal.PlainDate` object, an object with date properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainDate.prototype` and shared by all `Temporal.PlainDate` instances.

`Temporal.PlainDate.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainDate.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainDate` instances, the initial value is the `Temporal.PlainDate()` constructor.

`Temporal.PlainDate.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.PlainDate.prototype.dayOfWeek`

Returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. Calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

`Temporal.PlainDate.prototype.dayOfYear`

Returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. Calendar-dependent.

`Temporal.PlainDate.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.PlainDate.prototype.daysInWeek`

Returns a positive integer representing the number of days in the week of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week.

`Temporal.PlainDate.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.PlainDate.prototype.era`

Returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.PlainDate.prototype.eraYear`

Returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.PlainDate.prototype.inLeapYear`

Returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.PlainDate.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.PlainDate.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainDate.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.PlainDate.prototype.weekOfYear`

Returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. Calendar-dependent. Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year.

`Temporal.PlainDate.prototype.year`

Returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.PlainDate.prototype.yearOfWeek`

Returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. Calendar-dependent. Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1.

`Temporal.PlainDate.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainDate"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainDate.prototype.add()`

Returns a new `Temporal.PlainDate` object representing this date moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDate.prototype.equals()`

Returns `true` if this date is equivalent in value to another date (in a form convertible by `Temporal.PlainDate.from()`), and `false` otherwise. They are compared both by their date values and their calendars.

`Temporal.PlainDate.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another date (in a form convertible by `Temporal.PlainDate.from()`) to this date. The duration is positive if the other date is before this date, and negative if after.

`Temporal.PlainDate.prototype.subtract()`

Returns a new `Temporal.PlainDate` object representing this date moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDate.prototype.toJSON()`

Returns a string representing this date in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainDate.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this date.

`Temporal.PlainDate.prototype.toPlainDateTime()`

Returns a new `Temporal.PlainDateTime` object representing this date and a supplied time in the same calendar system.

`Temporal.PlainDate.prototype.toPlainMonthDay()`

Returns a new `Temporal.PlainMonthDay` object representing the `monthCode` and `day` of this date in the same calendar system.

`Temporal.PlainDate.prototype.toPlainYearMonth()`

Returns a new `Temporal.PlainYearMonth` object representing the `year` and `month` of this date in the same calendar system.

`Temporal.PlainDate.prototype.toString()`

Returns a string representing this date in the RFC 9557 format.

`Temporal.PlainDate.prototype.toZonedDateTime()`

Returns a new `Temporal.ZonedDateTime` object representing this date, a supplied time, and a supplied time zone, in the same calendar system.

`Temporal.PlainDate.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this date to another date (in a form convertible by `Temporal.PlainDate.from()`). The duration is positive if the other date is after this date, and negative if before.

`Temporal.PlainDate.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainDate` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainDate.prototype.with()`

Returns a new `Temporal.PlainDate` object representing this date with some fields replaced by new values.

`Temporal.PlainDate.prototype.withCalendar()`

Returns a new `Temporal.PlainDate` object representing this date interpreted in the new calendar system.

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.PlainDateTime`
- `Temporal.PlainMonthDay`
- `Temporal.PlainYearMonth`
- `Temporal.ZonedDateTime`

Temporal.PlainDate.prototype.add()

The **`add()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this date. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object representing the date specified by the original `PlainDate`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

The `duration` is handled in this way:

- Move forward by the number of years, keeping the `monthCode` and `day` the same. If the `monthCode` is invalid in the resulting year (impossible for Gregorian and ISO 8601, but possible for calendars with leap months), we adjust based on the `overflow` option: for `constrain`, we pick another month according to the cultural conventions of that calendar's users. For example, because the leap month is usually thought of as a duplicate of another month, we may pick the month that it is a duplicate of.
- Move forward by the number of months, adjusting the year if necessary, keeping the `day` the same. If the `day` is invalid in the resulting month (e.g., February 30), we adjust based on the `overflow` option: for `constrain`, we pick the closest valid day (e.g., February 28 or 29).
- All commonly supported calendars use fixed-length weeks, so the number of weeks is just converted to the number of days. If the rule is more complex, we may take an approach similar to shifting months.
- For all non-calendar units (days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds), they are converted to the number of days. Fractional part of a day is ignored. Then, we move forward by that number of days, adjusting the month and year if necessary.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration in ISO 8601 calendar

```
const start = Temporal.PlainDate.from("2021-01-01");
const end = start.add({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2022-03-26

const end2 = start.add({ years: -1, months: -2, weeks: -3, days: -4 });
console.log(end2.toString()); // 2019-10-07

const distance = Temporal.PlainDate.from("2020-01-01").until("2021-01-01"); // 366 days
const end3 = start.add(distance);
console.log(end3.toString()); // 2022-01-02
```

Adding a duration in a non-ISO calendar

```
const start = Temporal.PlainDate.from("2021-01-01[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 11/18/2020
const end = start.add({ months: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 12/18/2020
```

Adding a duration with overflow

If we move a few months and the corresponding day is invalid in this month, then we adjust the day based on the `overflow` option.

```
const start = Temporal.PlainDate.from("2021-01-31");
const end = start.add({ months: 1 });
console.log(end.toString()); // 2021-02-28

// Any further day additions are based on the clamped month-day:
const end2 = start.add({ months: 1, days: 31 });
console.log(end2.toString()); // 2021-03-31

// Compare with the same addition in a different order that results in no overflow:
const end3 = start.add({ days: 31 }).add({ months: 1 });
console.log(end3.toString()); // 2021-04-03
```

Overflow can also occur for the month, for calendars where different years have different numbers of months (usually due to leap months).

```
const start = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 2bis/11/2023; "bis" means leap month
```

```
const end = start.add({ years: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 3/11/2024

// Compare:
const start = Temporal.PlainDate.from("2023-04-30[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 3/11/2023
const end = start.add({ years: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 3/11/2024; same day as above!
```

Note that the following is not an overflow because the month can just increment:

```
const start = Temporal.PlainDate.from("2021-01-01");
const end = start.add({ days: 100 });
console.log(end.toString()); // 2021-04-11
```

You can also throw an error if the date component is out of range:

```
const start = Temporal.PlainDate.from("2021-01-31");
const end = start.add({ months: 1 }, { overflow: "reject" }); // RangeError: date value "day" not in 1..28: 31

const start = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
const end = start.add({ years: 1 }, { overflow: "reject" }); // RangeError: invalid "monthCode" calendar field: M02L
```

Adding time durations

Fractional parts of a day are ignored.

```
const start = Temporal.PlainDate.from("2021-01-01");
const end = start.add({ hours: 25 }); // Same as adding 1 day
console.log(end.toString()); // 2021-01-02
```

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.subtract()`

Temporal.PlainDate.prototype.calendarId

The **`calendarId`** accessor property of `Temporal.PlainDate` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. Use the `withCalendar()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using calendarId

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.calendarId); // "iso8601"; default

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.calendarId); // "chinese"

const date3 = date2.withCalendar("hebrew");
console.log(date3.calendarId); // "hebrew"
```

See also

- `Temporal.PlainDate`

Temporal.PlainDate.compare()

The **`Temporal.PlainDate.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first date comes before, is the same as, or comes after the second date. Equivalent to comparing the year, month, and day fields of the underlying ISO 8601 dates.

Syntax

```
Temporal.PlainDate.compare(date1, date2)
```

Parameters

`date1`

> A string, an object, or a `Temporal.PlainDate` instance representing the first date to compare. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`.

`date2`

> The second date to compare, converted to a `Temporal.PlainDate` object using the same algorithm as `date1`.

Return value

Returns `-1` if `date1` comes before `date2`, `0` if they are the same, and `1` if `date1` comes after `date2`. They are compared by their underlying date values, ignoring their calendars.

Examples

Using Temporal.PlainDate.compare()

```
const date1 = Temporal.PlainDate.from("2021-08-01");
const date2 = Temporal.PlainDate.from("2021-08-02");
console.log(Temporal.PlainDate.compare(date1, date2)); // -1

const date3 = Temporal.PlainDate.from("2021-07-31");
console.log(Temporal.PlainDate.compare(date1, date3)); // 1
```

Comparing dates in different calendars

```
const date1 = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const date2 = Temporal.PlainDate.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "islamic-umalqura",
});
const date3 = Temporal.PlainDate.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "hebrew",
});
console.log(date1.toString()); // "2021-08-01"
console.log(date2.toString()); // "2582-12-17[u-ca=islamic-umalqura]"
console.log(date3.toString()); // "-001739-04-06[u-ca=hebrew]"
console.log(Temporal.PlainDate.compare(date1, date2)); // -1
console.log(Temporal.PlainDate.compare(date1, date3)); // 1
```

Sorting an array of dates

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const dates = [
  Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 }),
  Temporal.PlainDate.from({
    year: 2021,
    month: 8,
    day: 1,
    calendar: "islamic-umalqura",
  }),
  Temporal.PlainDate.from({ year: 2021, month: 8, day: 1, calendar: "hebrew" }),
];

dates.sort(Temporal.PlainDate.compare);
console.log(dates.map((d) => d.toString()));
// [ "-001739-04-06[u-ca=hebrew]", "2021-08-01", "2582-12-17[u-ca=islamic-umalqura]" ]
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.equals()`

Temporal.PlainDate.prototype.day

The **day** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

It generally starts at 1 and is continuous, but not always. If you want to loop through all the days in a month, first use `with()` with `{ day: 1 }` (which sets to the beginning of the month, even if the actual number is not `1`), then repeatedly use `add()` with `{ days: 1 }`, until the month changes.

> **Note:** Usually, the day index only changes when transitioning from one calendar system into another, such as from the Julian to the Gregorian calendar. In practice, all currently built-in calendars are proleptic, meaning the calendar system is extended indefinitely into the past and future. Assuming `day` is non-continuous guards against future introductions of non-proleptic calendars.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using day

```
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.day); // 1

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.day); // 22; it is May 22 in the Chinese calendar
```

Looping through all days in a month

```
const month = Temporal.PlainDate.from("2021-07-14"); // An arbitrary date in the month
for (
  let day = month.with({ day: 1 });
  day.month === month.month;
  day = day.add({ days: 1 })
) {
  console.log(day.day);
}
```

Changing day

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.with({ day: 15 });
console.log(newDate.toString()); // 2021-07-15
```

You can also use `add()` or `subtract()` to move a certain number of days from the current date.

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.add({ days: 14 });
console.log(newDate.toString()); // 2021-07-15
```

By default, `with()` constrains the day to the range of valid values. So you can use `{ day: 1 }` to set the day to the first day of the month, even if the first day does not have the number `1`. Similarly, the following will set the day to the last day of the month:

```
const date = Temporal.PlainDate.from("2021-07-01");
const lastDay = date.with({ day: Number.MAX_VALUE }); // 2021-07-31
```

> **Note:** Avoid using `daysInMonth` to set the day to the last day of the month. The last day of the month is not always the same as the number of days in the month, in the rare case where a month may have a few days skipped.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.dayOfYear`

Temporal.PlainDate.prototype.dayOfWeek

The **`dayOfWeek`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. It is calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

All commonly supported calendars use 7-day weeks, and you could generally expect this property to return the same value for the same date across different calendars.

The set accessor of `dayOfWeek` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDate` object with the desired new `dayOfWeek` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

Examples

Using dayOfWeek

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.dayOfWeek); // 4; Thursday

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.dayOfWeek); // 4
```

Changing dayOfWeek

`PlainDate` does not support changing `dayOfWeek` directly. To change the day of the week, you need to first figure out the difference in days to your desired day of the week, then use `add` or `subtract` to adjust the date accordingly. For example, to change to the Friday of this week (whether before or after):

```
function getDayInSameWeek(date, destDayOfWeek) {
  return date.add({ days: destDayOfWeek - date.dayOfWeek });
}

console.log(
  getDayInSameWeek(Temporal.PlainDate.from("2021-07-01"), 5).toString(),
); // 2021-07-02
console.log(
  getDayInSameWeek(Temporal.PlainDate.from("2021-07-03"), 5).toString(),
); // 2021-07-02
```

To change to the next Friday:

```
function getNextDayInWeek(date, destDayOfWeek) {
  const distance = destDayOfWeek - date.dayOfWeek;
  return date.add({
    days: distance < 0 ? date.daysInWeek + distance : distance,
  });
}

console.log(
  getNextDayInWeek(Temporal.PlainDate.from("2021-07-01"), 5).toString(),
); // 2021-07-02
console.log(
  getNextDayInWeek(Temporal.PlainDate.from("2021-07-03"), 5).toString(),
); // 2021-07-09
```

To change to the previous Friday:

```
function getPreviousDayInWeek(date, destDayOfWeek) {
  const distance = date.dayOfWeek - destDayOfWeek;
  return date.subtract({
    days: distance < 0 ? date.daysInWeek + distance : distance,
  });
}

console.log(
  getPreviousDayInWeek(Temporal.PlainDate.from("2021-07-01"), 5).toString(),
); // 2021-06-25
console.log(
  getPreviousDayInWeek(Temporal.PlainDate.from("2021-07-03"), 5).toString(),
); // 2021-07-02
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.dayOfYear`
- `Temporal.PlainDate.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.weekOfYear`
- `Temporal.PlainDate.prototype.yearOfWeek`

Temporal.PlainDate.prototype.dayOfYear

The **`dayOfYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. It is calendar-dependent.

The set accessor of `dayOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDate` object with the desired new `dayOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

Examples

Using dayOfYear

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.dayOfYear); // 182

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.dayOfYear); // 140

const date3 = Temporal.PlainDate.from("2020-07-01");
console.log(date3.dayOfYear); // 183; 2020 is a leap year
```

Changing dayOfYear

`PlainDate` does not support changing `dayOfYear` directly. To change the day of the year, you need to first figure out the difference in days to your desired day of the year, then use `add` or `subtract` to adjust the date accordingly. For example, to change to the 100th day of this year (whether before or after):

```
function getDayInSameYear(date, destDayOfYear) {
  return date.add({ days: destDayOfYear - date.dayOfYear });
}

console.log(
  getDayInSameYear(Temporal.PlainDate.from("2021-07-01"), 100).toString(),
); // 2021-04-10
console.log(
  getDayInSameYear(Temporal.PlainDate.from("2021-01-01"), 100).toString(),
); // 2021-04-10
console.log(
  getDayInSameYear(Temporal.PlainDate.from("2020-01-01"), 100).toString(),
); // 2020-04-09
```

By default, `with()` constrains the day to the range of valid values. So you can always use `{ month: 1, day: 1 }` to set the day to the first day of the year, even if the first day does not have the number `1`. Similarly, the following will set the day to the last day of the year, regardless of how many days are in the last month or year:

```
const date = Temporal.PlainDate.from("2021-07-01");
const lastDay = date.with({ month: Number.MAX_VALUE, day: Number.MAX_VALUE }); // 2021-12-31
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDate.prototype.daysInMonth

The **`daysInMonth`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

Note that the days in month is not always equal to the `day` of the last day of the month, in the rare case where a month may have a few days skipped.

The set accessor of `daysInMonth` is `undefined`. You cannot change this property directly.

Examples

Using daysInMonth

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.daysInMonth); // 31

const date2 = Temporal.PlainDate.from("2021-02-01");
console.log(date2.daysInMonth); // 28; 2021 is not a leap year

const date3 = Temporal.PlainDate.from("2020-02-01");
console.log(date3.daysInMonth); // 29; 2020 is a leap year

const date4 = Temporal.PlainDate.from("2021-04-01[u-ca=chinese]");
console.log(date4.month); // 2
console.log(date4.daysInMonth); // 30; the Chinese 2nd month has 30 days
```

Changing to the second last day of the month

You can use `daysInMonth` to change to the second last day of the month:

```
const date = Temporal.PlainDate.from("2021-07-01");
const secondLastDay = date.with({ day: date.daysInMonth - 1 });
console.log(secondLastDay.toString()); // 2021-07-30
```

This is not totally safe, though, because `daysInMonth` is not guaranteed to have any connection with the day index. Here's a safer way to get the second last day:

```
const date = Temporal.PlainDate.from("2021-07-01");
const secondLastDay = date
  .with({ day: Number.MAX_SAFE_INTEGER })
  .subtract({ days: 1 });
console.log(secondLastDay.toString()); // 2021-07-30
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`

- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDate.prototype.daysInWeek

The **`daysInWeek`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of days in the week of this date. It is calendar-dependent.

For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week. All commonly supported calendars use 7-day weeks.

The set accessor of `daysInWeek` is `undefined`. You cannot change this property directly.

Examples

Using daysInWeek

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.daysInWeek); // 7

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.daysInWeek); // 7
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.weekOfYear`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDate.prototype.daysInYear

The **`daysInYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

For the ISO 8601 calendar, this is 365, or 366 in a leap year. In other calendar systems, it likely differs, especially in non-solar calendars.

The set accessor of `daysInWeek` is `undefined`. You cannot change this property directly.

Examples

Using daysInYear

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.daysInYear); // 365

const date2 = Temporal.PlainDate.from("2020-07-01");
console.log(date2.daysInYear); // 366; 2020 is a leap year

const date3 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date3.daysInYear); // 354

const date4 = Temporal.PlainDate.from("2023-07-01[u-ca=chinese]");
console.log(date4.daysInYear); // 384; 2023 is a Chinese leap year
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.dayOfYear`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.daysInWeek`

Temporal.PlainDate.prototype.equals()

The **`equals()`** method of `Temporal.PlainDate` instances returns `true` if this date is equivalent in value to another date (in a form convertible by `Temporal.PlainDate.from()`), and `false` otherwise. They are compared both by their date values and their calendars, so two dates from different calendars may be considered equal by `Temporal.PlainDate.compare()` but not by `equals()`.

Syntax

```
equals(other)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainDate` instance representing the other date to compare. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`.

Return value

`true` if this date is equal to `other` both in their date value and their calendar, `false` otherwise.

Examples

Using equals()

```
const date1 = Temporal.PlainDate.from("2021-08-01");
const date2 = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
console.log(date1.equals(date2)); // true

const date3 = Temporal.PlainDate.from("2021-08-01[u-ca=japanese]");
console.log(date1.equals(date3)); // false

const date4 = Temporal.PlainDate.from("2021-08-02");
console.log(date1.equals(date4)); // false
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.compare()`

Temporal.PlainDate.prototype.era

The **`era`** accessor property of `Temporal.PlainDate` instances returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

Value

All specified calendars have eras fully defined by the spec.

- The following calendars have a single era:
  - `buddhist`: `"be"`
  - `coptic`: `"am"`
  - `ethioaa`: `"aa"`
  - `hebrew`: `"am"`
  - `indian`: `"shaka"`
  - `persian`: `"ap"`
- The following calendars have two eras. One is the *epoch era*, in which `eraYear` starts at 1 and is the same as `year`. The other is the inverse era, in which `eraYear` also starts at 1 and is equal to `1 - year` (so `eraYear: 1` corresponds to year `0`, `eraYear: 2` to year `-1`, etc.):
  - `gregory`: epoch era `"ce"`, inverse era `"bce"`
  - `islamic-civil`, `islamic-tbla`, `islamic-umalqura`: epoch era `"ah"`, inverse era `"bh"`
  - `roc`: epoch era `"roc"`, inverse era `"broc"`
- The `ethiopic` calendar has an `"am"` era which is the epoch era. Years before `1` belong to the `"aa"` era, whose `eraYear` is equal to `year - 5500` (so `eraYear: -1000` corresponds to year `-6500`, `eraYear: 1` corresponds to year `-5499`, up to `eraYear: 5500` as year `0`).
- The `japanese` calendar adds an era for each new emperor, so the output year and era for a future date may not match the input year and era when your code runs on a future engine version, and we won't enumerate them here. Each era's year starts at 1. It is also the only calendar known to have eras starting in the middle of a year, which means that the same `year` may correspond to different `(era, eraYear)` pairs depending on the month and day.

  > **Warning:** As of October 2025, in the `japanese` calendar, dates prior to 1868-10-23 ISO (the start date of the year 1 Meiji) don't work as expected in browsers in two ways. First, CLDR had the wrong start date for the Meiji era, which causes calendar implementations to extend the Meiji era further to the past than it actually did. Second, the upcoming Intl era and monthCode Proposal specifies that dates prior to 1873-01-01 ISO should use Gregorian eras, but browsers have traditionally used approximations of prior Japanese eras instead. The `japanese` calendar was taken into use on January 1, 6 Meiji / 1873-01-01 ISO, so these problems only affect proleptic dates.
- Other specified calendars: `chinese`, `dangi`, `iso8601`, don't use eras and return `undefined`.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value. When setting eras, the `"ad"` and `"bc"` aliases are also accepted for the `"ce"` and `"bce"` eras of the `gregory` or `japanese` calendars.

> **Note:** This string is not intended for display to users. Use `toLocaleString()` with the appropriate options to get a localized string.

Examples

Using era

```
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.era); // undefined

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
console.log(date2.era); // ce

const date3 = Temporal.PlainDate.from("-002021-07-01[u-ca=gregory]");
console.log(date3.era); // bce

const date4 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(date4.era); // reiwa
```

Changing era

You can only set `era` for calendars that support them. For example, the ISO 8601 calendar does not have eras. Note that you must provide `era` and `eraYear` together.

```
const date = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
const newDate = date.with({ era: "bc", eraYear: 100 });
console.log(newDate.toString()); // -000099-07-01[u-ca=gregory]

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
const newDate2 = date2.with({ era: "meiji", eraYear: 1 });
console.log(newDate2.toString()); // 1868-07-01[u-ca=japanese]
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.eraYear`

Temporal.PlainDate.prototype.eraYear

The **`eraYear`** accessor property of `Temporal.PlainDate` instances returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

Unlike `year`, `era` and `eraYear` may change in the middle of a calendar year. For example, Japan started the Reiwa era on May 1, 2019, so dates from 2019-01-01 to 2019-04-30 have `{ era: "heisei", eraYear: 31 }`, and dates from 2019-05-01 onwards have `{ era: "reiwa", eraYear: 1 }`, but the `year` is always 2019 (because the Japanese calendar uses the ISO 8601 year as the arithmetic year).

For more information about the values of `era` and `eraYear` for different calendars, see the `era` property.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using eraYear

```
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.eraYear); // undefined

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
console.log(date2.eraYear); // 2021

const date3 = Temporal.PlainDate.from("-002021-07-01[u-ca=gregory]");
console.log(date3.eraYear); // 2022; 0000 is used for the year 1 BC

const date4 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(date4.eraYear); // 3
```

Changing eraYear

You can only set `eraYear` for calendars that support them. For example, the ISO 8601 calendar does not have eras. Note that you must provide `era` and `eraYear` together.

```
const date = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
const newDate = date.with({ era: "bc", eraYear: 100 });
console.log(newDate.toString()); // -000099-07-01[u-ca=gregory]

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
const newDate2 = date2.with({ era: "meiji", eraYear: 1 });
console.log(newDate2.toString()); // 1868-07-01[u-ca=japanese]
```

See also

- `Temporal.PlainDate`

- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.era`

Temporal.PlainDate.from()

The **`Temporal.PlainDate.from()`** static method creates a new `Temporal.PlainDate` object from another `Temporal.PlainDate` object, an object with date properties, or an RFC 9557 string.

Syntax

```
Temporal.PlainDate.from(info)
Temporal.PlainDate.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainDate` instance, which creates a copy of the instance.
- A `Temporal.PlainDateTime` instance, which provides the calendar date in the same fashion as `Temporal.PlainDateTime.prototype.toPlainDate()`.
- A `Temporal.ZonedDateTime` instance, which provides the calendar date in the same fashion as `Temporal.ZonedDateTime.prototype.toPlainDate()`.
- An RFC 9557 string containing a date and optionally a calendar.
- An object containing the following properties (in the order they are retrieved and validated):

  `calendar` Optional

  A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. All other properties are interpreted in this calendar system (unlike the `Temporal.PlainDate()` constructor, which interprets the values in the ISO calendar system).

  `day`

  An integer that corresponds to the `day` property. Must be positive regardless of the `overflow` option.

  `era` and `eraYear`

  A string and an integer that correspond to the `era` and `eraYear` properties. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. At least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

  `month`

  Corresponds to the `month` property. Must be positive regardless of the `overflow` option. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

  `monthCode`

  Corresponds to the `monthCode` property. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

  `year`

  Corresponds to the `year` property. At least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

  The info should explicitly specify a year (as `year` or `era` and `eraYear`), a month (as `month` or `monthCode`), and a day.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object, representing the date specified by `info` in the specified `calendar`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month` (or `monthCode`), and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainDate from an object

```
// Year, month, and day
const d1 = Temporal.PlainDate.from({ year: 2021, month: 7, day: 1 });
console.log(d1.toString()); // "2021-07-01"

// Year, month code, and day
const d2 = Temporal.PlainDate.from({ year: 2021, monthCode: "M07", day: 1 });
console.log(d2.toString()); // "2021-07-01"

// Year, month, day in a different calendar
const d3 = Temporal.PlainDate.from({
  year: 2021,
  month: 7,
  day: 1,
  calendar: "hebrew",
});
// Note: when you construct a date with an object, the date components
// are in *that* calendar, not the ISO calendar. However, toString() always
// outputs the date in the ISO calendar. For example, the year "2021" in
// the Hebrew calendar is actually 1740 BCE in the ISO calendar.
console.log(d3.toString()); // "-001739-03-07[u-ca=hebrew]"

// Era, eraYear, month, and day
const d4 = Temporal.PlainDate.from({
  era: "meiji",
  eraYear: 4,
  month: 7,
  day: 1,
  calendar: "japanese",
});
console.log(d4.toString()); // "1871-07-01[u-ca=japanese]"
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range:

```
const d1 = Temporal.PlainDate.from({ year: 2021, month: 13, day: 1 });
console.log(d1.toString()); // "2021-12-01"

const d2 = Temporal.PlainDate.from({ year: 2021, month: 2, day: 29 });
console.log(d2.toString()); // "2021-02-28"

const d3 = Temporal.PlainDate.from("2021-02-29");
console.log(d3.toString()); // "2021-02-28"
```

You can change this behavior to throw an error instead:

```
const d3 = Temporal.PlainDate.from(
  { year: 2021, month: 13, day: 1 },
  { overflow: "reject" },
);
// RangeError: date value "month" not in 1..12: 13
```

Creating a PlainDate from a string

```
const d = Temporal.PlainDate.from("2021-07-01");
console.log(d.toLocaleString("en-US", { dateStyle: "full" }));
// Thursday, July 1, 2021

// Providing a calendar
const d2 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
```

```
console.log(
  d2.toLocaleString("ja-JP", { calendar: "japanese", dateStyle: "full" }),
);
// 令和3年7月1日木曜日

// Providing a time and an offset (ignored)
const d3 = Temporal.PlainDate.from("2021-07-01T00:00+08:00");
console.log(d3.toString()); // "2021-07-01"
```

Creating a PlainDate from another Temporal instance

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:00");
const d = Temporal.PlainDate.from(dt);
console.log(d.toString()); // "2021-07-01"

const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T00:00+08:00[Asia/Shanghai]",
);
const d2 = Temporal.PlainDate.from(zdt);
console.log(d2.toString()); // "2021-07-01"

const d3 = Temporal.PlainDate.from(d);
console.log(d3.toString()); // "2021-07-01"
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate()`
- `Temporal.PlainDate.prototype.with()`

Temporal.PlainDate.prototype.inLeapYear

The **`inLeapYear`** accessor property of `Temporal.PlainDate` instances returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

For the ISO 8601 calendar, a leap year is a year that is evenly divisible by 4, except for years that are evenly divisible by 100, unless the year is also evenly divisible by 400. For the ISO 8601 calendar, leap years have 366 days, while common years have 365 days. For other calendar systems, the rules likely differ, and leap years may have more days added (such as a leap month).

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

Examples

Using inLeapYear

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.inLeapYear); // false
console.log(date.daysInYear); // 365
console.log(date.monthsInYear); // 12

const date2 = Temporal.PlainDate.from("2020-07-01");
console.log(date2.inLeapYear); // true
console.log(date2.daysInYear); // 366
console.log(date2.monthsInYear); // 12

const date3 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date3.inLeapYear); // false
console.log(date3.daysInYear); // 354
console.log(date3.monthsInYear); // 12

const date4 = Temporal.PlainDate.from("2023-07-01[u-ca=chinese]");
console.log(date4.inLeapYear); // true
console.log(date4.daysInYear); // 384
console.log(date4.monthsInYear); // 13
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.daysInYear`
- `Temporal.PlainDate.prototype.monthsInYear`

Temporal.PlainDate.prototype.month

The **`month`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

> **Note:** Do not use this property to identify the actual month, including its name. Use `monthCode` for that purpose. Use `month` only for identifying months within the context of a year, or to figure out their order.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using month

```
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7

const date2 = Temporal.PlainDate.from("2021-05-01[u-ca=chinese]");
console.log(date2.monthCode); // "M03"
console.log(date2.month); // 3; it is March 20 in the Chinese calendar

const date3 = Temporal.PlainDate.from("2023-05-01[u-ca=chinese]");
console.log(date3.monthCode); // "M03"
console.log(date3.month); // 4, although it is also March (M03)!

const date4 = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
console.log(date4.monthCode); // "M02L"
console.log(date4.month); // 3, this month is a leap month, i.e. a duplicate February
```

Looping through all months in a year

```
const year = Temporal.PlainDate.from("2021-07-14"); // An arbitrary date in the year
for (
  let month = year.with({ month: 1 });
  month.year === year.year;
  month = month.add({ months: 1 })
) {
  console.log(month.month);
}
```

Alternatively, this is also a safe way (unlike the day example):

```
for (let month = 1; month <= year.monthsInYear; month++) {
  const monthDate = year.with({ month });
}
```

Changing month

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.with({ month: 2 });
console.log(newDate.toString()); // 2021-02-01
```

You can also use `add()` or `subtract()` to move a certain number of months from the current date.

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.add({ months: 3 });
console.log(newDate.toString()); // 2021-10-01
```

By default, `with()` constrains the day to the range of valid values. Both of the following will set the month to the last month of the year:

```
const date = Temporal.PlainDate.from("2021-07-01");
const lastMonth = date.with({ month: date.monthsInYear }); // 2021-12-01
const lastMonth2 = date.with({ month: Number.MAX_VALUE }); // 2021-12-01
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.monthCode`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

Temporal.PlainDate.prototype.monthCode

The **`monthCode`** accessor property of `Temporal.PlainDate` instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Value

The basic format of `monthCode` is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`).

All calendars have at least 12 months, with codes from `"M01"` to `"M12"`.

All specified calendars have month codes fully defined by the spec. Most don't have month rules distinct from `iso8601`. The `coptic`, `ethioaa`, and `ethiopic` calendars have an additional `M13` month. The `chinese` and `dangi` calendars have 12 additional leap months possible, with codes from `"M01L"` to `"M12L"`. The `hebrew` calendar has one leap month, `"M05L"` (Adar I).

> **Note:** Don't assume that `monthCode` is a user-friendly string; use `toLocaleString()` to format your date instead. Generally, don't cache the name of months in an array or object. Even though `monthCode` usually maps to the month's name within one calendar, we recommend always computing the month's name using, for example, `date.toLocaleString("en-US", { calendar: date.calendarId, month: "long" })`.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

When setting the date to a different year, the `monthCode` remains the same, but the `month` may change if the target year has a different leap month structure. If the current `monthCode` does not exist in the target year and the method is not configured to reject, then for the `chinese` and `dangi` calendars, the previous month is used instead (e.g., from `"M03L"` to `"M03"`, which is from 闰三月 to 三月). For `hebrew`, the *next* month is used instead (from `"M05L"` to `"M06"`, which is from Adar I to Adar II).

Examples

Using monthCode

```
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7

const date2 = Temporal.PlainDate.from("2021-05-01[u-ca=chinese]");
console.log(date2.monthCode); // "M03"
console.log(date2.month); // 3; it is March 20 in the Chinese calendar

const date3 = Temporal.PlainDate.from("2023-05-01[u-ca=chinese]");
console.log(date3.monthCode); // "M03"
console.log(date3.month); // 4, although it is also March (M03)!

const date4 = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
console.log(date4.monthCode); // "M02L"
console.log(date4.month); // 3, this month is a leap month, i.e. a duplicate February
```

Changing monthCode

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.with({ month: 2 });
console.log(newDate.toString()); // 2021-02-01
```

You can also use `add()` or `subtract()` to move a certain number of months from the current date.

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.add({ months: 3 });
console.log(newDate.toString()); // 2021-10-01
```

By default, `with()` constrains the day to the range of valid values. Both of the following will set the month to the last month of the year:

```
const date = Temporal.PlainDate.from("2021-07-01");
const lastMonth = date.with({ month: date.monthsInYear }); // 2021-12-01
const lastMonth2 = date.with({ month: Number.MAX_VALUE }); // 2021-12-01
```

Formatting month names

Don't do this:

```
const names = [
  "January", "February", "March", "April", "May", "June",
  "July", "August", "September", "October", "November", "December",
];

const date = Temporal.PlainDate.from("2021-07-01");
console.log(names[date.month - 1]); // July
```

Also don't do this:

```
const names = {
  "M01": "January", "M02": "February", "M03": "March", "M04": "April",
  "M05": "May", "M06": "June", "M07": "July", "M08": "August",
  "M09": "September", "M10": "October", "M11": "November", "M12": "December",
};
```

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(names[date.monthCode]); // July
```

Instead, always do this, which is more user-friendly and less error-prone, and easily generalizes to other calendars:

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(
  date.toLocaleString("en-US", { calendar: date.calendarId, month: "long" }),
); // July
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

Temporal.PlainDate.prototype.monthsInYear

The **`monthsInYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ. For example, in calendars using leap months, leap years will have one more month than common years.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

Examples

Using monthsInYear

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.monthsInYear); // 12

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.monthsInYear); // 12

const date3 = Temporal.PlainDate.from("2023-07-01[u-ca=chinese]");
console.log(date3.monthsInYear); // 13; 2023 is a Chinese leap year
```

Changing to the second last month of the year

You can use `monthsInYear` to change to the second last month of the year:

```
const date = Temporal.PlainDate.from("2021-07-01");
const secondLastMonth = date.with({ month: date.monthsInYear - 1 });
console.log(secondLastMonth.toString()); // 2021-11-01
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.monthCode`
- `Temporal.PlainDate.prototype.daysInMonth`

Temporal.PlainDate() constructor

The **`Temporal.PlainDate()`** constructor creates `Temporal.PlainDate` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainDate` objects using the `Temporal.PlainDate.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.PlainDate(year, month, day)
new Temporal.PlainDate(year, month, day, calendar)
```

> **Note:** `Temporal.PlainDate()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`year`

A number, truncated to an integer, representing the year in the ISO calendar system.

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`day`

A number, truncated to an integer, representing the day of the month in the ISO calendar system.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `year`, `month`, and `day` must be in the ISO 8601 calendar system.

Return value

A new `Temporal.PlainDate` object, representing the date specified by `year`, `month`, `day` (in the ISO calendar), interpreted in the calendar system specified by `calendar`.

Exceptions

`TypeError`

Thrown if `calendar` is not a string or `undefined`.

`RangeError`

Thrown in one of the following cases:

- `year`, `month`, or `day` is not a finite number.
- The `year`, `month`, and `day` combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainDate()

```
const plainDate = new Temporal.PlainDate(2021, 7, 1);
console.log(plainDate.toString()); // 2021-07-01

// Note that the date is stored internally as ISO 8601, even when it's
// interpreted in a different calendar system. For example, even though
// 2021-07-01 ISO is 5781-10-21 in the Hebrew calendar, you still pass the
// ISO date to the constructor.
const plainDate2 = new Temporal.PlainDate(2021, 7, 1, "hebrew");
console.log(plainDate2.toString()); // 2021-07-01[u-ca=hebrew]
console.log(plainDate2.year); // 5781
console.log(plainDate2.month); // 10
console.log(plainDate2.day); // 21
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.from()`

Temporal.PlainDate.prototype.since()

The **`since()`** method of `Temporal.PlainDate` instances returns a new `Temporal.Duration` object representing the duration from another date (in a form convertible by `Temporal.PlainDate.from()`) to this date. The duration is positive if the other date is before this date, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDate` instance representing a date to subtract from this date. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`. It must have the same calendar as `this`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"years"`, `"months"`, `"weeks"`, `"days"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"days"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"days"`. The current date is used as the `relativeTo` option. Note that using units larger than `"days"` may make the duration not portable to other calendars or dates.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this date. The duration is positive if `other` is before this date, and negative if after.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using since()

```
const date = Temporal.PlainDate.from("2022-12-25");
const now = Temporal.Now.plainDateISO();
const duration = now.since(date);
const formatter = new Intl.DurationFormat("en-US", { style: "long" });
console.log(`It's been ${formatter.format(duration)} since that Christmas...`);
// Expected output: "It's been [number] days since that Christmas..."

const duration2 = now.since(date, { smallestUnit: "months" });
console.log(`It's been ${formatter.format(duration2)} since that Christmas...`);
// Expected output: "It's been [number] months since that Christmas..."

const duration3 = now.since(date, {
  largestUnit: "years",
  smallestUnit: "months",
});
console.log(`It's been ${formatter.format(duration3)} since that Christmas...`);
// Expected output: "It's been [number] years, [number] months since that Christmas..."
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```
const date1 = Temporal.PlainDate.from("2022-01-01");
const date2 = Temporal.PlainDate.from("2022-01-28");
const duration = date2.since(date1, {
  smallestUnit: "days",
  roundingIncrement: 5,
  roundingMode: "ceil",
});
console.log(duration.toString()); // "P30D"
```

Comparing different calendars

By default, the two dates must have the same calendar. This is to avoid ambiguity in the meaning of months and years. If you want to compare dates from different calendars, you can convert them to the same calendar first.

```
const date1 = Temporal.PlainDate.from("2022-01-01");
const date2 = Temporal.PlainDate.from("2022-01-28[u-ca=chinese]");
const duration = date2.withCalendar("iso8601").since(date1);
console.log(duration.toString()); // "P27D"
```

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.until()`

Temporal.PlainDate.prototype.subtract()

The **`subtract()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two dates and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this date. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object representing the date specified by the original `PlainDate`, minus the duration.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Subtracting a duration

```
const start = Temporal.PlainDate.from("2022-01-01");
const end = start.subtract({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2020-10-07
```

For more examples, see `add()`.

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.since()`
- `Temporal.PlainDate.prototype.until()`

Temporal.PlainDate.prototype.toJSON()

The **`toJSON()`** method of `Temporal.PlainDate` instances returns a string representing this date in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given date in the RFC 9557 format, with the calendar annotation included if it is not `"iso8601"`.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.PlainDate` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.PlainDate` objects during JSON serialization, which can then be deserialized using the `Temporal.PlainDate.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const date = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const dateStr = date.toJSON(); // '2021-08-01'
const d2 = Temporal.PlainDate.from(dateStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainDate` can be serialized as JSON without extra effort, and how to parse it back.

```
const date = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const jsonStr = JSON.stringify({ date }); // '{"date":"2021-08-01"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "date") {
    return Temporal.PlainDate.from(value);
  }
  return value;
});
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.toString()`
- `Temporal.PlainDate.prototype.toLocaleString()`

Temporal.PlainDate.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.PlainDate` instances returns a string with a language-sensitive representation of this date. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If this date's calendar is not `"iso8601"`, the `calendar` option must be provided with the same value; otherwise, if this date's calendar is `"iso8601"`, the `calendar` option can be any value. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year`, `month`, and `day` will default to `"numeric"`.
- Provide `dateStyle` only: it expands to `weekday`, `era`, `year`, `month`, and `day` formats.
- Provide some date-time component options, where at least one of them is a date option (`weekday`, `year`, `month`, `day`). Only the specified date components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const date = Temporal.PlainDate.from("2021-08-01");

console.log(date.toLocaleString()); // 8/1/2021 (assuming en-US locale)
```

If the date's calendar doesn't match the locale's default calendar, and the date's calendar is not `iso8601`, an explicit `calendar` option must be provided with the same value.

```
const date = Temporal.PlainDate.from("2021-08-01[u-ca=japanese]");
// The ja-JP locale uses the Gregorian calendar by default
date.toLocaleString("ja-JP", { calendar: "japanese" }); // R3/8/1
```

Using toLocaleString() with options

You can customize which parts of the date are included in the output by providing the `options` parameter.

```
const date = Temporal.PlainDate.from("2021-08-01");
date.toLocaleString("en-US", { dateStyle: "full" }); // Sunday, August 1, 2021
date.toLocaleString("en-US", {
  year: "numeric",
  month: "long",
  day: "numeric",
}); // August 1, 2021
date.toLocaleString("en-US", { year: "numeric", month: "long" }); // August 2021
```

See also

- `Temporal.PlainDate`
- `Intl.DateTimeFormat`
- `Temporal.PlainDate.prototype.toJSON()`
- `Temporal.PlainDate.prototype.toString()`

Temporal.PlainDate.prototype.toPlainDateTime()

The **`toPlainDateTime()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDateTime` object representing this date and a supplied time in the same calendar system.

Syntax

```
toPlainDateTime()
toPlainDateTime(plainTime)
```

Parameters

`plainTime` Optional

> A string, an object, or a `Temporal.PlainTime` instance representing the time component of the resulting `PlainDateTime`. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. Defaults to `"00:00:00"`.

Return value

A new `Temporal.PlainDateTime` object representing the date and time specified by this date and `plainTime`, interpreted in the calendar system of this date.

Examples

Using toPlainDateTime()

```
const date = Temporal.PlainDate.from("2021-07-01");
const dateTime = date.toPlainDateTime("12:34:56");
console.log(dateTime.toString()); // 2021-07-01T12:34:56

const midnight = date.toPlainDateTime();
console.log(midnight.toString()); // 2021-07-01T00:00:00

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
const dateTime2 = date2.toPlainDateTime("12:34:56");
console.log(dateTime2.toString()); // 2021-07-01T12:34:56[u-ca=chinese]
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDateTime`
- `Temporal.PlainTime`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainDateTime.prototype.toPlainDate()`

Temporal.PlainDate.prototype.toPlainMonthDay()

The **`toPlainMonthDay()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainMonthDay` object representing the `monthCode` and `day` of this date in the same calendar system.

Note that `PlainMonthDay` objects do not have a `month` component, because months with the same name can have different `month` indexes in different years due to leap months.

Syntax

```
toPlainMonthDay()
```

Parameters

None.

Return value

A new `Temporal.PlainMonthDay` object representing the `monthCode` and `day` of this date in the same calendar system.

Examples

Using toPlainMonthDay()

```
const date = Temporal.PlainDate.from("2021-07-01");
const monthDay = date.toPlainMonthDay();
console.log(monthDay.toString()); // 07-01
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainMonthDay`
- `Temporal.PlainDate.prototype.toPlainDateTime()`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainMonthDay.prototype.toPlainDate()`

Temporal.PlainDate.prototype.toPlainYearMonth()

The **`toPlainYearMonth()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainYearMonth` object representing the `year` and `month` of this date in the same calendar system.

Syntax

```
toPlainYearMonth()
```

Parameters

None.

Return value

A new `Temporal.PlainYearMonth` object representing the `year` and `month` of this date in the same calendar system.

Examples

Using toPlainYearMonth()

```
const date = Temporal.PlainDate.from("2021-07-01");
const yearMonth = date.toPlainYearMonth();
console.log(yearMonth.toString()); // 2021-07
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainYearMonth`
- `Temporal.PlainDate.prototype.toPlainDateTime()`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainYearMonth.prototype.toPlainDate()`

Temporal.PlainDate.prototype.toString()

The **`toString()`** method of `Temporal.PlainDate` instances returns a string representing this date in the RFC 9557 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainDate` instance, although the date value still remains the same.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

Return value

A string in the RFC 9557 format representing this date. The calendar annotation is included as specified.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```
const date = Temporal.PlainDate.from("2021-08-01");
console.log(date.toString()); // '2021-08-01'
```

Using options

```
const isoDate = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const date = Temporal.PlainDate.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "islamic-umalqura",
});
console.log(isoDate.toString({ calendarName: "auto" })); // '2021-08-01'
console.log(date.toString({ calendarName: "auto" })); // '2582-12-17[u-ca=islamic-umalqura]'
console.log(isoDate.toString({ calendarName: "always" })); // '2021-08-01[u-ca=iso8601]'
console.log(date.toString({ calendarName: "always" })); // '2582-12-17[u-ca=islamic-umalqura]'
console.log(date.toString({ calendarName: "never" })); // '2582-12-17'
console.log(isoDate.toString({ calendarName: "critical" })); // '2021-08-01[!u-ca=iso8601]'
console.log(date.toString({ calendarName: "critical" })); // '2582-12-17[!u-ca=islamic-umalqura]'
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.toJSON()`
- `Temporal.PlainDate.prototype.toLocaleString()`

Temporal.PlainDate.prototype.toZonedDateTime()

The **`toZonedDateTime()`** method of `Temporal.PlainDate` instances returns a new `Temporal.ZonedDateTime` object representing this date, a supplied time, and a supplied time zone, in the same calendar system.

Syntax

```
toZonedDateTime(timeZone)
toZonedDateTime(info)
```

Parameters

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the `timeZone` option. This is a convenience overload, so `toZonedDateTime(timeZone)` is equivalent to `toZonedDateTime({ timeZone })`, where `timeZone` is a string or `Temporal.ZonedDateTime`. This overload is chosen when the first argument is not an object, or the object's `timeZone` property is `undefined` (because `ZonedDateTime` instances have a `timeZoneId` property instead).

`info`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`plainTime` Optional

A string, an object, or a `Temporal.PlainTime` instance representing the time component of the resulting `ZonedDateTime`. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. Defaults to the first valid time in this time zone on this calendar date, which is usually `"00:00:00"`, but may be different if, for example, daylight saving time skips midnight.

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

A new `Temporal.ZonedDateTime` object representing the date and time specified by this date, `plainTime`, and `timeZone`, interpreted in the calendar system of this date.

In the case of ambiguities, the `compatible` behavior is always used: if the time falls into a gap, we move forward by the gap length; if the time falls into an ambiguity, we choose the earlier of the two possibilities. This means the resulting `ZonedDateTime` may have a potentially different date or time than the input.

Exceptions

`TypeError`

Thrown if `timeZone` is not a string or a `Temporal.ZonedDateTime` instance.

`RangeError`

Thrown if `timeZone` is a string that is not a valid time zone identifier.

Examples

Using toZonedDateTime()

```
const summer = Temporal.PlainDate.from("2021-07-01");
// Just time zone
const summerTime = summer.toZonedDateTime("America/New_York");
console.log(summerTime.toString()); // 2021-07-01T00:00:00-04:00[America/New_York]

const winter = Temporal.PlainDate.from("2021-01-01");
// Time zone and time
const winterTime = winter.toZonedDateTime({
  plainTime: "12:34:56",
  timeZone: "America/New_York",
});
console.log(winterTime.toString()); // 2021-01-01T12:34:56-05:00[America/New_York]

const spring = Temporal.PlainDate.from("2021-03-01");
// Time zone as object and time as object
const springTime = spring.toZonedDateTime({
  plainTime: summerTime.toPlainTime(),
  timeZone: winterTime,
});
console.log(springTime.toString()); // 2021-03-01T00:00:00-05:00[America/New_York]
```

See also

- `Temporal.PlainDate`
- `Temporal.ZonedDateTime`
- `Temporal.PlainTime`
- `Temporal.PlainDate.prototype.toPlainDateTime()`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`

Temporal.PlainDate.prototype.until()

The **`until()`** method of `Temporal.PlainDate` instances returns a new `Temporal.Duration` object representing the duration from this date to another date (in a form convertible by `Temporal.PlainDate.from()`). The duration is positive if the other date is after this date, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDate` instance representing a date to subtract this date from. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this date *until* `other`. The duration is positive if `other` is after this date, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using until()

```
const launch = Temporal.PlainDate.from("2035-01-01");
const now = Temporal.Now.plainDateISO();
const duration = now.until(launch);
console.log(`It will be ${duration.toLocaleString("en-US")} until the launch`);
```

For more examples, see `since()`.

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.since()`

Temporal.PlainDate.prototype.valueOf()

The **`valueOf()`** method of `Temporal.PlainDate` instances throws a `TypeError`, which prevents `Temporal.PlainDate` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `date1 > date2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainDate` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainDate.prototype.toString()`, or use the `Temporal.PlainDate.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainDate

All arithmetic and comparison operations on `Temporal.PlainDate` instances should use the dedicated methods or convert them to primitives explicitly.

```
const date1 = Temporal.PlainDate.from("2022-01-01");
const date2 = Temporal.PlainDate.from("2022-07-01");
date1 > date2; // TypeError: can't convert PlainDate to primitive type
Temporal.PlainDate.compare(date1, date2); // -1
```

```
date2 - date1; // TypeError: can't convert PlainDate to primitive type
date2.since(date1).toString(); // "P181D"
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.toString()`
- `Temporal.PlainDate.prototype.toJSON()`
- `Temporal.PlainDate.prototype.toLocaleString()`

Temporal.PlainDate.prototype.weekOfYear

The **`weekOfYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. It is calendar-dependent.

Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year. Namely, if a week crosses two years, then it belongs to the year that has the majority of its days. To get the year that the `weekOfYear` belongs to, use the `yearOfWeek` property, not the `year` property.

The set accessor of `weekOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDate` object with the desired new `weekOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `weeks`.

Examples

Using weekOfYear

```
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.weekOfYear); // 26

// If 01-01 is a Friday/Saturday/Sunday, it belongs to the last week of the previous year
const date2 = Temporal.PlainDate.from("2021-01-01");
console.log(date2.dayOfWeek); // 5
console.log(date2.weekOfYear); // 53; 2020 has 53 weeks
console.log(date2.yearOfWeek); // 2020

// Otherwise, it belongs to the first week of the year
const date3 = Temporal.PlainDate.from("2020-01-01");
console.log(date3.dayOfWeek); // 3
console.log(date3.weekOfYear); // 1
console.log(date3.yearOfWeek); // 2020

// Similarly, if 12-31 is a Monday/Tuesday/Wednesday, it belongs to the first week of the next year
const date4 = Temporal.PlainDate.from("2019-12-31");
console.log(date4.dayOfWeek); // 2
console.log(date4.weekOfYear); // 1
console.log(date4.yearOfWeek); // 2020
```

Changing weekOfYear

`PlainDate` does not support changing `weekOfYear` directly. To change the week, you need to first figure out the difference in weeks to your desired week, then use `add` or `subtract` to adjust the date accordingly. For example, to change to the previous week:

```
const date = Temporal.PlainDate.from("2021-07-01");
const previousWeek = date.subtract({ weeks: 1 });
console.log(previousWeek.toString()); // 2021-06-24
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDate.prototype.with()

The **`with()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date's fields.

To replace the `calendarId` property, use the `withCalendar()` method instead.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainDate.from()` (except `calendar`): `day`, `era` and `eraYear`, `month`, `monthCode`, `year`. Unspecified properties use the values from the original date. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```
const date = Temporal.PlainDate.from("2021-07-06");
const newDate = date.with({ day: date.daysInMonth });
console.log(newDate.toString()); // 2021-07-31
const nextDecade = date.with({ year: date.year + 10 });
console.log(nextDecade.toString()); // 2031-07-06
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.withCalendar()`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`

Temporal.PlainDate.prototype.withCalendar()

The **`withCalendar()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date interpreted in the new calendar system. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date's `calendarId` property.

To replace the date component properties, use the `with()` method instead.

Syntax

```
withCalendar(calendar)
```

Parameters

`calendar`

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

Return value

A new `Temporal.PlainDate` object, representing the date specified by the original `PlainDate`, interpreted in the new calendar system.

Exceptions

`TypeError`

Thrown if `calendar` is not a string.

`RangeError`

Thrown if `calendar` is not a valid calendar identifier.

Examples

Using withCalendar()

```
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.withCalendar("islamic-umalqura");
console.log(newDate.toLocaleString("en-US", { calendar: "islamic-umalqura" }));
// 11/21/1442 AH
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.calendarId`

Temporal.PlainDate.prototype.year

The **`year`** accessor property of `Temporal.PlainDate` instances returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. This property has the same function as the `era`/`eraYear` pair as a unique identifier of a year in a calendar. It is calendar-dependent.

Value

Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. Because `year` is relative to the start of the epoch year, not the epoch date, if the epoch is in the middle of the year (only known to happen for the `japanese` calendar), that year will have the same `year` value before and after the start date of the era (for the `japanese` calendar, `year` is the same as the ISO 8601 year).

All specified calendars have arithmetic years fully defined by the spec.

- The following calendars have the same epoch year as ISO 8601: `chinese`, `dangi`, `gregory`, `japanese`, in which `year: 1` corresponds to the ISO year `1`.
- The `buddhist` calendar uses the Buddhist epoch of 543 BCE, so `year: 1` corresponds to the ISO year `-542`.
- The `coptic` calendar uses the Coptic epoch of 284 CE, so `year: 1` corresponds to the ISO year `284`.
- The `ethioaa` calendar uses the Anno Mundi epoch of 5493 BCE, so `year: 1` corresponds to the ISO year `-5492`.
- The `ethiopic` calendar uses the Ethiopic epoch of 8 CE, so `year: 1` corresponds to the ISO year `8`.
- The `hebrew` calendar uses the Anno Mundi epoch of 3761 BCE, so `year: 1` corresponds to the ISO year `-3760`.
- The `indian` calendar uses the Śaka epoch of 79 CE, so `year: 1` corresponds to the ISO year `79`.
- The following calendars use the Hijri epoch of 622 CE: `islamic-civil`, `islamic-tbla`, `islamic-umalqura`, `persian`, in which `year: 1` corresponds to the ISO year `622`.
- The `roc` calendar uses the Minguo epoch of 1912 CE, so `year: 1` corresponds to the ISO year `1912`.

> **Note:** For the `chinese` and `dangi` calendars, the CLDR data uses the Huangdi epoch of 2637 BCE by default, but Temporal defined it to use the ISO 8601 epoch for simplicity.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using year

```
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.year); // 2021

const date2 = Temporal.PlainDate.from("-002021-07-01");
console.log(date2.year); // -2021

const date3 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(date3.year); // 2021; although the Japanese calendar uses eras,
// there's no obvious "default era", so the year is the same as the ISO year

const date4 = Temporal.PlainDate.from("2021-07-01[u-ca=hebrew]");
console.log(date4.year); // 5781; the Hebrew calendar uses the Anno Mundi epoch, which starts in 3761 BC
```

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.era`

- `Temporal.PlainDate.prototype.eraYear`
- `Temporal.PlainDate.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.day`

Temporal.PlainDate.prototype.yearOfWeek

The **`yearOfWeek`** accessor property of `Temporal.PlainDate` instances returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. It is calendar-dependent.

Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1. See `weekOfYear` for more details.

The set accessor of `yearOfWeek` is `undefined`. You cannot change this property directly.

Examples

See the examples in the `weekOfYear` page.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.weekOfYear`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDateTime

The **`Temporal.PlainDateTime`** object represents a date (calendar date) and time (wall-clock time) without a time zone. It is fundamentally represented as a combination of a date (with an associated calendar system) and a time.

Description

A `PlainDateTime` is essentially the combination of a `Temporal.PlainDate` and a `Temporal.PlainTime`. Because the date and time information don't have much interaction, all general information about date properties is documented in the `PlainDate` object, and all general information about time properties is documented in the `PlainTime` object.

If the date-time represents a specific instant that should remain invariant across time zones, you should use the `Temporal.ZonedDateTime` object instead. Use `PlainDateTime` when you need to represent an event happening at a specific wall-clock time that may be a different instant in different time zones.

RFC 9557 format

`PlainDateTime` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss [u-ca=calendar_id]
```

`YYYY`

: Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

: A two-digit number from `01` to `12`.

`DD`

: A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`T` Optional

: The date-time separator, which can be `T`, `t`, or a space. Present if and only if `HH` is present.

`HH` Optional

: A two-digit number from `00` to `23`. Defaults to `00`.

`mm` Optional

: A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

: A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the offset and time zone identifier, in the same format as `ZonedDateTime`, but they will be ignored. Note that the offset must *not* be `Z`. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits, whether to display the calendar ID, and whether to add a critical flag for it.

Constructor

`Temporal.PlainDateTime()`

Creates a new `Temporal.PlainDateTime` object by directly supplying the underlying data.

Static methods

`Temporal.PlainDateTime.compare()`

Returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. Equivalent to first comparing their dates, then comparing their times if the dates are the same.

`Temporal.PlainDateTime.from()`

Creates a new `Temporal.PlainDateTime` object from another `Temporal.PlainDateTime` object, an object with date and time properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainDateTime.prototype` and shared by all `Temporal.PlainDateTime` instances.

`Temporal.PlainDateTime.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainDateTime.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainDateTime` instances, the initial value is the `Temporal.PlainDateTime()` constructor.

`Temporal.PlainDateTime.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.PlainDateTime.prototype.dayOfWeek`

Returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. Calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

`Temporal.PlainDateTime.prototype.dayOfYear`

Returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. Calendar-dependent.

`Temporal.PlainDateTime.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.PlainDateTime.prototype.daysInWeek`

Returns a positive integer representing the number of days in the week of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week.

`Temporal.PlainDateTime.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.PlainDateTime.prototype.era`

Returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.PlainDateTime.prototype.eraYear`

Returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.PlainDateTime.prototype.hour`

Returns an integer from 0 to 23 representing the hour component of this time.

`Temporal.PlainDateTime.prototype.inLeapYear`

Returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.PlainDateTime.prototype.microsecond`

Returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

`Temporal.PlainDateTime.prototype.millisecond`

Returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

`Temporal.PlainDateTime.prototype.minute`

Returns an integer from 0 to 59 representing the minute component of this time.

`Temporal.PlainDateTime.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.PlainDateTime.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainDateTime.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.PlainDateTime.prototype.nanosecond`

Returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

`Temporal.PlainDateTime.prototype.second`

Returns an integer from 0 to 59 representing the second component of this time.

`Temporal.PlainDateTime.prototype.weekOfYear`

Returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. Calendar-dependent. Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year.

`Temporal.PlainDateTime.prototype.year`

Returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.PlainDateTime.prototype.yearOfWeek`

Returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. Calendar-dependent. Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1.

`Temporal.PlainDateTime.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainDateTime"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainDateTime.prototype.add()`

Returns a new `Temporal.PlainDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDateTime.prototype.equals()`

Returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`), and `false` otherwise. They are compared both by their date and time values and their calendars, so two date-times from different calendars may be considered equal by `Temporal.PlainDateTime.compare()` but not by `equals()`.

`Temporal.PlainDateTime.prototype.round()`

Returns a new `Temporal.PlainDateTime` object representing this date-time rounded to the given unit.

`Temporal.PlainDateTime.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.PlainDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

`Temporal.PlainDateTime.prototype.subtract()`

Returns a new `Temporal.PlainDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDateTime.prototype.toJSON()`

Returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainDateTime.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this date-time.

`Temporal.PlainDateTime.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing the date part (year, month, day) of this date-time in the same calendar system.

`Temporal.PlainDateTime.prototype.toPlainTime()`

Returns a new `Temporal.PlainTime` object representing the time part (hour, minute, second, and subsecond components) of this date-time.

`Temporal.PlainDateTime.prototype.toString()`

Returns a string representing this date-time in the RFC 9557 format.

`Temporal.PlainDateTime.prototype.toZonedDateTime()`

Returns a new `Temporal.ZonedDateTime` instance representing the same date-time as this plain date-time, but in the specified time zone.

`Temporal.PlainDateTime.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

`Temporal.PlainDateTime.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainDateTime.prototype.with()`

Returns a new `Temporal.PlainDateTime` object representing this date-time with some fields replaced by new values.

`Temporal.PlainDateTime.prototype.withCalendar()`

Returns a new `Temporal.PlainDateTime` object representing this date-time interpreted in the new calendar system.

`Temporal.PlainDateTime.prototype.withPlainTime()`

Returns a new `Temporal.PlainDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`).

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.PlainDate`
- `Temporal.PlainTime`
- `Temporal.ZonedDateTime`

Temporal.PlainDateTime.prototype.add()

The **`add()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object representing the date-time specified by the original `PlainDateTime`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

For how calendar durations are added, see `Temporal.PlainDate.prototype.add()`.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration

```
const start = Temporal.PlainDateTime.from("2021-01-01T12:34:56");
const end = start.add({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2022-03-26T17:41:03.008
```

For more examples, especially with how different calendars and the `overflow` option interact with calendar durations, see `Temporal.PlainDate.prototype.add()`.

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.subtract()`

Temporal.PlainDateTime.prototype.calendarId

The **`calendarId`** accessor property of `Temporal.PlainDateTime` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. Use the `withCalendar()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

Examples

Using calendarId

```
const dt = Temporal.PlainDateTime.from("2021-07-01T08:00:00");
console.log(dt.calendarId); // "iso8601"; default

const dt2 = Temporal.PlainDateTime.from("2021-07-01T08:00:00[u-ca=chinese]");
console.log(dt2.calendarId); // "chinese"

const dt3 = dt2.withCalendar("hebrew");
console.log(dt3.calendarId); // "hebrew"
```

See also

- `Temporal.PlainDateTime`

Temporal.PlainDateTime.compare()

The **`Temporal.PlainDateTime.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. Equivalent to first comparing their dates, then comparing their times if the dates are the same.

Syntax

```
Temporal.PlainDateTime.compare(dateTime1, dateTime2)
```

Parameters

`dateTime1`

A string, an object, or a `Temporal.PlainDateTime` instance representing the first date-time to compare. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`.

`dateTime2`

The second date-time to compare, converted to a `Temporal.PlainDateTime` object using the same algorithm as `dateTime1`.

Return value

Returns `-1` if `dateTime1` comes before `dateTime2`, `0` if they are the same, and `1` if `dateTime1` comes after `dateTime2`. They are compared by their underlying date and time values, ignoring their calendars.

Examples

Using Temporal.PlainDateTime.compare()

```
const dt1 = Temporal.PlainDateTime.from("2021-08-01T01:00:00");
const dt2 = Temporal.PlainDateTime.from("2021-08-02T00:00:00");
console.log(Temporal.PlainDateTime.compare(dt1, dt2)); // -1

const dt3 = Temporal.PlainDateTime.from("2021-08-01T00:00:00");
console.log(Temporal.PlainDateTime.compare(dt1, dt3)); // 1
```

Comparing dates in different calendars

```
const dt1 = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
const dt2 = Temporal.PlainDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "islamic-umalqura",
});
const dt3 = Temporal.PlainDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "hebrew",
});
console.log(dt1.toString()); // "2021-08-01T00:00:00"
console.log(dt2.toString()); // "2582-12-17T00:00:00[u-ca=islamic-umalqura]"
console.log(dt3.toString()); // "-001739-04-06T00:00:00[u-ca=hebrew]"
console.log(Temporal.PlainDateTime.compare(dt1, dt2)); // -1
console.log(Temporal.PlainDateTime.compare(dt1, dt3)); // 1
```

Sorting an array of date-times

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const dateTimes = [
  Temporal.PlainDateTime.from("2021-08-01"),
  Temporal.PlainDateTime.from("2021-08-02"),
  Temporal.PlainDateTime.from("2021-08-01T01:00:00"),
];

dateTimes.sort(Temporal.PlainDateTime.compare);
console.log(dateTimes.map((d) => d.toString()));
// [ "2021-08-01T00:00:00", "2021-08-01T01:00:00", "2021-08-02T00:00:00" ]
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.equals()`

Temporal.PlainDateTime.prototype.day

The **`day`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.day`.

Examples

Using day

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.day); // 1
```

See also

- Temporal.PlainDateTime
- Temporal.PlainDateTime.prototype.with()
- Temporal.PlainDateTime.prototype.add()
- Temporal.PlainDateTime.prototype.subtract()
- Temporal.PlainDateTime.prototype.year
- Temporal.PlainDateTime.prototype.month
- Temporal.PlainDateTime.prototype.daysInMonth
- Temporal.PlainDateTime.prototype.dayOfWeek
- Temporal.PlainDateTime.prototype.dayOfYear
- Temporal.PlainDate.prototype.day

Temporal.PlainDateTime.prototype.dayOfWeek

The **dayOfWeek** accessor property of Temporal.PlainDateTime instances returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from 1 to daysInWeek, with each number mapping to its name. It is calendar-dependent.

The set accessor of dayOfWeek is undefined. You cannot change this property directly. To create a new Temporal.PlainDateTime object with the desired new dayOfWeek value, use the add() or subtract() method with the appropriate number of days.

For general information and more examples, see Temporal.PlainDate.prototype.dayOfWeek.

Examples

Using dayOfWeek

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.dayOfWeek); // 4; Thursday
```

See also

- Temporal.PlainDateTime
- Temporal.PlainDateTime.prototype.with()
- Temporal.PlainDateTime.prototype.add()
- Temporal.PlainDateTime.prototype.subtract()
- Temporal.PlainDateTime.prototype.day
- Temporal.PlainDateTime.prototype.dayOfYear
- Temporal.PlainDateTime.prototype.daysInWeek
- Temporal.PlainDateTime.prototype.weekOfYear
- Temporal.PlainDateTime.prototype.yearOfWeek
- Temporal.PlainDate.prototype.dayOfWeek

Temporal.PlainDateTime.prototype.dayOfYear

The **dayOfYear** accessor property of Temporal.PlainDateTime instances returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is 1, and the last day is the daysInYear. It is calendar-dependent.

The set accessor of dayOfYear is undefined. You cannot change this property directly. To create a new Temporal.PlainDateTime object with the desired new dayOfYear value, use the add() or subtract() method with the appropriate number of days.

For general information and more examples, see Temporal.PlainDate.prototype.dayOfYear.

Examples

Using dayOfYear

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.dayOfYear); // 182
```

See also

- Temporal.PlainDateTime
- Temporal.PlainDateTime.prototype.with()
- Temporal.PlainDateTime.prototype.add()
- Temporal.PlainDateTime.prototype.subtract()
- Temporal.PlainDateTime.prototype.year
- Temporal.PlainDateTime.prototype.day
- Temporal.PlainDateTime.prototype.dayOfWeek
- Temporal.PlainDateTime.prototype.daysInYear
- Temporal.PlainDate.prototype.dayOfYear

Temporal.PlainDateTime.prototype.daysInMonth

The **daysInMonth** accessor property of Temporal.PlainDateTime instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

The set accessor of daysInMonth is undefined. You cannot change this property directly.

For general information and more examples, see Temporal.PlainDate.prototype.daysInMonth.

Examples

Using daysInMonth

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.daysInMonth); // 31
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.day`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInMonth`

Temporal.PlainDateTime.prototype.daysInWeek

The **`daysInWeek`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the number of days in the week of this date. It is calendar-dependent.

The set accessor of `daysInWeek` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInWeek`.

Examples

Using daysInWeek

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.daysInWeek); // 7
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.yearOfWeek`
- `Temporal.PlainDateTime.prototype.weekOfYear`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInWeek`

Temporal.PlainDateTime.prototype.daysInYear

The **`daysInYear`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

The set accessor of `daysInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInYear`.

Examples

Using daysInYear

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.daysInYear); // 365
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.dayOfYear`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDateTime.prototype.equals()

The **`equals()`** method of `Temporal.PlainDateTime` instances returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`), and `false` otherwise. They are compared both by their date and time values and their calendars, so two date-times from different

calendars may be considered equal by `Temporal.PlainDateTime.compare()` but not by `equals()`.

Syntax

```
equals(other)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainDateTime` instance representing the other date-time to compare. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`.

Return value

`true` if this date-time is equal to `other` both in their date/time value and their calendar, `false` otherwise.

Examples

Using equals()

```
const dt1 = Temporal.PlainDateTime.from("2021-08-01");
const dt2 = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
console.log(dt1.equals(dt2)); // true

const dt3 = Temporal.PlainDateTime.from("2021-08-01[u-ca=japanese]");
console.log(dt1.equals(dt3)); // false

const dt4 = Temporal.PlainDateTime.from("2021-08-01T01:00:00");
console.log(dt1.equals(dt4)); // false
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.compare()`

Temporal.PlainDateTime.prototype.era

The **`era`** accessor property of `Temporal.PlainDateTime` instances returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.era`.

Examples

Using era

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.era); // undefined

const dt2 = Temporal.PlainDateTime.from("2021-07-01[u-ca=gregory]");
console.log(dt2.era); // ce
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.eraYear`
- `Temporal.PlainDate.prototype.era`

Temporal.PlainDateTime.prototype.eraYear

The **`eraYear`** accessor property of `Temporal.PlainDateTime` instances returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.eraYear`.

Examples

Using eraYear

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.eraYear); // undefined

const dt2 = Temporal.PlainDateTime.from("2021-07-01[u-ca=gregory]");
console.log(dt2.eraYear); // 2021

const dt3 = Temporal.PlainDateTime.from("-002021-07-01[u-ca=gregory]");
console.log(dt3.eraYear); // 2022; 0000 is used for the year 1 BC

const dt4 = Temporal.PlainDateTime.from("2021-07-01[u-ca=japanese]");
console.log(dt4.eraYear); // 3
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

Temporal.PlainDateTime.from()

The **`Temporal.PlainDateTime.from()`** static method creates a new `Temporal.PlainDateTime` object from another `Temporal.PlainDateTime` object, an object with date and time properties, or an RFC 9557 string.

Syntax

```
Temporal.PlainDateTime.from(info)
Temporal.PlainDateTime.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainDateTime` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date, optionally a time, and optionally a calendar.
- An object containing properties that are recognized by either `Temporal.PlainDate.from()` (`calendar`, `era`, `eraYear`, `year`, `month`, `monthCode`, `day`) or `Temporal.PlainTime.from()` (`hour`, `minute`, `second`, `millisecond`, `microsecond`, `nanosecond`). The info should explicitly specify a year (as `year` or `era` and `eraYear`), a month (as `month` or `monthCode`), and a day; others are optional and will be set to their default values.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object, representing the date and time specified by `info` in the specified `calendar`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month` (or `monthCode`), and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainDateTime from an object

```
// Year + month + day + hour + minute + second
const dt = Temporal.PlainDateTime.from({
  year: 2021,
  month: 7,
  day: 1,
  hour: 12,
  minute: 34,
  second: 56,
});
console.log(dt.toString()); // "2021-07-01T12:34:56"
```

Creating a PlainDateTime from a string

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
console.log(dt.toLocaleString()); // "7/1/2021, 12:34:56 PM" (assuming en-US locale)
```

For more examples, especially regarding different calendars and overflow settings, see `Temporal.PlainDate.from()` and `Temporal.PlainTime.from()`.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime()`
- `Temporal.PlainDateTime.prototype.with()`

Temporal.PlainDateTime.prototype.hour

The **`hour`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 23 representing the hour component of this time.

The set accessor of `hour` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.hour`.

Examples

Using hour

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.hour); // 12
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.hour`

Temporal.PlainDateTime.prototype.inLeapYear

The **`inLeapYear`** accessor property of `Temporal.PlainDateTime` instances returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.inLeapYear`.

Examples

Using inLeapYear

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.inLeapYear); // false
console.log(dt.daysInYear); // 365
console.log(dt.monthsInYear); // 12
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.inLeapYear`

Temporal.PlainDateTime.prototype.microsecond

The **`microsecond`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

The set accessor of `microsecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.microsecond`.

Examples

Using microsecond

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.microsecond); // 456
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.second`
- `Temporal.PlainDateTime.prototype.millisecond`
- `Temporal.PlainDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.microsecond`

Temporal.PlainDateTime.prototype.millisecond

The **`millisecond`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

The set accessor of `millisecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.millisecond`.

Examples

Using millisecond

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.millisecond); // 123
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.second`
- `Temporal.PlainDateTime.prototype.microsecond`
- `Temporal.PlainDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.millisecond`

Temporal.PlainDateTime.prototype.minute

The **`minute`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 59 representing the minute component of this time.

The set accessor of `minute` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.minute`.

Examples

Using minute

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.minute); // 34
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.minute`

Temporal.PlainDateTime.prototype.month

The **`month`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.month`.

Examples

Using month

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.monthCode); // "M07"
console.log(dt.month); // 7
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.day`
- `Temporal.PlainDateTime.prototype.monthCode`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.month`

Temporal.PlainDateTime.prototype.monthCode

The **`monthCode`** accessor property of `Temporal.PlainDateTime` instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`). If the leap month is the first month of the year, the code is `M00L`.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.monthCode`.

Examples

Using monthCode

```
const date = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.monthCode`

Temporal.PlainDateTime.prototype.monthsInYear

The **`monthsInYear`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.monthsInYear`.

Examples

Using monthsInYear

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.monthsInYear); // 12
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.monthCode`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

Temporal.PlainDateTime.prototype.nanosecond

The **`nanosecond`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

The set accessor of `nanosecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.nanosecond`.

Examples

Using nanosecond

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.nanosecond); // 789
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.second`
- `Temporal.PlainDateTime.prototype.millisecond`
- `Temporal.PlainDateTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

Temporal.PlainDateTime() constructor

The **`Temporal.PlainDateTime()`** constructor creates `Temporal.PlainDateTime` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainDateTime` objects using the `Temporal.PlainDateTime.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.PlainDateTime(year, month, day)
new Temporal.PlainDateTime(year, month, day, hour)
new Temporal.PlainDateTime(year, month, day, hour, minute)
new Temporal.PlainDateTime(year, month, day, hour, minute, second)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond, microsecond)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond, microsecond, nanosecond)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond, microsecond, nanosecond, calendar)
```

> **Note:** `Temporal.PlainDateTime()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`year`

A number, truncated to an integer, representing the year in the ISO calendar system.

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`day`

A number, truncated to an integer, representing the day of the month in the ISO calendar system.

`hour` Optional

A number, truncated to an integer, representing the hour component.

`minute` Optional

A number, truncated to an integer, representing the minute component.

`second` Optional

A number, truncated to an integer, representing the second component.

`millisecond` Optional

A number, truncated to an integer, representing the millisecond component.

`microsecond` Optional

A number, truncated to an integer, representing the microsecond component.

`nanosecond` Optional

A number, truncated to an integer, representing the nanosecond component.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `year`, `month`, and `day` must be in the ISO 8601 calendar system.

Return value

A new `Temporal.PlainDateTime` object, representing the date-time specified by the parameters.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any date-time component is not a finite number.
- The date-time component combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainDateTime()

```
const dt = new Temporal.PlainDateTime(2021, 7, 1);
console.log(dt.toString()); // 2021-07-01T00:00:00

const dt2 = new Temporal.PlainDateTime(2021, 7, 1, 0, 0, 0, 0, 0, 0, "hebrew");
console.log(dt2.toString()); // 2021-07-01T00:00:00[u-ca=hebrew]
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.from()`

Temporal.PlainDateTime.prototype.round()

The **`round()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time rounded to the given unit.

Syntax

```
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. For all values of `smallestUnit` except `"day"`, the increment must be a divisor of the maximum value of the unit; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12. For `"day"`, the increment must be 1.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"day"`, `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.PlainDateTime` object representing this date-time rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
const nearestMillisecond = dt.round("millisecond");
console.log(nearestMillisecond.toString()); // 2021-07-01T12:34:56.123

const nearestHalfHour = dt.round({
  smallestUnit: "minute",
  roundingIncrement: 30,
});
console.log(nearestHalfHour.toString()); // 2021-07-01T12:30:00

const nextDay = dt.round({ smallestUnit: "day", roundingMode: "ceil" });
console.log(nextDay.toString()); // 2021-07-02T00:00:00
```

See also

- `Temporal.PlainDateTime`

Temporal.PlainDateTime.prototype.second

The **`second`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 59 representing the second component of this time.

The set accessor of `second` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.second`.

Examples

Using second

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.second); // 56
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.millisecond`
- `Temporal.PlainDateTime.prototype.microsecond`
- `Temporal.PlainDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.second`

Temporal.PlainDateTime.prototype.since()

The **`since()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.PlainDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDateTime` instance representing a date-time to subtract from this date-time. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` accept all possible units. For `largestUnit`, the default value `"auto"` means `"days"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"nanoseconds"`. The current date is used as the `relativeTo` option. Note that using units larger than `"days"` may make the duration not portable to other calendars or dates.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this date-time. The duration is positive if `other` is before this date-time, and negative if after.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using since()

```
let lastBilling = Temporal.PlainDateTime.from({
  year: Temporal.Now.plainDateISO().year,
  month: 4,
  day: 1,
});
const now = Temporal.Now.plainDateTimeISO().round("second");
if (Temporal.PlainDateTime.compare(lastBilling, now) > 0) {
  lastBilling = lastBilling.subtract({ years: 1 });
}
const duration = now.since(lastBilling);
console.log(`${duration.toLocaleString("en-US")} since last billing`);
// Expected output: "[number] days, [number] hr, [number] min, [number] sec since last billing"

const duration2 = now.since(lastBilling, { smallestUnit: "days" });
console.log(`${duration2.toLocaleString("en-US")} since last billing`);
// Expected output: "[number] days since last billing"

const duration3 = now.since(lastBilling, {
  largestUnit: "years",
  smallestUnit: "days",
});
console.log(`${duration3.toLocaleString("en-US")} since last billing`);
// Expected output: "[number] months, [number] days since last billing"
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```
const dt1 = Temporal.PlainDateTime.from("2022-01-01T00:00:00");
const dt2 = Temporal.PlainDateTime.from("2022-01-28T12:34:56");
const duration = dt2.since(dt1, {
  smallestUnit: "days",
  roundingIncrement: 5,
  roundingMode: "ceil",
});
console.log(duration.toString()); // "P30D"
```

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.until()`

Temporal.PlainDateTime.prototype.subtract()

The **`subtract()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two date-times and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object representing the date-time specified by the original `PlainDateTime`, minus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a duration

```
const start = Temporal.PlainDateTime.from("2022-01-01T12:34:56");
const end = start.subtract({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2020-10-07T07:28:48.992
```

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.since()`
- `Temporal.PlainDateTime.prototype.until()`

Temporal.PlainDateTime.prototype.toJSON()

The **`toJSON()`** method of `Temporal.PlainDateTime` instances returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given date-time in the RFC 9557 format, with the calendar annotation included if it is not `"iso8601"`.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.PlainDateTime` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.PlainDateTime` objects during JSON serialization, which can then be deserialized using the `Temporal.PlainDateTime.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const dt = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
const dtStr = dt.toJSON(); // '2021-08-01T00:00:00'
const dt2 = Temporal.PlainDateTime.from(dtStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainDateTime` can be serialized as JSON without extra effort, and how to parse it back.

```
const dt = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
const jsonStr = JSON.stringify({ nextBilling: dt }); // '{"nextBilling":"2021-08-01T00:00:00"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "nextBilling") {
    return Temporal.PlainDateTime.from(value);
  }
  return value;
});
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.toString()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`

Temporal.PlainDateTime.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.PlainDateTime` instances returns a string with a language-sensitive representation of this date-time. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If this date-time's calendar is not `"iso8601"`, the `calendar` option must be provided with the same value; otherwise, if this date-time's calendar is `"iso8601"`, the `calendar` option can be any value. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year`, `month`, `day`, `hour`, `minute`, and `second` will default to `"numeric"`.
- Provide at least one of `dateStyle` or `timeStyle`: the date-time components will be set according to the specified style and the locale.
- Provide some date-time component options. Only the specified date-time components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given date-time according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(dateTime)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");

console.log(dt.toLocaleString()); // 8/1/2021, 12:34:56 PM (assuming en-US locale)
```

If the date's calendar doesn't match the locale's default calendar, and the date's calendar is not `iso8601`, an explicit `calendar` option must be provided with the same value.

```
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56[u-ca=japanese]");
// The ja-JP locale uses the Gregorian calendar by default
dt.toLocaleString("ja-JP", { calendar: "japanese" }); // R3/8/1 12:34:56
```

Using toLocaleString() with options

You can customize which parts of the date are included in the output by providing the `options` parameter.

```
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");
dt.toLocaleString("en-US", { dateStyle: "full", timeStyle: "full" }); // Sunday, August 1, 2021 at 12:34:56 PM
dt.toLocaleString("en-US", {
  year: "numeric",
  month: "long",
  hour: "numeric",
}); // August 2021 at 12 PM
dt.toLocaleString("en-US", {
  year: "numeric",
  hour: "numeric",
  minute: "numeric",
}); // 2021, 12:34 PM
```

See also

- `Temporal.PlainDateTime`
- `Intl.DateTimeFormat`
- `Temporal.PlainDateTime.prototype.toJSON()`
- `Temporal.PlainDateTime.prototype.toString()`

Temporal.PlainDateTime.prototype.toPlainDate()

The **`toPlainDate()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDate` object representing the date part (year, month, day) of this date-time in the same calendar system.

Syntax

```
toPlainDate()
```

Parameters

None.

Return value

A new `Temporal.PlainDate` object representing the date part (year, month, day) of this date-time in the same calendar system.

Examples

Using toPlainDate()

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const date = dt.toPlainDate();
console.log(date.toString()); // '2021-07-01'
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDate`
- `Temporal.PlainDateTime.prototype.toPlainTime()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainDate.prototype.toPlainDateTime()`

Temporal.PlainDateTime.prototype.toPlainTime()

The **`toPlainTime()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainTime` object representing the time part (hour, minute, second, and subsecond components) of this date-time.

Syntax

```
toPlainTime()
```

Parameters

None.

Return value

A new `Temporal.PlainTime` object representing the time part (hour, minute, second, and subsecond components) of this date-time.

Examples

Using toPlainTime()

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const time = dt.toPlainTime();
console.log(time.toString()); // '12:34:56'
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainTime`
- `Temporal.PlainDateTime.prototype.toPlainDate()`
- `Temporal.PlainDateTime.prototype.toZonedDateTime()`

Temporal.PlainDateTime.prototype.toString()

The **`toString()`** method of `Temporal.PlainDateTime` instances returns a string representing this date-time in the RFC 9557 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainDateTime` instance, although the date value still remains the same.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

Return value

A string in the RFC 9557 format representing this date-time. The calendar annotation is included as specified.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");
console.log(dt.toString()); // '2021-08-01T12:34:56'
```

For examples with rounding times, see `Temporal.PlainTime.prototype.toString()`. For examples with displaying calendars, see `Temporal.PlainDate.prototype.toString()`.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.toJSON()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`

Temporal.PlainDateTime.prototype.toZonedDateTime()

The **`toZonedDateTime()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.ZonedDateTime` instance representing the same date-time as this plain date-time, but in the specified time zone.

Syntax

```
toZonedDateTime(timeZone)
toZonedDateTime(timeZone, options)
```

Parameters

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

`options` Optional

An object containing the following property:

`disambiguation` Optional

A string specifying what to do when this plain time corresponds to zero or more than one instants in the time zone, usually because of daylight saving time shifts. Possible values are `"compatible"`, `"earlier"`, `"later"`, and `"reject"`. Defaults to `"compatible"`. For more information about these values, see ambiguity and gaps from local time to UTC time.

Return value

A new `Temporal.ZonedDateTime` instance representing the same date-time as this plain date-time, but in the specified time zone.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the options is invalid.
- `timeZone` is not a valid time zone identifier.
- The wall-clock time is ambiguous in the time zone, and `options.disambiguation` is set to `"reject"`.

`TypeError`

Thrown if any of the arguments are not of the expected type.

Examples

Using toZonedDateTime()

```
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");
const zdt = dt.toZonedDateTime("America/New_York");
console.log(zdt.toString()); // '2021-08-01T12:34:56-04:00[America/New_York]'

const dt2 = Temporal.PlainDateTime.from("2021-01-01T12:34:56");
```

```
const zdt2 = dt2.toZonedDateTime("America/New_York");
console.log(zdt2.toString()); // '2021-01-01T12:34:56-05:00[America/New_York]'
```

Handling ambiguous times

Below, we have two wall-clock times that we want to interpret in the `America/New_York` time zone. The first one, `dtNotExist`, never existed because of a forward daylight saving time shift, so we need to choose from the times `01:05:00-05:00` or `03:05:00-04:00`. The second one, `dtAmbiguous`, appeared twice because of a backward daylight saving time shift, so we need to choose from the times `01:05:00-04:00` or `01:05:00-05:00`. For a more detailed explanation of this situation, see ambiguity and gaps from local time to UTC time.

```
const dtNotExist = Temporal.PlainDateTime.from("2024-03-10T02:05:00");
const dtAmbiguous = Temporal.PlainDateTime.from("2024-11-03T01:05:00");

// Default: compatible
console.log(dtNotExist.toZonedDateTime("America/New_York").toString());
// '2024-03-10T03:05:00-04:00[America/New_York]'
console.log(dtAmbiguous.toZonedDateTime("America/New_York").toString());
// '2024-11-03T01:05:00-04:00[America/New_York]'

// Use the earlier time for ambiguous times
console.log(
  dtNotExist
    .toZonedDateTime("America/New_York", { disambiguation: "earlier" })
    .toString(),
);
// '2024-03-10T01:05:00-05:00[America/New_York]'
console.log(
  dtAmbiguous
    .toZonedDateTime("America/New_York", { disambiguation: "earlier" })
    .toString(),
);
// '2024-11-03T01:05:00-04:00[America/New_York]'

// Use the later time for ambiguous times
console.log(
  dtNotExist
    .toZonedDateTime("America/New_York", { disambiguation: "later" })
    .toString(),
);
// '2024-03-10T03:05:00-04:00[America/New_York]'
console.log(
  dtAmbiguous
    .toZonedDateTime("America/New_York", { disambiguation: "later" })
    .toString(),
);
// '2024-11-03T01:05:00-05:00[America/New_York]'

// Throw an error for ambiguous times
dtNotExist.toZonedDateTime("America/New_York", { disambiguation: "reject" });
// RangeError: instant is ambiguous
```

See also

- `Temporal.PlainDateTime`
- `Temporal.ZonedDateTime`
- `Temporal.PlainDateTime.prototype.toPlainDate()`
- `Temporal.PlainDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`

Temporal.PlainDateTime.prototype.until()

The **`until()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainDateTime` instance representing a date-time to subtract this date-time from. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

> The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this date-time *until* `other`. The duration is positive if `other` is after this date-time, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using until()

```
let nextBilling = Temporal.PlainDateTime.from({
  year: Temporal.Now.plainDateISO().year,
  month: 4,
  day: 1,
});
const now = Temporal.Now.plainDateTimeISO().round("second");
if (Temporal.PlainDateTime.compare(nextBilling, now) < 0) {
  nextBilling = nextBilling.add({ years: 1 });
}
const duration = now.until(nextBilling);
console.log(`${duration.toLocaleString("en-US")} until next billing`);
```

For more examples, see `since()`.

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.since()`

Temporal.PlainDateTime.prototype.valueOf()

The **`valueOf()`** method of `Temporal.PlainDateTime` instances throws a `TypeError`, which prevents `Temporal.PlainDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `dateTime1 > dateTime2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainDateTime` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainDateTime.prototype.toString()`, or use the `Temporal.PlainDateTime.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainDateTime

All arithmetic and comparison operations on `Temporal.PlainDateTime` instances should use the dedicated methods or convert them to primitives explicitly.

```
const dt1 = Temporal.PlainDateTime.from("2022-01-01T00:00:00");
const dt2 = Temporal.PlainDateTime.from("2022-07-01T00:00:00");
dt1 > dt2; // TypeError: can't convert PlainDateTime to primitive type
Temporal.PlainDateTime.compare(dt1, dt2); // -1

dt2 - dt1; // TypeError: can't convert PlainDateTime to primitive type
dt2.since(dt1).toString(); // "P181D"
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.toString()`
- `Temporal.PlainDateTime.prototype.toJSON()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`

Temporal.PlainDateTime.prototype.weekOfYear

The **`weekOfYear`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. It is calendar-dependent.

The set accessor of `weekOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDateTime` object with the desired new `weekOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `weeks`.

For general information and more examples, see `Temporal.PlainDate.prototype.weekOfYear`.

Examples

Using weekOfYear

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.weekOfYear); // 26
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.yearOfWeek`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.weekOfYear`

Temporal.PlainDateTime.prototype.with()

The **`with()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's fields.

To replace the `calendarId` property, use the `withCalendar()` method instead.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainDateTime.from()` (except `calendar`): `day`, `era` and `eraYear`, `hour`, `microsecond`, `millisecond`, `minute`, `month`, `monthCode`, `nanosecond`, `second`, `year`. Unspecified properties use the values from the original date-time. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date-time.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const newDT = dt.with({ hour: 13 });
console.log(newDT.toString()); // "2021-07-01T13:34:56"
const newDT2 = dt.with({ month: 2, day: 22, millisecond: 222 });
console.log(newDT2.toString()); // "2021-02-22T13:34:56.222"
const nextDecade = dt.with({ year: dt.year + 10 });
console.log(nextDecade.toString()); // "2031-07-01T13:34:56"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.withCalendar()`
- `Temporal.PlainDateTime.prototype.withPlainTime()`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`

Temporal.PlainDateTime.prototype.withCalendar()

The **`withCalendar()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time interpreted in the new calendar system. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's `calendarId` property.

To replace the date-time component properties, use the `with()` method instead.

Syntax

```
withCalendar(calendar)
```

Parameters

`calendar`

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

Return value

A new `Temporal.PlainDateTime` object, representing the date-time specified by the original `PlainDateTime`, interpreted in the new calendar system.

Exceptions

`TypeError`

Thrown if `calendar` is not a string.

`RangeError`

Thrown if `calendar` is not a valid calendar identifier.

Examples

Using withCalendar()

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const newDT = dt.withCalendar("islamic-umalqura");
console.log(newDT.toLocaleString("en-US", { calendar: "islamic-umalqura" }));
// 11/21/1442 AH, 12:34:56 PM
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.withPlainTime()`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.calendarId`

Temporal.PlainDateTime.prototype.withPlainTime()

The **`withPlainTime()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`)

This method will replace all time properties, defaulting to `0` where properties are unspecified. If you only want to replace some of the time properties, use the `with()` method instead.

Syntax

```
withPlainTime()
withPlainTime(plainTime)
```

Parameters

`plainTime` Optional

> A string, an object, or a `Temporal.PlainTime` instance representing the new time. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. If not specified, the time part is set to `00:00:00`.

Return value

A new `Temporal.PlainDateTime` object, with the date part copied from the original date-time and the time part replaced by the new time.

Examples

Using withPlainTime()

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");

// You can pass a string
const newDT = dt.withPlainTime("13:45:00");
console.log(newDT.toString()); // "2021-07-01T13:45:00"

// You can only specify some time properties, and the rest default to 0;
// for the with() method, they would be copied from the original date-time
const newDT2 = dt.withPlainTime({ hour: 13 });
console.log(newDT2.toString()); // "2021-07-01T13:00:00"

// You can pass nothing to set the time to midnight
const newDT3 = dt.withPlainTime();
console.log(newDT3.toString()); // "2021-07-01T00:00:00"
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.withCalendar()`
- `Temporal.PlainDateTime.prototype.toPlainTime()`
- `Temporal.PlainDateTime.from()`

Temporal.PlainDateTime.prototype.year

The **`year`** accessor property of `Temporal.PlainDateTime` instances returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. It is calendar-dependent.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.year`.

Examples

Using year

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.year); // 2021
```

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.era`
- `Temporal.PlainDateTime.prototype.eraYear`
- `Temporal.PlainDateTime.prototype.yearOfWeek`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.day`
- `Temporal.PlainDate.prototype.year`

Temporal.PlainDateTime.prototype.yearOfWeek

The **`yearOfWeek`** accessor property of `Temporal.PlainDateTime` instances returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. It is calendar-dependent.

The set accessor of `yearOfWeek` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.yearOfWeek`.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.weekOfYear`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.yearOfWeek`

Temporal.PlainMonthDay

The **`Temporal.PlainMonthDay`** object represents the month and day of a calendar date, without a year or time zone; for example, an event on a calendar that recurs every year and happens during the whole day. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The year is used to disambiguate the month-day in non-ISO calendar systems.

Description

A `PlainMonthDay` is essentially the month-day part of a `Temporal.PlainDate` object, without the year. Because the meaning of a month-day can change from year to year (for example, whether it exists, or what the month-day of the next day is), this object doesn't provide much functionality on its own, such as comparison, addition, or subtraction. It doesn't even have a `month` property, because the month index is not meaningful without a year (for example, two months from two years with the same index can have different names in the case of leap months).

RFC 9557 format

`PlainMonthDay` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD [u-ca=calendar_id]
```

`YYYY` Optional

> Either a four-digit number, or a six-digit number with a `+` or `-` sign. It is required for non-ISO calendars, and optional otherwise. If omitted, you can either replace `YYYY-` with `--` (so the string looks like `--MM-DD` or `--MMDD`), or omit the `YYYY-` part entirely (so the string looks like `MM-DD` or `MMDD`). Note that the reference year actually stored may be different from the one you provide, but the represented month-day is the same. See `Temporal.PlainMonthDay.from()` for more information.

`MM`

> A two-digit number from `01` to `12`.

`DD`

> A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`[u-ca=calendar_id]` Optional

> Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the time, offset, and time zone identifier, in the same format as `PlainDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure whether to display the calendar ID, and whether to add a critical flag for it.

Constructor

`Temporal.PlainMonthDay()`

> Creates a new `Temporal.PlainMonthDay` object by directly supplying the underlying data.

Static methods

`Temporal.PlainMonthDay.from()`

> Creates a new `Temporal.PlainMonthDay` object from another `Temporal.PlainMonthDay` object, an object with month and day properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainMonthDay.prototype` and shared by all `Temporal.PlainMonthDay` instances.

`Temporal.PlainMonthDay.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainMonthDay.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainMonthDay` instances, the initial value is the `Temporal.PlainMonthDay()` constructor.

`Temporal.PlainMonthDay.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.PlainMonthDay.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainMonthDay.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainMonthDay"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainMonthDay.prototype.equals()`

Returns `true` if this month-day is equivalent in value to another month-day (in a form convertible by `Temporal.PlainMonthDay.from()`), and `false` otherwise. They are compared both by their date values and their calendars.

`Temporal.PlainMonthDay.prototype.toJSON()`

Returns a string representing this month-day in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainMonthDay.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this month-day.

`Temporal.PlainMonthDay.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing this month-day and a supplied year in the same calendar system.

`Temporal.PlainMonthDay.prototype.toString()`

Returns a string representing this month-day in the RFC 9557 format.

`Temporal.PlainMonthDay.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainMonthDay` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainMonthDay.prototype.with()`

Returns a new `Temporal.PlainMonthDay` object representing this month-day with some fields replaced by new values.

Examples

Getting the next occurrence of a festival

```
// Chinese New Years are on 1/1 in the Chinese calendar
const chineseNewYear = Temporal.PlainMonthDay.from({
  monthCode: "M01",
  day: 1,
  calendar: "chinese",
});
const currentYear = Temporal.Now.plainDateISO().withCalendar("chinese").year;
let nextCNY = chineseNewYear.toPlainDate({ year: currentYear });
if (Temporal.PlainDate.compare(nextCNY, Temporal.Now.plainDateISO()) <= 0) {
  nextCNY = nextCNY.add({ years: 1 });
}
console.log(
  `The next Chinese New Year is on ${nextCNY.withCalendar("iso8601").toLocaleString()}`,
);
```

See also

- `Temporal`
- `Temporal.PlainDate`
- `Temporal.PlainYearMonth`

Temporal.PlainMonthDay.prototype.calendarId

The **`calendarId`** accessor property of `Temporal.PlainMonthDay` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. There's no obvious way to create a new `Temporal.PlainMonthDay` object with a different calendar that represents the same month-day, so you need to convert it to a `Temporal.PlainDate` object first using `toPlainDate()`, change the calendar, and then convert it back.

Examples

Using calendarId

```
const md = Temporal.PlainMonthDay.from("07-01");
console.log(md.calendarId); // "iso8601"; default

const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=chinese]");
console.log(md2.calendarId); // "chinese"
```

Changing calendarId

```
const md = Temporal.PlainMonthDay.from("07-01");
const newMD = md
  .toPlainDate({ year: 2021 })
  .withCalendar("chinese")
  .toPlainMonthDay();
console.log(newMD.monthCode, newMD.day); // "M05" 22

const newMD2 = md
  .toPlainDate({ year: 2022 })
  .withCalendar("chinese")
  .toPlainMonthDay();
console.log(newMD2.monthCode, newMD2.day); // "M06" 3
```

See also

- `Temporal.PlainMonthDay`

Temporal.PlainMonthDay.prototype.day

The **day** accessor property of `Temporal.PlainMonthDay` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainMonthDay` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.day`.

Examples

Using day

```
const md = Temporal.PlainMonthDay.from("07-01"); // ISO 8601 calendar
console.log(md.day); // 1

const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=chinese]");
console.log(md2.day); // 22; it is May 22 in the Chinese calendar
```

Changing day

```
const md = Temporal.PlainMonthDay.from("07-01");
const newMD = md.with({ day: 15 });
console.log(newMD.toString()); // 07-15
```

By default, `with()` constrains the day to the range of valid values. So you can use `{ day: 1 }` to set the day to the first day of the month, even if the first day does not have the number `1`. Similarly, the following will set the day to the last day of the month:

```
const md = Temporal.PlainMonthDay.from("07-01");
const lastMD = md.with({ day: Number.MAX_VALUE }); // 07-31
```

For the purpose of `PlainMonthDay`, February is always considered to have 29 days.

```
const md = Temporal.PlainMonthDay.from("02-01");
const lastMD = md.with({ day: Number.MAX_VALUE }); // 02-29
console.log(lastMD.day); // 29
```

For other calendars, as long as there exists a year in which the month-day is valid, the month-day is considered valid, and the underlying reference year may therefore change. For example:

```
const md = Temporal.PlainMonthDay.from({
  monthCode: "M02",
  day: 29,
  calendar: "hebrew",
});
console.log(md.toString()); // 1972-11-06[u-ca=hebrew]
console.log(md.toLocaleString("en-US", { calendar: "hebrew" })); // 29 Heshvan
const lastMD = md.with({ day: Number.MAX_VALUE });
```

```
// 30 Heshvan does not exist in 1972, so the reference year changes to 1971
console.log(lastMD.toString()); // 1971-11-18[u-ca=hebrew]
console.log(lastMD.toLocaleString("en-US", { calendar: "hebrew" })); // 30 Heshvan
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.prototype.with()`
- `Temporal.PlainMonthDay.prototype.monthCode`

Temporal.PlainMonthDay.prototype.equals()

The **`equals()`** method of `Temporal.PlainMonthDay` instances returns `true` if this month-day is equivalent in value to another month-day (in a form convertible by `Temporal.PlainMonthDay.from()`), and `false` otherwise. They are compared both by their underlying ISO date values and their calendars.

> **Note:** `PlainMonthDay` objects keep track of a reference ISO year, which is also used in the comparison. This year is automatically set when using the `Temporal.PlainMonthDay.from()` method, but can be set manually using the `Temporal.PlainMonthDay()` constructor, causing two equivalent month-days to be considered different if they have different reference years. For this reason, you should avoid using the constructor directly and prefer the `from()` method.

Syntax

```
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainMonthDay` instance representing the other month-day to compare. It is converted to a `Temporal.PlainMonthDay` object using the same algorithm as `Temporal.PlainMonthDay.from()`.

Return value

`true` if this month-day is equal to `other` both in their date value and their calendar, `false` otherwise.

Examples

Using equals()

```
const md1 = Temporal.PlainMonthDay.from("2021-08-01");
const md2 = Temporal.PlainMonthDay.from({ year: 2020, month: 8, day: 1 }); // Year doesn't matter
console.log(md1.equals(md2)); // true

const md3 = Temporal.PlainMonthDay.from("2021-08-01[u-ca=japanese]");
console.log(md1.equals(md3)); // false

const md4 = Temporal.PlainMonthDay.from("2021-08-02");
console.log(md1.equals(md4)); // false
```

See also

- `Temporal.PlainMonthDay`

Temporal.PlainMonthDay.from()

The **`Temporal.PlainMonthDay.from()`** static method creates a new `Temporal.PlainMonthDay` object from another `Temporal.PlainMonthDay` object, an object with month and day properties, or an RFC 9557 string.

Syntax

```
Temporal.PlainMonthDay.from(info)
Temporal.PlainMonthDay.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainMonthDay` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date and optionally a calendar. If the calendar is not `iso8601`, a year is required.
- An object containing the following properties (in the order they are retrieved and validated):
  `calendar` Optional

  A string that corresponds to the `calendarId` property. Defaults to `"iso8601"`. All other properties are interpreted in this calendar system (unlike the `Temporal.PlainMonthDay()` constructor, which interprets the values in the ISO calendar system). See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

  `day`

  An integer that corresponds to the `day` property. Must be positive regardless of the `overflow` option.

`era` and `eraYear`

A string and an integer that can be used instead of `year`. See `era` and `eraYear` of `PlainDate`. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. If `month` is specified, at least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

`month`

A positive integer that can be used instead of `monthCode`. See `month` of `PlainDate`. Must be positive regardless of the `overflow` option. If `month` is provided, and the calendar is not `iso8601`, then `year` (or `eraYear` together with `era` as a substitution) must be provided too, because the same `month` may map to multiple possible `monthCode` values in different years. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

`monthCode`

Corresponds to the `monthCode` property. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

`year`

An integer used to disambiguate `month` if provided, because for some calendars, the same `month` can mean different `monthCode` in different years. See `year` of `PlainDate`. If a year is provided, then the `overflow` option validates the month-day in the given year, not just any year. If `month` is specified, at least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainMonthDay` object, representing the month and day specified by `info` in the specified `calendar`.

Each `PlainMonthDay` stores a whole ISO 8601 date internally, which has the same month-day in the target calendar as what's exposed. The reference year is visible when stringifying with `toString()`, which outputs an ISO date. The reference year is chosen arbitrarily but consistently (that is, every `(monthCode, day)` pair always maps to the same ISO reference year). It does not use the year provided in the input. Instead, the reference year is chosen by finding the latest date before December 31, 1972 that has the same month-day in the target calendar, or the earliest date after December 31, 1972 if no such date exists.

For example, for Gregorian-derived calendars, the reference year is 1972. For the Hebrew calendar, the reference year is 1972 in the Gregorian calendar, but if the month is Adar I (`M05L`), which is a leap month, the reference year is 1970 (5730 in Hebrew calendar) instead, because the next leap year is 1973 (5733 in Hebrew calendar), which is after 1972.

This reference year canonicalization ensures that `equals()` can directly compare the underlying ISO dates without extra computation.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month`, and a `day`, or a `monthCode` and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainMonthDay from an object

```
// Month code + day
const md = Temporal.PlainMonthDay.from({ monthCode: "M05", day: 2 });
console.log(md.toString()); // 05-02

// Month + day (only for ISO calendar)
```

```
const md2 = Temporal.PlainMonthDay.from({ month: 7, day: 1 });
console.log(md2.toString()); // 07-01

// Year + month + day
const md3 = Temporal.PlainMonthDay.from({ year: 2021, month: 7, day: 1 });
console.log(md3.toString()); // 07-01

// Year + month + day in a different calendar (where year is required)
const md4 = Temporal.PlainMonthDay.from({
  year: 2021,
  month: 7,
  day: 1,
  calendar: "hebrew",
});
console.log(md4.toString()); // 1972-03-16[u-ca=hebrew]

// Month code + day in a different calendar
const md5 = Temporal.PlainMonthDay.from({
  monthCode: "M05L",
  day: 1,
  calendar: "hebrew",
});
console.log(md5.toString()); // 1970-02-07[u-ca=hebrew]
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range. A month-day without an explicit reference year is valid as long as there exists one year in which it is valid, even if it doesn't appear every year. If year, month, and day are all given, then the rules for mapping to a valid month-day could be complex and specific to each calendar, but here's the usual behavior:

- If the `year`/`month` combination is invalid, the `month` is clamped to obtain a valid `monthCode` in the year.
- If the `year`/`monthCode` combination is invalid, a different year is chosen to keep the `monthCode` as-is.
- The `day` is clamped in the given year-month to obtain a valid month-day.

This is slightly different from usual date clamping, which favors the year over the month code.

```
// Month always out of range
const md1 = Temporal.PlainMonthDay.from({ month: 13, day: 1 });
console.log(md1.toString()); // 12-01

// Month out of range for the specific year: 5732 is not a Hebrew leap year,
// so month is clamped to 12 to resolve to a valid monthCode
const md2 = Temporal.PlainMonthDay.from({
  year: 5732,
  month: 13,
  day: 1,
  calendar: "hebrew",
});
console.log(md2.toLocaleString("en-US", { calendar: "hebrew" })); // 1 Elul
const underlyingDate = Temporal.PlainDate.from(md2.toString());
console.log(underlyingDate.year, underlyingDate.month); // 5732 12

// Month code exists but not for the specific year: 5731 is not a Hebrew leap year,
// so a different year is chosen to keep the monthCode as M05L
const md3 = Temporal.PlainMonthDay.from({
  year: 5731,
  monthCode: "M05L",
  day: 1,
  calendar: "hebrew",
});
console.log(md3.toLocaleString("en-US", { calendar: "hebrew" })); // 1 Adar I
const underlyingDate2 = Temporal.PlainDate.from(md3.toString());
console.log(underlyingDate2.year, underlyingDate2.monthCode); // 5730 M05L

// Day always out of range
const md4 = Temporal.PlainMonthDay.from({ month: 2, day: 30 });
console.log(md4.toString()); // 02-29

// Day out of range for the specific year-month
const md5 = Temporal.PlainMonthDay.from({ year: 2021, month: 2, day: 29 });
console.log(md5.toString()); // 02-28
```

You can change this behavior to throw an error instead:

```
Temporal.PlainMonthDay.from(
  { year: 2021, month: 13, day: 1 },
  { overflow: "reject" },
);
// RangeError: date value "month" not in 1..12: 13
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay()`

- `Temporal.PlainMonthDay.prototype.with()`

Temporal.PlainMonthDay.prototype.monthCode

The **`monthCode`** accessor property of `Temporal.PlainMonthDay` instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`). If the leap month is the first month of the year, the code is `M00L`.

Because `month` is an index within a year, but `PlainMonthDay` doesn't have a year, there's no `month` property for `PlainMonthDay`. Therefore, `monthCode` is used to represent the month in a way that is independent of the year.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainMonthDay` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.monthCode`.

Examples

Using monthCode

```
const md = Temporal.PlainMonthDay.from("07-01"); // ISO 8601 calendar
console.log(md.monthCode); // "M07"

const md2 = Temporal.PlainMonthDay.from("2021-05-01[u-ca=chinese]");
console.log(md2.monthCode); // "M03"

const md3 = Temporal.PlainMonthDay.from("2023-04-01[u-ca=chinese]");
console.log(md3.monthCode); // "M02L"
```

Changing monthCode

```
const md = Temporal.PlainMonthDay.from("07-01");
const newMD = md.with({ monthCode: "M03" });
console.log(newMD.toString()); // 03-01
```

For other calendars, as long as there exists a year in which the month-day is valid, the month-day is considered valid, and the underlying reference year may therefore change. For example:

```
const md = Temporal.PlainMonthDay.from({
  monthCode: "M02",
  day: 30,
  calendar: "hebrew",
});
console.log(md.toString()); // 1971-11-18[u-ca=hebrew]
console.log(md.toLocaleString("en-US", { calendar: "hebrew" })); // 30 Heshvan
// 30 Heshvan only exists in 1971, but this year is not a leap year
const newMD = md.with({ monthCode: "M05L" });
console.log(newMD.toString()); // 1970-03-08[u-ca=hebrew]
console.log(newMD.toLocaleString("en-US", { calendar: "hebrew" })); // 30 Adar I
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.prototype.with()`
- `Temporal.PlainMonthDay.prototype.day`

Temporal.PlainMonthDay() constructor

The **`Temporal.PlainMonthDay()`** constructor creates `Temporal.PlainMonthDay` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainMonthDay` objects using the `Temporal.PlainMonthDay.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.PlainMonthDay(month, day)
new Temporal.PlainMonthDay(month, day, calendar)
new Temporal.PlainMonthDay(month, day, calendar, referenceYear)
```

> **Note:** `Temporal.PlainMonthDay()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

> **Warning:** Avoid using the `calendar` and `referenceYear` parameters, because `equals()` will consider the reference year for equality, causing two equivalent month-days to be considered different if they have different reference years. To create a `Temporal.PlainMonthDay` object with a non-ISO calendar, use the `Temporal.PlainMonthDay.from()` static method.

Parameters

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`day`

A number, truncated to an integer, representing the day of the month in the ISO calendar system.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `referenceYear`, `month`, and `day` must be in the ISO 8601 calendar system.

`referenceYear` Optional

A number, truncated to an integer, representing the year in the ISO calendar system. Defaults to `1972`. The same ISO month-day can represent different dates in different years with non-ISO calendars. For example, the days 2021-07-01 and 1972-07-01 may fall on different month-days in a non-Gregorian calendar, and just specifying "07-01" is insufficient to unambiguously determine a month-day in the target calendar. Therefore, you virtually always want to specify a `referenceYear` when using a non-ISO calendar.

Return value

A new `Temporal.PlainMonthDay` object, representing the month-day of the date specified by `referenceYear`, `month`, `day` (in the ISO calendar), interpreted in the calendar system specified by `calendar`.

Exceptions

`TypeError`

Thrown if `calendar` is not a string or `undefined`.

`RangeError`

Thrown in one of the following cases:

- `referenceYear`, `month`, or `day` is not a finite number.
- The `referenceYear`, `month`, and `day` combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is ±(10^8 + 1) days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainMonthDay()

```
const md = new Temporal.PlainMonthDay(7, 1);
console.log(md.toString()); // 07-01

const md2 = new Temporal.PlainMonthDay(7, 1, "chinese");
console.log(md2.toString()); // 1972-07-01[u-ca=chinese]

const md3 = new Temporal.PlainMonthDay(7, 1, "chinese", 2021);
console.log(md3.toString()); // 2021-07-01[u-ca=chinese]
```

Improper usage

You should avoid using the `calendar` and `referenceYear` parameters, unless you know that the `referenceYear` is the canonical reference year that would be selected by `Temporal.PlainMonthDay.from()` for the same month-day.

```
const md = new Temporal.PlainMonthDay(7, 1, "chinese", 2021);
const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=chinese]");
console.log(md.equals(md2)); // false
console.log(md.toString()); // 2021-07-01[u-ca=chinese]
console.log(md2.toString()); // 1972-07-02[u-ca=chinese]
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`

Temporal.PlainMonthDay.prototype.toJSON()

The **`toJSON()`** method of `Temporal.PlainMonthDay` instances returns a string representing this month-day in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given month-day in the RFC 9557 format, with the year and calendar annotation included if it is not `"iso8601"`.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.PlainMonthDay` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.PlainMonthDay` objects during JSON serialization, which can then be deserialized using the `Temporal.PlainMonthDay.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const md = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
const mdStr = md.toJSON(); // '08-01'
const md2 = Temporal.PlainMonthDay.from(mdStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainMonthDay` can be serialized as JSON without extra effort, and how to parse it back.

```
const md = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
const jsonStr = JSON.stringify({ birthday: md }); // '{"birthday":"08-01"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "birthday") {
    return Temporal.PlainMonthDay.from(value);
  }
  return value;
});
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`
- `Temporal.PlainMonthDay.prototype.toString()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`

Temporal.PlainMonthDay.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.PlainMonthDay` instances returns a string with a language-sensitive representation of this month-day. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. The `calendar` option must be provided with the same value as this month-day's calendar. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `month` and `day` will default to `"numeric"`.
- Provide `dateStyle` only: it expands to `month` and `day` formats.
- Provide some date-time component options, where at least one of them is `month` or `day`. Only the specified date components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given month-day according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(monthDay)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
// Note that just specifying "08-01" defaults to the ISO 8601 calendar,
// which throws an error if the locale's default calendar is not ISO 8601.
const md = Temporal.PlainMonthDay.from("2021-08-01[u-ca=gregory]");

console.log(md.toLocaleString()); // 8/1 (assuming en-US locale and Gregorian calendar)
```

If the month-day's calendar doesn't match the locale's default calendar, even when its calendar is `iso8601`, an explicit `calendar` option must be provided with the same value.

```
const md = Temporal.PlainMonthDay.from("08-01");
md.toLocaleString("en-US", { calendar: "iso8601" }); // 08-01
```

Using toLocaleString() with options

You can customize which parts of the month-day are included in the output by providing the `options` parameter.

```
const md = Temporal.PlainMonthDay.from("2021-08-01[u-ca=gregory]");
md.toLocaleString("en-US", { dateStyle: "full" }); // August 1
md.toLocaleString("en-US", { month: "long" }); // August
md.toLocaleString("en-US", { day: "numeric" }); // 1
```

See also

- `Temporal.PlainMonthDay`
- `Intl.DateTimeFormat`
- `Temporal.PlainMonthDay.prototype.toJSON()`
- `Temporal.PlainMonthDay.prototype.toString()`

Temporal.PlainMonthDay.prototype.toPlainDate()

The **`toPlainDate()`** method of `Temporal.PlainMonthDay` instances returns a new `Temporal.PlainDate` object representing this month-day and a supplied year in the same calendar system.

Syntax

```
toPlainDate(yearInfo)
```

Parameters

`yearInfo`

An object representing the year component of the resulting `PlainDate`, containing the following properties (in the order they are retrieved and validated):

`era` and `eraYear`

A string and an integer that correspond to the `era` and `eraYear` properties. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. If they are not provided, then `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

`year`

Corresponds to the `year` property.

Return value

A new `Temporal.PlainDate` object representing the date specified by this month-day and the year in `yearInfo`, interpreted in the calendar system of this month-day.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `yearInfo` is not an object.

Examples

Using toPlainDate()

```
const md = Temporal.PlainMonthDay.from("07-01");
const date = md.toPlainDate({ year: 2021 });
console.log(date.toString()); // 2021-07-01

const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=japanese]");
const date2 = md2.toPlainDate({ era: "reiwa", eraYear: 1 });
console.log(date2.toString()); // 2019-07-01[u-ca=japanese]
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`

Temporal.PlainMonthDay.prototype.toString()

The **`toString()`** method of `Temporal.PlainMonthDay` instances returns a string representing this month-day in the RFC 9557 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`. The reference year is included if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation. The reference year is always included too.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainMonthDay` instance, although the month-day value still remains the same. The reference year is included if the calendar is not `"iso8601"`.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself. The reference year is always included too.

Return value

A string in the RFC 9557 format representing this month-day. The calendar annotation is included as specified. The reference year is included if a calendar annotation is included or if the calendar is not `"iso8601"`.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```
const md = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
console.log(md.toString()); // '08-01'

const md2 = Temporal.PlainMonthDay.from({
```

```
  monthCode: "M08",
  day: 1,
  calendar: "chinese",
});
console.log(md2.toString()); // '1972-09-08[u-ca=chinese]'
```

Using options

```
const isoMD = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
const md = Temporal.PlainMonthDay.from({
  monthCode: "M08",
  day: 1,
  calendar: "chinese",
});
console.log(isoMD.toString({ calendarName: "auto" })); // '08-01'
console.log(md.toString({ calendarName: "auto" })); // '1972-09-08[u-ca=chinese]'
console.log(isoMD.toString({ calendarName: "always" })); // '1972-08-01[u-ca=iso8601]'
console.log(md.toString({ calendarName: "always" })); // '1972-09-08[u-ca=chinese]'
console.log(isoMD.toString({ calendarName: "never" })); // '08-01'
console.log(md.toString({ calendarName: "never" })); // '1972-09-08'
console.log(isoMD.toString({ calendarName: "critical" })); // '1972-08-01[!u-ca=iso8601]'
console.log(md.toString({ calendarName: "critical" })); // '1972-09-08[!u-ca=chinese]'
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`
- `Temporal.PlainMonthDay.prototype.toJSON()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`

Temporal.PlainMonthDay.prototype.valueOf()

The **`valueOf()`** method of `Temporal.PlainMonthDay` instances throws a `TypeError`, which prevents `Temporal.PlainMonthDay` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `monthDay1 > monthDay2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainMonthDay` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainMonthDay.prototype.toString()`.

Examples

Arithmetic and comparison operations on Temporal.PlainMonthDay

All arithmetic and comparison operations on `Temporal.PlainMonthDay` instances should use the dedicated methods or convert them to primitives explicitly.

```
const md1 = Temporal.PlainMonthDay.from("01-01");
const md2 = Temporal.PlainMonthDay.from("07-01");
md1 > md2; // TypeError: can't convert PlainMonthDay to primitive type
Temporal.PlainDate.compare(
  md1.toPlainDate({ year: 2021 }),
  md2.toPlainDate({ year: 2021 }),
); // -1

md2 - md1; // TypeError: can't convert PlainMonthDay to primitive type
md2
  .toPlainDate({ year: 2021 })
  .since(md1.toPlainDate({ year: 2021 }))
  .toString(); // "P181D"
```

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.prototype.toString()`

- `Temporal.PlainMonthDay.prototype.toJSON()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`

Temporal.PlainMonthDay.prototype.with()

The **`with()`** method of `Temporal.PlainMonthDay` instances returns a new `Temporal.PlainMonthDay` object representing this month-day with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the month-day's fields.

There's no obvious way to create a new `Temporal.PlainMonthDay` object that represents the same month-day in a different calendar, so to replace its `calendarId` property, you need to convert it to a `Temporal.PlainDate` object using `toPlainDate()`, change the calendar, and then convert it back.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainMonthDay.from()` (except `calendar`): `day`, `era` and `eraYear`, `month`, `monthCode`, `year`. Unspecified properties use the values from the original month-day. You need to provide the year if and only if you provide `month` and the calendar is not `iso8601`. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainMonthDay` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- If the year is provided, the calendar is not `iso8601`, and the year is not in the representable range or years, which is from `-271821` to `275760`.

Examples

Using with()

```
const md = Temporal.PlainMonthDay.from("07-01");
const newMd = md.with({ day: 2 });
console.log(newMd.toString()); // "07-02"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`

Temporal.PlainTime

The **`Temporal.PlainTime`** object represents a time without a date or time zone; for example, a recurring event that happens at the same time every day. It is fundamentally represented as a combination of hour, minute, second, millisecond, microsecond, and nanosecond values.

Description

A `PlainTime` is essentially the time part of a `Temporal.PlainDateTime` object, with the date information removed. Because the date and time information don't have much interaction, all general information about time properties is documented here.

RFC 9557 format

`PlainTime` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form:

```
HH:mm:ss.sssssssss
```

`HH`

: A two-digit number from `00` to `23`. It may be prefixed by the time designator `T` or `t`.

`mm` Optional

: A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

: A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

As an input, you may optionally include the date, offset, time zone identifier, and calendar, in the same format as `PlainDateTime`, but they will be ignored. A date-only string will be rejected. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits.

Constructor

`Temporal.PlainTime()`

: Creates a new `Temporal.PlainTime` object by directly supplying the underlying data.

Static methods

`Temporal.PlainTime.compare()`

: Returns a number (-1, 0, or 1) indicating whether the first time comes before, is the same as, or comes after the second time. Equivalent to comparing the hour, minute, second, millisecond, microsecond, and nanosecond fields one by one.

`Temporal.PlainTime.from()`

: Creates a new `Temporal.PlainTime` object from another `Temporal.PlainTime` object, an object with time properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainTime.prototype` and shared by all `Temporal.PlainTime` instances.

`Temporal.PlainTime.prototype.constructor`

: The constructor function that created the instance object. For `Temporal.PlainTime` instances, the initial value is the `Temporal.PlainTime()` constructor.

`Temporal.PlainTime.prototype.hour`

: Returns an integer from 0 to 23 representing the hour component of this time.

`Temporal.PlainTime.prototype.microsecond`

: Returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

`Temporal.PlainTime.prototype.millisecond`

: Returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

`Temporal.PlainTime.prototype.minute`

: Returns an integer from 0 to 59 representing the minute component of this time.

`Temporal.PlainTime.prototype.nanosecond`

: Returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

`Temporal.PlainTime.prototype.second`

: Returns an integer from 0 to 59 representing the second component of this time.

`Temporal.PlainTime.prototype[Symbol.toStringTag]`

: The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainTime"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainTime.prototype.add()`

Returns a new `Temporal.PlainTime` object representing this time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

`Temporal.PlainTime.prototype.equals()`

Returns `true` if this time is equivalent in value to another time (in a form convertible by `Temporal.PlainTime.from()`), and `false` otherwise. They are compared by their time values. Equivalent to `Temporal.PlainTime.compare(this, other) === 0`.

`Temporal.PlainTime.prototype.round()`

Returns a new `Temporal.PlainTime` object representing this time rounded to the given unit.

`Temporal.PlainTime.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another time (in a form convertible by `Temporal.PlainTime.from()`) to this time. The duration is positive if the other time is before this time, and negative if after.

`Temporal.PlainTime.prototype.subtract()`

Returns a new `Temporal.PlainTime` object representing this time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

`Temporal.PlainTime.prototype.toJSON()`

Returns a string representing this time in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainTime.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this time.

`Temporal.PlainTime.prototype.toString()`

Returns a string representing this time in the RFC 9557 format.

`Temporal.PlainTime.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this time to another time (in a form convertible by `Temporal.PlainTime.from()`). The duration is positive if the other time is after this time, and negative if before.

`Temporal.PlainTime.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainTime.prototype.with()`

Returns a new `Temporal.PlainTime` object representing this time with some fields replaced by new values.

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.PlainDateTime`

Temporal.PlainTime.prototype.add()

The **`add()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

Syntax

```
add(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.PlainTime` object representing the time specified by the original `PlainTime`, plus the duration. Any units above `hours` are ignored, and if the time goes past midnight, it wraps around to the next day.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration

```
const start = Temporal.PlainTime.from("12:34:56");
const end = start.add({ hours: 1, minutes: 30 });
console.log(end.toString()); // 14:04:56
```

```
const end2 = start.add({ hours: -1, minutes: -30 });
console.log(end2.toString()); // 11:04:56

const distance = Temporal.PlainTime.from("00:00:00").until("01:23:45"); // 1h 23m 45s
const end3 = start.add(distance);
console.log(end3.toString()); // 13:58:41
```

Time wrapping

If the time goes past midnight, it wraps around to the next day:

```
const start = Temporal.PlainTime.from("12:34:56");
const end = start.add({ hours: 12 });
console.log(end.toString()); // 00:34:56
```

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.subtract()`

Temporal.PlainTime.compare()

The **`Temporal.PlainTime.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first time comes before, is the same as, or comes after the second time. It is equivalent to comparing the hour, minute, second, millisecond, microsecond, and nanosecond fields one by one.

Syntax

```
Temporal.PlainTime.compare(time1, time2)
```

Parameters

`time1`

> A string, an object, or a `Temporal.PlainTime` instance representing the first time to compare. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`.

`time2`

> The second time to compare, converted to a `Temporal.PlainTime` object using the same algorithm as `time1`.

Return value

Returns `-1` if `time1` comes before `time2`, `0` if they are the same, and `1` if `time1` comes after `time2`.

Examples

Using Temporal.PlainTime.compare()

```
const time1 = Temporal.PlainTime.from("12:34:56");
const time2 = Temporal.PlainTime.from("12:34:57");
console.log(Temporal.PlainTime.compare(time1, time2)); // -1

const time3 = Temporal.PlainTime.from("11:34:56");
console.log(Temporal.PlainTime.compare(time1, time3)); // 1
```

Sorting an array of times

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const times = ["12:34:56", "11:34:56", "12:34:57"];

times.sort(Temporal.PlainTime.compare);
console.log(times);
// [ "11:34:56", "12:34:56", "12:34:57" ]
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.equals()`

Temporal.PlainTime.prototype.equals()

The **`equals()`** method of `Temporal.PlainTime` instances returns `true` if this time is equivalent in value to another time (in a form convertible by `Temporal.PlainTime.from()`), and `false` otherwise. They are compared by their time values. It is equivalent to `Temporal.PlainTime.compare(this, other) === 0`.

Syntax

```
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainTime` instance representing the other time to compare. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`.

Return value

`true` if this time is equal to `other` both in their time value and their calendar, `false` otherwise.

Examples

Using equals()

```
const time1 = Temporal.PlainTime.from("12:34:56");
const time2 = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
console.log(time1.equals(time2)); // true

const time3 = Temporal.PlainTime.from("00:34:56");
console.log(time1.equals(time3)); // false
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.compare()`

Temporal.PlainTime.from()

The **`Temporal.PlainTime.from()`** static method creates a new `Temporal.PlainTime` object from another `Temporal.PlainTime` object, an object with time properties, or an RFC 9557 string.

Syntax

```
Temporal.PlainTime.from(info)
Temporal.PlainTime.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainTime` instance, which creates a copy of the instance.
- A `Temporal.PlainDateTime` instance, which provides the time in the same fashion as `Temporal.PlainDateTime.prototype.toPlainTime()`.
- A `Temporal.ZonedDateTime` instance, which provides the time in the same fashion as `Temporal.ZonedDateTime.prototype.toPlainTime()`.
- An RFC 9557 string containing a time.
- An object containing at least one of the following properties (in the order they are retrieved and validated):
  - `hour`
  - `microsecond`
  - `millisecond`
  - `minute`
  - `nanosecond`
  - `second`

  They are truncated to be integers. Out-of-range values are handled by the `overflow` option.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a time component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The time component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the time component is out of range.

Return value

A new `Temporal.PlainTime` object, representing the time specified by `info`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object with at least one recognized property or a string.
- `options` is not an object or `undefined`.

`RangeError`

Thrown if the provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.

Examples

Creating a PlainTime from an object

```
const t1 = Temporal.PlainTime.from({ hour: 0 });
console.log(t1.toString()); // "00:00:00"

const t2 = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
console.log(t2.toString()); // "12:34:56"

const t3 = Temporal.PlainTime.from({
  hour: 12,
  minute: 34,
  second: 56,
  millisecond: 123,
  microsecond: 456,
  nanosecond: 789,
});
console.log(t3.toString()); // "12:34:56.123456789"
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range:

```
const t1 = Temporal.PlainTime.from({ hour: 25 });
console.log(t1.toString()); // "23:00:00"

const t2 = Temporal.PlainTime.from({ hour: 25, minute: 60 });
console.log(t2.toString()); // "23:59:00"
```

You can change this behavior to throw an error instead:

```
Temporal.PlainTime.from({ hour: 25 }, { overflow: "reject" });
// RangeError: time value "hour" not in 0..23: 25
```

Creating a PlainTime from a string

```
const t1 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(t1.toLocaleString("en-US", { timeStyle: "full" }));
// 12:34:56 PM
```

Creating a PlainTime from another Temporal instance

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:00");
const t = Temporal.PlainTime.from(dt);
console.log(t.toString()); // "12:00:00"

const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T00:00+08:00[Asia/Shanghai]",
);
const t2 = Temporal.PlainTime.from(zdt);
console.log(t2.toString()); // "00:00:00"

const t3 = Temporal.PlainTime.from(t);
console.log(t3.toString()); // "12:00:00"
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime()`
- `Temporal.PlainTime.prototype.with()`

Temporal.PlainTime.prototype.hour

The **`hour`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 23 representing the hour component of this time.

The set accessor of `hour` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using hour

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.hour); // 12
```

Changing hour

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ hour: 15 });
console.log(newTime.toString()); // 15:34:56
```

You can also use `add()` or `subtract()` to move a certain number of hours from the current time.

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ hours: 3 });
console.log(newTime.toString()); // 15:34:56
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`

Temporal.PlainTime.prototype.microsecond

The **`microsecond`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

The set accessor of `microsecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using microsecond

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.microsecond); // 0

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.microsecond); // 456
```

Changing microsecond

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ microsecond: 100 });
console.log(newTime.toString()); // 12:34:56.0001
```

You can also use `add()` or `subtract()` to move a certain number of microseconds from the current time.

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ microseconds: 100 });
console.log(newTime.toString()); // 12:34:56.0001
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.second`
- `Temporal.PlainTime.prototype.millisecond`
- `Temporal.PlainTime.prototype.nanosecond`

Temporal.PlainTime.prototype.millisecond

The **`millisecond`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

The set accessor of `millisecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using millisecond

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.millisecond); // 0

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.millisecond); // 123
```

Changing millisecond

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ millisecond: 100 });
console.log(newTime.toString()); // 12:34:56.1
```

You can also use `add()` or `subtract()` to move a certain number of milliseconds from the current time.

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ milliseconds: 100 });
console.log(newTime.toString()); // 12:34:56.1
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.second`
- `Temporal.PlainTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

Temporal.PlainTime.prototype.minute

The **`minute`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 59 representing the minute component of this time.

The set accessor of `minute` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using minute

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.minute); // 34
```

Changing minute

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ minute: 58 });
console.log(newTime.toString()); // 12:58:56
```

You can also use `add()` or `subtract()` to move a certain number of minutes from the current time.

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ minutes: 24 });
console.log(newTime.toString()); // 12:58:56
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`

Temporal.PlainTime.prototype.nanosecond

The **`nanosecond`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

The set accessor of `nanosecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using nanosecond

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.nanosecond); // 0

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.nanosecond); // 789
```

Changing nanosecond

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ nanosecond: 100 });
console.log(newTime.toString()); // 12:34:56.0000001
```

You can also use `add()` or `subtract()` to move a certain number of nanoseconds from the current time.

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ nanoseconds: 100 });
console.log(newTime.toString()); // 12:34:56.0000001
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.second`
- `Temporal.PlainTime.prototype.millisecond`
- `Temporal.PlainTime.prototype.microsecond`

Temporal.PlainTime() constructor

The **`Temporal.PlainTime()`** constructor creates `Temporal.PlainTime` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainTime` objects using the `Temporal.PlainTime.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.PlainTime()
new Temporal.PlainTime(hour)
new Temporal.PlainTime(hour, minute)
new Temporal.PlainTime(hour, minute, second)
new Temporal.PlainTime(hour, minute, second, millisecond)
new Temporal.PlainTime(hour, minute, second, millisecond, microsecond)
new Temporal.PlainTime(hour, minute, second, millisecond, microsecond, nanosecond)
```

> **Note:** `Temporal.PlainTime()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`hour` Optional

A number, truncated to an integer, representing the hour component.

`minute` Optional

A number, truncated to an integer, representing the minute component.

`second` Optional

A number, truncated to an integer, representing the second component.

`millisecond` Optional

A number, truncated to an integer, representing the millisecond component.

`microsecond` Optional

A number, truncated to an integer, representing the microsecond component.

`nanosecond` Optional

A number, truncated to an integer, representing the nanosecond component.

Return value

A new `Temporal.PlainTime` object, representing the time specified by the parameters.

Exceptions

`RangeError`

Thrown if any of the components is not a finite number, or they don't represent a valid time.

Examples

Using Temporal.PlainTime()

```
const time = new Temporal.PlainTime(12, 34, 56, 123, 456, 789);
console.log(time.toString()); // 12:34:56.123456789
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`

Temporal.PlainTime.prototype.round()

The **`round()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time rounded to the given unit.

Syntax

```
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. The increment must be a divisor of the maximum value of `smallestUnit`; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.PlainTime` object representing this time rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```
const time = Temporal.PlainTime.from("12:34:56.123456789");
const nearestMillisecond = time.round("millisecond");
console.log(nearestMillisecond.toString()); // 12:34:56.123

const nearestHalfHour = time.round({
  smallestUnit: "minute",
  roundingIncrement: 30,
});
console.log(nearestHalfHour.toString()); // 12:30:00

const nextHour = time.round({ smallestUnit: "hour", roundingMode: "ceil" });
console.log(nextHour.toString()); // 13:00:00
```

See also

- `Temporal.PlainTime`

Temporal.PlainTime.prototype.second

The **`second`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 59 representing the second component of this time.

The set accessor of `second` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using second

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.second); // 56
```

Changing second

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ second: 15 });
console.log(newTime.toString()); // 12:34:15
```

You can also use `add()` or `subtract()` to move a certain number of seconds from the current time.

```
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.subtract({ seconds: 41 });
console.log(newTime.toString()); // 12:34:15
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.millisecond`
- `Temporal.PlainTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

Temporal.PlainTime.prototype.since()

The **`since()`** method of `Temporal.PlainTime` instances returns a new `Temporal.Duration` object representing the duration from another time (in a form convertible by `Temporal.PlainTime.from()`) to this time. The duration is positive if the other time is before this time, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```
since(other)
since(other, options)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainTime` instance representing a time to subtract from this time. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`.

`options` Optional

> An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"hours"`. For `smallestUnit`, the default value is `"nanoseconds"`.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this time. The duration is positive if `other` is before this time, and negative if after.

Exceptions

`RangeError`

> Thrown if any of the options is invalid.

Examples

Using since()

```
const lunchTime = Temporal.PlainTime.from("12:30:00");
const now = Temporal.Now.plainTimeISO();
const duration = now.since(lunchTime);
console.log(`You had lunch ${duration.toLocaleString("en-US")} ago`);
// Example output: "You had lunch 3 hr, 42 min, 21 sec, 343 ms, 131 µs, 718 ns ago"

const duration2 = now.since(lunchTime, { smallestUnit: "minutes" });
console.log(`You had lunch ${duration2.toLocaleString("en-US")} ago`);
// Example output: "You had lunch 3 hr, 42 min ago"

const duration3 = now.since(lunchTime, {
  largestUnit: "minutes",
  smallestUnit: "minutes",
});
console.log(`You had lunch ${duration3.toLocaleString("en-US")} ago`);
// Example output: "You had lunch 222 min ago"
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```
const time1 = Temporal.PlainTime.from("12:30:00");
const time2 = Temporal.PlainTime.from("12:30:01");
```

```
const duration = time2.since(time1, {
  smallestUnit: "seconds",
  roundingIncrement: 15,
  roundingMode: "ceil",
});
console.log(duration.toString()); // "PT15S"
```

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.until()`

Temporal.PlainTime.prototype.subtract()

The **`subtract()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

If you want to subtract two times and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
```

Parameters

`duration`

> A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.PlainTime` object representing the time specified by the original `PlainTime`, minus the duration.

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a duration

```
const start = Temporal.PlainTime.from("12:34:56");
const end = start.subtract({ hours: 1, minutes: 30 });
console.log(end.toString()); // 11:04:56
```

For more examples, see `add()`.

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.since()`
- `Temporal.PlainTime.prototype.until()`

Temporal.PlainTime.prototype.toJSON()

The **`toJSON()`** method of `Temporal.PlainTime` instances returns a string representing this time in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given time in the RFC 9557 format.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.PlainTime` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.PlainTime` objects during JSON serialization, which can then be deserialized using the `Temporal.PlainTime.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const time = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
const timeStr = time.toJSON(); // '12:34:56'
const t2 = Temporal.PlainTime.from(timeStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainTime` can be serialized as JSON without extra effort, and how to parse it back.

```
const time = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
const jsonStr = JSON.stringify({ time }); // '{"time":"12:34:56"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "time") {
    return Temporal.PlainTime.from(value);
  }
  return value;
});
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`
- `Temporal.PlainTime.prototype.toString()`
- `Temporal.PlainTime.prototype.toLocaleString()`

Temporal.PlainTime.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.PlainTime` instances returns a string with a language-sensitive representation of this time. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `hour`, `minute`, and `second` will default to `"numeric"`.
- Provide `timeStyle` only: it expands to `dayPeriod`, `hour`, `minute`, `second`, and `fractionalSecondDigits` formats.
- Provide some date-time component options, where at least one of them is a time option (`dayPeriod`, `hour`, `minute`, `second`, `fractionalSecondDigits`). Only the specified time components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given time according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(time)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const time = Temporal.PlainTime.from("12:34:56");

console.log(time.toLocaleString()); // 12:34:56 PM (assuming en-US locale)
```

Using toLocaleString() with options

You can customize which parts of the time are included in the output by providing the `options` parameter.

```
const time = Temporal.PlainTime.from("12:34:56");
time.toLocaleString("en-US", { timeStyle: "short" }); // 12:34 PM
time.toLocaleString("en-US", { hour: "2-digit" }); // 12 PM
```

See also

- `Temporal.PlainTime`
- `Intl.DateTimeFormat`
- `Temporal.PlainTime.prototype.toJSON()`
- `Temporal.PlainTime.prototype.toString()`

Temporal.PlainTime.prototype.toString()

The **`toString()`** method of `Temporal.PlainTime` instances returns a string representing this time in the RFC 9557 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

Return value

A string in the RFC 9557 format representing this time.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.toString()); // '12:34:56'
```

Using options

```
const time1 = Temporal.PlainTime.from("12:00:00");
console.log(time1.toString()); // '12:00:00'
console.log(time1.toString({ fractionalSecondDigits: 1 })); // '12:00:00.0'
console.log(time1.toString({ smallestUnit: "minute" })); // '12:00'
console.log(time1.toString({ smallestUnit: "nanosecond" })); // '12:00:00.000000000'

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.toString({ fractionalSecondDigits: 4 })); // '12:34:56.1234'
console.log(
  time2.toString({ fractionalSecondDigits: 4, roundingMode: "halfExpand" }),
); // '12:34:56.1235'
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`
- `Temporal.PlainTime.prototype.toJSON()`
- `Temporal.PlainTime.prototype.toLocaleString()`

Temporal.PlainTime.prototype.until()

The **`until()`** method of `Temporal.PlainTime` instances returns a new `Temporal.Duration` object representing the duration from this time to another time (in a form convertible by `Temporal.PlainTime.from()`). The duration is positive if the other time is after this time, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainTime` instance representing a time to subtract this time from. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. It must have the same calendar as `this`.

`options` Optional

> The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this time *until* `other`. The duration is positive if `other` is after this time, and negative if before.

Exceptions

`RangeError`

> Thrown if any of the options is invalid.

Examples

Using until()

```
const lunchTime = Temporal.PlainTime.from("12:30:00");
const now = Temporal.Now.plainTimeISO();
const duration = now.until(lunchTime);
console.log(`It will be ${duration.toLocaleString("en-US")} until lunch`);
// Example output: "It will be 3 hr, 42 min, 21 sec, 343 ms, 131 µs, 718 ns until lunch"
```

For more examples, see `since()`.

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.since()`

Temporal.PlainTime.prototype.valueOf()

The **`valueOf()`** method of `Temporal.PlainTime` instances throws a `TypeError`, which prevents `Temporal.PlainTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `time1 > time2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainTime` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainTime.prototype.toString()`, or use the `Temporal.PlainTime.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainTime

All arithmetic and comparison operations on `Temporal.PlainTime` instances should use the dedicated methods or convert them to primitives explicitly.

```
const time1 = Temporal.PlainTime.from("00:00:00");
const time2 = Temporal.PlainTime.from("12:00:00");
time1 > time2; // TypeError: can't convert PlainTime to primitive type
Temporal.PlainTime.compare(time1, time2); // -1

time2 - time1; // TypeError: can't convert PlainTime to primitive type
time2.since(time1).toString(); // "PT12H"
```

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.toString()`
- `Temporal.PlainTime.prototype.toJSON()`
- `Temporal.PlainTime.prototype.toLocaleString()`

Temporal.PlainTime.prototype.with()

The **`with()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the time's fields.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainTime.from()`: `hour`, `microsecond`, `millisecond`, `minute`, `nanosecond`, `second`. Unspecified properties use the values from the original time.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a time component is out of range. Possible values are:

`"constrain"` (default)

The time component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the time component is out of range.

Return value

A new `Temporal.PlainTime` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original time.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object with at least one recognized property or a string.

- `options` is not an object or `undefined`.

`RangeError`

Thrown if the provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.

Examples

Using with()

```
const time = Temporal.PlainTime.from("12:34:56.123456789");
const newTime = time.with({ hour: 23 });
console.log(newTime.toString()); // '23:34:56.123456789'
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`

Temporal.PlainYearMonth

The **`Temporal.PlainYearMonth`** object represents the year and month of a calendar date, without a day or time zone; for example, an event on a calendar that happens during the whole month. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The day is used to disambiguate the year-month in non-ISO calendar systems.

Description

A `PlainYearMonth` is essentially the year-month part of a `Temporal.PlainDate` object, without the day.

RFC 9557 format

`PlainYearMonth` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD [u-ca=calendar_id]
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD` Optional

A two-digit number from `01` to `31`. It is required for non-ISO calendars, and optional otherwise. If omitted, the string looks like `YYYY-MM` or `YYYYMM`. Note that the reference day actually stored may be different from the one you provide, but the represented year-month is the same. See `Temporal.PlainYearMonth.from()` for more information. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the time, offset, and time zone identifier, in the same format as `PlainDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure whether to display the calendar ID, and whether to add a critical flag for it.

Constructor

`Temporal.PlainYearMonth()`

Creates a new `Temporal.PlainYearMonth` object by directly supplying the underlying data.

Static methods

`Temporal.PlainYearMonth.compare()`

Returns a number (-1, 0, or 1) indicating whether the first year-month comes before, is the same as, or comes after the second year-month. Equivalent to comparing their underlying ISO 8601 dates. Two year-months from different calendars may be considered equal if they start on the same ISO date.

`Temporal.PlainYearMonth.from()`

Creates a new `Temporal.PlainYearMonth` object from another `Temporal.PlainYearMonth` object, an object with year and month properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainYearMonth.prototype` and shared by all `Temporal.PlainYearMonth` instances.

`Temporal.PlainYearMonth.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainYearMonth.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainYearMonth` instances, the initial value is the `Temporal.PlainYearMonth()` constructor.

`Temporal.PlainYearMonth.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.PlainYearMonth.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.PlainYearMonth.prototype.era`

Returns a calendar-specific lowercase string representing the era of this year-month, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.PlainYearMonth.prototype.eraYear`

Returns a non-negative integer representing the year of this year-month within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.PlainYearMonth.prototype.inLeapYear`

Returns a boolean indicating whether this year-month is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.PlainYearMonth.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this year-month. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.PlainYearMonth.prototype.monthCode`

Returns a calendar-specific string representing the month of this year-month. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainYearMonth.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.PlainYearMonth.prototype.year`

Returns an integer representing the number of years of this year-month relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.PlainYearMonth.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainYearMonth"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainYearMonth.prototype.add()`

Returns a new `Temporal.PlainYearMonth` object representing this year-month moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainYearMonth.prototype.equals()`

Returns `true` if this year-month is equivalent in value to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`), and `false` otherwise. They are compared both by their underlying ISO date values and their calendars, so two year-months from different calendars may be considered equal by `Temporal.PlainYearMonth.compare()` but not by `equals()`.

`Temporal.PlainYearMonth.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`) to this year-month. The duration is positive if the other month is before this month, and negative if after.

`Temporal.PlainYearMonth.prototype.subtract()`

Returns a new `Temporal.PlainYearMonth` object representing this year-month moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainYearMonth.prototype.toJSON()`

Returns a string representing this year-month in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainYearMonth.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this year-month.

`Temporal.PlainYearMonth.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing this year-month and a supplied day in the same calendar system.

`Temporal.PlainYearMonth.prototype.toString()`

Returns a string representing this year-month in the RFC 9557 format.

`Temporal.PlainYearMonth.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this year-month to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`). The duration is positive if the other month is after this month, and negative if before.

`Temporal.PlainYearMonth.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainYearMonth` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainYearMonth.prototype.with()`

Returns a new `Temporal.PlainYearMonth` object representing this year-month with some fields replaced by new values.

See also

- `Temporal`
- `Temporal.PlainDate`
- `Temporal.PlainMonthDay`

Temporal.PlainYearMonth.prototype.add()

The **`add()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainYearMonth` object representing this year-month moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this year-month. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object representing the year-month specified by the original `PlainYearMonth`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

The `duration` is handled in this way:

- Move forward by the number of years, keeping the `monthCode` the same. If the `monthCode` is invalid in the resulting year (impossible for Gregorian and ISO 8601, but possible for calendars with leap months), we adjust based on the `overflow` option: for `constrain`, we pick another month according to the cultural

conventions of that calendar's users. For example, because the leap month is usually thought of as a duplicate of another month, we may pick the month that it is a duplicate of.
- Move forward by the number of months, adjusting the year if necessary.
- For all units smaller than `months` (weeks, days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds), they are converted to the number of days. All commonly supported calendars use fixed-length weeks, so the number of weeks is just converted to the number of days. If the rule is more complex, we may take an approach similar to shifting months. Then, we move forward by that number of days, starting on the first day of the month, adjusting the month and year if necessary. Durations smaller than the current month's length therefore have no effect.

The internal reference day is then chosen to be the first valid day of the month, irrespective of the original reference day or the number of days in the duration. For the Gregorian calendar, overflow cannot happen because every year always has 12 months, and any increment less than a month is just ignored.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration in ISO 8601 calendar

```
const start = Temporal.PlainYearMonth.from("2021-01");
const end = start.add({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2022-03

const end2 = start.add({ years: -1, months: -2, weeks: -3, days: -4 });
console.log(end2.toString()); // 2019-11

const distance = Temporal.PlainYearMonth.from("2020-01").until("2021-01"); // 366 days
const end3 = start.add(distance);
console.log(end3.toString()); // 2022-01
```

Adding a duration in a non-ISO calendar

```
const start = Temporal.PlainYearMonth.from("2021-02-01[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 12/2020
console.log(start.toString()); // 2021-01-13[u-ca=chinese]
const end = start.add({ months: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 1/2021
console.log(end.toString()); // 2021-02-12[u-ca=chinese]

// Adding an extra day has no effect at all
const end2 = start.add({ months: 1, days: 1 });
console.log(end2.toLocaleString("en-US", { calendar: "chinese" })); // 1/2021
// The reference day doesn't change, because it's always the first day of the Chinese month
console.log(end2.toString()); // 2021-02-12[u-ca=chinese]

// Start in a leap month
const start2 = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
console.log(start2.toLocaleString("en-US", { calendar: "hebrew" })); // Adar I 5730
// End in another leap month
const end3 = start2.add({ years: 3 });
console.log(end3.toLocaleString("en-US", { calendar: "hebrew" })); // Adar I 5733
```

Adding a duration with overflow

If we move a few years and the corresponding month is invalid in this year, then we adjust the month based on the `overflow` option.

```
// Start in a leap month
const start = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
// Hebrew leap years occur every 2 or 3 years, and 5731 is not a leap year
const end = start.add({ years: 1 });
console.log(end.toLocaleString("en-US", { calendar: "hebrew" })); // Adar 5731
console.log(end.monthCode); // M06
console.log(end.toString()); // 1971-02-26[u-ca=hebrew]

// Any further month additions are based on the clamped year-month
const end2 = start.add({ years: 1, months: 2 });
console.log(end2.monthCode); // M08
console.log(end2.toString()); // 1971-04-26[u-ca=hebrew]

// Compare with the same addition in a different order that results in no overflow:
const end3 = start.add({ months: 2 }).add({ years: 1 });
console.log(end3.monthCode); // M07
console.log(end3.toString()); // 1971-03-27[u-ca=hebrew]
```

Note that the following is not an overflow because the year can just increment:

```
const start = Temporal.PlainYearMonth.from("2021-01");
const end = start.add({ months: 100 });
console.log(end.toString()); // 2029-05
```

You can also throw an error if the date component is out of range:

```
const start = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
const end = start.add({ years: 1 }, { overflow: "reject" }); // RangeError: invalid "monthCode" calendar field: M05L
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.subtract()`

Temporal.PlainYearMonth.prototype.calendarId

The **`calendarId`** accessor property of `Temporal.PlainYearMonth` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. There's no obvious way to create a new `Temporal.PlainYearMonth` object with a different calendar that represents the same year-month, so you need to convert it to a `Temporal.PlainDate` object first using `toPlainDate()`, change the calendar, and then convert it back.

Examples

Using calendarId

```
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.calendarId); // "iso8601"; default

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=chinese]");
console.log(ym2.calendarId); // "chinese"
```

Changing calendarId

```
const ym = Temporal.PlainYearMonth.from("2021-07");
const newYM = ym
  .toPlainDate({ day: 1 })
  .withCalendar("chinese")
  .toPlainYearMonth();
console.log(newYM.year, newYM.monthCode); // 2021 "M05"

const newYM2 = ym
  .toPlainDate({ day: 31 })
  .withCalendar("chinese")
  .toPlainYearMonth();
console.log(newYM2.year, newYM2.monthCode); // 2021 "M06"
```

See also

- `Temporal.PlainYearMonth`

Temporal.PlainYearMonth.compare()

The **`Temporal.PlainYearMonth.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first year-month comes before, is the same as, or comes after the second year-month. It is equivalent to comparing their underlying ISO 8601 dates. Two year-months from different calendars may be considered equal if they start on the same ISO date.

> **Note:** `PlainYearMonth` objects keep track of a reference ISO day, which is also used in the comparison. This day is automatically set when using the `Temporal.PlainYearMonth.from()` method, but can be set manually using the `Temporal.PlainYearMonth()` constructor, causing two equivalent year-months to be considered different if they have different reference days. For this reason, you should avoid using the constructor directly and prefer the `from()` method.

Syntax

```
Temporal.PlainYearMonth.compare(yearMonth1, yearMonth2)
```

Parameters

`yearMonth1`

> A string, an object, or a `Temporal.PlainYearMonth` instance representing the first year-month to compare. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`.

`yearMonth2`

The second year-month to compare, converted to a `Temporal.PlainYearMonth` object using the same algorithm as `yearMonth1`.

Return value

Returns `-1` if `yearMonth1` comes before `yearMonth2`, `0` if they are the same, and `1` if `yearMonth1` comes after `yearMonth2`. They are compared by their underlying date values (usually the first day of the month), ignoring their calendars.

Examples

Using Temporal.PlainYearMonth.compare()

```
const ym1 = Temporal.PlainYearMonth.from("2021-08");
const ym2 = Temporal.PlainYearMonth.from("2021-09");
console.log(Temporal.PlainYearMonth.compare(ym1, ym2)); // -1

const ym3 = Temporal.PlainYearMonth.from("2021-07");
console.log(Temporal.PlainYearMonth.compare(ym1, ym3)); // 1
```

Comparing year-months in different calendars

```
const ym1 = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ym2 = Temporal.PlainYearMonth.from({
  year: 2021,
  month: 8,
  calendar: "islamic-umalqura",
});
const ym3 = Temporal.PlainYearMonth.from({
  year: 2021,
  month: 8,
  calendar: "hebrew",
});
console.log(ym1.toString()); // "2021-08"
console.log(ym2.toString()); // "2582-12-17[u-ca=islamic-umalqura]"
console.log(ym3.toString()); // "-001739-04-06[u-ca=hebrew]"
console.log(Temporal.PlainYearMonth.compare(ym1, ym2)); // -1
console.log(Temporal.PlainYearMonth.compare(ym1, ym3)); // 1
```

Sorting an array of year-months

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const months = [
  Temporal.PlainYearMonth.from({ year: 2021, month: 8 }),
  Temporal.PlainYearMonth.from({
    year: 2021,
    month: 8,
    calendar: "islamic-umalqura",
  }),
  Temporal.PlainYearMonth.from({ year: 2021, month: 8, calendar: "hebrew" }),
];

months.sort(Temporal.PlainYearMonth.compare);
console.log(months.map((d) => d.toString()));
// [ "-001739-04-06[u-ca=hebrew]", "2021-08", "2582-12-17[u-ca=islamic-umalqura]" ]
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.equals()`

Temporal.PlainYearMonth.prototype.daysInMonth

The **`daysInMonth`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

The set accessor of `daysInMonth` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInMonth`.

Examples

Using daysInMonth

```
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.daysInMonth); // 31
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`

- `Temporal.PlainYearMonth.prototype.month`
- `Temporal.PlainYearMonth.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInMonth`

Temporal.PlainYearMonth.prototype.daysInYear

The **`daysInYear`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

The set accessor of `daysInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInYear`.

Examples

Using daysInYear

```
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.daysInYear); // 365
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainYearMonth.prototype.equals()

The **`equals()`** method of `Temporal.PlainYearMonth` instances returns `true` if this year-month is equivalent in value to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`), and `false` otherwise. They are compared both by their underlying ISO date values and their calendars, so two year-months from different calendars may be considered equal by `Temporal.PlainYearMonth.compare()` but not by `equals()`.

> **Note:** `PlainYearMonth` objects keep track of a reference ISO day, which is also used in the comparison. This day is automatically set when using the `Temporal.PlainYearMonth.from()` method, but can be set manually using the `Temporal.PlainYearMonth()` constructor, causing two equivalent year-months to be considered different if they have different reference days. For this reason, you should avoid using the constructor directly and prefer the `from()` method.

Syntax

```
equals(other)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainYearMonth` instance representing the other year-month to compare. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`.

Return value

`true` if this year-month is equal to `other` both in their date value and their calendar, `false` otherwise.

Examples

Using equals()

```
const ym1 = Temporal.PlainYearMonth.from("2021-08");
const ym2 = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
console.log(ym1.equals(ym2)); // true

const ym3 = Temporal.PlainYearMonth.from("2021-08-01[u-ca=japanese]");
console.log(ym1.equals(ym3)); // false

const ym4 = Temporal.PlainYearMonth.from("2021-09");
console.log(ym1.equals(ym4)); // false
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.compare()`

Temporal.PlainYearMonth.prototype.era

The **`era`** accessor property of `Temporal.PlainYearMonth` instances returns a calendar-specific lowercase string representing the era of this year-month, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.era`.

Examples

Using era

```
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.era); // undefined

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=gregory]");
console.log(ym2.era); // ce
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.eraYear`
- `Temporal.PlainDate.prototype.era`

Temporal.PlainYearMonth.prototype.eraYear

The **`eraYear`** accessor property of `Temporal.PlainYearMonth` instances returns a non-negative integer representing the year of this year-month within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.eraYear`.

Examples

Using eraYear

```
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.eraYear); // undefined

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=gregory]");
console.log(ym2.eraYear); // 2021

const ym3 = Temporal.PlainYearMonth.from("-002021-07-01[u-ca=gregory]");
console.log(ym3.eraYear); // 2022; 0000 is used for the year 1 BC

const ym4 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=japanese]");
console.log(ym4.eraYear); // 3
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

Temporal.PlainYearMonth.from()

The **`Temporal.PlainYearMonth.from()`** static method creates a new `Temporal.PlainYearMonth` object from another `Temporal.PlainYearMonth` object, an object with year and month properties, or an RFC 9557 string.

Syntax

```
Temporal.PlainYearMonth.from(info)
Temporal.PlainYearMonth.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainYearMonth` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date and optionally a calendar. If the calendar is not `iso8601`, a day is required.
- An object containing the following properties (in the order they are retrieved and validated):

`calendar` Optional

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. All other properties are interpreted in this calendar system (unlike the `Temporal.PlainYearMonth()` constructor, which interprets the values in the ISO calendar system).

`era` and `eraYear`

A string and an integer that correspond to the `era` and `eraYear` properties. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. If they are not provided, then `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

`month`

Corresponds to the `month` property. Must be positive regardless of the `overflow` option.

`monthCode`

Corresponds to the `monthCode` property. If it is not provided, then `month` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

`year`

Corresponds to the `year` property.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object, representing the year and month specified by `info` in the specified `calendar`.

Each `PlainYearMonth` stores a whole ISO 8601 date internally, which has the same year-month in the target calendar as what's exposed. The reference day is visible when stringifying with `toString()`, which outputs an ISO date. The reference day is chosen arbitrarily but consistently; that is, every `(year, month)` pair always maps to the same ISO reference day. It does not use the day provided in the input. Instead, the reference day is always chosen to be the first valid day of the month.

This reference day canonicalization ensures that `equals()` can directly compare the underlying ISO dates without extra computation.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`) and a `month` (or a `monthCode`).

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainYearMonth from an object

```
// Year + month code
const ym = Temporal.PlainYearMonth.from({ year: 2021, monthCode: "M05" });
console.log(ym.toString()); // 2021-05

// Year + month
const ym2 = Temporal.PlainYearMonth.from({ year: 2021, month: 7 });
console.log(ym2.toString()); // 2021-07

// Year + month in a different calendar
const ym3 = Temporal.PlainYearMonth.from({
  year: 5730,
```

```
  month: 6,
  calendar: "hebrew",
});
console.log(ym3.toString()); // 1970-02-07[u-ca=hebrew]

// Year + month code in a different calendar
const ym4 = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
console.log(ym4.toString()); // 1970-02-07[u-ca=hebrew]
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range.

```
const ym1 = Temporal.PlainYearMonth.from({ year: 2021, month: 13 });
console.log(ym1.toString()); // 2021-12

// 5732 is not a Hebrew leap year, so a different monthCode is chosen
const ym2 = Temporal.PlainYearMonth.from({
  year: 5732,
  monthCode: "M05L",
  calendar: "hebrew",
});
console.log(ym2.toLocaleString("en-US", { calendar: "hebrew" })); // Adar 5732
const underlyingDate = Temporal.PlainDate.from(ym2.toString());
console.log(underlyingDate.year, underlyingDate.monthCode); // 5732 M06
```

You can change this behavior to throw an error instead:

```
Temporal.PlainYearMonth.from({ year: 2021, month: 13 }, { overflow: "reject" });
// RangeError: date value "month" not in 1..12: 13
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth()`
- `Temporal.PlainYearMonth.prototype.with()`

Temporal.PlainYearMonth.prototype.inLeapYear

The **`inLeapYear`** accessor property of `Temporal.PlainYearMonth` instances returns a boolean indicating whether this year-month is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.inLeapYear`.

Examples

Using inLeapYear

```
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.inLeapYear); // false
console.log(ym.daysInYear); // 365
console.log(ym.monthsInYear); // 12
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.daysInYear`
- `Temporal.PlainYearMonth.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.inLeapYear`

Temporal.PlainYearMonth.prototype.month

The **`month`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the 1-based month index in the year of this year-month. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.month`.

Examples

Using month

```
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.monthCode); // "M07"
console.log(ym.month); // 7
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.monthCode`
- `Temporal.PlainYearMonth.prototype.daysInMonth`
- `Temporal.PlainYearMonth.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.month`

Temporal.PlainYearMonth.prototype.monthCode

The **`monthCode`** accessor property of `Temporal.PlainYearMonth` instances returns a calendar-specific string representing the month of this year-month. It is calendar-dependent.

Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`). If the leap month is the first month of the year, the code is `M00L`.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.monthCode`.

Examples

Using monthCode

```
const date = Temporal.PlainYearMonth.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.month`
- `Temporal.PlainYearMonth.prototype.daysInMonth`
- `Temporal.PlainYearMonth.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.monthCode`

Temporal.PlainYearMonth.prototype.monthsInYear

The **`monthsInYear`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.monthsInYear`.

Examples

Using monthsInYear

```
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.monthsInYear); // 12
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.month`
- `Temporal.PlainYearMonth.prototype.monthCode`
- `Temporal.PlainYearMonth.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

Temporal.PlainYearMonth() constructor

The **`Temporal.PlainYearMonth()`** constructor creates `Temporal.PlainYearMonth` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainYearMonth` objects using the `Temporal.PlainYearMonth.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.PlainYearMonth(year, month)
new Temporal.PlainYearMonth(year, month, calendar)
new Temporal.PlainYearMonth(year, month, calendar, referenceDay)
```

> **Note:** `Temporal.PlainYearMonth()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

> **Warning:** Avoid using the `calendar` and `referenceDay` parameters, because `equals()` and `compare()` will consider the reference day for comparison, causing two equivalent year-months to be considered different if they have different reference days. To create a `Temporal.PlainYearMonth` object with a non-ISO calendar, use the `Temporal.PlainYearMonth.from()` static method.

Parameters

`year` Optional

A number, truncated to an integer, representing the year in the ISO calendar system.

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `year`, `month`, and `referenceDay` must be in the ISO 8601 calendar system.

`referenceDay`

A number, truncated to an integer, representing the day of the month in the ISO calendar system. Defaults to `1`. The same ISO year-month can represent different months on different days with non-ISO calendars. For example, the days 2021-07-01 and 2021-07-31 may fall in different months in a non-Gregorian calendar, and just specifying "2021-07" is insufficient to unambiguously determine a month in the target calendar. Therefore, you virtually always want to specify a `referenceDay` when using a non-ISO calendar.

Return value

A new `Temporal.PlainYearMonth` object, representing the year-month of the date specified by `year`, `month`, and `referenceDay` (in the ISO calendar), interpreted in the calendar system specified by `calendar`.

Exceptions

`TypeError`

Thrown if `calendar` is not a string or `undefined`.

`RangeError`

Thrown in one of the following cases:

- `year`, `month`, or `referenceDay` is not a finite number.
- The `year`, `month`, and `referenceDay` combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainYearMonth()

```
const ym = new Temporal.PlainYearMonth(2021, 7);
console.log(ym.toString()); // 2021-07

const ym2 = new Temporal.PlainYearMonth(2021, 7, "chinese");
console.log(ym2.toString()); // 2021-07-01[u-ca=chinese]

const ym3 = new Temporal.PlainYearMonth(2021, 7, "chinese", 31);
console.log(ym3.toString()); // 2021-07-31[u-ca=chinese]
```

Improper usage

You should avoid using the `calendar` and `referenceDay` parameters, unless you know that the `referenceDay` is the canonical reference day that would be selected by `Temporal.PlainYearMonth.from()` for the same year-month.

```
const ym = new Temporal.PlainYearMonth(2021, 7, "chinese", 1);
const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=chinese]");
```

```
console.log(ym.equals(ym2)); // false
console.log(ym.toString()); // 2021-07-01[u-ca=chinese]
console.log(ym2.toString()); // 2021-06-10[u-ca=chinese]
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`

Temporal.PlainYearMonth.prototype.since()

The **`since()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.Duration` object representing the duration from another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`) to this year-month. The duration is positive if the other month is before this month, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```
since(other)
since(other, options)
```

Parameters

`other`

> A string, an object, or a `Temporal.PlainYearMonth` instance representing a year-month to subtract from this year-month. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`. It must have the same calendar as `this`.

`options` Optional

> An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"years"`, `"months"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"years"`. For `smallestUnit`, the default value is `"months"`. The current date is used as the `relativeTo` option.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this year-month. The duration is positive if `other` is before this year-month, and negative if after.

Exceptions

`RangeError`

> Thrown in one of the following cases:
>
> - `other` has a different calendar than `this`.
> - Any of the options is invalid.

Examples

Using since()

```
const lastUpdated = Temporal.PlainYearMonth.from("2022-01");
const now = Temporal.Now.plainDateISO().toPlainYearMonth();
const duration = now.since(lastUpdated);
console.log(`Last updated ${duration.toLocaleString("en-US")} ago`);
// Expected output: "Last updated [number] years, [number] months ago"

const duration2 = now.since(lastUpdated, { largestUnit: "months" });
console.log(`Last updated ${duration2.toLocaleString("en-US")} ago`);
// Expected output: "Last updated [number] months ago"

const duration3 = now.since(lastUpdated, { smallestUnit: "years" });
console.log(`Last updated ${duration3.toLocaleString("en-US")} ago`);
// Expected output: "Last updated [number] years ago"
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```
const ym1 = Temporal.PlainYearMonth.from("2022-01");
const ym2 = Temporal.PlainYearMonth.from("2022-11");
const duration = ym2.since(ym1, {
  smallestUnit: "years",
  roundingMode: "ceil",
});
console.log(duration.toString()); // "P1Y"
```

Getting the result in days

By default, the resulting duration never contains days, because `PlainYearMonth` does not offer day-level precision. You can get the result in days by converting it to a `Temporal.PlainDate` first with an unambiguous day.

```
const ym1 = Temporal.PlainYearMonth.from("2022-01");
const ym2 = Temporal.PlainYearMonth.from("2022-11");
const duration = ym2.toPlainDate({ day: 1 }).since(ym1.toPlainDate({ day: 1 }));
console.log(duration.toString()); // "P304D"
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.until()`

Temporal.PlainYearMonth.prototype.subtract()

The **`subtract()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainYearMonth` object representing this year-month moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two year-months and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this year-month. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object representing the year-month specified by the original `PlainYearMonth`, minus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply. Subtracting a positive duration starts from the end of the year-month and moves backward, so any increment smaller than the month's length is ignored.

Examples

Subtracting a duration

```
const start = Temporal.PlainYearMonth.from("2022-01");
const end = start.subtract({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2020-11
```

For more examples, see `add()`.

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.since()`
- `Temporal.PlainYearMonth.prototype.until()`

Temporal.PlainYearMonth.prototype.toJSON()

The **`toJSON()`** method of `Temporal.PlainYearMonth` instances returns a string representing this year-month in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given date in the RFC 9557 format, with the calendar annotation included if it is not `"iso8601"`.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.PlainYearMonth` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.PlainYearMonth` objects during JSON serialization, which can then be deserialized using the `Temporal.PlainYearMonth.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const ym = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ymStr = ym.toJSON(); // '2021-08'
const ym2 = Temporal.PlainYearMonth.from(ymStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainYearMonth` can be serialized as JSON without extra effort, and how to parse it back.

```
const ym = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ymStr = JSON.stringify({ event: ym }); // '{"event":"2021-08"}'
const obj = JSON.parse(ymStr, (key, value) => {
  if (key === "event") {
    return Temporal.PlainYearMonth.from(value);
  }
  return value;
});
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`
- `Temporal.PlainYearMonth.prototype.toString()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`

Temporal.PlainYearMonth.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.PlainYearMonth` instances returns a string with a language-sensitive representation of this year-month. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. The `calendar` option must be provided with the same value as this year-month's calendar. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year` and `month` will default to `"numeric"`.

- Provide `dateStyle` only: it expands to `era`, `year`, and `month` formats.
- Provide some date-time component options, where at least one of them is `year` or `month`. Only the specified date components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given year-month according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(yearMonth)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
// Note that just specifying "2021-08" defaults to the ISO 8601 calendar,
// which throws an error if the locale's default calendar is not ISO 8601.
const ym = Temporal.PlainYearMonth.from("2021-08-01[u-ca=gregory]");

console.log(ym.toLocaleString()); // 8/2021 (assuming en-US locale and Gregorian calendar)
```

If the year-month's calendar doesn't match the locale's default calendar, even when its calendar is `iso8601`, an explicit `calendar` option must be provided with the same value.

```
const ym = Temporal.PlainYearMonth.from("2021-08");
ym.toLocaleString("en-US", { calendar: "iso8601" }); // 2021-08
```

Using toLocaleString() with options

You can customize which parts of the year-month are included in the output by providing the `options` parameter.

```
const ym = Temporal.PlainYearMonth.from("2021-08-01[u-ca=gregory]");
ym.toLocaleString("en-US", { dateStyle: "full" }); // August 2021
ym.toLocaleString("en-US", { year: "2-digit" }); // 21
ym.toLocaleString("en-US", { month: "long" }); // August
```

See also

- `Temporal.PlainYearMonth`
- `Intl.DateTimeFormat`
- `Temporal.PlainYearMonth.prototype.toJSON()`
- `Temporal.PlainYearMonth.prototype.toString()`

Temporal.PlainYearMonth.prototype.toPlainDate()

The **`toPlainDate()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainDate` object representing this year-month and a supplied day in the same calendar system.

Syntax

```
toPlainDate(dayInfo)
```

Parameters

`dayInfo` Optional

An object representing the day component of the resulting `PlainDate`, containing the following property:

`day`

Corresponds to the `day` property.

Return value

A new `Temporal.PlainDate` object representing the date specified by this year-month and the day in `dayInfo`, interpreted in the calendar system of this year-month.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `dayInfo` is not an object.

Examples

Using toPlainDate()

```
const ym = Temporal.PlainYearMonth.from("2021-07");
const date = ym.toPlainDate({ day: 1 });
console.log(date.toString()); // 2021-07-01

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=chinese]");
const date2 = ym2.toPlainDate({ day: 15 });
console.log(date2.toString()); // 2021-06-24[u-ca=chinese]
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`

Temporal.PlainYearMonth.prototype.toString()

The **`toString()`** method of `Temporal.PlainYearMonth` instances returns a string representing this year-month in the RFC 9557 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`. The reference day is included if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation. The reference day is always included too.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainYearMonth` instance, although the year-month value still remains the same. The reference day is included if the calendar is not `"iso8601"`.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself. The reference day is always included too.

Return value

A string in the RFC 9557 format representing this year-month. The calendar annotation is included as specified. The reference day is included if a calendar annotation is included or if the calendar is not `"iso8601"`.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```
const ym = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
console.log(ym.toString()); // '2021-08'

const ym2 = Temporal.PlainYearMonth.from({
  year: 5781,
  monthCode: "M08",
  calendar: "hebrew",
});
console.log(ym2.toString()); // '2021-04-13[u-ca=hebrew]'
```

Using options

```
const isoYM = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ym = Temporal.PlainYearMonth.from({
  year: 5781,
  monthCode: "M08",
  calendar: "hebrew",
});
console.log(isoYM.toString({ calendarName: "auto" })); // '2021-08'
console.log(ym.toString({ calendarName: "auto" })); // '2021-04-13[u-ca=hebrew]'
console.log(isoYM.toString({ calendarName: "always" })); // '2021-08-01[u-ca=iso8601]'
console.log(ym.toString({ calendarName: "always" })); // '2021-04-13[u-ca=hebrew]'
console.log(isoYM.toString({ calendarName: "never" })); // '2021-08'
console.log(ym.toString({ calendarName: "never" })); // '2021-04-13'
console.log(isoYM.toString({ calendarName: "critical" })); // '2021-08-01[!u-ca=iso8601]'
console.log(ym.toString({ calendarName: "critical" })); // '2021-04-13[!u-ca=hebrew]'
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`
- `Temporal.PlainYearMonth.prototype.toJSON()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`

Temporal.PlainYearMonth.prototype.until()

The **`until()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.Duration` object representing the duration from this year-month to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`). The duration is positive if the other month is after this month, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainYearMonth` instance representing a year-month to subtract this year-month from. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this year-month *until* `other`. The duration is positive if `other` is after this year-month, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using until()

```
const launch = Temporal.PlainYearMonth.from("2035-01");
const now = Temporal.Now.plainDateISO().toPlainYearMonth();
const duration = now.until(launch);
console.log(`It will be ${duration.toLocaleString("en-US")} until the launch`);
```

For more examples, see `since()`.

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.since()`

Temporal.PlainYearMonth.prototype.valueOf()

The **`valueOf()`** method of `Temporal.PlainYearMonth` instances throws a `TypeError`, which prevents `Temporal.PlainYearMonth` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `yearMonth1 > yearMonth2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainYearMonth` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainYearMonth.prototype.toString()`, or use the `Temporal.PlainYearMonth.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainYearMonth

All arithmetic and comparison operations on `Temporal.PlainYearMonth` instances should use the dedicated methods or convert them to primitives explicitly.

```
const ym1 = Temporal.PlainYearMonth.from("2021-01");
const ym2 = Temporal.PlainYearMonth.from("2021-07");
ym1 > ym2; // TypeError: can't convert PlainYearMonth to primitive type
Temporal.PlainYearMonth.compare(ym1, ym2); // -1

ym2 - ym1; // TypeError: can't convert PlainYearMonth to primitive type
ym2.since(ym1).toString(); // "P6M"
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.toString()`
- `Temporal.PlainYearMonth.prototype.toJSON()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`

Temporal.PlainYearMonth.prototype.with()

The **`with()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainYearMonth` object representing this year-month with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the year-month's fields.

There's no obvious way to create a new `Temporal.PlainYearMonth` object that represents the same year-month in a different calendar, so to replace its `calendarId` property, you need to convert it to a `Temporal.PlainDate` object first using `toPlainDate()`, change the calendar, and then convert it back.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainYearMonth.from()` (except `calendar`): `era` and `eraYear`, `month`, `monthCode`, `year`. Unspecified properties use the values from the original year-month. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```
const ym = Temporal.PlainYearMonth.from("2021-07");
const newYM = ym.with({ year: 2024 });
console.log(newYM.toString()); // "2024-07"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`

Temporal.PlainYearMonth.prototype.year

The **`year`** accessor property of `Temporal.PlainYearMonth` instances returns an integer representing the number of years of this year-month relative to the start of a calendar-specific epoch year. It is calendar-dependent.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.year`.

Examples

Using year

```
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.year); // 2021
```

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.era`
- `Temporal.PlainYearMonth.prototype.eraYear`
- `Temporal.PlainYearMonth.prototype.month`
- `Temporal.PlainDate.prototype.year`

Temporal.ZonedDateTime

The **`Temporal.ZonedDateTime`** object represents a date and time with a time zone. It is fundamentally represented as a combination of an instant, a time zone, and a calendar system.

Description

A `ZonedDateTime` functions as a bridge between an exact time and a wall-clock time: it simultaneously represents an instant in history (like a `Temporal.Instant`) and a local, wall-clock time (like a `Temporal.PlainDateTime`). It does so by storing the instant, the time zone, and the calendar system. The time zone is used to convert between the instant and the local time, and the calendar system is used to interpret the local time.

`ZonedDateTime` is the only `Temporal` class that is time zone-aware. The addition of a time zone makes `ZonedDateTime` objects have important behavior differences from `Temporal.PlainDateTime` objects. Namely, you can no longer assume that "the time 1 minute afterwards" is the same every day, or that a day has 24 hours. In the worst case, an entire day may be missing from the local calendar. Below, we offer a quick overview of time zones and offsets and how they affect conversion between UTC time and local time.

Time zones and offsets

All times in JavaScript have one golden standard: the UTC time, which increments continuously and uniformly as physical time progresses. By contrast, users are more interested in their local time, which is the time they read on their calendars and clocks. The process of converting between UTC time and local time involves a time zone *offset*, which is calculated as:

```
local time = UTC time + offset
```

For example, if the UTC time is 1970-01-01T00:00:00, and the offset is "-05:00", then the local time is:

```
1970-01-01T00:00:00 + -05:00 = 1969-12-31T19:00:00
```

By suffixing this local time with the offset, thus expressing this time as "1969-12-31T19:00:00-05:00", it can now be unambiguously understood as an instant in history.

To know the offset, we need two pieces of information, the *time zone*, and the *instant*. The time zone is a region on Earth where the same offset is used at all times. Two clocks in the same time zone will always show the same time simultaneously, but the offset is not necessarily constant: that is, these clocks' times may change abruptly. This commonly happens during daylight saving time transitions, where the offset changes by an hour, which happens twice a year. Offsets can also change permanently due to political changes, e.g., a country switching time zones.

The time zones are stored in the IANA Time Zone Database. Each IANA time zone has:

- A *primary time zone identifier* that uniquely identifies the time zone. It usually refers to a geographic area anchored by a city (e.g., `Europe/Paris` or `Africa/Kampala`), but can also denote single-offset time zones like `UTC` (a consistent `+00:00` offset) or `Etc/GMT+5` (which for historical reasons is a negative offset `-05:00`). For historical reasons, the primary name for the UTC time zone is `UTC`, though in IANA it is `Etc/UTC`.
- A *time zone definition* in the form of a table that maps UTC date/time ranges (including future ranges) to specific offsets.
- Zero or more *non-primary time zone identifiers* that are aliases to the primary time zone identifier. These are usually historical names that are no longer in use, but are kept for compatibility reasons. See below for more information.

As input, named identifiers are matched case-insensitively. Internally, they are stored in their preferred case, and non-primary identifiers will *not* be converted to their primary identifier.

> **Note:** When setting the time zone name, you rarely want to set it to `"UTC"`. `ZonedDateTime` is intended to be displayed to users, but no human lives in the "UTC" time zone. If you don't know the time zone at construction time but know the wall-clock time, use a `Temporal.PlainDateTime`. If you know the exact instant, use a `Temporal.Instant`.

When a `Temporal` API accepts a *time zone identifier*, in addition to primary time zone identifiers and non-primary time zone identifiers, it also accepts an *offset time zone identifier*, which is in the same form as the offset, except subminute precision is not allowed. For example, `+05:30`, `-08`, `+0600` are all valid offsets identifiers. Internally, offset identifiers are stored in the `±HH:mm` form.

> **Note:** Avoid using offset identifiers if there is a named time zone you can use instead. Even if a region has always used a single offset, it is better to use the named identifier to guard against future political changes to the offset.
>
> If a region uses (or has used) multiple offsets, then using its named time zone is even more important. This is because `Temporal.ZonedDateTime` can use methods like `add` or `with` to create new instances at a different instant. If those derived instances correspond to an instant that uses a different offset (for example, after a Daylight Saving Time transition) then your calculations will have an incorrect local time. Using a named time zone ensures that local dates and times are always adjusted for the correct offset for that instant.

For convenience, when providing a time zone identifier to `Temporal` APIs such as `Temporal.ZonedDateTime.prototype.withTimeZone()` and the `timeZoneId` option of `Temporal.ZonedDateTime.from()`, you can provide it in a few other forms:

- As another `ZonedDateTime` instance, whose `timeZoneId` will be used.
- As an RFC 9557 string with a time zone annotation, whose time zone identifier will be used.
- As an ISO 8601 / RFC 3339 string containing an offset, whose offset will be used as an offset identifier; or, if using `Z`, then the `"UTC"` time zone is used. This usage is generally not recommended, because as discussed above, offset identifiers lack the ability to safely derive other `Temporal.ZonedDateTime` instances across an offset transition like when daylight saving time starts or ends. Instead, consider just using `Temporal.Instant` or fetching the user's actual named time zone.

The IANA time zone database changes from time to time, usually to add new time zones in response to political changes. However, on rare occasions, IANA time zone identifiers are renamed to match updated English translation of a city name or to update outdated naming conventions. For example, here are a few notable name changes:

Current IANA primary identifier	Old, now non-primary identifier
`America/Argentina/Buenos_Aires`	`America/Buenos_Aires`
`Asia/Kolkata`	`Asia/Calcutta`
`Asia/Ho_Chi_Minh`	`Asia/Saigon`
`Europe/Kyiv`	`Europe/Kiev`

Historically, these renames caused problems for programmers because the Unicode CLDR database (a library used by browsers to supply time zone identifiers and data) did not follow IANA's renaming for stability reasons. As a result, some browsers like Chrome and Safari reported CLDR's outdated identifiers, while other browsers like Firefox overrode CLDR's defaults and reported the up-to-date primary identifiers.

With the introduction of Temporal, this behavior is now more standardized:

- CLDR data now includes an `"_iana"` attribute that indicates the most up-to-date identifier if the older, stable identifier has been renamed. Browsers can use this new attribute to provide up-to-date identifiers to callers.
- Time zone identifiers provided by the programmer will never be replaced with an alias. For example, if the caller provides `Asia/Calcutta` or `Asia/Kolkata` as the identifier input to `Temporal.ZonedDateTime.from()`, then the same identifier is returned in the resulting instance's `timeZoneId`. Note that the letter case of outputs are normalized to match IANA, so that `ASIA/calCuTTa` as input generates a `timeZoneId` of `Asia/Calcutta` as output.
- When a time zone identifier is not provided by a caller but is instead sourced from the system itself (for example, when using `Temporal.Now.timeZoneId()`), modern identifiers are returned in all browsers. For city renames, there is a two-year lag before these system-provided-identifier APIs expose the new name, thereby giving other components (like a Node server) time to update their copies of the IANA database to recognize the new name.

Note that the attribution of primary identifiers preserves the country code: for example, the IANA database records `Atlantic/Reykjavik` as an alias for `Africa/Abidjan`, but because they correspond to different countries (Iceland and Côte d'Ivoire, respectively), they are treated as distinct primary identifiers.

This standardization applies outside of `Temporal` as well. For example, the `timeZone` option returned by `Intl.DateTimeFormat.prototype.resolvedOptions()` is also never replaced with an alias, though browsers have traditionally canonicalized these identifiers prior to standardization by Temporal. On the other hand, `Intl.Locale.prototype.getTimeZones()` and `Intl.supportedValuesOf()` (`timeZone` option) will return the most up-to-date identifier, while some browsers used to return the old, non-primary identifier.

RFC 9557 format

`ZonedDateTime` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss Z/±HH:mm [time_zone_id] [u-ca=calendar_id]
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD`

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`T` Optional

The date-time separator, which can be `T`, `t`, or a space. Present if and only if `HH` is present.

`HH` Optional

A two-digit number from `00` to `23`. Defaults to `00`.

`mm` Optional

A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

`Z/±HH:mm` Optional

Either the UTC designator `Z` or `z`, or an offset from UTC in the form `+` or `-` followed by the same format as the time component. Note that subminute precision (`:ss.sssssssss`) may be unsupported by other systems, and is accepted but never output. If omitted, the offset is derived from the time zone identifier. If present, then the time must be provided too. `Z` is not the same as `+00:00`: the former means that the time is given in UTC form regardless of the time zone identifier, while the latter means that the time is given in local time which happens to be UTC+0, and will be validated against the time zone identifier via the `offset` option.

`[time_zone_id]`

Replace `time_zone_id` with the time zone identifier (named or offset) as described above. May have a *critical flag* by prefixing the identifier with `!`: e.g., `[!America/New_York]`. This flag generally tells other systems that it cannot be ignored if they don't support it. Note that it is required for `Temporal.ZonedDateTime.from()`: omitting it causes a `RangeError`. If you want to parse ISO 8601 / RFC 3339 strings without time zone identifier annotations, use `Temporal.Instant.from()` instead.

`[u-ca=calendar_id]` Optional

> Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, other annotations in the `[key=value]` format are ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits, whether to display the offset/time zone ID/calendar ID, and whether to add a critical flag for the annotations.

Ambiguity and gaps from local time to UTC time

Given a time zone, conversion from UTC to local time is straightforward: you first get the offset using the time zone name and the instant, then add the offset to the instant. The reverse is not true: conversion from local time to UTC time, without an explicit offset, is ambiguous, because one local time can correspond to zero, one, or many UTC times. Consider the most common cause: daylight saving time transitions. Take New York as an example. Its standard offset is UTC-5, but during DST, all clocks are set forward by an hour, so the offset becomes UTC-4. In the US, transitions happen at 2:00 AM local time, so consider these two transition days:

UTC time	New York time
2024-03-10T06:58:00Z	2024-03-10T01:58:00-05:00
2024-03-10T06:59:00Z	2024-03-10T01:59:00-05:00
2024-03-10T07:00:00Z	2024-03-10T03:00:00-04:00
2024-11-03T05:58:00Z	2024-11-03T01:58:00-04:00
2024-11-03T05:59:00Z	2024-11-03T01:59:00-04:00
2024-11-03T06:00:00Z	2024-11-03T01:00:00-05:00

As you can see, in March, one hour disappeared from the local time, and in November, we have two hours that have the same wall-clock time. Suppose that we stored a `PlainDateTime` that says "2024-03-10T02:05:00", and we want to interpret it in the `America/New_York` time zone, there will be no time that corresponds to it, while a `PlainDateTime` that says "2024-11-03T01:05:00" can correspond to two different instants.

When constructing a `ZonedDateTime` from a local time (using `Temporal.ZonedDateTime.from()`, `Temporal.ZonedDateTime.prototype.with()`, `Temporal.PlainDateTime.prototype.toZonedDateTime()`), the behavior for ambiguity and gaps is configurable via the `disambiguation` option:

`earlier`

> If there are two possible instants, choose the earlier one. If there is a gap, go back by the gap duration.

`later`

> If there are two possible instants, choose the later one. If there is a gap, go forward by the gap duration.

`compatible` (default)

> Same behavior as `Date`: use `later` for gaps and `earlier` for ambiguities.

`reject`

> Throw a `RangeError` whenever there is an ambiguity or a gap.

There are several cases where there's no ambiguity when constructing a `ZonedDateTime`:

- If the time is specified in UTC via the `Z` offset.
- If the offset is explicitly provided and used (see below).

Offset ambiguity

We already demonstrated how ambiguity may arise from interpreting a local time in a time zone, without providing an explicit offset. However, if you provide an explicit offset, then another conflict arises: between the offset as specified, and the offset as calculated from the time zone and the local time. This is an unavoidable real-world issue: if you store a time in the future, with an anticipated offset, then before that time comes, the time zone definition may have changed due to political reasons. For example, suppose in 2018, we set a reminder at the time `2019-12-23T12:00:00-02:00[America/Sao_Paulo]` (which is a daylight saving time; Brazil is in the southern hemisphere, so it enters DST in October and exits in February). But before that time comes, in early 2019, Brazil decides to stop observing DST, so the real offset becomes `-03:00`. Should the reminder now still fire at noon (keeping the local time), or should it fire at 11:00 AM (keeping the exact time)?

For offset ambiguity to exist, a timestamp string must be parsed using different IANA Time Zone Database rules than the rules used when the timestamp was originally generated. This will never happen when timestamps are generated during the same execution of a JavaScript program, because the ECMAScript specification requires that IANA Time Zone Database rules must be consistent for the lifetime of a JavaScript program.

However, offset ambiguity may exist when a JavaScript program is parsing timestamps that were stored earlier, as in the `America/Sao_Paulo` example above, and the IANA Time Zone Database has been updated since the timestamp was originally generated. It can also happen when communicating timestamps between computers (or, rarely, between different software on the same computer!) that use different versions of the IANA Time Zone Database. The IANA Time Zone Database also has build options (for example, using or not using deprecated rules in `backzone`) that can cause offset ambiguity when communicating timestamps between computers using different software, even when the IANA Time Zone Database version is the same.

Offset ambiguity is rarely encountered and will almost always apply only to timestamps before 1970 or for timestamps that are months or years in the future. But when this problem happens, a `RangeError` will be thrown by default. When constructing a `ZonedDateTime` with `Temporal.ZonedDateTime.from()` or when updating it using the `with()` method, you can prevent this exception by using the `offset` option to decide whether the offset or the time zone identifier "wins":

`use`

Use the offset to calculate the exact time. This option "uses" the offset to decide the instant intended by the timestamp string, even if the offset at that instant has changed. The time zone identifier is still used to then infer the (possibly updated) offset and use that offset to convert the exact time to local time. In the `2019-12-23T12:00:00-02:00[America/Sao_Paulo]` example above, this option would cause the reminder to fire at 11:00 AM local time.

`ignore`

Use the time zone identifier to re-calculate the offset, ignoring the offset that's specified in the string. This option keeps the same local time as originally calculated when we stored the time, but may result in a different instant. Note that by ignoring the offset, the same local time interpretation ambiguity may arise, which is resolved using the `disambiguation` option. In the `2019-12-23T12:00:00-02:00[America/Sao_Paulo]` example above, this option would cause the reminder to fire at 12:00 PM local time.

`reject`

Throw a `RangeError` whenever there is a conflict between the offset and the time zone identifier. This is the default for `Temporal.ZonedDateTime.from()`.

`prefer`

Use the offset if it's valid, otherwise calculate the offset from the time zone identifier. This is the default for `Temporal.ZonedDateTime.prototype.with()` (see that method for more details). This is different from `ignore` because in the case of local time ambiguity, the offset is used to resolve it rather than the `disambiguation` option.

If you know in advance how you want to handle offset ambiguity, consider using the `offset` option to avoid exceptions that are thrown by default. For example, a calendar application likely wants the time zone identifier to "win" so that recurring meetings are shown in the most up-to-date local time for that time zone, so `offset: "ignore"` is appropriate. On the other hand, a task scheduler application that executes a task exactly 3 hours from now should probably pick `offset: "use"` because changes to time zone rules shouldn't change the meaning of "3 hours from now".

In some cases, you may not know which `offset` option is the correct one without getting input from the user. In these cases, you may want to catch the `RangeError` and then ask your user which local time is the correct one, and then retry parsing using a different `offset` option corresponding to the user's choice.

Note that the `Z` offset is not equivalent to `+00:00`. The `Z` offset means "the time in UTC is known, but the offset to local time is unknown", as per RFC 9557. When the time string uses the `Z` offset, the `offset` option is ignored, and the offset is derived from the time zone ID. On the other hand, the `+00:00` offset is interpreted as a local time offset that happens to match UTC and is validated against the time zone ID.

> **Note:** Although `Temporal.Instant.from()` also takes an RFC 9557 string in the same form, there is no ambiguity because it always ignores the time zone identifier and reads the offset only.

Constructor

`Temporal.ZonedDateTime()`

Creates a new `Temporal.ZonedDateTime` object by directly supplying the underlying data.

Static methods

`Temporal.ZonedDateTime.compare()`

Returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. Equivalent to comparing the `epochNanoseconds` of the two date-times.

`Temporal.ZonedDateTime.from()`

Creates a new `Temporal.ZonedDateTime` object from another `Temporal.ZonedDateTime` object, an object with date, time, and time zone properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.ZonedDateTime.prototype` and shared by all `Temporal.ZonedDateTime` instances.

`Temporal.ZonedDateTime.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.ZonedDateTime.prototype.constructor`

The constructor function that created the instance object. For `Temporal.ZonedDateTime` instances, the initial value is the `Temporal.ZonedDateTime()` constructor.

`Temporal.ZonedDateTime.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.ZonedDateTime.prototype.dayOfWeek`

Returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. Calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

`Temporal.ZonedDateTime.prototype.dayOfYear`

Returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. Calendar-dependent.

`Temporal.ZonedDateTime.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.ZonedDateTime.prototype.daysInWeek`

Returns a positive integer representing the number of days in the week of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week.

`Temporal.ZonedDateTime.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.ZonedDateTime.prototype.epochMilliseconds`

Returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. Equivalent to dividing `epochNanoseconds` by `1e6` and flooring the result.

`Temporal.ZonedDateTime.prototype.epochNanoseconds`

Returns a `BigInt` representing the number of nanoseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

`Temporal.ZonedDateTime.prototype.era`

Returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.ZonedDateTime.prototype.eraYear`

Returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.ZonedDateTime.prototype.hour`

Returns an integer from 0 to 23 representing the hour component of this time.

`Temporal.ZonedDateTime.prototype.hoursInDay`

Returns a positive integer representing the number of hours in the day of this date in the time zone. It may be more or less than 24 in the case of offset changes such as daylight saving time.

`Temporal.ZonedDateTime.prototype.inLeapYear`

Returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.ZonedDateTime.prototype.microsecond`

Returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

`Temporal.ZonedDateTime.prototype.millisecond`

Returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

`Temporal.ZonedDateTime.prototype.minute`

Returns an integer from 0 to 59 representing the minute component of this time.

`Temporal.ZonedDateTime.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.ZonedDateTime.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.ZonedDateTime.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.ZonedDateTime.prototype.nanosecond`

Returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

`Temporal.ZonedDateTime.prototype.offset`

Returns a string representing the offset used to interpret the internal instant, in the form `±HH:mm` (or `±HH:mm:ss.sssssssss` with as much subminute precision as necessary).

`Temporal.ZonedDateTime.prototype.offsetNanoseconds`

Returns an integer representing the offset used to interpret the internal instant, as a number of nanoseconds (positive or negative).

`Temporal.ZonedDateTime.prototype.second`

Returns an integer from 0 to 59 representing the second component of this time.

`Temporal.ZonedDateTime.prototype.timeZoneId`

Returns a string representing the time zone identifier used to interpret the internal instant. It uses the same string used when constructing the `Temporal.ZonedDateTime` object, which is either an IANA time zone name or a fixed offset.

`Temporal.ZonedDateTime.prototype.weekOfYear`

Returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. Calendar-dependent. Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year.

`Temporal.ZonedDateTime.prototype.year`

Returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.ZonedDateTime.prototype.yearOfWeek`

Returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. Calendar-dependent. Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1.

`Temporal.ZonedDateTime.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.ZonedDateTime"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.ZonedDateTime.prototype.add()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.ZonedDateTime.prototype.equals()`

Returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`), and `false` otherwise. They are compared both by their instant values, time zones, and their calendars, so two date-times from different calendars or time zones may be considered equal by `Temporal.ZonedDateTime.compare()` but not by `equals()`.

`Temporal.ZonedDateTime.prototype.getTimeZoneTransition()`

Returns a `Temporal.ZonedDateTime` object representing the first instant after or before this instant at which the time zone's UTC offset changes, or `null` if there is no such transition. This is useful for finding out the offset rules of a time zone, such as its daylight saving time pattern.

`Temporal.ZonedDateTime.prototype.round()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time rounded to the given unit.

`Temporal.ZonedDateTime.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

`Temporal.ZonedDateTime.prototype.startOfDay()`

Returns a `Temporal.ZonedDateTime` object representing the first instant of this date in the time zone. It usually has a time of `00:00:00`, but may be different if the midnight doesn't exist due to offset changes, in which case the first time that exists is returned.

`Temporal.ZonedDateTime.prototype.subtract()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.ZonedDateTime.prototype.toInstant()`

Returns a new `Temporal.Instant` object representing the instant of this date-time.

`Temporal.ZonedDateTime.prototype.toJSON()`

Returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.ZonedDateTime.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this date-time.

`Temporal.ZonedDateTime.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing the date portion of this date-time.

`Temporal.ZonedDateTime.prototype.toPlainDateTime()`

Returns a new `Temporal.PlainDateTime` object representing the date and time portions of this date-time. Only the time zone information is removed.

`Temporal.ZonedDateTime.prototype.toPlainTime()`

Returns a new `Temporal.PlainTime` object representing the time portion of this date-time.

`Temporal.ZonedDateTime.prototype.toString()`

Returns a string representing this date-time in the RFC 9557 format.

`Temporal.ZonedDateTime.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

`Temporal.ZonedDateTime.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.ZonedDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.ZonedDateTime.prototype.with()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time with some fields replaced by new values.

`Temporal.ZonedDateTime.prototype.withCalendar()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time interpreted in the new calendar system.

`Temporal.ZonedDateTime.prototype.withPlainTime()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`)

`Temporal.ZonedDateTime.prototype.withTimeZone()`

Returns a new `Temporal.ZonedDateTime` object representing the same instant as this date-time but in the new time zone.

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.Instant`
- `Temporal.PlainDateTime`
- `Temporal.PlainDate`
- `Temporal.PlainTime`

Temporal.ZonedDateTime.prototype.add()

The **`add()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object representing the date-time specified by the original `ZonedDateTime`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

For how calendar durations are added, see `Temporal.PlainDate.prototype.add()`.

Addition and subtraction are performed according to rules defined in RFC 5545 (iCalendar):

- Add/subtract the date portion of a duration using calendar arithmetic; in other words, add the date portion to its `PlainDateTime` using `Temporal.PlainDateTime.prototype.add()`, and then interpret the result in the same time zone. The result will automatically adjust for daylight saving time using the rules of this instance's `timeZone` field. For example, `2024-11-03T01:00:00-04:00[America/New_York]` plus one day is `2024-11-04T01:00:00-05:00[America/New_York]`, as if the day has 25 hours.
  - If the date-time is ambiguous or invalid due to a time zone offset transition, it is resolved using the `disambiguation: "compatible"` behavior: the later of the two possible instants will be used for time-skipped transitions and the earlier of the two possible instants will be used for time-repeated transitions. For example, `2024-03-09T02:05:00-05:00[America/New_York]` plus one day is supposedly `2024-03-10T02:05:00-05:00[America/New_York]`, but this time doesn't exist, so the wall-clock time one hour after, `2024-03-10T03:05:00-04:00[America/New_York]`, is returned. Similarly, both `2024-11-02T01:00:00-04:00[America/New_York]` plus one day and `2024-11-04T01:00:00-05:00[America/New_York]` minus one day produce `2024-11-03T01:00:00-04:00[America/New_York]`, the earlier of the two possible instants.
  - If the resulting date-time's components are out of bounds, they are resolved using the `overflow` option. For example, `2024-08-31` plus one month is `2024-09-31` which doesn't exist, so it is clamped to `2024-09-30` by default.
- Add/subtract the time portion of a duration using real-world time; in other words, add the time portion to its `Instant` using `Temporal.Instant.prototype.add()`, and then interpret the result in the same time zone. For example, `2024-11-03T01:00:00-04:00[America/New_York]` plus one hour is `2024-11-03T01:00:00-05:00[America/New_York]`.

These rules make arithmetic with `Temporal.ZonedDateTime` "DST-safe", which means that the results most closely match the expectations of both real-world users and implementers of other standards-compliant calendar applications. These expectations include:

- Adding or subtracting days should keep clock time consistent across DST transitions. For example, if you have an appointment on Saturday at 1:00PM and you ask to reschedule it 1 day later, you would expect the reschedule appointment to still be at 1:00PM, even if there was a DST transition overnight.
- Adding or subtracting the time portion of a duration should ignore DST transitions. For example, a friend you've asked to meet in 2 hours will be annoyed if you show up 1 hour or 3 hours later. There should be a consistent and relatively-unsurprising order of operations.
- If results are at or near a DST transition, ambiguities should be handled automatically (no crashing) and deterministically.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration

```
const start = Temporal.ZonedDateTime.from(
  "2021-11-01T12:34:56-04:00[America/New_York]",
);
const end = start.add({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2023-01-26T17:41:03.008-05:00[America/New_York]
```

For more examples, especially with how different calendars and the `overflow` option interact with calendar durations, see `Temporal.PlainDate.prototype.add()`.

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.subtract()`

Temporal.ZonedDateTime.prototype.calendarId

The **`calendarId`** accessor property of `Temporal.ZonedDateTime` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. Use the `withCalendar()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

Examples

Using calendarId

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T08:00:00-04:00[America/New_York]",
);
console.log(dt.calendarId); // "iso8601"; default

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T08:00:00+08:00[Asia/Shanghai][u-ca=chinese]",
);
console.log(dt2.calendarId); // "chinese"

const dt3 = dt2.withCalendar("hebrew");
console.log(dt3.calendarId); // "hebrew"
```

See also

- `Temporal.ZonedDateTime`

Temporal.ZonedDateTime.compare()

The **`Temporal.ZonedDateTime.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. It is equivalent to comparing the `epochNanoseconds` of the two date-times.

Syntax

```
Temporal.ZonedDateTime.compare(dateTime1, dateTime2)
```

Parameters

`dateTime1`

> A string, an object, or a `Temporal.ZonedDateTime` instance representing the first date-time to compare. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`.

`dateTime2`

> The second date-time to compare, converted to a `Temporal.ZonedDateTime` object using the same algorithm as `dateTime1`.

Return value

Returns `-1` if `dateTime1` comes before `dateTime2`, `0` if they are the same, and `1` if `dateTime1` comes after `dateTime2`. They are compared by their underlying instant values, ignoring their calendars or time zones.

Examples

Using Temporal.ZonedDateTime.compare()

```
const dt1 = Temporal.ZonedDateTime.from("2021-08-01T01:00:00[Europe/London]");
const dt2 = Temporal.ZonedDateTime.from("2021-08-02T00:00:00[Europe/London]");
console.log(Temporal.ZonedDateTime.compare(dt1, dt2)); // -1

const dt3 = Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Europe/London]");
console.log(Temporal.ZonedDateTime.compare(dt1, dt3)); // 1
```

Sorting an array of date-times

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```
const dateTimes = [
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[America/New_York]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Asia/Hong_Kong]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Europe/London]"),
];

dateTimes.sort(Temporal.ZonedDateTime.compare);
console.log(dateTimes.map((d) => d.toString()));
// [ "2021-08-01T00:00:00+08:00[Asia/Hong_Kong]", "2021-08-01T00:00:00+01:00[Europe/London]", "2021-08-01T00:00:00-04:00[America/New_York]" ]
```

Note that they are compared by their instant values. In the very rare case where you want to compare them by their wall-clock times, convert them to `PlainDateTime` first.

```
const dateTimes = [
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[America/New_York]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Asia/Hong_Kong]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Europe/London]"),
];

dateTimes.sort((a, b) =>
  Temporal.PlainDateTime.compare(a.toPlainDateTime(), b.toPlainDateTime()),
);
console.log(dateTimes.map((d) => d.toString()));
// [ "2021-08-01T00:00:00-04:00[America/New_York]", "2021-08-01T00:00:00+08:00[Asia/Hong_Kong]", "2021-08-01T00:00:00+01:00[Europe/London]" ]
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.equals()`

Temporal.ZonedDateTime.prototype.day

The **`day`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.day`.

For `PlainDate`, `day` can only be non-continuous if the calendar skips days. For `ZonedDateTime`, `day` can also be non-continuous if the time zone changes its offset by 24 hours; this actually happened. See the example below.

Examples

Using day

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.day); // 1
```

Non-continuous day

To better align times with its trading partners in Asia, the country of Samoa changed its time zone to the other side of the International Date Line, shifting its offset from -10:00 to +14:00 (daylight saving time). This resulted in a 24-hour abrupt change in the local time, therefore skipping the day December 30, 2011 entirely. `2011-12-29T23:59:59-10:00[Pacific/Apia]` is immediately followed by `2011-12-31T00:00:00+14:00[Pacific/Apia]`.

```
const dt = Temporal.ZonedDateTime.from(
  "2011-12-29T23:59:59-10:00[Pacific/Apia]",
);
console.log(dt.day); // 29
const nextDay = dt.add({ seconds: 1 });
console.log(nextDay.day); // 31
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `day` property.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.PlainDate.prototype.day`

Temporal.ZonedDateTime.prototype.dayOfWeek

The **`dayOfWeek`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. It is calendar-dependent.

The set accessor of `dayOfWeek` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `dayOfWeek` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

For general information and more examples, see `Temporal.PlainDate.prototype.dayOfWeek`.

Examples

Using dayOfWeek

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.dayOfWeek); // 4; Thursday
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.ZonedDateTime.prototype.weekOfYear`

- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.dayOfWeek`

Temporal.ZonedDateTime.prototype.dayOfYear

The **`dayOfYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. It is calendar-dependent.

The set accessor of `dayOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `dayOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

For general information and more examples, see `Temporal.PlainDate.prototype.dayOfYear`.

Examples

Using dayOfYear

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.dayOfYear); // 182
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.dayOfYear`

Temporal.ZonedDateTime.prototype.daysInMonth

The **`daysInMonth`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

The set accessor of `daysInMonth` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInMonth`.

Examples

Using daysInMonth

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.daysInMonth); // 31
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInMonth`

Temporal.ZonedDateTime.prototype.daysInWeek

The **`daysInWeek`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of days in the week of this date. It is calendar-dependent.

The set accessor of `daysInWeek` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInWeek`.

Examples

Using daysInWeek

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.daysInWeek); // 7
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`

- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.ZonedDateTime.prototype.weekOfYear`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInWeek`

Temporal.ZonedDateTime.prototype.daysInYear

The **`daysInYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

The set accessor of `daysInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInYear`.

Examples

Using daysInYear

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.daysInYear); // 365
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.ZonedDateTime.prototype.epochMilliseconds

The **`epochMilliseconds`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. It is equivalent to dividing `epochNanoseconds` by `1e6` and flooring the result.

The set accessor of `epochMilliseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `epochMilliseconds` value, see below.

Examples

Using epochMilliseconds

```
const zdt = Temporal.ZonedDateTime.from("2021-08-01T12:34:56.789Z[UTC]");
console.log(zdt.epochMilliseconds); // 1627821296789

const zdt2 = Temporal.ZonedDateTime.from("1969-08-01T12:34:56.789Z[UTC]");
console.log(zdt2.epochMilliseconds); // -13173903211
```

Creating a ZonedDateTime object from an epochMilliseconds value

You can create a `Temporal.ZonedDateTime` object from an `epochMilliseconds` value by first constructing a `Temporal.Instant` object using `Temporal.Instant.fromEpochMilliseconds()`, and then converting it to a `Temporal.ZonedDateTime` object using `Temporal.Instant.prototype.toZonedDateTimeISO()`:

```
const epochMilliseconds = 1627821296789;
const instant = Temporal.Instant.fromEpochMilliseconds(epochMilliseconds);
const zdt = instant.toZonedDateTimeISO("UTC");
console.log(zdt.toString()); // 2021-08-01T12:34:56.789+00:00[UTC]
```

Alternatively, use the `Temporal.ZonedDateTime()` constructor, but convert the milliseconds to nanoseconds first:

```
const epochMilliseconds = 1627821296789;
const epochNanoseconds = BigInt(epochMilliseconds) * 1_000_000n;
const zdt = new Temporal.ZonedDateTime(epochNanoseconds, "UTC");
console.log(zdt.toString()); // 2021-08-01T12:34:56.789+00:00[UTC]
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.epochNanoseconds`

Temporal.ZonedDateTime.prototype.epochNanoseconds

The **`epochNanoseconds`** accessor property of `Temporal.ZonedDateTime` instances returns a `BigInt` representing the number of nanoseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

The set accessor of `epochNanoseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `epochNanoseconds` value, use the `new Temporal.ZonedDateTime()` constructor instead.

An instant can only represent $\pm 10^8$ days (about ±273,972.6 years) around the epoch, which is ±8.64e21 nanoseconds. Attempting to set `epochNanoseconds` beyond this boundary throws a `RangeError`.

Examples

Using epochNanoseconds

```
const zdt = Temporal.ZonedDateTime.from("2021-08-01T12:34:56.789Z[UTC]");
console.log(zdt.epochNanoseconds); // 1627821296789000000n

const zdt2 = Temporal.ZonedDateTime.from("1969-08-01T12:34:56.789Z[UTC]");
console.log(zdt2.epochNanoseconds); // -13173903211000000n
```

Creating a ZonedDateTime object from an epochNanoseconds value

You can create a `Temporal.ZonedDateTime` object from an `epochNanoseconds` value using the `Temporal.ZonedDateTime()` constructor.

```
const epochNanoseconds = 1627821296789000000n;
const zdt = new Temporal.ZonedDateTime(epochNanoseconds, "UTC");
console.log(zdt.toString()); // 2021-08-01T12:34:56.789+00:00[UTC]
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.epochMilliseconds`

Temporal.ZonedDateTime.prototype.equals()

The **`equals()`** method of `Temporal.ZonedDateTime` instances returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`), and `false` otherwise. They are compared both by their instant values, time zones, and their calendars, so two date-times from different calendars or time zones may be considered equal by `Temporal.ZonedDateTime.compare()` but not by `equals()`.

Syntax

```
equals(other)
```

Parameters

`other`

> A string, an object, or a `Temporal.ZonedDateTime` instance representing the other date-time to compare. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`.

Return value

`true` if this date-time is equal to `other` both in their instant value, time zone, and their calendar, `false` otherwise.

Note that the time zones are canonicalized before comparison, so if their time zone IDs are both named and identify the same time zone, then they would be considered the same even when the exact names may be aliases of each other. Offset identifiers are compared by the offset values they represent. Offset identifiers never compare equal to named identifiers even when the named identifier's time zone always uses that offset.

Examples

Using equals()

```
// Asia/Kolkata and Asia/Calcutta are aliases of each other
const dt1 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56+05:30[Asia/Kolkata]",
);
const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56+05:30[Asia/Calcutta]",
);
console.log(dt1.equals(dt2)); // true

const dt3 = Temporal.ZonedDateTime.from("2021-07-01T12:34:56+05:30[+05:30]");
console.log(dt1.equals(dt3)); // false

const dt4 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56+05:30[Asia/Kolkata][u-ca=buddhist]",
);
console.log(dt1.equals(dt4)); // false
```

Testing if two time zone identifiers are equivalent

```
function sameTimeZone(timeZone1, timeZone2) {
  const dt1 = Temporal.ZonedDateTime.from({
```

```
    year: 2021,
    month: 7,
    day: 1,
    timeZone: timeZone1,
  });
  const dt2 = Temporal.ZonedDateTime.from({
    year: 2021,
    month: 7,
    day: 1,
    timeZone: timeZone2,
  });
  return dt1.equals(dt2);
}

console.log(sameTimeZone("Asia/Kolkata", "Asia/Calcutta")); // true
console.log(sameTimeZone("Asia/Shanghai", "Asia/Taipei")); // false
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.compare()`

Temporal.ZonedDateTime.prototype.era

The **`era`** accessor property of `Temporal.ZonedDateTime` instances returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.era`.

Examples

Using era

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.era); // undefined

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01[America/New_York][u-ca=gregory]",
);
console.log(dt2.era); // ce
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.eraYear`
- `Temporal.PlainDate.prototype.era`

Temporal.ZonedDateTime.prototype.eraYear

The **`eraYear`** accessor property of `Temporal.ZonedDateTime` instances returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.eraYear`.

Examples

Using eraYear

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.eraYear); // undefined

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01[America/New_York][u-ca=gregory]",
);
console.log(dt2.eraYear); // 2021

const dt3 = Temporal.ZonedDateTime.from(
  "-002021-07-01[America/New_York][u-ca=gregory]",
);
console.log(dt3.eraYear); // 2022; 0000 is used for the year 1 BC

const dt4 = Temporal.ZonedDateTime.from(
```

```
  "2021-07-01[America/New_York][u-ca=japanese]",
);
console.log(dt4.eraYear); // 3
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

Temporal.ZonedDateTime.from()

The **`Temporal.ZonedDateTime.from()`** static method creates a new `Temporal.ZonedDateTime` object from another `Temporal.ZonedDateTime` object, an object with date, time, and time zone properties, or an RFC 9557 string.

Syntax

```
Temporal.ZonedDateTime.from(info)
Temporal.ZonedDateTime.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.ZonedDateTime` instance, which creates a copy of the instance.
- An RFC 9557 format string containing a date, optionally a time, optionally an offset, a time zone annotation, and optionally a calendar.
- An object containing properties that are accepted by either `Temporal.PlainDate.from()` (`calendar`, `era`, `eraYear`, `year`, `month`, `monthCode`, `day`) or `Temporal.PlainTime.from()` (`hour`, `minute`, `second`, `millisecond`, `microsecond`, `nanosecond`). The info should explicitly specify a year (as `year` or as `era` and `eraYear`), a month (as `month` or `monthCode`), and a day; others are optional and will be set to their default values. The following properties should be provided too:

  `timeZone`

  Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information). The time properties are interpreted in this time zone.

  `offset` Optional

  An offset string, in the same format as the RFC 9557 offset but with optional seconds and subsecond components (`±HH:mm:ss.sssssssss`), representing the offset from UTC. If omitted, it will be calculated from the time zone and the date-time. `"Z"` is not allowed.

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`disambiguation` Optional

What to do if the local date-time is ambiguous in the given time zone (there are more than one instants with such local time, or the local time does not exist). Possible values are `"compatible"`, `"earlier"`, `"later"`, and `"reject"`. Defaults to `"compatible"`. For more information about these values, see ambiguity and gaps from local time to UTC time.

`offset` Optional

What to do if the offset is explicitly provided in `info` but the offset is invalid for the given time zone in the given local time. Possible values are `"use"`, `"ignore"`, `"reject"`, and `"prefer"`. Defaults to `"reject"`. For more information about these values, see offset ambiguity.

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object, representing the date and time specified by `info` in the specified `calendar` and `timeZone`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month` (or `monthCode`), and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The wall-clock time is ambiguous in the time zone, and `options.disambiguation` is set to `"reject"`.
- The info is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a ZonedDateTime from an object

```
// Year + month + day + hour + minute + second
const zdt = Temporal.ZonedDateTime.from({
  timeZone: "America/New_York",
  year: 2021,
  month: 7,
  day: 1,
  hour: 12,
  minute: 34,
  second: 56,
});
console.log(zdt.toString()); // "2021-07-01T12:34:56-04:00[America/New_York]"
```

Creating a ZonedDateTime from a string

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56-04:00[America/New_York]",
);
console.log(zdt.toLocaleString()); // "7/1/2021, 12:34:56 PM EDT" (assuming en-US locale)

// Time given as UTC, and converted to local
const zdt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56Z[America/New_York]",
);
console.log(zdt2.toString()); // "2021-07-01T08:34:56-04:00[America/New_York]"
```

Creating a ZonedDateTime from an ISO 8601 / RFC 3339 string

Note that `Temporal.ZonedDateTime.from()` rejects ISO 8601 strings, which do not include a time zone identifier. This is to ensure that the time zone is always known and can be used to derive different offsets as the local time changes.

If you want to parse an ISO 8601 string, first construct a `Temporal.Instant` object and then convert it to a `Temporal.ZonedDateTime` object. You can provide any time zone, even if it doesn't match the offset originally given in the string, and the local time will be adjusted accordingly.

```
const isoString = "2021-07-01T12:34:56+02:00";
const instant = Temporal.Instant.from(isoString);
const zdt = instant.toZonedDateTimeISO("America/New_York");
console.log(zdt.toString()); // "2021-07-01T06:34:56-04:00[America/New_York]"
```

Local time disambiguation

See ambiguity and gaps from local time to UTC time for an introduction to this situation.

```
const localTimeNotExist = "2024-03-10T02:05:00[America/New_York]";
// For non-existent times, "compatible" is equivalent to "later"
const zdt = Temporal.ZonedDateTime.from(localTimeNotExist);
console.log(zdt.toString()); // "2024-03-10T03:05:00-04:00[America/New_York]"

const zdt2 = Temporal.ZonedDateTime.from(localTimeNotExist, {
  disambiguation: "earlier",
});
console.log(zdt2.toString()); // "2024-03-10T01:05:00-05:00[America/New_York]"

const localTimeAmbiguous = "2024-11-03T01:05:00[America/New_York]";
// For ambiguous times, "compatible" is equivalent to "earlier"
const zdt3 = Temporal.ZonedDateTime.from(localTimeAmbiguous);
console.log(zdt3.toString()); // "2024-11-03T01:05:00-04:00[America/New_York]"

const zdt4 = Temporal.ZonedDateTime.from(localTimeAmbiguous, {
  disambiguation: "later",
});
console.log(zdt4.toString()); // "2024-11-03T01:05:00-05:00[America/New_York]"
```

Resolving offset ambiguity

See offset ambiguity for an introduction to this situation.

```
const offsetAmbiguous = "2019-12-23T12:00:00-02:00[America/Sao_Paulo]";

Temporal.ZonedDateTime.from(offsetAmbiguous);
// RangeError: date-time can't be represented in the given time zone
Temporal.ZonedDateTime.from(offsetAmbiguous, { offset: "use" }).toString();
// "2019-12-23T11:00:00-03:00[America/Sao_Paulo]"
Temporal.ZonedDateTime.from(offsetAmbiguous, { offset: "ignore" }).toString();
// "2019-12-23T12:00:00-03:00[America/Sao_Paulo]"
```

For more examples, especially regarding different calendars and overflow settings, see `Temporal.PlainDate.from()` and `Temporal.PlainTime.from()`.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime()`
- `Temporal.ZonedDateTime.prototype.with()`

Temporal.ZonedDateTime.prototype.getTimeZoneTransition()

The **`getTimeZoneTransition()`** method of `Temporal.ZonedDateTime` instances returns a `Temporal.ZonedDateTime` object representing the closest instant after or before this instant at which the time zone's UTC offset changes (the returned instant is the first instant after the change), or `null` if there is no such transition. This is useful for finding out the offset rules of a time zone, such as its daylight saving time pattern.

Note that instants returned about the future may be unreliable, for example due to changes to the time zone definitions.

Syntax

```
getTimeZoneTransition(direction)
getTimeZoneTransition(options)
```

Parameters

`direction`

> A string representing the `direction` option. This is a convenience overload, so `getTimeZoneTransition(direction)` is equivalent to `getTimeZoneTransition({ direction })`, where `direction` is a string.

`options`

> An object containing the following property:
>
> `direction`
>
> > Whether to search before or after the current instant. Must be one of `"next"` or `"previous"`.

Return value

A `Temporal.ZonedDateTime` object with instant `t`, such that:

- The time zone offset at `t` is different from the offset one nanosecond before `t`.
- `t < this.epochNanoseconds` if `direction` is `"previous"`, or `t > this.epochNanoseconds` if `direction` is `"next"`.
- For all instants between `this.epochNanoseconds` and `t`, exclusive, the offset is constant.

If there is no such transition, `null` is returned.

Examples

Finding the next time zone transition

```
const dt = Temporal.ZonedDateTime.from("2024-01-01T00-05:00[America/New_York]");
const transition = dt.getTimeZoneTransition("next");
console.log(transition.toString()); // "2024-03-10T03:00:00-04:00[America/New_York]"

const transition2 = transition.getTimeZoneTransition("next");
console.log(transition2.toString()); // "2024-11-03T01:00:00-05:00[America/New_York]"

const transition3 = dt.getTimeZoneTransition("previous");
console.log(transition3.toString()); // "2023-11-05T01:00:00-05:00[America/New_York]"

const dt2 = Temporal.ZonedDateTime.from("2024-01-01T00Z[UTC]");
console.log(dt2.getTimeZoneTransition("next")); // null
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.timeZoneId`
- `Temporal.ZonedDateTime.prototype.offset`

Temporal.ZonedDateTime.prototype.hour

The **`hour`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 23 representing the hour component of this time.

The set accessor of `hour` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.hour`.

For `ZonedDateTime`, `hour` can be non-continuous due to offset changes such as daylight saving time transitions. In this case, the hour may be repeated or skipped.

Examples

Using hour

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.hour); // 12
```

Non-continuous hour

Non-continuous hour is very common due to daylight saving time transitions, which is explained more in Ambiguity and gaps from local time to UTC time.

```
const dt = Temporal.ZonedDateTime.from(
  "2024-11-03T01:59:00-04:00[America/New_York]",
);
console.log(dt.hour); // 1
const dt2 = dt.add({ minutes: 1 });
console.log(dt2.hour); // 1
console.log(dt2.toString()); // 2024-11-03T01:00:00-05:00[America/New_York]

const dt3 = Temporal.ZonedDateTime.from(
  "2024-03-10T01:59:00-05:00[America/New_York]",
);
console.log(dt3.hour); // 1
const dt4 = dt3.add({ minutes: 1 });
console.log(dt4.hour); // 3
console.log(dt4.toString()); // 2024-03-10T03:00:00-04:00[America/New_York]
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `hour` property.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.hour`

Temporal.ZonedDateTime.prototype.hoursInDay

The **`hoursInDay`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of hours in the day of this date in the time zone. It may be more or less than 24 in the case of offset changes such as daylight saving time.

Because `ZonedDateTime` is the only class that's time zone-aware, and hours in a day can only change by offset changes, all other classes assume 24-hour days.

The set accessor of `hoursInDay` is `undefined`. You cannot change this property directly.

Examples

Using hoursInDay

```
const dt = Temporal.ZonedDateTime.from(
  "2024-03-10T01:58:00-05:00[America/New_York]",
);
console.log(dt.hoursInDay); // 23; this is the day of transition into DST

const dt2 = Temporal.ZonedDateTime.from(
  "2024-11-03T01:58:00-04:00[America/New_York]",
);
console.log(dt2.hoursInDay); // 25; this is the day of transition out of DST

const dt3 = Temporal.ZonedDateTime.from(
  "2024-11-04T01:58:00-05:00[America/New_York]",
);
console.log(dt3.hoursInDay); // 24
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`

- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.hour`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.daysInWeek`

Temporal.ZonedDateTime.prototype.inLeapYear

The **`inLeapYear`** accessor property of `Temporal.ZonedDateTime` instances returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.inLeapYear`.

Examples

Using inLeapYear

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.inLeapYear); // false
console.log(dt.daysInYear); // 365
console.log(dt.monthsInYear); // 12
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.ZonedDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.inLeapYear`

Temporal.ZonedDateTime.prototype.microsecond

The **`microsecond`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

The set accessor of `microsecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.microsecond`.

Examples

Using microsecond

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.microsecond); // 456
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.second`
- `Temporal.ZonedDateTime.prototype.millisecond`
- `Temporal.ZonedDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.microsecond`

Temporal.ZonedDateTime.prototype.millisecond

The **`millisecond`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

The set accessor of `millisecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.millisecond`.

Examples

Using millisecond

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
```

```
);
console.log(dt.millisecond); // 123
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.second`
- `Temporal.ZonedDateTime.prototype.microsecond`
- `Temporal.ZonedDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.millisecond`

Temporal.ZonedDateTime.prototype.minute

The **`minute`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 59 representing the minute component of this time.

The set accessor of `minute` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.minute`.

For `ZonedDateTime`, `minute` can be non-continuous due to offset changes. While much rarer than `hour` changes (because daylight saving time shifts are usually by whole hours), it can still happen.

Examples

Using minute

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.minute); // 34
```

Non-continuous minute

Typically, `minute` always goes from 0 to 59 and then back to 0, even when passing through a daylight saving time transition. There's one particular case where DST has a 30-minute offset: Lord Howe Island in Australia, which shifts between +10:30 and +11:00. In this case, the minute can be non-continuous.

```
const dt = Temporal.ZonedDateTime.from(
  "2021-10-03T01:59:00+10:30[Australia/Lord_Howe]",
);
console.log(dt.minute); // 59
const dt2 = dt.add({ minutes: 1 });
console.log(dt2.minute); // 30
console.log(dt2.toString()); // 2021-10-03T02:30:00+11:00[Australia/Lord_Howe]
```

There's a second peculiar case where the minute can be non-continuous: the standardization of hourly time zones. In the early 20th century, most countries were using their own time zones which were often not a whole hour offset from UTC. For example, Paris used to have an offset of UTC+0:09:21, which was changed to UTC+0 on March 11, 1911.

```
const dt = Temporal.ZonedDateTime.from(
  "1911-03-10T23:59:00+00:09:21[Europe/Paris]",
);
console.log(dt.minute); // 59
const dt2 = dt.add({ minutes: 1 });
console.log(dt2.minute); // 50
console.log(dt2.toString()); // 1911-03-10T23:50:39+00:00[Europe/Paris]
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `minute` property.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.minute`

Temporal.ZonedDateTime.prototype.month

The **`month`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.month`.

Examples

Using month

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.monthCode); // "M07"
console.log(dt.month); // 7
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.ZonedDateTime.prototype.monthCode`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.month`

Temporal.ZonedDateTime.prototype.monthCode

The **`monthCode`** accessor property of `Temporal.ZonedDateTime` instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`). If the leap month is the first month of the year, the code is `M00L`.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.monthCode`.

Examples

Using monthCode

```
const date = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.monthCode`

Temporal.ZonedDateTime.prototype.monthsInYear

The **`monthsInYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.monthsInYear`.

Examples

Using monthsInYear

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.monthsInYear); // 12
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.monthCode`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

Temporal.ZonedDateTime.prototype.nanosecond

The **`nanosecond`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

The set accessor of `nanosecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.nanosecond`.

Examples

Using nanosecond

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.nanosecond); // 789
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.second`
- `Temporal.ZonedDateTime.prototype.millisecond`
- `Temporal.ZonedDateTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

Temporal.ZonedDateTime.prototype.offset

The **`offset`** accessor property of `Temporal.ZonedDateTime` instances returns a string representing the offset used to interpret the internal instant, in the form `±HH:mm` (or `±HH:mm:ss.sssssssss` with as much subminute precision as necessary). This offset is guaranteed to be valid for the given instant and time zone at construction time.

The set accessor of `offset` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value (usually also changing the date/time), or use the `withTimeZone()` method to create a new `Temporal.ZonedDateTime` object in another time zone.

Examples

Using offset

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
console.log(dt.offset); // "-07:00"

const dt2 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00-07[-07]");
console.log(dt2.offset); // "-07:00"

const dt3 = Temporal.ZonedDateTime.from(
  "1900-01-01T00:00:00+00:09:21[Europe/Paris]",
);
console.log(dt3.offset); // "+00:09:21"

const dt4 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00Z[Asia/Shanghai]");
console.log(dt4.offset); // "+08:00"
```

Changing offset

If the local time happens to have two valid offsets, such as within a DST transition, you can change the offset without changing anything else:

```
const zdt = Temporal.ZonedDateTime.from(
  "2024-11-03T01:05:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ offset: "-05:00" });
console.log(newZDT.toString()); // "2024-11-03T01:05:00-05:00[America/New_York]"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.timeZoneId`
- `Temporal.ZonedDateTime.prototype.offsetNanoseconds`

Temporal.ZonedDateTime.prototype.offsetNanoseconds

The **`offsetNanoseconds`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the offset used to interpret the internal instant, as a number of nanoseconds (positive or negative). The value is a safe integer because it's less than a day, which is 8.64e15 nanoseconds.

The set accessor of `offsetNanoseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new `offset` value (which will change this property too), or use the `withTimeZone()` method to create a new `Temporal.ZonedDateTime` object in another

time zone.

Examples

Using offsetNanoseconds

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
console.log(dt.offsetNanoseconds); // -25200000000000

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+08:00[Asia/Shanghai]",
);
console.log(dt2.offsetNanoseconds); // 28800000000000

const dt3 = Temporal.ZonedDateTime.from(
  "1900-01-01T00:00:00+00:09:21[Europe/Paris]",
);
console.log(dt3.offsetNanoseconds); // 561000000000
```

Here's one way to get a `ZonedDateTime` representing the same wall-clock time in UTC:

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
const dtInUTC = dt.add({ nanoseconds: dt.offsetNanoseconds });
console.log(dtInUTC.withTimeZone("UTC").toString()); // "2021-07-01T12:00:00+00:00[UTC]"
```

Here's a better way to get the same result:

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
const dtInUTC = dt.toPlainDateTime().toZonedDateTime("UTC");
console.log(dtInUTC.toString()); // "2021-07-01T12:00:00+00:00[UTC]"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.offset`
- `Temporal.ZonedDateTime.prototype.timeZoneId`

Temporal.ZonedDateTime.prototype.round()

The **`round()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time rounded to the given unit.

Syntax

```
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. For all values of `smallestUnit` except `"day"`, the increment must be a divisor of the maximum value of the unit; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12. For `"day"`, the increment must be 1.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"day"`, `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.ZonedDateTime` object representing this date-time rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

If `smallestUnit` is `"day"`, the returned date-time will be the start of day of this date or the next day, depending on the `roundingMode` and the distance to these two instants. Otherwise, the rounding is first performed on its `PlainDateTime` (same as `Temporal.PlainDateTime.prototype.round()`), and then re-interpreted in the same time zone, with `disambiguation: "compatible", offset: "prefer"`. See ambiguity and gaps from local time to UTC time and offset ambiguity.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789[America/New_York]",
);
const nearestMillisecond = zdt.round("millisecond");
console.log(nearestMillisecond.toString()); // 2021-07-01T12:34:56.123-04:00[America/New_York]

const nearestHalfHour = zdt.round({
  smallestUnit: "minute",
  roundingIncrement: 30,
});
console.log(nearestHalfHour.toString()); // 2021-07-01T12:30:00-04:00[America/New_York]

const nextDay = zdt.round({ smallestUnit: "day", roundingMode: "ceil" });
console.log(nextDay.toString()); // 2021-07-02T00:00:00-04:00[America/New_York]
```

Ambiguity after rounding

It's possible that the rounded date-time is ambiguous in the given time zone. The ambiguity is always resolved using `disambiguation: "compatible", offset: "prefer"`. Here's a quick example:

```
const zdt = Temporal.ZonedDateTime.from(
  "2024-03-10T01:00:00-05:00[America/New_York]",
);
const rounded = zdt.round({ smallestUnit: "hour", roundingIncrement: 2 });
// The result is supposed to be 2024-03-10T02:00:00-05:00[America/New_York],
// but this time does not exist. `disambiguation: "compatible"` tells us to move
// forward by 1 hour.
console.log(rounded.toString()); // 2024-03-10T03:00:00-04:00[America/New_York]
```

See also

- `Temporal.ZonedDateTime`

Temporal.ZonedDateTime.prototype.second

The **`second`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 59 representing the second component of this time.

The set accessor of `second` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.second`.

For `ZonedDateTime`, `second` can be non-continuous due to offset changes. While much rarer than `hour` or `minute` changes (because daylight saving time shifts are usually by whole hours), it can still happen.

Examples

Using second

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.second); // 56
```

Non-continuous second

Typically, `second` always goes from 0 to 59 and then back to 0, even when passing through a daylight saving time transition. There's a peculiar case where the second can be non-continuous: the standardization of hourly time zones. In the early 20th century, most countries were using their own time zones which were often not a whole hour offset from UTC. For example, Paris used to have an offset of UTC+0:09:21, which was changed to UTC+0 on March 11, 1911.

```
const dt = Temporal.ZonedDateTime.from(
  "1911-03-10T23:59:59+00:09:21[Europe/Paris]",
);
console.log(dt.second); // 59
const dt2 = dt.add({ seconds: 1 });
console.log(dt2.second); // 39
console.log(dt2.toString()); // 1911-03-10T23:50:39+00:00[Europe/Paris]
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `second` property.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.millisecond`
- `Temporal.ZonedDateTime.prototype.microsecond`
- `Temporal.ZonedDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.second`

Temporal.ZonedDateTime.prototype.since()

The **`since()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```
since(other)
since(other, options)
```

Parameters

`other`

> A string, an object, or a `Temporal.ZonedDateTime` instance representing a date-time to subtract from this date-time. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

> An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` accept all possible units. For `largestUnit`, the default value `"auto"` means `"hours"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"nanoseconds"`. The current date is used as the `relativeTo` option. Note that using units larger than `"hours"` may make the duration not portable to other calendars, dates, or time zones.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this date-time. The duration is positive if `other` is before this date-time, and negative if after.

Exceptions

`RangeError`

> Thrown in one of the following cases:
>
> - `other` has a different calendar than `this`.
> - Any of the options is invalid.
> - `other` has a different time zone than `this`, and `largestUnit` is `"days"` or above.

Description

The duration returned is a "hybrid" duration. This means that the duration's date portion represents full calendar days like `Temporal.PlainDateTime.prototype.since()` would return, while its time portion represents real-world elapsed time like `Temporal.Instant.prototype.since()` would return. This "hybrid duration" approach automatically adjusts for DST and matches widely-adopted industry standards like RFC 5545 (iCalendar). See below for examples.

Examples

Offset transitions

When transitions happen, a day may not have exactly 24 hours.

```
const start = Temporal.ZonedDateTime.from(
  "2024-11-03T01:00:00-04:00[America/New_York]",
);
const end = Temporal.ZonedDateTime.from(
  "2024-11-04T01:00:00-05:00[America/New_York]",
);
console.log(end.since(start).toString()); // PT25H
console.log(end.since(start, { largestUnit: "days" }).toString()); // PT1D

const start2 = Temporal.ZonedDateTime.from(
  "2024-03-10T01:00:00-05:00[America/New_York]",
);
const end2 = Temporal.ZonedDateTime.from(
  "2024-03-11T01:00:00-04:00[America/New_York]",
);
console.log(end2.since(start2).toString()); // PT23H
console.log(end2.since(start2, { largestUnit: "days" }).toString()); // PT1D
```

For this reason, the returned duration is purely time-based with no date portion by default, so that it stays unambiguous.

Different time zones

The time portion of the returned duration is purely based on instants and is not affected by time zones. However, if you want to include any date units like `day`, then the start and end must be in the same time zone.

```
const start = Temporal.ZonedDateTime.from(
  "2024-11-03T01:00:00-04:00[America/New_York]",
);
// Peru does not use DST so its offset remains -05:00 year-round
const end = Temporal.ZonedDateTime.from(
  "2024-11-04T01:00:00-05:00[America/Lima]",
);

end.since(start); // PT25H
end.since(start, { largestUnit: "days" }); // RangeError: time zones "America/Lima" and "America/New_York" aren't compatible

end.withTimeZone("America/New_York").since(start, { largestUnit: "days" }); // P1D
end.since(start.withTimeZone("America/Lima"), { largestUnit: "days" }); // P1D1H
```

For more examples about how to use `since()`, especially with rounding, see `Temporal.PlainDate.prototype.since()` and `Temporal.PlainTime.prototype.since()`.

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.until()`

Temporal.ZonedDateTime.prototype.startOfDay()

The **`startOfDay()`** method of `Temporal.ZonedDateTime` instances returns a `Temporal.ZonedDateTime` object representing the first instant of this date in the time zone. It usually has a time of `00:00:00`, but may be different if the midnight doesn't exist due to offset changes, in which case the first time that exists is returned.

It is equivalent to calling `withPlainTime()` with no arguments.

Syntax

```
startOfDay()
```

Parameters

None.

Return value

A `Temporal.ZonedDateTime` object with instant `t`, such that:

- The date at `t` is different from the date one nanosecond before `t`.
- The date at `t` is the same as the date of `this`.

Examples

Using startOfDay()

```
// In the US, DST transitions happen at 2am, so the midnight exists
const dt = Temporal.ZonedDateTime.from(
  "2024-03-10T12:00:00-04:00[America/New_York]",
);
console.log(dt.startOfDay().toString()); // "2024-03-10T00:00:00-05:00[America/New_York]"

// In Brazil, DST transitions happened at midnight, so the midnight didn't exist
const dt2 = Temporal.ZonedDateTime.from(
  "2015-10-18T12:00-02:00[America/Sao_Paulo]",
);
console.log(dt2.startOfDay().toString()); // "2015-10-18T01:00:00-02:00[America/Sao_Paulo]"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.hour`
- `Temporal.ZonedDateTime.prototype.hoursInDay`
- `Temporal.ZonedDateTime.prototype.timeZoneId`
- `Temporal.ZonedDateTime.prototype.getTimeZoneTransition()`

Temporal.ZonedDateTime.prototype.subtract()

The **`subtract()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two date-times and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object representing the date-time specified by the original `ZonedDateTime`, minus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a duration

```
const start = Temporal.ZonedDateTime.from(
  "2021-01-01T12:34:56-05:00[America/New_York]",
);
const end = start.subtract({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2019-10-07T07:28:48.992-04:00[America/New_York]
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.since()`
- `Temporal.ZonedDateTime.prototype.until()`

Temporal.ZonedDateTime.prototype.timeZoneId

The **`timeZoneId`** accessor property of `Temporal.ZonedDateTime` instances returns a string representing the time zone identifier used to interpret the internal instant. The string is either a named identifier in the preferred case (such as `"America/New_York"`), or an offset in the form `"±hh:mm"`. If the time zone has aliases, the `timeZoneId` is the identifier used to create the `ZonedDateTime`, without canonicalization to the primary identifier.

The set accessor of `timeZoneId` is `undefined`. You cannot change this property directly. Use the `withTimeZone()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

> **Note:** This string is not intended for display to users. Use `toLocaleString()` with the appropriate options to get a localized string.

Examples

Using timeZoneId

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
console.log(dt.timeZoneId); // "America/Los_Angeles"

const dt2 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00-07:00[-07:00]");
console.log(dt2.timeZoneId); // "-07:00"

const dt3 = dt2.withTimeZone("Asia/Shanghai");
console.log(dt3.timeZoneId); // "Asia/Shanghai"
```

The `timeZoneId` is never canonicalized to the primary identifier; it is the same as the one used to create the `ZonedDateTime`.

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+07:00[Asia/Ho_Chi_Minh]",
);
const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+07:00[Asia/Saigon]",
);
console.log(dt.timeZoneId); // "Asia/Ho_Chi_Minh"
console.log(dt2.timeZoneId); // "Asia/Saigon"
```

However, presentational differences will get canonicalized.

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+07:00[asia/ho_chi_minh]",
);
console.log(dt.timeZoneId); // "Asia/Ho_Chi_Minh"

const dt2 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00+07:00[+07]");
console.log(dt2.timeZoneId); // "+07:00"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.offset`
- `Temporal.ZonedDateTime.prototype.offsetNanoseconds`

Temporal.ZonedDateTime.prototype.toInstant()

The **`toInstant()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.Instant` object representing the instant of this date-time.

Syntax

```
toInstant()
```

Parameters

None.

Return value

A new `Temporal.Instant` object representing the instant of this date-time.

Examples

Using toInstant()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const instant = zdt.toInstant();
console.log(instant.toString()); // 2021-07-01T16:34:56.987654321Z
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.Instant`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`
- `Temporal.Instant.prototype.toZonedDateTimeISO()`

Temporal.ZonedDateTime.prototype.toJSON()

The **`toJSON()`** method of `Temporal.ZonedDateTime` instances returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given date-time in the RFC 9557 format, with the calendar annotation included if it is not `"iso8601"`, and the offset and time zone annotation always included.

Description

The `toJSON()` method is automatically called by `JSON.stringify()` when a `Temporal.ZonedDateTime` object is stringified. This method is generally intended to, by default, usefully serialize `Temporal.ZonedDateTime` objects during JSON serialization, which can then be deserialized using the `Temporal.ZonedDateTime.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```
const zdt = Temporal.ZonedDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  timeZone: "America/New_York",
});
const zdtStr = zdt.toJSON(); // '2021-08-01T00:00:00-04:00[America/New_York]'
const zdt2 = Temporal.ZonedDateTime.from(zdtStr);
```

JSON serialization and parsing

This example shows how `Temporal.ZonedDateTime` can be serialized as JSON without extra effort, and how to parse it back.

```
const zdt = Temporal.ZonedDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  timeZone: "America/New_York",
});
const jsonStr = JSON.stringify({ meeting: zdt }); // '{"meeting":"2021-08-01T00:00:00-04:00[America/New_York]"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "meeting") {
    return Temporal.ZonedDateTime.from(value);
  }
  return value;
});
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.toString()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

Temporal.ZonedDateTime.prototype.toLocaleString()

The **`toLocaleString()`** method of `Temporal.ZonedDateTime` instances returns a string with a language-sensitive representation of this date-time. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat` and passes this date-time converted to a `Temporal.Instant` (because `Intl.DateTimeFormat` cannot directly format a `Temporal.ZonedDateTime`).

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context. However, currently `Intl.DateTimeFormat` does not support formatting `Temporal.ZonedDateTime` objects, so you must convert them to `Temporal.Instant` objects first before passing them to `format()`.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If this date-time's calendar is not `"iso8601"`, the `calendar` option must be provided with the same value; otherwise, if this date-time's calendar is `"iso8601"`, the `calendar` option can be any value. The `timeZone` option must not be provided, as it is automatically set to be the date-time's `timeZoneId`. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year`, `month`, `day`, `hour`, `minute`, and `second` will default to `"numeric"`.
- Provide at least one of `dateStyle` or `timeStyle`: the date-time components will be set according to the specified style and the locale.
- Provide some date-time component options. Only the specified date-time components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given date-time according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, { ...options, timeZone: dateTime.timeZoneId }).format(dateTime.toInstant())`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56-04:00[America/New_York]",
);

console.log(zdt.toLocaleString()); // 8/1/2021, 12:34:56 PM EDT (assuming en-US locale)
```

If the date's calendar doesn't match the locale's default calendar, and the date's calendar is not `iso8601`, an explicit `calendar` option must be provided with the same value.

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56+09:00[Asia/Tokyo][u-ca=japanese]",
);
// The ja-JP locale uses the Gregorian calendar by default
zdt.toLocaleString("ja-JP", { calendar: "japanese" }); // R3/8/1 12:34:56 JST
```

Using toLocaleString() with options

You can customize which parts of the date are included in the output by providing the `options` parameter.

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56+09:00[Asia/Tokyo][u-ca=japanese]",
);
zdt.toLocaleString("ja-JP", {
  calendar: "japanese",
  dateStyle: "full",
  timeStyle: "full",
}); // 令和3年8月1日日曜日 12時34分56秒 日本標準時
zdt.toLocaleString("ja-JP", {
  calendar: "japanese",
  year: "numeric",
  month: "long",
  hour: "numeric",
```

```
  timeZoneName: "shortGeneric",
}); // 令和3年8月 12時 JST
zdt.toLocaleString("ja-JP", {
  calendar: "japanese",
  year: "numeric",
  hour: "numeric",
  minute: "numeric",
}); // 令和3年 12:34
```

See also

- `Temporal.ZonedDateTime`
- `Intl.DateTimeFormat`
- `Temporal.ZonedDateTime.prototype.toJSON()`
- `Temporal.ZonedDateTime.prototype.toString()`

Temporal.ZonedDateTime.prototype.toPlainDate()

The **`toPlainDate()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.PlainDate` object representing the date portion of this date-time.

Syntax

```
toPlainDate()
```

Parameters

None.

Return value

A new `Temporal.PlainDate` object representing the date portion of this date-time.

Examples

Using toPlainDate()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const plainDate = zdt.toPlainDate();
console.log(plainDate.toString()); // 2021-07-01
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.PlainDate`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`
- `Temporal.ZonedDateTime.prototype.toInstant()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`

Temporal.ZonedDateTime.prototype.toPlainDateTime()

The **`toPlainDateTime()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.PlainDateTime` object representing the date and time portions of this date-time. Only the time zone information is removed.

> **Warning:** After a `Temporal.ZonedDateTime` is converted to `Temporal.PlainDateTime`, it's no longer time-zone-aware. Subsequent operations like arithmetic or `with()` operations will not adjust for DST and may not yield the same results as equivalent operations with the original `Temporal.ZonedDateTime`. However, unless you perform those operations across a time zone offset transition, it's impossible to notice the difference. Therefore, be very careful when performing this conversion because subsequent results may be correct most of the time, but only turn out incorrect when moving across offset transitions like when DST starts or ends.

Syntax

```
toPlainDateTime()
```

Parameters

None.

Return value

A new `Temporal.PlainDateTime` object representing the date and time portions of this date-time.

Examples

Using toPlainDateTime()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
```

```
const plainDateTime = zdt.toPlainDateTime();
console.log(plainDateTime.toString()); // 2021-07-01T12:34:56.987654321
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.PlainDateTime`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toInstant()`
- `Temporal.PlainDateTime.prototype.toZonedDateTime()`

Temporal.ZonedDateTime.prototype.toPlainTime()

The **`toPlainTime()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.PlainTime` object representing the time portion of this date-time.

> **Warning:** After a `Temporal.ZonedDateTime` is converted to `Temporal.PlainTime`, it's no longer time-zone-aware. Subsequent operations like arithmetic or `with()` operations will not adjust for DST and may not yield the same results as equivalent operations with the original `Temporal.ZonedDateTime`. However, unless you perform those operations across a time zone offset transition, it's impossible to notice the difference. Therefore, be very careful when performing this conversion because subsequent results may be correct most of the time, but only turn out incorrect when moving across offset transitions like when DST starts or ends.

Syntax

```
toPlainTime()
```

Parameters

None.

Return value

A new `Temporal.PlainTime` object representing the time portion of this date-time.

Examples

Using toPlainTime()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const plainTime = zdt.toPlainTime();
console.log(plainTime.toString()); // 12:34:56.987654321
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.PlainTime`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`
- `Temporal.ZonedDateTime.prototype.toInstant()`

Temporal.ZonedDateTime.prototype.toString()

The **`toString()`** method of `Temporal.ZonedDateTime` instances returns a string representing this date-time in the RFC 9557 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.ZonedDateTime` instance, although the date value still remains the same.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

`timeZoneName` Optional

Whether to show the time zone name (`[time_zone_id]`) in the return value. Possible values are:

`"auto"` (default)

Always include the time zone name.

`"never"`

Never include the time zone name. This makes the returned string not recoverable to the same `Temporal.ZonedDateTime` instance.

`"critical"`

Always include the time zone name, and add a critical flag: `[!time)zone_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

`offset` Optional

Whether to show the offset (`±HH:mm`) in the return value. Possible values are:

`"auto"` (default)

Always include the offset.

`"never"`

Never include the offset. This makes the returned string not recoverable to the same `Temporal.ZonedDateTime` instance, if the time zone is included but the time is ambiguous, or if the time zone is also not included.

Return value

A string in the RFC 9557 format representing this date-time. The offset and calendar/time zone annotations are included as specified.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56[America/New_York]",
);
console.log(zdt.toString()); // '2021-08-01T12:34:56-04:00[America/New_York]'
```

Even for the `UTC` time zone, the offset is `+00:00`, not `Z`:

```
const zdt = Temporal.ZonedDateTime.from("2021-08-01T12:34:56[UTC]");
console.log(zdt.toString()); // '2021-08-01T12:34:56+00:00[UTC]'
```

Using options

For examples with rounding times, see `Temporal.PlainTime.prototype.toString()`. For examples with displaying calendars, see `Temporal.PlainDate.prototype.toString()`. Here we show controlling the display of time zone and offset:

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56[America/New_York]",
);
console.log(zdt.toString({ timeZoneName: "auto", offset: "never" })); // '2021-08-01T12:34:56[America/New_York]'
console.log(zdt.toString({ timeZoneName: "never", offset: "auto" })); // '2021-08-01T12:34:56-04:00'
console.log(zdt.toString({ timeZoneName: "never", offset: "never" })); // '2021-08-01T12:34:56'
console.log(zdt.toString({ timeZoneName: "critical", offset: "never" })); // '2021-08-01T12:34:56[!America/New_York]'
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.toJSON()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

Temporal.ZonedDateTime.prototype.until()

The **`until()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.ZonedDateTime` instance representing a date-time to subtract this date-time from. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this date-time *until* `other`. The duration is positive if `other` is after this date-time, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.
- `other` has a different time zone than `this`, and `largestUnit` is `"days"` or above.

Examples

Using until()

```
const flight = Temporal.ZonedDateTime.from(
  "2024-12-21T13:31:00-05:00[America/New_York]",
);
const now = Temporal.Now.zonedDateTimeISO("America/New_York").round("second");
if (Temporal.ZonedDateTime.compare(flight, now) < 0) {
  console.error(
    "The flight is already in the past. The result may not make sense.",
  );
}
const duration = now.until(flight, { largestUnit: "days" });
console.log(`The flight is in ${duration.toLocaleString("en-US")}`);
```

For more examples, see `since()`.

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.since()`

Temporal.ZonedDateTime.prototype.valueOf()

The **`valueOf()`** method of `Temporal.ZonedDateTime` instances throws a `TypeError`, which prevents `Temporal.ZonedDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `yearMonth1 > yearMonth2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.ZonedDateTime` instances prevent such implicit conversions. You need to explicitly convert them to numbers using `Temporal.ZonedDateTime.prototype.epochNanoseconds`, or use the `Temporal.ZonedDateTime.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.ZonedDateTime

All arithmetic and comparison operations on `Temporal.ZonedDateTime` instances should use the dedicated methods or convert them to primitives explicitly.

```
const zdt1 = Temporal.ZonedDateTime.from(
  "2022-01-01T00:00:00[America/New_York]",
);
const zdt2 = Temporal.ZonedDateTime.from(
  "2022-07-01T00:00:00[America/New_York]",
);
zdt1 > zdt2; // TypeError: can't convert ZonedDateTime to primitive type
Temporal.ZonedDateTime.compare(zdt1, zdt2); // -1

zdt2 - zdt1; // TypeError: can't convert ZonedDateTime to primitive type
zdt2.since(zdt1).toString(); // "PT4343H"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.toString()`
- `Temporal.ZonedDateTime.prototype.toJSON()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

Temporal.ZonedDateTime.prototype.weekOfYear

The **`weekOfYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. It is calendar-dependent.

The set accessor of `weekOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `weekOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `weeks`.

For general information and more examples, see `Temporal.PlainDate.prototype.weekOfYear`.

Examples

Using weekOfYear

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.weekOfYear); // 26
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.ZonedDateTime.prototype.daysInYear`

- `Temporal.PlainDate.prototype.weekOfYear`

Temporal.ZonedDateTime.prototype.with()

The **`with()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's fields.

To replace the `calendarId` property, use the `withCalendar()` method. To replace the `timeZoneId` property, use the `withTimeZone()` method.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.ZonedDateTime.from()` (except `calendar` and `timeZone`): `day`, `era` and `eraYear`, `hour`, `microsecond`, `millisecond`, `minute`, `month`, `monthCode`, `nanosecond`, `offset`, `second`, `year`. Unspecified properties use the values from the original date-time. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`disambiguation` Optional

What to do if the local date-time is ambiguous in the given time zone (there are more than one instants with such local time, or the local time does not exist). Possible values are `"compatible"`, `"earlier"`, `"later"`, and `"reject"`. Defaults to `"compatible"`. For more information about these values, see ambiguity and gaps from local time to UTC time.

`offset` Optional

What to do if the offset is explicitly provided in `info` but the offset is invalid for the given time zone in the given local time. Possible values are `"use"`, `"ignore"`, `"reject"`, and `"prefer"`. Defaults to `"prefer"`. For more information about these values, see offset ambiguity.

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date-time.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The wall-clock time represented by the provided properties is ambiguous in the time zone, and `options.disambiguation` is set to `"reject"`.
- The result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56[America/New_York]",
);
const newZDT = zdt.with({ hour: 13 });
console.log(newZDT.toString()); // "2021-07-01T13:34:56-04:00[America/New_York]"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Offset during date changes

By default, the `offset` option is set to `"prefer"`, which means we use the original offset (or that provided in `info`) if it's valid, and recalculate otherwise. This means if you set to another date that has a different offset due to a DST transition, the offset will be recalculated:

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ month: 12 });
// The offset is recalculated to -05:00
console.log(newZDT.toString()); // "2021-12-01T12:00:00-05:00[America/New_York]"
```

And if you set the time to within the DST transition, the offset is used to resolve the ambiguity:

```
const zdt = Temporal.ZonedDateTime.from(
  "2024-11-02T01:05:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ day: 3 });
console.log(newZDT.toString()); // "2024-11-03T01:05:00-04:00[America/New_York]"

const zdt2 = Temporal.ZonedDateTime.from(
  "2024-11-04T01:05:00-05:00[America/New_York]",
);
const newZDT2 = zdt2.with({ day: 3 });
console.log(newZDT2.toString()); // "2024-11-03T01:05:00-05:00[America/New_York]"
```

If you use `offset: "use"`, then the offset will be used as-is to obtain the exact time first, and *then* recalculate the offset:

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ month: 12 }, { offset: "use" });
// The offset is recalculated to -05:00, but the wall-clock time changes
console.log(newZDT.toString()); // "2021-12-01T11:00:00-05:00[America/New_York]"
```

You can also set `offset: "reject"` to throw an error if the original offset is invalid, forcing an explicit new offset to be specified:

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-04:00[America/New_York]",
);
zdt.with({ month: 12 }, { offset: "reject" });
// RangeError: date-time can't be represented in the given time zone
zdt.with({ month: 12, offset: "-05:00" }, { offset: "reject" }).toString();
// "2021-12-01T12:00:00-05:00[America/New_York]"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.withCalendar()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.withPlainTime()`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`

Temporal.ZonedDateTime.prototype.withCalendar()

The **`withCalendar()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time interpreted in the new calendar system. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's `calendarId` property.

To replace the date-time component properties, use the `with()` method. To replace its time zone, use the `withTimeZone()` method.

Syntax

```
withCalendar(calendar)
```

Parameters

`calendar`

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

Return value

A new `Temporal.ZonedDateTime` object, representing the date-time specified by the original `ZonedDateTime`, interpreted in the new calendar system.

Exceptions

`TypeError`

Thrown if `calendar` is not a string.

`RangeError`

Thrown if `calendar` is not a valid calendar identifier.

Examples

Using withCalendar()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56[America/New_York]",
);
const newZDT = zdt.withCalendar("islamic-umalqura");
console.log(newZDT.toLocaleString("en-US", { calendar: "islamic-umalqura" }));
// 11/21/1442 AH, 12:34:56 PM EDT
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.withPlainTime()`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.calendarId`

Temporal.ZonedDateTime.prototype.withPlainTime()

The **`withPlainTime()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`)

This method will replace all time properties, defaulting to `0` where properties are unspecified. If you only want to replace some of the time properties, use the `with()` method instead.

Syntax

```
withPlainTime()
withPlainTime(plainTime)
```

Parameters

`plainTime` Optional

A string, an object, or a `Temporal.PlainTime` instance representing the new time. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. If not specified, the time part is set to the start of the day (which is usually `00:00:00` unless it doesn't exist due to offset transitions). Disambiguation always happens in the `"compatible"` mode; if you want to use a different mode, use the `with()` method instead.

Return value

A new `Temporal.ZonedDateTime` object, with the date part and the time zone copied from the original date-time and the time part replaced by the new time.

Examples

Using withPlainTime()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56[America/New_York]",
);

// You can pass a string
const newZDT = zdt.withPlainTime("13:45:00");
console.log(newZDT.toString()); // "2021-07-01T13:45:00-04:00[America/New_York]"

// You can only specify some time properties, and the rest default to 0;
// for the with() method, they would be copied from the original date-time
const newZDT2 = zdt.withPlainTime({ hour: 13 });
console.log(newZDT2.toString()); // "2021-07-01T13:00:00-04:00[America/New_York]"

// You can pass nothing to set the time to midnight
const newZDT3 = zdt.withPlainTime();
console.log(newZDT3.toString()); // "2021-07-01T00:00:00-04:00[America/New_York]"

// But, if midnight doesn't exist, it may be a different time
const zdt2 = Temporal.ZonedDateTime.from(
  "2015-10-18T12:00-02:00[America/Sao_Paulo]",
);
console.log(zdt2.withPlainTime().toString()); // "2015-10-18T01:00:00-02:00[America/Sao_Paulo]"
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withCalendar()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`

- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.from()`

Temporal.ZonedDateTime.prototype.withTimeZone()

The **`withTimeZone()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing the same instant as this date-time but in the new time zone. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's `timeZoneId` property.

To replace the date-time component properties, use the `with()` method. To replace its calendar, use the `withCalendar()` method.

Syntax

```
withTimeZone(timeZone)
```

Parameters

`timeZone`

> Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

A new `Temporal.ZonedDateTime` object representing the same instant as this date-time but in the new time zone.

Exceptions

`TypeError`

> Thrown if `timeZone` is not a string or a `Temporal.ZonedDateTime` instance.

`RangeError`

> Thrown if the time zone name is invalid.

Examples

Using withTimeZone()

```
const meetingTime = Temporal.ZonedDateTime.from(
  "2021-08-01T12:00[America/New_York]",
);
const meetingTimeInParis = meetingTime.withTimeZone("Europe/Paris");
console.log(meetingTimeInParis.toString()); // 2021-08-01T18:00:00+02:00[Europe/Paris]
```

Replacing the time zone while keeping the same wall-clock time

In the rare case where you want to keep the wall-clock time the same but change the time zone (and result in a different instant), convert it to a `Temporal.PlainDateTime` first:

```
const meetingTime = Temporal.ZonedDateTime.from(
  "2021-08-01T12:00[America/New_York]",
);
const meetingTimeInParis = meetingTime
  .toPlainDateTime()
  .toZonedDateTime("Europe/Paris");
console.log(meetingTimeInParis.toString()); // 2021-08-01T12:00:00+02:00[Europe/Paris]
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withCalendar()`
- `Temporal.ZonedDateTime.prototype.withPlainTime()`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.timeZoneId`

Temporal.ZonedDateTime.prototype.year

The **`year`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. It is calendar-dependent.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.year`.

Examples

Using year

```
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.year); // 2021
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.era`
- `Temporal.ZonedDateTime.prototype.eraYear`
- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.PlainDate.prototype.year`

Temporal.ZonedDateTime.prototype.yearOfWeek

The **`yearOfWeek`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. It is calendar-dependent.

The set accessor of `yearOfWeek` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.yearOfWeek`.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.weekOfYear`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.yearOfWeek`

Temporal.ZonedDateTime() constructor

The **`Temporal.ZonedDateTime()`** constructor creates `Temporal.ZonedDateTime` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.ZonedDateTime` objects using the `Temporal.ZonedDateTime.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.ZonedDateTime(epochNanoseconds, timeZone)
new Temporal.ZonedDateTime(epochNanoseconds, timeZone, calendar)
```

> **Note:** `Temporal.ZonedDateTime()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`epochNanoseconds`

A BigInt that corresponds to the `epochNanoseconds` property.

`timeZone`

A string that corresponds to the `timeZoneId` property. Unlike all other `Temporal` APIs, it must be a valid time zone identifier (either named or offset) as-is, and cannot be a `Temporal.ZonedDateTime` instance or a date-time string.

`calendar` Optional

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`.

Return value

A new `Temporal.ZonedDateTime` object, representing the specific instant specified by the parameters.

Exceptions

`TypeError`

Thrown if `timeZone` or `calendar` is not a string.

`RangeError`

Thrown in one of the following cases:

- `epochNanoseconds` is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.
- `timeZone` is not a valid time zone identifier.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.ZonedDateTime()

```
const zdt = new Temporal.ZonedDateTime(0n, "America/New_York");
console.log(zdt.toString()); // '1969-12-31T19:00:00-05:00[America/New_York]'
```

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.from()`

TypedArray

A ***TypedArray*** object describes an array-like view of an underlying binary data buffer. There is no global property named `TypedArray`, nor is there a directly visible `TypedArray` constructor. Instead, there are a number of different global properties, whose values are typed array constructors for specific element types, listed below. On the following pages you will find common properties and methods that can be used with any typed array containing elements of any type.

Description

The `TypedArray` constructor (often referred to as `%TypedArray%` to indicate its "intrinsicness", since it does not correspond to any global exposed to a JavaScript program) serves as the common superclass of all `TypedArray` subclasses. Think about `%TypedArray%` as an "abstract class" providing a common interface of utility methods for all typed array subclasses. This constructor is not directly exposed: there is no global `TypedArray` property. It is only accessible through `Object.getPrototypeOf(Int8Array)` and similar.

When creating an instance of a `TypedArray` subclass (e.g., `Int8Array`), an array buffer is created internally in memory or, if an `ArrayBuffer` object is given as constructor argument, that `ArrayBuffer` is used instead. The buffer address is saved as an internal property of the instance and all the methods of `%TypedArray%.prototype` will set and get values based on that array buffer address.

TypedArray objects

Type	Value Range	Size in bytes	Web IDL type
`Int8Array`	-128 to 127	1	`byte`
`Uint8Array`	0 to 255	1	`octet`
`Uint8ClampedArray`	0 to 255	1	`octet`
`Int16Array`	-32768 to 32767	2	`short`
`Uint16Array`	0 to 65535	2	`unsigned short`
`Int32Array`	-2147483648 to 2147483647	4	`long`
`Uint32Array`	0 to 4294967295	4	`unsigned long`
`Float16Array`	`-65504` to `65504`	2	N/A
`Float32Array`	`-3.4e38` to `3.4e38`	4	`unrestricted float`
`Float64Array`	`-1.8e308` to `1.8e308`	8	`unrestricted double`
`BigInt64Array`	-2^{63} to $2^{63} - 1$	8	`bigint`
`BigUint64Array`	0 to $2^{64} - 1$	8	`bigint`

Value encoding and normalization

All typed arrays operate on `ArrayBuffer`s, where you can observe the exact byte representation of each element, so how the numbers are encoded in binary format is significant.

- Unsigned integer arrays (`Uint8Array`, `Uint16Array`, `Uint32Array`, and `BigUint64Array`) store the number directly in binary.
- Signed integer arrays (`Int8Array`, `Int16Array`, `Int32Array`, and `BigInt64Array`) store the number using two's complement.
- Floating-point arrays (`Float16Array`, `Float32Array`, and `Float64Array`) store the number using IEEE 754 floating-point format. The `Number` reference has more information about the exact format. JavaScript numbers use double precision floating point format by default, which is the same as `Float64Array`. `Float32Array` uses 23 (instead of 52) bits for the mantissa and 8 (instead of 11) bits for the exponent. `Float16Array` uses 10 bits for the mantissa and 5 bits for the exponent. Note that the spec requires all `NaN` values to use the same bit encoding, but the exact bit pattern is implementation-dependent.
- `Uint8ClampedArray` is a special case. It stores the number in binary like `Uint8Array` does, but when you store a number outside the range, it *clamps* the number to the range 0 to 255 by mathematical value, instead of truncating the most significant bits.

All typed arrays except `Int8Array`, `Uint8Array`, and `Uint8ClampedArray` store each element using multiple bytes. These bytes can either be ordered from most significant to least significant (big-endian) or from least significant to most significant (little-endian). See Endianness for more explanation. Typed arrays always use the platform's native byte order. If you want to specify the endianness when writing and reading from buffers, you should use a `DataView` instead.

When writing to these typed arrays, values that are outside the representable range are normalized.

- All integer arrays (except `Uint8ClampedArray`) use fixed-width number conversion, which first truncates the decimal part of the number and then takes the lowest bits.
- `Uint8ClampedArray` first clamps the number to the range 0 to 255 (values greater than 255 become 255 and values less than 0 become 0). It then *rounds* (instead of flooring) the result to the nearest integer, with half-to-even; meaning if the number is exactly between two integers, it rounds to the nearest even integer. For

example, `0.5` becomes `0`, `1.5` becomes `2`, and `2.5` becomes `2`.
- `Float16Array` and `Float32Array` perform a "round to even" to convert 64-bit floating point numbers to 32-bit and 16-bit. This is the same algorithm as provided by `Math.fround()` and `Math.f16round()`.

Behavior when viewing a resizable buffer

When a `TypedArray` is created as a view of a resizable buffer, resizing the underlying buffer will have different effects on the size of the `TypedArray` depending on whether the `TypedArray` is constructed as length-tracking.

If a typed array is created without a specific size by omitting the third parameter or passing `undefined`, the typed array will become *length-tracking*, and will automatically resize to fit the underlying `buffer` as the latter is resized:

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2

buffer.resize(12);

console.log(float32.byteLength); // 12
console.log(float32.length); // 3
```

If a typed array is created with a specific size using the third `length` parameter, it won't resize to contain the `buffer` as the latter is grown:

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer, 0, 2);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2
console.log(float32[0]); // 0, the initial value

buffer.resize(12);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2
console.log(float32[0]); // 0, the initial value
```

When a `buffer` is shrunk, the viewing typed array may become out of bounds, in which case the typed array's observed size will decrease to 0. This is the only case where a non-length-tracking typed array's length may change.

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer, 0, 2);

buffer.resize(7);

console.log(float32.byteLength); // 0
console.log(float32.length); // 0
console.log(float32[0]); // undefined
```

If you then grow the `buffer` again to bring the typed array back in bounds, the typed array's size will be restored to its original value.

```
buffer.resize(8);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2
console.log(float32[0]); // 0 - back in bounds again!
```

The same can happen for length-tracking typed arrays as well, if the buffer is shrunk beyond the `byteOffset`.

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer, 4);
// float32 is length-tracking, but it only extends from the 4th byte
// to the end of the buffer, so if the buffer is resized to be shorter
// than 4 bytes, the typed array will become out of bounds
buffer.resize(3);
console.log(float32.byteLength); // 0
```

Constructor

This object cannot be instantiated directly — attempting to construct it with `new` throws a `TypeError`.

```
new (Object.getPrototypeOf(Int8Array))();
// TypeError: Abstract class TypedArray not directly constructable
```

Instead, you create an instance of a typed array of a particular type, such as an `Int8Array` or a `BigInt64Array`. These objects all have a common syntax for their constructors:

```
new TypedArray()
new TypedArray(length)
new TypedArray(typedArray)
new TypedArray(object)
```

```
new TypedArray(buffer)
new TypedArray(buffer, byteOffset)
new TypedArray(buffer, byteOffset, length)
```

Where `TypedArray` is a constructor for one of the concrete types.

> **Note:** All `TypedArray` subclasses' constructors can only be constructed with `new`. Attempting to call one without `new` throws a `TypeError`.

Parameters

`typedArray`

When called with an instance of a `TypedArray` subclass, the `typedArray` gets copied into a new typed array. For a non-bigint `TypedArray` constructor, the `typedArray` parameter can only be of one of the non-bigint types (such as `Int32Array`). Similarly, for a bigint `TypedArray` constructor (`BigInt64Array` or `BigUint64Array`), the `typedArray` parameter can only be of one of the bigint types. Each value in `typedArray` is converted to the corresponding type of the constructor before being copied into the new array. The length of the new typed array will be same as the length of the `typedArray` argument.

`object`

When called with an object that's not a `TypedArray` instance, a new typed array is created in the same way as the `TypedArray.from()` method.

`length` Optional

When called with a non-object, the parameter will be treated as a number specifying the length of the typed array. An internal array buffer is created in memory, of size `length` multiplied by `BYTES_PER_ELEMENT` bytes, filled with zeros. Omitting all parameters is equivalent to using `0` as `length`.

`buffer`, `byteOffset` Optional, `length` Optional

When called with an `ArrayBuffer` or `SharedArrayBuffer` instance, and optionally a `byteOffset` and a `length` argument, a new typed array view is created that views the specified buffer. The `byteOffset` (in bytes) and `length` (in number of elements, each occupying `BYTES_PER_ELEMENT` bytes) parameters specify the memory range that will be exposed by the typed array view. If both are omitted, all of `buffer` is viewed; if only `length` is omitted, the remainder of `buffer` starting from `byteOffset` is viewed. If `length` is omitted, the typed array becomes length-tracking.

Exceptions

All `TypeArray` subclass constructors operate in the same way. They would all throw the following exceptions:

`TypeError`

Thrown in one of the following cases:

- A `typedArray` is passed but it is a bigint type while the current constructor is not, or vice versa.
- A `typedArray` is passed but the buffer it's viewing is detached, or a detached `buffer` is directly passed.

`RangeError`

Thrown in one of the following cases:

- The new typed array's length is too large.
- The length of `buffer` (if the `length` parameter is not specified) or `byteOffset` is not an integral multiple of the new typed array's element size.
- `byteOffset` is not a valid array index (an integer between 0 and $2^{53} - 1$).
- When creating a view from a buffer, the bounds are outside the buffer. In other words, `byteOffset + length * TypedArray.BYTES_PER_ELEMENT > buffer.byteLength`.

Static properties

These properties are defined on the `TypedArray` constructor object and are thus shared by all `TypedArray` subclass constructors.

`TypedArray[Symbol.species]`

The constructor function used to create derived objects.

All `TypedArray` subclasses also have the following static properties:

`TypedArray.BYTES_PER_ELEMENT`

Returns a number value of the element size for the different `TypedArray` objects.

Static methods

These methods are defined on the `TypedArray` constructor object and are thus shared by all `TypedArray` subclass constructors.

`TypedArray.from()`

Creates a new `TypedArray` from an array-like or iterable object. See also `Array.from()`.

`TypedArray.of()`

Creates a new `TypedArray` with a variable number of arguments. See also `Array.of()`.

Instance properties

These properties are defined on `TypedArray.prototype` and shared by all `TypedArray` subclass instances.

`TypedArray.prototype.buffer`

Returns the `ArrayBuffer` referenced by the typed array.

`TypedArray.prototype.byteLength`

Returns the length (in bytes) of the typed array.

`TypedArray.prototype.byteOffset`

Returns the offset (in bytes) of the typed array from the start of its `ArrayBuffer`.

`TypedArray.prototype.constructor`

The constructor function that created the instance object. `TypedArray.prototype.constructor` is the hidden `TypedArray` constructor function, but each typed array subclass also defines its own `constructor` property.

`TypedArray.prototype.length`

Returns the number of elements held in the typed array.

`TypedArray.prototype[Symbol.toStringTag]`

The initial value of the `TypedArray.prototype[Symbol.toStringTag]` property is a getter that returns the same string as the typed array constructor's name. It returns `undefined` if the `this` value is not one of the typed array subclasses. This property is used in `Object.prototype.toString()`. However, because `TypedArray` also has its own `toString()` method, this property is not used unless you call `Object.prototype.toString.call()` with a typed array as `thisArg`.

All `TypedArray` subclasses also have the following instance properties:

`TypedArray.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size for the different `TypedArray` objects.

Instance methods

These methods are defined on the `TypedArray` prototype object and are thus shared by all `TypedArray` subclass instances.

`TypedArray.prototype.at()`

Takes an integer value and returns the item at that index. This method allows for negative integers, which count back from the last item.

`TypedArray.prototype.copyWithin()`

Copies a sequence of array elements within the array. See also `Array.prototype.copyWithin()`.

`TypedArray.prototype.entries()`

Returns a new *array iterator* object that contains the key/value pairs for each index in the array. See also `Array.prototype.entries()`.

`TypedArray.prototype.every()`

Returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`. See also `Array.prototype.every()`.

`TypedArray.prototype.fill()`

Fills all the elements of an array from a start index to an end index with a static value. See also `Array.prototype.fill()`.

`TypedArray.prototype.filter()`

Creates a new array with all of the elements of this array for which the provided filtering function returns `true`. See also `Array.prototype.filter()`.

`TypedArray.prototype.find()`

Returns the first `element` in the array that satisfies a provided testing function, or `undefined` if no appropriate element is found. See also `Array.prototype.find()`.

`TypedArray.prototype.findIndex()`

Returns the first index value in the array that has an element that satisfies a provided testing function, or `-1` if no appropriate element was found. See also `Array.prototype.findIndex()`.

`TypedArray.prototype.findLast()`

Returns the value of the last element in the array that satisfies a provided testing function, or `undefined` if no appropriate element is found. See also `Array.prototype.findLast()`.

`TypedArray.prototype.findLastIndex()`

Returns the index of the last element in the array that satisfies a provided testing function, or `-1` if no appropriate element was found. See also `Array.prototype.findLastIndex()`.

`TypedArray.prototype.forEach()`

Calls a function for each element in the array. See also `Array.prototype.forEach()`.

`TypedArray.prototype.includes()`

Determines whether a typed array includes a certain element, returning `true` or `false` as appropriate. See also `Array.prototype.includes()`.

`TypedArray.prototype.indexOf()`

Returns the first (least) index of an element within the array equal to the specified value, or `-1` if none is found. See also `Array.prototype.indexOf()`.

`TypedArray.prototype.join()`

Returns a new string that is the concatenation of all elements in this typed array, separated by commas or a specified separator string. See also `Array.prototype.join()`.

`TypedArray.prototype.keys()`

Returns a new array iterator that contains the keys for each index in the array. See also `Array.prototype.keys()`.

`TypedArray.prototype.lastIndexOf()`

Returns the last (greatest) index of an element within the array equal to the specified value, or `-1` if none is found. See also `Array.prototype.lastIndexOf()`.

`TypedArray.prototype.map()`

Creates a new array with the results of calling a provided function on every element in this array. See also `Array.prototype.map()`.

`TypedArray.prototype.reduce()`

Apply a function against an accumulator and each value of the array (from left-to-right) as to reduce it to a single value. See also `Array.prototype.reduce()`.

`TypedArray.prototype.reduceRight()`

Apply a function against an accumulator and each value of the array (from right-to-left) as to reduce it to a single value. See also `Array.prototype.reduceRight()`.

`TypedArray.prototype.reverse()`

Reverses the order of the elements of an array — the first becomes the last, and the last becomes the first. See also `Array.prototype.reverse()`.

`TypedArray.prototype.set()`

Stores multiple values in the typed array, reading input values from a specified array.

`TypedArray.prototype.slice()`

Extracts a section of an array and returns a new array. See also `Array.prototype.slice()`.

`TypedArray.prototype.some()`

Returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`. See also `Array.prototype.some()`.

`TypedArray.prototype.sort()`

Sorts the elements of an array in place and returns the array. See also `Array.prototype.sort()`.

`TypedArray.prototype.subarray()`

Returns a new `TypedArray` from the given start and end element index.

`TypedArray.prototype.toLocaleString()`

Returns a localized string representing the array and its elements. See also `Array.prototype.toLocaleString()`.

`TypedArray.prototype.toReversed()`

Returns a new array with the elements in reversed order, without modifying the original array.

`TypedArray.prototype.toSorted()`

Returns a new array with the elements sorted in ascending order, without modifying the original array.

`TypedArray.prototype.toString()`

Returns a string representing the array and its elements. See also `Array.prototype.toString()`.

`TypedArray.prototype.values()`

Returns a new *array iterator* object that contains the values for each index in the array. See also `Array.prototype.values()`.

`TypedArray.prototype.with()`

Returns a new array with the element at the given index replaced with the given value, without modifying the original array.

`TypedArray.prototype[Symbol.iterator]()`

Returns a new *array iterator* object that contains the values for each index in the array.

Examples

Property access

You can reference elements in the array using standard array index syntax (that is, using bracket notation). However, getting or setting indexed properties on typed arrays will not search in the prototype chain for this property, even when the indices are out of bound. Indexed properties will consult the `ArrayBuffer` and will never look at object properties. You can still use named properties, just like with all objects.

```
// Setting and getting using standard array syntax
const int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42

// Indexed properties on prototypes are not consulted (Fx 25)
Int8Array.prototype[20] = "foo";
new Int8Array(32)[20]; // 0
// even when out of bound
Int8Array.prototype[20] = "foo";
new Int8Array(8)[20]; // undefined
// or with negative integers
Int8Array.prototype[-1] = "foo";
new Int8Array(8)[-1]; // undefined

// Named properties are allowed, though (Fx 30)
Int8Array.prototype.foo = "bar";
new Int8Array(32).foo; // "bar"
```

Cannot be frozen

`TypedArrays` that aren't empty cannot be frozen, as their underlying `ArrayBuffer` could be mutated through another `TypedArray` view of the buffer. This would mean that the object would never genuinely be frozen.

```
const i8 = Int8Array.of(1, 2, 3);
Object.freeze(i8);
// TypeError: Cannot freeze array buffer views with elements
```

ByteOffset must be aligned

When constructing a `TypedArray` as a view onto an `ArrayBuffer`, the `byteOffset` argument must be aligned to its element size; in other words, the offset must be a multiple of `BYTES_PER_ELEMENT`.

```
const i32 = new Int32Array(new ArrayBuffer(4), 1);
// RangeError: start offset of Int32Array should be a multiple of 4
```

```
const i32 = new Int32Array(new ArrayBuffer(4), 0);
```

ByteLength must be aligned

Like the `byteOffset` parameter, the `byteLength` property of an `ArrayBuffer` passed to a `TypedArray`'s constructor must be a multiple of the constructor's `BYTES_PER_ELEMENT`.

```
const i32 = new Int32Array(new ArrayBuffer(3));
// RangeError: byte length of Int32Array should be a multiple of 4
```

```
const i32 = new Int32Array(new ArrayBuffer(4));
```

See also

- Polyfill of typed arrays in `core-js`
- JavaScript typed arrays guide
- `ArrayBuffer`
- `DataView`
- `TextDecoder`

TypedArray.prototype.at()

The **`at()`** method of `TypedArray` instances takes an integer value and returns the item at that index, allowing for positive and negative integers. Negative integers count back from the last item in the typed array. This method has the same algorithm as `Array.prototype.at()`.

Try it

```
const int8 = new Int8Array([0, 10, -10, 20, -30, 40, -50]);

let index = 1;

console.log(`An index of ${index} returns the item ${int8.at(index)}`);
// Expected output: "An index of 1 returns the item 10"

index = -2;

console.log(`An index of ${index} returns the item ${int8.at(index)}`);
// Expected output: "An index of -2 returns the item 40"
```

Syntax

```
at(index)
```

Parameters

`index`

> Zero-based index of the typed array element to be returned, converted to an integer. Negative index counts back from the end of the typed array — if `index < 0`, `index + array.length` is accessed.

Return value

The element in the typed array matching the given index. Always returns `undefined` if `index < -array.length` or `index >= array.length` without attempting to access the corresponding property.

Description

See `Array.prototype.at()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Return the last value of a typed array

The following example provides a function which returns the last element found in a specified array.

```
const uint8 = new Uint8Array([1, 2, 4, 7, 11, 18]);

// A function which returns the last item of a given array
function returnLast(arr) {
  return arr.at(-1);
}

const lastItem = returnLast(uint8);
console.log(lastItem); // 18
```

Comparing methods

Here we compare different ways to select the penultimate (last but one) item of a `TypedArray`. While all below methods are valid, it highlights the succinctness and readability of the `at()` method.

```
// Our typed array with values
const uint8 = new Uint8Array([1, 2, 4, 7, 11, 18]);

// Using length property
const lengthWay = uint8[uint8.length - 2];
console.log(lengthWay); // 11

// Using slice() method. Note an array is returned
const sliceWay = uint8.slice(-2, -1);
console.log(sliceWay[0]); // 11

// Using at() method
const atWay = uint8.at(-2);
console.log(atWay); // 11
```

See also

- Polyfill of `TypedArray.prototype.at` in `core-js`
- es-shims polyfill of `TypedArray.prototype.at`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.with()`
- `Array.prototype.at()`
- `String.prototype.at()`

TypedArray.prototype.buffer

The **`buffer`** accessor property of `TypedArray` instances returns the `ArrayBuffer` or `SharedArrayBuffer` referenced by this typed array at construction time.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint16 = new Uint16Array(buffer);

console.log(uint16.buffer.byteLength);
// Expected output: 8
```

Description

The `buffer` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the *TypedArray* is constructed and cannot be changed. *TypedArray* is one of the TypedArray objects.

Because a typed array is a *view* of a buffer, the underlying buffer may be longer than the typed array itself.

Examples

Using the buffer property

```
const buffer = new ArrayBuffer(8);
const uint16 = new Uint16Array(buffer);
uint16.buffer; // ArrayBuffer { byteLength: 8 }
```

Accessing the underlying buffer from a sliced array view

```
const buffer = new ArrayBuffer(1024);
const arr = new Uint8Array(buffer, 64, 128);
console.log(arr.byteLength); // 128
console.log(arr.buffer.byteLength); // 1024
console.log(arr.buffer === buffer); // true
```

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray.prototype.byteLength

The **`byteLength`** accessor property of `TypedArray` instances returns the length (in bytes) of this typed array.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer, 2);

console.log(uint8.byteLength);
// Expected output: 6
```

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. If the typed array is length-tracking, then its length depends on the length of the underlying buffer, and may change if the buffer is resized. Otherwise, the value is established when the typed array is constructed and cannot be changed. Whether length-tracking or not, the `byteLength` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteLength property

```
const buffer = new ArrayBuffer(8);

const uint8 = new Uint8Array(buffer);
uint8.byteLength; // 8 (matches the byteLength of the buffer)

const uint8newLength = new Uint8Array(buffer, 1, 5);
uint8newLength.byteLength; // 5 (as specified when constructing the Uint8Array)

const uint8offset = new Uint8Array(buffer, 2);
uint8offset.byteLength; // 6 (due to the offset of the constructed Uint8Array)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const uint8lengthTracking = new Uint8Array(buffer2, 4);
uint8lengthTracking.byteLength; // 12 (16 - 4)
buffer2.resize(20);
uint8lengthTracking.byteLength; // 16 (20 - 4)
buffer2.resize(3);
uint8lengthTracking.byteLength; // 0 (viewed range is no longer valid)
```

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray.prototype.byteOffset

The **`byteOffset`** accessor property of `TypedArray` instances returns the offset (in bytes) of this typed array from the start of its `ArrayBuffer` or `SharedArrayBuffer`.

Description

The `byteOffset` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the typed array is constructed and cannot be changed. However, the `byteOffset` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteOffset property

```
const buffer = new ArrayBuffer(8);

const uint8array1 = new Uint8Array(buffer);
uint8array1.byteOffset; // 0 (no offset specified)

const uint8array2 = new Uint8Array(buffer, 3);
uint8array2.byteOffset; // 3 (as specified when constructing Uint8Array)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const uint8lengthTracking = new Uint8Array(buffer2, 4);
uint8lengthTracking.byteOffset; // 4
buffer2.resize(3);
uint8lengthTracking.byteOffset; // 0 (viewed range is no longer valid)
```

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray.BYTES_PER_ELEMENT

The **`TypedArray.BYTES_PER_ELEMENT`** static data property represents the size in bytes of each element in a typed array.

Try it

```
console.log(Float64Array.BYTES_PER_ELEMENT);
// Expected output: 8

console.log(Int8Array.BYTES_PER_ELEMENT);
// Expected output: 1
```

Value

A number whose value depends on the type of `TypedArray`.

Property attributes of `TypedArray.BYTES_PER_ELEMENT`	
Writable	no
Enumerable	no
Configurable	no

Description

`TypedArray` objects differ from each other in the number of bytes per element and in the way the bytes are interpreted. The `BYTES_PER_ELEMENT` constant contains the number of bytes each element in the given `TypedArray` has.

The `BYTES_PER_ELEMENT` property is both an *instance property* and a *static property*. It's available on both `TypedArray` subclass constructors and on instances of those constructors.

As an instance property, `BYTES_PER_ELEMENT` is defined on the constructor's `prototype`.

```
console.log(Object.hasOwn(Int8Array.prototype, "BYTES_PER_ELEMENT")); // true
```

Examples

Using BYTES_PER_ELEMENT

As a static property:

```
Int8Array.BYTES_PER_ELEMENT; // 1
Uint8Array.BYTES_PER_ELEMENT; // 1
Uint8ClampedArray.BYTES_PER_ELEMENT; // 1
Int16Array.BYTES_PER_ELEMENT; // 2
Uint16Array.BYTES_PER_ELEMENT; // 2
Float16Array.BYTES_PER_ELEMENT; // 2
Int32Array.BYTES_PER_ELEMENT; // 4
Uint32Array.BYTES_PER_ELEMENT; // 4
Float32Array.BYTES_PER_ELEMENT; // 4
Float64Array.BYTES_PER_ELEMENT; // 8
BigInt64Array.BYTES_PER_ELEMENT; // 8
BigUint64Array.BYTES_PER_ELEMENT; // 8
```

As an instance property:

```
new Int8Array([]).BYTES_PER_ELEMENT; // 1
new Uint8Array([]).BYTES_PER_ELEMENT; // 1
new Uint8ClampedArray([]).BYTES_PER_ELEMENT; // 1
new Int16Array([]).BYTES_PER_ELEMENT; // 2
```

```
new Uint16Array([]).BYTES_PER_ELEMENT; // 2
new Float16Array([]).BYTES_PER_ELEMENT; // 2
new Int32Array([]).BYTES_PER_ELEMENT; // 4
new Uint32Array([]).BYTES_PER_ELEMENT; // 4
new Float32Array([]).BYTES_PER_ELEMENT; // 4
new Float64Array([]).BYTES_PER_ELEMENT; // 8
new BigInt64Array([]).BYTES_PER_ELEMENT; // 8
new BigUint64Array([]).BYTES_PER_ELEMENT; // 8
```

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray.prototype.copyWithin()

The **`copyWithin()`** method of `TypedArray` instances shallow copies part of this typed array to another location in the same typed array and returns this typed array without modifying its length. This method has the same algorithm as `Array.prototype.copyWithin()`.

Try it

```
const uint8 = new Uint8Array([1, 2, 3, 4, 5, 6, 7, 8]);

// Insert position, start position, end position
uint8.copyWithin(3, 1, 3);

console.log(uint8);
// Expected output: Uint8Array [1, 2, 3, 2, 3, 6, 7, 8]
```

Syntax

```
copyWithin(target, start)
copyWithin(target, start, end)
```

Parameters

`target`

Zero-based index at which to copy the sequence to, converted to an integer. This corresponds to where the element at `start` will be copied to, and all elements between `start` and `end` are copied to succeeding indices.

`start`

Zero-based index at which to start copying elements from, converted to an integer.

`end` Optional

Zero-based index at which to end copying elements from, converted to an integer. `copyWithin()` copies up to but not including `end`.

Return value

The modified typed array.

Description

See `Array.prototype.copyWithin()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using copyWithin()

```
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
uint8.set([1, 2, 3]);
console.log(uint8); // Uint8Array [ 1, 2, 3, 0, 0, 0, 0, 0 ]
uint8.copyWithin(3, 0, 3);
console.log(uint8); // Uint8Array [ 1, 2, 3, 1, 2, 3, 0, 0 ]
```

See also

- Polyfill of `TypedArray.prototype.copyWithin` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `Array.prototype.copyWithin()`

TypedArray.prototype.entries()

The **`entries()`** method of `TypedArray` instances returns a new *array iterator* object that contains the key/value pairs for each index in the typed array. This method has the same algorithm as `Array.prototype.entries()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);
const eArr = uint8.entries();

eArr.next();
eArr.next();

console.log(eArr.next().value);
// Expected output: Array [2, 30]
```

Syntax

```
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

See `Array.prototype.entries()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Iteration using for...of loop

```
const array = new Uint8Array([10, 20, 30, 40, 50]);
const arrayEntries = arr.entries();
for (const element of arrayEntries) {
  console.log(element);
}
```

Alternative iteration

```
const array = new Uint8Array([10, 20, 30, 40, 50]);
const arrayEntries = arr.entries();

console.log(arrayEntries.next().value); // [0, 10]
console.log(arrayEntries.next().value); // [1, 20]
console.log(arrayEntries.next().value); // [2, 30]
console.log(arrayEntries.next().value); // [3, 40]
console.log(arrayEntries.next().value); // [4, 50]
```

See also

- Polyfill of `TypedArray.prototype.entries` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.keys()`
- `TypedArray.prototype.values()`
- `TypedArray.prototype[Symbol.iterator]()`
- `Array.prototype.entries()`
- Iteration protocols

TypedArray.prototype.every()

The **`every()`** method of `TypedArray` instances returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`. This method has the same algorithm as `Array.prototype.every()`.

Try it

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([-10, -20, -30, -40, -50]);

console.log(int8.every(isNegative));
// Expected output: true
```

Syntax

```
every(callbackFn)
every(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `every()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`true` unless `callbackFn` returns a falsy value for a typed array element, in which case `false` is immediately returned.

Description

See `Array.prototype.every()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Testing size of all typed array elements

The following example tests whether all elements in the typed array are 10 or bigger.

```
function isBigEnough(element, index, array) {
  return element >= 10;
}
new Uint8Array([12, 5, 8, 130, 44]).every(isBigEnough); // false
new Uint8Array([12, 54, 18, 130, 44]).every(isBigEnough); // true
```

See also

- Polyfill of `TypedArray.prototype.every` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.forEach()`
- `TypedArray.prototype.some()`
- `TypedArray.prototype.find()`
- `Array.prototype.every()`

TypedArray.prototype.fill()

The **`fill()`** method of `TypedArray` instances changes all elements within a range of indices in a typed array to a static value. It returns the modified typed array. This method has the same algorithm as `Array.prototype.fill()`.

Try it

```
const uint8 = new Uint8Array([0, 0, 0, 0]);
// Value, start position, end position
uint8.fill(4, 1, 3);

console.log(uint8);
// Expected output: Uint8Array [0, 4, 4, 0]
```

Syntax

```
fill(value)
fill(value, start)
fill(value, start, end)
```

Parameters

`value`

Value to fill the typed array with.

`start` Optional

Zero-based index at which to start filling, converted to an integer.

`end` Optional

Zero-based index at which to end filling, converted to an integer. `fill()` fills up to but not including `end`.

Return value

The modified typed array, filled with `value`.

Description

See `Array.prototype.fill()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using fill()

```
new Uint8Array([1, 2, 3]).fill(4); // Uint8Array [4, 4, 4]
new Uint8Array([1, 2, 3]).fill(4, 1); // Uint8Array [1, 4, 4]
new Uint8Array([1, 2, 3]).fill(4, 1, 2); // Uint8Array [1, 4, 3]
new Uint8Array([1, 2, 3]).fill(4, 1, 1); // Uint8Array [1, 2, 3]
new Uint8Array([1, 2, 3]).fill(4, -3, -2); // Uint8Array [4, 2, 3]
```

See also

- Polyfill of `TypedArray.prototype.fill` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `Array.prototype.fill()`

TypedArray.prototype.filter()

The **`filter()`** method of `TypedArray` instances creates a copy of a portion of a given typed array, filtered down to just the elements from the given typed array that pass the test implemented by the provided function. This method has the same algorithm as `Array.prototype.filter()`.

Try it

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([-10, 20, -30, 40, -50]);
const negInt8 = int8.filter(isNegative);

console.log(negInt8);
// Expected output: Int8Array [-10, -30, -50]
```

Syntax

```
filter(callbackFn)
filter(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to keep the element in the resulting typed array, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `filter()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A copy of the given typed array containing just the elements that pass the test. If no elements pass the test, an empty typed array is returned.

Description

See `Array.prototype.filter()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Filtering out all small values

The following example uses `filter()` to create a filtered typed array that has all elements with values less than 10 removed.

```
function isBigEnough(element, index, array) {
  return element >= 10;
}
new Uint8Array([12, 5, 8, 130, 44]).filter(isBigEnough);
// Uint8Array [ 12, 130, 44 ]
```

See also

- Polyfill of `TypedArray.prototype.filter` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.forEach()`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.map()`
- `TypedArray.prototype.some()`
- `TypedArray.prototype.reduce()`
- `Array.prototype.filter()`

TypedArray.prototype.find()

The **`find()`** method of `TypedArray` instances returns the first element in the provided typed array that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned. This method has the same algorithm as `Array.prototype.find()`.

Try it

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([10, 0, -10, 20, -30, 40, -50]);

console.log(int8.find(isNegative));
// Expected output: -10
```

Syntax

```
find(callbackFn)
find(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `find()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The first element in the typed array that satisfies the provided testing function. Otherwise, `undefined` is returned.

Description

See `Array.prototype.find()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the first prime number in a typed array

The following example returns the first element in the typed array that is a prime number, or `undefined` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
```

```
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

const uint8 = new Uint8Array([4, 5, 8, 12]);
console.log(uint8.find(isPrime)); // 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

See also

- Polyfill of `TypedArray.prototype.find` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLast()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.includes()`
- `TypedArray.prototype.filter()`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.some()`
- `Array.prototype.find()`

TypedArray.prototype.findIndex()

The **`findIndex()`** method of `TypedArray` instances returns the index of the first element in a typed array that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned. This method has the same algorithm as `Array.prototype.findIndex()`.

Try it

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([10, -20, 30, -40, 50]);

console.log(int8.findIndex(isNegative));
// Expected output: 1
```

Syntax

```
findIndex(callbackFn)
findIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `findIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the first element in the typed array that passes the test. Otherwise, `-1`.

Description

See `Array.prototype.findIndex()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the index of the first prime number in a typed array

The following example returns the index of the first element in the typed array that is a prime number, or `-1` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

const uint8 = new Uint8Array([4, 6, 8, 12]);
const uint16 = new Uint16Array([4, 6, 7, 12]);

console.log(uint8.findIndex(isPrime)); // -1, not found
console.log(uint16.findIndex(isPrime)); // 2
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

See also

- Polyfill of `TypedArray.prototype.findIndex` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findLast()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.lastIndexOf()`
- `Array.prototype.findIndex()`

TypedArray.prototype.findLast()

The **`findLast()`** method of `TypedArray` instances iterates the typed array in reverse order and returns the value of the first element that satisfies the provided testing function. If no elements satisfy the testing function, `undefined` is returned. This method has the same algorithm as `Array.prototype.findLast()`.

Try it

```
function isNegative(element /*, index, array */) {
  return element < 0;
}

const int8 = new Int8Array([10, 0, -10, 20, -30, 40, 50]);

console.log(int8.findLast(isNegative));
// Expected output: -30
```

Syntax

```
findLast(callbackFn)
findLast(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `findLast()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The last (highest-index) element in the typed array that satisfies the provided testing function; `undefined` if no matching element is found.

Description

See `Array.prototype.findLast()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the last prime number in a typed array

The following example returns the last element in the typed array that is a prime number, or `undefined` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

let uint8 = new Uint8Array([4, 6, 8, 12]);
console.log(uint8.findLast(isPrime)); // undefined (no primes in array)
uint8 = new Uint8Array([4, 5, 7, 8, 9, 11, 12]);
console.log(uint8.findLast(isPrime)); // 11
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

See also

- Polyfill of `TypedArray.prototype.findLast` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.includes()`
- `TypedArray.prototype.filter()`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.some()`
- `Array.prototype.findLast()`

TypedArray.prototype.findLastIndex()

The **`findLastIndex()`** method of `TypedArray` instances iterates the typed array in reverse order and returns the index of the first element that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned. This method has the same algorithm as `Array.prototype.findLastIndex()`.

Try it

```
function isNegative(element /*, index, array */) {
  return element < 0;
}

const int8 = new Int8Array([10, -20, 30, -40, 50]);

console.log(int8.findLastIndex(isNegative));
// Expected output: 3
```

Syntax

```
findLastIndex(callbackFn)
findLastIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `findLastIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the last (highest-index) element in the typed array that passes the test. Otherwise `-1` if no matching element is found.

Description

See `Array.prototype.findLastIndex()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the index of the last prime number in a typed array

The following example returns the index of the last element in the typed array that is a prime number, or `-1` if there is no prime number.

```
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

let uint8 = new Uint8Array([4, 6, 8, 12]);
console.log(uint8.findLastIndex(isPrime));
// -1 (no primes in array)
uint8 = new Uint8Array([4, 5, 7, 8, 9, 11, 12]);
console.log(uint8.findLastIndex(isPrime));
// 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

See also

- Polyfill of `TypedArray.prototype.findLastIndex` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLast()`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.lastIndexOf()`
- `Array.prototype.findLastIndex()`

TypedArray.prototype.forEach()

The **`forEach()`** method of `TypedArray` instances executes a provided function once for each typed array element. This method has the same algorithm as `Array.prototype.forEach()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30]);

uint8.forEach((element) => console.log(element));

// Expected output: 10
// Expected output: 20
// Expected output: 30
```

Syntax

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value is discarded. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `forEach()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

None (`undefined`).

Description

See `Array.prototype.forEach()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Logging the contents of a typed array

The following code logs a line for each element in a typed array:

```
function logArrayElements(element, index, array) {
  console.log(`a[${index}] = ${element}`);
}

new Uint8Array([0, 1, 2, 3]).forEach(logArrayElements);
// Logs:
// a[0] = 0
// a[1] = 1
// a[2] = 2
// a[3] = 3
```

See also

- Polyfill of `TypedArray.prototype.forEach` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.map()`
- `TypedArray.prototype.filter()`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.some()`
- `Array.prototype.forEach()`
- `Map.prototype.forEach()`
- `Set.prototype.forEach()`

TypedArray.from()

The **`TypedArray.from()`** static method creates a new typed array from an array-like or iterable object. This method is nearly the same as `Array.from()`.

Try it

```
const uint16 = Int16Array.from("12345");

console.log(uint16);
// Expected output: Int16Array [1, 2, 3, 4, 5]
```

Syntax

```
TypedArray.from(arrayLike, mapFn)
TypedArray.from(arrayLike, mapFn, thisArg)
```

Where `TypedArray` is one of:

- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`
- `Float16Array`
- `Float32Array`
- `Float64Array`
- `BigInt64Array`
- `BigUint64Array`

Parameters

`arrayLike`

An iterable or array-like object to convert to a typed array.

`mapFn` Optional

A function to call on every element of the typed array. If provided, every value to be added to the array is first passed through this function, and `mapFn`'s return value is added to the typed array instead. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`thisArg` Optional

Value to use as `this` when executing `mapFn`.

Return value

A new `TypedArray` instance.

Description

See `Array.from()` for more details.

There are some subtle distinctions between `Array.from()` and `TypedArray.from()` (note: the `this` value mentioned below is the `this` value that `TypedArray.from()` was called with, not the `thisArg` argument used to invoke `mapFn`):

- If the `this` value of `TypedArray.from()` is not a constructor, `TypedArray.from()` will throw a `TypeError`, while `Array.from()` defaults to creating a new `Array`.
- The object constructed by `this` must be a `TypedArray` instance, while `Array.from()` allows its `this` value to be constructed to any object.
- When the `source` parameter is an iterator, `TypedArray.from()` first collects all the values from the iterator, then creates an instance of `this` using the count, and finally sets the values on the instance. `Array.from()` sets each value as it receives them from the iterator, then sets its `length` at the end.
- `TypedArray.from()` uses `[[Set]]` while `Array.from()` uses `[[DefineOwnProperty]]`. Hence, when working with `Proxy` objects, it calls `handler.set()` to create new elements rather than `handler.defineProperty()`.
- When `Array.from()` gets an array-like which isn't an iterator, it respects holes. `TypedArray.from()` will ensure the result is dense.

Examples

From an iterable object (Set)

```
const s = new Set([1, 2, 3]);
Uint8Array.from(s);
// Uint8Array [ 1, 2, 3 ]
```

From a string

```
Int16Array.from("123");
// Int16Array [ 1, 2, 3 ]
```

Use with arrow function and map

Using an arrow function as the map function to manipulate the elements

```
Float32Array.from([1, 2, 3], (x) => x + x);
// Float32Array [ 2, 4, 6 ]
```

Generate a sequence of numbers

```
Uint8Array.from({ length: 5 }, (v, k) => k);
// Uint8Array [ 0, 1, 2, 3, 4 ]
```

Calling from() on non-TypedArray constructors

The `this` value of `from()` must be a constructor that returns a `TypedArray` instance.

```
function NotArray(len) {
  console.log("NotArray called with length", len);
}

Int8Array.from.call({}, []); // TypeError: #<Object> is not a constructor
Int8Array.from.call(NotArray, []);
// NotArray called with length 0
// TypeError: Method %TypedArray%.from called on incompatible receiver #<NotArray>
```

```
function NotArray2(len) {
  console.log("NotArray2 called with length", len);
  return new Uint8Array(len);
}
console.log(Int8Array.from.call(NotArray2, [1, 2, 3]));
// NotArray2 called with length 3
// Uint8Array(3) [ 1, 2, 3 ]
```

See also

- Polyfill of `TypedArray.from` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.of()`
- `TypedArray.prototype.map()`
- `Array.from()`

TypedArray.prototype.includes()

The **`includes()`** method of `TypedArray` instances determines whether a typed array includes a certain value among its entries, returning `true` or `false` as appropriate. This method has the same algorithm as `Array.prototype.includes()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.includes(20));
// Expected output: true

// Check from position 3
console.log(uint8.includes(20, 3));
// Expected output: false
```

Syntax

```
includes(searchElement)
includes(searchElement, fromIndex)
```

Parameters

`searchElement`

The value to search for.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

Return value

A boolean value which is `true` if the value `searchElement` is found within the typed array (or the part of the typed array indicated by the index `fromIndex`, if specified).

Description

See `Array.prototype.includes()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using includes()

```
const uint8 = new Uint8Array([1, 2, 3]);
uint8.includes(2); // true
uint8.includes(4); // false
uint8.includes(3, 3); // false

// NaN handling (only relevant for floating point arrays)
new Uint8Array([NaN]).includes(NaN); // false, since the NaN passed to the constructor gets converted to 0
new Float32Array([NaN]).includes(NaN); // true
```

See also

- Polyfill of `TypedArray.prototype.includes` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findIndex()`
- `Array.prototype.includes()`
- `String.prototype.includes()`

TypedArray.prototype.indexOf()

The **`indexOf()`** method of `TypedArray` instances returns the first index at which a given element can be found in the typed array, or -1 if it is not present. This method has the same algorithm as `Array.prototype.indexOf()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.indexOf(50));
// Expected output: 4

// From position 3
console.log(uint8.indexOf(20, 3));
// Expected output: -1

console.log(uint8.indexOf(51));
// Expected output: -1
```

Syntax

```
indexOf(searchElement)
indexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the typed array.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

Return value

The first index of `searchElement` in the typed array; `-1` if not found.

Description

See `Array.prototype.indexOf()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using indexOf()

```
const uint8 = new Uint8Array([2, 5, 9]);
uint8.indexOf(2); // 0
uint8.indexOf(7); // -1
uint8.indexOf(9, 2); // 2
uint8.indexOf(2, -1); // -1
uint8.indexOf(2, -3); // 0
```

See also

- Polyfill of `TypedArray.prototype.indexOf` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.lastIndexOf()`
- `Array.prototype.indexOf()`
- `String.prototype.indexOf()`

TypedArray.prototype.join()

The **`join()`** method of `TypedArray` instances returns a new string that is the concatenation of all elements in this typed array, separated by commas or a specified separator string. If the typed array has only one item, that item's stringification is returned without using the separator. This method has the same algorithm as `Array.prototype.join()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.join());
// Expected output: "10,20,30,40,50"

console.log(uint8.join(""));
// Expected output: "1020304050"

console.log(uint8.join("-"));
// Expected output: "10-20-30-40-50"
```

Syntax

```
join()
join(separator)
```

Parameters

`separator` Optional

A string to separate each pair of adjacent elements of the typed array. If omitted, the typed array elements are separated with a comma (",").

Return value

A string with all typed array elements joined. If `array.length` is `0`, the empty string is returned.

Description

See `Array.prototype.join()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using join()

```
const uint8 = new Uint8Array([1, 2, 3]);
uint8.join(); // '1,2,3'
uint8.join(" / "); // '1 / 2 / 3'
uint8.join(""); // '123'
```

See also

- Polyfill of `TypedArray.prototype.join` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.toString()`
- `Array.prototype.join()`
- `String.prototype.split()`

TypedArray.prototype.keys()

The **`keys()`** method of `TypedArray` instances returns a new *array iterator* object that contains the keys for each index in the typed array. This method has the same algorithm as `Array.prototype.keys()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);
const keys = uint8.keys();

keys.next();
keys.next();

console.log(keys.next().value);
// Expected output: 2
```

Syntax

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

See `Array.prototype.keys()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Iteration using for...of loop

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const arrKeys = arr.keys();
for (const n of arrKeys) {
  console.log(n);
}
```

Alternative iteration

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const arrKeys = arr.keys();
console.log(arrKeys.next().value); // 0
console.log(arrKeys.next().value); // 1
console.log(arrKeys.next().value); // 2
console.log(arrKeys.next().value); // 3
console.log(arrKeys.next().value); // 4
```

See also

- Polyfill of `TypedArray.prototype.keys` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.entries()`
- `TypedArray.prototype.values()`
- `TypedArray.prototype[Symbol.iterator]()`
- `Array.prototype.keys()`
- Iteration protocols

TypedArray.prototype.lastIndexOf()

The **`lastIndexOf()`** method of `TypedArray` instances returns the last index at which a given element can be found in the typed array, or -1 if it is not present. The typed array is searched backwards, starting at `fromIndex`. This method has the same algorithm as `Array.prototype.lastIndexOf()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 50, 50, 50, 60]);

console.log(uint8.lastIndexOf(50, 5));
// Expected output: 4

console.log(uint8.lastIndexOf(50, 3));
// Expected output: 3
```

Syntax

```
lastIndexOf(searchElement)
lastIndexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the typed array.

`fromIndex` Optional

Zero-based index at which to start searching backwards, converted to an integer.

Return value

The last index of `searchElement` in the typed array; `-1` if not found.

Description

See `Array.prototype.lastIndexOf()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using lastIndexOf()

```
const uint8 = new Uint8Array([2, 5, 9, 2]);
uint8.lastIndexOf(2); // 3
uint8.lastIndexOf(7); // -1
uint8.lastIndexOf(2, 3); // 3
uint8.lastIndexOf(2, 2); // 0
uint8.lastIndexOf(2, -2); // 0
uint8.lastIndexOf(2, -1); // 3
```

See also

- Polyfill of `TypedArray.prototype.lastIndexOf` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`
- `String.prototype.lastIndexOf()`

TypedArray.prototype.length

The **`length`** accessor property of `TypedArray` instances returns the length (in elements) of this typed array.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer, 2);

console.log(uint8.length);
// Expected output: 6
```

Description

The `length` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. If the typed array is length-tracking, then its length depends on the length of the underlying buffer, and may change if the buffer is resized. Otherwise, the value is established when the typed array is constructed and cannot be changed. Whether length-tracking or not, the `length` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the length property

```
const buffer = new ArrayBuffer(8);

const uint8 = new Uint8Array(buffer);
uint8.length; // 8 (matches the length of the buffer)

const uint8newLength = new Uint8Array(buffer, 1, 5);
uint8newLength.length; // 5 (as specified when constructing the Uint8Array)

const uint8offset = new Uint8Array(buffer, 2);
uint8offset.length; // 6 (due to the offset of the constructed Uint8Array)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const uint8lengthTracking = new Uint8Array(buffer2, 4);
uint8lengthTracking.length; // 12 (16 - 4)
buffer2.resize(20);
uint8lengthTracking.length; // 16 (20 - 4)
buffer2.resize(3);
uint8lengthTracking.length; // 0 (viewed range is no longer valid)
```

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray.prototype.map()

The **`map()`** method of `TypedArray` instances creates a new typed array populated with the results of calling a provided function on every element in the calling typed array. This method has the same algorithm as `Array.prototype.map()`.

Try it

```
const uint8 = new Uint8Array([25, 36, 49]);
const roots = uint8.map(Math.sqrt);

console.log(roots);
// Expected output: Uint8Array [5, 6, 7]
```

Syntax

```
map(callbackFn)
map(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value is added as a single element in the new typed array. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `map()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A new typed array with each element being the result of the callback function.

Description

See `Array.prototype.map()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Mapping a typed array to a typed array of square roots

The following code takes a typed array and creates a new typed array containing the square roots of the numbers in the first typed array.

```
const numbers = new Uint8Array([1, 4, 9]);
const roots = numbers.map(Math.sqrt);
// roots is now: Uint8Array [1, 2, 3],
// numbers is still Uint8Array [1, 4, 9]
```

Mapping a typed array of numbers using a function containing an argument

The following code shows how `map()` works when a function requiring one argument is used with it. The argument will automatically be assigned to each element of the typed array as `map()` loops through the original typed array.

```
const numbers = new Uint8Array([1, 4, 9]);
const doubles = numbers.map((num) => num * 2);
// doubles is now Uint8Array [2, 8, 18]
// numbers is still Uint8Array [1, 4, 9]
```

See also

- Polyfill of `TypedArray.prototype.map` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.forEach()`
- `TypedArray.from()`
- `Array.prototype.map()`
- `Map`

TypedArray.of()

The **`TypedArray.of()`** static method creates a new typed array from a variable number of arguments. This method is nearly the same as `Array.of()`.

Try it

```
const int16array = Int16Array.of("10", "20", "30", "40", "50");

console.log(int16array);
// Expected output: Int16Array [10, 20, 30, 40, 50]
```

Syntax

```
TypedArray.of()
TypedArray.of(element1)
TypedArray.of(element1, element2)
TypedArray.of(element1, element2, /* …, */ elementN)
```

Where `TypedArray` is one of:

- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`

- `Float16Array`
- `Float32Array`
- `Float64Array`
- `BigInt64Array`
- `BigUint64Array`

Parameters

`element1, ..., elementN`

Elements used to create the typed array.

Return value

A new `TypedArray` instance.

Description

See `Array.of()` for more details. There are some subtle distinctions between `Array.of()` and `TypedArray.of()`:

- If the `this` value passed to `TypedArray.of()` is not a constructor, `TypedArray.of()` will throw a `TypeError`, while `Array.of()` defaults to creating a new `Array`.
- `TypedArray.of()` uses `[[Set]]` while `Array.of()` uses `[[DefineOwnProperty]]`. Hence, when working with `Proxy` objects, it calls `handler.set()` to create new elements rather than `handler.defineProperty()`.

Examples

Using of()

```
Uint8Array.of(1); // Uint8Array [ 1 ]
Int8Array.of("1", "2", "3"); // Int8Array [ 1, 2, 3 ]
Float32Array.of(1, 2, 3); // Float32Array [ 1, 2, 3 ]
Int16Array.of(undefined); // Int16Array [ 0 ]
```

See also

- Polyfill of `TypedArray.of` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.from()`
- `Array.of()`

TypedArray.prototype.reduce()

The **`reduce()`** method of `TypedArray` instances executes a user-supplied "reducer" callback function on each element of the typed array, in order, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements of the typed array is a single value. This method has the same algorithm as `Array.prototype.reduce()`.

Try it

```
const uint8 = new Uint8Array([0, 1, 2, 3]);

function sum(accumulator, currentValue) {
  return accumulator + currentValue;
}

console.log(uint8.reduce(sum));
// Expected output: 6
```

Syntax

```
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduce()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is `array[0]`.

`currentValue`

The value of the current element. On the first call, its value is `array[0]` if `initialValue` is specified; otherwise its value is `array[1]`.

`currentIndex`

The index position of `currentValue` in the typed array. On the first call, its value is `0` if `initialValue` is specified, otherwise `1`.

`array`

The typed array `reduce()` was called upon.

`initialValue` Optional

A value to which `accumulator` is initialized the first time the callback is called. If `initialValue` is specified, `callbackFn` starts executing with the first value in the typed array as `currentValue`. If `initialValue` is *not* specified, `accumulator` is initialized to the first value in the typed array, and `callbackFn` starts executing with the second value in the typed array as `currentValue`. In this case, if the typed array is empty (so that there's no first value to return as `accumulator`), an error is thrown.

Return value

The value that results from running the "reducer" callback function to completion over the entire typed array.

Exceptions

`TypeError`

Thrown if the typed array contains no elements and `initialValue` is not provided.

Description

See `Array.prototype.reduce()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Sum up all values within an array

```
const total = new Uint8Array([0, 1, 2, 3]).reduce((a, b) => a + b);
// total === 6
```

See also

- Polyfill of `TypedArray.prototype.reduce` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.map()`
- `TypedArray.prototype.reduceRight()`
- `Array.prototype.reduce()`
- `Object.groupBy()`
- `Map.groupBy()`

TypedArray.prototype.reduceRight()

The **`reduceRight()`** method of `TypedArray` instances applies a function against an accumulator and each value of the typed array (from right-to-left) to reduce it to a single value. This method has the same algorithm as `Array.prototype.reduceRight()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30]);

const result = uint8.reduceRight(
  (accumulator, currentValue) => `${accumulator}, ${currentValue}`,
);

console.log(result);
// Expected output: "30, 20, 10"
```

Syntax

```
reduceRight(callbackFn)
reduceRight(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduceRight()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is the last element of the typed array.

`currentValue`

The value of the current element. On the first call, its value is the last element if `initialValue` is specified; otherwise its value is the second-to-last element.

`currentIndex`

The index position of `currentValue` in the typed array. On the first call, its value is `array.length - 1` if `initialValue` is specified, otherwise `array.length - 2`.

`array`

The typed array `reduceRight()` was called upon.

`initialValue` Optional

Value to use as accumulator to the first call of the `callbackFn`. If no initial value is supplied, the last element in the typed array will be used and skipped. Calling `reduceRight()` on an empty typed array without an initial value creates a `TypeError`.

Return value

The value that results from the reduction.

Description

See `Array.prototype.reduceRight()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Sum up all values within an array

```
const total = new Uint8Array([0, 1, 2, 3]).reduceRight((a, b) => a + b);
// total === 6
```

See also

- Polyfill of `TypedArray.prototype.reduceRight` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.map()`
- `TypedArray.prototype.reduce()`
- `Array.prototype.reduceRight()`
- `Object.groupBy()`
- `Map.groupBy()`

TypedArray.prototype.reverse()

The **`reverse()`** method of `TypedArray` instances reverses a typed array *in place* and returns the reference to the same typed array, the first typed array element now becoming the last, and the last typed array element becoming the first. In other words, elements order in the typed array will be turned towards the direction opposite to that previously stated. This method has the same algorithm as `Array.prototype.reverse()`.

Try it

```
const uint8 = new Uint8Array([1, 2, 3]);
uint8.reverse();

console.log(uint8);
// Expected output: Uint8Array [3, 2, 1]
```

Syntax

```
reverse()
```

Parameters

None.

Return value

The reference to the original typed array, now reversed. Note that the typed array is reversed *in place*, and no copy is made.

Description

See `Array.prototype.reverse()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using reverse()

```
const uint8 = new Uint8Array([1, 2, 3]);
uint8.reverse();

console.log(uint8); // Uint8Array [3, 2, 1]
```

See also

- Polyfill of `TypedArray.prototype.reverse` in `core-js`
- JavaScript typed arrays guide

- `TypedArray`
- `TypedArray.prototype.join()`
- `TypedArray.prototype.sort()`
- `TypedArray.prototype.toReversed()`
- `Array.prototype.reverse()`

TypedArray.prototype.set()

The **`set()`** method of `TypedArray` instances stores multiple values in the typed array, reading input values from a specified array.

Try it

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);

// Copy the values into the array starting at index 3
uint8.set([1, 2, 3], 3);

console.log(uint8);
// Expected output: Uint8Array [0, 0, 0, 1, 2, 3, 0, 0]
```

Syntax

```
set(array)
set(array, targetOffset)

set(typedarray)
set(typedarray, targetOffset)
```

Parameters

`array`

> The array from which to copy values. All values from the source array are copied into the target array, unless the length of the source array plus the target offset exceeds the length of the target array, in which case an exception is thrown.

`typedarray`

> If the source array is a typed array, the two arrays may share the same underlying `ArrayBuffer`; the JavaScript engine will intelligently **copy** the source range of the buffer to the destination range.

`targetOffset` Optional

> The offset into the target array at which to begin writing values from the source array. If this value is omitted, 0 is assumed (that is, the source array will overwrite values in the target array starting at index 0).

Return value

None (`undefined`).

Exceptions

`RangeError`

> Thrown in one of the following cases:
>
> - An element will be stored beyond the end of the typed array, either because `targetOffset` is too large or because `array` or `typedarray` is too large.
> - `targetOffset` is negative.

Examples

Using set()

```
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);

uint8.set([1, 2, 3], 3);

console.log(uint8); // Uint8Array [ 0, 0, 0, 1, 2, 3, 0, 0 ]
```

See also

- Polyfill of `TypedArray.prototype.set` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`

TypedArray.prototype.slice()

The **`slice()`** method of `TypedArray` instances returns a copy of a portion of a typed array into a new typed array object selected from `start` to `end` (`end` not included) where `start` and `end` represent the index of items in that typed array. The original typed array will not be modified. This method has the same algorithm as

`Array.prototype.slice()`.

Try it

```
const bytes = new Uint8Array([10, 20, 30, 40, 50]);
const byteSlice = bytes.slice(1, 3);

console.log(byteSlice);
// Expected output: Uint8Array [20, 30]
```

Syntax

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

Return value

A new typed array containing the extracted elements.

Description

See `Array.prototype.slice()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Return a portion of an existing typed array

```
const bytes = new Uint8Array([1, 2, 3]);
bytes.slice(1); // Uint8Array [ 2, 3 ]
bytes.slice(2); // Uint8Array [ 3 ]
bytes.slice(-2); // Uint8Array [ 2, 3 ]
bytes.slice(0, 1); // Uint8Array [ 1 ]
```

See also

- Polyfill of `TypedArray.prototype.slice` in `core-js`
- es-shims polyfill of `TypedArray.prototype.slice`
- JavaScript typed arrays guide
- `TypedArray`
- `Array.prototype.slice()`
- `String.prototype.slice()`

TypedArray.prototype.some()

The **`some()`** method of `TypedArray` instances returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`. This method has the same algorithm as `Array.prototype.some()`.

Try it

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([-10, 20, -30, 40, -50]);
const positives = new Int8Array([10, 20, 30, 40, 50]);

console.log(int8.some(isNegative));
// Expected output: true

console.log(positives.some(isNegative));
// Expected output: false
```

Syntax

```
some(callbackFn)
some(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `some()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`false` unless `callbackFn` returns a truthy value for a typed array element, in which case `true` is immediately returned.

Description

See `Array.prototype.some()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Testing size of all typed array elements

The following example tests whether any element in the typed array is bigger than 10.

```
function isBiggerThan10(element, index, array) {
  return element > 10;
}
new Uint8Array([2, 5, 8, 1, 4]).some(isBiggerThan10); // false
new Uint8Array([12, 5, 8, 1, 4]).some(isBiggerThan10); // true
```

See also

- Polyfill of `TypedArray.prototype.some` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.forEach()`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.includes()`
- `Array.prototype.some()`

TypedArray.prototype.sort()

The **`sort()`** method of `TypedArray` instances sorts the elements of a typed array *in place* and returns the reference to the same typed array, now sorted. This method has the same algorithm as `Array.prototype.sort()`, except that it sorts the values numerically instead of as strings by default.

Try it

```
const uint8 = new Uint8Array([40, 10, 50, 20, 30]);
uint8.sort();

console.log(uint8);
// Expected output: Uint8Array [10, 20, 30, 40, 50]
```

Syntax

```
sort()
sort(compareFn)
```

Parameters

`compareFn` Optional

A function that determines the order of the elements. The function is called with the following arguments:

`a`

The first element for comparison.

`b`

The second element for comparison.

It should return a number where:

- A negative value indicates that `a` should come before `b`.
- A positive value indicates that `a` should come after `b`.
- Zero or `NaN` indicates that `a` and `b` are considered equal.

To memorize this, remember that `(a, b) => a - b` sorts numbers in ascending order.

If omitted, the typed array elements are sorted according to numeric value.

Return value

The reference to the original typed array, now sorted. Note that the typed array is sorted *in place*, and no copy is made.

Description

See `Array.prototype.sort()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using sort()

For more examples, see also the `Array.prototype.sort()` method.

```
let numbers = new Uint8Array([40, 1, 5, 200]);
numbers.sort();
// Uint8Array [ 1, 5, 40, 200 ]
// Unlike plain Arrays, a compare function is not required
// to sort the numbers numerically.

// Regular Arrays require a compare function to sort numerically:
numbers = [40, 1, 5, 200];
numbers.sort();
// [1, 200, 40, 5]

numbers.sort((a, b) => a - b); // compare numbers
// [ 1, 5, 40, 200 ]
```

See also

- Polyfill of `TypedArray.prototype.sort` with modern behavior like stable sort in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.reverse()`
- `TypedArray.prototype.toSorted()`
- `Array.prototype.sort()`

TypedArray.prototype.subarray()

The **`subarray()`** method of `TypedArray` instances returns a new typed array on the same `ArrayBuffer` store and with the same element types as for this typed array. The begin offset is **inclusive** and the end offset is **exclusive**.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.subarray(1, 3));
// Expected output: Uint8Array [20, 30]

console.log(uint8.subarray(1));
// Expected output: Uint8Array [20, 30, 40, 50]
```

Syntax

```
subarray()
subarray(begin)
subarray(begin, end)
```

Parameters

`begin` Optional

Element to begin at. The offset is inclusive. The whole array will be included in the new view if this value is not specified.

`end` Optional

Element to end at. The offset is exclusive. If not specified, all elements from the one specified by `begin` to the end of the array are included in the new view.

Return value

A new `TypedArray` object.

Description

The range specified by `begin` and `end` is clamped to the valid index range for the current array; if the computed length of the new array would be negative, it's clamped to zero. If either `begin` or `end` is negative, it refers to an index from the end of the array instead of from the beginning.

Also note that this is creating a new view on the existing buffer; changes to the new object's contents will impact the original object and vice versa.

Examples

Using the subarray() method

```
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
uint8.set([1, 2, 3]);

console.log(uint8); // Uint8Array [ 1, 2, 3, 0, 0, 0, 0, 0 ]

const sub = uint8.subarray(0, 4);

console.log(sub); // Uint8Array [ 1, 2, 3, 0 ]
```

See also

- Polyfill of `TypedArray.prototype.subarray` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`

TypedArray.prototype[Symbol.iterator]()

The **`[Symbol.iterator]()`** method of `TypedArray` instances implements the iterable protocol and allows typed arrays to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns an array iterator object that yields the value of each index in the typed array.

The initial value of this property is the same function object as the initial value of the `TypedArray.prototype.values` property.

Try it

```
const uint8 = new Uint8Array([10, 20, 30]);
const iterator = uint8[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
}

// Expected output: 10
// Expected output: 20
// Expected output: 30
```

Syntax

```
typedArray[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `TypedArray.prototype.values()`: a new iterable iterator object that yields the value of each index in the typed array.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes typed arrays iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
for (const n of arr) {
  console.log(n);
}
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const arrIter = arr[Symbol.iterator]();
console.log(arrIter.next().value); // 10
console.log(arrIter.next().value); // 20
console.log(arrIter.next().value); // 30
console.log(arrIter.next().value); // 40
console.log(arrIter.next().value); // 50
```

See also

- Polyfill of `TypedArray.prototype[Symbol.iterator]` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.entries()`
- `TypedArray.prototype.keys()`
- `TypedArray.prototype.values()`
- `Symbol.iterator`
- Iteration protocols

TypedArray[Symbol.species]

The **`TypedArray[Symbol.species]`** static accessor property returns the constructor used to construct return values from typed array methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
TypedArray[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from typed array methods that create new typed arrays.

Description

The `[Symbol.species]` accessor property returns the default constructor for typed array objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class TypedArray {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubTypedArray extends Int8Array {}
SubTypedArray[Symbol.species] === SubTypedArray; // true
```

When calling typed array methods that do not mutate the existing array but return a new array instance (for example, `filter()` and `map()`), the array's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the typed array method.

However, unlike `Array[Symbol.species]`, when using `[Symbol.species]` to create new typed arrays, the language will make sure that the newly created array is a proper typed array and has the same content type as the original array — for example, you can't create a `BigInt64Array` from a `Float64Array`, or create a non-BigInt array from a BigInt array. Doing so throws a `TypeError`.

```
class BadArray extends Int8Array {
  static get [Symbol.species]() {
    return Array;
  }
}
new BadArray(1).map(() => 0); // TypeError: Method %TypedArray%.prototype.map called on incompatible receiver [object Array]

class BadArray2 extends Int8Array {
  static get [Symbol.species]() {
    return BigInt64Array;
  }
}
new BadArray2(1).map(() => 0n); // TypeError: TypedArray.prototype.map constructed typed array of different content type from |this|
```

> **Note:** Due to a bug in both SpiderMonkey and V8, the content type match is not checked. Only Safari will throw a `TypeError` in the second example.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is one of the typed array constructors itself for any given typed array constructor.

```
Int8Array[Symbol.species]; // function Int8Array()
Uint8Array[Symbol.species]; // function Uint8Array()
Float32Array[Symbol.species]; // function Float32Array()
```

Species in derived objects

In an instance of a custom `TypedArray` subclass, such as `MyTypedArray`, the `MyTypedArray` species is the `MyTypedArray` constructor. However, you might want to overwrite this, in order to return a parent typed array object in your derived class methods:

```
class MyTypedArray extends Uint8Array {
  // Overwrite MyTypedArray species to the parent Uint8Array constructor
  static get [Symbol.species]() {
    return Uint8Array;
  }
}
```

See also

- `TypedArray`
- `Symbol.species`

TypedArray.prototype.toLocaleString()

The **`toLocaleString()`** method of `TypedArray` instances returns a string representing the elements of the typed array. The elements are converted to strings using their `toLocaleString` methods and these strings are separated by a locale-specific string (such as a comma ","). This method has the same algorithm as `Array.prototype.toLocaleString()`.

Try it

```
const uint8 = new Uint32Array([500, 8123, 12]);

console.log(uint8.toLocaleString());
// Expected output: "500,8123,12"

console.log(uint8.toLocaleString("en-GB"));
// Expected output: "500,8,123,12"

console.log(
  uint8.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// Expected output: "500,00 €,8.123,00 €,12,00 €"
```

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

`locales` Optional

> A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

> An object with configuration properties. See `Number.prototype.toLocaleString()`.

Return value

A string representing the elements of the typed array.

Description

See `Array.prototype.toLocaleString()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using toLocaleString()

```
const uint = new Uint32Array([2000, 500, 8123, 12, 4212]);

uint.toLocaleString();
// if run in a de-DE locale
// "2.000,500,8.123,12,4.212"

uint.toLocaleString("en-US");
// "2,000,500,8,123,12,4,212"

uint.toLocaleString("ja-JP", { style: "currency", currency: "JPY" });
// "￥2,000,￥500,￥8,123,￥12,￥4,212"
```

See also

- JavaScript typed arrays guide
- `TypedArray`

- `TypedArray.prototype.toString()`
- `Array.prototype.toLocaleString()`
- `Intl`
- `Intl.ListFormat`
- `Number.prototype.toLocaleString()`

TypedArray.prototype.toReversed()

The **`toReversed()`** method of `TypedArray` instances is the copying counterpart of the `reverse()` method. It returns a new typed array with the elements in reversed order. This method has the same algorithm as `Array.prototype.toReversed()`.

Syntax

```
toReversed()
```

Parameters

None.

Return value

A new typed array containing the elements in reversed order.

Description

See `Array.prototype.toReversed()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using toReversed()

```
const uint8 = new Uint8Array([1, 2, 3]);
const reversedUint8 = uint8.toReversed();
console.log(reversedUint8); // Uint8Array [3, 2, 1]
console.log(uint8); // Uint8Array [1, 2, 3]
```

See also

- Polyfill of `TypedArray.prototype.toReversed` in `core-js`
- JavaScript typed arrays guide
- `TypedArray.prototype.reverse()`
- `TypedArray.prototype.toSorted()`
- `TypedArray.prototype.with()`
- `Array.prototype.toReversed()`

TypedArray.prototype.toSorted()

The **`toSorted()`** method of `TypedArray` instances is the copying version of the `sort()` method. It returns a new typed array with the elements sorted in ascending order. This method has the same algorithm as `Array.prototype.toSorted()`, except that it sorts the values numerically instead of as strings by default.

Syntax

```
toSorted()
toSorted(compareFn)
```

Parameters

`compareFn` Optional

> A function that determines the order of the elements. If omitted, the typed array elements are sorted according to numeric value. See `sort()` for more information.

Return value

A new typed array with the elements sorted in ascending order.

Description

See `Array.prototype.toSorted()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Sorting an array

For more examples, see also the `Array.prototype.sort()` method.

```
const numbers = new Uint8Array([40, 1, 5, 200]);
const numberSorted = numbers.toSorted();
console.log(numberSorted); // Uint8Array [ 1, 5, 40, 200 ]
// Unlike plain Arrays, a compare function is not required
// to sort the numbers numerically.
console.log(numbers); // Uint8Array [ 40, 1, 5, 200 ]
```

See also

- Polyfill of `TypedArray.prototype.toSorted` in `core-js`
- JavaScript typed arrays guide
- `TypedArray.prototype.sort()`
- `TypedArray.prototype.toReversed()`
- `TypedArray.prototype.with()`
- `Array.prototype.toSorted()`

TypedArray.prototype.toString()

The **`toString()`** method of `TypedArray` instances returns a string representing the specified typed array and its elements. This method has the same algorithm as `Array.prototype.toString()`.

Try it

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

const uint8String = uint8.toString();

console.log(uint8String.startsWith("10"));
// Expected output: true
```

Syntax

```
toString()
```

Parameters

None.

Return value

A string representing the elements of the typed array.

Description

See `Array.prototype.toString()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Converting a typed array to a string

```
const uint8 = new Uint8Array([1, 2, 3]);
// Explicit conversion
console.log(uint8.toString()); // 1,2,3
// Implicit conversion
console.log(`${uint8}`); // 1,2,3
```

See also

- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.join()`
- `TypedArray.prototype.toLocaleString()`
- `Array.prototype.toString()`
- `String.prototype.toString()`

TypedArray.prototype.values()

The **`values()`** method of `TypedArray` instances returns a new *array iterator* object that iterates the value of each item in the typed array. This method has the same algorithm as `Array.prototype.values()`.

Try it

```
const bytes = new Uint8Array([10, 20, 30, 40, 50]);
const iterator = bytes.values();

iterator.next();
iterator.next();

console.log(iterator.next().value);
// Expected output: 30
```

Syntax

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

See `Array.prototype.values()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Iteration using for...of loop

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const values = arr.values();
for (const n of values) {
  console.log(n);
}
```

Alternative iteration

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const values = arr.values();
console.log(values.next().value); // 10
console.log(values.next().value); // 20
console.log(values.next().value); // 30
console.log(values.next().value); // 40
console.log(values.next().value); // 50
```

See also

- Polyfill of `TypedArray.prototype.values` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.entries()`
- `TypedArray.prototype.keys()`
- `TypedArray.prototype[Symbol.iterator]()`
- `Array.prototype.values()`
- Iteration protocols

TypedArray.prototype.with()

The **`with()`** method of `TypedArray` instances is the copying version of using the bracket notation to change the value of a given index. It returns a new typed array with the element at the given index replaced with the given value. This method has the same algorithm as `Array.prototype.with()`.

Syntax

```
arrayInstance.with(index, value)
```

Parameters

`index`

Zero-based index at which to change the typed array, converted to an integer.

`value`

Any value to be assigned to the given index.

Return value

A new typed array with the element at `index` replaced with `value`.

Exceptions

`RangeError`

Thrown if `index >= array.length` or `index < -array.length`.

Description

See `Array.prototype.with()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using with()

```
const arr = new Uint8Array([1, 2, 3, 4, 5]);
console.log(arr.with(2, 6)); // Uint8Array [1, 2, 6, 4, 5]
console.log(arr); // Uint8Array [1, 2, 3, 4, 5]
```

See also

- Polyfill of `TypedArray.prototype.with` in `core-js`
- JavaScript typed arrays guide
- `TypedArray.prototype.toReversed()`
- `TypedArray.prototype.toSorted()`
- `TypedArray.prototype.at()`
- `Array.prototype.with()`

TypeError

The **`TypeError`** object represents an error when an operation could not be performed, typically (but not exclusively) when a value is not of the expected type.

A `TypeError` may be thrown when:

- an operand or argument passed to a function is incompatible with the type expected by that operator or function; or
- when attempting to modify a value that cannot be changed; or
- when attempting to use a value in an inappropriate way.

`TypeError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`TypeError` is a subclass of `Error`.

Constructor

`TypeError()`

Creates a new `TypeError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `TypeError.prototype` and shared by all `TypeError` instances.

`TypeError.prototype.constructor`

The constructor function that created the instance object. For `TypeError` instances, the initial value is the `TypeError` constructor.

`TypeError.prototype.name`

Represents the name for the type of error. For `TypeError.prototype.name`, the initial value is `"TypeError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a TypeError

```
try {
  null.f();
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "null has no properties"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

Creating a TypeError

```
try {
  throw new TypeError("Hello");
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`

TypeError() constructor

The **`TypeError()`** constructor creates `TypeError` objects.

Syntax

```
new TypeError()
new TypeError(message)
new TypeError(message, options)
new TypeError(message, fileName)
new TypeError(message, fileName, lineNumber)
```

```
TypeError()
TypeError(message)
TypeError(message, options)
TypeError(message, fileName)
TypeError(message, fileName, lineNumber)
```

> **Note:** `TypeError()` can be called with or without `new`. Both create a new `TypeError` instance.

Parameters

`message` Optional

Human-readable description of the error

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception

`lineNumber` Optional

The line number of the code that caused the exception

Examples

Catching a TypeError

```
try {
  null.f();
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "null has no properties"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

Creating a TypeError

```
try {
  throw new TypeError("Hello");
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`

Uint16Array

The **`Uint16Array`** typed array represents an array of 16-bit unsigned integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint16Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Uint16Array()`

Creates a new `Uint16Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint16Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of `Uint16Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint16Array.prototype` and shared by all `Uint16Array` instances.

`Uint16Array.prototype.BYTES_PER_ELEMENT`

> Returns a number value of the element size. `2` in the case of a `Uint16Array`.

`Uint16Array.prototype.constructor`

> The constructor function that created the instance object. For `Uint16Array` instances, the initial value is the `Uint16Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Uint16Array

```
// From a length
const uint16 = new Uint16Array(2);
uint16[0] = 42;
console.log(uint16[0]); // 42
console.log(uint16.length); // 2
console.log(uint16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Uint16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Uint16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint16FromIterable = new Uint16Array(iterable);
console.log(uint16FromIterable);
// Uint16Array [1, 2, 3]
```

See also

- Polyfill of `Uint16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint16Array() constructor

The **`Uint16Array()`** constructor creates `Uint16Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Uint16Array()
new Uint16Array(length)
new Uint16Array(typedArray)
new Uint16Array(object)

new Uint16Array(buffer)
new Uint16Array(buffer, byteOffset)
new Uint16Array(buffer, byteOffset, length)
```

> **Note:** `Uint16Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint16Array

```
// From a length
const uint16 = new Uint16Array(2);
uint16[0] = 42;
console.log(uint16[0]); // 42
console.log(uint16.length); // 2
console.log(uint16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Uint16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Uint16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint16FromIterable = new Uint16Array(iterable);
console.log(uint16FromIterable);
// Uint16Array [1, 2, 3]
```

See also

- Polyfill of `Uint16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint32Array

The **`Uint32Array`** typed array represents an array of 32-bit unsigned integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint32Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Uint32Array()`

Creates a new `Uint32Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint32Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of `Uint32Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint32Array.prototype` and shared by all `Uint32Array` instances.

`Uint32Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of a `Uint32Array`.

`Uint32Array.prototype.constructor`

The constructor function that created the instance object. For `Uint32Array` instances, the initial value is the `Uint32Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Uint32Array

```
// From a length
const uint32 = new Uint32Array(2);
uint32[0] = 42;
console.log(uint32[0]); // 42
console.log(uint32.length); // 2
console.log(uint32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Uint32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Uint32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint32FromIterable = new Uint32Array(iterable);
console.log(uint32FromIterable);
// Uint32Array [1, 2, 3]
```

See also

- Polyfill of `Uint32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint32Array() constructor

The **`Uint32Array()`** constructor creates `Uint32Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Uint32Array()
new Uint32Array(length)
new Uint32Array(typedArray)
new Uint32Array(object)

new Uint32Array(buffer)
new Uint32Array(buffer, byteOffset)
new Uint32Array(buffer, byteOffset, length)
```

> **Note:** `Uint32Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint32Array

```
// From a length
const uint32 = new Uint32Array(2);
uint32[0] = 42;
console.log(uint32[0]); // 42
console.log(uint32.length); // 2
console.log(uint32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Uint32Array([21, 31]);
```

```
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Uint32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint32FromIterable = new Uint32Array(iterable);
console.log(uint32FromIterable);
// Uint32Array [1, 2, 3]
```

See also

- Polyfill of `Uint32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint8Array

The **`Uint8Array`** typed array represents an array of 8-bit unsigned integers. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint8Array` is a subclass of the hidden `TypedArray` class.

Description

`Uint8Array` is currently the only `TypedArray` subclass that has additional methods compared to other typed arrays. Because of its nature as a generic byte array, it is the most suitable for working with arbitrary binary data. It supports two sets of methods for the creation, serialization, and modification of `Uint8Array` data to/from hex strings and base64 strings.

- `Uint8Array.fromBase64()`, `Uint8Array.prototype.toBase64()`, and `Uint8Array.prototype.setFromBase64()` for working with base64 strings, where 3 bytes are encoded by 4 characters that are either 0–9, A–Z, a–z, "+", and "/" (or "-" and "_", if using URL-safe base64).
- `Uint8Array.fromHex()`, `Uint8Array.prototype.toHex()`, and `Uint8Array.prototype.setFromHex()` for working with hex strings, where every byte is encoded by two characters, each one being either 0–9 or A–F (case-insensitive).

Constructor

`Uint8Array()`

> Creates a new `Uint8Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint8Array.BYTES_PER_ELEMENT`

> Returns a number value of the element size. `1` in the case of `Uint8Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

`Uint8Array.fromBase64()`

> Creates a new `Uint8Array` object from a base64-encoded string.

`Uint8Array.fromHex()`

> Creates a new `Uint8Array` object from a hex-encoded string.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint8Array.prototype` and shared by all `Uint8Array` instances.

`Uint8Array.prototype.BYTES_PER_ELEMENT`

> Returns a number value of the element size. `1` in the case of a `Uint8Array`.

`Uint8Array.prototype.constructor`

> The constructor function that created the instance object. For `Uint8Array` instances, the initial value is the `Uint8Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

`Uint8Array.prototype.setFromBase64()`

> Populates this `Uint8Array` object with bytes from a base64-encoded string, returning an object indicating how many bytes were read and written.

`Uint8Array.prototype.setFromHex()`

> Populates this `Uint8Array` object with bytes from a hex-encoded string, returning an object indicating how many bytes were read and written.

`Uint8Array.prototype.toBase64()`

> Returns a base64-encoded string based on the data in this `Uint8Array` object.

`Uint8Array.prototype.toHex()`

> Returns a hex-encoded string based on the data in this `Uint8Array` object.

Examples

Different ways to create a Uint8Array

```
// From a length
const uint8 = new Uint8Array(2);
uint8[0] = 42;
console.log(uint8[0]); // 42
console.log(uint8.length); // 2
console.log(uint8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8FromIterable = new Uint8Array(iterable);
console.log(uint8FromIterable);
// Uint8Array [1, 2, 3]
```

See also

- Polyfill of `Uint8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint8Array.fromBase64()

The **`Uint8Array.fromBase64()`** static method creates a new `Uint8Array` object from a base64-encoded string.

This method should be preferred over `Window.atob()` because it results in a byte array, which is easier to work with than a string containing raw bytes, unless your decoded binary data is actually intended to be ASCII text. If you already have an array buffer allocated and you want to populate it, use the instance method `Uint8Array.prototype.setFromBase64()` instead.

Syntax

```
Uint8Array.fromBase64(string)
Uint8Array.fromBase64(string, options)
```

Parameters

`string`

> A base64 string encoding bytes to convert to a `Uint8Array`. The string must only contain characters in the base64 alphabet, which includes A–Z, a–z, 0–9, and two special characters, which are either `+` and `/` (if using `alphabet: "base64"` in `options`) or `-` and `_` (if using `alphabet: "base64url"` in `options`). It may have padding `=` characters at the end. Any ASCII white space characters within the string are ignored.

`options` Optional

> An object customizing the base64 string interpretation process. It can contain the following properties:

`alphabet` Optional

A string specifying the base64 alphabet to use. It can be one of the following:

`"base64"` (default)

Accept input encoded with the standard base64 alphabet, which uses `+` and `/`.

`"base64url"`

Accept input encoded with the URL-safe base64 alphabet, which uses `-` and `_`.

`lastChunkHandling` Optional

A string specifying how to handle the last chunk of the base64 string. Because every 4 characters in base64 encodes 3 bytes, the string is separated into chunks of 4 characters. If the last chunk has fewer than 4 characters, it needs to be handled differently. It can be one of the following:

`"loose"` (default)

The last chunk can either be 2 or 3 base64 characters, or exactly 4 characters long with padding `=` characters. The last chunk is decoded and appended to the result.

`"strict"`

The last chunk must be exactly 4 characters long with padding `=` characters. Furthermore, overflow bits (trailing bits from the last base64 character that don't represent any data) must be 0. The last chunk is decoded and appended to the result.

`"stop-before-partial"`

If the last chunk is exactly 4 characters long with padding `=` characters, then it's decoded and appended to the result. Otherwise, the last partial chunk is ignored (but if it contains one base64 character followed by `=`, then a syntax error is still thrown). This is useful if the string is coming from a stream and the last chunk is not yet complete. To know how many characters of the input were read, use `Uint8Array.prototype.setFromBase64()` instead (the linked page also contains an example of stream decoding using `"stop-before-partial"`).

Return value

A new `Uint8Array` object containing the decoded bytes from the base64-encoded string.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the specified alphabet, or if the last chunk does not satisfy the `lastChunkHandling` option.

`TypeError`

Thrown in one of the following cases:

- The input string is not a string.
- The `options` object is not an object or `undefined`.
- The options are not of the expected values or `undefined`.

Examples

Decoding a base64 string

This example uses the default `alphabet` and `lastChunkHandling` options to decode a base64 string. Note that:

- The whitespace in the space is ignored.
- The string has 14 base64 characters, not a multiple of 4. This is only valid and decodable with `lastChunkHandling: "loose"`.
- The last chunk, `Ph`, ends in the character `h` which is `0b100001` in base64, so the last `0001` bits are "overflow bits" and are ignored. This is only valid and decodable with `lastChunkHandling: "loose"`.

```
const uint8Array = Uint8Array.fromBase64("PGI+ TURO PC9i Ph");
console.log(uint8Array); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]
```

Decoding a URL-safe base64 string

This example uses the `alphabet` option to decode a URL-safe base64 string.

```
const uint8Array = Uint8Array.fromBase64("PGI-TUROPC9iPg", {
  alphabet: "base64url",
});
console.log(uint8Array); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]
```

Decoding a base64 string with strict last chunk handling

This example uses the `lastChunkHandling` option to decode a base64 string, where the last chunk must be exactly 4 characters long with padding `=` characters, and the overflow bits must be 0.

```
const array1 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg==", {
  lastChunkHandling: "strict",
});
```

```
console.log(array1); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]

const array2 = Uint8Array.fromBase64("PGI+ TURO PC9i Ph==", {
  lastChunkHandling: "strict",
});
// Throws a SyntaxError because h is 0b100001, where the last 4 bits are not 0

const array3 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg", {
  lastChunkHandling: "strict",
});
// Throws a SyntaxError because the last chunk is not exactly 4 characters long
```

Decoding a base64 string with partial last chunk handling

This example uses the `lastChunkHandling` option to decode a base64 string, ignoring any partial last chunk.

```
// The last chunk is complete
const array1 = Uint8Array.fromBase64("PGI+ TURO PC9i", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array1); // Uint8Array(9) [60, 98, 62, 77, 68, 78, 60, 47, 98]

// The last chunk is also complete with padding
const array2 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg==", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array2); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]

// The last chunk is partial; it's ignored
const array3 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array3); // Uint8Array(9) [60, 98, 62, 77, 68, 78, 60, 47, 98]

// The last chunk is partial with padding; it's still ignored
const array4 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg=", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array4); // Uint8Array(9) [60, 98, 62, 77, 68, 78, 60, 47, 98]

// The last chunk is partial, but it contains one base64 character followed by `=`
const array5 = Uint8Array.fromBase64("PGI+ TURO PC9i P=", {
  lastChunkHandling: "stop-before-partial",
});
// Throws a SyntaxError because this cannot possibly be part of a valid base64 string
```

See also

- Polyfill of `Uint8Array.fromBase64` in `core-js`
- es-shims polyfill of `Uint8Array.fromBase64`
- `Uint8Array`
- `Uint8Array.prototype.setFromBase64()`
- `Uint8Array.prototype.toBase64()`
- `Window.atob()`

Uint8Array.fromHex()

The **`Uint8Array.fromHex()`** static method creates a new `Uint8Array` object from a hexadecimal string.

This method parses the string into a byte array. To convert the string into a single number, use the `parseInt()` function with `radix` set to `16` instead.

Syntax

```
Uint8Array.fromHex(string)
```

Parameters

`string`

A hexadecimal string encoding bytes to convert to a `Uint8Array`. The string must:

- Have an even number of characters because two characters encode one byte.
- Only contain characters in the hexadecimal alphabet, which includes 0–9 and A–F (case-insensitive).
- Not contain whitespace (unlike `Uint8Array.prototype.setFromBase64()`).

Return value

A new `Uint8Array` object containing the decoded bytes from the hexadecimal string.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the hex alphabet, or its length is odd.

`TypeError`

Thrown if the input string is not a string.

Examples

Decoding a hexadecimal string

This example decodes a hexadecimal string into a `Uint8Array`.

```
const hexString = "cafed00d";
const bytes = Uint8Array.fromHex(hexString);
console.log(bytes); // Uint8Array [ 202, 254, 208, 13 ]
```

Uppercase characters are also supported:

```
const hexString = "CAFEd00d";
const bytes = Uint8Array.fromHex(hexString);
console.log(bytes); // Uint8Array [ 202, 254, 208, 13 ]
```

See also

- Polyfill of `Uint8Array.fromHex` in `core-js`
- es-shims polyfill of `Uint8Array.fromHex`
- `Uint8Array`
- `Uint8Array.prototype.setFromHex()`
- `Uint8Array.prototype.toHex()`
- `parseInt()`

Uint8Array.prototype.setFromBase64()

The **`setFromBase64()`** method of `Uint8Array` instances populates this `Uint8Array` object with bytes from a base64-encoded string, returning an object indicating how many bytes were read and written.

This method is most suitable for populating a pre-allocated array buffer. If you just want to create a new `Uint8Array` object from a base64-encoded string, use the static method `Uint8Array.fromBase64()` instead.

Syntax

```
setFromBase64(string)
setFromBase64(string, options)
```

Parameters

`string`

A base64 string encoding bytes to write into a `Uint8Array`. It has the same requirements as the `string` parameter of `Uint8Array.fromBase64()`. Note that the string is only read up to the point where the array is filled, so any invalid base64 syntax after that point is ignored.

`options` Optional

An object customizing the base64 string interpretation process. It has the same requirements as the `options` parameter of `Uint8Array.fromBase64()`.

Return value

An object containing the following properties:

`read`

The number of base64 characters read from the input string. If the decoded data fits into the array, this is the length of the input string (including padding); otherwise, it is the length up to the last complete 4-character chunk that fits into the array. Chunks will never be split (because the remaining bits cannot be partially "put back" into the base64 without completely re-encoding it); if the next chunk cannot fit into the remainder of the array, it will be entirely unread, resulting in the last one or two bytes of the array not being written.

`written`

The number of bytes written to the `Uint8Array`. Will never be greater than this `Uint8Array`'s `byteLength`.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the specified alphabet, or if the last chunk does not satisfy the `lastChunkHandling` option.

`TypeError`

Thrown in one of the following cases:

- The input string is not a string.
- The `options` object is not an object or `undefined`.
- The options are not of the expected values or `undefined`.

Examples

Decoding a base64 string

This example uses the default `alphabet` and `lastChunkHandling` options to decode a base64 string into an existing `Uint8Array`.

```
const uint8Array = new Uint8Array(16);
const result = uint8Array.setFromBase64("PGI+ TURO PC9i Pg==");
console.log(result); // { read: 19, written: 10 }
console.log(uint8Array);
// Uint8Array(16) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62, 0, 0, 0, 0, 0, 0]
```

Decoding a big string into a small array

If the string contains more data than the array can hold, the method will only write as many bytes as the array can hold, without discarding any bits.

```
const uint8Array = new Uint8Array(8);
const result = uint8Array.setFromBase64("PGI+ TURO PC9i Pg==");
console.log(result); // { read: 9, written: 6 }
console.log(uint8Array);
// Uint8Array(8) [60, 98, 62, 77, 68, 78, 0, 0]
```

Note how the last two bytes of the array are not written. To decode these two bytes, we need to read at least three more base64 characters, which represent 18 bits. These can't fit into the remaining two bytes of the array, so we can only write 2 chunks, or 6 bytes.

Setting data at a specific offset

The `setFromBase64()` method always starts writing at the beginning of the `Uint8Array`. If you want to write to the middle of the array, you can write to a `TypedArray.prototype.subarray()` instead.

```
const uint8Array = new Uint8Array(16);
// Start writing at offset 2
const result = uint8Array.subarray(2).setFromBase64("PGI+ TURO PC9i Pg==");
console.log(result); // { read: 19, written: 10 }
console.log(uint8Array);
// Uint8Array(16) [0, 0, 60, 98, 62, 77, 68, 78, 60, 47, 98, 62, 0, 0, 0, 0]
```

Stream decoding

This example is adapted from the original proposal. It mimics the `TextDecoder` API with the `stream` option. Note the use of `lastChunkHandling: "stop-before-partial"` to handle incomplete chunks.

```
class Base64Decoder {
  #extra = "";

  decode(chunk = "", options = {}) {
    const opts = { ...options };
    // match TextEncoder API
    if (opts.stream) {
      opts.lastChunkHandling = "stop-before-partial";
    }
    chunk = this.#extra + chunk;
    this.#extra = "";
    // For simplicity, allocate new memory every time
    // the calculation below is guaranteed to be enough,
    // but may be too much if there is whitespace
    // if you're really concerned about memory, a TextDecoder style API is a bad choice
    let buffer = new Uint8Array(Math.ceil((chunk.length * 3) / 4));
    const { read, written } = buffer.setFromBase64(chunk, opts);
    buffer = buffer.subarray(0, written);
    this.#extra = chunk.slice(read);
    return buffer;
  }
}

const decoder = new Base64Decoder();

console.log(decoder.decode("SG Vsb ", { stream: true }));
// Uint8Array(3) [72, 101, 108]
console.log(decoder.decode("G8gV29ybGR ", { stream: true }));
// Uint8Array(6) [108, 111, 32, 87, 111, 114]
console.log(decoder.decode(""));
// Uint8Array(2) [108, 100]
```

See also

- Polyfill of `Uint8Array.prototype.setFromBase64` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.setFromBase64`
- `Uint8Array`
- `Uint8Array.fromBase64()`
- `Uint8Array.prototype.toBase64()`
- `Window.atob()`

Uint8Array.prototype.setFromHex()

The **`setFromHex()`** method of `Uint8Array` instances populates this `Uint8Array` object with bytes from a hex-encoded string, returning an object indicating how many bytes were read and written.

This method parses the string into a byte array. To convert the string into a single number, use the `parseInt()` function with `radix` set to `16` instead.

Syntax

```
setFromHex(string)
```

Parameters

`string`

A hexadecimal string encoding bytes to write into a `Uint8Array`. The string must:

- Have an even number of characters because two characters encode one byte.
- Only contain characters in the hexadecimal alphabet, which includes 0–9 and A–F (case-insensitive).
- Not contain whitespace (unlike `Uint8Array.prototype.setFromBase64()`).

Note that the string is only read up to the point where the array is filled, so any invalid hex syntax after that point is ignored.

Return value

An object containing the following properties:

`read`

The number of hex characters read from the input string. If the decoded data fits into the array, it is the length of the input string: otherwise, it is the number of complete hex characters that fit into the array.

`written`

The number of bytes written to the `Uint8Array`. Will never be greater than this `Uint8Array`'s `byteLength`.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the hex alphabet, or its length is odd.

`TypeError`

Thrown if the input string is not a string.

Examples

Decoding a hexadecimal string

This example decodes a hexadecimal string into an existing `Uint8Array`.

```
const uint8Array = new Uint8Array(8);
const result = uint8Array.setFromHex("cafed00d");
console.log(result); // { read: 8, written: 4 }
console.log(uint8Array); // Uint8Array(8) [202, 254, 208, 13, 0, 0, 0, 0]
```

Decoding a big string into a small array

If the string contains more data than the array can hold, the method will only write as many bytes as the array can hold.

```
const uint8Array = new Uint8Array(4);
const result = uint8Array.setFromHex("cafed00d-some random stuff");
console.log(result); // { read: 8, written: 4 }
console.log(uint8Array); // Uint8Array(4) [202, 254, 208, 13]
```

Excess characters are ignored, even if they are invalid. However the total length of the input string must be even.

Setting data at a specific offset

The `setFromHex()` method always starts writing at the beginning of the `Uint8Array`. If you want to write to the middle of the array, you can write to a `TypedArray.prototype.subarray()` instead.

```
const uint8Array = new Uint8Array(8);
// Start writing at offset 2
const result = uint8Array.subarray(2).setFromHex("cafed00d");
console.log(result); // { read: 8, written: 4 }
console.log(uint8Array);
// Uint8Array(8) [0, 0, 202, 254, 208, 13, 0, 0]
```

See also

- Polyfill of `Uint8Array.prototype.setFromHex` in `core-js`

- es-shims polyfill of `Uint8Array.prototype.setFromHex`
- `Uint8Array`
- `Uint8Array.fromHex()`
- `Uint8Array.prototype.toHex()`
- `parseInt()`

Uint8Array.prototype.toBase64()

The **`toBase64()`** method of `Uint8Array` instances returns a base64-encoded string based on the data in this `Uint8Array` object.

This method should be preferred over `Window.btoa()`, especially if you already have a `Uint8Array` holding the object, because you don't need to convert it to a string first.

Syntax

```
toBase64()
toBase64(options)
```

Parameters

`options` Optional

An object customizing the base64 string format. It can contain the following properties:

`alphabet` Optional

A string specifying the base64 alphabet to use. It can be one of the following:

`"base64"` (default)

Encode input with the standard base64 alphabet, which uses `+` and `/`.

`"base64url"`

Encode input with the URL-safe base64 alphabet, which uses `-` and `_`.

`omitPadding` Optional

A boolean specifying whether to omit padding characters (`=`) at the end of the base64 string. The default is `false`.

Return value

A base64-encoded string representing the data in the `Uint8Array`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- The `options` object is not an object or `undefined`.
- The `options.alphabet` is not of the expected values or `undefined`.

Examples

Encoding binary data

This example uses the default `alphabet` and `omitPadding` options to encode data from a `Uint8Array` into a base64 string.

```
const uint8Array = new Uint8Array([29, 233, 101, 161]);
console.log(uint8Array.toBase64()); // "HelloQ=="
```

Encoding data without padding

```
const uint8Array = new Uint8Array([29, 233, 101, 161]);
console.log(uint8Array.toBase64({ omitPadding: true })); // "HelloQ"
```

Encoding data with URL-safe alphabet

This example populates a `URLSearchParams` object with a base64-encoded string using the URL-safe alphabet.

```
const uint8Array = new Uint8Array([46, 139, 222, 255, 42, 46]);
const base64 = uint8Array.toBase64({ alphabet: "base64url" });
const params = new URLSearchParams();
params.set("data", base64);
console.log(params.toString()); // "data=Love_you"
```

Stream encoding

This example is adapted from the original proposal, showcasing how to implement streaming in userland. It mimics the `TextEncoder` API with the `stream` option.

```
class Base64Encoder {
  #extra;
```

```
  #extraLength;
  constructor() {
    this.#extra = new Uint8Array(3);
    this.#extraLength = 0;
  }

  // Partly derived from https://github.com/lucacasonato/base64_streams/blob/main/src/iterator/encoder.ts
  encode(chunk = Uint8Array.of(), options = {}) {
    const stream = options.stream ?? false;

    if (this.#extraLength > 0) {
      const bytesNeeded = 3 - this.#extraLength;
      const bytesAvailable = Math.min(bytesNeeded, chunk.length);
      this.#extra.set(chunk.subarray(0, bytesAvailable), this.#extraLength);
      chunk = chunk.subarray(bytesAvailable);
      this.#extraLength += bytesAvailable;
    }

    if (!stream) {
      // assert: this.#extraLength.length === 0 || this.#extraLength === 3 || chunk.length === 0
      const prefix = this.#extra.subarray(0, this.#extraLength).toBase64();
      this.#extraLength = 0;
      return prefix + chunk.toBase64();
    }

    let extraReturn = "";

    if (this.#extraLength === 3) {
      extraReturn = this.#extra.toBase64();
      this.#extraLength = 0;
    }

    const remainder = chunk.length % 3;
    if (remainder > 0) {
      this.#extra.set(chunk.subarray(chunk.length - remainder));
      this.#extraLength = remainder;
      chunk = chunk.subarray(0, chunk.length - remainder);
    }

    return extraReturn + chunk.toBase64();
  }
}

const encoder = new Base64Encoder();

console.log(
  encoder.encode(Uint8Array.of(72, 101, 108, 108, 111), { stream: true }),
);
// "SGVs"
console.log(
  encoder.encode(Uint8Array.of(32, 87, 111, 114, 108, 100), { stream: true }),
);
// "bG8gV29y"
console.log(encoder.encode());
// "bGQ="
```

See also

- Polyfill of `Uint8Array.prototype.toBase64` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.toBase64`
- `Uint8Array`
- `Uint8Array.fromBase64()`
- `Uint8Array.prototype.setFromBase64()`
- `Window.btoa()`

Uint8Array.prototype.toHex()

The **`toHex()`** method of `Uint8Array` instances returns a hex-encoded string based on the data in this `Uint8Array` object.

This method creates strings from a byte array. To convert individual numbers into hex, use the `Number.prototype.toString()` method with `radix` set to `16` instead.

Syntax

```
toHex()
```

Parameters

None.

Return value

A hex-encoded string representing the data in the `Uint8Array`.

Examples

Encoding binary data

This example encodes data from a `Uint8Array` into a hex string.

```
const uint8Array = new Uint8Array([202, 254, 208, 13]);
console.log(uint8Array.toHex()); // "cafed00d"

const data = new Uint8Array([255, 0, 0, 0, 255, 0, 0, 0, 255]);
for (let i = 0; i < data.length; i += 3) {
  console.log(data.slice(i, i + 3).toHex());
}
// "ff0000"
// "00ff00"
// "0000ff"
```

See also

- Polyfill of `Uint8Array.prototype.toHex` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.toHex`
- `Uint8Array`
- `Uint8Array.fromHex()`
- `Uint8Array.prototype.setFromHex()`
- `Number.prototype.toString()`

Uint8Array() constructor

The **`Uint8Array()`** constructor creates `Uint8Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Uint8Array()
new Uint8Array(length)
new Uint8Array(typedArray)
new Uint8Array(object)

new Uint8Array(buffer)
new Uint8Array(buffer, byteOffset)
new Uint8Array(buffer, byteOffset, length)
```

> **Note:** `Uint8Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint8Array

```
// From a length
const uint8 = new Uint8Array(2);
uint8[0] = 42;
console.log(uint8[0]); // 42
console.log(uint8.length); // 2
console.log(uint8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8FromIterable = new Uint8Array(iterable);
console.log(uint8FromIterable);
// Uint8Array [1, 2, 3]
```

See also

- Polyfill of `Uint8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint8ClampedArray

The **`Uint8ClampedArray`** typed array represents an array of 8-bit unsigned integers clamped to 0–255. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint8ClampedArray` is a subclass of the hidden `TypedArray` class.

Constructor

`Uint8ClampedArray()`

Creates a new `Uint8ClampedArray` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint8ClampedArray.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of `Uint8ClampedArray`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint8ClampedArray.prototype` and shared by all `Uint8ClampedArray` instances.

`Uint8ClampedArray.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of a `Uint8ClampedArray`.

`Uint8ClampedArray.prototype.constructor`

The constructor function that created the instance object. For `Uint8ClampedArray` instances, the initial value is the `Uint8ClampedArray` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Uint8ClampedArray

```
// From a length
const uint8c = new Uint8ClampedArray(2);
uint8c[0] = 42;
uint8c[1] = 1337;
console.log(uint8c[0]); // 42
console.log(uint8c[1]); // 255 (clamped)
console.log(uint8c.length); // 2
console.log(uint8c.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8ClampedArray([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8ClampedArray(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8ClampedArray(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8cFromIterable = new Uint8ClampedArray(iterable);
```

```
console.log(uint8cFromIterable);
// Uint8ClampedArray [1, 2, 3]
```

See also

- Polyfill of `Uint8ClampedArray` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Uint8ClampedArray() constructor

The **`Uint8ClampedArray()`** constructor creates `Uint8ClampedArray` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```
new Uint8ClampedArray()
new Uint8ClampedArray(length)
new Uint8ClampedArray(typedArray)
new Uint8ClampedArray(object)

new Uint8ClampedArray(buffer)
new Uint8ClampedArray(buffer, byteOffset)
new Uint8ClampedArray(buffer, byteOffset, length)
```

> **Note:** `Uint8ClampedArray()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint8ClampedArray

```
// From a length
const uint8c = new Uint8ClampedArray(2);
uint8c[0] = 42;
uint8c[1] = 1337;
console.log(uint8c[0]); // 42
console.log(uint8c[1]); // 255 (clamped)
console.log(uint8c.length); // 2
console.log(uint8c.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8ClampedArray([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8ClampedArray(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8ClampedArray(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8cFromIterable = new Uint8ClampedArray(iterable);
console.log(uint8cFromIterable);
// Uint8ClampedArray [1, 2, 3]
```

See also

- Polyfill of `Uint8ClampedArray` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

undefined

The **`undefined`** global property represents the primitive value `undefined`. It is one of JavaScript's primitive types.

Try it

```
function test(t) {
  if (t === undefined) {
    return "Undefined value!";
  }
  return t;
}

let x;

console.log(test(x));
// Expected output: "Undefined value!"
```

Value

The primitive value `undefined`.

Property attributes of `undefined`	
Writable	no
Enumerable	no
Configurable	no

Description

`undefined` is a property of the *global object*. That is, it is a variable in global scope.

In all non-legacy browsers, `undefined` is a non-configurable, non-writable property. Even when this is not the case, avoid overriding it.

A variable that has not been assigned a value is of type `undefined`. A function returns `undefined` if a value was not `returned`. Accessing a property that does not exist also returns `undefined`. The `void` operator always returns `undefined`.

> **Note:** While you can use `undefined` as an identifier (variable name) in any scope other than the global scope (because `undefined` is not a reserved word), doing so is a very bad idea that will make your code difficult to maintain and debug.
>
> ```
> // DON'T DO THIS
>
> (() => {
>   const undefined = "foo";
>   console.log(undefined, typeof undefined); // foo string
> })();
>
> ((undefined) => {
>   console.log(undefined, typeof undefined); // foo string
> })("foo");
> ```

Examples

Strict equality and undefined

You can use `undefined` and the strict equality and inequality operators to determine whether a variable has a value. In the following code, the variable `x` is not initialized, and the `if` statement evaluates to true.

```
let x;
if (x === undefined) {
  // these statements execute
} else {
  // these statements do not execute
}
```

> **Note:** The *strict equality* operator (as opposed to the *loose equality* operator) must be used here, because `x == undefined` also checks whether `x` is `null`, while strict equality doesn't. This is because `null` is not equivalent to `undefined`.
>
> See Equality comparison and sameness for details.

typeof operator and undefined

`typeof` can also determine whether a variable is `undefined`:

```
let x;
if (typeof x === "undefined") {
  // these statements execute
}
```

One reason to use `typeof` is that it does not throw an error if the variable does not exist in the current scope.

```
// x has not been declared before
// evaluates to true without errors
if (typeof x === "undefined") {
  // these statements execute
}

// Throws a ReferenceError
if (x === undefined) {
}
```

It also works with variables declared with `var` *after* the check, because the declaration is hoisted to the top of the scope with value `undefined`.

```
if (typeof x === "undefined") {
  // these statements execute
}
var x = 1;
```

This technique is usually only useful for testing global variables. You can know if a variable exists at any other scope (blocks, functions, modules, etc.) just by looking at the source code. The global scope is bound to the global object, so checking the existence of a variable in the global context can be done by checking the existence of a property on the *global object*, such as by using the `in` operator:

```
if ("x" in window) {
  // These statements execute only if x is defined globally
}
```

However, none of the techniques above work if the variable is declared with `let`, `const`, or other lexical declarations. Using `typeof` before the line of declaration still produces a `ReferenceError`, due to the temporal dead zone (TDZ).

```
if (typeof z === "undefined") {
  // Uncaught ReferenceError: Cannot access 'z' before initialization
}
let z = 1;
```

Furthermore, `let` and `const` declarations do not create properties on the global object, so they cannot be checked with the `in` operator either.

```
let z;
if ("z" in window) {
  // false, even if z is declared globally with let or const
}
```

If you want to share global variables across different scripts, it is more advisable to use `var` or explicitly attach them to the global object:

```
window.myGlobalVar = "foo";
```

void operator and undefined

The `void` operator can also be used to produce the `undefined` value. This is very commonly seen in minified code because `void 0` is 3 bytes shorter and cannot be overridden. You should usually avoid this pattern in your own code.

```
let x;
if (x === void 0) {
  // these statements execute
}

// y has not been declared before
if (y === void 0) {
  // throws Uncaught ReferenceError: y is not defined
}
```

See also

- JavaScript data types and data structures
- `null`

unescape()

> **Note:** `unescape()` is a non-standard function implemented by browsers and was only standardized for cross-engine compatibility. It is not required to be implemented by all JavaScript engines and may not work everywhere. Use `decodeURIComponent()` or `decodeURI()` if possible.

The **`unescape()`** function computes a new string in which hexadecimal escape sequences are replaced with the characters that they represent. The escape sequences might be introduced by a function like `escape()`.

Syntax

```
unescape(str)
```

Parameters

`str`

A string to be decoded.

Return value

A new string in which certain characters have been unescaped.

Description

`unescape()` is a function property of the global object.

The `unescape()` function replaces any escape sequence with the character that it represents. Specifically, it replaces any escape sequence of the form `%xx` or `%uxxxx` (where `x` represents one hexadecimal digit) with the character that has the hexadecimal value `xx`/`xxxx`. If the escape sequence is not a valid escape sequence (for example, if `%` is followed by one or no hex digit), it is left as-is.

> **Note:** This function was used mostly for percent-encoding and is partly based on the escape format in RFC 1738. The `unescape()` function does *not* evaluate escape sequences in string literals. You can replace `\xXX` with `%XX` and `\uXXXX` with `%uXXXX` to get a string that can be handled by `unescape()`.

Examples

Using unescape()

```
unescape("abc123"); // "abc123"
unescape("%E4%F6%FC"); // "äöü"
unescape("%u0107"); // "ć"
```

See also

- Polyfill of `unescape` in `core-js`
- `decodeURI`
- `decodeURIComponent`
- `escape`

URIError

The **`URIError`** object represents an error when a global URI handling function was used in a wrong way.

`URIError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`URIError` is a subclass of `Error`.

Constructor

`URIError()`

Creates a new `URIError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `URIError.prototype` and shared by all `URIError` instances.

`URIError.prototype.constructor`

The constructor function that created the instance object. For `URIError` instances, the initial value is the `URIError` constructor.

`URIError.prototype.name`

Represents the name for the type of error. For `URIError.prototype.name`, the initial value is `"URIError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a URIError

```
try {
  decodeURIComponent("%");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "malformed URI sequence"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

Creating a URIError

```
try {
  throw new URIError("Hello");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`

URIError() constructor

The **`URIError()`** constructor creates `URIError` objects.

Syntax

```
new URIError()
new URIError(message)
new URIError(message, options)
new URIError(message, fileName)
new URIError(message, fileName, lineNumber)

URIError()
URIError(message)
URIError(message, options)
URIError(message, fileName)
URIError(message, fileName, lineNumber)
```

> **Note:** `URIError()` can be called with or without `new`. Both create a new `URIError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional

The name of the file containing the code that caused the exception.

`lineNumber` Optional

The line number of the code that caused the exception.

Examples

Catching a URIError

```
try {
  decodeURIComponent("%");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "malformed URI sequence"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

Creating a URIError

```
try {
  throw new URIError("Hello");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "URIError"
```

```
  console.log(e.stack); // Stack of the error
}
```

See also

- `Error`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`

WeakMap

A **`WeakMap`** is a collection of key/value pairs whose keys must be objects or non-registered symbols, with values of any arbitrary JavaScript type, and which does not create strong references to its keys. That is, an object's presence as a key in a `WeakMap` does not prevent the object from being garbage collected. Once an object used as a key has been collected, its corresponding values in any `WeakMap` become candidates for garbage collection as well — as long as they aren't strongly referred to elsewhere. The only primitive type that can be used as a `WeakMap` key is symbol — more specifically, non-registered symbols — because non-registered symbols are guaranteed to be unique and cannot be re-created.

`WeakMap` allows associating data to objects in a way that doesn't prevent the key objects from being collected, even if the values reference the keys. However, a `WeakMap` doesn't allow observing the liveness of its keys, which is why it doesn't allow enumeration; if a `WeakMap` exposed any method to obtain a list of its keys, the list would depend on the state of garbage collection, introducing non-determinism. If you want to have a list of keys, you should use a `Map` rather than a `WeakMap`.

You can learn more about `WeakMap` in the WeakMap object section of the Keyed collections guide.

Description

Keys of WeakMaps must be garbage-collectable. Most primitive data types can be arbitrarily created and don't have a lifetime, so they cannot be used as keys. Objects and non-registered symbols can be used as keys because they are garbage-collectable.

Key equality

Like regular `Map`, value equality is based on the SameValueZero algorithm, which is the same as the `===` operator because `WeakMap` can only hold object and symbol keys. This means that for object keys, equality is based on object identity. They are compared by reference, not by value.

Why WeakMap?

A map API *could* be implemented in JavaScript with two arrays (one for keys, one for values) shared by the four API methods. Setting elements on this map would involve pushing a key and value onto the end of each of those arrays simultaneously. As a result, the indices of the key and value would correspond to both arrays. Getting values from the map would involve iterating through all keys to find a match, then using the index of this match to retrieve the corresponding value from the array of values.

Such an implementation would have two main inconveniences:

1. The first one is an `O(n)` set and search (*n* being the number of keys in the map) since both operations must iterate through the list of keys to find a matching value.
2. The second inconvenience is a memory leak because the arrays ensure that references to each key and each value are maintained indefinitely. These references prevent the keys from being garbage collected, even if there are no other references to the object. This would also prevent the corresponding values from being garbage collected.

By contrast, in a `WeakMap`, a key object refers strongly to its contents as long as the key is not garbage collected, but weakly from then on. As such, a `WeakMap`:

- does not prevent garbage collection, which eventually removes references to the key object
- allows garbage collection of any values if their key objects are not referenced from somewhere other than a `WeakMap`

A `WeakMap` can be a particularly useful construct when mapping keys to information about the key that is valuable *only if* the key has not been garbage collected.

But because a `WeakMap` doesn't allow observing the liveness of its keys, its keys are not enumerable. There is no method to obtain a list of the keys. If there were, the list would depend on the state of garbage collection, introducing non-determinism. If you want to have a list of keys, you should use a `Map`.

Constructor

`WeakMap()`

> Creates a new `WeakMap` object.

Instance properties

These properties are defined on `WeakMap.prototype` and shared by all `WeakMap` instances.

`WeakMap.prototype.constructor`

> The constructor function that created the instance object. For `WeakMap` instances, the initial value is the `WeakMap` constructor.

`WeakMap.prototype[Symbol.toStringTag]`

> The initial value of the `[Symbol.toStringTag]` property is the string `"WeakMap"`. This property is used in `Object.prototype.toString()`.

Instance methods

`WeakMap.prototype.delete()`

Removes the entry specified by the key from this `WeakMap`.

`WeakMap.prototype.get()`

Returns the value corresponding to the key in this `WeakMap`, or `undefined` if there is none.

`WeakMap.prototype.getOrInsert()`

Returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

`WeakMap.prototype.getOrInsertComputed()`

Returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

`WeakMap.prototype.has()`

Returns a boolean indicating whether an entry with the specified key exists in this `WeakMap` or not.

`WeakMap.prototype.set()`

Adds a new entry with a specified key and value to this `WeakMap`, or updates an existing entry if the key already exists.

Examples

Using WeakMap

```
const wm1 = new WeakMap();
const wm2 = new WeakMap();
const wm3 = new WeakMap();
const o1 = {};
const o2 = () => {};
const o3 = window;

wm1.set(o1, 37);
wm1.set(o2, "azerty");
wm2.set(o1, o2); // a value can be anything, including an object or a function
wm2.set(o2, undefined);
wm2.set(wm1, wm2); // keys and values can be any objects. Even WeakMaps!

wm1.get(o2); // "azerty"
wm2.get(o2); // undefined, because that is the set value
wm2.get(o3); // undefined, because there is no key for o3 on wm2

wm1.has(o2); // true
wm2.has(o2); // true (even if the value itself is 'undefined')
wm2.has(o3); // false

wm3.set(o1, 37);
wm3.get(o1); // 37

wm1.has(o1); // true
wm1.delete(o1);
wm1.has(o1); // false
```

Implementing a WeakMap-like class with a .clear() method

```
class ClearableWeakMap {
  #wm;
  constructor(init) {
    this.#wm = new WeakMap(init);
  }
  clear() {
    this.#wm = new WeakMap();
  }
  delete(k) {
    return this.#wm.delete(k);
  }
  get(k) {
    return this.#wm.get(k);
  }
  has(k) {
    return this.#wm.has(k);
  }
  set(k, v) {
    this.#wm.set(k, v);
    return this;
  }
}
```

Emulating private members

Developers can use a `WeakMap` to associate private data to an object, with the following benefits:

- Compared to a `Map`, a WeakMap does not hold strong references to the object used as the key, so the metadata shares the same lifetime as the object itself, avoiding memory leaks.
- Compared to using non-enumerable and/or `Symbol` properties, a WeakMap is external to the object and there is no way for user code to retrieve the metadata through reflective methods like `Object.getOwnPropertySymbols`.
- Compared to a closure, the same WeakMap can be reused for all instances created from a constructor, making it more memory-efficient, and allows different instances of the same class to read the private members of each other.

```
let Thing;

{
  const privateScope = new WeakMap();
  let counter = 0;

  Thing = function () {
    this.someProperty = "foo";

    privateScope.set(this, {
      hidden: ++counter,
    });
  };

  Thing.prototype.showPublic = function () {
    return this.someProperty;
  };

  Thing.prototype.showPrivate = function () {
    return privateScope.get(this).hidden;
  };
}

console.log(typeof privateScope);
// "undefined"

const thing = new Thing();

console.log(thing);
// Thing {someProperty: "foo"}

thing.showPublic();
// "foo"

thing.showPrivate();
// 1
```

This is roughly equivalent to the following, using private fields:

```
class Thing {
  static #counter = 0;
  #hidden;
  constructor() {
    this.someProperty = "foo";
    this.#hidden = ++Thing.#counter;
  }
  showPublic() {
    return this.someProperty;
  }
  showPrivate() {
    return this.#hidden;
  }
}

const thing = new Thing();

console.log(thing);
// Thing {someProperty: "foo"}

thing.showPublic();
// "foo"

thing.showPrivate();
// 1
```

Associating metadata

A `WeakMap` can be used to associate metadata with an object, without affecting the lifetime of the object itself. This is very similar to the private members example, since private members are also modeled as external metadata that doesn't participate in prototypical inheritance.

This use case can be extended to already-created objects. For example, on the web, we may want to associate extra data with a DOM element, which the DOM element may access later. A common approach is to attach the data as a property:

```
const buttons = document.querySelectorAll(".button");
buttons.forEach((button) => {
  button.clicked = false;
  button.addEventListener("click", () => {
```

```
    button.clicked = true;
    const currentButtons = [...document.querySelectorAll(".button")];
    if (currentButtons.every((button) => button.clicked)) {
      console.log("All buttons have been clicked!");
    }
  });
});
```

This approach works, but it has a few pitfalls:

- The `clicked` property is enumerable, so it will show up in `Object.keys(button)`, `for...in` loops, etc. This can be mitigated by using `Object.defineProperty()`, but that makes the code more verbose.
- The `clicked` property is a normal string property, so it can be accessed and overwritten by other code. This can be mitigated by using a `Symbol` key, but the key would still be accessible via `Object.getOwnPropertySymbols()`.

Using a `WeakMap` fixes these:

```
const buttons = document.querySelectorAll(".button");
const clicked = new WeakMap();
buttons.forEach((button) => {
  clicked.set(button, false);
  button.addEventListener("click", () => {
    clicked.set(button, true);
    const currentButtons = [...document.querySelectorAll(".button")];
    if (currentButtons.every((button) => clicked.get(button))) {
      console.log("All buttons have been clicked!");
    }
  });
});
```

Here, only code that has access to `clicked` knows the clicked state of each button, and external code can't modify the states. In addition, if any of the buttons gets removed from the DOM, the associated metadata will automatically get garbage-collected.

Caching

You can associate objects passed to a function with the result of the function, so that if the same object is passed again, the cached result can be returned without re-executing the function. This is useful if the function is pure (i.e., it doesn't mutate any outside objects or cause other observable side effects).

```
const cache = new WeakMap();
function handleObjectValues(obj) {
  if (cache.has(obj)) {
    return cache.get(obj);
  }
  const result = Object.values(obj).map(heavyComputation);
  cache.set(obj, result);
  return result;
}
```

This only works if your function's input is an object. Moreover, even if the input is never passed in again, the result still remains forever in the cache as long as the key (input) is alive. A more effective way is to use a `Map` paired with `WeakRef` objects, which allows you to associate any type of input value with its respective (potentially large) computation result. See the WeakRefs and FinalizationRegistry example for more details.

See also

- Polyfill of `WeakMap` in `core-js`
- Keyed collections
- `Map`
- `Set`
- `WeakSet`

WeakMap.prototype.delete()

The **`delete()`** method of `WeakMap` instances removes the entry specified by the key from this `WeakMap`.

Try it

```
const weakmap = new WeakMap();
const object = {};

weakmap.set(object, 42);

console.log(weakmap.delete(object));
// Expected output: true

console.log(weakmap.has(object));
// Expected output: false
```

Syntax

```
weakMapInstance.delete(key)
```

Parameters

`key`

The key of the entry to remove from the `WeakMap` object. Object keys are compared by reference, not by value.

Return value

`true` if an entry in the `WeakMap` object has been removed successfully. `false` if the key is not found in the `WeakMap`. Always returns `false` if `key` is not an object or a non-registered symbol.

Examples

Using delete()

```
const wm = new WeakMap();
wm.set(window, "foo");

wm.delete(window); // Returns true. Successfully removed.

wm.has(window); // Returns false. The window object is no longer in the WeakMap.
```

See also

- `WeakMap`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`

WeakMap.prototype.get()

The **`get()`** method of `WeakMap` instances returns the value corresponding to the key in this `WeakMap`, or `undefined` if there is none. Object values are returned as the same reference that was originally stored, not as a copy, so mutations to the returned object will be reflected anywhere that reference is held, including inside the `WeakMap`.

Try it

```
const weakmap = new WeakMap();
const object1 = {};
const object2 = {};

weakmap.set(object1, 42);

console.log(weakmap.get(object1));
// Expected output: 42

console.log(weakmap.get(object2));
// Expected output: undefined
```

Syntax

```
get(key)
```

Parameters

`key`

The key of the value to return from the `WeakMap` object. Object keys are compared by reference, not by value.

Return value

The value associated with the specified key in the `WeakMap` object. If the key can't be found, `undefined` is returned. Always returns `undefined` if `key` is not an object or a non-registered symbol.

Examples

Using get()

```
const wm = new WeakMap();
wm.set(window, "foo");

wm.get(window); // Returns "foo".
wm.get("baz"); // Returns undefined.
```

See also

- `WeakMap`
- `WeakMap.prototype.delete()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`

WeakMap.prototype.getOrInsert()

The **`getOrInsert()`** method of `WeakMap` instances returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

If the computation of the default value is expensive, consider using `WeakMap.prototype.getOrInsertComputed()` instead, which takes a callback to compute the default value only if it's actually needed.

Try it

```
const map = new WeakMap([[window, "foo"]]);
console.log(map.getOrInsert(window, "default"));
// Expected output: "foo"

console.log(map.getOrInsert({}, "default"));
// Expected output: "default"
```

Syntax

```
getOrInsert(key, defaultValue)
```

Parameters

`key`

The key of the value to return from the `WeakMap` object. Must be either an object or a non-registered symbol. Object keys are compared by reference, not by value.

`defaultValue`

The value to insert and return if the key is not already present in the `WeakMap` object.

Return value

The value associated with the specified key in the `WeakMap` object. If the key can't be found, `defaultValue` is inserted and returned.

Exceptions

`TypeError`

Thrown if `key` is not an object or a non-registered symbol.

Examples

Using getOrInsert()

```
const wm = new WeakMap();
const obj = {};

console.log(wm.get(obj)); // undefined
console.log(wm.getOrInsert(obj, "default")); // "default"
console.log(wm.get(obj)); // "default"
console.log(wm.getOrInsert(obj, "another default")); // "default"
```

See also

- Polyfill of `WeakMap.prototype.getOrInsert` in `core-js`
- es-shims polyfill of `WeakMap.prototype.getOrInsert`
- `WeakMap`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`
- `WeakMap.prototype.getOrInsertComputed()`

WeakMap.prototype.getOrInsertComputed()

The **`getOrInsertComputed()`** method of `WeakMap` instances returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

Use this method instead of `WeakMap.prototype.getOrInsert()` when the default value is expensive to compute, and you want to avoid computing it unless it's actually needed.

Try it

```
const alan = { name: "Alan" };
const map = new WeakMap([[alan, 35]]);
const defaultCreator = (obj) => `${obj.name}'s age is unknown`;

console.log(map.getOrInsertComputed(alan, defaultCreator));
// Expected output: 35

const brett = { name: "Brett" };
console.log(map.getOrInsertComputed(brett, defaultCreator));
// Expected output: "Brett's age is unknown"
```

Syntax

```
getOrInsertComputed(key, callback)
```

Parameters

`key`

The key of the element to return from the `Map` object. Must be either an object or a non-registered symbol. Object keys are compared by reference, not by value.

`callback`

A function that returns the value to insert and return if the key is not already present in the `Map` object. The function is called with the following argument:

`key`

The same key that was passed to `getOrInsertComputed()`.

Return value

The value associated with the specified key in the `WeakMap` object. If the key can't be found, the result of `callback(key)` is inserted and returned.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `key` is not an object or a non-registered symbol.
- `callback` isn't callable.

Examples

Caching

Caching involves building a mapping from the parameter of an expensive function to its return value, so that future calls with the same parameter can return the cached value instead of recomputing it. When no cached value exists, it needs to be computed and inserted into the cache.

We use a `WeakMap` here instead of a `Map`, so that the cached argument values do not prevent them from being garbage-collected if they are no longer needed elsewhere in the program. If your cached function takes non-object arguments, you can use a `Map` instead.

```
// Any expensive function you want to cache
function computeExpensiveValue(requestOptions) {
  // Imagine expensive computation, like fetching data or complex calculations
  console.log(`Fetching from ${requestOptions.url}`);
  return new Response("Fake response");
}

// A higher-order function that adds caching to any function
function withCache(fn) {
  const cache = new WeakMap();
  return (param) => cache.getOrInsertComputed(param, fn);
}

const computeWithCache = withCache(computeExpensiveValue);
const options = { url: "https://example.com/a", method: "GET" };
const value1 = computeWithCache(options); // Logs "Fetching from https://example.com/a"
const value2 = computeWithCache(options); // No log
```

See also

- Polyfill of `WeakMap.prototype.getOrInsertComputed` in `core-js`
- es-shims polyfill of `WeakMap.prototype.getOrInsertComputed`
- `WeakMap`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`
- `WeakMap.prototype.getOrInsert()`

WeakMap.prototype.has()

The **`has()`** method of `WeakMap` instances returns a boolean indicating whether an entry with the specified key exists in this `WeakMap` or not.

Try it

```
const weakmap = new WeakMap();
const object1 = {};
const object2 = {};

weakmap.set(object1, "foo");

console.log(weakmap.has(object1));
// Expected output: true
```

```
console.log(weakmap.has(object2));
// Expected output: false
```

Syntax

```
has(key)
```

Parameters

`key`

The key of the entry to test for presence in the `WeakMap` object. Object keys are compared by reference, not by value.

Return value

Returns `true` if an entry with the specified key exists in the `WeakMap` object; otherwise `false`. Always returns `false` if `key` is not an object or a non-registered symbol.

Examples

Using has()

```
const wm = new WeakMap();
wm.set(window, "foo");

wm.has(window); // returns true
wm.has("baz"); // returns false
```

See also

- `WeakMap`
- `WeakMap.prototype.delete()`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`

WeakMap.prototype.set()

The **`set()`** method of `WeakMap` instances adds a new entry with a specified key and value to this `WeakMap`, or updates an existing entry if the key already exists.

Try it

```
const weakmap = new WeakMap();
const object1 = {};
const object2 = {};

weakmap.set(object1, "foo");
weakmap.set(object2, "bar");

console.log(weakmap.get(object1));
// Expected output: "foo"

console.log(weakmap.get(object2));
// Expected output: "bar"
```

Syntax

```
set(key, value)
```

Parameters

`key`

The key of the entry to add to or modify within the `WeakMap` object. Must be either an object or a non-registered symbol. Object keys are compared by reference, not by value.

`value`

The value of the entry to add to or modify within the `WeakMap` object. Can be any value.

Return value

The `WeakMap` object.

Exceptions

`TypeError`

Thrown if `key` is not an object or a non-registered symbol.

Examples

Using set()

```
const wm = new WeakMap();
const obj = {};

// Add new elements to the WeakMap
wm.set(obj, "foo").set(window, "bar"); // chainable

// Update an element in the WeakMap
wm.set(obj, "baz");

// Using a non-registered symbol as key
const sym = Symbol("foo");
wm.set(sym, "baz");
wm.set(Symbol.iterator, "qux");
```

See also

- `WeakMap`
- `WeakMap.prototype.delete()`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.has()`

WeakMap() constructor

The **`WeakMap()`** constructor creates `WeakMap` objects.

Syntax

```
new WeakMap()
new WeakMap(iterable)
```

> **Note:** `WeakMap()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable`

An `Array` or other iterable object that produces a two-element array-like object whose first element is a value that will be used as a `WeakMap` key and whose second element is the value to associate with that key. Each key-value pair will be added to the new `WeakMap`. `null` is treated as `undefined`.

Examples

Using WeakMap

```
const wm1 = new WeakMap();
const wm2 = new WeakMap();
const wm3 = new WeakMap();
const o1 = {};
const o2 = () => {};
const o3 = window;

wm1.set(o1, 37);
wm1.set(o2, "azerty");
wm2.set(o1, o2); // a value can be anything, including an object or a function
wm2.set(o3, undefined);
wm2.set(wm1, wm2); // keys and values can be any objects. Even WeakMaps!

wm1.get(o2); // "azerty"
wm2.get(o2); // undefined, because there is no key for o2 on wm2
wm2.get(o3); // undefined, because that is the set value

wm1.has(o2); // true
wm2.has(o2); // false
wm2.has(o3); // true (even if the value itself is 'undefined')

wm3.set(o1, 37);
wm3.get(o1); // 37

wm1.has(o1); // true
wm1.delete(o1);
wm1.has(o1); // false
```

See also

- Polyfill of `WeakMap` in `core-js`
- `WeakMap` in the JavaScript guide
- `Map`
- `Set`
- `WeakSet`

WeakRef

A **`WeakRef`** object lets you hold a weak reference to another object, without preventing that object from getting garbage-collected.

Description

A `WeakRef` object contains a weak reference to an object, which is called its *target* or *referent*. A *weak reference* to an object is a reference that does not prevent the object from being reclaimed by the garbage collector. In contrast, a normal (or *strong*) reference keeps an object in memory. When an object no longer has any strong references to it, the JavaScript engine's garbage collector may destroy the object and reclaim its memory. If that happens, you can't get the object from a weak reference anymore.

Because non-registered symbols are also garbage collectable, they can also be used as the target of a `WeakRef` object. However, the use case of this is limited.

Avoid where possible

Correct use of `WeakRef` takes careful thought, and it's best avoided if possible. It's also important to avoid relying on any specific behaviors not guaranteed by the specification. When, how, and whether garbage collection occurs is down to the implementation of any given JavaScript engine. Any behavior you observe in one engine may be different in another engine, in another version of the same engine, or even in a slightly different situation with the same version of the same engine. Garbage collection is a hard problem that JavaScript engine implementers are constantly refining and improving their solutions to.

Here are some specific points included by the authors in the proposal that introduced `WeakRef`:

> Garbage collectors are complicated. If an application or library depends on GC cleaning up a WeakRef or calling a finalizer [cleanup callback] in a timely, predictable manner, it's likely to be disappointed: the cleanup may happen much later than expected, or not at all. Sources of variability include:
>
> - One object might be garbage-collected much sooner than another object, even if they become unreachable at the same time, e.g., due to generational collection.
> - Garbage collection work can be split up over time using incremental and concurrent techniques.
> - Various runtime heuristics can be used to balance memory usage, responsiveness.
> - The JavaScript engine may hold references to things which look like they are unreachable (e.g., in closures, or inline caches).
> - Different JavaScript engines may do these things differently, or the same engine may change its algorithms across versions.
> - Complex factors may lead to objects being held alive for unexpected amounts of time, such as use with certain APIs.

Notes on WeakRefs

- If your code has just created a `WeakRef` for a target object, or has gotten a target object from a `WeakRef`'s `deref` method, that target object will not be reclaimed until the end of the current JavaScript job (including any promise reaction jobs that run at the end of a script job). That is, you can only "see" an object get reclaimed between turns of the event loop. This is primarily to avoid making the behavior of any given JavaScript engine's garbage collector apparent in code — because if it were, people would write code relying on that behavior, which would break when the garbage collector's behavior changed. (Garbage collection is a hard problem; JavaScript engine implementers are constantly refining and improving how it works.)
- If multiple `WeakRef`s have the same target, they're consistent with one another. The result of calling `deref` on one of them will match the result of calling `deref` on another of them (in the same job), you won't get the target object from one of them but `undefined` from another.
- If the target of a `WeakRef` is also in a `FinalizationRegistry`, the `WeakRef`'s target is cleared at the same time or before any cleanup callback associated with the registry is called; if your cleanup callback calls `deref` on a `WeakRef` for the object, it will receive `undefined`.
- You cannot change the target of a `WeakRef`, it will always only ever be the original target object or `undefined` when that target has been reclaimed.
- A `WeakRef` might never return `undefined` from `deref`, even if nothing strongly holds the target, because the garbage collector may never decide to reclaim the object.

Constructor

`WeakRef()`

Creates a new `WeakRef` object.

Instance properties

These properties are defined on `WeakRef.prototype` and shared by all `WeakRef` instances.

`WeakRef.prototype.constructor` Optional

The constructor function that created the instance object. For `WeakRef` instances, the initial value is the `WeakRef` constructor.

> **Note:** This property is marked as "normative optional" in the specification, which means a conforming implementation may not expose the `constructor` property. This prevents arbitrary code from obtaining the `WeakRef` constructor and being able to observe garbage collection. However, all major engines do expose it by default.

`WeakRef.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"WeakRef"`. This property is used in `Object.prototype.toString()`.

Instance methods

`WeakRef.prototype.deref()`

Returns the `WeakRef` object's target object, or `undefined` if the target object has been reclaimed.

Examples

Using a WeakRef object

This example starts a counter shown in a DOM element, stopping when the element doesn't exist anymore:

```
class Counter {
  constructor(element) {
    // Remember a weak reference to the DOM element
    this.ref = new WeakRef(element);
    this.start();
  }

  start() {
    if (this.timer) {
      return;
    }

    this.count = 0;

    const tick = () => {
      // Get the element from the weak reference, if it still exists
      const element = this.ref.deref();
      if (element) {
        element.textContent = ++this.count;
      } else {
        // The element doesn't exist anymore
        console.log("The element is gone.");
        this.stop();
        this.ref = null;
      }
    };

    tick();
    this.timer = setInterval(tick, 1000);
  }

  stop() {
    if (this.timer) {
      clearInterval(this.timer);
      this.timer = 0;
    }
  }
}

const counter = new Counter(document.getElementById("counter"));
setTimeout(() => {
  document.getElementById("counter").remove();
}, 5000);
```

See also

- `FinalizationRegistry`
- `WeakSet`
- `WeakMap`

WeakRef.prototype.deref()

The **`deref()`** method of `WeakRef` instances returns this `WeakRef`'s target value, or `undefined` if the target value has been garbage-collected.

Syntax

```
deref()
```

Parameters

None.

Return value

The target value of the WeakRef, which is either an object or a non-registered symbol. Returns `undefined` if the value has been garbage-collected.

Description

See the Notes on WeakRefs section of the `WeakRef` page for some important notes.

Examples

Using deref()

See the Examples section of the `WeakRef` page for the complete example.

```
const tick = () => {
  // Get the element from the weak reference, if it still exists
  const element = this.ref.deref();
  if (element) {
    element.textContent = ++this.count;
  } else {
```

```
    // The element doesn't exist anymore
    console.log("The element is gone.");
    this.stop();
    this.ref = null;
  }
};
```

See also

- `WeakRef`

WeakRef() constructor

The **`WeakRef()`** constructor creates `WeakRef` objects.

Syntax

```
new WeakRef(target)
```

> **Note:** `WeakRef()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`target`

The target value the WeakRef should refer to (also called the *referent*). Must be an object or a non-registered symbol.

Return value

A new `WeakRef` object referring to the given target value.

Exceptions

`TypeError`

Thrown if `target` is not an object or a non-registered symbol.

Examples

Creating a new WeakRef object

See the main `WeakRef` page for a complete example.

```
class Counter {
  constructor(element) {
    // Remember a weak reference to a DOM element
    this.ref = new WeakRef(element);
    this.start();
  }
}
```

See also

- `WeakRef`

WeakSet

A **`WeakSet`** is a collection of garbage-collectable values, including objects and non-registered symbols. A value in the `WeakSet` may only occur once. It is unique in the `WeakSet`'s collection.

Description

Values of WeakSets must be garbage-collectable. Most primitive data types can be arbitrarily created and don't have a lifetime, so they cannot be stored. Objects and non-registered symbols can be stored because they are garbage-collectable.

The main differences to the `Set` object are:

- `WeakSet`s are collections of **objects and symbols only**. They cannot contain arbitrary values of any type, as `Set`s can.
- The `WeakSet` is *weak*, meaning references to objects in a `WeakSet` are held *weakly*. If no other references to a value stored in the `WeakSet` exist, those values can be garbage collected.

  > **Note:** This also means that there is no list of current values stored in the collection. `WeakSet`s are not enumerable.

Key equality

Like regular `Set`, value equality is based on the SameValueZero algorithm, which is the same as the `===` operator because `WeakSet` can only hold object and symbol values. This means that for object values, equality is based on object identity. They are compared by reference, not by value.

Constructor

`WeakSet()`

Creates a new `WeakSet` object.

Instance properties

These properties are defined on `WeakSet.prototype` and shared by all `WeakSet` instances.

`WeakSet.prototype.constructor`

The constructor function that created the instance object. For `WeakSet` instances, the initial value is the `WeakSet` constructor.

`WeakSet.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"WeakSet"`. This property is used in `Object.prototype.toString()`.

Instance methods

`WeakSet.prototype.add()`

Inserts the specified value into this set, if it is not already present.

`WeakSet.prototype.delete()`

Removes the specified value from this set, if it is in the set.

`WeakSet.prototype.has()`

Returns a boolean indicating whether the specified value exists in this `WeakSet` or not.

Examples

Using the WeakSet object

```
const ws = new WeakSet();
const foo = {};
const bar = {};

ws.add(foo);
ws.add(bar);

ws.has(foo); // true
ws.has(bar); // true

ws.delete(foo); // removes foo from the set
ws.has(foo); // false, foo has been removed
ws.has(bar); // true, bar is retained
```

Note that `foo !== bar`. While they are similar objects, *they are not* ***the same object***. And so they are both added to the set.

Detecting circular references

Functions that call themselves recursively need a way of guarding against circular data structures by tracking which objects have already been processed.

`WeakSet`s are ideal for this purpose:

```
// Execute a callback on everything stored inside an object
function execRecursively(fn, subject, _refs = new WeakSet()) {
  // Avoid infinite recursion
  if (_refs.has(subject)) {
    return;
  }

  fn(subject);
  if (typeof subject === "object" && subject) {
    _refs.add(subject);
    for (const key in subject) {
      execRecursively(fn, subject[key], _refs);
    }
    _refs.delete(subject);
  }
}

const foo = {
  foo: "Foo",
  bar: {
    bar: "Bar",
  },
};

foo.bar.baz = foo; // Circular reference!
execRecursively((obj) => console.log(obj), foo);
```

Here, a `WeakSet` is created on the first run, and passed along with every subsequent function call (using the internal `_refs` parameter).

The number of objects or their traversal order is immaterial, so a `WeakSet` is more suitable (and performant) than a `Set` for tracking object references, especially if a very large number of objects is involved.

See also

- Polyfill of `WeakSet` in `core-js`
- `Map`
- `Set`
- `WeakMap`

WeakSet.prototype.add()

The **`add()`** method of `WeakSet` instances inserts the specified value into this set, if it is not already present.

Try it

```
const weakset = new WeakSet();
const object = {};

weakset.add(object);
console.log(weakset.has(object));
// Expected output: true

try {
  weakset.add(1);
} catch (error) {
  console.log(error);
  // Expected output (Chrome): TypeError: Invalid value used in weak set
  // Expected output (Firefox): TypeError: WeakSet value must be an object, got 1
  // Expected output (Safari): TypeError: Attempted to add a non-object key to a WeakSet
}
```

Syntax

```
add(value)
```

Parameters

`value`

The value to add to the `WeakSet` object. Must be either an object or a non-registered symbol. Objects are compared by reference, not by value.

Return value

The `WeakSet` object.

Exceptions

`TypeError`

Thrown if `value` is not an object or a non-registered symbol.

Examples

Using add()

```
const ws = new WeakSet();

ws.add(window); // add the window object to the WeakSet

ws.has(window); // true

// WeakSet only takes objects as arguments
ws.add(1);
// results in "TypeError: Invalid value used in weak set" in Chrome
// and "TypeError: 1 is not a non-null object" in Firefox
```

See also

- `WeakSet`
- `WeakSet.prototype.delete()`
- `WeakSet.prototype.has()`

WeakSet.prototype.delete()

The **`delete()`** method of `WeakSet` instances removes the specified value from this set, if it is in the set.

Try it

```
const weakset = new WeakSet();
const object = {};

weakset.add(object);
```

```
console.log(weakset.has(object));
// Expected output: true

weakset.delete(object);

console.log(weakset.has(object));
// Expected output: false
```

Syntax

```
weakSetInstance.delete(value)
```

Parameters

`value`

The value to remove from the `WeakSet` object. Objects are compared by reference, not by value.

Return value

`true` if a value in the `WeakSet` object has been removed successfully. `false` if the value is not found in the `WeakSet`. Always returns `false` if `value` is not an object or a non-registered symbol.

Examples

Using delete()

```
const ws = new WeakSet();
const obj = {};

ws.add(window);

ws.delete(obj); // Returns false. No obj found to be deleted.
ws.delete(window); // Returns true. Successfully removed.

ws.has(window); // Returns false. The window is no longer present in the WeakSet.
```

See also

- `WeakSet`
- `WeakSet.prototype.add()`
- `WeakSet.prototype.has()`

WeakSet.prototype.has()

The **`has()`** method of `WeakSet` instances returns a boolean indicating whether the specified value exists in this `WeakSet` or not.

Try it

```
const weakset = new WeakSet();
const object1 = {};
const object2 = {};

weakset.add(object1);

console.log(weakset.has(object1));
// Expected output: true

console.log(weakset.has(object2));
// Expected output: false
```

Syntax

```
has(value)
```

Parameters

`value`

The value to test for presence in the `WeakSet` object. Objects are compared by reference, not by value.

Return value

Returns `true` if the specified value exists in the `WeakSet` object; otherwise `false`. Always returns `false` if `value` is not an object or a non-registered symbol.

Examples

Using has()

```
const ws = new WeakSet();
const obj = {};
ws.add(window);
```

```
ws.has(window); // returns true
ws.has(obj); // returns false

// Storing a non-registered symbol
const sym = Symbol("foo");
ws.add(sym);
ws.add(Symbol.iterator);
```

See also

- `WeakSet`
- `WeakSet.prototype.add()`
- `WeakSet.prototype.delete()`

WeakSet() constructor

The **`WeakSet()`** constructor creates `WeakSet` objects.

Syntax

```
new WeakSet()
new WeakSet(iterable)
```

> **Note:** `WeakSet()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable` Optional

If an iterable object is passed, all of its elements will be added to the new `WeakSet`. `null` is treated as `undefined`.

Examples

Using the WeakSet object

```
const ws = new WeakSet();
const foo = {};
const bar = {};

ws.add(foo);
ws.add(bar);

ws.has(foo); // true
ws.has(bar); // true

ws.delete(foo); // removes foo from the set
ws.has(foo); // false, foo has been removed
ws.has(bar); // true, bar is retained
```

Note that `foo !== bar`. While they are similar objects, *they are not **the same object***. And so they are both added to the set.

See also

- Polyfill of `WeakSet` in `core-js`
- `WeakSet`

Iteration protocols

Iteration protocols aren't new built-ins or syntax, but *protocols*. These protocols can be implemented by any object by following some conventions.

There are two protocols: The iterable protocol and the iterator protocol.

The iterable protocol

The iterable protocol allows JavaScript objects to define or customize their iteration behavior, such as what values are looped over in a `for...of` construct. Some built-in types are built-in iterables with a default iteration behavior, such as `Array` or `Map`, while other types (such as `Object`) are not.

In order to be **iterable**, an object must implement the **`[Symbol.iterator]()`** method, meaning that the object (or one of the objects up its prototype chain) must have a property with a `[Symbol.iterator]` key which is available via constant `Symbol.iterator`:

`[Symbol.iterator]()`

A zero-argument function that returns an object, conforming to the iterator protocol.

Whenever an object needs to be iterated (such as at the beginning of a `for...of` loop), its `[Symbol.iterator]()` method is called with no arguments, and the returned **iterator** is used to obtain the values to be iterated.

Note that when this zero-argument function is called, it is invoked as a method on the iterable object. Therefore inside of the function, the `this` keyword can be used to access the properties of the iterable object, to decide what to provide during the iteration.

This function can be an ordinary function, or it can be a generator function, so that when invoked, an iterator object is returned. Inside of this generator function, each entry can be provided by using `yield`.

The iterator protocol

The iterator protocol defines a standard way to produce a sequence of values (either finite or infinite), and potentially a return value when all values have been generated.

An object is an iterator when it implements a **`next()`** method with the following semantics:

`next()`

> A function that accepts zero or one argument and returns an object conforming to the `IteratorResult` interface (see below). If a non-object value gets returned (such as `false` or `undefined`) when a built-in language feature (such as `for...of`) is using the iterator, a `TypeError` (`"iterator.next() returned a non-object value"`) will be thrown.

All iterator protocol methods (`next()`, `return()`, and `throw()`) are expected to return an object implementing the `IteratorResult` interface. It must have the following properties:

`done` Optional

> A boolean that's `false` if the iterator was able to produce the next value in the sequence. (This is equivalent to not specifying the `done` property altogether.)
>
> Has the value `true` if the iterator has completed its sequence. In this case, `value` optionally specifies the return value of the iterator.

`value` Optional

> Any JavaScript value returned by the iterator. Can be omitted when `done` is `true`.

In practice, neither property is strictly required; if an object without either property is returned, it's effectively equivalent to `{ done: false, value: undefined }`.

If an iterator returns a result with `done: true`, any subsequent calls to `next()` are expected to return `done: true` as well, although this is not enforced on the language level.

The `next` method can receive a value which will be made available to the method body. No built-in language feature will pass any value. The value passed to the `next` method of generators will become the value of the corresponding `yield` expression.

Optionally, the iterator can also implement the **`return(value)`** and **`throw(exception)`** methods, which, when called, tells the iterator that the caller is done with iterating it and can perform any necessary cleanup (such as closing database connection).

`return(value)` Optional

> A function that accepts zero or one argument and returns an object conforming to the `IteratorResult` interface, typically with `value` equal to the `value` passed in and `done` equal to `true`. Calling this method tells the iterator that the caller does not intend to make any more `next()` calls and can perform any cleanup actions. When built-in language features call `return()` for cleanup, `value` is always `undefined`.

`throw(exception)` Optional

> A function that accepts zero or one argument and returns an object conforming to the `IteratorResult` interface, typically with `done` equal to `true`. Calling this method tells the iterator that the caller detects an error condition, and `exception` is typically an `Error` instance. No built-in language feature calls `throw()` for cleanup purposes — it's a special feature of generators for the symmetry of `return`/`throw`.

> **Note:** It is not possible to know reflectively (i.e., without actually calling `next()` and validating the returned result) whether a particular object implements the iterator protocol.

It is very easy to make an iterator also iterable: just implement a `[Symbol.iterator]()` method that returns `this`.

```
// Satisfies both the Iterator Protocol and Iterable
const myIterator = {
  next() {
    // …
  },
  [Symbol.iterator]() {
    return this;
  },
};
```

Such object is called an *iterable iterator*. Doing so allows an iterator to be consumed by the various syntaxes expecting iterables — therefore, it is seldom useful to implement the Iterator Protocol without also implementing Iterable. (In fact, almost all syntaxes and APIs expect *iterables*, not *iterators*.) The generator object is an example:

```
const generatorObject = (function* () {
  yield 1;
  yield 2;
  yield 3;
})();

console.log(typeof generatorObject.next);
// "function" — it has a next method (which returns the right result), so it's an iterator

console.log(typeof generatorObject[Symbol.iterator]);
```

```
// "function" — it has a [Symbol.iterator] method (which returns the right iterator), so it's an iterable

console.log(generatorObject[Symbol.iterator]() === generatorObject);
// true — its [Symbol.iterator] method returns itself (an iterator), so it's an iterable iterator
```

All built-in iterators inherit from `Iterator.prototype`, which implements the `[Symbol.iterator]()` method as returning `this`, so that built-in iterators are also iterable.

However, when possible, it's better for `iterable[Symbol.iterator]()` to return different iterators that always start from the beginning, like `Set.prototype[Symbol.iterator]()` does.

The async iterator and async iterable protocols

There are another pair of protocols used for async iteration, named **async iterator** and **async iterable** protocols. They have very similar interfaces compared to the iterable and iterator protocols, except that each return value from the calls to the iterator methods is wrapped in a promise.

An object implements the async iterable protocol when it implements the following methods:

`[Symbol.asyncIterator]()`

> A zero-argument function that returns an object, conforming to the async iterator protocol.

An object implements the async iterator protocol when it implements the following methods:

`next()`

> A function that accepts zero or one argument and returns a promise. The promise fulfills to an object conforming to the `IteratorResult` interface, and the properties have the same semantics as those of the sync iterator's.

`return(value)` Optional

> A function that accepts zero or one argument and returns a promise. The promise fulfills to an object conforming to the `IteratorResult` interface, and the properties have the same semantics as those of the sync iterator's.

`throw(exception)` Optional

> A function that accepts zero or one argument and returns a promise. The promise fulfills to an object conforming to the `IteratorResult` interface, and the properties have the same semantics as those of the sync iterator's.

Interactions between the language and iteration protocols

The language specifies APIs that either produce or consume iterables and iterators.

Built-in iterables

`String`, `Array`, `TypedArray`, `Map`, `Set`, and `Segments` (returned by `Intl.Segmenter.prototype.segment()`) are all built-in iterables, because each of their `prototype` objects implements a `[Symbol.iterator]()` method. In addition, the `arguments` object and some DOM collection types such as `NodeList` are also iterables. There is no object in the core JavaScript language that is async iterable. Some web APIs, such as `ReadableStream`, have the `Symbol.asyncIterator` method set by default.

Generator functions return generator objects, which are iterable iterators. Async generator functions return async generator objects, which are async iterable iterators.

The iterators returned from built-in iterables actually all inherit from a common class `Iterator`, which implements the aforementioned `[Symbol.iterator]() { return this; }` method, making them all iterable iterators. The `Iterator` class also provides additional helper methods in addition to the `next()` method required by the iterator protocol. You can inspect an iterator's prototype chain by logging it in a graphical console.

```
console.log([][Symbol.iterator]());

Array Iterator {}
  [[Prototype]]: Array Iterator     ==> This is the prototype shared by all array iterators
    next: ƒ next()
    Symbol(Symbol.toStringTag): "Array Iterator"
    [[Prototype]]: Object           ==> This is the prototype shared by all built-in iterators
      Symbol(Symbol.iterator): ƒ [Symbol.iterator]()
      [[Prototype]]: Object         ==> This is Object.prototype
```

Built-in APIs accepting iterables

There are many APIs that accept iterables. Some examples include:

- `Map()`
- `WeakMap()`
- `Set()`
- `WeakSet()`
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.race()`
- `Promise.any()`
- `Array.from()`
- `Object.groupBy()`
- `Map.groupBy()`

```
const myObj = {};
```

```
new WeakSet(
  (function* () {
    yield {};
    yield myObj;
    yield {};
  })(),
).has(myObj); // true
```

Syntaxes expecting iterables

Some statements and expressions expect iterables, for example the `for...of` loops, array and parameter spreading, `yield*`, and array destructuring:

```
for (const value of ["a", "b", "c"]) {
  console.log(value);
}
// "a"
// "b"
// "c"

console.log([..."abc"]); // ["a", "b", "c"]

function* gen() {
  yield* ["a", "b", "c"];
}

console.log(gen().next()); // { value: "a", done: false }

[a, b, c] = new Set(["a", "b", "c"]);
console.log(a); // "a"
```

When built-in syntaxes are iterating an iterator, and the last result's `done` is `false` (i.e., the iterator is able to produce more values) but no more values are needed, the `return` method will get called if present. This can happen, for example, if a `break` or `return` is encountered in a `for...of` loop, or if all identifiers are already bound in an array destructuring.

```
const obj = {
  [Symbol.iterator]() {
    let i = 0;
    return {
      next() {
        i++;
        console.log("Returning", i);
        if (i === 3) return { done: true, value: i };
        return { done: false, value: i };
      },
      return() {
        console.log("Closing");
        return { done: true };
      },
    };
  },
};

const [a] = obj;
// Returning 1
// Closing

const [b, c, d] = obj;
// Returning 1
// Returning 2
// Returning 3
// Already reached the end (the last call returned `done: true`),
// so `return` is not called
console.log([b, c, d]); // [1, 2, undefined]; the value associated with `done: true` is not reachable

for (const b of obj) {
  break;
}
// Returning 1
// Closing
```

The `for await...of` loop and `yield*` in async generator functions (but not sync generator functions) are the only ways to interact with async iterables. Using `for...of`, array spreading, etc. on an async iterable that's not also a sync iterable (i.e., it has `[Symbol.asyncIterator]()` but no `[Symbol.iterator]()`) will throw a TypeError: x is not iterable.

Error handling

Because iteration involves transferring control back and forth between the iterator and the consumer, error handling happens in both ways: how the consumer handles errors thrown by the iterator, and how the iterator handles errors thrown by the consumer. When you are using one of the built-in ways of iteration, the language may also throw errors because the iterable breaks certain invariants. We will describe how built-in syntaxes generate and handle errors, which can be used as a guideline for your own code if you are manually stepping the iterator.

Non-well-formed iterables

Errors may happen when acquiring the iterator from the iterable. The language invariant enforced here is that the iterable must produce a valid iterator:

- It has a callable `[Symbol.iterator]()` method.
- The `[Symbol.iterator]()` method returns an object.
- The object returned by `[Symbol.iterator]()` has a callable `next()` method.

When using built-in syntax to initiate iteration on a non-well-formed iterable, a TypeError is thrown.

```
const nonWellFormedIterable = { [Symbol.iterator]: 1 };
[...nonWellFormedIterable]; // TypeError: nonWellFormedIterable is not iterable
nonWellFormedIterable[Symbol.iterator] = () => 1;
[...nonWellFormedIterable]; // TypeError: [Symbol.iterator]() returned a non-object value
nonWellFormedIterable[Symbol.iterator] = () => ({});
[...nonWellFormedIterable]; // TypeError: nonWellFormedIterable[Symbol.iterator]().next is not a function
```

For async iterables, if its `[Symbol.asyncIterator]()` property has value `undefined` or `null`, JavaScript falls back to using the `[Symbol.iterator]` property instead (and wraps the resulting iterator into an async iterator by forwarding the methods). Otherwise, the `[Symbol.asyncIterator]` property must conform to the above invariants too.

This type of errors can be prevented by first validating the iterable before attempting to iterate it. However, it's fairly rare because usually you know the type of the object you are iterating over. If you are receiving this iterable from some other code, you should just let the error propagate to the caller so they know an invalid input was provided.

Errors during iteration

Most errors happen when stepping the iterator (calling `next()`). The language invariant enforced here is that the `next()` method must return an object (for async iterators, an object after awaiting). Otherwise, a TypeError is thrown.

If the invariant is broken or the `next()` method throws an error (for async iterators, it may also return a rejected promise), the error is propagated to the caller. For built-in syntaxes, the iteration in progress is aborted without retrying or cleanup (with the assumption that if the `next()` method threw the error, then it has cleaned up already). If you are manually calling `next()`, you may catch the error and retry calling `next()`, but in general you should assume the iterator is already closed.

If the caller decides to exit iteration for any reason other than the errors in the previous paragraph, such as when it enters an error state in its own code (for example, while handling an invalid value produced by the iterator), it should call the `return()` method on the iterator, if one exists. This allows the iterator to perform any cleanup. The `return()` method is only called for premature exits—if `next()` returns `done: true`, the `return()` method is not called, with the assumption that the iterator has already cleaned up.

The `return()` method might be invalid too! The language also enforces that the `return()` method must return an object and throws a TypeError otherwise. If the `return()` method throws an error, the error is propagated to the caller. However, if the `return()` method is called because the caller encountered an error in its own code, then this error overrides the error thrown by the `return()` method.

Usually, the caller implements error handling like this:

```
try {
  for (const value of iterable) {
    // …
  }
} catch (e) {
  // Handle the error
}
```

The `catch` will be able to catch errors thrown when `iterable` is not a valid iterable, when `next()` throws an error, when `return()` throws an error (if the `for` loop exits early), and when the `for` loop body throws an error.

Most iterators are implemented with generator functions, so we will demonstrate how generator functions typically handle errors:

```
function* gen() {
  try {
    yield doSomething();
    yield doSomethingElse();
  } finally {
    cleanup();
  }
}
```

The lack of `catch` here causes errors thrown by `doSomething()` or `doSomethingElse()` to propagate to the caller of `gen`. If these errors are caught within the generator function (which is equally advisable), the generator function can decide to continue yielding values or to exit early. However, the `finally` block is necessary for generators that keep open resources. The `finally` block is guaranteed to run, either when the last `next()` is called or when `return()` is called.

Forwarding errors

Some built-in syntaxes wrap an iterator into another iterator. They include the iterator produced by `Iterator.from()`, iterator helper methods (`map()`, `filter()`, `take()`, `drop()`, and `flatMap()`), `yield*`, and a hidden wrapper when you use async iteration (`for await...of`, `Array.fromAsync`) on sync iterators. The wrapped iterator is then responsible for forwarding errors between the inner iterator and the caller.

- Wrapper iterators generally directly forward the `next()` method of the inner iterator, including its return value and thrown errors. The async wrapper of a sync iterator awaits the returned object's `value` property and converts thrown errors into promise rejections. If a yielded promise rejects, the wrapper calls the inner iterator's `return()` method, if one exists, before rejecting its own promise. If the inner iterator's `next()` method itself throws, the wrapper rejects its promise without calling `return()`.
- Wrapper iterators generally directly forward the `return()` method of the inner iterator. If the `return()` method doesn't exist on the inner iterator, it returns `{ done: true, value: undefined }` instead. In the case of iterator helpers: if the iterator helper's `next()` method has not been called, after trying to call `return()` on

the inner iterator, the current iterator always returns `{ done: true, value: undefined }`. This is consistent with generator functions where execution hasn't entered the `yield*` expression yet.
- `yield*` is the only built-in syntax that forwards the `throw()` method of the inner iterator. For information on how `yield*` forwards the `return()` and `throw()` methods, see its own reference.

Examples

User-defined iterables

You can make your own iterables like this:

```
const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};

console.log([...myIterable]); // [1, 2, 3]
```

Basic iterator

Iterators are stateful by nature. If you don't define it as a generator function (as the example above shows), you would likely want to encapsulate the state in a closure.

```
function makeIterator(array) {
  let nextIndex = 0;
  return {
    next() {
      return nextIndex < array.length
        ? {
            value: array[nextIndex++],
            done: false,
          }
        : {
            done: true,
          };
    },
  };
}

const it = makeIterator(["yo", "ya"]);

console.log(it.next().value); // 'yo'
console.log(it.next().value); // 'ya'
console.log(it.next().done); // true
```

Infinite iterator

```
function idMaker() {
  let index = 0;
  return {
    next() {
      return {
        value: index++,
        done: false,
      };
    },
  };
}

const it = idMaker();

console.log(it.next().value); // 0
console.log(it.next().value); // 1
console.log(it.next().value); // 2
// …
```

Defining an iterable with a generator

```
function* makeGenerator(array) {
  let nextIndex = 0;
  while (nextIndex < array.length) {
    yield array[nextIndex++];
  }
}

const gen = makeGenerator(["yo", "ya"]);

console.log(gen.next().value); // 'yo'
console.log(gen.next().value); // 'ya'
console.log(gen.next().done); // true

function* idMaker() {
```

```
  let index = 0;
  while (true) {
    yield index++;
  }
}

const it = idMaker();

console.log(it.next().value); // 0
console.log(it.next().value); // 1
console.log(it.next().value); // 2
// …
```

Defining an iterable with a class

State encapsulation can be done with private fields as well.

```
class SimpleClass {
  #data;

  constructor(data) {
    this.#data = data;
  }

  [Symbol.iterator]() {
    // Use a new index for each iterator. This makes multiple
    // iterations over the iterable safe for non-trivial cases,
    // such as use of break or nested looping over the same iterable.
    let index = 0;

    return {
      // Note: using an arrow function allows `this` to point to the
      // one of `[Symbol.iterator]()` instead of `next()`
      next: () => {
        if (index >= this.#data.length) {
          return { done: true };
        }
        return { value: this.#data[index++], done: false };
      },
    };
  }
}

const simple = new SimpleClass([1, 2, 3, 4, 5]);

for (const val of simple) {
  console.log(val); // 1 2 3 4 5
}
```

Overriding built-in iterables

For example, a `String` is a built-in iterable object:

```
const someString = "hi";
console.log(typeof someString[Symbol.iterator]); // "function"
```

`String`'s default iterator returns the string's code points one by one:

```
const iterator = someString[Symbol.iterator]();
console.log(`${iterator}`); // "[object String Iterator]"

console.log(iterator.next()); // { value: "h", done: false }
console.log(iterator.next()); // { value: "i", done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

You can redefine the iteration behavior by supplying our own `[Symbol.iterator]()`:

```
// need to construct a String object explicitly to avoid auto-boxing
const someString = new String("hi");

someString[Symbol.iterator] = function () {
  return {
    // this is the iterator object, returning a single element (the string "bye")
    next() {
      return this._first
        ? { value: "bye", done: (this._first = false) }
        : { done: true };
    },
    _first: true,
  };
};
```

Notice how redefining `[Symbol.iterator]()` affects the behavior of built-in constructs that use the iteration protocol:

```
console.log([...someString]); // ["bye"]
console.log(`${someString}`); // "hi"
```

Concurrent modifications when iterating

Almost all iterables have the same underlying semantic: they don't copy the data at the time when iteration starts. Rather, they keep a pointer and move it around. Therefore, if you add, delete, or modify elements in the collection while iterating over the collection, you may inadvertently change whether other *unchanged* elements in the collection are visited. This is very similar to how iterative array methods work.

Consider the following case using a `URLSearchParams`:

```
const searchParams = new URLSearchParams(
  "deleteme1=value1&key2=value2&key3=value3",
);

// Delete unwanted keys
for (const [key, value] of searchParams) {
  console.log(key);
  if (key.startsWith("deleteme")) {
    searchParams.delete(key);
  }
}

// Output:
// deleteme1
// key3
```

Note how it never logs `key2`. This is because a `URLSearchParams` is underlyingly a list of key-value pairs. When `deleteme1` is visited and deleted, all other entries are shifted to the left by one, so `key2` occupies the position that `deleteme1` used to be in, and when the pointer moves to the next key, it lands on `key3`.

Certain iterable implementations avoid this problem by setting "tombstone" values to avoid shifting the remaining values. Consider the similar code using a `Map`:

```
const myMap = new Map([
  ["deleteme1", "value1"],
  ["key2", "value2"],
  ["key3", "value3"],
]);

for (const [key, value] of myMap) {
  console.log(key);
  if (key.startsWith("deleteme")) {
    myMap.delete(key);
  }
}

// Output:
// deleteme1
// key2
// key3
```

Note how it logs all keys. This is because `Map` doesn't shift the remaining keys when one is deleted. If you want to implement something similar, here's how it may look:

```
const tombstone = Symbol("tombstone");

class MyIterable {
  #data;
  constructor(data) {
    this.#data = data;
  }
  delete(deletedKey) {
    for (let i = 0; i < this.#data.length; i++) {
      if (this.#data[i][0] === deletedKey) {
        this.#data[i] = tombstone;
        return true;
      }
    }
    return false;
  }
  *[Symbol.iterator]() {
    for (const data of this.#data) {
      if (data !== tombstone) {
        yield data;
      }
    }
  }
}

const myIterable = new MyIterable([
  ["deleteme1", "value1"],
  ["key2", "value2"],
  ["key3", "value3"],
]);
for (const [key, value] of myIterable) {
  console.log(key);
```

```
  if (key.startsWith("deleteme")) {
    myIterable.delete(key);
  }
}
```

> **Warning:** Concurrent modifications, in general, are very bug-prone and confusing. Unless you know precisely how the iterable is implemented, it's best to avoid modifying the collection while iterating over it.

See also

- Iterators and generators guide
- `function*`
- `Symbol.iterator`
- `Iterator`

JavaScript technologies overview

Whereas HTML defines a webpage's structure and content and CSS sets the formatting and appearance, JavaScript adds interactivity to a webpage and creates rich web applications.

However, the umbrella term "JavaScript" as understood in a web browser context contains several very different elements. One of them is the core language (ECMAScript), another is the collection of the Web APIs, including the DOM (Document Object Model).

JavaScript, the core language (ECMAScript)

The core language of JavaScript is standardized by the ECMA TC39 committee as a language named ECMAScript. "ECMAScript" is the term for the language standard, but "ECMAScript" and "JavaScript" can be used interchangeably.

This core language is also used in non-browser environments, for example in Node.js.

What falls under the ECMAScript scope?

Among other things, ECMAScript defines:

- Language syntax (parsing rules, keywords, control flow, object literal initialization, ...)
- Error handling mechanisms (`throw`, `try...catch`, ability to create user-defined `Error` types)
- Types (boolean, number, string, function, object, ...)
- A prototype-based inheritance mechanism
- Built-in objects and functions, including `JSON`, `Math`, Array methods, `parseInt`, `decodeURI`, etc.
- Strict mode
- A module system
- Basic memory model

Standardization process

ECMAScript editions are approved and published as a standard by the ECMA General Assembly on a yearly basis. All development is public on the Ecma TC39 GitHub organization, which hosts proposals, the official specification text, and meeting notes.

Before the 6th edition of ECMAScript (known as ES6), specifications were published once every several years, and are commonly referred by their major version numbers — ES3, ES5, etc. After ES6, the specification is named by the publishing year — ES2017, ES2018, etc. ES6 is synonymous with ES2015. *ESNext* is a dynamic name that refers to whatever the next version is at the time of writing. ESNext features are more correctly called proposals, because, by definition, the specification has not been finalized yet.

The current committee-approved snapshot of ECMA-262 is available in PDF and HTML format on Ecma International's ECMA-262 language specification page. ECMA-262 and ECMA-402 are continuously maintained and kept up to date by the specification editors; the TC39 website hosts the latest, up-to-date ECMA-262 and ECMA-402 versions.

New language features, including introduction of new syntaxes and APIs and revision of existing behaviors, are discussed in the form of proposals. Each proposal goes through a 4-stage process, and is typically implemented by JavaScript engines at stage 3 or stage 4 and thus available for public consumption.

See Wikipedia ECMAScript entry for more information on ECMAScript history.

Internationalization API

The ECMAScript Internationalization API Specification is an addition to the ECMAScript Language Specification, also standardized by Ecma TC39. The internationalization API provides collation (string comparison), number formatting, and date-and-time formatting for JavaScript applications, letting the applications choose the language and tailor the functionality to their needs. The initial standard was approved in December 2012; the status of implementations in browsers is tracked in the documentation of the `Intl` object. The Internationalization specification is nowadays also ratified on a yearly basis and browsers constantly improve their implementation.

Related resources

There are a variety of ways you can participate in or just track current work on the ECMAScript Language Specification and the ECMAScript Internationalization API Specification and related resources:

- ECMAScript Language Specification repo
- ECMAScript Internationalization API Specification repo
- ECMAScript proposals repo
- ECMAScript conformance test suite repo

- TC39 meeting notes
- ECMAScript spec discussion; current mailing list
- ECMAScript spec discussion; historical mailing-list archives (until March 2021)

DOM APIs

WebIDL

The WebIDL specification provides the glue between the DOM technologies and ECMAScript.

The Core of the DOM

The Document Object Model (DOM) is a cross-platform, **language-independent convention** for representing and interacting with objects in HTML, XHTML and XML documents. Objects in the **DOM tree** may be addressed and manipulated by using methods on the objects. Nowadays, the DOM core spec is maintained by WHATWG (superseding the W3C version). It defines language-agnostic interfaces that abstract HTML and XML documents as objects, and also defines mechanisms to manipulate this abstraction. This includes: `Node`, `Element`, `DocumentFragment`, `Document`, `DOMImplementation`, `Event`, `EventTarget`, and more.

From the ECMAScript point of view, objects defined in the DOM specification are called "host objects".

HTML DOM

HTML, the Web's markup language, is specified in terms of the DOM. Layered above the abstract concepts defined in DOM Core, HTML also defines the *meaning* of elements. The HTML DOM includes such things as the `className` property on HTML elements, or APIs such as `Document.body`.

The HTML specification also defines restrictions on documents; for example, it requires all children of a `<ul>` element, which represents an unordered list, to be `<li>` elements, as those represent list items. In general, it also forbids using elements and attributes that aren't defined in a standard.

Looking for the `Document` object, `Window` object, and the other DOM elements? Read the DOM documentation.

Other notable APIs

- The `setTimeout()` and `setInterval()` functions were first specified on the `Window` interface in HTML Standard.
- XMLHttpRequest makes it possible to send asynchronous HTTP requests.
- The Fetch API provides a more ergonomic abstraction for network requests.
- The CSS Object Model abstract CSS rules as objects.
- WebWorkers allows parallel computation.
- WebSockets allows low-level bidirectional communication.
- Canvas 2D Context is a drawing API for `<canvas>`.
- The WebAssembly interface provides utilities for communication between JavaScript code and WebAssembly modules.

Non-browser environments (like Node.js) often do not have DOM APIs — because they don't interact with a document — but they still usually implement many web APIs, such as `fetch()` and `setTimeout()`.

JavaScript implementations

JavaScript engines used in current web browsers include:

- Mozilla's SpiderMonkey, used in Firefox, Servo, and Flow. Other non-browser usage includes MongoDB, CouchDB, and more. This was the first *ever* JavaScript engine, created by Brendan Eich at Netscape.
- Google's V8, used in Chrome and Chromium-based browsers such as Opera, Edge, and Brave. Other non-browser usage includes Node.js, Deno, Electron, and more.
- Apple's JavaScriptCore (also known as SquirrelFish/Nitro), used in Safari and other WebKit-based browsers. Other non-browser usage includes Bun.
- LibJS, used in Ladybird.

Some JavaScript engines used in earlier browsers include:

- Carakan, used in Opera before it became a Chromium-based browser.
- Microsoft's Chakra, used in Internet Explorer (although the language it implements is formally called "JScript" to avoid trademark issues). Earlier versions of Edge used another JavaScript engine, confusingly also called Chakra, before it became a Chromium-based browser.

Some JavaScript engines specifically tailored for non-browser purposes include:

- Engine262, written in JavaScript and intended essentially as a reference implementation of the language.
- Meta's Hermes, optimized for React Native.
- Mozilla's Rhino, written in Java.
- Oracle's GraalJS, written in Java and built on top of GraalVM.
- Moddable XS, intended for IoT/embedded systems.
- QuickJS, intended to be small and lightweight.

JavaScript engines expose a public API which application developers can use to integrate JavaScript into their software. By far, the most common host environment for JavaScript is web browsers. Web browsers typically use the public API to create **host objects** responsible for reflecting the DOM into JavaScript.

Another common application for JavaScript is as a (Web) server-side scripting language. A JavaScript web server exposes host objects representing an HTTP request and response objects, which can then be manipulated by a JavaScript program to dynamically generate web pages. Node.js is a popular example of this.

Shells

A JavaScript shell allows you to quickly test snippets of JavaScript code without having to reload a web page. They are extremely useful for developing and debugging code.

Standalone JavaScript shells

The following JavaScript shells are stand-alone environments, like Perl or Python.

- Node.js - Node.js is a platform for easily building fast, scalable network applications.
- ShellJS - Portable Unix shell commands for Node.js.

Browser-based JavaScript shells

The following JavaScript shells run code through the browser's JavaScript engine.

- Firefox has a built-in JavaScript console, which support multi-line editing.
- Babel REPL - A browser-based REPL for experimenting with future JavaScript.
- TypeScript playground — A browser-based playground for experimenting both new JavaScript features (via the tsc compiler) and TypeScript syntax.

Tools & resources

Helpful tools for writing and debugging your JavaScript code.

Firefox Developer Tools

> Web Console, JavaScript Profiler, Debugger, and more.

Learn JavaScript

> An excellent resource for aspiring web developers — Learn JavaScript in an interactive environment, with short lessons and interactive tests, guided by automated assessment. The first 40 lessons are free, and the complete course is available for a small one-time payment.

TogetherJS

> Collaboration made easy. By adding TogetherJS to your site, your users can help each other out on a website in real-time!

Stack Overflow

> Stack Overflow questions tagged with "JavaScript".

JSFiddle

> Edit JavaScript, CSS, and HTML and get live results. Use external resources and collaborate with your team online.

Plunker

> Plunker is an online community for creating, collaborating on, and sharing your web development ideas. Edit your JavaScript, CSS, and HTML files and get live results and file structure.

JS Bin

> JS Bin is an open-source collaborative web development debugging tool.

CodePen

> CodePen is another collaborative web development tool used as a live result playground.

StackBlitz

> StackBlitz is another online playground/debugging tool, which can host and deploy full-stack applications using React, Angular, etc.

RunJS

> RunJS is a desktop playground/scratchpad tool, which provides live results and access to both Node and Browser APIs.

Lexical grammar

This page describes JavaScript's lexical grammar. JavaScript source text is just a sequence of characters — in order for the interpreter to understand it, the string has to be *parsed* to a more structured representation. The initial step of parsing is called lexical analysis, in which the text gets scanned from left to right and is converted into a sequence of individual, atomic input elements. Some input elements are insignificant to the interpreter, and will be stripped after this step — they include white space and comments. The others, including identifiers, keywords, literals, and punctuators (mostly operators), will be used for further syntax analysis. Line terminators and multiline comments are also syntactically insignificant, but they guide the process for automatic semicolons insertion to make certain invalid token sequences become valid.

Format-control characters

Format-control characters have no visual representation but are used to control the interpretation of the text.

Code point	Name	Abbreviation	Description
U+200C	Zero width non-joiner	<ZWNJ>	Placed between characters to prevent being connected into ligatures in certain languages (Wikipedia).
U+200D	Zero width joiner	<ZWJ>	Placed between characters that would not normally be connected in order to cause the characters to be rendered using their connected form in certain languages (Wikipedia).
U+FEFF	Byte order mark	<BOM>	Used at the start of the script to mark it as Unicode and to allow detection of the text's encoding and byte order (Wikipedia).

In JavaScript source text, <ZWNJ> and <ZWJ> are treated as identifier parts, while <BOM> (also called a zero-width no-break space <ZWNBSP> when not at the start of text) is treated as white space.

White space

White space characters improve the readability of source text and separate tokens from each other. These characters are usually unnecessary for the functionality of the code. Minification tools are often used to remove whitespace in order to reduce the amount of data that needs to be transferred.

Code point	Name	Abbreviation	Description	Escape sequence
U+0009	Character tabulation	<TAB>	Horizontal tabulation	\t
U+000B	Line tabulation	<VT>	Vertical tabulation	\v
U+000C	Form feed	<FF>	Page breaking control character (Wikipedia).	\f
U+0020	Space	<SP>	Normal space	
U+00A0	No-break space	<NBSP>	Normal space, but no point at which a line may break	
U+FEFF	Zero-width no-break space	<ZWNBSP>	When not at the start of a script, the BOM marker is a normal whitespace character.	
Others	Other Unicode space characters	<USP>	Characters in the "Space_Separator" general category	

> **Note:** Of those characters with the "White_Space" property but are not in the "Space_Separator" general category, U+0009, U+000B, and U+000C are still treated as white space in JavaScript; U+0085 NEXT LINE has no special role; others become the set of line terminators.

> **Note:** Changes to the Unicode standard used by the JavaScript engine may affect programs' behavior. For example, ES2016 upgraded the reference Unicode standard from 5.1 to 8.0.0, which caused U+180E MONGOLIAN VOWEL SEPARATOR to be moved from the "Space_Separator" category to the "Format (Cf)" category, and made it a non-whitespace. Subsequently, the result of `"\u180E".trim().length` changed from `0` to `1`.

Line terminators

In addition to white space characters, line terminator characters are used to improve the readability of the source text. However, in some cases, line terminators can influence the execution of JavaScript code as there are a few places where they are forbidden. Line terminators also affect the process of automatic semicolon insertion.

Outside the context of lexical grammar, white space and line terminators are often conflated. For example, `String.prototype.trim()` removes all white space and line terminators from the beginning and end of a string. The `\s` character class escape in regular expressions matches all white space and line terminators.

Only the following Unicode code points are treated as line terminators in ECMAScript, other line breaking characters are treated as white space (for example, Next Line, NEL, U+0085 is considered as white space).

Code point	Name	Abbreviation	Description	Escape sequence
U+000A	Line Feed	<LF>	New line character in UNIX systems.	\n
U+000D	Carriage Return	<CR>	New line character in Commodore and early Mac systems.	\r
U+2028	Line Separator	<LS>	Wikipedia	
U+2029	Paragraph Separator	<PS>	Wikipedia	

Comments

Comments are used to add hints, notes, suggestions, or warnings to JavaScript code. This can make it easier to read and understand. They can also be used to disable code to prevent it from being executed; this can be a valuable debugging tool.

JavaScript has two long-standing ways to add comments to code: line comments and block comments. In addition, there's a special hashbang comment syntax.

Line comments

The first way is the `//` comment; this makes all text following it on the same line into a comment. For example:

```
function comment() {
  // This is a one line JavaScript comment
  console.log("Hello world!");
}
comment();
```

Block comments

The second way is the `/* */` style, which is much more flexible.

For example, you can use it on a single line:

```
function comment() {
  /* This is a one line JavaScript comment */
  console.log("Hello world!");
```

```
}
comment();
```

You can also make multiple-line comments, like this:

```
function comment() {
  /* This comment spans multiple lines. Notice
     that we don't need to end the comment until we're done. */
  console.log("Hello world!");
}
comment();
```

You can also use it in the middle of a line, if you wish, although this can make your code harder to read so it should be used with caution:

```
function comment(x) {
  console.log("Hello " + x /* insert the value of x */ + " !");
}
comment("world");
```

In addition, you can use it to disable code to prevent it from running, by wrapping code in a comment, like this:

```
function comment() {
  /* console.log("Hello world!"); */
}
comment();
```

In this case, the `console.log()` call is never issued, since it's inside a comment. Any number of lines of code can be disabled this way.

Block comments that contain at least one line terminator behave like line terminators in automatic semicolon insertion.

Hashbang comments

There's a special third comment syntax, the **hashbang comment**. A hashbang comment behaves exactly like a single line-only (`//`) comment, except that it begins with `#!` and **is only valid at the absolute start of a script or module**. Note also that no whitespace of any kind is permitted before the `#!`. The comment consists of all the characters after `#!` up to the end of the first line; only one such comment is permitted.

Hashbang comments in JavaScript resemble shebangs in Unix which provide the path to a specific JavaScript interpreter that you want to use to execute the script. Before the hashbang comment became standardized, it had already been de-facto implemented in non-browser hosts like Node.js, where it was stripped from the source text before being passed to the engine. An example is as follows:

```
#!/usr/bin/env node

console.log("Hello world");
```

The JavaScript interpreter will treat it as a normal comment — it only has semantic meaning to the shell if the script is directly run in a shell.

> **Warning:** If you want scripts to be runnable directly in a shell environment, encode them in UTF-8 without a BOM. Although a BOM will not cause any problems for code running in a browser — because it's stripped during UTF-8 decoding, before the source text is analyzed — a Unix/Linux shell will not recognize the hashbang if it's preceded by a BOM character.

You must only use the `#!` comment style to specify a JavaScript interpreter. In all other cases just use a `//` comment (or multiline comment).

Identifiers

An *identifier* is used to link a value with a name. Identifiers can be used in various places:

```
const decl = 1; // Variable declaration (may also be `let` or `var`)
function fn() {} // Function declaration
const obj = { key: "value" }; // Object keys
// Class declaration
class C {
  #priv = "value"; // Private field
}
lbl: console.log(1); // Label
```

In JavaScript, identifiers are commonly made of alphanumeric characters, underscores (_), and dollar signs ($). Identifiers are not allowed to start with numbers. However, JavaScript identifiers are not only limited to ASCII — many Unicode code points are allowed as well. Namely:

- Start characters can be any character in the ID_Start category plus _ and $.
- After the first character, you can use any character in the ID_Continue category, which contains U+200C (ZWNJ) and U+200D (ZWJ).

> **Note:** If, for some reason, you need to parse some JavaScript source yourself, do not assume all identifiers follow the pattern `/[A-Za-z_$][\w$]*/` (i.e., ASCII-only)! The range of identifiers can be described by the regex `/[$_\p{ID_Start}][$\p{ID_Continue}]*/u` (excluding unicode escape sequences).

In addition, JavaScript allows using Unicode escape sequences in the form of `\u0000` or `\u{000000}` in identifiers, which encode the same string value as the actual Unicode characters. For example, 你好 and `\u4f60\u597d` are the same identifiers:

```
const 你好 = "Hello";
console.log(\u4f60\u597d); // Hello
```

Not all places accept the full range of identifiers. Certain syntaxes, such as function declarations, function expressions, and variable declarations require using identifier names that are not reserved words.

```
function import() {} // Illegal: import is a reserved word.
```

Most notably, private elements and object properties allow reserved words.

```
const obj = { import: "value" }; // Legal despite `import` being reserved
class C {
  #import = "value";
}
```

Keywords

Keywords are tokens that look like identifiers but have special meanings in JavaScript. For example, the keyword `async` before a function declaration indicates that the function is asynchronous.

Some keywords are *reserved*, meaning that they cannot be used as an identifier for variable declarations, function declarations, etc. They are often called *reserved words*. A list of these reserved words is provided below. Not all keywords are reserved — for example, `async` can be used as an identifier anywhere. Some keywords are only *contextually reserved* — for example, `await` is only reserved within the body of an async function, and `let` is only reserved in strict mode code, or `const` and `let` declarations.

Identifiers are always compared by *string value*, so escape sequences are interpreted. For example, this is still a syntax error:

```
const els\u{65} = 1;
// `els\u{65}` encodes the same identifier as `else`
```

Reserved words

These keywords cannot be used as identifiers for variables, functions, classes, etc. anywhere in JavaScript source.

- `break`
- `case`
- `catch`
- `class`
- `const`
- `continue`
- `debugger`
- `default`
- `delete`
- `do`
- `else`
- `export`
- `extends`
- `false`
- `finally`
- `for`
- `function`
- `if`
- `import`
- `in`
- `instanceof`
- `new`
- `null`
- `return`
- `super`
- `switch`
- `this`
- `throw`
- `true`
- `try`
- `typeof`
- `var`
- `void`
- `while`
- `with`

The following are only reserved when they are found in strict mode code:

- `let` (also reserved in `const`, `let`, and class declarations)
- `static`
- `yield` (also reserved in generator function bodies)

The following are only reserved when they are found in module code or async function bodies:

- `await`

Future reserved words

The following are reserved as future keywords by the ECMAScript specification. They have no special functionality at present, but they might at some future time, so they cannot be used as identifiers.

These are always reserved:

- `enum`

The following are only reserved when they are found in strict mode code:

- `implements`
- `interface`
- `package`
- `private`
- `protected`
- `public`

Future reserved words in older standards

The following are reserved as future keywords by older ECMAScript specifications (ECMAScript 1 till 3).

- `abstract`
- `boolean`
- `byte`
- `char`
- `double`
- `final`
- `float`
- `goto`
- `int`
- `long`
- `native`
- `short`
- `synchronized`
- `throws`
- `transient`
- `volatile`

Identifiers with special meanings

A few identifiers have a special meaning in some contexts without being reserved words of any kind. They include:

- `arguments` (not a keyword, but cannot be declared as identifier in strict mode)
- `as` (`import * as ns from "mod"`)
- `async`
- `eval` (not a keyword, but cannot be declared as identifier in strict mode)
- `from` (`import x from "mod"`)
- `get`
- `of`
- `set`

Literals

> **Note:** This section discusses literals that are atomic tokens. Object literals and array literals are expressions that consist of a series of tokens.

Null literal

See also `null` for more information.

```
null
```

Boolean literal

See also boolean type for more information.

```
true
false
```

Numeric literals

The Number and BigInt types use numeric literals.

Decimal

```
1234567890
42
```

Decimal literals can start with a zero (`0`) followed by another decimal digit, but if all digits after the leading `0` are smaller than 8, the number is interpreted as an octal number. This is considered a legacy syntax, and number literals prefixed with `0`, whether interpreted as octal or decimal, cause a syntax error in strict mode — so, use the `0o` prefix instead.

```
0888 // 888 parsed as decimal
0777 // parsed as octal, 511 in decimal
```

Exponential

The decimal exponential literal is specified by the following format: `beN`; where `b` is a base number (integer or floating), followed by an `E` or `e` character (which serves as separator or *exponent indicator*) and `N`, which is *exponent* or *power* number – a signed integer.

```
0e-5   // 0
0e+5   // 0
5e1    // 50
175e-2 // 1.75
1e3    // 1000
1e-3   // 0.001
1E3    // 1000
```

Binary

Binary number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "B" (`0b` or `0B`). Any character after the `0b` that is not 0 or 1 will terminate the literal sequence.

```
0b10000000000000000000000000000000 // 2147483648
0b01111111100000000000000000000000 // 2139095040
0B00000000011111111111111111111111 // 8388607
```

Octal

Octal number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "O" (`0o` or `0O`). Any character after the `0o` that is outside the range (01234567) will terminate the literal sequence.

```
0O755 // 493
0o644 // 420
```

Hexadecimal

Hexadecimal number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "X" (`0x` or `0X`). Any character after the `0x` that is outside the range (0123456789ABCDEF) will terminate the literal sequence.

```
0xFFFFFFFFFFFFF // 4503599627370495
0xabcdef123456  // 188900967593046
0XA             // 10
```

BigInt literal

The BigInt type is a numeric primitive in JavaScript that can represent integers with arbitrary precision. BigInt literals are created by appending `n` to the end of an integer.

```
123456789123456789n     // 123456789123456789
0o777777777777n         // 68719476735
0x123456789ABCDEFn      // 81985529216486895
0b11101001010101010101n // 955733
```

BigInt literals cannot start with `0` to avoid confusion with legacy octal literals.

```
0755n; // SyntaxError: invalid BigInt syntax
```

For octal `BigInt` numbers, always use zero followed by the letter "o" (uppercase or lowercase):

```
0o755n;
```

For more information about `BigInt`, see also JavaScript data structures.

Numeric separators

To improve readability for numeric literals, underscores (_, `U+005F`) can be used as separators:

```
1_000_000_000_000
1_050.95
0b1010_0001_1000_0101
0o2_2_5_6
0xA0_B0_C0
1_000_000_000_000_000_000_000n
```

Note these limitations:

```
// More than one underscore in a row is not allowed
100__000; // SyntaxError

// Not allowed at the end of numeric literals
100_; // SyntaxError

// Can not be used after leading 0
0_1; // SyntaxError
```

String literals

A string literal is zero or more Unicode code points enclosed in single or double quotes. Unicode code points may also be represented by an escape sequence. All code points may appear literally in a string literal except for these code points:

- U+005C \ (backslash)
- U+000D <CR>
- U+000A <LF>
- The same kind of quote that begins the string literal

Any code points may appear in the form of an escape sequence. String literals evaluate to ECMAScript String values. When generating these String values Unicode code points are UTF-16 encoded.

```
'foo'
"bar"
```

The following subsections describe various escape sequences (`\` followed by one or more characters) available in string literals. Any escape sequence not listed below becomes an "identity escape" that becomes the code point itself. For example, `\z` is the same as `z`. There's a deprecated octal escape sequence syntax described in the Deprecated and obsolete features page. Many of these escape sequences are also valid in regular expressions — see Character escape.

Escape sequences

Special characters can be encoded using escape sequences:

Escape sequence	Unicode code point
`\0`	null character (U+0000 NULL)
`\'`	single quote (U+0027 APOSTROPHE)
`\"`	double quote (U+0022 QUOTATION MARK)
`\\`	backslash (U+005C REVERSE SOLIDUS)
`\n`	newline (U+000A LINE FEED; LF)
`\r`	carriage return (U+000D CARRIAGE RETURN; CR)
`\v`	vertical tab (U+000B LINE TABULATION)
`\t`	tab (U+0009 CHARACTER TABULATION)
`\b`	backspace (U+0008 BACKSPACE)
`\f`	form feed (U+000C FORM FEED)
`\` followed by a line terminator	empty string

The last escape sequence, `\` followed by a line terminator, is useful for splitting a string literal across multiple lines without changing its meaning.

```
const longString =
  "This is a very long string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable.";
```

Make sure there is no space or any other character after the backslash (except for a line break), otherwise it will not work. If the next line is indented, the extra spaces will also be present in the string's value.

You can also use the `+` operator to append multiple strings together, like this:

```
const longString =
  "This is a very long string which needs " +
  "to wrap across multiple lines because " +
  "otherwise my code is unreadable.";
```

Both of the above methods result in identical strings.

Hexadecimal escape sequences

Hexadecimal escape sequences consist of `\x` followed by exactly two hexadecimal digits representing a code unit or code point in the range 0x0000 to 0x00FF.

```
"\xA9"; // "©"
```

Unicode escape sequences

A Unicode escape sequence consists of exactly four hexadecimal digits following `\u`. It represents a code unit in the UTF-16 encoding. For code points U+0000 to U+FFFF, the code unit is equal to the code point. Code points U+10000 to U+10FFFF require two escape sequences representing the two code units (a surrogate pair) used to encode the character; the surrogate pair is distinct from the code point.

See also `String.fromCharCode()` and `String.prototype.charCodeAt()`.

```
"\u00A9"; // "©" (U+A9)
```

Unicode code point escapes

A Unicode code point escape consists of `\u{`, followed by a code point in hexadecimal base, followed by `}`. The value of the hexadecimal digits must be in the range 0 and 0x10FFFF inclusive. Code points in the range U+10000 to U+10FFFF do not need to be represented as a surrogate pair.

See also `String.fromCodePoint()` and `String.prototype.codePointAt()`.

```
"\u{2F804}"; // CJK COMPATIBILITY IDEOGRAPH-2F804 (U+2F804)

// the same character represented as a surrogate pair
"\uD87E\uDC04";
```

Regular expression literals

Regular expression literals are enclosed by two forward slashes (`/`). The lexer consumes all characters up to the next unescaped forward slash or the end of the line, unless the forward slash appears within a character class (`[]`). Some characters (namely, those that are identifier parts) can appear after the closing slash, denoting flags.

The lexical grammar is very lenient: not all regular expression literals that get identified as one token are valid regular expressions.

See also `RegExp` for more information.

```
/ab+c/g;
/[/]/;
```

A regular expression literal cannot start with two forward slashes (`//`), because that would be a line comment. To specify an empty regular expression, use `/(?:)/`.

Template literals

One template literal consists of several tokens: `` `xxx${ `` (template head), `}xxx${` (template middle), and `}xxx` `` (template tail) are individual tokens, while any expression may come between them.

See also template literals for more information.

```
`string text`;

`string text line 1
 string text line 2`;

`string text ${expression} string text`;

tag`string text ${expression} string text`;
```

Automatic semicolon insertion

Some JavaScript statements' syntax definitions require semicolons (`;`) at the end. They include:

- `var`, `let`, `const`, `using`, `await using`
- Expression statements
- `do...while`
- `continue`, `break`, `return`, `throw`
- `debugger`
- Class field declarations (public or private)
- `import`, `export`

However, to make the language more approachable and convenient, JavaScript is able to automatically insert semicolons when consuming the token stream, so that some invalid token sequences can be "fixed" to valid syntax. This step happens after the program text has been parsed to tokens according to the lexical grammar. There are three cases when semicolons are automatically inserted:

1. When a token not allowed by the grammar is encountered, and it's separated from the previous token by at least one line terminator (including a block comment that includes at least one line terminator), or the token is "}", then a semicolon is inserted before the token.

```
{ 1
2 } 3

// is transformed by ASI into:

{ 1
;2 ;} 3;

// Which is valid grammar encoding three statements,
// each consisting of a number literal
```

The ending ")" of `do...while` is taken care of as a special case by this rule as well.

```
do {
  // …
} while (condition) /* ; */ // ASI here
const a = 1
```

However, semicolons are not inserted if the semicolon would then become the separator in the `for` statement's head.

```
for (
  let a = 1 // No ASI here
  a < 10 // No ASI here
  a++
) {}
```

Semicolons are also never inserted as empty statements. For example, in the code below, if a semicolon is inserted after ")", then the code would be valid, with an empty statement as the `if` body and the `const` declaration being a separate statement. However, because automatically inserted semicolons cannot become empty statements, this causes a declaration to become the body of the `if` statement, which is not valid.

```
if (Math.random() > 0.5)
const x = 1 // SyntaxError: Unexpected token 'const'
```

2. When the end of the input stream of tokens is reached, and the parser is unable to parse the single input stream as a complete program, a semicolon is inserted at the end.

```
const a = 1 /* ; */ // ASI here
```

This rule is a complement to the previous rule, specifically for the case where there's no "offending token" but the end of input stream.

3. When the grammar forbids line terminators in some place but a line terminator is found, a semicolon is inserted. These places include:

- `expr <here> ++`, `expr <here> --`
- `continue <here> lbl`
- `break <here> lbl`
- `return <here> expr`
- `throw <here> expr`
- `yield <here> expr`
- `yield <here> * expr`
- `(param) <here> => {}`
- `async <here> function`, `async <here> prop()`, `async <here> function*`, `async <here> *prop()`, `async <here> (param) <here> => {}`
- `using <here> id`, `await <here> using <here> id`

Here `++` is not treated as a postfix operator applying to variable `b`, because a line terminator occurs between `b` and `++`.

```
a = b
++c

// is transformed by ASI into

a = b;
++c;
```

Here, the `return` statement returns `undefined`, and the `a + b` becomes an unreachable statement.

```
return
a + b

// is transformed by ASI into

return;
a + b;
```

Note that ASI would only be triggered if a line break separates tokens that would otherwise produce invalid syntax. If the next token can be parsed as part of a valid structure, semicolons would not be inserted. For example:

```
const a = 1
(1).toString()

const b = 1
[1, 2, 3].forEach(console.log)
```

Because `()` can be seen as a function call, it would usually not trigger ASI. Similarly, `[]` may be a member access. The code above is equivalent to:

```
const a = 1(1).toString();

const b = 1[1, 2, 3].forEach(console.log);
```

This happens to be valid syntax. `1[1, 2, 3]` is a property accessor with a comma-joined expression. Therefore, you would get errors like "1 is not a function" and "Cannot read properties of undefined (reading 'forEach')" when running the code.

Within classes, class fields and generator methods can be a pitfall as well.

```
class A {
  a = 1
  *gen() {}
}
```

It is seen as:

```
class A {
  a = 1 * gen() {}
}
```

And therefore will be a syntax error around `{`.

There are the following rules-of-thumb for dealing with ASI, if you want to enforce semicolon-less style:

- Write postfix `++` and `--` on the same line as their operands.

  ```
  const a = b
  ++
  console.log(a) // ReferenceError: Invalid left-hand side expression in prefix operation
  ```

  ```
  const a = b++
  console.log(a)
  ```

- The expressions after `return`, `throw`, or `yield` should be on the same line as the keyword.

  ```
  function foo() {
    return
      1 + 1 // Returns undefined; 1 + 1 is ignored
  }
  ```

  ```
  function foo() {
    return 1 + 1
  }

  function foo() {
    return (
      1 + 1
    )
  }
  ```

- Similarly, the label identifier after `break` or `continue` should be on the same line as the keyword.

  ```
  outerBlock: {
    innerBlock: {
      break
        outerBlock // SyntaxError: Illegal break statement
    }
  }
  ```

  ```
  outerBlock: {
    innerBlock: {
      break outerBlock
    }
  }
  ```

- The `=>` of an arrow function should be on the same line as the end of its parameters.

  ```
  const foo = (a, b)
    => a + b
  ```

  ```
  const foo = (a, b) =>
    a + b
  ```

- The `async` of async functions, methods, etc. cannot be directly followed by a line terminator.

  ```
  async
  function foo() {}
  ```

  ```
  async function
  foo() {}
  ```

- The `using` keyword in `using` and `await using` statements should be on the same line as the first identifier it declares.

  ```
  using
  resource = acquireResource()
  ```

```
using resource
  = acquireResource()
```

- If a line starts with one of `(`, `[`, `` ` ``, `+`, `-`, `/` (as in regex literals), prefix it with a semicolon, or end the previous line with a semicolon.

```
// The () may be merged with the previous line as a function call
(() => {
  // …
})()

// The [ may be merged with the previous line as a property access
[1, 2, 3].forEach(console.log)

// The ` may be merged with the previous line as a tagged template literal
`string text ${data}`.match(pattern).forEach(console.log)

// The + may be merged with the previous line as a binary + expression
+a.toString()

// The - may be merged with the previous line as a binary - expression
-a.toString()

// The / may be merged with the previous line as a division expression
/pattern/.exec(str).forEach(console.log)
```

```
;(() => {
  // …
})()
;[1, 2, 3].forEach(console.log)
;`string text ${data}`.match(pattern).forEach(console.log)
;+a.toString()
;-a.toString()
;/pattern/.exec(str).forEach(console.log)
```

- Class fields should preferably always be ended with semicolons — in addition to the previous rule (which includes a field declaration followed by a computed property, since the latter starts with `[`), semicolons are also required between a field declaration and a generator method.

```
class A {
  a = 1
  [b] = 2
  *gen() {} // Seen as a = 1[b] = 2 * gen() {}
}
```

```
class A {
  a = 1;
  [b] = 2;
  *gen() {}
}
```

See also

- Grammar and types guide
- Micro-feature from ES6, now in Firefox Aurora and Nightly: binary and octal numbers by Jeff Walden (2013)
- JavaScript character escape sequences by Mathias Bynens (2011)

Magic comments

Magic comments (also called **comment directives**, **annotations**, etc.) are special types of comments that are recognized by certain engines, bundlers, type checkers, debuggers, etc. (collectively referred to as *consumers*) to enable opt-in functionality. Because they're comments, consumers that don't understand them ignore them. This article introduces the various types of magic comments commonly seen in JavaScript source code and their intended consumers.

Usually, the presence or absence of magic comments doesn't change the program's runtime behavior—for example, they may optimize execution, enable/disable static checking, or provide additional metadata. This contrasts with *directives*, which are strings that do alter runtime behavior. Examples include the standard `"use strict"` directive to opt into strict mode and React's `"use server"` and `"use client"` directives that determine if a component is executed server-side or client-side.

To avoid collision with regular comments, magic comments are usually marked by *sigils*, such as a leading `#` or `@` character: `//# my-setting-name` or `//@ my-setting-name`. The exact syntax varies between comment types.

> **Note:** There's no universal name for this type of feature, and the "alternatives" we listed may have subtle differences: *pragmas* are usually placed at the beginning of the file and inform compilers or engines about the entire script; *annotations* may be placed anywhere and apply to a specific construct; *directives* are generic terms but may be ambiguous with the JavaScript `"use strict"` style convention. In our writing, we use all these words interchangeably.

Compile hint pragmas

Compile hint pragmas give JavaScript engine hints about how to compile the code ahead-of-time. Their interpretation depends on the engine.

They are specified in the WICG proposal Explicit JavaScript Compile Hints.

Eager compilation

The following pragma enables eager compilation of all functions in the current script.

```
//# allFunctionsCalledOnLoad
```

The syntax and placement requirements are as follows:

- This comment must be placed at the beginning of the script before any code or whitespace (only other comments, single-line or block, are allowed to precede it).
- The comment can be either a line comment or a block comment.
- There must be no space between `#` and `//` or `/*`.
- There can be any amount of whitespace before and after `allFunctionsCalledOnLoad`.

In the following example, the magic comment indicates that the functions in the file are likely to be called during page load:

```
//# allFunctionsCalledOnLoad
function init() {
  console.log("init");
}

init();
```

Without the hint, the engine may defer compilation of a function until it is called.

Eager compilation offers the following benefits:

- It avoids double parsing: During initialization, the engine already does "light parsing" to know where the function starts and ends, and it has to do another full parse when the function is called. When the function can be compiled eagerly, the engine does the full parse up front, without a separate round of light parsing.
- It can be parallelized: During execution, if a function call triggers lazy compilation, the compilation must block the main thread to preserve synchronicity. Script loading is asynchronous and therefore parsing can be more efficiently scheduled, and even happen as the script gets fetched.

However, compiling functions that are never called can waste time and memory. Like everything about performance, you must do benchmarks and weigh the different tradeoffs. The good heuristic is, as the pragma itself says: only enable this mode if the functions are *called on load*.

See also Faster JavaScript Startup with Explicit Compile Hints on v8.dev.

Source map annotations

Source map annotations associate JavaScript code with source files, making generated, evaluated, or minified code easier to debug.

They are specified in the TC39 specification ECMA-426 Source map format.

Source URLs

The following annotation gives a piece of code a URL identifier.

```
//# sourceURL=<url>
```

The syntax and placement requirements are as follows:

- This comment be must placed at the *end* of the script *after* any code (other single-line comments, whitespace, and line terminators are allowed to follow it).
- The comment must be a line comment.
- The sigil may be a `@` instead of `#`, but `#` is preferred (`//@` might conflict with Internet Explorer pragmas).
- There can be any amount of whitespace before and after `sourceURL=<url>`.
- The `<url>` must not contain whitespace characters (they should be percent-encoded as `%20`).

Source URLs are especially useful for code not originating from resources already associated with URLs, such as code executed with `eval()`:

```
eval(
  'console.log("Hello"); throw new Error("error");\n//# sourceURL=generated-code.js',
);
```

The above may generate the following output in the browser console:

```
Hello       generated-code.js:1
Uncaught Error: error
    <anonymous> generated-code.js:1
    <anonymous> debugger eval code:1
```

This annotation is used by many features and is primarily associated with debugging:

- Console/debugger output, as demonstrated above.
- The `Error` `stack` property.
- Resolving the `sourceMapURL`.

See also naming evaluated code with `sourceURL` on developer.chrome.com and Give your eval a name with `//@ sourceURL` on Firebug.

Source map URLs

The following annotation links generated code to a source map.

```
//# sourceMappingURL=<url>
```

The syntax and placement requirements are the same as those of `sourceURL`.

The source map URL can be a relative URL, in which case it can be resolved to the `sourceURL`, the `<script>` element's `src` attribute, the `<script>` element's containing document's origin, etc., as defined in ECMA 426. It can also be a `data:` URL containing an inline source map. The HTTP `SourceMap` header takes precedence over this comment.

A debugger can use the map to display the original source and map breakpoints to a minified bundle. This is also useful in editors for go-to-definition, finding references, etc.

Build tools (bundlers, transpilers, etc.) generate this comment in the compiled output along with the map. Regular developers usually do not need to hand-write them.

Bundler annotations

Bundlers, minifiers, and transpilers use magic comments to control code generation, optimization, and dependency handling. These annotations are consumed during the build, rather than by the JavaScript engine running the output.

There is no specification, and bundlers often invent their own annotations. This section only documents some annotations that are popular and somewhat widely supported. You must find if a bundler supports the particular annotation in its own documentation.

Tree shaking

The `/*#__PURE__*/` and `/*@__PURE__*/` annotations mark a particular function call or constructor invocation as safe to remove if the result is unused:

```
function createPoint(x, y) {
  return { x, y };
}

const point = /*#__PURE__*/ createPoint(1, 2);
```

> **Note:** This doesn't imply purity in a functional programming sense. The result might be random, depend on external state, etc. But as long as removing it doesn't change the application behavior in a way that matters, it can be annotated as pure.

Without the annotation, the bundler may need to keep the function call even if the result variable, `point`, isn't used:

```
// -- Compiler output --
function createPoint(x, y) {
  return { x, y };
}

createPoint(1, 2);
```

With the annotation, the bundler can remove the function call entirely, and if there's no other call to `createPoint()`, the function definition as well. An incorrect annotation can cause required behavior to be removed.

The annotation applies to the particular function call. The side-effects of evaluating the argument expressions are separately decided. For example:

```
/*#__PURE__*/ createPoint(1, getY());
```

In this example, the `createPoint()` call is pure, and bundlers know that evaluating `1` has no side-effects. However, it cannot determine if `getY()` is pure, so the output contains the `getY()` call (and the `createPoint()` call may be retained as well). To elide everything, each function call must be separately annotated:

```
/*#__PURE__*/ createPoint(1, /*#__PURE__*/ getY());
```

See also esbuild's pure annotations and Terser's annotations.

Some bundlers also recognize `/*#__NO_SIDE_EFFECTS__*/` and `/*@__NO_SIDE_EFFECTS__*/`, which annotate a function declaration, or a supported variable declaration containing a function, so calls to that function can be treated as side-effect-free:

```
/*#__NO_SIDE_EFFECTS__*/
function createPoint(x, y) {
  return { x, y };
}
```

See Rollup's tree-shaking annotations.

Minimization

There are two important minimization techniques which may be unsafe to apply: *inlining* and *property mangling*. Inlining directly replaces a function call with the function body, while property mangling replaces property names with shorter strings.

You can use `/*@__INLINE__*/` and `/*@__NOINLINE__*/` on particular function calls to opt-in or opt-out of inlining. Things to consider:

- Inlining may improve call performance because it avoids pushing/popping stack frames. However, the engine itself is capable of inlining and source-level inlining may change its decisions in unpredictable ways.
- Inlining may decrease the bundle size if the function is used only once, and increase it if the function is used many times. Minimizers usually make decisions that optimize bundle size.

During property mangling, the minimizer globally replaces property names with shorter strings (while keeping distinct property names distinct). This is unsafe because the object may be accessible outside (passed to external functions, returned from exported functions, etc.), so they are usually opt-in. If you opt into mangling, you

probably want to constrain mangling to name patterns known to be internal, such as names prefixed with underscores.

The `/*@__KEY__*/` annotation marks a string literal as a property name to rename. By default, the minimizer can only check for specific syntaxes, so strings passed to arbitrary functions must be explicitly marked.

```
const record = { _internalValue: 42 };
Object.getOwnPropertyDescriptor(record, /*@__KEY__*/ "_internalValue");
```

The `/*@__MANGLE_PROP__*/` annotation can explicitly enable mangling for a particular property or class field.

See also Terser Annotations.

JSX transformation

JSX transpilers use file-level pragmas to select how JSX is converted into JavaScript. For example:

```
/** @jsxRuntime automatic */
/** @jsxImportSource preact */

const heading = <h1>Hello</h1>;
```

This generates:

```
import { jsx as _jsx } from "preact/jsx-runtime";

const heading = _jsx("h1", {
  children: "Hello",
});
```

These settings can also be globally configured in the transpiler; pragmas are only needed if a particular file uses a different setting.

See also Babel's JSX transform and esbuild's JSX configuration.

Static checker directives

Static checkers and related development tools also interpret comments as configuration or metadata. Some examples include:

- **JSDoc**: JSDoc annotations such as `@type` and `@param` supply type information. They also power editor features such as pop-up descriptions.
- **TypeScript (`tsc`)**: `// @ts-check` and `// @ts-nocheck` enable or disable type checking for a JavaScript file. `// @ts-ignore` suppresses diagnostics on the next line; `// @ts-expect-error` also reports an error if there was no error to suppress.
- **ESLint**: Configuration comments such as `/* eslint no-console: "warn" */` configure rules, while `// eslint-disable-next-line no-console` suppresses a rule for the next line.
- **Prettier**: `// prettier-ignore` excludes the next syntax node from formatting.
- **Flow**: `// @flow` enables type checking for a file. Comment types, such as `/*: number */`, embed type syntax inside JavaScript comments.

Legacy conditional compilation

> **Warning:** This section talks about an IE-specific mechanism. Like IE itself, it is now obsolete, but you may continue to see it in old code, and it continues to influence language design—such as why `//# sourceMapURL` is favored over `//@ sourceMapURL`. It is kept for historical interest.

Internet Explorer's JScript engine supported *conditional compilation*, an obsolete, non-standard mechanism that could interpret code inside specially marked comments. Unlike optimization hints or debugging metadata, these comments could change which code executed.

The `@cc_on` statement enabled conditional compilation. The `@set` statement defined conditional compilation variables, and `@if`, `@elif`, `@else`, and `@end` selected code based on those variables. Predefined variables included `@_jscript_version`, which identified the JScript engine version.

```
var supportsConditionalCompilation = false;
/*@cc_on
  supportsConditionalCompilation = true;
@*/
```

In a supporting engine, the assignment inside the comment would execute. Other engines would treat the entire block as an ordinary comment, leaving the variable as `false`. A line-comment form, `//@cc_on`, also existed.

See also Microsoft's historical JScript Conditional Compilation draft.

Expressions and operators

This chapter documents all the JavaScript language operators, expressions and keywords.

Expressions and operators by category

For an alphabetical listing see the sidebar on the left.

Primary expressions

Basic keywords and general expressions in JavaScript. These expressions have the highest precedence (higher than operators).

`this`

The `this` keyword refers to a special property of an execution context.

Literals

Basic `null`, boolean, number, and string literals.

`[]`

Array initializer/literal syntax.

`{}`

Object initializer/literal syntax.

`function`

The `function` keyword defines a function expression.

`class`

The `class` keyword defines a class expression.

`function*`

The `function*` keyword defines a generator function expression.

`async function`

The `async function` defines an async function expression.

`async function*`

The `async function*` keywords define an async generator function expression.

`/ab+c/i`

Regular expression literal syntax.

`` `string` ``

Template literal syntax.

`( )`

Grouping operator.

Left-hand-side expressions

Left values are the destination of an assignment.

Property accessors

Member operators provide access to a property or method of an object (`object.property` and `object["property"]`).

`?.`

The optional chaining operator returns `undefined` instead of causing an error if a reference is nullish (`null` or `undefined`).

`new`

The `new` operator creates an instance of a constructor.

`new.target`

In constructors, `new.target` refers to the constructor that was invoked by `new`.

`import.meta`

An object exposing context-specific metadata to a JavaScript module.

`super`

The `super` keyword calls the parent constructor or allows accessing properties of the parent object.

`import()`

The `import()` syntax allows loading a module asynchronously and dynamically into a potentially non-module environment.

`import.defer()`

The `import.defer()` syntax loads a module dynamically, deferring synchronous evaluation until the returned namespace's properties are accessed.

`import.source()`

The `import.source()` syntax results in an object representing the module's compiled source, without loading its dependencies, linking it, or evaluating it.

Increment and decrement

Postfix/prefix increment and postfix/prefix decrement operators.

`A++`

: Postfix increment operator.

`A--`

: Postfix decrement operator.

`++A`

: Prefix increment operator.

`--A`

: Prefix decrement operator.

Unary operators

A unary operation is an operation with only one operand.

`delete`

: The `delete` operator deletes a property from an object.

`void`

: The `void` operator evaluates an expression and discards its return value.

`typeof`

: The `typeof` operator determines the type of a given object.

`+`

: The unary plus operator converts its operand to Number type.

`-`

: The unary negation operator converts its operand to Number type and then negates it.

`~`

: Bitwise NOT operator.

`!`

: Logical NOT operator.

`await`

: Pause and resume an async function and wait for the promise's fulfillment/rejection.

Arithmetic operators

Arithmetic operators take numerical values (either literals or variables) as their operands and return a single numerical value.

`**`

: Exponentiation operator.

`*`

: Multiplication operator.

`/`

: Division operator.

`%`

: Remainder operator.

`+` (Plus)

: Addition operator.

`-`

: Subtraction operator.

Relational operators

A comparison operator compares its operands and returns a boolean value based on whether the comparison is true.

`<` (Less than)

Less than operator.

`>` (Greater than)

Greater than operator.

`<=`

Less than or equal operator.

`>=`

Greater than or equal operator.

`instanceof`

The `instanceof` operator determines whether an object is an instance of another object.

`in`

The `in` operator determines whether an object has a given property.

> **Note:** `=>` is not an operator, but the notation for Arrow functions.

Equality operators

The result of evaluating an equality operator is always of type boolean based on whether the comparison is true.

`==`

Equality operator.

`!=`

Inequality operator.

`===`

Strict equality operator.

`!==`

Strict inequality operator.

Bitwise shift operators

Operations to shift all bits of the operand.

`<<`

Bitwise left shift operator.

`>>`

Bitwise right shift operator.

`>>>`

Bitwise unsigned right shift operator.

Binary bitwise operators

Bitwise operators treat their operands as a set of 32 bits (zeros and ones) and return standard JavaScript numerical values.

`&`

Bitwise AND.

`|`

Bitwise OR.

`^`

Bitwise XOR.

Binary logical operators

Logical operators implement boolean (logical) values and have short-circuiting behavior.

`&&`

Logical AND.

`||`

Logical OR.

`??`

Nullish Coalescing Operator.

Conditional (ternary) operator

`(condition ? ifTrue : ifFalse)`

The conditional operator returns one of two values based on the logical value of the condition.

Assignment operators

An assignment operator assigns a value to its left operand based on the value of its right operand.

`=`

Assignment operator.

`*=`

Multiplication assignment.

`/=`

Division assignment.

`%=`

Remainder assignment.

`+=`

Addition assignment.

`-=`

Subtraction assignment

`<<=`

Left shift assignment.

`>>=`

Right shift assignment.

`>>>=`

Unsigned right shift assignment.

`&=`

Bitwise AND assignment.

`^=`

Bitwise XOR assignment.

`|=`

Bitwise OR assignment.

`**=`

Exponentiation assignment.

`&&=`

Logical AND assignment.

`||=`

Logical OR assignment.

`??=`

Nullish coalescing assignment.

`[a, b] = arr, { a, b } = obj`

Destructuring allows you to assign the properties of an array or object to variables using syntax that looks similar to array or object literals.

Yield operators

`yield`

Pause and resume a generator function.

`yield*`

Delegate to another generator function or iterable object.

Spread syntax

`...obj`

Spread syntax allows an iterable, such as an array or string, to be expanded in places where zero or more arguments (for function calls) or elements (for array literals) are expected. In an object literal, the spread syntax enumerates the properties of an object and adds the key-value pairs to the object being created.

Comma operator

`,`

The comma operator allows multiple expressions to be evaluated in a single statement and returns the result of the last expression.

What are operators?

As the What are statements, declarations, and expressions? section explains, an expression is a fundamental building block that evaluates to a value. Statements, declarations, and expressions can all define specific slots where expressions are accepted. Where an expression contains slots for further nested expressions, the part(s) that are not slots are known as operators.

For example, the syntax for an addition expression is `expression + expression` (if you read the spec, the operands are called *AdditiveExpression* and *MultiplicativeExpression*, which are both subsets of *Expression*, but that's the spec's mechanism for defining precedence and associativity and is irrelevant for our purposes). Apart from the two expression slots, the code entity it introduces is just `+`: the *addition operator*. Similarly, the syntax for a yield expression is `yield expression`, so `yield` is known as the operator. In other words, each operator corresponds to an expression.

MDN also regards expressions without slots such as `null` as operators per the definition above, although we nearly always just refer to them as "syntax" or "expression".

An expression does not need to take a fixed number of slots. For example, the array literal expression, `[expression, expression, expression]`, can take an arbitrary number of expression slots. The `[,,]` part might be called an "operator". MDN avoids this usage, but you may see it in functional programming languages like Haskell.

The definition of operators gets fuzzier with certain other code entities: what if an expression has a slot that's not an expression, or a code entity combined with an expression does not make an expression? Do we still refer to that code entity as an operator?

- In the optional chaining expression `foo?.bar`, `foo` is an expression, but `bar` must be an identifier and is not evaluated to a value. Do we still regard `?.` as an operator?
- In the arrow function expression `arg => body`, `body` might be an expression (although it can also be a block body), and `arg` is just an argument list. Do we still regard `=>` as an operator?
- In the spread syntax `...foo`, `foo` is an expression, but the whole thing is not an expression because it does not evaluate to a value—it only makes sense in certain other expressions like function calls, array literals, and object literals. Do we still regard `...` as an operator?

The term "operator" is not precisely defined in JavaScript, so MDN does not give a definitive answer. Our approach is to group all these constructs under "Operators" but avoid formally referring to them as operators. Many useful concepts about operators, such as precedence, still apply to them regardless of their exact nature.

See also

- Operator precedence

Addition (+)

The **addition (+)** operator produces the sum of numeric operands or string concatenation.

Try it

```
console.log(2 + 2);
// Expected output: 4

console.log(2 + true);
// Expected output: 3

console.log("hello " + "everyone");
// Expected output: "hello everyone"

console.log(2001 + ": A Space Odyssey");
// Expected output: "2001: A Space Odyssey"
```

Syntax

```
x + y
```

Description

The + operator is overloaded for two distinct operations: numeric addition and string concatenation. When evaluating, it first coerces both operands to primitives. Then, the two operands' types are tested:

- If one side is a string, the other operand is also converted to a string and they are concatenated.
- If they are both BigInts, BigInt addition is performed. If one side is a BigInt but the other is not, a `TypeError` is thrown.

- Otherwise, both sides are converted to numbers, and numeric addition is performed.

String concatenation is often thought to be equivalent with template literals or `String.prototype.concat()`, but they are not. Addition coerces the expression to a *primitive*, which calls `valueOf()` in priority; on the other hand, template literals and `concat()` coerce the expression to a *string*, which calls `toString()` in priority. If the expression has a `[Symbol.toPrimitive]()` method, string concatenation calls it with `"default"` as hint, while template literals use `"string"`. This is important for objects that have different string and primitive representations — such as `Temporal`, whose objects' `valueOf()` methods all throw.

```
const t = Temporal.Now.instant();
"" + t; // Throws TypeError
`${t}`; // '2022-07-31T04:48:56.113918308Z'
"".concat(t); // '2022-07-31T04:48:56.113918308Z'
```

You are advised to not use `"" + x` to perform string coercion.

Examples

Addition using numbers

```
1 + 2; // 3
```

Other non-string, non-BigInt values are coerced to numbers:

```
true + 1; // 2
false + false; // 0
```

Addition using BigInts

```
1n + 2n; // 3n
```

You cannot mix BigInt and number operands in addition. `null`, `undefined`, and boolean values are coerced to numbers and are forbidden as well.

```
1n + 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 + 1n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Strings have priority over other types, so adding a string to a BigInt produces string concatenation rather than a `TypeError`.

```
"1" + 2n; // "12"
```

To do addition with a BigInt and a non-BigInt, convert either operand:

```
1n + BigInt(2); // 3n
Number(1n) + 2; // 3
```

Addition using strings

If one of the operands is a string, the other is converted to a string and they are concatenated:

```
"foo" + "bar"; // "foobar"
5 + "foo"; // "5foo"
"foo" + false; // "foofalse"
"2" + 2; // "22"
```

See also

- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

Addition assignment (+=)

The **addition assignment (+=)** operator performs addition (which is either numeric addition or string concatenation) on the two operands and assigns the result to the left operand.

Try it

```
let a = 2;
let b = "hello";

console.log((a += 3)); // Addition
// Expected output: 5

console.log((b += " world")); // Concatenation
// Expected output: "hello world"
```

Syntax

```
x += y
```

Description

`x += y` is equivalent to `x = x + y`, except that the expression `x` is only evaluated once.

Examples

Addition assignment using numbers

```
let bar = 5;
bar += 2; // 7
```

Other non-string, non-BigInt values are coerced to numbers:

```
let baz = true;
baz += 1; // 2
baz += false; // 2
```

Addition assignment using BigInts

```
let x = 1n;
x += 2n; // 3n

x += 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Addition assignment using strings

```
let foo = "foo";
foo += false; // "foofalse"
foo += "bar"; // "foofalsebar"

let bar = 5;
bar += "foo"; // "5foo"
```

See also

- Assignment operators in the JS guide
- Addition (+)

Assignment (=)

The **assignment (=)** operator is used to assign a value to a variable or property. The assignment expression itself has a value, which is the assigned value. This allows multiple assignments to be chained in order to assign a single value to multiple variables.

Try it

```
let x = 2;
const y = 3;

console.log(x);
// Expected output: 2

console.log((x = y + 1)); // 3 + 1
// Expected output: 4

console.log((x = x * y)); // 4 * 3
// Expected output: 12
```

Syntax

```
x = y
```

Parameters

`x`

A valid assignment target, including an identifier or a property accessor. It can also be a destructuring pattern.

`y`

An expression specifying the value to be assigned to `x`.

Return value

The value of `y`.

Exceptions

`ReferenceError`

Thrown in strict mode if assigning to an identifier that is not declared in the scope.

`TypeError`

Thrown in strict mode if assigning to a property that is not modifiable.

Description

The assignment operator is completely different from the equals (`=`) sign used as syntactic separators in other locations, which include:

- Initializers of `var`, `let`, and `const` declarations
- Default values of destructuring
- Default parameters
- Initializers of class fields

All these places accept an assignment expression on the right-hand side of the `=`, so if you have multiple equals signs chained together:

```
const x = y = 5;
```

This is equivalent to:

```
const x = (y = 5);
```

Which means `y` must be a pre-existing variable, and `x` is a newly declared `const` variable. `y` is assigned the value `5`, and `x` is initialized with the value of the `y = 5` expression, which is also `5`. If `y` is not a pre-existing variable, a global variable `y` is implicitly created in non-strict mode, or a `ReferenceError` is thrown in strict mode. To declare two variables within the same declaration, use:

```
const x = 5,
  y = 5;
```

Examples

Basic assignment and chaining

```
let x = 5;
let y = 10;
let z = 25;

x = y; // x is 10
x = y = z; // x, y and z are all 25
```

Value of assignment expressions

The assignment expression itself evaluates to the value of the right-hand side, so you can log the value and assign to a variable at the same time.

```
let x;
console.log(x); // undefined
console.log(x = 2); // 2
console.log(x); // 2
```

Unqualified identifier assignment

The global object sits at the top of the scope chain. When attempting to resolve a name to a value, the scope chain is searched. This means that properties on the global object are conveniently visible from every scope, without having to qualify the names with `globalThis.` or `window.` or `global.`.

Because the global object has a `String` property (`Object.hasOwn(globalThis, "String")`), you can use the following code:

```
function foo() {
  String("s"); // The function `String` is globally available
}
```

So the global object will ultimately be searched for unqualified identifiers. You don't have to type `globalThis.String`; you can just type the unqualified `String`. To make this feature more conceptually consistent, assignment to unqualified identifiers will assume you want to create a property with that name on the global object (with `globalThis.` omitted), if there is no variable of the same name declared in the scope chain.

```
foo = "f"; // In non-strict mode, assumes you want to create a property named `foo` on the global object
Object.hasOwn(globalThis, "foo"); // true
```

In strict mode, assignment to an unqualified identifier in strict mode will result in a `ReferenceError`, to avoid the accidental creation of properties on the global object.

Note that the implication of the above is that, contrary to popular misinformation, JavaScript does not have implicit or undeclared variables. It just conflates the global object with the global scope and allows omitting the global object qualifier during property creation.

Assignment with destructuring

The left-hand side can also be an assignment pattern. This allows assigning to multiple variables at once.

```
const result = /(a+)(b+)(c+)/.exec("aaabcc");
let a = "",
  b = "",
```

```
  c = "";
[, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

See also

- Assignment operators in the JS guide
- Destructuring

async function expression

The **`async function`** keywords can be used to define an async function inside an expression.

You can also define async functions using the `async function` declaration or the arrow syntax.

Syntax

```
async function (param0) {
  statements
}
async function (param0, param1) {
  statements
}
async function (param0, param1, /* …, */ paramN) {
  statements
}

async function name(param0) {
  statements
}
async function name(param0, param1) {
  statements
}
async function name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** An expression statement cannot begin with the keywords `async function` to avoid ambiguity with an `async function` declaration. The `async function` keywords only begin an expression when they appear in a context that cannot accept statements.

Parameters

`name` Optional

> The function name. Can be omitted, in which case the function is *anonymous*. The name is only local to the function body.

`paramN` Optional

> The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

> The statements which comprise the body of the function.

Description

An `async function` expression is very similar to, and has almost the same syntax as, an `async function` declaration. The main difference between an `async function` expression and an `async function` declaration is the *function name*, which can be omitted in `async function` expressions to create *anonymous* functions. An `async function` expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined, allowing you to mimic top-level await. See also the chapter about functions for more information.

Examples

Using async function expression

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

// async function expression assigned to a variable
const add = async function (x) {
  const a = await resolveAfter2Seconds(20);
  const b = await resolveAfter2Seconds(30);
  return x + a + b;
};
```

```
add(10).then((v) => {
  console.log(v); // prints 60 after 4 seconds.
});

// async function expression used as an IIFE
(async function (x) {
  const p1 = resolveAfter2Seconds(20);
  const p2 = resolveAfter2Seconds(30);
  return x + (await p1) + (await p2);
})(10).then((v) => {
  console.log(v); // prints 60 after 2 seconds.
});
```

Async IIFE

An `async` IIFE allows you to use `await` and `for...await` in contexts where top-level await is not available. Here we use an arrow function to define the IIFE, but `async function` expressions can also be used.

```
const getFileStream = async (url) => {
  // implementation
};

(async () => {
  const stream = await getFileStream("https://domain.name/path/file.ext");
  for await (const chunk of stream) {
    console.log({ chunk });
  }
})();
```

See also

- Functions guide
- Functions
- `async function`
- `AsyncFunction`
- `await`

await

The **`await`** operator is used to wait for a `Promise` and get its fulfillment value. It can only be used inside an async function or at the top level of a module.

Syntax

```
await expression
```

Parameters

`expression`

A `Promise`, a thenable object, or any value to wait for.

Return value

The fulfillment value of the promise or thenable object, or, if the expression is not thenable, the expression's own value.

Exceptions

Throws the rejection reason if the promise or thenable object is rejected.

Description

`await` is usually used to unwrap promises by passing a `Promise` as the `expression`. Using `await` pauses the execution of its surrounding `async` function until the promise is settled (that is, fulfilled or rejected). When execution resumes, the value of the `await` expression becomes that of the fulfilled promise.

If the promise is rejected, the `await` expression throws the rejected value. The function containing the `await` expression will appear in the stack trace of the error. Otherwise, if the rejected promise is not awaited or is immediately returned, the caller function will not appear in the stack trace.

The `expression` is resolved in the same way as `Promise.resolve()`: it's always converted to a native `Promise` and then awaited. If the `expression` is a:

- Native `Promise` (which means `expression` belongs to `Promise` or a subclass, and `expression.constructor === Promise`): The promise is directly used and awaited natively, without calling `then()`.
- Thenable object (including non-native promises, polyfill, proxy, child class, etc.): A new promise is constructed with the native `Promise()` constructor by calling the object's `then()` method and passing in a handler that calls the `resolve` callback.
- Non-thenable value: An already-fulfilled `Promise` is constructed and used.

Even when the used promise is already fulfilled, the async function's execution still pauses until the next tick. In the meantime, the caller of the async function resumes execution. See example below.

Because `await` is only valid inside async functions and modules, which themselves are asynchronous and return promises, the `await` expression never blocks the main thread and only defers execution of code that actually depends on the result, i.e., anything after the `await` expression.

Examples

Awaiting a promise to be fulfilled

If a `Promise` is passed to an `await` expression, it waits for the `Promise` to be fulfilled and returns the fulfilled value.

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function f1() {
  const x = await resolveAfter2Seconds(10);
  console.log(x); // 10
}

f1();
```

Thenable objects

Thenable objects are resolved just the same as actual `Promise` objects.

```
async function f2() {
  const thenable = {
    then(resolve) {
      resolve("resolved!");
    },
  };
  console.log(await thenable); // "resolved!"
}

f2();
```

They can also be rejected:

```
async function f2() {
  const thenable = {
    then(_, reject) {
      reject(new Error("rejected!"));
    },
  };
  await thenable; // Throws Error: rejected!
}

f2();
```

Conversion to promise

If the value is not a `Promise`, `await` converts the value to a resolved `Promise`, and waits for it. The awaited value's identity doesn't change as long as it doesn't have a `then` property that's callable.

```
async function f3() {
  const y = await 20;
  console.log(y); // 20

  const obj = {};
  console.log((await obj) === obj); // true
}

f3();
```

Handling rejected promises

If the `Promise` is rejected, the rejected value is thrown.

```
async function f4() {
  try {
    const z = await Promise.reject(new Error("rejected!"));
  } catch (e) {
    console.error(e); // Error: rejected!
  }
}

f4();
```

You can handle rejected promises without a `try` block by chaining a `catch()` handler before awaiting the promise.

```
const response = await promisedFunction().catch((err) => {
  console.error(err);
  return "default response";
```

```
});
// response will be "default response" if the promise is rejected
```

This is built on the assumption that `promisedFunction()` never synchronously throws an error, but always returns a rejected promise. This is the case for most properly-designed promise-based functions, which usually look like:

```
function promisedFunction() {
  // Immediately return a promise to minimize chance of an error being thrown
  return new Promise((resolve, reject) => {
    // do something async
  });
}
```

However, if `promisedFunction()` does throw an error synchronously, the error won't be caught by the `catch()` handler. In this case, the `try...catch` statement is necessary.

Top level await

You can use the `await` keyword on its own (outside of an async function) at the top level of a module. This means that modules with child modules that use `await` will wait for the child modules to execute before they themselves run, all while not blocking other child modules from loading.

Here is an example of a module using the Fetch API and specifying await within the `export` statement. Any modules that include this will wait for the fetch to resolve before running any code.

```
// fetch request
const colors = fetch("../data/colors.json").then((response) => response.json());

export default await colors;
```

Control flow effects of await

When an `await` is encountered in code (either in an async function or in a module), the awaited expression is executed, while all code that depends on the expression's value is paused. Control exits the function and returns to the caller. When the awaited expression's value is resolved, another microtask that continues the paused code gets scheduled. This happens even if the awaited value is an already-resolved promise or not a promise: execution doesn't return to the current function until all other already-scheduled microtasks are processed. For example, consider the following code:

```
async function foo(name) {
  console.log(name, "start");
  console.log(name, "middle");
  console.log(name, "end");
}

foo("First");
foo("Second");

// First start
// First middle
// First end
// Second start
// Second middle
// Second end
```

In this case, the function `foo` is synchronous in effect, because it doesn't contain any `await` expression. The three statements happen in the same tick. Therefore, the two function calls execute all statements in sequence. In promise terms, the function corresponds to:

```
function foo(name) {
  return new Promise((resolve) => {
    console.log(name, "start");
    console.log(name, "middle");
    console.log(name, "end");
    resolve();
  });
}
```

However, as soon as there's one `await`, the function becomes asynchronous, and execution of following statements is deferred to the next tick.

```
async function foo(name) {
  console.log(name, "start");
  await console.log(name, "middle");
  console.log(name, "end");
}

foo("First");
foo("Second");

// First start
// First middle
// Second start
// Second middle
// First end
// Second end
```

This corresponds to:

```
function foo(name) {
  return new Promise((resolve) => {
    console.log(name, "start");
    resolve(console.log(name, "middle"));
  }).then(() => {
    console.log(name, "end");
  });
}
```

The extra `then()` handler can be merged with the executor passed to the constructor because it's not waiting on any asynchronous operation. However, its existence splits the code into one additional microtask for each call to `foo`. These microtasks are scheduled and executed in an intertwined manner, which can both make your code slower and introduce unnecessary race conditions. Therefore, make sure to use `await` only when necessary (to unwrap promises into their values).

Microtasks are scheduled not only by promise resolution but by other web APIs as well, and they execute with the same priority. This example uses `queueMicrotask()` to demonstrate how the microtask queue is processed when each `await` expression is encountered.

```
let i = 0;

queueMicrotask(function test() {
  i++;
  console.log("microtask", i);
  if (i < 3) {
    queueMicrotask(test);
  }
});

(async () => {
  console.log("async function start");
  for (let i = 1; i < 3; i++) {
    await null;
    console.log("async function resume", i);
  }
  await null;
  console.log("async function end");
})();

queueMicrotask(() => {
  console.log("queueMicrotask() after calling async function");
});

console.log("script sync part end");

// Logs:
// async function start
// script sync part end
// microtask 1
// async function resume 1
// queueMicrotask() after calling async function
// microtask 2
// async function resume 2
// microtask 3
// async function end
```

In this example, the `test()` function is always called before the async function resumes, so the microtasks they each schedule are always executed in an intertwined fashion. On the other hand, because both `await` and `queueMicrotask()` schedule microtasks, the order of execution is always based on the order of scheduling. This is why the "queueMicrotask() after calling async function" log happens after the async function resumes for the first time.

Improving stack trace

Sometimes, the `await` is omitted when a promise is directly returned from an async function.

```
async function noAwait() {
  // Some actions...

  return /* await */ lastAsyncTask();
}
```

However, consider the case where `lastAsyncTask` asynchronously throws an error.

```
async function lastAsyncTask() {
  await null;
  throw new Error("failed");
}

async function noAwait() {
  return lastAsyncTask();
}

noAwait();
```

```
// Error: failed
//    at lastAsyncTask
```

Only `lastAsyncTask` appears in the stack trace, because the promise is rejected after it has already been returned from `noAwait` — in some sense, the promise is unrelated to `noAwait`. To improve the stack trace, you can use `await` to unwrap the promise, so that the exception gets thrown into the current function. The exception will then be immediately wrapped into a new rejected promise, but during error creation, the caller will appear in the stack trace.

```
async function lastAsyncTask() {
  await null;
  throw new Error("failed");
}

async function withAwait() {
  return await lastAsyncTask();
}

withAwait();

// Error: failed
//    at lastAsyncTask
//    at async withAwait
```

Contrary to some popular belief, `return await promise` is at least as fast as `return promise`, due to how the spec and engines optimize the resolution of native promises. There's a proposal to make `return promise` faster and you can also read about V8's optimization on async functions. Therefore, except for stylistic reasons, `return await` is almost always preferable.

See also

- `async function`
- `async function` expression
- `AsyncFunction`
- Top-level await on v8.dev (2019)
- typescript-eslint rule: `return-await`

Bitwise AND (&)

The **bitwise AND (`&`)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bits of both operands are `1`.

Try it

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a & b); // 00000000000000000000000000000001
// Expected output: 1
```

Syntax

```
x & y
```

Description

The `&` operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt AND if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number bitwise AND. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the operands' bit representations in two's complement. Each bit in the first operand is paired with the corresponding bit in the second operand: *first bit* to *first bit*, *second bit* to *second bit*, and so on. The operator is applied to each pair of bits, and the result is constructed bitwise.

The truth table for the AND operation is:

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 & 9 (base 10) = 00000000000000000000000000001000 (base 2) = 8 (base 10)
```

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `& -1` to truncate numbers to integers. Bitwise ANDing any number `x` with `-1` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise AND

```
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 & 9;
// 8  (00000000000000000000000000001000)

14n & 9n; // 8n
```

See also

- Bitwise operators in the JS guide
- Bitwise AND assignment (`&=`)

Bitwise AND assignment (&=)

The **bitwise AND assignment (`&=`)** operator performs bitwise AND on the two operands and assigns the result to the left operand.

Try it

```
let a = 5; // 00000000000000000000000000000101
a &= 3; // 00000000000000000000000000000011

console.log(a); // 00000000000000000000000000000001
// Expected output: 1
```

Syntax

```
x &= y
```

Description

`x &= y` is equivalent to `x = x & y`, except that the expression `x` is only evaluated once.

Examples

Using bitwise AND assignment

```
let a = 5;
// 5:     00000000000000000000000000000101
// 2:     00000000000000000000000000000010
a &= 2; // 0

let b = 5n;
b &= 2n; // 0n
```

See also

- Assignment operators in the JS guide
- Bitwise AND (`&`)

Bitwise NOT (~)

The **bitwise NOT (`~`)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bit of the operand is `0`, and a `0` otherwise.

Try it

```
const a = 5; // 00000000000000000000000000000101
const b = -3; // 11111111111111111111111111111101

console.log(~a); // 11111111111111111111111111111010
// Expected output: -6

console.log(~b); // 00000000000000000000000000000010
// Expected output: 2
```

Syntax

```
~x
```

Description

The ~ operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt NOT if the operand becomes a BigInt; otherwise, it converts the operand to a 32-bit integer and performs number bitwise NOT.

The operator operates on the operands' bit representations in two's complement. The operator is applied to each bit, and the result is constructed bitwise.

The truth table for the NOT operation is:

x	NOT x
0	1
1	0

```
 9 (base 10) = 00000000000000000000000000001001 (base 2)
               --------------------------------
~9 (base 10) = 11111111111111111111111111110110 (base 2) = -10 (base 10)
```

Bitwise NOTing any 32-bit integer `x` yields `-(x + 1)`. For example, `~-5` yields `4`.

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `~~` to truncate numbers to integers. Bitwise NOTing any number `x` twice returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise NOT

```
~0; // -1
~-1; // 0
~1; // -2

~0n; // -1n
~4294967295n; // -4294967296n
```

See also

- Bitwise operators in the JS guide

Bitwise OR (|)

The **bitwise OR (|)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bits of either or both operands are `1`.

Try it

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a | b); // 00000000000000000000000000000111
// Expected output: 7
```

Syntax

```
x | y
```

Description

The | operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt OR if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number bitwise OR. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the operands' bit representations in two's complement. Each bit in the first operand is paired with the corresponding bit in the second operand: *first bit* to *first bit*, *second bit* to *second bit*, and so on. The operator is applied to each pair of bits, and the result is constructed bitwise.

The truth table for the OR operation is:

x	y	x OR y
0	0	0

x	y	x OR y
0	1	1
1	0	1
1	1	1

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 | 9 (base 10) = 00000000000000000000000000001111 (base 2) = 15 (base 10)
```

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `| 0` to truncate numbers to integers. Bitwise ORing any number `x` with `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise OR

```
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 | 9;
// 15 (00000000000000000000000000001111)

14n | 9n; // 15n
```

See also

- Bitwise operators in the JS guide
- Bitwise OR assignment (`|=`)

Bitwise OR assignment (|=)

The **bitwise OR assignment (`|=`)** operator performs bitwise OR on the two operands and assigns the result to the left operand.

Try it

```
let a = 5; // 00000000000000000000000000000101
a |= 3; // 00000000000000000000000000000011

console.log(a); // 00000000000000000000000000000111
// Expected output: 7
```

Syntax

```
x |= y
```

Description

`x |= y` is equivalent to `x = x | y`, except that the expression `x` is only evaluated once.

Examples

Using bitwise OR assignment

```
let a = 5;
a |= 2; // 7
// 5: 00000000000000000000000000000101
// 2: 00000000000000000000000000000010
// ----------------------------------
// 7: 00000000000000000000000000000111

let b = 5n;
b |= 2n; // 7n
```

See also

- Assignment operators in the JS guide
- Bitwise OR (`|`)

- Logical OR assignment (||=)

Bitwise XOR (^)

The **bitwise XOR (^)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bits of either but not both operands are `1`.

Try it

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a ^ b); // 00000000000000000000000000000110
// Expected output: 6
```

Syntax

```
x ^ y
```

Description

The `^` operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt XOR if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number bitwise XOR. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the operands' bit representations in two's complement. Each bit in the first operand is paired with the corresponding bit in the second operand: *first bit* to *first bit*, *second bit* to *second bit*, and so on. The operator is applied to each pair of bits, and the result is constructed bitwise.

The truth table for the XOR operation is:

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 ^ 9 (base 10) = 00000000000000000000000000000111 (base 2) = 7 (base 10)
```

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `^ 0` to truncate numbers to integers. Bitwise XORing any number `x` with `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise XOR

```
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 ^ 9;
// 7  (00000000000000000000000000000111)

14n ^ 9n; // 7n
```

See also

- Bitwise operators in the JS guide
- Bitwise XOR assignment (^=)

Bitwise XOR assignment (^=)

The **bitwise XOR assignment (^=)** operator performs bitwise XOR on the two operands and assigns the result to the left operand.

Try it

```
let a = 5; // 00000000000000000000000000000101
a ^= 3; // 00000000000000000000000000000011

console.log(a); // 00000000000000000000000000000110
// Expected output: 6
```

Syntax

```
x ^= y
```

Description

`x ^= y` is equivalent to `x = x ^ y`, except that the expression `x` is only evaluated once.

Examples

Using bitwise XOR assignment

```
let a = 5; // (00000000000000000000000000000101)
a ^= 3; // (00000000000000000000000000000011)

console.log(a); // 6 (00000000000000000000000000000110)

let b = 5; // (00000000000000000000000000000101)
b ^= 0; // (00000000000000000000000000000000)

console.log(b); // 5 (00000000000000000000000000000101)

let c = 5n;
c ^= 3n;
console.log(c); // 6n
```

See also

- Assignment operators in the JS guide
- Bitwise XOR (^)

class expression

The **`class`** keyword can be used to define a class inside an expression.

You can also define classes using the `class` declaration.

Try it

```
const Rectangle = class {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
  area() {
    return this.height * this.width;
  }
};

console.log(new Rectangle(5, 8).area());
// Expected output: 40
```

Syntax

```
class {
  // class body
}
class name {
  // class body
}
```

> **Note:** An expression statement cannot begin with the keyword `class` to avoid ambiguity with a `class` declaration. The `class` keyword only begins an expression when it appears in a context that cannot accept statements.

Description

A `class` expression is very similar to, and has almost the same syntax as, a `class` declaration. As with `class` declarations, the body of a `class` expression is executed in strict mode. The main difference between a `class` expression and a `class` declaration is the *class name*, which can be omitted in `class` expressions to create *anonymous* classes. Class expressions allow you to redefine classes, while redeclaring a class using `class` declarations throws a `SyntaxError`. See also the chapter about classes for more information.

Examples

A basic class expression

This is just an anonymous class expression which you can refer to using the variable `Foo`.

```
const Foo = class {
  constructor() {}
  bar() {
    return "Hello World!";
  }
};

const instance = new Foo();
instance.bar(); // "Hello World!"
Foo.name; // "Foo"
```

Named class expressions

If you want to refer to the current class inside the class body, you can create a *named class expression*. The name is only visible within the scope of the class expression itself.

```
const Foo = class NamedFoo {
  constructor() {}
  whoIsThere() {
    return NamedFoo.name;
  }
};
const bar = new Foo();
bar.whoIsThere(); // "NamedFoo"
NamedFoo.name; // ReferenceError: NamedFoo is not defined
Foo.name; // "NamedFoo"
```

See also

- `class`
- Classes

Comma operator (,)

The **comma (,)** operator evaluates each of its operands (from left to right) and returns the value of the last operand. This is commonly used to provide multiple updaters to a `for` loop's afterthought.

Try it

```
let x = 1;

x = (x++, x);

console.log(x);
// Expected output: 2

x = (2, 3);

console.log(x);
// Expected output: 3
```

Syntax

```
expr1, expr2, expr3/* , … */
```

Parameters

`expr1, expr2, expr3, ...`

> One or more expressions, the last of which is returned as the value of the compound expression.

Description

You can use the comma operator when you want to include multiple expressions in a location that requires a single expression. The most common usage of this operator is to supply multiple updaters in a `for` loop. For an idiom allowing multiple *statements* in a location that requires a single expression, you may use an IIFE.

Because all expressions except the last are evaluated and then discarded, these expressions must have side effects to be useful. Common expressions that have side effects are assignments, function calls, and `++` and `--` operators. Others may also have side effects if they invoke getters or trigger type coercions.

The comma operator has the lowest precedence of all operators. If you want to incorporate a comma-joined expression into a bigger expression, you must parenthesize it.

The comma operator is completely different from commas used as syntactic separators in other locations, which include:

- Elements in array initializers (`[1, 2, 3]`)
- Properties in object initializers (`{ a: 1, b: 2 }`)
- Parameters in function declarations/expressions (`function f(a, b) { … }`)
- Arguments in function calls (`f(1, 2)`)
- Binding lists in `let`, `const`, or `var` declarations (`const a = 1, b = 2;`)
- Import lists in `import` declarations (`import { a, b } from "c";`)

- Export lists in `export` declarations (`export { a, b };`)

In fact, although some of these places accept almost all expressions, they don't accept comma-joined expressions because that would be ambiguous with the syntactic comma separators. In this case, you must parenthesize the comma-joined expression. For example, the following is a `const` declaration that declares two variables, where the comma is not the comma operator:

```
const a = 1, b = 2;
```

It is different from the following, where `b = 2` is an assignment expression, not a declaration. The value of `a` is `2`, the return value of the assignment, while the value of `1` is discarded:

```
const a = (1, b = 2);
```

Comma operators cannot appear as trailing commas.

Examples

Using the comma operator in a for loop

If `a` is a 2-dimensional array with 10 elements on each side, the following code uses the comma operator to increment `i` and decrement `j` at once, thus printing the values of the diagonal elements in the array:

```
const a = Array.from({ length: 10 }, () =>
  Array.from({ length: 10 }, Math.random),
); // A 10×10 array of random numbers

for (let i = 0, j = 9; i <= 9; i++, j--) {
  console.log(`a[${i}][${j}] = ${a[i][j]}`);
}
```

Using the comma operator to join assignments

Because commas have the lowest precedence — even lower than assignment — commas can be used to join multiple assignment expressions. In the following example, `a` is set to the value of `b = 3` (which is 3). Then, the `c = 4` expression evaluates and its result becomes the return value of the entire comma expression.

```
let a, b, c;

a = b = 3, c = 4; // Returns 4
console.log(a); // 3 (left-most)

let x, y, z;

x = (y = 5, z = 6); // Returns 6
console.log(x); // 6 (right-most)
```

Processing and then returning

Another example that one could make with the comma operator is processing before returning. As stated, only the last element will be returned but all others are going to be evaluated as well. So, one could do:

```
function myFunc() {
  let x = 0;

  return (x += 1, x); // the same as return ++x;
}
```

This is especially useful for one-line arrow functions. The following example uses a single `map()` to get both the sum of an array and the squares of its elements, which would otherwise require two iterations, one with `reduce()` and one with `map()`:

```
let sum = 0;
const squares = [1, 2, 3, 4, 5].map((x) => ((sum += x), x * x));
console.log(squares); // [1, 4, 9, 16, 25]
console.log(sum); // 15
```

Discarding reference binding

The comma operator always returns the last expression as a *value* instead of a *reference*. This causes some contextual information such as the `this` binding to be lost. For example, a property access returns a reference to the function, which also remembers the object that it's accessed on, so that calling the property works properly. If the method is returned from a comma expression, then the function is called as if it's a new function value, and `this` is `undefined`.

```
const obj = {
  value: "obj",
  method() {
    console.log(this.value);
  },
};

obj.method(); // "obj"
(obj.method)(); // "obj" (the grouping operator still returns the reference)
(0, obj.method)(); // undefined (the comma operator returns a new value)
```

You can enter indirect eval with this technique, because direct eval requires the function call to happen on the reference to the `eval()` function.

```
globalThis.isDirectEval = false;

{
  const isDirectEval = true;
  console.log(eval("isDirectEval")); // true
  console.log((eval)("isDirectEval")); // true (the grouping operator still returns a reference to `eval`)
  console.log((0, eval)("isDirectEval")); // false (the comma operator returns a new value)
}
```

See also

- `for`
- IIFE

Conditional (ternary) operator

The **conditional (ternary) operator** is the only JavaScript operator that takes three operands: a condition followed by a question mark (`?`), then an expression to execute if the condition is truthy followed by a colon (`:`), and finally the expression to execute if the condition is falsy. This operator is frequently used as an alternative to an `if...else` statement.

Try it

```
function getFee(isMember) {
  return isMember ? "$2.00" : "$10.00";
}

console.log(getFee(true));
// Expected output: "$2.00"

console.log(getFee(false));
// Expected output: "$10.00"

console.log(getFee(null));
// Expected output: "$10.00"
```

Syntax

```
condition ? exprIfTrue : exprIfFalse
```

Parameters

`condition`

An expression whose value is used as a condition.

`exprIfTrue`

An expression which is executed if the `condition` evaluates to a truthy value (one which equals or can be converted to `true`).

`exprIfFalse`

An expression which is executed if the `condition` is falsy (that is, has a value which can be converted to `false`).

Description

Besides `false`, possible falsy expressions are: `null`, `NaN`, `0`, the empty string (`""`), and `undefined`. If `condition` is any of these, the result of the conditional expression will be the result of executing the expression `exprIfFalse`.

Examples

A basic example

```
const age = 26;
const beverage = age >= 21 ? "Beer" : "Juice";
console.log(beverage); // "Beer"
```

Handling null values

One common usage is to handle a value that may be `null`:

```
const greeting = (person) => {
  const name = person ? person.name : "stranger";
  return `Howdy, ${name}`;
};

console.log(greeting({ name: "Alice" })); // "Howdy, Alice"
console.log(greeting(null)); // "Howdy, stranger"
```

Conditional chains

The ternary operator is right-associative, which means it can be "chained" in the following way, similar to an `if … else if … else if … else` chain:

```
function example() {
  return condition1 ? value1
    : condition2 ? value2
    : condition3 ? value3
    : value4;
}
```

This is equivalent to the following `if...else` chain.

```
function example() {
  if (condition1) {
    return value1;
  } else if (condition2) {
    return value2;
  } else if (condition3) {
    return value3;
  } else {
    return value4;
  }
}
```

See also

- `if...else`
- Nullish coalescing operator (`??`)
- Optional chaining (`?.`)
- Learn: Making decisions in your code — conditionals
- Expressions and operators guide

Decrement (--)

The **decrement (--)** operator decrements (subtracts one from) its operand and returns the value before or after the decrement, depending on where the operator is placed.

Try it

```
let x = 3;
const y = x--;

console.log(`x:${x}, y:${y}`);
// Expected output: "x:2, y:3"

let a = 3;
const b = --a;

console.log(`a:${a}, b:${b}`);
// Expected output: "a:2, b:2"
```

Syntax

```
x--
--x
```

Description

The `--` operator is overloaded for two types of operands: number and BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt decrement if the operand becomes a BigInt; otherwise, it performs number decrement.

If used postfix, with operator after operand (for example, `x--`), the decrement operator decrements and returns the value before decrementing.

If used prefix, with operator before operand (for example, `--x`), the decrement operator decrements and returns the value after decrementing.

The decrement operator can only be applied on operands that are references (variables and object properties; i.e., valid assignment targets). `--x` itself evaluates to a value, not a reference, so you cannot chain multiple decrement operators together.

```
--(--x); // SyntaxError: Invalid left-hand side expression in prefix operation
```

Examples

Postfix decrement

```
let x = 3;
const y = x--;
// x is 2; y is 3

let x2 = 3n;
const y2 = x2--;
// x2 is 2n; y2 is 3n
```

Prefix decrement

```
let x = 3;
const y = --x;
// x is 2; y = 2

let x2 = 3n;
const y2 = --x2;
// x2 is 2n; y2 is 2n
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Unary negation (-)
- Unary plus (+)

delete

The **`delete`** operator removes a property from an object. If the property's value is an object and there are no more references to the object, the object held by that property is eventually released automatically.

Try it

```
const employee = {
  firstName: "Maria",
  lastName: "Sanchez",
};

console.log(employee.firstName);
// Expected output: "Maria"

delete employee.firstName;

console.log(employee.firstName);
// Expected output: undefined
```

Syntax

```
delete object.property
delete object[property]
```

> **Note:** The syntax allows a wider range of expressions following the `delete` operator, but only the above forms lead to meaningful behaviors.

Parameters

`object`

The name of an object, or an expression evaluating to an object.

`property`

The property to delete.

Return value

`true` for all cases except when the property is an own non-configurable property, in which case `false` is returned in non-strict mode.

Exceptions

`TypeError`

Thrown in strict mode if the property is an own non-configurable property.

`ReferenceError`

Thrown if `object` is `super`.

Description

The `delete` operator has the same precedence as other unary operators like `typeof`. Therefore, it accepts any expression formed by higher-precedence operators. However, the following forms lead to early syntax errors in strict mode:

```
delete identifier;
delete object.#privateProperty;
```

Because classes are automatically in strict mode, and private elements can only be legally referenced in class bodies, this means private elements can never be deleted. While `delete identifier` may work if `identifier` refers to a configurable property of the global object, you should avoid this form and prefix it with `globalThis` instead.

While other expressions are accepted, they don't lead to meaningful behaviors:

```
delete console.log(1);
// Logs 1, returns true, but nothing deleted
```

The `delete` operator removes a given property from an object. On successful deletion, it will return `true`, else `false` will be returned. Unlike what common belief suggests (perhaps due to other programming languages like delete in C++), the `delete` operator has **nothing** to do with directly freeing memory. Memory management is done indirectly via breaking references. See the memory management page for more details.

It is important to consider the following scenarios:

- If the property which you are trying to delete does not exist, `delete` will not have any effect and will return `true`.
- `delete` only has an effect on own properties. If a property with the same name exists on the object's prototype chain, then after deletion, the object will use the property from the prototype chain.
- Non-configurable properties cannot be removed. This includes properties of built-in objects like `Math`, `Array`, `Object` and properties that are created as non-configurable with methods like `Object.defineProperty()`.
- Deleting variables, including function parameters, never works. `delete variable` will throw a `SyntaxError` in strict mode, and will have no effect in non-strict mode.
  - Any variable declared with `var` cannot be deleted from the global scope or from a function's scope, because while they may be attached to the global object, they are not configurable.
  - Any variable declared with `let` or `const` cannot be deleted from the scope within which they were defined, because they are not attached to an object.

Examples

Using delete

> **Note:** The following example uses non-strict-mode only features, like implicitly creating global variables and deleting identifiers, which are forbidden in strict mode.

```
// Creates the property empCount on the global scope.
// Since we are using var, this is marked as non-configurable.
var empCount = 43;

// Creates the property EmployeeDetails on the global scope.
// Since it was defined without "var", it is marked configurable.
EmployeeDetails = {
  name: "xyz",
  age: 5,
  designation: "Developer",
};

// delete can be used to remove properties from objects.
delete EmployeeDetails.name; // returns true

// Even when the property does not exist, delete returns "true".
delete EmployeeDetails.salary; // returns true

// EmployeeDetails is a property of the global scope.
delete EmployeeDetails; // returns true

// On the contrary, empCount is not configurable
// since var was used.
delete empCount; // returns false

// delete also does not affect built-in static properties
// that are non-configurable.
delete Math.PI; // returns false

function f() {
  var z = 44;

  // delete doesn't affect local variable names
  delete z; // returns false
}
```

delete and the prototype chain

In the following example, we delete an own property of an object while a property with the same name is available on the prototype chain:

```
function Foo() {
  this.bar = 10;
}

Foo.prototype.bar = 42;

const foo = new Foo();

// foo.bar is associated with the
```

```
// own property.
console.log(foo.bar); // 10

// Delete the own property within the
// foo object.
delete foo.bar; // returns true

// foo.bar is still available in the
// prototype chain.
console.log(foo.bar); // 42

// Delete the property on the prototype.
delete Foo.prototype.bar; // returns true

// The "bar" property can no longer be
// inherited from Foo since it has been
// deleted.
console.log(foo.bar); // undefined
```

Deleting array elements

When you delete an array element, the array `length` is not affected. This holds even if you delete the last element of the array.

When the `delete` operator removes an array element, that element is no longer in the array. In the following example, `trees[3]` is removed with `delete`.

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
delete trees[3];
console.log(3 in trees); // false
```

This creates a sparse array with an empty slot. If you want an array element to exist but have an undefined value, use the `undefined` value instead of the `delete` operator. In the following example, `trees[3]` is assigned the value `undefined`, but the array element still exists:

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
trees[3] = undefined;
console.log(3 in trees); // true
```

If instead, you want to remove an array element by changing the contents of the array, use the `splice()` method. In the following example, `trees[3]` is removed from the array completely using `splice()`:

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
trees.splice(3, 1);
console.log(trees); // ["redwood", "bay", "cedar", "maple"]
```

Deleting non-configurable properties

When a property is marked as non-configurable, `delete` won't have any effect, and will return `false`. In strict mode, this will raise a `TypeError`.

```
const Employee = {};
Object.defineProperty(Employee, "name", { configurable: false });

console.log(delete Employee.name); // returns false
```

`var` creates non-configurable properties that cannot be deleted with the `delete` operator:

```
// Since "nameOther" is added using with the
// var keyword, it is marked as non-configurable
var nameOther = "XYZ";

// We can access this global property using:
Object.getOwnPropertyDescriptor(globalThis, "nameOther");
// {
//   value: "XYZ",
//   writable: true,
//   enumerable: true,
//   configurable: false
// }

delete globalThis.nameOther; // return false
```

In strict mode, this would raise an exception.

Deleting global properties

If a global property is configurable (for example, via direct property assignment), it can be deleted, and subsequent references to them as global variables will produce a `ReferenceError`.

```
globalThis.globalVar = 1;
console.log(globalVar); // 1
// In non-strict mode, you can use `delete globalVar` as well
delete globalThis.globalVar;
console.log(globalVar); // ReferenceError: globalVar is not defined
```

See also

- `Reflect.deleteProperty()`
- `Map.prototype.delete()`

Destructuring

The **destructuring** syntax is a JavaScript syntax that makes it possible to unpack values from arrays, or properties from objects, into distinct variables. It can be used in locations that receive data (such as the left-hand side of an assignment or anywhere that creates new identifier bindings).

Try it

```
let a, b, rest;
[a, b] = [10, 20];

console.log(a);
// Expected output: 10

console.log(b);
// Expected output: 20

[a, b, ...rest] = [10, 20, 30, 40, 50];

console.log(rest);
// Expected output: Array [30, 40, 50]
```

Syntax

```
const [a, b] = array;
const [a, , b] = array;
const [a = aDefault, b] = array;
const [a, b, ...rest] = array;
const [a, , b, ...rest] = array;
const [a, b, ...{ pop, push }] = array;
const [a, b, ...[c, d]] = array;

const { a, b } = obj;
const { a: a1, b: b1 } = obj;
const { a: a1 = aDefault, b = bDefault } = obj;
const { a, b, ...rest } = obj;
const { a: a1, b: b1, ...rest } = obj;
const { [key]: a } = obj;

let a, b, a1, b1, c, d, rest, pop, push;
[a, b] = array;
[a, , b] = array;
[a = aDefault, b] = array;
[a, b, ...rest] = array;
[a, , b, ...rest] = array;
[a, b, ...{ pop, push }] = array;
[a, b, ...[c, d]] = array;

({ a, b } = obj); // parentheses are required
({ a: a1, b: b1 } = obj);
({ a: a1 = aDefault, b = bDefault } = obj);
({ a, b, ...rest } = obj);
({ a: a1, b: b1, ...rest } = obj);
```

Description

The object and array literal expressions provide an easy way to create *ad hoc* packages of data.

```
const arr = [a, b, c];
```

The destructuring uses similar syntax but uses it on the left-hand side of the assignment instead. It performs the reverse operation of an array declaration, by declaring each element in the collection as a separate variable.

```
const arr = [1, 2, 3];
const [a, b, c] = arr;
// a = 1, b = 2, c = 3
```

As for objects, compare the two pairs of lines below, and see how there's a direct correspondence within each pair.

```
const obj = { a, b, c };
const { a, b, c } = obj;
// Equivalent to:
// const a = obj.a, b = obj.b, c = obj.c;

const obj = { prop1: x, prop2: y, prop3: z };
const { prop1: x, prop2: y, prop3: z } = obj;
// Equivalent to:
// const x = obj.prop1, y = obj.prop2, z = obj.prop3;
```

This capability is similar to features present in languages such as Perl and Python.

For features specific to array or object destructuring, refer to the individual examples below.

Binding and assignment

For both object and array destructuring, there are two kinds of destructuring patterns: *binding pattern* and *assignment pattern*, with slightly different syntaxes.

In binding patterns, the pattern starts with a declaration keyword (`var`, `let`, or `const`). Then, each individual property must either be bound to a variable or further destructured.

```
const obj = { a: 1, b: { c: 2 } };
const {
  a,
  b: { c: d },
} = obj;
// Two variables are bound: `a` and `d`
```

All variables share the same declaration, so if you want some variables to be re-assignable but others to be read-only, you may have to destructure twice — once with `let`, once with `const`.

```
const obj = { a: 1, b: { c: 2 } };
const { a } = obj; // a is constant
let {
  b: { c: d },
} = obj; // d is re-assignable
```

In many other syntaxes where the language binds a variable for you, you can use a binding destructuring pattern. These include:

- The looping variable of `for...in` `for...of`, and `for await...of` loops;
- Function parameters;
- The `catch` binding variable.

In assignment patterns, the pattern does not start with a keyword. Each destructured property is assigned to a target of assignment — which may either be declared beforehand with `var` or `let`, or is a property of another object — in general, anything that can appear on the left-hand side of an assignment expression.

```
const numbers = [];
const obj = { a: 1, b: 2 };
({ a: numbers[0], b: numbers[1] } = obj);
// The properties `a` and `b` are assigned to properties of `numbers`
```

> **Note:** The parentheses `( ... )` around the assignment statement are required when using object literal destructuring without a declaration.
>
> `{ a, b } = { a: 1, b: 2 }` is not valid stand-alone syntax, as the `{ a, b }` on the left-hand side is considered a block and not an object literal according to the rules of expression statements. However, `({ a, b } = { a: 1, b: 2 })` is valid, as is `const { a, b } = { a: 1, b: 2 }`.
>
> If your coding style does not include trailing semicolons, the `( ... )` expression needs to be preceded by a semicolon, or it may be used to execute a function on the previous line.

Note that the equivalent *binding pattern* of the code above is not valid syntax:

```
const numbers = [];
const obj = { a: 1, b: 2 };
const { a: numbers[0], b: numbers[1] } = obj;

// This is equivalent to:
//   const numbers[0] = obj.a;
//   const numbers[1] = obj.b;
// Which definitely is not valid.
```

You can only use assignment patterns as the left-hand side of the assignment operator. You cannot use them with compound assignment operators such as `+=` or `*=`.

Default value

Each destructured property can have a *default value*. The default value is used when the property is not present, or has value `undefined`. It is not used if the property has value `null`.

```
const [a = 1] = []; // a is 1
const { b = 2 } = { b: undefined }; // b is 2
const { c = 2 } = { c: null }; // c is null
```

The default value can be any expression. It will only be evaluated when necessary.

```
const { b = console.log("hey") } = { b: 2 };
// Does not log anything, because `b` is defined and there's no need
// to evaluate the default value.
```

Rest properties and rest elements

You can end a destructuring pattern with a rest property `...rest`. For array destructuring, it collects remaining elements of the iterable into a new array called `rest` (or any name you give it). For object destructuring, it copies all enumerable own properties of the object that are not already picked off by the destructuring pattern into a new object called `rest`.

More formally, the `...rest` syntax is called "rest elements" in array destructuring and "rest properties" in object destructuring, but we often just collectively call them "rest property".

```
const { a, ...others } = { a: 1, b: 2, c: 3 };
console.log(others); // { b: 2, c: 3 }

const [first, ...others2] = [1, 2, 3];
console.log(others2); // [2, 3]
```

The rest property must be the last in the pattern, and must not have a trailing comma.

```
const [a, ...b,] = [1, 2, 3];

// SyntaxError: rest element may not have a trailing comma
// Always consider using rest operator as the last element
```

Like any other JavaScript language feature, there is no implicit deep-copying. The rest property creates a new object (or array), but all its member values, if they are object references, remain the same.

Examples

Array destructuring

Basic variable assignment

```
const foo = ["one", "two", "three"];

const [red, yellow, green] = foo;
console.log(red); // "one"
console.log(yellow); // "two"
console.log(green); // "three"
```

Destructuring with more elements than the source

In an array destructuring from an array of length *N* specified on the right-hand side of the assignment, if the number of variables specified on the left-hand side of the assignment is greater than *N*, only the first *N* variables are assigned values. The values of the remaining variables will be undefined.

```
const foo = ["one", "two"];

const [red, yellow, green, blue] = foo;
console.log(red); // "one"
console.log(yellow); // "two"
console.log(green); // undefined
console.log(blue); // undefined
```

Swapping variables

Two variables values can be swapped in one destructuring expression.

Without destructuring, swapping two values requires a temporary variable (or, in some low-level languages, the XOR-swap trick).

```
let a = 1;
let b = 3;

[a, b] = [b, a];
console.log(a); // 3
console.log(b); // 1

const arr = [1, 2, 3];
[arr[2], arr[1]] = [arr[1], arr[2]];
console.log(arr); // [1, 3, 2]
```

Parsing an array returned from a function

It's always been possible to return an array from a function. Destructuring can make working with an array return value more concise.

In this example, `f()` returns the values `[1, 2]` as its output, which can be parsed in a single line with destructuring.

```
function f() {
  return [1, 2];
}

const [a, b] = f();
console.log(a); // 1
console.log(b); // 2
```

Ignoring some returned values

You can ignore return values that you're not interested in:

```
function f() {
  return [1, 2, 3];
}

const [a, , b] = f();
console.log(a); // 1
console.log(b); // 3

const [c] = f();
console.log(c); // 1
```

You can also ignore all returned values:

```
[, ,] = f();
```

Though in this case, it's probably clearer to just call the function and not use destructuring at all. You don't have to use the return value.

Using a binding pattern as the rest property

The rest property of array destructuring can be another array or object binding pattern. The inner destructuring destructures from the array created after collecting the rest elements, so you cannot access any properties present on the original iterable in this way.

```
const [a, b, ...{ length }] = [1, 2, 3];
console.log(a, b, length); // 1 2 1
```

```
const [a, b, ...[c, d]] = [1, 2, 3, 4];
console.log(a, b, c, d); // 1 2 3 4
```

These binding patterns can even be nested, as long as each rest property is the last in the list.

```
const [a, b, ...[c, d, ...[e, f]]] = [1, 2, 3, 4, 5, 6];
console.log(a, b, c, d, e, f); // 1 2 3 4 5 6
```

On the other hand, object destructuring can only have an identifier as the rest property.

```
const { a, ...{ b } } = { a: 1, b: 2 };
// SyntaxError: `...` must be followed by an identifier in declaration contexts

let a, b;
({ a, ...{ b } } = { a: 1, b: 2 });
// SyntaxError: `...` must be followed by an assignable reference in assignment contexts
```

Unpacking values from a regular expression match

When the regular expression `exec()` method finds a match, it returns an array containing first the entire matched portion of the string and then the portions of the string that matched each parenthesized group in the regular expression. Destructuring allows you to unpack the parts out of this array easily, ignoring the full match if it is not needed.

```
function parseProtocol(url) {
  const parsedURL = /^(\w+):\/\/([^/]+)\/(.*)$/.exec(url);
  if (!parsedURL) {
    return false;
  }
  console.log(parsedURL);
  // ["https://developer.mozilla.org/en-US/docs/Web/JavaScript",
  // "https", "developer.mozilla.org", "en-US/docs/Web/JavaScript"]

  const [, protocol, fullHost, fullPath] = parsedURL;
  return protocol;
}

console.log(
  parseProtocol("https://developer.mozilla.org/en-US/docs/Web/JavaScript"),
);
// "https"
```

Using array destructuring on any iterable

Array destructuring calls the iterable protocol of the right-hand side. Therefore, any iterable, not necessarily arrays, can be destructured.

```
const [a, b] = new Map([
  [1, 2],
  [3, 4],
]);
console.log(a, b); // [1, 2] [3, 4]
```

Non-iterables cannot be destructured as arrays.

```
const obj = { 0: "a", 1: "b", length: 2 };
const [a, b] = obj;
// TypeError: obj is not iterable
```

Iterables are only iterated until all bindings are assigned.

```
const obj = {
  *[Symbol.iterator]() {
    for (const v of [0, 1, 2, 3]) {
      console.log(v);
      yield v;
    }
  },
};
const [a, b] = obj; // Only logs 0 and 1
```

The rest binding is eagerly evaluated and creates a new array, instead of using the old iterable.

```
const obj = {
  *[Symbol.iterator]() {
    for (const v of [0, 1, 2, 3]) {
      console.log(v);
      yield v;
    }
  },
};
const [a, b, ...rest] = obj; // Logs 0 1 2 3
console.log(rest); // [2, 3] (an array)
```

Object destructuring

Basic assignment

```
const user = {
  id: 42,
  isVerified: true,
};

const { id, isVerified } = user;

console.log(id); // 42
console.log(isVerified); // true
```

Assigning to new variable names

A property can be unpacked from an object and assigned to a variable with a different name than the object property.

```
const o = { p: 42, q: true };
const { p: foo, q: bar } = o;

console.log(foo); // 42
console.log(bar); // true
```

Here, for example, `const { p: foo } = o` takes from the object `o` the property named `p` and assigns it to a local variable named `foo`.

Assigning to new variable names and providing default values

A property can be both

- Unpacked from an object and assigned to a variable with a different name.
- Assigned a default value in case the unpacked value is `undefined`.

```
const { a: aa = 10, b: bb = 5 } = { a: 3 };

console.log(aa); // 3
console.log(bb); // 5
```

Unpacking properties from objects passed as a function parameter

Objects passed into function parameters can also be unpacked into variables, which may then be accessed within the function body. As for object assignment, the destructuring syntax allows for the new variable to have the same name or a different name than the original property, and to assign default values for the case when the original object does not define the property.

Consider this object, which contains information about a user.

```
const user = {
  id: 42,
  displayName: "jdoe",
  fullName: {
    firstName: "Jane",
    lastName: "Doe",
```

```
  },
};
```

Here we show how to unpack a property of the passed object into a variable with the same name. The parameter value `{ id }` indicates that the `id` property of the object passed to the function should be unpacked into a variable with the same name, which can then be used within the function.

```
function userId({ id }) {
  return id;
}

console.log(userId(user)); // 42
```

You can define the name of the unpacked variable. Here we unpack the property named `displayName`, and rename it to `dname` for use within the function body.

```
function userDisplayName({ displayName: dname }) {
  return dname;
}

console.log(userDisplayName(user)); // "jdoe"
```

Nested objects can also be unpacked. The example below shows the property `fullname.firstName` being unpacked into a variable called `name`.

```
function whois({ displayName, fullName: { firstName: name } }) {
  return `${displayName} is ${name}`;
}

console.log(whois(user)); // "jdoe is Jane"
```

Setting a function parameter's default value

Default values can be specified using `=`, and will be used as variable values if a specified property does not exist in the passed object.

Below we show a function where the default size is `'big'`, default co-ordinates are `x: 0, y: 0` and default radius is 25.

```
function drawChart({
  size = "big",
  coords = { x: 0, y: 0 },
  radius = 25,
} = {}) {
  console.log(size, coords, radius);
  // do some chart drawing
}

drawChart({
  coords: { x: 18, y: 30 },
  radius: 30,
});
```

In the function signature for `drawChart` above, the destructured left-hand side has a default value of an empty object `= {}`.

You could have also written the function without that default. However, if you leave out that default value, the function will look for at least one argument to be supplied when invoked, whereas in its current form, you can call `drawChart()` without supplying any parameters. Otherwise, you need to at least supply an empty object literal.

For more information, see Default parameters > Destructured parameter with default value assignment.

Nested object and array destructuring

```
const metadata = {
  title: "Scratchpad",
  translations: [
    {
      locale: "de",
      localizationTags: [],
      lastEdit: "2014-04-14T08:43:37",
      url: "/de/docs/Tools/Scratchpad",
      title: "JavaScript-Umgebung",
    },
  ],
  url: "/en-US/docs/Tools/Scratchpad",
};

const {
  title: englishTitle, // rename
  translations: [
    {
      title: localeTitle, // rename
    },
  ],
} = metadata;

console.log(englishTitle); // "Scratchpad"
console.log(localeTitle); // "JavaScript-Umgebung"
```

For of iteration and destructuring

```
const people = [
  {
    name: "Mike Smith",
    family: {
      mother: "Jane Smith",
      father: "Harry Smith",
      sister: "Samantha Smith",
    },
    age: 35,
  },
  {
    name: "Tom Jones",
    family: {
      mother: "Norah Jones",
      father: "Richard Jones",
      brother: "Howard Jones",
    },
    age: 25,
  },
];

for (const {
  name: n,
  family: { father: f },
} of people) {
  console.log(`Name: ${n}, Father: ${f}`);
}

// "Name: Mike Smith, Father: Harry Smith"
// "Name: Tom Jones, Father: Richard Jones"
```

Computed object property names and destructuring

Computed property names, like on object literals, can be used with destructuring.

```
const key = "z";
const { [key]: foo } = { z: "bar" };

console.log(foo); // "bar"
```

Invalid JavaScript identifier as a property name

Destructuring can be used with property names that are not valid JavaScript identifiers by providing an alternative identifier that is valid.

```
const foo = { "fizz-buzz": true };
const { "fizz-buzz": fizzBuzz } = foo;

console.log(fizzBuzz); // true
```

Destructuring primitive values

Object destructuring is almost equivalent to property accessing. This means if you try to destruct a primitive value, the value will get wrapped into the corresponding wrapper object and the property is accessed on the wrapper object.

```
const { a, toFixed } = 1;
console.log(a, toFixed); // undefined ƒ toFixed() { [native code] }
```

Same as accessing properties, destructuring `null` or `undefined` throws a `TypeError`.

```
const { a } = undefined; // TypeError: Cannot destructure property 'a' of 'undefined' as it is undefined.
const { b } = null; // TypeError: Cannot destructure property 'b' of 'null' as it is null.
```

This happens even when the pattern is empty.

```
const {} = null; // TypeError: Cannot destructure 'null' as it is null.
```

Combined array and object destructuring

Array and object destructuring can be combined. Say you want the third element in the array `props` below, and then you want the `name` property in the object, you can do the following:

```
const props = [
  { id: 1, name: "Fizz" },
  { id: 2, name: "Buzz" },
  { id: 3, name: "FizzBuzz" },
];

const [, , { name }] = props;

console.log(name); // "FizzBuzz"
```

The prototype chain is looked up when the object is deconstructed

When deconstructing an object, if a property is not accessed in itself, it will continue to look up along the prototype chain.

```
const obj = {
  self: "123",
  __proto__: {
    prot: "456",
  },
};
const { self, prot } = obj;

console.log(self); // "123"
console.log(prot); // "456"
```

See also

- Assignment operators
- ES6 in Depth: Destructuring on hacks.mozilla.org (2015)

Division (/)

The **division (/)** operator produces the quotient of its operands where the left operand is the dividend and the right operand is the divisor.

Try it

```
console.log(12 / 2);
// Expected output: 6

console.log(3 / 2);
// Expected output: 1.5

console.log(6 / "3");
// Expected output: 2

console.log(2 / 0);
// Expected output: Infinity
```

Syntax

```
x / y
```

Description

The `/` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt division if both operands become BigInts; otherwise, it performs number division. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

For BigInt division, the result is the quotient of the two operands truncated towards zero, and the remainder is discarded. A `RangeError` is thrown if the divisor `y` is `0n`. This is because number division by zero returns `Infinity` or `-Infinity`, but BigInt has no concept of infinity.

Examples

Division using numbers

```
1 / 2; // 0.5
Math.floor(3 / 2); // 1
1.0 / 2.0; // 0.5

2 / 0; // Infinity
2.0 / 0.0; // Infinity, because 0.0 === 0
2.0 / -0.0; // -Infinity
```

Other non-BigInt values are coerced to numbers:

```
5 / "2"; // 2.5
5 / "foo"; // NaN
```

Division using BigInts

```
1n / 2n; // 0n
5n / 3n; // 1n
-1n / 3n; // 0n
1n / -3n; // 0n

2n / 0n; // RangeError: BigInt division by zero
```

You cannot mix BigInt and number operands in division.

```
2n / 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 / 2n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do division with a BigInt and a non-BigInt, convert either operand:

```
2n / BigInt(2); // 1n
Number(2n) / 2; // 1
```

See also

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

Division assignment (/=)

The **division assignment (/=)** operator performs division on the two operands and assigns the result to the left operand.

Try it

```
let a = 3;

a /= 2;
console.log(a);
// Expected output: 1.5

a /= 0;
console.log(a);
// Expected output: Infinity

a /= "hello";
console.log(a);
// Expected output: NaN
```

Syntax

```
x /= y
```

Description

`x /= y` is equivalent to `x = x / y`, except that the expression `x` is only evaluated once.

Examples

Division assignment using numbers

```
let bar = 5;

bar /= 2; // 2.5
bar /= 2; // 1.25
bar /= 0; // Infinity
```

Other non-BigInt values are coerced to numbers:

```
let bar = 5;
bar /= "2"; // 2.5
bar /= "foo"; // NaN
```

Division assignment using BigInts

```
let foo = 3n;
foo /= 2n; // 1n
foo /= 2n; // 0n

foo /= 0n; // RangeError: BigInt division by zero
foo /= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

See also

- Assignment operators in the JS guide
- Division (/)

Equality (==)

The **equality (==)** operator checks whether its two operands are equal, returning a Boolean result. Unlike the strict equality operator, it attempts to convert and compare operands that are of different types.

Try it

```
console.log(1 == 1);
// Expected output: true

console.log("hello" == "hello");
// Expected output: true

console.log("1" == 1);
// Expected output: true

console.log(0 == false);
// Expected output: true
```

Syntax

```
x == y
```

Description

The equality operators (`==` and `!=`) provide the IsLooselyEqual semantic. This can be roughly summarized as follows:

1. If the operands have the same type, they are compared as follows:
   - Object: return `true` only if both operands reference the same object.
   - String: return `true` only if both operands have the same characters in the same order.
   - Number: return `true` only if both operands have the same value. `+0` and `-0` are treated as the same value. If either operand is `NaN`, return `false`; so, `NaN` is never equal to `NaN`.
   - Boolean: return `true` only if operands are both `true` or both `false`.
   - BigInt: return `true` only if both operands have the same value.
   - Symbol: return `true` only if both operands reference the same symbol.
2. If one of the operands is `null` or `undefined`, the other must also be `null` or `undefined` to return `true`. Otherwise return `false`.
3. If one of the operands is an object and the other is a primitive, convert the object to a primitive.
4. At this step, both operands are converted to primitives (one of String, Number, Boolean, Symbol, and BigInt). The rest of the conversion is done case-by-case.
   - If they are of the same type, compare them using step 1.
   - If one of the operands is a Symbol but the other is not, return `false`.
   - If one of the operands is a Boolean but the other is not, convert the boolean to a number: `true` is converted to 1, and `false` is converted to 0. Then compare the two operands loosely again.
   - Number to String: convert the string to a number. Conversion failure results in `NaN`, which will guarantee the equality to be `false`.
   - Number to BigInt: compare by their mathematical value. If the number is ±Infinity or `NaN`, return `false`.
   - String to BigInt: convert the string to a BigInt using the same algorithm as the `BigInt()` constructor. If conversion fails, return `false`.

Loose equality is *symmetric*: `A == B` always has identical semantics to `B == A` for any values of `A` and `B` (except for the order of applied conversions).

The most notable difference between this operator and the strict equality (`===`) operator is that the strict equality operator does not attempt type conversion. Instead, the strict equality operator always considers operands of different types to be different. The strict equality operator essentially carries out only step 1, and then returns `false` for all other cases.

There's a "willful violation" of the above algorithm: if one of the operands is `document.all`, it is treated as if it's `undefined`. This means that `document.all == null` is `true`, but `document.all === undefined || document.all === null` is `false`.

Examples

Comparison with no type conversion

```
1 == 1; // true
"hello" == "hello"; // true
```

Comparison with type conversion

```
"1" == 1; // true
1 == "1"; // true
0 == false; // true
0 == null; // false
0 == undefined; // false
0 == !!null; // true, look at Logical NOT operator
0 == !!undefined; // true, look at Logical NOT operator
null == undefined; // true

const number1 = new Number(3);
const number2 = new Number(3);
number1 == 3; // true
number1 == number2; // false
```

Comparison of objects

```
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};
```

```
console.log(object1 == object2); // false
console.log(object1 == object1); // true
```

Comparing strings and String objects

Note that strings constructed using `new String()` are objects. If you compare one of these with a string literal, the `String` object will be converted to a string literal and the contents will be compared. However, if both operands are `String` objects, then they are compared as objects and must reference the same object for comparison to succeed:

```
const string1 = "hello";
const string2 = String("hello");
const string3 = new String("hello");
const string4 = new String("hello");

console.log(string1 == string2); // true
console.log(string1 == string3); // true
console.log(string2 == string3); // true
console.log(string3 == string4); // false
console.log(string4 == string4); // true
```

Comparing Dates and strings

```
const d = new Date("1995-12-17T03:24:00");
const s = d.toString(); // for example: "Sun Dec 17 1995 03:24:00 GMT-0800 (Pacific Standard Time)"
console.log(d == s); // true
```

Comparing arrays and strings

```
const a = [1, 2, 3];
const b = "1,2,3";
a == b; // true, `a` converts to string

const c = [true, 0.5, "hey"];
const d = c.toString(); // "true,0.5,hey"
c == d; // true
```

See also

- Inequality (`!=`)
- Strict equality (`===`)
- Strict inequality (`!==`)

Exponentiation (**)

The **exponentiation (**)** operator returns the result of raising the first operand to the power of the second operand. It is equivalent to `Math.pow()`, except it also accepts BigInts as operands.

Try it

```
console.log(3 ** 4);
// Expected output: 81

console.log(10 ** -2);
// Expected output: 0.01

console.log(2 ** (3 ** 2));
// Expected output: 512

console.log((2 ** 3) ** 2);
// Expected output: 64
```

Syntax

```
x ** y
```

Description

The `**` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt exponentiation if both operands become BigInts; otherwise, it performs number exponentiation. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

For both numbers and BigInts, `0` raised to a positive power returns `0`, and `0` raised to a power of `0` returns `1`. For numbers, `0` raised to a negative number returns `Infinity`, while `-0` raised to a negative number returns `-Infinity`.

`NaN ** 0` (and the equivalent `Math.pow(NaN, 0)`) is the only case where `NaN` doesn't propagate through mathematical operations — it returns `1` despite the operand being `NaN`. In addition, the behavior where `base` is 1 and `exponent` is non-finite (±Infinity or `NaN`) is different from IEEE 754, which specifies that the result should be 1, whereas JavaScript returns `NaN` to preserve backward compatibility with its original behavior.

For BigInt exponentiation, a `RangeError` is thrown if the exponent `y` is negative. This is because any negative exponent would likely result in a value between 0 and 1 (unless the base is `1`, `-1`, or `0`), which is rounded to zero, and is likely a developer mistake.

The exponentiation operator is right-associative: `a ** b ** c` is equal to `a ** (b ** c)`.

In most languages, such as PHP, Python, and others that have an exponentiation operator (`**`), the exponentiation operator is defined to have a higher precedence than unary operators, such as unary `+` and unary `-`, but there are a few exceptions. For example, in Bash, the `**` operator is defined to have a lower precedence than unary operators.

In JavaScript, it is impossible to write an ambiguous exponentiation expression. That is, you cannot put a unary operator (with precedence 14, including `+`/`-`/`~`/`!`/`++`/`--`/`delete`/`void`/`typeof`/`await`) immediately before the base number; doing so will cause a SyntaxError.

For example, `-2 ** 2` is 4 in Bash, but is -4 in other languages (such as Python). This is invalid in JavaScript, as the operation is ambiguous. You have to parenthesize either side — for example, as `-(2 ** 2)` — to make the intention unambiguous.

Note that some programming languages use the caret symbol `^` for exponentiation, but JavaScript uses that symbol for the bitwise XOR operator.

Examples

Exponentiation using numbers

```
2 ** 3; // 8
3 ** 2; // 9
3 ** 2.5; // 15.588457268119896
10 ** -1; // 0.1
2 ** 1024; // Infinity
NaN ** 2; // NaN
NaN ** 0; // 1
1 ** Infinity; // NaN
```

Other non-BigInt values are coerced to numbers:

```
2 ** "3"; // 8
2 ** "hello"; // NaN
```

Exponentiation using BigInts

```
2n ** 3n; // 8n
2n ** 1024n; // A very large number, but not Infinity
```

You cannot mix BigInt and number operands in exponentiation.

```
2n ** 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 ** 2n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do exponentiation with a BigInt and a non-BigInt, convert either operand:

```
2n ** BigInt(2); // 4n
Number(2n) ** 2; // 4
```

Associativity

```
2 ** 3 ** 2; // 512
2 ** (3 ** 2); // 512
(2 ** 3) ** 2; // 64
```

Usage with unary operators

To invert the sign of the result of an exponentiation expression:

```
-(2 ** 2); // -4
```

To force the base of an exponentiation expression to be a negative number:

```
(-2) ** 2; // 4
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

Exponentiation assignment (**=)

The **exponentiation assignment (**=)** operator performs exponentiation on the two operands and assigns the result to the left operand.

Try it

```
let a = 3;

console.log((a **= 2));
// Expected output: 9

console.log((a **= 0));
// Expected output: 1

console.log((a **= 'hello'));
// Expected output: NaN
```

Syntax

```
x **= y
```

Description

`x **= y` is equivalent to `x = x ** y`, except that the expression `x` is only evaluated once.

Examples

Exponentiation assignment using numbers

```
let bar = 5;
bar **= 2; // 25
```

Other non-BigInt values are coerced to numbers:

```
let baz = 5;
baz **= "foo"; // NaN
```

Exponentiation assignment using BigInts

```
let foo = 3n;
foo **= 2n; // 9n
foo **= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

See also

- Assignment operators in the JS guide
- Exponentiation (`**`)

function expression

The **`function`** keyword can be used to define a function inside an expression.

You can also define functions using the `function` declaration or the arrow syntax.

Try it

```
const getRectArea = function (width, height) {
  return width * height;
};

console.log(getRectArea(3, 4));
// Expected output: 12
```

Syntax

```
function (param0) {
  statements
}
function (param0, param1) {
  statements
}
function (param0, param1, /* …, */ paramN) {
  statements
}

function name(param0) {
  statements
}
function name(param0, param1) {
  statements
}
function name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** An expression statement cannot begin with the keyword `function` to avoid ambiguity with a `function` declaration. The `function` keyword only begins an expression when it appears in a context that cannot accept statements.

Parameters

`name` Optional

The function name. Can be omitted, in which case the function is *anonymous*. The name is only local to the function body.

`paramN` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements which comprise the body of the function.

Description

A `function` expression is very similar to, and has almost the same syntax as, a `function` declaration. The main difference between a `function` expression and a `function` declaration is the *function name*, which can be omitted in `function` expressions to create *anonymous* functions. A `function` expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined. See also the chapter about functions for more information.

Function expression hoisting

Function expressions in JavaScript are not hoisted, unlike function declarations. You can't use function expressions before you create them:

```
console.log(notHoisted); // undefined
// Even though the variable name is hoisted,
// the definition isn't. so it's undefined.
notHoisted(); // TypeError: notHoisted is not a function

var notHoisted = function () {
  console.log("bar");
};
```

Named function expression

If you want to refer to the current function inside the function body, you need to create a named function expression. This name is then local only to the function body (scope). This avoids using the deprecated `arguments.callee` property to call the function recursively.

```
const math = {
  factorial: function factorial(n) {
    console.log(n);
    if (n <= 1) {
      return 1;
    }
    return n * factorial(n - 1);
  },
};

math.factorial(3); // 3;2;1;
```

If a function expression is named, the `name` property of the function is set to that name, instead of the implicit name inferred from syntax (such as the variable the function is assigned to).

Unlike declarations, the name of the function expressions is read-only.

```
"use strict";

function foo() {
  foo = 1;
}
foo();
console.log(foo); // 1
(function foo() {
  foo = 1; // TypeError: Assignment to constant variable.
})();
```

Examples

Using function expression

The following example defines an unnamed function and assigns it to `x`. The function returns the square of its argument:

```
const x = function (y) {
  return y * y;
};
```

Using a function as a callback

More commonly it is used as a callback:

```
button.addEventListener("click", function (event) {
  console.log("button is clicked!");
});
```

Using an Immediately Invoked Function Expression (IIFE)

IIFEs are a common pattern used to execute arbitrarily many statements in their own scope (and possibly return a value), in a location that requires a single expression. Many traditional use cases of IIFEs have been obsoleted by new syntax features such as modules and block-scoped declarations. IIFEs themselves are more commonly written with arrow functions now, but the idea remains the same. In general, IIFEs look like this:

```
// standard IIFE
(function () {
  // statements…
})();

// IIFE with arguments
(function (a, b) {
  console.log(a + b);
})(1, 2); // logs 3

// IIFE being used to initialize a variable
const value = (() => {
  const randomValue = Math.random();
  if (randomValue > 0.5) {
    return "heads";
  }
  return "tails";
})();
```

Here, we introduce several use cases with examples.

Avoid polluting the global namespace in script code

The top-level scope of all scripts is shared, which could include many functions and global variables from different files, so to avoid name conflicts, it's important to limit the number of globally declared names (this is greatly mitigated in modules, but sometimes limiting the scope of temporary variables is still useful, especially when the file is very long). If we have some initialization code that we don't need to use again, we could use the IIFE pattern, which is better than using a function declaration or a function expression because it ensures that the code is only run here and once.

```
// top-level of a script (not a module)

var globalVariable = (() => {
  // some initialization code
  let firstVariable = something();
  let secondVariable = somethingElse();
  return firstVariable + secondVariable;
})();

// firstVariable and secondVariable cannot be accessed outside of the function body.
```

The module pattern

We would also use IIFE to create private and public variables and methods. For a more sophisticated use of the module pattern and other use of IIFE, you could see the book Learning JavaScript Design Patterns by Addy Osmani.

```
const makeWithdraw = (balance) =>
  ((copyBalance) => {
    let balance = copyBalance; // This variable is private
    const doBadThings = () => {
      console.log("I will do bad things with your money");
    };
    doBadThings();
    return {
      withdraw(amount) {
        if (balance >= amount) {
          balance -= amount;
          return balance;
        }
        return "Insufficient money";
      },
    };
  })(balance);

const firstAccount = makeWithdraw(100); // "I will do bad things with your money"
console.log(firstAccount.balance); // undefined
console.log(firstAccount.withdraw(20)); // 80
console.log(firstAccount.withdraw(30)); // 50
console.log(firstAccount.doBadThings); // undefined; this method is private
const secondAccount = makeWithdraw(20); // "I will do bad things with your money"
console.log(secondAccount.withdraw(30)); // "Insufficient money"
console.log(secondAccount.withdraw(20)); // 0
```

For loop with var before ES6

We could see the following use of IIFE in some old code, before the introduction of the block-scoped `let` and `const` declarations. With the statement `var`, we have only function scopes and the global scope. Suppose we want to create 2 buttons with the texts Button 0 and Button 1 and when we click them, we would like them to alert 0 and 1. The following code doesn't work:

```
for (var i = 0; i < 2; i++) {
  const button = document.createElement("button");
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i);
  };
  document.body.appendChild(button);
}
console.log(i); // 2
```

When clicked, both Button 0 and Button 1 alert 2 because `i` is global, with the last value 2. To fix this problem before ES6, we could use the IIFE pattern:

```
for (var i = 0; i < 2; i++) {
  const button = document.createElement("button");
  button.innerText = `Button ${i}`;
  button.onclick = (function (copyOfI) {
    return function () {
      console.log(copyOfI);
    };
  })(i);
  document.body.appendChild(button);
}
console.log(i); // 2
```

When clicked, Buttons 0 and 1 alert 0 and 1. The variable `i` is globally defined. Using the statement `let`, we could simply do:

```
for (let i = 0; i < 2; i++) {
  const button = document.createElement("button");
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i);
  };
  document.body.appendChild(button);
}
console.log(i); // Uncaught ReferenceError: i is not defined.
```

When clicked, these buttons alert 0 and 1.

Control flow statements in expression positions

IIFEs enable us to use language constructs such as `switch` in an expression.

```
someObject.property = (() => {
  switch (someVariable) {
    case 0:
      return "zero";
    case 1:
      return "one";
    default:
      return "unknown";
  }
})();
```

This approach can be especially useful in scenarios where you want to make a variable `const`, but are forced to use `let` or `var` during initialization:

```
let onlyAssignedOnce;
try {
  onlyAssignedOnce = someFunctionThatMightThrow();
} catch (e) {
  onlyAssignedOnce = null;
}
```

Using IIFEs, we can make the variable `const`:

```
const onlyAssignedOnce = (() => {
  try {
    return someFunctionThatMightThrow();
  } catch (e) {
    return null;
  }
})();
```

See also

- Functions guide
- Functions

- `function`
- `Function`
- Arrow functions

Greater than (>)

The **greater than (>)** operator returns `true` if the left operand is greater than the right operand, and `false` otherwise.

Try it

```
console.log(5 > 3);
// Expected output: true

console.log(3 > 3);
// Expected output: false

// Compare bigint to number
console.log(3n > 5);
// Expected output: false

console.log("ab" > "aa");
// Expected output: true
```

Syntax

```
x > y
```

Description

The operands are compared using the same algorithm as the Less than operator, except the two operands are swapped. `x > y` is generally equivalent to `y < x`, except that `x > y` coerces `x` to a primitive before `y`, while `y < x` coerces `y` to a primitive before `x`. Because coercion may have side effects, the order of the operands may matter.

Examples

String to string comparison

```
"a" > "b"; // false
"a" > "a"; // false
"a" > "3"; // true
```

String to number comparison

```
"5" > 3; // true
"3" > 3; // false
"3" > 5; // false

"hello" > 5; // false
5 > "hello"; // false

"5" > 3n; // true
"3" > 5n; // false
```

Number to Number comparison

```
5 > 3; // true
3 > 3; // false
3 > 5; // false
```

Number to BigInt comparison

```
5n > 3; // true
3 > 5n; // false
```

Comparing Boolean, null, undefined, NaN

```
true > false; // true
false > true; // false

true > 0; // true
true > 1; // false

null > 0; // false
1 > null; // true

undefined > 3; // false
3 > undefined; // false

3 > NaN; // false
NaN > 3; // false
```

See also

- Greater than or equal (`>=`)
- Less than (`<`)
- Less than or equal (`<=`)

Greater than or equal (`>=`)

The **greater than or equal (`>=`)** operator returns `true` if the left operand is greater than or equal to the right operand, and `false` otherwise.

Try it

```
console.log(5 >= 3);
// Expected output: true

console.log(3 >= 3);
// Expected output: true

// Compare bigint to number
console.log(3n >= 5);
// Expected output: false

console.log("ab" >= "aa");
// Expected output: true
```

Syntax

```
x >= y
```

Description

The operands are compared using the same algorithm as the Less than operator, with the result negated. `x >= y` is generally equivalent to `!(x < y)`, except for two cases where `x >= y` and `x < y` are both `false`:

- If one of the operands gets converted to a BigInt, while the other gets converted to a string that cannot be converted to a BigInt value (it throws a syntax error when passed to `BigInt()`).
- If one of the operands gets converted to `NaN`. (For example, strings that cannot be converted to numbers, or `undefined`.)

`x >= y` is generally equivalent to `x > y || x == y`, except for a few cases:

- When one of `x` or `y` is `null`, and the other is something that's not `null` and becomes 0 when coerced to numeric (including `0`, `0n`, `false`, `""`, `"0"`, `new Date(0)`, etc.): `x >= y` is `true`, while `x > y || x == y` is `false`.
- When one of `x` or `y` is `undefined`, and the other is one of `null` or `undefined`: `x >= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are the same object that becomes `NaN` after the first step of Less than (such as `new Date(NaN)`): `x >= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are different objects that become the same value after the first step of Less than: `x >= y` is `true`, while `x > y || x == y` is `false`.

Examples

String to string comparison

```
"a" >= "b"; // false
"a" >= "a"; // true
"a" >= "3"; // true
```

String to number comparison

```
"5" >= 3; // true
"3" >= 3; // true
"3" >= 5; // false

"hello" >= 5; // false
5 >= "hello"; // false
```

Number to Number comparison

```
5 >= 3; // true
3 >= 3; // true
3 >= 5; // false
```

Number to BigInt comparison

```
5n >= 3; // true
3 >= 3n; // true
3 >= 5n; // false
```

Comparing Boolean, null, undefined, NaN

```
true >= false; // true
true >= true; // true
false >= true; // false
```

```
true >= 0; // true
true >= 1; // true

null >= 0; // true
1 >= null; // true

undefined >= 3; // false
3 >= undefined; // false

3 >= NaN; // false
NaN >= 3; // false
```

See also

- Greater than (>)
- Less than (<)
- Less than or equal (<=)

Grouping operator ( )

The **grouping** `( )` operator controls the precedence of evaluation in expressions. It also acts as a container for arbitrary expressions in certain syntactic constructs, where ambiguity or syntax errors would otherwise occur.

Try it

```
console.log(1 + 2 * 3); // 1 + 6
// Expected output: 7

console.log(1 + (2 * 3)); // 1 + 6
// Expected output: 7

console.log((1 + 2) * 3); // 3 * 3
// Expected output: 9

console.log(1 * 3 + 2 * 3); // 3 + 6
// Expected output: 9
```

Syntax

```
(expression)
```

Parameters

`expression`

Any expression to be evaluated, including comma-joined expressions.

Description

The grouping operator consists of a pair of parentheses around an expression that groups the contents. The operator overrides the normal operator precedence, so that operators with lower precedence (as low as the comma operator) can be evaluated before an operator with higher precedence.

Examples

Using the grouping operator

Evaluating addition and subtraction before multiplication and division.

```
const a = 1;
const b = 2;
const c = 3;

// default precedence
a + b * c; // 7
// evaluated by default like this
a + (b * c); // 7

// now overriding precedence
// addition before multiplication
(a + b) * c; // 9

// which is equivalent to
a * c + b * c; // 9
```

Notice in these examples that the order in which the *operators* evaluate has changed, but the order in which the *operands* evaluate has not. For example, in this code, the function invocations `a()`, `b()`, and `c()` are evaluated left-to-right (the normal order of evaluation) before the operator order is considered.

```
a() * (b() + c());
```

The function `a` will be called before the function `b`, which will be called before the function `c`. For more on operator precedence, see its reference page.

Using the grouping operator to eliminate parsing ambiguity

An expression statement cannot start with the keyword `function`, because the parser would see it as the start of a function declaration. This means the following IIFE syntax is invalid:

```
function () {
  // code
}();
```

The grouping operator can be used to eliminate this ambiguity, since when the parser sees the left parenthesis, it knows that what follows must be an expression instead of a declaration.

```
(function () {
  // code
})();
```

You may also use the `void` operator to eliminate ambiguity.

In an arrow function expression body (one that directly returns an expression without the keyword `return`), the grouping operator can be used to return an object literal expression, because otherwise the left curly brace would be interpreted as the start of the function body.

```
const f = () => ({ a: 1 });
```

If a property is accessed on a number literal, the property accessor dot `.` may be ambiguous with a decimal point, unless the number already has a decimal point. You can wrap integer literals in parentheses to eliminate this ambiguity.

```
(1).toString(); // "1"
```

Grouping operator and automatic semicolon insertion

The grouping operator can mitigate automatic semicolon insertion (ASI) pitfalls. For example, the `return` keyword and the returned expression cannot have a line break in between:

```
function sum(a, b) {
  return
    a + b;
}
```

This code will return `undefined`, because a semicolon is inserted directly after the `return` keyword, which causes the function to return immediately without evaluating `a + b`. In case the returned expression is long and you want to keep it well-formatted, you may use the grouping operator to signify that the `return` keyword is followed by an expression and prevent semicolon insertion:

```
function sum(a, b) {
  return (
    a + b
  );
}
```

However, grouping may also *introduce* ASI hazards. When a line starts with a left parenthesis and the previous line ends with an expression, the parser will not insert a semicolon before the line break, because it could be the middle of a function call. For example:

```
const a = 1
(1).toString()
```

This code would be parsed as:

```
const a = 1(1).toString();
```

Which throws "TypeError: 1 is not a function". If your coding style does not use semicolons, remember that when a line starts with a left parenthesis, *prefix* it with a semicolon. This practice is recommended by several formatters and/or style guides, including Prettier and standard.

```
const a = 1
;(1).toString()
```

For more advice on working with ASI, see its reference section.

See also

- Operator precedence
- `delete`
- `typeof`

import()

The **`import()`** syntax, commonly called *dynamic import*, is a function-like expression that allows loading an ECMAScript module asynchronously and dynamically into a potentially non-module environment.

Unlike the declaration-style counterpart, dynamic imports are only evaluated when needed, and permit greater syntactic flexibility.

Syntax

```
import(moduleName)
import(moduleName, options)
```

The `import()` call is a syntax that closely resembles a function call, but `import` itself is a keyword, not a function. You cannot alias it like `const myImport = import`, which will throw a `SyntaxError`.

Trailing commas are only allowed if the runtime also supports `options`. Check browser compatibility.

Parameters

`moduleName`

The module to import from. The evaluation of the specifier is host-specified, but always follows the same algorithm as static import declarations.

`options`

An object containing import options. The following key is recognized:

`with`

The import attributes.

Return value

Returns a promise which:

- If the referenced module is loaded and evaluated successfully, fulfills to a module namespace object: an object containing all exports from `moduleName`.
- If the coercion to string of `moduleName` throws, rejects with the thrown error.
- If module fetching and loading fails for any reason, rejects with an implementation-defined error (Node uses a generic `Error`, while all browsers use `TypeError`). Common causes may include:
  - In a file-system-based module system (Node.js, for example), if accessing the file system fails (permission denied, file not found, etc.).
  - In a web-based module system (browsers, for example), if the network request fails (not connected to the Internet, CORS issue, etc.) or an HTTP error occurs (404, 500, etc.).
- If evaluation of the referenced module throws, rejects with the thrown error.

> **Note:** `import()` never synchronously throws an error.

Description

The import declaration syntax (`import something from "somewhere"`) is static and will always result in the imported module being evaluated at load time. Dynamic imports allow one to circumvent the syntactic rigidity of import declarations and load a module conditionally or on demand. The following are some reasons why you might need to use dynamic import:

- When importing statically significantly slows the loading of your code or increases your program's memory usage, and there is a low likelihood that you will need the code you are importing, or you will not need it until a later time.
- When the module you are importing does not exist at load time.
- When the import specifier string needs to be constructed dynamically. (Static import only supports static specifiers.)
- When the module being imported has side effects, and you do not want those side effects unless some condition is true. (It is recommended not to have any side effects in a module, but you sometimes cannot control this in your module dependencies.)
- When you are in a non-module environment (for example, `eval` or a script file).

Use dynamic import only when necessary. The static form is preferable for loading initial dependencies, and can benefit more readily from static analysis tools and tree shaking. If your only goal is to defer the execution of the loaded code until when it's used, consider using `import defer`.

If your file is not run as a module (if it's referenced in an HTML file, the script tag must have `type="module"`), you will not be able to use static import declarations. On the other hand, the asynchronous dynamic import syntax is always available, allowing you to import modules into non-module environments.

The `options` parameter allows different kinds of import options. For example, import attributes:

```
import("./data.json", { with: { type: "json" } });
```

Dynamic module import is not permitted in all execution contexts. For example, `import()` can be used in the main thread, a shared worker, or a dedicated worker, but will throw if called within a service worker or a worklet.

"Phase modifiers" for `import` statements can also be used with dynamic imports:

- `import.defer()`
- `import.source()`

Each of these syntaxes is considered a distinct type of expression.

Module namespace object

A *module namespace object* is an object that describes all exports from a module. It is a static object that is created when the module is evaluated. There are two ways to access the module namespace object of a module: through a namespace import (`import * as name from moduleName`), or through the fulfillment value of a dynamic import.

The module namespace object is a sealed object with `null` prototype. This means all string keys of the object correspond to the exports of the module and there are never extra keys. All keys are enumerable in lexicographic order (i.e., the default behavior of `Array.prototype.sort()`), with the default export available as a key called

`default`. In addition, the module namespace object has a `[Symbol.toStringTag]` property with the value `"Module"`, used in `Object.prototype.toString()`.

The string properties are non-configurable and writable when you use `Object.getOwnPropertyDescriptors()` to get their descriptors. However, they are effectively read-only, because you cannot re-assign a property to a new value. This behavior mirrors the fact that static imports create "live bindings" — the values can be re-assigned by the module exporting them, but not by the module importing them. The writability of the properties reflects the possibility of the values changing, because non-configurable and non-writable properties must be constant. For example, you can re-assign the exported value of a variable, and the new value can be observed in the module namespace object.

Each (normalized) module specifier corresponds to a unique module namespace object, so the following is generally true:

```
import * as mod from "/my-module.js";

import("/my-module.js").then((mod2) => {
  console.log(mod === mod2); // true
});
```

Except in one curious case: because a promise never fulfills to a thenable, if the `my-module.js` module exports a function called `then()`, that function will automatically get called when the dynamic import's promise is fulfilled, as part of the promise resolution process.

```
// my-module.js
export function then(resolve) {
  console.log("then() called");
  resolve(1);
}
```

```
// main.js
import * as mod from "/my-module.js";

import("/my-module.js").then((mod2) => {
  // Logs "then() called"
  console.log(mod === mod2); // false
});
```

> **Warning:** Do not export a function called `then()` from a module. This will cause the module to behave differently when imported dynamically than when imported statically.

This aggressive caching ensures that a piece of JavaScript code is never executed more than once, even if it is imported multiple times. Future imports don't even result in HTTP requests or disk access. If you do need to re-import and re-evaluate a module without restarting the entire JavaScript environment, one possible trick is to use a unique query parameter in the module specifier. This works in non-browser runtimes that support URL specifiers too.

```
import(`/my-module.js?t=${Date.now()}`);
```

Note that this can lead to memory leaks in a long-running application, because the engine cannot safely garbage-collect any module namespace objects. Currently, there is no way to manually clear the cache of module namespace objects.

You can also use the Fetch API to fetch module source code as text, and then evaluate the module manually depending on the module type:

- For JavaScript modules, you can dynamically import the source code as a `blob:` URL in browsers, or use `vm.Module` to evaluate it in Node.js.
- For JSON modules, you can parse the source code using `JSON.parse()`.
- For CSS modules, you can create a new `CSSStyleSheet` object and use its `replace()` method to populate it with the source code.

However, this is semantically not the same as dynamic import, because user-agent settings like fetch destination, CSP, or module resolution may not be applied correctly.

Module namespace object caching only applies to modules that are loaded and linked *successfully*. A module is imported in three steps: loading (fetching the module), linking (mostly, parsing the module), and evaluating (executing the parsed code). Only evaluation failures are cached; if a module fails to load or link, the next import may try to load and link the module again. The browser may or may not cache the result of the fetch operation, but it should follow typical HTTP semantics, so handling such network failures should not be different from handling `fetch()` failures.

Examples

Import a module for its side effects only

```
(async () => {
  if (somethingIsTrue) {
    // import module for side effects
    await import("/modules/my-module.js");
  }
})();
```

If your project uses packages that export ESM, you can also import them for side effects only. This will run the code in the package entry point file (and any files it imports) only.

Importing defaults

If you are destructuring the imported module namespace object, then you must rename the `default` key because `default` is a reserved word.

```
(async () => {
  if (somethingIsTrue) {
```

```
    const {
      default: myDefault,
      foo,
      bar,
    } = await import("/modules/my-module.js");
  }
})();
```

Importing on-demand in response to user action

This example shows how to load functionality on to a page based on a user action, in this case a button click, and then call a function within that module. This is not the only way to implement this functionality. The `import()` function also supports `await`.

```
const main = document.querySelector("main");
for (const link of document.querySelectorAll("nav > a")) {
  link.addEventListener("click", (e) => {
    e.preventDefault();

    import("/modules/my-module.js")
      .then((module) => {
        module.loadPageInto(main);
      })
      .catch((err) => {
        main.textContent = err.message;
      });
  });
}
```

Importing different modules based on environment

In processes such as server-side rendering, you may need to load different logic on server or in browser because they interact with different globals or modules (for example, browser code has access to web APIs like `document` and `navigator`, while server code has access to the server file system). You can do so through a conditional dynamic import.

```
let myModule;

if (typeof window === "undefined") {
  myModule = await import("module-used-on-server");
} else {
  myModule = await import("module-used-in-browser");
}
```

Importing modules with a non-literal specifier

Dynamic imports allow any expression as the module specifier, not necessarily string literals.

Here, we load 10 modules, `/modules/module-0.js`, `/modules/module-1.js`, etc., concurrently, and call the `load` functions that each one exports.

```
Promise.all(
  Array.from({ length: 10 }).map(
    (_, index) => import(`/modules/module-${index}.js`),
  ),
).then((modules) => modules.forEach((module) => module.load()));
```

Using import attributes with dynamic import

Import attributes are accepted as the second parameter of the `import()` syntax.

```
const data = await import("./data.json", {
  with: { type: "json" },
});
```

See also

- `import`

import.defer()

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import.defer()`** syntax behaves like regular `import()` syntax, except that it results in a deferred module namespace object. The module and its dependencies are fetched and linked up front, but their synchronous evaluation is deferred until the namespace's properties are accessed.

For more information about deferred evaluation, including its interaction with top-level `await`, see the `import defer` declaration form.

Syntax

```
import.defer(moduleName)
import.defer(moduleName, options)
```

`import.defer()` is special syntax (a "meta property"), not a method on an `import` object.

Parameters

See `import()`.

Return value

Returns a promise that fulfills with a deferred module namespace object after the module graph is loaded and linked, and any eagerly evaluated top-level `await` dependencies have finished evaluating.

Like regular `import()`, the promise rejects if the module or its dependencies cannot be loaded, parsed, or linked. It also rejects if an eagerly evaluated module throws. Errors from evaluation that remains deferred are instead thrown synchronously by the namespace operation that triggers evaluation.

Examples

Using import.defer()

> **Note:** It's guaranteed that awaiting the resulting promise never accidentally calls an exported `then` method—a gotcha associated with the regular module namespace object—because the deferred module namespace object never exposes a property called `then`.

```
const ts = await import.defer("typescript");

function compilePath(path) {
  // Evaluation of the typescript module subgraph starts here
  const program = ts.createProgram([path], {});
}
```

Never immediately destructure the returned namespace, because it triggers evaluation:

```
const { createProgram } = await import.defer("typescript");
// The typescript module has now been evaluated.
```

See also

- JavaScript modules guide
- `import()`
- `import defer`

import.source()

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import.source()`** syntax behaves like regular `import()` syntax, except that it results in an object that represents the module's compiled source code. The module is fetched and compiled, but its dependencies are not loaded and it is not linked or evaluated. It can be imperatively evaluated later, such as by using dynamic import or `WebAssembly.instantiate()`.

To use `import.source()`, the target module must be of a kind that supports source phase imports. Currently, only WebAssembly modules support source phase imports, and result in `WebAssembly.Module` objects. JavaScript module source objects will be added by the ECMAScript Module Phase Imports proposal.

For more information about the semantics of source phase imports, see the `import source` declaration form.

Syntax

```
import.source(moduleName)
import.source(moduleName, options)
```

`import.source()` is special syntax (a "meta property"), not a method on an `import` object.

Parameters

See `import()`.

Return value

Returns a promise that fulfills with an `AbstractModuleSource` object representing the module's compiled source after the module is loaded and compiled successfully.

Like regular `import()`, the promise rejects if the module cannot be loaded or parsed. It also rejects with a `SyntaxError` if the module type does not support source phase imports. The import does not load dependencies, link, or evaluate the module, so errors from those later steps are not reported.

Examples

Using import.source()

```
const myModuleSource = await import.source("./my-module.wasm");

const instance = await WebAssembly.instantiate(myModuleSource, {
  env: { log: console.log },
```

```
});
const { exports } = instance;
```

See also

- JavaScript modules guide
- `import()`
- `import source`
- `AbstractModuleSource`

import.meta

The **`import.meta`** meta-property exposes context-specific metadata to a JavaScript module. It contains information about the module, such as the module's URL.

Syntax

```
import.meta
```

Value

The `import.meta` object is created by the host environment, as an extensible `null`-prototype object where all properties are writable, configurable, and enumerable. The spec doesn't specify any properties to be defined on it, but hosts usually implement the following properties:

`url`

> The full URL to the module, includes query parameters and/or hash (following the `?` or `#`). In browsers, this is either the URL from which the script was obtained (for external scripts), or the URL of the containing document (for inline scripts). In Node.js, this is the file path (including the `file://` protocol).

`resolve`

> Resolves a module specifier to a URL using the current module's URL as base.

Description

The `import.meta` syntax consists of the keyword `import`, a dot, and the identifier `meta`. Because `import` is a reserved word, not an identifier, this is not a property accessor, but a special expression syntax.

The `import.meta` meta-property is available in JavaScript modules; using `import.meta` outside of a module (including direct `eval()` within a module) is a syntax error.

Examples

Passing query parameters

Using query parameters in the `import` specifier allows module-specific argument passing, which may be complementary to reading parameters from the application-wide `window.location` (or on Node.js, through `process.argv`). For example, with the following HTML:

```
<script type="module">
  import "./index.mjs?someURLInfo=5";
</script>
```

The `index.mjs` module is able to retrieve the `someURLInfo` parameter through `import.meta`:

```
// index.mjs
new URL(import.meta.url).searchParams.get("someURLInfo"); // 5
```

The same applies when a module imports another:

```
// index.mjs
import "./index2.mjs?someURLInfo=5";

// index2.mjs
new URL(import.meta.url).searchParams.get("someURLInfo"); // 5
```

The ES module implementation in Node.js supports resolving module specifiers containing query parameters (or the hash), as in the latter example. However, you cannot use queries or hashes when the module is specified through the CLI command (like `node index.mjs?someURLInfo=5`), because the CLI entrypoint uses a more CommonJS-like resolution mode, treating the path as a file path rather than a URL. To pass parameters to the entrypoint module, use CLI arguments and read them through `process.argv` instead (like `node index.mjs --someURLInfo=5`).

Resolving a file relative to the current one

In Node.js CommonJS modules, there's a `__dirname` variable that contains the absolute path to the folder containing current module, which is useful for resolving relative paths. However, ES modules cannot have contextual variables except for `import.meta`. Therefore, to resolve a relative file you can use `import.meta.url`. Note that this uses URLs rather than filesystem paths.

Before (CommonJS):

```
const fs = require("fs/promises");
const path = require("path");

const filePath = path.join(__dirname, "someFile.txt");
fs.readFile(filePath, "utf8").then(console.log);
```

After (ES modules):

```
import fs from "node:fs/promises";

const fileURL = new URL("./someFile.txt", import.meta.url);
fs.readFile(fileURL, "utf8").then(console.log);
```

See also

- `import`
- `export`

import.meta.resolve()

`import.meta.resolve()` is a built-in function defined on the `import.meta` object of a JavaScript module that resolves a module specifier to a URL using the current module's URL as base.

Syntax

```
import.meta.resolve(moduleName)
```

Parameters

`moduleName`

> A string that specifies a potentially importable module. This may be a relative path (such as `"./lib/helper.js"`), a bare name (such as `"my-module"`), or an absolute URL (such as `"https://example.com/lib/helper.js"`).

Return value

Returns a string corresponding to the path that would be imported if the argument were passed to `import()`.

Description

`import.meta.resolve()` allows a script to access the *module specifier resolution* algorithm for a name, like this:

```
// Script at https://example.com/main.js

const helperPath = import.meta.resolve("./lib/helper.js");
console.log(helperPath); // "https://example.com/lib/helper.js"
```

Note that `import.meta.resolve()` only performs resolution and does not attempt to load or import the resulting path. Therefore, its return value is the same *regardless of whether the returned path corresponds to a file that exists, and regardless of whether that file contains valid code for a module*. This allows `import.meta.resolve()` to be a *synchronous* operation.

It is different from dynamic import, because although both accept a module specifier as the first argument, `import.meta.resolve()` returns the path that *would be imported* without making any attempt to access that path. Therefore, the following two are effectively the same code:

```
// Approach 1
console.log(await import("./lib/helper.js"));

// Approach 2
const helperPath = import.meta.resolve("./lib/helper.js");
console.log(await import(helperPath));
```

However, even if `"./lib/helper.js"` cannot be successfully imported, the second snippet will not encounter an error until it attempts to perform the import on line 2.

Bare module names

You can pass a bare module name (also known as a bare module specifier) to `import.meta.resolve()`, as long as module resolution is defined for the name. For example, you can define this using an import map inside a browser:

```
<!-- index.html -->
<script type="importmap">
  {
    "imports": {
      "my-module": "./modules/my-module/index.js"
    }
  }
</script>

<script type="module">
  const moduleEntryPath = import.meta.resolve("my-module");
  console.log(moduleEntryPath);
</script>
```

Again, since this snippet does not try to import `moduleEntryPath` — neither does the import map — it prints the resolved URL regardless of whether `./modules/my-module/index.js` actually exists.

Comparison with new URL()

The `URL()` constructor accepts a second *base URL* argument. When the first argument is a relative path and the base URL is `import.meta.url`, the effect is similar to `import.meta.resolve()`.

```
const helperPath = new URL("./lib/helper.js", import.meta.url).href;
console.log(helperPath);
```

This is also a useful replacement syntax when targeting older browsers. However, there are some differences:

- `import.meta.resolve()` returns a string, while `new URL()` returns a `URL` object. It is possible to use `href` or `toString()` on the constructed `URL`, but this may still not produce the exact same result in some JavaScript environments or when using tools like bundlers to statically analyze the code.
- `import.meta.resolve()` is aware of additional resolution configurations, such as resolving bare module names using import maps, as described above. `new URL()` is not aware of import maps and treats bare module names as relative paths (i.e., `new URL("my-module", import.meta.url)` means `new URL("./my-module", import.meta.url)`).

Some tools recognize `new URL("./lib/helper.js", import.meta.url).href` as a dependency on `"./lib/helper.js"` (similar to an import), and take this into account for features like bundling, rewriting imports for moved files, "go to source" functionality, etc. However, since `import.meta.resolve()` is less ambiguous and specifically designed to indicate a module path resolution dependency, you should use `import.meta.resolve(moduleName)` instead of `new URL(moduleName, import.meta.url)` for these use cases wherever possible.

Not an ECMAScript feature

`import.meta.resolve()` is not specified or documented as part of the ECMAScript specification for JavaScript modules. Instead, the specification defines the `import.meta` object but leaves all its properties as "host-defined". The WHATWG HTML standard picks up where the ECMAScript standard leaves off, and defines `import.meta.resolve` using its module specifier resolution.

This means that `import.meta.resolve()` is not required to be implemented by all conformant JavaScript implementations. However, `import.meta.resolve()` may also be available in non-browser environments:

- Deno implements compatibility with browser behavior.
- Node.js also implements the `import.meta.resolve()` function, but adds an additional `parent` parameter if you use the `--experimental-import-meta-resolve` flag.

Examples

Resolve a path for the Worker() constructor

`import.meta.resolve()` is particularly valuable for APIs that take a path to a script file as an argument, such as the `Worker()` constructor:

```
// main.js
const workerPath = import.meta.resolve("./worker.js");
const worker = new Worker(workerPath, { type: "module" });
worker.addEventListener("message", console.log);
```

```
// worker.js
self.postMessage("hello!");
```

This is also useful to calculate paths for other workers, such as service workers and shared workers. However, if you are using a relative path to calculate the URL of a service worker, keep in mind that the directory of the resolved path determines its registration scope by default (although a different scope can be specified during registration).

See also

- `import`
- `import()`
- `import.meta`

in

The **`in`** operator returns `true` if the specified property is in the specified object or its prototype chain.

The `in` operator cannot be used to search for values in other collections. To test if a certain value exists in an array, use `Array.prototype.includes()`. For sets, use `Set.prototype.has()`.

Try it

```
const car = { make: "Honda", model: "Accord", year: 1998 };

console.log("make" in car);
// Expected output: true

delete car.make;
if ("make" in car === false) {
  car.make = "Suzuki";
}

console.log(car.make);
// Expected output: "Suzuki"
```

Syntax

```
prop in object
```

```
#prop in object
```

Parameters

`prop`

A string or symbol representing a property name (non-symbols will be coerced to strings). Can also be a private element identifier.

`object`

Object to check if it (or its prototype chain) contains the property with specified name (`prop`).

Exceptions

`TypeError`

Thrown if `object` is not an object (i.e., a primitive).

Description

The `in` operator tests if a string or symbol property is present in an object or its prototype chain. If you want to check for only *non-inherited* properties, use `Object.hasOwn()` instead.

A property may be present in an object but have value `undefined`. Therefore, `"x" in obj` is not the same as `obj.x !== undefined`. To make `in` return `false` after a property is added, use the `delete` operator instead of setting that property's value to `undefined`.

You can also use the `in` operator to check whether a particular private class field or method has been defined in an object. The operator returns `true` if the property is defined, and `false` otherwise. This is known as a *branded check*, because it returns `true` if and only if the object was created with that class constructor, after which you can safely access other private elements as well.

This is a special syntax — the left-hand side of the `in` operator is a property identifier instead of an expression, but unquoted (because otherwise it's a string property, not a private element).

Because accessing private elements on objects unrelated to the current class throws a `TypeError` instead of returning `undefined`, this syntax allows you to shorten:

```
class C {
  #x;
  static isC(obj) {
    try {
      obj.#x;
      return true;
    } catch {
      return false;
    }
  }
}
```

To:

```
class C {
  #x;
  static isC(obj) {
    return #x in obj;
  }
}
```

It also generally avoids the need for dealing with error handling just to access a private element that may be nonexistent.

However, the `in` operator still requires the private element to be declared beforehand in the enclosing class — otherwise, it would throw a `SyntaxError` ("Private field '#x' must be declared in an enclosing class"), the same one as when you try to access an undeclared private element.

```
class C {
  foo() {
    #x in this;
  }
}

new C().foo(); // SyntaxError: Private field '#x' must be declared in an enclosing class
```

Examples

Basic usage

The following examples show some uses of the `in` operator.

```
// Arrays
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
0 in trees; // returns true
3 in trees; // returns true
6 in trees; // returns false
"bay" in trees; // returns false (you must specify the index number, not the value at that index)
"length" in trees; // returns true (length is an Array property)
Symbol.iterator in trees; // returns true
```

```
// Predefined objects
"PI" in Math; // returns true

// Custom objects
const myCar = { make: "Honda", model: "Accord", year: 1998 };
"make" in myCar; // returns true
"model" in myCar; // returns true
```

You must specify an object on the right side of the `in` operator. For example, you can specify a string created with the `String` constructor, but you cannot specify a string literal.

```
const color1 = new String("green");
"length" in color1; // returns true

const color2 = "coral";
// generates an error (color2 is not a String object)
"length" in color2;
```

Using the in operator with deleted or undefined properties

If you delete a property with the `delete` operator, the `in` operator returns `false` for that property.

```
const myCar = { make: "Honda", model: "Accord", year: 1998 };
delete myCar.make;
"make" in myCar; // returns false

const trees = ["redwood", "bay", "cedar", "oak", "maple"];
delete trees[3];
3 in trees; // returns false
```

If you set a property to `undefined` but do not delete it, the `in` operator returns true for that property.

```
const myCar = { make: "Honda", model: "Accord", year: 1998 };
myCar.make = undefined;
"make" in myCar; // returns true
```

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
trees[3] = undefined;
3 in trees; // returns true
```

The `in` operator will return `false` for empty array slots, even if accessing it directly returns `undefined`.

```
const empties = new Array(3);
empties[2]; // returns undefined
2 in empties; // returns false
```

To avoid this, make sure a new array is always filled with non-empty values or not write to indexes past the end of array.

```
const empties = new Array(3).fill(undefined);
2 in empties; // returns true
```

Inherited properties

The `in` operator returns `true` for properties in the prototype chain. This may be undesirable if you are using objects to store arbitrary key-value pairs.

```
const ages = { alice: 18, bob: 27 };

function hasPerson(name) {
  return name in ages;
}

hasPerson("hasOwnProperty"); // true
```

You can use `Object.hasOwn()` to check if the object has the key.

```
const ages = { alice: 18, bob: 27 };

function hasPerson(name) {
  return Object.hasOwn(ages, name);
}

hasPerson("hasOwnProperty"); // false
```

Alternatively, you should consider using a null prototype object or a `Map` for storing `ages`, to avoid other bugs.

```
const ages = new Map([
  ["alice", 18],
  ["bob", 27],
]);
```

```
function hasPerson(name) {
  return ages.has(name);
}

hasPerson("hasOwnProperty"); // false
```

Using the in operator to implement branded checks

The code fragment below demonstrates a static function that tells if an object was created with the `Person` constructor and therefore can perform other methods safely.

```
class Person {
  #age;
  constructor(age) {
    this.#age = age;
  }
  static isPerson(o) {
    return #age in o;
  }
  ageDifference(other) {
    return this.#age - other.#age;
  }
}

const p1 = new Person(20);
const p2 = new Person(30);
console.log(p1.ageDifference(p2)); // -10
console.log(Person.isPerson(p1)); // true

if (Person.isPerson(p1) && Person.isPerson(p2)) {
  console.log(p1.ageDifference(p2)); // -10
}
```

It helps to prevent the following case:

```
const p2 = {};

p1.ageDifference(p2); // TypeError: Cannot read private member #age from an object whose class did not declare it
```

Without the `in` operator, you would have to use a `try...catch` block to check if the object has the private element.

You can also implement this as a `[Symbol.hasInstance]()` method of the class, so that you can use the `instanceof` operator to perform the same check (which, by default, only checks for the existence of `Person.prototype` in the object's prototype chain).

```
class Person {
  #age;
  constructor(age) {
    this.#age = age;
  }
  static [Symbol.hasInstance](o) {
    // Testing `this` to prevent false-positives when
    // calling `instanceof SubclassOfPerson`
    return this === Person && #age in o;
  }
  ageDifference(other) {
    return this.#age - other.#age;
  }
}

const p1 = new Person(20);
const p2 = new Person(30);

if (p1 instanceof Person && p2 instanceof Person) {
  console.log(p1.ageDifference(p2)); // -10
}
```

For more examples, see Private elements and the class guide.

See also

- `for...in`
- `delete`
- `Object.hasOwn()`
- `Reflect.has()`
- Enumerability and ownership of properties

Increment (++)

The **increment (++)** operator increments (adds one to) its operand and returns the value before or after the increment, depending on where the operator is placed.

Try it

```
let x = 3;
const y = x++;

console.log(`x:${x}, y:${y}`);
// Expected output: "x:4, y:3"

let a = 3;
const b = ++a;

console.log(`a:${a}, b:${b}`);
// Expected output: "a:4, b:4"
```

Syntax

```
x++
++x
```

Description

The ++ operator is overloaded for two types of operands: number and BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt increment if the operand becomes a BigInt; otherwise, it performs number increment.

If used postfix, with operator after operand (for example, x++), the increment operator increments and returns the value before incrementing.

If used prefix, with operator before operand (for example, ++x), the increment operator increments and returns the value after incrementing.

The increment operator can only be applied on operands that are references (variables and object properties; i.e., valid assignment targets). ++x itself evaluates to a value, not a reference, so you cannot chain multiple increment operators together.

```
++(++x); // SyntaxError: Invalid left-hand side expression in prefix operation
```

Examples

Postfix increment

```
let x = 3;
const y = x++;
// x is 4; y is 3

let x2 = 3n;
const y2 = x2++;
// x2 is 4n; y2 is 3n
```

Prefix increment

```
let x = 3;
const y = ++x;
// x is 4; y is 4

let x2 = 3n;
const y2 = ++x2;
// x2 is 4n; y2 is 4n
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

Inequality (!=)

The **inequality (!=)** operator checks whether its two operands are not equal, returning a Boolean result. Unlike the strict inequality operator, it attempts to convert and compare operands that are of different types.

Try it

```
console.log(1 != 1);
// Expected output: false

console.log("hello" != "hello");
// Expected output: false

console.log("1" != 1);
// Expected output: false
```

```
console.log(0 != false);
// Expected output: false
```

Syntax

```
x != y
```

Description

The inequality operator checks whether its operands are not equal. It is the negation of the equality operator so the following two lines will always give the same result:

```
x != y;

!(x == y);
```

For details of the comparison algorithm, see the page for the equality operator.

Like the equality operator, the inequality operator will attempt to convert and compare operands of different types:

```
3 != "3"; // false
```

To prevent this, and require that different types are considered to be different, use the strict inequality operator instead:

```
3 !== "3"; // true
```

Examples

Comparison with no type conversion

```
1 != 2; // true
"hello" != "hola"; // true

1 != 1; // false
"hello" != "hello"; // false
```

Comparison with type conversion

```
"1" != 1; // false
1 != "1"; // false
0 != false; // false
0 != null; // true
0 != undefined; // true
0 != !!null; // false, look at Logical NOT operator
0 != !!undefined; // false, look at Logical NOT operator
null != undefined; // false

const number1 = new Number(3);
const number2 = new Number(3);
number1 != 3; // false
number1 != number2; // true
```

Comparison of objects

```
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 != object2); // true
console.log(object1 != object1); // false
```

See also

- Equality (==)
- Strict equality (===)
- Strict inequality (!==)

instanceof

The **`instanceof`** operator tests to see if the `prototype` property of a constructor appears anywhere in the prototype chain of an object. The return value is a boolean value. Its behavior can be customized with `Symbol.hasInstance`.

Try it

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
```

```
  this.year = year;
}
const auto = new Car("Honda", "Accord", 1998);

console.log(auto instanceof Car);
// Expected output: true

console.log(auto instanceof Object);
// Expected output: true
```

Syntax

```
object instanceof constructor
```

Parameters

`object`

The object to test.

`constructor`

Constructor to test against.

Exceptions

`TypeError`

Thrown if `constructor` is not an object. If `constructor` doesn't have a `[Symbol.hasInstance]()` method, it must also be a function.

Description

The `instanceof` operator tests the presence of `constructor.prototype` in `object`'s prototype chain. This usually (though not always) means `object` was constructed with `constructor`.

```
// defining constructors
function C() {}
function D() {}

const o = new C();

// true, because: Object.getPrototypeOf(o) === C.prototype
o instanceof C;

// false, because D.prototype is nowhere in o's prototype chain
o instanceof D;

o instanceof Object; // true, because:
C.prototype instanceof Object; // true

// Re-assign `constructor.prototype`: you should
// rarely do this in practice.
C.prototype = {};
const o2 = new C();

o2 instanceof C; // true

// false, because C.prototype is nowhere in
// o's prototype chain anymore
o instanceof C;

D.prototype = new C(); // add C to [[Prototype]] linkage of D
const o3 = new D();
o3 instanceof D; // true
o3 instanceof C; // true since C.prototype is now in o3's prototype chain
```

Note that the value of an `instanceof` test can change if `constructor.prototype` is re-assigned after creating the object (which is usually discouraged). It can also be changed by changing `object`'s prototype using `Object.setPrototypeOf`.

Classes behave in the same way, because classes also have the `prototype` property.

```
class A {}
class B extends A {}

const o1 = new A();
// true, because Object.getPrototypeOf(o1) === A.prototype
o1 instanceof A;
// false, because B.prototype is nowhere in o1's prototype chain
o1 instanceof B;

const o2 = new B();
// true, because Object.getPrototypeOf(Object.getPrototypeOf(o2)) === A.prototype
o2 instanceof A;
```

```
// true, because Object.getPrototypeOf(o2) === B.prototype
o2 instanceof B;
```

For bound functions, `instanceof` looks up for the `prototype` property on the target function, since bound functions don't have `prototype`.

```
class Base {}
const BoundBase = Base.bind(null, 1, 2);
console.log(new Base() instanceof BoundBase); // true
```

instanceof and Symbol.hasInstance

If `constructor` has a `Symbol.hasInstance` method, the method will be called in priority, with `object` as its only argument and `constructor` as `this`.

```
// This class allows plain objects to be disguised as this class's instance,
// as long as the object has a particular flag as its property.
class Forgeable {
  static isInstanceFlag = Symbol("isInstanceFlag");

  static [Symbol.hasInstance](obj) {
    return Forgeable.isInstanceFlag in obj;
  }
}

const obj = { [Forgeable.isInstanceFlag]: true };
console.log(obj instanceof Forgeable); // true
```

Because all functions inherit from `Function.prototype` by default, most of the time, the `Function.prototype[Symbol.hasInstance]()` method specifies the behavior of `instanceof` when the right-hand side is a function. See the `Symbol.hasInstance` page for the exact algorithm of `instanceof`.

instanceof and multiple realms

JavaScript execution environments (windows, frames, etc.) are each in their own *realm*. This means that they have different built-ins (different global object, different constructors, etc.). This may result in unexpected results. For instance, `[] instanceof window.frames[0].Array` will return `false`, because `Array.prototype !== window.frames[0].Array.prototype` and arrays in the current realm inherit from the former.

This may not make sense at first, but for scripts dealing with multiple frames or windows, and passing objects from one context to another via functions, this will be a valid and strong issue. For instance, you can securely check if a given object is in fact an Array using `Array.isArray()`, neglecting which realm it comes from.

For example, to check if a `Node` is an `SVGElement` in a different context, you can use `myNode instanceof myNode.ownerDocument.defaultView.SVGElement`.

Examples

Using instanceof with String

The following example shows the behavior of `instanceof` with `String` objects.

```
const literalString = "This is a literal string";
const stringObject = new String("String created with constructor");

literalString instanceof String; // false, string primitive is not a String
stringObject instanceof String; // true

literalString instanceof Object; // false, string primitive is not an Object
stringObject instanceof Object; // true

stringObject instanceof Date; // false
```

Using instanceof with Map

The following example shows the behavior of `instanceof` with `Map` objects.

```
const myMap = new Map();

myMap instanceof Map; // true
myMap instanceof Object; // true
myMap instanceof String; // false
```

Objects created using Object.create()

The following example shows the behavior of `instanceof` with objects created using `Object.create()`.

```
function Shape() {}

function Rectangle() {
  Shape.call(this); // call super constructor.
}

Rectangle.prototype = Object.create(Shape.prototype);

Rectangle.prototype.constructor = Rectangle;

const rect = new Rectangle();
```

```
rect instanceof Object; // true
rect instanceof Shape; // true
rect instanceof Rectangle; // true
rect instanceof String; // false

const literalObject = {};
const nullObject = Object.create(null);
nullObject.name = "My object";

literalObject instanceof Object; // true, every object literal has Object.prototype as prototype
({}) instanceof Object; // true, same case as above
nullObject instanceof Object; // false, prototype is end of prototype chain (null)
```

Demonstrating that myCar is of type Car and type Object

The following code creates an object type `Car` and an instance of that object type, `myCar`. The `instanceof` operator demonstrates that the `myCar` object is of type `Car` and of type `Object`.

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
const myCar = new Car("Honda", "Accord", 1998);
const a = myCar instanceof Car; // returns true
const b = myCar instanceof Object; // returns true
```

Not an instanceof

To test if an object is not an `instanceof` a specific constructor, you can do:

```
if (!(myCar instanceof Car)) {
  // Do something, like:
  // myCar = new Car(myCar)
}
```

This is really different from:

```
if (!myCar instanceof Car) {
  // unreachable code
}
```

This will always be `false`. (`!myCar` will be evaluated before `instanceof`, so you always try to know if a boolean is an instance of `Car`).

Overriding the behavior of instanceof

A common pitfall of using `instanceof` is believing that, if `x instanceof C`, then `x` was created using `C` as constructor. This is not true, because `x` could be directly assigned with `C.prototype` as its prototype. In this case, if your code reads private fields of `C` from `x`, it would still fail:

```
class C {
  #value = "foo";
  static getValue(x) {
    return x.#value;
  }
}

const x = { __proto__: C.prototype };

if (x instanceof C) {
  console.log(C.getValue(x)); // TypeError: Cannot read private member #value from an object whose class did not declare it
}
```

To avoid this, you can override the behavior of `instanceof` by adding a `Symbol.hasInstance` method to `C`, so that it does a branded check with `in`:

```
class C {
  #value = "foo";

  static [Symbol.hasInstance](x) {
    return #value in x;
  }

  static getValue(x) {
    return x.#value;
  }
}

const x = { __proto__: C.prototype };

if (x instanceof C) {
  // Doesn't run, because x is not a C
  console.log(C.getValue(x));
}
```

Note that you may want to limit this behavior to the current class; otherwise, it could lead to false positives for subclasses:

```
class D extends C {}
console.log(new C() instanceof D); // true; because D inherits [Symbol.hasInstance] from C
```

You could do this by checking that `this` is the current constructor:

```
class C {
  #value = "foo";

  static [Symbol.hasInstance](x) {
    return this === C && #value in x;
  }
}

class D extends C {}
console.log(new C() instanceof D); // false
console.log(new C() instanceof C); // true
console.log({ __proto__: C.prototype } instanceof C); // false
```

See also

- `typeof`
- `Symbol.hasInstance`
- `Object.prototype.isPrototypeOf`

Left shift (<<)

The **left shift (<<)** operator returns a number or BigInt whose binary representation is the first operand shifted by the specified number of bits to the left. Excess bits shifted off to the left are discarded, and zero bits are shifted in from the right.

Try it

```
const a = 5; // 00000000000000000000000000000101
const b = 2; // 00000000000000000000000000000010

console.log(a << b); // 00000000000000000000000000010100
// Expected output: 20
```

Syntax

```
x << y
```

Description

The `<<` operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt left shift if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number left shift. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the left operand's bit representation in two's complement. For example, `9 << 2` yields 36:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
                  --------------------------------
9 << 2 (base 10): 00000000000000000000000000100100 (base 2) = 36 (base 10)
```

Bitwise a 32-bit integer `x` to the left by `y` bits yields `x * 2 ** y`. So for example, `9 << 3` is equivalent to `9 * (2 ** 3) = 9 * (8) = 72`.

If the left operand is a number with more than 32 bits, it will get the most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

The right operand will be converted to an unsigned 32-bit integer and then taken modulo 32, so the actual shift offset will always be a positive integer between 0 and 31, inclusive. For example, `100 << 32` is the same as `100 << 0` (and produces `100`) because 32 modulo 32 is 0.

> **Warning:** You may see people using `<< 0` to truncate numbers to integers. Left shifting any number `x` by `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using left shift

```
9 << 3; // 72

// 9 * (2 ** 3) = 9 * (8) = 72
```

```
9n << 3n; // 72n
```

See also

- Bitwise operators in the JS guide
- Left shift assignment (<<=)

Left shift assignment (<<=)

The **left shift assignment (<<=)** operator performs left shift on the two operands and assigns the result to the left operand.

Try it

```
let a = 5; // 00000000000000000000000000000101

a <<= 2; // 00000000000000000000000000010100

console.log(a);
// Expected output: 20
```

Syntax

```
x <<= y
```

Description

`x <<= y` is equivalent to `x = x << y`, except that the expression `x` is only evaluated once.

Examples

Using left shift assignment

```
let a = 5;
// 00000000000000000000000000000101

a <<= 2; // 20
// 00000000000000000000000000010100

let b = 5n;
b <<= 2n; // 20n
```

See also

- Assignment operators in the JS guide
- Left shift (<<)

Less than (<)

The **less than (<)** operator returns `true` if the left operand is less than the right operand, and `false` otherwise.

Try it

```
console.log(5 < 3);
// Expected output: false

console.log(3 < 3);
// Expected output: false

// Compare bigint to number
console.log(3n < 5);
// Expected output: true

console.log("aa" < "ab");
// Expected output: true
```

Syntax

```
x < y
```

Description

The operands are compared with multiple rounds of coercion, which can be summarized as follows:

- First, objects are converted to primitives by calling its `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. The left operand is always coerced before the right one. Note that although `[Symbol.toPrimitive]()` is called with the `"number"` hint (meaning there's a slight preference for the object to become a number), the return value is not converted to a number, since strings are still specially handled.
- If both values are strings, they are compared as strings, based on the values of the UTF-16 code units (not Unicode code points) they contain.
- Otherwise JavaScript attempts to convert non-numeric types to numeric values:
  - Boolean values `true` and `false` are converted to 1 and 0 respectively.
  - `null` is converted to 0.

    - `undefined` is converted to `NaN`.
    - Strings are converted based on the values they contain, and are converted as `NaN` if they do not contain numeric values.
- If either value is `NaN`, the operator returns `false`.
- Otherwise the values are compared as numeric values. BigInt and number values can be compared together.

Other operators, including `>`, `>=`, and `<=`, use the same algorithm as `<`. There are two cases where all four operators return `false`:

- If one of the operands gets converted to a BigInt, while the other gets converted to a string that cannot be converted to a BigInt value (it throws a syntax error when passed to `BigInt()`).
- If one of the operands gets converted to `NaN`. (For example, strings that cannot be converted to numbers, or `undefined`.)

For all other cases, the four operators have the following relationships:

```
x < y === !(x >= y);
x <= y === !(x > y);
x > y === y < x;
x >= y === y <= x;
```

> **Note:** One observable difference between `<` and `>` is the order of coercion, especially if the coercion to primitive has side effects. All comparison operators coerce the left operand before the right operand.

Examples

String to string comparison

```
"a" < "b"; // true
"a" < "a"; // false
"a" < "3"; // false

"\uD855\uDE51" < "\uFF3A"; // true
```

String to number comparison

```
"5" < 3; // false
"3" < 3; // false
"3" < 5; // true

"hello" < 5; // false
5 < "hello"; // false

"5" < 3n; // false
"3" < 5n; // true
```

Number to Number comparison

```
5 < 3; // false
3 < 3; // false
3 < 5; // true
```

Number to BigInt comparison

```
5n < 3; // false
3 < 5n; // true
```

Comparing Boolean, null, undefined, NaN

```
true < false; // false
false < true; // true

0 < true; // true
true < 1; // false

null < 0; // false
null < 1; // true

undefined < 3; // false
3 < undefined; // false

3 < NaN; // false
NaN < 3; // false
```

Comparison with side effects

Comparisons always coerce their operands to primitives. This means the same object may end up having different values within one comparison expression. For example, you may have two values that are both greater than and less than the other.

```
class Mystery {
  static #coercionCount = -1;
  valueOf() {
    Mystery.#coercionCount++;
```

```
    // The left operand is coerced first, so this will return 0
    // Then it returns 1 for the right operand
    return Mystery.#coercionCount % 2;
  }
}

const l = new Mystery();
const r = new Mystery();
console.log(l < r && r < l);
// true
```

> **Warning:** This can be a source of confusion. If your objects provide custom primitive conversion logic, make sure it is *idempotent*: multiple coercions should return the same value.

See also

- Greater than (`>`)
- Greater than or equal (`>=`)
- Less than or equal (`<=`)

Less than or equal (`<=`)

The **less than or equal (`<=`)** operator returns `true` if the left operand is less than or equal to the right operand, and `false` otherwise.

Try it

```
console.log(5 <= 3);
// Expected output: false

console.log(3 <= 3);
// Expected output: true

// Compare bigint to number
console.log(3n <= 5);
// Expected output: true

console.log("aa" <= "ab");
// Expected output: true
```

Syntax

```
x <= y
```

Description

The operands are compared using the same algorithm as the Less than operator, with the operands swapped and the result negated. `x <= y` is generally equivalent to `!(y < x)`, except for two cases where `x <= y` and `x > y` are both `false`:

- If one of the operands gets converted to a BigInt, while the other gets converted to a string that cannot be converted to a BigInt value (it throws a syntax error when passed to `BigInt()`).
- If one of the operands gets converted to `NaN`. (For example, strings that cannot be converted to numbers, or `undefined`.)

In addition, `x <= y` coerces `x` to a primitive before `y`, while `y < x` coerces `y` to a primitive before `x`. Because coercion may have side effects, the order of the operands may matter.

`x <= y` is generally equivalent to `x < y || x == y`, except for a few cases:

- When one of `x` or `y` is `null`, and the other is something that's not `null` and becomes 0 when coerced to numeric (including `0`, `0n`, `false`, `""`, `"0"`, `new Date(0)`, etc.): `x <= y` is `true`, while `x < y || x == y` is `false`.
- When one of `x` or `y` is `undefined`, and the other is one of `null` or `undefined`: `x <= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are the same object that becomes `NaN` after the first step of Less than (such as `new Date(NaN)`): `x <= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are different objects that become the same value after the first step of Less than: `x <= y` is `true`, while `x < y || x == y` is `false`.

Examples

String to string comparison

```
"a" <= "b"; // true
"a" <= "a"; // true
"a" <= "3"; // false
```

String to number comparison

```
"5" <= 3; // false
"3" <= 3; // true
"3" <= 5; // true

"hello" <= 5; // false
5 <= "hello"; // false
```

Number to Number comparison

```
5 <= 3; // false
3 <= 3; // true
3 <= 5; // true
```

Number to BigInt comparison

```
5n <= 3; // false
3 <= 3n; // true
3 <= 5n; // true
```

Comparing Boolean, null, undefined, NaN

```
true <= false; // false
true <= true; // true
false <= true; // true

true <= 0; // false
true <= 1; // true

null <= 0; // true
1 <= null; // false

undefined <= 3; // false
3 <= undefined; // false

3 <= NaN; // false
NaN <= 3; // false
```

See also

- Greater than (>)
- Greater than or equal (>=)
- Less than (<)

Logical AND (&&)

The **logical AND (`&&`)** (logical conjunction) operator for a set of boolean operands will be `true` if and only if all the operands are `true`. Otherwise it will be `false`.

More generally, the operator returns the value of the first falsy operand encountered when evaluating from left to right, or the value of the last operand if they are all truthy.

Try it

```
const a = 3;
const b = -2;

console.log(a > 0 && b > 0);
// Expected output: false
```

Syntax

```
x && y
```

Description

Logical AND (`&&`) evaluates operands from left to right, returning immediately with the value of the first falsy operand it encounters; if all values are truthy, the value of the last operand is returned.

If a value can be converted to `true`, the value is so-called truthy. If a value can be converted to `false`, the value is so-called falsy.

Examples of expressions that can be converted to false are:

- `false`;
- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or ``` ``);
- `undefined`.

The AND operator preserves non-Boolean values and returns them as they are:

```
result = "" && "foo"; // result is assigned "" (empty string)
result = 2 && 0; // result is assigned 0
result = "foo" && 4; // result is assigned 4
```

Even though the `&&` operator can be used with non-Boolean operands, it is still considered a boolean operator since its return value can always be converted to a boolean primitive. To explicitly convert its return value (or any expression in general) to the corresponding boolean value, use a double NOT operator or the `Boolean` constructor.

Short-circuit evaluation

The logical AND expression is a short-circuit operator. As each operand is converted to a boolean, if the result of one conversion is found to be `false`, the AND operator stops and returns the original value of that falsy operand; it does **not** evaluate any of the remaining operands.

Consider the pseudocode below.

```
(some falsy expression) && expr
```

The `expr` part is **never evaluated** because the first operand `(some falsy expression)` is evaluated as falsy. If `expr` is a function, the function is never called. See the example below:

```
function A() {
  console.log("called A");
  return false;
}
function B() {
  console.log("called B");
  return true;
}

console.log(A() && B());
// Logs "called A" to the console due to the call for function A,
// && evaluates to false (function A returns false), then false is logged to the console;
// the AND operator short-circuits here and ignores function B
```

Operator precedence

The AND operator has a higher precedence than the OR operator, meaning the `&&` operator is executed before the `||` operator (see operator precedence).

```
true || false && false; // true
true && (false || false); // false
(2 === 3) || (4 < 0) && (1 === 1); // false
```

Examples

Using AND

The following code shows examples of the `&&` (logical AND) operator.

```
a1 = true && true; // t && t returns true
a2 = true && false; // t && f returns false
a3 = false && true; // f && t returns false
a4 = false && 3 === 4; // f && f returns false
a5 = "Cat" && "Dog"; // t && t returns "Dog"
a6 = false && "Cat"; // f && t returns false
a7 = "Cat" && false; // t && f returns false
a8 = "" && false; // f && f returns ""
a9 = false && ""; // f && f returns false
```

Conversion rules for booleans

Converting AND to OR

The following operation involving **booleans**:

```
bCondition1 && bCondition2
```

is always equal to:

```
!(!bCondition1 || !bCondition2)
```

Converting OR to AND

The following operation involving **booleans**:

```
bCondition1 || bCondition2
```

is always equal to:

```
!(!bCondition1 && !bCondition2)
```

Removing nested parentheses

As logical expressions are evaluated left to right, it is always possible to remove parentheses from a complex expression provided that certain rules are followed.

The following composite operation involving **booleans**:

```
bCondition1 || (bCondition2 && bCondition3)
```

is always equal to:

```
bCondition1 || bCondition2 && bCondition3
```

See also

- `Boolean`
- Truthy
- Falsy

Logical AND assignment (&&=)

The **logical AND assignment (`&&=`)** operator only evaluates the right operand and assigns to the left if the left operand is truthy.

Try it

```
let a = 1;
let b = 0;

a &&= 2;
console.log(a);
// Expected output: 2

b &&= 2;
console.log(b);
// Expected output: 0
```

Syntax

```
x &&= y
```

Description

Logical AND assignment *short-circuits*, meaning that `x &&= y` is equivalent to `x && (x = y)`, except that the expression `x` is only evaluated once.

No assignment is performed if the left-hand side is not truthy, due to short-circuiting of the logical AND operator. For example, the following does not throw an error, despite `x` being `const`:

```
const x = 0;
x &&= 2;
```

Neither would the following trigger the setter:

```
const x = {
  get value() {
    return 0;
  },
  set value(v) {
    console.log("Setter called");
  },
};

x.value &&= 2;
```

In fact, if `x` is not truthy, `y` is not evaluated at all.

```
const x = 0;
x &&= console.log("y evaluated");
// Logs nothing
```

Examples

Using logical AND assignment

```
let x = 0;
let y = 1;

x &&= 0; // 0
x &&= 1; // 0
y &&= 1; // 1
y &&= 0; // 0
```

See also

- Logical AND (`&&`)
- Nullish coalescing operator (`??`)
- Bitwise AND assignment (`&=`)
- Truthy
- Falsy

Logical NOT (!)

The **logical NOT (`!`)** (logical complement, negation) operator takes truth to falsity and vice versa. It is typically used with boolean (logical) values. When used with non-Boolean values, it returns `false` if its single operand can be converted to `true`; otherwise, returns `true`.

Try it

```
const a = 3;
const b = -2;

console.log(!(a > 0 || b > 0));
// Expected output: false
```

Syntax

```
!x
```

Description

Returns `false` if its single operand can be converted to `true`; otherwise, returns `true`.

If a value can be converted to `true`, the value is so-called truthy. If a value can be converted to `false`, the value is so-called falsy.

Examples of expressions that can be converted to false are:

- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or `` ``);
- `undefined`.

Even though the `!` operator can be used with operands that are not Boolean values, it can still be considered a boolean operator since its return value can always be converted to a boolean primitive. To explicitly convert its return value (or any expression in general) to the corresponding boolean value, use a double NOT operator (`!!`) or the `Boolean` constructor.

Examples

Using NOT

The following code shows examples of the `!` (logical NOT) operator.

```
!true; // !t returns false
!false; // !f returns true
!""; // !f returns true
!"Cat"; // !t returns false
```

Double NOT (<code>!!</code>)

It is possible to use a couple of NOT operators in series to explicitly force the conversion of any value to the corresponding boolean primitive. The conversion is based on the "truthiness" or "falsiness" of the value (see truthy and falsy).

The same conversion can be done through the `Boolean()` function.

```
!!true; // !!truthy returns true
!!{}; // !!truthy returns true: any object is truthy…
!!new Boolean(false); // … even Boolean objects with a false .valueOf()!
!!false; // !!falsy returns false
!!""; // !!falsy returns false
!!Boolean(false); // !!falsy returns false
```

Converting between NOTs

The following operation involving **booleans**:

```
!!bCondition
```

is always equal to:

```
bCondition
```

See also

- `Boolean`
- Truthy
- Falsy

Logical OR (||)

The **logical OR (`||`)** (logical disjunction) operator for a set of operands is true if and only if one or more of its operands is true. It is typically used with boolean (logical) values. When it is, it returns a Boolean value. However, the `||` operator actually returns the value of one of the specified operands, so if this operator is used with non-Boolean values, it will return a non-Boolean value.

Try it

```
const a = 3;
const b = -2;

console.log(a > 0 || b > 0);
// Expected output: true
```

Syntax

```
x || y
```

Description

If `x` can be converted to `true`, returns `x`; else, returns `y`.

If a value can be converted to `true`, the value is so-called truthy. If a value can be converted to `false`, the value is so-called falsy.

Examples of expressions that can be converted to false are:

- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or ``` ``);
- `undefined`.

Even though the `||` operator can be used with operands that are not Boolean values, it can still be considered a boolean operator since its return value can always be converted to a boolean primitive. To explicitly convert its return value (or any expression in general) to the corresponding boolean value, use a double NOT operator or the `Boolean()` constructor.

Short-circuit evaluation

The logical OR expression is evaluated left to right, it is tested for possible "short-circuit" evaluation using the following rule:

`(some truthy expression) || expr` is short-circuit evaluated to the truthy expression.

Short circuit means that the `expr` part above is **not evaluated**, hence any side effects of doing so do not take effect (e.g., if `expr` is a function call, the calling never takes place). This happens because the value of the operator is already determined after the evaluation of the first operand. See example:

```
function A() {
  console.log("called A");
  return false;
}
function B() {
  console.log("called B");
  return true;
}

console.log(B() || A());
// Logs "called B" due to the function call,
// then logs true (which is the resulting value of the operator)
```

Operator precedence

The following expressions might seem equivalent, but they are not, because the `&&` operator is executed before the `||` operator (see operator precedence).

```
true || false && false; // returns true, because && is executed first
(true || false) && false; // returns false, because grouping has the highest precedence
```

Examples

Using OR

The following code shows examples of the `||` (logical OR) operator.

```
true || true; // t || t returns true
false || true; // f || t returns true
true || false; // t || f returns true
false || 3 === 4; // f || f returns false
"Cat" || "Dog"; // t || t returns "Cat"
false || "Cat"; // f || t returns "Cat"
"Cat" || false; // t || f returns "Cat"
"" || false; // f || f returns false
false || ""; // f || f returns ""
false || varObject; // f || object returns varObject
```

> **Note:** If you use this operator to provide a default value to some variable, be aware that any *falsy* value will not be used. If you only need to filter out `null` or `undefined`, consider using the nullish coalescing operator.

Conversion rules for booleans

Converting AND to OR

The following operation involving **booleans**:

```
bCondition1 && bCondition2
```

is always equal to:

```
!(!bCondition1 || !bCondition2)
```

Converting OR to AND

The following operation involving **booleans**:

```
bCondition1 || bCondition2
```

is always equal to:

```
!(!bCondition1 && !bCondition2)
```

Removing nested parentheses

As logical expressions are evaluated left to right, it is always possible to remove parentheses from a complex expression following some rules.

The following composite operation involving **booleans**:

```
bCondition1 && (bCondition2 || bCondition3)
```

is always equal to:

```
!(!bCondition1 || !bCondition2 && !bCondition3)
```

See also

- Nullish coalescing operator (`??`)
- `Boolean`
- Truthy
- Falsy

Logical OR assignment (`||=`)

The **logical OR assignment (`||=`)** operator only evaluates the right operand and assigns to the left if the left operand is falsy.

Try it

```
const a = { duration: 50, title: "" };

a.duration ||= 10;
console.log(a.duration);
// Expected output: 50

a.title ||= "title is empty.";
console.log(a.title);
// Expected output: "title is empty."
```

Syntax

```
x ||= y
```

Description

Logical OR assignment *short-circuits*, meaning that `x ||= y` is equivalent to `x || (x = y)`, except that the expression `x` is only evaluated once.

No assignment is performed if the left-hand side is not falsy, due to short-circuiting of the logical OR operator. For example, the following does not throw an error, despite `x` being `const`:

```
const x = 1;
x ||= 2;
```

Neither would the following trigger the setter:

```
const x = {
  get value() {
    return 1;
  },
  set value(v) {
    console.log("Setter called");
  },
};
```

```
x.value ||= 2;
```

In fact, if `x` is not falsy, `y` is not evaluated at all.

```
const x = 1;
x ||= console.log("y evaluated");
// Logs nothing
```

Examples

Setting default content

If the "lyrics" element is empty, display a default value:

```
document.getElementById("lyrics").textContent ||= "No lyrics.";
```

Here the short-circuit is especially beneficial, since the element will not be updated unnecessarily and won't cause unwanted side-effects such as additional parsing or rendering work, or loss of focus, etc.

Note: Pay attention to the value returned by the API you're checking against. If an empty string is returned (a falsy value), `||=` must be used, so that "No lyrics." is displayed instead of a blank space. However, if the API returns `null` or `undefined` in case of blank content, `??=` should be used instead.

See also

- Logical OR (`||`)
- Nullish coalescing operator (`??`)
- Bitwise OR assignment (`|=`)
- Truthy
- Falsy

Multiplication (*)

The **multiplication (*)** operator produces the product of the operands.

Try it

```
console.log(3 * 4);
// Expected output: 12

console.log(-3 * 4);
// Expected output: -12

console.log("3" * 2);
// Expected output: 6

console.log("foo" * 2);
// Expected output: NaN
```

Syntax

```
x * y
```

Description

The `*` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt multiplication if both operands become BigInts; otherwise, it performs number multiplication. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

Examples

Multiplication using numbers

```
2 * 2; // 4
-2 * 2; // -4

Infinity * 0; // NaN
Infinity * Infinity; // Infinity
```

Other non-BigInt values are coerced to numbers:

```
"foo" * 2; // NaN
"2" * 2; // 4
```

Multiplication using BigInts

```
2n * 2n; // 4n
-2n * 2n; // -4n
```

You cannot mix BigInt and number operands in multiplication.

```
2n * 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 * 2n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do multiplication with a BigInt and a non-BigInt, convert either operand:

```
2n * BigInt(2); // 4n
Number(2n) * 2; // 4
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

Multiplication assignment (*=)

The **multiplication assignment (*=)** operator performs multiplication on the two operands and assigns the result to the left operand.

Try it

```
let a = 2;

console.log((a *= 3));
// Expected output: 6

console.log((a *= "hello"));
// Expected output: NaN
```

Syntax

```
x *= y
```

Description

`x *= y` is equivalent to `x = x * y`, except that the expression `x` is only evaluated once.

Examples

Multiplication assignment using numbers

```
let bar = 5;
bar *= 2; // 10
```

Other non-BigInt values are coerced to numbers:

```
let bar = 5;
bar *= "foo"; // NaN
```

Multiplication assignment using BigInts

```
let foo = 3n;
foo *= 2n; // 6n
foo *= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

See also

- Assignment operators in the JS guide
- Multiplication (*)

new

The **`new`** operator lets developers create an instance of a user-defined object type or of one of the built-in object types that has a constructor function.

Try it

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}

const car1 = new Car("Eagle", "Talon TSi", 1993);
```

```
console.log(car1.make);
// Expected output: "Eagle"
```

Syntax

```
new constructor
new constructor()
new constructor(arg1)
new constructor(arg1, arg2)
new constructor(arg1, arg2, /* …, */ argN)
```

Parameters

`constructor`

A class or function that specifies the type of the object instance. The expression can be anything with sufficient precedence, including an identifier, a property access, or another `new` expression, but optional chaining is not allowed.

`arg1, arg2, ..., argN`

A list of values that the `constructor` will be called with. `new Foo` is equivalent to `new Foo()`, i.e., if no argument list is specified, `Foo` is called without arguments.

Description

When a function is called with the **`new`** keyword, the function will be used as a constructor. `new` will do the following things:

1. Creates a blank, plain JavaScript object. For convenience, let's call it `newInstance`.
2. Points `newInstance`'s [[Prototype]] to the constructor function's `prototype` property, if the `prototype` is an `Object`. Otherwise, `newInstance` stays as a plain object with `Object.prototype` as its [[Prototype]].

   > **Note:** Properties/objects added to the constructor function's `prototype` property are therefore accessible to all instances created from the constructor function.

3. Executes the constructor function with the given arguments, binding `newInstance` as the `this` context (i.e., all references to `this` in the constructor function now refer to `newInstance`).
4. If the constructor function returns a non-primitive, this return value becomes the result of the whole `new` expression. Otherwise, if the constructor function doesn't return anything or returns a primitive, `newInstance` is returned instead. (Normally constructors don't return a value, but they can choose to do so to override the normal object creation process.)

Classes can only be instantiated with the `new` operator — attempting to call a class without `new` will throw a `TypeError`.

Creating an object with a user-defined constructor function requires two steps:

1. Define the object type by writing a function that specifies its name and properties. For example, a constructor function to create an object `Foo` might look like this:

   ```
   function Foo(bar1, bar2) {
     this.bar1 = bar1;
     this.bar2 = bar2;
   }
   ```

2. Create an instance of the object with `new`.

   ```
   const myFoo = new Foo("Bar 1", 2021);
   ```

> **Note:** An object can have a property that is itself another object. See the examples below.

You can always add a property to a previously defined object instance. For example, the statement `car1.color = "black"` adds a property `color` to `car1`, and assigns it a value of `"black"`.

However, this does not affect any other objects. To add the new property to all objects of the same type, you must add the property to the constructor's `prototype` property. This defines a property that is shared by all objects created with that function, rather than by just one instance of the object type. The following code adds a `color` property with value `"original color"` to all objects of type `Car`, and then overwrites that value with the string `"black"` only in the instance object `car1`. For more information, see prototype.

```
function Car() {}
const car1 = new Car();
const car2 = new Car();

console.log(car1.color); // undefined

Car.prototype.color = "original color";
console.log(car1.color); // 'original color'

car1.color = "black";
console.log(car1.color); // 'black'
```

```
console.log(Object.getPrototypeOf(car1).color); // 'original color'
console.log(Object.getPrototypeOf(car2).color); // 'original color'
console.log(car1.color); // 'black'
console.log(car2.color); // 'original color'
```

> **Note:** While the constructor function can be invoked like any regular function (i.e., without the `new` operator), in this case a new object is not created and the value of `this` is also different.

A function can know whether it is invoked with `new` by checking `new.target`. `new.target` is only `undefined` when the function is invoked without `new`. For example, you can have a function that behaves differently when it's called versus when it's constructed:

```
function Car(color) {
  if (!new.target) {
    // Called as function.
    return `${color} car`;
  }
  // Called with new.
  this.color = color;
}

const a = Car("red"); // a is "red car"
const b = new Car("red"); // b is `Car { color: "red" }`
```

Prior to ES6, which introduced classes, most JavaScript built-ins are both callable and constructible, although many of them exhibit different behaviors. To name a few:

- `Array()`, `Error()`, and `Function()` behave the same when called as a function or a constructor.
- `Boolean()`, `Number()`, and `String()` coerce their argument to the respective primitive type when called, and return wrapper objects when constructed.
- `Date()` returns a string representing the current date when called, equivalent to `new Date().toString()`.

After ES6, the language is stricter about which are constructors and which are functions. For example:

- `Symbol()` and `BigInt()` can only be called without `new`. Attempting to construct them will throw a `TypeError`.
- `Proxy` and `Map` can only be constructed with `new`. Attempting to call them will throw a `TypeError`.

Examples

Object type and object instance

Suppose you want to create an object type for cars. You want this type of object to be called `Car`, and you want it to have properties for make, model, and year. To do this, you would write the following function:

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
```

Now you can create an object called `myCar` as follows:

```
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

This statement creates `myCar` and assigns it the specified values for its properties. Then the value of `myCar.make` is the string "Eagle", `myCar.year` is the integer 1993, and so on.

You can create any number of `car` objects by calls to `new`. For example:

```
const kensCar = new Car("Nissan", "300ZX", 1992);
```

Object property that is itself another object

Suppose you define an object called `Person` as follows:

```
function Person(name, age, sex) {
  this.name = name;
  this.age = age;
  this.sex = sex;
}
```

And then instantiate two new `Person` objects as follows:

```
const rand = new Person("Rand McNally", 33, "M");
const ken = new Person("Ken Jones", 39, "M");
```

Then you can rewrite the definition of `Car` to include an `owner` property that takes a `Person` object, as follows:

```
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
```

```
  this.year = year;
  this.owner = owner;
}
```

To instantiate the new objects, you then use the following:

```
const car1 = new Car("Eagle", "Talon TSi", 1993, rand);
const car2 = new Car("Nissan", "300ZX", 1992, ken);
```

Instead of passing a literal string or integer value when creating the new objects, the above statements pass the objects `rand` and `ken` as the parameters for the owners. To find out the name of the owner of `car2`, you can access the following property:

```
car2.owner.name;
```

Using <code>new</code> with classes

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

const p = new Person("Caroline");
p.greet(); // Hello, my name is Caroline
```

See also

- `Function`
- `Reflect.construct()`
- `Object`

new.target

The **`new.target`** meta-property lets you detect whether a function or constructor was called using the `new` operator. In constructors and functions invoked using the `new` operator, `new.target` returns a reference to the constructor or function that `new` was called upon. In normal function calls, `new.target` is `undefined`.

Try it

```
function Foo() {
  if (!new.target) {
    throw new TypeError("calling Foo constructor without new is invalid");
  }
}

try {
  Foo();
} catch (e) {
  console.log(e);
  // Expected output: TypeError: calling Foo constructor without new is invalid
}
```

Syntax

```
new.target
```

Value

`new.target` is guaranteed to be a constructable function value or `undefined`.

- In class constructors, it refers to the class that `new` was called upon, which may be a subclass of the current constructor, because subclasses transitively call the superclass's constructor through `super()`.
- In ordinary functions, if the function is constructed directly with `new`, `new.target` refers to the function itself. If the function is called without `new`, `new.target` is `undefined`. Functions can be used as the base class for `extends`, in which case `new.target` may refer to the subclass.
- If a constructor (class or function) is called via `Reflect.construct()`, then `new.target` refers to the value passed as `newTarget` (which defaults to `target`).
- In arrow functions, `new.target` is inherited from the surrounding scope. If the arrow function is not defined within another class or function which has a `new.target` binding, then a syntax error is thrown.
- In static initialization blocks, `new.target` is `undefined`.

Description

The `new.target` syntax consists of the keyword `new`, a dot, and the identifier `target`. Because `new` is a reserved word, not an identifier, this is not a property accessor, but a special expression syntax.

The `new.target` meta-property is available in all function/class bodies; using `new.target` outside of functions or classes is a syntax error.

Examples

new.target in function calls

In normal function calls (as opposed to constructor function calls), `new.target` is `undefined`. This lets you detect whether a function was called with `new` as a constructor.

```
function Foo() {
  if (!new.target) {
    throw new Error("Foo() must be called with new");
  }
  console.log("Foo instantiated with new");
}

new Foo(); // Logs "Foo instantiated with new"
Foo(); // Throws "Foo() must be called with new"
```

new.target in constructors

In class constructors, `new.target` refers to the constructor that was directly invoked by `new`. This is also the case if the constructor is in a parent class and was delegated from a child constructor. `new.target` points to the class that `new` was called upon. For example, when `b` was initialized using `new B()`, the name of `B` was printed; and similarly, in case of `a`, the name of class `A` was printed.

```
class A {
  constructor() {
    console.log(new.target.name);
  }
}

class B extends A {
  constructor() {
    super();
  }
}

const a = new A(); // Logs "A"
const b = new B(); // Logs "B"
```

new.target using Reflect.construct()

Before `Reflect.construct()` or classes, it was common to implement inheritance by passing the value of `this`, and letting the base constructor mutate it.

```
function Base() {
  this.name = "Base";
}

function Extended() {
  // Only way to make the Base() constructor work on the existing
  // `this` value instead of a new object that `new` creates.
  Base.call(this);
  this.otherProperty = "Extended";
}

Object.setPrototypeOf(Extended.prototype, Base.prototype);
Object.setPrototypeOf(Extended, Base);

console.log(new Extended()); // Extended { name: 'Base', otherProperty: 'Extended' }
```

However, `call()` and `apply()` actually *call* the function instead of *constructing* it, so `new.target` has value `undefined`. This means that if `Base()` checks whether it's constructed with `new`, an error will be thrown, or it may behave in other unexpected ways. For example, you can't extend `Map` this way, because the `Map()` constructor cannot be called without `new`.

All built-in constructors directly construct the entire prototype chain of the new instance by reading `new.target.prototype`. So to make sure that (1) `Base` is constructed with `new`, and (2) `new.target` points to the subclass instead of `Base` itself, we need to use `Reflect.construct()`.

```
function BetterMap(entries) {
  // Call the base class constructor, but setting `new.target` to the subclass,
  // so that the instance created has the correct prototype chain.
  return Reflect.construct(Map, [entries], BetterMap);
}

BetterMap.prototype.upsert = function (key, actions) {
  if (this.has(key)) {
    this.set(key, actions.update(this.get(key)));
  } else {
    this.set(key, actions.insert());
  }
};

Object.setPrototypeOf(BetterMap.prototype, Map.prototype);
Object.setPrototypeOf(BetterMap, Map);

const map = new BetterMap([["a", 1]]);
map.upsert("a", {
  update: (value) => value + 1,
  insert: () => 1,
});
console.log(map.get("a")); // 2
```

> **Note:** In fact, due to the lack of `Reflect.construct()`, it is not possible to properly subclass built-ins (like `Error` subclassing) when transpiling to pre-ES6 code.

However, if you are writing ES6 code, prefer using classes and `extends` instead, as it's more readable and less error-prone.

```
class BetterMap extends Map {
  // The constructor is omitted because it's just the default one

  upsert(key, actions) {
    if (this.has(key)) {
      this.set(key, actions.update(this.get(key)));
    } else {
      this.set(key, actions.insert());
    }
  }
}

const map = new BetterMap([["a", 1]]);
map.upsert("a", {
  update: (value) => value + 1,
  insert: () => 1,
});
console.log(map.get("a")); // 2
```

See also

- Functions
- Classes
- `new`
- `this`

null

The **`null`** keyword refers to the `null` primitive value, which represents the intentional absence of any object value.

Try it

```
function getVowels(str) {
  const m = str.match(/[aeiou]/gi);
  if (m === null) {
    return 0;
  }
  return m.length;
}

console.log(getVowels("sky"));
// Expected output: 0
```

Syntax

```
null
```

Description

The keyword `null` is a literal for the `null` value. Unlike `undefined`, which is a global variable, `null` is not an identifier but a syntax keyword.

`null` has the following behaviors:

- Like `undefined`, accessing any property on `null` throws a `TypeError` instead of returning `undefined` or searching prototype chains.
- Like `undefined`, `null` is treated as falsy for boolean operations, and nullish for nullish coalescing and optional chaining.
- The `typeof null` result is `"object"`. This is a bug in JavaScript that cannot be fixed due to backward compatibility.
- Unlike `undefined`, `JSON.stringify()` can represent `null` faithfully.

JavaScript is unique to have two nullish values: `null` and `undefined`. Semantically, their difference is very minor: `undefined` represents the absence of a value, while `null` represents the absence of an *object*. For example, the end of the prototype chain is `null` because the prototype chain is composed of objects; `document.querySelector()` returns `null` if it doesn't match, because had it matched, the result would be an object. If you are designing an API, you should likely accept `null` and `undefined` as equivalent inputs, because many codebases have stylistic rules about when to use `null` or `undefined` by default.

Examples

Difference between <code>null</code> and <code>undefined</code>

When checking for `null` or `undefined`, beware of the differences between equality (==) and identity (===) operators, as the former performs type-conversion.

```
typeof null; // "object" (not "null" for legacy reasons)
typeof undefined; // "undefined"
null === undefined; // false
null == undefined; // true
null === null; // true
null == null; // true
!null; // true
```

```
Number.isNaN(1 + null); // false
Number.isNaN(1 + undefined); // true
```

See also

- `undefined`
- `NaN`
- `void`

Nullish coalescing operator (??)

The **nullish coalescing (??)** operator is a logical operator that returns its right-hand side operand when its left-hand side operand is `null` or `undefined`, and otherwise returns its left-hand side operand.

Try it

```
const foo = null ?? "default string";
console.log(foo);
// Expected output: "default string"

const baz = 0 ?? 42;
console.log(baz);
// Expected output: 0
```

Syntax

```
leftExpr ?? rightExpr
```

Description

The nullish coalescing operator can be seen as a special case of the logical OR (`||`) operator. The latter returns the right-hand side operand if the left operand is *any* falsy value, not only `null` or `undefined`. In other words, if you use `||` to provide some default value to another variable `foo`, you may encounter unexpected behaviors if you consider some falsy values as usable (e.g., `''` or `0`). See below for more examples.

The nullish coalescing operator has the fifth-lowest operator precedence, directly lower than `||` and directly higher than the conditional (ternary) operator.

It is not possible to combine either the AND (`&&`) or OR operators (`||`) directly with `??`. A syntax error will be thrown in such cases.

```
null || undefined ?? "foo"; // raises a SyntaxError
true && undefined ?? "foo"; // raises a SyntaxError
```

Instead, provide parenthesis to explicitly indicate precedence:

```
(null || undefined) ?? "foo"; // returns "foo"
```

Examples

Using the nullish coalescing operator

In this example, we will provide default values but keep values other than `null` or `undefined`.

```
const nullValue = null;
const emptyText = ""; // falsy
const someNumber = 42;

const valA = nullValue ?? "default for A";
const valB = emptyText ?? "default for B";
const valC = someNumber ?? 0;

console.log(valA); // "default for A"
console.log(valB); // "" (as the empty string is not null or undefined)
console.log(valC); // 42
```

Assigning a default value to a variable

Earlier, when one wanted to assign a default value to a variable, a common pattern was to use the logical OR operator (`||`):

```
let foo;

// foo is never assigned any value so it is still undefined
const someDummyText = foo || "Hello!";
```

However, due to `||` being a boolean logical operator, the left-hand-side operand was coerced to a boolean for the evaluation and any *falsy* value (including `0`, `''`, `NaN`, `false`, etc.) was not returned. This behavior may cause unexpected consequences if you consider `0`, `''`, or `NaN` as valid values.

```
const count = 0;
const text = "";

const qty = count || 42;
const message = text || "hi!";
```

```
console.log(qty); // 42 and not 0
console.log(message); // "hi!" and not ""
```

The nullish coalescing operator avoids this pitfall by only returning the second operand when the first one evaluates to either `null` or `undefined` (but no other falsy values):

```
const myText = ""; // An empty string (which is also a falsy value)

const notFalsyText = myText || "Hello world";
console.log(notFalsyText); // Hello world

const preservingFalsy = myText ?? "Hi neighborhood";
console.log(preservingFalsy); // '' (as myText is neither undefined nor null)
```

Short-circuiting

Like the 'OR' and 'AND' logical operators, the right-hand side expression is not evaluated if the left-hand side proves to be neither `null` nor `undefined`.

```
function a() {
  console.log("a was called");
  return undefined;
}
function b() {
  console.log("b was called");
  return false;
}
function c() {
  console.log("c was called");
  return "foo";
}

console.log(a() ?? c());
// Logs "a was called" then "c was called" and then "foo"
// as a() returned undefined so both expressions are evaluated

console.log(b() ?? c());
// Logs "b was called" then "false"
// as b() returned false (and not null or undefined), the right
// hand side expression was not evaluated
```

Relationship with the optional chaining operator (?.)

The nullish coalescing operator treats `undefined` and `null` as specific values. So does the optional chaining operator (`?.`), which is useful to access a property of an object which may be `null` or `undefined`. Combining them, you can safely access a property of an object which may be nullish and provide a default value if it is.

```
const foo = { someFooProp: "hi" };

console.log(foo.someFooProp?.toUpperCase() ?? "not available"); // "HI"
console.log(foo.someBarProp?.toUpperCase() ?? "not available"); // "not available"
```

See also

- Nullish coalescing assignment (`??=`)
- Optional chaining (`?.`)
- Logical OR (`||`)
- Default parameters

Nullish coalescing assignment (??=)

The **nullish coalescing assignment (`??=`)** operator, also known as the **logical nullish assignment** operator, only evaluates the right operand and assigns to the left if the left operand is nullish (`null` or `undefined`).

Try it

```
const a = { duration: 50 };

a.speed ??= 25;
console.log(a.speed);
// Expected output: 25

a.duration ??= 10;
console.log(a.duration);
// Expected output: 50
```

Syntax

```
x ??= y
```

Description

Nullish coalescing assignment *short-circuits*, meaning that `x ??= y` is equivalent to `x ?? (x = y)`, except that the expression `x` is only evaluated once.

No assignment is performed if the left-hand side is not nullish, due to short-circuiting of the nullish coalescing operator. For example, the following does not throw an error, despite `x` being `const`:

```
const x = 1;
x ??= 2;
```

Neither would the following trigger the setter:

```
const x = {
  get value() {
    return 1;
  },
  set value(v) {
    console.log("Setter called");
  },
};

x.value ??= 2;
```

In fact, if `x` is not nullish, `y` is not evaluated at all.

```
const x = 1;
x ??= console.log("y evaluated");
// Logs nothing
```

Examples

Using nullish coalescing assignment

You can use the nullish coalescing assignment operator to apply default values to object properties. Compared to using destructuring and default values, `??=` also applies the default value if the property has value `null`.

```
function config(options) {
  options.duration ??= 100;
  options.speed ??= 25;
  return options;
}

config({ duration: 125 }); // { duration: 125, speed: 25 }
config({}); // { duration: 100, speed: 25 }
```

See also

- Nullish coalescing operator (`??`)
- Nullish
- Truthy
- Falsy

Object initializer

An **object initializer** is a comma-delimited list of zero or more pairs of property names and associated values of an object, enclosed in curly braces (`{}`). Objects can also be initialized using `Object.create()` or by invoking a constructor function with the `new` operator.

Try it

```
const object1 = { a: "foo", b: 42, c: {} };

console.log(object1.a);
// Expected output: "foo"

const a = "foo";
const b = 42;
const c = {};
const object2 = { a: a, b: b, c: c };

console.log(object2.b);
// Expected output: 42

const object3 = { a, b, c };

console.log(object3.a);
// Expected output: "foo"
```

Syntax

```
o = {
  a: "foo",
  b: 42,
  c: {},
  1: "number literal property",
  "foo:bar": "string literal property",
```

```
  shorthandProperty,

  method(parameters) {
    // …
  },

  get property() {},
  set property(value) {},

  [expression]: "computed property",

  __proto__: prototype,

  ...spreadProperty,
};
```

Description

An object initializer is an expression that describes the initialization of an `Object`. Objects consist of *properties*, which are used to describe an object. The values of object properties can either contain primitive data types or other objects.

Object literal syntax vs. JSON

The object literal syntax is not the same as the **J**ava**S**cript **O**bject **N**otation (JSON). Although they look similar, there are differences between them:

- JSON *only* permits property definition using the `"property": value` syntax. The property name must be double-quoted, and the definition cannot be a shorthand. Computed property names are not allowed either.
- JSON object property values can only be strings, numbers, `true`, `false`, `null`, arrays, or another JSON object. This means JSON cannot express methods or non-plain objects like `Map` or `RegExp`.
- In JSON, `"__proto__"` is a normal property key. In an object literal, it sets the object's prototype.

JSON is a *strict subset* of the object literal syntax, meaning that every valid JSON text can be parsed as an object literal, and would likely not cause syntax errors. The only exception is that the object literal syntax prohibits duplicate `__proto__` keys, which does not apply to `JSON.parse()`. The latter treats `__proto__` like a normal property and takes the last occurrence as the property's value. The only time when the object value they represent (a.k.a. their semantic) differ is also when the source contains the `__proto__` key — for object literals, it sets the object's prototype; for JSON, it's a normal property.

```
console.log(JSON.parse('{ "__proto__": 0, "__proto__": 1 }')); // {__proto__: 1}
console.log({ "__proto__": 0, "__proto__": 1 }); // SyntaxError: Duplicate __proto__ fields are not allowed in object literals

console.log(JSON.parse('{ "__proto__": {} }')); // { __proto__: {} }
console.log({ "__proto__": {} }); // {} (with {} as prototype)
```

Examples

Creating objects

An empty object with no properties can be created like this:

```
const object = {};
```

However, the advantage of the *literal* or *initializer* notation is, that you are able to quickly create objects with properties inside the curly braces. You notate a list of `key: value` pairs delimited by commas.

The following code creates an object with three properties and the keys are `"foo"`, `"age"` and `"baz"`. The values of these keys are a string `"bar"`, the number `42`, and another object.

```
const object = {
  foo: "bar",
  age: 42,
  baz: { myProp: 12 },
};
```

Accessing properties

Once you have created an object, you might want to read or change them. Object properties can be accessed by using the dot notation or the bracket notation. (See property accessors for detailed information.)

```
object.foo; // "bar"
object["age"]; // 42
object.baz; // {myProp: 12}
object.baz.myProp; // 12
```

Property definitions

We have already learned how to notate properties using the initializer syntax. Oftentimes, there are variables in your code that you would like to put into an object. You will see code like this:

```
const a = "foo";
const b = 42;
const c = {};
```

```
const o = {
  a: a,
  b: b,
  c: c,
};
```

There is a shorter notation available to achieve the same:

```
const a = "foo";
const b = 42;
const c = {};

// Shorthand property names
const o = { a, b, c };

// In other words,
console.log(o.a === { a }.a); // true
```

Duplicate property names

When using the same name for your properties, the second property will overwrite the first.

```
const a = { x: 1, x: 2 };
console.log(a); // {x: 2}
```

After ES2015, duplicate property names are allowed everywhere, including strict mode. You can also have duplicate property names in classes. The only exception is private elements, which must be unique in the class body.

Method definitions

A property of an object can also refer to a function or a getter or setter method.

```
const o = {
  property: function (parameters) {},
  get property() {
    return 1;
  },
  set property(value) {},
};
```

A shorthand notation is available, so that the keyword `function` is no longer necessary.

```
// Shorthand method names
const o = {
  property(parameters) {},
};
```

There is also a way to concisely define generator methods.

```
const o = {
  *generator() {
    // …
  },
};
```

Which is equivalent to this ES5-like notation (but note that ECMAScript 5 has no generators):

```
const o = {
  generator: function* () {
    // …
  },
};
```

For more information and examples about methods, see method definitions.

Computed property names

The object initializer syntax also supports computed property names. That allows you to put an expression in square brackets `[]`, that will be computed and used as the property name. This is reminiscent of the bracket notation of the property accessor syntax, which you may have used to read and set properties already.

Now you can use a similar syntax in object literals, too:

```
// Computed property names
let i = 0;
const a = {
  [`foo${++i}`]: i,
  [`foo${++i}`]: i,
  [`foo${++i}`]: i,
};

console.log(a.foo1); // 1
console.log(a.foo2); // 2
```

```
console.log(a.foo3); // 3

const items = ["A", "B", "C"];
const obj = {
  [items]: "Hello",
};
console.log(obj); // A,B,C: "Hello"
console.log(obj["A,B,C"]); // "Hello"

const param = "size";
const config = {
  [param]: 12,
  [`mobile${param.charAt(0).toUpperCase()}${param.slice(1)}`]: 4,
};

console.log(config); // {size: 12, mobileSize: 4}
```

Spread properties

Object literals support the spread syntax. It copies own enumerable properties from a provided object onto a new object.

Shallow-cloning (excluding `prototype`) or merging objects is now possible using a shorter syntax than `Object.assign()`.

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };

const clonedObj = { ...obj1 };
// { foo: "bar", x: 42 }

const mergedObj = { ...obj1, ...obj2 };
// { foo: "baz", x: 42, y: 13 }
```

> **Warning:** Note that `Object.assign()` triggers setters, whereas the spread syntax doesn't!

Prototype setter

A property definition of the form `__proto__: value` or `"__proto__": value` does not create a property with the name `__proto__`. Instead, if the provided value is an object or `null`, it points the `[[Prototype]]` of the created object to that value. (If the value is not an object or `null`, the object is not changed.)

Note that the `__proto__` key is standardized syntax, in contrast to the non-standard and non-performant `Object.prototype.__proto__` accessors. It sets the `[[Prototype]]` during object creation, similar to `Object.create` — instead of mutating the prototype chain.

```
const obj1 = {};
console.log(Object.getPrototypeOf(obj1) === Object.prototype); // true

const obj2 = { __proto__: null };
console.log(Object.getPrototypeOf(obj2)); // null

const protoObj = {};
const obj3 = { "__proto__": protoObj };
console.log(Object.getPrototypeOf(obj3) === protoObj); // true

const obj4 = { __proto__: "not an object or null" };
console.log(Object.getPrototypeOf(obj4) === Object.prototype); // true
console.log(Object.hasOwn(obj4, "__proto__")); // false
```

Only a single prototype setter is permitted in an object literal. Multiple prototype setters are a syntax error.

Property definitions that do not use "colon" notation are not prototype setters. They are property definitions that behave identically to similar definitions using any other name.

```
const __proto__ = "variable";

const obj1 = { __proto__ };
console.log(Object.getPrototypeOf(obj1) === Object.prototype); // true
console.log(Object.hasOwn(obj1, "__proto__")); // true
console.log(obj1.__proto__); // "variable"

const obj2 = { __proto__() { return "hello"; } };
console.log(obj2.__proto__()); // "hello"

const obj3 = { ["__proto__"]: 17 };
console.log(obj3.__proto__); // 17

// Mixing prototype setter with normal own properties with "__proto__" key
const obj4 = { ["__proto__"]: 17, __proto__: {} }; // {__proto__: 17} (with {} as prototype)
const obj5 = {
  ["__proto__"]: 17,
  __proto__: {},
  __proto__: null, // SyntaxError: Duplicate __proto__ fields are not allowed in object literals
};
const obj6 = {
```

```
  ["__proto__"]: 17,
  ["__proto__"]: "hello",
  __proto__: null,
}; // {__proto__: "hello"} (with null as prototype)
const obj7 =  {
  ["__proto__"]: 17,
  __proto__,
  __proto__: null,
}; // {__proto__: "variable"} (with null as prototype)
```

See also

- Property accessors
- `get`
- `set`
- Method definitions
- Lexical grammar

Operator precedence

Operator precedence determines how operators are parsed concerning each other. Operators with higher precedence become the operands of operators with lower precedence.

Try it

```
console.log(3 + 4 * 5); // 3 + 20
// Expected output: 23

console.log(4 * 3 ** 2); // 4 * 9
// Expected output: 36

let a;
let b;

console.log((a = b = 5));
// Expected output: 5
```

Precedence and associativity

Consider an expression describable by the representation below, where both `OP1` and `OP2` are fill-in-the-blanks for OPerators.

```
a OP1 b OP2 c
```

The combination above has two possible interpretations:

```
(a OP1 b) OP2 c
a OP1 (b OP2 c)
```

Which one the language decides to adopt depends on the identity of `OP1` and `OP2`.

If `OP1` and `OP2` have different precedence levels (see the table below), the operator with the higher *precedence* goes first and associativity does not matter. Observe how multiplication has higher precedence than addition and executed first, even though addition is written first in the code.

```
console.log(3 + 10 * 2); // 23
console.log(3 + (10 * 2)); // 23, because parentheses here are superfluous
console.log((3 + 10) * 2); // 26, because the parentheses change the order
```

Within operators of the same precedence, the language groups them by *associativity*. *Left-associativity* (left-to-right) means that it is interpreted as `(a OP1 b) OP2 c`, while *right-associativity* (right-to-left) means it is interpreted as `a OP1 (b OP2 c)`. Assignment operators are right-associative, so you can write:

```
a = b = 5; // same as writing a = (b = 5);
```

with the expected result that `a` and `b` get the value 5. This is because the assignment operator returns the value that is assigned. First, `b` is set to 5. Then the `a` is also set to 5 — the return value of `b = 5`, a.k.a. right operand of the assignment.

As another example, the unique exponentiation operator has right-associativity, whereas other arithmetic operators have left-associativity.

```
const a = 4 ** 3 ** 2; // Same as 4 ** (3 ** 2); evaluates to 262144
const b = 4 / 3 / 2; // Same as (4 / 3) / 2; evaluates to 0.6666...
```

Operators are first grouped by precedence, and then, for adjacent operators that have the same precedence, by associativity. So, when mixing division and exponentiation, the exponentiation always comes before the division. For example, `2 ** 3 / 3 ** 2` results in 0.8888888888888888 because it is the same as `(2 ** 3) / (3 ** 2)`.

For prefix unary operators, suppose we have the following pattern:

```
OP1 a OP2 b
```

where `OP1` is a prefix unary operator and `OP2` is a binary operator. If `OP1` has higher precedence than `OP2`, then it would be grouped as `(OP1 a) OP2 b`; otherwise, it would be `OP1 (a OP2 b)`.

```
const a = 1;
const b = 2;
typeof a + b; // Equivalent to (typeof a) + b; result is "number2"
```

If the unary operator is on the second operand:

```
a OP2 OP1 b
```

Then the binary operator `OP2` must have lower precedence than the unary operator `OP1` for it to be grouped as `a OP2 (OP1 b)`. For example, the following is invalid:

```
function* foo() {
  a + yield 1;
}
```

Because `+` has higher precedence than `yield`, this would become `(a + yield) 1` — but because `yield` is a reserved word in generator functions, this would be a syntax error. Luckily, most unary operators have higher precedence than binary operators and do not suffer from this pitfall.

If we have two prefix unary operators:

```
OP1 OP2 a
```

Then the unary operator closer to the operand, `OP2`, must have higher precedence than `OP1` for it to be grouped as `OP1 (OP2 a)`. It's possible to get it the other way and end up with `(OP1 OP2) a`:

```
async function* foo() {
  await yield 1;
}
```

Because `await` has higher precedence than `yield`, this would become `(await yield) 1`, which is awaiting an identifier called `yield`, and a syntax error. Similarly, if you have `new !A;`, because `!` has lower precedence than `new`, this would become `(new !) A`, which is obviously invalid. (This code looks nonsensical to write anyway, since `!A` always produces a boolean, not a constructor function.)

For postfix unary operators (namely, `++` and `--`), the same rules apply. Luckily, both operators have higher precedence than any binary operator, so the grouping is always what you would expect. Moreover, because `++` evaluates to a *value*, not a *reference*, you can't chain multiple increments together either.

```
let a = 1;
a++++; // SyntaxError: Invalid left-hand side in postfix operation.
```

Operator precedence will be handled *recursively*. For example, consider this expression:

```
1 + 2 ** 3 * 4 / 5 >> 6
```

First, we group operators with different precedence by decreasing levels of precedence.

1. The `**` operator has the highest precedence, so it's grouped first.
2. Looking around the `**` expression, it has `*` on the right and `+` on the left. `*` has higher precedence, so it's grouped first. `*` and `/` have the same precedence, so we group them together for now.
3. Looking around the `*//` expression grouped in 2, because `+` has higher precedence than `>>`, the former is grouped.

```
   (1 + ( (2 ** 3) * 4 / 5) ) >> 6
// |    | └ 1. ┘        | |
// |    └──────── 2. ─────────┘ |
// └──────────── 3. ────────────┘
```

Within the `*//` group, because they are both left-associative, the left operand would be grouped.

```
   (1 + ( ( (2 ** 3) * 4 ) / 5) ) >> 6
// |    | | └ 1. ┘     |    | |
// |    └─|──────── 2. ──┤────┘ |
// └──────┤──── 3. ──────┤──────┘
//        └──── 4. ──────┘
```

Note that operator precedence and associativity only affect the order of evaluation of *operators* (the implicit grouping), but not the order of evaluation of *operands*. The operands are always evaluated from left-to-right. The higher-precedence expressions are always evaluated first, and their results are then composed according to the order of operator precedence.

```
function echo(name, num) {
  console.log(`Evaluating the ${name} side`);
  return num;
}
// Exponentiation operator (**) is right-associative,
// but all call expressions (echo()), which have higher precedence,
// will be evaluated before ** does
console.log(echo("left", 4) ** echo("middle", 3) ** echo("right", 2));
// Evaluating the left side
// Evaluating the middle side
// Evaluating the right side
// 262144
```

```
// Exponentiation operator (**) has higher precedence than division (/),
// but evaluation always starts with the left operand
console.log(echo("left", 4) / echo("middle", 3) ** echo("right", 2));
// Evaluating the left side
// Evaluating the middle side
// Evaluating the right side
// 0.4444444444444444
```

If you are familiar with binary trees, think about it as a post-order traversal.

```
               /
       ┌───────┴────────┐
echo("left", 4)         **
               ┌────────┴────────┐
        echo("middle", 3)  echo("right", 2)
```

After all operators have been properly grouped, the binary operators would form a binary tree. Evaluation starts from the outermost group — which is the operator with the lowest precedence (`/` in this case). The left operand of this operator is first evaluated, which may be composed of higher-precedence operators (such as a call expression `echo("left", 4)`). After the left operand has been evaluated, the right operand is evaluated in the same fashion. Therefore, all leaf nodes — the `echo()` calls — would be visited left-to-right, regardless of the precedence of operators joining them.

Short-circuiting

In the previous section, we said "the higher-precedence expressions are always evaluated first" — this is generally true, but it has to be amended with the acknowledgement of *short-circuiting*, in which case an operand may not be evaluated at all.

Short-circuiting is jargon for conditional evaluation. For example, in the expression `a && (b + c)`, if `a` is falsy, then the sub-expression `(b + c)` will not even get evaluated, even if it is grouped and therefore has higher precedence than `&&`. We could say that the logical AND operator (`&&`) is "short-circuited". Along with logical AND, other short-circuited operators include logical OR (`||`), nullish coalescing (`??`), and optional chaining (`?.`).

```
a || (b * c); // evaluate `a` first, then produce `a` if `a` is "truthy"
a && (b < c); // evaluate `a` first, then produce `a` if `a` is "falsy"
a ?? (b || c); // evaluate `a` first, then produce `a` if `a` is not `null` and not `undefined`
a?.b.c; // evaluate `a` first, then produce `undefined` if `a` is `null` or `undefined`
```

When evaluating a short-circuited operator, the left operand is always evaluated. The right operand will only be evaluated if the left operand cannot determine the result of the operation.

> **Note:** The behavior of short-circuiting is baked in these operators. Other operators would *always* evaluate both operands, regardless if that's actually useful — for example, `NaN * foo()` will always call `foo`, even when the result would never be something other than `NaN`.

The previous model of a post-order traversal still stands. However, after the left subtree of a short-circuiting operator has been visited, the language will decide if the right operand needs to be evaluated. If not (for example, because the left operand of `||` is already truthy), the result is directly returned without visiting the right subtree.

Consider this case:

```
function A() { console.log('called A'); return false; }
function B() { console.log('called B'); return false; }
function C() { console.log('called C'); return true; }

console.log(C() || B() && A());

// Logs:
// called C
// true
```

Only `C()` is evaluated, despite `&&` having higher precedence. This does not mean that `||` has higher precedence in this case — it's exactly *because* `(B() && A())` has higher precedence that causes it to be neglected as a whole. If it's re-arranged as:

```
console.log(A() && B() || C());
// Logs:
// called A
// called C
// true
```

Then the short-circuiting effect of `&&` would only prevent `B()` from being evaluated, but because `A() && B()` as a whole is `false`, `C()` would still be evaluated.

However, note that short-circuiting does not change the final evaluation outcome. It only affects the evaluation of *operands*, not how *operators* are grouped — if evaluation of operands doesn't have side effects (for example, logging to the console, assigning to variables, throwing an error), short-circuiting would not be observable at all.

The assignment counterparts of these operators (`&&=`, `||=`, `??=`) are short-circuited as well. They are short-circuited in a way that the assignment does not happen at all.

Table

The following table lists operators in order from highest precedence (18) to lowest precedence (1).

Several general notes about the table:

1. Not all syntax included here are "operators" in the strict sense. For example, spread `...` and arrow `=>` are typically not regarded as operators. However, we still included them to show how tightly they bind compared to other operators/expressions.
2. Some operators have certain operands that require expressions narrower than those produced by higher-precedence operators. For example, the right-hand side of member access `.` (precedence 17) must be an identifier instead of a grouped expression. The left-hand side of arrow `=>` (precedence 2) must be an arguments list or a single identifier instead of some random expression.
3. Some operators have certain operands that accept expressions wider than those produced by higher-precedence operators. For example, the bracket-enclosed expression of bracket notation `[ … ]` (precedence 17) can be any expression, even comma (precedence 1) joined ones. These operators act as if that operand is "automatically grouped". In this case we will omit the associativity.

Precedence	Associativity	Individual operators	Notes
18: grouping	n/a	Grouping `(x)`	[1]
17: access and call	left-to-right	Member access `x.y`	[2]
		Optional chaining `x?.y`	
	n/a	Computed member access `x[y]`	[3]
		`new` with argument list `new x(y)`	[4]
		Function call `x(y)`	
		`import(x)`	
16: new	n/a	`new` without argument list `new x`	
15: postfix operators	n/a	Postfix increment `x++`	[5]
		Postfix decrement `x--`	
14: prefix operators	n/a	Prefix increment `++x`	[6]
		Prefix decrement `--x`	
		Logical NOT `!x`	
		Bitwise NOT `~x`	
		Unary plus `+x`	
		Unary negation `-x`	
		`typeof x`	
		`void x`	
		`delete x`	[7]
		`await x`	
13: exponentiation	right-to-left	Exponentiation `x ** y`	[8]
12: multiplicative operators	left-to-right	Multiplication `x * y`	
		Division `x / y`	
		Remainder `x % y`	
11: additive operators	left-to-right	Addition `x + y`	
		Subtraction `x - y`	
10: bitwise shift	left-to-right	Left shift `x << y`	
		Right shift `x >> y`	
		Unsigned right shift `x >>> y`	

9: relational operators	left-to-right	Less than `x < y`	
		Less than or equal `x <= y`	
		Greater than `x > y`	
		Greater than or equal `x >= y`	
		`x in y`	
		`x instanceof y`	
8: equality operators	left-to-right	Equality `x == y`	
		Inequality `x != y`	
		Strict equality `x === y`	
		Strict inequality `x !== y`	
7: bitwise AND	left-to-right	Bitwise AND `x & y`	
6: bitwise XOR	left-to-right	Bitwise XOR `x ^ y`	
5: bitwise OR	left-to-right	Bitwise OR `x | y`	
4: logical AND	left-to-right	Logical AND `x && y`	
3: logical OR, nullish coalescing	left-to-right	Logical OR `x || y`	
		Nullish coalescing operator `x ?? y`	[9]
2: assignment and miscellaneous	right-to-left	Assignment `x = y`	[10]
		Addition assignment `x += y`	
		Subtraction assignment `x -= y`	
		Exponentiation assignment `x **= y`	
		Multiplication assignment `x *= y`	
		Division assignment `x /= y`	
		Remainder assignment `x %= y`	
		Left shift assignment `x <<= y`	
		Right shift assignment `x >>= y`	
		Unsigned right shift assignment `x >>>= y`	
		Bitwise AND assignment `x &= y`	
		Bitwise XOR assignment `x ^= y`	
		Bitwise OR assignment `x |= y`	
		Logical AND assignment `x &&= y`	
		Logical OR assignment `x ||= y`	
		Nullish coalescing assignment `x ??= y`	

	right-to-left	Conditional (ternary) operator `x ? y : z`	[11]
	right-to-left	Arrow `x => y`	[12]
	n/a	`yield x`	
		`yield* x`	
		Spread `...x`	[13]
1: comma	left-to-right	Comma operator `x, y`	

Notes:

1. The operand can be any expression.
2. The "right-hand side" must be an identifier.
3. The "right-hand side" can be any expression.
4. The "right-hand side" is a comma-separated list of any expression with precedence > 1 (i.e., not comma expressions). The constructor of a `new` expression cannot be an optional chain.
5. The operand must be a valid assignment target (identifier or property access). Its precedence means `new Foo++` is `(new Foo)++` (a syntax error) and not `new (Foo++)` (a TypeError: (Foo++) is not a constructor).
6. The operand must be a valid assignment target (identifier or property access).
7. The operand cannot be an identifier or a private element access.
8. The left-hand side cannot have precedence 14.
9. The operands cannot be a logical OR `||` or logical AND `&&` operator without grouping.
10. The "left-hand side" must be a valid assignment target (identifier or property access).
11. The associativity means the two expressions after `?` are implicitly grouped.
12. The "left-hand side" is a single identifier or a parenthesized parameter list.
13. Only valid inside object literals, array literals, or argument lists.

The precedence of groups 17 and 16 may be a bit ambiguous. Here are a few examples to clarify.

- Optional chaining is always substitutable for its respective syntax without optionality (barring a few special cases where optional chaining is forbidden). For example, any place that accepts `a?.b` also accepts `a.b` and vice versa, and similarly for `a?.()`, `a()`, etc.
- Member expressions and computed member expressions are always substitutable for each other.
- Call expressions and `import()` expressions are always substitutable for each other.
- This leaves four classes of expressions: member access, `new` with arguments, function call, and `new` without arguments.
  - The "left-hand side" of a member access can be: a member access (`a.b.c`), `new` with arguments (`new a().b`), and function call (`a().b`).
  - The "left-hand side" of `new` with arguments can be: a member access (`new a.b()`) and `new` with arguments (`new new a()()`).
  - The "left-hand side" of a function call can be: a member access (`a.b()`), `new` with arguments (`new a()()`), and function call (`a()()`).
  - The operand of `new` without arguments can be: a member access (`new a.b`), `new` with arguments (`new new a()`), and `new` without arguments (`new new a`).

Optional chaining (?.)

The **optional chaining (`?.`)** operator accesses an object's property or calls a function. If the object accessed or function called using this operator is `undefined` or `null`, the expression short circuits and evaluates to `undefined` instead of throwing an error.

Try it

```
const adventurer = {
  name: "Alice",
  cat: {
    name: "Dinah",
  },
};

const dogName = adventurer.dog?.name;
console.log(dogName);
// Expected output: undefined

console.log(adventurer.someNonExistentMethod?.());
// Expected output: undefined
```

Syntax

```
obj?.prop
obj?.[expr]
func?.(args)
```

Description

The `?.` operator is like the `.` chaining operator, except that instead of causing an error if a reference is nullish (`null` or `undefined`), the expression short-circuits with a return value of `undefined`. When used with function calls, it returns `undefined` if the given function does not exist.

This results in shorter and simpler expressions when accessing chained properties when the possibility exists that a reference may be missing. It can also be helpful while exploring the content of an object when there's no known guarantee as to which properties are required.

For example, consider an object `obj` which has a nested structure. Without optional chaining, looking up a deeply-nested subproperty requires validating the references in between, such as:

```
const nestedProp = obj.first && obj.first.second;
```

The value of `obj.first` is confirmed to be non-`null` (and non-`undefined`) before accessing the value of `obj.first.second`. This prevents the error that would occur if you accessed `obj.first.second` directly without testing `obj.first`.

This is an idiomatic pattern in JavaScript, but it gets verbose when the chain is long, and it's not safe. For example, if `obj.first` is a Falsy value that's not `null` or `undefined`, such as `0`, it would still short-circuit and make `nestedProp` become `0`, which may not be desirable.

With the optional chaining operator (`?.`), however, you don't have to explicitly test and short-circuit based on the state of `obj.first` before trying to access `obj.first.second`:

```
const nestedProp = obj.first?.second;
```

By using the `?.` operator instead of just `.`, JavaScript knows to implicitly check to be sure `obj.first` is not `null` or `undefined` before attempting to access `obj.first.second`. If `obj.first` is `null` or `undefined`, the expression automatically short-circuits, returning `undefined`.

This is equivalent to the following, except that the temporary variable is in fact not created:

```
const temp = obj.first;
const nestedProp =
  temp === null || temp === undefined ? undefined : temp.second;
```

Optional chaining cannot be used on a non-declared root object, but can be used with a root object with value `undefined`.

```
undeclaredVar?.prop; // ReferenceError: undeclaredVar is not defined
```

Optional chaining with function calls

You can use optional chaining when attempting to call a method which may not exist. This can be helpful, for example, when using an API in which a method might be unavailable, either due to the age of the implementation or because of a feature which isn't available on the user's device.

Using optional chaining with function calls causes the expression to automatically return `undefined` instead of throwing an exception if the method isn't found:

```
const result = someInterface.customMethod?.();
```

However, if there is a property with such a name which is not a function, using `?.` will still raise a `TypeError` exception "someInterface.customMethod is not a function".

> **Note:** If `someInterface` itself is `null` or `undefined`, a `TypeError` exception will still be raised ("someInterface is null"). If you expect that `someInterface` itself may be `null` or `undefined`, you have to use `?.` at this position as well: `someInterface?.customMethod?.()`.

`eval?.()` is the shortest way to enter *indirect eval* mode.

Optional chaining with expressions

You can also use the optional chaining operator with bracket notation, which allows passing an expression as the property name:

```
const propName = "x";
const nestedProp = obj?.[propName];
```

This is particularly useful for arrays, since array indices must be accessed with square brackets.

```
function printMagicIndex(arr) {
  console.log(arr?.[42]);
}

printMagicIndex([0, 1, 2, 3, 4, 5]); // undefined
printMagicIndex(); // undefined; if not using ?., this would throw an error: "Cannot read properties of undefined (reading '42')"
```

Invalid optional chaining

It is invalid to try to assign to the result of an optional chaining expression:

```
const object = {};
object?.property = 1; // SyntaxError: Invalid left-hand side in assignment
```

Template literal tags cannot be an optional chain (see SyntaxError: tagged template cannot be used with optional chain):

```
String?.raw`Hello, world!`;
String.raw?.`Hello, world!`; // SyntaxError: Invalid tagged template on optional chain
```

The constructor of `new` expressions cannot be an optional chain (see SyntaxError: new keyword cannot be used with an optional chain):

```
new Intl?.DateTimeFormat(); // SyntaxError: Invalid optional chain from new expression
new Map?.();
```

Short-circuiting

When using optional chaining with expressions, if the left operand is `null` or `undefined`, the expression will not be evaluated. For instance:

```
const potentiallyNullObj = null;
let x = 0;
const prop = potentiallyNullObj?.[x++];

console.log(x); // 0 as x was not incremented
```

Subsequent property accesses will not be evaluated either.

```
const potentiallyNullObj = null;
const prop = potentiallyNullObj?.a.b;
// This does not throw, because evaluation has already stopped at
// the first optional chain
```

This is equivalent to:

```
const potentiallyNullObj = null;
const prop =
  potentiallyNullObj === null || potentiallyNullObj === undefined
    ? undefined
    : potentiallyNullObj.a.b;
```

However, this short-circuiting behavior only happens along one continuous "chain" of property accesses. If you group one part of the chain, then subsequent property accesses will still be evaluated.

```
const potentiallyNullObj = null;
const prop = (potentiallyNullObj?.a).b;
// TypeError: Cannot read properties of undefined (reading 'b')
```

This is equivalent to:

```
const potentiallyNullObj = null;
const temp = potentiallyNullObj?.a;
const prop = temp.b;
```

Except the `temp` variable isn't created.

Examples

Basic example

This example looks for the value of the `name` property for the member `CSS` in a map when there is no such member. The result is therefore `undefined`.

```
const myMap = new Map();
myMap.set("JS", { name: "Josh", desc: "I maintain things" });

const nameBar = myMap.get("CSS")?.name;
```

Dealing with optional callbacks or event handlers

If you use callbacks or fetch methods from an object with a destructuring pattern, you may have non-existent values that you cannot call as functions unless you have tested their existence. Using `?.`, you can avoid this extra test:

```
// Code written without optional chaining
function doSomething(onContent, onError) {
  try {
    // Do something with the data
  } catch (err) {
    // Testing if onError really exists
    if (onError) {
      onError(err.message);
    }
  }
}
```

```
// Using optional chaining with function calls
function doSomething(onContent, onError) {
  try {
    // Do something with the data
  } catch (err) {
    onError?.(err.message); // No exception if onError is undefined
  }
}
```

Stacking the optional chaining operator

With nested structures, it is possible to use optional chaining multiple times:

```
const customer = {
  name: "Carl",
  details: {
    age: 82,
    location: "Paradise Falls", // Detailed address is unknown
  },
};
const customerCity = customer.details?.address?.city;

// This also works with optional chaining function call
const customerName = customer.name?.getName?.(); // Method does not exist, customerName is undefined
```

Combining with the nullish coalescing operator

The nullish coalescing operator may be used after optional chaining in order to build a default value when none was found:

```
function printCustomerCity(customer) {
  const customerCity = customer?.city ?? "Unknown city";
  console.log(customerCity);
}

printCustomerCity({
  name: "Nathan",
  city: "Paris",
}); // "Paris"
printCustomerCity({
  name: "Carl",
  details: { age: 82 },
}); // "Unknown city"
```

See also

- Nullish coalescing operator (`??`)

Property accessors

Property accessors provide access to an object's properties by using the dot notation or the bracket notation.

Try it

```
const person1 = {};
person1["firstName"] = "Mario";
person1["lastName"] = "Rossi";

console.log(person1.firstName);
// Expected output: "Mario"

const person2 = {
  firstName: "John",
  lastName: "Doe",
};

console.log(person2["lastName"]);
// Expected output: "Doe"
```

Syntax

```
object.propertyName
object[expression]
object.#privateProperty
```

Description

One can think of an object as an *associative array* (a.k.a. *map*, *dictionary*, *hash*, *lookup table*). The *keys* in this array are the names of the object's properties.

There are two ways to access properties: *dot notation* and *bracket notation*.

Dot notation

In the `object.propertyName` syntax, the `propertyName` must be a valid JavaScript identifier which can also be a reserved word. For example, `object.$1` is valid, while `object.1` is not.

```
const variable = object.propertyName;
object.propertyName = value;
```

```
const object = {};
object.$1 = "foo";
console.log(object.$1); // 'foo'
```

```
const object = {};
object.1 = "bar"; // SyntaxError
console.log(object.1); // SyntaxError
```

Here, the method named `createElement` is retrieved from `document` and is called.

```
document.createElement("pre");
```

If you use a method for a numeric literal, and the numeric literal has no exponent and no decimal point, you should leave white-space(s) before the dot preceding the method call, so that the dot is not interpreted as a decimal point.

```
77 .toExponential();
// or
77
.toExponential();
// or
(77).toExponential();
// or
77..toExponential();
// or
77.0.toExponential();
// because 77. === 77.0, no ambiguity
```

In addition, private elements can only be accessed using dot notation within the class that defines them.

Bracket notation

In the `object[expression]` syntax, the `expression` should evaluate to a string or Symbol that represents the property's name. So, it can be any string literal, for example, including `'1foo'`, `'!bar!'`, or even `' '` (a space).

```
const variable = object[propertyName];
object[propertyName] = value;
```

This does the exact same thing as the previous example.

```
document["createElement"]("pre");
```

A space before bracket notation is allowed.

```
document ["createElement"]("pre");
```

Passing expressions that evaluate to property name will do the same thing as directly passing the property name.

```
const key = "name";
const getKey = () => "name";
const Obj = { name: "Michel" };

Obj["name"]; // returns "Michel"
Obj[key]; // evaluates to Obj["name"], and returns "Michel"
Obj[getKey()]; // evaluates to Obj["name"], and returns "Michel"
```

However, beware of using square brackets to access properties whose names are given by external input. This may make your code susceptible to object injection attacks.

Property names

Each property name is a string or a Symbol. Any other value, including a number, is coerced to a string. This outputs `'value'`, since `1` is coerced into `'1'`.

```
const object = {};
object["1"] = "value";
console.log(object[1]);
```

This also outputs `'value'`, since both `foo` and `bar` are converted to the same string (`"[object Object]"`).

```
const foo = { uniqueProp: 1 };
const bar = { uniqueProp: 2 };
const object = {};
object[foo] = "value";
console.log(object[bar]);
```

Method binding

It's typical when speaking of an object's properties to make a distinction between properties and methods. However, the property/method distinction is little more than a convention. A method is a property that can be called (for example, if it has a reference to a `Function` instance as its value).

A method is not bound to the object that it is a property of. Specifically, `this` is not fixed in a method and does not necessarily refer to the object containing the method. Instead, `this` is "passed" by the function call. See the reference for `this`.

Examples

Bracket notation vs. eval()

JavaScript novices often make the mistake of using `eval()` where the bracket notation can be used instead.

For example, the following syntax is often seen in many scripts.

```
const x = eval(`document.forms.form_name.elements.${strFormControl}.value`);
```

`eval()` is slow and should be avoided whenever possible. Also, `strFormControl` would have to hold an identifier, which is not required for names and `id`s of form controls. It is better to use bracket notation instead:

```
const x = document.forms.form_name.elements[strFormControl].value;
```

See also

- `Object`
- `Object.defineProperty()`
- Optional chaining (`?.`)

Remainder (%)

The **remainder (%)** operator returns the remainder left over when one operand is divided by a second operand. It always takes the sign of the dividend.

Try it

```
console.log(13 % 5);
// Expected output: 3

console.log(-13 % 5);
// Expected output: -3

console.log(4 % 2);
// Expected output: 0

console.log(-4 % 2);
// Expected output: -0
```

Syntax

```
x % y
```

Description

The `%` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt remainder if both operands become BigInts; otherwise, it performs number remainder. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

For the operation `n % d`, `n` is called the dividend and `d` is called the divisor. The operation returns `NaN` if one of the operands is `NaN`, `n` is ±Infinity, or if `d` is ±0. Otherwise, if `d` is ±Infinity or if `n` is ±0, the dividend `n` is returned.

When both operands are non-zero and finite, the remainder `r` is calculated as `r := n - d * q` where `q` is the integer such that `r` has the same sign as the dividend `n` while being as close to 0 as possible.

Note that while in most languages, '%' is a remainder operator, in some (e.g., Python, Perl) it is a modulo operator. Modulo is defined as `k := n - d * q` where `q` is the integer such that `k` has the same sign as the divisor `d` while being as close to 0 as possible. For two values of the same sign, the two are equivalent, but when the operands are of different signs, the modulo result always has the same sign as the *divisor*, while the remainder has the same sign as the *dividend*, which can make them differ by one unit of `d`. To obtain a modulo in JavaScript, in place of `n % d`, use `((n % d) + d) % d`. In JavaScript, the modulo operation (which doesn't have a dedicated operator) is used to normalize the second operand of bitwise shift operators (`<<`, `>>`, etc.), making the offset always a positive value.

For BigInt division, a `RangeError` is thrown if the divisor `y` is `0n`. This is because number remainder by zero returns `NaN`, but BigInt has no concept of `NaN`.

Examples

Remainder with positive dividend

```
13 % 5; // 3
1 % -2; // 1
1 % 2; // 1
2 % 3; // 2
5.5 % 2; // 1.5

3n % 2n; // 1n
```

Remainder with negative dividend

```
-13 % 5; // -3
-1 % 2; // -1
-4 % 2; // -0

-3n % 2n; // -1n
```

Remainder with NaN

```
NaN % 2; // NaN
```

Remainder with Infinity

```
Infinity % 2; // NaN
Infinity % 0; // NaN
Infinity % Infinity; // NaN
2 % Infinity; // 2
0 % Infinity; // 0
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)
- Remainder operator vs. modulo operator by Dr. Axel Rauschmayer (2019)

Remainder assignment (%=)

The **remainder assignment (%=)** operator performs remainder on the two operands and assigns the result to the left operand.

Try it

```
let a = 3;

console.log((a %= 2));
// Expected output: 1

console.log((a %= 0));
// Expected output: NaN

console.log((a %= "hello"));
// Expected output: NaN
```

Syntax

```
x %= y
```

Description

`x %= y` is equivalent to `x = x % y`, except that the expression `x` is only evaluated once.

Examples

Using remainder assignment

```
let bar = 5;

bar %= 2; // 1
bar %= "foo"; // NaN
bar %= 0; // NaN

let foo = 3n;
foo %= 2n; // 1n
```

See also

- Assignment operators in the JS guide
- Remainder (%)

Right shift (>>)

The **right shift (>>)** operator returns a number or BigInt whose binary representation is the first operand shifted by the specified number of bits to the right. Excess bits shifted off to the right are discarded, and copies of the leftmost bit are shifted in from the left. This operation is also called "sign-propagating right shift" or "arithmetic right shift", because the sign of the resulting number is the same as the sign of the first operand.

Try it

```
const a = 5; //  00000000000000000000000000000101
const b = 2; //  00000000000000000000000000000010
const c = -5; //  11111111111111111111111111111011

console.log(a >> b); //  00000000000000000000000000000001
// Expected output: 1

console.log(c >> b); //  11111111111111111111111111111110
// Expected output: -2
```

Syntax

```
x >> y
```

Description

The `>>` operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt right shift if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number right shift. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

Since the new leftmost bit has the same value as the previous leftmost bit, the sign bit (the leftmost bit) does not change. Hence the name "sign-propagating".

The operator operates on the left operand's bit representation in two's complement. Consider the 32-bit binary representations of the decimal (base 10) numbers `9` and `-9`:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
    -9 (base 10): 11111111111111111111111111110111 (base 2)
```

The binary representation under two's complement of the negative decimal (base 10) number `-9` is formed by inverting all the bits of its opposite number, which is `9` and `00000000000000000000000000001001` in binary, and adding `1`.

In both cases, the sign of the binary number is given by its leftmost bit: for the positive decimal number `9`, the leftmost bit of the binary representation is `0`, and for the negative decimal number `-9`, the leftmost bit of the binary representation is `1`.

Given those binary representations of the decimal (base 10) numbers `9`, and `-9`:

`9 >> 2` yields 2:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
                  --------------------------------
9 >> 2 (base 10): 00000000000000000000000000000010 (base 2) = 2 (base 10)
```

Notice how two rightmost bits, `01`, have been shifted off, and two copies of the leftmost bit, `0` have been shifted in from the left.

`-9 >> 2` yields `-3`:

```
     -9 (base 10): 11111111111111111111111111110111 (base 2)
                   --------------------------------
-9 >> 2 (base 10): 11111111111111111111111111111101 (base 2) = -3 (base 10)
```

Notice how two rightmost bits, `11`, have been shifted off. But as far as the leftmost bits: in this case, the leftmost bit is `1`. So two copies of that leftmost `1` bit have been shifted in from the left — which preserves the negative sign.

The binary representation `11111111111111111111111111111101` is equal to the negative decimal (base 10) number `-3`, because all negative integers are stored as two's complements, and this one can be calculated by inverting all the bits of the binary representation of the positive decimal (base 10) number `3`, which is `00000000000000000000000000000011`, and then adding one.

If the left operand is a number with more than 32 bits, it will get the most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

The right operand will be converted to an unsigned 32-bit integer and then taken modulo 32, so the actual shift offset will always be a positive integer between 0 and 31, inclusive. For example, `100 >> 32` is the same as `100 >> 0` (and produces `100`) because 32 modulo 32 is 0.

> **Warning:** You may see people using `>> 0` to truncate numbers to integers. Right shifting any number `x` by `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using right shift

```
9 >> 2; // 2
-9 >> 2; // -3

9n >> 2n; // 2n
```

See also

- Bitwise operators in the JS guide
- Right shift assignment (>>=)
- Unsigned right shift (>>>)

Right shift assignment (>>=)

The **right shift assignment (>>=)** operator performs right shift on the two operands and assigns the result to the left operand.

Try it

```
let a = 5; //  00000000000000000000000000000101

a >>= 2; //  00000000000000000000000000000001
console.log(a);
// Expected output: 1

let b = -5; //  11111111111111111111111111111011

b >>= 2; //  11111111111111111111111111111110
console.log(b);
// Expected output: -2
```

Syntax

```
x >>= y
```

Description

`x >>= y` is equivalent to `x = x >> y`, except that the expression `x` is only evaluated once.

Examples

Using right shift assignment

```
let a = 5; //   (00000000000000000000000000000101)
a >>= 2; //   1 (00000000000000000000000000000001)

let b = -5; //  (-00000000000000000000000000000101)
b >>= 2; //  -2 (-00000000000000000000000000000010)

let c = 5n;
c >>= 2n; // 1n
```

See also

- Assignment operators in the JS guide
- Right shift (>>)

Spread syntax (...)

The **spread (`...`)** syntax allows an iterable, such as an array or string, to be expanded in places where zero or more arguments (for function calls) or elements (for array literals) are expected. In an object literal, the spread syntax enumerates the properties of an object and adds the key-value pairs to the object being created.

Spread syntax looks exactly like rest syntax. In a way, spread syntax is the opposite of rest syntax. Spread syntax "expands" an array into its elements, while rest syntax collects multiple elements and "condenses" them into a single element. See rest parameters and rest property.

Try it

```
function sum(x, y, z) {
  return x + y + z;
}

const numbers = [1, 2, 3];

console.log(sum(...numbers));
// Expected output: 6

console.log(sum.apply(null, numbers));
// Expected output: 6
```

Syntax

```
myFunction(a, ...iterableObj, b)
[1, ...iterableObj, '4', 'five', 6]
{ ...obj, key: 'value' }
```

Description

Spread syntax can be used when all elements from an object or array need to be included in a new array or object, or should be applied one-by-one in a function call's arguments list. There are three distinct places that accept the spread syntax:

- Function arguments list (`myFunction(a, ...iterableObj, b)`)
- Array literals (`[1, ...iterableObj, '4', 'five', 6]`)
- Object literals (`{ ...obj, key: 'value' }`)

Although the syntax looks the same, they come with slightly different semantics.

Only iterable values, like `Array` and `String`, can be spread in array literals and argument lists. Many objects are not iterable, including all plain objects that lack a `Symbol.iterator` method:

```
const obj = { key1: "value1" };
const array = [...obj]; // TypeError: obj is not iterable
```

On the other hand, spreading in object literals enumerates the own properties of the value. For typical arrays, all indices are enumerable own properties, so arrays can be spread into objects.

```
const array = [1, 2, 3];
const obj = { ...array }; // { 0: 1, 1: 2, 2: 3 }
```

All primitives can be spread in objects. Only strings have enumerable own properties, and spreading anything else doesn't create properties on the new object.

```
const obj = { ...true, ..."test", ...10 };
// { '0': 't', '1': 'e', '2': 's', '3': 't' }
```

When using spread syntax for function calls, be aware of the possibility of exceeding the JavaScript engine's argument length limit. See `Function.prototype.apply()` for more details.

Examples

Spread in function calls

Replace apply()

It is common to use `Function.prototype.apply()` in cases where you want to use the elements of an array as arguments to a function.

```
function myFunction(x, y, z) {}
const args = [0, 1, 2];
myFunction.apply(null, args);
```

With spread syntax the above can be written as:

```
function myFunction(x, y, z) {}
const args = [0, 1, 2];
myFunction(...args);
```

Any argument in the argument list can use spread syntax, and the spread syntax can be used multiple times.

```
function myFunction(v, w, x, y, z) {}
const args = [0, 1];
myFunction(-1, ...args, 2, ...[3]);
```

Apply for new operator

When calling a constructor with `new`, it's not possible to **directly** use an array and `apply()`, because `apply()` *calls* the target function instead of *constructing* it, which means, among other things, that `new.target` will be `undefined`. However, an array can be easily used with `new` thanks to spread syntax:

```
const dateFields = [1970, 0, 1]; // 1 Jan 1970
const d = new Date(...dateFields);
```

Spread in array literals

A more powerful array literal

Without spread syntax, the array literal syntax is no longer sufficient to create a new array using an existing array as one part of it. Instead, imperative code must be used using a combination of methods, including `push()`, `splice()`, `concat()`, etc. With spread syntax, this becomes much more succinct:

```
const parts = ["shoulders", "knees"];
const lyrics = ["head", ...parts, "and", "toes"];
//  ["head", "shoulders", "knees", "and", "toes"]
```

Just like spread for argument lists, `...` can be used anywhere in the array literal, and may be used more than once.

Copying an array

You can use spread syntax to make a shallow copy of an array. Each array element retains its identity without getting copied.

```
const arr = [1, 2, 3];
const arr2 = [...arr]; // like arr.slice()

arr2.push(4);
// arr2 becomes [1, 2, 3, 4]
// arr remains unaffected
```

Spread syntax effectively goes one level deep while copying an array. Therefore, it may be unsuitable for copying multidimensional arrays. The same is true with `Object.assign()` — no native operation in JavaScript does a deep clone. The web API method `structuredClone()` allows deep copying values of certain supported types. See shallow copy for more details.

```
const a = [[1], [2], [3]];
const b = [...a];
```

```
b.shift().shift();
// 1

// Oh no! Now array 'a' is affected as well:
console.log(a);
// [[], [2], [3]]
```

A better way to concatenate arrays

`Array.prototype.concat()` is often used to concatenate an array to the end of an existing array. Without spread syntax, this is done as:

```
let arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

// Append all items from arr2 onto arr1
arr1 = arr1.concat(arr2);
```

With spread syntax this becomes:

```
let arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

arr1 = [...arr1, ...arr2];
// arr1 is now [0, 1, 2, 3, 4, 5]
```

`Array.prototype.unshift()` is often used to insert an array of values at the start of an existing array. Without spread syntax, this is done as:

```
const arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

//  Prepend all items from arr2 onto arr1
Array.prototype.unshift.apply(arr1, arr2);
console.log(arr1); // [3, 4, 5, 0, 1, 2]
```

With spread syntax, this becomes:

```
let arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

arr1 = [...arr2, ...arr1];
console.log(arr1); // [3, 4, 5, 0, 1, 2]
```

> **Note:** Unlike `unshift()`, this creates a new `arr1`, instead of modifying the original `arr1` array in-place.

Conditionally adding values to an array

You can make an element present or absent in an array literal, depending on a condition, using a conditional operator.

```
const isSummer = false;
const fruits = ["apple", "banana", ...(isSummer ? ["watermelon"] : [])];
// ['apple', 'banana']
```

When the condition is `false`, we spread an empty array, so that nothing gets added to the final array. Note that this is different from the following:

```
const fruits = ["apple", "banana", isSummer ? "watermelon" : undefined];
// ['apple', 'banana', undefined]
```

In this case, an extra `undefined` element is added when `isSummer` is `false`, and this element will be visited by methods such as `Array.prototype.map()`.

Spread in object literals

Copying and merging objects

You can use spread syntax to merge multiple objects into one new object.

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { bar: "baz", y: 13 };

const mergedObj = { ...obj1, ...obj2 };
// { foo: "bar", x: 42, bar: "baz", y: 13 }
```

A single spread creates a shallow copy of the original object (but without non-enumerable properties and without copying the prototype), similar to copying an array.

```
const clonedObj = { ...obj1 };
// { foo: "bar", x: 42 }
```

Overriding properties

When one object is spread into another object, or when multiple objects are spread into one object, and properties with identical names are encountered, the property takes the last value assigned while remaining in the position it was originally set.

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };

const mergedObj = { x: 41, ...obj1, ...obj2, y: 9 }; // { x: 42, foo: "baz", y: 9 }
```

Conditionally adding properties to an object

You can make an element present or absent in an object literal, depending on a condition, using a conditional operator.

```
const isSummer = false;
const fruits = {
  apple: 10,
  banana: 5,
  ...(isSummer ? { watermelon: 30 } : {}),
};
// { apple: 10, banana: 5 }
```

The case where the condition is `false` is an empty object, so that nothing gets spread into the final object. Note that this is different from the following:

```
const fruits = {
  apple: 10,
  banana: 5,
  watermelon: isSummer ? 30 : undefined,
};
// { apple: 10, banana: 5, watermelon: undefined }
```

In this case, the `watermelon` property is always present and will be visited by methods such as `Object.keys()`.

Because primitives can be spread into objects as well, and from the observation that all falsy values do not have enumerable properties, you can simply use a logical AND operator:

```
const isSummer = false;
const fruits = {
  apple: 10,
  banana: 5,
  ...(isSummer && { watermelon: 30 }),
};
```

In this case, if `isSummer` is any falsy value, no property will be created on the `fruits` object.

Comparing with Object.assign()

Note that `Object.assign()` can be used to mutate an object, whereas spread syntax can't.

```
const obj1 = { foo: "bar", x: 42 };
Object.assign(obj1, { x: 1337 });
console.log(obj1); // { foo: "bar", x: 1337 }
```

In addition, `Object.assign()` triggers setters on the target object, whereas spread syntax does not.

```
const objectAssign = Object.assign(
  {
    set foo(val) {
      console.log(val);
    },
  },
  { foo: 1 },
);
// Logs "1"; objectAssign.foo is still the original setter

const spread = {
  set foo(val) {
    console.log(val);
  },
  ...{ foo: 1 },
};
// Nothing is logged; spread.foo is 1
```

You cannot naively re-implement the `Object.assign()` function through a single spreading:

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };
const merge = (...objects) => ({ ...objects });

const mergedObj1 = merge(obj1, obj2);
// { 0: { foo: 'bar', x: 42 }, 1: { foo: 'baz', y: 13 } }

const mergedObj2 = merge({}, obj1, obj2);
// { 0: {}, 1: { foo: 'bar', x: 42 }, 2: { foo: 'baz', y: 13 } }
```

In the above example, the spread syntax does not work as one might expect: it spreads an *array* of arguments into the object literal, due to the rest parameter. Here is an implementation of `merge` using the spread syntax, whose behavior is similar to `Object.assign()`, except that it doesn't trigger setters, nor mutates any object:

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };
const merge = (...objects) =>
  objects.reduce((acc, cur) => ({ ...acc, ...cur }));

const mergedObj = merge(obj1, obj2);
// { foo: 'baz', x: 42, y: 13 }
```

See also

- Rest parameters
- Rest property
- `Function.prototype.apply()`

Strict equality (===)

The **strict equality (`===`)** operator checks whether its two operands are equal, returning a Boolean result. Unlike the equality operator, the strict equality operator always considers operands of different types to be different.

Try it

```
console.log(1 === 1);
// Expected output: true

console.log("hello" === "hello");
// Expected output: true

console.log("1" === 1);
// Expected output: false

console.log(0 === false);
// Expected output: false
```

Syntax

```
x === y
```

Description

The strict equality operators (`===` and `!==`) provide the IsStrictlyEqual semantic.

- If the operands are of different types, return `false`.
- If both operands are objects, return `true` only if they refer to the same object.
- If both operands are `null` or both operands are `undefined`, return `true`.
- If either operand is `NaN`, return `false`.
- Otherwise, compare the two operand's values:
  - Numbers must have the same numeric values. `+0` and `-0` are considered to be the same value.
  - Strings must have the same characters in the same order.
  - Booleans must be both `true` or both `false`.

The most notable difference between this operator and the equality (`==`) operator is that if the operands are of different types, the `==` operator attempts to convert them to the same type before comparing.

Examples

Comparing operands of the same type

```
"hello" === "hello"; // true
"hello" === "hola"; // false

3 === 3; // true
3 === 4; // false

true === true; // true
true === false; // false

null === null; // true
```

Comparing operands of different types

```
"3" === 3; // false
true === 1; // false
null === undefined; // false
3 === new Number(3); // false
```

Comparing objects

```
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 === object2); // false
console.log(object1 === object1); // true
```

See also

- Equality (==)
- Inequality (!=)
- Strict inequality (!==)

Strict inequality (!==)

The **strict inequality (!==)** operator checks whether its two operands are not equal, returning a Boolean result. Unlike the inequality operator, the strict inequality operator always considers operands of different types to be different.

Try it

```
console.log(1 !== 1);
// Expected output: false

console.log("hello" !== "hello");
// Expected output: false

console.log("1" !== 1);
// Expected output: true

console.log(0 !== false);
// Expected output: true
```

Syntax

```
x !== y
```

Description

The strict inequality operator checks whether its operands are not equal. It is the negation of the strict equality operator so the following two lines will always give the same result:

```
x !== y;

!(x === y);
```

For details of the comparison algorithm, see the page for the strict equality operator.

Like the strict equality operator, the strict inequality operator will always consider operands of different types to be different:

```
3 !== "3"; // true
```

Examples

Comparing operands of the same type

```
"hello" !== "hello"; // false
"hello" !== "hola"; // true

3 !== 3; // false
3 !== 4; // true

true !== true; // false
true !== false; // true

null !== null; // false
```

Comparing operands of different types

```
"3" !== 3; // true
true !== 1; // true
null !== undefined; // true
```

Comparing objects

```
const object1 = {
  key: "value",
};
```

```
const object2 = {
  key: "value",
};

console.log(object1 !== object2); // true
console.log(object1 !== object1); // false
```

See also

- Equality (==)
- Inequality (!=)
- Strict equality (===)

Subtraction (-)

The **subtraction (-)** operator subtracts the two operands, producing their difference.

Try it

```
console.log(5 - 3);
// Expected output: 2

console.log(3.5 - 5);
// Expected output: -1.5

console.log(5 - "hello");
// Expected output: NaN

console.log(5 - true);
// Expected output: 4
```

Syntax

```
x - y
```

Description

The `-` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt subtraction if both operands become BigInts; otherwise, it performs number subtraction. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

Examples

Subtraction using numbers

```
5 - 3; // 2
3 - 5; // -2
```

Other non-BigInt values are coerced to numbers:

```
"foo" - 3; // NaN; "foo" is converted to the number NaN
5 - "3"; // 2; "3" is converted to the number 3
```

Subtraction using BigInts

```
2n - 1n; // 1n
```

You cannot mix BigInt and number operands in subtraction.

```
2n - 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 - 1n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do subtraction with a BigInt and a non-BigInt, convert either operand:

```
2n - BigInt(1); // 1n
Number(2n) - 1; // 1
```

See also

- Addition (+)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

Subtraction assignment (-=)

The **subtraction assignment (-=)** operator performs subtraction on the two operands and assigns the result to the left operand.

Try it

```
let a = 2;

console.log((a -= 3));
// Expected output: -1

console.log((a -= "Hello"));
// Expected output: NaN
```

Syntax

```
x -= y
```

Description

`x -= y` is equivalent to `x = x - y`, except that the expression `x` is only evaluated once.

Examples

Subtraction assignment using numbers

```
let bar = 5;

bar -= 2; // 3
```

Other non-BigInt values are coerced to numbers:

```
bar -= "foo"; // NaN
```

Subtraction assignment using BigInts

```
let foo = 3n;
foo -= 2n; // 1n
foo -= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

See also

- Assignment operators in the JS guide
- Subtraction (-)

super

The **`super`** keyword is used to access properties on an object literal or class's [[Prototype]], or invoke a superclass's constructor.

The `super.prop` and `super[expr]` expressions are valid in any method definition in both classes and object literals. The `super(...args)` expression is valid in class constructors.

Try it

```
class Foo {
  constructor(name) {
    this.name = name;
  }

  getNameSeparator() {
    return "-";
  }
}

class FooBar extends Foo {
  constructor(name, index) {
    super(name);
    this.index = index;
  }

  // Does not get called
  getNameSeparator() {
    return "/";
  }

  getFullName() {
    return this.name + super.getNameSeparator() + this.index;
  }
}

const firstFooBar = new FooBar("foo", 1);

console.log(firstFooBar.name);
```

```
// Expected output: "foo"

console.log(firstFooBar.getFullName());
// Expected output: "foo-1"
```

Syntax

```
super()
super(arg1)
super(arg1, arg2)
super(arg1, arg2, /* …, */ argN)

super.propertyOnParent
super[expression]
```

Description

The `super` keyword can be used in two ways: as a "function call" (`super(...args)`), or as a "property lookup" (`super.prop` and `super[expr]`).

> **Note:** `super` is a keyword and these are special syntactic constructs. `super` is not a variable that points to the prototype object. Attempting to read `super` itself is a `SyntaxError`.
>
> ```
> const child = {
>   myParent() {
>     console.log(super); // SyntaxError: 'super' keyword unexpected here
>   },
> };
> ```

In the constructor body of a derived class (with `extends`), the `super` keyword may appear as a "function call" (`super(...args)`), which must be called before the `this` keyword is used, and before the constructor returns. It calls the parent class's constructor and binds the parent class's public fields, after which the derived class's constructor can further access and modify `this`.

The "property lookup" form can be used to access methods and properties of an object literal's or class's [[Prototype]]. Within a class's body, the reference of `super` can be either the superclass's constructor itself, or the constructor's `prototype`, depending on whether the execution context is instance creation or class initialization. See the Examples section for more details.

Note that the reference of `super` is determined by the class or object literal `super` was declared in, not the object the method is called on. Therefore, unbinding or re-binding a method doesn't change the reference of `super` in it (although they do change the reference of `this`). You can see `super` as a variable in the class or object literal scope, which the methods create a closure over. (But also beware that it's not actually a variable, as explained above.)

When setting properties through `super`, the property is set on `this` instead.

Examples

Using super in classes

This code snippet is taken from the classes sample (live demo). Here `super()` is called to avoid duplicating the constructor parts' that are common between `Rectangle` and `Square`.

```
class Rectangle {
  constructor(height, width) {
    this.name = "Rectangle";
    this.height = height;
    this.width = width;
  }
  sayName() {
    console.log(`Hi, I am a ${this.name}.`);
  }
  get area() {
    return this.height * this.width;
  }
  set area(value) {
    this._area = value;
  }
}

class Square extends Rectangle {
  constructor(length) {
    // Here, it calls the parent class's constructor with lengths
    // provided for the Rectangle's width and height
    super(length, length);

    // Note: In derived classes, super() must be called before you
    // can use 'this'. Moving this to the top causes a ReferenceError.
    this.name = "Square";
  }
}
```

Super-calling static methods

You are also able to call super on static methods.

```
class Rectangle {
  static logNbSides() {
    return "I have 4 sides";
  }
}

class Square extends Rectangle {
  static logDescription() {
    return `${super.logNbSides()} which are all equal`;
  }
}
Square.logDescription(); // 'I have 4 sides which are all equal'
```

Accessing super in class field declaration

`super` can also be accessed during class field initialization. The reference of `super` depends on whether the current field is an instance field or a static field.

```
class Base {
  static baseStaticField = 90;
  baseMethod() {
    return 10;
  }
}

class Extended extends Base {
  extendedField = super.baseMethod(); // 10
  static extendedStaticField = super.baseStaticField; // 90
}
```

Note that instance fields are set on the instance instead of the constructor's `prototype`, so you can't use `super` to access the instance field of a superclass.

```
class Base {
  baseField = 10;
}

class Extended extends Base {
  extendedField = super.baseField; // undefined
}
```

Here, `extendedField` is `undefined` instead of 10, because `baseField` is defined as an own property of the `Base` instance, instead of `Base.prototype`. `super`, in this context, only looks up properties on `Base.prototype`, because that's the [[Prototype]] of `Extended.prototype`.

Deleting super properties will throw an error

You cannot use the `delete` operator and `super.prop` or `super[expr]` to delete a parent class' property — it will throw a `ReferenceError`.

```
class Base {
  foo() {}
}
class Derived extends Base {
  delete() {
    delete super.foo; // this is bad
  }
}

new Derived().delete(); // ReferenceError: invalid delete involving 'super'.
```

Using super.prop in object literals

Super can also be used in the object initializer notation. In this example, two objects define a method. In the second object, `super` calls the first object's method. This works with the help of `Object.setPrototypeOf()` with which we are able to set the prototype of `obj2` to `obj1`, so that `super` is able to find `method1` on `obj1`.

```
const obj1 = {
  method1() {
    console.log("method 1");
  },
};

const obj2 = {
  method2() {
    super.method1();
  },
};

Object.setPrototypeOf(obj2, obj1);
obj2.method2(); // Logs "method 1"
```

Methods that read super.prop do not behave differently when bound to other objects

Accessing `super.x` behaves like `Reflect.get(Object.getPrototypeOf(objectLiteral), "x", this)`, which means the property is always sought on the object literal/class declaration's prototype, and unbinding and re-binding a method won't change the reference of `super`.

```
class Base {
  baseGetX() {
    return 1;
  }
}
class Extended extends Base {
  getX() {
    return super.baseGetX();
  }
}

const e = new Extended();
console.log(e.getX()); // 1
const { getX } = e;
console.log(getX()); // 1
```

The same happens in object literals.

```
const parent1 = { prop: 1 };
const parent2 = { prop: 2 };

const child = {
  myParent() {
    console.log(super.prop);
  },
};

Object.setPrototypeOf(child, parent1);
child.myParent(); // Logs "1"

const myParent = child.myParent;
myParent(); // Still logs "1"

const anotherChild = { __proto__: parent2, myParent };
anotherChild.myParent(); // Still logs "1"
```

Only resetting the entire inheritance chain will change the reference of `super`.

```
class Base {
  baseGetX() {
    return 1;
  }
  static staticBaseGetX() {
    return 3;
  }
}
class AnotherBase {
  baseGetX() {
    return 2;
  }
  static staticBaseGetX() {
    return 4;
  }
}
class Extended extends Base {
  getX() {
    return super.baseGetX();
  }
  static staticGetX() {
    return super.staticBaseGetX();
  }
}

const e = new Extended();
// Reset instance inheritance
Object.setPrototypeOf(Extended.prototype, AnotherBase.prototype);
console.log(e.getX()); // Logs "2" instead of "1", because the prototype chain has changed
console.log(Extended.staticGetX()); // Still logs "3", because we haven't modified the static part yet
// Reset static inheritance
Object.setPrototypeOf(Extended, AnotherBase);
console.log(Extended.staticGetX()); // Now logs "4"
```

Calling methods from super

When calling `super.prop` as a function, the `this` value inside the `prop` function is the current `this`, not the object that `super` points to. For example, the `super.getName()` call logs `"Extended"`, despite the code looking like it's equivalent to `Base.getName()`.

```
class Base {
  static getName() {
    console.log(this.name);
  }
}

class Extended extends Base {
```

```
  static getName() {
    super.getName();
  }
}

Extended.getName(); // Logs "Extended"
```

This is especially important when interacting with static private elements.

Setting super.prop sets the property on this instead

Setting properties of `super`, such as `super.x = 1`, behaves like `Reflect.set(Object.getPrototypeOf(objectLiteral), "x", 1, this)`. This is one of the cases where understanding `super` as simply "reference of the prototype object" falls short, because it actually sets the property on `this` instead.

```
class A {}
class B extends A {
  setX() {
    super.x = 1;
  }
}

const b = new B();
b.setX();
console.log(b); // B { x: 1 }
console.log(Object.hasOwn(b, "x")); // true
```

`super.x = 1` will look for the property descriptor of `x` on `A.prototype` (and invoke the setters defined there), but the `this` value will be set to `this`, which is `b` in this context. You can read `Reflect.set` for more details on the case when `target` and `receiver` differ.

This means that while methods that *get* `super.prop` are usually not susceptible to changes in the `this` context, those that *set* `super.prop` are.

```
/* Reusing same declarations as above */

const b2 = new B();
b2.setX.call(null); // TypeError: Cannot assign to read only property 'x' of object 'null'
```

However, `super.x = 1` still consults the property descriptor of the prototype object, which means you cannot rewrite non-writable properties, and setters will be invoked.

```
class X {
  constructor() {
    // Create a non-writable property
    Object.defineProperty(this, "prop", {
      configurable: true,
      writable: false,
      value: 1,
    });
  }
}

class Y extends X {
  constructor() {
    super();
  }
  foo() {
    super.prop = 2; // Cannot overwrite the value.
  }
}

const y = new Y();
y.foo(); // TypeError: "prop" is read-only
console.log(y.prop); // 1
```

See also

- Classes

this

The **`this`** keyword refers to the context where a piece of code, such as a function's body, is supposed to run. Most typically, it is used in object methods, where `this` refers to the object that the method is attached to, thus allowing the same method to be reused on different objects.

The value of `this` in JavaScript depends on how a function is invoked (runtime binding), not how it is defined. When a regular function is invoked as a method of an object (`obj.method()`), `this` points to that object. When invoked as a standalone function (not attached to an object: `func()`), `this` typically refers to the global object (in non-strict mode) or `undefined` (in strict mode). The `Function.prototype.bind()` method can create a function whose `this` binding doesn't change, and methods `Function.prototype.apply()` and `Function.prototype.call()` can also set the `this` value for a particular call.

Arrow functions differ in their handling of `this`: they inherit `this` from the parent scope at the time they are defined. This behavior makes arrow functions particularly useful for callbacks and preserving context. However, arrow functions do not have their own `this` binding. Therefore, their `this` value cannot be set by `bind()`, `apply()` or `call()` methods, nor does it point to the current object in object methods.

Try it

```
const test = {
  prop: 42,
  func() {
    return this.prop;
  },
};

console.log(test.func());
// Expected output: 42
```

Syntax

```
this
```

Value

In non–strict mode, `this` is always a reference to an object. In strict mode, it can be any value. For more information on how the value is determined, see the description below.

Description

The value of `this` depends on in which context it appears: function, class, or global.

Function context

Inside a function, the value of `this` depends on how the function is called. Think about `this` as a hidden parameter of a function — just like the parameters declared in the function definition, `this` is a binding that the language creates for you when the function body is evaluated.

For a regular function (not an arrow function, bound function, etc.), the value of `this` is the object that the function is accessed on. In other words, if the function call is in the form `obj.f()`, then `this` refers to `obj`. For example:

```
function getThis() {
  return this;
}

const obj1 = { name: "obj1" };
const obj2 = { name: "obj2" };

obj1.getThis = getThis;
obj2.getThis = getThis;

console.log(obj1.getThis()); // { name: 'obj1', getThis: [Function: getThis] }
console.log(obj2.getThis()); // { name: 'obj2', getThis: [Function: getThis] }
```

Note how the function is the same, but based on how it's invoked, the value of `this` is different. This is analogous to how function parameters work.

The value of `this` is not the object that has the function as an own property, but the object that is used to call the function. You can prove this by calling a method of an object up in the prototype chain.

```
const obj3 = {
  __proto__: obj1,
  name: "obj3",
};

console.log(obj3.getThis()); // { name: 'obj3' }
```

The value of `this` always changes based on how a function is called, even when the function was defined on an object at creation:

```
const obj4 = {
  name: "obj4",
  getThis() {
    return this;
  },
};

const obj5 = { name: "obj5" };

obj5.getThis = obj4.getThis;
console.log(obj5.getThis()); // { name: 'obj5', getThis: [Function: getThis] }
```

If the value that the method is accessed on is a primitive, `this` will be a primitive value as well — but only if the function is in strict mode.

```
function getThisStrict() {
  "use strict"; // Enter strict mode
  return this;
}

// Only for demonstration — you should not mutate built-in prototypes
Number.prototype.getThisStrict = getThisStrict;
console.log(typeof (1).getThisStrict()); // "number"
```

If the function is called without being accessed on anything, `this` will be `undefined` — but only if the function is in strict mode.

```
console.log(typeof getThisStrict()); // "undefined"
```

In non-strict mode, a special process called `this` substitution ensures that the value of `this` is always an object. This means:

- If a function is called with `this` set to `undefined` or `null`, `this` gets substituted with `globalThis`.
- If the function is called with `this` set to a primitive value, `this` gets substituted with the primitive value's wrapper object.

```
function getThis() {
  return this;
}

// Only for demonstration — you should not mutate built-in prototypes
Number.prototype.getThis = getThis;
console.log(typeof (1).getThis()); // "object"
console.log(getThis() === globalThis); // true
```

In typical function calls, `this` is implicitly passed like a parameter through the function's prefix (the part before the dot). You can also explicitly set the value of `this` using the `Function.prototype.call()`, `Function.prototype.apply()`, or `Reflect.apply()` methods. Using `Function.prototype.bind()`, you can create a new function with a specific value of `this` that doesn't change regardless of how the function is called. When using these methods, the `this` substitution rules above still apply if the function is non-strict.

Callbacks

When a function is passed as a callback, the value of `this` depends on how the callback is called, which is determined by the implementor of the API. Callbacks are *typically* called with a `this` value of `undefined` (calling it directly without attaching it to any object), which means if the function is non–strict, the value of `this` is the global object (`globalThis`). This is the case for iterative array methods, the `Promise()` constructor, etc.

```
function logThis() {
  "use strict";
  console.log(this);
}

[1, 2, 3].forEach(logThis); // undefined, undefined, undefined
```

Some APIs allow you to set a `this` value for invocations of the callback. For example, all iterative array methods and related ones like `Set.prototype.forEach()` accept an optional `thisArg` parameter.

```
[1, 2, 3].forEach(logThis, { name: "obj" });
// { name: 'obj' }, { name: 'obj' }, { name: 'obj' }
```

Occasionally, a callback is called with a `this` value other than `undefined`. For example, the `reviver` parameter of `JSON.parse()` and the `replacer` parameter of `JSON.stringify()` are both called with `this` set to the object that the property being parsed/serialized belongs to.

Arrow functions

In arrow functions, `this` retains the value of the enclosing lexical context's `this`. In other words, when evaluating an arrow function's body, the language does not create a new `this` binding.

For example, in global code, `this` is always `globalThis` regardless of strictness, because of the global context binding:

```
const globalObject = this;
const foo = () => this;
console.log(foo() === globalObject); // true
```

Arrow functions create a closure over the `this` value of its surrounding scope, which means arrow functions behave as if they are "auto-bound" — no matter how it's invoked, `this` is bound to what it was when the function was created (in the example above, the global object). The same applies to arrow functions created inside other functions: their `this` remains that of the enclosing lexical context. See example below.

Furthermore, when invoking arrow functions using `call()`, `bind()`, or `apply()`, the `thisArg` parameter is ignored. You can still pass other arguments using these methods, though.

```
const obj = { name: "obj" };

// Attempt to set this using call
console.log(foo.call(obj) === globalObject); // true

// Attempt to set this using bind
const boundFoo = foo.bind(obj);
console.log(boundFoo() === globalObject); // true
```

Constructors

When a function is used as a constructor (with the `new` keyword), its `this` is bound to the new object being constructed, no matter which object the constructor function is accessed on. The value of `this` becomes the value of the `new` expression unless the constructor returns another non–primitive value.

```
function C() {
  this.a = 37;
}

let o = new C();
```

```
console.log(o.a); // 37

function C2() {
  this.a = 37;
  return { a: 38 };
}

o = new C2();
console.log(o.a); // 38
```

In the second example (`C2`), because an object was returned during construction, the new object that `this` was bound to gets discarded. (This essentially makes the statement `this.a = 37;` dead code. It's not exactly dead because it gets executed, but it can be eliminated with no outside effects.)

super

When a function is invoked in the `super.method()` form, the `this` inside the `method` function is the same value as the `this` value around the `super.method()` call, and is generally not equal to the object that `super` refers to. This is because `super.method` is not an object member access like the ones above — it's a special syntax with different binding rules. For examples, see the `super` reference.

Class context

A class can be split into two contexts: static and instance. Constructors, methods, and instance field initializers (public or private) belong to the instance context. Static methods, static field initializers, and static initialization blocks belong to the static context. The `this` value is different in each context.

Class constructors are always called with `new`, so their behavior is the same as function constructors: the `this` value is the new instance being created. Class methods behave like methods in object literals — the `this` value is the object that the method was accessed on. If the method is not transferred to another object, `this` is generally an instance of the class.

Static methods are not properties of `this`. They are properties of the class itself. Therefore, they are generally accessed on the class, and `this` is the value of the class (or a subclass). Static initialization blocks are also evaluated with `this` set to the current class.

Field initializers are also evaluated in the context of the class. Instance fields are evaluated with `this` set to the instance being constructed. Static fields are evaluated with `this` set to the current class. This is why arrow functions in field initializers are bound to the instance for instance fields and to the class for static fields.

```
class C {
  instanceField = this;
  static staticField = this;
}

const c = new C();
console.log(c.instanceField === c); // true
console.log(C.staticField === C); // true
```

Derived class constructors

Unlike base class constructors, derived constructors have no initial `this` binding. Calling `super()` creates a `this` binding within the constructor and essentially has the effect of evaluating the following line of code, where `Base` is the base class:

```
this = new Base();
```

> **Warning:** Referring to `this` before calling `super()` will throw an error.

Derived classes must not return before calling `super()`, unless the constructor returns an object (so the `this` value is overridden) or the class has no constructor at all.

```
class Base {}
class Good extends Base {}
class AlsoGood extends Base {
  constructor() {
    return { a: 5 };
  }
}
class Bad extends Base {
  constructor() {}
}

new Good();
new AlsoGood();
new Bad(); // ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from derived constructor
```

Global context

In the global execution context (outside of any functions or classes; may be inside blocks or arrow functions defined in the global scope), the `this` value depends on what execution context the script runs in. Like callbacks, the `this` value is determined by the runtime environment (the caller).

At the top level of a script, `this` refers to `globalThis` whether in strict mode or not. This is generally the same as the global object — for example, if the source is put inside an HTML `<script>` element and executed as a script, `this === window`.

> **Note:** `globalThis` is generally the same concept as the global object (i.e., adding properties to `globalThis` makes them global variables) — this is the case for

> browsers and Node — but hosts are allowed to provide a different value for `globalThis` that's unrelated to the global object.

```
// In web browsers, the window object is also the global object:
console.log(this === window); // true

this.b = "MDN";
console.log(window.b); // "MDN"
console.log(b); // "MDN"
```

If the source is loaded as a module (for HTML, this means adding `type="module"` to the `<script>` tag), `this` is always `undefined` at the top level.

If the source is executed with `eval()`, `this` is the same as the enclosing context for direct eval, or `globalThis` (as if it's run in a separate global script) for indirect eval.

```
function test() {
  // Direct eval
  console.log(eval("this") === this);
  // Indirect eval, non-strict
  console.log(eval?.("this") === globalThis);
  // Indirect eval, strict
  console.log(eval?.("'use strict'; this") === globalThis);
}

test.call({ name: "obj" }); // Logs 3 "true"
```

Note that some source code, while looking like the global scope, is actually wrapped in a function when executed. For example, Node.js CommonJS modules are wrapped in a function and executed with the `this` value set to `module.exports`. Event handler attributes are executed with `this` set to the element they are attached to.

Object literals don't create a `this` scope — only functions (methods) defined within the object do. Using `this` in an object literal inherits the value from the surrounding scope.

```
const obj = {
  a: this,
};

console.log(obj.a === window); // true
```

Examples

this in function contexts

The value of the `this` parameter depends on how the function is called, not on how it's defined.

```
// An object can be passed as the first argument to 'call'
// or 'apply' and 'this' will be bound to it.
const obj = { a: "Custom" };

// Variables declared with var become properties of 'globalThis'.
var a = "Global";

function whatsThis() {
  return this.a; // 'this' depends on how the function is called
}

whatsThis(); // 'Global'; the 'this' parameter defaults to 'globalThis' in non-strict mode
obj.whatsThis = whatsThis;
obj.whatsThis(); // 'Custom'; the 'this' parameter is bound to obj
```

Using `call()` and `apply()`, you can pass the value of `this` as if it's an explicit parameter.

```
function add(c, d) {
  return this.a + this.b + c + d;
}

const o = { a: 1, b: 3 };

// The first argument is bound to the implicit 'this' parameter; the remaining
// arguments are bound to the named parameters.
add.call(o, 5, 7); // 16

// The first argument is bound to the implicit 'this' parameter; the second
// argument is an array whose members are bound to the named parameters.
add.apply(o, [10, 20]); // 34
```

this and object conversion

In non–strict mode, if a function is called with a `this` value that's not an object, the `this` value is substituted with an object. `null` and `undefined` become `globalThis`. Primitives like `7` or `'foo'` are converted to an object using the related constructor, so the primitive number `7` is converted to a `Number` wrapper class and the string `'foo'` to a `String` wrapper class.

```
function bar() {
  console.log(Object.prototype.toString.call(this));
```

```
}

bar.call(7); // [object Number]
bar.call("foo"); // [object String]
bar.call(undefined); // [object Window]
```

The bind() method

Calling `f.bind(someObject)` creates a new function with the same body and scope as `f`, but the value of `this` is permanently bound to the first argument of `bind`, regardless of how the function is being called.

```
function f() {
  return this.a;
}

const g = f.bind({ a: "azerty" });
console.log(g()); // azerty

const h = g.bind({ a: "yoo" }); // bind only works once!
console.log(h()); // azerty

const o = { a: 37, f, g, h };
console.log(o.a, o.f(), o.g(), o.h()); // 37 37 azerty azerty
```

this in arrow functions

Arrow functions create closures over the `this` value of the enclosing execution context. In the following example, we create `obj` with a method `getThisGetter` that returns a function that returns the value of `this`. The returned function is created as an arrow function, so its `this` is permanently bound to the `this` of its enclosing function. The value of `this` inside `getThisGetter` can be set in the call, which in turn sets the return value of the returned function. We will assume that `getThisGetter` is a non-strict function, which means it's contained in a non-strict script and not further nested in a class or strict function.

```
const obj = {
  getThisGetter() {
    const getter = () => this;
    return getter;
  },
};
```

We can call `getThisGetter` as a method of `obj`, which binds `this` to `obj` inside its body. The returned function is assigned to a variable `fn`. Now, when calling `fn`, the value of `this` returned is still the one set by the call to `getThisGetter`, which is `obj`. If the returned function was not an arrow function, such calls would cause the `this` value to be `globalThis`, because `getThisGetter` is non-strict.

```
const fn = obj.getThisGetter();
console.log(fn() === obj); // true
```

But be careful if you unbind the method of `obj` without calling it, because `getThisGetter` is still a method that has a varying `this` value. Calling `fn2()()` in the following example returns `globalThis`, because it follows the `this` from `fn2()`, which is `globalThis` since it's called without being attached to any object.

```
const fn2 = obj.getThisGetter;
console.log(fn2()() === globalThis); // true in non-strict mode
```

This behavior is very useful when defining callbacks. Usually, each function expression creates its own `this` binding, which shadows the `this` value of the upper scope. Now, you can define functions as arrow functions if you don't care about the `this` value, and only create `this` bindings where you do (e.g., in class methods). See example with `setTimeout()`.

this with a getter or setter

`this` in getters and setters is based on which object the property is accessed on, not which object the property is defined on. A function used as getter or setter has its `this` bound to the object from which the property is being set or gotten.

```
function sum() {
  return this.a + this.b + this.c;
}

const o = {
  a: 1,
  b: 2,
  c: 3,
  get average() {
    return (this.a + this.b + this.c) / 3;
  },
};

Object.defineProperty(o, "sum", {
  get: sum,
  enumerable: true,
  configurable: true,
});

console.log(o.average, o.sum); // 2 6
```

this in DOM event handlers

When a function is used as an event handler, its `this` parameter is bound to the DOM element on which the listener is placed (some browsers do not follow this convention for listeners added dynamically with methods other than `addEventListener()`).

```
// When called as a listener, turns the related element blue
function bluify(e) {
  // Always true
  console.log(this === e.currentTarget);
  // true when currentTarget and target are the same object
  console.log(this === e.target);
  this.style.backgroundColor = "#A5D9F3";
}

// Get a list of every element in the document
const elements = document.getElementsByTagName("*");

// Add bluify as a click listener so when the
// element is clicked on, it turns blue
for (const element of elements) {
  element.addEventListener("click", bluify);
}
```

this in inline event handlers

When the code is called from an inline event handler attribute, its `this` is bound to the DOM element on which the listener is placed:

```
<button onclick="alert(this.tagName.toLowerCase());">Show this</button>
```

The above alert shows `button`. Note, however, that only the outer scope has its `this` bound this way:

```
<button onclick="alert((function () { return this; })());">
  Show inner this
</button>
```

In this case, the `this` parameter of the inner function is bound to `globalThis` (i.e., the default object in non–strict mode where `this` isn't passed in the call).

Bound methods in classes

Just like with regular functions, the value of `this` within methods depends on how they are called. Sometimes it is useful to override this behavior so that `this` within classes always refers to the class instance. To achieve this, bind the class methods in the constructor:

```
class Car {
  constructor() {
    // Bind sayBye but not sayHi to show the difference
    this.sayBye = this.sayBye.bind(this);
  }

  sayHi() {
    console.log(`Hello from ${this.name}`);
  }

  sayBye() {
    console.log(`Bye from ${this.name}`);
  }

  get name() {
    return "Ferrari";
  }
}

class Bird {
  get name() {
    return "Tweety";
  }
}

const car = new Car();
const bird = new Bird();

// The value of 'this' in methods depends on their caller
car.sayHi(); // Hello from Ferrari
bird.sayHi = car.sayHi;
bird.sayHi(); // Hello from Tweety

// For bound methods, 'this' doesn't depend on the caller
bird.sayBye = car.sayBye;
bird.sayBye(); // Bye from Ferrari
```

> **Note:** Classes are always in strict mode. Calling methods with an undefined `this` will throw an error if the method tries to access properties on `this`.

```
const carSayHi = car.sayHi;
carSayHi(); // TypeError because the 'sayHi' method tries to access 'this.name', but 'this' is undefined in strict mode.
```

Note, however, that auto-bound methods suffer from the same problem as using arrow functions for class properties: each instance of the class will have its own copy of the method, which increases memory usage. Only use it where absolutely necessary. You can also mimic the implementation of `Intl.NumberFormat.prototype.format()`: define the property as a getter that returns a bound function when accessed and saves it, so that the function is only created once and only created when necessary.

this in with statements

Although `with` statements are deprecated and not available in strict mode, they still serve as an exception to the normal `this` binding rules. If a function is called within a `with` statement and that function is a property of the scope object, the `this` value is bound to the scope object, as if the `obj.` prefix exists.

```
const obj = {
  foo() {
    return this;
  },
};

with (obj) {
  console.log(foo() === obj); // true
}
```

See also

- Strict mode
- `globalThis`

typeof

The **`typeof`** operator returns a string indicating the type of the operand's value.

Try it

```
console.log(typeof 42);
// Expected output: "number"

console.log(typeof "blubber");
// Expected output: "string"

console.log(typeof true);
// Expected output: "boolean"

console.log(typeof undeclaredVariable);
// Expected output: "undefined"
```

Syntax

```
typeof operand
```

Parameters

`operand`

An expression representing the object or primitive whose type is to be returned.

Description

The following table summarizes the possible return values of `typeof`. For more information about types and primitives, see also the JavaScript data structure page.

Type	Result
Undefined	`"undefined"`
Null	`"object"` (reason)
Boolean	`"boolean"`
Number	`"number"`
BigInt	`"bigint"`
String	`"string"`
Symbol	`"symbol"`
Function (implements [[Call]] in ECMA-262 terms; classes are functions as well)	`"function"`
Any other object	`"object"`

This list of values is exhaustive. No spec-compliant engines are reported to produce (or had historically produced) values other than those listed.

Examples

Basic usage

```
// Numbers
typeof 37 === "number";
typeof 3.14 === "number";
typeof 42 === "number";
typeof Math.LN2 === "number";
typeof Infinity === "number";
typeof NaN === "number"; // Despite being "Not-A-Number"
typeof Number("1") === "number"; // Number tries to parse things into numbers
typeof Number("shoe") === "number"; // including values that cannot be type coerced to a number

typeof 42n === "bigint";

// Strings
typeof "" === "string";
typeof "bla" === "string";
typeof `template literal` === "string";
typeof "1" === "string"; // note that a number within a string is still typeof string
typeof typeof 1 === "string"; // typeof always returns a string
typeof String(1) === "string"; // String converts anything into a string, safer than toString

// Booleans
typeof true === "boolean";
typeof false === "boolean";
typeof Boolean(1) === "boolean"; // Boolean() will convert values based on if they're truthy or falsy
typeof !!1 === "boolean"; // two calls of the ! (logical NOT) operator are equivalent to Boolean()

// Symbols
typeof Symbol() === "symbol";
typeof Symbol("foo") === "symbol";
typeof Symbol.iterator === "symbol";

// Undefined
typeof undefined === "undefined";
typeof declaredButUndefinedVariable === "undefined";
typeof undeclaredVariable === "undefined";

// Objects
typeof { a: 1 } === "object";

// use Array.isArray or Object.prototype.toString.call
// to differentiate regular objects from arrays
typeof [1, 2, 4] === "object";

typeof new Date() === "object";
typeof /regex/ === "object";

// The following are confusing, dangerous, and wasteful. Avoid them.
typeof new Boolean(true) === "object";
typeof new Number(1) === "object";
typeof new String("abc") === "object";

// Functions
typeof function () {} === "function";
typeof class C {} === "function";
typeof Math.sin === "function";
```

typeof null

```
// This stands since the beginning of JavaScript
typeof null === "object";
```

In the first implementation of JavaScript, JavaScript values were represented as a type tag and a value. The type tag for objects was `0`. `null` was represented as the NULL pointer (`0x00` in most platforms). Consequently, `null` had `0` as type tag, hence the `typeof` return value `"object"`. (reference)

A fix was proposed for ECMAScript (via an opt-in), but was rejected. It would have resulted in `typeof null === "null"`.

Using new operator

All constructor functions called with `new` will return non-primitives (`"object"` or `"function"`). Most return objects, with the notable exception being `Function`, which returns a function.

```
const str = new String("String");
const num = new Number(100);

typeof str; // "object"
typeof num; // "object"

const func = new Function();

typeof func; // "function"
```

Need for parentheses in syntax

The `typeof` operator has higher precedence than binary operators like addition (`+`). Therefore, parentheses are needed to evaluate the type of an addition result.

```
// Parentheses can be used for determining the data type of expressions.
const someData = 99;

typeof someData + " foo"; // "number foo"
typeof (someData + " foo"); // "string"
```

Interaction with undeclared and uninitialized variables

`typeof` works with undeclared identifiers, returning `"undefined"` instead of throwing an error.

```
typeof undeclaredVariable; // "undefined"
```

However, using `typeof` on lexical declarations (`let const`, `using await using`, and `class`) in the same block before the place of declaration will throw a `ReferenceError`. Block scoped variables are in a *temporal dead zone* from the start of the block until the initialization is processed, during which it will throw an error if accessed.

```
typeof newLetVariable; // ReferenceError
typeof newConstVariable; // ReferenceError
typeof newClass; // ReferenceError

let newLetVariable;
const newConstVariable = "hello";
class newClass {}
```

See `typeof` operator and `undefined` for more details.

Exceptional behavior of document.all

All current browsers expose a non-standard host object `document.all` with type `undefined`.

```
typeof document.all === "undefined";
```

Although `document.all` is also falsy and loosely equal to `undefined`, it is not `undefined`. The case of `document.all` having type `"undefined"` is classified in the web standards as a "willful violation" of the original ECMAScript standard for web compatibility.

Custom method that gets a more specific type

`typeof` is very useful, but it's not as versatile as might be required. For example, `typeof []` is `"object"`, as well as `typeof new Date()`, `typeof /abc/`, etc.

For greater specificity in checking types, here we present a custom `type(value)` function, which mostly mimics the behavior of `typeof`, but for non-primitives (i.e., objects and functions), it returns a more granular type name where possible.

```
function type(value) {
  if (value === null) {
    return "null";
  }
  const baseType = typeof value;
  // Primitive types
  if (!["object", "function"].includes(baseType)) {
    return baseType;
  }

  // Symbol.toStringTag often specifies the "display name" of the
  // object's class. It's used in Object.prototype.toString().
  const tag = value[Symbol.toStringTag];
  if (typeof tag === "string") {
    return tag;
  }

  // If it's a function whose source code starts with the "class" keyword
  if (
    baseType === "function" &&
    Function.prototype.toString.call(value).startsWith("class")
  ) {
    return "class";
  }

  // The name of the constructor; for example `Array`, `GeneratorFunction`,
  // `Number`, `String`, `Boolean` or `MyCustomClass`
  const className = value.constructor.name;
  if (typeof className === "string" && className !== "") {
    return className;
  }

  // At this point there's no robust way to get the type of value,
  // so we use the base implementation.
  return baseType;
}
```

For checking potentially non-existent variables that would otherwise throw a `ReferenceError`, use `typeof nonExistentVar === "undefined"` because this behavior cannot be mimicked with custom code.

See also

- `instanceof`
- `document.all` willful violation of the standard

Unary negation (-)

The **unary negation (-)** operator precedes its operand and negates it.

Try it

```
const x = 4;
const y = -x;

console.log(y);
// Expected output: -4

const a = "4";
const b = -a;

console.log(b);
// Expected output: -4
```

Syntax

```
-x
```

Description

The - operator is overloaded for two types of operands: number and BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt negation if the operand becomes a BigInt; otherwise, it performs number negation.

Examples

Negating numbers

```
const x = 3;
const y = -x;
// y is -3; x is 3
```

Negating non-numbers

The unary negation operator can convert a non-number into a number.

```
const x = "4";
const y = -x;

// y is -4
```

BigInts can be negated using the unary negation operator.

```
const x = 4n;
const y = -x;

// y is -4n
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary plus (+)

Unary plus (+)

The **unary plus (+)** operator precedes its operand and evaluates to its operand but attempts to convert it into a number, if it isn't already.

Try it

```
const x = 1;
const y = -1;

console.log(+x);
// Expected output: 1

console.log(+y);
// Expected output: -1
```

```
console.log(+"");
// Expected output: 0

console.log(+true);
// Expected output: 1

console.log(+false);
// Expected output: 0

console.log(+"hello");
// Expected output: NaN
```

Syntax

```
+x
```

Description

Although unary negation (-) also can convert non-numbers, unary plus is the fastest and preferred way of converting something into a number, because it does not perform any other operations on the number.

Unary plus does the exact same steps as normal number coercion used by most built-in methods expecting numbers. It can convert string representations of integers and floats, as well as the non-string values `true`, `false`, and `null`. Integers in both decimal and hexadecimal (`0x`-prefixed) formats are supported. Negative numbers are supported (though not for hex). If it cannot parse a particular value, it will evaluate to `NaN`. Unlike other arithmetic operators, which work with both numbers and BigInts, using the + operator on BigInt values throws a `TypeError`.

Examples

Usage with numbers

```
const x = 1;
const y = -1;

console.log(+x);
// 1
console.log(+y);
// -1
```

Usage with non-numbers

```
+true  // 1
+false // 0
+null  // 0
+[]    // 0
+function (val) { return val; } // NaN
+1n    // throws TypeError: Cannot convert BigInt value to number
```

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)

Unsigned right shift (>>>)

The **unsigned right shift (>>>)** operator returns a number whose binary representation is the first operand shifted by the specified number of bits to the right. Excess bits shifted off to the right are discarded, and zero bits are shifted in from the left. This operation is also called "zero-filling right shift", because the sign bit becomes `0`, so the resulting number is always positive. Unsigned right shift does not accept BigInt values.

Try it

```
const a = 5; //  00000000000000000000000000000101
const b = 2; //  00000000000000000000000000000010
const c = -5; //  11111111111111111111111111111011

console.log(a >>> b); //  00000000000000000000000000000001
// Expected output: 1

console.log(c >>> b); //  00111111111111111111111111111110
// Expected output: 1073741822
```

Syntax

```
x >>> y
```

Description

Unlike other arithmetic and bitwise operators, the unsigned right shift operator does not accept BigInt values. This is because it fills the leftmost bits with zeroes, but conceptually, BigInts have an infinite number of leading sign bits, so there's no "leftmost bit" to fill with zeroes.

The operator operates on the left operand's bit representation in two's complement. Consider the 32-bit binary representations of the decimal (base 10) numbers `9` and `-9`:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
    -9 (base 10): 11111111111111111111111111110111 (base 2)
```

The binary representation under two's complement of the negative decimal (base 10) number `-9` is formed by inverting all the bits of its opposite number, which is `9` and `00000000000000000000000000001001` in binary, and adding `1`.

In both cases, the sign of the binary number is given by its leftmost bit: for the positive decimal number `9`, the leftmost bit of the binary representation is `0`, and for the negative decimal number `-9`, the leftmost bit of the binary representation is `1`.

Given those binary representations of the decimal (base 10) numbers `9`, and `-9`:

For the positive number `9`, zero-fill right shift and sign-propagating right shift yield the same result: `9 >>> 2` yields `2`, the same as `9 >> 2`:

```
      9 (base 10): 00000000000000000000000000001001 (base 2)
                   --------------------------------
9 >>  2 (base 10): 00000000000000000000000000000010 (base 2) = 2 (base 10)
9 >>> 2 (base 10): 00000000000000000000000000000010 (base 2) = 2 (base 10)
```

Notice how two rightmost bits, `01`, have been shifted off, and two zeroes have been shifted in from the left.

However, notice what happens for `-9`: `-9 >> 2` (sign-propagating right shift) yields `-3`, but `-9 >>> 2` (zero-fill right shift) yields 1073741821:

```
      -9 (base 10): 11111111111111111111111111110111 (base 2)
                    --------------------------------
-9 >>  2 (base 10): 11111111111111111111111111111101 (base 2) = -3 (base 10)
-9 >>> 2 (base 10): 00111111111111111111111111111101 (base 2) = 1073741821 (base 10)
```

Notice how two rightmost bits, `11`, have been shifted off. For `-9 >> 2` (sign-propagating right shift), two copies of the leftmost `1` bit have been shifted in from the left, which preserves the negative sign. On the other hand, for `-9 >>> 2` (zero-fill right shift), zeroes have instead been shifted in from the left, so the negative sign of the number is not preserved, and the result is instead a (large) positive number.

If the left operand is a number with more than 32 bits, it will get the most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

The right operand will be converted to an unsigned 32-bit integer and then taken modulo 32, so the actual shift offset will always be a positive integer between 0 and 31, inclusive. For example, `100 >>> 32` is the same as `100 >>> 0` (and produces `100`) because 32 modulo 32 is 0.

Examples

Using unsigned right shift

```
9 >>> 2; // 2
-9 >>> 2; // 1073741821
```

Unsigned right shift doesn't work with BigInts.

```
9n >>> 2n; // TypeError: BigInts have no unsigned right shift, use >> instead
```

See also

- Bitwise operators in the JS guide
- Unsigned right shift assignment (>>>=)

Unsigned right shift assignment (>>>=)

The **unsigned right shift assignment (>>>=)** operator performs unsigned right shift on the two operands and assigns the result to the left operand.

Try it

```
let a = 5; //  00000000000000000000000000000101

a >>>= 2; //  00000000000000000000000000000001
console.log(a);
// Expected output: 1

let b = -5; // -00000000000000000000000000000101

b >>>= 2; //  00111111111111111111111111111110
console.log(b);
// Expected output: 1073741822
```

Syntax

```
x >>>= y
```

Description

`x >>>= y` is equivalent to `x = x >>> y`, except that the expression `x` is only evaluated once.

Examples

Using unsigned right shift assignment

```
let a = 5; // (00000000000000000000000000000101)
a >>>= 2; // 1 (00000000000000000000000000000001)

let b = -5; // (-00000000000000000000000000000101)
b >>>= 2; // 1073741822 (00111111111111111111111111111110)

let c = 5n;
c >>>= 2n; // 1n
```

See also

- Assignment operators in the JS guide
- Unsigned right shift (>>>)

void operator

The **`void`** operator evaluates the given `expression` and then returns `undefined`.

Try it

```
const output = void 1;
console.log(output);
// Expected output: undefined

void console.log("expression evaluated");
// Expected output: "expression evaluated"

void (function iife() {
  console.log("iife is executed");
})();
// Expected output: "iife is executed"

void function test() {
  console.log("test function executed");
};
try {
  test();
} catch (e) {
  console.log("test function is not defined");
  // Expected output: "test function is not defined"
}
```

Syntax

```
void expression
```

Description

This operator allows evaluating expressions that produce a value into places where an expression that evaluates to `undefined` is desired.

The `void` operator is often used merely to obtain the `undefined` primitive value, usually using `void(0)` (which is equivalent to `void 0`). In these cases, the global variable `undefined` can be used.

It should be noted that the precedence of the `void` operator should be taken into account and that parentheses can help clarify the resolution of the expression following the `void` operator:

```
void 2 === "2"; // (void 2) === '2', returns false
void (2 === "2"); // void (2 === '2'), returns undefined
```

Examples

Immediately Invoked Function Expressions

When using an immediately-invoked function expression, the `function` keyword cannot be at the immediate start of the statement, because that would be parsed as a function declaration, and would generate a syntax error when the parentheses representing invocation is reached — if the function is unnamed, it would immediately be a syntax error if the function is parsed as a declaration.

```
function iife() {
  console.log("Executed!");
}(); // SyntaxError: Unexpected token ')'
```

```
function () {
  console.log("Executed!");
}(); // SyntaxError: Function statements require a function name
```

In order for the function to be parsed as an expression, the `function` keyword has to appear at a position that only accepts expressions, not statements. This can be achieved by prefixing the keyword with a unary operator, which only accepts expressions as operands. Function invocation has higher precedence than unary operators, so it will be executed first. Its return value (which is almost always `undefined`) will be passed to the unary operator and then immediately discarded.

Of all the unary operators, `void` offers the best semantic, because it clearly signals that the return value of the function invocation should be discarded.

```
void function () {
  console.log("Executed!");
}();

// Logs "Executed!"
```

This is a bit longer than wrapping the function expression in parentheses, which has the same effect of forcing the `function` keyword to be parsed as the start of an expression instead of a statement.

```
(function () {
  console.log("Executed!");
})();
```

Note that this trick only applies to IIFEs defined with the `function` keyword. Attempting to use the `void` operator to avoid parentheses for an arrow function results in a syntax error. Arrow function expressions always require parentheses around them when being called.

```
void () => { console.log("iife!"); }(); // SyntaxError: Malformed arrow function parameter list
```

JavaScript URIs

When a browser follows a `javascript:` URI, it evaluates the code in the URI and then replaces the contents of the page with the returned value, unless the returned value is `undefined`. The `void` operator can be used to return `undefined`. For example:

```
<a href="javascript:void(0);">Click here to do nothing</a>

<a href="javascript:void(document.body.style.backgroundColor='green');">
  Click here for green background
</a>
```

> **Note:** `javascript:` pseudo protocol is discouraged over other alternatives, such as unobtrusive event handlers.

Non-leaking Arrow Functions

Arrow functions introduce a short-hand braceless syntax that returns an expression. This can cause unintended side effects if the expression is a function call where the returned value changes from `undefined` to some other value.

For example, if `doSomething()` returns `false` in the code below, the checkbox will no longer be marked as checked or unchecked when the checkbox is clicked (returning `false` from the handler disables the default action).

```
checkbox.onclick = () => doSomething();
```

This is unlikely to be desired behavior! To be safe, when the return value of a function is not intended to be used, it can be passed to the `void` operator to ensure that (for example) changing APIs do not cause arrow functions' behaviors to change.

```
checkbox.onclick = () => void doSomething();
```

See also

- `undefined`

yield

The **`yield`** operator is used to pause and resume a generator function.

Try it

```
function* foo(index) {
  while (index < 2) {
    yield index;
    index++;
  }
}

const iterator = foo(0);

console.log(iterator.next().value);
// Expected output: 0
```

```
console.log(iterator.next().value);
// Expected output: 1
```

Syntax

```
yield
yield expression
```

Parameters

`expression` Optional

The value to yield from the generator function via the iterator protocol. If omitted, `undefined` is yielded.

Return value

Returns the optional value passed to the generator's `next()` method to resume its execution.

> **Note:** This means `next()` is asymmetric: it always sends a value to the currently suspended `yield`, but returns the operand of the next `yield`. The argument passed to the first `next()` call cannot be retrieved because there's no currently suspended `yield`.

Description

The `yield` keyword pauses generator function execution and the value of the expression following the `yield` keyword is returned to the generator's caller. It can be thought of as a generator-based version of the `return` keyword.

`yield` can only be used directly within the generator function that contains it. It cannot be used within nested functions.

Calling a generator function constructs a `Generator` object. Each time the generator's `next()` method is called, the generator resumes execution, and runs until it reaches one of the following:

- A `yield` expression. In this case, the generator pauses, and the `next()` method return an iterator result object with two properties: `value` and `done`. The `value` property is the value of the expression after the `yield` operator, and `done` is `false`, indicating that the generator function has not fully completed.
- The end of the generator function. In this case, execution of the generator ends, and the `next()` method returns an iterator result object where the `value` is `undefined` and `done` is `true`.
- A `return` statement. In this case, execution of the generator ends, and the `next()` method returns an iterator result object where the `value` is the specified return value and `done` is `true`.
- A `throw` statement. In this case, execution of the generator halts entirely, and the `next()` method throws the specified exception.

Once paused on a `yield` expression, the generator's code execution remains paused until the generator's `next()` method is called again. If an optional value is passed to the generator's `next()` method, that value becomes the value returned by the generator's current `yield` operation. The first `next()` call does not have a corresponding suspended `yield` operation, so there's no way to get the argument passed to the first `next()` call.

If the generator's `return()` or `throw()` method is called, it acts as if a `return` or `throw` statement was executed at the paused `yield` expression. You can use `try...catch...finally` within the generator function body to handle these early completions. If the `return()` or `throw()` method is called but there's no suspended `yield` expression (because `next()` has not been called yet, or because the generator has already completed), then the early completions cannot be handled and always terminate the generator.

Examples

Using yield

The following code is the declaration of an example generator function.

```
function* countAppleSales() {
  const saleList = [3, 7, 5];
  for (const sale of saleList) {
    yield sale;
  }
}
```

Once a generator function is defined, it can be used by constructing an iterator as shown.

```
const appleStore = countAppleSales(); // Generator { }
console.log(appleStore.next()); // { value: 3, done: false }
console.log(appleStore.next()); // { value: 7, done: false }
console.log(appleStore.next()); // { value: 5, done: false }
console.log(appleStore.next()); // { value: undefined, done: true }
```

You can also send a value with `next(value)` into the generator. `step` evaluates as a return value of the `yield` expression — although the value passed to the generator's `next()` method the first time `next()` is called is ignored.

```
function* counter(value) {
  while (true) {
    const step = yield value++;

    if (step) {
      value += step;
    }
```

```
  }
}

const generatorFunc = counter(0);
console.log(generatorFunc.next().value); // 0
console.log(generatorFunc.next().value); // 1
console.log(generatorFunc.next().value); // 2
console.log(generatorFunc.next().value); // 3
console.log(generatorFunc.next(10).value); // 14
console.log(generatorFunc.next().value); // 15
console.log(generatorFunc.next(10).value); // 26
```

See also

- Iteration protocols
- `function*`
- `function*` expression
- `yield*`

Regular expressions

A **regular expression** (*regex* for short) allow developers to match strings against a pattern, extract submatch information, or simply test if the string conforms to that pattern. Regular expressions are used in many programming languages, and JavaScript's syntax is inspired by Perl.

You are encouraged to read the regular expressions guide to get an overview of the available regex syntaxes and how they work.

Description

Regular expressions are an important concept in formal language theory. They are a way to describe a possibly infinite set of character strings (called a *language*). A regular expression, at its core, needs the following features:

- A set of *characters* that can be used in the language, called the *alphabet*.
- *Concatenation*: `ab` means "the character `a` followed by the character `b`".
- *Union*: `a|b` means "either `a` or `b`".
- *Kleene star*: `a*` means "zero or more `a` characters".

Assuming a finite alphabet (such as the 26 letters of the English alphabet, or the entire Unicode character set), all regular languages can be generated by the features above. Of course, many patterns are very tedious to express this way (such as "10 digits" or "a character that's not a space"), so JavaScript regular expressions include many shorthands, introduced below.

> **Note:** JavaScript regular expressions are in fact not regular, due to the existence of backreferences (regular expressions must have finite states). However, they are still a very useful feature.

Creating regular expressions

A regular expression is typically created as a literal by enclosing a pattern in forward slashes (`/`):

```
const regex1 = /ab+c/g;
```

Regular expressions can also be created with the `RegExp()` constructor:

```
const regex2 = new RegExp("ab+c", "g");
```

They have no runtime differences, although they may have implications on performance, static analyzability, and authoring ergonomic issues with escaping characters. For more information, see the `RegExp` reference.

Regex flags

Flags are special parameters that can change the way a regular expression is interpreted or the way it interacts with the input text. Each flag corresponds to one accessor property on the `RegExp` object.

Flag	Description	Corresponding property
`d`	Generate indices for substring matches.	`hasIndices`
`g`	Global search.	`global`
`i`	Case-insensitive search.	`ignoreCase`
`m`	Makes `^` and `$` match the start and end of each line instead of those of the entire string.	`multiline`
`s`	Allows `.` to match newline characters.	`dotAll`
`u`	"Unicode"; treat a pattern as a sequence of Unicode code points.	`unicode`
`v`	An upgrade to the `u` mode with more Unicode features.	`unicodeSets`
`y`	Perform a "sticky" search that matches starting at the current position in the target string.	`sticky`

The `i`, `m`, and `s` flags can be enabled or disabled for specific parts of a regex using the modifier syntax.

The sections below list all available regex syntaxes, grouped by their syntactic nature.

Assertions

Assertions are constructs that test whether the string meets a certain condition at the specified position, but not consume characters. Assertions cannot be quantified.

Buffer boundary assertion: `\A`, `\z`, `\Z`

> Asserts that the current position in the string is strictly at the start or end of the entire string (`\Z` also allows a trailing newline), regardless of the presence of the `m` flag.

Input boundary assertion: `^`, `$`

> Asserts that the current position is the start or end of input, or start or end of a line if the `m` flag is set.

Lookahead assertion: `(?=...)`, `(?!...)`

> Asserts that the current position is followed or not followed by a certain pattern.

Lookbehind assertion: `(?<=...)`, `(?<!...)`

> Asserts that the current position is preceded or not preceded by a certain pattern.

Word boundary assertion: `\b`, `\B`

> Asserts that the current position is a word boundary.

Atoms

Atoms are the most basic units of a regular expression. Each atom *consumes* one or more characters in the string, and either fails the match or allows the pattern to continue matching with the next atom.

Backreference: `\1`, `\2`

> Matches a previously matched subpattern captured with a capturing group.

Capturing group: `(...)`

> Matches a subpattern and remembers information about the match.

Character class: `[...]`, `[^...]`

> Matches any character in or not in a set of characters. When the `v` flag is enabled, it can also be used to match finite-length strings.

Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`

> Matches any character in or not in a predefined set of characters.

Character escape: `\n`, `\u{...}`

> Matches a character that may not be able to be conveniently represented in its literal form.

Literal character: `a`, `b`

> Matches a specific character.

Modifier: `(?ims-ims:...)`

> Overrides flag settings in a specific part of a regular expression.

Named backreference: `\k<name>`

> Matches a previously matched subpattern captured with a named capturing group.

Named capturing group: `(?<name>...)`

> Matches a subpattern and remembers information about the match. The group can later be identified by a custom name instead of by its index in the pattern.

Non-capturing group: `(?:...)`

> Matches a subpattern without remembering information about the match.

Unicode character class escape: `\p{...}`, `\P{...}`

> Matches a set of characters specified by a Unicode property. When the `v` flag is enabled, it can also be used to match finite-length strings.

Wildcard: `.`

> Matches any character except line terminators, unless the `s` flag is set.

Other features

These features do not specify any pattern themselves, but are used to compose patterns.

Disjunction: `|`

> Matches any of a set of alternatives separated by the `|` character.

Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

Matches an atom a certain number of times.

Escape sequences

Escape sequences in regexes refer to any kind of syntax formed by `\` followed by one or more characters. They may serve very different purposes depending on what follow `\`. Below is a list of all valid "escape sequences":

Escape sequence	Followed by	Meaning
`\A`	None	Buffer boundary assertion
`\B`	None	Non-word-boundary assertion
`\D`	None	Character class escape representing non-digit characters
`\P`	`{`, a Unicode property and/or value, then `}`	Unicode character class escape representing characters without the specified Unicode property
`\S`	None	Character class escape representing non-white-space characters
`\W`	None	Character class escape representing non-word characters
`\Z`	None	Buffer boundary assertion
`\b`	None	Word boundary assertion; inside character classes, represents U+0008 (BACKSPACE)
`\c`	A letter from `A` to `Z` or `a` to `z`	A character escape representing the control character with value equal to the letter's character value modulo 32
`\d`	None	Character class escape representing digit characters (`0` to `9`)
`\f`	None	Character escape representing U+000C (FORM FEED)
`\k`	`<`, an identifier, then `>`	A named backreference
`\n`	None	Character escape representing U+000A (LINE FEED)
`\p`	`{`, a Unicode property and/or value, then `}`	Unicode character class escape representing characters with the specified Unicode property
`\q`	`{`, a string, then a `}`	Only valid inside v-mode character classes; represents the string to be matched literally
`\r`	None	Character escape representing U+000D (CARRIAGE RETURN)
`\s`	None	Character class escape representing whitespace characters
`\t`	None	Character escape representing U+0009 (CHARACTER TABULATION)
`\u`	4 hexadecimal digits; or `{`, 1 to 6 hexadecimal digits, then `}`	Character escape representing the character with the given code point
`\v`	None	Character escape representing U+000B (LINE TABULATION)
`\w`	None	Character class escape representing word characters (`A` to `Z`, `a` to `z`, `0` to `9`, `_`)
`\x`	2 hexadecimal digits	Character escape representing the character with the given value
`\z`	None	Buffer boundary assertion
`\0`	None	Character escape representing U+0000 (NULL)

`\` followed by `0` and another digit becomes a legacy octal escape sequence, which is forbidden in Unicode-aware mode. `\` followed by any other digit sequence becomes a backreference.

In addition, `\` can be followed by some non-letter-or-digit characters, in which case the escape sequence is always a character escape representing the escaped character itself:

- `\$`, `\(`, `\)`, `\*`, `\+`, `\.`, `\/`, `\?`, `\[`, `\\`, `\]`, `\^`, `\{`, `\|`, `\}`: valid everywhere
- `\-`: only valid inside character classes
- `\!`, `\#`, `\%`, `\&`, `\,`, `\:`, `\;`, `\<`, `\=`, `\>`, `\@`, `` \` ``, `\~`: only valid inside v-mode character classes

The other ASCII characters, namely space character, `"`, `'`, `_`, and any letter character not mentioned above, are not valid escape sequences. In Unicode-unaware mode, escape sequences that are not one of the above become *identity escapes*: they represent the character that follows the backslash. For example, `\a` represents the character `a`. This behavior limits the ability to introduce new escape sequences without causing backward compatibility issues, and is therefore forbidden in Unicode-aware mode.

See also

- Regular expressions guide
- `RegExp`

Backreference: \1, \2

A **backreference** refers to the submatch of a previous capturing group and matches the same text as that group. For named capturing groups, you may prefer to use the named backreference syntax.

Syntax

```
\N
```

> **Note:** `N` is not a literal character.

Parameters

`N`

A positive integer referring to the number of a capturing group.

Description

A backreference is a way to match the same text as previously matched by a capturing group. Capturing groups count from 1, so the first capturing group's result can be referenced with `\1`, the second with `\2`, and so on. `\0` is a character escape for the NUL character.

In case-insensitive matching, the backreference may match text with different casing from the original text.

```
/(b)\1/i.test("bB"); // true
```

The backreference must refer to an existent capturing group. If the number it specifies is greater than the total number of capturing groups, a syntax error is thrown.

```
/(a)\2/u; // SyntaxError: Invalid regular expression: Invalid escape
```

In Unicode-unaware mode, invalid backreferences become a legacy octal escape sequence. This is a deprecated syntax for web compatibility, and you should not rely on it.

```
/(a)\2/.test("a\x02"); // true
```

If the referenced capturing group is unmatched (for example, because it belongs to an unmatched alternative in a disjunction), or the group hasn't matched yet (for example, because it lies to the right of the backreference), the backreference always succeeds (as if it matches the empty string).

```
/(?:a|(b))\1c/.test("ac"); // true
/\1(a)/.test("a"); // true
```

Examples

Pairing quotes

The following function matches the `title='xxx'` and `title="xxx"` patterns in a string. To ensure the quotes match, we use a backreference to refer to the first quote. Accessing the second capturing group (`[2]`) returns the string between the matching quote characters:

```
function parseTitle(metastring) {
  return metastring.match(/title=(["'])(.*?)\1/)[2];
}

parseTitle('title="foo"'); // 'foo'
parseTitle("title='foo' lang='en'"); // 'foo'
parseTitle('title="Named capturing groups\' advantages"'); // "Named capturing groups' advantages"
```

Matching duplicate words

The following function finds duplicate words in a string (which are usually typos). Note that it uses the `\w` character class escape, which only matches English letters but not any accented letters or other alphabets. If you want more generic matching, you may want to split the string by whitespace and iterate over the resulting array.

```
function findDuplicates(text) {
  return text.match(/\b(\w+)\s+\1\b/i)?.[1];
}

findDuplicates("foo foo bar"); // 'foo'
findDuplicates("foo bar foo"); // undefined
findDuplicates("Hello hello"); // 'Hello'
findDuplicates("Hello hellos"); // undefined
```

See also

- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Named capturing group: `(?<name>...)`
- Named backreference: `\k<name>`

Buffer boundary assertion: \A, \z, \Z

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

A **buffer boundary assertion** checks if the current position in the string is strictly at the start or end of the entire string (`\z` also allows a trailing newline), regardless of the presence of the `m` flag (which changes the meanings of the `^` and `$` input boundary assertions). It's only supported in Unicode-aware mode.

Syntax

```
\A
\z
\Z
```

Description

`\A` asserts that the current position is the start of the entire string. `\z` asserts that the current position is the end of the entire string. `\Z` is like `\z`, but also matches before a line terminator or `\r\n` (CRLF) sequence at the end of the string. All of them are *assertions*, so they don't consume any characters.

More precisely, `\A` asserts that the character to the left is out of bounds of the string; `\z` asserts that the character to the right is out of bounds of the string; `\Z` is equivalent to `(?=(?:\r?\n?|[\u{2028}\u{2029}]?)\z)`.

These assertions only make sense when no characters are expected to the left or right of them. For example, `f\Ao` never matches because it's not possible for `\A` to both be at the start of the string and have a character to its left.

The `\A` and `\z` assertions are only useful when the `m` flag is used. Without `m`, they behave just like `^` and `$`. If your pattern only needs to match the start or end of the entire string or a line (and never a mix of both), the recommended approach is to still use the `^` and `$` assertions, and set the `m` flag as needed, because they are more widely supported than the buffer boundary assertions. If you need to match both types of boundaries in the same pattern, you can technically use modifiers to enable or disable the `m` flag in different parts of the pattern, but using these escape sequences make the code much more readable.

Examples

Mixing buffer and input boundary assertions

Suppose you have a pattern that needs to match either at the start of the whole string or the start of one line. You can turn on `m` so that you can use `^` to refer to the latter, and then use `\A` to refer to the former (you also need to turn on `u` so you can use `\A`).

This example matches line comments on their own lines, which can be a hashbang at start of a file or a line comment on any line. It does not match line comments at the end of a line containing code.

```
function findLineComments(code) {
  // Matches the hashbang syntax: #!... (which is only valid at start of file)
  // and line comment syntax: //... (which is valid everywhere)
  const pattern = /\A#!.*|^\s*\/\/.*/gmu;
  return code.match(pattern);
}

const program = `#!/usr/env/node

function findLineComments(code) {
  // Matches the hashbang syntax: #!... (which is only valid at start of file)
  // and line comment syntax: //... (which is valid everywhere)
  const pattern = /\\A#!.*|^\\/\\/.*/gmu;
  return code.match(pattern);
}
`;

console.log(findLineComments(program));
// [
//   '#!/usr/env/node',
//   '  // Matches the hashbang syntax: #!... (which is only valid at start of file)',
//   '  // and line comment syntax: //... (which is valid everywhere)'
// ]
```

Another major use case of buffer boundary assertions is when you cannot modify the flags, such as regular expression search in text editors. These cases usually enable `m` by default, so you need to use `\A` and `\z` to "opt out".

Matching the end-of-file but allowing an optional trailing newline

File formats often allow an optional newline at the end of the file. If you want to match an end-of-file pattern that allows the trailing newline, you can use `\Z`. This feature is useful with or without the `m` flag.

```
const endOfPDF = /%%EOF\Z/u;

console.log(endOfPDF.test("%%EOF")); // true
console.log(endOfPDF.test("%%EOF\n")); // true
console.log(endOfPDF.test("%%EOF\r\n")); // true
console.log(endOfPDF.test("%%EOF\n\n")); // false
console.log(endOfPDF.test("%%EOF\nsomething")); // false
```

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Word boundary assertion: `\b`, `\B`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`

Capturing group: (...)

A **capturing group** groups a subpattern, allowing you to apply a quantifier to the entire group or use disjunctions within it. It memorizes information about the subpattern match, so that you can refer back to it later with a backreference, or access the information through the match results.

If you don't need the result of the subpattern match, use a non-capturing group instead, which improves performance and avoids refactoring hazards.

Syntax

```
(pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

A capturing group acts like the grouping operator in JavaScript expressions, allowing you to use a subpattern as a single atom.

Capturing groups are numbered by the order of their opening parentheses. The first capturing group is numbered `1`, the second `2`, and so on. Named capturing groups are also capturing groups and are numbered together with other (unnamed) capturing groups. The information of the capturing group's match can be accessed through:

- The return value (which is an array) of `RegExp.prototype.exec()`, `String.prototype.match()`, and `String.prototype.matchAll()`
- The `pN` parameters of the `String.prototype.replace()` and `String.prototype.replaceAll()` methods' `replacement` callback function
- Backreferences within the same pattern

> **Note:** Even in `exec()`'s result array, capturing groups are accessed by numbers `1`, `2`, etc., because the `0` element is the entire match. `\0` is not a backreference, but a character escape for the NUL character.

Capturing groups in the regex source code correspond to their results one-to-one. If a capturing group is not matched (for example, it belongs to an unmatched alternative in a disjunction), the corresponding result is `undefined`.

```
/(ab)|(cd)/.exec("cd"); // ['cd', undefined, 'cd']
```

Capturing groups can be quantified. In this case, the match information corresponding to this group is the last match of the group.

```
/([ab])+/.exec("abc"); // ['ab', 'b']; because "b" comes after "a", this result overwrites the previous one
```

Capturing groups can be used in lookahead and lookbehind assertions. Because lookbehind assertions match their atoms backwards, the final match corresponding to this group is the one that appears to the *left* end of the string. However, the indices of the match groups still correspond to their relative locations in the regex source.

```
/c(?=(ab))/.exec("cab"); // ['c', 'ab']
/(?<=(a)(b))c/.exec("abc"); // ['c', 'a', 'b']
/(?<=([ab])+)c/.exec("abc"); // ['c', 'a']; because "a" is seen by the lookbehind after the lookbehind has seen "b"
```

Capturing groups can be nested, in which case the outer group is numbered first, then the inner group, because they are ordered by their opening parentheses. If a nested group is repeated by a quantifier, then each time the group matches, the subgroups' results are all overwritten, sometimes with `undefined`.

```
/((a+)?(b+)?(c))*/.exec("aacbbbcac"); // ['aacbbbcac', 'ac', 'a', undefined, 'c']
```

In the example above, the outer group is matched three times:

1. Matches `"aac"`, with subgroups `"aa"`, `undefined`, and `"c"`.
2. Matches `"bbbc"`, with subgroups `undefined`, `"bbb"`, and `"c"`.
3. Matches `"ac"`, with subgroups `"a"`, `undefined`, and `"c"`.

The `"bbb"` result from the second match is not preserved, because the third match overwrites it with `undefined`.

You can get the start and end indices of each capturing group in the input string by using the `d` flag. This creates an extra `indices` property on the array returned by `exec()`.

You can optionally specify a name to a capturing group, which helps avoid pitfalls related to group positions and indexing. See Named capturing groups for more information.

Parentheses have other purposes in different regex syntaxes. For example, they also enclose lookahead and lookbehind assertions. Because these syntaxes all start with `?`, and `?` is a quantifier which normally cannot occur directly after `(`, this does not lead to ambiguities.

Examples

Matching date

The following example matches a date in the format `YYYY-MM-DD`:

```
function parseDate(input) {
  const parts = /^(\d{4})-(\d{2})-(\d{2})$/.exec(input);
  if (!parts) {
    return null;
  }
```

```
  return parts.slice(1).map((p) => parseInt(p, 10));
}

parseDate("2019-01-01"); // [2019, 1, 1]
parseDate("2019-06-19"); // [2019, 6, 19]
```

Pairing quotes

The following function matches the `title='xxx'` and `title="xxx"` patterns in a string. To ensure the quotes match, we use a backreference to refer to the first quote. Accessing the second capturing group (`[2]`) returns the string between the matching quote characters:

```
function parseTitle(metastring) {
  return metastring.match(/title=(["'])(.*?)\1/)[2];
}

parseTitle('title="foo"'); // 'foo'
parseTitle("title='foo' lang='en'"); // 'foo'
parseTitle('title="Named capturing groups\' advantages"'); // "Named capturing groups' advantages"
```

See also

- Groups and backreferences guide
- Regular expressions
- Non-capturing group: `(?:...)`
- Named capturing group: `(?<name>...)`
- Backreference: `\1`, `\2`

Character class: [...], [^...]

A **character class** matches any character in or not in a custom set of characters. When the `v` flag is enabled, it can also be used to match finite-length strings.

Syntax

```
[]
[abc]
[A-Z]

[^]
[^abc]
[^A-Z]

// `v` mode only
[operand1&&operand2]
[operand1--operand2]
[\q{substring}]
```

Parameters

`operand1`, `operand2`

> Can be a single character, another square-bracket-enclosed character class, a character class escape, a Unicode character class escape, or a string using the `\q` syntax.

`substring`

> A literal string.

Description

A character class specifies a list of characters between square brackets and matches any character in the list. The `v` flag drastically changes how character classes are parsed and interpreted. The following syntaxes are available in both `v` mode and non-`v` mode:

- A single character: matches the character itself.
- A range of characters: matches any character in the inclusive range. The range is specified by two characters separated by a dash (`-`). The first character must be smaller in character value than the second character. The *character value* is the Unicode code point of the character. Because Unicode code points are usually assigned to alphabets in order, `[a-z]` specifies all lowercase Latin characters, while `[α-ω]` specifies all lowercase Greek characters. In Unicode-unaware mode, regexes are interpreted as a sequence of BMP characters. Therefore, surrogate pairs in character classes represent two characters instead of one; see below for details.
- Escape sequences: `\b`, `\-`, character class escapes, Unicode character class escapes, and other character escapes.

These syntaxes can occur any number of times, and the character sets they represent are unioned. For example, `/[a-zA-Z0-9]/` matches any letter or digit.

The `^` prefix in a character class creates a *complement class*. For example, `[^abc]` matches any character except `a`, `b`, or `c`. The `^` character is a literal character when it appears in the middle of a character class — for example, `[a^b]` matches the characters `a`, `^`, and `b`.

The lexical grammar does a very rough parse of regex literals, so that it does not end the regex literal at a `/` character which appears within a character class. This means `/[/]/` is valid without needing to escape the `/`.

The boundaries of a character range must not specify more than one character, which happens if you use a character class escape. For example:

```
/[\s-9]/u; // SyntaxError: Invalid regular expression: Invalid character class
```

In Unicode-unaware mode, character ranges where one boundary is a character class makes the `-` become a literal character. This is a deprecated syntax for web compatibility, and you should not rely on it.

```
/[\s-9]/.test("-"); // true
```

In Unicode-unaware mode, regexes are interpreted as a sequence of BMP characters. Therefore, surrogate pairs in character classes represent two characters instead of one.

```
/[😄]/.test("\ud83d"); // true
/[😄]/u.test("\ud83d"); // false

/[😄-😆]/.test("😅"); // SyntaxError: Invalid regular expression: /[😄-😆]/: Range out of order in character class
/[😄-😆]/u.test("😅"); // true
```

Even if the pattern ignores case, the case of the two ends of a range is significant in determining which characters belong to the range. For example, the pattern `/[E-F]/i` only matches `E`, `F`, `e`, and `f`, while the pattern `/[E-f]/i` matches all uppercase and lowercase ASCII letters (because it spans over `E`–`Z` and `a`–`f`), as well as `[`, `\`, `]`, `^`, `_`, and `` ` ``.

Non-v-mode character class

Non-`v`-mode character classes interpret most character literally and have less restrictions about the characters they can contain. For example, `.` is the literal dot character, not the wildcard. The only characters that cannot appear literally are `\`, `]`, and `-`.

- In character classes, most escape sequences are supported, except `\b`, `\B`, and backreferences. `\b` indicates a backspace character instead of a word boundary, while the other two cause syntax errors. To use `\` literally, escape it as `\\`.
- The `]` character indicates the end of the character class. To use it literally, escape it as `\]`.
- The dash (`-`) character, when used between two characters, indicates a range. When it appears at the start or end of a character class, it is a literal character. It's also a literal character when it's used in the boundary of a range. For example, `[a-]` matches the characters `a` and `-`, `[!--]` matches the characters `!` to `-`, and `[--9]` matches the characters `-` to `9`. You can also escape it as `\-` if you want to use it literally anywhere.

v-mode character class

The basic idea of character classes in `v` mode remains the same: you can still use most characters literally, use `-` to denote character ranges, and use escape sequences. One of the most important features of the `v` flag is *set notation* within character classes. As previously mentioned, normal character classes can express unions by concatenating two ranges, such as using `[A-Z0-9]` to mean "the union of the set `[A-Z]` and the set `[0-9]`". However, there's no easy way to represent other operations with character sets, such as intersection and difference.

With the `v` flag, intersection is expressed with `&&`, and subtraction with `--`. The absence of both implies union. The two operands of `&&` or `--` can be a character, character escape, character class escape, or even another character class. For example, to express "a word character that's not an underscore", you can use `[\w--_]`. You cannot mix operators on the same level. For example, `[\w&&[A-z]--_]` is a syntax error. However, because you can nest character classes, you can be explicit by writing `[\w&&[[A-z]--_]]` or `[[\w&&[A-z]]--_]` (which both mean `[A-Za-z]`). Similarly, `[AB--C]` is invalid and you need to write `[A[B--C]]` (which just means `[AB]`).

In `v` mode, the Unicode character class escape `\p` can match finite-length strings, such as emojis. For symmetry, regular character classes can also match more than one character. To write a "string literal" in a character class, you wrap the string in `\q{...}`. The only regex syntax supported here is disjunction — apart from this, `\q` must completely enclose literals (including escaped characters). This ensures that character classes can only match finite-length strings with finitely many possibilities.

Unlike regular disjunction, the order of alternatives within `\q` does not matter, neither does the order of the `\q` escape with the rest of the operands being unioned. During matching, all alternatives specified in a character class are always attempted in descending order of length, so the match is greedy. For example, `[\q{a|ab}]`, `[\q{ab|a}]`, `[\q{ab}a]`, and `[a\q{ab}]` applied to `"ab"` will all match `"ab"`.

Complement character classes `[^...]` cannot possibly be able to match strings longer than one character. For example, `[\q{ab|c}]` is valid and matches the string `"ab"`, but `[^\q{ab|c}]` is invalid because it's unclear how many characters should be consumed. The check is done by checking if all `\q` contain single characters and all `\p` specify character properties — for unions, all operands must be purely characters; for intersections, at least one operand must be purely characters; for subtraction, the leftmost operand must be purely characters. The check is syntactic without looking at the actual character set being specified, which means although `/[^\q{ab|c}--\q{ab}]/v` is equivalent to `/[^c]/v`, it's still rejected.

Because the character class syntax is now more sophisticated, more characters are reserved and forbidden from appearing literally.

- In addition to `]` and `\`, the following characters must be escaped in character classes if they represent literal characters: `(`, `)`, `[`, `{`, `}`, `/`, `-`, `|`. This list is somewhat similar to the list of syntax characters, except that `^`, `$`, `*`, `+`, and `?` are not reserved inside character classes, while `/` and `-` are not reserved outside character classes (although `/` may delimit a regex literal and therefore still needs to be escaped). All these characters may also be optionally escaped in `u`-mode character classes.
- The following "double punctuator" sequences must be escaped as well (but they don't make much sense without the `v` flag anyway): `&&`, `!!`, `##`, `$$`, `%%`, `**`, `++`, `,,`, `..`, `::`, `;;`, `<<`, `==`, `>>`, `??`, `@@`, `^^`, `` `` ``, `~~`. In `u` mode, some of these characters can only appear literally within character classes and cause a syntax error when escaped. In `v` mode, they must be escaped when appearing in pairs, but can be optionally escaped when appearing alone. For example, `/[\!]/u` is invalid because it's an identity escape, but both `/[\!]/v` and `/[!]/v` are valid, while `/[!!]/v` is invalid. The literal character reference has a detailed table of which characters can appear escaped or unescaped.

Complement classes and case-insensitive matching

Case-insensitive matching works by case-folding both the expected character set and the matched string. When specifying complement classes, the order in which JavaScript performs case-folding and complementing is important. In brief, `[^...]` in `u` mode matches `allCharacters - caseFold(original)`, while in `v` mode matches `caseFold(allCharacters) - caseFold(original)`. This ensures that all complement class syntaxes, including `[^...]`, `\P`, `\W`, etc., cancel each other out.

Consider the following two regexes (to simplify things, let's assume that Unicode characters are one of three kinds: lowercase, uppercase, and caseless, and each uppercase letter has a unique lowercase counterpart, and vice versa):

```
const r1 = /\p{Lowercase_Letter}/iu;
const r2 = /[^\P{Lowercase_Letter}]/iu;
```

The `r2` is a double negation and seems to be equivalent with `r1`. But in fact, `r1` matches all lower- and uppercase ASCII letters, while `r2` matches none. Here's a step-by-step explanation:

- In `r1`, `\p{Lowercase_Letter}` constructs a set of all lowercase characters. Characters in this set are then case-folded to their lowercase form, so they stay the same. The input string is also case-folded to lowercase. Therefore, `"A"` and `"a"` are both folded to `"a"` and matched by `r1`.
- In `r2`, `\P{Lowercase_Letter}` first constructs a set of all non-lowercase characters, i.e., uppercase letters and caseless characters. Characters in this set are then case-folded to their lowercase form, so the character set becomes all lowercase letters and caseless characters. `[^...]` negates the match, causing it to match anything that's *not* in this set, i.e., an uppercase letter. However, the input is still case-folded to lowercase, so `"A"` is folded to `"a"` and not matched by `r2`.

The main observation here is that after `[^...]` negates the match, the expected character set may not be a subset of the set of case-folded Unicode characters, causing the case-folded input to not be in the expected character set. In `v` mode, the set of all characters is also case-folded. The `\P` character class itself also works slightly differently in `v` mode (see Unicode character class escape). All of these ensure that `[^\P{Lowercase_Letter}]` and `\p{Lowercase_Letter}` are strictly equivalent.

Examples

Matching hexadecimal digits

The following function determines whether a string contains a valid hexadecimal number:

```
function isHexadecimal(str) {
  return /^[0-9A-F]+$/i.test(str);
}

isHexadecimal("2F3"); // true
isHexadecimal("beef"); // true
isHexadecimal("undefined"); // false
```

Using intersection

The following function matches Greek letters.

```
function greekLetters(str) {
  return str.match(/[\p{Script_Extensions=Greek}&&\p{Letter}]/gv);
}

// 𐆊 is U+1018A GREEK ZERO SIGN
greekLetters("π𐆊P0零αAΣ"); // [ 'π', 'α', 'Σ' ]
```

Using subtraction

The following function matches all non-ASCII numbers.

```
function nonASCIINumbers(str) {
  return str.match(/[\p{Decimal_Number}--\d]/gv);
}

// 𑜹 is U+11739 AHOM DIGIT NINE
nonASCIINumbers("𐆊0零14𑜹a"); // [ '4', '𑜹' ]
```

Matching strings

The following function matches all line terminator sequences, including the line terminator characters and the sequence `\r\n` (CRLF). The order in which the alternatives are specified does not matter, because the longest alternative, `\r\n`, is always preferred.

```
function getLineTerminatorSequences(str) {
  return str.match(/[\r\n\u2028\u2029\q{\r\n}]/gv);
}

getLineTerminatorSequences(`
A poem\r
Is split\rInto many
Stanzas
`); // ['\n', '\r\n', '\r', '\n', '\n']
```

This regular expression is exactly equivalent to `/(?:\r\n|\r|\n|\u2028|\u2029)/gu` or `/(?:\r\n|[\r\n\u2028\u2029])/gu`. However, in regular disjunction, the order of alternatives matters, so `\r\n` must be specified first in order for it to be preferred. Using a character class not only is shorter but also avoids this pitfall.

The most useful case of `\q{}` is when doing subtraction and intersection. Previously, this was possible with multiple lookaheads. The following function matches flags that are not one of the American, Chinese, Russian, British, and French flags.

```
function notUNSCPermanentMember(flag) {
  return /^[\p{RGI_Emoji_Flag_Sequence}--\q{🇺🇸|🇨🇳|🇷🇺|🇬🇧|🇫🇷}]$/v.test(flag);
}

notUNSCPermanentMember("🇺🇸"); // false
notUNSCPermanentMember("🇩🇪"); // true
```

This example is mostly equivalent to `/^(?!🇺🇸|🇨🇳|🇷🇺|🇬🇧|🇫🇷)\p{RGI_Emoji_Flag_Sequence}$/v`, except perhaps more performant.

See also

- Character classes guide
- Regular expressions
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Literal character: `a`, `b`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`
- RegExp v flag with set notation and properties of strings on v8.dev (2022)

Character class escape: \d, \D, \w, \W, \s, \S

A **character class escape** is an escape sequence that represents a set of characters.

Syntax

```
\d, \D
\s, \S
\w, \W
```

> **Note:** , is not part of the syntax.

Description

Unlike character escapes, character class escapes represent a predefined *set* of characters, much like a character class. The following character classes are supported:

`\d`

: Matches any digit character. Equivalent to `[0-9]`.

`\w`

: Matches any word character, where a word character includes letters (A–Z, a–z), numbers (0–9), and underscore (_). If the regex is Unicode-aware and the `i` flag is set, it also matches other Unicode characters that get canonicalized to one of the characters above through case folding.

`\s`

: Matches any whitespace or line terminator character.

The uppercase forms `\D`, `\W`, and `\S` create complement character classes for `\d`, `\w`, and `\s`, respectively. They match any character that is not in the set of characters matched by the lowercase form.

Unicode character class escapes start with `\p` and `\P`, but they are only supported in Unicode-aware mode. In Unicode-unaware mode, they are identity escapes for the `p` or `P` character.

Character class escapes can be used in character classes. However, they cannot be used as boundaries of character ranges, which is only allowed as a deprecated syntax for web compatibility, and you should not rely on it.

Examples

Splitting by whitespace

The following example splits a string into an array of words, supporting all kinds of whitespace separators:

```
function splitWords(str) {
  return str.split(/\s+/);
}

splitWords(`Look at the stars
Look  how they\tshine for you`);
// ['Look', 'at', 'the', 'stars', 'Look', 'how', 'they', 'shine', 'for', 'you']
```

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`

Character escape: \n, \u{...}

A **character escape** represents a character that may not be able to be conveniently represented in its literal form.

Syntax

```
\f, \n, \r, \t, \v
\cA, \cB, …, \cz
\0
\^, \$, \\, \., \*, \+, \?, \(, \), \[, \], \{, \}, \|, \/
```

```
\xHH
\uHHHH
\u{H…H}
```

> **Note:** , is not part of the syntax.

Parameters

`H…H`

A hexadecimal number representing the Unicode code point of the character. The `\xHH` form must have two hexadecimal digits; the `\uHHHH` form must have four; the `\u{H…H}` form may have 1 to 6 hexadecimal digits.

Description

The following character escapes are recognized in regular expressions:

`\f`, `\n`, `\r`, `\t`, `\v`

Same as those in string literals, except `\b`, which represents a word boundary in regexes unless in a character class.

`\c` followed by a letter from `A` to `Z` or `a` to `z`

Represents the control character with value equal to the letter's character value modulo 32. For example, `\cJ` represents line break (`\n`), because the code point of `J` is 74, and 74 modulo 32 is 10, which is the code point of line break. Because an uppercase letter and its lowercase form differ by 32, `\cJ` and `\cj` are equivalent. You can represent control characters from 1 to 26 in this form.

`\0`

Represents the U+0000 NUL character. Cannot be followed by a digit (which makes it a legacy octal escape sequence).

`\^`, `\$`, `\\`, `\.`, `\*`, `\+`, `\?`, `\(`, `\)`, `\[`, `\]`, `\{`, `\}`, `\|`, `\/`

Represents the character itself. For example, `\\` represents a backslash, and `\(` represents a left parenthesis. These are syntax characters in regexes (`/` is the delimiter of a regex literal), so they require escaping unless in a character class.

`\xHH`

Represents the character with the given hexadecimal Unicode code point. The hexadecimal number must be exactly two digits long.

`\uHHHH`

Represents the character with the given hexadecimal Unicode code point. The hexadecimal number must be exactly four digits long. Two such escape sequences can be used to represent a surrogate pair in Unicode-aware mode. (In Unicode-unaware mode, they are always two separate characters.)

`\u{H…H}`

(Unicode-aware mode only) Represents the character with the given hexadecimal Unicode code point. The hexadecimal number can be from 1 to 6 digits long.

In Unicode-unaware mode, escape sequences that are not one of the above become *identity escapes*: they represent the character that follows the backslash. For example, `\a` represents the character `a`. This behavior limits the ability to introduce new escape sequences without causing backward compatibility issues, and is therefore forbidden in Unicode-aware mode.

In Unicode-unaware mode, `]`, `{`, and `}` may appear literally if it's not possible to parse them as the end of a character class or quantifier delimiters. This is a deprecated syntax for web compatibility, and you should not rely on it.

In Unicode-unaware mode, escape sequences within character classes of the form `\cX` where `X` is a number or `_` are decoded in the same way as those with ASCII letters: `\c0` is the same as `\cP` when taken modulo 32. In addition, if the form `\cX` is encountered anywhere where `X` is not one of the recognized characters, then the backslash is treated as a literal character. These syntaxes are also deprecated.

```
/[\c0]/.test("\x10"); // true
/[\c_]/.test("\x1f"); // true
/[\c*]/.test("\\"); // true
/\c/.test("\\c"); // true
/\c0/.test("\\c0"); // true (the \c0 syntax is only supported in character classes)
```

Examples

Using character escapes

Character escapes are useful when you want to match a character that is not easily represented in its literal form. For example, you cannot use a line break literally in a regex literal, so you must use a character escape:

```
const pattern = /a\nb/;
const string = `a
b`;
console.log(pattern.test(string)); // true
```

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Literal character: `a`, `b`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Backreference: `\1`, `\2`
- Named backreference: `\k<name>`
- Word boundary assertion: `\b`, `\B`

Disjunction: |

A **disjunction** specifies multiple alternatives. Any alternative matching the input causes the entire disjunction to be matched.

Syntax

```
alternative1|alternative2
alternative1|alternative2|alternative3|…
```

Parameters

`alternativeN`

One alternative pattern, composed of a sequence of atoms and assertions. Successfully matching one alternative causes the entire disjunction to be matched.

Description

The `|` regular expression operator separates two or more *alternatives*. The pattern first tries to match the first alternative; if it fails, it tries to match the second one, and so on. For example, the following matches `"a"` instead of `"ab"`, because the first alternative already matches successfully:

```
/a|ab/.exec("abc"); // ['a']
```

The `|` operator has the lowest precedence in a regular expression. If you want to use a disjunction as a part of a bigger pattern, you must group it.

When a grouped disjunction has more expressions after it, the matching begins by selecting the first alternative and attempting to match the rest of the regular expression. If the rest of the regular expression fails to match, the matcher tries the next alternative instead. For example,

```
/(?:(a)|(ab))(?:(c)|(bc))/.exec("abc"); // ['abc', 'a', undefined, undefined, 'bc']
// Not ['abc', undefined, 'ab', 'c', undefined]
```

This is because by selecting `a` in the first alternative, it's possible to select `bc` in the second alternative and result in a successful match. This process is called *backtracking*, because the matcher first goes beyond the disjunction and then comes back to it when subsequent matching fails.

Note also that any capturing parentheses inside an alternative that's not matched produce `undefined` in the resulting array.

An alternative can be empty, in which case it matches the empty string (in other words, always matches).

Alternatives are always attempted left-to-right, regardless of the direction of matching (which is reversed in a lookbehind).

Examples

Matching file extensions

The following example matches file extensions, using the same code as the input boundary assertion article:

```
function isImage(filename) {
  return /\.(?:png|jpe?g|webp|avif|gif)$/i.test(filename);
}

isImage("image.png"); // true
isImage("image.jpg"); // true
isImage("image.pdf"); // false
```

See also

- Character classes guide
- Regular expressions
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`
- Character class: `[...]`, `[^...]`

Input boundary assertion: ^, $

An **input boundary assertion** checks if the current position in the string is an input boundary. An input boundary is the start or end of the string; or, if the `m` flag is set, the start or end of a line.

> **Note:** To match the start and end of the entire string in `m` mode, use the buffer boundary assertions `\A`, `\z`, and `\Z`.

Syntax

```
^
$
```

Description

`^` asserts that the current position is the start of input. `$` asserts that the current position is the end of input. Both are *assertions*, so they don't consume any characters.

More precisely, `^` asserts that the character to the left is out of bounds of the string; `$` asserts that the character to the right is out of bounds of the string. If the `m` flag is set, `^` also matches if the character to the left is a line terminator character, and `$` also matches if the character to the right is a line terminator.

Unless the `m` flag is set, these assertions only make sense when no characters are expected to the left or right of them. For example, `f^o` never matches because it's not possible for `^` to both be at the start of the string and have a character to its left.

These assertions don't change the match locations when the `y` flag is used — see also anchored sticky flag.

Examples

Removing trailing slashes

The following example removes trailing slashes from a URL string:

```
function removeTrailingSlash(url) {
  return url.replace(/\/$/, "");
}

removeTrailingSlash("https://example.com/"); // "https://example.com"
removeTrailingSlash("https://example.com/docs/"); // "https://example.com/docs"
```

Matching file extensions

The following example checks file types by matching the file extension, which always comes at the end of the string:

```
function isImage(filename) {
  return /\.(?:png|jpe?g|webp|avif|gif)$/i.test(filename);
}

isImage("image.png"); // true
isImage("image.jpg"); // true
isImage("image.pdf"); // false
```

Matching entire input

Sometimes you want to make sure that your regex matches the entire input, not just a substring of the input. For example, if you are determining if a string is a valid identifier, you can add input boundary assertions to both ends of the pattern:

```
function isValidIdentifier(str) {
  return /^[$_\p{ID_Start}][$_\p{ID_Continue}]*$/u.test(str);
}

isValidIdentifier("foo"); // true
isValidIdentifier("$1"); // true
isValidIdentifier("1foo"); // false
isValidIdentifier("  foo  "); // false
```

This function is useful when doing codegen (generating code using code), because you can use valid identifiers differently from other string properties, such as dot notation instead of bracket notation:

```
const variables = ["foo", "foo:bar", "  foo  "];

function toAssignment(key) {
  if (isValidIdentifier(key)) {
    return `globalThis.${key} = undefined;`;
  }
  // JSON.stringify() escapes quotes and other special characters
  return `globalThis[${JSON.stringify(key)}] = undefined;`;
}

const statements = variables.map(toAssignment).join("\n");

console.log(statements);
// globalThis.foo = undefined;
// globalThis["foo:bar"] = undefined;
// globalThis["  foo  "] = undefined;
```

See also

- Assertions guide
- Regular expressions
- Buffer boundary assertion: `\A`, `\z`, `\Z`
- Word boundary assertion: `\b`, `\B`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`

Literal character: a, b

A **literal character** specifies exactly itself to be matched in the input text.

Syntax

```
c
```

Parameters

`c`

A single character that is not one of the syntax characters described below.

Description

In regular expressions, most characters can appear literally. They are usually the most basic building blocks of patterns. For example, here is a pattern from the Removing HTML tags example:

```
const pattern = /<.+?>/g;
```

In this example, `.`, `+`, and `?` are called *syntax characters*. They have special meanings in regular expressions. The rest of the characters in the pattern (`<` and `>`) are literal characters. They match themselves in the input text: the left and right angle brackets.

The following characters are syntax characters in regular expressions, and they cannot appear as literal characters:

- `^`, `$`
- `\`
- `*`, `+`, `?`, `{`, `}`
- `(`, `)`
- `[`, `]`
- `|`

Within character classes, more characters can appear literally. For more information, see the Character class page. For example `\.` and `[.]` both match a literal `.`. In v-mode character classes, however, there are a different set of characters reserved as syntax characters. To be most comprehensive, below is a table of ASCII characters and whether they may appear escaped or unescaped in different contexts, where "✅" means the character represents itself, "❌" means it throws a syntax error, and "⚠️" means the character is valid but means something other than itself.

Characters	Outside character classes in `u` or `v` mode		In `u`-mode character classes		In `v`-mode character classes	
	Unescaped	Escaped	Unescaped	Escaped	Unescaped	Escaped
`123456789 "' ACEFGHIJKLMN OPQRTUVXYZ_ aceghijklmop quxyz`	✅	❌	✅	❌	✅	❌
`!#%&,:;<=>@`~`	✅	❌	✅	❌	✅	✅
`]`	❌	✅	❌	✅	❌	✅
`(){}`	❌	✅	✅	✅	❌	✅
`*+?`	❌	✅	✅	✅	✅	✅
`/`	✅	✅	✅	✅	❌	✅
`0DSWbdfnrstvw`	✅	⚠️	✅	⚠️	✅	⚠️
`B`	✅	⚠️	✅	❌	✅	❌
`$.`	⚠️	✅	✅	✅	✅	✅
`|`	⚠️	✅	✅	✅	❌	✅
`-`	✅	❌	✅⚠️	✅	❌⚠️	✅
`^`	⚠️	✅	✅⚠️	✅	✅⚠️	✅
`\`	❌⚠️	✅	❌⚠️	✅	❌⚠️	✅

> **Note:** The characters that can both be escaped and unescaped in `v`-mode character classes are exactly those forbidden as "double punctuators". See v-mode character classes for more information.

Whenever you want to match a syntax character literally, you need to escape it with a backslash (`\`). For example, to match a literal `*` in a pattern, you need to write `\*` in the pattern. Using syntax characters as literal characters either leads to unexpected results or causes syntax errors — for example, `/*/` is not a valid regular expression because the quantifier is not preceded by a pattern. In Unicode-unaware mode, `]`, `{`, and `}` may appear literally if it's not possible to parse them as the end of a character class or quantifier delimiters. This is a deprecated syntax for web compatibility, and you should not rely on it.

Regular expression literals cannot be specified with certain non-syntax literal characters. `/` cannot appear as a literal character in a regular expression literal, because `/` is used as the delimiter of the literal itself. You need to escape it as `\/` if you want to match a literal `/`. Line terminators cannot appear as literal characters in a regular expression literal either, because a literal cannot span multiple lines. You need to use a character escape like `\n` instead. There are no such restrictions when using the `RegExp()` constructor, although string literals have their own escaping rules (for example, `"\\"` actually denotes a single backslash character, so `new RegExp("\\*")` and `/\*/` are equivalent).

In Unicode-unaware mode, the pattern is interpreted as a sequence of UTF-16 code units. This means surrogate pairs actually represent two literal characters. This causes unexpected behaviors when paired with other features:

```
/^[😄]$/.test("😄"); // false, because the pattern is interpreted as /^[\ud83d\udc04]$/
/^😄+$/.test("😄😄"); // false, because the pattern is interpreted as /^\ud83d\udc04+$/
```

In Unicode-aware mode, the pattern is interpreted as a sequence of Unicode code points, and surrogate pairs do not get split. Therefore, you should always prefer to use the `u` flag.

Examples

Using literal characters

The following example is copied from Character escape. The `a` and `b` characters are literal characters in the pattern, and `\n` is an escaped character because it cannot appear literally in a regular expression literal.

```
const pattern = /a\nb/;
const string = `a
b`;
console.log(pattern.test(string)); // true
```

See also

- Character classes guide
- Regular expressions
- Character escape: `\n`, `\u{...}`

Lookahead assertion: (?=...), (?!...)

A **lookahead assertion** "looks ahead": it attempts to match the subsequent input with the given pattern, but it does not consume any of the input — if the match is successful, the current position in the input stays the same.

Syntax

```
(?=pattern)
(?!pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

A regular expression generally matches from left to right. This is why lookahead and lookbehind assertions are called as such — lookahead asserts what's on the right, and lookbehind asserts what's on the left.

In order for a `(?=pattern)` assertion to succeed, the `pattern` must match the text after the current position, but the current position is not changed. The `(?!pattern)` form negates the assertion — it succeeds if the `pattern` does not match at the current position.

The `pattern` can contain capturing groups. See the capturing groups page for more information on the behavior in this case.

Unlike other regular expression operators, there's no backtracking into a lookahead — this behavior is inherited from Perl. This only matters when the `pattern` contains capturing groups and the pattern following the lookahead contains backreferences to those captures. For example:

```
/(?=(a+))a*b\1/.exec("baabac"); // ['aba', 'a']
// Not ['aaba', 'a']
```

The matching of the pattern above happens as follows:

1. The lookahead `(a+)` succeeds before the first `"a"` in `"baabac"`, and `"aa"` is captured because the quantifier is greedy.
2. `a*b` matches the `"aab"` in `"baabac"` because lookaheads don't consume their matched strings.
3. `\1` does not match the following string, because that requires 2 `"a"`s, but only 1 is available. So the matcher backtracks, but it doesn't go into the lookahead, so the capturing group cannot be reduced to 1 `"a"`, and the entire match fails at this point.
4. `exec()` re-attempts matching at the next position — before the second `"a"`. This time, the lookahead matches `"a"`, and `a*b` matches `"ab"`. The backreference `\1` matches the captured `"a"`, and the match succeeds.

If the regex is able to backtrack into the lookahead and revise the choice made in there, then the match would succeed at step 3 by `(a+)` matching the first `"a"` (instead of the first two `"a"`s) and `a*b` matching `"aab"`, without even re-attempting the next input position.

Negative lookaheads can contain capturing groups as well, but backreferences only make sense within the `pattern`, because if matching continues, `pattern` would necessarily be unmatched (otherwise the assertion fails). This means outside of the `pattern`, backreferences to those capturing groups in negative lookaheads always succeed. For example:

```
/(.*?)a(?!(a+)b\2c)\2(.*)/.exec("baaabaac"); // ['baaabaac', 'ba', undefined, 'abaac']
```

The matching of the pattern above happens as follows:

1. The `(.*?)` pattern is non-greedy, so it starts by matching nothing. However, the next character is `a`, which fails to match `"b"` in the input.

2. The `(.*?)` pattern matches `"b"` so that the `a` in the pattern matches the first `"a"` in `"baaabaac"`.
3. At this position, the lookahead succeeds to match, because if `(a+)` matches `"aa"`, then `(a+)b\2c` matches `"aabaac"`. This causes the assertion to fail, so the matcher backtracks.
4. The `(.*?)` pattern matches the `"ba"` so that the `a` in the pattern matches the second `"a"` in `"baaabaac"`.
5. At this position, the lookahead fails to match, because the remaining input does not follow the pattern "any number of `"a"`s, a `"b"`, the same number of `"a"`s, a `c`". This causes the assertion to succeed.
6. However, because nothing was matched within the assertion, the `\2` backreference has no value, so it matches the empty string. This causes the rest of the input to be consumed by the `(.*)` at the end.

Normally, assertions cannot be quantified. However, in Unicode-unaware mode, lookahead assertions can be quantified. This is a deprecated syntax for web compatibility, and you should not rely on it.

```
/(?=a)?b/.test("b"); // true; the lookahead is matched 0 time
```

Examples

Matching strings without consuming them

Sometimes it's useful to validate that the matched string is followed by something without returning that as the result. The following example matches a string that is followed by a comma/period, but the punctuation is not included in the result:

```
function getFirstSubsentence(str) {
  return /^.*?(?=[,.])/.exec(str)?.[0];
}

getFirstSubsentence("Hello, world!"); // "Hello"
getFirstSubsentence("Thank you."); // "Thank you"
```

A similar effect can be achieved by capturing the submatch you are interested in.

Pattern subtraction and intersection

Using lookahead, you can match a string multiple times with different patterns, which allows you to express complex relationships like subtraction (is X but not Y) and intersection (is both X and Y).

The following example matches any identifier that's not a reserved word (only showing three reserved words here for brevity; more reserved words can be added to this disjunction). The `[$_\p{ID_Start}][$\p{ID_Continue}]*` syntax describes exactly the set of identifier strings in the language spec; you can read more about identifiers in lexical grammar and the `\p` escape in Unicode character class escape.

```
function isValidIdentifierName(str) {
  const re = /^(?!(?:break|case|catch)$)[$_\p{ID_Start}][$\p{ID_Continue}]*$/u;
  return re.test(str);
}

isValidIdentifierName("break"); // false
isValidIdentifierName("foo"); // true
isValidIdentifierName("cases"); // true
```

The following example matches a string that's both ASCII and can be used as an identifier part:

```
function isASCIIIDPart(char) {
  return /^(?=\p{ASCII}$)\p{ID_Start}$/u.test(char);
}

isASCIIIDPart("a"); // true
isASCIIIDPart("α"); // false
isASCIIIDPart(":"); // false
```

If you are doing intersection and subtraction with finitely many characters, you may want to use the character set intersection syntax enabled with the `v` flag.

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Word boundary assertion: `\b`, `\B`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Capturing group: `(...)`

Lookbehind assertion: (?<=...), (?<!...)

A **lookbehind assertion** "looks behind": it attempts to match the previous input with the given pattern, but it does not consume any of the input — if the match is successful, the current position in the input stays the same. It matches each atom in its pattern in the reverse order.

Syntax

```
(?<=pattern)
(?<!pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

A regular expression generally matches from left to right. This is why lookahead and lookbehind assertions are called as such — lookahead asserts what's on the right, and lookbehind asserts what's on the left.

In order for a `(?<=pattern)` assertion to succeed, the `pattern` must match the input immediately to the left of the current position, but the current position is not changed before matching the subsequent input. The `(?<!pattern)` form negates the assertion — it succeeds if the `pattern` does not match the input immediately to the left of the current position.

Lookbehind generally has the same semantics as lookahead — however, within a lookbehind assertion, the regular expression matches *backwards*. For example,

```
/(?<=([ab]+)([bc]+))$/.exec("abc"); // ['', 'a', 'bc']
// Not ['', 'ab', 'c']
```

If the lookbehind matches from left to right, it should first greedily match `[ab]+`, which makes the first group capture `"ab"`, and the remaining `"c"` is captured by `[bc]+`. However, because `[bc]+` is matched first, it greedily grabs `"bc"`, leaving only `"a"` for `[ab]+`.

This behavior is reasonable — the matcher does not know where to *start* the match (because the lookbehind may not be fixed-length), but it does know where to *end* (at the current position). Therefore, it starts from the current position and works backwards. (Regexes in some other languages forbid non-fixed-length lookbehind to avoid this issue.)

For quantified capturing groups inside the lookbehind, the match furthest to the left of the input string — instead of the one on the right — is captured because of backward matching. See the capturing groups page for more information. Backreferences inside the lookbehind must appear on the *left* of the group it's referring to, also due to backward matching. However, disjunctions are still attempted left-to-right.

Examples

Matching strings without consuming them

Similar to lookaheads, lookbehinds can be used to match strings without consuming them so that only useful information is extracted. For example, the following regex matches the number in a price label:

```
function getPrice(label) {
  return /(?<=\$)\d+(?:\.\d*)?/.exec(label)?.[0];
}

getPrice("$10.53"); // "10.53"
getPrice("10.53"); // undefined
```

A similar effect can be achieved by capturing the submatch you are interested in.

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Word boundary assertion: `\b`, `\B`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Capturing group: `(...)`

Modifier: (?ims-ims:...)

A **modifier** overrides flag settings in a specific part of a regular expression. It can be used to enable or disable flags that change the meanings of certain regex syntax elements. These flags are `i`, `m`, and `s`.

Syntax

```
(?flags1:pattern)
(?flags1-flags2:pattern)
```

> **Note:** JavaScript only has the "bounded" modifier form, where the pattern is placed inside the modifier group. Most other languages that support modifiers have an "unbounded" form, where the modifier is applied until the end of the closest containing group.

Parameters

`flags1` Optional

A string of flags to enable. Can contain any combination of `i`, `m`, and `s`.

`flags2` Optional

A string of flags to disable. Can contain any combination of `i`, `m`, and `s`, but must not contain any flags included in `flags1`.

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Description

Some flags change the meanings of regex syntax elements:

- The `i` flag makes the regex case-insensitive by making all literal characters and character classes implicitly be lowercase.
- The `m` flag changes the behavior of input boundary assertions `^` and `$` to match the start and end of each line, in addition to the start and end of the input string.
- The `s` flag changes the behavior of the wildcard `.` character to match any character, including line terminators.

Sometimes you may want these changes to only take effect in a specific part of a regex pattern. You can do so by wrapping that part in a modifier. For example:

```
/(?i:Hello) world/;
```

In this regex, the `i` flag is only enabled for the `Hello` part of the pattern. The `world` part is case-sensitive. Therefore, it matches `Hello world`, `hello world`, and `HELLO world`, but not `HELLO WORLD`. The following is equivalent, by enabling the `i` flag globally, and then disabling it for the `world` part:

```
/Hello (?-i:world)/i;
```

The `flags1` and `flags2` parameters can contain any combination of `i`, `m`, and `s`. However, the flags must all be unique between `flags1` and `flags2`—you cannot enable or disable a flag twice, or enable a flag and then immediately disable it.

The `flags1` and `flags2` parameters are optional, but at least one must be non-empty. `(?flags1-:pattern)` is a modifier that only enables flags (equivalent to `(?flags1:pattern)`). `(?-flags2:pattern)` is a modifier that only disables flags. `(?:pattern)` is just a non-capturing group, and `(?-:pattern)` is a syntax error.

Other flags don't make sense in a modifier and are thus syntax errors if included:

- The `g` and `y` flags determine how multiple calls to `exec()` behave and affect matching behavior of the whole regex.
- The `d` flag enables additional information in the `exec()` result and affects matching behavior of the whole regex.
- The `u` and `v` flags change the behavior of the regex engine in a way that's too complex to be locally modified. They also have global effects on the regex, such as how the `lastIndex` is advanced.

Examples

Matching a multiline format only at the start of the string

The following regex defines a format for a multiline string. The first `^` represents the start of the whole input string, by being inside a `(?-m:)` modifier, while all other `^` characters represent the start of a line:

```
const pattern = /(?-m:^)---\n^title:.*^slug:.*^---/ms;

const input = `---
title: "Modifier: (?ims-ims:...)"
slug: Web/JavaScript/Reference/Regular_expressions/Modifier
---`;

pattern.test(input); // true

// Extra line break at the start of string
const input2 = `\n${input}`;

pattern.test(input2); // false
```

Matching certain words case-insensitively

Imagine you are finding all variable declarations called `foo` or `bar` (because they are bad names). The word may appear in any case, but you know the keyword is always lowercase, so you can do this:

```
const pattern = /(?:var|let|const) (?i:foo|bar)\b/;

pattern.test("let foo;"); // true
pattern.test("const BAR = 1;"); // true
pattern.test("Let foo be a number"); // false
```

See also

- Groups and backreferences guide
- Regular expressions
- Non-capturing group: `(?:...)`

Named backreference: \k<name>

A **named backreference** refers to the submatch of a previous named capturing group and matches the same text as that group. For unnamed capturing groups, you need to use the normal backreference syntax.

Syntax

```
\k<name>
```

Parameters

`name`

The name of the group. Must be a valid identifier and refer to an existent named capturing group.

Description

Named backreferences are very similar to normal backreferences: it refers to the text matched by a capturing group and matches the same text. The difference is that you refer to the capturing group by name instead of by number. This makes the regular expression more readable and easier to refactor and maintain.

In Unicode-unaware mode, the sequence `\k` only starts a named backreference if the regex contains at least one named capturing group. Otherwise, it is an identity escape and is the same as the literal character `k`. This is a deprecated syntax for web compatibility, and you should not rely on it.

```
/\k/.test("k"); // true
```

Examples

Pairing quotes

The following function matches the `title='xxx'` and `title="xxx"` patterns in a string. To ensure the quotes match, we use a backreference to refer to the first quote. Accessing the second capturing group (`[2]`) returns the string between the matching quote characters:

```
function parseTitle(metastring) {
  return metastring.match(/title=(?<quote>["'])(.*?)\k<quote>/)[2];
}

parseTitle('title="foo"'); // 'foo'
parseTitle("title='foo' lang='en'"); // 'foo'
parseTitle('title="Named capturing groups\' advantages"'); // "Named capturing groups' advantages"
```

See also

- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Named capturing group: `(?<name>...)`
- Backreference: `\1`, `\2`

Named capturing group: (?<name>...)

A **named capturing group** is a particular kind of capturing group that allows you to give a name to the group. The group's matching result can later be identified by this name instead of by its index in the pattern.

Syntax

```
(?<name>pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

`name`

The name of the group. Must be a valid identifier.

Description

Named capturing groups can be used just like capturing groups — they also have their match index in the result array, and they can be referenced through `\1`, `\2`, etc. The only difference is that they can be *additionally* referenced by their name. The information of the capturing group's match can be accessed through:

- The `groups` property of the return value of `RegExp.prototype.exec()`, `String.prototype.match()`, and `String.prototype.matchAll()`
- The `groups` parameter of the `String.prototype.replace()` and `String.prototype.replaceAll()` methods' `replacement` callback function
- Named backreferences within the same pattern

All names must be unique within the same pattern. Multiple named capturing groups with the same name result in a syntax error.

```
/(?<name>)(?<name>)/; // SyntaxError: Invalid regular expression: Duplicate capture group name
```

This restriction is relaxed if the duplicate named capturing groups are not in the same disjunction alternative, so for any string input, only one named capturing group can actually be matched. This is a much newer feature, so check browser compatibility before using it.

```
/(?<year>\d{4})-\d{2}|\d{2}-(?<year>\d{4})/;
// Works; "year" can either come before or after the hyphen
```

Named capturing groups will all be present in the result. If a named capturing group is not matched (for example, it belongs to an unmatched alternative in a disjunction), the corresponding property on the `groups` object has value `undefined`.

```
/(?<ab>ab)|(?<cd>cd)/.exec("cd").groups; // [Object: null prototype] { ab: undefined, cd: 'cd' }
```

You can get the start and end indices of each named capturing group in the input string by using the `d` flag. In addition to accessing them on the `indices` property on the array returned by `exec()`, you can also access them by their names on `indices.groups`.

Compared to unnamed capturing groups, named capturing groups have the following advantages:

- They allow you to provide a descriptive name for each submatch result.
- They allow you to access submatch results without having to remember the order in which they appear in the pattern.
- When refactoring code, you can change the order of capturing groups without worrying about breaking other references.

Examples

Using named capturing groups

The following example parses a timestamp and an author name from a Git log entry (output with `git log --format=%ct,%an -- filename`):

```
function parseLog(entry) {
  const { author, timestamp } = /^(?<timestamp>\d+),(?<author>.+)$/.exec(
    entry,
  ).groups;
  return `${author} committed on ${new Date(
    parseInt(timestamp, 10) * 1000,
  ).toLocaleString()}`;
}

parseLog("1560979912,Caroline"); // "Caroline committed on 6/19/2019, 5:31:52 PM"
```

See also

- Polyfill of named capturing groups in `core-js`
- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Non-capturing group: `(?:...)`
- Named backreference: `\k<name>`
- ESLint rule: `prefer-named-capture-group`

Non-capturing group: (?:...)

A **non-capturing group** groups a subpattern, allowing you to apply a quantifier to the entire group or use disjunctions within it. It acts like the grouping operator in JavaScript expressions, and unlike capturing groups, it does not memorize the matched text, allowing for better performance and avoiding confusion when the pattern also contains useful capturing groups.

Syntax

```
(?:pattern)
```

Parameters

`pattern`

A pattern consisting of anything you may use in a regex literal, including a disjunction.

Examples

Grouping a subpattern and applying a quantifier

In the following example, we test if a file path ends with `styles.css` or `styles.[a hex hash].css`. Because the entire `\.[\da-f]+` part is optional, in order to apply the `?` quantifier to it, we need to group it into a new atom. Using a non-capturing group improves performance by not creating the extra match information that we don't need.

```
function isStylesheet(path) {
  return /styles(?:\.[\da-f]+)?\.css$/.test(path);
}

isStylesheet("styles.css"); // true
isStylesheet("styles.1234.css"); // true
isStylesheet("styles.cafe.css"); // true
isStylesheet("styles.1234.min.css"); // false
```

Grouping a disjunction

A disjunction has the lowest precedence in a regular expression. If you want to use a disjunction as a part of a bigger pattern, you must group it. You are advised to use a non-capturing group unless you rely on the matched text of the disjunction. The following example matches file extensions, using the same code as the input boundary assertion article:

```
function isImage(filename) {
  return /\.(?:png|jpe?g|webp|avif|gif)$/i.test(filename);
}

isImage("image.png"); // true
isImage("image.jpg"); // true
isImage("image.pdf"); // false
```

Avoiding refactoring hazards

Capturing groups are accessed by their position in the pattern. If you add or remove a capturing group, you must also update the positions of the other capturing groups, if you are accessing them through match results or backreferences. This can be a source of bugs, especially if most groups are purely for syntactic purposes (to apply quantifiers or to group disjunctions). Using non-capturing groups avoids this problem, and allows the indices of actual capturing groups to be easily tracked.

For example, suppose we have a function that matches the `title='xxx'` pattern in a string (example taken from capturing group). To ensure the quotes match, we use a backreference to refer to the first quote.

```
function parseTitle(metastring) {
  return metastring.match(/title=(["'])(.*?)\1/)[2];
}

parseTitle('title="foo"'); // 'foo'
```

If we later decided to add `name='xxx'` as an alias for `title=`, we will need to group the disjunction in another group:

```
function parseTitle(metastring) {
  // Oops — the backreference and index access are now off by one!
  return metastring.match(/(title|name)=(["'])(.*?)\1/)[2];
}

parseTitle('name="foo"'); // Cannot read properties of null (reading '2')
// Because \1 now refers to the "name" string, which isn't found at the end.
```

Instead of locating all places where we are referring to the capturing groups' indices and updating them one-by-one, it's better to avoid using a capturing group:

```
function parseTitle(metastring) {
  // Do not capture the title|name disjunction
  // because we don't use its value
  return metastring.match(/(?:title|name)=(["'])(.*?)\1/)[2];
}

parseTitle('name="foo"'); // 'foo'
```

Named capturing groups are another way to avoid refactoring hazards. It allows capturing groups to accessed by a custom name, which is unaffected when other capturing groups are added or removed.

See also

- Groups and backreferences guide
- Regular expressions
- Capturing group: `(...)`
- Named capturing group: `(?<name>...)`

Quantifier: *, +, ?, {n}, {n,}, {n,m}

A **quantifier** repeats an atom a certain number of times. The quantifier is placed after the atom it applies to.

Syntax

```
// Greedy
atom?
atom*
atom+
atom{count}
atom{min,}
atom{min,max}

// Non-greedy
atom??
atom*?
atom+?
atom{count}?
atom{min,}?
atom{min,max}?
```

> **Note:** Adding `?` after `{count}` is syntactically valid but practically useless. Since `{count}` always matches exactly `count` times, `atom{count}?` behaves the same as `atom{count}`.

Parameters

`atom`

A single atom.

`count`

A non-negative integer. The number of times the atom should be repeated.

`min`

A non-negative integer. The minimum number of times the atom can be repeated.

`max` Optional

A non-negative integer. The maximum number of times the atom can be repeated. If omitted, the atom can be repeated as many times as needed.

Description

A quantifier is placed after an atom to repeat it a certain number of times. It cannot appear on its own. Each quantifier is able to specify a minimum and maximum number that a pattern must be repeated for.

Quantifier	Minimum	Maximum
`?`	0	1
`*`	0	Infinity
`+`	1	Infinity
`{count}`	`count`	`count`
`{min,}`	`min`	Infinity
`{min,max}`	`min`	`max`

For the `{count}`, `{min,}`, and `{min,max}` syntaxes, there cannot be white spaces around the numbers — otherwise, it becomes a literal pattern.

```
const re = /a{1, 3}/;
re.test("aa"); // false
re.test("a{1, 3}"); // true
```

This behavior is fixed in Unicode-aware mode, where braces cannot appear literally without escaping. The ability to use `{` and `}` literally without escaping is a deprecated syntax for web compatibility, and you should not rely on it.

```
/a{1, 3}/u; // SyntaxError: Invalid regular expression: Incomplete quantifier
```

It is a syntax error if the minimum is greater than the maximum.

```
/a{3,2}/; // SyntaxError: Invalid regular expression: numbers out of order in {} quantifier
```

Quantifiers can cause capturing groups to match multiple times. See the capturing groups page for more information on the behavior in this case.

Each repeated match doesn't have to be the same string.

```
/[ab]*/.exec("aba"); // ['aba']
```

Quantifiers are *greedy* by default, which means they try to match as many times as possible until the maximum is reached, or until it's not possible to match further. You can make a quantifier *non-greedy* by adding a `?` after it. In this case, the quantifier will try to match as few times as possible, only matching more times if it's impossible to match the rest of the pattern with this many repetitions.

```
/a*/.exec("aaa"); // ['aaa']; the entire input is consumed
/a*?/.exec("aaa"); // ['']; it's possible to consume no characters and still match successfully
/^a*?$/.exec("aaa"); // ['aaa']; it's not possible to consume fewer characters and still match successfully
```

However, as soon as the regex successfully matches the string at some index, it will not try subsequent indices, although that may result in fewer characters being consumed.

```
/a*?$/.exec("aaa"); // ['aaa']; the match already succeeds at the first character, so the regex never attempts to start matching at the second character
```

Greedy quantifiers may try fewer repetitions if it's otherwise impossible to match the rest of the pattern.

```
/[ab]+[abc]c/.exec("abbc"); // ['abbc']
```

In this example, `[ab]+` first greedily matches `"abb"`, but `[abc]c` is not able to match the rest of the pattern (`"c"`), so the quantifier is reduced to match only `"ab"`.

Greedy quantifiers avoid matching infinitely many empty strings. If the minimum number of matches is reached and no more characters are being consumed by the atom at this position, the quantifier stops matching. This is why `/(a*)*/.exec("b")` does not result in an infinite loop.

Greedy quantifiers try to match as many *times* as possible; it does not maximize the *length* of the match. For example, `/(aa|aabaac|ba)*/.exec("aabaac")` matches `"aa"` and then `"ba"` instead of `"aabaac"`.

Quantifiers apply to a single atom. If you want to quantify a longer pattern or a disjunction, you must group it. Quantifiers cannot be applied to assertions.

```
/^*/; // SyntaxError: Invalid regular expression: nothing to repeat
```

In Unicode-aware mode, lookahead assertions can be quantified. This is a deprecated syntax for web compatibility, and you should not rely on it.

```
/(?=a)?b/.test("b"); // true; the lookahead is matched 0 time
```

Examples

Removing HTML tags

The following example removes HTML tags enclosed in angle brackets. Note the use of `?` to avoid consuming too many characters at once.

```
function stripTags(str) {
  return str.replace(/<.+?>/g, "");
}

stripTags("<p><em>lorem</em> <strong>ipsum</strong></p>"); // 'lorem ipsum'
```

The same effect can be achieved with a greedy match, but not allowing the repeated pattern to match `>`.

```
function stripTags(str) {
  return str.replace(/<[^>]+>/g, "");
}

stripTags("<p><em>lorem</em> <strong>ipsum</strong></p>"); // 'lorem ipsum'
```

> **Warning:** This is for demonstration only — it doesn't handle `>` in attribute values. Use a proper HTML sanitizer like the HTML sanitizer API instead.

Locating Markdown paragraphs

In Markdown, paragraphs are separated by one or more blank lines. The following example counts all paragraphs in a string by matching two or more line breaks.

```
function countParagraphs(str) {
  return str.match(/(?:\r?\n){2,}/g).length + 1;
}

countParagraphs(`
Paragraph 1

Paragraph 2
Containing some line breaks, but still the same paragraph

Another paragraph
`); // 3
```

> **Warning:** This is for demonstration only — it doesn't handle line breaks in code blocks or other Markdown block elements like headings. Use a proper Markdown parser instead.

See also

- Quantifiers guide
- Regular expressions
- Disjunction: `|`
- Character class: `[...]`, `[^...]`

Unicode character class escape: \p{...}, \P{...}

A **unicode character class escape** is a kind of character class escape that matches a set of characters specified by a Unicode property. It's only supported in Unicode-aware mode. When the `v` flag is enabled, it can also be used to match finite-length strings.

Try it

```
const sentence = "A ticket to 大阪 costs ¥2000 👌.";

const regexpEmojiPresentation = /\p{Emoji_Presentation}/gu;
console.log(sentence.match(regexpEmojiPresentation));
// Expected output: Array ["👌"]

const regexpNonLatin = /\P{Script_Extensions=Latin}+/gu;
console.log(sentence.match(regexpNonLatin));
// Expected output: Array [" ", " ", " 大阪 ", " ¥2000 👌."]

const regexpCurrencyOrPunctuation = /\p{Sc}|\p{P}/gu;
console.log(sentence.match(regexpCurrencyOrPunctuation));
// Expected output: Array ["¥", "."]
```

Syntax

```
\p{loneProperty}
\P{loneProperty}

\p{property=value}
\P{property=value}
```

Parameters

`loneProperty`

> A lone Unicode property name or value, following the same syntax as `value`. It specifies the value for the `General_Category` property, or a binary property name. In `v` mode, it can also be a binary Unicode property of strings.

> **Note:** ICU syntax allows omitting the `Script` property name as well, but JavaScript does not support this, because most of the time `Script_Extensions` is more useful than `Script`.

`property`

A Unicode property name. Must be made of ASCII letters (`A–Z`, `a–z`) and underscores (_), and must be one of the non-binary property names.

`value`

A Unicode property value. Must be made of ASCII letters (`A–Z`, `a–z`), underscores (_), and digits (`0–9`), and must be one of the supported values listed in `PropertyValueAliases.txt`.

Description

`\p` and `\P` are only supported in Unicode-aware mode. In Unicode-unaware mode, they are identity escapes for the `p` or `P` character.

Every Unicode character has a set of properties that describe it. For example, the character `a` has the `General_Category` property with value `Lowercase_Letter`, and the `Script` property with value `Latn`. The `\p` and `\P` escape sequences allow you to match a character based on its properties. For example, `a` can be matched by `\p{Lowercase_Letter}` (the `General_Category` property name is optional) as well as `\p{Script=Latn}`. `\P` creates a *complement class* that consists of code points without the specified property.

When the `i` flag is set, `\P` character classes are handled slightly differently in `u` and `v` modes. In `u` mode, case-folding happens after subtraction; in `v` mode, case-folding happens before subtraction. More concretely, in `u` mode, `\P{property}` matches `caseFold(allCharacters - charactersWithProperty)`. This means `/\P{Lowercase_Letter}/iu` still matches `"a"`, because `A` is not a `Lowercase_Letter`. In `v` mode, `\P{property}` matches `caseFold(allCharacters) - caseFold(charactersWithProperty)`. This means `/\P{Lowercase_Letter}/iv` does not match `"a"`, because `A` is not even in the set of all case-folded Unicode characters. See also complement classes and case-insensitive matching.

To compose multiple properties, use the character set intersection syntax enabled with the `v` flag, or see pattern subtraction and intersection.

In `v` mode, `\p` may match a sequence of code points, defined in Unicode as "properties of strings". This is most useful for emojis, which are often composed of multiple code points. However, `\P` can only complement character properties.

> **Note:** There are plans to port the properties of strings feature to `u` mode as well.

Examples

General categories

General categories are used to classify Unicode characters and subcategories are available to define a more precise categorization. It is possible to use both short or long forms in Unicode property escapes.

They can be used to match letters, numbers, symbols, punctuations, spaces, etc. For a more exhaustive list of general categories, please refer to the Unicode specification.

```
// finding all the letters of a text
const story = "It's the Cheshire Cat: now I shall have somebody to talk to.";

// Most explicit form
story.match(/\p{General_Category=Letter}/gu);

// It is not mandatory to use the property name for General categories
story.match(/\p{Letter}/gu);

// This is equivalent (short alias):
story.match(/\p{L}/gu);

// This is also equivalent (conjunction of all the subcategories using short aliases)
story.match(/\p{Lu}|\p{Ll}|\p{Lt}|\p{Lm}|\p{Lo}/gu);
```

Scripts and script extensions

Some languages use different scripts for their writing system. For instance, English and Spanish are written using the Latin script while Arabic and Russian are written with other scripts (respectively Arabic and Cyrillic). The `Script` and `Script_Extensions` Unicode properties allow regular expression to match characters according to the script they are mainly used with (`Script`) or according to the set of scripts they belong to (`Script_Extensions`).

For example, `A` belongs to the `Latin` script and `ε` to the `Greek` script.

```
const mixedCharacters = "aεЛ";

// Using the canonical "long" name of the script
mixedCharacters.match(/\p{Script=Latin}/u); // a

// Using a short alias (ISO 15924 code) for the script
mixedCharacters.match(/\p{Script=Grek}/u); // ε

// Using the short name sc for the Script property
mixedCharacters.match(/\p{sc=Cyrillic}/u); // Л
```

For more details, refer to the Unicode specification, the Scripts table in the ECMAScript specification, and the ISO 15924 list of script codes.

If a character is used in a limited set of scripts, the `Script` property will only match for the "predominant" used script. If we want to match characters based on a "non-predominant" script, we could use the `Script_Extensions` property (`scx` for short).

```
// ٢ is the digit 2 in Arabic-Indic notation
// while it is predominantly written within the Arabic script
// it can also be written in the Thaana script

"٢".match(/\p{Script=Thaana}/u);
// null as Thaana is not the predominant script

"٢".match(/\p{Script_Extensions=Thaana}/u);
// ["٢", index: 0, input: "٢", groups: undefined]
```

Unicode property escapes vs. character classes

With JavaScript regular expressions, it is also possible to use character classes and especially `\w` or `\d` to match letters or digits. However, such forms only match characters from the *Latin* script (in other words, `a` to `z` and `A` to `Z` for `\w` and `0` to `9` for `\d`). As shown in this example, it might be a bit clumsy to work with non Latin texts.

Unicode property escapes categories encompass much more characters and `\p{Letter}` or `\p{Number}` will work for any script.

```
// Trying to use ranges to avoid \w limitations:

const nonEnglishText = "Приключения Алисы в Стране чудес";
const regexpBMPWord = /([\u0000-\u0019\u0021-\uFFFF])+/gu;
// BMP goes through U+0000 to U+FFFF but space is U+0020

console.table(nonEnglishText.match(regexpBMPWord));

// Using Unicode property escapes instead
const regexpUPE = /\p{L}+/gu;
console.table(nonEnglishText.match(regexpUPE));
```

Matching prices

The following example matches prices in a string:

```
function getPrices(str) {
  // Sc stands for "currency symbol"
  return [...str.matchAll(/\p{Sc}\s*[\d.,]+/gu)].map((match) => match[0]);
}

const str = `California rolls $6.99
Crunchy rolls $8.49
Shrimp tempura $10.99`;
console.log(getPrices(str)); // ["$6.99", "$8.49", "$10.99"]

const str2 = `US store $19.99
Europe store €18.99
Japan store ¥2000`;
console.log(getPrices(str2)); // ["$19.99", "€18.99", "¥2000"]
```

Matching strings

With the `v` flag, `\p{…}` can match strings that are potentially longer than one character by using a property of strings:

```
const flag = "🇺🇳";
console.log(flag.length); // 4 (two code points, each a surrogate pair)
console.log([...flag].length); // 2
console.log(/\p{RGI_Emoji_Flag_Sequence}/v.exec(flag)); // [ '🇺🇳' ]
```

However, you can't use `\P` to match "a string that does not have a property", because it's unclear how many characters should be consumed.

```
/\P{RGI_Emoji_Flag_Sequence}/v; // SyntaxError: Invalid regular expression: Invalid property name
```

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`
- Unicode character property on Wikipedia
- ES2018: RegExp Unicode property escapes by Dr. Axel Rauschmayer (2017)
- Unicode regular expressions § Properties
- Unicode Utilities: UnicodeSet
- RegExp v flag with set notation and properties of strings on v8.dev (2022)

Wildcard: .

A **wildcard** matches all characters except line terminators. It also matches line terminators if the `s` flag is set.

Syntax

```
.
```

Description

`.` matches any character except line terminators. If the `s` flag is set, `.` also matches line terminators.

The exact character set matched by `.` depends on whether the regex is Unicode-aware. If it is Unicode-aware, `.` matches any Unicode code point; otherwise, it matches any UTF-16 code unit. For example:

```
/../.test("😍"); // true; matches two UTF-16 code units as a surrogate pair
/../u.test("😍"); // false; input only has one Unicode character
```

Examples

Usage with quantifiers

Wildcards are often used with quantifiers to match any character sequence, until the next character of interest is found. For example, the following example extracts the title of a Markdown page in the form `# Title`:

```
function parseTitle(entry) {
  // Use multiline mode because the title may not be at the start of
  // the file. Note that the m flag does not make . match line
  // terminators, so the title must be on a single line
  // Return text matched by the first capturing group.
  return /^#[ \t]+(.+)$/m.exec(entry)?.[1];
}

parseTitle("# Hello world"); // "Hello world"
parseTitle("## Subsection"); // undefined
parseTitle(`
---
slug: Web/JavaScript/Reference/Regular_expressions/Wildcard
---

# Wildcard: .

A **wildcard** matches all characters except line terminators.
`); // "Wildcard: ."
```

Matching code block content

The following example matches the content of a code block enclosed by three backticks in Markdown. It uses the `s` flag to make `.` match line terminators, because the content of a code block may span multiple lines:

```
function parseCodeBlock(entry) {
  return /^```.*?^(.+?)\n```/ms.exec(entry)?.[1];
}

parseCodeBlock(`
\`\`\`js
console.log("Hello world");
\`\`\`
`); // "console.log("Hello world");"

parseCodeBlock(`
A \`try...catch\` statement must have the blocks enclosed in curly braces.

\`\`\`js example-bad
try
  doSomething();
catch (e)
  console.log(e);
\`\`\`
`); // "try\n  doSomething();\ncatch (e)\n  console.log(e);"
```

> **Warning:** These examples are for demonstration only. If you want to parse Markdown, use a dedicated Markdown parser because there are many edge cases to consider.

See also

- Character classes guide
- Regular expressions
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`

Word boundary assertion: \b, \B

A **word boundary assertion** checks if the current position in the string is a word boundary. A word boundary is where the next character is a word character and the previous character is not a word character, or vice versa.

Syntax

```
\b
\B
```

Description

`\b` asserts that the current position in the string is a word boundary. `\B` negates the assertion: it asserts that the current position is not a word boundary. Both are *assertions*, so unlike other character escapes or character class escapes, `\b` and `\B` don't consume any characters.

A word character includes the following:

- Letters (A–Z, a–z), numbers (0–9), and underscore (_).
- If the regex is Unicode-aware and the `i` flag is set, other Unicode characters that get canonicalized to one of the characters above through case folding.

Word characters are also matched by the `\w` character class escape.

Out-of-bounds input positions are considered non-word characters. For example, the following are successful matches:

```
/\ba/.exec("abc");
/c\b/.exec("abc");

/\B /.exec(" abc");
/ \B/.exec("abc ");
```

Examples

Detecting words

The following example detects if a string contains the word "thanks" or "thank you":

```
function hasThanks(str) {
  return /\b(thanks|thank you)\b/i.test(str);
}

hasThanks("Thanks! You helped me a lot."); // true
hasThanks("Just want to say thank you for all your work."); // true
hasThanks("Thanksgiving is around the corner."); // false
```

> **Warning:** Not all languages have clearly defined word boundaries. If you are working with languages like Chinese or Thai, where there are no whitespace separators, use a more advanced library like `Intl.Segmenter` to search for words instead.

See also

- Assertions guide
- Regular expressions
- Input boundary assertion: `^`, `$`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Character escape: `\n`, `\u{...}`

Statements and declarations

JavaScript code is built from statements and declarations, which can contain expressions. This reference groups the constructs used to control execution and declare bindings. A statement can span multiple lines, and a line can contain multiple statements.

Statements and declarations by category

For an alphabetical listing see the sidebar on the left.

Control flow

`return`

: Specifies the value to be returned by a function.

`break`

: Terminates the current loop, switch, or label statement and transfers program control to the statement following the terminated statement.

`continue`

: Terminates execution of the statements in the current iteration of the current or labeled loop, and continues execution of the loop with the next iteration.

`throw`

: Throws a user-defined exception.

`if...else`

Executes a statement if a specified condition is true. If the condition is false, another statement can be executed.

`switch`

Evaluates an expression, matching the expression's value to a case clause, and executes statements associated with that case.

`try...catch`

Marks a block of statements to try, and specifies a response, should an exception be thrown.

Declaring variables

`var`

Declares a variable, optionally initializing it to a value.

`let`

Declares a block-scoped variable, optionally initializing it to a value.

`const`

Declares a block-scoped variable that cannot be re-assigned, which must be initialized at declaration.

`using`

Declares a variable like `const` that is *synchronously disposed*.

`await using`

Declares a variable like `const` that is *asynchronously disposed*.

Functions and classes

`function`

Declares a function with the specified parameters.

`function*`

Generator Functions enable writing iterators more easily.

`async function`

Declares an async function with the specified parameters.

`async function*`

Asynchronous Generator Functions enable writing async iterators more easily.

`class`

Declares a class.

Iterations

`do...while`

Creates a loop that executes a specified statement until the test condition evaluates to false. The condition is evaluated after executing the statement, resulting in the specified statement executing at least once.

`for`

Creates a loop that consists of three optional expressions, enclosed in parentheses and separated by semicolons, followed by a statement executed in the loop.

`for...in`

Iterates over the enumerable properties of an object, in arbitrary order. For each distinct property, statements can be executed.

`for...of`

Iterates over iterable objects (including arrays, array-like objects, iterators and generators), invoking a custom iteration hook with statements to be executed for the value of each distinct property.

`for await...of`

Iterates over async iterable objects, array-like objects, iterators and generators, invoking a custom iteration hook with statements to be executed for the value of each distinct property.

`while`

Creates a loop that executes a specified statement as long as the test condition evaluates to true. The condition is evaluated before executing the statement.

Others

Empty

An empty statement is used to provide no statement, although the JavaScript syntax would expect one.

Block

A block statement is used to group zero or more statements. The block is delimited by a pair of curly braces.

Expression statement

An expression statement evaluates an expression and discards its result. It allows the expression to perform side effects, such as executing a function or updating a variable.

`debugger`

Invokes any available debugging functionality. If no debugging functionality is available, this statement has no effect.

`export`

Used to export functions to make them available for imports in external modules, and other scripts.

`import`

Used to import functions exported from an external module, another script.

`import defer`

Loads a module as a namespace, deferring synchronous evaluation until the returned namespace's properties are accessed.

`import source`

Results in an object representing the module's compiled source, without loading its dependencies, linking it, or evaluating it.

label

Provides a statement with an identifier that you can refer to using a `break` or `continue` statement.

`with`

Extends the scope chain for a statement.

What are statements, declarations, and expressions?

All JavaScript programs are composed of a sequence of top-level constructs, using one of the syntaxes listed above. These are known as *statements* and *declarations*. In MDN docs, we often colloquially refer to both as *statements*, but they are technically two disjoint sets of grammars.

The following are declarations:

- `let`
- `const`
- `using`
- `await using`
- `function`
- `function*`
- `async function`
- `async function*`
- `class`
- `export` (Note: it can only appear at the top-level of a module)
- `import` (Note: it can only appear at the top-level of a module)

Everything else in the list above is a statement.

The terms "statement" and "declaration" have a precise meaning in the formal syntax of JavaScript that affects where they may be placed in code. For example, in most control-flow structures, the body only accepts statements — such as the two arms of an `if...else`:

```
if (condition)
  statement1;
else
  statement2;
```

If you use a declaration instead of a statement, it would be a `SyntaxError`. For example, a `let` declaration is not a statement, so you can't use it in its bare form as the body of an `if` statement.

```
if (condition)
  let i = 0; // SyntaxError: Lexical declaration cannot appear in a single-statement context
```

On the other hand, `var` is a statement, so you can use it on its own as the `if` body.

```
if (condition)
  var i = 0;
```

You can see declarations as "binding identifiers to values", and statements as "carrying out actions". The fact that `var` is a statement instead of a declaration is a special case, because it doesn't follow normal lexical scoping rules and may create side effects — in the form of creating global variables, mutating existing `var`-defined

variables, and defining variables that are visible outside of its block (because `var`-defined variables aren't block-scoped).

As another example, labels can only be attached to statements.

```
label: const a = 1; // SyntaxError: Lexical declaration cannot appear in a single-statement context
```

> **Note:** There's a legacy grammar that allows function declarations to have labels, but it's only standardized for compatibility with web reality.

To get around this, you can wrap the declaration in braces — this makes it part of a block statement.

```
label: {
  const a = 1;
}

if (condition) {
  let i = 0;
}
```

In JavaScript, statements and declarations produce values, but these values are almost never observable (other than `eval()`). Their purpose is to manipulate the surrounding environment and produce side effects—creating variable bindings, outputting things, modifying variable values, etc. The values they use come from evaluating *expressions*.

Expressions are not top-level constructs; they can only be used in specific slots inside statements and declarations, such as `if (expression)`, `const x = expression`, etc. The expression statement syntax allows most expressions to be used as statements, but that's just a statement with a single expression slot.

You can map out JavaScript syntax structures using just statements, declarations, and expressions:

- Statements can contain statements, declarations, and expressions (like `if (expression) statement` and block statements `{ statement; declaration }`)
- Declarations can contain statements, declarations, and expressions (like `function x() { statement; declaration }` and `const x = expression`)
- Expressions can contain statements, declarations, and expressions (like `() => { statement; declaration }` and `console.log(expression)`)

In the reference documentation for each piece of syntax, we describe all its "slots" and say whether each slot is a statement, declaration, or expression.

Operators is another important concept in JavaScript grammar, but they do not work like building blocks. See What are operators? for more information.

See also

- Expressions and operators

async function

The **`async function`** declaration creates a binding of a new async function to a given name. The `await` keyword is permitted within the function body, enabling asynchronous, promise-based behavior to be written in a cleaner style and avoiding the need to explicitly configure promise chains.

You can also define async functions using the `async function` expression.

Try it

```
function resolveAfter2Seconds() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("resolved");
    }, 2000);
  });
}

async function asyncCall() {
  console.log("calling");
  const result = await resolveAfter2Seconds();
  console.log(result);
  // Expected output: "resolved"
}

asyncCall();
```

Syntax

```
async function name(param0) {
  statements
}
async function name(param0, param1) {
  statements
}
async function name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** There cannot be a line terminator between `async` and `function`, otherwise a semicolon is automatically inserted, causing `async` to become an identifier and the rest to become a `function` declaration.

Parameters

`name`

The function's name.

`param` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements comprising the body of the function. The `await` mechanism may be used.

Description

An `async function` declaration creates an `AsyncFunction` object. Each time when an async function is called, it returns a new `Promise` which will be resolved with the value returned by the async function, or rejected with an exception uncaught within the async function.

Async functions can contain zero or more `await` expressions. Await expressions make promise-returning functions behave as though they're synchronous by suspending execution until the returned promise is fulfilled or rejected. The resolved value of the promise is treated as the return value of the await expression. Use of `async` and `await` enables the use of ordinary `try` / `catch` blocks around asynchronous code.

> **Note:** The `await` keyword is only valid inside async functions within regular JavaScript code. If you use it outside of an async function's body, you will get a `SyntaxError`.
>
> `await` can be used on its own with JavaScript modules.

> **Note:** The purpose of `async/await` is to simplify the syntax necessary to consume promise-based APIs. The behavior of `async/await` is similar to combining generators and promises.

Async functions always return a promise. If the return value of an async function is not explicitly a promise, it will be implicitly wrapped in a promise.

For example, consider the following code:

```
async function foo() {
  return 1;
}
```

It is similar to:

```
function foo() {
  return Promise.resolve(1);
}
```

Note that even though the return value of an async function behaves as if it's wrapped in a `Promise.resolve`, they are not equivalent. An async function will return a different *reference*, whereas `Promise.resolve` returns the same reference if the given value is a promise. It can be a problem when you want to check the equality of a promise and a return value of an async function.

```
const p = new Promise((res, rej) => {
  res(1);
});

async function asyncReturn() {
  return p;
}

function basicReturn() {
  return Promise.resolve(p);
}

console.log(p === basicReturn()); // true
console.log(p === asyncReturn()); // false
```

The body of an async function can be thought of as being split by zero or more await expressions. Top-level code, up to and including the first await expression (if there is one), is run synchronously. In this way, an async function without an await expression will run synchronously. If there is an await expression inside the function body, however, the async function will always complete asynchronously.

For example:

```
async function foo() {
  await 1;
}
```

It is also equivalent to:

```
function foo() {
  return Promise.resolve(1).then(() => undefined);
```

```
}
```

Code after each await expression can be thought of as existing in a `.then` callback. In this way a promise chain is progressively constructed with each reentrant step through the function. The return value forms the final link in the chain.

In the following example, we successively await two promises. Progress moves through function `foo` in three stages.

1. The first line of the body of function `foo` is executed synchronously, with the await expression configured with the pending promise. Progress through `foo` is then suspended and control is yielded back to the function that called `foo`.
2. Some time later, when the first promise has either been fulfilled or rejected, control moves back into `foo`. The result of the first promise fulfillment (if it was not rejected) is returned from the await expression. Here `1` is assigned to `result1`. Progress continues, and the second await expression is evaluated. Again, progress through `foo` is suspended and control is yielded.
3. Some time later, when the second promise has either been fulfilled or rejected, control re-enters `foo`. The result of the second promise resolution is returned from the second await expression. Here `2` is assigned to `result2`. Control moves to the return expression (if any). The default return value of `undefined` is returned as the resolution value of the current promise.

```
async function foo() {
  const result1 = await new Promise((resolve) =>
    setTimeout(() => resolve("1")),
  );
  const result2 = await new Promise((resolve) =>
    setTimeout(() => resolve("2")),
  );
}
foo();
```

Note how the promise chain is not built-up in one go. Instead, the promise chain is constructed in stages as control is successively yielded from and returned to the async function. As a result, we must be mindful of error handling behavior when dealing with concurrent asynchronous operations.

For example, in the following code an unhandled promise rejection error will be thrown, even if a `.catch` handler has been configured further along the promise chain. This is because `p2` will not be "wired into" the promise chain until control returns from `p1`.

```
async function foo() {
  const p1 = new Promise((resolve) => setTimeout(() => resolve("1"), 1000));
  const p2 = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("failed")), 500),
  );
  const results = [await p1, await p2]; // Do not do this! Use Promise.all or Promise.allSettled instead.
}
foo().catch(() => {}); // Attempt to swallow all errors...
```

`async function` declarations behave similar to `function` declarations — they are hoisted to the top of their scope and can be called anywhere in their scope, and they can be redeclared only in certain contexts.

Examples

Async functions and execution order

```
function resolveAfter2Seconds() {
  console.log("starting slow promise");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("slow");
      console.log("slow promise is done");
    }, 2000);
  });
}

function resolveAfter1Second() {
  console.log("starting fast promise");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("fast");
      console.log("fast promise is done");
    }, 1000);
  });
}

async function sequentialStart() {
  console.log("== sequentialStart starts ==");

  // 1. Start a timer, log after it's done
  const slow = resolveAfter2Seconds();
  console.log(await slow);

  // 2. Start the next timer after waiting for the previous one
  const fast = resolveAfter1Second();
  console.log(await fast);

  console.log("== sequentialStart done ==");
}

async function sequentialWait() {
```

```
  console.log("== sequentialWait starts ==");

  // 1. Start two timers without waiting for each other
  const slow = resolveAfter2Seconds();
  const fast = resolveAfter1Second();

  // 2. Wait for the slow timer to complete, and then log the result
  console.log(await slow);
  // 3. Wait for the fast timer to complete, and then log the result
  console.log(await fast);

  console.log("== sequentialWait done ==");
}

async function concurrent1() {
  console.log("== concurrent1 starts ==");

  // 1. Start two timers concurrently and wait for both to complete
  const results = await Promise.all([
    resolveAfter2Seconds(),
    resolveAfter1Second(),
  ]);
  // 2. Log the results together
  console.log(results[0]);
  console.log(results[1]);

  console.log("== concurrent1 done ==");
}

async function concurrent2() {
  console.log("== concurrent2 starts ==");

  // 1. Start two timers concurrently, log immediately after each one is done
  await Promise.all([
    (async () => console.log(await resolveAfter2Seconds()))(),
    (async () => console.log(await resolveAfter1Second()))(),
  ]);
  console.log("== concurrent2 done ==");
}

sequentialStart(); // after 2 seconds, logs "slow", then after 1 more second, "fast"

// wait above to finish
setTimeout(sequentialWait, 4000); // after 2 seconds, logs "slow" and then "fast"

// wait again
setTimeout(concurrent1, 7000); // same as sequentialWait

// wait again
setTimeout(concurrent2, 10000); // after 1 second, logs "fast", then after 1 more second, "slow"
```

await and concurrency

In `sequentialStart`, execution suspends 2 seconds for the first `await`, and then another second for the second `await`. The second timer is not created until the first has already fired, so the code finishes after 3 seconds.

In `sequentialWait`, both timers are created and then `await`ed. The timers run concurrently, which means the code finishes in 2 rather than 3 seconds, i.e., the slowest timer. However, the `await` calls still run in series, which means the second `await` will wait for the first one to finish. In this case, the result of the fastest timer is processed after the slowest.

If you wish to safely perform other jobs after two or more jobs run concurrently and are complete, you must await a call to `Promise.all()` or `Promise.allSettled()` before that job.

> **Warning:** The functions `sequentialWait` and `concurrent1` are not functionally equivalent.
>
> In `sequentialWait`, if promise `fast` rejects before promise `slow` is fulfilled, then an unhandled promise rejection error will be raised, regardless of whether the caller has configured a catch clause.
>
> In `concurrent1`, `Promise.all` wires up the promise chain in one go, meaning that the operation will fail-fast regardless of the order of rejection of the promises, and the error will always occur within the configured promise chain, enabling it to be caught in the normal way.

Rewriting a Promise chain with an async function

An API that returns a `Promise` will result in a promise chain, and it splits the function into many parts. Consider the following code:

```
function getProcessedData(url) {
  return downloadData(url) // returns a promise
    .catch((e) => downloadFallbackData(url)) // returns a promise
    .then((v) => processDataInWorker(v)); // returns a promise
}
```

it can be rewritten with a single async function as follows:

```
async function getProcessedData(url) {
  let v;
  try {
    v = await downloadData(url);
  } catch (e) {
    v = await downloadFallbackData(url);
  }
  return processDataInWorker(v);
}
```

Alternatively, you can chain the promise with `catch()`:

```
async function getProcessedData(url) {
  const v = await downloadData(url).catch((e) => downloadFallbackData(url));
  return processDataInWorker(v);
}
```

In the two rewritten versions, notice there is no `await` statement after the `return` keyword, although that would be valid too: The return value of an async function is implicitly wrapped in `Promise.resolve` - if it's not already a promise itself (as in the examples).

See also

- Functions guide
- Using promises guide
- Functions
- `AsyncFunction`
- `async function` expression
- `function`
- `function*`
- `async function*`
- `await`
- `Promise`
- Decorating async JavaScript functions on innolitics.com (2016)

await using

The **`await using`** declaration declares block-scoped local variables that are *asynchronously disposed*. Like `const`, variables declared with `await using` must be initialized and cannot be reassigned. The variable's value must be either `null`, `undefined`, or an object with a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method. When the variable goes out of scope, the `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method of the object is called and awaited, to ensure that resources are freed.

Syntax

```
await using name1 = value1;
await using name1 = value1, name2 = value2;
await using name1 = value1, name2 = value2, /* …, */ nameN = valueN;
```

`nameN`

> The name of the variable to declare. Each must be a legal JavaScript identifier and *not* a destructuring binding pattern.

`valueN`

> Initial value of the variable. It can be any legal expression but its value must be either `null`, `undefined`, or an object with a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method.

Description

This declaration can only be used in places where both `await` and `using` can be used, which include:

- Inside a block (if the block is also in an async context)
- Inside an async function or async generator function body
- At the top level of a module
- In the initializer of a `for`, `for...of` (if the `for` loop is also in an async context), or `for await...of` loop

An `await using` declares an async disposable resource that's tied to the lifetime of the variable's scope (block, function, module, etc.). When the scope exits, the resource is disposed of asynchronously. Its syntax may be somewhat confusing, because the `await` does not have an awaiting effect when the variable is first declared, but only when the variable goes out of scope.

When a variable is first declared and its value is non-nullish, a *disposer* is retrieved from the object. The `[Symbol.asyncDispose]` property is tried first, and falls back to `[Symbol.dispose]` if `[Symbol.asyncDispose]` is `undefined`. If neither property contains a function, a `TypeError` is thrown. Notably, the `[Symbol.dispose]()` method is wrapped into a function that looks like `async () => { object[Symbol.dispose](); }`, which means if it returns a promise, that promise is *not* awaited. This disposer is saved to the scope.

When the variable goes out of scope, the disposer is called and awaited. If the scope contains multiple `using` or `await using` declarations, all disposers are run in sequence in the reverse order of declaration, regardless of the type of declaration. All disposers are guaranteed to run (much like the `finally` block in `try...catch...finally`). All errors thrown during disposal, including the initial error that caused the scope exit (if applicable), are all aggregated inside one `SuppressedError`, with each earlier exception as the `suppressed` property and the later exception as the `error` property. This `SuppressedError` is thrown after disposal is complete.

The variable is allowed to have value `null` or `undefined`, so the resource can be optionally present. As long as one `await using` variable is declared in this scope, at least one `await` is guaranteed to happen on scope exit, even if the variable actually has value `null` or `undefined`. This prevents the disposal from happening synchronously, causing timing issues (see control flow effects of `await`).

`await using` ties resource management to lexical scopes, which is both convenient and sometimes confusing. See below for some examples where it may not behave how you expect. If you want to hand-manage resource disposal, while maintaining the same error handling guarantees, you can use `AsyncDisposableStack` instead.

Examples

You should also check `using` for more examples, especially some general caveats with respect to scope-based resource management.

Basic usage

Usually, you use `await using` on some library-provided resource that already implements the async disposable protocol. For example, the Node.js `FileHandle` is async disposable:

```
import fs from "node:fs/promises";

async function example() {
  await using file = await fs.open("example.txt", "r");
  console.log(await file.read());
  // Before `file` goes out of scope, it is disposed by calling `file[Symbol.asyncDispose]()` and awaited.
}
```

Note that there are two `await` operations in the declaration for `file`, which do different things and are both necessary. `await fs.open()` causes an await during *acquisition*: it waits for the file to be opened and unwraps the returned promise into a `FileHandle` object. `await using file` causes an await during *disposal*: it makes `file` disposed asynchronously when the variable goes out of scope.

<code>await using</code> with <code>for await...of</code>

It's very easy to confuse the following three syntaxes:

- `for await (using x of y) { ... }`
- `for (await using x of y) { ... }`
- `for (using x of await y) { ... }`

It may be even more confusing to know that they can be used together.

```
for await (await using x of await y) {
  // ...
}
```

First, `await y` does what you expect: we `await` the promise `y`, which is expected to resolve to an object we iterate over. Let us set this variant aside.

The `for await...of` loop requires the `y` object to be an *async iterable*. This means that the object must have a `[Symbol.asyncIterator]` method that returns an *async iterator*, whose `next()` method returns a promise representing the result. This is for when the iterable doesn't know what the next value is, or even if it's done yet, until some async operation is complete.

On the other hand, the `await using x` syntax requires the `x` object, as yielded from the iterable, to be an *async disposable*. This means that the object must have a `[Symbol.asyncDispose]` method that returns a promise representing the disposal operation. This is a separate concern from the iteration itself, and is only called when the variable `x` goes out of scope.

In other words, all of the following four combinations are valid and do different things:

- `for (using x of y)`: `y` is synchronously iterated, yielding one result at a time, which can be disposed synchronously.
- `for await (using x of y)`: `y` is asynchronously iterated, yielding one result at a time after awaiting, but the result value can be disposed synchronously.
- `for (await using x of y)`: `y` is synchronously iterated, yielding one result at a time, but the result value can only be disposed asynchronously.
- `for await (await using x of y)`: `y` is asynchronously iterated, yielding one result at a time after awaiting, and the result value can only be disposed asynchronously.

Below, we create some fictitious values of `y` to demonstrate their use cases. For asynchronous APIs, we base our code on the Node.js `fs/promises` module.

```
const syncIterableOfSyncDisposables = [
  stream1.getReader(),
  stream2.getReader(),
];
for (using reader of syncIterableOfSyncDisposables) {
  console.log(reader.read());
}
```

```
async function* requestMany(urls) {
  for (const url of urls) {
    const res = await fetch(url);
    yield res.body.getReader();
  }
}
const asyncIterableOfSyncDisposables = requestMany([
  "https://example.com",
  "https://example.org",
]);
for await (using reader of asyncIterableOfSyncDisposables) {
```

```
  console.log(reader.read());
}
```

```
const syncIterableOfAsyncDisposables = await Promise.all(
  fs.globSync("*.txt").map((path) => fs.open(path, "r")),
);
for (await using file of syncIterableOfAsyncDisposables) {
  console.log(await file.read());
}
```

```
async function* globHandles(pattern) {
  for await (const path of fs.glob(pattern)) {
    yield await fs.open(path, "r");
  }
}
const asyncIterableOfAsyncDisposables = globHandles("*.txt");
for await (await using file of asyncIterableOfAsyncDisposables) {
  console.log(await file.read());
}
```

Implicit await on scope exit

As soon as one `await using` is declared in a scope, the scope will always have an `await` on exit, even if the variable is `null` or `undefined`. This ensures stable execution order and error handling. The Control flow effects of await examples have more details on this.

In the example below, the `example()` call below doesn't resolve until one tick after, because of an implicit `await` when the function returns.

```
async function example() {
  await using nothing = null;
  console.log("Example call");
}

example().then(() => console.log("Example done"));
Promise.resolve().then(() => console.log("Microtask done"));
// Output:
// Example call
// Microtask done
// Example done
```

Consider the same code but with a synchronous `using` instead. This time, the `example()` call immediately resolves, so the two `then()` handlers are called in the same tick.

```
async function example() {
  using nothing = null;
  console.log("Example call");
}

example().then(() => console.log("Example done"));
Promise.resolve().then(() => console.log("Microtask done"));
// Output:
// Example call
// Example done
// Microtask done
```

For a more realistic example, consider two concurrent calls to a function:

```
class Resource {
  #name;
  constructor(name) {
    this.#name = name;
  }
  async [Symbol.asyncDispose]() {
    console.log(`Disposing resource ${this.#name}`);
  }
}

async function example(id, createOptionalResource) {
  await using required = new Resource(`required ${id}`);
  await using optional = createOptionalResource
    ? new Resource("optional")
    : null;
  await using another = new Resource(`another ${id}`);
}

example(1, true);
example(2, false);
// Output:
// Disposing resource another 1
// Disposing resource another 2
// Disposing resource optional
// Disposing resource required 1
// Disposing resource required 2
```

As you can see, the `required 2` resource is disposed in the same tick as `required 1`. If the `optional` resource did not cause a redundant `await`, then `required 2` would have been disposed earlier, which would be simultaneous with `optional`.

See also

- JavaScript resource management
- `const`
- `let`
- `using`
- `Symbol.asyncDispose`
- `AsyncDisposableStack`

Block statement

A **block statement** is used to group zero or more statements. The block is delimited by a pair of braces ("curly braces") and contains a list of zero or more statements and declarations.

Try it

```
var x = 1;
let y = 1;

if (true) {
  var x = 2;
  let y = 2;
}

console.log(x);
// Expected output: 2

console.log(y);
// Expected output: 1
```

Syntax

```
{
  StatementList
}
```

`StatementList`

Statements and declarations grouped within the block statement.

Description

The block statement is often called the *compound statement* in other languages. It allows you to use multiple statements where JavaScript expects only one statement. Combining statements into blocks is a common practice in JavaScript, especially when used in association with control flow statements like `if...else` and `for`. The opposite behavior is possible using an empty statement, where you provide no statement, although one is required.

In addition, combined with block-scoped declarations like `let`, `const`, and `class`, blocks can prevent temporary variables from polluting the global namespace, just like IIFEs do.

Block scoping rules with var or function declaration in non-strict mode

Variables declared with `var` or created by function declarations in non-strict mode **do not** have block scope. Variables introduced within a block are scoped to the containing function or script, and the effects of setting them persist beyond the block itself. For example:

```
var x = 1;
{
  var x = 2;
}
console.log(x); // 2
```

This logs 2 because the `var x` statement within the block is in the same scope as the `var x` statement before the block.

In non-strict code, function declarations inside blocks behave strangely. Do not use them.

Block scoping rules with let, const, class, or function declaration in strict mode

By contrast, identifiers declared with `let`, `const`, and `class` do have block scope:

```
let x = 1;
{
  let x = 2;
}
console.log(x); // 1
```

The `x = 2` is limited in scope to the block in which it was defined.

The same is true of `const`:

```
const c = 1;
{
  const c = 2;
}
console.log(c); // 1; does not throw SyntaxError
```

Note that the block-scoped `const c = 2` *does not* throw a `SyntaxError: Identifier 'c' has already been declared` because it can be declared uniquely within the block.

In strict mode, function declarations inside blocks are scoped to that block and are hoisted to the top of the block.

```
"use strict";

{
  foo(); // Logs "foo"
  function foo() {
    console.log("foo");
  }
}

foo(); // ReferenceError: foo is not defined
```

Examples

Using a block statement as the body of a for loop

A `for` loop accepts a single statement as its body.

```
for (let i = 0; i < 10; i++) console.log(i);
```

If you want to use more than one statement in the loop body, you can group them into one block statement:

```
for (let i = 0; i < 10; i++) {
  console.log(i);
  console.log(i ** 2);
}
```

Using a block statement to encapsulate data

`let` and `const` declarations are scoped to the containing block. This allows you to hide data from the global scope without wrapping it in a function.

```
let sector;
{
  // These variables are scoped to this block and are not
  // accessible after the block
  const angle = Math.PI / 3;
  const radius = 10;
  sector = {
    radius,
    angle,
    area: (angle / 2) * radius ** 2,
    perimeter: 2 * radius + angle * radius,
  };
}
console.log(sector);
// {
//   radius: 10,
//   angle: 1.0471975511965976,
//   area: 52.35987755982988,
//   perimeter: 30.471975511965976
// }
console.log(typeof radius); // "undefined"
```

<code>using</code> declarations in a block

You can declare variables with `using` or `await using` in a block, which causes the object stored in the variable to be disposed when control exits the block. For more information, see resource management.

```
{
  using reader1 = stream1.getReader();
  using reader2 = stream2.getReader();

  // do something with reader1 and reader2

  // Before we exit the block, reader1 and reader2 are automatically released
}
```

See also

- `while`
- `if...else`
- `let`

break

The **`break`** statement terminates the current loop or `switch` statement and transfers program control to the statement following the terminated statement. It can also be used to jump past a labeled statement when used within that labeled statement.

Try it

```
let i = 0;

while (i < 6) {
  if (i === 3) {
    break;
  }
  i += 1;
}

console.log(i);
// Expected output: 3
```

Syntax

```
break;
break label;
```

`label` Optional

Identifier associated with the label of the statement to break to. If the `break` statement is not nested within a loop or `switch`, then the label identifier is required.

Description

When `break;` is encountered, the program breaks out of the innermost `switch` or looping statement and continues executing the next statement after that.

When `break label;` is encountered, the program breaks out of the statement labeled with `label` and continues executing the next statement after that. The `break` statement needs to be nested within the referenced label. The labeled statement can be any statement (commonly a block statement); it does not have to be another loop statement.

A `break` statement, with or without a following label, cannot be used at the top level of a script, module, function's body, or static initialization block, even when the function or class is further contained within a loop.

Examples

break in while loop

The following function has a `break` statement that terminates the `while` loop when `i` is 3, and then returns the value `3 * x`.

```
function testBreak(x) {
  let i = 0;

  while (i < 6) {
    if (i === 3) {
      break;
    }
    i += 1;
  }

  return i * x;
}
```

break in switch statements

The following code has a `break` statement that terminates the `switch` statement when a case is matched and the corresponding code has run.

```
const food = "sushi";

switch (food) {
  case "sushi":
    console.log("Sushi is originally from Japan.");
    break;
  case "pizza":
    console.log("Pizza is originally from Italy.");
    break;
  default:
    console.log("I have never heard of that dish.");
    break;
}
```

break in labeled blocks

The following code uses `break` statements with labeled blocks. By using `break outerBlock`, control is transferred to the end of the block statement marked as `outerBlock`.

```
outerBlock: {
  innerBlock: {
```

```
    console.log("1");
    break outerBlock; // breaks out of both innerBlock and outerBlock
    console.log(":-("); // skipped
  }
  console.log("2"); // skipped
}
```

Unsyntactic break statements

A `break` statement must be nested within any label it references. The following code also uses `break` statements with labeled blocks, but generates a syntax error because its `break` statement references `block2` but it's not nested within `block2`.

```
block1: {
  console.log("1");
  break block2; // SyntaxError: label not found
}

block2: {
  console.log("2");
}
```

Syntax errors are also generated in the following code examples which use `break` statements within functions that are nested within a loop, or labeled block that the `break` statements are intended to break out of.

```
function testBreak(x) {
  let i = 0;

  while (i < 6) {
    if (i === 3) {
      (() => {
        break;
      })();
    }
    i += 1;
  }

  return i * x;
}

testBreak(1); // SyntaxError: Illegal break statement
```

```
block1: {
  console.log("1");
  (() => {
    break block1; // SyntaxError: Undefined label 'block1'
  })();
}
```

See also

- `continue`
- label
- `switch`

class

The **`class`** declaration creates a binding of a new class to a given name.

You can also define classes using the `class` expression.

Try it

```
class Polygon {
  constructor(height, width) {
    this.area = height * width;
  }
}

console.log(new Polygon(4, 3).area);
// Expected output: 12
```

Syntax

```
class name {
  // class body
}
class name extends otherName {
  // class body
}
```

Description

The class body of a class declaration is executed in strict mode. The `class` declaration is very similar to `let`:

- `class` declarations are scoped to blocks as well as functions.
- `class` declarations can only be accessed after the place of declaration is reached (see temporal dead zone). For this reason, `class` declarations are commonly regarded as non-hoisted (unlike function declarations).
- `class` declarations do not create properties on `globalThis` when declared at the top level of a script (unlike function declarations).
- `class` declarations cannot be redeclared by any other declaration in the same scope.

Outside the class body, `class` declarations can be re-assigned like `let`, but you should avoid doing so. Within the class body, the binding is constant like `const`.

```
class Foo {
  static {
    Foo = 1; // TypeError: Assignment to constant variable.
  }
}

class Foo2 {
  bar = (Foo2 = 1); // TypeError: Assignment to constant variable.
}

class Foo3 {}
Foo3 = 1;
console.log(Foo3); // 1
```

Examples

A class declaration

In the following example, we first define a class named `Rectangle`, then extend it to create a class named `FilledRectangle`.

Note that `super()`, used in the `constructor`, can only be used in constructors, and *must* be called *before* the `this` keyword can be used.

```
class Rectangle {
  constructor(height, width) {
    this.name = "Rectangle";
    this.height = height;
    this.width = width;
  }
}

class FilledRectangle extends Rectangle {
  constructor(height, width, color) {
    super(height, width);
    this.name = "Filled rectangle";
    this.color = color;
  }
}
```

See also

- `function`
- `class` expression
- Classes

const

The **`const`** declaration declares block-scoped local variables. The value of a constant can't be changed through reassignment using the assignment operator, but if a constant is an object, its properties can be added, updated, or removed.

Try it

```
const number = 42;

try {
  number = 99;
} catch (err) {
  console.log(err);
  // Expected output: TypeError: invalid assignment to const 'number'
  // (Note: the exact output may be browser-dependent)
}

console.log(number);
// Expected output: 42
```

Syntax

```
const name1 = value1;
const name1 = value1, name2 = value2;
const name1 = value1, name2 = value2, /* …, */ nameN = valueN;
```

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier or a destructuring binding pattern.

`valueN`

Initial value of the variable. It can be any legal expression.

Description

The `const` declaration is very similar to `let`:

- `const` declarations are scoped to blocks as well as functions.
- `const` declarations can only be accessed after the place of declaration is reached (see temporal dead zone). For this reason, `const` declarations are commonly regarded as non-hoisted.
- `const` declarations do not create properties on `globalThis` when declared at the top level of a script.
- `const` declarations cannot be redeclared by any other declaration in the same scope.
- `const` begins *declarations, not statements*. That means you cannot use a lone `const` declaration as the body of a block (which makes sense, since there's no way to access the variable).

  ```
  if (true) const a = 1; // SyntaxError: Lexical declaration cannot appear in a single-statement context
  ```

An initializer for a constant is required. You must specify its value in the same declaration. (This makes sense, given that it can't be changed later.)

```
const FOO; // SyntaxError: Missing initializer in const declaration
```

The `const` declaration creates an immutable reference to a value. It does *not* mean the value it holds is immutable — just that the variable identifier cannot be reassigned. For instance, in the case where the content is an object, this means the object's contents (e.g., its properties) can be altered. You should understand `const` declarations as "create a variable whose *identity* remains constant", not "whose *value* remains constant" — or, "create immutable bindings", not "immutable values".

Many style guides (including MDN's) recommend using `const` over `let` whenever a variable is not reassigned in its scope. This makes the intent clear that a variable's type (or value, in the case of a primitive) can never change. Others may prefer `let` for non-primitives that are mutated.

The list that follows the `const` keyword is called a *binding list* and is separated by commas, where the commas are *not* comma operators and the `=` signs are *not* assignment operators. Initializers of later variables can refer to earlier variables in the list.

Examples

Basic const usage

Constants can be declared with uppercase or lowercase, but a common convention is to use all-uppercase letters, especially for primitives because they are truly immutable.

```
// define MY_FAV as a constant and give it the value 7
const MY_FAV = 7;

console.log(`my favorite number is: ${MY_FAV}`);
```

```
// Re-assigning to a constant variable throws an error
MY_FAV = 20; // TypeError: Assignment to constant variable

// Redeclaring a constant throws an error
const MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
var MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
let MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
```

Block scoping

It's important to note the nature of block scoping.

```
const MY_FAV = 7;

if (MY_FAV === 7) {
  // This is fine because it's in a new block scope
  const MY_FAV = 20;
  console.log(MY_FAV); // 20

  // var declarations are not scoped to blocks so this throws an error
  var MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
}

console.log(MY_FAV); // 7
```

const in objects and arrays

`const` also works on objects and arrays. Attempting to overwrite the object throws an error "Assignment to constant variable".

```
const MY_OBJECT = { key: "value" };
MY_OBJECT = { OTHER_KEY: "value" };
```

However, object keys are not protected, so the following statement is executed without problem.

```
MY_OBJECT.key = "otherValue";
```

You would need to use `Object.freeze()` to make an object immutable.

The same applies to arrays. Assigning a new array to the variable throws an error "Assignment to constant variable".

```
const MY_ARRAY = [];
MY_ARRAY = ["B"];
```

Still, it's possible to push items into the array and thus mutate it.

```
MY_ARRAY.push("A"); // ["A"]
```

Declaration with destructuring

The left-hand side of each `=` can also be a binding pattern. This allows creating multiple variables at once.

```
const result = /(a+)(b+)(c+)/.exec("aaabcc");
const [, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

See also

- `var`
- `let`
- Constants in the JavaScript Guide

continue

The **`continue`** statement terminates execution of the statements in the current iteration of the current or labeled loop, and continues execution of the loop with the next iteration.

Try it

```
let text = "";

for (let i = 0; i < 10; i++) {
  if (i === 3) {
    continue;
  }
  text += i;
}

console.log(text);
// Expected output: "012456789"
```

Syntax

```
continue;
continue label;
```

`label` Optional

Identifier associated with the label of the statement.

Description

In contrast to the `break` statement, `continue` does not terminate the execution of the loop entirely, but instead:

- In a `while` or `do...while` loop, it jumps back to the condition.
- In a `for` loop, it jumps to the update expression.
- In a `for...in`, `for...of`, or `for await...of` loop, it jumps to the next iteration.

The `continue` statement can include an optional label that allows the program to jump to the next iteration of a labeled loop statement instead of the innermost loop. In this case, the `continue` statement needs to be nested within this labeled statement.

A `continue` statement, with or without a following label, cannot be used at the top level of a script, module, function's body, or static initialization block, even when the function or class is further contained within a loop.

Examples

Using continue with while

The following example shows a `while` loop that has a `continue` statement that executes when the value of `i` is 3. Thus, `n` takes on the values 1, 3, 7, and 12.

```
let i = 0;
let n = 0;

while (i < 5) {
```

```
  i++;

  if (i === 3) {
    continue;
  }

  n += i;
}
```

Using continue with a label

In the following example, a statement labeled `checkIAndJ` contains a statement labeled `checkJ`. If `continue` is encountered, the program continues at the top of the `checkJ` statement. Each time `continue` is encountered, `checkJ` reiterates until its condition returns false. When false is returned, the remainder of the `checkIAndJ` statement is completed.

If `continue` had a label of `checkIAndJ`, the program would continue at the top of the `checkIAndJ` statement.

```
let i = 0;
let j = 8;

checkIAndJ: while (i < 4) {
  console.log(`i: ${i}`);
  i += 1;

  checkJ: while (j > 4) {
    console.log(`j: ${j}`);
    j -= 1;

    if (j % 2 === 0) continue;
    console.log(`${j} is odd.`);
  }
  console.log(`i = ${i}`);
  console.log(`j = ${j}`);
}
```

Output:

```
i: 0

// start checkJ
j: 8
7 is odd.
j: 7
j: 6
5 is odd.
j: 5
// end checkJ

i = 1
j = 4

i: 1
i = 2
j = 4

i: 2
i = 3
j = 4

i: 3
i = 4
j = 4
```

Unsyntactic continue statements

`continue` cannot be used within loops across function boundaries.

```
for (let i = 0; i < 10; i++) {
  (() => {
    continue; // SyntaxError: Illegal continue statement: no surrounding iteration statement
  })();
}
```

When referencing a label, the labeled statement must contain the `continue` statement.

```
label: for (let i = 0; i < 10; i++) {
  console.log(i);
}

for (let i = 0; i < 10; i++) {
  continue label; // SyntaxError: Undefined label 'label'
}
```

The labeled statement must be a loop.

```
label: {
  for (let i = 0; i < 10; i++) {
    continue label; // SyntaxError: Illegal continue statement: 'label' does not denote an iteration statement
  }
}
```

See also

- `break`
- label

debugger

The **`debugger`** statement invokes any available debugging functionality, such as setting a breakpoint. If no debugging functionality is available, this statement has no effect.

Syntax

```
debugger;
```

Examples

Using the debugger statement

The following example shows code where a `debugger` statement has been inserted, to invoke a debugger (if one exists) when the function is called.

```
function potentiallyBuggyCode() {
  debugger;
  // do potentially buggy stuff to examine, step through, etc.
}
```

When the debugger is invoked, execution is paused at the `debugger` statement. It is like a breakpoint in the script source.

A browser with developer tools open to the debugger panel showing how execution is paused at the debugger statement to allow close inspection of variables, scopes, events, etc.

See also

- The Firefox JavaScript Debugger¶ in the Firefox source docs

do...while

The **`do...while`** statement creates a loop that executes a specified statement as long as the test condition evaluates to true. The condition is evaluated after executing the statement, resulting in the specified statement executing at least once.

Try it

```
let result = "";
let i = 0;

do {
  i += 1;
  result += i;
} while (i < 5);

console.log(result);
// Expected output: "12345"
```

Syntax

```
do
  statement
while (condition);
```

`statement`

> A statement that is executed at least once and re-executed as long as the condition evaluates to true. You can use a block statement to execute multiple statements.

`condition`

> An expression evaluated *after* each pass through the loop. If this condition evaluates to true, `statement` is re-executed. When condition evaluates to false, execution continues with the statement after the `do...while` loop.

Description

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and re-evaluates `condition`.

The `do...while` statement syntax requires a semicolon at the end, but the automatic semicolon insertion process may insert one for you if the lack of a semicolon results in invalid syntax.

Examples

Using do...while

In the following example, the `do...while` loop iterates at least once and reiterates until `i` is no longer less than 5.

```
let result = "";
let i = 0;
do {
  i += 1;
  result += `${i} `;
} while (i > 0 && i < 5);
// Despite i === 0 this will still loop as it starts off without the test

console.log(result);
```

Using false as do...while condition

Because the statement is always executed once, `do...while (false)` is the same as executing the statement itself. This is a common idiom in C-like languages, which allows you to use `break` to break out of branching logic early.

```
do {
  if (!user.loggedIn) {
    console.log("You are not logged in");
    break;
  }
  const friends = user.getFriends();
  if (!friends.length) {
    console.log("No friends found");
    break;
  }
  for (const friend of friends) {
    handleFriend(friend);
  }
} while (false);
// The rest of code
```

In JavaScript, there are some alternatives, such as using a labeled block statement with `break`:

```
handleFriends: {
  if (!user.loggedIn) {
    console.log("You are not logged in");
    break handleFriends;
  }
  const friends = user.getFriends();
  if (!friends.length) {
    console.log("No friends found");
    break handleFriends;
  }
  for (const friend of friends) {
    handleFriend(friend);
  }
}
```

Or using a function:

```
function handleFriends() {
  if (!user.loggedIn) {
    console.log("You are not logged in");
    return;
  }
  const friends = user.getFriends();
  if (!friends.length) {
    console.log("No friends found");
    return;
  }
  for (const friend of friends) {
    handleFriend(friend);
  }
}
```

Using an assignment as a condition

In some cases, it can make sense to use an assignment as a condition, such as this:

```
do {
  // …
} while ((match = regexp.exec(str)));
```

But when you do, there are readability tradeoffs. The `while` documentation has a Using an assignment as a condition section with our recommendations.

See also

- `while`
- `for`
- `break`
- `continue`

Empty statement

An **empty statement** is used to provide no statement, although the JavaScript syntax would expect one.

Try it

```
const array = [1, 2, 3];

// Assign all array values to 0
for (let i = 0; i < array.length; array[i++] = 0 /* empty statement */);

console.log(array);
// Expected output: Array [0, 0, 0]
```

Syntax

```
;
```

Description

The empty statement is a semicolon (`;`) indicating that no statement will be executed, even if JavaScript syntax requires one.

The opposite behavior, where you want multiple statements, but JavaScript only allows a single one, is possible using a block statement, which combines several statements into a single one.

Examples

Empty loop body

The empty statement is sometimes used with loop statements. See the following example with an empty loop body:

```
const arr = [1, 2, 3];

// Assign all array values to 0
for (let i = 0; i < arr.length; arr[i++] = 0) /* empty statement */ ;

console.log(arr);
// [0, 0, 0]
```

Unintentional usage

It is a good idea to comment *intentional* use of the empty statement, as it is not really obvious to distinguish from a normal semicolon.

In the following example, the usage is probably not intentional:

```
if (condition);      // Caution, this "if" does nothing!
  killTheUniverse(); // So this always gets executed!!!
```

See also

- Block statement

export

The **`export`** declaration is used to export values from a JavaScript module. Exported values can then be imported into other programs with the `import` declaration or dynamic import. The value of an imported binding is subject to change in the module that exports it — when a module updates the value of a binding that it exports, the update will be visible in its imported value.

In order to use the `export` declaration in a source file, the file must be interpreted by the runtime as a module. In HTML, this is done by adding `type="module"` to the `<script>` tag, or by being imported by another module. Modules are automatically interpreted in strict mode.

Syntax

```
// Exporting declarations
export let name1, name2/*, … */; // also var
export const name1 = 1, name2 = 2/*, … */; // also var, let
export function functionName() { /* … */ }
export class ClassName { /* … */ }
export function* generatorFunctionName() { /* … */ }
export const { name1, name2: bar } = o;
export const [ name1, name2 ] = array;

// Export list
export { name1, /* …, */ nameN };
export { variable1 as name1, variable2 as name2, /* …, */ nameN };
```

```
export { variable1 as "string name" };
export { name1 as default /*, … */ };

// Default exports
export default expression;
export default function functionName() { /* … */ }
export default class ClassName { /* … */ }
export default function* generatorFunctionName() { /* … */ }
export default function () { /* … */ }
export default class { /* … */ }
export default function* () { /* … */ }

// Aggregating modules
export * from "module-name";
export * as name1 from "module-name";
export { name1, /* …, */ nameN } from "module-name";
export { import1 as name1, import2 as name2, /* …, */ nameN } from "module-name";
export { default, /* …, */ } from "module-name";
export { default as name1 } from "module-name";
```

`nameN`

> Identifier to be exported (so that it can be imported via `import` in another script). If you use an alias with `as`, the actual exported name can be specified as a string literal, which may not be a valid identifier.

Description

Every module can have two different types of export, *named export* and *default export*. You can have multiple named exports per module but only one default export. Each type corresponds to one of the above syntax.

Named exports:

```
// export features declared elsewhere
export { myFunction2, myVariable2 };

// export individual features (can export var, let,
// const, function, class)
export let myVariable = Math.sqrt(2);
export function myFunction() {
  // …
}
```

After the `export` keyword, you can use `let`, `const`, and `var` declarations, as well as function or class declarations. You can also use the `export { name1, name2 }` syntax to export a list of names declared elsewhere. Note that `export {}` does not export an empty object — it's a no-op declaration that exports nothing (an empty name list).

You cannot use `export` on a `using` or `await using` declaration. You can, however, export a variable that was declared elsewhere using `using` or `await using`. Doing so is still strongly discouraged, because the variable is disposed as soon as the module finishes executing, causing all importers to receive a value that's already disposed.

```
export using resource1 = getResource(); // SyntaxError

// Allowed by syntax but discouraged
using resource2 = getResource();
export { resource2 };
```

Export declarations are not subject to temporal dead zone rules. You can declare that the module exports `x` before the name `x` itself is declared.

```
export { x };
const x = 1;
// This works, because `export` is only a declaration, but doesn't
// utilize the value of `x`.
```

Default exports:

```
// export feature declared elsewhere as default
export { myFunction as default };
// This is equivalent to:
export default myFunction;

// export individual features as default
export default function () { /* … */ }
export default class { /* … */ }
```

> **Note:** Names for export declarations must be distinct from each other. Having exports with duplicate names or using more than one `default` export will result in a `SyntaxError` and prevent the module from being evaluated.

The `export default` syntax allows any expression.

```
export default 1 + 1;
```

As a special case, functions and classes are exported as *declarations*, not expressions, and these declarations can be anonymous. This means functions will be hoisted.

```
// Works because `foo` is a function declaration,
// not a function expression
foo();

export default function foo() {
  console.log("Hi");
}

// It's still technically a declaration, but it's allowed
// to be anonymous
export default function () {
  console.log("Hi");
}
```

Named exports are useful when you need to export several values. When importing this module, named exports must be referred to by the exact same name (optionally renaming it with `as`), but the default export can be imported with any name. For example:

```
// file test.js
const k = 12;
export default k;
```

```
// some other file
import m from "./test"; // note that we have the freedom to use import m instead of import k, because k was default export

console.log(m); // 12
```

You can also rename named exports to avoid naming conflicts:

```
export { myFunction as function1, myVariable as variable };
```

You can rename a name to something that's not a valid identifier by using a string literal. For example:

```
export { myFunction as "my-function" };
```

Re-exporting / Aggregating

A module can also "relay" values exported from other modules without the hassle of writing two separate import/export statements. This is often useful when creating a single module concentrating various exports from various modules (usually called a "barrel module").

This can be achieved with the "export from" syntax:

```
export { default as function1, function2 } from "bar.js";
```

Which is comparable to a combination of import and export, except that `function1` and `function2` do not become available inside the current module:

```
import { default as function1, function2 } from "bar.js";

export { function1, function2 };
```

Most of the "import from" syntaxes have "export from" counterparts.

```
export { x } from "mod";
export { x as v } from "mod";
export * as ns from "mod";
```

There is also `export * from "mod"`, although there's no `import * from "mod"`. This re-exports all **named** exports from `mod` as the named exports of the current module, but the default export of `mod` is not re-exported. If there are two wildcard exports statements that implicitly re-export the same name, neither one is re-exported.

```
// -- mod1.js --
export const a = 1;

// -- mod2.js --
export const a = 3;

// -- barrel.js --
export * from "./mod1.js";
export * from "./mod2.js";

// -- main.js --
import * as ns from "./barrel.js";

console.log(ns.a); // undefined
```

Attempting to import the duplicate name directly will throw an error.

```
import { a } from "./barrel.js";
// SyntaxError: The requested module './barrel.js' contains conflicting star exports for name 'a'
```

The following is syntactically invalid despite its import equivalent:

```
export DefaultExport from "bar.js"; // Invalid
```

> **Note:** Due to the lack of this syntax, `import source` also has no re-exporting counterpart.

The correct way of doing this is to rename the export:

```
export { default as DefaultExport } from "bar.js";
```

The "export from" syntax allows the `as` token to be omitted, which makes the default export still re-exported as default export.

```
export { default, function2 } from "bar.js";
```

`export from` supports all features that `import` supports — for example, import attributes:

```
export { default } from "./data.json" with { type: "json" };
```

> **Note:** The `import defer * as ns` declaration has no re-exporting counterpart, despite `import * as ns` having one. Deferred re-exports are being developed in a separate proposal, because they can also avoid loading unused modules, which requires additional tree-shaking semantics. You can instead import and export in two separate statements, although this immediately loads the module.

Examples

Using named exports

In a module `my-module.js`, we could include the following code:

```
// module "my-module.js"
function cube(x) {
  return x * x * x;
}

const foo = Math.PI + Math.SQRT2;

const graph = {
  options: {
    color: "white",
    thickness: "2px",
  },
  draw() {
    console.log("From graph draw function");
  },
};

export { cube, foo, graph };
```

Then in the top-level module included in your HTML page, we could have:

```
import { cube, foo, graph } from "./my-module.js";

graph.options = {
  color: "blue",
  thickness: "3px",
};

graph.draw(); // Logs "From graph draw function"
console.log(cube(3)); // 27
console.log(foo); // 4.555806215962888
```

It is important to note the following:

- You need to include this script in your HTML with a `<script>` element of `type="module"`, so that it gets recognized as a module and dealt with appropriately.
- You can't run JS modules via a `file://` URL — you'll get CORS errors. You need to run it via an HTTP server.

Using the default export

If we want to export a single value representing an entire module, we could use a default export:

```
// module "cube.js"

export default function cube(x) {
  return x * x * x;
}
```

Then, in another script, it is straightforward to import the default export:

```
import cube from "./cube.js";
```

```
console.log(cube(3)); // 27
```

Using export from

Let's take an example where we have the following hierarchy:

- `childModule1.js`: exporting `myFunction` and `myVariable`
- `childModule2.js`: exporting `MyClass`
- `parentModule.js`: acting as an aggregator (and doing nothing else)
- top level module: consuming the exports of `parentModule.js`

This is what it would look like using code snippets:

```
// In childModule1.js
function myFunction() {
  console.log("Hello!");
}
const myVariable = 1;
export { myFunction, myVariable };
```

```
// In childModule2.js
class MyClass {
  constructor(x) {
    this.x = x;
  }
}

export { MyClass };
```

```
// In parentModule.js
// Only aggregating the exports from childModule1 and childModule2
// to re-export them
export { myFunction, myVariable } from "childModule1.js";
export { MyClass } from "childModule2.js";
```

```
// In top-level module
// We can consume the exports from a single module since parentModule
// "collected"/"bundled" them in a single source
import { myFunction, myVariable, MyClass } from "parentModule.js";
```

See also

- `import`
- JavaScript modules guide
- ES6 in Depth: Modules on hacks.mozilla.org (2015)
- ES modules: A cartoon deep-dive on hacks.mozilla.org (2018)

Expression statement

An **expression statement** is an expression used in a place where a statement is expected. The expression is evaluated and its result is discarded — therefore, it makes sense only for expressions that have side effects, such as executing a function or updating a variable.

Syntax

```
expression;
```

`expression`

An arbitrary expression to be evaluated. There are certain expressions that may be ambiguous with other statements and are thus forbidden.

Description

Apart from the dedicated statement syntaxes, you can also use almost any expression as a statement on its own. The expression statement syntax requires a semicolon at the end, but the automatic semicolon insertion process may insert one for you if the lack of a semicolon results in invalid syntax.

Because the expression is evaluated and then discarded, the result of the expression is not available. Therefore, the expression must have some side effect for it to be useful. Expression statements are commonly:

- Function calls (`console.log("Hello");`, `[1, 2, 3].forEach((i) => console.log(i));`)
- Tagged template literals
- Assignment expressions, including compound assignments
- Increment and decrement operators
- `delete`
- `import()`
- `yield` and `yield*`

Others may also have side effects if they invoke getters or trigger type coercions.

Forbidden expressions

In order for an expression to be used as a statement, it must not be ambiguous with other statement syntaxes. Therefore, the expression must not start with any of the following tokens:

- `function`: which would be a `function` declaration or `function*` declaration, not a `function` expression or `function*` expression
- `async function`: which would be an `async function` declaration or `async function*` declaration, not an `async function` expression or `async function*` expression
- `class`: which would be a `class` declaration, not a `class` expression
- `let[`: which would be a `let` declaration with array destructuring, not a property accessor on a variable called `let` (`let` can only be an identifier in non-strict mode)
- `{`: which would be a block statement, not an object literal

Therefore, all of the following are invalid:

```
function foo() {
  console.log("foo");
}(); // SyntaxError: Unexpected token '('

// For some reason, you have a variable called `let`
var let = [1, 2, 3];
let[0] = 4; // SyntaxError: Invalid destructuring assignment target

{
  foo: 1,
  bar: 2, // SyntaxError: Unexpected token ':'
};
```

More dangerously, sometimes the code happens to be valid syntax, but is not what you intend.

```
// For some reason, you have a variable called `let`
var let = [1, 2, 3];

function setIndex(index, value) {
  if (index >= 0) {
    // Intend to assign to the array `let`, but instead creates an extra variable!
    let[index] = value;
  }
}

setIndex(0, [1, 2]);
console.log(let); // [1, 2, 3]

// This is not an object literal, but a block statement,
// where `foo` is a label and `1` is an expression statement.
// This often happens in the console
{ foo: 1 };
```

To avoid these problems, you can use parentheses, so that the statement is unambiguously an expression statement.

```
(function foo() {
  console.log("foo");
})();
```

Examples

Avoiding control flow statements

You can avoid almost all use of control flow statements using expression statements. For example, `if...else` can be replaced with ternary operators and logical operators. Iterative statements like `for` or `for...of` can be replaced with array methods.

```
// Using control flow statements
function range(start, end) {
  if (start > end) {
    [start, end] = [end, start];
  }
  const result = [];
  for (let i = start; i < end; i++) {
    result.push(i);
  }
  return result;
}

// Using expression statements
function range2(start, end) {
  start > end && ([start, end] = [end, start]);
  return Array.from({ length: end - start }, (_, i) => start + i);
}
```

> **Warning:** This only demonstrates a capability of the language. Excessive use of expression statements as a substitute for control-flow statements can make code much less readable.

See also

- Statements and declarations
- Expressions and operators

for

The **`for`** statement creates a loop that consists of three optional expressions, enclosed in parentheses and separated by semicolons, followed by a statement (usually a block statement) to be executed in the loop.

Try it

```
let str = "";

for (let i = 0; i < 9; i++) {
  str += i;
}

console.log(str);
// Expected output: "012345678"
```

Syntax

```
for (initialization; condition; afterthought)
  statement
```

`initialization` Optional

> An expression (including assignment expressions) or variable declaration evaluated once before the loop begins. Typically used to initialize a counter variable. This expression may optionally declare new variables with `var` or `let` keywords. Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for` loop is in. Variables declared with `let` are local to the statement.
>
> The result of this expression is discarded.

`condition` Optional

> An expression to be evaluated before each loop iteration. If this expression evaluates to true, `statement` is executed. If the expression evaluates to false, execution exits the loop and goes to the first statement after the `for` construct.
>
> This conditional test is optional. If omitted, the condition always evaluates to true.

`afterthought` Optional

> An expression to be evaluated at the end of each loop iteration. This occurs before the next evaluation of `condition`. Generally used to update or increment the counter variable.

`statement`

> A statement that is executed as long as the condition evaluates to true. You can use a block statement to execute multiple statements. To execute no statement within the loop, use an empty statement (`;`).

Description

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and re-evaluates `afterthought` then `condition`.

Examples

Using for

The following `for` statement starts by declaring the variable `i` and initializing it to `0`. It checks that `i` is less than nine, performs the two succeeding statements, and increments `i` by 1 after each pass through the loop.

```
for (let i = 0; i < 9; i++) {
  console.log(i);
  // more statements
}
```

Initialization block syntax

The initialization block accepts both expressions and variable declarations. However, expressions cannot use the `in` operator unparenthesized, because that is ambiguous with a `for...in` loop.

```
for (let i = "start" in window ? window.start : 0; i < 9; i++) {
  console.log(i);
}
// SyntaxError: 'for-in' loop variable declaration may not have an initializer.
```

```
// Parenthesize the whole initializer
for (let i = ("start" in window ? window.start : 0); i < 9; i++) {
  console.log(i);
}
```

```
// Parenthesize the `in` expression
for (let i = ("start" in window) ? window.start : 0; i < 9; i++) {
  console.log(i);
}
```

Optional for expressions

All three expressions in the head of the `for` loop are optional. For example, it is not required to use the `initialization` block to initialize variables:

```
let i = 0;
for (; i < 9; i++) {
  console.log(i);
  // more statements
}
```

Like the `initialization` block, the `condition` part is also optional. If you are omitting this expression, you must make sure to break the loop in the body in order to not create an infinite loop.

```
for (let i = 0; ; i++) {
  console.log(i);
  if (i > 3) break;
  // more statements
}
```

You can also omit all three expressions. Again, make sure to use a `break` statement to end the loop and also modify (increase) a variable, so that the condition for the break statement is true at some point.

```
let i = 0;

for (;;) {
  if (i > 3) break;
  console.log(i);
  i++;
}
```

However, in the case where you are not fully using all three expression positions — especially if you are not declaring variables with the first expression but mutating something in the upper scope — consider using a `while` loop instead, which makes the intention clearer.

```
let i = 0;

while (i <= 3) {
  console.log(i);
  i++;
}
```

Lexical declarations in the initialization block

Declaring a variable within the initialization block has important differences from declaring it in the upper scope, especially when creating a closure within the loop body. For example, for the code below:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

It logs `0`, `1`, and `2`, as expected. However, if the variable is defined in the upper scope:

```
let i = 0;
for (; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

It logs `3`, `3`, and `3`. The reason is that each `setTimeout` creates a new closure that closes over the `i` variable, but if the `i` is not scoped to the loop body, all closures will reference the same variable when they eventually get called — and due to the asynchronous nature of `setTimeout()`, it will happen after the loop has already exited, causing the value of `i` in all queued callbacks' bodies to have the value of `3`.

This also happens if you use a `var` statement as the initialization, because variables declared with `var` are only function-scoped, but not lexically scoped (i.e., they can't be scoped to the loop body).

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
// Logs 3, 3, 3
```

The scoping effect of the initialization block can be understood as if the declaration happens within the loop body, but just happens to be accessible within the `condition` and `afterthought` parts. More precisely, `let` declarations are special-cased by `for` loops — if `initialization` is a `let` declaration, then every time, after the loop body is evaluated, the following happens:

1. A new lexical scope is created with new `let`-declared variables.
2. The binding values from the last iteration are used to re-initialize the new variables.
3. `afterthought` is evaluated in the new scope.

So re-assigning the new variables within `afterthought` does not affect the bindings from the previous iteration.

A new lexical scope is also created after `initialization`, just before `condition` is evaluated for the first time. These details can be observed by creating closures, which allow you to get hold of a binding at any particular point. For example, in this code a closure created within the `initialization` section does not get updated by re-assignments of `i` in the `afterthought`:

```
for (let i = 0, getI = () => i; i < 3; i++) {
  console.log(getI());
}
// Logs 0, 0, 0
```

This does not log "0, 1, 2", like what would happen if `getI` is declared in the loop body. This is because `getI` is not re-evaluated on each iteration — rather, the function is created once and closes over the `i` variable, which refers to the variable declared when the loop was first initialized. Subsequent updates to the value of `i` actually create new variables called `i`, which `getI` does not see. A way to fix this is to re-compute `getI` every time `i` updates:

```
for (let i = 0, getI = () => i; i < 3; i++, getI = () => i) {
  console.log(getI());
}
// Logs 0, 1, 2
```

The `i` variable inside the `initialization` is distinct from the `i` variable inside every iteration, including the first. So, in this example, `getI` returns 0, even though the value of `i` inside the iteration is incremented beforehand:

```
for (let i = 0, getI = () => i; i < 3;) {
  i++;
  console.log(getI());
}
// Logs 0, 0, 0
```

In fact, you can capture this initial binding of the `i` variable and re-assign it later, and this updated value will not be visible to the loop body, which sees the next new binding of `i`.

```
for (
  let i = 0, getI = () => i, incrementI = () => i++;
  getI() < 3;
  incrementI()
) {
  console.log(i);
}
// Logs 0, 0, 0
```

This logs "0, 0, 0", because the `i` variable in each loop evaluation is actually a separate variable, but `getI` and `incrementI` both read and write the *initial* binding of `i`, not what was subsequently declared.

Using for without a body

The following `for` cycle calculates the offset position of a node in the `afterthought` section, and therefore it does not require the use of a `statement` section, a semicolon is used instead.

```
function showOffsetPos(id) {
  let left = 0;
  let top = 0;
  for (
    let itNode = document.getElementById(id); // initialization
    itNode; // condition
    left += itNode.offsetLeft,
      top += itNode.offsetTop,
      itNode = itNode.offsetParent // afterthought
  ); // semicolon

  console.log(
    `Offset position of "${id}" element:
left: ${left}px;
top: ${top}px;`,
  );
}

showOffsetPos("content");

// Logs:
// Offset position of "content" element:
// left: 0px;
// top: 153px;
```

Note that the semicolon after the `for` statement is mandatory, because it stands as an empty statement. Otherwise, the `for` statement acquires the following `console.log` line as its `statement` section, which makes the `log` execute multiple times.

Using for with two iterating variables

You can create two counters that are updated simultaneously in a for loop using the comma operator. Multiple `let` and `var` declarations can also be joined with commas.

```
const arr = [1, 2, 3, 4, 5, 6];
for (let l = 0, r = arr.length - 1; l < r; l++, r--) {
  console.log(arr[l], arr[r]);
}
// 1 6
// 2 5
// 3 4
```

See also

- Empty statement
- `break`
- `continue`
- `while`
- `do...while`
- `for...in`
- `for...of`

for await...of

The **`for await...of`** statement creates a loop iterating over async iterable objects as well as sync iterables. This statement can only be used in contexts where `await` can be used, which includes inside an async function body and in a module.

Try it

```
async function* foo() {
  yield 1;
  yield 2;
}

(async () => {
  for await (const num of foo()) {
    console.log(num);
    // Expected output: 1

    break; // Closes iterator, triggers return
  }
})();
```

Syntax

```
for await (variable of iterable)
  statement
```

`variable`

> Receives a value from the sequence on each iteration. May be either a declaration with `const`, `let`, or `var`, or an assignment target (e.g., a previously declared variable, an object property, or a destructuring pattern). Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for await...of` loop is in.

`iterable`

> An async iterable or sync iterable. The source of the sequence of values on which the loop operates.

`statement`

> A statement to be executed on every iteration. May reference `variable`. You can use a block statement to execute multiple statements.

Description

When a `for await...of` loop iterates over an iterable, it first gets the iterable's `[Symbol.asyncIterator]()` method and calls it, which returns an async iterator. If the `@asyncIterator` method does not exist, it then looks for a `[Symbol.iterator]()` method, which returns a sync iterator. The sync iterator returned is then wrapped into an async iterator by wrapping every object returned from the `next()`, `return()`, and `throw()` methods into a resolved or rejected promise, with the `value` property resolved if it's also a promise. The loop then repeatedly calls the final async iterator's `next()` method and awaits the returned promise, producing the sequence of values to be assigned to `variable`.

A `for await...of` loop exits when the iterator has completed (the awaited `next()` result is an object with `done: true`). Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and goes to the next iteration of the loop.

If the `for await...of` loop exited early (e.g., a `break` statement is encountered or an error is thrown), the `return()` method of the iterator is called to perform any cleanup. The returned promise is awaited before the loop exits.

`for await...of` generally functions the same as the `for...of` loop and shares many of the same syntax and semantics. There are a few differences:

- `for await...of` works on both sync and async iterables, while `for...of` only works on sync iterables.
- `for await...of` can only be used in contexts where `await` can be used, which includes inside an async function body and in a module. Even when the iterable is sync, the loop still awaits the return value for every iteration, leading to slower execution due to repeated promise unwrapping.
- If the `iterable` is a sync iterable that yields promises, `for await...of` would produce a sequence of resolved values, while `for...of` would produce a sequence of promises.
- For `for await...of`, the `variable` can be the identifier `async` (e.g., `for await (async of foo)`); `for...of` forbids this case.

Like `for...of`, if you use a `using` declaration, then the variable cannot be called `of`:

```
for await (using of of []); // SyntaxError
```

This is to avoid syntax ambiguity with the valid code `for await (using of [])`, before `using` was introduced.

Examples

Iterating over async iterables

You can also iterate over an object that explicitly implements async iterable protocol:

```
const LIMIT = 3;

const asyncIterable = {
  [Symbol.asyncIterator]() {
    let i = 0;
    return {
      next() {
        const done = i === LIMIT;
        const value = done ? undefined : i++;
        return Promise.resolve({ value, done });
      },
      return() {
        // This will be reached if the consumer called 'break' or 'return' early in the loop.
        return { done: true };
      },
    };
  },
};

(async () => {
  for await (const num of asyncIterable) {
    console.log(num);
  }
})();
// 0
// 1
// 2
```

Iterating over async generators

Since the return values of async generator functions conform to the async iterable protocol, they can be looped using `for await...of`.

```
async function* asyncGenerator() {
  let i = 0;
  while (i < 3) {
    yield i++;
  }
}

(async () => {
  for await (const num of asyncGenerator()) {
    console.log(num);
  }
})();
// 0
// 1
// 2
```

For a more concrete example of iterating over an async generator using `for await...of`, consider iterating over data from an API.

This example first creates an async iterable for a stream of data, then uses it to find the size of the response from the API.

```
async function* streamAsyncIterable(stream) {
  const reader = stream.getReader();
  try {
    while (true) {
      const { done, value } = await reader.read();
      if (done) return;
      yield value;
```

```
    }
  } finally {
    reader.releaseLock();
  }
}

// Fetches data from URL and calculates response size using the async generator.
async function getResponseSize(url) {
  const response = await fetch(url);
  // Will hold the size of the response, in bytes.
  let responseSize = 0;
  // The for-await-of loop. Async iterates over each portion of the response.
  for await (const chunk of streamAsyncIterable(response.body)) {
    // Incrementing the total response length.
    responseSize += chunk.length;
  }

  console.log(`Response Size: ${responseSize} bytes`); // "Response Size: 1071472"
  return responseSize;
}
getResponseSize("https://jsonplaceholder.typicode.com/photos");
```

Iterating over sync iterables and generators

`for await...of` loop also consumes sync iterables and generators. In that case it internally awaits emitted values before assign them to the loop control variable.

```
function* generator() {
  yield 0;
  yield 1;
  yield Promise.resolve(2);
  yield Promise.resolve(3);
  yield 4;
}

(async () => {
  for await (const num of generator()) {
    console.log(num);
  }
})();
// 0
// 1
// 2
// 3
// 4

// compare with for-of loop:

for (const numOrPromise of generator()) {
  console.log(numOrPromise);
}
// 0
// 1
// Promise { 2 }
// Promise { 3 }
// 4
```

If a sync generator yields a rejected promise, `for await...of` calls the generator's `return()` method before throwing the rejection reason, allowing `finally` blocks within that generator to run.

```
function* generatorWithRejectedPromises() {
  try {
    yield 0;
    yield 1;
    yield Promise.resolve(2);
    yield Promise.reject(new Error("failed"));
    yield 4;
    throw new Error("throws");
  } finally {
    console.log("called finally");
  }
}

(async () => {
  try {
    for await (const num of generatorWithRejectedPromises()) {
      console.log(num);
    }
  } catch (e) {
    console.log("caught", e);
  }
})();
// 0
// 1
// 2
```

```
// called finally
// caught Error: failed
```

See also

- `Symbol.asyncIterator`
- `for...of`

for...in

The **`for...in`** statement iterates over all enumerable string properties of an object (ignoring properties keyed by symbols), including inherited enumerable properties.

Try it

```
const object = { a: 1, b: 2, c: 3 };

for (const property in object) {
  console.log(`${property}: ${object[property]}`);
}

// Expected output:
// "a: 1"
// "b: 2"
// "c: 3"
```

Syntax

```
for (variable in object)
  statement
```

Parameters

`variable`

> Receives a string property name on each iteration. May be either a declaration with `const`, `let`, or `var`, or an assignment target (e.g., a previously declared variable, an object property, or a destructuring pattern). Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for...in` loop is in.

`object`

> Object whose non-symbol enumerable properties are iterated over.

`statement`

> A statement to be executed on every iteration. May reference `variable`. You can use a block statement to execute multiple statements.

Description

The loop will iterate over all enumerable properties of the object itself and those the object inherits from its prototype chain (properties of nearer prototypes take precedence over those of prototypes further away from the object in its prototype chain).

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and goes to the next iteration of the loop.

A `for...in` loop only iterates over enumerable, non-symbol properties. Objects created from built–in constructors like `Array` and `Object` have inherited non–enumerable properties from `Array.prototype` and `Object.prototype`, such as `Array`'s `indexOf()` method or `Object`'s `toString()` method, which will not be visited in the `for...in` loop.

The traversal order, as of modern ECMAScript specification, is well-defined and consistent across implementations. Within each component of the prototype chain, all non-negative integer keys (those that can be array indices) will be traversed first in ascending order by value, then other string keys in ascending chronological order of property creation.

The `variable` part of `for...in` accepts anything that can come before the `=` operator. You can use `const` to declare the variable as long as it's not reassigned within the loop body (it can change between iterations, because those are two separate variables). Otherwise, you can use `let`. You can use destructuring to assign multiple local variables, or use a property accessor like `for (x.y in iterable)` to assign the value to an object property. However, `using` and `await using` are not allowed, because the variable is always a string or symbol while these declarations require a disposable object.

A legacy syntax allows `var` declarations of the loop variable to have an initializer. This throws a syntax error in strict mode and is ignored in non–strict mode.

Deleted, added, or modified properties

`for...in` visits property keys in the following fashion:

1. It first gets all own string keys of the current object, in a fashion very similar to `Object.getOwnPropertyNames()`.
2. For each key, if no string with the same value has ever been visited, the property descriptor is retrieved and the property is only visited if it is enumerable. However, this property string will be marked as visited even if it's not enumerable.
3. Then, the current object is replaced with its prototype, and the process is repeated.

This means:

- Any property added to the currently visited object during iteration will not be visited, because all own properties of the current object have already been saved beforehand.
- If multiple objects in the prototype chain have a property with the same name, only the first one will be considered, and it is only visited if it's enumerable. If it is non-enumerable, no other properties with the same name further up the prototype chain will be visited, even if they are enumerable.

In general, it is best not to add, modify, or remove properties from the object during iteration, other than the property currently being visited. The spec explicitly allows the implementation to not follow the algorithm above in one of the following cases:

- The object's prototype chain is modified during iteration.
- A property is deleted from the object or its prototype chain during iteration.
- A property is added to the object's prototype chain during iteration.
- A property's enumerability is changed during iteration.

In these cases, implementations may behave differently from what you may expect, or even from each other.

Array iteration and for...in

Array indexes are just enumerable properties with integer names and are otherwise identical to general object properties. The `for...in` loop will traverse all integer keys before traversing other keys, and in strictly increasing order, making the behavior of `for...in` close to normal array iteration. However, the `for...in` loop will return all enumerable properties, including those with non–integer names and those that are inherited. Unlike `for...of`, `for...in` uses property enumeration instead of the array's iterator. In sparse arrays, `for...of` will visit the empty slots, but `for...in` will not.

It is better to use a `for` loop with a numeric index, `Array.prototype.forEach()`, or the `for...of` loop, because they will return the index as a number instead of a string, and also avoid non-index properties.

Iterating over own properties only

If you only want to consider properties attached to the object itself, and not its prototypes, you can use one of the following techniques:

- `Object.keys()`
- `Object.getOwnPropertyNames()`

`Object.keys` will return a list of enumerable own string properties, while `Object.getOwnPropertyNames` will also contain non-enumerable ones.

Many JavaScript style guides and linters recommend against the use of `for...in`, because it iterates over the entire prototype chain which is rarely what one wants, and may be a confusion with the more widely-used `for...of` loop. `for...in` is most practically used for debugging purposes, being an easy way to check the properties of an object (by outputting to the console or otherwise). In situations where objects are used as ad hoc key-value pairs, `for...in` allows you check if any of those keys hold a particular value.

Examples

Using for...in

The `for...in` loop below iterates over all of the object's enumerable, non-symbol properties and logs a string of the property names and their values.

```
const obj = { a: 1, b: 2, c: 3 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}

// Logs:
// "obj.a = 1"
// "obj.b = 2"
// "obj.c = 3"
```

Iterating own properties

The following function illustrates the use of `Object.hasOwn()`: the inherited properties are not displayed.

```
const triangle = { a: 1, b: 2, c: 3 };

function ColoredTriangle() {
  this.color = "red";
}

ColoredTriangle.prototype = triangle;

const obj = new ColoredTriangle();

for (const prop in obj) {
  if (Object.hasOwn(obj, prop)) {
    console.log(`obj.${prop} = ${obj[prop]}`);
  }
}

// Logs:
// "obj.color = red"
```

Concurrent modification

> **Warning:** You should not write code like this yourself. It is only included here to illustrate the behavior of `for...in` in some corner cases.

Properties added to the current object during iteration are never visited:

```
const obj = { a: 1, b: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  obj.c = 3;
}

// Logs:
// obj.a = 1
// obj.b = 2
```

Shadowed properties are only visited once:

```
const proto = { a: 1 };
const obj = { __proto__: proto, a: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}

// Logs:
// obj.a = 2

Object.defineProperty(obj, "a", { enumerable: false });

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}
// Logs nothing, because the first "a" property visited is non-enumerable.
```

In addition, consider the following cases, where the behavior is unspecified, and implementations tend to diverge from the specified algorithm:

Changing the prototype during iteration:

```
const obj = { a: 1, b: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  Object.setPrototypeOf(obj, { c: 3 });
}
```

Deleting a property during iteration:

```
const obj = { a: 1, b: 2, c: 3 };

// Deleting a property before it is visited
for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  delete obj.c;
}

const obj2 = { a: 1, b: 2, c: 3 };

// Deleting a property after it is visited
for (const prop in obj2) {
  console.log(`obj2.${prop} = ${obj2[prop]}`);
  delete obj2.a;
}
```

Enumerable properties added to the prototype during iteration:

```
const proto = {};
const obj = { __proto__: proto, a: 1, b: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  proto.c = 3;
}
```

Changing the enumerability of a property during iteration:

```
const obj = { a: 1, b: 2, c: 3 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  Object.defineProperty(obj, "c", { enumerable: false });
}
```

See also

- `for...of`
- `for`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `Object.hasOwn()`
- `Array.prototype.forEach()`

for...of

The **`for...of`** statement executes a loop that operates on a sequence of values sourced from an iterable object. Iterable objects include instances of built-ins such as `Array`, `String`, `TypedArray`, `Map`, `Set`, `NodeList` (and other DOM collections), as well as the `arguments` object, generators produced by generator functions, and user-defined iterables.

Try it

```
const array = ["a", "b", "c"];

for (const element of array) {
  console.log(element);
}

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

```
for (variable of iterable)
  statement
```

`variable`

> Receives a value from the sequence on each iteration. May be either a declaration with `const`, `let`, `var`, `using`, `await using`, or an assignment target (e.g., a previously declared variable, an object property, or a destructuring pattern). Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for...of` loop is in.

`iterable`

> An iterable object. The source of the sequence of values on which the loop operates.

`statement`

> A statement to be executed on every iteration. May reference `variable`. You can use a block statement to execute multiple statements.

Description

A `for...of` loop operates on the values sourced from an iterable one by one in sequential order. Each operation of the loop on a value is called an *iteration*, and the loop is said to *iterate over the iterable*. Each iteration executes statements that may refer to the current sequence value.

When a `for...of` loop iterates over an iterable, it first calls the iterable's `[Symbol.iterator]()` method, which returns an iterator, and then repeatedly calls the resulting iterator's `next()` method to produce the sequence of values to be assigned to `variable`.

A `for...of` loop exits when the iterator has completed (the `next()` result is an object with `done: true`). Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and goes to the next iteration of the loop.

If the `for...of` loop exited early (e.g., a `break` statement is encountered or an error is thrown), the `return()` method of the iterator is called to perform any cleanup.

The `variable` part of `for...of` accepts anything that can come before the `=` operator. You can use `const` to declare the variable as long as it's not reassigned within the loop body (it can change between iterations, because those are two separate variables). Otherwise, you can use `let`.

```
const iterable = [10, 20, 30];

for (let value of iterable) {
  value += 1;
  console.log(value);
}
// 11
// 21
// 31
```

> **Note:** Each iteration creates a new variable. Reassigning the variable inside the loop body does not affect the original value in the iterable (an array, in this case).

Variables declared using the `using` or `await using` declaration are disposed every time a loop iteration is done (and `await using` causes an implicit `await` at the end of the iteration). However, if the loop early-exits, any values left in the iterator that haven't been visited are not disposed (although the current value is).

```
const resources = [dbConnection1, dbConnection2, dbConnection3];

for (using dbConnection of resources) {
  dbConnection.query("...");
  // dbConnection is disposed here
}
```

You can use destructuring to assign multiple local variables, or use a property accessor like `for (x.y of iterable)` to assign the value to an object property.

However, a special rule forbids using `async` as the variable name. This is invalid syntax:

```
let async;
for (async of [1, 2, 3]); // SyntaxError: The left-hand side of a for-of loop may not be 'async'.
```

This is to avoid syntax ambiguity with the valid code `for (async of => {};;)`, which is a `for` loop.

Similarly, if you use the `using` declaration, then the variable cannot be called `of`:

```
for (using of of []); // SyntaxError
```

This is to avoid syntax ambiguity with the valid code `for (using of [])`, before `using` was introduced.

Examples

Iterating over an Array

```
const iterable = [10, 20, 30];

for (const value of iterable) {
  console.log(value);
}
// 10
// 20
// 30
```

Iterating over a string

Strings are iterated by Unicode code points.

```
const iterable = "boo";

for (const value of iterable) {
  console.log(value);
}
// "b"
// "o"
// "o"
```

Iterating over a TypedArray

```
const iterable = new Uint8Array([0x00, 0xff]);

for (const value of iterable) {
  console.log(value);
}
// 0
// 255
```

Iterating over a Map

```
const iterable = new Map([
  ["a", 1],
  ["b", 2],
  ["c", 3],
]);

for (const entry of iterable) {
  console.log(entry);
}
// ['a', 1]
// ['b', 2]
// ['c', 3]

for (const [key, value] of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over a Set

```
const iterable = new Set([1, 1, 2, 2, 3, 3]);

for (const value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over the arguments object

You can iterate over the `arguments` object to examine all parameters passed into a function.

```
function foo() {
  for (const value of arguments) {
    console.log(value);
  }
}

foo(1, 2, 3);
// 1
// 2
// 3
```

Iterating over a NodeList

The following example adds a `read` class to paragraphs that are direct descendants of the `<article>` element by iterating over a `NodeList` DOM collection.

```
const articleParagraphs = document.querySelectorAll("article > p");
for (const paragraph of articleParagraphs) {
  paragraph.classList.add("read");
}
```

Iterating over a user-defined iterable

Iterating over an object with a `[Symbol.iterator]()` method that returns a custom iterator:

```
const iterable = {
  [Symbol.iterator]() {
    let i = 1;
    return {
      next() {
        if (i <= 3) {
          return { value: i++, done: false };
        }
        return { value: undefined, done: true };
      },
    };
  },
};

for (const value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over an object with a `[Symbol.iterator]()` generator method:

```
const iterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};

for (const value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterable iterators (iterators with a `[Symbol.iterator]()` method that returns `this`) are a fairly common technique to make iterators usable in syntaxes expecting iterables, such as `for...of`.

```
let i = 1;

const iterator = {
  next() {
    if (i <= 3) {
```

```
      return { value: i++, done: false };
    }
    return { value: undefined, done: true };
  },
  [Symbol.iterator]() {
    return this;
  },
};

for (const value of iterator) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over a generator

```
function* source() {
  yield 1;
  yield 2;
  yield 3;
}

const generator = source();

for (const value of generator) {
  console.log(value);
}
// 1
// 2
// 3
```

Early exiting

Execution of the `break` statement in the first loop causes it to exit early. The iterator is not finished yet, so the second loop will continue from where the first one stopped at.

```
const source = [1, 2, 3];

const iterator = source[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
  if (value === 1) {
    break;
  }
  console.log("This string will not be logged.");
}
// 1

// Another loop using the same iterator
// picks up where the last loop left off.
for (const value of iterator) {
  console.log(value);
}
// 2
// 3

// The iterator is used up.
// This loop will execute no iterations.
for (const value of iterator) {
  console.log(value);
}
// [No output]
```

Generators implement the `return()` method, which causes the generator function to early return when the loop exits. This makes generators not reusable between loops.

```
function* source() {
  yield 1;
  yield 2;
  yield 3;
}

const generator = source();

for (const value of generator) {
  console.log(value);
  if (value === 1) {
    break;
  }
  console.log("This string will not be logged.");
}
```

```
// 1

// The generator is used up.
// This loop will execute no iterations.
for (const value of generator) {
  console.log(value);
}
// [No output]
```

Difference between for...of and for...in

Both `for...in` and `for...of` statements iterate over something. The main difference between them is in what they iterate over.

The `for...in` statement iterates over the enumerable string properties of an object, while the `for...of` statement iterates over values that the iterable object defines to be iterated over.

The following example shows the difference between a `for...of` loop and a `for...in` loop when used with an `Array`.

```
Object.prototype.objCustom = function () {};
Array.prototype.arrCustom = function () {};

const iterable = [3, 5, 7];
iterable.foo = "hello";

for (const i in iterable) {
  console.log(i);
}
// "0", "1", "2", "foo", "arrCustom", "objCustom"

for (const i in iterable) {
  if (Object.hasOwn(iterable, i)) {
    console.log(i);
  }
}
// "0" "1" "2" "foo"

for (const i of iterable) {
  console.log(i);
}
// 3 5 7
```

The object `iterable` inherits the properties `objCustom` and `arrCustom` because it contains both `Object.prototype` and `Array.prototype` in its prototype chain.

The `for...in` loop logs only enumerable properties of the `iterable` object. It doesn't log array *elements* `3`, `5`, `7` or `"hello"` because those are not *properties* — they are *values*. It logs array *indexes* as well as `arrCustom` and `objCustom`, which are actual properties. If you're not sure why these properties are iterated over, there's a more thorough explanation of how array iteration and `for...in` work.

The second loop is similar to the first one, but it uses `Object.hasOwn()` to check if the found enumerable property is the object's own, i.e., not inherited. If it is, the property is logged. Properties `0`, `1`, `2` and `foo` are logged because they are own properties. Properties `arrCustom` and `objCustom` are not logged because they are inherited.

The `for...of` loop iterates and logs *values* that `iterable`, as an array (which is iterable), defines to be iterated over. The object's *elements* `3`, `5`, `7` are shown, but none of the object's *properties* are.

See also

- `Array.prototype.forEach()`
- `Map.prototype.forEach()`
- `Object.entries()`

function

The **`function`** declaration creates a binding of a new function to a given name.

You can also define functions using the `function` expression.

Try it

```
function calcRectArea(width, height) {
  return width * height;
}

console.log(calcRectArea(5, 6));
// Expected output: 30
```

Syntax

```
function name(param0) {
  statements
}
function name(param0, param1) {
  statements
}
```

```
function name(param0, param1, /* …, */ paramN) {
  statements
}
```

Parameters

`name`

The function name.

`param` Optional

The name of a formal parameter for the function. Maximum number of arguments varies in different engines. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements which comprise the body of the function.

Description

A `function` declaration creates a `Function` object. Each time when a function is called, it returns the value specified by the last executed `return` statement, or `undefined` if the end of the function body is reached. See functions for detailed information on functions.

`function` declarations behave like a mix of `var` and `let`:

- Like `let`, in strict mode, function declarations are scoped to the most closely containing block.
- Like `let`, function declarations at the top level of a module or within blocks in strict mode cannot be redeclared by any other declaration.
- Like `var`, function declarations at the top level of a script (strict or non-strict) become properties on `globalThis`. Function declarations at the top level of a script or function body (strict or non-strict) can be redeclared by another `function` or `var`.
- Like both, function declarations can be re-assigned, but you should avoid doing so.
- Unlike either, function declarations are hoisted together with its value and can be called anywhere in its scope.

Block-level function declaration

> **Warning:** In non-strict mode, function declarations inside blocks behave strangely. Only declare functions in blocks if you are in strict mode.

Functions can be conditionally declared — that is, a function statement can be nested within an `if` statement. However, in non-strict mode, the results are inconsistent across implementations.

```
console.log(
  `'foo' name ${
    "foo" in globalThis ? "is" : "is not"
  } global. typeof foo is ${typeof foo}`,
);
if (false) {
  function foo() {
    return 1;
  }
}

// In Chrome:
// 'foo' name is global. typeof foo is undefined
//
// In Firefox:
// 'foo' name is global. typeof foo is undefined
//
// In Safari:
// 'foo' name is global. typeof foo is function
```

The scoping and hoisting effect won't change regardless of whether the `if` body is actually executed.

```
console.log(
  `'foo' name ${
    "foo" in globalThis ? "is" : "is not"
  } global. typeof foo is ${typeof foo}`,
);
if (true) {
  function foo() {
    return 1;
  }
}

// In Chrome:
// 'foo' name is global. typeof foo is undefined
//
// In Firefox:
// 'foo' name is global. typeof foo is undefined
//
// In Safari:
// 'foo' name is global. typeof foo is function
```

In strict mode, block-level function declarations are scoped to that block and are hoisted to the top of the block.

```
"use strict";

{
  foo(); // Logs "foo"
  function foo() {
    console.log("foo");
  }
}

console.log(
  `'foo' name ${
    "foo" in globalThis ? "is" : "is not"
  } global. typeof foo is ${typeof foo}`,
);
// 'foo' name is not global. typeof foo is undefined
```

Hoisting

Function declarations in JavaScript are hoisted to the top of the enclosing function or global scope. You can use the function before you declared it:

```
hoisted(); // Logs "foo"

function hoisted() {
  console.log("foo");
}
```

Note that function expressions are not hoisted:

```
notHoisted(); // TypeError: notHoisted is not a function

var notHoisted = function () {
  console.log("bar");
};
```

Redeclarations

Whether `function` declarations can be redeclared in the same scope depends on what scope it's contained in.

At the top level of a script, `function` declarations behave like `var` and can be redeclared by another `function` or `var` but not by `let`, `const`, or `class`.

```
function a(b) {}
function a(b, c) {}
console.log(a.length); // 2
let a = 2; // SyntaxError: Identifier 'a' has already been declared
```

When `function` declarations are redeclared by `var`, the `var` declaration's initializer always overrides the function's value, regardless of their relative position. This is because function declarations are hoisted before any initializer gets evaluated, so the initializer comes later and overrides the value.

```
var a = 1;
function a() {}
console.log(a); // 1
```

At the top level of a function's body, `function` also behaves like `var` and can be redeclared or have the same name as a parameter.

```
function foo(a) {
  function a() {}
  console.log(typeof a);
}

foo(2); // Logs "function"
```

At the top level of a module or a block in strict mode, `function` declarations behave like `let` and cannot be redeclared by any other declaration.

```
// Assuming current source is a module
function foo() {}
function foo() {} // SyntaxError: Identifier 'foo' has already been declared
```

```
"use strict";
{
  function foo() {}
  function foo() {} // SyntaxError: Identifier 'foo' has already been declared
}
```

A `function` declaration within a `catch` block cannot have the same name as the `catch`-bound identifier, even in non-strict mode.

```
try {
} catch (e) {
  function e() {} // SyntaxError: Identifier 'e' has already been declared
}
```

Examples

Using function

The following code declares a function that returns the total amount of sales, when given the number of units sold of three products.

```
function calcSales(unitsA, unitsB, unitsC) {
  return unitsA * 79 + unitsB * 129 + unitsC * 699;
}
```

See also

- Functions guide
- Functions
- `Function`
- `function` expression
- `function*`
- `async function`
- `async function*`

if...else

The **`if...else`** statement executes a statement if a specified condition is truthy. If the condition is falsy, another statement in the optional `else` clause will be executed.

Try it

```
function testNum(a) {
  let result;
  if (a > 0) {
    result = "positive";
  } else {
    result = "NOT positive";
  }
  return result;
}

console.log(testNum(-5));
// Expected output: "NOT positive"
```

Syntax

```
if (condition)
  statement1

// With an else clause
if (condition)
  statement1
else
  statement2
```

`condition`

An expression that is considered to be either truthy or falsy.

`statement1`

Statement that is executed if *condition* is truthy. Can be any statement, including further nested `if` statements. To execute multiple statements, use a block statement (`{ /* ... */ }`) to group those statements. To execute no statements, use an empty statement.

`statement2`

Statement that is executed if `condition` is falsy and the `else` clause exists. Can be any statement, including block statements and further nested `if` statements.

Description

Multiple `if...else` statements can be nested to create an `else if` clause. Note that there is no `elseif` (in one word) keyword in JavaScript.

```
if (condition1)
  statement1
else if (condition2)
  statement2
else if (condition3)
  statement3
// …
else
  statementN
```

To see how this works, this is how it would look if the nesting were properly indented:

```
if (condition1)
  statement1
else
```

```
  if (condition2)
    statement2
  else
    if (condition3)
      statement3
// …
```

The conditions are evaluated in order until one evaluates to `true`. At that point, the associated statement is executed and the rest of the `else if` clauses are skipped.

To execute multiple statements within a clause, use a block statement (`{ /* ... */ }`) to group those statements.

```
if (condition) {
  statements1
} else {
  statements2
}
```

Not using blocks may lead to confusing behavior, especially if the code is hand-formatted. For example:

```
function checkValue(a, b) {
  if (a === 1)
    if (b === 2)
      console.log("a is 1 and b is 2");
  else
    console.log("a is not 1");
}
```

This code looks innocent — however, executing `checkValue(1, 3)` will log "a is not 1". This is because in the case of dangling else, the `else` clause will be connected to the closest `if` clause. Therefore, the code above, with proper indentation, would look like:

```
function checkValue(a, b) {
  if (a === 1)
    if (b === 2)
      console.log("a is 1 and b is 2");
    else
      console.log("a is not 1");
}
```

In general, it is a good practice to always use block statements, especially in code involving nested `if` statements.

```
function checkValue(a, b) {
  if (a === 1) {
    if (b === 2) {
      console.log("a is 1 and b is 2");
    }
  } else {
    console.log("a is not 1");
  }
}
```

Do not confuse the primitive Boolean values `true` and `false` with truthiness or falsiness of the `Boolean` object. Any value that is not `false`, `undefined`, `null`, `0`, `-0`, `NaN`, or the empty string (`""`), and any object, including a Boolean object whose value is `false`, is considered truthy when used as the condition. For example:

```
const b = new Boolean(false);
if (b) {
  console.log("b is truthy"); // "b is truthy"
}
```

Examples

Using if...else

```
if (cipherChar === fromChar) {
  result += toChar;
  x++;
} else {
  result += clearChar;
}
```

Using else if

Note that there is no `elseif` syntax in JavaScript. However, you can write it with a space between `else` and `if`:

```
if (x > 50) {
  /* do something */
} else if (x > 5) {
  /* do something */
} else {
  /* do something */
}
```

Using an assignment as a condition

You should almost never have an `if...else` with an assignment like `x = y` as a condition:

```
if ((x = y)) {
  // …
}
```

Because unlike `while` loops, the condition is only evaluated once, so the assignment is only performed once. The code above is equivalent to:

```
x = y;
if (x) {
  // …
}
```

Which is much clearer. However, in the rare case you find yourself wanting to do something like that, the `while` documentation has a Using an assignment as a condition section with our recommendations.

See also

- `block`
- `switch`
- Conditional (ternary) operator

import

The static **`import`** declaration is used to import read-only live bindings which are exported by another module. The imported bindings are called *live bindings* because they are updated by the module that exported the binding, but cannot be re-assigned by the importing module.

In order to use the `import` declaration in a source file, the file must be interpreted by the runtime as a module. In HTML, this is done by adding `type="module"` to the `<script>` tag. Modules are automatically interpreted in strict mode.

There is also a function-like dynamic `import()`, which does not require scripts of `type="module"`.

Syntax

```
import defaultExport from "module-name";
import * as name from "module-name";
import { export1 } from "module-name";
import { export1 as alias1 } from "module-name";
import { default as alias } from "module-name";
import { export1, export2 } from "module-name";
import { export1, export2 as alias2, /* … */ } from "module-name";
import { "string name" as alias } from "module-name";
import defaultExport, { export1, /* … */ } from "module-name";
import defaultExport, * as name from "module-name";
import "module-name";
```

`defaultExport`

> Name that will refer to the default export from the module. Must be a valid JavaScript identifier.

`module-name`

> The module to import from. Only single- and double-quoted string literals are allowed. The evaluation of the specifier is host-specified. Most hosts align with browsers and resolve the specifiers as URLs relative to the current module URL (see `import.meta.url`). Node, bundlers, and other non-browser environments often define their own features on top of this, so you should find documentation for them to understand the exact rules. The module specifier resolution section also has more information.

`name`

> Name of the module object that will be used as a kind of namespace when referring to the imports. Must be a valid JavaScript identifier.

`exportN`

> Name of the exports to be imported. The name can be either an identifier or a string literal, depending on what `module-name` declares to export. If it is a string literal, it must be aliased to a valid identifier.

`aliasN`

> Names that will refer to the named imports. Must be a valid JavaScript identifier.

The `"module-name"` may be followed by a set of import attributes, starting with the `with` keyword.

Description

`import` declarations can only be present in modules, and only at the top-level (i.e., not inside blocks, functions, etc.). If an `import` declaration is encountered in non-module contexts (for example, `<script>` tags without `type="module"`, `eval`, `new Function`, which all have "script" or "function body" as parsing goals), a `SyntaxError` is thrown. To load modules in non-module contexts, use the dynamic import syntax instead.

All imported bindings cannot be in the same scope as any other declaration, including `let`, `const`, `class`, `function`, `var`, and `import` declaration.

`import` declarations are designed to be syntactically rigid (for example, only string literal specifiers, only permitted at the top-level, all bindings must be identifiers), which allows modules to be statically analyzed and linked before getting evaluated. This is the key to making modules asynchronous by nature, powering features like top-level await.

The `import` keyword may be followed by a "phase modifier" that stops the module import process at a particular phase:

- `import defer`
- `import source`

Each of these syntaxes is considered a distinct type of declaration.

Forms of import declarations

There are four forms of `import` declarations:

- Named import: `import { export1, export2 } from "module-name";`
- Default import: `import defaultExport from "module-name";`
- Namespace import: `import * as name from "module-name";`
- Side effect import: `import "module-name";`

Below are examples to clarify the syntax.

Named import

Given a value named `myExport` which has been exported from the module `my-module` either implicitly as `export * from "another.js"` or explicitly using the `export` statement, this inserts `myExport` into the current scope.

```
import { myExport } from "/modules/my-module.js";
```

You can import multiple names from the same module.

```
import { foo, bar } from "/modules/my-module.js";
```

You can rename an export when importing it. For example, this inserts `shortName` into the current scope.

```
import { reallyReallyLongModuleExportName as shortName } from "/modules/my-module.js";
```

A module may also export a member as a string literal which is not a valid identifier, in which case you must alias it in order to use it in the current module.

```
// /modules/my-module.js
const a = 1;
export { a as "a-b" };
```

```
import { "a-b" as a } from "/modules/my-module.js";
```

> **Note:** `import { x, y } from "mod"` is not equivalent to `import defaultExport from "mod"` and then destructuring `x` and `y` from `defaultExport`. Named and default imports are distinct syntaxes in JavaScript modules.

Default import

Default exports need to be imported with the corresponding default import syntax. This version directly imports the default:

```
import myDefault from "/modules/my-module.js";
```

Since the default export doesn't explicitly specify a name, you can give the identifier any name you like.

It is also possible to specify a default import with namespace imports or named imports. In such cases, the default import will have to be declared first. For instance:

```
import myDefault, * as myModule from "/modules/my-module.js";
// myModule.default and myDefault point to the same binding
```

or

```
import myDefault, { foo, bar } from "/modules/my-module.js";
```

Importing a name called `default` has the same effect as a default import. It is necessary to alias the name because `default` is a reserved word.

```
import { default as myDefault } from "/modules/my-module.js";
```

Namespace import

The following code inserts `myModule` into the current scope, containing all the exports from the module located at `/modules/my-module.js`.

```
import * as myModule from "/modules/my-module.js";
```

Here, `myModule` represents a *namespace* object which contains all exports as properties. For example, if the module imported above includes an export `doAllTheAmazingThings()`, you would call it like this:

```
myModule.doAllTheAmazingThings();
```

`myModule` is a sealed object with `null` prototype. The default export is available as a key called `default`. For more information, see module namespace object.

> **Note:** JavaScript does not have wildcard imports like `import * from "module-name"`, because of the high possibility of name conflicts.

Import a module for its side effects only

Import an entire module for side effects only, without importing anything. This runs the module's global code, but doesn't actually import any values.

```
import "/modules/my-module.js";
```

This is often used for polyfills, which mutate the global variables.

Hoisting

Import declarations are hoisted. In this case, that means that the identifiers the imports introduce are available in the entire module scope, and their side effects are produced before the rest of the module's code runs.

```
myModule.doAllTheAmazingThings(); // myModule.doAllTheAmazingThings is imported by the next line

import * as myModule from "/modules/my-module.js";
```

Module specifier resolution

The ECMAScript specification does not define how module specifiers are resolved and leaves it to the host environment (e.g., browsers, Node.js, Deno). Browser behavior is specified by the HTML spec, and this has become the *de facto* baseline for all environments.

There are three types of specifiers widely recognized, as implemented by the HTML spec, Node, and many others:

- *Relative specifiers* that start with `/`, `./`, or `../`, which are resolved relative to the current module URL.
- *Absolute specifiers* that are parsable URLs, which are resolved as-is.
- *Bare specifiers* that are not one of the above.

The most notable caveat for relative specifiers, especially for people familiar with the CommonJS conventions, is that browsers forbid one specifier to implicitly resolve to many potential candidates. In CommonJS, if you have `main.js` and `utils/index.js`, then all of the following will import the "default export" from `utils/index.js`:

```
// main.js
const utils = require("./utils"); // Omit the "index.js" file name
const utils = require("./utils/index"); // Omit only the ".js" extension
const utils = require("./utils/index.js"); // The most explicit form
```

On the web, this is costly because if you write `import x from "./utils"`, the browser needs to send requests to `utils`, `utils/index.js`, `utils.js`, and potentially many other URLs until it finds an importable module. Therefore, in the HTML spec, the specifier by default can only be a URL resolved relative to the current module URL. You cannot omit the file extension or the `index.js` file name. This behavior has been inherited by Node's ESM implementation, but it is not a part of the ECMAScript specification.

Note that this does not mean that `import x from "./utils"` never works on the web. The browser still sends a request to that URL, and if the server can respond with the correct content, the import will succeed. This requires the server to implement some custom resolution logic, because usually extension-less requests are understood as requests for HTML files.

Absolute specifiers can be any kind of URL that resolve to importable source code. Most notably:

- HTTP URLs are always supported on the web since most scripts already have HTTP URLs. It's supported natively by Deno (which initially predicated its entire module system on HTTP URLs), but it only has experimental support in Node via custom HTTPS loaders.
- `file:` URLs are supported by many non-browser runtimes such as Node, since scripts there already have `file:` URLs, but they are not supported by browsers due to security reasons.
- Data URLs are supported by many runtimes including browsers, Node, Deno, etc. They are useful for embedding small modules directly into the source code. Supported MIME types are those that designate importable source code, such as `text/javascript` for JavaScript, `application/json` for JSON modules, `application/wasm` for WebAssembly modules, etc. (They may still require import attributes.)

  ```
  // HTTP URLs
  import x from "https://example.com/x.js";
  // Data URLs
  import x from "data:text/javascript,export default 42;";
  // Data URLs for JSON modules
  import x from 'data:application/json,{"foo":42}' with { type: "json" };
  ```

  `text/javascript` data URLs are still interpreted as modules, but they cannot use relative imports — because the `data:` URL scheme is not hierarchical. That is, `import x from "data:text/javascript,import y from './y.js';"` will throw an error because the relative specifier `'./y.js'` cannot be resolved.
- `node:` URLs resolve to built-in Node.js modules. They are supported by Node and other runtimes that claim compatibility with Node, such as Bun.

Bare specifiers, popularized by CommonJS, are resolved within the `node_modules` directory. For example, if you have `import x from "foo"`, then the runtime will look for the `foo` package within any `node_modules` directory in the parent directories of the current module. This behavior can be reproduced in browsers using import maps, which also enable you to customize resolution in other ways.

The module resolution algorithm can also be executed programmatically using the `import.meta.resolve` function defined by the HTML spec.

Examples

Standard Import

In this example, we create a re-usable module that exports a function to get all primes within a given range.

```
// getPrimes.js
/**
 * Returns a list of prime numbers that are smaller than `max`.
 */
export function getPrimes(max) {
  const isPrime = Array.from({ length: max }, () => true);
  isPrime[0] = isPrime[1] = false;
  isPrime[2] = true;
  for (let i = 2; i * i < max; i++) {
    if (isPrime[i]) {
      for (let j = i ** 2; j < max; j += i) {
        isPrime[j] = false;
      }
    }
  }
  return [...isPrime.entries()]
    .filter(([, isPrime]) => isPrime)
    .map(([number]) => number);
}
```

```
import { getPrimes } from "/modules/getPrimes.js";

console.log(getPrimes(10)); // [2, 3, 5, 7]
```

Imported values can only be modified by the exporter

The identifier being imported is a *live binding*, because the module exporting it may re-assign it and the imported value would change. However, the module importing it cannot re-assign it. Still, any module holding an exported object can mutate the object, and the mutated value can be observed by all other modules importing the same value.

You can also observe the new value through the module namespace object.

```
// my-module.js
export let myValue = 1;
setTimeout(() => {
  myValue = 2;
}, 500);
```

```
// main.js
import { myValue } from "/modules/my-module.js";
import * as myModule from "/modules/my-module.js";

console.log(myValue); // 1
console.log(myModule.myValue); // 1
setTimeout(() => {
  console.log(myValue); // 2; my-module has updated its value
  console.log(myModule.myValue); // 2
  myValue = 3; // TypeError: Assignment to constant variable.
  // The importing module can only read the value but can't re-assign it.
}, 1000);
```

Importing non-JavaScript modules

Non-JavaScript modules can also be imported using the `import` statement, but their types need to be explicitly declared using import attributes. For example, to import a JSON module, you need to specify the `type: "json"` attribute.

```
import data from "./data.json" with { type: "json" };
```

See also

- `export`
- `import()`
- `import.meta`
- Import attributes
- Previewing ES6 Modules and more from ES2015, ES2016 and beyond on blogs.windows.com (2016)
- ES6 in Depth: Modules on hacks.mozilla.org (2015)
- ES modules: A cartoon deep-dive on hacks.mozilla.org (2018)
- Exploring JS, Ch.16: Modules by Dr. Axel Rauschmayer
- Export and Import on javascript.info

import defer

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import defer`** declaration behaves like regular `import` declarations, except that it results in a deferred module namespace object. The module and its dependencies are fetched and linked up front, but their synchronous evaluation is deferred until the namespace's properties are accessed. Modules that use top-level `await` are evaluated eagerly.

Syntax

```
import defer * as name from "module-name";
```

`name`

Name that will refer to the deferred module namespace object. Must be a valid JavaScript identifier.

`module-name`

The module to import from. Handled the same way as the `module-name` in regular `import` declarations.

Import attributes are also supported, using a `with` clause after the module specifier.

`defer` is not a reserved word. For example, `import defer from "./module.js"` is still a regular default import whose local binding is named `defer`.

Description

By default, the `import` declaration performs many tasks at once: resolving the module specifier, fetching the module source code, parsing (potentially discovering transitive dependencies), linking, and evaluating it. This form of eager evaluation is not always desirable: it may cause slower startup, the environment for its evaluation may not be fully prepared, or the module may not need to be evaluated at all.

The *import phase modifier* allows the module import process to stop at a particular phase. By adding `defer` after `import`, the source code is linked but remains unevaluated, provided that it can be evaluated synchronously (i.e., does not use top-level `await`). Accessing an export through the deferred namespace synchronously evaluates the module and any dependencies that need to be evaluated before it. The access returns the export's value after evaluation finishes. This executes the module's top-level code, not just the code needed to initialize the requested export. Transitive dependencies imported with their own `import defer` declarations can remain deferred.

By ensuring that the paused subgraph can be evaluated synchronously, `import defer` can be "dropped in" with almost no code changes to the places that use the module:

```
// Before:
import * as ts from "typescript";
// The full typescript module graph evaluates here

function compileFile(path) {
  const program = ts.createProgram([path], {});
}
```

```
// After:
import defer * as ts from "typescript";
// No code is evaluated; potentially faster startup

function compileFile(path) {
  // The typescript module graph evaluates when `ts.createProgram`
  // is accessed, i.e., when `compileFile` is called.
  // If `compileFile` is never called, then the module graph
  // is never evaluated.
  const program = ts.createProgram([path], {});
}
```

> **Warning:** Deferring an import changes when its side effects occur. Do not defer modules whose side effects are needed before the rest of your code runs, such as modules that install polyfills.

Unlike `import source`, a deferred module is still linked up front. Linking up front lets the module loader resolve dependencies, catching missing dependencies or invalid imports before the module is used. Leaving the module unlinked avoids loading dependencies you may not need and allows you to control how it is instantiated.

Unlike `import()`, the deferred module is still fetched, parsed, and linked up front, avoiding unnecessary async coloring (entire chain of function calls forced to become async). `import defer` also enjoys most benefits of a static declaration, such as better static analysis.

Note that only the "namespace import" syntax is supported. You cannot use `import defer { property } from "./my-module.js"`, etc., because the execution is triggered by property access on the namespace object.

Caching semantics

The modifier applies to an import, not to the module itself. If another part of the application imports the same module without `defer`, the module is evaluated as usual. Both forms share the same module state, and the module's code executes at most once. Changing the import phase does not create a separate module in the cache:

```
import defer * as ts from "typescript";
// No code is evaluated
import * as ts2 from "typescript";
// The full typescript module graph evaluates here

function compileFile(path) {
  // Accessing `ts.createProgram` no longer evaluates the subgraph
```

```
  // because it's already evaluated.
  const program = ts.createProgram([path], {});
}
```

In contrast, import attributes can affect module identity. For example, in a host that supports text modules, these two declarations request different module types:

```
import * as mod from "./module.js";
import text from "./module.js" with { type: "text" };
```

The two imports are considered to be from different modules that happen to share the same string specifier (on the web, they will be requested with different HTTP headers). The supported attributes and their effects on loading and module identity are defined by the host.

Deferred module namespace object

A deferred module namespace object behaves much like a regular module namespace object: it has a `null` prototype, is non-extensible and sealed, and exposes read-only live bindings to the module's exports. Its string keys are enumerable and sorted in lexicographic order. The default export is available as the property named `default`.

There are three differences from a regular namespace:

- Operations that inspect exports can trigger evaluation and throw evaluation errors, as described below.
- Its `[Symbol.toStringTag]` property is `"Deferred Module"` instead of `"Module"`. This remains the case after evaluation.
- It does not expose an export named `then`, even after evaluation. Reading `namespace.then` always returns `undefined`. This prevents promise resolution from treating the namespace as a thenable and triggering evaluation. To access such an export, use a regular import, or introduce an intermediate module that re-exports `then` under a different name.

The deferred and regular namespaces for the same module are distinct objects, even after evaluation. Repeated deferred imports of the same module, whether static or dynamic, share the same deferred namespace object.

In order to implement the behavior of "triggering module evaluation when keys are accessed", the deferred module namespace is essentially a `Proxy` that intercepts the following actions to trigger module evaluation:

- `defineProperty()` for any string key other than `then`: for example, `Object.defineProperty(namespace, "value", {})`.

  > **Note:** Because the module namespace object is non-extensible and sealed, you can't meaningfully add or change any property descriptor, including its value. Nevertheless, even if the operation fails, evaluation is still triggered.
  >
  > The proxy does not intercept `set()`. Setting properties like `namespace.value = 1;` always fails.

- `deleteProperty()` for any string key other than `then`: for example, `delete namespace.value`.

  > **Note:** You cannot actually delete any property that the namespace has. Nevertheless, even if the operation fails, evaluation is still triggered.

- `get()` and `getOwnPropertyDescriptor()` for any string key other than `then`: for example, `namespace.value`, `namespace["missing"]`, `const { default: value } = namespace`, `Object.getOwnPropertyDescriptor(namespace, "value")`.

  > **Note:** Destructuring an export at the top level therefore defeats the deferral of that module.

- `has()` for any string key other than `then`: for example, `"value" in namespace`, `Object.hasOwn(namespace, "missing")`.
- `ownKeys()`: for example, `Object.keys(namespace)`, `Object.getOwnPropertySymbols(namespace)`, `for (const key in namespace) {}`. Even enumerating only symbol keys triggers evaluation.

Merely referring to the namespace, assigning it to another variable, comparing its identity, or passing it to a function does not trigger evaluation. Neither does reading `then` or a symbol-keyed property, such as `namespace[Symbol.toStringTag]`. Calling `Object.getPrototypeOf()` or `Object.isExtensible()` does not trigger evaluation either. However, `Object.isSealed()` and `Object.isFrozen()` enumerate keys and therefore do trigger evaluation.

Top-level await

Reading a namespace property is synchronous, so it cannot wait for asynchronous module evaluation. Modules that contain top-level `await` are evaluated eagerly, along with the dependencies required to evaluate them. This includes modules reached through further deferred imports. The importing module waits for this asynchronous evaluation before running its own body.

If the directly imported module contains top-level `await`, its evaluation is not deferred. If only some of its dependencies contain top-level `await`, those dependencies are evaluated eagerly, but the synchronous parts of the graph that are not required for their evaluation can remain deferred. See Deferring a module with an asynchronous dependency.

Errors

Loading, parsing, and linking errors are not deferred. For example, a missing module, a syntax error in a dependency, or an unresolved named import prevents the importing module from running, even if the deferred namespace is never accessed. Errors from eagerly evaluated asynchronous dependencies also prevent the importing module from running.

Errors thrown during deferred evaluation are thrown synchronously by the operation that triggers evaluation. You can catch them with `try...catch` around that operation. The error is cached: subsequent operations that trigger evaluation throw the same error instead of retrying the module's code. This also applies if another import previously caused the module's evaluation to fail.

An operation that triggers evaluation throws a `TypeError` if the module or its dependencies are not ready for synchronous evaluation. This can happen with cyclic imports, when an access would require a module that is still being evaluated. An `import defer` declaration does not make every cyclic dependency safe to access during initialization. A readiness failure itself does not mark the requested module as having failed evaluation: a later access can succeed once its dependencies are ready.

Examples

Evaluating a module on first use

The following module initializes a lookup table when its top-level code runs:

```
// -- squares.js --
console.log("Initializing squares");
const squares = Array.from({ length: 10000 }, (_, index) => index ** 2);

export function getSquare(index) {
  return squares[index];
}
```

The importing module can expose a synchronous function without initializing the table until it is needed:

```
// -- main.js --
import defer * as squares from "./squares.js";

console.log("Ready");

export function showSquare(index) {
  console.log(squares.getSquare(index));
}

showSquare(3); // Logs "Initializing squares", then 9
showSquare(4); // Logs 16; initialization is not repeated
```

`"Ready"` is logged before `"Initializing squares"`. If `showSquare()` is never called and no other import causes `squares.js` to be evaluated, the lookup table is never initialized.

Deferring a module with an asynchronous dependency

In this example, `report.js` (which itself is synchronous) depends on both an asynchronous configuration module and a synchronous formatting module.

```
// -- config.js --
export const locale = await Promise.resolve("en-US");
console.log("Configuration ready");
```

```
// -- format.js --
console.log("Formatting module evaluated");

export function format(value, locale) {
  return new Intl.NumberFormat(locale).format(value);
}
```

```
// -- report.js --
import { locale } from "./config.js";
import { format } from "./format.js";

console.log("Report module evaluated");

export function createReport(value) {
  return format(value, locale);
}
```

```
// -- main.js --
import defer * as report from "./report.js";

console.log("Main module evaluated");
console.log(report.createReport(1000));
```

The output is:

```
Configuration ready
Main module evaluated
Formatting module evaluated
Report module evaluated
1,000
```

`config.js` is evaluated before `main.js` because it contains top-level `await`. Neither `format.js` nor `report.js` needs to run to evaluate `config.js`, so their evaluation is deferred until `report.createReport` is accessed.

Catching evaluation errors

```
// -- broken.js --
export const value = 1;
throw new Error("Initialization failed");
```

```
// -- main.js --
import defer * as broken from "./broken.js";

for (let attempt = 0; attempt < 2; attempt++) {
  try {
    console.log(broken.value);
  } catch (error) {
    console.log(error.message); // "Initialization failed" on both attempts
  }
}
```

Even though `value` was initialized before the exception was thrown, accessing it through the deferred namespace throws the cached evaluation error.

Exporting a deferred namespace

There is no `export defer` syntax (see `export` for more information). You can import a deferred namespace and then export its binding without triggering evaluation:

```
// -- features.js --
import defer * as report from "./report.js";

export { report };
```

```
// -- main.js --
import { report } from "./features.js";

// Accessing an export through report triggers its deferred evaluation.
console.log(report.createReport(1000));
```

See also

- JavaScript modules guide
- `import`
- `import.defer()`

import source

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import source`** declaration is similar to the regular `import` declaration, except that it results in an object that represents the module's compiled source code. The module is fetched and compiled, but its dependencies are not loaded and it is not linked or evaluated. It can be imperatively evaluated later, such as by using dynamic import or `WebAssembly.instantiate()`.

To use `import source`, the target module must be of a kind that supports source phase imports. Currently, only WebAssembly modules support source phase imports, and result in `WebAssembly.Module` objects. JavaScript module source objects will be added by the ECMAScript Module Phase Imports proposal.

Syntax

```
import source x from "module-name";
```

`x`

Name that will refer to the module source object. Must be a valid JavaScript identifier.

`module-name`

The module to import from. Handled the same way as the `module-name` in regular `import` declarations.

Import attributes are also supported, using a `with` clause after the module specifier.

`source` is not a reserved word. For example, `import source from "./module.js"` is still a regular default import whose local binding is named `source`.

Exceptions

`SyntaxError`

Thrown if the target module does not support source phase imports.

Description

By default, the `import` declaration performs many tasks at once: resolving the module specifier, fetching the module source code, parsing (potentially discovering transitive dependencies), linking, and evaluating it. This form of eager evaluation is not always desirable, especially when the module source code is intended to be evaluated in some other context, such as a worker thread.

The *import phase modifier* allows the module import process to stop at a particular phase. By adding `source` after `import`, the source code is parsed and compiled but remains unlinked and unevaluated. The source import does not load the module's transitive dependencies.

Traditionally, the only way to run a part of the pipeline was to manually implement the entirety of it. For example, with WebAssembly, you first use `fetch()` to retrieve the module's binary bytes, and later compile them using `WebAssembly.compile()`, or compile the response stream using `WebAssembly.compileStreaming()`, into a `WebAssembly.Module` object.

```
const myModuleSource = await WebAssembly.compileStreaming(
  fetch("./my-module.wasm"),
);
```

This kind of manual workflow is problematic for several reasons:

- It involves too much low-level manipulation, which may be inconsistent with the default module loader's behavior, such as caching, HTTP header settings, and module specifier resolution.
- It is imperative, making it difficult to reason about the module's dependencies and their evaluation order, especially for bundlers or other analysis tools.
- Userland evaluation of the raw source code is incompatible with strict CSP settings.

Using source phase imports avoids these problems.

```
import source myModuleSource from "./my-module.wasm";
```

The object obtained from this import is an instance of a subclass of `AbstractModuleSource`. Each module type that supports source phase imports defines its own subclass. For WebAssembly, the subclass is `WebAssembly.Module`. This object is cached for the specific module so that later source imports of the same specifier return the exact same object.

Unlike `import defer`, using source import also defers linking. Linking up front lets the module loader resolve dependencies, catching missing dependencies or invalid imports before the module is used. Leaving the module unlinked avoids loading dependencies you may not need and allows you to control how it is instantiated. For example, you can instantiate a source-imported WebAssembly module multiple times with different imports, or send it to a worker for instantiation there.

Unlike `import()`, the source-imported module is fetched and compiled up front without evaluating it. `import source` also enjoys most benefits of a static declaration, such as better static analysis.

Note that only the "default import" syntax is supported. You cannot use `import source { property } from "./my-module.wasm"`, etc.

Importing WebAssembly source

For WebAssembly, you supply the imported values when instantiating the module:

```
import source myModuleSource from "./my-module.wasm";

const instance = await WebAssembly.instantiate(myModuleSource, {
  env: { log: console.log },
});
const { exports } = instance;
```

Unlike an ordinary WebAssembly module import, this creates a new instance with the imports you provide, rather than using the cached instance linked by the module loader. Its `exports` object is not a JavaScript module namespace object. For example, WebAssembly globals are exposed as `WebAssembly.Global` objects through `instance.exports`, whereas the ESM integration exposes their values to JavaScript importers.

> **Note:** Support for source phase imports of WebAssembly does not imply support for ordinary WebAssembly module imports. A host may support `import source mod from "./mod.wasm"` while rejecting `import * as ns from "./mod.wasm"`.

Source imports use the host's module loader, including its compilation settings. They are not necessarily equivalent to calling `WebAssembly.compileStreaming()` with default options. For example, Node.js enables JavaScript string builtins for Wasm imports and rejects reserved import and export names. Use the direct WebAssembly compilation APIs when you need custom compilation options.

Importing JavaScript source

For JavaScript (assuming the proposed `import(moduleSource)` support, also in ECMAScript Module Phase Imports):

```
import * as namespace from "./my-module.js";

// Is equivalent to:

import source myModuleSource from "./my-module.js";

const namespace = await import(myModuleSource);
```

Caching semantics

The modifier applies to an import, not to the module itself. If another part of the application imports the same module without `source`, the module is evaluated as usual. Both forms share the same module state, and the module's code executes at most once. Changing the import phase does not create a separate module in the cache:

```
import * as x from "foo";
import source xSource from "foo";

console.log((await import(xSource)) === x); // true
```

> **Note:** This cached instance is reused when you use `import()`, but not when you manually instantiate WebAssembly. Each call to `WebAssembly.instantiate(modSource, imports)` or `new WebAssembly.Instance(modSource, imports)` creates a new instance and runs its initialization, including any start function. In contrast, `WebAssembly.compileStreaming()` only compiles; it does not execute the module or use the ESM module cache.

In contrast, import attributes can affect module identity. For example, in a host that supports text modules, these two declarations request different module types:

```
import * as mod from "./module.js";
import text from "./module.js" with { type: "text" };
```

The two imports are considered to be from different modules that happen to share the same string specifier (on the web, they will be requested with different HTTP headers). The supported attributes and their effects on loading and module identity are defined by the host.

Examples

Instantiating a module with different imports

Suppose `counter.wasm` imports an immutable `i32` global named `initial` from `env`, initializes an internal counter from it, and exports a function named `increment`. You can reuse its compiled source with different imports and independent internal counters:

```
import source counterSource from "./counter.wasm";

const first = await WebAssembly.instantiate(counterSource, {
  env: { initial: 0 },
});
const second = await WebAssembly.instantiate(counterSource, {
  env: { initial: 100 },
});

console.log(first.exports.increment()); // 1
console.log(first.exports.increment()); // 2
console.log(second.exports.increment()); // 101
```

Each instance has its own internal state, although explicitly imported memories, tables, or other mutable objects can be shared between instances. You can also send the compiled source to a worker for instantiation there.

Exporting a module source

There is no `export source` syntax (see `export` for more information). You can import a module source and then export its binding without instantiating or evaluating the module:

```
// -- sources.js --
import source counterSource from "./counter.wasm";

export { counterSource };
```

```
// -- main.js --
import { counterSource } from "./sources.js";

const instance = new WebAssembly.Instance(counterSource, {
  env: { initial: 0 },
});
```

See also

- JavaScript modules guide
- `import`
- `import.source()`
- `AbstractModuleSource`

Import attributes

> **Note:** A previous version of this proposal used the `assert` keyword instead of `with`. The assertion feature is now non-standard. Check the browser compatibility table for details.

The **import attributes** feature instructs the runtime about how a module should be loaded, including the behavior of module resolution, fetching, parsing, and evaluation. It's supported in `import` declarations, `export...from` declarations, and dynamic `import()`.

Attributes can be attached to any kind of `import`/`export from` statement, including default import, namespace import, etc. They follow the module specifier string and start with the `with` keyword. When used with `import()`, the attributes are specified in the `options` parameter as the `with` property.

Syntax

```
import { names } from "module-name" with {};
import { names } from "module-name" with { key: "data" };
import { names } from "module-name" with { key: "data", key2: "data2" };
import { names } from "module-name" with { key: "data", key2: "data2", /* …, */ keyN: "dataN" };

export { names } from "module-name" with {};
```

```
export { names } from "module-name" with { key: "data" };
export { names } from "module-name" with { key: "data", key2: "data2" };
export { names } from "module-name" with { key: "data", key2: "data2", /* …, */ keyN: "dataN" };
```

Parameters

`keyN`

An attribute key. Can be an identifier or a string literal. All keys must be unique and must be known to the runtime.

`"dataN"`

An attribute value. Must be a string literal.

Exceptions

`SyntaxError`

An unsupported `key` was specified in a *static import*.

`TypeError`

An unsupported `key` was specified in a *dynamic import*.

Note that specifying an unsupported value for a supported key may also result in an exception in some cases, depending on the key.

Description

Import attributes tell the runtime how a particular module should be loaded.

The primary use case is to load non-JS modules, such as JSON modules and CSS modules. Consider the following statement:

```
import data from "https://example.com/data.json";
```

On the web, each import statement results in an HTTP request. The response is then prepared into a JavaScript value and made available to the program by the runtime. For example, the response may look like this:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
...
{"name":"Maria"}
```

Modules are identified and parsed only according to their served media type (MIME type) — the file extension in the URL cannot be used to identify a file's type. In this case, the MIME type is `application/json`, which tells the browser that the file is JSON and must be parsed as JSON. If, for some reason (e.g., the server is hijacked or bogus), the media type in the server response is set to `text/javascript` (for JavaScript source), then the file would be parsed and executed as code. If the "JSON" file actually contains malicious code, the `import` declaration would unintentionally execute external code, posing a serious security threat.

Import attributes fix this problem by allowing the author to explicitly specify how a module should be validated. In particular, the `type` attribute allows you to validate that the file is served with a particular media type, and fails the import if a different media type is used.

For example, the code above can be written to specify that the expected type is `"json"` and the import would fail if it was served with the `text/javascript` (or any media type other than `application/json`):

```
import data from "https://example.com/data.json" with { type: "json" };
```

The `type` attribute allows you to specify that modules are served as JSON, CSS, or plain text (and implicitly as JavaScript).

Other attributes may also be supported, and can affect the behavior of different parts of the loading process. A syntax error is thrown if an unknown attribute is used.

Standard attributes

The available attributes depend on the language and runtime environment. The ECMAScript standard defines the `type` attribute with the values `"json"` and `"text"`.

The HTML specification also defines the `type` attribute with values `"json"`, `"text"` and `"css"` — these are the attributes that are supported in browser environments.

JSON Modules (`{ type: "json" }`)

The `json` type indicates that the imported file must contain JSON. You can load JSON from a file into the `data` object using the following code:

```
import data from "https://example.com/data.json" with { type: "json" };
```

If the file is served with any other media type than `"application/json"`, the import will fail.

The `type` attribute changes how the module is fetched (the browser sends the request with `Accept: application/json` header), but does *not* change how the module is parsed or evaluated. The runtime already knows to parse the module as JSON given the response MIME type. It only uses the attribute to do *after-the-fact* checking that the `data.json` module is, in fact, a JSON module. For example, if the response header changes to `Content-Type: text/javascript` instead, the program will fail with a similar error as above.

The specification explicitly calls out `type: "json"` to be supported — if a module is asserted to be `type: "json"` and the runtime does not fail this import, then it must be parsed as JSON.

However, there's no behavior requirement otherwise: for imports without a `type: "json"` attribute, the runtime may still parse it as JSON if security is not an issue in this environment.

Browsers, on the other hand, implicitly assume that the module is JavaScript if the `type` is not specified, and fail if the module is not JavaScript (for example, JSON). This ensures that module types are always strictly validated and prevents any security risks. Non-browser runtimes such as Node and Deno align with browser semantics and enforce `type` for JSON modules.

In other words, if you omit the `type` and attempt to import a file as `"application/json"` you will usually get an error like the following:

```
Failed to load module script: Expected a JavaScript module script but the server responded with a MIME type of "application/json". Strict MIME type checking is enforced for module scripts per HTML spec.
```

CSS Modules (`{ type: "css" }`)

The HTML spec defines the `css` type, which imports a stylesheet into a script as a `CSSStyleSheet` object.

The code below shows how you might import a style and add it to your document. The import will throw an exception if `example_styles.css` is served with any media type other than `"text/css"`.

```
import exampleStyles from "https://example.com/example_styles.css" with { type: "css" };

document.adoptedStyleSheets.push(exampleStyles);
```

Note that importing CSS modules into workers is usually not supported, because the CSSOM specification only exposes `CSSStyleSheet` in the window context.

Text Modules (`{ type: "text" }`)

The `text` type allows importing a module's source as a string value. You can load text from a file into the `text` string using the following code:

```
import text from "https://example.com/file.txt" with { type: "text" };
```

The file will be requested with an `Accept: text/plain` header, but the value of the response's `Content-Type` header is ignored, and all files are parsed as UTF-8. It can contain any textual data, even JavaScript code (which is treated as plain text).

If possible, prefer `import source` to obtain a compiled module for later instantiation, or `import defer` to defer the synchronous evaluation of a module. Using `with { type: "text" }` means the module cannot reuse the cache if it was imported somewhere else, and userland evaluation of string sources may be incompatible with strict CSP settings.

Intended semantics for import attributes

An attribute can change the runtime's behavior at every stage of the module loading process:

- Resolution: the attribute is part of the module specifier (the string in the `from` clause). Therefore, given the same string path, different attributes may lead to entirely different modules being loaded. For example, TypeScript supports the `resolution-mode` attribute.

  ```
  import type { TypeFromRequire } from "pkg" with {
    "resolution-mode": "require",
  };
  ```

- Fetching: for example, CSS modules are fetched with the `destination` set to `"style"`, JSON modules are fetched with `destination: "json"`, and text modules are fetched with `destination: "text"`. This means given the same destination URL, the server may still return different content.
- Parsing and evaluation: the runtime may use the attribute to determine how to parse and evaluate the module.

Examples

Importing JSON modules with the type attribute

In `data.json`:

```
{
  "name": "Shilpa"
}
```

In `index.html`:

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <script type="module">
      import data from "./data.json" with { type: "json" };

      const p = document.createElement("p");
      p.textContent = `name: ${data.name}`;
      document.body.appendChild(p);
    </script>
  </head>
  <body></body>
</html>
```

Start a local HTTP server (see troubleshooting) and go to the `index.html` page. You should see `Shilpa` on the page.

> **Note:** JSON modules only have one default export. You cannot do named imports from them (like `import { name } from "data.json"`).

Using import attributes with dynamic import

Import attributes are also accepted as the second parameter of the `import()` syntax.

```
const data = await import("./data.json", {
  with: { type: "json" },
});
```

Note that, like static imports, dynamic imports are cached for the lifetime of the environment (e.g., a page or worker). If you expect this data to change (such as the latest news or a user's credits), use the Fetch API instead.

See also

- `import`
- `export`
- `import()`
- Import attributes proposal
- JSON modules proposal
- Import Text proposal

Labeled statement

A **labeled statement** is any statement that is prefixed with an identifier. You can jump to this label using a `break` or `continue` statement nested within the labeled statement.

Try it

```
let i, j;

loop1: for (i = 0; i < 3; i++) {
  loop2: for (j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break loop1;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}

// Expected output:
// "i = 0, j = 0"
// "i = 0, j = 1"
// "i = 0, j = 2"
// "i = 1, j = 0"
```

Syntax

```
label:
  statement
```

`label`

: Any JavaScript identifier that is not a reserved word.

`statement`

: A JavaScript statement. `break` can be used within any labeled statement, and `continue` can be used within labeled looping statements.

Description

You can use a label to identify a statement, and later refer to it using a `break` or `continue` statement. Note that JavaScript has *no* `goto` statement; you can only use labels with `break` or `continue`.

Any `break` or `continue` that references `label` must be contained within the `statement` that's labeled by `label`. Think about `label` as a variable that's only available in the scope of `statement`.

If a `break label;` statement is encountered when executing `statement`, execution of `statement` terminates, and execution continues at the statement immediately following the labeled statement.

`continue label;` can only be used if `statement` is one of the looping statements. If a `continue label;` statement is encountered when executing `statement`, execution of `statement` continues at the next iteration of the loop. `continue;` without a label can only continue the innermost loop, while `continue label;` allows continuing any given loop even when the statement is nested within other loops.

A statement can have multiple labels. In this case, the labels are all functionally equivalent.

Examples

Using a labeled continue with for loops

```
// The first for statement is labeled "loop1"
loop1: for (let i = 0; i < 3; i++) {
  // The second for statement is labeled "loop2"
  loop2: for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      continue loop1;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}

// Logs:
// i = 0, j = 0
// i = 0, j = 1
// i = 0, j = 2
// i = 1, j = 0
// i = 2, j = 0
// i = 2, j = 1
// i = 2, j = 2
```

Notice how it skips both "i = 1, j = 1" and "i = 1, j = 2".

Using a labeled break with for loops

```
let i, j;

// The first for statement is labeled "loop1"
loop1: for (i = 0; i < 3; i++) {
  // The second for statement is labeled "loop2"
  loop2: for (j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break loop1;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}

// Logs:
// i = 0, j = 0
// i = 0, j = 1
// i = 0, j = 2
// i = 1, j = 0
```

Notice the difference with the previous `continue` example: when `break loop1` is encountered, the execution of the outer loop is terminated, so there are no further logs beyond "i = 1, j = 0"; when `continue loop1` is encountered, the execution of the outer loop continues at the next iteration, so only "i = 1, j = 1" and "i = 1, j = 2" are skipped.

Using a labeled continue statement

Given an array of items and an array of tests, this example counts the number of items that pass all the tests.

```
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let itemsPassed = 0;

itemIteration: for (const item of items) {
  for (const test of tests) {
    if (!test.pass(item)) {
      continue itemIteration;
    }
  }

  itemsPassed++;
}
```

Note how the `continue itemIteration;` statement skips the rest of the tests for the current item as well as the statement that updates the `itemsPassed` counter, and continues with the next item. If you don't use a label, you would need to use a boolean flag instead.

```
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let itemsPassed = 0;
```

```
for (const item of items) {
  let passed = true;
  for (const test of tests) {
    if (!test.pass(item)) {
      passed = false;
      break;
    }
  }
  if (passed) {
    itemsPassed++;
  }
}
```

Using a labeled break statement

Given an array of items and an array of tests, this example determines whether all items pass all tests.

```
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let allPass = true;

itemIteration: for (const item of items) {
  for (const test of tests) {
    if (!test.pass(item)) {
      allPass = false;
      break itemIteration;
    }
  }
}
```

Again, if you don't use a label, you would need to use a boolean flag instead.

```
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let allPass = true;

for (const item of items) {
  let passed = true;
  for (const test of tests) {
    if (!test.pass(item)) {
      passed = false;
      break;
    }
  }
  if (!passed) {
    allPass = false;
    break;
  }
}
```

Using a labeled block with break

You can label statements other than loops, such as simple blocks, but only `break` statements can reference non-loop labels.

```
foo: {
  console.log("face");
  break foo;
  console.log("this will not be executed");
}
console.log("swap");

// Logs:
// "face"
// "swap"
```

Labeled function declarations

Labels can only be applied to statements, not declarations. There is a legacy grammar that allows function declarations to be labeled in non-strict code:

```
L: function F() {}
```

In strict mode code, however, this will throw a `SyntaxError`:

```
"use strict";
L: function F() {}
// SyntaxError: functions cannot be labelled
```

Non-plain functions, such as generator functions and async functions can neither be labeled in strict code, nor in non-strict code:

```
L: function* F() {}
// SyntaxError: generator functions cannot be labelled
```

The labeled function declaration syntax is deprecated and you should not use it, even in non-strict code. You cannot actually jump to this label within the function body.

See also

- `break`
- `continue`

let

The **`let`** declaration declares re-assignable, block-scoped local variables, optionally initializing each to a value.

Try it

```
let x = 1;

if (x === 1) {
  let x = 2;

  console.log(x);
  // Expected output: 2
}

console.log(x);
// Expected output: 1
```

Syntax

```
let name1;
let name1 = value1;
let name1 = value1, name2 = value2;
let name1, name2 = value2;
let name1 = value1, name2, /* …, */ nameN = valueN;
```

Parameters

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier or a destructuring binding pattern.

`valueN` Optional

Initial value of the variable. It can be any legal expression. Default value is `undefined`.

Description

The scope of a variable declared with `let` is one of the following curly-brace-enclosed syntaxes that most closely contains the `let` declaration:

- Block statement
- `switch` statement
- `try...catch` statement
- Body of one of the `for` statements, if the `let` is in the header of the statement
- Function body
- Static initialization block

Or if none of the above applies:

- The current module, for code running in module mode
- The global scope, for code running in script mode.

Compared with `var`, `let` declarations have the following differences:

- `let` declarations are scoped to blocks as well as functions.
- `let` declarations can only be accessed after the place of declaration is reached (see temporal dead zone). For this reason, `let` declarations are commonly regarded as non-hoisted.
- `let` declarations do not create properties on `globalThis` when declared at the top level of a script.
- `let` declarations cannot be redeclared by any other declaration in the same scope.
- `let` begins *declarations, not statements*. That means you cannot use a lone `let` declaration as the body of a block (which makes sense, since there's no way to access the variable).

```
if (true) let a = 1; // SyntaxError: Lexical declaration cannot appear in a single-statement context
```

Note that `let` is allowed as an identifier name when declared with `var` or `function` in non-strict mode, but you should avoid using `let` as an identifier name to prevent unexpected syntax ambiguities.

Many style guides (including MDN's) recommend using `const` over `let` whenever a variable is not reassigned in its scope. This makes the intent clear that a variable's type (or value, in the case of a primitive) can never change. Others may prefer `let` for non-primitives that are mutated.

The list that follows the `let` keyword is called a *binding list* and is separated by commas, where the commas are *not* comma operators and the `=` signs are *not* assignment operators. Initializers of later variables can refer to earlier variables in the list.

Temporal dead zone (TDZ)

A variable declared with `let`, `const`, or `class` is said to be in a "temporal dead zone" (TDZ) from the start of the block until code execution reaches the place where the variable is declared and initialized.

While inside the TDZ, the variable has not been initialized with a value, and any attempt to access it will result in a `ReferenceError`. The variable is initialized with a value when execution reaches the place in the code where it was declared. If no initial value was specified with the variable declaration, it will be initialized with a value of `undefined`.

This differs from `var` variables, which will return a value of `undefined` if they are accessed before they are declared. The code below demonstrates the different result when `let` and `var` are accessed in code before the place where they are declared.

```
{
  // TDZ starts at beginning of scope
  console.log(bar); // "undefined"
  console.log(foo); // ReferenceError: Cannot access 'foo' before initialization
  var bar = 1;
  let foo = 2; // End of TDZ (for foo)
}
```

The term "temporal" is used because the zone depends on the order of execution (time) rather than the order in which the code is written (position). For example, the code below works because, even though the function that uses the `let` variable appears before the variable is declared, the function is *called* outside the TDZ.

```
{
  // TDZ starts at beginning of scope
  const func = () => console.log(letVar); // OK

  // Within the TDZ letVar access throws `ReferenceError`

  let letVar = 3; // End of TDZ (for letVar)
  func(); // Called outside TDZ!
}
```

Using the `typeof` operator for a variable in its TDZ will throw a `ReferenceError`:

```
{
  typeof i; // ReferenceError: Cannot access 'i' before initialization
  let i = 10;
}
```

This differs from using `typeof` for undeclared variables, and variables that hold a value of `undefined`:

```
console.log(typeof undeclaredVariable); // "undefined"
```

> **Note:** `let` and `const` declarations are only processed when the current script gets processed. If you have two `<script>` elements running in script mode within one HTML, the first script is not subject to the TDZ restrictions for top-level `let` or `const` variables declared in the second script, although if you declare a `let` or `const` variable in the first script, declaring it again in the second script will cause a redeclaration error.

Redeclarations

`let` declarations cannot be in the same scope as any other declaration, including `let`, `const`, `class`, `function`, `var`, and `import` declaration.

```
{
  let foo;
  let foo; // SyntaxError: Identifier 'foo' has already been declared
}
```

A `let` declaration within a function's body cannot have the same name as a parameter. A `let` declaration within a `catch` block cannot have the same name as the `catch`-bound identifier.

```
function foo(a) {
  let a = 1; // SyntaxError: Identifier 'a' has already been declared
}
try {
} catch (e) {
  let e; // SyntaxError: Identifier 'e' has already been declared
}
```

If you're experimenting in a REPL, such as the Firefox web console (**Tools** > **Web Developer** > **Web Console**), and you run two `let` declarations with the same name in two separate inputs, you may get the same re-declaration error. See further discussion of this issue in Firefox bug 1580891. The Chrome console allows `let` re-declarations between different REPL inputs.

You may encounter errors in `switch` statements because there is only one block.

```
let x = 1;

switch (x) {
  case 0:
    let foo;
    break;
  case 1:
    let foo; // SyntaxError: Identifier 'foo' has already been declared
    break;
}
```

To avoid the error, wrap each `case` in a new block statement.

```
let x = 1;

switch (x) {
  case 0: {
    let foo;
    break;
  }
  case 1: {
    let foo;
    break;
  }
}
```

Examples

Scoping rules

Variables declared by `let` have their scope in the block for which they are declared, as well as in any contained sub-blocks. In this way, `let` works very much like `var`. The main difference is that the scope of a `var` variable is the entire enclosing function:

```
function varTest() {
  var x = 1;
  {
    var x = 2; // same variable!
    console.log(x); // 2
  }
  console.log(x); // 2
}

function letTest() {
  let x = 1;
  {
    let x = 2; // different variable
    console.log(x); // 2
  }
  console.log(x); // 1
}
```

At the top level of programs and functions, `let`, unlike `var`, does not create a property on the global object. For example:

```
var x = "global";
let y = "global";
console.log(this.x); // "global"
console.log(this.y); // undefined
```

TDZ combined with lexical scoping

The following code results in a `ReferenceError` at the line shown:

```
function test() {
  var foo = 33;
  if (foo) {
    let foo = foo + 55; // ReferenceError
  }
}
test();
```

The `if` block is evaluated because the outer `var foo` has a value. However due to lexical scoping this value is not available inside the block: the identifier `foo` *inside* the `if` block is the `let foo`. The expression `foo + 55` throws a `ReferenceError` because initialization of `let foo` has not completed — it is still in the temporal dead zone.

This phenomenon can be confusing in a situation like the following. The instruction `let n of n.a` is already inside the scope of the `for...of` loop's block. So, the identifier `n.a` is resolved to the property `a` of the `n` object located in the first part of the instruction itself (`let n`). This is still in the temporal dead zone as its declaration statement has not been reached and terminated.

```
function go(n) {
  // n here is defined!
  console.log(n); // { a: [1, 2, 3] }

  for (let n of n.a) {
    //          ^ ReferenceError
    console.log(n);
  }
}

go({ a: [1, 2, 3] });
```

Other situations

When used inside a block, `let` limits the variable's scope to that block. Note the difference between `var`, whose scope is inside the function where it is declared.

```
var a = 1;
var b = 2;

{
  var a = 11; // the scope is global
  let b = 22; // the scope is inside the block

  console.log(a); // 11
  console.log(b); // 22
}

console.log(a); // 11
console.log(b); // 2
```

However, this combination of `var` and `let` declarations below is a `SyntaxError` because `var` not being block-scoped, leading to them being in the same scope. This results in an implicit re-declaration of the variable.

```
let x = 1;

{
  var x = 2; // SyntaxError for re-declaration
}
```

Declaration with destructuring

The left-hand side of each `=` can also be a binding pattern. This allows creating multiple variables at once.

```
const result = /(a+)(b+)(c+)/.exec("aaabcc");
let [, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

See also

- `var`
- `const`
- Hoisting
- ES6 In Depth: `let` and `const` on hacks.mozilla.org (2015)
- Breaking changes in `let` and `const` in Firefox 44 on blog.mozilla.org (2015)
- You Don't Know JS: Scope & Closures, Ch.3: Function vs. Block Scope by Kyle Simpson
- What is the Temporal Dead Zone? on Stack Overflow
- What is the difference between using `let` and `var`? on Stack Overflow
- Why was the name 'let' chosen for block-scoped variable declarations in JavaScript? on Stack Overflow

return

The **`return`** statement ends function execution and specifies a value to be returned to the function caller.

Try it

```
function getRectArea(width, height) {
  if (width > 0 && height > 0) {
    return width * height;
  }
  return 0;
}

console.log(getRectArea(3, 4));
// Expected output: 12

console.log(getRectArea(-3, 4));
// Expected output: 0
```

Syntax

```
return;
return expression;
```

`expression` Optional

The expression whose value is to be returned. If omitted, `undefined` is returned.

Description

The `return` statement can only be used within function bodies. When a `return` statement is used in a function body, the execution of the function is stopped. The `return` statement has different effects when placed in different functions:

- In a plain function, the call to that function evaluates to the return value.
- In an async function, the produced promise is resolved with the returned value.
- In a generator function, the produced generator object's `next()` method returns `{ done: true, value: returnedValue }`.
- In an async generator function, the produced async generator object's `next()` method returns a promise fulfilled with `{ done: true, value: returnedValue }`.

If a `return` statement is executed within a `try` block, its `finally` block, if present, is first executed, before the value is actually returned.

Automatic semicolon insertion

The syntax forbids line terminators between the `return` keyword and the expression to be returned.

```
return
a + b;
```

The code above is transformed by automatic semicolon insertion (ASI) into:

```
return;
a + b;
```

This makes the function return `undefined` and the `a + b` expression is never evaluated. This may generate a warning in the console.

To avoid this problem (to prevent ASI), you could use parentheses:

```
return (
  a + b
);
```

Examples

Interrupt a function

A function immediately stops at the point where `return` is called.

```
function counter() {
  // Infinite loop
  for (let count = 1; ; count++) {
    console.log(`${count}A`); // Until 5
    if (count === 5) {
      return;
    }
    console.log(`${count}B`); // Until 4
  }
  console.log(`${count}C`); // Never appears
}

counter();

// Logs:
// 1A
// 1B
// 2A
// 2B
// 3A
// 3B
// 4A
// 4B
// 5A
```

Returning a function

See also the article about Closures.

```
function magic() {
  return function calc(x) {
    return x * 42;
  };
}

const answer = magic();
answer(1337); // 56154
```

See also

- Functions
- Closures

switch

The **`switch`** statement evaluates an expression, matching the expression's value against a series of `case` clauses, and executes statements after the first `case` clause with a matching value, until a `break` statement is encountered. The `default` clause of a `switch` statement will be jumped to if no `case` matches the expression's value.

Try it

```
const expr = "Papayas";
switch (expr) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Mangoes":
  case "Papayas":
    console.log("Mangoes and papayas are $2.79 a pound.");
    // Expected output: "Mangoes and papayas are $2.79 a pound."
    break;
  default:
    console.log(`Sorry, we are out of ${expr}.`);
}
```

Syntax

```
switch (expression) {
  case caseExpression1:
    statements
  case caseExpression2:
    statements
  // …
  case caseExpressionN:
    statements
  default:
    statements
}
```

`expression`

An expression whose result is matched against each `case` clause.

`caseExpressionN` Optional

A `case` clause used to match against `expression`. If the value of `expression` matches the value of any `caseExpressionN`, execution starts from the first statement after that `case` clause until either the end of the `switch` statement or the first encountered `break`.

`default` Optional

A `default` clause; if provided, this clause is executed if the value of `expression` doesn't match any of the `case` clauses. A `switch` statement can only have one `default` clause.

Description

A `switch` statement first evaluates its expression. It then looks for the first `case` clause whose expression evaluates to the same value as the result of the input expression (using the strict equality comparison) and transfers control to that clause, executing all statements following that clause.

The clause expressions are only evaluated when necessary — if a match is already found, subsequent `case` clause expressions will not be evaluated, even when they will be visited by fall-through.

```
switch (undefined) {
  case console.log(1):
  case console.log(2):
}
// Only logs 1
```

If no matching `case` clause is found, the program looks for the optional `default` clause, and if found, transfers control to that clause, executing statements following that clause. If no `default` clause is found, the program continues execution at the statement following the end of `switch`. By convention, the `default` clause is the last clause, but it does not need to be so. A `switch` statement may only have one `default` clause; multiple `default` clauses will result in a `SyntaxError`.

Breaking and fall-through

You can use the `break` statement within a `switch` statement's body to break out early, often when all statements between two `case` clauses have been executed. Execution will continue at the first statement following `switch`.

If `break` is omitted, execution will proceed to the next `case` clause, even to the `default` clause, regardless of whether the value of that clause's expression matches. This behavior is called "fall-through".

```
const foo = 0;
switch (foo) {
  case -1:
```

```
    console.log("negative 1");
    break;
  case 0: // Value of foo matches this criteria; execution starts from here
    console.log(0);
  // Forgotten break! Execution falls through
  case 1: // no break statement in 'case 0:' so this case will run as well
    console.log(1);
    break; // Break encountered; will not continue into 'case 2:'
  case 2:
    console.log(2);
    break;
  default:
    console.log("default");
}
// Logs 0 and 1
```

In the appropriate context, other control-flow statements also have the effect of breaking out of the `switch` statement. For example, if the `switch` statement is contained in a function, then a `return` statement terminates the execution of the function body and therefore the `switch` statement. If the `switch` statement is contained in a loop, then a `continue` statement stops the `switch` statement and jumps to the next iteration of the loop.

Lexical scoping

The `case` and `default` clauses are like labels: they indicate possible places that control flow may jump to. However, they don't create lexical scopes themselves (neither do they automatically break out — as demonstrated above). For example:

```
const action = "say_hello";
switch (action) {
  case "say_hello":
    const message = "hello";
    console.log(message);
    break;
  case "say_hi":
    const message = "hi";
    console.log(message);
    break;
  default:
    console.log("Empty action received.");
}
```

This example will output the error "Uncaught SyntaxError: Identifier 'message' has already been declared", because the first `const message = 'hello';` conflicts with the second `const message = 'hi';` declaration, even when they're within their own separate case clauses. Ultimately, this is due to both `const` declarations being within the same block scope created by the `switch` body.

To fix this, whenever you need to use `let` or `const` declarations in a `case` clause, wrap it in a block.

```
const action = "say_hello";
switch (action) {
  case "say_hello": {
    const message = "hello";
    console.log(message);
    break;
  }
  case "say_hi": {
    const message = "hi";
    console.log(message);
    break;
  }
  default: {
    console.log("Empty action received.");
  }
}
```

This code will now output `hello` in the console as it should, without any errors.

Examples

Using switch

In the following example, if `expr` evaluates to `Bananas`, the program matches the value with case `case 'Bananas'` and executes the associated statement. When `break` is encountered, the program breaks out of `switch` and executes the statement following `switch`. If `break` were omitted, the statement for the `case 'Cherries'` would also be executed.

```
switch (expr) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Apples":
    console.log("Apples are $0.32 a pound.");
    break;
  case "Bananas":
    console.log("Bananas are $0.48 a pound.");
    break;
  case "Cherries":
```

```
    console.log("Cherries are $3.00 a pound.");
    break;
  case "Mangoes":
  case "Papayas":
    console.log("Mangoes and papayas are $2.79 a pound.");
    break;
  default:
    console.log(`Sorry, we are out of ${expr}.`);
}

console.log("Is there anything else you'd like?");
```

Putting the default clause between two case clauses

If no match is found, execution will start from the `default` clause, and execute all statements after that.

```
const foo = 5;
switch (foo) {
  case 2:
    console.log(2);
    break; // it encounters this break so will not continue into 'default:'
  default:
    console.log("default");
  // fall-through
  case 1:
    console.log("1");
}
```

It also works when you put `default` before all other `case` clauses.

Taking advantage of fall-through

This method takes advantage of the fact that if there is no `break` below a `case` clause, execution will continue to the next `case` clause regardless if that `case` meets the criteria.

The following is an example of a single operation sequential `case` statement, where four different values perform exactly the same.

```
const Animal = "Giraffe";
switch (Animal) {
  case "Cow":
  case "Giraffe":
  case "Dog":
  case "Pig":
    console.log("This animal is not extinct.");
    break;
  case "Dinosaur":
  default:
    console.log("This animal is extinct.");
}
```

The following is an example of a multiple-operation sequential `case` clause, where, depending on the provided integer, you can receive different output. This shows you that it will traverse in the order that you put the `case` clauses, and it does not have to be numerically sequential. In JavaScript, you can even mix in definitions of strings into these `case` statements as well.

```
const foo = 1;
let output = "Output: ";
switch (foo) {
  case 0:
    output += "So ";
  case 1:
    output += "What ";
    output += "Is ";
  case 2:
    output += "Your ";
  case 3:
    output += "Name";
  case 4:
    output += "?";
    console.log(output);
    break;
  case 5:
    output += "!";
    console.log(output);
    break;
  default:
    console.log("Please pick a number from 0 to 5!");
}
```

The output from this example:

Value	Log text
`foo` is `NaN` or not `1`, `2`, `3`, `4`, `5`, or `0`	Please pick a number from 0 to 5!

Value	Log text
`0`	Output: So What Is Your Name?
`1`	Output: What Is Your Name?
`2`	Output: Your Name?
`3`	Output: Name?
`4`	Output: ?
`5`	Output: !

An alternative to if...else chains

You may often find yourself doing a series of `if...else` matches.

```
if ("fetch" in globalThis) {
  // Fetch a resource with fetch
} else if ("XMLHttpRequest" in globalThis) {
  // Fetch a resource with XMLHttpRequest
} else {
  // Fetch a resource with some custom AJAX logic
}
```

This pattern is not doing a sequence of `===` comparisons, but you can still convert it to a `switch` construct.

```
switch (true) {
  case "fetch" in globalThis:
    // Fetch a resource with fetch
    break;
  case "XMLHttpRequest" in globalThis:
    // Fetch a resource with XMLHttpRequest
    break;
  default:
    // Fetch a resource with some custom AJAX logic
    break;
}
```

The `switch (true)` pattern as an alternative to `if...else` is especially useful if you want to utilize the fall-through behavior.

```
switch (true) {
  case isSquare(shape):
    console.log("This shape is a square.");
  // Fall-through, since a square is a rectangle as well!
  case isRectangle(shape):
    console.log("This shape is a rectangle.");
  case isQuadrilateral(shape):
    console.log("This shape is a quadrilateral.");
    break;
  case isCircle(shape):
    console.log("This shape is a circle.");
    break;
}
```

See also

- `if...else`

throw

The **`throw`** statement throws a user-defined exception. Execution of the current function will stop (the statements after `throw` won't be executed), and control will be passed to the first `catch` block in the call stack. If no `catch` block exists among caller functions, the program will terminate.

Try it

```
function getRectArea(width, height) {
  if (isNaN(width) || isNaN(height)) {
    throw new Error("Parameter is not a number!");
  }
}

try {
  getRectArea(3, "A");
} catch (e) {
  console.error(e);
  // Expected output: Error: Parameter is not a number!
}
```

Syntax

```
throw expression;
```

`expression`

The expression to throw.

Description

The `throw` statement is valid in all contexts where statements can be used. Its execution generates an exception that penetrates through the call stack. For more information on error bubbling and handling, see Control flow and error handling.

The `throw` keyword can be followed by any kind of expression, for example:

```
throw error; // Throws a previously defined value (e.g. within a catch block)
throw new Error("Required"); // Throws a new Error object
```

In practice, the exception you throw should *always* be an `Error` object or an instance of an `Error` subclass, such as `RangeError`. This is because code that catches the error may expect certain properties, such as `message`, to be present on the caught value. For example, web APIs typically throw `DOMException` instances, which inherit from `Error.prototype`.

Automatic semicolon insertion

The syntax forbids line terminators between the `throw` keyword and the expression to be thrown.

```
throw
new Error();
```

The code above is transformed by automatic semicolon insertion (ASI) into:

```
throw;
new Error();
```

This is invalid code, because unlike `return`, `throw` must be followed by an expression.

To avoid this problem (to prevent ASI), you could use parentheses:

```
throw (
  new Error()
);
```

Examples

Throwing a user-defined error

This example defines a function that throws a `TypeError` if the input is not of the expected type.

```
function isNumeric(x) {
  return ["number", "bigint"].includes(typeof x);
}

function sum(...values) {
  if (!values.every(isNumeric)) {
    throw new TypeError("Can only add numbers");
  }
  return values.reduce((a, b) => a + b);
}

console.log(sum(1, 2, 3)); // 6
try {
  sum("1", "2");
} catch (e) {
  console.error(e); // TypeError: Can only add numbers
}
```

Throwing an existing object

This example calls a callback-based async function, and throws an error if the callback receives an error.

```
readFile("foo.txt", (err, data) => {
  if (err) {
    throw err;
  }
  console.log(data);
});
```

Errors thrown this way are not catchable by the caller and will cause the program to crash unless (a) the `readFile` function itself catches the error, or (b) the program is running in a context that catches top-level errors. You can handle errors more naturally by using the `Promise()` constructor.

```
function readFilePromise(path) {
  return new Promise((resolve, reject) => {
    readFile(path, (err, data) => {
      if (err) {
        reject(err);
      }
      resolve(data);
    });
```

```
  });
}

try {
  const data = await readFilePromise("foo.txt");
  console.log(data);
} catch (err) {
  console.error(err);
}
```

See also

- `try...catch`
- `Error`

try...catch

The **`try...catch`** statement is comprised of a `try` block and either a `catch` block, a `finally` block, or both. The code in the `try` block is executed first, and if it throws an exception, the code in the `catch` block will be executed. The code in the `finally` block will always be executed before control flow exits the entire construct.

Try it

```
try {
  nonExistentFunction();
} catch (error) {
  console.error(error);
  // Expected output: ReferenceError: nonExistentFunction is not defined
  // (Note: the exact output may be browser-dependent)
}
```

Syntax

```
try {
  tryStatements
} catch (exceptionVar) {
  catchStatements
} finally {
  finallyStatements
}
```

`tryStatements`

The statements to be executed.

`catchStatements`

Statement that is executed if an exception is thrown in the `try` block.

`exceptionVar` Optional

An optional identifier or pattern to hold the caught exception for the associated `catch` block. If the `catch` block does not use the exception's value, you can omit the `exceptionVar` and its surrounding parentheses.

`finallyStatements`

Statements that are executed before control flow exits the `try...catch...finally` construct. These statements execute regardless of whether an exception was thrown or caught.

Description

The `try` statement always starts with a `try` block. Then, a `catch` block or a `finally` block must be present. It's also possible to have both `catch` and `finally` blocks. This gives us three forms for the `try` statement:

- `try...catch`
- `try...finally`
- `try...catch...finally`

Unlike other constructs such as `if` or `for`, the `try`, `catch`, and `finally` blocks must be *blocks*, instead of single statements.

```
try doSomething(); // SyntaxError
catch (e) console.log(e);
```

A `catch` block contains statements that specify what to do if an exception is thrown in the `try` block. If any statement within the `try` block (or in a function called from within the `try` block) throws an exception, control is immediately shifted to the `catch` block. If no exception is thrown in the `try` block, the `catch` block is skipped.

The `finally` block will always execute before control flow exits the `try...catch...finally` construct. It always executes, regardless of whether an exception was thrown or caught.

You can nest one or more `try` statements. If an inner `try` statement does not have a `catch` block, the enclosing `try` statement's `catch` block is used instead.

You can also use the `try` statement to handle JavaScript exceptions. See the JavaScript Guide for more information on JavaScript exceptions.

Catch binding

When an exception is thrown in the `try` block, `exceptionVar` (i.e., the `e` in `catch (e)`) holds the exception value. You can use this binding to get information about the exception that was thrown. This binding is only available in the `catch` block's scope.

It doesn't need to be a single identifier. You can use a destructuring pattern to assign multiple identifiers at once.

```
try {
  throw new TypeError("oops");
} catch ({ name, message }) {
  console.log(name); // "TypeError"
  console.log(message); // "oops"
}
```

The bindings created by the `catch` clause live in the same scope as the `catch` block, so any variables declared in the `catch` block cannot have the same name as the bindings created by the `catch` clause. (There's one exception to this rule, but it's a deprecated syntax.)

```
try {
  throw new TypeError("oops");
} catch ({ name, message }) {
  var name; // SyntaxError: Identifier 'name' has already been declared
  let message; // SyntaxError: Identifier 'message' has already been declared
}
```

The exception binding is writable. For example, you may want to normalize the exception value to make sure it's an `Error` object.

```
try {
  throw "Oops; this is not an Error object";
} catch (e) {
  if (!(e instanceof Error)) {
    e = new Error(e);
  }
  console.error(e.message);
}
```

If you don't need the exception value, you can omit it along with the enclosing parentheses.

```
function isValidJSON(text) {
  try {
    JSON.parse(text);
    return true;
  } catch {
    return false;
  }
}
```

The finally block

The `finally` block contains statements to execute after the `try` block and `catch` block(s) execute, but before the statements following the `try...catch...finally` block. Control flow will always enter the `finally` block, which can proceed in one of the following ways:

- Immediately after the control flow exits the `try` block in a `try...finally` construct (either after the last statement or a `throw`, `return`, `break`, or `continue` statement);
- Immediately after the control flow exits the `catch` block in a `try...catch...finally` construct;
- Immediately after the control flow exits the `try` block in a `try...catch...finally` construct, unless it exits via a `throw` statement (in which case control flow enters the `catch` block first).

If the `finally` block is entered after a control flow statement (`return`, `throw`, `break`, `continue`) in the `try` or `catch` block, the effect of this statement is deferred until after the last statement executed in the `finally` block. For example, if an exception is thrown from the `try` block, even when there's no `catch` block to handle the exception, the `finally` block still executes, and the exception is thrown immediately after the `finally` block finishes executing.

However, there is one exception to this rule: if the last statement executed in the `finally` block is itself a control flow statement, that statement will override the effect of the previous one (no deferral); see returning from a `finally` block for examples. It is generally a bad idea to use control flow statements (`return`, `throw`, `break`, `continue`) in the `finally` block because they can override the effect of previously executed control flow statements, which is rarely intended. Most of the time, the `finally` block should be reserved for cleanup code that does not modify the main logic.

Examples

Unconditional catch block

When a `catch` block is used, the `catch` block is executed when any exception is thrown from within the `try` block. For example, when the exception occurs in the following code, control transfers to the `catch` block.

```
try {
  throw new Error("My exception"); // generates an exception
} catch (e) {
  // statements to handle any exceptions
  logMyErrors(e); // pass exception object to error handler
}
```

The `catch` block specifies an identifier (`e` in the example above) that holds the value of the exception; this value is only available in the scope of the `catch` block.

Conditional catch blocks

You can create "Conditional `catch` blocks" by combining `try...catch` blocks with `if...else if...else` structures, like this:

```
try {
  myRoutine(); // may throw three types of exceptions
} catch (e) {
  if (e instanceof TypeError) {
    // statements to handle TypeError exceptions
  } else if (e instanceof RangeError) {
    // statements to handle RangeError exceptions
  } else if (e instanceof EvalError) {
    // statements to handle EvalError exceptions
  } else {
    // statements to handle any unspecified exceptions
    logMyErrors(e); // pass exception object to error handler
  }
}
```

A common use case for this is to only catch (and silence) a small subset of expected errors, and then re-throw the error in other cases:

```
try {
  myRoutine();
} catch (e) {
  if (e instanceof RangeError) {
    // statements to handle this very common expected error
  } else {
    throw e; // re-throw the error unchanged
  }
}
```

This may mimic the syntax from other languages, like Java:

```
try {
  myRoutine();
} catch (RangeError e) {
  // statements to handle this very common expected error
}
// Other errors are implicitly re-thrown
```

Nested try blocks

First, let's see what happens with this:

```
try {
  try {
    throw new Error("oops");
  } finally {
    console.log("finally");
  }
} catch (ex) {
  console.error("outer", ex.message);
}

// Logs:
// "finally"
// "outer" "oops"
```

Now, if we already caught the exception in the inner `try` block by adding a `catch` block:

```
try {
  try {
    throw new Error("oops");
  } catch (ex) {
    console.error("inner", ex.message);
  } finally {
    console.log("finally");
  }
} catch (ex) {
  console.error("outer", ex.message);
}

// Logs:
// "inner" "oops"
// "finally"
```

And now, let's rethrow the error.

```
try {
  try {
    throw new Error("oops");
  } catch (ex) {
    console.error("inner", ex.message);
    throw ex;
  } finally {
```

```
    console.log("finally");
  }
} catch (ex) {
  console.error("outer", ex.message);
}

// Logs:
// "inner" "oops"
// "finally"
// "outer" "oops"
```

Any given exception will be caught only once by the nearest enclosing `catch` block unless it is rethrown. Of course, any new exceptions raised in the "inner" block (because the code in `catch` block may do something that throws), will be caught by the "outer" block.

Resource cleanup using finally

The following example shows one use case for the `finally` block. The code opens a file and then executes statements that use the file; the `finally` block makes sure the file always closes after it is used even if an exception was thrown.

```
openMyFile();
try {
  // tie up a resource
  writeMyFile(theData);
} finally {
  closeMyFile(); // always close the resource
  // any uncaught exception is deferred here
}
```

In the same way, the effect of any `return` statement in the `try` block is deferred at the end of the `finally` block, although the return value expression is evaluated before entering the `finally` block.

```
function safeWriteMyFile() {
  openMyFile();
  try {
    return writeMyFile(theData); // function call is evaluated
  } finally {
    closeMyFile(); // always close the resource
    // return is deferred here
  }
}
```

Returning from a finally block

The following example illustrates how control flow statements in the `finally` block behave. When control flow exits the `try` block via the first `return` statement, the return value expression (`order.sort()`) is evaluated before entering the `finally` block, and the function is planned to return that value after the `finally` block finishes executing. However, the `return` statement in the `finally` block overrides the effect of the previous `return` statement, including its return value.

```
function doIt() {
  const order = ["z"];
  try {
    order.push("try");
    return order.sort(); // "z" is now after "try"
  } finally {
    order.push("finally");
    return order;
  }
}
doIt();
// returns ["try", "z", "finally"], not ["finally", "try", "z"] or ["try", "z"]
```

The same logic applies to other control flow statements. Here, the function is first planned to throw the value `"catch"`, but instead returns the value `"finally"`.

```
function doIt() {
  try {
    throw "try"; // makes control flow enter the `catch` block
  } catch {
    throw "catch"; // makes control flow enter the `finally` block
  } finally {
    return "finally"; // returns "finally" instead of throwing "catch"
  }
}
doIt(); // returns "finally"
```

Again, control flow statements are discouraged in the `finally` block because this effect is likely not intended.

See also

- `Error`
- `throw`

using

The **`using`** declaration declares block-scoped local variables that are *synchronously disposed*. Like `const`, variables declared with `using` must be initialized and cannot be reassigned. The variable's value must be either `null`, `undefined`, or an object with a `[Symbol.dispose]()` method. When the variable goes out of scope, the `[Symbol.dispose]()` method of the object is called, to ensure that resources are freed.

Syntax

```
using name1 = value1;
using name1 = value1, name2 = value2;
using name1 = value1, name2 = value2, /* …, */ nameN = valueN;
```

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier and *not* a destructuring binding pattern.

`valueN`

Initial value of the variable. It can be any legal expression but its value must be either `null`, `undefined`, or an object with a `[Symbol.dispose]()` method.

Description

This declaration can be used:

- Inside a block
- Inside any function body or class static initialization block
- At the top level of a module
- In the initializer of a `for`, `for...of`, or `for await...of` loop

Most notably, it cannot be used:

- At the top level of a script, because script scopes are persistent.
- At the top level of a `switch` statement.
- In the initializer of a `for...in` loop. Because the loop variable can only be a string or symbol, this doesn't make sense.

A `using` declares a disposable resource that's tied to the lifetime of the variable's scope (block, function, module, etc.). When the scope exits, the resource is disposed of synchronously. The variable is allowed to have value `null` or `undefined`, so the resource can be optionally present.

When the variable is first declared and its value is non-nullish, a *disposer* is retrieved from the object. If the `[Symbol.dispose]` property doesn't contain a function, a `TypeError` is thrown. This disposer is saved to the scope.

When the variable goes out of scope, the disposer is called. If the scope contains multiple `using` or `await using` declarations, all disposers are run in the reverse order of declaration, regardless of the type of declaration. All disposers are guaranteed to run (much like the `finally` block in `try...catch...finally`). All errors thrown during disposal, including the initial error that caused the scope exit (if applicable), are all aggregated inside one `SuppressedError`, with each earlier exception as the `suppressed` property and the later exception as the `error` property. This `SuppressedError` is thrown after disposal is complete.

`using` ties resource management to lexical scopes, which is both convenient and sometimes confusing. There are many ways to preserve the variable's value when the variable itself is out of scope, so you may hold a reference to an already-disposed resource. See below for some examples where it may not behave how you expect. If you want to hand-manage resource disposal, while maintaining the same error handling guarantees, you can use `DisposableStack` instead.

Examples

In the following examples, we assume a simple `Resource` class that has a `getValue` method and a `[Symbol.dispose]()` method:

```
class Resource {
  value = Math.random();
  #isDisposed = false;

  getValue() {
    if (this.#isDisposed) {
      throw new Error("Resource is disposed");
    }
    return this.value;
  }

  [Symbol.dispose]() {
    this.#isDisposed = true;
    console.log("Resource disposed");
  }
}
```

<code>using</code> in a block

The resource declared with `using` is disposed when exiting the block.

```
{
  using resource = new Resource();
  console.log(resource.getValue());
  // resource disposed here
}
```

<code>using</code> in a function

You can use `using` in a function body. In this case, the resource is disposed when the function finishes executing, immediately before the function returns.

```
function example() {
  using resource = new Resource();
  return resource.getValue();
}
```

Here, `resource[Symbol.dispose]()` will be called after `getValue()`, before the `return` statement executes.

The resource may outlive the declaration, in case it's captured by a closure:

```
function example() {
  using resource = new Resource();
  return () => resource.getValue();
}
```

In this case, if you call `example()()`, you will always execute `getValue` on a resource that's already disposed, because the resource was disposed when `example` returns. In case you want to dispose the resource immediately after the callback has been called once, consider this pattern:

```
function example() {
  const resource = new Resource();
  return () => {
    using resource2 = resource;
    return resource2.getValue();
  };
}
```

Here, we *alias* a `const`-declared resource to a `using`-declared resource, so that the resource is only disposed after the callback is called; note that if it is never called then the resource will never be cleaned up.

<code>using</code> in a module

You can use `using` at the top level of a module. In this case, the resource is disposed when the module finishes executing.

```
using resource = new Resource();
export const value = resource.getValue();
// resource disposed here
```

`export using` is invalid syntax, but you can `export` a variable declared elsewhere using `using`:

```
using resource = new Resource();
export { resource };
```

This is still discouraged, because the importer will always receive a disposed resource. Similar to the closure problem, this causes the value of resource to live longer than the variable.

<code>using</code> with <code>for...of</code>

You can use `using` in the initializer of a `for...of` loop. In this case, the resource is disposed on every loop iteration.

```
const resources = [new Resource(), new Resource(), new Resource()];
for (using resource of resources) {
  console.log(resource.getValue());
  // resource disposed here
}
```

Multiple <code>using</code>

The following are two equivalent ways to declare multiple disposable resources:

```
using resource1 = new Resource(),
  resource2 = new Resource();

// OR

using resource1 = new Resource();
using resource2 = new Resource();
```

In both cases, when the scope exits, `resource2` is disposed before `resource1`. This is because `resource2` may have a dependency on `resource1`, so it's disposed first to ensure that `resource1` is still available when `resource2` is disposed.

Optional <code>using</code>

`using` allows the variable to have a value of `null` or `undefined`, so the resource can be optionally present. This means that, for example, if you are checking the availability of a resource of some kind:

```
function acquireResource() {
  // Imagine some real-world relevant condition here,
  // such as whether there's space to allocate for this resource
  if (Math.random() < 0.5) {
    return null;
  }
```

```
  return new Resource();
}
```

You don't have to do this:

```
const maybeResource = acquireResource();

if (maybeResource) {
  using resource = maybeResource;
  console.log(resource.getValue());
} else {
  console.log(undefined);
}
```

And can instead do this:

```
using resource = acquireResource();
console.log(resource?.getValue());
```

<code>using</code> declaration without using the variable

You can achieve automatic resource disposing using `using`, without even using the variable. This is very useful for setting up a context within a block, such as creating a lock:

```
{
  using _ = new Lock();
  // Perform concurrent operations here
  // Lock disposed (released) here
}
```

Note that _ is a normal identifier, but it's a convention to use it as a "throwaway" variable. To create multiple unused variables, you need to use distinct names, for example by using a variable name prefixed with _.

Initialization and temporal dead zones

`using` variables are subject to the same temporal dead zone restriction as `let` and `const` variables. This means that you can't access the variable before the initialization —the valid lifetime of the resource is strictly from its initialization to the end of its scope. This is enables RAII-style resource management.

```
let useResource;
{
  useResource = () => resource.getValue();
  useResource(); // Error: Cannot access 'resource' before initialization
  using resource = new Resource();
  useResource(); // Valid
}
useResource(); // Error: Resource is disposed
```

Error handling

The `using` declaration is the most useful for managing resource disposal in the presence of errors. If you are not careful, some resources may leak because the error prevents code afterwards from executing.

```
function handleResource(resource) {
  if (resource.getValue() > 0.5) {
    throw new Error("Resource value too high");
  }
}

try {
  using resource = new Resource();
  handleResource(resource);
} catch (e) {
  console.error(e);
}
```

This will successfully catch the error thrown by `handleResource` and log it, and no matter if `handleResource` throws an error or not, the resource is disposed before exiting the `try` block.

Here, if you don't use `using`, you may do something like:

```
try {
  const resource = new Resource();
  handleResource(resource);
  resource[Symbol.dispose]();
} catch (e) {
  console.error(e);
}
```

But, if `handleResource()` throws an error, then control never reaches `resource[Symbol.dispose]()`, and the resource is leaked. Furthermore, if you have two resources, then errors thrown in earlier disposals may prevent later disposals from running, leading to more leaks.

Consider a more complicated case where the disposer itself throws an error:

```
class CantDisposeMe {
  #name;
  constructor(name) {
    this.#name = name;
  }
  [Symbol.dispose]() {
    throw new Error(`Can't dispose ${this.#name}`);
  }
}

let error;

try {
  using resource1 = new CantDisposeMe("resource1");
  using resource2 = new CantDisposeMe("resource2");
  throw new Error("Error in main block");
} catch (e) {
  error = e;
}
```

You can inspect the error thrown in your browser's console. It has the following structure:

```
SuppressedError: An error was suppressed during disposal
  suppressed: SuppressedError: An error was suppressed during disposal
    suppressed: Error: Can't dispose resource1
    error: Error: Error in main block
  error: Error: Can't dispose resource2
```

As you can see, `error` contains all the errors that were thrown during disposal, as a `SuppressedError`. Each additional error is added as the `error` property, and the original error is added as the `suppressed` property.

See also

- JavaScript resource management
- `const`
- `let`
- `await using`
- `Symbol.dispose`
- `DisposableStack`

var

The **`var`** statement declares function-scoped or globally-scoped variables, optionally initializing each to a value.

Try it

```
var x = 1;

if (x === 1) {
  var x = 2;

  console.log(x);
  // Expected output: 2
}

console.log(x);
// Expected output: 2
```

Syntax

```
var name1;
var name1 = value1;
var name1 = value1, name2 = value2;
var name1, name2 = value2;
var name1 = value1, name2, /* …, */ nameN = valueN;
```

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier or a destructuring binding pattern.

`valueN` Optional

Initial value of the variable. It can be any legal expression. Default value is `undefined`.

Description

The scope of a variable declared with `var` is one of the following curly-brace-enclosed syntaxes that most closely contains the `var` statement:

- Function body
- Static initialization block

Or if none of the above applies:

- The current module, for code running in module mode
- The global scope, for code running in script mode.

```
function foo() {
  var x = 1;
  function bar() {
    var y = 2;
    console.log(x); // 1 (function `bar` closes over `x`)
    console.log(y); // 2 (`y` is in scope)
  }
  bar();
  console.log(x); // 1 (`x` is in scope)
  console.log(y); // ReferenceError, `y` is scoped to `bar`
}

foo();
```

Importantly, other block constructs, including block statements, `try...catch`, `switch`, headers of one of the `for` statements, do not create scopes for `var`, and variables declared with `var` inside such a block can continue to be referenced outside the block.

```
for (var a of [1, 2, 3]);
console.log(a); // 3
```

In a script, a variable declared using `var` is added as a non-configurable property of the global object. This means its property descriptor cannot be changed and it cannot be deleted using `delete`. JavaScript has automatic memory management, and it would make no sense to be able to use the `delete` operator on a global variable.

```
"use strict";
var x = 1;
Object.hasOwn(globalThis, "x"); // true
delete globalThis.x; // TypeError in strict mode. Fails silently otherwise.
delete x; // SyntaxError in strict mode. Fails silently otherwise.
```

In both Node.js CommonJS modules and native ECMAScript modules, top-level variable declarations are scoped to the module, and are not added as properties to the global object.

The list that follows the `var` keyword is called a *binding list* and is separated by commas, where the commas are *not* comma operators and the `=` signs are *not* assignment operators. Initializers of later variables can refer to earlier variables in the list and get the initialized value.

Hoisting

`var` declarations, wherever they occur in a script, are processed before any code within the script is executed. Declaring a variable anywhere in the code is equivalent to declaring it at the top. This also means that a variable can appear to be used before it's declared. This behavior is called *hoisting*, as it appears that the variable declaration is moved to the top of the function, static initialization block, or script source in which it occurs.

> **Note:** `var` declarations are only hoisted to the top of the current script. If you have two `<script>` elements within one HTML, the first script cannot access variables declared by the second before the second script has been processed and executed.

```
bla = 2;
var bla;
```

This is implicitly understood as:

```
var bla;
bla = 2;
```

For that reason, it is recommended to always declare variables at the top of their scope (the top of global code and the top of function code) so it's clear which variables are scoped to the current function.

Only a variable's declaration is hoisted, not its initialization. The initialization happens only when the assignment statement is reached. Until then the variable remains `undefined` (but declared):

```
function doSomething() {
  console.log(bar); // undefined
  var bar = 111;
  console.log(bar); // 111
}
```

This is implicitly understood as:

```
function doSomething() {
  var bar;
  console.log(bar); // undefined
  bar = 111;
  console.log(bar); // 111
}
```

Redeclarations

Duplicate variable declarations using `var` will not trigger an error, even in strict mode, and the variable will not lose its value, unless the declaration has an initializer.

```
var a = 1;
var a = 2;
console.log(a); // 2
var a;
console.log(a); // 2; not undefined
```

`var` declarations can also be in the same scope as a `function` declaration. Both the function declaration and the `var` declaration are hoisted to the top, so the function value is only accessible from the start of its scope until the variable's initializer or first assignment, regardless of the two declarations' relative positions in the source code.

```
console.log(typeof a); // "function"
function a() {}
var a;
console.log(typeof a); // "function"
```

```
var a = 1;
function a() {}
console.log(a); // 1
```

`var` declarations cannot be in the same scope as a `let`, `const`, `class`, or `import` declaration.

```
var a = 1;
let a = 2; // SyntaxError: Identifier 'a' has already been declared
```

Because `var` declarations are not scoped to blocks, this also applies to the following case:

```
let a = 1;
{
  var a = 1; // SyntaxError: Identifier 'a' has already been declared
}
```

It does not apply to the following case, where `let` is in a child scope of `var`, not the same scope:

```
var a = 1;
{
  let a = 2;
}
```

A `var` declaration within a function's body can have the same name as a parameter.

```
function foo(a) {
  var a = 1;
  console.log(a);
}

foo(2); // Logs 1
```

A `var` declaration within a `catch` block can have the same name as the `catch`-bound identifier, but only if the `catch` binding is a simple identifier, not a destructuring pattern. This is a deprecated syntax and you should not rely on it. In this case, the declaration is hoisted to outside the `catch` block, but any value assigned within the `catch` block is not visible outside.

```
try {
  throw new Error();
} catch (e) {
  var e = 2; // Works
}
console.log(e); // undefined
```

Examples

Declaring and initializing two variables

```
var a = 0,
  b = 0;
```

Assigning two variables with single string value

```
var a = "A";
var b = a;
```

This is equivalent to:

```
var a, b = a = "A";
```

Be mindful of the order:

```
var x = y,
  y = "A";
console.log(x, y); // undefined A
```

Here, `x` and `y` are declared before any code is executed, but the assignments occur later. At the time `x = y` is evaluated, `y` exists so no `ReferenceError` is thrown and its value is `undefined`. So, `x` is assigned the undefined value. Then, `y` is assigned the value `"A"`.

Initialization of several variables

Be careful of the `var x = y = 1` syntax — `y` is not actually declared as a variable, so `y = 1` is an unqualified identifier assignment, which creates a global variable in non-strict mode.

```
var x = 0;
function f() {
  var x = y = 1; // Declares x locally; declares y globally.
}
f();

console.log(x, y); // 0 1

// In non-strict mode:
// x is the global one as expected;
// y is leaked outside of the function, though!
```

The same example as above but with a strict mode:

```
"use strict";

var x = 0;
function f() {
  var x = y = 1; // ReferenceError: y is not defined
}
f();

console.log(x, y);
```

Implicit globals and outer function scope

Variables that appear to be implicit globals may be references to variables in an outer function scope:

```
var x = 0; // Declares x within file scope, then assigns it a value of 0.

console.log(typeof z); // "undefined", since z doesn't exist yet

function a() {
  var y = 2; // Declares y within scope of function a, then assigns it a value of 2.

  console.log(x, y); // 0 2

  function b() {
    x = 3; // Assigns 3 to existing file scoped x.
    y = 4; // Assigns 4 to existing outer y.
    z = 5; // Creates a new global variable z, and assigns it a value of 5.
    // (Throws a ReferenceError in strict mode.)
  }

  b(); // Creates z as a global variable.
  console.log(x, y, z); // 3 4 5
}

a(); // Also calls b.
console.log(x, z); // 3 5
console.log(typeof y); // "undefined", as y is local to function a
```

Declaration with destructuring

The left-hand side of each `=` can also be a binding pattern. This allows creating multiple variables at once.

```
const result = /(a+)(b+)(c+)/.exec("aaabcc");
var [, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

See also

- `let`
- `const`

while

The **`while`** statement creates a loop that executes a specified statement as long as the test condition evaluates to true. The condition is evaluated before executing the statement.

Try it

```
let n = 0;

while (n < 3) {
  n++;
}

console.log(n);
// Expected output: 3
```

Syntax

```
while (condition)
  statement
```

`condition`

> An expression evaluated *before* each pass through the loop. If this condition evaluates to true, `statement` is executed. When condition evaluates to false, execution continues with the statement after the `while` loop.

`statement`

> A statement that is executed as long as the condition evaluates to true. You can use a block statement to execute multiple statements.

Description

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and re-evaluates `condition`.

Examples

Using while

The following `while` loop iterates as long as `n` is less than three.

```
let n = 0;
let x = 0;

while (n < 3) {
  n++;
  x += n;
}
```

Each iteration, the loop increments `n` and adds it to `x`. Therefore, `x` and `n` take on the following values:

- After the first pass: `n` = 1 and `x` = 1
- After the second pass: `n` = 2 and `x` = 3
- After the third pass: `n` = 3 and `x` = 6

After completing the third pass, the condition `n < 3` is no longer true, so the loop terminates.

Using an assignment as a condition

In some cases, it can make sense to use an assignment as a condition. This comes with readability tradeoffs, so there are certain stylistic recommendations that would make the pattern more obvious for everyone.

Consider the following example, which iterates over a document's comments, logging them to the console.

```
const iterator = document.createNodeIterator(document, NodeFilter.SHOW_COMMENT);
let currentNode;
while (currentNode = iterator.nextNode()) {
  console.log(currentNode.textContent.trim());
}
```

That's not completely a good-practice example, due to the following line specifically:

```
while (currentNode = iterator.nextNode()) {
```

The *effect* of that line is fine — in that, each time a comment node is found:

1. `iterator.nextNode()` returns that comment node, which gets assigned to `currentNode`.
2. The value of `currentNode = iterator.nextNode()` is therefore truthy.
3. So the `console.log()` call executes and the loop continues.

...and then, when there are no more comment nodes in the document:

1. `iterator.nextNode()` returns `null`.
2. The value of `currentNode = iterator.nextNode()` is therefore also `null`, which is falsy.
3. So the loop ends.

The problem with this line is: conditions typically use comparison operators such as `===`, but the `=` in that line isn't a comparison operator — instead, it's an assignment operator. So that `=` *looks like* it's a typo for `===` — even though it's *not* actually a typo.

Therefore, in cases like that one, some code-linting tools such as ESLint's `no-cond-assign` rule — in order to help you catch a possible typo so that you can fix it — will report a warning such as the following:

> Expected a conditional expression and instead saw an assignment.

Many style guides recommend more explicitly indicating the intention for the condition to be an assignment. You can do that minimally by putting additional parentheses as a grouping operator around the assignment:

```
const iterator = document.createNodeIterator(document, NodeFilter.SHOW_COMMENT);
let currentNode;
while ((currentNode = iterator.nextNode())) {
  console.log(currentNode.textContent.trim());
}
```

In fact, this is the style enforced by ESLint's `no-cond-assign`'s default configuration, as well as Prettier, so you'll likely see this pattern a lot in the wild.

Some people may further recommend adding a comparison operator to turn the condition into an explicit comparison:

```
while ((currentNode = iterator.nextNode()) !== null) {
```

There are other ways to write this pattern, such as:

```
while ((currentNode = iterator.nextNode()) && currentNode) {
```

Or, forgoing the idea of using a `while` loop altogether:

```
const iterator = document.createNodeIterator(document, NodeFilter.SHOW_COMMENT);
for (
  let currentNode = iterator.nextNode();
  currentNode;
  currentNode = iterator.nextNode()
) {
  console.log(currentNode.textContent.trim());
}
```

If the reader is sufficiently familiar with the assignment as condition pattern, all these variations should have equivalent readability. Otherwise, the last form is probably the most readable, albeit the most verbose.

See also

- `do...while`
- `for`
- `break`
- `continue`

with

> **Note:** Use of the `with` statement is not recommended, as it may be the source of confusing bugs and compatibility issues, makes optimization impossible, and is forbidden in strict mode. The recommended alternative is to assign the object whose properties you want to access to a temporary variable.

The **`with`** statement extends the scope chain for a statement.

Syntax

```
with (expression)
  statement
```

`expression`

Adds the given expression to the scope chain used when evaluating the statement. The parentheses around the expression are required.

`statement`

Any statement. To execute multiple statements, use a block statement (`{ ... }`) to group those statements.

Description

There are two types of identifiers: a *qualified* identifier and an *unqualified* identifier. An unqualified identifier is one that does not indicate where it comes from.

```
foo; // unqualified identifier
```

```
foo.bar; // bar is a qualified identifier
```

Normally, an unqualified identifier is resolved by searching the scope chain for a variable with that name, while a qualified identifier is resolved by searching the prototype chain of an object for a property with that name.

```
const foo = { bar: 1 };
console.log(foo.bar);
// foo is found in the scope chain as a variable;
// bar is found in foo as a property
```

One exception to this is the global object, which sits on top of the scope chain, and whose properties automatically become global variables that can be referred to without qualifiers.

```
console.log(globalThis.Math === Math); // true
```

The `with` statement adds the given object to the head of this scope chain during the evaluation of its statement body. Every unqualified name would first be searched within the object (through an `in` check) before searching in the upper scope chain.

Note that if the unqualified reference refers to a method of the object, the method is called with the object as its `this` value.

```
with ([1, 2, 3]) {
  console.log(toString()); // 1,2,3
}
```

The object may have a `[Symbol.unscopables]` property, which defines a list of properties that should not be added to the scope chain (for backward compatibility). See the `Symbol.unscopables` documentation for more information.

The reasons to use a `with` statement include saving one temporary variable and reducing file size by avoiding repeating a lengthy object reference. However, there are far more reasons why `with` statements are not desirable:

- Performance: The `with` statement forces the specified object to be searched first for all name lookups. Therefore, all identifiers that aren't members of the specified object will be found more slowly in a `with` block. Moreover, the optimizer cannot make any assumptions about what each unqualified identifier refers to, so it must repeat the same property lookup every time the identifier is used.
- Readability: The `with` statement makes it hard for a human reader or JavaScript compiler to decide whether an unqualified name will be found along the scope chain, and if so, in which object. For example:

  ```
  function f(x, o) {
    with (o) {
      console.log(x);
    }
  }
  ```

  If you look just at the definition of `f`, it's impossible to tell what the `x` in the `with` body refers to. Only when `f` is called can `x` be determined to be `o.x` or `f`'s first formal parameter. If you forget to define `x` in the object you pass as the second parameter, you won't get an error — instead you'll just get unexpected results. It's also unclear what the actual intent of such code would be.
- Forward compatibility: Code using `with` may not be forward compatible, especially when used with something other than a plain object, which may gain more properties in the future. Consider this example:

  ```
  function f(foo, values) {
    with (foo) {
      console.log(values);
    }
  }
  ```

  If you call `f([1, 2, 3], obj)` in an ECMAScript 5 environment, the `values` reference inside the `with` statement will resolve to `obj`. However, ECMAScript 2015 introduces a `values` property on `Array.prototype` (so it will be available on every array). So, after upgrading the environment, the `values` reference inside the `with` statement resolves to `[1, 2, 3].values` instead, and is likely to cause bugs.

  In this particular example, `values` is defined as unscopable through `Array.prototype[Symbol.unscopables]`, so it still correctly resolves to the `values` parameter. If it were not defined as unscopable, one can see how this would be a difficult issue to debug.

Examples

Using the with statement

The following `with` statement specifies that the `Math` object is the default object. The statements following the `with` statement refer to the `PI` property and the `cos` and `sin` methods, without specifying an object. JavaScript assumes the `Math` object for these references.

```
let a, x, y;
const r = 10;

with (Math) {
  a = PI * r * r;
  x = r * cos(PI);
  y = r * sin(PI / 2);
}
```

Avoiding the with statement by destructuring properties into the current scope

You can usually avoid using `with` through property destructuring. Here we create an extra block to mimic the behavior of `with` creating an extra scope — but in actual usage, this block can usually be omitted.

```
let a, x, y;
const r = 10;

{
  const { PI, cos, sin } = Math;
  a = PI * r * r;
  x = r * cos(PI);
  y = r * sin(PI / 2);
}
```

Avoiding the with statement by using an IIFE

If you're producing an expression that must reuse a long-named reference multiple times, and your goal is to eliminate that lengthy name within your expression, you can wrap the expression in an IIFE and provide the long name as an argument.

```
const objectHavingAnEspeciallyLengthyName = { foo: true, bar: false };

if (((o) => o.foo && !o.bar)(objectHavingAnEspeciallyLengthyName)) {
  // This branch runs.
}
```

Creating dynamic namespaces using the with statement and a proxy

`with` will transform every variable lookup to a property lookup, while Proxies allow trapping every property lookup call. You can create a dynamic namespace by combining them.

```
const namespace = new Proxy(
  {},
  {
    has(target, key) {
      // Avoid trapping global properties like `console`
      if (key in globalThis) {
        return false;
      }
      // Trap all property lookups
      return true;
    },
    get(target, key) {
      return key;
    },
  },
);

with (namespace) {
  console.log(a, b, c); // "a" "b" "c"
}
```

See also

- block
- Strict mode
- `Symbol.unscopables`
- `Array.prototype[Symbol.unscopables]`

Strict mode

> **Note:** Sometimes you'll see the default, non-strict mode referred to as *sloppy mode*. This isn't an official term, but be aware of it, just in case.

JavaScript's strict mode is a way to *opt in* to a restricted variant of JavaScript, thereby implicitly opting-out of "sloppy mode". Strict mode isn't just a subset: it *intentionally* has different semantics from normal code. Strict mode code and non-strict mode code can coexist, so scripts can opt into strict mode incrementally.

Strict mode makes several changes to normal JavaScript semantics:

1. Eliminates some JavaScript silent errors by changing them to throw errors.
2. Fixes mistakes that make it difficult for JavaScript engines to perform optimizations: strict mode code can sometimes be made to run faster than identical code that's not strict mode.
3. Prohibits some syntax likely to be defined in future versions of ECMAScript.

Invoking strict mode

Strict mode applies to *entire scripts* or to *individual functions*. It doesn't apply to block statements enclosed in `{}` braces; attempting to apply it to such contexts does nothing. `eval` code, `Function` code, event handler attributes, strings passed to `setTimeout()`, and related functions are either function bodies or entire scripts, and invoking strict mode in them works as expected.

Strict mode for scripts

To invoke strict mode for an entire script, put the *exact* statement `"use strict";` (or `'use strict';`) before any other statements.

```
// Whole-script strict mode syntax
"use strict";
const v = "Hi! I'm a strict mode script!";
```

Strict mode for functions

Likewise, to invoke strict mode for a function, put the *exact* statement `"use strict";` (or `'use strict';`) in the function's body before any other statements.

```
function myStrictFunction() {
  // Function-level strict mode syntax
  "use strict";
  function nested() {
    return "And so am I!";
  }
  return `Hi! I'm a strict mode function! ${nested()}`;
}
function myNotStrictFunction() {
  return "I'm not strict.";
}
```

The `"use strict"` directive can only be applied to the body of functions with simple parameters. Using `"use strict"` in functions with rest, default, or destructured parameters is a syntax error.

```
function sum(a = 1, b = 2) {
  // SyntaxError: "use strict" not allowed in function with default parameter
  "use strict";
  return a + b;
}
```

Strict mode for modules

The entire contents of JavaScript modules are automatically in strict mode, with no statement needed to initiate it.

```
function myStrictFunction() {
  // because this is a module, I'm strict by default
}
export default myStrictFunction;
```

Strict mode for classes

All parts of a class's body are strict mode code, including both class declarations and class expressions.

```
class C1 {
  // All code here is evaluated in strict mode
  test() {
    delete Object.prototype;
  }
}
new C1().test(); // TypeError, because test() is in strict mode

const C2 = class {
  // All code here is evaluated in strict mode
};

// Code here may not be in strict mode
delete Object.prototype; // Will not throw error
```

Changes in strict mode

Strict mode changes both syntax and runtime behavior. Changes generally fall into these categories:

- changes converting mistakes into errors (as syntax errors or at runtime)
- changes simplifying how variable references are resolved
- changes simplifying `eval` and `arguments`
- changes making it easier to write "secure" JavaScript
- changes anticipating future ECMAScript evolution.

Converting mistakes into errors

Strict mode changes some previously-accepted mistakes into errors. JavaScript was designed to be easy for novice developers, and sometimes it gives operations which should be errors non-error semantics. Sometimes this fixes the immediate problem, but sometimes this creates worse problems in the future. Strict mode treats these mistakes as errors so that they're discovered and promptly fixed.

Assigning to undeclared variables

Strict mode makes it impossible to accidentally create global variables. In sloppy mode, mistyping a variable in an assignment creates a new property on the global object and continues to "work". Assignments which would accidentally create global variables throw an error in strict mode:

```
"use strict";
let mistypeVariable;

// Assuming no global variable mistypeVarible exists
```

```
// this line throws a ReferenceError due to the
// misspelling of "mistypeVariable" (lack of an "a")
mistypeVarible = 17;
```

Failing to assign to object properties

In strict mode, certain assignments throw errors instead of failing silently. There are three ways to fail a property assignment:

- assignment to a non-writable data property
- assignment to a getter-only accessor property
- assignment to a new property on a non-extensible object

For example, `NaN` is a non-writable global variable. In sloppy mode, assigning to `NaN` does nothing; the developer receives no failure feedback. In strict mode, assigning to `NaN` throws an exception.

```
"use strict";

// Assignment to a non-writable global
undefined = 5; // TypeError
Infinity = 5; // TypeError

// Assignment to a non-writable property
const obj1 = {};
Object.defineProperty(obj1, "x", { value: 42, writable: false });
obj1.x = 9; // TypeError

// Assignment to a getter-only property
const obj2 = {
  get x() {
    return 17;
  },
};
obj2.x = 5; // TypeError

// Assignment to a new property on a non-extensible object
const fixed = {};
Object.preventExtensions(fixed);
fixed.newProp = "ohai"; // TypeError
```

Failing to delete object properties

Attempts to delete a non-configurable or otherwise undeletable (e.g., it's intercepted by a proxy's `deleteProperty` handler which returns `false`) property throw in strict mode (where before the attempt would have no effect):

```
"use strict";
delete Object.prototype; // TypeError
delete [].length; // TypeError
```

Strict mode also forbids deleting plain names. `delete name` in strict mode is a syntax error:

```
"use strict";

var x;
delete x; // syntax error
```

If the name is a configurable global property, prefix it with `globalThis` to delete it.

```
"use strict";

delete globalThis.x;
```

Duplicate parameter names

Strict mode requires that function parameter names be unique. In sloppy mode, the last duplicated argument hides previous identically-named arguments. Those previous arguments remain available through `arguments`, so they're not completely inaccessible. Still, this hiding makes little sense and is probably undesirable (it might hide a typo, for example), so in strict mode, duplicate argument names are a syntax error:

```
function sum(a, a, c) {
  // syntax error
  "use strict";
  return a + a + c; // wrong if this code ran
}
```

It is also a syntax error in non-strict mode to have duplicate parameter names, if the function has a default parameter, rest parameter, or destructured parameter.

Legacy octal literals

Strict mode forbids a `0`-prefixed octal literal. In sloppy mode, a number beginning with a `0`, such as `0644`, is interpreted as an octal number (`0644 === 420`), if all digits are smaller than 8. Novice developers sometimes believe a leading-zero prefix has no semantic meaning, so they might use it as an alignment device — but this changes the number's meaning! A leading-zero syntax for the octal is rarely useful and can be mistakenly used, so strict mode makes it a syntax error:

```
"use strict";
const sum =
  015 + // syntax error
  197 +
  142;
```

The standardized way to denote octal literals is via the `0o` prefix. For example:

```
const sumWithOctal = 0o10 + 8;
console.log(sumWithOctal); // 16
```

Octal escape sequences, such as `"\45"`, which is equal to `"%"`, can be used to represent characters by extended-ASCII character code numbers in octal. In strict mode, this is a syntax error. More formally, it's disallowed to have `\` followed by any decimal digit other than `0`, or `\0` followed by a decimal digit; for example `\9` and `\07`.

Setting properties on primitive values

Strict mode forbids setting properties on primitive values. Accessing a property on a primitive implicitly creates a wrapper object that's unobservable, so in sloppy mode, setting properties is ignored (no-op). In strict mode, a `TypeError` is thrown.

```
"use strict";

false.true = ""; // TypeError
(14).sailing = "home"; // TypeError
"with".you = "far away"; // TypeError
```

Duplicate property names

Duplicate property names used to be considered a `SyntaxError` in strict mode. With the introduction of computed property names, making duplication possible at runtime, this restriction was removed in ES2015.

```
"use strict";
const o = { p: 1, p: 2 }; // syntax error prior to ECMAScript 2015
```

> **Note:** Making code that used to error become non-errors is always considered backwards-compatible. This is a good part of the language being strict about throwing errors: it leaves room for future semantic changes.

Simplifying scope management

Strict mode simplifies how variable names map to particular variable definitions in the code. Many compiler optimizations rely on the ability to say that variable *X* is stored in *that* location: this is critical to fully optimizing JavaScript code. JavaScript sometimes makes this basic mapping of name to variable definition in the code impossible to perform until runtime. Strict mode removes most cases where this happens, so the compiler can better optimize strict mode code.

Removal of the with statement

Strict mode prohibits `with`. The problem with `with` is that any name inside the block might map either to a property of the object passed to it, or to a variable in surrounding (or even global) scope, at runtime; it's impossible to know which beforehand. Strict mode makes `with` a syntax error, so there's no chance for a name in a `with` to refer to an unknown location at runtime:

```
"use strict";
const x = 17;
with (obj) {
  // Syntax error
  // If this weren't strict mode, would this be const x, or
  // would it instead be obj.x? It's impossible in general
  // to say without running the code, so the name can't be
  // optimized.
  x;
}
```

The alternative of assigning the object to a short name variable, then accessing the corresponding property on that variable, stands ready to replace `with`.

Non-leaking eval

In strict mode, `eval` does not introduce new variables into the surrounding scope. In sloppy mode, `eval("var x;")` introduces a variable `x` into the surrounding function or the global scope. This means that, in general, in a function containing a call to `eval`, every name not referring to an argument or local variable must be mapped to a particular definition at runtime (because that `eval` might have introduced a new variable that would hide the outer variable). In strict mode, `eval` creates variables only for the code being evaluated, so `eval` can't affect whether a name refers to an outer variable or some local variable:

```
var x = 17;
var evalX = eval("'use strict'; var x = 42; x;");
console.assert(x === 17);
console.assert(evalX === 42);
```

Whether the string passed to `eval()` is evaluated in strict mode depends on how `eval()` is invoked (direct eval or indirect eval).

Block-scoped function declarations

The JavaScript language specification, since its start, had not allowed function declarations nested in block statements. However, it was so intuitive that most browsers implemented it as an extension grammar. Unfortunately, the implementations' semantics diverged, and it became impossible for the language specification to

reconcile all implementations. Therefore, block-scoped function declarations are only explicitly specified in strict mode (whereas they were once disallowed in strict mode), while sloppy mode behavior remains divergent among browsers.

Making eval and arguments simpler

Strict mode makes `arguments` and `eval` less bizarrely magical. Both involve a considerable amount of magical behavior in sloppy mode: `eval` to add or remove bindings and to change binding values, and `arguments` syncing named arguments with its indexed properties. Strict mode makes great strides toward treating `eval` and `arguments` as keywords.

Preventing binding or assigning eval and arguments

The names `eval` and `arguments` can't be bound or assigned in language syntax. All these attempts to do so are syntax errors:

```
"use strict";
eval = 17;
arguments++;
++eval;
const obj = { set p(arguments) {} };
let eval;
try {
} catch (arguments) {}
function x(eval) {}
function arguments() {}
const y = function eval() {};
const f = new Function("arguments", "'use strict'; return 17;");
```

No syncing between parameters and arguments indices

Strict mode code doesn't sync indices of the `arguments` object with each parameter binding. In a sloppy mode function whose first argument is `arg`, setting `arg` also sets `arguments[0]`, and vice versa (unless no arguments were provided or `arguments[0]` is deleted). `arguments` objects for strict mode functions store the original arguments when the function was invoked. `arguments[i]` does not track the value of the corresponding named argument, nor does a named argument track the value in the corresponding `arguments[i]`.

```
function f(a) {
  "use strict";
  a = 42;
  return [a, arguments[0]];
}
const pair = f(17);
console.assert(pair[0] === 42);
console.assert(pair[1] === 17);
```

"Securing" JavaScript

Strict mode makes it easier to write "secure" JavaScript. Some websites now provide ways for users to write JavaScript which will be run by the website *on behalf of other users*. JavaScript in browsers can access the user's private information, so such JavaScript must be partially transformed before it is run, to censor access to forbidden functionality. JavaScript's flexibility makes it effectively impossible to do this without many runtime checks. Certain language functions are so pervasive that performing runtime checks has a considerable performance cost. A few strict mode tweaks, plus requiring that user-submitted JavaScript be strict mode code and that it be invoked in a certain manner, substantially reduce the need for those runtime checks.

No this substitution

The value passed as `this` to a function in strict mode is not forced into being an object (a.k.a. "boxed"). For a sloppy mode function, `this` is always an object: either the provided object, if called with an object-valued `this`; or the boxed value of `this`, if called with a primitive as `this`; or the global object, if called with `undefined` or `null` as `this`. (Use `call`, `apply`, or `bind` to specify a particular `this`.) Not only is automatic boxing a performance cost, but exposing the global object in browsers is a security hazard because the global object provides access to functionality that "secure" JavaScript environments must restrict. Thus for a strict mode function, the specified `this` is not boxed into an object, and if unspecified, `this` is `undefined` instead of `globalThis`:

```
"use strict";
function fun() {
  return this;
}
console.assert(fun() === undefined);
console.assert(fun.call(2) === 2);
console.assert(fun.apply(null) === null);
console.assert(fun.call(undefined) === undefined);
console.assert(fun.bind(true)() === true);
```

Removal of stack-walking properties

In strict mode it's no longer possible to "walk" the JavaScript stack. Many implementations used to implement some extension features that make it possible to detect the upstream caller of a function. When a function `fun` is in the middle of being called, `fun.caller` is the function that most recently called `fun`, and `fun.arguments` is the `arguments` for that invocation of `fun`. Both extensions are problematic for "secure" JavaScript because they allow "secured" code to access "privileged" functions and their (potentially unsecured) arguments. If `fun` is in strict mode, both `fun.caller` and `fun.arguments` are non-deletable properties which throw when set or retrieved:

```
function restricted() {
  "use strict";
  restricted.caller; // throws a TypeError
  restricted.arguments; // throws a TypeError
}
function privilegedInvoker() {
```

```
  return restricted();
}
privilegedInvoker();
```

Similarly, `arguments.callee` is no longer supported. In sloppy mode, `arguments.callee` refers to the enclosing function. This use case is weak: name the enclosing function! Moreover, `arguments.callee` substantially hinders optimizations like inlining functions, because it must be made possible to provide a reference to the un-inlined function if `arguments.callee` is accessed. `arguments.callee` for strict mode functions is a non-deletable property which throws an error when set or retrieved:

```
"use strict";
function f() {
  return arguments.callee;
}
f(); // throws a TypeError
```

Future-proofing JavaScript

Extra reserved words

Reserved words are identifiers that can't be used as variable names. Strict mode reserves some more names than sloppy mode, some of which are already used in the language, and some of which are reserved for the future to make future syntax extensions easier to implement.

- `implements`
- `interface`
- `let`
- `package`
- `private`
- `protected`
- `public`
- `static`
- `yield`

Transitioning to strict mode

Strict mode has been designed so that the transition to it can be made gradually. It is possible to change each file individually and even to transition code to strict mode down to the function granularity.

You can migrate a codebase to strict mode by first adding `"use strict"` to a piece of source code, and then fixing all execution errors, while watching out for semantic differences.

Syntax errors

When adding `'use strict';`, the following cases will throw a `SyntaxError` before the script is executing:

- Octal syntax `const n = 023;`
- `with` statement
- Using `delete` on a variable name `delete myVariable;`
- Using `eval` or `arguments` as variable or function argument name
- Using one of the newly reserved keywords (in prevision for future language features): `implements`, `interface`, `let`, `package`, `private`, `protected`, `public`, `static`, and `yield`
- Declaring two function parameters with the same name `function f(a, b, b) {}`
- Declaring the same property name twice in an object literal `{a: 1, b: 3, a: 7}`. This constraint was later removed (bug 1041128).

These errors are good, because they reveal plain errors or bad practices. They occur before the code is running, so they are easily discoverable as long as the code gets parsed by the runtime.

New runtime errors

JavaScript used to silently fail in contexts where what was done should be an error. Strict mode throws in such cases. If your code base contains such cases, testing will be necessary to be sure nothing is broken. You can screen for such errors at the function granularity level.

- Assigning to an undeclared variable throws a `ReferenceError`. This used to set a property on the global object, which is rarely the expected effect. If you really want to set a value to the global object, explicitly assign it as a property on `globalThis`.
- Failing to assign to an object's property (e.g., it's read-only) throws a `TypeError`. In sloppy mode, this would silently fail.
- Deleting a non-deletable property throws a `TypeError`. In sloppy mode, this would silently fail.
- Accessing `arguments.callee`, `strictFunction.caller`, or `strictFunction.arguments` throws a `TypeError` if the function is in strict mode. If you are using `arguments.callee` to call the function recursively, you can use a named function expression instead.

Semantic differences

These differences are very subtle differences. It's possible that a test suite doesn't catch this kind of subtle difference. Careful review of your code base will probably be necessary to be sure these differences don't affect the semantics of your code. Fortunately, this careful review can be done gradually down the function granularity.

`this`

> In sloppy mode, function calls like `f()` would pass the global object as the `this` value. In strict mode, it is now `undefined`. When a function was called with `call` or `apply`, if the value was a primitive value, this one was boxed into an object (or the global object for `undefined` and `null`). In strict mode, the value is passed directly without conversion or replacement.

`arguments`

In sloppy mode, modifying a value in the `arguments` object modifies the corresponding named argument. This made optimizations complicated for JavaScript engine and made code harder to read/understand. In strict mode, the `arguments` object is created and initialized with the same values than the named arguments, but changes to either the `arguments` object or the named arguments aren't reflected in one another.

`eval`

In strict mode code, `eval` doesn't create a new variable in the scope from which it was called. Also, of course, in strict mode, the string is evaluated with strict mode rules. Thorough testing will need to be performed to make sure nothing breaks. Not using eval if you don't really need it may be another pragmatic solution.

Block-scoped function declarations

In sloppy mode, a function declaration inside a block may be visible outside the block and even callable. In strict mode, a function declaration inside a block is only visible inside the block.

See also

- JavaScript modules guide
- Lexical grammar

Template literals (Template strings)

Template literals are literals delimited with backtick (`` ` ``) characters, allowing for multi-line strings, string interpolation with embedded expressions, and special constructs called tagged templates.

Template literals are sometimes informally called *template strings*, because they are used most commonly for string interpolation (to create strings by doing substitution of placeholders). However, a tagged template literal may not result in a string; it can be used with a custom tag function to perform whatever operations you want on the different parts of the template literal.

Syntax

```
`string text`

`string text line 1
 string text line 2`

`string text ${expression} string text`

tagFunction`string text ${expression} string text`
```

Parameters

`string text`

The string text that will become part of the template literal. Almost all characters are allowed literally, including line breaks and other whitespace characters. However, invalid escape sequences will cause a syntax error, unless a tag function is used.

`expression`

An expression to be inserted in the current position, whose value is converted to a string or passed to `tagFunction`.

`tagFunction`

If specified, it will be called with the template strings array and substitution expressions, and the return value becomes the value of the template literal. See tagged templates.

Description

Template literals are enclosed by backtick (`` ` ``) characters instead of double or single quotes.

Along with having normal strings, template literals can also contain other parts called *placeholders*, which are embedded expressions delimited by a dollar sign and curly braces: `${expression}`. The strings and placeholders get passed to a function — either a default function, or a function you supply. The default function (when you don't supply your own) just performs string interpolation to do substitution of the placeholders and then concatenate the parts into a single string.

To supply a function of your own, precede the template literal with a function name; the result is called a **tagged template**. In that case, the template literal is passed to your tag function, where you can then perform whatever operations you want on the different parts of the template literal.

To escape a backtick in a template literal, put a backslash (`\`) before the backtick.

```
`\`` === "`"; // true
```

Dollar signs can be escaped as well to prevent interpolation.

```
`\${1}` === "${1}"; // true
```

Multi-line strings

Any newline characters inserted in the source are part of the template literal.

Using normal strings, you would have to use the following syntax in order to get multi-line strings:

```
console.log("string text line 1\nstring text line 2");
// "string text line 1
// string text line 2"
```

Using template literals, you can do the same with this:

```
console.log(`string text line 1
string text line 2`);
// "string text line 1
// string text line 2"
```

Like normal string literals, you can write a single-line string across multiple lines for source code readability, by escaping the newline with a backslash (`\`):

```
console.log(`string text line 1 \
string text line 2`);
// "string text line 1 string text line 2"
```

String interpolation

Without template literals, when you want to combine output from expressions with strings, you'd concatenate them using the addition operator `+`:

```
const a = 5;
const b = 10;
console.log("Fifteen is " + (a + b) + " and\nnot " + (2 * a + b) + ".");
// "Fifteen is 15 and
// not 20."
```

That can be hard to read – especially when you have multiple expressions.

With template literals, you can avoid the concatenation operator — and improve the readability of your code — by using placeholders of the form `${expression}` to perform substitutions for embedded expressions:

```
const a = 5;
const b = 10;
console.log(`Fifteen is ${a + b} and
not ${2 * a + b}.`);
// "Fifteen is 15 and
// not 20."
```

Note that there's a mild difference between the two syntaxes. Template literals coerce their expressions directly to strings, while addition coerces its operands to primitives first. For more information, see the reference page for the `+` operator.

Nesting templates

In certain cases, nesting a template is the easiest (and perhaps more readable) way to have configurable strings. Within a backtick-delimited template, it is simple to allow inner backticks by using them inside an `${expression}` placeholder within the template.

For example, without template literals, if you wanted to return a certain value based on a particular condition, you could do something like the following:

```
let classes = "header";
classes += isLargeScreen()
  ? ""
  : item.isCollapsed
    ? " icon-expander"
    : " icon-collapser";
```

With a template literal but without nesting, you could do this:

```
const classes = `header ${
  isLargeScreen() ? "" : item.isCollapsed ? "icon-expander" : "icon-collapser"
}`;
```

With nesting of template literals, you can do this:

```
const classes = `header ${
  isLargeScreen() ? "" : `icon-${item.isCollapsed ? "expander" : "collapser"}`
}`;
```

Tagged templates

A more advanced form of template literals are *tagged* templates.

Tags allow you to parse template literals with a function. The first argument of a tag function contains an array of string values. The remaining arguments are related to the expressions.

The tag function can then perform whatever operations on these arguments you wish, and return the manipulated string. (Alternatively, it can return something completely different, as described in one of the following examples.)

The name of the function used for the tag can be whatever you want.

```
const person = "Mike";
const age = 28;

function myTag(strings, personExp, ageExp) {
  const str0 = strings[0]; // "That "
  const str1 = strings[1]; // " is a "
  const str2 = strings[2]; // "."

  const ageStr = ageExp < 100 ? "youngster" : "centenarian";

  // We can even return a string built using a template literal
  return `${str0}${personExp}${str1}${ageStr}${str2}`;
}

const output = myTag`That ${person} is a ${age}.`;

console.log(output);
// That Mike is a youngster.
```

The tag does not have to be a plain identifier. You can use any expression with precedence greater than 16, which includes property access, function call, new expression, or even another tagged template literal.

```
console.log`Hello`; // [ 'Hello' ]
console.log.bind(1, 2)`Hello`; // 2 [ 'Hello' ]
new Function("console.log(arguments)")`Hello`; // [Arguments] { '0': [ 'Hello' ] }

function recursive(strings, ...values) {
  console.log(strings, values);
  return recursive;
}
recursive`Hello``World`;
// [ 'Hello' ] []
// [ 'World' ] []
```

While technically permitted by the syntax, *untagged* template literals are strings and will throw a `TypeError` when chained.

```
console.log(`Hello``World`); // TypeError: "Hello" is not a function
```

The only exception is optional chaining, which will throw a syntax error.

```
console.log?.`Hello`; // SyntaxError: Invalid tagged template on optional chain
console?.log`Hello`; // SyntaxError: Invalid tagged template on optional chain
```

Note that these two expressions are still parsable. This means they would not be subject to automatic semicolon insertion, which will only insert semicolons to fix code that's otherwise unparsable.

```
// Still a syntax error
const a = console?.log
`Hello`
```

Tag functions don't even need to return a string!

```
function template(strings, ...keys) {
  return (...values) => {
    const dict = values[values.length - 1] || {};
    const result = [strings[0]];
    keys.forEach((key, i) => {
      const value = Number.isInteger(key) ? values[key] : dict[key];
      result.push(value, strings[i + 1]);
    });
    return result.join("");
  };
}

const t1Closure = template`${0}${1}${0}!`;
// const t1Closure = template(["","","","!"],0,1,0);
t1Closure("Y", "A"); // "YAY!"

const t2Closure = template`${0} ${"foo"}!`;
// const t2Closure = template(["","," ","!"],0,"foo");
t2Closure("Hello", { foo: "World" }); // "Hello World!"

const t3Closure = template`I'm ${"name"}. I'm almost ${"age"} years old.`;
// const t3Closure = template(["I'm ", ". I'm almost ", " years old."], "name", "age");
t3Closure("foo", { name: "MDN", age: 30 }); // "I'm MDN. I'm almost 30 years old."
t3Closure({ name: "MDN", age: 30 }); // "I'm MDN. I'm almost 30 years old."
```

The first argument received by the tag function is an array of strings. For any template literal, its length is equal to the number of substitutions (occurrences of `${…}`) plus one, and is therefore always non-empty.

For any particular tagged template literal expression, the tag function will always be called with the exact same literal array, no matter how many times the literal is evaluated.

```
const callHistory = [];

function tag(strings, ...values) {
  callHistory.push(strings);
  // Return a freshly made object
  return {};
}

function evaluateLiteral() {
  return tag`Hello, ${"world"}!`;
}

console.log(evaluateLiteral() === evaluateLiteral()); // false; each time `tag` is called, it returns a new object
console.log(callHistory[0] === callHistory[1]); // true; all evaluations of the same tagged literal would pass in the same strings array
```

This allows the tag to cache the result based on the identity of its first argument. To further ensure the array value's stability, the first argument and its `raw` property are both frozen, so you can't mutate them in any way.

Raw strings

The special `raw` property, available on the first argument to the tag function, allows you to access the raw strings as they were entered, without processing escape sequences.

```
function tag(strings) {
  console.log(strings.raw[0]);
}

tag`string text line 1 \n string text line 2`;
// Logs "string text line 1 \n string text line 2",
// including the two characters '\' and 'n'
```

> **Note:** The template literal syntax is still processed in the same way, which means unescaped backticks and `${` have special syntactic meaning, but escaping these characters create extra backslashes in the raw string. See raw strings containing template literal syntax for more information.

In addition, the `String.raw()` method exists to create raw strings just like the default template function and string concatenation would create.

```
const str = String.raw`Hi\n${2 + 3}!`;
// "Hi\\n5!"

str.length;
// 6

Array.from(str).join(",");
// "H,i,\\,n,5,!"
```

`String.raw` functions like an "identity" tag if the literal doesn't contain any escape sequences. In case you want an actual identity tag that always works as if the literal is untagged, you can make a custom function that passes the "cooked" (i.e., escape sequences are processed) literal array to `String.raw`, pretending they are raw strings.

```
const identity = (strings, ...values) =>
  String.raw({ raw: strings }, ...values);
console.log(identity`Hi\n${2 + 3}!`);
// Hi
// 5!
```

This is useful for many tools which give special treatment to literals tagged by a particular name.

```
const html = (strings, ...values) => String.raw({ raw: strings }, ...values);
// Some formatters will format this literal's content as HTML
const doc = html`<!doctype html>
  <html lang="en-US">
    <head>
      <title>Hello</title>
    </head>
    <body>
      <h1>Hello world!</h1>
    </body>
  </html>`;
```

Tagged templates and escape sequences

In normal template literals, the escape sequences in string literals are all allowed. Any other non-well-formed escape sequence is a syntax error. This includes:

- `\` followed by any decimal digit other than `0`, or `\0` followed by a decimal digit; for example `\9` and `\07` (which is a deprecated syntax)
- `\x` followed by fewer than two hex digits (including none); for example `\xz`
- `\u` not followed by `{` and followed by fewer than four hex digits (including none); for example `\uz`
- `\u{}` enclosing an invalid Unicode code point — it contains a non-hex digit, or its value is greater than `10FFFF`; for example `\u{110000}` and `\u{z}`

> **Note:** `\` followed by other characters, while they may be useless since nothing is escaped, are not syntax errors.

However, this is problematic for tagged templates, which, in addition to the "cooked" literal, also have access to the raw literals (escape sequences are preserved as-is).

Tagged templates enable the embedding of arbitrary string content, where escape sequences may follow a different syntax. Consider for an example where we embed LaTeX source text in JavaScript via `String.raw`. We want to still be able to use LaTeX macros that start with `u` or `x` without following JavaScript syntax restrictions. Therefore, the syntax restriction of well-formed escape sequences is removed from tagged templates. The example below uses MathJax to render LaTeX in one element:

```
const node = document.getElementById("formula");
MathJax.typesetClear([node]);
// Throws in older ECMAScript versions (ES2016 and earlier)
// SyntaxError: malformed Unicode character escape sequence
node.textContent = String.raw`$\underline{u}$`;
MathJax.typesetPromise([node]);
```

However, illegal escape sequences must still be represented in the "cooked" representation. They will show up as `undefined` element in the "cooked" array:

```
function log(str) {
  console.log("Cooked:", str[0]);
  console.log("Raw:", str.raw[0]);
}

log`\unicode`;
// Cooked: undefined
// Raw: \unicode
```

Note that the escape-sequence restriction is only dropped from *tagged* templates, but not from *untagged* template literals:

```
const bad = `bad escape sequence: \unicode`;
```

See also

- Numbers and strings guide
- `String`
- `String.raw()`
- Lexical grammar
- ES6 in Depth: Template strings on hacks.mozilla.org (2015)

Trailing commas

Trailing commas (sometimes called "final commas") can be useful when adding new elements, parameters, or properties to JavaScript code. If you want to add a new property, you can add a new line without modifying the previously last line if that line already uses a trailing comma. This makes version-control diffs cleaner and editing code might be less troublesome.

JavaScript has allowed trailing commas in array literals since the beginning. Trailing commas are now also allowed in object literals, function parameters, named imports, named exports, and more.

JSON, however, disallows all trailing commas.

Description

JavaScript allows trailing commas wherever a comma-separated list of values is accepted and more values may be expected after the last item. This includes:

- Array literals
- Object literals
- Parameter definitions
- Function calls
- Named imports
- Named exports
- Dynamic import
- Array and object destructuring

In all these cases, the trailing comma is entirely optional and doesn't change the program's semantics in any way.

It is particularly useful when adding, removing, or reordering items in a list that spans multiple lines, because it reduces the number of lines that need to be changed, which helps with both editing and reviewing the diff.

```
  [
    "foo",
+   "baz",
    "bar",
-   "baz",
  ]
```

Examples

Trailing commas in literals

Arrays

JavaScript ignores trailing commas in array literals:

```
const arr = [
  1,
  2,
  3,
];

arr; // [1, 2, 3]
arr.length; // 3
```

If more than one trailing comma is used, an elision (or hole) is produced. An array with holes is called *sparse* (a *dense* array has no holes). When iterating arrays for example with `Array.prototype.forEach()` or `Array.prototype.map()`, array holes are skipped. Sparse arrays are generally unfavorable, so you should avoid having multiple trailing commas.

```
const arr = [1, 2, 3, , ,];
arr.length; // 5
```

Objects

Trailing commas in object literals are legal as well:

```
const object = {
  foo: "bar",
  baz: "qwerty",
  age: 42,
};
```

Trailing commas in functions

Trailing commas are also allowed in function parameter lists.

Parameter definitions

The following function definition pairs are legal and equivalent to each other. Trailing commas don't affect the `length` property of function declarations or their `arguments` object.

```
function f(p) {}
function f(p,) {}

(p) => {};
(p,) => {};
```

The trailing comma also works with method definitions for classes or objects:

```
class C {
  one(a,) {}
  two(a, b,) {}
}

const obj = {
  one(a,) {},
  two(a, b,) {},
};
```

Function calls

The following function invocation pairs are legal and equivalent to each other.

```
f(p);
f(p,);

Math.max(10, 20);
Math.max(10, 20,);
```

Illegal trailing commas

Function parameter definitions or function invocations only containing a comma will throw a `SyntaxError`. Furthermore, when using rest parameters, trailing commas are not allowed:

```
function f(,) {} // SyntaxError: missing formal parameter
(,) => {};       // SyntaxError: expected expression, got ','
f(,)             // SyntaxError: expected expression, got ','

function f(...p,) {} // SyntaxError: parameter after rest parameter
(...p,) => {}        // SyntaxError: expected closing parenthesis, got ','
```

Trailing commas in destructuring

A trailing comma is also allowed within a destructuring pattern:

```
// array destructuring with trailing comma
[a, b,] = [1, 2];

// object destructuring with trailing comma
const o = {
  p: 42,
  q: true,
};
const { p, q, } = o;
```

However, a trailing comma is not allowed after the rest element, if present

```
const [a, ...b,] = [1, 2, 3];
// SyntaxError: rest element may not have a trailing comma
```

Trailing commas in JSON

As JSON is based on a very restricted subset of JavaScript syntax, **trailing commas are not allowed in JSON**.

Both lines will throw a `SyntaxError`:

```
JSON.parse("[1, 2, 3, 4, ]");
JSON.parse('{"foo" : 1, }');
// SyntaxError JSON.parse: unexpected character
// at line 1 column 14 of the JSON data
```

Omit the trailing commas to parse the JSON correctly:

```
JSON.parse("[1, 2, 3, 4 ]");
JSON.parse('{"foo" : 1 }');
```

Trailing commas in named imports and named exports

Trailing commas are valid in named imports and named exports.

Named imports

```
import {
  A,
  B,
  C,
} from "D";

import { X, Y, Z, } from "W";

import { A as B, C as D, E as F, } from "Z";
```

Named exports

```
export {
  A,
  B,
  C,
};

export { A, B, C, };

export { A as B, C as D, E as F, };
```

Trailing commas in dynamic import

Trailing commas are only allowed in dynamic imports if the runtime also implements the second `options` parameter.

```
import("D",);
import(
  "D",
  { with: { type: "json" } },
);
```

Quantifier prefix

> **Note:** The trailing comma in a quantifier actually changes its semantics from matching "exactly `n`" to matching "at least `n`".

```
/x{2}/; // Exactly 2 occurrences of "x"; equivalent to /xx/
/x{2,}/; // At least 2 occurrences of "x"; equivalent to /xx+/
/x{2,4}/; // 2 to 4 occurrences of "x"; equivalent to /xxx?x?/
```

See also

- Grammar and types guide